Red Hat Ceph Storage

User Guide

9



Contents

Red Hat Ceph Storage Compatibility 13

Compatibility matrix for Red Hat Ceph Storage 9.0 13

Red Hat Ceph Storage Getting started 16

Object storage 16

Object storage interfaces 16
Common workloads 16
Common use cases 16
Getting started with object storage 17

Block storage 18

Common workloads 18
Common use cases 18
Example workflow for using Ceph Block Device snapshots and clones 18
Getting started with block storage 19

File storage 20

Common use cases 20
Example use cases for file storage as a service (FSaaS) 20
Example workflow for using Ceph File System 20
Example workflow for using CephFS snapshot 21
Getting started with file storage 21

Red Hat Ceph Storage Overview 24

Introduction to Red Hat Ceph Storage 24
Understanding Red Hat Ceph Storage terminology 26
Architecture 31

Ceph architecture 31
Ceph client components 32
Core Ceph components 40

Data security and hardening 51

Introduction to Red Hat Ceph Storage 51
Supporting software 52
Threat and vulnerability management 52
Encryption and key management 55
Identity and access management 60
Infrastructure security 76
Data retention 80
Federal Information Processing Standard (FIPS) 81

Ceph File System 82

Use cases for CephFS 82

Red Hat Ceph Storage Planning 84

Considerations and recommendations 84

Basic considerations 84
Workload considerations 85
Network considerations for Red Hat Ceph Storage 88
Considerations for using a RAID controller with OSD hosts 90
Tuning considerations for Red Hat Ceph Storage 90
Colocation 93
Operating system requirements 98
Minimum hardware considerations 99

Hardware 101

General principles for selecting hardware 101
Optimize workload performance domains 104
Server and rack solutions 105
Recommended minimum hardware requirements for the Red Hat Ceph Storage Dashboard 108

Storage strategies 108

Configuring storage strategies 110
CRUSH admin overview 111
Placement groups 137
Pools overview 154
Erasure code pools overview 169

Installing Red Hat Ceph Storage 182

Initial installation 182

`cephadm` utility 182
Registering the Red Hat Ceph Storage nodes 185
Configuring Ansible inventory location 186
Enabling SSH login as root user on Red Hat Enterprise Linux 9 and 10 187
Creating an Ansible user with `sudo` access 188
Enabling password-less SSH for Ansible 190

Running the preflight playbook 191
Bootstrapping a new storage cluster 195
Distributing SSH keys 206
Disconnected (air gap) installation 207
Post-bootstrap setup 217
Using the cephadm shell 217
cephadm commands 218
Verifying the cluster installation 222
Adding hosts 223
Removing hosts 227
Labeling hosts 229
Adding Monitor service 234
Setting up a custom SSH key on an existing cluster 236
Setting up the admin node 238
Adding Manager service 240
Adding OSDs 241
Purging the Ceph storage cluster 242
Deploying client nodes 243
Managing a Red Hat Ceph Storage cluster using cephadm-ansible modules 245
cephadm-ansible modules 245
cephadm-ansible modules options 245
Bootstrapping a storage cluster using the cephadm_ansible modules 248
Adding or removing hosts using the ceph_orch_host module 250
Setting configuration options using the ceph_config module 253
Applying a service specification using the ceph_orch_apply module 255
Managing Ceph daemon states using the ceph_orch_daemon module 256
Day Two operations 257
Upgrading Red Hat Ceph Storage 259
Supported upgrade paths 259
Red Hat upgrade paths 259
Upgrading a Red Hat Ceph Storage cluster using cephadm 259
Compatibility considerations between RHCS and podman versions 259
Upgrading the Red Hat Ceph Storage cluster 260
Upgrading cluster in a disconnected (air-gapped) environment 263
Updating Ceph Object Gateway to use cephadm self-signed certificates in a multi-site deployment 266
Upgrading a host operating system from RHEL 9 to RHEL 10 268
Upgrading to 9 with RHEL 10 with stretch mode enabled from previous versions 269
Running a staggered upgrade 272
Monitoring and managing upgrade 275
Configuring Red Hat Ceph Storage 277
Ceph configuration 277
Configuration database 277
Using the Ceph metavariables 279
Viewing the Ceph configuration at runtime 280
Viewing a specific configuration at runtime 280
Setting a specific configuration at runtime 280
OSD Memory Target 282
Automatically tuning OSD memory 283
MDS memory cache limit 284
General configuration options 286
Ceph network configuration 287
Network configuration for Ceph 288
Network messenger 289
Configuring a public network 290
Configuring a private network 291
Configuring multiple public networks to the cluster 293
Verifying firewall rules are configured for default Ceph ports 294
Firewall settings for Ceph Monitor node 295
Firewall settings for Ceph OSDs 296
Network configuration options 297
Ceph Monitor configuration 299
Viewing the Ceph Monitor configuration database 300
Cluster maps 301
Ceph Monitor quorum 301
Ceph Monitor consistency 302
Bootstrap the Ceph Monitor 302
Minimum configuration for a Ceph Monitor 302
Unique identifier for Ceph 303
Ceph Monitor data store 303
Storage capacity 304
Ceph heartbeat 305
Ceph Monitor synchronization role 305
Time synchronization 306
Ceph Monitor configuration options 307
Ceph authentication configuration 313
Cephx authentication 313

Enabling Cephx 313
Disabling Cephx 315
Cephx user keyrings 315
CephX daemon keyrings 315
Cephx message signatures 318
Cephx configuration options 319
Pools, placement groups, and CRUSH configuration 320
Pools, placement groups and CRUSH 321
Pools, placement groups, and CRUSH configuration options 321
Ceph Object Storage Daemon (OSD) configuration 324
Ceph OSD configuration 324
Scrubbing the OSD 325
Backfilling an OSD 325
OSD recovery 325
Object Storage Daemon (OSD) configuration options 325
Ceph Monitor and OSD interaction configuration 331
Ceph Monitor and OSD interaction 331
OSD heartbeat 331
Reporting an OSD as down 332
Reporting a peering failure 333
OSD reporting status 333
Ceph Monitor and OSD configuration options 334
Debugging and logging configuration 337
Debugging and logging configuration options 337
Scrubbing options 340
BlueStore configuration options 343
Red Hat Ceph Storage Edge clusters 344
Pools overview 344
Creating a cluster handle and connecting to the cluster 344
Creating a pool I/O context 344
Resilient and non-resilient data pools 345
Cluster utilization with data pools 345
Erasure-coding 346
Reference 347
Erasure code pools overview 347
Creating a sample erasure-coded pool 348
Bluestore server-side compression 349
Benefits 349
Considerations 349
How compression works 349
Compression behavior 350
Configuring BlueStore compression for a specific pool 350
Cluster topology and colocation 353
Administering Red Hat Ceph Storage 354
Administration 354
Ceph administration 354
Understanding process management 354
Monitor Red Hat Ceph Storage clusters 366
Stretch clusters for Ceph storage 400
Generalized stretch cluster configuration for three availability zones 431
Override Ceph behavior 443
Ceph user management 445
Using `ceph-volume` utility 451
Ceph performance benchmark 464
Ceph performance counters 469
mClock OSD scheduler 488
BlueStore 509
`cephadm` operations 524
Using `cephadm-ansible` modules 529
Operations 542
Introduction to the Ceph Orchestrator 542
Managing services 543
Managing hosts 550
Managing monitors 558
Managing manager daemons 566
Managing OSDs 586
Managing the monitoring stack 614
Basic client setup 617
Managing the MDS service using the Ceph Orchestrator 619
Managing the Ceph Object Gateway 624
Handling a node failure 635
Configuring SNMP traps 643
Handling a data center failure 649
Certificate management 652
Red Hat Ceph Storage Dashboard 659

Security features 659
Configuring and managing the dashboard interface 659
Ceph management with the dashboard 660
Monitoring with the dashboard 661
Components 662
Architecture 662
Accessing the Ceph dashboard after bootstrap 663
Understanding the landing page of the Ceph dashboard 664
Configuring the Dashboard 667
Enabling Red Hat Ceph Storage Dashboard manually 667
Setting message of the day (MOTD) 668
Toggling Ceph dashboard features 669
Changing the dashboard password 671
Changing the Ceph dashboard password using the command line interface 672
Setting `admin` user password for Grafana 672
Configuring Prometheus data retention 673
Expanding the cluster 674
Upgrading a cluster 676
Managing access and single sign-on with the dashboard 677
Creating an admin account for syncing users to the Ceph dashboard 678
Syncing users from Red Hat Sign-On to the Ceph dashboard 679
Enabling single sign-on with SAML 2.0 683
Configuring Red Hat SSO as the identity provider using OAuth2 Protocol 684
Configuring IBM Security Verify as the identity provider using OAuth2 protocol 685
Enabling OAuth2 single sign-on 686
Disabling single sign-on for the Ceph Dashboard 687
Managing roles 688
User roles and permissions 688
Creating roles 689
Editing roles 690
Cloning roles 691
Deleting roles 691
Managing users 692
Creating users 692
Editing users 693
Deleting users 694
Managing user capabilities 695
Capabilities 695
Accessing capabilities 695
Creating user capabilities 697
Editing user capabilities 699
Importing user capabilities 699
Exporting user capabilities 700
Deleting user capabilities 700
Managing Ceph services 701
Running Ceph services 701
Managing hosts 702
Prerequisites 702
Entering maintenance mode 703
Exiting maintenance mode 703
Removing hosts 704
Managing pools 704
Creating pools 704
Editing pools 705
Deleting pools 705
Managing Ceph OSDs from the Ceph Dashboard 706
Managing the OSDs 706
Replacing the failed OSDs 709
Managing alerts 712
Critical alerts 712
Warning alerts 712
CephHealthError 713
CephHealthWarning 713
CephMonDownQuorumAtRisk 713
CephMonDiskspaceCritical 714
CephOSDDownHigh 715
CephOSDFull 716
CephDeviceFailurePredictionTooHigh 716
CephFilesystemDamaged 716
CephFilesystemOffline 717
CephFilesystemDegraded 717
CephFilesystemFailureNoStandby 717
CephFilesystemReadOnly 718
CephMgrModuleCrash 718
CephMgrPrometheusModuleInactive 718
CephPGsInactive 718

CephPGsDamaged 719
CephPGRecoveryAtRisk 719
CephNodeRootFilesystemFull 719
CephPoolFull 719
CephadmUpgradeFailed 720
CephadmDaemonFailed 720
HardwareStorageError 720
HardwareMemoryError 720
HardwareProcessorError 721
HardwareNetworkError 721
HardwarePowerError 721
HardwareFanError 721
PrometheusJobMissing 722
CephObjectMissing 722
CephDaemonCrash 722
CephRBDMirrorImagesPerDaemonHigh 723
CephPGsUnclean 723
CephPGNotScrubbed 723
CephPGsHighPerOSD 723
CephPGNotDeepScrubbed 724
CephNodeNetworkPacketDrops 724
CephNodeNetworkPacketErrors 724
CephNodeNetworkBondDegraded 724
CephNodeDiskspaceWarning 724
CephNodeInconsistentMTU 725
CephPoolGrowthWarning 725
CephPoolBackfillFull 725
CephPoolNearFull 725
CephSlowOps 725
CephDaemonSlowOps 726
CephadmPaused 726
Enabling monitoring stack 726
Configuring a Grafana certificate 728
Adding Alertmanager webhooks 729
Viewing alerts 730
Creating a silence 731
Recreating a silence 732
Editing a silence 732
Expiring a silence 732
Managing Ceph Object Gateway 733
Manually adding Ceph Object Gateway login credentials to the dashboard 733
Managing Ceph Object Gateway users 735
Managing user accounts 739
Managing Ceph Object Gateway buckets 742
Managing Ceph Object Gateway notification destinations 746
Managing bucket notifications in Ceph Object Gateway 749
Viewing Performance Metrics for Multi-Site Sync Deltas in Ceph Object Gateway 751
Viewing Ceph Object Gateway per-user and per-bucket performance counters 751
Managing Ceph Object Gateway bucket policies 753
Managing S3 bucket lifecycle policies 754
Creating a tiering storage class 756
Managing buckets of a multi-site object configuration 757
Configuring a multi-site 760
Managing Ceph File Systems (CephFS) 762
Configuring CephFS volumes 762
Configuring CephFS subvolume groups 763
Configuring CephFS subvolumes 764
Managing CephFS snapshots 766
Scheduling CephFS snapshots 769
Managing block devices 772
Managing block device images 772
Managing mirroring functions 780
Activating and deactivating telemetry 785
Deactivating telemetry 787
Red Hat Ceph Storage Object Gateways 788
Introduction to WORM 789
Reference 790
Key Factors for deploying Ceph Object Gateway 790
Prerequisites 790
Reference 790
Network Requirements in Red Hat Ceph Storage 790
Workload Management in Red Hat Ceph Storage 791
Basic Considerations for Red Hat Ceph Storage Deployment 793
Deployment Guidelines 795
Developing CRUSH hierarchies 798
Multi-site considerations 803
Performance adjustments 806
Tuning considerations for the Linux kernel when running Ceph 807

Advantages of colocating Ceph daemons ... 808
Deployment ... 811
Prerequisites ... 812
Deploying the Ceph Object Gateway using the command line interface ... 812
Deploying the Ceph Object Gateway using the service specification ... 814
Deploying a multi-site Ceph Object Gateway using the Ceph Orchestrator ... 818
Removing the Ceph Object Gateway using the Ceph Orchestrator ... 822
Using the Ceph Manager `rgw` module ... 823
High availability for the Ceph Object Gateway ... 829
Adjusting logging and debugging output ... 834
Configuration ... 835
Multi-site configuration and administration ... 857
Requirements and assumptions ... 858
Pools ... 861
Migrating a single site system to multi-site ... 861
Establishing a secondary zone ... 863
3-site failover and disaster recovery ... 871
Configuring multiple realms in the same storage cluster ... 872
Using multi-site sync policies ... 880
Using destination parameters ... 890
Bucket granular sync policies ... 891
Feature specific configuration ... 905
Bucket management ... 905
Identity and Access Management (IAM) ... 943
User management ... 948
Role management ... 956
Static web hosting ... 962
Storage classes ... 965
Ceph Object Gateway data layout ... 966
Metadata ... 967
Bucket index ... 967
Data ... 968
Object lookup path ... 968
Multiple data pools ... 968
Optimize the Ceph Object Gateway's garbage collection ... 968
Bucket and object listing ... 970
Object Gateway data layout parameters ... 971
Rate limits for ingesting data ... 971
Managing user rate limits ... 973
Managing bucket rate limits ... 974
Managing global rate limits ... 975
Authenticating with Ceph Object Gateway ... 977
LDAP integration ... 978
Active Directory integration ... 983
OpenStack Keystone integration ... 987
Secure Token Service ... 993
Security ... 1008
Prerequisites ... 1008
QAT acceleration for encryption and compression ... 1008
Server-side encryption ... 1013
Multi-factor authentication ... 1031
Administrating Ceph Object Gateway ... 1036
Prerequisites ... 1036
Creating storage policies ... 1036
Configure bucket index resharding ... 1038
Creating indexless buckets ... 1052
Enabling compression ... 1053
Quota management ... 1055
Usage ... 1057
Deduplication statistics ... 1058
Optimize the Ceph Object Gateway's data object storage ... 1060
Policy based data archival and retrieval to S3 compatible platforms ... 1061
Viewing Ceph Object Gateway per-user and per-bucket usage statistics (Technology Preview) ... 1082
Testing and validation ... 1084
Prerequisites ... 1084
Create an S3 user ... 1084
Create a Swift user ... 1085
Test S3 access ... 1087
Test Swift access ... 1088
Configuration reference ... 1088
General settings ... 1088
About pools ... 1093
Lifecycle settings ... 1094
Swift settings ... 1095
Logging settings ... 1096
Keystone settings ... 1098
Keystone integration configuration options ... 1098
LDAP settings ... 1100

Red Hat Ceph Storage NFS cluster and share management (OpenStack Manila users only) 1101
High availability for NFS services 1101
NFSv4 ACL behavior and limitations 1101
NFS failover behavior 1102
Load-balancing considerations 1103
Creating an NFS cluster 1103
Customize an NFS configuration 1104
Remove customized NFS configurations 1106
Sharing Ceph File System namespaces over the NFS protocol 1106
Modifying NFS shares 1109
Creating custom NFS shares 1111
Prerequisites 1112
Procedure 1112
Deleting NFS shares from the CLI 1113
Deleting an NFS cluster 1113
Before you begin 1113
Procedure 1113
Windows 11 desktop client side mount options for NFSv3 fileshare 1114
Rate-limiting in NFS 1114
Bandwidth rate-limiting 1115
Input/output operations (IOPS) rate-limiting 1115
Managing rate-limiting for NFS clusters 1116
Listing QoS configuration for NFS cluster 1121
Managing rate-limiting for NFS shares 1121
Listing QoS configuration for NFS share 1125
TLS security in NFS 1126
TLS versus mTLS 1126
Use Cases 1127
Configuring share security options for NFS 1127
Configuring TLS settings for NFS 1128
Setting up TLS in NFS client using Stunnel 1129
Red Hat Ceph Storage File Systems 1131
Using CephFS 1131
Understanding the CephFS data flow 1132
Comparison with other FSaaS Solutions 1133
Data Protection in CephFS 1133
Configuring CephFS 1133
Health messages 1133
Configuring Metadata Server daemons 1135
Configuring journaler 1141
Configuring Ceph File System mirrors 1142
CephFS and the Metadata Server 1144
Role of Metadata Server in CephFS 1144
Optimizing MDS performance 1145
MDS memory sizing considerations 1145
CephFS volumes, subvolumes, and subvolume groups 1146
Differences and Usage 1146
Volume Management 1147
File system components 1147
CephFS and SELinux 1148
CephFS limitations and the POSIX standards 1149
CephFS Metadata Server 1149
MDS resource recommendations 1149
Metadata Server daemon states 1150
Metadata Server ranks 1151
Metadata Server cache size limits 1151
File system affinity 1152
Managing the MDS service using the Ceph Orchestrator 1152
Configuring file system affinity 1156
Configuring multiple active Metadata Server daemons 1157
Configuring the number of standby daemons 1158
Configuring the standby-replay Metadata Server 1159
Ephemeral pinning policies 1159
Manually pinning directory trees to a particular rank 1161
Decreasing the number of active Metadata Server daemons 1161
Viewing metrics for Ceph metadata server clients 1163
Deploying the CephFS 1165
Layout, quota, snapshot, and network restrictions 1166
Creating Ceph File Systems 1167
Adding an erasure-coded pool to a Ceph File System 1169
Creating client users for a Ceph File System 1171
CephFS client capabilities 1172

Authorizing CephFS clients ... 1173
Mounting the Ceph File System as a kernel client ... 1174
Mounting the Ceph File System as a FUSE client ... 1177
Managing CephFS volumes ... 1180
CephFS volumes ... 1181
CephFS subvolumes ... 1183
CephFS subvolume groups ... 1193
Metadata information on CephFS subvolumes ... 1198
Administering CephFS ... 1202
Using the `cephfs-top` utility ... 1202
Using the MDS autoscaler module ... 1205
Unmounting Ceph File Systems mounted as kernel clients ... 1205
Unmounting Ceph File Systems mounted as FUSE clients ... 1206
Mapping directory trees to Metadata Server daemon ranks ... 1206
Disassociating directory trees from Metadata Server daemon ranks ... 1207
Multiple data pools use-cases ... 1208
Adding data pools ... 1209
Taking down a CephFS cluster ... 1210
Removing a CephFS ... 1211
Using the **`ceph mds fail`** command ... 1212
Client features ... 1212
CephFS client evictions ... 1213
Blocklist CephFS clients ... 1214
Manually evicting a CephFS client ... 1214
Removing a CephFS client from the blocklist ... 1216
Disabling volumes plug-in ... 1216
Pausing asynchronous subvolume purge and clone operations ... 1217
CephFS subvolume quiesce ... 1218
Operations ... 1218
CephFS subvolume quiesce parameters ... 1220
CephFS quotas ... 1224
Limitations ... 1224
Viewing quotas ... 1224
Setting quotas ... 1225
Removing quotas ... 1226
File and directory layouts ... 1226
Layouts inheritance ... 1227
Setting file and directory layout fields ... 1227
Viewing file and directory layout fields ... 1227
Viewing individual layout fields ... 1228
Removing directory layouts ... 1228
CephFS snapshots ... 1229
CephFS snapshots usage ... 1229
CephFS limitations ... 1229
Creating a snapshot for a Ceph File System ... 1229
Cloning subvolumes from snapshots ... 1230
CephFS snapshot schedules ... 1232
Adding a snapshot schedule for a CephFS ... 1233
Adding a snapshot schedule for CephFS subvolume ... 1235
Activating snapshot schedule for a CephFS ... 1239
Activating snapshot schedule for a CephFS subvolume ... 1240
Deactivating snapshot schedule for a CephFS ... 1241
Deactivating snapshot schedule for a CephFS subvolume ... 1241
Removing a snapshot schedule for a CephFS ... 1242
Removing a snapshot schedule for a CephFS subvolume ... 1243
Removing snapshot schedule retention policy for a CephFS ... 1244
Removing snapshot schedule retention policy for a CephFS subvolume ... 1244
CephFS snapshot mirroring (Technology Preview) ... 1246
Ceph Manager Module and interfaces ... 1246
Snapshot incarnation ... 1246
High availability ... 1247
Readdition of Ceph File System mirror peers ... 1247
Configuring a snapshot mirror for a Ceph File System ... 1247
Viewing the mirror status for a Ceph File System ... 1250
Viewing metrics for Ceph File System snapshot mirroring ... 1253
Red Hat Ceph Storage Block Devices ... 1256
Using Ceph Block Devices ... 1256
Displaying command help ... 1256
Creating block device pools ... 1257
Creating images ... 1257
Listing images ... 1258
Retrieving image information ... 1258
Resizing images ... 1259
Removing images ... 1259
Moving images to trash ... 1260
Schedule automatic trash purge ... 1261
Enabling and disabling image features ... 1262

Working with image metadata ... 1263
Moving images between pools ... 1264
Migrating pools ... 1266
rbdmap service ... 1266
Configuring rbdmap service ... 1267
Persistent write log cache ... 1267
Monitoring performance of Ceph Block Devices ... 1271
Ceph user and keyring ... 1272
Live migration of images ... 1273
Live migration formats ... 1273
Live migration streams ... 1275
Preparing for default live migration ... 1276
Preparing import-only migration ... 1278
Running the live migration ... 1280
Committing the live migration ... 1281
Stopping a live migration ... 1282
Image encryption ... 1282
Encryption format ... 1283
Encryption load ... 1283
Supported formats ... 1284
Adding encryption format to images and clones ... 1285
Managing snapshots ... 1286
Creating snapshots ... 1287
Listing snapshots ... 1287
Roll back snapshots ... 1288
Deleting snapshot ... 1288
Purging snapshots ... 1289
Renaming snapshots ... 1289
Ceph Block Device layering ... 1290
Protecting snapshots ... 1291
Cloning block device snapshot ... 1291
Unprotecting snapshots ... 1293
Listing children of a snapshot ... 1293
Flattening cloned images ... 1294
Mirroring Ceph Block Devices ... 1294
One-way replication ... 1295
Two-way replication ... 1296
Configuring Ceph Block Device mirroring ... 1297
Administering Ceph Block Device mirroring ... 1301
Setting up mirroring between the default and non-default namespaces ... 1318
Disaster recovery ... 1321
Managing ceph-immutable-object-cache daemons ... 1328
Reasons to use ceph-immutable-object-cache daemons ... 1328
Connection to the cluster ... 1328
Key components of the daemon ... 1329
How librbd uses the cache daemon ... 1329
Configuration ... 1329
Generic settings ... 1331
QoS settings for ceph-immutable-object-cache daemons ... 1331
rbd kernel module ... 1332
Creating a Ceph Block Device and using it from a Linux kernel module client ... 1332
Mapping a block device ... 1335
Displaying mapped block devices ... 1336
Unmapping a block device ... 1336
Ceph Block Device python module ... 1336
Prerequisites ... 1336
Procedure ... 1336
Ceph Block Device configuration reference ... 1337
Default options ... 1337
General options ... 1341
Caching options ... 1343
Parent and child read options ... 1344
Read ahead options ... 1345
Block-list options ... 1346
Journal options ... 1346
Configuration override options ... 1347
Input and output options ... 1348
Red Hat Ceph Storage Block Devices and OpenStack ... 1351
Ceph and OpenStack integration models ... 1351
OpenStack services that use Ceph Block Devices ... 1351
Image format and storage considerations ... 1351
Installing and configuring Ceph for OpenStack ... 1352
Creating Ceph pools for OpenStack ... 1352
Installing the Ceph client on OpenStack ... 1353
Copying the Ceph configuration file to OpenStack ... 1353
Configuring Ceph client authentication ... 1354
Configuring OpenStack to use Ceph Block Devices ... 1355

Configuring Cinder to use Ceph Block Devices ... 1355
Configuring Cinder Backup to use Ceph Block Devices ... 1356
Configuring Glance to use Ceph Block Devices ... 1357
Configuring Nova to use Ceph Block Devices ... 1358
Restarting the OpenStack services ... 1359
Red Hat Ceph Storage Developer ... 1360
Ceph RESTful API ... 1360
API documentation ... 1360
Understanding API parameters ... 1361
Understanding API responses ... 1361
Understanding API errors ... 1361
Authentication and authorization for the Ceph API ... 1361
Enabling and securing the Ceph API ... 1362
Using the Ceph RESTful API ... 1362
Versioning for the Ceph API ... 1362
Authentication and authorization for the Ceph API ... 1362
Enabling and Securing the Ceph API ... 1364
Using the Ceph RESTful API ... 1366
Changing OSD configurations ... 1379
Administering the cluster ... 1383
Ceph Object Gateway administrative API ... 1386
Administration operations ... 1387
Administration authentication requests ... 1387
Creating an administrative user ... 1392
Get user information ... 1394
Create a user ... 1396
Modify a user ... 1402
Remove a user ... 1407
Create a subuser ... 1408
Modify a subuser ... 1410
Remove a subuser ... 1413
Add capabilities to a user ... 1414
Remove capabilities from a user ... 1415
Create a key ... 1417
Remove a key ... 1420
Bucket notifications ... 1421
Get bucket information ... 1431
Check a bucket index ... 1434
Remove a bucket ... 1435
Link a bucket ... 1436
Unlink a bucket ... 1438
Get a bucket or object policy ... 1439
Remove an object ... 1440
Ceph Object Gateway quota operation ... 1441
Get usage information ... 1442
Remove usage information ... 1446
Standard error responses ... 1447
Ceph Object Gateway and the S3 API ... 1448
Prerequisites ... 1448
S3 limitations ... 1448
Accessing the Ceph Object Gateway with the S3 API ... 1448
S3 bucket operations ... 1464
S3 object operations ... 1501
S3 select operations ... 1537
Ceph Object Gateway and the Swift API ... 1554
Swift API limitations ... 1554
Swift function feature support ... 1554
Create a Swift user ... 1555
Swift authenticating a user ... 1557
Swift container operations ... 1558
Swift object operations ... 1564
Swift temporary URL operations ... 1569
Swift multi-tenancy container operations ... 1570
Ceph RESTful API specifications ... 1571
S3 common request headers ... 1571
S3 common response status codes ... 1571
S3 supported and unsupported verbs ... 1572
S3 unsupported header fields ... 1575
Swift request headers ... 1576
Swift response headers ... 1576
Examples using the Secure Token Service APIs ... 1576
Reference ... 1578
Red Hat Ceph Storage Troubleshooting ... 1579
Initial troubleshooting ... 1579
Identifying problems ... 1579
Diagnosing health ... 1579

Understanding Ceph health ... 1580
Muting health alerts ... 1580
Understanding Ceph logs ... 1582
Generating an `sos report` ... 1582
Providing information for support ... 1583
Configuring logging ... 1584
Ceph subsystems ... 1584
Configuring logging at runtime ... 1586
Configuring logging in configuration file ... 1587
Accelerating log rotation ... 1588
Creating and collecting operation logs for Ceph Object Gateway ... 1589
Troubleshooting networking issues ... 1589
Basic networking troubleshooting ... 1590
Basic chrony NTP troubleshooting ... 1593
`cephadm` troubleshooting ... 1594
Pause or disable cephadm ... 1594
Per service and per daemon event ... 1594
Check `cephadm` logs ... 1595
Gather log files ... 1595
Collect systemd status ... 1596
List all downloaded container images ... 1596
Manually run containers ... 1596
CIDR network error ... 1597
Access admin socket ... 1598
Manually deploying a `mgr` daemon ... 1598
Troubleshooting Ceph Monitors ... 1599
Most common Ceph Monitor errors ... 1599
Injecting a `monmap` ... 1605
Replacing a failed Monitor ... 1606
Compacting Monitor store ... 1607
Opening port for Ceph Manager ... 1608
Recovering Ceph Monitor store ... 1609
Troubleshooting Ceph OSDs ... 1609
Most common Ceph OSD errors ... 1610
Stopping and starting rebalancing ... 1619
Mounting the OSD data partition ... 1619
Replacing an OSD drive ... 1620
Increasing PID count ... 1623
Deleting data from a full storage cluster ... 1623
Troubleshooting multi-site Ceph Object Gateway ... 1624
Error code definitions for Ceph Object Gateway ... 1624
Syncing a multi-site Ceph Object Gateway ... 1625
Synchronizing data in a multi-site Ceph Object Gateway configuration ... 1627
Troubleshooting **`radosgw-admin`** commands after upgrading a cluster ... 1628
Troubleshooting Ceph placement groups ... 1628
Most common Ceph placement groups errors ... 1629
Listing placement groups stuck in *stale*, *inactive*, or *unclean* states ... 1635
Listing placement group inconsistencies ... 1635
Repairing inconsistent placement groups ... 1639
Increasing the placement group ... 1639
Interpreting placement group dump output ... 1641
Troubleshooting Ceph objects ... 1644
Troubleshooting high-level object operations ... 1644
Troubleshooting low-level object operations ... 1646
Troubleshooting clusters in stretch mode ... 1653
Forcing stretch cluster into recovery or healthy mode ... 1654
Troubleshooting scrub and deep-scrub issues ... 1654
Addressing scrub slowness issue after upgrading from Red Hat Ceph Storage 6 to Red Hat Ceph Storage 8 and Red Hat Ceph Storage 7 to Red Hat Ceph Storage 8 ... 1654
Ceph subsystems default logging level values ... 1655
Troubleshooting upgrade error messages ... 1657
Health messages of a Ceph cluster ... 1657

Red Hat Ceph Storage Compatibility

This section describes compatibility of Red Hat Ceph Storage with other products.

Compatibility matrix for Red Hat Ceph Storage 9.0

Use this information for products supported by Red Hat Ceph Storage 9.0.

The following tables list supported compatibility with Red Hat Ceph Storage 9.0.

- Supported host operating system
- Supported products
- Analytics and AI
- Supported products for Red Hat Ceph Storage as a backup target
- Independent software vendors
- Supported S3 endpoints for Object Storage policy based archive and retrieval

For more information and the latest supported configurations, see IBM Storage Ceph Supported configurations.

Table 1: Supported host operating system

Host operating system	Version	Notes
Red Hat Enterprise Linux (RHEL)	– 9.6, 9.7 – 10, 10.1	– Standard lifecycle RHEL is included in the product. – FIPS mode is supported. For more information and access to FIPS builds, contact your Red Hat representative.

Use Ceph client packages included with any currently supported Red Hat Enterprise Linux distribution. Red Hat supports RHEL clients as part of the product subscription. Third party drivers can be utilized with the Red Hat Ceph Storage cluster, with client support from their respective vendor. Ideally, the Ceph client packages should correspond with the Ceph release that is deployed in the Ceph cluster that the client needs to connect to.

Important: All nodes in the cluster and their clients must use the supported OS version(s) to ensure that the version of the `ceph` package is the same on all nodes. Using different versions of the `ceph` package is not supported.

Table 2: Supported products

Product	Version	Notes
Ansible	Supported in a limited capacity.	Supported for upgrade and conversion to `cephadm` and for other minimal playbooks.
Red Hat OpenShift Data Foundation	See the Red Hat OpenShift Data Foundation Supportability and Interoperability Checker for detailed external mode version compatibility.	None
Red Hat OpenStack Services on OpenShift	18.0.17 Feature Release 5 or later	Supports Red Hat Ceph Storage or with HyperConverged Infrastructure (HCI).

Product	Version	Notes
Red Hat Satellite	6.x	Only registering with the Content Delivery Network (CDN) is supported. Registering with Red Hat Network (RHN) is deprecated and not supported.

Table 3: Analytics and AI

Client connector	Version	Notes
Hadoop S3A	2.8.x, 3.2.x, and trunk	None
Polaris Catalog	0.9.0 or later	`S3_COMPATIBLE` allows Credential Vending (IAM/STS)
Weka Data Platform	3.12.2	None

Table 4: Supported products for Red Hat Ceph Storage as a backup target

Red Hat Ceph Storage as a backup target	Version	Notes
CommVault Platform	v11.20 or later	S3 target
IBM Spectrum Protect Plus	10.1.5	None
IBM Spectrum Protect server	8.1.18	S3 target (Object Lock)
NetApp AltaVault	4.3.2 and 4.4	None
Rubrik Cloud Data Management (CDM)	3.2 or later	None
Trilio, TrilioVault	3.0	S3 target
Veeam (object storage)	– Repository Primary Target Veeam v12 – Repository Capacity Target with Immutability Veeam v12	Supported on Red Hat Ceph Storage object storage with the S3 protocol
Veritas NetBackup Cloud Storage	10.x or later	S3 target

Table 5: Independent software vendors

Independent software vendors	Version	Notes
IBM Spectrum Discover	2.0.3	None
Komprise Intelligent Data Management	6.3.0 or later	None

Table 6: External key managers

External key manager	Version	Notes
IBM Security Key Lifecycle Manager (GKLM/SKLM)	5.1	SSE-KMS

Table 7: Supported identity providers (SSO)

Supported identity providers	Version	Notes
Red Hat Build of Keycloak (RHBK)	26.2 or later	Dashboard SSO, Object STS/OIDC auth

Table 8: Supported S3 endpoints for Object Storage policy based archive and retrieval

S3 endpoint	Version	Notes
AWS S3	N/A	S3 API

S3 endpoint	Version	Notes
Red Hat Ceph Storage	7.1 or later	S3 API
PoINT Archival Gateway (Tape)	4.1 or later	S3 API

For more information, see “Policy based data archival and retrieval to S3 compatible platforms” on page 1061.

Red Hat Ceph Storage Getting started

Learn about how to get started with Red Hat Ceph Storage and its basic features. This information is designed for customers that are new to Red Hat Ceph Storage or established customers wanting an overview of how Red Hat Ceph Storage works and where to start in the workflow.

This information provides basic workflows for Red Hat Ceph Storage. For detailed instructions links to the correct documentation section are provided.

Before you begin to start working with Red Hat Ceph Storage, familiarize yourself with the following information:

- “Compatibility matrix for Red Hat Ceph Storage 9.0” on page 13
- “Minimum hardware considerations” on page 99
- “Network considerations for Red Hat Ceph Storage” on page 88
- “Basic considerations” on page 84

Object storage

The Ceph Object Gateway client is a leading storage backend for cloud platforms that provides a RESTful S3-compliant and Swift-compliant object storage for objects like audio, bitmap, video, and other data. Ceph Object Gateway, also known as RADOS Gateway (RGW), is an object storage interface built on top of the librados library to provide applications with a RESTful gateway to Ceph storage clusters.

Go through the following sections to get started with Ceph Object Gateways (object storage)

Object storage interfaces

The following are the object storage interfaces:

Administrative API

Provides an administrative interface for managing the Ceph Object Gateways. For more information, see Ceph Object Gateway administrative API.

S3

Provides object storage functionality with an interface that is compatible with a large subset of the Amazon S3 RESTful API. For more information, see Ceph Object Gateway and the S3 API.

Swift

Provides object storage functionality with an interface that is compatible with a large subset of the OpenStack Swift API. The Ceph Object Gateway is a service interacting with a Ceph storage cluster. For more information, see Ceph Object Gateway and the Swift API.

Common workloads

The following are the most common workloads for using Ceph Object Gateway:

Data efficiency

Use for erasure coding, thin provisioning, lifecycle management, and compression.

Data security

Use for object lock, key management, at rest and inflight encryption, and WORM.

Data resilience

Use for backup, snapshots, cloning, and business continuity.

Common use cases

The following are some of the most common uses cases for Ceph Object Gateway.

Storage as a Service (SaaS)

Provides scalability and performance for both small and large object stores.

AI, Analytics and Big Data including Data Lake and Data Lake House

Cloud native data lake, with massive scalability and high availability to support demanding workloads. For example, you can use Red Hat Ceph Storage to provide an external S3 object store for IBM watsonx.data.

Backup and archive or large amounts of unstructured data

A unique new way of architecting the dataflow in applications which is through event driven architectures.

Data intensive workloads like Cloud Native (S3) object data

Back up and recover into and from an object storage helps improve recovery time objectives (RTO) and recovery point objectives (RPO).

Internet of Things (IoT)

Object gateways serve as intermediaries in IoT systems, aggregating data from various devices, translating communication protocols, and enabling edge processing. They enhance security, facilitate device management, and ensure interoperability, streamlining the overall IoT ecosystem.

Getting started with object storage

Get to know the relevant tasks that are required for working with Ceph Object Gateway.

See Compatibility matrix for a list of backup vendors that are certified with Red Hat Ceph Storage as an S3 target.

There are specific network and hardware requirements to work with Ceph object storage. For more information, see Considerations and recommendations.

Managing Object Gateway through the dashboard

For detailed information, see “Managing Ceph Object Gateway” on page 624.

Deploying the Ceph Object Gateway using CLI

For detailed information, see Deploying the Ceph Object Gateway using CLI.

Setting up Ingress service

For detailed information, see High availability service.

Setting up S3 server-side security

For detailed information, see Server side encryption.

Multi-site Asynchronous replication for High Availability and Disaster Recovery

For detailed information, see Failover and disaster recovery.

Deploying Ceph Object Gateway with Ceph Orchestrator

Ceph Object Gateway is deployed by either using the Ceph Orchestrator with the command line interface or by using the service specification. You can also configure multi-site Ceph Object Gateways, and remove the Ceph Object Gateway using the Ceph Orchestrator. The **cephadm** command deploys the Ceph Object Gateway as a collection of daemons that manages a single-cluster deployment or a particular realm and zone in a multi-site deployment.

For full Ceph Object Gateway deployment information and instructions, see Deployment.

Alternatively, you can deploy Ceph Object Gateway using the command-line interface. For more information, see Deploying the Ceph Object Gateway using the command line interface.

Creating S3 users and testing S3 access

- For detailed information on creating S3 users, see Create an S3 user.
- For detailed information on testing S3 access, see Test S3 access.
- For detailed information about managing Ceph Object Gateway S3 users, see Management of Ceph Object Gateway users.

Block storage

Red Hat Ceph Storage uses block storage, and refers to this as Ceph Block Devices. Block-based storage interfaces are the most common way to store data with rotating media such as hard drives and both SSD and NVMe flash storage.

Ceph Block Devices interact with OSDs by using the `librbd` library.

Ceph Block Devices deliver high performance with infinite scalability to Kernel Virtual Machines (KVMs), such as Quick Emulator (QEMU), and cloud-based computing systems, like OpenStack, that rely on the `libvirt` and QEMU utilities to integrate with Ceph Block Devices. You can use the same storage cluster to operate the Ceph Object Gateway and Ceph Block Devices simultaneously.

Ceph Block Devices can easily be managed through either the Ceph dashboard or command-line interface (CLI) commands. For detailed information about Ceph Block Devices, see Ceph Block Devices.

Common workloads

The following are the most common workloads for using Ceph Block Devices:

Database store

Use for data protection application database backup.

Device mirroring

Use to protect against data loss or site failures.

Data resiliency

Use for replication and erasure coding.

Common use cases

The following are some of the most common uses cases for Ceph Block Devices:

- Storage for Virtualization, Hypervisors, and private clouds.
- Persistent storage for containers (Kubernetes).
- Storage for applications that require low-latency and high performance, such as databases.
- Virtual machines for VMware, OpenStack and OpenShift.
- Unified file and object storage.
  - Ingest data via file protocols from existing (legacy) enterprise applications into a common repository based on an object store.
  - Once it is in the repository, access the data via S3 for analytics, machine-learning, and so on.

Example workflow for using Ceph Block Device snapshots and clones

Learn the generic working configurations that can be configured within the context of your environment. For detailed information about the procedures see the relevant sections within the documentation.

This information provides an example workflow for using Ceph Block Device snapshots and clones. This workflow includes creating, listing, renaming, layering, protecting, and cloning Ceph Block Device snapshots.

For detailed information, see Managing snapshots.

1. Create, list, and rename Ceph Block Device snapshots.

```
[root@rbd-client ~]# rbd --pool pool1 snap create --snap snap1 image1
[root@rbd-client ~]# rbd --pool pool1 --image image1 snap ls
[root@rbd-client ~]# rbd snap ls pool1/image1
[root@rbd-client ~]# rbd snap rename data/dataset@snap1 data/dataset@snap2
```

2. Layer Ceph Block Device snapshots.
   Within the `ceph.conf` file, enable this feature by adding `rbd_clone_copy_on_read = true` into either the `[global]` or `[client]` section.

   For detailed information, see Ceph Block Device layering.

3. Protect and clone Ceph Block Device snapshots.

```
[root@rbd-client ~]# ceph osd set-require-min-compat-client mimic
[root@rbd-client ~]# rbd clone --pool pool1 --image image1 --snap snap2 --dest-pool
pool2 --dest childimage1
[root@rbd-client ~]# rbd clone pool1/image1@snap1 pool1/childimage1
```

Getting started with block storage

Get to know the relevant tasks and most common commands that are required for working with Ceph Block Devices.

Managing Ceph Block Devices with the dashboard

Manage Ceph Block Devices by using the Red Hat Ceph Storage dashboard. As a storage administrator, you can manage and monitor block device images on the Red Hat Ceph Storage dashboard. The functionality is divided between generic image functions and mirroring functions. For example, you can create new images, view the state of images that are mirrored across clusters, and set IOPS limits on an image.

For detailed information, see Managing block devices.

Common block image CLI commands

This information is for a quick reference of basic block image CLI commands. For a full list and more detailed information about each command, see Ceph Block Devices.

Creating images

```
rbd create IMAGE_NAME --size MEGABYTES --pool POOL_NAME
```

> **Important:** A pool must be created before creating a block image. For details, see "Creating block device pools" on page 1257.

Listing images

```
rbd ls POOL_NAME
```

Retrieving image information from a particular image in the default pool

```
rbd --image IMAGE_NAME info
```

Retrieving information from an image within a pool

```
rbd --image IMAGE_NAME -p POOL_NAME info
```

Resizing images

Increasing the maximum size of a Ceph Block Device image for the default `rbd` pool

```
rbd resize --image IMAGE_NAME --size SIZE
```

Increasing the maximum size of a Ceph Block Deivce image for a specific pool

```
rbd resize --image POOL_NAME/IMAGE_NAME --size SIZE
```

Decreasing the maximum size of a Ceph Block Device image for the default `rbd` pool

```
rbd resize --image IMAGE_NAME --size SIZE --allow-shrink
```

Decreasing the maximum size of a Ceph Block Device image for a specific pool

```
rbd resize --image POOL_NAME/IMAGE_NAME --size SIZE --allow-shrink
```

Moving images to the trash

```
rbd trash mv POOL_NAME/IMAGE_NAME
```

Restoring an image from the trash

```
rbd trash restore POOL_NAME/IMAGE_NAME
```

Ensuring the `rbd_support` Ceph Manager module is enabled

```
ceph mgr module ls
```

File storage

Red Hat Ceph Storage provides file storage with the Ceph File System (CephFS).

CephFS provides shared file access to a Red Hat Ceph Storage cluster and uses POSIX semantics wherever possible. CephFS is built on top of the Ceph distributed object store, called RADOS (Reliable Autonomic Distributed Object Store). CephFS inherits all of the architectural benefits of RADOS, providing high availability, built-in data redundancy, shared and parallel data access and an infinitely scalable architecture.

Common use cases

The following are some of the most common use cases for CephFS:

- File System as a Service (FSaaS).
- General purpose network file system.
- Home directories.
- Applications that require multiple readers, writers, or both. For example, High Performance Computing (HPC).
- Media and content repositories.

Example use cases for file storage as a service (FSaaS)

Get to know the generic working configurations that can be configured within the context of your environment. For detailed information about the procedures see the relevant sections within the documentation.

See the following workflows to get to know the main use cases for FSaaS:

- “Example workflow for using Ceph File System” on page 20
- “Example workflow for using CephFS snapshot” on page 21

Example workflow for using Ceph File System

This information provides an example workflow for using CephFS.

1. Create a Ceph File system volume or subvolume.
   For detailed information, see “Managing CephFS volumes” on page 1180.

   ```
   [root@client01 ~]# subscription-manager repos --enable=rhceph-9-tools-for-rhel-9-
   x86_64-rpms
   [root@client01 ~]# dnf install ceph-common
   [root@host01 ~]# cephadm shell
   [root@client ~]# dnf install ceph-fuse
   [root@client ~]# scp root@192.168.0.1:/etc/ceph/ceph.client.1.keyring /etc/ceph/
   [root@client ~]# scp root@192.168.0.1:/etc/ceph/ceph.conf /etc/ceph/ceph.conf
   [root@client ~]# chmod 644 /etc/ceph/ceph.conf
   [root@mon ~]# ceph fs volume create cephfs01
   ```

2. Set Ceph client credentials for CephFS.
   For detailed information, see Creating client users for a Ceph File System.

   Part of setting client credentials is to restrict the client to either the entire `temp` directory or to only write in the `temp` directory of file system `cephfs_a`. This example provides the instructions for restricting the client to the `temp` directory.

```
[ceph: root@host01 ~]# ceph fs authorize cephfs_a client.1 /temp rw
[ceph: root@host01 ~]# ceph auth get client.1

client.1
  key = AQBSdFhcGZFUDRAAcKhG9Cl2HPiDMMRv4DC43A==
  caps mds = "allow r, allow rw path=/temp"
  caps mon = "allow r"
  caps osd = "allow rw tag cephfs data=cephfs_a"
[ceph: root@host01 ~]# ceph auth get client.1 -o ceph.client.1.keyring
exported keyring for client.1
[ceph: root@host01 ~]# scp /ceph.client.1.keyring root@client01:/etc/ceph/
ceph.client.1.keyring
[root@client01 ~]# chmod 644 /etc/ceph/ceph.client.1.keyring
```

3. Mount a CephFS volume on a client. The volume can be mounted as either a kernel or a FUSE client.

   Example for mounting file storage as a kernel client with automatic mounting

   ```
   [root@client01 ~]# ceph auth get-key client.1 > /etc/ceph/1.secreat
   mount -t ceph 10.0.195.41,10.0.195.177,10.0.195.78:/ /mnt/cephfs -o
   name=1,secretfile=/etc/ceph/1.secret,fs=cephfs-ec
   ```

   For detailed information, see Deploying the CephFS.

   Example for mounting file storage as a FUSE client

   ```
   [root@client ~]# mkdir /mnt/mycephfs
   [root@client ~]# ceph-fuse /mnt/mycephfs/ -n client.1 --client-fs=cephfs01
   ceph-fuse[555001]: starting ceph client
   2022-05-09T07:33:27.158+0000 7f11feb81200 -1 init, newargv = 0x55fc4269d5d0
   newargc=15
   ceph-fuse[555001]: starting fuse
   ```

Example workflow for using CephFS snapshot

This information provides an example workflow for using CephFS snapshots. For detailed information, see “Creating a snapshot for a Ceph File System” on page 1229.

1. Create a snapshot for a CephFS.

   ```
   [ceph: root@host01 ~]# cd /mnt/cephfs
   [ceph: root@host01 ~]# touch before_snapshot
   [ceph: root@host01 cephfs ~]# mkdir .snap/newsnap
   ```

   Note: The CLI command can be used to create subvolume snapshots. While `mkdir` and `rmdir` can still be used to create and delete snapshots, the CLI command is the most current method and is recommended for use.

2. Verify and use the snapshot.

   ```
   [ceph: root@host01 cephfs ~]# touch after_snapshot
   [ceph: root@host01 cephfs ~]# ls -al .snap/newsnap
   ```

3. Delete the snapshots from a CephFS.

   Important: Attempting to delete root-level snapshots, which might contain underlying snapshots, will fail.

   ```
   [ceph: root@host01 ~]# rmdir .snap/newsnap
   [ceph: root@host01 ~]# ls -al .snap/
   ```

Getting started with file storage

Get to know the relevant tasks that are required for working with Ceph File Systems (CephFS).

Limitations

To know about the limitations and POSIX standards to consider when working with Ceph File System, see “CephFS limitations and the POSIX standards” on page 1149.

Deploying the Ceph File System

Detailed deployment instructions for Ceph File System is found in “Deploying the CephFS” on page 1165.

Managing Ceph File Systems through the dashboard

For detailed information on managing Ceph File Systems from the dashboard, see “Managing Ceph File Systems (CephFS)” on page 762.

Creating client users for a Ceph File System

For detailed information, see “Creating client users for a Ceph File System” on page 1171 .

Setting up the Ceph File System

Use the following procedures to setup a Ceph File System.

1. “Creating client users for a Ceph File System” on page 1171.
2. “Mounting the Ceph File System as a kernel client” on page 1174 or “Mounting the Ceph File System as a FUSE client” on page 1177.
3. “Client features” on page 1212.

Basic Ceph File System CLI commands

This information is for a quick reference of basic CephFS CLI commands. For a full list and more detailed information about each command, see “Managing CephFS volumes” on page 1180.

Creating a CephFS volume

```
ceph fs volume create VOLUME_NAME
```

Creating a CephFS subvolume

```
ceph fs subvolume create VOLUME_NAME SUBVOLUME_NAME [--size SIZE_IN_BYTES --group_name
SUBVOLUME_GROUP_NAME --pool_layout DATA_POOL_NAME --uid UID --gid GID --mode OCTAL_MODE]
[--namespace-isolated]
```

Creating a CephFS subvolume group

```
ceph fs subvolumegroup create VOLUME_NAME GROUP_NAME [--pool_layout DATA_POOL_NAME --uid
UID --gid GID --mode OCTAL_MODE]
```

Listing CephFS volumes

```
ceph fs volume ls
```

Listing CephFS subvolumes

```
ceph fs subvolume ls VOLUME_NAME
```

Listing CephFS subvolume groups

```
ceph fs subvolumegroup ls VOLUME_NAME
```

Viewing information about a CephFS volume

```
ceph fs volume info VOLUME_NAME
```

Removing a CephFS volume

```
ceph fs volume rm VOLUME_NAME [--yes-i-really-mean-it]
```

Removing a CephFS subvolume

```
ceph fs subvolume rm VOLUME_NAME SUBVOLUME_NAME
```

Removing a CephFS subvolume group

```
ceph fs subvolumegroup rm VOLUME_NAME GROUP_NAME [--force]
```

Creating a snapshot of a CephFS subvolume

```
ceph fs subvolume snapshot create VOLUME_NAME SUBVOLUME_NAME SNAP_NAME [--group_name
GROUP_NAME]
```

Red Hat Ceph Storage Overview

Learn about the architecture, data security, and hardening concepts for Red Hat Ceph Storage.

Introduction to Red Hat Ceph Storage

Red Hat Ceph Storage cluster is a distributed data object store designed to provide excellent performance, reliability and scalability.

Red Hat Ceph Storage is a scalable, open, software-defined storage platform that combines an enterprise-hardened version of the Ceph storage system, with a Ceph management platform, deployment utilities, and support services. Red Hat Ceph Storage is designed for cloud infrastructure and web-scale object storage.

Distributed object stores are the future of storage, because they accommodate unstructured data, and because clients can use modern object interfaces and legacy interfaces simultaneously.

For example:

- APIs in many languages (C/C++, Java, Python)
- RESTful interfaces (S3/Swift)
- Block device interface
- File system interface

The power of Red Hat Ceph Storage cluster can transform your organization's IT infrastructure and your ability to manage vast amounts of data, especially for cloud computing platforms like Red Hat Enterprise Linux OSP. The cluster delivers extraordinary scalability–thousands of clients accessing petabytes to exabytes of data and beyond.

At the heart of every Ceph deployment is the Red Hat Ceph Storage cluster. It consists of three types of daemons:

Ceph OSD Daemon

Ceph OSDs store data on behalf of Ceph clients. Additionally, Ceph OSDs utilize the CPU, memory and networking of Ceph nodes to perform data replication, erasure coding, rebalancing, recovery, monitoring and reporting functions.

Ceph Monitor

A Ceph Monitor maintains a master copy of the Red Hat Ceph Storage cluster map with the current state of the cluster. Monitors require high consistency, and use Paxos to ensure agreement about the state of the cluster.

Ceph Manager

The Ceph Manager maintains detailed information about placement groups, process metadata and host metadata in lieu of the Ceph Monitor—significantly improving performance at scale. The Ceph Manager handles execution of many of the read-only Ceph CLI queries, such as placement group statistics. The Ceph Manager also provides the RESTful monitoring APIs.

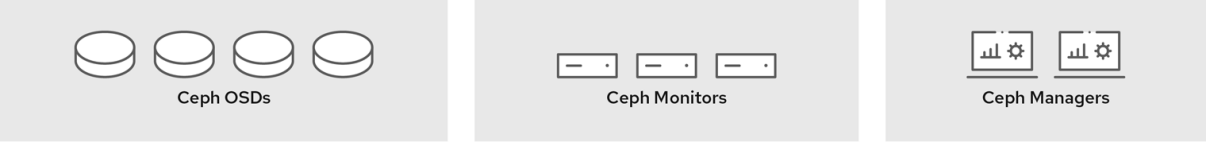


Figure 1: Daemons

Ceph client interfaces read data from and write data to the Red Hat Ceph Storage cluster. Clients need the following data to communicate with the Red Hat Ceph Storage cluster:

- The Ceph configuration file, or the cluster name (usually `ceph`) and the monitor address.
- The pool name.

– The user name and the path to the secret key.

Ceph clients maintain object IDs and the pool names where they store the objects. However, they do not need to maintain an object-to-OSD index or communicate with a centralized object index to look up object locations. Then, Ceph clients provide an object name and pool name to `librados`, which computes an object's placement group and the primary OSD for storing and retrieving data using the CRUSH (Controlled Replication Under Scalable Hashing) algorithm. The Ceph client connects to the primary OSD where it may perform read and write operations. There is no intermediary server, broker or bus between the client and the OSD.

When an OSD stores data, it receives data from a Ceph client, whether the client is a Ceph Block Device, a Ceph Object Gateway, a Ceph File System or another interface and it stores the data as an object.

> **Note:** An object ID is unique across the entire cluster, not just an OSD's storage media.

Ceph OSDs store all data as objects in a flat namespace. There are no hierarchies of directories. An object has a cluster-wide unique identifier, binary data, and metadata consisting of a set of name/value pairs.

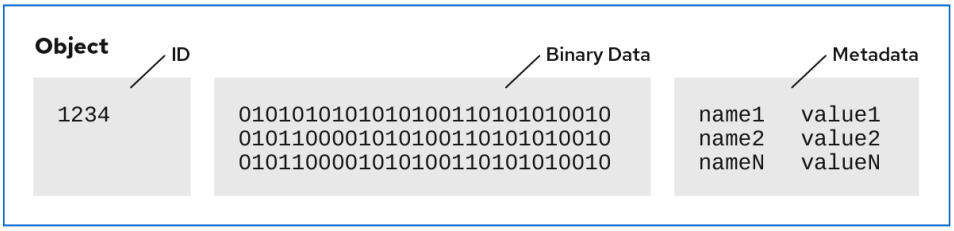


Figure 2: Object

Ceph clients define the semantics for the client's data format. For example, the Ceph Block Device maps a block device image to a series of objects stored across the cluster.

> **Note:** Objects consisting of a unique ID, data, and name/value paired metadata can represent both structured and unstructured data, as well as legacy and leading edge data storage interfaces.

Red Hat Ceph Storage clusters consist of the following types of nodes:

Ceph Monitor

Each Ceph Monitor node runs the `ceph-mon` daemon, which maintains a primary copy of the storage cluster map. The storage cluster map includes the storage cluster topology. A client connecting to the Ceph storage cluster retrieves the current copy of the storage cluster map from the Ceph Monitor, enabling the client to read from and write data to the storage cluster.

> **Important:** The storage cluster can run with just one Ceph Monitor; however, to ensure high availability in a production storage cluster, Red Hat supports deployments with at least three Ceph Monitor nodes. Deploy a total of 5 Ceph Monitors for storage clusters exceeding 750 Ceph OSDs.

Ceph Manager

The Ceph Manager daemon, `ceph-mgr`, co-exists with the Ceph Monitor daemons running on Ceph Monitor nodes to provide extra services. The Ceph Manager provides an interface for other monitoring and management systems using Ceph Manager modules. Running the Ceph Manager daemons is a requirement for normal storage cluster operations.

Ceph OSD

Each Ceph Object Storage Device (OSD) node runs the `ceph-osd` daemon, which interacts with logical disks that are attached to the node. The storage cluster stores data on these Ceph OSD nodes.
Ceph can run with few OSD nodes, of which the default is three, but production storage clusters realize better performance beginning at modest scales. For example, 50 Ceph OSDs in a storage cluster. Ideally, a

Ceph storage cluster has multiple OSD nodes, allowing for the possibility to isolate failure domains by configuring the CRUSH map.

Ceph MDS

Each Ceph Metadata Server (MDS) node runs the `ceph-mds` daemon, which manages metadata related to files stored on the Ceph File System (CephFS). The Ceph MDS daemon also coordinates access to the shared storage cluster.

Ceph Object Gateway

Ceph Object Gateway node runs the `ceph-radosgw` daemon, and is an object storage interface built on top of `librados` to provide applications with a RESTful access point to the Ceph storage cluster. The Ceph Object Gateway supports two interfaces:

S3

Provides object storage functionality with an interface that is compatible with a large subset of the Amazon S3 RESTful API.

Swift

Provides object storage functionality with an interface that is compatible with a large subset of the OpenStack Swift API.

Note:

- For more information about Ceph architecture, see Architecture.
- For the minimum hardware recommendations, see Hardware.

Understanding Red Hat Ceph Storage terminology

Understand terms and definitions for the Red Hat Ceph Storage storage system.

BlueStore

BlueStore stores objects directly on raw block devices or partitions, and does not interact with mounted file systems. BlueStore uses RocksDB's key/value database to map object names to block locations on disk.

Bucket

In the context of Ceph Object Gateway, a bucket is a group of objects. In a filesystem-based analogy in which objects are the counterpart of files, buckets are the counterpart of directories. Multi-site sync policies can be set on buckets to provide fine-grained control of data movement from one zone to another zone. The concept of the bucket has been taken from AWS S3. See also "Ceph Object Gateway" on page 27.

Ceph Block Device

Also called *RADOS Block Device* and *RBD*. A software instrument that orchestrates the storage of block-based data in Red Hat Ceph Storage. Ceph Block Device splits block-based application data into "chunks". RADOS stores these chunks as objects. Ceph Block Device orchestrates the storage of those objects across the storage cluster.

Ceph Block Storage

One of the three kinds of storage supported by Red Hat Ceph Storage (the other two are object storage and file storage). Ceph Block Storage is the block storage product, which refers to block-storage related services and capabilities when used in conjunction with the collection of the following:

- `librbd`, a python module that provides file-like access to RBD images.
- A hypervisor such as QEMU or Xen.
- A hypervisor abstraction layer such as `libvirt`.

Ceph client

Any of the Red Hat Ceph Storage components that can access a Ceph Storage Cluster. This includes the Ceph Object Gateway, the Ceph Block Device, the Ceph File System, and their corresponding libraries. It also includes kernel modules, and FUSEs (Filesystems in USERspace).

Ceph client libraries

The collection of libraries that can be used to interact with components of the Red Hat Ceph Storage Cluster.

Ceph cluster map

See cluster map.

Ceph File System

The Ceph File System, or *CephFS*, is a POSIX-compliant file system built on top of Ceph's distributed object store, RADOS. See also "RADOS" on page 30.

CephFS

See "Ceph File System" on page 27.

Ceph kernel modules

The collection of kernel modules that can be used to interact with the Red Hat Ceph Storage cluster, such as `ceph.ko` and `rbd.ko`.

Ceph Manager

The Ceph Manager daemon. Also referred to as *ceph-mgr*. The Ceph Manager is a daemon that runs alongside monitor daemons to provide monitoring and interfacing to external monitoring and management systems.

Ceph Metadata Server

The Ceph MetaData Server daemon. Also referred to as *ceph-mds* and *MDS*. The MDS daemon must be running in any Red Hat Ceph Storage cluster that runs the CephFS. The MDS stores all filesystem metadata.

Ceph Monitor

A daemon that maintains a map of the state of the cluster. This cluster state includes the monitor map, the manager map, the OSD map, and the CRUSH map. a Red Hat Ceph Storage cluster must contain a minimum of three running monitors to be both redundant and highly-available. Ceph monitors and the nodes on which they run are often referred to as *mons*.

Ceph node

A Ceph node is a unit of the Ceph Cluster that communicates with other nodes in the Ceph Cluster in order to replicate and redistribute data. All of the nodes together are called the Ceph Storage Cluster. Ceph nodes include OSDs, Ceph Monitors, Ceph Managers, and MDSes. The term *node* is usually equivalent to *host* in the Red Hat Ceph Storage documentation. If you have a running Ceph Cluster, you can list all of the nodes in it by running the **`ceph node ls all`** command.

Ceph Object Gateway

Also called *RADOS Gateway* and *RGW*. An object storage interface built on top of `librados`. Ceph Object Gateway provides a RESTful gateway between applications and Ceph storage clusters. See also "RADOS Gateway" on page 30.

Ceph Object Storage

A Ceph Object Store consists of a Ceph Storage Cluster and a Ceph Object Gateway (RGW).

Ceph Object Store

See "Ceph Object Storage" on page 27.

Ceph OSD

Ceph Object Storage Daemon (OSD). The Ceph OSD software, which interacts with logical disks.

Ceph OSD Daemon

See "Ceph OSD" on page 27.

Ceph stack

A collection of two or more components of Red Hat Ceph Storage.

Ceph Storage Cluster

The collection of Ceph Monitors, Ceph Managers, Ceph Metadata Servers, and OSDs that work together to store and replicate data for use by applications, Ceph Users, and Ceph Clients. Ceph Storage Clusters receive data from Red Hat Ceph Storage Clients.

CephX

The Ceph authentication protocol. CephX authenticates users and daemons. CephX operates like Kerberos, but it has no single point of failure.

Clone

A full data copy from an existing volume or disk drive to a new volume or disk drive. Clearly distinguish from a virtual machine clone, which copies both storage and configuration.

Cloud stacks

Third party cloud provisioning platforms such as OpenStack, CloudStack, OpenNebula, and Proxmox VE.

Cloud storage cluster

The collection of Ceph Monitors, Ceph Managers, Ceph Metadata Servers, and OSDs that work together to store and replicate data for use by applications, Ceph Users, and Ceph Clients. Ceph Storage Clusters receive data from Ceph Clients.

Cluster map

The cluster map represents all components that are being used in a Ceph cluster that together report the state of the Ceph cluster. The components include the following: monitor map, OSD map, placement group map, MDS map, and CRUSH map.

Container storage interface

Container Storage Interface (CSI) is a software interface for Kubernetes and other container orchestrators. The CSI allows storage providers to create plug-ins so that their applications can be exposed to containerized applications as persistent storage without a need for users to understand all of the specifics and complexity of the underlying storage infrastructure. Using the CSI, users can address requests against the CSI API in a common way.

CRUSH

Controlled Replication Under Scalable Hashing (*CRUSH*). The algorithm that Red Hat Ceph Storage uses to compute object storage locations.

CRUSH rule

The CRUSH data placement rule that applies to a particular pool or pools. See also "CRUSH" on page 28.

DAS

Direct-Attached Storage (*DAS*). Storage that is attached directly to the computer accessing it, without passing through a network. Contrast with NAS and SAN.

Distributed file system

A file system that presents files from a number of different storage devices, potentially on many different machines and in many different locations, as a single interface to an end user or consuming service.

Distributed volume

A storage volume that distributes data across multiple storage nodes and resources.

Ephemeral storage

A temporary storage location that allows pods to run internal resources required for their functioning. Ephemeral storage only exists when a pod exists. It is non-persistent storage and non-sharable with other pods processes.

Epoch

In the context of Ceph Object Gateway, an epoch is a group of time periods.

File System in User Space

File System In User Space (FUSE). A software interface for Linux and Linux-like systems that lets non-privileged users create and configure their own file systems without touching kernel code.

FQDN

See "Fully Qualified Domain Name" on page 28.

Fully Qualified Domain Name

Fully Qualified Domain Name (FQDN). A domain name that is applied to a node in a network and that specifies the node's exact location in the tree hierarchy of the DNS.
In the context of Ceph cluster administration, FQDNs are often applied to hosts. In this documentation, the term *FQDN* is used mostly to distinguish between FQDNs and relatively simpler hostnames, which do not specify the exact location of the host in the tree hierarchy of the DNS but merely name the host.

FUSE

See "File System in User Space" on page 28.

Georeplication

A way to distribute data over long distances and over latency networks with the goal to provide a copy of the data for disaster recovery purposes.

Hybrid OSD

Refers to an OSD that has both HDD and SSD drives.

Logical volume

A logical volume (LV) is a virtual block storage device that a file system, database, or application can use. To create a Logical Volume Manager (LVM) logical volume, the physical volumes are combined into a volume group. This creates a pool of disk space out of which LVM logical volumes can be allocated.

LVM cache

A caching mechanism used to improve the performance of a logical volume (LV). Typically, a smaller and faster device is used to improve I/O performance of a larger and slower LV. See also “Logical volume” on page 29.

LVM tags

Logical Volume Manager tags. Extensible metadata for LVM volumes and groups. They are used to store Ceph-specific information about devices and its relationship with OSDs.

MDS

See “Ceph Metadata Server” on page 27.

MGR

See “Ceph Manager” on page 27.

MON

See “Ceph Monitor” on page 27.

Object bucket

A logical container for objects to be stored in.

Object bucket claim

A request by a user or application for a S3-compatible bucket that is used for workload processing.

Object storage

A repository of data objects for unstructured data.

OSD

See “Ceph OSD” on page 27.

OSD FSID

This is a unique identifier used to identify an OSD. It is found in the OSD path in a file called `osd_fsid`. The term *FSID* is used interchangeably with *UUID*. See also “OSD UUID” on page 29.

OSD ID

The integer that defines an OSD. It is generated by the monitors during the creation of each OSD.

OSD UUID

This is the unique identifier of an OSD. The term *UUID* is used interchangeably with *FSID*. See also “OSD FSID” on page 29.

Period

In the context of Ceph Object Gateway, a period is the configuration state of the *Realm*. The period stores the configuration state of a multi-site configuration. When the period is updated, the “epoch” is said thereby to have been changed. See also “RADOS Gateway” on page 30 and “Realm” on page 30.

Persistent volume

A storage volume for data in an OpenShift cluster that provides persistent storage, rather than the ephemeral storage provided by default. A request for this kind of storage is a Persistent Volume Claim (PVC).
See also “Persistent volume claim” on page 29.

Persistent volume claim

Persistent volume claim (PVC). A request by a user or application for storage that is associated to a persistent volume or disk drive.

RADOS

Reliable Autonomic Distributed Object Store (*RADOS*). RADOS is the object store that provides a scalable service for variably-sized objects. The RADOS object store is the core component of a Ceph cluster. It is the core set of storage software which stores the user data (MON and OSD). See also “Ceph Monitor” on page 27 and “Ceph OSD” on page 27.

RADOS cluster

A proper subset of the Ceph Cluster consisting of OSDs, Ceph Monitors, and Ceph Managers.

RADOS Gateway

Also called *Ceph Object Gateway* or *RGW*. The component of Ceph that provides a gateway to both the Amazon S3 RESTful API and the OpenStack Swift API. See “Ceph Object Gateway” on page 27.

RBD

RADOS Block Device. See “Ceph Block Device” on page 26.

Realm

In the context of Ceph Object Gateway, a realm is a globally unique namespace that consists of one or more zonegroups.

RGW

See “RADOS Gateway” on page 30.

Scale out

Adding servers to achieve better performance, availability, and in the case of storage, more capacity.

Scale up

Increasing capacity or performance by adding more storage devices or CPUs into a server.

Scrubs

The processes by which Red Hat Ceph Storage ensures data integrity. During the process of scrubbing, Red Hat Ceph Storage generates a catalog of all objects in a placement group, then ensures that none of the objects are missing or mismatched by comparing each primary object against its replicas, which are stored across other OSDs. Any placement group (PG) is determined to have a copy of an object that is different than the other copies or is missing entirely is marked `inconsistent` (that is, the PG is marked “inconsistent”).
There are two kinds of scrubbing: light scrubbing and deep scrubbing (also called “normal scrubbing” and “deep scrubbing”, respectively). Light scrubbing is performed daily and does nothing more than confirm that a given object exists and that its metadata is correct. Deep scrubbing is performed weekly and reads the data and uses checksums to ensure data integrity.

Scrubbing

See “Scrubs” on page 30.

Secrets

Secrets are credentials used to perform digital authentication whenever privileged users must access systems that require authentication. Secrets can be passwords, API keys, tokens, SSH keys, private certificates, or encryption keys.

Shard

A shard is a small part of a larger container. Shards replicate independently in parallel with other shards. Operations on shards take less time than operations on the whole container, which makes replication and maintenance more reliable.

Storage class

A storage class defines the type of storage and eventual storage specifics. Storage classes provide a way for administrators to describe the classes of storage they offer. Classes can map to storage type, quality of service levels, backup policies, storage profiles, storage types, and storage features.
This is different than a Kubernetes StorageClass object.

Teuthology

The collection of software that performs scripted tests on Ceph.

Zone

In the context of Ceph Object Gateway, a zone is a logical group that consists of one or more Ceph Object Gateway instances. A zone’s configuration state is stored in the period. See also “Ceph Object Gateway” on page 27.

Architecture

Get to know the architecture information for Ceph Storage Clusters and their clients.

Ceph architecture

Red Hat Ceph Storage cluster is a distributed data object store designed to provide excellent performance, reliability and scalability. The power of Red Hat Ceph Storage cluster can transform your organization's IT infrastructure and your ability to manage vast amounts of data, especially for cloud computing platforms like Red Hat Enterprise Linux OSP. The cluster delivers extraordinary scalability–thousands of clients accessing petabytes to exabytes of data and beyond.

Distributed object stores are the future of storage, because they accommodate unstructured data, and because clients can use modern object interfaces and legacy interfaces simultaneously.

For example:

- APIs in many languages (C/C++, Java, Python)
- RESTful interfaces (S3/Swift)
- Block device interface
- Filesystem interface

At the heart of every Ceph deployment is the Red Hat Ceph Storage cluster. It consists of three types of daemons:

Ceph OSD Daemon

Ceph OSDs store data on behalf of Ceph clients. Additionally, Ceph OSDs utilize the CPU, memory and networking of Ceph nodes to perform data replication, erasure coding, rebalancing, recovery, monitoring and reporting functions.

Ceph Monitor

A Ceph Monitor maintains a master copy of the Red Hat Ceph Storage cluster map with the current state of the cluster. Monitors require high consistency, and use Paxos to ensure agreement about the state of the cluster.

Ceph Manager

The Ceph Manager maintains detailed information about placement groups, process metadata and host metadata in lieu of the Ceph Monitor—significantly improving performance at scale. The Ceph Manager handles execution of many of the read-only Ceph CLI queries, such as placement group statistics. The Ceph Manager also provides the RESTful monitoring APIs.

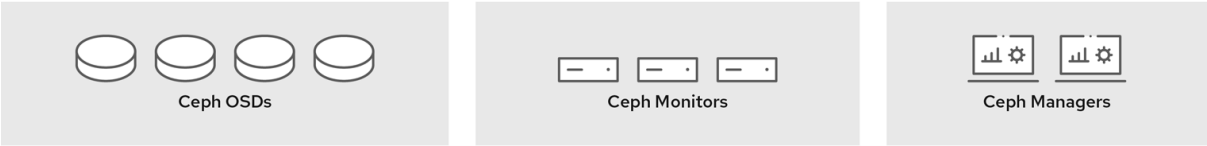


Figure 3: Daemons

Ceph client interfaces read data from and write data to the Red Hat Ceph Storage cluster. Clients need the following data to communicate with the Red Hat Ceph Storage cluster:

- The Ceph configuration file, or the cluster name (usually `ceph`) and the monitor address.
- The pool name.
- The user name and the path to the secret key.

Ceph clients maintain object IDs and the pool names where they store the objects. However, they do not need to maintain an object-to-OSD index or communicate with a centralized object index to look up object locations. Then, Ceph clients provide an object name andI pool name to `librados`, which computes an object's placement group and the primary OSD for storing and retrieving data using the CRUSH (Controlled Replication Under Scalable Hashing) algorithm. The Ceph client connects to the primary OSD where it may perform read and write operations. There is no intermediary server, broker or bus between the client and the OSD.

When an OSD stores data, it receives data from a Ceph client—whether the client is a Ceph Block Device, a Ceph Object Gateway, a Ceph Filesystem or another interface and it stores the data as an object.

> **Note:** An object ID is unique across the entire cluster, not just an OSD's storage media.

Ceph OSDs store all data as objects in a flat namespace. There are no hierarchies of directories. An object has a cluster-wide unique identifier, binary data, and metadata consisting of a set of name/value pairs.

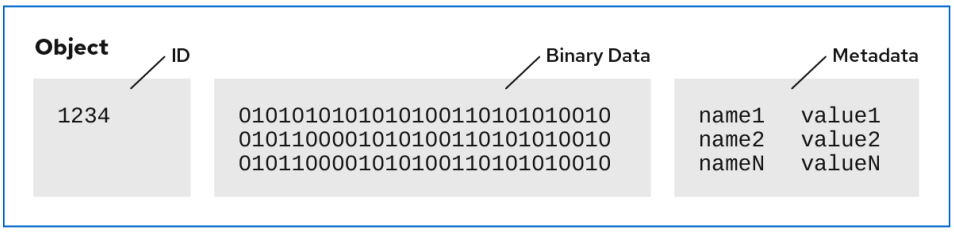


Figure 4: Object

Ceph clients define the semantics for the client's data format. For example, the Ceph block device maps a block device image to a series of objects stored across the cluster.

> **Note:** Objects consisting of a unique ID, data, and name/value paired metadata can represent both structured and unstructured data, as well as legacy and leading edge data storage interfaces.

Ceph client components

Ceph clients differ materially in how they present data storage interfaces. A Ceph Block Device presents block storage that mounts just like a physical storage drive. A Ceph gateway presents an object storage service with S3-compliant and Swift-compliant RESTful interfaces with its own user management. However, all Ceph clients use the Reliable Autonomic Distributed Object Store (RADOS) protocol to interact with the Red Hat Ceph Storage cluster.

Ceph clients all have the same basic needs:

- The Ceph configuration file, and the Ceph monitor address.
- The pool name.
- The user name and the path to the secret key.

Ceph clients tend to follow some similar patterns, such as object-watch-notify and striping. The following sections describe a little bit more about RADOS, librados and common patterns used in Ceph clients.

Prerequisites

Be sure to have a basic understanding of distributed storage systems before learning about the Ceph client components.

Ceph client native protocol

Modern applications need a simple object storage interface with asynchronous communication capability. The Red Hat Ceph Storage Cluster provides a simple object storage interface with asynchronous communication capability.

The interface provides direct, parallel access to objects throughout the cluster.

- Pool Operations
- Snapshots
- Read/Write Objects

    - Create or Remove
    - Entire Object or Byte Range
    - Append or Truncate
- Create/Set/Get/Remove XATTRs
- Create/Set/Get/Remove Key/Value Pairs
- Compound operations and dual-ack semantics

Ceph client object watch and notify

A Ceph client can register a persistent interest with an object and keep a session to the primary OSD open. The client can send a notification message and payload to all watchers and receive notification when the watchers receive the notification. This enables a client to use any object as a synchronization/communication channel.

"Figure: Ceph client object watch and notify" on page 33 illustrates the Ceph client object watch and notify workflow.

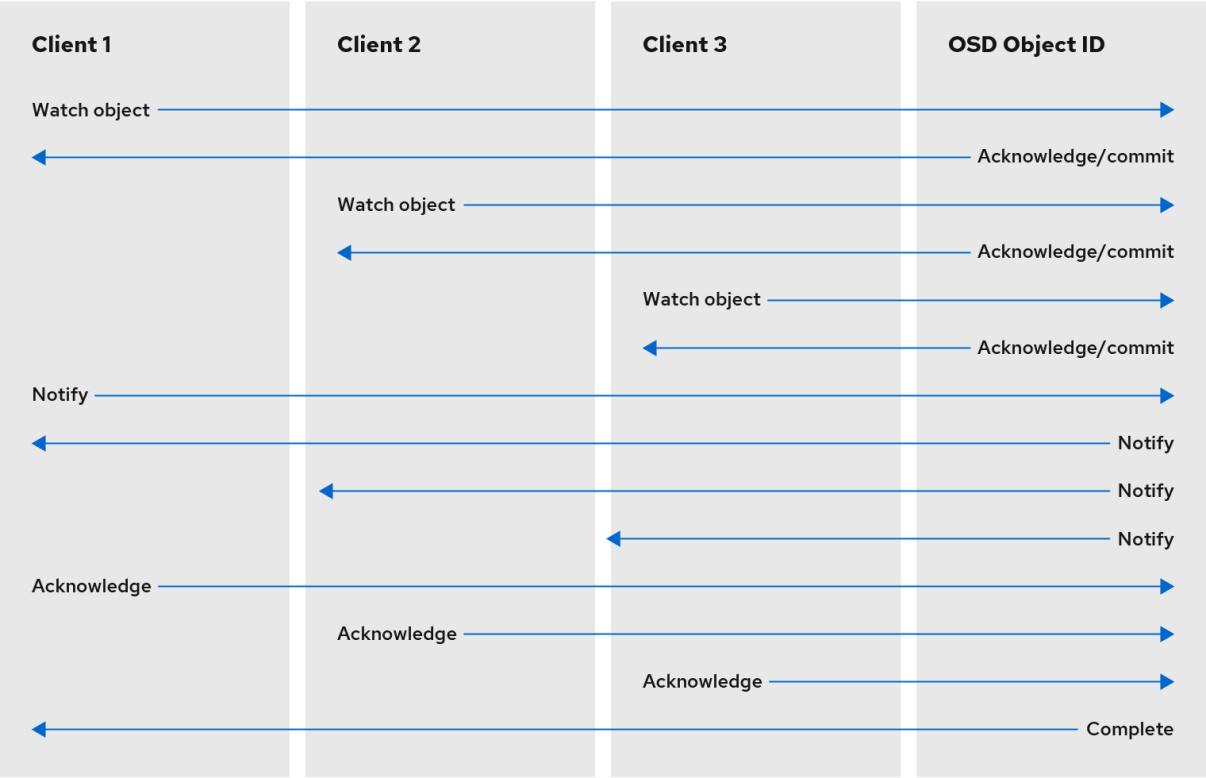


Figure 5: Ceph client object watch and notify

Ceph client `Mandatory Exclusive Locks`

Mandatory Exclusive Locks is a feature that locks an RBD to a single client, if multiple mounts are in place. Enabling this feature means that only one client can modify an RBD device at a time and provides protection for failed clients.

Mandatory Exclusive Locks helps address the write conflict situation when multiple mounted clients try to write to the same object. This feature is built on `object-watch-notify` explained in the previous section. So, when writing, if one client first establishes an exclusive lock on an object, another mounted client will first check to see if a peer has placed a lock on the object before writing.

With this feature enabled, only one client can modify an RBD device at a time, especially when changing internal RBD structures during operations like `snapshot create/delete`. It also provides some protection for failed clients. For instance, if a virtual machine seems to be unresponsive and you start a copy of it with the same disk elsewhere, the first one will be blacklisted in Ceph and unable to corrupt the new one.

Mandatory Exclusive Locks are not enabled by default. You have to explicitly enable it with `--image-feature` parameter when creating an image.

Example

```
[root@mon ~]# rbd create --size 102400 mypool/myimage --image-feature 13
```

In this example, the numeral 13 is a summation of 1, 4, and 8, where 1 enables layering support, 4 enables exclusive locking support, and 8 enables object map support. So, the command creates a 100 GB RBD image, enable layering, exclusive lock, and object map.

Mandatory Exclusive Locks is also a prerequisite for `object map`. Without enabling exclusive locking support, object map support cannot be enabled.

Mandatory Exclusive Locks also does some ground work for mirroring.

Ceph client object map

Object map is a feature that tracks the presence of backing RADOS objects when a client writes to an RBD image. When a write occurs, that write is converted to an offset within a backing RADOS object. When the object map feature is enabled, the presence of these RADOS objects is tracked, allowing Ceph to know if the objects actually exist.

Object map is kept in-memory on the librbd client so it can avoid querying the OSDs for objects that it knows don't exist. In other words, object map is an index of the objects that actually exist.

Object map is beneficial for certain operations, viz:

- Resize
- Export
- Copy
- Flatten
- Delete
- Read

A shrink **resize** operation is like a partial delete where the trailing objects are deleted.

An **export** operation knows which objects are to be requested from RADOS.

A **copy** operation knows which objects exist and need to be copied. It does not have to iterate over potentially hundreds and thousands of possible objects.

A **flatten** operation performs a copy-up for all parent objects to the clone so that the clone can be detached from the parent i.e, the reference from the child clone to the parent snapshot can be removed. So, instead of all potential objects, copy-up is done only for the objects that exist.

A **delete** operation deletes only the objects that exist in the image.

A **read** operation skips the read for objects it knows doesn't exist.

So, for operations like resize, shrinking only, exporting, copying, flattening, and deleting, these operations would need to issue an operation for all potentially affected RADOS objects, whether they exist or not. With object map enabled, if the object doesn't exist, the operation need not be issued.

For example, if we have a 1 TB sparse RBD image, it can have hundreds and thousands of backing RADOS objects. A delete operation without object map enabled would need to issue a `remove object` operation for each potential object in the image. But if object map is enabled, it only needs to issue `remove object` operations for the objects that exist.

Object map is valuable against clones that don't have actual objects but get objects from parents. When there is a cloned image, the clone initially has no objects and all reads are redirected to the parent. So, object map can improve reads as without the object map, first it needs to issue a read operation to the OSD for the clone, when that fails, it issues another read to the parent — with object map enabled. It skips the read for objects it knows doesn't exist.

Object map is not enabled by default. You have to explicitly enable it with `--image-features` parameter when creating an image. Also, `Mandatory Exclusive Locks` is a prerequisite for `object map`. Without enabling

exclusive locking support, object map support cannot be enabled. To enable object map support when creating a image, run:

Example

```
[root@mon ~]# rbd create --size 102400 mypool/myimage --image-feature 13
```

In this example, the numeral 13 is a summation of 1, 4, and 8, where 1 enables layering support, 4 enables exclusive locking support, and 8 enables object map support. So, the command creates a 100 GB RBD image, enable layering, exclusive lock and object map.

Ceph client data striping

Storage devices have throughput limitations, which impact performance and scalability. As a result, storage systems often support striping to increase throughput and performance. Striping is the storing sequential pieces of information across multiple storage devices. The most common form of data striping comes from RAID. The RAID type most similar to Ceph's striping is RAID 0, or a *striped volume*. Ceph's striping offers the throughput of RAID 0 striping, the reliability of n-way RAID mirroring and faster recovery.

Ceph provides three types of clients: Ceph Block Device, Ceph Filesystem, and Ceph Object Storage. A Ceph Client converts its data from the representation format it provides to its users, such as a block device image, RESTful objects, CephFS filesystem directories, into objects for storage in the Ceph Storage Cluster.

> **Tip:** The objects Ceph stores in the Ceph Storage Cluster are not striped. Ceph Object Storage, Ceph Block Device, and the Ceph Filesystem stripe their data over multiple Ceph Storage Cluster objects. Ceph Clients that write directly to the Ceph storage cluster using `librados` must perform the striping, and parallel I/O for themselves to obtain these benefits.

The simplest Ceph striping format involves a stripe count of 1 object. Ceph Clients write stripe units to a Ceph Storage Cluster object until the object is at its maximum capacity, and then create another object for additional stripes of data. The simplest form of striping may be sufficient for small block device images, S3 or Swift objects. However, this simple form doesn't take maximum advantage of Ceph's ability to distribute data across placement groups, and consequently does not improve performance very much. The following diagram depicts the simplest form of striping:

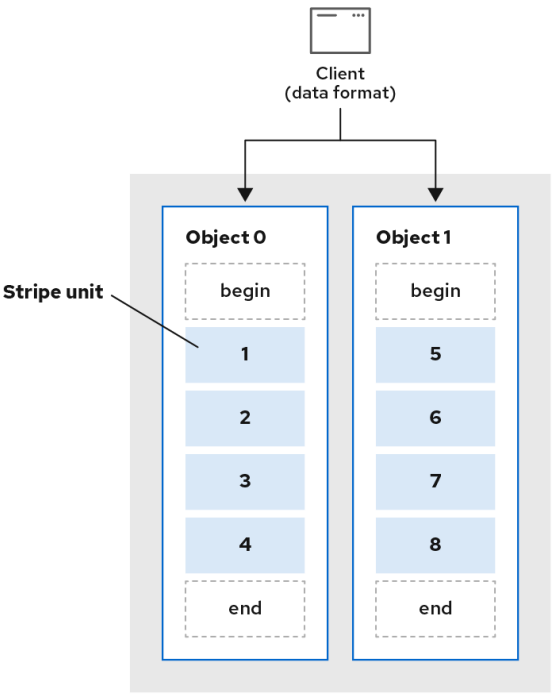


Figure 6: Data Striping

If you anticipate large images sizes, large S3 or Swift objects for example, video, you may see considerable read/write performance improvements by striping client data over multiple objects within an object set. Significant write performance occurs when the client writes the stripe units to their corresponding objects in parallel. Since objects get mapped to different placement groups and further mapped to different OSDs, each write occurs in parallel at the maximum write speed. A write to a single disk would be limited by the head movement for example, 6ms per seek and bandwidth of that one device for example, 100MB/s. By spreading that write over multiple objects, which map to different placement groups and OSDs, Ceph can reduce the number of seeks per drive and combine the throughput of multiple drives to achieve much faster write or read speeds.

> **Note:** Striping is independent of object replicas. Since CRUSH replicates objects across OSDs, stripes get replicated automatically.

In the following diagram, client data gets striped across an object set (`object set 1` in the following diagram) consisting of 4 objects, where the first stripe unit is `stripe unit 0` in `object 0`, and the fourth stripe unit is `stripe unit 3` in `object 3`. After writing the fourth stripe, the client determines if the object set is full. If the object set is not full, the client begins writing a stripe to the first object again, see `object 0` in the following diagram. If the object set is full, the client creates a new object set, see `object set 2` in the following diagram, and begins writing to the first stripe, with a stripe unit of 16, in the first object in the new object set, see `object 4` in the diagram.

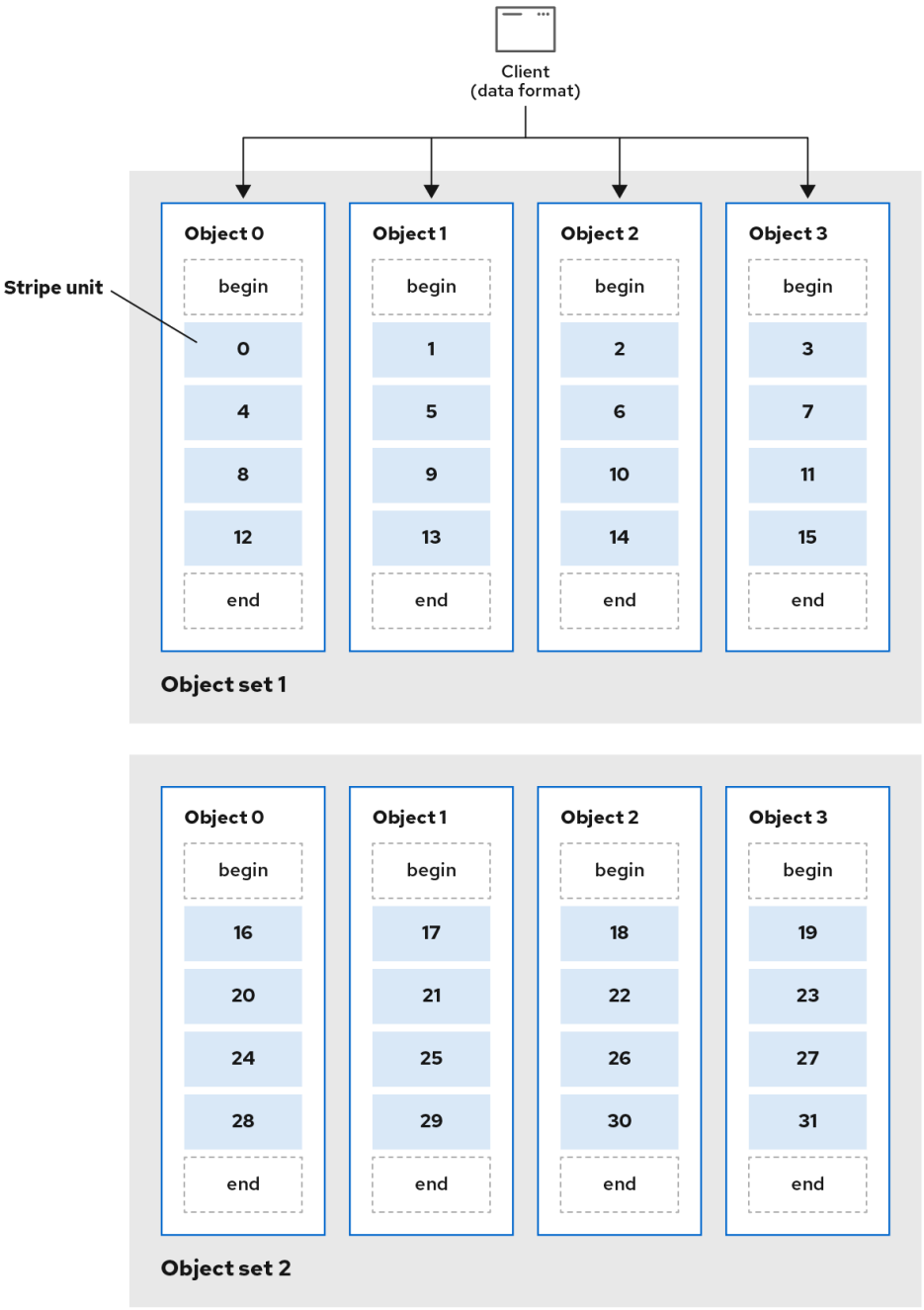


Figure 7: Client Data Stripping

Three important variables determine how Ceph stripes data:

Object Size

Objects in the Ceph Storage Cluster have a maximum configurable size, such as 2 MB, or 4 MB. The object size should be large enough to accommodate many stripe units, and should be a multiple of the stripe unit.

> **Important:** Red Hat recommends a safe maximum value of 16 MB.

Stripe Width

Stripes have a configurable unit size, for example 64 KB. The Ceph Client divides the data it will write to objects into equally sized stripe units, except for the last stripe unit. A stripe width should be a fraction of the Object Size so that an object may contain many stripe units.

Stripe Count

The Ceph Client writes a sequence of stripe units over a series of objects determined by the stripe count. The series of objects is called an object set. After the Ceph Client writes to the last object in the object set, it returns to the first object in the object set.

Important: Test the performance of your striping configuration before putting your cluster into production. You **CANNOT** change these striping parameters after you stripe the data and write it to objects.

Once the Ceph Client has striped data to stripe units and mapped the stripe units to objects, Ceph's CRUSH algorithm maps the objects to placement groups, and the placement groups to Ceph OSD Daemons before the objects are stored as files on a storage disk.

Note: Since a client writes to a single pool, all data striped into objects get mapped to placement groups in the same pool. So they use the same CRUSH map and the same access controls.

Ceph on-wire encryption

You can enable encryption for all Ceph traffic over the network with the introduction of the messenger version 2 protocol. The `secure` mode setting for messenger v2 encrypts communication between Ceph daemons and Ceph clients, giving you end-to-end encryption.

The second version of Ceph's on-wire protocol, `msgr2`, includes several new features:

- A secure mode encrypting all data moving through the network.
- Encapsulation improvement of authentication payloads.
- Improvements to feature advertisement and negotiation.

The Ceph daemons bind to multiple ports allowing both the legacy, v1-compatible, and the new, v2-compatible, Ceph clients to connect to the same storage cluster. Ceph clients or other Ceph daemons connecting to the Ceph Monitor daemon will try to use the `v2` protocol first, if possible, but if not, then the legacy `v1` protocol will be used. By default, both messenger protocols, `v1` and `v2`, are enabled. The new v2 port is 3300, and the legacy v1 port is 6789, by default.

The messenger v2 protocol has two configuration options that control whether the v1 or the v2 protocol is used:

`ms_bind_msgr1`

This option controls whether a daemon binds to a port speaking the v1 protocol; it is `true` by default.

`ms_bind_msgr2`

This option controls whether a daemon binds to a port speaking the v2 protocol; it is `true` by default.

Similarly, two options control based on IPv4 and IPv6 addresses used:

`ms_bind_ipv4`

This option controls whether a daemon binds to an IPv4 address; it is `true` by default.

`ms_bind_ipv6`

This option controls whether a daemon binds to an IPv6 address; it is `true` by default.

The `msgr2` protocol supports two connection modes:

- `crc`
  - Provides strong initial authentication when a connection is established with `cephx`.
  - Provides a `crc32c` integrity check to protect against bit flips.
  - Does not provide protection against a malicious man-in-the-middle attack.
  - Does not prevent an eavesdropper from seeing all post-authentication traffic.

- `secure`
  - Provides strong initial authentication when a connection is established with `cephx`.
  - Provides full encryption of all post-authentication traffic.
  - Provides a cryptographic integrity check.

The default mode is `crc`.

Ensure that you consider cluster CPU requirements when you plan the Red Hat Ceph Storage deployment to include encryption overhead.

> **Important:** Using `secure` mode is supported by Ceph clients using `librbd`, such as OpenStack Nova, Glance, and Cinder.

Address Changes

For both versions of the messenger protocol to co-exist in the same storage cluster, the address formatting has changed:

Syntax for old address format

```
IP_ADDR:PORT/CLIENT_ID
```

For example, `1.2.3.4:5678/91011`

Syntax for new address format

```
PROTOCOL_VERSION:IP_ADDR:PORT/CLIENT_ID
```

For example, `v2:1.2.3.4:5678/91011`, where *PROTOCOL_VERSION* can be either `v1` or `v2`.

Because the Ceph daemons now bind to multiple ports, the daemons display multiple addresses instead of a single address. Here is an example from a dump of the monitor map:

```
epoch 1
fsid 50fcf227-be32-4bcb-8b41-34ca8370bd17
last_changed 2021-12-12 11:10:46.700821
created 2021-12-12 11:10:46.700821
min_mon_release 14 (nautilus)
0: [v2:10.0.0.10:3300/0,v1:10.0.0.10:6789/0] mon.a
1: [v2:10.0.0.11:3300/0,v1:10.0.0.11:6789/0] mon.b
2: [v2:10.0.0.12:3300/0,v1:10.0.0.12:6789/0] mon.c
```

Also, the `mon_host` configuration option and specifying addresses on the command line using `-m` supports the new address format.

Connection Phases

There are four phases for making an encrypted connection:

Banner

On connection, both the client and the server send a banner. Currently, the Ceph banner is `ceph 0 0n`.

Authentication Exchange

All data, sent or received, is contained in a frame for the duration of the connection. The server decides if authentication has completed, and what the connection mode will be. The frame format is fixed, and can be in three different forms depending on the authentication flags being used.

Message Flow Handshake Exchange

The peers identify each other and establish a session. The client sends the first message, and the server will reply with the same message. The server can close connections if the client talks to the wrong daemon. For new sessions, the client and server proceed to exchanging messages. Client cookies are used to identify a session, and can reconnect to an existing session.

Message Exchange

The client and server start exchanging messages, until the connection is closed.

Reference

See the "Data security and hardening" on page 51 for details on enabling the `msgr2` protocol.

Core Ceph components

A Red Hat Ceph Storage cluster can have a large number of Ceph nodes for limitless scalability, high availability and performance. Use this information to learn how CRUSH enables Ceph to perform seamless operations.

Each node leverages non-proprietary hardware and intelligent Ceph daemons that communicate with each other to:

- Write and read data
- Compress data
- Ensure durability by replicating or erasure coding data
- Monitor and report on cluster health--also called 'heartbeating'
- Redistribute data dynamically--also called 'backfilling'
- Ensure data integrity; and,
- Recover from failures.

To the Ceph client interface that reads and writes data, an Red Hat Ceph Storage cluster looks like a simple pool where it stores data. However, `librados` and the storage cluster perform many complex operations in a manner that is completely transparent to the client interface. Ceph clients and Ceph OSDs both use the CRUSH (Controlled Replication Under Scalable Hashing) algorithm.

Prerequisites

Be sure to have a basic understanding of distributed storage systems before learning about the Ceph client components.

Pools

The Ceph storage cluster stores data objects in logical partitions called *pools*. Ceph administrators can create pools for particular types of data, such as for Ceph Block Devices, Ceph Object Gateways, or simply just to separate one group of users from another. Pools play an important role in data durability, performance, and high availability to Red Hat Ceph Storage.

From the perspective of a Ceph client, the storage cluster is very simple. When a Ceph client reads or writes data using an I/O context, it *always* connects to a storage pool in the Ceph storage cluster. The client specifies the pool name, a user and a secret key, so the pool appears to act as a logical partition with access controls to its data objects.

A Ceph pool is not only a logical partition for storing object data. A pool plays a critical role in how the Ceph storage cluster distributes and stores data. However, these complex operations are transparent to the Ceph client.

Ceph pools define:

Pool type

In early versions of Ceph, a pool simply maintained multiple deep copies of an object. Today, Ceph can maintain multiple copies of an object, or it can use erasure coding to ensure durability. The data durability method is pool-wide, and does not change after creating the pool. The pool type defines the data durability method when creating the pool. Pool types are completely transparent to the client.

Placement groups

In an exabyte scale storage cluster, a Ceph pool might store millions of data objects or more. Ceph must handle many types of operations, including data durability via replicas or erasure code chunks, data integrity by scrubbing or CRC checks, replication, rebalancing and recovery. Consequently, managing data on a per-object basis presents a scalability and performance bottleneck. Ceph addresses this bottleneck by sharding a pool into placement groups. The CRUSH algorithm computes the placement group for storing an object and computes the Acting Set of OSDs for the placement group. CRUSH puts each object into a

placement group. Then, CRUSH stores each placement group in a set of OSDs. System administrators set the placement group count when creating or modifying a pool.

CRUSH ruleset

CRUSH can detect failure domains and performance domains. CRUSH can identify OSDs by storage media type and organize OSDs hierarchically into nodes, racks, and rows. CRUSH enables Ceph OSDs to store object copies across failure domains. For example, copies of an object may get stored in different server rooms, aisles, racks and nodes. If a large part of a cluster fails, such as a rack, the cluster can still operate in a degraded state until the cluster recovers.
Additionally, CRUSH enables clients to write data to particular types of hardware, such as SSDs, hard drives with SSD journals, or hard drives with journals on the same drive as the data. The CRUSH ruleset determines failure domains and performance domains for the pool. Administrators set the CRUSH ruleset when creating a pool.

> **Note:** An administrator CANNOT change a pool's ruleset after creating the pool.

Durability

In exabyte scale storage clusters, hardware failure is an expectation and not an exception. When using data objects to represent larger grained storage interfaces such as a block device, losing one or more data objects for that larger grained interface can compromise the integrity of the larger grained storage entity potentially rendering it useless. So data loss is intolerable. Ceph provides high data durability in two ways:

- Replica pools store multiple deep copies of an object using the CRUSH failure domain to physically separate one data object copy from another. That is, copies get distributed to separate physical hardware. This increases durability during hardware failures.
- Erasure coded pools store each object as `K+M` chunks, where `K` represents data chunks and `M` represents coding chunks. The sum represents the number of OSDs used to store the object and the `M` value represents the number of OSDs that can fail and still restore data should the `M` number of OSDs fail.

Ceph authentication

To identify users and protect against man-in-the-middle attacks, Ceph provides its `cephx` authentication system, which authenticates users and daemons.

> **Note:** The `cephx` protocol does not address data encryption for data transported over the network or data stored in OSDs.

Cephx uses shared secret keys for authentication, meaning both the client and the monitor cluster have a copy of the client's secret key. The authentication protocol enables both parties to prove to each other that they have a copy of the key without actually revealing it. This provides mutual authentication, which means the cluster is sure the user possesses the secret key, and the user is sure that the cluster has a copy of the secret key.

Cephx

The `cephx` authentication protocol operates in a manner similar to Kerberos.

A user/actor invokes a Ceph client to contact a monitor. Unlike Kerberos, each monitor can authenticate users and distribute keys, so there is no single point of failure or bottleneck when using `cephx`. The monitor returns an authentication data structure similar to a Kerberos ticket that contains a session key for use in obtaining Ceph services. This session key is itself encrypted with the user's permanent secret key, so that only the user can request services from the Ceph monitors. The client then uses the session key to request its desired services from the monitor, and the monitor provides the client with a ticket that will authenticate the client to the OSDs that actually handle data. Ceph monitors and OSDs share a secret, so the client can use the ticket provided by the monitor with any OSD or metadata server in the cluster. Like Kerberos, `cephx` tickets expire, so an attacker cannot use an expired ticket or session key obtained surreptitiously. This form of authentication will prevent attackers with access to the communications medium from either creating bogus messages under another user's identity or altering another user's legitimate messages, as long as the user's secret key is not divulged before it expires.

To use `cephx`, an administrator must set up users first. In the following diagram, the `client.admin` user invokes `ceph auth get-or-create-key` from the command line to generate a username and secret key. Ceph's `auth` subsystem generates the username and key, stores a copy with the monitor(s) and transmits the user's secret back to the `client.admin` user. This means that the client and the monitor share a secret key.

> **Note:** The `client.admin` user must provide the user ID and secret key to the user in a secure manner.

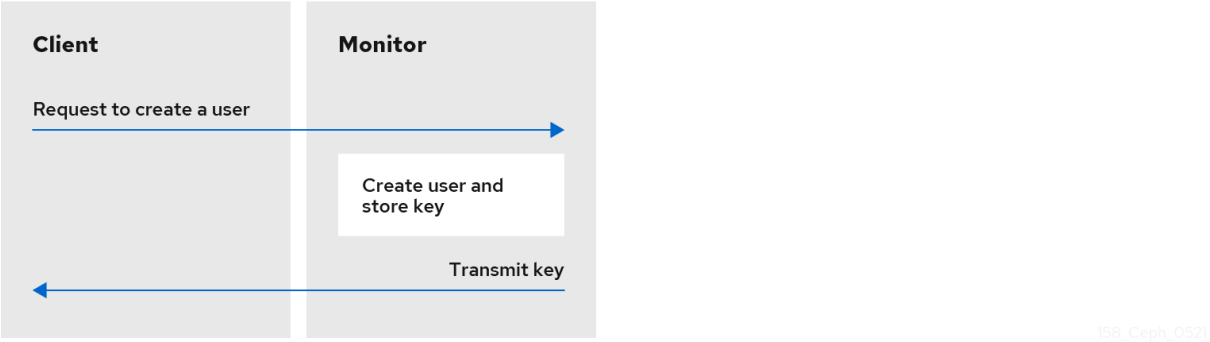


Figure 8: CephX

Placement groups

Storing millions of objects in a cluster and managing them individually is resource intensive. Ceph uses placement groups (PGs) to make managing a huge number of objects more efficient.

A PG is a subset of a pool that serves to contain a collection of objects. Ceph shards a pool into a series of PGs. Then, the CRUSH algorithm takes the cluster map and the status of the cluster into account and distributes the PGs evenly and pseudo-randomly to OSDs in the cluster.

Here is how it works:

When a system administrator creates a pool, CRUSH creates a user-defined number of PGs for the pool. Generally, the number of PGs should be a reasonably fine-grained subset of the data. For example, 100 PGs per OSD per pool would mean that each PG contains approximately 1% of the pool's data.

The number of PGs has a performance impact when Ceph needs to move a PG from one OSD to another OSD. If the pool has too few PGs, Ceph will move a large percentage of the data simultaneously and the network load will adversely impact the cluster's performance. If the pool has too many PGs, Ceph will use too much CPU and RAM when moving tiny percentages of the data and thereby adversely impact the cluster's performance. For details on calculating the number of PGs to achieve optimal performance, see PG Count

Ceph ensures against data loss by storing replicas of an object or by storing erasure code chunks of an object. Since Ceph stores objects or erasure code chunks of an object within PGs, Ceph replicates each PG in a set of OSDs called the *Acting Set* for each copy of an object or each erasure code chunk of an object. A system administrator can determine the number of PGs in a pool and the number of replicas or erasure code chunks. However, the CRUSH algorithm calculates which OSDs are in the acting set for a particular PG.

The CRUSH algorithm and PGs make Ceph dynamic. Changes in the cluster map or the cluster state may result in Ceph moving PGs from one OSD to another automatically.

Here are a few examples:

- **Expanding the Cluster:** When adding a new host and its OSDs to the cluster, the cluster map changes. Since CRUSH evenly and pseudo-randomly distributes PGs to OSDs throughout the cluster, adding a new host and its OSDs means that CRUSH will reassign some of the pool's placement groups to those new OSDs. That means that system administrators do not have to rebalance the cluster manually. Also, it means that the new OSDs contain approximately the same amount of data as the other OSDs. This also means that new OSDs do not contain newly written OSDs, preventing *hot spots* in the cluster.
- **An OSD Fails:** When an OSD fails, the state of the cluster changes. Ceph temporarily loses one of the replicas or erasure code chunks, and needs to make another copy. If the primary OSD in the acting set fails, the next OSD in the acting set becomes the primary and CRUSH calculates a new OSD to store the additional copy or erasure code chunk.

By managing millions of objects within the context of hundreds to thousands of PGs, the Ceph storage cluster can grow, shrink and recover from failure efficiently.

For Ceph clients, the CRUSH algorithm via `librados` makes the process of reading and writing objects very simple. A Ceph client simply writes an object to a pool or reads an object from a pool. The primary OSD in the acting set can write replicas of the object or erasure code chunks of the object to the secondary OSDs in the acting set on behalf of the Ceph client.

If the cluster map or cluster state changes, the CRUSH computation for which OSDs store the PG will change too. For example, a Ceph client may write object `foo` to the pool `bar`. CRUSH will assign the object to PG `1.a`, and store it on `OSD 5`, which makes replicas on `OSD 10` and `OSD 15` respectively. If `OSD 5` fails, the cluster state changes. When the Ceph client reads object `foo` from pool `bar`, the client via `librados` will automatically retrieve it from `OSD 10` as the new primary OSD dynamically.

The Ceph client via `librados` connects directly to the primary OSD within an acting set when writing and reading objects. Since I/O operations do not use a centralized broker, network oversubscription is typically NOT an issue with Ceph.

The following diagram depicts how CRUSH assigns objects to PGs, and PGs to OSDs. The CRUSH algorithm assigns the PGs to OSDs such that each OSD in the acting set is in a separate failure domain, which typically means the OSDs will always be on separate server hosts and sometimes in separate racks.

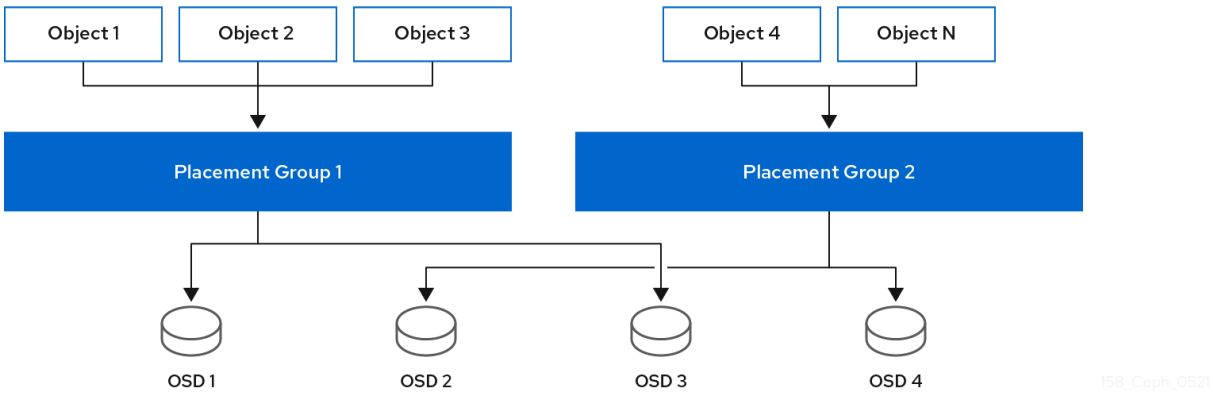


Figure 9: Placement Groups

CRUSH ruleset

Ceph assigns a CRUSH ruleset to a pool. When a Ceph client stores or retrieves data in a pool, Ceph identifies the CRUSH ruleset, a rule within the rule set, and the top-level bucket in the rule for storing and retrieving data. As Ceph processes the CRUSH rule, it identifies the primary OSD that contains the placement group for an object. That enables the client to connect directly to the OSD, access the placement group and read or write object data.

To map placement groups to OSDs, a CRUSH map defines a hierarchical list of bucket types. The list of bucket types are located under `types` in the generated CRUSH map. The purpose of creating a bucket hierarchy is to segregate the leaf nodes by their failure domains and/or performance domains, such as drive type, hosts, chassis, racks, power distribution units, pods, rows, rooms, and data centers.

With the exception of the leaf nodes representing OSDs, the rest of the hierarchy is arbitrary. Administrators may define it according to their own needs if the default types don't suit their requirements. CRUSH supports a directed acyclic graph that models the Ceph OSD nodes, typically in a hierarchy. So Ceph administrators can support multiple hierarchies with multiple root nodes in a single CRUSH map. For example, an administrator can create a hierarchy representing higher cost SSDs for high performance, and a separate hierarchy of lower cost hard drives with SSD journals for moderate performance.

Input/output operations

Ceph clients retrieve a *Cluster Map* from a Ceph monitor, bind to a pool, and perform input/output (I/O) on objects within placement groups in the pool. The pool's CRUSH ruleset and the number of placement groups are the main factors that determine how Ceph will place the data.

With the latest version of the cluster map, the client knows about all of the monitors and OSDs in the cluster and their current state. **However, the client doesn't know anything about object locations.**

The only inputs required by the client are the object ID and the pool name. It is simple: Ceph stores data in named pools. When a client wants to store a named object in a pool it takes the object name, a hash code, the number of PGs in the pool and the pool name as inputs; then, CRUSH (Controlled Replication Under Scalable Hashing) calculates the ID of the placement group and the primary OSD for the placement group.

Ceph clients use the following steps to compute PG IDs.

1. The client inputs the pool ID and the object ID. For example, `pool = liverpool` and `object-id = john`.
2. CRUSH takes the object ID and hashes it.
3. CRUSH calculates the hash modulo of the number of PGs to get a PG ID. For example, `58`.
4. CRUSH calculates the primary OSD corresponding to the PG ID.
5. The client gets the pool ID given the pool name. For example, the pool `liverpool` is pool number `4`.
6. The client prepends the pool ID to the PG ID. For example, `4.58`.
7. The client performs an object operation such as write, read, or delete by communicating directly with the Primary OSD in the Acting Set.

The topology and state of the Ceph storage cluster are relatively stable during a session. Empowering a Ceph client via `librados` to compute object locations is much faster than requiring the client to make a query to the storage cluster over a chatty session for each read/write operation. The CRUSH algorithm allows a client to compute where objects *should* be stored, and **enables the client to contact the primary OSD in the acting set directly** to store or retrieve data in the objects. Since a cluster at the exabyte scale has thousands of OSDs, network oversubscription between a client and a Ceph OSD is not a significant problem. If the cluster state changes, the client can simply request an update to the cluster map from the Ceph monitor.

Replication

Similar to Ceph clients, Ceph OSDs can contact Ceph monitors to retrieve the latest copy of the cluster map. Ceph OSDs also use the CRUSH algorithm, but they use it to compute where to store replicas of objects. In a typical write scenario, a Ceph client uses the CRUSH algorithm to compute the placement group ID and the primary OSD in the Acting Set for an object. When the client writes the object to the primary OSD, the primary OSD finds the number of replicas that it should store.

The value is found in the `osd_pool_default_size` setting. Then, the primary OSD takes the object ID, pool name and the cluster map and uses the CRUSH algorithm to calculate the IDs of secondary OSDs for the acting set. The primary OSD writes the object to the secondary OSDs. When the primary OSD receives an acknowledgment from the secondary OSDs and the primary OSD itself completes its write operation, it acknowledges a successful write operation to the Ceph client.

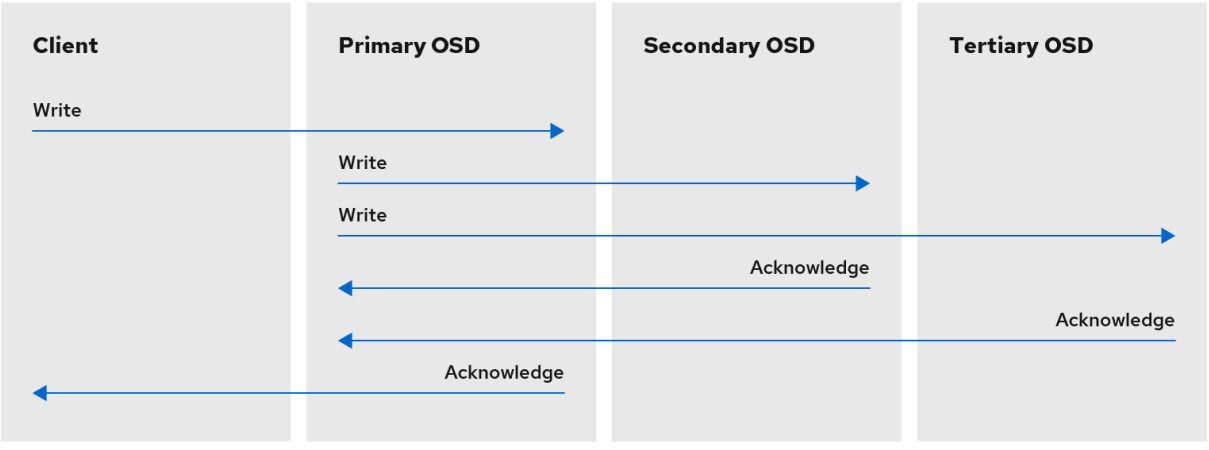


Figure 10: Replicated IO

With the ability to perform data replication on behalf of Ceph clients, Ceph OSD Daemons relieve Ceph clients from that duty, while ensuring high data availability and data safety.

Note: The primary OSD and the secondary OSDs are typically configured to be in separate failure domains. CRUSH computes the IDs of the secondary OSDs with consideration for the failure domains.

Data copies

In a replicated storage pool, Ceph needs multiple copies of an object to operate in a degraded state. Ideally, a Ceph storage cluster enables a client to read and write data even if one of the OSDs in an acting set fails. For this reason, Ceph defaults to making three copies of an object with a minimum of two copies clean for write operations. Ceph will still preserve data even if two OSDs fail. However, it will interrupt write operations.

In an erasure-coded pool, Ceph needs to store chunks of an object across multiple OSDs so that it can operate in a degraded state. Similar to replicated pools, ideally an erasure-coded pool enables a Ceph client to read and write in a degraded state.

Important: The following *jerasure* coding values for *k*, and *m* are supported:

- k=8 m=3
- k=8 m=4
- k=4 m=2

Erasure-coding

Ceph can load one of many erasure code algorithms. The earliest and most commonly used is the `Reed-Solomon` algorithm. An erasure code is a forward error correction (FEC) code. FEC code transforms a message of `K` chunks into a longer message that is called a code word of `N` chunks, such that Ceph can recover the original message from a subset of the `N` chunks.

More specifically, `N = K+M` where the variable `K` is the original number of data chunks. The variable `M` stands for the extra or redundant chunks that the erasure code algorithm adds to provide protection from failures. The variable `N` is the total number of chunks that are created after the erasure coding process. The value of `M` is `N-K` which means that the algorithm computes `N-K` redundant chunks from `K` original data chunks. This approach ensures that that Ceph can access all the original data. The system is resilient to arbitrary `N-K` failures. For instance, in a 10 `K` of 16 `N` configuration, or erasure coding `10/16`, the erasure code algorithm adds six extra chunks to the 10 base chunks `K`. For example, in a `M = K-N` or `16-10 = 6` configuration, Ceph will spread the 16 chunks `N` across 16 OSDs. The original file might be reconstructed from the 10 verified `N` chunks even if 6 OSDs fail—ensuring that the Red Hat Ceph Storage cluster will not lose data, and thereby ensures a high level of fault tolerance.

Like replicated pools, in an erasure-coded pool, the primary OSD in the up set receives all write operations. In replicated pools, Ceph makes a deep copy of each object in the placement group on the secondary OSDs in the set. For erasure coding, the process is a bit different. An erasure coded pool stores each object as `K+M` chunks. It is divided into `K` data chunks and `M` coding chunks. The pool is configured to have a size of `K+M` so that Ceph stores each chunk in an OSD in the acting set. Ceph stores the rank of the chunk as an attribute of the object. The primary OSD is responsible for encoding the payload into `K+M` chunks and sends them to the other OSDs. The primary OSD is also responsible for maintaining an authoritative version of the placement group logs.

For example, in a typical configuration, a system administrator creates an erasure-coded pool to use six OSDs and sustain the loss of two of them. That is, (`K+M = 6`) such that (`M = 2`).

When Ceph writes the object **NYAN** containing `ABCDEFGHIJKL` to the pool, the erasure encoding algorithm splits the content into four data chunks by simply dividing the content into four parts: `ABC`, `DEF`, `GHI`, and `JKL`. The algorithm will pad the content if the content length is not a multiple of `K`. The function also creates two coding chunks: the fifth with `YXY` and the sixth with `QGC`. Ceph stores each chunk on an OSD in the acting set with a shard_id corresponding to the acting set position, where it stores the chunks in objects that have the same name, **NYAN**, but reside on different OSDs. For example, Chunk 1 contains `ABC` and Ceph stores it on **OSD5** while chunk 5 contains `YXY` and Ceph stores it on **OSD4**.

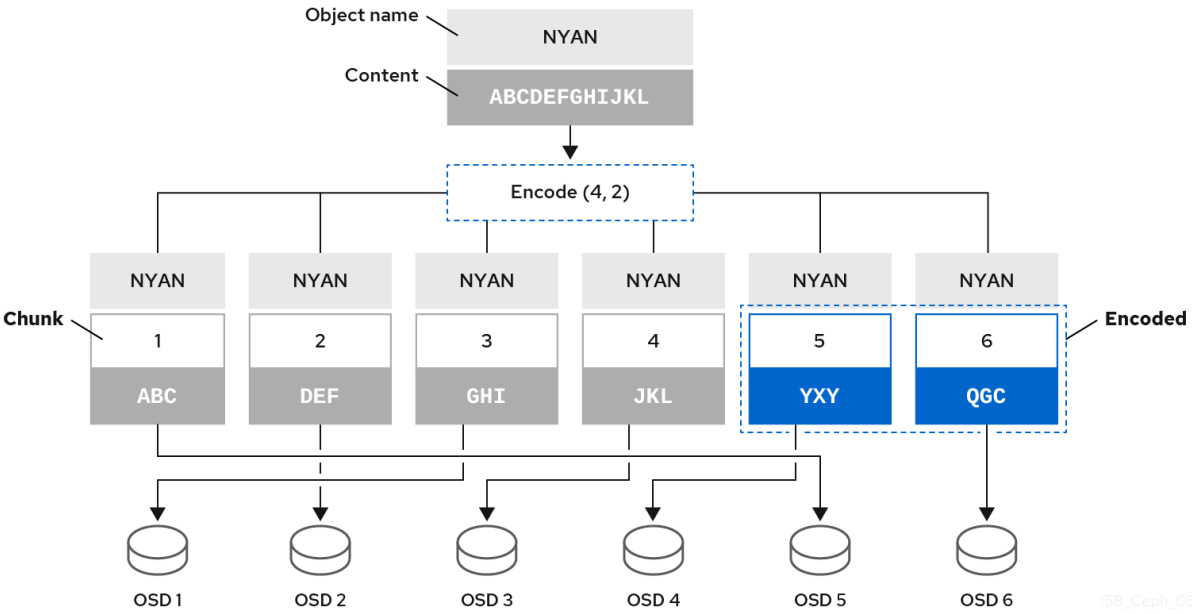


Figure 11: Ceph client object watch and notify

In a recovery scenario, the client attempts to read the object **NYAN** from the erasure-coded pool by reading chunks 1 through 6. The OSD informs the algorithm that chunks 2 and 6 are missing. For example, the primary OSD might not read chunk 6 because the **OSD6** is out, and might not read chunk 2, because **OSD2** was the slowest and its chunk was not taken into account. However, when the algorithm has four chunks, it reads the four chunks: chunk 1 containing `ABC`, chunk 3 containing `GHI`, chunk 4 containing `JKL`, and chunk 5 containing `YXY`. Then, it rebuilds the original content of the object `ABCDEFGHIJKL`, and original content of chunk 6, which contained `QGC`.

Splitting data into chunks is independent from object placement. The CRUSH ruleset along with the erasure-coded pool profile determines the placement of chunks on the OSDs. For instance, that uses the Locally Repairable Code (`lrc`) plug-in in the erasure code profile creates extra chunks and requires fewer OSDs to recover from. For example, in an `lrc` profile configuration `K=4 M=2 L=3`, the algorithm creates six chunks (`K+M`), just as the `jerasure` plug-in would, but the locality value (`L=3`) requires that the algorithm create 2 more chunks locally. The algorithm creates the additional chunks as such, `(K+M)/L`. If the OSD containing chunk 0 fails, this chunk can be recovered by using chunks 1, 2 and the first local chunk. In this case, the algorithm only requires 3 chunks for recovery instead of 5.

Note: Using erasure-coded pools disables Object Map.

Important: For an erasure-coded pool with 2+2 configuration, replace the input string from ABCDEFGHIJKL to ABCDEF and replace the coding chunks from 4 to 2.

Multi-step-retry (MSR) is a type of CRUSH rule in the Ceph cluster that defines how data is distributed across storage devices. MSR ensures efficient data retrieval, balancing, and fault tolerance. For more information, see “Creating CRUSH rules” on page 800

Reference

- For more information about CRUSH, the erasure-coding profiles, and plug-ins, see Storage strategies.
- For more details on Object Map, see Ceph client object map.

`ObjectStore`

`ObjectStore` provides a low-level interface to an OSD's raw block device. When a client reads or writes data, it interacts with the `ObjectStore` interface. Ceph write operations are essentially ACID transactions: that is, they provide *Atomicity*, *Consistency*, *Isolation*, and *Durability*.

`ObjectStore` ensures that a `Transaction` is all-or-nothing to provide Atomicity. The `ObjectStore` also handles object semantics. An object stored in the storage cluster has a unique identifier, object data and metadata. So `ObjectStore` provides `Consistency` by ensuring that Ceph object semantics are correct. `ObjectStore` also provides the **Isolation** portion of an ACID transaction by invoking a `Sequencer` on write operations to ensure that Ceph write operations occur sequentially. In contrast, an OSDs replication or erasure coding functionality provides the **Durability** component of the ACID transaction. Since `ObjectStore` is a low-level interface to storage media, it also provides performance statistics.

Ceph implements several concrete methods for storing data:

BlueStore

A production grade implementation using a raw block device to store object data.

Memstore

A developer implementation for testing read/write operations directly in RAM.

K/V Store

An internal implementation for Ceph's use of key/value databases.

Since administrators will generally only address `BlueStore`, the following sections will only describe those implementations in greater detail.

BlueStore

`BlueStore` is the next generation storage implementation for Ceph. As the market for storage devices now includes solid state drives or SSDs and non-volatile memory over PCI Express or NVMe, their use in Ceph reveals some of the limitations of the `FileStore` storage implementation. While `FileStore` has many improvements to facilitate SSD and NVMe storage, other limitations remain. Among them, increasing placement groups remains computationally expensive, and the double write penalty remains. Whereas, `FileStore` interacts with a file system on a block device, `BlueStore` eliminates that layer of indirection and directly consumes a raw block device for object storage. `BlueStore` uses the very light weight `BlueFS` file system on a small partition for its k/v databases. `BlueStore` eliminates the paradigm of a directory representing a placement group, a file representing an object and file XATTRs representing metadata. `BlueStore` also eliminates the double write penalty of `FileStore`, so write operations are nearly twice as fast with `BlueStore` under most workloads.

`BlueStore` stores data as:

Object Data

In `BlueStore`, Ceph stores objects as blocks directly on a raw block device. The portion of the raw block device that stores object data does NOT contain a filesystem. The omission of the filesystem eliminates a layer of indirection and thereby improves performance. However, much of the `BlueStore` performance improvement comes from the block database and write-ahead log.

Block Database

In `BlueStore`, the block database handles the object semantics to guarantee **Consistency**. An object's unique identifier is a key in the block database. The values in the block database consist of a series of block addresses that refer to the stored object data, the object's placement group, and object metadata. The block database may reside on a `BlueFS` partition on the same raw block device that stores the object data, or it may reside on a separate block device, usually when the primary block device is a hard disk drive and an SSD or NVMe will improve performance. The block database provides a number of improvements over `FileStore`; namely, the key/value semantics of `BlueStore` do not suffer from the limitations of filesystem XATTRs. `BlueStore` may assign objects to other placement groups quickly within the block database without the overhead of moving files from one directory to another, as is the case in `FileStore`. `BlueStore` also introduces new features. The block database can store the checksum of the stored object data and its metadata, allowing full data checksum operations for each read, which is more efficient than periodic scrubbing to detect bit rot. `BlueStore` can compress an object and the block database can store the algorithm used to compress an object—ensuring that read operations select the appropriate algorithm for decompression.

Write-ahead Log

In `BlueStore`, the write-ahead log ensures **Atomicity**, similar to the journaling functionality of `FileStore`. Like `FileStore`, `BlueStore` logs all aspects of each transaction. However, the `BlueStore` write-ahead log or WAL can perform this function simultaneously, which eliminates the double write penalty of `FileStore`. Consequently, `BlueStore` is nearly twice as fast as `FileStore` on write operations for most workloads. BlueStore can deploy the WAL on the same device for storing object data, or it may deploy the WAL on another device, usually when the primary block device is a hard disk drive and an SSD or NVMe will improve performance.

Note: It is only helpful to store a block database or a write-ahead log on a separate block device if the separate device is faster than the primary storage device. For example, SSD and NVMe devices are generally faster than HDDs. Placing the block database and the WAL on separate devices may also have performance benefits due to differences in their workloads.

Self management operations

Ceph clusters perform a lot of self monitoring and management operations automatically.

Ceph OSDs can, for example, check the cluster health and report back to the Ceph monitors. By using CRUSH to assign objects to placement groups and placement groups to a set of OSDs, Ceph OSDs can use the CRUSH algorithm to rebalance the cluster or recover from OSD failures dynamically.

Heartbeat

Ceph OSDs join a cluster and report to Ceph Monitors on their status.

At the lowest level, the Ceph OSD status is `up` or `down` reflecting whether or not it is running and able to service Ceph client requests. If a Ceph OSD is `down` and `in` the Ceph storage cluster, this status may indicate the failure of the Ceph OSD. If a Ceph OSD is not running for example, it crashes the Ceph OSD cannot notify the Ceph Monitor that it is `down`. The Ceph Monitor can ping a Ceph OSD daemon periodically to ensure that it is running. However, heartbeating also empowers Ceph OSDs to determine if a neighboring OSD is `down`, to update the cluster map and to report it to the Ceph Monitors. This means that Ceph Monitors can remain light weight processes.

Peering

Ceph stores copies of placement groups on multiple OSDs. Each copy of a placement group has a status. These OSDs *peer* check each other to ensure that they agree on the status of each copy of the placement group. Peering issues usually resolve themselves.

Note: When Ceph monitors agree on the state of the OSDs storing a placement group, that does not mean that the placement group has the latest contents.

When Ceph stores a placement group in an acting set of OSDs, refer to them as *Primary*, *Secondary*, and so forth. By convention, the *Primary* is the first OSD in the *Acting Set*. The *Primary* that stores the first copy of a placement group is responsible for coordinating the peering process for that placement group. The *Primary* is the **ONLY** OSD that will accept client-initiated writes to objects for a given placement group where it acts as the *Primary*.

An *Acting Set* is a series of OSDs that are responsible for storing a placement group. An *Acting Set* may refer to the Ceph OSD Daemons that are currently responsible for the placement group, or the Ceph OSD Daemons that were responsible for a particular placement group as of some epoch.

The Ceph OSD daemons that are part of an *Acting Set* may not always be `up`. When an OSD in the *Acting Set* is `up`, it is part of the *Up Set*. The *Up Set* is an important distinction, because Ceph can remap PGs to other Ceph OSDs when an OSD fails.

Note: In an *Acting Set* for a PG containing `osd.25`, `osd.32` and `osd.61`, the first OSD, `osd.25`, is the *Primary*. If that OSD fails, the *Secondary*, `osd.32`, becomes the *Primary*, and Ceph will remove `osd.25` from the *Up Set*.

Rebalancing and recovery

When an administrator adds a Ceph OSD to a Ceph storage cluster, Ceph updates the cluster map. This change to the cluster map also changes object placement, because the modified cluster map changes an input for the CRUSH calculations. CRUSH places data evenly, but pseudo randomly. So only a small amount of data moves when an administrator adds a new OSD. The amount of data is usually the number of new OSDs divided by the total amount of data in the cluster. For example, in a cluster with 50 OSDs, 1/50th or 2% of the data might move when adding an OSD.

"Figure: Rebalancing and recovery" on page 49 depicts the rebalancing process where some, but not all of the PGs migrate from existing OSDs, OSD 1 and 2 in the diagram, to the new OSD, OSD 3, in the diagram. Even when rebalancing, CRUSH is stable. Many of the placement groups remain in their original configuration, and each OSD gets some added capacity, so there are no load spikes on the new OSD after the cluster rebalances.

There are 2 types of balancer in Ceph: capacity balancing and read balancing.

Capacity balancing

Capacity balancing is a functional need. When one device is full, the system can not take write requests anymore. To avoid filling up devices, it is important to balance capacity across the devices in a fair way. Each device must get a capacity proportional to its size so all devices have the same fullness level. Capacity balancing creates fair share workloads on the OSDs for write requests from a performance perspective. Capacity balancing creates fair share workloads on the OSDs for write requests from a performance perspective. Capacity balancing requires data movement and is a time-consuming operation as it takes time to balance the system.

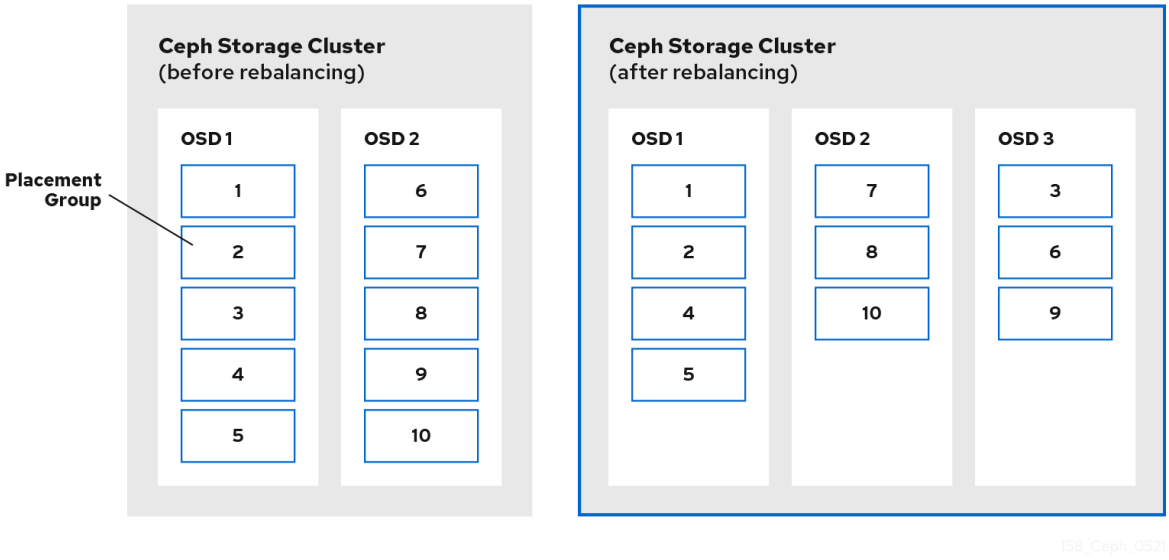


Figure 12: Rebalancing and recovery

For optimal write performance, ensure that all devices are homogeneous (same size and performance).

Note: You can run the optimizer on heterogeneous devices, but the results would not be optimal.

Read balancing (Technology Preview)

> **Important:** Technology Preview features are not supported with production service level agreements (SLAs), might not be functionally complete, and does not recommend using them for production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

Read balancing is a performance need. It helps the system perform better by ensuring that each device gets its fair share of primary OSDs so read requests get distributed across OSDs in the cluster, evenly. Unbalanced read requests lead to bad performance under load due to the weakest link in the chain effect and to reduced cluster read bandwidth. Read balancing is cheap and the operation is fast as there is no data movement involved. It is a metadata operation, where the osdmap is updated to change which participating OSD in a PG is primary.

Read balancing supports replicated pools only, erasure coded pools are not supported. Read balancing does not take into account the device class of the OSDs and the availability zones of DR solutions. An offline tool is available to use the read balancing feature and you need to run the procedure on each pool of the cluster. You need to run the read balancing procedure again after every autoscale change.

For optimal read performance, ensure that all devices are homogeneous (same size and performance) and that you have balanced the capacity.

> **Note:** You can run the optimizer on heterogeneous devices, but the results would not be optimal.

Data integrity

As part of maintaining data integrity, Ceph provides numerous mechanisms to guard against bad disk sectors and bit rot.

Scrubbing

Ceph OSD Daemons can scrub objects within placement groups. That is, Ceph OSD Daemons can compare object metadata in one placement group with its replicas in placement groups stored on other OSDs. Scrubbing usually performed daily catches bugs or storage errors. Ceph OSD Daemons also perform deeper scrubbing by comparing data in objects bit-for-bit. Deep scrubbing usually performed weekly finds bad sectors on a drive that weren't apparent in a light scrub.

CRC Checks

When using `BlueStore`, Ceph can ensure data integrity by conducting a cyclical redundancy check (CRC) on write operations; then, store the CRC value in the block database. On read operations, Ceph can retrieve the CRC value from the block database and compare it with the generated CRC of the retrieved data to ensure data integrity instantly.

High availability

In addition to the high scalability enabled by the CRUSH algorithm, Ceph must also maintain high availability.

Maintaining high availability means that Ceph clients must be able to read and write data even when the cluster is in a degraded state, or when a monitor fails.

Clustering the Ceph Monitor

For added reliability and fault tolerance, Ceph supports a cluster of monitors.

Before Ceph clients can read or write data, they must contact a Ceph Monitor to obtain the most recent copy of the cluster map. A Red Hat Ceph Storage cluster can operate with a single monitor; however, this introduces a single point of failure. That is, if the monitor goes down, Ceph clients cannot read or write data.

In a cluster of Ceph Monitors, latency and other faults can cause one or more monitors to fall behind the current state of the cluster. For this reason, Ceph must have agreement among various monitor instances regarding the state of the storage cluster. Ceph always uses a majority of monitors and the Paxos algorithm to establish a consensus among the monitors about the current state of the storage cluster. Ceph Monitors nodes require NTP to prevent clock drift.

Storage administrators usually deploy Ceph with an odd number of monitors so determining a majority is efficient. For example, a majority may be 1, 2:3, 3:5, 4:6, and so forth.

Data security and hardening

Use this information to learn about data security and hardening information for Red Hat Ceph Storage Clusters and their clients. The information here also provides advice and good practices information for hardening the security of Red Hat Ceph Storage, with a focus on the Ceph Orchestrator using `cephadm` for Red Hat Ceph Storage deployments.

> **Important:** While following these instructions helps harden the security of your environment, security and compliance is not guaranteed from following these recommendations.

Security is an important concern and should be a strong focus of any Red Hat Ceph Storage deployment. Data breaches and downtime are costly and difficult to manage, laws may require passing audits and compliance processes, and projects have an expectation of a certain level of data privacy and security. The information in this section provides a general introduction to security for Red Hat Ceph Storage, as well as the role of Red Hat in supporting your system's security.

This information provides only a general introduction to security for Red Hat Ceph Storage. Contact Red Hat Support for additional help.

Introduction to Red Hat Ceph Storage

Red Hat Ceph Storage is a highly scalable and reliable object storage solution, which is typically deployed in conjunction with cloud computing solutions like OpenStack, as a standalone storage service, or as network attached storage using interfaces.

All Red Hat Ceph Storage deployments consist of a storage cluster commonly referred to as the Ceph Storage Cluster or RADOS (Reliable Autonomous Distributed Object Store), which consists of three types of daemons:

- **Ceph Monitors (`ceph-mon`):** Ceph monitors provide a few critical functions such as establishing an agreement about the state of the cluster, maintaining a history of the state of the cluster such as whether an OSD is up and running and in the cluster, providing a list of pools through which clients write and read data, and providing authentication for clients and the Ceph Storage Cluster daemons.
- **Ceph Managers (`ceph-mgr`):** Ceph manager daemons track the status of peering between copies of placement groups distributed across Ceph OSDs, a history of the placement group states, and metrics about the Ceph cluster. They also provide interfaces for external monitoring and management systems.
- **Ceph OSDs (`ceph-osd`):** Ceph Object Storage Daemons (OSDs) store and serve client data, replicate client data to secondary Ceph OSD daemons, track and report to Ceph Monitors on their health and on the health of neighboring OSDs, dynamically recover from failures, and backfill data when the cluster size changes, among other functions.

All Red Hat Ceph Storage deployments store end-user data in the Ceph Storage Cluster or RADOS (Reliable Autonomous Distributed Object Store). Generally, users **DO NOT** interact with the Ceph Storage Cluster directly; rather, they interact with a Ceph client.

There are three primary Ceph Storage Cluster clients:

- **Ceph Object Gateway (`radosgw`):** The Ceph Object Gateway, also known as RADOS Gateway, `radosgw` or `rgw` provides an object storage service with RESTful APIs. Ceph Object Gateway stores data on behalf of its clients in the Ceph Storage Cluster or RADOS.
- **Ceph Block Device (`rbd`):** The Ceph Block Device provides copy-on-write, thin-provisioned, and cloneable virtual block devices to a Linux kernel via Kernel RBD (`krbd`) or to cloud computing solutions like OpenStack via `librbd`.
- **Ceph File System (`cephfs`):** The Ceph File System consists of one or more Metadata Servers (`mds`), which store the inode portion of a file system as objects on the Ceph Storage Cluster. Ceph file systems can be mounted via a kernel client, a FUSE client, or via the `libcephfs` library for cloud computing solutions like OpenStack.

Additional clients include `librados`, which enables developers to create custom applications to interact with the Ceph Storage cluster and command line interface clients for administrative purposes.

Supporting software

An important aspect of Red Hat Ceph Storage security is to deliver solutions that have security built-in upfront, that Red Hat supports over time.

Specific steps which Red Hat takes with Red Hat Ceph Storage include:

- Maintaining upstream relationships and community involvement to help focus on security from the start.
- Selecting and configuring packages based on their security and performance track records.
- Building binaries from associated source code (instead of simply accepting upstream builds).
- Applying a suite of inspection and quality assurance tools to prevent an extensive array of potential security issues and regressions.
- Digitally signing all released packages and distributing them through cryptographically authenticated distribution channels.
- Providing a single, unified mechanism for distributing patches and updates.

In addition, Red Hat maintains a dedicated security team that analyzes threats and vulnerabilities against our products, and provides relevant advice and updates through the Customer Portal. This team determines which issues are important, as opposed to those that are mostly theoretical problems. The Red Hat Product Security team maintains expertise in, and makes extensive contributions to the upstream communities associated with our subscription products. A key part of the process, Red Hat Security Advisories, deliver proactive notification of security flaws affecting Red Hat solutions, along with patches that are frequently distributed on the same day the vulnerability is first published.

Threat and vulnerability management

Red Hat Ceph Storage is typically deployed in conjunction with cloud computing solutions, so it can be helpful to think about a Red Hat Ceph Storage deployment abstractly as one of many series of components in a larger deployment. These deployments typically have shared security concerns, which this guide refers to as *security zones*. Threat actors and vectors are classified based on their motivation and access to resources. The intention is to provide you with a sense of the security concerns for each zone, depending on your objectives.

Threat actors

A threat actor is an abstract way to refer to a class of adversary that you might attempt to defend against. The more capable the actor, the more rigorous the security controls that are required for successful attack mitigation and prevention. Security is a matter of balancing convenience, defense, and cost, based on requirements.

In some cases, it's impossible to secure a Red Hat Ceph Storage deployment against all threat actors described here. When deploying Red Hat Ceph Storage, you must decide where the balance lies for your deployment and usage.

As part of your risk assessment, you must also consider the type of data you store and any accessible resources, as this will also influence certain actors. However, even if your data is not appealing to threat actors, they could simply be attracted to your computing resources.

Nation-State Actors

This is the most capable adversary. Nation-state actors can bring tremendous resources against a target and have capabilities beyond that of any other actor. It is difficult to defend against these actors without stringent controls in place, both human and technical.

Serious Organized Crime

This class describes highly capable and financially driven groups of attackers. They are able to fund in-house exploit development and target research. In recent years, the rise of organizations such as the Russian Business Network, a massive cyber-criminal enterprise, has demonstrated how cyber attacks have become a commodity. Industrial espionage falls within the serious organized crime group.

Highly Capable Groups

This refers to hacktivist-type organizations that are not typically commercially funded but can pose a serious threat to service providers and cloud operators.

Motivated Individuals Acting Alone

These attackers come in many guises, such as rogue or malicious employees, disaffected customers, or small-scale industrial espionage.

Script Kiddies

These attackers do not target a specific organization but run automated vulnerability scanning and exploitation. They are often a nuisance; however, compromise by one of these actors is a major risk to an organization's reputation.

The following practices can help mitigate some of the risks identified:

Security Updates

You must consider the end-to-end security posture of your underlying physical infrastructure, including networking, storage, and server hardware. These systems will require their own security hardening practices. For your Red Hat Ceph Storage deployment, you should have a plan to regularly test and deploy security updates.

Product Updates

Red Hat recommends running product updates as they become available. Updates are typically released every six weeks (and occasionally more frequently). Red Hat endeavors to make point releases and z-stream releases fully compatible within a major release in order to not require additional integration testing.

Access Management

Access management includes authentication, authorization, and accounting. Authentication is the process of verifying the user's identity. Authorization is the process of granting permissions to an authenticated user. Accounting is the process of tracking which user performed an action. When granting system access to users, apply the *principle of least privilege*, and only grant users the granular system privileges they actually need. This approach can also help mitigate the risks of both malicious actors and typographical errors from system administrators.

Manage Insiders

You can help mitigate the threat of malicious insiders by applying careful assignment of role-based access control (minimum required access), using encryption on internal interfaces, and using authentication and authorization security such as centralized identity management. You can also consider additional non-technical options, such as separation of duties and irregular job role rotation.

Security zones

A security zone comprises users, applications, servers, or networks that share common trust requirements and expectations within a system. Typically they share the same authentication, authorization requirements, and users. Although you might refine these zone definitions further, this section refers to four distinct security zones, three of which form the bare minimum that is required to deploy a security-hardened Red Hat Ceph Storage cluster.

The following list are the security zones from least to most trusted:

- **Public Security Zone:** The public security zone is an entirely untrusted area of the cloud infrastructure. It can refer to the Internet as a whole or simply to networks that are external to your OpenStack deployment over which you have no authority. Any data with confidentiality or integrity requirements that traverse this zone should be protected using compensating controls such as encryption. The public security zone **SHOULD NOT** be confused with the Ceph Storage Cluster's front- or client-side network, which is referred to as the `public_network` in Red Hat Ceph Storage and is usually **NOT** part of the public security zone or the Ceph client security zone.

- **Ceph Client Security Zone:** With Red Hat Ceph Storage, the Ceph client security zone refers to networks accessing Ceph clients such as Ceph Object Gateway, Ceph Block Device, Ceph Filesystem, or `librados`. The Ceph client security zone is typically behind a firewall separating itself from the public security zone. However, Ceph clients are not always protected from the public security zone. It is possible to expose the Ceph Object Gateway's S3 and Swift APIs in the public security zone.

- **Storage Access Security Zone:** The storage access security zone refers to internal networks providing Ceph clients with access to the Ceph Storage Cluster. We use the phrase *storage access security zone* so that this document is consistent with the terminology used in the OpenStack Platform Security and Hardening Guide. The storage access security zone includes the Ceph Storage Cluster's front- or client-side network, which is referred to as the `public_network` in Red Hat Ceph Storage.

- **Ceph Cluster Security Zone:** The Ceph cluster security zone refers to the internal networks providing the Ceph Storage Cluster's OSD daemons with network communications for replication, heartbeating, backfilling, and recovery. The Ceph cluster security zone includes the Ceph Storage Cluster's backside network, which is referred to as the `cluster_network` in Red Hat Ceph Storage.

  These security zones can be mapped separately, or combined to represent the majority of the possible areas of trust within a given Red Hat Ceph Storage deployment. Security zones should be mapped out against your specific deployment topology. The zones and their trust requirements will vary depending upon whether the storage cluster is operating in a standalone capacity or is serving a public, private, or hybrid cloud.

  For a visual representation of these security zones, see Security-Optimized Architecture.

Reference

For more information, see Network Communication

Connecting security zones

Any component that spans across multiple security zones with different trust levels or authentication requirements must be carefully configured. These connections are often the weak points in network architecture, and should always be configured to meet the security requirements of the highest trust level of any of the zones being connected. In many cases, the security controls of the connected zones should be a primary concern due to the likelihood of attack. The points where zones meet do present an opportunity for attackers to migrate or target their attack to more sensitive parts of the deployment.

In some cases, Red Hat Ceph Storage administrators might want to consider securing integration points at a higher standard than any of the zones in which the integration point resides. For example, the Ceph Cluster Security Zone can be isolated from other security zones easily, because there is no reason for it to connect to other security zones. By contrast, the Storage Access Security Zone must provide access to port `6789` on Ceph monitor nodes, and ports `6800-7300` on Ceph OSD nodes. However, port `3000` should be exclusive to the Storage Access Security Zone, because it provides access to Ceph Grafana monitoring information that should be exposed to Ceph administrators only. A Ceph Object Gateway in the Ceph Client Security Zone will need to access the Ceph Cluster Security Zone's monitors (port `6789`) and OSDs (ports `6800-7300`), and may expose its S3 and Swift APIs to the Public Security Zone such as over HTTP port `80` or HTTPS port `443`; yet, it may still need to restrict access to the admin API.

The design of Red Hat Ceph Storage is such that the separation of security zones is difficult. As core services usually span at least two zones, special consideration must be given when applying security controls to them.

Security-optimized architecture

A Red Hat Ceph Storage cluster's daemons typically run on nodes that are subnet isolated and behind a firewall, which makes it relatively simple to secure a cluster. By contrast, Red Hat Ceph Storage clients such as Ceph Block Device (rbd), Ceph Filesystem (cephfs), and Ceph Object Gateway (rgw) access the Red Hat storage cluster, but expose their services to other cloud computing platforms.

"Figure: Security-optimized architecture" on page 55 illustrates the security-optimized architecture for the different Red Hat Ceph Storage cluster daemon and client types.

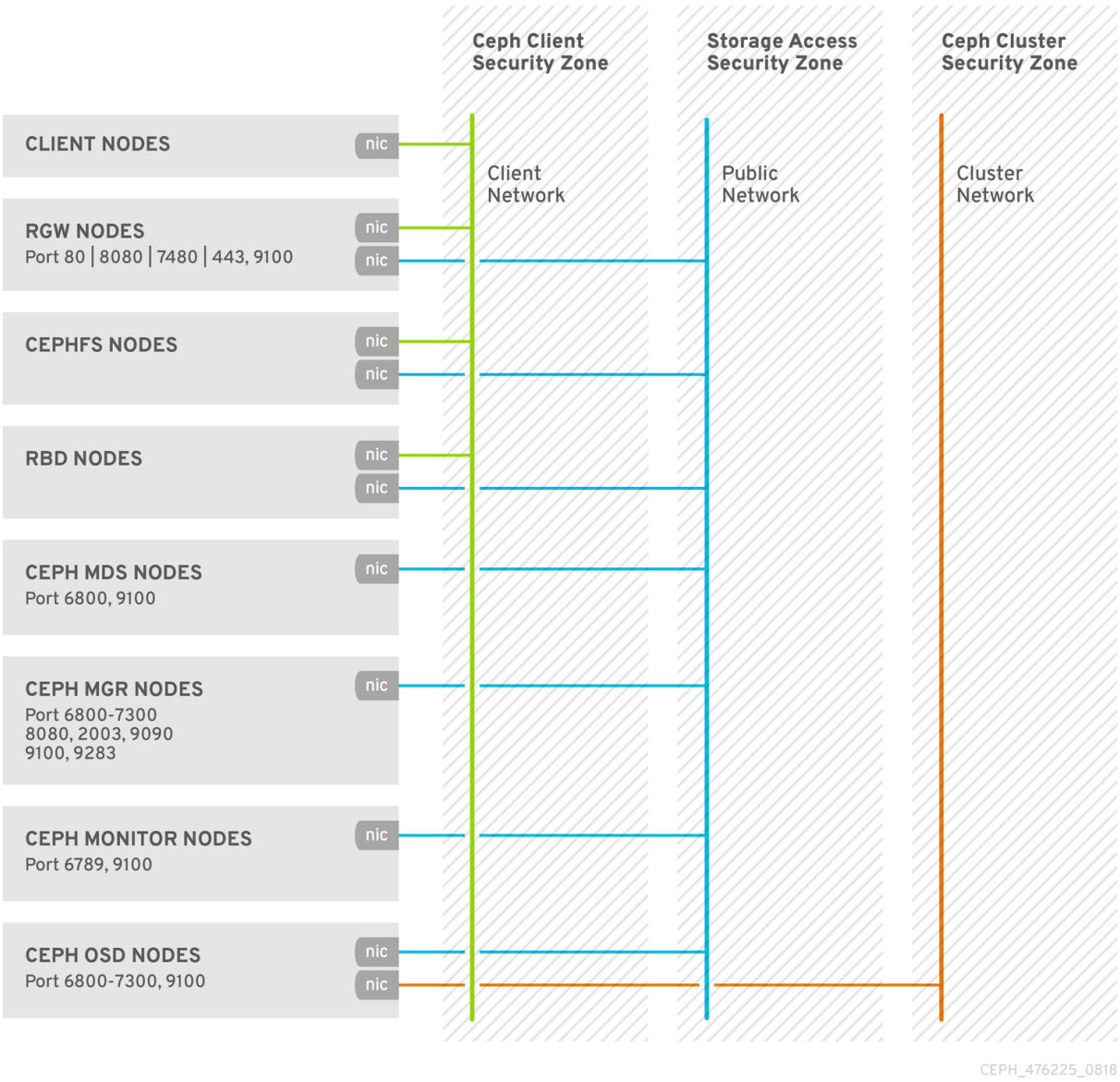


Figure 13: Security-optimized architecture

Encryption and key management

The Red Hat Ceph Storage cluster typically resides in its own network security zone, especially when using a private storage cluster network. In some cases, there are security requirements to assure the confidentiality or integrity of network traffic, and where Red Hat Ceph Storage uses encryption and key management.

> **Note:** Security zone separation might be insufficient for protection if an attacker gains access to Ceph clients on the public network.

The following cases are examples of when encryption and key management are used:

- SSH
- SSL Termination
- Messenger v2 protocol
- Encryption in Transit
- Compression modes of messenger v2 protocol

- Encryption at Rest
- Key rotation

SSH

All nodes in the Red Hat Ceph Storage cluster use SSH as part of deploying the cluster.

On each node:

- A `cephadm` user exists with password-less root privileges.
- The SSH service is enabled and by extension port 22 is open.
- A copy of the `cephadm` user's public SSH key is available.

> **Important:** Any person with access to the `cephadm` user by extension has permission to run commands as `root` on any node in the Red Hat Ceph Storage cluster.

For more information, see How `cephadm` works

SSL Termination

The Ceph Object Gateway may be deployed in conjunction with HAProxy and `keepalived` for load balancing and failover. Understand the implications of terminating SSL with HAProxy and `keepalived` and their encryption status.

The Ceph Object Gateway may be deployed in conjunction with HAProxy and `keepalived` for load balancing and failover. Earlier versions of Civetweb do not support SSL and later versions support SSL with some performance limitations.

You can configure the Beast front-end web server to use the OpenSSL library to provide Transport Layer Security (TLS).

When using HAProxy and `keepalived` to terminate SSL connections, the HAProxy and `keepalived` components use encryption keys.

When using HAProxy and `keepalived` to terminate SSL, the connection between the load balancer and the Ceph Object Gateway is **NOT** encrypted.

Reference

For more information, see the following:

- "Configuring HTTPS for the Ceph Object Gateway" on page 838
- High availability service

Messenger v2 protocol

Messenger v2 (`msgr2`) is the new protocol for Ceph clients, or other Ceph daemons, to connect to the Ceph Monitor daemon. Ceph daemons bind to multiple ports, allowing both the legacy v1-compatible, and the new, v2-compatible Ceph clients to connect to the same storage cluster.

The second version of Ceph's on-wire protocol, `msgr2`, has the following features:

- A secure mode encrypting all data moving through the network.
- Encapsulation improvement of authentication payloads, enabling future integration of new authentication modes.
- Improvements to feature advertisement and negotiation.

Ceph clients or other Ceph daemons connecting to the Ceph Monitor daemon uses the `v2` protocol first, if possible, but if not, then the legacy `v1` protocol is used. By default, both messenger protocols, `v1` and `v2`, are enabled. The new v2 port is 3300, and the legacy v1 port is 6789, by default.

The messenger v2 protocol has two configuration options that control whether the v1 or the v2 protocol is used:

- `ms_bind_msgr1` - This option controls whether a daemon binds to a port speaking the v1 protocol; it is `true` by default.
- `ms_bind_msgr2` - This option controls whether a daemon binds to a port speaking the v2 protocol; it is `true` by default.

Similarly, two options control based on IPv4 and IPv6 addresses used:

- `ms_bind_ipv4` - This option controls whether a daemon binds to an IPv4 address; it is `true` by default.
- `ms_bind_ipv6` - This option controls whether a daemon binds to an IPv6 address; it is `true` by default.

> **Note:** The ability to bind to multiple ports has paved the way for dual-stack IPv4 and IPv6 support.

The `msgr2` protocol supports two connection modes:

- `crc`
  - Provides strong initial authentication when a connection is established with `cephx`.
  - Provides a `crc32c` integrity check to protect against bit flips.
  - Does not provide protection against a malicious man-in-the-middle attack.
  - Does not prevent an eavesdropper from seeing all post-authentication traffic.
- `secure`
  - Provides strong initial authentication when a connection is established with `cephx`.
  - Provides full encryption of all post-authentication traffic.
  - Provides a cryptographic integrity check.

The default mode is `crc`.

Ceph Object Gateway Encryption

Also, the Ceph Object Gateway supports encryption with customer-provided keys using its S3 API.

> **Important:** To comply with regulatory compliance standards requiring strict encryption in transit, administrators **MUST** deploy the Ceph Object Gateway with client-side encryption.

Ceph Block Device Encryption

System administrators integrating Ceph as a backend for OpenStack Platform 13 **MUST** encrypt Ceph Block Device volumes using `dm_crypt` for RBD Cinder to ensure on-wire encryption within the Ceph storage cluster.

> **Important:** To comply with regulatory compliance standards requiring strict encryption in transit, system administrators **MUST** use `dmcrypt` for RBD Cinder to ensure on-wire encryption within the Ceph storage cluster.

Reference

For more information, see Configuring

Encryption in transit

Encryption for all Ceph traffic over the network is enabled by default, with the introduction of the messenger version 2 protocol. The `secure` mode setting for messenger v2 encrypts communication between Ceph daemons and Ceph clients, providing end-to-end encryption.

You can check for encryption of the messenger v2 protocol with the `ceph config dump` command, `netstat -Ip | grep ceph-osd` command, or verify the Ceph daemon on the v2 ports.

Reference

For more information, see the following:

- "SSL Termination" on page 56
- "Messenger v2 protocol" on page 56
- S3 server-side encryption
- Server side encryption

Compression modes of messenger v2 protocol

The messenger v2 protocol supports the compression feature. Compression is not enabled by default.

Compressing and encrypting the same message is not recommended as the level of security of messages between peers is reduced. If encryption is enabled, a request to enable compression is ignored until the configuration option `ms_osd_compress_mode` is set to `true`.

It supports two compression modes:

- `force`
  - In multi-availability zones deployment, compresses replication messages between OSDs saves latency.
  - In the public cloud, minimizes message size, thereby reducing network costs to cloud provider.
  - Instances on public clouds with NVMe provides low network bandwidth relative to the device bandwidth. Does not provide protection against a malicious man-in-the-middle attack.
- `none`
  - The messages are transmitted without compression.

To ensure that compression of the message is enabled, run the `debug_ms` command and check some debug entries for connections. Also, you can run the `ceph config get` command to get details about the different configuration options for the network messages.

Reference

- Network configuration options

Encryption at Rest

Red Hat Ceph Storage supports encryption at rest for both the Ceph Storage Cluster as well as for Ceph Object Gateway in various scenarios.

1. **Ceph Storage Cluster:** The Ceph Storage Cluster supports Linux Unified Key Setup or LUKS encryption of Ceph OSDs and their corresponding journals, write-ahead logs, and metadata databases. In this scenario, Ceph will encrypt all data at rest irrespective of whether the client is a Ceph Block Device, Ceph Filesystem, or a custom application built on `librados`.
2. **Ceph Object Gateway:** The Ceph storage cluster supports encryption of client objects. Additionally, the data transmitted is between the Ceph Object Gateway and the Ceph Storage Cluster is in encrypted form.

Ceph Storage Cluster Encryption

The Ceph storage cluster supports encrypting data stored in Ceph OSDs. Red Hat Ceph Storage can encrypt logical volumes with `lvm` by specifying `dmcrypt`; that is, `lvm`, invoked by `ceph-volume`, encrypts an OSD's logical volume, not its physical volume. It can encrypt non-LVM devices like partitions using the same OSD key. Encrypting logical volumes allows for more configuration flexibility.

Ceph uses LUKS v1 rather than LUKS v2, because LUKS v1 has the broadest support among Linux distributions.

When creating an OSD, `lvm` will generate a secret key and pass the key to the Ceph Monitors securely in a JSON payload via `stdin`. The attribute name for the encryption key is `dmcrypt_key`.

Important: System administrators must explicitly enable encryption.

By default, Ceph does not encrypt data stored in Ceph OSDs. System administrators must enable `dmcrypt` to encrypt data stored in Ceph OSDs. When using a Ceph Orchestrator service specification file for adding Ceph OSDs to the storage cluster, set the following option in the file to encrypt Ceph OSDs:

Example

```
...
encrypted: true
...
```

Note: LUKS and `dmcrypt` only address encryption for data at rest, not encryption for data in transit.

Ceph Object Gateway Encryption

The Ceph Object Gateway supports encryption with customer-provided keys using its S3 API. When using customer-provided keys, the S3 client passes an encryption key along with each request to read or write encrypted data. It is the customer's responsibility to manage those keys. Customers must remember which key the Ceph Object Gateway used to encrypt each object. For full information about Ceph Object Gateway encryption, see Server side encryption.

Reference

For more information, see the following:

- S3 API server-side encryption

Enabling key rotation

Key rotation helps ensure that current industry and security compliance requirements are met.

Before you begin

Before you begin, be sure that you have a running Red Hat Ceph Storage cluster and admin user privileges.

About this task

Note: The active Ceph cluster includes nodes within the Ceph client role with parallel key changes.

Ceph and Ceph Object Gateway daemons within the Ceph cluster have a secret key. This key is used to connect to and authenticate with the cluster. You can update an active security key within an active Ceph cluster with minimal service interruption, by using the key rotation feature.

For more information about enabling key rotation on Ceph dashboard, see User capabilities.

Procedure

1. Rotate the key.

   ```
   ceph orch daemon rotate-key NAME
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch daemon rotate-key mgr.ceph-key-host01
   Scheduled to rotate-key mgr.ceph-key-host01 on host 'my-host-host01-installer'
   ```

2. If using a daemon other than MDS, OSD, or MGR, restart the daemon to switch to the new key. MDS, OSD, and MGR daemons do not require daemon restart.

```
ceph orch restart SERVICE_TYPE
```

For example,

```
[ceph: root@host01 /]# ceph orch restart rgw
```

Identity and access management

Red Hat Ceph Storage provides identity and access management for Ceph Storage Cluster users, Ceph Object Gateway users, Ceph Object Gateway LDAP and AD, and for Ceph Object Gateway OpenStack Keystone.

Ceph Storage cluster user access

To identify users and protect against man-in-the-middle attacks, Ceph provides its `cephx` authentication system to authenticate users and daemons.

For more information about `cephx`, see Ceph user management.

> **Important:** The `cephx` protocol **DOES NOT** address data encryption in transport or encryption at rest.

Cephx uses shared secret keys for authentication, meaning both the client and the monitor cluster have a copy of the client's secret key. The authentication protocol is such that both parties are able to prove to each other they have a copy of the key without actually revealing it. This provides mutual authentication, which means the cluster is sure the user possesses the secret key, and the user is sure that the cluster has a copy of the secret key.

In "Figure: User access" on page 60, users are either individuals or system actors such as applications, which use Ceph clients to interact with the Red Hat Ceph Storage cluster daemons.

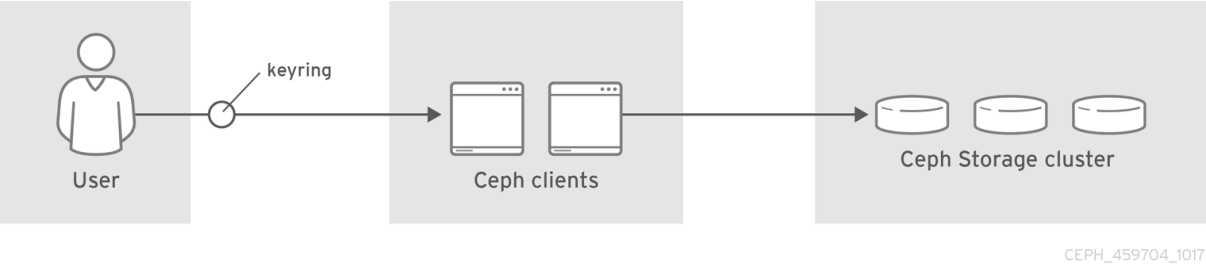


Figure 14: User access

Ceph runs with authentication and authorization enabled by default. Ceph clients may specify a user name and a keyring containing the secret key of the specified user, usually by using the command line. If the user and keyring are not provided as arguments, Ceph will use the `client.admin` administrative user as the default. If a keyring is not specified, Ceph will look for a keyring by using the `keyring` setting in the Ceph configuration.

> **Important:** To harden a Ceph cluster, keyrings **SHOULD ONLY** have read and write permissions for the current user and `root`. The keyring containing the `client.admin` administrative user key must be restricted to the `root` user.

Reference

For more information, see the following:

- Configuring
- Ceph authentication configuration

Ceph Object Gateway user access

The Ceph Object Gateway provides a RESTful application programming interface (API) service with its own user management that authenticates and authorizes users to access S3 and Swift APIs containing user data.

Authentication consists of:

- **S3 User:** An access key and secret for a user of the S3 API.
- **Swift User:** An access key and secret for a user of the Swift API. The Swift user is a subuser of an S3 user. Deleting the S3 *parent* user will delete the Swift user.
- **Administrative User:** An access key and secret for a user of the administrative API. Administrative users should be created sparingly, as the administrative user will be able to access the Ceph Admin API and run its functions, such as creating users, and giving them permissions to access buckets or containers and their objects among other things.

The Ceph Object Gateway stores all user authentication information in Ceph Storage cluster pools. Additional information may be stored about users including names, email addresses, quotas, and usage.

Reference

For more information, see the following:

- User management
- Creating an administrative user

Ceph Object Gateway LDAP or AD authentication

Red Hat Ceph Storage supports Light-weight Directory Access Protocol (LDAP) servers for authenticating Ceph Object Gateway users. When configured to use LDAP or Active Directory (AD), Ceph Object Gateway defers to an LDAP server to authenticate users of the Ceph Object Gateway.

Ceph Object Gateway controls whether to use LDAP. However, once configured, it is the LDAP server that is responsible for authenticating users.

To secure communications between the Ceph Object Gateway and the LDAP server, Red Hat recommends deploying configurations with LDAP Secure or LDAPS.

> **Important:**
>
> - When configuring LDAP, ensure the `ceph` and `cephadm` users are created in LDAP.
> - When using LDAP, ensure that access to the `rgw_ldap_secret = PATH_TO_SECRET_FILE` secret file is secure.

Reference

For more information, see the following:

- Configuring LDAP and Ceph Object Gateway
-

Ceph Object Gateway OpenStack Keystone authentication

Red Hat Ceph Storage supports using OpenStack Keystone to authenticate Ceph Object Gateway Swift API users. The Ceph Object Gateway can accept a Keystone token, authenticate the user and create a corresponding Ceph Object Gateway user. When Keystone validates a token, the Ceph Object Gateway considers the user authenticated.

Ceph Object Gateway controls whether to use OpenStack Keystone for authentication. However, once configured, it is the OpenStack Keystone service that is responsible for authenticating users.

Configuring the Ceph Object Gateway to work with Keystone requires converting the OpenSSL certificates that Keystone uses for creating the requests to the `nss db` format.

Reference

For more information, see the following:

- Ceph Object Gateway and OpenStack Keystone

Ceph Dashboard SSO using OpenID Connect (OIDC) protocol

The Red Hat Ceph Management stack is built by using a modular, service-based architecture that relies on nginx and `oauth2-proxy` components.

The Ceph Management gateway (`mgmt-gateway`) provides unified access, improved user experience, and high availability for both the Ceph Dashboard and monitoring.

The OAuth2 Proxy (`oauth2-proxy`) service provides enhanced security, seamless SSO, and centralized authentication management.

Enable the `mgmt-gateway` service by itself to provide a single-entry point to the cluster, high availability for monitoring and dashboard, as well as for improved security. Enable the service together with the `oauth2-proxy` to use SSO.

With the Ceph Management gateway (`mgmt-gateway`) and the OAuth2 Proxy (`oauth2-proxy`) in place, nginx automatically directs the user through the `oauth2-proxy` to the configured Identity Provider (*IdP*), when single sign-on (SSO) is configured. The IdP then runs the authentication and login.

This configuration provides the following advantages:

- Unified access.
- Improved user experience.
- High availability for the Ceph Dashboard and monitoring.
- Enhanced security.
- Seamless SSO.
- Centralized authentication management.

For more information about this modular SSO solution and implementation with the Ceph Dashboard, see Using single sign-on with the dashboard.

For more information about dashboard SSO options, see Using single sign-on with the dashboard

Using the Ceph Management gateway (`mgmt-gateway`)

The Ceph Management gateway (`mgmt-gateway`) service provides a modular, service-based architecture design.`cephadm` manages the `mgmt-gateway` service, which is built on top of nginx. This gateway acts as the front-end and single entry point to the Ceph cluster, providing unified access to all Ceph applications, including the Ceph Dashboard and the monitoring stack.

Using nginx enhances security and simplifies access management due to its robust community support and high-security standards. The `mgmt-gateway` service acts as a reverse proxy that routes requests to the appropriate Ceph application instances. Use the `mgmt-gateway` either by itself or with the OAuth2 Proxy service for authentication to enable single-sign on (SSO) access to the Ceph Dashboard. For more information about the OAuth2 Proxy service, see “Using the OAuth2 Proxy (oauth2-proxy) service” on page 71.

Using the Ceph Management gateway provides multiple benefits.

Unified access

Consolidated access through nginx improves security and provide a single entry point to services.

Improved user experience

Users no longer needs to keep track of where each application is running (IP or host).

High availability for Ceph Dashboard

nginx high availability (HA) mechanisms are used to provide high availability for the Ceph Dashboard.

High availability for monitoring

nginx high availability mechanisms are used to provide high availability for monitoring.

High availability for mg mt-gateway

Users can have continuous access to Ceph management tools, such as the Ceph Dashboard, Prometheus, Grafana, and Alertmanager, by enabling high availability (HA) for the Ceph Management gateway (`mgmt-gateway`). For more information about enabling high availability (HA) for the Ceph Management gateway, see “Enabling high availability for Ceph Management gateway” on page 67

Deploying the `mgmt-gateway` provides enhanced security. Once the `mgmt-gateway` service is deployed users cannot access monitoring services without authentication through the Ceph Dashboard. After the service is deployed, Prometheus, Grafana, and Alertmanager applications are accessible through links on the Ceph Dashboard. To get to the application links, go to **Administration › Services** on the Ceph Dashboard.

> **Note:** In `mgmt-gateway` mode, Prometheus and Alertmanager require the user to define a unique username and password. The default is `admin:admin`. If OAuth2-proxy SSO is not enabled, the user must manually define these credentials.

nginx HA mechanisms are used to provide high availability for all the Ceph management applications including the Ceph Dashboard and monitoring stack. The `mgmt-gateway` service handles manager failover transparently and redirects the user to the active manager.

With monitoring, the Ceph Management service takes care of handling HA when several instances of Prometheus, Alertmanager or Grafana are available. The reverse proxy automatically detects healthy instances and uses them to process user requests.

> **Important:** Design high availability configurations carefully to avoid introducing single points of failure.

Ceph Management gateway limitations

Using the `mgmt-gateway` service has the following limitations:

- Services must bind to the appropriate ports based on the applications being proxied. Ensure that there are no port conflicts that might disrupt service availability.
- The `mgmt-gateway` service internally makes use of the nginx reverse proxy.
  Use the following to find the default container image is used by default:

  ```
  DEFAULT_NGINX_IMAGE = 'quay.io/ceph/NGINX_IMAGE'
  ```

  Storage administrators can change the image being used by changing the **`container_image_nginx`**`cephadm`module option. If other daemons were running, you must redeploy the daemons to use the new image.

  ```
  ceph config set mgr mgr/cephadm/container_image_nginx NEW_NGINX_IMAGE
  ceph orch redeploy mgmt-gateway
  ```

Reconfigure the dashboard to use `cephadm`-signed certificates

Replace custom dashboard certificates with `cephadm`-signed certificates required for mTLS when the management gateway is enabled.

Before you begin

Conditions

Perform this task only if the dashboard currently uses custom (non-`cephadm`) certificates.

Impact

Reloading the dashboard briefly interrupts access to the Ceph Dashboard.

About this task

While reconfiguring the dashboard can be done at any time, be sure to reconfigure enabling the Ceph Management gateway.

Procedure

1. Generate `cephadm`-signed certificates for the dashboard.

```
ceph orch certmgr generate-certificates dashboard \

tee \
>(jq -r '.cert' > dashboard.cert.pem) \
>(jq -r '.key' > dashboard.key.pem) \
> /dev/null
```

   Two files are created in the current directory. `dashboard.cert.pem` and `dashboard.key.pem`.

2. Set the dashboard certificate.

```
ceph dashboard set-ssl-certificate -i dashboard.cert.pem
```

3. Set the dashboard key.

```
ceph dashboard set-ssl-certificate-key -i dashboard.key.pem
```

4. Reload the dashboard module to apply the new certificates.

```
ceph mgr module disable dashboard; ceph mgr module enable dashboard --force
```

What to do next

Verify the dashboard is using the new `cephadm`-signed certificates and that all services start without errors.

- Open the dashboard URL and confirm HTTPS loads without certificate errors.
- Check the MGR logs for successful dashboard startup and TLS messages.
- (Optional) Validate that backend services (Prometheus, Alertmanager) connect without mTLS errors after enabling `mgmt-gateway`.

Enabling the Ceph Management gateway

Enable the Ceph Management gateway for SSO access to the Dashboard and the Ceph cluster. Enabling the `mgmt-gateway` service either with the `cephadm` CLI commands or by using a service specification file.

Before you begin

Before you begin, if the dashboard uses custom certificates, you must replace them with `cephadm`-signed certificates before you enable the management gateway. For more information, see “Reconfigure the dashboard to use cephadm-signed certificates” on page 63.

About this task

After deploying the `mgmt-gateway` service, direct access to services like Prometheus, Grafana, and Alertmanager is no longer allowed. These services are now accessible only through the Ceph Dashboard by the provided links in **Administration › Services**.

Enabling the Ceph Management gateway with the command-line interface

Procedure

- Deploy the `mgmt-gateway` service.

```
ceph orch apply mgmt-gateway [--placement=DESTINATION_HOST] [--enable-auth=true]
```

> **Note:** The **`--enable-auth=true`** parameter is mandatory to enable SSO with the `oauth2-proxy`.

   For example,

```
[ceph: root@host01 /]# ceph orch apply mgmt-gateway --placement=host01
```

What to do next

Verify that the service has been deployed, as expected.

1. Run the **ceph orch ls** command to get the service status.
2. Run the **ceph orch ps** command to get the status of the corresponding daemons.

Enabling the Ceph Management gateway with a service specification file

Before you begin

Before enabling the Ceph Management gateway, be sure that the following are on each Ceph node that the `mgmt-gateway` service will run on.

- The port for gateway service use.
- A running Red Hat Ceph Storage cluster.
- (Optional) SSL protocols being used.
- (Optional) SSL ciphers.
- (Optional) SSL certificates and certificate keys.

For more information about SSL protocols, ciphers, certificates, and certificate keys, see the *Deploying web servers and reverse proxies* in the Red Hat Enterprise Linux documentation.

Procedure

1. Create a YAML file for the `mgmt-gateway` service.
   For example,

```
[root@host01 ~]# touch mgmt-gateway.yaml
```

2. Edit the YAML file to include the following details.

> **Important:** The following fields are optional: **ssl_protocols**, **ssl_ciphers**, and **ssl_certificate**. When omitted, the `mgmt-gateway` service uses a safe and secure configuration. Changing these fields are permitted but should be done with care as this can compromise the security of your system.

```
service_type: mgmt-gateway
placement:
  hosts:
    - HOST
spec:
 port: PORT
 ssl_protocols:   # Optional
   - TLSv1.3
 ssl_ciphers:     # Optional
   - AES128-SHA
   - AES256-SHA
   - RC4-SHA
 ssl_certificate: |   # Optional
   -----BEGIN CERTIFICATE-----
   < YOU CERT DATA HERE >
   -----END CERTIFICATE-----
 ssl_certificate_key: |
  -----BEGIN RSA PRIVATE KEY-----
   < YOU PRIV KEY DATA HERE >
  -----END RSA PRIVATE KEY-----
```

> **Important:** Use the TLSv1.3 SSL protocol. TLSv1.3 includes a set of secure ciphers by default. While TLSv1.2 is supported, it is crucial to use only a subset of secure ciphers when using this protocol. Using weak or outdated ciphers can significantly compromise the security of your system.

For example,

```
service_type: mgmt-gateway
service_id: gateway
placement:
  hosts:
    - ceph0
spec:
 port: 5000
 ssl_protocols:
   - TLSv1.3
   - ...
 ssl_ciphers:
   - AES128-SHA
   - AES256-SHA
   - ...
 ssl_certificate: |
   -----BEGIN CERTIFICATE-----
   MIIDtTCCAp2gAwIBAgIYMC4xNzc1NDQxNjEzMzc2MjMyXzxvQ7EcMA0GCSqGSIb3
   DQEBCwUAMG0xCzAJBgNVBAYTAlVTMQ0wCwYDVQQIDARVdGFoMRcwFQYDVQQHDA5T
   [...]
   -----END CERTIFICATE-----
ssl_certificate_key: |
   -----BEGIN PRIVATE KEY-----
   MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBKcwggSjAgEAAoIBAQC5jdYbjtNTAKW4
   /CwQr/7wOiLGzVxChn3mmCIF3DwbL/qvTFTX2d8bDf6LjGwLYloXHscRfxszX/4h
   [...]
   -----END PRIVATE KEY-----
```

“mgmt-gateway specific fields in the spec file” on page 66 lists fields that are specific to the `mgmt-gateway` service section of the spec file (`ceph.deployment.service_spec.MgmtGatewaySpec`).

Table 9: `mgmt-gateway` specific fields in the spec file

Field	Description
`disable_https`	Is a flag to disable HTTPS. If `True`, the server will use unsecure HTTP.
`enable_auth`	Is a flag to enable SSO auth. Requires `oauth2-proxy` to be active for SSO authentication.
`networks`	A list of network identities instructing the daemons to only bind on the particular networks in that list. In case the cluster is distributed across multiple networks. You can add multiple networks.
`placement`	For the orchestrator to deploy a service, it needs to know where to deploy daemons, and how many to deploy. This is the role of a placement specification. Placement specifications can either be passed as command line arguments or in a YAML files. For more information, see Managing services.
`port`	The port number on which the server will listen.
`server_tokens`	Flag control server tokens in responses: `on`, `off`, `build`, `string`.
`ssl_cert`	A multi-line string that contains the SSL certificate.
`ssl_key`	A multi-line string that contains the SSL key.
`ssl_ciphers`	List of supported secure SSL ciphers. Changing this list can reduce system security.
`ssl_prefer_server_ciphers`	Prefer server ciphers over client ciphers: `on`, `off`.
`ssl_protocols`	A list of supported SSL protocols (as supported by nginx).
`ssl_session_cache`	Duration of an SSL/TLS session is cached: `off`, `none`, `[builtin[:size]]`, `[shared:name:size]`.

Field	Description
ssl_session_tickets	A multi-option flag to control session tickets: on, off.
ssl_session_timeout	The duration for SSL session timeout. Syntax: time (for example, 5m).
ssl_stapling	Flag to enable or disable SSL stapling: on, off.
ssl_stapling_verify	Flag to control verification of SSL stapling: on, off.

3. Apply the specification file.

```
[root@host01 ~]# ceph orch apply -i mgmt-gateway.yaml
```

What to do next

Verify that the service has been deployed, as expected.

1. Run the **ceph orch ls** command to get the service status.
2. Run the **ceph orch ps** command to get the status of the corresponding daemons.

Enabling high availability for Ceph Management gateway

Ensure continuous access to Ceph management tools, such as the Ceph Dashboard, Prometheus, Grafana, and Alertmanager, by enabling high availability (HA) for the Ceph Management gateway (mgmt-gateway).

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A subset of hosts have the mgmt label set on them. These are the hosts where the daemons are deployed. Use the **ceph orch host ls** command to see which hosts have the mgmt labels set.
- Have an available Virtual IP (VIP).

About this task

By deploying multiple mgmt-gateway instances in an active/standby setup, the system can automatically switch to a backup instance if there is failure. Change to a backup instance by using the keepalived for failover. The oauth2-proxy service operates statelessly, with nginx as a load balancer to evenly distribute traffic. They following are the key components of HA for mgmt-gateway:

Keepalived

Provides failover support.

OAuth2-Proxy

Manages authentication.

Nginx

Runs as a load balancer and reverse-proxy for Ceph Management stack services.

Virtual IP (VIP)

Helps ensure smooth access by other external and internal services to the mgmt-gateway.

Enable Enabling HA to help ensure that tool access is always available, even during a component failure. Enable high availability service either with the cephadm CLI commands or by using a service specification file.

After deploying the mgmt-gateway service, direct access to services like Prometheus, Grafana, and Alertmanager is no longer allowed. These services are now accessible only through the Ceph Dashboard by the links that are provided in **Administration › Services**.

> **Note:** To enable high availability (HA) for the management gateway, the ingress service must also be configured—this is demonstrated in the comprehensive example provided. Without ingress, the virtual IP (VIP) will not function correctly.

Enabling HA for Ceph Management gateway with the command-line interface

Procedure

- Deploy the `mgmt-gateway` service in a high-availability configuration.

  > **Note:** To enable Single Sign-On (SSO), configure and deploy an OAuth2-proxy service. For more information about OAuth2-proxy service, see "Using the OAuth2 Proxy (oauth2-proxy) service" on page 71

  To enable SSO with the `oauth2-proxy`, the **`--enable-auth=true`** parameter is mandatory.

  ```
  ceph orch apply mgmt-gateway --virtual_ip VIP --enable-auth=true --
  placement="label:mgmt"
  ```

  For example,

  ```
  [ceph: root@host01 /]# ceph orch apply mgmt-gateway --virtual_ip 192.168.100.220 --
  enable-auth=true --placement="label:mgmt"
  ```

What to do next

Verify that the service is deployed, as expected.

1. Run the **`ceph orch ls`** command to get the service status.
2. Run the **`ceph orch ps`** command to get the status of the corresponding daemons.

Enabling HA for Ceph Management gateway with a service specification file

Before you begin

Before enabling the Ceph Management gateway, be sure that the following are on each Ceph node that the `mgmt-gateway` service will run on.

- The port for gateway service use.
- A running Red Hat Ceph Storage cluster.
- (Optional) SSL protocols and ciphers for secure communication.
- (Optional) SSL certificates and private key data for secure connections.

For more information about SSL protocols, ciphers, certificates, and certificate keys, see the *Deploying web servers and reverse proxies* in the Red Hat Enterprise Linux documentation.

Procedure

1. Create a YAML file for the `mgmt-gateway` service.
   For example,

   ```
   [root@host01 ~]# touch mgmt-gateway.yaml
   ```

2. Edit the YAML file to include the following details.

   > **Important:** The following fields are optional: **`ssl_protocols`**, **`ssl_ciphers`**, and **`ssl_cert`**. When omitted, the `mgmt-gateway` service uses a safe and secure configuration. Change these fields with care, as they can compromise the security of your system.

   ```
   service_type: mgmt-gateway
   placement:
     hosts: - ceph-node-1
   spec:
     enable_auth: true
     virtual_ip: 192.168.100.220
     ssl_protocols:
       - TLSv1.3
   ```

```
  ssl_ciphers:
    - AES128-SHA
    - AES256-SHA
  ssl_cert: |
    -----BEGIN CERTIFICATE-----
    < YOUR CERT DATA HERE >
    -----END CERTIFICATE-----
  ssl_key: |
    -----BEGIN PRIVATE KEY-----
    < YOUR PRIVATE KEY DATA HERE >
    -----END PRIVATE KEY-----
```

Important: SSL-related fields such as **`ssl_protocols`**, **`ssl_ciphers`**, and **`ssl_cert`** are optional but recommended for secure configurations. If omitted, default secure settings are used. However, incorrect modifications can weaken your system's security. Ensure that any changes are in line with current security best practices.

"mgmt-gateway specific fields in the spec file" on page 69 lists fields that are specific to the `mgmt-gateway` service section of the spec file (`ceph.deployment.service_spec.MgmtGatewaySpec`).

Table 10: `mgmt-gateway` specific fields in the spec file

Field	Description
`disable_https`	Is a flag to disable HTTPS. If `True`, the server uses unsecure HTTP.
`enable_auth`	Is a flag to enable SSO auth. Requires `oauth2-proxy` to be active for SSO authentication.
`networks`	A list of network identities that instruct the daemons to only bind on the particular networks in that list. In case the cluster is distributed across multiple networks, you can add multiple networks.
`placement`	For the orchestrator to deploy a service, it needs to know where to deploy daemons, and how many to deploy. This is the role of a placement specification. Placement specifications can either be passed as command line arguments or in a YAML files. For more information, see Managing services.
`virtual_ip`	The high availability virtual IP number on which the server will reside.
`server_tokens`	Flag control server tokens in responses: `on`, `off`, `build`, `string`.
`ssl_cert`	A multi-line string that contains the SSL certificate.
`ssl_key`	A multi-line string that contains the SSL key.
`ssl_ciphers`	List of supported secure SSL ciphers. Changing this list can reduce system security.
`ssl_prefer_server_ciphers`	Prefer server ciphers over client ciphers: `on`, `off`.
`ssl_protocols`	A list of supported SSL protocols (as supported by nginx).
`ssl_session_cache`	Duration that an SSL/TLS session is cached: `off`, `none`, `[builtin[:size]]`, `[shared:name:size]`.
`ssl_session_tickets`	A multi-option flag to control session tickets: `on`, `off`.
`ssl_session_timeout`	The duration for SSL session timeout. Syntax: time (for example, `5m`).

Field	Description
`ssl_stapling`	Flag to enable or disable SSL stapling: `on`, `off`.
`ssl_stapling_verify`	Flag to control verification of SSL stapling: `on`, `off`.

3. Apply the specification file.

```
[root@host01 ~]# ceph orch apply -i mgmt-gateway.yaml
```

Applying the specification file applies the HA configuration including the virtual IP and optional SSL.

What to do next

Verify that the service has been deployed, as expected.

1. Run the **`ceph orch ls`** command to get the service status.
2. Run the **`ceph orch ps`** command to get the status of the corresponding daemons.

Example configurations for `mgmt-gateway` with high availability

Use these examples to configure multiple `mgmt-gateway` instances with high availability (HA) in an active/standby configuration with keepalived for seamless failover.

Example `mgmt-gateway` configuration

```
service_type: mgmt-gateway
placement:
  label: mgmt
spec:
  enable_auth: true
  virtual_ip: 192.168.100.220  # HA Virtual IP
```

Example of ingress configuration for keepalived

```
service_type: ingress
service_id: ingress-mgmt-gw
placement:
  label: mgmt
virtual_ip: 192.168.100.220
backend_service: mgmt-gateway
keepalive_only: true
```

Example for deploying the `mgmt-gateway` in a high availability configuration

The following is a complete example that shows all the components required to deploy the Ceph management gateway in HA mode with SSO enabled. The configuration includes the nodes, the management gateway, `oauth2-proxy`, and ingress.

```
---
service_type: host
hostname: ceph-node-1
addr: 192.168.100.101
labels:
  - mgmt
---
service_type: host
hostname: ceph-node-2
addr: 192.168.100.102
labels:
  - mgmt
---
service_type: mgmt-gateway
placement:
  label: mgmt
spec:
  enable_auth: true
  virtual_ip: <your-virtual-ip>
---
service_type: oauth2-proxy
placement:
  label: mgmt
spec:
  provider_display_name: "<your-oidc-provider>"
  client_id: <your-client-id>
  client_secret: <your-client-secret>
  oidc_issuer_url: <your-oidc-url>
```

```
  allowlist_domains:
    - 192.168.100.1:8080
---
service_type: ingress
service_id: ingress-mgmt-gw
placement:
  label: mgmt
virtual_ip: <your-virtual-ip>
backend_service: mgmt-gateway
keepalive_only: true
```

Using the OAuth2 Proxy (`oauth2-proxy`) service

The OAuth2 Proxy service provides an advanced method for managing authentication and access control for Ceph applications. The oauth2-proxy service integrates with external Identity Providers (*IdPs*) to provide secure, flexible authentication with the OpenID Connect (*OIDC*) protocol. oauth2-proxy acts as an authentication gateway, ensuring that access to Ceph applications are tightly controlled.

Using the OAuth2 Proxy service with authentication requires using the Ceph Management gateway (`mgmt-gateway`). For more information, see "Using the Ceph Management gateway (mgmt-gateway)" on page 62.

Using the `oauth2-proxy` service provides multiple benefits.

Enhanced security

Provides robust authentication through integration with external IdPs by using the OIDC protocol.

Seamless single sign-on (*SSO*)

Enables seamless SSO across all Ceph monitoring applications, improving user access control. For more information about enabling OAuth2 SSO for the Ceph Dashboard, see Enabling OAuth2 single sign-on.

Centralized authentication

Centralizes authentication management, reducing complexity and improving control over access.

Deploying the `oauth2-proxy` service provides enhanced security. Once the `oauth2-proxy` service is deployed all access to Ceph applications must be authenticated through the external IdP by using OIDC. Authentication prevents unauthorized users from accessing sensitive information. Users are redirected to the IdP for login and then returned to the requested application. This setup ensures secure access and integrates seamlessly with the Ceph management stack. The Prometheus, Alertmanager, and Grafana Ceph applications all require authentication through the required IdP.

The `oauth2-proxy` service uses the OAuth Provider open-source project, enabling easier service integration with a variety of external IdPs, providing a secure and flexible authentication mechanism. For a full list of valid OAuth providers, see **OAuth Provider Configuration** on OAuth2 Proxy Docs.

OAuth2 Proxy limitations

The `oauth2-proxy` service does not support high availability configurations.

> **Important:** Proper configuration of the IdP and OAuth2 Proxy parameters is crucial to avoid authentication failures. Misconfiguration can lead to access issues.

Creating an admin account with Red Hat Single Sign-On 7.6.0

Create an admin account to synchronize OAuth2 service to the Ceph dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Admin-level access to the dashboard.
- Users are added to the dashboard.
- Root-level access on all the hosts.

- Java OpenJDK installed. For more information, see the Installing a JRE on RHEL by using yum section of the *Installing and using OpenJDK 8 for RHEL* guide for OpenJDK on Red Hat Documentation.
- Red Hat Single Sign-On installed from a ZIP file. For more information, see *Installing RH-SSO from a ZIP file* in the Red Hat Single Sign-On 7.6 Server Installation and Configuration Guide on Red Hat Documentation.

About this task

Use Red Hat Single Sign-on (SSO) to synchronize users to the Ceph dashboard. See Syncing users to the Ceph dashboard using Red Hat Single Sign-On.

For more information, see the following:

- Creating roles.
- Creating users.

Procedure

1. Download the *Red Hat Single Sign-On 7.6.0 Server* on the system where Red Hat Ceph Storage is installed. For download package and information, see Software Details for *Red Hat Single Sign-On 7.6.0 Server* on the Red Hat Customer Portal.
2. Extract the folder.

   ```
   unzip rhsso-7.6.0.zip
   ```

   For example,

   ```
   [root@host01 ~]# unzip rhsso-7.6.0.zip
   ```

3. Go to the `standalone/configuration` directory under the newly created `rhsso-7.6.0` directory and open the `standalone.xml` file for editing.
   For example,

   ```
   [root@host01 ~]# cd standalone/configuration
   [root@host01 configuration]# vi standalone.xml
   ```

4. Replace the **`localhost`** and `127.0.0.1` with the IP address of the server where the Red Hat Single-Sign On is installed.
5. From the `bin` directory of the newly created `rhsso-7.6.0` folder, run the `add-user-keycloak` script to add the initial administrator user.

   ```
   ./add-user-keycloak.sh -u admin
   ```

   For example,

   ```
   [root@host01 bin]# add-user-keycloak.sh -u admin
   ```

6. Start the server.
   From the `bin` directory of `rh-sso-7.6` folder, run the `standalone` boot script.

   ```
   ./standalone.sh
   ```

   For example,

   ```
   [root@host01 bin]# ./standalone.sh
   ```

7. Log in to the admin account in `https:IP_ADDRESS:8080/auth` with the new username and password created.

   > **Note:** You must create an admin account only the first time that you log into the console.

Setting up IdP for OAuth2-proxy using Red Hat SSO with OAuth2 protocol

Procedure

Complete these steps from the Red Hat Single Sign-On console.

1. Create a realm.
   The realm must be **Enabled**.
2. Go to **Realm Settings** and complete the required settings.
   - Toggle **Enabled** to **ON**.
   - Toggle **User-Managed Access** to **ON**.
   - Click the link address of the OpenID Endpoint Configuration to retrieve the required endpoint for OAuth2 configuration. The issuer endpoint is used during OAuth2 service configuration.
3. Create a client.
   Set the **Client Protocol** as **openid-connect**.
4. Update the client settings.
   - Set the **Access Type** as **confidential**.
   - Enter the **Valid Redirect URIs**. For example: `<https://<ip|hostname>/oauth2/callback>` and `<https://<ip|hostname>/oauth2/sign_out>`
5. Verify the client credentials.
   Check that the **Client Authenticator** is set as **Client Id and Secret**. The secret is used as the **`client_secret`** parameter when configuring the `oauth2-proxy` service.
6. Create a new administrator role.
   Enter `administrator` as the role name.
7. Update the administrator information.
   a. Go to **Manage › Users › Add User**.
   Complete the **Username**, **Email**, **First Name**, **Last Name**, and toggle **Email Verified** to **ON**.
   b. From **Credentials**, set a new password credentials for the administrator user.
   c. From **Role Mappings**, set the **Client Roles** as the client that was created in step "3" on page 73 and click **Add selected** for **administrator** role created in step "6" on page 73.

Enabling the OAuth2 Proxy service

Enable the OAuth2 Proxy service for SSO access to the Dashboard and the Ceph cluster. Enabling the `oauth2-proxy` service either with the `cephadm` CLI commands or by using a service specification file.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Ceph Management gateway (`mgmt-gateway`) service is enabled and **`--enable-auth`** is set to `true`. For more information, see "Enabling the Ceph Management gateway" on page 64.
- You should either have an admin account with Red Hat Single-Sign-On 7.6.0 with OAuth2 protocol running or IBM Security Verify with OAuth2 protocol running. For more information, see Configuring IBM Security Verify as the identity provider using OAuth2 protocol.

About this task

Find the `oauth2-proxy` container image by running the **`ceph config get`** command.

```
ceph config get mgr mgr/cephadm/container_image_oauth2_proxy
```

Storage administrators can specify a custom image by changing the **`container_image_oauth2_proxy`** `cephadm` module option. If other daemons were running, you must redeploy the daemons to use the new image.

```
ceph config set mgr mgr/cephadm/container_image_oauth2_proxy NEW_OAUTH2_PROXY_IMAGE
ceph orch redeploy oauth2_proxy
```

Enabling the OAuth2 Proxy service with the command-line interface

Procedure

1. Deploy the `oauth2-proxy` service.

```
ceph orch apply oauth2-proxy [--placement=DESTINATION_HOST]
```

   For example,

```
[ceph: root@host01 /]# ceph orch apply oauth2-proxy [--placement=host01]
```

2. Verify that the service was correctly deployed.

Enabling the OAuth2 Proxy service with a service specification file

Before you begin

Before enabling the OAuth2 Proxy service, make sure that the following are on each Ceph node that the `oauth2-proxy` service will run on.

- Your client ID.
- The OIDC issuer URL.
- Client secret.
- The relevant domain addresses to allow.

> **Note:** The domain can be the same or different from the **`oidc_issuer_url`**.

- A running Red Hat Ceph Storage cluster.
- (Optional) The host HTTPS address and host port.
- (Optional) Cookie secret.
- (Optional) SSL certificates and certificate keys.

For more information about SSL protocols, ciphers, certificates, and certificate keys, see the *Deploying web servers and reverse proxies* section in the Red Hat Enterprise Linux documentation.

Procedure

1. Create a YAML file for the `oauth2-proxy` service.
   For example,

```
[root@host01 ~]# touch oauth2-proxy.yaml
```

2. Edit the YAML file to include the following details.

```
service_type: oauth2-proxy
service_id: auth-proxy
placement:
  hosts:
    - <target-node>
spec:
 https_address: HTTPS_ADDRESS:PORT
 provider_display_name: MY OIDC PROVIDER
 client_id: CLIENT_ID
 oidc_issuer_url: OIDC ISSUER URL
 allowlist_domains:
    - HTTPS_ADDRESS:PORT
 client_secret: CLIENT_SECRET
 cookie_secret: COOKIE_SECRET
 ssl_cert: |
   -----BEGIN CERTIFICATE-----
   < YOU CERT DATA HERE >
   -----END CERTIFICATE-----
 ssl_key: |
  -----BEGIN RSA PRIVATE KEY-----
   < YOU PRIV KEY DATA HERE >
  -----END RSA PRIVATE KEY-----
```

For example,

```
service_type: oauth2-proxy
service_id: auth-proxy
placement:
  hosts:
    - ceph0
spec:
 https_address: "0.0.0.0:4180"
 provider_display_name: "My OIDC Provider"
 client_id: "your-client-id"
 oidc_issuer_url: "http://192.168.100.1:5556/realms/ceph"
 allowlist_domains:
    - 192.168.100.1:8080
    - 192.168.200.1:5000
 client_secret: "your-client-secret"
 cookie_secret: "your-cookie-secret"
 ssl_cert: |
   -----BEGIN CERTIFICATE-----
   MIIDtTCCAp2gAwIBAgIYMC4xNzc1NDQxNjEzMzc2MjMyXzxvQ7EcMA0GCSqGSIb3
   DQEBCwUAMG0xCzAJBgNVBAYTAlVTMQ0wCwYDVQQIDARVdGFoMRcwFQYDVQQHDA5T
   [...]
   -----END CERTIFICATE-----
ssl_key: |
   -----BEGIN PRIVATE KEY-----
   MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBKcwggSjAgEAAoIBAQC5jdYbjtNTAKW4
   /CwQr/7wOiLGzVxChn3mmCIF3DwbL/qvTFTX2d8bDf6LjGwLYloXHscRfxszX/4h
   [...]
   -----END PRIVATE KEY-----
```

“oauth2-proxy specific fields in the spec file” on page 75 lists fields that are specific to the `oauth2-proxy` service section of the spec file (`ceph.deployment.service_spec.OAuth2ProxySpec`).

Table 11: `oauth2-proxy` specific fields in the spec file

Field	Description
`allowlist_domains`	List of allowed domains for safe redirection after login or logout, preventing unauthorized redirects. This can be the same or different from the **`oidc_issuer_url`**.
`client_id`	The client ID for authenticating with the identity provider.
`client_secret`	The client secret for authenticating with the identity provider.
`cookie_secret`	The secret key is used for signing cookies. Its length must be 16 bytes, 24 bytes, or 32 bytes to create an AES cipher.
`https_address`	The address for HTTPS connections, which are formatted as `host:port`.
`networks`	A list of network identities that instruct the daemons to only bind on the particular networks in that list. You can add multiple networks if the cluster is distributed across multiple networks.
`oidc_issuer_url`	The URL of the OpenID Connect (OIDC) issuer.
`placement`	For the orchestrator to deploy a service, it needs to know where to deploy daemons, and how many to deploy. This is the role of a placement specification. Placement specifications can either be passed as command line arguments or in a YAML files. For more information, see Managing services.
`provider_display_name`	The display name for the identity provider (IdP) in the UI.

Field	Description
`redirect_url`	Optional. The URL oauth2-proxy will redirect to after a successful login. If not provided, the`cephadm`automatically calculates the value of this URL.
`ssl_cert`	Optional. The multi-line SSL certificate for encrypting communications.
`ssl_key`	Optional. The multi-line SSL certificate private key for decrypting communications.

3. Apply the specification file.

```
[root@host01 ~]# ceph orch apply -i oauth2-proxy.yaml
```

Infrastructure security

A proper Red Hat Ceph Storage security plan requires Red Hat Enterprise Linux security and SELinux prerequisites.

For full information, see the following Red Hat Enterprise Linux 9 user documentation on Red Hat Documentation.

- *Security Hardening Guide*
- *Using SELinux Guide*

Administration

Use command line tools to administer Red Hat Ceph Storage clusters. The CLI tools require an administrator key for administrator access privileges to the cluster.

By default, Ceph stores the administrator key in the `/etc/ceph` directory. The default file name is `ceph.client.admin.keyring`. Take steps to secure the keyring so that only a user with administrative privileges to the cluster can access the keyring.

Network communication

Red Hat Ceph Storage provides two networks, both a public and a cluster network.

All Ceph daemons and Ceph clients require access to the public network, which is part of the *storage access security zone*. By contrast, **ONLY** the OSD daemons require access to the cluster network, which is part of the *Ceph cluster security zone*.

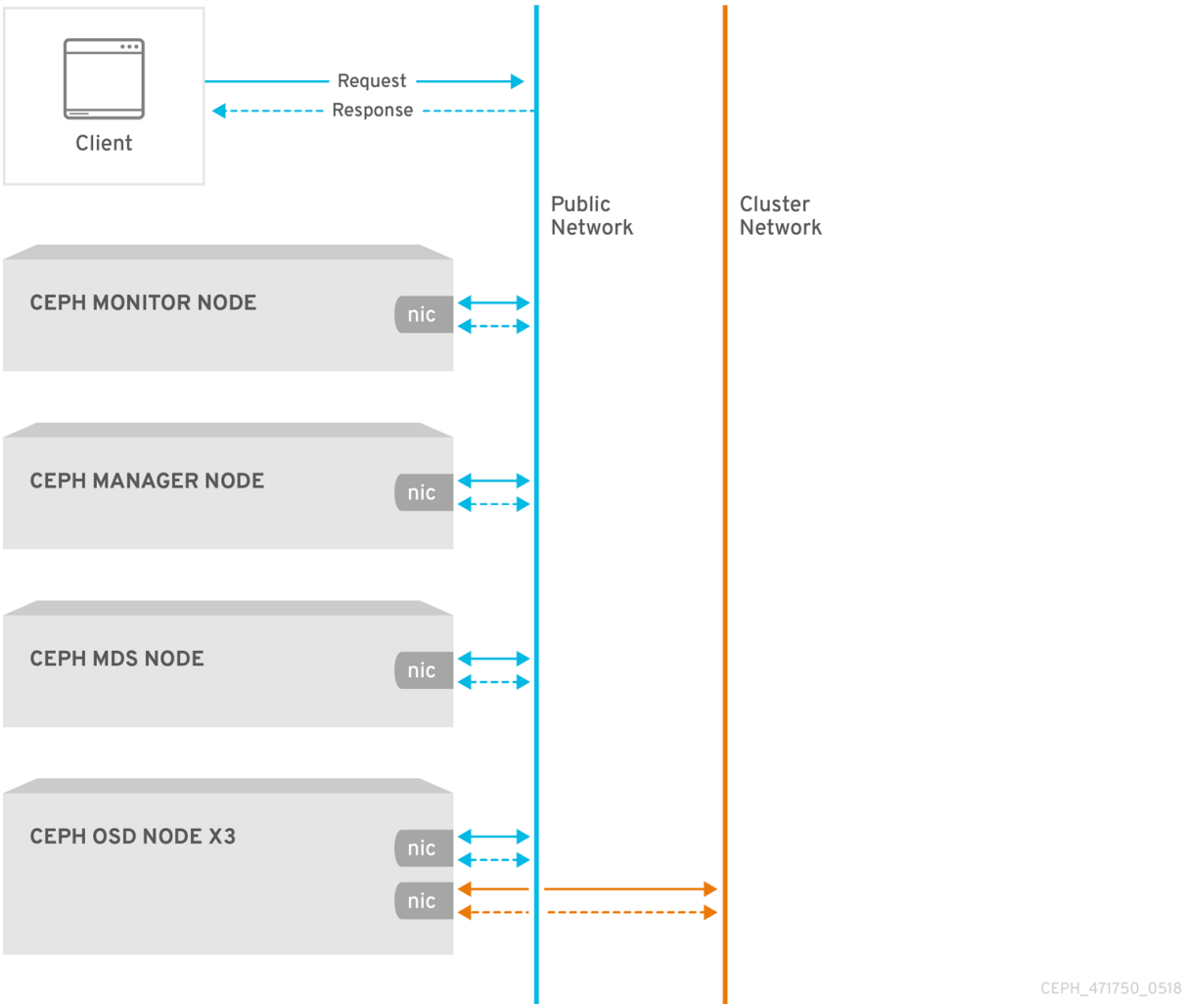


Figure 15: Network architecture

The Ceph configuration contains `public_network` and `cluster_network` settings. For hardening purposes, specify the IP address and the netmask using CIDR notation. Specify multiple comma-delimited IP address and netmask entries if the cluster will have multiple sub-nets.

```
public_network = <public-network/netmask>[,<public-network/netmask>]

cluster_network = <cluster-network/netmask>[,<cluster-network/netmask>]
```

Reference

For more information, see the following:

- Ceph network configuration

Network service security and port architecture

Understand the network security zones, port usage, and default firewall posture for Red Hat Ceph Storage clusters deployed on Red Hat Enterprise Linux 9.

When deploying Red Hat Ceph Storage on Red Hat Enterprise Linux 9 Server, SELinux is enabled by default and the host firewall blocks all inbound traffic except for the SSH service on port 22.

Network security zones

For hardening purposes, segment the cluster into dedicated network zones using separate subnets:

Ceph cluster security zone

Includes cluster daemons (`ceph-mon`, `ceph-mgr`, and `ceph-osd`) and their host nodes. This zone should use its own dedicated subnet.

Storage access security zone

Includes Ceph clients (`ceph-radosgw`, `ceph-mds`, `ceph-fuse`, `libcephfs`, `rbd`, `librbd`, and `librados`) and their hosts.

Ceph daemon ports

Each Ceph daemon uses one or more ports to communicate across the cluster. Administrators typically only modify the default port for the Ceph Object Gateway (`ceph-radosgw`) daemon.

Table 12: Ceph Ports

TCP/UDP Port	Daemon	Configuration Option
6789, 3300	`ceph-mon`	N/A
6800-7568	`ceph-osd`	`ms_bind_port_min` to `ms_bind_port_max`
6800-7568	`ceph-mgr`	`ms_bind_port_min` to `ms_bind_port_max`
6800	`ceph-mds`	N/A
8080	`ceph-radosgw`	`rgw_frontends`

Next steps

For instructions on verifying SELinux, enabling the firewall, and applying zone access rules, see Hardening the network service.

Hardening the network service

Verify SELinux enforcement, enable the host firewall, and restrict network traffic between Ceph security zones on Red Hat Enterprise Linux 9.

Before you begin

Ensure that you have root or `sudo` privileges on each cluster node.

Procedure

1. Start and enable the `firewalld` service on boot:

```
# systemctl enable firewalld
# systemctl start firewalld
# systemctl status firewalld
```

2. Take an inventory of all open ports:

```
# firewall-cmd --list-all
```

On a new installation, the `sources:` line is empty, and the `services` line shows only `ssh` (port 22) and `dhcpv6-client`:

```
sources:
services: ssh dhcpv6-client
```

3. Verify that SELinux is running and in `Enforcing` mode:

```
# getenforce
Enforcing
```

– If SELinux is set to `Permissive`, switch to enforcing mode:

```
# setenforce 1
```

- If SELinux is disabled, enable it following the operating system guidelines. For more information, see *Red Hat Enterprise Linux 9 Using SELinux*.

4. On hosts in the Ceph Storage Cluster zone, restrict inbound traffic so that only designated client hosts can connect to cluster daemons:

```
# firewall-cmd --zone=<zone-name> --add-rich-rule="rule family=\"ipv4\" \
source address=\"<ip-address>/<netmask>\" port protocol=\"tcp\" \
port=\"<port-number>\" accept"
```

   Replace `<zone-name>` with the firewalld zone name, `<ip-address>/<netmask>` with the client subnet in CIDR notation, and `<port-number>` with the daemon port or port range.

5. Apply the rich rule permanently to persist across system reboots:

```
# firewall-cmd --zone=<zone-name> --add-rich-rule="rule family=\"ipv4\" \
source address=\"<ip-address>/<netmask>\" port protocol=\"tcp\" \
port=\"<port-number>\" accept" --permanent
```

Reporting

Red Hat Ceph Storage provides system monitoring and reporting. System administrators can retrieve configuration settings at runtime.

Red Hat Ceph Storage provides basic system monitoring and reporting with the `ceph-mgr` daemon plug-ins, namely, the RESTful API, the dashboard, and other plug-ins such as `Prometheus` and `Zabbix`. Ceph collects this information using `collectd` and sockets to retrieve settings, configuration details, and statistical information.

In addition to default system behavior, system administrators may configure `collectd` to report on security matters, such as configuring the `IP-Tables` or `ConnTrack` plug-ins to track open ports and connections respectively.

Reference

For more information, see Viewing the Ceph configuration at runtime.

Auditing administrator actions

An important aspect of system security is to periodically audit administrator actions on the cluster.

Red Hat Ceph Storage stores a history of administrator actions in the `/var/log/ceph/`*`CLUSTER_FSID`*`/ceph.audit.log` file.

Example

```
[root@host04 ~]# cat /var/log/ceph/6c58dfb8-4342-11ee-a953-fa163e843234/ceph.audit.log
```

Each entry will contain:

- **Timestamp:** Indicates when the command was run.
- **Monitor Address:** Identifies the monitor modified.
- **Client Node:** Identifies the client node initiating the change.
- **Entity:** Identifies the user making the change.
- **Command:** Identifies the command used.

The following is an output of the Ceph audit log:

```
2023-09-01T10:20:21.445990+0000 mon.host01 (mon.0) 122301 : audit [DBG] from='mgr.14189
10.0.210.22:0/1157748332' entity='mgr.host01.mcadea' cmd=[{"prefix": "config generate-
minimal-conf"}]: dispatch
2023-09-01T10:20:21.446972+0000 mon.host01 (mon.0) 122302 : audit [INF] from='mgr.14189
10.0.210.22:0/1157748332' entity='mgr.host01.mcadea' cmd=[{"prefix": "auth get", "entity":
"client.admin"}]: dispatch
2023-09-01T10:20:21.453790+0000 mon.host01 (mon.0) 122303 : audit [INF] from='mgr.14189
10.0.210.22:0/1157748332' entity='mgr.host01.mcadea'
2023-09-01T10:20:21.457119+0000 mon.host01 (mon.0) 122304 : audit [DBG] from='mgr.14189
10.0.210.22:0/1157748332' entity='mgr.host01.mcadea' cmd=[{"prefix": "osd tree", "states":
```

```
["destroyed"], "format": "json"}]: dispatch
2023-09-01T10:20:30.671816+0000 mon.host01 (mon.0) 122305 : audit [DBG] from='mgr.14189
10.0.210.22:0/1157748332' entity='mgr.host01.mcadea' cmd=[{"prefix": "osd blocklist ls",
"format": "json"}]: dispatch
```

In distributed systems such as Ceph, actions may begin on one instance and get propagated to other nodes in the cluster. When the action begins, the log indicates `dispatch`. When the action ends, the log indicates `finished`.

Data retention

Red Hat Ceph Storage stores user data, but usually in an indirect manner. Customer data retention may involve other applications, such as the OpenStack Platform.

Ceph Storage Cluster

The Ceph Storage Cluster, often referred to as the Reliable Autonomic Distributed Object Store or RADOS, stores data as objects within pools. In most cases, these objects are the atomic units representing client data, such as Ceph Block Device images, Ceph Object Gateway objects, or Ceph File System files. However, custom applications built on top of `librados` might bind to a pool and store data too.

Cephx controls access to the pools storing object data. However, Ceph Storage Cluster users are typically Ceph clients, and not users. Consequently, users generally DO NOT have the ability to write, read or delete objects directly in a Ceph Storage Cluster pool.

Ceph Block Device

The most popular use of Red Hat Ceph Storage, the Ceph Block Device interface, also referred to as RADOS Block Device, or RBD, creates virtual volumes, images, and compute instances and stores them as a series of objects within pools. Ceph assigns these objects to placement groups and distributes or places them pseudo-randomly in OSDs throughout the cluster.

Depending upon the application consuming the Ceph Block Device interface, usually OpenStack Platform, users may create, modify, and delete volumes and images. Ceph handles the create, retrieve, update, and delete operations of each individual object.

Deleting volumes and images destroys the corresponding objects in an unrecoverable manner. However, residual data artifacts may continue to reside on storage media until overwritten. Data may also remain in backup archives.

Ceph File System

The Ceph File System interface creates virtual file systems and stores them as a series of objects within pools. Ceph assigns these objects to placement groups and distributes or places them pseudo-randomly in OSDs throughout the cluster.

Typically, the Ceph File System uses two pools:

- **Metadata:** The metadata pool stores the data of the Ceph Metadata Server (MDS), which generally consists of inodes; that is, the file ownership, permissions, creation date and time, last modified or accessed date and time, parent directory, and so on.
- **Data:** The data pool stores file data. Ceph may store a file as one or more objects, typically representing smaller chunks of file data such as extents.

Depending upon the application consuming the Ceph File System interface, usually OpenStack Platform, users may create, modify, and delete files in a Ceph File System. Ceph handles the create, retrieve, update, and delete operations of each individual object representing the file.

Deleting files destroys the corresponding objects in an unrecoverable manner. However, residual data artifacts may continue to reside on storage media until overwritten. Data may also remain in backup archives.

Ceph Object Gateway

From a data security and retention perspective, the Ceph Object Gateway interface has some important differences when compared to the Ceph Block Device and Ceph File System interfaces. The Ceph Object

Gateway provides a service to users. The Ceph Object Gateway may store user authentication information, user data, and logging.

- **User Authentication Information:** User authentication information generally consists of user IDs, user access keys, and user secrets. It may also comprise a user's name and email address if provided. Ceph Object Gateway will retain user authentication data unless the user is explicitly deleted from the system.
- **User Data:** User data generally comprises user- or administrator-created buckets or containers, and the user-created S3 or Swift objects contained within them. The Ceph Object Gateway interface creates one or more Ceph Storage cluster objects for each S3 or Swift object and stores the corresponding Ceph Storage cluster objects within a data pool. Ceph assigns the Ceph Storage cluster objects to placement groups and distributes or places them pseudo-randomly in OSDs throughout the cluster. The Ceph Object Gateway may also store an index of the objects contained within a bucket or index to enable services such as listing the contents of an S3 bucket or Swift container. Additionally, when implementing multi-part uploads, the Ceph Object Gateway may temporarily store partial uploads of S3 or Swift objects.

  Users may create, modify, and delete buckets or containers, and the objects contained within them in a Ceph Object Gateway. Ceph handles the create, retrieve, update, and delete operations of each individual Ceph Storage cluster object representing the S3 or Swift object.

  Deleting S3 or Swift objects destroys the corresponding Ceph Storage cluster objects in an unrecoverable manner. However, residual data artifacts may continue to reside on storage media until overwritten. Data may also remain in backup archives.
- **Logging:** Ceph Object Gateway also stores logs of user operations that the user intends to accomplish and operations that have been run. This data provides traceability about who created, modified or deleted a bucket or container, or an S3 or Swift object residing in an S3 bucket or Swift container. When users delete their data, the logging information is not affected and will remain in storage until deleted by a system administrator or removed automatically by expiration policy.

Bucket Lifecycle

Ceph Object Gateway also supports bucket lifecycle features, including object expiration. Data retention regulations like the General Data Protection Regulation may require administrators to set object expiration policies and disclose them to users among other compliance factors.

Multi-site

Ceph Object Gateway is often deployed in a multi-site context whereby a user stores an object at one site and the Ceph Object Gateway creates a replica of the object in another cluster possibly at another geographic location. For example, if a primary cluster fails, a secondary cluster may resume operations. In another example, a secondary cluster may be in a different geographic location, such as an edge network or content-delivery network such that a client may access the closest cluster to improve response time, throughput, and other performance characteristics. In multi-site scenarios, administrators must ensure that each site has implemented security measures. Additionally, if geographic distribution of data would occur in a multi-site scenario, administrators must be aware of any regulatory implications when the data crosses political boundaries.

Federal Information Processing Standard (FIPS)

Red Hat Ceph Storage uses FIPS validated cryptography modules when run on the latest certified Red Hat Enterprise Linux version.

Enable FIPS mode on Red Hat Enterprise Linux either during system installation or after the installation process.

For container deployments, follow the instructions in the Red Hat Enterprise Linux 9 Security Hardening Guide on Red Hat Documentation.

Reference

For more information, see the following:

- *US Government Standards*

Ceph File System

Ceph File System (CephFS) is a distributed file system that integrates seamlessly with the Ceph storage architecture. By leveraging the Ceph RADOS (Reliable Autonomic Distributed Object Store), CephFS provides a scalable and robust file system interface, adhering to POSIX standards.

“Figure: Ceph cluster integration” on page 82 illustrates integration of Ceph within the Ceph cluster.

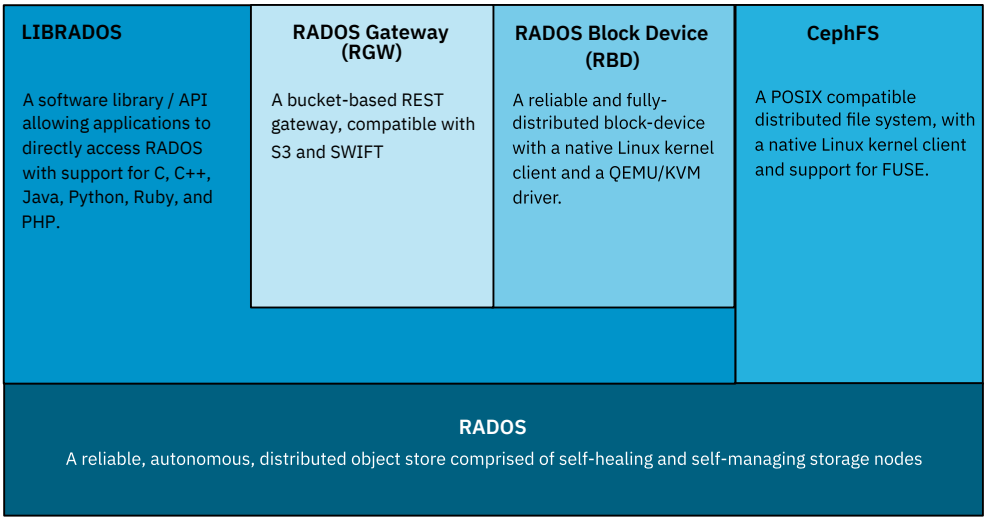


Figure 16: Ceph cluster integration

CephFS-based file sharing offers the same desirable features as other storage services on the Ceph platform, including scalability, high availability, and operational efficiency. The following are key Ceph components that enable CephFS to achieve these benefits:

Object Storage Daemons (OSDs)

OSDs are responsible for storing the actual file data in CephFS. Files and directories are represented as objects within RADOS, with OSDs managing data storage, retrieval, and replication to ensure durability and availability.

Ceph Monitors (MONs)

MONs maintain the cluster map and monitor the health of the Ceph cluster. They track the state of OSDs and other components, ensuring operational stability and fault tolerance. For CephFS, MONs facilitate the mapping of file system operations to the underlying object storage.

Ceph Managers (MGRs)

MGRs handle cluster management, monitoring, and administrative tasks. They provide an interface for managing the Ceph cluster and integrate with external services like OpenStack Manila and Kubernetes CSI for additional functionalities.

Ceph Metadata Servers (MDSs)

MDSs manage CephFS metadata, including file and directory operations, access control, and namespace organization. They coordinate with OSDs to ensure consistent and efficient metadata operations.

Scalability

CephFS scales horizontally with the Ceph cluster, allowing for the addition of OSDs, MONs, and MDSs to increase storage capacity and performance, accommodating large data volumes and high client request rates.

Fault tolerance

Inheriting Ceph fault tolerance, CephFS ensures data durability and availability through replication and erasure coding provided by RADOS, even in the event of hardware failures.

Unified storage platform

CephFS integrates with other Ceph storage services such as block storage, object storage, offering a unified platform that simplifies management and supports diverse storage needs.

Use cases for CephFS

CephFS supports a range of use cases, demonstrating its scalability and flexibility to meet various storage needs. These include:

Native CephFS clients

Clients can mount CephFS shares directly using the kernel client (kcephfs) or FUSE (ceph-fuse), benefiting from Ceph's high throughput and low latency.

Kubernetes storage

CephFS can serve as backend storage for Kubernetes via the Container Storage Interface (CSI) driver, facilitating dynamic provisioning and management of volumes for containerized applications.

Red Hat Ceph Storage Planning

Planning involves considering the supported compatibility, physical configuration, and various storage strategy prerequisites before working with Red Hat Ceph Storage.

Considerations and recommendations

Use this information to understand all considerations and recommendations before you run your Red Hat Ceph Storage cluster.

Red Hat Ceph Storage can be used for different workloads based on a particular business need or set of requirements. Doing the necessary planning before installing a Red Hat Ceph Storage is critical to the success of running a Ceph storage cluster efficiently and achieving the business requirements.

For the latest architectural requirements for a supportable cluster, see Red Hat Ceph Storage: Supported configurations.

> **Note:** For more Red Hat Ceph Storage planning help, contact your Red Hat Support account team.

Basic considerations

Use these basic considerations for running your Red Hat Ceph Storage cluster.

Before using Red Hat Ceph Storage, it is important to develop a proper storage strategy for your data. A storage strategy is a method of storing data that serves a particular use case. If storing volumes and images for a cloud platform, such as OpenStack, is necessary, you can store data on faster Serial Attached SCSI (SAS) drives with Solid State Drives (SSD) for journals. By contrast, if you need to store object data for an S3- or Swift-compliant gateway, you can choose to use something more economical, like traditional Serial Advanced Technology Attachment (SATA) drives.

Both cloud platform and S3- and Swift-compliant gateway scenarios are supported within the same Ceph storage cluster. When a storage cluster needs to support both scenarios, a fast storage strategy to the cloud platform and traditional storage for the object store must be provided.

One of the most important steps in a successful Ceph deployment is identifying a price-to-performance profile suitable for the storage cluster's use case and workload. It is important to choose the right hardware for the use case. For example,

- An IOPS-optimized hardware for a cold storage application can increase hardware costs unnecessarily.
- Capacity-optimized hardware might have a more attractive price point but can lead to slow performance with IOPS-intensive workloads.

For more hardware considerations, see "Minimum hardware considerations" on page 99.

Use cases

Red Hat Ceph Storage use cases can support multiple storage strategies. Use cases, cost versus benefit performance tradeoffs, and data durability are the primary considerations that help develop a sound storage strategy.

Ceph provides massive storage capacity, and it supports numerous use cases, such as:

- The Ceph Block Device client is a leading storage backend for cloud platforms that provides limitless storage for volumes and images with high-performance features like copy-on-write cloning.
- The Ceph Object Gateway client is a leading storage backend for cloud platforms that provides a RESTful S3-compliant and Swift-compliant object storage for objects like audio, bitmap, video, and other data.
- The Ceph File System (CephFS) for traditional file storage.

Cost versus benefit of performance

Faster is better. Bigger is better. High durability is better. However, there is a price for each superlative quality, and a corresponding cost versus benefit tradeoff. Consider the following use cases from a performance perspective:

- SSDs can provide fast storage for relatively small amounts of data and journaling. Storing a database or object index can benefit from a pool of very fast SSDs, but proves too expensive for other data.
- SAS drives with SSD journaling provide fast performance at an economical price for volumes and images.
- SATA drives without SSD journaling provide cheap storage with lower overall performance.

When you create a CRUSH hierarchy of OSDs, you need to consider the use case and an acceptable cost versus performance tradeoff.

Data durability

In large-scale storage clusters, hardware failure is an expectation, not an exception. However, data loss and service interruption remain unacceptable. For this reason, data durability is important. Ceph addresses data durability with multiple replica copies of an object or with erasure coding and multiple coding chunks. Multiple copies or multiple coding chunks present an extra cost versus benefit tradeoff: it is cheaper to store fewer copies or coding chunks, but it can lead to the inability to service write requests in a degraded state. Generally, one object with two more copies, or two coding chunks can allow a storage cluster to service writes in a degraded state while the storage cluster recovers.

Replication stores one or more redundant copies of the data across failure domains in case of a hardware failure. However, redundant copies of data can become expensive at scale. For example, to store 1 petabyte of data with triple replication would require a cluster with at least 3 petabytes of storage capacity.

Erasure coding stores data as data chunks and coding chunks. If there is a lost data chunk, erasure coding can recover the lost data chunk with the remaining data chunks and coding chunks. Erasure coding is substantially more economical than replication. For example, by using erasure coding with eight data chunks and three coding chunks provides the same redundancy as three copies of the data. However, such an encoding scheme uses approximately 1.5-times the initial data stored compared to three-times with replication.

The CRUSH algorithm aids this process by ensuring that Ceph stores extra copies or coding chunks in different locations within the storage cluster. This ensures that the failure of a single storage device or host does not lead to losing all copies or coding chunks necessary to preclude data loss. You can plan a storage strategy with cost versus benefit tradeoffs, and data durability in mind, then present it to a Ceph client as a storage pool.

Important:

- ONLY the data storage pool can use erasure coding. Pools storing service data and bucket indexes use replication.
- Erasure coding is supported only for Ceph Object Gateway bucket data pools and CephFS data pools. Index and metadata pools must use replication. Erasure-coded Ceph Block Device pools are supported only for archival data.
- Ceph’s object copies or coding chunks make RAID solutions obsolete. Do not use RAID, as Ceph already handles data durability. A degraded RAID has a negative impact on performance and recovering data that uses RAID is substantially slower than using deep copies or erasure coding chunks.

Workload considerations

One of the key benefits of a Ceph storage cluster is the ability to support different types of workloads within the same storage cluster using performance domains.

Performance domains allow Red Hat Ceph Storage clusters to support different workloads within the same cluster. Each domain uses hardware optimized for its performance profile. Storage administrators can deploy pools on the appropriate domain to meet application requirements. Selecting correctly sized and optimized servers for these domains is essential for cluster design.

To the Ceph client interface that reads and writes data, a Ceph storage cluster appears as a simple pool where the client stores data. However, the storage cluster performs many complex operations in a manner that is visible to the client interface. Ceph clients and Ceph object storage daemons, referred to as Ceph OSDs, or simply OSDs, both use the Controlled Replication Under Scalable Hashing (CRUSH) algorithm for the storage and retrieval of objects. Ceph OSDs can run in containers within the storage cluster.

A CRUSH map describes a topography of cluster resources, and the map exists both on client hosts as well as Ceph Monitor hosts within the cluster. Ceph clients and Ceph OSDs both use the CRUSH map and the CRUSH algorithm. Ceph clients communicate directly with OSDs, eliminating a centralized object lookup and a potential performance bottleneck. With awareness of the CRUSH map and communication with their peers, OSDs can handle replication, backfilling, and recovery—allowing for dynamic failure recovery.

Ceph uses the CRUSH map to implement failure domains. Ceph also uses the CRUSH map to implement performance domains, which take the performance profile of the underlying hardware into consideration. The CRUSH map describes how Ceph stores data, and it is implemented as a simple hierarchy, specifically an acyclic graph, and a ruleset. The CRUSH map can support multiple hierarchies to separate one type of hardware performance profile from another. Ceph implements performance domains with device "classes".

For example, you can have these performance domains coexisting in the same Red Hat Ceph Storage cluster:

- Hard disk drives (HDDs) are typically appropriate for cost and capacity-focused workloads.
- Throughput-sensitive workloads typically use HDDs with Ceph write journals on solid-state drives (SSDs).
- IOPS-intensive workloads, such as MySQL and MariaDB, often use SSDs, including those with NVMe, SAS, or SATA interfaces.

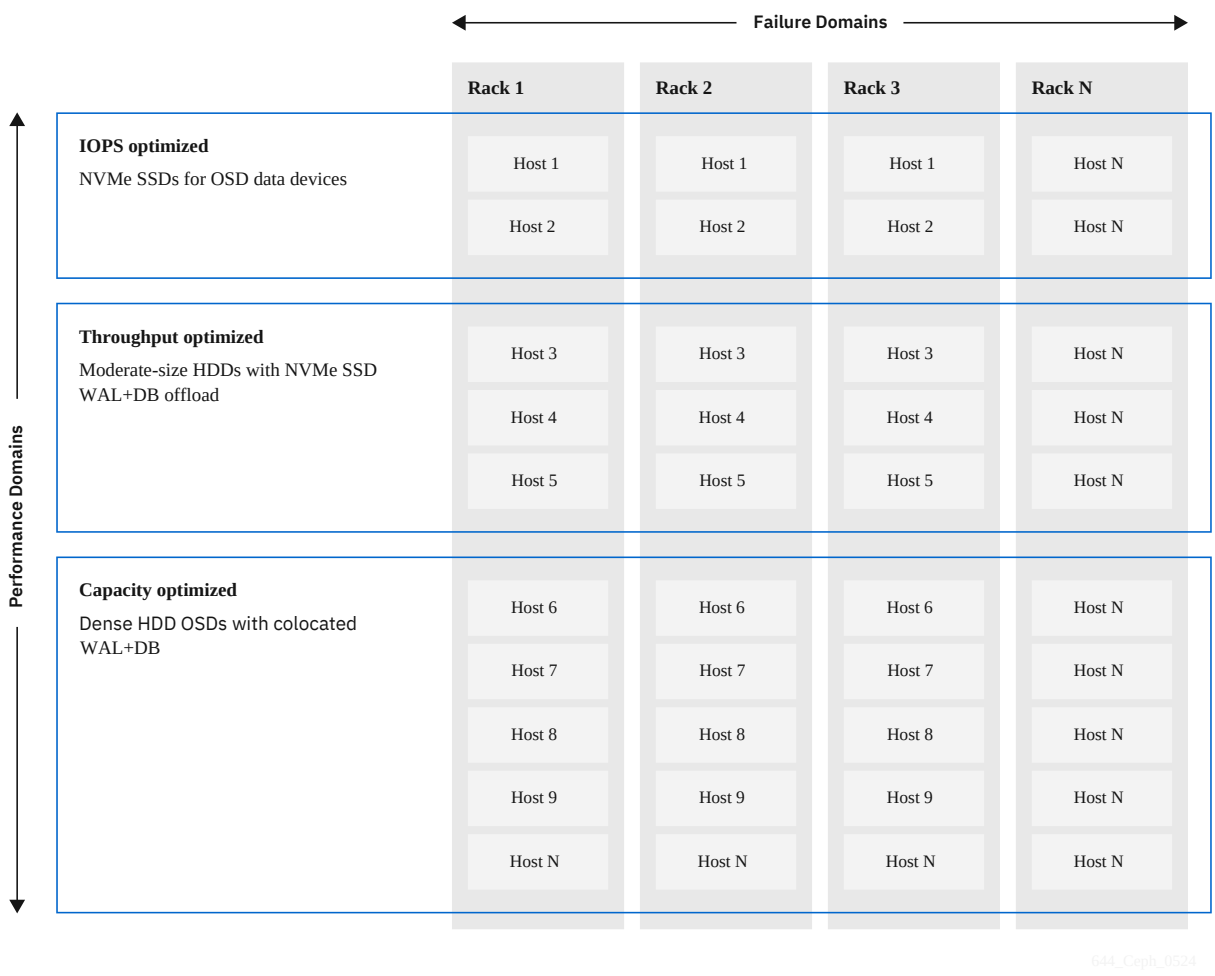


Figure 17: Performance and Failure Domains

Important: Operating System swap must not be enabled. Configuring the operating system to use swap to provide additional virtual memory for daemons is not recommended for modern systems and can result in degraded performance.

Workloads

Red Hat Ceph Storage is optimized for three primary workloads:

Important: Carefully consider the workload being run by Red Hat Ceph Storage clusters **BEFORE** considering what hardware to purchase because it can significantly impact the price and performance of the storage cluster. For example, if the workload is capacity-optimized and the hardware is better suited to a throughput-optimized workload, then hardware will be more expensive than necessary. Conversely, if the workload is throughput-optimized and the hardware is better suited to a capacity-optimized workload, then the storage cluster can suffer from poor performance.

IOPS optimized

Input, output per second (IOPS) optimization deployments are suitable for cloud computing operations, such as running MYSQL or MariaDB instances as virtual machines on OpenStack. IOPS optimized deployments require higher performance storage such as 15k RPM SAS drives and separate SSD journals to handle frequent write operations. Some high IOPS scenarios use all flash storage to improve IOPS and total throughput.
An IOPS-optimized storage cluster has the following properties:

- Lowest cost per IOPS.
- Highest IOPS per GB.
- 99th percentile latency consistency.

Uses for an IOPS-optimized storage cluster are:

- Typically block storage.
- 3x replication for hard disk drives (HDDs) or 2x replication for solid-state drives (SSDs).
- MySQL on OpenStack clouds.

Throughput optimized

Throughput-optimized deployments are suitable for serving up significant amounts of data, such as graphic, audio, and video content. Throughput-optimized deployments require high-bandwidth networking hardware, controllers, and hard disk drives with fast sequential read and write characteristics. If fast data access is a requirement, then use a throughput-optimized storage strategy. Also, if fast write performance is a requirement, using SSDs for journals can substantially improve write performance.
A throughput-optimized storage cluster has the following properties:

- Lowest cost per MBps (throughput).
- Highest MBps per TB.
- Highest MBps per BTU.
- Highest MBps per Watt.
- 97th percentile latency consistency.

Uses for a throughput-optimized storage cluster are:

- Block or object storage.
- 3x replication.
- Active performance storage for video, audio, and images.
- Streaming media, such as 4k video.

Capacity optimized

Capacity-optimized deployments are suitable for storing significant amounts of data as inexpensively as possible. Capacity-optimized deployments typically trade performance for a more attractive price point. For example, capacity-optimized deployments often use slower and less expensive SATA drives and colocate journals rather than using SSDs for journaling.
A cost and capacity-optimized storage cluster has the following properties:

- Lowest cost per TB.
- Lowest BTU per TB.
- Lowest Watts required per TB.

A cost and capacity-optimized storage cluster has the following properties:

- Typically object storage.
- Erasure coding for maximizing usable capacity
- Object archive.
- Video, audio, and image object repositories.

Network considerations for Red Hat Ceph Storage

Network latency and bandwidth constraints can exhaust the IOPS and throughput of a storage cluster long before capacity runs out, so size the network for your workloads.

Storage administrators prefer that a storage cluster recovers as quickly as possible. Carefully consider bandwidth requirements for the storage cluster network, be mindful of network link oversubscription, and separate the intra-cluster traffic from the client-to-cluster traffic. Network performance is increasingly important when considering the use of Solid State Drives (SSDs), including NVMe SSDs, and other high performing storage devices.

Ceph supports a public network and a storage cluster network. The public network handles client traffic and communication with Ceph Monitors. The storage cluster network handles Ceph OSD heartbeats, replication, backfilling, and recovery traffic. Use a **minimum** of a single 10 Gb/s Ethernet link for storage hardware, and and add more 10 Gb/s Ethernet links for connectivity and throughput.

Important:

- On replicated pools, size the bandwidth of the storage cluster network as a multiple of the public network bandwidth. Use the value of the **`osd_pool_default_size`** parameter as the basis for the multiple.
- Use 10 Gb/s Ethernet for Red Hat Ceph Storage deployments in production. A 1 Gb/s Ethernet network is not suitable for production storage clusters.

In the case of a drive failure, replicating 1 TB of data across a 1 Gb/s network takes 3 hours and replicating 10 TB across a 1 Gb/s network takes 30 hours. Using 10 TB is the typical drive configuration. By contrast, with a 10 Gb/s Ethernet network, the replication times would be 20 minutes for 1 TB and 1 hour for 10 TB.

Note: When a Ceph OSD fails, the storage cluster recovers by replicating the data that it contained to other Ceph OSDs.

The failure of a larger domain such as a rack means that the storage cluster uses considerably more bandwidth. When building a storage cluster consisting of multiple racks, which is common for large storage implementations, consider using as much network bandwidth between switches in a "fat tree" design for optimal performance.

A typical 10 Gb/s Ethernet switch has 48 10 Gb/s ports and four 40 Gb/s ports. Use the 40 Gb/s ports on the spine for maximum throughput. Alternatively, consider aggregating unused 10 Gb/s ports with QSFP+ and SFP+ cables into more 40 Gb/s ports to connect to other rack and spine routers.

Use Link Aggregation Control Protocol (LACP) mode 4 to bond network interfaces.

Use jumbo frames with a maximum transmission unit (MTU) of 9000, especially on the backend or cluster network.

Before installing and testing a Red Hat Ceph Storage cluster, verify the network throughput. Most performance-related problems in Ceph begin with a networking issue. Simple network issues such as a kinked or bent Cat-6 cable can result in degraded bandwidth. Use a minimum of 10 Gb/s Ethernet for the front side network. For large clusters, consider using 40 Gb/s Ethernet for the backend or cluster network.

> **Important:** For network optimization, use jumbo frames for a better CPU per bandwidth ratio, and a non-blocking network switch backplane. Red Hat Ceph Storage requires the same MTU value throughout all networking devices in the communication path, end-to-end for both public and cluster networks. Verify that the MTU value is the same on all hosts and networking equipment in the environment before using a Red Hat Ceph Storage cluster in production.

Network ports and protocols used by Red Hat Ceph Storage services

"Network ports and protocols by service for Red Hat Ceph Storage " on page 89 lists the ports, protocols, and traffic direction for each service. Use this information to plan firewall rules and network access for your cluster.

Table 13: Network ports and protocols by service for Red Hat Ceph Storage

Service category	Component	Port	Protocol	Direction	Purpose
Cluster core	MON (Messenger v2)	3300	TCP	Client ↔ Cluster	Cluster discovery and monitor communication
Cluster core	MON (Messenger v1)	6789	TCP	Client ↔ Cluster	Legacy monitor communication
Cluster core	OSD	6800–7300	TCP	Internal	Data I/O, replication, recovery, backfill
Cluster core	MGR	6800–7300	TCP	Internal	Cluster management services
Cluster core	MDS	6800–7300	TCP	Internal and client	CephFS metadata operations
Block storage	Ceph Block Device clients	3300, 6789, 6800–7300	TCP	Client ↔ Cluster	Native Ceph Block Device access
File storage	CephFS native Linux clients	3300, 6789, 6800–7300	TCP	Client ↔ Cluster	Native CephFS access
NFS services	NFS service	2049	TCP	Client ↔ NFS	NFSv3/v4 file access
NFS services	NFS over TLS	2049	TCP/TLS	Client ↔ NFS	Encrypted NFS access
Object storage	Ceph Object Gateway (S3 HTTPS)	443	TCP	Client ↔ Ceph Object Gateway	Secure S3 object storage access
Object storage	Ceph Object Gateway (S3 HTTP)	7480	TCP	Client ↔ Ceph Object Gateway	S3 object storage access (HTTP)
Administration	Ceph Dashboard	8443	TCP	Admin ↔ Cluster	Web administration
Monitoring	Prometheus Exporter	9283	TCP	Monitoring ↔ Cluster	Metrics collection
Monitoring	Grafana	3000	TCP	Admin ↔ Monitoring	Monitoring dashboards
Monitoring	Alertmanager	9093–9094	TCP	Monitoring	Alerting
Monitoring	Node Exporter	9100	TCP	Monitoring	Host metrics

Related information

Configuring multiple public networks to the cluster
Configuring a private network
Configuring a public network

Considerations for using a RAID controller with OSD hosts

Plan RAID controller usage carefully for Ceph OSD hosts to avoid performance issues, data corruption, and unnecessary complexity.

> **Note:** Just a Bunch of Drives (JBOD), also known as Integrated Target (IT) mode, HBAs are recommended for SAS/SATA OSDs. RAID-capable HBAs do not offer significant benefit yet incur additional cost, monitoring demands, operational complexity, and decreased reliability.
> When using OSD drives, avoid using RAID HBAs. Alternatively, the Linux boot volume can be mirrored by using MD software RAID (Linux `mdadm` utility).

If RAID controllers are already in use or required for specific configurations, review the following considerations to ensure optimal performance and reliability:

- If an OSD host has a RAID controller with 1 - 2 Gb of cache installed, enabling the write-back cache might result in increased small I/O write throughput. However, the cache must be nonvolatile.
- Most modern RAID controllers have super capacitors that provide enough power to drain volatile memory to nonvolatile NAND memory during a power-loss event. It is important to understand how a particular controller and its firmware behave after power is restored.
- Some RAID controllers require manual intervention. Hard disks typically advertise to the operating system whether their disk caches can be enabled or disabled by default. However, certain RAID controllers and some firmware do not provide such information. Verify that disk level caches are disabled to avoid file system corruption.
- Create a single RAID 0 volume with write-back for each Ceph OSD data drive with write-back cache enabled.
- If Serial Attached SCSI (SAS) or SATA connected Solid-state Drive (SSD) disks are also present on the RAID controller, then investigate whether the controller and firmware support *pass-through* mode. Enabling *pass-through* mode helps avoid caching logic, and generally results in lower latency for fast media.

Related information

Avoid using RAID and SAN solutions
Server and rack solutions

Tuning considerations for Red Hat Ceph Storage

Production Red Hat Ceph Storage clusters benefit from tuning operating system and BIOS / BMC settings. BMC refers to an embedded service processor used to configure and manage servers independently of the operating system, for example Dell iDRAC or HPE iLO.

When adjusting settings, be sure to apply the adjustments consistently across all hosts in the storage cluster. For more information, contact Red Hat Support.

There are various optimizations to help improve performance and reliability for many Red Hat Ceph Storage deployments. Coordinate with networking and infrastructure teams to align with fleet management. You can use TuneD or the `sysctl` utility for operation system tuning.

The following optimizations improve performance and reliability for many Red Hat Ceph Storage deployments.

TuneD

TuneD manages performance profiles that adjust system settings for specific use cases.

sysctl

The **`sysctl`** utility manages Linux kernel tunables. The following settings have shown performance and reliability benefits. Evaluate these settings against your environment and automation.

BIOS and BMC settings

Available settings and their possible values vary by server chassis manufacturer and model. Use the following recommendations as guidelines, and adapt them to your hardware and environment.

Thermal and performance profiles

Many modern server chassis include performance and thermal profile settings that affect CPU core management. By default, these are set to `Balanced`. Select the `Performance` or equivalent profile to improve throughput and reduce latency. This change may increase power consumption.

Cooling

Server chassis BMCs often adjust fan speeds dynamically based on detected hardware. The default settings may not provide optimal cooling. Select **Enhanced Cooling** or a medium fan offset for more effective cooling of storage devices, with the tradeoff of increased noise.

Hyper-Threading

Red Hat Ceph Storage is an integer workload that benefits from Hyper-Threading. Each physical CPU core appears to the operating system as two virtual cores (vcores) or *threads*. Keep Hyper-Threading enabled for most deployments.

Important: Operating System swap must not be enabled. Configuring the operating system to use swap to provide additional virtual memory for daemons is not recommended for modern systems and can result in degraded performance.

Operating system tuning with TuneD

TuneD manages performance profiles that adjust system settings for specific use cases.

1. Install TuneD.

```
# dnf install -y tuned
# tuned-adm active
Current active profile: balanced
```

2. Activate the `network-latency` profile.
   The **`network-latency`** profile prevents deep CPU C-state transitions. These transitions are slow and can cause performance degradation or packet loss.

```
# tuned-adm profile network-latency
# tuned-adm active
Current active profile: network-latency
```

3. Verify the system C-states.
   On the **idle stats** tab, CPUs should report being in the following states most of the time: `C0` , `C1`, or `C1E`. The CPU should never be in the `C6` state. This improves throughput and latency, but may increase power consumption and heat generation.

4. After verifying that TuneD is running with the correct profile, install the `powertop` utility to verify that TuneD is running correctly.

```
# dnf install -y powertop
# powertop
```

A rolling reboot is recommended to apply all tunings.

Note: Do not modify `/etc/sysctl.conf` or `/etc/sysctl.d/99-sysctl.conf` . Audit existing `/etc/sysctl.d` files to avoid overrides.

Operating system tuning with `sysctl`

The **`sysctl`** utility manages Linux kernel tunables. The following settings have shown performance and reliability benefits. Evaluate these settings against your environment and automation.

1. Create the `/etc/sysctl.d/99-ceph.conf` file with the following contents:

```
# Memory tuning
vm.min_free_kbytes = 4194304
vm.vfs_cache_pressure = 10
vm.swappiness = 1
vm.zone_reclaim_mode = 0
vm.dirty_ratio = 80
vm.dirty_background_ratio = 3

# I/O performance
fs.aio-max-nr = 50000000

# Network performance
net.ipv4.tcp_timestamps = 0
net.ipv4.tcp_sack = 1
net.ipv4.tcp_dsack = 0
net.ipv4.tcp_fack = 0
net.ipv4.tcp_congestion_control = bbr
net.core.default_qdisc = fq
net.ipv4.tcp_mtu_probing = 1
net.ipv4.tcp_syncookies = 1
net.core.somaxconn = 5000
net.ipv4.conf.all.send_redirects = 0
net.ipv4.conf.all.accept_source_route = 0
net.ipv4.tcp_fin_timeout = 20
net.ipv4.tcp_tw_reuse = 1
net.ipv4.tcp_window_scaling = 1
net.ipv4.tcp_low_latency = 1
net.ipv4.tcp_adv_win_scale = 1
net.ipv4.tcp_slow_start_after_idle = 0

# High-impact memory and backlog tuning
net.core.netdev_max_backlog = 250000
net.ipv4.tcp_max_tw_buckets = 2000000
net.ipv4.tcp_max_syn_backlog = 100000
net.ipv4.udp_mem = 4096 87380 33554432
net.ipv4.tcp_rmem = 4096 87380 33554432
net.ipv4.tcp_wmem = 4096 65536 33554432
net.core.rmem_max = 67108864
net.core.wmem_max = 67108864

# Connection tracking
net.netfilter.nf_conntrack_max = 1048576
net.nf_conntrack_max = 1048576
```

2. Complete the `nf_conntrack` table expansion by running the following commands:

```
echo 131072 > /sys/module/nf_conntrack/parameters/hashsize
echo "options nf_conntrack hashsize=131072" >/etc/modprobe.d/nf_conntrack.conf
```

   Ensure that your fleet automation persists these files by using Ansible, or a similar fleet configuration management tool.

A rolling reboot is recommended to apply all tunings.

> **Note:** Do not modify `/etc/sysctl.conf` or `/etc/sysctl.d/99-sysctl.conf`. Audit existing `/etc/sysctl.d` files to avoid overrides.

Volatile HDD cache

Modern hard drives include up to 256 MB of cache. This cache is volatile and lacks Power Loss Protection (PLP). If power is lost, unwritten data may be corrupted. When volatile write cache is enabled, the Linux kernel flushes every write synchronously to media. This prevents data loss but often reduces OSD performance, especially average and tail write latencies.

You can improve performance by disabling the volatile cache. This cache is different from controller-level caches, such as those on RAID HBAs. The impact varies by drive and controller. Test changes in a lab or pre-GA environment before production deployment. Monitor `hdparm` and `sdparm` latency before and after changes by using your cluster dashboard.

“Figure: OSD Latencies dashboard output” on page 93 an example OSD latency output on the dashboard.

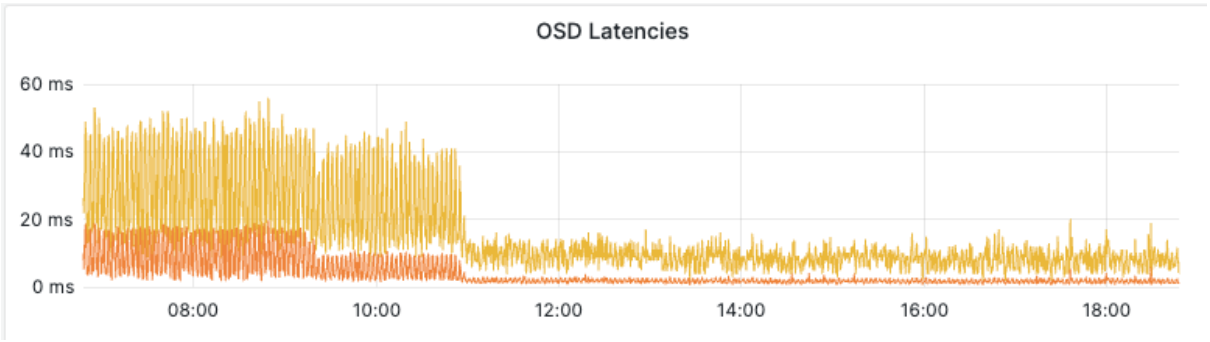


*Figure 18: **OSD Latencies** dashboard output*

> **Note:** Persistence across reboot depends on drive firmware.

Use the `hdparm` and `sdparm` commands for live testing of cache settings.

Example commands for disabling the volatile cache

```
hdparm -W 0 /dev/sdXX   # SATA drives
sdparm --clear=WCE /dev/sdXX   # SAS drives
```

Example commands for enabling the volatile cache

```
hdparm -W 1 /dev/sdXX   # SATA drives
sdparm --set=WCE /dev/sdXX   # SAS drives
```

To disable volatile write cache reliably at boot, create a `udev` rule.

The following is an example for all SAS drives reporting to the manufacturer SPINNERSRUS.

```
cat <<EOF >/etc/udev/rules.d/90-wce.rules SUBSYSTEM=="block", ENV{SCSI_VENDOR}
=="SPINNERSRUS", ACTION=="add|change", RUN+="/usr/bin/sdparm --clear=WCE /dev/%k" EOF
```

To apply the changes immediately, reload udev.

```
udevadm control --reload
```

Colocation

Use this information to understand how colocation works and its advantages.

You can colocate containerized Ceph daemons on the same host.

The following are the advantages of colocating some of the services that Ceph offers:

- Significant improvement in total cost of ownership (TCO) at small scale
- Reduction from six hosts to three for the minimum configuration
- Easier upgrade
- Better resource isolation

How colocation works

With the help of the `cephadm` orchestrator, you can colocate one daemon from the following list with one or more OSD daemons (`ceph-osd`):

- Ceph Monitor (`ceph-mon`) and Ceph Manager (`ceph-mgr`) daemons
- RBD Mirror (`rbd-mirror`)
- Observability Stack (Grafana)

Additionally, for Ceph Object Gateway (`radosgw`) and Ceph File System (`ceph-mds`), you can colocate either with an OSD daemon plus a daemon from the previous list, excluding RBD mirror.

Note:

- Colocating two of the same kind of daemons on a specific node is not supported.
- Because `ceph-mon` and `ceph-mgr` work together closely they do not count as two separate daemons for the purposes of colocation.
- Colocate the Ceph Object Gateway with Ceph OSD containers to increase performance.

Colocation examples

The following examples demonstrate minimum cluster sizes that comply with the listed rules.

Example 1

"Colocated daemons: example 1" on page 94 and "Figure: Colocated daemons: example 1" on page 95 provide an example of colocated daemons with the following specifications:

Media

Full flash systems (SSDs)

Use case

Block (Ceph Block Device) and File (CephFS), or Object (Ceph Object Gateway)

Number of nodes

3

Replication scheme

2

Table 14: Colocated daemons: example 1

Host	Daemon	Daemon	Daemon
host1	OSD	Monitor/Manager	Grafana
host2	OSD	Monitor/Manager	RGW or CephFS
host3	OSD	Monitor/Manager	RGW or CaphFS

Note: The minimum size for a storage cluster with three replicas is four nodes. Similarly, the size of a storage cluster with two replicas is a three node cluster. It is a requirement to have a certain number of nodes for the replication factor with an extra node in the cluster to avoid extended periods with the cluster in a degraded state.

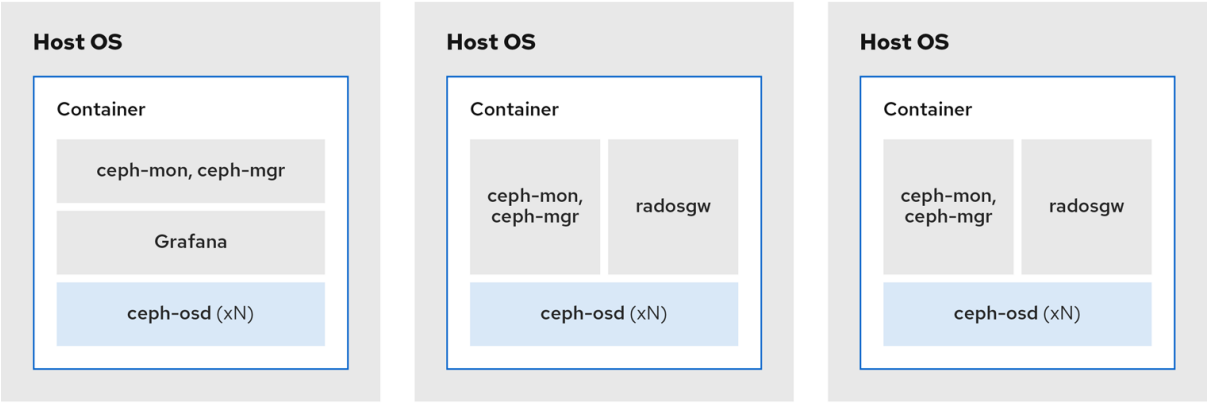


Figure 19: Colocated daemons: example 1

Example 2

“Colocated daemons: example 2” on page 95 and “Figure: Colocated daemons: example 2” on page 96 provide an example of colocated daemons with the following specifications:

Media

Full flash systems (SSDs) or spinning devices (HDDs)

Use case

Block (Ceph Block Device), File (CephFS), and Object (Ceph Object Gateway)

Number of nodes

4

Replication scheme

3

Table 15: Colocated daemons: example 2

Host	Daemon	Daemon	Daemon
host1	OSD	Grafana	CephFS
host2	OSD	Monitor/Manager	RGW
host3	OSD	Monitor/Manager	RGW
host4	OSD	Monitor/Manager	CephFS



Figure 20: Colocated daemons: example 2

The differences between colocation and non-colocated daemons

"Figure: Example of colocated daemons" on page 97 and "Figure: Non-colocated Daemons" on page 98 show the differences between storage clusters with colocated and non-colocated daemons.



Figure 21: Example of colocated daemons



Figure 22: Non-colocated Daemons

Operating system requirements

Before running Red Hat Ceph Storage, be sure to comply with all operating system requirements listed here.

- Operating System swap must not be enabled.
- Red Hat Enterprise Linux entitlements are included in the Red Hat Ceph Storage subscription.
- For the latest supported Red Hat Enterprise Linux versions, see Compatibility matrix.
- Red Hat Ceph Storage is supported on container-based deployments only.
- Use the same architecture and deployment type across all nodes.
  For example, do not use a mixture of nodes with both AMD64 and Intel 64 architectures or a mixture of nodes with container-based deployments.

> **Important:** All nodes in the cluster must use the same CPU architecture and deployment type. Mixing CPU architectures or deployment types across nodes is not supported.

- By default, SELinux is set to `Enforcing` mode and the `ceph-selinux` packages are installed. For details, see Using SELinux for your OS version, on the Red Hat Customer Portal.
- Red Hat Ceph Storage 9 and later requires GCC 13 or later in the host operating system and Ceph container environment.

> **Important:** Earlier GCC versions are not supported and can result in unexpected behavior or service failures

Minimum hardware considerations

Before running Red Hat Ceph Storage, be sure to comply with all minimum hardware requirements listed here.

Red Hat Ceph Storage can run on non-proprietary commodity hardware. Small production clusters and development clusters can run without performance optimization with modest hardware.

> **Note:** Disk space requirements are based on the Ceph daemons' default path under `/var/lib/ceph/` directory.

For more information about the Red Hat Ceph Storage internal components and the strategies around those components, see Storage strategies.

> **Important:** Hardware accelerated compression in Ceph Object Gateway requires RHEL 9.4 on a Sapphire or Emerald Rapids Xeon CPU (or newer) with QAT devices. For more information, see Which 4th Gen Intel® Xeon® Scalable Processors Do Support Intel® QuickAssist Technology (Intel® QAT)? on Intel Product Support.

Red Hat Ceph Storage is supported on Intel and AMD x86-64 microprocessors.

- Red Hat Ceph Storage 9 introduces full support for clusters built with ARM (aarch64).

Use the following information for each process container and their minimum recommended requirements for various components.

For the latest architectural requirements for a supportable cluster, see Red Hat Ceph Storage: Supported configurations .

`ceph-osd-container`

- 1 × AMD64 or Intel 64 CPU vcore/hyperthread per OSD container.
- Minimum of 5 GB of RAM per OSD container.
- 1 × OS disk per host.
- 1 × storage drive per OSD container.

  > **Note:** The storage drive cannot be shared with the OS disk.

- Optional WAL and DB offload for HDD OSDs.
  - Red Hat recommended.
  - 1 × SSD or NVMe or Optane partition or lvm per daemon.
  - Sizing is 4% of `block.data` for BlueStore for object, file, and mixed workloads and 1% of `block.data` for the BlueStore for Block Device.
- Minimum of 2 × 10 GB Ethernet NICs network connection.

`ceph-mon-container`

- 1 × AMD64 or Intel 64 CPU vcore/hyperthread per `mon-container`.
- 3 GB of RAM per `mon-container`.
- 10 GB of disk space per `mon-container`.

  > **Note:** 50 GB disk space is recommended.

- Prometheus requires 20 GB to 50 GB under `/var/lib/ceph/` directory created as a separate file system to protect the contents under `/var/` directory.

`ceph-mgr-container`

- 1 × AMD64 or Intel 64 CPU vcore/hyperthread per `mgr-container`.
- 3 GB RAM per `mgr-container`.
- 2 × 1 GB Ethernet NICs network connection.

  > **Note:** 10 GB network is recommended.

`ceph-radosgw-container`

- 1 × AMD64 or Intel 64 CPU vcore/hyperthread per `radosgw-container`.
- 1 GB RAM per daemon.
- 5 GB disk space per daemon.
- 1 × 1 GB Ethernet NICs network connection.

`ceph-mds-container`

- 1 × AMD64 or Intel 64 CPU vcore/hyperthread per `mds-container`.
  This number is highly dependent on the configurable MDS cache size. The RAM requirement is typically twice as much as the amount set in the `mds_cache_memory_limit` configuration setting.

  > **Note:** This is the memory for the daemon, not the overall system memory.

- 3 GB RAM per `mds-container`.
- 2 GB disk space per `mds-container`, plus considering any additional space required for possible debug logging.

  > **Note:** Minimum of 20 GiB of available space in the CephFS metadata pool.

- Minimum of 2 × 1 GB Ethernet NICs network connection.

  > **Note:**
  > - 10 GB network is recommended.
  > - This is the same network as the OSD containers. If you have a 10 GB network on your OSDs you should use the same on your MDS so that the MDS is not disadvantaged when it comes to latency.

Hardware

Get a high level guidance on selecting hardware for use with Red Hat Ceph Storage.

Software-defined storage presents many advantages to organizations that seek scale-out solutions to meet demanding applications and escalating storage needs. With a proven methodology and extensive testing with multiple vendors, Red Hat simplifies the process of selecting hardware to meet the demands of any environment. Importantly, the guidelines and example systems that are listed in this document are not a substitute for quantifying the impact of production workloads on sample systems.

Red Hat Ceph Storage significantly reduces the cost of storing enterprise data and helps organizations manage exponential data growth. The software is a robust and modern petabyte-scale storage platform for public or private cloud deployments.

Red Hat offers mature interfaces for enterprise block and object storage These interfaces make Red Hat Ceph Storage an optimal solution for active archive, rich media, and cloud infrastructure workloads characterized by tenant-agnostic OpenStack® environments[1]. Delivered as a unified, software-defined, scale-out storage platform, Red Hat Ceph Storage allows businesses to focus on improving application innovation and availability by offering capabilities such as:

- Scaling to hundreds of petabytes[2].
- No single point of failure in the cluster.
- Lower capital expenses (CapEx) by running on commodity server hardware.
- Lower operational expenses (OpEx) with self-managing and self-healing properties.

Red Hat Ceph Storage can run on myriad industry-standard hardware configurations to satisfy diverse needs. To simplify and accelerate the cluster design process, Red Hat conducts extensive performance and suitability testing with participating hardware vendors. This testing allows evaluation of selected hardware under load and generates essential performance and sizing data for diverse workloads ultimately simplifying Ceph hardware selection. Multiple hardware vendors now provide server and rack-level solutions optimized for Red Hat Ceph Storage deployments with IOPS, throughput, and cost and capacity-optimized solutions as available options.

General principles for selecting hardware

As a storage administrator, you must select the appropriate hardware for running a production Red Hat Ceph Storage cluster. When selecting hardware for Red Hat Ceph Storage, review these following general principles. These principles will help save time, avoid common mistakes, save money and achieve a more effective solution.

For a list of supported configurations, see .

Prerequisites

- A planned use for Red Hat Ceph Storage.
- Linux System Administration Advance level with Red Hat Enterprise Linux certification.
- Storage administrator with Ceph Certification.

Identify performance use case

One of the most important steps in a successful Ceph deployment is identifying a price-to-performance profile suitable for the cluster's use case and workload. It is important to choose the correct hardware for the use case. For example, choosing IOPS-optimized hardware for a cloud storage application increases hardware costs unnecessarily. This is as opposed to selecting capacity-optimized hardware for its more attractive price point in an IOPS-intensive workload likely leads to unhappy users complaining about slow performance.

The following lists the primary use cases for Ceph.

IOPS optimized

IOPS optimized deployments are suitable for cloud computing operations, such as running MYSQL or MariaDB instances as virtual machines on OpenStack. IOPS optimized deployments require higher

1. Ceph is and has been the leading storage for OpenStack according to several semi-annual OpenStack user surveys.
2. See Yahoo Cloud Object Store - Object Storage at Exabyte Scale for details.

performance storage such as 15k RPM SAS drives and separate SSD journals to handle frequent write operations. Some high IOPS scenarios use all flash storage to improve IOPS and total throughput.

Throughput optimized

Throughput-optimized deployments are suitable for serving up significant amounts of data, such as graphic, audio and video content. Throughput-optimized deployments require networking hardware, controllers, and hard disk drives with acceptable total throughput characteristics. In cases where write performance is a requirement, SSD journals substantially improve write performance.

Capacity optimized

Capacity-optimized deployments are suitable for storing significant amounts of data as inexpensively as possible. Capacity-optimized deployments typically trade performance for a more attractive price point. For example, capacity-optimized deployments often use slower and less expensive SATA drives and colocate journals rather than using SSDs for journaling.

Use the information that is detailed within "General principles for selecting hardware" on page 101 and its subsections provides examples of Red Hat tested hardware suitable for these use cases.

Consider storage density

Hardware planning should include distributing Ceph daemons and other processes that use Ceph across many hosts to maintain high availability in the event of hardware faults. Balance storage density considerations with the need to rebalance the cluster in the event of hardware faults.

A common hardware selection mistake is to use very high storage density in small clusters, which can overload networking during backfill and recovery operations.

Identical hardware configuration

Create pools and define CRUSH hierarchies such that the OSD hardware within the pool is identical. Using the same hardware within a pool provides a consistent performance profile, simplifies provisioning, and streamlines troubleshooting.

Identical hardware configuration includes the following:

- Same controller.
- Same drive size.
- Same RPMs.
- Same seek times.
- Same I/O.
- Same network throughput.
- Same journal configuration.

Network considerations for Red Hat Ceph Storage

Carefully consider bandwidth requirements for the storage cluster network. An important aspect of a cloud storage solution is that storage clusters can run out of IOPS due to network latency, and other factors. Also, the storage cluster can run out of throughput due to bandwidth constraints long before the storage clusters run out of storage capacity. This means that the network hardware configuration must support the chosen workloads to meet price versus performance requirements.

Storage administrators prefer that a storage cluster recovers as quickly as possible. Carefully consider bandwidth requirements for the storage cluster network, be mindful of network link oversubscription, and segregate the intra-cluster traffic from the client-to-cluster traffic. Also consider that network performance is increasingly important when considering the use of Solid State Disks (SSD), flash, NVMe, and other high performing storage devices.

Ceph supports a public network and a storage cluster network. The public network handles client traffic and communication with Ceph Monitors. The storage cluster network handles Ceph OSD heartbeats, replication, backfilling, and recovery traffic. At a *minimum*, a single 10 GB Ethernet link should be used for storage hardware, and you can add additional 10 GB Ethernet links for connectivity and throughput.

Important: Red Hat recommends allocating bandwidth to the storage cluster network, such that it is a multiple of the public network using the `osd_pool_default_size` as the basis for the multiple on replicated pools. Red Hat also recommends running the public and storage cluster networks on separate network cards.
Red Hat recommends using 10 GB Ethernet for Red Hat Ceph Storage deployments in production. A 1 GB Ethernet network is not suitable for production storage clusters.

In the case of a drive failure, replicating 1 TB of data across a 1 GB Ethernet network takes 3 hours, and 3 TB takes 9 hours. Using 3 TB is the typical drive configuration. By contrast, with a 10 GB Ethernet network, the replication times would be 20 minutes and 1 hour. Remember that when a Ceph OSD fails, the storage cluster will recover by replicating the data it contained to other Ceph OSDs within the pool.

The failure of a larger domain such as a rack means that the storage cluster utilizes considerably more bandwidth. When building a storage cluster consisting of multiple racks, which is common for large storage implementations, consider utilizing as much network bandwidth between switches in a "fat tree" design for optimal performance. A typical 10 GB Ethernet switch has 48 10 GB ports and four 40 GB ports. Use the 40 GB ports on the spine for maximum throughput. Alternatively, consider aggregating unused 10 GB ports with QSFP+ and SFP+ cables into more 40 GB ports to connect to other rack and spine routers. Also, consider using LACP mode 4 to bond network interfaces. Additionally, use jumbo frames, with a maximum transmission unit (MTU) of 9000, especially on the backend or cluster network.

Before installing and testing a Red Hat Ceph Storage cluster, verify the network throughput. Most performance-related problems in Ceph usually begin with a networking issue. Simple network issues like a kinked or bent Cat-6 cable could result in degraded bandwidth. Use a minimum of 10 GB ethernet for the front side network. For large clusters, consider using 40 GB ethernet for the backend or cluster network.

Important: For network optimization, Red Hat recommends using jumbo frames for a better CPU per bandwidth ratio, and a non-blocking network switch back-plane. Red Hat Ceph Storage requires the same MTU value throughout all networking devices in the communication path, end-to-end for both public and cluster networks. Verify that the MTU value is the same on all hosts and networking equipment in the environment before using a Red Hat Ceph Storage cluster in production.

Avoid using RAID and SAN solutions

Ceph can replicate or erasure code objects. RAID duplicates this functionality on the block level and reduces available capacity. Consequently, RAID is an unnecessary expense. Additionally, a degraded RAID will have a **negative** impact on performance.

Important: Red Hat does not support SAN as a storage backend for OSDs.

Important: Red Hat recommends that each hard drive be exported separately from the RAID controller as a single volume with write-back caching enabled.

This requires a battery-backed, or a non-volatile flash memory device on the storage controller. It is important to make sure the battery is working, as most controllers will disable write-back caching if the memory on the controller can be lost as a result of a power failure. Periodically, check the batteries and replace them if necessary, as they do degrade over time. See the storage controller vendor's documentation for details. Typically, the storage controller vendor provides storage management utilities to monitor and adjust the storage controller configuration without any downtime.

Using Just a Bunch of Drives (JBOD) in independent drive mode with Ceph is supported when using all Solid State Drives (SSDs), or for configurations with high numbers of drives per controller. For example, 60 drives attached to one controller. In this scenario, the write-back caching can become a source of I/O contention. Since JBOD disables write-back caching, it is ideal in this scenario. One advantage of using JBOD mode is the ease of adding or replacing drives and then exposing the drive to the operating system immediately after it is physically plugged in.

Summary of common mistakes when selecting hardware

Use the examples provided within this documentation of Red Hat tested configurations for different workloads to avoid some of the most common hardware selection mistakes.

These mistakes include the following:

- Repurposing underpowered legacy hardware for use with Ceph.
- Using dissimilar hardware in the same pool.
- Using 1 Gbps networks instead of 10Gbps or greater.
- Neglecting to setup both public and cluster networks.
- Using RAID instead of JBOD.
- Selecting drives on a price basis without regard to performance or throughput.
- Journaling on OSD data drives when the use case calls for an SSD journal.
- Having a disk controller with insufficient throughput characteristics.

Optimize workload performance domains

One of the key benefits of Ceph storage is the ability to support different types of workloads within the same cluster using Ceph performance domains. Dramatically different hardware configurations can be associated with each performance domain. Ceph system administrators can deploy storage pools on the appropriate performance domain, providing applications with storage tailored to specific performance and cost profiles. Selecting appropriately sized and optimized servers for these performance domains is an essential aspect of designing a Red Hat Ceph Storage cluster.

The following lists provide the criteria Red Hat uses to identify optimal Red Hat Ceph Storage cluster configurations on storage servers. These categories are provided as general guidelines for hardware purchases and configuration decisions, and can be adjusted to satisfy unique workload blends. Actual hardware configurations chosen will vary depending on specific workload mix and vendor capabilities.

IOPS optimized

An IOPS-optimized storage cluster typically has the following properties:

- Lowest cost per IOPS.
- Highest IOPS per GB.
- 99th percentile latency consistency.

Typically uses for an IOPS-optimized storage cluster are:

- Typically block storage.
- 3x replication for hard disk drives (HDDs) or 2x replication for solid state drives (SSDs).
- MySQL on OpenStack clouds.

Throughput optimized

A throughput-optimized storage cluster typically has the following properties:

- Lowest cost per MBps (throughput).
- Highest MBps per TB.
- Highest MBps per BTU.
- Highest MBps per Watt.
- 97th percentile latency consistency.

Typically uses for an throughput-optimized storage cluster are:

- Block or object storage.
- 3x replication.

- Active performance storage for video, audio, and images.
- Streaming media.

Cost and capacity optimized

A cost- and capacity-optimized storage cluster typically has the following properties:

- Lowest cost per TB.
- Lowest BTU per TB.
- Lowest Watts required per TB.

Typically uses for an cost- and capacity-optimized storage cluster are:

- Typically object storage.
- Erasure coding common for maximizing usable capacity
- Object archive.
- Video, audio, and image object repositories.

How performance domains work

To the Ceph client interface that reads and writes data, a Ceph storage cluster appears as a simple pool where the client stores data. However, the storage cluster performs many complex operations in a manner that is completely transparent to the client interface. Ceph clients and Ceph object storage daemons (Ceph OSDs, or simply OSDs) both use the controlled replication under scalable hashing (CRUSH) algorithm for storage and retrieval of objects. OSDs run on OSD hosts—the storage servers within the cluster.

A CRUSH map describes a topography of cluster resources, and the map exists both on client nodes as well as Ceph Monitor (MON) nodes within the cluster. Ceph clients and Ceph OSDs both use the CRUSH map and the CRUSH algorithm. Ceph clients communicate directly with OSDs, eliminating a centralized object lookup and a potential performance bottleneck. With awareness of the CRUSH map and communication with their peers, OSDs can handle replication, backfilling, and recovery—allowing for dynamic failure recovery.

Ceph uses the CRUSH map to implement failure domains. Ceph also uses the CRUSH map to implement performance domains, which simply take the performance profile of the underlying hardware into consideration. The CRUSH map describes how Ceph stores data, and it is implemented as a simple hierarchy (acyclic graph) and a ruleset. The CRUSH map can support multiple hierarchies to separate one type of hardware performance profile from another.

The following examples describe performance domains.

- Hard disk drives (HDDs) are typically appropriate for cost- and capacity-focused workloads.
- Throughput-sensitive workloads typically use HDDs with Ceph write journals on solid state drives (SSDs).
- IOPS-intensive workloads such as MySQL and MariaDB often use SSDs.

All of these performance domains can coexist in a Ceph storage cluster.

Server and rack solutions

Ceph provides both optimized server-level and rack-level solution SKUs. Hardware vendors provide both optimized server-level and rack-level solution SKUs. Validated through joint testing with Red Hat, these solutions offer predictable price-to-performance ratios for Ceph deployments, with a convenient modular approach to expand Ceph storage for specific workloads.

Many hardware vendors now offer both Ceph-optimized servers and rack-level solutions that are designed for distinct workload profiles. Red Hat works with multiple storage server vendors to test and evaluate specific cluster options for different cluster sizes and workload profiles. This work is to simplify the hardware selection process and reduce risk for organizations. Red Hat’s exacting methodology combines performance testing with proven guidance for a broad range of cluster capabilities and sizes.

With appropriate storage servers and rack-level solutions, Red Hat Ceph Storage can provide storage pools that serve various workloads from throughput-sensitive and cost and capacity-focused workloads to emerging IOPS intensive workloads.

IOPS-optimized solutions

With the growing use of flash storage, organizations increasingly host IOPS-intensive workloads on Ceph storage clusters enabling emulation of high-performance public cloud solutions with private cloud storage. These workloads commonly involve structured data from MySQL-, MariaDB-, or PostgreSQL-based applications.

"Typical IOPS-optimized server configuration" on page 106 lists a typical server configuration.

Table 16: Typical IOPS-optimized server configuration

Element	Specification
CPU	5 physical cores (10 hyperthreads) per NVMe SSD
RAM	24 GB baseline, plus 6 GB per OSD
Networking	10-Gigabit Ethernet (GbE) per 2 OSDs
OSD media	High performance enterprise NVMe SSDs, read-intensive or mixed-use
OSDs	One OSD per SSD for most deployments
BlueStore WAL/DB	High performance enterprise NVMe SSD, colocated on the OSD
Controller	Native PCIe bus

"Solutions SKUs for IOPS-optimized Ceph workloads, by cluster size" on page 106 provides SKU solutions for IOPs-optimized Ceph workloads, by cluster size.

For more information, see Supermicro® storage products.

Table 17: Solutions SKUs for IOPS-optimized Ceph workloads, by cluster size

Vendor	Small (250 TB)	Medium (1 PB)	Large (2 PB+)
SuperMicro [1]	SSG-122B, SSG-222B, SSG-121, SSG-110P-NTR10	SSG-122B-NE316R, SSG-110P-NTR10	SSG-222B-NE3X24R

Throughput-optimized solutions

Throughput-optimized Ceph solutions are usually centered around semi-structured or unstructured data. Large-block sequential I/O is typical.

"Typical throughput-optimized server configuration" on page 106 lists a typical server configuration.

Table 18: Typical throughput-optimized server configuration

Element	Specification
CPU	– 2 physical cores (4 hyperthreads) per OSD – 1 physical core (2 hyperthreads) per HDD
RAM	24 GB baseline, plus 6 GB per OSD
Networking	Redundant 25GE or 100GE links
OSD media	7,200 RPM Enterprise HDDs or NVMe SSDs
OSDs	One per storage drive
BlueStore WAL/DB	Offloaded to shared high-performance NVMe SSDs
Controller	RAID HBAs are not recommended; for SAS/SATA drives a JBOD style HBA is preferred.

Several vendors provide pre-configured server and rack-level solutions for throughput-optimized Ceph workloads. Extensive testing and evaluation of servers from Supermicro and Quanta Cloud Technologies (QCT) has been conducted.

This table describes individual OSD servers.

Table 19: Individual OSD servers			
Vendor	**Small (250 TB)**	**Medium (1 PB)**	**Large (2 PB+)**
SuperMicro	SSG-110P-NTR10, SYS-112B-WR, SSG-122B-NE316R	SSG-122B-NE316R	SSG-222B-NE3X24R
QCT[1]	QxStor RCT-200	QxStor RCT-400	QxStor RCT-400

[1]QxStor Ceph Storage Open Source Edition

This table describes other servers configurable for throughput-optimized Ceph OSD workloads.

Table 20: Other configurable servers for throughput-optimized Ceph OSD workloads			
Vendor	**Small (250 TB)**	**Medium (1 PB)**	**Large (2 PB+)**
Dell	R670	R670, R770	R7715
Cisco	C220 M8	C240 M8	C245 M8
Lenovo	SR645 V3	SR645 V3	SR665 V3

Cost and capacity-optimized solutions

Cost- and capacity-optimized solutions typically focus on higher capacity, or longer archival scenarios. Data can be either semi-structured or unstructured. Workloads include media archives, big data analytics archives, and machine image backups. Large-block sequential I/O is typical.

"Typical cost- and capacity-optimized server configuration" on page 107 lists a typical configuration.

Table 21: Typical cost- and capacity-optimized server configuration	
Element	**Specification**
CPU	2 physical cores (4 hyperthreads) per OSD
RAM	24 GB baseline, plus 5 GB per OSD
Networking	Redundant 10 GE, 25 GE, or 100 GE links
OSD media	7,200 RPM Enterprise HDDs or QLC-class SSDs
OSDs	One per storage drive
BlueStore WAL/DB	Colocated on the storage drive
Controller	RAID HBAs are not recommended; for SAS/SATA drives a JBOD style HBA is preferred.

Table 22: Other configurable servers for cost- and capacity-optimized workloads			
Vendor	**Small (250 TB)**	**Medium (1 PB)**	**Large (2 PB+)**
Dell	R470	R570	R7715
Cisco	C225 M8	C245 M8	C245M8
Lenovo	SR630 V3	SR645 V3	SR665 V3

For more information, see the following related information:

- *Deploying MySQL Databases on Red Hat Ceph Storage*
- *Intel® Data Center Blocks for Cloud – Red Hat OpenStack Platform with Red Hat Ceph Storage*

Recommended minimum hardware requirements for the Red Hat Ceph Storage Dashboard

The Red Hat Ceph Storage Dashboard has minimum hardware requirements.

The following are minimum requirements for using the Red Hat Ceph Storage Dashboard:

- 4 core processor at 2.5 GHz or higher
- 8 GB RAM
- 50 GB hard disk drive
- 1 Gigabit Ethernet network interface

Related information

High-level monitoring

Storage strategies

A storage strategy is a method of storing data that serves a particular use case. Creating storage strategies for Red Hat Ceph Storage clusters includes creating CRUSH hierarchies, estimating the number of placement groups, determining which type of storage pool to create, and managing pools.

Red Hat Ceph Storage supports the following, among other, storage strategies within a single cluster:

Storage for OpenStack volumes and images

When storing volumes and images for a cloud platform such as OpenStack, consider the workload characteristics. Cinder handles random small-block I/O, while Glance serves sequential, high-volume reads. These workloads require fast storage. Use enterprise or datacenter grade NVMe SSDs in replicated pools.

Storage for S3- or Swift-compatible object data

For object data storage in S3- or Swift-compatible services, a more economical strategy may be appropriate. Enterprise SATA or SAS HDDs using CMR technology can be used, depending on object size and access patterns. Use 4Kn sector drives where possible. Very small objects (hundreds of KB or less) are not ideal for rotational media and should be stored on SSDs. Object storage clusters must include a modest pool of NVMe SSDs for indexes and metadata. For mixed workloads, use multiple storage classes: replicated NVMe SSDs for small objects, and erasure-coded HDDs for cost-effective bulk storage.

Interacting with the Ceph storage cluster is made simple from the perspective of the Ceph client. Storage strategies are invisible to the Ceph client in all but the storage capacity and performance.

The interface enables the Ceph client to select one of the defined storage strategies. The interaction is as follows:

1. Connect to the cluster.
2. Create a pool I/O context.

“Figure: Ceph storage architecture data flow” on page 109 shows the logical data flow starting from the client into the Red Hat Ceph Storage cluster.

The diagram illustrates the Ceph object storage workflow, starting from data creation by the Ceph client, which sends data to RADOS for conversion and RADOS Object creation. RADOS Objects are organized into pools, divided into placement groups (`pg_num = 4`), and then mapped by the CRUSH algorithm to Object Storage Daemons (OSDs) for writing and replication.

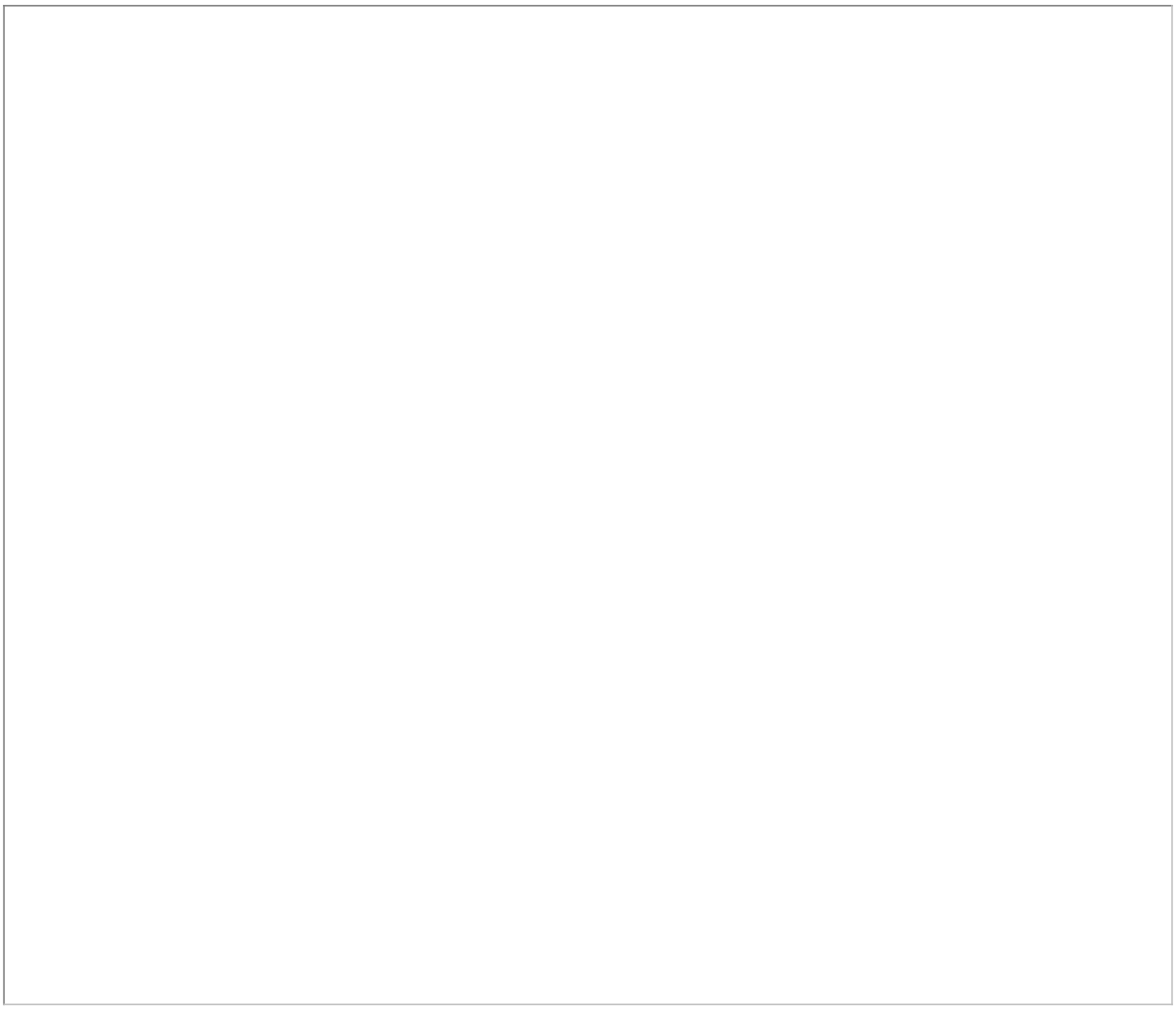


Figure 23: Ceph storage architecture data flow

Storage strategies include:

- Storage hardware components, including HDDs, SSDs, and other related storage devices.
- The CRUSH maps that set up performance and failure domains for the storage media.
- The number of placement groups.
- The pool interface.

Red Hat Ceph Storage supports multiple storage strategies. Use cases, cost/benefit performance tradeoffs, and data durability are the primary considerations that drive storage strategies.

Use cases

Ceph provides massive storage capacity, and it supports numerous use cases. For example, the Ceph Block Device client is a leading storage backend for cloud platforms like OpenStack that provides limitless storage for volumes and images with high-performance features like copy-on-write cloning. Likewise, Ceph can provide container-based storage for OpenShift environments. By contrast, the Ceph Object Gateway client is a leading storage backend for cloud platforms that provides RESTful S3-compliant and Swift-compliant object storage for objects like audio, bitmap, video, and other data.

Cost/benefit performance tradeoff

High capacity and high durability are desirable qualities, but each has a cost. When you create a CRUSH hierarchy of OSDs, consider the use case and an acceptable cost/performance tradeoff.

HDDs

Enterprise SATA and SAS HDDs are both suitable for Ceph. SAS drives offer a lower uncorrectable bit error rate (UBER). Use only CMR (conventional magnetic recording) drives; avoid SMR drives. Both 4Kn and 512e sector formats have been tested; 4Kn drives are recommended.

NVMe SSDs

NVMe is an interface, not a distinct drive type. SSDs use NAND for data storage and can connect over SATA, SAS, or NVMe interfaces. The NAND is the primary cost driver and is the same regardless of interface; skip SATA and SAS SSD interfaces and use NVMe. Use only enterprise or datacenter grade NVMe drives with a minimum mean time between failures (MTBF) of 2 million hours (or equivalent annualized failure rate (AFR)).

When evaluating NVMe drive endurance, measure petabyte writes (PBW) rather than drive writes per day (DWPD). A larger drive with lower DWPD can support more total writes than a smaller drive with higher DWPD. For example, a drive that is 4 times larger with 1 DWPD supports more writes than one that is 4 times smaller with 3 DWPD.

Do not assume that using multiple OSDs per NVMe drive is the correct configuration. Use multiple OSDs per NVMe drive only when there is an abundance of CPU available beyond the needs of co-resident components such as gateways. For example, a 64-core, 128-thread machine with 24 NVMe drives (a common 2U system shape) provides only 2.5 cores and 5 threads per OSD. A 64-core, 128-thread machine with 10 to 12 NVMe drives (a common 1U system shape) may be a better candidate. When the ratio approaches 5 cores and 10 threads per OSD, run empirical testing with your configuration before committing to multiple OSDs per drive, and weigh any performance gain against the added operational complexity.

Hybrid OSDs

Ceph uses device classes to describe OSD storage configurations. An `nvme` device class describes an OSD that stores both its data and BlueStore metadata and WAL on a single NVMe device. An `hdd` device class can describe either an OSD that stores its data and BlueStore metadata and WAL entirely on a single HDD, or an OSD that stores its data on an HDD and its BlueStore metadata and WAL on an NVMe SSD.

Tested HDD-to-metadata-SSD ratios include 12:1, 12:2, 35:2, and 21:1. Higher ratios reduce cluster-level durability. Do not mirror SSDs used for BlueStore metadata and WAL; drives in that role experience identical wear patterns and fail concurrently.

Durability

In large-scale clusters, hardware failure is an expectation, not an exception. However, data loss and service interruption remain unacceptable. For this reason, data durability is important. Ceph addresses data durability with multiple deep copies of an object or with erasure coding and multiple coding chunks. Multiple copies or multiple coding chunks present an extra cost/benefit tradeoff. It is cheaper to store fewer copies or coding chunks, but it might lead to the inability to service write requests in a degraded state. Generally, one object with 2 more copies (that is, `size = 3`) or 2 coding chunks might allow a cluster to service writes in a degraded state while the cluster recovers. The CRUSH algorithm aids this process by ensuring that Ceph stores more copies or coding chunks in different locations within the cluster. This ensures that the failure of a single storage device or node does not lead to a loss of all the copies or coding chunks necessary to preclude data loss.

You can capture use cases, cost/benefit performance tradeoffs, and data durability in a storage strategy and present it to a Ceph client as a storage pool.

> **Important:** Ceph's object copies or coding chunks make RAID obsolete. Do not use RAID because Red Hat Ceph Storage already handles data durability. A degraded RAID has a negative impact on performance and recovering data by using RAID is substantially slower than using deep copies or erasure coding chunks.

Configuring storage strategies

Configuring storage strategies is about assigning Ceph OSDs to a CRUSH hierarchy, defining the number of placement groups for a pool, and creating a pool.

Procedure

1. Define a storage strategy - Storage strategies require you to analyze your use case, cost/benefit performance tradeoffs and data durability. Then, you create OSDs suitable for that use case. For example, you can create SSD-backed OSDs for a high-performance pool; SAS drive/SSD journal-backed OSDs for

high-performance block device volumes and images; or SATA-backed OSDs for low-cost storage. Ideally, each OSD for a use case must have the same hardware configuration so that you have a consistent performance profile.

2. Define a CRUSH hierarchy - Ceph rules select a node, usually the `root`, in a CRUSH hierarchy, and identify the appropriate OSDs for storing placement groups and the objects they contain. Create a CRUSH hierarchy and a CRUSH rule for your storage strategy. CRUSH hierarchies get assigned directly to a pool by the CRUSH rule setting.
3. Calculate placement groups - Ceph shards a pool into placement groups. You do not have to manually set the number of placement groups for your pool. PG autoscaler sets an appropriate number of placement groups for your pool that remains within a healthy maximum number of placement groups when you assign multiple pools to the same CRUSH rule.
4. Create a pool - Finally, you must create a pool and determine whether it uses replicated or erasure-coded storage. Set the number of placement groups for the pool, the rule for the pool and the durability, such as size or `K+M` coding chunks.

What to do next

Remember, the pool is the Ceph client's interface to the storage cluster, but the storage strategy is transparent to the Ceph client, except for capacity and performance.

CRUSH admin overview

The Controlled Replication Under Scalable Hashing (CRUSH) algorithm determines how to store and retrieve data by computing data storage locations.

CRUSH introduction

The CRUSH map for your storage cluster describes your device locations within CRUSH hierarchies and a rule for each hierarchy that determines how Ceph stores data.

The CRUSH map contains at least one hierarchy of nodes and leaves. The nodes of a hierarchy, called "buckets" in Ceph, are any aggregation of storage locations as defined by their type. For example, rows, racks, chassis, hosts, and devices. Each leaf of the hierarchy consists essentially of one of the storage devices in the list of storage devices. A leaf is always contained in one node or "bucket." A CRUSH map also has a list of rules that determine how CRUSH stores and retrieves data.

> **Note:** Storage devices are added to the CRUSH map when adding an OSD to the cluster.

The CRUSH algorithm distributes data objects among storage devices according to a per-device weight value, approximating a uniform probability distribution. CRUSH distributes objects and their replicas or erasure-coding chunks according to the hierarchical cluster map an administrator defines. The CRUSH map represents the available storage devices and the logical buckets that contain them for the rule, and by extension each pool that uses the rule.

To map placement groups to OSDs across failure domains or performance domains, a CRUSH map defines a hierarchical list of bucket types; that is, under `types` in the generated CRUSH map. The purpose of creating a bucket hierarchy is to segregate the leaf nodes by their failure domains or performance domains or both. Failure domains include hosts, chassis, racks, power distribution units, pods, rows, rooms, and data centers. Performance domains include failure domains and OSDs of a particular configuration. For example, SSDs, SAS drives with SSD journals, SATA drives, and so on. Devices have the notion of a `class`, such as `hdd`, `ssd` and `nvme` to more rapidly build CRUSH hierarchies with a class of devices.

With the exception of the leaf nodes representing OSDs, the rest of the hierarchy is arbitrary, and you can define it according to your own needs if the default types do not suit your requirements. We recommend adapting your CRUSH map bucket types to your organization's hardware naming conventions and using instance names that reflect the physical hardware names. Your naming practice can make it easier to administer the cluster and troubleshoot problems when an OSD or other hardware malfunctions and the administrator needs remote or physical access to the host or other hardware.

In the following example, the bucket hierarchy has four leaf buckets (`osd 1-4`), two node buckets (`host 1-2`) and one rack node (`rack 1`).

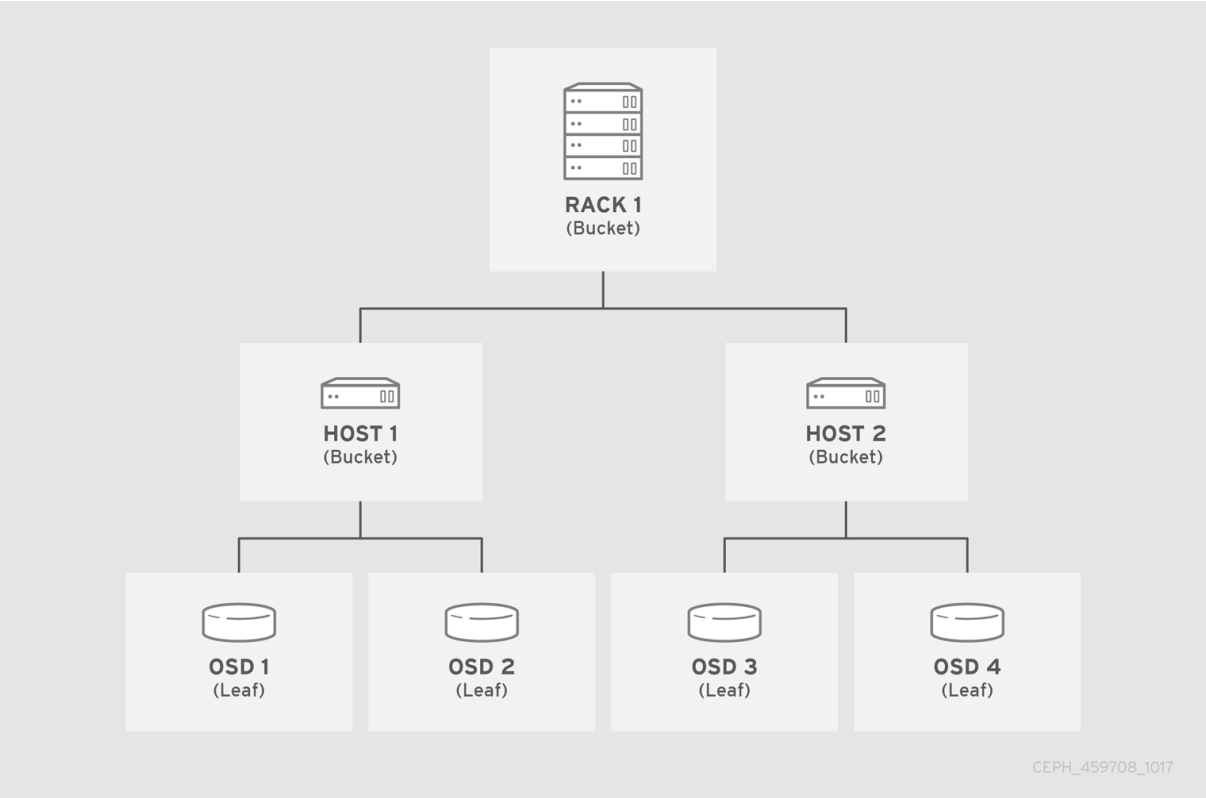


Figure 24: CRUSH hierarchy

Since leaf nodes reflect storage devices declared under the `devices` list at the beginning of the CRUSH map, there is no need to declare them as bucket instances. The second lowest bucket type in the hierarchy usually aggregates the devices; that is, it is usually the computer containing the storage media, and uses whatever term administrators prefer to describe it, such as "node", "computer", "server," "host", "machine", and so on. In high density environments, it is increasingly common to see multiple hosts/nodes per card and per chassis. Make sure to account for card and chassis failure too, for example, the need to pull a card or chassis if a node fails can result in bringing down numerous hosts/nodes and their OSDs.

When declaring a bucket instance, specify its type, give it a unique name as a string, assign it an optional unique ID expressed as a negative integer, specify a weight relative to the total capacity or capability of its items, specify the bucket algorithm such as `straw2`, and the hash that is usually `0` reflecting hash algorithm `rjenkins1`. A bucket can have one or more items. The items can consist of node buckets or leaves. Items can have a weight that reflects the relative weight of the item.

Dynamic data placement

Ceph Clients and Ceph OSDs both use the CRUSH map and the CRUSH algorithm.

- **Ceph Clients:** By distributing CRUSH maps to Ceph clients, CRUSH empowers Ceph clients to communicate with OSDs directly. This means that Ceph clients avoid a centralized object look-up table that could act as a single point of failure, a performance bottleneck, a connection limitation at a centralized look-up server and a physical limit to the storage cluster's scalability.
- **Ceph OSDs:** By distributing CRUSH maps to Ceph OSDs, Ceph empowers OSDs to handle replication, backfilling and recovery. This means that the Ceph OSDs handle storage of object replicas (or coding chunks) on behalf of the Ceph client. It also means that Ceph OSDs know enough about the cluster to re-balance the cluster (backfilling) and recover from failures dynamically.

CRUSH failure domain

Having multiple object replicas or `M` erasure coding chunks helps prevent data loss, but it is not sufficient to address high availability. By reflecting the underlying physical organization of the Ceph Storage Cluster, CRUSH can model, and thereby address, potential sources of correlated device failures. By encoding the cluster's topology into the cluster map, CRUSH placement policies can separate object replicas or erasure coding chunks across different failure domains while still maintaining the desired pseudo-random distribution.

For example, to address the possibility of concurrent failures, it might be desirable to ensure that data replicas or erasure coding chunks are on devices using different shelves, racks, power supplies, controllers or physical locations. This helps to prevent data loss and allows the cluster to operate in a degraded state.

CRUSH performance domain

Ceph can support multiple hierarchies to separate one type of hardware performance profile from another type of hardware performance profile. For example, CRUSH can create one hierarchy for hard disk drives and another hierarchy for SSDs. *Performance domains* are hierarchies that take the performance profile of the underlying hardware into consideration. Performance domains are increasingly popular due to the need to support different performance characteristics. Operationally, these are just CRUSH maps with more than one `root` type bucket.

Some use case examples of CRUSH performance domains include object storage, cold storage, and SSD-backed pools.

- **Object storage:** Ceph hosts that serve as an object storage back end for S3 and Swift interfaces might take advantage of less expensive storage media such as SATA drives that might not be suitable for VMs—reducing the cost per gigabyte for object storage, while separating more economical storage hosts from more performing ones intended for storing volumes and images on cloud platforms. HTTP tends to be the bottleneck in object storage systems.
- **Cold storage**: Systems designed for cold storage—infrequently accessed data, or data retrieval with relaxed performance requirements—might take advantage of less expensive storage media and erasure coding. However, erasure coding might require a bit of additional RAM and CPU, and thus differ in RAM and CPU requirements from a host used for object storage or VMs.
- **SSD-backed pools:** SSDs are expensive, but they provide significant advantages over hard disk drives. SSDs have no seek time and they provide high total throughput. In addition to using SSDs for journaling, a cluster can support SSD-backed pools. Common use cases include high performance SSD pools. For example, it is possible to map the `.rgw.buckets.index` pool for the Ceph Object Gateway to SSDs instead of SATA drives.

A CRUSH map supports the notion of a device `class`. Ceph can discover aspects of a storage device and automatically assign a class such as `hdd`, `ssd` or `nvme`. However, CRUSH is not limited to these defaults. For example, CRUSH hierarchies might also be used to separate different types of workloads. For example, an SSD might be used for a journal or write-ahead log, a bucket index or for raw object storage. CRUSH can support different device classes, such as `ssd-bucket-index` or `ssd-object-storage` so Ceph does not use the same storage media for different workloads—making performance more predictable and consistent.

Behind the scenes, Ceph generates a crush root for each device-class. These roots should only be modified by setting or changing device classes on OSDs. You can view the generated roots using the following command:

Example

```
[ceph: root@host01 /]# ceph osd crush tree --show-shadow

 ID   CLASS  WEIGHT    TYPE NAME
-24    ssd   4.54849  root default~ssd
-19    ssd   0.90970      host ceph01~ssd
  8    ssd   0.90970          osd.8
-20    ssd   0.90970      host ceph02~ssd
  7    ssd   0.90970          osd.7
-21    ssd   0.90970      host ceph03~ssd
  3    ssd   0.90970          osd.3
-22    ssd   0.90970      host ceph04~ssd
  5    ssd   0.90970          osd.5
-23    ssd   0.90970      host ceph05~ssd
  6    ssd   0.90970          osd.6
 -2    hdd  50.94173  root default~hdd
 -4    hdd   7.27739      host ceph01~hdd
 10    hdd   7.27739          osd.10
-12    hdd  14.55478      host ceph02~hdd
  0    hdd   7.27739          osd.0
 12    hdd   7.27739          osd.12
 -6    hdd  14.55478      host ceph03~hdd
  4    hdd   7.27739          osd.4
 11    hdd   7.27739          osd.11
-10    hdd   7.27739      host ceph04~hdd
  1    hdd   7.27739          osd.1
 -8    hdd   7.27739      host ceph05~hdd
  2    hdd   7.27739          osd.2
 -1         55.49022  root default
```

```
 -3          8.18709      host ceph01
 10    hdd   7.27739          osd.10
  8    ssd   0.90970          osd.8
-11         15.46448      host ceph02
  0    hdd   7.27739          osd.0
 12    hdd   7.27739          osd.12
  7    ssd   0.90970          osd.7
 -5         15.46448      host ceph03
  4    hdd   7.27739          osd.4
 11    hdd   7.27739          osd.11
 3    ssd   0.90970          osd.3
 -9          8.18709      host ceph04
  1    hdd   7.27739          osd.1
  5    ssd   0.90970          osd.5
 -7          8.18709      host ceph05
  2    hdd   7.27739          osd.2
  6    ssd   0.90970          osd.6
```

Using different device classes

To create performance domains, use device classes and a single CRUSH hierarchy.

To create performance domains, add OSDs to the CRUSH hierarchy, then do the following:

1. Add a class to each device. For example:

   Syntax

   ```
   ceph osd crush set-device-class <class> <osdId> [<osdId>]
   ceph osd crush set-device-class hdd osd.0 osd.1 osd.4 osd.5
   ceph osd crush set-device-class ssd osd.2 osd.3 osd.6 osd.7
   ```

2. Then, create rules to use the devices.

   Syntax

   ```
   ceph osd crush rule create-replicated <rule-name> <root> <failure-domain-type>
   <class>
   ceph osd crush rule create-replicated cold default host hdd
   ceph osd crush rule create-replicated hot default host ssd
   ```

3. Finally, set pools to use the rules.

   Syntax

   ```
   ceph osd pool set <poolname> crush_rule <rule-name>
   ceph osd pool set cold_tier crush_rule cold
   ceph osd pool set hot_tier crush_rule hot
   ```

> **Note:** There is no need to manually edit the CRUSH map.

CRUSH hierarchy

The CRUSH map is a directed acyclic graph, so it can accommodate multiple hierarchies, for example, performance domains.

The easiest way to create and modify a CRUSH hierarchy is with the Ceph CLI; however, you can also decompile a CRUSH map, edit it, recompile it, and activate it.

When declaring a bucket instance with the Ceph CLI, you must specify its type and give it a unique string name. Ceph automatically assigns a bucket ID, sets the algorithm to `straw2`, sets the hash to `0` reflecting `rjenkins1` and sets a weight. When modifying a decompiled CRUSH map, assign the bucket a unique ID expressed as a negative integer (optional), specify a weight relative to the total capacity/capability of its item(s), specify the bucket algorithm (usually `straw2`), and the hash (usually `0`, reflecting hash algorithm `rjenkins1`).

A bucket can have one or more items. The items can consist of node buckets (for example, racks, rows, hosts) or leaves (for example, an OSD disk). Items can have a weight that reflects the relative weight of the item.

When modifying a decompiled CRUSH map, you can declare a node bucket with the following syntax:

```
[bucket-type] [bucket-name] {
    id [a unique negative numeric ID]
    weight [the relative capacity/capability of the item(s)]
    alg [the bucket type: uniform | list | tree | straw2 ]
    hash [the hash type: 0 by default]
    item [item-name] weight [weight]
}
```

For example, we would define two host buckets and one rack bucket. The OSDs are declared as items within the host buckets:

```
host node1 {
    id -1
    alg straw2
    hash 0
    item osd.0 weight 1.00
    item osd.1 weight 1.00
}

host node2 {
    id -2
    alg straw2
    hash 0
    item osd.2 weight 1.00
    item osd.3 weight 1.00
}

rack rack1 {
    id -3
    alg straw2
    hash 0
    item node1 weight 2.00
    item node2 weight 2.00
}
```

Note: In the foregoing example, note that the rack bucket does not contain any OSDs. Rather it contains lower level host buckets, and includes the sum total of their weight in the item entry.

CRUSH location

A CRUSH location is the position of an OSD in terms of the CRUSH map's hierarchy.

When you express a CRUSH location on the command line interface, a CRUSH location specifier takes the form of a list of name/value pairs describing the OSD's position. For example, if an OSD is in a particular row, rack, chassis and host, and is part of the `default` CRUSH tree, its crush location could be described as:

```
root=default row=a rack=a2 chassis=a2a host=a2a1
```

Note:

1. The order of the keys does not matter.
2. The key name (left of =) must be a valid CRUSH `type`. By default these include `root`, `datacenter`, `room`, `row`, `pod`, `pdu`, `rack`, `chassis` and `host`. You might edit the CRUSH map to change the types to suit your needs.
3. You do not need to specify all the buckets/keys. For example, by default, Ceph automatically sets a `ceph-osd` daemon's location to be `root=default host={HOSTNAME}` (based on the output from `hostname -s`).

Adding a bucket

Add a bucket type for each CRUSH hierarchy.

To add a bucket instance to your CRUSH hierarchy, specify the bucket name and its type. Bucket names must be unique in the CRUSH map.

```
ceph osd crush add-bucket {name} {type}
```

If you plan to use multiple hierarchies, for example, for different hardware performance profiles, consider naming buckets based on their type of hardware or use case.

For example, you could create a hierarchy for solid state drives (`ssd`), a hierarchy for SAS disks with SSD journals (`hdd-journal`), and another hierarchy for SATA drives (`hdd`):

```
ceph osd crush add-bucket ssd-root root
ceph osd crush add-bucket hdd-journal-root root
ceph osd crush add-bucket hdd-root root
```

The Ceph CLI outputs:

```
added bucket ssd-root type root to crush map
added bucket hdd-journal-root type root to crush map
added bucket hdd-root type root to crush map
```

> **Important:** Using colons (`:`) in bucket names is not supported.

Add an instance of each bucket type you need for your hierarchy. The following example demonstrates adding buckets for a row with a rack of SSD hosts and a rack of hosts for object storage.

```
ceph osd crush add-bucket ssd-row1 row
ceph osd crush add-bucket ssd-row1-rack1 rack
ceph osd crush add-bucket ssd-row1-rack1-host1 host
ceph osd crush add-bucket ssd-row1-rack1-host2 host
ceph osd crush add-bucket hdd-row1 row
ceph osd crush add-bucket hdd-row1-rack1 rack
ceph osd crush add-bucket hdd-row1-rack1-host1 host
ceph osd crush add-bucket hdd-row1-rack1-host2 host
ceph osd crush add-bucket hdd-row1-rack1-host3 host
ceph osd crush add-bucket hdd-row1-rack1-host4 host
```

Once you have completed these steps, view your tree.

```
ceph osd tree
```

Notice that the hierarchy remains flat. You must move your buckets into a hierarchical position after you add them to the CRUSH map.

Moving a bucket

When you create your initial cluster, Ceph has a default CRUSH map with a root bucket named `default` and your initial OSD hosts appear under the `default` bucket. When you add a bucket instance to your CRUSH map, it appears in the CRUSH hierarchy, but it does not necessarily appear under a particular bucket. Moving a bucket instance to a particular location in your CRUSH hierarchy, in these cases, is necessary.

To move a bucket instance to a particular location in your CRUSH hierarchy, specify the bucket name and its type.

Example

```
ceph osd crush move ssd-row1 root=ssd-root
ceph osd crush move ssd-row1-rack1 row=ssd-row1
ceph osd crush move ssd-row1-rack1-host1 rack=ssd-row1-rack1
ceph osd crush move ssd-row1-rack1-host2 rack=ssd-row1-rack1
```

Once you have completed these steps, you can view your tree.

```
ceph osd tree
```

Note: You can also use `ceph osd crush create-or-move` to create a location while moving an OSD.

Removing a bucket

Learn how to remove a bucket.

To remove a bucket instance from your CRUSH hierarchy, specify the bucket name. For example:

```
ceph osd crush remove {bucket-name}
```

Or:

```
ceph osd crush rm {bucket-name}
```

Note: The bucket must be empty in order to remove it.

If you are removing higher level buckets (for example, a root like `default`), check to see if a pool uses a CRUSH rule that selects that bucket. If so, you need to modify your CRUSH rules; otherwise, peering fails.

CRUSH bucket algorithms

When you create buckets using the Ceph CLI, Ceph sets the algorithm to `straw2` by default. Ceph supports four bucket algorithms, each representing a tradeoff between performance and reorganization efficiency.

If you are unsure of which bucket type to use, we recommend using a `straw2` bucket. The bucket algorithms are uniform, list, tree, and straw2.

1. **Uniform:** Uniform buckets aggregate devices with **exactly** the same weight. For example, when firms commission or decommission hardware, they typically do so with many machines that have exactly the same physical configuration (for example, bulk purchases). When storage devices have exactly the same weight, you can use the `uniform` bucket type, which allows CRUSH to map replicas into uniform buckets in constant time. With non-uniform weights, you should use another bucket algorithm.
2. **List**: List buckets aggregate their content as linked lists. Based on the RUSH (Replication Under Scalable Hashing) $_{P}$ algorithm, a list is a natural and intuitive choice for an **expanding cluster**: either an object is relocated to the newest device with some appropriate probability, or it remains on the older devices as before. The result is optimal data migration when items are added to the bucket. Items removed from the middle or tail of the list, however, can result in a significant amount of unnecessary movement, making list buckets most suitable for circumstances in which they **never, or very rarely shrink**.
3. **Tree**: Tree buckets use a binary search tree. They are more efficient than listing buckets when a bucket contains a larger set of items. Based on the RUSH (Replication Under Scalable Hashing) $_{R}$ algorithm, tree buckets reduce the placement time to zero (log $_{n}$), making them suitable for managing much larger sets of devices or nested buckets.
4. **Straw2 (default):** List and Tree buckets use a divide and conquer strategy in a way that either gives certain items precedence, for example, those at the beginning of a list or obviates the need to consider entire subtrees of items at all. That improves the performance of the replica placement process, but can also introduce suboptimal reorganization behavior when the contents of a bucket change due an addition, removal, or re-weighting of an item. The `straw2` bucket type allows all items to fairly “compete” against each other for replica placement through a process analogous to a draw of straws.

Ceph OSDs in CRUSH

Once you have a CRUSH hierarchy for the OSDs, add OSDs to the CRUSH hierarchy. You can also move or remove OSDs from an existing hierarchy.

“CRUSH rule values” on page 118 describes the Ceph CLI usage values.

Table 23: CRUSH rule values				
Value	**Description**	**Type**	**Required**	**Example**
`id`	The numeric ID of the OSD.	Integer	Yes	`0`
`name`	The full name of the OSD.	String	Yes	`osd.0`
`weight`	The CRUSH weight for the OSD.	Double	Yes	`2.0`
`root`	The name of the root bucket of the hierarchy or tree in which the OSD resides.	Key-value pair.	Yes	`root=default`, `root=replicated_rule`, and so on
`bucket-type`	One or more name-value pairs, where the name is the bucket type and the value is the bucket's name. You can specify a CRUSH location for an OSD in the CRUSH hierarchy.	Key-value pair.	No	`datacenter=dc1 room=room1 row=foo rack=bar host=foo-bar-1`

Viewing OSDs in CRUSH

The **`ceph osd crush tree`** command outputs CRUSH buckets and items in a tree view. Use this command to determine a list of OSDs in a particular bucket. It gives output similar to the **`ceph osd tree`** command.

To return additional details, run the following:

```
# ceph osd crush tree -f json-pretty
```

The command returns an output similar to the following:

```
[
    {
        "id": -2,
        "name": "ssd",
        "type": "root",
        "type_id": 10,
        "items": [
            {
                "id": -6,
                "name": "dell-per630-11-ssd",
                "type": "host",
                "type_id": 1,
                "items": [
                    {
                        "id": 6,
                        "name": "osd.6",
                        "type": "osd",
                        "type_id": 0,
                        "crush_weight": 0.099991,
                        "depth": 2
                    }
                ]
            },
            {
                "id": -7,
                "name": "dell-per630-12-ssd",
                "type": "host",
                "type_id": 1,
                "items": [
                    {
                        "id": 7,
                        "name": "osd.7",
```

```
                        "type": "osd",
                        "type_id": 0,
                        "crush_weight": 0.099991,
                        "depth": 2
                    }
                ]
            },
            {
                "id": -8,
                "name": "dell-per630-13-ssd",
                "type": "host",
                "type_id": 1,
                "items": [
                    {
                        "id": 8,
                        "name": "osd.8",
                        "type": "osd",
                        "type_id": 0,
                        "crush_weight": 0.099991,
                        "depth": 2
                    }
                ]
            }
        ]
    },
    {
        "id": -1,
        "name": "default",
        "type": "root",
        "type_id": 10,
        "items": [
            {
                "id": -3,
                "name": "dell-per630-11",
                "type": "host",
                "type_id": 1,
                "items": [
                    {
                        "id": 0,
                        "name": "osd.0",
                        "type": "osd",
                        "type_id": 0,
                        "crush_weight": 0.449997,
                        "depth": 2
                    },
                    {
                        "id": 3,
                        "name": "osd.3",
                        "type": "osd",
                        "type_id": 0,
                        "crush_weight": 0.289993,
                        "depth": 2
                    }
                ]
            },
            {
                "id": -4,
                "name": "dell-per630-12",
                "type": "host",
                "type_id": 1,
                "items": [
                    {
                        "id": 1,
                        "name": "osd.1",
                        "type": "osd",
                        "type_id": 0,
                        "crush_weight": 0.449997,
                        "depth": 2
                    },
                    {
                        "id": 4,
                        "name": "osd.4",
                        "type": "osd",
                        "type_id": 0,
                        "crush_weight": 0.289993,
                        "depth": 2
                    }
                ]
            },
            {
                "id": -5,
```

```
                "name": "dell-per630-13",
                "type": "host",
                "type_id": 1,
                "items": [
                    {
                        "id": 2,
                        "name": "osd.2",
                        "type": "osd",
                        "type_id": 0,
                        "crush_weight": 0.449997,
                        "depth": 2
                    },
                    {
                        "id": 5,
                        "name": "osd.5",
                        "type": "osd",
                        "type_id": 0,
                        "crush_weight": 0.289993,
                        "depth": 2
                    }
                ]
            }
        ]
    }
]
```

Adding an OSD to CRUSH

Adding a Ceph OSD to a CRUSH hierarchy is the final step before you might start an OSD (rendering it `up` and `in`) and Ceph assigns placement groups to the OSD. You must prepare a Ceph OSD before you add it to the CRUSH hierarchy.

Deployment utilities, such as the Ceph Orchestrator, can prepare a CephOSD before adding it to the CRUSH hierarchy. For example creating a Ceph OSD on a single node:

Syntax

```
ceph orch daemon add osd HOST:DEVICE,[DEVICE]
```

The CRUSH hierarchy is notional, so the `ceph osd crush add` command allows you to add OSDs to the CRUSH hierarchy wherever you wish. The location you specify *should* reflect its actual location. If you specify at least one bucket, the command places the OSD into the most specific bucket you specify, *and* it moves that bucket underneath any other buckets you specify.

To add an OSD to a CRUSH hierarchy:

Syntax

```
ceph osd crush add ID_OR_NAME WEIGHT [BUCKET_TYPE=BUCKET_NAME ...]
```

Important: If you specify only the root bucket, the command attaches the OSD directly to the root. However, CRUSH rules expect OSDs to be inside of hosts or chassis, and host or chassis *should* be inside of other buckets reflecting your cluster topology.

If you specify only the root bucket, the command attaches the OSD directly to the root. However, CRUSH rules expect OSDs to be inside of hosts or chassis, and host or chassis *should* be inside of other buckets reflecting your cluster topology.

The following example adds `osd.0` to the hierarchy:

```
ceph osd crush add osd.0 1.0 root=default datacenter=dc1 room=room1 row=foo rack=bar
host=foo-bar-1
```

Note: You can also use `ceph osd crush set` or `ceph osd crush create-or-move` to add an OSD to the CRUSH hierarchy.

Moving an OSD within a CRUSH Hierarchy

If the storage cluster topology changes, you can move an OSD in the CRUSH hierarchy to reflect its actual location.

> **Important:** Moving an OSD in the CRUSH hierarchy means that Ceph will recompute which placement groups get assigned to the OSD, potentially resulting in significant redistribution of data

To move an OSD within the CRUSH hierarchy:

Syntax

```
ceph osd crush set ID_OR_NAME WEIGHT root=POOL_NAME  [BUCKET_TYPE=BUCKET_NAME...]
```

> **Note:** You can also use `ceph osd crush create-or-move` to move an OSD within the CRUSH hierarchy.

Removing an OSD from a CRUSH Hierarchy

Removing an OSD from a CRUSH hierarchy is the first step when you want to remove an OSD from your cluster. When you remove the OSD from the CRUSH map, CRUSH recomputes which OSDs get the placement groups and data re-balances accordingly.

To remove an OSD from the CRUSH map of a running cluster, run the following:

Syntax

```
ceph osd crush remove NAME
```

Device class

The Ceph CRUSH map provides a lot of flexibility in controlling data placement.

The flexibility in controlling data placement is one of Ceph's greatest strengths. Early Ceph deployments used hard disk drives almost exclusively. Today, Ceph clusters are frequently built with multiple types of storage devices: HDD, SSD, NVMe, or even various classes of the foregoing. For example, it is common in Ceph Object Gateway deployments to have storage policies where clients can store data on slower HDDs and other storage policies for storing data on fast SSDs. Ceph Object Gateway deployments might even have a pool backed by fast SSDs for bucket indices. Additionally, OSD nodes also frequently have SSDs used exclusively for journals or write-ahead logs that do NOT appear in the CRUSH map. These complex hardware scenarios historically required manually editing the CRUSH map, which can be time-consuming and tedious. It is not required to have different CRUSH hierarchies for different classes of storage devices.

CRUSH rules work in terms of the CRUSH hierarchy. However, if different classes of storage devices reside in the same hosts, the process becomes more complicated—requiring users to create multiple CRUSH hierarchies for each class of device, and then disable the `osd crush update on start` option that automates much of the CRUSH hierarchy management. Device classes eliminate this tediousness by telling the CRUSH rule what class of device to use, dramatically simplifying CRUSH management tasks.

> **Note:** The **`ceph osd tree`** command has a column reflecting a device class.

Reference

For more information, see "Using different device classes" on page 114 and "CRUSH storage strategies examples" on page 136.

Setting a device class

Use this information to set a device class for an OSD.

To set a device class for an OSD, run the following:

Syntax

```
ceph osd crush set-device-class  CLASS OSD_ID [OSD_ID..]
```

Example

```
[ceph: root@host01 /]# ceph osd crush set-device-class hdd osd.0 osd.1
[ceph: root@host01 /]# ceph osd crush set-device-class ssd osd.2 osd.3
[ceph: root@host01 /]# ceph osd crush set-device-class bucket-index osd.4
```

Note: Ceph might assign a class to a device automatically. However, class names are simply arbitrary strings. There is no requirement to adhere to `hdd`, `ssd` or `nvme`. In the foregoing example, a device class named `bucket-index` might indicate an SSD device that a Ceph Object Gateway pool uses exclusively bucket index workloads. To change a device class that was already set, use `ceph osd crush rm-device-class` first.

Removing a device class

Use this information to remove a device class for an OSD.

Remove a device class for an OSD by running the following command:

```
ceph osd crush rm-device-class OSD_ID [OSD_ID..]
```

For example,

```
[ceph: root@host01 /]# ceph osd crush rm-device-class 20
done removing class of osd(s): 20
[ceph: root@host01 /]# ceph osd crush rm-device-class 19 21
done removing class of osd(s): 19,21
```

Renaming a device class

Use this information to rename a device class for OSDs.

To rename a device class for all OSDs that use that class, run the following:

Syntax

```
ceph osd crush class rename OLD_NAME NEW_NAME
```

Example

```
[ceph: root@host01 /]# ceph osd crush class rename hdd sas15k
```

Listing a device class

Use this information to list device classes within a CRUSH map.

To list device classes in the CRUSH map, run the following:

Syntax

```
ceph osd crush class ls
```

The output will look something like this:

Example

```
[
    "hdd",
    "ssd",
```

```
    "bucket-index"
]
```

Listing OSDs of a device class

Use this information to list oSDs that belong to a device class.

To list all OSDs that belong to a particular class, run the following:

Syntax

```
ceph osd crush class ls-osd CLASS
```

Example

```
[ceph: root@host01 /]# ceph osd crush class ls-osd hdd
```

The output is simply a list of OSD numbers. For example:

```
0
1
2
3
4
5
6
```

Listing CRUSH rules by class

Use this information to list CRUSH rules that reference the same class.

To list all CRUSH rules that reference the same class, run the following:

Syntax

```
ceph osd crush rule ls-by-class CLASS
```

Example

```
[ceph: root@host01 /]# ceph osd crush rule ls-by-class hdd
```

CRUSH weights

The CRUSH algorithm assigns a weight value in terabytes (by convention) per OSD device. The assigned weight is done with the objective of approximating a uniform probability distribution for write requests that assign new data objects to placement groups (PGs) and PGs to OSDs.

As a result, it is best practice to create CRUSH hierarchies with devices of the same type and size, and to assign the same weight. In addition, use devices with the same I/O and throughput characteristics so that you will also have uniform performance characteristics in your CRUSH hierarchy, even though performance characteristics do not affect data distribution.

Since using uniform hardware is not always practical, you might incorporate OSD devices of different sizes and use a relative weight so that Ceph will distribute more data to larger devices and less data to smaller devices.

Setting CRUSH weights of OSDs

Set CRUSH weights in terabytes with a CRUSH map.

To set an OSD CRUSH weight in terabytes within the CRUSH map, run the following command, with the value information provided in “OSD CRUSH reweight values” on page 124:

```
ceph osd crush reweight NAME WEIGHT
```

Table 24: OSD CRUSH reweight values

Value	Description	Type	Required	Example
`name`	The full name of the OSD.	String	Yes.	`osd.0`
`weight`	The CRUSH weight for the OSD. This should be the size of the OSD in Terabytes, where `1.0` is 1 Terabyte. **Note:** This setting is used when creating an OSD or adjusting the CRUSH weight immediately after adding the OSD. It usually does not change over the life of the OSD.	Double	Yes.	`2.0`

Setting a bucket's OSD weights

Set all Ceph OSD weights under a bucket by running a single command.

Using the **`ceph osd crush reweight`** command can be time-consuming. You can set (or reset) all Ceph OSD weights under a bucket (row, rack, node, and so on) by running the **`ceph osd crush reweight-subtree`** command.

```
ceph osd crush reweight-subtree NAME
```

Where *NAME* is the name of the CRUSH bucket.

Set an OSD's `in` weight

In some cases, it is necessary to change the `in` weight of a particular OSD. Use this information to set an OSD's `in` weight.

For the purposes of `ceph osd in` and `ceph osd out`, an OSD is either `in` the cluster or `out` of the cluster. That is how a monitor records an OSD's status. However, even though an OSD is `in` the cluster, it might be experiencing a malfunction such that you do not want to rely on it as much until you fix it (for example, replace a storage drive, change out a controller, and so on).

You can increase or decrease the `in` weight of a particular OSD (that is, without changing its weight in terabytes) by executing:

Syntax

```
ceph osd reweight ID WEIGHT
```

Where:

- `id` is the OSD number.

- `weight` is a range from 0.0-1.0, where `0` is not `in` the cluster (that is, it does not have any PGs assigned to it) and 1.0 is `in` the cluster (that is, the OSD receives the same number of PGs as other OSDs).

Setting the OSDs weight by utilization

CRUSH is designed to approximate a uniform probability distribution for write requests that assign new data objects PGs and PGs to OSDs. Despite the CRUSH design, it is possible for clusters to become imbalanced for various reasons. If this occurs, set the OSD weight by utilization.

OSD weight imbalance can occur from various reasons, for example:

- **Multiple Pools:** You can assign multiple pools to a CRUSH hierarchy, but the pools might have different numbers of placement groups, size (number of replicas to store), and object size characteristics.
- **Custom Clients:** Ceph clients such as block device, object gateway, and filesystem share data from their clients and stripe the data as objects across the cluster as uniform-sized smaller RADOS objects. So except for the foregoing scenario, CRUSH usually achieves its goal. However, there is another case where a cluster can become imbalanced: namely, using `librados` to store data without normalizing the size of objects. This scenario can lead to imbalanced clusters (for example, storing 100 1-MB objects and 10 4-MB objects will make a few OSDs have more data than the others).
- **Probability:** A uniform distribution will result in some OSDs with more PGs and some with less. For clusters with a large number of OSDs, the statistical outliers will be further out.

You can reweight OSDs by utilization by executing the following:

Syntax

```
ceph osd reweight-by-utilization [THRESHOLD] [WEIGHT_CHANGE_AMOUNT] [NUMBER_OF_OSDS] [--no-increasing]
```

Example

```
[ceph: root@host01 /]# ceph osd test-reweight-by-utilization 110 .5 4 --no-increasing
```

Where:

- `threshold` is a percentage of utilization such that OSDs facing higher data storage loads will receive a lower weight and thus fewer PGs assigned to them. The default value is `120`, reflecting 120%. Any value from `100+` is a valid threshold. Optional.
- `weight_change_amount` is the amount to change the weight. Valid values are greater than `0.0 - 1.0`. The default value is `0.05`. Optional.
- `number_of_OSDs` is the maximum number of OSDs to reweight. For large clusters, limiting the number of OSDs to reweight prevents significant rebalancing. Optional.
- `no-increasing` is **off** by default. Increasing the osd weight is allowed when using the `reweight-by-utilization` or `test-reweight-by-utilization` commands. If this option is used with these commands, it prevents the OSD weight from increasing, even if the OSD is underutilized. Optional.

> **Important:** Executing `reweight-by-utilization` is recommended and somewhat inevitable for large clusters. Utilization rates might change over time, and as your cluster size or hardware changes, the weightings might need to be updated to reflect changing utilization. If you elect to reweight by utilization, you might need to re-run this command as utilization, hardware or cluster size change.

Executing this or other weight commands that assign a weight will override the weight assigned by this command (for example, `osd reweight-by-utilization`, `osd crush weight`, `osd weight`, `in` or `out`).

Setting an OSD's weight by placement group distribution

In CRUSH hierarchies with a smaller number of OSDs, it's possible for some OSDs to get more placement groups (PGs) than other OSDs, resulting in a higher load. You can reweight OSDs by PG distribution to address this situation.

Reweight OSDs by PG distribution, by running the following:

Syntax

```
osd reweight-by-pg POOL_NAME
```

Where:

- `poolname` is the name of the pool. Ceph will examine how the pool assigns PGs to OSDs and reweight the OSDs according to this pool's PG distribution. Note that multiple pools could be assigned to the same CRUSH hierarchy. Reweighting OSDs according to one pool's distribution could have unintended effects for other pools assigned to the same CRUSH hierarchy if they do not have the same size (number of replicas) and PGs.

Recalculating a CRUSH tree bucket weight

When manually editing CRUSH map weights, recalculate the CRUSH bucket tree weight.

CRUSH tree buckets should be the sum of their leaf weights. If you manually edit the CRUSH map weights, you should run the following to ensure that the CRUSH bucket tree accurately reflects the sum of the leaf OSDs under the bucket.

Syntax

```
osd crush reweight-all
```

Primary affinity

When a Ceph Client reads or writes data, it always contacts the primary OSD in the acting set. Sometimes an OSD is not well suited to act as a primary compared to other OSDs (for example, it has a slow disk or a slow controller). To prevent performance bottlenecks (especially on read operations) while maximizing utilization of your hardware, you can set a Ceph OSD's primary affinity so that CRUSH is less likely to use the OSD as a primary in an acting set.

For example, in set `[2, 3, 4]`, `osd.2` is the primary.

Syntax

```
ceph osd primary-affinity OSD_ID WEIGHT
```

Primary affinity is `1` by default (*that is,* an OSD might act as a primary). You might set the OSD primary range from `0-1`, where `0` means that the OSD might **NOT** be used as a primary and `1` means that an OSD might be used as a primary. When the weight is `< 1`, it is less likely that CRUSH will select the Ceph OSD Daemon to act as a primary.

CRUSH rules

Controlled Replication Under Scalable Hashing (CRUSH) rules define how a Ceph client selects buckets and the primary OSD within them to store objects, and how the primary OSD selects buckets and the secondary OSDs to store replicas or coding chunks.

In some cases, you might create a rule that selects a pair of target OSDs backed by SSDs for two object replicas, and another rule that selects three target OSDs backed by SAS drives in different data centers for three replicas.

Consider the following best practices when creating CRUSH rules:

- Use device classes whenever possible when creating CRUSH rules.
- Use the Ceph command-line interface (CLI) to create and manage CRUSH rules instead of manually editing the CRUSH map.
- Device classes help ensure that data is placed on the intended storage media type, such as HDD, SSD, or NVMe.
- CRUSH rules that do not specify a device class can select all eligible OSDs in the specified CRUSH hierarchy, which might not be the intended behavior in clusters that contain multiple media types.
- For examples of creating CRUSH rules by using CLI commands, see Creating CRUSH rules for replicated pools and Creating CRUSH rules for erasure coded pools.

A rule takes the following form, where the values information is provided in "CRUSH rule values" on page 127:

```
rule <rulename> {

    id <unique number>
    type [replicated | erasure]
    min_size <min-size>
    max_size <max-size>
    step take <bucket-type> [class <class-name>]
    step [choose|chooseleaf] [firstn|indep] <N> <bucket-type>
    step emit
}
```

Note: Specify a device class in the `step take` operation when creating new CRUSH rules.

Table 25: CRUSH rule values

Value	Description	Type	Required	Example/Default
id	A unique whole number for identifying the rule. Purpose: A component of the rule mask.	Integer	Yes	Default: `0`
type	Describes a rule for either a replicated or an erasure-coded storage drive. Purpose: A component of the rule mask. Valid value: `replicated`	String	Yes	Default: `replicated`
min_size	If a pool makes fewer replicas than this number, CRUSH does not select this rule. Purpose: A component of the rule mask.	Integer	Yes	Default: `1`
max_size	If a pool makes more replicas than this number, CRUSH does not select this rule. Purpose: A component of the rule mask.	Integer	Yes	Default: `10`

Value	Description	Type	Required	Example/Default
`step take <bucket-name> [class <class-name>]`	Takes a bucket name, and begins iterating down the tree. Purpose: A component of the rule.	N/A	Yes	Example: `step take data step take default class ssd` **Note:** Specify a device class whenever possible. Rules that do not specify a device class select all eligible OSDs in the selected CRUSH hierarchy.
`step choose firstn <num> type <bucket-type>`	Selects the number of buckets of the given type. The number is usually the number of replicas in the pool (that is, pool size). – If `<num> == 0`, choose `pool-num-replicas` buckets (all available). – If `<num> > 0 && < pool-num-replicas`, choose that many buckets. – If `<num> < 0`, it means `pool-num-replicas - {num}`. Purpose: A component of the rule. Prerequisite: Follows `step take` or `step choose`.	N/A	N/A	Example: `step choose firstn 1 type row`

Value	Description	Type	Required	Example/Default
`step chooseleaf firstn <num> type <bucket-type>`	Selects a set of buckets of `{bucket-type}` and chooses a leaf node from the subtree of each bucket in the set of buckets. The number of buckets in the set is usually the number of replicas in the pool (that is, pool size). – If `<num> == 0`, choose `pool-num-replicas` buckets (all available). – If `<num> > 0 && < pool-num-replicas`, choose that many buckets. – If `<num> < 0`, it means `pool-num-replicas -<num>`. Purpose: A component of the rule. Usage removes the need to select a device using two steps. Prerequisite: Follows `step take` or `step choose`.	N/A	N/A	Example: `step chooseleaf firstn 0 type row`
`step emit`	Outputs the current value and empties the stack. Typically used at the end of a rule, but might also be used to pick from different trees in the same rule. Purpose: A component of the rule. Prerequisite: Follows `step choose`.	N/A	N/A	Example: `step emit`

Value	Description	Type	Required	Example/Default
`firstn versus indep`	Controls the replacement strategy CRUSH uses when OSDs are marked down in the CRUSH map. If this rule is to be used with replicated pools it should be `firstn` and if it is for erasure-coded pools it should be `indep`. For example, you have a PG stored on OSDs 1, 2, 3, 4, 5 in which 3 goes down. With the `firstn` mode, CRUSH adjusts its calculation to select 1 and 2, then selects 3 but discovers it is down, so it retries and selects 4 and 5, and then goes on to select a new OSD 6. The final CRUSH mapping change is from `1, 2, 3, 4, 5` to `1, 2, 4, 5, 6`. With `indep` mode on an erasure-coded pool, CRUSH attempts to select the failed OSD 3, tries again and picks OSD 6, for a final transformation from `1, 2, 3, 4, 5` to `1, 2, 6, 4, 5`.	N/A	N/A	N/A

Important: A given CRUSH rule can be assigned to multiple pools, but a pool can have only one CRUSH rule.

The following example shows a replicated rule that uses a device class.

```
rule rgw-service {
 type replicated
 min_size 1
 max_size 10
 step take default class hdd
 step chooseleaf firstn 0 type rack
 step emit
}
```

The following example shows an alternative replicated rule that uses an SSD device class.

```
rule rgw-service {
    type replicated
    min_size 1
    max_size 10
    step take default class ssd
    step chooseleaf firstn 0 type rack
    step emit
}
```

Listing CRUSH rules

Use this information to list CRUSH rules from the command-line.

To list CRUSH rules from the command line, run the following:

Syntax

```
ceph osd crush rule list
ceph osd crush rule ls
```

Dumping CRUSH rules

Use this information to ldump the contents of a specific CRUSH rule from the command-line.

To dump the contents of a specific CRUSH rule, run the following:

Syntax

```
ceph osd crush rule dump NAME
```

Adding CRUSH rules

Use this information to add CRUSH rules from the command-line.

To add a CRUSH rule, you must specify a rule name, the root node of the hierarchy you wish to use, the type of bucket you want to replicate across (for example, *rack*, *row*, and so on and the mode for choosing the bucket.

Syntax

```
ceph osd crush rule create-simple RULENAME ROOT BUCKET_NAME FIRSTN_OR_INDEP
```

Ceph creates a rule with `chooseleaf` and one bucket of the type you specify.

Example

```
[ceph: root@host01 /]# ceph osd crush rule create-simple deleteme default host firstn
```

Create the following rule:

```
{ "id": 1,
  "rule_name": "deleteme",
  "type": 1,
  "min_size": 1,
  "max_size": 10,
  "steps": [
        { "op": "take",
          "item": -1,
          "item_name": "default"},
        { "op": "chooseleaf_firstn",
          "num": 0,
          "type": "host"},
        { "op": "emit"}]}
```

Creating CRUSH rules for replicated pools

Use this information to create a CRUSH rule for a replicated pool from the command-line.

To create a CRUSH rule for a replicated pool, run the following:

Syntax

```
ceph osd crush rule create-replicated NAME ROOT FAILURE_DOMAIN CLASS
```

Where:

- `<name>`: The name of the rule.
- `<root>`: The root of the CRUSH hierarchy.
- `<failure-domain>`: The failure domain. For example: `host` or `rack`.
- `<class>`: The storage device class. For example: `hdd` or `ssd`.

Example

```
[ceph: root@host01 /]# ceph osd crush rule create-replicated fast default host ssd
```

Creating CRUSH rules for erasure coded pools

To add a CRUSH rule for use with an erasure-coded pool, you might specify a rule name and an erasure coded profile.

Procedure

- Specify the rule name and an erasure-coded profile.

  ```
  ceph osd crush rule create-erasure RULE_NAME PROFILE_NAME
  ```

  For example,

  ```
  [ceph: root@host01 /]# ceph osd crush rule create-erasure default default
  ```

  For more information, see “Erasure code profiles” on page 170.

Removing CRUSH rules

Use this information to remove a CRUSH rule from the command-line.

To remove a rule, run the following and specify the CRUSH rule name:

Syntax

```
ceph osd crush rule rm NAME
```

CRUSH tuning

Ceph provides the ability to adjust certain parameters of the CRUSH algorithm, also known as CRUSH tuning.

Important considerations

Before adjusting CRUSH parameters, be sure to consider the following:

- Adjusting CRUSH values might result in the shift of some placement groups (PGs) between storage nodes. If the Ceph cluster is already storing a lot of data, be prepared for some fraction of the data to move.
- The `ceph-osd` and `ceph-mon` daemons will start requiring the feature bits of new connections as soon as they receive an updated map. However, already-connected clients are effectively grandfathered in, and will misbehave if they do not support the new feature. Make sure when you upgrade your Ceph Storage Cluster daemons that you also update your Ceph clients.
- If the CRUSH tunables are set to non-legacy values and then later changed back to the legacy values, `ceph-osd` daemons will not be required to support the feature. However, the OSD peering process requires examining and understanding old maps. Therefore, you should not run old versions of the `ceph-osd` daemon if the cluster has previously used non-legacy CRUSH values, even if the latest version of the map has been switched back to using the legacy defaults.
- Clusters running supported releases should set optimal tunables, by using the following command:

```
ceph osd crush tunables optimal
```

Tuning by changing to a known profile

Tuning by changing to a known profile is the preferred method.

> **Important:** Before you tune CRUSH, ensure that all Ceph clients and all Ceph daemons use the same version. If you have recently upgraded, ensure that you have restarted daemons and reconnected clients.

Set the CRUSH tunables after you upgrade, or if you receive a warning. Starting with version v0.74, Ceph issues a health warning if the CRUSH tunables are not set to their optimal values, the optimal values are the default as of v0.73.

To adjust the CRUSH tunables, change to a known profile.

You can select a profile on a running cluster with the following command:

```
ceph osd crush tunables PROFILE
```

> **Note:** This might result in some data movement.

Available known profiles are as follows:

`legacy`

The legacy behavior from v0.47 (pre-Argonaut) and earlier.

`argonaut`

The legacy values supported by v0.48 (Argonaut) release.

`bobtail`

The values supported by the v0.56 (Bobtail) release.

`firefly`

The values supported by the v0.80 (Firefly) release.

`hammer`

The values supported by the v0.94 (Hammer) release.

`jewel`

The values supported by the v10.0.2 (Jewel) release.

`optimal`

The current best values.

`default`

The current default values for a new cluster.

If a health warning is emitted that the CRUSH tunables are not set to their optimal values, run one of the following commands:

- Adjust the tunables on the existing cluster.
  Adjusting the tunables on the cluster results in up to 10% data movement. This is the preferred route, but should be taken with care on a production cluster where the data movement might affect performance.Enable tunalbes by using the **`crush tunables`** command.

  ```
  ceph osd crush tunables optimal
  ```

  If at any time you need to revert to an earlier profile, use the following command:

  ```
  ceph osd crush tunables PROFILE
  ```

Examples of when reverting might be necessary are the following:

- Too much load.
- Not enough progress.
- An issue with client compatibility, such as old kernel cephfs or rbd clients or pre-bobtail librados clients.

The following example restores the pre-v0.48 (Argonaut) values.

```
ceph osd crush tunables legacy
```

– Add the `warn on legacy crush tunables = false` option to the `mon` section of the `ceph.conf` file.

```
mon warn on legacy crush tunables = false
```

After changing the configuration file, either restart the monitors or apply the following option to the running monitors:

```
ceph tell mon.\* injectargs --no-mon-warn-on-legacy-crush-tunables
```

Tuning by modifying the CRUSH map

> **Important:** Only use this option if you can ensure that all clients are running recent code.

Adjust the tunables by extracting the CRUSH map, modifying the values, and then reinjecting the map into the cluster.

1. Extract the latest CRUSH map.

   ```
   ceph osd getcrushmap -o /tmp/crush
   ```

2. Adjust tunables.

   > **Important:** Use this option with extreme care.

   ```
   crushtool -i /tmp/crush --set-choose-local-tries 0 --set-choose-local-fallback-tries 0
   --set-choose-total-tries 50 -o /tmp/crush.new
   ```

   In addition to the values specified here, you also need to specify the **`--enable-unsafe-tunables`** argument to **`crushtool`** for this to work.

3. Reinject modified map.

   ```
   ceph osd setcrushmap -i /tmp/crush.new
   ```

Setting CRUSH legacy values

Legacy values for the CRUSH tunables can be set, when required.

– The special **`--enable-unsafe-tunables`** option is required.

– Be careful running old versions of the `ceph-osd` daemon after reverting to legacy values

```
crushtool -i /tmp/crush --set-choose-local-tries 2 --set-choose-local-fallback-tries 5 --
set-choose-total-tries 19 --set-chooseleaf-descend-once 0 --set-chooseleaf-vary-r 0 -o /
tmp/crush.legacy
```

Edit a CRUSH map

Generally, modifying your CRUSH map at runtime with the Ceph CLI is more convenient than editing the CRUSH map manually. However, there are times when you might choose to edit it, such as changing the default bucket types, or using a bucket algorithm other than `straw2`.

To edit an existing CRUSH map:

1. Get the CRUSH map, as described in "Getting the CRUSH map" on page 135.
2. Decompile the CRUSH map, as described in "Decompiling the CRUSH map" on page 135 .
3. Edit at least one of the devices, and buckets and rules.
4. Recompile the CRUSH map, as described in "Compiling the CRUSH map" on page 135.
5. Set the CRUSH map, as described in "Setting a CRUSH map" on page 135.

To activate a CRUSH Map rule for a specific pool, identify the common rule number and specify that rule number for the pool when creating the pool.

Getting the CRUSH map

Getting the CRUSH map is the necessary first step in editing the map.

To get the CRUSH map for your cluster, run the following:

Syntax

```
ceph osd getcrushmap -o COMPILED_CRUSHMAP_FILENAME
```

Ceph will output (-o) a compiled CRUSH map to the file name you specified. Since the CRUSH map is in a compiled form, you must decompile it first before you can edit it.

Decompiling the CRUSH map

Decompile the CRUSH map

To decompile a CRUSH map, run the following:

Syntax

```
crushtool -d COMPILED_CRUSHMAP_FILENAME -o DECOMPILED_CRUSHMAP_FILENAME
```

Ceph decompiles (-d) the compiled CRUSH map and send the output (-o) to the file name you specified.

Compiling the CRUSH map

To compile a CRUSH map, run the following:

Syntax

```
crushtool -c DECOMPILED_CRUSHMAP_FILENAME -o COMPILED_CRUSHMAP_FILENAME
```

Ceph will store a compiled CRUSH map to the file name you specified.

Setting a CRUSH map

To set the CRUSH map for your cluster, run the following:

Syntax

```
ceph osd setcrushmap -i COMPILED_CRUSHMAP_FILENAME
```

Ceph inputs the compiled CRUSH map of the file name you specified as the CRUSH map for the cluster.

CRUSH storage strategies examples

Set the CRUSH rules with device classes.

About this task

CRUSH can handle scenarios when you want to have most pools default to OSDs backed by large hard disk drives or some pools mapped to OSDs are backed by fast solid-state drives (SSDs). Use device classes to set rules for CRUSH to handle specific scenarios.

Procedure

1. Add a class to each device.

   ```
   ceph osd crush set-device-class CLASS OSD_ID [OSD_ID]
   ```

   For example,

   ```
   [ceph:root@host01 /]# ceph osd crush set-device-class hdd osd.0 osd.1 osd.4 osd.5
   [ceph:root@host01 /]# ceph osd crush set-device-class ssd osd.2 osd.3 osd.6 osd.7
   ```

2. Create rules to use the devices.

   ```
   ceph osd crush rule create-replicated RULENAME ROOT FAILURE_DOMAIN_TYPE DEVICE_CLASS
   ```

   For example,

   ```
   [ceph:root@host01 /]# ceph osd crush rule create-replicated cold default host hdd
   [ceph:root@host01 /]# ceph osd crush rule create-replicated hot default host ssd
   ```

3. Set the pools to use the rules.

   ```
   ceph osd pool set POOL_NAME crush_rule RULENAME
   ```

   For example,

   ```
   [ceph:root@host01 /]# ceph osd pool set cold crush_rule hdd
   [ceph:root@host01 /]# ceph osd pool set hot crush_rule ssd
   ```

   You do not need to manually edit the CRUSH map because one hierarchy can serve multiple classes of devices.
   For example,

```
device 0 osd.0 class hdd
device 1 osd.1 class hdd
device 2 osd.2 class ssd
device 3 osd.3 class ssd
device 4 osd.4 class hdd
device 5 osd.5 class hdd
device 6 osd.6 class ssd
device 7 osd.7 class ssd

  host ceph-osd-server-1 {
      id -1
      alg straw2
      hash 0
      item osd.0 weight 1.00
      item osd.1 weight 1.00
      item osd.2 weight 1.00
      item osd.3 weight 1.00
  }

  host ceph-osd-server-2 {
      id -2
      alg straw2
      hash 0
      item osd.4 weight 1.00
      item osd.5 weight 1.00
      item osd.6 weight 1.00
      item osd.7 weight 1.00
  }

  root default {
      id -3
      alg straw2
      hash 0
      item ceph-osd-server-1 weight 4.00
      item ceph-osd-server-2 weight 4.00
  }


  rule cold {
      ruleset 0
      type replicated
      min_size 2
      max_size 11
      step take default class hdd
      step chooseleaf firstn 0 type host
      step emit
  }


  rule hot {
      ruleset 1
      type replicated
      min_size 2
      max_size 11
      step take default class ssd
      step chooseleaf firstn 0 type host
      step emit
  }
```

Placement groups

Placement groups (PGs) are invisible to Ceph clients, but they play an important role in Ceph storage clusters. Ceph assigns objects to placement groups, and placement groups to OSDs to make re-balancing dynamic and efficient.

A Ceph Storage Cluster might require many thousands of OSDs to reach an exabyte level of storage capacity. Ceph clients store objects in pools, which are a logical subset of the overall cluster. The number of objects stored in a pool might easily run into the millions and beyond. A system with millions of objects or more cannot realistically track placement on a per-object basis and still perform well. Ceph assigns objects to placement groups, and placement groups to OSDs to make re-balancing dynamic and efficient.

About placement groups

Tracking object placement on a per-object basis within a pool is computationally expensive at scale. To facilitate high performance at scale, Ceph subdivides a pool into placement groups, assigns each individual object to a placement group, and assigns the placement group to a *primary* OSD. If an OSD fails or the cluster

re-balances, Ceph can move or replicate an entire placement group, meaning all of the objects in the placement groups, without having to address each object individually. This allows a Ceph cluster to re-balance or recover efficiently.

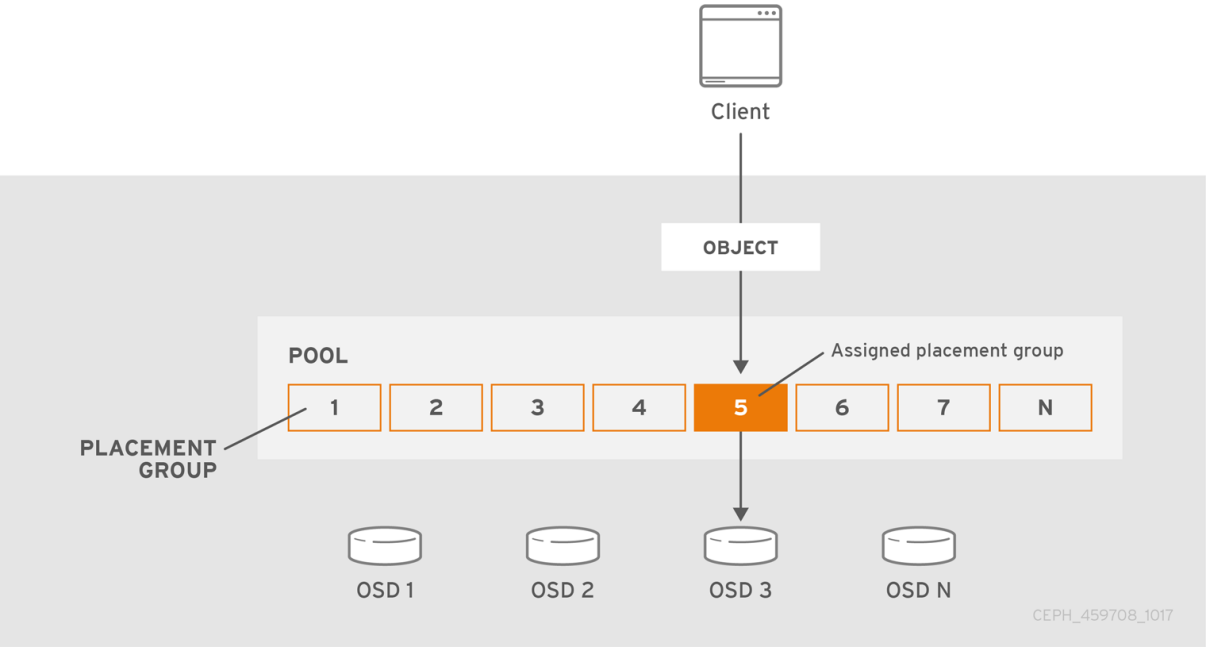


Figure 25: About PGs

When CRUSH assigns a placement group to an OSD, it calculates a series of OSDs—the first being the primary. The `osd_pool_default_size` setting minus `1` for replicated pools, and the number of coding chunks `M` for erasure-coded pools determine the number of OSDs storing a placement group that can fail without losing data permanently. Primary OSDs use CRUSH to identify the secondary OSDs and copy the placement group's contents to the secondary OSDs. For example, if CRUSH assigns an object to a placement group, and the placement group is assigned to OSD 5 as the primary OSD, if CRUSH calculates that OSD 1 and OSD 8 are secondary OSDs for the placement group, the primary OSD 5 will copy the data to OSDs 1 and 8. By copying data on behalf of clients, Ceph simplifies the client interface and reduces the client workload. The same process allows the Ceph cluster to recover and rebalance dynamically.

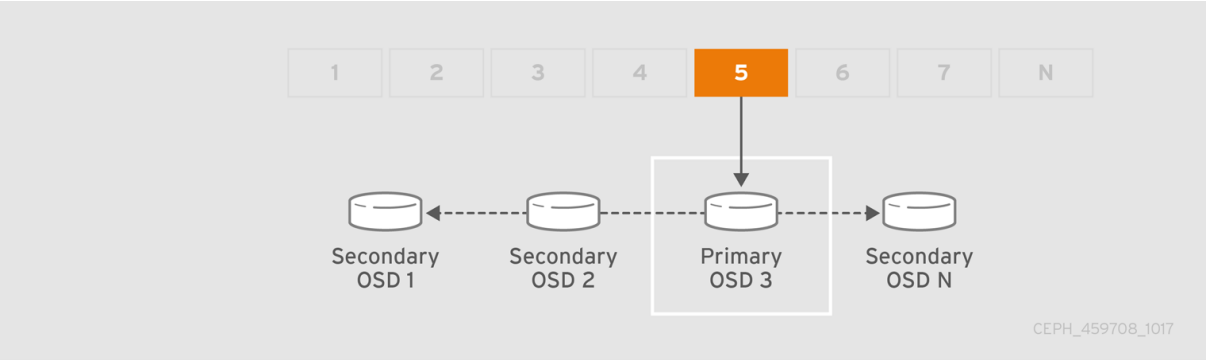


Figure 26: CRUSH hierarchy

When the primary OSD fails and gets marked out of the cluster, CRUSH assigns the placement group to another OSD, which receives copies of objects in the placement group. Another OSD in the `Up Set` will assume the role of the primary OSD.

When you increase the number of object replicas or coding chunks, CRUSH will assign each placement group to additional OSDs as required.

> **Note:** PGs do not own OSDs. CRUSH assigns many placement groups to each OSD pseudo-randomly to ensure that data gets distributed evenly across the cluster.

Placement group states

When you check the storage cluster's status with the `ceph -s` or `ceph -w` commands, Ceph reports on the status of the placement groups (PGs). A PG has one or more states. The optimum state for PGs in the PG map is an `active + clean` state. Use this information to get to know the different placement group states.

`activating`

The PG is peered, but not yet active.

`active`

Ceph processes requests to the PG.

`backfill_toofull`

A backfill operation is waiting because the destination OSD is over the backfillfull ratio.

`backfill_unfound`

Backfill stopped due to unfound objects.

`backfill_wait`

The PG is waiting in line to start backfill.

`backfilling`

Ceph is scanning and synchronizing the entire contents of a PG instead of inferring what contents need to be synchronized from the logs of recent operations. Backfill is a special case of recovery.

`clean`

Ceph replicated all objects in the PG accurately.

`creating`

Ceph is still creating the PG.

`deep`

Ceph is checking the PG data against stored checksums.

`degraded`

Ceph has not replicated some objects in the PG accurately yet.

`down`

A replica with necessary data is down, so the PG is offline. A PG with less than `min_size` replicas is marked as down. Use `ceph health detail` to understand the backing OSD state.

`forced_backfill`

High backfill priority of that PG is enforced by user.

`forced_recovery`

High recovery priority of that PG is enforced by user.

`incomplete`

Ceph detects that a PG is missing information about writes that might have occurred, or does not have any healthy copies. If you see this state, try to start any failed OSDs that might contain the needed information. In the case of an erasure coded pool, temporarily reducing `min_size` might allow recovery.

`inconsistent`

Ceph detects inconsistencies in one or more replicas of an object in the PG, such as objects are the wrong size, objects are missing from one replica after recovery finished.

`peering`

The PG is undergoing the peering process. A peering process should clear off without much delay, but if it stays and the number of PGs in a peering state does not reduce in number, the peering might be stuck.

`peered`

The PG has peered, but cannot serve client IO due to not having enough copies to reach the pool's configured `min_size` parameter. Recovery might occur in this state, so the PG might heal up to `min_size` eventually.

`recovering`

Ceph is migrating or synchronizing objects and their replicas.

recovery_toofull

A recovery operation is waiting because the destination OSD is over its full ratio.

recovery_unfound

Recovery stopped due to unfound objects.

recovery_wait

The PG is waiting in line to start recovery.

remapped

The PG is temporarily mapped to a different set of OSDs from what CRUSH specified.

repair

Ceph is checking the PG and repairing any inconsistencies it finds, if possible.

replay

The PG is waiting for clients to replay operations after an OSD crashed.

snaptrim

Trimming snaps.

snaptrim_error

Error stopped trimming snaps.

snaptrim_wait

Queued to trim snaps.

scrubbing

Ceph is checking the PG metadata for inconsistencies.

splitting

Ceph is splitting the PG into multiple PGs.

stale

The PG is in an unknown state; the monitors have not received an update for it since the PG mapping changed.

undersized

The PG has fewer copies than the configured pool replication level.

unknown

The `ceph-mgr` has not yet received any information about the PG's state from an OSD since Ceph Manager started up.

Related information

What are the possible Placement Group states in an RHCS/Ceph cluster on Red Hat Customer Portal

Placement group tradeoffs

Data durability and data distribution among all OSDs call for more placement groups but their number should be reduced to the minimum required for maximum performance to conserve CPU and memory resources.

Data durability

Ceph strives to prevent the permanent loss of data. However, after an OSD fails, the risk of permanent data loss increases until the data it had is fully recovered. Permanent data loss, though rare, is still possible.

The following scenario describes how Ceph could permanently lose data in a single placement group with three copies of the data:

- An OSD fails and all copies of the object it contains are lost. For all objects within a placement group stored on the OSD, the number of replicas suddenly drops from three to two.
- Ceph starts recovery for each placement group stored on the failed OSD by choosing a new OSD to re-create the third copy of all objects for each placement group.
- The second OSD containing a copy of the same placement group fails before the new OSD is fully populated with the third copy. Some objects will then only have one surviving copy.
- Ceph picks yet another OSD and keeps copying objects to restore the desired number of copies.
- The third OSD containing a copy of the same placement group fails before recovery is complete. If this OSD contained the only remaining copy of an object, the object is lost permanently.

Hardware failure isn't an exception, but an expectation. To prevent the foregoing scenario, ideally the recovery process should be as fast as reasonably possible. The size of your cluster, your hardware configuration and the number of placement groups play an important role in total recovery time.

Small clusters don't recover as quickly.

In a cluster containing 10 OSDs with 512 placement groups in a three replica pool, CRUSH will give each placement group three OSDs. Each OSD will end up hosting `(512 * 3) / 10 = ~150` placement groups. When the first OSD fails, the cluster will start recovery for all 150 placement groups simultaneously.

It is likely that Ceph stored the remaining 150 placement groups randomly across the 9 remaining OSDs. Therefore, each remaining OSD is likely to send copies of objects to all other OSDs and also receive some new objects, because the remaining OSDs become responsible for some of the 150 placement groups now assigned to them.

The total recovery time depends upon the hardware supporting the pool. For example, in a 10 OSD cluster, if a host contains one OSD with a 1 TB SSD, and a 10 GB/s switch connects each of the 10 hosts, the recovery time will take `M` minutes. By contrast, if a host contains two SATA OSDs and a 1 GB/s switch connects the five hosts, recovery will take substantially longer. Interestingly, in a cluster of this size, the number of placement groups has almost no influence on data durability. The placement group count could be 128 or 8192 and the recovery would not be slower or faster.

However, growing the same Ceph cluster to 20 OSDs instead of 10 OSDs is likely to speed up recovery and therefore improve data durability significantly. Why? Each OSD now participates in only 75 placement groups instead of 150. The 20 OSD cluster will still require all 19 remaining OSDs to perform the same amount of copy operations in order to recover. In the 10 OSD cluster, each OSDs had to copy approximately 100 GB. In the 20 OSD cluster each OSD only has to copy 50 GB each. If the network was the bottleneck, recovery will happen twice as fast. In other words, recovery time decreases as the number of OSDs increases.

In large clusters, PG count is important!

If the exemplary cluster grows to 40 OSDs, each OSD will only host 35 placement groups. If an OSD dies, recovery time will decrease unless another bottleneck precludes improvement. However, if this cluster grows to 200 OSDs, each OSD will only host approximately 7 placement groups. If an OSD dies, recovery will happen between at most of 21 `(7 * 3)` OSDs in these placement groups: recovery takes longer than when there were 40 OSDs, meaning the number of placement groups should be increased!

> **Important:** No matter how short the recovery time, there is a chance for another OSD storing the placement group to fail while recovery is in progress.

In the 10 OSD cluster described, if any OSD fails, then approximately 8 placement groups (that is `75 pgs / 9 osds` being recovered) will only have one surviving copy. And if any of the 8 remaining OSDs fail, the last objects of one placement group are likely to be lost (that is `8 pgs / 8 osds` with only one remaining copy being recovered). This is why starting with a somewhat larger cluster is preferred (for example, 50 OSDs).

When the size of the cluster grows to 20 OSDs, the number of placement groups damaged by the loss of three OSDs drops. The second OSD lost will degrade approximately 2 (that is `35 pgs / 19 osds` being recovered) instead of 8 and the third OSD lost will only lose data if it is one of the two OSDs containing the surviving copy. In other words, if the probability of losing one OSD is `0.0001%` during the recovery time frame, it goes from `8 * 0.0001%` in the cluster with 10 OSDs to `2 * 0.0001%` in the cluster with 20 OSDs. Having 512 or 4096 placement groups is roughly equivalent in a cluster with less than 50 OSDs as far as data durability is concerned.

> **Tip:** More OSDs means faster recovery and a lower risk of cascading failures leading to the permanent loss of a placement group and its objects.

When you add an OSD to the cluster, it might take a long time to populate the new OSD with placement groups and objects. However there is no degradation of any object and adding the OSD has no impact on data durability.

Data distribution

Ceph seeks to avoid hot spots, that is, some OSDs receive substantially more traffic than other OSDs. Ideally, CRUSH assigns objects to placement groups evenly so that when the placement groups get assigned to OSDs (also pseudo randomly), the primary OSDs store objects such that they are evenly distributed across the cluster and hot spots and network over-subscription problems cannot develop because of data distribution.

Since CRUSH computes the placement group for each object, but does not actually know how much data is stored in each OSD within this placement group, **the ratio between the number of placement groups and the number of OSDs might influence the distribution of the data significantly.**

For instance, if there was only one placement group with ten OSDs in a three replica pool, Ceph would only use three OSDs to store data because CRUSH would have no other choice. When more placement groups are available, CRUSH is more likely to evenly spread objects across OSDs. CRUSH also evenly assigns placement groups to OSDs.

As long as there are one or two orders of magnitude more placement groups than OSDs, the distribution should be even. For instance, 256 placement groups for 3 OSDs, 512 or 1024 placement groups for 10 OSDs, and so forth.

The ratio between OSDs and placement groups usually solves the problem of uneven data distribution for Ceph clients that implement advanced features like object striping. For example, a 4 TB block device might get sharded up into 4 MB objects.

The ratio between OSDs and placement groups does not address uneven data distribution in other cases, because CRUSH does not take object size into account. Using the `librados` interface to store some relatively small objects and some very large objects can lead to uneven data distribution. For example, one million 4K objects totaling 4 GB are evenly spread among 1000 placement groups on 10 OSDs. They will use `4 GB / 10 = 400 MB` on each OSD. If one 400 MB object is added to the pool, the three OSDs supporting the placement group in which the object has been placed will be filled with `400 MB + 400 MB = 800 MB` while the seven others will remain occupied with only 400 MB.

Resource usage

For each placement group, OSDs and Ceph monitors need memory, network, and CPU at all times, and even more during recovery. Sharing this overhead by clustering objects within a placement group is one of the main reasons placement groups exist.

Minimize the number of placement groups to save significant amounts of resources.

Placement group count

The number of placement groups (PGs) in a pool plays a significant role in how a cluster peers, distributes data, and rebalances.

Small clusters do not see as many performance improvements compared to large clusters by increasing the number of placement groups. However, clusters that have many pools accessing the same OSDs might need to carefully consider PG count so that Ceph OSDs use resources efficiently.

> **Note:** Use 100–200 placement group replicas per OSD.

Placement group calculator

The placement group (PG) calculator calculates the number of placement groups for you and addresses specific use cases.

The PG calculator is helpful when using Ceph clients like the Ceph Object Gateway where there are many pools typically using the same rule (CRUSH hierarchy). You might still calculate PGs manually using the guidelines in "Placement group count for small clusters" on page 143 and "Calculating placement group count" on page 143. However, the PG calculator is the preferred method of calculating PGs.

See Ceph Placement Groups (PGs) per Pool Calculator on the Red Hat Customer Portal for details.

Configuring default placement group count

When you create a pool, you also create a number of placement groups for the pool. While Ceph uses the default value of 8, this should increased along with the default value.

If you don't specify the number of placement groups, Ceph uses the default value of 8, which is unacceptably low. You can increase the number of placement groups for a pool, but we recommend setting reasonable default values too.

```
osd pool default pg num = 100
osd pool default pgp num = 100
```

You need to set both the number of placement groups (total), and the number of placement groups used for objects (used in PG splitting). They should be equal.

Placement group count for small clusters

Small clusters don't benefit from large numbers of placement groups, therefore it is important to use the PG calculator with small clusters.

As the number of OSDs increase, choosing the correct value for `pg_num` and `pgp_num` becomes more important because it has a significant influence on the behavior of the cluster as well as the durability of the data when something goes wrong (that is the probability that a catastrophic event leads to data loss).

For more information about the PG calculator, see "Placement group calculator" on page 142.

Calculating placement group count

Plan for 100–200 PG replicas per OSD in large clusters to balance durability, resource usage, and data distribution.

Clusters with more than 50 OSDs should plan for each OSD to host 100-200 placement group replicas. This balances resource usage, data durability, and uniform data distribution. If you have fewer than 50 OSDs, choosing among the PG count for small clusters as is recommended, as described in the previous section.

For a single pool of objects, you can use the following formula to get a baseline:

```
pg_num = (num_of_OSDs × ratio) / replication
ratio = (pg_num × replication) / num_of_OSDs
```

The PG ratio is the average number of PG replicas on each OSD.

Where pool size is either the number of replicas for replicated pools or the `K+M` sum for erasure coded pools (as returned by `ceph osd erasure-code-profile get`).

The result must be rounded to a power of 2.

> **Note:** When in doubt, err on the side of the higher power of 2.

The result should be rounded up to the nearest power of two. Rounding up is optional, but recommended for CRUSH to evenly balance the number of objects among placement groups.

For a cluster with 200 OSDs and a pool size of 3 replicas, you would estimate your number of PGs as follows:

```
   (200 * 200)
   ----------- = 13333
        3
```

In this example, the nearest power of 2 is 16384.

With 16384 placement groups, 49152 placement group replicas are distributed across 200 OSDs. Thus in this single-pool example, each OSD will on average host 245 placement group replicas. Consider the number of pools you are likely to use in your cluster, since additional pools will create additional placement groups too. With multiple RADOS pools, we solve for the aggregate number of placement group replicas.

The placement group calculator referenced in the above section is a valuable tool for manually calculating `pg_num` values when multiple data pools are present in a cluster, again targeting 100–200 PG replicas per OSD. Most clusters benefit by using the PG autoscaler however, which is described in the next section.

Ensure that you have a reasonable maximum placement group count. For more information, see "Maximum placement group count" on page 144

Maximum placement group count

Balance placement groups (PGs) across pools and OSDs to reduce variance and avoid resource strain or slow peering.

When using multiple data pools to store objects, ensure that the number of PGs per pool is balanced with the number of PGs per OSD. This helps maintain a reasonable total PG count and minimizes variance per OSD, avoiding excessive resource usage and slow peering.

For example, in a Red Hat Ceph Storage cluster with 10 pools, each configured with 512 PGs across 10 OSDs, the system manages 5,120 PGs total, which is 512 PGs per OSD. This configuration may be acceptable depending on your hardware. However, creating 1,000 pools with 512 PGs each results in ~50,000 PGs per OSD, which significantly increases resource demands. Excessive PGs per OSD can degrade performance, especially during rebalancing or recovery, and may lead to `OOMkilling` errors.

Red Hat Ceph Storage includes safeguards against excessive PG counts. By default, an OSD will not activate if it exceeds 250 PG replicas. You can adjust this limit using the following configuration command:

```
ceph config set mon_max_pg_per_osd 600
```

Setting the value to 600 provides a balance between protection and flexibility when OSDs fail or are added.

> **Tip:** Ceph Object Gateways deploy with 10-15 pools. To maintain a reasonable PG count, consider using fewer than 100 PGs per OSD.

Auto-scaling placement groups

Auto-scaling the number of PGs can make managing the cluster easier. The number of placement groups (PGs) in a pool plays a significant role in how a cluster peers, distributes data, and rebalances.

The auto-scaler analyzes pools and adjusts on a per-subtree basis.

Because each pool can map to a different CRUSH rule, and each rule can distribute data across different devices, Ceph considers utilization of each subtree of the hierarchy independently. For example, a pool that maps to OSDs of class `ssd`, and a pool that maps to OSDs of class `hdd`, will each have optimal PG counts that depend on the number of those respective device types.

The `pg-autoscaling` command provides recommendations for scaling PGs, or automatically scales PGs based on how the cluster is being used.

- To enable, or disable auto-scaling, see "Setting placement group auto-scaling modes" on page 146.
- To view placement group scaling recommendations, see "Viewing placement group scaling recommendations" on page 148.
- To set placement group auto-scaling, see "Configuring placement group auto-scaling" on page 150.
- To manually update autoscaler profile, see Manually updating autoscaler profile for a pool.
- To update the autoscaler globally, see "Updating noautoscale flag" on page 151.
- To set target pool size, see "Specifying target pool size" on page 151.

Placement group splitting and merging

Red Hat Ceph Storage can split existing placement groups (PGs) into smaller PGs, which increases the total number of PGs for a given pool. Red Hat Ceph Storage can also merge two existing PGs into a larger PG, which decreases the total number of PGs.

Splitting

Splitting existing placement groups (PGs) allows a small Red Hat Ceph Storage cluster to scale over time as storage requirements increase. The PG auto-scaling feature can increase the `pg_num` value, which causes the existing PGs to split as the storage cluster expands. If the PG auto-scaling feature is disabled, then you can manually increase the `pg_num` value, which triggers the PG split process to begin. For example, increasing the `pg_num` value from 4 to 16, will split into four pieces. Increasing the `pg_num` value will also increase the `pgp_num` value, but the `pgp_num` value increases at a gradual rate. This gradual increase is done to minimize the impact to a storage cluster's performance and to a client's workload, because migrating object data adds a significant load to the system. By default, Ceph queues and moves no more than 5% of the object data that is in a "misplaced" state. This default percentage can be adjusted with the `target_max_misplaced_ratio` option.

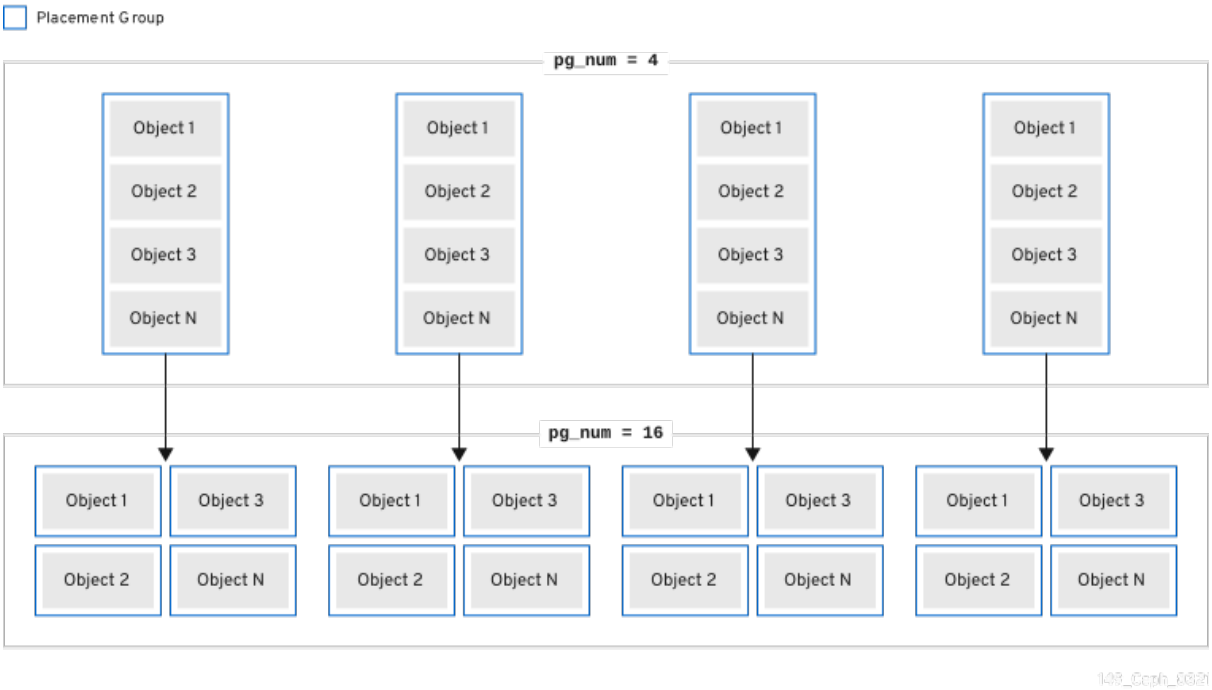


Figure 27: Splitting

Merging

Merging two PGs together can be useful, especially when the relative amount of objects in a pool decreases over time, or when the initial number of PGs chosen was too large. While merging PGs can be useful, it is also a complex and delicate process. When doing a merge, pausing I/O to the PG occurs, and only one PG is merged at a time to minimize the impact to a storage cluster's performance. Ceph works slowly on merging the object data until the new `pg_num` value is reached.

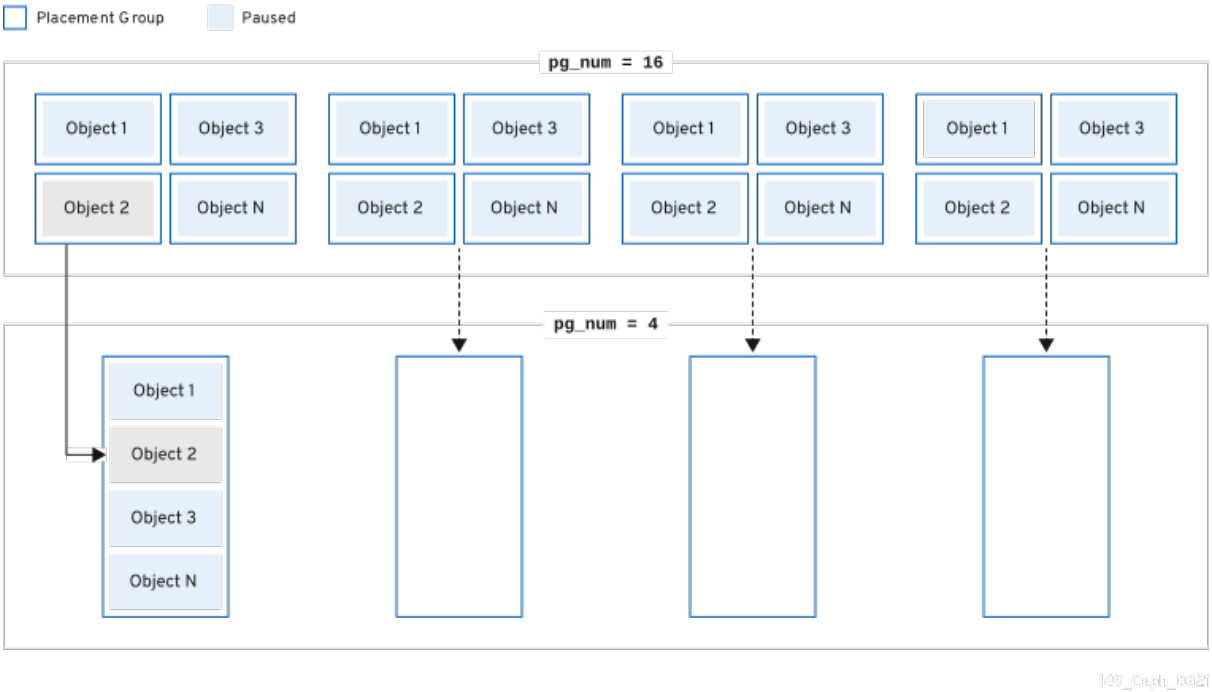


Figure 28: Merging

Setting placement group auto-scaling modes

Each pool in the Red Hat Ceph Storage cluster has a `pg_autoscale_mode` property for PGs that you can set to `off`, `on`, or `warn`.

- `off`: Disables auto-scaling for the pool. It is up to the administrator to choose an appropriate PG number for each pool. For more information, see "Placement group count" on page 142.
- `on`: Enables automated adjustments of the PG count for the given pool.
- `warn`: Raises health alerts when the PG count needs adjustment.

> **Note:** `pg_autoscale_mode` is on by default. Upgraded storage clusters retain the existing `pg_autoscale_mode` setting. The `pg_auto_scale` mode is `on` for the newly created pools. PG count is automatically adjusted, and `ceph status` might display a recovering state during PG count adjustment.

The autoscaler uses the `bulk` flag to determine which pool should start with a full complement of PGs and only scales down when the usage ratio across the pool is not even. However, if the pool does not have the `bulk` flag, the pool starts with minimal PGs and only when there is more usage in the pool.

> **Note:** The autoscaler identifies any overlapping roots and prevents the pools with such roots from scaling because overlapping roots can cause problems with the scaling process.

Procedure

- Enable auto-scaling on an existing pool:

  Syntax

  ```
  ceph osd pool set POOL_NAME pg_autoscale_mode on
  ```

  Example

```
[ceph: root@host01 /]# ceph osd pool set testpool pg_autoscale_mode on
```

- Enable auto-scaling on a newly created pool:

  Syntax

  ```
  ceph config set global osd_pool_default_pg_autoscale_mode MODE
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph config set global osd_pool_default_pg_autoscale_mode on
  ```

- Create a pool with the `bulk` flag:

  Syntax

  ```
  ceph osd pool create POOL_NAME [--bulk]
  ```

  Example

  ```
  [ceph: root@host01 /]#  ceph osd pool create testpool --bulk
  ```

- Set or unset the `bulk` flag for an existing pool:

  Important: The values must be written as `true`, `false`, `1`, or `0`. `1` is equivalent to `true` and `0` is equivalent to `false`. If written with different capitalization, or with other content, an error is emitted. The following is an example of the command written with the wrong syntax:

  ```
  [ceph: root@host01 /]# ceph osd pool set ec_pool_overwrite bulk
  TrueError EINVAL: expecting value 'true', 'false', '0', or '1'
  ```

  Syntax

  ```
  ceph osd pool set POOL_NAME bulk true/false/1/0
  ```

  Example

  ```
  [ceph: root@host01 /]#  ceph osd pool set testpool bulk true
  ```

- Get the `bulk` flag of an existing pool:

  Syntax

  ```
  ceph osd pool get POOL_NAME bulk
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph osd pool get testpool bulk
  bulk: true
  ```

Setting minimum and maximum number of placement groups for pools

Specify the minimum and maximum value of placement groups (PGs) in order to limit the auto-scaling range.

If a minimum value is set, Ceph does not automatically reduce, or recommend to reduce, the number of PGs to a value below the set minimum value.

If a minimum value is set, Ceph does not automatically increase, or recommend to increase, the number of PGs to a value above the set maximum value.

The minimum and maximum values can be set together or separately.

In addition to the this procedure, the **ceph osd pool create** command has two command-line options that can be used to specify the minimum or maximum PG count at the time of pool creation.

```
ceph osd pool create --pg-num-min NUMBER
ceph osd pool create --pg-num-max NUMBER
```

Example

For example:

```
ceph osd pool create --pg-num-min 50
ceph osd pool create --pg-num-max 150
```

Prerequisites

- A running Red Hat Ceph Storage cluster
- Root-level access to the node

Procedure

- Set the minimum number of PGs for a pool.

  Syntax

  ```
  ceph osd pool set POOL_NAME pg_num_min NUMBER
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph osd pool set testpool pg_num_min 50
  ```

- Set the maximum number of PGs for a pool.

  Syntax

  ```
  ceph osd pool set POOL_NAME pg_num_max NUMBER
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph osd pool set testpool pg_num_max 150
  ```

Reference

For more information, see:

- Setting placement group auto-scaling modes
- Placement Group count

Viewing placement group scaling recommendations

You can view the pool, its relative utilization, and any suggested changes to the placement group (PG) count within the storage cluster.

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- A `cephadm` shell or shell on a node with the cluster's admin key.

View each pool, its relative utilization, and any suggested changes to the PG count by using the **pool autoscale-status** command.

```
ceph osd pool autoscale-status
```

The command output looks similar to the following:

```
[ceph: root@host01 /]# ceph osd pool autoscale-status
OOL                      SIZE  TARGET SIZE  RATE  RAW CAPACITY   RATIO  TARGET RATIO
 EFFECTIVE RATIO  BIAS  PG_NUM  NEW PG_NUM  AUTOSCALE  BULK
  device_health_metrics      0                 3.0        374.9G  0.0000
                1.0       1              on         False
  cephfs.cephfs.meta     24632                 3.0        374.9G  0.0000
                4.0      32              on         False
  cephfs.cephfs.data         0                 3.0        374.9G  0.0000
                1.0      32              on         False
  .rgw.root                1323                 3.0        374.9G  0.0000
                1.0      32              on         False
  default.rgw.log          3702                 3.0        374.9G  0.0000
                1.0      32              on         False
  default.rgw.control        0                 3.0        374.9G  0.0000
                1.0      32              on         False
  default.rgw.meta         382                 3.0        374.9G  0.0000
                4.0       8              on         False
```

See the following output definitions for understanding the scaling status.

SIZE

The amount of data stored in the pool.

TARGET SIZE

If present, the amount of data the administrator expects to eventually be stored in this pool. The system uses the larger of the two values for its calculation.

RATE

The multiplier for the pool that determines how much raw storage capacity the pool uses. For example, a 3-replica pool has a ratio of 3.0, while a k=4,m=2 erasure coded pool has a ratio of 1.5.

RAW CAPACITY

The total amount of raw storage capacity on the OSDs responsible for storing the pool's data.

RATIO

The ratio of the total capacity that the pool is consuming, calculated as:
ratio = size × rate ÷ raw capacity

TARGET RATIO

If present, the ratio of storage the administrator expects the pool to consume relative to other pools with target ratios set. If both target size bytes and ratio are specified, the ratio takes precedence. The default value is 0 unless specified during pool creation. The higher the `--target_ratio`, the larger the PGs expected for the pool.

EFFECTIVE RATIO

The target ratio after adjusting in two ways:

1. Subtracting capacity expected to be used by pools with target size set.
2. Normalizing target ratios among pools with target ratio set so they collectively target the remaining space.

For example, four pools with target ratio 1.0 would each have an effective ratio of 0.25. The system uses the larger of the actual ratio and the effective ratio for its calculation.

BIAS

A multiplier to manually adjust a pool's PG count based on prior expectations. The default value is 1.0 unless specified during pool creation. The higher the `--bias`, the larger the PGs expected for the pool.

PG_NUM

The current number of PGs for the pool, or the number the pool is working towards if a `pg_num` change is in progress.

NEW PG_NUM

If present, the suggested number of PGs (`pg_num`). It is always a power of 2 and only appears if the suggested value differs from the current value by more than a factor of 3.

AUTOSCALE

The pool's `pg_autoscale_mode`, which can be `on`, `off`, or `warn`.

BULK

Determines which pool should start with a full complement of PGs. `BULK` only scales down when usage ratio across the pool is uneven. Pools without this flag start with minimal PGs and scale up with increased usage. Valid values are `true`, `false`, `1`, or `0`, where `1` is equivalent to `true`. The default is `false`. Set during or after pool creation.

For more information on using the `bulk` flag, see "Creating pools" on page 156 and "Configuring placement group auto-scaling" on page 150.

Configuring placement group auto-scaling

Allowing the cluster to automatically scale placement groups (PGs) based on cluster usage is the simplest approach to scaling PGs.

Red Hat Ceph Storage takes the total available storage and the target number of PGs for the whole system, compares how much data is stored in each pool, and apportions the PGs accordingly.

Configuring placement group autoscaling solves for the number of PG replicas per OSD.

> **Note:** This is not the same as dividing the pool's `pg_num` value by the number of OSDs.

A replicated `size=3` pool has 3×`pg_num` PG replicas.

An erasure-coded pool has (k + m)×`pg_num` replicas.

Each OSD's current number of PG replicas, the PG ratio, can be seen by using the **`ceph osd df`** command.

In the following example, `osd.217` holds 127 PG replicas.

```
[ceph: root@host01 /]# ceph osd df | head
ID  CLASS WEIGHT    REWEIGHT SIZE   RAW    USE    DATA   OMAP   META    AVAIL %USE VAR PGS
STATUS
217 hdd   18.53969  1.00000  19 TiB 15 TiB 14 TiB 21 KiB 53 GiB 3.9 TiB 79.20 1.01 127 up
```

The command only makes changes to a pool whose current number of PGs (`pg_num`) is more than three times off from the calculated or suggested PG number. This threshold can be modified by adjusting the value at pool granularity.

```
ceph osd pool set POOL threshold THRESHOLD
```

The following example adjusts the threshold for a pool named `pool01` to 2.5.

```
[ceph: root@host01 /]# ceph osd pool set pool01 threshold 2.5
```

The target number of PG replicas per OSD is determined by the **`mon_target_pg_per_osd`** central configuration option. The default value is `100` but most clusters benefit from setting to 250.

```
ceph config set global mon_target_pg_per_osd VALUE
```

For example,

```
[ceph: root@host01 /]# ceph config set global mon_target_pg_per_osd 250
```

> **Important:** When raising the target number of PG replicas per OSD, it is important to also raise the central configuration option **`mon_max_pg_per_osd`**. This value is a failsafe that guards against accidents. The default value is 250, but it is recommended to raise the value to 600.
>
> ```
> ceph config set global mon_max_pg_per_osd 600
> ```

Updating `noautoscale` flag

If you want to enable or disable the autoscaler for all the pools at same time, you can use the `noautoscale` global flag. This global flag is useful during upgradation of the storage cluster when some OSDs are bounced or when the cluster is under maintenance. You can set the flag before any activity and unset it once the activity is complete.

By default, the `noautoscale` flag is set to `off`. When this flag is set, then all the pools have `pg_autoscale_mode` as `off` and all the pools have the autoscaler disabled.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to all the nodes.

Procedure

1. Get the value of the `noautoscale` flag:

   Example

   ```
   [ceph: root@host01 /]# ceph osd pool get noautoscale
   ```

2. Set the `noautoscale` flag before any activity:

   Example

   ```
   [ceph: root@host01 /]# ceph osd pool set noautoscale
   ```

3. Unset the `noautoscale` flag on completion of the activity:

   Example

   ```
   [ceph: root@host01 /]# ceph osd pool unset noautoscale
   ```

Specifying target pool size

Specify target pool size.

A newly created pool consumes a small fraction of the total cluster capacity and appears to the system that it will need a small number of PGs. However, in most cases, cluster administrators know which pools are expected to consume most of the system capacity over time. If you provide this information, known as the `target size` to Red Hat Ceph Storage, such pools can use a more appropriate number of PGs (`pg_num`) from the beginning. This approach prevents subsequent changes in `pg_num` and the overhead associated with moving data around when making those adjustments.

You can specify target size of a pool by either using the absolute size of the pool or the total cluster capacity.

Specifying target size using the absolute size of the pool

Specify the target size of a pool by using the absolute size of the pool.

1. Set the `target size` using the absolute size of the pool in bytes:

   ```
   ceph osd pool set pool-name target_size_bytes value
   ```

   For example, to instruct the system that `mypool` is expected to consume 100T of space:

   ```
   $ ceph osd pool set mypool target_size_bytes 100T
   ```

You can also set the target size of a pool at creation time by adding the optional `--target-size-bytes <bytes>` argument to the **`ceph osd pool create`** command.

Specifying target size using the total cluster capacity

Specify the target size of a pool by using the total cluster capacity.

1. Set the `target size` using the ratio of the total cluster capacity:

   Syntax

   ```
   ceph osd pool set pool-name target_size_ratio ratio
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph osd pool set mypool target_size_ratio 1.0
   ```

   tells the system that the pool `mypool` is expected to consume 1.0 relative to the other pools with `target_size_ratio` set. If `mypool` is the only pool in the cluster, this means an expected use of 100% of the total capacity. If there is a second pool with `target_size_ratio` as 1.0, both pools would expect to use 50% of the cluster capacity.

You can also set the target size of a pool at creation time by adding the optional `--target-size-ratio <ratio>` argument to the `ceph osd pool create` command.

NOTE If you specify impossible target size values, for example, a capacity larger than the total cluster, or ratios that sum to more than 1.0, the cluster raises a POOL_TARGET_SIZE_RATIO_OVERCOMMITTED or POOL_TARGET_SIZE_BYTES_OVERCOMMITTED health warning. If you specify both target_size_ratio and target_size_bytes for a pool, the cluster considers only the ratio, and raises a POOL_HAS_TARGET_SIZE_BYTES_AND_RATIO health warning.

Placement group command line interface

The `ceph` CLI allows you to set and get the number of placement groups for a pool, view the PG map and retrieve PG statistics.

Setting number of placement groups in a pool

To set the number of placement groups in a pool, you must specify the number of placement groups at the time you create the pool.

Once you set placement groups for a pool, you can increase or decrease the number of placement groups. For more information, see “Creating pools” on page 156.

To change the number of placement groups, use the following command:

Syntax

```
ceph osd pool set POOL_NAME pg_num PG_NUM
```

Example

```
[ceph: root@host01 /]# ceph osd pool set pool1 pg_num 60
set pool 2 pg_num to 60
```

Once you increase or decrease the number of placement groups, you must also adjust the number of placement groups for placement (`pgp_num`) before your cluster rebalances. The `pgp_num` should be equal to the `pg_num`. To increase the number of placement groups for placement, run the following:

Syntax

```
ceph osd pool set POOL_NAME pgp_num PGP_NUM
```

Example

```
[ceph: root@host01 /]# ceph osd pool set pool1 pgp_num 60
set pool 2 pgp_num to 60
```

Getting number of placement groups in a pool

Learn how to get the number of placement groups in a pool.

To get the number of placement groups in a pool, run the following:

Syntax

```
ceph osd pool get POOL_NAME pg_num
```

Example

```
[ceph: root@host01 /]# ceph osd pool get testpool 60
```

Getting statistics for placement groups

Learn how to get the statistics for the placement groups in a storage cluster.

To get the statistics for the placement groups in your storage cluster, run the following:

Syntax

```
ceph pg dump [--format FORMAT]
```

Valid formats are `plain` (default) and `json`.

Getting statistics for stuck placement groups

Learn how to get the statistics for all placements groups stuck in a specific state.

To get the statistics for all placement groups stuck in a specified state, run the following:

Syntax

```
ceph pg dump_stuck {inactive|unclean|stale|undersized|degraded [inactive|unclean|stale|
undersized|degraded...]} INTEGER
```

Inactive Placement groups cannot process reads or writes because they are waiting for an OSD with the most up-to-date data to come up and in.

Unclean Placement groups contain objects that are not replicated the desired number of times. They should be recovering.

Stale Placement groups are in an unknown state - the OSDs that host them have not reported to the monitor cluster in a while (configured by `mon_osd_report_timeout`).

Valid formats are `plain` (default) and `json`. The threshold defines the minimum number of seconds the placement group is stuck before including it in the returned statistics (default 300 seconds).

Getting placement group maps

Learn how to get the placement group map for a particular placement group.

To get the placement group map for a particular placement group, run the following:

Syntax

```
ceph pg map PG_ID
```

Example

```
[ceph: root@host01 /]# ceph pg map 1.6c
osdmap e13 pg 1.6c (1.6c) -> up [1,0] acting [1,0]
```

Ceph returns the placement group map, the placement group, and the OSD status:

Getting a placement group statistics

Learn how to get statistics for a particular placement group.

Retrieve statistics for a particular placement group:

Syntax

```
ceph pg PG_ID query
```

Scrubbing placement groups

Learn how to scrub a placement group.

To scrub a placement group, run the following:

Syntax

```
ceph pg scrub PG_ID
```

Ceph checks the primary and any replica nodes, generates a catalog of all objects in the placement group and compares them to ensure that no objects are missing or mismatched, and their contents are consistent. Assuming the replicas all match, a final semantic sweep ensures that all of the snapshot-related object metadata is consistent. Errors are reported via logs.

Marking `unfound` objects

If the cluster has lost one or more objects, and you have decided to abandon the search for the lost data, you must mark the unfound objects as `lost`.

If all possible locations have been queried and objects are still lost, you might have to give up on the lost objects. This is possible given unusual combinations of failures that allow the cluster to learn about writes that were performed before the writes themselves are recovered.

Currently the only supported option is "revert", which will either roll back to a previous version of the object or (if it was a new object) forget about it entirely. To mark the "unfound" objects as "lost", run the following:

Syntax

```
ceph pg PG_ID mark_unfound_lost revert|delete
```

> **Important:** Use this feature with caution, because it might confuse applications that expect the object(s) to exist.

Pools overview

Ceph clients store data in pools. When you create pools, you create an I/O interface for clients to store data.

From a Ceph client perspective, (that is, block device, gateway, and the rest), interacting with the Ceph storage cluster is remarkably simple where you perform the following actions:

- Create a cluster handle.
- Connect the cluster handle to the cluster.
- Create an I/O context for reading and writing objects and their extended attributes.

Creating a cluster handle and connecting to the cluster

To connect to the Ceph storage cluster, the Ceph client needs the following details:

- The cluster name (which Ceph by default) - not using usually because it sounds ambiguous.
- An initial monitor address.

Ceph clients usually retrieve these parameters that use the default path for the Ceph configuration file and then read it from the file. You can also specify the parameters on the command line interface.

The Ceph client also provides a username and secret key (authentication is on by default) for the clients to contact the Ceph monitor cluster and retrieve a recent copy of the cluster map, including its monitors, OSDs, and pools.

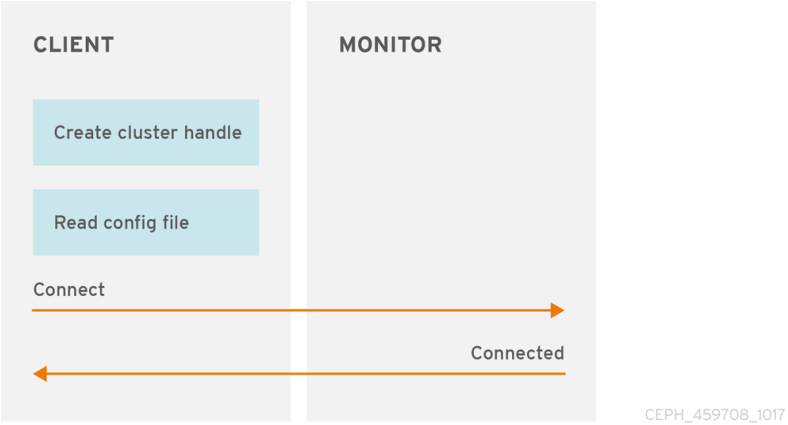


Figure 29: Create handle

Creating a pool I/O context

To read and write data, the Ceph client creates an I/O context to a specific pool in the Ceph storage cluster. If the specified user has the permission to the pool, the Ceph client can read from and write to the specified pool.

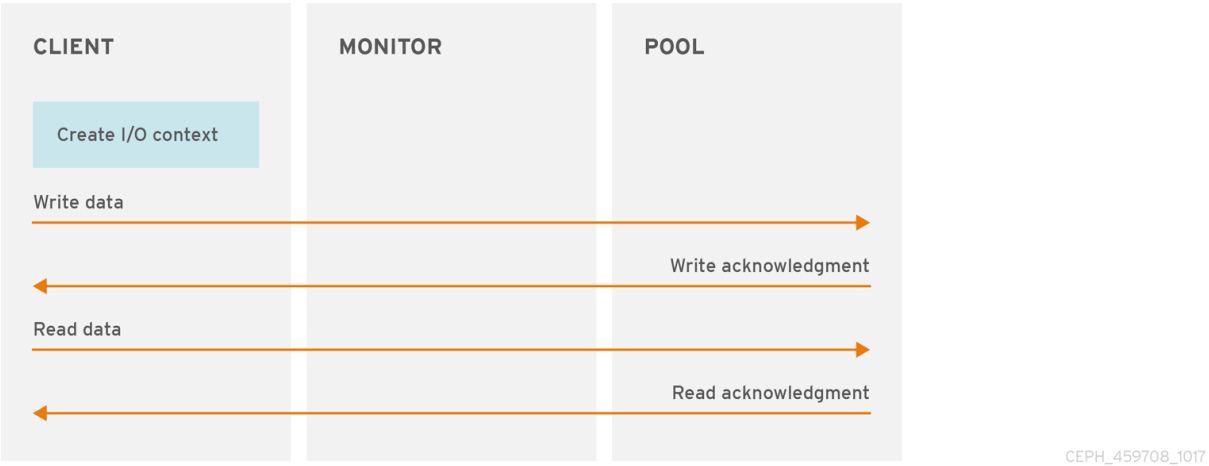


Figure 30: I/O Context

The architecture of Ceph enables the storage cluster to provide a simple interface to Ceph clients so that clients can select one of the storage strategies you define simply by specifying a pool name and creating an I/O context. Storage strategies are invisible to the Ceph client in all but capacity and performance. Similarly, the complexities of Ceph clients (mapping objects into a block device representation, providing an S3/Swift RESTful service) are invisible to the Ceph storage cluster.

A pool provides you with resilience, placement groups, CRUSH rules, and quotas.

Resilience

Set how many OSD are allowed to fail without losing data. For replicated pools, it is the wanted number of copies/replicas of an object. A typical configuration stores an object and one more copy (that is, `size = 2`), but you can determine the number of copies/replicas. For erasure coded pools, it is the number of coding chunks (that is `m=2` in the erasure code profile).

Placement groups

Set the number of placement groups for the pool. A typical configuration uses approximately 50-100 placement groups per OSD to provide optimal balancing without using up too many computing resources. When setting up multiple pools, ensure that you set a reasonable number of placement groups for both the pool and the cluster as a whole.

CRUSH rules

When you store data in a pool, a CRUSH rule mapped to the pool enables CRUSH to identify the rule for the placement of each object and its replicas or chunks for erasure coded pools in your cluster. You can create a custom CRUSH rule for your pool.

Quotas

When you set quotas on a pool with the **ceph osd pool set-quota** command you can limit the maximum number of objects or the maximum number of bytes stored in the specified pool.

You can list, create, and remove pools. You can also view the utilization statistics for each pool.

Listing pool

Learn how to list pools in a cluster.

To list your cluster's pools, run:

```
ceph osd lspools
```

Creating pools

Learn how to create a pool. Pools can be replicated, erasure-coded, or as bulk.

Before you begin

See Pools, placement groups, and CRUSH configuration.

> **Note:** The system administrators must expressly enable a pool to receive I/O operations from Ceph clients. See "Enabling a client application" on page 161 for details. Failure to enable a pool results in a `HEALTH_WARN` status.

About this task

When you create a pool, set the number of placement groups to a reasonable value (for example to 100). Consider the total number of placement groups per OSD too. Placement groups are computationally expensive, so performance degrades when you have many pools with many placement groups, for example, 50 pools with 100 placement groups each. The point of diminishing returns depends upon the power of the OSD host.

Based on the requirement, you can create a replicated pool, an erasure-coded pool, or a bulk pool.

- "Creating a replicated pool" on page 158
- "Creating an erasure-coded pool" on page 158
- "Creating a bulk pool" on page 158

In the examples, replace the variable names as follows:

POOL_NAME

Description

The name of the pool. It must be unique.

Type

String

Required

Yes. If not specified, it is set to the default value or manually set with the **ceph config set** command.

Default

`ceph`

PG_NUM

Description

The total number of placement groups for the pool. For more information about calculating a suitable number, see "Placement groups" on page 137 and Ceph Placement Groups (PGs) per Pool Calculator on the Red Hat Customer Portal. The default value `8` is not suitable for most systems.

Type

Integer

Required

Yes

Default

`8`

PGP_NUM

Description

The total number of placement groups for placement purposes. This value must be equal to the total number of placement groups, except for placement group splitting scenarios.

Type

Integer

Required

Yes. If not specified it is set to the value listed or to the default value.

Default

`8`

replicated* or *erasure

Description

The pool type can be either `replicated` to recover from lost OSDs by keeping multiple copies of the objects or `erasure` to get a generalized RAID5 capability. The replicated pools require more raw storage but implement all Ceph operations. The erasure-coded pools require less raw storage but implement only a subset of the available operations.

Type

String

Required

No

Default

`replicated`

crush-rule-name

Description

The name of the crush rule for the pool. The rule must exist. For replicated pools, the name is the rule that is specified by the `osd_pool_default_crush_rule` configuration setting. For erasure-coded pools the name is `erasure-code` if you specify the default erasure code profile or *POOL_NAME* otherwise. Ceph creates this rule with the specified name implicitly if the rule doesn't exist.

Type

String

Required

No

Default

Uses `erasure-code` for an erasure-coded pool. For replicated pools, it uses the value of the `osd_pool_default_crush_rule` variable from the Ceph configuration.

expected-num-objects

Description

The expected number of objects for the pool. Ceph splits the placement groups at pool creation time to avoid the latency impact to perform runtime directory splitting.

Type

Integer

Required

No

Default

`0`, no splitting at the pool creation time.

erasure-code-profile

Description

For erasure-coded pools only. Use the erasure code profile. It must be an existing profile as defined by the `osd erasure-code-profile set` variable in the Ceph configuration file. For more information, see "Erasure code profiles" on page 170.

Type

String

Required

No

Creating a replicated pool

- To create a replicated pool, use the following command:

```
ceph osd pool create POOL_NAME PG_NUM PGP_NUM [replicated] [CRUSH_RULE_NAME]
[EXPECTED_NUMBER_OBJECTS]
```

Creating an erasure-coded pool

- To create an erasure-coded pool, use the following command:

```
ceph osd pool create POOL_NAME PG_NUM PGP_NUM [erasure] ERASURE_CODE_PROFILE
         [CRUSH_RULE_NAME] [EXPECTED_NUMBER_OBJECTS]
```

Creating a bulk pool

- To create bulk pool, use the following command:

```
ceph osd pool create POOL_NAME [--bulk]
```

Setting pool quota

You can set pool quotas for the maximum number of bytes or the maximum number of objects per pool or for both.

Syntax

```
ceph osd pool set-quota POOL_NAME [max_objects OBJECT_COUNT>] [max_bytes BYTES]
```

Example

```
[ceph: root@host01 /]# ceph osd pool set-quota data max_objects 10000
```

To remove a quota, set its value to `0`.

Note: In-flight write operations might overrun pool quotas for a short time until Ceph propagates the pool usage across the cluster. This is normal behavior. Enforcing pool quotas on in-flight write operations would impose significant performance penalties.

Deleting a pool

Learn how to delete a pool.

To delete a pool, run:

Syntax

```
ceph osd pool delete POOL_NAME [POOL_NAME --yes-i-really-really-mean-it]
```

Important: To protect data, storage administrators cannot delete pools by default. Set the `mon_allow_pool_delete` configuration option before deleting pools.

If a pool has its own rule, consider removing it after deleting the pool. If a pool has users strictly for its own use, consider deleting those users after deleting the pool.

Renaming a pool

Learn how to rename a pool.

To rename a pool, run:

Syntax

```
ceph osd pool rename CURRENT_POOL_NAME NEW_POOL_NAME
```

If you rename a pool and you have per-pool capabilities for an authenticated user, you must update the user's capabilities (that is, caps) with the new pool name.

Migrating a pool

Sometimes it is necessary to migrate all objects from one pool to another. This is done in cases such as needing to change parameters that cannot be modified on a specific pool. For example, needing to reduce the number of placement groups of a pool.

Before you begin

- If using the **`rados cppool`** command:
  - Read-only access to the pool is required.
  - Only use this command if you do not have RBD images and its snaps and `user_version` consumed by librados.
- If using the local drive RADOS commands, verify that sufficient cluster space is available. Two, three, or more copies of data will be present as per pool replication factor.

About this task

Important: When a workload is using *only* Ceph Block Device images, use the following procedures:

- "Moving images between pools" on page 1264
- "Migrating pools" on page 1266

The migration methods described for Ceph Block Device are more recommended than those documented here. using the cppool does not preserve all snapshots and snapshot related metadata, resulting in an unfaithful copy of the data. For example, copying an RBD pool does not completely copy the image. In this case, snaps are not present and will not work properly. The cppool also does not preserve the `user_version` field that some librados users may rely on.

If migrating a pool is necessary and your user workloads contain images other than Ceph Block Devices, continue with one of the procedures documented here.

Migrating directly

Copy all objects with the **`rados cppool`** command.

> **Important:** Read-only access to the pool is required during copy.

```
ceph osd pool create NEW_POOL PG_NUM [ <other new pool parameters> ]
rados cppool SOURCE_POOL NEW_POOL
ceph osd pool rename SOURCE_POOL NEW_SOURCE_POOL_NAME
ceph osd pool rename NEW_POOL SOURCE_POOL
```

For example,

```
[ceph: root@host01 /]# ceph osd pool create pool1 250
[ceph: root@host01 /]# rados cppool pool2 pool1
[ceph: root@host01 /]# ceph osd pool rename pool2 pool3
[ceph: root@host01 /]# ceph osd pool rename pool1 pool2
```

Migrating through a local drive

Procedure

1. Use the **`rados export`** and **`rados import`** commands and a temporary local directory to save all exported data.

   ```
   ceph osd pool create NEW_POOL PG_NUM [ <other new pool parameters> ]
   rados export --create SOURCE_POOL FILE_PATH
   rados import FILE_PATH NEW_POOL
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph osd pool create pool1 250
   [ceph: root@host01 /]# rados export --create pool2 <path of export file>
   [ceph: root@host01 /]# rados import <path of export file> pool1
   ```

2. Stop all I/O to the source pool.
3. Resynchronize all modified objects.

   ```
   rados export --workers 5 SOURCE_POOL FILE_PATH
   rados import --workers 5 FILE_PATH NEW_POOL
   ```

   For example,

   ```
   [ceph: root@host01 /]# rados export --workers 5 pool2 <path of export file>
   [ceph: root@host01 /]# rados import --workers 5 <path of export file> pool1
   ```

Viewing pool statistics

Learn how to view a pool's utilization statistics.

To show a pool’s utilization statistics, run the following command:

```
rados df
```

Setting pool values

Set pool values.

To set a value to a pool, run the following command:

```
ceph osd pool set POOL_NAME KEY VALUE
```

For more information about the available key-value pairs, see “Pool values” on page 163.

Getting pool values

Get a value from a pool.

To get a value from a pool, run the following command:

```
ceph osd pool get POOL_NAME KEY
```

For more information about the available key-value pairs, see “Pool values” on page 163.

Enabling a client application

Red Hat Ceph Storage provides additional protection for pools to prevent unauthorized types of clients from writing data to the pool. This means that system administrators must expressly enable pools to receive I/O operations from Ceph Block Device, Ceph Object Gateway, Ceph Filesystem or for a custom application.

To enable a client application to conduct I/O operations on a pool, execute the following:

Syntax

```
ceph osd pool application enable POOL_NAME APP {--yes-i-really-mean-it}
```

Where `APP` is:

- `cephfs` for the Ceph Filesystem.
- `rbd` for the Ceph Block Device
- `rgw` for the Ceph Object Gateway

Important: A pool that is not enabled will generate a `HEALTH_WARN` status.
In that scenario, the output for `ceph health detail -f json-pretty` gives the following output:

```
{
    "checks": {
        "POOL_APP_NOT_ENABLED": {
            "severity": "HEALTH_WARN",
            "summary": {
                "message": "application not enabled on 1 pool(s)"
            },
            "detail": [
                {
                    "message": "application not enabled on pool 'POOL_NAME'"
                },
                {
                    "message": "use 'ceph osd pool application enable POOL_NAME APP',
where APP is 'cephfs', 'rbd', 'rgw', or freeform for custom applications."
                }
            ]
        }
    },
    "status": "HEALTH_WARN",
    "overall_status": "HEALTH_WARN",
    "detail": [
        "'ceph health' JSON format has changed in luminous. If you see this your
monitoring system is scraping the wrong fields. Disable this with 'mon health
preluminous compat warning = false'"
    ]
}
```

Note:

- Specify a different `APP` value for a custom application.
- Initialize pools for the Ceph Block Device with `rbd pool init POOL_NAME`.

Disabling a client application

To disable a client application from conducting I/O operations on a pool, run the following:

Syntax

```
ceph osd pool application disable POOL_NAME APP {--yes-i-really-mean-it}
```

Where `APP` is:

- `cephfs` for the Ceph Filesystem.
- `rbd` for the Ceph Block Device
- `rgw` for the Ceph Object Gateway

Note: Specify a different `APP` value for a custom application.

Setting application metadata

Set client application metadata on a pool to provide the functionality to set key-value pairs describing attributes of the client application.

To set client application metadata on a pool, run the following:

```
ceph osd pool application set POOL_NAME APP KEY VALUE
```

Replace *APP* with one of the following:

- `cephfs` for Ceph File System.
- `rbd` for Ceph Block Device
- `rgw` for Ceph Object Gateway

Note:

- Specify a different *APP* value for a custom application.
- This command is different than the **`ceph osd pool application enable POOL APP`** command.

Removing application metadata

To remove client application metadata on a pool, run the following:

Syntax

```
ceph osd pool application rm POOL_NAME APP KEY
```

Where `APP` is:

- `cephfs` for the Ceph Filesystem.
- `rbd` for the Ceph Block Device

– `rgw` for the Ceph Object Gateway

> **Note:** Specify a different `APP` value for a custom application.

Setting the number of object replicas

Learn how to set the number of object replicas.

To set the number of object replicas on a replicated pool, run the following command:

Syntax

```
ceph osd pool set POOL_NAME size NUMBER_OF_REPLICAS
```

You can run this command for each pool.

– The **`NUMBER_OF_REPLICAS`** parameter includes the object itself. If you want to include the object and two copies of the object for a total of three instances of the object, specify `3`.
Example

```
[ceph: root@host01 /]# ceph osd pool set data size 3
```

– An object might accept I/O operations in degraded mode with fewer replicas than specified by the `pool size` setting. To set a minimum number of required replicas for I/O, use the `min_size` setting.
Example

```
ceph osd pool set data min_size 2
```

This ensures that no object in the data pool will receive I/O with fewer replicas than specified by the `min_size` setting.

Getting the number of object replicas

Learn how to get the number of object replicas.

To get the number of object replicas, run the following command:

```
ceph osd dump | grep 'replicated size'
```

Ceph will list the pools, with the `replicated size` attribute highlighted. By default, Ceph creates two replicas of an object, that is a total of three copies, or a size of `3`.

Pool values

This information contains key-values pairs that you can set or get.

For further information, see “Setting pool values” on page 161 and “Getting pool values” on page 161.

Table 26: Pool values

Value	Description	Type	Required	Default
size	Specifies the number of replicas for objects in the pool. See the “Setting the number of object replicas” on page 163 section for further details. Applicable for the replicated pools only.	Integer	No	None

Value	Description	Type	Required	Default
min_size	Specifies the minimum number of replicas required for I/O. See the "Setting the number of object replicas" on page 163 section for further details. For erasure-coded pools, this should be set to a value greater than k. If I/O is allowed at the value k, then there is no redundancy and data is lost in the event of a permanent OSD failure. For more information, see "Erasure code pools overview" on page 169.	Integer	No	None
crash_replay_interval	Specifies the number of seconds to allow clients to replay acknowledged, but uncommitted requests.	Integer	No	None
pg_num	The total number of placement groups for the pool. See the Pools, placement groups, and CRUSH configuration options for details on calculating a suitable number. **Note:** The default value 8 is not suitable for most systems.	Integer	Yes	8

Value	Description	Type	Required	Default
pgp_num	The total number of placement groups for placement purposes. This should be equal to the total number of placement groups, except for placement group splitting scenarios. Valid range: Equal to or less than what specified by the `pg_num` variable.	Integer	Yes. Picks up default or Ceph configuration value if not specified.	None
crush_rule	The rule to use for mapping object placement in the cluster.	String	No	None
hashpspool	Enable or disable the **HASHPSPOOL** flag on a given pool. With this option enabled, pool hashing and placement group mapping are changed to improve the way pools and placement groups overlap. Valid settings: `1` enables the flag, `0` disables the flag. **Important:** Do not enable this option on production pools of a cluster with a large amount of OSDs and data. All placement groups in the pool would have to be remapped causing too much data movement.	Integer	No	None

Value	Description	Type	Required	Default
fast_read	On a pool that uses erasure coding, if this flag is enabled, the read request issues subsequent reads to all shards, and waits until it receives enough shards to decode to serve the client. In the case of the `jerasure` and `isa erasure` plug-ins, once the first K replies return, the client's request is served immediately using the data decoded from these replies. This helps to allocate some resources for better performance. Currently this flag is only supported for erasure coding pools.	Boolean	No	`0`
allow_ec_overwrites	Whether writes to an erasure coded pool can update part of an object, so the Ceph Filesystem and Ceph Block Device can use it.	Boolean	No	None
compression_algorithm	Sets inline compression algorithm to use with the BlueStore storage backend. This setting overrides the **`bluestore_compression_algorithm`** configuration setting. Valid settings: `lz4`, `snappy`, `zlib`, `zstd`	String	No	None

Value	Description	Type	Required	Default
compression_mode	Sets the policy for the inline compression algorithm for the BlueStore storage backend. This setting overrides the **bluestore_compression_algorithm** configuration setting. Valid settings: `none`, `passive`, `aggressive`, `force`	String	No	None
compression_min_blob_size	BlueStore does not compress chunks smaller than this size. This setting overrides the **bluestore_compression_min_blob_size** configuration setting.	Unsigned Integer	No	None
compression_max_blob_size	BlueStore breaks chunks larger than this size into smaller blobs of **compression_max_blob_size** before compressing the data.	Unsigned Integer	No	None
nodelete	Set or unset the **NODELETE** flag on a given pool. Valid settings: `1` enables the flag, `0` disables the flag.	Integer	No	None
nopgchange	Set or unset the **NOPGCHANGE** flag on a given pool.	Integer	No	None
nosizechange	Set or unset the **NOSIZECHANGE** flag on a given pool. Valid settings: `1` enables the flag, `0` disables the flag.	Integer	No	None
write_fadvise_dontneed	Set or unset the **WRITE_FADVISE_DONTNEED** flag on a given pool. Valid settings: `1` enables the flag, `0` disables the flag.	Integer	No	None

Value	Description	Type	Required	Default
noscrub	Set or unset the **NOSCRUB** flag on a given pool. Valid settings: `1` enables the flag, `0` disables the flag.	Integer	No	None
nodeep-scrub	Set or unset the **NODEEP_SCRUB** flag on a given pool. Valid settings: `1` enables the flag, `0` disables the flag.	Integer	No	None
scrub_min_interval	The minimum interval in seconds for pool scrubbing when load is low. If it is `0`, Ceph uses the **osd_scrub_min_interval** configuration setting.	Double	No	`0`
scrub_max_interval	The maximum interval in seconds for pool scrubbing irrespective of cluster load. If it is `0`, Ceph uses the **osd_scrub_max_interval** configuration setting.	Double	No	`0`
deep_scrub_interval	The interval in seconds for pool *deep scrubbing*. If it is `0`, Ceph uses the `osd_deep_scrub_interval` configuration setting.	Double	No	`0`
peering_crush_bucket_count	The value is used along with peering_crush_bucket_barrier to determined whether the set of OSDs in the chosen acting set can peer with each other, based on the number of distinct buckets there are in the acting set.	Integer	No	None

Value	Description	Type	Required	Default
peering_crush_bucket_target	This value is used along with peering_crush_bucket_barrier and size to calculate the value `bucket_max` which limits the number of OSDs in the same bucket from getting chose to be in the acting set of a PG.	Integer	No	None
peering_crush_bucket_barrier	The type of bucket a pool is stretched across. For example, rack, row, or datacenter.	String	No	None

Erasure code pools overview

Ceph uses replicated pools by default, meaning that Ceph copies every object from a primary OSD node to one or more secondary OSDs. The erasure-coded pools reduce the amount of disk space required to ensure data durability but it is computationally a bit more expensive than replication.

Ceph storage strategies involve defining data durability requirements. Data durability means the ability to sustain the loss of one or more OSDs without losing data.

Ceph stores data in pools and there are two types of the pools:

- replicated
- erasure-coded

Erasure coding is a method of storing an object in the Ceph storage cluster durably where the erasure code algorithm breaks the object into data chunks (`k`) and coding chunks (`m`), and stores those chunks in different OSDs.

In an event of the failure of an OSD, Ceph retrieves the remaining data (`k`) and coding (`m`) chunks from the other OSDs and the erasure code algorithm restores the object from those chunks.

> **Note:** Use `min_size` for erasure-coded pools to be `K+1` or more to prevent loss of writes and data. For a 2+2 pool, the `min_size` should be k+1 or 3.

Erasure coding uses storage capacity more efficiently than replication. The n-replication approach maintains `n` copies of an object (3 times by default in Ceph), whereas erasure coding maintains only `k + m` chunks. For example, 3 data and 2 coding chunks use 1.5 times the storage space of the original object.

While erasure coding uses less storage overhead than replication, the erasure code algorithm uses more RAM and CPU than replication when it accesses or recovers objects. Erasure coding is advantageous when data storage must be durable and fault tolerant, but do not require fast read performance (for example, cold storage, historical records, and so on).

For the mathematical and detailed explanation on how erasure code works in Ceph, see the Erasure coding.

Ceph creates a default erasure code profile when initializing a cluster with **k=2** and **m=2**. This means that Ceph spreads the object data over four OSDs **(k+m = 4)** and Ceph can lose up to two of those OSDs without losing data. To know more about erasure code profiling see “Erasure code profiles” on page 170 section.

> **Important:** Configure only the `.rgw.buckets` pool as erasure-coded and all other Ceph Object Gateway pools as replicated, otherwise an attempt to create a new bucket fails with the following error:

```
set_req_state_err err_no=95 resorting to 500
```

The reason for this is that erasure-coded pools do not support the `omap` operations and certain Ceph Object Gateway metadata pools require the `omap` support.

Creating a sample erasure-coded pool

Create an erasure-coded pool and specify the placement groups.

About this task

The **`ceph osd pool create`** command creates an erasure-coded pool with the default profile, unless another profile is specified. Profiles define the redundancy of data by setting two parameters, **k**, and **k**. These parameters define the number of chunks a piece of data is split and the number of coding chunks are created.

The simplest erasure-coded pool is equivalent to RAID5 and requires at least four hosts. You can create an erasure-coded pool with 2+2 profile.

Procedure

1. Set the following configuration for an erasure-coded pool on four nodes with 2+2 configuration.

   ```
   ceph config set mon mon_osd_down_out_subtree_limit host
   ```

   Important: This is not needed for an erasure-coded pool in general.

   Note: For an EC cluster with four nodes the value of K+M is 2+2. If a node fails completely, it does not recover as four chunks and only three nodes are available. When you set the value of **`mon_osd_down_out_subtree_limit`** to `host`, during a host down scenario, it prevents the OSDs from marked out, so as to prevent the data from re balancing and the waits until the node is up again.

2. For an erasure-coded pool with a 2+2 configuration, set the profile.

   ```
   ceph osd erasure-code-profile set ec22 k=2 m=2 crush-failure-domain=host
   ```

   ```
   [ceph: root@host01 /]# ceph osd erasure-code-profile set ec22 k=2 m=2 crush-failure-
   domain=host

   Pool : ceph osd pool create test-ec-22 erasure ec22
   ```

3. Create an erasure-coded pool.

   ```
   ceph osd pool create ecpool 32 32 erasure
   pool 'ecpool' created
   $ echo ABCDEFGHI | rados --pool ecpool put NYAN -
   $ rados --pool ecpool get NYAN -
   ABCDEFGHI
   ```

   `32` is the number of placement groups.

Erasure code profiles

Ceph defines an erasure-coded pool with a *profile*. Ceph uses a profile when creating an erasure-coded pool and the associated CRUSH rule.

Ceph creates a default erasure code profile when initializing a cluster and it provides the same level of redundancy as two copies in a replicated pool. This default profile defines **k=2** and **m=2**, meaning Ceph spreads the object data over four OSDs (**k+m=4**) and Ceph can lose one of those OSDs without losing data. EC 2+2

requires a minimum deployment footprint of 4 nodes (5 nodes recommended) and can cope with the temporary loss of 1 OSD node.

To display the default profile use the following command:

```
$ ceph osd erasure-code-profile get default
k=2
m=2
plugin=jerasure
technique=reed_sol_van
```

You can create a new profile to improve redundancy without increasing raw storage requirements. For instance, a profile with **k=8** and **m=4** can sustain the loss of four (**m=4**) OSDs by distributing an object on 12 (**k+m=12**) OSDs. Ceph divides the object into **8** chunks and computes **4** coding chunks for recovery. For example, if the object size is 8 MB, each data chunk is 1 MB and each coding chunk has the same size as the data chunk, that is also 1 MB. The object is not lost even if four OSDs fail simultaneously.

The most important parameters of the profile are `k`, `m`, and `crush-failure-domain`, because they define the storage overhead and the data durability.

> **Important:** Choosing the correct profile is important because you cannot change the profile after you create the pool. To modify a profile, you must create a new pool with a different profile and migrate the objects from the old pool to the new pool.

For instance, if the desired architecture must sustain the loss of two racks with a storage overhead of 40% overhead, the following profile can be defined:

```
$ ceph osd erasure-code-profile set myprofile \
   k=4 \
   m=2 \
   crush-failure-domain=rack
$ ceph osd pool create ecpool 12 12 erasure *myprofile*
$ echo ABCDEFGHIJKL | rados --pool ecpool put NYAN -
$ rados --pool ecpool get NYAN -
ABCDEFGHIJKL
```

The primary OSD will divide the *NYAN* object into four (*k=4*) data chunks and create two additional chunks (*m=2*). The value of *m* defines how many OSDs can be lost simultaneously without losing any data. The *crush-failure-domain=rack* will create a CRUSH rule that ensures no two chunks are stored in the same rack.

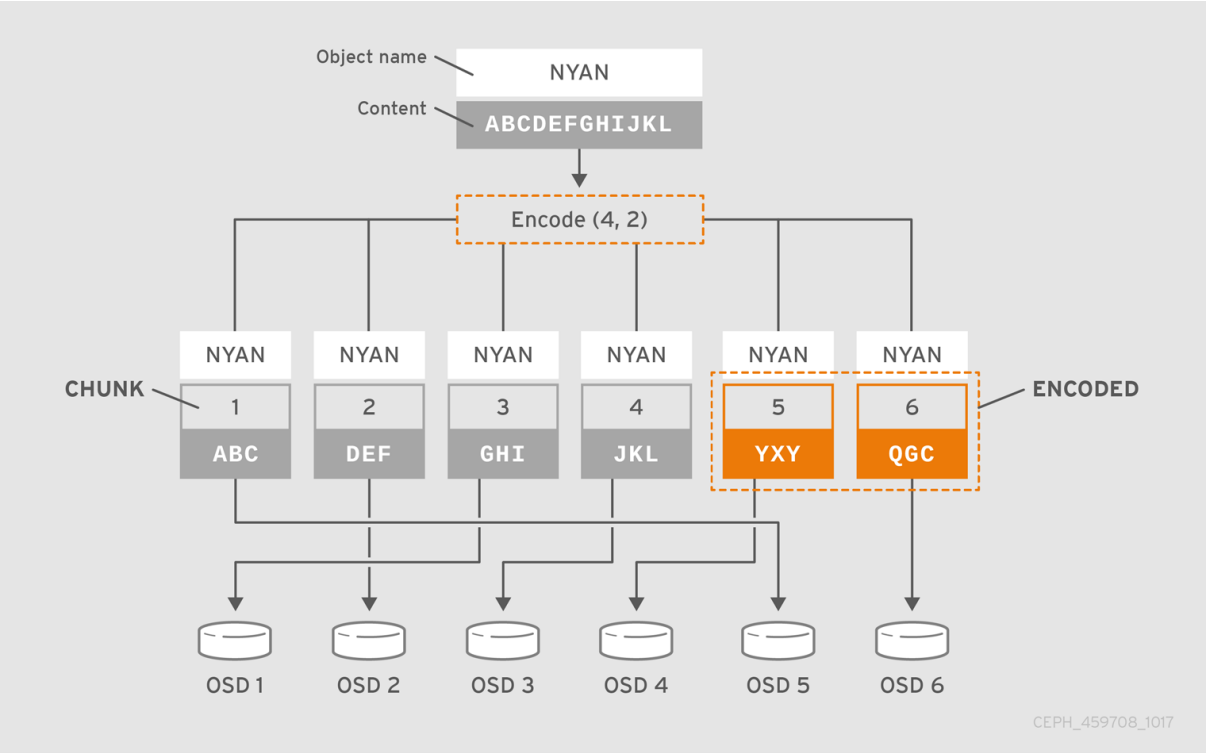


Figure 31: Erasure code

Important:

- The following *jerasure* coding values are supported for *k*, and *m*:
  - k=8 m=3
  - k=8 m=4
  - k=4 m=2
- If the number of OSDs lost equals the number of coding chunks (`m`), some placement groups in the erasure coding pool will go into incomplete state. If the number of OSDs lost is less than `m`, no placement groups will go into incomplete state. In either situation, no data loss will occur. If placement groups are in incomplete state, temporarily reducing `min_size` of an erasure coded pool will allow recovery.

Supported erasure coding profiles lists the supported erasure coding profiles.

Table 27: Supported erasure coding profiles

k	M	Supported Plugins	Minimum OSD Hosts (K+M)	Recommended OSD Hosts	Minimum OSDs	Redundancy Domain
2	2	jerasure, isa	4	4 + hosts (K+M)	4 (1 per node)	Loss of 1 nod 2 OSDs
2	2	jerasure, isa	5	5 + hosts(K+M+1)	5 (1 per node)	Loss of up to OSDs

k	M	Supported Plugins	Minimum OSD Hosts (K+M)	Recommended OSD Hosts	Minimum OSDs	Redundancy Domain
4	2	jerasure, isa	6	7 + hosts (K+M+1)	7 (1 per node)	Loss of up to OSDs
5	2	jerasure, isa	7	8 + hosts (K+M+1)	8 (1 per node)	Loss of up to OSDs
6	2	jerasure, isa	8	9 + hosts (K+M+1)	9 (1 per node)	Loss of up to OSDs
8	3	jerasure, isa	11	12 + hosts (K+M+1)	12 (1 per node)	Loss of up to OSDs
8	4	jerasure, isa	12	13 + hosts (K+M+1)	13 (1 per node)	Loss of up to OSDs
8	6	jerasure, isa	4	4 hosts{}	4 (4 per node)	Loss of 1 nod OR 6 OSDs

Note:
For clusters initialized on Red Hat Ceph Storage 9 or later, the default erasure code profile uses the ISA-L plugin instead of Jerasure.

For clusters upgraded to 9, the existing default Jerasure profile is preserved. You can create new profiles using either the ISA-L or Jerasure plugin depending on performance and compatibility requirements.

Creating a new `erasure-code-profile`

Learn how to create a new erasure code profile.

To create a new erasure code profile, use the following syntax, with the value information provided in "Erasure code profile values" on page 174.

```
ceph osd erasure-code-profile set NAME \
         [<directory=DIRECTORY>] \
         [<plugin=PLUGIN>] \
         [<stripe_unit=STRIPE_UNIT>] \
         [<CRUSH_DEVICE_CLASS>]\
         [<CRUSH_FAILURE_DOMAIN>]\
         [<key=value> ...] \
         [--force]
```

Table 28: Erasure code profile values

Value	Description	Type	Required	Default
`directory`	Set the directory name from which the erasure code plug-in is loaded.	String	No.	`/usr/lib/ceph/erasure-code`
`plugin`	Use the erasure code plug-in to compute coding chunks and recover missing details. For more information, see "Erasure code plugins" on page 178.	String	No.	`isa` (for new clusters created on Ceph 9.0 or later), `jerasure` (for clusters upgraded from earlier versions) Jerasure
`stripe_unit`	The amount of data in a data chunk, per stripe. For example, a profile with 2 data chunks and `stripe_unit=4K` would put the range 0-4K in chunk 0, 4K-8K in chunk 1, then 8K-12K in chunk 0 again. This should be a multiple of 4K for best performance. The default value is taken from the monitor config option `osd_pool_erasure_code_stripe_unit` when a pool is created. The stripe_width of a pool using this profile will be the number of data chunks multiplied by this `stripe_unit`.	String	No.	`4K`
`crush-device-class`	The device class, such as `hdd` or `ssd`.	String	No.	`none`, meaning CRUSH uses all devices regardless of class.
`crush-failure-domain`	The failure domain, such as `host` or `rack`.	String	No.	`host`
`key`	The semantic of the remaining key-value pairs is defined by the erasure code plug-in.	String	No.	N/A

Value	Description	Type	Required	Default
`--force`	Override an existing profile by the same name.	String	No.	N/A

Note: The plugin default depends on how the cluster was created. New 9.0 clusters use ISA-L; upgraded clusters continue with Jerasure unless specified otherwise.

Removing an existing `erasure-code-profile`

Learn how to remove an erasure code profile.

Procedure

- Remove an erasure code profile.

```
ceph osd erasure-code-profile rm NAME
```

where *NAME* is the name of the profile.

Important: If the profile is referenced by a pool, the deletion fails.

Warning: Removing an erasure code profile using **`osd erasure-code-profile rm`** command does not automatically delete the associated CRUSH rule that is associated with the erasure code profile. Manually remove the associated CRUSH rule using **`ceph osd crush rule remove RULE_NAME`** command to avoid unexpected behavior.

Fetching details for an `erasure-code-profile`

Learn how to display an erasure code profile.

To display an erasure code profile:

Syntax

```
ceph osd erasure-code-profile get NAME
```

where *NAME* is the name of the profile.

Listing all `erasure-code-profile` on cluster

Learn how to list the names of all erasure code profiles.

To list the names of all erasure code profiles:

Syntax

```
ceph osd erasure-code-profile ls
```

Erasure coding with overwrites

By default, erasure coded pools only work with the Ceph Object Gateway, which performs full object writes and appends.

Using erasure coded pools with overwrites allows Ceph Block Devices and CephFS store their data in an erasure coded pool:

Syntax

```
ceph osd pool set ERASURE_CODED_POOL_NAME allow_ec_overwrites true
```

Example

```
[ceph: root@host01 /]# ceph osd pool set ec_pool allow_ec_overwrites true
```

Enabling erasure coded pools with overwrites can only reside in a pool using BlueStore OSDs. Since BlueStore's checksumming is used to detect bit rot or other corruption during deep scrubs. Using FileStore with erasure coded overwrites is unsafe, and yields lower performance when compared to BlueStore.

Erasure coded pools do not support omap. To use erasure coded pools with Ceph Block Devices and CephFS, store the data in an erasure coded pool, and the metadata in a replicated pool.

For Ceph Block Devices, use the `--data-pool` option during image creation:

Syntax

```
rbd create --size IMAGE_SIZE_M|G|T --data-pool _ERASURE_CODED_POOL_NAME
REPLICATED_POOL_NAME/IMAGE_NAME
```

Example

```
[ceph: root@host01 /]# rbd create --size 1G --data-pool ec_pool rep_pool/image01
```

If using erasure coded pools for CephFS, then setting the overwrites must be done in a file layout.

Erasure coding optimizations

Erasure coding (EC) optimizations improve performance for small I/O operations and reduce storage overhead in erasure-coded pools by eliminating padding.

Starting with the Red Hat Ceph Storage 9 release, EC optimizations are enabled by default for newly created pools. This behavior is controlled by the global flag described below.

Workload impact
EC optimizations affect workloads differently depending on how applications store and access data:

RADOS Block Device (RBD) and Ceph File System (CephFS)

Gain higher performance and reduced capacity usage.

Ceph Object Gateway with large sequential objects

Experience minimal benefit.

Ceph Object Gateway with many small objects or random access

Gain improved performance and reduced capacity usage.

Configuration
When you enable EC optimizations, you cannot disable them because they change how data is stored. You can configure EC optimizations at either the pool level or the global level:

Pool level

Use the `allow_ec_optimizations` option to enable EC optimizations for a specific pool.

Global level

Use the `osd_pool_default_flag_ec_optimizations` option to set EC optimizations as the default for all newly created erasure-coded pools.

Requirements
You must meet the following requirements before you enable EC optimizations:

- Upgrade all Monitors and OSDs to Red Hat Ceph Storage 9 or later.
- Clients and gateways do not require upgrading.
- Supported with the following EC config combinations:
  Plugin

  Jerasure — Technique: `reed_sol_van`

Plugin

ISA-L — Technique: `reed_sol_van` or `cauchy`

> **Note:** Any EC pool created with the above plugins and technique will support EC enhancements, irrespective of `k, m` values. The appropriate technique is automatically selected based on the `k,m` configuration.

Limitations
Before enabling EC optimizations, review the following limitations to avoid configuration errors:

- Red Hat Ceph Storage blocks the operation with an error if you attempt to enable EC optimizations with an unsupported plugin or technique.

Enabling coding optimizations

Enable erasure coding (EC) optimizations, also known as Fast EC, on a pool to improve performance for small I/O operations and reduce capacity usage.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- All monitors and OSDs are upgraded to Red Hat Ceph Storage 9 or later.
- The erasure-coded pool is configured using one of the following supported combinations:
  Plugin

  Jerasure — Technique: `reed_sol_van`

  Plugin

  ISA-L — Technique: `reed_sol_van` or `cauchy`

  > **Note:** Any EC pool created with the above plugins and technique will support EC enhancements, irrespective of `k, m` values. The appropriate technique is automatically selected based on the `k,m` configuration.

- Gateways and clients do not require upgrading.

Procedure

1. Run the following command to enable EC optimizations on a pool.

   ```
   ceph osd pool set EC_POOL allow_ec_optimizations true
   ```

2. Configure optimizations as the default for new pools by setting the **`osd_pool_default_flag_ec_optimizations`** configuration parameter.

   > **Note:** Starting with Red Hat Ceph Storage 9, this parameter is set to `true` by default. This enables EC optimizations automatically for newly created pools that use supported erasure-coded configurations.

   For example,

   ```
   ceph config set global osd_pool_default_flag_ec_optimizations true
   ```

Result

The pool is optimized for workloads with small objects or random read/write operations. EC pools become more suitable for Ceph Block Device and CephFS workloads. Ceph Object Gateway workloads with many small objects benefit, while large sequential objects see minimal improvement.

Note:

- Once enabled, EC optimizations cannot be disabled because they change how data is stored.
- EC optimizations are supported only with the Jerasure and ISA-L plugins using the reed_sol_van or cauchy technique. Unsupported combinations are blocked with an error.

Erasure code plugins

Ceph supports erasure coding with a plug-in architecture, which means you can create erasure-coded pools by using different erasure coding algorithms.

Starting with Red Hat Ceph Storage 9, the default erasure code plug-in for new clusters changes from Jerasure to ISA-L. ISA-L (Intel® Intelligent Storage Acceleration Library) provides optimized Reed-Solomon–based erasure coding with improved encoding and recovery performance on modern CPUs.Clusters that are upgraded to Red Hat Ceph Storage 9 continue to use the existing Jerasure-based default profile. You can still create custom profiles using either plug-in.

Ceph supports the following erasure code plug-ins:

- ISA-L - Default for new clusters created in Red Hat Ceph Storage 9 and later.
- Jerasure - Default for clusters created before Red Hat Ceph Storage 9 and still supported.
- Intel Storage Acceleration (ISA) library (Default) Jerasure

Creating a new erasure code profile using Jerasure erasure code plugin

The ISA-L plugin provides high-performance Reed-Solomon erasure coding using the Intel® Intelligent Storage Acceleration Library. Starting with Red Hat Ceph Storage 9, new clusters created on this release use ISA-L as the default erasure code plugin.

The ISA-L plug-in encapsulates the JerasureH library. For detailed information about the parameters, see the Jerasure documentation.

To create a new erasure code profile using the Jerasure plug-in, run the following command, with the value information provided in “New EC profile using Jerasure values” on page 178:

```
ceph osd erasure-code-profile set NAME \
     plugin=jerasure \
     k=DATA_CHUNKS \
     m=DATA_CHUNKS \
     technique=TECHNIQUE \
     [crush-root=ROOT] \
     [crush-failure-domain=BUCKET_TYPE] \
     [directory=DIRECTORY] \
     [--force]
```

Table 29: New EC profile using Jerasure values

Value	Description	Type	Required	Example/Default
k	Each object is split in data-chunks parts, each stored on a different OSD.	Integer	Yes.	Example: 4

Value	Description	Type	Required	Example/Default
`m`	Compute coding chunks for each object and store them on different OSDs. The number of coding chunks is also the number of OSDs that can be down without losing data.	Integer	Yes.	Example: 2
`technique`	The more flexible technique is **reed_sol_van**; it is enough to set `k` and `m`. The **cauchy_good** technique can be faster but you need to choose the `packetsize` carefully. All of `reed_sol_r6_op`, `liberation`, `blaum_roth`, `liber8tion` are RAID6 equivalents in the sense that they can only be configured with `m=2`. Valid settings are as follows: `reed_sol_van`, `reed_sol_r6_op`, `cauchy_orig`, `cauchy_good`, `liberation`, `blaum_roth`, `liber8tion`	String	No.	Default: `reed_sol_van`
`packetsize`	The encoding will be done on packets of bytes size at a time. Choosing the correct packet size is difficult. The Jerasure documentation contains extensive information on this topic.	Integer	No.	Default: `2048`
`crush-root`	The name of the CRUSH bucket used for the first step of the rule. For instance **step take default**.	String	No.	Default: default

Value	Description	Type	Required	Example/Default
`crush-failure-domain`	Ensure that no two chunks are in a bucket with the same failure domain. For instance, if the failure domain is `host`, no two chunks will be stored on the same host. It is used to create a rule step such as step chooseleaf host.	String	No.	Default: `host`
`directory`	Set the **directory** name from which the erasure code plug-in is loaded.	String	No.	Default: `/usr/lib/ceph/erasure-code`
`--force`	Override an existing profile by the same name.	String	No.	N/A

Creating a new erasure code profile using the ISA-L erasure code plugin

The ISA-L plugin provides high-performance Reed-Solomon erasure coding using the Intel® Intelligent Storage Acceleration Library. Starting with Red Hat Ceph Storage 9, new clusters created on this release use ISA-L as the default erasure code plugin. Clusters upgrading to Red Hat Ceph Storage 9 continue using their existing Jerasure-based default profile, but they may create new ISA-L or Jerasure profiles as needed.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- The cluster is running Red Hat Ceph Storage 9 or later.
- You have administrator privileges.
- All OSD nodes have ISA-L support installed.

Procedure

1. To create an ISA-L–based erasure code profile, run the following commands:

```
ceph osd erasure-code-profile set <NAME> \
     plugin=isa \
     k=DATA_CHUNKS \
     m=CODING_CHUNKS \
     technique=TECHNIQUE \
     [crush-root=ROOT] \
     [crush-failure-domain=BUCKET_TYPE] \
     [directory=DIRECTORY] \
     [--force]
```

   The following example creates a 4+2 ISA-L profile:

```
ceph osd erasure-code-profile set isal_4_2 \
     plugin=isa \
     k=4 \
     m=2 \
     technique=reed_sol_van \
     crush-failure-domain=host
```

2. Create a pool using the profile:

```
ceph osd pool create ecpool 12 12 erasure isal_4_2
```

Result

The ISA-L erasure code profile is created and available for use. You can verify the profile by running the following command:

```
ceph osd erasure-code-profile get <NAME>
```

Controlling CRUSH placement

Learn how to control CRUSH placement.

The default CRUSH rule provides OSDs that are on different hosts. For instance:

```
chunk nr    01234567

step 1      _cDD_cDD
step 2      cDDD____
step 3      ____cDDD
```

needs exactly *8* OSDs, one for each chunk. If the hosts are in two adjacent racks, the first four chunks can be placed in the first rack and the last four in the second rack. Recovering from the loss of a single OSD does not require using bandwidth between the two racks.

For instance:

```
crush-steps='[ [ "choose", "rack", 2 ], [ "chooseleaf", "host", 4 ] ]'
```

creates a rule that selects two crush buckets of type *rack* and for each of them choose four OSDs, each of them located in a different bucket of type *host*.

The rule can also be created manually for finer control.

Installing Red Hat Ceph Storage

This information provides instructions on installing Red Hat Ceph Storage on Red Hat Enterprise Linux running on AMD64 and Intel 64 architectures.

Before starting to install Red Hat Ceph Storage, see Installation and upgrade requirements.

Initial installation

Use the `cephadm` utility to deploy new Red Hat Ceph Storage clusters.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Operating System swap must not be enabled.
- At least one running virtual machine (VM) or bare-metal server with an active internet connection.
- Red Hat Enterprise Linux 9.6, 9.7, 10, or 10.1 with `ansible-core` available from the corresponding RHEL AppStream repository.
- A valid Red Hat subscription with the appropriate entitlements.
- Root-level access to all nodes.
- An active Red Hat Network or service account to access the Red Hat Registry.
- Remove troubling configurations in iptables so that refresh of iptables services does not cause issues to the cluster. For an example, see “Verifying firewall rules are configured for default Ceph ports” on page 294.

For the latest supported Red Hat Enterprise Linux versions, see Compatibility matrix.

About this task

The `cephadm` utility manages the entire life cycle of a Ceph cluster. Installation and management tasks comprise two types of operations:

- Day One operations involve installing and bootstrapping a bare-minimum, containerized Ceph storage cluster, running on a single node. Day One also includes deploying the Monitor and Manager daemons and adding Ceph OSDs.
- Day Two operations use the Ceph orchestration interface, `cephadm orch`, or the Red Hat Ceph Storage Dashboard to expand the storage cluster by adding other Ceph services to the storage cluster.

> **Important:** Operating System swap must not be enabled. Configuring the operating system to use swap to provide additional virtual memory for daemons is not recommended for modern systems and can result in degraded performance.

`cephadm` utility

The `cephadm` utility deploys and manages a Ceph storage cluster. It is tightly integrated with both the command-line interface (CLI) and the Red Hat Ceph Storage Dashboard web interface so that you can manage storage clusters from either environment. `cephadm` uses SSH to connect to hosts from the manager daemon to add, remove, or update Ceph daemon containers. It does not rely on external configuration or orchestration tools such as Ansible or Rook.

> **Note:** The `cephadm` utility is available after running the preflight playbook on a host.

The `cephadm` utility consists of two main components:

- The `cephadm` shell.
- The `cephadm` orchestrator.

The `cephadm` shell

The `cephadm` shell starts a `bash` shell within a container. Use the shell to complete "Day One" cluster setup tasks, such as installation and bootstrapping, and to use `ceph` commands.

For more information about how to start the `cephadm` shell, see "Using the cephadm shell" on page 217.

The `cephadm` orchestrator

Use the `cephadm` orchestrator to perform "Day Two" Ceph functions, such as expanding the storage cluster and provisioning Ceph daemons and services. You can use the `cephadm` orchestrator through either the command-line interface (CLI) or the web-based Red Hat Ceph Storage Dashboard. Orchestrator commands take the form `ceph orch`.

The `cephadm` script interacts with the Ceph orchestration module that is used by the Ceph Manager.

How `cephadm` works

Use the `cephadm` command to manage the lifecycle of a Red Hat Ceph Storage cluster.

The `cephadm` command can perform the following operations:

- Bootstrap a new Red Hat Ceph Storage cluster.
- Launch a containerized shell that works with the Red Hat Ceph Storage command-line interface (CLI).
- Debug containerized daemons.

The `cephadm` command uses `ssh` to communicate with the nodes in the storage cluster. This allows you to add, remove, or update Red Hat Ceph Storage containers without using external tools. Generate the `ssh` key pair during the bootstrapping process, or use your own `ssh` key.

The `cephadm` bootstrapping process creates a small storage cluster on a single node, consisting of one Ceph Monitor and one Ceph Manager, as well as any required dependencies. You then use the orchestrator CLI or the Red Hat Ceph Storage Dashboard to expand the storage cluster to include nodes, and to provision all of the Red Hat Ceph Storage daemons and services. You can perform management functions through the CLI or from the Red Hat Ceph Storage Dashboard web interface.

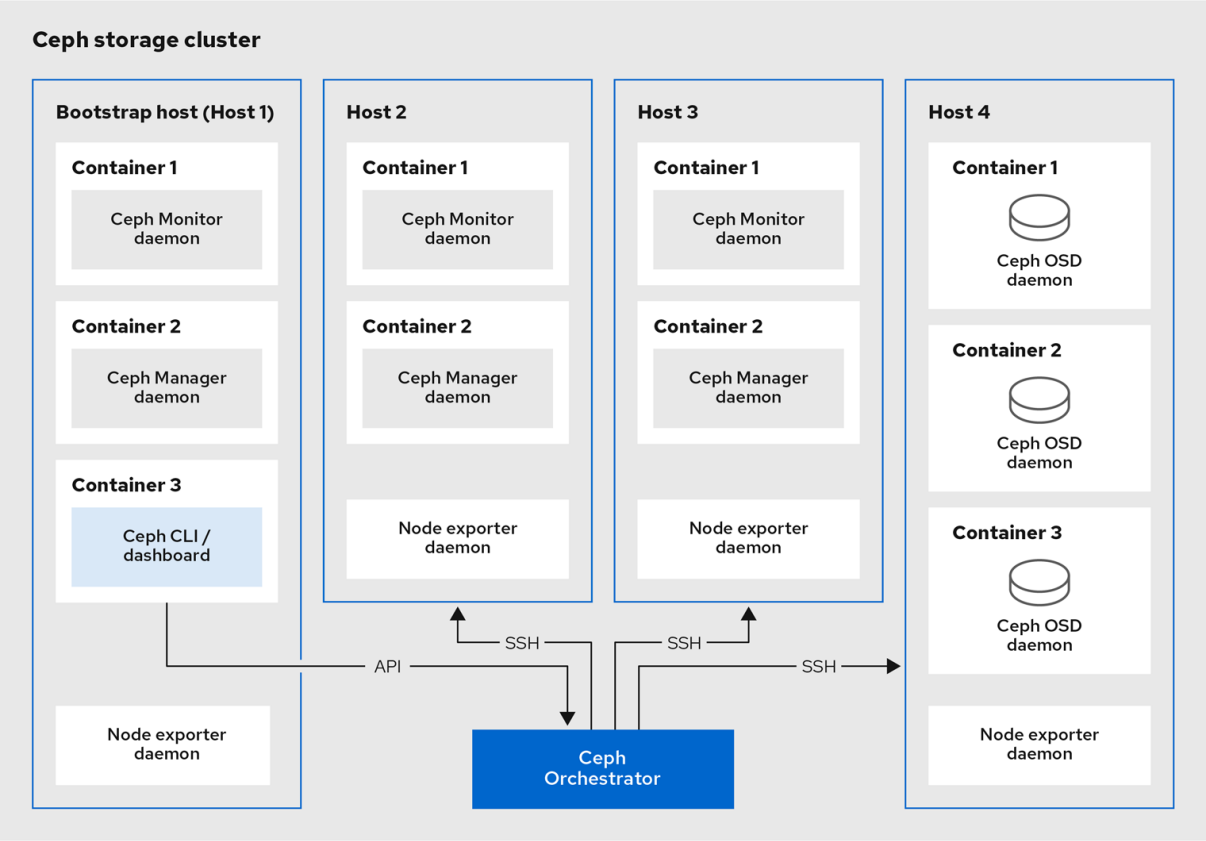


Figure 32: Ceph storage cluster deployment

`cephadm-ansible` playbooks

The `cephadm-ansible` package is a collection of Ansible playbooks to simplify workflows that are not covered by `cephadm`. After installation, the playbooks are located in `/usr/share/cephadm-ansible/`.

The `cephadm-ansible` package includes the following playbooks:

- `cephadm-preflight.yml`
- `cephadm-clients.yml`
- `cephadm-purge-cluster.yml`

The `cephadm-preflight` playbook

Use the `cephadm-preflight` playbook to initially setup hosts before bootstrapping the storage cluster and before adding new nodes or clients to your storage cluster. This playbook configures the Ceph repository and installs some prerequisites such as `podman`, `lvm2`, `chrony`, and `cephadm`.
For more information, see Running the preflight playbook.

The `cephadm-clients` playbook

Use the `cephadm-clients` playbook to set up client hosts. This playbook handles the distribution of configuration and keyring files to a group of Ceph clients.
For more information, see “Deploying client nodes” on page 243.

The `cephadm-purge-cluster` playbook

Use the `cephadm-purge-cluster` playbook to remove a Ceph cluster. This playbook purges a Ceph cluster that is managed with cephadm.
For more information, see Purging the Ceph storage cluster.

Registering the Red Hat Ceph Storage nodes

Register the Red Hat Ceph Storage nodes on Red Hat Enterprise Linux.

Before you begin

Before registering the Red Hat Ceph Storage nodes, be sure that you have:

- At least one running virtual machine (VM) or bare-metal server with an active internet connection.
- Red Hat Enterprise Linux 9.6, 9.7, 10, or 10.1 with `ansible-core` available from the corresponding RHEL AppStream repository.
- A valid Red Hat subscription with the appropriate entitlements.

  Note: If you purchased an Red Hat Ceph Storage Premium license, complete the Red Hat entitlement activation process before beginning deployment.
  For more information, see Accessing Red Hat entitlements from IBM Storage Ceph.

- Root-level access to all nodes.

Note: Red Hat now supports only Simple Content Access (SCA). As a result, customers no longer need to attach any subscription to a system. Registering the system and enabling the required repositories is now sufficient for registration use. For more information, see How to register and subscribe a RHEL system to the Red Hat Customer Portal using Red Hat Subscription-Manager? on the Red Hat Customer Portal.

For the latest supported Red Hat Enterprise Linux versions, see Compatibility matrix.

Procedure

1. Register the system, and when prompted, enter your Red Hat customer portal credentials.

   ```
   subscription-manager register
   ```

   For example,

   ```
   [root@admin ~]# subscription-manager register

   Registering to: subscription.rhsm.redhat.com:443/subscription
   Username: USERNAME
   Password: PASSWORD
   The system has been registered with ID: ID
   The registered system name is: SYSTEM_NAME
   ```

2. Pull the latest subscription.

   ```
   subscription-manager refresh
   ```

   For example,

   ```
   [root@admin ~]# subscription-manager refresh
   All local data refreshed
   ```

3. Disable the software repositories.

   ```
   subscription-manager repos --disable=*
   ```

   For example,

   ```
   [root@admin ~]# subscription-manager repos --disable=*
   ```

4. Enable the Red Hat Enterprise Linux BaseOS and appstream repositories.
   - Enabling repositories with Red Hat Enterprise Linux 9.x:

```
subscription-manager repos --enable=rhel-9-for-x86_64-baseos-rpms
```

```
subscription-manager repos --enable=rhel-9-for-x86_64-appstream-rpms
```

- Enabling repositories with Red Hat Enterprise Linux 10.x:

```
subscription-manager repos --enable=rhel-10-for-x86_64-baseos-rpms
```

```
subscription-manager repos --enable=rhel-10-for-x86_64-appstream-rpms
```

5. Update the system to receive the latest packages for Red Hat Enterprise Linux.

```
dnf update
```

For example,

```
[root@admin ~]# dnf update
```

6. Enable the `ceph-tools` repository.
   - Enabling repositories with Red Hat Enterprise Linux 9.x:

```
subscription-manager repos --enable=rhceph-9-tools-for-rhel-9-x86_64-rpms
```

   - Enabling repositories with Red Hat Enterprise Linux 10.x:

```
subscription-manager repos --enable=rhceph-9-tools-for-rhel-10-x86_64-rpms
```

7. Repeat steps “1” on page 185 through “6” on page 186 on all the nodes of the storage cluster.
8. Install `cephadm-ansible`.

```
dnf install cephadm-ansible
```

Configuring Ansible inventory location

You can configure inventory location files for the `cephadm-ansible` staging and production environments. The Ansible inventory hosts file contains all the hosts that are part of the storage cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- An Ansible administration node.
- Root-level access to the Ansible administration node.
- The `cephadm-ansible` package is installed on the node.

About this task

You can list nodes individually in the inventory hosts file or you can create groups such as `[mons]`,`[osds]`, and `[rgws]` to provide clarity to your inventory and ease the usage of the `--limit` option to target a group or node when running a playbook.

> **Note:** If deploying clients, client nodes must be defined in a dedicated `[clients]` group.

Procedure

1. Navigate to the `/usr/share/cephadm-ansible/` directory.

```
cd /usr/share/cephadm-ansible
```

For example,

```
[root@admin ~]# cd /usr/share/cephadm-ansible
```

2. Optional: Create subdirectories for staging and production.

```
mkdir -p inventory/staging inventory/production
```

For example,

```
[root@admin cephadm-ansible]# mkdir -p inventory/staging inventory/production
```

3. Optional: Edit the `ansible.cfg` file and add the following line to assign a default inventory location.

```
[defaults]
inventory = ./inventory/staging
```

4. Optional: Create an inventory `hosts` file for each environment.

```
touch inventory/staging/hosts
touch inventory/production/hosts
```

For example,

```
[root@admin cephadm-ansible]# touch inventory/staging/hosts
[root@admin cephadm-ansible]# touch inventory/production/hosts
```

5. Open and edit each `hosts` file and add the nodes and `[admin]` group.

```
NODE_NAME_1
NODE_NAME_2

[admin]
ADMIN_NODE_NAME_1
```

   - Replace *NODE_NAME_1* and *NODE_NAME_2* with the Ceph nodes such as monitors, OSDs, MDSs, and gateway nodes.
   - Replace *ADMIN_NODE_NAME_1* with the name of the node where the admin keyring is stored.

For example,

```
host02
host03
host04

[admin]
host01
```

> **Note:** If you set the inventory location in the `ansible.cfg` file to staging, you need to run the playbooks in the staging environment as follows:
>
> ```
> ansible-playbook -i inventory/staging/hosts PLAYBOOK.yml
> ```
>
> To run the playbooks in the production environment:
>
> ```
> ansible-playbook -i inventory/production/hosts PLAYBOOK.yml
> ```

Enabling SSH login as root user on Red Hat Enterprise Linux 9 and 10

Enable the SSH login as root user on Red Hat Enterprise Linux 9 and 10, as it is not automatically supported.

Before you begin

Before you begin, make sure that you have root-level access to all nodes.

About this task

Red Hat Enterprise Linux 9 and 10 do not support SSH login as a root user even if `PermitRootLogin` parameter is set to `yes` in the `/etc/ssh/sshd_config file`. See the following error example for using SSH login with RHEL 9.

```
[root@host01 ~]# ssh root@myhostname
root@myhostname password:
Permission denied, please try again.
```

You can run one of the following methods to enable login as a root user:

- Use the`Allow root SSH login with password`flag while setting the root password during installation of Red Hat Enterprise Linux 9.
- Manually set the `PermitRootLogin` parameter after Red Hat Enterprise Linux 9 installation.

This section describes manual setting of the `PermitRootLogin` parameter.

For more information, see Not able to login as root user via ssh in RHEL 9 server on the Red Hat Customer Portal.

Procedure

1. Open the `etc/ssh/sshd_config` file and set the `PermitRootLogin` to `yes`.
   For example,

   ```
   [root@admin ~]# echo 'PermitRootLogin yes' >> /etc/ssh/sshd_config.d/01-
   permitrootlogin.conf
   ```

2. Restart the SSH service.
   For example,

   ```
   [root@admin ~]# systemctl restart sshd.service
   ```

3. Login to the node as the `root` user.

   ```
   ssh root@HOST_NAME
   ```

   Replace HOST_NAME with the host name of the Ceph node.

   For example,

   ```
   [root@admin ~]# ssh root@host01
   ```

4. When prompted, enter the `root` password.

Creating an Ansible user with sudo access

You can create an Ansible user with password-less `root` access on all nodes in the Red Hat Ceph Storage to run the `cephadm-ansible` playbooks.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root-level access to all nodes.
- For Red Hat Enterprise 9, to log in as a root user, see “Enabling SSH login as root user on Red Hat Enterprise Linux 9 and 10” on page 187.

About this task

The Ansible user must be able to log into all the Red Hat Ceph Storage nodes as a user who has `root` privileges to install software and create configuration files without prompting for a password.

Important: Only use these steps if you are a non-root user. If you are a root user, see the "Enabling SSH login as root user on Red Hat Enterprise Linux 9 and 10" on page 187 section.

For more information about creating user accounts, see **Configuring basic system settings › Getting started with managing user accounts** in the Red Hat Enterprise Linux documentation on Red Hat Documentation.

Procedure

Complete these steps on each node in the storage cluster.

1. Log in to the node as the `root` user.

```
ssh root@HOST_NAME
```

Replace *HOST_NAME* with the host name of the Ceph node.
For example,

```
[root@admin ~]# ssh root@host01
```

Enter the `root` password when prompted.

2. Create a new Ansible user.

```
adduser USER_NAME
```

Replace *USER_NAME* with the new user name for the Ansible user.

Important: Do not use `ceph` as the user name. The `ceph` user name is reserved for the Ceph daemons. A uniform user name across the cluster can improve ease of use, but avoid using obvious user names, because intruders typically use them for brute-force attacks.

For example,

```
[root@host01 ~]# adduser ceph-admin
```

3. Set a new password for this user.

```
passwd USER_NAME
```

Replace *USER_NAME* with the new user name for the Ansible user.
For example,

```
[root@host01 ~]# passwd ceph-admin
```

Enter the new password twice when prompted.

4. Configure `sudo` access for the newly created user.

```
cat << EOF >/etc/sudoers.d/USER_NAME
$USER_NAME ALL = (root) NOPASSWD:ALL
EOF
```

Replace *USER_NAME* with the user name for the Ansible user, created in step "2" on page 189.
For example,

```
[root@host01 ~]# cat << EOF >/etc/sudoers.d/ceph-admin
ceph-admin ALL = (root) NOPASSWD:ALL
EOF
```

5. Assign the correct file permissions to the new file.

```
chmod 0440 /etc/sudoers.d/USER_NAME
```

Replace *USER_NAME* with the user name for the Ansible user, created in step “2” on page 189.
For example,

```
[root@host01 ~]# chmod 0440 /etc/sudoers.d/ceph-admin
```

Related information

Managing sudo access

Enabling password-less SSH for Ansible

Generate an SSH key pair on the Ansible administration node and distribute the public key to each node in the storage cluster so that Ansible can access the nodes without being prompted for a password.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Access to the Ansible administration node.
- Ansible user with sudo access to all nodes in the storage cluster.
- For Red Hat Enterprise 9, to log in as a root user, see “Enabling SSH login as root user on Red Hat Enterprise Linux 9 and 10” on page 187.

About this task

Important: Only use these steps if you are a non-root user. If you are a root user, go to “Bootstrapping a new storage cluster” on page 195.

Procedure

1. Generate the SSH key pair, accept the default file name and leave the passphrase empty.

   ```
   ssh-keygen
   ```

   For example,

   ```
   [ansible@admin ~]$ ssh-keygen
   ```

2. Copy the public key to all nodes in the storage cluster.

   ```
   ssh-copy-id USER_NAME@HOST_NAME
   ```

   Replace *USER_NAME* with the new user name for the Ansible user. Replace *HOST_NAME* with the host name of the Ceph node.

   For example,

   ```
   [ansible@admin ~]$ ssh-copy-id ceph-admin@host01
   ```

3. Create the user’s SSH `config` file.

   ```
   touch ~/.ssh/config
   ```

   For example,

   ```
   [ansible@admin ~]$ touch ~/.ssh/config
   ```

4. Open the `config` file for editing.
   Set values for the **`Hostname`** and **`User`** options for each node in the storage cluster.

> **Important:** By configuring the ~/.ssh/config file you do not have to specify the `-u USER_NAME` option each time you execute the **ansible-playbook** command.

```
Host host01
   Hostname HOST_NAME
   User USER_NAME
Host host02
   Hostname HOST_NAME
   User USER_NAME
...
```

Replace *HOST_NAME* with the host name of the Ceph node. Replace *USER_NAME* with the new user name for the Ansible user.

For example,

```
Host host01
   Hostname host01
   User ceph-admin
Host host02
   Hostname host02
   User ceph-admin
Host host03
   Hostname host03
   User ceph-admin
```

5. Set the correct file permissions for the ~/.ssh/config file.

```
chmod 600 ~/.ssh/config
```

Related information

ssh_config(5) man page
Using secure communications between two systems with OpenSSH in Red Hat Enterprise Linux documentation

Running the preflight playbook

The Ansible playbook configures the Ceph repository and prepares the storage cluster for bootstrapping. It also installs some prerequisites, such as podman, lvm2, chrony, and cephadm packages. The default location for cephadm-ansible and cephadm-preflight.yml is /usr/share/cephadm-ansible.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- cephadm-ansible package installed using the **dnf install cephadm-ansible** command.
- Root-level access to the Ansible administration node.
- Ansible user with sudo and password-less ssh access to all nodes in the storage cluster.

About this task

> **Important:** Run the preflight playbook before you bootstrap the initial host.

The preflight playbook uses the cephadm-ansible inventory file to identify the admin node and all other nodes in the storage cluster.

The default location for the inventory file is /usr/share/cephadm-ansible/hosts.

The following example shows the structure of a typical inventory file:

```
host02
host03
```

```
host04

[admin]
host01
```

The `[admin]` group in the inventory file contains the name of the node where the admin keyring is stored. On a new storage cluster, the node in the `[admin]` group will be the bootstrap node. To add additional admin hosts after bootstrapping the cluster see "Setting up the admin node" on page 238.

The preflight playbook provides a comprehensive set of checks and tasks to ensure optimal system configuration for Red Hat Ceph Storage deployments.

The playbook checks and tasks include the following:

- Correct operating system configurations.
  - Red Hat Enterprise Linux version.
  - The correct software profile.
  - All mandatory packages are installed. If any packages are missing, the playbook installs the mandatory missing packages.
- Correct hardware configurations.
  - CPU configuration
  - RAM configuration
  - Memory swap space
  - Network Interface Card (NIC) configurations
- The separation of `root-filesystem` and `/var`.

Important: If you are performing a disconnected installation, see "Running the preflight playbook for a disconnected installation" on page 211.

When running the preflight playbook, `cephadm-ansible` automatically installs `chrony` and `ceph-common` packages on the client nodes.

The Ansible Preflight performs a check to detect whether swap space is configured on the host. This check is informational only and does not block deployment. Operational practices vary across environments:

- Some deployments disable swap on OSD nodes to avoid increased latency under memory pressure.
- Other deployments keep swap enabled but configure a very low `vm.swappiness` value (for example,`0`or`1`) to minimize swapping activity.

Note: Specific swap configurations are not enforced. Follow the appropriate guidance for your specific environment.

To configure multiple sources or if you have a disconnected environment, see the following documentation for more information:

- "Running the preflight playbook for a disconnected installation" on page 211
- Basic chrony NTP troubleshooting

Procedure

Note: In the following procedure, `host01` is the bootstrap node.

1. Navigate to the `/usr/share/cephadm-ansible` directory.
2. Open and edit the `hosts` file and add your nodes.

For example,

```
host02
host03
host04

[admin]
host01
```

3. Run the preflight playbook.
   Run the preflight playbook either by running the playbook for all hosts in the cluster or for a selected set of hosts in the cluster.

   > **Note:** When running the preflight playbook, `cephadm-ansible` automatically installs `chrony` and `ceph-common` packages on the client nodes.

   After installation is complete, `cephadm` resides in the `/usr/sbin/` directory.

   a. Run the playbook for all hosts in the cluster.

      ```
      ansible-playbook -i INVENTORY_FILE cephadm-preflight.yml --extra-vars
      "ceph_origin=rhcs"
      ```

      A **Preflight Check Report** and **Summary** generates and highlights passed, failed, and informational checks.

      For example,

```
================================================
              **  Preflight Check Report **
================================================

 System Checks
------------------------------------------------
- OS Version: ✅ Passed

- Tuned Profile: ❌ Failed
    - Reason: Incorrect tuned profile. Expected: throughput-performance

- RHEL Profile: ❌ Failed
    - Reason: Incorrect RHEL software profile. Expected: Server with File and Storage Server.

- Firewalld Running: ✅ Passed

- Podman Installed: ✅ Passed

- SELinux: ✅ Passed

- Required Packages Installed: ✅ Passed

- Minimum RAM (8GB): ❌ Failed
    - Reason: System has only 7684 MB RAM, required: 8192MB

- Swap Space (1.5x RAM): ❌ Failed
    - Reason: System has only 5119 MB Swap, required: 11526 MB

- CPU x86-64-v2: ✅ Passed

- CPU Cores >= 4: ✅ Passed

- /var is a separate partition: ❌ Failed
    - Reason: /var is not a separate partition

- Root Filesystem >= 100GB: ❌ Failed
    - Reason: Root FS is only 43GB, required: 100GB

- NIC Configuration: ℹ️INFO
    - Reason: Available network interfaces: ens3 | Speeds (Mbps): -1

- Jumbo Frames Enabled: ❌ Failed
    - Reason: MTU is 1500, recommended > 1500

- NIC Static IP Configuration: ❌ Failed
    - Reason: NIC is using DHCP, static IP is recommended

- NIC Bandwidth (10GbE Recommended): ❌ Failed
    - Reason: NIC speed is -1 Mbps, recommended is 10GbE

- Network Latency: ℹ️INFO
    - Reason: Average latency (ms): ['0.037']

================================================
** Summary **
------------------------------------------------
ℹ️ The following checks did not meet the recommended configuration:

   - Tuned Profile
   - RHEL Profile
   - Minimum RAM
   - Swap Space
   - /var Partition
   - Root Filesystem
   - Jumbo Frames Enabled
   - NIC Static IP Configuration
   - NIC Bandwidth

   These values differ from expected baselines and may impact Ceph performance.
   Please review and adjust the system configuration.
```

Each output type in the report has a corresponding visual display. Use “Preflight Check Report key” on page 195 to understand the report.

> **Important:** All checks are information only. Failures do not stop the check from running.

Table 30: Preflight Check Report key		
Report type	**Corresponding icon**	**Description**
Passed	✅	Meets baseline or recommended values.
INFO	ℹ️	General system or environment information details.
Failed	❌	Does not meet recommended values. This is informational only.

At the end of the report a **Summary** section is provided. The **Summary** provides information of failed checks.

> **Note:** The report is saved in the current working directory, from where the playbook was run as `./preflight_report.txt`. The path to the report is displayed after the summary report.

b. Run the preflight playbook on a selected set of hosts in a storage cluster by using the **`--limit`** option.

```
ansible-playbook -i INVENTORY_FILE cephadm-preflight.yml --extra-vars
"ceph_origin=rhcs" --limit GROUP_NAME|NODE_NAME
```

Replace *GROUP_NAME* with a group name from your inventory file and *NODE_NAME* with a specific node name from your inventory file.

> **Note:** Optionally, you can group your nodes in your inventory file by group name such as `[mons]`, `[osds]`, and `[mgrs]`. However, admin nodes must be added to the `[admin]` group and clients must be added to the `[clients]` group.

For example,

```
[ansible@admin cephadm-ansible]$ ansible-playbook -i hosts cephadm-preflight.yml
--extra-vars "ceph_origin=rhcs" --limit clients
[ansible@admin cephadm-ansible]$ ansible-playbook -i hosts cephadm-preflight.yml
--extra-vars "ceph_origin=rhcs" --limit host01
```

What to do next

1. Update or fix any issues that were flagged during the preflight check.
2. Verify that the packages were installed successfully.
   For example,

   ```
   [ansible@admin cephadm-ansible]$ rpm -q chrony ceph-common
   ```

   Update or fix any issues that were flagged during the preflight check. Verify that the packages were installed successfully.

Bootstrapping a new storage cluster

Use the `cephadm` utility to bootstrap a new storage cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- An IP address for the first Ceph Monitor container, which is also the IP address for the first node in the storage cluster.
- Login access to `registry.redhat.io`.
- A minimum of 10 GB of free space for `/var/lib/containers/`.
- Root-level access to all nodes.

About this task

The `cephadm` utility performs the following tasks during the bootstrap process:

- Installs and starts a Ceph Monitor daemon and a Ceph Manager daemon for a new Red Hat Ceph Storage cluster on the local node as containers.
- Creates the `/etc/ceph` directory.
- Writes a copy of the public key to `/etc/ceph/ceph.pub` for the Red Hat Ceph Storage cluster and adds the SSH key to the root user's `/root/.ssh/authorized_keys` file.
- Applies the `_admin` label to the bootstrap node.
- Writes a minimal configuration file needed to communicate with the new cluster to `/etc/ceph/ceph.conf`.
- Writes a copy of the `client.admin` administrative secret key to `/etc/ceph/ceph.client.admin.keyring`.
- Deploys a basic monitoring stack with Prometheus, Grafana, and other tools such as `node-exporter` and `Alertmanager`.

Important:

- If you are performing a disconnected installation, see "Performing a disconnected installation" on page 213.
- If you are deploying a monitoring stack, see Deploying the monitoring stack using the Ceph Orchestrator.
- Bootstrapping provides the default user name and password for the initial login to the Dashboard. Bootstrap requires you to change the password after you log in.
- Before you begin the bootstrapping process, make sure that the container image that you want to use has the same version of Red Hat Ceph Storage as `cephadm`. If the two versions do not match, bootstrapping fails at the `Creating initial admin user` stage.

- If you have an existing Prometheus services that you want to run with the new storage cluster, or if you are deploying Ceph with Rook, pass the **`--skip-monitoring-stack`** option with the `cephadm bootstrap` command. This option bypasses the basic monitoring stack so that you can manually configure it later.
- Before you begin the bootstrapping process, you must create a username and password for the `registry.redhat.io` container registry. For more information about Red Hat container registry, see Red Hat Container Registry Authentication on Red Hat Customer Portal.

For more information, see:

- "Recommended cephadm bootstrap command options" on page 197
- Using a JSON file to protect login information

Procedure

Important: Run the **`cephadm bootstrap`** command on the node that you want to be the initial Monitor node in the cluster. The *`IP_ADDRESS`* option should be the IP address of the node you are using to run the **`cephadm bootstrap`** command.

Note:

- If the storage cluster includes multiple networks and interfaces, be sure to choose a network that is accessible by any node that uses the storage cluster.
- If the local node uses fully-qualified domain names (FQDN), then add the **`--allow-fqdn-hostname`** option to the **`cephadm bootstrap`** command.
- To deploy a storage cluster using IPV6 addresses, use the IPV6 address format for the `--mon-ip` *`IP_ADDRESS`* option. For example: **`cephadm bootstrap --mon-ip 2620:52:0:880:225:90ff:fefc:2536 --registry-json /etc/mylogin.json`**.
- To route the internal cluster traffic over the public network, omit the `--cluster-network` *`NETWORK_CIDR`* option.

– Bootstrap a storage cluster.

```
cephadm bootstrap --cluster-network NETWORK_CIDR --mon-ip IP_ADDRESS --registry-url
registry.redhat.io --registry-username cp --registry-password PASSWORD
```

For example,

```
[root@host01 ~]# cephadm bootstrap --cluster-network 10.10.128.0/24 --mon-ip
10.10.128.68 --registry-url registry.redhat.io --registry-username cp --registry-
password mypassword1
```

The script takes a few minutes to complete. Once the script completes, it provides the credentials to the Red Hat Ceph Storage Dashboard URL, a command to access the Ceph command-line interface (CLI), and a request to enable telemetry.

```
Ceph Dashboard is now available at:

             URL: https://host01:8443/
            User: admin
        Password: i8nhu7zham

Enabling client.admin keyring and conf on hosts with "admin" label
You can access the Ceph CLI with:

        sudo /usr/sbin/cephadm shell --fsid 266ee7a8-2a05-11eb-b846-5254002d4916 -c /
etc/ceph/ceph.conf -k /etc/ceph/ceph.client.admin.keyring

Please consider enabling telemetry to help improve Ceph:

        ceph telemetry on

For more information see:

        https://docs.ceph.com/docs/master/mgr/telemetry/

Bootstrap complete.
```

Recommended `cephadm bootstrap` command options

Use a basic set of command options for `cephadm bootstrap`. You can configure additional options after your initial cluster is up and running.

The **`cephadm bootstrap`** command has multiple options that allow you to specify file locations, configure `ssh` settings, set passwords, and perform other initial configuration tasks.

Specify the recommended options by using the following command:

```
cephadm bootstrap --ssh-user USER_NAME --mon-ip IP_ADDRESS --allow-fqdn-hostname --
registry-json REGISTRY_JSON
```

For example,

```
[root@host01 ~]# cephadm bootstrap --ssh-user ceph --mon-ip 10.10.128.68 --allow-fqdn-
hostname --registry-json /etc/mylogin.json
```

For non-root users, see “Creating an Ansible user with sudo access” on page 188 and “Enabling password-less SSH for Ansible” on page 190 for more details.

Reference

- For more information about the `--registry-json` option, see Using a JSON file to protect login information.
- For more information about all available `cephadm bootstrap` options, see “Bootstrap command options” on page 204.
- For more information about bootstrapping the storage cluster as a non-root user, see “Bootstrapping the storage cluster as a non-root user” on page 203.

Using a JSON file to protect login information

As a storage administrator, you might choose to add login and password information to a JSON file and then refer to the JSON file for bootstrapping. Using a JSON file protects the login credentials from exposure.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- An IP address for the first Ceph Monitor container, which is also the IP address for the first node in the storage cluster.
- Login access to `registry.redhat.io`.
- A minimum of 10 GB of available space for `/var/lib/containers/`.
- Root-level access to all nodes.

About this task

> **Note:** A JSON file can also be used with the **`cephadm --registry-login`** command.

Procedure

In these steps, the JSON file name is `mylogin.json`.

1. Create the JSON file.

```
{
 "url":"registry.redhat.io",
 "username":"USER_NAME",
 "password":"PASSWORD"
}
```

2. Bootstrap a storage cluster.

```
cephadm bootstrap --mon-ip IP_ADDRESS --registry-json /etc/mylogin.json
```

For example,

```
[root@host01 ~]# cephadm bootstrap --mon-ip 10.10.128.68 --registry-json /etc/
mylogin.json
```

Bootstrap storage cluster along with Dashboard

As a system administrator, you can access the dashboard with the credentials that are provided on bootstrapping the cluster.

`cephadm` installs the dashboard by default. The following snippet is an example of the dashboard URL.

```
URL: https://host01:8443/
User: admin
Password: zbiql951ar
```

Note: Update the browser and clear the cookies to access the dashboard URL.

The following are the `cephadm` bootstrap options that are available for the Ceph dashboard configurations.

–initial-dashboard-user *INITIAL_DASHBOARD_USER*

Use this option during bootstrap to set initial-dashboard-user.

–initial-dashboard-password *INITIAL_DASHBOARD_PASSWORD*

Use this option during bootstrap to set the initial-dashboard-password.

–ssl-dashboard-port *SSL_DASHBOARD_PORT*

Use this option during bootstrap to set a custom dashboard port other than the default 8443.

–dashboard-key *DASHBOARD_KEY*

Use this option during bootstrap to set a custom key for SSL.

–dashboard-crt *DASHBOARD_CRT*

Use this option during bootstrap to set a Custom certificate for SSL.

–skip-dashboard

Use this option during bootstrap to deploy Ceph without the dashboard.

–dashboard-password-noupdate

Use this option during bootstrap if you used `–skip-dashboard` and `–dashboard-crt` options and do not want to reset password at the first time login.

–allow-fqdn-hostname

Use this option during bootstrap to allow a hostname that is fully qualified.

–skip-prepare-host

Use this option during bootstrap to skip preparing the host.

Note:

- To avoid connectivity issues with dashboard related external URL, use the fully qualified domain names (FQDN) for hostnames. For example, `host01.ceph.redhat.com`.
- Open the Grafana URL directly in the client internet browser and accept the security exception to see the graphs on the Ceph dashboard. Reload the browser to view the changes.

Example

```
[root@host01 ~]# cephadm bootstrap --mon-ip 127.0.0.1 --registry-json cephadm.txt  --
initial-dashboard-user  admin --initial-dashboard-password zbiql951ar --dashboard-password-
noupdate --allow-fqdn-hostname
```

Note:

- While bootstrapping the storage cluster that uses `cephadm`, you can use the `--image` option for either custom container images or local container images.
- Change the password the first time that you log in to the dashboard with the credentials that are provided on bootstrapping only if `--dashboard-password-noupdate` option is not used while bootstrapping. You can find the Ceph dashboard credentials in the `var/log/ceph/cephadm.log` file.

Search with the “Ceph Dashboard is now available at” string. For example, `host01.ceph.redhat.com`.

Network port requirements for Ceph Dashboard

The Ceph dashboard components use certain TCP network ports which must be accessible. By default, the network ports are automatically opened in `firewalld` during installation of Red Hat Ceph Storage.

Table 31: TCP port requirements

Port	Use	Originating Host	Destination Host
8443	The dashboard web interface	IP addresses that need access to Ceph Dashboard UI and the host under Grafana server, since the AlertManager service can also initiate connections to the Dashboard for reporting alerts.	The Ceph Manager hosts.
3000	Grafana	IP addresses that need access to Grafana Dashboard UI and all Ceph Manager hosts and Grafana server.	The host or hosts running Grafana server.
9095	Default Prometheus server for basic Prometheus graphs	IP addresses that need access to Prometheus UI and all Ceph Manager hosts and Grafana server or Hosts running Prometheus.	The host or hosts running Prometheus.
9093	Prometheus Alertmanager	IP addresses that need access to Alertmanager Web UI and all Ceph Manager hosts and Grafana server or Hosts running Prometheus.	All Ceph Manager hosts and the host under Grafana server.
9094	Prometheus Alertmanager for configuring a highly available cluster made from multiple instances	All Ceph Manager hosts and the host under Grafana server.	Prometheus Alertmanager High Availability (peer daemon sync), so both `src` and `dst` should be hosts running Prometheus Alertmanager.
9100	The Prometheus `node-exporter` daemon	Hosts running Prometheus that need to view Node Exporter metrics Web UI and All Ceph Manager hosts and Grafana server or Hosts running Prometheus.	All storage cluster hosts, including MONs, OSDS, Grafana server host.
9283	Ceph Manager Prometheus exporter module	Hosts running Prometheus that need access to Ceph Exporter metrics Web UI and Grafana server.	All Ceph Manager hosts.

Reference

For more information, see the following:

- Installing.
- Using and configuring firewalld.

Providing configuration settings during bootstrap

Use a configuration file with the `cephadm` bootstrap command to apply cluster settings during bootstrap.

Before you begin

Ensure that the bootstrap host is prepared and that the configuration file contains the settings that you want to apply during bootstrap.

About this task

You can provide cluster configuration settings during bootstrap by using the **`--config`** option. During bootstrap, `cephadm` reads the specified configuration file and assimilates the settings into the cluster configuration.

Procedure

1. Create a configuration file that contains the settings that you want to apply during bootstrap.
   Example configuration file:

   ```
   [global]
   log_to_file = true

   [mon]
   mon_cluster_log_to_file = true

   [osd]
   osd_crush_initial_weight = 0
   ```

2. Run the bootstrap command and specify the configuration file.

   ```
   cephadm bootstrap \
     --mon-ip 10.10.128.68 \
     --config /root/bootstrap.conf \
     --registry-json /etc/mylogin.json
   ```

Result

During bootstrap, `cephadm` assimilates the configuration settings into the cluster configuration.

What to do next

> **Important:** Do not maintain the bootstrap configuration file as `/etc/ceph/ceph.conf`. `cephadm` can overwrite that file during later cluster management operations. Store bootstrap configuration files separately and use them only as input to the bootstrap process.

Bootstrapping a storage cluster using a service configuration file

Use the **`cephadm boostrap`** command with the **`--apply-spec`** option to bootstrap the storage cluster and configure additional hosts and daemons using a service configuration file. The configuration file is a YAML file that contains the service type, placement, and designated nodes for services that you want to deploy.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- At least one running virtual machine (VM) or server.
- Root-level access to all nodes.

- Login access to `registry.redhat.io`.
- Passwordless `ssh` is set up on all hosts in the storage cluster.
- `cephadm` is installed on the node that you want to be the initial Monitor node in the storage cluster.

For the latest supported Red Hat Enterprise Linux versions, see Compatibility matrix.

About this task

Note: If you want to use a non-default realm or zone for applications such as multi-site, configure your Ceph Object Gateway daemons after you bootstrap the storage cluster, instead of adding them to the configuration file and using the `--apply-spec` option. This gives you the opportunity to create the realm or zone you need for the Ceph Object Gateway daemons before deploying them.

Note: If you run the bootstrap command with `--apply-spec` option, ensure to include the IP address of the bootstrap host in the specification file. This prevents resolving the IP address to loopback address while re-adding the bootstrap host where active Ceph® Manager is already running. If you do not use the `--apply spec` option during bootstrap and instead use **`ceph orch apply`** command with another specification file which includes re-adding the host and contains an active Ceph® Manager running, then ensure to explicitly provide the **`addr`** field. This is applicable for applying any specification file after bootstrapping.

For more information, see "Bootstrap command options" on page 204.

Procedure

1. Log in to the bootstrap host.
2. Create the service configuration YAML file for your storage cluster.
   The following example file directs `cephadm bootstrap` to configure the initial host and two additional hosts, and it specifies that OSDs be created on all available disks.

```
service_type: host
addr: host01
hostname: host01
---
service_type: host
addr: host02
hostname: host02
---
service_type: host
addr: host03
hostname: host03
---
service_type: host
addr: host04
hostname: host04
---
service_type: mon
placement:
  host_pattern: "host[0-2]"
---
service_type: osd
service_id: my_osds
placement:
  host_pattern: "host[1-3]"
data_devices:
  all: true
```

3. Bootstrap the storage cluster with the **`--apply-spec`** option.

```
cephadm bootstrap --apply-spec CONFIGURATION_FILE_NAME --mon-ip MONITOR_IP_ADDRESS --
registry-url registry.redhat.io --registry-username USER_NAME --registry-password
PASSWORD
```

   For example,

```
[root@host01 ~]# cephadm bootstrap --apply-spec initial-config.yaml --mon-ip
10.10.128.68 --registry-url registry.redhat.io --registry-username myuser1--registry-
password mypassword1
```

The script takes a few minutes to complete. Once the script completes, it provides the credentials to the Red Hat Ceph Storage Dashboard URL, a command to access the Ceph command-line interface (CLI), and a request to enable telemetry.

What to do next

Once your storage cluster is up and running, see Operations for more information about configuring additional daemons and services.

Bootstrapping the storage cluster as a non-root user

You can bootstrap the storage cluster as a non-root user if you have passwordless sudo privileges.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- An IP address for the first Ceph Monitor container, which is also the IP address for the initial Monitor node in the storage cluster.
- Login access to `registry.redhat.io`.
- A minimum of 10 GB of free space for `/var/lib/containers/`.
- Optional: SSH public and private keys.
- Passwordless `sudo` access to the bootstrap node.
- Non-root users have passwordless sudo access on all nodes intended to be part of the cluster.
- `cephadm` installed on the node that you want to be the initial Monitor node in the storage cluster.

About this task

To bootstrap the Red Hat Ceph Storage cluster as a non-root user on the bootstrap node, use the `--ssh-user` option with the **`cephadm bootstrap`** command. `--ssh-user` specifies a user for SSH connections to cluster nodes.

Non-root users must have passwordless `sudo` access.

For more information, see:

- “Creating an Ansible user with sudo access” on page 188
- “Enabling password-less SSH for Ansible” on page 190
- “Bootstrap command options” on page 204

Procedure

1. Change to `sudo` on the bootstrap node.

   ```
   su - SSH_USER_NAME
   ```

   For example,

   ```
   [root@host01 ~]$ su - ceph
   Last login: Tue Sep 14 12:00:29 EST 2021 on pts/0
   ```

2. Check the SSH connection to the bootstrap node.
   For example,

   ```
   [ceph@host01 ~]$ ssh host01
   Last login: Tue Sep 14 12:03:29 EST 2021 on pts/0
   ```

3. Run the **`cephadm bootstrap`** command.

Note: Using private and public keys is optional.
If SSH keys have not previously been created, these can be created during this step.

Run the command with the **--ssh-private-key** and **--ssh-public-key** options.

```
sudo cephadm bootstrap --ssh-user USER_NAME --mon-ip IP_ADDRESS --ssh-private-key
PRIVATE_KEY --ssh-public-key PUBLIC_KEY --registry-url registry.redhat.io --registry-
username USER_NAME --registry-password PASSWORD
```

For example,

```
sudo cephadm bootstrap --ssh-user USER_NAME --mon-ip IP_ADDRESS --ssh-private-key
PRIVATE_KEY --ssh-public-key PUBLIC_KEY --registry-url registry.redhat.io --registry-
username myuser1 --registry-password mypassword1
```

Related information

Managing sudo access
Red Hat Ceph Storage 5 rootless deployment utilizing ansible ad-hoc commands

Bootstrap command options

The **cephadm bootstrap** command bootstraps a Red Hat Ceph Storage cluster on the local host. It deploys a MON daemon and a MGR daemon on the bootstrap node, automatically deploys the monitoring stack on the local host, and calls **ceph orch host add HOSTNAME**.

"General cephadm bootstrap command options" on page 204 lists the basic available options for the **cephadm bootstrap** command.

Table 32: General **cephadm bootstrap** *command options*

cephadm bootstrap option	**Description**
`--config CONFIG_FILE, -c CONFIG_FILE`	Uses the specified Ceph configuration file during bootstrap. `cephadm` assimilates the configuration settings into the cluster as part of the bootstrap process.
`--cluster-network NETWORK_CIDR`	Use the subnet defined by *NETWORK_CIDR* for internal cluster traffic. This is specified in CIDR notation. For example: `10.10.128.0/24`.
`--mon-id MON_ID`	Bootstraps on the host named *MON_ID*. Default value is the local host.
`--mon-addrv MON_ADDRV`	mon IPs (e.g., `[v2:localipaddr:3300,v1:localipaddr:6789]`)
`--mon-ip IP_ADDRESS`	IP address of the node you are using to run **cephadm bootstrap**.
`--mgr-id MGR_ID`	Host ID where a MGR node should be installed. Default: randomly generated.
`--fsid FSID`	Cluster FSID.
`--output-dir OUTPUT_DIR`	Use this directory to write config, keyring, and pub key files.
`--output-keyring OUTPUT_KEYRING`	Use this location to write the keyring file with the new cluster admin and mon keys.

cephadm bootstrap option	**Description**
`--output-config OUTPUT_CONFIG`	Use this location to write the configuration file to connect to the new cluster.
`--output-pub-ssh-key OUTPUT_PUB_SSH_KEY`	Use this location to write the public SSH key for the cluster.
`--skip-ssh`	Skip the setup of the ssh key on the local host.
`--initial-dashboard-user INITIAL_DASHBOARD_USER`	Initial user for the dashboard.
`--initial-dashboard-password INITIAL_DASHBOARD_PASSWORD`	Initial password for the initial dashboard user.
`--ssl-dashboard-port SSL_DASHBOARD_PORT`	Port number used to connect with the dashboard using SSL.
`--dashboard-key DASHBOARD_KEY`	Dashboard key.
`--dashboard-crt DASHBOARD_CRT`	Dashboard certificate.
`--ssh-config SSH_CONFIG`	SSH config.
`--ssh-private-key SSH_PRIVATE_KEY`	SSH private key.
`--ssh-public-key SSH_PUBLIC_KEY`	SSH public key.
`--ssh-user SSH_USER`	Sets the user for SSH connections to cluster hosts. Passwordless sudo is needed for non-root users.
`--skip-mon-network`	Sets mon public_network based on the bootstrap mon ip.
`--skip-dashboard`	Do not enable the Ceph Dashboard.
`--dashboard-password-noupdate`	Disable forced dashboard password change.
`--no-minimize-config`	Do not assimilate and minimize the configuration file.
`--skip-ping-check`	Do not verify that the mon IP is pingable.
`--skip-pull`	Do not pull the latest image before bootstrapping.
`--skip-firewalld`	Do not configure firewalld.
`--allow-overwrite`	Allow the overwrite of existing –`output-*` `config/keyring/ssh` files.
`--allow-fqdn-hostname`	Allow fully qualified host name.
`--skip-prepare-host`	Do not prepare host.
`--orphan-initial-daemons`	Do not create initial mon, mgr, and crash service specs.
`--skip-monitoring-stack`	Do not automatically provision the monitoring stack] (prometheus, grafana, alertmanager, node-exporter).
`--apply-spec APPLY_SPEC`	Apply cluster spec file after bootstrap (copy ssh key, add hosts and apply services).

cephadm bootstrap option	Description
--registry-url REGISTRY_URL	Specifies the URL of the custom registry to log into. For example: registry.redhat.io.
--registry-username REGISTRY_USERNAME	User name of the login account to the custom registry.
--registry-password REGISTRY_PASSWORD	Password of the login account to the custom registry.
--registry-json REGISTRY_JSON	JSON file containing registry login information.

Reference

- For more information about the --skip-monitoring-stack option, see "Adding hosts" on page 223.
- For more information about logging into the registry with the registry-json option, see help for the registry-login command.
- For more information about cephadm options, see help for cephadm.

Distributing SSH keys

You can use the cephadm-distribute-ssh-key.yml playbook to distribute the SSH keys instead of creating and distributing the keys manually.

Before you begin

- Ansible is installed on the administration node.
- Access to the Ansible administration node.
- Ansible user with sudo access to all nodes in the storage cluster.
- Bootstrapping is completed. See "Bootstrapping a new storage cluster" on page 195 for more details.

About this task

The playbook distributes an SSH public key over all hosts in the inventory. You can also generate an SSH key pair on the Ansible administration node and distribute the public key to each node in the storage cluster so that Ansible can access the nodes without being prompted for a password.

Procedure

1. Navigate to the /usr/share/cephadm-ansible directory on the Ansible administration node.

```
[ansible@admin ~]$ cd /usr/share/cephadm-ansible
```

2. From the Ansible administration node, distribute the SSH keys. The optional cephadm_pubkey_path parameter is the full path name of the SSH public key file on the ansible controller host.

> **Note:** If cephadm_pubkey_path is not specified, the playbook gets the key from the cephadm get-pub-key command. This implies that you have at least bootstrapped a minimal cluster.

```
ansible-playbook -i INVENTORY_HOST_FILE cephadm-distribute-ssh-key.yml -e
cephadm_ssh_user=USER_NAME -e cephadm_pubkey_path=home/cephadm/ceph.key -e
admin_node=ADMIN_NODE_NAME_1
```

```
[ansible@admin cephadm-ansible]$ ansible-playbook -i hosts cephadm-distribute-ssh-
key.yml -e cephadm_ssh_user=ceph-admin -e cephadm_pubkey_path=/home/cephadm/ceph.key
-e admin_node=host01

[ansible@admin cephadm-ansible]$ ansible-playbook -i hosts cephadm-distribute-ssh-
key.yml -e cephadm_ssh_user=ceph-admin -e admin_node=host01
```

Disconnected (air gap) installation

As a storage administrator, you can install Red Hat Ceph Storage in a disconnected, also known as *air-gapped* environment, to install `cephadm` and bootstrap your storage cluster on a private network. A disconnected installation uses a private registry for installation.

Before starting to install Red Hat Ceph Storage, see Installation and upgrade requirements.

Configuring a private registry for a disconnected installation

Use a disconnected installation procedure to install `cephadm` and bootstrap your storage cluster on a private network. A disconnected installation uses private registry for installation. Use this procedure when the Red Hat Ceph Storage nodes do not have access to the internet during deployment, such as air gap.

Before you begin

- At least one running virtual machine or server with an active internet connection.
- Red Hat Enterprise Linux 9.6, 9.7, 10, or 10.1 with `ansible-core` available from the corresponding RHEL AppStream repository.
- Root-level access to all nodes.

> **Note:** Red Hat now supports only Simple Content Access (SCA). As a result, customers no longer need to attach any subscription to a system. Registering the system and enabling the required repositories is now sufficient for registration use. For more information, see How to register and subscribe a RHEL system to the Red Hat Customer Portal using Red Hat Subscription-Manager? on the Red Hat Customer Portal.

For the latest supported Red Hat Enterprise Linux versions, see Compatibility matrix.

About this task

Follow this procedure to set up a secure private registry using authentication and a self-signed certificate. Run these steps on a node that has both internet access and access to the local cluster.

> **Note:** Do not use an insecure registry for production.

Procedure

1. Register the hosts, and when prompted, enter the appropriate Red Hat Customer Portal credentials.

   ```
   subscription-manager register
   ```

   For example,

   ```
   subscription-manager register
   ```

2. Get the latest subscription.

   ```
   subscription-manager refresh
   ```

3. Disable the software repositories.

   ```
   subscription-manager repos --disable=*
   ```

4. Enable the Red Hat Enterprise Linux baseos and appstream repositories.
   - Enabling repositories with Red Hat Enterprise Linux 9.x:

     ```
     subscription-manager repos --enable=rhel-9-for-x86_64-baseos-rpms
     ```

```
subscription-manager repos --enable=rhel-9-for-x86_64-appstream-rpms
```

- Enabling repositories with Red Hat Enterprise Linux 10.x:

```
subscription-manager repos --enable=rhel-10-for-x86_64-baseos-rpms
```

```
subscription-manager repos --enable=rhel-10-for-x86_64-appstream-rpms
```

For example,

```
# subscription-manager repos --enable=rhel-9-for-x86_64-baseos-rpms
# subscription-manager repos --enable=rhel-9-for-x86_64-appstream-rpms
```

5. Update the system.

```
dnf update
```

6. Install the `podman` and `httpd-tools` packages.

```
dnf install -y podman httpd-tools
```

7. Create folders for the private registry.

```
mkdir -p /opt/registry/{auth,certs,data}
```

The registry is stored in `/opt/registry` and the directories are mounted in the container running the registry.

- The `auth` directory stores the `htpasswd` file the registry uses for authentication.
- The `certs` directory stores the certificates the registry uses for authentication.
- The `data` directory stores the registry images.

8. Create credentials for accessing the private registry.

```
htpasswd -bBc /opt/registry/auth/htpasswd PRIVATE_REGISTRY_USERNAME
PRIVATE_REGISTRY_PASSWORD
```

- Theboption provides the password from the command line.
- TheBoption stores the password using `Bcrypt` encryption.
- Thecoption creates the `htpasswd` file.
- Replace *`PRIVATE_REGISTRY_USERNAME`* with the username to create for the private registry.
- Replace *`PRIVATE_REGISTRY_PASSWORD`* with the password to create for the private registry username.

For example,

```
# htpasswd -bBc /opt/registry/auth/htpasswd myregistryusername myregistrypassword1
```

9. Create a self-signed certificate.

```
openssl req -newkey rsa:4096 -nodes -sha256 -keyout /opt/registry/certs/domain.key
-x509 -days 365 -out /opt/registry/certs/domain.crt -addext "subjectAltName =
DNS:LOCAL_NODE_FQDN"
```

Replace *`LOCAL_NODE_FQDN`* with the fully qualified host name of the private registry node.
There is a prompt for the respective options for your certificate. The `CN=` value is the host name of your node and should be resolvable by DNS or the `/etc/hosts` file.

For example,

```
# openssl req -newkey rsa:4096 -nodes -sha256 -keyout /opt/registry/certs/domain.key
-x509 -days 365 -out /opt/registry/certs/domain.crt -addext "subjectAltName =
DNS:admin.lab.redhat.com"
```

> **Note:** When creating a self-signed certificate, be sure to create a certificate with a proper Subject Alternative Name (SAN). Podman commands that require TLS verification for certificates that do not include a proper SAN, return the following error:
>
> `x509: certificate relies on legacy Common Name field, use SANs or temporarily enable Common Name matching with GODEBUG=x509ignoreCN=0`

10. Create a symbolic link to `domain.cert`.
This allows skopeo to locate the certificate with the file extension `.cert`.
For example,

```
# ln -s /opt/registry/certs/domain.crt /opt/registry/certs/domain.cert
```

11. Add the certificate to the trusted list on the private registry node.

```
cp /opt/registry/certs/domain.crt /etc/pki/ca-trust/source/anchors/
update-ca-trust
trust list | grep -i "LOCAL_NODE_FQDN"
```

Replace *LOCAL_NODE_FQDN* with the FQDN of the private registry node.
For example,

```
# cp /opt/registry/certs/domain.crt /etc/pki/ca-trust/source/anchors/
# update-ca-trust
# trust list | grep -i "admin.lab.redhat.com"

     label: admin.lab.redhat.com
```

12. Copy the certificate to any nodes that will access the private registry for installation and update the trusted list.

```
# scp /opt/registry/certs/domain.crt root@HOST_NAME:/etc/pki/ca-trust/source/anchors/
ssh root@HOST_NAME
update-ca-trust
trust list | grep -i "LOCAL_NODE_FQDN"
```

Replace *LOCAL_NODE_FQDN* with the FQDN of the private registry node.
For example,

```
# scp /opt/registry/certs/domain.crt root@host01:/etc/pki/ca-trust/source/anchors/
# ssh root@host01
# update-ca-trust
# trust list | grep -i "admin.lab.redhat.com"

    label: admin.lab.redhat.com
```

13. Download and install the mirror registry.
    a. Download the mirror-registry from the Red Hat Hybrid Cloud Console.
    b. Install the mirror registry.

```
./mirror-registry install --sslKey /opt/registry/certs/domain.key --sslCert /opt/
registry/certs/domain.crt --initUser myregistryuser --initPassword myregistrypass
```

14. On the local registry node, verify that `registry.redhat.io` is in the container registry search path.
    a. Open `/etc/containers/registries.conf` for editing.
    b. If needed, add `registry.redhat.io` to the `unqualified-search-registries` list.

```
unqualified-search-registries = ["registry.redhat.io",
"registry.access.redhat.com","registry.fedoraproject.org", "registry.centos.org",
"docker.io"]
```

15. Copy the following images to the private registry.

Table 33: Custom image details for the monitoring stack

Monitoring stack component	Image details

Ceph image	`registry.redhat.io/rhceph/rhceph-9-rhel9:latest`
	or
	`registry.redhat.io/rhceph/rhceph-9-rhel10:latest`

Ceph image (image mode)	`registry.redhat.io/rhceph/alloy-rhel10:latest`
Prometheus	`registry.redhat.io/openshift4/ose-prometheus:v4.15`
Grafana	`registry.redhat.io/rhceph/grafana-rhel10:latest`
Node-exporter	`registry.redhat.io/openshift4/ose-prometheus-nodeexporter:v4.15`
AlertManager	`registry.redhat.io/openshift4/ose-prometheusalertmanager:v4.15`
Keepalived	`registry.redhat.io/rhceph/keepalived-rhel10:latest`
SNMP Gateway	`registry.redhat.io/rhceph/snmp-notifier-rhel10:latest`
OAuth2 Proxy	`registry.redhat.io/rhceph/oauth2-proxy-rhel9:v7.14.0`

```
podman run -v /CERTIFICATE_DIRECTORY_PATH:/certs:Z -v /CERTIFICATE_DIRECTORY_PATH/
domain.cert:/certs/domain.cert:Z --rm registry.redhat.io/rhel9/skopeo skopeo copy --
remove-signatures --src-creds
RED_HAT_CUSTOMER_PORTAL_LOGIN:RED_HAT_CUSTOMER_PORTAL_PASSWORD --dest-cert-dir=./
certs/ --dest-creds PRIVATE_REGISTRY_USERNAME:PRIVATE_REGISTRY_PASSWORD docker://
registry.redhat.io/SRC_IMAGE:SRC_TAG docker://LOCAL_NODE_FQDN:8433/DST_IMAGE:DST_TAG
```

Note: If needed, replace the `rhel9` image input with `rhel10`

- Replace *`CERTIFICATE_DIRECTORY_PATH`* with the directory path to the self-signed certificates.
- Replace *`RED_HAT_CUSTOMER_PORTAL_LOGIN`* and *`RED_HAT_CUSTOMER_PORTAL_PASSWORD`* with your Red Hat Customer Portal credentials.

- Replace *PRIVATE_REGISTRY_USERNAME* and *PRIVATE_REGISTRY_PASSWORD* with the private registry credentials.
- Replace *SRC_IMAGE* and *SRC_TAG* with the name and tag of the image to copy from `registry.redhat.io`.
- Replace *DST_IMAGE* and *DST_TAG* with the name and tag of the image to copy to the private registry.
- Replace *LOCAL_NODE_FQDN* with the FQDN of the private registry.

For example,

```
# podman run -v /opt/registry/certs:/certs:Z -v/opt/registry/certs/domain.cert:/certs/
domain.cert:Z --rm registry.redhat.io/rhel9/skopeoskopeo copy --remove-signatures --
src-creds myusername:mypassword1 --dest-cert-dir=./certs/ --dest-creds
myregistryusername:myregistrypassword1 docker://registry.redhat.io/rhceph/
rhceph-9-rhel9:latest docker://admin.lab.redhat.com:5000/rhceph/rhceph-9-rhel9:latest

# podman run -v /opt/registry/certs:/certs:Z -v /opt/registry/certs/domain.cert:/
certs/domain.cert:Z --rm registry.redhat.io/rhel9/skopeo skopeo copy --remove-
signatures --src-creds myusername:mypassword1 --dest-cert-dir=./certs/ --dest-creds
myregistryusername:myregistrypassword1 docker://registry.redhat.io/openshift4/ose-
prometheus-node-exporter:v4.15 docker://admin.lab.redhat.com:5000/openshift4/ose-
prometheus-node-exporter:v4.15

# podman run -v /opt/registry/certs:/certs:Z -v /opt/registry/certs/domain.cert:/
certs/domain.cert:Z --rm registry.redhat.io/rhel9/skopeo skopeo copy --remove-
signatures --src-creds myusername:mypassword1 --dest-cert-dir=./certs/ --dest-creds
myregistryusername:myregistrypassword1  docker://registry.redhat.io/rhceph/grafana-
rhel10:latest docker://admin.lab.redhat.com:5000/rhceph/grafana-rhel10:latest

# podman run -v /opt/registry/certs:/certs:Z -v /opt/registry/certs/domain.cert:/
certs/domain.cert:Z --rm registry.redhat.io/rhel9/skopeo skopeo copy --remove-
signatures --src-creds myusername:mypassword1 --dest-cert-dir=./certs/ --dest-creds
myregistryusername:myregistrypassword1 docker://registry.redhat.io/openshift4/ose-
prometheus:v4.15 docker://admin.lab.redhat.com:5000/openshift4/ose-prometheus:v4.15

# podman run -v /opt/registry/certs:/certs:Z -v /opt/registry/certs/domain.cert:/
certs/domain.cert:Z --rm registry.redhat.io/rhel9/skopeo skopeo copy --remove-
signatures --src-creds myusername:mypassword1 --dest-cert-dir=./certs/ --dest-creds
myregistryusername:myregistrypassword1 docker://registry.redhat.io/openshift4/ose-
prometheus-alertmanager:v4.15 docker://admin.lab.redhat.com:5000/openshift4/ose-
prometheus-alertmanager:v4.15
```

> **Note:** For more image Ceph package versions, see What are the Red Hat Ceph Storage releases and corresponding Ceph package versions? on Red Hat Customer Portal.

16. Using the Ceph Dashboard, verify that the images are in the local registry.
For more information, see "Monitoring services of the Ceph cluster" on page 392.

Running the preflight playbook for a disconnected installation

Use the `cephadm-preflight.yml` Ansible playbook to configure the Ceph repository and prepare the storage cluster for bootstrapping in an air-gapped environment. It also installs some prerequisites, such as `podman`, `lvm2`, `chrony`, and `cephadm`.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Nodes configured to access a local YUM repository server with the following repositories enabled on respective Red Hat Enterprise Linux versions.
  - Red Hat Enterprise Linux 9 repositories:
    - `rhel-9-for-x86_64-baseos-rpms`
    - `rhel-9-for-x86_64-appstream-rpms`
    - `rhceph-9-tools-for-rhel-9-x86_64-rpms`
  - Red Hat Enterprise Linux 10 repositories:

- rhel-10-for-x86_64-baseos-rpms
- rhel-10-for-x86_64-appstream-rpms
- rhceph-9-tools-for-rhel-10-x86_64-rpms

> **Note:** For more information about setting up a local YUM repository, see Creating a Local Repository and Sharing with Disconnected/Offline/Air-gapped Systems on Red Hat Customer Portal.

- The cephadm-ansible package is installed on the Ansible administration node.

```
[root@admin ~]# dnf install cephadm-ansible
```

- Root-level access to all nodes in the storage cluster.
- Passwordless ssh is set up on all hosts in the storage cluster.

About this task

> **Important:** Run the preflight playbook before you bootstrap the initial host.

The preflight playbook uses the cephadm-ansible inventory hosts file to identify all the nodes in the storage cluster. The default location for cephadm-ansible, cephadm-preflight.yml, and the inventory hosts file is /usr/share/cephadm-ansible/.

The following example shows the structure of a typical inventory file:

```
host02
host03
host04

[admin]
host01
```

The [admin] group in the inventory file contains the name of the node where the admin keyring is stored.

The Ansible Preflight performs a check to detect whether swap space is configured on the host. This check is informational only and does not block deployment. Operational practices vary across environments:

- Some deployments disable swap on OSD nodes to avoid increased latency under memory pressure.
- Other deployments keep swap enabled but configure a very low vm.swappiness value (for example,0or1) to minimize swapping activity.

> **Note:** Specific swap configurations are not enforced. Follow the appropriate guidance for your specific environment.

Procedure

1. Navigate to the /usr/share/cephadm-ansible directory on the Ansible administration node.
2. Open and edit the hosts file and add your nodes.
3. Run the preflight playbook.

   > **Note:** When running the preflight playbook, cephadm-ansible automatically installs chrony and ceph-common on the client nodes.

   Use one of the following options:

   - Use **ceph_origin** parameter set tocustomto use a local YUM repository.

```
ansible-playbook -i INVENTORY_FILE cephadm-preflight.yml --extra-vars
"ceph_origin=custom" -e "custom_repo_url=CUSTOM_REPO_URL"
```

For example,

```
[ansible@admin cephadm-ansible]$ ansible-playbook -i hosts cephadm-preflight.yml
--extra-vars "ceph_origin=custom" -e "custom_repo_url=http://
mycustomrepo.lab.redhat.com/x86_64/os/"
```

After installation is complete, `cephadm` resides in the `/usr/sbin/` directory.

> **Note:** Populate the contents of the `registries.conf` file with the Ansible playbook.
>
> ```
> ansible-playbook -vvv i INVENTORY_HOST_FILE cephadm-set-container-insecure-
> registries.yml -e insecure_registry=REGISTRY_URL
> ```

- Use the **`--limit`** option to run the preflight playbook on a selected set of hosts in the storage cluster.

```
ansible-playbook -i INVENTORY_FILE cephadm-preflight.yml --extra-vars
"ceph_origin=custom" -e "custom_repo_url=CUSTOM_REPO_URL" --limit GROUP_NAME|
NODE_NAME
```

Replace *GROUP_NAME* with a group name from your inventory file and *NODE_NAME* with a specific node name from your inventory file.
For example,

```
[ansible@admin cephadm-ansible]$ ansible-playbook -i hosts cephadm-preflight.yml
--extra-vars "ceph_origin=custom" -e "custom_repo_url=http://
mycustomrepo.lab.redhat.com/x86_64/os/" --limit clients
[ansible@admin cephadm-ansible]$ ansible-playbook -i hosts cephadm-preflight.yml
--extra-vars "ceph_origin=custom" -e "custom_repo_url=http://
mycustomrepo.lab.redhat.com/x86_64/os/" --limit host02
```

Performing a disconnected installation

Before you can perform the air-gapped environment installation, you must obtain A Red Hat Ceph Storage container image, either from a proxy host that has access to the Red Hat registry or by copying the image to your local registry.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- At least one running virtual machine (VM) or server.
- Root-level access to all nodes.
- Passwordless `ssh` is set up on all hosts in the storage cluster.
- The preflight playbook has been run on the bootstrap host in the storage cluster. For more information, see Running the preflight playbook for a disconnected installation.
- A private registry has been configured and the bootstrap node has access to it. For more information, see Configuring a private registry for a disconnected installation.
- A Red Hat Ceph Storage container image resides in the custom registry.

About this task

> **Important:** Before you begin the bootstrapping process, make sure that the container image that you want to use has the same version of Red Hat Ceph Storage as `cephadm`. If the two versions do not match, bootstrapping fails at the `Creating initial admin user` stage.

Note: If your local registry uses a self-signed certificate with a local registry, ensure you have added the trusted root certificate to the bootstrap host. For more information, see Configuring a private registry for a disconnected installation.

Procedure

1. Log in to the bootstrap host.
2. Bootstrap the storage cluster.

```
cephadm --image PRIVATE_REGISTRY_NODE_FQDN:5000/CUSTOM_IMAGE_NAME:IMAGE_TAG bootstrap
--mon-ip IP_ADDRESS --registry-url PRIVATE_REGISTRY_NODE_FQDN:5000 --registry-username
PRIVATE_REGISTRY_USERNAME --registry-password PRIVATE_REGISTRY_PASSWORD
```

   - Replace *PRIVATE_REGISTRY_NODE_FQDN* with the fully qualified domain name of your private registry.
   - Replace *CUSTOM_IMAGE_NAME* and *IMAGE_TAG* with the name and tag of the Red Hat Ceph Storage container image that resides in the private registry.
   - Replace *IP_ADDRESS* with the IP address of the node you are using to run `cephadm bootstrap`.
   - Replace *PRIVATE_REGISTRY_USERNAME* with the username to create for the private registry.
   - Replace *PRIVATE_REGISTRY_PASSWORD* with the password to create for the private registry username.

   For example,

```
[root@host01 ~]# cephadm --image admin.lab.redhat.com:5000/rhceph-9-rhel9:latest
bootstrap --mon-ip 10.10.128.68 --registry-url admin.lab.redhat.com:5000 --registry-
username myregistryusername --registry-password myregistrypassword1
```

   The script takes a few minutes to complete. Once the script completes, it provides the credentials to the Red Hat Ceph Storage Dashboard URL, a command to access the Ceph command-line interface (CLI), and a request to enable telemetry.

```
Ceph Dashboard is now available at:

             URL: https://host01:8443/
            User: admin
        Password: i8nhu7zham

Enabling client.admin keyring and conf on hosts with "admin" label
You can access the Ceph CLI with:

        sudo /usr/sbin/cephadm shell --fsid 266ee7a8-2a05-11eb-b846-5254002d4916 -c /
etc/ceph/ceph.conf -k /etc/ceph/ceph.client.admin.keyring

Please consider enabling telemetry to help improve Ceph:

        ceph telemetry on

For more information see:

        https://docs.ceph.com/docs/master/mgr/telemetry/

Bootstrap complete.
```

What to do next

After the bootstrap process is complete, configure the container images, as detailed in Changing configurations of custom container images for disconnected installations.

Once your storage cluster is up and running, configure additional daemons and services. For more information, see Operations.

Changing configurations of custom container images for disconnected installations

After you perform the initial bootstrap for disconnected nodes, you must specify custom container images for monitoring stack daemons. You can override the default container images for monitoring stack daemons, since the nodes do not have access to the default container registry in air-gapped environments.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- At least one running virtual machine (VM) or server.
- Red Hat Enterprise Linux 9.6, 9.7, 10, or 10.1 with `ansible-core` available from the corresponding RHEL AppStream repository.
- Root-level access to all nodes.
- Passwordless `ssh` is set up on all hosts in the storage cluster.

For the latest supported Red Hat Enterprise Linux versions, see Compatibility matrix.

About this task

> **Note:** Make sure that the bootstrap process on the initial host is complete before making any configuration changes.

By default, the monitoring stack components are deployed based on the primary Ceph image. For disconnected environment of the storage cluster, you can use the latest available monitoring stack component images.

> **Note:** When using a custom registry, be sure to log in to the custom registry on newly added nodes before adding any Ceph daemons.
>
> ```
> ceph cephadm registry-login --registry-url CUSTOM_REGISTRY_NAME --registry_username
> REGISTRY_USERNAME --registry_password REGISTRY_PASSWORD
> ```
>
> For example,
>
> ```
> # ceph cephadm registry-login --registry-url myregistry --registry_username
> myregistryusername --registry_password myregistrypassword1
> ```

For more information, see “Performing a disconnected installation” on page 213.

Procedure

1. Set the custom container images with the **`ceph config`** command.

   ```
   ceph config set mgr mgr/cephadm/OPTION_NAME
   CUSTOM_REGISTRY_NAME/IMAGE_NAME
   ```

   Use the following options for *`OPTION_NAME`*:

   ```
   container_image_prometheus
   container_image_grafana
   container_image_alertmanager
   container_image_node_exporter
   ```

   For example,

```
[root@host01 ~]# ceph config set mgr mgr/cephadm/container_image_prometheus
private_registry/prometheus
[root@host01 ~]# ceph config set mgr mgr/cephadm/container_image_grafana
private_registry/grafana
[root@host01 ~]# ceph config set mgr mgr/cephadm/container_image_alertmanager
private_registry/alertmanager
[root@host01 ~]# ceph config set mgr mgr/cephadm/container_image_node_exporter
private_registry/node_exporter
```

2. Redeploy `node-exporter`.

```
ceph orch redeploy node-exporter
```

> **Note:**
> - If any of the services do not deploy, you can redeploy them with the **`ceph orch redeploy`** command.
> - By setting a custom image, the default values for the configuration image name and tag will be overridden, but not overwritten. The default values change when updates become available. By setting a custom image, you will not be able to configure the component for which you have set the custom image for automatic updates. You will need to manually update the configuration image name and tag to be able to install updates.

What to do next

To revert to the default configuration, reset the customer container image. This can be done at any time. Use the **`ceph config rm`** to reset the configuration option.

```
ceph config rm mgr mgr/cephadm/OPTION_NAME
```

For example,

```
[root@host01 ~]# ceph config rm mgr mgr/cephadm/container_image_prometheus
```

Adding hosts in disconnected deployments

If you are running a storage cluster on a private network, such as an air-gapped environment, and your host domain name server (DNS) cannot be reached through private IP, you must include both the host name and the IP address for each host you want to add to the storage cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to all hosts in the storage cluster.

Procedure

1. Log into the `cephadm` shell.

```
cephadm shell
```

2. Add the host.

```
ceph orch host add HOST_NAME HOST_ADDRESS
```

For example,

```
[ceph: root@host01 /]# ceph orch host add host03 10.10.128.70
```

Post-bootstrap setup

After the Red Hat Ceph Storage cluster has been installed and bootstrapped, start verifying and setting up your cluster.

Using the cephadm shell

The **cephadm shell** command opens a bash shell in a container with all Ceph packages installed. Use the shell to run "Day One" cluster setup tasks, such as installation and bootstrapping, and to run ceph commands.

Entering the cephadm shell

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to all nodes in the storage cluster.

About this task

Note: If the node contains configuration and keyring files in /etc/ceph/, the container environment uses the values in those files as defaults for the cephadm shell. If you run the cephadm shell on a MON node, the cephadm shell inherits its default configuration from the MON container, instead of using the default configuration.

Procedure

Open the cephadm shell in one of the following ways:

- Enter **cephadm shell** at the system prompt.
  This example runs the **ceph -s** command from within the shell.
  For example,

```
[root@host01 ~]# cephadm shell
[ceph: root@host01 /]# ceph -s
```

- At the system prompt, type cephadm shell and the command you want to run.
  For example,

```
[root@host01 ~]# cephadm shell ceph -s
cluster:
    id:     f64f341c-655d-11eb-8778-fa163e914bcc
    health: HEALTH_OK

services:
    mon: 3 daemons, quorum host01,host02,host03 (age 94m)
    mgr: host01.lbnhug(active, since 59m), standbys: host02.rofgay, host03.ohipra
    mds: 1/1 daemons up, 1 standby
    osd: 18 osds: 18 up (since 10m), 18 in (since 10m)
    rgw: 4 daemons active (2 hosts, 1 zones)

data:
    volumes: 1/1 healthy
    pools:   8 pools, 225 pgs
    objects: 230 objects, 9.9 KiB
    usage:   271 MiB used, 269 GiB / 270 GiB avail
    pgs:     225 active+clean

io:
    client:   85 B/s rd, 0 op/s rd, 0 op/s wr
```

Exiting the cephadm shell

To exit the cephadm shell, use the **exit** command.

```
[ceph: root@host01 /]# exit
[root@host01 ~]#
```

cephadm commands

The cephadm is a command line tool to manage the local host for the cephadm Orchestrator. It provides commands to investigate and modify the state of the current host.

Some of the commands are generally used for debugging.

Note: cephadm is not required on all hosts, however, it is useful when investigating a particular daemon. The cephadm-ansible-preflight playbook installs cephadm on all hosts and the cephadm-ansible purge playbook requires cephadm be installed on all hosts to work properly.

Table 34: The **cephadm** *commands*

Command	Description	Syntax	Example
adopt	Convert an upgraded storage cluster daemon to run cephadm.	`cephadm adopt [-h] --name DAEMON_NAME --style STYLE [--cluster CLUSTER] --legacy-dir [LEGACY_DIR] --config-json CONFIG_JSON] [--skip-firewalld] [--skip-pull]`	`[root@host01 ~]# cephadm adopt --style=legacy --name prometheus.host02`
ceph-volume	This command is used to list all the devices on the particular host. Run the ceph-volume command inside a container Deploys OSDs with different device technologies like lvm or physical disks using pluggable tools and follows a predictable, and robust way of preparing, activating, and starting OSDs.	`cephadm ceph-volume inventory/ simple/raw/lvm [-h] [--fsid FSID] [--config-json CONFIG_JSON] [--config CONFIG, -c CONFIG] [--keyring KEYRING, -k KEYRING]`	`[root@host01 ~]# cephadm ceph-volume inventory --fsid f64f341c-655d-11eb-8778-fa163e914bcc`
check-host	Check the host configuration that is suitable for a Ceph cluster.	`cephadm check-host [--expect-hostname HOSTNAME]`	`[root@host01 ~]# cephadm check-host --expect-hostname host02`

Command	Description	Syntax	Example
deploy	Deploys a daemon on the local host.	`cephadm shell deploy DAEMON_TYPE [-h] [--name DAEMON_NAME] [--fsid FSID] [--config CONFIG, -c CONFIG] [--config-json CONFIG_JSON] [--keyring KEYRING] [--key KEY] [--osd-fsid OSD_FSID] [--skip-firewalld] [--tcp-ports TCP_PORTS] [--reconfig] [--allow-ptrace] [--memory-request MEMORY_REQUEST] [--memory-limit MEMORY_LIMIT] [--meta-json META_JSON]`	`[root@host01 ~]# cephadm shell deploy mon --fsid f64f341c-655d-11eb-8778-fa163e914bcc`
enter	Run an interactive shell inside a running daemon container.	`cephadm enter [-h] [--fsid FSID] --name NAME [command [command …]]`	`[root@host01 ~]# cephadm enter --name 52c611f2b1d9`
help	View all the commands supported by `cephadm`.	`cephadm help`	`[root@host01 ~]# cephadm help`
install	Install the packages.	`cephadm install PACKAGES`	`[root@host01 ~]# cephadm install ceph-common ceph-osd`
inspect-image	Inspect the local Ceph container image.	`cephadm --image IMAGE_ID inspect-image`	`[root@host01 ~]# cephadm --image 13ea90216d0be03003d12d7869f72ad9de5cec9e54a27fd308e01e467c0d4a0a inspect-image`
list-networks	List the IP networks.	`cephadm list-networks`	`[root@host01 ~]# cephadm list-networks`

Command	Description	Syntax	Example
ls	List daemon instances known to `cephadm` on the hosts. You can use `--no-detail` for the command to run faster, which gives details of the daemon name, fsid, style, and systemd unit per daemon. You can use `--legacy-dir` option to specify a legacy base directory to search for daemons.	`cephadm ls [--no-detail] [--legacy-dir LEGACY_DIR]`	`[root@host01 ~]# cephadm ls --no-detail`

logs	Print `journald` logs for a daemon container. This is similar to the `journalctl` command.	`cephadm logs [--fsid FSID] --name DAEMON_NAME`
		`cephadm logs [--fsid FSID] --name DAEMON_NAME -- -n NUMBER # Last N lines`
		`cephadm logs [--fsid FSID] --name DAEMON_NAME -- -f # Follow the logs`
			`[root@host01 ~]# cephadm logs --fsid 57bddb48-ee04-11eb-9962-001a4a000672 --name osd.8`
			`[root@host01 ~]# cephadm logs --fsid 57bddb48-ee04-11eb-9962-001a4a000672 --name osd.8 -- -n 20`
			`[root@host01 ~]# cephadm logs --fsid 57bddb48-ee04-11eb-9962-001a4a000672 --name osd.8 -- -f`

prepare-host	Prepare a host for `cephadm`.	`cephadm prepare-host [--expect-hostname HOSTNAME]`	`[root@host01 ~]# cephadm prepare-host` `[root@host01 ~]# cephadm prepare-host --expect-hostname host01`
pull	Pull the Ceph image.	`cephadm [-h] [--image IMAGE_ID] pull`	`[root@host01 ~]# cephadm --image 13ea90216d0be03003d12d7869f72ad9de5cec9e54a27fd308e01e467c0d4a0a pull`

Command	Description	Syntax	Example

registry-login	Give cephadm login information for an authenticated registry. cephadm attempts to log the calling host into that registry.	`cephadm registry-login --registry-url REGISTRY_URL --registry-username USERNAME --registry-password PASSWORD [--fsid FSID] [--registry-json JSON_FILE]`
		
		You can also use a JSON registry file containing the login info formatted as:
		`cat REGISTRY_FILE`
		`{ "url":"REGISTRY_URL", "username":"REGISTRY_USERNAME", "password":"REGISTRY_PASSWORD" }`
			`[root@host01 ~]# cephadm registry-login --registry-url registry.redhat.io --registry-username myuser1 --registry-password mypassword1`
			
			`[root@host01 ~]# cat registry_file`
			`{ "url":"registry.redhat.io", "username":"myuser", "password":"mypass" }`
			`[root@host01 ~]# cephadm registry-login -i registry_file`

	Give cephadm login information for an authenticated registry. cephadm attempts to log the calling host into that registry.	`cephadm registry-login --registry-url REGISTRY_URL --registry-username USERNAME --registry-password PASSWORD [--fsid FSID] [--registry-json JSON_FILE]`
		
		You can also use a JSON registry file containing the login info formatted as:
		`cat REGISTRY_FILE`
		`{ "url":"REGISTRY_URL", "username":"REGISTRY_USERNAME", "password":"REGISTRY_PASSWORD" }`
			`[root@host01 ~]# cephadm registry-login --registry-url cp.icr.io/cp --registry-username myuser1 --registry-password mypassword1`
			
			`[root@host01 ~]# cat registry_file`
			`{ "url":"cp.icr.io/cp", "username":"myuser", "password":"mypass" }`
			`[root@host01 ~]# cephadm registry-login -i registry_file`

rm-daemon	Removes a specific daemon instance. Use this command only for debugging or experimental purposes. If you run it directly on the host, cephadm removes the daemon, but the manager module detects the missing service and redeploys it.	`cephadm rm-daemon --fsid FSID --name DAEMON_NAME [--force ] [--force-delete-data]`	`[root@host01 ~]# cephadm rm-daemon --fsid f64f341c-655d-11eb-8778-fa163e914bcc --name osd.8`

Command	Description	Syntax	Example
rm-cluster	Removes all daemons from the storage cluster on the host where the command is run. If the Ceph Orchestrator is not paused and the removed daemons belong to managed services, `cephadm` redeploys them automatically.	`cephadm rm-cluster --fsid FSID [--force]`	`[root@host01 ~]# cephadm rm-cluster --fsid f64f341c-655d-11eb-8778-fa163e914bcc`
rm-repo	Remove a package repository configuration. This is mainly used for the disconnected installation of Red Hat Ceph Storage.	`cephadm rm-repo [-h]`	`[root@host01 ~]# cephadm rm-repo`
run	Run a Ceph daemon, in a container, in the foreground.	`cephadm run [--fsid FSID] --name DAEMON_NAME`	`[root@host01 ~]# cephadm run --fsid f64f341c-655d-11eb-8778-fa163e914bcc --name osd.8`
shell	Run an interactive shell with access to Ceph commands over the inferred or specified Ceph cluster. You can enter the shell using the `cephadm shell` command and run all the orchestrator commands within the shell.	`cephadm shell [--fsid FSID] [--name DAEMON_NAME, -n DAEMON_NAME] [--config CONFIG, -c CONFIG] [--mount MOUNT, -m MOUNT] [--keyring KEYRING, -k KEYRING] [--env ENV, -e ENV]`	`[root@host01 ~]# cephadm shell --ceph orch ls [root@host01 ~]# cephadm shell`
unit	Start, stop, restart, enable, and disable the daemons with this operation. This operates on the daemon's `systemd` unit.	`cephadm unit [--fsid FSID] --name DAEMON_NAME start/stop/restart/enable/disable`	`[root@host01 ~]# cephadm unit --fsid f64f341c-655d-11eb-8778-fa163e914bcc --name osd.8 start`
version	Provides the version of the storage cluster.	`cephadm version`	`[root@host01 ~]# cephadm version`

Verifying the cluster installation

After the cluster installation is complete, you can verify that the Red Hat Ceph Storage installation is running properly.

There are two ways of verifying the storage cluster installation as a root user:

- Run the **`podman ps`** command.
- Run the **`cephadm shell ceph -s`** command.

Prerequisites

Before you begin, make sure that you have the following prerequisites in place:

- Root-level access to all nodes in the storage cluster.

Procedure

- Run the **podman ps** command:
  For example,

```
[root@host01 ~]# podman ps
```

> **Note:** In the NAMES column of the podman output, the unit files now include the FSID.

- Run the **cephadm shell ceph -s** command:
  For example,

```
[root@host01 ~]# cephadm shell ceph -s
  cluster:
    id:     8f43ad64-dd03-11ed-8ec8-525400c632ec
    health: HEALTH_WARN
            OSD count 0 < osd_pool_default_size 3

  services:
    mon: 1 daemons, quorum rhcs01 (age 14m)
    mgr: rhcs01.axgipe(active, since 109s)
    osd: 0 osds: 0 up, 0 in

  data:
    pools:   0 pools, 0 pgs
    objects: 0 objects, 0 B
    usage:   0 B used, 0 B / 0 B avail
    pgs:
```

> **Note:** The health of the storage cluster is in HEALTH_WARN status as the hosts and the daemons are not added.

Adding hosts

Bootstrapping the Red Hat Ceph Storage installation creates a working storage cluster, consisting of one Monitor daemon and one Manager daemon within the same container. As a storage administrator, you can add additional hosts to the storage cluster and configure them.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level or user with sudo access to all nodes in the storage cluster.
- Register the nodes to Red Hat subscription.
- Ansible user with sudo and passwordless ssh access to all nodes in the storage cluster.

About this task

> **Important:** For adding hosts with disconnected installations, see “Adding hosts in disconnected deployments” on page 216.

> **Note:**
>
> - Running the preflight playbook installs podman, lvm2, chrony, and cephadm on all hosts listed in the Ansible inventory file.

- When using a custom registry, be sure to log in to the custom registry on newly added nodes before adding any Ceph daemons.

```
ceph cephadm registry-login --registry-url CUSTOM_REGISTRY_NAME --
registry_username REGISTRY_USERNAME --registry_password REGISTRY_PASSWORD
```

For example,

```
# ceph cephadm registry-login --registry-url myregistry --registry_username
myregistryusername --registry_password myregistrypassword1
```

Procedure

In the following procedure, use either `root`, as indicated, or the username with which the user is bootstrapped.

1. From the node that contains the admin keyring, install the storage cluster's public SSH key in the root user's `authorized_keys` file on the new host.

```
ssh-copy-id -f -i /etc/ceph/ceph.pub user@NEWHOST
```

For example,

```
[root@host01 ~]# ssh-copy-id -f -i /etc/ceph/ceph.pub root@host02
[root@host01 ~]# ssh-copy-id -f -i /etc/ceph/ceph.pub root@host03
```

2. Navigate to the `/usr/share/cephadm-ansible` directory on the Ansible administration node.
For example,

```
[ansible@admin ~]$ cd /usr/share/cephadm-ansible
```

3. From the Ansible administration node, add the new host to the Ansible inventory file.
The default location for the file is `/usr/share/cephadm-ansible/hosts`.
The following example shows the structure of a typical inventory file.

> **Note:** If you have previously added the new host to the Ansible inventory file and run the preflight playbook on the host, skip to step "4" on page 224.

```
[ansible@admin ~]$ cat hosts

host02
host03
host04

[admin]
host01
```

4. Run the preflight playbook with the **`--limit`** option.

```
ansible-playbook -i INVENTORY_FILE cephadm-preflight.yml --extra-vars
"ceph_origin=rhcs" --limit NEWHOST
```

For example,

```
[ansible@admin cephadm-ansible]$ ansible-playbook -i hosts cephadm-preflight.yml --
extra-vars "ceph_origin=rhcs" --limit host02
```

5. Install `podman`, `lvm2`, `chrony`, and `cephadm` manually.

```
dnf install podman lvm2 chrony cephadm
```

For example,

```
[root@host01 ~]# dnf install podman lvm2 chrony cephadm
```

`cephadm` resides in the `/usr/sbin/` directory.

6. From the bootstrap node, use the `cephadm` orchestrator to add the new host to the storage cluster.

```
ceph orch host add NEWHOST
```

For example,

```
[ceph: root@host01 /]# ceph orch host add host02
Added host 'host02' with addr '10.10.128.69'
[ceph: root@host01 /]# ceph orch host add host03
Added host 'host03' with addr '10.10.128.70'
```

7. Optional: Add nodes to the host, by IP address.
Adding nodes can be done before and after running the preflight playbook. If you do not have DNS configured in your storage cluster environment, you can add the hosts by IP address, along with the host names.

```
ceph orch host add HOSTNAME IP_ADDRESS
```

For example,

```
[ceph: root@host01 /]# ceph orch host add host02 10.10.128.69
Added host 'host02' with addr '10.10.128.69'
```

8. Optional: Set labels to the host.
For more information, see "Labeling hosts" on page 229.

What to do next

View the status of the storage cluster and verify that the new host has been added.

> **Note:** The `status` output of the hosts is blank.
>
> ```
> ceph orch host ls
> ```

Related information

Registering the Red Hat Ceph Storage nodes
Creating an Ansible user with sudo access

Using the `addr` option to identify hosts

The `addr` option offers an additional way to contact a host. Add the IP address of the host to the `addr` option. If `ssh` cannot connect to the host by its hostname, then it uses the value stored in `addr` to reach the host by its IP address.

Prerequisites

Before you begin, make sure that you have the following prerequisites in place:

- A storage cluster is installed and bootstrapped.
- Root-level access to all nodes in the storage cluster.

Procedure

1. Log in to the `cephadm` shell:

Example

```
[root@host01 ~]# cephadm shell
```

2. Add the IP address:

   Syntax

   ```
   ceph orch host add HOSTNAME IP_ADDR
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph orch host add host02 10.10.128.68
   ```

   Note: If adding a host by hostname results in that host being added with an IPv6 address instead of an IPv4 address, use the **`ceph orch host`** command to specify the IP address of that host:
   Syntax

   ```
   ceph orch host set-addr HOSTNAME IP_ADDR
   ```

   To convert the IP address from IPv6 format to IPv4 format for a host you have added, use the following command:

   ```
   ceph orch host set-addr HOSTNAME IPV4_ADDRESS
   ```

Adding multiple hosts

Add multiple hosts at a time to the storage cluster, by using a YAML file.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A storage cluster is installed and bootstrapped.
- Root-level access to all nodes in the storage cluster.

About this task

Note: Be sure to create the `hosts.yaml` file within a host container, or create the file on the local host and then use the `cephadm` shell to mount the file within the container. The `cephadm` shell automatically places mounted files in `/mnt`. If you create the file directly on the local host and then apply the `hosts.yaml` file instead of mounting it, a File does not exist error message displays.

Procedure

1. Copy the public `ssh` key to each of the hosts that you want to add.
2. Use a text editor to create a `hosts.yaml` file.
3. Add the host descriptions to the `hosts.yaml` file.
   Include the labels to identify placements for the daemons that you want to deploy on each host. Separate each host description with three dashes (`---`).

   ```
   service_type: host
   addr:
   hostname: host02
   labels:
   - mon
   - osd
   - mgr
   ---
   service_type: host
   ```

```
addr:
hostname: host03
labels:
- mon
- osd
- mgr
---
service_type: host
addr:
hostname: host04
labels:
- mon
- osd
```

4. Mount the `hosts.yaml` file.
   - If you created the `hosts.yaml` file directly on the local host, use the `cephadm` shell to mount the file.

     ```
     cephadm shell --mount hosts.yaml -- ceph orch apply -i /mnt/hosts.yaml
     ```

     For example,

     ```
     [root@host01 ~]# cephadm shell --mount hosts.yaml -- ceph orch apply -i /mnt/
     hosts.yaml
     ```

   - If you created the `hosts.yaml` file within the host container, run the **`ceph orch apply`** command.
     For example,

     ```
     [ceph: root@host01 /]# ceph orch apply -i hosts.yaml
     Added host 'host02' with addr '10.10.128.69'
     Added host 'host03' with addr '10.10.128.70'
     Added host 'host04' with addr '10.10.128.71'
     ```

5. View the list of hosts and their labels.

   ```
   [ceph: root@host01 /]# ceph orch host ls
   HOST      ADDR      LABELS          STATUS
   host02   host02    mon,osd,mgr
   host03   host03    mon,osd,mgr
   host04   host04    mon,osd
   ```

   Note: If a host is online and operating normally, its status is blank. An offline host shows a status of `OFFLINE`, and a host in maintenance mode shows a status of `MAINTENANCE`.

Removing hosts

Remove Ceph cluster hosts with the Ceph Orchestrators.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to all the nodes.
- Hosts are added to the storage cluster.
- All the services are deployed.
- `cephadm` is deployed on the nodes where the services have to be removed.

About this task

All daemons are removed with the **`drain`** option which adds the `_no_schedule` label to ensure that you cannot deploy any daemons or a cluster till the operation is complete.

Important: If you are removing the bootstrap host, be sure to copy the admin keyring and the configuration file to another host in the storage cluster before you remove the host.

For more information, see the following operations information:

- Adding hosts using the Ceph Orchestrator
- Listing hosts using the Ceph Orchestrator

Procedure

1. Log into the `cephadm` shell.

   ```
   cephadm shell
   ```

   For example,

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Get the host details.

   ```
   ceph orch host ls
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch host ls
   ```

3. Drain all the daemons from the host.

   ```
   ceph orch host drain HOSTNAME
   ```

   Note: The ceph orch host drain command supports the **`--zap-osd-devices`** flag. When you set this flag, `cephadm` zaps the devices of the OSDs it removes during the drain process.

   For example,

   ```
   [ceph: root@host01 /]# ceph orch host drain host02
   ```

   The `_no_schedule` label is automatically applied to the host which blocks deployment.

4. Check the status of OSD removal.

   ```
   ceph orch osd rm status
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch osd rm status
   ```

   When no placement groups (PGs) remain on the OSD, the OSD is decommissioned and removed from the storage cluster.

5. Check if all the daemons are removed from the storage cluster.

   ```
   ceph orch ps HOSTNAME
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch ps host02
   ```

6. Remove the host from the orchestrator and the CRUSH map.
   Use the **`--rm-crush-entry`** flag to remove the host from both the orchestrator and the CRUSH map in a single operation. This ensures the host is completely removed from the cluster and does not appear in the `ceph osd tree` output.

```
ceph orch host rm HOSTNAME --rm-crush-entry
```

For example,

```
[ceph: root@host01 /]# ceph orch host rm host02 --rm-crush-entry
Removed host 'host02'
```

Alternative method: If you have already removed the host without the **`--rm-crush-entry`** flag, you can manually remove the host from the CRUSH map.

```
ceph osd crush remove HOSTNAME
```

For example,

```
[ceph: root@host01 /]# ceph osd crush remove host02
removed item id -5 name 'host02' from crush map
```

7. Verify that the host has been removed from both the orchestrator and the CRUSH map.
   a. Check that the host is not listed in the orchestrator.

      ```
      ceph orch host ls
      ```

      For example,

      ```
      [ceph: root@host01 /]# ceph orch host ls
      ```

   b. Verify that the host does not appear in the OSD tree.

      ```
      ceph osd tree
      ```

      For example,

      ```
      [ceph: root@host01 /]# ceph osd tree
      ```

      The host should not appear in either output.

Labeling hosts

The Ceph Orchestrator supports assigning both general labels and topological labels to hosts. Labels can be free-form with any text string. Topological labels are input with a hardware identifier and a user label. Each host can have multiple labels.

Labels

Labels are free-form user specified text. For example, use a label such as `mycluster_mon` on all hosts on which you want to deploy Ceph Monitor daemons. A second label of `mycluster_mgr` can then be placed on all hosts on which you want to deploy Ceph Manager daemons.
Labeling all the hosts in the storage cluster helps to simplify system management tasks by allowing you to quickly identify the daemons running on each host. In addition, you can use the Ceph Orchestrator or a YAML file to deploy or remove daemons on hosts that have specific host labels.

Topological labels

Use topological labels in multi-rack environments to help create distinct services for each rack. Topological labels enable upgrade operations on specific hosts.
For more information, see Running a staggered upgrade.

Before labeling hosts, be sure that you have a storage cluster installed and bootstrapped.

Adding labels to a host

Use the Ceph Orchestrator to add a label to a host. Labels can be used to specify placement of daemons.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A storage cluster is installed and bootstrapped.
- Root-level access to all nodes in the storage cluster.
- Hosts are added to the storage cluster.

About this task

A few examples of labels are `mgr`, `mon`, and `osd` based on the service deployed on the hosts. Each host can have multiple labels.

The following labels have special meaning to `cephadm`.

> **Note:** Special labels begin with _ (underscore).

`_no_schedule`

This label prevents `cephadm` from scheduling or deploying daemons on the host. If it is added to an existing host that already contains Ceph daemons, it causes `cephadm` to move those daemons elsewhere, except OSDs which are not removed automatically. When a host is added with the `_no_schedule` label, no daemons are deployed on it. When the daemons are drained before the host is removed, the `_no_schedule` label is set on that host.

`_no_autotune_memory`

This label does not autotune memory on the host. It prevents the daemon memory from being tuned even when the **`osd_memory_target_autotune`** option or other similar options are enabled for one or more daemons on that host.

`_admin`

By default, the `_admin` label is applied to the bootstrapped host in the storage cluster and the `client.admin` key is set to be distributed to that host with the `ceph orch client-keyring [ls|set|rm]` function. Adding this label to additional hosts normally causes `cephadm` to deploy configuration and keyring files in the `/etc/ceph` directory.

Procedure

1. Launch the `cephadm` shell.
   For example,

   ```
   [root@host01 ~]# cephadm shell
   [ceph: root@host01 /]#
   ```

2. Add a label to a host.

   ```
   ceph orch host label add HOSTNAME LABEL
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch host label add host02 mon
   ```

   > **Note:** If your Red Hat Ceph Storage deployment comprises multiple CRUSH bucket types, such as `chassis` or `rack`, also known as failure domains, that are higher than `host`, assign the `mon` label to no more than 2 hosts per failure domain. Limiting the `mon` label spreads the daemons across failure domains, ensuring Monitor quorum when entire failure domain is lost.

What to do next

Verify that the label is added to the host, by using the **`ceph orch host ls`** command.

Removing labels from a host

Use the Ceph Orchestrator to remove a label from a host.

Before you begin

- A Red Hat Ceph Storage cluster is installed and bootstrapped.
- Root-level access to all nodes in the storage cluster.
- Hosts are added to the storage cluster.

Procedure

1. Log into the `cephadm` shell.

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Remove the label.

   ```
   ceph orch host label rm HOST_NAME LABEL_NAME
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch host label rm host02 mon
   ```

What to do next

Verify that the label is moved from the host, by using the **`ceph orch host ls`** command.

Using host labels to deploy daemons on specific hosts

Use host labels to deploy daemons to specific hosts.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A Red Hat Ceph Storage cluster is installed and bootstrapped.
- Root-level access to all nodes in the storage cluster.

About this task

There are two ways to use host labels to deploy daemons on specific hosts:

- By using a YAML service specification file.

  > **Note:** Using the YAML file is the preferred method, as it aligns with Infrastructure as Code best practice.

- By using the **`--placement`** option from the command line.
  For example, **`ceph orch apply DAEMON --placement="label:LABEL"`**

Deploying daemons from the command line

Procedure

1. Log in to the `cephadm` shell.
   For example,

   ```
   [root@host01 ~]# cephadm shell
   ```

2. List all current hosts and labels by using the **`ceph orch host ls`** command.
   For example,

   ```
   [ceph: root@host01 /]# ceph orch host ls

   HOST     ADDR     LABELS               STATUS
   host01   host01   _admin,mon,osd,mgr
   host02   host02   mon,osd,mgr,mylabel
   ```

3. Use the **--placement** option to deploy a daemon from the command line.

```
ceph orch apply DAEMON --placement="label:LABEL"
```

For example,

```
[ceph: root@host01 /]# ceph orch apply prometheus --placement="label:mylabel"
```

Deploying daemons with a YAML file

Procedure

1. Log in to the `cephadm` shell.
For example,

```
[root@host01 ~]# cephadm shell
```

2. List all current hosts and labels by using the **ceph orch host ls** command.
For example,

```
[ceph: root@host01 /]# ceph orch host ls

HOST     ADDR     LABELS               STATUS
host01   host01   _admin,mon,osd,mgr
host02   host02   mon,osd,mgr,mylabel
```

3. Assign the daemon to a specific host label in a YAML file.
Be sure to specify the service type and label, by using your preferred text editor.
   a. Create the `placement.yml` file.
   For example,

```
[ceph: root@host01 /]# vi placement.yml
```

   b. Specify the service type and label within the YAML file.

```
service_type: prometheus
placement:
  label: "mylabel"
```

   c. Apply the daemon placement file.

```
ceph orch apply -i FILENAME
```

   For example,

```
[ceph: root@host01 /]# ceph orch apply -i placement.yml
Scheduled prometheus update…
```

What to do next

Verify the status of the daemons.

```
ceph orch ps --daemon_type=DAEMON_NAME
```

For example,

```
[ceph: root@host01 /]# ceph orch ps --daemon_type=prometheus
NAME              HOST   PORTS   STATUS        REFRESHED  AGE  MEM USE  MEM LIM  VERSION
 IMAGE ID      CONTAINER ID
prometheus.host02  host02  *:9095  running (2h)     8m ago   2h    85.3M        -  2.22.2
ac25aac5d567  ad8c7593d7c0
```

Adding topological labels to a host

Use topological labels in multi-rack environments to help `cephadm` be aware of the topological layout of the hosts in the cluster. Topological labels enable upgrade operations on specific hosts.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A storage cluster is installed and bootstrapped.
- Root-level access to all nodes in the storage cluster.

About this task

Add topological labels either with the command-line interface or with the host specification file.

Adding topological labels with the command-line interface

Add topological labels with the `cephadm` shell with either the **`host add`** or **`set-topological-labels`** commands.

About this task

Labels are written in the following format: `hardware_identifier=user_label`. Hardware identifiers can be data center or rack. Multiple labels can be used with a comma separator.

Procedure

Use one of the following commands to add topological labels to a host:

- Use the **`host add`** command with the **`--topological-labels`** option.

```
ceph orch host add HOSTNAME IP_ADDRESS --topological-labels
TOPOLOGICAL_LABEL1,TOPOLOGICAL_LABEL2
```

For example,

```
[root@host01] # ceph orch host add host02 10.10.128.69 --topological-labels
datacenter=A,rack=3
```

- Use the **`set-topological-labels`** command.

> **Important:** The **`set-topological-labels`** command overwrites any existing topological labels. Running the command with no labels specified clears all labels for the host. Running the command with labels replaces existing labels on the host. Labels used while running this command does not add to existing labels.

```
ceph orch host set-topological-labels HOSTNAME TOPOLOGICAL_LABEL1,TOPOLOGICAL_LABEL2
```

For example,

```
[root@host01] # ceph orch host set-topological-labels host02 datacenter=A,rack=3
```

Adding topological labels with a service specification file

Set topological labels from within the host specification file.

Procedure

- Add topical labels to the host service specification file.

```
service_type: host
hostname: HOSTNAME
```

```
addr: IP_ADDRESS
toplogical_labels:
  HARDWARE_IDENTIFIER1: USER_LABEL1
  HARDWARE_IDENTIFIER2: USER_LABEL2
```

For example,

```
service_type: host
hostname: host02
addr: 10.10.128.69
toplogical_labels:
  datacenter: A
  rack: 3
```

What to do next

Find existing topological labels for a host by using the **ceph orch host ls -f yaml** command.

Related information

Adding hosts
Adding multiple hosts
Running a staggered upgrade

Adding Monitor service

A typical Red Hat Ceph Storage storage cluster has three or five monitor daemons deployed on different hosts. If your storage cluster has five or more hosts, deploy five monitor nodes.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root-level access to all hosts in the storage cluster.
- A running storage cluster.

About this task

Note:

- In the case of a firewall, see "Firewall settings for Ceph Monitor node" on page 295.
- The bootstrap node is the initial monitor of the storage cluster. Be sure to include the bootstrap node in the list of hosts to which you want to deploy.
- If you want to apply monitor service to more than one specific host, be sure to specify all of the host names within the same **ceph orch apply** command. If you specify **ceph orch apply mon --placement host1** and then specify **ceph orch apply mon --placement host2**, the second command removes the monitor service on host1 and applies a monitor service to host2.

If your monitor nodes or your entire cluster are located on a single subnet, then `cephadm` automatically adds up to five Monitor daemons as you add new hosts to the cluster. `cephadm` automatically configures the monitor daemons on the new hosts. The new hosts reside on the same subnet as the first (bootstrap) host in the storage cluster. `cephadm` can also deploy and scale monitors to correspond to changes in the size of the storage cluster.

Important: In director-deployed Red Hat Ceph Storage environments, replacing Controller nodes can affect the Ceph Monitor service and lead to storage outages if Monitor IP addresses change. For more information, see "Managing the Ceph Monitor service with Red Hat OpenStack Platform" on page 563.

Procedure

1. Apply five monitor daemons to five random hosts in the storage cluster.

```
ceph orch apply mon 5
```

2. Disable automatic monitor deployment.

```
ceph orch apply mon --unmanaged
```

Deploying Ceph monitor nodes using host labels

A typical Red Hat Ceph Storage storage cluster has three or five Ceph Monitor daemons deployed on different hosts. If your storage cluster has five or more hosts, deploy five Ceph Monitor nodes. Use host labels to identify the hosts that contain Ceph Monitor nodes.

If your Ceph Monitor nodes or your entire cluster are located on a single subnet, then `cephadm` automatically adds up to five Ceph Monitor daemons as you add new nodes to the cluster. `cephadm` automatically configures the Ceph Monitor daemons on the new nodes. The new nodes reside on the same subnet as the first (bootstrap) node in the storage cluster. `cephadm` can also deploy and scale monitors to correspond to changes in the size of the storage cluster.

Prerequisites

Before you begin, make sure that you have the following prerequisites in place:

- Root-level access to all nodes in the storage cluster.
- A running storage cluster.

Procedure

1. Assign the mon label to the host:

   Syntax

   ```
   ceph orch host label add HOSTNAME mon
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph orch host label add host02 mon
   [ceph: root@host01 /]# ceph orch host label add host03 mon
   ```

2. View the current hosts and labels:

   Syntax

   ```
   ceph orch host ls
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph orch host ls
   HOST   ADDR   LABELS  STATUS
   host01        mon,mgr,_admin
   host02        mon
   host03        mon
   host04
   host05
   host06
   ```

3. Deploy Ceph Monitor daemons based on the host label:

   Syntax

   ```
   ceph orch apply mon label:mon
   ```

4. Deploy Ceph Monitor daemons on a specific set of hosts:

   Syntax

```
ceph orch apply mon HOSTNAME1,HOSTNAME2,HOSTNAME3
```

Example

```
[ceph: root@host01 /]# ceph orch apply mon host01,host02,host03
```

Note: Be sure to include the bootstrap node in the list of hosts to which you want to deploy.

Adding Ceph Monitor nodes by IP address or network name

A typical Red Hat Ceph Storage storage cluster has three or five Ceph Monitor daemons deployed on different hosts. If your storage cluster has five or more hosts, deploy five Ceph Monitor nodes. Add Ceph Monitor nodes by IP address or network name.

A typical Red Hat Ceph Storage storage cluster has three or five monitor daemons deployed on different hosts. If your storage cluster has five or more hosts, Red Hat recommends that you deploy five Monitor nodes.

If your Monitor nodes or your entire cluster are located on a single subnet, then `cephadm` automatically adds up to five Monitor daemons as you add new nodes to the cluster. You do not need to configure the Monitor daemons on the new nodes. The new nodes reside on the same subnet as the first node in the storage cluster. The first node in the storage cluster is the bootstrap node. `cephadm` can also deploy and scale monitors to correspond to changes in the size of the storage cluster.

Prerequisites

- Root-level access to all nodes in the storage cluster.
- A running storage cluster.

Procedure

- To deploy each additional Ceph Monitor node:

  Syntax

  ```
  ceph orch apply mon NODE:IP_ADDRESS_OR_NETWORK_NAME
  [NODE:IP_ADDRESS_OR_NETWORK_NAME...]
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph orch apply mon host02:10.10.128.69 host03:mynetwork
  ```

Setting up a custom SSH key on an existing cluster

As a storage administrator, with `cephadm`, you can use an SSH key to securely authenticate with remote hosts. The SSH key is stored in the monitor to connect to remote hosts.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- An Ansible administration node.
- Root-level access to the Ansible administration node.
- The `cephadm-ansible` package is installed on the node.

About this task

When the cluster is bootstrapped, this SSH key is generated automatically and no additional configuration is necessary. However, you can generate a new SSH key with the **`ceph cephadm generate-key`** command.

Procedure

1. Navigate to the `/usr/share/cephadm-ansible/` directory.
   For example,

   ```
   [root@admin ~]# cd /usr/share/cephadm-ansible
   ```

2. Generate a new SSH key.

   ```
   ceph cephadm generate-key
   ```

   For example,

   ```
   [ceph-admin@admin cephadm-ansible]$ ceph cephadm generate-key
   ```

3. Retrieve the public portion of the SSH key.

   ```
   ceph cephadm get-pub-key
   ```

   For example,

   ```
   [ceph-admin@admin cephadm-ansible]$ ceph cephadm get-pub-key
   ```

4. Delete the currently stored SSH key.

   ```
   ceph cephadm clear-key
   ```

   For example,

   ```
   [ceph-admin@admin cephadm-ansible]$ceph cephadm clear-key
   ```

5. Restart the `mgr` daemon to reload the configuration.

   ```
   ceph mgr fail
   ```

   For example,

   ```
   [ceph-admin@admin cephadm-ansible]$ ceph mgr fail
   ```

Configuring a different SSH user

As a storage administrator, you can configure a non-root SSH user who can log in to all the Ceph cluster nodes. Configure users with enough privileges to download container images, start containers, and run commands without prompting for a password.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- An Ansible administration node.
- Root-level access to the Ansible administration node.
- The `cephadm-ansible` package is installed on the node.
- Add the cluster SSH keys to the user's `authorized_keys`.
- Enable `passwordless` sudo access for the non-root users.

About this task

> **Important:** Before configuring a non-root SSH user, the cluster SSH key needs to be added to the user's `authorized_keys` file and non-root users must have `passwordless` sudo access.

Procedure

1. Navigate to the `/usr/share/cephadm-ansible` directory.
2. Provide the `cephadm` the name of the user who is going to perform all the `cephadm` operations.

   ```
   ceph cephadm set-user USER
   ```

   For example,

   ```
   ceph cephadm set-user USER
   ```

3. Retrieve the SSH public key.

   ```
   ceph cephadm get-pub-key > ~/ceph.pub
   ```

   For example,

   ```
   [ceph-admin@admin cephadm-ansible]$ ceph cephadm get-pub-key > ~/ceph.pub
   ```

4. Copy the SSH keys to all the hosts.

   ```
   ssh-copy-id -f -i ~/ceph.pub USER@HOST
   ```

   For example,

   ```
   [ceph-admin@admin cephadm-ansible]$ ssh-copy-id ceph-admin@host01
   ```

Setting up the admin node

Use an admin node to administer the storage cluster.

An admin node contains both the cluster configuration file and the admin keyring. Both of these files are stored in the directory `/etc/ceph` and use the name of the storage cluster as a prefix.

For example, the default ceph cluster name is `ceph`. In a cluster using the default name, the admin keyring is named `/etc/ceph/ceph.client.admin.keyring`. The corresponding cluster configuration file is named `/etc/ceph/ceph.conf`.

To set up additional hosts in the storage cluster as admin nodes, apply the `_admin` label to the host you want to designate as an administrator node.

> **Note:** By default, after applying the `_admin` label to a node, `cephadm` copies the `ceph.conf` and `client.admin` keyring files to that node. The `_admin` label is automatically applied to the bootstrap node unless the `--skip-admin-label` option was specified with the `cephadm bootstrap` command.

Prerequisites

- A running storage cluster with `cephadm` installed.
- The storage cluster has running Monitor and Manager nodes.
- Root-level access to all nodes in the cluster.

Procedure

1. Use **`ceph orch host ls`** to view the hosts in your storage cluster:

   Example

   ```
   [root@host01 ~]# ceph orch host ls
   HOST   ADDR   LABELS  STATUS
   host01        mon,mgr,_admin
   host02        mon
   host03        mon,mgr
   ```

```
host04
host05
host06
```

2. Use the `_admin` label to designate the admin host in your storage cluster. For best results, this host should have both Monitor and Manager daemons running.

   Syntax

   ```
   ceph orch host label add HOSTNAME _admin
   ```

   Example

   ```
   [root@host01 ~]#  ceph orch host label add host03 _admin
   ```

3. Verify that the admin host has the `_admin` label.

   Example

   ```
   [root@host01 ~]#  ceph orch host ls
   HOST   ADDR   LABELS  STATUS
   host01        mon,mgr,_admin
   host02        mon
   host03        mon,mgr,_admin
   host04
   host05
   host06
   ```

4. Log in to the admin node to manage the storage cluster.

Removing the admin label from a host

You can use the Ceph orchestrator to remove the admin label from a host.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running storage cluster with `cephadm` installed and bootstrapped.
- The storage cluster has running Monitor and Manager nodes.
- Root-level access to all nodes in the cluster.

Procedure

1. Use the **`ceph orch host ls`** command to view the hosts and identify the admin host in your storage cluster.
   For example,

   ```
   [root@host01 ~]# ceph orch host ls
    HOST   ADDR   LABELS  STATUS
    host01        mon,mgr,_admin
    host02        mon
    host03        mon,mgr,_admin
    host04
    host05
    host06
   ```

2. Log into the `cephadm` shell.
   For example,

   ```
   [root@host01 ~]# cephadm shell
   [ceph: root@host01 /]#
   ```

3. Use the Ceph Orchetrator to remove the `_admin` label from the host.

   ```
   ceph orch host label rm HOSTNAME LABEL
   ```

For example,

```
[ceph: root@host01 /]# ceph orch host label rm host03 _admin
```

What to do next

Verify that the admin host has the `_admin` label.

```
[root@host01 ~]#  ceph orch host ls
 HOST   ADDR   LABELS  STATUS
 host01        mon,mgr,_admin
 host02        mon
 host03        mon,mgr
 host04
 host05
 host06
```

Important:

- After removing the admin label from a node, ensure you remove the `ceph.conf` and `client.admin` keyring files from that node.
- The node must be removed from the admin Ansible inventory file.

Adding Manager service

`cephadm` automatically installs a Manager daemon on the bootstrap node during the bootstrapping process. Use the Ceph orchestrator to deploy additional Manager daemons.

The Ceph orchestrator deploys two Manager daemons by default. To deploy a different number of Manager daemons, specify a different number. If you do not specify the hosts where the Manager daemons should be deployed, the Ceph orchestrator randomly selects the hosts and deploys the Manager daemons to them.

Note: If you want to apply Manager daemons to more than one specific host, be sure to specify all of the host names within the same **`ceph orch apply`** command. If you specify **`ceph orch apply mgr --placement host1`** and then specify **`ceph orch apply mgr --placement host2`**, the second command removes the Manager daemon on host1 and applies a Manager daemon to host2.

Use the **`--placement`** option to deploy to specific hosts.

Before you begin, be sure to have a running storage cluster.

Important: In director-deployed Red Hat Ceph Storage environments, replacing Controller nodes can affect the Ceph Monitor service and lead to storage outages if Monitor IP addresses change. For more information, see “Managing the Ceph Monitor service with Red Hat OpenStack Platform” on page 563.

Procedure

- Specify that you want to apply a certain number of Manager daemons to randomly selected hosts.

  ```
  ceph orch apply mgr NUMBER_OF_DAEMONS
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph orch apply mgr 3
  ```

- Add Manager daemons to specific hosts in your storage cluster.

  ```
  ceph orch apply mgr --placement "HOSTNAME1 HOSTNAME2 HOSTNAME3"
  ```

Example

```
[ceph: root@host01 /]# ceph orch apply mgr --placement "host02 host03 host04"
```

Adding OSDs

Use this information to deploy OSDs on a specific device or host.

Before you begin

Before beginning make sure that you have a running Red Hat Ceph Storage cluster.

About this task

`cephadm` does not provision an OSD on a device that is not available. A storage device is considered available if it meets all of the following conditions:

- The device must have no partitions.
- The device must not be mounted.
- The device must not contain a file system.
- The device must not contain a Ceph BlueStore OSD.
- The device must be larger than 5 GB.

Important: By default, the **`osd_memory_target_autotune`** parameter is set to `true` in Red Hat Ceph Storage.

- For information about tuning OSD memory automatically, see Automatically tuning OSD memory.
- For more information about drive specifications for OSDs, see Advanced service specifications and filters for deploying OSDs.
- For more information about zapping devices to clear data on devices, see Zapping devices for Ceph OSD deployment.

Procedure

1. List the available devices to deploy OSDs.

   ```
   ceph orch device ls [--hostname=HOSTNAME1 HOSTNAME2] [--wide] [--refresh]
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch device ls --wide --refresh
   ```

2. You can either deploy the OSDs on specific hosts or on all the available devices.
   - Create an OSD from a specific device on a specific host, by using the following command:

     ```
     ceph orch daemon add osd HOSTNAME:DEVICE_PATH
     ```

     For example,

     ```
     [ceph: root@host01 /]# ceph orch daemon add osd host02:/dev/sdb
     ```

   - Deploy OSDs on any available and unused devices, by using use the **`--all-available-devices`** parameter.

     Note: This command creates colocated WAL and DB daemons. To create non-colocated daemons, do not use this command.

```
ceph orch apply osd --all-available-devices
```

For example,

```
[ceph: root@host01 /]# ceph orch apply osd --all-available-devices
```

What to do next

Verify that the OSDs are added by checking that the Ceph status is in a `Healthy` state with OSD counts.
For example,

```
[root@host01 ~]# cephadm shell ceph -s
  cluster:
    id:     8f43ad64-dd03-11ed-8ec8-525400c632ec
    health: HEALTH_OK

  services:
    mon: 3 daemons, quorum host01,host02,host03 (age 94m)
    mgr: host01.lbnhug(active, since 59m), standbys: host02.rofgay, host03.ohipra
    mds: 1/1 daemons up, 1 standby
    osd: 18 osds: 18 up (since 10m), 18 in (since 10m)
    rgw: 4 daemons active (2 hosts, 1 zones)

  data:
    volumes: 1/1 healthy
    pools:   8 pools, 225 pgs
    objects: 230 objects, 9.9 KiB
    usage:   271 MiB used, 269 GiB / 270 GiB
    avail pgs: 225 active+clean

  io:
    client: 85 B/s rd, 0 op/s rd, 0 op/s wr
```

Purging the Ceph storage cluster

Purging the Ceph storage cluster clears any data or connections that remain from previous deployments on your server.

Before you begin

Before you begin, be sure that you have a running Red Hat Ceph Storage.

About this task

For Red Hat Enterprise Linux 9, use the **`cephadm rm-cluster`** command, since Ansible is not supported.

Procedure

1. Disable `cephadm` to stop all the orchestration operations to avoid deploying new daemons.

   ```
   ceph mgr module disable cephadm
   ```

   For example,

   ```
   [ceph: root#host01 /]# ceph mgr module disable cephadm
   ```

2. Get the FSID of the cluster.

   ```
   ceph fsid
   ```

   For example,

   ```
   [ceph: root#host01 /]# ceph fsid
   ```

3. Exit the `cephadm` shell.
   For example,

   ```
   [ceph: root@host01 /]# exit
   ```

4. Purge the Ceph daemons from all hosts in the cluster.

```
cephadm rm-cluster --force --zap-osds --fsid FSID
```

For example,

```
[root@host01 ~]# cephadm rm-cluster --force --zap-osds  --fsid 2d2fd136-6df1-11ea-
ae74-002590e526e8
```

Deploying client nodes

As a storage administrator, you can deploy client nodes by running the `cephadm-preflight.yml` and `cephadm-clients.yml` playbooks.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root-level access to the Ansible administration node.
- Ansible user with sudo and passwordless `ssh` access to all nodes in the storage cluster.
- The `cephadm-ansible` package is installed.
- The prefight playbook has been run on the initial host in the storage cluster.
  For more information, see "Running the preflight playbook" on page 191.
- The `[clients]` group variable must be specified in the Ansible inventory file.
- The `[admin]` group is defined in the inventory file with a node where the admin keyring is present at `/etc/ceph/ceph.client.admin.keyring`.
  For more information about admin keys, see "Ceph user management" on page 445.

Red Hat Ceph Storage is supported on Intel and AMD x86-64 microprocessors.

- Red Hat Ceph Storage 9 introduces full support for clusters built with ARM (aarch64).

About this task

The `cephadm-preflight.yml` playbook configures the Ceph repository and prepares the storage cluster for bootstrapping. It also installs some prerequisites, such as `podman`, `lvm2`, `chrony`, and `cephadm`.

The `cephadm-clients.yml` playbook handles the distribution of configuration and keyring files to a group of Ceph clients.

> **Note:** If you are not using the `cephadm-ansible` playbooks, after upgrading your Ceph cluster, you must upgrade the `ceph-common` package and client libraries on your client nodes. For more information, see "Upgrading the Red Hat Ceph Storage cluster" on page 260.

Procedure

1. As an Ansible user, navigate to the `/usr/share/cephadm-ansible` directory on the Ansible administration node.
   For example,

```
[ceph-admin@admin ~]$ cd /usr/share/cephadm-ansible
```

2. Open and edit the `hosts` inventory file and add the `[clients]` group and clients to your inventory.

```
host02
host03
host04

[clients]
client01
client02
client03

[admin]
host01
```

3. Run the `cephadm-preflight.yml` playbook to install the prerequisites on the clients.

```
ansible-playbook -i INVENTORY_FILE cephadm-preflight.yml --limit CLIENT_GROUP_NAME|
CLIENT_NODE_NAME
```

For example,

```
[ceph-admin@admin cephadm-ansible]$ ansible-playbook -i hosts cephadm-preflight.yml --
limit clients
```

4. Run the `cephadm-clients.yml` playbook to distribute the keyring and Ceph configuration files to a set of clients.
   a. Copy the keyring with a custom destination keyring name.

```
ansible-playbook -i INVENTORY_FILE cephadm-clients.yml --extra-vars
'{"fsid":"FSID","keyring":"KEYRING_PATH","client_group":"CLIENT_GROUP_NAME","conf
":"CEPH_CONFIGURATION_PATH","keyring_dest":"KEYRING_DESTINATION_PATH"}'
```

Replace the following values:

INVENTORY_FILE

Repalce with the Ansible inventory file name.

FSID

Replace with the FSID of the cluster.
To get the FSID, use the **ceph fsid** command.

proc_osd_adding-osds*KEYRING_PATH*

Replace with the full path name to the keyring on the admin host that you want to copy to the client.

CLIENT_GROUP_NAME

Optional. Replace with the Ansible group name for the clients to set up.

CEPH_CONFIGURATION_PATH

Optional. Replace with the full path to the Ceph configuration file on the admin node.

KEYRING_DESTINATION_PATH

Optional. Replace with the full path name of the destination where the keyring will be copied.

> **Note:** If you do not specify a configuration file with the `conf` option when you run the playbook, the playbook generates and distributes a minimal configuration file. By default, the generated file is located at `/etc/ceph/ceph.conf`.

For example,

```
[ceph-admin@host01 cephadm-ansible]$ ansible-playbook -i hosts cephadm-
clients.yml --extra-vars '{"fsid":"266ee7a8-2a05-11eb-
b846-5254002d4916","keyring":"/etc/ceph/
ceph.client.admin.keyring","client_group":"clients","conf":"/etc/ceph/
ceph.conf","keyring_dest":"/etc/ceph/custom.name.ceph.keyring"}'
```

b. Copy a keyring with the default destination keyring name of `ceph.keyring` and using the default group of `clients`.

```
ansible-playbook -i INVENTORY_FILE cephadm-clients.yml --extra-vars
'{"fsid":"FSID","keyring":"KEYRING_PATH","conf":"CONF_PATH"}'
```

For example,

```
[ceph-admin@host01 cephadm-ansible]$ ansible-playbook -i hosts cephadm-
clients.yml --extra-vars '{"fsid":"266ee7a8-2a05-11eb-
b846-5254002d4916","keyring":"/etc/ceph/ceph.client.admin.keyring","conf":"/etc/
ceph/ceph.conf"}'
```

What to do next

Log into the client nodes and verify that the keyring and configuration files exist.
For example,

```
[user@client01 ~]# ls -l /etc/ceph/

  -rw-------. 1 ceph ceph 151 Jul 11 12:23 custom.name.ceph.keyring
  -rw-------. 1 ceph ceph 151 Jul 11 12:23 ceph.keyring
  -rw-------. 1 ceph ceph 269 Jul 11 12:23 ceph.conf
```

Managing a Red Hat Ceph Storage cluster using `cephadm-ansible` modules

Use the `cephadm-ansible` modules in Ansible playbooks to manage your Red Hat Ceph Storage cluster. The `cephadm-ansible` package includes several modules that wrap `cephadm` commands, allowing you to create custom Ansible playbooks to administer your cluster. These modules help ensure consistent configuration and automate tasks across your Ceph deployment.

> **Note:** `cephadm-ansible` modules do not cover all tasks. Any operation that is not covered by `cephadm-ansible` modules must be completed by using either the `command` or `shell` Ansible modules in your playbooks.

`cephadm-ansible` modules

The `cephadm-ansible` modules are a collection of modules that simplify writing Ansible playbooks by providing a wrapper around `cephadm` and `ceph orch` commands. You can use the modules to write your own unique Ansible playbooks to administer your cluster using one or more of the modules.

The `cephadm-ansible` package includes the following modules:

- `cephadm_bootstrap`
- `ceph_orch_host`
- `ceph_config`
- `ceph_orch_apply`
- `ceph_orch_daemon`
- `cephadm_registry_login`

`cephadm-ansible` modules options

This section lists the available options for the `cephadm-ansible` modules.

Options listed in the following tables with the **required** value as`true`need to be set when using the modules in your Ansible playbooks. Options that are listed with a default value of `true` indicate that the option is automatically set when using the modules and you do not need to specify it in your playbook. For example, for the `cephadm_bootstrap` module, the Ceph Dashboard is installed unless you set `dashboard: false`.

Table 35: Available options for the `cephadm_bootstrap` module

cephadm_bootstrap	Description	Required	Default
mon_ip	Ceph Monitor IP address.	true	None
image	Ceph container image.	false	None
docker	Use `docker` instead of `podman`.	false	None
fsid	Define the Ceph FSID.	false	None
pull	Pull the Ceph container image.	false	true
dashboard	Deploy the Ceph Dashboard.	false	true
dashboard_user	Specify a specific Ceph Dashboard user.	false	None
dashboard_password	Ceph Dashboard password.	false	None
monitoring	Deploy the monitoring stack.	false	true
firewalld	Manage firewall rules with firewalld.	false	true
allow_overwrite	Allow overwrite of existing **`--output-config`**, **`--output-keyring`**, or **`--output-pub-ssh-key`** files.	false	false
registry_url	URL for custom registry.	false	None
registry_username	Username for custom registry.	false	None
registry_password	Password for custom registry.	false	None
registry_json	JSON file with custom registry login information.	false	None
ssh_user	SSH user to use for `cephadm` ssh to hosts.	false	None
ssh_config	SSH config file path for `cephadm` SSH client.	false	None
allow_fqdn_hostname	Allow hostname that is a fully-qualified domain name (FQDN).	false	false
cluster_network	Subnet to use for cluster replication, recovery and heartbeats.	false	None

Table 36: Available options for the `ceph_orch_host` module

ceph_orch_host	Description	Required	Default
fsid	The FSID of the Ceph cluster to interact with.	false	None
image	The Ceph container image to use.	false	None
name	Name of the host to add, remove, or update.	true	None

ceph_orch_host	Description	Required	Default
address	IP address of the host.	true when state is present.	None
set_admin_label	Set the _admin label on the specified host.	false	false
labels	The list of labels to apply to the host.	false	[]
state	If set to present, it ensures the name specified in name is present. If set to absent, it removes the host specified in name. If set to drain, it schedules to remove all daemons from the host specified in name.	false	present

Table 37: Available options for the ceph_config module

ceph_config	Description	Required	Default
fsid	The FSID of the Ceph cluster to interact with.	false	None
image	The Ceph container image to use.	false	None
action	Whether to set or get the parameter specified in option.	false	set
who	Which daemon to set the configuration to.	true	None
option	Name of the parameter to set or get.	true	None
value	Value of the parameter to set.	true if action is set	None

Table 38: Available options for the ceph_orch_apply module

ceph_orch_apply	Description	Required
fsid	The FSID of the Ceph cluster to interact with.	false
image	The Ceph container image to use.	false
spec	The service specification to apply.	true

Table 39: Available options for the ceph_orch_daemon module

ceph_orch_daemon	Description	Required
fsid	The FSID of the Ceph cluster to interact with.	false
image	The Ceph container image to use.	false

ceph_orch_daemon	Description	Required
state	The desired state of the service specified in name.	true – If started, it ensures the service is started. – If stopped, it ensures the service is stopped. – If restarted, it will restart the service.
daemon_id	The ID of the service.	true
daemon_type	The type of service.	true

Table 40: Available options for the cephadm_registry_login module

cephadm_registry_login	Description	Required	Default
state	Login or logout of a registry.	false	login
docker	Use docker instead of podman.	false	None
registry_url	The URL for custom registry.	false	None
registry_username	Username for custom registry.	true when state is login.	None
registry_password	Password for custom registry.	true when state is login.	None
registry_json	The path to a JSON file. This file must be present on remote hosts prior to running this task. This option is currently not supported.	None	None

Bootstrapping a storage cluster using the cephadm_ansible modules

As a storage administrator, you can bootstrap a storage cluster using the cephadm-ansible modules such as cephadm_bootstrap and cephadm_registry_login playbook.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- An IP address for the first Ceph Monitor container, which is also the IP address for the first node in the storage cluster.
- Login access to registry.redhat.io.
- A minimum of 10 GB of free space for /var/lib/containers/.
- Red Hat Enterprise Linux 9.6, 9.7, 10, or 10.1 with ansible-core available from the corresponding RHEL AppStream repository.
- Installation of the cephadm-ansible package on the Ansible administration node.
- Passwordless SSH is set up on all hosts in the storage cluster.
- Hosts are registered with CDN.

For the latest supported Red Hat Enterprise Linux versions, see Compatibility matrix.

Procedure

1. Log in to the Ansible administration node.
2. Navigate to the `/usr/share/cephadm-ansible` directory on the Ansible administration node.
   For example,

   ```
   [ansible@admin ~]$ cd /usr/share/cephadm-ansible
   ```

3. Create the `hosts` file and add hosts, labels, and monitor IP address of the first host in the storage cluster.

   ```
   sudo vi INVENTORY_FILE

   HOST1 labels="[LABEL1, LABEL2]"
   HOST2 labels="[LABEL1, LABEL2]"
   HOST3 labels="[LABEL1]"

   [admin]
   ADMIN_HOST monitor_address=MONITOR_IP_ADDRESS labels="[ADMIN_LABEL, LABEL1, LABEL2]"
   ```

   For example,

   ```
   ansible@admin cephadm-ansible]$ sudo vi hosts

   host02 labels="['mon', 'mgr']"
   host03 labels="['mon', 'mgr']"
   host04 labels="['osd']"
   host05 labels="['osd']"
   host06 labels="['osd']"

   [admin]
   host01 monitor_address=10.10.128.68 labels="['_admin', 'mon', 'mgr']"
   ```

4. Run the preflight playbook.

   ```
   ansible-playbook -i INVENTORY_FILE cephadm-preflight.yml --extra-vars
   "ceph_origin=rhcs"
   ```

   For example,

   ```
   [ansible@admin cephadm-ansible]$ ansible-playbook -i hosts cephadm-preflight.yml --
   extra-vars "ceph_origin=rhcs"
   ```

5. Create a playbook to bootstrap your cluster.

   ```
   sudo vi PLAYBOOK_FILENAME.yml

   ---
   - name: NAME_OF_PLAY
     hosts: BOOTSTRAP_HOST
     become: USE_ELEVATED_PRIVILEGES
     gather_facts: GATHER_FACTS_ABOUT_REMOTE_HOSTS
     tasks:
       - name: NAME_OF_TASK
         cephadm_registry_login:
           state: STATE
           registry_url: REGISTRY_URL
           registry_username: REGISTRY_USER_NAME
           registry_password: REGISTRY_PASSWORD

       - name: NAME_OF_TASK
         cephadm_bootstrap:
           mon_ip: ""{{ monitor_address }}""
           dashboard_user: DASHBOARD_USER
           dashboard_password: DASHBOARD_PASSWORD
           allow_fqdn_hostname: ALLOW_FQDN_HOSTNAME
           cluster_network: NETWORK_CIDR
   ```

   For example,

```
[ansible@admin cephadm-ansible]$ sudo vi bootstrap.yml

---
- name: bootstrap the cluster
  hosts: host01
  become: true
  gather_facts: false
  tasks:
    - name: login to registry
      cephadm_registry_login:
        state: login
        registry_url: registry.redhat.io
        registry_username: user1
        registry_password: mypassword1

    - name: bootstrap initial cluster
      cephadm_bootstrap:
        mon_ip: "{{ monitor_address }}"
        dashboard_user: mydashboarduser
        dashboard_password: mydashboardpassword
        allow_fqdn_hostname: true
        cluster_network: 10.10.128.0/28
```

6. Run the playbook.

```
ansible-playbook -i INVENTORY_FILE PLAYBOOK_FILENAME.yml -vvv
```

For example,

```
ansible@admin cephadm-ansible]$ ansible-playbook -i hosts bootstrap.yml -vvv
```

What to do next

Review the Ansible output after running the playbook.

Adding or removing hosts using the `ceph_orch_host` module

Add and remove hosts in your storage cluster by using the `ceph_orch_host` module in your Ansible playbook.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Register the nodes to the CDN and attach subscriptions.
- Ansible user with sudo and passwordless SSH access to all nodes in the storage cluster.
- Installation of the `cephadm-ansible` package on the Ansible administration node.
- New hosts have the storage cluster's public SSH key. For more information about copying the storage cluster's public SSH keys to new hosts, see "Adding hosts" on page 223.

Add new hosts to the cluster

1. Log in to the Ansible administration node.
2. Navigate to the `/usr/share/cephadm-ansible` directory on the Ansible administration node.
   For example,

```
[ansible@admin ~]$ cd /usr/share/cephadm-ansible
```

3. Add the new hosts and labels to the Ansible inventory file.

```
sudo vi INVENTORY_FILE

NEW_HOST1 labels="[LABEL1, LABEL2]"
NEW_HOST2 labels="[LABEL1, LABEL2]"
NEW_HOST3 labels="[LABEL1]"

[admin]
ADMIN_HOST monitor_address=MONITOR_IP_ADDRESS labels="[ADMIN_LABEL, LABEL1, LABEL2]"
```

For example,

```
[ansible@admin cephadm-ansible]$ sudo vi hosts

host02 labels="['mon', 'mgr']"
host03 labels="['mon', 'mgr']"
host04 labels="['osd']"
host05 labels="['osd']"
host06 labels="['osd']"

[admin]
host01 monitor_address= 10.10.128.68 labels="['_admin', 'mon', 'mgr']"
```

4. Run the preflight playbook with the **--limit** option.

```
ansible-playbook -i INVENTORY_FILE cephadm-preflight.yml --extra-vars
"ceph_origin=rhcs" --limit NEWHOST
```

For example ,

```
[ansible@admin cephadm-ansible]$ ansible-playbook -i hosts cephadm-preflight.yml --
extra-vars "ceph_origin=rhcs" --limit host02
```

The preflight playbook installs `podman`, `lvm2`, `chrony`, and `cephadm` on the new host. After installation is complete, `cephadm` resides in the `/usr/sbin/` directory.

5. Create a playbook to add the new hosts to the cluster.

```
sudo vi PLAYBOOK_FILENAME.yml

---
- name: PLAY_NAME
  hosts: HOSTS_OR_HOST_GROUPS
  become: USE_ELEVATED_PRIVILEGES
  gather_facts: GATHER_FACTS_ABOUT_REMOTE_HOSTS
  tasks:
    - name: NAME_OF_TASK
      ceph_orch_host:
        name: "{{ ansible_facts[hostname] }}"
        address: "{{ ansible_facts[default_ipv4][address] }}"
        labels: "{{ labels }}"
      delegate_to: HOST_TO_DELEGATE_TASK_TO

    - name: NAME_OF_TASK
      when: inventory_hostname in groups[admin]
      ansible.builtin.shell:
        cmd: CEPH_COMMAND_TO_RUN
      register: REGISTER_NAME

    - name: NAME_OF_TASK
      when: inventory_hostname in groups[admin]
      debug:
        msg: "{{ REGISTER_NAME.stdout }}"
```

> **Note:** By default, Ansible runs all tasks on the host that matches the `hosts` line of your playbook. The **ceph orch** commands must run on the host that contains the admin keyring and the Ceph configuration file. Use the `delegate_to` keyword to specify the admin host in your cluster.

In the following example, the playbook adds the new hosts to the cluster and displays a current list of hosts.

```
[ansible@admin cephadm-ansible]$ sudo vi add-hosts.yml

---
- name: add additional hosts to the cluster
  hosts: all
  become: true
  gather_facts: true
  tasks:
    - name: add hosts to the cluster
      ceph_orch_host:
        name: "{{ ansible_facts['hostname'] }}"
        address: "{{ ansible_facts['default_ipv4']['address'] }}"
        labels: "{{ labels }}"
      delegate_to: host01

    - name: list hosts in the cluster
      when: inventory_hostname in groups['admin']
      ansible.builtin.shell:
        cmd: ceph orch host ls
      register: host_list

    - name: print current list of hosts
      when: inventory_hostname in groups['admin']
      debug:
        msg: "{{ host_list.stdout }}"
```

6. Run the playbook to add additional hosts to the cluster.

```
ansible-playbook -i INVENTORY_FILE PLAYBOOK_FILENAME.yml
```

For example,

```
[ansible@admin cephadm-ansible]$ ansible-playbook -i hosts add-hosts.yml
```

Remove hosts from the cluster

1. Log in to the Ansible administration node.
2. Navigate to the `/usr/share/cephadm-ansible` directory on the Ansible administration node. For example,

```
[ansible@admin ~]$ cd /usr/share/cephadm-ansible
```

3. Create a playbook to remove a host or hosts from the cluster.

```
sudo vi PLAYBOOK_FILENAME.yml

---
- name: NAME_OF_PLAY
  hosts: ADMIN_HOST
  become: USE_ELEVATED_PRIVILEGES
  gather_facts: GATHER_FACTS_ABOUT_REMOTE_HOSTS
  tasks:
    - name: NAME_OF_TASK
      ceph_orch_host:
        name: HOST_TO_REMOVE
        state: STATE

    - name: NAME_OF_TASK
      ceph_orch_host:
        name: HOST_TO_REMOVE
        state: STATE
      retries: NUMBER_OF_RETRIES
      delay: DELAY
      until: CONTINUE_UNTIL
      register: REGISTER_NAME

    - name: NAME_OF_TASK
      ansible.builtin.shell:
        cmd: ceph orch host ls
      register: REGISTER_NAME

    - name: NAME_OF_TASK
        debug:
          msg: "{{ REGISTER_NAME.stdout }}"
```

In the following example the playbook tasks drain all daemons on `host07`, removes the host from the cluster, and displays a current list of hosts.

```
[ansible@admin cephadm-ansible]$ sudo vi remove-hosts.yml

---
- name: remove host
  hosts: host01
  become: true
  gather_facts: true
  tasks:
    - name: drain host07
      ceph_orch_host:
        name: host07
        state: drain

    - name: remove host from the cluster
      ceph_orch_host:
        name: host07
        state: absent
      retries: 20
      delay: 1
      until: result is succeeded
      register: result

     - name: list hosts in the cluster
       ansible.builtin.shell:
         cmd: ceph orch host ls
       register: host_list

     - name: print current list of hosts
       debug:
         msg: "{{ host_list.stdout }}"
```

In this example, the playbook tasks drain all daemons on `host07`, removes the host from the cluster, and displays a current list of hosts.

4. Run the playbook to remove host from the cluster.

```
ansible-playbook -i INVENTORY_FILE PLAYBOOK_FILENAME.yml
```

For example,

```
[ceph-admin@admin cephadm-ansible]$ ansible-playbook -i hosts remove-hosts.yml
```

Verification

– Review the Ansible task output displaying the current list of hosts in the cluster.

```
TASK [print current hosts]
*****************************************************************************************
***************
Friday 24 June 2022  14:52:40 -0400 (0:00:03.365)       0:02:31.702 ***********
ok: [host01] =>
  msg: |-
    HOST    ADDR           LABELS          STATUS
    host01  10.10.128.68   _admin mon mgr
    host02  10.10.128.69   mon mgr
    host03  10.10.128.70   mon mgr
    host04  10.10.128.71   osd
    host05  10.10.128.72   osd
    host06  10.10.128.73   osd
```

Setting configuration options using the `ceph_config` module

As a storage administrator, you can set or get Red Hat Ceph Storage configuration options using the `ceph_config` module.

Prerequisites

– A running Red Hat Ceph Storage cluster.

- Ansible user with sudo and passwordless SSH access to all nodes in the storage cluster.
- Installation of the `cephadm-ansible` package on the Ansible administration node.
- The Ansible inventory file contains the cluster and admin hosts. For more information about adding hosts to your storage cluster, see “Adding or removing hosts using the ceph_orch_host module” on page 250.

Procedure

1. Log in to the Ansible administration node.
2. Navigate to the `/usr/share/cephadm-ansible` directory on the Ansible administration node:

   Example

   ```
   [ansible@admin ~]$ cd /usr/share/cephadm-ansible
   ```

3. Create a playbook with configuration changes:

   Syntax

   ```
   sudo vi PLAYBOOK_FILENAME.yml

   ---
   - name: PLAY_NAME
     hosts: ADMIN_HOST
     become: USE_ELEVATED_PRIVILEGES
     gather_facts: GATHER_FACTS_ABOUT_REMOTE_HOSTS
     tasks:
       - name: NAME_OF_TASK
         ceph_config:
           action: GET_OR_SET
           who: DAEMON_TO_SET_CONFIGURATION_TO
           option: CEPH_CONFIGURATION_OPTION
           value: VALUE_OF_PARAMETER_TO_SET

       - name: NAME_OF_TASK
         ceph_config:
           action: GET_OR_SET
           who: DAEMON_TO_SET_CONFIGURATION_TO
           option: CEPH_CONFIGURATION_OPTION
         register: REGISTER_NAME

       - name: NAME_OF_TASK
         debug:
           msg: "MESSAGE_TO_DISPLAY {{ REGISTER_NAME.stdout }}"
   ```

   Example

   ```
   [ansible@admin cephadm-ansible]$ sudo vi change_configuration.yml

   ---
   - name: set pool delete
     hosts: host01
     become: true
     gather_facts: false
     tasks:
       - name: set the allow pool delete option
         ceph_config:
           action: set
           who: mon
           option: mon_allow_pool_delete
           value: true

       - name: get the allow pool delete setting
         ceph_config:
           action: get
           who: mon
           option: mon_allow_pool_delete
         register: verify_mon_allow_pool_delete

       - name: print current mon_allow_pool_delete setting
         debug:
           msg: "the value of 'mon_allow_pool_delete' is
   {{ verify_mon_allow_pool_delete.stdout }}"
   ```

In this example, the playbook first sets the `mon_allow_pool_delete` option to `false`. The playbook then gets the current `mon_allow_pool_delete` setting and displays the value in the Ansible output.

4. Run the playbook:

   Syntax

   ```
   ansible-playbook -i INVENTORY_FILE _PLAYBOOK_FILENAME.yml
   ```

   Example

   ```
   [ansible@admin cephadm-ansible]$ ansible-playbook -i hosts change_configuration.yml
   ```

Verification

- Review the output from the playbook tasks.

  Example

  ```
  TASK [print current mon_allow_pool_delete setting]
  *************************************************************
  Wednesday 29 June 2022  13:51:41 -0400 (0:00:05.523)       0:00:17.953 ********
  ok: [host01] =>
    msg: the value of 'mon_allow_pool_delete' is true
  ```

Reference

- For more information, see “Configuring Red Hat Ceph Storage” on page 277.

Applying a service specification using the `ceph_orch_apply` module

As a storage administrator, you can apply service specifications to your storage cluster using the `ceph_orch_apply` module in your Ansible playbooks. A service specification is a data structure to specify the service attributes and configuration settings that is used to deploy the Ceph service. You can use a service specification to deploy Ceph service types like `mon`, `crash`, `mds`, `mgr`, `osd`, `rbd`, or `rbd-mirror`.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Ansible user with sudo and passwordless SSH access to all nodes in the storage cluster.
- Installation of the `cephadm-ansible` package on the Ansible administration node.
- The Ansible inventory file contains the cluster and admin hosts. For more information about adding hosts to your storage cluster, see “Adding or removing hosts using the ceph_orch_host module” on page 250.

Procedure

1. Log in to the Ansible administration node.
2. Navigate to the `/usr/share/cephadm-ansible` directory on the Ansible administration node:

   Example

   ```
   [ansible@admin ~]$ cd /usr/share/cephadm-ansible
   ```

3. Create a playbook with the service specifications:

   Syntax

   ```
   sudo vi PLAYBOOK_FILENAME.yml

   ---
   - name: PLAY_NAME
     hosts: HOSTS_OR_HOST_GROUPS
     become: USE_ELEVATED_PRIVILEGES
   ```

```
  gather_facts: GATHER_FACTS_ABOUT_REMOTE_HOSTS
  tasks:
    - name: NAME_OF_TASK
      ceph_orch_apply:
        spec: |
          service_type: SERVICE_TYPE
          service_id: UNIQUE_NAME_OF_SERVICE
          placement:
            host_pattern: HOST_PATTERN_TO_SELECT_HOSTS
            label: LABEL
          spec:
            SPECIFICATION_OPTIONS:
```

Example

```
[ansible@admin cephadm-ansible]$ sudo vi deploy_osd_service.yml

---
- name: deploy osd service
  hosts: host01
  become: true
  gather_facts: true
  tasks:
    - name: apply osd spec
      ceph_orch_apply:
        spec: |
          service_type: osd
          service_id: osd
          placement:
            host_pattern: '*'
            label: osd
          spec:
            data_devices:
              all: true
```

In this example, the playbook deploys the Ceph OSD service on all hosts with the label `osd`.

4. Run the playbook:

   Syntax

   ```
   ansible-playbook -i INVENTORY_FILE _PLAYBOOK_FILENAME.yml
   ```

   Example

   ```
   [ansible@admin cephadm-ansible]$ ansible-playbook -i hosts deploy_osd_service.yml
   ```

Verification

- Review the output from the playbook tasks.

Managing Ceph daemon states using the `ceph_orch_daemon` module

Start, stop, and restart Ceph daemons on hosts using the `ceph_orch_daemon` module in your Ansible playbooks.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Ansible user with sudo and passwordless SSH access to all nodes in the storage cluster.
- Installation of the `cephadm-ansible` package on the Ansible administration node.
- The Ansible inventory file contains the cluster and admin hosts. For more information about adding hosts to your storage cluster, see “Adding or removing hosts using the ceph_orch_host module” on page 250.

Procedure

1. Log in to the Ansible administration node.

2. Navigate to the `/usr/share/cephadm-ansible` directory on the Ansible administration node:

   Example

   ```
   [ansible@admin ~]$ cd /usr/share/cephadm-ansible
   ```

3. Create a playbook with daemon state changes:

   Syntax

   ```
   sudo vi PLAYBOOK_FILENAME.yml

   ---
   - name: PLAY_NAME
     hosts: ADMIN_HOST
     become: USE_ELEVATED_PRIVILEGES
     gather_facts: GATHER_FACTS_ABOUT_REMOTE_HOSTS
     tasks:
       - name: NAME_OF_TASK
         ceph_orch_daemon:
           state: STATE_OF_SERVICE
           daemon_id: DAEMON_ID
           daemon_type: TYPE_OF_SERVICE
   ```

   Example

   ```
   [ansible@admin cephadm-ansible]$ sudo vi restart_services.yml

   ---
   - name: start and stop services
     hosts: host01
     become: true
     gather_facts: false
     tasks:
       - name: start osd.0
         ceph_orch_daemon:
           state: started
           daemon_id: 0
           daemon_type: osd

       - name: stop mon.host02
         ceph_orch_daemon:
           state: stopped
           daemon_id: host02
           daemon_type: mon
   ```

   In this example, the playbook starts the OSD with an ID of `0` and stops a Ceph Monitor with an id of `host02`.

4. Run the playbook:

   Syntax

   ```
   ansible-playbook -i INVENTORY_FILE _PLAYBOOK_FILENAME.yml
   ```

   Example

   ```
   [ansible@admin cephadm-ansible]$ ansible-playbook -i hosts restart_services.yml
   ```

Verification

- Review the output from the playbook tasks.

Day Two operations

As a storage administrator after installing and configuring the Red Hat Ceph Storage, you can begin performing "Day Two" operations for your storage cluster. These tasks include adding metadata servers (MDS) and Ceph Object Gateways (RGW), and configuring other services to optimize and manage your cluster's performance and capabilities.

For more information about how to use the `cephadm` orchestrator to run "Day Two" operations, see “Operations” on page 542.

To deploy, configure, and administer the Ceph Object Gateway on "Day Two" operations, see “Red Hat Ceph Storage Object Gateways” on page 80.

Use the Ceph Dashboard to monitor clusters. For more information, see “Monitor Red Hat Ceph Storage clusters” on page 366.

Upgrading Red Hat Ceph Storage

Upgrade to an Red Hat Ceph Storage cluster running Red Hat Enterprise Linux on AMD64 and Intel 64 architectures.

Supported upgrade paths

Upgrade to Red Hat Ceph Storage based on the support that is provided by Red Hat.

Red Hat upgrade paths

Supported upgrade paths details the upgrade paths that are supported to upgrade from older Red Hat Ceph Storage versions to Red Hat Ceph Storage 9.

Table 41: Supported Red Hat Ceph Storage upgrade paths

Pre-upgrade	Post-upgrade
Red Hat Ceph Storage 7[1]	Red Hat Ceph Storage 9
Red Hat Ceph Storage 8	Red Hat Ceph Storage 9

[1]To upgrade Ceph Object Gateway (RGW) with multi-site to use`cephadm` self-signed certificates from Red Hat Ceph Storage 7.1, see "Updating Ceph Object Gateway to use cephadm self-signed certificates in a multi-site deployment" on page 266.

For more information and the latest supported configurations, see Red Hat Ceph Storage: Supported configurations.

Upgrading a Red Hat Ceph Storage cluster using `cephadm`

As a storage administrator, you can use the `cephadm` Orchestrator to upgrade to the latest version of Red Hat Ceph Storage by using the **`ceph orch upgrade`** command.

The automated upgrade process follows Ceph best practices.

- The upgrade order starts with Ceph Managers, Ceph Monitors, then other daemons.
- Each daemon is restarted only after Ceph indicates that the cluster will remain available.

For information about upgrade paths, see "Supported upgrade paths" on page 259.

Before starting to upgrade Red Hat Ceph Storage, see Installation and upgrade requirements.

The storage cluster health status is likely to switch to `HEALTH_WARNING` during the upgrade. When the upgrade is complete, the health status should switch back to `HEALTH_OK`.

> **Note:** You do not get a message once the upgrade is successful. Run **`ceph versions`** and **`ceph orch ps`** commands to verify the new image ID and the version of the storage cluster.

Compatibility considerations between RHCS and `podman` versions

`podman` and Red Hat Ceph Storage have different end-of-life strategies that might make it challenging to find compatible versions.

Red Hat recommends to use the `podman` version shipped with the corresponding Red Hat Enterprise Linux version for Red Hat Ceph Storage. See the Red Hat Ceph Storage: Supported configurations knowledge base article for more details. See the Contacting Red Hat support for service section in "Red Hat Ceph Storage Troubleshooting" on page 1579 for additional assistance.

The following table shows version compatibility between Red Hat Ceph Storage 9 and versions of `podman`.

Important: You must use a version of Podman that is 2.0.0 or higher.

Table 42: Red Hat Ceph Storage 9 and podman version compatibility

Ceph	Podman					
	1.9	2.0	2.1	2.2	3.0	>3.0
Red Hat Ceph Storage 9	false	true	true	false	true	true

Upgrading the Red Hat Ceph Storage cluster

Use the **ceph orch upgrade** command to upgrade an Red Hat Ceph Storage cluster. Alternatively, update the cluster through the dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A supported Red Hat Ceph Storage cluster version. For more information, see Red Hat Ceph Storage: Supported configurations.
- Root-level access to all the nodes.
- At least two Ceph Manager nodes in the storage cluster: one active and one standby.
- Ansible user with sudo and passwordless ssh access to all nodes in the storage cluster.

Note: Red Hat Ceph Storage includes a health check function that returns a DAEMON_OLD_VERSION warning if it detects daemons in the storage cluster are running multiple versions of Red Hat Ceph Storage. The warning is triggered when the daemons continue to run multiple versions of Red Hat Ceph Storage beyond the time value set in the `mon_warn_older_version_delay` option. By default, the `mon_warn_older_version_delay` option is set to 1 week. This setting allows most upgrades to proceed without falsely seeing the warning.
If the upgrade process is paused for an extended time period, you can mute the health warning by using the **ceph health mute** command.

```
ceph health mute DAEMON_OLD_VERSION --sticky
```

After the upgrade has finished, unmute the health warning, by using the **ceph health unmute** command.

```
ceph health unmute DAEMON_OLD_VERSION
```

About this task

Use either the command-line interface (CLI) or the Ceph Dashboard to upgrade a cluster. To update the cluster through the dashboard, see Upgrading a cluster. To upgrade through the CLI, continue with the instructions provided here.
For more information, see Troubleshooting upgrade error messages.

If a cluster host is or becomes unavailable the upgrade will be paused until it is restored.

To run a staggered upgrade, see “Running a staggered upgrade” on page 272.

Procedure

1. Register the system, and when prompted, enter your Red Hat Customer Portal credentials.
   For example,

   ```
   [root@admin ~]# subscription-manager register
   ```

2. Pull the latest subscription.

```
subscription-manager refresh
```

> **Note:** Red Hat customers now use Simple Content Access (SCA), as a result, attaching a subscription to the system is no longer necessary. Registering the system and enabling the required repositories is sufficient. For more information, see How to register and subscribe a RHEL system to the Red Hat Customer Portal using Red Hat Subscription-Manager? on the Red Hat Customer Portal.

3. Enable the Ceph Ansible repositories on the Ansible administration node.

```
subscription-manager repos --enable=rhceph-9-tools-for-rhel-9-x86_64-rpms
```

4. Update the system.

```
dnf update
```

5. Update the `cephadm`, `cephadm-ansible`, and `crun` packages on the admin node where `cephadm-ansible` is installed.

```
dnf update cephadm
```

```
dnf update cephadm-ansible
```

```
dnf update crun
```

For example,

```
[root@admin ~]# dnf update cephadm
[root@admin ~]# dnf update cephadm-ansible
[root@admin ~]# dnf update crun
```

6. Navigate to the `/usr/share/cephadm-ansible/` directory.

```
cd /usr/share/cephadm-ansible
```

7. Run the preflight playbook with the **upgrade_ceph_packages** parameter set to `true` on the bootstrapped host in the storage cluster.
This package upgrades `cephadm` on all the nodes.

```
ansible-playbook -i INVENTORY_FILE cephadm-preflight.yml --extra-vars
"ceph_origin=rhcs upgrade_ceph_packages=true"
```

For example,

```
[ansible@admin cephadm-ansible]$ ansible-playbook -i hosts cephadm-preflight.yml --
extra-vars "ceph_origin=rhcs upgrade_ceph_packages=true"
```

8. Log in to the `cephadm` shell.

```
cephadm shell
```

For example,

```
[root@host01 ~]# cephadm shell
[ceph: root@host01 /]#
```

9. Ensure all the hosts are online and that the storage cluster is healthy, by using the **ceph -s** command.
For example,

```
[ceph: root@host01 /]# ceph -s
```

10. Set the OSD `noout`, `noscrub`, and `nodeep-scrub` flags.
Setting these flags prevents OSDs from getting marked out during upgrade and to avoid unnecessary load on the cluster.

```
ceph osd set noout
```

```
ceph osd set noscrub
```

```
ceph osd set nodeep-scrub
```

For example,

```
[ceph: root@host01 /]# ceph osd set noout
[ceph: root@host01 /]# ceph osd set noscrub
[ceph: root@host01 /]# ceph osd set nodeep-scrub
```

11. Check service versions and the available target containers.

> **Note:** The following procedure provides syntax and examples for installing with Red Hat Enterprise Linux 9.6 or 9.7. For installing with Red Hat Enterprise Linux 10 or 10.1, replace `rhel-9` with `rhel-10`, in each instance.

```
ceph orch upgrade check IMAGE_NAME
```

For example,

```
[ceph: root@host01 /]# ceph orch upgrade check registry.redhat.io/rhceph/
rhceph-9-rhel9:latest
```

12. Upgrade the storage cluster.

> **Note:** The following procedure provides syntax and examples for installing with Red Hat Enterprise Linux 9.6 or 9.7. For installing with Red Hat Enterprise Linux 10 or 10.1, replace `rhel-9` with `rhel-10`, in each instance.

```
ceph orch upgrade start --image IMAGE_NAME
```

For example,

```
ceph orch upgrade start registry.redhat.io/rhceph/rhceph-9-rhel9:latest
```

While the upgrade is underway, a progress bar appears in the **`ceph status`** command output. For example,

```
[ceph: root@host01 /]# ceph status
[...]
progress:
    Upgrade to 20.1.0-145.el9cp (1s)
      [............................]
```

For more information about monitoring the upgrade status, see “Monitoring and managing upgrade” on page 275.

13. Verify the new *`IMAGE_ID`* and *`VERSION`* of the Ceph cluster.

```
ceph versions
```

```
ceph orch ps
```

Note: If you are not using the `cephadm-ansible` playbooks and are not running Ceph commands from `cephadm` shell, after upgrading your Ceph cluster, upgrade the Ceph client packages and libraries on each client nodes, and then verify that the latest version is installed.
For example,

```
[root@client01 ~] dnf update ceph-common librados2 librbd1 python3-rados
python3-rbd
[root@client01 ~] ceph --version
```

Important: If you are upgrading from Red Hat Ceph Storage versions earlier than 6.1, the `ceph-exporter` service is not deployed automatically during the upgrade. The `ceph-exporter` service is required for the Dashboard and Prometheus to display Ceph cluster metrics. If the service is not deployed, the Dashboard shows `No Data`.

14. If required, deploy the `ceph-exporter` service.
For example,

```
[ceph: root@host01 /]# ceph orch apply ceph-exporter --placement='*'
```

    a. Verify that the `ceph-exporter` is running.

```
ceph orch ls
```

15. When the upgrade is complete, unset the `noout`, `noscrub`, and `nodeep-scrub` flags.
For example,

```
[ceph: root@host01 /]# ceph osd unset noout
[ceph: root@host01 /]# ceph osd unset noscrub
[ceph: root@host01 /]# ceph osd unset nodeep-scrub
```

Upgrading cluster in a disconnected (air-gapped) environment

Upgrade the Red Hat Ceph Storage cluster in a disconnected environment.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root-level access to all the nodes.
- At least two Ceph Manager nodes in the storage cluster: one active and one standby.
- Register the nodes to CDN and attach subscriptions.
- Check for the customer container images in a disconnected environment and change the configuration, if required. For more information, see Changing configurations of custom container images for disconnected installations.

About this task

Upgrade the storage cluster in a disconnected environment by using the **`--image`** tag.
By default, the monitoring stack components are deployed based on the primary Ceph image. For disconnected environment of the storage cluster, you have to use the latest available monitoring stack component images.

"Custom image details for the monitoring stack" on page 264 details the custom image details for each monitoring stack component.

Table 43: Custom image details for the monitoring stack	
Monitoring stack component	**Image details**

Ceph image	`registry.redhat.io/rhceph/rhceph-9-rhel9:latest`
	or
	`registry.redhat.io/rhceph/rhceph-9-rhel10:latest`

Ceph image (image mode)	`registry.redhat.io/rhceph/alloy-rhel10:latest`
Prometheus	`registry.redhat.io/openshift4/ose-prometheus:v4.15`
Grafana	`registry.redhat.io/rhceph/grafana-rhel10:latest`
Node-exporter	`registry.redhat.io/openshift4/ose-prometheus-nodeexporter:v4.15`
AlertManager	`registry.redhat.io/openshift4/ose-prometheusalertmanager:v4.15`
Keepalived	`registry.redhat.io/rhceph/keepalived-rhel10:latest`
SNMP Gateway	`registry.redhat.io/rhceph/snmp-notifier-rhel10:latest`
OAuth2 Proxy	`registry.redhat.io/rhceph/oauth2-proxy-rhel9:v7.14.0`

Use the **`ceph orch upgrade`** command to start and check an upgrade to a specific tagged version.

Procedure

1. Update the `cephadm` and `cephadm-ansible` package.

   ```
   dnf update cephadm
   dnf update cephadm-ansible
   ```

   For example,

   ```
   [root@admin cephadm-ansible]# dnf update cephadm
    [root@admin cephadm-ansible]# dnf update cephadm-ansible
   ```

2. Navigate to the `/usr/share/cephadm-ansible/` directory.

   ```
   cd /usr/share/cephadm-ansible
   ```

3. Run the preflight playbook with the **upgrade_ceph_packages** parameter set to `true` and the **ceph_origin** parameter set to `custom` on the bootstrapped host in the storage cluster.

```
ansible-playbook -i INVENTORY_FILE cephadm-preflight.yml --extra-vars
"ceph_origin=custom upgrade_ceph_packages=true"
```

For example,

```
[ansible@admin cephadm-ansible]$ ansible-playbook -i /etc/ansible/hosts cephadm-
preflight.yml --extra-vars "ceph_origin=custom upgrade_ceph_packages=true"
```

This package upgrades `cephadm` on all the nodes.

4. Log into the `cephadm` shell.

```
cephadm shell
```

5. Ensure all the hosts are online and that the storage cluster is healthy.

```
ceph -s
```

6. Check service versions and the available target containers.

```
ceph orch upgrade check IMAGE_NAME
```

For example,

```
[ceph: root@host01 /]# ceph orch upgrade check LOCAL_NODE_FQDN:5000/rhceph/
rhceph-9-rhel9
```

7. Set the OSD `noout`, `noscrub`, and `nodeep-scrub` flags to prevent OSDs from getting marked out during upgrade and to avoid unnecessary load on the cluster.

```
ceph osd set noout
ceph osd set noscrub
ceph osd set nodeep-scrub
```

```
[ceph: root@host01 /]# ceph osd set noout
 [ceph: root@host01 /]# ceph osd set noscrub
 [ceph: root@host01 /]# ceph osd set nodeep-scrub
```

8. Upgrade the storage cluster.

> **Note:** The following procedure provides syntax and examples for installing with Red Hat Enterprise Linux 9.6 or 9.7. For installing with Red Hat Enterprise Linux 10 or 10.1, replace `rhel-9` with `rhel-10`, in each instance.

```
ceph orch upgrade start --image IMAGE_NAME
```

For example,

```
ceph orch upgrade start registry.redhat.io/rhceph/rhceph-9-rhel9:latest
```

While the upgrade is underway, a progress bar appears in the **ceph status** command output. For example,

```
[ceph: root@host01 /]# ceph status
[...]
progress:
    Upgrade to 20.1.0-145.el9cp (1s)
      [............................]
```

For more information about monitoring the upgrade status, see “Monitoring and managing upgrade” on page 275.

9. Verify the new *IMAGE_ID* and *VERSION* of the cluster.

```
ceph version
ceph versions
ceph orch ps
```

For example,

```
[ceph: root@host01 /]# ceph version
[ceph: root@host01 /]# ceph versions
[ceph: root@host01 /]# ceph orch ps
```

10. When the upgrade is complete, unset the `noout`, `noscrub`, and`nodeep-scrub` flags.

```
ceph osd unset noout
ceph osd unset noscrub
ceph osd unset nodeep-scrub
```

```
[ceph: root@host01 /]# ceph osd unset noout
[ceph: root@host01 /]# ceph osd unset noscrub
[ceph: root@host01 /]# ceph osd unset nodeep-scrub
```

Related information

Registering Red Hat Ceph Storage nodes to the CDN and attaching subscriptions
Configuring a private registry for a disconnected installation

Updating Ceph Object Gateway to use `cephadm` self-signed certificates in a multi-site deployment

Starting from Red Hat Ceph Storage 8.1, Red Hat Ceph Storage introduced a new root certificate authority (CA) format. As a result, when updating the Ceph Object Gateway in multi-site deployments from Red Hat Ceph Storage versions 8.0 or earlier, you must complete additional steps to enable `cephadm` self-signed certificates.

Before you begin

Before you begin, make sure that your Red Hat Ceph Storage 9 storage system is using the earlier old-style root CA.

```
ceph orch certmgr cert ls
```

- If the root CA `commonName` has the `cephadm-root` format, it is in the old style. Continue with the procedure documented here.
- If the root CA `commonName` has the `cephadm-root-FSID` format, your cluster has already been upgraded to the new format and no further actions are required.

About this task

Use this procedure in any of the following scenarios:

- You are upgrading from Red Hat Ceph Storage versions 8.0 or earlier and are planning on using `cephadm` self-signed certificates.
- `cephadm` self-signed certificates are expired.

If you do not plan on using `cephadm` self-signed certificates or the root CA `commonName` has the `cephadm-root-FSID` format and the certificates are valid, this procedure is not required.

Procedure

1. Remove the previous root CA.

```
ceph orch certmgr rm cephadm_root_ca_cert
```

2. Reload the `certmgr`, to force creating the new root CA.

```
ceph orch certmgr reload
```

3. Verify that the `certmgr` has been loaded correctly.

```
ceph orch certmgr cert ls
```

Ensure that the `cephadm_root_ca_cert` subject has the following format: `cephadm-root-`*`FSID`*
The root CA is upgraded.

4. Reissue new certificates for all services that use `cephadm` self-signed certificates.
5. Refresh any existing Grafana services.
   a. Remove all Grafana certificates.
   This step must be done for each Grafana service that uses self-signed certificates.

```
ceph orch certmgr cert rm grafana_cert --hostname=HOSTNAME
```

   Only continue once all certificates are removed.

   b. Redeploy Grafana.

```
ceph orch redeploy grafana
```

   c. Verify that all Grafana services are running correctly.

```
ceph orch ps --service-name grafana
```

   d. Verify that new `cephadm`-signed certificates have been created for Grafana.

```
ceph orch certmgr cert ls
```

6. Refresh any existing Ceph Object Gateway (RGW) services.
   This step must be done for each Ceph Object Gateway service that uses self-signed certificates.

```
ceph orch certmgr cert rm rgw_frontend_ssl_cert --service-name RGW-SERVICE-NAME
ceph orch redeploy RGW-SERVICE-NAME
```

7. Verify that the Ceph Object Gateway (RGW) service is deployed and running as expected.

```
ceph orch ps --service-name RGW_SERVICE_NAME
```

8. Verify the certificates.

```
ceph orch certmgr cert ls
```

9. Upgrade the Ceph Dashboard certificates.
   a. Generate the `cephadm`-signed certificates for the dashboard.
   From the `cephadm` command-line interface (CLI), run the following commands:

```
json="$(ceph orch certmgr generate-certificates dashboard)"
printf '%s' "$json" | jq -r '.cert' > dashboard.crt
printf '%s' "$json" | jq -r '.key'  > dashboard.key
```

   b. Set the new generated certificate and key.
   From the Ceph command-line interface (CLI), run the following commands:

```
ceph dashboard set-ssl-certificate -i dashboard.crt
ceph dashboard set-ssl-certificate-key -i dashboard.key
```

The `dashboard.crt` and `dashboard.key` pair files are generated.

What to do next

Enable and then disable the Ceph Dashboard to load the new certificates.

```
ceph mgr module disable dashboard
ceph mgr module enable dashboard
```

Upgrading a host operating system from RHEL 9 to RHEL 10

You can perform an Red Hat Ceph Storage host operating system upgrade from Red Hat Enterprise Linux (RHEL) 9 to Red Hat Enterprise Linux 10 using the Leapp utility.

Before you begin

Be sure that you have a running Red Hat Ceph Storage cluster that is supported for upgrade. For more information, see “Supported upgrade paths” on page 259.

About this task

Important: This host operating system upgrade *must* be performed before upgrading the Red Hat Ceph Storage cluster.

The following are the supported combinations of containerized Ceph daemons. For more information, see “Colocation” on page 93.

- Ceph Metadata Server (`ceph-mds`), Ceph OSD (`ceph-osd`), and Ceph Object Gateway (`radosgw`)
- Ceph Monitor (`ceph-mon`) *or* Ceph Manager (`ceph-mgr`), Ceph OSD (`ceph-osd`), and Ceph Object Gateway (`radosgw`)
- Ceph Monitor (`ceph-mon`), Ceph Manager (`ceph-mgr`), Ceph OSD (`ceph-osd`), and Ceph Object Gateway (`radosgw`)

Procedure

1. Deploy your existing Red Hat Ceph Storage on the latest Red Hat Enterprise Linux 9 supported version with service.

   Note: Verify that the cluster contains two admin nodes, so that while performing host upgrade in one admin node (with `_admin` label), the second admin node can be used for managing clusters.

   For full instructions, see Initial installation and “Deploying the Ceph daemons using the service specification” on page 549.

2. Set the `noout` flag on the Ceph OSD.
   For example,

   ```
   [ceph: root@host01 /]# ceph osd set noout
   ```

3. Run the host upgrade one node at a time using the Leapp utility.
   a. Put the respective node in maintenance mode before performing host upgrade using Leapp.

      ```
      ceph orch host maintenance enter HOST
      ```

      For example,

      ```
      ceph orch host maintenance enter host01
      ```

   b. Enable the Ceph tools repo manually when executing the Leapp command with the `--enablerepo` parameter.
      For example,

      ```
      leapp upgrade --enablerepo rhceph-9-tools-for-rhel-10-x86_64-rpms
      ```

   c. Upgrade Red Hat Enterprise Linux from version 9 to 10.

For full instructions, follow the instructions provided within the Upgrading from RHEL 9 to RHEL 10 guide within the Red Hat Enterprise Linux product documentation on the Red Hat Customer Portal.

> **Important:** After performing an in-place upgrade from Red Hat Enterprise Linux 9 to Red Hat Enterprise Linux 10, manually enable and start the `logrotate.timer` service.
>
> ```
> systemctl start logrotate.timer
> systemctl enable logrotate.timer
> ```

4. Verify the new *IMAGE_ID* and *VERSION* of the Ceph cluster.

```
[ceph: root@node0 /]# ceph version
[ceph: root@node0 /]# ceph orch ps
```

What to do next

Continue with the existing Red Hat Ceph Storage version to Red Hat Ceph Storage 9 upgrade, by following "Upgrading a Red Hat Ceph Storage cluster using cephadm" on page 259.

Upgrading to 9 with RHEL 10 with stretch mode enabled from previous versions

You can perform an upgrade from Red Hat Ceph Storage 7.1.0 or 8.1.0 to Red Hat Ceph Storage 9 involving Red Hat Enterprise Linux 9 to Red Hat Enterprise Linux 10 with the stretch mode enabled.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Red Hat Ceph Storage on Red Hat Enterprise Linux 9 with necessary hosts and daemons that run with stretch mode enabled.
- Backup of Ceph binary (`/usr/sbin/cephadm`), `ceph.pub` (`/etc/ceph`), and the Ceph cluster's public SSH keys from the admin node.

About this task

> **Important:** Upgrade to the latest version of Red Hat Ceph Storage 7.1.0 or 8.1.0 before upgrading to the latest version of Red Hat Ceph Storage 9.

Procedure

1. Log in to the `cephadm` shell.

```
cephadm shell
```

2. Label a second node as the admin in the cluster to manage the cluster when the admin node is reprovisioned.

```
ceph orch host label add HOSTNAME_admin
```

For example,

```
[ceph: root@host01 /]# ceph orch host label add host02_admin
```

3. Set the `noout` flag.

```
ceph osd set noout
```

4. Drain all daemons from the host.

```
ceph orch host drain HOSTNAME --force
```

The `_no_schedule` label is automatically applied to the host that blocks deployment.

5. Verify that all daemons are removed from the storage cluster.

```
ceph orch ps HOSTNAME
```

6. Zap the devices.
Zapping the devices helps ensure that the hosts being drained have OSDs present. After the hosts are drained, they can be used to redeploy OSDs when the host is added back.

```
ceph orch device zap HOSTNAME DISK --force
```

For example,

```
[ceph: root@host01 /]# ceph orch device zap ceph-host02 /dev/vdb --force

zap successful for /dev/vdb on ceph-host02
```

7. Verify the status of the OSD removal.

```
ceph orch host rm HOSTNAME --force
```

For example,

```
[ceph: root@host01 /]# ceph orch host rm host02 --force
```

8. Reprovision the respective hosts from RHEL 9 to RHEL 10.
Use the instructions, as detailed in *Upgrading from RHEL 9 to RHEL 10*, on Red Hat Documentation.

9. Go to the `/usr/share/cephadm-ansible/` directory.

```
cd /usr/share/cephadm-ansible
```

10. Run the preflight playbook with the `--limit` option.

```
ansible-playbook -i INVENTORY_FILE cephadm-preflight.yml --limit NEW_HOSTNAME
```

For example,

```
[ceph: root@host01 cephadm ansible]# ansible-playbook -i hosts cephadm-preflight.yml
--extra-vars "ceph_origin={storage-product}" --limit host02
```

The preflight playbook installs `podman`, `lvm2`, `chrony`, and `cephadm` on the new host. After installation is complete, `cephadm` is found in the `/usr/sbin/` directory.

11. Extract the cluster's public SSH keys to a folder.

```
ceph cephadm get-pub-key ~/PATH
```

For example,

```
[ceph: root@host01 /]# ceph cephadm get-pub-key ~/ceph.pub
```

12. Copy the Ceph cluster public SSH keys to the reprovisioned node.

```
ssh-copy-id -f -i ~/PATH root@HOSTNAME_2
```

For example,

```
[ceph: root@host01 /]# ssh-copy-id -f -i ~/ceph.pub root@host02
```

13. Optional: If the removed host has a monitor daemon, before adding the host to the cluster, add the `--unmanaged` flag to monitor deployment.

```
ceph orch apply mon PLACEMENT --unmanaged
```

14. Re-add the host to the cluster and re-add the labels that were present earlier.

```
ceph orch host add HOSTNAME IP_ADDRESS --labels=LABELS
```

The following examples give different ways to re-add the host to the cluster.

- `ceph mon add HOSTNAME IP_LOCATION`

  For example,

  ```
  [ceph: root@host01 /]# ceph mon add ceph-host02 10.0.211.62 datacenter=DC2
  ```

- `ceph orch daemon add mon HOSTNAME`

  For example,

  ```
  [ceph: root@host01 /]# ceph orch daemon add mon ceph-host02
  ```

15. Verify the daemons on the reprovisioned host running successfully with the same Ceph version.

```
ceph orch ps
```

16. Set the monitor daemon placement to `managed`.

```
ceph orch apply mon managed
```

17. Repeat all previous steps for each host.
18. Reprovision the arbiter monitor.
    The arbiter monitor cannot be drained or removed from the host. As a result, the arbiter monitor must be reprovisioned to another tie-breaker node, and then drained or removed from host. For more information, see Replacing a tiebreaker with a new monitor.
19. Reprovision the administrator nodes.
    Use a second administrator node to manage clusters. For more information, see Replacing a tiebreaker with a new monitor.
20. Re-add the backup files to the node.
21. Re-add the administrator noes to the cluster, by using the second administrator node.
    Set the `mon` deployment to `unmanaged`.

```
ceph orch apply mon unmanaged
```

22. Re-add the original arbiter monitor and remove the temporary monitor that was created in step “18” on page 271.
    For more information, see Replacing a tiebreaker with a new monitor.
23. Unset the `noout` flag.

```
ceph osd unset noout
```

What to do next

1. Verify the Ceph version and the cluster status to help ensure that all demons are working as expected after the OS upgrade.
2. Follow the “Upgrading a Red Hat Ceph Storage cluster using cephadm” on page 259 to upgrade from Red Hat Ceph Storage 7.1.0 or 8.1.0 to 9.

Running a staggered upgrade

As a storage administrator, you can upgrade Red Hat Ceph Storage components in phases rather than all at once. You can use the **ceph orch upgrade** options to limit which daemons are upgraded with a single upgrade command.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- The latest version of a supported Red Hat Ceph Storage cluster. For a full list of supported upgrade paths, see "Supported upgrade paths" on page 259.
- Root-level access to all the nodes.
- At least two Ceph Manager nodes in the storage cluster: one active and one standby.
- Topological labels set on a host. Use topological labels to upgrade specific hosts. For more information, see Adding topological labels to a host.

About this task

> **Note:** If you want to upgrade from a version that does not support staggered upgrades, you must first manually upgrade the Ceph Manager (`ceph-mgr`) daemons.

The **ceph orch upgrade** command supports several options to upgrade cluster components in phases. "Staggered upgrade options" on page 272 lists the staggered upgrade options.

Table 44: Staggered upgrade options

Command option	Description
--daemon_types	The **--daemon_types** option takes a comma-separated list of daemon types and only upgrades daemons of those types. Valid daemon types for this option include `mgr`, `mon`, `crash`, `osd`, `mds`, `rgw`, `rbd-mirror`, and `cephfs-mirror`.
--services	The **--services** option is mutually exclusive with **--daemon_types**. **--services** takes only services of one type at a time, and upgrades only daemons belonging to those services. For example, you cannot provide an OSD and RGW service simultaneously.
--hosts	You can combine the **--hosts** option with**--daemon_types**,**--services**, or use it on its own. The **--hosts** option parameter follows the same format as the command line options for orchestrator CLI placement specification.
--limit	The **--limit** option takes an integer greater than zero and provides a numerical limit on the number of daemons `cephadm` upgrades. You can combine the **--limit** option with **--daemon_types**, **--services**, or **--hosts**. For example, if you specify to upgrade daemons of type `osd` on `host01` with a limit set to `3`, `cephadm` upgrades up to three OSD daemons on host01.
--topological-labels	The **--topological-labels** option upgrades a specific set of hosts that have the specified topological label. For example, if the label `datacenter=A` is used on a set of hosts, you can target these specific hosts for upgrade, without needing to manually input each host.

cephadm strictly enforces an order for the upgrade of daemons that is still present in staggered upgrade scenarios. The current upgrade order is as follows:

1. Ceph Manager (mgr) nodes
2. Ceph Monitor (mon) nodes
3. Ceph crash daemons
4. Ceph osd nodes
5. Ceph Metadata Server (mds) nodes
6. Ceph Object Gateway (rgw) nodes
7. Ceph Block Device mirror (rbd-mirror) node
8. Ceph File System mirror (cephfs-mirror) node

Note:

- If you specify parameters that upgrade daemons out of order, the upgrade command blocks and notes which daemons you need to upgrade before you proceed.
- Restarting instances does not require a specific order. It is recommended to restart the instance pointing to the pool with primary images followed by the instance pointing to the mirrored pool.

The following example shows a staggered upgrade that was started out of order:

```
[ceph: root@host01 /]# ceph orch upgrade start --image registry.redhat.io/rhceph-9/
rhceph-9-rhel9:latest --hosts host02

Error EINVAL: Cannot start upgrade. Daemons with types earlier in upgrade order than
daemons on given host need upgrading.
Please first upgrade mon.ceph-host01
NOTE: Enforced upgrade order is: mgr -> mon -> crash -> osd -> mds -> rgw -> rbd-
mirror -> cephfs-mirror
```

Procedure

1. Log in to the cephadm shell and check that all hosts are online and that the storage cluster is in a healthy state.
   For more information about starting the cephadm shell, see “Using the cephadm shell” on page 217.
   For example,

   ```
   [root@host01 ~]# cephadm shell
   [ceph: root@host01 /]# ceph -s
   ```

2. Set the OSD noout, noscrub, and nodeep-scrub flags to prevent OSDs from getting marked out during upgrade and to avoid unnecessary load on the cluster.

   ```
   ceph osd set noout
   ```

   ```
   ceph osd set noscrub
   ```

   ```
   ceph osd set nodeep-scrub
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph osd set noout
    [ceph: root@host01 /]# ceph osd set noscrub
    [ceph: root@host01 /]# ceph osd set nodeep-scrub
   ```

3. Check service versions and the available target containers.

   ```
   ceph orch upgrade check IMAGE_NAME
   ```

For example,

```
[ceph: root@host01 /]# ceph orch upgrade check registry.redhat.io/rhceph-9/
rhceph-9-rhel9:latest
```

4. Upgrade the storage cluster.
Upgrade clusters in one of the following ways:
   - Upgrade specific daemon types on specific hosts.

     ```
     ceph orch upgrade start --image IMAGE_NAME --daemon-types
     DAEMON_TYPE1,DAEMON_TYPE2 --hosts HOST1,HOST2
     ```

     For example,

     ```
     [ceph: root@host01 /]# ceph orch upgrade start --image registry.redhat.io/
     rhceph-9/rhceph-9-rhel9:latest --daemon-types mgr,mon --hosts host02,host03
     ```

   - Specify specific services and limit the number of daemons to upgrade.

     Note:
     - In staggered upgrade scenarios, if using a limiting parameter, the monitoring stack daemons, including Prometheus and `node-exporter`, are refreshed after the upgrade of the Ceph Manager daemons. As a result of the limiting parameter, Ceph Manager upgrades take longer to complete. The versions of monitoring stack daemons might not change between Ceph releases, in which case, they are only redeployed.
     - Upgrade commands with limiting parameters validates the options before beginning the upgrade, which can require pulling the new container image. As a result, the **`upgrade start`** command might take a while to return when you provide limiting parameters.

     ```
     ceph orch upgrade start --image IMAGE_NAME --services SERVICE1,SERVICE2 --limit
     LIMIT_NUMBER
     ```

     For example,

     ```
     [ceph: root@host01 /]# ceph orch upgrade start --image registry.redhat.io/
     rhceph-9/rhceph-9-rhel9:latest --services rgw.example1,rgw1.example2 --limit 2
     ```

   - Specify specific topological labels to run the upgrade on specific hosts.

     Note: Using topological labels does not allow changing from the required daemon upgrade order.

     ```
     ceph orch upgrade start --topological-labels TOPOLOGICAL_LABEL
     ```

     For example,

     ```
     [ceph: root@host01 /]# ceph orch upgrade start --topological-labels datacenter=A
     ```

   - Specify specific topological labels to run the upgrade on specific hosts, with the **`--daemon-types`** option.
     Use the **`--daemon-types`** option for hosts with a similar scenario to the following example:

     A host exists with both Monitor and OSD daemons that match `datacenter=A` but nonupgraded Monitor daemons on host that do not match `datacenter=A`. In this case, you only want to upgrade the Monitor daemons on the `datacenter=A` host, to not break the upgrade order. In this scenario, use the following command to upgrade only the Monitor daemons on `datacenter=A`, but not the OSD daemons.

```
ceph orch upgrade start IMAGE_NAME --daemon-types DAEMON_TYPE1 --topological-
labels TOPOLOGICAL_LABEL
```

For example,

```
[ceph: root@host01 /]# ceph orch upgrade start registry.redhat.io/rhceph-9/
rhceph-9-rhel9:latest --daemon-types mon --topological-labels datacenter=A
```

Use the **ceph orch upgrade status** command to verify what was selected for upgrade. View the `"which"` field in the output. For example,

```
[ceph: root@host01 /]# ceph orch upgrade status
{
    "in_progress": true,
    "target_image": "registry.redhat.io/rhceph-9/rhceph-9-rhel9:latest",
    "services_complete": [
        "mon"
    ],
    "which": "Upgrading daemons of type(s) mon on host(s) host01",
    "progress": "1/1 daemons upgraded",
    "message": "Currently upgrading mon daemons",
    "is_paused": false
}
```

5. Check which daemons still need to upgrade by running either the **ceph orch upgrade check** or **ceph versions** command.

```
ceph orch upgrade check --image registry.redhat.io/rhceph-9/rhceph-9-rhel9:latest
```

6. Verify the upgrade of all remaining services.

```
ceph orch upgrade start --image IMAGE_NAME
```

For example,

```
[ceph: root@host01 /]# ceph orch upgrade start --image registry.redhat.io/rhceph-9/
rhceph-9-rhel9:latest
```

7. Verify the new *IMAGE_ID* and *VERSION* of the Ceph cluster.

```
ceph versions
ceph orch ps
```

What to do next

After the upgrade is complete, unset the upgrade is complete, unset the `noout`, `noscrub`, and `nodeep-scrub` flags.

```
ceph osd unset noout
ceph osd unset noscrub
ceph osd unset nodeep-scrub
```

Monitoring and managing upgrade

You can monitor and manage the upgrade of the storage cluster during the upgrade process.

After running the **ceph orch upgrade start** command to upgrade the Red Hat Ceph Storage cluster, you can check the status, pause, resume, or stop the upgrade process. The health of the cluster changes to `HEALTH_WARNING` during an upgrade.

> **Note:** Upgrade one daemon type after the other.

The upgrade is paused in the following situations:

- The host of the cluster is offline.

– A daemon cannot be upgraded.

> **Note:** If a cluster is or becomes unavailable the upgrade will be paused until it is restored.

Before you begin, make sure that you have the following prerequisites in place:

– A running Red Hat Ceph Storage supported existing.

– Root-level access to all the nodes.

– At least two Ceph Manager nodes in the storage cluster: one active and one standby.

– Upgrade for the storage cluster initiated.

Use the **ceph orch upgrade** command to take you through the upgrade process. “Upgrade management command options” on page 276 provides the different options that can be used to help manage and monitor during the upgrade process.

Table 45: Upgrade management command options

Command option	Syntax	Notes
`status`	`ceph orch upgrade status`	– Use the `status` option to determine whether an upgrade is in progress and the version to which the cluster is upgrading. – You do not get a message when the upgrade is successful. Run **ceph versions** and **ceph orch ps** commands to verify the new image ID and the version of the storage cluster.
`pause`	`ceph orch upgrade pause`	None
`resume`	`ceph orch upgrade resume`	None
`stop`	`ceph orch upgrade stop`	None

Configuring Red Hat Ceph Storage

Use this information to help configure Red Hat Ceph Storage at start and run time.

As a storage administrator, you need to have a basic understanding of how to view the Ceph configuration, and how to set the Ceph configuration options for the Red Hat Ceph Storage cluster. You can view and set the Ceph configuration options at runtime.

As part of the Ceph authentication configuration, consider key rotation for your Ceph and gateway daemons for increased security. Key rotation can be done either through the command-line or the Ceph dashboard. For more information see Creating user capabilities and "Enabling key rotation" on page 59.

Be sure that the Red Hat Ceph Storage software is installed before configuring.

Ceph configuration

Red Hat Ceph Storage clusters typically have a configuration file that is created and defined through a deployment tool. Configuration files can also be created manually.

Deployment tools, such as `cephadm`, typically create an initial Ceph configuration file for you. However, you can manually create one, if you prefer to bootstrap a Red Hat Ceph Storage cluster without using a deployment tool.

Red Hat Ceph Storage clusters have a configuration, which defines:

- Cluster Identity
- Authentication settings
- Ceph daemons
- Network configuration
- Node names and addresses
- Paths to keyrings
- Paths to OSD log files
- Other runtime options

For more information about `cephadm` and the Ceph orchestrator, see Operations.

Configuration database

The Ceph Monitor manages a configuration database of Ceph options that centralize configuration management by storing configuration options for the entire storage cluster. By centralizing the Ceph configuration in a database, this simplifies storage cluster administration.

The priority order that Ceph uses to set options is:

1. Compiled-in default values
2. Ceph cluster configuration database
3. Local `ceph.conf` file
4. Runtime override, using the **`ceph daemon DAEMON-NAME config set`** or **`ceph tell DAEMON-NAME injectargs`** commands

There are still a few Ceph options that can be defined in the local Ceph configuration file, which is `/etc/ceph/ceph.conf` by default.

`cephadm` uses a basic `ceph.conf` file that only contains a minimal set of options for connecting to Ceph Monitors, authenticating, and fetching configuration information. In most cases, `cephadm` uses only the `mon_host` option. To avoid using `ceph.conf` only for the `mon_host` option, use DNS SRV records to perform operations with Monitors.

Important: Use the **`assimilate-conf`** administrative command to move valid options into the configuration database from the `ceph.conf` file. For more information about **`assimilate-conf`**, see "Administrative commands" on page 278.

Ceph allows you to make changes to the configuration of a daemon at runtime. This capability can be useful for increasing or decreasing the logging output, by enabling or disabling debug settings, and can even be used for runtime optimization.

Note: When the same option exists in the configuration database and the Ceph configuration file, the configuration database option has a lower priority than what is set in the Ceph configuration file.

Sections and masks

Just as you can configure Ceph options globally, per daemon type, or by a specific daemon in the Ceph configuration file, you can also configure the Ceph options in the configuration database according to these sections. These sections are described in "Configuration database sections" on page 278.

Table 46: Configuration database sections

Section	Description
`global`	Affects all daemons and clients.
`mon`	Affects all Ceph Monitors.
`mgr`	Affects all Ceph Managers.
`osd`	Affects all Ceph OSDs.
`mds`	Affects all Ceph Metadata Servers.
`client`	Affects all Ceph Clients, including mounted file systems, block devices, and RADOS Gateways.

Ceph configuration options can have a mask associated with them. These masks can further restrict which daemons or clients the options apply to.

Masks have two forms:

`type:location`

The `type` is a CRUSH property, for example, `rack` or `host`. The `location` is a value for the property type. For example, `host:foo` limits the option only to daemons or clients running on the `foo` host.

For example:

```
ceph config set osd/host:magna045 debug_osd 20
```

`class:device-class`

The `device-class` is the name of the CRUSH device class, such as `hdd` or `ssd`. For example, `class:ssd` limits the option only to Ceph OSDs backed by solid state drives (SSD). This mask has no effect on non-OSD daemons of clients.

For example:

```
ceph config set osd/class:hdd osd_max_backfills 8
```

Administrative commands

The Ceph configuration database can be administered with the subcommand **`ceph config ACTION`**. These are the actions you can do:

`ls`

Lists the available configuration options.

dump

Dumps the entire configuration database of options for the storage cluster.

get WHO

Dumps the configuration for a specific daemon or client. For example, *WHO* can be a daemon, like `mds.a`.

set WHO OPTION VALUE

Sets a configuration option in the Ceph configuration database, where *WHO* is the target daemon, *OPTION* is the option to set, and *VALUE* is the desired value.

show WHO

Shows the reported running configuration for a running daemon. These options might be different from those stored by the Ceph Monitors if there is a local configuration file in use or options have been overridden on the command line or at run time. Also, the source of the option values is reported as part of the output.

assimilate-conf -i INPUT_FILE -o OUTPUT_FILE

Assimilate a configuration file from the *INPUT_FILE* and move any valid options into the Ceph Monitors' configuration database. Any options that are unrecognized, invalid, or cannot be controlled by the Ceph Monitor return in an abbreviated configuration file stored in the *OUTPUT_FILE*. This command can be useful for transitioning from legacy configuration files to a centralized configuration database. Note that when you assimilate a configuration and the Monitors or other daemons have different configuration values set for the same set of options, the end result depends on the order in which the files are assimilated.

help OPTION -f json-pretty

Displays help for a particular *OPTION* using a JSON-formatted output.

For more information, see "Setting a specific configuration at runtime" on page 280.

Using the Ceph metavariables

Metavariables simplify Ceph storage cluster configuration dramatically. When a metavariable is set in a configuration value, Ceph expands the metavariable into a concrete value.

Metavariables are powerful when used within the `[global]`, `[osd]`, `[mon]`, or `[client]` sections of the Ceph configuration file. However, you can also use them with the administration socket. Ceph metavariables are similar to Bash shell expansion.

Ceph supports the following metavariables, as listed in "Supported Ceph metavariables" on page 279:

Table 47: Supported Ceph metavariables

Metavariable	Description	Example	Default
`$cluster`	Expands to the Ceph storage cluster name. Useful when running multiple Ceph storage clusters on the same hardware.	`/etc/ceph/$cluster.keyring`	`ceph`
`$type`	Expands to one of `osd` or `mon`, depending on the type of instant daemon.	`/var/lib/ceph/$type`	None.
`$id`	Expands to the daemon identifier. For `osd.0`, this would be `0`.	`/var/lib/ceph/$type/$cluster-$id`	None.
`$host`	Expands to the host name of the instant daemon.	N/A	None.
`$name`	Expands to `$type.$id`.	`/var/run/ceph/$cluster-$name.asok`	None.

Viewing the Ceph configuration at runtime

The Ceph configuration files can be viewed at boot time and run time.

Prerequisites

- Root-level access to the Ceph node.
- Access to admin keyring.

Procedure

1. To view a runtime configuration, log in to a Ceph node running the daemon and run the following command.

   Syntax

   ```
   ceph daemon DAEMON_TYPE.ID config show
   ```

   To see the configuration for `osd.0`, you can log into the node containing `osd.0` and run this command:

   Example

   ```
   [root@osd ~]# ceph daemon osd.0 config show
   ```

2. For additional options, specify a daemon and `help`.

   Example

   ```
   [root@osd ~]# ceph daemon osd.0 help
   ```

Viewing a specific configuration at runtime

Configuration settings for Red Hat Ceph Storage can be viewed at runtime from the Ceph Monitor node.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph Monitor node.

Procedure

1. Log into a Ceph node and and run the following command.

   Syntax

   ```
   ceph daemon DAEMON_TYPE.ID config get PARAMETER
   ```

   Example

   ```
   [root@mon ~]# ceph daemon osd.0 config get public_addr
   ```

Setting a specific configuration at runtime

To set a specific Ceph configuration at runtime, use the **`ceph config set`** command.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph Monitor or OSD nodes.

Procedure

1. Set the configuration on all Monitor or OSD daemons :

   Syntax

```
ceph config set DAEMON CONFIG-OPTION VALUE
```

Example

```
[root@mon ~]# ceph config set osd debug_osd 10
```

2. Validate that the option and value are set:

   Example

   ```
   [root@mon ~]# ceph config dump
   osd      advanced debug_osd  10/10
   ```

   - To remove the configuration option from all daemons:

     Syntax

     ```
     ceph config rm DAEMON CONFIG-OPTION VALUE
     ```

     Example

     ```
     [root@mon ~]# ceph config rm osd debug_osd
     ```

   - To set the configuration for a specific daemon:

     Syntax

     ```
     ceph config set DAEMON.DAEMON-NUMBER CONFIG-OPTION VALUE
     ```

     Example

     ```
     [root@mon ~]# ceph config set osd.0 debug_osd 10
     ```

   - To validate that the configuration is set for the specified daemon:

     Example

     ```
     [root@mon ~]# ceph config dump
     osd.0      advanced debug_osd     10/10
     ```

   - To remove the configuration for a specific daemon:

     Syntax

     ```
     ceph config rm DAEMON.DAEMON-NUMBER CONFIG-OPTION
     ```

     Example

     ```
     [root@mon ~]# ceph config rm osd.0 debug_osd
     ```

Note: If you use a client that does not support reading options from the configuration database, or if you still need to use `ceph.conf` to change your cluster configuration for other reasons, run the following command:

```
ceph config set mgr mgr/cephadm/manage_etc_ceph_ceph_conf false
```

Be sure to maintain and distribute the `ceph.conf` file across the storage cluster.

OSD Memory Target

BlueStore keeps OSD heap memory usage under a designated target size with the `osd_memory_target` configuration option.

The option `osd_memory_target` sets OSD memory based upon the available RAM in the system. Use this option when TCMalloc is configured as the memory allocator, and when the `bluestore_cache_autotune` option in BlueStore is set to `true`.

Ceph OSD memory caching is more important when the block device is slow; for example, traditional hard drives, because the benefit of a cache hit is much higher than it would be with a solid state drive. However, this must be weighed into a decision to collocate OSDs with other services, such as in a hyper-converged infrastructure (HCI) or other applications.

Setting the OSD memory target

Use the `osd_memory_target` option to set the maximum memory threshold for all OSDs in the storage cluster, or for specific OSDs.

An OSD with an `osd_memory_target` option set to 16 GB might use up to 16 GB of memory.

> **Note:** Configuration options for individual OSDs take precedence over the settings for all OSDs.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to all hosts in the storage cluster.

Procedure

1. To set `osd_memory_target` for all OSDs in the storage cluster:

   Syntax

   ```
   ceph config set osd osd_memory_target VALUE
   ```

   VALUE is the number of GBytes of memory to be allocated to each OSD in the storage cluster.

2. To set `osd_memory_target` for a specific OSD in the storage cluster:

   Syntax

   ```
   ceph config set osd.id osd_memory_target VALUE
   ```

   `.id` is the ID of the OSD and *VALUE* is the number of GB of memory to be allocated to the specified OSD. For example, to configure the OSD with ID 8 to use up to 16 GBytes of memory:

   Example

   ```
   [ceph: root@host01 /]# ceph config set osd.8 osd_memory_target 16G
   ```

3. To set an individual OSD to use one maximum amount of memory and configure the rest of the OSDs to use another amount, specify the individual OSD first:

   Example

   ```
   [ceph: root@host01 /]# ceph config set osd osd_memory_target 16G
   [ceph: root@host01 /]# ceph config set osd.8 osd_memory_target 8G
   ```

Reference

- To configure Red Hat Ceph Storage to autotune OSD memory usage, see Automatically tuning OSD memory.

Automatically tuning OSD memory

The OSD daemons adjust the memory consumption based on the `osd_memory_target` configuration option. The option `osd_memory_target` sets OSD memory based upon the available RAM in the system.

If Red Hat Ceph Storage is deployed on dedicated nodes that do not share memory with other services, `cephadm` automatically adjusts the per-OSD consumption based on the total amount of RAM and the number of deployed OSDs.

Important: Operating System swap must not be enabled. Configuring the operating system to use swap to provide additional virtual memory for daemons is not recommended for modern systems and can result in degraded performance.

Important: By default, the **`osd_memory_target_autotune`** parameter is set to **`true`** in the Red Hat Ceph Storage cluster.

```
ceph config set osd osd_memory_target_autotune true
```

`cephadm` starts with a fraction `mgr/cephadm/autotune_memory_target_ratio`, which defaults to `0.7` of the total RAM in the system, subtract off any memory consumed by non-autotuned daemons such as non-OSDS and for OSDs for which `osd_memory_target_autotune` is false, and then divide by the remaining OSDs.

The `osd_memory_target` parameter is calculated as follows:

```
osd_memory_target = TOTAL_RAM_OF_THE_OSD * (1048576) *
(autotune_memory_target_ratio) / NUMBER_OF_OSDS_IN_THE_OSD_NODE -
(SPACE_ALLOCATED_FOR_OTHER_DAEMONS)
```

SPACE_ALLOCATED_FOR_OTHER_DAEMONS can optionally include the following daemon space allocations:

- Alertmanager: 1 GB
- Grafana: 1 GB
- Ceph Manager: 4 GB
- Ceph Monitor: 2 GB
- Node-exporter: 1 GB
- Prometheus: 1 GB

For example, if a node has 24 OSDs and has 251 GB RAM space, then `osd_memory_target` is `7860684936`.

The final targets are reflected in the configuration database with options. You can view the limits and the current memory consumed by each daemon from the `ceph orch ps` output under `MEM LIMIT` column.

Note:
The default setting of `osd_memory_target_autotune` `true` is unsuitable for hyperconverged infrastructures where compute and Ceph storage services are colocated. In a hyperconverged infrastructure, the `autotune_memory_target_ratio` can be set to `0.2` to reduce the memory consumption of Ceph.

```
[ceph: root@host01 /]# ceph config set mgr mgr/cephadm/autotune_memory_target_ratio
0.2
```

You can manually set a specific memory target for an OSD in the storage cluster.

```
[ceph: root@host01 /]# ceph config set osd.123 osd_memory_target 7860684936
```

You can manually set a specific memory target for an OSD host in the storage cluster.

```
ceph config set osd/host:HOSTNAME osd_memory_target TARGET_BYTES
```

```
[ceph: root@host01 /]# ceph config set osd/host:host01 osd_memory_target 1000000000
```

Note:
Enabling `osd_memory_target_autotune` overwrites existing manual OSD memory target settings. To prevent daemon memory from being tuned even when the `osd_memory_target_autotune` option or other similar options are enabled, set the `_no_autotune_memory` label on the host.

```
ceph orch host label add HOSTNAME _no_autotune_memory
```

You can exclude an OSD from memory autotuning by disabling the autotune option and setting a specific memory target.

```
[ceph: root@host01 /]# ceph config set osd.123 osd_memory_target_autotune false
[ceph: root@host01 /]# ceph config set osd.123 osd_memory_target 16G
```

MDS memory cache limit

Metadata Server (MDS) daemons store Ceph File System (CephFS) metadata in memory to reduce access to the metadata pool and improve file-system performance.

The MDS daemons store persistent metadata in a separate metadata pool, such as `cephfs_metadata`, and use Ceph Object Storage Daemons (OSDs) to access the pool.

An MDS daemon supports an entire CephFS, not a single storage device in the storage cluster. Therefore, the MDS memory requirements can be significant. Workloads with small or medium-sized files typically require more metadata cache because the ratio of metadata to data is higher.

Cache-sizing recommendations

The `mds_cache_memory_limit` option specifies the target maximum memory that an MDS daemon uses for its metadata cache.

The required cache size depends on:

- The metadata working set
- The number and size of files
- The directory structure
- The number of concurrent clients
- The rate of metadata operations
- The application workload

An undersized MDS cache can cause frequent cache eviction. Under a metadata-intensive workload, excessive eviction can reduce throughput and substantially increase client request latency.

For production environments with metadata-intensive workloads, set the initial MDS cache size to 128 GiB. In performance testing with a metadata-intensive software-build workload, a 128 GiB cache sustained approximately 10,000–15,000 operations per second on four dedicated CPU cores. With a cache smaller than 64 GiB, throughput decreased substantially and latency increased after the metadata working set exceeded the available cache.

The optimal MDS cache size depends on the workload. After you set the initial value, validate it by using representative application workloads. Adjust the value based on memory use, cache pressure, throughput, and latency.

The 128 GiB value is a starting point, not a universal requirement. MDS cache requirements vary by workload. Validate the cache size by using representative application workloads, and adjust the value based on observed memory use, cache pressure, throughput, and latency.

A larger MDS cache can retain a larger metadata working set and reduce cache eviction. However, increasing the cache:

- Increases the memory required by each MDS daemon
- Requires equivalent memory capacity for standby MDS daemons
- Can increase failover time because the replacement MDS daemon must rebuild a larger metadata cache

Maintain sufficient memory for the operating system and other services on the MDS node. Do not allocate all available system memory to the MDS cache.

> **Note:** For a large Red Hat Ceph Storage cluster with a metadata-intensive workload, do not place an MDS daemon on the same node as other memory-intensive services. Separating these services enables you to allocate more memory to the MDS daemon, including cache sizes greater than 100 GiB.

Setting the MDS memory cache limit

Configure the MDS memory cache limit for all MDS daemons or for a specific MDS daemon.

About this task

For metadata-intensive production workloads, use 128 GiB as a recommended initial MDS cache size. Validate the configured value with representative application workloads and adjust it based on memory usage, cache pressure, throughput, and latency.

Set the cache limit for all MDS daemons

Set **`mds_cache_memory_limit`** for all MDS daemons.

Procedure

1. Set the MDS cache memory limit.
   Syntax:

   ```
   ceph config set mds mds_cache_memory_limit VALUEGi
   ```

   Replace *VALUE* with the cache size in GiB.

   For example:

   ```
   [ceph: root@host01 /]# ceph config set mds mds_cache_memory_limit 128Gi
   ```

2. Verify the configured value.

   ```
   ceph config get mds mds_cache_memory_limit
   ```

   Example output:

   ```
   137438953472
   ```

   The output displays the configured value in bytes.

Set the cache limit for a specific MDS daemon

Set **`mds_cache_memory_limit`** for an individual MDS daemon when different cache limits are required.

Procedure

1. Set the cache limit for the MDS daemon.
   Syntax:

   ```
   ceph config set mds.MDS_ID mds_cache_memory_limit VALUEGi
   ```

Replace:

- *MDS_ID* with the ID of the MDS daemon.
- *VALUE* with the cache size in GiB.

For example:

```
[ceph: root@host01 /]# ceph config set mds.mycephfs.host01.abcdef
mds_cache_memory_limit 128Gi
```

2. View the effective configuration for the running MDS daemon.
Syntax:

```
ceph config show mds.MDS_ID mds_cache_memory_limit
```

Configure a legacy configuration override

If the deployment workflow uses configuration overrides, specify the cache limit in bytes.

Procedure

- Specify the cache limit in bytes in the configuration overrides.
For example, to set **mds_cache_memory_limit** to 2000000000 bytes, use:

```
ceph_conf_overrides:
  mds:
    mds_cache_memory_limit: 2000000000
```

> **Note:** For clusters managed by `cephadm`, use **ceph config set** unless the deployment workflow requires configuration overrides.

Monitoring the cache configuration

Monitor MDS memory usage and client performance after changing the cache limit.

About this task

An insufficient cache can produce the following symptoms:

- Frequent cache eviction
- Decreased throughput as the workload increases
- Increased client request latency
- High MDS CPU usage caused by cache maintenance
- Slow metadata operations, such as directory listing, file creation, file removal, and attribute lookup

Additional CPU cores can improve throughput and latency under load, but additional CPU resources cannot compensate for an insufficient cache. Performance testing showed that increasing the allocation from one CPU core to four CPU cores improved MDS scalability when the cache size was fixed at 64 GiB.

Use equivalent cache and CPU allocations for active and standby MDS daemons. If the metadata workload exceeds the practical capacity of one active MDS daemon, configure multiple active MDS daemons instead of continuing to increase the cache of a single daemon.

For more information, see Metadata Server cache size limits.

General configuration options

Understand the general configuration options for Ceph.

Note: Typically, these configuration options are set automatically by deployment tools, such as `cephadm`.

`fsid`

Description: The file system ID. One per cluster.

Type: >UUID

Required: No.

Default: N/A. Usually generated by deployment tools.

`admin_socket`

Description: The socket for executing administrative commands on a daemon, irrespective of whether Ceph monitors have established a quorum.

Type: >String

Required: No

Default: `/var/run/ceph/$cluster-$name.asok`

`pid_file`

Description: The file in which the monitor or OSD will write its PID. For instance, `/var/run/$cluster/$type.$id.pid` creates `/var/run/ceph/mon.a.pid` for the `mon` with id `a` running in the `ceph` cluster. The `pid file` is removed when the daemon stops gracefully. If the process is not daemonized (meaning it runs with the `-f` or `-d` option), the `pid file` is not created.

Type: >String

Required: No

Default: No

`chdir`

Description: The directory Ceph daemons change to once they are up and running. Default `/` directory recommended.

Type: >String

Required: No

Default: `/`

`max_open_files`

Description: If set, when the Red Hat Ceph Storage cluster starts, Ceph sets the `max_open_fds` at the OS level (that is, the max # of file descriptors). It helps prevent Ceph OSDs from running out of file descriptors.

Type: >64-bit Integer

Required: No

Default: `0`

`fatal_signal_handlers`

Description: If set, we will install signal handlers for SEGV, ABRT, BUS, ILL, FPE, XCPU, XFSZ, SYS signals to generate a useful log message.

Type: >Boolean

Default: `true`

Ceph network configuration

It is important to understand the network environment that the Red Hat Ceph Storage cluster will operate in, in order to configure the Red Hat Ceph Storage.

Understanding and configuring the Ceph network options ensures optimal performance and reliability of the overall storage cluster.

Be sure to have active network connectivity and a Red Hat Ceph Storage cluster installed before configuring your Ceph network.

For more information, see “Network configuration options” on page 297 and Ceph on-wire encryption.

Network configuration for Ceph

The Ceph storage cluster does not perform request routing or dispatching on behalf of the Ceph client. Instead, Ceph clients make requests directly to Ceph OSD daemons. Ceph OSDs perform data replication on behalf of Ceph clients, which means replication and other factors impose additional loads on the networks of Ceph storage clusters.

Ceph has one network configuration requirement that applies to all daemons. The Ceph configuration file must specify the `host` for each daemon.

Some deployment utilities, such as `cephadm` creates a configuration file for you. Do not set these values if the deployment utility does it for you.

Important:

- The `host` option is the short name of the node, not its FQDN. It is not an IP address.
- All Ceph clusters must use a public network. However, unless you specify an internal cluster network, Ceph assumes a single public network. Ceph can function with a public network only, but for large storage clusters, you will see significant performance improvement with a second private network for carrying only cluster-related traffic.
- It is recommended to run a Ceph storage cluster with two networks. One public network and one private network.

To support two networks, each Ceph Node needs to have more than one network interface card (NIC).

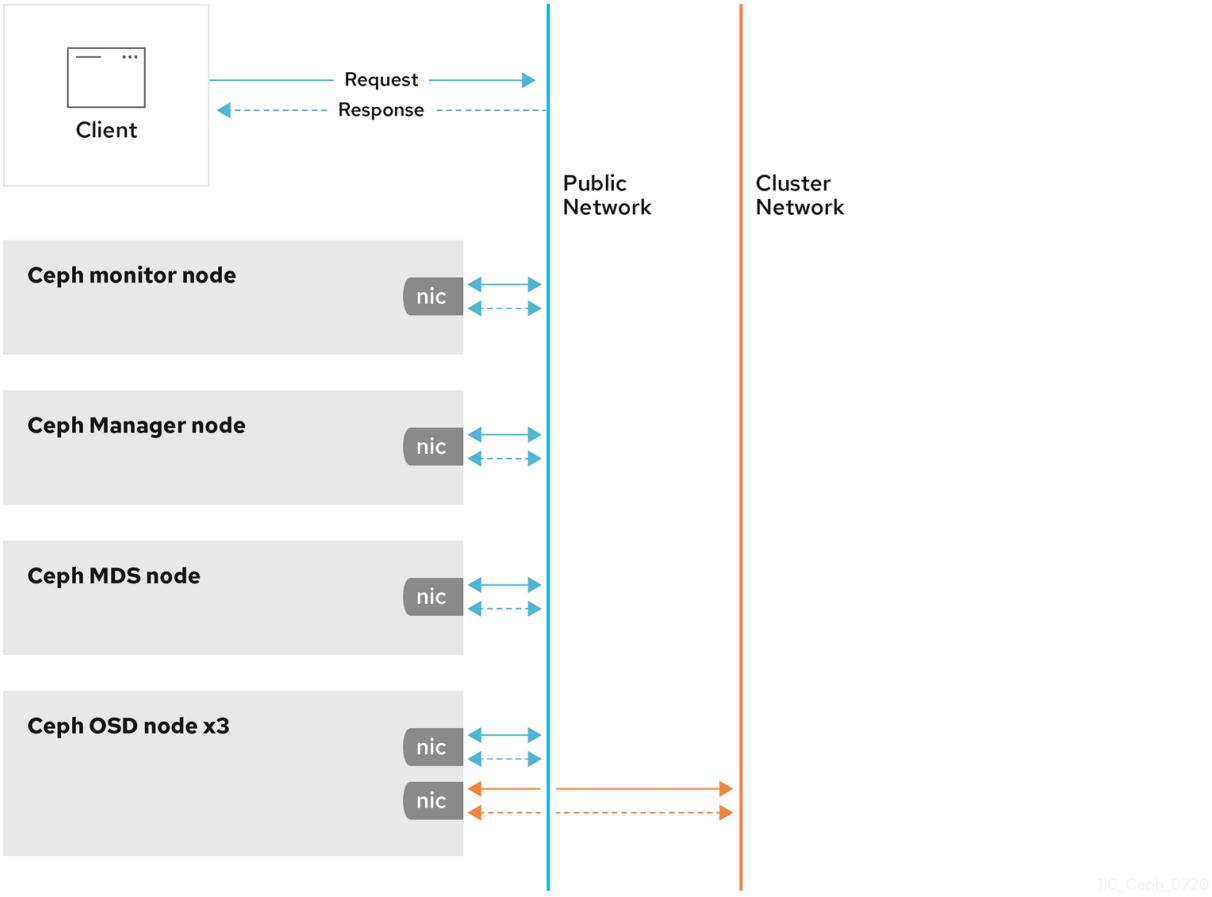


Figure 33: Network architecture

Consider operating two separate networks for better performance and security.

- **Performance:** Ceph OSDs handle data replication for the Ceph clients. When Ceph OSDs replicate data more than once, the network load between Ceph OSDs easily dwarfs the network load between Ceph clients and the Ceph storage cluster. This can introduce latency and create a performance problem. Recovery and rebalancing can also introduce significant latency on the public network.
- **Security**: While most people are generally civil, some actors will engage in what is known as a Denial of Service (DoS) attack. When traffic between Ceph OSDs gets disrupted, peering may fail and placement groups may no longer reflect an `active + clean` state, which may prevent users from reading and writing data. A great way to defeat this type of attack is to maintain a completely separate cluster network that does not connect directly to the internet.

Network configuration settings are not required. Ceph can function with a public network only, assuming a public network is configured on all hosts running a Ceph daemon. However, Ceph allows you to establish much more specific criteria, including multiple IP networks and subnet masks for your public network. You can also establish a separate cluster network to handle OSD heartbeat, object replication, and recovery traffic.

Do not confuse the IP addresses you set in the configuration with the public-facing IP addresses network clients might use to access your service. Typical internal IP networks are often `192.168.0.0` or `10.0.0.0`.

Important: If you specify more than one IP address and subnet mask for either the public or the private network, the subnets within the network must be capable of routing to each other. Additionally, make sure you include each IP address and subnet in your IP tables and open ports for them as necessary.

Note: Ceph uses CIDR notation for subnets, for example, `10.0.0.0/24`.

When you configured the networks, you can restart the cluster or restart each daemon. Ceph daemons bind dynamically, so you do not have to restart the entire cluster at once if you change the network configuration.

For common option descriptions and usage information, see “Network configuration options” on page 297.

Network messenger

Messenger is the Ceph network layer implementation. Both `simple` and `async` messenger types are supported.

The default messenger type is `async`. To change the messenger type, specify the `ms_type` configuration setting in the `[global]` section of the Ceph configuration file.

Note: For the `async` messenger, Red Hat supports the `posix` transport type, but does not currently support `rdma` or `dpdk`. By default, the `ms_type` setting in Red Hat Ceph Storage reflects `async+posix`, where `async` is the messenger type and `posix` is the transport type.

`SimpleMessenger`

The `SimpleMessenger` implementation uses TCP sockets with two threads per socket. Ceph associates each logical session with a connection. A pipe handles the connection, including the input and output of each message. While `SimpleMessenger` is effective for the `posix` transport type, it is not effective for other transport types such as `rdma` or `dpdk`.

`AsyncMessenger`

`AsyncMessenger` is the default messenger type. `AsyncMessenger` implementation uses TCP sockets with a fixed-size thread pool for connections, which should be equal to the highest number of replicas or erasure-code chunks. The thread count can be set to a lower value if performance degrades due to a low CPU count or a high number of OSDs per server.

Note: Other transport types, such as `rdma` and `dpdk` are not currently supported.

For more information about using on-wire encryption with the Ceph messenger version 2 protocol, see Ceph on-wire encryption.

For more information about asynchronous messenger options, see “Network configuration options” on page 297.

Configuring a public network

While Ceph functions well with only a public network, you can establish more specific criteria, including multiple IP networks, for your public network.

Before you begin

Before you begin, make sure that you have the Red Hat Ceph Storage software installed.

About this task

To configure Ceph networks, use the **`config set`** command within the `cephadm` shell.

> **Note:** The network addresses that you set in your network configuration are different from the public-facing IP addresses that network clients might use to access your service.

Ceph can work with only a public network. You can also establish more specific criteria, including multiple IP networks for your public network.

You can also establish a separate, private cluster network to handle OSD heartbeat, object replication, and recovery traffic. For more information about the private network, see “Configuring a private network” on page 291.

> **Note:**
> - Ceph uses CIDR notation for subnets, for example, 10.0.0.0/24. Typical internal IP networks are often 192.168.0.0/24 or 10.0.0.0/24.
> - If you specify more than one network address for either the public or the cluster network, the subnets within the network must be capable of routing to each other. Make sure that you include each IP address in your firewall rules and open ports for them as necessary.

Using the public network configuration allows you to specifically define network addresses and subnets for the public network.

> **Note:** The `public_network` option cannot be updated at runtime. After updating the Ceph configuration database, reconfigure and restart each Monitor daemon to apply the new value.

For common option descriptions and usage information, see “Network configuration options” on page 297.

Procedure

1. Log in to the `cephadm` shell.
   For example,

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Configure the public network with the network address in CIDR notation.

   ```
   ceph config set global public_network NETWORK_ADDRESS_CIDR
   ```

   For example,

```
[ceph: root@host01 /]# ceph config set global public_network
"10.58.145.0/24,10.58.146.0/24,10.58.148.0/24"
```

3. Get the list of Monitor daemons in the storage cluster.
   For example,

```
[ceph: root@host01 /]# ceph orch ps --daemon_type mon
```

4. Verify the current value in the Monitor bootstrap configuration file.

```
cat /var/lib/ceph/FSID/mon.MON_ID/config
```

   For example,

```
[root@host01 ~]# cat /var/lib/ceph/a4b15188-c43e-11ee-9388-5cba2c8ed070/mon.host01/
config

[mon.host01]
public network = 10.58.145.0/24,10.58.146.0/24
```

5. Reconfigure the Monitor daemon to update the Monitor bootstrap configuration file.

```
ceph orch daemon reconfig mon.MON_ID
```

   For example,

```
[ceph: root@host01 /]# ceph orch daemon reconfig mon.host01
```

6. Verify that the Monitor bootstrap configuration file contains the updated value.

```
cat /var/lib/ceph/FSID/mon.MON_ID/config
```

   For example,

```
[root@host01 ~]# cat /var/lib/ceph/a4b15188-c43e-11ee-9388-5cba2c8ed070/mon.host01/
config

[mon.host01]
public network = 10.58.145.0/24,10.58.146.0/24,10.58.148.0/24
```

7. Restart the Monitor daemon.

```
ceph orch daemon restart mon.MON_ID
```

   > **Note:** Wait for the Monitor daemon to regain quorum before reconfiguring and restarting the next Monitor daemon. Repeat this procedure for each Monitor daemon in the cluster.

   For example,

```
[ceph: root@host01 /]# ceph orch daemon restart mon.host01
```

8. Optional: To restart the cluster, from the admin node as a root user run the **`systemctl`** command.

```
systemctl restart ceph-FSID_OF_CLUSTER.target
```

   For example,

```
[root@host01 ~]# systemctl restart ceph-1ca9f6a8-d036-11ec-8263-fa163ee967ad.target
```

Configuring a private network

Network configuration settings are not required. Ceph assumes a public network with all hosts operating on it, unless you specifically configure a cluster network, also known as a *private network*.

If you create a cluster network, OSDs routes heartbeat, object replication, and recovery traffic over the cluster network. This can improve performance, compared to using a single network.

> **Important:** For added security, the cluster network should not be reachable from the public network or the Internet.

To assign a cluster network, use the **`--cluster-network`** option with the `cephadm bootstrap` command. The cluster network that you specify must define a subnet in CIDR notation (for example, 10.90.90.0/24 or fe80::/64).

You can also configure the `cluster_network` after bootstrap.

For more information about invoking `cephadm bootstrap`, see Bootstrapping a new storage cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Access to the Ceph software repository.
- Root-level access to all nodes in the storage cluster.

Procedure

1. Run the `cephadm bootstrap command` from the initial node that you want to use as the Monitor node in the storage cluster. Include the `--cluster-network` option in the command.

   ```
   cephadm bootstrap --mon-ip IP-ADDRESS --registry-url registry.redhat.io --registry-
   username USER_NAME --registry-password PASSWORD --cluster-network NETWORK-IP-ADDRESS
   ```

   For example,

   ```
   [root@host01 ~]# cephadm bootstrap --mon-ip 10.10.128.68 --registry-url
   registry.redhat.io --registry-username myuser1 --registry-password mypassword1 --
   cluster-network 10.10.0.0/24
   ```

2. To configure the `cluster_network` after bootstrap, run the `config set` command and redeploy the daemons

   a. Log in to the `cephadm` shell.
   For example,

   ```
   [root@host01 ~]# cephadm shell
   ```

   b. Configure the cluster network with the subnet.

   ```
   ceph config set global cluster_network IP_ADDRESS_WITH_SUBNET
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph config set global cluster_network 10.10.0.0/24
   ```

   c. Get the list of services in the storage cluster, by running the **`ceph orch ls`** command.
   For example,f

   ```
   [ceph: root@host01 /]# ceph orch ls
   ```

   d. Restart the daemons. Ceph daemons bind dynamically, so you do not have to restart the entire cluster at once if you change the network configuration for a specific daemon.
   For example,

   ```
   [ceph: root@host01 /]# ceph orch restart mon
   ```

   e. If you want to restart the cluster, on the admin node as a root user, run **`systemctl restart`** command.

```
systemctl restart ceph-FSID_OF_CLUSTER.target
```

For example,

```
[root@host01 ~]# systemctl restart ceph-1ca9f6a8-d036-11ec-8263-
fa163ee967ad.target
```

Configuring multiple public networks to the cluster

When the user wants to place the Ceph Monitor daemons on hosts belonging to multiple network subnets, configuring multiple public networks to the cluster is necessary.

An example of usage is a stretch cluster mode used for Advanced Cluster Management (ACM) in Metro DR for OpenShift Data Foundation.

You can configure multiple public networks to the cluster during bootstrap and once bootstrap is complete.

Before you begin

Before adding a host be sure that you have a running Red Hat Ceph Storage cluster.

Procedure

1. Bootstrap a Ceph cluster configured with multiple public networks.
   a. Prepare a `ceph.conf` file containing a `mon` public network section.

   Important: At least one of the provided public networks must be configured on the current host used for bootstrap.

   ```
   [mon]
    public_network = PUBLIC_NETWORK1, PUBLIC_NETWORK2
   ```

   The following is an example with three public networks to be provided for bootstrap.

   ```
   [mon]
    public_network = 10.40.0.0/24, 10.41.0.0/24, 10.42.0.0/24
   ```

   b. Bootstrap the cluster by providing the `ceph.conf` file as input.

   Note: During the bootstrap you can include any other arguments that you want to provide.

   ```
   cephadm --image IMAGE_URL bootstrap --mon-ip MONITOR_IP -c PATH_TO_CEPH_CONF
   ```

   Note: Alternatively, an *IMAGE_ID* (such as, `13ea90216d0be03003d12d7869f72ad9de5cec9e54a27fd308e01e467c0d4a0a`) can be used instead of *IMAGE_URL*.

   For example,

   ```
   cephadm --image admin.lab.redhat.com:5000/rhceph-9-rhel9:latest bootstrap –mon-ip
   10.40.0.0/24 -c /etc/ceph/ceph.conf
   ```

2. Add new hosts to the subnets.

   Note: The host being added must be reachable from the host that the active manager is running on.

a. Install the cluster's public SSH key in the new host's root user's `authorized_keys` file.

```
# ssh-copy-id -f -i /etc/ceph/ceph.pub root@NEW-HOST
```

For example,

```
[root@host01 ~]# ssh-copy-id -f -i /etc/ceph/ceph.pub root@host02
[root@host01 ~]# ssh-copy-id -f -i /etc/ceph/ceph.pub root@host03
```

b. Add the new node to the Ceph cluster.

```
ceph orch host add NEW_HOST IP [LABEL1 ...]
```

For example,

```
ceph orch host add host02 10.41.0.102 label1
ceph orch host add host03 10.42.0.103 label2
```

Note:

- It is best to explicitly provide the host IP address. If an IP is not provided, then the host name will be immediately resolved via DNS and that IP will be used.
- One or more labels can also be included to immediately label the new host. For example, by default the `_admin` label will make `cephadm` maintain a copy of the `ceph.conf` file and a `client.admin` keyring file in `/etc/ceph`.

3. Add the networks configurations for the public network parameters to a running cluster. Be sure that the subnets are separated by commas and that the subnets are listed in subnet/mask format.

```
ceph config set global public_network "SUBNET_1,SUBNET_2, ..."
```

For example,

```
[root@host01 ~]# ceph config set global public_network
"10.40.0.0/24,10.41.0.0/24,10.42.0.0/24"
```

Note: For an existing cluster, configure `public_network` in the global section of the Ceph configuration database. After updating the configuration database, reconfigure and restart the Monitor daemons to apply the new value. For more information, see "Configuring a public network" on page 290.

If necessary, update the `mon` specifications to place the `mon` daemons on hosts within the specified subnets.

Reference

- For more information about adding hosts, see Adding hosts.
- For more information about stretch clusters, see Stretch clusters for Ceph storage.

Verifying firewall rules are configured for default Ceph ports

Verify that the host's firewall allows connection on the default TCP ports.

By default, Red Hat Ceph Storage daemons use TCP ports 6800—7100 to communicate with other hosts in the cluster.

Note: If your network has a dedicated firewall, you might need to verify its configuration in addition to following this procedure. See the firewall's documentation for more information.

See the firewall's documentation for more information.

Prerequisites

- Root-level access to the host.

Procedure

1. Verify the host's `iptables` configuration.
   a. List active rules.

   ```
   [root@host1 ~]# iptables -L
   ```

   b. Verify the absence of rules that restrict connectivity on TCP ports 6800—7100.
   Example

   ```
   REJECT all -- anywhere anywhere reject-with icmp-host-prohibited
   ```

2. Verify the host's `firewalld` configuration.
   a. List ports open on the host.
   Syntax

   ```
   firewall-cmd --zone ZONE --list-ports
   ```

   Example

   ```
   [root@host1 ~]# firewall-cmd --zone default --list-ports
   ```

   b. Verify the range is inclusive of TCP ports 6800—7100.

Firewall settings for Ceph Monitor node

You can enable encryption for all Ceph traffic over the network with the introduction of the messenger version 2 protocol. The `secure` mode setting for messenger v2 encrypts communication between Ceph daemons and Ceph clients, giving you end-to-end encryption.

Messenger v2 Protocol

The second version of Ceph's on-wire protocol, `msgr2`, includes several new features:

- A secure mode encrypts all data moving through the network.
- Encapsulation improvement of authentication payloads.
- Improvements to feature advertisement and negotiation.

The Ceph daemons bind to multiple ports allowing both the legacy, v1-compatible, and the new, v2-compatible, Ceph clients to connect to the same storage cluster. Ceph clients or other Ceph daemons connecting to the Ceph Monitor daemon will try to use the `v2` protocol first, if possible, but if not, then the legacy `v1` protocol will be used. By default, both messenger protocols, `v1` and `v2`, are enabled. The new v2 port is 3300, and the legacy v1 port is 6789, by default.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Access to the Ceph software repository.
- Root-level access to the Ceph Monitor node.

Procedure

1. Add rules using the following example.

```
[root@mon ]# sudo iptables -A INPUT -i IFACE -p tcp -s IP-ADDRESS/NETMASK --dport 6789
-j ACCEPT
[root@mon ]# sudo iptables -A INPUT -i IFACE -p tcp -s IP-ADDRESS/NETMASK --dport 3300
-j ACCEPT
```

a. Replace `IFACE` with the public network interface (for example, `eth0`, `eth1`, and so on).

b. Replace `IP-ADDRESS` with the IP address of the public network and `NETMASK` with the netmask for the public network.

2. For the `firewalld` daemon, run the following commands.

```
[root@mon ~]# firewall-cmd --zone=public --add-port=6789/tcp
[root@mon ~]# firewall-cmd --zone=public --add-port=6789/tcp --permanent
[root@mon ~]# firewall-cmd --zone=public --add-port=3300/tcp
[root@mon ~]# firewall-cmd --zone=public --add-port=3300/tcp --permanent
```

Firewall settings for Ceph OSDs

By default, Ceph OSDs bind to the first available ports on a Ceph node beginning at port 6800. Open at least four ports beginning at port 6800 for each OSD that runs on the node.

The four ports are used as follows:

- One for talking to clients and monitors on the public network.
- One for sending data to other OSDs on the cluster network.
- Two for sending heartbeat packets on the cluster network.

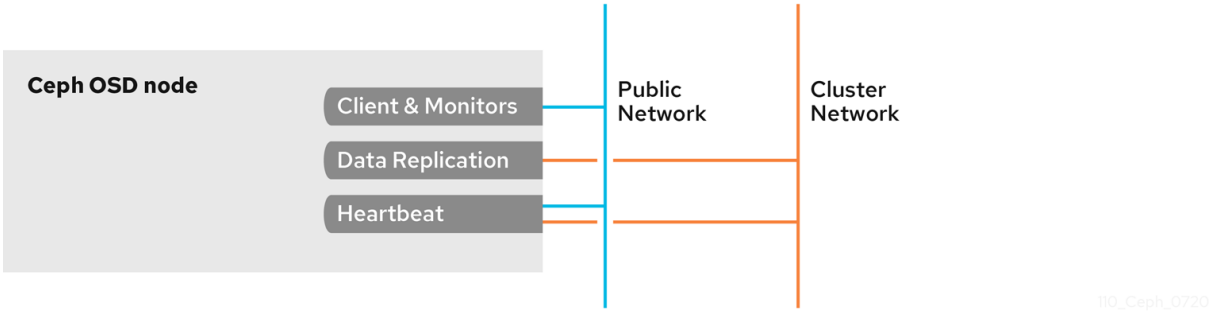


Figure 34: OSD firewall

Ports are node-specific. However, you might need to open more ports than the number of ports needed by Ceph daemons running on that Ceph node in the event that processes get restarted and the bound ports do not get released. Consider opening a few additional ports in case a daemon fails and restarts without releasing the port such that the restarted daemon binds to a new port. Also, consider opening the port range of 6800—7300 on each OSD node.

If you set separate public and cluster networks, you must add rules for both the public network and the cluster network, because clients will connect using the public network and other Ceph OSD Daemons will connect using the cluster network.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Access to the Ceph software repository.
- Root-level access to the Ceph OSD nodes.

Procedure

1. Add rules using the following example:

```
[root@mon ~]# sudo iptables -A INPUT -i IFACE  -m multiport -p tcp -s IP-ADDRESS/
NETMASK --dports 6800:7300 -j ACCEPT
```

a. Replace `IFACE` with the public network interface (for example, `eth0`, `eth1`, and so on).

b. Replace `IP-ADDRESS` with the IP address of the public network and `NETMASK` with the netmask for the public network.

2. For the `firewalld` daemon, run the following:

```
[root@mon ~] # firewall-cmd --zone=public --add-port=6800-7300/tcp
[root@mon ~] # firewall-cmd --zone=public --add-port=6800-7300/tcp --permanent
```

If you put the cluster network into another zone, open the ports within that zone as appropriate.

Network configuration options

Understand the various network configuration options for Ceph.

– "Common options" on page 297
– "Host options" on page 298
– "TCP options" on page 298
– "Bind options" on page 299
– "Asynchronous messenger options" on page 299

Common options

These are the common network configuration options for Ceph.

`public_network`

Description: The network address and CIDR bit count of the public (front-side) network (for example, `192.168.0.0/24`). Set in `[global]`. You can specify comma-delimited subnets.

> **Note:** The `public_network` option cannot be updated at runtime. After updating the Ceph configuration database, reconfigure and restart the Monitor daemons to apply the new value. For more information, see "Configuring a public network" on page 290.

Type: `<network-address>/<CIDR-bit-count> [, <network-address>/<CIDR-bit-count>]`

Required: No

Default: N/A

`public_addr`

Description: The IP address for the public (front-side) network. Set for each daemon.

Type: IP Address

Required: No

Default: N/A

`cluster_network`

Description: The network address and CIDR bit count of the cluster network (for example, `10.0.0.0/24`). Set in `[global]`. You can specify comma-delimited subnets.

Type: `<network-address>/<CIDR-bit-count> [, <network-address>/<CIDR-bit-count>]`

Required: No

Default: N/A

`cluster_addr`

Description: The IP address for the cluster network. Set for each daemon.

Type: Address

Required: No

Default: N/A

`ms_type`

Description: The messenger type for the network transport layer. Red Hat supports the `simple` and the `async` messenger type using `posix` semantics.

Type: String.

Required: No.

Default: `async+posix`

`ms_public_type`

Description: The messenger type for the network transport layer of the public network. It operates identically to `ms_type`, but is applicable only to the public or front-side network. This setting enables Ceph to use a different messenger type for the public or front-side and cluster or back-side networks.

Type: String.

Required: No.

Default: None.

`ms_cluster_type`

Description: The messenger type for the network transport layer of the cluster network. It operates identically to `ms_type`, but is applicable only to the cluster or back-side network. This setting enables Ceph to use a different messenger type for the public or front-side and cluster or back-side networks.

Type: String.

Required: No.

Default: None.

Host options

You must declare at least one Ceph Monitor in the Ceph configuration file, with a `mon addr` setting under each declared monitor. Ceph expects a `host` setting under each declared monitor, metadata server and OSD in the Ceph configuration file.

> **Important:** Do not use `localhost`. Use the short name of the node, not the fully-qualified domain name (FQDN). Do not specify any value for `host` when using a third party deployment system that retrieves the node name for you.

`mon_addr`

Description: A list of `<hostname>:<port>` entries that clients can use to connect to a Ceph monitor. If not set, Ceph searches `[mon.*]` sections.

Type: String

Required: No

Default: N/A

`host`

Description: The host name. Use this setting for specific daemon instances (for example, `[osd.0]`).

Type: String

Required: Yes, for daemon instances.

Default: `localhost`

TCP options

Ceph disables TCP buffering by default.

`ms_tcp_nodelay`

Description: Ceph enables `ms_tcp_nodelay` so that each request is sent immediately (no buffering). Disabling Nagle's algorithm increases network traffic, which can introduce congestion. If you experience large numbers of small packets, you may try disabling `ms_tcp_nodelay`, but be aware that disabling it will generally increase latency.

Type: Boolean

Required: No

Default: `true`

`ms_tcp_rcvbuf`

Description: The size of the socket buffer on the receiving end of a network connection. Disabled by default.

Type: 32-bit Integer

Required: No

Default: 0

Bind options

The bind options configure the default port ranges for the Ceph OSD daemons. The default range is `6800:7100`. You can also enable Ceph daemons to bind to IPv6 addresses.

> **Important:** Verify that the firewall configuration allows you to use the configured port range.

`ms_bind_port_min`

Description: The minimum port number to which an OSD daemon will bind.

Type: 32-bit Integer

Default: `6800`

Required: No

`ms_bind_ipv6`

Description: Enables Ceph daemons to bind to IPv6 addresses.

Type: Boolean

Default: `false`

Required: No

Asynchronous messenger options

These Ceph messenger options configure the behavior of `AsyncMessenger`.

`ms_async_op_threads`

Description: Initial number of worker threads used by each `AsyncMessenger` instance. This configuration setting **SHOULD** equal the number of replicas or erasure code chunks, but it may be set lower if the CPU core count is low or the number of OSDs on a single server is high.

Type: 64-bit Unsigned Integer

Required: No

Default: `3`

Ceph Monitor configuration

Use the default configuration values for the Ceph Monitor or customize them according to the intended workload.

All storage clusters have at least one monitor. A Ceph Monitor configuration usually remains fairly consistent, but you can add, remove or replace a Ceph Monitor in a storage cluster.

Ceph monitors maintain a primary copy of the cluster map. That means a Ceph client can determine the location of all Ceph monitors and Ceph OSDs just by connecting to one Ceph monitor and retrieving a current cluster map.

> **Important:** Be sure to have a Red Hat Ceph Storage cluster that is installed before customizing values for Ceph Monitor.

Before Ceph clients can read from or write to Ceph OSDs, they must connect to a Ceph Monitor first. With a current copy of the cluster map and the CRUSH algorithm, a Ceph client can compute the location for any object. The ability to compute object locations allows a Ceph client to talk directly to Ceph OSDs, which is an important aspect of Ceph's high scalability and performance.

The primary role of the Ceph Monitor is to maintain a primary copy of the cluster map. Ceph Monitors also provide authentication and logging services. Ceph Monitors write all changes in the monitor services to a single Paxos instance, and Paxos writes the changes to a key-value store for strong consistency. Ceph Monitors can query the most recent version of the cluster map during synchronization operations. Ceph Monitors use the key-value store's snapshots and iterators, by using the `rocksdb` database to complete store-wide synchronization. "Figure: Paxos" on page 300 shows the flow of the Ceph Monitors toward a single Paxos instance.

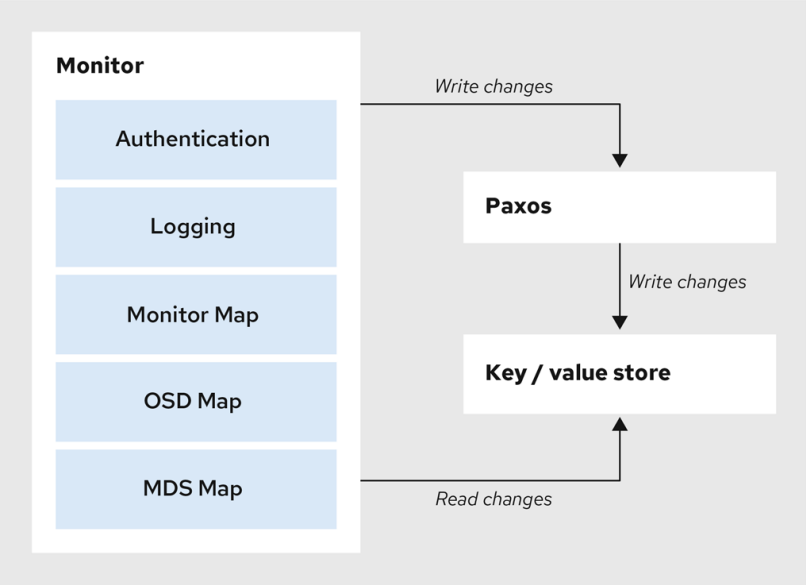


Figure 35: Paxos

> **Important:** In director-deployed Red Hat Ceph Storage environments, replacing Controller nodes can affect the Ceph Monitor service and lead to storage outages if Monitor IP addresses change. For more information, see "Managing the Ceph Monitor service with Red Hat OpenStack Platform" on page 563.

For more information, see "Ceph Monitor configuration options" on page 307.

Viewing the Ceph Monitor configuration database

You can view Ceph Monitor configuration in the configuration database.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to a Ceph Monitor host.

Procedure

1. Log into the `cephadm` shell.

```
[root@host01 ~]# cephadm shell
```

2. Use the `ceph config` command to view the configuration database.

   Example

```
[ceph: root@host01 /]# ceph config get mon
```

For more information about the options available for the `ceph config` command, use `ceph config -h`.

Cluster maps

The cluster map is a composite of maps, including the monitor map, the OSD map, and the placement group map.

The cluster map tracks a number of important events:

- Which processes are `in` the Red Hat Ceph Storage cluster.
- Which processes that are `in` the Red Hat Ceph Storage cluster are `up` and running or `down`.
- Whether, the placement groups are `active` or `inactive`, and `clean` or in some other state.
- Other details that reflect the current state of the cluster, such as:
  - the total amount of storage space or
  - the amount of storage used.

When there is a significant change in the state of the cluster, for example, a Ceph OSD goes down, a placement group falls into a degraded state, and so on. The cluster map gets updated to reflect the current state of the cluster. Additionally, the Ceph monitor also maintains a history of the prior states of the cluster. The monitor map, OSD map, and placement group map each maintain a history of their map versions. Each version is called an **epoch**.

When operating the Red Hat Ceph Storage cluster, keeping track of these states is an important part of the cluster administration.

Ceph Monitor quorum

Run a production Red Hat Ceph Storage cluster with at least three Ceph Monitors to ensure high availability. When you run multiple monitors, you can specify the initial monitors that must be members of the storage cluster to establish a quorum. This may reduce the time it takes for the storage cluster to come online.

A cluster will run sufficiently with a single monitor. However, a single monitor is a single-point-of-failure. To ensure high availability in a production Ceph storage cluster, run Ceph with multiple monitors so that the failure of a single monitor will not cause a failure of the entire storage cluster.

When a Ceph storage cluster runs multiple Ceph Monitors for high availability, Ceph Monitors use the Paxos algorithm to establish consensus about the primary cluster map. A consensus requires a majority of monitors running to establish a quorum for consensus about the cluster map. For example, 1; 2 out of 3; 3 out of 5; 4 out of 6; and so on.

```
[mon]
mon_initial_members = a,b,c
```

> **Note:** Majority of the monitors in the storage cluster must be able to reach each other in to establish a quorum. You can decrease the initial number of monitors to establish a quorum with the `mon_initial_members` option.

Ceph Monitor consistency

When you add monitor settings to the Ceph configuration file, you need to be aware of some of the architectural aspects of Ceph Monitors. Ceph imposes strict consistency requirements for a Ceph Monitor when discovering another Ceph Monitor within the cluster. Whereas Ceph clients and other Ceph daemons use the Ceph configuration file to discover monitors, monitors discover each other using the monitor map (`monmap`), not the Ceph configuration file.

A Ceph Monitor always refers to the local copy of the monitor map when discovering other Ceph Monitors in the Red Hat Ceph Storage cluster. Using the monitor map instead of the Ceph configuration file avoids errors that could break the cluster. For example, typos in the Ceph configuration file when specifying a monitor address or port. Since monitors use monitor maps for discovery and they share monitor maps with clients and other Ceph daemons, the monitor map provides monitors with a strict guarantee that their consensus is valid.

Strict consistency when applying updates to the monitor maps

As with any other updates on the Ceph Monitor, changes to the monitor map always run through a distributed consensus algorithm called Paxos. The Ceph Monitors must agree on each update to the monitor map, such as adding or removing a Ceph Monitor, to ensure that each monitor in the quorum has the same version of the monitor map. Updates to the monitor map are incremental so that Ceph Monitors have the latest agreed-upon version and a set of previous versions.

Maintaining history

Maintaining a history enables a Ceph Monitor that has an older version of the monitor map to catch up with the current state of the Red Hat Ceph Storage cluster.

If Ceph Monitors discovered each other through the Ceph configuration file instead of through the monitor map, it would introduce additional risks because the Ceph configuration files are not updated and distributed automatically. Ceph Monitors might inadvertently use an older Ceph configuration file, fail to recognize a Ceph Monitor, fall out of a quorum, or develop a situation where Paxos is not able to determine the current state of the system accurately.

Bootstrap the Ceph Monitor

In most configuration and deployment cases, tools that deploy Ceph, such as `cephadm`, might help bootstrap the Ceph monitors by generating a monitor map for you.

A Ceph monitor requires a few explicit settings:

- **File System ID**: The `fsid` is the unique identifier for your object store. Since you can run multiple storage clusters on the same hardware, you must specify the unique ID of the object store when bootstrapping a monitor. Using deployment tools, such as `cephadm`, will generate a file system identifier, but you can also specify the `fsid` manually.
- **Monitor ID**: A monitor ID is a unique ID assigned to each monitor within the cluster. By convention, the ID is set to the monitor's hostname. This option can be set using a deployment tool, using the `ceph` command, or in the Ceph configuration file. In the Ceph configuration file, sections are formed as follows:

  Example

  ```
  [mon.host1]
  [mon.host2]
  ```

- **Keys**: The monitor must have secret keys.

Reference

- For more information about `cephadm` and the Ceph orchestrator, see Operations.

Minimum configuration for a Ceph Monitor

The bare minimum monitor settings for a Ceph Monitor in the Ceph configuration file includes a host name for each monitor if it is not configured for DNS and the monitor address. The Ceph Monitors run on port 6789 and 3300 by default.

Important: Do not edit the Ceph configuration file.

Note: This minimum configuration for monitors assumes that a deployment tool generates the `fsid` and the `mon.` key for you.

You can use the following commands to set or read the storage cluster configuration options.

`ceph config dump`

Dumps the entire configuration database for the whole storage cluster.

`ceph config generate-minimal-conf`

Generates a minimal `ceph.conf` file.

`ceph config get WHO`

Dumps the configuration for a specific daemon or a client, as stored in the Ceph Monitor's configuration database.

`ceph config set WHO OPTION VALUE`

Sets the configuration option in the Ceph Monitor's configuration database.

`ceph config show WHO`

Shows the reported running configuration for a running daemon.

`ceph config assimilate-conf -i INPUT_FILE -o OUTPUT_FILE`

Ingests a configuration file from the input file and moves any valid options into the Ceph Monitors' configuration database.

Here, the *WHO* parameter might be name of the section or a Ceph daemon, *OPTION* is a configuration file, and *VALUE* can be either `true` or `false`.

Important: When a Ceph daemon needs a config option prior to getting the option from the config store, you can set the configuration by running the following command:

```
ceph cephadm set-extra-ceph-conf
```

This command adds text to all the daemon's `ceph.conf` files. It is a workaround and is NOT a recommended operation.

Unique identifier for Ceph

Each Red Hat Ceph Storage cluster has a unique identifier (`fsid`).

If specified, the unique identifier usually appears under the `[global]` section of the configuration file. Deployment tools usually generate the `fsid` and store it in the monitor map, so the value may not appear in a configuration file. The `fsid` makes it possible to run daemons for multiple clusters on the same hardware.

Important: Do not set the `fsid` value if you use a deployment tool that does this automatically.

Ceph Monitor data store

Ceph provides a default path where Ceph monitors store data.

Important: For optimal performance in a production Red Hat Ceph Storage cluster, run Ceph monitors on separate drives from Ceph OSDs.

Note: A dedicated `/var/lib/ceph` partition should be used for the MON database with a size between 50 and 100 GB.

Ceph monitors call the `fsync()` function often, which can interfere with Ceph OSD workloads.

Ceph monitors store their data as key-value pairs. Using a data store prevents recovering Ceph monitors from running corrupted versions through Paxos, and it enables multiple modification operations in one single atomic batch, among other advantages.

Important: Only change the default data location if necessary. If you modify the default location, make it uniform across Ceph monitors by setting it in the `[mon]` section of the configuration file.

Storage capacity

Monitor your storage capacity to protect against data loss.

When a Red Hat Ceph Storage cluster gets close to its maximum capacity (specified by the `mon_osd_full_ratio` parameter), Ceph prevents you from writing to or reading from Ceph OSDs as a safety measure to prevent data loss. Therefore, letting a production Red Hat Ceph Storage cluster approach its full ratio is not a good practice, because it sacrifices high availability. The default full ratio is `.95`, or 95% of capacity. This a very aggressive setting for a test cluster with a small number of OSDs.

Tip: When monitoring a cluster, be alert to warnings related to the `nearfull` ratio. This means that a failure of some OSDs could result in a temporary service disruption if one or more OSDs fails. Consider adding more OSDs to increase storage capacity.

A common scenario for test clusters involves a system administrator removing a Ceph OSD from the Red Hat Ceph Storage cluster to watch the cluster re-balance. Then, removing another Ceph OSD, and so on until the Red Hat Ceph Storage cluster eventually reaches the full ratio and locks up.

Important: Red Hat recommends a bit of capacity planning even with a test cluster. Planning enables you to gauge how much spare capacity you will need in to maintain high availability.

Ideally, you want to plan for a series of Ceph OSD failures where the cluster can recover to an `active + clean` state without replacing those Ceph OSDs immediately. You can run a cluster in an `active + degraded` state, but this is not ideal for normal operating conditions.

The following diagram depicts a simplistic Red Hat Ceph Storage cluster containing 33 Ceph Nodes with one Ceph OSD per host, each Ceph OSD Daemon reading from and writing to a 3TB drive. So this exemplary Red Hat Ceph Storage cluster has a maximum actual capacity of 99TB. With a `mon osd full ratio` of `0.95`, if the Red Hat Ceph Storage cluster falls to 5 TB of remaining capacity, the cluster will not allow Ceph clients to read and write data. So, the Red Hat Ceph Storage cluster's operating capacity is 95 TB, not 99 TB.



Figure 36: Storage capacity

It is normal in such a cluster for one or two OSDs to fail. A less frequent but reasonable scenario involves a rack's router or power supply failing, which brings down multiple OSDs simultaneously, for example, OSDs 7-12. In such a scenario, you should still strive for a cluster that can remain operational and achieve an `active + clean` state, even if that means adding a few hosts with additional OSDs in short order. If your capacity utilization is too high, you might not lose data, but you could still sacrifice data availability while resolving an outage within a failure domain if capacity utilization of the cluster exceeds the full ratio. For this reason, Red Hat recommends at least some rough capacity planning.

Identify two numbers for your cluster:

- the number of OSDs
- the total capacity of the cluster

To determine the mean average capacity of an OSD within a cluster, divide the total capacity of the cluster by the number of OSDs in the cluster. Consider multiplying that number by the number of OSDs you expect to fail simultaneously during normal operations (a relatively small number). Finally, multiply the capacity of the cluster by the full ratio to arrive at a maximum operating capacity. Then, subtract the amount of data from the OSDs you expect to fail to arrive at a reasonable full ratio. Repeat the foregoing process with a higher number of OSD failures (for example, a rack of OSDs) to arrive at a reasonable number for a near full ratio.

Ceph heartbeat

Ceph monitors know about the cluster by requiring reports from each OSD, and by receiving reports from OSDs about the status of their neighboring OSDs.

Ceph provides reasonable default settings for interaction between monitor and OSD, however, you can modify them as needed.

Ceph Monitor synchronization role

When you run a production cluster with multiple monitors which is recommended, each monitor checks to see if a neighboring monitor has a more recent version of the cluster map. For example, a map in a neighboring monitor with one or more epoch numbers higher than the most current epoch in the map of the instant monitor. Periodically, one monitor in the cluster might fall behind the other monitors to the point where it must leave the quorum, synchronize to retrieve the most current information about the cluster, and then rejoin the quorum.

Synchronization roles

For the purposes of synchronization, monitors can assume one of three roles:

Leader

The Leader is the first monitor to achieve the most recent Paxos version of the cluster map.

Provider

The Provider is a monitor that has the most recent version of the cluster map, but was not the first to achieve the most recent version.

Requester

The Requester is a monitor that has fallen behind the leader and must synchronize to retrieve the most recent information about the cluster before it can rejoin the quorum.

These roles enable a leader to delegate synchronization duties to a provider, which prevents synchronization requests from overloading the leader and improving performance.

In “Figure: Ceph Monitor synchronization roles” on page 306, the requester has learned that it has fallen behind the other monitors. The requester asks the leader to synchronize, and the leader tells the requester to synchronize with a provider.

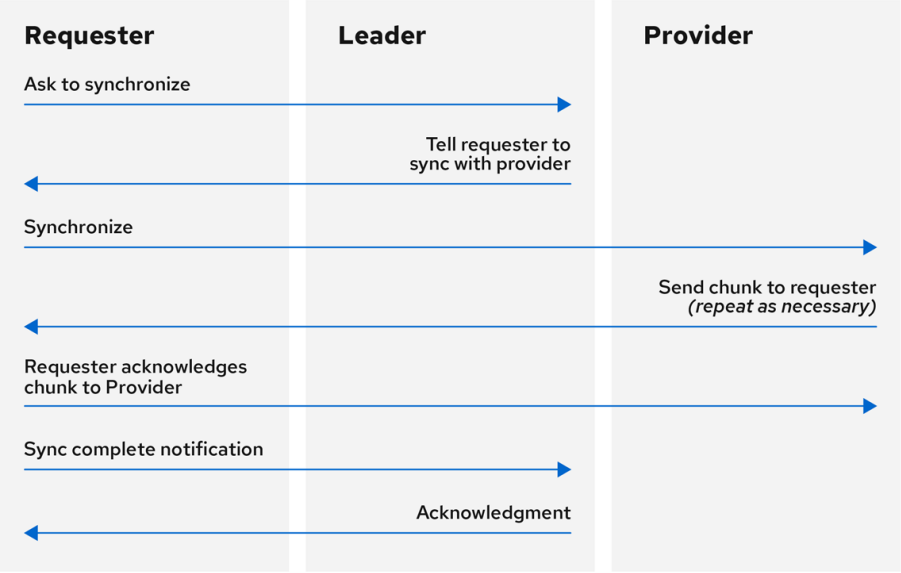


Figure 37: Ceph Monitor synchronization roles

Monitor synchronization

Synchronization always occurs when a new monitor joins the cluster. During runtime operations, monitors can receive updates to the cluster map at different times. This means the leader and provider roles may migrate from one monitor to another. If this happens while synchronizing, for example, a provider falls behind the leader, the provider can terminate synchronization with a requester.

Once synchronization is complete, Ceph requires trimming across the cluster. Trimming requires that the placement groups are `active + clean`.

Time synchronization

Ceph daemons pass critical messages to each other, which must be processed before daemons reach a timeout threshold. If the clocks in Ceph monitors are not synchronized, it can lead to a number of anomalies.

For example:

- Daemons ignoring received messages such as outdated timestamps.
- Timeouts triggered too soon or late when a message was not received in time.

> **Tip:** Install NTP on the Ceph monitor hosts to ensure that the monitor cluster operates with synchronized clocks.

Clock drift may still be noticeable with NTP even though the discrepancy is not yet harmful. Ceph clock drift and clock skew warnings can get triggered even though NTP maintains a reasonable level of synchronization. Increasing your clock drift may be tolerable under such circumstances. However, a number of factors such as workload, network latency, configuring overrides to default timeouts, and other synchronization options can influence the level of acceptable clock drift without compromising Paxos guarantees.

Ceph Monitor configuration options

Understand the various Ceph monitor configuration options that can be set up during deployment.

You can set these configuration options with the **`ceph config set mon CONFIGURATION_OPTION VALUE`** command.

Table 48: Table title, optional

Configuration option	Description	Type	Default
`mon_force_quorum_join`	Force monitor to join quorum even if it has been previously removed from the map	Boolean	`False`
`mon_dns_srv_name`	The service name used for querying the DNS for the monitor hosts/ addresses.	String	`ceph-mon`
`fsid`	The cluster ID. One per cluster.	UUID	N/A. May be generated by a deployment tool if not specified.
`mon_data_size_warn`	Ceph issues a HEALTH_WARN status in the cluster log when the monitor's data store reaches this threshold. The default value is 15GB.	Integer	`15*1024*1024*1024*`
`mon_data_avail_warn`	Ceph issues a HEALTH_WARN status in the cluster log when the available disk space of the monitor's data store is lower than or equal to this percentage.	Integer	`30`
`mon_data_avail_crit`	Ceph issues a HEALTH_ERR status in the cluster log when the available disk space of the monitor's data store is lower or equal to this percentage.	Integer	`5`
`mon_warn_on_cache_pools_without_hit_sets`	Ceph issues a HEALTH_WARN status in the cluster log if a cache pool does not have the **`hit_set_type`** parameter set.	Boolean	True

Configuration option	Description	Type	Default
mon_warn_on_crush_straw_calc_version_zero	Ceph issues a HEALTH_WARN status in the cluster log if the CRUSH's straw_calc_version is zero. See CRUSH tunables overview for details.	Boolean	True
mon_warn_on_legacy_crush_tunables	Ceph issues a HEALTH_WARN status in the cluster log if CRUSH tunables are too old (older than mon_min_crush_required_version).	Boolean	True
mon_crush_min_required_version	This setting defines the minimum tunable profile version required by the cluster.	String	hammer
mon_warn_on_osd_down_out_interval_zero	Ceph issues a HEALTH_WARN status in the cluster log if the mon_osd_down_out_interval setting is zero, because the Leader behaves in a similar manner when the **noout** flag is set. Administrators find it easier to troubleshoot a cluster by setting the **noout** flag. Ceph issues the warning to ensure administrators know that the setting is zero.	Boolean	True
mon_health_to_clog	This setting enables Ceph to send a health summary to the cluster log periodically.	Boolean	True
mon_health_detail_to_clog	This setting enable Ceph to send a health details to the cluster log periodically.	Boolean	True
mon_op_complaint_time	Number of seconds after which the Ceph Monitor operation is considered blocked after no updates.	Integer	30
mon_health_to_clog_tick_interval	How often (in seconds) the monitor sends a health summary to the cluster log. A non-positive number disables it. If the current health summary is empty or identical to the last time, the monitor will not send the status to the cluster log.	Float	60.000000

Configuration option	Description	Type	Default
`mon_health_to_clog_interval`	How often (in seconds) the monitor sends a health summary to the cluster log. A non-positive number disables it. The monitor will always send the summary to the cluster log.	Integer	`600`
`mon_sync_timeout`	The number of seconds the monitor will wait for the next update message from its sync provider before it gives up and bootstraps again.	Double	`60.000000`
`mon_sync_max_payload_size`	The maximum size for a sync payload (in bytes).	32-bit Integer	`1045676`
`paxos_max_join_drift`	The maximum Paxos iterations before we must first sync the monitor data stores. When a monitor finds that its peer is too far ahead of it, it will first sync with data stores before moving on.	Integer	`10`
`paxos_stash_full_interval`	How often (in commits) to stash a full copy of the PaxosService state. Currently this setting only affects `mds`, `mon`, `auth`, and `mgr` PaxosServices.	`Integer`	`25`
`paxos_propose_interval`	Gather updates for this time interval before proposing a map update.	`Double`	`1.0`
`paxos_min`	The minimum number of paxos states to keep around	`Integer`	`500`
`paxos_min_wait`	The minimum amount of time to gather updates after a period of inactivity.	`Double`	`0.05`
`paxos_trim_min`	Number of extra proposals tolerated before trimming	`Integer`	`250`
`paxos_trim_max`	The maximum number of extra proposals to trim at a time	`Integer`	`500`
`paxos_service_trim_min`	The minimum amount of versions to trigger a trim (0 disables it)	`Integer`	`250`
`paxos_service_trim_max`	The maximum amount of versions to trim during a single proposal (0 disables it)	`Integer`	`500`

Configuration option	Description	Type	Default
`mon_mds_force_trim_to`	Force monitor to trim mdsmaps to this point (0 disables it. Dangerous, use with care)	`Integer`	`0`
`mon_osd_force_trim_to`	Force monitor to trim osdmaps to this point, even if there is PGs not clean at the specified epoch (0 disables it. dangerous, use with care)	`Integer`	`0`
`mon_osd_cache_size`	The size of osdmaps cache, not to rely on underlying store's cache	`Integer`	`500`
`mon_election_timeout`	On election proposer, maximum waiting time for all ACKs in seconds.	`Float`	`5`
`mon_lease_renew_interval_factor`	`mon lease` * `mon lease renew interval factor` will be the interval for the Leader to renew the other monitor's leases. The factor should be less than `1.0`.	`Float`	`0.6`
`mon_lease_ack_timeout_factor`	The Leader will wait `mon lease` * `mon lease ack timeout factor` for the Providers to acknowledge the lease extension.	`Float`	`2.0`
`mon_min_osdmap_epochs`	Minimum number of OSD map epochs to keep at all times.	32-bit Integer	`500`
`mon_max_pgmap_epochs`	Maximum number of PG map epochs the monitor should keep.	32-bit Integer	`500`
`mon_max_log_epochs`	Maximum number of Log epochs the monitor should keep.	32-bit Integer	`500`
`mon_tick_interval`	A monitor's tick interval in seconds.	32-bit Integer	`5`
`mon_clock_drift_allowed`	The clock drift in seconds allowed between monitors.	Float	`.050`
`mon_clock_drift_warn_backoff`	Exponential backoff for clock drift warnings.	Float	`5`
`mon_timecheck_interval`	The time check interval (clock drift check) in seconds for the leader.	Float	`300.0`
`mon_timecheck_skew_interval`	The time check interval (clock drift check) in seconds when in the presence of a skew in seconds for the Leader.	Float	`30.0`

Configuration option	Description	Type	Default
mon_max_osd	The maximum number of OSDs allowed in the cluster.	32-bit Integer	10000
mon_globalid_prealloc	The number of global IDs to pre-allocate for clients and daemons in the cluster.	32-bit Integer	10000
mon_subscribe_interval	The refresh interval, in seconds, for subscriptions. The subscription mechanism enables obtaining the cluster maps and log information.	Double	86400.000000
mon_stat_smooth_intervals	Ceph will smooth statistics over the last N PG maps.	Integer	6
mon_probe_timeout	Number of seconds the monitor will wait to find peers before bootstrapping.	Double	2.0
mon_daemon_bytes	The message memory cap for metadata server and OSD messages (in bytes).	64-bit Integer Unsigned	400ul << 20
mon_max_log_entries_per_event	The maximum number of log entries per event.	Integer	4096
mon_osd_prime_pg_temp	Enables or disable priming the PGMap with the previous OSDs when an out OSD comes back into the cluster. With the true setting, the clients will continue to use the previous OSDs until the newly in OSDs as that PG peered.	Boolean	true
mon_osd_prime_pg_temp_max_time	How much time in seconds the monitor should spend trying to prime the PGMap when an out OSD comes back into the cluster.	Float	0.5
mon_osd_pool_ec_fast_read	Whether turn on fast read on the pool or not. It will be used as the default setting of newly created erasure pools if fast_read is not specified at create time.	Boolean	False

Configuration option	Description	Type	Default
mon_mds_skip_sanity	Skip safety assertions on FSMap, in case of bugs where we want to continue anyway. Monitor terminates if the FSMap sanity check fails, but we can disable it by enabling this option.	Boolean	False
mon_max_mdsmap_epochs	The maximum amount of mdsmap epochs to trim during a single proposal.	Integer	500
mon_config_key_max_entry_size	The maximum size of config-key entry (in bytes).	Integer	65536
mon_warn_pg_not_scrubbed_ratio	The percentage of the scrub max interval past the scrub max interval to warn.	float	0.5
mon_warn_pg_not_deep_scrubbed_ratio	The percentage of the deep scrub interval past the deep scrub interval to warn	float	0.75
mon_scrub_interval	How often, in seconds, the monitor scrub its store by comparing the stored checksums with the computed ones of all the stored keys.	Integer	3600*24
mon_scrub_timeout	The timeout to restart scrub of mon quorum participant does not respond for the latest chunk.	Integer	5 min
mon_scrub_max_keys	The maximum number of keys to scrub each time.	Integer	100
mon_scrub_inject_missing_keys	The probability of injecting missing keys into mon scrub.	Float	0
mon_compact_on_start	Compact the database used as Ceph Monitor store on ceph-mon start. A manual compaction helps to shrink the monitor database and improve its performance if the regular compaction fails to work.	Boolean	False
mon_compact_on_bootstrap	Compact the database used as Ceph Monitor store on bootstrap. The monitor starts probing each other for creating a quorum after bootstrap. If it times out before joining the quorum, it will start over and bootstrap itself again.	Boolean	False

Configuration option	Description	Type	Default
mon_compact_on_trim	Compact a certain prefix (including paxos) when we trim its old states.	Boolean	True
mon_osd_mapping_pgs_per_chunk	We calculate the mapping from the placement group to OSDs in chunks. This option specifies the number of placement groups per chunk.	Integer	4096
rados_mon_op_timeout	Number of seconds to wait for a response from the monitor before returning an error from a RADOS operation. 0 means at limit, or no wait time.	Double	0

Ceph authentication configuration

Authenticating users and services is important to the security of the Red Hat Ceph Storage cluster. Red Hat Ceph Storage includes the Cephx protocol, as the default, for cryptographic authentication, and the tools to manage authentication in the storage cluster.

Be sure to have a Red Hat Ceph Storage cluster installed before configuring user authentication.

For more information, see "Cephx configuration options" on page 319.

Cephx authentication

The cephx protocol is enabled by default.

Cryptographic authentication has some computational costs, though they are generally low. If the network environment connecting clients and hosts is considered safe and you cannot afford authentication computational costs, you can disable it. When deploying a Ceph storage cluster, the deployment tool creates the client.admin user and keyring.

> **Important:** Use authentication. If authentication is disabled you are at risk of a man-in-the-middle attack altering client and server messages, which can lead to significant security issues.

Enabling and disabling Cephx

Enabling Cephx requires that you have deployed keys for the Ceph Monitors and OSDs. When toggling Cephx authentication on or off, you do not have to repeat the deployment procedures.

Enabling Cephx

When cephx is enabled, Ceph will look for the keyring in the default search path, which includes /etc/ceph/$cluster.$name.keyring. You can override this location by adding a keyring option in the [global] section of the Ceph configuration file, but this is not recommended.

Run the following procedures to enable cephx on a cluster with authentication disabled. If you or your deployment utility have already generated the keys, you may skip the steps related to generating keys.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph Monitor node.

Procedure

1. Create a `client.admin` key, and save a copy of the key for your client host:

```
[root@mon ~]# ceph auth get-or-create client.admin mon 'allow *' osd 'allow *' -o /
etc/ceph/ceph.client.admin.keyring
```

> **Warning:** This will erase the contents of any existing `/etc/ceph/client.admin.keyring` file. Do not perform this step if a deployment tool has already done it for you.

2. Create a keyring for the monitor cluster and generate a monitor secret key:

```
 [root@mon ~]# ceph-authtool --create-keyring /tmp/ceph.mon.keyring --gen-key -n mon.
--cap mon 'allow *'
```

3. Copy the monitor keyring into a `ceph.mon.keyring` file in every monitor `mon data` directory. For example, to copy it to `mon.a` in cluster `ceph`, use the following:

```
 [root@mon ~]# cp /tmp/ceph.mon.keyring /var/lib/ceph/mon/ceph-a/keyring
```

4. Generate a secret key for every OSD, where `_ID_` is the OSD number:

```
 ceph auth get-or-create osd.ID mon allow rwx osd allow * -o /var/lib/ceph/osd/ceph-
ID/keyring
```

5. By default the `cephx` authentication protocol is enabled.

> **Note:** If the `cephx` authentication protocol was disabled previously by setting the authentication options to `none`, then by removing the following lines under the `[global]` section in the Ceph configuration file (`/etc/ceph/ceph.conf`) will reenable the `cephx` authentication protocol:
>
> ```
> auth_cluster_required = none
>  auth_service_required = none
>  auth_client_required = none
> ```

6. Start or restart the Ceph storage cluster.

> **Important:**
> Enabling `cephx` requires downtime because the cluster needs to be completely restarted, or it needs to be shut down and then started while client I/O is disabled. These flags need to be set before restarting or shutting down the storage cluster:
>
> ```
> [root@mon ~]# ceph osd set noout
>  [root@mon ~]# ceph osd set norecover
>  [root@mon ~]# ceph osd set norebalance
>  [root@mon ~]# ceph osd set nobackfill
>  [root@mon ~]# ceph osd set nodown
>  [root@mon ~]# ceph osd set pause
> ```

Once `cephx` is enabled and all PGs are active and clean, unset the flags:

```
[root@mon ~]# ceph osd unset noout
 [root@mon ~]# ceph osd unset norecover
 [root@mon ~]# ceph osd unset norebalance
 [root@mon ~]# ceph osd unset nobackfill
 [root@mon ~]# ceph osd unset nodown
 [root@mon ~]# ceph osd unset pause
```

Disabling Cephx

If your cluster environment is relatively safe, you can offset the computation expense of the running authentication, by disabling Cephx.

The following procedure describes how to disable Cephx.

> **Important:** Red Hat recommends enabling authentication.

However, it may be easier during setup or troubleshooting to temporarily disable authentication.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph Monitor node.

Procedure

1. Disable `cephx` authentication by setting the following options in the `[global]` section of the Ceph configuration file:

   Example

   ```
   auth_cluster_required = none
   auth_service_required = none
   auth_client_required = none
   ```

2. Start or restart the Ceph storage cluster.

Cephx user keyrings

When you run Ceph with authentication enabled, the `ceph` administrative commands and Ceph clients require authentication keys to access the Ceph storage cluster.

The most common way to provide these keys to the `ceph` administrative commands and clients is to include a Ceph keyring under the `/etc/ceph/` directory. The file name is usually `ceph.client.admin.keyring` or `$cluster.client.admin.keyring`. If you include the keyring under the `/etc/ceph/` directory, you do not need to specify a `keyring` entry in the Ceph configuration file.

> **Important:** Copy the Red Hat Ceph Storage cluster keyring file to nodes where you will run administrative commands, because it contains the `client.admin` key.

To do so, run the following command:

```
# scp USER@HOSTNAME:/etc/ceph/ceph.client.admin.keyring /etc/ceph/ceph.client.admin.keyring
```

Replace *USER* with the user name used on the host with the `client.admin` key and *HOSTNAME* with the host name of that host.

> **Note:** Ensure the `ceph.keyring` file has appropriate permissions set on the client machine.

You can specify the key itself in the Ceph configuration file using the `key` setting, which is not recommended, or a path to a key file using the `keyfile` setting.

CephX daemon keyrings

A daemon keyring stores a secret key and capabilities for a specific Ceph daemon. CephX uses these keyrings to authenticate each daemon with the Monitor before the daemon can participate in the cluster.

How daemon authentication works

When a Ceph daemon starts, it reads its keyring file and presents its credentials to the Monitor. The Monitor verifies the key and issues a time-limited session ticket that the daemon uses for all subsequent cluster communication. Each daemon has its own unique keyring containing a secret key and a set of capabilities that define what the daemon is authorized to do.

Administrative users or deployment tools such as `cephadm` generate daemon keyrings in the same way as generating user keyrings. By default, Ceph stores daemon keyrings inside the daemon data directory.

> **Important:** The Monitor keyring contains a key but no capabilities, and is not part of the cluster `auth` database. Monitor keyrings are used only during initial cluster bootstrap for peer-to-peer Monitor authentication.

Daemon keyrings and client keyrings

Both daemon keyrings and client keyrings use the CephX protocol, but they serve different purposes and are managed differently.

Attribute	Daemon keyrings	Client keyrings
Entity name format	`TYPE.ID` — for example, `osd.0`, `mgr.node1`, `mds.cephfs1`	`client.NAME` — for example, `client.admin`, `client.rgw`
Purpose	Authenticate a running cluster daemon with the Monitor	Grant a user, application, or service access to cluster data
Capabilities	Scoped to the minimum permissions required for that daemon type	Defined by the administrator based on access requirements
Default storage location	`/var/lib/ceph/FSID/DAEMON-NAME/keyring` (cephadm deployments)	`/etc/ceph/ceph.client.admin.keyring` or application-specific path
Management	Created and rotated automatically by `cephadm`	Created and managed by the cluster administrator

Default keyring locations

In deployments that use the `cephadm` orchestrator, the daemon data directory defaults to the following path on the host where the daemon runs:

```
/var/lib/ceph/FSID/DAEMON-TYPE.DAEMON-ID/keyring
```

For example, the keyring for `osd.2` in a cluster with FSID `a7f3b1c2-4d5e-6f78-...` is stored at:

```
/var/lib/ceph/a7f3b1c2-4d5e-6f78-.../osd.2/keyring
```

In legacy (non-cephadm) deployments, the data directory defaults to directories of the form:

```
/var/lib/ceph/$type/CLUSTER-ID
```

For example:

```
/var/lib/ceph/osd/ceph-12
```

Keep the default locations to ensure that `cephadm` and other management tools can locate the keyring automatically.

> **CAUTION:**
> **The `cephadm` orchestrator bind-mounts each daemon keyring into the daemon container at `/etc/ceph/keyring`. Do not manually copy or modify keyring files managed by `cephadm`. Manual changes may conflict with the credentials stored in the Monitor authentication database and cause authentication failures.**

Anatomy of a daemon keyring file

A keyring file is a plain-text file in INI format. Each section corresponds to a CephX entity. The following example shows the keyring for `osd.0`:

```
[osd.0]
        key = AQCvCKRkAAAAABAAX1mxBEDDMnHkubu0hJPk2A==
        caps mgr = "allow profile osd"
        caps mon = "allow profile osd"
        caps osd = "allow *"
```

`[osd.0]`

The entity header in the format `DAEMON-TYPE.DAEMON-ID`. This value must match the CephX entity registered in the Monitor authentication database.

`key`

The Base64-encoded secret key used to generate and verify CephX authentication tickets. Treat this value as a sensitive credential and restrict read access to the keyring file.

`caps`

Capabilities that define what operations this entity is authorized to perform against each daemon type. The `allow profile osd` shorthand grants the standard permissions required for an OSD to function. The `allow *` on the `osd` capability grants the OSD full access to peer OSDs, which is required for replication and recovery operations.

Capabilities by daemon type

The following table summarizes the standard capability profiles that `cephadm` assigns when it creates a daemon keyring. The `allow profile TYPE` shorthand is a built-in CephX profile that expands to the exact capability set required for that daemon type.

Daemon type	Default capabilities
Monitor (`mon.ID`)	`caps mon = "allow *"` Monitors require full access to the cluster map and authentication database. The Monitor keyring is used only during bootstrap and contains no capabilities in the cluster auth database.

OSD (`osd.ID`)	`caps mgr = "allow profile osd"`
	`caps mon = "allow profile osd"`
	`caps osd = "allow *"`
	
	OSDs report status to the Manager and Monitor, and communicate with peer OSDs for replication and recovery.

Manager (`mgr.ID`)	`caps mds = "allow *"`
	`caps mon = "allow profile mgr"`
	`caps osd = "allow *"`
	
	The Manager aggregates cluster-wide metrics and runs orchestration and monitoring modules, requiring broad read and write access.

MDS (`mds.ID`)	`caps mds = "allow"`
	`caps mon = "allow profile mds"`
	`caps osd = "allow rwx pool=CEPHFS-POOLS"`
	
	Each MDS instance requires access to the CephFS metadata and data pools.

Keyring lifecycle in cephadm deployments

`cephadm` manages the keyring lifecycle automatically.

Creation

When `cephadm` deploys a daemon, it calls `ceph auth get-or-create` to register a CephX entity in the Monitor authentication database and writes the returned keyring to the host storage path.

Startup authentication

When the daemon container starts, it reads the keyring and presents its credentials to the Monitor. The Monitor verifies the key and issues a time-limited session ticket used for all subsequent cluster communication.

Key rotation

To rotate a daemon key without redeploying the daemon, run:

```
ceph orch daemon rotate-key DAEMON-TYPE.DAEMON-ID
```

`cephadm` generates a new key, updates the Monitor authentication database, and rewrites the keyring file on the host.

Daemon removal

When you remove a daemon with `ceph orch daemon rm`, `cephadm` removes the CephX entity from the Monitor authentication database and deletes the keyring from host storage.

Inspecting daemon keyrings

Use the following commands to inspect and verify daemon keyring configuration.

List all authentication entities in the cluster

```
ceph auth ls
```

Display the keyring for a specific entity

```
ceph auth get ENTITY
```

For example, to inspect the credentials for `osd.0`:

```
ceph auth get osd.0
```

Example output:

```
[osd.0]
        key = AQCvCKRkAAAAABAAX1mxBEDDMnHkubu0hJPk2A==
        caps mgr = "allow profile osd"
        caps mon = "allow profile osd"
        caps osd = "allow *"
```

Export a keyring to a file

```
ceph auth get ENTITY -o /path/to/keyring
```

Verify daemon health and status

```
ceph orch ps
```

If a daemon fails to authenticate, its status in `ceph orch ps` shows as `error` or the daemon restarts repeatedly. Check the daemon log for authentication errors:

```
ceph orch daemon logs DAEMON-TYPE.DAEMON-ID
```

Cephx message signatures

Ceph provides fine-grained control so you can enable or disable signatures for service messages between the client and Ceph.

Signatures for messages can be enabled or disabled between Ceph daemons.

Important: Ceph should authenticate all ongoing messages between the entities by using the session key set up for that initial authentication.

Note: Ceph kernel modules do not currently support signatures.

Cephx configuration options

Understand the various Cephx configuration options that can be set up during deployment.

`auth_cluster_required`

Description: Valid settings are `cephx` or `none`.

Type: String

Required: No

Default: `cephx`.

`auth_service_required`

Description: Valid settings are `cephx` or `none`.

Type: String

Required: No

Default: `cephx`.

`auth_client_required`

Description: If enabled, the Red Hat Ceph Storage cluster daemons require Ceph clients to authenticate with the Red Hat Ceph Storage cluster in order to access Ceph services. Valid settings are `cephx` or `none`.

Type: String

Required: No

Default: `cephx`.

`keyring`

Description: The path to the keyring file.

Type: String

Required: No

Default: `/etc/ceph/$cluster.$name.keyring`, `/etc/ceph/$cluster.keyring`, `/etc/ceph/keyring`, `/etc/ceph/keyring.bin`

`keyfile`

Description: The path to a key file (that is. a file containing only the key).

Type: String

Required: No

Default: None

`key`

Description: The key (that is, the text string of the key itself). Not recommended.

Type: String

Required: No

Default: None

`ceph-mon`

Location: `$mon_data/keyring`

Capabilities: `mon 'allow *'`

`ceph-osd`

Location: `$osd_data/keyring`

Capabilities: `mon 'allow profile osd' osd 'allow *'`

`radosgw`

Location: `$rgw_data/keyring`

Capabilities: `mon 'allow rwx' osd 'allow rwx'`

`cephx_require_signatures`

Description: If set to `true`, Ceph requires signatures on all message traffic between the Ceph client and the Red Hat Ceph Storage cluster, and between daemons comprising the Red Hat Ceph Storage cluster.

Type: Boolean

Required: No

Default: `false`

`cephx_cluster_require_signatures`

Description: If set to `true`, Ceph requires signatures on all message traffic between Ceph daemons comprising the Red Hat Ceph Storage cluster.

Type: Boolean

Required: No

Default: `false`

`cephx_service_require_signatures`

Description: If set to `true`, Ceph requires signatures on all message traffic between Ceph clients and the Red Hat Ceph Storage cluster.

Type: Boolean

Required: No

Default: `false`

`cephx_sign_messages`

Description: If the Ceph version supports message signing, Ceph will sign all messages so they cannot be spoofed.

Type: Boolean

Default: `true`

`auth_service_ticket_ttl`

Description: When the Red Hat Ceph Storage cluster sends a Ceph client a ticket for authentication, the cluster assigns the ticket a time to live.

Type: Double

Default: `60*60`

Pools, placement groups, and CRUSH configuration

As a storage administrator, you can choose to use the Red Hat Ceph Storage default options for pools, placement groups, and the CRUSH algorithm or customize them for the intended workload.

Pools, placement groups and CRUSH

When you create pools and set the number of placement groups for the pool, Ceph uses default values when you do not specifically override the defaults.

> **Important:** It is best to override some of the defaults. Specifically, set a pool's replica size and override the default number of placement groups.

You can set these values when running pool commands.

By default, Ceph makes three replicas of objects. If you want to set four copies of an object as the default value, a primary copy and three replica copies, reset the default values as shown in `osd_pool_default_size`. If you want to allow Ceph to write a lesser number of copies in a degraded state, set `osd_pool_default_min_size` to a number less than the `osd_pool_default_size` value.

For example,

```
[ceph: root@host01 /]# ceph config set global osd_pool_default_size 4  # Write an object
four times.
[ceph: root@host01 /]# ceph config set global osd_pool_default_min_size 1  # Allow writing
one copy in a degraded state.
```

Ensure you have a realistic number of placement groups. Red Hat recommends approximately 100 per OSD. Total number of OSDs multiplied by 100 divided by the number of replicas, that is, `osd_pool_default_size`. For 10 OSDs and `osd_pool_default_size` = 4, we would recommend approximately (100*10)/4=250.

For example,

```
[ceph: root@host01 /]# ceph config set global osd_pool_default_pg_num 250
[ceph: root@host01 /]# ceph config set global osd_pool_default_pgp_num 250
```

Pools, placement groups, and CRUSH configuration options

Understand the various Ceph options that govern pools, placement groups, and the CRUSH algorithm.

"Ceph pool, placement group, and CRUSH configuration options" on page 321 lists the options that govern pools, placement groups, and the CRUSH algorithm, with their types and default values.

Table 51: Ceph pool, placement group, and CRUSH configuration options

Options	Description	Type	Default
`mon_allow_pool_delete`	Allows a monitor to delete a pool. In RHCS 3 and later releases, the monitor cannot delete the pool by default as an added measure to protect data.	Boolean	`false`
`mon_max_pool_pg_num`	The maximum number of placement groups per pool.	Integer	`65536`
`mon_pg_create_interval`	Number of seconds between PG creation in the same Ceph OSD Daemon.	Float	`30.0`
`mon_pg_stuck_threshold`	Number of seconds after which PGs can be considered as being stuck.	32-bit Integer	`300`

Options	Description	Type	Default
mon_pg_min_inactive	Ceph issues a HEALTH_ERR status in the cluster log if the number of PGs that remain inactive longer than the mon_pg_stuck_threshold exceeds this setting. The default setting is one PG. A non-positive number disables this setting.	Integer	1
mon_pg_warn_max_per_osd	Ceph issues a HEALTH_WARN status in the cluster log if the average number of PGs per OSD in the cluster is greater than this setting. A non-positive number disables this setting.	Integer	300
mon_pg_warn_min_per_osd	Ceph issues a HEALTH_WARN status in the cluster log if the average number of PGs per OSD in the cluster is less than this setting. A non-positive number disables this setting.	Integer	30
mon_pg_warn_min_objects	Do not warn if the total number of objects in the cluster is below this number.	Integer	1000
mon_pg_warn_min_pool_objects	Do not warn on pools whose object number is below this number.	Integer	1000
mon_pg_check_down_all_threshold	The threshold of down OSDs by percentage after which Ceph checks all PGs to ensure they are not stuck or stale.	Float	0.5
mon_pg_warn_max_object_skew	Ceph issue a HEALTH_WARN status in the cluster log if the average number of objects in a pool is greater than mon pg warn max object skew times the average number of objects for all pools. A non-positive number disables this setting.	Float	10

Options	Description	Type	Default
`mon_delta_reset_interval`	The number of seconds of inactivity before Ceph resets the PG delta to zero. Ceph keeps track of the delta of the used space for each pool to aid administrators in evaluating the progress of recovery and performance.	Integer	`10`
`mon_osd_max_op_age`	The maximum age in seconds for an operation to complete before issuing a `HEALTH_WARN` status.	Float	`32.0`
`osd_pg_bits`	Placement group bits per Ceph OSD Daemon.	32-bit Integer	`6`
`osd_pgp_bits`	The number of bits per Ceph OSD Daemon for Placement Groups for Placement purpose (PGPs).	32-bit Integer	`6`
`osd_crush_chooseleaf_type`	The bucket type to use for `chooseleaf` in a CRUSH rule. Uses ordinal rank rather than name.	32-bit Integer	`1`. Typically a host containing one or more Ceph OSD Daemons.
`osd_pool_default_crush_replicated_ruleset`	The default CRUSH ruleset to use when creating a replicated pool.	8-bit Integer	`0`
`osd_pool_erasure_code_stripe_unit`	Sets the default size, in bytes, of a chunk of an object stripe for erasure coded pools. Every object of size S will be stored as N stripes, with each data chunk receiving `stripe unit` bytes. Each stripe of `N * stripe unit` bytes will be encoded/decoded individually. This option can be overridden by the `stripe_unit` setting in an erasure code profile.	Unsigned 32-bit Integer	`4096`
`osd_pool_default_size`	Sets the number of replicas for objects in the pool. The default value is the same as **`ceph osd pool set POOL NAME size SIZE`**.	32-bit Integer	`3`

Options	Description	Type	Default
`osd_pool_default_min_size`	Sets the minimum number of written replicas for objects in the pool in order to acknowledge a write operation to the client. If the minimum is not met, Ceph will not acknowledge the write to the client. This setting ensures a minimum number of replicas when operating in `degraded` mode.	32-bit Integer	`0`, which means no particular minimum. If `0`, minimum is `size - (size / 2)`.
`osd_pool_default_pg_num`	The default number of placement groups for a pool. The default value is the same as `pg_num` with `mkpool`.	32-bit Integer	`32`
`osd_pool_default_pgp_num`	The default number of placement groups for placement for a pool. The default value is the same as `pgp_num` with `mkpool`. PG and PGP should be equal.	32-bit Integer	`0`
`osd_pool_default_flags`	The default flags for new pools.	32-bit Integer	`0`
`osd_max_pgls`	The maximum number of placement groups to list. A client requesting a large number can tie up the Ceph OSD Daemon.	Unsigned 64-bit Integer	`1024`
`osd_min_pg_log_entries`	The minimum number of placement group logs to maintain when trimming log files.	32-bit Int Unsigned	`250`
`osd_default_data_pool_replay_window`	The time, in seconds, for an OSD to wait for a client to replay a request.	32-bit Integer	`45`

Ceph Object Storage Daemon (OSD) configuration

Configure the Ceph Object Storage Daemon (OSD) to be redundant and optimized based on the intended workload.

For more information, see “Object Storage Daemon (OSD) configuration options” on page 325.

Ceph OSD configuration

All Ceph clusters have a configuration. A deployment tool, such as `cephadm`, will typically create an initial Ceph configuration file for you. However, you can create one yourself if you prefer to bootstrap a cluster without using a deployment tool.

All Ceph clusters have a configuration, which defines:

- Cluster identity
- Authentication settings

- Ceph daemon membership in the cluster
- Network configuration
- Host names and addresses
- Paths to keyrings
- Paths to OSD log files
- Other runtime options

For your convenience, each daemon has a series of default values. Many are set by the `ceph/src/common/config_opts.h` script. You can override these settings with a Ceph configuration file or at runtime by using the monitor `tell` command or connecting directly to a daemon socket on a Ceph node.

> **Important:** Changing the default paths makes it more difficult to troubleshoot Ceph later.

Reference

- For more information about `cephadm` and the Ceph orchestrator, see Operations.

Scrubbing the OSD

In addition to making multiple copies of objects, Ceph ensures data integrity by scrubbing placement groups. Ceph scrubbing is analogous to the **`fsck`** command on the object storage layer.

For each placement group, Ceph generates a catalog of all objects and compares each primary object and its replicas to ensure that no objects are missing or mismatched.

Light scrubbing (daily) checks the object size and attributes. Deep scrubbing (weekly) reads the data and uses checksums to ensure data integrity.

Scrubbing is important for maintaining data integrity, but it can reduce performance. Adjust the following settings to increase or decrease scrubbing operations.

For more information, see “Scrubbing options” on page 340.

Backfilling an OSD

When you add Ceph OSDs to a cluster or remove them from the cluster, the CRUSH algorithm rebalances the cluster by moving placement groups to or from Ceph OSDs to restore the balance. The process of migrating placement groups and the objects they contain can reduce the cluster operational performance considerably. To maintain operational performance, Ceph performs this migration with the *backfill* process, which allows Ceph to set backfill operations to a lower priority than requests to read or write data.

OSD recovery

When the cluster starts or when a Ceph OSD terminates unexpectedly and restarts, the OSD begins peering with other Ceph OSDs before a write operation can occur.

If a Ceph OSD crashes and comes back online, usually it will be out of sync with other Ceph OSDs containing more recent versions of objects in the placement groups. When this happens, the Ceph OSD goes into recovery mode and seeks to get the latest copy of the data and bring its map back up to date. Depending upon how long the Ceph OSD was down, the OSD’s objects and placement groups may be significantly out of date. Also, if a failure domain went down, for example, a rack, more than one Ceph OSD might come back online at the same time. This can make the recovery process time consuming and resource intensive.

To maintain operational performance, Ceph performs recovery with limitations on the number of recovery requests, threads, and object chunk sizes which allows Ceph to perform well in a degraded state.

Object Storage Daemon (OSD) configuration options

Understand the various Ceph Object Storage Daemon (OSD) configuration options that can be set during deployment.

You can set these configuration options with the **`ceph config set osd CONFIGURATION_OPTION VALUE`** command.

"Ceph OSD configuration options" on page 326 lists the OSD configuration options, their types, and default values.

Table 52: Ceph OSD configuration options

Option	Description	Type	Default
`osd_uuid`	The universally unique identifier (UUID) for the Ceph OSD.	UUID	The UUID. **Note:** The **`osd_uuid`** option applies to a single Ceph OSD. The **`fsid`** option applies to the entire cluster.
`osd_data`	The path to the OSD's data. You must create the directory when deploying Ceph. Mount a drive for OSD data at this mount point.	String	`/var/lib/ceph/osd/$cluster-$id` **Important:** Red Hat does not recommend changing the default.
`osd_max_write_size`	The maximum size of a write in megabytes.	32-bit Integer	`90`
`osd_client_message_size_cap`	The largest client data message allowed in memory.	64-bit Integer Unsigned	500 MB. `500*1024L*1024L`
`osd_class_dir`	The class path for RADOS class plug-ins.	String	`$libdir/rados-classes`
`osd_max_scrubs`	The maximum number of simultaneous scrub operations for a Ceph OSD.	32-bit Int	`1`
`osd_scrub_thread_timeout`	The maximum time in seconds before timing out a scrub thread.	32-bit Integer	`60`
`osd_scrub_finalize_thread_timeout`	The maximum time in seconds before timing out a scrub finalize thread.	32-bit Integer	`60*10`

Option	Description	Type	Default
osd_scrub_begin_hour	This restricts scrubbing to this hour of the day or later. Use `osd_scrub_begin_hour = 0` and `osd_scrub_end_hour = 0` to allow scrubbing the entire day. Along with **osd_scrub_end_hour**, they define a time window, in which the scrubs can happen. But a scrub is performed no matter whether the time window allows or not, as long as the placement group's scrub interval exceeds **osd_scrub_max_interval**. The allowed range is 0–23.	Integer	`0`
osd_scrub_end_hour	This restricts scrubbing to the hour earlier than this. Use `osd_scrub_begin_hour = 0` and `osd_scrub_end_hour = 0` to allow scrubbing for the entire day. Along with **osd_scrub_begin_hour**, they define a time window, in which the scrubs can happen. But a scrub is performed no matter whether the time window allows or not, as long as the placement group's scrub interval exceeds **osd_scrub_max_interval**. The allowed range is 0–23.	Integer	`0`
osd_scrub_load_threshold	The maximum load. Ceph will not scrub when the system load (as defined by the `getloadavg()` function) is higher than this number.	Float	`0.5`
osd_scrub_min_interval	The minimum interval in seconds for scrubbing the Ceph OSD when the Red Hat Ceph Storage cluster load is low.	Float	Once per day. `60*60*24`
osd_scrub_max_interval	The maximum interval in seconds for scrubbing the Ceph OSD irrespective of cluster load.	Float	Once per week. `7*60*60*24`
osd_scrub_interval_randomize_ratio	Takes the ratio and randomizes the scheduled scrub between **osd_scrub_min_interval** and **osd_scrub_max_interval**.	Float	`0.5`
mon_warn_not_scrubbed	Number of seconds after **osd_scrub_interval** to warn about any PGs that were not scrubbed.	Integer	`0` (no warning)
osd_scrub_chunk_min	The object store is partitioned into chunks which end on hash boundaries. For chunky scrubs, Ceph scrubs objects one chunk at a time with writes blocked for that chunk. The **osd_scrub_chunk_min** setting represents the minimum number of chunks to scrub.	32-bit Integer	`5`
osd_scrub_chunk_max	The maximum number of chunks to scrub.	32-bit Integer	`25`
osd_scrub_sleep	The time to sleep between deep scrub operations.	Float	`0` (or off)
osd_scrub_during_recovery	Allows scrubbing during recovery.	Boolean	`false`
osd_scrub_invalid_stats	Forces extra scrub to fix stats marked as invalid.	Boolean	`true`
osd_scrub_priority	Controls queue priority of scrub operations versus client I/O.	Unsigned 32-bit Integer	`5`
osd_requested_scrub_priority	The priority set for user requested scrub on the work queue. If this value were to be smaller than **osd_client_op_priority**, it can be boosted to the value of **osd_client_op_priority** when scrub is blocking client operations.	Unsigned 32-bit Integer	`120`

Option	Description	Type	Default
osd_scrub_cost	Cost of scrub operations in megabytes for queue scheduling purposes.	Unsigned 32-bit Integer	52428800
osd_deep_scrub_interval	The interval for deep scrubbing, that is fully reading all data. The **osd_scrub_load_threshold** parameter does not affect this setting.	Float	Once per week. 60*60*24*7
osd_deep_scrub_stride	Read size when doing a deep scrub.	32-bit Integer	512 KB. 524288
mon_warn_not_deep_scrubbed	Number of seconds after **osd_deep_scrub_interval** to warn about any PGs that were not scrubbed.	Integer	0 (no warning)
osd_deep_scrub_randomize_ratio	The rate at which scrubs will randomly become deep scrubs (even before **osd_deep_scrub_interval** has passed).	Float	0.15 or 15%
osd_deep_scrub_update_digest_min_age	How many seconds old objects must be before scrub updates the whole-object digest.	Integer	7200 (2 hours)
osd_deep_scrub_large_omap_object_key_threshold	Warning when you encounter an object with more OMAP keys than this.	Integer	200000
osd_deep_scrub_large_omap_object_value_sum_threshold	Warning when you encounter an object with more OMAP key bytes than this.	Integer	1 G
osd_delete_sleep	Time in seconds to sleep before the next removal transaction. This throttles the placement group deletion process.	Float	0.0
osd_delete_sleep_hdd	Time in seconds to sleep before the next removal transaction for HDDs.	Float	5.0
osd_delete_sleep_ssd	Time in seconds to sleep before the next removal transaction for SSDs.	Float	1.0
osd_delete_sleep_hybrid	Time in seconds to sleep before the next removal transaction when Ceph OSD data is on HDD and OSD journal or WAL and DB is on SSD.	Float	1.0
osd_op_num_shards	The number of shards for client operations.	32-bit Integer	0
osd_op_num_threads_per_shard	The number of threads per shard for client operations.	32-bit Integer	0
osd_op_num_shards_hdd	The number of shards for HDD operations.	32-bit Integer	5
osd_op_num_threads_per_shard_hdd	The number of threads per shard for HDD operations.	32-bit Integer	1
osd_op_num_shards_ssd	The number of shards for SSD operations.	32-bit Integer	8
osd_op_num_threads_per_shard_ssd	The number of threads per shard for SSD operations.	32-bit Integer	2
osd_client_op_priority	The priority set for client operations. It is relative to **osd_recovery_op_priority**. The allowed range is 1–63.	32-bit Integer	63

Option	Description	Type	Default
osd_recovery_op_priority	The priority set for recovery operations. It is relative to **osd_client_op_priority**. The allowed range is 1–63.	32-bit Integer	3
osd_op_thread_timeout	The Ceph OSD operation thread timeout in seconds.	32-bit Integer	15
osd_op_complaint_time	An operation becomes complaint worthy after the specified number of seconds have elapsed.	Float	30
osd_disk_threads	The number of disk threads, which are used to perform background disk intensive OSD operations such as scrubbing and snap trimming.	32-bit Integer	1
osd_op_history_size	The maximum number of completed operations to track.	32-bit Unsigned Integer	20
osd_op_history_duration	The oldest completed operation to track.	32-bit Unsigned Integer	600
osd_op_log_threshold	How many operations logs to display at once.	32-bit Integer	5
osd_op_timeout	The time in seconds after which running OSD operations time out.	Integer	0 **Important:** Do not set the **osd_op_timeout** option unless your clients can handle the consequences. For example, setting this parameter on clients running in virtual machines can lead to data corruption because the virtual machines interpret this timeout as a hardware failure.
osd_max_backfills	The maximum number of backfill operations allowed to or from a single OSD.	64-bit Unsigned Integer	1
osd_backfill_scan_min	The minimum number of objects per backfill scan.	32-bit Integer	64

Option	Description	Type	Default
osd_backfill_scan_max	The maximum number of objects per backfill scan.	32-bit Integer	512
osd_backfill_full_ratio	Refuse to accept backfill requests when the Ceph OSD's full ratio is above this value.	Float	0.85
osd_backfill_retry_interval	The number of seconds to wait before retrying backfill requests.	Double	30.000000
osd_map_dedup	Enable removing duplicates in the OSD map.	Boolean	true
osd_map_cache_size	The size of the OSD map cache in megabytes.	32-bit Integer	50
osd_map_cache_bl_size	The size of the in-memory OSD map cache in OSD daemons.	32-bit Integer	50
osd_map_cache_bl_inc_size	The size of the in-memory OSD map cache incrementals in OSD daemons.	32-bit Integer	100
osd_map_message_max	The maximum map entries allowed per MOSDMap message.	32-bit Integer	40
osd_snap_trim_thread_timeout	The maximum time in seconds before timing out a snap trim thread.	32-bit Integer	60*60*1
osd_pg_max_concurrent_snap_trims	The max number of parallel snap trims/PG. This controls how many objects per PG to trim at once.	32-bit Integer	2
osd_snap_trim_sleep	Insert a sleep between every trim operation a PG issues.	32-bit Integer	0
osd_max_trimming_pgs	The max number of trimming PGs.	32-bit Integer	2
osd_backlog_thread_timeout	The maximum time in seconds before timing out a backlog thread.	32-bit Integer	60*60*1
osd_default_notify_timeout	The OSD default notification timeout (in seconds).	32-bit Integer Unsigned	30
osd_check_for_log_corruption	Check log files for corruption. Can be computationally expensive.	Boolean	false
osd_remove_thread_timeout	The maximum time in seconds before timing out a remove OSD thread.	32-bit Integer	60*60
osd_command_thread_timeout	The maximum time in seconds before timing out a command thread.	32-bit Integer	10*60
osd_command_max_records	Limits the number of lost objects to return.	32-bit Integer	256
osd_auto_upgrade_tmap	Uses `tmap` for `omap` on old objects.	Boolean	true
osd_tmapput_sets_users_tmap	Uses `tmap` for debugging only.	Boolean	false
osd_preserve_trimmed_log	Preserves trimmed log files, but uses more disk space.	Boolean	false
osd_recovery_delay_start	After peering completes, Ceph delays for the specified number of seconds before starting to recover objects.	Float	0
osd_recovery_max_active	The number of active recovery requests per OSD at one time. More requests will accelerate recovery, but the requests place an increased load on the cluster.	32-bit Integer	0

Option	Description	Type	Default
osd_recovery_max_chunk	The maximum size of a recovered chunk of data to push.	64-bit Integer Unsigned	8388608
osd_recovery_threads	The number of threads for recovering data.	32-bit Integer	1
osd_recovery_thread_timeout	The maximum time in seconds before timing out a recovery thread.	32-bit Integer	30
osd_recover_clone_overlap	Preserves clone overlap during recovery. Should always be set to `true`.	Boolean	true
rados_osd_op_timeout	Number of seconds that RADOS waits for a response from the OSD before returning an error from a RADOS operation. A value of 0 means no limit.	Double	0

Ceph Monitor and OSD interaction configuration

Be sure to properly configure the interactions between the Ceph Monitors and OSDs to ensure a stable working environment.

For more information, see "Ceph Monitor and OSD configuration options" on page 334.

Ceph Monitor and OSD interaction

After you have completed your initial Ceph configuration, you can deploy and run Ceph. When you run a command such as **ceph health** or **ceph -s**, the Ceph Monitor reports on the current state of the Ceph storage cluster.

The Ceph Monitor knows about the Ceph storage cluster by requiring reports from each Ceph OSD daemon, and by receiving reports from Ceph OSD daemons about the status of their neighboring Ceph OSD daemons. If the Ceph Monitor does not receive reports, or if it receives reports of changes in the Ceph storage cluster, the Ceph Monitor updates the status of the Ceph cluster map.

Ceph provides reasonable default settings for Ceph Monitor and OSD interaction. However, you can override the defaults. The following sections describe how Ceph Monitors and Ceph OSD daemons interact for the purposes of monitoring the Ceph storage cluster.

OSD heartbeat

Each Ceph OSD daemon checks the heartbeat of other Ceph OSD daemons every 6 seconds. The heartbeat interval can be changed.

To change the heartbeat interval, change the value at runtime:

Syntax

```
ceph config set osd osd_heartbeat_interval TIME_IN_SECONDS
```

Example

```
[ceph: root@host01 /]# ceph config set osd osd_heartbeat_interval 60
```

If a neighboring Ceph OSD daemon does not send heartbeat packets within a 20 second grace period, the Ceph OSD daemon might consider the neighboring Ceph OSD daemon `down`. It can report it back to a Ceph Monitor, which updates the Ceph cluster map. To change the grace period, set the value at runtime:

Syntax

```
ceph config set osd osd_heartbeat_grace TIME_IN_SECONDS
```

Example

```
[ceph: root@host01 /]# ceph config set osd osd_heartbeat_grace 30
```



Figure 38: Check heartbeats

Reporting an OSD as down

By default, two Ceph OSD Daemons *from different hosts* must report to the Ceph Monitors that another Ceph OSD Daemon is `down` before the Ceph Monitors acknowledge that the reported Ceph OSD Daemon is `down`.

There is the possibility that all the OSDs reporting the failure are in different hosts in a rack with a bad switch that causes connection problems between OSDs.

To avoid a "false alarm," Ceph considers the peers reporting the failure as a proxy for a "subcluster" that is similarly laggy. While this is not always the case, it may help administrators localize the grace correction to a subset of the system that is performing poorly.

Ceph uses the `mon_osd_reporter_subtree_level` setting to group the peers into the "subcluster" by their common ancestor type in the CRUSH map.

By default, only two reports from **a different subtree** are required to report another Ceph OSD Daemon `down`. Administrators can change the number of reporters from unique subtrees and the common ancestor type required to report a Ceph OSD Daemon `down` to a Ceph Monitor by setting the `mon_osd_min_down_reporters` and `mon_osd_reporter_subtree_level` values at runtime:

Syntax

```
ceph config set mon mon_osd_min_down_reporters NUMBER
```

Example

```
[ceph: root@host01 /]# ceph config set mon mon_osd_min_down_reporters 4
```

Syntax

```
ceph config set mon mon_osd_reporter_subtree_level CRUSH_ITEM
```

Example

```
[ceph: root@host01 /]# ceph config set mon mon_osd_reporter_subtree_level host
[ceph: root@host01 /]# ceph config set mon mon_osd_reporter_subtree_level rack
[ceph: root@host01 /]# ceph config set mon mon_osd_reporter_subtree_level osd
```

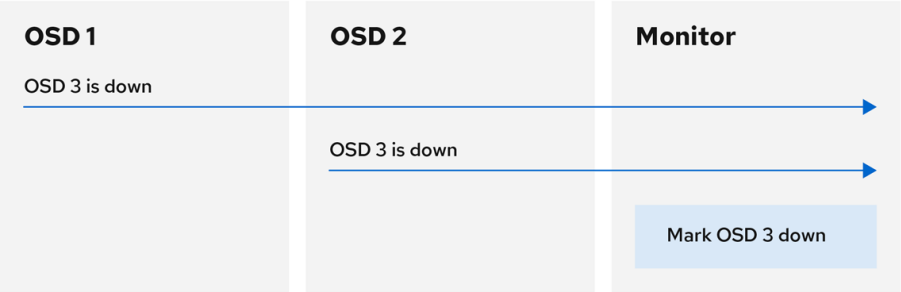


Figure 39: Report down OSDs

Reporting a peering failure

If a Ceph OSD daemon cannot peer with any of the Ceph OSD daemons defined in its Ceph configuration file or the cluster map, it pings a Ceph Monitor for the most recent copy of the cluster map every 30 seconds. The Ceph Monitor heartbeat interval can be changed.

You can change the Ceph Monitor heartbeat interval by setting the value at runtime:

Syntax

```
ceph config set osd osd_mon_heartbeat_interval TIME_IN_SECONDS
```

Example

```
[ceph: root@host01 /]# ceph config set osd osd_mon_heartbeat_interval 60
```

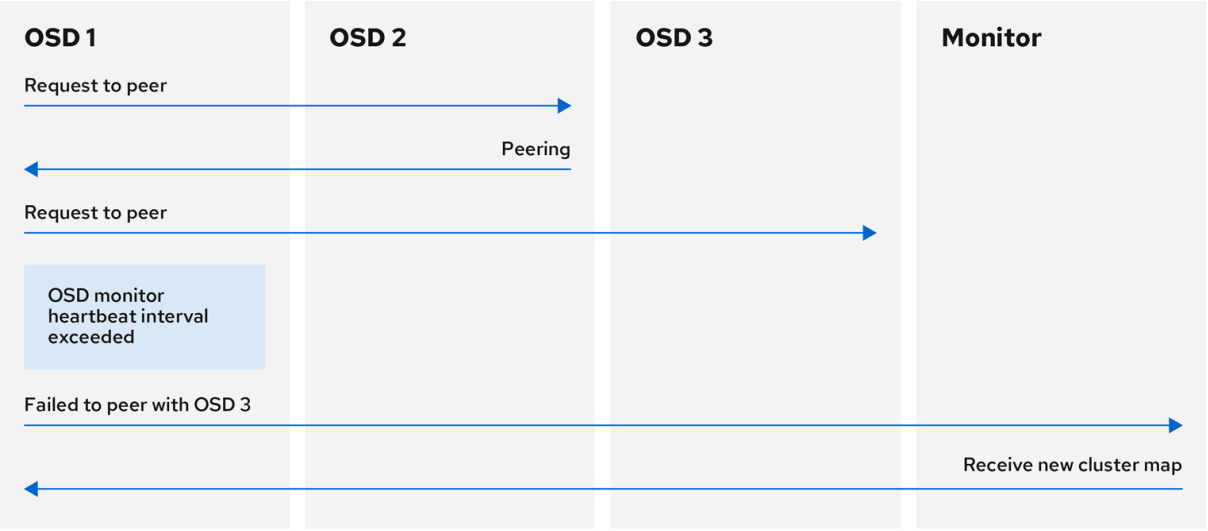


Figure 40: Report peering failure

OSD reporting status

If a Ceph OSD Daemon does not report to a Ceph Monitor, the Ceph Monitor marks the Ceph OSD Daemon `down` after the `mon_osd_report_timeout`, which is 900 seconds, elapses. A Ceph OSD Daemon sends a report to a

Ceph Monitor when a reportable event such as a failure, a change in placement group stats, a change in `up_thru` or when it boots within 5 seconds. the minimum report interval can be changed.

You can change the Ceph OSD Daemon minimum report interval by setting the `osd_mon_report_interval` value at runtime:

Syntax

```
ceph config set osd osd_mon_report_interval TIME_IN_SECONDS
```

To get, set, and verify the config you can use the following example:

Example

```
[ceph: root@host01 /]# ceph config get osd osd_mon_report_interval
5

[ceph: root@host01 /]# ceph config set osd osd_mon_report_interval
20

[ceph: root@host01 /]# ceph config dump | grep osd
global                  advanced  osd_pool_default_crush_rule                -1
osd                   basic     osd_memory_target                          4294967296
osd                   advanced  osd_mon_report_interval                    20
```

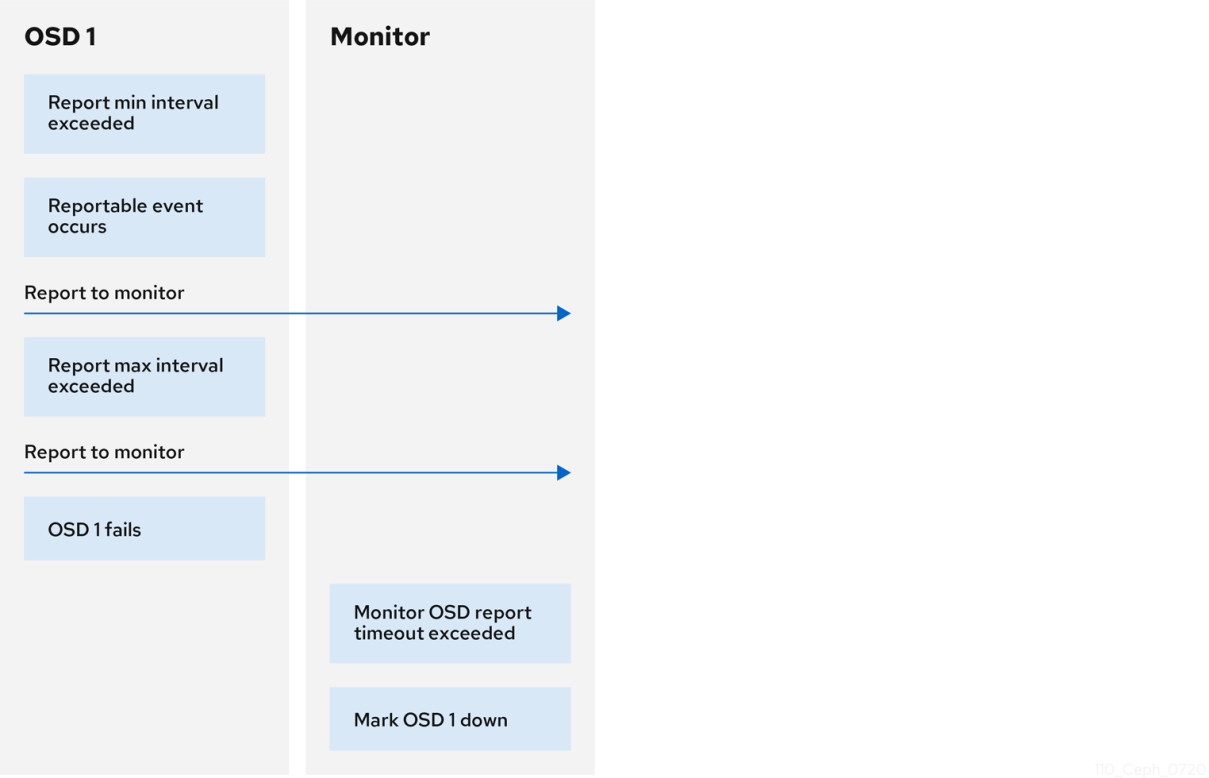


Figure 41: Ceph configuration update

Ceph Monitor and OSD configuration options

Understand the various Ceph Monitor and OSD configuration options.

When modifying heartbeat settings, include them in the `[global]` section of the Ceph configuration file.

`mon_osd_min_up_ratio`

Description: The minimum ratio of `up` Ceph OSD Daemons before Ceph will mark Ceph OSD Daemons `down`.

Type: Double

Default: `.3`

mon_osd_min_in_ratio

Description: The minimum ratio of `in` Ceph OSD Daemons before Ceph will mark Ceph OSD Daemons `out`.

Type Double

Default: `0.750000`

mon_osd_laggy_halflife

Description: The number of seconds `laggy` estimates will decay.

Type: Integer

Default: `60*60`

mon_osd_laggy_weight

Description: The weight for new samples in `laggy` estimation decay.

Type: Double

Default: `0.3`

mon_osd_laggy_max_interval

Description: Maximum value of `laggy_interval` in laggy estimations (in seconds). The monitor uses an adaptive approach to evaluate the `laggy_interval` of a certain OSD. This value will be used to calculate the grace time for that OSD.

Type: Integer

Default: `300`

mon_osd_adjust_heartbeat_grace

Description: If set to `true`, Ceph will scale based on `laggy` estimations.

Type: Boolean

Default: `true`

mon_osd_adjust_down_out_interval

Description: If set to `true`, Ceph will scaled based on `laggy` estimations.

Type: Boolean

Default: `true`

mon_osd_auto_mark_in

Description: Ceph will mark any booting Ceph OSD Daemons as `in` the Ceph Storage Cluster.

Type: Boolean

Default: `false`

mon_osd_auto_mark_auto_out_in

Description: Ceph will mark booting Ceph OSD Daemons auto marked `out` of the Ceph Storage Cluster as `in` the cluster.

Type: Boolean

Default: `true`

mon_osd_auto_mark_new_in

Description: Ceph will mark booting new Ceph OSD Daemons as `in` the Ceph Storage Cluster.

Type: Boolean

Default: `true`

mon_osd_down_out_interval

Description: The number of seconds Ceph waits before marking a Ceph OSD Daemon `down` and `out` if it does not respond.

Type: 32-bit Integer

Default: `600`

`mon_osd_downout_subtree_limit`

Description: The largest CRUSH unit type that Ceph will automatically mark `out`.

Type: String

Default: `rack`

`mon_osd_reporter_subtree_level`

Description: This setting defines the parent CRUSH unit type for the reporting OSDs. The OSDs send failure reports to the monitor if they find an unresponsive peer. The monitor may mark the reported OSD `down` and then `out` after a grace period.

Type: String

Default: `host`

`mon_osd_report_timeout`

Description: The grace period in seconds before declaring unresponsive Ceph OSD Daemons `down`.

Type: 32-bit Integer

Default: `900`

`mon_osd_min_down_reporters`

Description: The minimum number of Ceph OSD Daemons required to report a `down` Ceph OSD Daemon.

Type: 32-bit Integer

Default: `2`

`osd_heartbeat_address`

Description: A Ceph OSD Daemon's network address for heartbeats.

Type: Address

Default: The host address.

`osd_heartbeat_interval`

Description: How often a Ceph OSD Daemon pings its peers (in seconds).

Type: 32-bit Integer

Default: `6`

`osd_heartbeat_grace`

Description: The elapsed time when a Ceph OSD Daemon has not shown a heartbeat that the Ceph Storage Cluster considers it `down`.

Type: 32-bit Integer

Default: `20`

`osd_mon_heartbeat_interval`

Description: Frequency of Ceph OSD Daemon pinging a Ceph Monitor if it has no Ceph OSD Daemon peers.

Type: 32-bit Integer

Default: `30`

`osd_mon_report_interval_max`

Description The maximum time in seconds that a Ceph OSD Daemon can wait before it must report to a Ceph Monitor.

Type: 32-bit Integer

Default: `120`

`osd_mon_report_interval_min`

Description: The minimum number of seconds a Ceph OSD Daemon may wait from startup or another reportable event before reporting to a Ceph Monitor.

Type: 32-bit Integer

Default: `5`

Valid Range: Should be less than `osd mon report interval max`

`osd_mon_ack_timeout`

Description: The number of seconds to wait for a Ceph Monitor to acknowledge a request for statistics.

Type: 32-bit Integer

Default: `30`

Debugging and logging configuration

Increase the amount of debugging and logging information in `cephadm` to help diagnose problems with Red Hat Ceph Storage.

Be sure to have a running Red Hat Ceph Storage cluster before beginning to debugging and logging information in `cephadm`.

- For more information about troubleshooting cephadm, see `cephadm` troubleshooting.
- For more information about `cephadm` logging, see `cephadm` operations.

Debugging and logging configuration options

Logging and debugging settings are not required in a Ceph configuration file, but you can override default settings as needed.

The options take a single item that is assumed to be the default for all daemons regardless of channel. For example, specifying "info" is interpreted as "default=info". However, options can also take key/value pairs. For example, `default=daemon audit=local0` is interpreted as "default all to *daemon*, override *audit* with *local0*."

`log_file`

Description: The location of the logging file for the cluster.

Type: String

Required: No

Default: `/var/log/ceph/$cluster-$name.log`

`mon_cluster_log_file`

Description The location of the monitor cluster's log file.

Type String

Required No

Default: `/var/log/ceph/$cluster.log`

`log_max_new`

Description: The maximum number of new log files.

Type: Integer

Required: No

Default: `1000`

`log_max_recent`

Description: The maximum number of recent events to include in a log file.

Type: Integer

Required: No

Default: `10000`

`log_flush_on_exit`

Description: Determines if Ceph flushes the log files after exit.

Type: Boolean

Required: No

Default: `true`

`mon_cluster_log_file_level`

Description: The level of file logging for the monitor cluster. Valid settings include "debug", "info", "sec", "warn", and "error".

Type: String

Default: `"info"`

`log_to_stderr`

Description: Determines if logging messages appear in `stderr`.

Type: Boolean

Required: No

Default: `true`

`err_to_stderr`

Description: Determines if error messages appear in `stderr`.

Type: Boolean

Required: No

Default: `true`

`log_to_syslog`

Description: Determines if logging messages appear in `syslog`.

Type: Boolean

Required: No

Default: `false`

`err_to_syslog`

Description: Determines if error messages appear in `syslog`.

Type: Boolean

Required: No

Default: `false`

`clog_to_syslog`

Description: Determines if `clog` messages will be sent to `syslog`.

Type: Boolean

Required: No

Default: `false`

`mon_cluster_log_to_syslog`

Description: Determines if the cluster log will be output to `syslog`.

Type: Boolean

Required: No

Default: `false`

`mon_cluster_log_to_syslog_level`

Description: The level of syslog logging for the monitor cluster. Valid settings include "debug", "info", "sec", "warn", and "error".

Type: String

Default: `"info"`

`mon_cluster_log_to_syslog_facility`

Description: The facility generating the syslog output. This is usually set to "daemon" for the Ceph daemons.

Type: String

Default: `"daemon"`

`clog_to_monitors`

Description: Determines if `clog` messages will be sent to monitors.

Type: Boolean

Required: No

Default: `true`

`mon_cluster_log_to_graylog`

Description: Determines if the cluster will output log messages to graylog.

Type: String

Default: `"false"`

`mon_cluster_log_to_graylog_host`

Description: The IP address of the graylog host. If the graylog host is different from the monitor host, override this setting with the appropriate IP address.

Type: String

Default: `"127.0.0.1"`

`mon_cluster_log_to_graylog_port`

Description: Graylog logs will be sent to this port. Ensure the port is open for receiving data.

Type: String

Default: `"12201"`

`osd_preserve_trimmed_log`

Description: Preserves trimmed logs after trimming.

Type: Boolean

Required: No

Default: `false`

`osd_tmapput_sets_uses_tmap`

Description: Uses `tmap`. For debug only.

Type: Boolean

Required: No

Default: `false`

`osd_min_pg_log_entries`

Description: The minimum number of log entries for placement groups.

Type: 32-bit Unsigned Integer

Required: No

Default: 1000

osd_op_log_threshold

Description: Number of op log messages to show up in one pass.

Type: Integer

Required: No

Default: 5

Scrubbing options

Ceph ensures data integrity by scrubbing placement groups.

The following are the Ceph scrubbing options that you can adjust to increase or decrease scrubbing operations.

You can set these configuration options with the **ceph config set global CONFIGURATION_OPTION VALUE** command.

mds_max_scrub_ops_in_progress

Description: The maximum number of scrub operations performed in parallel. You can set this value with **ceph config set mds_max_scrub_ops_in_progress VALUE** command.

Type: integer

Default: 5

osd_max_scrubs

Description: The maximum number of simultaneous scrub operations for a Ceph OSD Daemon.

Type: integer

Default: 1

osd_scrub_begin_hour

Description: The specific hour at which the scrubbing begins. Along with `osd_scrub_end_hour`, you can define a time window in which the scrubs can happen. Use `osd_scrub_begin_hour = 0` and `osd_scrub_end_hour = 0` to allow scrubbing the entire day.

Type: integer

Default: 0

Allowed range: `[0, 23]`

osd_scrub_end_hour

Description: The specific hour at which the scrubbing ends. Along with `osd_scrub_begin_hour`, you can define a time window, in which the scrubs can happen. Use `osd_scrub_begin_hour = 0` and `osd_scrub_end_hour = 0` to allow scrubbing for the entire day.

Type: integer

Default: 0

Allowed range: `[0, 23]`

osd_scrub_begin_week_day

Description: The specific day on which the scrubbing begins. 0 = Sunday, 1 = Monday, etc. Along with `osd_scrub_end_week_day`, you can define a time window in which scrubs can happen. Use `osd_scrub_begin_week_day = 0` and `osd_scrub_end_week_day = 0` to allow scrubbing for the entire week.

Type: integer

Default: 0

Allowed range: `[0, 6]`

osd_scrub_end_week_day

Description: This defines the day on which the scrubbing ends. 0 = Sunday, 1 = Monday, etc. Along with `osd_scrub_begin_week_day`, they define a time window, in which the scrubs can happen. Use

`osd_scrub_begin_week_day = 0` and `osd_scrub_end_week_day = 0` to allow scrubbing for the entire week.

Type: integer

Default: 0

Allowed range: `[0, 6]`

`osd_scrub_during_recovery`

Description: Allow scrub during recovery. Setting this to `false` disables scheduling new scrub, and deep-scrub, while there is an active recovery. The already running scrubs continue which is useful to reduce load on busy storage clusters.

Type: boolean

Default: false

`osd_scrub_extended_sleep`

Description: Duration to inject a delay during scrubbing out of scrubbing hours or seconds.

Type: float

Default: 0.0

`osd_scrub_backoff_ratio`

Description: Backoff ratio for scheduling scrubs. This is the percentage of ticks that do NOT schedule scrubs, 66% means that 1 out of 3 ticks schedules scrubs.

Type: float

Default: 0.66

`osd_scrub_load_threshold`

Description: The normalized maximum load. Scrubbing does not happen when the system load, as defined by `getloadavg()`/number of online CPUs, is higher than this defined number.

Type: float

Default: 0.5

`osd_scrub_min_interval`

Description: The minimal interval in seconds for scrubbing the Ceph OSD daemon when the Ceph storage Cluster load is low.

Type: float

Default: 1 day

`osd_scrub_max_interval`

Description: The maximum interval in seconds for scrubbing the Ceph OSD daemon irrespective of cluster load.

Type: float

Default: 7 days

`osd_scrub_chunk_min`

Description: The minimal number of object store chunks to scrub during a single operation. Ceph blocks writes to a single chunk during scrub.

Type: integer

Default: 5

`osd_scrub_chunk_max`

Description: The maximum number of object store chunks to scrub during a single operation.

Type: integer

Default: 25

`osd_scrub_sleep`

Description: Time to sleep before scrubbing the next group of chunks. Increasing this value slows down the overall rate of scrubbing, so that client operations are less impacted.

Type: float

Default: 0.0

`osd_deep_scrub_interval`

Description: The interval for `deep` scrubbing, fully reading all data. The `osd_scrub_load_threshold` does not affect this setting.

Type: float

Default: 7 days

`osd_debug_deep_scrub_sleep`

Description: Inject an expensive sleep during deep scrub IO to make it easier to induce preemption.

Type: float

Default: 0

`osd_scrub_interval_randomize_ratio`

Description: Add a random delay to `osd_scrub_min_interval` when scheduling the next scrub job for a placement group. The delay is a random value less than `osd_scrub_min_interval` * `osd_scrub_interval_randomized_ratio`. The default setting spreads scrubs throughout the allowed time window of `[1, 1.5]` * `osd_scrub_min_interval`.

Type: float

Default: 0.5

`osd_deep_scrub_stride`

Description: Read size when doing a deep scrub.

Type: size

Default: 512 KB

`osd_scrub_auto_repair_num_errors`

Description: Auto repair does not occur if more than this many errors are found.

Type: integer

Default: 5

`osd_scrub_auto_repair`

Description: Setting this to `true` enables automatic Placement Group (PG) repair when errors are found by scrubs or deep-scrubs. However, if more than `osd_scrub_auto_repair_num_errors` errors are found, a repair is NOT performed.

Type: boolean

Default: false

`osd_scrub_max_preemptions`

Description: Set the maximum number of times you need to preempt a deep scrub due to a client operation before blocking client IO to complete the scrub.

Type: integer

Default: 5

`osd_deep_scrub_keys`

Description: Number of keys to read from an object at a time during deep scrub.

Type: integer

Default: 1024

BlueStore configuration options

Ceph BlueStore configuration options can be configured during deployment.

"BlueStore configuration options" on page 343 describes the available BlueStore configuration options.

Table 53: BlueStore configuration options

Option	Description	Type	Default
`rocksdb_cache_size`	The size of the RocksDB cache in MB.	32-bit integer	`512`
`bluestore_throttle_bytes`	Maximum bytes available before the user throttles input or output (I/O) submission.	Size	64 MB
`bluestore_throttle_deferred_bytes`	Maximum bytes for deferred writes before the user throttles I/O submission.	Size	128 MB
`bluestore_throttle_cost_per_io`	Overhead added to the transaction cost in bytes for each I/O.	Size	0 B
`bluestore_throttle_cost_per_io_hdd`	Default `bluestore_throttle_cost_per_io` value for HDDs.	Unsigned integer	`67 000`
`bluestore_throttle_cost_per_io_ssd`	Default `bluestore_throttle_cost_per_io` value for SSDs.	Unsigned integer	`4 000`
`bluestore_debug_enforce_settings`	`hdd` enforces settings intended for BlueStore on rotational drives. `ssd` enforces settings intended for BlueStore on solid-state drives.	`default`, `hdd`, `ssd`	`default` **Note:** After changing this option, restart the OSD.

Red Hat Ceph Storage Edge clusters

Edge clusters are a solution for cost-efficient object storage configurations.

Red Hat supports the following minimum configuration of a Red Hat Ceph Storage cluster:

- A three node cluster with two replicas for SSDs.
- A four-node cluster with three replicas for HDDs.
- A four-node cluster with EC pool with 2+2 configuration.
- A four-node cluster with 8+6 CRUSH MSR configuration.

With smaller clusters, the utilization goes down because of the amount of usage and the loss of resiliency.

Pools overview

Ceph clients store data in pools. When you create pools, you create an I/O interface for clients to store data.

From a Ceph client perspective, (that is, block device, gateway, and the rest), interacting with the Ceph storage cluster is remarkably simple where you perform the following actions:

- Create a cluster handle.
- Connect the cluster handle to the cluster.
- Create an I/O context for reading and writing objects and their extended attributes.

Creating a cluster handle and connecting to the cluster

To connect to the Ceph storage cluster, the Ceph client needs the following details:

- The cluster name (which Ceph by default) - not using usually because it sounds ambiguous.
- An initial monitor address.

Ceph clients usually retrieve these parameters that use the default path for the Ceph configuration file and then read it from the file. You can also specify the parameters on the command line interface.

The Ceph client also provides a username and secret key (authentication is on by default) for the clients to contact the Ceph monitor cluster and retrieve a recent copy of the cluster map, including its monitors, OSDs, and pools.

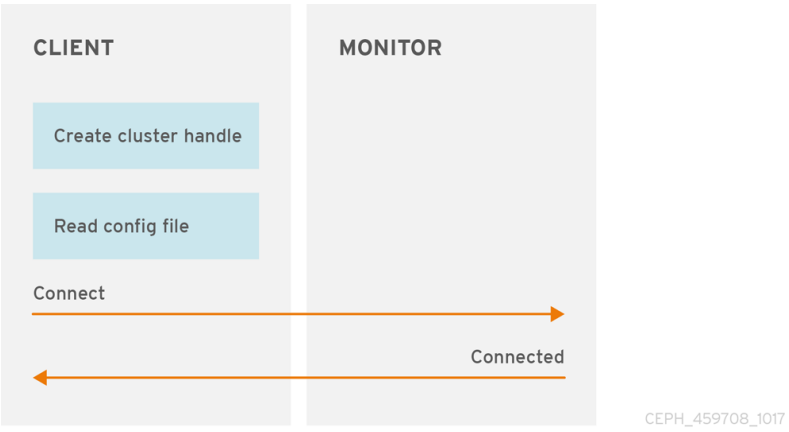


Figure 42: Create handle

Creating a pool I/O context

To read and write data, the Ceph client creates an I/O context to a specific pool in the Ceph storage cluster. If the specified user has the permission to the pool, the Ceph client can read from and write to the specified pool.

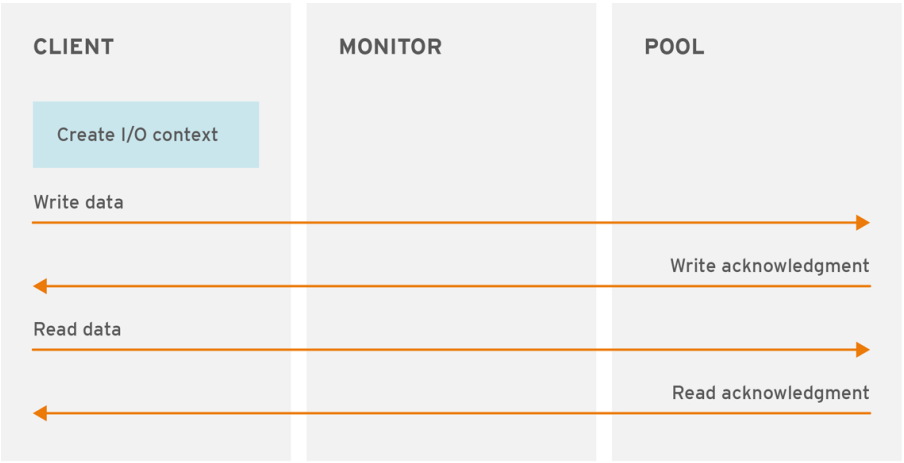


Figure 43: I/O Context

The architecture of Ceph enables the storage cluster to provide a simple interface to Ceph clients so that clients can select one of the storage strategies you define simply by specifying a pool name and creating an I/O context. Storage strategies are invisible to the Ceph client in all but capacity and performance. Similarly, the complexities of Ceph clients (mapping objects into a block device representation, providing an S3/Swift RESTful service) are invisible to the Ceph storage cluster.

A pool provides you with resilience, placement groups, CRUSH rules, and quotas.

Resilience

Set how many OSD are allowed to fail without losing data. For replicated pools, it is the wanted number of copies/replicas of an object. A typical configuration stores an object and one more copy (that is, `size = 2`), but you can determine the number of copies/replicas. For erasure coded pools, it is the number of coding chunks (that is `m=2` in the erasure code profile).

Placement groups

Set the number of placement groups for the pool. A typical configuration uses approximately 50-100 placement groups per OSD to provide optimal balancing without using up too many computing resources. When setting up multiple pools, ensure that you set a reasonable number of placement groups for both the pool and the cluster as a whole.

CRUSH rules

When you store data in a pool, a CRUSH rule mapped to the pool enables CRUSH to identify the rule for the placement of each object and its replicas or chunks for erasure coded pools in your cluster. You can create a custom CRUSH rule for your pool.

Quotas

When you set quotas on a pool with the **`ceph osd pool set-quota`** command you can limit the maximum number of objects or the maximum number of bytes stored in the specified pool.

You can list, create, and remove pools. You can also view the utilization statistics for each pool.

Resilient and non-resilient data pools

Resilient data pools enable data to replicate or encode its data. A non-resilient data pool does not enable to replicate or encode its data. Non-resilient pools are also known as *replical*.

A non-resilient pool is not protected from data loss. A small cluster with non-resilient data pools does its own replication and does not need replication from Red Hat Ceph Storage as it does its own replication.

Cluster utilization with data pools

With smaller configurations, the cluster utilization reduces because of the loss of resiliency. The recovery is limited by host utilization and might potentially impact production input/output operations per second (IOPS). When there is a single node of failure, you can add a third node to limit the host utilization and for Red Hat Ceph Storage to recover to full replication.

Erasure-coding

Ceph can load one of many erasure code algorithms. The earliest and most commonly used is the `Reed-Solomon` algorithm. An erasure code is a forward error correction (FEC) code. FEC code transforms a message of `K` chunks into a longer message that is called a code word of `N` chunks, such that Ceph can recover the original message from a subset of the `N` chunks.

More specifically, `N = K+M` where the variable `K` is the original number of data chunks. The variable `M` stands for the extra or redundant chunks that the erasure code algorithm adds to provide protection from failures. The variable `N` is the total number of chunks that are created after the erasure coding process. The value of `M` is `N-K` which means that the algorithm computes `N-K` redundant chunks from `K` original data chunks. This approach ensures that that Ceph can access all the original data. The system is resilient to arbitrary `N-K` failures. For instance, in a 10 `K` of 16 `N` configuration, or erasure coding `10/16`, the erasure code algorithm adds six extra chunks to the 10 base chunks `K`. For example, in a `M = K-N` or `16-10 = 6` configuration, Ceph will spread the 16 chunks `N` across 16 OSDs. The original file might be reconstructed from the 10 verified `N` chunks even if 6 OSDs fail—ensuring that the Red Hat Ceph Storage cluster will not lose data, and thereby ensures a high level of fault tolerance.

Like replicated pools, in an erasure-coded pool, the primary OSD in the up set receives all write operations. In replicated pools, Ceph makes a deep copy of each object in the placement group on the secondary OSDs in the set. For erasure coding, the process is a bit different. An erasure coded pool stores each object as `K+M` chunks. It is divided into `K` data chunks and `M` coding chunks. The pool is configured to have a size of `K+M` so that Ceph stores each chunk in an OSD in the acting set. Ceph stores the rank of the chunk as an attribute of the object. The primary OSD is responsible for encoding the payload into `K+M` chunks and sends them to the other OSDs. The primary OSD is also responsible for maintaining an authoritative version of the placement group logs.

For example, in a typical configuration, a system administrator creates an erasure-coded pool to use six OSDs and sustain the loss of two of them. That is, (`K+M = 6`) such that (`M = 2`).

When Ceph writes the object **NYAN** containing `ABCDEFGHIJKL` to the pool, the erasure encoding algorithm splits the content into four data chunks by simply dividing the content into four parts: `ABC`, `DEF`, `GHI`, and `JKL`. The algorithm will pad the content if the content length is not a multiple of `K`. The function also creates two coding chunks: the fifth with `YXY` and the sixth with `QGC`. Ceph stores each chunk on an OSD in the acting set with a shard_id corresponding to the acting set position, where it stores the chunks in objects that have the same name, **NYAN**, but reside on different OSDs. For example, Chunk 1 contains `ABC` and Ceph stores it on **OSD5** while chunk 5 contains `YXY` and Ceph stores it on **OSD4**.

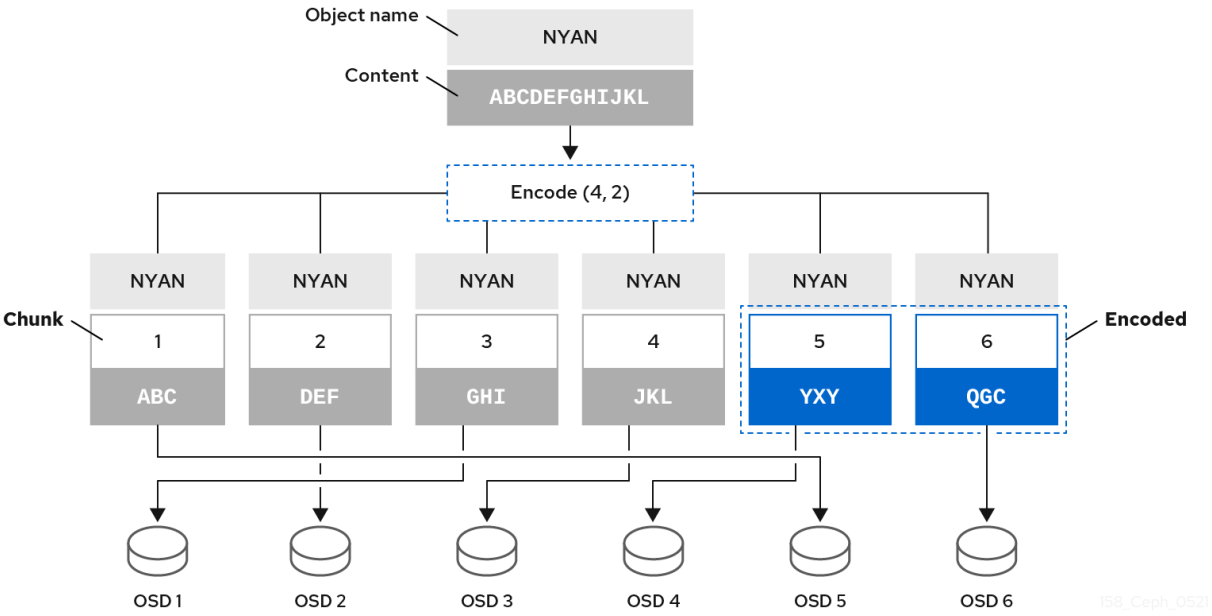


Figure 44: Ceph client object watch and notify

In a recovery scenario, the client attempts to read the object **NYAN** from the erasure-coded pool by reading chunks 1 through 6. The OSD informs the algorithm that chunks 2 and 6 are missing. For example, the primary OSD might not read chunk 6 because the **OSD6** is out, and might not read chunk 2, because **OSD2** was the slowest and its chunk was not taken into account. However, when the algorithm has four chunks, it reads the four chunks: chunk 1 containing `ABC`, chunk 3 containing `GHI`, chunk 4 containing `JKL`, and chunk 5 containing

YXY. Then, it rebuilds the original content of the object ABCDEFGHIJKL, and original content of chunk 6, which contained QGC.

Splitting data into chunks is independent from object placement. The CRUSH ruleset along with the erasure-coded pool profile determines the placement of chunks on the OSDs. For instance, that uses the Locally Repairable Code (lrc) plug-in in the erasure code profile creates extra chunks and requires fewer OSDs to recover from. For example, in an lrc profile configuration K=4 M=2 L=3, the algorithm creates six chunks (K+M), just as the jerasure plug-in would, but the locality value (L=3) requires that the algorithm create 2 more chunks locally. The algorithm creates the additional chunks as such, (K+M)/L. If the OSD containing chunk 0 fails, this chunk can be recovered by using chunks 1, 2 and the first local chunk. In this case, the algorithm only requires 3 chunks for recovery instead of 5.

Note: Using erasure-coded pools disables Object Map.

Important: For an erasure-coded pool with 2+2 configuration, replace the input string from ABCDEFGHIJKL to ABCDEF and replace the coding chunks from 4 to 2.

Multi-step-retry (MSR) is a type of CRUSH rule in the Ceph cluster that defines how data is distributed across storage devices. MSR ensures efficient data retrieval, balancing, and fault tolerance. For more information, see "Creating CRUSH rules" on page 800

Reference

- For more information about CRUSH, the erasure-coding profiles, and plug-ins, see Storage strategies.
- For more details on Object Map, see Ceph client object map.

Erasure code pools overview

Ceph uses replicated pools by default, meaning that Ceph copies every object from a primary OSD node to one or more secondary OSDs. The erasure-coded pools reduce the amount of disk space required to ensure data durability but it is computationally a bit more expensive than replication.

Ceph storage strategies involve defining data durability requirements. Data durability means the ability to sustain the loss of one or more OSDs without losing data.

Ceph stores data in pools and there are two types of the pools:

- replicated
- erasure-coded

Erasure coding is a method of storing an object in the Ceph storage cluster durably where the erasure code algorithm breaks the object into data chunks (k) and coding chunks (m), and stores those chunks in different OSDs.

In an event of the failure of an OSD, Ceph retrieves the remaining data (k) and coding (m) chunks from the other OSDs and the erasure code algorithm restores the object from those chunks.

Note: Use min_size for erasure-coded pools to be K+1 or more to prevent loss of writes and data. For a 2+2 pool, the min_size should be k+1 or 3.

Erasure coding uses storage capacity more efficiently than replication. The n-replication approach maintains n copies of an object (3 times by default in Ceph), whereas erasure coding maintains only k + m chunks. For example, 3 data and 2 coding chunks use 1.5 times the storage space of the original object.

While erasure coding uses less storage overhead than replication, the erasure code algorithm uses more RAM and CPU than replication when it accesses or recovers objects. Erasure coding is advantageous when data storage must be durable and fault tolerant, but do not require fast read performance (for example, cold storage, historical records, and so on).

For the mathematical and detailed explanation on how erasure code works in Ceph, see the Erasure coding.

Ceph creates a default erasure code profile when initializing a cluster with **k=2** and **m=2**. This means that Ceph spreads the object data over four OSDs **(k+m = 4)** and Ceph can lose up to two of those OSDs without losing data. To know more about erasure code profiling see "Erasure code profiles" on page 170 section.

> **Important:** Configure only the `.rgw.buckets` pool as erasure-coded and all other Ceph Object Gateway pools as replicated, otherwise an attempt to create a new bucket fails with the following error:
>
> `set_req_state_err err_no=95 resorting to 500`
>
> The reason for this is that erasure-coded pools do not support the `omap` operations and certain Ceph Object Gateway metadata pools require the `omap` support.

Creating a sample erasure-coded pool

Create an erasure-coded pool and specify the placement groups.

About this task

The **`ceph osd pool create`** command creates an erasure-coded pool with the default profile, unless another profile is specified. Profiles define the redundancy of data by setting two parameters, **k**, and **k**. These parameters define the number of chunks a piece of data is split and the number of coding chunks are created.

The simplest erasure-coded pool is equivalent to RAID5 and requires at least four hosts. You can create an erasure-coded pool with 2+2 profile.

Procedure

1. Set the following configuration for an erasure-coded pool on four nodes with 2+2 configuration.

   ```
   ceph config set mon mon_osd_down_out_subtree_limit host
   ```

   > **Important:** This is not needed for an erasure-coded pool in general.

   > **Note:** For an EC cluster with four nodes the value of K+M is 2+2. If a node fails completely, it does not recover as four chunks and only three nodes are available. When you set the value of **`mon_osd_down_out_subtree_limit`** to `host`, during a host down scenario, it prevents the OSDs from marked out, so as to prevent the data from re balancing and the waits until the node is up again.

2. For an erasure-coded pool with a 2+2 configuration, set the profile.

   ```
   ceph osd erasure-code-profile set ec22 k=2 m=2 crush-failure-domain=host
   ```

   ```
   [ceph: root@host01 /]# ceph osd erasure-code-profile set ec22 k=2 m=2 crush-failure-
   domain=host

   Pool : ceph osd pool create test-ec-22 erasure ec22
   ```

3. Create an erasure-coded pool.

   ```
   ceph osd pool create ecpool 32 32 erasure
   pool 'ecpool' created
   $ echo ABCDEFGHI | rados --pool ecpool put NYAN -
   $ rados --pool ecpool get NYAN -
   ABCDEFGHI
   ```

   `32` is the number of placement groups.

Bluestore server-side compression

BlueStore applies compression based on pool settings, the configured compression mode, and object characteristics. Each blob (data fragment) is evaluated individually.

How compression is applied depends on pool settings and object access patterns. Object characteristics determine which blob size BlueStore uses, not whether compression is enabled:

- If an object is immutable, append-only, or expected to have sequential reads, BlueStore uses `compression_max_blob_size`.
- If the object is expected to have random reads or writes, BlueStore uses `compression_min_blob_size`.

Benefits

Following are the benefits of using the Bluestore server-side compression feature:

- Store more data without adding physical capacity.
- Reduce raw disk usage and infrastructure costs.
- Enable compression selectively per pool to balance performance and efficiency.

Considerations

Compression introduces CPU overhead and may slightly increase read and write latency. In some cases, small overwrites on compressed objects can increase space usage because overlapping data fragments require both the old and new blobs to be retained. Enable compression only on pools or workloads where data-reduction benefits outweigh performance costs, and avoid enabling it cluster-wide without testing workload patterns.

How compression works

BlueStore applies compression based on pool settings and object characteristics. Each blob (data fragment) is evaluated individually. Compression is applied when an object is marked as immutable or append-only, not expected to have random writes or reads, and expected to have sequential reads. If these conditions are met, BlueStore uses the `compression_max_blob_size` parameter; otherwise, it uses `compression_min_blob_size`. Blob size determines how data is divided and aligned before compression. Larger blobs generally improve compression efficiency, while smaller blobs improve read performance because less data must be decompressed.

Figure 1 shows how BlueStore selects a blob size based on object properties.

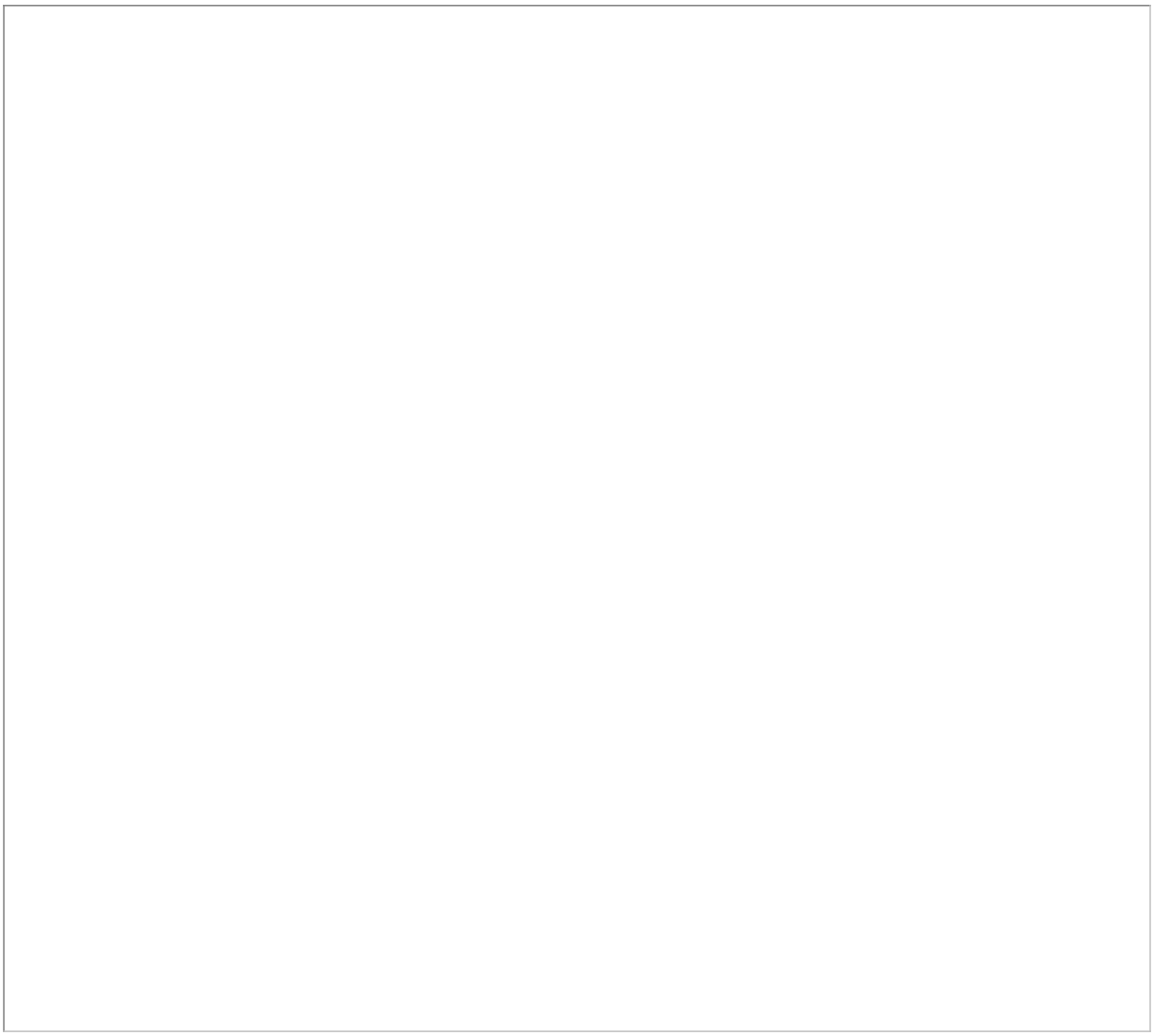


Figure 45: Bluestore compression workflow

BlueStore does not use a continuous size range; instead, it selects one of the two blob sizes based on allocation hints sent through the `rados_set_alloc_hint2` API call. Ceph clients such as RBD, CephFS, and RGW already use these hints to optimize performance.

Compression behavior

BlueStore compresses each write operation according to its configured mode, ratio threshold, and blob-size limits. In *none*mode, compression is never applied. In *passive* mode, it occurs only if the client provides a compressible hint. In *aggressive* mode, BlueStore compresses unless the client marks data as incompressible. In *force* mode, BlueStore always attempts to compress data.

The `compression_required_ratio` parameter controls the minimum acceptable ratio. For example, if set to 0.7, compression occurs only when the resulting data is 70 percent or smaller than the original. If the threshold is not met, the data is stored uncompressed. This check is performed for each blob individually to prevent unnecessary CPU usage and space amplification.

Blob size is governed by `compression_min_blob_size` and `compression_max_blob_size`, and BlueStore supports the snappy, zlib, lz4, and zstd algorithms. When configured appropriately, BlueStore balances capacity savings with performance efficiency across Red Hat Ceph Storage clusters.

Configuring BlueStore compression for a specific pool

Use these commands to configure compression settings for a specific pool. These settings override global defaults.

Procedure

1. Set the compression algorithm for the pool.

```
ceph osd pool set POOL_NAME compression_algorithm ALGORITHM
```

   For example,

```
[ceph: root@host01 /]# ceph osd pool set test_pool compression_algorithm snappy
set pool 2 compression_algorithm to snappy
```

2. Set the compression mode for the pool.

```
ceph osd pool set POOL_NAME compression_mode MODE
```

   For example,

```
[ceph: root@host01 /]# ceph osd pool set test_pool compression_mode force
set pool 2 compression_mode to force
```

3. Set the required compression ratio for the pool.

```
ceph osd pool set POOL_NAME compression_required_ratio RATIO
```

   Note: Compression occurs only when compressed data size is less than or equal to 70% of the original size.

   For example,

```
[ceph: root@host01 /]# ceph osd pool set test_pool compression_required_ratio 0.7
```

4. Set minimum and maximum blob sizes for the pool.

```
ceph osd pool set POOL_NAME compression_min_blob_size SIZE
ceph osd pool set POOL_NAME compression_max_blob_size SIZE
```

5. Verify the per-pool compression settings.

```
ceph osd pool get POOL_NAME OPTION_NAME
```

   For example,

```
[ceph: root@host01 /]# ceph osd pool get test_pool compression_algorithm
compression_algorithm: snappy
```

Configuring global BlueStore compression

Use the **ceph config set** commands to configure compression across all OSDs in the cluster.

Procedure

1. Set the global compression algorithm.

```
ceph config set osd bluestore_compression_algorithm ALGORITHM
```

   The default value is **snappy**. This parameter is optional.

2. Set the global compression mode.

```
ceph config set osd bluestore_compression_mode MODE
```

   The supported *MODE* inputs are:

   - none (default)

- `passive`
- `aggressive`
- `force`

3. Set the global compression ratio.

```
ceph config set osd bluestore_compression_required_ratio RATIO
```

The default value is `0.875`. A value of `0.7` means that compression is applied only if the compressed data is 70% or smaller than the original.

4. Set the minimum and maximum blob sizes.

```
ceph config set osd bluestore_compression_min_blob_size SIZE
ceph config set osd bluestore_compression_max_blob_size SIZE
```

The following are the default blob sizes:

SSD

65536 bytes (minimum and maximum)

HDD

8192 bytes (minimum) and 65536 bytes (maximum)

Removing global BlueStore compression

Remove global BlueStore compression settings when you want all OSDs to revert to their default values. Removing these options stops applying the overridden compression behavior for future writes; existing data is not modified.

Procedure

1. Remove the global compression algorithm.
This command overrides any set algorithms so that OSDs stop forcing a specific compression algorithm.

```
ceph config rm osd bluestore_compression_algorithm
```

2. Remove the global compression mode.
This command clears the compression mode (`none`, `passive`, `aggressive`, or `force`) and returns OSDs to the default mode.

```
ceph config rm osd bluestore_compression_mode
```

3. Remove the global required ratio.
This command removes the threshold used to decide whether compressed data should be kept.

```
ceph config rm osd bluestore_compression_required_ratio
```

4. Remove the minimum blob size.
This command removes the configured minimum blob size so that OSDs use the default value based on the device type.

```
ceph config rm osd bluestore_compression_min_blob_size
```

5. Remove the maximum blob size.
This command removes the configured maximum blob size so that OSDs use the default value based on the device type.

```
ceph config rm osd bluestore_compression_max_blob_size
```

Cluster topology and colocation

Understand the topology needed and the factors to be considers for collocation for an edge cluster.

For information on cluster topology, hyper convergence with OpenStack, collocating nodes On OpenStack, and limitations of OpenStack minimum configuration, see Ceph configuration overrides for HCI.

For more information on colocation, see Colocation.

Administering Red Hat Ceph Storage

Learn how to properly administer and operate Red Hat Ceph Storage.

Administration

Learn how to manage processes, monitor cluster states, manage users, and add and remove daemons for Red Hat Ceph Storage.

Ceph administration

A Red Hat Ceph Storage cluster is the foundation for all Ceph deployments. After deploying the cluster, there are administrative operations for keeping the cluster healthy and perform optimally.

This section helps storage administrators to perform such tasks as:

- How do I check the health of my cluster?
- How do I start and stop the services?
- How do I add or remove an OSD from a running cluster?
- How do I manage user authentication and access controls to the objects stored in the cluster?
- I want to understand how to use overrides with the cluster.
- I want to monitor the performance of the cluster.

A basic Ceph storage cluster consist of two types of daemons:

- A Ceph Object Storage Device (OSD) stores data as objects within placement groups assigned to the OSD
- A Ceph Monitor maintains a master copy of the cluster map

A production system will have three or more Ceph Monitors for high availability and typically a minimum of 50 OSDs for acceptable load balancing, data re-balancing and data recovery.

Reference

- For more information, see"Initial installation" on page 182.

Understanding process management

As a storage administrator, you can manipulate the various Ceph daemons by type or instance in a Red Hat Ceph Storage cluster. Manipulating these daemons allows you to start, stop, and restart all of the Ceph services as needed.

Process management

In Red Hat Ceph Storage, all process management is done through the systemd service. Each time you want to `start`, `restart`, and `stop` the Ceph daemons, you must specify the daemon type or the daemon instance.

For more information on using systemd, see the **Introduction to systemd** and **Managing system services with systemctl** chapters within the *Configuring basic system settings* guide within the Product Documentation for Red Hat Enterprise Linux for your OS version, on the Managing system services with systemctl.

Starting, stopping, and restarting all Ceph services

You can start, stop, and restart all Ceph services from the host where you want to manage the Ceph services.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Have root-node access
- Log into the `cephadm` shell.
  For example,

```
[root@host01 ~]# cephadm shell
```

About this task

Ceph services are logical groups of Ceph daemons of the same type, configured to run in the same Red Hat Ceph Storage cluster. The orchestration layer in Ceph allows the user to manage these services in a centralized way, making it easy to run operations that affect all the Ceph daemons that belong to the same logical service. The Ceph daemons running in each host are managed through the Systemd service.

Important: To start, stop, or restart a specific Ceph daemon in a specific host, you need to use the SystemD service. To obtain a list of the SystemD services running in a specific host, connect to the host, and run the following command:

```
[root@host01 ~]# systemctl list-units “ceph*”
```

The output provides a list of the service names that you can use to manage each Ceph daemon.

Get a list of Ceph services

Procedure

- Get a list of Ceph services configured in the Red Hat Ceph Storage cluster and to get the specific service ID.

```
ceph orch ls
```

For example,

```
[ceph: root@host01 /]# ceph orch ls
NAME                       RUNNING  REFRESHED  AGE  PLACEMENT  IMAGE NAME
                                                 IMAGE ID
alertmanager                   1/1  4m ago     4M   count:1    registry.redhat.io/
openshift4/ose-prometheus-alertmanager:v4.15   b7bae610cd46
crash                          3/3  4m ago     4M   *          registry.redhat.io/
rhceph-alpha/rhceph-9-rhel9:latest             c88a5d60f510
grafana                        1/1  4m ago     4M   count:1    registry.redhat.io/
rhceph-alpha/rhceph-9-dashboard-rhel9:latest   bd3d7748747b
mgr                            2/2  4m ago     4M   count:2    registry.redhat.io/
rhceph-alpha/rhceph-9-rhel9:latest             c88a5d60f510
mon                            2/2  4m ago     10w  count:2    registry.redhat.io/
rhceph-alpha/rhceph-9-rhel9:latest             c88a5d60f510
nfs.foo                        0/1  -          -    count:1    <unknown>
                                                 <unknown>
node-exporter                  1/3  4m ago     4M   *          registry.redhat.io/
openshift4/ose-prometheus-node-exporter:v4.5   mix
osd.all-available-devices      5/5  4m ago     3M   *          registry.redhat.io/
rhceph-alpha/rhceph-9-rhel9:latest             c88a5d60f510
prometheus                     1/1  4m ago     4M   count:1    registry.redhat.io/
openshift4/ose-prometheus:v4.15                bebb0ddef7f0
rgw.test_realm.test_zone       2/2  4m ago     3M   count:2    registry.redhat.io/
rhceph-alpha/rhceph-9-rhel9:latest             c88a5d60f510
```

Start a service

Procedure

- Start a specific service by running the **`ceph orch start SERVICE_ID`** command.
  For example,

```
[ceph: root@host01 /]# ceph orch start node-exporter
```

Stop a service

Procedure

- Stop a specific service by running the following command.

> **Important:** The **`ceph orch stop SERVICE_ID`** command results in the Red Hat Ceph Storage cluster being inaccessible, only for the MON and MGR service. It is recommended to use the **`systemctl stop SERVICE_ID`** command to stop a specific daemon in the host.
> For more information, see Starting, stopping, and restarting all Ceph daemons using the systemctl command.

```
ceph orch stop SERVICE_ID
```

In the following example the **`ceph orch stop node-exporter`** command removes all the daemons of the `node exporter` service.

```
[ceph: root@host01 /]# ceph orch stop node-exporter
```

Restart a service

Procedure

- Restart a specific service by running the following command.

```
ceph orch restart SERVICE_ID
```

For example,

```
[ceph: root@host01 /]# ceph orch restart node-exporter
```

Viewing log files of Ceph daemons

Use the `journald` daemon from the container host to view a log file of a Ceph daemon from a container.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

1. To view the entire Ceph log file, run a `journalctl` command as `root` composed in the following format:

   Syntax

   ```
   journalctl -u ceph SERVICE_ID
   ```

   Example

   ```
   [root@host01 ~]# journalctl -u ceph-499829b4-832f-11eb-8d6d-001a4a000635@osd.8.service
   ```

   In the example, you can view the entire log for the OSD with ID `osd.8`.

2. To show only the recent journal entries, use the `-f` option.

   Syntax

   ```
   journalctl -fu SERVICE_ID
   ```

Example

```
[root@host01 ~]# journalctl -fu
ceph-499829b4-832f-11eb-8d6d-001a4a000635@osd.8.service
```

Note: The `sosreport` utility can be used to view the `journald` logs. For more details about SOS reports, see the What is an sosreport and how to create one in Red Hat Enterprise Linux? solution on the Red Hat Customer Portal.

Reference

- The `journalctl` manual page.

Powering down and rebooting the cluster

Use either the Ceph Orchestrator or `systemctl` commands to power down and restart the Red Hat Ceph Storage cluster.

Before you begin powering down Red Hat Ceph Storage, be sure that you have root-level access.

Powering down and rebooting with Ceph Orchestrator

Use the Ceph Orchestrator to shut down and restart the Red Hat Ceph Storage cluster. In most cases, logging in to a single system is sufficient to power off the cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

About this task

The Ceph Orchestrator supports `start`, `stop`, and `restart` operations for powering down or rebooting the cluster.

Alternatively, use `systemctl` commands for powering down and rebooting the Ceph cluster. For more information, see “Powering down and rebooting by using systemctl commands” on page 360.

Use the following basic flow to power down an Red Hat Ceph Storage cluster. The procedure provides full in-depth information and commands for each step.

1. Stop all clients from using the Block Device image and Ceph Object Gateway on this cluster and on any other clients.
2. Stop the Ceph File System (CephFS) cluster and the MDS service.
3. Stop all component gateways related to block and file. These include: NFS, SMB, and NVMe-oF gateways.
4. Stop all ingress services and RADOS gateways.
5. If mirror daemons are used, stop all mirror daemons.
6. Stop the monitoring service components.
7. Set the **noout** flag.
8. Shut down all nodes.

Powering down the Red Hat Ceph Storage cluster

Procedure

1. Stop all clients from using user Block Device images, CephFS volumes, and Ceph Object Gateways.
2. Log in to the `cephadm` shell.

For example,

```
[root@host01 ~]# cephadm shell
```

Before proceeding, the cluster must be in healthy state (`Health_OK` and all PGs `active+clean`). View the cluster state by using the **ceph -s** command.

3. When using CephFS, bring down the CephFS cluster.

```
ceph fs fail FS_NAME
ceph status
ceph fs set FS_NAME joinable false
ceph mds fail FS_NAME:N
```

For example,

```
[ceph: root@host01 /]# ceph fs fail cephfs
[ceph: root@host01 /]# ceph status
[ceph: root@host01 /]# ceph fs set cephfs joinable false
[ceph: root@host01 /]# ceph mds fail cephfs:1
```

4. Stop the MDS service.
   a. Get the MDS service name.

```
ceph orch ls --service-type mds
```

For example,

```
[ceph: root@host01 /]# ceph orch ls --service-type mds
```

   b. Stop the MDS service that uses the name output in the previous step.

```
ceph orch stop SERVICE_NAME
```

5. Stop all component gateways that are related to block and file systems.
Component gateways that must be stopped are NFS, SMB, and NVMe-oF gateways.
   a. Find all relevant services that are running by using the **ceph orch ls** command.
   b. Stop each block and file system service.

```
ceph orch stop SERVICE_NAME
```

For example,

```
[ceph: root@host01 /]# ceph orch stop nvmeof
```

6. Stop the Ceph Object Gateway services.
Repeat for each deployed service.
   a. Find the Ceph Object Gateway service name.

```
ceph orch ls --service-type SERVICE_TYPE
```

   b. Stop the Ceph Object Gateway service that uses the fetched service name.

```
ceph orch stop SERVICE_NAME
```

7. Stop the Ceph Object Gateway ingress services.
Repeat for each deployed service.
   a. Fetch the deployed ingress services.

```
ceph orch ls --service-type ingress
```

   b. Stop each deployed ingress service.

```
ceph orch stop SERVICE_NAME
```

8. Stop the monitoring service components.
    a. Stop the Alertmanager service.

    ```
    ceph orch stop alertmanager
    ```

    b. Stop the node-exporter service, which is part of the monitoring stack.

    ```
    ceph orch stop node-exporter
    ```

    c. Stop the Prometheus service.

    ```
    ceph orch stop prometheus
    ```

    d. Stop the Grafana dashboard service.

    ```
    ceph orch stop grafana
    ```

    e. Stop the crash service.

    ```
    ceph orch stop crash
    ```

9. Set the **noout** flag.
Set the **noout** flag during maintenance to prevent CRUSH from automatically rebalancing the cluster.

    > **Note:** Use the **pause** flag to block any forgotten client I/Os.

    ```
    ceph osd set noout
    ```

    For example,

    ```
    [ceph: root@host01 /]# ceph osd set noout
    ```

10. Stop the OSD services from the `cephadm` node, one by one.
If there are large number of OSD daemons, you may proceed to shutdown the OSD nodes directly.
    a. Fetch the OSD IDs.

    ```
    ceph orch ps
    ```

    b. Stop the OSD daemons.

    ```
    ceph orch daemon stop OSD_ID
    ```

    For example,

    ```
    [ceph: root@host01 /]# ceph orch daemon stop osd.1
    Scheduled to stop osd.1 on host 'host02'
    ```

    c. Identify the OSD hosts.

    ```
    ceph orch ls osd
    ```

    d. Shut down each of the OSD hosts from the hosts listed in step “10.a” on page 359.

11. Stop the monitors.
    a. Identify the hosts with monitors.

    ```
    ceph orch ls mon
    ```

    Note the listed monitor hosts.

    b. Shut down the monitor hosts that are listed in the previous step.

Shut down the monitor hosts one by one.

```
systemctl stop SERVICE_NAME
```

12. Shut down any remaining standalone hosts.
    Shut down hosts with services, such as Ceph Object Gateway and NVMe-oF gateways that are still running.

Rebooting the Red Hat Ceph Storage cluster

Before you begin

Before you begin rebooting the cluster, power on and stabilize any network equipment before powering on the Ceph hosts and nodes.

Procedure

1. Power on all the Ceph hosts.
2. Log in to the administration node from the `cephadm` shell.
   For example,

   ```
   [root@host01 ~]# cephadm shell
   ```

3. Verify that all services are in the `running` state.

   ```
   ceph orch ls
   ```

4. Ensure that the cluster health is in the `Health_OK` status, by running the **`ceph -s`** command.
5. Unset the **`noout`** flag that was set in “Powering down the Red Hat Ceph Storage cluster” on page 357.

   ```
   ceph osd unset noout
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph osd unset noout
   ```

6. When using Ceph File System (CephFS), make the CephFS cluster available by setting the **`joinable`** flag to `true`.

   ```
   ceph fs set FS_NAME joinable true
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph fs set cephfs joinable true
   ```

What to do next

Verify that the cluster is in healthy state (`Health_OK` and all PGs `active+clean`). Run the **`ceph -s`** status command on a node with the client keyrings, for example, the Ceph Monitor or OpenStack controller nodes, to ensure that the cluster is healthy.

Powering down and rebooting by using `systemctl` commands

Use `systemctl` commands to shut down and restart the Red Hat Ceph Storage cluster. This method aligns with standard Linux service management practices.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

About this task

Alternatively, use the Ceph Orchestrator to power down and reboot the Ceph cluster. For more information, see "Powering down and rebooting with Ceph Orchestrator" on page 357.

Powering down the Red Hat Ceph Storage cluster

About this task

Repeat these steps for each node in the cluster.

Procedure

1. Stop all clients from using user Block Device Images, CephFS volumes, and Ceph Object Gateways.
2. Log in to the `cephadm` shell.
   For example,

   ```
   [root@host01 ~]# cephadm shell
   ```

   Before proceeding, the cluster must be in healthy state (`Health_OK` and all PGs `active+clean`). View the cluster state by using the **ceph -s** command.
3. When using CephFS, bring down the CephFS cluster.

   ```
   ceph fs fail FS_NAME
    ceph status
    ceph fs set FS_NAME joinable false
    ceph mds fail FS_NAME:N
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph fs fail cephfs
   [ceph: root@host01 /]# ceph status
   [ceph: root@host01 /]# ceph fs set cephfs joinable false
   [ceph: root@host01 /]# ceph mds fail cephfs:1
   ```

4. If the MDS and Ceph Object Gateway nodes are on their own dedicated nodes, stop the MDS service.
   a. Get the MDS service name.

      ```
      ceph orch ls --service-type mds
      ```

      For example,

      ```
      [ceph: root@host01 /]# ceph orch ls --service-type mds
      ```

   b. Stop the MDS service that uses the name output in the previous step.

      ```
      ceph orch stop SERVICE_NAME
      ```

5. Set the **noout** flag.
   Set the **noout** flag during maintenance to prevent CRUSH from automatically rebalancing the cluster.

   > **Note:** Use the **pause** flag to block any forgotten client I/Os.

   ```
   ceph osd set noout
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph osd set noout
   ```

6. Get the systemd target of the daemons.

   ```
   systemctl list-units --type target | grep ceph
   ```

   For example,

```
[root@host01 ~]# systemctl list-units --type target | grep ceph
ceph-0b007564-ec48-11ee-b736-525400fd02f8.target loaded active active Ceph cluster
0b007564-ec48-11ee-b736-525400fd02f8
ceph.target                                       loaded active active All Ceph
clusters and services
```

7. Disable the target that includes the cluster FSID.

```
systemctl disable CLUSTER_FSID.target
```

For example,

```
[root@host01 ~]# systemctl disable ceph-0b007564-ec48-11ee-b736-525400fd02f8.target
Removed "/etc/systemd/system/multi-user.target.wants/ceph-0b007564-ec48-11ee-
b736-525400fd02f8.target".
Removed "/etc/systemd/system/ceph.target.wants/ceph-0b007564-ec48-11ee-
b736-525400fd02f8.target".
```

8. Stop all daemons on the host.

```
systemctl stop CLUSTER_FSID.target
```

For example,

```
[root@host01 ~]# systemctl stop ceph-0b007564-ec48-11ee-b736-525400fd02f8.target
```

9. Shut down the node by using the **shutdown** command.
For example,

```
[root@host01 ~]# shutdown
Shutdown scheduled for Wed 2024-03-27 11:47:19 EDT, use 'shutdown -c' to cancel.
```

What to do next

Repeat the shutdown steps for all nodes in the cluster.

Rebooting the Red Hat Ceph Storage cluster

Procedure

1. Enable the systemd target to run all daemons.

```
systemctl enable CLUSTER_FSID.target
```

For example,

```
[root@host01 ~]# systemctl enable ceph-0b007564-ec48-11ee-b736-525400fd02f8.target
Created symlink /etc/systemd/system/multi-user.target.wants/ceph-0b007564-ec48-11ee-
b736-525400fd02f8.target → /etc/systemd/system/ceph-0b007564-ec48-11ee-
b736-525400fd02f8.target.
Created symlink /etc/systemd/system/ceph.target.wants/ceph-0b007564-ec48-11ee-
b736-525400fd02f8.target → /etc/systemd/system/ceph-0b007564-ec48-11ee-
b736-525400fd02f8.target.
```

2. Start the systemd target.

```
systemctl start CLUSTER_FSID.target
```

For example,

```
[root@host01 ~]# systemctl start ceph-0b007564-ec48-11ee-b736-525400fd02f8.target
```

Wait for all the nodes to come up.

3. Log in to the `cephadm` shell.

```
cephadm shell
```

4. Verify that all services are up and there are no connectivity issues between the nodes.

```
ceph orch ls
```

5. Unset the **noout** flags.
   Run the following commands on a node with the client keyrings, for example, the Ceph Monitor or OpenStack controller node.

```
ceph osd unset noout
```

> **Note:** If the **pause** flag was set, unset the pause flag.

   For example,

```
ceph: root@host01 /]# ceph osd unset noout
```

6. When using Ceph File System (CephFS), bring the CephFS cluster back up by setting the `joinable` flag to `true`.

```
ceph fs set FS_NAME joinable true
```

   For example,

```
[ceph: root@host01 /]# ceph fs set cephfs joinable true
```

What to do next

Verify that the cluster is in healthy state (`Health_OK` and all PGs `active+clean`). Run the **ceph -s** status command on a node with the client keyrings, for example, the Ceph Monitor or OpenStack controller nodes, to ensure that the cluster is healthy.

Powering down and rebooting by using `cephadm` cluster commands

Power down and start the Red Hat Ceph Storage cluster by using automated `cephadm` cluster shutdown and startup commands.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

1. Ensure that the cluster is in a healthy state (`HEALTH_OK`) and that all placement groups are `active+clean`.
2. Ensure that all clients are stopped
   - Unmount Ceph File System (CephFS).
   - Disconnect Ceph Block Device (RBD) clients.
   - Stop Ceph Object Gateway (RGW) clients.
3. Log in to the administration host with root-level access.
4. If `cephadm` cannot automatically determine the cluster FSID, specify it by using the **--fsid** option.

> **Important:** If your cluster has RGW multisite configured, complete the following requirements before you start the shutdown procedure:
>
> - Ensure that no client I/O operations are in progress on any site.
> - Power down and power on sites one at a time. Do not shut down all sites simultaneously.

- Always shut down the secondary site first and start it up first. Shut down the primary site last and start it up last.

About this task

Use `cephadm` commands to perform a coordinated shutdown and startup of all cluster services and hosts from the administration host.

Important: Use the `--force` option only at the explicit direction of IBM Support. Using this option on an unhealthy cluster can result in data loss.

If you prefer manual control, see the procedures for powering down and rebooting by using Ceph Orchestrator or systemctl commands. For more information, see "Powering down and rebooting with Ceph Orchestrator" on page 357 and "Powering down and rebooting by using systemctl commands" on page 360.

Procedure

1. Preview the shutdown operation.

   ```
   cephadm cluster-shutdown --dry-run
   ```

2. Shut down the cluster.

   ```
   cephadm cluster-shutdown --yes-i-really-mean-it
   ```

   The system performs a coordinated shutdown of cluster services and hosts.

3. If RGW multisite is configured, power down each site separately.
   Perform the following substeps on the secondary site first, then repeat them on the primary site.

   a. Check the cluster health status.

      ```
      ceph -s
      ```

      For example,

      ```
      [ceph: root@host01 /]# ceph -s
      ```

   b. Set the OSD cluster flags to prevent data movement and I/O during shutdown.

      ```
      ceph osd set noout
      ceph osd set norecover
      ceph osd set norebalance
      ceph osd set nobackfill
      ceph osd set nodown
      ceph osd set pause
      ```

      For example,

      ```
      [ceph: root@host01 /]# ceph osd set noout
      [ceph: root@host01 /]# ceph osd set norecover
      [ceph: root@host01 /]# ceph osd set norebalance
      [ceph: root@host01 /]# ceph osd set nobackfill
      [ceph: root@host01 /]# ceph osd set nodown
      [ceph: root@host01 /]# ceph osd set pause
      ```

   c. Shut down the RGW daemons, then the OSD daemons, and then the MON and MGR daemons.

      ```
      cephadm cluster-shutdown --yes-i-really-mean-it
      ```

      The automated shutdown command stops daemons in the correct order. If you are performing a manual shutdown, stop daemons in the following order: RGW → OSD → MON/MGR.

4. Start the cluster.

```
cephadm cluster-start [--parallel MAX_NUMBER]
```

Use the --parallel option to set the maximum number of host clusters to start in parallel. If not set, the default number is 5.

```
[ceph: root@host01 /]# cephadm cluster-start --parallel 10
```

5. If RGW multisite is configured, power on each site separately.
Perform the following substeps on the secondary site first, then repeat them on the primary site.
   a. Start the cluster. MON and MGR daemons start first, followed by OSD daemons, and then RGW daemons.

      ```
      cephadm cluster-start
      ```

      For example,

      ```
      [ceph: root@host01 /]# cephadm cluster-start
      ```

   b. Unset the OSD cluster flags that were set before shutdown.

      ```
      ceph osd unset noout
      ceph osd unset norecover
      ceph osd unset norebalance
      ceph osd unset nobackfill
      ceph osd unset nodown
      ceph osd unset pause
      ```

      For example,

      ```
      [ceph: root@host01 /]# ceph osd unset noout
      [ceph: root@host01 /]# ceph osd unset norecover
      [ceph: root@host01 /]# ceph osd unset norebalance
      [ceph: root@host01 /]# ceph osd unset nobackfill
      [ceph: root@host01 /]# ceph osd unset nodown
      [ceph: root@host01 /]# ceph osd unset pause
      ```

   c. Verify that the cluster has returned to a healthy state.

      ```
      ceph -s
      ```

      For example,

      ```
      [ceph: root@host01 /]# ceph -s
      ```

      The cluster status shows HEALTH_OK.

6. Check the cluster status.

   ```
   cephadm cluster-status
   ```

Result

The cluster is shut down and restarted by using an automated workflow. Verify that the cluster returns to a healthy state.

- The shutdown process creates a state file:

  ```
  /var/lib/ceph/<fsid>/cluster-shutdown-state.json
  ```

- `cephadm` temporarily stores an SSH private key on the administration host to enable startup operations. The key is removed after startup completes.

If RGW multisite is configured, verify that multisite replication has resumed after the cluster is up on all sites.

1. Check the RGW sync status on each site.

   ```
   radosgw-admin sync status
   ```

Confirm that the output shows an active sync relationship with the peer site and that no sync errors are reported.

2. Write a test object on one site and verify that it replicates to the peer site.

```
radosgw-admin bucket sync status --bucket BUCKET_NAME
```

> **Note:** Sync replication can take up to 20 minutes to complete due to the `rgw_sync_lease_period` setting. If objects do not appear on the peer site after this period, check the sync status output for errors.

Related information

Troubleshooting powering down and starting the cluster

Monitor Red Hat Ceph Storage clusters

You can monitor the health, performance, and status of Red Hat Ceph Storage clusters by using the Ceph Dashboard or the command-line interface (CLI). These tools provide visibility into cluster health, capacity usage, and component status at different levels of detail.

Monitor a Ceph cluster with the CLI

As a storage administrator, you can monitor the overall health of the Red Hat Ceph Storage cluster, along with monitoring the health of the individual components of Ceph.

Once you have a running cluster, you might begin monitoring the storage cluster to ensure that the Ceph Monitor and Ceph OSD daemons are running, at a high-level. Ceph storage cluster clients connect to a Ceph Monitor and receive the latest version of the storage cluster map before they can read and write data to the Ceph pools within the storage cluster. So the monitor cluster must have agreement on the state of the cluster before Ceph clients can read and write data.

Ceph OSDs must peer the placement groups on the primary OSD with the copies of the placement groups on secondary OSDs. If faults arise, peering will reflect something other than the `active + clean` state.

High-level monitoring

As a storage administrator, you can monitor the health of the Ceph daemons to ensure that they are up and running.

High level monitoring also involves checking the storage cluster capacity to ensure that the storage cluster does not exceed its `full ratio`. The Red Hat Ceph Storage Dashboard is the most common way to conduct high-level monitoring. However, you can also use the command-line interface, the Ceph admin socket or the Ceph API to monitor the storage cluster.

Checking storage cluster health

After you start the Ceph storage cluster, and before you start reading or writing data, check the storage cluster's health first.

Prerequisites

- A running cluster.
- Root-level access to the node.

Procedure

1. Log into the `cephadm` shell:

   Example

```
[root@host01 ~]# cephadm shell
```

2. You can check on the health of the Ceph storage cluster with the following command:

Example

```
[ceph: root@host01 /]# ceph health
HEALTH_OK
```

3. You can check the status of the Ceph storage cluster by running `ceph status` command:

   Example

   ```
   [ceph: root@host01 /]# ceph status
   ```

   The output provides the following information:

   - Cluster ID
   - Cluster health status
   - The monitor map epoch and the status of the monitor quorum.
   - The OSD map epoch and the status of OSDs.
   - The status of Ceph Managers.
   - The status of Ceph Object Gateways.
   - The placement group map version.
   - The number of placement groups and pools.
   - The notional amount of data stored and the number of objects stored.
   - The total amount of data stored.
   - The IO client operations.
   - An update on the upgrade process if the cluster is upgrading.

     Upon starting the Ceph cluster, you will likely encounter a health warning such as `HEALTH_WARN XXX num placement groups stale`. Wait a few moments and check it again. When the storage cluster is ready, `ceph health` should return a message such as `HEALTH_OK`. At that point, it is okay to begin using the cluster.

Watching storage cluster events

You can watch events that are happening with the Ceph storage cluster using the command-line interface.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   root@host01 ~]# cephadm shell
   ```

2. To watch the cluster's ongoing events, run the following command:

   Example

   ```
   [ceph: root@host01 /]# ceph -w
     cluster:
       id:     8c9b0072-67ca-11eb-af06-001a4a0002a0
       health: HEALTH_OK

     services:
       mon: 2 daemons, quorum Ceph5-2,Ceph5-adm (age 3d)
       mgr: Ceph5-1.nqikfh(active, since 3w), standbys: Ceph5-adm.meckej
       osd: 5 osds: 5 up (since 2d), 5 in (since 8w)
       rgw: 2 daemons active (test_realm.test_zone.Ceph5-2.bfdwcn,
   test_realm.test_zone.Ceph5-adm.acndrh)
   ```

```
  data:
    pools:   11 pools, 273 pgs
    objects: 459 objects, 32 KiB
    usage:   2.6 GiB used, 72 GiB / 75 GiB avail
    pgs:     273 active+clean

  io:
    client:   170 B/s rd, 730 KiB/s wr, 0 op/s rd, 729 op/s wr

2022-11-02 15:45:21.655871 osd.0 [INF] 17.71 deep-scrub ok
2022-11-02 15:45:47.880608 osd.1 [INF] 1.0 scrub ok
2022-11-02 15:45:48.865375 osd.1 [INF] 1.3 scrub ok
2022-11-02 15:45:50.866479 osd.1 [INF] 1.4 scrub ok
2022-11-02 15:45:01.345821 mon.0 [INF] pgmap v41339: 952 pgs: 952 active+clean; 17130
MB data, 115 GB used, 167 GB / 297 GB avail
2022-11-02 15:45:05.718640 mon.0 [INF] pgmap v41340: 952 pgs: 1
active+clean+scrubbing+deep, 951 active+clean; 17130 MB data, 115 GB used, 167 GB /
297 GB avail
2022-11-02 15:45:53.997726 osd.1 [INF] 1.5 scrub ok
2022-11-02 15:45:06.734270 mon.0 [INF] pgmap v41341: 952 pgs: 1
active+clean+scrubbing+deep, 951 active+clean; 17130 MB data, 115 GB used, 167 GB /
297 GB avail
2022-11-02 15:45:15.722456 mon.0 [INF] pgmap v41342: 952 pgs: 952 active+clean; 17130
MB data, 115 GB used, 167 GB / 297 GB avail
2022-11-02 15:46:06.836430 osd.0 [INF] 17.75 deep-scrub ok
2022-11-02 15:45:55.720929 mon.0 [INF] pgmap v41343: 952 pgs: 1
active+clean+scrubbing+deep, 951 active+clean; 17130 MB data, 115 GB used, 167 GB /
297 GB avail
```

How Ceph calculates data usage

The `used` value reflects the *actual* amount of raw storage used. The `xxx GB / xxx GB` value means the amount available, the lesser of the two numbers, of the overall storage capacity of the cluster.

The notional number reflects the size of the stored data before it is replicated, cloned or snapshotted. Therefore, the amount of data actually stored typically exceeds the notional amount stored, because Ceph creates replicas of the data and may also use storage capacity for cloning and snapshotting.

Understanding storage clusters usage stats

To check a cluster's data usage and data distribution among pools, use the `df` option. It is similar to the Linux **df** command. You can run either the **ceph df** command or **ceph df detail** command.

The `SIZE`/`AVAIL`/`RAW USED` in the `ceph df` and **ceph status** command output are different if some OSDs are marked `OUT` of the cluster compared to when all OSDs are `IN`. The `SIZE`/`AVAIL`/`RAW USED` is calculated from sum of `SIZE` (osd disk size), `RAW USE` (total used space on disk), and `AVAIL` of all OSDs which are in `IN` state. You can see the total of `SIZE`/`AVAIL`/`RAW USED` for all OSDs in **ceph osd df tree** command output.

Example

```
[ceph: root@host01 /]#ceph df
--- RAW STORAGE ---
CLASS   SIZE    AVAIL     USED  RAW USED  %RAW USED
hdd    5 TiB  2.9 TiB  2.1 TiB   2.1 TiB      42.98

TOTAL  5 TiB  2.9 TiB  2.1 TiB   2.1 TiB      42.98


--- POOLS ---
POOL                        ID  PGS   STORED  OBJECTS     USED  %USED  MAX AVAIL
.mgr                         1    1  5.3 MiB        3   16 MiB      0    629 GiB
.rgw.root                    2   32  1.3 KiB        4   48 KiB      0    629 GiB
default.rgw.log              3   32  3.6 KiB      209  408 KiB      0    629 GiB
default.rgw.control          4   32      0 B        8      0 B      0    629 GiB
default.rgw.meta             5   32  1.7 KiB       10   96 KiB      0    629 GiB
default.rgw.buckets.index    7   32  5.5 MiB       22   17 MiB      0    629 GiB
default.rgw.buckets.data     8   32  807 KiB        3  2.4 MiB      0    629 GiB
default.rgw.buckets.non-ec   9   32  1.0 MiB        1  3.1 MiB      0    629 GiB
source-ecpool-86            11   32  1.2 TiB  391.13k  2.1 TiB  53.49    1.1 TiB
```

The `ceph df detail` command gives more details about other pool statistics such as quota objects, quota bytes, used compression, and under compression.

The `RAW STORAGE` section of the output provides an overview of the amount of storage the storage cluster manages for data.

- `CLASS`: The class of OSD device.
- `SIZE`: The amount of storage capacity managed by the storage cluster.

  In the example, if the `SIZE` is 90 GiB, it is the total size without the replication factor, which is three by default. The total available capacity with the replication factor is 90 GiB/3 = 30 GiB. Based on the full ratio, which is 0.85% by default, the maximum available space is 30 GiB * 0.85 = 25.5 GiB
- `AVAIL`: The amount of free space available in the storage cluster.

  In the example, if the `SIZE` is 90 GiB and the `USED` space is 6 GiB, then the `AVAIL` space is 84 GiB. The total available space with the replication factor, which is three by default, is 84 GiB/3 = 28 GiB
- `USED`: The amount of raw storage consumed by user data.

  In the example, 100 MiB is the total space available after considering the replication factor. The actual available size is 33 MiB.
- `RAW USED`: The amount of raw storage consumed by user data, internal overhead, or reserved capacity.
- `% RAW USED`: The percentage of **RAW USED**. Use this number in conjunction with the `full ratio` and `near full ratio` to ensure that you are not reaching the storage cluster's capacity.

The `POOLS` section of the output provides a list of pools and the notional usage of each pool. The output from this section **DOES NOT** reflect replicas, clones or snapshots. For example, if you store an object with 1 MB of data, the notional usage will be 1 MB, but the actual usage may be 3 MB or more depending on the number of replicas for example, `size = 3`, clones and snapshots.

- **POOL:** The name of the pool.
- **ID:** The pool ID.
- **STORED:** The actual amount of data stored by the user in the pool. This value changes based on the raw usage data based on (k+M)/K values, number of object copies, and the number of objects degraded at the time of pool stats calculation.
- **OBJECTS:** The notional number of objects stored per pool. It is `STORED` size * replication factor.
- **USED:** The notional amount of data stored in kilobytes, unless the number appends **M** for megabytes or **G** for gigabytes.
- **%USED:** The notional percentage of storage used per pool.
- **MAX AVAIL:** An estimate of the notional amount of data that can be written to this pool. It is the amount of data that can be used before the first OSD becomes full. It considers the projected distribution of data across disks from the CRUSH map and uses the first OSD to fill up as the target.

  In the example, `MAX AVAIL` is 153.85 MB without considering the replication factor, which is three by default.

  See the Knowledgebase article titled *`ceph df MAX AVAIL` is incorrect for simple replicated pool* to calculate the value of `MAX AVAIL`.
- **QUOTA OBJECTS:** The number of quota objects.
- **QUOTA BYTES:** The number of bytes in the quota objects.
- **USED COMPR:** The amount of space allocated for compressed data including his includes compressed data, allocation, replication and erasure coding overhead.
- **UNDER COMPR:** The amount of data passed through compression and beneficial enough to be stored in a compressed form.

Note:

- The numbers in the **POOLS** section are notional. They are not inclusive of the number of replicas, snapshots or clones. As a result, the sum of the **USED** and **%USED** amounts will not add up to the **RAW USED** and **%RAW USED** amounts in the **GLOBAL** section of the output.

- The **MAX AVAIL** value is a complicated function of the replication or erasure code used, the CRUSH rule that maps storage to devices, the utilization of those devices, and the configured `mon_osd_full_ratio`.
- For more information, see "How Ceph calculates data usage" on page 368 and "Understanding OSD usage stats" on page 372.

Checking Ceph Monitor status

If the storage cluster has multiple Ceph Monitors, which is a requirement for a production Red Hat Ceph Storage cluster, then you can check the Ceph Monitor quorum status after starting the storage cluster, and before doing any reading or writing of data.

Before you begin

Before you begin, be sure you have root-level access to the node.

About this task

A quorum must be present when multiple Ceph Monitors are running.

Check the Ceph Monitor status periodically to ensure that they are running. If there is a problem with the Ceph Monitor, that prevents an agreement on the state of the storage cluster, the fault can prevent Ceph clients from reading and writing data.

Procedure

- Log into the `cephadm` shell.
  For example,

  ```
  [root@host01 ~]# cephadm shell
  ```

- Display the Ceph Monitor map.
  Display the map either by running the **`ceph mon stat`** or **`ceph mon dump`** command.
  For example,

  ```
  [ceph: root@host01 /]# ceph mon stat
  ```

  or

  ```
  [ceph: root@host01 /]# ceph mon dump
  ```

- Check the quorum status for the storage cluster.

  ```
  ceph quorum_status -f json-pretty
  ```

  For example,

  ```
  [ceph: root@host01 /]# ceph quorum_status -f json-pretty
  {
      "election_epoch": 6686,
      "quorum": [
          0,
          1,
          2
      ],
      "quorum_names": [
          "host01",
          "host03",
          "host02"
      ],
      "quorum_leader_name": "host01",
      "quorum_age": 424884,
      "features": {
          "quorum_con": "4540138297136906239",
          "quorum_mon": [
              "kraken",
  ```

```
            "luminous",
            "mimic",
            "osdmap-prune",
            "nautilus",
            "octopus",
            "pacific",
            "elector-pinging"
        ]
    },
    "monmap": {
        "epoch": 3,
        "fsid": "499829b4-832f-11eb-8d6d-001a4a000635",
        "modified": "2021-03-15T04:51:38.621737Z",
        "created": "2021-03-12T12:35:16.911339Z",
        "min_mon_release": 16,
        "min_mon_release_name": "pacific",
        "election_strategy": 1,
        "disallowed_leaders: ": "",
        "stretch_mode": false,
        "features": {
            "persistent": [
                "kraken",
                "luminous",
                "mimic",
                "osdmap-prune",
                "nautilus",
                "octopus",
                "pacific",
                "elector-pinging"
            ],
            "optional": []
        },
        "mons": [
            {
                "rank": 0,
                "name": "host01",
                "public_addrs": {
                    "addrvec": [
                        {
                            "type": "v2",
                            "addr": "10.74.255.0:3300",
                            "nonce": 0
                        },
                        {
                            "type": "v1",
                            "addr": "10.74.255.0:6789",
                            "nonce": 0
                        }
                    ]
                },
                "addr": "10.74.255.0:6789/0",
                "public_addr": "10.74.255.0:6789/0",
                "priority": 0,
                "weight": 0,
                "crush_location": "{}"
            },
            {
                "rank": 1,
                "name": "host03",
                "public_addrs": {
                    "addrvec": [
                        {
                            "type": "v2",
                            "addr": "10.74.251.164:3300",
                            "nonce": 0
                        },
                        {
                            "type": "v1",
                            "addr": "10.74.251.164:6789",
                            "nonce": 0
                        }
                    ]
                },
                "addr": "10.74.251.164:6789/0",
                "public_addr": "10.74.251.164:6789/0",
                "priority": 0,
                "weight": 0,
                "crush_location": "{}"
            },
            {
                "rank": 2,
```

```
                "name": "host02",
                "public_addrs": {
                    "addrvec": [
                        {
                            "type": "v2",
                            "addr": "10.74.249.253:3300",
                            "nonce": 0
                        },
                        {
                            "type": "v1",
                            "addr": "10.74.249.253:6789",
                            "nonce": 0
                        }
                    ]
                },
                "addr": "10.74.249.253:6789/0",
                "public_addr": "10.74.249.253:6789/0",
                "priority": 0,
                "weight": 0,
                "crush_location": "{}"
            }
        ]
    }
}
```

Understanding OSD usage stats

Use the **ceph osd df** command to view OSD utilization stats.

Example

```
[ceph: root@host01 /]# ceph osd df
ID CLASS WEIGHT  REWEIGHT SIZE    USE     DATA    OMAP    META    AVAIL   %USE VAR  PGS
 3   hdd 0.90959  1.00000  931GiB 70.1GiB 69.1GiB      0B    1GiB  861GiB 7.53 2.93  66
 4   hdd 0.90959  1.00000  931GiB 1.30GiB  308MiB      0B    1GiB  930GiB 0.14 0.05  59
 0   hdd 0.90959  1.00000  931GiB 18.1GiB 17.1GiB      0B    1GiB  913GiB 1.94 0.76  57
MIN/MAX VAR: 0.02/2.98  STDDEV: 2.91
```

- **ID:** The name of the OSD.
- **CLASS:** The type of devices the OSD uses.
- **WEIGHT:** The weight of the OSD in the CRUSH map.
- **REWEIGHT:** The default reweight value.
- **SIZE:** The overall storage capacity of the OSD.
- **USE:** The OSD capacity.
- **DATA:** The amount of OSD capacity that is used by user data.
- **OMAP:** An estimate value of the `bluefs` storage that is being used to store object map (`omap`) data (key value pairs stored in `rocksdb`).
- **META:** The `bluefs` space allocated, or the value set in the `bluestore_bluefs_min` parameter, whichever is larger, for internal metadata which is calculated as the total space allocated in `bluefs` minus the estimated `omap` data size.
- **AVAIL:** The amount of free space available on the OSD.
- **%USE:** The notional percentage of storage used by the OSD
- **VAR:** The variation above or below average utilization.
- **PGS:** The number of placement groups in the OSD.
- **MIN/MAX VAR:** The minimum and maximum variation across all OSDs.

Reference

For more information, see:

- “How Ceph calculates data usage” on page 368
- “CRUSH weights” on page 123

Understanding Ceph OSD status

Understand the different Ceph OSD states for the storage cluster.

A Ceph OSD's status is either `in` the storage cluster, or `out` of the storage cluster. It is either `up` and running, or it is `down` and not running. If a Ceph OSD is in an `up` state, it can be either `in` the storage cluster, where data can be read and written, or it is `out` of the storage cluster. If it was `in` the storage cluster and recently moved `out` of the storage cluster, Ceph starts migrating placement groups to other Ceph OSDs. If a Ceph OSD is `out` of the storage cluster, CRUSH does not assign placement groups to the Ceph OSD. If a Ceph OSD is in a `down` state, it is also `out`.

> **Note:** If a Ceph OSD is in the `down` and `in` state there is a problem and the storage cluster will not be in a healthy state.

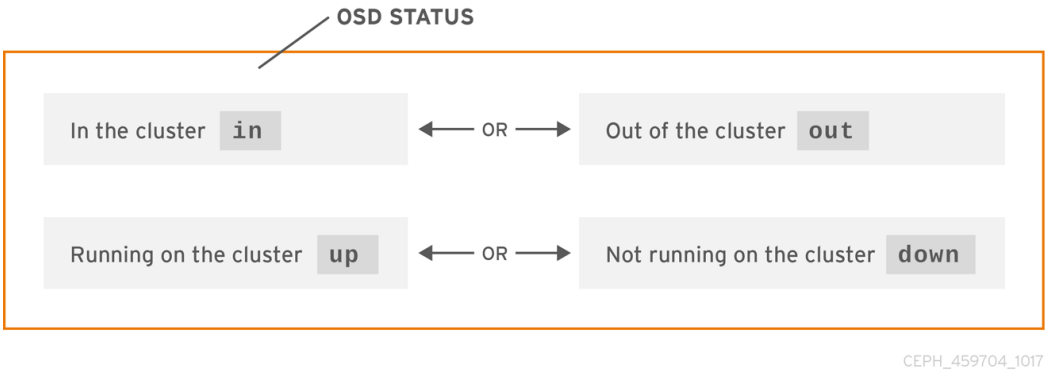


Figure 46: OSD states

If you run a command such as **`ceph health`**, **`ceph -s`** or **`ceph -w`**, the storage cluster does not always echo back HEALTH OK. Not displaying the HEALTH OK output is expected in the following circumstances:

- You have not started the storage cluster yet, and it is not responding.
- You have just started or restarted the storage cluster, and it is not ready yet, because the placement groups are getting created and the Ceph OSDs are in the process of peering.
- You just added or removed a Ceph OSD.
- You just modified the storage cluster map.

An important aspect of monitoring Ceph OSDs is to ensure that when the storage cluster is up and running that all Ceph OSDs that are `in` the storage cluster are `up` and running, too.

To see whether all OSDs are running, run one of the following commands:

- `ceph osd stat`

  For example,

  ```
  [ceph: root@host01 /]# ceph osd stat
  ```

- `ceph osd dump`

  For example,

  ```
  [ceph: root@host01 /]# ceph osd dump
  ```

The command results shown as

```
eNNNN: x osds: y up, z in
```

where,

- *eNNNN* is the map epoch

- x is the total number of OSDs
- y is the number of OSDs that are in an `up` state
- z is the number OSDs that are in an `in` state

If the number of Ceph OSDs that are `in` the storage cluster are more than the number of Ceph OSDs that are `up`. Run the **ceph osd tree** command to identify the `ceph-osd` daemons that are not running.

```
[ceph: root@host01 /]# ceph osd tree

# id    weight  type name   up/down reweight
-1  3   pool default
-3  3       rack mainrack
-2  3           host osd-host
0   1               osd.0   up  1
1   1               osd.1   up  1
2   1               osd.2   up  1
```

Tip: The ability to search through a well-designed CRUSH hierarchy can help you troubleshoot the storage cluster by identifying the physical locations faster.

If a Ceph OSD is `down`, connect to the node and start it. Use either the Red Hat Ceph Storage Console or the command line to restart the Ceph OSD daemon.

From the command line:

```
systemctl start CEPH_OSD_SERVICE_ID
```

For example,

```
[root@host01 ~]# systemctl start ceph-499829b4-832f-11eb-8d6d-001a4a000635@osd.6.service
```

Related information

Dashboard

Checking Ceph Monitor status

If the storage cluster has multiple Ceph Monitors, which is a requirement for a production Red Hat Ceph Storage cluster, then you can check the Ceph Monitor quorum status after starting the storage cluster, and before doing any reading or writing of data.

Before you begin

Before you begin, be sure you have root-level access to the node.

About this task

A quorum must be present when multiple Ceph Monitors are running.

Check the Ceph Monitor status periodically to ensure that they are running. If there is a problem with the Ceph Monitor, that prevents an agreement on the state of the storage cluster, the fault can prevent Ceph clients from reading and writing data.

Procedure

- Log into the `cephadm` shell.
  For example,

  ```
  [root@host01 ~]# cephadm shell
  ```

- Display the Ceph Monitor map.
  Display the map either by running the **ceph mon stat** or **ceph mon dump** command.
  For example,

```
[ceph: root@host01 /]# ceph mon stat
```

or

```
[ceph: root@host01 /]# ceph mon dump
```

– Check the quorum status for the storage cluster.

```
ceph quorum_status -f json-pretty
```

For example,

```
[ceph: root@host01 /]# ceph quorum_status -f json-pretty
{
    "election_epoch": 6686,
    "quorum": [
        0,
        1,
        2
    ],
    "quorum_names": [
        "host01",
        "host03",
        "host02"
    ],
    "quorum_leader_name": "host01",
    "quorum_age": 424884,
    "features": {
        "quorum_con": "4540138297136906239",
        "quorum_mon": [
            "kraken",
            "luminous",
            "mimic",
            "osdmap-prune",
            "nautilus",
            "octopus",
            "pacific",
            "elector-pinging"
        ]
    },
    "monmap": {
        "epoch": 3,
        "fsid": "499829b4-832f-11eb-8d6d-001a4a000635",
        "modified": "2021-03-15T04:51:38.621737Z",
        "created": "2021-03-12T12:35:16.911339Z",
        "min_mon_release": 16,
        "min_mon_release_name": "pacific",
        "election_strategy": 1,
        "disallowed_leaders: ": "",
        "stretch_mode": false,
        "features": {
            "persistent": [
                "kraken",
                "luminous",
                "mimic",
                "osdmap-prune",
                "nautilus",
                "octopus",
                "pacific",
                "elector-pinging"
            ],
            "optional": []
        },
        "mons": [
            {
                "rank": 0,
                "name": "host01",
                "public_addrs": {
                    "addrvec": [
                        {
                            "type": "v2",
                            "addr": "10.74.255.0:3300",
                            "nonce": 0
                        },
                        {
                            "type": "v1",
                            "addr": "10.74.255.0:6789",
                            "nonce": 0
```

```
                        }
                    ]
                },
                "addr": "10.74.255.0:6789/0",
                "public_addr": "10.74.255.0:6789/0",
                "priority": 0,
                "weight": 0,
                "crush_location": "{}"
            },
            {
                "rank": 1,
                "name": "host03",
                "public_addrs": {
                    "addrvec": [
                        {
                            "type": "v2",
                            "addr": "10.74.251.164:3300",
                            "nonce": 0
                        },
                        {
                            "type": "v1",
                            "addr": "10.74.251.164:6789",
                            "nonce": 0
                        }
                    ]
                },
                "addr": "10.74.251.164:6789/0",
                "public_addr": "10.74.251.164:6789/0",
                "priority": 0,
                "weight": 0,
                "crush_location": "{}"
            },
            {
                "rank": 2,
                "name": "host02",
                "public_addrs": {
                    "addrvec": [
                        {
                            "type": "v2",
                            "addr": "10.74.249.253:3300",
                            "nonce": 0
                        },
                        {
                            "type": "v1",
                            "addr": "10.74.249.253:6789",
                            "nonce": 0
                        }
                    ]
                },
                "addr": "10.74.249.253:6789/0",
                "public_addr": "10.74.249.253:6789/0",
                "priority": 0,
                "weight": 0,
                "crush_location": "{}"
            }
        ]
    }
}
```


Using Ceph administration socket

Use the administration socket to interact with a given daemon directly by using a UNIX socket file.

For example, the socket enables you to:

- List the Ceph configuration at runtime
- Set configuration values at runtime directly without relying on Monitors. This is useful when Monitors are down.
- Dump historic operations
- Dump the operation priority queue state
- Dump operations without rebooting
- Dump performance counters

In addition, using the socket is helpful when troubleshooting problems related to Ceph Monitors or OSDs.

Regardless, if the daemon is not running, a following error is returned when attempting to use the administration socket:

```
Error 111: Connection Refused
```

Important: The administration socket is only available while a daemon is running. When you shut down the daemon properly, the administration socket is removed. However, if the daemon terminates unexpectedly, the administration socket might persist.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. To use the socket:

   Syntax

   ```
   ceph daemon MONITOR_ID COMMAND
   ```

   Replace:

   - `MONITOR_ID` of the daemon
   - `COMMAND` with the command to run. Use `help` to list the available commands for a given daemon.

     To view the status of a Ceph Monitor:

     Example

     ```
     [ceph: root@host01 /]# ceph daemon mon.host01 help
     {
         "add_bootstrap_peer_hint": "add peer address as potential bootstrap peer for
     cluster bringup",
         "add_bootstrap_peer_hintv": "add peer address vector as potential bootstrap
     peer for cluster bringup",
         "compact": "cause compaction of monitor's leveldb/rocksdb storage",
         "config diff": "dump diff of current config and default config",
         "config diff get": "dump diff get <field>: dump diff of current and default
     config setting <field>",
         "config get": "config get <field>: get the config value",
         "config help": "get config setting schema and descriptions",
         "config set": "config set <field> <val> [<val> ...]: set a config variable",
         "config show": "dump current config settings",
         "config unset": "config unset <field>: unset a config variable",
         "connection scores dump": "show the scores used in connectivity-based
     elections",
         "connection scores reset": "reset the scores used in connectivity-based
     elections",
         "counter dump": "dump all labeled and non-labeled counters and their values",
         "counter schema": "dump all labeled and non-labeled counters schemas",
         "dump_historic_ops": "show recent ops",
         "dump_historic_slow_ops": "show recent slow ops",
         "dump_mempools": "get mempool stats",
         "get_command_descriptions": "list available commands",
         "git_version": "get git sha1",
         "heap": "show heap usage info (available only if compiled with tcmalloc)",
         "help": "list available commands",
         "injectargs": "inject configuration arguments into running daemon",
         "log dump": "dump recent log entries to log file",
         "log flush": "flush log entries to log file",
         "log reopen": "reopen log file",
         "mon_status": "report status of monitors",
         "ops": "show the ops currently in flight",
     ```

```
    "perf dump": "dump non-labeled counters and their values",
    "perf histogram dump": "dump perf histogram values",
    "perf histogram schema": "dump perf histogram schema",
    "perf reset": "perf reset <name>: perf reset all or one perfcounter name",
    "perf schema": "dump non-labeled counters schemas",
    "quorum enter": "force monitor back into quorum",
    "quorum exit": "force monitor out of the quorum",
    "sessions": "list existing sessions",
    "smart": "Query health metrics for underlying device",
    "sync_force": "force sync of and clear monitor store",
    "version": "get ceph version"
}
```

Example

```
[ceph: root@host01 /]# ceph daemon mon.host01 mon_status

{
    "name": "host01",
    "rank": 0,
    "state": "leader",
    "election_epoch": 120,
    "quorum": [
        0,
        1,
        2
    ],
    "quorum_age": 206358,
    "features": {
        "required_con": "2449958747317026820",
        "required_mon": [
            "kraken",
            "luminous",
            "mimic",
            "osdmap-prune",
            "nautilus",
            "octopus",
            "pacific",
            "elector-pinging"
        ],
        "quorum_con": "4540138297136906239",
        "quorum_mon": [
            "kraken",
            "luminous",
            "mimic",
            "osdmap-prune",
            "nautilus",
            "octopus",
            "pacific",
            "elector-pinging"
        ]
    },
    "outside_quorum": [],
    "extra_probe_peers": [],
    "sync_provider": [],
    "monmap": {
        "epoch": 3,
        "fsid": "81a4597a-b711-11eb-8cb8-001a4a000740",
        "modified": "2021-05-18T05:50:17.782128Z",
        "created": "2021-05-17T13:13:13.383313Z",
        "min_mon_release": 16,
        "min_mon_release_name": "pacific",
        "election_strategy": 1,
        "disallowed_leaders: ": "",
        "stretch_mode": false,
        "features": {
            "persistent": [
                "kraken",
                "luminous",
                "mimic",
                "osdmap-prune",
                "nautilus",
                "octopus",
                "pacific",
                "elector-pinging"
            ],
            "optional": []
        },
        "mons": [
            {
```

```
            "rank": 0,
            "name": "host01",
            "public_addrs": {
                "addrvec": [
                    {
                        "type": "v2",
                        "addr": "10.74.249.41:3300",
                        "nonce": 0
                    },
                    {
                        "type": "v1",
                        "addr": "10.74.249.41:6789",
                        "nonce": 0
                    }
                ]
            },
            "addr": "10.74.249.41:6789/0",
            "public_addr": "10.74.249.41:6789/0",
            "priority": 0,
            "weight": 0,
            "crush_location": "{}"
        },
        {
            "rank": 1,
            "name": "host02",
            "public_addrs": {
                "addrvec": [
                    {
                        "type": "v2",
                        "addr": "10.74.249.55:3300",
                        "nonce": 0
                    },
                    {
                        "type": "v1",
                        "addr": "10.74.249.55:6789",
                        "nonce": 0
                    }
                ]
            },
            "addr": "10.74.249.55:6789/0",
            "public_addr": "10.74.249.55:6789/0",
            "priority": 0,
            "weight": 0,
            "crush_location": "{}"
        },
        {
            "rank": 2,
            "name": "host03",
            "public_addrs": {
                "addrvec": [
                    {
                        "type": "v2",
                        "addr": "10.74.249.49:3300",
                        "nonce": 0
                    },
                    {
                        "type": "v1",
                        "addr": "10.74.249.49:6789",
                        "nonce": 0
                    }
                ]
            },
            "addr": "10.74.249.49:6789/0",
            "public_addr": "10.74.249.49:6789/0",
            "priority": 0,
            "weight": 0,
            "crush_location": "{}"
        }
    ]
},
"feature_map": {
    "mon": [
        {
            "features": "0x3f01cfb9fffdffff",
            "release": "luminous",
            "num": 1
        }
    ],
    "osd": [
        {
            "features": "0x3f01cfb9fffdffff",
```

```
                "release": "luminous",
                "num": 3
            }
        ]
    },
    "stretch_mode": false
}
```

3. Alternatively, specify the Ceph daemon by using its socket file:

   Syntax

   ```
   ceph daemon /var/run/ceph/SOCKET_FILE COMMAND
   ```

4. To view the status of a Ceph OSD named `osd.0` on the specific host:

   Example

   ```
   [ceph: root@host01 /]# ceph daemon /var/run/ceph/ceph-osd.0.asok status
   {
       "cluster_fsid": "9029b252-1668-11ee-9399-001a4a000429",
       "osd_fsid": "1de9b064-b7a5-4c54-9395-02ccda637d21",
       "whoami": 0,
       "state": "active",
       "oldest_map": 1,
       "newest_map": 58,
       "num_pgs": 33
   }
   ```

   Note: You can use `help` instead of `status` for the various options that are available for the specific daemon.

5. List all socket files for the Ceph processes:

   Example

   ```
   [ceph: root@host01 /]# ls /var/run/ceph
   ```

Reference

- For more information, see “Identifying problems” on page 1579.

Track the data availability score of a cluster (Technology Preview)

Ceph tracks the data availability of each pool and calculates a score that represents long-term data accessibility for operational assessment.

Important: Technology Preview features are not supported with production service level agreements (SLAs), might not be functionally complete, and does not recommend using them for production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

Ceph measures pool reliability over time by tracking when a pool is available or unavailable. From these measurements, Ceph calculates a pool’s availability score, which reflects how consistently the pool has remained accessible.

The score is based on the following formula:

```
availability_score = MTBF / (MTBF + MTTR)
```

A score of 1 indicates continuous data availability, while lower values indicate interruptions.

Use data availability scoring to:

- Understand long-term reliability trends for each pool.

- Correlate incidents with data availability impact for more accurate root-cause analysis.
- Control noise in monitoring by adjusting update frequency or pausing tracking during planned work.
- Reset baselines after structural changes to a pool (for example, new CRUSH mappings).

How Ceph determines pool availability

Available

All placement groups (PGs) in the pool are active and no unfound objects exist.

Unavailable

At least one PG becomes inactive or an unfound object appears.

How uptime and downtime update

Ceph tracks how long each pool remains available or unavailable. At each interval, the system compares the pool's previous state to its current state and updates timers accordingly.

Ceph recalculates MTBF, MTTR, and the data availability score using the updated values, as detailed in "Pool state transition rules" on page 381.

Table 54: Pool state transition rules

Previous state	Current state	Uptime update	Downtime update
Available	Available	+ diff time	No update
Available	Unavailable	+ diff time	No update
Unavailable	Available	+ diff time	No update
Unavailable	Unavailable	No update	+ diff time

Enable or disable data availability tracking

Tracking is disabled by default. To enable data availability, run the following command:

```
ceph config set mon enable_availability_tracking true
```

- To start or resume data availability tracking, set **enable_availability_tracking** to `true`.
- To disable data availability tracking, set **enable_availability_tracking** to `false`. While disabled, the last calculated score is preserved.

View pool availability status

Run the following command to display data availability information for each pool:

```
ceph osd pool availability-status
```

Use JSON output to view exact time values:

```
ceph osd pool availability-status --format json-pretty
```

For example,

```
POOL         UPTIME  DOWNTIME  NUMFAILURES  MTBF  MTTR  SCORE     AVAILABLE
rbd             2m     21s           1        2m   21s  0.888889     1
.mgr             86s     0s           0        0s   0s         1       1
cephfs.a.meta    77s     0s           0        0s   0s         1       1
cephfs.a.data    76s     0s           0        0s   0s         1       1
```

Update interval control

Ceph updates data availability metrics every second by default. You can modify the update interval with the following command:

```
ceph config set mon pool_availability_update_interval 2
```

> **Note:** You cannot set the update interval lower than the value of **paxos_propose_interval**.

Transient state changes that occur and resolve between update cycles are not recorded.

Clear a pool's availability status
To remove historical availability data for a pool, run the following command:

```
ceph osd pool clear-availability-status pool-name
```

> **Note:** You cannot clear a pool's data availability status when tracking is disabled.

Low-level monitoring

As a storage administrator, you can monitor the health of a Red Hat Ceph Storage cluster from a low-level perspective.

Low-level monitoring typically involves ensuring that Ceph OSDs are peering properly. When peering faults occur, placement groups operate in a degraded state. This degraded state can be the result of many different things, such as hardware failure, a hung or crashed Ceph daemon, network latency, or a complete site outage.

Monitoring placement group sets

Learn and understand about monitoring placement group sets.

When CRUSH assigns placement groups to Ceph OSDs, it looks at the number of replicas for the pool and assigns the placement group to Ceph OSDs such that each replica of the placement group gets assigned to a different Ceph OSD. For example, if the pool requires three replicas of a placement group, CRUSH may assign them to `osd.1`, `osd.2` and `osd.3` respectively. CRUSH actually seeks a pseudo-random placement that will take into account failure domains you set in the CRUSH map, so you will rarely see placement groups assigned to nearest neighbor Ceph OSDs in a large cluster. We refer to the set of Ceph OSDs that should contain the replicas of a particular placement group as the **Acting Set**. In some cases, an OSD in the Acting Set is `down` or otherwise not able to service requests for objects in the placement group. When these situations arise, do not panic. Common examples include:

- You added or removed an OSD. Then, CRUSH reassigned the placement group to other Ceph OSDs, thereby changing the composition of the acting set and spawning the migration of data with a "backfill" process.
- A Ceph OSD was `down`, was restarted and is now `recovering`.
- A Ceph OSD in the acting set is `down` or unable to service requests, and another Ceph OSD has temporarily assumed its duties.

Ceph processes a client request using the **Up Set**, which is the set of Ceph OSDs that actually handle the requests. In most cases, the up set and the Acting Set are virtually identical. When they are not, it can indicate that Ceph is migrating data, a Ceph OSD is recovering, or that there is a problem, that is, Ceph usually echoes a `HEALTH WARN` state with a "stuck stale" message in such scenarios.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. To retrieve a list of placement groups:

   Example

   ```
   [ceph: root@host01 /]# ceph pg dump
   ```

3. View which Ceph OSDs are in the **Acting Set** or in the **Up Set** for a given placement group:

   Syntax

   ```
   ceph pg map PG_NUM
   ```

Example

```
[ceph: root@host01 /]# ceph pg map 128
```

Note: If the Up Set and Acting Set do not match, this may be an indicator that the storage cluster rebalancing itself or of a potential problem with the storage cluster.

Ceph OSD peering

Before you can write data to a placement group, it must be in an `active` state, and it **should** be in a `clean` state. For Ceph to determine the current state of a placement group, the primary OSD of the placement group that is, the first OSD in the acting set, peers with the secondary and tertiary OSDs to establish agreement on the current state of the placement group.

"Figure: Peering" on page 383 is an example that assumes a pool with three replicas of the PG.

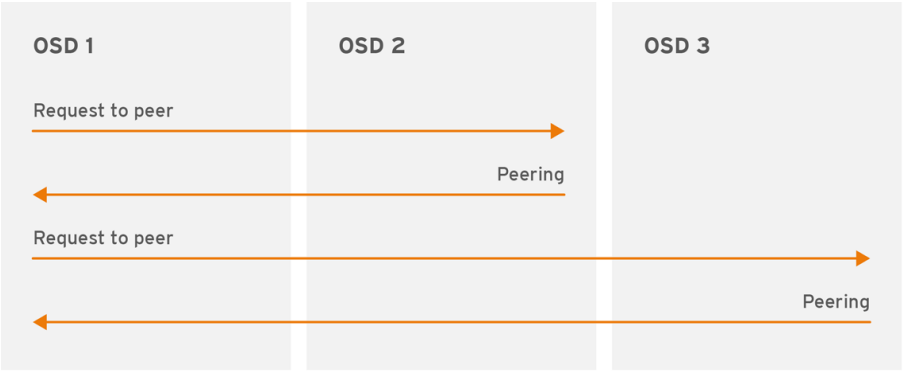


Figure 47: Peering

Placement group states

Learn about and understand the different placement group (PG) states.

When running commands such as `ceph health`, `ceph -s` or `ceph -w`, you may notice that the cluster does not always echo back `HEALTH OK`. After you check to see if the OSDs are running, you should also check placement group states. You should expect that the cluster does **NOT** echo `HEALTH OK` in a number of placement group peering-related circumstances:

- You have just created a pool and placement groups have not peered yet.
- The placement groups are recovering.
- You have just added an OSD to or removed an OSD from the cluster.
- You have just modified the CRUSH map and the placement groups are migrating.
- There is inconsistent data in different replicas of a placement group.
- Ceph is scrubbing a placement group's replicas.
- Ceph does not have enough storage capacity to complete backfilling operations.

If one of the foregoing circumstances causes Ceph to echo `HEALTH WARN`, do not panic. In many cases, the cluster will recover on its own. In some cases, you may need to take action. An important aspect of monitoring placement groups is to ensure that when the cluster is up and running that all placement groups are `active`, and preferably in the `clean` state.

To see the status of all placement groups, run:

Example

```
[ceph: root@host01 /]# ceph pg stat
```

The result should tell you the placement group map version, `vNNNNNN`, the total number of placement groups, `x`, and how many placement groups, `y`, are in a particular state such as `active+clean`:

```
vNNNNNN: x pgs: y active+clean; z bytes data, aa MB used, bb GB / cc GB avail
```

> **Note:** It is common for Ceph to report multiple states for placement groups.

Snapshot Trimming PG States

When snapshots exist, two additional PG states will be reported.

- `snaptrim` : The PGs are currently being trimmed
- `snaptrim_wait` : The PGs are waiting to be trimmed

Example Output:

```
244 active+clean+snaptrim_wait
 32 active+clean+snaptrim
```

In addition to the placement group states, Ceph will also echo back the amount of data used, `aa`, the amount of storage capacity remaining, `bb`, and the total storage capacity `cc` for the placement group. These numbers can be important in a few cases:

- You are reaching the `near full ratio` or `full ratio`.
- Your data isn't getting distributed across the cluster due to an error in the CRUSH configuration.

Placement Group IDs

Placement group IDs consist of the pool number, and not the pool name, followed by a period (.) and the placement group ID - a hexadecimal number. You can view pool numbers and their names from the output of `ceph osd lspools`. The default pool names `data`, `metadata` and `rbd` correspond to pool numbers `0`, `1` and `2` respectively. A fully qualified placement group ID has the following form:

Syntax

```
POOL_NUM.PG_ID
```

Example output:

```
0.1f
```

- To retrieve a list of placement groups:

  Example

  ```
  [ceph: root@host01 /]# ceph pg dump
  ```

- To format the output in JSON format and save it to a file:

  Syntax

  ```
  ceph pg dump -o FILE_NAME --format=json
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph pg dump -o test --format=json
  ```

- Query a particular placement group:

  Syntax

  ```
  ceph pg POOL_NUM.PG_ID query
  ```

  Example

```
[ceph: root@host01 /]#  ceph pg 5.fe query
{
    "snap_trimq": "[]",
    "snap_trimq_len": 0,
    "state": "active+clean",
    "epoch": 2449,
    "up": [
        3,
        8,
        10
    ],
    "acting": [
        3,
        8,
        10
    ],
    "acting_recovery_backfill": [
        "3",
        "8",
        "10"
    ],
    "info": {
        "pgid": "5.ff",
        "last_update": "0'0",
        "last_complete": "0'0",
        "log_tail": "0'0",
        "last_user_version": 0,
        "last_backfill": "MAX",
        "purged_snaps": [],
        "history": {
            "epoch_created": 114,
            "epoch_pool_created": 82,
            "last_epoch_started": 2402,
            "last_interval_started": 2401,
            "last_epoch_clean": 2402,
            "last_interval_clean": 2401,
            "last_epoch_split": 114,
            "last_epoch_marked_full": 0,
            "same_up_since": 2401,
            "same_interval_since": 2401,
            "same_primary_since": 2086,
            "last_scrub": "0'0",
            "last_scrub_stamp": "2022-10-17T01:32:03.763988+0000",
            "last_deep_scrub": "0'0",
            "last_deep_scrub_stamp": "2022-10-17T01:32:03.763988+0000",
            "last_clean_scrub_stamp": "2022-10-17T01:32:03.763988+0000",
            "prior_readable_until_ub": 0
        },
        "stats": {
            "version": "0'0",
            "reported_seq": "2989",
            "reported_epoch": "2449",
            "state": "active+clean",
            "last_fresh": "2022-10-18T05:16:59.401080+0000",
            "last_change": "2022-10-17T01:32:03.764162+0000",
            "last_active": "2022-10-18T05:16:59.401080+0000",
....
```

Reference

- For more details about the snapshot trimming settings, see Object Storage Daemon (OSD) configuration options.

Placement group creating state

When you create a pool, it creates the number of placement groups you specified.

.Ceph echos `creating` when it is creating one or more placement groups. Once they are created, the OSDs that are part of a placement group's Acting Set peers. Once peering is complete, the placement group status should be `active+clean`, which means a Ceph client can begin writing to the placement group.

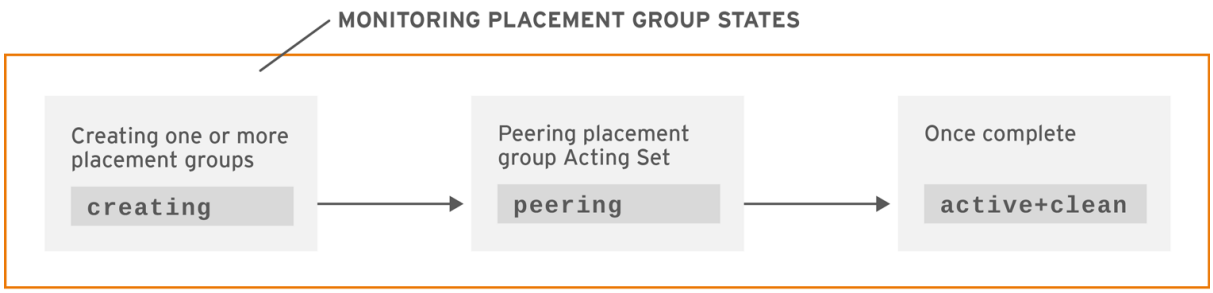


Figure 48: Creating PGs

Placement group peering state

When Ceph is peering a placement group, Ceph is bringing the OSDs that store the replicas of the placement group into **agreement about the state** of the objects and metadata in the placement group. When Ceph completes peering, this means that the OSDs that store the placement group agree about the current state of the placement group.

Completion of the peering process does **NOT** mean that each replica has the latest contents.

Authoritative History

Ceph does **NOT** acknowledge a write operation to a client, until all OSDs of the acting set persist the write operation. This practice ensures that at least one member of the acting set will have a record of every acknowledged write operation since the last successful peering operation.

With an accurate record of each acknowledged write operation, Ceph can construct and disseminate a new authoritative history of the placement group. A complete, and fully ordered set of operations that, if performed, would bring an OSD's copy of a placement group up to date.

Placement group active state

The `active` state means that the data in the placement group is generally available in the primary placement group and the replicas for read and write operations.

Once Ceph completes the peering process, a placement group may become `active`.

Placement group clean state

When a placement group is in the `clean` state, the primary OSD and the replica OSDs have successfully peered and there are no stray replicas for the placement group.

The `clean` state indicates that Ceph replicated all objects in the placement group the correct number of times.

Placement group degraded state

When a client writes an object to the primary OSD, the primary OSD is responsible for writing the replicas to the replica OSDs. After the primary OSD writes the object to storage, the placement group will remain in a `degraded` state until the primary OSD has received an acknowledgment from the replica OSDs that Ceph created the replica objects successfully.

The reason a placement group can be `active+degraded` is that an OSD may be `active` even though it doesn't hold all of the objects yet. If an OSD goes `down`, Ceph marks each placement group assigned to the OSD as `degraded`. The Ceph OSDs must peer again when the Ceph OSD comes back online. However, a client can still write a new object to a `degraded` placement group if it is `active`.

If an OSD is `down` and the `degraded` condition persists, Ceph may mark the `down` OSD as `out` of the cluster and remap the data from the `down` OSD to another OSD. The time between being marked `down` and being marked `out` is controlled by `mon_osd_down_out_interval`, which is set to `600` seconds by default.

A placement group can also be `degraded`, because Ceph cannot find one or more objects that Ceph thinks should be in the placement group. While you cannot read or write to unfound objects, you can still access all of the other objects in the `degraded` placement group.

For example, if there are nine OSDs in a three way replica pool. If OSD number 9 goes down, the PGs assigned to OSD 9 goes into a degraded state. If OSD 9 does not recover, it goes out of the storage cluster and the storage cluster rebalances. In that scenario, the PGs are degraded and then recover to an active state.

Placement Group recovering state

Ceph was designed for fault-tolerance at a scale where hardware and software problems are ongoing. When an OSD goes `down`, its contents may fall behind the current state of other replicas in the placement groups. When the OSD is back `up`, the contents of the placement groups must be updated to reflect the current state. During that time period, the OSD may reflect a `recovering` state.

Recovery is not always trivial, because a hardware failure might cause a cascading failure of multiple Ceph OSDs. For example, a network switch for a rack or cabinet may fail, which can cause the OSDs of a number of host machines to fall behind the current state of the storage cluster. Each one of the OSDs must recover once the fault is resolved.

Ceph provides a number of settings to balance the resource contention between new service requests and the need to recover data objects and restore the placement groups to the current state. The `osd recovery delay start` setting allows an OSD to restart, re-peer and even process some replay requests before starting the recovery process. The `osd recovery threads` setting limits the number of threads for the recovery process, by default one thread. The `osd recovery thread timeout` sets a thread timeout, because multiple Ceph OSDs can fail, restart and re-peer at staggered rates. The `osd recovery max active` setting limits the number of recovery requests a Ceph OSD works on simultaneously to prevent the Ceph OSD from failing to serve. The `osd recovery max chunk` setting limits the size of the recovered data chunks to prevent network congestion.

Back fill state

When a new Ceph OSD joins the storage cluster, CRUSH will reassign placement groups from OSDs in the cluster to the newly added Ceph OSD. Forcing the new OSD to accept the reassigned placement groups immediately can put excessive load on the new Ceph OSD. Backfilling the OSD with the placement groups allows this process to begin in the background. Once backfilling is complete, the new OSD will begin serving requests when it is ready.

During the backfill operations, you might see one of several states:

- `backfill_wait` indicates that a backfill operation is pending, but isn't underway yet
- `backfill` indicates that a backfill operation is underway
- `backfill_too_full` indicates that a backfill operation was requested, but couldn't be completed due to insufficient storage capacity.

When a placement group cannot be backfilled, it can be considered `incomplete`.

Ceph provides a number of settings to manage the load spike associated with reassigning placement groups to a Ceph OSD, especially a new Ceph OSD. By default, `osd_max_backfills` sets the maximum number of concurrent backfills to or from a Ceph OSD to 10. The `osd backfill full ratio` enables a Ceph OSD to refuse a backfill request if the OSD is approaching its full ratio, by default 85%. If an OSD refuses a backfill request, the `osd backfill retry interval` enables an OSD to retry the request, by default after 10 seconds. OSDs can also set `osd backfill scan min` and `osd backfill scan max` to manage scan intervals, by default 64 and 512.

For some workloads, it is beneficial to avoid regular recovery entirely and use backfill instead. Since backfilling occurs in the background, this allows I/O to proceed on the objects in the OSD. You can force a backfill rather than a recovery by setting the `osd_min_pg_log_entries` option to `1`, and setting the `osd_max_pg_log_entries` option to `2`. Contact your Red Hat Support account team for details on when this situation is appropriate for your workload.

Placement group remapped state

When the Acting Set that services a placement group changes, the data migrates from the old acting set to the new acting set. It may take some time for a new primary OSD to service requests. So it may ask the old primary to continue to service requests until the placement group migration is complete. Once data migration completes, the mapping uses the primary OSD of the new acting set.

Placement group stale state

If the Primary OSD of a placement group's acting set fails to report to the monitor or if other OSDs have reported the primary OSD `down`, the monitors will mark the placement group `stale`.

While Ceph uses heartbeats to ensure that hosts and daemons are running, the `ceph-osd` daemons may also get into a `stuck` state where they are not reporting statistics in a timely manner. For example, a temporary network fault. By default, OSD daemons report their placement group, up through boot and failure statistics every half a second, that is, `0.5`, which is more frequent than the heartbeat thresholds.

When you start the storage cluster, it is common to see the `stale` state until the peering process completes. After the storage cluster has been running for a while, seeing placement groups in the `stale` state indicates that the primary OSD for those placement groups is `down` or not reporting placement group statistics to the monitor.

Placement group misplaced state

There are some temporary backfilling scenarios where a PG gets mapped temporarily to an OSD. When that `temporary` situation should no longer be the case, the PGs might still reside in the temporary location and not in the proper location. In which case, they are said to be `misplaced`. That's because the correct number of extra copies actually exist, but one or more copies is in the wrong place.

For example, there are 3 OSDs: 0,1,2 and all PGs map to some permutation of those three. If you add another OSD (OSD 3), some PGs will now map to OSD 3 instead of one of the others. However, until OSD 3 is backfilled, the PG will have a temporary mapping allowing it to continue to serve I/O from the old mapping. During that time, the PG is `misplaced`, because it has a temporary mapping, but not `degraded`, since there are three copies.

Syntax

```
pg 1.5: up=acting: [0,1,2]
ADD_OSD_3
pg 1.5: up: [0,3,1] acting: [0,1,2]
```

`[0,1,2]` is a temporary mapping, so the `up` set is not equal to the `acting` set and the PG is `misplaced` but not `degraded` since `[0,1,2]` is still three copies.

Example

```
pg 1.5: up=acting: [0,3,1]
```

OSD 3 is now backfilled and the temporary mapping is removed, not degraded and not misplaced.

Placement group incomplete state

A PG goes into a `incomplete` state when there is incomplete content and peering fails, that is, when there are no complete OSDs which are current enough to perform recovery.

For example, OSD 1, 2, and 3 are the acting OSD set and it switches to OSD 1, 4, and 3, then `osd.1` requests a temporary acting set of OSD 1, 2, and 3 while backfilling 4. During this time, if OSD 1, 2, and 3 all go down, `osd.4` is the only one which might not have fully backfilled all the data. At this time, the PG goes `incomplete` indicating that there are no complete OSDs which are current enough to perform recovery.

Alternately, if `osd.4` is not involved and the acting set is simply OSD 1, 2, and 3 when OSD 1, 2, and 3 go down, the PG would likely go `stale` indicating that the monitors have not heard anything on that PG since the acting set changed. The reason being there are no more OSDs to notify the new OSDs.

Identifying stuck placement groups

A placement group is not necessarily problematic just because it is not in a `active+clean` state. Generally, Ceph's ability to self repair might not be working when placement groups get stuck.

Before you begin

Before you begin, make sure that you have root-level access to the node.

About this task

The stuck states include `unclean`, `inactive`, and `stale`.

Unclean

Placement groups contain objects that are not replicated the desired number of times. They should be recovering.

Inactive

Placement groups cannot process reads or writes because they are waiting for an OSD with the most up-to-date data to come back `up`.

Stale

Placement groups are in an unknown state, because the OSDs that host them have not reported to the monitor cluster in a while, and can be configured with the `mon osd report timeout` setting.

Procedure

- Identify the stuck placement group by running the **`pg dump_stuck`** command.

```
ceph pg dump_stuck {inactive|unclean|stale|undersized|degraded [inactive|unclean|stale|
undersized|degraded...]} {INT}
```

For example,

```
[ceph: root@host01 /]# ceph pg dump_stuck stale
OK
```

Finding object's location

The Ceph client retrieves the latest cluster map and the CRUSH algorithm calculates how to map the object to a placement group, and then calculates how to assign the placement group to an OSD dynamically.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

To find the object location, all you need is the object name and the pool name:

Syntax

```
ceph osd map POOL_NAME OBJECT_NAME
```

Example

```
[ceph: root@host01 /]# ceph osd map mypool myobject
```

Monitor a Ceph cluster with the Dashboard

As a storage administrator, you can use Red Hat Ceph Storage Dashboard to monitor specific aspects of the cluster based on types of hosts, services, data access methods, and more.

Monitor a Ceph cluster with the Dashboard

As a storage administrator, you can use Red Hat Ceph Storage Dashboard to monitor specific aspects of the cluster based on types of hosts, services, data access methods, and more.

Monitoring hosts of the Ceph cluster

Monitor the hosts of the cluster on the Red Hat Ceph Storage Dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.

- Hosts are added to the storage cluster.
- All the services, monitor, manager and OSD daemons are deployed on the storage cluster.

About this task

The following are the different tabs on the hosts page. Each tab contains a table with the relavent information. The tables are searchable and customizable by column and row.

To change the order of the columns, select the column name and drag to place within the table.

To select which columns are displaying, click the **toggle columns** button and select or clear column names. Enter the number of rows to be displayed in the row selector field.

Devices

This tab has a table that details the device ID, state of the device health, life expectancy, device name, prediction creation date, and the daemons on the hosts.

Physical Disks

This tab has a table that details all disks attached to a selected host, as well as their type, size and others. It has details such as device path, type of device, available, vendor, model, size, and the OSDs deployed.

To identify which disk is where on the physical device, select the device and click **Identify**. Select the duration of how long the LED should blink for to find the selected disk.

Daemons

This tab has a table that details all services that have been deployed on the selected host, which container they are running in, and their current status.
The table has details such as daemon name, daemon version, status, when the daemon was last refreshed, CPU usage, memory usage (in MiB), and daemon events.

Daemon actions can be run from this tab. For more details, see “Running Ceph services” on page 701.

Performance details

This tab has details such as OSDs deployed, CPU utilization, RAM usage, network load, network drop rate, and OSD disk performance statistics. View performance information through the embedded Grafana Dashboard.

Device health

For SMART-enabled devices, you can get the individual health status and SMART data only on the OSD deployed hosts.

Procedure

1. From the dashboard navigation, go to **Cluster › Hosts**.
2. On the **Hosts List** tab, expand the host row and select the host with the daemon to perform the action on.
3. On the **Daemons** tab of the host, select the row with the daemon.

   > **Note:** The **Daemons** table can be searched and filtered.

4. Select the action that needs to be run on the daemon. The options are **Start**, **Stop**, **Restart**, and **Redeploy**.

Viewing and editing the configuration of the Ceph cluster

View various configuration options of the Ceph cluster on the dashboard. You can edit only some configuration options.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.

- All the services are deployed on the storage cluster.

Procedure

1. From the dashboard navigation, go to **Administration › Configuration**.
2. View the details of the configuration by expanding the row contents.
3. Optional: Use the search field to find a configuration.
4. Optional: You can filter for a specific configuration.
   To filter, use the following filters:
   - *Level* - Basic, advanced or dev
   - *Service* - Any, mon, mgr, osd, mds, common, mds_client, rgw, and similar filters.
   - *Source* - Any, mon, and similar filters
   - *Modified* - yes or no
5. Edit a configuration by selecting the configuration row and click **Edit**.
   a. Use the **Edit** form to edit the required parameters, and click **Update**.
   A notification displays that the configuration was updated successfully.

Viewing and editing the manager modules of the Ceph cluster

Manager modules are used to manage module-specific configuration settings. For example, you can enable alerts for the health of the cluster. You can view, enable or disable, and edit the manager modules of a cluster on the Red Hat Ceph Storage dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.

Procedure

1. From the dashboard navigation, go to **Administration › Manager Modules**.
2. View the details of a specific manager module by expanding the row contents.

Enabling a manger module

Select the row and click **Enable** from the action drop-down.

Disabling a manger module

Select the row and click **Disable** from the action drop-down.

Editing a manger module

Procedure

1. Select the row and click **Edit**.

   > **Note:** Not all modules have configurable parameters. If a module is not configurable, the **Edit** button is disabled.

2. Edit the required parameters and click **Update**.
   A notification displays that the module was updated successfully.

Monitoring monitors of the Ceph cluster

Monitor the performance of the Ceph monitors on the landing page of the Red Hat Ceph Storage dashboard. You can also view the details such as status, quorum, number of open session, and performance counters of the monitors in the **Monitors** panel.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Monitors are deployed in the storage cluster.

About this task

For more information about monitoring Ceph clusters, see:

- Ceph monitors.
- Ceph performance counters.

Procedure

1. From the dashboard navigation, go to **Cluster › Monitors**.
   The **Monitors** panel displays information about the overall monitor status and monitor hosts that are in and out of quorum.
2. To see the number of open sessions, hover the cursor over the **Open Sessions**.
3. To see performance counters for any monitor, click **Name** in the **In Quorum** and **Not In Quorum** tables.

Monitoring services of the Ceph cluster

Monitor the services of the cluster on the Red Hat Ceph Storage Dashboard. You can view the details such as hostname, daemon type, daemon ID, container ID, container image name, container image ID, version status, and last refreshed time.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Hosts are added to the storage cluster.
- All the services are deployed on the storage cluster.

Procedure

1. From the dashboard navigation, go to **Administration › Services**.
2. To view the details of a specific service, click the **Expand/Collapse** icon on it’s row.

Monitoring Ceph OSDs

Monitor the status of the Ceph OSDs on the landing page of the Red Hat Ceph Storage Dashboard. You can also view the details such as host, status, device class, number of placement groups (PGs), size flags, usage, and read or write operations time in the *OSDs* tab.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Hosts are added to the storage cluster.

- All the services including OSDs are deployed on the storage cluster.

About this task

The following are the different tabs on the OSDs page:

- **Devices** - This tab has details such as Device ID, state of health, life expectancy, device name, and the daemons on the hosts.
- **Attributes (OSD map)** - This tab shows the cluster address, details of heartbeat, OSD state, and the other OSD attributes.
- **Metadata** - This tab shows the details of the OSD object store, the devices, the operating system, and the kernel details.
- **Device health** - For SMART-enabled devices, you can get the individual health status and SMART data.
- **Performance counter** - This tab gives details of the bytes written on the devices.
- **Performance Details** - This tab has details such as OSDs deployed, CPU utilization, RAM usage, network load, network drop rate, and OSD disk performance statistics.

Procedure

1. From the dashboard navigation, go to **Cluster › OSDs**.
2. To view more information about a specific service, expand the OSD row.

Monitoring HAProxy

The Ceph Object Gateway allows you to assign many instances of the object gateway to a single zone, so that you can scale out as load increases. Since each object gateway instance has its own IP address, you can use HAProxy to balance the load across Ceph Object Gateway servers.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Admin-level access on the storage dashboard.
- An existing Ceph Object Gateway service, without SSL. If you want SSL service, the certificate should be configured on the ingress service, not the Ceph Object Gateway service.
- Ingress service deployed using the Ceph Orchestrator.
- Monitoring stack components are created on the dashboard.

About this task

You can monitor the following HAProxy metrics:

- Total responses by HTTP code.
- Total requests/responses.
- Total number of connections.
- Current total number of incoming / outgoing bytes.

You can also get the Grafana details by running the **`ceph dashboard get-grafana-api-url`** command.

Procedure

1. Log in to the Grafana URL and select **RGW Overview** panel.

   ```
   https://DASHBOARD_URL:3000
   ```

   For example,

   ```
   https://dashboard_url:3000
   ```

2. Verify the HAProxy metrics on the Grafana URL.
3. From the Ceph dashboard navigation, go to **Object › Gateways**.
4. From the **Overall Performance** tab, verify the Ceph Object Gateway HAProxy metrics.



Figure 49: Viewing the Ceph Object Gateway HAProxy metrics

Related information

Configuring high availability for the Ceph Object Gateway

Viewing the CRUSH map of the Ceph cluster

View the The CRUSH map that contains a list of OSDs and related information on the Red Hat Ceph Storage dashboard. Together, the CRUSH map and CRUSH algorithm determine how and where data is stored. The dashboard allows you to view different aspects of the CRUSH map, including OSD hosts, OSD daemons, ID numbers, device class, and more.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Ceph Dashboard is installed.
- OSD daemons deployed on the storage cluster.

About this task

The CRUSH map allows you to determine which host a specific OSD ID is running on. This is helpful if there is an issue with an OSD.

Procedure

1. From the dashboard navigation, go to **Cluster › CRUSH map**.
2. To view CRUSH map information, select a CRUSH map from the **CRUSH map viewer**.

Filtering logs of the Ceph cluster

View and filter logs of the Red Hat Ceph Storage cluster based on several criteria. The criteria includes *Priority*, *Keyword*, *Date*, and *Time range*. For further analysis, the logs are available for download to the system or copying to the clipboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- The Ceph Dashboard is installed.
- Log entries have been generated since the Ceph Monitor was last started.

About this task

Note: The Dashboard logging feature only displays the thirty latest high level events. The events are stored in memory by the Ceph Monitor. The entries disappear after restarting the Monitor. If you need to review detailed or older logs, refer to the file based logs.

For more information about logs, see

- Configuring Logging.
- Understanding Ceph Logs.

Procedure

1. From the dashboard navigation, go to **Observability › Logs**.
2. View and filter logs.
   The logs are divided up by type in different tabs: **Cluster Logs**, **Audit Logs**, and **Daemon Logs**.
   - To filter by priority, click the **Priority** drop-down menu and select either *Debug*, *Info*, *Warning*, *Error*, or *All*.
   - To filter by keyword, enter text into the **Keyword** field.
   - To filter by date, click the **Date** field and either use the date picker to select a date from the menu, or enter a date in the form of `YYYY-MM-DD`.
   - To filter by time, enter a range in the **Time range** fields using the `HH:MM - HH:MM` format. Hours must be entered using numbers `0` to `23`.
   - To combine filters, set two or more filters.
3. Download or copy the logs.
   - Download, by clicking the **Download** icon.
   - Copy, by clicking the **Copy to Clipboard** icon.

Viewing centralized logs of the Ceph cluster

Use the Ceph Dashboard to view logs from all the clients in a centralized space in the Red Hat Ceph Storage cluster. Centralized logs provide efficient monitoring by using Loki and alloy.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Grafana is configured and the cluster is logged in to.

About this task

Loki is a log aggregation system that is designed to store and query logs. Alloy is an agent that provides the contents of local logs to a private Grafana Loki instance.

Procedure

1. From the dashboard navigation, go to **Administration › Services**.
2. From **Services**, click **Create**.
3. In the **Create Service** form, from the **Type** list, select **loki**.
4. Complete the rest of the form.
5. To save the service, click **Create Service**.
6. Repeat steps “2” on page 395 through “5” on page 395 to create the `alloy` service.
   When creating the `alloy` service, select **alloy** from the **Type** list, instead of **loki**.

> **Note:** By default, the alloy service is deployed on all running hosts.

To see whether the services were created successfully, verify that **loki** and **alloy** services are running in the services list.

7. Enable logging to the file.
   a. Go to **Administration › Configuration**.
   b. From the search bar, search for `log_to_file`.

      > **Note:** If the file does not show in the search results, clear any filter settings that might be selected.

   c. Select `log_to_file` and click **Edit**.
      The **Edit log_to_file** form opens.
   d. In the **Values** section of the form, from the **global** list, select **true**.
   e. Click **Update**.
8. The **Updated config option log_to_file** notification displays and you are returned to the **Configuration** table.
9. Repeat step "7" on page 396 to configure the **global** value to **true** for the `mon_cluster_log_to_file` file. Search for `mon_cluster_log_to_file` instead of `log_to_file`.

   > **Important:** Both the `log_to_file` and `mon_cluster_log_to_file` files need to be configured.

10. Optional: To view the Ceph Object Gateway *ops_log*, `rgw_enable_ops_log` must be set to `true` by using the following command:

    ```
    $ ceph config set client.rgw rgw_enable_ops_log true
    ```

    a. Go to **Administration** -> **Configuration**.
    b. Change level from *basic* to *Dev*.
    c. Search for `rgw_enable_ops_log` and edit the value to `true`.
    d. From the **Daemon Logs** tab, locate the logs file in the **filename** field and run the query to view the ops log.
11. View the centralized logs by going to **Cluster › Logs** and select the **Daemon Logs** tab.
    The **Logs** view is displayed.

    > **Note:** If the **Daemon Logs** tab has a general **Welcome to Grafana** page and does not display the **Logs** view, sign in to Grafana and reload the page.

12. Use the **Log browser** field to select the relevant file and label values.
    "Figure: Using the Log browser" on page 397 shows an example of selecting the relevant labels and values for the search query.

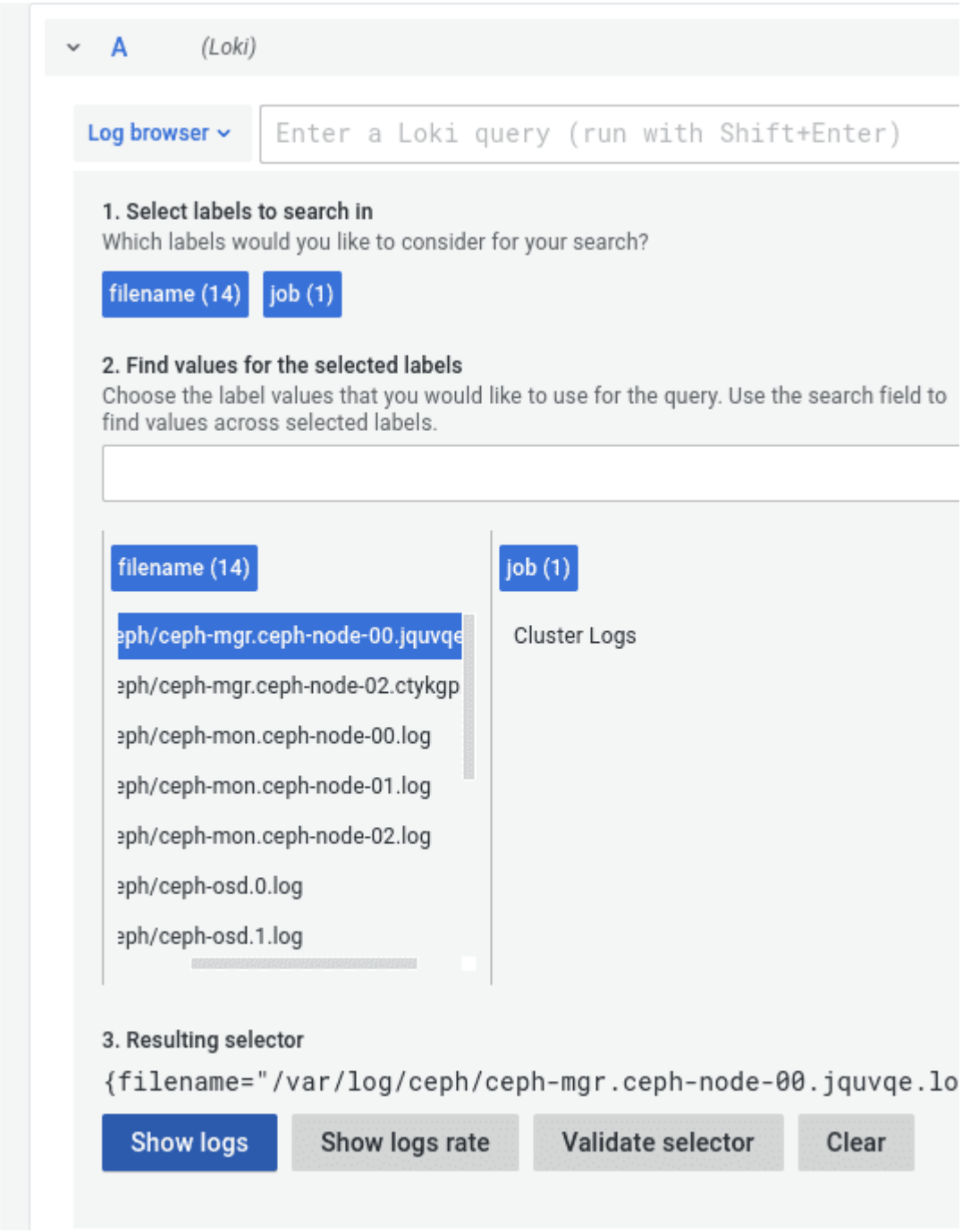


*Figure 50: Using the **Log browser***

13. Click **Show logs** to view the logs from the selected file.
"Figure: Viewing centralized logs" on page 398 shows the **Daemon Logs** view after a log query has been set.

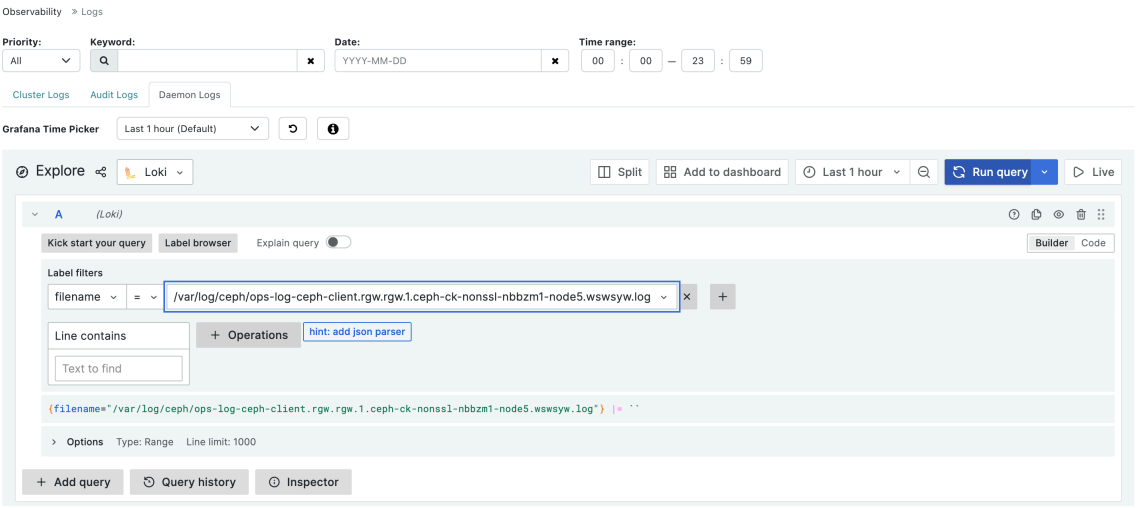


Figure 51: Viewing centralized logs

Viewing and managing dashboard data

Learn how to use the Red Hat Ceph Storage Dashboard to view cluster health, explore system data, and manage alerts.

Before you begin, make sure that you have the following prerequisites in place:

- Ensure that you can access the Red Hat Ceph Storage Dashboard. The Dashboard module is enabled by default in Red Hat-supported installations. No additional configuration is required.
- Sign in with username and password generated during bootstrap.

You can monitor cluster data, review system health, and manage alerts directly from the Dashboard interface.

The dashboard displays temporary loading indicators or empty states while data loads. Large tables are paginated to improve loading time and navigation. The **Status card** card summarizes cluster health and displays active alerts. You can view and verify alert states directly from the **Status card** card.

- To view cluster status and alerts, open the **Status** card on the main dashboard.
- To navigate large tables, open a page that lists multiple records, such as **OSDs** or **Hosts**. Use the pagination controls to move between pages or change the number of rows displayed. Enter search or filter criteria, and wait for the table to refresh.
- To view cluster status and alerts, open the **Status** card on the main dashboard. The **Status** card shows summarized cluster health and active alerts.
- Click **View alerts** to open the **Active Alerts** page.
  Expand an alert row to view details, including the description, severity, state, and Total occurrences. To view the alert source, click **Source** in the alert row. Alerts update automatically as cluster conditions change.

Monitoring pools of the Ceph cluster

A pool plays a critical role in how the Ceph storage cluster distributes and stores data. If you have deployed a cluster without creating a pool, Ceph uses the default pools for storing data.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Pools are created

About this task

Procedure

1. From the dashboard navigation, go to **Cluster › Pools**.
2. View the **Pools List** tab, which gives the details of Data protection and the application for which the pool is enabled.
   Hover the mouse over **Usage**, **Read bytes**, and **Write bytes** for the required details.
3. Expand the pool row for detailed information about a specific pool.

Monitoring Ceph File Systems

Use the Red Hat Ceph Storage Dashboard to monitor Ceph File Systems (CephFS) and related components.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- MDS service is deployed on at least one of the hosts.
- Ceph File System is installed.

About this task

For each File System listed, the following tabs are available:

Details

View the metadata servers (MDS) and their rank plus any standby daemons, pools and their usage,and performance counters.

Directories

View list of directories, their quotas and snapshots. Select a directory to set and unset maximum file and size quotas and to create and delete snapshots for the specific directory.

Subvolumes

Create, edit, and view subvolume information. These can be filtered by subvolume groups.

Subvolume groups

Create, edit, and view subvolume group information.

Snapshots

Create, clone, and view snapshot information. These can be filtered by subvolume groups and subvolumes.

Snapshot schedules

Enable, create, edit, and delete snapshot schedules.

Clients

View and evict Ceph File System client information.

Performance Details

View the performance of the file systems through the embedded Grafana Dashboard.

Procedure

1. From the dashboard navigation, go to **File › File Systems**.
2. To view more information about an individual file system, expand the file system row.

Monitoring Ceph Object Gateway daemons

Use the Red Hat Ceph Storage Dashboard to monitor Ceph Object Gateway daemons. You can view the details, performance counters and performance details of the Ceph Object Gateway daemons.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- At least one Ceph Object Gateway daemon configured in the storage cluster.

Procedure

1. From the dashboard navigation, go to **Object › Gateways**.
2. To view more information about a Ceph Object Gateway daemon, expand the Ceph Object Gateway row.
3. Optional: If you have configured multiple Ceph Object Gateway daemons, click on **Sync Performance** tab and view the multi-site performance counters.

Monitoring block device images

Use the Red Hat Ceph Storage Dashboard to monitor and manage block device images. You can view the details, snapshots, configuration details, and performance details of the images.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool with the `rbd` application enabled is created.
- An image is created.

Procedure

1. From the dashboard navigation, go to **Block › Images**.
2. View image information from the **Images** tab. Expand image rows for more information.
3. View namespace information from the **Namespaces** tab. Expand namespace rows for more information.

Stretch clusters for Ceph storage

As a storage administrator, you can configure a two-site stretched cluster by enabling stretch mode in Ceph.

Red Hat Ceph Storage, when configured with stretch mode on the cluster, offer the ability to expand the failure domain beyond the OSD or host level to span entire data centers or cloud availability zones, enhancing fault tolerance and availability in geographically distributed deployments.

“Figure: Stretch clusters for Ceph storage” on page 401 depicts a simplified representation of a Ceph cluster operating in stretch mode, where the tiebreaker host is provisioned in data center (DC) 3.

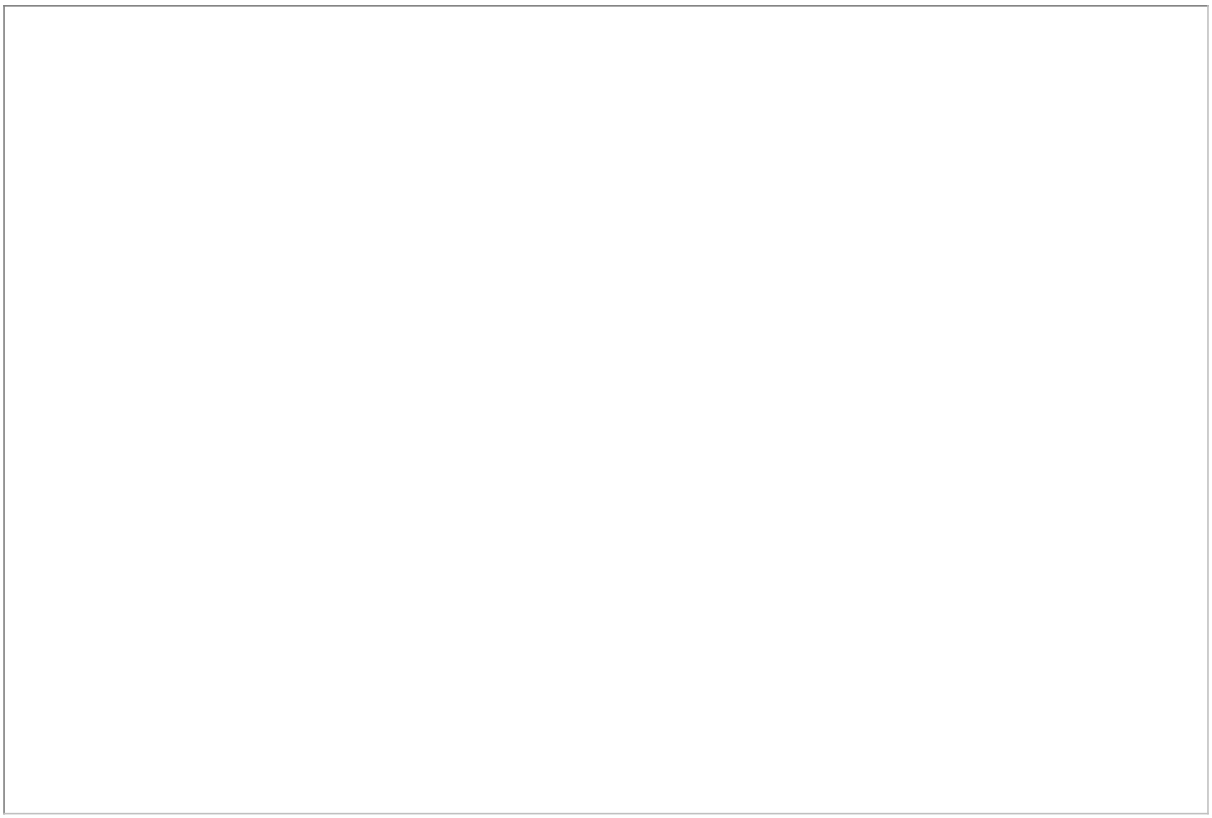


Figure 52: Stretch clusters for Ceph storage

A stretch cluster operates over a Wide Area Network (WAN), unlike a typical Ceph cluster, which operates over a Local Area Network (LAN).

For illustration purposes, a data center is chosen as the failure domain, though this could also represent a cloud availability zone. Data Center 1 (DC1) and Data Center 2 (DC2) contain OSDs and Monitors within their respective domains, while Data Center 3 (DC3) contains only a single monitor. The latency between DC1 and DC2 should not exceed 10 ms RTT, as higher latency can significantly impact Ceph performance in terms of replication, recovery, and related operations. However, DC3, a non-data site typically hosted on a virtual machine, can tolerate higher latency compared to the two data sites.

A stretch cluster, like the one depicted in “Figure: Stretch clusters for Ceph storage” on page 401, can withstand a complete data center failure or a network partition between data centers as long as at least two sites remain connected.

There are no additional steps for powering down a stretch cluster. For more information, see “Powering down and rebooting the cluster” on page 357.

Stretch mode

To improve availability in Stretched Clusters (geographically distributed deployments), you must enter the stretch mode. When stretch mode is enabled, the Ceph OSDs only take placement groups (PGs) as active when they peer across data centers, or whichever other CRUSH bucket type you specified, assuming both are active. Pools increase in size from the default three to four, with two copies on each site.

In stretch mode, Ceph OSDs are only allowed to connect to monitors within the same data center. New monitors are not allowed to join the cluster without specified location.

If all the OSDs and monitors from a data center become inaccessible at once, the surviving data center will enter a `degraded` stretch mode. This issues a warning, reduces the `min_size` to `1`, and allows the cluster to reach an `active` state with the data from the remaining site.

Stretch mode is designed to handle netsplit scenarios between two data centers and the loss of one data center. Stretch mode handles the netsplit scenario by choosing the surviving data center with a better connection to the *tiebreaker monitor*. Stretch mode handles the loss of one data center by reducing the `min_size` of all pools to `1`, allowing the cluster to continue operating with the remaining data center. When the lost data center comes back, the cluster will recover the lost data and return to normal operation.

Note: In a stretch cluster, when a site goes down and the cluster enters a `degraded` state, the `min_size` of the pool may be temporarily reduced (e.g., to 1) to allow the placement groups (PGs) to become active and continue serving I/O. However, the `size` of the pool remains unchanged. The **`peering_crush_bucket_count`** stretch mode flag ensures that PGs does not become active unless they are backed by OSDs in a minimum number of distinct CRUSH buckets (e.g., different data centers). This mechanism prevents the system from creating redundant copies solely within the surviving site, ensuring that data is only fully replicated once the downed site recovers."

When the missing data center becomes accessible again, the cluster enters `recovery` stretch mode. This changes the warning and allows peering, but still requires only the OSDs from the data center, which was up the whole time.

When all PGs are in a known state and are not degraded or incomplete, the cluster goes back to the regular stretch mode, ends the warning, and restores `min_size` to its starting value `2`. The cluster again requires both sites to peer, not only the site that stayed up the whole time, therefore you can fail over to the other site, if necessary.

For troubleshooting information, see Troubleshooting clusters in stretch mode.

Stretch mode limitations

- It is not possible to exit from stretch mode once it is entered.
- You cannot use erasure-coded pools with clusters in stretch mode. You can neither enter the stretch mode with erasure-coded pools, nor create an erasure-coded pool when the stretch mode is active.
- Device class is not supported in stretch mode. In the following example, the `class hdd` is not supported.

```
rule stretch_replicated_rule
{id 2
type replicated class hdd
step take default
step choose firstn 0 type datacenter
step chooseleaf firstn 2 type host
step emit
}
```

To achieve same weights on both sites, the Ceph OSDs deployed in the two sites should be of equal size, that is, storage capacity in the first site is equivalent to storage capacity in the second site.

- While it is not enforced, you should run two Ceph monitors on each site and a tiebreaker, for a total of five. This is because OSDs can only connect to monitors in their own site when in stretch mode.
- You have to create your own CRUSH rule, which provides two copies on each site, which totals to four on both sites.
- You cannot enable stretch mode if you have existing pools with non-default size or `min_size`.
- Because the cluster runs with `min_size 1` when degraded, you should only use stretch mode with all-flash OSDs. This minimizes the time needed to recover once connectivity is restored, and minimizes the potential for data loss.

Stretch peering rule

In Ceph stretch cluster mode, a critical safeguard is enforced through the *stretch peering rule*, which ensures that a Placement Group (PG) cannot become active if all acting replicas reside within a single failure domain, such as a single data center or cloud availability zone.

This behavior is essential for protecting data integrity during site failures. If a PG were allowed to go active with all replicas confined to one site, write operations could be falsely acknowledged without true redundancy. In the event of a site outage, this would result in complete data loss for those PGs. By enforcing zone diversity in the acting set, Ceph stretch clusters maintain high availability while minimizing the risk of data inconsistency or loss.

.

Deployment requirements

This information details important hardware, software, and network requirements that are needed for deploying a generalized stretch cluster configuration for three availability zones.

Software requirements

Red Hat Ceph Storage 9

Hardware requirements

Use the following minimum hardware requirements before deploying a stretch cluster configuration.

- ceph-osd hardware requirements
- ceph-mon hardware requirements
- ceph-mds hardware requirements

Table 55: ceph-osd hardware requirements

Hardware criteria	Minimum and recommended
Processor	– 1 core minimum, 2 recommended. – 1 core per 200-500 MB/s throughput. – 1 core per 1000-3000 IOPS. – Results are before replication. – Results can vary across CPU and drive models and Ceph configuration (erasure coding, and compression). – ARM processors specifically can require more cores for performance. – SSD OSDs, especially NVMe, benefit from extra cores per OSD. – Actual performance depends on various factors, including drives, network, and client throughput and latency. Bench marking is recommended to assess performance accurately.
RAM	– 4 GB or more per daemon is required (higher is recommended). – 2-4 GB can work, but performance might be slower. – Less than 2 GB is not recommended for optimal performance.
Network	A single 1 Gb/s (bonded 10+ Gb/s recommended). The network latency between the two data sites should be less than 10ms.

Table 56: ceph-mon hardware requirements

Hardware criteria	Minimum and recommended
Processor	2 cores minimum
Storage drives	100 GB per daemon. SSD is recommended.
Network	A single 1 Gb/s (10+ Gb/s recommended)

Table 57: ceph-mds hardware requirements	
Hardware criteria	**Minimum and recommended**
Processor	2 cores minimum
RAM	2 GB per daemon (more for production)
Disk space	1 GB per daemon
Network	A single 1 Gb/s (10+ Gb/s recommended)

Daemon placement

"Daemon placement" on page 404 lists the daemon placement details across various hosts and data centers.

Table 58: Daemon placement		
Hostname	**Data center**	**Services**
host01	DC1	OSD+MON+MGR
host02	DC1	OSD+MON+MGR
host03	DC1	OSD+MDS+RGW
host04	DC2	OSD+MON+MGR
host05	DC2	OSD+MON+MGR
host06	DC2	OSD+MDS+RGW
host07	DC3 (Tiebreaker)	MON

Network configuration requirements

Use the following network configuration requirements before deploying stretch cluster configuration.

> **Note:** You can use different subnets for each of the data centers.

- Have two separate networks, one public network and one cluster network.
- The latencies between data centers that run the Red Hat Ceph Storage Object Storage Devices (OSDs) cannot exceed 10 ms RTT.

The following is an example of a basic network configuration:

- DC1
  Ceph public/private network: 10.0.40.0/24
- DC2
  Ceph public/private network: 10.0.40.0/24
- Tiebreaker
  Ceph public/private network: 10.0.40.0/24

Cluster setup requirements

Ensure that the hostname is configured by using the bare or short hostname in all hosts.

```
hostnamectl set-hostname SHORT_NAME
```

> **Important:** The **`hostname`** command should only return the short hostname, when run on all nodes. If the FQDN is returned, the cluster configuration will not be successful.

Configuring a cluster for stretch mode

As a storage administrator, you can configure stretch mode in Ceph to distribute your cluster across multiple data centers, ensuring high availability and fault tolerance.

The following is an example user case for using the stretch mode feature.

A global e-commerce company with multiple regional data centers can implement stretch mode to ensure their Ceph storage cluster remains highly available and resilient. In the event of a failure in one data center, stretch mode ensures that the cluster continues to operate by redistributing workloads to other active sites, minimizing downtime and preventing data loss, thereby maintaining continuous service for customers worldwide.

Setting CRUSH location for daemons

Before you enter the stretch mode, you need to prepare the cluster by setting the CRUSH location to the daemons in the cluster. Add details of host locations in crush map for mon daemons & OSD hosts. This can either be done with the cluster deployment with spec file, or can be added on existing clusters as well.

Prepare the stretch mode in one of the following ways:

- Bootstrap the cluster through a service configuration file, where the locations are added to the hosts as part of deployment.
- Set the locations manually through **ceph osd crush add-bucket** and **ceph osd crush move** commands after the cluster is deployed.

Setting the CRUSH location during bootstrap

For hosts with multiple public networks, update the public networks in CIDR format, within the configuration file. Enhance the specification file by setting the monitor CRUSH location. Set this information during the bootstrap procedure.

Before you begin

Before you begin, be sure that you have root-level access to the nodes.

About this task

For more information about Ceph bootstrapping and different **cephadm bootstrap** command options, see Bootstrapping a new storage cluster.

Procedure

1. Create a `cluster-spec.yaml` file.
   The specification file adds the nodes to the Red Hat Ceph Storage cluster and also sets specific labels for where the services run.
   For example,

```
service_type: host
addr: <host01 address>
hostname: host01
location:
 root: default
 datacenter: DC1
labels:
 - osd
 - mon
 - mgr
---
service_type: host
addr: <host02 address>
hostname: host02
location:
 datacenter: DC1
labels:
 - osd
 - mon
 - mgr
---
service_type: host
addr: <host03 address>
hostname: host03
location:
 datacenter: DC1
labels:
 - osd
 - mds
 - rgw
```

```
---
service_type: host
addr: <host04 address>
hostname: host04
location:
 root: default
 datacenter: DC2
labels:
 - osd
 - mon
 - mgr
---
service_type: host
addr: host05
hostname: host05
location:
 datacenter: DC2
labels:
 - osd
 - mon
 - mgr
---
service_type: host
addr: <host06 address>
hostname: host06
location:
 datacenter: DC2
labels:
 - osd
 - mds
 - rgw
---
service_type: host
addr: <host07 address>
hostname: host07
labels:
 - mon
------
service_type: mon
spec:
 crush_locations:
   host01:
     - datacenter=DC1
   host02:
     - datacenter=DC1
   host04:
     - datacenter=DC2
   host05:
     - datacenter=DC2
placement:
  label: mon
---
service_type: mgr
service_name: mgr
placement:
 label: "mgr"
---
service_type: osd
service_id: all-available-devices
service_name: osd.all-available-devices
placement:
 label: "osd"
spec:
 data_devices:
   all: true
---
service_type: rgw
service_id: objectgw
service_name: rgw.objectgw
placement:
 count: 2
 label: "rgw"
spec:
 rgw_frontend_port: 8080
```

2. Run the **cephadm bootstrap** command as the root user on the node that will serve as the initial Monitor node in the cluster.
   The MONITOR_IP_ADDRESS value is the node's IP address that you are using to run the **cephadm bootstrap** command. Run one of the following commands, based on your configuration needs.

- For hosts that are present on the same network.

```
cephadm bootstrap --apply-spec CONFIGURATION_FILE_NAME --mon-ip
MONITOR_IP_ADDRESS --ssh-private-key PRIVATE_KEY --ssh-public-key PUBLIC_KEY --
registry-url REGISTRY_URL --registry-username USER_NAME --registry-password
PASSWORD
```

- For hosts that are present on different networks.

```
cephadm bootstrap --apply-spec CONFIGURATION_FILE_NAME --mon-ip
MONITOR_IP_ADDRESS --ssh-private-key PRIVATE_KEY --ssh-public-key PUBLIC_KEY --
registry-url REGISTRY_URL --registry-username USER_NAME --registry-password
PASSWORD --config ceph.conf
```

The following configuration in the `ceph.conf` file defines the public network settings for the cluster:

```
[global]
public_network = 10.1.172.0/23,10.0.64.0/22
```

After the bootstrap process completes, the following output is emitted.

```
Or, if you are only running a single cluster on this host:
    sudo /usr/sbin/cephadm shell
Please consider enabling telemetry to help improve Ceph:
    ceph telemetry on
For more information see:
    https://docs.ceph.com/en/latest/mgr/telemetry/
Bootstrap complete.
```

What to do next

1. Verify the network configuration.

```
ceph config dump | grep network
```

In the following example, the output confirms that there are multiple networks and that `10.1.172.0/23` and `10.0.64.0/22` are properly configured.

```
[ceph: root@host01 /]#  ceph config dump | grep network
global   advanced   public_network   10.1.172.0/23,10.0.64.0/22   *
```

2. Verify that the status of storage cluster deployment is in the HEALTH_OK state.
For example,

```
[ceph: root@host01 /]# ceph -s

 cluster:
   id:     ff19789c-f5c7-11ef-8e1c-fa163e4e1f7e
   health: HEALTH_OK

 services:
   mon: 5 daemons, quorum host01,host05,host02,host07-installer,host04 (age 10m)
   mgr: host05.aswlzq(active, since 43m), standbys: host02.ctajlt, host01.napqyw,
host04.wdglem
   mds: 1/1 daemons up, 1 standby
   osd: 24 osds: 24 up (since 31m), 24 in (since 32m)
   rgw: 4 daemons active (2 hosts, 1 zones)

 data:
   volumes: 1/1 healthy
   pools:   7 pools, 1019 pgs
   objects: 216 objects, 456 KiB
   usage:   2.7 GiB used, 357 GiB / 360 GiB avail
```

3. Verify that all the nodes that were added in the `cluster-spec.yaml` file are added to the cluster.

```
ceph orch host ls
```

For example,

```
[ceph: root@host01 /]# ceph orch host ls
HOST                                ADDR         LABELS       STATUS
host01            10.0.56.37   mgr,mon,osd
host02            10.0.59.35   mgr,mon,osd
host03            10.0.58.106  osd,mds,rgw
host04            10.0.56.13   osd,mon,mgr
host05            10.0.59.188  mgr,mon,osd
host06            10.0.56.223  rgw,mds,osd
host07            10.0.56.189  _admin,mon
7 hosts in cluster
```

4. Use the **ceph osd tree** command to verify the following:
   - CRUSH locations for the OSD hosts
   - That each host has one OSD configured
   - That each host OSD is in the up state
   - That each node is in the correct data center bucket

   For example,

```
[ceph: root@host01 /]# ceph osd tree
ID   CLASS  WEIGHT   TYPE NAME               STATUS REWEIGHT PRI-AFF
-1          0.35010  root default
-3          0.17505      datacenter DC1
-2          0.05835        host host01
5    hdd    0.01459              osd.5       up     1.00000  1.00000
11   hdd    0.01459             osd.11       up     1.00000  1.00000
17   hdd    0.01459             osd.17       up     1.00000  1.00000
23   hdd    0.01459             osd.23       up     1.00000  1.00000
-4          0.05835      host host02
1    hdd    0.01459              osd.1       up     1.00000  1.00000
6    hdd    0.01459              osd.6       up     1.00000  1.00000
12   hdd    0.01459             osd.12       up     1.00000  1.00000
18   hdd    0.01459             osd.18       up     1.00000  1.00000
-5          0.05835      host host03
3    hdd    0.01459              osd.3       up     1.00000  1.00000
10   hdd    0.01459             osd.10       up     1.00000  1.00000
16   hdd    0.01459             osd.16       up     1.00000  1.00000
22   hdd    0.01459             osd.22       up     1.00000  1.00000
-7          0.17505      datacenter DC2
-6          0.05835      host host04
2    hdd    0.01459              osd.2       up     1.00000  1.00000
8    hdd    0.01459              osd.8       up     1.00000  1.00000
14   hdd    0.01459             osd.14       up     1.00000  1.00000
20   hdd    0.01459             osd.20       up     1.00000  1.00000
-8          0.05835      host host05
0    hdd    0.01459              osd.0       up     1.00000  1.00000
7    hdd    0.01459              osd.7       up     1.00000  1.00000
13   hdd    0.01459             osd.13       up     1.00000  1.00000
19   hdd    0.01459             osd.19       up     1.00000  1.00000
-9          0.05835      host host06
4    hdd    0.01459              osd.4       up     1.00000  1.00000
9    hdd    0.01459              osd.9       up     1.00000  1.00000
15   hdd    0.01459             osd.15       up     1.00000  1.00000
21   hdd    0.01459             osd.21       up     1.00000  1.00000
```

5. Verify the CRUSH locations for the MON hosts. Check the `mon map` to ensure that each MON host has a `crush_location` specified.

```
ceph mon dump
```

   The output displays details about the MON map, including the `crush_location` for each host.

   For example,

```
[ceph: root@host01 /]# ceph mon dump

epoch 5

fsid 4158287e-169e-11f0-b1ad-fa163e98b991

last_changed 2025-04-11T06:32:20.332479+0000

created 2025-04-11T06:29:24.974553+0000

min_mon_release 19 (squid)

election_strategy: 1

0: [v2:10.0.57.33:3300/0,v1:10.0.57.33:6789/0] mon.host07

1: [v2:10.0.58.200:3300/0,v1:10.0.58.200:6789/0] mon.host05; crush_location
{datacenter=DC2}

2: [v2:10.0.58.47:3300/0,v1:10.0.58.47:6789/0] mon.host02; crush_location
{datacenter=DC1}

3: [v2:10.0.58.104:3300/0,v1:10.0.58.104:6789/0] mon.host04; crush_location
{datacenter=DC2}

4: [v2:10.0.58.38:3300/0,v1:10.0.58.38:6789/0] mon.host01; crush_location
{datacenter=DC1}
```

Manually setting the CRUSH location for daemons

Set the locations manually through **ceph osd crush add-bucket** and **ceph osd crush move** commands after the cluster is deployed.

Before you begin

Before you begin, be sure that you have root-level access to the nodes.

Procedure

1. Add two buckets to which you plan to set the location of your non-tiebreaker monitors to the CRUSH map. Specify the bucket type as `datacenter`.

   ```
   ceph osd crush add-bucket BUCKET_NAME datacenter
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph osd crush add-bucket DC1 datacenter
   [ceph: root@host01 /]# ceph osd crush add-bucket DC2 datacenter
   ```

2. . `root=default`.

   ```
   ceph osd crush move BUCKET_NAME root=default
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph osd crush move DC1 root=default
   [ceph: root@host01 /]# ceph osd crush move DC2 root=default
   ```

3. Move the OSD hosts, according to the required CRUSH placement.

   ```
   ceph osd crush move HOST datacenter=DATACENTER
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph osd crush move host01 datacenter=DC1
   ```

4. Verify the CRUSH locations for OSD hosts.

   ```
   ceph osd tree
   ```

For example,

```
[ceph: root@host01 /]# ceph osd tree
ID   CLASS  WEIGHT   TYPE NAME               STATUS REWEIGHT PRI-AFF
-1         0.35010  root default
-3         0.17505      datacenter DC1
-2         0.05835        host host01
5    hdd    0.01459              osd.5       up     1.00000  1.00000
11   hdd    0.01459             osd.11       up     1.00000  1.00000
17   hdd    0.01459             osd.17       up     1.00000  1.00000
23   hdd    0.01459             osd.23       up     1.00000  1.00000
-4         0.05835        host host02
1    hdd    0.01459              osd.1       up     1.00000  1.00000
6    hdd    0.01459              osd.6       up     1.00000  1.00000
12   hdd    0.01459             osd.12       up     1.00000  1.00000
18   hdd    0.01459             osd.18       up     1.00000  1.00000
-5         0.05835        host host03
3    hdd    0.01459              osd.3       up     1.00000  1.00000
10   hdd    0.01459             osd.10       up     1.00000  1.00000
16   hdd    0.01459             osd.16       up     1.00000  1.00000
22   hdd    0.01459             osd.22       up     1.00000  1.00000
-7         0.17505      datacenter DC2
-6         0.05835        host host04
2    hdd    0.01459              osd.2       up     1.00000  1.00000
8    hdd    0.01459              osd.8       up     1.00000  1.00000
14   hdd    0.01459             osd.14       up     1.00000  1.00000
20   hdd    0.01459             osd.20       up     1.00000  1.00000
-8         0.05835        host host05
0    hdd    0.01459              osd.0       up     1.00000  1.00000
7    hdd    0.01459              osd.7       up     1.00000  1.00000
13   hdd    0.01459             osd.13       up     1.00000  1.00000
19   hdd    0.01459             osd.19       up     1.00000  1.00000
-9         0.05835        host host06
4    hdd    0.01459              osd.4       up     1.00000  1.00000
9    hdd    0.01459              osd.9       up     1.00000  1.00000
15   hdd    0.01459             osd.15       up     1.00000  1.00000
21   hdd    0.01459             osd.21       up     1.00000  1.00000
```

5. Set the location of each monitor, matching your CRUSH map.

```
ceph mon set_location HOST datacenter=DATACENTER
```

For example,

```
[ceph: root@host01 /]# ceph mon set_location host01 datacenter=DC1
[ceph: root@host01 /]# ceph mon set_location host02 datacenter=DC1
[ceph: root@host01 /]# ceph mon set_location host04 datacenter=DC2
[ceph: root@host01 /]# ceph mon set_location host05 datacenter=DC2
```

6. Verify the CRUSH locations for the MON hosts.
Check the `mon map` to ensure that each MON host has a `crush_location` specified.

```
ceph mon dump
```

The output displays details about the MON map, including the `crush_location` for each host.

For example,

```
[ceph: root@host01 /]# ceph mon dump

epoch 5

fsid 4158287e-169e-11f0-b1ad-fa163e98b991

last_changed 2025-04-11T06:32:20.332479+0000

created 2025-04-11T06:29:24.974553+0000

min_mon_release 19 (squid)

election_strategy: 1

0: [v2:10.0.57.33:3300/0,v1:10.0.57.33:6789/0] mon.host07

1: [v2:10.0.58.200:3300/0,v1:10.0.58.200:6789/0] mon.host05; crush_location
{datacenter=DC2}

2: [v2:10.0.58.47:3300/0,v1:10.0.58.47:6789/0] mon.host02; crush_location
{datacenter=DC1}

3: [v2:10.0.58.104:3300/0,v1:10.0.58.104:6789/0] mon.host04; crush_location
{datacenter=DC2}

4: [v2:10.0.58.38:3300/0,v1:10.0.58.38:6789/0] mon.host01; crush_location
{datacenter=DC1}
```

Configuring a CRUSH map for stretch mode

Use this information to configure a CRUSH map for stretch mode. In this step, we modify few configs on the cluster, add new crush rule and enable stretch mode on the cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root-level access to the nodes.
- The CRUSH location is set to the hosts.

Procedure

1. Create a CRUSH rule that makes use of this OSD crush topology by installing the ceph-base RPM package in order to use the **`crushtool`** command.

   ```
   dnf -y install ceph-base
   ```

2. Get the compiled CRUSH map from the cluster.

   ```
   ceph osd getcrushmap > /etc/ceph/crushmap.bin
   ```

3. Decompile the CRUSH map and convert it to a text file to edit it.

   ```
   crushtool -d /etc/ceph/crushmap.bin -o /etc/ceph/crushmap.txt
   ```

4. Add the following rule to the CRUSH map by editing the `/etc/ceph/crushmap.txt` at the end of the file.
   This rule distributes reads and writes evenly across the data center.

   ```
   rule stretch_rule {
           id 1
           type replicated
           step take default
           step choose firstn 0 type datacenter
           step chooseleaf firstn 2 type host
           step emit
    }
   ```

   a. Optionally have the cluster with a read/write affinity towards data center 1.

```
rule stretch_rule {
         id 1
         type replicated
         step take DC1
         step chooseleaf firstn 2 type host
         step emit
         step take DC2
         step chooseleaf firstn 2 type host
         step emit
 }
```

```
The CRUSH rule declared contains the following information:
     Rule name
          Description: A unique name for identifying the rule.
          Value: stretch_rule
     id
          Description: A unique whole number for identifying the rule.
          Value: 1
     type
          Description: Describes a rule for either a storage drive replicated or
erasure-coded.
          Value: replicated
     step take default
          Description: Takes the root bucket called default, and begins iterating
down the tree.
     step take DC1
          Description: Takes the bucket called DC1, and begins iterating down the
tree.
     step choose firstn 0 type datacenter
          Description: Selects the datacenter bucket, and goes into its subtrees.
     step chooseleaf firstn 2 type host
          Description: Selects the number of buckets of the given type. In this
case, it is two different hosts located in the datacenter it entered at the
previous level.
     step emit
          Description: Outputs the current value and empties the stack. Typically
used at the end of a rule, but may also be used to pick from different trees in
the same rule.
```

5. Compile the new CRUSH map from `/etc/ceph/crushmap.txt` and convert it to a binary file `/etc/ceph/crushmap2.bin`.

   ```
   crushtool -c /path/to/crushmap.txt -o /path/to/crushmap2.bin
   ```

   For example,

   ```
   [ceph: root@host01 /]# crushtool -c /etc/ceph/crushmap.txt -o /etc/ceph/crushmap2.bin
   ```

6. Inject the newly created CRUSH map back into the cluster.

   ```
   ceph osd setcrushmap -i /path/to/compiled_crushmap
   ```

   For example,

   ```
   ceph osd setcrushmap -i /etc/ceph/crushmap2.bin
   ```

   Note: The number 17 is a counter and increases (18,19, and so on) depending on the changes that are made to the CRUSH map.

What to do next

Verify that the newly created `stretch_rule` available for use.

```
ceph osd crush rule ls
```

For example,

```
[ceph: root@host01 /]# ceph osd crush rule ls

replicated_rule
stretch_rule
```

Changing stretch mode

Change the stretch mode state by entering or exiting stretch mode as needed to support your cluster's availability and data-placement requirements.

Entering stretch mode

Stretch mode is designed to handle two sites. There is a lesser risk of component availability outages with 2-site clusters.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root-level access to the nodes.
- The CRUSH location is set to the hosts.
- The CRUSH map configured to include stretch rule.
- No erasure coded pools in the cluster.
- Weights of the two sites are the same.

Procedure

1. Check the current election strategy being used by the monitors.

   ```
   ceph mon dump | grep election_strategy
   ```

   > **Note:** The Ceph cluster `election_strategy` is set to `1`, by default.

   For example,

   ```
   [ceph: root@host01 /]# ceph mon dump | grep election_strategy

   dumped monmap epoch 9
   election_strategy: 1
   ```

2. Change the election strategy to `connectivity`.

   ```
   ceph mon set election_strategy connectivity
   ```

   For more information about configuring the election strategy, see Configuring monitor election strategy.

3. Use the **ceph mon dump** command to verify that the election strategy was updated to `3`.
   For example,

   ```
   [ceph: root@host01 /]# ceph mon dump | grep election_strategy

   dumped monmap epoch 22
   election_strategy: 3
   ```

4. Set the location of the tiebreaker monitor so that it is split across the data centers.

   ```
   ceph mon set_location TIEBREAKER_HOST datacenter=DC3
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph mon set_location host07 datacenter=DC3
   ```

5. Verify that the tiebreaker monitor is set as expected.

```
ceph mon dump
```

For example,

```
[ceph: root@host01 /]# ceph mon dump

epoch 8

fsid 4158287e-169e-11f0-b1ad-fa163e98b991

last_changed 2025-04-11T07:14:48.652801+0000

created 2025-04-11T06:29:24.974553+0000

min_mon_release 19 (squid)

election_strategy: 3

0: [v2:10.0.57.33:3300/0,v1:10.0.57.33:6789/0] mon.host07; crush_location
{datacenter=DC3}

1: [v2:10.0.58.200:3300/0,v1:10.0.58.200:6789/0] mon.host05; crush_location
{datacenter=DC2}

2: [v2:10.0.58.47:3300/0,v1:10.0.58.47:6789/0] mon.host02; crush_location
{datacenter=DC1}

3: [v2:10.0.58.104:3300/0,v1:10.0.58.104:6789/0] mon.host04; crush_location
{datacenter=DC2}

4: [v2:10.0.58.38:3300/0,v1:10.0.58.38:6789/0] mon.host01; crush_location
{datacenter=DC1}

dumped monmap epoch 8
0
```

6. Enter stretch mode.

```
ceph mon enable_stretch_mode TIEBREAKER_HOST STRETCH_RULE STRETCH_BUCKET
```

In the following example:

- The tiebreaker node is set as `host07`.
- The stretch rule is `stretch_rule`, as created in “Configuring a CRUSH map for stretch mode” on page 411.
- The stretch bucket is set as `datacenter`.

```
[ceph: root@host01 /]# ceph mon enable_stretch_mode host07 stretch_rule datacenter
```

What to do next

Verify that stretch mode was implemented correctly by continuing to “Verifying stretch mode” on page 414.

Exiting stretch mode

Disable stretch mode by moving pools to a specified CRUSH rule or to the default replicated rule.

Procedure

- Disable stretch mode.
  You can specify a CRUSH rule to move all pools to. If you do not specify a rule, Ceph moves the pools to the default replicated CRUSH rule.

```
ceph mon disable_stretch_mode CRUSH_RULE --yes-i-really-mean-it
```

Verifying stretch mode

Use this information to verify that stretch mode was created correctly with the implemented CRUSH rules.

Procedure

1. Verify that all pools are using the CRUSH rule that was created in the Ceph cluster.
   In these examples, the CRUSH rule is set as `stretch_rule`, per the settings that were created in "Configuring a CRUSH map for stretch mode" on page 411.

   ```
   for pool in $(rados lspools);do echo -n "Pool: ${pool}; ";ceph osd pool get ${pool}
   crush_rule;done
   ```

   For example,

   ```
   [ceph: root@host01 /]# for pool in $(rados lspools);do echo -n "Pool: ${pool}; ";ceph
   osd pool get ${pool} crush_rule;done
   Pool: device_health_metrics; crush_rule: stretch_rule
   Pool: cephfs.cephfs.meta; crush_rule: stretch_rule
   Pool: cephfs.cephfs.data; crush_rule: stretch_rule
   Pool: .rgw.root; crush_rule: stretch_rule
   Pool: default.rgw.log; crush_rule: stretch_rule
   Pool: default.rgw.control; crush_rule: stretch_rule
   Pool: default.rgw.meta; crush_rule: stretch_rule
   Pool: rbdpool; crush_rule: stretch_rule
   ```

2. Verify that stretch mode is enabled.
   Ensure that `stretch_mode_enabled` is set to `true`.

   ```
   ceph osd dump
   ```

   The output includes the following information:

   `stretch_mode_enabled`

   Set to `true` if stretch mode is enabled.

   `stretch_bucket_count`

   The number of data centers with OSDs.

   `degraded_stretch_mode`

   Output of `0` if not degraded. If the stretch mode is degraded, this outputs the number of up sites.

   `recovering_stretch_mode`

   Output of `0` if not recovering. If the stretch mode is recovering, the output is `1`.

   `stretch_mode_bucket`

   A unique value set for each CRUSH bucket type. This value is usually set to 8, for data center.

   For example,

   ```
   stretch_mode_enabled true

   stretch_bucket_count 2

   degraded_stretch_mode 0

   recovering_stretch_mode 0

   stretch_mode_bucket 8
   ```

3. Verify that stretch mode is using the mon map, by using the **`ceph mon dump`** command.
   Ensure the following:
   - `stretch_mode_enabled` is set to `1`
   - The correct mon host is set as `tiebreaker_mon`
   - The correct mon host is set as `disallowed_leaders`

   ```
   ceph mon dump
   ```

   For example,

```
[ceph: root@host01 /]# ceph mon dump
epoch 16
fsid ff19789c-f5c7-11ef-8e1c-fa163e4e1f7e
last_changed 2025-02-28T12:12:51.089706+0000
created 2025-02-28T11:34:59.325503+0000
min_mon_release 19 (squid)
election_strategy: 3
stretch_mode_enabled 1
tiebreaker_mon host07
disallowed_leaders host07
0: [v2:10.0.56.37:3300/0,v1:10.0.56.37:6789/0] mon.host01; crush_location
{datacenter=DC1}
1: [v2:10.0.59.188:3300/0,v1:10.0.59.188:6789/0] mon.host05; crush_location
{datacenter=DC2}
2: [v2:10.0.59.35:3300/0,v1:10.0.59.35:6789/0] mon.host02; crush_location
{datacenter=DC1}
3: [v2:10.0.56.189:3300/0,v1:10.0.56.189:6789/0] mon.host07; crush_location
{datacenter=DC3}
4: [v2:10.0.56.13:3300/0,v1:10.0.56.13:6789/0] mon.host04; crush_location
{datacenter=DC2}
dumped monmap epoch 16
```

What to do next

1. Deploy, configure, and administer a Ceph Object Gateway. For more information, see Ceph Object Gateway.
2. Manage, create, configure, and use Ceph Block Devices. For more information, see Using Ceph Block Devices.
3. Create, mount, and work the Ceph File System (CephFS). For more information, see Ceph File Systems.

Using and maintaining stretch mode

Use and maintain stretch mode by adding OSD hosts, managing data center monitor service hosts, and replacing tiebreakers with a monitor both with and without a quorum.

Adding OSD hosts in stretch mode

You can add Ceph OSDs in the stretch mode. The procedure is similar to the addition of the OSD hosts on a cluster where stretch mode is not enabled.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Stretch mode is enabled on a cluster.
- Root-level access to the nodes.

Procedure

1. List the available devices to deploy OSDs.

   ```
   ceph orch device ls [--hostname=HOST_1 HOST_2] [--wide] [--refresh]
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch device ls
   ```

2. Deploy the OSDs on specific hosts or on all the available devices.
   - Create an OSD from a specific device on a specific host.

     ```
     ceph orch daemon add osd HOST:DEVICE_PATH
     ```

     For example,

```
[ceph: root@host01 /]# ceph orch daemon add osd host03:/dev/sdb
```

- Deploy OSDs on any available and unused devices.

  Important: This command creates collocated WAL and DB devices. Do not use this command to create non-collocated devices.

  ```
  ceph orch apply osd --all-available-devices
  ```

3. Move the OSD hosts under the CRUSH bucket.

   ```
   ceph osd crush move HOST datacenter=DATACENTER
   ```

   Note: Ensure you add the same topology nodes on both sites. Issues can occur if hosts are added only on one site.

   For example,

   ```
   [ceph: root@host01 /]# ceph osd crush move host03 datacenter=DC1
    [ceph: root@host01 /]# ceph osd crush move host06 datacenter=DC2
   ```

What to do next

Verify that the OSD hosts and OSDs are added in the CRUSH bucket, by using the **`ceph osd tree`** command.

Related information

Adding OSDs

Managing data center monitor service hosts in stretch mode

Use this information to add and remove data center monitor service (`mon`) hosts in stretch mode. Managing data centers can be done by using the specification file or directly on the Ceph cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Stretch mode is enabled on a cluster.
- Root-level access to the nodes.

Managing a `mon` service with a service specification file

About this task

These steps detail how to add a `mon` service. To remove the service, use the same steps of updating the service specification file, with removing the needed information.

Procedure

1. Export the specification file for `mon` and save the output to `mon-spec.yaml`.

   ```
   ceph orch ls mon --export > mon-spec.yaml
   ```

   After the file is exported, the YAML file can be edited.

2. Add the new host details.
   In the following example, `host08` is being added to the cluster into the DC2 data center bucket.

```
service_type: host

addr: 10.1.172.225

hostname: host08

labels:

- mon

---

service_type: mon

service_name: mon

placement:

label: mon

spec:

crush_locations:

host01:

- datacenter=DC1

host02:

- datacenter=DC1

host04:

- datacenter=DC2

host05:

- datacenter=DC2

host08:

- datacenter=DC2
```

3. Apply the specification file.

```
ceph orch apply -i mon-spec.yaml
```

For example,

```
[ceph: root@host01 /]# eph orch apply -i mon-spec.yaml
Added host 'host08' with addr '10.1.172.225'
Scheduled mon update...
```

What to do next

1. Use the **ceph mon dump** command to verify that the `mon` service was deployed and that the appropriate CRUSH location was added to the monitor.
For example,

```
[ceph: root@host01 /]# ceph mon dump

epoch 10

fsid 4158287e-169e-11f0-b1ad-fa163e98b991

last_changed 2025-04-11T08:45:13.049347+0000

created 2025-04-11T06:29:24.974553+0000

min_mon_release 19 (squid)

election_strategy: 3

stretch_mode_enabled 1

tiebreaker_mon host07

disallowed_leaders host07

0: [v2:10.0.57.33:3300/0,v1:10.0.57.33:6789/0] mon.host07; crush_location
{datacenter=DC3}

1: [v2:10.0.58.200:3300/0,v1:10.0.58.200:6789/0] mon.host05; crush_location
{datacenter=DC2}

2: [v2:10.0.58.47:3300/0,v1:10.0.58.47:6789/0] mon.host02; crush_location
{datacenter=DC1}

3: [v2:10.0.58.104:3300/0,v1:10.0.58.104:6789/0] mon.host04; crush_location
{datacenter=DC2}

4: [v2:10.0.58.38:3300/0,v1:10.0.58.38:6789/0] mon.host01; crush_location
{datacenter=DC1}

5: [v2:10.0.66.96:3300/0,v1:10.0.66.96:6789/0] mon.host08; crush_location
{datacenter=DC2}

dumped monmap epoch 10
```

2. Use the **`ceph orch host ls`** command to verify that the host was added to the cluster. For example,

```
[ceph: root@host01 /]# ceph orch host ls

HOST ADDR LABELS STATUS

host01 10.0.56.37 mgr,mon,osd

host02 10.0.59.35 mgr,mon,osd

host03 10.0.58.106 osd,mds,rgw

host04 10.0.56.13 osd,mon,mgr

host05 10.0.59.188 mgr,mon,osd

host06 10.0.56.223 rgw,mds,osd

host07 10.0.56.189 _admin,mon

host08 10.1.172.225 mon

8 hosts in cluster
```

Managing a `mon` with the command-line interface

About this task

These steps detail how to add a `mon` service. To remove the service, use the same steps of updating with the CLI, with removing the needed information.

Procedure

1. Set the monitor service tounmanaged.

```
ceph orch set-unmanaged mon
```

2. Optional: Use the **ceph orch ls** command to verify that the service was set, as expected.
For example,

```
[ceph: root@host01 /]# ceph orch host ls
NAME                                PORTS             RUNNING  REFRESHED  AGE
 PLACEMENT
mon                                                       8/8  10m ago    19s
 <unmanaged>
```

3. Add a new host with the mon label.

```
ceph orch host add HOST_NAME IP_ADDRESS_OF_HOST [--label=LABEL_NAME_1,LABEL_NAME_2]
```

For example,

```
[ceph: root@host01 /]# ceph orch host add host08 10.1.172.205 --labels=mon
```

4. Add a monitor service with CRUSH locations.

> **Note:** At this point, the mon is not running and is not managed by cephadm.

```
ceph mon add NODE:IP_ADDRESS datacenter=DC2
```

For example,

```
[ceph: root@host01 /]# ceph mon add host08 10.1.172.205 datacenter=DC2
```

5. Deploy the monitor daemon using cephadm.

```
ceph orch daemon add mon host08
```

For example,

```
[ceph: root@host01 /]# ceph orch daemon add mon host08
Deployed mon.host08 on host 'host08'
```

6. Enable cephadm management for the monitor service.

```
ceph orch set-managed mon
```

7. Start the newly added mon daemon.

```
ceph orch set-managed mgr
```

What to do next

Verify that the service, monitor, and host are added and running.

1. Use the **ceph orch ls** command to verify that the service is running.
For example,

```
[ceph: root@host01 /]# ceph orch ls

NAME PORTS RUNNING  REFRESHED  AGE PLACEMENT

mon   8/8  7m ago 4d label:mon
```

2. Use the **ceph mon dump** command to verify that the mon service was deployed and that the appropriate CRUSH location was added to the monitor.
For example,

```
[ceph: root@host01 /]# ceph mon dump
epoch 16
fsid ff19789c-f5c7-11ef-8e1c-fa163e4e1f7e
last_changed 2025-02-28T12:12:51.089706+0000
created 2025-02-28T11:34:59.325503+0000
min_mon_release 19 (squid)
election_strategy: 3
stretch_mode_enabled 1
tiebreaker_mon host07
disallowed_leaders host07
0: [v2:10.0.56.37:3300/0,v1:10.0.56.37:6789/0] mon.host01; crush_location
{datacenter=DC1}
1: [v2:10.0.59.188:3300/0,v1:10.0.59.188:6789/0] mon.host05; crush_location
{datacenter=DC2}
2: [v2:10.0.59.35:3300/0,v1:10.0.59.35:6789/0] mon.host02; crush_location
{datacenter=DC1}
3: [v2:10.0.56.189:3300/0,v1:10.0.56.189:6789/0] mon.host07; crush_location
{datacenter=DC3}
4: [v2:10.0.56.13:3300/0,v1:10.0.56.13:6789/0] mon.host04; crush_location
{datacenter=DC2}
dumped monmap epoch 16
```

3. Use the **ceph orch host ls** command to verify that the host was added to the cluster.
   For example,

```
[ceph: root@host01 /]# ceph orch host ls

HOST ADDR LABELS STATUS

host01 10.0.56.37 mgr,mon,osd

host02 10.0.59.35 mgr,mon,osd

host03 10.0.58.106 osd,mds,rgw

host04 10.0.56.13 osd,mon,mgr

host05 10.0.59.188 mgr,mon,osd

host06 10.0.56.223 rgw,mds,osd

host07 10.0.56.189 _admin,mon

host08 10.1.172.205 mon

8 hosts in cluster
```

Replacing a tiebreaker with a monitor in quorum

If your tiebreaker monitor fails, you can replace it with an existing monitor in quorum and remove it from the cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Stretch mode is enabled on a cluster.

Procedure

1. Disable automated monitor deployment.

```
ceph orch apply mon --unmanaged
```

   For example,

```
[ceph: root@host01 /]# ceph orch apply mon --unmanaged

 Scheduled mon update…
```

2. View the monitors in quorum, by using the **ceph -s** command.
   For example,

```
[ceph: root@host01 /]# ceph -s

 mon: 5 daemons, quorum host01, host02, host04, host05 (age 30s), out of quorum:
host07
```

3. Set the monitor in quorum as a new tiebreaker.

```
ceph mon set_new_tiebreaker NEW_HOST
```

For example,

```
[ceph: root@host01 /]# ceph mon set_new_tiebreaker host02
```

Important: If the monitor is in the same location as existing nontiebreaker monitors, an error message is displayed.
For example,

```
[ceph: root@host01 /]# ceph mon set_new_tiebreaker host02

  Error EINVAL: mon.host02 has location DC1, which matches mons host02 on the
datacenter dividing bucket for stretch mode.
```

If there is an error message, change the location of the monitor.

```
ceph mon set_location HOST datacenter=DATACENTER
```

For example,

```
[ceph: root@host01 /]# ceph mon set_location host02 datacenter=DC3
```

4. Remove the failed tiebreaker monitor.

```
ceph orch daemon rm FAILED_TIEBREAKER_MONITOR --force
```

For example,

```
[ceph: root@host01 /]# ceph orch daemon rm mon.host07 --force

 Removed mon.host07 from host 'host07'
```

5. Redeploy the monitor after the monitor is removed from the host.

```
ceph mon add HOST IP_ADDRESS datacenter=DATACENTER
 ceph orch daemon add mon HOST
```

For example,

```
[ceph: root@host01 /]# ceph mon add host07 213.222.226.50 datacenter=DC1
 [ceph: root@host01 /]# ceph orch daemon add mon host07
```

6. Ensure that there are 5 monitors in quorum, by using the **ceph -s** command.

```
[ceph: root@host01 /]# ceph -s

 mon: 5 daemons, quorum host01, host02, host04, host05, host07 (age 15s)
```

7. Verify that everything is configured properly.

```
ceph mon dump
```

For example,

```
[ceph: root@host01 /]# ceph mon dump

 epoch 19
 fsid 1234ab78-1234-11ed-b1b1-de456ef0a89d
 last_changed 2023-01-17T04:12:05.709475+0000
 created 2023-01-16T05:47:25.631684+0000
 min_mon_release 16 (pacific)
 election_strategy: 3
 stretch_mode_enabled 1
 tiebreaker_mon host02
 disallowed_leaders host02
 0: [v2:132.224.169.63:3300/0,v1:132.224.169.63:6789/0] mon.host02; crush_location
{datacenter=DC3}
 1: [v2:220.141.179.34:3300/0,v1:220.141.179.34:6789/0] mon.host04; crush_location
{datacenter=DC2}
 2: [v2:40.90.220.224:3300/0,v1:40.90.220.224:6789/0] mon.host01; crush_location
{datacenter=DC1}
 3: [v2:60.140.141.144:3300/0,v1:60.140.141.144:6789/0] mon.host07; crush_location
{datacenter=DC1}
 4: [v2:186.184.61.92:3300/0,v1:186.184.61.92:6789/0] mon.host03; crush_location
{datacenter=DC2}
 dumped monmap epoch 19
```

8. Redeploy the monitors.

```
ceph orch apply mon --placement="HOST_1, HOST_2, HOST_3, HOST_4, HOST_5”
```

For example,

```
[ceph: root@host01 /]# ceph orch apply mon --placement="host01, host02, host04,
host05, host07"

 Scheduled mon update...
```

Replacing a tiebreaker with a new monitor

When a tiebreaker monitor fails you can replace it with a new monitor and remove the failed one from the cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Stretch mode in enabled on a cluster.

Procedure

1. Add a new monitor to the cluster.
   a. Manually add the `crush_location` to the new monitor.

   > **Note:** The new monitor must be in a different location than existing non-tiebreaker monitors.

   ```
   ceph mon add NEW_HOST IP_ADDRESS datacenter=DATACENTER
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph mon add host06 213.222.226.50 datacenter=DC3

    adding mon.host06 at [v2:213.222.226.50:3300/0,v1:213.222.226.50:6789/0]
   ```

   b. Disable the automated monitor deployment.
   For example,

   ```
   [ceph: root@host01 /]# ceph orch apply mon --unmanaged

    Scheduled mon update…
   ```

c. Deploy the new monitor.

```
ceph orch daemon add mon NEW_HOST
```

For example,

```
[ceph: root@host01 /]# ceph orch daemon add mon host06
```

2. Ensure that there are six monitors.
Five of the monitors must be in quorum.
For example,

```
[ceph: root@host01 /]# ceph -s

 mon: 6 daemons, quorum host01, host02, host04, host05, host06 (age 30s), out of
quorum: host07
```

3. Set the new monitor as a new tiebreaker.

```
ceph mon set_new_tiebreaker NEW_HOST
```

For example,

```
[ceph: root@host01 /]# ceph mon set_new_tiebreaker host06
```

4. Remove the failed tiebreaker monitor.

```
ceph orch daemon rm FAILED_TIEBREAKER_MONITOR --force
```

For example,

```
[ceph: root@host01 /]# ceph orch daemon rm mon.host07 --force

  Removed mon.host07 from host 'host07'
```

5. Verify that everything is properly configured.
For example,

```
[ceph: root@host01 /]# ceph mon dump

 epoch 19
 fsid 1234ab78-1234-11ed-b1b1-de456ef0a89d
 last_changed 2023-01-17T04:12:05.709475+0000
 created 2023-01-16T05:47:25.631684+0000
 min_mon_release 16 (pacific)
 election_strategy: 3
 stretch_mode_enabled 1
 tiebreaker_mon host06
 disallowed_leaders host06
 0: [v2:213.222.226.50:3300/0,v1:213.222.226.50:6789/0] mon.host06; crush_location
{datacenter=DC3}
 1: [v2:220.141.179.34:3300/0,v1:220.141.179.34:6789/0] mon.host04; crush_location
{datacenter=DC2}
 2: [v2:40.90.220.224:3300/0,v1:40.90.220.224:6789/0] mon.host01; crush_location
{datacenter=DC1}
 3: [v2:60.140.141.144:3300/0,v1:60.140.141.144:6789/0] mon.host02; crush_location
{datacenter=DC1}
 4: [v2:186.184.61.92:3300/0,v1:186.184.61.92:6789/0] mon.host05; crush_location
{datacenter=DC2}
 dumped monmap epoch 19
```

6. Redeploy the monitors.

```
ceph orch apply mon --placement="HOST_1, HOST_2, HOST_3, HOST_4, HOST_5”
```

For example,

```
[ceph: root@host01 /]# ceph orch apply mon --placement="host01, host02, host04,
host05, host06"

 Scheduled mon update…
```

Read Affinity in stretch clusters

Read Affinity reduces cross-zone traffic by keeping the data access within the respective data centers.

For stretched clusters deployed in multi-zone environments, the read affinity topology implementation provides a mechanism to help keep traffic within the data center it originated from. Ceph Object Gateway volumes have the ability to read data from an OSD in proximity to the client, according to OSD locations defined in the CRUSH map and topology labels on nodes.

For example, a stretch cluster contains a Ceph Object Gateway primary OSD and replicated OSDs spread across two data centers A and B. If a *GET* action is performed on an *Object* in data center A, the *READ* operation is performed on the data of the OSDs closest to the client in data center A.

Performing localized reads

Run a localized read on a replicated pool in a stretch cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A stretch cluster with two data centers and a Ceph Object Gateway that is configured on both.
- A user who is created with a bucket having primary and replicated OSDs.

About this task

When a localized read request is made on a replicated pool, Ceph selects the local OSDs closest to the client based on the client location specified in **`crush_location`**.

Procedure

- To run a localized read, set **`rados_replica_read_policy`** to `localize` in the OSD daemon configuration by using the **`ceph config set`** command.

```
ceph config set client.rgw.rgw.1 rados_replica_read_policy localize
```

What to do next

Verify the localized read from an OSD set.

1. Run the **`ceph osd tree`** command to view the OSDs and the data centers.
   For example,

```
[ceph: root@host01 /]# ceph osd tree

ID  CLASS  WEIGHT   TYPE NAME                                STATUS  REWEIGHT  PRI-
AFF
-1         0.58557  root default
-3         0.29279      datacenter DC1
-2         0.09760          host ceph-ci-fbv67y-ammmck-node2
 2    hdd  0.02440              osd.2                            up   1.00000
 1.00000
11    hdd  0.02440              osd.11                           up   1.00000
 1.00000
17    hdd  0.02440              osd.17                           up   1.00000
 1.00000
22    hdd  0.02440              osd.22                           up   1.00000
 1.00000
-4         0.09760          host ceph-ci-fbv67y-ammmck-node3
 0    hdd  0.02440              osd.0                            up   1.00000
 1.00000
 6    hdd  0.02440              osd.6                            up   1.00000
 1.00000
12    hdd  0.02440              osd.12                           up   1.00000
 1.00000
18    hdd  0.02440              osd.18                           up   1.00000
 1.00000
-5         0.09760          host ceph-ci-fbv67y-ammmck-node4
 5    hdd  0.02440              osd.5                            up   1.00000
 1.00000
10    hdd  0.02440              osd.10                           up   1.00000
 1.00000
16    hdd  0.02440              osd.16                           up   1.00000
 1.00000
23    hdd  0.02440              osd.23                           up   1.00000
 1.00000
-7         0.29279      datacenter DC2
-6         0.09760          host ceph-ci-fbv67y-ammmck-node5
 3    hdd  0.02440              osd.3                            up   1.00000
 1.00000
 8    hdd  0.02440              osd.8                            up   1.00000
 1.00000
14    hdd  0.02440              osd.14                           up   1.00000
 1.00000
20    hdd  0.02440              osd.20                           up   1.00000
 1.00000
-8         0.09760          host ceph-ci-fbv67y-ammmck-node6
 4    hdd  0.02440              osd.4                            up   1.00000
 1.00000
 9    hdd  0.02440              osd.9                            up   1.00000
 1.00000
15    hdd  0.02440              osd.15                           up   1.00000
 1.00000
21    hdd  0.02440              osd.21                           up   1.00000
 1.00000
-9         0.09760          host ceph-ci-fbv67y-ammmck-node7
 1    hdd  0.02440              osd.1                            up   1.00000
 1.00000
 7    hdd  0.02440              osd.7                            up   1.00000
 1.00000
13    hdd  0.02440              osd.13                           up   1.00000
 1.00000
19    hdd  0.02440              osd.19                           up   1.00000
 1.00000
```

2. Run the **ceph orch** command to identify the Ceph Object Gateway daemons in the data centers. For example,

```
[ceph: root@host01 /]# ceph orch ps | grep rg

rgw.rgw.1.ceph-ci-fbv67y-ammmck-node4.dmsmex         ceph-ci-fbv67y-ammmck-node4
     *:80              running (4h)     10m ago  22h    93.3M        -
 19.1.0-55.el9cp  0ee0a0ad94c7  34f27723ccd2
rgw.rgw.1.ceph-ci-fbv67y-ammmck-node7.pocecp         ceph-ci-fbv67y-ammmck-node7
     *:80              running (4h)     10m ago  22h    96.4M        -
 19.1.0-55.el9cp  0ee0a0ad94c7  40e4f2a6d4c4
```

3. Verify if the localized read took place by running the **vim** command on the Ceph Object Gateway logs.

```
vim /var/log/ceph/<fsid>/<ceph-client-rgw>.log
```

For example,

```
[ceph: root@host01 /]# vim /var/log/ceph/<fsid>/<ceph-client-rgw>.log

2024-08-26T08:07:45.471+0000 7fc623e63640  1 ====== starting new request
req=0x7fc5b93694a0 =====
2024-08-26T08:07:45.471+0000 7fc623e63640  1 -- 10.0.67.142:0/279982082 -->
[v2:10.0.66.23:6816/73244434,v1:10.0.66.23:6817/73244434] -- osd_op(unknown.0.0:9081
11.55 11:ab26b168:::3acf4091-c54c-43b5-a495-c505fe545d25.27842.1_f1:head
[getxattrs,stat] snapc 0=[]
ondisk+read+localize_reads+known_if_redirected+supports_pool_eio e3533) --
0x55f781bd2000 con 0x55f77f0e8c00
```

You can see in the logs that a localized read has taken place.

Important: To be able to view the debug logs, you must first enable `debug_ms 1` in the configuration by running the **`ceph config set`** command.
For example,

```
[ceph: root@host01 /]#ceph config set client.rgw.rgw.1.ceph-ci-gune2w-mysx73-
node4.dgvrmx    advanced  debug_ms    1/1

[ceph: root@host01 /]#ceph config set client.rgw.rgw.1.ceph-ci-gune2w-mysx73-
node7.rfkqqq    advanced  debug_ms    1/1
```

Performing balanced reads

Perform a balanced read on a pool to retrieve evenly distributed OSDs across data centers.

Before you begin

- A stretch cluster with two data centers and Ceph Object Gateway configured on both.
- A user created with a bucket and OSDs - primary and replicated OSDs.

About this task

When a balanced *READ* is issued on a pool, the read operations are distributed evenly across all OSDs that are spread across the data centers.

Procedure

1. To perform a balanced read, set **`rados_replica_read_policy`** to `balance` for the Ceph Object Gateway daemon configuration using the `ceph config set` command.

   ```
   [ceph: root@host01 /]#ceph config set

   client.rgw.rgw.1 advanced rados_replica_read_policy balance
   ```

   The IO load is divided between the two data centers when a GET operation is performed.

2. **Verification**: Perform the below steps to verify the localized read from an OSD set.
   a. Run the `ceph osd tree` command to view the OSDs and the data centers.
   Example

```
[ceph: root@host01 /]# ceph osd tree

ID  CLASS  WEIGHT   TYPE NAME                                STATUS  REWEIGHT
 PRI-AFF
-1         0.58557  root default
-3         0.29279      datacenter DC1
-2         0.09760          host ceph-ci-fbv67y-ammmck-node2
 2    hdd  0.02440              osd.2                            up   1.00000
 1.00000
11    hdd  0.02440              osd.11                           up   1.00000
 1.00000
17    hdd  0.02440              osd.17                           up   1.00000
 1.00000
22    hdd  0.02440              osd.22                           up   1.00000
 1.00000
-4         0.09760          host ceph-ci-fbv67y-ammmck-node3
 0    hdd  0.02440              osd.0                            up   1.00000
 1.00000
 6    hdd  0.02440              osd.6                            up   1.00000
 1.00000
12    hdd  0.02440              osd.12                           up   1.00000
 1.00000
18    hdd  0.02440              osd.18                           up   1.00000
 1.00000
-5         0.09760          host ceph-ci-fbv67y-ammmck-node4
 5    hdd  0.02440              osd.5                            up   1.00000
 1.00000
10    hdd  0.02440              osd.10                           up   1.00000
 1.00000
16    hdd  0.02440              osd.16                           up   1.00000
 1.00000
23    hdd  0.02440              osd.23                           up   1.00000
 1.00000
-7         0.29279      datacenter DC2
-6         0.09760          host ceph-ci-fbv67y-ammmck-node5
 3    hdd  0.02440              osd.3                            up   1.00000
 1.00000
 8    hdd  0.02440              osd.8                            up   1.00000
 1.00000
14    hdd  0.02440              osd.14                           up   1.00000
 1.00000
20    hdd  0.02440              osd.20                           up   1.00000
 1.00000
-8         0.09760          host ceph-ci-fbv67y-ammmck-node6
 4    hdd  0.02440              osd.4                            up   1.00000
 1.00000
 9    hdd  0.02440              osd.9                            up   1.00000
 1.00000
15    hdd  0.02440              osd.15                           up   1.00000
 1.00000
21    hdd  0.02440              osd.21                           up   1.00000
 1.00000
-9         0.09760          host ceph-ci-fbv67y-ammmck-node7
 1    hdd  0.02440              osd.1                            up   1.00000
 1.00000
 7    hdd  0.02440              osd.7                            up   1.00000
 1.00000
13    hdd  0.02440              osd.13                           up   1.00000
 1.00000
19    hdd  0.02440              osd.19                           up   1.00000
 1.00000
```

b. Run the `ceph orch` command to identify the Ceph Object Gateway daemons in the data centers.
Example

```
ceph orch ps | grep rg

rgw.rgw.1.ceph-ci-fbv67y-ammmck-node4.dmsmex         ceph-ci-fbv67y-ammmck-node4
           *:80              running (4h)     10m ago  22h    93.3M        -
 19.1.0-55.el9cp  0ee0a0ad94c7  34f27723ccd2
rgw.rgw.1.ceph-ci-fbv67y-ammmck-node7.pocecp         ceph-ci-fbv67y-ammmck-node7
           *:80              running (4h)     10m ago  22h    96.4M        -
 19.1.0-55.el9cp  0ee0a0ad94c7  40e4f2a6d4c4
```

c. Verify if a balanced read has happened by running the `vim` command on the Ceph Object Gateway logs.
For example,

```
[ceph: root@host01 /]# vim /var/log/ceph/<fsid>/<ceph-client-rgw>.log

2024-08-27T09:32:25.510+0000 7f2a7a284640  1 ====== starting new request
req=0x7f2a31fcf4a0 =====
2024-08-27T09:32:25.510+0000 7f2a7a284640  1 -- 10.0.67.142:0/3116867178 -->
[v2:10.0.64.146:6816/2838383288,v1:10.0.64.146:6817/2838383288] --
osd_op(unknown.0.0:268731 11.55 11:ab26b168:::3acf4091-c54c-43b5-a495-
c505fe545d25.27842.1_f1:head [getxattrs,stat] snapc 0=[]
ondisk+read+balance_reads+known_if_redirected+supports_pool_eio e3554) --
0x55cd1b88dc00 con 0x55cd18dd6000
```

You can see in the logs that a balanced read has taken place.

Important: To be able to view the debug logs, you must first enable `debug_ms 1` in the configuration by running the `ceph config set` command.

```
[ceph: root@host01 /]#ceph config set client.rgw.rgw.1.ceph-ci-gune2w-
mysx73-node4.dgvrmx    advanced  debug_ms    1/1

[ceph: root@host01 /]#ceph config set client.rgw.rgw.1.ceph-ci-gune2w-
mysx73-node7.rfkqqq    advanced  debug_ms    1/1
```

Performing default reads

Perform a default read on a pool to retrieve data from primary data centers.

Before you begin

- A stretch cluster with two data centers and Ceph Object Gateway configured on both.
- A user created with a bucket and OSDs - primary and replicated OSDs.

About this task

When a default *READ* is issued on a pool, the IO operations are retrieved directly from each OSD in the data center.

Procedure

1. To perform a default read, set **`rados_replica_read_policy`** to `default` in the OSD daemon configuration by using the `ceph config set` command.

   ```
   [ceph: root@host01 /]#ceph config set

   client.rgw.rgw.1 advanced rados_replica_read_policy default
   ```

   The IO operations from the closest OSD in a data center are retrieved when a GET operation is performed.

2. **Verification**: Perform the below steps to verify the localized read from an OSD set.
   a. Run the `ceph osd tree` command to view the OSDs and the data centers.
   For example,

```
[ceph: root@host01 /]# ceph osd tree

ID  CLASS  WEIGHT   TYPE NAME                                STATUS  REWEIGHT
 PRI-AFF
-1         0.58557  root default
-3         0.29279      datacenter DC1
-2         0.09760          host ceph-ci-fbv67y-ammmck-node2
 2    hdd  0.02440              osd.2                            up   1.00000
 1.00000
11    hdd  0.02440              osd.11                           up   1.00000
 1.00000
17    hdd  0.02440              osd.17                           up   1.00000
 1.00000
22    hdd  0.02440              osd.22                           up   1.00000
 1.00000
-4         0.09760          host ceph-ci-fbv67y-ammmck-node3
 0    hdd  0.02440              osd.0                            up   1.00000
 1.00000
 6    hdd  0.02440              osd.6                            up   1.00000
 1.00000
12    hdd  0.02440              osd.12                           up   1.00000
 1.00000
18    hdd  0.02440              osd.18                           up   1.00000
 1.00000
-5         0.09760          host ceph-ci-fbv67y-ammmck-node4
 5    hdd  0.02440              osd.5                            up   1.00000
 1.00000
10    hdd  0.02440              osd.10                           up   1.00000
 1.00000
16    hdd  0.02440              osd.16                           up   1.00000
 1.00000
23    hdd  0.02440              osd.23                           up   1.00000
 1.00000
-7         0.29279      datacenter DC2
-6         0.09760          host ceph-ci-fbv67y-ammmck-node5
 3    hdd  0.02440              osd.3                            up   1.00000
 1.00000
 8    hdd  0.02440              osd.8                            up   1.00000
 1.00000
14    hdd  0.02440              osd.14                           up   1.00000
 1.00000
20    hdd  0.02440              osd.20                           up   1.00000
 1.00000
-8         0.09760          host ceph-ci-fbv67y-ammmck-node6
 4    hdd  0.02440              osd.4                            up   1.00000
 1.00000
 9    hdd  0.02440              osd.9                            up   1.00000
 1.00000
15    hdd  0.02440              osd.15                           up   1.00000
 1.00000
21    hdd  0.02440              osd.21                           up   1.00000
 1.00000
-9         0.09760          host ceph-ci-fbv67y-ammmck-node7
 1    hdd  0.02440              osd.1                            up   1.00000
 1.00000
 7    hdd  0.02440              osd.7                            up   1.00000
 1.00000
13    hdd  0.02440              osd.13                           up   1.00000
 1.00000
19    hdd  0.02440              osd.19                           up   1.00000
 1.00000
```

b. Run the `ceph orch` command to identify the Ceph Object Gateway daemons in the data centers. For example,

```
ceph orch ps | grep rg

rgw.rgw.1.ceph-ci-fbv67y-ammmck-node4.dmsmex         ceph-ci-fbv67y-ammmck-node4
           *:80              running (4h)     10m ago  22h    93.3M        -
 19.1.0-55.el9cp  0ee0a0ad94c7  34f27723ccd2
rgw.rgw.1.ceph-ci-fbv67y-ammmck-node7.pocecp         ceph-ci-fbv67y-ammmck-node7
           *:80              running (4h)     10m ago  22h    96.4M        -
 19.1.0-55.el9cp  0ee0a0ad94c7  40e4f2a6d4c4
```

c. Verify if a default read has happened by running the `vim` command on the Ceph Object Gateway logs.
For example,

```
[ceph: root@host01 /]# vim /var/log/ceph/<fsid>/<ceph-client-rgw>.log

2024-08-28T10:26:05.155+0000 7fe6b03dd640  1 ====== starting new request
req=0x7fe6879674a0 =====
2024-08-28T10:26:05.156+0000 7fe6b03dd640  1 -- 10.0.64.251:0/2235882725 -->
[v2:10.0.65.171:6800/4255735352,v1:10.0.65.171:6801/4255735352] --
osd_op(unknown.0.0:1123 11.6d 11:b69767fc:::699c2d80-5683-43c5-bdcd-
e8912107c176.24827.3_f1:head [getxattrs,stat] snapc 0=[]
ondisk+read+known_if_redirected+supports_pool_eio e4513) -- 0x5639da653800 con
0x5639d804d800
```

You can see in the logs that a default read has taken place.

Important: To be able to view the debug logs, you must first enable `debug_ms 1` in the configuration by running the `ceph config set` command.

```
[ceph: root@host01 /]#ceph config set client.rgw.rgw.1.ceph-ci-gune2w-
mysx73-node4.dgvrmx    advanced  debug_ms    1/1

[ceph: root@host01 /]#ceph config set client.rgw.rgw.1.ceph-ci-gune2w-
mysx73-node7.rfkqqq    advanced  debug_ms    1/1
```

Generalized stretch cluster configuration for three availability zones

As a storage administrator, you can configure a generalized stretch cluster configuration for three availability zones with Ceph OSDs.

Red Hat Ceph Storage can withstand the loss of Ceph OSDs because of its network and cluster, which are equally reliable with failures that are randomly distributed across the CRUSH map. If multiple OSDs are shut down, the remaining OSDs and monitors still manage to operate.

Using a single cluster limits data availability to a single location with a single point of failure. However, in some situations, higher availability might be required. Using three availability zones allows the cluster to withstand power loss and even a full data center loss in the event of a natural disaster.

With a generalized stretch cluster configuration for three availability zones, three data centers are supported, with each site holding two copies of the data. This helps ensure that even during a data center outage, the data remains accessible and writeable from another site. With this configuration, the pool replication size is `6` and the pool `min_size` is `3`.

Note: The standard Ceph configuration survives many failures of the network or data centers and it never compromises data consistency. If you restore enough Ceph servers following a failure, it recovers. Red Hat Ceph Storage maintains availability if you lose a data center, but can still form a quorum of monitors and have all the data available with enough copies to satisfy pools' `min_size`, or CRUSH rules that replicate again to meet the size.

Limitations

When using generalized stretch clusters, the following limitations should be considered.

- Generalized stretch cluster configuration for three availability zones does not support I/O operations during a netsplit scenario between two or more zones. While the cluster remains accessible for basic Ceph commands, I/O usage remains unavailable until the netsplit is resolved. This is different from stretch mode, where the tiebreaker monitor can isolate one zone of the cluster and continue I/O operations in degraded mode during a netsplit. For more information about stretch mode, see "Stretch mode" on page 401.
- In a three availability zone configuration, Red Hat Ceph Storage is designed to tolerate multiple host failures. However, if more than 25% of the OSDs in the cluster go down, Ceph may stop marking OSDs as `out`. This behavior is controlled by the **`mon_osd_min_in_ratio`** parameter. By default, **`mon_osd_min_in_ratio`** is set to `0.75`, meaning that at least 75% of the OSDs in the cluster must remain `in` (active) before any additional OSDs can be marked `out`. This setting prevents too many OSDs from being marked `out` as this might lead to significant data movement. The data movement can cause high client I/O impact and long recovery times when the OSDs are returned to service.

If Red Hat Ceph Storage stops marking OSDs as `out`, some placement groups (PGs) may fail to rebalance to surviving OSDs, potentially leading to inactive placement groups (PGs).

> **Important:** While adjusting the **`mon_osd_min_in_ratio`** value can allow more OSDs to be marked `out` and trigger rebalancing, this should be done with caution. For more information on the **`mon_osd_min_in_ratio`** parameter, see Ceph Monitor and OSD configuration options.

Generalized stretch cluster deployment requirements

This information details important hardware, software, and network requirements that are needed for deploying a generalized stretch cluster configuration for three availability zones.

Hardware requirements

Use the following minimum hardware requirements before deploying generalized stretch cluster configuration for three availability zones. "Hardware requirements" on page 432 lists the physical server locations and Ceph component layout for an example three availability zone deployment.

Table 59: Hardware requirements

Host name	Datacenter	Ceph services
host01	DC1	OSD+MON+MGR
host02	DC1	OSD+MON+MGR+RGW
host03	DC1	OSD+MON+MDS
host04	DC2	OSD+MON+MGR
host05	DC2	OSD+MON+MGR+RGW
host06	DC2	OSD+MON+MDS
host07	DC3	OSD+MON+MGR
host08	DC3	OSD+MON+MGR+RGW
host09	DC3	OSD+MON+MDS

Network configuration requirements

Use the following network configuration requirements before deploying generalized stretch cluster configuration for three availability zones.

- Have two separate networks, one public network and one cluster network.
- Have three different data centers that support VLANS and subnets for Ceph cluster and public networks for all data centers.

  > **Note:** You can use different subnets for each of the data centers.

- The latencies between data centers running the Red Hat Ceph Storage Object Storage Devices (OSDs) cannot exceed 10 ms RTT.

For more information about network considerations, see "Network considerations for Red Hat Ceph Storage" on page 88.

Cluster setup requirements

Ensure that the hostname is configured by using the bare or short hostname in all hosts.

```
hostnamectl set-hostname SHORT_NAME
```

> **Important:** The **`hostname`** command should only return the short hostname, when run on all nodes. If the FQDN is returned, the cluster configuration will not be successful.

Bootstrapping the Ceph cluster with a specification file

Deploy the generalized stretch cluster by setting the CRUSH location to the daemons in the cluster with the spec configuration file.

Before you begin

Before you begin, be sure that you have root-level access to the nodes.

About this task

Set the CRUSH location to the daemons in the cluster with a service configuration file. Use the configuration file to add the hosts to the proper locations during deployment.

For more information about Ceph bootstrapping and different **cephadm bootstrap** command options, see Bootstrapping a new storage cluster.

> **Important:** Run **cephadm bootstrap** on the node that you want to be the initial Monitor node in the cluster. The *IP_ADDRESS* option should be the IP address of the node you are using to run **cephadm bootstrap**.

> **Note:**
> - If the storage cluster includes multiple networks and interfaces, be sure to choose a network that is accessible by any node that uses the storage cluster.
> - To deploy a storage cluster by using IPV6 addresses, use the IPV6 address format for the `--mon-ip` *IP_ADDRESS* option. For example: **cephadm bootstrap --mon-ip 2620:52:0:880:225:90ff:fefc:2536 --registry-json /etc/mylogin.json**.
> - To route the internal cluster traffic over the public network, omit the `--cluster-network` *SUBNET* option.

Within this procedure the network Classless Inter-Domain Routing (CIDR) is referred to as *subnet*.

Procedure

1. Create the service configuration YAML file.
   The YAML file adds the nodes to the Red Hat Ceph Storage cluster and also sets specific labels for where the services run.
   The following example depends on the specific OSD and Ceph Object Gateway (RGW) configuration that is needed.

```
---
service_type: host
hostname: HOST01
addr: IP_ADDRESS01
labels: ['alertmanager', 'osd', 'installer', '_admin', 'mon', 'prometheus', 'mgr',
'grafana']
location:
  root: default
  datacenter: DC1
---
service_type: host
hostname: HOST02
addr: IP_ADDRESS02
labels: ['osd', 'mon', 'mgr', 'rgw']
location:
  root: default
  datacenter: DC1
---
service_type: host
hostname: HOST03
addr: IP_ADDRESS03
labels: ['osd', 'mon', 'mds']
location:
  root: default
```

```
  datacenter: DC1
---
service_type: host
hostname: HOST04
addr: IP_ADDRESS04
labels: ['osd', '_admin', 'mon', 'mgr']
location:
  root: default
  datacenter: DC2
---
service_type: host
hostname: HOST05
addr: IP_ADDRESS05
labels: ['osd', 'mon', 'mgr', 'rgw']
location:
  root: default
  datacenter: DC2
---
service_type: host
hostname: HOST06
addr: IP_ADDRESS06
labels: ['osd', 'mon', 'mds']
location:
  root: default
  datacenter: DC2
---
service_type: host
hostname: HOST07
addr: IP_ADDRESS07
labels: ['osd', '_admin', 'mon', 'mgr']
location:
  root: default
  datacenter: DC3
---
service_type: host
hostname: HOST08
addr: IP_ADDRESS08
labels: ['osd', 'mon', 'mgr', 'rgw']
location:
  root: default
  datacenter: DC3
---
service_type: host
hostname: HOST09
addr: IP_ADDRESS09
labels: ['osd', 'mon', 'mds']
location:
  root: default
  datacenter: DC3
---
service_type: mon
service_name: mon
placement:
  label: mon
spec:
  crush_locations:
    HOST01:
    - datacenter=DC1
    HOST02:
    - datacenter=DC1
    HOST03:
    - datacenter=DC1
    HOST04:
    - datacenter=DC2
    HOST05:
    - datacenter=DC2
    HOST06:
    - datacenter=DC2
    HOST07:
    - datacenter=DC3
    HOST08:
    - datacenter=DC3
    HOST09:
    - datacenter=DC3

---
service_type: mgr
service_name: mgr
placement:
  label: mgr
------
```

```
service_type: osd
service_id: osds
placement:
  label: osd
spec:
  data_devices:
     all: true
---------
service_type: rgw
service_id: rgw.rgw.1
placement:
label: rgw
------------
```

For more information about changing the custom spec for OSD and Object Gateway, see the following deployment instructions::

- "Deploying Ceph OSDs using service specifications" on page 597
- "Deploying the Ceph Object Gateway using the service specification" on page 626

2. Bootstrap the storage cluster with the **--apply-spec** option.

```
cephadm bootstrap --apply-spec CONFIGURATION_FILE_NAME --mon-ip MONITOR_IP_ADDRESS --
ssh-private-key PRIVATE_KEY --ssh-public-key PUBLIC_KEY --registry-url REGISTRY_URL --
registry-username USER_NAME --registry-password PASSWORD
```

For example,

```
[root@host01 ~]# cephadm bootstrap --apply-spec initial-config.yaml --mon-ip
10.10.128.68 --ssh-private-key /home/ceph/.ssh/id_rsa --ssh-public-key /home/
ceph/.ssh/id_rsa.pub --registry-url registry.redhat.io --registry-username myuser1 --
registry-password mypassword1
```

> **Important:** You can use different command options with the **cephadm bootstrap** command but always include the **--apply-spec** option to use the service configuration file and configure the host locations.

3. Log in to the `cephadm` shell.

```
[root@host01 ~]# cephadm shell
```

4. Configure the public network with the subnet.
For more information about configuring multiple public networks to the cluster, see "Configuring multiple public networks to the cluster" on page 293.

```
ceph config set global public_network "SUBNET_1,SUBNET_2, ..."
```

For example,

```
[ceph: root@host01 /]# ceph config set global public_network
"10.0.208.0/22,10.0.212.0/22,10.0.64.0/22,10.0.56.0/22"
```

5. Optional: Configure a cluster network.
For more information about configuring multiple cluster networks to the cluster, see "Configuring a private network" on page 291.

```
ceph config set global cluster_network "SUBNET_1,SUBNET_2, ..."
```

For example,

```
[ceph: root@host01 /]# ceph config set global cluster_network
"10.0.208.0/22,10.0.212.0/22,10.0.64.0/22,10.0.56.0/22"
```

6. Optional: Verify the network configurations.

```
[ceph: root@host01 /]# ceph config dump | grep network
```

7. Restart the daemons.
   Ceph daemons bind dynamically, so you do not have to restart the entire cluster at once if you change the network configuration for a specific daemon.

```
ceph orch restart mon
```

8. Optional: Restart the cluster on the admin node as a root user, by running the **systemctl restart** command.

> **Note:** To get the FSID of the cluster, use the **ceph fsid** command.

```
systemctl restart ceph-FSID_OF_CLUSTER.target
```

For example,

```
[root@host01 ~]# systemctl restart ceph-1ca9f6a8-d036-11ec-8263-fa163ee967ad.target
```

What to do next

Verify the specification file details and that the bootstrap was installed successfully.

1. Verify that all hosts were placed in the expected data centers, as specified in step “1” on page 433.

```
ceph osd tree
```

Check that there are three data centers under `root` and that the hosts are placed in each of the expected data centers.

> **Note:** The hosts with OSDs will only be present after bootstrap if OSDs are deployed during bootstrap with the specification file.

For example,

```
[root@host01 ~]# ceph osd tree
ID   CLASS  WEIGHT   TYPE NAME                                       STATUS
 REWEIGHT  PRI-AFF
 -1         0.87836  root default
 -3         0.29279      datacenter DC1
 -2         0.09760          host host01-installer
  0    hdd  0.02440              osd.0                                   up
1.00000  1.00000
 12    hdd  0.02440              osd.12                                  up
1.00000  1.00000
 21    hdd  0.02440              osd.21                                  up
1.00000  1.00000
 29    hdd  0.02440              osd.29                                  up
1.00000  1.00000
 -4         0.09760          host host02
  1    hdd  0.02440              osd.1                                   up
1.00000  1.00000
  9    hdd  0.02440              osd.9                                   up
1.00000  1.00000
 18    hdd  0.02440              osd.18                                  up
1.00000  1.00000
 28    hdd  0.02440              osd.28                                  up
1.00000  1.00000
 -5         0.09760          host host03
  8    hdd  0.02440              osd.8                                   up
1.00000  1.00000
 16    hdd  0.02440              osd.16                                  up
1.00000  1.00000
 24    hdd  0.02440              osd.24                                  up
1.00000  1.00000
 34    hdd  0.02440              osd.34                                  up
```

```
1.00000  1.00000
 -7         0.29279      datacenter DC2
 -6         0.09760          host host04
  4    hdd  0.02440              osd.4                                  up
1.00000  1.00000
 13    hdd  0.02440              osd.13                                 up
1.00000  1.00000
 20    hdd  0.02440              osd.20                                 up
1.00000  1.00000
 27    hdd  0.02440              osd.27                                 up
1.00000  1.00000
 -8         0.09760          host host05
  3    hdd  0.02440              osd.3                                  up
1.00000  1.00000
 10    hdd  0.02440              osd.10                                 up
1.00000  1.00000
 19    hdd  0.02440              osd.19                                 up
1.00000  1.00000
 30    hdd  0.02440              osd.30                                 up
1.00000  1.00000
 -9         0.09760          host host06
  7    hdd  0.02440              osd.7                                  up
1.00000  1.00000
 17    hdd  0.02440              osd.17                                 up
1.00000  1.00000
 26    hdd  0.02440              osd.26                                 up
1.00000  1.00000
 35    hdd  0.02440              osd.35                                 up
1.00000  1.00000
-11         0.29279      datacenter DC3
-10         0.09760          host host07
  5    hdd  0.02440              osd.5                                  up
1.00000  1.00000
 14    hdd  0.02440              osd.14                                 up
1.00000  1.00000
 23    hdd  0.02440              osd.23                                 up
1.00000  1.00000
 32    hdd  0.02440              osd.32                                 up
1.00000  1.00000
-12         0.09760          host host08
  2    hdd  0.02440              osd.2                                  up
1.00000  1.00000
 11    hdd  0.02440              osd.11                                 up
1.00000  1.00000
 22    hdd  0.02440              osd.22                                 up
1.00000  1.00000
 31    hdd  0.02440              osd.31                                 up
1.00000  1.00000
-13         0.09760          host host09
  6    hdd  0.02440              osd.6                                  up
1.00000  1.00000
 15    hdd  0.02440              osd.15                                 up
1.00000  1.00000
 25    hdd  0.02440              osd.25                                 up
1.00000  1.00000
 33    hdd  0.02440              osd.33                                 up
1.00000  1.00000
```

2. From the `cephadm` shell, verify that the mon daemons are deployed with CRUSH locations, as specified in step “1” on page 433.

```
ceph mon dump
```

Check that all mon daemons are in the output and that the correct CRUSH locations are added.

```
[root@host01 ~]# ceph mon dump
epoch 19
fsid b556497a-693a-11ef-b9d1-fa163e841fd7
last_changed 2024-09-03T12:47:08.419495+0000
created 2024-09-02T14:50:51.490781+0000
min_mon_release 19 (squid)
election_strategy: 3
0: [v2:10.0.67.43:3300/0,v1:10.0.67.43:6789/0] mon.host01-installer; crush_location
{datacenter=DC1}
1: [v2:10.0.67.20:3300/0,v1:10.0.67.20:6789/0] mon.host02; crush_location
{datacenter=DC1}
2: [v2:10.0.64.242:3300/0,v1:10.0.64.242:6789/0] mon.host03; crush_location
{datacenter=DC1}
3: [v2:10.0.66.17:3300/0,v1:10.0.66.17:6789/0] mon.host06; crush_location
{datacenter=DC2}
4: [v2:10.0.66.228:3300/0,v1:10.0.66.228:6789/0] mon.host09; crush_location
{datacenter=DC3}
5: [v2:10.0.65.125:3300/0,v1:10.0.65.125:6789/0] mon.host05; crush_location
{datacenter=DC2}
6: [v2:10.0.66.252:3300/0,v1:10.0.66.252:6789/0] mon.host07; crush_location
{datacenter=DC3}
7: [v2:10.0.64.145:3300/0,v1:10.0.64.145:6789/0] mon.host08; crush_location
{datacenter=DC3}
8: [v2:10.0.64.125:3300/0,v1:10.0.64.125:6789/0] mon.host04; crush_location
{datacenter=DC2}
dumped monmap epoch 19
```

3. Verify that the service spec and all location attributes are added correctly.
   a. Check the service name for mon daemons on the cluster, by using the **ceph orch ls** command.
   For example,

```
[root@host01 ~]# ceph orch ls
NAME                       PORTS        RUNNING  REFRESHED  AGE  PLACEMENT
alertmanager               ?:9093,9094      1/1  8m ago     6d   count:1
ceph-exporter                               9/9  8m ago     6d   *
crash                                       9/9  8m ago     6d   *
grafana                    ?:3000           1/1  8m ago     6d   count:1
mds.cephfs                                  3/3  8m ago     6d   label:mds
mgr                                         6/6  8m ago     6d   label:mgr
 mon                                         9/9  8m ago     5d   label:mon
node-exporter              ?:9100           9/9  8m ago     6d   *
osd.all-available-devices                    36  8m ago     6d   label:osd
prometheus                 ?:9095           1/1  8m ago     6d   count:1
rgw.rgw.1                  ?:80             3/3  8m ago     6d   label:rgw
```

   b. Confirm the mon daemon services, by using the **ceph orch ls mon --export** command.
   For example,

```
[root@host01 ~]# ceph orch ls mon --export
service_type: mon
service_name: mon
placement:
  label: mon
spec:
  crush_locations:
    host01-installer:
    - datacenter=DC1
    host02:
    - datacenter=DC1
    host03:
    - datacenter=DC1
    host04:
    - datacenter=DC2
    host05:
    - datacenter=DC2
    host06:
    - datacenter=DC2
    host07:
    - datacenter=DC3
    host08:
    - datacenter=DC3
    host09:
    - datacenter=DC3
```

4. Verify that the bootstrap was installed successfully, by running the **cephadm shell ceph -s** command.
   For more information, see “Verifying the cluster installation” on page 222.

Enabling three availability zones on the pool

Use this information to enable and integrate three availability zones within a generalized stretch cluster configuration.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root-level access to the nodes.
- The CRUSH location is set to the hosts.

Procedure

1. Get the most recent CRUSH map and decompile the map into a text file.

   ```
   ceph osd getcrushmap > COMPILED_CRUSHMAP_FILENAME
   crushtool -d COMPILED_CRUSHMAP_FILENAME -o DECOMPILED_CRUSHMAP_FILENAME
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph osd getcrushmap > crush.map.bin
   [ceph: root@host01 /]# crushtool -d crush.map.bin -o crush.map.txt
   ```

2. Add the new CRUSH rule into the decompiled CRUSH map file from step “1” on page 439.
   In this example, the rule name is `3az_rule`.

   ```
   rule 3az_rule {
            id 1
            type replicated
            step take default
            step choose firstn 3 type datacenter
            step chooseleaf firstn 2 type host
            step emit
    }
   ```

   With this rule, the placement groups will be replicated with two copies in each of the three data centers.

3. Inject the CRUSH map to make the rule available to the cluster.

   ```
   crushtool -c DECOMPILED_CRUSHMAP_FILENAME -o COMPILED_CRUSHMAP_FILENAME
   ceph osd setcrushmap -i COMPILED_CRUSHMAP_FILENAME
   ```

   For example,

   ```
   [ceph: root@host01 /]# crushtool -c crush.map.txt -o crush2.map.bin
   [ceph: root@host01 /]# ceph osd setcrushmap -i crush2.map.bin
   ```

   You can verify that the rule was injected successfully, by using the following steps.

   a. List the rules on the cluster.

      ```
      ceph osd crush rule ls
      ```

      For example,

      ```
      [ceph: root@host01 /]# ceph osd crush rule ls
      replicated_rule
      ec86_pool
      3az_rule
      ```

   b. Dump the CRUSH rule.

      ```
      ceph osd crush rule dump CRUSH_RULE
      ```

      For example,

```
[ceph: root@host01 /]# ceph osd crush rule dump 3az_rule
{
    "rule_id": 1,
    "rule_name": "3az_rule",
    "type": 1,
    "steps": [
        {
            "op": "take",
            "item": -1,
            "item_name": "default"
        },
        {
            "op": "choose_firstn",
            "num": 3,
            "type": "datacenter"
        },
        {
            "op": "chooseleaf_firstn",
            "num": 2,
            "type": "host"
        },
        {
            "op": "emit"
        }
    ]
}
```

4. Set the MON election strategy to `connectivity`.

```
ceph mon set election_strategy connectivity
```

When updated successfully, the `election_strategy` is updated to `3`. The default `election_strategy` is `1`.

5. Optional: Verify the election strategy that was set in step “4” on page 440.

```
ceph mon dump
```

Check that all mon daemons are in the output and that the correct CRUSH locations are added.

```
[ceph: root@host01 /]# ceph mon dump
epoch 19
fsid b556497a-693a-11ef-b9d1-fa163e841fd7
last_changed 2024-09-03T12:47:08.419495+0000
created 2024-09-02T14:50:51.490781+0000
min_mon_release 19 (squid)
election_strategy: 3
0: [v2:10.0.67.43:3300/0,v1:10.0.67.43:6789/0] mon.host01-installer; crush_location
{datacenter=DC1}
1: [v2:10.0.67.20:3300/0,v1:10.0.67.20:6789/0] mon.host02; crush_location
{datacenter=DC1}
2: [v2:10.0.64.242:3300/0,v1:10.0.64.242:6789/0] mon.host03; crush_location
{datacenter=DC1}
3: [v2:10.0.66.17:3300/0,v1:10.0.66.17:6789/0] mon.host06; crush_location
{datacenter=DC2}
4: [v2:10.0.66.228:3300/0,v1:10.0.66.228:6789/0] mon.host09; crush_location
{datacenter=DC3}
5: [v2:10.0.65.125:3300/0,v1:10.0.65.125:6789/0] mon.host05; crush_location
{datacenter=DC2}
6: [v2:10.0.66.252:3300/0,v1:10.0.66.252:6789/0] mon.host07; crush_location
{datacenter=DC3}
7: [v2:10.0.64.145:3300/0,v1:10.0.64.145:6789/0] mon.host08; crush_location
{datacenter=DC3}
8: [v2:10.0.64.125:3300/0,v1:10.0.64.125:6789/0] mon.host04; crush_location
{datacenter=DC2}
dumped monmap epoch 19
```

6. Set the pool to associate with three availability zone stretch clusters.
For more information about available pool values, see Pool values.

```
ceph osd pool stretch
set POOL_NAME PEERING_CRUSH_BUCKET_COUNT PEERING_CRUSH_BUCKET_TARGET PEERING_CRUSH_BUC
KET_BARRIER CRUSH_RULE SIZE MIN_SIZE [--yes-i-really-mean-it]
```

Replace the variables as follows:

POOL_NAME

The name of the pool. It must be an existing pool, this command doesn't create a new pool.

PEERING_CRUSH_BUCKET_COUNT

The value is used along with peering_crush_bucket_barrier to determined whether the set of OSDs in the chosen acting set can peer with each other, based on the number of distinct buckets there are in the acting set.

PEERING_CRUSH_BUCKET_TARGET

This value is used along with peering_crush_bucket_barrier and size to calculate the value `bucket_max` which limits the number of OSDs in the same bucket from getting chose to be in the acting set of a PG.

PEERING_CRUSH_BUCKET_BARRIER

The type of bucket a pool is stretched across. For example, rack, row, or datacenter.

CRUSH_RULE

The crush rule to use for the stretch pool. The type of pool must match the type of **`crush_rule`** (replicated or erasure).

SIZE

The number of replicas for objects in the stretch pool.

MIN_SIZE

The minimum number of replicas required for I/O in the stretch pool.

Important: The **`--yes-i-really-mean-it`** flag is required when setting the *PEERING_CRUSH_BUCKET_COUNT* and *PEERING_CRUSH_BUCKET_TARGET* to be more than the number of buckets in the CRUSH map. Use the optional flag to confirm that you want to bypass the safety checks and set the values for a stretch pool.

For example,

```
[ceph: root@host01 /]# ceph osd pool stretch set pool01 2 3 datacenter 3az_rule 6 3
```

Note: To revert a pool to a nonstretched cluster, use the **`ceph osd pool stretch unset POOL_NAME`** command. Using this command does not unset the `crush_rule`, `size`, and `min_size` values. If needed, these need to be reset manually.

A success message is emitted that the pool stretch values were set correctly.

7. Optional: Verify the pools associated with the stretch clusters, by using the **`ceph osd pool stretch show`** commands.
For example,

```
[ceph: root@host01 /]# ceph osd pool stretch show pool01
pool: pool01
pool_id: 1
is_stretch_pool: 1
peering_crush_bucket_count: 2
peering_crush_bucket_target: 3
peering_crush_bucket_barrier: 8
crush_rule: 3az_rule
size: 6
min_size: 3
```

Adding OSD hosts with three availability zones

You can add Ceph OSDs with three availability zones on a generalized stretch cluster. The procedure is similar to the addition of the OSD hosts on a cluster where a generalized stretch cluster is not enabled.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Three availability zones enabled on a cluster. For more information, see "Enabling three availability zones on the pool" on page 439.
- Root-level access to the nodes.

About this task

For more information, see Adding OSDs.

Procedure

1. From the node that contains the admin keyring, install the storage cluster's public SSH key in the root user's `authorized_keys` file on the new host.

   ```
   ssh-copy-id -f -i /etc/ceph/ceph.pub user@NEWHOST
   ```

   For example,

   ```
   [root@host10 ~]# ssh-copy-id -f -i /etc/ceph/ceph.pub root@host11
   [root@host10 ~]# ssh-copy-id -f -i /etc/ceph/ceph.pub root@host12
   ```

2. Add nodes by IP address.
   If you do not have DNS configured in your storage cluster environment, you can add the hosts by IP address, along with the host names.

   ```
   ceph orch host add HOSTNAME IP_ADDRESS
   ```

   For example,

   ```
   [ceph: root@host10 /]# ceph orch host add host11 10.10.128.69
   Added host 'host11' with addr '10.10.128.69'
   ```

3. Optional: Verify the status of the storage cluster and that each new host has been added by using the **`ceph orch host ls`** command.
   See that the new host has been added and that the **Status** of each host is blank in the output.

4. List the available devices to deploy OSDs.

   ```
   ceph orch device ls [--hostname=HOST11 HOST12] [--wide] [--refresh]
   ```

5. Deploy the OSDs on specific hosts or on all the available devices.
   Deploy in one of the following ways:
   - Create an OSD from a specific device on a specific host.

     ```
     ceph orch daemon add osd HOST:DEVICE_PATH
     ```

     For example,

     ```
     [ceph: root@host10 /]# ceph orch daemon add osd host11:/dev/sdb
     ```

   - Deploy OSDs on any available and unused devices.

     > **Important:** This command creates collocated WAL and DB devices. If you want to create non-collocated devices, do not use this command.

     ```
     ceph orch apply osd --all-available-devices
     ```

6. Move the OSD hosts under the CRUSH bucket.

```
ceph osd crush move HOST datacenter=DATACENTER
```

```
[ceph: root@host10 /]# ceph osd crush move host10 datacenter=DC1
[ceph: root@host10 /]# ceph osd crush move host11 datacenter=DC2
[ceph: root@host10 /]# ceph osd crush move host12 datacenter=DC3
```

Note: Ensure you add the same topology nodes on all sites. Issues might arise if hosts are added only on one site.

What to do next

Verify that all hosts are moved to the assigned data centers, by using the **ceph osd tree** command.

Override Ceph behavior

As a storage administrator, you need to understand how to use overrides for the Red Hat Ceph Storage cluster to change Ceph options during runtime.

Setting and unsetting override options

You can set and unset Ceph options to override Ceph's default behavior.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

1. To override Ceph's default behavior, use the `ceph osd set` command and the behavior you wish to override:

   Syntax

   ```
   ceph osd set FLAG
   ```

   Once you set the behavior, `ceph health` will reflect the override(s) that you have set for the cluster.

   Example

   ```
   [ceph: root@host01 /]# ceph osd set noout
   ```

2. To cease overriding Ceph's default behavior, use the `ceph osd unset` command and the override you wish to cease.

   Syntax

   ```
   ceph osd unset FLAG
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph osd unset noout
   ```

Flag	Description
`noin`	Prevents OSDs from being treated as `in` the cluster.
`noout`	Prevents OSDs from being treated as `out` of the cluster.

Flag	Description
noup	Prevents OSDs from being treated as up and running.
nodown	Prevents OSDs from being treated as down.
full	Makes a cluster appear to have reached its full_ratio, and thereby prevents write operations.
pause	Ceph stops processing read and write operations, but will not affect OSD in, out, up or down statuses.
nobackfill	Ceph prevents new backfill operations.
norebalance	Ceph prevents new rebalancing operations.
norecover	Ceph prevents new recovery operations.
noscrub	Ceph prevents new scrubbing operations.
nodeep-scrub	Ceph prevents new deep scrubbing operations.
notieragent	Ceph disables the process that is looking for cold/ dirty objects to flush and evict.

Override use cases

Understand various overrride use cases.

- noin: Commonly used with noout to address flapping OSDs.
- noout: If the mon osd report timeout is exceeded and an OSD has not reported to the monitor, the OSD will get marked out. If this happens erroneously, you can set noout to prevent the OSD(s) from getting marked out while you troubleshoot the issue.
- noup: Commonly used with nodown to address flapping OSDs.
- nodown: Networking issues may interrupt Ceph *heartbeat* processes, and an OSD may be up but still get marked down. You can set nodown to prevent OSDs from getting marked down while troubleshooting the issue.
- full: If a cluster is reaching its full_ratio, you can pre-emptively set the cluster to full and expand capacity.

  > **Note:** Setting the cluster to full will prevent write operations.

- pause: If you need to troubleshoot a running Ceph cluster without clients reading and writing data, you can set the cluster to pause to prevent client operations.
- nobackfill: If you need to take an OSD or node down temporarily, for example, upgrading daemons, you can set nobackfill so that Ceph will not backfill while the OSDs is down.
- norecover: If you need to replace an OSD disk and don't want the PGs to recover to another OSD while you are hotswapping disks, you can set norecover to prevent the other OSDs from copying a new set of PGs to other OSDs.
- noscrub and nodeep-scrubb: If you want to prevent scrubbing for example, to reduce overhead during high loads, recovery, backfilling, and rebalancing you can set noscrub and/or nodeep-scrub to prevent the cluster from scrubbing OSDs.
- notieragent: If you want to stop the tier agent process from finding cold objects to flush to the backing storage tier, you may set notieragent.

Ceph user management

As a storage administrator, you can manage the Ceph user base by providing authentication, and access control to objects in the Red Hat Ceph Storage cluster.

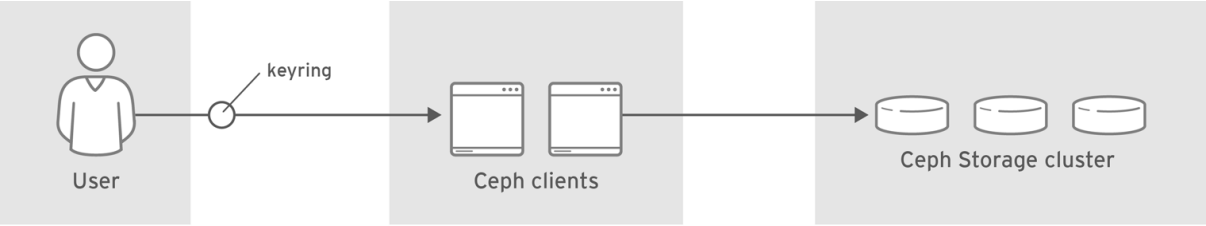


Figure 53: OSD states

> **Note:** `cephadm` manages the client keyrings for the Red Hat Ceph Storage cluster as long as the clients are within the scope of `cephadm`. Do not modify the keyrings that are managed by the `cephadm`, unless otherwise specified. If troubleshooting is needed, see "cephadm troubleshooting" on page 1594.

Ceph user management background

When Ceph runs with authentication and authorization that is enabled, you must specify a username. If a username is not specified, Ceph uses the `client.admin` administrative user as the default username.

The `CEPH_ARGS` environment variable may also be used to avoid re-entry of the username and secret.

Irrespective of the type of Ceph client, for example, block device, object store, file system, native API, or the Ceph command line, Ceph stores all data as objects within pools. Ceph users must have access to pools in order to read and write data. Also, administrative Ceph users must have permissions to run Ceph's administrative commands.

For more information on configuring the use of authentication, see "Configuring Red Hat Ceph Storage" on page 277.

The following concepts can help you understand Ceph user management:

- "Storage cluster users" on page 445
- "Authorization capabilities" on page 446
- "Pool" on page 447
- "Namespace" on page 447

Storage cluster users

A user of the Red Hat Ceph Storage cluster is either an individual or as an application. Creating users allows you to control who can access the storage cluster, its pools, and the data within those pools.

Ceph has the notion of a `type` of user. For the purposes of user management, the type is always `client`. Ceph identifies users in period (.) delimited form consisting of the user type and the user ID. For example, `TYPE.ID`, `client.admin`, or `client.user1`. The reason for user typing is that Ceph Monitors, and OSDs also use the Cephx protocol, but they are not clients. Distinguishing the user type helps to distinguish between client users and other users, streamlining access control, user monitoring, and traceability.

Sometimes Ceph's user type may seem confusing because the Ceph command line allows you to specify a user with or without the type, depending upon the command-line usage. If you specify `--user` or `--id`, you can omit the type. So `client.user1` can be entered, instead, as `user1`. If you specify `--name` or `-n`, you must specify the type and name, such as `client.user1`. Use the type and name, wherever possible.

> **Note:** A Red Hat Ceph Storage cluster user is not the same as a Ceph Object Gateway user. The object gateway uses a Red Hat Ceph Storage cluster user to communicate between the gateway daemon and the storage cluster. The gateway also has its own user management functionality for its end users.

Authorization capabilities

Ceph uses the term "capabilities" (caps) to describe authorizing an authenticated user to exercise the functionality of the Ceph Monitors and OSDs. Capabilities can also restrict access to data within a pool or a namespace within a pool. A Ceph administrative user sets a user's capabilities when creating or updating a user. Capability syntax follows the form:

```
DAEMON_TYPE 'allow CAPABILITY' [DAEMON_TYPE 'allow CAPABILITY']
```

Monitor Caps

Monitor capabilities include `r`, `w`, `x`, `allow profile CAP`, and `profile rbd`.
For example:

```
mon 'allow rwx'
mon 'allow profile osd'
```

OSD Caps

OSD capabilities include `r`, `w`, `x`, `class-read`, `class-write`, `profile osd`, `profile rbd`, and `profile rbd-read-only`. OSD capabilities also allow for pool and namespace settings.
For example:

```
osd 'allow CAPABILITY' [pool=POOL_NAME] [namespace=NAMESPACE_NAME]
```

> **Note:** The Ceph Object Gateway daemon (`radosgw`) is a client of the Ceph storage cluster, so it is not represented as a Ceph storage cluster daemon type.

"OSD Caps capabilities" on page 446 describes each capability.

Table 61: OSD Caps capabilities

Capability	Description
`allow`	Precedes access settings for a daemon.
`r`	Gives the user read access. Required with monitors to retrieve the CRUSH map.
`w`	Gives the user write access to objects.
`x`	Gives the user the capability to call class methods (that is, both read and write) and to conduct `auth` operations on monitors.
`class-read`	Gives the user the capability to call class read methods. Subset of `x`.
`class-write`	Gives the user the capability to call class write methods. Subset of `x`.
`*`	Gives the user read, write and run permissions for a particular daemon or pool, and the ability to run admin commands.
`profile osd`	Gives a user permissions to connect as an OSD to other OSDs or monitors. Conferred on OSDs to enable OSDs to handle replication heartbeat traffic and status reporting.
`profile bootstrap-osd`	Gives a user permissions to bootstrap an OSD so that they have permissions to add keys when bootstrapping an OSD.

Capability	Description
`profile rbd`	Gives a user read/write access to the Ceph Block Devices.
`profile rbd-read-only`	Gives a user read-only access to the Ceph Block Devices.

Pool

A pool defines a storage strategy for Ceph clients, and acts as a logical partition for that strategy.

In Ceph deployments, it is common to create a pool to support different types of use cases. For example, cloud volumes or images, object storage, hot storage, cold storage, and so on. When deploying Ceph as a back end for OpenStack, a typical deployment would have pools for volumes, images, backups and virtual machines, and users such as `client.glance`, `client.cinder`, and so on.

Namespace

Objects within a pool can be associated to a namespace, a logical group of objects within the pool. A user's access to a pool can be associated with a namespace such that reads and writes by the user take place only within the namespace. Objects written to a namespace within the pool can only be accessed by users who have access to the namespace.

> **Note:** Currently, namespaces are only useful for applications that are written on top of `librados`. Ceph clients such as block device and object storage do not currently support this feature.

The rationale for namespaces is that pools can be a computationally expensive method of segregating data by use case because each pool creates a set of placement groups that get mapped to OSDs. If multiple pools use the same CRUSH hierarchy and ruleset, OSD performance can degrade as load increases.

For example, a pool should have approximately 100 placement groups per OSD. So an exemplary cluster with 1000 OSDs would have 100,000 placement groups for one pool. Each pool mapped to the same CRUSH hierarchy and ruleset would create another 100,000 placement groups in the exemplary cluster. By contrast, writing an object to a namespace simply associates the namespace to the object name without the computational overhead of a separate pool. Rather than creating a separate pool for a user or set of users, you may use a namespace.

> **Note:** Only available using `librados` at this time.

Managing Ceph users

As a storage administrator, you can manage Ceph users by creating, modifying, deleting, and importing users.

A Ceph client user can be either individuals or applications, which use Ceph clients to interact with the Red Hat Ceph Storage cluster daemons.

Listing Ceph users

You can list the users in the storage cluster using the command-line interface.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

- To list the users in the storage cluster, run the following:

Example

```
[ceph: root@host01 /]# ceph auth list
installed auth entries:

osd.10
        key: AQBW7U5gqOsEExAAg/CxSwZ/gSh8iOsDV3iQOA==
        caps: [mgr] allow profile osd
        caps: [mon] allow profile osd
        caps: [osd] allow *
osd.11
        key: AQBX7U5gtj/JIhAAPsLBNG+SfC2eMVEFkl3vfA==
        caps: [mgr] allow profile osd
        caps: [mon] allow profile osd
        caps: [osd] allow *
osd.9
        key: AQBV7U5g1XDULhAAKo2tw6ZhH1jki5aVui2v7g==
        caps: [mgr] allow profile osd
        caps: [mon] allow profile osd
        caps: [osd] allow *
client.admin
        key: AQADYEtgFfD3ExAAwH+C1qO7MSLE4TWRfD2g6g==
        caps: [mds] allow *
        caps: [mgr] allow *
        caps: [mon] allow *
        caps: [osd] allow *
client.bootstrap-mds
        key: AQAHYEtgpbkANBAANqoFlvzEXFwD8oB0w3TF4Q==
        caps: [mon] allow profile bootstrap-mds
client.bootstrap-mgr
        key: AQAHYEtg3dcANBAAVQf6brq3sxTSrCrPe0pKVQ==
        caps: [mon] allow profile bootstrap-mgr
client.bootstrap-osd
        key: AQAHYEtgD/QANBAATS9DuP3DbxEl86MTyKEmdw==
        caps: [mon] allow profile bootstrap-osd
client.bootstrap-rbd
        key: AQAHYEtgjxEBNBAANho25V9tWNNvIKnHknW59A==
        caps: [mon] allow profile bootstrap-rbd
client.bootstrap-rbd-mirror
        key: AQAHYEtgdE8BNBAAr6rLYxZci0b2hoIgH9GXYw==
        caps: [mon] allow profile bootstrap-rbd-mirror
client.bootstrap-rgw
        key: AQAHYEtgwGkBNBAAuRzI4WSrnowBhZxr2XtTFg==
        caps: [mon] allow profile bootstrap-rgw
client.crash.host04
        key: AQCQYEtgz8lGGhAAy5bJS8VH9fMdxuAZ3CqX5Q==
        caps: [mgr] profile crash
        caps: [mon] profile crash
client.crash.host02
        key: AQDuYUtgqgfdOhAAsyX+Mo35M+HFpURGad7nJA==
        caps: [mgr] profile crash
        caps: [mon] profile crash
client.crash.host03
        key: AQB98E5g5jHZAxAAklWSvmDsh2JaL5G7FvMrrA==
        caps: [mgr] profile crash
        caps: [mon] profile crash
client.rgw.test_realm.test_zone.host01.hgbvnq
        key: AQD5RE9gAQKdCRAAJzxDwD/dJObbInp9J95sXw==
        caps: [mgr] allow rw
        caps: [mon] allow *
        caps: [osd] allow rwx tag rgw *=*
client.rgw.test_realm.test_zone.host02.yqqilm
        key: AQD0RE9gkxA4ExAAFXp3pLJWdIhsyTe2ZR6Ilw==
        caps: [mgr] allow rw
        caps: [mon] allow *
        caps: [osd] allow rwx tag rgw *=*
mgr.host01.hdhzwn
        key: AQAEYEtg3lhIBxAAmHodoIpdvnxK0llWF80ltQ==
        caps: [mds] allow *
        caps: [mon] profile mgr
        caps: [osd] allow *
mgr.host02.eobuuv
        key: AQAn6U5gzUuiABAA2Fed+jPM1xwb4XDYtrQxaQ==
        caps: [mds] allow *
        caps: [mon] profile mgr
        caps: [osd] allow *
mgr.host03.wquwpj
        key: AQAd6U5gIzWsLBAAbOKUKZlUcAVe9kBLfajMKw==
        caps: [mds] allow *
        caps: [mon] profile mgr
        caps: [osd] allow *
```

Note: The `TYPE.ID` notation for users applies such that `osd.0` is a user of type `osd` and its ID is `0`. `client.admin` is a user of type `client` and its ID is `admin`, that is, the default `client.admin` user. Note also that each entry has a `key: VALUE` entry, and one or more `caps:` entries.

You may use the `-o FILE_NAME` option with `ceph auth list` to save the output to a file.

Displaying Ceph user information

You can display a Ceph's user information using the command-line interface.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

1. To retrieve a specific user, key and capabilities, run the following:

   Syntax

   ```
   ceph auth export TYPE.ID
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph auth export mgr.host02.eobuuv
   ```

2. You can also use the `-o FILE_NAME` option.

   Syntax

   ```
   ceph auth export TYPE.ID -o FILE_NAME
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph auth export osd.9 -o filename
   export auth(key=AQBV7U5g1XDULhAAKo2tw6ZhH1jki5aVui2v7g==)
   ```

The `auth export` command is identical to `auth get`, but also prints out the internal `auid`, which isn't relevant to end users.

Adding a Ceph user

Adding a user creates a username, that is, `TYPE.ID`, a secret key and any capabilities included in the command you use to create the user.

A user's key enables the user to authenticate with the Red Hat Ceph Storage cluster. The user's capabilities authorize the user to read, write, or run on Ceph monitors (`mon`), Ceph OSDs (`osd`) or Ceph Metadata Servers (`mds`).

There are a few ways to add a user:

- `ceph auth add`: This command is the canonical way to add a user. It will create the user, generate a key and add any specified capabilities.
- `ceph auth get-or-create`: This command is often the most convenient way to create a user, because it returns a keyfile format with the user name (in brackets) and the key. If the user already exists, this command simply returns the user name and key in the keyfile format. You may use the `-o FILE_NAME` option to save the output to a file.
- `ceph auth get-or-create-key`: This command is a convenient way to create a user and return the user's key only. This is useful for clients that need the key only, for example, `libvirt`. If the user already exists, this command simply returns the key. You may use the `-o FILE_NAME` option to save the output to a file.

When creating client users, you may create a user with no capabilities. A user with no capabilities is useless beyond mere authentication, because the client cannot retrieve the cluster map from the monitor. However, you can create a user with no capabilities if you wish to defer adding capabilities later using the `ceph auth caps` command.

A typical user has at least read capabilities on the Ceph monitor and read and write capability on Ceph OSDs. Additionally, a user's OSD permissions are often restricted to accessing a particular pool. :

```
[ceph: root@host01 /]# ceph auth add client.john mon 'allow r' osd 'allow rw pool=mypool'
[ceph: root@host01 /]# ceph auth get-or-create client.paul mon 'allow r' osd 'allow rw
pool=mypool'
[ceph: root@host01 /]# ceph auth get-or-create client.george mon 'allow r' osd 'allow rw
pool=mypool' -o george.keyring
[ceph: root@host01 /]# ceph auth get-or-create-key client.ringo mon 'allow r' osd 'allow rw
pool=mypool' -o ringo.key
```

Important: If you provide a user with capabilities to OSDs, but you DO NOT restrict access to particular pools, the user will have access to ALL pools in the cluster.

Modifying Ceph user

The **`ceph auth caps`** command allows you to specify a user and change the user's capabilities.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

1. To add capabilities, use the form:

   Syntax

   ```
   ceph auth caps USERTYPE.USERID DAEMON 'allow [r|w|x|*|...] [pool=POOL_NAME]
   [namespace=NAMESPACE_NAME]'
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph auth caps client.john mon 'allow r' osd 'allow rw
   pool=mypool'
   [ceph: root@host01 /]# ceph auth caps client.paul mon 'allow rw' osd 'allow rwx
   pool=mypool'
   [ceph: root@host01 /]# ceph auth caps client.brian-manager mon 'allow *' osd 'allow *'
   ```

2. To remove a capability, you may reset the capability. If you want the user to have no access to a particular daemon that was previously set, specify an empty string:

   Example

   ```
   [ceph: root@host01 /]# ceph auth caps client.ringo mon ' ' osd ' '
   ```

Reference

For more information about capabilities, see "Ceph user management background" on page 445.

Deleting Ceph user

You can delete a user from the Ceph storage cluster using the command-line interface.

Prerequisites

- A running Red Hat Ceph Storage cluster.

- Root-level access to the node.

Procedure

1. To delete a user, use `ceph auth del`:

   Syntax

   ```
   ceph auth del TYPE.ID
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph auth del osd.6
   ```

Printing Ceph user key

You can display a Ceph user's key information using the command-line interface.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

- Print a user's authentication key to standard output:

  ```
  ceph auth print-key TYPE.ID
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph auth print-key osd.6
  AQBQ7U5gAry3JRAA3NoPrqBBThpFMcRL6Sr+5w==[ceph: root@host01 /]#
  ```

Using `ceph-volume` utility

Use the `ceph-volume` utility to prepare, list, create, activate, deactivate, batch, trigger, zap, and migrate Ceph OSDs.

The `ceph-volume` utility is a single-purpose command-line tool to deploy logical volumes as OSDs. It uses a plugin-type framework to deploy OSDs with different device technologies. The `ceph-volume` utility follows a similar workflow of the `ceph-disk` utility for deploying OSDs, with a predictable, and robust way of preparing, activating, and starting OSDs. Currently, the `ceph-volume` utility only supports the `lvm` plug-in, with the plan to support others technologies in the future.

> **Important:** The **`ceph-disk`** command is deprecated.

Running ceph-volume in a `cephadm` shell

> **Important:** Do not run `ceph-volume` from a container session that is started with `cephadm shell` when relying on the shell's default bind mounts.
> By design, `cephadm shell` omits several host bind mounts that `ceph-volume` expects, such as `/run/udev` and `/run/lvm`. As a result, running `ceph-volume` in this environment can fail or behave incorrectly.

Running `ceph-volume` manually, outside operations driven by `ceph orch` or `cephadm`, is not the normal operational path. This approach is intended mainly for debugging, testing, or development.

Important: Using `ceph-volume` directly outside the `ceph orch` (`cephadm`) workflow is not encouraged. The utility assumes a host environment with access to device metadata and system services that are normally available on the host when operating under the `cephadm`-driven workflow.
Although it is possible to extend the `cephadm shell` container environment by adding host bind mounts, this configuration is not typical. For guidance specific to your environment, contact IBM Support.

Ceph volume `lvm` plugin

By making use of LVM tags, the `lvm` sub-command is able to store and re-discover by querying devices associated with OSDs so they can be activated. This includes support for lvm-based technologies like `dm-cache` as well.

When using `ceph-volume`, the use of `dm-cache` is transparent, and treats `dm-cache` like a logical volume. The performance gains and losses when using `dm-cache` will depend on the specific workload. Generally, random and sequential reads will see an increase in performance at smaller block sizes. While random and sequential writes will see a decrease in performance at larger block sizes.

To use the LVM plugin, add `lvm` as a subcommand to the `ceph-volume` command within the `cephadm` shell:

```
[ceph: root@host01 /]# ceph-volume lvm
```

Following are the `lvm` subcommands:

- `prepare` - Format an LVM device and associate it with an OSD.
- `activate` - Discover and mount the LVM device associated with an OSD ID and start the Ceph OSD.
- `list` - List logical volumes and devices associated with Ceph.
- `batch` - Automatically size devices for multi-OSD provisioning with minimal interaction.
- `deactivate` - Deactivate OSDs.
- `create` - Create a new OSD from an LVM device.
- `trigger` - A systemd helper to activate an OSD.
- `zap` - Removes all data and filesystems from a logical volume or partition.
- `migrate` - Migrate BlueFS data from to another LVM device.
- `new-wal` - Allocate new WAL volume for the OSD at specified logical volume.
- `new-db` - Allocate new DB volume for the OSD at specified logical volume.

Note: Using the **`create`** subcommand combines the **`prepare`** and **`activate`** subcommands into one subcommand.

Reference

For more information, see “Creating Ceph OSDs” on page 457.

Why does `ceph-volume` replace `ceph-disk`?

Previous versions of Ceph used the `ceph-disk` utility to prepare, activate, and create OSDs. The `ceph-disk` is replaced by the `ceph-volume` utility that aims to be a single purpose command-line tool to deploy logical volumes as OSDs, while maintaining a similar API to `ceph-disk` when preparing, activating, and creating OSDs.

How does `ceph-volume` work?

The `ceph-volume` is a modular tool that currently supports two ways of provisioning hardware devices, legacy `ceph-disk` devices and LVM (Logical Volume Manager) devices. The `ceph-volume lvm` command uses the LVM tags to store information about devices specific to Ceph and its relationship with OSDs. It uses these tags to

later re-discover and query devices associated with OSDS so that it can activate them. It supports technologies based on LVM and `dm-cache` as well.

The `ceph-volume` utility uses `dm-cache` transparently and treats it as a logical volume. You might consider the performance gains and losses when using `dm-cache`, depending on the specific workload you are handling. Generally, the performance of random and sequential read operations increases at smaller block sizes; while the performance of random and sequential write operations decreases at larger block sizes. Using `ceph-volume` does not introduce any significant performance penalties.

Important: The `ceph-disk` utility is deprecated.

Note: The **`ceph-volume simple`** command can handle legacy `ceph-disk` devices, if these devices are still in use.

How does `ceph-disk` work?

The `ceph-disk` utility was required to support many different types of init systems, such as `upstart` or `sysvinit`, while being able to discover devices. For this reason, `ceph-disk` concentrates only on GUID Partition Table (GPT) partitions. Specifically on GPT GUIDs that label devices in a unique way to answer questions like:

- Is this device a `journal`?
- Is this device an encrypted data partition?
- Was the device partially prepared?

To solve these questions, `ceph-disk` uses UDEV rules to match the GUIDs.

What are disadvantages of using `ceph-disk`?

Using the UDEV rules to call `ceph-disk` can lead to a back-and-forth between the `ceph-disk systemd` unit and the `ceph-disk` executable. The process is very unreliable and time consuming and can cause OSDs to not come up at all during the boot process of a node. Moreover, it is hard to debug, or even replicate these problems given the asynchronous behavior of UDEV.

Because `ceph-disk` works with GPT partitions exclusively, it cannot support other technologies, such as Logical Volume Manager (LVM) volumes, or similar device mapper devices.

To ensure the GPT partitions work correctly with the device discovery workflow, `ceph-disk` requires a large number of special flags to be used. In addition, these partitions require devices to be exclusively owned by Ceph.

Preparing OSDs

The `prepare` subcommand prepares an OSD back-end object store and consumes logical volumes (LV) for both the OSD data and journal. It does not modify the logical volumes, except for adding some extra metadata tags using LVM. These tags make volumes easier to discover, and they also identify the volumes as part of the Ceph Storage Cluster and the roles of those volumes in the storage cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root-level access to the OSD nodes.
- Optionally, create logical volumes. If you provide a path to a physical device, the subcommand turns the device into a logical volume. This approach is simpler, but you cannot configure or change the way the logical volume is created.

About this task

The BlueStore OSD backend supports the following configurations:

- A block device, a `block.wal` device, and a `block.db` device

- A block device and a `block.wal` device
- A block device and a `block.db` device
- A single block device

The `prepare` subcommand accepts a whole device or partition, or a logical volume for `block`.

For more information, see:

- "Activating OSDs" on page 455
- "Creating Ceph OSDs" on page 457

Procedure

1. Extract the Ceph keyring.

   ```
   ceph auth get client.ID -o ceph.client.ID.keyring
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph auth get client.bootstrap-osd -o /var/lib/ceph/bootstrap-
   osd/ceph.keyring
   ```

2. Prepare the LVM volumes.

   ```
   ceph-volume lvm prepare --bluestore --data VOLUME_GROUP/LOGICAL_VOLUME
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph-volume lvm prepare --bluestore --data example_vg/data_lv
   ```

   a. To use a separate device for RocksDB, specify the **`--block.db`** and **`--block.wal`** options.

      ```
      ceph-volume lvm prepare --bluestore --block.db --block.wal --data VOLUME_GROUP/
      LOGICAL_VOLUME
      ```

      For example,

      ```
      ceph: root@host01 /]# ceph-volume lvm prepare --bluestore --block.db --block.wal
      --data example_vg/data_lv
      ```

   b. To encrypt data, use the **`--dmcrypt`** flag.

      ```
      ceph-volume lvm prepare --bluestore --dmcrypt --data VOLUME_GROUP/LOGICAL_VOLUME
      ```

      For example,

      ```
      [ceph: root@host01 /]# ceph-volume lvm prepare --bluestore --dmcrypt --data
      example_vg/data_lv
      ```

Listing devices

You can use the **`ceph-volume lvm list`** subcommand to list logical volumes and devices associated with a Ceph cluster, as long as they contain enough metadata to allow for that discovery. The output is grouped by the OSD ID associated with the devices. For logical volumes, the `devices key` is populated with the physical devices associated with the logical volume.

Sometimes, the output of the **`ceph -s`** command shows the following error message:

```
1 devices have fault light turned on
```

In such cases, you can list the devices with **`ceph device ls-lights`** command that gives the details about the lights on the devices. Based on the information, you can turn off the lights on the devices.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph OSD node.

Procedure

1. List the devices in the Ceph cluster:

   Example

   ```
   [ceph: root@host01 /]# ceph-volume lvm list


   ====== osd.6 =======

     [block]       /dev/ceph-83909f70-95e9-4273-880e-5851612cbe53/osd-
   block-7ce687d9-07e7-4f8f-a34e-d1b0efb89920

         block device              /dev/ceph-83909f70-95e9-4273-880e-5851612cbe53/osd-
   block-7ce687d9-07e7-4f8f-a34e-d1b0efb89920
         block uuid                4d7gzX-Nzxp-UUG0-bNxQ-Jacr-l0mP-IPD8cX
         cephx lockbox secret
         cluster fsid              1ca9f6a8-d036-11ec-8263-fa163ee967ad
         cluster name              ceph
         crush device class        None
         encrypted                 0
         osd fsid                  7ce687d9-07e7-4f8f-a34e-d1b0efb89920
         osd id                    6
         osdspec affinity          all-available-devices
         type                      block
         vdo                       0
         devices                   /dev/vdc
   ```

2. List the devices in the storage cluster with the lights.

   ```
   [ceph: root@host01 /]# ceph device ls-lights

   {
       "fault": [
           "SEAGATE_ST12000NM002G_ZL2KTGCK0000C149"
       ],
       "ident": []
   }
   ```

3. Turn off the lights on the device.

   ```
   ceph device light off DEVICE_NAME FAULT/INDENT --force
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph device light off SEAGATE_ST12000NM002G_ZL2KTGCK0000C149
   fault --force
   ```

Activating OSDs

The activation process for a Ceph OSD enables a `systemd` unit at boot time, which allows the correct OSD identifier and its UUID to be enabled and mounted.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph OSD node.
- Ceph OSDs prepared by the `ceph-volume` utility.

Procedure

1. Get the OSD ID and OSD FSID from an OSD node.

Example

```
[ceph: root@host01 /]# ceph-volume lvm list
```

2. Activate the OSD.

```
ceph-volume lvm activate --bluestore OSD_ID OSD_FSID
```

Example

```
[ceph: root@host01 /]# ceph-volume lvm activate --bluestore 10 7ce687d9-07e7-4f8f-
a34e-d1b0efb89920
```

To activate all OSDs that are prepared for activation, use the `--all` option.

Example

```
[ceph: root@host01 /]# ceph-volume lvm activate --all
```

3. Optionally, you can use the `trigger` subcommand. This command cannot be used directly, and it is used by `systemd` so that it proxies input to `ceph-volume lvm activate`. This parses the metadata coming from systemd and startup, detecting the UUID and ID associated with an OSD.

```
ceph-volume lvm trigger SYSTEMD_DATA
```

Here the *SYSTEMD_DATA* is in *OSD_ID-OSD_FSID* format.

Example

```
[ceph: root@host01 /]# ceph-volume lvm trigger 10 7ce687d9-07e7-4f8f-a34e-d1b0efb89920
```

Reference

For more information, see:

- “Preparing OSDs” on page 453
- “Creating Ceph OSDs” on page 457

Deactivating OSDs

You can deactivate the Ceph OSDs using the **`ceph-volume lvm`** subcommand. This subcommand removes the volume groups and the logical volume.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph OSD node.
- The Ceph OSDs are activated using the `ceph-volume` utility.

Procedure

1. Get the OSD ID from the OSD node.

```
[ceph: root@host01 /]# ceph-volume lvm list
```

2. Deactivate the OSD:

```
ceph-volume lvm deactivate OSD_ID
```

Example

```
[ceph: root@host01 /]# ceph-volume lvm deactivate 16
```

Reference

For more information, see:

- “Activating OSDs” on page 455
- “Preparing OSDs” on page 453
- “Creating Ceph OSDs” on page 457

Creating Ceph OSDs

The `create` subcommand calls the `prepare` subcommand, and then calls the `activate` subcommand.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph OSD nodes.

> **Note:** If you prefer to have more control over the creation process, you can use the **`prepare`** and **`activate`** subcommands separately to create the OSD instead of using **`create`**. You can use the two subcommands to gradually introduce new OSDs into a storage cluster, and avoid having to rebalance large amounts of data. Both approaches work the same way, except that using the **`create`** subcommand causes the OSD to become `up` and `in` immediately after completion.

Procedure

- Create an OSD.

  Syntax

  ```
  ceph-volume lvm create --bluestore --data VOLUME_GROUP/LOGICAL_VOLUME
  ```

  Example

  ```
  [root@osd ~]# ceph-volume lvm create --bluestore --data example_vg/data_lv
  ```

Reference

- “Preparing OSDs” on page 453
- “Activating OSDs” on page 455

Migrating BlueFS data

You can migrate the BlueStore file system (BlueFS) data, that is the RocksDB data, from the source volume to the target volume using the `migrate` LVM subcommand.

When using the `migrate` LVM subcommand, the source volume, except the main one, is removed on success.

LVM volumes are primarily for the target only.

The new volumes are attached to the OSD, replacing one of the source drives.

Following are the placement rules for the LVM volumes:

- If source list has DB or WAL volume, then the target device replaces it.
- If source list has slow volume only, then explicit allocation using the `new-db` or `new-wal` command is needed.

The `new-db` and `new-wal` commands attaches the given logical volume to the given OSD as a DB or a WAL volume respectively.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph OSD node.
- Ceph OSDs prepared by the `ceph-volume` utility.
- Volume groups and Logical volumes are created.

Procedure

1. Log in the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Stop the OSD to which you have to add the DB or the WAL device:

   Example

   ```
   [ceph: root@host01 /]# ceph orch daemon stop osd.1
   ```

3. Mount the new devices to the container:

   Example

   ```
   [root@host01 ~]# cephadm shell --mount /var/lib/ceph/72436d46-ca06-11ec-9809-
   ac1f6b5635ee/osd.1:/var/lib/ceph/osd/ceph-1
   ```

4. Attach the given logical volume to OSD as a DB/WAL device:

   > **Note:** This command fails if the OSD has an attached DB.

   Syntax

   ```
   ceph-volume lvm new-db --osd-id OSD_ID --osd-fsid OSD_FSID --target VOLUME_GROUP_NAME/
   LOGICAL_VOLUME_NAME
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph-volume lvm new-db --osd-id 1 --osd-fsid
   7ce687d9-07e7-4f8f-a34e-d1b0efb89921 --target vgname/new_db
   [ceph: root@host01 /]# ceph-volume lvm new-wal --osd-id 1 --osd-fsid
   7ce687d9-07e7-4f8f-a34e-d1b0efb89921 --target vgname/new_wal
   ```

5. You can migrate BlueFS data in the following ways:

   - Move BlueFS data from main device to LV that is already attached as DB:

     Syntax

     ```
     ceph-volume lvm migrate --osd-id OSD_ID --osd-fsid OSD_UUID --from data --target
     VOLUME_GROUP_NAME/LOGICAL_VOLUME_NAME
     ```

     Example

     ```
     [ceph: root@host01 /]# ceph-volume lvm migrate --osd-id 1 --osd-fsid
     0263644D-0BF1-4D6D-BC34-28BD98AE3BC8 --from data --target vgname/db
     ```

   - Move BlueFS data from shared main device to LV which shall be attached as a new DB:

Syntax

```
ceph-volume lvm migrate --osd-id OSD_ID --osd-fsid OSD_UUID --from data --target
VOLUME_GROUP_NAME/LOGICAL_VOLUME_NAME
```

Example

```
[ceph: root@host01 /]# ceph-volume lvm migrate --osd-id 1 --osd-fsid
0263644D-0BF1-4D6D-BC34-28BD98AE3BC8 --from data --target vgname/new_db
```

- Move BlueFS data from DB device to new LV, and replace the DB device:

  Syntax

  ```
  ceph-volume lvm migrate --osd-id OSD_ID --osd-fsid OSD_UUID --from db --target
  VOLUME_GROUP_NAME/LOGICAL_VOLUME_NAME
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph-volume lvm migrate --osd-id 1 --osd-fsid
  0263644D-0BF1-4D6D-BC34-28BD98AE3BC8 --from db --target vgname/new_db
  ```

- Move BlueFS data from main and DB devices to new LV, and replace the DB device:

  Syntax

  ```
  ceph-volume lvm migrate --osd-id OSD_ID --osd-fsid OSD_UUID --from data db --
  target VOLUME_GROUP_NAME/LOGICAL_VOLUME_NAME
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph-volume lvm migrate --osd-id 1 --osd-fsid
  0263644D-0BF1-4D6D-BC34-28BD98AE3BC8 --from data db --target vgname/new_db
  ```

- Move BlueFS data from main, DB, and WAL devices to new LV, remove the WAL device, and replace the DB device:

  Syntax

  ```
  ceph-volume lvm migrate --osd-id OSD_ID --osd-fsid OSD_UUID --from data db wal --
  target VOLUME_GROUP_NAME/LOGICAL_VOLUME_NAME
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph-volume lvm migrate --osd-id 1 --osd-fsid
  0263644D-0BF1-4D6D-BC34-28BD98AE3BC8 --from data db --target vgname/new_db
  ```

- Move BlueFS data from main, DB, and WAL devices to the main device, remove the WAL and DB devices:

  Syntax

  ```
  ceph-volume lvm migrate --osd-id OSD_ID --osd-fsid OSD_UUID --from db wal --
  target VOLUME_GROUP_NAME/LOGICAL_VOLUME_NAME
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph-volume lvm migrate --osd-id 1 --osd-fsid
  0263644D-0BF1-4D6D-BC34-28BD98AE3BC8 --from db wal --target vgname/data
  ```

Expanding BlueFS DB device

Expand the storage of BlueStore File System.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Ceph OSDs are prepared by the `ceph-volume` utility.
- Volume groups and Logical volumes are created.

> **Note:** Run these steps on the host where the OSD is deployed.

About this task

You can expand the storage of the BlueStore file system (BlueFS) data that is the RocksDB data of `ceph-volume` created OSDs with the `ceph-bluestore` tool.

Procedure

1. **Optional:** Inside the `cephadm` shell, list the devices in the Red Hat Ceph Storage cluster.

```
ceph-volume lvm list
```

For example,

```
[ceph: root@host01 /]# ceph-volume lvm list

====== osd.3 =======

  [db]          /dev/db-test/db1

      block device              /dev/test/lv1
      block uuid                N5zoix-FePe-uExe-UngY-D9YG-BMs0-1tTDyB
      cephx lockbox secret
      cluster fsid              1a6112da-ed05-11ee-bacd-525400565cda
      cluster name              ceph
      crush device class
      db device                 /dev/db-test/db1
      db uuid                   1TUaDY-3mEt-fReP-cyB2-JyZ1-oUPa-hKPfo6
      encrypted                 0
      osd fsid                  94ff742c-7bfd-4fb5-8dc4-843d10ac6731
      osd id                    3
      osdspec affinity          None
      type                      db
      vdo                       0
      devices                   /dev/vdh

  [block]       /dev/test/lv1

      block device              /dev/test/lv1
      block uuid                N5zoix-FePe-uExe-UngY-D9YG-BMs0-1tTDyB
      cephx lockbox secret
      cluster fsid              1a6112da-ed05-11ee-bacd-525400565cda
      cluster name              ceph
      crush device class
      db device                 /dev/db-test/db1
      db uuid                   1TUaDY-3mEt-fReP-cyB2-JyZ1-oUPa-hKPfo6
      encrypted                 0
      osd fsid                  94ff742c-7bfd-4fb5-8dc4-843d10ac6731
      osd id                    3
      osdspec affinity          None
      type                      block
      vdo                       0
      devices                   /dev/vdg
```

2. Get the volume group information.

```
vgs
```

For example,

```
[root@host01 ~]# vgs
VG                                        #PV #LV #SN Attr   VSize    VFree
db-test                                     1   1   0 wz--n- <200.00g <160.00g
test                                        1   1   0 wz--n- <200.00g <170.00g
```

3. Stop the Ceph OSD service.

```
systemctl stop SERVICE_ID
```

For example,

```
[root@host01 ~]# systemctl stop host01a6112da-ed05-11ee-
bacd-525400565cda@osd.3.service
```

4. Resize, shrink, and expand the logical volumes.

```
lvresize -l 100%FREE PATH_OF_DB_DEVICE
```

For example,

```
[root@host01 ~]# lvresize -l 100%FREE /dev/db-test/db1
Size of logical volume db-test/db1 changed from 40.00 GiB (10240 extents) to <160.00
GiB (40959 extents).
Logical volume db-test/db1 successfully resized.
```

5. Launch the `cephadm` shell.

```
cephadm shell -m /var/lib/ceph/CLUSTER_FSID/osd.OSD_ID:/var/lib/ceph/osd/ceph-OSD_ID:z
```

For example,

```
[root@host01 ~]# cephadm shell -m /var/lib/ceph/1a6112da-ed05-11ee-bacd-525400565cda/
osd.3:/var/lib/ceph/osd/ceph-3:z
```

The `ceph-bluestore-tool` needs to access the BlueStore data from within the `cephadm` shell container, so it must be bind-mounted. Use the `-m` option to make the BlueStore data available.

6. Check the size of the Rocks DB before expansion.

```
ceph-bluestore-tool show-label --path OSD_DIRECTORY_PATH
```

For example,

```
[ceph: root@host01 /]# ceph-bluestore-tool show-label --path /var/lib/ceph/osd/ceph-3/
inferring bluefs devices from bluestore path
{
    "/var/lib/ceph/osd/ceph-3/block": {
        "osd_uuid": "94ff742c-7bfd-4fb5-8dc4-843d10ac6731",
        "size": 32212254720,
        "btime": "2024-04-03T08:34:12.742848+0000",
        "description": "main",
        "bfm_blocks": "7864320",
        "bfm_blocks_per_key": "128",
        "bfm_bytes_per_block": "4096",
        "bfm_size": "32212254720",
        "bluefs": "1",
        "ceph_fsid": "1a6112da-ed05-11ee-bacd-525400565cda",
        "ceph_version_when_created": "ceph version 19.0.0-2493-gd82c9aa1
(d82c9aa17f09785fe698d262f9601d87bb79f962) squid (dev)",
        "created_at": "2024-04-03T08:34:15.637253Z",
        "elastic_shared_blobs": "1",
        "kv_backend": "rocksdb",
        "magic": "ceph osd volume v026",
        "mkfs_done": "yes",
        "osd_key": "AQCEFA1m9xuwABAAwKEHkASVbgB1GVt5jYC2Sg==",
        "osdspec_affinity": "None",
        "ready": "ready",
        "require_osd_release": "19",
        "whoami": "3"
    },
    "/var/lib/ceph/osd/ceph-3/block.db": {
        "osd_uuid": "94ff742c-7bfd-4fb5-8dc4-843d10ac6731",
        "size": 40794497536,
        "btime": "2024-04-03T08:34:12.748816+0000",
        "description": "bluefs db"
    }
}
```

7. Expand the BlueStore device.

```
ceph-bluestore-tool bluefs-bdev-expand --path OSD_DIRECTORY_PATH
```

For example,

```
[ceph: root@host01 /]# ceph-bluestore-tool bluefs-bdev-expand --path /var/lib/ceph/
osd/ceph-3/
inferring bluefs devices from bluestore path
1 : device size 0x27ffbfe000 : using 0x2300000(35 MiB)
2 : device size 0x780000000 : using 0x52000(328 KiB)
Expanding DB/WAL...
1 : expanding  to 0x171794497536
1 : size label updated to 171794497536
```

8. Verify the `block.db` is expanded.

```
ceph-bluestore-tool show-label --path OSD_DIRECTORY_PATH
```

For example,

```
[ceph: root@host01 /]# ceph-bluestore-tool show-label --path /var/lib/ceph/osd/ceph-3/
inferring bluefs devices from bluestore path
{
    "/var/lib/ceph/osd/ceph-3/block": {
        "osd_uuid": "94ff742c-7bfd-4fb5-8dc4-843d10ac6731",
        "size": 32212254720,
        "btime": "2024-04-03T08:34:12.742848+0000",
        "description": "main",
        "bfm_blocks": "7864320",
        "bfm_blocks_per_key": "128",
        "bfm_bytes_per_block": "4096",
        "bfm_size": "32212254720",
        "bluefs": "1",
        "ceph_fsid": "1a6112da-ed05-11ee-bacd-525400565cda",
        "ceph_version_when_created": "ceph version 19.0.0-2493-gd82c9aa1
(d82c9aa17f09785fe698d262f9601d87bb79f962) squid (dev)",
        "created_at": "2024-04-03T08:34:15.637253Z",
        "elastic_shared_blobs": "1",
        "kv_backend": "rocksdb",
        "magic": "ceph osd volume v026",
        "mkfs_done": "yes",
        "osd_key": "AQCEFA1m9xuwABAAwKEHkASVbgB1GVt5jYC2Sg==",
        "osdspec_affinity": "None",
        "ready": "ready",
        "require_osd_release": "19",
        "whoami": "3"
    },
    "/var/lib/ceph/osd/ceph-3/block.db": {
        "osd_uuid": "94ff742c-7bfd-4fb5-8dc4-843d10ac6731",
        "size": 171794497536,
        "btime": "2024-04-03T08:34:12.748816+0000",
        "description": "bluefs db"
    }
```

9. Exit the shell and restart the OSD.

```
systemctl start SERVICE_ID
```

For example,

```
[root@host01 ~]# systemctl start host01a6112da-ed05-11ee-
bacd-525400565cda@osd.3.service
osd.3              host01               running (15s)     0s ago  13m    46.9M
 4096M  19.0.0-2493-gd82c9aa1  3714003597ec  02150b3b6877
```

Using batch mode

The `batch` subcommand automates the creation of multiple OSDs when single devices are provided.

The `ceph-volume` command decides the best method to use to create the OSDs, based on drive type. Ceph OSD optimization depends on the available devices:

– If all devices are traditional hard drives, `batch` creates one OSD per device.

- If all devices are solid state drives, `batch` creates two OSDs per device.
- If there is a mix of traditional hard drives and solid state drives, `batch` uses the traditional hard drives for data, and creates the largest possible journal (`block.db`) on the solid state drive.

> **Note:** The **`batch`** subcommand does not support the creation of a separate logical volume for the write-ahead-log (`block.wal`) device.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph OSD nodes.

Procedure

1. To create OSDs on several drives:

```
ceph-volume lvm batch --bluestore PATH_TO_DEVICE [PATH_TO_DEVICE]
```

   Example

```
[ceph: root@host01 /]# ceph-volume lvm batch --bluestore /dev/sda /dev/sdb /dev/
nvme0n1
```

Reference

- For more information, see "Creating Ceph OSDs" on page 457.

Zapping data

The `zap` subcommand removes all data and file systems from a logical volume or partition. Use the `zap` subcommand to zap logical volumes, partitions, or raw devices that are used by Ceph OSDs for reuse. Any file system present on the given logical volume or partition are removed and all data is purged.

Optionally, you can use the `--destroy` flag for complete removal of a logical volume, partition, or the physical device.

> **Important:** Do not run **`ceph-volume`** from a default `cephadm` shell session. The default shell does not expose the bind mounts that **`ceph-volume`** requires. These mounts are reserved for dedicated, short-lived **`ceph-volume`** containers because they could cause unintended side effects in certain scenarios, particularly with long-running shell containers. As a result, **`ceph-volume`** commands might fail or behave unexpectedly in a default `cephadm` shell session.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph OSD node.

Procedure

- Zap the logical volume:

  Syntax

```
ceph-volume lvm zap VOLUME_GROUP_NAME/LOGICAL_VOLUME_NAME [--destroy]
```

  Example

```
[ceph: root@host01 /]# ceph-volume lvm zap osd-vg/data-lv
```

- Zap the partition:

  Syntax

  ```
  ceph-volume lvm zap DEVICE_PATH_PARTITION [--destroy]
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph-volume lvm zap /dev/sdc1
  ```

- Zap the raw device:

  Syntax

  ```
  ceph-volume lvm zap DEVICE_PATH --destroy
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph-volume lvm zap /dev/sdc --destroy
  ```

- Purge multiple devices with the OSD ID:

  Syntax

  ```
  ceph-volume lvm zap --destroy --osd-id OSD_ID
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph-volume lvm zap --destroy --osd-id 16
  ```

  Note: All the relative devices are zapped.

- Purge OSDs with the FSID:

  Syntax

  ```
  ceph-volume lvm zap --destroy --osd-fsid OSD_FSID
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph-volume lvm zap --destroy --osd-fsid 65d7b6b1-e41a-4a3c-
  b363-83ade63cb32b
  ```

  Note: All the relative devices are zapped.

Ceph performance benchmark

As a storage administrator, you can benchmark performance of the Red Hat Ceph Storage cluster.

Use this information to gain a basic understanding of Ceph's native benchmarking tools. These tools will provide some insight into how the Ceph storage cluster is performing. This is not the definitive guide to Ceph performance benchmarking, nor is it a guide on how to tune Ceph accordingly.

Performance baseline

The OSD, including the journal, disks and the network throughput should each have a performance baseline to compare against. You can identify potential tuning opportunities by comparing the baseline performance data with the data from Ceph’s native tools.

Red Hat Enterprise Linux has many built-in tools, along with a plethora of open source community tools, available to help accomplish these tasks.

Reference

- For more details about some of the available tools, see this Knowledge base article.

Benchmarking Ceph performance

Ceph includes the **rados bench** command to do performance benchmarking on a RADOS storage cluster. The command runs a write test and two types of read tests. The `--no-cleanup` option is important to use when testing both read and write performance.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

About this task

By default, the **rados bench** command will delete the objects it has written to the storage pool. Leaving behind these objects allows the two read tests to measure sequential and random read performance.

> **Note:** Before running these performance tests, drop all the file system caches by running the following:
>
> ```
> [ceph: root@host01 /]# echo 3 | sudo tee /proc/sys/vm/drop_caches && sudo sync
> ```

Procedure

1. Create a new storage pool.
   For example,

   ```
   [ceph: root@host01 /]# ceph osd pool create testbench 100 100
   ```

2. Run a write test for 10 seconds to the storage pool created in the previous step.
   For example,

```
[ceph: root@host01 /]# rados bench -p testbench 10 write --no-cleanup

Maintaining 16 concurrent writes of 4194304 bytes for up to 10 seconds or 0 objects
 Object prefix: benchmark_data_cephn1.home.network_10510
   sec Cur ops   started  finished  avg MB/s  cur MB/s  last lat   avg lat
     0       0         0         0         0         0         -         0
     1      16        16         0         0         0         -         0
     2      16        16         0         0         0         -         0
     3      16        16         0         0         0         -         0
     4      16        17         1  0.998879         1   3.19824   3.19824
     5      16        18         2   1.59849         4   4.56163   3.87993
     6      16        18         2   1.33222         0         -   3.87993
     7      16        19         3   1.71239         2   6.90712     4.889
     8      16        25         9   4.49551        24   7.75362   6.71216
     9      16        25         9   3.99636         0         -   6.71216
    10      16        27        11   4.39632         4   9.65085   7.18999
    11      16        27        11   3.99685         0         -   7.18999
    12      16        27        11   3.66397         0         -   7.18999
    13      16        28        12   3.68975   1.33333   12.8124   7.65853
    14      16        28        12   3.42617         0         -   7.65853
    15      16        28        12   3.19785         0         -   7.65853
    16      11        28        17   4.24726   6.66667   12.5302   9.27548
    17      11        28        17   3.99751         0         -   9.27548
    18      11        28        17   3.77546         0         -   9.27548
    19      11        28        17   3.57683         0         -   9.27548
 Total time run:         19.505620
Total writes made:      28
Write size:             4194304
Bandwidth (MB/sec):     5.742

Stddev Bandwidth:       5.4617
Max bandwidth (MB/sec): 24
Min bandwidth (MB/sec): 0
Average Latency:        10.4064
Stddev Latency:         3.80038
Max latency:            19.503
Min latency:            3.19824
```

3. Run a sequential read test for 10 seconds to the storage pool.
   For example,

```
[ceph: root@host01 /]# rados bench -p testbench 10 seq

sec Cur ops   started  finished  avg MB/s  cur MB/s  last lat   avg lat
  0       0         0         0         0         0         -         0
Total time run:        0.804869
Total reads made:      28
Read size:             4194304
Bandwidth (MB/sec):    139.153

Average Latency:       0.420841
Max latency:           0.706133
Min latency:           0.0816332
```

4. Run a random read test for 10 seconds to the storage pool.
   For example,

```
[ceph: root@host01 /]# rados bench -p testbench 10 rand

sec Cur ops   started  finished  avg MB/s  cur MB/s  last lat   avg lat
  0       0         0         0         0         0         -         0
  1      16        46        30   119.801       120  0.440184  0.388125
  2      16        81        65   129.408       140  0.577359  0.417461
  3      16       120       104   138.175       156  0.597435  0.409318
  4      15       157       142   141.485       152  0.683111  0.419964
  5      16       206       190   151.553       192  0.310578  0.408343
  6      16       253       237   157.608       188 0.0745175  0.387207
  7      16       287       271   154.412       136  0.792774   0.39043
  8      16       325       309   154.044       152  0.314254   0.39876
  9      16       362       346   153.245       148  0.355576  0.406032
 10      16       405       389   155.092       172   0.64734  0.398372
Total time run:        10.302229
Total reads made:      405
Read size:             4194304
Bandwidth (MB/sec):    157.248

Average Latency:       0.405976
Max latency:           1.00869
Min latency:           0.0378431
```

5. Use additional parameters, as needed.
   - Increase the number of concurrent reads and writes, by using the **-t** parameter.
     The default value is 16 threads.
   - Adjust the size of the object being written, by using the **-b** parameter.
     The default object size is 4 MB.

     The maximum object size is 16 MB.
   - Control the names of the objects that get written during the benchmark test, by using the **--run-name** option, with the *LABEL* value.

   > **Tip:** Run multiple copies of these benchmark tests to different pools to view the changes in performance from multiple clients.

   Run multiple **rados bench** commands simultaneously, by changing the **--run-name** label for each running command instance. Changing the label prevents potential I/O errors that can occur when multiple clients are trying to access the same object. The multiple labels also allows for different clients to access different objects.

   Use the **--run-name** option when trying to simulate a real world workload.

```
[ceph: root@host01 /]# rados bench -p testbench 10 write -t 4 --run-name client1

Maintaining 4 concurrent writes of 4194304 bytes for up to 10 seconds or 0 objects
 Object prefix: benchmark_data_node1_12631
   sec Cur ops   started  finished  avg MB/s  cur MB/s  last lat   avg lat
     0       0         0         0         0         0         -         0
     1       4         4         0         0         0         -         0
     2       4         6         2   3.99099         4   1.94755   1.93361
     3       4         8         4   5.32498         8     2.978   2.44034
     4       4         8         4   3.99504         0         -   2.44034
     5       4        10         6   4.79504         4   2.92419    2.4629
     6       3        10         7   4.64471         4   3.02498    2.5432
     7       4        12         8   4.55287         4   3.12204   2.61555
     8       4        14        10    4.9821         8   2.55901   2.68396
     9       4        16        12   5.31621         8   2.68769   2.68081
    10       4        17        13   5.18488         4   2.11937   2.63763
    11       4        17        13   4.71431         0         -   2.63763
    12       4        18        14   4.65486         2    2.4836   2.62662
    13       4        18        14   4.29757         0         -   2.62662
Total time run:         13.123548
Total writes made:      18
Write size:             4194304
Bandwidth (MB/sec):     5.486

Stddev Bandwidth:       3.0991
Max bandwidth (MB/sec): 8
Min bandwidth (MB/sec): 0
Average Latency:        2.91578
Stddev Latency:         0.956993
Max latency:            5.72685
Min latency:            1.91967
```

6. Remove the data created by the **rados bench** command.
   For example,

```
[ceph: root@host01 /]# rados -p testbench cleanup
```

Benchmarking Ceph block performance

Ceph includes the **rbd bench-write** command to test sequential writes to the block device measuring throughput and latency.

The default byte size is 4096, the default number of I/O threads is 16, and the default total number of bytes to write is 1 GB. These defaults can be modified by the `--io-size`, `--io-threads` and `--io-total` options respectively.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.
- Run the write performance test against the block device:

  Example

```
[root@host01 ~]# rbd bench --io-type write image01 --pool=testbench

bench-write  io_size 4096 io_threads 16 bytes 1073741824 pattern seq
  SEC       OPS   OPS/SEC   BYTES/SEC
    2     11127   5479.59  22444382.79
    3     11692   3901.91  15982220.33
    4     12372   2953.34  12096895.42
    5     12580   2300.05  9421008.60
    6     13141   2101.80  8608975.15
    7     13195    356.07  1458459.94
    8     13820    390.35  1598876.60
    9     14124    325.46  1333066.62
    ..
```

Reference

- For more information about the **rbd** command, see "Using Ceph Block Devices" on page 1256.

Benchmarking CephFS performance

Benchmark Ceph File System (CephFS) performance with the FIO tool.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.
- FIO tool is installed on the nodes.
- Block Device or the Ceph File System mounted on the node.

About this task

You can use the FIO tool to benchmark Ceph File System (CephFS) performance. This tool can also be used to benchmark Ceph Block Device.

Procedure

1. Navigate to the node or the application where the Block Device or the CephFS is mounted.

```
[root@host01 ~]# cd /mnt/ceph-block-device
 [root@host01 ~]# cd /mnt/ceph-file-system
```

2. Run FIO command. Start the `bs` value from 4k and repeat in power of 2 increments (4k, 8k, 16k, 32k ... 128k... 512k, 1 m, 2 m, 4 m) and with different `iodepth` settings. Also run tests at your expected workload operation size.
   For example, 4 K tests with different `iodepth` values:

```
fio --name=randwrite --rw=randwrite --direct=1 --ioengine=libaio --bs=4k --iodepth=32
--size=5G --runtime=60 --group_reporting=1
```

   For example, 8 K tests with different `iodepth` values:

```
fio --name=randwrite --rw=randwrite --direct=1 --ioengine=libaio --bs=8k --iodepth=32
 --size=5G --runtime=60 --group_reporting=1
```

> **Note:** For more information on the usage of `fio` command, see the `fio` man page.

Benchmarking Ceph Object Gateway performance

Benchmark Ceph Object Gateway performance with the `s3cmd` tool.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.
- `s3cmd` installed on the nodes.

About this task

You can use the `s3cmd` tool to benchmark Ceph Object Gateway performance.

Use `get` and `put` requests to determine the performance.

Procedure

1. Upload a file and measure the speed. The `time` command measures the duration of upload.

   ```
   time s3cmd put PATH_OF_SOURCE_FILE PATH_OF_DESTINATION_FILE
   ```

   For example,

   ```
   time s3cmd put /path-to-local-file s3://bucket-name/remote/file
   ```

   Replace `/path-to-local-file` with the file you want to upload and `s3://bucket-name/remote/file` with the destination in your S3 bucket.

2. Download a file and measure the speed. The `time` command measures the duration of download.

   ```
   time s3cmd get PATH_OF_DESTINATION_FILE DESTINATION_PATH
   ```

   For example,

   ```
   time s3cmd get s3://bucket-name/remote/file /path-to-local-destination
   ```

   Replace `s3://bucket-name/remote/file` with the S3 object that you want to download and `/path-to-local-destination` with the local directory where you want to save the file.

3. List all the objects in the specified bucket and measure response time.

   ```
   time s3cmd ls s3://BUCKET_NAME
   ```

   For example,

   ```
   time s3cmd ls s3://bucket-name
   ```

4. Analyze the output to calculate upload and download speed. Measure response time reported by the `time` command.

Ceph performance counters

As a storage administrator, you can gather performance metrics of the Red Hat Ceph Storage cluster. The Ceph performance counters are a collection of internal infrastructure metrics. The collection, aggregation, and graphing of this metric data can be done by an assortment of tools and can be useful for performance analytics.

Access to Ceph performance counters

The performance counters are available through a socket interface for the Ceph Monitors and the OSDs. The socket file for each respective daemon is located under `/var/run/ceph`, by default. The performance counters are grouped together into collection names. These collections names represent a subsystem or an instance of a subsystem.

The following is a list of the Monitor and the OSD collection name categories with a brief description for each.

Monitor collection name categories

Cluster metrics

Displays information about the storage cluster: Monitors, OSDs, pools, and placement groups.

Level database metrics

Displays information about the back-end `KeyValueStore` database.

Monitor metrics

Displays general monitor information.

Paxos metrics

Displays information on cluster quorum management.

Throttle metrics

Displays the statistics on how the monitor is throttling.

OSD collection name categories

Write back throttle metrics

Displays the statistics on how the write back throttle is tracking unflushed I/O.

Level database metrics

Displays information about the back-end `KeyValueStore` database.

Objecter metrics

Displays information on various object-based operations.

Read and write operations metrics

Displays information on various read and write operations.

Recovery state metrics

Displays latencies on various recovery states.

OSD throttle metrics

Display the statistics on how the OSD is throttling.

RADOS Gateway collection name categories

Object Gateway client metrics

Displays statistics on GET and PUT requests.

Objecter metrics

Displays information on various object-based operations.

Object Gateway throttle metrics

Displays the statistics on how the OSD is throttling.

Display Ceph performance counters

The `ceph daemon` *`DAEMON_NAME`* `perf schema` command outputs the available metrics. Each metric has an associated bit field value type.

Prerequisites

- A running Red Hat Ceph Storage cluster.

– Root-level access to the node.

Procedure

1. To view the metric's schema:

   Syntax

   ```
   ceph daemon DAEMON_NAME perf schema
   ```

   Note: Run the **ceph daemon** command from the node running the daemon.

2. Executing `ceph daemon DAEMON_NAME perf schema` command from the monitor node:

   Example

   ```
   [ceph: root@host01 /]# ceph daemon mon.host01 perf schema
   ```

3. Executing the `ceph daemon DAEMON_NAME perf schema` command from the OSD node:

   Example

   ```
   [ceph: root@host01 /]# ceph daemon osd.11 perf schema
   ```

Table 62: The bit field value definitions

Bit	Meaning
1	Floating point value
2	Unsigned 64-bit integer value
4	Average (Sum + Count)
8	Counter

Each value will have bit 1 or 2 set to indicate the type, either a floating point or an integer value. When bit 4 is set, there will be two values to read, a sum and a count. When bit 8 is set, the average for the previous interval would be the sum delta, since the previous read, divided by the count delta. Alternatively, dividing the values outright would provide the lifetime average value. Typically these are used to measure latencies, the number of requests and a sum of request latencies. Some bit values are combined, for example 5, 6 and 10. A bit value of 5 is a combination of bit 1 and bit 4. This means the average will be a floating point value. A bit value of 6 is a combination of bit 2 and bit 4. This means the average value will be an integer. A bit value of 10 is a combination of bit 2 and bit 8. This means the counter value will be an integer value.

Reference

– "Average count and sum" on page 474

Viewing the performance counters for users and buckets

The Ceph Object Gateway uses the performance counters to track metrics. You can visualize a cluster-wide view of the usage data over time in the Ceph Exporter port, which is usually, `9926`, which includes PUT operations for objects in a bucket.

Before you begin

1. A running Red Hat Ceph Storage cluster with Ceph Object Gateway installed.
2. Monitoring stack enabled which includes Prometheus and `ceph-exporter`.

About this task

To track the operation metrics by users, set the `rgw_user_counters_cache` to `true` and to track the operation metrics by buckets, set the `rgw_bucket_counters_cache` to `true`.

You can use both `rgw_user_counters_cache_size` and `rgw_bucket_counters_cache_size` to set the number of entries in each cache.

Counters are evicted from a cache once the number of counters in the cache are greater than the cache size configuration variable. The counters that are evicted are the least recently used (LRU).

For example, if the number of buckets exceeded `rgw_bucket_counters_cache_size` by `1` and the counters with label `bucket1` were the last to be updated, the counters for `bucket1` get evicted from the cache. If S3 operations tracked by the operation metrics were done on `bucket1` after eviction, all the metrics in the cache for `bucket1` start at `0`.

Cache sizing can depend on several factors, which include the following:

- Number of users in the cluster.
- Number of buckets in the cluster.
- Memory usage of the Ceph Object Gateway.
- Disk and memory usage of Prometheus.
- To help calculate the Ceph Object Gateway's memory usage of a cache, it should be noted that each cache entry, encompassing all the operation metrics, is 1360 bytes. This value is an estimate and subject to change if metrics are added or removed from the operation metrics list.

Important:
Since the operation metrics are labeled as performance counters, they live in memory. If the Ceph Object Gateway is restarted or crashes, all counters in the Ceph Object Gateway, whether in a cache or not, are lost.

Procedure

1. Set the performance counters for users and buckets.
   a. Set the performance counter for users.
   For example,

   ```
   [ceph: root@host01 /]# ceph config set client.rgw.rgw.1.host04
   rgw_user_counters_cache true
   ```

   b. Set the performance counter for buckets.
   For example,

   ```
   [ceph: root@host01 /]# ceph config set client.rgw.rgw.1.host04
   rgw_bucket_counters_cache true
   ```

   c. Restart the Ceph Object Gateway service.

   ```
   [ceph: root@host01 /]# ceph orch restart rgw.rgw.1
   ```

2. Create users. For more information, see "User management" on page 948.
3. Create buckets and upload objects into the bucket.
   a. Configure `s3cmd`.

   ```
   [root@host01 ~]# s3cmd --configure
   ```

   b. Create the S3 bucket.

   ```
   s3cmd mb s3://NAME_OF_THE_BUCKET_FOR_S3
   ```

   For example,

```
[root@host01 ~]# s3cmd mb s3://bucket
Bucket 's3://bucket/' created
```

c. Create your file, input all the data, upload buckets on S3.

```
s3cmd put FILE_NAME s3://NAME_OF_THE_BUCKET_FOR_S3
```

For example,

```
[root@host01 ~]# s3cmd put test.txt s3://bucket

upload: 'test.txt' -> 's3://bucket/test.txt'  [1 of 1]
 21 of 21   100% in    1s    16.75 B/s  done
```

d. Verify that the objects are uploaded.

```
config dump ceph daemon DAEMON_ID counter dump
```

4. Verify that the metrics are running on the local host.

```
http://RGW_IP_ADDRESS:CEPH-EXPORTER_PORT
```

For example,

```
http://10.0.210.100:9926/
ceph_rgw_op_get_obj_ops{Bucket="bucket",instance_id="host04"} 0
ceph_rgw_op_get_obj_ops{User="admin",instance_id="host04"} 0
```

5. Verify the metrics on Prometheus.

```
https://RGW_IP_ADDRESS:PROMETHEUS_PORT
```

For example,

```
https://10.0.210.100:9283/
```

Dump Ceph performance counters

The `ceph daemon .. perf dump` command outputs the current values and groups the metrics under the collection name for each subsystem.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

1. To view the current metric data:

Syntax

```
ceph daemon DAEMON_NAME perf dump
```

> **Note:** run the **ceph daemon** command from the node running the daemon.

2. Executing `ceph daemon .. perf dump` command from the Monitor node:

```
[ceph: root@host01 /]# ceph daemon mon.host01 perf dump
```

3. Executing the `ceph daemon .. perf dump` command from the OSD node:

```
[ceph: root@host01 /]# ceph daemon osd.11 perf dump
```

Reference

- To view a short description of each Monitor metric available, see “Ceph Monitor metrics” on page 474.

Average count and sum

The `avgcount` is the number of operations within this range and the `sum` is the total latency in seconds. To know approximately how much latency there is per operation, divide the `sum` by the `avgcount`.

All latency numbers have a bit field value of 5. This field contains floating point values for the average count and sum.

For a short description of each OSD metric available, see “Ceph OSD metrics” on page 478.

Ceph Monitor metrics

Use this information to understand different Ceph Monitor metrics.

- “Cluster metrics” on page 474
- “Level database metrics” on page 475
- “General monitor metrics” on page 475
- “Paxos metrics” on page 476
- “Throttle metrics” on page 477

Cluster metrics

“Cluster metrics table” on page 474 describes the cluster metrics for the `cluster` collection.

Table 63: Cluster metrics table

Metric Name	Bit Field Value	Short Description
`num_mon`	2	Number of monitors
`num_mon_quorum`	2	Number of monitors in quorum
`num_osd`	2	Total number of OSD
`num_osd_up`	2	Number of OSDs that are up
`num_osd_in`	2	Number of OSDs that are in cluster
`osd_epoch`	2	Current epoch of OSD map
`osd_bytes`	2	Total capacity of cluster in bytes
`osd_bytes_used`	2	Number of used bytes on cluster
`osd_bytes_avail`	2	Number of available bytes on cluster
`num_pool`	2	Number of pools
`num_pg`	2	Total number of placement groups
`num_pg_active_clean`	2	Number of placement groups in active+clean state

Metric Name	Bit Field Value	Short Description
num_pg_active	2	Number of placement groups in active state
num_pg_peering	2	Number of placement groups in peering state
num_object	2	Total number of objects on cluster
num_object_degraded	2	Number of degraded (missing replicas) objects
num_object_misplaced	2	Number of misplaced (wrong location in the cluster) objects
num_object_unfound	2	Number of unfound objects
num_bytes	2	Total number of bytes of all objects
num_mds_up	2	Number of MDSs that are up
num_mds_in	2	Number of MDS that are in cluster
num_mds_failed	2	Number of failed MDS
mds_epoch	2	Current epoch of MDS map

Level database metrics

"Level database metrics table" on page 475 describes the level database metrics for the `leveldb` collection.

Table 64: Level database metrics table

Metric Name	Bit Field Value	Short Description
leveldb_get	10	Gets
leveldb_transaction	10	Transactions
leveldb_compact	10	Compactions
leveldb_compact_range	10	Compactions by range
leveldb_compact_queue_merge	10	Merging of ranges in compaction queue
leveldb_compact_queue_len	2	Length of compaction queue

General monitor metrics

"General monitor metrics" on page 475 describes the general monitor metrics for the `mon` collection.

Table 65: General monitor metrics table

Metric Name	Bit Field Value	Short Description
num_sessions	2	Current number of opened monitor sessions
session_add	10	Number of created monitor sessions

Metric Name	Bit Field Value	Short Description
session_rm	10	Number of remove_session calls in monitor
session_trim	10	Number of trimmed monitor sessions
num_elections	10	Number of elections monitor took part in
election_call	10	Number of elections started by monitor
election_win	10	Number of elections won by monitor
election_lose	10	Number of elections lost by monitor

Paxos metrics

"Paxos metrics" on page 476 describes the Paxos metrics for the paxos collection.

Table 66: Paxos metrics table

Metric Name	Bit Field Value	Short Description
start_leader	10	Starts in leader role
start_peon	10	Starts in peon role
restart	10	Restarts
refresh	10	Refreshes
refresh_latency	5	Refresh latency
begin	10	Started and handled begins
begin_keys	6	Keys in transaction on begin
begin_bytes	6	Data in transaction on begin
begin_latency	5	Latency of begin operation
commit	10	Commits
commit_keys	6	Keys in transaction on commit
commit_bytes	6	Data in transaction on commit
commit_latency	5	Commit latency
collect	10	Peon collects
collect_keys	6	Keys in transaction on peon collect
collect_bytes	6	Data in transaction on peon collect
collect_latency	5	Peon collect latency

Metric Name	Bit Field Value	Short Description
collect_uncommitted	10	Uncommitted values in started and handled collects
collect_timeout	10	Collect timeouts
accept_timeout	10	Accept timeouts
lease_ack_timeout	10	Lease acknowledgment timeouts
lease_timeout	10	Lease timeouts
store_state	10	Store a shared state on disk
store_state_keys	6	Keys in transaction in stored state
store_state_bytes	6	Data in transaction in stored state
store_state_latency	5	Storing state latency
share_state	10	Sharing of state
share_state_keys	6	Keys in shared state
share_state_bytes	6	Data in shared state
new_pn	10	New proposal number queries
new_pn_latency	5	New proposal number getting latency

Throttle metrics

"Throttle metrics" on page 477 describes the throttle metrics for the `throttle-*` collection.

Table 67: Throttle Metrics Table

Metric Name	Bit Field Value	Short Description
val	10	Currently available throttle
max	10	Max value for throttle
get	10	Gets
get_sum	10	Got data
get_or_fail_fail	10	Get blocked during get_or_fail
get_or_fail_success	10	Successful get during get_or_fail
take	10	Takes
take_sum	10	Taken data
put	10	Puts
put_sum	10	Put data
wait	5	Waiting latency

Ceph OSD metrics

Use this information to understand different Ceph OSD metrics.

- "Write back throttle metrics" on page 478
- "Level database metrics" on page 478
- "Objecter metrics" on page 478
- "Read and write operations metrics" on page 481
- "Recovery state metrics" on page 483
- "OSD throttle metrics" on page 485

Write back throttle metrics

"Write back throttle metrics table" on page 478 describes the write back throttle metrics for the `WBThrottle` collection.

Table 68: Write back throttle metrics table

Metric Name	Bit Field Value	Short Description
`bytes_dirtied`	2	Dirty data
`bytes_wb`	2	Written data
`ios_dirtied`	2	Dirty operations
`ios_wb`	2	Written operations
`inodes_dirtied`	2	Entries waiting for write
`inodes_wb`	2	Written entries

Level database metrics

"Level database metrics" on page 478 describes the level database metrics for the `leveldb` collection.

Table 69: Level database metrics table

Collection Name	Metric Name	Bit Field Value	Short Description
`leveldb`	`leveldb_get`	10	Gets
	`leveldb_transaction`	10	Transactions
	`leveldb_compact`	10	Compactions
	`leveldb_compact_range`	10	Compactions by range
	`leveldb_compact_queue_merge`	10	Merging of ranges in compaction queue
	`leveldb_compact_queue_len`	2	Length of compaction queue

Objecter metrics

"Objecter metrics" on page 478 describes the objecter metrics for the `objecter` collection.

Table 70: Objecter metrics table

Metric Name	Bit Field Value	Short Description
op_active	2	Active operations
op_laggy	2	Laggy operations
op_send	10	Sent operations
op_send_bytes	10	Sent data
op_resend	10	Resent operations
op_ack	10	Commit callbacks
op_commit	10	Operation commits
op	10	Operation
op_r	10	Read operations
op_w	10	Write operations
op_rmw	10	Read-modify-write operations
op_pg	10	PG operation
osdop_stat	10	Stat operations
osdop_create	10	Create object operations
osdop_read	10	Read operations
osdop_write	10	Write operations
osdop_writefull	10	Write full object operations
osdop_append	10	Append operation
osdop_zero	10	Set object to zero operations
osdop_truncate	10	Truncate object operations
osdop_delete	10	Delete object operations
osdop_mapext	10	Map extent operations
osdop_sparse_read	10	Sparse read operations
osdop_clonerange	10	Clone range operations
osdop_getxattr	10	Get xattr operations
osdop_setxattr	10	Set xattr operations
osdop_cmpxattr	10	Xattr comparison operations
osdop_rmxattr	10	Remove xattr operations
osdop_resetxattrs	10	Reset xattr operations
osdop_tmap_up	10	TMAP update operations

Metric Name	Bit Field Value	Short Description
osdop_tmap_put	10	TMAP put operations
osdop_tmap_get	10	TMAP get operations
osdop_call	10	Call (run) operations
osdop_watch	10	Watch by object operations
osdop_notify	10	Notify about object operations
osdop_src_cmpxattr	10	Extended attribute comparison in multi operations
osdop_other	10	Other operations
linger_active	2	Active lingering operations
linger_send	10	Sent lingering operations
linger_resend	10	Resent lingering operations
linger_ping	10	Sent pings to lingering operations
poolop_active	2	Active pool operations
poolop_send	10	Sent pool operations
poolop_resend	10	Resent pool operations
poolstat_active	2	Active get pool stat operations
poolstat_send	10	Pool stat operations sent
poolstat_resend	10	Resent pool stats
statfs_active	2	Statfs operations
statfs_send	10	Sent FS stats
statfs_resend	10	Resent FS stats
command_active	2	Active commands
command_send	10	Sent commands
command_resend	10	Resent commands
map_epoch	2	OSD map epoch
map_full	10	Full OSD maps received
map_inc	10	Incremental OSD maps received
osd_sessions	2	Open sessions
osd_session_open	10	Sessions opened
osd_session_close	10	Sessions closed
osd_laggy	2	Laggy OSD sessions

Read and write operations metrics

"Read and write operations metrics" on page 481 describes the read and write operations metrics for the `osd` collection.

Table 71: Read and write operations metrics table

Metric Name	Bit Field Value	Short Description
`op_wip`	2	Replication operations currently being processed (primary)
`op_in_bytes`	10	Client operations total write size
`op_out_bytes`	10	Client operations total read size
`op_latency`	5	Latency of client operations (including queue time)
`op_process_latency`	5	Latency of client operations (excluding queue time)
`op_r`	10	Client read operations
`op_r_out_bytes`	10	Client data read
`op_r_latency`	5	Latency of read operation (including queue time)
`op_r_process_latency`	5	Latency of read operation (excluding queue time)
`op_w`	10	Client write operations
`op_w_in_bytes`	10	Client data written
`op_w_rlat`	5	Client write operation readable/ applied latency
`op_w_latency`	5	Latency of write operation (including queue time)
`op_w_process_latency`	5	Latency of write operation (excluding queue time)
`op_rw`	10	Client read-modify-write operations
`op_rw_in_bytes`	10	Client read-modify-write operations write in
`op_rw_out_bytes`	10	Client read-modify-write operations read out
`op_rw_rlat`	5	Client read-modify-write operation readable/applied latency
`op_rw_latency`	5	Latency of read-modify-write operation (including queue time)
`op_rw_process_latency`	5	Latency of read-modify-write operation (excluding queue time)

Metric Name	Bit Field Value	Short Description
subop	10	Sub operations
subop_in_bytes	10	Sub operations total size
subop_latency	5	Sub operations latency
subop_w	10	Replicated writes
subop_w_in_bytes	10	Replicated written data size
subop_w_latency	5	Replicated writes latency
subop_pull	10	Sub operations pull requests
subop_pull_latency	5	Sub operations pull latency
subop_push	10	Sub operations push messages
subop_push_in_bytes	10	Sub operations pushed size
subop_push_latency	5	Sub operations push latency
pull	10	Pull requests sent
push	10	Push messages sent
push_out_bytes	10	Pushed size
push_in	10	Inbound push messages
push_in_bytes	10	Inbound pushed size
recovery_ops	10	Started recovery operations
loadavg	2	CPU load
buffer_bytes	2	Total allocated buffer size
numpg	2	Placement groups
numpg_primary	2	Placement groups for which this osd is primary
numpg_replica	2	Placement groups for which this osd is replica
numpg_stray	2	Placement groups ready to be deleted from this osd
heartbeat_to_peers	2	Heartbeat (ping) peers we send to
heartbeat_from_peers	2	Heartbeat (ping) peers we recv from
map_messages	10	OSD map messages
map_message_epochs	10	OSD map epochs
map_message_epoch_dups	10	OSD map duplicates
stat_bytes	2	OSD size

Metric Name	Bit Field Value	Short Description
stat_bytes_used	2	Used space
stat_bytes_avail	2	Available space
copyfrom	10	RADOS *copy-from* operations
tier_promote	10	Tier promotions
tier_flush	10	Tier flushes
tier_flush_fail	10	Failed tier flushes
tier_try_flush	10	Tier flush attempts
tier_try_flush_fail	10	Failed tier flush attempts
tier_evict	10	Tier evictions
tier_whiteout	10	Tier whiteouts
tier_dirty	10	Dirty tier flag set
tier_clean	10	Dirty tier flag cleaned
tier_delay	10	Tier delays (agent waiting)
tier_proxy_read	10	Tier proxy reads
agent_wake	10	Tiering agent wake up
agent_skip	10	Objects skipped by agent
agent_flush	10	Tiering agent flushes
agent_evict	10	Tiering agent evictions
object_ctx_cache_hit	10	Object context cache hits
object_ctx_cache_total	10	Object context cache lookups
ceph_cluster_osd_blocklist_count	2	Number of clients blocklisted

Recovery state metrics

“Recovery state metrics” on page 483 describes the recovery state metrics for the `recoverystate_perf` collection.

Table 72: Recovery state metrics table

Metric Name	Bit Field Value	Short Description
initial_latency	5	Initial recovery state latency
started_latency	5	Started recovery state latency
reset_latency	5	Reset recovery state latency
start_latency	5	Start recovery state latency
primary_latency	5	Primary recovery state latency

Metric Name	Bit Field Value	Short Description
peering_latency	5	Peering recovery state latency
backfilling_latency	5	Backfilling recovery state latency
waitremotebackfillreserved_latency	5	Wait remote backfill reserved recovery state latency
waitlocalbackfillreserved_latency	5	Wait local backfill reserved recovery state latency
notbackfilling_latency	5	Notbackfilling recovery state latency
repnotrecovering_latency	5	Repnotrecovering recovery state latency
repwaitrecoveryreserved_latency	5	Rep wait recovery reserved recovery state latency
repwaitbackfillreserved_latency	5	Rep wait backfill reserved recovery state latency
RepRecovering_latency	5	RepRecovering recovery state latency
activating_latency	5	Activating recovery state latency
waitlocalrecoveryreserved_latency	5	Wait local recovery reserved recovery state latency
waitremoterecoveryreserved_latency	5	Wait remote recovery reserved recovery state latency
recovering_latency	5	Recovering recovery state latency
recovered_latency	5	Recovered recovery state latency
clean_latency	5	Clean recovery state latency
active_latency	5	Active recovery state latency
replicaactive_latency	5	Replicaactive recovery state latency
stray_latency	5	Stray recovery state latency
getinfo_latency	5	Getinfo recovery state latency
getlog_latency	5	Getlog recovery state latency
waitactingchange_latency	5	Waitactingchange recovery state latency
incomplete_latency	5	Incomplete recovery state latency
getmissing_latency	5	Get missing recovery state latency
waitupthru_latency	5	Waitupthru recovery state latency

OSD throttle metrics

"OSD throttle metrics" on page 485 describes the OSD throttle metrics for the `throttle-*` collection.

Table 73: OSD throttle metrics table

Metric Name	Bit Field Value	Short Description
`val`	10	Currently available throttle
`max`	10	Max value for throttle
`get`	10	Gets
`get_sum`	10	Got data
`get_or_fail_fail`	10	Get blocked during get_or_fail
`get_or_fail_success`	10	Successful get during get_or_fail
`take`	10	Takes
`take_sum`	10	Taken data
`put`	10	Puts
`put_sum`	10	Put data
`wait`	5	Waiting latency

Ceph Object Gateway metrics

Use this information to understand different Ceph Object Gateway metrics.

Ceph Object Gateway client metrics

The following table describes the Ceph Object Gateway client metrics for the `client.rgw.<rgw_node_name>` collection.

Table 74: Ceph Object Gateway client metrics table

Metric Name	Bit Field Value	Short Description
`req`	10	Requests
`failed_req`	10	Aborted requests
`copy_obj_ops`	10	Copy objects
`copy_obj_bytes`	10	Size of copy objects
`copy_obj_lat`	10	Copy object latency
`del_obj_ops`	10	Delete objects
`del_obj_bytes`	10	Size of delete objects
`del_obj_lat`	10	Delete object latency
`del_bucket_ops`	10	Delete Buckets
`del_bucket_lat`	10	Delete bucket latency
`get`	10	Gets
`get_b`	10	Size of gets
`get_initial_lat`	5	Get latency
`list_obj_ops`	10	List objects
`list_obj_lat`	10	List object latency

Metric Name	Bit Field Value	Short Description
list_buckets_ops	10	List buckets
list_buckets_lat	10	List buckets latency
put	10	Puts
put_b	10	Size of puts
put_initial_lat	5	Put latency
qlen	2	Queue length
qactive	2	Active requests queue
cache_hit	10	Cache hits
cache_miss	10	Cache miss
keystone_token_cache_hit	10	Keystone token cache hits
keystone_token_cache_miss	10	Keystone token cache miss

Objecter metrics

The following table describes the objecter metrics for the `objecter` collection.

Table 75: Objecter metrics table

Metric Name	Bit Field Value	Short Description
op_active	2	Active operations
op_laggy	2	Laggy operations
op_send	10	Sent operations
op_send_bytes	10	Sent data
op_resend	10	Resent operations
op_ack	10	Commit callbacks
op_commit	10	Operation commits
op	10	Operation
op_r	10	Read operations
op_w	10	Write operations
op_rmw	10	Read-modify-write operations
op_pg	10	PG operation
osdop_stat	10	Stat operations
osdop_create	10	Create object operations
osdop_read	10	Read operations
osdop_write	10	Write operations
osdop_writefull	10	Write full object operations
osdop_append	10	Append operation
osdop_zero	10	Set object to zero operations
osdop_truncate	10	Truncate object operations
osdop_delete	10	Delete object operations
osdop_mapext	10	Map extent operations
osdop_sparse_read	10	Sparse read operations
osdop_clonerange	10	Clone range operations
osdop_getxattr	10	Get xattr operations
osdop_setxattr	10	Set xattr operations

Metric Name	Bit Field Value	Short Description
osdop_cmpxattr	10	Xattr comparison operations
osdop_rmxattr	10	Remove xattr operations
osdop_resetxattrs	10	Reset xattr operations
osdop_tmap_up	10	TMAP update operations
osdop_tmap_put	10	TMAP put operations
osdop_tmap_get	10	TMAP get operations
osdop_call	10	Call (run) operations
osdop_watch	10	Watch by object operations
osdop_notify	10	Notify about object operations
osdop_src_cmpxattr	10	Extended attribute comparison in multi operations
osdop_other	10	Other operations
linger_active	2	Active lingering operations
linger_send	10	Sent lingering operations
linger_resend	10	Resent lingering operations
linger_ping	10	Sent pings to lingering operations
poolop_active	2	Active pool operations
poolop_send	10	Sent pool operations
poolop_resend	10	Resent pool operations
poolstat_active	2	Active get pool stat operations
poolstat_send	10	Pool stat operations sent
poolstat_resend	10	Resent pool stats
statfs_active	2	Statfs operations
statfs_send	10	Sent FS stats
statfs_resend	10	Resent FS stats
command_active	2	Active commands
command_send	10	Sent commands
command_resend	10	Resent commands
map_epoch	2	OSD map epoch
map_full	10	Full OSD maps received
map_inc	10	Incremental OSD maps received
osd_sessions	2	Open sessions
osd_session_open	10	Sessions opened
osd_session_close	10	Sessions closed
osd_laggy	2	Laggy OSD sessions

Ceph Object Gateway throttle metrics

The following table describes the Ceph Object Gateway throttle metrics for the `throttle-*` collection.

Table 76: Ceph Object Gateway throttle metrics table

Metric Name	Bit Field Value	Short Description
val	10	Currently available throttle
max	10	Max value for throttle
get	10	Gets

Metric Name	Bit Field Value	Short Description
get_sum	10	Got data
get_or_fail_fail	10	Get blocked during get_or_fail
get_or_fail_success	10	Successful get during get_or_fail
take	10	Takes
take_sum	10	Taken data
put	10	Puts
put_sum	10	Put data
wait	5	Waiting latency

mClock OSD scheduler

As a storage administrator, you can implement the Red Hat Ceph Storage's quality of service (QoS) using mClock queueing scheduler. This is based on an adaptation of the mClock algorithm called dmClock.

The mClock OSD scheduler provides the desired QoS using configuration profiles to allocate proper reservation, weight, and limit tags to the service types.

The mClock OSD scheduler performs the QoS calculations for the different device types, that is SSD or HDD, by using the OSD's IOPS capability (determined automatically) and maximum sequential bandwidth capability (See `osd_mclock_max_sequential_bandwidth_hdd` and `osd_mclock_max_sequential_bandwidth_ssd` in the mclock configuration options.

Comparison of mClock OSD scheduler with WPQ OSD scheduler

The mClock OSD scheduler replaces the Weighted Priority Queue (WPQ) OSD scheduler as a default scheduler in Red Hat Ceph Storage 7.

> **Important:** The mClock scheduler is supported for BlueStore OSDs. For Filestore OSDs the `osd_op_queue` is set to `wpq` and it is enforced even if the user attempts to change it.

The WPQ OSD scheduler features a strict sub-queue, which is de-queued before the normal queue. The WPQ removes operations from a queue in relation to their priorities to prevent depletion of any queue. This helps in cases where some Ceph OSDs are more overloaded than others.

The mClock OSD scheduler currently features an immediate queue, into which operations that require immediate response are queued. The immediate queue is not handled by mClock and functions as a simple first in, first out queue and is given the first priority.

Operations, such as OSD replication operations, OSD operation replies, peering, recoveries marked with the highest priority, and so forth, are queued into the immediate queue. All other operations are enqueued into the mClock queue that works according to the mClock algorithm.

The mClock queue, `mclock_scheduler`, prioritizes operations based on which bucket they belong to, that is `pg recovery`, `pg scrub`, `snap trim`, `client op`, and `pg deletion`.

With background operations in progress, the average client throughputs, that is the input and output operations per second (IOPS), are significantly higher and latencies are lesser with the mClock profiles when compared to the WPQ scheduler. That is because of mClock's effective allocation of the QoS parameters.

Reference

- For automated OSD capacity determination, see mClock profiles.

Allocation of input and output resources

This section describes how the QoS controls work internally with reservation, limit, and weight allocation. The user is not expected to set these controls as the mClock profiles automatically set them. Tuning these controls can only be performed using the available mClock profiles.

The dmClock algorithm allocates the input and output (I/O) resources of the Ceph cluster in proportion to weights. It implements the constraints of minimum reservation and maximum limitation to ensure the services can compete for the resources fairly.

Currently, the `mclock_scheduler` operation queue divides Ceph services involving I/O resources into following buckets:

- `client op`: the input and output operations per second (IOPS) issued by a client.
- `pg deletion`: the IOPS issued by primary Ceph OSD.
- `snap trim`: the snapshot trimming-related requests.
- `pg recovery`: the recovery-related requests.
- `pg scrub`: the scrub-related requests.

The resources are partitioned using the following three sets of tags, meaning that the share of each type of service is controlled by these three tags:

- Reservation
- Limit
- Weight

Reservation

The minimum IOPS allocated for the service. The more reservation a service has, the more resources it is guaranteed to possess, as long as it requires so.

For example, a service with the reservation set to 0.1 (or 10%) always has 10% of the OSD's IOPS capacity allocated for itself. Therefore, even if the clients start to issue large amounts of I/O requests, they do not exhaust all the I/O resources and the service's operations are not depleted even in a cluster with high load.

Limit

The maximum IOPS allocated for the service. The service does not get more than the set number of requests per second serviced, even if it requires so and no other services are competing with it. If a service crosses the enforced limit, the operation remains in the operation queue until the limit is restored.

Note: If the value is set to `0` (disabled), the service is not restricted by the limit setting and it can use all the resources if there is no other competing operation. This is represented as "MAX" in the mClock profiles.

Note: The reservation and limit parameter allocations are per-shard, based on the type of backing device, that is HDD or SSD, under the Ceph OSD.

Weight

The proportional share of capacity if extra capacity or system is not enough. The service can use a larger portion of the I/O resource, if its weight is higher than its competitor's.

Note: The reservation and limit values for a service are specified in terms of a proportion of the total IOPS capacity of the OSD. The proportion is represented as a percentage in the mClock profiles. The weight does not have a unit. The weights are relative to one another, so if one class of requests has a weight of 9 and another a weight of 1, then the requests are performed at a 9 to 1 ratio. However, that only happens once the reservations are met and those values include the operations performed under the reservation phase.

Important: If the weight is set to `W`, then for a given class of requests the next one that enters has a weight tag of `1/W` and the previous weight tag, or the current time, whichever is larger. That means, if `W` is

too large and thus `1/W` is too small, the calculated tag might never be assigned as it gets a value of the current time. Therefore, values for weight should be always under the number of requests expected to be serviced each second.

Factors impacting mClock operation queues

Understand the different factors that can reduce the impact of the mClock operation queues within Red Hat Ceph Storage.

There are three factors that can reduce the impact of the mClock operation queues within Red Hat Ceph Storage

- The number of shards for client operations.
- The number of operations in the operation sequencer.
- The usage of distributed system for Ceph OSDs

The number of shards for client operations

Requests to a Ceph OSD are sharded by their placement group identifier. Each shard has its own mClock queue and these queues neither interact, nor share information amongst them.

The number of shards can be controlled with these configuration options:

- `osd_op_num_shards`
- `osd_op_num_shards_hdd`
- `osd_op_num_shards_ssd`

A lesser number of shards increase the impact of the mClock queues, but might have other damaging effects.

Note: Use the default number of shards as defined by the configuration options `osd_op_num_shards`, `osd_op_num_shards_hdd`, and `osd_op_num_shards_ssd`.

The number of operations in the operation sequencer

Requests are transferred from the operation queue to the operation sequencer, in which they are processed. The mClock scheduler is located in the operation queue. It determines which operation to transfer to the operation sequencer.

The number of operations allowed in the operation sequencer is a complex issue. The aim is to keep enough operations in the operation sequencer so it always works on some, while it waits for disk and network access to complete other operations.

However, mClock no longer has control over an operation that is transferred to the operation sequencer. Therefore, to maximize the impact of mClock, the goal is also to keep as few operations in the operation sequencer as possible.

The configuration options that influence the number of operations in the operation sequencer are:

- `bluestore_throttle_bytes`
- `bluestore_throttle_deferred_bytes`
- `bluestore_throttle_cost_per_io`
- `bluestore_throttle_cost_per_io_hdd`
- `bluestore_throttle_cost_per_io_ssd`

Note: Use the default values as defined by the `bluestore_throttle_bytes` and `bluestore_throttle_deferred_bytes` options. However, these options can be determined during the benchmarking phase.

The usage of distributed system for Ceph OSDs

The third factor that affects the impact of the mClock algorithm is the usage of a distributed system, where requests are made to multiple Ceph OSDs, and each Ceph OSD can have multiple shards.

> **Note:** dmClock is the distributed version of mClock.

Reference

- For more details about `osd_op_num_shards_hdd` and `osd_op_num_shards_ssd` parameters, see Object Storage Daemon (OSD) configuration options.
- For more details about BlueStore throttle parameters, see BlueStore configuration options.
- Manually benchmarking OSDs

mClock configuration

The mClock profiles hide the low-level details from users, making it easier to configure and use mClock.

The following input parameters are required for an mClock profile to configure the quality of service (QoS) related parameters:

- The total capacity of input and output operations per second (IOPS) for each Ceph OSD. This is determined automatically.
- The maximum sequential bandwidth capacity (MiB/s) of each OS. See `osd_mclock_max_sequential_bandwidth_[hdd/ssd]` option.
- An mClock profile type to be enabled. The default is `balanced`.

Using the settings in the specified profile, a Ceph OSD determines and applies the lower-level mClock and Ceph parameters. The parameters applied by the mClock profile make it possible to tune the QoS between the client I/O and background operations in the OSD.

Reference

- For automated OSD capacity determination, see “Ceph OSD capacity determination” on page 499 .

mClock clients

The mClock scheduler handles requests from different types of Ceph services. Each service is considered by mClock as a type of client. Depending on the type of requests handled, mClock clients are classified into different bucket types.

The different bucket classification types are as follows:

- Client - Handles input and output (I/O) requests issued by external clients of Ceph.
- Background recovery - Handles internal recovery requests.
- Background best-effort - Handles internal backfill, scrub, snap trim, and placement group (PG) deletion requests.

The mClock scheduler derives the cost of an operation used in the QoS calculations from **`osd_mclock_max_capacity_iops_hdd`** | **`osd_mclock_max_capacity_iops_ssd`**, **`osd_mclock_max_sequential_bandwidth_hdd`** | **`osd_mclock_max_sequential_bandwidth_ssd`**, and **`osd_op_num_shards_hdd`** | **`osd_op_num_shards_ssd`** parameters.

mClock profiles

An mClock profile is a configuration setting. When applied to a running Red Hat Ceph Storage cluster, it enables the throttling of the IOPS operations belonging to different client classes, such as `background recovery`, `scrub`, `snap trim`, `client op`, and `pg deletion`.

The mClock profile uses the capacity limits and the mClock profile type selected by the user to determine the low-level mClock resource control configuration parameters and applies them transparently. Other Red Hat Ceph Storage configuration parameters are also applied. The low-level mClock resource control parameters are

the reservation, limit, and weight that provide control of the resource shares. The mClock profiles allocate these parameters differently for each client type.

mClock profile types

mClock profiles can be classified into *built-in* and *custom* profiles.

If any mClock profile is active, the following Red Hat Ceph Storage configuration sleep options get disabled, which means they are set to `0`:

- `osd_recovery_sleep`
- `osd_recovery_sleep_hdd`
- `osd_recovery_sleep_ssd`
- `osd_recovery_sleep_hybrid`
- `osd_scrub_sleep`
- `osd_delete_sleep`
- `osd_delete_sleep_hdd`
- `osd_delete_sleep_ssd`
- `osd_delete_sleep_hybrid`
- `osd_snap_trim_sleep`
- `osd_snap_trim_sleep_hdd`
- `osd_snap_trim_sleep_ssd`
- `osd_snap_trim_sleep_hybrid`

It is to ensure that mClock scheduler is able to determine when to pick the next operation from its operation queue and transfer it to the operation sequencer. This results in the desired QoS being provided across all its clients.

Custom profile

This profile gives users complete control over all the mClock configuration parameters.

Built-in profiles

When a `built-in` profile is enabled, the mClock scheduler calculates the low-level mClock parameters, that is, reservation, weight, and limit, based on the profile enabled for each client type.

The mClock parameters are calculated based on the maximum Ceph OSD capacity provided beforehand. Therefore, the following mClock configuration options cannot be modified when using any of the built-in profiles:

- `osd_mclock_scheduler_client_res`
- `osd_mclock_scheduler_client_wgt`
- `osd_mclock_scheduler_client_lim`
- `osd_mclock_scheduler_background_recovery_res`
- `osd_mclock_scheduler_background_recovery_wgt`
- `osd_mclock_scheduler_background_recovery_lim`
- `osd_mclock_scheduler_background_best_effort_res`
- `osd_mclock_scheduler_background_best_effort_wgt`
- `osd_mclock_scheduler_background_best_effort_lim`

> **Note:** These defaults cannot be changed using any of the config subsystem commands like `config set`, `config daemon` or `config tell` commands. Although the command(s) report success, the mclock QoS parameters are reverted to their respective built-in profile defaults.

The following recovery and backfill related Ceph options are overridden to mClock defaults:

> **Warning:** Do not change these options as the built-in profiles are optimized based on them. Changing these defaults can result in unexpected performance outcomes.

- `osd_max_backfills`
- `osd_recovery_max_active`
- `osd_recovery_max_active_hdd`
- `osd_recovery_max_active_ssd`

The following options show the mClock defaults which is same as the current defaults to maximize the performance of the foreground client operations:

Table 77: mClock recovery and backfill default values

Configuration option	Original default	mClock default
`osd_max_backfills`	`1`	`1`
`osd_recovery_max_active`	`0`	`0`
`osd_recovery_max_active_hdd`	`3`	`3`
`osd_recovery_max_active_ssd`	`10`	`10`

> **Note:** The mClock defaults can be modified, only if necessary, by enabling `osd_mclock_override_recovery_settings` that is by default set as `false`. See Modifying backfill and recovery options to modify these parameters.

Users can choose from the following *built-in* profile types:

- `balanced` (default)
- `high_client_ops`
- `high_recovery_ops`

> **Note:** The values mentioned in the list represent the proportion of the total IOPS capacity of the Ceph OSD allocated for the service type.

`balanced`

The default mClock profile is set to `balanced` because it represents a compromise between prioritizing client IO or recovery IO. It allocates equal reservation or priority to client operations and background recovery operations. Background best-effort operations are given lower reservation and therefore take longer to complete when there are competing operations. This profile meets the normal or steady state requirements of the cluster which is the case when external client performance requirements is not critical and there are other background operations that still need attention within the OSD.

There might be instances that necessitate giving higher priority to either client operations or recovery operations. To meet such requirements you can choose either the `high_client_ops` profile to prioritize client IO or the `high_recovery_ops` profile to prioritize recovery IO. These profiles are discussed further.

Table 78: Resource control parameters for the balanced profile			
Service type	**Reservation**	**Weight**	**Limit**
client	50%	1	MAX
background recovery	50%	1	MAX
background best-effort	MIN	1	90%

`high_client_ops`

This profile optimizes client performance over background activities by allocating more reservation and limit to client operations as compared to background operations in the Ceph OSD. This profile, for example, can be enabled to provide the needed performance for I/O intensive applications for a sustained period of time at the cost of slower recoveries. The list shows the resource control parameters set by the profile:

Table 79: Resource control parameters for the high_client_ops profile			
Service type	**Reservation**	**Weight**	**Limit**
client	60%	2	MAX
background recovery	40%	1	MAX
background best-effort	MIN	1	70%

`high_recovery_ops`

This profile optimizes background recovery performance as compared to external clients and other background operations within the Ceph OSD.

For example, it could be temporarily enabled by an administrator to accelerate background recoveries during non-peak hours. The list shows the resource control parameters set by the profile:

Table 80: Resource control parameters for the high_recovery_ops profile			
Service type	**Reservation**	**Weight**	**Limit**
client	30%	1	MAX
background recovery	70%	2	MAX
background best-effort	MIN	1	MAX

Reference

- For more information about mClock configuration options, see the mClock configuration options.

Changing mClock profiles

The default mClock profile is set to `balanced`. The other types of the `built-in` profile are `high_client_ops` and `high_recovery_ops`.

> **Note:** The `custom` profile is not recommended unless you are an advanced user.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph Monitor host.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Set the `osd_mclock_profile` option:

   Syntax

   ```
   ceph config set osd._OSD_ID_ osd_mclock_profile VALUE
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph config set osd.0 osd_mclock_profile high_recovery_ops
   ```

   This example changes the profile to allow faster recoveries on `osd.0`.

   Note: For optimal performance the profile must be set on all Ceph OSDs by using the following command:
   Syntax

   ```
   ceph config set osd osd_mclock_profile VALUE
   ```

Switching between `built-in` and `custom` profiles

Understand how to switch between the `built-in` and `custom` profiles.

In some cases, you might want to switch to the `custom` profile if you want complete control over all the mClock configuration options. However, it is recommended not to use the `custom` profile unless you are an advanced user.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph Monitor host.

Procedure

Switch from *built-in* profile to *custom* profile

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Switch to the *custom* profile:

   Syntax

   ```
   ceph config set osd.OSD_ID osd_mclock_profile custom
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph config set osd.0 osd_mclock_profile custom
   ```

   Note: For optimal performance the profile must be set on all Ceph OSDs by using the following command:

   ```
   [ceph: root@host01 /]# ceph config set osd osd_mclock_profile custom
   ```

3. Optional: After switching to the *custom* profile, modify the desired mClock configuration options:

Syntax

```
ceph config set osd.OSD_ID MCLOCK_CONFIGURATION_OPTION VALUE
```

Example

```
[ceph: root@host01 /]# ceph config set osd.0 osd_mclock_scheduler_client_res 0.5
```

This example changes the client reservation IOPS ratio for a specific OSD `osd.0` to 0.5 (50%)

> **Important:** Change the reservations of other services, such as background recovery and background best-effort accordingly to ensure that the sum of the reservations does not exceed the maximum proportion (1.0) of the IOPS capacity of the OSD.

Switch from *custom* profile to *built-in* profile

1. Log into the cephadm shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Set the desired *built-in* profile:

   Syntax

   ```
   ceph config set osd osd_mclock_profile MCLOCK_PROFILE
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph config set osd osd_mclock_profile high_client_ops
   ```

   This example sets the *built-in* profile to `high_client_ops` on all Ceph OSDs.

3. Determine the existing custom mClock configuration settings in the database:

   Example

   ```
   [ceph: root@host01 /]# ceph config dump
   ```

4. Remove the custom mClock configuration settings determined earlier:

   Syntax

   ```
   ceph config rm osd MCLOCK_CONFIGURATION_OPTION
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph config rm osd osd_mclock_scheduler_client_res
   ```

   This example removes the configuration option `osd_mclock_scheduler_client_res` that was set on all Ceph OSDs.

   After all existing custom mClock configuration settings are removed from the central configuration database, the configuration settings related to `high_client_ops` are applied.

5. Verify the settings on Ceph OSDs:

   Syntax

   ```
   ceph config show osd.OSD_ID
   ```

   Example

```
[ceph: root@host01 /]# ceph config show osd.0
```

Reference

- For the list of the mClock configuration options that cannot be modified with _*built-in*_ profiles, see mClock profile types.

Switching temporarily between mClock profiles

Understand how to temporarily switch between mClock profiles.

Warning: This section is for advanced users or for experimental testing. Do not use these commands on a running storage cluster as it could have unexpected outcomes.

Note: The configuration changes on a Ceph OSD using these commands are temporary and are lost when the Ceph OSD is restarted.

Important: The configuration options that are overridden using the commands described in this section cannot be modified further using the `ceph config set osd.OSD_ID` command. The changes do not take effect until a given Ceph OSD is restarted. This is intentional, as per the configuration subsystem design. However, any further modifications can still be made temporarily using these commands.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph Monitor host.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Run the following command to override the mClock settings:

   Syntax

   ```
   ceph tell osd.OSD_ID injectargs '--MCLOCK_CONFIGURATION_OPTION=VALUE'
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph tell osd.0 injectargs '--
   osd_mclock_profile=high_recovery_ops'
   ```

   This example overrides the `osd_mclock_profile` option on `osd.0`.

3. Optional: You can use the alternative to the previous `ceph tell osd.OSD_ID injectargs` command:

   Syntax

   ```
   ceph daemon osd.OSD_ID config set MCLOCK_CONFIGURATION_OPTION VALUE
   ```

   Example

```
[ceph: root@host01 /]# ceph daemon osd.0 config set osd_mclock_profile
high_recovery_ops
```

Note: The individual QoS related configuration options for the `custom` profile can also be modified temporarily using these commands.

Degraded and misplaced object recovery rate with mClock profiles

Degraded object recovery is categorized into the background recovery bucket. Across all mClock profiles, degraded object recovery is given higher priority when compared to misplaced object recovery because degraded objects present a data safety issue not present with objects that are merely misplaced. Backfill or the misplaced object recovery operation is categorized into the background best-effort bucket.

According to the `balanced` and `high_client_ops` mClock profiles, background best-effort client is not constrained by reservation (set to zero) but is limited to use a fraction of the participating OSD's capacity if there are no other competing services.

Therefore, with the `balanced` or `high_client_ops` profile and with other background competing services active, backfilling rates are expected to be slower when compared to the previous *WeightedPriorityQueue* (WPQ) scheduler.

If higher backfill rates are desired, follow the steps mentioned in "Modifying backfills and recovery options" on page 498.

Improving backfilling rates

For faster backfilling rate when using either `balanced` or `high_client_ops` profile, run the following steps:

- Switch to the 'high_recovery_ops' mClock profile for the duration of the backfills. See changing an clock profile to achieve this. Once the backfilling phase is complete, switch the mClock profile to the previously active profile. In case there is no significant improvement in the backfilling rate with the `high_recovery_ops` profile, continue to the next step.
- Switch the mClock profile back to the previously active profile.
- Modify `osd_max_backfills` to a higher value, for example, `3`. See Modifying backfills and recovery options.
- Once the backfilling is complete, *osd_max_backfills* can be reset to the default value of 1 by following the same procedure mentioned in step 3.

Important: Modifying `osd_max_backfills` may result in other operations, for example, client operations may experience higher latency during the backfilling phase. Therefore, users are recommended to increase `osd_max_backfills` in small increments to minimize performance impact to other operations in the cluster.

Modifying `backfills` and `recovery` options

Modify the `backfills` and `recovery` options with the **`ceph config set`** command.

The backfill or recovery options that can be modified are listed in mClock profile types.

Warning: This section is for advanced users or for experimental testing. Do not use these commands on a running storage cluster as it could have unexpected outcomes.
Modify the values only for experimental testing, or if the cluster is unable to handle the values or it shows poor performance with the default settings.

Important: The modification of the mClock default backfill or recovery options is restricted by the `osd_mclock_override_recovery_settings` option, which is set to `false` by default.
If you attempt to modify any default backfill or recovery options without setting `osd_mclock_override_recovery_settings` to `true`, it resets the options back to the mClock defaults along with a warning message logged in the cluster log.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph Monitor host.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Set the `osd_mclock_override_recovery_settings` configuration option to `true` on all Ceph OSDs:

   Example

   ```
   [ceph: root@host01 /]# ceph config set osd osd_mclock_override_recovery_settings true
   ```

3. Set the desired `backfills` or `recovery` option:

   Syntax

   ```
   ceph config set osd OPTION VALUE
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph config set osd osd_max_backfills 5
   ```

4. Wait a few seconds and verify the configuration for the specific OSD:

   Syntax

   ```
   ceph config show osd.OSD_ID | grep OPTION
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph config show osd.0 | grep osd_max_backfills
   ```

5. Reset the `osd_mclock_override_recovery_settings` configuration option to `false` on all OSDs:

   Example

   ```
   [ceph: root@host01 /]# ceph config set osd osd_mclock_override_recovery_settings false
   ```

Ceph OSD capacity determination

The Ceph OSD capacity in terms of total IOPS is determined automatically during the Ceph OSD initialization. This is achieved by running the Ceph OSD bench tool and overriding the default value of `osd_mclock_max_capacity_iops_[hdd, ssd]` option depending on the device type. No other action or input is expected from the user to set the Ceph OSD capacity.

Mitigation of unrealistic Ceph OSD capacity from the automated procedure

In certain conditions, the Ceph OSD bench tool might show unrealistic or inflated results depending on the drive configuration and other environment related conditions.

To mitigate the performance impact due to this unrealistic capacity, a couple of threshold configuration options depending on the OSD device type are defined and used:

- `osd_mclock_iops_capacity_threshold_hdd` = 500
- `osd_mclock_iops_capacity_threshold_ssd` = 80000

You can verify these parameters by running the following commands:

```
[ceph: root@host01 /]# ceph config show osd.0 osd_mclock_iops_capacity_threshold_hdd
500.000000
[ceph: root@host01 /]# ceph config show osd.0 osd_mclock_iops_capacity_threshold_ssd
80000.000000
```

> **Note:** If you want to manually benchmark OSD(s) or manually tune the BlueStore throttle parameters, see "Manually benchmarking OSDs" on page 501.

You can verify the capacity of an OSD after the cluster is up by running the following command:

Syntax

```
ceph config show osd.N osd_mclock_max_capacity_iops_[hdd, ssd]
```

Example

```
[ceph: root@host01 /]# ceph config show osd.0 osd_mclock_max_capacity_iops_ssd
```

In the example, you can view the maximum capacity for `osd.0` on a Red Hat Ceph Storage node whose underlying device is an SSD.

The following automated step is performed:

Fallback to using default OSD capacity

If the Ceph OSD bench tool reports a measurement that exceeds the threshold values, the fallback mechanism reverts to the default value of `osd_mclock_max_capacity_iops_hdd` or `osd_mclock_max_capacity_iops_ssd`. The threshold configuration options can be reconfigured based on the type of drive used.

A cluster warning is logged in case the measurement exceeds the threshold:

Example

```
3403 Sep 11 11:52:50 dell-r640-039.dsal.lab.eng.rdu2.redhat.com ceph-osd[70342]:
log_channel(cluster) log [WRN] : OSD bench result of 49691.213005 IOPS exceeded the
threshold limit of 500.000000 IOPS for osd.27. IOPS capacity is unchanged at 315.000000
IOPS. The recommendation is to establish the osd's IOPS capacity using other benchmark
tools (e.g. Fio) and then override osd_mclock_max_capacity_iops_[hdd|ssd].
```

> **Important:** If the default capacity does not accurately represent the Ceph OSD capacity, it is highly recommended to run a custom benchmark using the preferred tool, for example Fio, on the drive and then override the `osd_mclock_max_capacity_iops_[hdd, ssd]` option as described in Specifying maximum OSD capacity

Reference

- To manually benchmark Ceph OSDs or manually tune the BlueStore throttle parameters, see Manually benchmarking OSDs.
- For more information about the `osd_mclock_max_capacity_iops_[hdd, ssd]` and `osd_mclock_iops_capacity_threshold_[hdd, ssd]` options, see the mClock configuration options.

Verifying the capacity of an OSD

You can verify the capacity of a Ceph OSD after setting up the storage cluster.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph Monitor host.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Verify the capacity of a Ceph OSD:

   Syntax

   ```
   ceph config show osd.OSD_ID osd_mclock_max_capacity_iops_[hdd, ssd]
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph config show osd.0 osd_mclock_max_capacity_iops_ssd
   21500.000000
   ```

Manually benchmarking OSDs

To manually benchmark a Ceph OSD, any existing benchmarking tool, for example Fio, can be used. Regardless of the tool or command used, steps documented here remain the same.

Important: The number of shards and BlueStore throttle parameters have an impact on the mClock operation queues. Therefore, it is critical to set these values carefully in order to maximize the impact of the mclock scheduler. For more information on these values, see factors that impact mClock operation queues.

Note: The steps in this section are only necessary if you want to override the Ceph OSD capacity determined automatically during the OSD initialization.
If you have already determined the benchmark data and wish to manually override the maximum OSD capacity for a Ceph OSD, see Specifying maximum OSD capacity.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph Monitor host.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Benchmark a Ceph OSD:

   Syntax

   ```
   ceph tell osd.OSD_ID bench [TOTAL_BYTES] [BYTES_PER_WRITE] [OBJ_SIZE] [NUM_OBJS]
   ```

where:

- *TOTAL_BYTES*: Total number of bytes to write.
- *BYTES_PER_WRITE*: Block size per write.
- *OBJ_SIZE*: Bytes per object.
- *NUM_OBJS*: Number of objects to write.

Example

```
[ceph: root@host01 /]# ceph tell osd.0 bench 12288000 4096 4194304 100
{
    "bytes_written": 12288000,
    "blocksize": 4096,
    "elapsed_sec": 1.3718913019999999,
    "bytes_per_sec": 8956977.8466311768,
    "iops": 2186.7621695876896
}
```

Determining BlueStore throttle values

Determining BlueStore throttle values is optional. It is used to determine the correct BlueStore throttle values. The steps use the default shards.

Important: Before running the test, clear the caches to get an accurate measurement. Clear the OSD caches between each benchmark run using the following command:
Syntax

```
ceph tell osd.OSD_ID cache drop
```

Example

```
[ceph: root@host01 /]# ceph tell osd.0 cache drop
```

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph Monitor node hosting the OSDs that you wish to benchmark.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Run a simple 4KiB random write workload on an OSD:

   Syntax

   ```
   ceph tell osd.OSD_ID bench 12288000 4096 4194304 100
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph tell osd.0 bench 12288000 4096 4194304 100
   {
       "bytes_written": 12288000,
       "blocksize": 4096,
       "elapsed_sec": 1.3718913019999999,
       "bytes_per_sec": 8956977.8466311768,
   ```

```
    "iops": 2186.7621695876896 1
}
```

   a. The overall throughput obtained from the output of the `osd bench` command. This value is the baseline throughput, when the default BlueStore throttle options are in effect.

3. Note the overall throughput, that is IOPS, obtained from the output of the previous command.
4. If the intent is to determine the BlueStore throttle values for your environment, set `bluestore_throttle_bytes` and `bluestore_throttle_deferred_bytes` options to 32 KiB, that is, 32768 Bytes:

   Syntax

   ```
   ceph config set osd.OSD_ID bluestore_throttle_bytes 32768
   ceph config set osd.OSD_ID bluestore_throttle_deferred_bytes 32768
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph config set osd.0 bluestore_throttle_bytes 32768
   [ceph: root@host01 /]# ceph config set osd.0 bluestore_throttle_deferred_bytes 32768
   ```

   Otherwise, you can skip to the next section, specifying maximum OSD capacity.

5. Run the 4KiB random write test as before using an OSD bench command:

   Example

   ```
   [ceph: root@host01 /]# ceph tell osd.0 bench 12288000 4096 4194304 100
   ```

6. Notice the overall throughput from the output and compare the value against the baseline throughput recorded earlier.
7. If the throughput does not match with the baseline, increase the BlueStore throttle options by multiplying by 2.
8. Repeat the steps by running the 4KiB random write test, comparing the value against the baseline throughput, and increasing the BlueStore throttle options by multiplying by 2, until the obtained throughput is very close to the baseline value.

Note: For example, during benchmarking on a machine with NVMe SSDs, a value of 256 KiB for both BlueStore throttle and deferred bytes was determined to maximize the impact of mClock. For HDDs, the corresponding value was 40 MiB, where the overall throughput was roughly equal to the baseline throughput.
In general for HDDs, the BlueStore throttle values are expected to be higher when compared to SSDs.

Specifying maximum OSD capacity

You can override the maximum Ceph OSD capacity automatically set during OSD initialization.

These steps are optional. Perform the following steps if the default capacity does not accurately represent the Ceph OSD capacity.

Note: Ensure that you determine the benchmark data first, as described in manually benchmarking OSDs.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph Monitor host.

Procedure

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Set `osd_mclock_max_capacity_iops_[hdd, ssd]` option for an OSD:

   Syntax

   ```
   ceph config set osd.OSD_ID osd_mclock_max_capacity_iops_[hdd,ssd] VALUE
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph config set osd.0 osd_mclock_max_capacity_iops_hdd 350
   ```

   This example sets the maximum capacity for `osd.0`, where an underlying device type is HDD, to 350 IOPS.

mClock configuration options

Learn the different mClock configuration options.

Following are the list of mClock configuration options:

`osd_mclock_profile`

Description

It sets the type of mClock profile to provide the quality of service (QoS) based on operations that belong to different classes, such as background recovery, `backfill`, `pg scrub`, `snap trim`, `client op`, and `pg deletion`.

On enabling a `built-in` profile, the lower-level mClock resource control parameters, that is reservation, weight, and limit, and some Ceph configuration parameters are set transparently. This does not apply for the `custom` profile.

Type

String

Default

`balanced`

Valid choices

`balanced`, `high_recovery_ops`, `high_client_ops`, `custom`

`osd_mclock_max_capacity_iops_hdd`

Description

It sets a maximum random write IOPS capacity, at 4 KiB block size, to consider per OSD for rotational media. Contributes in QoS calculations when enabling a dmclock profile. It is only considered for `osd_op_queue = mclock_scheduler`

Type

Float

Default

`315.0`

`osd_mclock_max_capacity_iops_ssd`

Description

It sets a maximum random write IOPS capacity, at 4 KiB block size, to consider per OSD for solid-state media.

Type

Float

Default

`21500.0`

`osd_mclock_cost_per_byte_usec_ssd`

Description

Indicates cost per byte in microseconds to consider per OSD for SDDs. Contributes in QoS calculations when enabling a dmclock profile. It is only considered for `osd_op_queue = mclock_scheduler`

Type

Float

Default

`0.011`

`osd_mclock_max_sequential_bandwidth_hdd`

Description

Indicates the maximum sequential bandwidth in bytes to consider for an OSD whose underlying device type is rotational media. This option is considered by the mclock scheduler to derive the cost factor to be used in QoS calculations. Only considered for `osd_op_queue = mclock_scheduler`

Type

Size

Default

`150_M`

`osd_mclock_max_sequential_bandwidth_ssd`

Description

Indicates the maximum sequential bandwidth in bytes to consider for an OSD whose underlying device type is solid-state media. This is considered by the mclock scheduler to derive the cost factor to be used in QoS calculations. Only considered for `osd_op_queue = mclock_scheduler`

Type

Size

Default

`1200_M`

`osd_mclock_force_run_benchmark_on_init`

Description

This force-runs the OSD benchmark on OSD initialization or boot-up.

Type

Boolean

Default

False

See also

`osd_mclock_max_capacity_iops_hdd`, `osd_mclock_max_capacity_iops_ssd`

`osd_mclock_skip_benchmark`

Description

Setting this option skips the OSD benchmark on OSD initialization or boot-up.

Type

Boolean

Default

False

See also

`osd_mclock_max_capacity_iops_hdd`, `osd_mclock_max_capacity_iops_ssd`

`osd_mclock_override_recovery_settings`

Description

Setting this option enables the override of the recovery or backfill limits for the mClock scheduler as defined by the `osd_recovery_max_active_hdd`, `osd_recovery_max_active_ssd`, and `osd_max_backfills` options.

Type

Boolean

Default

False

See also

`osd_recovery_max_active_hdd`, `osd_recovery_max_active_ssd`, `osd_max_backfills`

`osd_mclock_iops_capacity_threshold_hdd`

Description

It indicates the threshold IOPS capacity, at 4 KiB block size, beyond which to ignore the Ceph OSD bench results for an OSD for HDDs.

Type

Float

Default

`500.0`

`osd_mclock_iops_capacity_threshold_ssd`

Description

It indicates the threshold IOPS capacity, at 4 KiB block size, beyond which to ignore the Ceph OSD bench results for an OSD for SSDs.

Type

Float

Default

`80000.0`

`osd_mclock_scheduler_client_res`

Description

It is the default I/O proportion that is reserved for each client. The default value of `0` specifies the lowest possible reservation. Any value greater than 0 and up to 1.0 specifies the minimum IO proportion to reserve for each client in terms of a fraction of the OSD's maximum IOPS capacity.

Type

Float

Default

`0`

min

0

max

1.0

`osd_mclock_scheduler_client_wgt`

Description

It is the default I/O share for each client over reservation.

Type

Unsigned integer

Default

`1`

`osd_mclock_scheduler_client_lim`

Description

It is the default I/O limit for each client over reservation. The default value of `0` specifies no limit enforcement, which means each client can use the maximum possible IOPS capacity of the OSD. Any value greater than 0 and up to 1.0 specifies the upper IO limit over reservation that each client receives in terms of a fraction of the OSD's maximum IOPS capacity.

Type

Float

Default

`0`

min

0

max

1.0

`osd_mclock_scheduler_background_recovery_res`

Description

It is the default I/O proportion reserved for background recovery. The default value of 0 specifies the lowest possible reservation. Any value greater than 0 and up to 1.0 specifies the minimum IO proportion to reserve for background recovery operations which is in terms of a fraction of the OSD's maximum IOPS capacity.

Type

Float

Default

`0`

min

0

max

1.0

`osd_mclock_scheduler_background_recovery_wgt`

Description

It indicates the I/O share for each background recovery over reservation.

Type

Unsigned integer

Default

`1`

`osd_mclock_scheduler_background_recovery_lim`

Description

It indicates the I/O limit for background recovery over reservation. The default value of 0 specifies no limit enforcement, which means background recovery operation can use the maximum possible IOPS capacity of the OSD. Any value greater than 0 and up to 1.0 specifies the upper IO limit over reservation that background recovery operation receives in terms of a fraction of the OSD's maximum IOPS capacity.

Type

Float

Default

`0`

min

0

max

1.0

`osd_mclock_scheduler_background_best_effort_res`

Description

It indicates the default I/O proportion that is reserved for background `best_effort`. The default value of 0 specifies the lowest possible reservation. Any value greater than 0 and up to 1.0 specifies the minimum IO proportion to reserve for background best_effort operations in terms of a fraction of the OSD's maximum IOPS capacity.

Type

Float

Default

`0`

min

0

max

1.0

`osd_mclock_scheduler_background_best_effort_wgt`

Description

It indicates the I/O share for each background `best_effort` over reservation.

Type

Unsigned integer

Default

`1`

`osd_mclock_scheduler_background_best_effort_lim`

Description

It indicates the I/O limit for background `best_effort` over reservation. The default value of 0 specifies no limit enforcement, which means background best_effort operation can use the maximum IOPS capacity of the OSD. Any value greater than 0 and up to 1.0 specifies the upper IO limit over reservation that background best_effort operation receives in terms of a fraction of the OSD's maximum IOPS capacity.

Type

float

Default

`0`

min

0

max

1.0

Reference

- For more details about this `osd_op_queue` option, see Object Storage Daemon (OSD) configuration options .

BlueStore

BlueStore is the back-end object store for the OSD daemons and puts objects directly on the block device.

> **Important:** BlueStore provides a high-performance backend for OSD daemons in a production environment. By default, BlueStore is configured to be self-tuning. If you determine that your environment performs better with BlueStore tuned manually, contact Red Hat Support and share the details of your configuration to help Red Hat improve the auto-tuning capability. Red Hat looks forward to your feedback and appreciates your recommendations.

BlueStore features

Learn about the main BlueStore features.

The following are some of the main features of using BlueStore:

Direct management of storage devices

BlueStore consumes raw block devices or partitions. This avoids any intervening layers of abstraction, such as local file systems like XFS, that might limit performance or add complexity.

Metadata management with RocksDB

BlueStore uses the RocksDB key-value database to manage internal metadata, such as the mapping from object names to block locations on a disk.

Full data and metadata checksumming

By default all data and metadata written to BlueStore is protected by one or more checksums. No data or metadata are read from disk or returned to the user without verification.

Efficient copy-on-write

The Ceph Block Device and Ceph File System snapshots rely on a copy-on-write clone mechanism that is implemented efficiently in BlueStore. This results in efficient I/O both for regular snapshots and for erasure coded pools which rely on cloning to implement efficient two-phase commits.

No large double-writes

BlueStore first writes any new data to unallocated space on a block device, and then commits a RocksDB transaction that updates the object metadata to reference the new region of the disk. Only when the write operation is under a configurable size threshold, it falls back to a write-ahead journaling scheme.

Multi-device support

BlueStore can use multiple block devices for storing different data. For example: Hard Disk Drive (HDD) for the data, Solid-state Drive (SSD) for metadata, Non-volatile Memory (NVM) or Non-volatile random-access memory (NVRAM) or persistent memory for the RocksDB write-ahead log (WAL). For more information, see "BlueStore devices" on page 510.

Efficient block device usage

Because BlueStore does not use any file system, it minimizes the need to clear the storage device cache.

BlueStore devices

BlueStore manages either one, two, or three storage devices in the backend. The backend devices include primary, write-ahead-log (WAL), and database (DB).

In the simplest case, BlueStore consumes a single primary storage device. The storage device is partitioned into two parts that contain:

OSD metadata

A small partition that contains basic metadata for the OSD. This data directory includes information about the OSD, such as its identifier, which cluster it belongs to, and its private keyring.

Data

A large partition occupying the rest of the device that is managed directly by BlueStore and that contains all of the OSD data. This primary device is identified by a block symbolic link in the data directory.

You can also use two additional devices:

A WAL (write-ahead-log) device

A device that stores BlueStore internal journal or write-ahead log. It is identified by the `block.wal` symbolic link in the data directory. Consider using a WAL device only if the device is faster than the primary device. For example, when the WAL device uses an SSD disk and the primary device uses an HDD disk.

A DB device

A device that stores BlueStore internal metadata. The embedded RocksDB database puts as much metadata as it can on the DB device instead of on the primary device to improve performance. If the DB device is full, it starts adding metadata to the primary device. Consider using a DB device only if the device is faster than the primary device.

Warning: If you have only less than a gigabyte storage available on fast devices, Red Hat recommends using it as a WAL device. If you have more fast devices available, consider using it as a DB device. The BlueStore journal is always placed on the fastest device, so using a DB device provides the same benefit that the WAL device provides while also allowing for storing additional metadata.

BlueStore caching

The BlueStore cache is a collection of buffers that, depending on configuration, can be populated with data as the OSD daemon does reading from or writing to the disk.

By default in Red Hat Ceph Storage, BlueStore will cache on reads, but not writes. This is because the `bluestore_default_buffered_write` option is set to `false` to avoid potential overhead associated with cache eviction.

If the `bluestore_default_buffered_write` option is set to `true`, data is written to the buffer first, and then committed to disk. Afterward, a write acknowledgment is sent to the client, allowing subsequent reads faster access to the data already in cache, until that data is evicted.

Read-heavy workloads will not see an immediate benefit from BlueStore caching. As more reading is done, the cache will grow over time and subsequent reads will see an improvement in performance. How fast the cache populates depends on the BlueStore block and database disk type, and the client's workload requirements.

Important: Before enabling the `bluestore_default_buffered_write` option, contact Red Hat Support.

Sizing considerations

When mixing traditional and solid state drives using BlueStore OSDs, it is important to size the RocksDB logical volume (`block.db`) appropriately.

The RocksDB logical volume be no less than 4% of the block size with object, file and mixed workloads. Red Hat Ceph Storage supports 1% of the BlueStore block size with RocksDB and OpenStack block workloads. For example, if the block size is 1 TB for an object workload, then at a minimum, create a 40 GB RocksDB logical volume.

When not mixing drive types, there is no requirement to have a separate RocksDB logical volume. BlueStore will automatically manage the sizing of RocksDB.

BlueStore's cache memory is used for the key-value pair metadata for RocksDB, BlueStore metadata, and object data.

> **Note:** The BlueStore cache memory values are in addition to the memory footprint already being consumed by the OSD.

Tuning BlueStore

Use this procedure for freshly deployed OSDs.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Ceph Monitors and Managers are deployed in the cluster.
- Servers that can be freshly provisioned as OSD nodes.
- The admin keyring.
- Check the value used to deploy the OSD by running the following command:

  ```
  ceph osd metadata OSD_ID | grep bluestore_min_alloc_size
  ```

  For example,

  ```
  [ceph: root@host01 /]# ceph osd metadata 11 | grep bluestore_min_alloc_size
  "bluestore_min_alloc_size": "4096",
  ```

About this task

In BlueStore, the raw underlying storage is allocated in chunks of **`bluestore_min_alloc_size`**. By default, **`bluestore_min_alloc_size`** is `4096`, equivalent to 4 KiB for HDDs and SSDs. Starting with Red Hat Ceph Storage 9, erasure-coded pools no longer add zero padding to unwritten areas in chunks. To enable erasure coding optimization, also known as FastEC, see "Enabling coding optimizations" on page 177.

> **Important:** Changing the value of **`bluestore_min_alloc_size`** is not recommended. For any assistance, contact Red Hat Support.

> **Note:**
>
> - You can change specifically the SSD and HDD options with the **`bluestore_min_alloc_size`** parameter within the **`config set`** command. For more information, contact Red Hat Support.
> - For already deployed OSDs, you cannot modify the **`bluestore_min_alloc_size`** parameter. To modify, remove and then redeploy each affected OSD.

For OSD removal and additional information, see Managing OSDs.

Procedure

1. From the `cephadm` shell, change the **`bluestore_min_alloc_size`** parameter value.
   Set the allocation size, in one of the following ways.

   > **Note:** The selected **`bluestore_min_alloc_size`** values should be power of 2 aligned.

   - Set the allocation size per OSD.

     ```
     ceph config set osd.OSD_ID bluestore_min_alloc_size VALUE
     ```

     In the following example, **`bluestore_min_alloc_size`** is set to 8192 bytes, which is equivalent to 8 KiB.

     ```
     [ceph: root@host01 /]# ceph config set osd.4 bluestore_min_alloc_size 8192
     ```

   - Set the allocation size per CRUSH device class.

     ```
     ceph config set class:CLASS bluestore_min_alloc_size VALUE
     ```

     Replace *CLASS* with the CRUSH device class, usually`hdd` or `ssd`.
     In the following example, the CRUSH device class is `hdd` and the **`bluestore_min_alloc_size`** is set to 8192 bytes, which is equivalent to 8 KiB.

     ```
     [ceph: root@host01 /]# ceph config set class:hdd bluestore_min_alloc_size
     8192
     ```

   ```
   ceph config set osd.OSD_ID bluestore_min_alloc_size VALUE
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph config set osd.4 bluestore_min_alloc_size 8192
   ```

2. Redeploy the affected OSDs.
   For more information, see Managing OSDs.

What to do next

Verify the BlueStore tuning settings, by using the **`ceph osd metadata`** command.

For example,

```
[ceph: root@host01 /]# ceph osd metadata OSD.1 | grep bluestore_min_alloc_size
"bluestore_min_alloc_size": "8192",
```

Resharding the RocksDB database

Use the BlueStore administration tool to reshard the RocksDB database. Resharding splits the BlueStore block.db into several column families without redeploying the OSDs, which can be beneficial for write performance.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- The object store configured as BlueStore.
- OSD nodes deployed on the hosts.
- Root level access to the all the hosts.
- The `ceph-common` and `cephadm` packages installed on all the hosts.

About this task

Reshard the RocksDB database to split the BlueStore block.db into several columns. Column families have the same features as the whole database, but allows users to operate on smaller data sets and apply different options. It leverages the different expected lifetime of keys stored. The keys are moved during the transformation without creating new keys or deleting existing keys.

Reshard the OSD in one of the following ways:

- "Use the rocksdb-resharding.yml playbook" on page 513.
- "Manually resharding the OSDs" on page 515.

For more information, see Initial installation.

Use the `rocksdb-resharding.yml` playbook

Reshard the database by using the `rocksdb-resharding.yml` playbook.

Procedure

1. As a root user, on the administration node, navigate to the `cephadm` folder where the playbook is installed.

   ```
   cd /usr/share/cephadm-ansible
   ```

2. Run the playbook.

   ```
   ansible-playbook -i hosts rocksdb-resharding.yml -e osd_id=OSD_ID -e
   admin_node=HOST_NAME
   ```

   For example,

```
[root@host01 ~]# ansible-playbook -i hosts rocksdb-resharding.yml -e osd_id=7 -e
admin_node=host03

.............
TASK [stop the osd]
*********************************************************************************
*********************************************************************************
******************
Wednesday 29 November 2023  11:25:18 +0000 (0:00:00.037)       0:00:03.864 ****
changed: [localhost -> host03]
TASK [set_fact ceph_cmd]
*********************************************************************************
*********************************************************************************
**************
Wednesday 29 November 2023  11:25:32 +0000 (0:00:14.128)       0:00:17.992 ****
ok: [localhost -> host03]

TASK [check fs consistency with fsck before resharding]
*********************************************************************************
*************************************************************
Wednesday 29 November 2023  11:25:32 +0000 (0:00:00.041)       0:00:18.034 ****
ok: [localhost -> host03]

TASK [show current sharding]
*********************************************************************************
*********************************************************************************
**********
Wednesday 29 November 2023  11:25:43 +0000 (0:00:11.053)       0:00:29.088 ****
ok: [localhost -> host03]

TASK [reshard]
*********************************************************************************
*********************************************************************************
************************
Wednesday 29 November 2023  11:25:45 +0000 (0:00:01.446)       0:00:30.534 ****
ok: [localhost -> host03]

TASK [check fs consistency with fsck after resharding]
*********************************************************************************
*************************************************************
Wednesday 29 November 2023  11:25:46 +0000 (0:00:01.479)       0:00:32.014 ****
ok: [localhost -> host03]

TASK [restart the osd]
*********************************************************************************
*********************************************************************************
****************
Wednesday 29 November 2023  11:25:57 +0000 (0:00:10.699)       0:00:42.714 ****
changed: [localhost -> host03]
```

What to do next

Verify that the sharding is complete.

1. Stop the OSD that is resharded.

```
[ceph: root@host01 /]# ceph orch daemon stop osd.7
```

2. Enter the OSD container.

```
[root@host03 ~]# cephadm shell --name osd.7
```

3. Check for resharding.

```
[ceph: root@host03 /]# ceph-bluestore-tool --path /var/lib/ceph/osd/ceph-9/ show-
sharding
    m(3) p(3,0-12) O(3,0-13) L P
```

4. Start the OSD.

```
[ceph: root@host01 /]# ceph orch daemon start osd.7
```

Manually resharding the OSDs

Manually reshard the OSDs with the `cephadm` shell.

Procedure

1. Log into the `cephadm` shell.
   For example,

   ```
   [root@host01 ~]# cephadm shell
   [ceph: root@host01 /]#
   ```

2. Fetch the *OSD_ID* and the host details from the administration node.

   ```
   ceph orch ps
   ```

3. Log into the respective host as a root user and stop the OSD.

   ```
   cephadm unit --name OSD_ID stop
   ```

   For example,

   ```
   [root@host02 ~]# cephadm unit --name osd.0 stop
   ```

4. Enter into the stopped OSD daemon container.

   ```
   cephadm shell --name OSD_ID
   ```

   For example,

   ```
   root@host02 ~]# cephadm shell --name osd.0
   ```

5. Log into the `cephadm shell` and check the file system consistency.

   ```
   ceph-bluestore-tool --path/var/lib/ceph/osd/ceph-OSD_ID/ fsck
   ```

   For example,

   ```
   [ceph: root@host02 /]# ceph-bluestore-tool --path /var/lib/ceph/osd/ceph-0/ fsck

   fsck success
   ```

6. Check the sharding status of the node.

   ```
   ceph-bluestore-tool --path /var/lib/ceph/osd/ceph-OSD_ID/ show-sharding
   ```

   For example,

   ```
   [ceph: root@host02 /]# ceph-bluestore-tool --path /var/lib/ceph/osd/ceph-9/ show-
   sharding

   m(3) p(3,0-12) O(3,0-13) L P
   ```

7. Reshard by running the **`ceph-bluestore-tool`** command.
   Use the command, along with the following given parameters.

   ```
   ceph-bluestore-tool --log-level 10 -l log.txt --path /var/lib/ceph/osd/ceph-OSD_ID/ --
   sharding="m(3) p(3,0-12) O(3,0-13)=block_cache={type=binned_lru} L P" reshard
   ```

   For example,

   ```
   [ceph: root@host02 /]# ceph-bluestore-tool --path /var/lib/ceph/osd/ceph-9/ --
   sharding="m(3) p(3,0-12) O(3,0-13)=block_cache={type=binned_lru} L P" reshard

   reshard success
   ```

8. Use the **`show-sharding`** command to check the sharding status of the OSD node.

```
ceph-bluestore-tool --path /var/lib/ceph/osd/ceph-OSD_ID/ show-sharding
```

For example,

```
[ceph: root@host02 /]# ceph-bluestore-tool --path /var/lib/ceph/osd/ceph-9/ show-
sharding

m(3) p(3,0-12) O(3,0-13)=block_cache={type=binned_lru} L P
```

9. Exit from the `cephadm` shell.
   For example,

```
[ceph: root@host02 /]# exit
[root@host02 ~]#
```

10. Log into the respective host as a root user and start the OSD.

```
cephadm unit --name OSD_ID start
```

For example,

```
[root@host02 ~]# cephadm unit --name osd.0 start
```

BlueStore fragmentation tool

As a storage administrator, you will want to periodically check the fragmentation level of your BlueStore OSDs. You can check fragmentation levels with one simple command for offline or online OSDs.

For BlueStore OSDs, the free space gets fragmented over time on the underlying storage device. Some fragmentation is normal, but when there is excessive fragmentation this causes poor performance.

The BlueStore fragmentation tool generates a score on the fragmentation level of the BlueStore OSD. This fragmentation score is given as a range, 0 through 1. A score of 0 means no fragmentation, and a score of 1 means severe fragmentation.

Table 81: Fragmentation scores' meaning

Score	Fragmentation Amount
0.0 - 0.4	None to tiny fragmentation.
0.4 - 0.7	Small and acceptable fragmentation.
0.7 - 0.9	Considerable, but safe fragmentation.
0.9 - 1.0	Severe fragmentation and that causes performance issues.

Important: For help resolving severe fragmentation, contact Red Hat Support.

Checking for fragmentation

Check the fragmentation level of BlueStore OSDs while the BlueStore OSD process is either online or offline.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- BlueStore OSDs.

Getting an online BlueStore fragmentation score

Procedure

Inspect a running BlueStore OSD process by getting either a simple or more detailed report.

- Get a simple report.

```
ceph daemon OSD_ID bluestore allocator score block
```

For example,

```
[ceph: root@host01 /]# ceph daemon osd.123 bluestore allocator score block
```

- Get a detailed report.

```
ceph daemon OSD_ID bluestore allocator dump block
```

For example,

```
[ceph: root@host01 /]# ceph daemon osd.123 bluestore allocator dump block
```

Getting an offline BlueStore fragmentation score

Procedure

1. Reshard to check the offline BlueStore OSD.
   For more information about resharding, see “Resharding the RocksDB database” on page 512.

```
[root@host01 ~]# cephadm shell --name osd.ID
```

For example,

```
[root@host01 ~]# cephadm shell --name osd.2
Inferring fsid 110bad0a-bc57-11ee-8138-fa163eb9ffc2
Inferring config /var/lib/ceph/110bad0a-bc57-11ee-8138-fa163eb9ffc2/osd.2/config
Using ceph image with id `17334f841482` and tag `ceph-9-rhel-9-containers-
candidate-59483-20240301201929` created on 2026-03-01 20:22:41 +0000 UTC
registry-proxy.engineering.redhat.com/rh-osbs/
rhceph@sha256:09fc3e5baf198614d70669a106eb87dbebee16d4e91484375778d4adbccadacd
```

2. Inspect a non-running BlueStore OSD process.
   Inspect a running BlueStore OSD process by getting either a simple or more detailed report.
   - Get a simple report.

```
ceph-bluestore-tool --path PATH_TO_OSD_DATA_DIRECTORY --allocator block free-
score
```

For example,

```
[root@host01 /]# ceph-bluestore-tool --path /var/lib/ceph/osd/ceph-123 --
allocator block free-score
```

   - Get a detailed report.

```
ceph-bluestore-tool --path PATH_TO_OSD_DATA_DIRECTORY --allocator block free-dump
block:
{
    "fragmentation_rating": 0.018290238194701977
}
```

For example,

```
[root@host01 /]# ceph-bluestore-tool --path /var/lib/ceph/osd/ceph-123 --
allocator block free-dump
block:
{
    "capacity": 21470642176,
    "alloc_unit": 4096,
    "alloc_type": "hybrid",
    "alloc_name": "block",
    "extents": [
        {
            "offset": "0x370000",
            "length": "0x20000"
        },
        {
            "offset": "0x3a0000",
            "length": "0x10000"
        },
        {
            "offset": "0x3f0000",
            "length": "0x20000"
        },
        {
            "offset": "0x460000",
            "length": "0x10000"
        },
```

Ceph BlueStore BlueFS

BlueStore block database stores metadata as key-value pairs in a RocksDB database. The block database resides on a small BlueFS partition on the storage device. BlueFS is a minimal file system that is designed to hold the RocksDB files.

BlueFS files

There are three types of files that RocksDB produces.

- Control files, for example `CURRENT`, `IDENTITY`, and `MANIFEST-00011`.
- Database (DB) table files, for example `004112.sst`.
- Write ahead logs (WAL), for example `00038.log`.

There is also an internal, hidden file that serves as BlueFS replay log, `ino 1`, that works as directory structure, file mapping, and operations log.

Fallback hierarchy

With BlueFS it is possible to put any file on any device. Parts of file can even reside on different devices, that is WAL, DB, and SLOW. There is an order to where BlueFS puts files. File is put to secondary storage only when primary storage is exhausted, and tertiary only when secondary is exhausted.

The order for the specific files is as follows, for each device type.

Write ahead logs

WAL, DB, SLOW

Replay log `ino 1`

DB, SLOW

Control and DB files

DB, SLOW
Control and DB file order when running out of space: SLOW

> **Important:** There is an exception to control and DB file order. When RocksDB detects that you are running out of space on DB file, it directly notifies you to put file to SLOW device.

Viewing the bluefs_buffered_io setting

As a storage administrator, you can view the current setting for the **bluefs_buffered_io** parameter.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph Monitor node.
- Log into the `cephadm` shell, using the **cephadm shell** command.

About this task

The option **bluefs_buffered_io** is set to `True` by default for Red Hat Ceph Storage. This option enable BlueFS to perform buffered reads in some cases, and enables the kernel page cache to act as a secondary cache for reads like RocksDB block reads.

> **Important:** Changing the value of **bluefs_buffered_io** is not recommended. Before changing the **bluefs_buffered_io** parameter, contact your Red Hat Support account team.

Procedure

View the current value of the **bluefs_buffered_io** parameter, using one of the following procedures.

- View the value stored in the configuration database.

```
ceph config get osd bluefs_buffered_io
```

For example,

```
[ceph: root@host01 /]# ceph config get osd bluefs_buffered_io
```

- View the value stored in the configuration database for a specific OSD.

```
ceph config get OSD_ID bluefs_buffered_io
```

For example,

```
[ceph: root@host01 /]# ceph config get osd.2 bluefs_buffered_io
```

- View the running value for an OSD where the running value is different from the value stored in the configuration database.

```
ceph config show OSD_ID bluefs_buffered_io
```

For example,

```
[ceph: root@host01 /]# ceph config show osd.3 bluefs_buffered_io
```

Viewing Ceph BlueFS statistics for Ceph OSDs

View the BluesFS related information about collocated and non-collocated Ceph OSDs with the **bluefs stats** command.

About this task

For more information about BlueStore devices, see “BlueStore devices” on page 510.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Root-level access to the OSD node.

- The object store configured as BlueStore.
- Log into the `cephadm` shell, using the **`cephadm shell`** command.

Procedure

View the BlueStore OSD statistics.

```
ceph daemon osd.OSD_ID bluefs stats
```

- Example for collocated OSDs

```
[ceph: root@host01 /]# ceph daemon osd.1 bluefs stats

1 : device size 0x3bfc00000 : using 0x1a428000(420 MiB)
wal_total:0, db_total:15296836403, slow_total:0
```

- Example for non-collocated OSDs

```
[ceph: root@host01 /]# ceph daemon osd.1 bluefs stats

0 :
1 : device size 0x1dfbfe000 : using 0x1100000(17 MiB)
2 : device size 0x27fc00000 : using 0x248000(2.3 MiB)
RocksDBBlueFSVolumeSelector: wal_total:0, db_total:7646425907, slow_total:10196562739,
db_avail:935539507
Usage matrix:
DEV/LEV     WAL         DB          SLOW        *           *           REAL
 FILES
LOG         0 B         4 MiB       0 B         0 B         0 B         756 KiB     1
WAL         0 B         4 MiB       0 B         0 B         0 B         3.3 MiB     1
DB          0 B         9 MiB       0 B         0 B         0 B         76 KiB      10
SLOW        0 B         0 B         0 B         0 B         0 B         0 B         0
TOTALS      0 B         17 MiB      0 B         0 B         0 B         0 B         12
MAXIMUMS:
LOG         0 B         4 MiB       0 B         0 B         0 B         756 KiB
WAL         0 B         4 MiB       0 B         0 B         0 B         3.3 MiB
DB          0 B         11 MiB      0 B         0 B         0 B         112 KiB
SLOW        0 B         0 B         0 B         0 B         0 B         0 B
TOTALS      0 B         17 MiB      0 B         0 B         0 B         0 B
```

In this example,

`0`

Refers to the dedicated WAL device, which is `block.wal`.

`1`

Refers to the dedicated DB device, which is `block.db`.

`2`

Refers to the main block device, which is `block` or `slow`.

`device size`

Represents an actual size of the device.

`using`

Represents the total usage. It is not restricted to BlueFS.

> **Note:** DB and WAL devices are used only by BlueFS. For a main device, usage from stored BlueStore data is also included. In this example, 2.3 MiB is the data from BlueStore.

`wal_total`, `db_total`, and `slow_total`

Values that reiterate the device values previously stated.

`db_avail`

Represents how many bytes can be taken from the SLOW device, if necessary.
Usage matrix:

Rows WAL, DB, and SLOW

Describes where the specific file was intended to be put.

Row LOG

Describes the BlueFS replay log `ino 1`.

Columns WAL, DB, and SLOW

Describe where data is actually put. The values are in allocation units. WAL and DB have bigger allocation units for performance reasons.

Columns *

Relate to virtual devices `new-db` and `new-wal` that are used for `ceph-bluestore-tool`. It should always show `0 B`.

Column REAL

Shows actual usage in bytes.

Column FILES

Shows count of files.

MAXIMUMS

This table captures the maximum value of each entry from the usage matrix.

Using the `ceph-bluestore-tool`

`ceph-bluestore-tool` is a utility to perform low-level administrative operations on a BlueStore instance.

About this task

The following commands are available with the `ceph-bluestore-tool`.

```
ceph-bluestore-tool COMMAND [ --dev DEVICE … ] [ -i OSD_ID ] [ --path OSD_PATH ] [ --out-dir DIR ] [ --log-file | -l filename ] [ --deep ]

ceph-bluestore-tool fsck|repair --path OSD_PATH [ --deep ]

ceph-bluestore-tool qfsck --path OSD_PATH

ceph-bluestore-tool allocmap --path OSD_PATH

ceph-bluestore-tool restore_cfb --path OSD_PATH

ceph-bluestore-tool show-label --dev DEVICE …

ceph-bluestore-tool prime-osd-dir --dev DEVICE --path OSD_PATH

ceph-bluestore-tool bluefs-export --path OSD_PATH --out-dir DIR

ceph-bluestore-tool bluefs-bdev-new-wal --path OSD_PATH --dev-target NEW_DEVICE

ceph-bluestore-tool bluefs-bdev-new-db --path OSD_PATH --dev-target NEW_DEVICE

ceph-bluestore-tool bluefs-bdev-migrate --path OSD_PATH --dev-target NEW_DEVICE --devs-source DEVICE1 [--devs-source DEVICE2]

ceph-bluestore-tool free-dump|free-score --path OSD_PATH [ --allocator block/bluefs-wal/bluefs-db/bluefs-slow ]

ceph-bluestore-tool reshard --path OSD_PATH --sharding NEW_SHARDING [ --sharding-ctrl CONTROL_STRING ]

ceph-bluestore-tool show-sharding --path OSD_PATH
```

Every BlueStore block device has a single block label at the beginning of the device. You can dump the contents of the label with:

```
ceph-bluestore-tool show-label --dev DEVICE
```

The main device contains a lot of metadata, including information that used to be stored in small files in the OSD data directory. The auxiliary devices (db and wal) only have the minimum required fields: OSD UUID, size, device type, and birth time).

Generate the content for an OSD data directory that can start up a BlueStore OSD with the **`prime-osd-dir`** command.

```
ceph-bluestore-tool prime-osd-dir --dev MAIN_DEVICE --path /var/lib/ceph/osd/ceph-ID
```

Table 82: `ceph-bluestore-tool` commands

Command	Description
`help`	Show help
`fsck [--deep]`	Options:`on`/`off`,`yes`/`no`,`1`/`0`, or`true`/`false`. Run consistency check on BlueStore metadata. If --deep is specified, also read all object data and verify checksums.
`repair`	Run a consistency check and repair any errors.
`qfsck`	Run consistency check on BlueStore metadata comparing allocator data with ONodes state. The allocator data comes from the RocksDB CFB, when exists, and if not uses allocation-file.
`allocmap`	Performs the same check done by **`qfsck`** and then stores a new allocation-file. This command is disabled by default and requires a special build.
`restore_cfb`	Reverses changes done by the new NCB code (either through **`ceph restart`** or when running the **`allocmap`** command) and restores RocksDB B Column-Family (allocator-map).
`bluefs-export`	Export the contents of BlueFS to an output directory.
`bluefs-bdev-sizes --path` *`OSD_PATH`*	Print the device sizes, as understood by BlueFS, to stdout.
`bluefs-bdev-expand --path` *`OSD_PATH`*	Instruct BlueFS to check the size of its block devices and, if they have expanded, make use of the additional space. Note that only the new files created by BlueFS will be allocated on the preferred block device if it has enough free space, and the existing files that have spilled over to the slow device will be gradually removed when RocksDB performs compaction. In other words, if there is any data spilled over to the slow device, it will be moved to the fast device over time.
`bluefs-bdev-new-wal --path` *`OSD_PATH`* `--dev-target` *`NEW_DEVICE`*	Adds WAL device to BlueFS, fails if WAL device already exists.
`bluefs-bdev-new-db --path` *`OSD_PATH`* `--dev-target` *`NEW_DEVICE`*	Adds DB device to BlueFS, fails if DB device already exists.

Command	Description
`bluefs-bdev-migrate --dev-target NEW_DEVICE --devs-source DEVICE1 [--devs-source DEVICE2]`	Moves BlueFS data from source device(s) to the target one, source devices (except the main one) are removed on success. Target device can be both already attached or new device. In the latter case it's added to OSD replacing one of the source devices. Following replacement rules apply (in the order of precedence, stop on the first match): (1) if source list has DB volume - target device replaces it. (2) if source list has WAL volume - target device replace it. (3) if source list has slow volume only - operation isn't permitted, requires explicit allocation via new **`-db/new-wal`** command.
`show-label --dev DEVICE […]`	Show any device labels.
`free-dump --path OSD_PATH [ --allocator block/bluefs-wal/bluefs-db/bluefs-slow ]`	Dump all free regions in allocator.
`free-score --path OSD_PATH [ --allocator block/bluefs-wal/bluefs-db/bluefs-slow ]`	Give a [0-1] number that represents quality of fragmentation in allocator. 0 represents case when all free space is in one chunk. 1 represents worst possible fragmentation.
`reshard --path OSD_PATH --sharding NEW_SHARDING [ --resharding-ctrl CONTROL_STRING ]`	Changes sharding of BlueStore's RocksDB. Sharding is build on top of RocksDB column families. This option allows to test performance of new sharding without need to redeploy OSD. Resharding is usually a long process, which involves walking through entire RocksDB key space and moving some of them to different column families. The **`--resharding-ctrl`** option provides performance control over resharding process. Interrupted resharding will prevent OSD from running. Interrupted resharding does not corrupt data. It is always possible to continue previous resharding, or select any other sharding scheme, including reverting to original one. For more information, see "Manually resharding the OSDs" on page 515.
`show-sharding --path OSD_PATH`	Show sharding that is currently applied to BlueStore's RocksDB.

Table 83: `ceph-bluestore-tool` command options

Command	Description
`--dev DEVICE`	Add the *`device`* to the list of devices to consider.
`-i OSD_ID`	Operate as OSD *`OSD_ID`*. Connect to monitor for OSD specific options. If monitor is unavailable, add **`--no-mon-config`** to read from ceph.conf instead.
`--devs-source DEVICE`	Add the _device_ to the list of devices to consider as sources for migration.
`--dev-target DEVICE`	Specify _target_ device migrate operation or device to add for adding new DB/WAL.
`--path OSD_PATH`	Specify an OSD path. In most cases, the device list is inferred from the symlinks present in osd path. This is usually simpler than explicitly specifying the device(s) with --dev. This option is not necessary if **`-i osd_id`** is provided.
`--out-dir DIR`	Output directory for `bluefs-export`

Command	Description
`-l, --log-file` *LOG_FILE*	The file to log to.
`--log-level` *NUM*	Debug log level. Default is 30 (extremely verbose), 20 is very verbose, 10 is verbose, and 1 is not very verbose.
`--deep`	Deep scrub/repair (read and validate object data, not just metadata).
`--allocator` *NAME*	Useful for free-dump and free-score actions. Selects allocator(s).
`--resharding-ctrl` *CONTROL_STRING*	Provides control over resharding process. Specifies how often refresh RocksDB iterator, and how large should commit batch be before committing to RocksDB. Option format is: `<iterator_refresh_bytes>/ <iterator_refresh_keys>/ <batch_commit_bytes>/<batch_commit_keys>` Default: 10000000/10000/1000000/1000

Procedure

1. Stop the OSD before using the `ceph-bluestore-tool`.

```
ceph orch daemon stop osd.ID
```

   For example,

```
[ceph: root@host01 /]# ceph orch daemon stop osd.2
```

2. From the OSD node, log into the target OSD container.

```
cephadm shell --name osd.ID
```

   For example,

```
[ceph: root@host01 /]# cephadm shell --name osd.2
```

3. Run the needed command.
   The following example is adding a new wal device.

```
[ceph: root@host01 /]# ceph-bluestore-tool bluefs-bdev-new-wal --dev-target /dev/test/
newdb --path /var/lib/ceph/osd/ceph-0
```

4. From the `cephadm` shell, restart the OSD.

```
ceph orch daemon start osd.ID
```

   For example,

```
[ceph: root@host01 /]# ceph orch daemon start osd.2
```

Related information

BlueStore configuration options

`cephadm` operations

As a storage administrator, you can carry out `cephadm` operations in the Red Hat Ceph Storage cluster.

Prerequisites

- A running Red Hat Ceph Storage cluster.

Monitor cephadm log messages

cephadm logs to the cephadm cluster log channel so you can monitor progress in real time.

- To monitor progress in real time, run the following command:

  Example

  ```
  [ceph: root@host01 /]# ceph -W cephadm
  ```

  Example

  ```
  2022-11-02T17:51:36.335728+0000 mgr.Ceph5-1.nqikfh [INF] refreshing Ceph5-adm facts
  2022-11-02T17:51:37.170982+0000 mgr.Ceph5-1.nqikfh [INF] deploying 1 monitor(s) instead
  of 2 so monitors may achieve consensus
  2022-11-02T17:51:37.173487+0000 mgr.Ceph5-1.nqikfh [ERR] It is NOT safe to stop
  ['mon.Ceph5-adm']: not enough monitors would be available (Ceph5-2) after stopping mons
  [Ceph5-adm]
  2022-11-02T17:51:37.179658+0000 mgr.Ceph5-1.nqikfh [INF] Found osd claims -> {}
  2022-11-02T17:51:37.180116+0000 mgr.Ceph5-1.nqikfh [INF] Found osd claims for
  drivegroup all-available-devices -> {}
  2022-11-02T17:51:37.182138+0000 mgr.Ceph5-1.nqikfh [INF] Applying all-available-devices
  on host Ceph5-adm...
  2022-11-02T17:51:37.182987+0000 mgr.Ceph5-1.nqikfh [INF] Applying all-available-devices
  on host Ceph5-1...
  2022-11-02T17:51:37.183395+0000 mgr.Ceph5-1.nqikfh [INF] Applying all-available-devices
  on host Ceph5-2...
  2022-11-02T17:51:43.373570+0000 mgr.Ceph5-1.nqikfh [INF] Reconfiguring node-
  exporter.Ceph5-1 (unknown last config time)...
  2022-11-02T17:51:43.373840+0000 mgr.Ceph5-1.nqikfh [INF] Reconfiguring daemon node-
  exporter.Ceph5-1 on Ceph5-1
  ```

- By default, the log displays info-level events. To see the debug-level messages, run the following commands:

  Example

  ```
  [ceph: root@host01 /]# ceph config set mgr mgr/cephadm/log_to_cluster_level debug
  [ceph: root@host01 /]# ceph -W cephadm --watch-debug
  [ceph: root@host01 /]# ceph -W cephadm --verbose
  ```

- Return debugging level to default info:

  Example

  ```
  [ceph: root@host01 /]# ceph config set mgr mgr/cephadm/log_to_cluster_level info
  ```

- To see the recent events, run the following command:

  Example

  ```
  [ceph: root@host01 /]# ceph log last cephadm
  ```

Theses events are also logged to ceph.cephadm.log file on the monitor hosts and to the monitor daemon's stderr.

Ceph daemon logs

You can view the Ceph daemon logs through stderr or files.

Logging to stderr directory

Traditionally, Ceph daemons have logged to /var/log/ceph. By default, cephadm daemons log to stderr and the logs are captured by the container runtime environment. For most systems, by default, these logs are sent to journald and accessible through the journalctl command.

- For example, to view the logs for the daemon on **host01** for a storage cluster with ID **5c5a50ae-272a-455d-99e9-32c6a013e694**:

```
[ceph: root@host01 /]# journalctl -u ceph-5c5a50ae-272a-455d-99e9-32c6a013e694@host01
```

This works well for normal `cephadm` operations when logging levels are low.

- To disable logging to `stderr`, set the following values:

```
[ceph: root@host01 /]# ceph config set global log_to_stderr false
[ceph: root@host01 /]# ceph config set global mon_cluster_log_to_stderr false
```

Logging to files

You can also configure Ceph daemons to log to files instead of `stderr`. When logging to files, Ceph logs are located in `/var/log/ceph/CLUSTER_FSID`.

- To enable logging to files, set the following values:

```
[ceph: root@host01 /]# ceph config set global log_to_file true
[ceph: root@host01 /]# ceph config set global mon_cluster_log_to_file true
```

Important: Log rotation to a non-default path is not currently supported.

Note: Disable logging to `stderr` to avoid double logs.

By default, `cephadm` sets up log rotation on each host to rotate these files. You can configure the logging retention schedule by modifying `/etc/logrotate.d/ceph.CLUSTER_FSID`.

Data location

Know where `cephadm` daemon data and logs are located.

- `/var/log/ceph/`*`CLUSTER_FSID`* contains all the storage cluster logs. Note that by default `cephadm` logs through `stderr` and the container runtime, so these logs are usually not present.
- `/var/lib/ceph/`*`CLUSTER_FSID`* contains all the cluster daemon data, besides logs.
- `/var/lib/ceph/`*`CLUSTER_FSID`*`/`*`DAEMON_NAME`* contains all the data for a specific daemon.
- `/var/lib/ceph/`*`CLUSTER_FSID`*`/crash` contains the crash reports for the storage cluster.
- `/var/lib/ceph/`*`CLUSTER_FSID`*`/removed` contains old daemon data directories for the stateful daemons, for example monitor or Prometheus, that have been removed by `cephadm`.

Disk usage

A few Ceph daemons may store a significant amount of data in `/var/lib/ceph`, notably the monitors and Prometheus daemon, hence move this directory to its own disk, partition, or logical volume so that the root file system is not filled up.

`cephadm` health checks

As a storage administrator, you can monitor the Red Hat Ceph Storage cluster with the additional health checks provided by the `cephadm` module. This is supplementary to the default health checks provided by the storage cluster.

`cephadm` operations health checks

Health checks are run when the `cephadm` module is active.

You can get the following health warnings:

`CEPHADM_PAUSED`

cephadm background work is paused with the ceph orch pause command. cephadm continues to perform passive monitoring activities such as checking the host and daemon status, but it does not make any changes like deploying or removing daemons. You can resume cephadm work with the ceph orch resume command.

CEPHADM_STRAY_HOST

One or more hosts have running Ceph daemons but are not registered as hosts managed by the cephadm module. This means that those services are not currently managed by cephadm, for example, a restart and upgrade that is included in the ceph orch ps command. You can manage the host(s) with the ceph orch host add HOST_NAME command but ensure that SSH access to the remote hosts is configured. Alternatively, you can manually connect to the host and ensure that services on that host are removed or migrated to a host that is managed by cephadm. You can also disable this warning with the setting ceph config set mgr mgr/cephadm/ warn_on_stray_hosts false

CEPHADM_STRAY_DAEMON

One or more Ceph daemons are running but are not managed by the cephadm module. This might be because they were deployed using a different tool, or because they were started manually. Those services are not currently managed by cephadm, for example, a restart and upgrade that is included in the ceph orch ps command.

If the daemon is a stateful one that is a monitor or OSD daemon, these daemons should be adopted by cephadm. For stateless daemons, you can provision a new daemon with the ceph orch apply command and then stop the unmanaged daemon.

You can disable this health warning with the setting ceph config set mgr mgr/cephadm/ warn_on_stray_daemons false.

CEPHADM_HOST_CHECK_FAILED

One or more hosts have failed the basic cephadm host check, which verifies that:name: value

- The host is reachable and you can run cephadm.
- The host meets the basic prerequisites, like a working container runtime that is Podman , and working time synchronization. If this test fails, cephadm wont be able to manage the services on that host.

You can manually run this check with the ceph cephadm check-host HOST_NAME command. You can remove a broken host from management with the ceph orch host rm HOST_NAME command. You can disable this health warning with the setting ceph config set mgr mgr/cephadm/warn_on_failed_host_check false.

cephadm configuration health checks

cephadm periodically scans each of the hosts in the storage cluster, to understand the state of the OS, disks, and NICs . These facts are analyzed for consistency across the hosts in the storage cluster to identify any configuration anomalies. The configuration checks are an optional feature.

Enabling cephadm configuration health checks

Enable cephadm configuration health checks by using the following command:

```
[ceph: root@host01 /]# ceph config set mgr mgr/cephadm/config_checks_enabled true
```

The configuration checks are triggered after each host scan, which is for a duration of one minute.

Viewing the current state

Use the **ceph -W cephadm** command to show log entries of the current state and display the output of the configuration checks.

Example output for disabled state

```
ALL cephadm checks are disabled, use ceph config set mgr mgr/cephadm/
config_checks_enabled true command to enable
```

Example output for enabled state

```
CEPHADM 8/8 checks enabled and executed (0 bypassed, 0 disabled). No issues detected
```

Managing configuration checks

The configuration checks themselves are managed through several `cephadm` subcommands.

Determine whether the configuration checks are enabled by running the following command:

```
ceph cephadm config-check status
```

This command returns the status of the configuration checker as either `Enabled` or `Disabled`.

List all the configuration checks and their current state by running the following command:

```
ceph cephadm config-check ls
```

For example,

```
[ceph: root@host01 /]# ceph cephadm config-check ls
NAME             HEALTHCHECK                      STATUS   DESCRIPTION
kernel_security  CEPHADM_CHECK_KERNEL_LSM         enabled  checks SELINUX/Apparmor profiles
are consistent across cluster hosts
os_subscription  CEPHADM_CHECK_SUBSCRIPTION       enabled  checks subscription states are
consistent for all cluster hosts
public_network   CEPHADM_CHECK_PUBLIC_MEMBERSHIP  enabled  check that all hosts have a NIC
on the Ceph public_netork
osd_mtu_size     CEPHADM_CHECK_MTU                enabled  check that OSD hosts share a
common MTU setting
osd_linkspeed    CEPHADM_CHECK_LINKSPEED          enabled  check that OSD hosts share a
common linkspeed
network_missing  CEPHADM_CHECK_NETWORK_MISSING    enabled  checks that the cluster/public
networks defined exist on the Ceph hosts
ceph_release     CEPHADM_CHECK_CEPH_RELEASE       enabled  check for Ceph version
consistency - ceph daemons should be on the same release (unless upgrade is active)
kernel_version   CEPHADM_CHECK_KERNEL_VERSION     enabled  checks that the MAJ.MIN of the
kernel on Ceph hosts is consistent
```

Each configuration check is described as follows:

`CEPHADM_CHECK_KERNEL_LSM`

Each host within the storage cluster is expected to operate within the same Linux Security Module (LSM) state. For example, if the majority of the hosts are running with SELINUX in `enforcing` mode, any host not running in this mode would be flagged as an anomaly and a healthcheck with a warning state is raised.

`CEPHADM_CHECK_SUBSCRIPTION`

This check relates to the status of the vendor subscription. This check is only performed for hosts using Red Hat Enterprise Linux, but helps to confirm that all the hosts are covered by an active subscription so that patches and updates are available.

`CEPHADM_CHECK_PUBLIC_MEMBERSHIP`

All members of the cluster should have NICs configured on at least one of the public network subnets. Hosts that are not on the public network will rely on routing which may affect performance.

`CEPHADM_CHECK_MTU`

The maximum transmission unit (MTU) of the NICs on OSDs can be a key factor in consistent performance. This check examines hosts that are running OSD services to ensure that the MTU is configured consistently within the cluster. This is determined by establishing the MTU setting that the majority of hosts are using, with any anomalies resulting in a Ceph health check.

`CEPHADM_CHECK_LINKSPEED`

Similar to the MTU check, linkspeed consistency is also a factor in consistent cluster performance. This check determines the linkspeed shared by the majority of the OSD hosts, resulting in a healthcheck for any hosts that are set at a lower linkspeed rate.

`CEPHADM_CHECK_NETWORK_MISSING`

The `public_network` and `cluster_network` settings support subnet definitions for IPv4 and IPv6. If these settings are not found on any host in the storage cluster a healthcheck is raised.

`CEPHADM_CHECK_CEPH_RELEASE`

Under normal operations, the Ceph cluster should be running daemons under the same Ceph release, for example all Red Hat Ceph Storage cluster releases. This check looks at the active release for each daemon, and reports any anomalies as a healthcheck. This check is bypassed if an upgrade process is active within the cluster.

CEPHADM_CHECK_KERNEL_VERSION

The OS kernel version is checked for consistency across the hosts. Once again, the majority of the hosts is used as the basis of identifying anomalies.

Using `cephadm-ansible` modules

Use `cephadm-ansible` modules in Ansible playbooks to administer your Red Hat Ceph Storage cluster.

The `cephadm-ansible` package provides several modules that wrap `cephadm` calls to let you write your own unique Ansible playbooks to administer your cluster.

> **Note:** `cephadm-ansible` modules only support the most important tasks. Any operation not covered by `cephadm-ansible` modules must be completed using either the `command` or `shell` Ansible modules in your playbooks.

`cephadm-ansible` modules

The `cephadm-ansible` modules are a collection of modules that simplify writing Ansible playbooks by providing a wrapper around `cephadm` and `ceph orch` commands. You can use the modules to write your own unique Ansible playbooks to administer your cluster using one or more of the modules.

The `cephadm-ansible` package includes the following modules:

- `cephadm_bootstrap`
- `ceph_orch_host`
- `ceph_config`
- `ceph_orch_apply`
- `ceph_orch_daemon`
- `cephadm_registry_login`

`cephadm-ansible` module options

This information lists available options for the `cephadm-ansible` modules.

Options listed as required need to be set when using the modules in your Ansible playbooks. Options listed with a default value of `true` indicate that the option is automatically set when using the modules and you do not need to specify it in your playbook. For example, for the `cephadm_bootstrap` module, the Ceph Dashboard is installed unless you set `dashboard: false`.

Table 84: Available options for the `cephadm_bootstrap` module

cephadm_bootstrap	Description	Required	Default
`mon_ip`	Ceph Monitor IP address.	true	
`image`	Ceph container image.	false	
`docker`	Use `docker` instead of `podman`.	false	
`fsid`	Define the Ceph FSID.	false	
`pull`	Pull the Ceph container image.	false	true
`dashboard`	Deploy the Ceph Dashboard.	false	true
`dashboard_user`	Specify a specific Ceph Dashboard user.	false	

cephadm_bootstrap	Description	Required	Default
dashboard_password	Ceph Dashboard password.	false	
monitoring	Deploy the monitoring stack.	false	true
firewalld	Manage firewall rules with firewalld.	false	true
allow_overwrite	Allow overwrite of existing --output-config, --output-keyring, or --output-pub-ssh-key files.	false	false
registry_url	URL for custom registry.	false	
registry_username	Username for custom registry.	false	
registry_password	Password for custom registry.	false	
registry_json	JSON file with custom registry login information.	false	
ssh_user	SSH user to use for cephadm ssh to hosts.	false	
ssh_config	SSH config file path for cephadm SSH client.	false	
allow_fqdn_hostname	Allow hostname that is a fully-qualified domain name (FQDN).	false	false
cluster_network	Subnet to use for cluster replication, recovery and heartbeats.	false	

Table 85: Available options for the ceph_orch_host module

ceph_orch_host	Description	Required	Default
fsid	The FSID of the Ceph cluster to interact with.	false	
image	The Ceph container image to use.	false	
name	Name of the host to add, remove, or update.	true	
address	IP address of the host.	true when state is present.	
set_admin_label	Set the _admin label on the specified host.	false	false
labels	The list of labels to apply to the host.	false	[]

ceph_orch_host	Description	Required	Default
state	If set to present, it ensures the name specified in name is present. If set to absent, it removes the host specified in name. If set to drain, it schedules to remove all daemons from the host specified in name.	false	present

Table 86: Available options for the ceph_config module

ceph_config	Description	Required	Default
fsid	The FSID of the Ceph cluster to interact with.	false	
image	The Ceph container image to use.	false	
action	Whether to set or get the parameter specified in option.	false	set
who	Which daemon to set the configuration to.	true	
option	Name of the parameter to set or get.	true	
value	Value of the parameter to set.	true if action is set	

Table 87: Available options for the ceph_orch_apply module

ceph_orch_apply	Description	Required
fsid	The FSID of the Ceph cluster to interact with.	false
image	The Ceph container image to use.	false
spec	The service specification to apply.	true

Table 88: Available options for the ceph_orch_daemon module

ceph_orch_daemon	Description	Required
fsid	The FSID of the Ceph cluster to interact with.	false
image	The Ceph container image to use.	false

ceph_orch_daemon	Description	Required
state	The desired state of the service specified in name.	true If started, it ensures the service is started. If stopped, it ensures the service is stopped. If restarted, it will restart the service.
daemon_id	The ID of the service.	true
daemon_type	The type of service.	true

Table 89: Available options for the cephadm_registry_login module

cephadm_registry_login	Description	Required	Default
state	Login or logout of a registry.	false	login
docker	Use docker instead of podman.	false	
registry_url	The URL for custom registry.	false	
registry_username	Username for custom registry.	true when state is login.	
registry_password	Password for custom registry.	true when state is login.	
registry_json	The path to a JSON file. This file must be present on remote hosts prior to running this task. This option is currently not supported.		

Bootstrapping a storage cluster with Ansible

As a storage administrator, you can bootstrap a storage cluster using Ansible by using the cephadm_bootstrap and cephadm_registry_login modules in your Ansible playbook.

Before you begin

- An IP address for the first Ceph Monitor container, which is also the IP address for the first node in the storage cluster.
- Login access to registry.redhat.io.
- A minimum of 10 GB of free space for /var/lib/containers/.
- Installation of the cephadm-ansible package on the Ansible administration node.
- Passwordless SSH is set up on all hosts in the storage cluster.
- Hosts are registered with CDN.

For the latest supported Red Hat Enterprise Linux versions, see Compatibility matrix.

Procedure

1. Log in to the Ansible administration node.
2. Navigate to the `/usr/share/cephadm-ansible` directory on the Ansible administration node.
   For example,

   ```
   [ansible@admin ~]$ cd /usr/share/cephadm-ansible
   ```

3. Create the `hosts` file and add hosts, labels, and monitor IP address of the first host in the storage cluster.

   ```
   sudo vi INVENTORY_FILE

   HOST1 labels="['LABEL1', 'LABEL2']"
   HOST2 labels="['LABEL1', 'LABEL2']"
   HOST3 labels="['LABEL1']"

   [admin]
   ADMIN_HOST monitor_address=MONITOR_IP_ADDRESS labels="['ADMIN_LABEL', 'LABEL1',
   'LABEL2']"
   ```

   For example,

   ```
   [ansible@admin cephadm-ansible]$ sudo vi hosts

   host02 labels="['mon', 'mgr']"
   host03 labels="['mon', 'mgr']"
   host04 labels="['osd']"
   host05 labels="['osd']"
   host06 labels="['osd']"

   [admin]
   host01 monitor_address=10.10.128.68 labels="['_admin', 'mon', 'mgr']"
   ```

4. Run the preflight playbook:

   ```
   ansible-playbook -i INVENTORY_FILE cephadm-preflight.yml --extra-vars
   "ceph_origin=rhcs"
   ```

   For example,

   ```
   [ansible@admin cephadm-ansible]$ ansible-playbook -i hosts cephadm-preflight.yml --
   extra-vars "ceph_origin=rhcs""
   ```

5. Create a playbook to bootstrap your cluster.

   ```
   sudo vi PLAYBOOK_FILENAME.yml

   ---
   - name: NAME_OF_PLAY
     hosts: BOOTSTRAP_HOST
     become: USE_ELEVATED_PRIVILEGES
     gather_facts: GATHER_FACTS_ABOUT_REMOTE_HOSTS
     tasks:
       -name: NAME_OF_TASK
        cephadm_registry_login:
          state: STATE
          registry_url: REGISTRY_URL
          registry_username: REGISTRY_USER_NAME
          registry_password: REGISTRY_PASSWORD

       - name: NAME_OF_TASK
         cephadm_bootstrap:
           mon_ip: "{{ monitor_address }}"
           dashboard_user: DASHBOARD_USER
           dashboard_password: DASHBOARD_PASSWORD
           allow_fqdn_hostname: ALLOW_FQDN_HOSTNAME
           cluster_network: NETWORK_CIDR
   ```

   For example,

```
sudo vi bootstrap.yml

---
- name: bootstrap the cluster
  hosts: host01
  become: true
  gather_facts: false
  tasks:
    - name: login to registry
      cephadm_registry_login:
        state: login
        registry_url: registry.redhat.io
        registry_username: user1
        registry_password: mypassword1

    - name: bootstrap initial cluster
      cephadm_bootstrap:
        mon_ip: "{{ monitor_address }}"
        dashboard_user: mydashboarduser
        dashboard_password: mydashboardpassword
        allow_fqdn_hostname: true
        cluster_network: 10.10.128.0/28
```

6. Run the playbook.

```
ansible-playbook -i INVENTORY_FILE PLAYBOOK_FILENAME.yml -vvv
```

For example,

```
ansible@admin cephadm-ansible]$ ansible-playbook -i hosts bootstrap.yml -vvv
```

Verification

Review the Ansible output after running the playbook.

Adding and removing hosts

Add and remove hosts in your storage cluster by using the `ceph_orch_host` module in your Ansible playbook.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Register the nodes to the CDN and attach subscriptions.
- Ansible user with sudo and passwordless SSH access to all nodes in the storage cluster.
- Installation of the `cephadm-ansible` package on the Ansible administration node.
- New hosts have the storage cluster's public SSH key.
  For more information about copying the storage cluster's public SSH keys to new hosts, see "Adding hosts" on page 223.

Adding hosts

Add new hosts to the storage cluster.

Procedure

1. Log in to the Ansible administration node.
2. Navigate to the `/usr/share/cephadm-ansible` directory on the Ansible administration node.
   For example,

```
[ansible@admin ~]$ cd /usr/share/cephadm-ansible
```

3. Add the new hosts and labels to the Ansible inventory file.

Note: If you have previously added the new hosts to the Ansible inventory file and ran the preflight playbook on the hosts, skip to step “5” on page 535.

```
sudo vi INVENTORY_FILE

NEW_HOST1 labels="['LABEL1', 'LABEL2']"
NEW_HOST2 labels="['LABEL1', 'LABEL2']"
NEW_HOST3 labels="['LABEL1']"

[admin]
ADMIN_HOST monitor_address=MONITOR_IP_ADDRESS labels="['ADMIN_LABEL', 'LABEL1',
'LABEL2']"
```

For example,

```
[ansible@admin cephadm-ansible]$ sudo vi hosts

host02 labels="['mon', 'mgr']"
host03 labels="['mon', 'mgr']"
host04 labels="['osd']"
host05 labels="['osd']"
host06 labels="['osd']"

[admin]
host01 monitor_address= 10.10.128.68 labels="['_admin', 'mon', 'mgr']"
```

4. Run the preflight playbook with the `--limit` option.

```
ansible-playbook -i INVENTORY_FILE cephadm-preflight.yml --extra-vars
"ceph_origin=rhcs" --limit NEWHOST
```

For example,

```
[ansible@admin cephadm-ansible]$ ansible-playbook -i hosts cephadm-preflight.yml --
extra-vars "ceph_origin=rhcs" --limit host02
```

The preflight playbook installs `podman`, `lvm2`, `chrony`, and `cephadm` on the new host. After installation is complete, `cephadm` resides in the `/usr/sbin/` directory.

5. Create a playbook to add the new hosts to the cluster.

```
sudo vi PLAYBOOK_FILENAME.yml

---
- name: PLAY_NAME
  hosts: HOSTS_OR_HOST_GROUPS
  become: USE_ELEVATED_PRIVILEGES
  gather_facts: GATHER_FACTS_ABOUT_REMOTE_HOSTS
  tasks:
    - name: NAME_OF_TASK
      ceph_orch_host:
        name: "{{ ansible_facts[hostname] }}"
        address: "{{ ansible_facts[default_ipv4][address] }}"
        labels: "{{ labels }}"
      delegate_to: HOST_TO_DELEGATE_TASK_TO

    - name: NAME_OF_TASK
      when: inventory_hostname in groups[admin]
      ansible.builtin.shell:
        cmd: CEPH_COMMAND_TO_RUN
      register: REGISTER_NAME

    - name: NAME_OF_TASK
      when: inventory_hostname in groups[admin]
      debug:
        msg: "{{ REGISTER_NAME.stdout }}"
```

> **Note:** By default, Ansible runs all tasks on the host that matches the `hosts` line of your playbook. The **ceph orch** commands must run on the host that contains the admin keyring and the Ceph configuration file. Use the `delegate_to` keyword to specify the admin host in your cluster.

In the following example, the playbook adds the new hosts to the cluster and displays a current list of hosts.

```
[ansible@admin cephadm-ansible]$ sudo vi add-hosts.yml

---
- name: add additional hosts to the cluster
  hosts: all
  become: true
  gather_facts: true
  tasks:
    - name: add hosts to the cluster
      ceph_orch_host:
        name: "{{ ansible_facts['hostname'] }}"
        address: "{{ ansible_facts['default_ipv4']['address'] }}"
        labels: "{{ labels }}"
      delegate_to: host01

    - name: list hosts in the cluster
      when: inventory_hostname in groups['admin']
      ansible.builtin.shell:
        cmd: ceph orch host ls
      register: host_list

    - name: print current list of hosts
      when: inventory_hostname in groups['admin']
      debug:
        msg: "{{ host_list.stdout }}"
```

6. Run the playbook to add additional hosts to the cluster.

```
ansible-playbook -i INVENTORY_FILE PLAYBOOK_FILENAME.yml
```

For example,

```
[ansible@admin cephadm-ansible]$ ansible-playbook -i hosts add-hosts.yml
```

What to do next

Review the Ansible task output displaying the current list of hosts in the cluster.
For example,

```
TASK [print current hosts]
*****************************************************************************************
***********
Friday 24 April 2026  14:52:40 -0400 (0:00:03.365)       0:02:31.702 ***********
ok: [host01] =>
  msg: |-
    HOST    ADDR           LABELS          STATUS
    host01  10.10.128.68   _admin mon mgr
    host02  10.10.128.69   mon mgr
    host03  10.10.128.70   mon mgr
    host04  10.10.128.71   osd
    host05  10.10.128.72   osd
    host06  10.10.128.73   osd
```

Removing hosts

Remove hosts from the cluster.

Procedure

1. Log in to the Ansible administration node.
2. Navigate to the `/usr/share/cephadm-ansible` directory on the Ansible administration node.
   For example,

```
[ansible@admin ~]$ cd /usr/share/cephadm-ansible
```

3. Create a playbook to remove a host or multiple hosts from the cluster.

```
sudo vi PLAYBOOK_FILENAME.yml

---
- name: NAME_OF_PLAY
  hosts: ADMIN_HOST
  become: USE_ELEVATED_PRIVILEGES
  gather_facts: GATHER_FACTS_ABOUT_REMOTE_HOSTS
  tasks:
    - name: NAME_OF_TASK
      ceph_orch_host:
        name: HOST_TO_REMOVE
        state: STATE

    - name: NAME_OF_TASK
      ceph_orch_host:
        name: HOST_TO_REMOVE
        state: STATE
      retries: NUMBER_OF_RETRIES
      delay: DELAY
      until: CONTINUE_UNTIL
      register: REGISTER_NAME

    - name: NAME_OF_TASK
      ansible.builtin.shell:
        cmd: ceph orch host ls
      register: REGISTER_NAME

    - name: NAME_OF_TASK
        debug:
          msg: "{{ REGISTER_NAME.stdout }}"
```

In the following example, the playbook tasks drain all daemons on `host07`, removes the host from the cluster, and displays a current list of hosts.

```
[ansible@admin cephadm-ansible]$ sudo vi remove-hosts.yml

---
- name: remove host
  hosts: host01
  become: true
  gather_facts: true
  tasks:
    - name: drain host07
      ceph_orch_host:
        name: host07
        state: drain

    - name: remove host from the cluster
      ceph_orch_host:
        name: host07
        state: absent
      retries: 20
      delay: 1
      until: result is succeeded
      register: result

     - name: list hosts in the cluster
       ansible.builtin.shell:
         cmd: ceph orch host ls
       register: host_list

     - name: print current list of hosts
       debug:
         msg: "{{ host_list.stdout }}"
```

4. Run the playbook to remove the hosts from the cluster.

```
ansible-playbook -i INVENTORY_FILE PLAYBOOK_FILENAME.yml
```

For example,

```
[ansible@admin cephadm-ansible]$ ansible-playbook -i hosts remove-hosts.yml
```

What to do next

Review the Ansible task output displaying the current list of hosts in the cluster.
For example,

```
TASK [print current hosts]
*****************************************************************************************
***********
Friday 24 April 2026  14:52:40 -0400 (0:00:03.365)       0:02:31.702 ***********
ok: [host01] =>
  msg: |-
    HOST    ADDR           LABELS          STATUS
    host01  10.10.128.68   _admin mon mgr
    host02  10.10.128.69   mon mgr
    host03  10.10.128.70   mon mgr
    host04  10.10.128.71   osd
    host05  10.10.128.72   osd
    host06  10.10.128.73   osd
```

Setting configuration options

As a storage administrator, you can set or get Red Hat Ceph Storage configuration options using the `ceph_config` module.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Ansible user with sudo and passwordless SSH access to all nodes in the storage cluster.
- Installation of the `cephadm-ansible` package on the Ansible administration node.
- The Ansible inventory file contains the cluster and admin hosts.

Procedure

1. Log in to the Ansible administration node.
2. Navigate to the `/usr/share/cephadm-ansible` directory on the Ansible administration node:

   Example

   ```
   [ansible@admin ~]$ cd /usr/share/cephadm-ansible
   ```

3. Create a playbook with configuration changes:

   Syntax

   ```
   sudo vi PLAYBOOK_FILENAME.yml

   ---
   - name: PLAY_NAME
     hosts: ADMIN_HOST
     become: USE_ELEVATED_PRIVILEGES
     gather_facts: GATHER_FACTS_ABOUT_REMOTE_HOSTS
     tasks:
       - name: NAME_OF_TASK
         ceph_config:
           action: GET_OR_SET
           who: DAEMON_TO_SET_CONFIGURATION_TO
           option: CEPH_CONFIGURATION_OPTION
           value: VALUE_OF_PARAMETER_TO_SET

       - name: NAME_OF_TASK
         ceph_config:
           action: GET_OR_SET
           who: DAEMON_TO_SET_CONFIGURATION_TO
           option: CEPH_CONFIGURATION_OPTION
         register: REGISTER_NAME

       - name: NAME_OF_TASK
         debug:
           msg: "MESSAGE_TO_DISPLAY {{ REGISTER_NAME.stdout }}"
   ```

Example

```
[ansible@admin cephadm-ansible]$ sudo vi change_configuration.yml

---
- name: set pool delete
  hosts: host01
  become: true
  gather_facts: false
  tasks:
    - name: set the allow pool delete option
      ceph_config:
        action: set
        who: mon
        option: mon_allow_pool_delete
        value: true

    - name: get the allow pool delete setting
      ceph_config:
        action: get
        who: mon
        option: mon_allow_pool_delete
      register: verify_mon_allow_pool_delete

    - name: print current mon_allow_pool_delete setting
      debug:
        msg: "the value of 'mon_allow_pool_delete' is
{{ verify_mon_allow_pool_delete.stdout }}"
```

In this example, the playbook first sets the `mon_allow_pool_delete` option to `false`. The playbook then gets the current `mon_allow_pool_delete` setting and displays the value in the Ansible output.

4. Run the playbook:

   Syntax

   ```
   ansible-playbook -i INVENTORY_FILE PLAYBOOK_FILENAME.yml
   ```

   Example

   ```
   [ansible@admin cephadm-ansible]$ ansible-playbook -i hosts change_configuration.yml
   ```

Verification

- Review the output from the playbook tasks.

  Example

  ```
  TASK [print current mon_allow_pool_delete setting]
  ************************************************************
  Wednesday 29 June 2022  13:51:41 -0400 (0:00:05.523)       0:00:17.953 ********
  ok: [host01] =>
    msg: the value of 'mon_allow_pool_delete' is true
  ```

Reference

- For more information about configuration options, see Configuring.

Applying a service specification

As a storage administrator, you can apply service specifications to your storage cluster using the `ceph_orch_apply` module in your Ansible playbooks.

A service specification is a data structure to specify the service attributes and configuration settings that is used to deploy the Ceph service. You can use a service specification to deploy Ceph service types like `mon`, `crash`, `mds`, `mgr`, `osd`, `rdb`, or `rbd-mirror`.

For more information about service specification options, see .

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Ansible user with sudo and passwordless SSH access to all nodes in the storage cluster.
- Installation of the `cephadm-ansible` package on the Ansible administration node.
- The Ansible inventory file contains the cluster and admin hosts.

Procedure

1. Log in to the Ansible administration node.
2. Navigate to the `/usr/share/cephadm-ansible` directory on the Ansible administration node:

   Example

   ```
   [ansible@admin ~]$ cd /usr/share/cephadm-ansible
   ```

3. Create a playbook with the service specifications.

   ```
   sudo vi PLAYBOOK_FILENAME.yml

   ---
   - name: PLAY_NAME
     hosts: HOSTS_OR_HOST_GROUPS
     become: USE_ELEVATED_PRIVILEGES
     gather_facts: GATHER_FACTS_ABOUT_REMOTE_HOSTS
     tasks:
       - name: NAME_OF_TASK
         ceph_orch_apply:
           spec: |
             service_type: SERVICE_TYPE
             service_id: UNIQUE_NAME_OF_SERVICE
             placement:
               host_pattern: HOST_PATTERN_TO_SELECT_HOSTS
               label: LABEL
             spec:
               SPECIFICATION_OPTIONS:
   ```

   Example

   ```
   [ansible@admin cephadm-ansible]$ sudo vi deploy_osd_service.yml

   ---
   - name: deploy osd service
     hosts: host01
     become: true
     gather_facts: true
     tasks:
       - name: apply osd spec
         ceph_orch_apply:
           spec: |
             service_type: osd
             service_id: osd
             placement:
               host_pattern: '*'
               label: osd
             spec:
               data_devices:
                 all: true
   ```

   In this example, the playbook deploys the Ceph OSD service on all hosts with the label `osd`.

4. Run the playbook.

   ```
   ansible-playbook -i INVENTORY_FILE PLAYBOOK_FILENAME.yml
   ```

   Example

   ```
   [ansible@admin cephadm-ansible]$ ansible-playbook -i hosts deploy_osd_service.yml
   ```

Verification

- Review the output from the playbook tasks.

Managing Ceph daemon states

As a storage administrator, you can start, stop, and restart Ceph daemons on hosts using the `ceph_orch_daemon` module in your Ansible playbooks.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Ansible user with sudo and passwordless SSH access to all nodes in the storage cluster.
- Installation of the `cephadm-ansible` package on the Ansible administration node.
- The Ansible inventory file contains the cluster and admin hosts.

Procedure

1. Log in to the Ansible administration node.
2. Navigate to the `/usr/share/cephadm-ansible` directory on the Ansible administration node:

   Example

   ```
   [ansible@admin ~]$ cd /usr/share/cephadm-ansible
   ```

3. Create a playbook with daemon state changes:

   Syntax

   ```
   sudo vi PLAYBOOK_FILENAME.yml

   ---
   - name: PLAY_NAME
     hosts: ADMIN_HOST
     become: USE_ELEVATED_PRIVILEGES
     gather_facts: GATHER_FACTS_ABOUT_REMOTE_HOSTS
     tasks:
       - name: NAME_OF_TASK
         ceph_orch_daemon:
           state: STATE_OF_SERVICE
           daemon_id: DAEMON_ID
           daemon_type: TYPE_OF_SERVICE
   ```

   Example

   ```
   [ansible@admin cephadm-ansible]$ sudo vi restart_services.yml

   ---
   - name: start and stop services
     hosts: host01
     become: true
     gather_facts: false
     tasks:
       - name: start osd.0
         ceph_orch_daemon:
           state: started
           daemon_id: 0
           daemon_type: osd

       - name: stop mon.host02
         ceph_orch_daemon:
           state: stopped
           daemon_id: host02
           daemon_type: mon
   ```

   In this example, the playbook starts the OSD with an ID of `0` and stops a Ceph Monitor with an id of `host02`.

4. Run the playbook:

   Syntax

   ```
   ansible-playbook -i INVENTORY_FILE PLAYBOOK_FILENAME.yml
   ```

   Example

   ```
   [ansible@admin cephadm-ansible]$ ansible-playbook -i hosts restart_services.yml
   ```

Verification

- Review the output from the playbook tasks.

Operations

Learn to do operational tasks for Red Hat Ceph Storage.

Introduction to the Ceph Orchestrator

Red Hat Ceph Storage Orchestrators are manager modules that primarily act as a bridge between a Red Hat Ceph Storage cluster and deployment tools like Rook and `cephadm` for a unified experience. They also integrate with the Ceph command line interface and Ceph Dashboard.

As a storage administrator, you can use the Ceph Orchestrator with `cephadm` utility that provides the ability to discover devices and create services in a Red Hat Ceph Storage cluster.

"Figure: Ceph Orchestrator" on page 542 is a workflow diagram of Ceph Orchestrator.

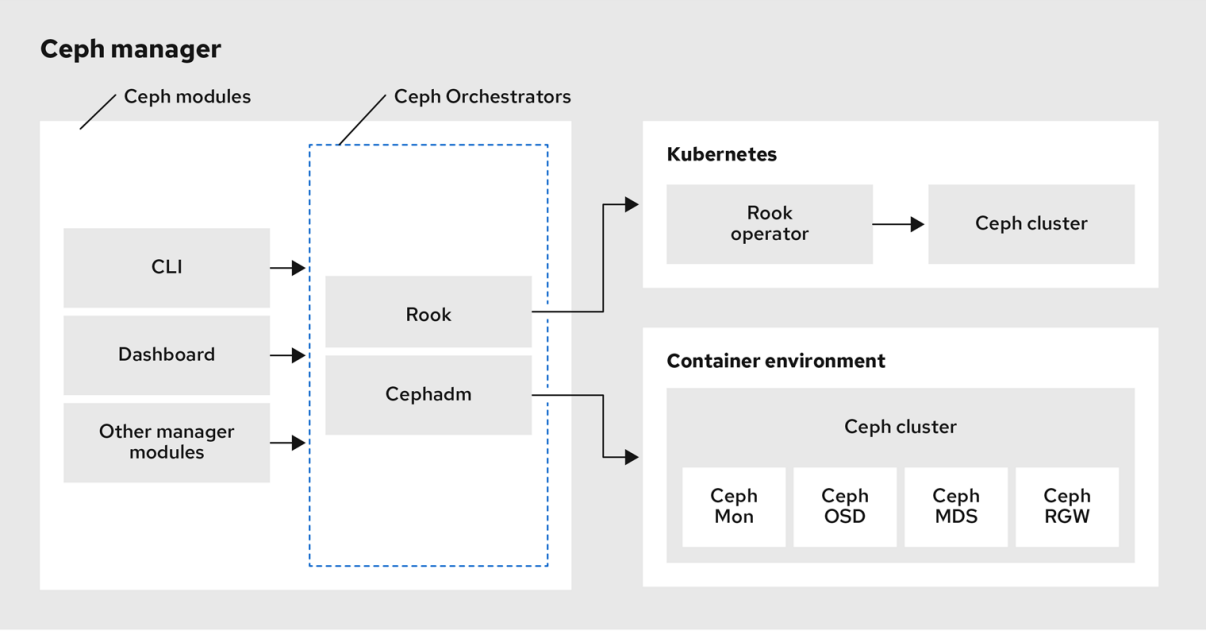


Figure 54: Ceph Orchestrator

Types of Red Hat Ceph Storage

There are three main types of Red Hat Ceph Storage Orchestrators:

Orchestrator CLI

These are common APIs used in Orchestrators and include a set of commands that can be implemented. These APIs also provide a common command line interface (CLI) to orchestrate `ceph-mgr` modules with external orchestration services. The following are the nomenclature that is used with the Ceph Orchestrator:

- Host: This is the host name of the physical host and not the pod name, DNS name, container name, or host name inside the container.
- Service type: This is the type of the service, such as `mds`, `osd`, `mon`, `rgw`, and `mgr`.
- Service: A functional service that is provided by a Ceph storage cluster such as monitors service, managers service, OSD services, and Ceph Object Gateway service.
- Daemon: A specific instance of a service that is deployed by one or more hosts such as Ceph Object Gateway services can have different Ceph Object Gateway daemons running in three different hosts.

`cephadm` Orchestrator

This is a Ceph Orchestrator module that does not rely on an external tool such as Rook or Ansible, but rather manages nodes in a cluster by establishing an SSH connection and issuing explicit management commands. This module is intended for day-one and day-two operations.

Using the `cephadm` Orchestrator is the recommended way of installing a Ceph storage cluster without using any deployment frameworks like Ansible. The idea is to provide the manager daemon with access to an SSH configuration and key that is able to connect to all nodes in a cluster to perform any management operations, like creating an inventory of storage devices, deploying and replacing OSDs, or starting and stopping Ceph daemons. In addition, the `cephadm` Orchestrator will deploy container images that are managed by `systemd` in order to allow independent upgrades of colocated services.

This orchestrator also highlights a tool that encapsulates all necessary operations to manage the deployment of container image-based services on the current host, including a command that bootstraps a minimal cluster that runs a Ceph Monitor and a Ceph Manager.

Rook Orchestrator

Rook is an orchestration tool that uses the Kubernetes Rook operator to manage a Ceph storage cluster running inside a Kubernetes cluster. The rook module provides integration between Ceph's Orchestrator framework and Rook. Rook is an open source cloud-native storage operator for Kubernetes.

Rook follows the "operator" model, in which a custom resource definition (CRD) object is defined in Kubernetes to describe a Ceph storage cluster and its desired state, and a rook operator daemon is running in a control loop that compares the current cluster state to desired state and takes steps to make them converge. The main object describing Ceph's desired state is the Ceph storage cluster CRD, which includes information about which devices should be consumed by OSDs, how many monitors should be running, and what version of Ceph must be used. Rook defines several other CRDs to describe RBD pools, CephFS file systems, and so on.

The Rook Orchestrator module is the glue that runs in the `ceph-mgr` daemon and implements the Ceph orchestration API by making changes to the Ceph storage cluster in Kubernetes that describe desired cluster state. A Rook cluster's `ceph-mgr` daemon is running as a Kubernetes pod, and hence, the rook module can connect to the Kubernetes API without any explicit configuration.

Managing services

As a storage administrator, after installing the Red Hat Ceph Storage cluster, you can monitor and manage the services in a storage cluster. A service is a group of daemons that are configured together.

Checking service status

Check the service status of the storage cluster, by using the **`ceph orch ls`** command.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Log in to the `cephadm` shell.

About this task

You can check the service status of the storage cluster with the **`ceph orch ls`** command. This command checks the status of the following services of the storage cluster:

- Print a list of services.

- Locate the service whose status you want to check.
- Print the status of the service.

Note: If the services are applied with the **ceph orch apply** command while bootstrapping, changing the service specification file is complicated. Instead, you can use the `--export` option with the **ceph orch ls** command to export the running specification, update the `yaml` file, and reapply the service.

Procedure

- Print a list of services.

```
ceph orch ls [--service_type SERVICE_TYPE] [--service_name SERVICE_NAME] [--export] [--
format FORMAT] [--refresh]
```

The format can be plain, `json`, `json-pretty`, `yaml`, `xml-pretty`, or `xml`.

For example,

```
[ceph: root@host01 /]# ceph orch ls
```

- Check the status of a particular service or a daemon.

```
ceph orch ls [--service_type SERVICE_TYPE] [--service_name SERVICE_NAME] [--refresh]
```

For example,

```
ceph: root@host01 /]# ceph orch ls --service-type mds
[ceph: root@host01 /]# ceph orch ls --service-name rgw.realm.myzone
```

- Export the service specification.
For example,

```
[ceph: root@host01 /]# ceph orch ls --service-type mgr --export > mgr.yaml
[ceph: root@host01 /]# ceph orch ls --export > cluster.yaml
```

This exports the file in the `.yaml` file format. This file can be used with the **ceph orch apply -i** command to retrieve the service specification of a single service.

Checking daemon status

A daemon is a `systemd` unit that is running and is part of the service. The daemon status can be checked by using the **ceph orch ps** command.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Log into the `cephadm` shell.

About this task

You can check the following status of the daemons of the storage cluster using the `ceph orch ps` command. This command checks provides the following information:

- Print a list of all the daemons.
- Query the status of the target daemon.

Procedure

1. Print a list of daemons.

```
ceph orch ps [--daemon-type DAEMON_TYPE] [--service_name SERVICE_NAME] [--
daemon_id DAEMON_ID] [--format FORMAT] [--refresh]
```

For example,

```
[ceph: root@host01 /]# ceph orch ps
```

2. Verify the status of the service instance.

```
ceph orch ls [--daemon-type DAEMON_TYPE] [--daemon_id DAEMON_ID] [--refresh]
```

For example,

```
[ceph: root@host01 /]# ceph orch ps --daemon_type osd --daemon_id 0
```

Placement specification of the Ceph Orchestrator

You can use the Ceph Orchestrator to deploy `osds`, `mons`, `mgrs`, `mds`, and `rgw` services. Red Hat recommends deploying services using placement specifications. You need to know where and how many daemons have to be deployed to deploy a service. Placement specifications can either be passed as command line arguments or as a service specification in a `yaml` file.

There are two ways of deploying the services using the placement specification:

- Using the placement specification directly in the command line interface. For example, if you want to deploy three monitors on the hosts, running the following command deploys three monitors on `host01`, `host02`, and `host03`.

  Example

```
[ceph: root@host01 /]# ceph orch apply mon --placement="3 host01 host02 host03"
```

- Using the placement specification in the YAML file. For example, if you want to deploy `node-exporter` on all the hosts, then you can specify the following in the `yaml` file.

  Example

```
service_type: node-exporter
placement:
host_pattern: '*'
extra_entrypoint_args:
    - "--collector.textfile.directory=/var/lib/node_exporter/textfile_collector2"
```

Deploying the Ceph daemons using the command line interface

Using the Ceph Orchestrator, you can deploy the daemons such as Ceph Manager, Ceph Monitors, Ceph OSDs, monitoring stack, and others using the `ceph orch` command. Placement specification is passed as `--placement` argument with the Orchestrator commands.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Hosts are added to the storage cluster.

Procedures

1. Log into the `cephadm` shell:

   Example

```
[root@host01 ~]# cephadm shell
```

2. Use one of the following methods to deploy the daemons on the hosts:
   - **Method 1**: Specify the number of daemons and the host names:

     Syntax

```
ceph orch apply SERVICE_NAME --placement="NUMBER_OF_DAEMONS HOST_NAME_1
HOST_NAME_2 HOST_NAME_3"
```

Example

```
[ceph: root@host01 /]# ceph orch apply mon --placement="3 host01 host02 host03"
```

- **Method 2**: Add the labels to the hosts and then deploy the daemons using the labels:
  - Add the labels to the hosts:

    Syntax

    ```
    ceph orch host label add HOSTNAME_1 LABEL
    ```

    Example

    ```
    [ceph: root@host01 /]# ceph orch host label add host01 mon
    ```

  - Deploy the daemons with labels:

    Syntax

    ```
    ceph orch apply DAEMON_NAME label:LABEL
    ```

    Example

    ```
    ceph orch apply mon label:mon
    ```

- **Method 3**: Add the labels to the hosts and deploy using the `--placement` argument:
  - Add the labels to the hosts:

    Syntax

    ```
    ceph orch host label add HOSTNAME_1 LABEL
    ```

    Example

    ```
    [ceph: root@host01 /]# ceph orch host label add host01 mon
    ```

  - Deploy the daemons using the label placement specification:

    Syntax

    ```
    ceph orch apply DAEMON_NAME --placement="label:LABEL"
    ```

    Example

    ```
    ceph orch apply mon --placement="label:mon"
    ```

Verification

- List the service:

  Example

  ```
  [ceph: root@host01 /]# ceph orch ls
  ```

- List the hosts, daemons, and processes:

  Syntax

  ```
  ceph orch ps --daemon_type=DAEMON_NAME
  ceph orch ps --service_name=SERVICE_NAME
  ```

Example

```
[ceph: root@host01 /]# ceph orch ps --daemon_type=mon
[ceph: root@host01 /]# ceph orch ps --service_name=mon
```

Reference

- Adding hosts

Deploying the Ceph daemons on a subset of hosts using the command line interface

You can use the `--placement` option to deploy daemons on a subset of hosts. You can specify the number of daemons in the placement specification with the name of the hosts to deploy the daemons.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Hosts are added to the cluster.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. List the hosts on which you want to deploy the Ceph daemons:

   Example

   ```
   [ceph: root@host01 /]# ceph orch host ls
   ```

3. Deploy the daemons:

   Syntax

   ```
   ceph orch apply SERVICE_NAME --placement="NUMBER_OF_DAEMONS HOST_NAME_1 _HOST_NAME_2
   HOST_NAME_3"
   ```

   Example

   ```
   ceph orch apply mgr --placement="2 host01 host02 host03"
   ```

   In this example, the `mgr` daemons are deployed only on two hosts.

Verification

- List the hosts:

  Example

  ```
  [ceph: root@host01 /]# ceph orch host ls
  ```

Reference

- *Listing hosts*

Service specification of the Ceph Orchestrator

A service specification is a data structure to specify the service attributes and configuration settings that is used to deploy the Ceph service.

The following is an example of the multi-document YAML file, `cluster.yaml`, for specifying service specifications:

Example

```
service_type: mon
placement:
  host_pattern: "mon*"
---
service_type: mgr
placement:
  host_pattern: "mgr*"
---
service_type: osd
service_id: default_drive_group
placement:
  host_pattern: "osd*"
data_devices:
  all: true
```

The following list are the parameters where the properties of a service specification are defined as follows:

- `service_type`: The type of service:
  - Ceph services like mon, crash, mds, mgr, osd, rbd, or rbd-mirror.
  - Ceph gateway like rgw.
  - Monitoring stack like Alertmanager, Prometheus, Grafana or Node-exporter.
  - Container for custom containers.
- `service_id`: A unique name of the service.
- `placement`: This is used to define where and how to deploy the daemons.
- `unmanaged`: If set to `true`, the Orchestrator will neither deploy nor remove any daemon associated with this service.

Stateless service of Orchestrators

A stateless service is a service that does not need information of the state to be available. For example, to start an `rgw` service, additional information is not needed to start or run the service. The `rgw` service does not create information about this state in order to provide the functionality. Regardless of when the `rgw` service starts, the state is the same.

Disabling automatic management of daemons

You can mark the `cephadm` services as `managed` or `unmanaged` without having to edit and re-apply the service specification.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Root-level access to all the nodes.

Procedure

1. Set `unmanaged` to `True` for service mon by using this command:

   ```
   [root@host01 ~]# ceph orch set-unmanaged mon
   ```

2. Set `unmanaged` to `False` for service mon by using this command:

   ```
   [root@host01 ~]# ceph orch set-managed mon
   ```

3. Set service `osd.all-available-devices` as unmanaged:

   ```
   [root@host01 ~]# ceph orch set-managed osd.all-available-devices
   ```

Deploying the Ceph daemons using the service specification

Using the Ceph Orchestrator, you can deploy daemons such as Ceph Manager, Ceph Monitors, Ceph OSDs, monitoring stack, and others using the service specification in a YAML file.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to all the nodes.

Procedure

1. Create the `yaml` file:

   Example

   ```
   [root@host01 ~]# touch mon.yaml
   ```

2. This file can be configured in two different ways:
   - Edit the file to include the host details in placement specification:

     Syntax

     ```
     service_type: SERVICE_NAME
     placement:
       hosts:
         - HOST_NAME_1
         - HOST_NAME_2
     ```

     Example

     ```
     service_type: mon
     placement:
       hosts:
         - host01
         - host02
         - host03
     ```

   - Edit the file to include the label details in placement specification:

     Syntax

     ```
     service_type: SERVICE_NAME
     placement:
       label: "LABEL_1"
     ```

     Example

     ```
     service_type: mon
     placement:
       label: "mon"
     ```

3. Optional: Use extra container arguments in the service specification files such as CPUs, CA certificates, and other files while deploying services. Use extra arguments to enable additional metrics in node-exporter deployed by `cephadm`.

   Example

   ```
   extra_container_args:
        - "-v"
        - "/etc/pki/ca-trust/extracted:/etc/pki/ca-trust/extracted:ro"
        - "--security-opt"
        - "label=disable"
        - "cpus=2"
        - "--collector.textfile.directory=/var/lib/node_exporter/textfile_collector2"
   ```

4. Mount the YAML file under a directory in the container:

Example

```
[root@host01 ~]# cephadm shell --mount mon.yaml:/var/lib/ceph/mon/mon.yaml
```

5. Navigate to the directory:

   Example

   ```
   [ceph: root@host01 /]# cd /var/lib/ceph/mon/
   ```

6. Deploy the Ceph daemons using service specification:

   Syntax

   ```
   ceph orch apply -i FILE_NAME.yaml
   ```

   Example

   ```
   [ceph: root@host01 mon]# ceph orch apply -i mon.yaml
   ```

Verification

- List the service:

  Example

  ```
  [ceph: root@host01 /]# ceph orch ls
  ```

- List the hosts, daemons, and processes:

  Syntax

  ```
  ceph orch ps --daemon_type=DAEMON_NAME
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph orch ps --daemon_type=mon
  ```

Reference

- *Listing hosts*

Managing hosts

As a storage administrator, you can use the Ceph Orchestrator with `cephadm` in the backend to add, list, and remove hosts in an existing Red Hat Ceph Storage cluster.

You can also add labels to hosts. Labels are free-form and have no specific meanings. Each host can have multiple labels. For example, apply the `mon` label to all hosts that have monitor daemons deployed, `mgr` for all hosts with manager daemons deployed, `rgw` for Ceph Object Gateways, and so on.

Labeling all the hosts in the storage cluster helps to simplify system management tasks by allowing you to quickly identify the daemons running on each host. In addition, you can use the Ceph Orchestrator or a YAML file to deploy or remove daemons on hosts that have specific host labels.

Adding hosts

You can use the Ceph Orchestrator with `cephadm` in the backend to add hosts to an existing Red Hat Ceph Storage.

Prerequisites

- A running Red Hat Ceph Storage cluster.

- Root-level access to all nodes in the storage cluster.
- Register the nodes to the CDN and attach subscriptions.
- Ansible user with sudo and passwordless `ssh` access to all nodes in the storage cluster.
- The IP addresses of the new hosts should be updated in the `/etc/hosts` file.

Procedure

1. From the Ceph administration node, log into the `cephadm` shell.
   For example,

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Extract the cluster's public SSH keys to a folder.

   ```
   ceph cephadm get-pub-key > ~/PATH
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph cephadm get-pub-key > ~/ceph.pub
   ```

3. Copy Ceph cluster's public SSH keys to the root user's `authorized_keys` file on the new host.

   ```
   ssh-copy-id -f -i ~/PATH root@HOST_NAME_2
   ```

   For example,

   ```
   [ceph: root@host01 /]# ssh-copy-id -f -i ~/ceph.pub root@host02
   ```

4. From the Ansible administration node, add the new host to the Ansible inventory file. The default location for the file is `/usr/share/cephadm-ansible/hosts`. The following example shows the structure of a typical inventory file:

   ```
   host01
   host02
   host03

   [admin]
   host00
   ```

   > **Note:** If you have previously added the new host to the Ansible inventory file and run the preflight playbook on the host, skip to step 6.

5. Run the preflight playbook with the `--limit` option:

   ```
   ansible-playbook -i INVENTORY_FILE cephadm-preflight.yml --extra-vars
   "ceph_origin=rhcs" --limit NEWHOST
   ```

   For example,

   ```
   [ansible@admin cephadm-ansible]$ ansible-playbook -i hosts cephadm-preflight.yml --
   extra-vars "ceph_origin=rhcs"
    --limit host02
   ```

   The preflight playbook installs `podman`, `lvm2`, `chrony`, and `cephadm` on the new host. After installation is complete, `cephadm` resides in the `/usr/sbin/` directory.

6. From the Ceph administration node, log into the `cephadm` shell.
   For example,

   ```
   [root@host01 ~]# cephadm shell
   ```

7. Use the `cephadm` orchestrator to add hosts to the storage cluster.

```
ceph orch host add HOST_NAME IP_ADDRESS_OF_HOST [--label=LABEL_NAME_1,LABEL_NAME_2]
```

For example,

```
[ceph: root@host01 /]# ceph orch host add host02 10.10.128.70 --labels=mon,mgr
```

Verification

Verify that the hosts were added successfully, by running the **ceph orch host ls** command.

For example,

```
[ceph: root@host01 /]# ceph orch host ls
```

Reference

For more information, see the following:

- Listing hosts
- “Running the preflight playbook” on page 191
- “Registering the Red Hat Ceph Storage nodes” on page 185
- “Creating an Ansible user with sudo access” on page 188

Adding multiple hosts

You can use the Ceph Orchestrator to add multiple hosts to a Red Hat Ceph Storage cluster at the same time using the service specification in YAML file format.

Prerequisites

- A running Red Hat Ceph Storage.

Procedure

1. Create the `hosts.yaml` file:

   Example

   ```
   [root@host01 ~]# touch hosts.yaml
   ```

2. Edit the `hosts.yaml` file to include the following details:

   Example

   ```
   service_type: host
   addr: host01
   hostname: host01
   labels:
   - mon
   - osd
   - mgr
   ---
   service_type: host
   addr: host02
   hostname: host02
   labels:
   - mon
   - osd
   - mgr
   ---
   service_type: host
   addr: host03
   hostname: host03
   labels:
   - mon
   - osd
   ```

3. Mount the YAML file under a directory in the container:

   Example

   ```
   [root@host01 ~]# cephadm shell --mount hosts.yaml:/var/lib/ceph/hosts.yaml
   ```

4. Navigate to the directory:

   Example

   ```
   [ceph: root@host01 /]# cd /var/lib/ceph/
   ```

5. Deploy the hosts using service specification:

   Syntax

   ```
   ceph orch apply -i FILE_NAME.yaml
   ```

   Example

   ```
   [ceph: root@host01 hosts]# ceph orch apply -i hosts.yaml
   ```

Verification

- List the hosts:

  Example

  ```
  [ceph: root@host01 /]# ceph orch host ls
  ```

Reference

- *Listing hosts*.

Listing hosts

> **Note:** The *STATUS* of the hosts is blank, in the output of the `ceph orch host ls` command.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Hosts are added to the cluster.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. List the hosts of the cluster:

   Example

   ```
   [ceph: root@host01 /]# ceph orch host ls
   ```

   You will see that the *STATUS* of the hosts is blank which is expected.

Adding labels to a host

Use the Ceph Orchestrator to add a label to a host. Labels can be used to specify placement of daemons.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A storage cluster is installed and bootstrapped.
- Root-level access to all nodes in the storage cluster.
- Hosts are added to the storage cluster.

About this task

A few examples of labels are `mgr`, `mon`, and `osd` based on the service deployed on the hosts. Each host can have multiple labels.

The following labels have special meaning to `cephadm`.

> **Note:** Special labels begin with _ (underscore).

`_no_schedule`

This label prevents `cephadm` from scheduling or deploying daemons on the host. If it is added to an existing host that already contains Ceph daemons, it causes `cephadm` to move those daemons elsewhere, except OSDs which are not removed automatically. When a host is added with the `_no_schedule` label, no daemons are deployed on it. When the daemons are drained before the host is removed, the `_no_schedule` label is set on that host.

`_no_autotune_memory`

This label does not autotune memory on the host. It prevents the daemon memory from being tuned even when the **`osd_memory_target_autotune`** option or other similar options are enabled for one or more daemons on that host.

`_admin`

By default, the `_admin` label is applied to the bootstrapped host in the storage cluster and the `client.admin` key is set to be distributed to that host with the `ceph orch client-keyring [ls|set|rm]` function. Adding this label to additional hosts normally causes `cephadm` to deploy configuration and keyring files in the `/etc/ceph` directory.

Procedure

1. Launch the `cephadm` shell.
   For example,

   ```
   [root@host01 ~]# cephadm shell
   [ceph: root@host01 /]#
   ```

2. Add a label to a host.

   ```
   ceph orch host label add HOSTNAME LABEL
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch host label add host02 mon
   ```

   > **Note:** If your Red Hat Ceph Storage deployment comprises multiple CRUSH bucket types, such as `chassis` or `rack`, also known as failure domains, that are higher than `host`, assign the `mon` label to no more than 2 hosts per failure domain. Limiting the `mon` label spreads the daemons across failure domains, ensuring Monitor quorum when entire failure domain is lost.

What to do next

Verify that the label is added to the host, by using the **`ceph orch host ls`** command.

Removing labels from a host

Use the Ceph Orchestrator to remove a label from a host.

Before you begin

- A Red Hat Ceph Storage cluster is installed and bootstrapped.
- Root-level access to all nodes in the storage cluster.
- Hosts are added to the storage cluster.

Procedure

1. Log into the `cephadm` shell.

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Remove the label.

   ```
   ceph orch host label rm HOST_NAME LABEL_NAME
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch host label rm host02 mon
   ```

What to do next

Verify that the label is moved from the host, by using the **`ceph orch host ls`** command.

Removing hosts

Remove Ceph cluster hosts with the Ceph Orchestrators.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to all the nodes.
- Hosts are added to the storage cluster.
- All the services are deployed.
- `cephadm` is deployed on the nodes where the services have to be removed.

About this task

All daemons are removed with the **`drain`** option which adds the `_no_schedule` label to ensure that you cannot deploy any daemons or a cluster till the operation is complete.

> **Important:** If you are removing the bootstrap host, be sure to copy the admin keyring and the configuration file to another host in the storage cluster before you remove the host.

For more information, see the following operations information:

- Adding hosts using the Ceph Orchestrator
- Listing hosts using the Ceph Orchestrator

Procedure

1. Log into the `cephadm` shell.

   ```
   cephadm shell
   ```

   For example,

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Get the host details.

   ```
   ceph orch host ls
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch host ls
   ```

3. Drain all the daemons from the host.

   ```
   ceph orch host drain HOSTNAME
   ```

   > **Note:** The ceph orch host drain command supports the **`--zap-osd-devices`** flag. When you set this flag, `cephadm` zaps the devices of the OSDs it removes during the drain process.

   For example,

   ```
   [ceph: root@host01 /]# ceph orch host drain host02
   ```

   The `_no_schedule` label is automatically applied to the host which blocks deployment.

4. Check the status of OSD removal.

   ```
   ceph orch osd rm status
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch osd rm status
   ```

   When no placement groups (PGs) remain on the OSD, the OSD is decommissioned and removed from the storage cluster.

5. Check if all the daemons are removed from the storage cluster.

   ```
   ceph orch ps HOSTNAME
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch ps host02
   ```

6. Remove the host from the orchestrator and the CRUSH map.
   Use the **`--rm-crush-entry`** flag to remove the host from both the orchestrator and the CRUSH map in a single operation. This ensures the host is completely removed from the cluster and does not appear in the `ceph osd tree` output.

   ```
   ceph orch host rm HOSTNAME --rm-crush-entry
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch host rm host02 --rm-crush-entry
   Removed host 'host02'
   ```

   Alternative method: If you have already removed the host without the **`--rm-crush-entry`** flag, you can manually remove the host from the CRUSH map.

```
ceph osd crush remove HOSTNAME
```

For example,

```
[ceph: root@host01 /]# ceph osd crush remove host02
removed item id -5 name 'host02' from crush map
```

7. Verify that the host has been removed from both the orchestrator and the CRUSH map.
   a. Check that the host is not listed in the orchestrator.

      ```
      ceph orch host ls
      ```

      For example,

      ```
      [ceph: root@host01 /]# ceph orch host ls
      ```

   b. Verify that the host does not appear in the OSD tree.

      ```
      ceph osd tree
      ```

      For example,

      ```
      [ceph: root@host01 /]# ceph osd tree
      ```

      The host should not appear in either output.

Placing hosts in the maintenance mode

You can use the Ceph Orchestrator to place the hosts in and out of the maintenance mode.

The `ceph orch host maintenance enter` command stops the `systemd target` which causes all the Ceph daemons to stop on the host. Similarly, the `ceph orch host maintenance exit` command restarts the `systemd target` and the Ceph daemons restart on their own.

The orchestrator adopts the following workflow when the host is placed in maintenance:

1. Confirms the removal of hosts does not impact data availability by running the `orch host ok-to-stop` command.
2. If the host has Ceph OSD daemons, it applies `noout` to the host subtree to prevent data migration from triggering during the planned maintenance slot.
3. Stops the Ceph target, thereby, stopping all the daemons.
4. Disables the `ceph target` on the host, to prevent a reboot from automatically starting Ceph services.

Exiting maintenance reverses this workflow sequence.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to all the nodes.
- Hosts added to the cluster.

Procedure

1. Log into the `cephadm` shell.

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. You can either place the host in maintenance mode or place it out of the maintenance mode:
   - Place the host in maintenance mode.

Syntax

```
ceph orch host maintenance enter HOST_NAME [--force]
```

Example

```
[ceph: root@host01 /]# ceph orch host maintenance enter host02 --force
```

The `--force` flag allows the user to bypass warnings, but not alerts.

- Place the host out of the maintenance mode.

  Syntax

  ```
  ceph orch host maintenance exit HOST_NAME
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph orch host maintenance exit host02
  ```

Verification

- List the hosts.

  Example

  ```
  [ceph: root@host01 /]# ceph orch host ls
  ```

Managing monitors

As a storage administrator, you can deploy additional monitors using placement specification, add monitors using service specification, add monitors to a subnet configuration, and add monitors to specific hosts. Apart from this, you can remove the monitors.

By default, a typical Red Hat Ceph Storage has three or five monitor daemons deployed on different hosts.

Red Hat recommends deploying five monitors if there are five or more nodes in a cluster.

Ceph deploys monitor daemons automatically as the cluster grows, and scales back monitor daemons automatically as the cluster shrinks. The smooth execution of this automatic growing and shrinking depends upon proper subnet configuration.

If your monitor nodes or your entire cluster are located on a single subnet, then `cephadm` automatically adds up to five monitor daemons as you add new hosts to the cluster. `cephadm` automatically configures the monitor daemons on the new hosts. The new hosts reside on the same subnet as the bootstrapped host in the storage cluster.

`cephadm` can also deploy and scale monitors to correspond to changes in the size of the storage cluster.

Ceph Monitors

Ceph Monitors are lightweight processes that maintain a primary copy of the storage cluster map. All Ceph clients contact a Ceph monitor and retrieve the current copy of the storage cluster map, enabling clients to bind to a pool and read and write data.

Ceph Monitors use a variation of the Paxos protocol to establish consensus about maps and other critical information across the storage cluster. Due to the nature of Paxos, Ceph requires a majority of monitors running to establish a quorum, thus establishing consensus.

> **Important:** Red Hat requires at least three monitors on separate hosts to receive support for a production cluster.

Red Hat recommends deploying an odd number of monitors. An odd number of Ceph Monitors has a higher resilience to failures than an even number of monitors. For example, to maintain a quorum on a two-monitor

deployment, Ceph cannot tolerate any failures; with three monitors, one failure; with four monitors, one failure; with five monitors, two failures. This is why an odd number is advisable. Summarizing, Ceph needs a majority of monitors to be running and to be able to communicate with each other, two out of three, three out of four, and so on.

For an initial deployment of a multi-node Ceph storage cluster, Red Hat requires three monitors, increasing the number two at a time if a valid need for more than three monitors exists.

Since Ceph Monitors are lightweight, it is possible to run them on the same host as OpenStack nodes. However, Red Hat recommends running monitors on separate hosts.

> **Important:** Red Hat ONLY supports collocating Ceph services in containerized environments.

When you remove monitors from a storage cluster, consider that Ceph Monitors use the Paxos protocol to establish a consensus about the primary storage cluster map. You must have a sufficient number of Ceph Monitors to establish a quorum.

Reference

- *Supported configurations* Knowledgebase article

Configuring monitor election strategy

The monitor election strategy identifies the net splits and handles failures. You can configure the election monitor strategy in three different modes: classic, disallow, and connectivity.

The monitor election strategy identifies the net splits and handles failures. You can configure the election monitor strategy in three different modes:

1. `classic` - This is the default mode in which the lowest ranked monitor is voted based on the elector module between the two sites.
2. `disallow` - This mode lets you mark monitors as disallowed, in which case they will participate in the quorum and serve clients, but cannot be an elected leader. This lets you add monitors to a list of disallowed leaders. If a monitor is in the disallowed list, it will always defer to another monitor.
3. `connectivity` - This mode is mainly used to resolve network discrepancies. It evaluates connection scores, based on pings that check liveness, provided by each monitor for its peers and elects the most connected and reliable monitor to be the leader. This mode is designed to handle net splits, which may happen if your cluster is stretched across multiple data centers or otherwise susceptible. This mode incorporates connection score ratings and elects the monitor with the best score. If a specific monitor is desired to be the leader, configure the election strategy so that the specific monitor is the first monitor in the list with a rank of `0`.

Red Hat recommends you to stay in the `classic` mode unless you require features in the other modes.

Before constructing the cluster, change the `election_strategy` to `classic`, `disallow`, or `connectivity` in the following command:

Syntax

```
ceph mon set election_strategy {classic|disallow|connectivity}
```

Deploying the Ceph monitor daemons using the command line interface

The Ceph Orchestrator deploys one monitor daemon by default. You can deploy additional monitor daemons by using the `placement` specification in the command line interface. To deploy a different number of monitor daemons, specify a different number. If you do not specify the hosts where the monitor daemons should be deployed, the Ceph Orchestrator randomly selects the hosts and deploys the monitor daemons to them.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Hosts are added to the cluster.

Procedure

1. Log into the `cephadm` shell.

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. There are four different ways of deploying Ceph monitor daemons:

Method 1

- Use placement specification to deploy monitors on hosts:

  > **Note:** Red Hat recommends that you use the `--placement` option to deploy on specific hosts.

  Syntax

  ```
  ceph orch apply mon --placement="HOST_NAME_1 HOST_NAME_2 HOST_NAME_3"
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph orch apply mon --placement="host01 host02 host03"
  ```

  > **Note:** Be sure to include the bootstrap node as the first node in the command.

  > **Important:**
  >
  > - Do not add the monitors individually as `ceph orch apply mon` supersedes and will not add the monitors to all the hosts. For example, if you run the following commands, then the first command creates a monitor on `host01`. Then the second command supersedes the monitor on host1 and creates a monitor on `host02`. Then the third command supersedes the monitor on `host02` and creates a monitor on `host03`. Eventually, there is a monitor only on the third host.
>
>   ```
>   # ceph orch apply mon host01
>   # ceph orch apply mon host02
>   # ceph orch apply mon host03
>   ```

Method 2

- Use placement specification to deploy specific number of monitors on specific hosts with labels:
  - Add the labels to the hosts:

    Syntax

    ```
    ceph orch host label add HOSTNAME_1 LABEL
    ```

    Example

    ```
    [ceph: root@host01 /]# ceph orch host label add host01 mon
    ```

  - Deploy the daemons.

    Syntax

    ```
    ceph orch apply mon --placement="HOST_NAME_1:mon HOST_NAME_2:mon HOST_NAME_3:mon"
    ```

    Example

```
[ceph: root@host01 /]# ceph orch apply mon --placement="host01:mon host02:mon
host03:mon"
```

Method 3

- Use placement specification to deploy specific number of monitors on specific hosts:

  Syntax

  ```
  ceph orch apply mon --placement="NUMBER_OF_DAEMONS HOST_NAME_1 HOST_NAME_2 HOST_NAME_3"
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph orch apply mon --placement="3 host01 host02 host03"
  ```

Method 4

- Deploy monitor daemons randomly on the hosts in the storage cluster:

  Syntax

  ```
  ceph orch apply mon NUMBER_OF_DAEMONS
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph orch apply mon 3
  ```

Verification

- List the service.

  Example

  ```
  [ceph: root@host01 /]# ceph orch ls
  ```

- List the hosts, daemons, and processes.

  Syntax

  ```
  ceph orch ps --daemon_type=DAEMON_NAME
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph orch ps --daemon_type=mon
  ```

Deploying the Ceph monitor daemons using the service specification

The Ceph Orchestrator deploys one monitor daemon by default. You can deploy additional monitor daemons by using the service specification, like a YAML format file.

Prerequisites

- A running Red Hat Ceph Storage .
- Hosts are added to the cluster.

Procedure

1. Create the `mon.yaml` file:

   Example

   ```
   [root@host01 ~]# touch mon.yaml
   ```

2. Edit the `mon.yaml` file to include the following details:

   Syntax

   ```
   service_type: mon
   placement:
     hosts:
       - HOST_NAME_1
       - HOST_NAME_2
   ```

   Example

   ```
   service_type: mon
   placement:
     hosts:
       - host01
       - host02
   ```

3. Mount the YAML file under a directory in the container:

   Example

   ```
   [root@host01 ~]# cephadm shell --mount mon.yaml:/var/lib/ceph/mon/mon.yaml
   ```

4. Navigate to the directory:

   Example

   ```
   [ceph: root@host01 /]# cd /var/lib/ceph/mon/
   ```

5. Deploy the monitor daemons:

   Syntax

   ```
   ceph orch apply -i FILE_NAME.yaml
   ```

   Example

   ```
   [ceph: root@host01 mon]# ceph orch apply -i mon.yaml
   ```

Verification

- List the service:

  Example

  ```
  [ceph: root@host01 /]# ceph orch ls
  ```

- List the hosts, daemons, and processes:

  Syntax

  ```
  ceph orch ps --daemon_type=DAEMON_NAME
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph orch ps --daemon_type=mon
  ```

Deploying the monitor daemons on specific network

The Ceph Orchestrator deploys one monitor daemon by default. You can explicitly specify the IP address or CIDR network for each monitor and control where each monitor is placed.

Prerequisites

- A running Red Hat Ceph Storage cluster.

- Hosts are added to the cluster.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Disable automated monitor deployment:

   Example

   ```
   [ceph: root@host01 /]# ceph orch apply mon --unmanaged
   ```

3. Deploy monitors on hosts on specific network:

   Syntax

   ```
   ceph orch daemon add mon _HOST_NAME_1_:_IP_OR_NETWORK_
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph orch daemon add mon host03:10.1.2.123
   ```

Verification

- List the service:

  Example

  ```
  [ceph: root@host01 /]# ceph orch ls
  ```

- List the hosts, daemons, and processes:

  Syntax

  ```
  ceph orch ps --daemon_type=DAEMON_NAME
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph orch ps --daemon_type=mon
  ```

Managing the Ceph Monitor service with Red Hat OpenStack Platform

Understand how replacing Controller nodes affects the Ceph Monitor service in director-deployed Red Hat Ceph Storage environments.

In director-deployed Red Hat Ceph Storage environments, it is important to understand the impact of replacing Controller nodes. The Ceph Monitor service runs on the Controller nodes and typically uses IP addresses assigned from the Storage network.

The Ceph Monitor service IP addresses are associated with virtual machine instances that use Red Hat Ceph Storage. These IP addresses are not dynamically updated if a Ceph Monitor service IP address changes during replacement of a Controller node.

As a result, replacing Controller nodes can lead to a storage outage, particularly if multiple Controller nodes are replaced. In this situation, each affected virtual machine instance must be migrated, rebooted, or shelved and unshelved to resolve the IP address change and restore storage access.

To avoid storage outages during Controller node replacement, reuse the IP addresses of the removed Ceph Monitor service instances instead of assigning new IP addresses.

For example,

```
- name: Controller
  count: 3
  defaults:
    network_config:
      template: /home/stack/templates/nic-config/myController.j2
      default_route_network:
      - external
  instances:
  - hostname: overcloud-controller-0
    name: node00
    networks:
    - network: ctlplane
      vif: true
    - network: external
      subnet: external_subnet
    - network: internal_api
      subnet: internal_api_subnet01
      fixed_ip: 172.21.11.100
    - network: storage
      subnet: storage_subnet01
    - network: storage_mgmt
      subnet: storage_mgmt_subnet01
    - network: tenant
      subnet: tenant_subnet01
```

Note: To find the current Ceph Monitor service IP addresses on a Controller node, run the following command:

```
sudo cephadm shell -- ceph mon stat
```

For more information, see the Red Hat OpenStack Platform documentation.

Removing the monitor daemons

To remove the monitor daemons from the host, you can just redeploy the monitor daemons on other hosts.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Hosts are added to the cluster.
- At least one monitor daemon deployed on the hosts.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Run the `ceph orch apply` command to deploy the required monitor daemons:

   Syntax

   ```
   ceph orch apply mon “NUMBER_OF_DAEMONS HOST_NAME_1 HOST_NAME_3”
   ```

   If you want to remove monitor daemons from `host02`, then you can redeploy the monitors on other hosts.

   Example

   ```
   [ceph: root@host01 /]# ceph orch apply mon “2 host01 host03”
   ```

Verification

- List the hosts,daemons, and processes:

Syntax

```
ceph orch ps --daemon_type=DAEMON_NAME
```

Example

```
[ceph: root@host01 /]# ceph orch ps --daemon_type=mon
```

Reference

- *Deploying the Ceph monitor daemons using the command line interface*
- *Deploying the Ceph monitor daemons using the service specification*

Removing a Ceph Monitor from an unhealthy storage cluster

You can remove a `ceph-mon` daemon from an unhealthy storage cluster. An unhealthy storage cluster is one that has placement groups persistently in not `active + clean` state.

Prerequisites

- Root-level access to the Ceph Monitor node.
- At least one running Ceph Monitor node.

Procedure

1. Identify a surviving monitor and log into the host:

   Syntax

   ```
   ssh root@MONITOR_ID
   ```

   Example

   ```
   [root@admin ~]# ssh root@host00
   ```

2. Log in to each Ceph Monitor host and stop all the Ceph Monitors:

   Syntax

   ```
   cephadm unit --name DAEMON_NAME.HOSTNAME stop
   ```

   Example

   ```
   [root@host00 ~]# cephadm unit --name mon.host00 stop
   ```

3. Set up the environment suitable for extended daemon maintenance and to run the daemon interactively:

   Syntax

   ```
   cephadm shell --name DAEMON_NAME.HOSTNAME
   ```

   Example

   ```
   [root@host00 ~]# cephadm shell --name mon.host00
   ```

4. Extract a copy of the `monmap` file:

   Syntax

   ```
   ceph-mon -i HOSTNAME --extract-monmap TEMP_PATH
   ```

   Example

```
[ceph: root@host00 /]# ceph-mon -i host01 --extract-monmap /tmp/monmap
2022-01-05T11:13:24.440+0000 7f7603bd1700 -1 wrote monmap to /tmp/monmap
```

5. Remove the non-surviving Ceph Monitor(s):

   Syntax

   ```
   monmaptool TEMPORARY_PATH --rm HOSTNAME
   ```

   Example

   ```
   [ceph: root@host00 /]# monmaptool /tmp/monmap --rm host01
   ```

6. Inject the surviving monitor map with the removed monitor(s) into the surviving Ceph Monitor:

   Syntax

   ```
   ceph-mon -i HOSTNAME --inject-monmap TEMP_PATH
   ```

   Example

   ```
   [ceph: root@host00 /]# ceph-mon -i host00 --inject-monmap /tmp/monmap
   ```

7. Start only the surviving monitors:

   Syntax

   ```
   cephadm unit --name DAEMON_NAME.HOSTNAME start
   ```

   Example

   ```
   [root@host00 ~]# cephadm unit --name mon.host00 start
   ```

8. Verify the monitors form a quorum:

   Example

   ```
   [ceph: root@host00 /]# ceph -s
   ```

9. Optional: Archive the removed Ceph Monitor's data directory in `/var/lib/ceph/CLUSTER_FSID/mon.HOSTNAME` directory.

Managing manager daemons

As a storage administrator, you can use the Ceph Orchestrator to deploy additional manager daemons. `cephadm` automatically installs a manager daemon on the bootstrap node during the bootstrapping process.

In general, you should set up a Ceph Manager on each of the hosts running the Ceph Monitor daemon to achieve same level of availability.

By default, whichever `ceph-mgr` instance comes up first is made active by the Ceph Monitors, and others are standby managers. There is no requirement that there should be a quorum among the `ceph-mgr` daemons.

If the active daemon fails to send a beacon to the monitors for more than the `mon mgr beacon grace`, then it is replaced by a standby.

If you want to pre-empt failover, you can explicitly mark a `ceph-mgr` daemon as failed with **`ceph mgr fail MANAGER_NAME`** command.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to all the nodes.

– Hosts are added to the cluster.

Deploying the manager daemons

The Ceph Orchestrator deploys two Manager daemons by default. You can deploy additional manager daemons using the `placement` specification in the command line interface. To deploy a different number of Manager daemons, specify a different number. If you do not specify the hosts where the Manager daemons should be deployed, the Ceph Orchestrator randomly selects the hosts and deploys the Manager daemons to them.

Prerequisites

> **Note:** Ensure your deployment has at least three Ceph Managers in each deployment.

– A running Red Hat Ceph Storage cluster.

– Hosts are added to the cluster.

Procedure

1. Log into the `cephadm` shell.

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. You can deploy manager daemons in two different ways:

Method 1

– Deploy manager daemons using placement specification on specific set of hosts:

> **Note:** Use the `--placement` option to deploy on specific hosts.

Syntax

```
ceph orch apply mgr --placement="HOST_NAME_1 HOST_NAME_2 HOST_NAME_3"
```

Example

```
[ceph: root@host01 /]# ceph orch apply mgr --placement="host01 host02 host03"
```

Method 2

– Deploy manager daemons randomly on the hosts in the storage cluster.

Syntax

```
ceph orch apply mgr NUMBER_OF_DAEMONS
```

Example

```
[ceph: root@host01 /]# ceph orch apply mgr 3
```

Verification

– List the service.

Example

```
[ceph: root@host01 /]# ceph orch ls
```

- List the hosts, daemons, and processes.

  Syntax

  ```
  ceph orch ps --daemon_type=DAEMON_NAME
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph orch ps --daemon_type=mgr
  ```

Removing the manager daemons

To remove the manager daemons from the host, you can just redeploy the daemons on other hosts.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to all the nodes.
- Hosts are added to the cluster.
- At least one manager daemon deployed on the hosts.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Run the `ceph orch apply` command to redeploy the required manager daemons:

   Syntax

   ```
   ceph orch apply mgr "NUMBER_OF_DAEMONS HOST_NAME_1 HOST_NAME_3"
   ```

   If you want to remove manager daemons from `host02`, then you can redeploy the manager daemons on other hosts.

   Example

   ```
   [ceph: root@host01 /]# ceph orch apply mgr "2 host01 host03"
   ```

Verification

- List the hosts,daemons, and processes:

  Syntax

  ```
  ceph orch ps --daemon_type=DAEMON_NAME
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph orch ps --daemon_type=mgr
  ```

Reference

- *Deploying the manager daemons*

Using Ceph Manager modules

Use the **`ceph mgr module ls`** command to see the available modules and the modules that are presently enabled.

Enable or disable modules with **ceph mgr module enable MODULE** command or **ceph mgr module disable MODULE** command respectively.

If a module is enabled, then the active `ceph-mgr` daemon loads and executes it. In the case of modules that provide a service, such as an HTTP server, the module might publish its address when it is loaded. To see the addresses of such modules, run the `ceph mgr services` command.

Some modules might also implement a special standby mode which runs on standby `ceph-mgr` daemon as well as the active daemon. This enables modules that provide services to redirect their clients to the active daemon, if the client tries to connect to a standby.

Following is an example to enable the dashboard module:

```
[ceph: root@host01 /]# ceph mgr module enable dashboard

[ceph: root@host01 /]# ceph mgr module ls

        MODULE
        balancer              on (always on)
        crash                 on (always on)
        devicehealth          on (always on)
        orchestrator          on (always on)
        pg_autoscaler         on (always on)
        progress              on (always on)
        rbd_support           on (always on)
        status                on (always on)
        telemetry             on (always on)
        volumes               on (always on)
        cephadm               on
        dashboard             on
        iostat                on
        prometheus            on
        restful               on
        alerts                -
        diskprediction_local  -
        influx                -
        insights              -
        k8sevents             -
        localpool             -
        mds_autoscaler        -
        mirroring             -
        osd_perf_query        -
        osd_support           -
        rgw                   -
        rook                  -
        selftest              -
        snap_schedule         -
        stats                 -
        telegraf              -
        test_orchestrator     -
        zabbix                -

[ceph: root@host01 /]# ceph mgr services
{
        "dashboard": "http://myserver.com:7789/",
        "restful": "https://myserver.com:8789/"
}
```

The first time the cluster starts, it uses the `mgr_initial_modules` setting to override which modules to enable. However, this setting is ignored through the rest of the lifetime of the cluster: only use it for bootstrapping. For example, before starting your monitor daemons for the first time, you might add a section like this to your `ceph.conf` file:

```
[mon]
    mgr initial modules = dashboard balancer
```

Where a module implements comment line hooks, the commands are accessible as ordinary Ceph commands and Ceph automatically incorporates module commands into the standard CLI interface and route them appropriately to the module:

```
[ceph: root@host01 /]# ceph <command | help>
```

Use the configuration parameters listed in “Configuration parameters” on page 570 with the **ceph** command.

Table 90: Configuration parameters			
Configuration	**Description**	**Type**	**Default**
`mgr module path`	Path to load modules from.	String	`"<library dir>/mgr"`
`mgr data`	Path to load daemon data (such as keyring)	String	`"/var/lib/ceph/mgr/$cluster-$id"`
`mgr tick period`	How many seconds between manager beacons to monitors, and other periodic checks.	Integer	`5`
`mon mgr beacon grace`	How long after last beacon should a manager be considered failed.	Integer	`30`

Using the Ceph Manager balancer module

The balancer is a module for Ceph Manager (`ceph-mgr`) that optimizes the placement of placement groups (PGs) across OSDs in order to achieve a balanced distribution, either automatically or in a supervised fashion.

> **Note:** The balancer module cannot be disabled but can be turned off to customize the configuration.

The following are the supported balancer modes:

crush-compat

The CRUSH compat mode uses the compat `weight-set` feature to manage an alternative set of weights for devices in the CRUSH hierarchy.
The normal weights should remain set to the size of the device to reflect the target amount of data that you want to store on the device. The balancer then optimizes the `weight-set` values, adjusting them up or down in small increments in order to achieve a distribution that matches the target distribution as closely as possible. Because PG placement is a pseudorandom process, there is a natural amount of variation in the placement; by optimizing the weights, the balancer counter-acts that natural variation.

This mode is fully backwards compatible with older clients. When an OSDMap and CRUSH map are shared with older clients, the balancer presents the optimized weights as the real weights.

The primary restriction of this mode is that the balancer cannot handle multiple CRUSH hierarchies with different placement rules if the subtrees of the hierarchy share any OSDs. Because this configuration makes managing space utilization on the shared OSDs difficult, it is generally not recommended. As such, this restriction is normally not an issue.

upmap

Use `upmap` to optimize PG placement and achieve uniform distribution across OSDs. In most cases, this distribution is nearly uniform, with each OSD hosting an equal number of PGs (+/-1 PG). In most cases, this distribution is "perfect", with an equal number of PGs on each OSD +/-1 PG, as they might not divide evenly.

1. Run the following command to check the PG distribution by device class:

   ```
   ceph osd df deviceclass
   ```

   If the `VAR` values for `hdd` or `ssd` are outside the range `[0.95 – 1.05]`, the balancer may be disabled or misconfigured.
   If any OSD shows a `REWEIGHT` value other than `1.000`, reset the OSD.

   ```
   ceph osd reweight osd.ID 1.000
   ```

2. Improve balancer effectiveness by ensuring that pools have enough PGs. If PGS per OSD is less than `100`, add PGs to the pools served by that device class.

When PG autoscaler is enabled, apply the following settings:

```
ceph config set global max_pg_per_osd 500
ceph config set global mon_target_pg_per_osd 300
ceph config set mgr/balancer/upmap_max_deviation 1
```

> **Note:** Using **`ceph config set mgr/balancer/upmap_max_deviation 1`** is useful when OSD sizes vary significantly.

3. Overlapping CRUSH roots can prevent proper balancing.
Verify the pool rules by using the following command:

```
ceph osd pool ls detail
```

4. After verifying the pool rules, inspect the CRUSH rule.

```
ceph osd crush rule dump | jq 'map(select(.rule_id == RULE_ID) | .steps)'
```

Each pool should use a rule that specifies a device class, for example, `default~ssd`. If any pool uses a rule without a device class, contact Red Hat Support for guidance.

read

The OSDMap can store explicit mappings for individual primary OSDs as exceptions to the normal CRUSH placement calculation. These **`pg-upmap-primary`** entries provide fine-grained control over primary PG mappings. This mode optimizes the placement of individual primary PGs in order to achieve balanced reads, or primary PGs, in a cluster. In `read` mode, upmap behavior is not exercised, so this mode is best for uses cases in which only read balancing is desired.

> **Important:** To allow use of this feature, you must tell the cluster that it only needs to support rRef or later clients with the following command:
>
> ```
> ceph osd set-require-min-compat-client reef
> ```
>
> This command fails if any pre-Reef clients or daemons are connected to the monitors. To work around this issue, use the `--yes-i-really-mean-it` flag:
>
> ```
> ceph osd set-require-min-compat-client reef --yes-i-really-mean-it
> ```
>
> You can check what client versions are in use with: the **`ceph features`** command. For example,
>
> ```
> [ceph: root@host01 /]# ceph features
> ```

upmap-read

This balancer mode combines optimization benefits of both `upmap` and `read` mode. Like in `read` mode, `upmap-read` makes use of **`pg-upmap-primary`**.
Use `upmap-read` for achieving the `upmap` mode's offering of balanced PG distribution as well as the `read` mode's offering of balanced reads.

> **Important:** To allow use of this feature, you must tell the cluster that it only needs to support Reef or later clients with the following command:
>
> ```
> ceph osd set-require-min-compat-client reef
> ```
>
> This command fails if any pre-Reef clients or daemons are connected to the monitors. To work around this issue, use the `--yes-i-really-mean-it` flag:

```
ceph osd set-require-min-compat-client reef --yes-i-really-mean-it
```

You can check what client versions are in use with: the **ceph features** command. For example,

```
[ceph: root@host01 /]# ceph features
```

Balancing Red Hat Ceph Storage cluster using the capacity balancer

Balance the cluster by using the capacity balancer module.

Before you begin

Before you begin, make sure that you have a running Red Hat Ceph Storage cluster.

Procedure

1. Check if the balancer module is enabled.

   ```
   ceph mgr module enable balancer
   ```

2. Turn on the balancer module.

   ```
   ceph balancer on
   ```

3. To change the mode use the following command. The default mode is `upmap`.

   ```
   ceph balancer mode upmap
   ```

4. Check the current status of the balancer.

   ```
   ceph balancer status
   ```

Automatic balancing

Use automatic balancing when turning on the balancer module.

Turn on the capacity balancer, by using the **ceph balancer on** command. By default, when turning on the balancer module, automatic balancing is used and the `upmap` mode is enabled.

```
[ceph: root@host01 /]# ceph balancer on
```

You can change the balancer mode from `upmap` to `crush-compact` mode. The `crush-compact` mode is backward compatible with older clients and makes small changes to the data distribution to ensure that OSDs are equally utilized.

After the balancer is on, you can verify the balancer status.

```
[ceph: root@host01 /]# ceph balancer on
[ceph: root@host01 /]# ceph balancer mode crush-compat
[ceph: root@host01 /]# ceph balancer status
{
    "active": true,
    "last_optimize_duration": "0:00:00.001174",
    "last_optimize_started": "Fri Nov 22 11:09:18 2024",
    "mode": "crush-compact",
    "no_optimization_needed": false,
    "optimize_result": "Unable to find further optimization, change balancer mode and retry
might help",
    "plans": []
}
```

To turn off the balancer module, use the **ceph balancer off** command. For example,

```
[ceph: root@host01 /]# ceph balancer off
[ceph: root@host01 /]#  ceph balancer status
{
    "active": false,
    "last_optimize_duration": "",
    "last_optimize_started": "",
    "mode": "crush-compat",
    "no_optimization_needed": false,
    "optimize_result": "",
    "plans": []
}
```

Throttling

Adjust the balancer with throttling.

When the cluster is healthy, the balancer throttles its changes such that the percentage of placement groups (PGs) that are misplaced, or need to be moved, is below a threshold of 5% by default. No adjustments are made to the PG distribution if the cluster is degraded. For example, if an OSD has failed and the system has not yet healed itself.

Adjust the balancer percentage by using the **target_max_misplaced_ratio** parameter.

```
ceph config-key set mgr target_max_misplaced_ratio THRESHOLD_PERCENTAGE
```

The following example increases the threshold to 7%.

```
[ceph: root@host01 /]# ceph config-key set mgr target_max_misplaced_ratio .07
```

Supervised optimization

Break down the balancer operations in a phased manner.

About this task

The balancer operation is broken into a few distinct phases.

Procedure

1. Build a `plan`.
2. Evaluate the quality of the data distribution, either for the current PG distribution, or the PG distribution that might result after running a `plan`.
3. Run the `plan`.
    a. Evaluate and score the current distribution.

    ```
    ceph balancer eval
    ```

    b. Evaluate the distribution for a single pool.

    ```
    ceph balancer eval POOL_NAME
    ```

    For example,

    ```
    [ceph: root@host01 /]# ceph balancer eval rbd
    ```

    c. See a greater detail for the evaluation.

    ```
    ceph balancer eval-verbose ...
    ```

    d. Generate a plan using the currently configured mode.

    ```
    ceph balancer optimize PLAN_NAME
    ```

    Replace *PLAN_NAME* with a custom plan name.

For example,

```
[ceph: root@host01 /]# ceph balancer optimize rbd_123
```

e. See the contents of a plan.

```
ceph balancer show PLAN_NAME
```

For example,

```
[ceph: root@host01 /]# ceph balancer show rbd_123
```

f. Discard old plans.

```
ceph balancer rm PLAN_NAME
```

For example,

```
[ceph: root@host01 /]# ceph balancer rm rbd_123
```

g. See currently recorded plans use the status command.

```
ceph balancer status
```

h. Calculate the quality of the distribution that would result after executing a plan.

```
ceph balancer eval PLAN_NAME
```

For example,

```
[ceph: root@host01 /]# ceph balancer eval rbd_123
```

i. Run the plan.

```
ceph balancer execute PLAN_NAME
```

For example,

```
[ceph: root@host01 /]# ceph balancer execute rbd_123
```

> **Note:** Only run the plan if it is expected to improve the distribution. After execution, the plan is discarded.

Balancing a Ceph cluster using read balancer

Balance read workloads across Object Storage Daemons (OSDs) by optimizing primary placement group (PG) ownership.

About this task

The Ceph Manager `balancer` module balances data placement and capacity utilization across OSDs. In some clusters, OSDs can still receive uneven read workloads even after the CRUSH algorithm does its best to distribute PGs evenly.

This situation commonly occurs in small clusters or in clusters where OSDs differ in size or device class. Larger OSDs host more placement groups and can therefore receive a larger share of read requests.

The read balancer complements capacity balancing by distributing primary placement group ownership more evenly across OSDs. You can perform read balancing by using online optimization, offline optimization, or supervised optimization.

Balancing reads by using online optimization

Balance read workloads automatically by enabling the balancer module and setting a read-balancing mode.

Procedure

1. Enable the balancer module.

```
ceph mgr module enable balancer
```

2. Ensure that the capacity balancer is turned on.

```
ceph balancer on
```

3. Configure the capacity balancer to maintain a more uniform placement group distribution.
This setting is recommended for clusters that contain multiple OSD sizes or device classes.

```
ceph config set mgr mgr/balancer/upmap_max_deviation 1
```

4. Update the **`min_compat_client`** setting to ensure compatibility.
Read balancing requires that no clients older than Red Hat Ceph Storage 7 connect to the cluster. Older clients are not supported and fail to connect when read balancing is enabled.

   To use online optimization, configure the cluster to require Reef or later clients.

```
ceph osd set-require-min-compat-client reef
```

   > **Note:** This command fails if any pre-Reef clients or daemons are connected to the monitors. To work around this issue, use the `--yes-i-really-mean-it` flag:
   >
   > ```
   > ceph osd set-require-min-compat-client reef --yes-i-really-mean-it
   > ```
   >
   > You can check which client versions are in use with the **`ceph features`** command. For example:
   >
   > ```
   > [ceph: root@host01 /]# ceph features
   > ```

5. Change the balancer mode to `upmap-read` or `read` to enable read balancing.
The default mode is `upmap`, which balances OSD capacity only.

   To enable read workload balancing, configure one of the following modes. The recommended mode is `upmap-read`, which balances both capacity utilization and read workloads.

   Use `read` mode only when maximizing read workload balancing is more important than maintaining optimal capacity balancing.

   - `ceph balancer mode upmap-read`
   - `ceph balancer mode read`

6. Check the current status of the balancer.

```
ceph balancer status
```

   For example,

```
[ceph: root@host01 /]# ceph balancer status
{
    "active": true,
    "last_optimize_duration": "0:00:00.013640",
    "last_optimize_started": "Mon Oct 19 14:47:57 2026",
    "mode": "upmap-read",
    "no_optimization_needed": true,
    "optimize_result": "Unable to find further optimization, or pool(s) pg_num is
decreasing, or distribution is already perfect",
    "plans": []
}
```

Balancing reads by using offline optimization (Technology Preview)

Correct unbalanced primary OSDs by running the offline read balancer on an exported copy of the OSD map.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Before running the offline read balancer on an exported copy of the OSD map, run the capacity balancer to balance PG placement across OSDs. This ensures optimal results. Complete the following steps to run the capacity balancer:
  - Get the latest copy of your `osdmap`.

    ```
    ceph osd getmap -o map
    ```

  - Run the `upmap` balancer.

    ```
    osdmaptool map --upmap balance.sh
    ```

  - The file `balance.sh` contains the proposed solution.

    Run the following command to apply the calculated optimizations.

    ```
    source balance.sh
    ```

  For more information, see “Balancing Red Hat Ceph Storage cluster using the capacity balancer” on page 572.

About this task

> **Important:** Technology Preview features are not supported with production service level agreements (SLAs), might not be functionally complete, and does not recommend using them for production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

If you have unbalanced primary OSDs, you can correct them by using the offline optimizer built into the **`osdmaptool`** utility. Run the capacity balancer before running the read balancer to ensure optimal results.

Procedure

1. Check the `read_balance_score` for each pool.

   ```
   ceph osd pool ls detail
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph osd pool ls detail
   ```

> **Note:**
> You can use `json` formatting to get additional details.

If the `read_balance_score` is considerably greater than 1, the pool has unbalanced primary OSDs.

For a homogeneous cluster the optimal score is [Ceil{(number of PGs/Number of OSDs)}/(number of PGs/Number of OSDs)]/[(number of PGs/Number of OSDs)/(number of PGs/Number of OSDs)].

For example, if you have a pool with 32 PGs and 10 OSDs, (number of PGs/Number of OSDs) = 32/10 = 3.2. The optimal score is 4/3.2 = 1.25. If you have another pool in the same system with 64 PGs, the optimal score is 7/6.4 = 1.09375.

The following is an example output:

```
$ ceph osd pool ls detail
pool 1 '.mgr' replicated size 3 min_size 1 crush_rule 0 object_hash rjenkins pg_num 1
pgp_num 1 autoscale_mode on last_change 17 flags hashpspool stripe_width 0 pg_num_max
32 pg_num_min 1 application mgr read_balance_score 3.00
pool 2 'cephfs.a.meta' replicated size 3 min_size 1 crush_rule 0 object_hash rjenkins
pg_num 16 pgp_num 16 autoscale_mode on last_change 55 lfor 0/0/25 flags hashpspool
stripe_width 0 pg_autoscale_bias 4 pg_num_min 16 recovery_priority 5 application
cephfs read_balance_score 1.50
pool 3 'cephfs.a.data' replicated size 3 min_size 1 crush_rule 0 object_hash rjenkins
pg_num 128 pgp_num 128 autoscale_mode on last_change 27 lfor 0/0/25 flags
hashpspool,bulk stripe_width 0 application cephfs read_balance_score 1.31
```

2. Get the latest copy of your `osdmap`.

```
ceph osd getmap -o om
```

For example,

```
[ceph: root@host01 /]# ceph osd getmap -o om
got osdmap epoch 56
```

3. Run the optimizer.
The `balance.sh` file contains the proposed solution.

```
osdmaptool om --read balance.sh --read-pool POOL_NAME [--vstart]
```

For example,

```
[ceph: root@host01 /]# osdmaptool om --read balance.sh --read-pool cephfs.a.meta
./bin/osdmaptool: osdmap file 'om'
writing upmap command output to: balance.sh
---------- BEFORE ------------
osd.0 | primary affinity: 1 | number of prims: 4
osd.1 | primary affinity: 1 | number of prims: 8
osd.2 | primary affinity: 1 | number of prims: 4
read_balance_score of 'cephfs.a.meta': 1.5
---------- AFTER ------------
osd.0 | primary affinity: 1 | number of prims: 5
osd.1 | primary affinity: 1 | number of prims: 6
osd.2 | primary affinity: 1 | number of prims: 5
read_balance_score of 'cephfs.a.meta': 1.13
num changes: 2
```

4. Apply the calculated optimizations to the cluster.
The `balance.sh` file contains the proposed solution.

```
source balance.sh
```

For example,

```
[ceph: root@host01 /]# source balance.sh
```

The following shows an example of offline balancer output when you run the commands.

```
$ cat balance.sh
ceph osd pg-upmap-primary 2.3 0
ceph osd pg-upmap-primary 2.4 2

$ source balance.sh
change primary for pg 2.3 to osd.0
change primary for pg 2.4 to osd.2
```

Note:

- These optimizations are applied through standard Ceph commands generated by the read balancer. Do not run the **ceph osd setcrushmap** command because an updated Controlled Replication Under Scalable Hashing (CRUSH) map is not required.
- If you are running the **ceph osd pg-upmap-primary** command for the first time, you might receive the warning `Error EPERM: min_compat_client luminous < reef, which is required for pg-upmap-primary`. In this case, run the **ceph osd set-require-min-compat-client reef** command to update the cluster minimum compatibility level.

What to do next

Consider rechecking the scores and rerunning the balancer if the number of PGs changes or if you add or remove OSDs. These operations can significantly affect read-balancing results.

Balancing reads by using supervised optimization

Use the read balancer with supervised optimization by setting the balancer mode to `upmap-read`.

You can also use the read balancer with supervised optimization. For more information, see "Supervised optimization" on page 573. Set the balancer mode to `upmap-read`.

Using the Ceph Manager alerts module

You can use the Ceph Manager alerts module to send simple alert messages about the Red Hat Ceph Storage cluster's health by email.

Note: This module is not intended to be a robust monitoring solution. The fact that it is run as part of the Ceph cluster itself is fundamentally limiting in that a failure of the `ceph-mgr` daemon prevents alerts from being sent. This module can, however, be useful for standalone clusters that exist in environments where existing monitoring infrastructure does not exist.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph Monitor node.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Enable the alerts module:

   Example

   ```
   [ceph: root@host01 /]# ceph mgr module enable alerts
   ```

3. Ensure the alerts module is enabled:

   Example

   ```
   [ceph: root@host01 /]# ceph mgr module ls | more
   {
       "always_on_modules": [
           "balancer",
           "crash",
           "devicehealth",
           "orchestrator",
           "pg_autoscaler",
           "progress",
           "rbd_support",
           "status",
           "telemetry",
           "volumes"
       ],
       "enabled_modules": [
           "alerts",
           "cephadm",
           "dashboard",
           "iostat",
           "prometheus",
           "restful"
       ]
   ```

4. Configure the Simple Mail Transfer Protocol (SMTP):

   Syntax

   ```
   ceph config set mgr mgr/alerts/smtp_host SMTP_SERVER
   ceph config set mgr mgr/alerts/smtp_destination RECEIVER_EMAIL_ADDRESS
   ceph config set mgr mgr/alerts/smtp_sender SENDER_EMAIL_ADDRESS
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph config set mgr mgr/alerts/smtp_host smtp.example.com
   [ceph: root@host01 /]# ceph config set mgr mgr/alerts/smtp_destination
   example@example.com
   [ceph: root@host01 /]# ceph config set mgr mgr/alerts/smtp_sender example2@example.com
   ```

5. Optional: By default, the alerts module uses SSL and port 465.

   Syntax

   ```
   ceph config set mgr mgr/alerts/smtp_port PORT_NUMBER
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph config set mgr mgr/alerts/smtp_port 587
   ```

   Do not set the `smtp_ssl` parameter while configuring alerts.

6. Authenticate to the SMTP server:

   Syntax

   ```
   ceph config set mgr mgr/alerts/smtp_user USERNAME
   ceph config set mgr mgr/alerts/smtp_password PASSWORD
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph config set mgr mgr/alerts/smtp_user admin1234
   [ceph: root@host01 /]# ceph config set mgr mgr/alerts/smtp_password admin1234
   ```

7. Optional: By default, SMTP `From` name is `Ceph`. To change that, set the `smtp_from_name` parameter:

   Syntax

   ```
   ceph config set mgr mgr/alerts/smtp_from_name CLUSTER_NAME
   ```

Example

```
[ceph: root@host01 /]# ceph config set mgr mgr/alerts/smtp_from_name 'Ceph Cluster
Test'
```

8. Optional: By default, the alerts module checks the storage cluster's health every minute, and sends a message when there is a change in the cluster health status. To change the frequency, set the `interval` parameter:

   Syntax

   ```
   ceph config set mgr mgr/alerts/interval INTERVAL
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph config set mgr mgr/alerts/interval "5m"
   ```

   In this example, the interval is set to 5 minutes.

9. Optional: Send an alert immediately:

   Example

   ```
   [ceph: root@host01 /]# ceph alerts send
   ```

Reference

- *Health messages of a Ceph cluster*

Using the Ceph manager crash module

By default, daemon crashdumps are dumped in `/var/lib/ceph/crash`. You can configure it with the option `crash dir`. Crash directories are named by time, date, and a randomly-generated UUID, and contain a metadata file `meta` and a recent log file, with a `crash_id` that is the same.

You can use `ceph-crash.service` to submit these crashes automatically and persist in the Ceph Monitors. The `ceph-crash.service` watches the `crashdump` directory and uploads them with `ceph crash post`.

The *RECENT_CRASH* heath message is one of the most common health messages in a Ceph cluster. This health message means that one or more Ceph daemons has crashed recently, and the crash has not yet been archived or acknowledged by the administrator. This might indicate a software bug, a hardware problem like a failing disk, or some other problem. The option `mgr/crash/warn_recent_interval` controls the time period of what recent means, which is two weeks by default. You can disable the warnings by running the following command:

Example

```
[ceph: root@host01 /]# ceph config set mgr/crash/warn_recent_interval 0
```

The option `mgr/crash/retain_interval` controls the period for which you want to retain the crash reports before they are automatically purged. The default for this option is one year.

Prerequisites

- A running Red Hat Ceph Storage cluster.

Procedure

1. Ensure the crash module is enabled:

   Example

   ```
   [ceph: root@host01 /]# ceph mgr module ls | more
   {
       "always_on_modules": [
           "balancer",
           "crash",
           "devicehealth",
   ```

```
        "orchestrator_cli",
        "progress",
        "rbd_support",
        "status",
        "volumes"
    ],
    "enabled_modules": [
        "dashboard",
        "pg_autoscaler",
        "prometheus"
    ]
```

2. Save a crash dump: The metadata file is a JSON blob stored in the crash dir as `meta`. You can invoke the ceph command `-i -` option, which reads from stdin.

   Example

   ```
   [ceph: root@host01 /]# ceph crash post -i meta
   ```

3. List the timestamp or the UUID crash IDs for all the new and archived crash info:

   Example

   ```
   [ceph: root@host01 /]# ceph crash ls
   ```

4. List the timestamp or the UUID crash IDs for all the new crash information:

   Example

   ```
   [ceph: root@host01 /]# ceph crash ls-new
   ```

5. List the timestamp or the UUID crash IDs for all the new crash information:

   Example

   ```
   [ceph: root@host01 /]# ceph crash ls-new
   ```

6. List the summary of saved crash information grouped by age:

   Example

   ```
   [ceph: root@host01 /]# ceph crash stat
   8 crashes recorded
   8 older than 1 days old:
   2022-05-20T08:30:14.533316Z_4ea88673-8db6-4959-a8c6-0eea22d305c2
   2022-05-20T08:30:14.590789Z_30a8bb92-2147-4e0f-a58b-a12c2c73d4f5
   2022-05-20T08:34:42.278648Z_6a91a778-bce6-4ef3-a3fb-84c4276c8297
   2022-05-20T08:34:42.801268Z_e5f25c74-c381-46b1-bee3-63d891f9fc2d
   2022-05-20T08:34:42.803141Z_96adfc59-be3a-4a38-9981-e71ad3d55e47
   2022-05-20T08:34:42.830416Z_e45ed474-550c-44b3-b9bb-283e3f4cc1fe
   2022-05-24T19:58:42.549073Z_b2382865-ea89-4be2-b46f-9a59af7b7a2d
   2022-05-24T19:58:44.315282Z_1847afbc-f8a9-45da-94e8-5aef0738954e
   ```

7. View the details of the saved crash:

   Syntax

   ```
   ceph crash info CRASH_ID
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph crash info 2022-05-24T19:58:42.549073Z_b2382865-ea89-4be2-
   b46f-9a59af7b7a2d
   {
       "assert_condition": "session_map.sessions.empty()",
       "assert_file": "/builddir/build/BUILD/ceph-16.1.0-486-g324d7073/src/mon/
   Monitor.cc",
       "assert_func": "virtual Monitor::~Monitor()",
       "assert_line": 287,
       "assert_msg": "/builddir/build/BUILD/ceph-16.1.0-486-g324d7073/src/mon/Monitor.cc:
   In function 'virtual Monitor::~Monitor()' thread 7f67a1aeb700 time
   ```

```
2022-05-24T19:58:42.545485+0000\n/builddir/build/BUILD/ceph-16.1.0-486-g324d7073/src/
mon/Monitor.cc: 287: FAILED ceph_assert(session_map.sessions.empty())\n",
    "assert_thread_name": "ceph-mon",
    "backtrace": [
        "/lib64/libpthread.so.0(+0x12b30) [0x7f679678bb30]",
        "gsignal()",
        "abort()",
        "(ceph::__ceph_assert_fail(char const*, char const*, int, char const*)+0x1a9)
[0x7f6798c8d37b]",
        "/usr/lib64/ceph/libceph-common.so.2(+0x276544) [0x7f6798c8d544]",
        "(Monitor::~Monitor()+0xe30) [0x561152ed3c80]",
        "(Monitor::~Monitor()+0xd) [0x561152ed3cdd]",
        "main()",
        "__libc_start_main()",
        "_start()"
    ],
    "ceph_version": "16.2.8-65.el8cp",
    "crash_id": "2022-07-06T19:58:42.549073Z_b2382865-ea89-4be2-b46f-9a59af7b7a2d",
    "entity_name": "mon.ceph-adm4",
    "os_id": "rhel",
    "os_name": "Red Hat Enterprise Linux",
    "os_version": "8.5 (Ootpa)",
    "os_version_id": "8.5",
    "process_name": "ceph-mon",
    "stack_sig": "957c21d558d0cba4cee9e8aaf9227b3b1b09738b8a4d2c9f4dc26d9233b0d511",
    "timestamp": "2022-07-06T19:58:42.549073Z",
    "utsname_hostname": "host02",
    "utsname_machine": "x86_64",
    "utsname_release": "4.18.0-240.15.1.el8_3.x86_64",
    "utsname_sysname": "Linux",
    "utsname_version": "#1 SMP Wed Jul 06 03:12:15 EDT 2022"
}
```

8. Remove saved crashes older than *KEEP* days: Here, *KEEP* must be an integer.

   Syntax

   ```
   ceph crash prune KEEP
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph crash prune 60
   ```

9. Archive a crash report so that it is no longer considered for the `RECENT_CRASH` health check and does not appear in the `crash ls-new` output. It appears in the `crash ls`.

   Syntax

   ```
   ceph crash archive CRASH_ID
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph crash archive 2022-05-24T19:58:42.549073Z_b2382865-
   ea89-4be2-b46f-9a59af7b7a2d
   ```

10. Archive all crash reports:

    Example

    ```
    [ceph: root@host01 /]# ceph crash archive-all
    ```

11. Remove the crash dump:

    Syntax

    ```
    ceph crash rm CRASH_ID
    ```

    Example

    ```
    [ceph: root@host01 /]# ceph crash rm 2022-05-24T19:58:42.549073Z_b2382865-ea89-4be2-
    b46f-9a59af7b7a2d
    ```

Reference

- *Health messages of a Ceph cluster*

Telemetry module

The telemetry module sends data about the storage cluster to help understand how Ceph is used and what problems are encountered during operations. The data is visualized on the public dashboard to view the summary statistics on how many clusters are reporting, their total capacity and OSD count, and version distribution trends.

The telemetry report is broken down into different channels, each with a different type of information. After the telemetry is enabled, you can turn on or turn off the individual channels.

The following are the available channels:

- `basic`
- `crash`
- `device`
- `ident`
- `perf`

`basic`

Default is `on`. This channel provides the basic information about the clusters, which includes the following information:

- The capacity of the cluster.
- The number of monitors, managers, OSDs, MDSs, Ceph Object Gateways, or other daemons.
- The software version that is currently being used.
- The number and types of RADOS pools and Ceph File Systems.
- The names of configuration options that are changed from their default (but not their values).

`crash`

Default is `on`. This channel provides information about the daemon crashes, which includes the following information:

- The type of daemon.
- The version of the daemon.
- The operating system, the OS distribution, and the kernel version.
- The stack trace that identifies where in the Ceph code the crash occurred.

`device`

Default is `on`. This channel provides information about the device metrics, which includes anonymized SMART metrics.

`ident`

Default is `off`. This channel provides the user-provided identifying information about the cluster such as cluster description, and contact email address.

`perf`

Default is `off`. This channel provides the various performance metrics of the cluster, which can be used for the following:

- Reveal overall cluster health.
- Identify workload patterns.
- Troubleshoot issues with latency, throttling, memory management, and other similar issues.
- Monitor cluster performance by daemon.

The data that is reported does not contain any sensitive data such as pool names, object names, object contents, hostnames, or device serial numbers.

It contains counters and statistics on how the cluster is deployed, Ceph version, host distribution, and other parameters that help the project to gain a better understanding of the way Ceph is used.

Data is secure and is sent to https://telemetry.ceph.com.

Enabling telemetry

Before enabling channels, ensure that the telemetry is `on`.

Enable telemetry by using the following command:

```
ceph telemetry on
```

Enabling and disabling channels

You can enable and disable either individual channels, multiple channels simultaneously, or all channels.

- Enable an individual channel by using the following command:

  ```
  ceph telemetry enable channel CHANNEL
  ```

  For example,

  ```
  [ceph: root@host01 /]# ceph telemetry enable channel basic
  ```

- Disable an individual channel by using the following command:

  ```
  ceph telemetry disable channel CHANNEL
  ```

  For example,

  ```
  [ceph: root@host01 /]# ceph telemetry disable channel basic
  ```

- Enable multiple individual channels with a single command by using the following command:

  ```
  ceph telemetry enable channel CHANNEL1 CHANNEL2 CHANNEL3
  ```

  For example,

  ```
  [ceph: root@host01 /]# ceph telemetry enable channel basic crash device
  ```

- Disable multiple individual channels with a single command by using the following command:

  ```
  ceph telemetry disable channel CHANNEL1 CHANNEL2 CHANNEL3
  ```

  For example,

  ```
  [ceph: root@host01 /]# ceph telemetry disable channel basic crash device
  ```

- Enable all channels simultaneously by using the following command:

  ```
  ceph telemetry enable channel all
  ```

  For example,

  ```
  [ceph: root@host01 /]# ceph telemetry enable channel all
  ```

- Disable all channels simultaneously by using the following command:

  ```
  ceph telemetry disable channel all
  ```

  For example,

  ```
  [ceph: root@host01 /]# ceph telemetry disable channel all
  ```

Sample reports

Use the following commands to generate and view sample reports.

- Review the data report at any time by generating a sample report.

```
ceph telemetry show
```

  - If telemetry is `off`, you can preview the sample report.

```
ceph telemetry preview
```

    It takes longer to generate a sample report for storage clusters with hundreds of OSDs or more.

- Device reports are generated separately to protect your privacy. Device reports output with data such as the host name and device serial number as anonymous. The device telemetry is sent to a different endpoint and does not associate the device data with a particular cluster.

```
ceph telemetry show-device
```

  - If telemetry is `off`, you can preview the sample device report.

```
ceph telemetry preview-device
```

- Generate a single output for both the data and device.

```
ceph telemetry show-all
```

  - If telemetry is `off`, you can preview both reports.

```
ceph telemetry preview-all
```

- Generate a sample report per channel.

```
ceph telemetry show CHANNEL_NAME
```

  - If telemetry is `off`, you can preview a sample report per channel.

```
ceph telemetry preview CHANNEL_NAME
```

Collections

Collections are different aspects of data that is collected within a channel.

- List the collections.

```
ceph telemetry collection ls
```

- See the difference between the collections that you are enrolled in, and the new, available collections.

```
ceph telemetry diff
```

- Enroll to the most recent collections.

```
ceph telemetry on
ceph telemetry enable channel CHANNEL_NAME
```

`interval`

The `interval` module compiles and sends a new report every 24 hours by default.

Use the **`mgr/telemetry/interval`** option to adjust the report interval.

```
ceph config set mgr mgr/telemetry/interval INTERVAL
```

The following example sets the report generation for every three days (72 hours).

```
[ceph: root@host01 /]# ceph config set mgr mgr/telemetry/interval 72
```

status

Use the `status` option to view the current configuration.

```
ceph telemetry status
```

Manually sending telemetry

Send an asynchronous, one-time set of telemetry data, by running the `send` command.

```
ceph telemetry send
```

If telemetry is `off`, include `--license sharing-1-0` to the command.

```
ceph telemetry send --license sharing-1-0
```

Sending telemetry through a proxy

If the cluster cannot connect directly to the configured telemetry endpoint, you can configure a HTTP/HTTPs proxy server.

```
ceph config set mgr mgr/telemetry/proxy PROXY_URL
```

For example,

```
[ceph: root@host01 /]# ceph config set mgr mgr/telemetry/proxy https://10.0.0.1:8080
```

Contact and description

A contact and description can be added to the report. This is optional and is disabled by default.

> **Note:** If `ident` flag is enabled, its details are not displayed in the leaderboard.

```
ceph config set mgr mgr/telemetry/contact 'CONTACT_NAME'
ceph config set mgr mgr/telemetry/description 'DESCRIPTION'
ceph config set mgr mgr/telemetry/channel_ident true
```

For example,

```
[ceph: root@host01 /]# ceph config set mgr mgr/telemetry/contact 'John Doe
<john.doe@example.com>'
[ceph: root@host01 /]# ceph config set mgr mgr/telemetry/description 'My first Ceph
cluster'
[ceph: root@host01 /]# ceph config set mgr mgr/telemetry/channel_ident true
```

Leaderboard

You can participate in a leaderboard on the public Telemetry dashboard.

The leaderboard displays basic information about the storage cluster. This board includes the total storage capacity and the number of OSDs.

```
ceph config set mgr mgr/telemetry/leaderboard true
```

Managing OSDs

As a storage administrator, you can use the Ceph Orchestrators to manage Red Hat Ceph Storage OSDs.

Deploy one OSD per physical storage device, using the entire device. This recommendation applies to all supported device types, including HDDs, SSDs, and NVMe drives.

- Do not deploy multiple OSDs per device for NVMe SSDs.
- This guidance concerns the main OSD block device, not SSDs used for WAL+DB metadata offload from HDD OSDs.

Note: In some cases, deploying multiple OSDs per device may be appropriate. In the following cases, contact Red Hat Support for configuration guidance.

- Ceph Object Gateway workloads with a large number of very small S3 objects or many object versions.
- Very large NVMe SSDs (30 TB or larger).

Check the capacity of a cluster regularly to see whether it is reaching the upper end of its storage capacity. As a storage cluster reaches its `near full` ratio, add one or more OSDs to expand the storage cluster's capacity. If the node has multiple storage drives, you might also need to remove one of the `ceph-osd` daemons for that drive.

Important:

- Ensure that when you remove an OSD, the storage cluster is not at its `near full` ratio.
- Do not allow a storage cluster to reach the `full` ratio before adding an OSD. OSD failures that occur after the storage cluster reaches the `near full` ratio can cause the storage cluster to exceed the `full` ratio. Ceph blocks write access to protect the data until you resolve the storage capacity issues. When removing an OSD, be sure to consider the impact on the `full` ratio first.

Configure Ceph OSDs and their supporting hardware in a similar manner as a storage strategy for any pools that will use the OSDs. Ceph prefers uniform hardware across pools for a consistent performance profile. For best performance, consider a CRUSH hierarchy with drives of the same type or size.

In cases where you add drives of dissimilar size, adjust their weights accordingly. When you add the OSD to the CRUSH map, consider the weight for the new OSD. Hard disk drive capacity grows approximately 40% per year, so newer OSD nodes might have larger hard disk drives than older nodes in the storage cluster, that is, they might have a greater weight.

Automatically tuning OSD memory

The OSD daemons adjust the memory consumption based on the `osd_memory_target` configuration option.

The option `osd_memory_target` sets OSD memory based upon the available RAM in the system.

Important: Operating System swap must not be enabled. Configuring the operating system to use swap to provide additional virtual memory for daemons is not recommended for modern systems and can result in degraded performance.

Syntax

```
ceph config set osd osd_memory_target_autotune true
```

`cephadm` starts with a fraction `mgr/cephadm/autotune_memory_target_ratio`, which defaults to `0.7` of the total RAM in the system, subtract off any memory consumed by non-autotuned daemons such as non-OSDS and for OSDs for which `osd_memory_target_autotune` is false, and then divide by the remaining OSDs.

By default, `autotune_memory_target_ratio` is 0.2 for hyper-converged infrastructure and 0.7 for other environments.

The `osd_memory_target` parameter is calculated as follows:

Syntax

```
osd_memory_target = TOTAL_RAM_OF_THE_OSD_NODE (in Bytes) * (autotune_memory_target_ratio) /
NUMBER_OF_OSDS_IN_THE_OSD_NODE - (SPACE_ALLOCATED_FOR_OTHER_DAEMONS (in Bytes))
```

SPACE_ALLOCATED_FOR_OTHER_DAEMONS may optionally include the following daemon space allocations:

- Alertmanager: 1 GB
- Grafana: 1 GB
- Ceph Manager: 4 GB
- Ceph Monitor: 2 GB
- Node-exporter: 1 GB
- Prometheus: 1 GB

For example, if a node has 24 OSDs and has 251 GB RAM space, then `osd_memory_target` is `7860684936`.

The final targets are reflected in the configuration database with options. You can view the limits and the current memory consumed by each daemon from the `ceph orch ps` output under `MEM LIMIT` column.

> **Note:**
> The default setting of `osd_memory_target_autotune true` is unsuitable for hyperconverged infrastructures where compute and Ceph storage services are colocated. In a hyperconverged infrastructure, the `autotune_memory_target_ratio` can be set to `0.2` to reduce the memory consumption of Ceph.
>
> ```
> [ceph: root@host01 /]# ceph config set mgr mgr/cephadm/autotune_memory_target_ratio
> 0.2
> ```

You can manually set a specific memory target for an OSD in the storage cluster.

Example

```
[ceph: root@host01 /]# ceph config set osd.123 osd_memory_target 7860684936
```

You can manually set a specific memory target for an OSD host in the storage cluster.

Syntax

```
ceph config set osd/host:_HOSTNAME_ osd_memory_target _TARGET_BYTES_
```

Example

```
[ceph: root@host01 /]# ceph config set osd/host:host01 osd_memory_target 1000000000
```

> **Note:** Enabling `osd_memory_target_autotune` overwrites existing manual OSD memory target settings. To prevent daemon memory from being tuned even when the `osd_memory_target_autotune` option or other similar options are enabled, set the `_no_autotune_memory` label on the host.

Syntax

```
ceph orch host label add HOSTNAME _no_autotune_memory
```

You can exclude an OSD from memory autotuning by disabling the autotune option and setting a specific memory target.

Example

```
[ceph: root@host01 /]# ceph config set osd.123 osd_memory_target_autotune false
[ceph: root@host01 /]# ceph config set osd.123 osd_memory_target 16G
```

Listing devices for Ceph OSD deployment

You can check the list of available devices before deploying OSDs using the Ceph Orchestrator.

The commands are used to print a list of devices discoverable by `cephadm`. A storage device is considered available if all of the following conditions are met:

- The device must have no partitions.
- The device must not have any LVM state.
- The device must not be mounted.
- The device must not contain a file system.
- The device must not contain a Ceph BlueStore OSD.
- The device must be larger than 5 GB.

> **Note:** Ceph will not provision an OSD on a device that is not available.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Hosts are added to the cluster.
- All manager and monitor daemons are deployed.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. List the available devices to deploy OSDs:

   Syntax

   ```
   ceph orch device ls [--hostname=HOSTNAME_1 HOSTNAME_2] [--wide] [--refresh]
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph orch device ls --wide --refresh
   ```

   Using the `--wide` option provides all details relating to the device, including any reasons that the device might not be eligible for use as an OSD. This option does not support NVMe devices.

3. Optional: To enable *Health*, *Ident*, and *Fault* fields in the output of `ceph orch device ls`, run the following commands:

   NOTE: These fields are supported by `libstoragemgmt` library and currently supports SCSI, SAS, and SATA devices.

   a. As root user outside the `cephadm` shell, check your hardware's compatibility with `libstoragemgmt` library to avoid unplanned interruption to services:

      Example

      ```
      [root@host01 ~]# cephadm shell lsmcli ldl
      ```

      In the output, you see the *Health Status* as *Good* with the respective *SCSI VPD 0x83* ID.

      NOTE: If you do not get this information, then enabling the fields might cause erratic behavior of devices.

   b. Log back into the `cephadm` shell and enable `libstoragemgmt` support:

      Example

```
[root@host01 ~]# cephadm shell
[ceph: root@host01 /]# ceph config set mgr mgr/cephadm/device_enhanced_scan true
```

Once this is enabled, `ceph orch device ls` gives the output of *Health* field as *Good*.

Verification

- List the devices:

  Example

  ```
  [ceph: root@host01 /]# ceph orch device ls
  ```

Zapping devices for Ceph OSD deployment

Before deploying OSDs, it is important to check the list of available devices. If there is no space available on the devices, you can clear the data on the devices by zapping them.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Hosts are added to the cluster.
- All manager and monitor daemons are deployed.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. List the available devices to deploy OSDs:

   Syntax

   ```
   ceph orch device ls [--hostname=HOSTNAME_1 HOSTNAME_2] [--wide] [--refresh]
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph orch device ls --wide --refresh
   ```

3. Clear the data of a device:

   Syntax

   ```
   ceph orch device zap HOSTNAME FILE_PATH --force
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph orch device zap host02 /dev/sdb --force
   ```

Verification

- Verify the space is available on the device:

  Example

  ```
  [ceph: root@host01 /]# ceph orch device ls
  ```

  You will see that the field under *Available* is *Yes*.

Reference

- *Listing devices for Ceph OSD deployment*

Deploying Ceph OSDs on all available devices

You can deploy all OSDS on all the available devices. `cephadm` allows the Ceph Orchestrator to discover and deploy the OSDs on any available and unused storage device.

To deploy OSDs all available devices, run the command without the **unmanaged** parameter and then re-run the command with the parameter to prevent from creating future OSDs.

> **Note:** The deployment of OSDs with `--all-available-devices` is generally used for smaller clusters. For larger clusters, use the OSD specification file.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Hosts are added to the cluster.
- All manager and monitor daemons are deployed.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. List the available devices to deploy OSDs:

   Syntax

   ```
   ceph orch device ls [--hostname=HOSTNAME_1 HOSTNAME_2] [--wide] [--refresh]
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph orch device ls --wide --refresh
   ```

3. Deploy OSDs on all available devices:

   Example

   ```
   [ceph: root@host01 /]# ceph orch apply osd --all-available-devices
   ```

   The effect of `ceph orch apply` is persistent which means that the Orchestrator automatically finds the device, adds it to the cluster, and creates new OSDs. This occurs under the following conditions:

   - New disks or drives are added to the system.
   - Existing disks or drives are zapped.
   - An OSD is removed and the devices are zapped.

     You can disable automatic creation of OSDs on all the available devices by using the `--unmanaged` parameter.

     Example

     ```
     [ceph: root@host01 /]# ceph orch apply osd --all-available-devices --
     unmanaged=true
     ```

     Setting the parameter `--unmanaged` to `true` disables the creation of OSDs and also there is no change if you apply a new OSD service.

Note: The command `ceph orch daemon add` creates new OSDs, but does not add an OSD service.

Verification

- List the service:

  Example

  ```
  [ceph: root@host01 /]# ceph orch ls
  ```

- View the details of the node and devices:

  Example

  ```
  [ceph: root@host01 /]# ceph osd tree
  ```

Reference

- *Listing devices for Ceph OSD deployment*

Deploying Ceph OSDs on specific devices and hosts

You can deploy all the Ceph OSDs on specific devices and hosts using the Ceph Orchestrator.

For more information, see "Listing devices for Ceph OSD deployment" on page 588.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Hosts are added to the cluster.
- All manager and monitor daemons are deployed.

Procedure

1. Log into the `cephadm` shell:
   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. List the available devices to deploy OSDs:

   ```
   ceph orch device ls [--hostname=HOSTNAME_1 HOSTNAME_2] [--wide] [--refresh]
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph orch device ls --wide --refresh
   ```

3. Deploy OSDs on specific devices and hosts:

   ```
   ceph orch daemon add osd _HOSTNAME_:_DEVICE_PATH_
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph orch daemon add osd host02:/dev/sdb
   ```

   To deploy ODSs on a raw physical device, without an LVM layer, use the `--method raw` option.

```
ceph orch daemon add osd --method raw _HOSTNAME_:_DEVICE_PATH_
```

Example

```
[ceph: root@host01 /]# ceph orch daemon add osd --method raw host02:/dev/sdb
```

> **Note:** If you have separate DB or WAL devices, the ratio of block to DB or WAL devices **MUST** be 1:1.

Verification

- List the service:
  Example

  ```
  [ceph: root@host01 /]# ceph orch ls osd
  ```

- View the details of the node and devices:
  Example

  ```
  [ceph: root@host01 /]# ceph osd tree
  ```

- List the hosts, daemons, and processes:

  ```
  ceph orch ps --service_name=SERVICE_NAME
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph orch ps --service_name=osd
  ```

Advanced service specifications and filters for deploying OSDs

Use OSD service specifications to tell Ceph which storage devices become OSDs without specifying device names and paths.

For each device and each host, define a YAML or JSON file. For more deployment information, see “Deploying Ceph OSDs using service specifications” on page 597.

To create an OSD with non-collocated components on the same host, specify the different types of devices that you use. The devices should be on the same host.

The devices that you use to deploy OSDs must be supported by `libstoragemgmt`. For more information, see “Listing devices for Ceph OSD deployment” on page 588.

“General settings for OSD specifications” on page 593 provides the supported OSD settings and information.

Table 91: General settings for OSD specifications

OSD setting	Additional options	Description
`service_type:osd`	None	Mandatory to create OSDs.
`service_id`	None	Use the service name or identification you prefer. A set of OSDs is created using the specification file. This name is used to manage all the OSDs together and represent an Orchestrator service.

OSD setting	Additional options	Description
`placement`	**`host_pattern:`** A hostname pattern to limit the hosts to which this service should be applied. Set to * to apply to all hosts. The host pattern is matched by using `fnmatch`, which supports UNIX shell-style wildcards. **`label: OSD_HOST`** Apply only to hosts with the specified ceph `orch host` label. **`hosts`: HOST01, HOST02** An explicit list of hostnames to which the service is to be applied.	This is used to define the hosts on which the OSDs need to be deployed.
`selection of devices`	None	Allows the user to select specific devices for OSD services. Fine-grained selection allows the use of multiple OSD services for multiple classes of OSD devices. BlueStore OSDs have three components, which can be deployed together on one storage device, or across multiple devices. Only create BlueStore OSDs which have three components: **OSD data** Contains all the OSD data. **BlueStore Write-Ahead Log (WAL)** By default this is colocated with the DB device. It is rarely appropriate to specify a different device just for the WAL. **DB** BlueStore internal metadata, including omaps used for Ceph Object Gateway index pools. By default the WAL is colocated on this device. Use `block` for OSD payload data. By default this is the device used for all three components.
`data_devices`	None	Define the devices to deploy OSD. In this case, OSDs are created in a collocated schema. You can use filters to select devices and folders.

OSD setting	Additional options	Description
`wal_devices`	None	Define the devices used for WAL offloading. You can use filters to select devices and folders. If not specified the WAL is colocated on `data_devices`.
`db_devices`	None	Define the devices for DB offload. You can use the filters to select devices and folders. If not specified, the DB is colocated on `data_devices`.
`encrypted`	None	An optional parameter to encrypt information on the OSD. Input options: `True` or `False`.
`unmanaged`	None	An optional parameter, set to `False` by default. You can set it to `True` if you do not want the Orchestrator to manage the OSD service. Input options:`True` or `False`.
`block_wal_size`	None	User-defined value, in bytes.
`block_db_size`	None	User-defined value, in bytes.
`osds_per_device`	None	User-defined value for deploying more than one OSD per device. Most deployments should let this default to 1. Contact Red Hat Support for additional guidance if your OSD devices are 30 TB or larger or if your workload includes large numbers of very small S3 objects or many object versions.
`method`	None	An optional parameter to specify if an OSD is created with an LVM layer or not. Set to `raw` if you want to create OSDs on raw physical devices that do not include an LVM layer. If you have separate DB or WAL devices, the ratio of block to DB or WAL devices **MUST** be 1:1.

"Filters for specifying devices" on page 596 defines the filters that are used in conjunction with the `data_devices`, `wal_devices`, and `db_devices` settings that are defined in "General settings for OSD specifications" on page 593.

Table 92: Filters for specifying devices

Name of the filter	Description	Syntax	Example
Model	Target specific SKUs. You can get details of the model by running one of the following commands: – **`lsblk -o NAME,FSTYPE,LABEL,MOUNTPOINT,SIZE,MODEL`** – **`smartctl -i /DEVICE_PATH`** – **`lsscsi`**	Model: *`STORAGE_MODEL_NAME`*	Model: MC-55-44-XZ
Vendor	Target only storage devices from a specific manufacturer or vendor.	Vendor: *`STORAGE_VENDOR_NAME`*	Vendor: Vendor Cs
Size Specification	Match only storage devices that are an exact specific size. **Note:** Using an exact size is not recommended. Using a size range is usually the better strategy.	size: *`EXACT`*	size: 10G
Size Specification	Match storage devices within a size range. This is the recommended filter. For example, when deploying nominal 20TB HDDs, one might specify `size: 18:22`.	size: *`LOW:HIGH`*	size: 10G:40G
Size Specification	Match storage devices less than or equal to in size.	size: *`:HIGH`*	size: :10G
Size Specification	Match storage devices greater than or equal in size. For example, if your OSDs are currently nominal 20 TB HDDs but in the future you might use 24 TB replacement or expansion drives, you could specify `18T:` so that both the current and future storage devices are matched.	size: *`LOW:`*	size: 40G:

Name of the filter	Description	Syntax	Example
Rotational	Rotational attribute of the storage devices. – `1` matches rotating HDDs – `0` matches SSDs. This matches against the kernel's determination of the device type, which in most cases is accurate. There are certain storage layers that may require a `udev` rule to override the kernel's determination. For more information, contact Red Hat Support .	`rotational`:`0`or`1`	rotational: 0
All	Matches all the available storage devices. **Note:** Storage devices with existing partitions, even if not in active use, are considered `unavailable` by the orchestrator and must be zapped to be considered `available`.	all: true	all: true
Limiter	Optional. This is an optional limit to the number of storage devices that another filter will match. For example, if there are 4 unused SSDs on each server, but one of them is to be reserved for non-Ceph purposes. Most deployments do not benefit from this directive. For more information, contact Red Hat Support.	limit: *NUMBER*	limit: 2

Deploying Ceph OSDs using service specifications

An OSD service specification describes a cluster storage layout by using disk properties, allowing you to define OSD configurations without specifying exact device names and paths.

You can deploy the OSD for each device and each host by defining a YAML or JSON file.

For more information, see:

- “Adding OSDs” on page 241
- “Advanced service specifications and filters for deploying OSDs” on page 593

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Hosts are added to the cluster.
- All manager and monitor daemons are deployed.

Procedure

1. List the available devices.

```
ceph orch device ls --wide --refresh
```

2. On the monitor node, create the `osd_spec.yaml` file.

```
touch osd_spec.yaml
```

3. Edit the `osd_spec.yaml` file. Include the following details:

```
service_type: osd
service_id: SERVICE_ID
placement:
  host_pattern: '*'
data_devices:
  paths:
    - /DEVICE_PATH
```

> **Note:** Use a unique **`service_id`** for each OSD service specification.

The following examples start with common deployment scenarios and then progress to advanced deployment layouts.

Common customer scenarios

a. Deploy OSDs on all available devices:
Use this scenario when all hosts use the same storage layout and all available devices must be deployed as OSDs.

For example,

```
service_type: osd
service_id: all-available-devices
placement:
  host_pattern: '*'
data_devices:
  all: true
```

b. Deploy OSDs on a specific host and device:
Use this scenario to deploy an OSD on a specific device on a single host.

For example,

```
service_type: osd
service_id: osd_spec_host01
placement:
  hosts:
    - host01
data_devices:
  paths:
    - /dev/sdb
```

c. Deploy OSDs using HDD data devices with SSD DB devices:
Use this scenario to assign dedicated solid-state drives (SSDs) for the BlueStore DB and WAL while using HDDs for data storage. The SSDs can use a SATA, SAS, or NVMe interface. The `rotational: 0` filter matches all non-rotational devices, whatever their interface.

> **Important:** If you do not specify the **`block_db_size`** parameter, each OSD can use an entire DB device. This can happen when you add OSDs to a host or retrofit offload devices to existing OSDs. Specify the **`block_db_size`** parameter to limit the DB size of each OSD.

For example,

```
service_type: osd
service_id: hdd_with_ssd_db
placement:
  host_pattern: '*'
data_devices:
  rotational: 1
db_devices:
  rotational: 0
```

d. Deploy OSDs by using size filters:
Use this scenario when devices are filtered based on disk capacity.

For example,

```
service_type: osd
service_id: osd_spec_size
placement:
  host_pattern: '*'
data_devices:
  size: '2TB:'
db_devices:
  size: ':2TB'
```

Advanced scenarios

a. Advanced scenario with multiple OSD specifications:
Use this scenario to apply different OSD specifications to the same set of hosts.

For example,

```
service_type: osd
service_id: osd_spec_hdd
placement:
  host_pattern: '*'
data_devices:
  rotational: 1
db_devices:
  model: Model-name
  limit: 2
---
service_type: osd
service_id: osd_spec_ssd
placement:
  host_pattern: '*'
data_devices:
  model: Model-name
db_devices:
  vendor: Vendor-name
```

b. Advanced scenario with non-uniform nodes: This applies different OSD specs to different hosts depending on the host_pattern key.
For example,

```
service_type: osd
service_id: osd_spec_node_one_to_five
placement:
  host_pattern: 'node[1-5]'
data_devices:
```

```
  rotational: 1
db_devices:
  rotational: 0
---
service_type: osd
service_id: osd_spec_six_to_ten
placement:
  host_pattern: 'node[6-10]'
data_devices:
  model: Model-name
db_devices:
  model: Model-name
```

c. Advanced scenario with dedicated WAL and DB devices:
For example,

```
service_type: osd
service_id: osd_using_paths
placement:
  hosts:
    - host01
    - host02
data_devices:
  paths:
    - /dev/sdb
db_devices:
  paths:
    - /dev/sdc
wal_devices:
  paths:
    - /dev/sdd
```

d. Advanced scenario with multiple OSDs per device:

> **Note:** Deploy multiple OSDs per device only for supported and validated configurations.

For example,

```
service_type: osd
service_id: multiple_osds
placement:
  hosts:
    - host01
    - host02
osds_per_device: 4
data_devices:
  paths:
    - /dev/sdb
```

e. Advanced scenario: Deploy OSDs by using pre-created logical volumes:

```
service_type: osd
service_id: SERVICE_ID
placement:
  hosts:
    - HOSTNAME
data_devices:
  paths:
    - /DEVICE_PATH
db_devices:
  paths:
    - /DEVICE_PATH
```

For example,

```
service_type: osd
service_id: osd_spec
placement:
  hosts:
    - machine1
data_devices:
```

```
  paths:
    - /dev/vg_hdd/lv_hdd
db_devices:
  paths:
    - /dev/vg_nvme/lv_nvmes
```

4. From within the `cephadm` shell, mount the YAML file under a directory in the container.

```
cephadm shell --mount osd_spec.yaml:/var/lib/ceph/osd/osd_spec.yaml
```

5. Navigate to the directory.
   For example,

```
[ceph: root@host01 /]# cd /var/lib/ceph/osd/
```

   Before deploying OSDs, do a dry run.

> **Note:** Always run a dry run before applying an OSD service specification. This step previews the deployment without deploying any daemons.

```
[ceph: root@host01 osd]# ceph orch apply -i osd_spec.yaml --dry-run
```

6. Deploy OSDs using service specification.

```
ceph orch apply -i FILE_NAME.yaml
```

   For example,

```
[ceph: root@host01 osd]# ceph orch apply -i osd_spec.yaml
```

Verification

Verify in any of the following ways, from the `cephadm` shell:

- List the service.

```
ceph orch ls osd
```

- View the details of the node and devices.

```
ceph osd tree
```

Removing OSDs

You can remove the OSD from a cluster by using `cephadm`.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage.
- Hosts are added to the cluster.
- Ceph Monitor, Ceph Manager, and Ceph OSD daemons are deployed on the storage cluster.

About this task

Removing an OSD from a cluster involves two steps:

1. Evacuates all placement groups (PGs) from the cluster.
2. Removes the PG-free OSDs from the cluster.

The **--zap** option removed the volume groups, logical volumes, and the LVM metadata.

Procedure

1. Log into the `cephadm` shell.
   For example,

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Check the device and the node from which the OSD has to be removed.

   ```
   ceph osd tree
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph osd tree
   ```

3. Remove the OSD.
   You can either remove a specific OSD from the storage cluster or multiple OSDs from a specific node.
   - Remove the OSD from the storage cluster.

     > **Important:** Removing the OSD from the storage cluster without an option, such as `--replace`, removes the device completely from the storage cluster completely. If you want to use the same device for deploying OSDs, first zap the device before adding it to the storage cluster.

     ```
     ceph orch osd rm OSD_ID [--replace] [--force] --zap
     ```

     For example,

     ```
     [ceph: root@host01 /]# ceph orch osd rm 0 --zap
     ```

   - Remove multiple OSDs from a specific node.

     ```
     ceph orch osd rm OSD_ID OSD_ID --zap
     ```

     For example,

     ```
     [ceph: root@host01 /]# ceph orch osd rm 2 5 --zap
     ```

4. Use the **osd rm status** command to check the removal status.
   For example,

   ```
   [ceph: root@host01 /]# ceph orch osd rm status
   OSD  HOST   STATE                    PGS  REPLACE  FORCE  ZAP   DRAIN STARTED AT

   9    host01 done, waiting for purge    0  False    False  True  2023-06-06
   17:50:50.525690
   10   host03 done, waiting for purge    0  False    False  True  2023-06-06
   17:49:38.731533
   11   host02 done, waiting for purge    0  False    False  True  2023-06-06
   17:48:36.641105
   ```

   When no PGs are left on the OSD, it is decommissioned and removed from the cluster.

What to do next

Verify the details of the devices and the nodes from which the Ceph OSDs are removed.

```
ceph osd tree
```

Related information

Deploying Ceph OSDs on all available devices

Deploying Ceph OSDs on specific devices and hosts
Zapping devices for Ceph OSD deployment

Replacing the OSDs

When disks fail, you can replace the physical storage device and reuse the same OSD ID to avoid having to reconfigure the CRUSH map.

You can replace the OSDs from the cluster by preserving the OSD ID using the **`ceph orch rm`** command.

> **Note:** If you want to replace a single OSD, Deploying Ceph OSDs on specific devices and hosts . If you want to deploy OSDs on all available devices, Deploying Ceph OSDs on all available devices.

The OSD is not permanently removed from the CRUSH hierarchy, but is assigned the `destroyed` flag. This flag is used to determine the OSD IDs that can be reused in the next OSD deployment. The `destroyed` flag is used to determine which OSD id is reused in the next OSD deployment.

If you use OSD specification for deployment, your newly added disk is assigned the OSD ID of their replaced counterparts.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Hosts are added to the cluster.
- Monitor, Manager, and OSD daemons are deployed on the storage cluster.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Check the device and the node from which the OSD has to be replaced:

   Example

   ```
   [ceph: root@host01 /]# ceph osd tree
   ```

3. Set the OSD service specification to **`unmanaged`** state:

   > **Note:** Set the **`unmanaged`** parameter to `true` in the OSD service specification before removal. If you skip this step, `cephadm` may automatically redeploy OSDs, causing conflicts between orchestrator-driven deployment and OSD removal.

   Example

   ```
   service_type:osd
   service_id:<service_id>
   unmanaged:true
   ```

4. Apply the updated specification:

   ```
   ceph orch apply -i <osds.yaml>
   ```

5. Run the **`orch ls ceph`** command to confirm that the OSD service is set to the unmanaged state.
6. Replace the OSD:

> **Important:** If the storage cluster has `health_warn` or other errors associated with it, check and try to fix any errors before replacing the OSD to avoid data loss.

Syntax

```
ceph orch osd rm OSD_ID --replace
```

Example

```
[ceph: root@host01 /]# ceph orch osd rm 0 --replace
```

7. Check the status of the OSD replacement:

   Example

   ```
   [ceph: root@host01 /]# ceph orch osd rm status
   ```

8. Set the OSD service specification back to managed state. After you replace the disk, set the OSD service back to a **managed** state so that `cephadm` can resume orchestration.
   a. Update the OSD service specification.

      ```
      unmanaged: false
      ```

   b. Apply the updated specification. This step allows `cephadm` to redeploy the OSD on the new device while reusing the existing OSD ID.

      ```
      ceph orch apply -i <osds.yaml>
      ```

Example

Verification

- Verify the details of the devices and the nodes from which the Ceph OSDs are replaced:

  Example

  ```
  [ceph: root@host01 /]# ceph osd tree
  ```

  You will see an OSD with the same id as the one you replaced running on the same host.

Reference

- *Deploying Ceph OSDs on all available devices*
- *Deploying Ceph OSDs on specific devices and hosts*

Replacing the OSDs with pre-created LVM

After purging the OSD with the **`ceph-volume lvm zap`** command, if the directory is not present, then you can replace the OSDs with the OSD service specification file with the pre-created LVM.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Failed OSD

Procedure

1. Log into the `cephadm` shell:

Example

```
[root@host01 ~]# cephadm shell
```

2. Set the OSD service to an **unmanaged** state to prevent `cephadm` from automatically redeploying OSDs during removal.

> **Note:** If you skip this step, `cephadm` may redeploy OSDs automatically, causing conflicts with the removal operation.

Example

```
service_type: osd
service_id: <service_id>
unmanaged: true
```

3. Apply the specification:

Example

```
ceph orch apply -i <osds.yaml>
```

4. Run the **`orch ls ceph`** command to confirm that the OSD service is set to the unmanaged state.
5. Remove the OSD:

Syntax

```
ceph orch osd rm OSD_ID [--replace]
```

Example

```
[ceph: root@host01 /]# ceph orch osd rm 8 --replace
Scheduled OSD(s) for removal
```

6. Verify the OSD is destroyed:

Example

```
[ceph: root@host01 /]# ceph osd tree

ID   CLASS  WEIGHT   TYPE NAME        STATUS     REWEIGHT  PRI-AFF
-1         0.32297  root default
-9         0.05177      host host10
 3    hdd  0.01520          osd.3           up   1.00000  1.00000
13    hdd  0.02489          osd.13          up   1.00000  1.00000
17    hdd  0.01169          osd.17          up   1.00000  1.00000
-13         0.05177      host host11
 2    hdd  0.01520          osd.2           up   1.00000  1.00000
15    hdd  0.02489          osd.15          up   1.00000  1.00000
19    hdd  0.01169          osd.19          up   1.00000  1.00000
-7         0.05835      host host12
20    hdd  0.01459          osd.20          up   1.00000  1.00000
21    hdd  0.01459          osd.21          up   1.00000  1.00000
22    hdd  0.01459          osd.22          up   1.00000  1.00000
23    hdd  0.01459          osd.23          up   1.00000  1.00000
-5         0.03827      host host04
 1    hdd  0.01169          osd.1           up   1.00000  1.00000
 6    hdd  0.01129          osd.6           up   1.00000  1.00000
 7    hdd  0.00749          osd.7           up   1.00000  1.00000
 9    hdd  0.00780          osd.9           up   1.00000  1.00000
-3         0.03816      host host05
 0    hdd  0.01169          osd.0           up   1.00000  1.00000
 8    hdd  0.01129          osd.8    destroyed         0  1.00000
12    hdd  0.00749          osd.12          up   1.00000  1.00000
16    hdd  0.00769          osd.16          up   1.00000  1.00000
-15         0.04237      host host06
 5    hdd  0.01239          osd.5           up   1.00000  1.00000
10    hdd  0.01540          osd.10          up   1.00000  1.00000
11    hdd  0.01459          osd.11          up   1.00000  1.00000
```

```
 -11         0.04227      host host07
  4    hdd  0.01239          osd.4           up   1.00000  1.00000
 14    hdd  0.01529          osd.14          up   1.00000  1.00000
 18    hdd  0.01459          osd.18          up   1.00000  1.00000
```

7. Zap and remove the OSD using the `ceph-volume` command:

   Syntax

   ```
   ceph-volume lvm zap --osd-id OSD_ID
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph-volume lvm zap --osd-id 8

   Zapping: /dev/vg1/data-lv2
   Closing encrypted path /dev/mapper/l4D6ql-Prji-IzH4-dfhF-xzuf-5ETl-jNRcXC
   Running command: /usr/sbin/cryptsetup remove /dev/mapper/l4D6ql-Prji-IzH4-dfhF-
  xzuf-5ETl-jNRcXC
   Running command: /usr/bin/dd if=/dev/zero of=/dev/vg1/data-lv2 bs=1M count=10
  conv=fsync
   stderr: 10+0 records in
   10+0 records out
   stderr: 10485760 bytes (10 MB, 10 MiB) copied, 0.034742 s, 302 MB/s
   Zapping successful for OSD: 8
   ```

8. Check the OSD topology:

   Example

   ```
   [ceph: root@host01 /]# ceph-volume lvm list
   ```

9. Recreate the OSD with a specification file corresponding to that specific OSD topology:

   Create the required volume group (VG) and logical volume (LV) on the replacement device before applying the OSD service specification. For more information about creating Logical Volume Manager (LVM) volumes, see the Red Hat Enterprise Linux 10 LVM documentation. If your hosts run a different RHEL version, see the documentation for that version.

   If an OSD service specification already exists for the service, review the existing specification instead of creating a new file. If the specification already contains the required VG and LV paths, reuse it. If the paths are different, update the existing specification with the new paths. Create a new specification only when an appropriate OSD service specification does not already exist.

   Example

   ```
   [ceph: root@host01 /]# cat osd.yml
   service_type: osd
   service_id: osd_service
   placement:
     hosts:
     - host03
   data_devices:
     paths:
     - /dev/vg1/data-lv2
   db_devices:
     paths:
     - /dev/vg1/db-lv1
   ```

   Note: Do not replace an existing OSD service specification with a file that contains only the replacement device information. Preserve the existing service definition and update only the required device paths so that future OSD deployments continue to be managed correctly.

10. Apply the updated specification file:

    Example

    ```
    ceph orch apply -i osd.yml
    ```

11. Set the OSD service specification back to managed state. After you replace the disk, set the OSD service back to a **managed** state so that `cephadm` can resume orchestration.
    a. Update the OSD service specification.

    ```
    unmanaged: false
    ```

    b. Apply the updated specification. This step allows `cephadm` to redeploy the OSD on the new device while reusing the existing OSD ID.

    ```
    ceph orch apply -i <osds.yaml>
    ```

12. Verify the OSD is back:

    Example

    ```
    [ceph: root@host01 /]# ceph -s
    [ceph: root@host01 /]# ceph osd tree
    ```

Replacing the OSDs in a non-colocated scenario

When the an OSD fails in a non-colocated scenario, you can replace the WAL/DB devices. The procedure is the same for DB and WAL devices. You need to edit the `paths` under `db_devices` for DB devices and `paths` under `wal_devices` for WAL devices.

Prerequisites

- A running Red Hat Ceph Storage.
- Daemons are non-colocated.
- Failed OSD

Procedure

1. Identify the devices in the cluster.

   Example

   ```
   [root@host01 ~]# lsblk

   NAME
                   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
   sda
                    8:0    0   20G  0 disk
   ├─sda1
                     8:1    0    1G  0 part /boot
   └─sda2
                     8:2    0   19G  0 part
     ├─rhel-root
                  253:0    0   17G  0 lvm  /
     └─rhel-swap
                  253:1    0    2G  0 lvm  [SWAP]
   sdb
                    8:16   0   10G  0 disk
    └─ceph--5726d3e9--4fdb--4eda--b56a--3e0df88d663f-osd--
   block--3ceb89ec--87ef--46b4--99c6--2a56bac09ff0 253:2    0   10G  0 lvm
   sdc
                    8:32   0   10G  0 disk
    └─ceph--d7c9ab50--f5c0--4be0--a8fd--e0313115f65c-osd--block--37c370df--1263--487f--
   a476--08e28bdbcd3c 253:4    0   10G  0 lvm
   sdd
                    8:48   0   10G  0 disk
    ├─ceph--1774f992--44f9--4e78--be7b--b403057cf5c3-osd--
   db--31b20150--4cbc--4c2c--9c8f--6f624f3bfd89    253:7    0  2.5G  0 lvm
    └─ceph--1774f992--44f9--4e78--be7b--b403057cf5c3-osd--db--1bee5101--dbab--4155--
   a02c--e5a747d38a56    253:9    0  2.5G  0 lvm
   sde
                    8:64   0   10G  0 disk
   sdf
                    8:80   0   10G  0 disk
    └─ceph--412ee99b--4303--4199--930a--0d976e1599a2-osd--
   ```

```
block--3a99af02--7c73--4236--9879--1fad1fe6203d 253:6    0   10G  0 lvm
 sdg
                 8:96   0   10G  0 disk
 └─ceph--316ca066--aeb6--46e1--8c57--f12f279467b4-osd--
block--58475365--51e7--42f2--9681--e0c921947ae6 253:8    0   10G  0 lvm
 sdh
                 8:112  0   10G  0 disk
 ├─ceph--d7064874--66cb--4a77--a7c2--8aa0b0125c3c-osd--db--0dfe6eca--
ba58--438a--9510--d96e6814d853    253:3    0    5G  0 lvm
 └─ceph--d7064874--66cb--4a77--a7c2--8aa0b0125c3c-osd--
db--26b70c30--8817--45de--8843--4c0932ad2429    253:5    0    5G  0 lvm
 sr0
```

2. Log into the `cephadm` shell.

   Example

```
[root@host01 ~]# cephadm shell
```

3. Identify the OSDs and their DB device.

   Example

```
 [ceph: root@host01 /]# ceph-volume lvm list /dev/sdh

 ====== osd.2 =======

   [db]          /dev/ceph-d7064874-66cb-4a77-a7c2-8aa0b0125c3c/osd-db-0dfe6eca-
ba58-438a-9510-d96e6814d853

       block device              /dev/ceph-5726d3e9-4fdb-4eda-b56a-3e0df88d663f/osd-
block-3ceb89ec-87ef-46b4-99c6-2a56bac09ff0
       block uuid                GkWLoo-f0jd-Apj2-Zmwj-ce0h-OY6J-UuW8aD
       cephx lockbox secret
       cluster fsid              fa0bd9dc-e4c4-11ed-8db4-001a4a00046e
       cluster name              ceph
       crush device class
       db device                 /dev/ceph-d7064874-66cb-4a77-a7c2-8aa0b0125c3c/osd-
db-0dfe6eca-ba58-438a-9510-d96e6814d853
       db uuid                   6gSPoc-L39h-afN3-rDl6-kozT-AX9S-XR20xM
       encrypted                 0
       osd fsid                  3ceb89ec-87ef-46b4-99c6-2a56bac09ff0
       osd id                    2
       osdspec affinity          non-colocated
       type                      db
       vdo                       0
       devices                   /dev/sdh

 ====== osd.5 =======

   [db]          /dev/ceph-d7064874-66cb-4a77-a7c2-8aa0b0125c3c/osd-
db-26b70c30-8817-45de-8843-4c0932ad2429

       block device              /dev/ceph-d7c9ab50-f5c0-4be0-a8fd-e0313115f65c/osd-
block-37c370df-1263-487f-a476-08e28bdbcd3c
       block uuid                Eay3I7-fcz5-AWvp-kRcI-mJaH-n03V-Zr0wmJ
       cephx lockbox secret
       cluster fsid              fa0bd9dc-e4c4-11ed-8db4-001a4a00046e
       cluster name              ceph
       crush device class
       db device                 /dev/ceph-d7064874-66cb-4a77-a7c2-8aa0b0125c3c/osd-
db-26b70c30-8817-45de-8843-4c0932ad2429
       db uuid                   mwSohP-u72r-DHcT-BPka-piwA-lSwx-w24N0M
       encrypted                 0
       osd fsid                  37c370df-1263-487f-a476-08e28bdbcd3c
       osd id                    5
       osdspec affinity          non-colocated
       type                      db
       vdo                       0
       devices                   /dev/sdh
```

4. In the `osds.yaml` file, set `unmanaged` parameter to `true`, else `cephadm` redeploys the OSDs.

   Example

```
 [ceph: root@host01 /]# cat osds.yml
 service_type: osd
```

```
service_id: non-colocated
unmanaged: true
placement:
  host_pattern: 'ceph*'
data_devices:
  paths:
  - /dev/sdb
  - /dev/sdc
  - /dev/sdf
  - /dev/sdg
db_devices:
  paths:
  - /dev/sdd
  - /dev/sdh
```

5. Apply the updated specification file.

   Example

```
[ceph: root@host01 /]# ceph orch apply -i osds.yml

Scheduled osd.non-colocated update...
```

6. Check the status.

   Example

```
[ceph: root@host01 /]# ceph orch ls

NAME           PORTS        RUNNING  REFRESHED  AGE  PLACEMENT
alertmanager   ?:9093,9094      1/1  9m ago     4d   count:1
crash                           3/4  4d ago     4d   *
grafana        ?:3000           1/1  9m ago     4d   count:1
mgr                             1/2  4d ago     4d   count:2
mon                             3/5  4d ago     4d   count:5
node-exporter  ?:9100           3/4  4d ago     4d   *
osd.non-colocated                 8  4d ago     5s   <unmanaged>
prometheus     ?:9095           1/1  9m ago     4d   count:1
```

7. Remove the OSDs. Ensure to use the `--zap` option to remove hte backend services and the `--replace` option to retain the OSD IDs.

   Example

```
[ceph: root@host01 /]# ceph orch osd rm 2 5 --zap --replace
Scheduled OSD(s) for removal
```

8. Check the status.

   Example

```
[ceph: root@host01 /]# ceph osd df tree | egrep -i "ID|host02|osd.2|osd.5"

ID  CLASS  WEIGHT   REWEIGHT  SIZE     RAW USE  DATA     OMAP  META     AVAIL   %USE
 VAR   PGS  STATUS     TYPE NAME
-5         0.04877         -   55 GiB   15 GiB  4.1 MiB   0 B   60 MiB  40 GiB  27.27
1.17    -                host02
2    hdd  0.01219   1.00000   15 GiB  5.0 GiB  996 KiB   0 B   15 MiB  10 GiB  33.33
1.43    0  destroyed          osd.2
5    hdd  0.01219   1.00000   15 GiB  5.0 GiB  1.0 MiB   0 B   15 MiB  10 GiB  33.33
1.43    0  destroyed          osd.5
```

9. Edit the `osds.yaml` specification file to change `unmanaged` parameter to `false` and replace the path to the DB device if it has changed after the device got physically replaced.

   Example

```
[ceph: root@host01 /]# cat osds.yml
service_type: osd
service_id: non-colocated
unmanaged: false
placement:
  host_pattern: 'ceph01*'
```

```
data_devices:
  paths:
  - /dev/sdb
  - /dev/sdc
  - /dev/sdf
  - /dev/sdg
db_devices:
  paths:
  - /dev/sdd
  - /dev/sde
```

In the above example, `/dev/sdh` is replaced with `/dev/sde`.

> **Important:** If you use the same host specification file to replace the faulty DB device on a single OSD node, modify the `host_pattern` option to specify only the OSD node, else the deployment fails and you cannot find the new DB device on other hosts.

10. Reapply the specification file with the `--dry-run` option to ensure the OSDs shall be deployed with the new DB device.

    Example

```
[ceph: root@host01 /]# ceph orch apply -i osds.yml --dry-run
WARNING! Dry-Runs are snapshots of a certain point in time and are bound
to the current inventory setup. If any of these conditions change, the
preview will be invalid. Please make sure to have a minimal
timeframe between planning and applying the specs.
####################
SERVICESPEC PREVIEWS
####################
+---------+------+--------+-------------+
|SERVICE  |NAME  |ADD_TO  |REMOVE_FROM  |
+---------+------+--------+-------------+
+---------+------+--------+-------------+
################
OSDSPEC PREVIEWS
################
+---------+-------+-------+----------+----------+-----+
|SERVICE  |NAME   |HOST   |DATA      |DB        |WAL  |
+---------+-------+-------+----------+----------+-----+
|osd      |non-colocated  |host02  |/dev/sdb  |/dev/sde  |-    |
|osd      |non-colocated  |host02  |/dev/sdc  |/dev/sde  |-    |
+---------+-------+-------+----------+----------+-----+
```

11. Apply the specification file:

    Example

```
[ceph: root@host01 /]# ceph orch apply -i osds.yml
Scheduled osd.non-colocated update...
```

12. Check the OSDs are redeployed.

    Example

```
[ceph: root@host01 /]# ceph osd df tree | egrep -i "ID|host02|osd.2|osd.5"

ID  CLASS  WEIGHT   REWEIGHT  SIZE     RAW USE  DATA     OMAP  META     AVAIL   %USE
VAR   PGS  STATUS  TYPE NAME
-5 0.04877 - 55 GiB 15 GiB 4.5 MiB 0 B 60 MiB 40 GiB 27.27 1.17 - host host02
2 hdd 0.01219 1.00000 15 GiB 5.0 GiB 1.1 MiB 0 B 15 MiB 10 GiB 33.33 1.43 0 up osd.2
5 hdd 0.01219 1.00000 15 GiB 5.0 GiB 1.1 MiB 0 B 15 MiB 10 GiB 33.33 1.43 0 up osd.5
```

-

Verification

– From the OSD host where the OSDS are redeployed, verify if they are on the new DB device.

  Example

```
 [ceph: root@host01 /]# ceph-volume lvm list /dev/sde

 ====== osd.2 =======

   [db]          /dev/ceph-15ce813a-8a4c-46d9-ad99-7e0845baf15e/osd-
db-1998a02e-5e67-42a9-b057-e02c22bbf461

       block device              /dev/ceph-a4afcb78-c804-4daf-b78f-3c7ad1ed0379/osd-
block-564b3d2f-0f85-4289-899a-9f98a2641979
       block uuid                ITPVPa-CCQ5-BbFa-FZCn-FeYt-c5N4-ssdU41
       cephx lockbox secret
       cluster fsid              fa0bd9dc-e4c4-11ed-8db4-001a4a00046e
       cluster name              ceph
       crush device class
       db device                 /dev/ceph-15ce813a-8a4c-46d9-ad99-7e0845baf15e/osd-
db-1998a02e-5e67-42a9-b057-e02c22bbf461
       db uuid                   HF1bYb-fTK7-0dcB-CHzW-xvNn-dCym-KKdU5e
       encrypted                 0
       osd fsid                  564b3d2f-0f85-4289-899a-9f98a2641979
       osd id                    2
       osdspec affinity          non-colocated
       type                      db
       vdo                       0
       devices                   /dev/sde

 ====== osd.5 =======

   [db]          /dev/ceph-15ce813a-8a4c-46d9-ad99-7e0845baf15e/osd-
db-6c154191-846d-4e63-8c57-fc4b99e182bd

       block device              /dev/ceph-b37c8310-77f9-4163-964b-f17b4c29c537/osd-
block-b42a4f1f-8e19-4416-a874-6ff5d305d97f
       block uuid                0LuPoz-ao7S-UL2t-BDIs-C9pl-ct8J-xh5ep4
       cephx lockbox secret
       cluster fsid              fa0bd9dc-e4c4-11ed-8db4-001a4a00046e
       cluster name              ceph
       crush device class
       db device                 /dev/ceph-15ce813a-8a4c-46d9-ad99-7e0845baf15e/osd-
db-6c154191-846d-4e63-8c57-fc4b99e182bd
       db uuid                   SvmXms-iWkj-MTG7-VnJj-r5Mo-Moiw-MsbqVD
       encrypted                 0
       osd fsid                  b42a4f1f-8e19-4416-a874-6ff5d305d97f
       osd id                    5
       osdspec affinity          non-colocated
       type                      db
       vdo                       0
       devices                   /dev/sde
```

Stopping the removal of the OSDs

You can stop the removal of only the OSDs that are queued for removal. This resets the initial state of the OSD and takes it off the removal queue.

If the OSD is in the process of removal, then you cannot stop the process.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Hosts are added to the cluster.
- Monitor, Manager and OSD daemons are deployed on the cluster.
- Remove OSD process initiated.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Check the device and the node from which the OSD was initiated to be removed:

Example

```
[ceph: root@host01 /]# ceph osd tree
```

3. Stop the removal of the queued OSD:

   Syntax

   ```
   ceph orch osd rm stop OSD_ID
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph orch osd rm stop 0
   ```

4. Check the status of the OSD removal:

   Example

   ```
   [ceph: root@host01 /]# ceph orch osd rm status
   ```

Verification

- Verify the details of the devices and the nodes from which the Ceph OSDs were queued for removal:

  Example

  ```
  [ceph: root@host01 /]# ceph osd tree
  ```

Reference

- *Removing the OSD daemons*

Activating the OSDs

You can activate the OSDs in the cluster in cases where the operating system of the host was reinstalled.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Hosts are added to the cluster.
- Monitor, Manager and OSD daemons are deployed on the storage cluster.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. After the operating system of the host is reinstalled, activate the OSDs:

   Syntax

   ```
   ceph cephadm osd activate HOSTNAME
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph cephadm osd activate host03
   ```

Verification

- List the service:

  Example

  ```
  [ceph: root@host01 /]# ceph orch ls
  ```

- List the hosts, daemons, and processes:

  Syntax

  ```
  ceph orch ps --service_name=SERVICE_NAME
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph orch ps --service_name=osd
  ```

Observing the data migration

When you add or remove an OSD to the CRUSH map, Ceph begins rebalancing the data by migrating placement groups to the new or existing OSD(s).

You can observe the data migration by using the **ceph-w** command.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Recently added or removed an OSD.

Procedure

1. To observe the data migration:

   Example

   ```
   [ceph: root@host01 /]# ceph -w
   ```

2. Watch as the placement group states change from `active+clean` to `active, some degraded objects`, and finally `active+clean` when migration completes.
3. To exit the utility, press `Ctrl + C`.

Recalculating the placement groups

Adjusting the number of placement groups can be done online. Recalculating is not only a recalculation of the PG numbers, but will involve data relocation, which will be a lengthy process. However, the data availability will be maintained at any time.

Placement groups (PGs) define the spread of any pool data across the available OSDs. A placement group is built upon the given redundancy algorithm to be used. For a 3-way replication, the redundancy is defined to use three different OSDs. For erasure-coded pools, the number of OSDs to use is defined by the number of chunks.

When defining a pool the number of placement groups defines the grade of granularity the data is spread with across all available OSDs. The higher the number the better the equalization of capacity load can be. However, since handling the placement groups is also important in case of reconstruction of data, the number is significant to be carefully chosen upfront. To support calculation a tool is available to produce agile environments.

During the lifetime of a storage cluster, a pool can grow above the initially anticipated limits. With the growing number of drives a recalculation is recommended. The number of placement groups per OSD should be around 100. When adding more OSDs to the storage cluster the number of PGs per OSD will lower over time. Starting with 120 drives initially in the storage cluster and setting the `pg_num` of the pool to 4000 will end up in 100 PGs per OSD, given with the replication factor of three. Over time, when growing to ten times the number of OSDs, the number of PGs per OSD will go down to ten only. Because a small number of PGs per OSD will tend to an unevenly distributed capacity, consider adjusting the PGs per pool.

Very high numbers of PGs per OSD should be avoided, because reconstruction of all PGs on a failed OSD will start at once. A high number of IOPS is required to perform reconstruction in a timely manner, which might not be available. This would lead to deep I/O queues and high latency rendering the storage cluster unusable or will result in long healing times.

Reference

- To calculate the values given by use case, see the *PG calculator*.
- *Erasure Code Pools*

Managing the monitoring stack

As a storage administrator, you can use the Ceph Orchestrator with `cephadm` in the backend to deploy monitoring and alerting stack. The monitoring stack consists of Prometheus, Prometheus exporters, Prometheus Alertmanager, and Grafana. Users need to either define these services with `cephadm` in a YAML configuration file, or they can use the command line interface to deploy them. When multiple services of the same type are deployed, a highly-available setup is deployed. The node exporter is an exception to this rule.

> **Note:** Red Hat Ceph Storage does not support custom images for deploying monitoring services such as Prometheus, Grafana, Alertmanager, and node-exporter.

The following monitoring services can be deployed with `cephadm`:

- Prometheus is the monitoring and alerting toolkit. It collects the data provided by Prometheus exporters and fires preconfigured alerts if predefined thresholds have been reached. The Prometheus manager module provides a Prometheus exporter to pass on Ceph performance counters from the collection point in `ceph-mgr`.

The Prometheus configuration, including scrape targets, such as metrics providing daemons, is set up automatically by `cephadm`. `cephadm` also deploys a list of default alerts, for example, health error, 10% OSDs down, or pgs inactive.

- Alertmanager handles alerts sent by the Prometheus server. It deduplicates, groups, and routes the alerts to the correct receiver. By default, the Ceph dashboard is automatically configured as the receiver. The Alertmanager handles alerts sent by the Prometheus server. Alerts can be silenced using the Alertmanager, but silences can also be managed using the Ceph Dashboard.
- Grafana is a visualization and alerting software. The alerting functionality of Grafana is not used by this monitoring stack. For alerting, the Alertmanager is used.

By default, traffic to Grafana is encrypted with TLS. You can either supply your own TLS certificate or use a self-signed one. If no custom certificate has been configured before Grafana has been deployed, then a self-signed certificate is automatically created and configured for Grafana. Custom certificates for Grafana can be configured using the following commands:

```
ceph config-key set mgr/cephadm/HOSTNAME/grafana_key -i PRESENT_WORKING_DIRECTORY/key.pem
ceph config-key set mgr/cephadm/HOSTNAME/grafana_crt -i PRESENT_WORKING_DIRECTORY/
certificate.pem
```

Node exporter is an exporter for Prometheus which provides data about the node on which it is installed. It is recommended to install the node exporter on all nodes. This can be done using the `monitoring.yml` file with the node-exporter service type.

Deploying the monitoring stack

You can deploy the monitoring stack using the service specification in YAML file format. All the monitoring services can have the network and port they bind to configured in the YAML file.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.

– Root-level access to the nodes.

About this task

Deploy the monitoring stack using the Ceph orchestrator. The monitoring stack consists of Prometheus, Prometheus exporters, Prometheus Alertmanager, Grafana, and Ceph Exporter. Ceph Dashboard makes use of these components to store and visualize detailed metrics on cluster usage and performance.

Procedure

1. Enable the Prometheus module in the Ceph Manager daemon. This exposes the internal Ceph metrics so that Prometheus can read them.

   ```
   ceph mgr module enable prometheus
   ```

   Important: Ensure this command is run before Prometheus is deployed. If the command was not run before the deployment, you must redeploy Prometheus to update the configuration.

   ```
   ceph orch redeploy prometheus
   ```

2. Navigate to the following directory.

   ```
   cd /var/lib/ceph/DAEMON_PATH/
   ```

   For example,

   ```
   [ceph: root@host01 mds/]# cd /var/lib/ceph/monitoring/
   ```

   Note:

   Note: If the directory `monitoring` does not exist, create it.

3. Create the `monitoring.yml` file.

   ```
   touch monitoring.yml
   ```

4. Edit the specification file with a content similar to the following example.
   For example,

```
service_type: prometheus
service_name: prometheus
placement:
  hosts:
  - host01
networks:
- 192.169.142.0/24
---
service_type: node-exporter
---
service_type: alertmanager
service_name: alertmanager
placement:
  hosts:
  - host01
networks:
- 192.169.142.0/24
---
service_type: grafana
service_name: grafana
placement:
  hosts:
  - host01
networks:
- 192.169.142.0/24
---
service_type: ceph-exporter
```

Important: Ensure the monitoring stack components **alertmanager**, **prometheus**, and **grafana** are deployed on the same host. The **node-exporter** and **ceph-exporter** components should be deployed on all the hosts.

5. Apply monitoring services.

```
ceph orch apply -i MONITORING_STACK_FILE.yml
```

For example,

```
[ceph: root@host01 monitoring]# ceph orch apply -i monitoring.yml
```

What to do next

Important: Prometheus, Grafana, and the Ceph dashboard are all automatically configured to talk to each other, resulting in a fully functional Grafana integration in the Ceph dashboard.

- List the service.

```
ceph orch ls
```

- List the hosts, daemons, and processes.

```
ceph orch ps --service_name=SERVICE_NAME
```

For example,

```
[ceph: root@host01 /]# ceph orch ps --service_name=prometheus
```

Removing the monitoring stack

You can remove the monitoring stack using the **ceph orch rm** command.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Monitoring stack deployed on the cluster.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Use the `ceph orch rm` command to remove the monitoring stack:

   Syntax

   ```
   ceph orch rm SERVICE_NAME --force
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph orch rm grafana
   [ceph: root@host01 /]# ceph orch rm prometheus
   [ceph: root@host01 /]# ceph orch rm node-exporter
   [ceph: root@host01 /]# ceph orch rm ceph-exporter
   [ceph: root@host01 /]# ceph orch rm alertmanager
   [ceph: root@host01 /]# ceph mgr module disable prometheus
   ```

3. Check the status of the process:

   Example

   ```
   [ceph: root@host01 /]# ceph orch status
   ```

Verification

- List the service:

  Example

  ```
  [ceph: root@host01 /]# ceph orch ls
  ```

- List the hosts, daemons, and processes:

  Syntax

  ```
  ceph orch ps
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph orch ps
  ```

Reference

- *Deploying the monitoring stack*

Basic client setup

As a storage administrator, you have to set up Red Hat Ceph Storage client machines with basic configuration to interact with the storage cluster.

Most client machines only need the `ceph-common package` and its dependencies installed. It will supply the basic `ceph` and `rados` commands, as well as other commands like `mount.ceph` and `rbd`.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to all the nodes.
- Hosts are added to the cluster.
- All manager, monitor and OSD daemons are deployed.

Configuring file setup on client machines

Client machines generally need a smaller configuration file than a full-fledged storage cluster member. You can generate a minimal configuration file which can give details to clients to reach the Ceph monitors.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root access to the nodes.

Procedure

1. On the node where you want to set up the files, create a directory `ceph` in the `/etc` folder:

   Example

   ```
   [root@host01 ~]# mkdir /etc/ceph/
   ```

2. Navigate to `/etc/ceph` directory:

   Example

   ```
   [root@host01 ~]# cd /etc/ceph/
   ```

3. Generate the configuration file in the `ceph` directory:

   Example

   ```
   [root@host01 ceph]# ceph config generate-minimal-conf

   # minimal ceph.conf for 417b1d7a-a0e6-11eb-b940-001a4a000740
   [global]
           fsid = 417b1d7a-a0e6-11eb-b940-001a4a000740
           mon_host = [v2:10.74.249.41:3300/0,v1:10.74.249.41:6789/0]
   ```

   The contents of this file should be installed in `/etc/ceph/ceph.conf` path. You can use this configuration file to reach the Ceph monitors.

Setting-up keyring on client machines

Most Ceph clusters are run with the authentication enabled, and the client needs the keys in order to communicate with cluster machines. You can generate the keyring which can give details to clients to reach the Ceph monitors.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root access to the nodes.

Procedure

1. On the node where you want to set up the keyring, create a directory `ceph` in the `/etc` folder:

   Example

```
[root@host01 ~]# mkdir /etc/ceph/
```

2. Navigate to `/etc/ceph` directory in the `ceph` directory:

   Example

   ```
   [root@host01 ~]# cd /etc/ceph/
   ```

3. Generate the keyring for the client:

   Syntax

   ```
   ceph auth get-or-create client.CLIENT_NAME -o /etc/ceph/NAME_OF_THE_FILE
   ```

   Example

   ```
   [root@host01 ceph]# ceph auth get-or-create client.fs -o /etc/ceph/ceph.keyring
   ```

4. Verify the output in the `ceph.keyring` file:

   Example

   ```
   [root@host01 ceph]# cat ceph.keyring

   [client.fs]
           key = AQAvoH5gkUCsExAATz3xCBLd4n6B6jRv+Z7CVQ==
   ```

   The resulting output should be put into a keyring file, for example `/etc/ceph/ceph.keyring`.

Managing the MDS service using the Ceph Orchestrator

Use the Ceph Orchestrator with `cephadm` in the backend to deploy the MDS service.

By default, a Ceph File System (CephFS) uses only one active MDS daemon. However, systems with many clients benefit from multiple active MDS daemons.

When managing the MDS service by using the Ceph Orchestrator, be sure to have:

- A running Red Hat Ceph Storage cluster.
- Root-level access to all the nodes.
- Hosts added to the cluster.
- All manager, monitor, and OSD daemons deployed.

Deploying the MDS service with the command-line interface

Using the Ceph Orchestrator, you can deploy the Metadata Server (MDS) service by using the `placement` specification in the command-line interface.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Hosts added to the cluster.
- All manager, monitor, and OSD daemons deployed.

About this task

Ceph File System (CephFS) requires one or more MDS.

Note: Ensure that you have at least two pools, one for Ceph file system (CephFS) data and one for CephFS metadata.

For more information about creating Ceph File Systems, see Creating Ceph File Systems.

To deploy the MDS service with the command-line interface, log in to the `cephadm` shell.

```
[root@host01 ~]# cephadm shell
```

Use one of the following options to deploy MDS daemons using placement specification.

Creating the MDS daemons through the command-line

Procedure

- Use **`ceph fs volume`** to create the MDS daemons.
  Creating the MDS daemons creates the CephFS volume and pools associated with the CephFS, and also starts the MDS service on the hosts.

  ```
  ceph fs volume create FILESYSTEM_NAME --
  placement="NUMBER_OF_DAEMONS HOST_NAME_1 HOST_NAME_2 HOST_NAME_3"
  ```

  Note: By default, replicated pools are created for this command.

  For example,

  ```
  [ceph: root@host01 /]# ceph fs volume create test --placement="2 host01 host02"
  ```

Creating the pools, CephFS, and then deploying MDS service using placement specification

Procedure

1. Create the pools for CephFS.

   ```
   ceph osd pool create DATA_POOL PG_NUM
   ceph osd pool create METADATA_POOL PG_NUM
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph osd pool create cephfs_data 64
   [ceph: root@host01 /]# ceph osd pool create cephfs_metadata 64
   ```

2. Create the file system for the data pools and metadata pools.

   ```
   ceph fs new FILESYSTEM_NAME METADATA_POOL DATA_POOL
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph fs new test cephfs_metadata cephfs_data
   ```

3. Deploy MDS service using the **`ceph orch apply`** command.

   ```
   ceph orch apply mds FILESYSTEM_NAME --
   placement="NUMBER_OF_DAEMONS HOST_NAME_1 HOST_NAME_2 HOST_NAME_3"
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch apply mds test --placement="2 host01 host02"
   ```

Verifying the MDS service deployment

Procedure

1. List the service.
   For example,

   ```
   [ceph: root@host01 /]# ceph orch ls
   ```

2. Check the CephFS status.
   For example,

   ```
   [ceph: root@host01 /]# ceph fs ls
   [ceph: root@host01 /]# ceph fs status
   ```

3. List the hosts, daemons, and processes.

   ```
   ceph orch ps --daemon_type=DAEMON_NAME
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch ps --daemon_type=mds
   ```

Deploying the MDS service using the service specification

Using the Ceph Orchestrator, you can deploy the MDS service using the service specification.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Hosts added to the cluster.
- All manager, monitor, and OSD daemons deployed.

About this task

> **Note:** Ensure that you have at least two pools, one for the Ceph File System (CephFS) data and one for the CephFS metadata.

Procedure

1. Create the `mds.yaml` file.

   ```
   touch mds.yaml
   ```

2. Edit the `mds.yaml` file.

   ```
   service_type: mds
   service_id: FILESYSTEM_NAME
   placement:
     hosts:
     - HOST_NAME_1
     - HOST_NAME_2
     - HOST_NAME_3
   ```

   For example,

   ```
   service_type: mds
   service_id: fs_name
   placement:
     hosts:
     - host01
     - host02
   ```

3. Mount the YAML file under a directory in the container.
   For example,

   ```
   [root@host01 ~]# cephadm shell --mount mds.yaml:/var/lib/ceph/mds/mds.yaml
   ```

4. Navigate to the ceph/mds/ directory.
   For example,

   ```
   [ceph: root@host01 /]# cd /var/lib/ceph/mds/
   ```

5. Log into the cephadm shell.

   ```
   cephadm shell
   ```

6. Navigate to the /ceph/mds/ directory.
   For example,

   ```
   [ceph: root@host01 /]# cd /var/lib/ceph/mds/
   ```

7. Deploy MDS service using service specification.

   ```
   ceph orch apply -i FILE_NAME.yaml
   ```

   For example,

   ```
   [ceph: root@host01 mds]# ceph orch apply -i mds.yaml
   ```

8. Once the MDS services is deployed and functional, create the CephFS.

   ```
   ceph fs new CEPHFS_NAME METADATA_POOL DATA_POOL
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph fs new test metadata_pool data_pool
   ```

What to do next

Verify the MDS service deployment.

1. List the service.

   ```
   [ceph: root@host01 /]# ceph orch ls
   ```

2. List the hosts, daemons, and processes.

   ```
   ceph orch ps --daemon_type=DAEMON_NAME
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch ps --daemon_type=mds
   ```

Removing the MDS service using the Ceph Orchestrator

Remove the service either by using the **ceph orch rm** command or by removing the file system and the associated pools.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Hosts added to the cluster.
- Root-level access to all the nodes.
- At least one MDS daemon deployed on the hosts.

About this task

To remove the MDS service with the command-line interface, log in to the `cephadm` shell.

```
[root@host01 ~]# cephadm shell
```

Use one of the following options to remove MDS daemons.

Removing the MDS service from the entire cluster through the command line

Procedure

Use the **`ceph orch rm`** command to remove the MDS service from the entire cluster.

1. List the service.

   ```
   ceph orch ls
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch ls
   ```

2. Remove the service.

   ```
   ceph orch rm SERVICE_NAME
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch rm mds.test
   ```

Removing the CephFS volume, associated pools, and the services

Procedure

1. Set the configuration parameter **`mon_allow_pool_delete`** to `true`.

   ```
   ceph config set mon mon_allow_pool_delete true
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph config set mon mon_allow_pool_delete true
   ```

2. Remove the file system, its data, and metadata pools.

   ```
   ceph fs volume rm FILESYSTEM_NAME --yes-i-really-mean-it
   ```

   This command also tries to remove the MDS using the enabled `ceph-mgr` Orchestrator module.

   ```
   ceph fs volume rm cephfs-new --yes-i-really-mean-it
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph fs volume rm cephfs-new --yes-i-really-mean-it
   ```

Verifying the MDS service removal

Procedure

- List the hosts, daemons, and processes, by using the **`ceph orch ps`** command.
  For example,

  ```
  [ceph: root@host01 /]# ceph orch ps
  ```

Managing the Ceph Object Gateway

As a storage administrator, you can deploy Ceph Object Gateway using the command line interface or by using the service specification.

You can also configure multi-site Ceph Object Gateways, and remove the Ceph Object Gateway.

`cephadm` deploys Ceph Object Gateway as a collection of daemons that manages a single-cluster deployment or a particular realm and zone in a multi-site deployment.

Note:

- With `cephadm`, the Ceph Object Gateway daemons are configured using the monitor configuration database instead of a `ceph.conf` or the command line. If that configuration is not already in the `client.rgw` section, then the Ceph Object Gateway will start up with default settings and bind to the port `80`.
- The `.default.rgw.buckets.index` pool is created only after the bucket is created in Ceph Object Gateway, while the `.default.rgw.buckets.data` pool is created after the data is uploaded to the bucket.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to all the nodes.
- Hosts are added to the cluster.
- All the managers, monitors, and OSDs are deployed in the storage cluster.

Deploying the Ceph Object Gateway using the command line interface

Deploy the Ceph Object Gateway by using the Ceph Orchestrator with the **`ceph orch`** command in the command line interface.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to all the nodes.
- Hosts are added to the cluster.
- All manager, monitor, and OSD daemons are deployed.

Log in to the `cephadm` shell by using the **`cephadm shell`** to deploy Ceph Object Gateway daemons.

About this task

Deploy using one of the following methods:

- “Creating a realm, zone group, and zone” on page 624
- “Using an arbitrary service name for a single cluster deployment” on page 625
- “Using an arbitrary service name on a labeled set of hosts” on page 625

Creating a realm, zone group, and zone

Procedure

Create a realm, zone group, zone, and then use the placement specification with the hostname.

1. Create a realm.

```
radosgw-admin realm create --rgw-realm=REALM_NAME --default
```

For example,

```
[ceph: root@host01 /]# radosgw-admin realm create --rgw-realm=test_realm --default
```

2. Create a zone group.

```
radosgw-admin zonegroup create --rgw-zonegroup=ZONE_GROUP_NAME  --master --default
```

For example,

```
[ceph: root@host01 /]# radosgw-admin zonegroup create --rgw-zonegroup=default  --
master --default
```

3. Create a zone.

```
radosgw-admin zone create --rgw-zonegroup=ZONE_GROUP_NAME --rgw-zone=ZONE_NAME --
master --default
```

For example,

```
[ceph: root@host01 /]# radosgw-admin zone create --rgw-zonegroup=default --rgw-
zone=test_zone --master --default
```

4. Commit the changes.

```
radosgw-admin period update --rgw-realm=REALM_NAME --commit
```

For example,

```
[ceph: root@host01 /]# radosgw-admin period update --rgw-realm=test_realm --commit
```

5. Apply the changes by using the **ceph orch apply** command.

```
ceph orch apply rgw NAME [--realm=REALM_NAME] [--zone=ZONE_NAME] [--
zonegroup=ZONE_GROUP_NAME] --placement="NUMBER_OF_DAEMONS [HOST_NAME_1 HOST_NAME_2]"
```

For example,

```
[ceph: root@host01 /]# ceph orch apply rgw test --realm=test_realm --zone=test_zone --
zonegroup=default --placement="2 host01 host02"
```

Using an arbitrary service name for a single cluster deployment

Use an arbitrary service name to deploy two Ceph Object Gateway daemons for a single cluster deployment.

```
ceph orch apply rgw SERVICE_NAME
```

For example,

```
[ceph: root@host01 /]# ceph orch apply rgw foo
```

Using an arbitrary service name on a labeled set of hosts

Use an arbitrary service name on a labeled set of hosts.

```
ceph orch host label add HOST_NAME_1 LABEL_NAME
ceph orch host label add HOSTNAME_2 LABEL_NAME
ceph orch apply rgw SERVICE_NAME --placement="label:LABEL_NAME count-per-
host:NUMBER_OF_DAEMONS" --port=8000
```

NUMBER_OF_DAEMONS controls the number of Ceph Object Gateways that are deployed on each host. To achieve the highest performance without incurring any extra cost, set this value to 2.
For example,

```
[ceph: root@host01 /]# ceph orch host label add host01 rgw  # the 'rgw' label can be
anything
[ceph: root@host01 /]# ceph orch host label add host02 rgw
[ceph: root@host01 /]# ceph orch apply rgw foo --placement="label:rgw count-per-host:2" --
port=8000
```

What to do next

Verify the Ceph Object Gateway deployment.

- List the service, by using the **`ceph orch ls`** command.
  For example,

  ```
  [ceph: root@host01 /]# ceph orch ls
  ```

- List the hosts, daemons, and processes.

  ```
  ceph orch ps --daemon_type=DAEMON_NAME
  ```

  For example,

  ```
  [ceph: root@host01 /]# ceph orch ps --daemon_type=rgw
  ```

Deploying the Ceph Object Gateway using the service specification

Deploy the Ceph Object Gateway using the service specification with either the default or the custom realms, zones, and zone groups.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the bootstrapped host.
- Hosts are added to the cluster.
- All Ceph Manager, Ceph Monitor, and Object Storage Daemon (OSD) daemons are deployed.

Procedure

1. Create a specification file.

   ```
   touch rgw_spec.yaml
   ```

   For example,

   ```
   [root@host01 ~]# touch rgw_spec.yaml
   ```

2. Configure S3 requests to wait for the duration defined in the **`rgw_exit_timeout_secs`** parameter for all outstanding requests to complete by setting **`rgw_graceful_stop`** to `true` during Ceph Object Gateway shutdown or restart.

   ```
   ceph config set client.rgw rgw_graceful_stop true

   ceph config set client.rgw rgw_exit_timeout_secs 120
   ```

   > **Important:** In containerized deployments, an additional `extra_container_args` configuration of `--stop-timeout=120` (or the value of the **`rgw_exit_timeout_secs`** configuration, if not default) is also necessary in order for it to work as expected with the **`ceph orch stop`** or **`ceph orch restart`** commands.
   > For example,

```
[root@host1 ~]$ cat rgw_spec.yaml
service_type: rgw
service_id: foo
placement:
  count_per_host: 1
  hosts:
    - rgw_node
spec:
  rgw_frontend_port: 8081
extra_container_args:
  - "--stop-timeout=120"
```

3. Edit the `rgw_spec.yaml` file to include the details for the default realm, zone, and zone group.

```
service_type: rgw
service_id: REALM_NAME.ZONE_NAME
placement:
  hosts:
    - HOST_NAME_1
    - HOST_NAME_2
  count_per_host: NUMBER_OF_DAEMONS
spec:
  rgw_realm: REALM_NAME
  rgw_zone: ZONE_NAME
  rgw_zonegroup: ZONE_GROUP_NAME
  rgw_frontend_port: FRONT_END_PORT

networks:
  - NETWORK_CIDR
```

The *NUMBER_OF_DAEMONS* parameter controls the number of Ceph Object Gateways deployed on each host. To achieve the highest performance without incurring an additional cost, set this value to 2.

For example,

```
service_type: rgw
service_id: default
placement:
  hosts:
    - host01
    - host02
    - host03
  count_per_host: 2
spec:
  rgw_realm: default
  rgw_zone: default
  rgw_zonegroup: default
  rgw_frontend_port: 1234

networks:
  - 192.169.142.0/24
```

4. Optional: Configure custom realm, zone, and zone group.
   a. Create the custom realm, zone, and zone group.

```
radosgw-admin realm create --rgw-realm=test_realm --default
radosgw-admin zonegroup create --rgw-zonegroup=test_zonegroup --default
radosgw-admin zone create --rgw-zonegroup=test_zonegroup --rgw-zone=test_zone --
default
radosgw-admin period update --rgw-realm=test_realm --commit
```

   For example,

```
[root@host01 ~]# radosgw-admin realm create --rgw-realm=test_realm --default
[root@host01 ~]# radosgw-admin zonegroup create --rgw-zonegroup=test_zonegroup --
default
[root@host01 ~]# radosgw-admin zone create --rgw-zonegroup=test_zonegroup --rgw-
zone=test_zone --default
[root@host01 ~]# radosgw-admin period update --rgw-realm=test_realm --commit
```

   b. Create the `rgw_spec.yaml` file with the custom details.

For example,

```
service_type: rgw
service_id: test_realm.test_zone
placement:
  hosts:
    - host01
    - host02
    - host03
  count_per_host: 2
spec:
  rgw_realm: test_realm
  rgw_zone: test_zone
  rgw_zonegroup: test_zonegroup
  rgw_frontend_port: 1234
networks:
  - 192.169.142.0/24
```

5. Optional: Deploy the HAProxy concentrator for local Ceph Object Gateway load balancing.
To load balance requests across Ceph Object Gateway daemons on the same host, configure the service with a local HAProxy concentrator.

   Note: The HAProxy concentrator does not currently support SSL.

   Note: Use this configuration only when `count_per_host > 1`.

   For example,

   ```
   service_type: rgw
   service_id: foo
   service_name: rgw.foo
   placement:
     count_per_host: 2
     hosts:
       - host1
       - host2
   spec:
     concentrator: haproxy
     concentrator_frontend_port: 8080
     concentrator_monitor_port: 1967
     concentrator_monitor_user: admin
   ```

6. Optional: In a multi-site deployment, if you want to dedicate some Ceph Object Gateway instances to client IO only, disable multi-site sync traffic for those instances.
These instances handle client IO but do not take part in multi-site sync operations.

   For those instances, set **`disable_multisite_sync_traffic`** to `true` under the `spec` section of the Ceph Object Gateway specification file. The `cephadm` utility then sets the **`rgw_run_sync_thread`** configuration to `false` for those Ceph Object Gateway daemons. This setting prevents the daemons from taking part in sync operations.

   This automation eliminates the need to manually configure **`rgw_run_sync_thread`** for those Ceph Object Gateway daemons. To configure it manually, run the following command, where RGW_DAEMON_NAME is the name of the Ceph Object Gateway daemon:

   ```
   ceph config set RGW_DAEMON_NAME rgw_run_sync_thread false
   ```

   In the following example, the Ceph Object Gateway service `rgw.io` has **`disable_multisite_sync_traffic`** set to `true`, so it handles only client IO. The service `rgw.sync` takes part in multi-site sync operations because **`disable_multisite_sync_traffic`** is `false` by default.

```
[root@host01 ~]# cat rgw_spec.yaml
service_type: rgw
service_id: io
placement:
  hosts:
    - host01
    - host02
spec:
  rgw_realm: test_realm
  rgw_zone: test_zone
  rgw_zonegroup: test_zonegroup
  rgw_frontend_port: 1234
  disable_multisite_sync_traffic: true
---
service_type: rgw
service_id: sync
placement:
  hosts:
    - host03
    - host04
spec:
  rgw_realm: test_realm
  rgw_zone: test_zone
  rgw_zonegroup: test_zonegroup
  rgw_frontend_port: 1234
```

7. Mount the `rgw_spec.yaml` file under a directory in the container.

   Important: Every time you exit the `cephadm` shell, you must mount the file in the container before deploying the daemon.

   ```
   cephadm shell --mount rgw_spec.yaml:/var/lib/ceph/radosgw/rgw_spec.yaml
   ```

   For example,

   ```
   [root@host01 ~]# cephadm shell --mount rgw_spec.yaml:/var/lib/ceph/radosgw/
   rgw_spec.yaml
   ```

8. From within the `cephadm` shell, deploy the Ceph Object Gateway by using the service specification.

   ```
   ceph orch apply -i FILE_NAME.yaml
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch apply -i /var/lib/ceph/radosgw/rgw_spec.yaml
   ```

9. Optional: If you disabled multi-site sync traffic for some Ceph Object Gateway instances, make sure that only the sync instances are listed as zone and zone group endpoints.
   Other zones send sync requests to the zone and zone group endpoints. Do not list the client IO instances as endpoints. Set the endpoints to the sync instances only, and then commit the period.

   ```
   radosgw-admin zonegroup modify --rgw-zonegroup=ZONE_GROUP_NAME --endpoints=http://
   SYNC_HOST_1:PORT,http://SYNC_HOST_2:PORT
   radosgw-admin zone modify --rgw-zone=ZONE_NAME --endpoints=http://
   SYNC_HOST_1:PORT,http://SYNC_HOST_2:PORT
   radosgw-admin period update --commit
   ```

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin zonegroup modify --rgw-zonegroup=test_zonegroup
   --endpoints=http://host03:1234,http://host04:1234
   [ceph: root@host01 /]# radosgw-admin zone modify --rgw-zone=test_zone --
   endpoints=http://host03:1234,http://host04:1234
   [ceph: root@host01 /]# radosgw-admin period update --commit
   ```

Result

Verify that the Ceph Object Gateway service is deployed successfully:

- List the service:

```
ceph orch ls
```

- List the hosts, daemons, and processes:

```
ceph orch ps --daemon_type=DAEMON_NAME
```

For example,

```
[ceph: root@host01 /]# ceph orch ps --daemon_type=rgw
```

Deploying a multi-site Ceph Object Gateway

Ceph Orchestrator supports multi-site configuration options for the Ceph Object Gateway.

You can configure each object gateway to work in an active-active zone configuration allowing writes to a non-primary zone. The multi-site configuration is stored within a container called a realm.

The realm stores zone groups, zones, and a time period. The `rgw` daemons handle the synchronization eliminating the need for a separate synchronization agent, thereby operating with an active-active configuration.

You can also deploy multi-site zones using the command line interface (CLI).

> **Note:** The following configuration assumes at least two Red Hat Ceph Storage clusters are in geographically separate locations. However, the configuration also works on the same site.

Prerequisites

- At least two running Red Hat Ceph Storage clusters.
- At least two Ceph Object Gateway instances, one for each Red Hat Ceph Storage cluster.
- Root-level access to all the nodes.
- Nodes or containers are added to the storage cluster.
- All Ceph Manager, Monitor and OSD daemons are deployed.

Procedure

1. In the `cephadm` shell, configure the primary zone:
   a. Create a realm:

      Syntax

      ```
      radosgw-admin realm create --rgw-realm=REALM_NAME --default
      ```

      Example

      ```
      [ceph: root@host01 /]# radosgw-admin realm create --rgw-realm=test_realm --
      default
      ```

      If the storage cluster has a single realm, then specify the `--default` flag.

   b. Create a primary zone group:

      Syntax

      ```
      radosgw-admin zonegroup create --rgw-zonegroup=ZONE_GROUP_NAME --
      endpoints=http://RGW_PRIMARY_HOSTNAME:RGW_PRIMARY_PORT_NUMBER_1 --master --
      default
      ```

      Example

```
[ceph: root@host01 /]# radosgw-admin zonegroup create --rgw-zonegroup=us --
endpoints=http://rgw1:80 --master --default
```

c. Create a primary zone:

Syntax

```
radosgw-admin zone create --rgw-zonegroup=PRIMARY_ZONE_GROUP_NAME --rgw-
zone=PRIMARY_ZONE_NAME --endpoints=http://
RGW_PRIMARY_HOSTNAME:RGW_PRIMARY_PORT_NUMBER_1 --access-key=SYSTEM_ACCESS_KEY --
secret=SYSTEM_SECRET_KEY
```

Example

```
[ceph: root@host01 /]# radosgw-admin zone create --rgw-zonegroup=us --rgw-
zone=us-east-1 --endpoints=http://rgw1:80 --access-key=LIPEYZJLTWXRKXS9LPJC --
secret-key=IsAje0AVDNXNw48LjMAimpCpI7VaxJYSnfD0FFKQ
```

d. Optional: Delete the default zone, zone group, and the associated pools.

> **Important:** Do not delete the default zone and its pools if you are using the default zone and zone group to store data. Also, removing the default zone group deletes the system user.

To access old data in the `default` zone and zonegroup, use `--rgw-zone default` and `--rgw-zonegroup default` in `radosgw-admin` commands.

Example

```
[ceph: root@host01 /]# radosgw-admin zonegroup delete --rgw-zonegroup=default
[ceph: root@host01 /]# ceph osd pool rm default.rgw.log default.rgw.log --yes-i-
really-really-mean-it
[ceph: root@host01 /]# ceph osd pool rm default.rgw.meta default.rgw.meta --yes-
i-really-really-mean-it
[ceph: root@host01 /]# ceph osd pool rm default.rgw.control default.rgw.control
--yes-i-really-really-mean-it
[ceph: root@host01 /]# ceph osd pool rm default.rgw.data.root
default.rgw.data.root --yes-i-really-really-mean-it
[ceph: root@host01 /]# ceph osd pool rm default.rgw.gc default.rgw.gc --yes-i-
really-really-mean-it
```

e. Create a system user:

Syntax

```
radosgw-admin user create --uid=USER_NAME --display-name="USER_NAME" --access-
key=SYSTEM_ACCESS_KEY --secret=SYSTEM_SECRET_KEY --system
```

Example

```
[ceph: root@host01 /]# radosgw-admin user create --uid=zone.user --display-
name="Zone user" --system
```

Make a note of the `access_key` and `secret_key`.

f. Add the access key and system key to the primary zone:

Syntax

```
radosgw-admin zone modify --rgw-zone=PRIMARY_ZONE_NAME --access-key=ACCESS_KEY --
secret=SECRET_KEY
```

Example

```
[ceph: root@host01 /]# radosgw-admin zone modify --rgw-zone=us-east-1 --access-
key=LIPEYZJLTWXRKXS9LPJC --secret-key=IsAje0AVDNXNw48LjMAimpCpI7VaxJYSnfD0FFKQ
```

g. Commit the changes:

Syntax

```
radosgw-admin period update --commit
```

Example

```
[ceph: root@host01 /]# radosgw-admin period update --commit
```

h. Outside the `cephadm` shell, fetch the `FSID` of the storage cluster and the processes:

Example

```
[root@host01 ~]#  systemctl list-units | grep ceph
```

i. Start the Ceph Object Gateway daemon:

Syntax

```
systemctl start ceph-FSID@DAEMON_NAME
systemctl enable ceph-FSID@DAEMON_NAME
```

Example

```
[root@host01 ~]# systemctl start ceph-62a081a6-88aa-11eb-
a367-001a4a000672@rgw.test_realm.us-east-1.host01.ahdtsw.service
[root@host01 ~]# systemctl enable ceph-62a081a6-88aa-11eb-
a367-001a4a000672@rgw.test_realm.us-east-1.host01.ahdtsw.service
```

2. In the `cephadm` shell, configure the secondary zone.

a. Pull the primary realm configuration from the host:

Syntax

```
radosgw-admin realm pull --url=URL_TO_PRIMARY_ZONE_GATEWAY --access-
key=ACCESS_KEY --secret-key=SECRET_KEY
```

Example

```
[ceph: root@host04 /]# radosgw-admin realm pull --url=http://10.74.249.26:80 --
access-key=LIPEYZJLTWXRKXS9LPJC --secret-
key=IsAje0AVDNXNw48LjMAimpCpI7VaxJYSnfD0FFKQ
```

b. Pull the primary period configuration from the host:

Syntax

```
radosgw-admin period pull --url=URL_TO_PRIMARY_ZONE_GATEWAY --access-
key=ACCESS_KEY --secret-key=SECRET_KEY
```

Example

```
[ceph: root@host04 /]# radosgw-admin period pull --url=http://10.74.249.26:80 --
access-key=LIPEYZJLTWXRKXS9LPJC --secret-
key=IsAje0AVDNXNw48LjMAimpCpI7VaxJYSnfD0FFKQ
```

c. Configure a secondary zone:

Syntax

```
radosgw-admin zone create --rgw-zonegroup=ZONE_GROUP_NAME
             --rgw-zone=_SECONDARY_ZONE_NAME_
             --access-key=_SYSTEM_ACCESS_KEY_ --secret=SYSTEM_SECRET_KEY
             --endpoints=http://RGW_SECONDARY_HOSTNAME:PORT_NUMBER"
             [--read-only]
```

Example

```
[ceph: root@host04 /]# radosgw-admin zone create --rgw-zonegroup=us --rgw-
zone=us-east-2 --access-key=LIPEYZJLTWXRKXS9LPJC --secret-
key=IsAje0AVDNXNw48LjMAimpCpI7VaxJYSnfD0FFKQ --endpoints=http://
rgw.example.com:80
```

d. Optional: Delete the default zone.

> **Important:** Do not delete the default zone and its pools if you are using the default zone and zone group to store data.

To access old data in the `default` zone and zonegroup, use `--rgw-zone default` and `--rgw-zonegroup default` in `radosgw-admin` commands.

Example

```
[ceph: root@host04 /]# radosgw-admin zone rm --rgw-zone=default
[ceph: root@host04 /]# ceph osd pool rm default.rgw.log default.rgw.log --yes-i-
really-really-mean-it
[ceph: root@host04 /]# ceph osd pool rm default.rgw.meta default.rgw.meta --yes-
i-really-really-mean-it
[ceph: root@host04 /]# ceph osd pool rm default.rgw.control default.rgw.control
--yes-i-really-really-mean-it
[ceph: root@host04 /]# ceph osd pool rm default.rgw.data.root
default.rgw.data.root --yes-i-really-really-mean-it
[ceph: root@host04 /]# ceph osd pool rm default.rgw.gc default.rgw.gc --yes-i-
really-really-mean-it
```

e. Update the Ceph configuration database:

Syntax

```
ceph config set SERVICE_NAME rgw_zone SECONDARY_ZONE_NAME
```

Example

```
[ceph: root@host04 /]# ceph config set rgw rgw_zone us-east-2
```

f. Commit the changes:

Syntax

```
radosgw-admin period update --commit
```

Example

```
[ceph: root@host04 /]# radosgw-admin period update --commit
```

g. Outside the `cephadm` shell, fetch the FSID of the storage cluster and the processes:

Example

```
[root@host04 ~]#  systemctl list-units | grep ceph
```

h. Start the Ceph Object Gateway daemon:

Syntax

```
systemctl start ceph-FSID@DAEMON_NAME
systemctl enable ceph-FSID@DAEMON_NAME
```

Example

```
[root@host04 ~]# systemctl start ceph-62a081a6-88aa-11eb-
a367-001a4a000672@rgw.test_realm.us-east-2.host04.ahdtsw.service
```

```
[root@host04 ~]# systemctl enable ceph-62a081a6-88aa-11eb-
a367-001a4a000672@rgw.test_realm.us-east-2.host04.ahdtsw.service
```

3. Optional: Deploy multi-site Ceph Object Gateways using the placement specification:

   Syntax

   ```
   ceph orch apply rgw NAME --realm=REALM_NAME --zone=PRIMARY_ZONE_NAME --
   placement="NUMBER_OF_DAEMONS HOST_NAME_1 HOST_NAME_2"
   ```

   Example

   ```
   [ceph: root@host04 /]# ceph orch apply rgw east --realm=test_realm --zone=us-east-1 --
   placement="2 host01 host02"
   ```

Verification

- Check the synchronization status to verify the deployment:

  Example

  ```
  [ceph: root@host04 /]# radosgw-admin sync status
  ```

Removing the Ceph Object Gateway

Use the **`ceph orch rm`** command to remove Ceph Object Gateway daemons.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to all the nodes.
- Hosts are added to the cluster.
- At least one Ceph Object Gateway daemon deployed on the hosts.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. List the service:

   Example

   ```
   [ceph: root@host01 /]# ceph orch ls
   ```

3. Remove the service:

   Syntax

   ```
   ceph orch rm SERVICE_NAME
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph orch rm rgw.test_realm.test_zone_bb
   ```

Verification

- List the hosts, daemons, and processes:

Syntax

```
ceph orch ps
```

Example

```
[ceph: root@host01 /]# ceph orch ps
```

Reference

- Deploying the Ceph Object Gateway using the command line interface
- Deploying the Ceph Object Gateway using the service specification

Handling a node failure

As a storage administrator, you can experience a whole node failing within the storage cluster, and handling a node failure is similar to handling a disk failure. With a node failure, instead of Ceph recovering placement groups (PGs) for only one disk, all PGs on the disks within that node must be recovered. Ceph will detect that the OSDs are all down and automatically start the recovery process, known as self-healing.

There are three node failure scenarios.

- Replacing the node by using the root and Ceph OSD disks from the failed node.
- Replacing the node by reinstalling the operating system and using the Ceph OSD disks from the failed node.
- Replacing the node by reinstalling the operating system and using all new Ceph OSD disks.

For a high level workflow for each node replacement scenario, see Workflow for replacing a node.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- A failed node.

Considerations before adding or removing a node

Ceph OSD nodes can be added or removed at run time. This means that you can resize the storage cluster capacity or replace hardware without taking down the storage cluster. Understand these considerations before adding or removing a node.

The ability to serve Ceph clients while the storage cluster is in a `degraded` state also has operational benefits. For example, you can add or remove or replace hardware during regular business hours, rather than working overtime or on weekends. However, adding and removing Ceph OSD nodes can have a significant impact on performance.

Before you add or remove Ceph OSD nodes, consider the effects on storage cluster performance:

- Whether you are expanding or reducing the storage cluster capacity, adding or removing Ceph OSD nodes induces backfilling as the storage cluster rebalances. During that rebalancing time period, Ceph uses additional resources, which can impact storage cluster performance.
- In a production Ceph storage cluster, a Ceph OSD node has a particular hardware configuration that facilitates a particular type of storage strategy.
- Since a Ceph OSD node is part of a CRUSH hierarchy, the performance impact of adding or removing a node typically affects the performance of pools that use the CRUSH ruleset.

Important: In director-deployed Red Hat Ceph Storage environments, replacing Controller nodes can affect the Ceph Monitor service and lead to storage outages if Monitor IP addresses change. For more information, see “Managing the Ceph Monitor service with Red Hat OpenStack Platform” on page 563.

Workflow for replacing a node

There are three node failure scenarios. Use these high-level workflows for each scenario when replacing a node.

There are three node failure scenarios.

- “Replacing the node by using the root and Ceph OSD disks from the failed node” on page 636
- “Replacing the node by reinstalling the operating system and using the Ceph OSD disks from the failed node” on page 636
- “Replacing the node by reinstalling the operating system and using all new Ceph OSD disks” on page 636

Prerequisites

- A running Red Hat Ceph Storage cluster.
- A failed node.

Replacing the node by using the root and Ceph OSD disks from the failed node

Use the root and Ceph OSD disks from the failed node to replace the node.

Procedure

1. Disable backfilling.

   ```
   ceph osd set noout
   ceph osd set noscrub
   ceph osd set nodeep-scrub
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph osd set noout
   [ceph: root@host01 /]# ceph osd set noscrub
   [ceph: root@host01 /]# ceph osd set nodeep-scrub
   ```

2. Replace the node, taking the disks from the old node, and adding them to the new node.
3. Enable backfilling.

   ```
   ceph osd unset noout
   ceph osd unset noscrub
   ceph osd unset nodeep-scrub
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph osd unset noout
   [ceph: root@host01 /]# ceph osd unset noscrub
   [ceph: root@host01 /]# ceph osd unset nodeep-scrub
   ```

Replacing the node by reinstalling the operating system and using the Ceph OSD disks from the failed node

Reinstall the operating system and use the Ceph OSD disks from the failed node to replace the node.

Procedure

1. Disable backfilling.

   ```
   ceph osd set noout
   ceph osd set noscrub
   ceph osd set nodeep-scrub
   ```

   For example,

```
[ceph: root@host01 /]# ceph osd set noout
[ceph: root@host01 /]# ceph osd set noscrub
[ceph: root@host01 /]# ceph osd set nodeep-scrub
```

2. Create a backup of the Ceph configuration.

```
cp /etc/ceph/ceph.conf /PATH_TO_BACKUP_LOCATION/ceph.conf
```

For example,

```
[ceph: root@host01 /]# cp /etc/ceph/ceph.conf /some/backup/location/ceph.conf
```

3. Replace the node and add the Ceph OSD disks from the failed node.
4. Configure disks as JBOD.

> **Note:** This should be done by the storage administrator.

5. Install the operating system.
For more information about operating system requirements, see Operating system requirements. For ore information about installing the operating system, see the Red Hat Enterprise Linux product documentation.

> **Note:** This should be done by the system administrator.

6. Restore the Ceph configuration.

```
cp /PATH_TO_BACKUP_LOCATION/ceph.conf /etc/ceph/ceph.conf
```

For example,

```
[ceph: root@host01 /]# cp /some/backup/location/ceph.conf /etc/ceph/ceph.conf
```

7. Add the new node to the storage cluster using the Ceph Orchestrator commands.
Ceph daemons are placed automatically on the respective node. For more information, see “Adding a Ceph OSD node” on page 640.
8. Enable backfilling.

```
ceph osd unset noout
ceph osd unset noscrub
ceph osd unset nodeep-scrub
```

For example,

```
[ceph: root@host01 /]# ceph osd unset noout
[ceph: root@host01 /]# ceph osd unset noscrub
[ceph: root@host01 /]# ceph osd unset nodeep-scrub
```

Replacing the node by reinstalling the operating system and using all new Ceph OSD disks

Reinstall the operating system and use all new Ceph OSD disks to replace the node.

Procedure

1. Disable backfilling.

```
ceph osd set noout
ceph osd set noscrub
ceph osd set nodeep-scrub
```

For example,

```
[ceph: root@host01 /]# ceph osd set noout
[ceph: root@host01 /]# ceph osd set noscrub
[ceph: root@host01 /]# ceph osd set nodeep-scrub
```

2. Remove all OSDs on the failed node from the storage cluster.
For more information, see “Removing a Ceph OSD node” on page 641.
3. Create a backup of the Ceph configuration.

```
cp /etc/ceph/ceph.conf /PATH_TO_BACKUP_LOCATION/ceph.conf
```

For example,

```
[ceph: root@host01 /]# cp /etc/ceph/ceph.conf /some/backup/location/ceph.conf
```

4. Replace the node and add the Ceph OSD disks from the failed node.
5. Configure disks as JBOD.

> **Note:** This should be done by the storage administrator.

6. Install the operating system.
For more information about operating system requirements, see Operating system requirements. For ore information about installing the operating system, see the Red Hat Enterprise Linux product documentation.

> **Note:** This should be done by the system administrator.

7. Add the new node to the storage cluster using the Ceph Orchestrator commands.
Ceph daemons are placed automatically on the respective node. For more information, see “Adding a Ceph OSD node” on page 640.
8. Enable backfilling.

```
ceph osd unset noout
ceph osd unset noscrub
ceph osd unset nodeep-scrub
```

For example,

```
[ceph: root@host01 /]# ceph osd unset noout
[ceph: root@host01 /]# ceph osd unset noscrub
[ceph: root@host01 /]# ceph osd unset nodeep-scrub
```

Performance considerations

Understand the performance considerations that affect a storage cluster's performance when adding or removing Ceph OSD nodes.

The following factors typically affect a storage cluster’s performance when adding or removing Ceph OSD nodes:

- Ceph clients place load on the I/O interface to Ceph; that is, the clients place load on a pool. A pool maps to a CRUSH ruleset. The underlying CRUSH hierarchy allows Ceph to place data across failure domains. If the underlying Ceph OSD node involves a pool that is experiencing high client load, the client load could significantly affect recovery time and reduce performance. Because write operations require data replication for durability, write-intensive client loads in particular can increase the time for the storage cluster to recover.
- Generally, the capacity you are adding or removing affects the storage cluster’s time to recover. In addition, the storage density of the node you add or remove might also affect recovery times. For example, a node with 36 OSDs typically takes longer to recover than a node with 12 OSDs.

- When removing nodes, you MUST ensure that you have sufficient spare capacity so that you will not reach `full ratio` or `near full ratio`. If the storage cluster reaches `full ratio`, Ceph will suspend write operations to prevent data loss.
- A Ceph OSD node maps to at least one Ceph CRUSH hierarchy, and the hierarchy maps to at least one pool. Each pool that uses a CRUSH ruleset experiences a performance impact when Ceph OSD nodes are added or removed.
- Replication pools tend to use more network bandwidth to replicate deep copies of the data, whereas erasure coded pools tend to use more CPU to calculate `k+m` coding chunks. The more copies that exist of the data, the longer it takes for the storage cluster to recover. For example, a larger pool or one that has a greater number of `k+m` chunks will take longer to recover than a replication pool with fewer copies of the same data.
- Drives, controllers and network interface cards all have throughput characteristics that might impact the recovery time. Generally, nodes with higher throughput characteristics, such as 10 Gbps and SSDs, recover more quickly than nodes with lower throughput characteristics, such as 1 Gbps and SATA drives.

Recommendations for adding or removing nodes

Understand these recommendations for adding or removing nodes.

Red Hat recommends adding or removing one OSD at a time within a node and allowing the storage cluster to recover before proceeding to the next OSD. This helps to minimize the impact on storage cluster performance. Note that if a node fails, you might need to change the entire node at once, rather than one OSD at a time.

To remove an OSD:

- Using *Removing the OSD daemons*.

To add an OSD:

- Using *Deploying Ceph OSDs on all available devices*.
- Using *Deploying Ceph OSDs using service specification*.
- Using *Deploying Ceph OSDs on specific devices and hosts*.

When adding or removing Ceph OSD nodes, consider that other ongoing processes also affect storage cluster performance. To reduce the impact on client I/O, Red Hat recommends the following:

Calculate capacity

Before removing a Ceph OSD node, ensure that the storage cluster can backfill the contents of all its OSDs without reaching the `full ratio`. Reaching the `full ratio` will cause the storage cluster to refuse write operations.

Temporarily disable scrubbing

Scrubbing is essential to ensuring the durability of the storage cluster's data; however, it is resource intensive. Before adding or removing a Ceph OSD node, disable scrubbing and deep-scrubbing and let the current scrubbing operations complete before proceeding.

```
ceph osd set noscrub
ceph osd set nodeep-scrub
```

Once you have added or removed a Ceph OSD node and the storage cluster has returned to an `active+clean` state, unset the `noscrub` and `nodeep-scrub` settings.

```
ceph osd unset noscrub
ceph osd unset nodeep-scrub
```

Limit backfill and recovery

If you have reasonable data durability, there is nothing wrong with operating in a `degraded` state. For example, you can operate the storage cluster with `osd_pool_default_size = 3` and `osd_pool_default_min_size = 2`. You can tune the storage cluster for the fastest possible recovery time, but doing so significantly affects Ceph client I/O performance. To maintain the highest Ceph client I/O performance, limit the backfill and recovery operations and allow them to take longer.

```
osd_max_backfills = 1
osd_recovery_max_active = 1
osd_recovery_op_priority = 1
```

You can also consider setting the sleep and delay parameters such as, `osd_recovery_sleep`.

Increase the number of placement groups

Finally, if you are expanding the size of the storage cluster, you may need to increase the number of placement groups. If you determine that you need to expand the number of placement groups, Red Hat recommends making incremental increases in the number of placement groups. Increasing the number of placement groups by a significant amount will cause a considerable degradation in performance.

Adding a Ceph OSD node

To expand the capacity of the Red Hat Ceph Storage cluster, you can add an OSD node.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- A provisioned node with a network connection.

Procedure

1. Verify that other nodes in the storage cluster can reach the new node by its short host name.
2. Temporarily disable scrubbing:

   Example

   ```
   [ceph: root@host01 /]# ceph osd set noscrub
   [ceph: root@host01 /]# ceph osd set nodeep-scrub
   ```

3. Limit the backfill and recovery features:

   Syntax

   ```
   ceph tell DAEMON_TYPE.* injectargs --OPTION_NAME VALUE [--OPTION_NAME VALUE]
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph tell osd.* injectargs --osd-max-backfills 1 --osd-
   recovery-max-active 1 --osd-recovery-op-priority 1
   ```

4. Extract the cluster's public SSH keys to a folder:

   Syntax

   ```
   ceph cephadm get-pub-key > ~/PATH
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph cephadm get-pub-key > ~/ceph.pub
   ```

5. Copy ceph cluster's public SSH keys to the root user's `authorized_keys` file on the new host:

   Syntax

   ```
   ssh-copy-id -f -i ~/PATH root@HOST_NAME_2
   ```

   Example

   ```
   [ceph: root@host01 /]# ssh-copy-id -f -i ~/ceph.pub root@host02
   ```

6. Add the new node to the CRUSH map:

Syntax

```
ceph orch host add NODE_NAME IP_ADDRESS
```

Example

```
[ceph: root@host01 /]# ceph orch host add host02 10.10.128.70
```

7. Add an OSD for each disk on the node to the storage cluster.
   - Using *Deploying Ceph OSDs on all available devices*.
   - Using *Deploying Ceph OSDs using service specification*.
   - Using *Deploying Ceph OSDs on specific devices and hosts*.

Important: When adding an OSD node to a Red Hat Ceph Storage cluster, Red Hat recommends adding one OSD daemon at a time and allowing the cluster to recover to an `active+clean` state before proceeding to the next OSD.

Reference

- Setting a specific configuration setting at runtime.
- Adding a bucket
- Moving a bucket

Removing a Ceph OSD node

To reduce the capacity of a storage cluster, remove an OSD node.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to all nodes in the storage cluster.

About this task

Warning: Before removing a Ceph OSD node, ensure that the storage cluster can backfill the contents of all OSDs without reaching the `full ratio`. Reaching the `full ratio` will cause the storage cluster to refuse write operations.

Procedure

1. Check the storage cluster's capacity.

```
ceph df
rados df
ceph osd df
```

2. Temporarily disable scrubbing.

```
ceph osd set noscrub
ceph osd set nodeep-scrub
```

3. Limit the backfill and recovery features.

```
ceph tell DAEMON_TYPE.* injectargs --OPTION_NAME VALUE [--OPTION_NAME VALUE]
```

For example,

```
[ceph: root@host01 /]# ceph tell osd.* injectargs --osd-max-backfills 1 --osd-
recovery-max-active 1 --osd-recovery-op-priority 1
```

4. Remove each OSD on the node from the storage cluster.

> **Important:** When removing an OSD node from the storage cluster, Red Hat recommends removing one OSD at a time within the node and allowing the cluster to recover to an `active+clean` state before proceeding to remove the next OSD.

5. Verify that the storage cluster is not getting to the `near-full ratio`.

```
ceph -s
ceph df
```

Repeat this step until all OSDs on the node are removed from the storage cluster.
Only continue after verifying that all OSDs are removed.

6. Remove the host.
For more information, see Removing hosts.

Related information

Setting a specific configuration at runtime

Simulating a node failure

To simulate a hard node failure, power off the node and reinstall the operating system.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to all nodes on the storage cluster.

Procedure

1. Check the storage cluster's capacity to understand the impact of removing the node:

Example

```
[ceph: root@host01 /]# ceph df
[ceph: root@host01 /]# rados df
[ceph: root@host01 /]# ceph osd df
```

2. Optionally, disable recovery and backfilling:

Example

```
[ceph: root@host01 /]# ceph osd set noout
[ceph: root@host01 /]# ceph osd set noscrub
[ceph: root@host01 /]# ceph osd set nodeep-scrub
```

3. Shut down the node.

4. If you are changing the host name, remove the node from CRUSH map:

Example

```
[ceph: root@host01 /]# ceph osd crush rm host03
```

5. Check the status of the storage cluster:

Example

```
[ceph: root@host01 /]# ceph -s
```

6. Reinstall the operating system on the node.
7. Add the new node:
   - For more information, see Adding hosts.
8. Optionally, enable recovery and backfilling:

   Example

```
[ceph: root@host01 /]# ceph osd unset noout
[ceph: root@host01 /]# ceph osd unset noscrub
[ceph: root@host01 /]# ceph osd unset nodeep-scrub
```

9. Check Ceph's health:

   Example

```
[ceph: root@host01 /]# ceph -s
```

Configuring SNMP traps

As a storage administrator, you can deploy and configure the simple network management protocol (SNMP) gateway in a Red Hat Ceph Storage cluster to receive alerts from the Prometheus Alertmanager and route them as SNMP traps to the cluster.

Simple network management protocol

Simple network management protocol (SNMP) is one of the most widely used open protocols, to monitor distributed systems and devices across a variety of hardware and software platforms. Ceph's SNMP integration focuses on forwarding alerts from its Prometheus Alertmanager cluster to a gateway daemon. The gateway daemon transforms the alert into an SNMP Notification and sends it on to a designated SNMP management platform. The gateway daemon is from the `snmp_notifier_project`, which provides SNMP V2c and V3 support with authentication and encryption.

The Red Hat Ceph Storage SNMP gateway service deploys one instance of the gateway by default. You can increase this by providing placement information. However, if you enable multiple SNMP gateway daemons, your SNMP management platform receives multiple notifications for the same event.

The SNMP traps are alert messages and the Prometheus Alertmanager sends these alerts to the SNMP notifier which then looks for object identifier (OID) in the given alerts' labels. Each SNMP trap has a unique ID which allows it to send additional traps with updated status to a given SNMP poller. SNMP hooks into the Ceph health checks so that every health warning generates a specific SNMP trap.

In order to work correctly and transfer information on device status to the user to monitor, SNMP relies on several components. There are four main components that makeup SNMP:

SNMP Manager

The SNMP manager, also called a management station, is a computer that runs network monitoring platforms. A platform that has the job of polling SNMP-enabled devices and retrieving data from them. An SNMP Manager queries agents, receives responses from agents and acknowledges asynchronous events from agents.

SNMP Agent

- An SNMP agent is a program that runs on a system to be managed and contains the MIB database for the system. These collect data like bandwidth and disk space, aggregates it, and sends it to the management information base (MIB).

Management information base (MIB)

These are components contained within the SNMP agents. The SNMP manager uses this as a database and asks the agent for access to particular information. This information is needed for the network management systems (NMS). The NMS polls the agent to take information from these files and then proceeds to translate it into graphs and displays that can be viewed by the user. MIBs contain statistical and control values that are determined by the network device.

SNMP Devices

These are devices used for SNMP.

"SNMP supported versions" on page 644 lists the compatible and supported SNMP versions for gateway implementation.

Table 93: SNMP supported versions

SNMP version	Description
V2c	Uses a community string without any authentication and is vulnerable to outside attacks.
V3 authNoPriv	Uses the username and password authentication without encryption.
V3 authPriv	Uses the username and password authentication with encryption to the SNMP management platform.

Important: When using SNMP traps, ensure that you have the correct security configuration for your version number to minimize the vulnerabilities that are inherent to SNMP and keep your network protected from unauthorized users.

Configuring `snmptrapd`

It is important to configure the simple network management protocol (SNMP) target before deploying the `snmp-gateway` because the `snmptrapd` daemon contains the auth settings that you need to specify when creating the `snmp-gateway` service.

The SNMP gateway feature provides a means of exposing the alerts that are generated in the Prometheus stack to an SNMP management platform. You can configure the SNMP traps to the destination based on the `snmptrapd` tool. This tool allows you to establish one or more SNMP trap listeners.

The following parameters are important for configuration:

- The `engine-id` is a unique identifier for the device, in hex, and required for SNMPV3 gateway. Red Hat recommends using `8000C53F_CLUSTER_FSID_WITHOUT_DASHES`for this parameter.
- The `snmp-community`, which is the *`SNMP_COMMUNITY_FOR_SNMPV2`* parameter, is `public` for SNMPV2c gateway.
- The `auth-protocol` which is the *`AUTH_PROTOCOL`*, is mandatory for SNMPV3 gateway and is `SHA` by default.
- The `privacy-protocol`, which is the *`PRIVACY_PROTOCOL`*, is mandatory for SNMPV3 gateway.
- The *`PRIVACY_PASSWORD`* is mandatory for SNMPV3 gateway with encryption.
- The *`SNMP_V3_AUTH_USER_NAME`* is the user name and is mandatory for SNMPV3 gateway.
- The *`SNMP_V3_AUTH_PASSWORD`* is the password and is mandatory for SNMPV3 gateway.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the nodes.
- Install `firewalld` on Red Hat Enterprise Linux system.

Procedure

1. On the SNMP management host, install the SNMP packages.

   Example

   ```
   [root@host01 ~]# dnf install -y net-snmp-utils net-snmp
   ```

2. Open the port 162 for SNMP to receive alerts.

   Example

   ```
   [root@host01 ~]# firewall-cmd --zone=public --add-port=162/udp
   [root@host01 ~]# firewall-cmd --zone=public --add-port=162/udp --permanent
   ```

3. Implement the management information base (MIB) to make sense of the SNMP notification and enhance SNMP support on the destination host. Copy the raw file from the main repository. https://github.com/ceph/ceph/blob/master/monitoring/snmp/CEPH-MIB.txt

   Example

   ```
   [root@host01 ~]# curl -o CEPH_MIB.txt -L https://raw.githubusercontent.com/ceph/ceph/
   master/monitoring/snmp/CEPH-MIB.txt
   [root@host01 ~]# scp CEPH_MIB.txt root@host02:/usr/share/snmp/mibs
   ```

4. Create the `snmptrapd` directory.

   Example

   ```
   [root@host01 ~]# mkdir /root/snmptrapd/
   ```

5. Create the configuration files in `snmptrapd` directory for each protocol based on the SNMP version.

   Syntax

   ```
   format2 %V\n% Agent Address: %A \n Agent Hostname: %B \n Date: %H - %J - %K - %L - %M
   - %Y \n Enterprise OID: %N \n Trap Type: %W \n Trap Sub-Type: %q \n Community/Infosec
   Context: %P \n Uptime: %T \n Description: %W \n PDU Attribute/Value Pair Array:\n%v \n
   -------------- \n
   createuser -e 0x_ENGINE_ID_ SNMPV3_AUTH_USER_NAME AUTH_PROTOCOL SNMP_V3_AUTH_PASSWORD
   PRIVACY_PROTOCOL PRIVACY_PASSWORD
   authuser log,execute SNMP_V3_AUTH_USER_NAME
   authCommunity   log,execute,net SNMP_COMMUNITY_FOR_SNMPV2
   ```

   – For SNMPV2c, create the `snmptrapd_public.conf` file as follows:

     Example

     ```
     format2 %V\n% Agent Address: %A \n Agent Hostname: %B \n Date: %H - %J - %K - %L
     - %M - %Y \n Enterprise OID: %N \n Trap Type: %W \n Trap Sub-Type: %q \n
     Community/Infosec Context: %P \n Uptime: %T \n Description: %W \n PDU Attribute/
     Value Pair Array:\n%v \n -------------- \n

     authCommunity   log,execute,net public
     ```

     The `public` setting here must match the `snmp_community` setting used when deploying the `snmp-gateway` service.

   – For SNMPV3 with authentication only, create the `snmptrapd_auth.conf` file.

     Example

     ```
     format2 %V\n% Agent Address: %A \n Agent Hostname: %B \n Date: %H - %J - %K - %L
     - %M - %Y \n Enterprise OID: %N \n Trap Type: %W \n Trap Sub-Type: %q \n
     Community/Infosec Context: %P \n Uptime: %T \n Description: %W \n PDU Attribute/
     Value Pair Array:\n%v \n -------------- \n
     createuser -e 0x8000C53Ff64f341c655d11eb8778fa163e914bcc myuser SHA mypassword
     authuser log,execute myuser
     ```

     The `0x8000C53Ff64f341c655d11eb8778fa163e914bcc` string is the `engine_id`, and `myuser` and `mypassword` are the credentials. The password security is defined by the `SHA` algorithm.

     This corresponds to the settings for deploying the `snmp-gateway` daemon.

     Example

     ```
     snmp_v3_auth_username: myuser
     snmp_v3_auth_password: mypassword
     ```

– For SNMPV3 with authentication and encryption, create the `snmptrapd_authpriv.conf` file.

  Example

```
format2 %V\n% Agent Address: %A \n Agent Hostname: %B \n Date: %H - %J - %K - %L
- %M - %Y \n Enterprise OID: %N \n Trap Type: %W \n Trap Sub-Type: %q \n
Community/Infosec Context: %P \n Uptime: %T \n Description: %W \n PDU Attribute/
Value Pair Array:\n%v \n -------------- \n
createuser -e 0x8000C53Ff64f341c655d11eb8778fa163e914bcc myuser SHA mypassword
DES mysecret
authuser log,execute myuser
```

  The `0x8000C53Ff64f341c655d11eb8778fa163e914bcc` string is the `engine_id`, and `myuser` and `mypassword` are the credentials. The password security is defined by the `SHA` algorithm and `DES` is the type of privacy encryption.

  This corresponds to the settings for deploying the `snmp-gateway` daemon.

  Example

```
snmp_v3_auth_username: myuser
snmp_v3_auth_password: mypassword
snmp_v3_priv_password: mysecret
```

6. Run the daemon on the SNMP management host.

   Syntax

```
/usr/sbin/snmptrapd -M /usr/share/snmp/mibs -m CEPH-MIB.txt -f -C -c /root/snmptrapd/
CONFIGURATION_FILE -Of -Lo :162
```

   Example

```
[root@host01 ~]# /usr/sbin/snmptrapd -M /usr/share/snmp/mibs -m CEPH-MIB.txt -f -C
-c /root/snmptrapd/snmptrapd_auth.conf -Of -Lo :162
```

7. If any alert is triggered on the storage cluster, you can monitor the output on the SNMP management host. Verify the SNMP traps and also the traps decoded by MIB.

   Example

```
NET-SNMP version 5.8
 Agent Address: 0.0.0.0
 Agent Hostname: <UNKNOWN>
 Date: 15 - 5 - 12 - 8 - 10 - 4461391
 Enterprise OID: .
 Trap Type: Cold Start
 Trap Sub-Type: 0
 Community/Infosec Context: TRAP2, SNMP v3, user myuser, context
 Uptime: 0
 Description: Cold Start
 PDU Attribute/Value Pair Array:
.iso.org.dod.internet.mgmt.mib-2.1.3.0 = Timeticks: (292276100) 3 days, 19:52:41.00
.iso.org.dod.internet.snmpV2.snmpModules.1.1.4.1.0 =
OID: .iso.org.dod.internet.private.enterprises.ceph.cephCluster.cephNotifications.prom
etheus.promMgr.promMgrPrometheusInactive
.iso.org.dod.internet.private.enterprises.ceph.cephCluster.cephNotifications.prometheu
s.promMgr.promMgrPrometheusInactive.1 = STRING:
"1.3.6.1.4.1.50495.1.2.1.6.2[alertname=CephMgrPrometheusModuleInactive]"
.iso.org.dod.internet.private.enterprises.ceph.cephCluster.cephNotifications.prometheu
s.promMgr.promMgrPrometheusInactive.2 = STRING: "critical"
.iso.org.dod.internet.private.enterprises.ceph.cephCluster.cephNotifications.prometheu
s.promMgr.promMgrPrometheusInactive.3 = STRING: "Status: critical
- Alert: CephMgrPrometheusModuleInactive
  Summary: Ceph's mgr/prometheus module is not available
  Description: The mgr/prometheus module at 10.70.39.243:9283 is unreachable. This
could mean that the module has been disabled or the mgr itself is down.
Without the mgr/prometheus module metrics and alerts will no longer function. Open a
shell to ceph and use 'ceph -s' to determine whether the mgr is active. If the mgr is
not active, restart it, otherwise you can check the mgr/prometheus module is loaded
with 'ceph mgr module ls'  and if it's not listed as enabled, enable it with 'ceph mgr
module enable prometheus'"
```

   In the above example, an alert is generated after the Prometheus module is disabled.

Reference

- *Deploying the SNMP gateway*

Deploying the SNMP gateway

You can deploy the simple network management protocol (SNMP) gateway using either SNMPV2c or SNMPV3.

There are two methods to deploy the SNMP gateway:

1. By creating a credentials file.
2. By creating one service configuration yaml file with all the details.

You can use the following parameters to deploy the SNMP gateway based on the versions:

- The `service_type` is the `snmp-gateway`.
- The `service_name` is any user-defined string.
- The `count` is the number of SNMP gateways to be deployed in a storage cluster.
- The `snmp_destination` parameter must be of the format hostname:port.
- The `engine-id` is a unique identifier for the device, in hex, and required for SNMPV3 gateway. Red Hat recommends to use `8000C53F_CLUSTER_FSID_WITHOUT_DASHES_`for this parameter.
- The `snmp_community` parameter is `public` for SNMPV2c gateway.
- The `auth-protocol` is mandatory for SNMPV3 gateway and is `SHA` by default.
- The `privacy-protocol` is mandatory for SNMPV3 gateway with authentication and encryption.
- The port is `9464` by default.
- You must provide a `-i _FILENAME_` to pass the secrets and passwords to the orchestrator.

Once the SNMP gateway service is deployed or updated, the Prometheus Alertmanager configuration is automatically updated to forward any alert that has an `objectidentifier` to the SNMP gateway daemon for further processing.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the nodes.
- Configuring `snmptrapd` on the destination host, which is the SNMP management host.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Create a label for the host on which SNMP gateway needs to be deployed:

   Syntax

   ```
   ceph orch host label add HOSTNAME snmp-gateway
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph orch host label add host02 snmp-gateway
   ```

3. Create a credentials file or a service configuration file based on the SNMP version:
   - For SNMPV2c, create the file as follows:

     Example

```
[ceph: root@host01 /]# cat snmp_creds.yml

snmp_community: public
```

OR

Example

```
[ceph: root@host01 /]# cat snmp-gateway.yml

service_type: snmp-gateway
service_name: snmp-gateway
placement:
  count: 1
spec:
  credentials:
    snmp_community: public
  port: 9464
  snmp_destination: 192.168.122.73:162
  snmp_version: V2c
```

– For SNMPV3 with authentication only, create the file as follows:

Example

```
[ceph: root@host01 /]# cat snmp_creds.yml

snmp_v3_auth_username: myuser
snmp_v3_auth_password: mypassword
```

OR

Example

```
[ceph: root@host01 /]# cat snmp-gateway.yml

service_type: snmp-gateway
service_name: snmp-gateway
placement:
  count: 1
spec:
  credentials:
    snmp_v3_auth_password: mypassword
    snmp_v3_auth_username: myuser
  engine_id: 8000C53Ff64f341c655d11eb8778fa163e914bcc
  port: 9464
  snmp_destination: 192.168.122.1:162
  snmp_version: V3
```

– For SNMPV3 with authentication and encryption, create the file.

Example

```
[ceph: root@host01 /]# cat snmp_creds.yml

snmp_v3_auth_username: myuser
snmp_v3_auth_password: mypassword
snmp_v3_priv_password: mysecret
```

OR

Example

```
[ceph: root@host01 /]# cat snmp-gateway.yml

service_type: snmp-gateway
service_name: snmp-gateway
placement:
  count: 1
spec:
  credentials:
    snmp_v3_auth_password: mypassword
    snmp_v3_auth_username: myuser
    snmp_v3_priv_password: mysecret
```

```
engine_id: 8000C53Ff64f341c655d11eb8778fa163e914bcc
port: 9464
snmp_destination: 192.168.122.1:162
snmp_version: V3
```

4. Run the `ceph orch` command:

   Syntax

   ```
   ceph orch apply snmp-gateway --snmp_version=V2c_OR_V3 --destination=SNMP_DESTINATION
   [--port=PORT_NUMBER]\
   [--engine-id=8000C53FCLUSTER_FSID_WITHOUT_DASHES] [--auth-protocol=MDS_OR_SHA] [--
   privacy_protocol=DES_OR_AES] -i FILENAME
   ```

   OR

   Syntax

   ```
   ceph orch apply -i FILENAME.yml
   ```

– For SNMPV2c, with the `snmp_creds` file, run the `ceph orch` command with the `snmp-version` as `V2c`:

  Example

  ```
  [ceph: root@host01 /]# ceph orch apply snmp-gateway --snmp-version=V2c --
  destination=192.168.122.73:162 --port=9464 -i snmp_creds.yml
  ```

– For SNMPV3 with authentication only, with the `snmp_creds` file, run the `ceph orch` command with the `snmp-version` as `V3` and `engine-id`:

  Example

  ```
  [ceph: root@host01 /]# ceph orch apply snmp-gateway --snmp-version=V3 --engine-
  id=8000C53Ff64f341c655d11eb8778fa163e914bcc--destination=192.168.122.73:162 -i
  snmp_creds.yml
  ```

– For SNMPV3 with authentication and encryption, with the `snmp_creds` file, run the `ceph orch` command with the `snmp-version` as `V3`, `privacy-protocol`, and `engine-id`:

  Example

  ```
  [ceph: root@host01 /]# ceph orch apply snmp-gateway --snmp-version=V3 --engine-
  id=8000C53Ff64f341c655d11eb8778fa163e914bcc--destination=192.168.122.73:162 --privacy-
  protocol=AES -i snmp_creds.yml
  ```

  OR

– For all the SNMP versions, with the `snmp-gateway` file, run the following command:

  Example

  ```
  [ceph: root@host01 /]# ceph orch apply -i snmp-gateway.yml
  ```

Reference

– *Configuring `snmptrapd`*

Handling a data center failure

As a storage administrator, you can take preventive measures to avoid a data center failure.

These preventive measures include:

– Configuring the data center infrastructure.

– Setting up failure domains within the CRUSH map hierarchy.

– Designating failure nodes within the domains.

Prerequisites

- A healthy running Red Hat Ceph Storage cluster.
- Root-level access to all nodes in the storage cluster.

Avoiding a data center failure

Use this information to understand how to avoid a data center failure.

Configuring the data center infrastructure

Each data center within a stretch cluster can have a different storage cluster configuration to reflect local capabilities and dependencies. Set up replication between the data centers to help preserve the data. If one data center fails, the other data centers in the storage cluster contain copies of the data.

Setting up failure domains within the CRUSH map hierarchy

Failure, or failover, domains are redundant copies of domains within the storage cluster. If an active domain fails, the failure domain becomes the active domain.

By default, the CRUSH map lists all nodes in a storage cluster within a flat hierarchy. However, for best results, create a logical hierarchical structure within the CRUSH map. The hierarchy designates the domains to which each node belongs and the relationships among those domains within the storage cluster, including the failure domains. Defining the failure domains for each domain within the hierarchy improves the reliability of the storage cluster.

When planning a storage cluster that contains multiple data centers, place the nodes within the CRUSH map hierarchy so that if one data center goes down, the rest of the storage cluster stays up and running.

Designating failure nodes within the domains

If you plan to use three-way replication for data within the storage cluster, consider the location of the nodes within the failure domain. If an outage occurs within a data center, it is possible that some data might reside in only one copy. When this scenario happens, there are two options:

- Leave the data in read-only status with the standard settings.
- Live with only one copy for the duration of the outage.

With the standard settings, and because of the randomness of data placement across the nodes, not all the data will be affected, but some data can have only one copy and the storage cluster would revert to read-only mode. However, if some data exist in only one copy, the storage cluster reverts to read-only mode.

Handling a data center failure

Red Hat Ceph Storage can withstand catastrophic failures to the infrastructure, such as losing one of the data centers in a stretch cluster. For the standard object store use case, configuring all three data centers can be done independently with replication set up between them. In this scenario, the storage cluster configuration in each of the data centers might be different, reflecting the local capabilities and dependencies.

A logical structure of the placement hierarchy should be considered. A proper CRUSH map can be used, reflecting the hierarchical structure of the failure domains within the infrastructure. Using logical hierarchical definitions improves the reliability of the storage cluster, versus using the standard hierarchical definitions. Failure domains are defined in the CRUSH map. The default CRUSH map contains all nodes in a flat hierarchy. In a three data center environment, such as a stretch cluster, the placement of nodes should be managed in a way that one data center can go down, but the storage cluster stays up and running. Consider which failure domain a node resides in when using 3-way replication for the data.

In the following example, the resulting map is derived from the initial setup of the storage cluster with 6 OSD nodes. In this example, all nodes have only one disk and hence one OSD. All of the nodes are arranged under the default *root*, that is the standard *root* of the hierarchy tree. Because there is a weight assigned to two of the OSDs, these OSDs receive fewer chunks of data than the other OSDs. These nodes were introduced later with bigger disks than the initial OSD disks. This does not affect the data placement to withstand a failure of a group of nodes.

For example:

```
[ceph: root@host01 /]# ceph osd tree
ID WEIGHT  TYPE NAME           UP/DOWN REWEIGHT PRIMARY-AFFINITY
-1 0.33554 root default
-2 0.04779     host host03
 0 0.04779         osd.0            up  1.00000          1.00000
-3 0.04779     host host02
 1 0.04779         osd.1            up  1.00000          1.00000
-4 0.04779     host host01
 2 0.04779         osd.2            up  1.00000          1.00000
-5 0.04779     host host04
 3 0.04779         osd.3            up  1.00000          1.00000
-6 0.07219     host host06
 4 0.07219         osd.4            up  0.79999          1.00000
-7 0.07219     host host05
 5 0.07219         osd.5            up  0.79999          1.00000
```

Using logical hierarchical definitions to group the nodes into same data center can achieve data placement maturity. Possible definition types of *root*, *datacenter*, *rack*, *row*, and *host* allow the reflection of the failure domains for the three data center (DC) stretch cluster:

- Nodes host01 and host02 reside in data center 1 (DC1)
- Nodes host03 and host05 reside in data center 2 (DC2)
- Nodes host04 and host06 reside in data center 3 (DC3)
- All data centers belong to the same structure (allDC)

Since all OSDs in a host belong to the host definition there is no change needed. All the other assignments can be adjusted during runtime of the storage cluster by:

- Defining the *bucket* structure with the following commands:

```
ceph osd crush add-bucket allDC root
ceph osd crush add-bucket DC1 datacenter
ceph osd crush add-bucket DC2 datacenter
ceph osd crush add-bucket DC3 datacenter
```

- Moving the nodes into the appropriate place within this structure by modifying the CRUSH map:

```
ceph osd crush move DC1 root=allDC
ceph osd crush move DC2 root=allDC
ceph osd crush move DC3 root=allDC
ceph osd crush move host01 datacenter=DC1
ceph osd crush move host02 datacenter=DC1
ceph osd crush move host03 datacenter=DC2
ceph osd crush move host05 datacenter=DC2
ceph osd crush move host04 datacenter=DC3
ceph osd crush move host06 datacenter=DC3
```

Within this structure any new hosts can be added too, as well as new disks. By placing the OSDs in the correct place in the hierarchy the CRUSH algorithm is changed to place redundant pieces into different failure domains within the structure. This results in the following:

```
[ceph: root@host01 /]# ceph osd tree
ID  WEIGHT  TYPE NAME              UP/DOWN REWEIGHT PRIMARY-AFFINITY
 -8 6.00000 root allDC
 -9 2.00000     datacenter DC1
 -4 1.00000         host host01
  2 1.00000             osd.2           up  1.00000          1.00000
 -3 1.00000         host host02
  1 1.00000             osd.1           up  1.00000          1.00000
-10 2.00000     datacenter DC2
 -2 1.00000         host host03
  0 1.00000             osd.0           up  1.00000          1.00000
 -7 1.00000         host host05
  5 1.00000             osd.5           up  0.79999          1.00000
-11 2.00000     datacenter DC3
 -6 1.00000         host host06
  4 1.00000             osd.4           up  0.79999          1.00000
 -5 1.00000         host host04
  3 1.00000             osd.3           up  1.00000          1.00000
 -1       0 root default
```

The listing shows the resulting CRUSH map by displaying the osd tree. Easy to see is now how the hosts belong to a data center and all data centers belong to the same top level structure but clearly distinguishing between locations.

> **Note:** Placing the data in the proper locations according to the map works only properly within the healthy cluster. Misplacement might happen under circumstances, when some OSDs are not available. Those misplacements will be corrected automatically once it is possible to do so.

Reference

- CRUSH administration

Certificate management

Manage TLS certificates for Red Hat Ceph Storage services with the **cephadm** certificate manager (**certmgr**). Supports self-signed and bring-your-own certificates.

certmgr acts as the Root Certificate Authority (CA) for all self-signed certificates generated by **cephadm**. For services that require SSL, administrators can either bring their own certificate or allow **cephadm** to generate a self-signed certificate. This ensures secure communication while offering flexibility for deployment preferences.

certmgr automatically detects whether a certificate is self-signed (generated by **cephadm**), user-provided as an embedded value in the service specification, or referenced externally. This distinction determines how expiration and renewal are handled.

Self-signed certificates

With self-signed certificates:

- **certmgr** fully automates renewal, helping to ensure seamless service operation.
- Automation is controlled by configuration parameters that define certificate duration, renewal thresholds, and whether automated rotation is enabled.

User-provided certificates

There are two types of user-provided certificates: inline and reference.

Inline certificates

Certificates are embedded in the service specification. These certificates cannot be modified directly with **certmgr**.

Reference certificates

Certificates are set and managed with **certmgr**. These certificates must be uploaded manually.

For more information about how **certmgr** handles user-provided certificate expiration, see “Certificate health monitoring” on page 653.

Lifecycle configuration

To manage certificate lifecycles, **certmgr** continuously monitors certificates and applies renewal policies based on the certificate type and configured parameters. **cephadm** provides several configuration options to manage certificate lifecycle and renewal.

“certmgr certificate lifecycle configuration options” on page 653 details the available configuration options, values, and descriptions.

Table 94: **certmgr** *certificate lifecycle configuration options*

Configuration	Values	Description

mgr/cephadm/certificate_automated_rotation_enabled	**Default**
	`true`
	Additional values
	`false`
		Enabled by default, this configuration option controls whether **cephadm** automatically rotates certificates upon expiration. When disabled, **cephadm** still checks certificates periodically, but instead of automatically renewing self-signed expired certificates, it issues a health error or warning when an issue is detected.

mgr/cephadm/certificate_duration_days	**Default**
	`3 * 365`
	Minimum
	`90`
	Maximum
	`10 * 365`
		Specifies the duration (in days) of self-signed certificates generated and signed by the **cephadm** root CA.

mgr/cephadm/certificate_renewal_threshold_days	**Default**
	`30`
	Minimum
	`10`
	Maximum
	`90`
		Defines the number of days before certificate expiration when **cephadm** initiates renewal. This applies to both self-signed and user-provided certificates. For user-provided certificates, **cephadm** issues a health error or warning alerting administrators to the upcoming expiration.

mgr/cephadm/certificate_check_period	**Default**
	`1`
	Minimum
	`0`
	Maximum
	`30`
		Specifies how often (in days) certificates are checked for validity. Setting this parameter to `0` disables certificate checking.

Certificate health monitoring

cephadm continuously monitors the status of all managed certificates, both self-signed and user-provided.

- If a certificate is invalid or expired, **cephadm** issues a health error (`CEPHADM_CERT_ERROR`).
- If a certificate is approaching its expiration date, as determined by **mgr/cephadm/certificate_renewal_threshold_days**, **cephadm** issues a health warning.

Self-signed certificates are automatically renewed by **cephadm** when automated rotation is enabled, but user-provided certificates must be updated manually.

SSL fields in service specifications

Service specifications that support SSL/TLS can define specific configuration fields. "Optional SSL configuration fields" on page 654 defines the optional configuration fields and their values.

> **Note:** If you set **ssl** to `true` without additional configuration, the certificate manager issues **cephadm**-signed certificates for the service by default.

Table 95: Optional SSL configuration fields	
Field	**Values**
ssl	Boolean to enable or disable SSL (`true` or `false`)
ssl_cert	Use for `inline`: certificate content
ssl_key	Use for `inline`: key content
certificate_source	One of the following: – `inline` – `reference` – `cephadm_signed`

Note: If **certificate_source** is set to `reference` and the certificate is missing, you must upload both the certificate and the key. Use the names shown in the output of the **ceph orch certmgr bindings ls** command:

```
ceph orch certmgr cert set --cert-name CERT_NAME --service-name SERVICE_NAME -i
CERT_FILE
ceph orch certmgr key set --key-name KEY_NAME --service-name SERVICE_NAME -i KEY_FILE
```

Certificate scopes

certmgr supports three scopes for certificate management: global, per-host, and per-service.

Table 96: Certificate scopes		
Scope	**Description**	**Example services**
Global	Certificates in this scope are shared across all service daemons regardless of the host on which they run.	`mgmt-gateway` certificate is a globally shared certificate used by all service daemons.
Per-host	Certificates are assigned per host, meaning each host has its own unique certificate. When configuring a custom certificate, specify the target host.	`grafana` service certificates are configured at the host level and apply specifically to a single machine. For step-by-step instructions, see Configuring a Grafana certificate.
Per-service	Certificates are configured per service name, where each service instance can have its own certificate. When specifying a custom certificate, define the target service name.	A Ceph Object Gateway (`rgw`) service certificate is assigned specifically to a Ceph Object Gateway service.

Related information

Configuring a Grafana certificate

Manage certificates with `certmgr`

Use `certmgr` to list, check, retrieve, upload, generate, and remove TLS certificates and keys for Red Hat Ceph Storage services.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster with `cephadm`.
- Administrator access to the `cephadm` shell.

List certificates

List all certificates that `certmgr` manages.

Procedure

- List all certificates, by using the **`certmgr cert ls`** command.

  > **Note:** By default, the command omits `cephadm`-signed certificates. To include the `cephadm`-signed certificates, use the **`--include-cephadm-signed`** option.

  ```
  ceph orch certmgr cert ls
  ```

  The **`certmgr cert ls`** command includes the following filtering options:

  `--show-details`

  The basic command displays an overview of all certificates currently managed by `cephadm`. Use the **`--show-details`** option to include additional information such as issuing authorities and certificate extensions.

  `--include-cephadm-signed`

  Use this option to include `cephadm`-signed certificates.

  `--filter-by`

  Filter by name, status, and scope.

  ```
  ceph orch certmgr cert ls [--show-details] [--include-cephadm-signed] [--filter-by
  EXPRESSION]
  ```

  For example,

  ```
  [ceph: root@host01 /]# ceph orch certmgr cert ls --include-cephadm-signed
  [ceph: root@host01 /]# ceph orch certmgr cert ls --filter-by "status=expired"
  [ceph: root@host01 /]# ceph orch certmgr cert ls --filter-by
  "scope=service,status=expiring"
  [ceph: root@host01 /]# ceph orch certmgr cert ls --include-cephadm-signed --filter-by
  "name=rgw*,status=valid"
  ```

List certificate bindings

Show bindings between services (consumers) and the certificates/keys managed by `certmgr`.

About this task

You will reuse these names when retrieving or setting scoped material.

Procedure

- Display all entities that are associated with managed certificates.

  ```
  ceph orch certmgr bindings ls
  ```

Check certificate status

Validate integrity and expiration across all managed certificates.

Procedure

– Check the validity and expiration for certificates that `certmgr` manages.

```
ceph orch certmgr cert check
```

List certificate keys

List private keys managed by `certmgr`.

Procedure

– List the private keys.

> **Note:** By default, the command omits `cephadm`-signed keys. To include the `cephadm`-signed keys, use the **`--include-cephadm-signed`** option.

```
ceph orch certmgr key ls [--include-cephadm-signed]
```

Retrieve a certificate

Retrieve a certificate for inspection or export.

Before you begin

Before you begin, identify the certificate name and relevant `service_name` or `hostname`. Use these when the certificate scope is per-service or per-host. For more information, see “List certificate bindings” on page 655.

Procedure

– Get the PEM content of a specific certificate for inspection or export.

```
ceph orch certmgr cert get CERTIFICATE_NAME [--service_name SERVICE_NAME] [--hostname
HOSTNAME] [--no-exception-when-missing]
```

Replace *`CERTIFICATE_NAME`* with the certificate name.

- Use the **`--service_name`** argument for certificates with service scope.
- Use the **`--hostname`** argument for certificates with host scope.

Retrieve a certificate key

Retrieve a private key associated with a specific certificate.

Before you begin

Before you begin, identify the key name and relevant `service_name` or `hostname`. Use these when the certificate scope is per-service or per-host. For more information, see “List certificate bindings” on page 655.

Procedure

– Get the private key.

```
ceph orch certmgr key get KEY_NAME [--service_name SERVICE_NAME] [--hostname HOSTNAME]
[--no-exception-when-missing]
```

Replace *`KEY_NAME`* with the certificate key name.

- Use the **`--service_name`** argument for certificates with service scope.
- Use the **`--hostname`** argument for certificates with host scope.

Set a certificate-key pair

Set a certificate-key pair to upload or replace an existing certificate-key pair for a certain service.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Identify the certificate name to use as the *SERVICE_TYPE*.
- Identify other relevant fields, as needed, such as the `service_name` or `hostname`.

For more information, see "List certificate bindings" on page 655.

Procedure

- Set the certificate-key pair for a service.

```
ceph orch certmgr cert-key set SERVICE_TYPE [--cert CERTIFICATE] [--key KEY] [--
service_name SERVICE_NAME] [--hostname HOSTNAME] [-i CERT_KEY_PATH] [--force]
```

  Use the **-i** option to specify a file containing a combined certificate and key in PEM format.

> **Note:** When specifying a combined certificate and key be sure that the file contains both the certificate and private key placed together in a single PEM file in sequence.

Set a certificate

Add or replace an existing certificate.

Procedure

- Set or update the certificate.

```
ceph orch certmgr cert set CERTIFICATE_NAME [--cert CERTIFICATE] [--service_name
SERVICE_NAME] [--hostname HOSTNAME] [-i CERT_KEY_PATH]
```

Set a certificate key

Administrators can provide new private keys for services.

Procedure

- Set or update the key.

```
ceph orch certmgr key set KEY_NAME [--key KEY] [--service_name SERVICE_NAME] [--
hostname HOSTNAME] [-i CERT_KEY_PATH]
```

Remove a certificate

Use this information to remove an existing certificate.

Before you begin

Removing a certificate requires a valid certificate name. To get the certificate name, use the **certmgr cert ls** command. for more information, see "List certificates" on page 655.

Procedure

- Remove an existing certificate.

```
ceph orch certmgr cert rm CERTIFICATE_NAME [--service_name SERVICE_NAME] [--hostname
HOSTNAME]
```

  - Use the **--service_name** argument for certificates with service scope.

- Use the **--hostname** argument for certificates with host scope.

Remove a certificate key

Use this information to remove an existing private keys.

Before you begin

Removing a private key requires a valid key name. To get the key name, use the **certmgr key ls** command. for more information, see “List certificate keys” on page 656.

Procedure

– Remove an existing key.

```
ceph orch certmgr key rm KEY_NAME [--key KEY] [--service_name SERVICE_NAME] [--hostname
HOSTNAME]
```

- Use the **--service_name** argument for certificates with service scope.
- Use the **--hostname** argument for certificates with host scope.

Generate certificates for a module

Automatically provision certificates for a Manager module.

About this task

Generating certificates for a specified Manager module is typically used for specified modules that require automatic TLS provisioning, such as the Ceph Dashboard.

Procedure

– Generate a new certificate.

```
ceph orch certmgr generate-certificates MODULE_NAME
```

Red Hat Ceph Storage Dashboard

As a storage administrator, the Red Hat Ceph Storage Dashboard provides management and monitoring capabilities. Use the dashboard to administer and configure the cluster, and visualize information and performance statistics that are related to it. The dashboard uses a web server that is hosted by the `ceph-mgr` daemon.

The dashboard is accessible from a web browser and includes many useful management and monitoring features. For example, use the dashboard to configure manager modules and monitor the state of OSDs.

> **Note:** To use the Red Hat Ceph Storage dashboard, have system administrator level experience.

The dashboard provides the following features:

Multi-user and role management

The dashboard supports multiple user accounts with different permissions and roles. User accounts and roles can be managed by using both the command line and the web user interface. The dashboard supports various methods to enhance password security. Password complexity rules can be configured, requiring users to change their password after the first login or after a configurable time period.
For more information, see “Managing roles” on page 688 and “Managing users” on page 692.

Single Sign-On (SSO)

The dashboard supports authentication with an external identity provider by using the SAML 2.0 protocol.
For more information, see “Enabling single sign-on with SAML 2.0” on page 683.

Auditing

The dashboard backend can be configured to log all PUT, POST, and DELETE API requests in the Ceph manager log.
For more information about using the manager modules with the dashboard, see “Viewing and editing the manager modules of the Ceph cluster” on page 391.

High availability (Technology Preview)

Both Ceph management and Ceph monitoring uses high availability. High availability provides the users with transparent manager failover of the `mgmt-gateway`. High availability redirects to the active manager and to the active instance of Prometheus, Alertmanager or Grafana, when several instances are available.

> **Important:** To use high availability with the Ceph Dashboard, ensure that there are no port conflicts that might disrupt service availability. High-availability configurations and clustering for the mgmt-gateway service itself are currently not supported. Services must bind to the appropriate ports based on the applications being proxied.

Security features

The dashboard provides the following security features.

SSL and TLS support

All HTTP communication between the web browser and the dashboard is secured through SSL. A self-signed certificate can be created with a built-in command, but it is also possible to import custom certificates that are signed and issued by a Certificate Authority (CA).
For more information, see “Bootstrap storage cluster along with Dashboard” on page 198.

Username and password protection

You can access the dashboard only by providing a configurable username and password.
For more information, see “Managing users” on page 692.

Configuring and managing the dashboard interface

Configure and manage the Ceph Dashboard.

Configuring manager modules

You can view and change parameters for Ceph manager modules.

For more information, see "Viewing and editing the manager modules of the Ceph cluster" on page 391.

Embedded Grafana dashboards

Ceph Dashboard Grafana dashboards might be embedded in external applications and web pages to surface information with Prometheus modules gathering the performance metrics.
For more information, see "Components" on page 662.

Viewing and filtering logs

You can view event and audit cluster logs and filter them based on priority, keyword, date, or time range.
For more information, see "Filtering logs of the Ceph cluster" on page 394.

Toggling dashboard components

You can enable and disable dashboard components so only the features you need are available.
For more information, see "Toggling Ceph dashboard features" on page 669.

Quality of service for images

You can set performance limits on images. For example, limiting IOPS or read BPS burst rates.
For more information, see "Managing block device images" on page 772.

Ceph management with the dashboard

Manage and monitor Red Hat Ceph Storage with the Ceph Dashboard.

Upgrading

You can upgrade the Ceph cluster version by using the dashboard.
For more information, see "Upgrading a cluster" on page 676.

Overall cluster health

Displays performance and capacity metrics. The cluster health also displays the overall cluster status and storage usage. For example, number of objects, raw capacity, usage per pool, a list of pools and their status and usage statistics.
For more information, see "Viewing and editing the configuration of the Ceph cluster" on page 390.

Configuration editor

Displays all the available configuration options, their descriptions, types, default, and currently set values. These values are editable.
For more information, see "Viewing and editing the configuration of the Ceph cluster" on page 390.

Managing OSD settings

You can set cluster-wide OSD flags by using the dashboard. You can also Mark OSDs up, down or out, purge and reweight OSDs, run scrub operations, modify various scrub-related configuration options, and select profiles to adjust the level of backfilling activity. You can set and change the device class of an OSD, display, and sort OSDs by device class. You can deploy OSDs on new drives and hosts.
For more information, see "Managing Ceph OSDs from the Ceph Dashboard" on page 706.

Pools

Lists and manages all Ceph pools and their details. For example, applications, placement groups, replication size, EC profile, quotas, and CRUSH ruleset.
For more information, see "Understanding the landing page of the Ceph dashboard" on page 664 and "Monitoring pools of the Ceph cluster" on page 398.

OSDs

Lists and manages all OSDs, their status, and usage statistics. **OSDs** also lists detailed information, for example, attributes, OSD map, metadata, and performance counters for read and write operations. **OSDs** also lists all drives that are associated with an OSD.
For more information, see "Monitoring Ceph OSDs" on page 392.

Images

Lists all Ceph Block Device (RBD) images and their properties such as size, objects, and features. Create, copy, modify, and delete RBD images. Create, delete, and rollback snapshots of selected images, protect or unprotect these snapshots against modification. Copy or clone snapshots and flatten cloned images.

> **Note:** The performance graph for I/O changes in the **Overall Performance** tab for a specific image shows values only after specifying the pool that includes that image by setting the **`rbd_stats_pool`** parameter in **Cluster › Manager modules › Prometheus**.

For more information, see “Monitoring block device images” on page 400.

Block device mirroring

Enables and configures Ceph Block Device (RBD) mirroring to a remote Ceph server. Lists all active sync daemons and their status, pools, and RBD images including their synchronization state.
For more information, see “Mirroring view” on page 780.

Ceph File Systems

Lists all active Ceph File System (CephFS) clients and associated pools, including their usage statistics. Evict active CephFS clients, manage CephFS quotas and snapshots, and browse a CephFS directory structure.
For more information, see “Monitoring Ceph File Systems” on page 399.

Object Gateway (RGW)

Lists all active object gateways and their performance counters. This window displays and manages, including add, edit, and delete, Ceph Object Gateway users and their details. This includes quotas and users’ buckets and their details, for example, owner or quotas.
For more information, see “Monitoring Ceph Object Gateway daemons” on page 399.

NFS

Manages NFS shares of CephFS and Ceph Object Gateway S3 buckets by using the Ceph NFS service..
For more information, see Managing NFS shares.

Ceph NVMe-oF gateway

Manages the Ceph NVMe-oF gateway and subsystems. You can configure NVMe-oF services, subsystems, listeners, and namespaces. You can also add or remove an initiator NQN masking per subsystem.
For more information, see Managing the Ceph NVMe-oF gateway.

Monitoring with the dashboard

Monitor the following components with the dashboard.

Viewing alerts

Use the alerts page to see details of current alerts.
For more information, see “Viewing alerts” on page 730.

Viewing cluster hierarchy

You can view the CRUSH map, for example, to determine which host a specific OSD ID is running on. This is helpful if an issue with an OSD occurs.
For more information, see “Viewing the CRUSH map of the Ceph cluster” on page 394.

Cluster logs

Displays and filters the latest updates to the cluster’s event and audit log files by priority, date, or keyword.
For more information, see “Filtering logs of the Ceph cluster” on page 394.

Centralized logs

Provides a centralized location for viewing logs in the Red Hat Ceph Storage cluster for more efficient monitoring.
For more information, see “Viewing centralized logs of the Ceph cluster” on page 395.

Hosts

Provides a list of all hosts that are associated with the cluster along with the running services and the installed Ceph version.

For more information, see “Monitoring hosts of the Ceph cluster” on page 389.

Performance counters

Displays detailed statistics for each running service.
For more information, see “Monitoring services of the Ceph cluster” on page 392.

Monitors

Lists all Monitors, their quorum status, and open sessions.
For more information, see “Monitoring monitors of the Ceph cluster” on page 392.

Device management

Lists all hosts that are known by the Orchestrator. Lists all drives that are attached to a host and their properties. Displays drive health predictions, SMART data, and blink enclosure LEDs.
For more information, see “Monitoring hosts of the Ceph cluster” on page 389.

View storage cluster capacity

You can view the raw storage capacity of the Red Hat Ceph Storage cluster in the Capacity pages of the Ceph dashboard.
For more information, see “Understanding the landing page of the Ceph dashboard” on page 664.

Components

Understand the different components of the dashboard.

The functionality of the dashboard is provided by multiple components.

- The `cephadm` application for deployment.
- The embedded dashboard `ceph-mgr` module.
- The embedded Prometheus `ceph-mgr` module.
- The Prometheus time-series database.
- The Prometheus node-exporter daemon, running on each host of the storage cluster.
- The Grafana platform to provide monitoring user interface and alerting.

Related information

Prometheus website
Grafana website

Architecture

The Dashboard architecture depends on the Ceph manager dashboard plugin and other components.

“Figure: Ceph Dashboard architecture” on page 663 illustrates how the Ceph manager and dashboard work together.

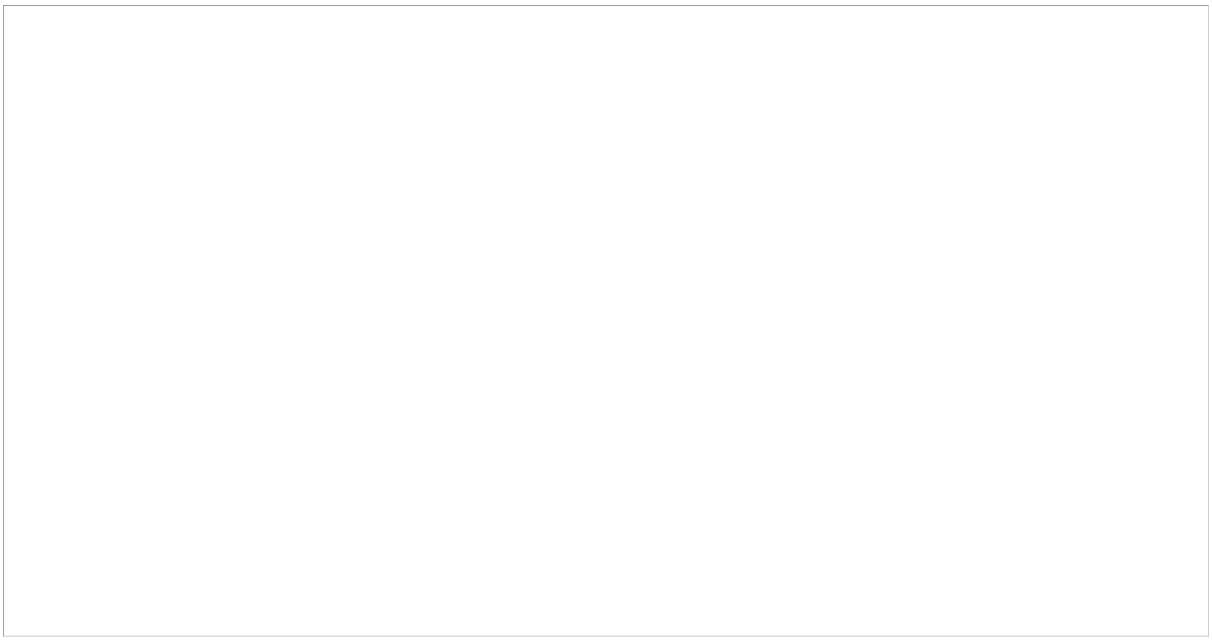


Figure 55: Ceph Dashboard architecture

Accessing the Ceph dashboard after bootstrap

You can access the Ceph dashboard to administer and monitor your Red Hat Ceph Storage cluster.

Before you begin

Be sure that the following is in place before accessing the Ceph dashboard:

- Successful installation of Red Hat Ceph Storage dashboard.
- NTP is synchronizing clocks properly.

Procedure

1. Log into the dashboard through a web browser.
   This can be done in one of the following ways:
   - Enter the following URL:

     ```
     https://HOST_NAME:PORT
     ```

     Replace:
     - *HOST_NAME* with the fully qualified domain name (FQDN) of the active manager host.
     - *PORT* with port `8443`.

     For example,

     ```
     https://host01:8443
     ```

   - From the `cephadm` shell, run the **`ceph mgr services`** command.
     For example,

     ```
     [ceph: root@host01 /]# ceph mgr services
     ```

     This command displays all endpoints that are currently configured. Look for the `dashboard` key to obtain the URL for accessing the dashboard.
2. From the login page, enter the username `admin` and the default password provided during bootstrapping.

> **Note:** During initial login to the Red Hat Ceph Storage dashboard, you must change the password. For more information, see “Changing the dashboard password” on page 671.

3. After logging in, the dashboard default landing page is displayed, which provides details, a high-level overview of status, performance, inventory, and capacity metrics of the Red Hat Ceph Storage cluster.

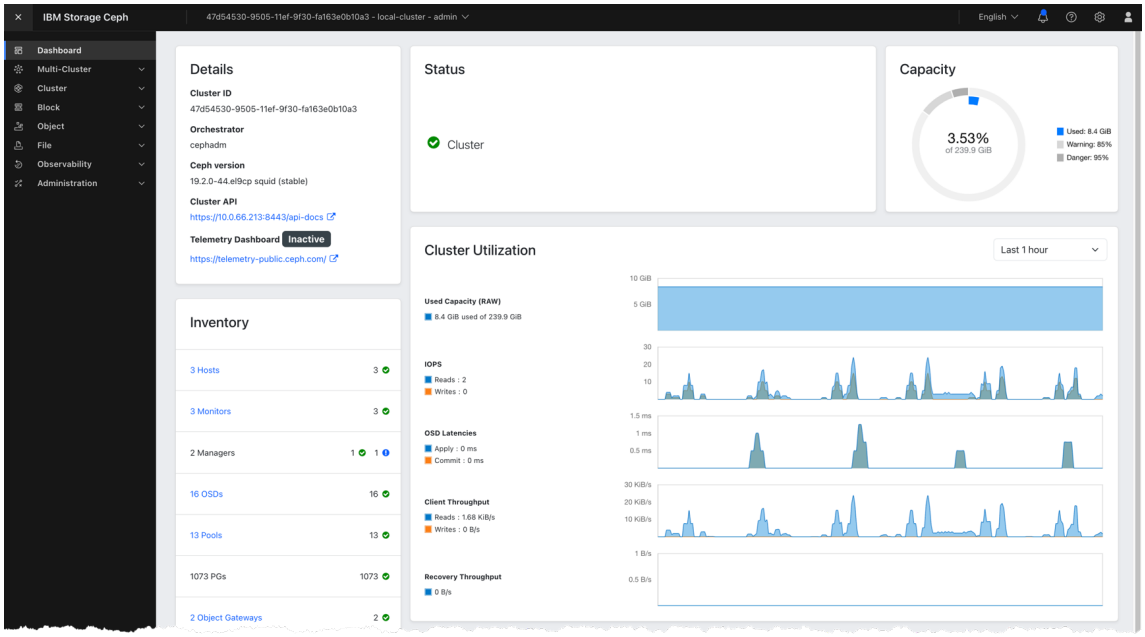


Figure 56: Red Hat Ceph Storage dashboard landing page

4. Click menu icon ( ) on the dashboard landing page to collapse or display the options in the vertical menu.

Understanding the landing page of the Ceph dashboard

The landing page displays an overview of the entire Ceph cluster using navigation bars and individual panels.

Menu bar

The menu bar provides the following options:

Tasks and Notifications

Provides task and notification messages.

Help

Provides links to the product and REST API documentation, details about the Red Hat Ceph Storage Dashboard, and a form to report an issue.

Dashboard Settings

Gives access to user management and telemetry configuration.

User

Use this menu to see log in status, to change a password, and to sign out of the dashboard.



The navigation menu can be opened or hidden by clicking the navigation menu icon ( ).

Dashboard

The main dashboard displays specific information about the state of the cluster. The main dashboard can be accessed at any time by clicking **Dashboard** from the navigation menu. The dashboard landing page organizes the panes into different categories.

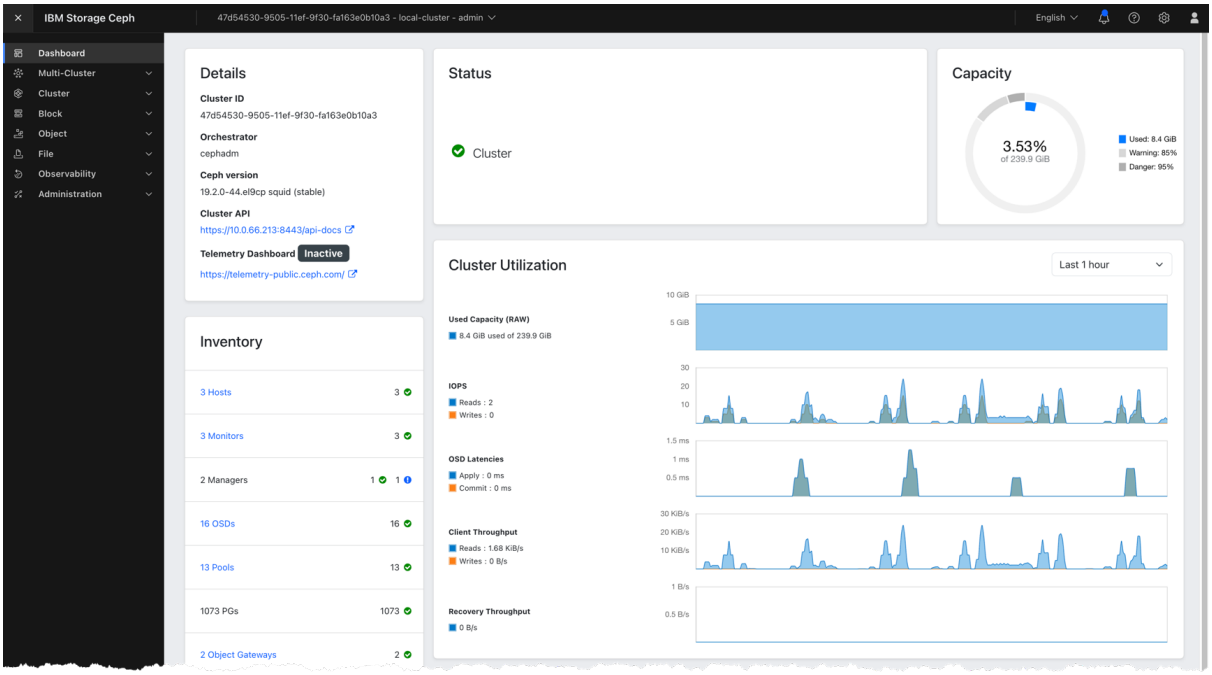


Figure 57: Ceph dashboard landing page

Details

Displays specific cluster information and if telemetry is active or inactive.

Inventory

Displays the different parts of the cluster, how many are available, and their status.
Link directly from **Inventory** to specific inventory items, where available.

Hosts

Displays the total number of hosts in the Ceph storage cluster.

Monitors

Displays the number of Ceph Monitors and the quorum status.

Managers

Displays the number and status of the Manager Daemons.

OSDs

Displays the total number of OSDs in the Ceph Storage cluster and the number that are *up*, and *in*.

Pools

Displays the number of storage pools in the Ceph cluster.

PGs

Displays the total number of placement groups (PGs). The PG states are divided into`Working`and `Warning`to simplify the display. Each one encompasses multiple states.
The`Working`state includes PGs with any of the following states:

- activating
- backfill_wait
- backfilling
- creating
- deep

- degraded
- forced_backfill
- forced_recovery
- peering
- peered
- recovering
- recovery_wait
- repair
- scrubbing
- snaptrim
- snaptrim_wait

TheWarningstate includes PGs with any of the following states:

- backfill_toofull
- backfill_unfound
- down
- incomplete
- inconsistent
- recovery_toofull
- recovery_unfound
- remapped
- snaptrim_error
- stale
- undersized

Object Gateways

Displays the number of Object Gateways in the Ceph storage cluster.

Metadata Servers

Displays the number and status of metadata servers for Ceph File Systems (CephFS).

Status

Displays the health of the cluster and host and daemon states. The current health status of the Ceph storage cluster is displayed. Danger and warning alerts are displayed directly on the landing page. Click **View alerts** for a full list of alerts.

Capacity

Displays storage usage metrics. This is displayed as a graph of used, warning, and danger. The numbers are in percentages and in GiB.

Cluster Utilization

The **Cluster Utilization** pane displays information related to data transfer speeds. Select the time range for the data output from the list. Select a range between the last 5 minutes to the last 24 hours.

Used Capacity (RAW)

Displays usage in GiB.

IOPS

Displays total I/O read and write operations per second.

OSD Latencies

Displays total applies and commits per millisecond.

Client Throughput

Displays total client read and write throughput in KiB per second.

Recovery Throughput

Displays the rate of cluster healing and balancing operations. For example, the status of any background data that may be moving due to a loss of disk is displayed. The information is displayed in bytes per second.

Related information

Monitor the cluster

Configuring the Dashboard

Configure the Red Hat Ceph Storage to provide a secure, web-based interface for monitoring cluster health, managing Ceph services, and performing administrative tasks.

Enabling Red Hat Ceph Storage Dashboard manually

If you have installed a Red Hat Ceph Storage cluster by using `--skip-dashboard` option during bootstrap, you can see that the dashboard URL and credentials are not available in the bootstrap output. You can enable the dashboard manually using the command-line interface. Although the monitoring stack components such as Prometheus, Grafana, Alertmanager, and node-exporter are deployed, they are disabled and you have to enable them manually.

Prerequisites

- A running Red Hat Ceph Storage cluster installed with `--skip-dashboard` option during bootstrap.
- Root-level access to the host on which the dashboard needs to be enabled.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Check the Ceph Manager services:

   Example

   ```
   [ceph: root@host01 /]# ceph mgr services
   {
       "prometheus": "http://10.8.0.101:9283/"
   }
   ```

   You can see that the Dashboard URL is not configured.

3. Enable the dashboard module:

   Example

   ```
   [ceph: root@host01 /]# ceph mgr module enable dashboard
   ```

4. Create the self-signed certificate for the dashboard access:

   Example

   ```
   [ceph: root@host01 /]# ceph dashboard create-self-signed-cert
   ```

> **Note:** You can disable the certificate verification to avoid certification errors.

5. Check the Ceph Manager services:

   Example

   ```
   [ceph: root@host01 /]# ceph mgr services

   {
       "dashboard": "https://10.8.0.101:8443/",
       "prometheus": "http://10.8.0.101:9283/"
   }
   ```

6. Create the admin user and password to access the dashboard:

   Syntax

   ```
   echo -n "PASSWORD" > PASSWORD_FILE
   ceph dashboard ac-user-create admin -i PASSWORD_FILE administrator
   ```

   Example

   ```
   [ceph: root@host01 /]# echo -n "p@ssw0rd" > password.txt
   [ceph: root@host01 /]# ceph dashboard ac-user-create admin -i password.txt
   administrator
   ```

7. Enable the monitoring stack. See Enabling monitoring stack.

Setting message of the day (MOTD)

Sometimes, Ceph Dashboard users need to know about the latest news, updates, and information on Red Hat Ceph Storage.

Before you begin

- A running Red Hat Ceph Storage cluster with the monitoring stack installed.
- Root-level access to the `cephadm` host.
- The `dashboard` module enabled.

About this task

As a storage administrator, you can configure a message of the day (MOTD) in the command-line interface (CLI). When the user logs in to the Ceph Dashboard, the configured MOTD is displayed in a banner within the Ceph Dashboard, similar to the Telemetry module. You can configure MOTD based on the severity, such as `info`, `warning`, or `danger`.

You can close an MOTD with a `info` or `warning` severity. The `info` MOTD is not displayed anymore until the local storage cookies are cleared or a new MOTD with a different severity is displayed. A MOTD with a `warning` severity is displayed again in a new session.

Procedure

1. Configure a MOTD for the dashboard.
   Syntax

   ```
   ceph dashboard motd set SEVERITY EXPIRES MESSAGE
   ```

   Replace:

   - `SEVERITY` can be `info`, `warning`, or `danger`.
   - `EXPIRES` can be for seconds (`s`), minutes (`m`), hours (`h`), days (`d`), weeks (`w`), or never expires (`0`).
   - `MESSAGE` can be any custom message that users can view when they log in to the dashboard.

For example,

```
[ceph: root@host01 /]# ceph dashboard motd set danger 2d "Custom login message"
 Message of the day has been set.
```

2. **Optional:** Set the MOTD that does not expire.
For example,

```
[ceph: root@host01 /]# ceph dashboard motd set danger 0 "Custom login message"
 Message of the day has been set.
```

3. Get the configured MOTD.
For example,

```
[ceph: root@host01 /]# ceph dashboard motd get
 Message="Custom login message", severity="danger",
expires="2022-09-08T07:38:52.963882Z"
```

4. **Optional:** Clear the configured MOTD with the `clear` option.
For example,

```
[ceph: root@host01 /]# ceph dashboard motd clear
 Message of the day has been cleared.
```

5. Verify the message of the day.

```
https://HOST_NAME:8443
```

Related information

Activating and deactivating telemetry

Toggling Ceph dashboard features

You can customize the Red Hat Ceph Storage dashboard components by enabling or disabling features on demand. All features are enabled by default.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Installation and configuration of the Red Hat Ceph Storage Dashboard software.
- User access to the Ceph Manager host or the dashboard web interface.
- Root level access to the Ceph Manager host.

About this task

When disabling a feature, the web-interface elements become hidden and the associated REST API end-points reject any further requests for that feature. Enabling and disabling dashboard features can be done from the command-line interface or the web interface.
Available features:

- Ceph Block Devices:
  - Image management, `rbd`
  - Mirroring, `mirroring`
- Ceph File System, `cephfs`
- Ceph Object Gateway, `rgw`

Note:

- By default, the Ceph Manager is collocated with the Ceph Monitor.

– You can disable multiple features at once.

Important: Once a feature is disabled, it can take up to 20 seconds to reflect the change in the web interface.

Toggling the dashboard features from the web interface

Procedure

1. From the dashboard navigation, go to **› Administration › Manager Modules**.
2. Click **Edit**.
3. In the **Edit Manager module** form, you can enable or disable the dashboard features by selecting or clearing the check boxes next to the different feature names.

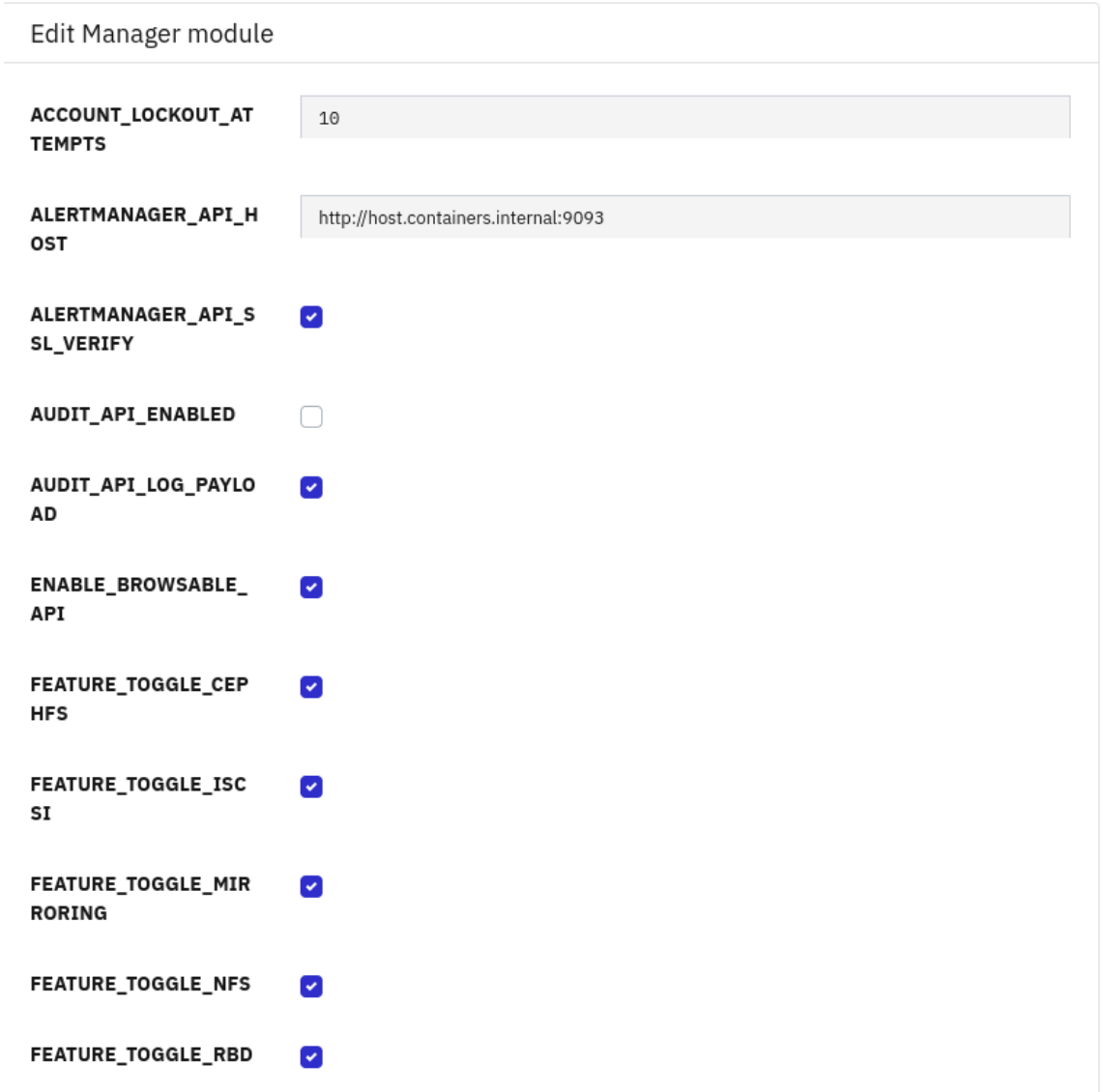


Figure 58: Editing manager modules

4. After the selections are made, click **Update**.

Toggling the dashboard features from the command-line interface

Procedure

1. Login to `cephadm` shell.

   ```
   cephadm shell
   ```

2. List the feature status.

   ```
   ceph dashboard feature status
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph dashboard feature status
   ```

3. Disable a feature.

   ```
   ceph dashboard feature disable FEATURE
   ```

   The following example disables the Ceph Object Gateway feature.

   ```
   [ceph: root@host01 /]# ceph dashboard feature disable rgw
   ```

4. Enable a feature.

   ```
   ceph dashboard feature enable FEATURE
   ```

   The following example enables the Ceph File System feature.

   ```
   [ceph: root@host01 /]# ceph dashboard feature enable cephfs
   ```

Changing the dashboard password

By default, the password for accessing dashboard is randomly generated by the system while bootstrapping the cluster. You have to change the password the first time you log in to the Red Hat Ceph Storage dashboard. You can change the password for the `admin` user using the dashboard.

Prerequisites

– A running Red Hat Ceph Storage cluster.

Procedure

1. Log in to the dashboard:

   ```
   https://HOST_NAME:8443
   ```

2. Click the *Dashboard Settings* icon and then click *User management*.
3. To change the password of **admin**, click it's row.
4. From the *Edit* drop-down menu, select *Edit*.
5. In the *Edit User* window, enter the new password, and change the other parameters, and then Click *Edit User*.

You will be logged out and redirected to the log-in screen. A notification appears confirming the password change.

Changing the Ceph dashboard password using the command line interface

If you have forgotten your Ceph dashboard password, you can change the password using the command line interface.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the host on which the dashboard is installed.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Create the `dashboard_password.yml` file:

   Example

   ```
   [ceph: root@host01 /]# touch dashboard_password.yml
   ```

3. Edit the file and add the new dashboard password:

   Example

   ```
   [ceph: root@host01 /]# vi dashboard_password.yml
   ```

4. Reset the dashboard password:

   Syntax

   ```
   ceph dashboard ac-user-set-password DASHBOARD_USERNAME -i PASSWORD_FILE
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph dashboard ac-user-set-password admin -i
   dashboard_password.yml
   {"username": "admin", "password":
   "$2b$12$i5RmvN1PolR61Fay0mPgt.GDpcga1QpYsaHUbJfoqaHd1rfFFx7XS", "roles":
   ["administrator"], "name": null, "email": null, "lastUpdate": , "enabled": true,
   "pwdExpirationDate": null, "pwdUpdateRequired": false}
   ```

Verification

- Log in to the dashboard with your new password.

Setting `admin` user password for Grafana

By default, `cephadm` does not create an admin user for Grafana. With the Ceph Orchestrator, you can create an admin user and set the password. With these credentials, you can log in to the storage cluster's Grafana URL with the given password for the admin user.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster with the monitoring stack installed.
- Root-level access to the `cephadm` host.
- The `dashboard` module enabled.

Procedure

1. As a root user, create a `grafana.yml` file.
   Provide the following details:

   ```
   service_type: grafana
   spec:
     initial_admin_password: PASSWORD
   ```

2. Mount the `grafana.yml` file under a directory in the container.
   For example,

   ```
   [root@host01 ~]# cephadm shell --mount grafana.yml:/var/lib/ceph/grafana.yml
   ```

   > **Note:** Every time you exit the shell, you have to mount the file in the container before deploying the daemon.

3. Optional: Verify and enable that the `dashboard` Ceph Manager module is enabled.
   a. Check that the `dashboard` module is enabled.

      ```
      ceph mgr module ls
      ```

   b. Enable the `dashboard` module.

      ```
      ceph mgr enable dashboard
      ```

4. Apply the specification file.

   ```
   ceph orch apply -i FILE_NAME.yml
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch apply -i /var/lib/ceph/grafana.yml
   ```

5. Redeploy the `grafana` service.
   This creates an admin user called `admin` with the given password and the user can log in to the Grafana URL with these credentials.

   ```
   ceph orch redeploy grafana
   ```

What to do next

Log into Grafana with the proper credentials.

```
https://HOST_NAME:PORT
```

For example,

```
https://host01:3000/
```

Configuring Prometheus data retention

You can control how long Prometheus stores monitoring data and how much disk space it consumes by configuring retention settings in the Prometheus service specification. Prometheus is the component responsible for storing metrics; therefore, retention policies must be applied to the Prometheus spec.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Prometheus is deployed and managed by Cephadm.
- You have cluster administrator privileges.

– Prometheus is running.

```
ceph orch ls | grep prometheus
```

Procedure

1. Create or edit the `prometheus.yaml` specification file.

```
service_type: prometheus
service_name: prometheus
placement:
  count: 1
spec:
  retention_time: "30d"     # Retain data for 30 days
  retention_size: "50GB"    # Limit storage to 50 GB
```

2. Review the parameter descriptions:

   `retention_time`

   Defines how long Prometheus retains monitoring data. Supported units: `y`, `w`, `d`, `h`, `m`, `s`. The default value is 15 days.

   `retention_size`

   Defines the maximum size of Prometheus time-series database (TSDB). Supported units: `B`, `KB`, `MB`, `GB`, `TB`, `PB`, `EB`. The default value is 0 (disabled)

3. Apply the updated specification file.

```
ceph orch apply -i prometheus.yaml
```

4. Redeploy Prometheus to activate changes.

```
ceph orch redeploy prometheus
```

5. Verify that the retention configuration is active:

```
ceph orch ps | grep prometheus
```

Example

Example
The following example configures Prometheus to retain monitoring data for one year or up to 100 GB.

```
service_type: prometheus
service_name: prometheus
placement:
  count: 1
spec:
  retention_time: "1y"
  retention_size: "100GB"
```

Prometheus automatically deletes older data when either the time-based or size-based threshold is reached.

Prometheus now enforces the configured retention policy. If Prometheus was already deployed before applying the updated specification, redeployment ensures that the new settings take effect.

Expanding the cluster

You can use the dashboard to expand the Red Hat Ceph Storage cluster for adding hosts, adding OSDs, and creating services such as Alertmanager, `cephadm`-exporter, CephFS-mirror, Grafana, ingress, MDS, node-exporter, Prometheus, RBD-mirror, and Ceph Object Gateway.

Once you bootstrap a new storage cluster, the Ceph Monitor and Ceph Manager daemons are created and the cluster is in *HEALTH_WARN* state. After creating all the services for the cluster on the dashboard, the health of the cluster changes from *HEALTH_WARN* to *HEALTH_OK* status.

Prerequisites

- Bootstrapped storage cluster. For more information, see Bootstrapping a new storage cluster.
- At least `cluster-manager` role for the user on the Red Hat Ceph Storage Dashboard. For more information, see "User roles and permissions" on page 688.

Procedure

1. Copy the admin key from the bootstrapped host to other hosts:

   Syntax

   ```
   ssh-copy-id -f -i /etc/ceph/ceph.pub root@HOST_NAME
   ```

   Example

   ```
   [ceph: root@host01 /]# ssh-copy-id -f -i /etc/ceph/ceph.pub root@host02
   [ceph: root@host01 /]# ssh-copy-id -f -i /etc/ceph/ceph.pub root@host03
   ```

2. Log in to the dashboard with the default credentials provided during bootstrap.
3. Change the password and log in to the dashboard with the new password .
4. On the landing page, click *Expand Cluster*.
5. Add hosts:
   a. In the *Add Hosts* window, click *+Add*.
   b. Provide the hostname. This is same as the hostname that was provided while copying the key from the bootstrapped host.

      Note: You can use the tool tip in the *Add Hosts* dialog box for more details.

   c. Optional: Provide the respective IP address of the host.
   d. Optional: Select the labels for the hosts on which the services are going to be created.
   e. Click *Add Host*.
   f. Follow the above steps for all the hosts in the storage cluster.
6. In the *Add Hosts* window, click *Next*.
7. Create OSDs:
   a. In the *Create OSDs* window, for Primary devices, Click *+Add*.
   b. In the *Primary Devices* window, filter for the device and select the device.
   c. Click *Add*.
   d. Optional: In the *Create OSDs* window, if you have any shared devices such as WAL or DB devices, then add the devices.
   e. Optional: Click on the check-box *Encryption* to encrypt the features.
   f. In the *Create OSDs* window, click *Next*.
8. Create services:
   a. In the *Create Services* window, click *+Create*.
   b. In the *Create Service* dialog box,
      i. Select the type of the service from the drop-down.
      ii. Provide the service ID, a unique name of the service.
      iii. Provide the placement by hosts or label.

iv. Select the hosts.

v. Provide the number of daemons or services that need to be deployed.

c. Click *Create Service*.

9. In the *Create Service* window, Click *Next*.
10. Review the *Cluster Resources*, *Hosts by Services*, *Host Details*. If you want to edit any parameter, click *Back* and follow the above steps.
11. Click *Expand Cluster*.
12. You get a notification that the cluster expansion was successful.
13. The cluster health changes to *HEALTH_OK* status on the dashboard.

Verification

1. Log in to the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Run the `ceph -s` command.

   Example

   ```
   [ceph: root@host01 /]# ceph -s
   ```

   The health of the cluster is *HEALTH_OK*.

Upgrading a cluster

Upgrade Ceph clusters by using the dashboard.

Before you begin

Verify that your upgrade version path and operating system is supported before starting the upgrade process. For more information, see Compatibility matrix and Supported upgrade and crossgrade paths.
Before starting to upgrade Red Hat Ceph Storage, see Installation and upgrade requirements.

Before you begin, make sure that you have the following prerequisites in place:

> **Important:** These items cannot be done through the dashboard and must be completed manually, through the command-line interface, before continuing to upgrade the cluster from the dashboard.

> **Note:** For detailed information, see Upgrading cluster in a disconnected environment and complete steps 1 through 9.

1. The latest `cephadm`.

   ```
   dnf update cephadm
   ```

2. The latest `cephadm-ansible`.

   ```
   dnf update cephadm-ansible
   ```

3. The latest `cephadm pre-flight playbook`.

   ```
   ansible-playbook -i INVENTORY_FILE cephadm-preflight.yml --extra-vars
   "ceph_origin=custom upgrade_ceph_packages=true"
   ```

4. Run the following Ceph commands to avoid alerts and rebalancing of the data during the cluster upgrade:

```
ceph health mute DAEMON_OLD_VERSION --sticky
ceph osd set noout
ceph osd set noscrub
ceph osd set nodeep-scrub
```

About this task

Cluster images are pulled automatically from `registry.redhat.io`. Optionally, use custom images for upgrade.

Procedure

1. View if cluster upgrades are available and upgrade as needed from the **Administration › Upgrade** page on the dashboard.

   > **Note:** If the dashboard displays the `Not retrieving upgrades` message, check if the registries were added to the container configuration files with the appropriate log in credentials to Podman or docker.
   > Due to known issue BZ #2168121, the updates are not retrieved properly during the **`ceph orch upgrade ls`** command that is being used. To upgrade, use the **Upgrade using custom image** option. Get the latest custom image name from Custom image details for the monitoring stack in Configuring a private registry for a disconnected installation.

   Click **Pause** or **Stop** during the upgrade process, if needed. The upgrade progress is shown in the progress bar along with information messages during the upgrade.

   > **Note:** When stopping the upgrade, the upgrade is first paused and then prompts you to stop the upgrade.

2. Optional: View cluster logs during the upgrade process from the **Cluster logs** section of the **Upgrade** page.
3. Verify that the upgrade is completed successfully by confirming that the cluster status displays OK state.

What to do next

After verifying that the upgrade is complete, unset the `noout`, `noscrub`, and `nodeep-scrub` flags.

```
ceph osd unset noout
ceph osd unset noscrub
ceph osd unset nodeep-scrub
```

For example,

```
[ceph: root@host01 /]# ceph osd unset noout
[ceph: root@host01 /]# ceph osd unset noscrub
[ceph: root@host01 /]# ceph osd unset nodeep-scrub
```

Managing access and single sign-on with the dashboard

The Ceph Dashboard supports external authentication of users with the choice of either the Security Assertion Markup Language (SAML) 2.0 protocol or with the OAuth2 Proxy (`oauth2-proxy`). Before using single sign-on (*SSO*) with the Ceph Dashboard, create the dashboard user accounts and assign any required roles. The Ceph Dashboard completes user authorization and then the existing Identity Provider (*IdP*) completes the authentication process. This process applies only to SAML 2.0 SSO. You can enable single sign-on using the SAML protocol or `oauth2-proxy`.

Red Hat Ceph Storage supports dashboard SSO and Multi-Factor Authentication with RHSSO (Keycloak) and IBM Security Verify.

To connect your SAML IdP with the Ceph Dashboard, use the following Service Provider (SP) configuration details. In each of the following configuration URLs, replace *DASHBOARD_HOST* with the fully qualified domain name (FQDN) and port of your dashboard. For example,

```
ceph-dashboard.example.com:8443
```

Entity ID (Audience URI)

The unique identifier for the Red Hat Ceph Storage Dashboard service provider.
`https://DASHBOARD_HOST/auth/saml2/metadata`

Assertion Consumer Service (ACS)

The endpoint where the IdP sends the SAML assertion (using POST binding).
`https://DASHBOARD_HOST/auth/saml2/`

Default Relay State (Start URL)

The page that users are directed to after a successful login.
`https://DASHBOARD_HOST/#/dashboard`

Target URL

`https://DASHBOARD_HOST/#/dashboard`

> **Note:** Some IdPs can automatically configure these fields from the service provider metadata. To provide the metadata, provide the following URL to your IdP:
> `https://DASHBOARD_HOST/auth/saml2/metadata`

OAuth2 SSO uses the `oauth2-proxy` service to work with the Ceph Management gateway (`mgmt-gateway`), providing unified access and improved user experience. For more information about the Ceph Management gateway and the OAuth2 Proxy service, see Using the Ceph Management gateway (`mgmt-gateway`) and Using the OAuth2 Proxy (`oauth2-proxy`) service.

For more information about Red Hat build of Keycloak, see Red Hat build of Keycloak on the Red Hat Customer Portal.

For more information about IBM Security Verify, see IBM Security Verify documentation. For a free trial, see IBM Security Verify (SaaS).

Creating an admin account for syncing users to the Ceph dashboard

Create an admin account to synchronize users to the Ceph dashboard.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Admin-level access to the dashboard.
- Users are added to the dashboard.
- Root-level access on all the hosts.
- Java OpenJDK installed. For more information, see the Installing a JRE on RHEL by using yum section of the *Installing and using OpenJDK 8 for RHEL* guide for OpenJDK on Red Hat Documentation.
- Red Hat Single Sign-On installed from a ZIP file. For more information, see the Installing RH-SSO from a ZIP File section of the *Server Installation and Configuration Guide* for Red Hat Single Sign-On on Red Hat Documentation.

About this task

After creating the account, use Red Hat Single Sign-on (SSO) to synchronize users to the Ceph dashboard. See Syncing users to the Ceph dashboard using Red Hat Single Sign-On.

For more information, see the following:

- Creating roles.
- Creating users.

Procedure

1. Download the *Red Hat Single Sign-On 7.4.0 Server* on the system where Red Hat Ceph Storage is installed. For download package and information, see Software Details for *Red Hat Single Sign-On 7.4.0 Server* on the Red Hat Customer Portal.
2. Extract the folder.

   ```
   unzip rhsso-7.4.0.zip
   ```

   For example,

   ```
   [root@host01 ~]# unzip rhsso-7.4.0.zip
   ```

3. Go to the `standalone/configuration` directory and open the `standalone.xml` file for editing. For example,

   ```
   [root@host01 ~]# cd standalone/configuration
   [root@host01 configuration]# vi standalone.xml
   ```

4. From the `bin` directory of the newly created `rhsso-7.4.0` folder, run the `add-user-keycloak` script to add the initial administrator user.

   ```
   ./add-user-keycloak.sh -u admin
   ```

   For example,

   ```
   [root@host01 bin]# add-user-keycloak.sh -u admin
   ```

5. Replace the **`localhost`** and `127.0.0.1` with the IP address of the server where the Red Hat SSO is installed.
6. Start the server.
   From the `bin` directory of `rh-sso-7.4` folder, run the `standalone` boot script.

   ```
   ./standalone.sh
   ```

   For example,

   ```
   [root@host01 bin]# ./standalone.sh
   ```

7. Create the admin account in `https: IP_ADDRESS :8080/auth` with a username and password.

   > **Note:** You must create an admin account only the first time that you log into the console.

8. Log in to the admin console with the credentials created.

Syncing users from Red Hat Sign-On to the Ceph dashboard

You can use Red Hat Single Sign-on (SSO) with Lightweight Directory Access Protocol (LDAP) integration to synchronize users to the Red Hat Ceph Storage Dashboard.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Ceph Dashboard installed.
- Admin-level access to the dashboard.
- Users added to the dashboard. See “Creating users” on page 692.

- Root-level access on all the hosts.
- Admin account created for syncing users. See “Creating an admin account for syncing users to the Ceph dashboard” on page 678.

About this task

The users are added to specific realms in which they can access the dashboard through SSO without any additional requirements of a password.

Procedure

1. To create a realm, select **Master** from the drop-down list.
   In this realm, you can provide access to users and applications.
2. In the **Add Realm** window, enter a case-sensitive realm name, and set the parameter **Enabled** to0Nand click **Create**.
3. In the **Realm Settings** tab, set the parameters and click **Save**.
   Set the parameters as follows:
   - `Enabled -ON`
   - `User-Managed-ON`

   Note the link address of SAML 2.0 Identity Provider Metadata to paste in **Client Settings**.
4. In the **Clients** tab, click **Create**.
5. In the **Add Client** window, set the client ID parameter, and click **Save**.
   `BASE_URL:8443/auth/saml2/metadata`

   For example,

   ```
   https://example.ceph.redhat.com:8443/auth/saml2/metadata ..
   Client Protocol - saml
   ```

6. In the **Client** window, **Settings** tab, set the parameters and click **Save**.
   Use the parameters lthat are isted in “Client Settings tab” on page 680.

*Table 97: **Client Settings** tab*

Name of the parameter	Syntax	Example
`Client ID`	BASE_URL:8443/auth/saml2/metadata	https://example.ceph.redhat.com:8443/auth/saml2/metadata
`Enabled`	ON	ON
`Client Protocol`	saml	saml
`Include AuthnStatement`	ON	ON
`Sign Documents`	ON	ON
`Signature Algorithm`	RSA_SHA1	RSA_SHA1
`SAML Signature Key Name`	KEY_ID	KEY_ID
`Valid Redirect URLs`	BASE_URL:8443/*	https://example.ceph.redhat.com:8443/
`Base URL`	BASE_URL:8443	https://example.ceph.redhat.com:8443/

Name of the parameter	Syntax	Example
`Master SAML Processing URL`	https://localhost:8080/auth/realms/REALM_NAME/protocol/saml/descriptor	https://localhost:8080/auth/realms/Ceph_LDAP/protocol/saml/descriptor **Note:** Paste the link of SAML 2.0 Identity Provider Metadata from **Realm Settings** tab.

7. In **Fine Grain SAML Endpoint Configuration**, set the parameters and click **Save**.
Use the parameters that are listed in "Fine Grain SAML Endpoint configuration" on page 681.

*Table 98: **Fine Grain SAML Endpoint configuration***

Name of the parameter	Syntax	Example
`Assertion Consumer Service POST Binding URL`	BASE_URL: 8443/#/dashboard	https://example.ceph.redhat.com:8443/#/dashboard
`Assertion Consumer Service Redirect Binding URL`	BASE_URL: 8443/#/dashboard	https://example.ceph.redhat.com:8443/#/dashboard
`Logout Service Redirect Binding URL`	BASE_URL:8443/	https://example.ceph.redhat.com:8443/

8. From **Clients › Mappers** tab, set the parameters, and click **Save**.
Use the parameters that are listed in "Client Mappers tab" on page 681.

*Table 99: **Client Mappers** tab*

Name of the parameter	Value
`Protocol`	saml
`Name`	username
`Mapper Property`	User Property
`Property`	username
`SAML Attribute name`	username

9. In the **Client Scope** tab, select the **role_list**.
    a. In the **Mappers** tab, select the **role list**, set the **Single Role Attribute** to`ON`.
10. Select the **User_Federation** tab.
11. In the **User Federation** window, select **ldap** from the drop-down list.
12. From the **User Federation** window, **Settings** tab, set the parameters and click **Save**.
Use the parameters that are listed in "User Federation Settings tab" on page 681.

*Table 100: **User Federation Settings** tab*

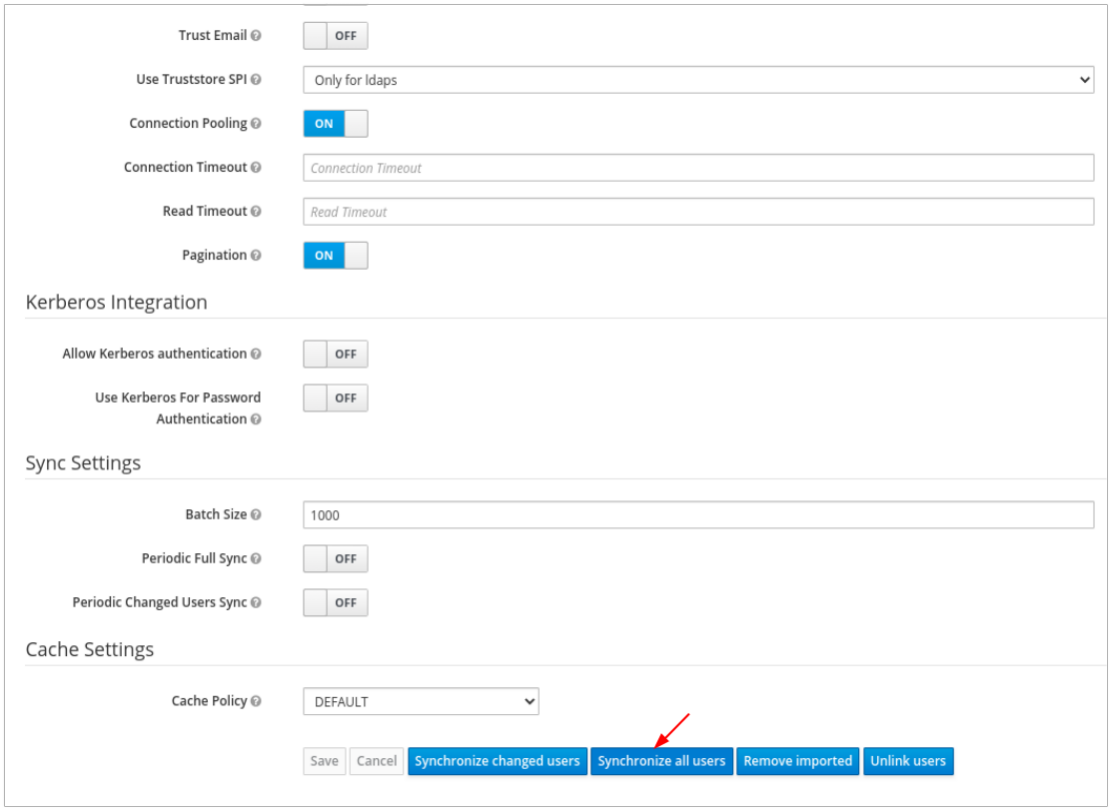
Name of the parameter	Value
`Console display name`	rh-ldap
`Import Users`	ON
`Edit_Mode`	READ_ONLY
`Username LDAP attribute`	UID
`RDN LDAP attribute`	UID

Name of the parameter	Value
`UUID LDAP attribute`	nsuniqueid
`User Object Classes`	inetOrgPerson, organizationalPerson, rhatPerson
`Connection URL`	Example: ldap://ldap.corp.redhat.com
`Users DN`	ou=users, dc=example, dc=com
`Bind Type`	None

13. Click **Test authentication**.
A notification displays with a message that the LDAP authentication is successful.
14. Click **Test Connection**.
A notification displays that the LDAP connection is successful.
15. In the **Mappers** tab, select the **first name** row, edit the **LDAP `Attribute`** parameter, and click **Save**.

```
- LDAP Attribute - uid
```

16. In the **User Federation Settings** tab, click **Synchronize all users**.



A notification displays with a message that the sync of users is finished successfully.

17. In the **Users** tab, search for the user added to the dashboard and click the Search icon.
To view the user, click the specific row. The federation link is displayed as the name provided for the **User Federation**.

> **Important:** LDAP does not synchronize users who are added manually. Click **Delete** if you have added the user manually.

> **Note:** If Red Hat SSO is used within your work environment, be sure to first enable SSO. For more information, see “Enabling single sign-on with SAML 2.0” on page 683.

What to do next

Users that are added to the realm and the dashboard can access the Ceph dashboard with their email address and password.
For example,

```
https://example.ceph.redhat.com:8443
```

Related information

Creating roles
Creating an admin account for syncing users to the Ceph dashboard

Enabling single sign-on with SAML 2.0

Enable single sign-on (SSO) for the Ceph Dashboard with SAML 2.0.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Dashboard.
- Root-level access to the Ceph Manager hosts.

Procedure

1. Configure SAML 2.0 SSO on the Ceph Dashboard.

   ```
   ceph dashboard sso setup
   saml2 CEPH_DASHBOARD_BASE_URL IDP_METADATA IDP_USERNAME_ATTRIBUTE IDP_ENTITY_ID SP_X_5
   09_CERT SP_PRIVATE_KEY
   ```

   Replace the following variables:

   - *CEPH_DASHBOARD_BASE_URL* with the base URL where Ceph Dashboard is accessible.
   - *IDP_METADATA* with the URL to remote or local path or content of the IdP metadata XML. The supported URL types are `http`, `https`, and `file`.
   - (Optional) *IDP_USERNAME_ATTRIBUTE* with the attribute used to get the username from the authentication response. Defaults to `uid`.
   - (Optional) *IDP_ENTITY_ID* with the IdP entity ID when more than one entity ID exists on the IdP metadata.
   - (Optional) *SP_X_509_CERT* with the file path of the certificate used by Ceph Dashboard for signing and encryption.
   - (Optional) *SP_PRIVATE_KEY* with the file path of the private key used by Ceph Dashboard for signing and encryption.

   For example,

   ```
   [ceph: root@host01 /]# ceph dashboard ceph dashboard sso setup saml2 https://
   dashboard_hostname.ceph.redhat.com:8443 idp-metadata.xml username https://
   10.70.59.125:8080/auth/realms/realm_name /home/certificate.txt /home/private-key.txt
   ```

2. Verify the SAML 2.0 configuration.

   ```
   cephadm shell ceph dashboard sso show saml2
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph dashboard ceph dashboard sso show saml2
   ```

3. Enable SSO.

   ```
   cephadm shell ceph dashboard sso enable saml2
   ```

For example,

```
[ceph: root@host01 /]# ceph dashboard ceph dashboard sso enable saml2
SSO is "enabled" with "SAML2" protocol.
```

What to do next

1. Open your dashboard URL.
   `https://dashboard_hostname.ceph.redhat.com:8443`
2. On the SSO page, enter the login credentials.
   The SSO redirects to the dashboard web interface.

Note: Check the SSO status at any time with the **`cephadm shell ceph dashboard sso status`** command. For example:

```
[ceph: root@host01 /]# ceph dashboard ceph dashboard sso status
SSO is "enabled" with "SAML2" protocol.
```

Configuring Red Hat SSO as the identity provider using OAuth2 Protocol

You can enable single sign-on (SSO) for the Red Hat Ceph Storage Dashboard by configuring Red Hat SSO as the Identity Provider (IdP).

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Dashboard.
  - Enable the Ceph Management gateway (`mgmt-gateway`) service. For more information, see Enabling the Ceph Management gateway.
  - Enable the OAuth2 Proxy service (`oauth2-proxy`). For more information, see Enabling the OAuth2 Proxy service.

  Ensure that oauth2 proxy and identity provider are configured before you can configure the Identity Provider.

Procedure

1. Create a realm in Red Hat SSO.
   a. Click the **Master** drop-down menu and select **Add Realm**.
   b. Enter a case-sensitive name for the realm.
   c. Set **Enabled** to **ON** and click **Create**.
2. Configure realm settings.
   a. Go to the **Realm Settings** tab.
   b. Set **Enabled** to **ON** and **User-Managed Access** to **ON**.
3. Click **Save**.
4. Copy the **OpenID Endpoint Configuration** URL. Use the issuer endpoint from this configuration for OAuth2 Proxy setup.
   For example,

   ```
   Issuer: http://<your-ip>:<port>/realms/<realm-name>
   ```
5. Create and configure the OIDC client.
   a. Create a client of type **OpenID Connect**.
   b. Enable client authentication and authorization.

c. Set redirect URLs.
   - **Valid redirect URL**: `https://<host_name|IP_address>/oauth2/callback`
   - **Valid logout redirect URL**: `https://<host_name|IP_address>/oauth2/sign_out`

d. Go to the **Credentials** tab and note the values for client_id and client_secret.
These credentials are required to configure the oauth2-proxy service.

6. Create a role for dashboard access.
   a. Navigate to the **Roles** tab.
   b. Create a role and assign it to the OIDC client.
   c. Use a role name supported by the Red Hat Ceph Storage dashboard.
7. Create a user and assign roles.
   a. Create a new user.
   b. Define user credentials.
   c. Open the **Role Mappings** tab and assign the previously created role to the user.
8. Enable SSO for the dashboard.

```
ceph dashboard sso enable
```

SSO is now enabled for the Ceph management stack. When users access the dashboard, they are redirected to the Red Hat SSO login page. After successful authentication, users are granted access across all monitoring stacks, including dashboard, Grafana, and Prometheus, based on the roles assigned in Red Hat SSO.

Configuring IBM Security Verify as the identity provider using OAuth2 protocol

You can enable single sign-on (SSO) for the Red Hat Ceph Storage Dashboard by configuring IBM Security Verify as the Identity Provider (*IdP*).

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Dashboard.
  - Enable the Ceph Management gateway (`mgmt-gateway`) service. For more information, see Enabling the Ceph Management gateway.
  - Enable the OAuth2 Proxy service (`oauth2-proxy`). For more information, see Enabling the OAuth2 Proxy service.

Procedure

1. From the IBM Security Verify application , navigate to the **Applications › Application** and click **Add An Application**.
2. Under Application Type , select **Custom Application** and click **Add Application**.
3. Add the company name under the **General** tab.
4. Navigate to **Sign-on** tab and select **Open ID Connect 1.0** from the **Sign-on Method** drop-down list.
5. Enter the mgmt-gateway URL under the **Application URL** section.
For example,
`https://<HOSTNAME/IP_OF_THE_MGMT-GATEWAY_HOST>`
6. Clear the **PKCE verification** checkbox.
7. From the **Redirect URLs** section, fill in the redirect the URLs.
Use the following redirect URL format.
   - `https://<HOSTNAME/IP_OF_THE_MGMT-GATEWAY_HOST>/oauth2/callback`

- `https://<HOSTNAME/IP_OF_THE_MGMT-GATEWAY_HOST>/oauth2/sign_out`

8. From the **Token settings**, select **JWT** as the access token format.
   For each of the following attributes, enter the details in the respective fields and then select the corresponding attribute source from the drop-down list:
   - `name | preferred_username`
   - `email | email`
9. From the **Consent Settings**, select **Do not ask for consent** from the drop-down list.
10. From the **Custom Scopes**, clear the **Restrict Custom Scopes** checkbox and click **Save**.
11. Add a user.
    a. Navigate to **Directory › Users & Groups** and click **Add User**.
    b. Provide the desired username and a valid email address.
       The initial user credentials are sent to the email address provided.
    c. Click **Save**.
12. Navigate to the **Directory › Attributes** and click **Add Attribute**.
    a. Select **Custom Attribute**, and then select the **Single Sign On (SSO)** checkbox.
    b. Click **Next**.
    c. In the **Attribute Name**field, enter `roles`.
    d. Under **Data Type**, select **Multivalue String** from the drop-down list.
    e. Click **Additional settings**, select **Lowercase** from the **Apply Transformation** drop-down list and click **Next**.
    f. Click **Add Attribute**.
13. Navigate to **Directory › Users & Groups** and edit the newly added user.
    a. Select **View Extended Profile**.
    b. From **Custom User Attributes**, add your desired dashboard role.
       For example, administrator, read-only, block-manager, and so on. This user inherit the dashboard roles automatically.
    c. Click **Save settings**.
14. Go to **Applications › Application** and click the **Settings** icon for the custom application created in step "2" on page 685.
    a. From the **Sign-on** tab, go to **Token settings**.
    b. For the roles attribute, enter the detail in the respective fields and then select the corresponding attribute source from the drop-down list:
       - `roles | roles`
    c. From the **Entitlements** tab add the newly created user and click **Save**.

Enabling OAuth2 single sign-on

Enable OAuth2 single sign-on (SSO) for the Ceph Dashboard. OAuth2 SSO uses the `oauth2-proxy` service.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Dashboard.
- Root-level access to the Ceph Manager hosts.
- You should either have an admin account with Red Hat Single-Sign-On 7.6.0 with OAuth2 protocol running or IBM Security Verify with OAuth2 protocol running. For more information, see "Configuring IBM Security Verify as the identity provider using OAuth2 protocol" on page 683.

- Enable the Ceph Management gateway (`mgmt-gateway`) service. For more information, see Enabling the Ceph Management gateway.
- Enable the OAuth2 Proxy service (`oauth2-proxy`). For more information, see Enabling the OAuth2 Proxy service.

Procedure

1. Enable Ceph Dashboard OAuth2 SSO access.

   ```
   ceph dashboard sso enable oauth2
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph dashboard sso enable oauth2
   SSO is "enabled" with "oauth2" protocol.
   ```

2. Set the valid redirect URL.
   `https://HOST_NAME|IP_ADDRESS/oauth2/callback`

   > **Note:** This URL must be the same redirect URL as configured in the OAuth2 Proxy service.

3. Configure a valid user role.

   > **Note:** For the Administrator role, configure the IdP user with an `administrator` or `read-only` access.

What to do next

1. Open your dashboard URL.
   `<https://<hostname/ip> address of mgmt-gateway-host>:<mgmt-gatway-port>`
2. On the SSO page, enter the Identity Provider (IdP) credentials.
   The SSO redirects to the dashboard web interface.

> **Note:** Check the SSO status at any time with the **`cephadm shell ceph dashboard sso status`** command. For example:
>
> ```
> [root@host01 ~]# cephadm shell ceph dashboard sso status
> SSO is "enabled" with "oauth2" protocol.
> ```

Disabling single sign-on for the Ceph Dashboard

You can disable single sign-on for Ceph Dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Dashboard.
- Root-level access to the Ceph Manager hosts.
- Single sign-on enabled for Ceph Dashboard.

About this task

You can disable SAML 2.0 and OAuth2 SSO for the Ceph Dashboard at any time.

Procedure

1. Optional: View the status of the SSO.

```
cephadm shell ceph dashboard sso status
```

For example,

```
[ceph: root@host01 /]# ceph dashboard ceph dashboard sso status
SSO is "enabled" with "SAML2" protocol.
```

2. Disable SSO.

```
cephadm shell ceph dashboard sso disable
```

For example,

```
[ceph: root@host01 /]# ceph dashboard ceph dashboard sso disable
SSO is "disabled".
```

Managing roles

As a storage administrator, you can create, edit, clone, and delete roles on the dashboard.

By default, there are eight system roles. You can create custom roles and give permissions to those roles. These roles can be assigned to users based on the requirements.

User roles and permissions

User accounts are associated with a set of roles that define the specific dashboard functionality which can be accessed. View user roles and permissions by going to **Dashboard settings › User management**.

The Red Hat Ceph Storage dashboard functionality or modules are grouped within a security scope. Security scopes are predefined and static. The current available security scopes on the Red Hat Ceph Storage dashboard are:

- `cephfs` - Includes all features related to CephFS management.
- `config-opt` - Includes all features related to management of Ceph configuration options.
- `dashboard-settings` - Allows to edit the dashboard settings.
- `grafana` - Include all features related to Grafana proxy.
- `hosts` - Includes all features related to the Hosts menu entry.
- `log` - Includes all features related to Ceph logs management.
- `manager` - Includes all features related to Ceph manager management.
- `monitor` - Includes all features related to Ceph monitor management.
- `osd` - Includes all features related to OSD management.
- `pool` - Includes all features related to pool management.
- `prometheus` - Include all features related to Prometheus alert management.
- `rbd-image` - Includes all features related to RBD image management.
- `rbd-mirroring` - Includes all features related to RBD mirroring management.
- `rgw` - Includes all features related to Ceph Object Gateway (RGW) management.

A role specifies a set of mappings between a security scope and a set of permissions. There are four types of `permissions`:

- `Read`
- `Create`

- `Update`
- `Delete`

The list of `system roles` are:

- `administrator` - Allows full permissions for all security scopes.
- `block-manager` - Allows full permissions for RBD-image and RBD-mirroring scopes.
- `cephfs-manager` - Allows full permissions for the Ceph File System scope.
- `cluster-manager` - Allows full permissions for the hosts, OSDs, monitor, manager, and config-opt scopes.
- `pool-manager` - Allows full permissions for the pool scope.
- `read-only` - Allows read permission for all security scopes except the dashboard settings and config-opt scopes.
- `rgw-manager` - Allows full permissions for the Ceph Object Gateway scope.

> **Note:** you need to provide `rgw-manager` access to the users for all Ceph Object Gateway operations.

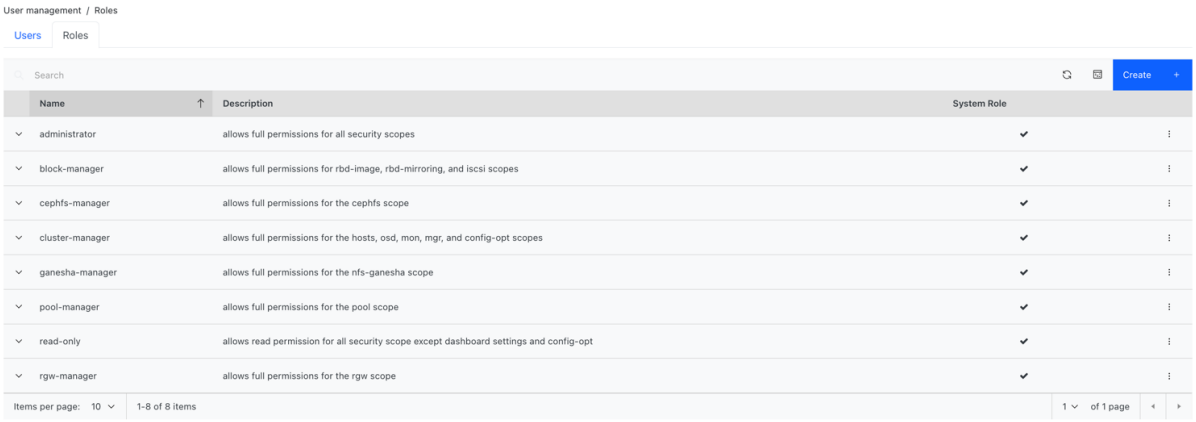


Figure 59: Displaying the user roles

For more information, see the following:

- Creating Ceph Object Gateway
- Creating roles on the Ceph dashboard

Creating roles

You can create custom roles on the dashboard and these roles can be assigned to users based on their roles.

Prerequisites

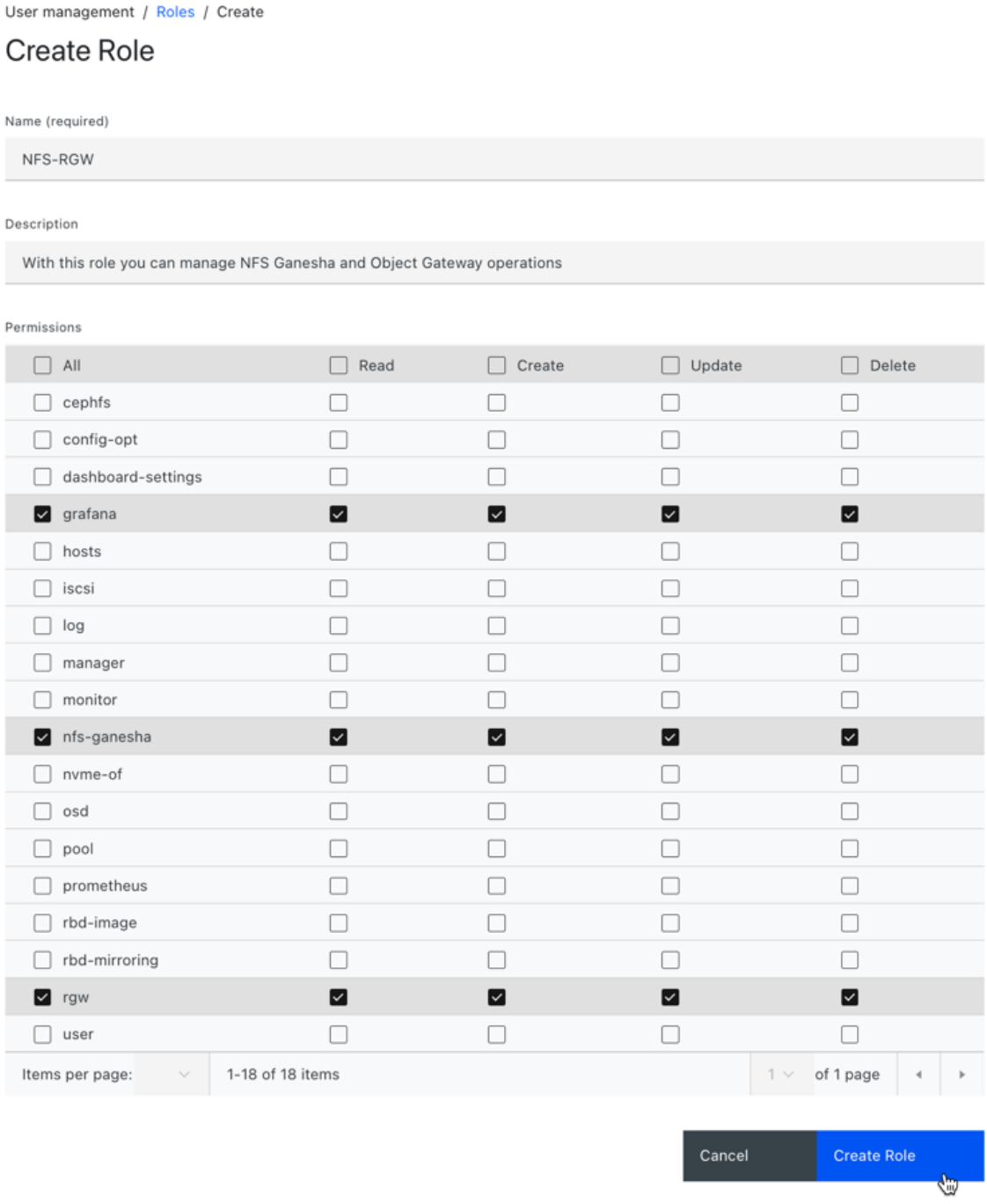
Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Admin-level of access to the Dashboard.

Procedure

1. Log in to the Dashboard.
2. Click the **Dashboard Settings** icon and then click **User management**.
3. On **Roles** tab, click **Create**.

4. In the **Create Role** form, enter the **Name**, **Description**, and select the **Permissions** for this role, and then click **Create Role**.



A notification displays that the role was created successfully.

5. Click on the **Expand/Collapse** icon of the row to view the details and permissions given to the roles.

Reference

For more information, see the following:

- "User roles and permissions" on page 688
- "Creating users" on page 692

Editing roles

The dashboard allows you to edit roles on the dashboard.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Admin-level of access to the Dashboard.
- A role is created on the dashboard.

Procedure

1. Log in to the Dashboard.
2. Click the **Dashboard Settings** icon and then click **User management**.
3. On **Roles** tab, select the role to edit and click **Edit**.
4. In the **Edit Role** form, edit the parameters, and then click **Edit Role**.

   A notification displays that the role was updated successfully.

Cloning roles

When you want to assign additional permissions to existing roles, you can clone the system roles and edit it on the Red Hat Ceph Storage Dashboard.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Admin-level of access to the dashboard.
- Roles are created on the dashboard.

Procedure

1. From the dashboard menu, click the **Dashboard Settings** icon and then click **User management**.
2. On **Roles** tab, click the role you want to clone.
3. Select **Clone** from action drop-down.
4. In the **Clone Role** dialog box, enter the details for the role, and then click **Clone Role**.

   Once you clone the role, you can customize the permissions as per the requirements.

Reference

For more information, see the following:

- Creating roles.

Deleting roles

You can delete the custom roles that you have created on the Red Hat Ceph Storage dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Admin-level of access to the dashboard.
- A custom role is created on the dashboard.

About this task

> **Note:** You cannot delete the system roles of the Ceph Dashboard.

Procedure

1. Log in to the Dashboard.
2. Click the **Dashboard Settings** icon and then select **User management**.
3. On the **Roles** tab, click the role you want to delete and select **Delete** from the action drop-down.
4. In the **Delete Role** notification, select **Yes, I am sure** and then click **Delete Role**.

Related information

Creating roles

Managing users

As a storage administrator, you can create, edit, and delete users with specific roles on the Red Hat Ceph Storage Dashboard. Role-based access control is given to each user based on their roles and requirements.

You can also create, edit, import, export, and delete Ceph client authentication keys on the dashboard. Once you create the authentication keys, you can rotate keys using command-line interface (CLI). Key rotation meets the current industry and security compliance requirements.

Creating users

You can create users on the Red Hat Ceph Storage Dashboard with adequate roles and permissions based on their roles. For example, if you want the user to manage Ceph Object Gateway operations, then you can give `rgw-manager` role to the user.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Admin-level of access to the Dashboard.

> **Note:** The Red Hat Ceph Storage Dashboard does not support any email verification when changing a users password. This behavior is intentional, because the Dashboard supports Single Sign-On (SSO) and this feature can be delegated to the SSO provider.

Procedure

1. Log in to the Dashboard.
2. Click the Dashboard Settings icon and then click **User management**.
3. On **Users** tab, click **Create**.
4. In the **Create User** window, set the **Username** and other parameters including the roles, and then click **Create User**.

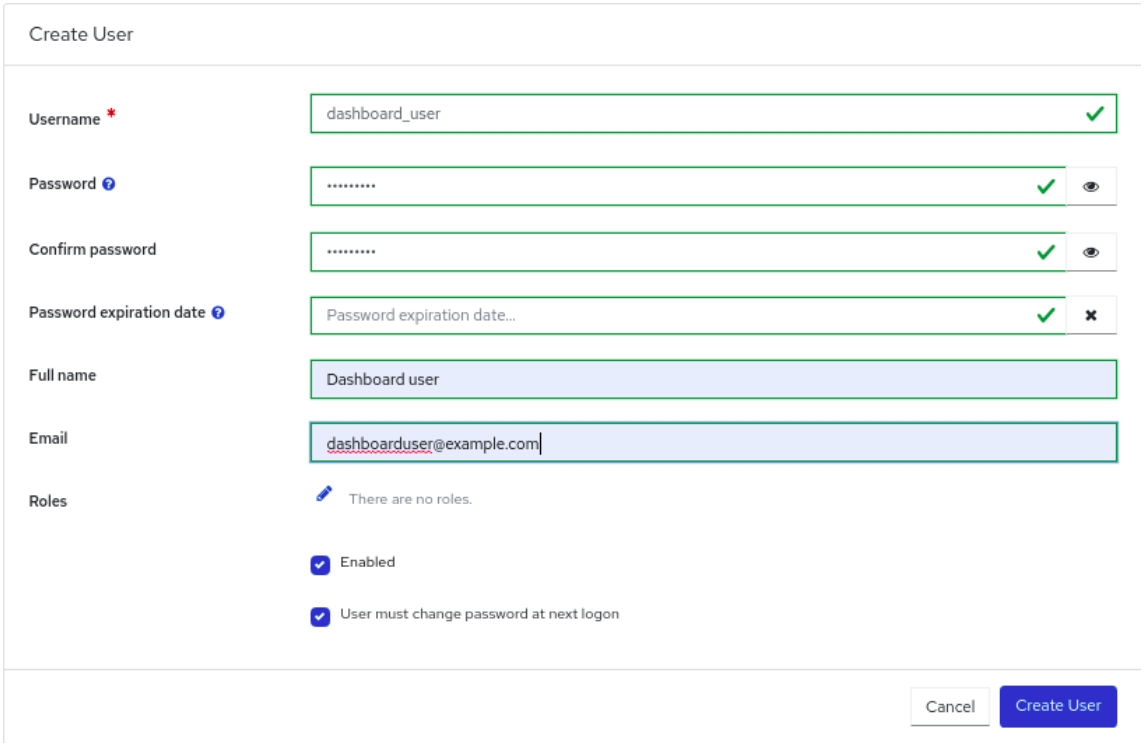


Figure 60: Create User

You get a notification that the user was created successfully.

Reference

For more information, see the following:

- Creating roles.
- User roles and permissions.

Editing users

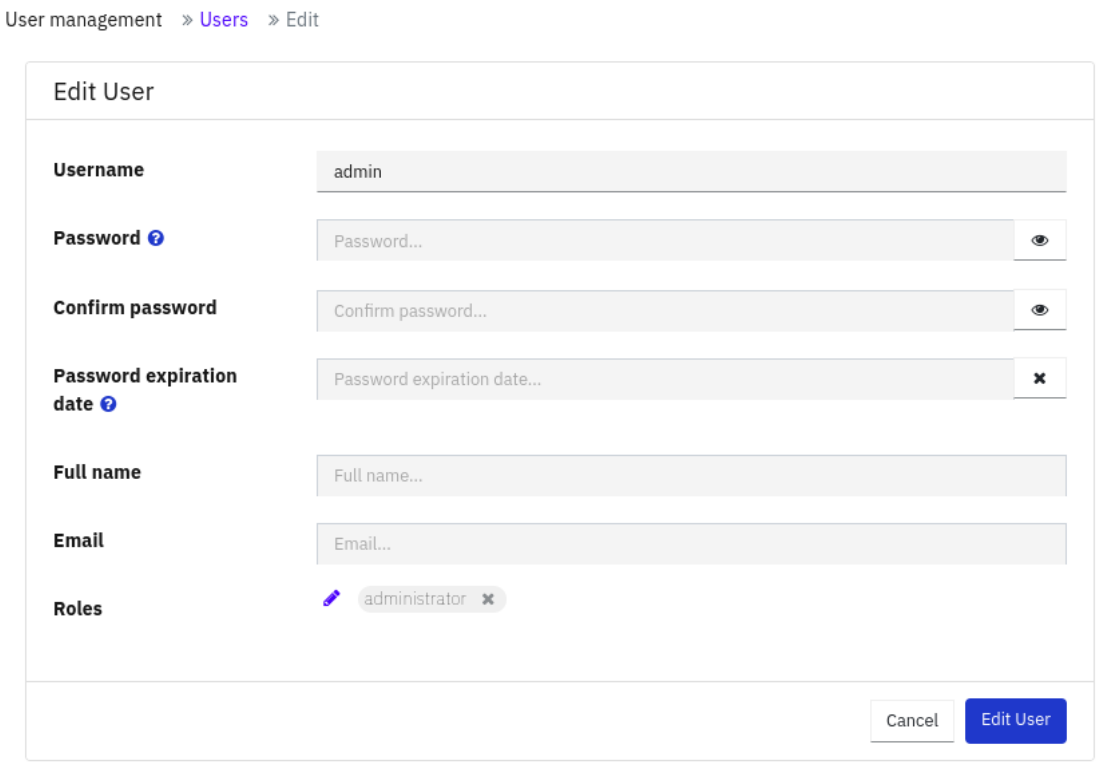
You can edit the users on the Red Hat Ceph Storage dashboard. You can modify the user's password and roles based on the requirements.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Admin-level of access to the Dashboard.
- User created on the dashboard.

Procedure

1. Log in to the Dashboard.
2. Click the **Dashboard Settings** icon and then click **User management**.
3. To edit the user, from the **Users** tab, select the user row and click **Edit**.
4. In the **Edit User** form, edit parameters like password and roles, and then click **Edit User**.



Note: If you want to disable any user's access to the Ceph dashboard, you can clear the **Enabled** option.

A notification displays that the user was created successfully.

Deleting users

You can delete users on the Ceph dashboard. Some users might be removed from the system. The access to such users can be deleted from the Ceph dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Admin-level of access to the Dashboard.
- User created on the dashboard.

Procedure

1. Log in to the Dashboard.
2. Click the **Dashboard Settings** icon and then select **User management**.
3. On the **Users** tab, click the user you want to delete and select **Delete** from the action drop-down.
4. In the **Delete User** dialog, select **Yes, I am sure** and then click **Delete User**.
   In the **Delete User** dialog, select **Yes, I am sure** and then click **Delete User**.

Related information

Creating users

Managing user capabilities

Ceph stores data RADOS objects within pools irrespective of the Ceph client used. Ceph users must have access to a given pool to read and write data, and must have executable permissions to use Ceph administrative commands. Creating users allows you to control their access to your Red Hat Ceph Storage cluster, its pools, and the data within the pools.

Ceph has a concept of `type` of user which is always `client`. You need to define the user with the `TYPE.ID` where ID is the user ID, for example, `client.admin`. This user typing is because the Cephx protocol is used not only by clients but also non-clients, such as Ceph Monitors, OSDs, and Metadata Servers. Distinguishing the user type helps to distinguish between client users and other users. This distinction streamlines access control, user monitoring, and traceability.

Capabilities

Ceph uses capabilities (caps) to describe the permissions granted to an authenticated user to exercise the functionality of the monitors, OSDs, and metadata servers. The capabilities restrict access to data within a pool, a namespace within a pool, or a set of pools based on their applications tags. A Ceph administrative user specifies the capabilities of a user when creating or updating the user.

You can set the capabilities to monitors, managers, OSDs, and metadata servers.

- The Ceph Monitor capabilities include `r`, `w`, and `x` access settings. These can be applied in aggregate from pre-defined profiles with `profile NAME`.
- The OSD capabilities include `r`, `w`, `x`, `class-read`, and `class-write` access settings. These can be applied in aggregate from pre-defined profiles with `profile NAME`.
- The Ceph Manager capabilities include `r`, `w`, and `x` access settings. These can be applied in aggregate from pre-defined profiles with `profile NAME`.
- For administrators, the metadata server (MDS) capabilities include `allow *`.

> **Note:** The Ceph Object Gateway daemon (`radosgw`) is a client of the Red Hat Ceph Storage cluster and is not represented as a Ceph storage cluster daemon type.

Reference

- Access capabilities

Accessing capabilities

Understand the different access or entity capabilities that can be given to a Ceph user or a Ceph client such as Block Device, Object Storage, File System, and native API.

Additionally, you can describe the capability profiles while assigning roles to clients.

`allow`

Description

Precedes access settings for a daemon. Implies `rw` for MDS only

`r`

Description

Gives the user *read* access. Required with monitors to retrieve the CRUSH map.

`w`

Description

Gives the user *write* access to objects.

`x`

Description

Gives the user the capability to call class methods, that is, both *read* and *write*, and to conduct `auth` operations on monitors.

`class-read`

Description

Gives the user the capability to call class read methods. Subset of `x`.

`class-write`

Description

Gives the user the capability to call class write methods. Subset of `x`.

***, `all`**

Description

Gives the user *read*, *write*, and *execute* permissions for a particular daemon or a pool, as well as the ability to run admin commands.

The following entries describe valid capability profile:

`profile osd`

Description

This is applicable to Ceph Monitor only. Gives a user permissions to connect as an OSD to other OSDs or monitors. Conferred on OSDs to enable OSDs to handle replication heartbeat traffic and status reporting.

`profile mds`

Description

This is applicable to Ceph Monitor only. Gives a user permissions to connect as an MDS to other MDSs or monitors.

`profile bootstrap-osd`

Description

This is applicable to Ceph Monitor only. Gives a user permissions to bootstrap an OSD. Conferred on deployment tools, such as `ceph-volume` and `cephadm`, so that they have permissions to add keys when bootstrapping an OSD.

`profile bootstrap-mds`

Description

This is applicable to Ceph Monitor only. Gives a user permissions to bootstrap a metadata server. Conferred on deployment tools, such as `cephadm`, so that they have permissions to add keys when bootstrapping a metadata server.

`profile bootstrap-rbd`

Description

This is applicable to Ceph Monitor only. Gives a user permissions to bootstrap an RBD user. Conferred on deployment tools, such as `cephadm`, so that they have permissions to add keys when bootstrapping an RBD user.

`profile bootstrap-rbd-mirror`

Description

This is applicable to Ceph Monitor only. Gives a user permissions to bootstrap an `rbd-mirror` daemon user. Conferred on deployment tools, such as `cephadm`, so that they have permissions to add keys when bootstrapping an `rbd-mirror` daemon.

`profile rbd`

Description

This is applicable to Ceph Monitor, Ceph Manager, and Ceph OSDs. Gives a user permissions to manipulate RBD images. When used as a Monitor cap, it provides the user with the minimal privileges required by an

RBD client application; such privileges include the ability to blocklist other client users. When used as an OSD cap, it provides an RBD client application with read-write access to the specified pool. The Manager cap supports optional `pool` and `namespace` keyword arguments.

`profile rbd-mirror`

Description

This is applicable to Ceph Monitor only. Gives a user permissions to manipulate RBD images and retrieve RBD mirroring config-key secrets. It provides the minimal privileges required for the user to manipulate the `rbd-mirror` daemon.

`profile rbd-read-only`

Description

This is applicable to Ceph Monitor and Ceph OSDS. Gives a user read-only permissions to RBD images. The Manager cap supports optional `pool` and `namespace` keyword arguments.

`profile simple-rados-client`

Description

This is applicable to Ceph Monitor only. Gives a user read-only permissions for monitor, OSD, and PG data. Intended for use by direct librados client applications.

`profile simple-rados-client-with-blocklist`

Description

This is applicable to Ceph Monitor only. Gives a user read-only permissions for monitor, OSD, and PG data. Intended for use by direct librados client applications. Also includes permissions to add blocklist entries to build high-availability (HA) applications.

`profile fs-client`

Description

This is applicable to Ceph Monitor only. Gives a user read-only permissions for monitor, OSD, PG, and MDS data. Intended for CephFS clients.

`profile role-definer`

Description

This is applicable to Ceph Monitor and Auth. Gives user **all** permissions for the auth subsystem, read-only access to monitors, and nothing else. Useful for automation tools. WARNING: Do not assign this unless you really, know what you are doing, as the security ramifications are substantial and pervasive.

`profile crash`

Description

This is applicable to Ceph Monitor and Ceph Manager. Gives a user read-only access to monitors. Used in conjunction with the manager crash module to upload daemon `crash` dumps into monitor storage for later analysis.

Reference

- User capabilities

Creating user capabilities

Create role-based access users with different capabilities on the Ceph dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.

– Admin-level access to the dashboard.

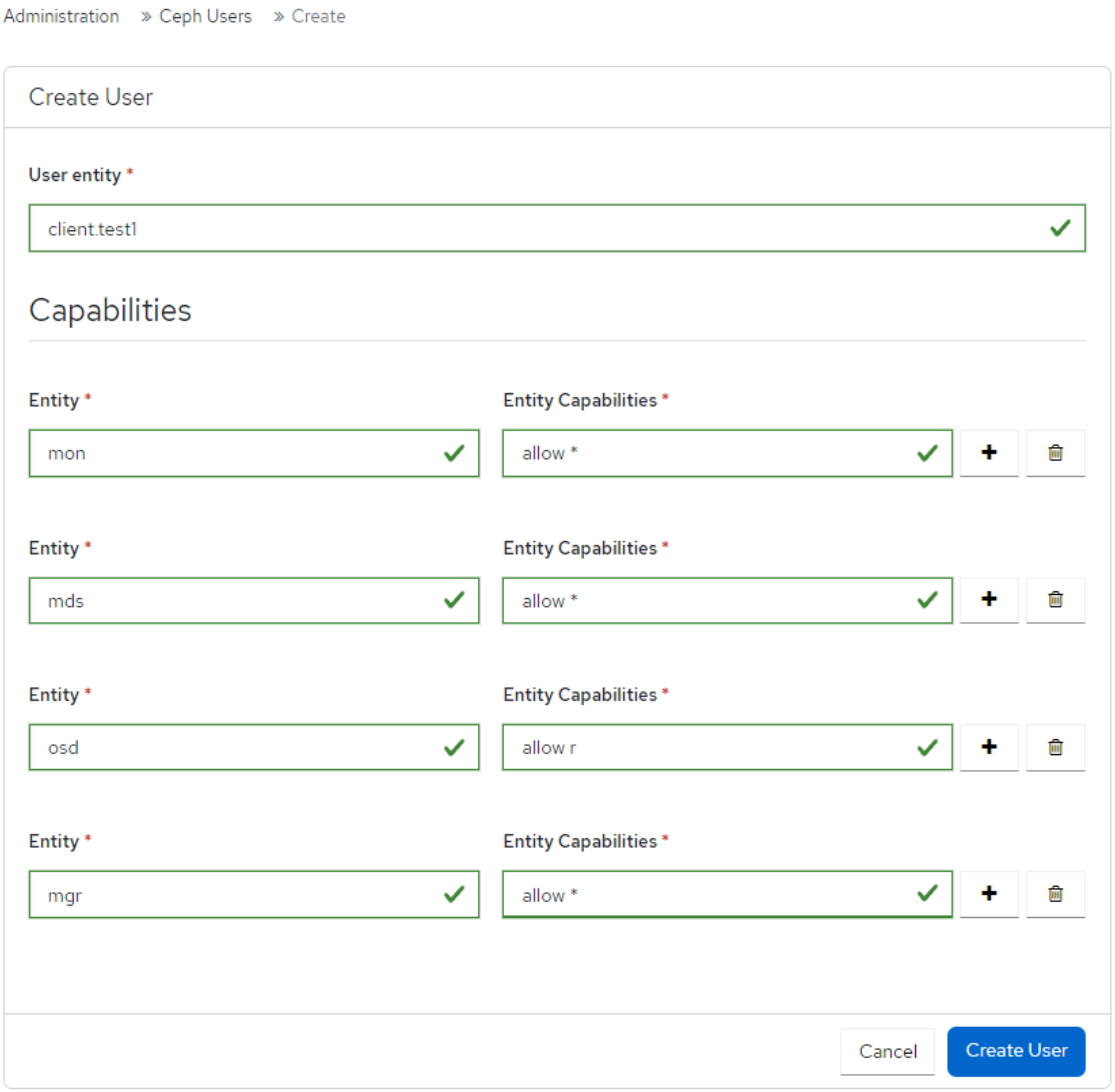
About this task

For details on different user capabilities, see User capabilities and Access capabilities.

Procedure

1. From the dashboard navigation, go to **Administration › Ceph Users**.
2. Click **Create**.
3. Fill in the **Create User** form.
   a. **User Entity**: Enter as *TYPE*.*ID*
   b. **Entity**: This can be `mon`, `mgr`, `osd`, or `mds`.
   c. **Entity Capabilities**: Enter the capabilities that you can to provide to the user. For example, `allow *` and `profile crash` are some of the capabilities that can be assigned to the client.

   > **Note:** You can add more entities to the user, based on the requirement.



4. Click **Create User**.
   A notification displays that the user is created successfully.

Editing user capabilities

Edit the roles of users or clients on the dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Admin-level access to the dashboard.

About this task

For details on different user capabilities, see User capabilities and Access capabilities.

Procedure

1. From the dashboard navigation, go to **Administration › Ceph Users**.
2. Click **Edit**.
3. Fill in the **Edit User** form.
   a. Edit the `entity` and `entity capabilities`.

   Note: You can add more entities to the user based on the requirement.

4. Click **Edit User**.
   A notification displays that the user is edited successfully.

Importing user capabilities

Import the roles of users or clients from the local host to the client on the dashboard.

For details on different user capabilities, see User capabilities and Access capabilities.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Admin-level access to the dashboard.

Procedure

1. Create a keyring file on the local host.

   For example,

   ```
   [localhost:~]$ cat import.keyring

   [client.test11]
           key = AQD9S29kmjgJFxAAkvhFar6Af3AWKDY2DsULRg==
           caps mds = "allow *"
           caps mgr = "allow *"
           caps mon = "allow *"
           caps osd = "allow r"
   ```

2. Log in to the Dashboard.
3. Go to **Administration › Ceph Users**.
4. Select the user whose roles you want to import and click **Import** from the action drop-down.
5. In the **Import User** window, click **Choose File** and select the appropriate file.

6. Click **Import User** .

   A notification displays that the keys are imported successfully.

Exporting user capabilities

Export the roles of the users or clients from the dashboard to a the local host.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Admin-level access to the dashboard.

About this task

For details on different user capabilities, see User capabilities and Access capabilities.

Procedure

1. From the dashboard navigation, go to **Administration › Ceph Users**.
2. Select the user whose roles you want to export.
3. Select **Export** from the action drop-down.
4. From the **Ceph user export data** dialog, click **Copy to Clipboard**.
   A notification displays that the keys are successfully copied.
5. Click **Close**.
6. On your local system, create a keyring file and paste the keys.

```
[localhost:~]$ cat exported.keyring

[client.test11]
        key = AQD9S29kmjgJFxAAkvhFar6Af3AWKDY2DsULRg==
        caps mds = "allow *"
        caps mgr = "allow *"
        caps mon = "allow *"
        caps osd = "allow r"
```

Deleting user capabilities

Delete the roles of users or clients on the dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Admin-level access to the dashboard.

Procedure

1. From the dashboard navigation, go to **Administration › Ceph Users**.
2. Select the user that you want to delete and select **Delete** from the action drop-down.
3. In the **Delete user** dialog, select **Yes, I am sure** and click **Delete user**.
   A notification displays that the user is deleted successfully.

Managing Ceph services

As a storage administrator, you can manage Ceph daemons on the Red Hat Ceph Storage Dashboard.

> **Note:** In the Ceph Dashboard, daemon processes are referred to as services. While this documentation may reference daemons when describing underlying behavior, all management actions in the dashboard are performed on services.

Running Ceph services

The Red Hat Ceph Storage dashboard allows you to start, stop, restart, and redeploy daemons.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- The dashboard is installed.
- At least one daemon is configured in the storage cluster.

About this task

> **Note:** In the Ceph Dashboard, Ceph daemon processes are managed as services. While the UI and this documentation may refer to daemons to reflect underlying Ceph components, all management actions in the dashboard are performed at the service level

About this task

> **Note:** Daemon actions are supported on all daemons except monitor and manager daemons.

Managing daemons can be done in the following ways:

- “Managing daemons from Services” on page 701
- “Managing daemons from Hosts” on page 702

Managing daemons from Services

Procedure

1. From the dashboard navigation, go to **Administration › Services**.
2. Expand the service with the daemon that will have the action run on.

   > **Note:** The row can be collapsed at any time.

3. On the **Daemons** tab, select the row with the daemon.

   > **Note:** The **Daemons** table can be searched and filtered.

4. Select the action that needs to be run on the daemon. The options are **Start**, **Stop**, **Restart**, and **Redeploy**.

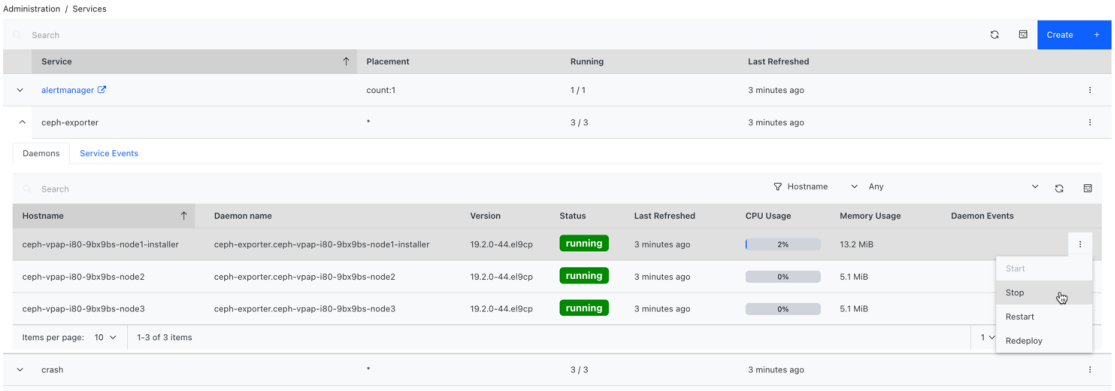


*Figure 61: Managing daemons from **Services***

Managing daemons from Hosts

Procedure

1. From the dashboard navigation, go to **Cluster › Host**.
2. On the **Hosts List** tab, expand the host row and select the host with the daemon to perform the action on.
3. On the **Daemons** tab of the host, select the row with the daemon.

> **Note:** The **Daemons** table can be searched and filtered.

4. Select the action that needs to be run on the daemon. The options are **Start**, **Stop**, **Restart**, and **Redeploy**.

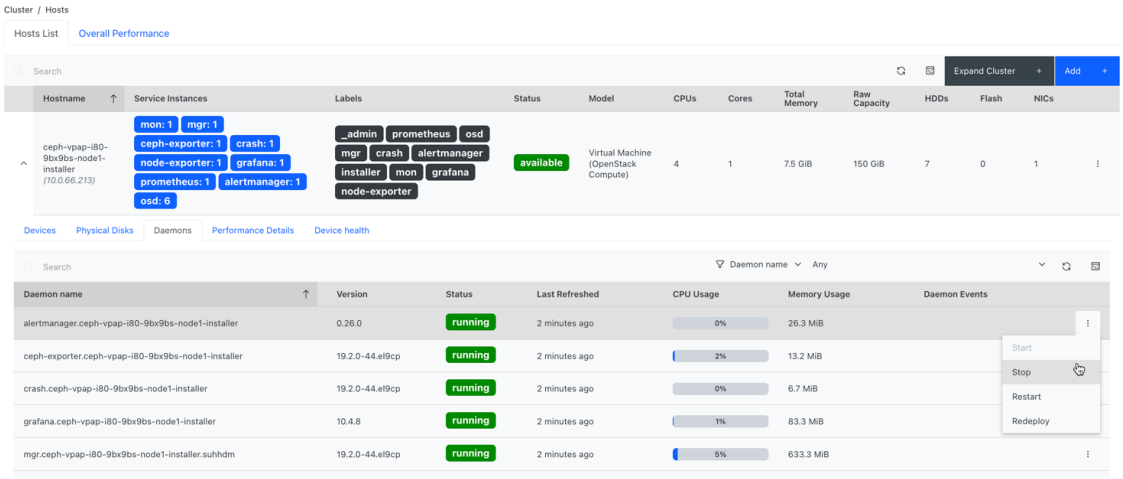


*Figure 62: Managing daemons from **Hosts***

Managing hosts

As a storage administrator, you can enable or disable maintenance mode for a host in the Red Hat Ceph Storage Dashboard. The maintenance mode ensures that shutting down the host, to perform maintenance activities, does not harm the cluster.

In addition to removing hosts, you can also remove hosts using *Start Drain* and *Remove* options in the Red Hat Ceph Storage Dashboard.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.

- Hosts, Ceph Monitors, and Ceph Manager Daemons are added to the storage cluster.

Entering maintenance mode

If the maintenance mode gets enabled successfully, the host is taken offline without any errors for the maintenance activity to be performed. If the maintenance mode fails, it indicates the reasons for failure and the actions you need to take before taking the host down.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- All other prerequisite checks are performed internally by Ceph and any probable errors are taken care of internally by Ceph.

Procedure

1. From the dashboard navigation, go to **Cluster › Hosts**.
2. From the **Hosts List**, select the host to put in maintenance.
3. Click **Enter Maintenance** from the action drop-down.

   > **Note:** When a host enters maintenance, all daemons are stopped. You can check the status of the daemons under the **Daemons** tab of a host.

   A notification is displayed that the host has entered maintenance successfully and the maintenance label is seen in the **Status** column.

   > **Note:** If the maintenance mode fails, you get a notification indicating the reasons for failure.

Exiting maintenance mode

To restart a host, you can move it out of maintenance mode on the Red Hat Ceph Storage Dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- All other prerequisite checks are performed internally by Ceph and any probable errors are taken care of internally by Ceph.

Procedure

1. From the dashboard navigation, go to **Cluster › Hosts**.
2. From the **Hosts List**, select the host that is in`maintenance`.

   > **Note:** You can identify the host in maintenance by checking for the **maintenance** label in the **Status** column.

3. Click **Exit Maintenance** from the action drop-down.
   After exiting the maintenance mode, you need to create the required services on the host by default-crash and the node-exporter gets deployed.

A notification is displayed that the host has been moved out of maintenance successfully and the maintenance label is removed from the **Status** column.

Removing hosts

To remove a host from a Ceph cluster, you can use`Start Drain`and`Remove`options in Red Hat Ceph Storage Dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- All other prerequisite checks are performed internally by Ceph and any probable errors are taken care of internally by Ceph.

Procedure

1. From the dashboard navigation, go to **Cluster › Hosts**.
2. Select the host that is to be removed, and click **Start Drain** from the action drop-down.
   This option drains all the daemons from the host.

   Note: The `_no_schedule` label is automatically applied to the host, which blocks the deployment of daemons on this host.

   a. To stop the draining of daemons from the host, click **Stop Drain** option from the action drop-down menu.
3. Check that all the daemons are removed from the host.
   a. Expand the host row.
   b. Go to the **Daemons** tab.
   No daemons should be listed.
4. Select the host that is to be removed, and click **Remove** from the action drop-down.
5. In the **Remove Host** dialog, select **Yes, I am sure** and click **Remove Host**.
   A notification displays that the host is removed successfully.

Managing pools

As a storage administrator, you can create, edit, and delete pools on the Red Hat Ceph Storage dashboard.

Creating pools

When you deploy a storage cluster without creating a pool, Ceph uses the default pools for storing data. You can create pools to logically partition your storage objects on the Red Hat Ceph Storage dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.

About this task

For more information about pools, see Pools.

Procedure

1. From the dashboard navigation, go to **Cluster › Pools**.
2. Click **Create**.
3. Fill out the **Create Pool** form.
   The form changes based off of selection. Not all fields are mandatory.
   - Set the name of the pool .
   - Select the **Pool type**, either `replicated` or `erasure`. Erasure is referred to as Erasure Coded (EC).
   - Optional: Select if the the **PG Autoscale** ison,`off`, or`warn`.
   - Enter the number of Placement groups (PGs).
   - Optional: If using a replicated pool type, set the replicated size.
   - Optional: If using an EC pool type configure the following additional settings.
   - Optional: To see the settings for the currently selected EC profile, click the **Information** icon.
   - Optional: Add a new EC profile by clicking the **Add** icon.
   - Optional: Click the **Edit** icon to select an application for the pool.
   - Optional: Set the CRUSH rule, if applicable.
   - Optional: If compression is required, select`passive`,`aggressive`, or`force`.
   - Optional: Set the Quotas.
   - Optional: Set the Quality of Service configuration.
4. Save the changes and complete creating the pool, by clicking **Create Pool**.
   A notification displays that the pool was created successfully.

Editing pools

Edit the pools on the Red Hat Ceph Storage Dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool is created

Procedure

1. From the dashboard navigation, go to **Cluster › Pools**.
2. Select the pool that needs to be edited, and click **Edit**.
3. In the **Edit Pool** form, edit the required parameters and click **Edit Pool**.
   A notification displays that the pool was updated successfully.

Deleting pools

Delete the pools on the Red Hat Ceph Storage Dashboard. Ensure that value of `mon_allow_pool_delete` is set to `True` in Manager modules.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool is created.

About this task

For more information about pools, see the following:

- Pools
- Pool values

Procedure

1. From the dashboard navigation, go to **Administration › Configuration**.
2. From the **Configuration** table, select `mon_allow_pool_delete`, and click **Edit**.

   > **Note:** If needed, clear filters and search for the configuration.

3. From the **Edit mon_allow_pool_delete** form, in **Values**, set all values to`true`.
4. Click **Update**.
   A notification displays that the configuration was updated successfully.
5. Go to **Cluster › Pools**.
6. Select the pool to be deleted, and click **Delete** from the action drop-down.
7. In the **Delete Pool** dialog, select **Yes, I am sure** and complete, by clicking **Delete Pool**.
   A notification displays that the pool was deleted successfully.

Managing Ceph OSDs from the Ceph Dashboard

As a storage administrator, you can monitor and manage OSDs on the Red Hat Ceph Storage Dashboard.

Some of the capabilities of the Red Hat Ceph Storage Dashboard are:

- List OSDs, their status, statistics, information such as attributes, metadata, device health, performance counters and performance details.
- Mark OSDs down, in, out, lost, purge, reweight, scrub, deep-scrub, destroy, delete, and select profiles to adjust backfilling activity.
- List all drives associated with an OSD.
- Set and change the device class of an OSD.
- Deploy OSDs on new drives and hosts.

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- `cluster-manager` level of access on the Red Hat Ceph Storage dashboard.

Managing the OSDs

Manage Ceph OSD lifecycle and maintenance tasks by using the Red Hat Ceph Storage Dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Hosts, Monitors, and Manager Daemons are added to the storage cluster.

About this task

Use the Red Hat Ceph Storage Dashboard to centrally manage Object Storage Daemons (OSDs) and perform common operational and maintenance tasks.

From the dashboard, you can complete the following OSD management actions:

- Create new OSDs.
- Edit the device class assigned to an OSD.
- Set OSD flags, such as **No Up**, **No Down**, **No In**, or **No Out**.
- Run scrub and deep-scrub operations on OSDs.
- Reweight OSDs to rebalance data placement.
- Change OSD state to **Out**, **In**, **Down**, or **Lost**.
- Purge, destroy, or delete OSDs as part of decommissioning workflows.

To access OSD management, from the dashboard navigation, click **Cluster › OSDs**.

Creating an OSD

Procedure

1. From the **OSDs List** page, click **Create**.

   > **Note:** Ensure you have an available host and a few available devices. You can check for available devices in **Cluster › Physical Disks**.

2. In the **Create OSDs** form, from the **Deployment Options** section, select a deployment option.
   Cost/Capacity-optimized

   The cluster gets deployed with all available HDDs.

   Throughput-optimized

   Slower devices are used to store data and faster devices are used to store journals/WALs.

   IOPS-optmized

   All the available NVMes are used to deploy OSDs.

3. Optional: Add a additional devices in the **Advanced Mode** section, by clicking **Add**.
   Primary devices

   Primary storage devices contain all OSD data.

   WAL devices

   Write-Ahead-Log devices are used for BlueStore's internal journal and are used only if the WAL device is faster than the primary device. For example, NVMes or SSDs.

   DB devices

   DB devices are used to store BlueStore's internal metadata and are used only if the DB device is faster than the primary device. For example, **NVMEs** or **SSDs**.

4. For security, select **Encryption** in the **Features** section.
5. Click **Create**.
   A notification displays that the OSD was created successfully and the OSD status changes from `in` and `down` to `in` and `up`.

Editing an OSD

Procedure

1. Select an OSD in the **OSDs List**, and click **Edit**.
2. In the **Edit OSD** dialog, edit the device class and click **Edit OSD**.
   A notification displays that the OSD was updated successfully.

Marking OSD flags

Procedure

1. Select the OSD in the **OSDs List**, and click **Flags** from the action drop-down.
2. Select the flags needed and click **Update**.
   A notification displays that the OSD flags updated succesfully.

Scrubbing OSDs

Procedure

1. Select the OSD in the **OSDs List**, and click **Scrub** from the action drop-down.
2. In the **OSDs Scrub** dialog, click **Update**.
   A notification displays that the OSD scrubbing was initiated successfully.

Deep scrubbing

Procedure

1. Select the OSD in the **OSDs List**, and click **Deep Scrub** from the action drop-down.
2. In the **OSDs Deep Scrub** dialog, click **Update**.
   A notification displays that the OSD deep scrubbing was initiated successfully.

Reweighting OSDs

Procedure

1. Select the OSD in the **OSDs List**, and click **Reweight** from the action drop-down.
2. In the **Reweight OSD** dialog, enter a value between 0 and 1.
3. Click **Reweight**.

Marking OSDsOut

Procedure

1. Select the OSD in the **OSDs List**, and click **Mark Out** from the action drop-down.
2. In the **Mark OSD out** notification, click **Mark out**.
   The OSD status changes toOut.

Marking OSDsIn

Procedure

1. Select the OSD in the **OSDs List**, and click **Mark In** from the action drop-down.
2. In the **Mark OSD in** notification, click **Mark in**.
   The OSD status changes toIn.

Marking OSDsDown

Procedure

1. Select the OSD in the **OSDs List**, and click **Mark Down** from the action drop-down.
2. In the **Mark OSD down** notification, click **Mark Down**.
   The OSD status changes toDown.

Marking OSDs

Lost

Procedure

1. Select the OSD that is in`out`and`down`status from the **OSDs List**.
2. Click **Mark Lost** from the action drop-down.
3. In the **Mark OSD Lost** dialog, select **Yes, I am sure**, and click **Mark Lost**.

Purging OSDs

Procedure

1. Select the OSD that is in the`down`status from the **OSDs List**.
2. Click **Purge** from the action drop-down.
3. In the **Purge OSDs** dialog, select **Yes, I am sure**, and click **Purge OSD**.
   All the flags are reset and the OSD is back in`in`and`up`status.

Destroying OSDs

Procedure

1. Select the OSD that is in the`down`status from the **OSDs List**.
2. Click **Detroy** from the action drop-down.
3. In the **Destroy OSDs** dialog, select **Yes, I am sure**, and click **Destroy OSD**.
   The status of the OSD changes to`destroyed`.

Deleting OSDs

Procedure

1. Select the OSD that is in the`down`status from the **OSDs List**.
2. Click **Delete** from the action drop-down.
3. In the **Delete OSDs** dialog, select **Yes, I am sure**, and click **Delete OSD**.

> **Note:** You can preserve the OSD_ID when you have to replace the failed OSD.

Replacing the failed OSDs

You can replace the failed OSDs in a Red Hat Ceph Storage cluster with the cluster-manager level of access on the dashboard. One of the highlights of this feature on the dashboard is that the OSD IDs can be preserved while replacing the failed OSDs.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- At least `cluster-manager` level of access to the Ceph Dashboard.
- At least one of the OSDs is `down`

Procedure

1. From the dashboard, identify the failed OSDs.
   The failed OSDs can be identified in any of the following ways:
   - Dashboard AlertManager pop-up notifications.
   - Dashboard landing page showing `HEALTH_WARN` status.

- Dashboard landing page showing failed OSDs.
- Dashboard OSD page showing failed OSDs.
  You can also view the LED blinking lights on the physical drive if one of the OSDs is down.

“Figure: Health status of OSDs on dashboard” on page 710 shows tone down and one `out` OSD from the Ceph Dashboard landing page.

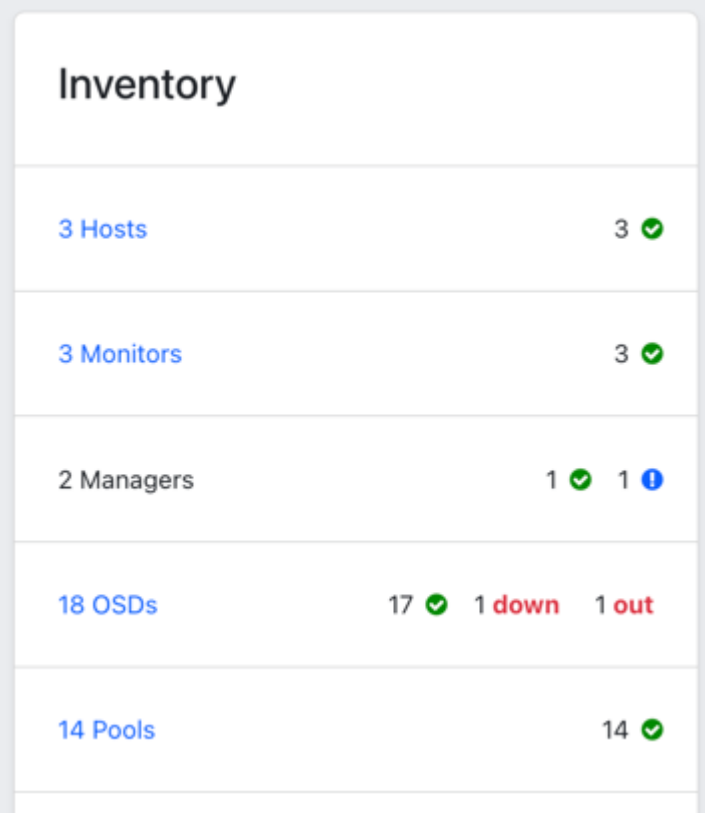


Figure 63: Health status of OSDs on dashboard

2. From **Cluster › OSDs** on the **OSDs List** table, select the`out`and`down`OSD.
   a. Click **Flags** from the action drop-down, select`No Up`in the **Individual OSD Flags** form, and click `Update`.
   b. Click **Delete** from the action drop-down. In the **Delete OSD** dialog, select **Preserve OSD ID(s) for replacement** and **Yes, I am sure** and click **Delete OSD**.
   c. Wait until the status of the OSD changes to`out`and`destroyed`.
3. Optional: To change the`No Up`flag for the entire cluster, from the **Cluster-wide configuration** menu, select **Flags**.
   a. In the **Cluster-wide configuration** form, select **No Up** and click **Update**.
4. Optional: If the OSDs are down due to a hard disk failure, replace the physical drive.
   - If the drive is hot-swappable, replace the failed drive with a new one.
   - If the drive is not hot-swappable and the host contains multiple OSDs, you might have to shut down the whole host and replace the physical drive. Consider preventing the cluster from backfilling. For more information, see Stopping and Starting Rebalancing.
   - When the drive appears under the `/dev/` directory, make a note of the drive path.
   - If you want to add the OSD manually, find the OSD drive and format the disk.
   - If the new disk has data, zap the disk:

     ```
     ceph orch device zap HOST_NAME PATH --force
     ```

     For example,

```
ceph orch device zap ceph-adm2 /dev/sdc --force
```

5. From **Cluster › OSDs** on the **OSDs List** table, click **Create**.
6. In the **Create OSDs** from **Advanced Mode** section, add a primary device.
   a. In the **Primary devices** dialog, select a **Hostname** filter.
   b. Select a device type from the list.

      > **Note:** You have to select the Hostname first and then at least one filter to add the devices. For example, from **Hostname** list, select **Type** and then **hdd**.

   c. Select **Vendor** from the device list, select **ATA**.
   d. Click **Add**.

"Figure: Using Primary devices filter" on page 711 shows how to select the **hdd** type in the Primary devices filter.

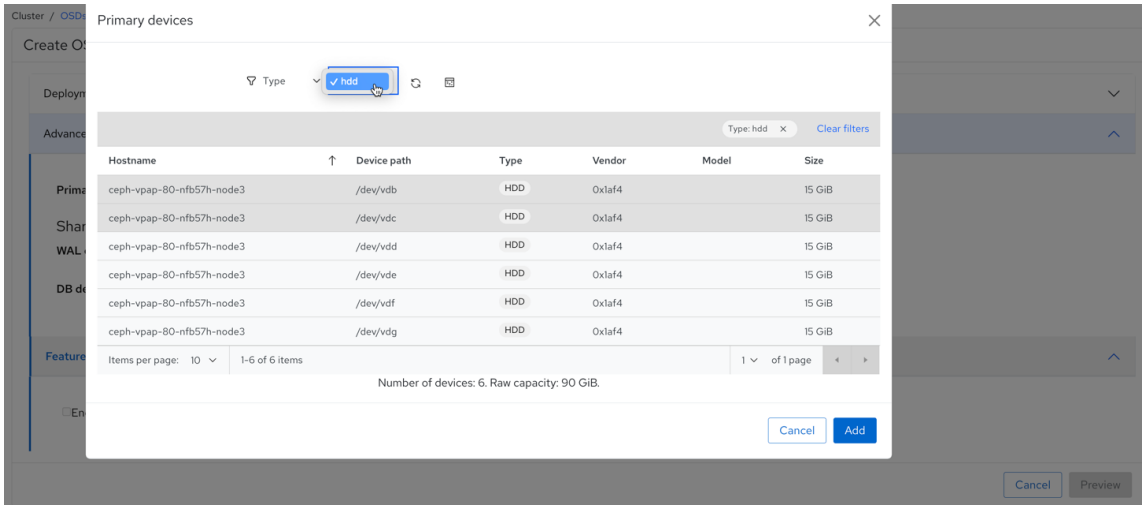


Figure 64: Using Primary devices filter

7. In the **Create OSDs** from, click **Preview**.
8. In the **OSD Creation Preview** dialog, click **Create**.
   A notification displays that the OSD is created successfully and the OSD changes to be in the`out`and`down` status.
9. Select the newly created OSD that has`out`and`down`status.
   a. From the action drop-down, click **Mark In**.
   b. In the **Mark OSD in** notification, click **Mark In**.
      The OSD status changes to`in`.
   c. Click **Flags** from the action drop-down.
   d. Clear the **No Up** selection and click **Update**.
10. Optional: If you have changed the`No Up`flag before for cluster-wide configuration, in the **Cluster-wide configuration** menu, select **Flags**.
    a. In **Cluster-wide OSDs Flags** form, clear the **No Up** selection and click **Update**.

What to do next

Verify that the OSD that was destroyed is created on the device and the OSD ID is preserved.

Managing alerts

As a storage administrator, you can view alert details and create silences for alerts on the IBM Storage Ceph Dashboard.

The following sections list the predefined alerts, grouped by severity. Use the alert index to go directly to a specific alert.

Critical alerts

- CephHealthError
- CephHealthWarning
- CephMonDownQuorumAtRisk
- CephMonDiskspaceCritical
- CephOSDDownHigh
- CephOSDFull
- CephDeviceFailurePredictionTooHigh
- CephFilesystemDamaged
- CephFilesystemOffline
- CephFilesystemDegraded
- CephFilesystemFailureNoStandby
- CephFilesystemReadOnly
- CephMgrModuleCrash
- CephMgrPrometheusModuleInactive
- CephPGsInactive
- CephPGsDamaged
- CephPGRecoveryAtRisk
- CephNodeRootFilesystemFull
- CephPoolFull
- CephadmUpgradeFailed
- CephadmDaemonFailed
- HardwareStorageError
- HardwareMemoryError
- HardwareProcessorError
- HardwareNetworkError
- HardwarePowerError
- HardwareFanError
- PrometheusJobMissing
- CephObjectMissing
- CephDaemonCrash
- CephRBDMirrorImagesPerDaemonHigh

Warning alerts

Warning alerts:

- CephPGsUnclean

- CephPGNotScrubbed
- CephPGsHighPerOSD
- CephPGNotDeepScrubbed
- CephNodeNetworkPacketDrops
- CephNodeNetworkPacketErrors
- CephNodeNetworkBondDegraded
- CephNodeDiskspaceWarning
- CephNodeInconsistentMTU
- CephPoolGrowthWarning
- CephPoolBackfillFull
- CephPoolNearFull
- CephSlowOps
- CephDaemonSlowOps
- CephadmPaused

CephHealthError

Severity: Critical

This alert is triggered when the cluster remains in the HEALTH_ERROR state for more than 5 minutes. This condition indicates a serious issue that affects cluster stability or data availability.

To resolve the issue, complete the following steps:

1. Check the cluster health:

   ```
   ceph -s
   ```

2. Review the detailed errors:

   ```
   ceph health detail
   ```

3. Resolve the reported issues.
4. Verify that the cluster returns to a healthy state.

CephHealthWarning

Severity: Critical

This alert is triggered when the cluster remains in the HEALTH_WARN state for more than 15 minutes. This condition indicates degraded performance or potential risks that require attention but are not yet critical.

To resolve the issue, complete the following steps:

1. Review the detailed health warning.
2. Resolve the reported issues.
3. Verify the cluster returns to healthy.

CephMonDownQuorumAtRisk

Severity: Critical

This alert is triggered when the monitor quorum is at risk. Quorum requires a majority of monitors (for example, 3 of 5) to be active. Without quorum, the cluster becomes inoperable and all services and clients are affected.

Perform the resolution steps on the node where the monitor daemon is down.

To resolve the issue, complete the following steps:

1. Identify the node where the monitor daemon is down:

```
ceph status
ceph orch ps | grep mon
```

The `ceph orch ps` output shows the host and daemon name for each monitor, which you use to locate the affected node.

2. On the affected node, check the current status of the monitor service:

```
systemctl status <mon-service-name>
```

3. Verify whether the service is active or failed. If the service is not running, start it on the affected node:

```
systemctl start <mon-service-name>
```

4. Recheck the service status to confirm that it is running.
5. If the service fails to start, check the logs and determine whether the monitor database is corrupted.
6. If corruption is detected, replace the monitor. For more information, see Ceph Monitor is out of quorum.

CephMonDiskspaceCritical

Severity: Critical

This alert is triggered when the available free space in the Ceph monitor store is critically low. If the disk becomes full, the monitor service might stop functioning, which can affect quorum and overall cluster stability.

The default store locations are:

- From the cephadm shell: `/var/lib/ceph/mon/ceph-<mon-name>/store.db`
- From the host: `/var/lib/ceph/<fsid>/mon.<daemon-name>/`

> **Note:** Do not stop multiple monitor daemons simultaneously. Always ensure that quorum is maintained while you perform maintenance.

To resolve the issue, complete the following steps:

1. Ensure that the cluster is healthy and that monitors are in quorum:

```
ceph -s
```

2. Check the overall disk space on the monitor nodes:

```
df -h
```

3. Identify disk consumption from logs and of the monitor data:

```
du -sh /var/lib/ceph/mon/*
```

> **Note:** In many deployments, the log directory (`/var/log/ceph/`) is mounted on a separate disk or volume. Its usage does not necessarily reflect the available space in the monitor store, so it is not included in this check. Log cleanup is covered in a later step.

4. Locate the correct monitor database path:
   - Standard location: `/var/lib/ceph/mon/ceph-<mon-name>/store.db`
   - cephadm deployments (host view): `/var/lib/ceph/<fsid>/mon.<daemon-name>/`

> **Note:** Inside the cephadm shell, paths appear differently.

5. Free up disk space safely:
   - Rotate or clean old log files under `/var/log/ceph/`.
   - Remove unnecessary temporary or debug files.
   - Ensure that sufficient free space is restored.

   Important: Do not delete or modify monitor database files (`store.db`), because doing so can cause monitor failure or data loss.

6. Investigate whether excessive monitor database growth is caused by frequent cluster map updates (for example, flapping OSDs), and address the root cause.
7. Expand storage if required. Increase the disk capacity of the monitor nodes.
8. Maintain quorum during operations.
9. Verify recovery and confirm that the alert is cleared and that monitors are stable:

```
ceph -s
```

CephOSDDownHigh

Severity: Critical

This alert is triggered when a significant number of OSDs are down, which can affect data redundancy and cluster performance.

To resolve the issue, complete the following steps:

1. Verify the overall cluster health and identify the down OSDs:

```
ceph -s
ceph osd stat
ceph osd tree
```

   Note: The `ceph osd tree` command is sufficient to identify down OSDs.

2. Identify whether an OSD went down because of a crash:

```
ceph crash ls
ceph crash info <crash-id>
```

   This information helps determine whether the issue is related to a bug or a recurring failure.

3. Restart the failed OSD by using the correct systemd unit format (with the FSID):

```
systemctl restart ceph-<fsid>@osd.<osd-id>.service
```

   For example:

```
systemctl restart ceph-861a07b2-1179-11f1-bb0e-52540048746d@osd.0.service
```

4. Check the logs for disk, host, or network-related problems:

```
journalctl -u ceph-<fsid>@osd.<osd-id>.service --no-pager
```

5. Address the root cause:
   - Fix hardware issues (disk, node, or network).
   - Investigate frequent failures or flapping OSDs.
   - Resolve bugs if crashes are observed.

6. Replace or recreate the OSD if required:
   - Replace the failed disk if necessary.
   - Recreate the OSD by using `ceph cephadm` only (not `ceph-volume` manually).
   - Follow the appropriate procedure based on the deployment type (colocated, non-colocated, or custom LVM).

   For more information, see Replacing an OSD drive.
7. Verify cluster recovery:

```
ceph -s
```

CephOSDFull

Severity: Critical

This alert is triggered when one or more OSDs reach the FULL threshold. When this occurs, writes to pools that use the affected OSDs are blocked. This condition can halt client I/O and place the cluster at risk until it is resolved. For more information about resolving OSD full issues, see Troubleshooting Ceph OSDs.

CephDeviceFailurePredictionTooHigh

Severity: Critical

This alert is triggered when the device health module predicts multiple device failures, but remediation cannot proceed automatically. Removing too many OSDs at once risks cluster performance and availability. To prevent data integrity issues, add new OSDs so that data can be relocated safely.

To resolve the issue, complete the following steps:

1. Check device health:

```
ceph device ls
ceph device info <device-id>
```

2. Review the SMART metrics:

```
smartctl -a /dev/<device>
```

3. Mark the corresponding OSD out. Monitor the cluster status and ensure that all placement groups (PGs) return to a healthy state:

```
ceph -s
```

> **Note:** Proceed only when the PGs are active and clean.

4. After the data is rebalanced safely, remove the OSD from the cluster:

```
ceph osd purge <osd-id> --yes-i-really-mean-it
```

5. Physically replace the disk and recreate the OSD by using the appropriate deployment method.
6. Verify cluster health:

```
ceph -s
```

CephFilesystemDamaged

Severity: Critical

This alert is triggered when CephFS metadata becomes corrupted, which might render data inaccessible. Analyze the metrics from the MDS daemon admin socket, or escalate the issue to support for recovery actions.

To resolve the issue, complete the following steps:

1. Check the cluster health:

```
ceph -s
```

2. Check for for a message similar to the following:

```
MDSs report damaged metadata
```

3. Check the filesystem status:

```
ceph fs status
```

4. Inspect the MDS status:

```
ceph mds stat
```

5. List the reported metadata damage:

```
ceph tell mds.<fs-name>:0 damage ls
```

Review the reported damage type. If the damage type is backtrace, follow the documented recovery procedure for metadata damage. Different damage types require different recovery procedures.If the reported damage type is not recognized or recovery is unsuccessful, contact IBM Support for further assistance. Avoid attempting manual metadata repair without guidance, because incorrect recovery actions can result in data loss.

CephFilesystemOffline

Severity: Critical

This alert is triggered when all Metadata Server (MDS) ranks are unavailable. With no active MDS daemons, the CephFS filesystem cannot serve metadata requests and becomes offline.

To resolve the issue, complete the following steps:

1. Check the MDS status:

```
ceph mds stat
```

2. Restart the MDS daemons.
3. Ensure that sufficient active MDS daemons are available.
4. Verify that the filesystem comes online.

CephFilesystemDegraded

Severity: Critical

This alert is triggered when one or more CephFS Metadata Server (MDS) ranks are failed or in a damaged state. This condition can lead to partial filesystem availability or, in severe cases, complete failure.

To resolve the issue, complete the following steps:

1. Check the MDS and OSD status:

```
ceph -s
```

2. Restore the failed components.
3. Monitor the recovery progress.

CephFilesystemFailureNoStandby

Severity: Critical

This alert is triggered when an active MDS daemon fails and no standby MDS is available to take over.

To resolve the issue, complete the following steps:

1. Add a standby MDS:

```
ceph orch apply mds <fs-name> --placement=<nodes>
```

2. Verify standby availability:

```
ceph mds stat
```

CephFilesystemReadOnly

Severity: Critical

This alert is triggered when the CephFS filesystem switches to read-only mode because of unexpected write errors in the metadata pool.

To resolve the issue, complete the following steps:

1. Check the cluster health:

```
ceph -s
```

2. Identify the underlying errors.
3. Resolve the OSD or MDS issues.
4. Restore write access.

CephMgrModuleCrash

Severity: Critical

This alert is triggered when one or more Ceph Manager modules crash.

To resolve the issue, complete the following steps:

1. Check the daemon status:

```
ceph -s
ceph mgr stat
```

2. Review the logs:

```
systemctl | grep mgr
ceph-eb4c0bd6-64cf-11f1-b1e9-525400e3b50b@mgr.testnode-0.fuvasw.service
loaded active running
Ceph mgr.testnode-0.fuvasw for eb4c0bd6-64cf-11f1-b1e9-525400e3b50b
```

3. Restart the daemon.

CephMgrPrometheusModuleInactive

Severity: Critical

This alert is triggered when the mgr/prometheus module becomes unreachable.

To resolve the issue, complete the following steps:

1. Check the module status:

```
ceph mgr module ls
```

2. Enable the module:

```
ceph mgr module enable prometheus
```

3. Verify that Prometheus scraping resumes.

CephPGsInactive

Severity: Critical

This alert is triggered when one or more placement groups (PGs) remain inactive for more than 5 minutes.

To resolve the issue, complete the following steps:

1. Check the cluster health:

```
ceph -s
```

2. Inspect the PG state:

```
ceph pg stat
```

3. Restore the failed OSDs.
4. Resolve any network issues. For more information, see Inactive placement groups.

CephPGsDamaged

Severity: Critical

This alert is triggered when one or more placement groups (PGs) are marked inconsistent or damaged.

To resolve the issue, complete the following steps:

1. Identify the affected PGs:

```
ceph health detail
```

2. Attempt a repair:

```
ceph pg repair <pg-id>
```

CephPGRecoveryAtRisk

Severity: Critical

This alert is triggered when OSDs reach the FULL threshold, which affects recovery.

To resolve the issue, complete the following steps:

1. Check the recovery status:

```
ceph -s
```

2. Ensure that sufficient free space is available.
3. Restore OSD availability.
4. Monitor the recovery and backfill progress. For more information see, Full OSDs.

CephNodeRootFilesystemFull

Severity: Critical

This alert is triggered when the root filesystem has less than 5% free space.

To resolve the issue, complete the following steps:

1. Check the filesystem usage:

```
df -h
```

2. Free up disk space.
3. Verify node stability.

CephPoolFull

Severity: Critical

This alert is triggered when a pool reaches the FULL threshold.

To resolve the issue, complete the following steps:

1. Check the pool usage:

```
ceph df
```

2. Increase the pool capacity.
3. Delete unused data.
4. Adjust the quotas if required.

CephadmUpgradeFailed

Severity: Critical

This alert is triggered when a cephadm upgrade fails.

To resolve the issue, complete the following steps:

1. Check the upgrade status:

```
ceph orch upgrade status
```

2. Review the logs:

```
ceph orch ps
```

3. Fix the blocking issues.
4. Retry the upgrade.

CephadmDaemonFailed

Severity: Critical

This alert is triggered when a daemon that is managed by cephadm becomes inactive or fails to respond. Depending on the failed service, this condition might affect the overall functionality of the cluster.

To resolve the issue, complete the following steps:

1. Check the daemon status:

```
ceph orch ps
```

2. Restart the failed daemon:

```
ceph orch daemon start <daemon-id>
```

3. Review the logs.

HardwareStorageError

Severity: Critical

This alert is triggered when one or more storage devices report hardware errors.

To resolve the issue, complete the following steps:

1. Check the disk or device status:

```
ceph device ls
ceph device info <device-id>
```

2. Review the system logs.
3. Check the SMART health:

```
smartctl -a /dev/<device>
```

4. Replace the failed disk.
5. Verify cluster health:

```
ceph -s
```

HardwareMemoryError

Severity: Critical

This alert is triggered when memory (DIMM) errors are detected on cluster nodes.

To resolve the issue, complete the following steps:

1. Check the system logs:

```
dmesg | grep -i memory
```

2. Verify ECC or memory errors by using hardware tools.
3. Verify node stability and cluster health:

```
ceph -s
```

HardwareProcessorError

Severity: Critical

This alert is triggered when CPU hardware errors are detected.

To resolve the issue, complete the following steps:

1. Check the CPU errors:

```
dmesg | grep -i cpu
```

2. Review the hardware logs (IPMI or vendor tools)..
3. Verify node and cluster health:

```
ceph -s
```

HardwareNetworkError

Severity: Critical

This alert is triggered when network-related hardware errors are detected.

To resolve the issue, complete the following steps:

1. Check network connectivity:

```
ping <peer-node>
```

2. Verify the interfaces:

```
ip addr
```

3. Check the Ceph network status:

```
ceph -s
```

4. Resolve the network issues.

HardwarePowerError

Severity: Critical

This alert is triggered when power supply units report hardware errors.

To resolve the issue, complete the following steps:

1. Check the system power logs (IPMI).
2. Verify the PSU status and redundancy.
3. Replace the faulty power supply.
4. Confirm node stability.

HardwareFanError

Severity: Critical

This alert is triggered when cooling fans report hardware errors.

To resolve the issue, complete the following steps:

1. Check the hardware health sensors (IPMI).
2. Verify the cooling system status.
3. Replace the failed fan.
4. Ensure that the node temperature is stable.

PrometheusJobMissing

Severity: Critical

This alert is triggered when the Prometheus scrape job is missing.

To resolve the issue, complete the following steps:

1. Check the Prometheus targets and jobs.
2. Verify the Ceph Manager module:

   ```
   ceph mgr module ls
   ```

3. Enable the module if needed:

   ```
   ceph mgr module enable prometheus
   ```

4. Restart the Prometheus service.
5. Verify metrics collection.

CephObjectMissing

Severity: Critical

This alert is triggered when a RADOS object cannot be found.

To resolve the issue, complete the following steps:

1. Check the cluster health:

   ```
   ceph health detail
   ```

2. Identify the affected PGs.
3. Attempt a repair:

   ```
   ceph pg repair <pg-id>
   ```

4. Verify cluster health:

   ```
   ceph -s
   ```

CephDaemonCrash

Severity: Critical

This alert is triggered when one or more Ceph daemons crash and are unacknowledged.

To resolve the issue, complete the following steps:

1. List the crashes:

   ```
   ceph crash ls
   ```

2. Review the crash details:

   ```
   ceph crash info <id>
   ```

3. Acknowledge the crashes:

```
ceph crash archive <id>
```

CephRBDMirrorImagesPerDaemonHigh

Severity: Critical

This alert is triggered when too many RBD images are handled by a single mirroring daemon.

To resolve the issue, complete the following steps:

1. Check the mirror status:

   ```
   rbd mirror pool status
   ```

2. Distribute the load across daemons.
3. Add more mirroring daemons if required.

CephPGsUnclean

Severity: Warning

This alert is triggered when one or more placement groups (PGs) remain in an unclean state for more than 15 minutes.

To resolve the issue, complete the following steps:

1. Check the cluster health:

   ```
   ceph -s
   ```

2. Review the recovery status:

   ```
   ceph pg stat
   ```

3. Identify and fix the underlying issues. For more information, see Unclean placement groups.

CephPGNotScrubbed

Severity: Warning

This alert is triggered when PGs are not scrubbed within the expected interval.

To resolve the issue, complete the following steps:

1. Check the PG scrub status:

   ```
   ceph health detail
   ```

2. Trigger a manual scrub:

   ```
   ceph pg scrub <pg-id>
   ```

3. Verify scrub completion.

CephPGsHighPerOSD

Severity: Warning

This alert is triggered when the PG count per OSD exceeds the configured limits.

To resolve the issue, complete the following steps:

1. Check the autoscaler status:

   ```
   ceph osd pool autoscale-status
   ```

2. Review the PG distribution. For more information, see Recalculating the placement groups.
3. Adjust the pool settings if required.

CephPGNotDeepScrubbed

Severity: Warning

This alert is triggered when PGs do not undergo a deep scrub within the expected interval.

To resolve the issue, complete the following steps:

1. Check the deep scrub status:

```
ceph health detail
```

2. Trigger a deep scrub:

```
ceph pg deep-scrub <pg-id>
```

3. Verify completion.

CephNodeNetworkPacketDrops

Severity: Warning

This alert is triggered when network packet drops exceed the threshold.

To resolve the issue, complete the following steps:

1. Check the interface statistics:

```
ip -s link
```

2. Verify network connectivity. For more information, see Basic networking troubleshooting.
3. Inspect the cables and NICs.

CephNodeNetworkPacketErrors

Severity: Warning

This alert is triggered when network packet errors exceed the threshold.

To resolve the issue, complete the following steps:

1. Check the interface errors:

```
ip -s link
```

2. Review the system logs:

```
dmesg
```

3. Fix the NIC or configuration issues.

CephNodeNetworkBondDegraded

Severity: Warning

This alert is triggered when a bonded interface has fewer active links than expected.

To resolve the issue, complete the following steps:

1. Check the bond status:

```
cat /proc/net/bonding/*
```

2. Identify the failed interfaces.
3. Restore or replace the interfaces.

CephNodeDiskspaceWarning

Severity: Warning

This alert is triggered when a filesystem is predicted to run out of space within 5 days.

To resolve the issue, complete the following steps:

1. Check the disk usage:

```
df -h
```

2. Clean unnecessary files.
3. Expand storage if needed.

CephNodeInconsistentMTU

Severity: Warning

This alert is triggered when the MTU size differs across nodes.

To resolve the issue, complete the following steps:

1. Check the MTU settings:

```
ip link
```

2. Standardize the MTU across nodes.
3. Restart networking if required.

CephPoolGrowthWarning

Severity: Warning

This alert is triggered when a pool is projected to reach full capacity soon.

To resolve the issue, complete the following steps:

1. Check the pool usage:

```
ceph df
```

2. Plan capacity expansion.
3. Clean unused data.

CephPoolBackfillFull

Severity: Warning

This alert is triggered when a pool nears the backfillfull threshold.

To resolve the issue, complete the following steps:

1. Check the cluster status:

```
ceph -s
```

2. Free up space or add capacity.
3. Monitor the recovery progress.

CephPoolNearFull

Severity: Warning

This alert is triggered when a pool exceeds the nearfull threshold.

To resolve the issue, complete the following steps:

1. Check the pool usage:

```
ceph df
```

2. Add capacity or delete unused data.
3. Adjust the quotas if required. For more information, see Setting pool quota.

CephSlowOps

Severity: Warning

This alert is triggered when OSD operations exceed the expected latency.

To resolve the issue, complete the following steps:

1. Check the cluster latency:

```
ceph -s
```

2. Review the OSD performance.
3. Investigate network or disk bottlenecks. For more information, see Slow requests or requests are blocked.

CephDaemonSlowOps

Severity: Warning

This alert is triggered when daemon operations are slow.

To resolve the issue, complete the following steps:

1. Check the daemon status:

```
ceph -s
```

2. Review the logs.
3. Resolve the performance bottlenecks.

CephadmPaused

Severity: Warning

This alert is triggered when cephadm operations are paused.

To resolve the issue, complete the following steps:

1. Check the orchestrator status:

```
ceph orch status
```

2. Resume operations:

```
ceph orch resume
```

3. Verify cluster operations.

Enabling monitoring stack

Alerts for the monitoring stack of the Red Hat Ceph Storage are viewed on the dashboard. However, enabling the monitoring stack is done through the command-line interface. The monitoring stack includes Prometheus, Alertmanager, and Grafana. You can use the Prometheus and Alertmanager API to manage alerts and silences.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to all the hosts.

Procedure

1. Log into the `cephadm` shell:

   Example

```
[root@host01 ~]# cephadm shell
```

2. Set the APIs for the monitoring stack:
   - Specify the host and port of the Alertmanager server:

Syntax

```
ceph dashboard set-alertmanager-api-host 'ALERTMANAGER_API_HOST:PORT'
```

Example

```
[ceph: root@host01 /]# ceph dashboard set-alertmanager-api-host 'http://
10.0.0.101:9093'
Option ALERTMANAGER_API_HOST updated
```

- To see the configured alerts, configure the URL to the Prometheus API. Using this API, the Ceph Dashboard UI verifies that a new silence matches a corresponding alert.

  Syntax

  ```
  ceph dashboard set-prometheus-api-host 'PROMETHEUS_API_HOST:PORT'
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph dashboard set-prometheus-api-host 'http://
  10.0.0.101:9095'
  Option PROMETHEUS_API_HOST updated
  ```

  After setting up the hosts, refresh your browser's dashboard window.

- Specify the host and port of the Grafana server:

  Syntax

  ```
  ceph dashboard set-grafana-api-url 'GRAFANA_API_URL:PORT'
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph dashboard set-grafana-api-url 'http://
  10.0.0.101:3000'
  Option GRAFANA_API_URL updated
  ```

3. Get the Prometheus, Alertmanager, and Grafana API host details:

   Example

   ```
   [ceph: root@host01 /]# ceph dashboard get-alertmanager-api-host
   http://10.0.0.101:9093
   [ceph: root@host01 /]# ceph dashboard get-prometheus-api-host
   http://10.0.0.101:9095
   [ceph: root@host01 /]# ceph dashboard get-grafana-api-url
   http://10.0.0.101:3000
   ```

4. Optional: If you are using a self-signed certificate in your Prometheus, Alertmanager, or Grafana setup, disable the certificate verification in the dashboard This avoids refused connections caused by certificates signed by an unknown Certificate Authority (CA) or that do not match the hostname.

   - For Prometheus:

     Example

     ```
     [ceph: root@host01 /]# ceph dashboard set-prometheus-api-ssl-verify False
     ```

   - For Alertmanager:

     Example

     ```
     [ceph: root@host01 /]# ceph dashboard set-alertmanager-api-ssl-verify False
     ```

   - For Grafana:

     Example

```
[ceph: root@host01 /]# ceph dashboard set-grafana-api-ssl-verify False
```

5. Get the details of the self-signed certificate verification setting for Prometheus, Alertmanager, and Grafana:

   Example

```
[ceph: root@host01 /]# ceph dashboard get-prometheus-api-ssl-verify
[ceph: root@host01 /]# ceph dashboard get-alertmanager-api-ssl-verify
[ceph: root@host01 /]# ceph dashboard get-grafana-api-ssl-verify
```

6. Optional: If the dashboard does not reflect the changes, you have to disable and then enable the dashboard:

   Example

```
[ceph: root@host01 /]# ceph mgr module disable dashboard
[ceph: root@host01 /]# ceph mgr module enable dashboard
```

Configuring a Grafana certificate

Configure a custom SSL/TLS certificate for Grafana by using the **certmgr** certificate manager.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- A valid SSL/TLS certificate.
- A corresponding private key.
- Access to the **cephadm** shell.

About this task

cephadm deploys Grafana by using certificates managed by the **certmgr** certificate manager. Grafana certificates are configured per host.

If you do not configure a custom certificate, **cephadm** generates and signs a certificate for each host where Grafana runs.

To use your own certificate, configure the Grafana service to use `certificate_source: reference` and upload the certificate and private key to **certmgr**.

Procedure

1. Log in to the **cephadm** shell.
   For example,

```
[root@host01 ~]# cephadm shell
```

2. Configure the Grafana service specification to reference certificates managed by **certmgr**.

```
service_type: grafana
placement:
  hosts:
  - ceph-node-hostname
spec:
  ssl: true
  certificate_source: reference
```

3. Upload the SSL/TLS certificate to **certmgr**.

```
ceph orch certmgr cert set \
--cert-name GRAFANA_CERTIFICATE \
--hostname CEPH_HOSTNAME \
-i $PWD/certificate.pem
```

For example,

```
[ceph: root@host01 /]# ceph orch certmgr cert set \
--cert-name grafana_ssl_cert \
--hostname ceph-node-hostname \
-i $PWD/certificate.pem
```

4. Upload the private key to **certmgr**.

```
ceph orch certmgr key set \
--key-name GRAFANA_KEY \
--hostname CEPH_HOSTNAME \
-i $PWD/key.pem
```

For example,

```
[ceph: root@host01 /]# ceph orch certmgr key set \
--key-name grafana_ssl_key \
--hostname ceph-node-hostname \
-i $PWD/key.pem
```

5. Apply the updated certificate configuration.

```
ceph orch reconfig grafana
```

Result

Grafana uses the user-provided SSL/TLS certificate and private key stored in **certmgr** for the specified host.

What to do next

Verify that the Ceph Dashboard and Grafana interface load successfully using the updated SSL/TLS certificate.

Adding Alertmanager webhooks

You can add new webhooks to an existing Alertmanager configuration to receive real-time alerts about the health of the storage cluster. You have to enable incoming webhooks to allow asynchronous messages into third-party applications.

For example, if an OSD is down in a Red Hat Ceph Storage cluster, you can configure the Alertmanager to send notification on Google chat.

Prerequisites

- A running Red Hat Ceph Storage cluster with monitoring stack components enabled.
- Incoming webhooks configured on the receiving third-party application.

Procedure

1. Log into the `cephadm` shell:

   Example

```
[root@host01 ~]# cephadm shell
```

2. Configure the Alertmanager to use the webhook for notification:

   Syntax

```
service_type: alertmanager
spec:
  user_data:
    default_webhook_urls:
    - "_URLS_"
```

The ‘default_webhook_urls is a list of additional URLs that are added to the default receivers’ webhook_configs` configuration.

Example

```
service_type: alertmanager
spec:
  user_data:
    webhook_configs:
    - url: 'http:127.0.0.10:8080'
```

3. Update Alertmanager configuration:

   Example

```
[ceph: root@host01 /]#  ceph orch reconfig alertmanager
```

Verification

– An example notification from Alertmanager to Gchat:

  Example

```
using: https://chat.googleapis.com/v1/spaces/(xx- space identifyer -xx)/messages
posting: {'status': 'resolved', 'labels': {'alertname': 'PrometheusTargetMissing',
'instance': 'postgres-exporter.host03.chest
response: 200
response: {
"name": "spaces/(xx- space identifyer -xx)/messages/3PYDBOsIofE.3PYDBOsIofE",
"sender": {
"name": "users/114022495153014004089",
"displayName": "monitoring",
"avatarUrl": "",
"email": "",
"domainId": "",
"type": "BOT",
"isAnonymous": false,
"caaEnabled": false
},
"text": "Prometheus target missing (instance postgres-exporter.cluster.local:9187)\n\nA
Prometheus target has disappeared. An e
"cards": [],
"annotations": [],
"thread": {
"name": "spaces/(xx- space identifyer -xx)/threads/3PYDBOsIofE"
},
"space": {
"name": "spaces/(xx- space identifyer -xx)",
"type": "ROOM",
"singleUserBotDm": false,
"threaded": false,
"displayName": "_privmon",
"legacyGroupChat": false
},
"fallbackText": "",
"argumentText": "Prometheus target missing (instance postgres-
exporter.cluster.local:9187)\n\nA Prometheus target has disappea
"attachment": [],
"createTime": "2022-06-06T06:17:33.805375Z",
"lastUpdateTime": "2022-06-06T06:17:33.805375Z"
```

Viewing alerts

After an alert has been triggered, you can view it on the Red Hat Ceph Storage Dashboard. You can edit the **Manager module** settings to trigger a mail when an alert is fired.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

– A running Red Hat Ceph Storage cluster.

– Dashboard is installed.

– A running simple mail transfer protocol (SMTP) configured.

– An alert fired.

Procedure

1. From the dashboard navigation, go to **Observability › Alerts**.
2. View the active Prometheus alerts from the **Active Alerts** tab.
3. View all alerts from the **Alerts** tab.
4. To view the source of an alert, click on its row and then click **Source**.

Creating a silence

You can create a silence for an alert for a specified amount of time on the Red Hat Ceph Storage Dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- An alert fired.

Procedure

1. From the dashboard navigation, go to **Observability › Alerts**.
2. On the **Silences** tab, click **Create**.
3. In the **Create Silence** form, fill in the required fields.
   Use the **Add matcher** to add silence requirements.

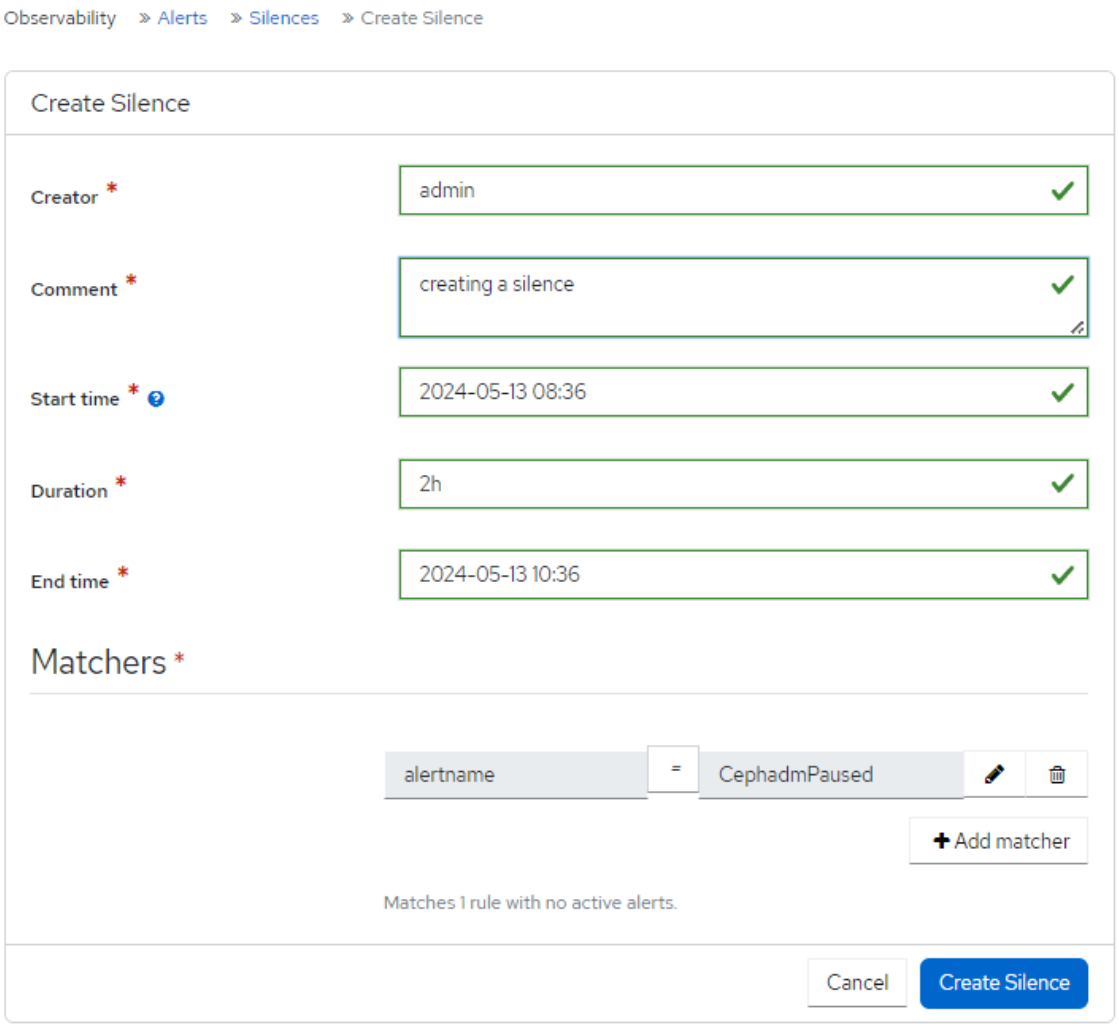


Figure 65: Creating a silence

A notification displays that the silence was created successfully and the **Alerts Silenced** updates in the **Silences** table.

Recreating a silence

You can recreate a silence from an expired silence on the Red Hat Ceph Storage Dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- An alert fired.
- A silence created for the alert.

Procedure

1. From the dashboard navigation, go to **Observability › Alerts**.
2. On the **Silences** tab, select the row with the alert that you want to recreate, and click **Recreate** from the action drop-down.
3. Edit any needed details, and click **Recreate Silence** button.
   A notification displays indicating that the silence was edited successfully and the status of the silence is now`active`.

Editing a silence

You can edit an active silence, for example, to extend the time it is active on the Red Hat Ceph Storage Dashboard. If the silence has expired, you can either recreate a silence or create a new silence for the alert.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- An alert fired.
- A silence created for the alert.

Procedure

1. From the dashboard navigation, go to **Observability › Alerts**.
2. On the **Silences** tab, select the row with the alert that you want ot edit, and click **Edit** from the action drop-down.

   > **Note:** To edit a matcher, click the **Edit** icon.

   A notification displays indicating that the silence was edited successfully.

Expiring a silence

You can expire a silence so any matched alerts will not be suppressed on the Red Hat Ceph Storage Dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.

- Dashboard is installed.
- An alert fired.
- A silence created for the alert.

Procedure

1. From the dashboard navigation, go to **Observability › Alerts**.
2. On the **Silences** tab, select the row with the alert that you want to expire, and click **Expire** from the action drop-down.
3. In the **Expire Silence** notification, select **Yes, I am sure** and click **Expire Silence**.
   A notification displays indicating that the silence was expired successfully and the Status of the alert is **expired**, in the Silences table.

Managing Ceph Object Gateway

As a storage administrator, the Ceph Object Gateway functions of the dashboard allow you to manage and monitor the Ceph Object Gateway.

You can also create the Ceph Object Gateway services with Secure Sockets Layer (SSL) using the dashboard.

For example, monitoring functions allow you to view details about a gateway daemon such as its zone name, or performance graphs of GET and PUT rates. Management functions allow you to view, create, and edit both users and buckets.

Ceph Object Gateway functions are divided between user functions and bucket functions.

Manually adding Ceph Object Gateway login credentials to the dashboard

The Red Hat Ceph Storage Dashboard can manage the Ceph Object Gateway, also known as the RADOS Gateway, or RGW. When Ceph Object Gateway is deployed with `cephadm`, the Ceph Object Gateway credentials used by the dashboard is automatically configured. You can also manually force the Ceph Object Gateway credentials to the Ceph dashboard using the command-line interface.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Ceph Object Gateway is installed.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Set up the credentials manually:

   Example

   ```
   [ceph: root@host01 /]# ceph dashboard set-rgw-credentials
   ```

   This creates a Ceph Object Gateway user with UID `dashboard` for each realm in the system.

3. Optional: If you have configured a custom `admin` resource in your Ceph Object Gateway admin API, you have to also set the the admin resource:

   Syntax

   ```
   ceph dashboard set-rgw-api-admin-resource RGW_API_ADMIN_RESOURCE
   ```

Example

```
[ceph: root@host01 /]# ceph dashboard set-rgw-api-admin-resource admin
Option RGW_API_ADMIN_RESOURCE updated
```

4. Optional: If you are using HTTPS with a self-signed certificate, disable certificate verification in the dashboard to avoid refused connections.

   Refused connections can happen when the certificate is signed by an unknown Certificate Authority, or if the host name used does not match the host name in the certificate.

   Syntax

   ```
   ceph dashboard set-rgw-api-ssl-verify false
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph dashboard set-rgw-api-ssl-verify False
   Option RGW_API_SSL_VERIFY updated
   ```

5. Optional: If the Ceph Object Gateway takes too long to process requests and the dashboard runs into timeouts, you can set the timeout value:

   Syntax

   ```
   ceph dashboard set-rest-requests-timeout _TIME_IN_SECONDS_
   ```

   The default value of 45 seconds.

   Example

   ```
   [ceph: root@host01 /]# ceph dashboard set-rest-requests-timeout 240
   ```

Creating the Ceph Object Gateway services with SSL using the dashboard

After installing a Red Hat Ceph Storage cluster, you can create the Ceph Object Gateway service with SSL by using the dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- SSL key from Certificate Authority (CA).

About this task

> **Note:** Obtain the SSL certificate from a CA that matches the hostname of the gateway host. Red Hat recommends obtaining a certificate from a CA that has subject alternate name fields and a wildcard for use with S3-style subdomains.

For more information, see “Configuring HTTPS for the Ceph Object Gateway” on page 838.

Procedure

1. From the dashboard navigation, go to **Administration › Services**.
2. Click **Create**.
3. Fill in the **Create Service** form.
   a. Select **rgw** from the **Type** service list.
   b. Enter the ID that is used in **`service_id`**.

c. Select **SSL**.

d. Click **Choose File** and upload the SSL certificate in `.pem` format.

4. Click **Create Service**.

5. Verify that the Ceph Object Gateway service is running.

Managing Ceph Object Gateway users

As a storage administrator, the Red Hat Ceph Storage Dashboard allows you to view and manage Ceph Object Gateway users.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- The Ceph Object Gateway is installed.
- Ceph Object Gateway login credentials are added to the dashboard.

Creating Ceph Object Gateway users

You can create Ceph Object Gateway users on the Red Hat Ceph Storage once the credentials are set-up using the CLI.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Ceph Object Gateway is installed.
- Ceph Object Gateway login credentials are added to the dashboard.

About this task

For more information, see the following:

- “Manually adding Ceph Object Gateway login credentials to the dashboard” on page 733.
- Ceph Object Gateway.

Procedure

1. From the dashboard navigation, go to **Object › Users**.
2. On the **Users** tab, click **Create**.
3. Set the parameters in the **Create User** form.
   a. Enter the **User ID** and **Full name**
   b. If required, edit the maximum number of buckets.
   c. Fill in an **Email address**.
   d. Select if the user is **Suspended** or a **System user**.
   e. In the **S3 key** section, set a custom access key and secret key by clearing the **Auto-generate key** selection.
   f. In the **User quota** section, select the user quota.
   The options for the user quota can is **Enabled**, **Unlimited size**, or has **Unlimited objects**. If there is a limited size enter the maximum size. If there are limited objects, enter the maximum objects.
   g. In the **Bucket quota** section, select the bucket quota.
   The options for the bucket quota can is **Enabled**, **Unlimited size**, or has **Unlimited objects**. If there is a limited size enter the maximum size. If there are limited objects, enter the maximum objects.
4. Click **Create User**.

A notification displays that the user was created successfully.

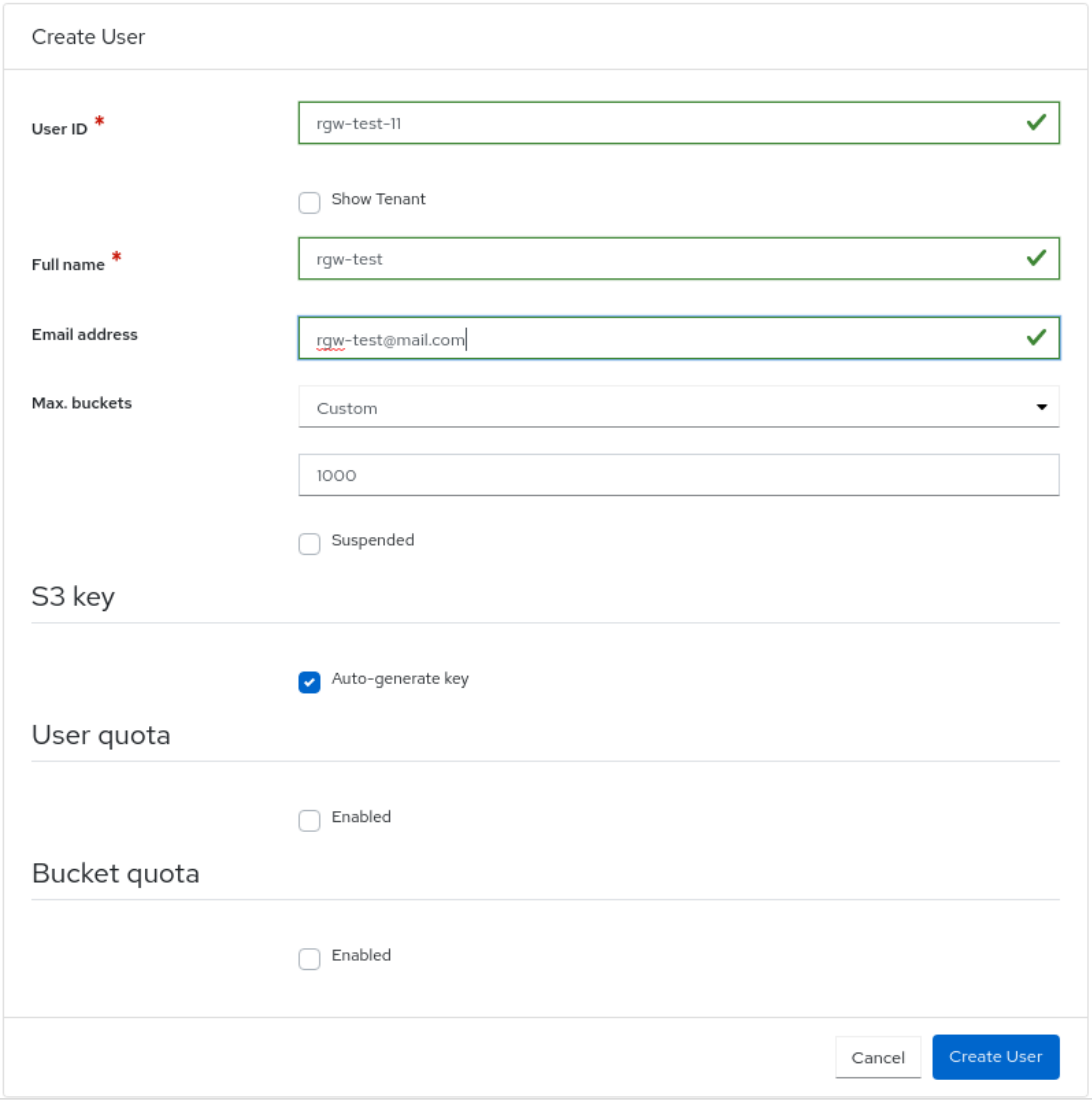


Figure 66: Create user

Creating Ceph Object Gateway subusers

A subuser is associated with a user of the S3 interface. You can create a subuser for a specific Ceph Object Gateway user on the Red Hat Ceph Storage dashboard.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- The Ceph Object Gateway is installed.
- Ceph Object Gateway login credentials are added to the dashboard.
- Ceph Object Gateway user is created.

Procedure

1. From the dashboard navigation, go to **Object › Users**.
2. On the **Users** tab, select a user and click **Edit**.

3. In the **Edit User** form, from the **Subusers** section, click **Create Subuser**.
4. In the **Create Subuser** dialog, enter the **Subuser** and select the appropriate permissions.
5. Select **Auto-generate secret** and click **Create Subuser**.

> **Note:** Selecting **Auto-generate-secret** automatically generates the Ceph Object Gateway secret key.

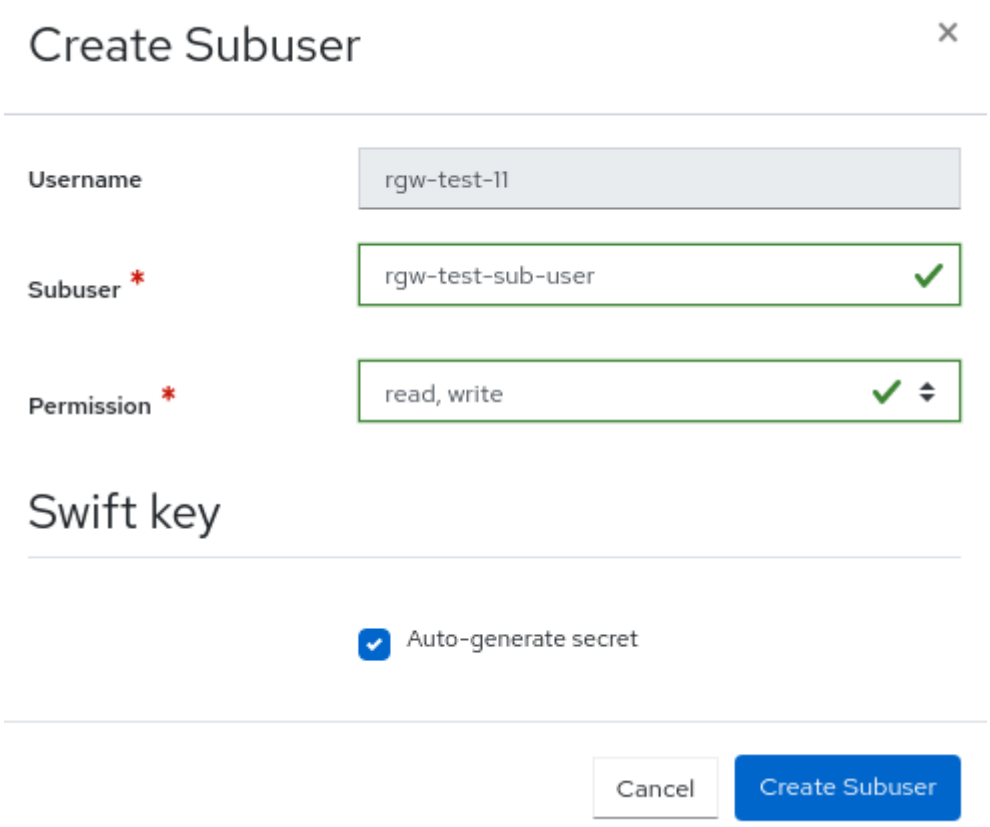


Figure 67: Create Subuser form

6. In the **Edit User** form, save your changes by clicking **Edit user**.

   A notification displays that the Object Gateway user was updated successfully.

Adding roles to Ceph Object Gateway users

You can add a role to a specific Ceph object gateway user on the Red Hat Ceph Storage dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Ceph Object Gateway is installed.
- Ceph Object gateway login credentials are added to the dashboard.
- Ceph Object gateway user is created.

Procedure

1. From the dashboard navigation, go to **Object › Users** and click on the **Roles** tab.
2. Click **Create**.
3. In the **Create Role** form, configure **Role name**, **Path**, and **Assume Role Policy Document**.

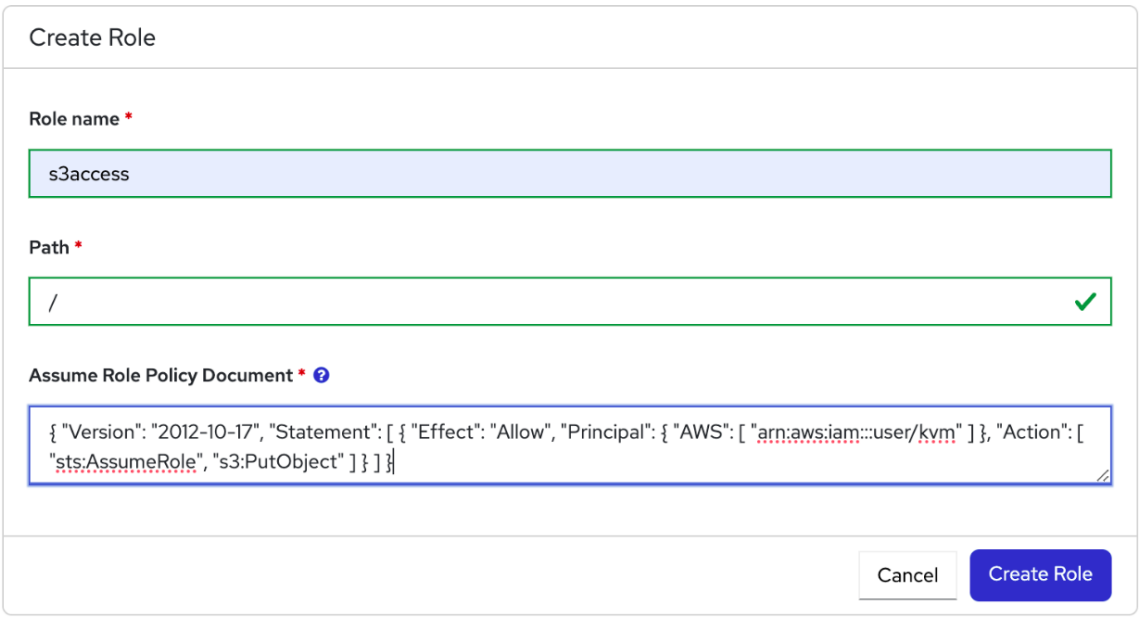


*Figure 68: **Create Role** form*

4. Click **Create Role**.
   A notification displays that the role was created successfully.

Editing Ceph Object Gateway users on the dashboard

Edit Ceph Object Gateway users on the Red Hat Ceph Storage, after the credentials are set-up using the CLI.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- The Ceph Object Gateway is installed.
- Ceph Object Gateway login credentials are added to the dashboard.
- A Ceph Object Gateway user is created.

Procedure

1. From the dashboard navigation, go to **Object › Users**.
2. From the **Users** tab, select the user row and click **Edit**.
3. In the **Edit User** form, edit the required parameters and click **Edit User**.
   A notification displays that the user was updated successfully.

Related information

Manually adding Ceph Object Gateway login credentials to the dashboard

Deleting Ceph Object Gateway users

Delete Ceph Object Gateway users on the Red Hat Ceph Storage, after the credentials are set-up using the CLI.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.

- The Ceph Object Gateway is installed.
- Ceph Object Gateway login credentials are added to the dashboard.
- A Ceph Object Gateway user is created.

Procedure

1. From the dashboard navigation, go to **Object › Users**.
2. Select the **Username** to delete, and click **Delete** from the action drop-down.
3. In the **Delete user** notification, select **Yes, I am sure** and click **Delete User**.
   The user is removed from the **Users** table.

Reference

For more information, see the following:

- "Manually adding Ceph Object Gateway login credentials to the dashboard" on page 733.

Managing user accounts

Accounts provide authenticated identities for accessing Object Gateway resources. A user can operate independently or belong to an account, where it inherits shared ownership, quotas, and centrally managed policies.

A user represents an identity within the Object Gateway. It can exist on its own, or it can be linked to an account for shared resource ownership and unified policy management. When a user operates independently (traditional Ceph Object Gateway user), they own their buckets and objects directly. When linked to an account, the ownership shifts, all buckets and objects created by the user are owned by the account rather than by the individual.

Linking a user to an account provides the following benefits:

Aggregated quotas and usage

Resource consumption is tracked and enforced at the account level.

Shared visibility

Users and roles within the same account can view and interact with account-owned buckets.

This structure helps create consistent identity boundaries and improves multitenancy governance.

Managing user accounts allows administrators to:

- Control who can access Object Gateway resources
- Maintain consistency across all identities under an account
- Track usage and enforce quotas
- Support multitenant application environments

Roles and responsibilities for user accounts

Different types of user identities exist under an account.

"Roles and responsibilities of users linked to an account" on page 739 lists these user types and their permissions.

Table 101: Roles and responsibilities of users linked to an account

User access	Description	Permission
User with root access	user has default permission owned by the account.	Full access to all account resources.
User without root access	user can select the required permissions.	Require explicit managed policies (S3 read-only and S3 full access).

Creating an account

Create an account to define a managed identity boundary and configure quotas, limits, and access controls for its users and buckets.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- You must have administrator access to the Red Hat Ceph Storage Dashboard.
- Ensure the account email address is unique.
- Plan the quotas, resource modes, and identity limits for the account.

Procedure

1. Go to **Object › User management › Accounts › Create**.
2. Enter the required account information.
3. Configure resource modes and limits, including users, roles, groups, and access keys.
4. Optional: Set account-level quota.
5. Optional: Set bucket-level quota.
6. Click **Create Account**.

Result

The account is created and appears in the **Accounts** list. The configured quotas, limits, and access settings are applied to the account, and the account is available for linking with user.

What to do next

After creating an account, the next step is to create an account user.

Editing a user account

Update a user account's attributes, permissions, or access settings.

Before you begin

Before you begin, make sure that you have administrator permission to edit user accounts.

> **Note:** Account-linked users cannot be removed at the account level. To modify their attributes or policies, use the **User** page.

Procedure

1. Go to **Object › User management › Accounts**.
2. In the accounts list, open the ⋮ (three-dot) menu for the account that you want to modify.
3. Select **Edit** and update the required identity details, limits, or managed policies.
4. Save the changes.

Result

The user account is updated with the new configuration. The changes take effect immediately and appear in the **User Details** panel.

Deleting a user account

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Ensure that the user does not own any buckets.
- You must have administrator permission to delete user accounts.

> **Note:** An Account cannot be deleted until all user linked the accounts are deleted.

Procedure

1. Go to **Object > User management > Accounts** .
2. In the accounts list, open the **⋮ (three-dot)** menu for the account that you want to remove.
3. Select **Delete** and confirm the action.

Result

The user account is deleted from the system. All associated credentials are deleted, and the user no longer appears in the Users list.

Linking a user to an account

Associate a user with an account so they inherit the account's quotas, policies, and resource ownership. Account membership is permanent. When a user is linked to an account, all existing buckets owned by that user are automatically reassigned to the account.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- At least one account must exist.
- Determine whether the user requires root-level access within the account
- Ensure the appropriate managed policies are available if the user needs predefined access permissions.

Procedure

1. Go to **Object › Users**.
2. Associate a user with an account.
   Choose one of the following ways to associate a user with an account:
   - Create a new user and link it to an account during creation
   - Edit an existing user and link it to an account.
3. In the **Link Account** field, select the account to associate with the user.
4. Optional: Select **Account Root User** if the user requires administrative privileges to manage other users or roles.
5. Optional: Apply **managed policies** if the user needs predefined full-access or read-only permissions.
6. Save the user configuration.

Result

The user is linked to the selected account. The user now inherits the account's quotas, ownership rules, and applicable managed policies. If configured as a root account user, the user receives elevated management capabilities, and this status is indicated in the Users list.

Link bucket with an account

You can link a bucket to an account so the bucket becomes part of the account's ownership scope and follows the account's quota, visibility, and policy rules.

Before you begin

Before you begin, make sure that you have the following prerequisites in place::

- The user must have sufficient permissions, through a managed policy, to create or modify bucket ownership.

Procedure

1. Choose one of the following methods to associate a bucket with an account:
   - Create a new bucket under a user who belongs to an account, and specify the account user as the bucket owner.
   - Edit an existing bucket created by a normal Ceph Object Gateway user and update its ownership to an account user.
2. In the bucket creation or edit form, select the `account user` from the owner list.
3. Save the bucket configuration.

Result

The bucket is linked to the account and appears in the bucket list with the updated ownership. Account users can view and manage the bucket according to their assigned permissions, and the bucket now follows the account's quota and policy settings.

Managing Ceph Object Gateway buckets

As a storage administrator, the Red Hat Ceph Storage Dashboard allows you to view and manage Ceph Object Gateway buckets.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- The Ceph Object Gateway is installed.
- At least one Ceph Object Gateway user is created.
- Ceph Object Gateway login credentials are added to the dashboard.

Creating Ceph Object Gateway buckets

Use the dashboard to create Ceph Object Gateway buckets on the Red Hat Ceph Storage once the credentials are set up using the CLI.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- The Ceph Object Gateway is installed.
- Ceph Object Gateway login credentials are added to the dashboard.
- Ceph Object Gateway user is created and not suspended.

Procedure

1. From the dashboard navigation, go to **Object › Buckets**.
2. Click **Create**.
   The **Create Bucket** displays.
3. Fill in the **Create Bucket** form.
4. Enter a **Name** for the bucket.
5. Select a **Owner**. The owner is a user that is not suspended.

6. Select a **Placement target**.

   Important: A bucket's placement target cannot be changed after creation.

7. Optional: In the **Locking** section, select **Enabled** to enable locking for the bucket objects.

   Important: Locking can only be enabled while creating a bucket and cannot be changed after creation.

   a. Select the **Mode**, either **Compliance** or **Governance**.
   b. In the **Days** field, select the default retention period that is applied to new objects placed in this bucket.

8. Optional: In the **Security** section, select **Security** to encrypt objects in the bucket.

   Note: For Server-Side Encryption with Key Management Service (SSE-KMS), supported key management implementations include HashiCorp Vault, IBM Guardium Key Lifecycle Manager (GKLM), and Thales CipherTrust Manager.

   a. Set the configuration values for SSE-S3. Click the **Encryption** information icon and then **Click here**.

      Note:
      - When using the `SSE-S3` encryption type, Ceph manages the encryption keys that are stored in the vault.
      - For `SSE-KMS` encryption, supported key management implementations include HashiCorp Vault, IBM Guardium Key Lifecycle Manager (GKLM), and Thales CipherTrust Manager.

   b. In the **Update RGW Encryption Configurations** dialog, ensure that **SSE-S3** is selected as the **Encryption Type**.
   c. Fill in the other required information.
   d. Click **Submit**.

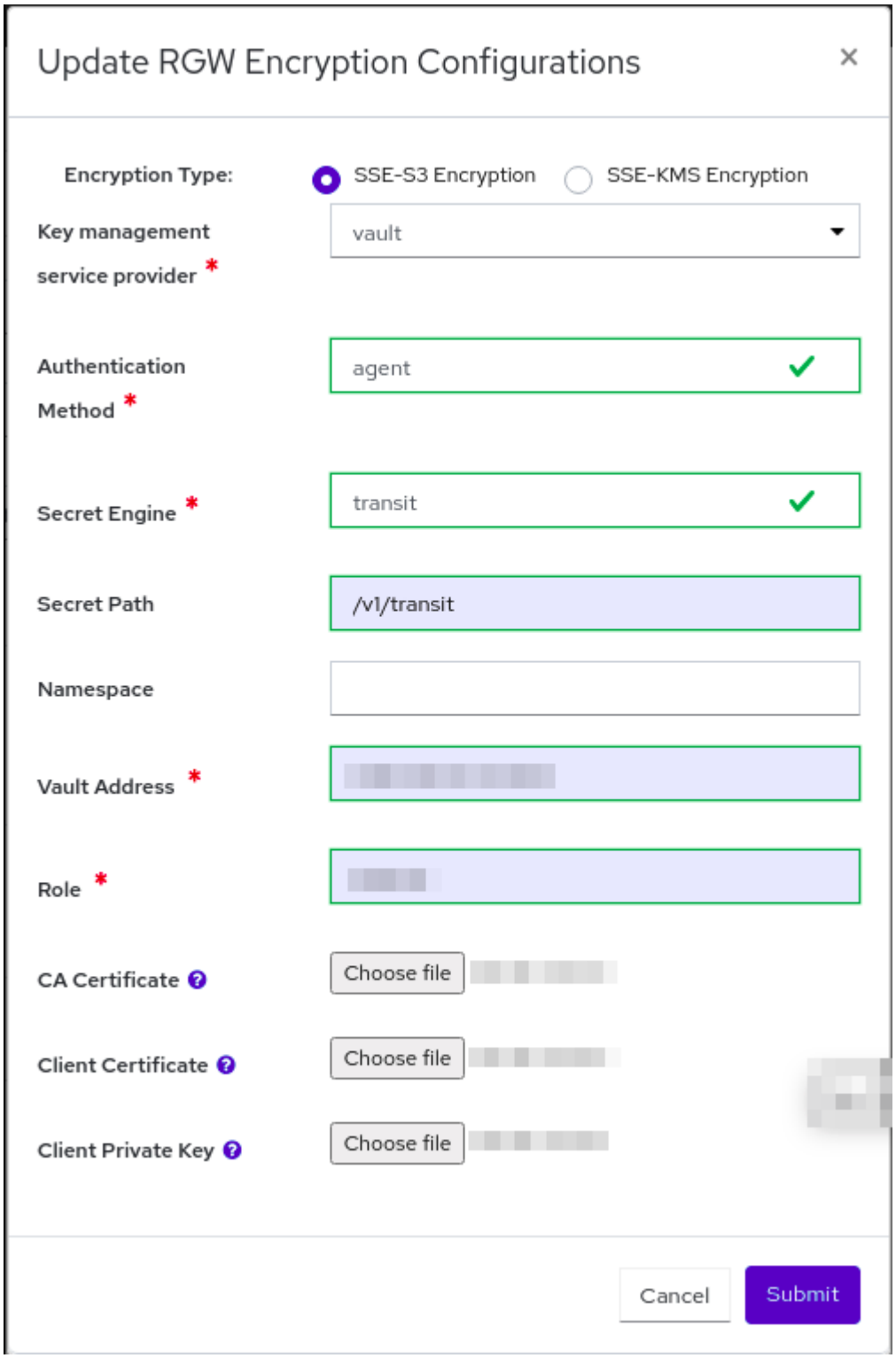


Figure 69: Encrypt objects in the bucket

9. In the **Tags** section, click **Add** to add bucket tags.
   These tags are equivalent to the S3 PutBucketTagging. Enter the tag **Key** and tag **Value** to categorize your storage buckets.

10. Set the bucket policies in the **Policies** section.

    a. Enter the **Bucket policy**.
    Use the **Policy generator** or **Policy examples** buttons to help create the bucket policies, as needed. Enter or modify the policy in JSON format.

    Use the following links from within the form to help create your bucket policy. These links open a new tab in your browser.

- **Policy generator** is an external tool from AWS to generate a bucket policy. For more information, see AWS Policy Generator.

  > **Note:** You can use the policy generator with the **S3 Bucket Policy** type as a guideline for building your Ceph Object Gateway bucket policies.

- **Policy examples** takes you to AWS documentation with examples of bucket policies.

For more information about managing bucket policies through the dashboard, see "Managing Ceph Object Gateway bucket policies" on page 753.

b. Set the Access Control Lists (ACL) grantee and permission information.

Table 102: ACL user operations

Permission	Bucket	Object
`READ`	Grantee can list the objects in the bucket.	Grantee can read the object.
`WRITE`	Grantee can write or delete objects in the bucket.	N/A
`FULL_CONTROL`	Grantee has full permissions for object in the bucket.	Grantee can read or write to the object ACL.

11. Click **Create Bucket**.
    A notification displays that the bucket was created successfully.

Editing Ceph Object Gateway buckets

You can edit Ceph Object Gateway buckets on the Red Hat Ceph Storage once the credentials are set-up using the CLI.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- The Ceph Object Gateway is installed.
- Ceph Object Gateway login credentials are added to the dashboard.
- Ceph Object Gateway user is created and not suspended.
- A Ceph Object Gateway bucket created.

Procedure

1. From the dashboard navigation, go to **Object › Buckets**.
2. Select the bucket row that needs to be updated, and click **Edit**.
   The **Edit Bucket** displays.
3. Optional: Enable **Versioning** to enable a versioning state for all objects in an existing bucket.
   - To enable versioning, you must be the owner of the bucket.
   - If `Locking` is enabled during bucket creation, you cannot disable the versioning.
   - All objects added to the bucket will receive a unique version ID.
   - If the versioning state has not been set on a bucket, then the bucket will not have a versioning state.
4. Optional: Select **Delete enabled** for Multi-Factor Authentication.
   Multi-Factor Authentication (MFA) ensures that users need to use a one-time password (OTP) when removing objects on certain buckets. Enter a value for **Token Serial Number** and **Token PIN**.

> **Note:** The buckets must be configured with versioning and MFA enabled which can be done through the S3 API.

5. Optional: As needed, update the **Tags** and **Policies**.
   Updating the **Policies** includes updating the **Bucket policy** and Access Control Lists (ACL) grantee and permission information. For more information, see steps “9” on page 744 and “10” on page 744 in “Creating Ceph Object Gateway buckets” on page 742.
6. Click **Edit Bucket** to save the changes.
   A notification displays that the bucket was updated successfully.

Deleting Ceph Object Gateway buckets

You can delete Ceph Object Gateway buckets on the Red Hat Ceph Storage once the credentials are set-up using the CLI.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- The Ceph Object Gateway is installed.
- Ceph Object Gateway login credentials are added to the dashboard.
- Ceph Object Gateway user is created and not suspended.
- A Ceph Object Gateway bucket created.
- All objects associated with the bucket are deleted.

Procedure

1. From the dashboard navigation, go to **Object › Buckets**.
2. Select the bucket to be deleted, and click **Delete** from the action drop-down.
3. In the **Delete bucket** dialog, enter the bucket name, select **Yes, I am sure**, and click **Delete bucket**.
   The bucket is deleted from the **Buckets** table.

> **Note:** When the bucket has associated objects, the **Delete** button is disabled. All objects must be removed before deletion.

Managing Ceph Object Gateway notification destinations

Learn how notification destinations in Red Hat Ceph Storage Object Gateway define where S3 bucket event messages are delivered.

A notification destination represents an external endpoint that receives S3 bucket event messages. These destinations define *how* and *where* events are sent—for example to HTTPS endpoints, Kafka topics, AMQP queues, or custom services. Notification destinations are created once and can be reused across multiple bucket notifications.

Why notification destinations are important

Typical consumers include:

- Stream processing services
- Data ingestion pipelines
- Automation tools
- Audit and compliance systems
- Analytics applications

Key components of a notification destination

A notification destination includes the following components:

Destination name

A unique identifier

Endpoint type

HTTPS, Kafka, AMQP, or other supported mechanisms

Endpoint details

URL, port, credentials, TLS settings, timeout, and retry configuration

Delivery format

How event data is packaged and transmitted

Associations

Bucket notifications that reference the destination

How notification destinations fit into the event architecture

The following sequence explains how destinations participate in event routing:

1. The administrator creates one or more notification destinations.
2. Bucket notifications reference these destinations.
3. The Object Gateway monitors bucket events.
4. When an event occurs, the gateway identifies the mapped destination.
5. The event is delivered to the configured endpoint.

Use cases

Notification destinations support scenarios such as:

- Sending new-object events to Kafka
- Triggering workflows through HTTPS webhooks
- Forwarding delete events to audit systems
- Integrating with AMQP or message queues
- Driving real-time data ingestion

Creating a notification destination

Create a new notification destination to receive S3 event messages.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- You have administrative permissions.
- You have the endpoint information, including URL, protocol, and authentication details.

Procedure

1. Go to **Object Gateway > Bucket notifications** and select the **Destinations** tab.
2. Click **Create destination**.
3. Enter the destination details, including the name, endpoint type, and required configuration fields.
4. Click **Save**.
5. Verify that the destination appears in the list.

Result

A new notification destination is created and available for bucket notification mapping.

Editing a notification destination

Modify the configuration of an existing notification destination.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- The destination is not used by critical or production bucket notifications.
- You have the updated endpoint or configuration details.

Procedure

1. Go to **Object Gateway > Bucket notifications** and select the **Destinations** tab.
2. Select the destination and click **Edit**.
3. Update the required fields.
4. Click **Save**.
5. Verify that the updated details appear in the list.

Result

The notification destination is updated..

Deleting a notification destination

Delete a notification destination that is no longer required.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- No active bucket notifications reference the destination.
- You have confirmed that deleting the destination will not impact downstream workflows.

Procedure

1. Go to **Object Gateway > Bucket notifications** and select the **Destinations** tab.
2. Click the **Destinations** tab.
3. Select the destination you want to delete.
4. Click **Delete**.
5. Confirm the deletion when prompted.
6. Verify that the destination is removed from the list.

Result

The selected notification destination is deleted.

Listing notification destinations

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- The Dashboard service is running.
- You are signed in with a user ID that has permission to view notification destinations.

Procedure

1. In the Dashboard navigation pane, select **Object Gateway > Bucket notifications** and select the **Destinations** tab
2. Review the list of configured notification destinations.
3. Optional. Use the search box or filters to refine the list.

Result

All available notification destinations are displayed..

Managing bucket notifications in Ceph Object Gateway

Learn how bucket notifications map S3 events to configured topics to enable event-driven workflows.

Bucket notifications define which S3 events generate messages and where those messages should be delivered. Each rule includes the bucket, event types, and a target topic. When the specified events occur, the Object Gateway sends structured event records to the mapped topics.Bucket notifications provide the dynamic part of the notification framework: topics define the destination, and notifications define the triggering conditions.

Bucket notifications have the following characteristics:

- Multiple notifications per bucket: Each bucket can have multiple event mappings.
- Reusability: A topic can be referenced by multiple notifications.
- Granularity: Notifications can be based on specific events or object paths.
- Dependency: A topic must exist before creating a bucket notification.

How bucket notifications work

The following workflow describes how bucket notifications function:

1. An administrator configures the notification in the Dashboard.
2. The notification references a previously created topic.
3. The Object Gateway tracks events on the bucket.
4. When an event occurs, the gateway generates an event record.
5. The gateway delivers the event to the topic endpoint.

Use cases

Bucket notifications support several real-world use cases, including:

- Triggering pipelines when new objects are uploaded.
- Synchronizing metadata to external systems.
- Automating cleanup or archival workflows
- Generating alerts when objects are deleted
- ntegrating bucket events with third-party analytics tools

Creating a bucket notification

Create a bucket notification rule that maps S3 events to a notification destination.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- At least one notification destination exists.
- You know the bucket name and the required event types.

Procedure

1. In the **Notifications** tab, click **Create notification**.

2. Enter the bucket name.
3. Select the event types to monitor.
4. Select the target notification destination.
5. Click **Save**.

Result

A new bucket notification is created and associated with the selected destination.

Editing a bucket notification

Modify an existing bucket notification rule.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- You have the updated event types or destination information.

Procedure

1. Open the **Notifications** tab.
2. Select the notification and click **Edit**.
3. Update the required fields.
4. Click **Save**.
5. Verify the updates in the table.

Result

The bucket notification is updated with the new configuration..

Deleting a bucket notification

Delete a bucket notification that is no longer required.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- The notification is not required by production services.

Procedure

1. Go to the **Notifications** tab.
2. Select the bucket notification.
3. Click **Delete**.
4. Confirm the deletion.

Result

The selected bucket notification is deleted..

Listing bucket notifications

View the list of bucket notifications configured for an Object Gateway bucket.

Before you begin

Before you begin, make sure that you have permission to view bucket notifications.

Procedure

1. Select**Object Gateway › Bucket notifications**.
2. Click the **Notifications** tab.
3. Review the table of existing bucket notifications.
4. Optional: Apply filters or search.

Result

All configured bucket notifications are displayed.

Viewing Performance Metrics for Multi-Site Sync Deltas in Ceph Object Gateway

You can view the replication performance of each shard between zones in the Grafana dashboard.

Before you begin

- A running Red Hat Ceph Storage cluster.
- ceph-exporter, Grafana, and Prometheus are installed.
- Ceph Object Gateway is installed.
- Object gateway login credentials are added to the dashboard.
- Object gateway users and buckets are created and not suspended.

Procedure

1. Log in to the Grafana URL.

   ```
   https://<ip-where-grafana-deployed>:3000
   ```

   Example

   ```
   https://<ip-where-grafana-deployed>:3000
   ```

2. Go to the **Dashboard** tab and search for **RGW Sync Overview**.
3. To view the performance metrics of the multi-site sync deltas, see the **Replication(Time) Delta per shard** panel

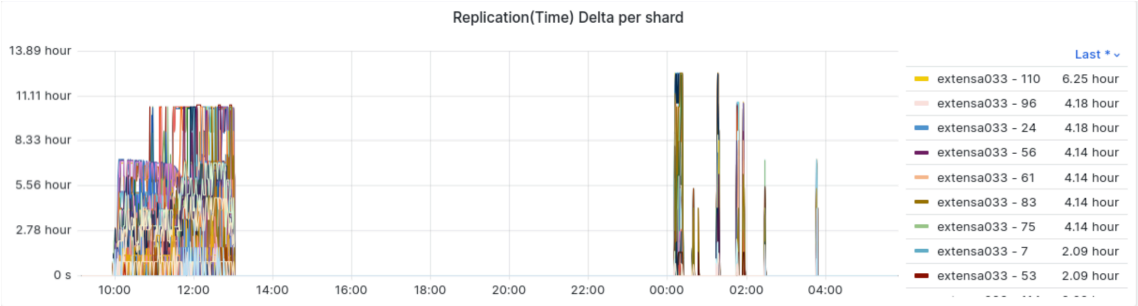


Figure 70: ***Replication (Time) Delta per shard***

The panel displays the time taken for each shard in the cluster to replicate.

Viewing Ceph Object Gateway per-user and per-bucket performance counters

You can view the Ceph Object Gateway performance counters per user per bucket in the Grafana dashboard.

Procedure

1. Log in to the Grafana URL.

```
https://DASHBOARD_URL:3000
```

Example

```
https://dashboard_url:3000
```

2. Go to the **Dashboard** tab and search for **RGW S3 Analytics**.
3. To view per-bucket Ceph Object gateway operations, select the 'Bucket' window:

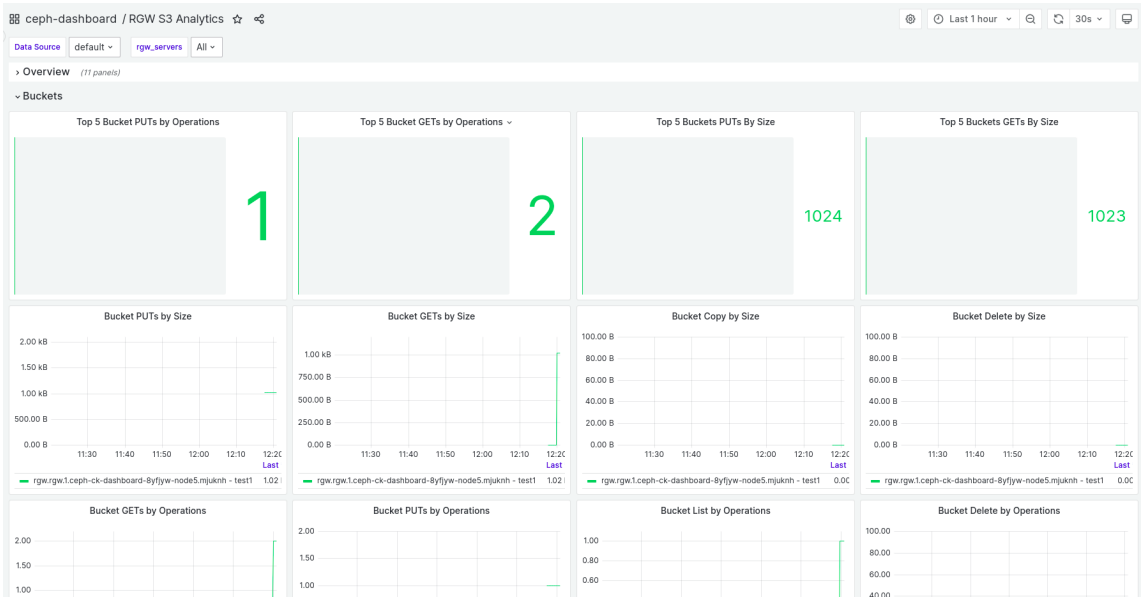


*Figure 71: **Bucket operations counter** form*

4. To view per-user Ceph Object gateway operations, select the 'User' window:

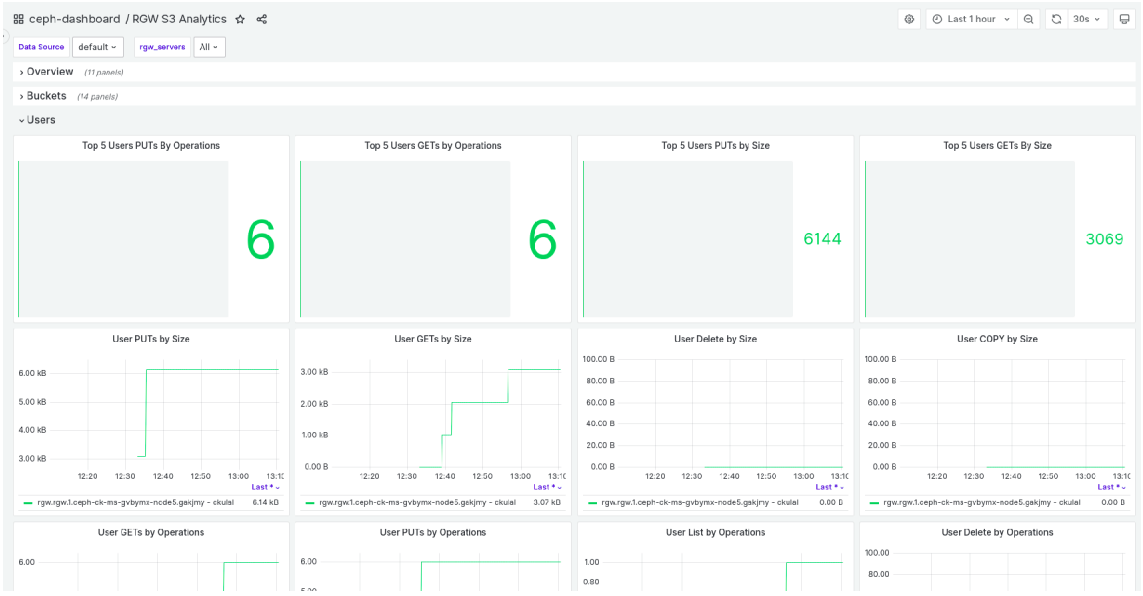


*Figure 72: **User operations counter** form*

> **Note:** The output of per-bucket and per-user `get operation count` command increases by two for each `get` operation that is run from client: s3cmd. This behaviour is a known issue.

Managing Ceph Object Gateway bucket policies

As a storage administrator, the Red Hat Ceph Storage Dashboard allows you to view and manage Ceph Object Gateway bucket policies.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- The Ceph Object Gateway is installed.
- At least one Ceph Object Gateway user is created.
- Ceph Object Gateway login credentials are added to the dashboard.
- At least one Ceph Object Gateway bucket. For information about creating a bucket, see “Creating Ceph Object Gateway buckets” on page 742.

Creating and editing Ceph Object Gateway bucket policies

Use the dashboard to create and edit a Ceph Object Gateway bucket policy on the Red Hat Ceph Storage dashboard.

Before you begin

Before creating or editing Ceph Object Gateway bucket policies, ensure the following are available:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- The Ceph Object Gateway is installed.
- At least one Ceph Object Gateway user is created.
- Ceph Object Gateway login credentials are added to the dashboard.
- At least one Ceph Object Gateway bucket. For information about creating a bucket, see “Creating Ceph Object Gateway buckets” on page 742.

Procedure

1. From the dashboard, go to **Object › Buckets**.
2. Create or modify a bucket policy for an existing bucket.

   > **Note:** To create a bucket policy during bucket creation, click **Create** and fill in the bucket policy information in the **Policies** section of the **Create Bucket** form.

   Select the bucket for which the bucket policy will be created or modified, and then click **Edit**.
3. In the **Edit Bucket** form, go to **Policies**.
4. Enter or modify the policy in JSON format.
   Use the following links from within the form to help create your bucket policy. These links open a new tab in your browser.
   - **Policy generator** is an external tool from AWS to generate a bucket policy. For more information, see AWS Policy Generator.

     > **Note:** You can use the policy generator with the **S3 Bucket Policy** type as a guideline for building your Ceph Object Gateway bucket policies.

   - **Policy examples** takes you to AWS documentation with examples of bucket policies.
5. To save the bucket policy, click **Edit Bucket**.

> **Note:** When creating a bucket policy during an initial bucket creation, click **Create Bucket**.

When the bucket policy is saved, the `Updated Object Gateway bucket `bucketname`` notification is displayed.

Deleting Ceph Object Gateway bucket policies

Use the dashboard to delete Ceph Object Gateway bucket policies on the Red Hat Ceph Storage dashboard.

Before you begin

Before deleting Ceph Object Gateway bucket policies, ensure the following are available:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- The Ceph Object Gateway is installed.
- At least one Ceph Object Gateway user is created.
- Ceph Object Gateway login credentials are added to the dashboard.
- At least one Ceph Object Gateway bucket. For information about creating a bucket, see "Creating Ceph Object Gateway buckets" on page 742.

Procedure

1. From the dashboard, go to **Object › Buckets**.
2. Select the bucket for which the bucket policy will be deleted, and then click **Edit**.
3. In the **Edit Bucket** form, go to **Policies**.
4. Click **Clear**.
5. To complete the bucket policy deletion, click **Edit Bucket**.
   When the bucket policy is deleted, the `Updated Object Gateway bucket `bucketname`` notification is displayed.

Managing S3 bucket lifecycle policies

As a storage administrator, the Red Hat Ceph Storage Dashboard allows you to view and manage Ceph Object Gateway bucket lifecycle policies.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- At least one Ceph Object Gateway user is created.
- Ceph Object Gateway login credentials are added to the dashboard.
- At least one Ceph Object Gateway bucket. For information about creating a bucket, see "Creating Ceph Object Gateway buckets" on page 742.

Applying and viewing bucket lifecycle policies

Use the dashboard to apply and manage a Ceph Object Gateway bucket lifecycle policy on the Red Hat Ceph Storage dashboard.

Procedure

1. From the dashboard, go to **Object › Buckets**.
2. Select the bucket for which the lifecycle policy needs to be applied and click **Edit**

> **Note:** Bucket lifecycle policy cannot be applied during the creation of the bucket. They can be applied only after a bucket is created.

3. In the **Edit Bucket** form, go to **Policies** and apply the lifecycle rule in the **Lifecycle** field in JSON format.
4. To save the bucket lifecycle policy, click **Edit Bucket**.

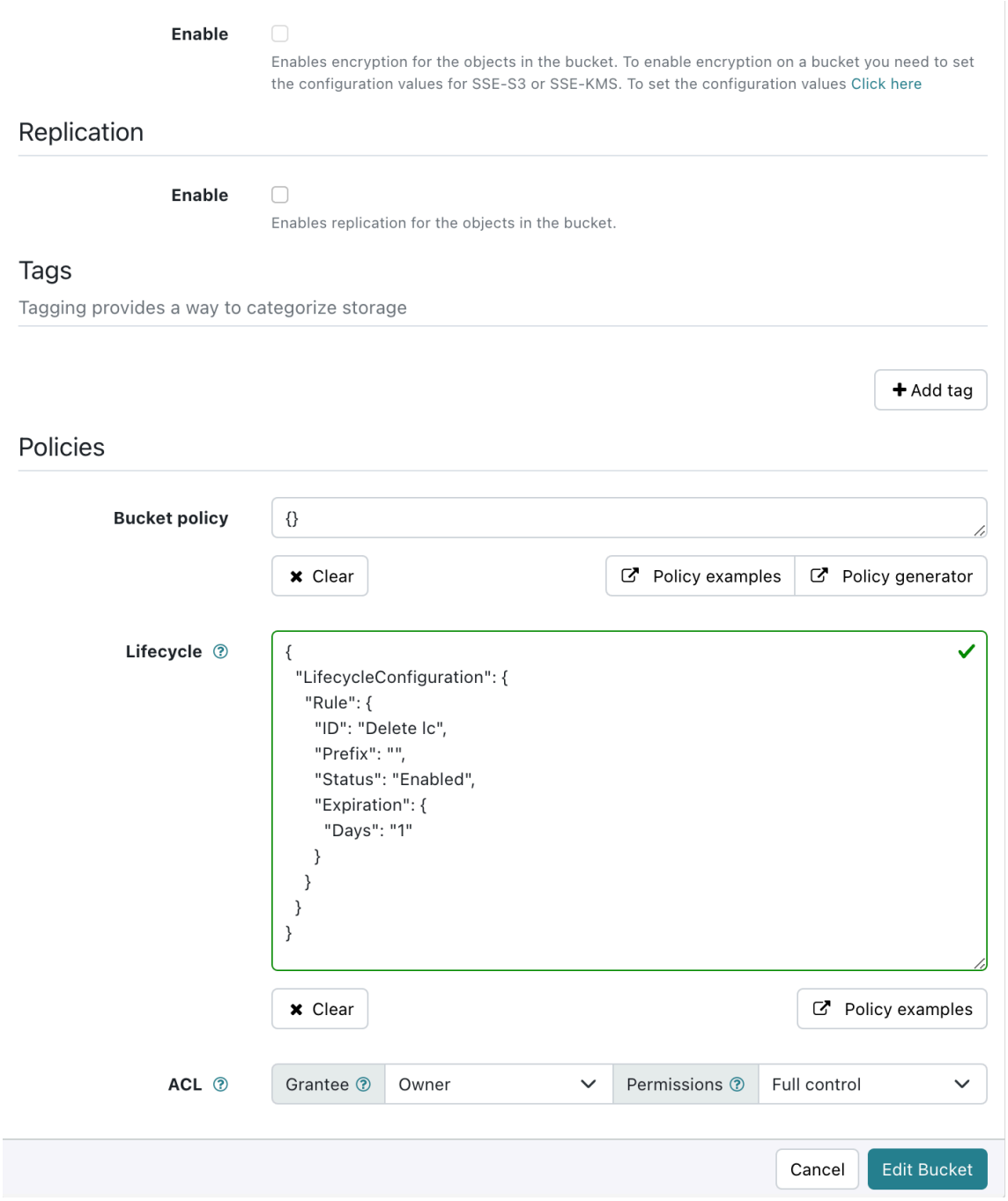


Figure 73: Apply bucket LC rule

5. After the bucket lifecycle policy is applied, it can viewed in the bucket listing screen by expanding the relevant bucket entry.

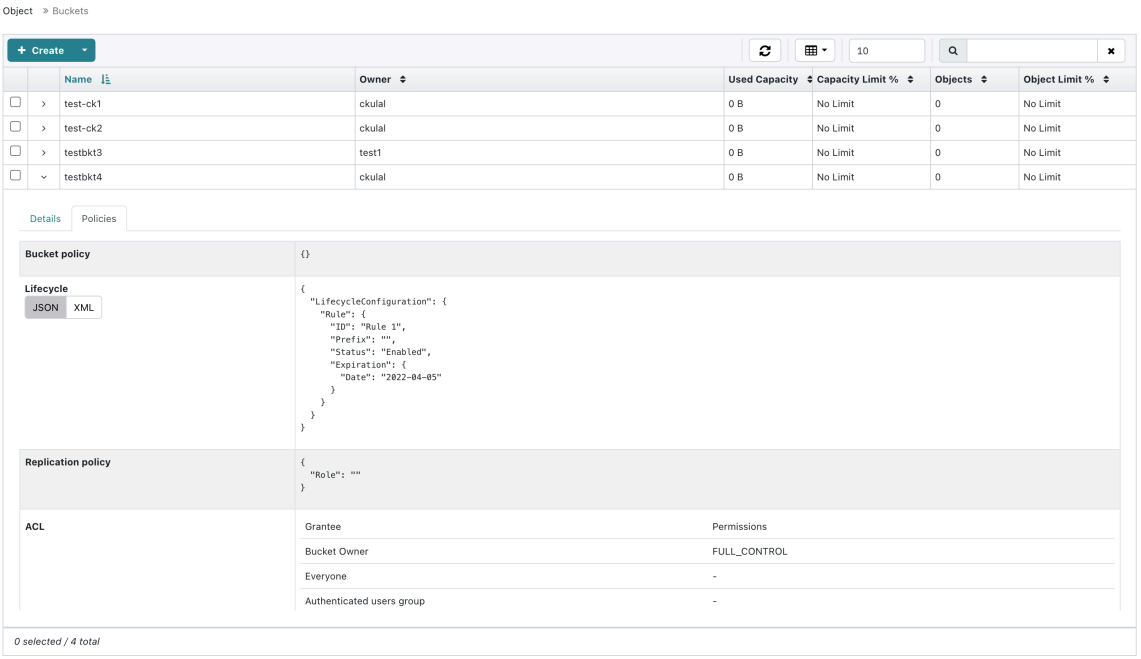


Figure 74: View bucket LC rule

Deleting bucket lifecycle policies

Use the dashboard to delete Ceph Object Gateway bucket lifecycle policies on the Red Hat Ceph Storage dashboard.

Procedure

1. From the dashboard, go to **Object › Buckets**.
2. Select the bucket for which the lifecycle policy needs to be deleted, and click **Edit**.
3. In the **Edit Bucket** form, go to **Policies**.
4. Click **Clear**.
5. To complete the bucket policy deletion, click **Edit Bucket**.

Creating a tiering storage class

Use the Dashboard to create a tiering storage class for local storage, Cloud S3, or Cloud S3 Glacier targets.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- You have administrator privileges to access the Ceph Dashboard.
- The Object Gateway service is running.
- You have the access key and secret key for the target storage system.

Procedure

1. Go to **Dashboard > Object >Tiering**.
2. Click **Create**.
3. In the **Create Tiering Storage** dialog, select the storage type: `Local`,`Cloud S3`, or `Cloud S3 Glacier`.
4. Enter the required details:
   Name

   Unique name for the storage class.

Target Region

Region of the storage endpoint.

Target Endpoint

URL endpoint of the remote cloud object storage service where Ceph transitions objects. Use this parameter to define the destination endpoint for data movement to an external S3-compatible cloud, such as AWS S3 or S3 Glacier. The value must include the protocol, host, and optional port number.

Access key and Secret key

Credentials for the target storage.

Target Path

Target path and bucket name in the target storage.

5. Specify **Read Through Restore Days**.
6. Select one of the restore options:
   - **Allow read through (default restore behavior)**
   - **Head object (Stub file)**
7. Select the **Restore Storage Class** from the dropdown list.
8. Select the **Glacier Restore Type**.
9. Expand the **Advanced** pane and configure the optional parameters:
   Multipart Sync Threshold

   object size that triggers multipart upload.

   Multipart Minimum Part Size

   Minimum size of each part.

   Placement Target

   Placement configuration that defines where transitioned objects are stored within a zonegroup. The placement target maps a storage class to a specific storage tier or cloud destination, ensuring objects are directed to the correct backend during transition.

   ACL Mapping

   Defines how access control list (ACL) entries from the source Ceph zone are translated to the destination cloud service. Use this parameter to maintain user or group permissions when objects transition to a cloud tier.
10. Click **Create** to save the tiering storage class.

Result

The new tiering storage class appears under **Dashboard > Object > Tiering**.

Managing buckets of a multi-site object configuration

As a storage administrator, you can edit buckets of one zone in another zone on the Red Hat Ceph Storage Dashboard. However, you can delete buckets of secondary sites in the primary site. You cannot delete the buckets of primary zones of primary sites in other sites. For example, If the buckets are created in a zone in the secondary site, you can edit and delete those buckets in the primary zone in the primary site.

For more information about multi-site and Ceph Object Gateway buckets, see:

- Multi-site configuration and administration
- “Manually adding Ceph Object Gateway login credentials to the dashboard” on page 733
- “Managing Ceph Object Gateway buckets” on page 742
- “User roles and permissions” on page 688

Prerequisites

Before you begin, make sure that you have the following prerequisites in place:

- At least one running Red Hat Ceph Storage cluster deployed on both the sites.
- Dashboard is installed.
- The multi-site Ceph Object Gateway is configured on the primary and secondary sites.
- Ceph Object Gateway login credentials of the primary and secondary sites are added to the dashboard.
- Ceph Object Gateway users are created on the primary site.
- Ceph Object Gateway buckets are created on the primary site.
- At least `rgw-manager` level of access on the Ceph dashboard.

Monitoring buckets of a multi-site object

Monitor the multi-site sync status of a bucket on the dashboard. You can view the source zones and sync status from the **Multi-site sync status** card on **Object › Overview** page.

The multi-site sync status is divided into two sections.

Primary Source Zone

Displays the default realm, zonegroup, and the zone the Ceph Object Gateway is connected to.

Source Zones

View both the metadata sync status and data sync information progress. When you click the status, a breakdown of the shard syncing is displayed.
The sync status shows the **Last Synced** time stamp with the relative time of the last sync occurrence in relation to the current time. When the sync is complete, this shows as **Up to Date**.

When a sync is not caught up the `status` shows as `Syncing`. However, the `Last sync` shows the number of days the sync is not caught up. By clicking `Syncing`, it displays the details about shards which are not synced.

Editing buckets of a multi-site Ceph Object Gateway configuration

You can edit and update the details of the buckets of one zone in another zone on the Red Hat Ceph Storage Dashboard in a multi-site Ceph Object Gateway configuration. You can edit the owner, versioning, multi-factor authentication and locking features of the buckets with this feature of the dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Minimum of one running Red Hat Ceph Storage cluster deployed on both sites.
- Dashboard is installed.
- The multi-site Ceph Object Gateway is configured on the primary and secondary sites.
- Ceph Object Gateway login credentials of the primary and secondary sites are added to the dashboard.
- Ceph Object Gateway users are created on the primary site.
- Ceph Object Gateway buckets are created on the primary site.
- Minimum of `rgw-manager` level of access on the Ceph dashboard.

Procedure

1. From the dashboard navigation of the secondary site, go to **Object › Buckets**.
   From the secondary landing page, view the Object Gateway buckets that were created for Object Gateway users on the primary site.
2. Click the row of the bucket that you want to edit and click **Edit**.
3. From the **Edit bucket** form, edit the required parameters.
4. Click **Edit Bucket** to save the changes.

A notification displays showing that the bucket was updated successfully.

Deleting buckets of a multi-site Ceph Object Gateway configuration

You can delete buckets of secondary sites in primary sites on the Red Hat Ceph Storage Dashboard in a multi-site Ceph Object Gateway configuration.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Minimum of one running Red Hat Ceph Storage cluster deployed on both sites.
- Dashboard is installed.
- The multi-site Ceph Object Gateway is configured on the primary and secondary sites.
- Ceph Object Gateway login credentials of the primary and secondary sites are added to the dashboard.
- Ceph Object Gateway users are created on the primary site.
- Ceph Object Gateway buckets are created on the primary site.
- Minimum of `rgw-manager` level of access on the Ceph dashboard.

About this task

Important: Do not delete buckets of primary site from secondary sites.

Procedure

1. From the dashboard navigation of the primary site, go to **Object › Buckets**.
2. Select the bucket of the secondary site to be deleted, and click **Delete** from the action drop-down.
3. In the **Delete Bucket** notification, select **Yes, I am sure** and click **Delete bucket**.
   The bucket is deleted from the **Buckets** table.

Monitoring multi-site Ceph Object Gateway configuration

The Red Hat Ceph Storage dashboard supports monitoring the users and buckets of one zone in another zone in a multi-site Ceph Object Gateway configuration. For example, if the users and buckets are created in a zone in the primary site, you can monitor those users and buckets in the secondary zone in the secondary site.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Minimum of one running Red Hat Ceph Storage cluster deployed on both sites.
- Dashboard is installed.
- The multi-site Ceph Object Gateway is configured on the primary and secondary sites.
- Ceph Object Gateway login credentials of the primary and secondary sites are added to the dashboard.
- Ceph Object Gateway users are created on the primary site.
- Ceph Object Gateway buckets are created on the primary site.

Procedure

1. From the dashboard navigation of the secondary site, go to **Object › Buckets**.
2. View the Object Gateway buckets on the secondary landing page that were created for the Object Gateway users on the primary site.

Configuring a multi-site

Configure multi-site on the Red Hat Ceph Storage dashboard.

Before you begin

- A Red Hat Ceph Storage cluster at both the sites.
- At least one Ceph Object Gateway service installed at both the sites.

Procedure

1. Enable the Ceph Object Gateway module on both the sites to enable import/export feature.
   a. From the dashboard navigation of the primary site, go to **Object › Multi-site**.
   b. In the **Information** box, click **Enable RGW Module**.
2. On the primary site dashboard, create a default realm, zonegroup, and zone.
   a. From the **Object › Multi-site**, click **Create Realm**.
   b. In the **Create Realm** dialog, provide a realm name, and select **Default**.
   c. Click **Create Realm**.
   d. Click **Create Zone Group** from the action drop-down.
   e. In the **Create Zone Group** dialog, provide a zone group name, the Ceph Object Gateway endpoints, and select **Default**.
   f. Click **Create Zonegroup**.
   g. Click **Create Zone** from the action drop-down.
   h. In the **Create Zone** dialog, provide a **Zone Name**, select **Default**, provide the Ceph Object Gateway endpoints of the primary site. For the user, provide the access and secret key of the user with system privileges.

      > **Note:** While creating a zone, provide the access key and secret key of the dashboard default user, `dashboard`.

3. After the zone is created, a warning is emitted to restart the Ceph Object Gateway Service.
   a. From the **Administration › Services**.
   b. Select the Ceph Object Gateway (`rgw`) service row and expand to see the service details.
   c. From the **Daemons** table, select the host and click **Restart** from the action drop-down.
4. When you go to **Object › Gateways**, you get an error that `The Object Gateway Service is not configured`. This bug is a BZ#2231072.
   a. As a workaround, set the Ceph Object Gateway credentials on the command-line interface.

      ```
      ceph dashboard set-rgw-credentials
      RGW credentials configured
      ```

   b. Go to **Object › Gateways**, to verify that you are able to access the Ceph Object Gateway on the dashboard.
5. Create a replication user on the primary site. You can use the following two options:
   - Create user using the CLI.

     ```
     [ceph: root@host01 /]# radosgw-admin user create --uid="uid" --display-
     name="displayname" --system
     ```

   - Create user from the dashboard and modify the user from the CLI.

     ```
     [ceph: root@host01 /]# radosgw-admin user modify --uid="uid" --system
     ```

6. From, **Object › Gateways**, on the **Users** table, select the user row created in the previous step.
    a. Expand the user row.
    b. Copy the **Access Key** and **Secret Key**.
    c. From, **Object › Multi-site**, select the zone.
    d. Click the edit icon and paste the access key and secret key that was copied earlier.
    e. Click **Edit Zone**.
7. Click **Export**.
    a. From the **Export Multi-site Realm Token** dialog box, copy the token.
    b. Close the dialog box.
8. On the secondary site, go to **Object › Multi-site**.
9. Click **Import** to import token from the primary zone.
10. In the **Import Multi-site Token** dialog box, paste the token that was copied earlier, and provide a secondary zone name.
    a. Under **Service** details, select the placement and the port where the new Ceph Object Gateway service is going to be created.
    b. Click **Import**.
11. After the token is imported, you get a warning to restart the Ceph Object Gateway Service.
    a. Go to **Administration › Services**.
    b. Select the Ceph Object Gateway (`rgw`) service row.
    c. From the **Daemons** table, select the host. Verify that the Ceph Object Gateway is configured on the secondary site.
    d. Click **Restart** from the action drop-down.
12. You need to wait for sometime till the users are synced to the secondary site. Verify that the sync is complete with the following commands.

```
[ceph: root@host01 /]# radosgw-admin sync status
[ceph: root@host01 /]# radosgw-admin user list
```

13. When you go to **Object › Gateways**, you get an error that `The Object Gateway Service is not configured`. This bug is a BZ#2231072.
    a. As a workaround, set the Ceph Object Gateway credentials on the command-line interface.

```
ceph dashboard set-rgw-credentials
RGW credentials configured
```

    b. Go to **Object › Gateways**, to verify that you are able to access the Ceph Object Gateway on the dashboard.
14. On the primary site, under an error displays on the **Multi Site Sync Status** panel of the **Object › Overview** page. This is because on the secondary zone, you can see that the endpoints are of the hostname and not the IP address. This bug is a known issue while configuring multi-site. See BZ#2242994.
    a. As a workaround, in the secondary site, go to **Object › Multi-site** and select the secondary zone.
    b. Edit the endpoints to reflect the IP address.
    c. Click **Edit Zone**.
15. On the primary site and secondary site dashboard, you can see the status displays on the **Multi Site Sync Status** panel of the **Object › Overview** page.

What to do next

To verify, create a user on the primary site. You see that the user syncs to the secondary site.

Managing Ceph File Systems (CephFS)

As a storage administrator, you can create, edit, delete, and manage accesses of file systems on the Red Hat Ceph Storage Dashboard.

Configuring CephFS volumes

As a storage administrator, you can configure Ceph File System (CephFS) volumes on the Red Hat Ceph Storage dashboard.

Creating CephFS volumes

You can create Ceph File System (CephFS) volumes on the Red Hat Ceph Storage Dashboard.

Before you begin

Before you begin, make sure that you have a working Red Hat Ceph Storage cluster with MDS deployed.

Procedure

1. From the dashboard navigation, go to **File › File Systems**.
2. Click **Create**.
3. In the **Create Volume** window, set the following parameters:
   a. **Name**: Set the name of the volume.
   b. **Placement (Optional)**: Select the placement of the volume. You can set it as either *Hosts* or *Label*.
   c. **Hosts/Label (Optional)**: If placement is selected as *Hosts* in the previous option, then select the appropriate host from the list. If placement is selected as *Label* in the previous option, then enter the label.

> **Important:** To identify the label that has been created for the hosts while using it as a placement, you can run the following command from the CLI:
>
> ```
> [ceph: root@ceph-hk-ds-uoayxl-node1-installer /]# ceph orch host ls
>
> HOST ADDR LABELS STATUS
>
> host01 10.0.210.182 _admin,installer,mon,mgr
> host02 10.0.96.72 mon,mgr
> host03 10.0.99.37 mon,mds
> host04 10.0.99.244 osd,mds
> host05 10.0.98.118 osd,mds
> host06 10.0.98.66 osd,mds
> host07 10.0.98.23 mds
> 7 hosts in cluster
> ```

4. Click **Create Volume**.
   A notification displays that the volume was created successfully.

Editing CephFS volumes

You can edit Ceph File System (CephFS) volumes on the Red Hat Ceph Storage dashboard.

Before you begin

- A working Red Hat Ceph Storage cluster with Ceph File System deployed.

Procedure

1. From the dashboard navigation, go to **File › File Systems**.

2. From the listed volumes, select the volume to be edited and click **Edit**.
3. In the **Edit File System** window, rename the volume as required and click **Edit File System**.
   A notification displays that the volume was edited successfully.

Removing CephFS volumes

You can remove Ceph File System (CephFS) volumes on the Red Hat Ceph Storage dashboard.

Before you begin

- A working Red Hat Ceph Storage cluster with Ceph File System deployed.

Procedure

1. From the dashboard navigation, go to **File › File Systems**.
2. From the listed volumes, select the volume to be removed and click **Remove** from the action drop-down.
3. In the **Remove File System** window, select **Yes, I am sure** and click **Remove File System**.
   A notification displays that the volume was removed successfully.

Configuring CephFS subvolume groups

As a storage administrator, you can configure Ceph File System (CephFS) subvolume groups on the Red Hat Ceph Storage dashboard.

Creating CephFS subvolume groups

Create Ceph File System (CephFS) subvolume groups on the Red Hat Ceph Storage dashboard. You can use subvolume groups to apply policies across a set of subvolumes.

Before you begin

- A working Red Hat Ceph Storage cluster with Ceph File System deployed.

About this task

You can create subvolume groups to create subvolume on the dashboard.

Procedure

1. From the dashboard navigation, go to **File › File Systems**.
2. From the listed volumes, select the row for which you want to create subvolumes and expand the row.
3. From the **Subvolume groups** tab, click **Create** to create a subvolume group.
4. In the **Create Subvolume group** window, enter the following parameters:
   - **Name**: Set the name of the subvolume group.
   - **Volume name**: Validate that the correct name of the volume is selected.
   - **Size**: Set the size of the subvolume group. If left blank or entered as 0, then, by default, the size will be set as infinite.
   - **Pool**: Set the pool of the subvolume group. By default, `data_pool_layout` of the parent directory is selected.
   - **UID**: Set the UID of the subvolume group.
   - **GID**: Set the GID of the subvolume group.
   - **Mode**: Set the permissions for the directory. By default, the mode is 755 which is `rwxr-xr-x`.
5. Click **Create Subvolume group**.
   A notification displays that the subvolume group was created successfully.

Editing CephFS subvolume groups

Edit Ceph File System (CephFS) subvolume groups on the Red Hat Ceph Storage dashboard.

Before you begin

- A working Red Hat Ceph Storage cluster with Ceph File System deployed.

About this task

You can edit the subvolume groups on the dashboard.

Procedure

1. From the dashboard navigation, go to **File › File Systems**.
2. From the listed volumes, select the row for which you want to edit a subvolume and expand the row.
3. Under the **Subvolume groups** tab, select the row that you want to edit.
4. Click **Edit** to edit a subvolume group.
5. In the **Edit Subvolume group** window, edit the needed parameters and click **Edit Subvolume group**
   A notification displays that the subvolume group was edited successfully.

Removing CephFS subvolume groups

You can remove Ceph File System (CephFS) subvolume groups on the Red Hat Ceph Storage dashboard.

Before you begin

- A working Red Hat Ceph Storage cluster with Ceph File System deployed.

About this task

You can remove a subvolume group from the cluster on the dashboard.

> **Warning:** Ensure to remove the subvolume within the subvolume group before removing the subvolume group.

Procedure

1. From the dashboard navigation, go to **File › File Systems**.
2. From the listed volumes, select the row for which you want to remove a subvolume and expand the row.
3. Under the **Subvolume groups** tab, select the subvolume group you want to remove, and click **Remove** from the action drop-down.
4. In the **Remove Subvolume group** window, select **Yes, I am sure**, and click **Remove Subvolume group**.
   A notification displays that the subvolume group was removed successfully.

Configuring CephFS subvolumes

As a storage administrator, you can configure Ceph File System (CephFS) subvolumes on the Red Hat Ceph Storage dashboard.

Creating CephFS subvolume

You can create Ceph File System (CephFS) subvolume on the Red Hat Ceph Storage dashboard.

Before you begin

- A working Red Hat Ceph Storage cluster with Ceph File System deployed.

Procedure

1. From the dashboard navigation, go to **File › File Systems**.
2. From the listed volumes, select the row.
3. Under the **Subvolume** tab, select the Subvolume group in which you want to create the subvolume.

   > **Note:** If you select the **Default** option, the subvolumes are not created under any subvolume groups.

4. Click **Create** to create a subvolume.
5. In the **Create Subvolume** window, enter the following parameters:
   - **Name**: Set the name of the subvolume group.
   - **Subvolume name**: Set the name of the volume.
   - **Size**: Set the size of the subvolume group. If left blank or entered as 0, then, by default, the size will be set as infinite.
   - **Pool**: Set the pool of the subvolume group. By default, `data_pool_layout` of the parent directory is selected.
   - **UID**: Set the UID of the subvolume group.
   - **GID**: Set the GID of the subvolume group.
   - **Mode**: Set the permissions for the directory. By default, the mode is 755 which is `rwxr-xr-x`.
   - **Isolated Namespace**: If you want to create the subvolume in a separate RADOS namespace, select this option.
6. Click **Create Subvolume**.
   A notification that the subvolume was created successfully is displayed.

Editing CephFS subvolume

You can edit Ceph File System (CephFS) subvolume on the Red Hat Ceph Storage dashboard.

Before you begin

- A working Red Hat Ceph Storage cluster with Ceph File System deployed.
- A subvolume created.

About this task

You can edit the subvolumes on the dashboard.

> **Note:** You can only edit the Size of the subvolumes on the dashboard.

Procedure

1. From the dashboard navigation, go to **File › File Systems**.
2. From the listed volumes, select the row.
3. Under the **Subvolume** tab, select the subvolume you want to edit, and click **Edit** within the volume.
4. In the **Edit Subvolume** window, edit the required parameters:
   - **Name**: Set the name of the subvolume group.
   - **Subvolume name**: Set the name of the volume.
   - **Size**: Set the size of the subvolume group. If left blank or entered as 0, then, by default, the size will be set as infinite.

- **Pool**: Set the pool of the subvolume group. By default, `data_pool_layout` of the parent directory is selected.
- **UID**: Set the UID of the subvolume group.
- **GID**: Set the GID of the subvolume group.
- **Mode**: Set the permissions for the directory. By default, the mode is 755 which is `rwxr-xr-x`.

5. Click **Edit Subvolume**.
   A notification that the subvolume was edited successfully is displayed.

Removing CephFS subvolume

You can remove Ceph File System (CephFS) subvolume on the Red Hat Ceph Storage dashboard.

Before you begin

- A working Red Hat Ceph Storage cluster with Ceph File System deployed.
- A subvolume created.

Procedure

1. From the dashboard navigation, go to **File › File Systems**.
2. From the listed volumes, select the row.
3. Navigate to **Subvolume** tab, select the subvolume you want to remove, and click **Remove**.
4. In the **Remove Subvolume** window, confirm whether you want to remove the selected subvolume and click **Remove Subvolume**.
   A notification that the subvolume was removed successfully is displayed.

Managing CephFS snapshots

As a storage administrator, you can cretate Ceph File System (CephFS) volume and subvolume snapshots on the Red Hat Ceph Storage dashboard.

The Ceph File System (CephFS) snapshots create an immutable, point-in-time view of a Ceph File System. CephFS snapshots are asynchronous and are kept in a special hidden directory in the CephFS directory named `.snap`. You can specify snapshot creation for any directory within a Ceph File System. When specifying a directory, the snapshot also includes all the subdirectories beneath it.

CephFS snapshot feature is enabled by default on new Ceph File Systems, but it must be manually enabled on existing Ceph File Systems.

> **Warning:** Each Ceph Metadata Server (MDS) cluster allocates the snap identifiers independently. Using snapshots for multiple Ceph File Systems that are sharing a single pool causes snapshot collisions, and results in missing file data.

Creating CephFS subvolume snapshots

As a storage administrator, you can create Ceph File System (CephFS) subvolume snapshot on the Red Hat Ceph Storage dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- A subvolume group with corresponding subvolumes.

About this task

You can create an immutable, point-in-time view of a Ceph File System (CephFS) by creating a snapshot.

Procedure

1. From the dashboard navigation, go to **File › File Systems**.
2. Expand the CephFS where you want to create the subvolume snapshot.
   If there are no file systems available, create a new file system. For more information, see “Creating CephFS volumes” on page 762.
3. Go to the **Snapshots** tab. There are three columns - **Groups**, **Subvolumes** and **Create**.
4. From the **Groups** and **Subvolumes** columns, select the group and the subvolume for which you want to create a snapshot.
5. Click **Create** in the third column. The *Create snapshot* form opens.
   - **Name**: A default name (date and time of the creation of snapshot) is already added. You can edit this name and add a new name.
   - **Volume name**: The volume name is the file system name. It is already added to the form as per your selection.
   - **Subvolume group**: The subvolume group name is already added as per your selection. Alternatively, you select a different subvolume group from the dropdown list.
   - **Subvolume**: The subvolume name is already added as per your selection. Alternatively, you can select a different subvolume from the dropdown list.
6. Click **Create Snapshot**.
   A notification is displayed that the snapshot is created successfully.
7. You can verify that the snapshot is created by going to the **Snapshots** tab, select the subvolume and subvoume group for which the snapshot is created.

Deleting CephFS subvolume snapshots

As a storage administrator, you can delete Ceph File System (CephFS) subvolume snapshot on the Red Hat Ceph Storage dashboard.

Before you begin

- A running Red Hat Ceph Storage cluster.
- A subvolume snapshot.

About this task

You can now delete Ceph File System (CephFS) subvolume snapshot on the dashboard.

Procedure

1. From the Ceph Dashboard, go to **File › File Systems**
2. Select the CephFS where you want to delete the subvolume snapshot.
3. Go to the **Snapshots** tab.
4. Select the snapshot you want to delete. You can list the snapshot by selecting the subvolume group and subvoume for which the snapshot is created.
5. In the **Delete Subvolume** dialog, select **Yes, I am sure** and then click **Delete Subvolume**.
   A notification is displayed that the subvolume snapshot is deleted successfully.

Cloning CephFS subvolume snapshots

As a storage administrator, you can clone Ceph File System (CephFS) subvolume snapshot on the Red Hat Ceph Storage dashboard.

Before you begin

- A running Red Hat Ceph Storage cluster.
- A subvolume snapshot.

About this task

You can now clone a Ceph File System (CephFS) subvolume snapshot on the dashboard.

Procedure

1. Log into the **dashboard**.
2. On the dashboard navigation menu, click **File → File Systems.**
3. Select the CephFS where you want to clone the subvolume snapshot.
4. Go to the **Snapshots** tab.
5. Select the snapshot you want to clone. You can list the snapshot by selecting the subvolume group and subvoume for which the snapshot is created.
6. Click the **arrow** next to the **Delete** button.
7. Click **Clone**. The *Create clone*form opens.
   - **Name**: A default name (date and time of the creation of clone) is already added. You can change this name and add a new name.
   - **Group name**: The subvolume group name is already added as per your selection. Alternatively, you select a different subvolume group from the dropdown list.
8. Click **Create Clone**.
   A notification is displayed that the clone is created successfully.
9. You can verify that the clone is created by going to the **Snapshots** tab, in the **Subvolume** column. Select the subvoume group for which the clone is created.

Creating CephFS volume snapshots

As a storage administrator, you can create Ceph File System (CephFS) volume snapshot on the Red Hat Ceph Storage dashboard.

Before you begin

- A running Red Hat Ceph Storage cluster.
- A subvolume group with corresponding subvolumes.

About this task

You can create an immutable, point-in-time view of a Ceph File System (CephFS) volume by creating a snapshot.

Procedure

1. Log into the **dashboard**.
2. On the dashboard navigation menu, click **File → File Systems.**
3. Select the CephFS where you want to create the subvolume snapshot.
   If there are no file systems available, create a new file system.
4. Go to the **Directories** tab.
5. From the list of the subvolume and volumes, select the volume for which you want to create the snapshot.
6. Click **Create** in the **Snapshot** row. The *Create Snapshot* form opens.
   - **Name**: A default name (date and time of the creation of snapshot) is already added. You can edit this name and add a new name.
7. Click **Create Snapshot**.
   A notification is displayed that the snapshot is created successfully.
8. You can verify that the snapshot is created from the**Snapshots** row in the **Directories** tab.

Deleting CephFS volume snapshots

As a storage administrator, you can delete Ceph File System (CephFS) volume snapshot on the Red Hat Ceph Storage dashboard.

Before you begin

- A running Red Hat Ceph Storage cluster.
- A volume snapshot.

About this task

You can now delete Ceph File System (CephFS) volume snapshot on the dashboard.

Procedure

1. Log into the **dashboard**.
2. On the dashboard navigation menu, click **File → File Systems.**
3. Select the CephFS where you want to delete the subvolume snapshot.
4. Go to the **Directories** tab.
5. Select the snapshot you want to delete. You can see list of snapshots in the **Snapshot** row.
6. Click **Delete**.
7. In the **Delete Snapshot** dialog, select **Yes, I am sure** and then click **Delete Snapshot**.
   A notification is displayed that the snapshot is deleted successfully.

Scheduling CephFS snapshots

As a storage administrator, you can schedule Ceph File System (CephFS) snapshots on the Red Hat Ceph Storage dashboard.

Scheduling Ceph File System (CephFS) snapshots ensures consistent reliable backups at regular intervals, reducing the risk of data loss. Scheduling snapshots also provides ease of management by reducing administrative overhead of manually managing backups.

A list of available snapshots for a particular file system, subvolume, or directory is found on the **File › File Systems** page. Use the snapshot list for creating scheduled backups.

Creating CephFS snapshot schedule

As a storage administrator, you can create Ceph File System (CephFS) snapshot schedules on the Red Hat Ceph Storage dashboard.

Before you begin

- A running Red Hat Ceph Storage cluster.

About this task

Create a policy for automatic creation of Ceph File System (CephFS) snapshot of a volume or a certain directory. You can also define how often you want to schedule it (for example: every hour, every day, every week, etc.)

You can specify the number of snapshots or a period for which you want to keep the snapshots. All the older snapshots are deleted when the number of snapshots exceeds or when the retention period is over

Procedure

1. From the dashboard navigation, go to **File › File Systems**.
   File system volumes are listed. Select the file system where you want to create the snapshot schedule.
2. Go to **Snapshot schedules** tab.

> **Note:** The **Enable** is available only if the *snapshot_scheduler* is disabled on the cluster.

a. Click **Enable** to enable the `snapshot_scheduler` module.
After enabling the scheduler, wait for the dashboard to reload and navigate back to the **Snapshot schedule** page.

b. Click **Create**.
The **Create Snapshot schedule** form opens.

c. Enter the directory name, start date, start time, and schedule.

- **Directory**: You can search by typing in the path of the directory or subvolume and select the directory from the suggested list where you want to create the snapshot schedule.
- **Start date**: Enter date on which you want the scheduler to start creating the snapshots. By default the current date is selected.
- **Start time**: Enter the time at which you want the scheduler to start creating the snapshots. By default the current time is added.
- **Schedule**: Enter the number of snapshots and the frequency at which you want to create the snapshots. Frequency can be hourly, daily, weekly, monthly, yearly or lastest snapshots.

3. Optional: Click **Add retention policy** if you want to add retention policy to the schedule. You can add multiple retention policies.
Enter the number of snapshots and the frequency at which you want to retain the snapshots. Frequency can be hourly, daily, weekly, monthly, yearly or lastest snapshots.
4. Click **Create snapshot schedule**.
A notification is displayed that the snapshot schedule is created successfully.

What to do next

Verify that the snapshots are created by going to the **Snapshots** tab after the first scheduled snapshot is triggered.

Editing CephFS snapshot schedule

As a storage administrator, you can edit Ceph File System (CephFS) snapshot schedules on the Red Hat Ceph Storage dashboard.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A snapshot schedule.

About this task

Edit an existing snapshot schedule using the snapshot scheduler.

You can only edit the retention policy of the snapshot schedule. You can add another retention policy or delete the existing policy.

Procedure

1. From the dashboard navigation, go to **File › File Systems**.
2. Select the CephFS where you want to edit snapshot schedules and click **Snapshot schedules**.
3. Select the snapshot schedule that you want to edit.
4. Click **Edit**. An **Edit Snapshot schedule** dialog box appears.
5. Optional: From the **Edit Snapshot schedule** dialog, add a schedule retention policy, by selecting **Add retention policy**.
Enter the number of snapshots and the frequency at which you want to retain the snapshots.
Frequency can be:

- Weekly or lastest snapshots. For example, enter the number of snapshots as 10 and select **lastest snapshots** to retain last 10 snapshots irrespective of the frequency at which they were created.
- Daily
- Hourly
- Monthly
- Yearly

6. Click **Edit snapshot schedule** to save.
   A notification is displayed that the retention policy is created successfully.
7. Click the **trash** icon to delete the existing retention policy.
   A notification is displayed that the retention policy is deleted successfully.

Deleting CephFS snapshot schedule

As a storage administrator, you can delete Ceph File System (CephFS) snapshot schedules on the Red Hat Ceph Storage dashboard.

Procedure

1. From the dashboard navigation, go to **File › File Systems**.
2. Select the CephFS where you want to edit snapshot schedules and click **Snapshot schedules**.
3. Select the snapshot schedule that you want to delete and click **Delete** from the action drop-down.
4. On the navigation menu, click **File › File Systems**.
5. Select the CephFS where you want to edit snapshot schedules.
6. Click **Snapshot schedules**.
7. Select the snapshot schedule that you want to delete.
8. Click the **Edit** dropdown menu.
9. Click **Delete**.
   The**Delete Snapshot schedule** dialog box displays.
10. Select **Yes, I am sure** to confirm if you want to delete the schedule.
11. Click **Delete snapshot schedule**.
    A notification is displayed that the snapshot schedule is deleted successfully.

Deactivating and activating CephFS snapshot schedule

As a storage administrator, you can deactivate and activate Ceph File System (CephFS) snapshot schedules on the Red Hat Ceph Storage dashboard.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A snapshot schedule.

About this task

The snapshot schedule is activated by default, it runs according to how it is configured. Deactivating excludes the snapshot from scheduling until it is activated again.

You can deactivate and activate snapshot schedules from the CephFS dashboard using the snapshot scheduler.

Procedure

1. From the dashboard navigation, go to **File › File Systems**.
2. Select the CephFS where you want to edit snapshot schedules.

3. Click **Snapshot schedules**.
4. Select the snapshot schedule that you want to deactivate and click **Deactivate** from the action drop-down.
5. Select **Yes, I am sure** to confirm if you want to deactivate the schedule.
6. Click **Deactivate snapshot schedule**.
   A notification is displayed that the snapshot schedule is deactivated successfully.
7. You can activate the snapshot schedule by clicking **Activate** in the action drop-down.

Managing block devices

As a storage administrator, you can manage and monitor block device images on the Red Hat Ceph Storage dashboard. The functionality is divided between generic image functions and mirroring functions. For example, you can create new images, view the state of images that are mirrored across clusters, and set IOPS limits on an image.

Managing block device images

As a storage administrator, you can create, edit, copy, purge, and delete images using the Red Hat Ceph Storage dashboard. In addition, you can also create, clone, copy, rollback, and delete snapshots of the images using the Ceph dashboard.

> **Note:** The Block Device images table is paginated for use with 10000+ image storage clusters to reduce Block Device information retrieval costs.

Creating images

You can create block device images on the Red Hat Ceph Storage dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool with the `rbd` application enabled is created.

About this task

For more information, see the following:

- Block devices
- Creating pools

Procedure

1. From the dashboard navigation, go to **Block › Images**.
2. From the **Images** tab, click **Create**.
3. Complete the **Create Images** form.
4. Optional: Click **Advanced** to set advanced parameters, such as **Striping** and **Quality of Service**.
5. Click **Create Image**.
   A notification displays that the image was created successfully.

Creating namespaces

Create namespaces for the block device images on the Red Hat Ceph Storage dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool with the `rbd` application enabled is created.

About this task

Once the namespaces are created, you can give access to the users for those namespaces.

For more information, see the *How to segregate RBD images within isolated namespaces in ceph?* article on the Red Hat Customer Portal.

Procedure

1. From the dashboard navigation, go to **Block › Images**.
2. From the **Namespaces** tab, click **Create**.
3. In the **Create Namespace** dialog, select the pool and enter a name for the namespace.
4. Click **Create**.
   A notification displays that the namespace was created successfully.

Editing images

Edit block device images on the Red Hat Ceph Storage dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool with the `rbd` application enabled is created.
- An image is created.

Procedure

1. From the dashboard navigation, go to **Block › Images**.
2. To edit the image, select its row and click **Edit**.
3. In the **Edit Image** form, edit the required parameters and click **Edit Image**.

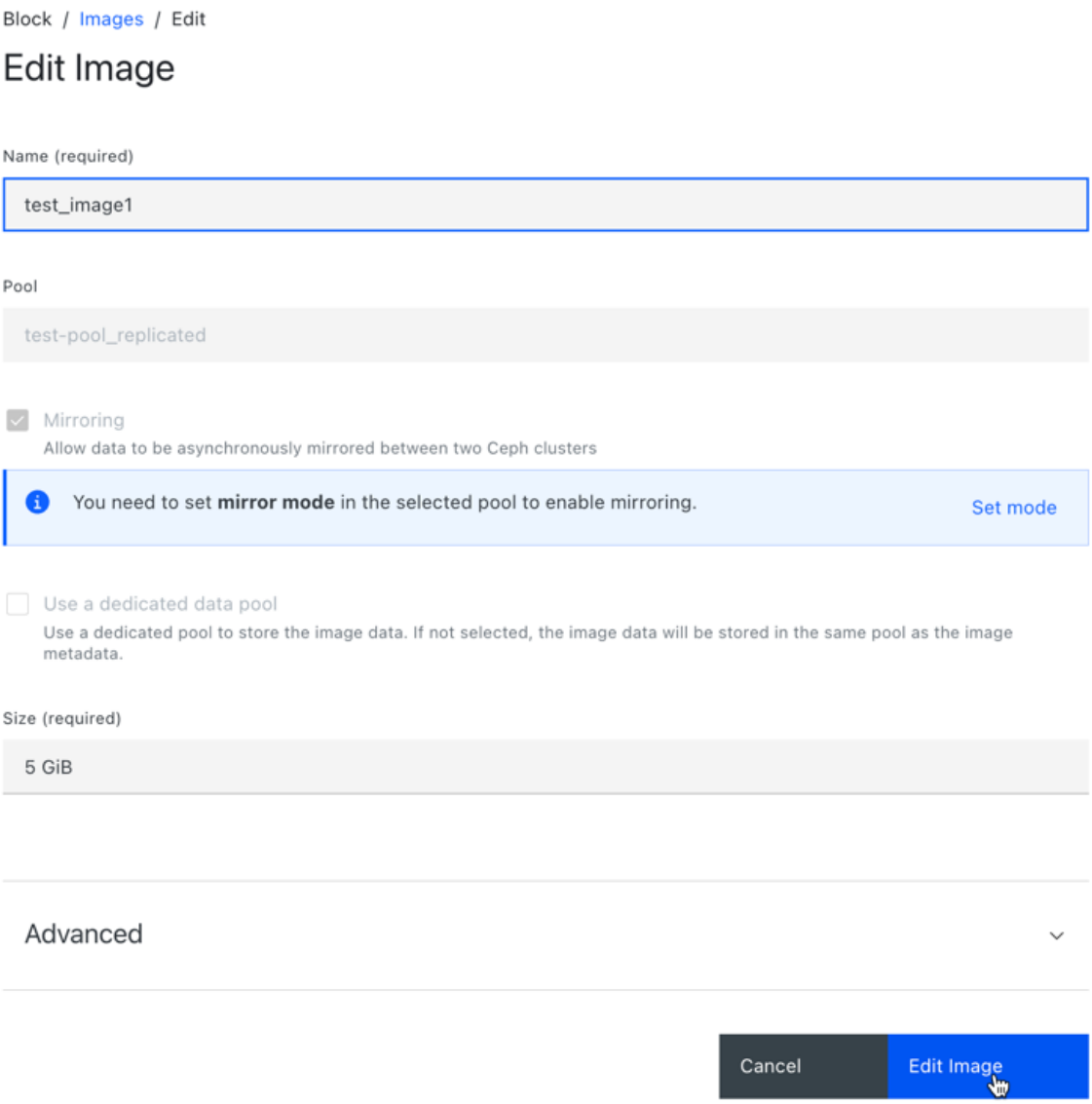


*Figure 75: **Edit Image** image form*

A notification displays that the image was updated successfully.

Copying images

Copy block device images on the Red Hat Ceph Storage dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool with the `rbd` application enabled is created.
- An image is created.

About this task

For more information, see the following:

- Block devices
- “Creating pools” on page 704

Procedure

1. From the dashboard navigation, go to **Block › Images**.
2. From the **Images** tab, click **Copy** from the image menu.
3. In the **Copy Image** form, set the required parameters and click **Copy Image**.
   A notification displays that the image was copied successfully.

Moving images to trash

Move the block device images to trash before it is deleted on the Red Hat Ceph Storage dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool with the `rbd` application enabled is created.
- An image is created.

Procedure

1. From the dashboard navigation, go to **Block › Images**.
2. From the **Images** tab, select the image to move to trash, and click **Move to Trash** from the image menu.
3. In the **Move an image to trash** dialog, change the **Protection expires at** field and click **Move**.
   A notification displays that the image was moved to trash successfully.

Purging trash

Purge trash using the Red Hat Ceph Storage dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool with the `rbd` application enabled is created.
- An image is trashed.

Procedure

1. From the dashboard navigation, go to **Block › Images**.
2. From the **Trash** tab, click **Purge Trash**.
3. In the **Purge Trash** dialog, select the pool and click **Purge Trash**.
   A notification displays that the pools in the trash were purged successfully.

Restoring images from trash

Restore the images that were trashed and has an expiry date on the Red Hat Ceph Storage Dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool with the `rbd` application enabled is created.

- An image is trashed.

Procedure

1. From the dashboard navigation, go to **Block › Images**.
2. From the **Trash** tab, select the row of the image to restore.
3. Click **Restore** from the image menu.
4. In the **Restore Image** dialog, enter the new name of the image and click **Restore**.
   A notification displays that the image was restored successfully.

Deleting images

Delete the images from the cluster on the dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool with the `rbd` application enabled is created.
- An image is created.

Procedure

1. From the dashboard navigation, go to **Block › Images**.
2. Select the row to be deleted and click **Delete** from the action drop-down.
3. In the **Delete RBD** notification, select **Yes, I am sure** and click **Delete RBD**.
   A notification displays that the image was deleted successfully.

Deleting namespaces

Delete the namespaces of the images on the Red Hat Ceph Storage Dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool with the `rbd` application enabled is created.
- A namespace is created in the pool.

Procedure

1. From the dashboard navigation, go to **Block › Images**.
2. From the **Namespaces** tab, select the namespace and click **Delete** from the action drop-down.
3. In the **Delete Namespace** notification, select **Yes, I am sure** and click **Delete Namespace**.
   A notification displays that the namespace was deleted successfully.

Creating snapshots of images

Take snapshots of the Ceph Block Device images on the Red Hat Ceph Storage Dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.

- Dashboard is installed.
- A pool with the `rbd` application enabled is created.
- An image is created.

About this task

For more information, see the following:

- Creating a block device snapshot
- “Creating pools” on page 704
- “Creating images” on page 772

Procedure

1. From the dashboard navigation, go to **Block › Images**.
2. From the **Images** tab, expand an image row.
3. In the **Snapshots** tab, click **Create**.
4. In the **Create RBD Snapshot** dialog, enter the snapshot name and click **Create RBD Snapshot**.
   A notification displays that the snapshot was created successfully.

Renaming snapshots of images

Rename the snapshots of the Ceph Block Device images on the Red Hat Ceph Storage Dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool with the `rbd` application enabled is created.
- An image is created.
- A snapshot of the image is created.

Procedure

1. From the dashboard navigation, go to **Block › Images**.
2. From the **Images** tab, expand an image row.
3. In the **Snapshots** tab, click **Rename**.
4. In the **Rename RBD Snapshot** dialog, enter the new name and click **Rename RBD Snapshot**.

Protecting snapshots of images

Protect the snapshots of the Ceph Block Device images on the Red Hat Ceph Storage Dashboard. Protecting snapshots of images is required when you need to clone the snapshots.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool with the `rbd` application enabled is created.
- An image is created.
- A snapshot of the image is created.

Procedure

1. From the dashboard navigation, go to **Block › Images**.
2. From the **Images** tab, expand an image row and click on the **Snapshots** tab.
3. Select the snapshot to protect, and click **Protect** from the action drop-down.
   The snapshot updates and the **State** changes from`Unprotected`to`Protected`.

Cloning snapshots of images

Clone the snapshots of images on the Red Hat Ceph Storage Dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool with the `rbd` application enabled is created.
- An image is created.
- A snapshot of the image is created and protected.

About this task

For more information, see the following:

- Protecting snapshots.
- “Protecting snapshots of images” on page 777

Procedure

1. From the dashboard navigation, go to **Block › Images**.
2. From the **Images** tab, expand an image row.
3. In the **Snapshots** tab, select the snapshot to clone and click **Clone** from the action drop-down.
4. In the **Clone RBD** form, edit the parameters, as required, and click **Clone RBD**.

Copying snapshots of images

Copy the snapshots of images on the Red Hat Ceph Storage Dashboard.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool with the `rbd` application enabled is created.
- An image is created.
- A snapshot of the image is created.

Procedure

1. From the dashboard navigation, go to **Block › Images**.
2. To protect the snapshot of the image, from the **Images** tab, select the row and then go to the **Snapshots** tab.
3. Click **Copy** from the action drop-down.
4. In the **Copy RBD** form, enter the parameters and click **Copy RBD**.

A notification displays that the snapshot was copied successfully. You can search for the copied image in the *Images* tab.

Reference

For more information, see the following:

- Creating pools.
- Creating images.

Unprotecting snapshots of images

You can unprotect the snapshots of the Ceph Block Device images on the Red Hat Ceph Storage Dashboard. Unprotecting snapshots of images is required when you need to delete the snapshots.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool with the `rbd` application enabled is created.
- An image is created.
- A snapshot of the image is created and protected.

Procedure

1. From the dashboard navigation, go to **Block › Images**.
2. From the **Images** tab, expand an image row and click on the **Snapshots** tab.
3. Select the protected snapshot, and click **Unprotect** from the action drop-down.
   The snapshot updates and the **State** changes from`Protected`to`Unprotected`.

Rolling back snapshots of images

You can rollback the snapshots of the Ceph Block Device images on the Red Hat Ceph Storage Dashboard. Rolling back an image to a snapshot means overwriting the current version of the image with data from a snapshot. The time it takes to run a rollback increases with the size of the image. It is faster to clone from a snapshot than to rollback an image to a snapshot, and it is the preferred method of returning to a preexisting state.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool with the `rbd` application enabled is created.
- An image is created.
- A snapshot of the image is created.

Procedure

1. From the dashboard navigation, go to **Block › Images**.
2. From the **Images** tab, expand an image row and click on the **Snapshots** tab.
3. Select the snapshot to rollback, and click **Rollback** from the snapshot menu.
4. In the **RBD snapshot rollback** dialog, click **Rollback**.

Deleting snapshots of images

Delete the snapshots of the Ceph Block Device images on the Red Hat Ceph Storage Dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool with the `rbd` application enabled is created.
- An image is created.
- A snapshot of the image is created and is unprotected.

About this task

For more information, see the following:

- Deleting a block device snapshot.
- "Unprotecting snapshots of images" on page 779.

Procedure

1. From the dashboard navigation, go to **Block › Images**.
2. From the **Images** tab, expand an image row.
3. In the **Snapshots** tab, select the snapshot to delete and click **Delete** from the action drop-down.
4. In the **Delete RBD Snapshot** dialog, select **Yes, I am sure** and click **Delete RBD Snapshot**.
   A notification displays that the snapshot was deleted successfully.

Managing mirroring functions

As a storage administrator, you can manage and monitor mirroring functions of the block devices on the Red Hat Ceph Storage Dashboard.

You can add another layer of redundancy to Ceph Block Devices by mirroring data images between storage clusters. Understanding and using Ceph Block Device mirroring can provide you protection against data loss, such as a site failure. There are two configurations for mirroring Ceph block devices, one-way mirroring or two-way mirroring, and you can configure mirroring on pools and individual images.

Mirroring view

View the Block device mirroring on the Red Hat Ceph Storage Dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- Mirroring is configured.

About this task

You can view the daemons, the site details, the pools, and the images that are configured for Block device mirroring.

Procedure

- From the dashboard navigation, go to **Block › Mirroring**.

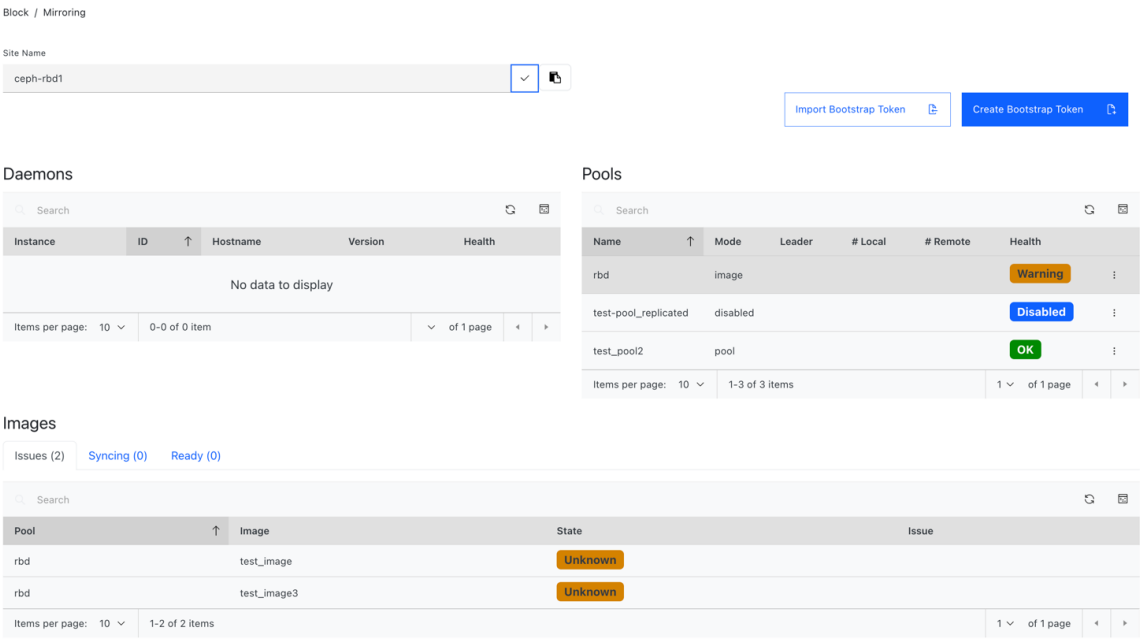


Figure 76: Ceph Block Device mirroring view

Editing mode of pools

Edit mode of the overall state of mirroring functions, which includes pools and images on the Red Hat Ceph Storage Dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool with the *rbd* application enabled is created.
- An image is created.
- Mirroring is configured.

Procedure

1. From the dashboard navigation, go to **Block › Mirroring**.
2. From the **Pools** table, select the pool that requires editing, and click **Edit Mode**.
3. In the **Edit pool mirror mode** dialog, select the mode from the drop-down, and then click **Update**.
   A notification displays that the pool was updated successfully.

Adding peer in mirroring

Add storage cluster peer for the `rbd-daemon` mirror to discover its peer storage cluster on the Red Hat Ceph Storage Dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Two healthy running Red Hat Ceph Storage clusters.
- The dashboard is installed on both the clusters.
- Pools created with the same name.
- `rbd` application enabled on both the clusters.

Note: Ensure that mirroring is enabled for the pool in which images are created.

About this task

For more information, see Configuring two-way mirroring.

Creating a bootstrap token from site A

1. From the dashboard navigation, go to **Block › Mirroring** and click **Create Bootstrap Token**.
2. On the **Navigation** menu, click the **Block** drop-down menu, and click **Mirroring**.
3. Click **Create Bootstrap token**.
   a. For the provided site name, select the pools to be mirrored.
   b. For the selected pools, generate a new bootstrap token by clicking **Generate**.
   c. Click **Copy to Clipboard**.
   d. Click **Close**.

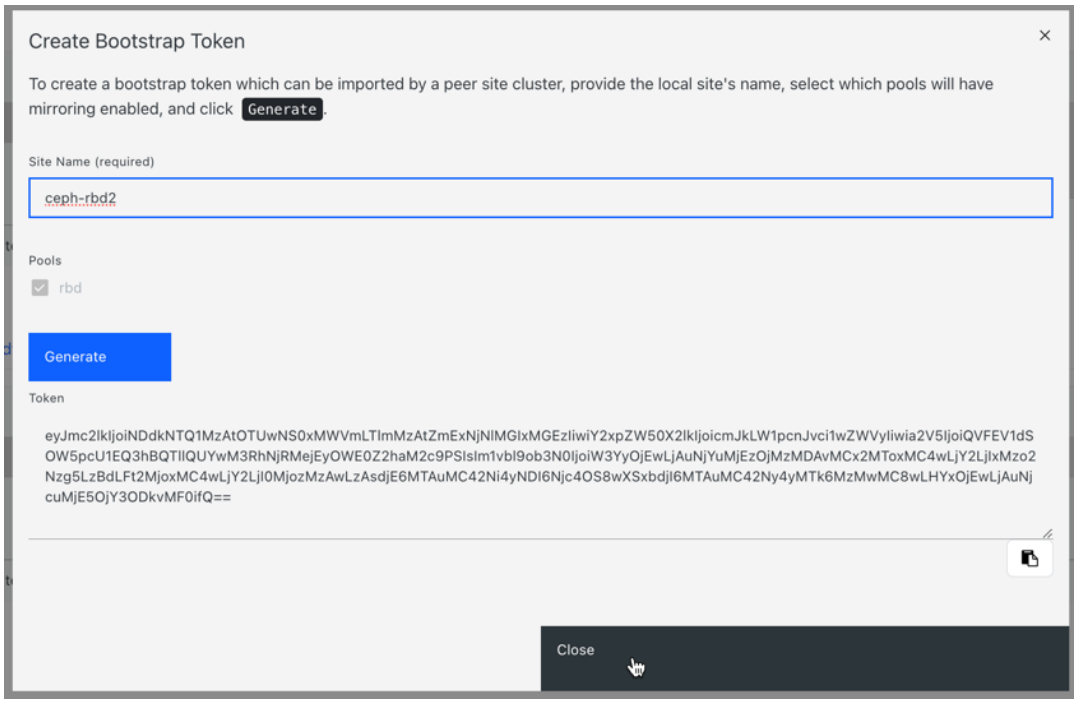


Figure 77: Create bootstrap token

4. Enable the pool mirror mode.
   a. Select the pool.
   b. Click **Edit Mode**.
   c. From the **Edit pool mirror mode** dialog, select **Image** from the **Mode** list.
   d. Click **Update**.
   A notification displays that the pool was updated successfully.

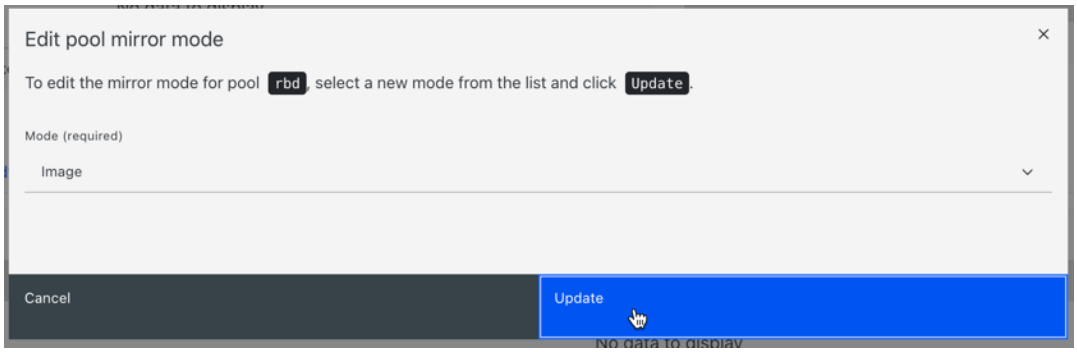


Figure 78: Edit pool mirror mode

Importing the bootstrap token on site B

1. From the dashboard navigation, go to **Block › Mirroring** and click **Import Bootstrap Token** from the action drop-down.

   > **Note:** Ensure that mirroring mode is enabled for the specific pool for which you are importing the bootstrap token.

2. In the **Import Bootstrap Token** dialog, select the direction, and paste the token copied earlier, from **Site A**.
3. Click **Submit**.

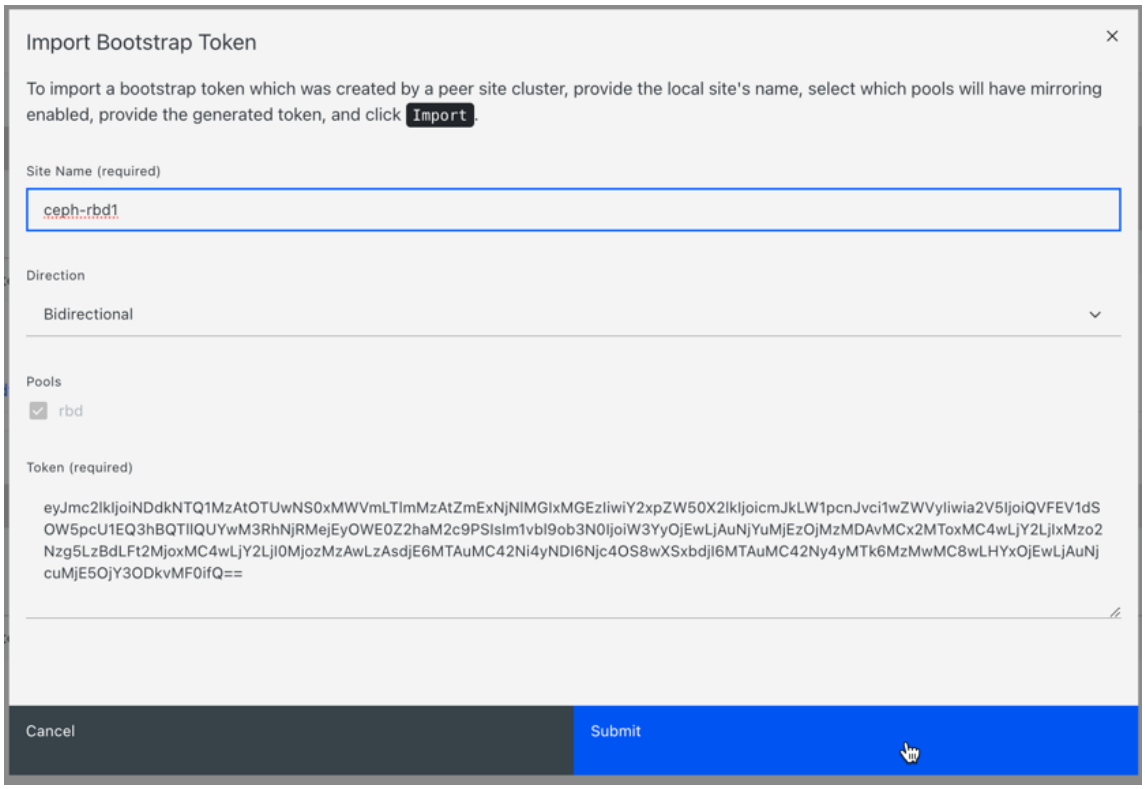


Figure 79: Import bootstrap token

The peer is added and the images are mirrored in the cluster at site B.

4. Verify that the health of the pool is in an **OK** state.
   - On the **Block › Mirroring** page, in the **Pool** table, verify the health of the pool.

Creating a Block Device image with mirroring on site A

1. Create a Block Device image with mirroring enabled.

a. From the dashboard navigation, go to **Block › Images**.

b. On the **Images** tab, click **Create**.

c. In the **Create Image** form, and complete the **Name**, **Size**, and **Pool**.

d. Select **Mirroring** .

> **Note:** Select mirroring with either **Journal** or **Snapshot**.

e. Click **Create Image**.

2. Verify that the image is available at both the sites.
   - From the **Images** table, verify that the image in Site A is set to `primary` and that the image in site B is set to `secondary`.

Editing peer in mirroring

Edit storage cluster peer for the `rbd-daemon` mirror to discover its peer storage cluster in the Red Hat Ceph Storage Dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool with the `rbd` application enabled is created.
- An image is created.
- Mirroring is configured.
- A peer is added.

Procedure

1. From the dashboard navigation, go to **Block › Mirroring**.
2. From the **Pools** table, click **Edit Peer** from the pool menu.
3. In the **Edit pool mirror peer** dialog, edit the parameters and click **Submit**.
   A notification displays that the peer was updated successfully.

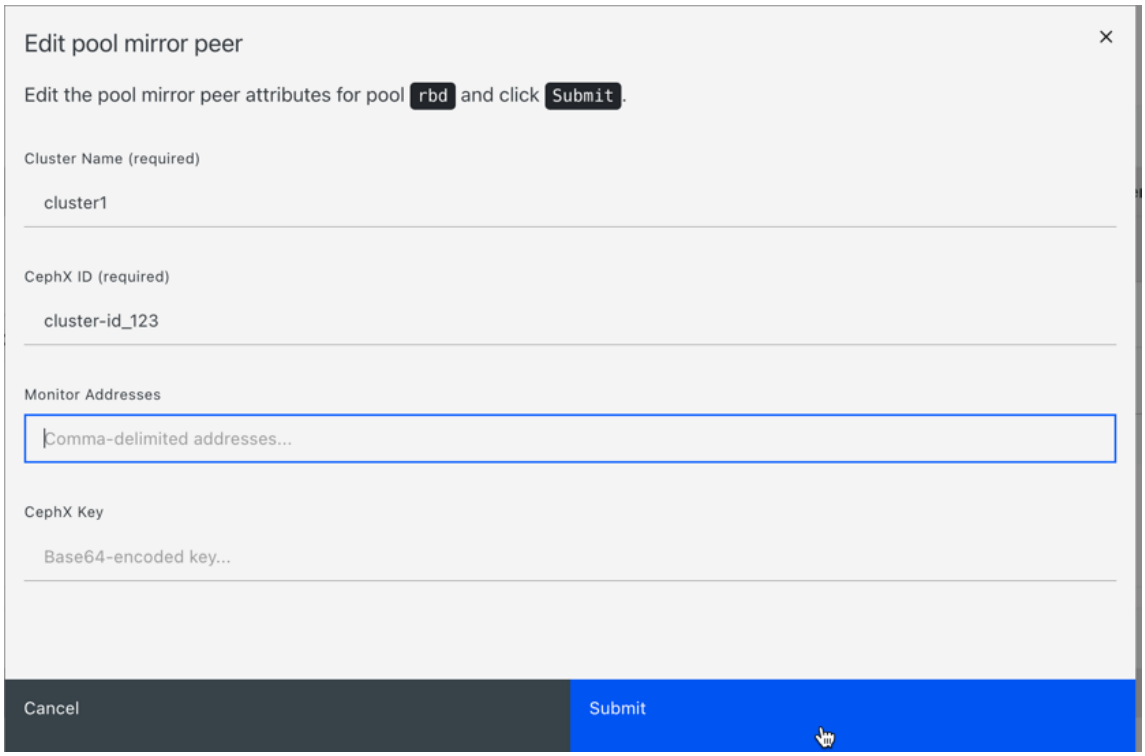


*Figure 80: Completing the **Edit pool mirror peer** form*

Deleting peer in mirroring

Delete a storage cluster peer for the `rbd-daemon` mirror in the Red Hat Ceph Storage Dashboard.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Dashboard is installed.
- A pool with the `rbd` application enabled is created.
- An image is created.
- Mirroring is configured.
- A peer is added.

About this task

For more information, see Adding peer in mirroring.

Procedure

1. From the dashboard navigation, go to **Block › Mirroring**.
2. From the **Pools** table, select the pool to edit and click **Delete Peer** from the action drop-down.
3. In the **Delete mirror peer** dialog, select **Yes, I am sure** and click **Delete mirror peer**.
   A notification displays that the peer deleted successfully.

Activating and deactivating telemetry

Activate the telemetry module to help Ceph developers understand how Ceph is used and what problems users might be experiencing, helping to improve the dashboard experience. Activating the telemetry module sends anonymous data about the cluster back to the Ceph developers.

About this task

View the telemetry data that is sent to the Ceph developers on the public telemetry dashboard. The public dashboard allows the community to easily see summary statistics on how many clusters are reporting, their total capacity and OSD count, and version distribution trends.

The telemetry report is broken down into several channels, each with a different type of information. After telemetry is enabled, you can turn on and off the individual channels. If telemetry is off, the per-channel setting has no effect.

Basic

Provides basic information about the cluster.

Crash

Provides information about daemon crashes.

Device

Provides information about device metrics.

Ident

Provides user-provided identifying information about the cluster.

Perf

Provides various performance metrics of the cluster.

The data reports contain information that help the developers gain a better understanding of the way Ceph is used. The data includes counters and statistics on how the cluster has been deployed, the version of Ceph, the distribution of the hosts, and other parameters.

Important: The data reports do not contain any sensitive data like pool names, object names, object contents, hostnames, or device serial numbers.

Note: Telemetry can also be managed by using an API. For more information, see Telemetry.

Procedure

1. Activate the telemetry module by going to **Settings › Telemetry configuration**.
2. Select each channel that telemetry should be enabled on.

   Note: For detailed information about each channel type, click **More Info** next to the channels.
3. Complete the **Contact Information** for the cluster.
   Enter the contact, Ceph cluster description, and organization.
4. Optional: Complete the **Advanced Settings** field options.

   Interval

   Set the interval by hour. The module compiles and sends a new report per this hour interval. The default interval is 24 hours.

   Proxy

   Use to configure an HTTP or HTTPs proxy server if the cluster cannot directly connect to the configured telemetry endpoint. Add the server in one of the following formats: `https://10.0.0.1:8080` or `https://ceph:telemetry@10.0.01:8080`

   The default endpoint is

   ```
   telemetry.ceph.com
   ```
5. Click **Next**.

This displays the **Telemetry report preview** before enabling telemetry.

6. Review the **Report preview**.

   > **Note:** The report can be downloaded and saved locally or copied to the clipboard.

7. Select **I agree to my telemetry data being submitted under the Community Data License Agreement**.
8. Enable the telemetry module by clicking **Update**.
   The following message is displayed, confirming the telemetry activation:
   `The Telemetry module has been configured and activated successfully`

Deactivating telemetry

Procedure

– To deactivate the telemetry module, go to **Settings › Telemetry configuration** and click **Deactivate**.

Red Hat Ceph Storage Object Gateways

Ceph Object Gateway, also known as RADOS Gateway (RGW), is an object storage interface built on top of the `librados` library to provide applications with a RESTful gateway to Ceph storage clusters. Use this information to understand how to deploy, configure, and administer a Ceph Object Gateway environment.

This uses a "Day Zero", "Day One", and "Day Two" organizational methodology, providing readers with a logical progression path.

Day Zero is where research and planning are done before implementing a potential solution.

Day One is where the actual deployment, and installation of the software happens.

Day Two is where all the basic, and advanced configuration happens.

Ceph Object Gateway supports three interfaces:

S3-compatibility

Provides object storage functionality with an interface that is compatible with a large subset of the Amazon S3 RESTful API.

Swift-compatibility

Provides object storage functionality with an interface that is compatible with a large subset of the OpenStack Swift API.
The Ceph Object Gateway is a service interacting with a Ceph storage cluster. Since it provides interfaces compatible with OpenStack Swift and Amazon S3, the Ceph Object Gateway has its own user management system. Ceph Object Gateway can store data in the same Ceph storage cluster used to store data from Ceph Block Device clients; however, it would involve separate pools and likely a different CRUSH hierarchy. The S3 and Swift APIs share a common namespace, so you can write data with one API and retrieve it with the other.

Administrative API

Provides an administrative interface for managing the Ceph Object Gateways.
Administrative API requests are done on a URI that starts with the `admin` resource end point. Authorization for the administrative API mimics the S3 authorization convention. Some operations require the user to have special administrative capabilities. The response type can be either XML or JSON by specifying the format option in the request, but defaults to the JSON format.

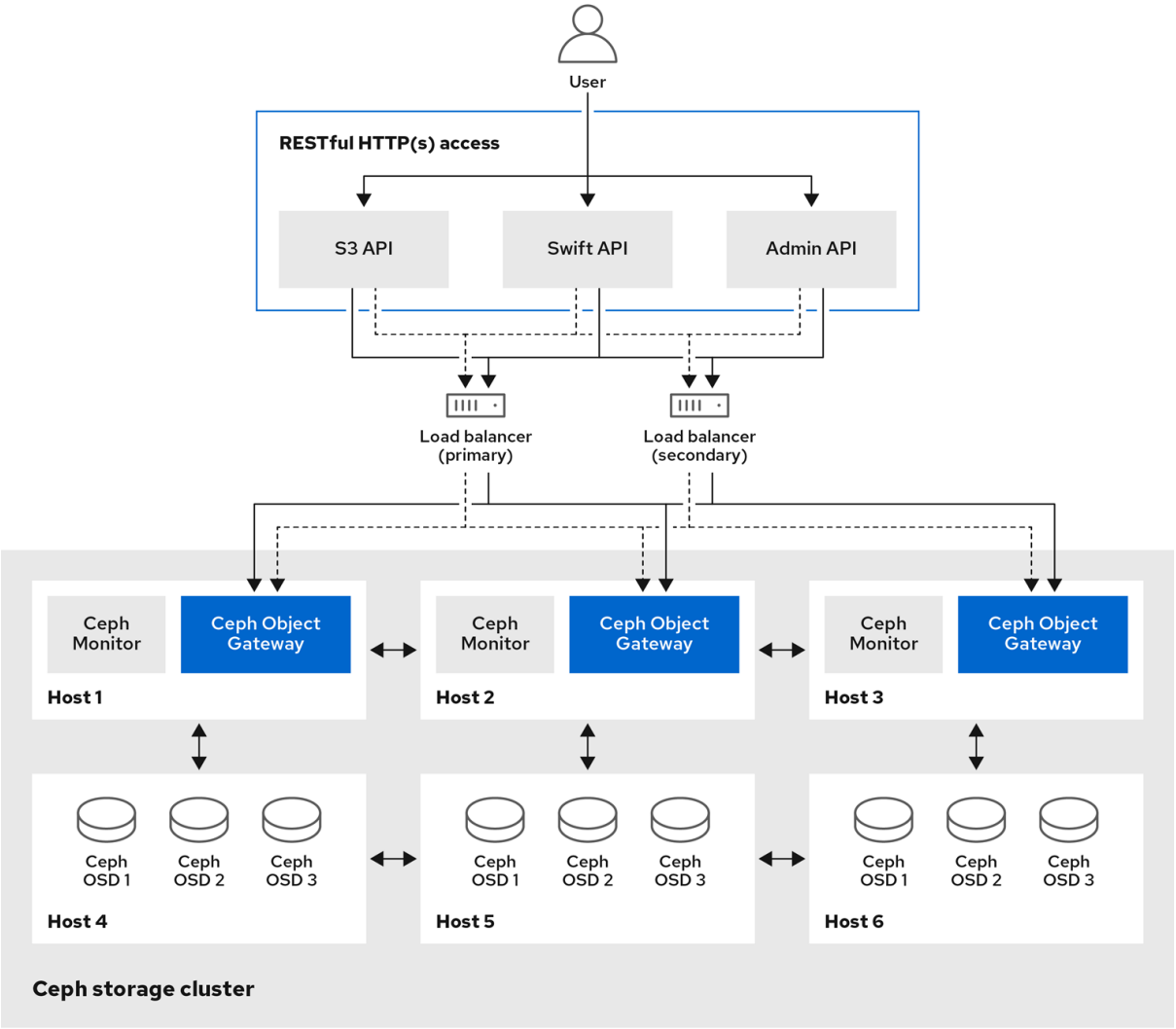


Figure 81: Basic access diagram

Introduction to WORM

Write-Once-Read-Many (WORM) is a secured data storage model that is used to guarantee data protection and data retrieval even in cases where objects and buckets are compromised in production zones.

In Red Hat Ceph Storage, data security is achieved through the use of S3 Object Lock with read-only capability that is used to store objects and buckets using a Write-Once-Read-Many (WORM) model, preventing them from being deleted or overwritten. They cannot be deleted even by the Red Hat Ceph Storage administrator.

S3 Object Lock provides two retention modes:

- GOVERNANCE
- COMPLIANCE

These retention modes apply different levels of protection to your objects. You can apply either retention mode to any object version that is protected by Object Lock.

In GOVERNANCE mode, users cannot overwrite or delete an object version or alter its lock settings unless they have special permissions. With GOVERNANCE mode, you can protect objects against deletion by most users, although you can still grant some users permission to alter the retention settings or delete the object if necessary.

In COMPLIANCE mode, a protected object version cannot be overwritten or deleted by any user. When an object is locked in COMPLIANCE mode, its retention mode cannot be changed or shortened.

Important:
Consulting company Cohasset concludes that Red Hat Ceph Storage, when properly configured and upon satisfying the additional considerations, meets the electronic record-keeping system requirements of **SEC Rules 17a-4(f)(2)**, **18a-6(e)(2)**, and **FINRA Rule 4511(c)**, as well as, supports the regulated entity in its compliance with the audit system requirements in **SEC Rules 17a-4(f)(3)(iii)** and **18a-6(e)(3)(iii)**. In addition, the assessed capabilities meet the principles-based electronic records requirements of **CFTC Rule 1.31(c)-(d)**. See the Cohasset certification for more information.

Reference

For more details about S3 Object Lock, see the following:

- "Enabling object lock for S3" on page 937

Key Factors for deploying Ceph Object Gateway

As a storage administrator, a basic understanding about what to consider before running a Ceph Object Gateway and implementing a multi-site Ceph Object Gateway solution is important. You can learn the hardware and network requirements, knowing what type of workloads work well with a Ceph Object Gateway, and Red Hat's recommendations.

Prerequisites

- Time to understand, consider, and plan a storage solution.

Reference

For more details about Ceph's various internal components and the strategies around those components, see "Storage strategies" on page 108.

Network Requirements in Red Hat Ceph Storage

An important aspect of a cloud storage solution is that storage clusters can run out of IOPS due to network latency, and other factors. Also, the storage cluster can run out of throughput due to bandwidth constraints long before the storage clusters run out of storage capacity. This means that the network hardware configuration must support the chosen workloads to meet price versus performance requirements.

Storage administrators prefer that a storage cluster recovers as quickly as possible. Carefully consider bandwidth requirements for the storage cluster network, be mindful of network link oversubscription, and segregate the intra-cluster traffic from the client-to-cluster traffic. Also consider that network performance is increasingly important when considering the use of Solid State Disks (SSD), flash, NVMe, and other high performing storage devices.

Ceph supports a public network and a storage cluster network. The public network handles client traffic and communication with Ceph Monitors. The storage cluster network handles Ceph OSD heartbeats, replication, backfilling, and recovery traffic. At a **minimum**, a single 10 GB Ethernet link should be used for storage hardware, and you can add additional 10 GB Ethernet links for connectivity and throughput.

Important: Red Hat recommends allocating bandwidth to the storage cluster network, such that it is a multiple of the public network using the `osd_pool_default_size` as the basis for the multiple on replicated pools. Red Hat also recommends running the public and storage cluster networks on separate network cards.

Important:
A 1 GB Ethernet network is not suitable for production storage clusters.

In the case of a drive failure, replicating 1 TB of data across a 1 GB Ethernet network takes 3 hours, and 3 TB takes 9 hours. Using 3 TB is the typical drive configuration. By contrast, with a 10 GB Ethernet network, the replication times would be 20 minutes and 1 hour. Remember that when a Ceph OSD fails, the storage cluster recovers by replicating the data it contained to other OSDs within the same failure domain and device class as the failed OSD.

The failure of a larger domain such as a rack means that the storage cluster utilizes considerably more bandwidth. When building a storage cluster consisting of multiple racks, which is common for large storage implementations, consider utilizing as much network bandwidth between switches in a "fat tree" design for optimal performance. A typical 10 GB Ethernet switch has 48 10 GB ports and four 40 GB ports. Use the 40 GB ports on the spine for maximum throughput. Alternatively, consider aggregating unused 10 GB ports with QSFP+ and SFP+ cables into more 40 GB ports to connect to other rack and spine routers. Also, consider using LACP mode 4 to bond network interfaces. Additionally, use jumbo frames, with a maximum transmission unit (MTU) of 9000, especially on the backend or cluster network.

Most performance-related problems in Ceph usually begin with a networking issue. Simple network issues like a kinked or bent Cat-6 cable could result in degraded bandwidth. Use a minimum of 10 GB ethernet for the front side network. For large clusters, consider using 40 GB ethernet for the backend or cluster network.

For network optimization, Red Hat recommends using jumbo frames for a better CPU per bandwidth ratio, and a non-blocking network switch back-plane.Red Hat Ceph Storage requires the same MTU value throughout all networking devices in the communication path, end-to-end for both public and cluster networks. Verify that the MTU value is the same on all hosts and networking equipment in the environment before using a Red Hat Ceph Storage cluster in production.

Workload Management in Red Hat Ceph Storage

One of the key benefits of a Ceph storage cluster is the ability to support different types of workloads within the same storage cluster by using performance domains. Different hardware configurations can be associated with each performance domain. Storage administrators can deploy storage pools on the appropriate performance domain, providing applications with storage tailored to specific performance and cost profiles. Selecting appropriately sized and optimized servers for these performance domains is an essential aspect of designing a Red Hat Ceph Storage cluster.

To the Ceph client interface that reads and writes data, a Ceph storage cluster appears as a simple pool where the client stores data. However, the storage cluster performs many complex operations in a manner that is visible to the client interface. Ceph clients and Ceph object storage daemons, referred to as *Ceph OSDs* or *OSDs*, both use the Controlled Replication Under Scalable Hashing (CRUSH) algorithm for the storage and retrieval of objects. Ceph OSDs can run in containers within the storage cluster.

A CRUSH map describes a topography of cluster resources, and the map exists both on client hosts and Ceph Monitor hosts within the cluster. Ceph clients and Ceph OSDs both use the CRUSH map and the CRUSH algorithm. Ceph clients communicate directly with OSDs, eliminating a centralized object lookup and a potential performance bottleneck. With awareness of the CRUSH map and communication with their peers, OSDs can handle replication, backfilling, and recovery, allowing for dynamic failure recovery.

Ceph uses the CRUSH map to implement failure domains. Ceph also uses the CRUSH map to implement performance domains, which takes the performance profile of the underlying hardware into consideration. The CRUSH map describes how Ceph stores data, and it is implemented as a simple hierarchy, specifically an acyclic graph, and a ruleset. The CRUSH map can support multiple hierarchies to separate one type of hardware performance profile from another. Ceph implements performance domains with device "classes".

- Hard disk drives (HDDs) are typically appropriate for cost and capacity-focused workloads.
- Throughput-sensitive workloads typically use HDDs with Ceph write journals on solid state drives (SSDs).
- IOPS-intensive workloads, such as MySQL and MariaDB, often use SSDs.

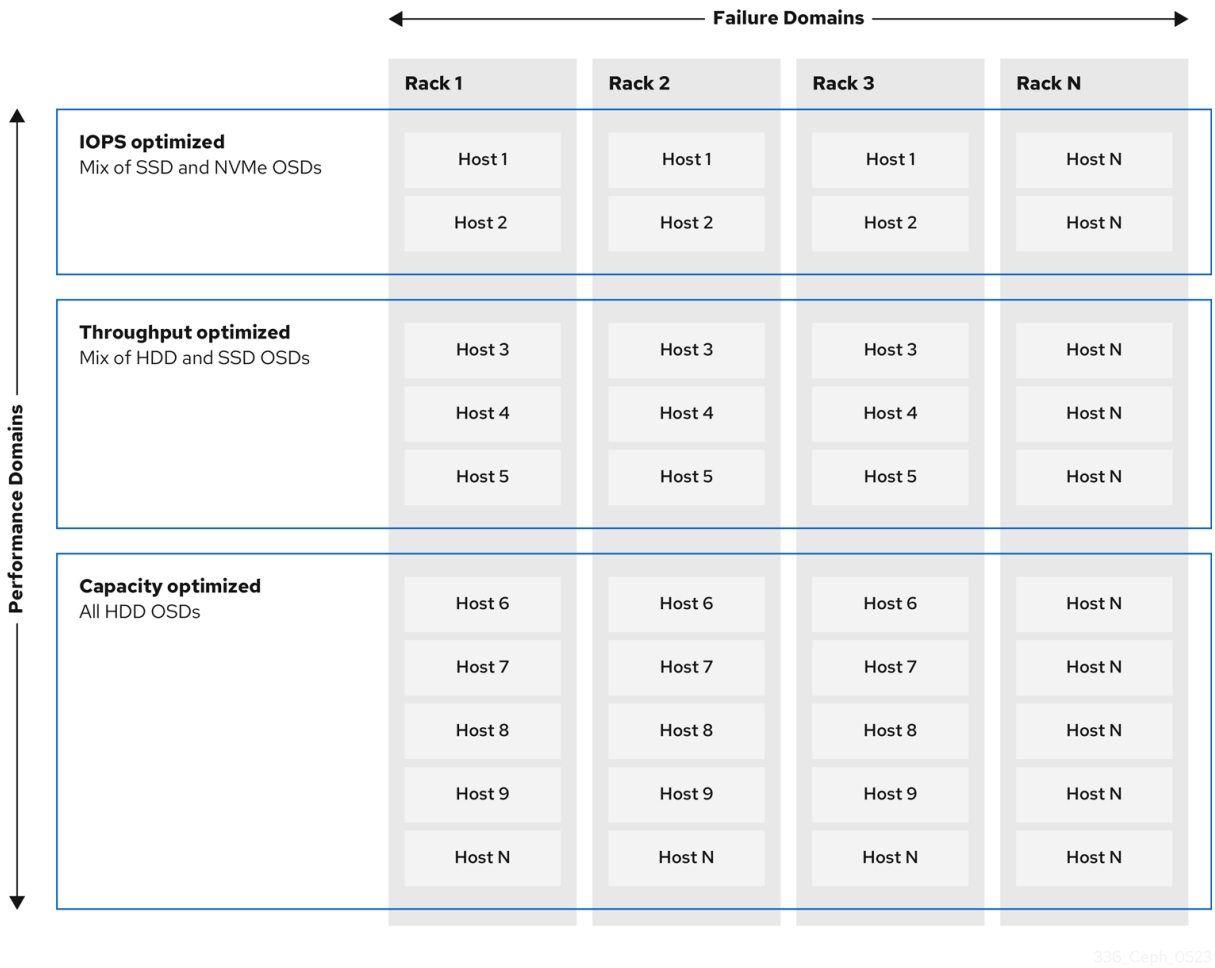


Figure 82: Performance and failure domains

Important: Operating System swap must not be enabled. Configuring the operating system to use swap to provide additional virtual memory for daemons is not recommended for modern systems and can result in degraded performance.

Workloads

Red Hat Ceph Storage is optimized for three primary workloads.

Important: Carefully consider the workload being run by Red Hat Ceph Storage clusters BEFORE considering what hardware to purchase because it can significantly impact the price and performance of the storage cluster. For example, if the workload is capacity-optimized and the hardware is better suited to a throughput-optimized workload, then hardware is more expensive than necessary. Conversely, if the workload is throughput-optimized and the hardware is better suited to a capacity-optimized workload, then the storage cluster can suffer from poor performance.

IOPS optimized

Input, output per second (IOPS) optimization deployments are suitable for cloud computing operations, such as running MYSQL or MariaDB instances as virtual machines on OpenStack. IOPS optimized deployments require higher performance storage such as 15k RPM SAS drives and separate SSD journals to handle frequent write operations. Some high IOPS scenarios use all flash storage to improve IOPS and total throughput.
An IOPS-optimized storage cluster has the following properties:

- Lowest cost per IOPS.

- Highest IOPS per GB.
- 99th percentile latency consistency.

Uses for an IOPS-optimized storage cluster are:

- Typically block storage.
- 3x replication for hard disk drives (HDDs) or 2x replication for solid state drives (SSDs).
- MySQL on OpenStack clouds.

Throughput optimized

Throughput-optimized deployments are suitable for serving up significant amounts of data, such as graphic, audio, and video content. Throughput-optimized deployments require high bandwidth networking hardware, controllers, and hard disk drives with fast sequential read and write characteristics. If fast data access is a requirement, then use a throughput-optimized storage strategy. Also, if fast write performance is a requirement, using Solid State Disks (SSD) for journals substantially improves write performance.
A throughput-optimized storage cluster has the following properties:

- Lowest cost per MBps (throughput).
- Highest MBps per TB.
- Highest MBps per BTU.
- Highest MBps per Watt.
- 97th percentile latency consistency.

Uses for a throughput-optimized storage cluster are as follows:

- Block or object storage.
- 3x replication.
- Active performance storage for video, audio, and images.
- Streaming media, such as 4k video.

Capacity optimized

Capacity-optimized deployments are suitable for storing significant amounts of data as inexpensively as possible. Capacity-optimized deployments typically trade performance for a more attractive price point. For example, capacity-optimized deployments often use slower and less expensive SATA drives and colocate journals rather than using SSDs for journaling.
A cost and capacity-optimized storage cluster has the following properties:

- Lowest cost per TB.
- Lowest BTU per TB.
- Lowest Watts required per TB.

Uses for a cost and capacity-optimized storage cluster are as follows:

- Typically object storage.
- Erasure coding for maximizing usable capacity.
- Object archive.
- Video, audio, and image object repositories.

Basic Considerations for Red Hat Ceph Storage Deployment

A storage strategy is a method of storing data that serves a particular use case. If you need to store volumes and images for a cloud platform like OpenStack, you can choose to store data on faster Serial Attached SCSI (SAS) drives with Solid State Drives (SSD) for journals. By contrast, if you need to store object data for an S3- or Swift-compliant gateway, you can choose to use something more economical, like traditional Serial Advanced Technology Attachment (SATA) drives.

One of the most important steps in a successful Ceph deployment is identifying a price-to-performance profile suitable for the storage cluster's use case and workload. It is important to choose the correct hardware for the use case. For example, choosing IOPS-optimized hardware for a cold storage application increases hardware

costs unnecessarily. Whereas, choosing capacity-optimized hardware for its more attractive price point in an IOPS-intensive workload will likely lead to unhappy users complaining about slow performance.

Use cases, cost versus benefit performance tradeoffs, and data durability are the primary considerations that help develop a sound storage strategy.

Use cases

Ceph provides massive storage capacity, and it supports numerous use cases, such as:

- The Ceph Block Device client is a leading storage backend for cloud platforms that provides limitless storage for volumes and images with high performance features like copy-on-write cloning.
- The Ceph Object Gateway client is a leading storage backend for cloud platforms that provides a RESTful S3-compliant and Swift-compliant object storage for objects like audio, bitmap, video, and other data.
- The Ceph File System for traditional file storage.

Cost vs. benefit of performance

Faster is better. Bigger is better. High durability is better. However, there is a price for each superlative quality, and a corresponding cost versus benefit tradeoff. Consider the following use cases from a performance perspective: SSDs can provide very fast storage for relatively small amounts of data and journaling. Storing a database or object index can benefit from a pool of very fast SSDs, but proves too expensive for other data. SAS drives with SSD journaling provide fast performance at an economical price for volumes and images. SATA drives without SSD journaling provide cheap storage with lower overall performance. When you create a CRUSH hierarchy of OSDs, you need to consider the use case and an acceptable cost versus performance tradeoff.

Data durability

In large scale storage clusters, hardware failure is an expectation, not an exception. However, data loss and service interruption remain unacceptable. For this reason, data durability is very important. Ceph addresses data durability with multiple replica copies of an object or with erasure coding and multiple coding chunks. Multiple copies or multiple coding chunks present an additional cost versus benefit tradeoff: it is cheaper to store fewer copies or coding chunks, but it can lead to the inability to service write requests in a degraded state. Generally, one object with two additional copies, or two coding chunks can allow a storage cluster to service writes in a degraded state while the storage cluster recovers.

Replication stores one or more redundant copies of the data across failure domains in case of a hardware failure. However, redundant copies of data can become expensive at scale. For example, to store 1 petabyte of data with triple replication would require a cluster with at least 3 petabytes of storage capacity.

Erasure coding stores data as data chunks and coding chunks. In the event of a lost data chunk, erasure coding can recover the lost data chunk with the remaining data chunks and coding chunks. Erasure coding is substantially more economical than replication. For example, using erasure coding with 8 data chunks and 3 coding chunks provides the same redundancy as 3 copies of the data. However, such an encoding scheme uses approximately 1.5 times the initial data stored compared to 3 times with replication.

The CRUSH algorithm aids this process by ensuring that Ceph stores additional copies or coding chunks in different locations within the storage cluster. This ensures that the failure of a single storage device or host does not lead to a loss of all of the copies or coding chunks necessary to preclude data loss. You can plan a storage strategy with cost versus benefit tradeoffs, and data durability in mind, then present it to a Ceph client as a storage pool.

Important:

- ONLY the data storage pool can use erasure coding. Pools storing service data and bucket indexes use replication.
- Ceph’s object copies or coding chunks make RAID solutions obsolete. Do not use RAID, because Ceph already handles data durability, a degraded RAID has a negative impact on performance, and recovering data using RAID is substantially slower than using deep copies or erasure coding chunks.

Deployment Guidelines

An important aspect of designing a storage cluster is to determine if the storage cluster will be in one data center site or span multiple data center sites. Use this information to learn more about considerations for setting up Ceph Object Gateway.

Multi-site storage clusters benefit from geographically distributed failover and disaster recovery, such as long-term power outages, earthquakes, hurricanes, floods or other disasters. Additionally, multi-site storage clusters can have an active-active configuration, which can direct client applications to the closest available storage cluster. This is a good storage strategy for content delivery networks. Consider placing data as close to the client as possible. This is important for throughput-intensive workloads, such as streaming 4k video.

Important: Red Hat recommends identifying realm, zone group and zone names BEFORE creating Ceph's storage pools. Prepend some pool names with the zone name as a standard naming convention.

Reference

For more information, see "Multi-site configuration and administration" on page 857.

Administrative data storage

A Ceph Object Gateway stores administrative data in a series of pools that are defined in an instance's zone configuration. For example, the buckets, users, user quotas, and usage statistics that are discussed in the subsequent sections are stored in pools in the Ceph storage cluster.

By default, Ceph Object Gateway creates the following pools and maps them to the default zone.

- `.rgw.root`
- `.default.rgw.control`
- `.default.rgw.meta`
- `.default.rgw.log`
- `.default.rgw.buckets.index`
- `.default.rgw.buckets.data`
- `.default.rgw.buckets.non-ec`

Note: The `.default.rgw.buckets.index` pool is created only after the bucket is created in Ceph Object Gateway, while the `.default.rgw.buckets.data` pool is created after the data is uploaded to the bucket.

Consider creating these pools manually so you can set the CRUSH ruleset and the number of placement groups. In a typical configuration, the pools that store the Ceph Object Gateway's administrative data usually use the same CRUSH ruleset and fewer placement groups. This is because there are 10 pools for the administrative data.

Have the `.rgw.root` pool and the service pools use the same CRUSH hierarchy, and use at least `node` as the failure domain in the CRUSH rule. Use `replicated` for data durability, and NOT `erasure` for the `.rgw.root` pool, and the service pools.

The `mon_pg_warn_max_per_osd` setting warns you if you assign too many placement groups to a pool, `300` by default. You can adjust the value to suit your needs and the capabilities of your hardware where `n` is the maximum number of placement groups (PGs) per OSD.

```
mon_pg_warn_max_per_osd = n
```

Note: For service pools, including `.rgw.root`, the suggested PG count from the Ceph placement groups (PGs) per pool calculator is substantially less than the target PGs per Ceph OSD. Also, ensure the number of Ceph OSDs is set in step 4 of the calculator.

Important: Garbage collection uses the `.log` pool with regular RADOS objects instead of OMAP. Use NVMe/SSD Ceph OSDs for the `.log` pool.

`.rgw.root` pool

The pool where the Ceph Object Gateway configuration is stored. This includes realms, zone groups, and zones. Conventionally, the pool name does not include the zone name as a prefix.

Service pools

The service pools store objects that are related to service control, garbage collection, logging, user information, and usage. Conventionally, the pool name includes the zone name as a prefix, as described in “Service pools with zone names” on page 796.

Table 103: Service pools with zone names

Zone name	Description
`.ZONE_NAME.rgw.control`	The control pool.
`.ZONE_NAME.log`	The log pool contains logs of all bucket, container, and object actions, such as create, read, update, and delete.
`.ZONE_NAME.rgw.buckets.index`	This pool stores index of the buckets.
`.ZONE_NAME.rgw.buckets.data`	This pool stores data of the buckets.
`.ZONE_NAME.rgw.meta`	The metadata pool stores user_keys and other critical metadata.
`.ZONE_NAME.rgw.meta:users.uid`	The user ID pool contains a map of unique user IDs.
`.ZONE_NAME.rgw.meta:users.keys`	The keys pool contains access keys and secret keys for each user ID.
`.ZONE_NAME.rgw.meta:users.email`	The email pool contains email addresses that are associated to a user ID.
`.ZONE_NAME.rgw.meta:users.swift`	The Swift pool contains the Swift subuser information for a user ID.

For more information, see “Pools” on page 861 and Storage strategies.

Index pool

Each node that hosts Object Storage Daemons (OSDs) for the Ceph Object Gateway needs at least one SSD to store the index pool.

When you select OSD hardware for the Ceph Object Gateway, each node that hosts OSDs must include at least one SSD to store the `.index` pool. SSDs can use a SATA, SAS, or NVMe interface. NVMe is an interface, not a separate type of storage medium, and NVMe drives are SSDs.

Use enterprise-class SSDs that have Power Loss Protection (PLP), whatever the interface. Read Intensive class SSDs usually suffice, and Mixed Use class SSDs also work. Ceph Object Gateway workloads are often well served by QLC SSDs, which can cost less than HDDs.

If you use HDDs for bulk object storage in the `.buckets` pools, place the index pool only on OSDs that are all SSD, or on OSDs that use an SSD to offload the BlueStore write-ahead log (WAL) and database (DB) data to a faster device.

For Red Hat Ceph Storage running BlueStore, Red Hat recommends deploying an NVMe SSD as a `block.db` device, rather than as a separate pool.

The Ceph Object Gateway writes index data only into an object map (OMAP). OMAP data for BlueStore resides on the `block.db` device on an OSD. When an NVMe SSD functions as a `block.db` device for an HDD OSD and HDD OSDs back the index pool, Ceph writes the index data only to the `block.db` device. If you size the `block.db` partition or logical volume at 4% of block, this configuration is all that BlueStore needs.

Restriction: Red Hat does not support HDD devices for index pools. For more information on supported configurations, see Red Hat Ceph Storage: Supported configurations.

An index entry is approximately 200 bytes of data for each S3 or Swift object, stored as an OMAP in `rocksdb`. While this is a trivial amount of data, some uses of the Ceph Object Gateway can result in tens or hundreds of millions of objects in a single bucket. By default, Ceph splits each bucket index into 11 shards. When a bucket grows to millions of objects, Ceph automatically increases the shard count.

Sharding affects hardware selection because operations on large buckets can traverse many shards, which amplifies the shortcomings of slow OSD media. You can map the index pool to a Controlled Replication Under Scalable Hashing (CRUSH) hierarchy of high performance storage media. The reduced latency provides a dramatic performance improvement when buckets contain very large numbers of objects.

If you use dedicated SSD OSDs, you can also place the `.log` pool and, especially, the `.meta` pool on those OSDs. In this case, constrain every CRUSH rule to either the `hdd` or the `ssd` CRUSH device class. If a pool mixes OSD device classes, its performance is limited to that of the slowest media.

Another strategy for HDD-backed `.buckets` pools is to deploy hybrid OSDs, which offload the WAL and DB to an external, often shared, faster drive. Follow these guidelines:

- Share each enterprise SATA or SAS SSD among at most 5 HDD OSDs. A server with 20 HDDs for bucket data needs at least 4 SSDs for WAL and DB offload.
- Share each enterprise NVMe SSD among at most 10 HDD OSDs. A server with 20 HDDs for bucket data needs at least 2 NVMe SSDs for WAL and DB offload.
- Provide WAL and DB offload space of at least 4% of the size of the slower data device for each OSD. For a 10 TB HDD, this is at least 400 GB. If an NVMe SSD serves 10 HDD OSDs of 10 TB each, the NVMe SSD needs at least 4 TB of capacity.

Hybrid OSDs also accelerate other OSD metadata operations and spread the index pool across more OSDs for more parallelism. Because bucket indexes are OMAP data, they perform as well on hybrid HDD and SSD OSDs as on dedicated SSD OSDs.

If you need SSD OSDs for other purposes, the index pool can usually share those OSDs. For example, an intensive Ceph Object Gateway deployment can use a modestly sized default storage class on replicated SSD OSDs for Reliable Autonomic Distributed Object Store (RADOS) HEAD objects and small objects, and place larger and tail objects on erasure-coded storage, QLC SSDs, or HDDs.

Important: In a production cluster, a typical OSD node has at least one SSD for storing the OSD journal and the index pool or `block.db` device, which use separate partitions or logical volumes for the same physical drive.

Data pool

The data pool is where the Ceph Object Gateway stores the object data for a particular storage policy. The data pool has a full complement of placement groups (PGs), not the reduced number of PGs for service pools. Consider using erasure coding for the data pool, as it is substantially more efficient than replication, and can significantly reduce the capacity requirements while maintaining data durability.

To use erasure coding, create an erasure code profile. See *Erasure code profiles*

Important: Choosing the correct profile is important because you cannot change the profile after you create the pool. To modify a profile, you must create a new pool with a different profile and migrate the objects from the old pool to the new pool.

The default configuration is two data chunks(**k**) and two encoding chunks(**m**), which means only one OSD can be lost. For higher resiliency, consider a larger number of data and encoding chunks. For example, some large scale systems use 8 data chunks and 3 encoding chunks, which allows 3 OSDs to fail without losing data.

Important: Each data and encoding chunk SHOULD get stored on a different node or host at a minimum. For smaller storage clusters, this makes using `rack` impractical as the minimum CRUSH failure domain for a larger number of data and encoding chunks. Consequently, it is common for the data pool to use a separate CRUSH hierarchy with `host` as the minimum CRUSH failure domain. Red Hat recommends `host` as the minimum failure domain. If erasure code chunks get stored on Ceph OSDs within the same host, a host failure, such as a failed journal or network card, could lead to data loss.

To create a data pool, run the `ceph osd pool create` command with the pool name, the number of PGs and PGPs, the `erasure` data durability method, the erasure code profile, and the name of the rule.

Data extra pool

The `data_extra_pool` is for data that cannot use erasure coding. For example, multi-part uploads allow uploading a large object, such as a movie in multiple parts. These parts must first be stored without erasure coding. Erasure coding applies to the whole object, not the partial uploads.

Note: The placement group (PG) per Pool Calculator recommends a smaller number of PGs per pool for the `data_extra_pool`; however, the PG count is approximately twice the number of PGs as the service pools and the same as the bucket index pool.

To create a data extra pool, run the `ceph osd pool create` command with the pool name, the number of PGs and PGPs, the `replicated` data durability method, and the name of the rule. For example:

```
# ceph osd pool create .us-west.rgw.buckets.non-ec 64 64 replicated rgw-service
```

Developing CRUSH hierarchies

As a storage administrator, when deploying a Ceph storage cluster and an object gateway, typically the Ceph Object Gateway has a default zone group and zone. The Ceph storage cluster will have default pools, which in turn will use a CRUSH map with a default CRUSH hierarchy and a default CRUSH rule.

Important: The default `rbd` pool can use the default CRUSH rule. **DO NOT** delete the default rule or hierarchy if Ceph clients have used them to store client data.

Production gateways typically use a custom realm, zone group and zone named according to the use and geographic location of the gateways. Additionally, the Ceph storage cluster will have a CRUSH map that has multiple CRUSH hierarchies.

- **Service Pools:** At least one CRUSH hierarchy will be for service pools and potentially for data. The service pools include `.rgw.root` and the service pools associated with the zone. Service pools typically fall under a single CRUSH hierarchy, and use replication for data durability. A data pool may also use the CRUSH hierarchy, but the pool will usually be configured with erasure coding for data durability.
- **Index:** At least one CRUSH hierarchy **SHOULD** be for the index pool, where the CRUSH hierarchy maps to high performance media, such as SSD or NVMe drives. Bucket indices can be a performance bottleneck. Red Hat recommends to use SSD or NVMe drives in this CRUSH hierarchy. Create partitions for indices on SSDs or NVMe drives used for Ceph OSD journals. Additionally, an index should be configured with bucket sharding.
- **Placement Pools:** The placement pools for each placement target include the bucket index, the data bucket, and the bucket extras. These pools can fall under separate CRUSH hierarchies. Since the Ceph Object Gateway can support multiple storage policies, the bucket pools of the storage policies may be associated with different CRUSH hierarchies, reflecting different use cases, such as IOPS-optimized, throughput-optimized, and capacity-optimized. The bucket index pool **SHOULD** use its own CRUSH hierarchy to map the bucket index pool to higher performance storage media, such as SSD or NVMe drives.

Creating CRUSH roots

From the command line on the administration node, create CRUSH roots in the CRUSH map for each CRUSH hierarchy. There **MUST** be at least one CRUSH hierarchy for service pools that may also potentially serve data storage pools. There **SHOULD** be at least one CRUSH hierarchy for the bucket index pool, mapped to high performance storage media, such as SSDs or NVMe drives.

In the following examples, the hosts named `data0`, `data1`, and `data2` use extended logical names, such as `data0-sas-ssd`, `data0-index`, and so forth in the CRUSH map, because there are multiple CRUSH hierarchies pointing to the same physical hosts.

A typical CRUSH root might represent nodes with SAS drives and SSDs for journals. For example:

```
##
# SAS-SSD ROOT DECLARATION
##

root sas-ssd {
  id -1   # do not change unnecessarily
  # weight 0.000
  alg straw
  hash 0  # rjenkins1
  item data2-sas-ssd weight 4.000
  item data1-sas-ssd weight 4.000
  item data0-sas-ssd weight 4.000
}
```

A CRUSH root for bucket indexes **SHOULD** represent high performance media, such as SSD or NVMe drives. Consider creating partitions on SSD or NVMe media that store OSD journals. For example:

```
##
# INDEX ROOT DECLARATION
##

root index {
  id -2    # do not change unnecessarily
  # weight 0.000
  alg straw
  hash 0  # rjenkins1
  item data2-index weight 1.000
  item data1-index weight 1.000
  item data0-index weight 1.000
}
```

Using logical host names in a CRUSH map

CRUSH supports the notion of a storage device "class". The clusters with hosts or nodes that contain multiple classes of a storage device, such as NVMe, SSD, or HDD, use a single CRUSH hierarchy with device classes to distinguish different classes of a storage device.

In the CRUSH map, host names must be unique and used only once. When the host serves multiple CRUSH hierarchies and use cases, a CRUSH map may use logical host names instead of the actual host name in order to ensure the host name is only used once. For example, a node may have multiple classes of drives such as SSDs, SAS drives with SSD journals, and SATA drives with co-located journals. For example, if the host name is `data2`, the CRUSH hierarchy might use logical names, such as `data2-sas-ssd` and `data2-index`:

```
host data2-sas-ssd {
  id -11   # do not change unnecessarily
  # weight 0.000
  alg straw
  hash 0  # rjenkins1
  item osd.0 weight 1.000
  item osd.1 weight 1.000
  item osd.2 weight 1.000
  item osd.3 weight 1.000
}
```

In the foregoing example, the host `data2` uses the logical name `data2-sas-ssd` to map the SAS drives with journals on SSDs into one hierarchy. The OSD IDs `osd.0` through `osd.3` represent SAS drives using SSD journals in a high throughput hardware configuration. These OSD IDs differ from the OSD ID in the following example.

In the following example, the host `data2` uses the logical name `data2-index` to map the SSD drive for a bucket index into a second hierarchy. The OSD ID `osd.4` represents an SSD drive or other high speed storage media used exclusively for a bucket index pool.

```
host data2-index {
  id -21   # do not change unnecessarily
  # weight 0.000
  alg straw
  hash 0  # rjenkins1
  item osd.4 weight 1.000
}
```

Important: When using logical host names, ensure that one of the following settings is present in the Ceph configuration file to prevent the OSD startup scripts from using the actual host names upon startup and thereby, failing to locate data in the CRUSH map.

When the CRUSH map uses logical host names, as in the foregoing examples, prevent the OSD startup scripts from identifying the hosts according to their actual host names at initialization. In the `[global]` section of the Ceph configuration file, add the following setting:

```
osd_crush_update_on_start = false
```

An alternative method of defining a logical host name is to define the location of the CRUSH map for each OSD in the `[osd.<ID>]` sections of the Ceph configuration file. This will override any locations the OSD startup script defines. From the foregoing examples, the entries might look like the following:

```
[osd.0]
osd crush location = "host=data2-sas-ssd"

[osd.1]
osd crush location = "host=data2-sas-ssd"

[osd.2]
osd crush location = "host=data2-sas-ssd"

[osd.3]
osd crush location = "host=data2-sas-ssd"

[osd.4]
osd crush location = "host=data2-index"
```

Important: If one of the foregoing approaches is not taken when a CRUSH map uses logical host names rather than actual host names, on restart, the Ceph Storage Cluster assumes that the OSDs map to the actual host names, the actual host names are not found in the CRUSH map, and Ceph Storage Cluster clients will not find the OSDs and their data.

Creating CRUSH rules

Like the default CRUSH hierarchy, the CRUSH map also contains a default CRUSH rule. For each CRUSH hierarchy, create a CRUSH rule.

Note: The default `rbd` pool may use this rule. Do not delete the default rule if other pools have used it to store customer data.

- For general details on CRUSH rules, see “CRUSH rules” on page 126
- To manually edit a CRUSH map, see “Edit a CRUSH map” on page 135
- For more information about administrating, see “CRUSH admin overview” on page 111

The following example illustrates a rule for the CRUSH hierarchy that will store the service pools, including `.rgw.root`. In this example, the root `sas-ssd` serves as the main CRUSH hierarchy. It uses the name

rgw-service to distinguish itself from the default rule. The step take sas-ssd line tells the pool to use the sas-ssd root created in "Creating CRUSH roots" on page 799 whose child buckets contain OSDs with SAS drives and high performance storage media, such as SSD or NVMe drives, for journals in a high throughput hardware configuration. The type rack portion of step chooseleaf is the failure domain. In this example, the failure domain is rack.

```
##
# SERVICE RULE DECLARATION
##

rule rgw-service {
 type replicated
 min_size 1
 max_size 10
 step take sas-ssd
 step chooseleaf firstn 0 type rack
 step emit
}
```

Note: In the following example, if data gets replicated three times, there should be at least three racks in the cluster containing a similar number of OSD nodes.

Tip: The type replicated setting has nothing to do with data durability, the number of replicas, or the erasure coding. Only replicated is supported.

The following example illustrates a rule for the CRUSH hierarchy that will store the data pool. In this example, the root sas-ssd serves as the main CRUSH hierarchy—the same CRUSH hierarchy as the service rule. It uses rgw-throughput to distinguish itself from the default rule and rgw-service. The step take sas-ssd line tells the pool to use the sas-ssd root created in "Creating CRUSH roots" on page 799, whose child buckets contain OSDs with SAS drives and high performance storage media, such as SSD or NVMe drives, in a high throughput hardware configuration. The type host portion of step chooseleaf is the failure domain. In the following example, it is a host. Notice that the rule uses the same CRUSH hierarchy, but a different failure domain.

```
##
# THROUGHPUT RULE DECLARATION
##

rule rgw-throughput {
 type replicated
 min_size 1
 max_size 10
 step take sas-ssd
 step chooseleaf firstn 0 type host
 step emit
}
```

Note: In the following example, if the pool uses erasure coding with a larger number of data and encoding chunks than the default, there should be at least as many racks in the cluster containing a similar number of OSD nodes to facilitate the erasure coding chunks. For smaller clusters, this may not be practical, so the foregoing example uses host as the CRUSH failure domain.

The following example illustrates a rule for the CRUSH hierarchy that stores the index pool. In this example, the root index serves as the main CRUSH hierarchy. It uses rgw-index to distinguish itself from rgw-service and rgw-throughput. The step take index line tells the pool to use the index root created in *Creating CRUSH roots*, whose child buckets contain high performance storage media, such as SSD or NVMe drives, or partitions on SSD or NVMe drives that also store OSD journals. The type rack portion of step chooseleaf is the failure domain. In the following example, it is a rack.

```
##
# INDEX RULE DECLARATION
##

rule rgw-index {
```

```
 type replicated
 min_size 1
 max_size 10
 step take index
 step chooseleaf firstn 0 type rack
 step emit
}
```

CRUSH Multi-Step Retry (MSR) rules

In Ceph, the **CRUSH** algorithm manages data placement across the cluster. However, conventional CRUSH rules have limitations when handling `out` OSDs (Object Storage Daemons) in multi-step configurations. Specifically, when an OSD is marked as `out` it is unavailable and conventional CRUSH rules cannot retry earlier steps in the rule, leading to potential issues such as uneven load distribution and undersized Placement Groups (PGs).

In standard CRUSH rules, if an OSD in a multi-step rule is marked as "out", subsequent steps may fail to find suitable OSDs for data placement. This can lead to:

Uneven load distribution

When an OSD is marked out on a particular host, other OSDs on the same host may become over-utilized, resulting in load imbalances.

Undersized placement groups (PGs)

If all OSDs on a host are marked out, the PGs mapped to that host may fail to find a valid location for data, causing them to be undersized or unable to map correctly.

To address these limitations, CRUSH MSR (Multi-Step Retry) is introduced. CRUSH MSR improves data remapping and load balancing by retrying each step of the CRUSH rule sequence whenever an `out` OSD is encountered. This ensures that data placement remains balanced, even when some OSDs are unavailable.

Consider an erasure-coded pool with an 8+6 coding scheme. The goal is to place 16 OSDs across four hosts, allowing the system to tolerate the loss of one entire host and one OSD on another. Without MSR, out OSDs on one host would result in PGs remapping to other OSDs on the same host, potentially causing load imbalance.

Example

```
rule ecpool-86 {
    type msr_indep
    step take default class hdd
    step choosemsr 4 type host
    step choosemsr 4 type osd
    step emit
}
```

In this example:

- `take defaultclass hdd` specifies the root bucket.
- `choosemsr 4 type host` selects 4 hosts, retrying previous steps if any host is unavailable.
- `choosemsr 4 type osd` selects 4 OSD from each chosen host, retrying as needed to ensure all hosts are utilized.
- `emit` finalizes the rule, mapping the PGs across chosen OSDs.

Before implementing CRUSH MSR, there are a few key considerations:

Compatibility

Before implementing CRUSH MSR ensure that your Red Hat Ceph Storage version supports CRUSH MSR. Older versions do not support this feature. Enable MSR for erasure-coded pools by restricting the cluster to support only Squid and later version clients.

```
ceph osd set-require-min-compat-client squid
```

Example for creating an MSR-based EC pool with a 4 node count

Create an MSR-based erasure coded pool for a 4 node count by creating a new erasure coding profile and then creating a pool with the new profile.

1. Create a new the erasure coding profile.
2. Create pool with the above profile.

The following procedure is an example to create a MSR-based Erase-Coded Pool for 4 Node Count.

1. Create an erasure code profile that defines the desired data (k) and parity (m) chunks, failure domain, and chunk distribution parameters.

```
ceph osd erasure-code-profile set ec86-profile \
   k=8 \
   m=6 \
   crush-failure-domain=host \
   crush-osds-per-failure-domain=4 \
   crush-num-failure-domains=4
```

   The following lists the command examples and their descriptions:

   k=8,m=6

   Defines an 8+6 erasure coding scheme, with eight data chunks and six parity chunks.

   crush-failure-domain=host

   Sets each host as a failure domain.

   crush-osds-per-failure-domain=4

   Allows each host to store up to four chunks.

   crush-num-failure-domains=4

   Limits the mapping to four hosts, each with multiple OSDs, to meet the resilience requirements.

2. Verify that the profile and the CRUSH rule were created successfully in one of the following ways.
   - List and check the new profile created.

```
ceph osd erasure-code-profile ls
ceph osd erasure-code-profile get PROFILE_NAME
```

   - List and check the CRUSH rule created.

```
ceph osd crush rule ls
ceph osd crush rule dump RULE_NAME
```

3. Create the erasure-coded pool with the MSR rule. This command creates the ec86-pool by using the `ec86-profile` profile and the custom MSR rule `ecpool-86`.

```
ceph osd pool create ec86-pool erasure ec86-profile
```

4. Verify that the pool was created with the correct CRUSH rule profile.

```
ceph osd pool ls detail
```

Multi-site considerations

A Ceph Object Gateway multi-site configuration requires at least two Red Hat Ceph Storage clusters, and at least two Ceph Object Gateway instances, one for each Red Hat Ceph Storage cluster. Typically, the two Red Hat Ceph Storage clusters are in geographically separate locations; however, this same multi-site configuration can work on two Red Hat Ceph Storage clusters at the same physical site.

Multi-site configurations require a primary zone group and a primary zone. Additionally, each zone group requires a primary zone. Zone groups might have one or more secondary zones.

Configure multi-site either through the CLI or through the dashboard. See Configuring a multi-site for more details.

> **Important:** The primary zone within the primary zone group of a realm is responsible for storing the primary copy of the realm's metadata, including users, quotas, and buckets. This metadata gets synchronized to secondary zones and secondary zone groups automatically. Metadata operations issued with the **`radosgw-admin`** command line interface (CLI) *MUST* be issued on a node within the primary zone of the primary zone group to ensure that they synchronize to the secondary zone groups and zones.

Currently, it is possible to issue metadata operations on secondary zones and zone groups, but it is *NOT* recommended because they *WILL NOT* be synchronized, which can lead to fragmentation of the metadata.

The following diagrams illustrate the possible one and two realm configurations in multi-site Ceph Object Gateway environments.

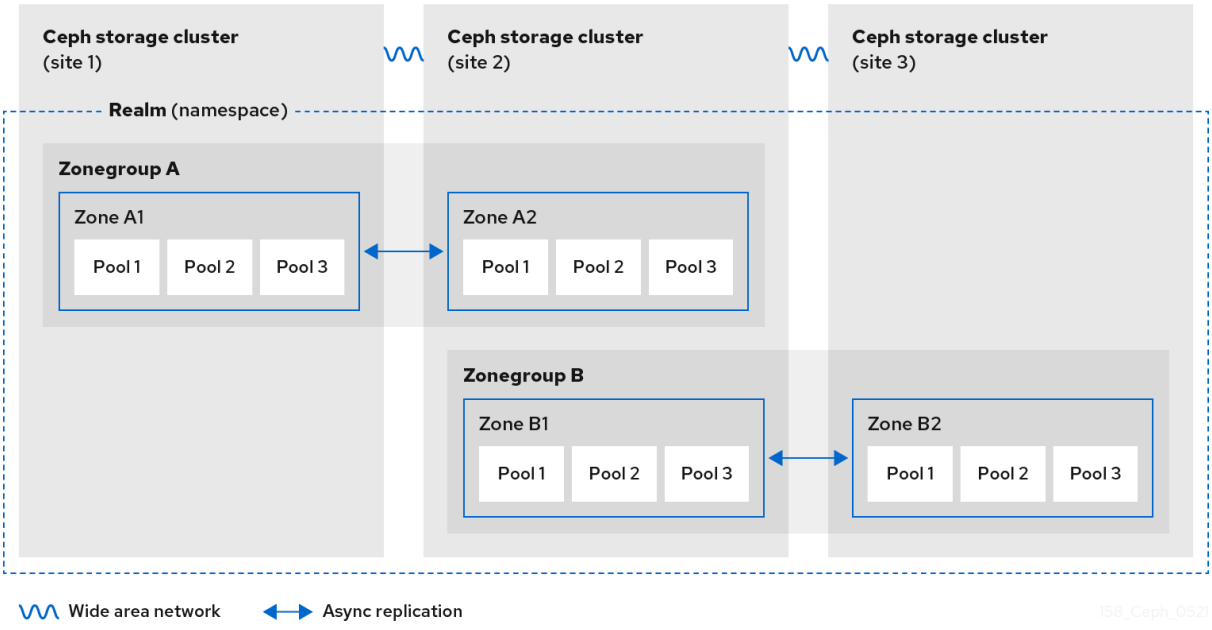


Figure 83: One realm configuration

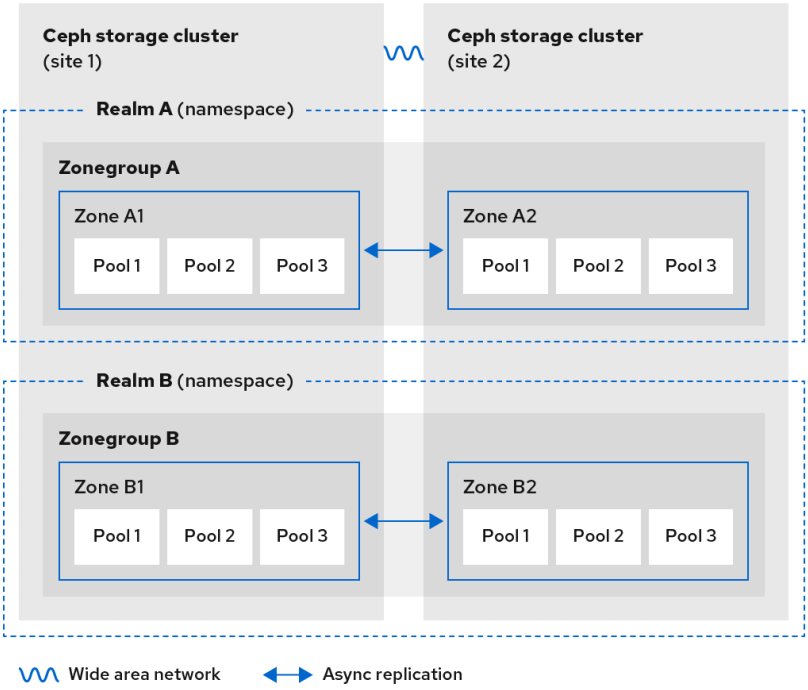


Figure 84: Two realm configuration

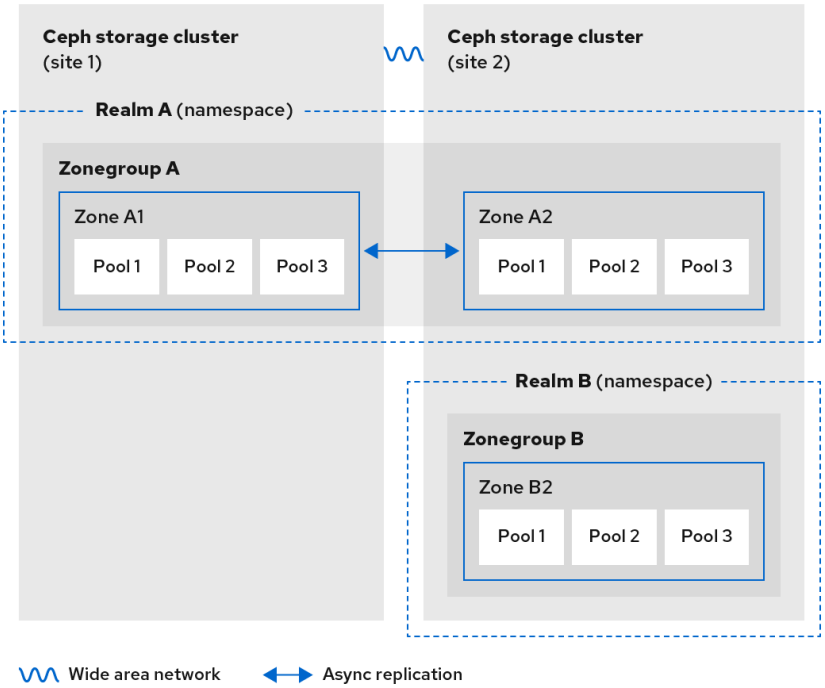


Figure 85: Two realms variant

Storage sizing

One of the most important factors in designing a cluster is to determine the storage requirements (sizing). Ceph Storage is designed to scale into petabytes and beyond.

The following examples are common sizes for Ceph storage clusters.

- **Small:** 250 terabytes
- **Medium:** 1 petabyte
- **Large:** 2 petabytes or more

Sizing includes current needs and near future needs. Consider the rate at which the gateway client will add new data to the cluster. That can differ from use-case to use-case. For example, recording 4k videos or storing medical images can add significant amounts of data faster than less storage-intensive information, such as financial market data. Additionally, consider that the data durability methods, such as replication versus erasure coding, can have a significant impact on the storage media required.

Reference

For more information, see the following:

- Hardware
-

Storage density

An important aspect of Ceph's design, includes storage density.

Generally, a storage cluster stores data across at least 10 nodes to ensure reasonable performance when replicating, backfilling, and recovery. If a node fails, with at least 10 nodes in the storage cluster, only 10% of the data has to move to the surviving nodes. If the number of nodes is substantially less, a higher percentage of the data must move to the surviving nodes. Additionally, the `full_ratio` and `near_full_ratio` options need to be set to accommodate a node failure to ensure that the storage cluster can write data. For this reason, it is important to consider storage density. Higher storage density is not necessarily a good idea.

Another factor that favors more nodes over higher storage density is erasure coding. When writing an object using erasure coding and using `node` as the minimum CRUSH failure domain, the Ceph storage cluster will need

as many nodes as data and coding chunks. For example, a cluster using `k=8, m=3` should have at least 11 nodes so that each data or coding chunk is stored on a separate node.

Hot-swapping is also an important consideration. Most modern servers support drive hot-swapping. However, some hardware configurations require removing more than one drive to replace a drive. Red Hat recommends avoiding such configurations, because they can bring down more Ceph OSDs than required when swapping out failed disks.

Disks for the Ceph Monitor nodes

Ceph Monitors use `rocksdb`, which is sensitive to synchronous write latency. Red Hat strongly recommends using SSD disks to store the Ceph Monitor data. Choose SSD disks that have sufficient sequential write and throughput characteristics.

Performance adjustments

To optimize the performance of the Ceph Object Gateway, several adjustments can be made in the configuration.

Following are the key factors to consider for performance adjustments.

- Load balancing
- OSD configuration
- Network optimization

Adjusting backfill and recovery settings

Backfill and recovery operations can affect client I/O performance during cluster expansion or recovery. Adjust the backfill and recovery configuration values to help balance recovery operations and client workload performance.

> **Note:** When using the mClock scheduler, enable recovery setting overrides before modifying backfill and recovery value.

1. Enable recovery setting overrides:

```
ceph config set osd osd_mclock_override_recovery_settings true
```

2. Set the recovery and backfill configuration values:

```
ceph config set osd osd_max_backfills VALUE
ceph config set osd osd_recovery_max_active VALUE
ceph config set osd osd_recovery_op_priority VALUE
```

Data about Data: For more information about the mClock scheduler, see mClock OSD scheduler.

Adjusting the cluster map size

Adjusting the cluster map size in the Ceph configuration can help reduce memory consumption, when needed.

By default, the `ceph-osd` daemon caches 500 previous osdmaps. Even with deduplication, the map might consume a lot of memory per daemon. Tuning the cache size in the Ceph configuration might help reduce memory consumption significantly. For example:

```
[ceph: root@host01 /]# ceph config set global osd_map_message_max 10
[ceph: root@host01 /]# ceph config set osd osd_map_cache_size 20
[ceph: root@host01 /]# ceph config set osd osd_map_share_max_epochs 10
[ceph: root@host01 /]# ceph config set osd osd_pg_epoch_persisted_max_stale 10
```

The `ceph-manager` daemon handles PG queries, so the cluster map should not impact performance.

Adjusting scrubbing

By default, Ceph performs light scrubbing daily and deep scrubbing weekly. Adjust the scrubbing schedules if the scrubbing schedule takes place at inopportune times.

Light scrubbing checks object sizes and checksums to ensure that PGs are storing the same object data. Over time, disk sectors can go bad irrespective of object sizes and checksums. Deep scrubbing checks an object's content with that of its replicas to ensure that the actual contents are the same. In this respect, deep scrubbing ensures data integrity in the manner of `fsck`, but the procedure imposes an I/O penalty on the cluster. Even light scrubbing can impact I/O.

The default settings may allow Ceph OSDs to initiate scrubbing at inopportune times, such as peak operating times or periods with heavy loads. End users may experience latency and poor performance when scrubbing operations conflict with end user operations.

To prevent end users from experiencing poor performance, Ceph provides a number of scrubbing settings that can limit scrubbing to periods with lower loads or during off-peak hours. For more information, see Scrubbing the OSD.

If the cluster experiences high loads during the day and low loads late at night, consider restricting scrubbing to night time hours. For example,

```
[osd]
osd_scrub_begin_hour = 23   #23:01H, or 10:01PM.
osd_scrub_end_hour = 6      #06:01H or 6:01AM.
```

If time constraints are not an effective method of determining a scrubbing schedule, consider using the **`osd_scrub_load_threshold`** command. The default value is `0.5`, but can be modified for low load conditions. For example,

```
[osd]
osd_scrub_load_threshold = 0.25
```

Increasing scalability

Improve scalability by editing Ceph Object Gateway configuration parameters.

For more information about configuring parameters, see "Configuration reference" on page 1088.

Increase the thread pool

Change the size of the thread pool by editing the value of the **`rgw_thread_pool_size`** parameter.

> **Note:** The `beast` frontend is not restricted by the thread pool size to accept new connections.

For example,

```
rgw_thread_pool_size = 512
```

Increase the maximum number of allowed unsent I/O requests

Change the maximum number of allowed unsent I/O requests by editing the value of the **`--objecter_inflight_ops`** parameter. The **`--objecter_inflight_ops`** parameter is used for client traffic control.

For example,

```
objecter_inflight_ops = 24576
```

Tuning considerations for the Linux kernel when running Ceph

Tune the operating system, specifically for limits and memory allocation.

Production Red Hat Storage clusters generally benefit from tuning the operating system, specifically around limits and memory allocation. Ensure that adjustments are set for all hosts within the storage cluster. For more guidance, contact Red Hat Support.

Important: Operating System swap must not be enabled. Configuring the operating system to use swap to provide additional virtual memory for daemons is not recommended for modern systems and can result in degraded performance.

Increasing the file descriptors

The Ceph Object Gateway can hang if it runs out of file descriptors. You can modify the `/etc/security/limits.conf` file on Ceph Object Gateway hosts to increase the file descriptors for the Ceph Object Gateway.

```
ceph       soft    nofile     unlimited
```

Adjusting the `ulimit` value for large storage clusters

Create a `/etc/security/limits.d/50-ceph.conf` file on each host that runs administrative commands, when running Ceph administrative commands on large storage clusters. An example of a large storage cluster is one with 1024 or more Ceph OSDs.

Note: The root user's `ulimit` value is already set to `unlimited` by default on Red Hat Enterprise Linux.

Include the following contents to the configuration file:

```
USER_NAME       soft    nproc     unlimited
```

Replace *USER_NAME* with the name of the non-root user account that runs the Ceph administrative commands.

Advantages of colocating Ceph daemons

Colocate containerized Ceph daemons on the same host to optimize resource usage, reduce hardware costs, and improve performance.

Colocating Ceph daemons helps you achieve better efficiency and scalability. This topic explains the advantages, best practices, and examples for colocating daemons using YAML files or the command-line interface (CLI).

The following are advantages of colocating Red Hat Ceph Storage daemons:

- Significantly improves total cost of ownership (TCO) for small-scale deployments.
- Increases overall performance.
- Reduces the number of physical hosts in a minimum configuration.
- Optimizes resource usage.
- Simplifies Red Hat Ceph Storage upgrades.

For detailed information about daemon-specific colocation considerations, see Red Hat Ceph Storage: Supported configurations. Be sure to avoid single points of failure and to distribute CPU and memory demands evenly.

"Daemon placement example" on page 809 provides an example configuration.

Note:

- The only clusters with 3 Ceph Monitors should be those that only have 3 or 4 nodes total.
- The Ceph Monitor (`ceph-mon`) and Ceph Manager (`ceph-mgr`) work closely together and therefore are not considered two separate daemons for the purposes of colocation.

Table 104: Daemon placement example			
Host Name	**Daemon**	**Daemon**	**Daemon**
host01	OSD	Monitor and Manager	Prometheus
host02	OSD	Monitor and Manager	RGW
host03	OSD	Monitor and Manager	RGW
host04	OSD	Metadata Server, Monitor, and Manager	Prometheus or Monitoring Stack
host5	OSD	Metadata Server, Monitor, and Manager	Prometheus or Monitoring Stack

Colocate Ceph daemons, either by using a service specification YAML file or with the CLI.

Colocating Ceph daemons with a service specification YAML file

Colocate Ceph daemons by using a service specification YAML file, based on host labels. Using a service specification file is best practice as it avoids explicitly placing daemons on specific hostnames, increasing resilience during host changes. For more information, see “Labeling hosts” on page 229.
For example,

```
service_type: mon
placement:
  hosts:
   - host01
   - host02
   - host03
   - host04
   - host05

[ceph: root@host01 /]# ceph orch apply -i mon.yml
```

Colocating Ceph daemons with the CLI

Colocate Ceph daemons by using the CLI, by using the **ceph orch** command with the **--placement** option.
For example,

```
[ceph: root@host01 /]# ceph orch apply mon --placement="host01 host02 host03 host04
host05"
```

To increase performance, colocate the Ceph Object Gateway with Ceph OSD. To increase performance without incurring extra hardware costs, deploy two Ceph Object Gateway daemons per host.

“Figure: Colocated daemons” on page 810 and “Figure: Non-colocated daemons” on page 811 show the difference between storage clusters with colocated and non-colocated daemons.



Figure 86: Colocated daemons



Figure 87: Non-colocated daemons

For more information, see Managing services.

Related information

Red Hat Ceph Storage RGW deployment strategies and sizing guidance

Deployment

As a storage administrator, you can deploy the Ceph Object Gateway using the Ceph Orchestrator with the command line interface or the service specification. You can also configure multi-site Ceph Object Gateways, and remove the Ceph Object Gateway using the Ceph Orchestrator.

The `cephadm` command deploys the Ceph Object Gateway as a collection of daemons that manages a single-cluster deployment or a particular realm and zone in a multi-site deployment.

> **Note:** With `cephadm`, the Ceph Object Gateway daemons are configured using the Ceph Monitor configuration database instead of the `ceph.conf` file or the command line options. If the configuration is

not in the `client.rgw` section, then the Ceph Object Gateway daemons start up with default settings and bind to port `80`.

Warning: If you want `cephadm` to handle the setting of a realm and zone, specify the realm and zone in the service specification during the deployment of the Ceph Object Gateway. If you want to change that realm or zone at a later point, ensure to update and reapply the `rgw_realm` and `rgw_zone` parameters in the specification file. If you want to handle these options manually without `cephadm`, do not include them in the service specification. `cephadm` still deploys the Ceph Object Gateway daemons without setting the configuration option for which realm or zone the daemons should use. In this case, the update of the specification file is not necessary.

Prerequisites

- A running, and healthy Red Hat Ceph Storage cluster.
- Root-level access to all the nodes.
- Available nodes on the storage cluster.
- All the managers, monitors, and OSDs are deployed in the storage cluster.

Deploying the Ceph Object Gateway using the command line interface

Deploy the Ceph Object Gateway by using the Ceph Orchestrator with the **`ceph orch`** command in the command line interface.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to all the nodes.
- Hosts are added to the cluster.
- All manager, monitor, and OSD daemons are deployed.

Log in to the `cephadm` shell by using the **`cephadm shell`** to deploy Ceph Object Gateway daemons.

About this task

Deploy using one of the following methods:

- “Creating a realm, zone group, and zone” on page 624
- “Using an arbitrary service name for a single cluster deployment” on page 625
- “Using an arbitrary service name on a labeled set of hosts” on page 625

Creating a realm, zone group, and zone

Procedure

Create a realm, zone group, zone, and then use the placement specification with the hostname.

1. Create a realm.

```
radosgw-admin realm create --rgw-realm=REALM_NAME --default
```

For example,

```
[ceph: root@host01 /]# radosgw-admin realm create --rgw-realm=test_realm --default
```

2. Create a zone group.

```
radosgw-admin zonegroup create --rgw-zonegroup=ZONE_GROUP_NAME  --master --default
```

For example,

```
[ceph: root@host01 /]# radosgw-admin zonegroup create --rgw-zonegroup=default  --
master --default
```

3. Create a zone.

```
radosgw-admin zone create --rgw-zonegroup=ZONE_GROUP_NAME --rgw-zone=ZONE_NAME --
master --default
```

For example,

```
[ceph: root@host01 /]# radosgw-admin zone create --rgw-zonegroup=default --rgw-
zone=test_zone --master --default
```

4. Commit the changes.

```
radosgw-admin period update --rgw-realm=REALM_NAME --commit
```

For example,

```
[ceph: root@host01 /]# radosgw-admin period update --rgw-realm=test_realm --commit
```

5. Apply the changes by using the **ceph orch apply** command.

```
ceph orch apply rgw NAME [--realm=REALM_NAME] [--zone=ZONE_NAME] [--
zonegroup=ZONE_GROUP_NAME] --placement="NUMBER_OF_DAEMONS [HOST_NAME_1 HOST_NAME_2]"
```

For example,

```
[ceph: root@host01 /]# ceph orch apply rgw test --realm=test_realm --zone=test_zone --
zonegroup=default --placement="2 host01 host02"
```

Using an arbitrary service name for a single cluster deployment

Use an arbitrary service name to deploy two Ceph Object Gateway daemons for a single cluster deployment.

```
ceph orch apply rgw SERVICE_NAME
```

For example,

```
[ceph: root@host01 /]# ceph orch apply rgw foo
```

Using an arbitrary service name on a labeled set of hosts

Use an arbitrary service name on a labeled set of hosts.

```
ceph orch host label add HOST_NAME_1 LABEL_NAME
ceph orch host label add HOSTNAME_2 LABEL_NAME
ceph orch apply rgw SERVICE_NAME --placement="label:LABEL_NAME count-per-
host:NUMBER_OF_DAEMONS" --port=8000
```

NUMBER_OF_DAEMONS controls the number of Ceph Object Gateways that are deployed on each host. To achieve the highest performance without incurring any extra cost, set this value to 2.
For example,

```
[ceph: root@host01 /]# ceph orch host label add host01 rgw  # the 'rgw' label can be
anything
[ceph: root@host01 /]# ceph orch host label add host02 rgw
[ceph: root@host01 /]# ceph orch apply rgw foo --placement="label:rgw count-per-host:2" --
port=8000
```

What to do next

Verify the Ceph Object Gateway deployment.

- List the service, by using the **ceph orch ls** command.
  For example,

```
[ceph: root@host01 /]# ceph orch ls
```

- List the hosts, daemons, and processes.

```
ceph orch ps --daemon_type=DAEMON_NAME
```

  For example,

```
[ceph: root@host01 /]# ceph orch ps --daemon_type=rgw
```

Deploying the Ceph Object Gateway using the service specification

Deploy the Ceph Object Gateway using the service specification with either the default or the custom realms, zones, and zone groups.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the bootstrapped host.
- Hosts are added to the cluster.
- All Ceph Manager, Ceph Monitor, and Object Storage Daemon (OSD) daemons are deployed.

Procedure

1. Create a specification file.

```
touch rgw_spec.yaml
```

   For example,

```
[root@host01 ~]# touch rgw_spec.yaml
```

2. Configure S3 requests to wait for the duration defined in the **rgw_exit_timeout_secs** parameter for all outstanding requests to complete by setting **rgw_graceful_stop** to `true` during Ceph Object Gateway shutdown or restart.

```
ceph config set client.rgw rgw_graceful_stop true

ceph config set client.rgw rgw_exit_timeout_secs 120
```

   Important: In containerized deployments, an additional `extra_container_args` configuration of `--stop-timeout=120` (or the value of the **rgw_exit_timeout_secs** configuration, if not default) is also necessary in order for it to work as expected with the **ceph orch stop** or **ceph orch restart** commands.
   For example,

```
[root@host1 ~]$ cat rgw_spec.yaml
service_type: rgw
service_id: foo
placement:
  count_per_host: 1
  hosts:
    - rgw_node
spec:
  rgw_frontend_port: 8081
extra_container_args:
  - "--stop-timeout=120"
```

3. Edit the `rgw_spec.yaml` file to include the details for the default realm, zone, and zone group.

```
service_type: rgw
service_id: REALM_NAME.ZONE_NAME
placement:
  hosts:
    - HOST_NAME_1
    - HOST_NAME_2
  count_per_host: NUMBER_OF_DAEMONS
spec:
  rgw_realm: REALM_NAME
  rgw_zone: ZONE_NAME
  rgw_zonegroup: ZONE_GROUP_NAME
  rgw_frontend_port: FRONT_END_PORT

networks:
  - NETWORK_CIDR
```

   The *NUMBER_OF_DAEMONS* parameter controls the number of Ceph Object Gateways deployed on each host. To achieve the highest performance without incurring an additional cost, set this value to 2.

   For example,

```
service_type: rgw
service_id: default
placement:
  hosts:
    - host01
    - host02
    - host03
  count_per_host: 2
spec:
  rgw_realm: default
  rgw_zone: default
  rgw_zonegroup: default
  rgw_frontend_port: 1234

networks:
  - 192.169.142.0/24
```

4. Optional: Configure custom realm, zone, and zone group.

   a. Create the custom realm, zone, and zone group.

```
radosgw-admin realm create --rgw-realm=test_realm --default
radosgw-admin zonegroup create --rgw-zonegroup=test_zonegroup --default
radosgw-admin zone create --rgw-zonegroup=test_zonegroup --rgw-zone=test_zone --
default
radosgw-admin period update --rgw-realm=test_realm --commit
```

      For example,

```
[root@host01 ~]# radosgw-admin realm create --rgw-realm=test_realm --default
[root@host01 ~]# radosgw-admin zonegroup create --rgw-zonegroup=test_zonegroup --
default
[root@host01 ~]# radosgw-admin zone create --rgw-zonegroup=test_zonegroup --rgw-
zone=test_zone --default
[root@host01 ~]# radosgw-admin period update --rgw-realm=test_realm --commit
```

   b. Create the `rgw_spec.yaml` file with the custom details.
      For example,

```
service_type: rgw
service_id: test_realm.test_zone
placement:
  hosts:
    - host01
    - host02
    - host03
  count_per_host: 2
spec:
  rgw_realm: test_realm
  rgw_zone: test_zone
  rgw_zonegroup: test_zonegroup
  rgw_frontend_port: 1234
networks:
  - 192.169.142.0/24
```

5. Optional: Deploy the HAProxy concentrator for local Ceph Object Gateway load balancing.
To load balance requests across Ceph Object Gateway daemons on the same host, configure the service with a local HAProxy concentrator.

   > **Note:** The HAProxy concentrator does not currently support SSL.

   > **Note:** Use this configuration only when `count_per_host > 1`.

   For example,

   ```
   service_type: rgw
   service_id: foo
   service_name: rgw.foo
   placement:
     count_per_host: 2
     hosts:
       - host1
       - host2
   spec:
     concentrator: haproxy
     concentrator_frontend_port: 8080
     concentrator_monitor_port: 1967
     concentrator_monitor_user: admin
   ```

6. Optional: In a multi-site deployment, if you want to dedicate some Ceph Object Gateway instances to client IO only, disable multi-site sync traffic for those instances.
These instances handle client IO but do not take part in multi-site sync operations.

   For those instances, set **`disable_multisite_sync_traffic`** to `true` under the `spec` section of the Ceph Object Gateway specification file. The `cephadm` utility then sets the **`rgw_run_sync_thread`** configuration to `false` for those Ceph Object Gateway daemons. This setting prevents the daemons from taking part in sync operations.

   This automation eliminates the need to manually configure **`rgw_run_sync_thread`** for those Ceph Object Gateway daemons. To configure it manually, run the following command, where RGW_DAEMON_NAME is the name of the Ceph Object Gateway daemon:

   ```
   ceph config set RGW_DAEMON_NAME rgw_run_sync_thread false
   ```

   In the following example, the Ceph Object Gateway service `rgw.io` has **`disable_multisite_sync_traffic`** set to `true`, so it handles only client IO. The service `rgw.sync` takes part in multi-site sync operations because **`disable_multisite_sync_traffic`** is `false` by default.

```
[root@host01 ~]# cat rgw_spec.yaml
service_type: rgw
service_id: io
placement:
  hosts:
    - host01
    - host02
spec:
  rgw_realm: test_realm
  rgw_zone: test_zone
  rgw_zonegroup: test_zonegroup
  rgw_frontend_port: 1234
  disable_multisite_sync_traffic: true
---
service_type: rgw
service_id: sync
placement:
  hosts:
    - host03
    - host04
spec:
  rgw_realm: test_realm
  rgw_zone: test_zone
  rgw_zonegroup: test_zonegroup
  rgw_frontend_port: 1234
```

7. Mount the `rgw_spec.yaml` file under a directory in the container.

   Important: Every time you exit the `cephadm` shell, you must mount the file in the container before deploying the daemon.

   ```
   cephadm shell --mount rgw_spec.yaml:/var/lib/ceph/radosgw/rgw_spec.yaml
   ```

   For example,

   ```
   [root@host01 ~]# cephadm shell --mount rgw_spec.yaml:/var/lib/ceph/radosgw/
   rgw_spec.yaml
   ```

8. From within the `cephadm` shell, deploy the Ceph Object Gateway by using the service specification.

   ```
   ceph orch apply -i FILE_NAME.yaml
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch apply -i /var/lib/ceph/radosgw/rgw_spec.yaml
   ```

9. Optional: If you disabled multi-site sync traffic for some Ceph Object Gateway instances, make sure that only the sync instances are listed as zone and zone group endpoints.
   Other zones send sync requests to the zone and zone group endpoints. Do not list the client IO instances as endpoints. Set the endpoints to the sync instances only, and then commit the period.

   ```
   radosgw-admin zonegroup modify --rgw-zonegroup=ZONE_GROUP_NAME --endpoints=http://
   SYNC_HOST_1:PORT,http://SYNC_HOST_2:PORT
   radosgw-admin zone modify --rgw-zone=ZONE_NAME --endpoints=http://
   SYNC_HOST_1:PORT,http://SYNC_HOST_2:PORT
   radosgw-admin period update --commit
   ```

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin zonegroup modify --rgw-zonegroup=test_zonegroup
   --endpoints=http://host03:1234,http://host04:1234
   [ceph: root@host01 /]# radosgw-admin zone modify --rgw-zone=test_zone --
   endpoints=http://host03:1234,http://host04:1234
   [ceph: root@host01 /]# radosgw-admin period update --commit
   ```

Result

Verify that the Ceph Object Gateway service is deployed successfully:

- List the service:

```
ceph orch ls
```

- List the hosts, daemons, and processes:

```
ceph orch ps --daemon_type=DAEMON_NAME
```

For example,

```
[ceph: root@host01 /]# ceph orch ps --daemon_type=rgw
```

Deploying a multi-site Ceph Object Gateway using the Ceph Orchestrator

Ceph Orchestrator supports multi-site configuration options for the Ceph Object Gateway.

You can configure each object gateway to work in an active-active zone configuration allowing writes to a non-primary zone. The multi-site configuration is stored within a container called a realm.

The realm stores zone groups, zones, and a time period. The `rgw` daemons handle the synchronization eliminating the need for a separate synchronization agent, thereby operating with an active-active configuration.

You can also deploy multi-site zones using the command line interface (CLI).

> **Note:** The following configuration assumes at least two Red Hat Ceph Storage clusters are in geographically separate locations. However, the configuration also works on the same site.

Prerequisites

- At least two running Red Hat Ceph Storage clusters.
- At least two Ceph Object Gateway instances, one for each Red Hat Ceph Storage cluster.
- Root-level access to all the nodes.
- Nodes or containers are added to the storage cluster.
- All Ceph Manager, Monitor and OSD daemons are deployed.

Procedure

1. In the `cephadm` shell, configure the primary zone:
   a. Create a realm:

      Syntax

      ```
      radosgw-admin realm create --rgw-realm=REALM_NAME --default
      ```

      Example

      ```
      [ceph: root@host01 /]# radosgw-admin realm create --rgw-realm=test_realm --
      default
      ```

      If the storage cluster has a single realm, then specify the `--default` flag.

   b. Create a primary zone group:

      Syntax

      ```
      radosgw-admin zonegroup create --rgw-zonegroup=_ZONE_GROUP_NAME_ --
      endpoints=http://RGW_PRIMARY_HOSTNAME:RGW_PRIMARY_PORT_NUMBER_1 --master --
      default
      ```

      Example

```
[ceph: root@host01 /]# radosgw-admin zonegroup create --rgw-zonegroup=us --
endpoints=http://rgw1:80 --master --default
```

c. Create a primary zone:

Syntax

```
radosgw-admin zone create --rgw-zonegroup=PRIMARY_ZONE_GROUP_NAME --rgw-
zone=PRIMARY_ZONE_NAME --endpoints=http://
RGW_PRIMARY_HOSTNAME:RGW_PRIMARY_PORT_NUMBER_1 --access-key=_SYSTEM_ACCESS_KEY_
--secret=_SYSTEM_SECRET_KEY_
```

Example

```
[ceph: root@host01 /]# radosgw-admin zone create --rgw-zonegroup=us --rgw-
zone=us-east-1 --endpoints=http://rgw1:80 --access-key=LIPEYZJLTWXRKXS9LPJC --
secret-key=IsAje0AVDNXNw48LjMAimpCpI7VaxJYSnfD0FFKQ
```

d. Optional: Delete the default zone, zone group, and the associated pools.

> **Important:**
> Do not delete the default zone and its pools if you are using the default zone and zone group to store data. Also, removing the default zone group deletes the system user.
>
> To access old data in the `default` zone and zonegroup, use `--rgw-zone default` and `--rgw-zonegroup default` in `radosgw-admin` commands.

Example

```
[ceph: root@host01 /]# radosgw-admin zonegroup delete --rgw-zonegroup=default
[ceph: root@host01 /]# ceph osd pool rm default.rgw.log default.rgw.log --yes-i-
really-really-mean-it
[ceph: root@host01 /]# ceph osd pool rm default.rgw.meta default.rgw.meta --yes-
i-really-really-mean-it
[ceph: root@host01 /]# ceph osd pool rm default.rgw.control default.rgw.control
--yes-i-really-really-mean-it
[ceph: root@host01 /]# ceph osd pool rm default.rgw.data.root
default.rgw.data.root --yes-i-really-really-mean-it
[ceph: root@host01 /]# ceph osd pool rm default.rgw.gc default.rgw.gc --yes-i-
really-really-mean-it
```

e. Create a system user:

Syntax

```
radosgw-admin user create --uid=USER_NAME --display-name="USER_NAME" --access-
key=SYSTEM_ACCESS_KEY --secret=SYSTEM_SECRET_KEY --system
```

Example

```
[ceph: root@host01 /]# radosgw-admin user create --uid=zone.user --display-
name="Zone user" --system
```

Make a note of the `access_key` and `secret_key`.

f. Add the access key and system key to the primary zone:

Syntax

```
radosgw-admin zone modify --rgw-zone=PRIMARY_ZONE_NAME --access-key=ACCESS_KEY --
secret=SECRET_KEY
```

Example

```
[ceph: root@host01 /]# radosgw-admin zone modify --rgw-zone=us-east-1 --access-
key=NE48APYCAODEPLKBCZVQ --secret=u24GHQWRE3yxxNBnFBzjM4jn14mFIckQ4EKL6LoW
```

g. Commit the changes:

Syntax

```
radosgw-admin period update --commit
```

Example

```
[ceph: root@host01 /]# radosgw-admin period update --commit
```

h. Outside the `cephadm` shell, fetch the `FSID` of the storage cluster and the processes:

Example

```
[root@host01 ~]#  systemctl list-units | grep ceph
```

i. Start the Ceph Object Gateway daemon:

Syntax

```
systemctl start ceph-FSID@DAEMON_NAME
systemctl enable ceph-FSID@DAEMON_NAME
```

Example

```
[root@host01 ~]# systemctl start ceph-62a081a6-88aa-11eb-
a367-001a4a000672@rgw.test_realm.us-east-1.host01.ahdtsw.service
[root@host01 ~]# systemctl enable ceph-62a081a6-88aa-11eb-
a367-001a4a000672@rgw.test_realm.us-east-1.host01.ahdtsw.service
```

2. In the `cephadm` shell, configure the secondary zone.

a. Pull the primary realm configuration from the host:

Syntax

```
radosgw-admin realm pull --rgw-realm=PRIMARY_REALM --
url=URL_TO_PRIMARY_ZONE_GATEWAY --access-key=ACCESS_KEY --secret-key=SECRET_KEY
--default
```

Example

```
[ceph: root@host04 /]# radosgw-admin realm pull --rgw-realm=test_realm --
url=http://10.74.249.26:80 --access-key=LIPEYZJLTWXRKXS9LPJC --secret-
key=IsAje0AVDNXNw48LjMAimpCpI7VaxJYSnfD0FFKQ --default
```

b. Pull the primary period configuration from the host:

Syntax

```
radosgw-admin period pull --url=URL_TO_PRIMARY_ZONE_GATEWAY --access-
key=ACCESS_KEY --secret-key=SECRET_KEY
```

Example

```
[ceph: root@host04 /]# radosgw-admin period pull --url=http://10.74.249.26:80 --
access-key=LIPEYZJLTWXRKXS9LPJC --secret-
key=IsAje0AVDNXNw48LjMAimpCpI7VaxJYSnfD0FFKQ
```

c. Configure a secondary zone:

Syntax

```
radosgw-admin zone create --rgw-zonegroup=ZONE_GROUP_NAME
             --rgw-zone=SECONDARY_ZONE_NAME
             --access-key=SYSTEM_ACCESS_KEY --secret=SYSTEM_SECRET_KEY
             --endpoints=http://FQDN:80
             [--read-only]
```

Example

```
[ceph: root@host04 /]# radosgw-admin zone create --rgw-zonegroup=us --rgw-
zone=us-east-2 --endpoints=http://rgw2:80 --access-key=LIPEYZJLTWXRKXS9LPJC --
secret-key=IsAje0AVDNXNw48LjMAimpCpI7VaxJYSnfD0FFKQ
```

d. Optional: Delete the default zone.

> **Important:** Do not delete the default zone and its pools if you are using the default zone and zone group to store data. To access old data in the `default` zone and zonegroup, use `--rgw-zone default` and `--rgw-zonegroup default` in `radosgw-admin` commands.

Example

```
[ceph: root@host04 /]# radosgw-admin zone rm --rgw-zone=default
[ceph: root@host04 /]# ceph osd pool rm default.rgw.log default.rgw.log --yes-i-
really-really-mean-it
[ceph: root@host04 /]# ceph osd pool rm default.rgw.meta default.rgw.meta --yes-
i-really-really-mean-it
[ceph: root@host04 /]# ceph osd pool rm default.rgw.control default.rgw.control
--yes-i-really-really-mean-it
[ceph: root@host04 /]# ceph osd pool rm default.rgw.data.root
default.rgw.data.root --yes-i-really-really-mean-it
[ceph: root@host04 /]# ceph osd pool rm default.rgw.gc default.rgw.gc --yes-i-
really-really-mean-it
```

e. Update the Ceph configuration database:

Syntax

```
ceph config set SERVICE_NAME rgw_zone SECONDARY_ZONE_NAME
```

Example

```
[ceph: root@host04 /]# ceph config set rgw rgw_zone us-east-2
```

f. Commit the changes:

Syntax

```
radosgw-admin period update --commit
```

Example

```
[ceph: root@host04 /]# radosgw-admin period update --commit
```

g. Outside the `cephadm` shell, fetch the FSID of the storage cluster and the processes:

Example

```
[root@host04 ~]#  systemctl list-units | grep ceph
```

h. Start the Ceph Object Gateway daemon:

Syntax

```
systemctl start ceph-FSID@DAEMON_NAME
systemctl enable ceph-FSID@DAEMON_NAME
```

Example

```
[root@host04 ~]# systemctl start ceph-62a081a6-88aa-11eb-
a367-001a4a000672@rgw.test_realm.us-east-2.host04.ahdtsw.service
[root@host04 ~]# systemctl enable ceph-62a081a6-88aa-11eb-
a367-001a4a000672@rgw.test_realm.us-east-2.host04.ahdtsw.service
```

3. Optional: Deploy multi-site Ceph Object Gateways using the placement specification:

   Syntax

   ```
   ceph orch apply rgw NAME --realm=REALM_NAME --zone=PRIMARY_ZONE_NAME --
   placement="NUMBER_OF_DAEMONS HOST_NAME_1 HOST_NAME_2"
   ```

   Example

   ```
   [ceph: root@host04 /]# ceph orch apply rgw east --realm=test_realm --zone=us-east-1 --
   placement="2 host01 host02"
   ```

Verification

- Check the synchronization status to verify the deployment:

  Example

  ```
  [ceph: root@host04 /]# radosgw-admin sync status
  ```

Removing the Ceph Object Gateway using the Ceph Orchestrator

Remove the Ceph Object Gateway daemons using the **`ceph orch rm`** command.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to all the nodes.
- Hosts are added to the cluster.
- At least one Ceph Object Gateway daemon deployed on the hosts.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. List the service:

   Example

   ```
   [ceph: root@host01 /]# ceph orch ls
   ```

3. Remove the service:

   Syntax

   ```
   ceph orch rm SERVICE_NAME
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph orch rm rgw.test_realm.test_zone_bb
   ```

Verification

- List the hosts, daemons, and processes:

  Syntax

  ```
  ceph orch ps
  ```

Example

```
[ceph: root@host01 /]# ceph orch ps
```

Reference

For more information, see the following:

- Deploying the Ceph Object Gateway using the command line interface
- Deploying the Ceph Object Gateway using the service specification

Using the Ceph Manager `rgw` module

As a storage administrator, you can deploy Ceph Object Gateway, single site and multi-site, using the `rgw` module. It helps with bootstrapping and configuring Ceph Object realm, zonegroup, and the different related entities.

You can use the available tokens for the newly created or existing realms. This token is a base64 string that encapsulates the realm information and its primary zone endpoint authentication data.

In a multi-site configuration, these tokens can be used to pull a realm to create a secondary zone on a different cluster that syncs with the primary zone on the primary cluster by using the `rgw zone create` command.

Deploying Ceph Object Gateway using the `rgw` module

Bootstrapping the Ceph Object Gateway realm creates a new realm entity, a new zonegroup, and a new zone. The `rgw` module instructs the orchestrator to create and deploy the corresponding Ceph Object Gateway daemons.

Before you begin

- A running Red Hat Ceph Storage cluster.

About this task

Enable the `rgw` module by using the `ceph mgr module enable rgw` command. Pass the arguments in the CLI or use the YAML specification file to bootstrap the realm.

Procedure

1. Log in to the `cephadm` shell.

   ```
   cephadm shell
   ```

   For example,

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Enable the `rgw` module.

   ```
   ceph mgr module enable rgw
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph mgr module enable rgw
   ```

3. Bootstrap the Ceph Object Gateway realm by using either the CLI or the YAML specification file.
   - Use the command-line interface.

     ```
     ceph rgw realm bootstrap [--realm name REALM_NAME] [--zonegroup-
     name ZONEGROUP_NAME] [--zone-name ZONE_NAME] [--port PORT_NUMBER] [--
     placement HOSTNAME] [--start-radosgw]
     ```

     Foe example,

```
[ceph: root@host01 /]# ceph rgw realm bootstrap --realm-name myrealm --zonegroup-
name myzonegroup --zone-name myzone --port 5500 --placement="host01 host02" --
start-radosgw
Realm(s) created correctly. Please, use 'ceph rgw realm tokens' to get the token.
```

- Use YAML specification file.
  - As a root user, create the YAML file:

    ```
    rgw_realm: REALM_NAME
    rgw_zonegroup: ZONEGROUP_NAME
    rgw_zone: ZONE_NAME
    placement:
      hosts:
        - HOSTNAME_1
        - HOSTNAME_2
    ```

    For example,

    ```
    [root@host01 ~]# cat rgw.yaml

    rgw_realm: myrealm
    rgw_zonegroup: myzonegroup
    rgw_zone: myzone
    placement:
      hosts:
        - host01
        - host02
    ```

  - You can add *hostnames* to the zonegroup during realm bootstrap:

    ```
    service_type: rgw
    placement:
      hosts:
      - host1
      - host2
    spec:
      rgw_realm: my_realm
      rgw_zonegroup: my_zonegroup
      rgw_zone: my_zone
      zonegroup_hostnames:
      - hostname1
      - hostname2
    ```

    For example,

    ```
    service_type: rgw
    placement:
      hosts:
      - _host1_
      - _host2_
    spec:
      rgw_realm: my_realm
      rgw_zonegroup: my_zonegroup
      rgw_zone: my_zone
      zonegroup_hostnames:
      - foo
      - bar
    ```

  - Mount the YAML file under a directory in the container.

    ```
    cephadm shell --mount rgw.yaml:/var/lib/ceph/rgw/rgw.yaml
    ```

    For example,

    ```
    [root@host01 ~]# cephadm shell --mount rgw.yaml:/var/lib/ceph/rgw/rgw.yaml
    ```

  - Bootstrap the realm.

> **Note:** The specification file that is used by the `rgw` module has the same format as the one used by the orchestrator. Therefore, you can provide any orchestration supported Ceph Object Gateway parameters, including advanced configuration features such as SSL certificates.

```
ceph rgw realm bootstrap -i /var/lib/ceph/rgw/rgw.yaml
```

For example,

```
[ceph: root@host01 /]# ceph rgw realm bootstrap -i /var/lib/ceph/rgw/
rgw.yaml
```

4. List the available tokens.

> **Note:** If you run this command before the Ceph Object Gateway daemons are fully deployed, a 'no tokens' message is displayed. This is because of no endpoints existing yet.

```
ceph rgw realm tokens | jq
```

For example,

```
[ceph: root@host01 /]# ceph rgw realm tokens | jq

[
  {
    "realm": "myrealm",
    "token":
"ewogICAgInJlYWxtX25hbWUiOiAibXlyZWFsbSIsCiAgICAicmVhbG1faWQiOiAiZDA3YzAwZWYtOTA0MS00Z
jZlLTg4MDQtN2Q0MDI0MDU1NmFlIiwKICAgICJlbmRwb2ludCI6ICJodHRwOi8vdm0tMDA6NDMyMSIsCiAgICA
iYWNjZXNzX2tleSI6ICI5NTY1VFZSMVFWTExFRzdVNFIxRCIsCiAgICAic2VjcmV0IjogImQ3b0FJQXZrNEdYe
Xpyd3Q2QVZ6bEZNQmNnRG53RVdMMHFDenE3cjUiCn1="
  }
]
```

What to do next

– Verify the Ceph Object Gateway deployment.
For example,

```
[ceph: root@host01 /]# ceph orch list --daemon-type=rgw
NAME                                                  HOST                    PORTS
 STATUS          REFRESHED  AGE  MEM USE  MEM LIM  VERSION          IMAGE ID
 CONTAINER ID
rgw.myrealm.myzonegroup.ceph-saya-6-osd-host01.eburst  ceph-saya-6-osd-host01  *:80
running (111m)  9m ago     111m  82.3M   -        17.2.6-22.el9cp  2d5b080de0b0
 2f3eaca7e88e
```

– Verify the *hostnames* added via realm bootstrap:
For example,

```
radosgw-admin zonegroup get --rgw-zonegroup zone_group_name
```

Foe example,

```
[ceph: root@host01 /]# radosgw-admin zonegroup get --rgw-zonegroup my_zonegroup

{
    "id": "02a175e2-7f23-4882-8651-6fbb15d25046",
    "name": "my_zonegroup_ck",
    "api_name": "my_zonegroup_ck",
    "is_master": true,
    "endpoints": [
        "http://vm-00:80"
    ],
    "hostnames": [
        "foo"
        "bar"
    ],
    "hostnames_s3website": [],
    "master_zone": "f42fea84-a89e-4995-996e-61b7223fb0b0",
    "zones": [
        {
            "id": "f42fea84-a89e-4995-996e-61b7223fb0b0",
            "name": "my_zone_ck",
            "endpoints": [
                "http://vm-00:80"
            ],
            "log_meta": false,
            "log_data": false,
            "bucket_index_max_shards": 11,
            "read_only": false,
            "tier_type": "",
            "sync_from_all": true,
            "sync_from": [],
            "redirect_zone": "",
            "supported_features": [
                "compress-encrypted",
                "resharding"
            ]
        }
    ],
    "placement_targets": [
        {
            "name": "default-placement",
            "tags": [],
            "storage_classes": [
                "STANDARD"
            ]
        }
    ],
    "default_placement": "default-placement",
    "realm_id": "439e9c37-4ddc-43a3-99e9-ea1f3825bb51",
    "sync_policy": {
        "groups": []
    },
    "enabled_features": [
        "resharding"
    ]
}
```

See the *hostnames* section of the zonegroup for the list of hostnames. They are specified in *zonegroup_hostnames* in the Ceph Object Gateway specification file.

Deploying Ceph Object Gateway multi-site using the `rgw` module

Bootstrap a Ceph Object Gateway realm to create a realm, zonegroup, zone, and system user for multi-site sync operations.

Before you begin

Before you begin, make sure that you have a running Red Hat Ceph Storage cluster with at least one OSD deployed.

About this task

Bootstrapping the Ceph Object Gateway realm creates a new realm entity, a new zonegroup, and a new zone. It configures a new system user that you can use for multi-site sync operations. The `rgw` module instructs the orchestrator to create and deploy the corresponding Ceph Object Gateway daemons.

Enable the `rgw` module by using the **`ceph mgr module enable rgw`** command. After you enable the `rgw` module, either pass the arguments in the command line or use the YAML specification file to bootstrap the realm.

Procedure

1. Log in to the `cephadm` shell.

```
cephadm shell
```

2. Enable the `rgw` module.

```
ceph mgr module enable rgw
```

For example,

```
[ceph: root@host01 /]# ceph mgr module enable rgw
```

3. Bootstrap the Ceph Object Gateway realm.
Use either the command line or the YAML specification file to bootstrap the realm.

> **Note:** To dedicate some Ceph Object Gateway daemons to client IO only, so that they do not take part in multi-site sync operations, see "Deploying the Ceph Object Gateway using the service specification" on page 626.

- Use the command line.

```
ceph rgw realm bootstrap [--realm-name REALM_NAME] [--zonegroup-name
ZONEGROUP_NAME] [--zone-name ZONE_NAME] [--port PORT_NUMBER] [--placement
HOSTNAME] [--start-radosgw]
```

For example,

```
[ceph: root@host01 /]# ceph rgw realm bootstrap --realm-name myrealm --zonegroup-
name myzonegroup --zone-name myzone --port 5500 --placement="host01 host02" --
start-radosgw
Realm(s) created correctly. Please, use 'ceph rgw realm tokens' to get the token.
```

- Use the YAML specification file.
  - As a root user, create the YAML file.

```
rgw_realm: REALM_NAME
rgw_zonegroup: ZONEGROUP_NAME
rgw_zone: ZONE_NAME
placement:
  hosts:
    - HOSTNAME_1
    - HOSTNAME_2
spec:
  rgw_frontend_port: PORT_NUMBER
  zone_endpoints: http://RGW_HOSTNAME_1:RGW_PORT_NUMBER_1, http://
RGW_HOSTNAME_2:RGW_PORT_NUMBER_2
```

For example,

```
[root@host01 ~]# cat rgw.yaml

rgw_realm: myrealm
rgw_zonegroup: myzonegroup
rgw_zone: myzone
placement:
  hosts:
    - host01
    - host02
spec:
  rgw_frontend_port: 5500
  zone_endpoints: http://<rgw_host1>:<rgw_port1>, http://
<rgw_host2>:<rgw_port2>
```

- Mount the YAML file under a directory in the container, by using the **cephadm shell --mount** command.
  For example,

  ```
  [root@host01 ~]# cephadm shell --mount rgw.yaml:/var/lib/ceph/rgw/rgw.yaml
  ```

- Bootstrap the realm, by using the **ceph rgw realm bootstrap** command.

  > **Note:** The specification file that the `rgw` module uses has the same format as the one that the orchestrator uses. As a result, you can provide any Ceph Object Gateway parameters that the orchestrator supports, including advanced configuration features such as SSL certificates.

  For example,

  ```
  [ceph: root@host01 /]# ceph rgw realm bootstrap -i /var/lib/ceph/rgw/
  rgw.yaml
  ```

4. List the available tokens, by using the **ceph rgw realm tokens** command.

   > **Note:** If you run this command before the Ceph Object Gateway daemons get deployed, it displays a message that there are no tokens as there are no endpoints yet.

   For example,

   ```
   [ceph: root@host01 /]# ceph rgw realm tokens | jq

   [
     {
       "realm": "myrealm",
       "token":
   "ewogICAgInJlYWxtX25hbWUiOiAibXlyZWFsbSIsCiAgICAicmVhbG1faWQiOiAiZDA3YzAwZWYtOTA0MS00Z
   jZlLTg4MDQtN2Q0MDI0MDU1NmFlIiwKICAgICJlbmRwb2ludCI6ICJodHRwOi8vdm0tMDA6NDMyMSIsCiAgICA
   iYWNjZXNzX2tleSI6ICI5NTY1VFZSMVFWTExFRzdVNFIxRCIsCiAgICAic2VjcmV0IjogImQ3b0FJQXZrNEdYe
   Xpyd3Q2QVZ6bEZNQmNnRG53RVdMMHFDenE3cjUiCn1="
     }
   ]
   ```

5. Create the secondary zone by using these tokens and join the existing realms.

   a. As a root user, create the YAML file.
      For example,

      ```
      [root@host01 ~]# cat zone-spec.yaml
      rgw_zone: my-secondary-zone
      rgw_realm_token: <token>
      placement:
        hosts:
          - ceph-node-1
          - ceph-node-2
      spec:
        rgw_frontend_port: 5500
      ```

   b. Mount the `zone-spec.yaml` file under a directory in the container, by using the **cephadm shell --mount** command.
      For example,

      ```
      [root@host01 ~]# cephadm shell --mount zone-spec.yaml:/var/lib/ceph/radosgw/zone-
      spec.yaml
      ```

   c. Enable the `rgw` module on the secondary zone.

      ```
      ceph mgr module enable rgw
      ```

   d. Create the secondary zone, by using the **ceph rgw zone create** command.
      For example,

```
[ceph: root@host01 /]# ceph rgw zone create -i /var/lib/ceph/radosgw/zone-
spec.yaml
```

What to do next

Verify the Ceph Object Gateway multi-site deployment, by using the **`radosgw-admin realm list`** command. For example,

```
[ceph: root@host01 /]# radosgw-admin realm list
{
    "default_info": "d07c00ef-9041-4f6e-8804-7d40240556ae",
    "realms": [
        "myrealm"
    ]
}
```

High availability for the Ceph Object Gateway

As a storage administrator, you can assign many instances of the Ceph Object Gateway to a single zone. This allows you to scale out as the load increases, that is, the same zone group and zone; however, you do not need a federated architecture to use a highly available proxy. Since each Ceph Object Gateway daemon has its own IP address, you can use the `ingress` service to balance the load across many Ceph Object Gateway daemons or nodes. The `ingress` service manages HAProxy and `keepalived` daemons for the Ceph Object Gateway environment. You can also terminate HTTPS traffic at the HAProxy server, and use HTTP between the HAProxy server and the Beast front-end web server instances for the Ceph Object Gateway.

Prerequisites

- At least two Ceph Object Gateway daemons running on different hosts.
- Capacity for at least two instances of the ingress service running on different hosts

High availability service

The `ingress` service provides a highly available endpoint for the Ceph Object Gateway. The `ingress` service can be deployed to any number of hosts as needed. Red Hat recommends having at least two supported Red Hat Enterprise Linux servers, each server configured with the `ingress` service. You can run a high availability (HA) service with a minimum set of configuration options. The Ceph orchestrator deploys the `ingress` service, which manages the `haproxy` and `keepalived` daemons, by providing load balancing with a floating virtual IP address. The active `haproxy` distributes all Ceph Object Gateway requests to all the available Ceph Object Gateway daemons.

A virtual IP address is automatically configured on one of the `ingress` hosts at a time, known as the primary host. The Ceph orchestrator selects the first network interface based on existing IP addresses that are configured as part of the same subnet. In cases where the virtual IP address does not belong to the same subnet, you can define a list of sub-nets for the Ceph orchestrator to match with existing IP addresses. If the `keepalived` daemon and the active `haproxy` are not responding on the primary host, then the virtual IP address moves to a backup host. This backup host becomes the new primary host.

> **Warning:** Currently, you can not configure a virtual IP address on a network interface that does not have a configured IP address.

> **Important:** To use the secure socket layer (SSL), SSL must be terminated by the `ingress` service and not at the Ceph Object Gateway.

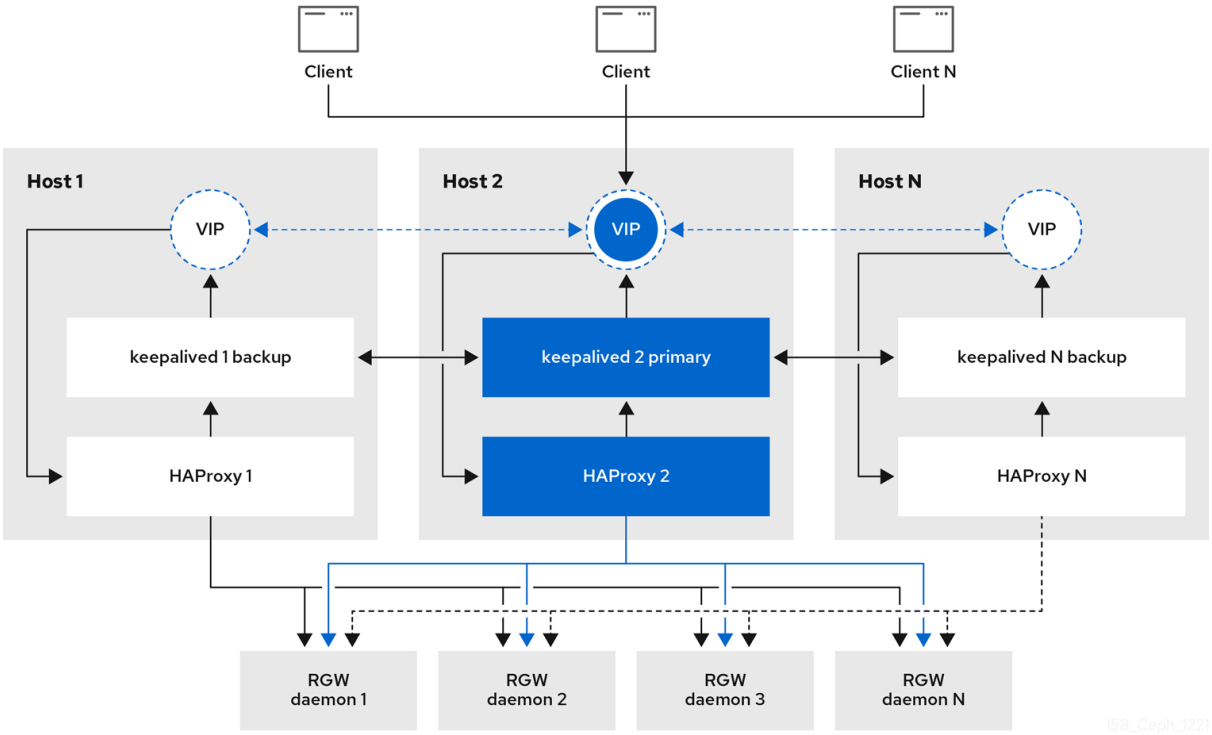


Figure 88: High availability architecture

Reference

For more information, see "Configuring high availability for the Ceph Object Gateway" on page 830.

Configuring high availability for the Ceph Object Gateway

To configure high availability (HA) for the Ceph Object Gateway you write a YAML configuration file, and the Ceph orchestrator does the installation, configuration, and management of the `ingress` service. The `ingress` service uses the `haproxy` and `keepalived` daemons to provide high availability for the Ceph Object Gateway.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A minimum of two hosts running Red Hat Enterprise Linux 9 or higher, for installing the `ingress` service on.
- A healthy running Red Hat Ceph Storage cluster.
- A minimum of two Ceph Object Gateway daemons running on different hosts.
- Root-level access to the host running the `ingress` service.
- If using a firewall, then open port 80 for HTTP and port 443 for HTTPS traffic.

About this task

You can deploy an ingress service with Ceph Object Gateway as the backend, where the `use_tcp_mode_over_rgw` option is set to true in the `spec` section of the ingress specification.
For more information, see "High availability service" on page 829, and *Performing a standard RHEL 9 installation* within the product documentation for your supported Red Hat Enterprise Linux version on Red Hat Documentation.

> **Note:** In `cephadm` deployments, HTTPS is configured by using certificate management. The ingress service handles SSL termination rather than configuring SSL directly on the Ceph Object Gateway frontend.

Procedure

1. Create a new `ingress.yaml` file.

```
touch ingress.yaml
```

   For example,

```
[root@host01 ~] touch ingress.yaml
```

2. Open the `ingress.yaml` file for editing.
   Add in the following options, and add values applicable to the environment.

```
service_type: ingress
service_id: SERVICE_ID    # adjust to match your existing RGW service
placement:
 hosts:
   - HOST1
   - HOST2
   - HOST3
spec:
 backend_service: SERVICE_ID              # adjust to match your existing RGW service
 virtual_ip: IP_ADDRESS/CIDR              # ex: 192.168.20.1/24
 frontend_port: INTEGER                   # ex: 8080
 monitor_port: INTEGER                    # ex: 1967, used by haproxy for load
balancer status
 virtual_interface_networks: [ ... ]      # optional: list of CIDR networks
 use_keepalived_multicast: TRUE/FALSE     # optional: Default is False.
 vrrp_interface_network: IP_ADDRESS/CIDR  # optional: ex: 192.168.20.0/24
 health_check_interval: TIME              # optional: Default is 2s.
 ssl: true
 certificate_source: inline               # optional: Default is cephadm-signed
 ssl_cert: |                              # optional: SSL certificate and key
   -----BEGIN CERTIFICATE-----
   ...
   -----END CERTIFICATE-----
 ssl_key: |
   -----BEGIN PRIVATE KEY-----
   ...
   -----END PRIVATE KEY-----
 enable_stats: true
 monitor_ssl: BOOLEAN
 monitor_cert_source: inline               # optional: default is reuse_service_cert
 monitor_ssl_cert: |                       # optional: SSL certificate and key
   -----BEGIN CERTIFICATE-----
   ...
   -----END CERTIFICATE-----
 monitor_ssl_key: |
   -----BEGIN PRIVATE KEY-----
   ...
   -----END PRIVATE KEY-----
 monitor_networks: [..]
 monitor_ip_addrs:
   host: IP_ADDRESSSS/CIDR
```

"Ingress service configuration" on page 831 provides the ingress service configuration parameters and descriptions.

Table 105: Ingress service configuration

Parameter	Description
service_type	Mandatory and set to `ingress`.
service_id	Name of the service, for example, `rgw.foo`.
placement hosts	Hosts for HA daemons (haproxy and keepalived). Need not match RGW nodes.
virtual_ip	Virtual IP in CIDR format for ingress service.
virtual_ips_list	List of virtual IPs in CIDR format. Each IP is primary on one ingress node.
virtual_interface_networks	Networks to identify ethernet interface for virtual IP.

Parameter	Description
frontend_port	Port used to access the ingress service.
ssl_cert	SSL certificate in .pem format. Required when `certificate_source` is `inline` or `reference`.
ssl	Enable SSL for ingress service.
certificate_source	Source of certificate: `inline`, `reference`, or `cephadm-signed`. Default is `cephadm-signed`.
ssl_key	SSL key in `.pem` format if **`certificate_source`** is not `cephadm-signed`.
use_keepalived_multicast	Default is `False`. Uses unicast IPs for keepalived config.
health_check_interval	Interval between haproxy health checks. Default is 2 seconds.
enable_stats	Enable haproxy stats. Default is `False`.
monitor_ssl	Enable SSL for monitoring. Requires service SSL enabled.
monitor_cert_source	Source of monitor certificate: `reuse_service_cert`, `inline`, `reference`, or `cephadm-signed`. Default is `reuse_service_cert`.
monitor_ssl_cert	Monitor SSL certificate in .pem format if **`monitor_cert_source`** is not`cephadm-signed`.
monitor_ssl_key	Monitor SSL key in `.pem` format if **`monitor_cert_source`** is not`cephadm-signed`.
monitor_ip_addrs	If assigned to host, it will be used. Otherwise, **`monitor_networks`** will be checked.
monitor_networks	If specified, an IP matching one of the networks will be used. Otherwise, default host IP is used.

Following is an example of providing an SSL cert:

```
service_type: ingress
service_id: rgw.foo
placement:
  hosts:
    - host01.example.com
    - host02.example.com
    - host03.example.com
spec:
  backend_service: rgw.foo
  virtual_ip: 192.168.1.2/24
  frontend_port: 8080
  monitor_port: 1967
  virtual_interface_networks:
    - 10.10.0.0/16
  ssl_cert: |
    -----BEGIN CERTIFICATE-----
    MIIEpAIBAAKCAQEA+Cf4l9OagD6x67HhdCy4Asqw89Zz9ZuGbH50/7ltIMQpJJU0
    gu9ObNtIoC0zabJ7n1jujueYgIpOqGnhRSvsGJiEkgN81NLQ9rqAVaGpadjrNLcM
    bpgqJCZj0vzzmtFBCtenpb5l/EccMFcAydGtGeLP33SaWiZ4Rne56GBInk6SATI/
    JSKweGD1y5GiAWipBR4C74HiAW9q6hCOuSdp/2WQxWT3T1j2sjlqxkHdtInUtwOm
    j5Ism276IndeQ9hR3reFR8PJnKIPx73oTBQ7p9CMR1J4ucq9Ny0J12wQYT00fmJp
    -----END CERTIFICATE-----
    -----BEGIN PRIVATE KEY-----
    MIIEBTCCAu2gAwIBAgIUGfYFsj8HyA9Zv2l600hxzT8+gG4wDQYJKoZIhvcNAQEL
    BQAwgYkxCzAJBgNVBAYTAklOMQwwCgYDVQQIDANLQVIxDDAKBgNVBAcMA0JMUjEM
    MAoGA1UECgwDUkhUMQswCQYDVQQLDAJCVTEkMCIGA1UEAwwbY2VwaC1zc2wtcmhj
    czUtOGRjeHY2LW5vZGU1MR0wGwYJKoZIhvcNAQkBFg5hYmNAcmVkaGF0LmNvbTAe
    -----END PRIVATE KEY-----
```

Following is an example of an ingress file without providing an SSL cert:

```
service_type: ingress
```

```
service_id: rgw.ssl    # adjust to match your existing RGW service

placement:
hosts:
- hostname1
- hostname2
spec:
backend_service: rgw.rgw.ssl.ceph13   # adjust to match your existing RGW service
virtual_ip: IP_ADDRESS/CIDR           # ex: 192.168.20.1/24
frontend_port: INTEGER                # ex: 443
monitor_port: INTEGER                 # ex 1969
use_tcp_mode_over_rgw: True
```

3. Launch the `cephadm` shell.

```
cephadm shell --mount ingress.yaml:FILEPATH/ingress.yaml
```

For example,

```
[root@host01 ~]# cephadm shell --mount ingress.yaml:/var/lib/ceph/radosgw/ingress.yaml
```

4. Configure the latest `haproxy` and `keepalived` images.

```
ceph config set mgr mgr/cephadm/container_image_haproxy HAPROXY_IMAGE_ID
ceph config set mgr mgr/cephadm/container_image_keepalived KEEPALIVED_IMAGE_ID
```

For example,

```
[ceph: root@host01 /]# ceph config set mgr mgr/cephadm/container_image_haproxy
registry.redhat.io/rhceph/rhceph-haproxy-rhel9:latest
[ceph: root@host01 /]# ceph config set mgr mgr/cephadm/container_image_keepalived
registry.redhat.io/rhceph/keepalived-rhel9:latest
```

5. Install and configure the new `ingress` service using the Ceph orchestrator.

```
ceph orch apply -i FILEPATH/ingress.yaml
```

For example,

```
[ceph: root@host01 /]# ceph orch apply -i /var/lib/ceph/radosgw/ingress.yaml
```

6. After the Ceph orchestrator completes, verify the HA configuration.
   a. On the host running the `ingress` service, check that the virtual IP address displays.

```
ip addr show
```

For example,

```
[root@host01 ~]# ip addr show
```

   b. Try reaching the Ceph Object Gateway from a Ceph client.

```
wget HOST_NAME
```

For example,

```
[root@client ~]# wget host01.example.com
```

The HA configuration for the Ceph Object Gateway is working properly if this returns an `index.html` file, similar to the following example:

```
<?xml version="1.0" encoding="UTF-8"?>
        <ListAllMyBucketsResult xmlns="http://s3.amazonaws.com/doc/2006-03-01/">
                <Owner>
                        <ID>anonymous</ID>
                        <DisplayName></DisplayName>
                </Owner>
                <Buckets>
                </Buckets>
        </ListAllMyBucketsResult>
```

Adjusting logging and debugging output

Debug output can be changed and log output can be adjusted to save as files.

After finishing the setup procedure, check your logging output to ensure it meets your needs. By default, the Ceph daemons log to `journald`, and you can view the logs using the `journalctl` command. Alternatively, you can also have the Ceph daemons log to files, which are located under the `/var/log/ceph/CEPH_CLUSTER_ID/` directory.

> **Important:** Verbose logging can generate over 1 GB of data per hour. This type of logging can potentially fill up the operating system's disk, causing the operating system to stop functioning.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway software.

Procedure

1. Set the following parameter to increase the Ceph Object Gateway logging output:

   Syntax

   ```
   ceph config set client.rgw debug_rgw VALUE
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph config set client.rgw debug_rgw 20
   ```

   a. You can also modify these settings at runtime:

      Syntax

      ```
      ceph --admin-daemon /var/run/ceph/CEPH_CLUSTER_ID/ceph-client.rgw.NAME.asok
      config set debug_rgw VALUE
      ```

      Example

      ```
      [ceph: root@host01 /]# ceph --admin-daemon /var/run/ceph/62a081a6-88aa-11eb-
      a367-001a4a000672/ceph-client.rgw.rgw.asok config set debug_rgw 20
      ```

2. Optionally, you can configure the Ceph daemons to log their output to files. Set the `log_to_file`, and `mon_cluster_log_to_file` options to `true`:

   Example

   ```
   [ceph: root@host01 /]# ceph config set global log_to_file true
   [ceph: root@host01 /]# ceph config set global mon_cluster_log_to_file true
   ```

Reference

For more information, see the following:

– Debugging and logging configuration options

Configuration

As a storage administrator, learning the basics of configuring the Ceph Object Gateway is important. You can learn about the defaults and the embedded web server called Beast.

For troubleshooting issues with the Ceph Object Gateway, you can adjust the logging and debugging output generated by the Ceph Object Gateway. Also, you can provide a High-Availability proxy for storage cluster access using the Ceph Object Gateway.

Add a wildcard to the DNS

Add the wildcard such as hostname to the DNS record of the DNS server.

Prerequisites

– A running Red Hat Ceph Storage cluster.

– Ceph Object Gateway installed.

– Root-level access to the admin node.

Procedure

1. To use Ceph with S3-style sub-domains, add a wildcard to the DNS record of the DNS server that the `ceph-radosgw` daemon uses to resolve domain names:

   Syntax

   ```
   bucket-name.domain-name.com
   ```

   For `dnsmasq`, add the following address setting with a dot (.) prepended to the host name:

   Syntax

   ```
   address=/.HOSTNAME_OR_FQDN/HOST_IP_ADDRESS
   ```

   Example

   ```
   address=/.gateway-host01/192.168.122.75
   ```

   For `bind`, add a wildcard to the DNS record:

   Example

   ```
   $TTL    604800
   @       IN      SOA     gateway-host01. root.gateway-host01. (
                                 2         ; Serial
                            604800         ; Refresh
                             86400         ; Retry
                           2419200         ; Expire
                            604800 )       ; Negative Cache TTL
   ;
   @       IN      NS      gateway-host01.
   @       IN      A       192.168.122.113
   *       IN      CNAME   @
   ```

2. Restart the DNS server and ping the server with a subdomain to ensure that the `ceph-radosgw` daemon can process the subdomain requests:

   Syntax

   ```
   ping mybucket.HOSTNAME
   ```

   Example

```
[root@host01 ~]# ping mybucket.gateway-host01
```

3. If the DNS server is on the local machine, you might need to modify `/etc/resolv.conf` by adding a nameserver entry for the local machine.

4. Add the host name in the Ceph Object Gateway zone group:

   a. Get the zone group:

      Syntax

      ```
      radosgw-admin zonegroup get --rgw-zonegroup=ZONEGROUP_NAME > zonegroup.json
      ```

      Example

      ```
      [ceph: root@host01 /]# radosgw-admin zonegroup get --rgw-zonegroup=us >
      zonegroup.json
      ```

   b. Take a back-up of the JSON file:

      Example

      ```
      [ceph: root@host01 /]# cp zonegroup.json zonegroup.backup.json
      ```

   c. View the `zonegroup.json` file:

      Example

      ```
      [ceph: root@host01 /]# cat zonegroup.json
      {
          "id": "d523b624-2fa5-4412-92d5-a739245f0451",
          "name": "asia",
          "api_name": "asia",
          "is_master": "true",
          "endpoints": [],
          "hostnames": [],
          "hostnames_s3website": [],
          "master_zone": "d2a3b90f-f4f3-4d38-ac1f-6463a2b93c32",
          "zones": [
              {
                  "id": "d2a3b90f-f4f3-4d38-ac1f-6463a2b93c32",
                  "name": "india",
                  "endpoints": [],
                  "log_meta": "false",
                  "log_data": "false",
                  "bucket_index_max_shards": 11,
                  "read_only": "false",
                  "tier_type": "",
                  "sync_from_all": "true",
                  "sync_from": [],
                  "redirect_zone": ""
              }
          ],
          "placement_targets": [
              {
                  "name": "default-placement",
                  "tags": [],
                  "storage_classes": [
                      "STANDARD"
                  ]
              }
          ],
          "default_placement": "default-placement",
          "realm_id": "d7e2ad25-1630-4aee-9627-84f24e13017f",
          "sync_policy": {
              "groups": []
          }
      }
      ```

   d. Update the `zonegroup.json` file with new host name:

      Example

      ```
      "hostnames": ["host01", "host02","host03"],
      ```

e. Set the zone group back in the Ceph Object Gateway:

Syntax

```
radosgw-admin zonegroup set --rgw-zonegroup=ZONEGROUP_NAME --
infile=zonegroup.json
```

Example

```
[ceph: root@host01 /]# radosgw-admin zonegroup set --rgw-zonegroup=us --
infile=zonegroup.json
```

f. Update the period:

Example

```
[ceph: root@host01 /]# radosgw-admin period update --commit
```

g. Restart the Ceph Object Gateway so that the DNS setting takes effect.

Reference

For more information, see the following:

– “Configuration database” on page 277

Beast front-end web server

The Ceph Object Gateway provides Beast, a C/C embedded front-end web server. Beast uses the `Boost.Beast` C library to parse HTTP, and `Boost.Asio` for asynchronous network I/O.

In `cephadm` deployments, HTTPS for the Ceph Object Gateway is configured by using certificate management. The Beast frontend does not handle SSL configuration directly.

Related information

Boost C++ Libraries

Beast configuration options

Use this information to know which Beast configuration options can be passed to the embedded web server in the Ceph configuration file for the Ceph Object Gateway.

“Beast configuration options that can be passed to the embedded web server” on page 837 lists the Beast configuration options that can be passed to the embedded web server in the Ceph configuration file for the Ceph Object Gateway. Each option has a default value. If a value is not specified, the default value is empty.

> **Note:** In `cephadm` deployments, HTTPS is configured by using certificate management. These Beast SSL configuration options are not used for enabling HTTPS on the Ceph Object Gateway.

Table 106: Beast configuration options that can be passed to the embedded web server

Option	Description	Default
`endpoint` and `ssl_endpoint`	Sets the listening address in the form `address[:port]` where the address is an IPv4 address string in dotted decimal form, or an IPv6 address in hexadecimal notation surrounded by brackets, [ ]. The optional port defaults to `8080` for `endpoint` and `443` for `ssl_endpoint`. It can be specified multiple times as in `endpoint=[::1] endpoint=192.168.0.100:8000`.	EMPTY

Option	Description	Default
ssl_certificate	Path to the SSL certificate file used for SSL-enabled endpoints. This option is not used for HTTPS configuration in cephadm deployments.	EMPTY
ssl_private_key	Optional path to the private key file used for SSL-enabled endpoints. This option is not used for HTTPS configuration in cephadm deployments.	EMPTY
tcp_nodelay	Performance optimization in some environments.	EMPTY

Note: By default, the Beast front end writes an access log line recording all requests that are processed by the server to the RADOS Gateway log file.

Example /etc/ceph/ceph.conf file with Beast options using SSL

The following example shows legacy SSL configuration using Beast. This method is not used in cephadm deployments.

```
...

[client.rgw.node1]
rgw frontends = beast ssl_endpoint=192.168.0.100:443 ssl_certificate=<path to SSL
certificate>
```

Configuring HTTPS for the Ceph Object Gateway

Enable HTTPS for the Ceph Object Gateway (RGW) by using the cephadm certificate manager.

Before you begin

- The Ceph Object Gateway is deployed using cephadm.
- You have access to the RGW service specification.
- Certificate files are available if you are using custom certificates.

About this task

Starting with version 9.0, HTTPS for the Ceph Object Gateway is configured by using the cephadm certificate manager (certmgr). This method replaces legacy frontend SSL configuration and provides a consistent way to manage certificates across services.

Important: The rgw_frontend_ssl_certificate configuration option is deprecated and is not supported in 9.0 and later releases. Do not use this option for new deployments.

Procedure

1. Define the RGW service specification with HTTPS enabled.
2. Configure HTTPS by using cephadm-signed certificates (default).
   Use cephadm to automatically generate and manage certificates for the RGW service. Set ssl: true and certificate_source: cephadm-signed in the servicespecification.

   ```
   service_type: rgw
   service_id: my-rgw
   spec:
     ssl: true
     certificate_source: cephadm-signed
   ```

   Apply the service specification.

```
ceph orch apply -i rgw.yaml
```

cephadm generates a certificate and enables HTTPS for the RGW endpoints.

> **Note:** The `generate_cert: true` field remains supported when used together with `ssl: true` and produces the same cephadm-managed self-signed certificate behavior. For new deployments, `certificate_source: cephadm-signed` is the preferred method.

3. Configure HTTPS by using inline certificates.
   Provide a certificate and private key directly in the service specification. Set `ssl: true`, `certificate_source: inline`, and embed the PEM certificate andprivate key in the `ssl_cert` and `ssl_key` fields.

```
service_type: rgw
service_id: my-rgw
spec:
  ssl: true
  certificate_source: inline
  ssl_cert: |
    -----BEGIN CERTIFICATE-----
    ...
    -----END CERTIFICATE-----
  ssl_key: |
    -----BEGIN PRIVATE KEY-----
    ...
    -----END PRIVATE KEY-----
```

   Apply the service specification:

```
ceph orch apply -i rgw.yaml
```

   cephadm deploys the RGW service with the custom certificate and key.

   > **Note:** Both `ssl_cert` and `ssl_key` are required when certificate_source is inline.

4. Configure HTTPS for the Ceph Object Gateway by using referenced, externally managed certificates.
   The certificates are not generated by `cephadm`; instead, they are stored and referenced in the service specification.

   First, register the certificate and key with certmgr:

```
ceph orch certmgr cert set --cert-name rgw_ssl_cert --service-name rgw.<service_id> -i
$PWD/server_cert.pem
```

```
ceph orch certmgr key set --key-name rgw_ssl_key --service-name rgw.<service_id> -i
$PWD/server_key
```

   Then configure the RGW service to use the referenced certificate:

```
service_type: rgw
service_id: my-rgw
spec:
  ssl: true
  certificate_source: reference
```

   Apply the updated specification:

```
ceph orch apply -i my-rgw.yaml
```

5. Configure wildcard SAN support if required.

To support virtual-host-style S3 access, enable wildcard Subject Alternative Names (SAN) in the `cephadm-signed` certificate. Set `wildcard_enabled` to `true` and list the zone group hostnames in `zonegroup_hostnames`.

```
service_type: rgw
service_id: my-rgw
spec:
  ssl: true
  certificate_source: cephadm-signed
  wildcard_enabled: true
  zonegroup_hostnames:
    - "*.example.com"
```

Apply the updated specification:

```
ceph orch apply -i my-rgw.yaml
```

When `wildcard_enabled` is set to `true`, `cephadm` includes a wildcard SAN entry in the generated certificate (for example, *.s3.example.com), allowing bucket access using virtual-host-style DNS names.

Result

After completing this task, HTTPS is enabled for the Ceph Object Gateway. Certificates are managed by `cephadm` through `certmgr`, and RGW endpoints are accessible over secure connections.

Verify the deployment.

```
ceph orch ls and ceph orch ps
```

The RGW service should show the HTTPS port (for example, ?:443).

Deploying self-signed `cephadm` certificates with SAN configuration

Configure HTTPS for the Ceph Object Gateway (RGW) by using cephadm-managed self-signed certificates with Subject Alternative Name (SAN) entries.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running and healthy Red Hat Ceph Storage cluster.
- The Ceph Object Gateway is deployed by using cephadm.
- Root-level access to the Ceph cluster.

About this task

This task extends the HTTPS configuration for the Ceph Object Gateway by enabling self-signed certificates with SAN entries. SAN configuration allows access by using multiple domain names or hostnames, which is required for virtual-host–style S3 access and multi-site deployments.

`cephadm` automatically generates and manages certificates when `certificate_source: cephadm-signed` is used (or `generate_cert` is set). You can specify SAN entries by using `zonegroup_hostnames`.

Procedure

1. Create a service specification file for the RGW deployment.

   ```
   [ceph: root@host01 ~]# touch rgw.yaml
   ```

2. Define the RGW service specification with HTTPS and SAN configuration.
   Enable certificate generation and specify hostnames for SAN entries.

   ```
   service_type: rgw
   service_id: my-rgw
   placement:
     hosts:
   ```

```
    - host01
spec:
  ssl: true
  certificate_source: cephadm-signed
  wildcard_enabled: true
  zonegroup_hostnames:
    - s3.example.com
    - s3.zone1.example.com
```

Note: `generate_cert: true` remains supported; `certificate_source: cephadm-signed` is the preferred method for new configurations.

3. Deploy the RGW service by using the specification file.

Note: Ensure that `rgw_realm`, `rgw_zone`, and `rgw_zonegroup` are created before deploying the RGW service.

```
ceph orch apply -i rgw.yaml
```

4. Verify that the certificate includes the SAN entries.
Retrieve the generated certificate:

```
ceph orch certmgr cert get cephadm_root_ca_cert > cephadm-root-ca.crt
```

Inspect the certificate:

```
openssl x509 -in cephadm-root-ca.crt -noout -text
```

The output includes SAN entries similar to:

```
X509v3 Subject Alternative Name:
DNS:s3.example.com, DNS:s3.zone1.example.com
```

The SAN values are derived from `zonegroup_hostnames`.

Result

The Ceph Object Gateway is configured with HTTPS using cephadm-managed certificates. The generated certificate includes SAN entries that allow access through multiple hostnames or domains.

What to do next

You can extend this configuration for multi-site deployments. For more information, see:

- “Configuring RGW multi-site with HTTPS using cephadm” on page 841

Configuring RGW multi-site with HTTPS using `cephadm`

Configure HTTPS trust between Ceph Object Gateway (RGW) deployments in a multi-site environment by using cephadm-managed certificates.

In a multi-site deployment, each site uses cephadm-generated certificates. To enable secure communication between sites, you must copy each site's root Certificate Authority (CA) certificate to the other sites and trust these certificates.

Prerequisites

- A running and healthy Red Hat Ceph Storage.
- RGW services are deployed on both primary and secondary sites by using `cephadm` with HTTPS enabled.
- Access to both primary and secondary cluster nodes.

Procedure

1. Retrieve the `cephadm` root CA certificate and store it on the primary site.

   ```
   ceph orch certmgr cert get cephadm_root_ca_cert > cephadm-root-ca.crt
   ```

   Example output:

   ```
   -----BEGIN CERTIFICATE-----
   MIIC...
   -----END CERTIFICATE-----
   ```

2. From the primary site, copy the root CA certificate to the trusted CA directory on the secondary site.

   ```
   scp cephadm-root-ca.crt root@node-sec:/etc/pki/ca-trust/source/anchors/
   ```

   On the secondary site, verify that the certificate is present:

   ```
   ls -l /etc/pki/ca-trust/source/anchors/cephadm-root-ca.crt
   ```

3. Update the trusted certificate store.

   ```
   update-ca-trust
   ```

4. Verify connectivity from the secondary site to the primary RGW endpoint.
   Run an RGW admin command:

   ```
   radosgw-admin realm pull --rgw-realm RGW_REALM --url https://PRIMARY_RGW_ENDPOINT:PORT
   --access-key ACCESS_KEY --secret SECRET_KEY --default
   ```

   Example output:

   ```
   {
     "name": "RGW_REALM",
     "epoch": 2
   }
   ```

   From the secondary site, verify HTTPS access to the primary RGW endpoint:

   ```
   curl https://PRIMARY_RGW_ENDPOINT
   ```

   Example response:

   ```
   <ListAllMyBucketsResult>...</ListAllMyBucketsResult>
   ```

5. Repeat these steps on the primary site by copying the secondary site's `cephadm_root_ca_cert` to `/etc/pki/ca-trust/source/anchors/` and running `update-ca-trust` extract to complete the reverse CA exchange.

   After completing these steps, both sites trust each other's cephadm root CA certificates, enabling secure multi-site replication and operations over HTTPS.

After completing these steps, RGW services in both sites trust each other's certificates, enabling secure multi-site replication and operations over HTTPS.

D3N Data Cache

Datacenter-Data-Delivery Network (D3N) uses high-speed storage, such as `NVMe`, to cache datasets on the access side. Such caching allows big data jobs to use the compute and fast-storage resources available on each Ceph Object Gateway node at the edge. The Ceph Object Gateways act as cache servers for the back-end object store (OSDs), storing data locally for reuse.

> **Note:** Each time the Ceph Object Gateway is restarted the content of the cache directory is purged.

Adding D3N cache directory

To enable D3N cache on Ceph Object Gateway, you need to also include the D3N cache directory in the podman `unit.run` file.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Ceph Object Gateway installed.
- Root-level access to the admin node.
- A fast NVMe drive in each Ceph Object Gateway node to serve as the local cache storage.

Procedure

1. Create a mount point for the NVMe drive.

   ```
   mkfs.ext4 NVME_DRIVE_PATH
   ```

   For example,

   ```
   [ceph: root@host01 /]# mkfs.ext4 /dev/nvme0n1
   mount /dev/nvme0n1 /mnt/nvme0n1/
   ```

2. Create a cache directory path.

   ```
   mkdir NVME_MOUNT_PATH/CACHE_DIRECTORY_NAME
   ```

   For example,

   ```
   [ceph: root@host01 /]# mkdir /mnt/nvme0n1/rgw_datacache
   ```

3. Provide `a+rwx` permission to *NVME_MOUNT_PATH* and to the *rgw_d3n_l1_datacache_persistent_path*.

   ```
   chmod a+rwx NVME_MOUNT_PATH ; chmod a+rwx rgw_d3n_l1_datacache_persistent_path
   ```

   For example,

   ```
   [ceph: root@host01 /]# chmod a+rwx /mnt/nvme0n1 ; chmod a+rwx /mnt/nvme0n1/
   rgw_datacache/
   ```

4. Create or modify the Ceph Object Gateway specification file with `extra_container_args` to add *rgw_d3n_l1_datacache_persistent_path* into podman `unit.run` file.

   ```
   "extra_container_args:
             "-v"
             "rgw_d3n_l1_datacache_persistent_path:rgw_d3n_l1_datacache_persistent_path"
               "
   ```

   For example,

   ```
   [ceph: root@host01 /]# cat rgw-spec.yml
       service_type: rgw
       service_id: rgw.test
       placement:
           hosts:
               host1
               host2
           extra_container_args:
               "-v"
               "/mnt/nvme0n1/rgw_datacache/:/mnt/nvme0n1/rgw_datacache/"
   ```

   If there are multiple instances of Ceph Object Gateway in a single host, then a separate *rgw_d3n_l1_datacache_persistent_path* must be created for each instance and add each path in `extra_container_args`.

For two instances of Ceph Object Gateway in each host, create two separate instances of `cache-directory` under *`rgw_d3n_l1_datacache_persistent_path`*, `/mnt/nvme0n1/rgw_datacache/rgw1` and `/mnt/nvme0n1/rgw_datacache/rgw2`.

Example for `extra_container_args` in a Ceph Object Gateway specification file

```
"extra_container_args:
    "-v"
    "/mnt/nvme0n1/rgw_datacache/rgw1/:/mnt/nvme0n1/rgw_datacache/rgw1/"
    "-v"
    "/mnt/nvme0n1/rgw_datacache/rgw2/:/mnt/nvme0n1/rgw_datacache/rgw2/"
"
```

Example for Ceph Object Gateway specification YAML file

```
[ceph: root@host01 /]# cat rgw-spec.yml
    service_type: rgw
        service_id: rgw.test
        placement:
            hosts:
                host1
                host2
            count-per-host: 2
            extra_container_args:
                "-v"
                "/mnt/nvme0n1/rgw_datacache/rgw1/:/mnt/nvme0n1/rgw_datacache/
rgw1/"
                "-v"
                "/mnt/nvme0n1/rgw_datacache/rgw2/:/mnt/nvme0n1/rgw_datacache/
rgw2/"
```

5. Redeploy the Ceph Object Gateway service by using the **`ceph orch apply`** command.
   For example,

```
[ceph: root@host01 /]# ceph orch apply -i rgw-spec.yml
```

Configuring D3N

You can configure the D3N data cache on an existing Ceph Object Gateway to improve the performance of big-data jobs running in Red Hat Ceph Storage clusters.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Ceph Object Gateway installed.
- Root-level access to the admin node.
- A fast NVMe drive in each RGW node to serve as the local cache storage.

About this task

For more information about using high-availability, see Ceph subsystems default logging level values and Understanding Ceph logs .
To enable D3N on an existing Ceph Object Gateway, use the following command to set the D3N-related configuration:

```
ceph config set CLIENT.RGW CONFIGURATION_OPTION VALUE
```

`rgw_d3n_l1_local_datacache_enabled=true`

```
ceph config set CLIENT.RGW rgw_d3n_l1_local_datacache_enabled TRUE/FALSE
```

For example,

```
[ceph: root@host01 /]# ceph config set client.rgw.host01
rgw_d3n_l1_local_datacache_enabled true
```

rgw_d3n_l1_datacache_persistent_path=*PATH_TO_CACHE_DIRECTORY*

```
ceph config set CLIENT.RGW rgw_d3n_l1_datacache_persistent_path PATH_TO_CACHE_DIRECTORY
```

For example,

```
[ceph: root@host01 /]# ceph config set client.rgw.host01
rgw_d3n_l1_datacache_persistent_path /mnt/nvme/rgw_datacache/
```

rgw_d3n_l1_datacache_size=*max_size_of_cache_in_bytes*

```
ceph config set CLIENT.RGW rgw_d3n_l1_datacache_size VALUE
```

For example,

```
[ceph: root@host01 /]# ceph config set client.rgw.host01 rgw_d3n_l1_datacache_size
10737418240
```

Example

1. Create a test object.

   Note: The test object needs to be larger than 4 MB to cache.

   ```
   [ceph: root@host01 /]# fallocate -l 1G ./1G.dat
   [ceph: root@host01 /]# s3cmd mb s3://bkt
   [ceph: root@host01 /]# s3cmd put ./1G.dat s3://bkt
   ```

2. Run `GET` of an object.

   ```
   [ceph: root@host01 /]# s3cmd get s3://bkt/1G.dat /dev/shm/1G_get.dat
   download: 's3://bkt/1G.dat' -> './1G_get.dat'  [1 of 1]
   1073741824 of 1073741824   100% in    13s    73.94 MB/s  done
   ```

3. Verify cache creation. Cache is created with the name consisting of object `key-name` within a configured *`rgw_d3n_l1_datacache_persistent_path`*.

   ```
   [ceph: root@host01 /]# ls -lh /mnt/nvme/rgw_datacache

   rw-rr. 1 ceph ceph 1.0M Jun  2 06:18
   cc7f967c-0021-43b2-9fdf-23858e868663.615391.1_shadow.ZCiCtMWeu_19wb100JIEZ-o4tv2IyA_1
   ```

   After the cache is created for an object, the next `GET` operation for that object will access from cache resulting in faster access.
   After the cache is created, you can see the cache acceleration by writing to the RAM drive, in this example `/dev/shm`.

   ```
   [ceph: root@host01 /]# s3cmd get s3://bkt/1G.dat /dev/shm/1G_get.dat
   download: 's3://bkt/1G.dat' -> './1G_get.dat'  [1 of 1]
   1073741824 of 1073741824   100% in    6s   155.07 MB/s  done
   ```

Multi-site Ceph Object Gateway command line usage

As a storage administrator, you can have a good understanding of how to use the Ceph Object Gateway in a multi-site environment. You can learn how to better manage the realms, zone groups, and zones in a multi-site environment.

Realms

A realm represents a globally unique namespace consisting of one or more zonegroups containing one or more zones, and zones containing buckets, which in turn contain objects. A realm enables the Ceph Object Gateway to support multiple namespaces and their configuration on the same hardware.

A realm contains the notion of periods. Each period represents the state of the zone group and zone configuration in time. Each time you make a change to a zonegroup or zone, update the period and commit it.

> **Important:** Create realms for new clusters.

Creating a realm

Use the **`realm create`** command to create a realm.

About this task

Make one realm in the list of realms be the default realm. If there is only one realm and it wasn't specified as the default realm when it was created, make it the default realm.

Procedure

- Create a realm.

  ```
  radosgw-admin realm create --rgw-realm=REALM_NAME
  ```

  For example,

  ```
  [ceph: root@host01 /]# radosgw-admin realm create --rgw-realm=test_realm
  ```

  In this example, `test_realm` becomes the default realm.

  > **Important:** Do not use the realm with the **`--default`** flag if the data and metadata are stored in the `default.rgw.data` and `default.rgw.index` pools. If a new realm is set as the default and these pools contain important data, the `radosgw-admin` utility can fail to manage this data properly. Only use the **`--default`** flag if necessary to specify the realm as the default and if you do not need the existing data or metadata in the `default.rgw` pools. If the existing data or metadata is needed, either migrate the default configuration to a multi-site or realm setup or avoid setting the new realm as the default. For more information about migrating to a multi-site, see "Migrating a single site system to multi-site" on page 861. By specifying **`--default`**, the realm is called implicitly with each `radosgw-admin` call unless `--rgw-realm` and the realm name are explicitly provided.

- Optional: Change the default realm.

  ```
  radosgw-admin realm default --rgw-realm=REALM_NAME
  ```

  For example,

  ```
  [ceph: root@host01 /]# radosgw-admin realm default --rgw-realm=test_realm1
  ```

Deleting a realm

Use this information to delete a Ceph Object Gateway realm.

Procedure

1. Delete a realm, by using the **`realm rm`** command.

   ```
   radosgw-admin realm rm --rgw-realm=REALM_NAME
   ```

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin realm rm --rgw-realm=test_realm
   ```

> **Important:** If the deleted realm was configured as the default realm, continue to step "2" on page 847. If the deleted realm was not configured as default, no further steps are required.

2. Remove the default realm ID from the `default_info` in the realm list.
   This step is required for realms that were configured as the default.

   ```
   radosgw-admin realm default rm
   ```

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin realm default rm
   ```

What to do next

Verify that the realm and the realm ID are no longer listed.

```
radosgw-admin realm list
```

Getting a realm

Use the **`realm get`** command to get a realm.

Syntax

```
radosgw-admin realm get --rgw-realm=REALM_NAME
```

Example

```
[ceph: root@host01 /]# radosgw-admin realm get --rgw-realm=test_realm >filename.json
```

The CLI will echo a JSON object with the realm properties.

```
{
    "id": "0a68d52e-a19c-4e8e-b012-a8f831cb3ebc",
    "name": "test_realm",
    "current_period": "b0c5bbef-4337-4edd-8184-5aeab2ec413b",
    "epoch": 1
}
```

Use > and an output file name to output the JSON object to a file.

Listing realms

Use the **`realm list`** command to list realms.

Example

```
[ceph: root@host01 /]# radosgw-admin realm list
```

Setting a realm

Set a specific realm with an input file name.

To set a realm, run the **`realm set`** command, specify the realm name, and `--infile=` with an input file name.

Syntax

```
radosgw-admin realm set --rgw-realm=REALM_NAME --infile=IN_FILENAME
```

Example

```
[ceph: root@host01 /]# radosgw-admin realm set --rgw-realm=test_realm --
infile=filename.json
```

Listing realm periods

Use the **`realm list-periods`** command to list realm periods.

Example

```
[ceph: root@host01 /]# radosgw-admin realm list-periods
```

Pulling a realm

Pull a realm from the node containing the primary zone group and primary zone to a node containing a secondary zone group or zone.

To pull a realm from the node containing the primary zone group and primary zone to a node containing a secondary zone group or zone, run the **`realm pull`** command on the node that will receive the realm configuration.

Syntax

```
radosgw-admin realm pull --url=URL_TO_MASTER_ZONE_GATEWAY--access-key=ACCESS_KEY --
secret=SECRET_KEY
```

Renaming a realm

Renaming a realm is applied locally, as a realm is not part of the period. When renaming a realm with multiple zones, run the command on each zone.

About this task

Renaming a realm does not get pulled with the **`realm pull`** command.

Procedure

1. Rename the realm.

   ```
   radosgw-admin realm rename --rgw-realm=REALM_NAME --realm-new-name=NEW_REALM_NAME
   ```

   > **Note:** Do NOT use **`realm set`** command to change the **name** parameter. That changes the internal name only. Specifying **`--rgw-realm`** would still use the old realm name.

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin realm rename --rgw-realm=test_realm --realm-new-
   name=test_realm2
   ```

2. Commit the changes.

   ```
   radosgw-admin period update --commit
   ```

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin period update --commit
   ```

Zone groups

Zone groups define the geographic location of one or more Ceph Object Gateway instances within one or more zones.

The Ceph Object Gateway supports multi-site deployments and a global namespace by using the notion of zone groups.

> **Note:** Zone groups were previously referred to as regions.

Configuring zone groups differs from typical configuration procedures, because not all of the settings end up in a Ceph configuration file. You can list zone groups, get a zone group configuration, and set a zone group configuration.

> **Note:** The `radosgw-admin zonegroup` operations can be performed on any node within the realm, because the step of updating the period propagates the changes throughout the cluster. However, `radosgw-admin zone` operations **MUST** be performed on a host within the zone.

Creating a zone group

Use the **`zonegroup create`** command to create a zone group. A default zone group must always exist.

About this task

Make one zone group in the list of zone groups be the default zone group. If there is only one zone group and it wasn't specified as the default zone group when it was created, make it the default zone group.

> **Note:** Only one default realm can exist at a time.

Creating a zone group consists of specifying the zone group name. Creating a zone assumes that it is created in the default realm unless `--rgw-realm=`*`REALM_NAME`* is specified.

Specify whether the zone group is the primary zone group, by using the `--master` flag.

> **Important:** Do not use the zone group with the **`--default`** flag if the data and metadata are stored in the `default.rgw.data` and `default.rgw.index` pools. If a new zone group is set as the default and these pools contain important data, the `radosgw-admin` utility can fail to manage this data properly.
> Only use the **`--default`** flag if necessary to specify the zone group as the default and if you do not need the existing data or metadata in the `default.rgw` pools. If the existing data or metadata is needed, either migrate the default configuration to a multi-site or zone group setup or avoid setting the new zone group as the default. For more information about migrating to a multi-site, see "Migrating a single site system to multi-site" on page 861. By specifying **`--default`**, the zone group is called implicitly with each `radosgw-admin` call unless `--rgw-zonegroup` and the zone group name are explicitly provided.

Procedure

1. Create a zone group.

   ```
   radosgw-admin zonegroup create --rgw-zonegroup=ZONE_GROUP_NAME [--rgw-
   realm=REALM_NAME] [--master]
   ```

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin zonegroup create --rgw-zonegroup=zonegroup1 --
   rgw-realm=test_realm --default
   ```

   In this example, `zonegroup1` becomes the default zone group.

   > **Note:** When the zone group is default, the command line assumes `--rgw-zonegroup=`*`ZONE_GROUP_NAME`* as an argument.

2. Optional: Change a zone group setting.

   ```
   radosgw-admin zonegroup modify --rgw-zonegroup=ZONE_GROUP_NAME
   ```

   For example,

```
[ceph: root@host01 /]# radosgw-admin zonegroup modify --rgw-zonegroup=zonegroup1
```

3. Optional: Change the default zone group.

```
radosgw-admin zonegroup default --rgw-zonegroup=ZONE_GROUP_NAME
```

For example,

```
[ceph: root@host01 /]# radosgw-admin zonegroup default --rgw-zonegroup=zonegroup2
```

4. Commit the change.

```
radosgw-admin period update --commit
```

For example,

```
[ceph: root@host01 /]# radosgw-admin period update --commit
```

Renaming a zone group

Use the **zonegroup rename** command to rename a zone group.

To rename a zone group, run the following command:

Syntax

```
radosgw-admin zonegroup rename --rgw-zonegroup=ZONE_GROUP_NAME --zonegroup-new-
name=NEW_ZONE_GROUP_NAME
```

> **Note:** The **zonegroup rename** command changes only the zone group name. The `api_name` value is not updated automatically. To use the new name as the API name, update the zone group configuration manually.

Export the zone group configuration:

```
radosgw-admin zonegroup get > zg_get.json
```

Update the `api_name` value in the `zg_get.json` file to match the new zone group name.

Apply the updated configuration:

```
radosgw-admin zonegroup set --infile zg_get.json
```

Commit the changes:

```
radosgw-admin period update --commit
```

Deleting a zone group

Use the **zonegroup delete** command to delete a zone group.

To delete a zone group, run the following command:

Syntax

```
radosgw-admin zonegroup delete --rgw-zonegroup=ZONE_GROUP_NAME
```

Then, update the period:

Example

```
[ceph: root@host01 /]# radosgw-admin period update --commit
```

Listing zone groups

Use the **zonegroup list** command to list zone groups within a Ceph cluster.

To list the zone groups, run the following command:

```
[ceph: root@host01 /]# radosgw-admin zonegroup list
```

The `radosgw-admin` returns a JSON formatted list of zone groups.

```
{
    "default_info": "90b28698-e7c3-462c-a42d-4aa780d24eda",
    "zonegroups": [
        "us"
    ]
}
```

Getting a zone group

Use the **zonegroup get** command to retrieve a zone group configuration.

To view the configuration of a zone group, run the following command:

Syntax

```
radosgw-admin zonegroup get [--rgw-zonegroup=ZONE_GROUP_NAME]
```

Example of a zone group configuration:

```
{
    "id": "90b28698-e7c3-462c-a42d-4aa780d24eda",
    "name": "us",
    "api_name": "us",
    "is_master": "true",
    "endpoints": [
        "http://rgw1:80"
    ],
    "hostnames": [],
    "hostnames_s3website": [],
    "master_zone": "9248cab2-afe7-43d8-a661-a40bf316665e",
    "zones": [
        {
            "id": "9248cab2-afe7-43d8-a661-a40bf316665e",
            "name": "us-east",
            "endpoints": [
                "http://rgw1"
            ],
            "log_meta": "true",
            "log_data": "true",
            "bucket_index_max_shards": 11,
            "read_only": "false"
        },
        {
            "id": "d1024e59-7d28-49d1-8222-af101965a939",
            "name": "us-west",
            "endpoints": [
                "http://rgw2:80"
            ],
            "log_meta": "false",
            "log_data": "true",
            "bucket_index_max_shards": 11,
            "read_only": "false"
        }
    ],
    "placement_targets": [
        {
            "name": "default-placement",
            "tags": []
        }
    ],
    "default_placement": "default-placement",
    "realm_id": "ae031368-8715-4e27-9a99-0c9468852cfe"
}
```

Setting a zone group

Set a zone group by creating a JSON object and specifying minimally the required settings.

Specify the following settings in the zone group JSON object:

`name`

The name of the zone group. Required.

`api_name`

The API name for the zone group. Optional.

`is_master`

Determines if the zone group is the primary zone group. Required.

> **Note:** You can only have one primary zone group.

`endpoints`

A list of all the endpoints in the zone group. For example, you may use multiple domain names to refer to the same zone group. Remember to escape the forward slashes (/). You may also specify a port (`fqdn:port`) for each endpoint. Optional.

`hostnames`

A list of all the hostnames in the zone group. For example, you may use multiple domain names to refer to the same zone group. Optional. The **`rgw dns name`** setting will automatically be included in this list. You should restart the gateway daemon(s) after changing this setting.

`master_zone`

The primary zone for the zone group. Optional. Uses the default zone if not specified.

> **Note:** You can only have one primary zone per zone group.

`zones`

A list of all zones within the zone group. Each zone has a name (required), a list of endpoints (optional), and whether or not the gateway will log metadata and data operations (false by default).

`placement_targets`

A list of placement targets (optional). Each placement target contains a name (required) for the placement target and a list of tags (optional) so that only users with the tag can use the placement target (i.e., the user's **`placement_tags`** field in the user info).

`default_placement`

The default placement target for the object index and object data. Set to `default-placement` by default. You may also set a per-user default placement in the user info for each user.

To set a zone group, create a JSON object consisting of the required fields, save the object to a file, for example, `zonegroup.json`; then, run the following command:

Example

```
[ceph: root@host01 /]# radosgw-admin zonegroup set --infile zonegroup.json
```

Where `zonegroup.json` is the JSON file you created.

> **Important:** The `default` zone group **`is_master`** setting is `true` by default. If you create a new zone group and want to make it the primary zone group, you must either set the `default` zone group **`is_master`** setting to `false`, or delete the `default` zone group.

Finally, update the period:

Example

```
[ceph: root@host01 /]# radosgw-admin period update --commit
```

Zones

Ceph Object Gateway supports the notion of zones. A zone defines a logical group consisting of one or more Ceph Object Gateway instances.

Configuring zones differs from typical configuration procedures, because not all of the settings end up in a Ceph configuration file. You can list zones, get a zone configuration, and set a zone configuration.

> **Important:** All `radosgw-admin zone` operations **MUST** be issued on a host that operates or will operate within the zone.

Creating a zone

Create a zone by specifying a zone name.

About this task

If it is a primary zone, specify the `--master` option. Only one zone in a zone group can be a primary zone. To add the zone to a zone group, specify the `--rgw-zonegroup` option with the zone group name.

> **Important:** Zones must be created on a Ceph Object Gateway node that will be within the zone.

> **Important:** Do not use the zone with the **`--default`** flag if the data and metadata are stored in the `default.rgw.data` and `default.rgw.index` pools. If a new zone is set as the default and these pools contain important data, the `radosgw-admin` utility can fail to manage this data properly.
> Only use the **`--default`** flag if necessary to specify the zone as the default and if you do not need the existing data or metadata in the `default.rgw` pools. If the existing data or metadata is needed, either migrate the default configuration to a multi-site or zone setup or avoid setting the new zone as the default. For more information about migrating to a multi-site, see "Migrating a single site system to multi-site" on page 861. By specifying **`--default`**, the zone is called implicitly with each `radosgw-admin` call unless `--rgw-zone` and the zone name are explicitly provided.

Procedure

1. Create the zone.

   ```
   radosgw-admin zone create --rgw-zone=ZONE_NAME
                  [--zonegroup=ZONE_GROUP_NAME]
                  [--endpoints=ENDPOINT_PORT [,<endpoint:port>]
                  [--master] [--default]
                  --access-key ACCESS_KEY --secret SECRET_KEY
   ```

2. Commit the change.

   ```
   radosgw-admin period update --commit
   ```

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin period update --commit
   ```

Deleting a zone

Learn how to delete a zone.

Procedure

To delete a zone, start by removing it from the zonegroup.

1. Remove the zone from the zonegroup.

   Important: Do not delete a zone without removing it from a zone group first. Otherwise, updating the period fails.

   ```
   radosgw-admin zonegroup remove --rgw-zonegroup=ZONE_GROUP_NAME
                                  --rgw-zone=ZONE_NAME
   ```

2. Update the period.

   ```
   radosgw-admin period update --commit
   ```

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin period update --commit
   ```

3. Delete the zone.

   Important: This procedure must be used on a host within the zone.

   ```
   radosgw-admin zone delete --rgw-zone=ZONE_NAME
   ```

4. Update the period.

   ```
   radosgw-admin period update --commit
   ```

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin period update --commit
   ```

5. Delete the pools.
   If the pools for the deleted zone will not be used anywhere else, consider deleting the pools. Replace *DELETED_ZONE_NAME* in the following example with the deleted zone's name.

   Important:

   - Once Ceph deletes the zone pools, it deletes all of the data within them in an unrecoverable manner. Only delete the zone pools if Ceph clients no longer need the pool contents.
   - In a multi-realm cluster, deleting the `.rgw.root` pool along with the zone pools removes ALL the realm information for the cluster. Ensure that `.rgw.root` does not contain other active realms before deleting the `.rgw.root` pool.

   ```
   ceph osd pool delete DELETED_ZONE_NAME.rgw.control DELETED_ZONE_NAME.rgw.control --
   yes-i-really-really-mean-it
   ceph osd pool delete DELETED_ZONE_NAME.rgw.data.root DELETED_ZONE_NAME.rgw.data.root
   --yes-i-really-really-mean-it
   ceph osd pool delete DELETED_ZONE_NAME.rgw.log DELETED_ZONE_NAME.rgw.log --yes-i-
   really-really-mean-it
   ceph osd pool delete DELETED_ZONE_NAME.rgw.users.uid DELETED_ZONE_NAME.rgw.users.uid
   --yes-i-really-really-mean-it
   ```

6. Restart the Ceph Object Gateway process.

Modifying a zone

To modify a zone, specify the zone name and the parameters that you want to modify.

> **Important:** Zones should be modified on a Ceph Object Gateway node that will be within the zone.

Syntax

```
radosgw-admin zone modify [options]

`--access-key=<key>`
`--secret/--secret-key=<key>`
`--master`
`--default`
`--endpoints=<list>`
```

Then, update the period:

Example

```
[ceph: root@host01 /]# radosgw-admin period update --commit
```

Listing zones

Use the **`zone list`** command to list zones in a cluster.

As `root`, to list the zones in a cluster, run the following command:

Example

```
[ceph: root@host01 /]# radosgw-admin zone list
```

Getting a zone

Use the **`zone get`** command to retrieve a zone configuration.

As `root`, to get the configuration of a zone, run the following command:

Syntax

```
radosgw-admin zone get [--rgw-zone=ZONE_NAME]
```

The `default` zone looks like this:

```
{ "domain_root": ".rgw",
  "control_pool": ".rgw.control",
  "gc_pool": ".rgw.gc",
  "log_pool": ".log",
  "intent_log_pool": ".intent-log",
  "usage_log_pool": ".usage",
  "user_keys_pool": ".users",
  "user_email_pool": ".users.email",
  "user_swift_pool": ".users.swift",
  "user_uid_pool": ".users.uid",
  "system_key": { "access_key": "", "secret_key": ""},
  "placement_pools": [
      {  "key": "default-placement",
         "val": { "index_pool": ".rgw.buckets.index",
                  "data_pool": ".rgw.buckets"}
      }
    ]
  }
```

Setting a zone

Configuring a zone involves specifying a series of Ceph Object Gateway pools.

For consistency, use a pool prefix that is the same as the zone name. See the Pools for details on configuring pools.

> **Important:** Zones should be set on a Ceph Object Gateway node that will be within the zone.

To set a zone, create a JSON object consisting of the pools, save the object to a file, for example, `zone.json`; then, run the following command, replacing `ZONE_NAME` with the name of the zone:

Example

```
[ceph: root@host01 /]# radosgw-admin zone set --rgw-zone=test-zone --infile zone.json
```

Where `zone.json` is the JSON file you created.

Then, as `root`, update the period:

Example

```
[ceph: root@host01 /]# radosgw-admin period update --commit
```

Renaming a zone

use the **zone reanme** command to rename a zone on a host.

To rename a zone, specify the zone name and the new zone name. Issue the following command on a host within the zone:

Syntax

```
radosgw-admin zone rename --rgw-zone=ZONE_NAME --zone-new-name=NEW_ZONE_NAME
```

Then, update the period:

Example

```
[ceph: root@host01 /]# radosgw-admin period update --commit
```

Adding a zone to a zone group

Use the **zonegroup add** command to add a zone to a zone group.

This command must be run on a host that will be in the zone. To add a zone to a zonegroup, run the following command:

Syntax

```
radosgw-admin zonegroup add --rgw-zonegroup=ZONE_GROUP_NAME --rgw-zone=ZONE_NAME
```

Then, update the period:

Example

```
[ceph: root@host01 /]# radosgw-admin period update --commit
```

Removing a zone from a zone group

Use the **zonegroup remove** command to remove a zone from a zone group.

To remove a zone from a zone group, run the following command:

Syntax

```
radosgw-admin zonegroup remove --rgw-zonegroup=ZONE_GROUP_NAME --rgw-zone=ZONE_NAME
```

Then, update the period:

Example

```
[ceph: root@host01 /]# radosgw-admin period update --commit
```

Multi-site configuration and administration

Configure and administer multiple Ceph Object Gateways across realms, zone groups, and zones for disaster recovery, failover, and multi-site synchronization.

A single zone configuration typically consists of one zone group containing one zone and one or more `ceph-radosgw` instances where you can load-balance gateway client requests between the instances. In a single zone configuration, multiple gateway instances typically point to a single Ceph storage cluster. However, Red Hat supports several multi-site configuration options for the Ceph Object Gateway:

Multi-zone

Consists of one zone group and multiple zones, where each zone has one or more `ceph-radosgw` instances. Each zone is backed by its own Ceph Storage Cluster. Multiple zones in a zone group provide disaster recovery for the zone group if one zone experiences a significant failure. Each zone is active and can receive write operations. In addition to disaster recovery, multiple active zones can serve as a foundation for content delivery networks.

Multi-zone-group

Formerly called "regions", the Ceph Object Gateway also supports multiple zone groups, with each zone group containing one or more zones. Objects stored in zone groups within the same realm share a global namespace, ensuring unique object IDs across zone groups and zones.

Multiple realms

The Ceph Object Gateway supports realms, which can contain a single zone group or multiple zone groups with a globally unique namespace. Multiple realms provide the ability to support various configurations and namespaces.

Note: When objects are synchronized between zones, the destination zone must have the corresponding storage class configured to preserve storage class placement. If the source storage class is not configured in the destination zone, synchronization continues and the object is stored by using the destination zone's `STANDARD` storage class.

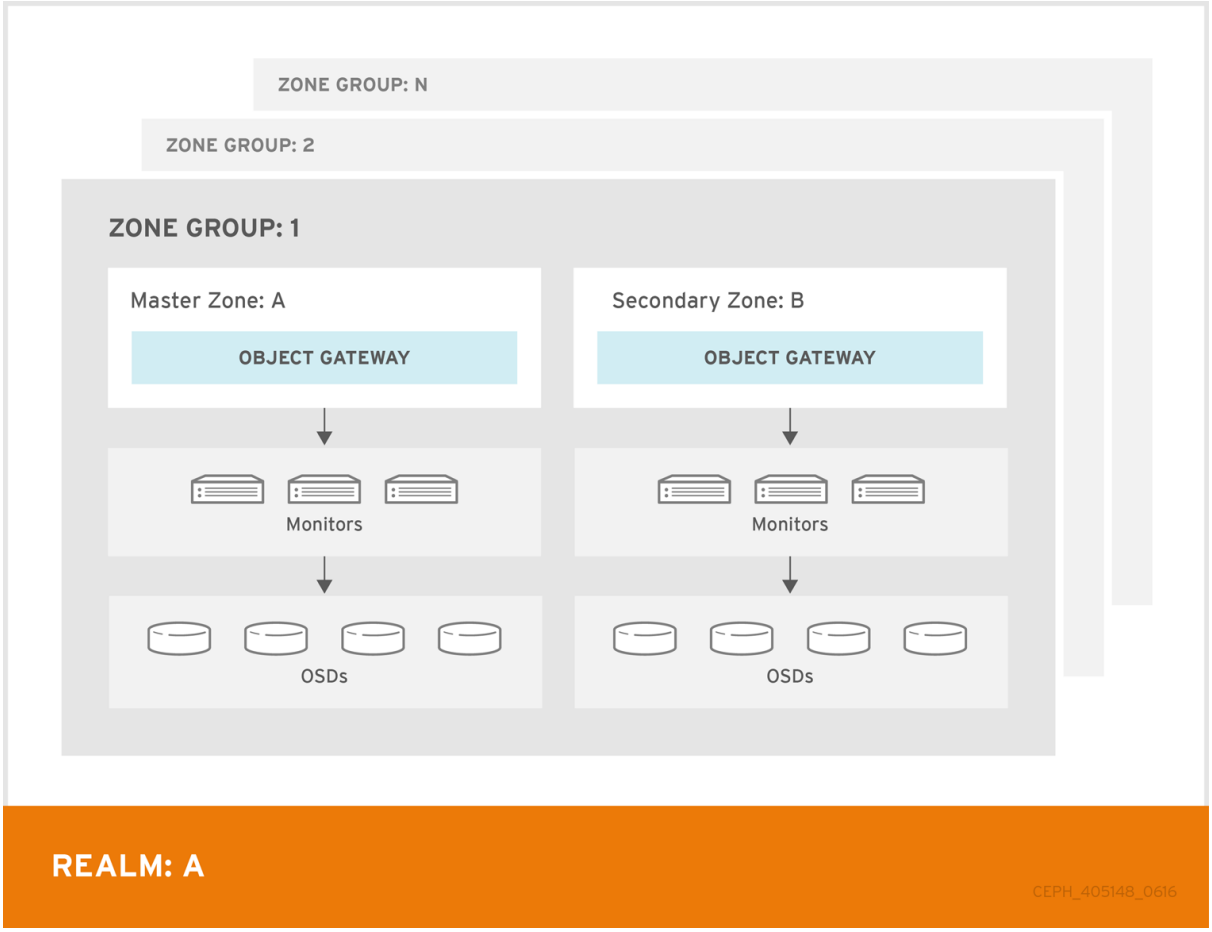


Figure 89: Ceph Object Gateway realm

Related information

Deployment

Requirements and assumptions

A multi-site configuration requires at least two Ceph storage clusters, and at least two Ceph Object Gateway instances, one for each Ceph storage cluster. The information in this documentation assumes at least two Ceph storage clusters in geographically separate locations; however, the configuration can work on the same physical site. This information also assumes four Ceph Object Gateway servers named `rgw1`, `rgw2`, `rgw3`, and `rgw4` respectively.

A multi-site configuration requires a primary zone group and a primary zone. Also, each zone group requires a primary zone. Zone groups might have one or more secondary or non-primary zones.

Important: When planning network considerations for multi-site, it is important to understand the relation bandwidth and latency that is observed on the multi-site synchronization network and the clients ingest rate in direct correlation with the current sync state of the objects owed to the secondary site. Multi-site synchronization is asynchronous and one of the limitations is the rate at which the sync gateways can process data across the link. An example to look at in terms of network inter-connectivity speed might be 1 GbE or inter-datacenter connectivity, for every 8 TB or cumulative receive data, per client gateway. Thus, if you replicate to two other sites, and ingest 16 TB a day, you need 6 GbE of dedicated bandwidth for multi-site replication.

Red Hat also recommends private Ethernet or Dense wavelength-division multiplexing (DWDM) as a VPN over the internet is not ideal due to the additional overhead incurred.

Important: The primary zone within the primary zone group of a realm is responsible for storing the primary copy of the realm's metadata, including users, quotas, and buckets (created by the `radosgw-admin` CLI). This metadata gets synchronized to secondary zones and secondary zone groups automatically. Metadata operations run with the `radosgw-admin` CLI **MUST be run** on a host within the primary zone of the primary zone group in order to ensure that they get synchronized to the secondary zone groups and zones. Currently, it is *possible* to run metadata operations on secondary zones and zone groups, but it is **NOT recommended** because they **WILL NOT** be synchronized, leading to fragmented metadata.

Note: For new Ceph Object Gateway deployment in multi-site, it takes around 20 minutes to sync metadata operations to the secondary site.

In the following examples, the `rgw1` host serves as the primary zone of the primary zone group; the `rgw2` host serves as the secondary zone of the primary zone group; the `rgw3` host serves as the primary zone of the secondary zone group; and the `rgw4` host serves as the secondary zone of the secondary zone group.

Important: Red Hat recommends provisioning 3 Ceph Object Gateway daemons that are dedicated to sync processing in a multi-site deployment. Prevent the other nonsyncing Ceph Object Gateway daemons in the multi-site configuration that are to be dedicated to client I/O operations through an ingress service or other load balancer from running sync operations. Prevent the sync operations, by running the following command:

```
ceph config set client.rgw.CLIENT_NODE rgw_run_sync_thread false
```

After running the command, restart those client-facing Ceph Object Gateway daemons. These **`ceph config`** commands must substitute your actual host or service names for *`CLIENT_NODE`*. This should be backported to previous releases as well, as this is purely improvement to language and clarity.

The following is a typical example configuration file for HAProxy for syncing gateways:

```
[root@host01 ~]# cat ./haproxy.cfg

global

  log         127.0.0.1 local2

  chroot      /var/lib/haproxy
  pidfile     /var/run/haproxy.pid
  maxconn     7000
  user        haproxy
  group       haproxy
  daemon

  stats socket /var/lib/haproxy/stats

defaults
  mode                    http
  log                     global
  option                  httplog
  option                  dontlognull
  option http-server-close
  option forwardfor       except 127.0.0.0/8
  option                  redispatch
  retries                 3
  timeout http-request    10s
  timeout queue           1m
  timeout connect         10s
  timeout client          30s
  timeout server          30s
  timeout http-keep-alive 10s
  timeout check           10s
        timeout client-fin 1s
        timeout server-fin 1s
  maxconn                 6000


listen  stats
  bind    0.0.0.0:1936
      mode            http
      log             global

      maxconn 256


      clitimeout      10m
      srvtimeout      10m
      contimeout      10m
      timeout queue   10m

# JTH start
      stats enable
      stats hide-version
      stats refresh 30s
      stats show-node
##        stats auth admin:password
      stats uri  /haproxy?stats
      stats admin if TRUE


frontend  main
  bind     *:5000
  acl url_static       path_beg       -i /static /images /javascript /stylesheets
  acl url_static       path_end       -i .jpg .gif .png .css .js

  use_backend static          if url_static
  default_backend             app
        maxconn 6000

backend static
  balance     roundrobin
    fullconn 6000
    server  app8 host01:8080 check maxconn 2000
    server  app9 host02:8080 check maxconn 2000
    server  app10 host03:8080 check maxconn 2000


backend app
  balance     roundrobin
    fullconn 6000
    server  app8 host01:8080 check maxconn 2000
```

```
    server  app9 host02:8080 check maxconn 2000
    server  app10 host03:8080 check maxconn 2000
```

Pools

Calculate a suitable number of placement groups for the pools the `radosgw` daemon will create. Set the calculated values as defaults in the Ceph configuration database.

Use the *Ceph Placement Group's per Pool Calculator* to calculate a suitable number of placement groups for the pools the `radosgw` daemon will create.

For example,

```
[ceph: root@host01 /]#  ceph config set osd osd_pool_default_pg_num = 32
[ceph: root@host01 /]#  ceph config set osd osd_pool_default_pgp_num = 32
```

Note: Making this change to the Ceph configuration will use those defaults when the Ceph Object Gateway instance creates the pools. Alternatively, you can create the pools manually.

Pool names particular to a zone follow the naming convention `ZONE_NAME.POOL_NAME`. For example, a zone named `us-east` will have the following pools:

- `.rgw.root`
- `us-east.rgw.control`
- `us-east.rgw.meta`
- `us-east.rgw.log`
- `us-east.rgw.buckets.index`
- `us-east.rgw.buckets.data`
- `us-east.rgw.buckets.non-ec`
- `us-east.rgw.meta:users.keys`
- `us-east.rgw.meta:users.email`
- `us-east.rgw.meta:users.swift`
- `us-east.rgw.meta:users.uid`

Reference

For more information, see the following:

- *Pools*

Migrating a single site system to multi-site

You can migrate from a single site system with a `default` zone group and zone to a multi-site system.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- A Ceph Object Gateway installed.

Procedure

1. Create a realm.

```
radosgw-admin realm create --rgw-realm REALM_NAME --default
```

Replace *REALM_NAME* with the realm name.

2. Rename the default zone and zone group.

```
radosgw-admin zonegroup rename --rgw-zonegroup default --zonegroup-new-name
NEW_ZONE_GROUP_NAME
radosgw-admin zone rename --rgw-zone default --zone-new-name NEW_ZONE_NAME --rgw-
zonegroup NEW_ZONE_GROUP_NAME
```

   Replace *NEW_ZONE_GROUP_NAME* and *NEW_ZONE_NAME* with the zone group and zone name, respectively.

3. Rename the default zone group's **api_name**.

```
radosgw-admin zonegroup modify --api-name NEW_ZONE_GROUP_NAME --rgw-zonegroup
NEW_ZONE_GROUP_NAME
```

   Replace *NEW_ZONE_GROUP_NAME* with the zone group name.

   The **api_name** field in the zone group map refers to the name of the RADOS API used for data replication across different zones. This field helps clients interact with the correct APIs for accessing and managing data within the Ceph storage cluster.

4. Configure the primary zone group.

```
radosgw-admin zonegroup modify --rgw-realm REALM_NAME --rgw-zonegroup
NEW_ZONE_GROUP_NAME --endpoints http://ENDPOINT --master --default
```

   Replace the following:

   - *REALM_NAME* with realm name.
   - *NEW_ZONE_GROUP_NAME* with the zone group name.
   - *ENDPOINT* with the fully qualified domain names in the zone group.

5. Configure the primary zone.

```
radosgw-admin zone modify --rgw-realm REALM_NAME --rgw-zonegroup NEW_ZONE_GROUP_NAME
--rgw-zone NEW_ZONE_NAME --endpoints http://ENDPOINT --master --default
```

   Replace the following:

   - *REALM_NAME* with realm name.
   - *NEW_ZONE_GROUP_NAME* with the zone group name.
   - *NEW_ZONE_NAME* with the zone name.
   - *ENDPOINT* with the fully qualified domain names in the zone group.

6. Create a system user.

```
radosgw-admin user create --uid USER_ID --display-name DISPLAY_NAME --access-key
ACCESS_KEY --secret SECRET_KEY --system
```

   Important: The `--system` flag is intended for multi-site replication tasks. It behaves like the `--admin` flag but also overrides quota checks and adds extra metadata to S3 responses. Do not use the `--system` flag outside a multi-site replication context.

   Replace the following:

   - *USER_ID* with the username.
   - *DISPLAY_NAME* with the display name. The display name can contain spaces.

7. Commit the updated configuration.

```
radosgw-admin period update --commit
```

For example,

```
[ceph: root@host01 /]# radosgw-admin period update --commit
```

8. Restart the Ceph Object Gateway.

```
systemctl restart ceph-radosgw@rgw.`hostname -s`
ceph orch restart RGW_SERVICE_NAME
```

For example,

```
[ceph: root@host01 /]# systemctl restart ceph-radosgw@rgw.`hostname -s`
[ceph: root@host01 /]# ceph orch restart rgw.rgwsvcid.mons-1.jwgwwp
```

Establishing a secondary zone

Zones within a zone group replicate all data to ensure that each zone has the same data. When creating the secondary zone, issue all of the `radosgw-admin zone` operations on a host identified to serve the secondary zone.

Before you begin

- At least two Red Hat Ceph Storage clusters.
- At least two Ceph Object Gateway instances, one for each Red Hat Ceph Storage cluster.
- Root-level access to all the nodes.
- Nodes or containers are added to the storage cluster.
- All Ceph Manager, Monitor, and OSD daemons are deployed.

About this task

Learn how to establish a secondary zone.

Note: To add additional zones, follow the same procedures as for adding the secondary zone. Use a different zone name.

Important:

- Run the metadata operations, such as user creation and quotas, on a host within the primary zone of the primary zonegroup. The primary zone and the secondary zone can receive bucket operations from the RESTful APIs, but the secondary zone redirects bucket operations to the primary zone. If the primary zone is down, bucket operations will fail. If you create a bucket using the `radosgw-admin` CLI, you must run it on a host within the primary zone of the primary zone group so that the buckets will synchronize with other zone groups and zones.
- Bucket creation for a particular user is not supported, even if you create a user in the secondary zone with **`--yes-i-really-mean-it`**.
- Creating a user from a non-primary site is successful when using the **`--yes-i-really-mean-it`** however the user does not sync to the primary site.

Procedure

1. Log into the `cephadm` shell.
   For example,

```
[root@host04 ~]# cephadm shell
```

2. Pull the primary realm configuration from the host.

```
radosgw-admin realm pull --url=URL_TO_PRIMARY_ZONE_GATEWAY --access-key=ACCESS_KEY --
secret-key=SECRET_KEY
```

For example,

```
[ceph: root@host04 /]# radosgw-admin realm pull --url=http://10.74.249.26:80 --access-
key=LIPEYZJLTWXRKXS9LPJC --secret-key=IsAje0AVDNXNw48LjMAimpCpI7VaxJYSnfD0FFKQ
```

3. Pull the primary period configuration from the host.

```
radosgw-admin period pull --url=URL_TO_PRIMARY_ZONE_GATEWAY --access-key=ACCESS_KEY --
secret-key=SECRET_KEY
```

For exaple,

```
[ceph: root@host04 /]# radosgw-admin period pull --url=http://10.74.249.26:80 --
access-key=LIPEYZJLTWXRKXS9LPJC --secret-key=IsAje0AVDNXNw48LjMAimpCpI7VaxJYSnfD0FFKQ
```

4. Configure a secondary zone.

> **Note:** All zones run in an active-active configuration by default; that is, a gateway client might write data to any zone and the zone will replicate the data to all other zones within the zone group. If the secondary zone should not accept write operations, specify the `--read-only` flag to create an active-passive configuration between the primary zone and the secondary zone. Additionally, provide the `access_key` and `secret_key` of the generated system user stored in the primary zone of the primary zone group.

```
radosgw-admin zone create --rgw-zonegroup=ZONE_GROUP_NAME \
             --rgw-zone=SECONDARY_ZONE_NAME --endpoints=http://
RGW_SECONDARY_HOSTNAME:RGW_PRIMARY_PORT_NUMBER_1 \
             --access-key=SYSTEM_ACCESS_KEY --secret=SYSTEM_SECRET_KEY \
             [--read-only]
```

For example,

```
[ceph: root@host04 /]# radosgw-admin zone create --rgw-zonegroup=us --rgw-zone=us-
east-2 --endpoints=http://rgw2:80 --access-key=LIPEYZJLTWXRKXS9LPJC --secret-
key=IsAje0AVDNXNw48LjMAimpCpI7VaxJYSnfD0FFKQ
```

5. Optional: Delete the default zone.

> **Important:** Do not delete the default zone and its pools if you are using the default zone and zone group to store data.

For example,

```
[ceph: root@host04 /]# radosgw-admin zone rm --rgw-zone=default
[ceph: root@host04 /]# ceph osd pool rm default.rgw.log default.rgw.log --yes-i-
really-really-mean-it
[ceph: root@host04 /]# ceph osd pool rm default.rgw.meta default.rgw.meta --yes-i-
really-really-mean-it
[ceph: root@host04 /]# ceph osd pool rm default.rgw.control default.rgw.control --yes-
i-really-really-mean-it
[ceph: root@host04 /]# ceph osd pool rm default.rgw.data.root default.rgw.data.root --
yes-i-really-really-mean-it
[ceph: root@host04 /]# ceph osd pool rm default.rgw.gc default.rgw.gc --yes-i-really-
really-mean-it
```

6. Optional: If you specified the realm and zone in the service specification during the deployment of the Ceph Object Gateway, update the `spec` section of the specification file.

```
spec:
rgw_realm: REALM_NAME
rgw_zone: ZONE_NAME
```

7. Update the Ceph configuration database.

```
ceph config set client.rgw.SERVICE_NAME rgw_realm REALM_NAME
ceph config set client.rgw.SERVICE_NAME rgw_zonegroup ZONE_GROUP_NAME
ceph config set client.rgw.SERVICE_NAME rgw_zone SECONDARY_ZONE_NAME
```

For example,

```
[ceph: root@host04 /]# ceph config set client.rgw.rgwsvcid.mons-1.jwgwwp rgw_realm
test_realm
[ceph: root@host04 /]# ceph config set client.rgw.rgwsvcid.mons-1.jwgwwp rgw_zonegroup
us
[ceph: root@host04 /]# ceph config set client.rgw.rgwsvcid.mons-1.jwgwwp rgw_zone us-
east-2
```

8. Commit the changes.

```
radosgw-admin period update --commit
```

For example,

```
[ceph: root@host04 /]# radosgw-admin period update --commit
```

9. Outside the `cephadm` shell, fetch the FSID of the storage cluster and the processes.
For example,

```
[root@host04 ~]#  systemctl list-units | grep ceph
```

10. Start the Ceph Object Gateway daemon.

```
systemctl start ceph-FSID@DAEMON_NAME
systemctl enable ceph-FSID@DAEMON_NAME
```

For example,

```
[root@host04 ~]# systemctl start ceph-62a081a6-88aa-11eb-
a367-001a4a000672@rgw.test_realm.us-east-2.host04.ahdtsw.service
[root@host04 ~]# systemctl enable ceph-62a081a6-88aa-11eb-
a367-001a4a000672@rgw.test_realm.us-east-2.host04.ahdtsw.service
```

Configuring the archive zone

Archive object data residing on Red Hat Ceph Storage using the Object Storage Archive Zone Feature.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Root-level access to a Ceph Monitor node.
- Ceph Object Gateway software is installed.

About this task

> **Note:** Ensure you have a realm before configuring a zone as an archive. Without a realm, you cannot archive data through an archive zone for default zone/zonegroups.

The archive zone uses multi-site replication and S3 object versioning feature in Ceph Object Gateway. The archive zone retains all version of all the objects available, even when deleted in the production file.
The archive zone has a history of versions of S3 objects that can be eliminated only through the gateways that are associated with the archive zone. It captures all the data updates and metadata to consolidate them as versions of S3 objects. Bucket granular replication to the archive zone can be used after creating an archive zone.

You can control the storage space usage of an archive zone through the bucket Lifecycle policies, where you can define the number of versions you would like to keep for an object.

An archive zone helps protect your data against logical or physical errors. It can save users from logical failures, such as accidentally deleting a bucket in the production zone. It can also save your data from massive hardware failures, like a complete production site failure. Additionally, it provides an immutable copy, which can help build a ransomware protection strategy.

To implement the bucket granular replication, use the sync policies commands for enabling and disabling policies. For more information, see "Creating a sync policy group" on page 883 and "Modifying a sync policy group" on page 883.

> **Note:** Using the sync policy group procedures is optional and only necessary to use enabling and disabling with bucket granular replication. For using the archive zone without bucket granular replication, it is not necessary to use the sync policy procedures.

If you want to migrate the storage cluster from single site, see "Migrating a single site system to multi-site" on page 861.

Procedure

- During new zone creation, use the `archive` tier to configure the archive zone.

  ```
  radosgw-admin zone create --rgw-zonegroup={ZONE_GROUP_NAME} --rgw-zone={ZONE_NAME} --
  endpoints={http://FQDN:PORT},{http://FQDN:PORT} --tier-type=archive
  ```

  For example,

  ```
  [ceph: root@host01 /]# radosgw-admin zone create --rgw-zonegroup=us --rgw-zone=us-east
  --endpoints={http://example.com:8080} --tier-type=archive
  ```

- From the archive zone, modify the archive zone to sync from only the primary zone and perform a period update commit.

  ```
  $ radosgw-admin zone modify --rgw-zone archive --sync_from primary --sync_from_all
  false --sync-from-rm secondary

  $ radosgw-admin period update --commit
  ```

  > **Note:** The recommendation is to reduce the `max_objs_per_shard` to 50K to account for the omap olh entries in the archive zone. This helps in keeping the number of omap entries per bucket index shard object in check to prevent large omap warnings. For example,
  >
  > ```
  > ceph config set client.rgw rgw_max_objs_per_shard 50000
  > ```

Related information

Deploying a multi-site Ceph Object Gateway using the Ceph Orchestrator
S3 add an object to a bucket

Deleting objects in archive zone

Use the S3 lifecycle policy extension to delete objects within an **<ArchiveZone>** element.

About this task

> **Important:** Archive zone objects can only be deleted using the **expiration** lifecycle policy rule.

- If any **<Rule>** section contains an **<ArchiveZone>** element, that rule runs in archive zone and is the only rule that runs in an archive zone.
- Rules marked **<ArchiveZone>** do NOT run in nonarchive zones.

The rules within the lifecycle policy determine when and what objects to delete. For more information about lifecycle creation and management, see “Configuring Bucket lifecycle” on page 922.

Procedure

1. Set the **<ArchiveZone>** lifecycle policy rule.
   For more information about creating a lifecycle policy, see “Creating a lifecycle management policy” on page 923.
   For example,

```
<?xml version="1.0" ?>
<LifecycleConfiguration xmlns="http://s3.amazonaws.com/doc/2006-03-01/">
        <Rule>
                <ID>delete-1-days-az</ID>
                <Filter>
          <Prefix></Prefix>
          <ArchiveZone />
                </Filter>
                <Status>Enabled</Status>
                <Expiration>
                        <Days>1</Days>
                </Expiration>
        </Rule>
</LifecycleConfiguration>
```

2. Optional: Verify whether a specific lifecycle policy contains an archive zone rule.

```
radosgw-admin lc get --bucket BUCKET_NAME
```

   The following example shows the output of a lifecycle policy with an archive zone rule.

```
[ceph: root@host01 /]# radosgw-admin lc get --bucket test-bkt

{
    "prefix_map": {
        "": {
            "status": true,
            "dm_expiration": true,
            "expiration": 0,
            "noncur_expiration": 2,
            "mp_expiration": 0,
            "transitions": {},
            "noncur_transitions": {}
        }
    },
    "rule_map": [
        {
            "id": "Rule 1",
            "rule": {
                "id": "Rule 1",
                "prefix": "",
                "status": "Enabled",
                "expiration": {
                    "days": "",
                    "date": ""
                },
                "noncur_expiration": {
                    "days": "2",
                    "date": ""
                },
                "mp_expiration": {
                    "days": "",
                    "date": ""
                },
                "filter": {
                    "prefix": "",
                    "obj_tags": {
                        "tagset": {}
                    },
                    "archivezone": ""
                },
                "transitions": {},
                "noncur_transitions": {},
                "dm_expiration": true
            }
        }
    ]
}
```

3. If the Ceph Object Gateway user is deleted, the buckets at the archive site owned by that user is inaccessible. Link those buckets to another Ceph Object Gateway user to access the data.

```
radosgw-admin bucket link --uid NEW_USER_ID --bucket BUCKET_NAME --yes-i-really-mean-
it
```

```
[ceph: root@host01 /]# radosgw-admin bucket link --uid arcuser1 --bucket arc1-deleted-
da473fbbaded232dc5d1e434675c1068 --yes-i-really-mean-it
```

Related information

S3 bucket lifecycle
Managing your storage lifecycle
Configuring Bucket lifecycle

Failover and disaster recovery

Recover your data from different failover and disaster scenarios.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Root-level access to a Ceph Monitor node.

– Installation of the Ceph Object Gateway software.

About this task

Use any of the following procedures, according to your needs:

– “Primary zone failure with a failover to the secondary zone for disaster recovery” on page 869

– “Switching archive zone sync from primary to secondary zone with I/Os in progress” on page 870

– “Archive zone syncs only from primary zone with primary zone failure” on page 870

Primary zone failure with a failover to the secondary zone for disaster recovery

Procedure

1. Make the secondary zone the primary and default zone.

   ```
   radosgw-admin zone modify --rgw-zone=ZONE_NAME --master --default
   ```

   By default, Ceph Object Gateway runs in an active-active configuration. If the cluster was configured to run in an active-passive configuration, the secondary zone is a read-only zone. Remove the `--read-only` status to allow the zone to receive write operations.

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin zone modify --rgw-zone=ZONE_NAME --master --
   default --read-only=false
   ```

2. Update the period to make the changes take effect.

   ```
   radosgw-admin period update --commit
   ```

3. Restart the Ceph Object Gateway.

   Note: Use the output from the **ceph orch ps** command, under the `NAME` column, to get the *SERVICE_TYPE.ID* information.

   – Restart the Ceph Object Gateway on an individual node in the storage cluster.

   ```
   systemctl restart ceph-CLUSTER_ID@SERVICE_TYPE.ID.service
   ```

   For example,

   ```
   [root@host01 ~]# systemctl restart ceph-c4b34c6f-8365-11ba-
   dc31-529020a7702d@rgw.realm.zone.host01.gwasto.service
   ```

   – Restart the Ceph Object Gateways on all nodes in the storage cluster.

   ```
   ceph orch restart SERVICE_TYPE
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch restart rgw
   ```

   If the former primary zone recovers, revert the operation.

4. From the recovered zone, pull the realm from the current primary zone.

   ```
   radosgw-admin realm pull --url=URL_TO_PRIMARY_ZONE_GATEWAY
                            --access-key=ACCESS_KEY --secret=SECRET_KEY
   ```

5. Make the recovered zone the primary and default zone.

   ```
   adosgw-admin zone modify --rgw-zone=ZONE_NAME --master --default
   ```

6. Update the period to make the changes take effect.

```
radosgw-admin period update --commit
```

7. Restart the Ceph Object Gateway in the recovered zone.

```
ceph orch restart SERVICE_TYPE
```

For example,

```
[ceph: root@host01 /]# ceph orch restart rgw
```

8. If the secondary zone needs to be a read-only configuration, update the secondary zone.

```
radosgw-admin zone modify --rgw-zone=ZONE_NAME --read-only
radosgw-admin zone modify --rgw-zone=ZONE_NAME --read-only
```

a. Update the period to make the changes take effect.

```
radosgw-admin period update --commit
```

b. Restart the Ceph Object Gateway in the recovered zone.
For example,

```
[ceph: root@host01 /]# ceph orch restart rgw
```

Switching archive zone sync from primary to secondary zone with I/Os in progress

Procedure

1. Switch the sync source to the secondary zone.

```
radosgw-admin zone modify --rgw-zone archive --sync_from secondary --sync_from_all
false --sync-from-rm primary radosgw-admin period update --commit
```

2. Verify the sync status and data consistency in the archive zone.
3. Switch the sync source back to the primary zone.

```
radosgw-admin zone modify --rgw-zone archive --sync_from primary --sync_from_all false
--sync-from-rm secondary radosgw-admin period update --commit
```

4. Verify the sync status and data consistency in the archive zone after the switch.
As a result:
   - The archive zone starts syncing from the secondary zone after the first modification.
   - Data consistency is maintained throughout the switch.
   - Upon switching back to the primary zone, the archive zone resumes syncing from the primary zone without data loss or corruption.
   - Sync remains consistent in all the zones.

Archive zone syncs only from primary zone with primary zone failure

Procedure

1. Ensure that the archive zone is set to sync only from the primary zone.
2. Stop the primary zone gateways to simulate a primary zone failure.
3. Failover to the secondary zone and run the period update commit.
4. Observe the sync status of the archive zone.
5. Post a time interval of about 30 minutes, restart the primary zone gateways.

6. Verify that the archive zone resumes syncing from the primary zone.
   As a result:
   - The archive zone stops syncing when the primary zone fails.
   - After restoring the primary zone, the archive zone automatically resumes syncing from the primary zone.
   - Data integrity and sync status is maintained throughout the process.

3-site failover and disaster recovery

Recover your data from different failover and disaster scenarios.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Root-level access to a Ceph Monitor node.
- Installation of the Ceph Object Gateway software.

Procedure

1. Make the either the secondary or tertiary zone the primary and default zone.

   ```
   radosgw-admin zone modify --rgw-zone=ZONE_NAME --master --default
   ```

   By default, Ceph Object Gateway runs in an active-active-active configuration. If the cluster was configured to run in an active-passive-passive configuration, the secondary and tertiary zones are read-only zones. Remove the `--read-only` status to allow the zone to receive write operations.

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin zone modify --rgw-zone=ZONE_NAME --master --
   default --read-only=false
   ```

2. Update the period to make the changes take effect.

   ```
   radosgw-admin period update --commit
   ```

3. Restart the Ceph Object Gateway.

   Note: Use the output from the **`ceph orch ps`** command, under the `NAME` column, to get the *SERVICE_TYPE.ID* information.

   - Restart the Ceph Object Gateway on an individual node in the storage cluster.

     ```
     systemctl restart ceph-CLUSTER_ID@SERVICE_TYPE.ID.service
     ```

     For example,

     ```
     [root@host01 ~]# systemctl restart ceph-c4b34c6f-8365-11ba-
     dc31-529020a7702d@rgw.realm.zone.host01.gwasto.service
     ```

   - Restart the Ceph Object Gateways on all nodes in the storage cluster.

     ```
     ceph orch restart SERVICE_TYPE
     ```

     For example,

     ```
     [ceph: root@host01 /]# ceph orch restart rgw
     ```

     If the former primary zone recovers, revert the operation.

4. From the recovered zone, pull the realm from the current primary zone.

```
radosgw-admin realm pull --url=URL_TO_PRIMARY_ZONE_GATEWAY
                           --access-key=ACCESS_KEY --secret=SECRET_KEY
```

5. Make the recovered zone the primary and default zone.

```
adosgw-admin zone modify --rgw-zone=ZONE_NAME --master --default
```

6. Update the period to make the changes take effect.

```
radosgw-admin period update --commit
```

7. Restart the Ceph Object Gateway in the recovered zone.

```
ceph orch restart SERVICE_TYPE
```

   For example,

```
[ceph: root@host01 /]# ceph orch restart rgw
```

8. If the secondary and tertiary zones need to be a read-only configuration, update each zone.

```
radosgw-admin zone modify --rgw-zone=ZONE_NAME --read-only
radosgw-admin zone modify --rgw-zone=ZONE_NAME --read-only
```

   a. Update the period to make the changes take effect.

```
radosgw-admin period update --commit
```

   b. Restart the Ceph Object Gateway in the recovered zones.
   For example,

```
[ceph: root@host01 /]# ceph orch restart rgw
```

Configuring multiple realms in the same storage cluster

You can configure multiple realms in the same storage cluster. This is a more advanced use case for multi-site. Configuring multiple realms in the same storage cluster enables you to use a local realm to handle local Ceph Object Gateway client traffic, as well as a replicated realm for data that will be replicated to a secondary site.

> **Note:** Each realm should have its own Ceph Object Gateway.

Prerequisites

- Two running Red Hat Ceph Storage data centers in a storage cluster.
- The access key and secret key for each data center in the storage cluster.
- Root-level access to all the Ceph Object Gateway nodes.
- Each data center has its own local realm. They share a realm that replicates on both sites.

Procedure

1. Create one local realm on the first data center in the storage cluster:

   Syntax

```
radosgw-admin realm create --rgw-realm=REALM_NAME --default
```

   Example

```
[ceph: root@host01 /]# radosgw-admin realm create --rgw-realm=ldc1 --default
```

2. Create one local primary zonegroup on the first data center:

   Syntax

   ```
   radosgw-admin zonegroup create --rgw-zonegroup=ZONE_GROUP_NAME --endpoints=http://
   RGW_NODE_NAME:80 --rgw-realm=REALM_NAME --master --default
   ```

   Example

   ```
   [ceph: root@host01 /]# radosgw-admin zonegroup create --rgw-zonegroup=ldc1zg --
   endpoints=http://rgw1:80 --rgw-realm=ldc1 --master --default
   ```

3. Create one local zone on the first data center:

   Syntax

   ```
   radosgw-admin zone create --rgw-zonegroup=ZONE_GROUP_NAME --rgw-zone=ZONE_NAME --
   master --default --endpoints=HTTP_FQDN[,HTTP_FQDN]
   ```

   Example

   ```
   [ceph: root@host01 /]# radosgw-admin zone create --rgw-zonegroup=ldc1zg --rgw-
   zone=ldc1z --master --default --endpoints=http://rgw.example.com
   ```

4. Commit the period:

   Example

   ```
   [ceph: root@host01 /]# radosgw-admin period update --commit
   ```

5. Optional: If you specified the realm and zone in the service specification during the deployment of the Ceph Object Gateway, update the `spec` section of the specification file:

   Syntax

   ```
   spec:
       rgw_realm: REALM_NAME
       rgw_zone: ZONE_NAME
   ```

6. You can either deploy the Ceph Object Gateway daemons with the appropriate realm and zone or update the configuration database:

   - Deploy the Ceph Object Gateway using placement specification:

     Syntax

     ```
     ceph orch apply rgw SERVICE_NAME --realm=REALM_NAME --zone=ZONE_NAME --
     placement="NUMBER_OF_DAEMONS HOST_NAME_1 HOST_NAME_2"
     ```

     Example

     ```
     [ceph: root@host01 /]# ceph orch apply rgw rgw --realm=ldc1 --zone=ldc1z --
     placement="1 host01"
     ```

   - Update the Ceph configuration database:

     Syntax

     ```
     ceph config set client.rgw.SERVICE_NAME rgw_realm REALM_NAME
     ceph config set client.rgw.SERVICE_NAME rgw_zonegroup ZONE_GROUP_NAME
     ceph config set client.rgw.SERVICE_NAME rgw_zone ZONE_NAME
     ```

     Example

```
[ceph: root@host01 /]# ceph config set client.rgw.rgwsvcid.mons-1.jwgwwp
rgw_realm ldc1
[ceph: root@host01 /]# ceph config set client.rgw.rgwsvcid.mons-1.jwgwwp
rgw_zonegroup ldc1zg
[ceph: root@host01 /]# ceph config set client.rgw.rgwsvcid.mons-1.jwgwwp rgw_zone
ldc1z
```

7. Restart the Ceph Object Gateway.

> **Note:** Use the output from the `ceph orch ps` command, under the `NAME` column, to get the *SERVICE_TYPE.ID* information.

a. To restart the Ceph Object Gateway on an individual node in the storage cluster:

Syntax

```
systemctl restart ceph-CLUSTER_ID@SERVICE_TYPE.ID.service
```

Example

```
[root@host01 ~]# systemctl restart ceph-c4b34c6f-8365-11ba-
dc31-529020a7702d@rgw.realm.zone.host01.gwasto.service
```

b. To restart the Ceph Object Gateways on all nodes in the storage cluster:

Syntax

```
ceph orch restart SERVICE_TYPE
```

Example

```
[ceph: root@host01 /]# ceph orch restart rgw
```

8. Create one local realm on the second data center in the storage cluster:

Syntax

```
radosgw-admin realm create --rgw-realm=REALM_NAME --default
```

Example

```
[ceph: root@host04 /]# radosgw-admin realm create --rgw-realm=ldc2 --default
```

9. Create one local primary zonegroup on the second data center:

Syntax

```
radosgw-admin zonegroup create --rgw-zonegroup=ZONE_GROUP_NAME --endpoints=http://
RGW_NODE_NAME:80 --rgw-realm=REALM_NAME --master --default
```

Example

```
[ceph: root@host04 /]# radosgw-admin zonegroup create --rgw-zonegroup=ldc2zg --
endpoints=http://rgw2:80 --rgw-realm=ldc2 --master --default
```

10. Create one local zone on the second data center:

Syntax

```
radosgw-admin zone create --rgw-zonegroup=ZONE_GROUP_NAME --rgw-zone=ZONE_NAME --
master --default --endpoints=HTTP_FQDN[, HTTP_FQDN]
```

Example

```
[ceph: root@host04 /]# radosgw-admin zone create --rgw-zonegroup=ldc2zg --rgw-
zone=ldc2z --master --default --endpoints=http://rgw.example.com
```

11. Commit the period:

    Example

    ```
    [ceph: root@host04 /]# radosgw-admin period update --commit
    ```

12. Optional: If you specified the realm and zone in the service specification during the deployment of the Ceph Object Gateway, update the `spec` section of the specification file:

    Syntax

    ```
    spec:
        rgw_realm: REALM_NAME
        rgw_zone: ZONE_NAME
    ```

13. You can either deploy the Ceph Object Gateway daemons with the appropriate realm and zone or update the configuration database:

    - Deploy the Ceph Object Gateway using placement specification:

      Syntax

      ```
      ceph orch apply rgw SERVICE_NAME --realm=REALM_NAME --zone=ZONE_NAME --
      placement="NUMBER_OF_DAEMONS HOST_NAME_1 HOST_NAME_2"
      ```

      Example

      ```
      [ceph: root@host01 /]# ceph orch apply rgw rgw --realm=ldc2 --zone=ldc2z --
      placement="1 host01"
      ```

    - Update the Ceph configuration database:

      Syntax

      ```
      ceph config set client.rgw.SERVICE_NAME rgw_realm REALM_NAME
      ceph config set client.rgw.SERVICE_NAME rgw_zonegroup ZONE_GROUP_NAME
      ceph config set client.rgw.SERVICE_NAME rgw_zone ZONE_NAME
      ```

      Example

      ```
      [ceph: root@host01 /]# ceph config set client.rgw.rgwsvcid.mons-1.jwgwwp
      rgw_realm ldc2
      [ceph: root@host01 /]# ceph config set client.rgw.rgwsvcid.mons-1.jwgwwp
      rgw_zonegroup ldc2zg
      [ceph: root@host01 /]# ceph config set client.rgw.rgwsvcid.mons-1.jwgwwp rgw_zone
      ldc2z
      ```

14. Restart the Ceph Object Gateway.

    Note: Use the output from the `ceph orch ps` command, under the `NAME` column, to get the *SERVICE_TYPE.ID* information.

    a. To restart the Ceph Object Gateway on individual node in the storage cluster:

       Syntax

       ```
       systemctl restart ceph-CLUSTER_ID@SERVICE_TYPE.ID.service
       ```

       Example

       ```
       [root@host04 ~]# systemctl restart ceph-c4b34c6f-8365-11ba-
       dc31-529020a7702d@rgw.realm.zone.host01.gwasto.service
       ```

b. To restart the Ceph Object Gateways on all nodes in the storage cluster:

Syntax

```
ceph orch restart SERVICE_TYPE
```

Example

```
[ceph: root@host04 /]# ceph orch restart rgw
```

15. Create a replicated realm on the first data center in the storage cluster:

Syntax

```
radosgw-admin realm create --rgw-realm=REPLICATED_REALM_1 --default
```

Example

```
[ceph: root@host01 /]# radosgw-admin realm create --rgw-realm=rdc1 --default
```

Use the `--default` flag to make the replicated realm default on the primary site.

16. Create a primary zonegroup for the first data center:

Syntax

```
radosgw-admin zonegroup create --rgw-zonegroup=RGW_ZONE_GROUP --endpoints=http://
_RGW_NODE_NAME:80 --rgw-realm=_RGW_REALM_NAME --master --default
```

Example

```
[ceph: root@host01 /]# radosgw-admin zonegroup create --rgw-zonegroup=rdc1zg --
endpoints=http://rgw1:80 --rgw-realm=rdc1 --master --default
```

17. Create a primary zone on the first data center:

Syntax

```
radosgw-admin zone create --rgw-zonegroup=RGW_ZONE_GROUP --rgw-
zone=_MASTER_RGW_NODE_NAME --master --default --endpoints=HTTP_FQDN[,HTTP_FQDN]
```

Example

```
[ceph: root@host01 /]# radosgw-admin zone create --rgw-zonegroup=rdc1zg --rgw-
zone=rdc1z --master --default --endpoints=http://rgw.example.com
```

18. Create a synchronization user and add the system user to the primary zone for multi-site:

Syntax

```
radosgw-admin user create --uid="SYNCHRONIZATION_USER" --display-name="Synchronization
User" --system
radosgw-admin zone modify --rgw-zone=RGW_ZONE --access-key=ACCESS_KEY --
secret=SECRET_KEY
```

Example

```
radosgw-admin user create --uid="synchronization-user" --display-name="Synchronization
User" --system
[ceph: root@host01 /]# radosgw-admin zone modify --rgw-zone=rdc1zg --access-
key=3QV0D6ZMMCJZMSCXJ2QJ --secret=VpvQWcsfI9OPzUCpR4kynDLAbqa1OIKqRB6WEnH8
```

19. Commit the period:

Example

```
[ceph: root@host01 /]# radosgw-admin period update --commit
```

20. Optional: If you specified the realm and zone in the service specification during the deployment of the Ceph Object Gateway, update the `spec` section of the specification file:

    Syntax

    ```
    spec:
        rgw_realm: REALM_NAME
        rgw_zone: ZONE_NAME
    ```

21. You can either deploy the Ceph Object Gateway daemons with the appropriate realm and zone or update the configuration database:

    - Deploy the Ceph Object Gateway using placement specification:

      Syntax

      ```
      ceph orch apply rgw SERVICE_NAME --realm=REALM_NAME --zone=ZONE_NAME --
      placement="NUMBER_OF_DAEMONS HOST_NAME_1 HOST_NAME_2"
      ```

      Example

      ```
      [ceph: root@host01 /]# ceph orch apply rgw rgw --realm=rdc1 --zone=rdc1z --
      placement="1 host01"
      ```

    - Update the Ceph configuration database:

      Syntax

      ```
      ceph config set client.rgw.SERVICE_NAME  rgw_realm REALM_NAME
      ceph config set client.rgw.SERVICE_NAME  rgw_zonegroup ZONE_GROUP_NAME
      ceph config set client.rgw.SERVICE_NAME  rgw_zone ZONE_NAME
      ```

      Example

      ```
      [ceph: root@host01 /]# ceph config set client.rgw.rgwsvcid.mons-1.jwgwwp
      rgw_realm rdc1
      [ceph: root@host01 /]# ceph config set client.rgw.rgwsvcid.mons-1.jwgwwp
      rgw_zonegroup rdc1zg
      [ceph: root@host01 /]# ceph config set client.rgw.rgwsvcid.mons-1.jwgwwp rgw_zone
      rdc1z
      ```

22. Restart the Ceph Object Gateway.

    Note: Use the output from the `ceph orch ps` command, under the `NAME` column, to get the *SERVICE_TYPE.ID* information.

    a. To restart the Ceph Object Gateway on individual node in the storage cluster:

       Syntax

       ```
       systemctl restart ceph-CLUSTER_ID@SERVICE_TYPE.ID.service
       ```

       Example

       ```
       [root@host01 ~]# systemctl restart ceph-c4b34c6f-8365-11ba-
       dc31-529020a7702d@rgw.realm.zone.host01.gwasto.service
       ```

    b. To restart the Ceph Object Gateways on all nodes in the storage cluster:

       Syntax

       ```
       ceph orch restart SERVICE_TYPE
       ```

       Example

```
[ceph: root@host01 /]# ceph orch restart rgw
```

23. Pull the replicated realm on the second data center:

    Syntax

    ```
    radosgw-admin realm pull --url=https://tower-osd1.cephtips.com --access-key=ACCESS_KEY
    --secret-key=SECRET_KEY
    ```

    Example

    ```
    [ceph: root@host01 /]# radosgw-admin realm pull --url=https://tower-osd1.cephtips.com
    --access-key=3QV0D6ZMMCJZMSCXJ2QJ --secret-
    key=VpvQWcsfI9OPzUCpR4kynDLAbqa1OIKqRB6WEnH8
    ```

24. Pull the period from the first data center:

    Syntax

    ```
    radosgw-admin period pull --url=https://tower-osd1.cephtips.com --access-
    key=ACCESS_KEY --secret-key=SECRET_KEY
    ```

    Example

    ```
    [ceph: root@host01 /]# radosgw-admin period pull --url=https://tower-osd1.cephtips.com
    --access-key=3QV0D6ZMMCJZMSCXJ2QJ --secret-
    key=VpvQWcsfI9OPzUCpR4kynDLAbqa1OIKqRB6WEnH8
    ```

25. Create the secondary zone on the second data center:

    Syntax

    ```
    radosgw-admin zone create --rgw-zone=RGW_ZONE --rgw-zonegroup=RGW_ZONE_GROUP --
    endpoints=https://tower-osd4.cephtips.com --access-key=_ACCESS_KEY --secret-
    key=SECRET_KEY
    ```

    Example

    ```
    [ceph: root@host04 /]# radosgw-admin zone create --rgw-zone=rdc2z --rgw-
    zonegroup=rdc1zg --endpoints=https://tower-osd4.cephtips.com --access-
    key=3QV0D6ZMMCJZMSCXJ2QJ --secret-key=VpvQWcsfI9OPzUCpR4kynDLAbqa1OIKqRB6WEnH8
    ```

26. Commit the period:

    Example

    ```
    [ceph: root@host04 /]# radosgw-admin period update --commit
    ```

27. Optional: If you specified the realm and zone in the service specification during the deployment of the Ceph Object Gateway, update the `spec` section of the specification file:

    Syntax

    ```
    spec:
        rgw_realm: REALM_NAME
        rgw_zone: ZONE_NAME
    ```

28. You can either deploy the Ceph Object Gateway daemons with the appropriate realm and zone or update the configuration database:

    – Deploy the Ceph Object Gateway using placement specification:

      Syntax

      ```
      ceph orch apply rgw SERVICE_NAME --realm=REALM_NAME --zone=ZONE_NAME --
      placement="NUMBER_OF_DAEMONS HOST_NAME_1 HOST_NAME_2"
      ```

      Example

```
[ceph: root@host04 /]# ceph orch apply rgw rgw --realm=rdc1 --zone=rdc2z --
placement="1 host04"
```

- Update the Ceph configuration database:

  Syntax

  ```
  ceph config set client.rgw.SERVICE_NAME rgw_realm REALM_NAME
  ceph config set client.rgw.SERVICE_NAME rgw_zonegroup ZONE_GROUP_NAME
  ceph config set client.rgw.SERVICE_NAME rgw_zone ZONE_NAME
  ```

  Example

  ```
  [ceph: root@host04 /]# ceph config set client.rgw.rgwsvcid.mons-1.jwgwwp
  rgw_realm rdc1
  [ceph: root@host04 /]# ceph config set client.rgw.rgwsvcid.mons-1.jwgwwp
  rgw_zonegroup rdc1zg
  [ceph: root@host04 /]# ceph config set client.rgw.rgwsvcid.mons-1.jwgwwp rgw_zone
  rdc2z
  ```

29. Restart the Ceph Object Gateway.

    Note: Use the output from the `ceph orch ps` command, under the `NAME` column, to get the *SERVICE_TYPE.ID* information.

    a. To restart the Ceph Object Gateway on individual node in the storage cluster:

       Syntax

       ```
       systemctl restart ceph-CLUSTER_ID@SERVICE_TYPE.ID.service
       ```

       Example

       ```
       [root@host02 ~]# systemctl restart ceph-c4b34c6f-8365-11ba-
       dc31-529020a7702d@rgw.realm.zone.host01.gwasto.service
       ```

    b. To restart the Ceph Object Gateways on all nodes in the storage cluster:

       Syntax

       ```
       ceph orch restart SERVICE_TYPE
       ```

       Example

       ```
       [ceph: root@host04 /]# ceph orch restart rgw
       ```

30. Log in as `root` on the endpoint for the second data center.
31. Verify the synchronization status on the primary realm:

    Syntax

    ```
    radosgw-admin sync status
    ```

    Example

    ```
    [ceph: root@host04 /]# radosgw-admin sync status
              realm 59762f08-470c-46de-b2b1-d92c50986e67 (ldc2)
          zonegroup 7cf8daf8-d279-4d5c-b73e-c7fd2af65197 (ldc2zg)
               zone 034ae8d3-ae0c-4e35-8760-134782cb4196 (ldc2z)
      metadata sync no sync (zone is master)
    ```

32. Log in as `root` on the endpoint for the first data center.
33. Verify the synchronization status for the replication-synchronization realm:

Syntax

```
radosgw-admin sync status --rgw-realm RGW_REALM_NAME
```

Example

```
[ceph: root@host01 /]# radosgw-admin sync status --rgw-realm rdc1
          realm 73c7b801-3736-4a89-aaf8-e23c96e6e29d (rdc1)
      zonegroup d67cc9c9-690a-4076-89b8-e8127d868398 (rdc1zg)
           zone 67584789-375b-4d61-8f12-d1cf71998b38 (rdc2z)
  metadata sync syncing
                full sync: 0/64 shards
                incremental sync: 64/64 shards
                metadata is caught up with master
      data sync source: 705ff9b0-68d5-4475-9017-452107cec9a0 (rdc1z)
                        syncing
                        full sync: 0/128 shards
                        incremental sync: 128/128 shards
                        data is caught up with source
          realm 73c7b801-3736-4a89-aaf8-e23c96e6e29d (rdc1)
      zonegroup d67cc9c9-690a-4076-89b8-e8127d868398 (rdc1zg)
           zone 67584789-375b-4d61-8f12-d1cf71998b38 (rdc2z)
  metadata sync syncing
                full sync: 0/64 shards
                incremental sync: 64/64 shards
                metadata is caught up with master
      data sync source: 705ff9b0-68d5-4475-9017-452107cec9a0 (rdc1z)
                        syncing
                        full sync: 0/128 shards
                        incremental sync: 128/128 shards
                        data is caught up with source
```

34. To store and access data in the local site, create the user for local realm:

Syntax

```
radosgw-admin user create --uid="LOCAL_USER" --display-name="Local user" --rgw-
realm=_REALM_NAME --rgw-zonegroup=ZONE_GROUP_NAME --rgw-zone=ZONE_NAME
```

Example

```
[ceph: root@host04 /]# radosgw-admin user create --uid="local-user" --display-
name="Local user" --rgw-realm=ldc1 --rgw-zonegroup=ldc1zg --rgw-zone=ldc1z
```

Important: By default, users are created under the default realm. For the users to access data in the local realm, the `radosgw-admin` command requires the `--rgw-realm` argument.

Using multi-site sync policies

As a storage administrator, you can use multi-site sync policies at the bucket level to control data movement between buckets in different zones. These policies are called bucket-granularity sync policies. Previously, all buckets within zones were treated symmetrically. This means that each zone contained a mirror copy of a given bucket, and the copies of buckets were identical in all of the zones. The sync process assumed that the bucket sync source and the bucket sync destination referred to the same bucket.

Important: Bucket sync policies apply to data only, and metadata is synced across all the zones in the multi-site irrespective of the presence of the bucket sync policies. Objects that were created, modified, or deleted when the bucket sync policy was in `allowed` or `forbidden` place, it does not automatically sync when policy takes effect. Run the `bucket sync run` command to sync these objects.

Important: If there are multiple sync policies defined at zonegroup level, only one policy can be in enabled state at any time. We can toggle between policies if needed

The sync policy supersedes the old zone group coarse configuration (`sync_from*`). The sync policy can be configured at the zone group level. If it is configured, it replaces the old-style configuration at the zone group level, but it can also be configured at the bucket level.

When the sync policy of a bucket or zonegroup moves from `disabled` to `enabled` state, the following behavioral changes are observed:

- **Greenfield deployment** - To set up bucket granular sync replication, a new zonegroup, or zone must be configured at a minimum.
- **Brownfield deployment** - Migrate or upgrade Ceph Object Gateway multi-site replication configurations to the newly featured bucket granular sync policy replication.
- **Data flow** - directional, symmetrical - Both unidirectional and bi-directional (symmetrical) replication can be configured.

Note: The bucket sync policies are applicable to the archive zones. The movement from an archive zone is not bidirectional wherein all the objects can be moved from active zone to archive zone. However, you cannot move objects from the archive zone to active zone since archive zone is read-only.

Important: In this release, the following features are not supported:

- Source filters
- Storage class
- Destination owner translation
- User mode

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to a Ceph Monitor node.
- Installation of the Ceph Object Gateway.

Multi-site sync policy group state

In the sync policy, multiple groups that can contain lists of data-flow configurations can be defined and listed. The data-flow defines the flow of data between the different zones. It can define symmetrical data flow, in which multiple zones sync data from each other. It can also define directional data flow, in which the data moves in one way from one zone to another.

A pipe defines the actual buckets that can use these data flows, and the properties that are associated with it, such as the source object prefix.

A sync policy group can be in three states:

- `enabled` - sync is allowed and enabled.
- `allowed` - sync is allowed.
- `forbidden` - sync, as defined by this group, is not allowed. Sync states in this group can override other groups.

When the zones replicate, you can disable replication for specific buckets using the sync policy. The following are the semantics that need to be followed to resolve the policy conflicts:

Table 107: Semantics

Zonegroup	Bucket	Result
enabled	enabled	enabled
enabled	allowed	enabled

Zonegroup	Bucket	Result
enabled	forbidden	disabled
allowed	enabled	enabled
allowed	allowed	disabled
allowed	forbidden	disabled
forbidden	enabled	disabled
forbidden	allowed	disabled
forbidden	forbidden	disabled

For multiple group polices that are set to reflect for any sync pair (*SOURCE_ZONE*,*SOURCE_BUCKET*), (*DESTINATION_ZONE*,*DESTINATION_BUCKET*), the following rules are applied in the following order:

- Even if one sync policy is `forbidden`, the sync is `disabled`.
- At least one policy should be `enabled` for the sync to be `allowed`.

A policy can be defined at the bucket level. A bucket-level sync policy inherits the data flow of the zonegroup policy, and can define a subset of what the zonegroup allows only.

A wildcard zone, and a wildcard bucket parameter in the policy defines all relevant zones, or all relevant buckets. In the context of a bucket policy, it means the current bucket instance. A disaster recovery configuration where entire zones are mirrored does not require configuring anything on the buckets. However, for a fine grained bucket sync it would be better to configure the pipes to be synced by allowing (`status=allowed`) them at the zonegroup level (for example, by using wildcard). However, enable the specific sync at the bucket level (`status=enabled`) only. If needed, the policy at the bucket level can limit the data movement to specific relevant zones.

> **Important:** Any changes to the zonegroup policy need to be applied on the zonegroup primary zone, and require period update and commit. Changes to the bucket policy need to be applied on the zonegroup primary zone. Ceph Object Gateway handles these changes dynamically.

References

- See "S3 bucket replication API" on page 918 for more details.

Retrieving the current policy

You can use the **`get`** command to retrieve the current zonegroup sync policy, or a specific bucket policy.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root or `sudo` access.
- The Ceph Object Gateway is installed.

Procedure

- Retrieve the current zonegroup sync policy or bucket policy. To retrieve a specific bucket policy, use the `--bucket` option:

  Syntax

  ```
  radosgw-admin sync policy get --bucket=BUCKET_NAME
  ```

  Example

  ```
  [ceph: root@host01 /]# radosgw-admin sync policy get --bucket=mybucket
  ```

Creating a sync policy group

You can create a sync policy group for the current zone group, or for a specific bucket.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Root or sudo access.
- The Ceph Object Gateway is installed.
- When creating for an archive zone, be sure that the archive zone is created before the sync policy group.

About this task

When creating a sync policy for bucket granular replication for a sync policy group that has changed from `forbidden` to `enabled`, a manual update might be necessary to complete the sync process.

For example, if any data is written to `bucket1` when its policy is `forbidden`, the data might not sync properly across zones after the policy is changed to `enabled`. To properly sync the changes, run the **`bucket sync run`** command on the sync policy. This step is also necessary if the bucket is resharded when the policy is `forbidden`. In this case the **`bucket sync run`** command must also be used after enabling the policy.

For more information about configuring an archive zone and bucket granular replication, see "Configuring the archive zone" on page 865.

Procedure

1. Create a sync policy group or a bucket policy, using the `.--bucket` option.

   ```
   radosgw-admin sync group create --bucket=BUCKET_NAME --group-id=GROUP_ID --
   status=enabled | allowed | forbidden
   ```

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin sync group create --group-id=mygroup1 --
   status=enabled
   ```

2. Optional: Manually complete the sync process for bucket granular replication.

   > **Note:** This step is mandatory when using as part of an archive zone with bucket granular replication if the policy has data written or the bucket was resharded when the policy was `forbidden`.

   ```
   radosgw-admin bucket sync run
   ```

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin bucket sync run
   ```

Modifying a sync policy group

You can modify an existing sync policy group for the current zone group, or for a specific bucket.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Root or sudo access.
- The Ceph Object Gateway is installed.
- When creating for an archive zone, be sure that the archive zone is created before the sync policy group.

About this task

When creating a sync policy for bucket granular replication for a sync policy group that has changed from `forbidden` to `enabled`, a manual update might be necessary to complete the sync process.

For example, if any data is written to `bucket1` when its policy is `forbidden`, the data might not sync properly across zones after the policy is changed to `enabled`. To properly sync the changes, run the **`bucket sync run`** command on the sync policy. This step is also necessary if the bucket is resharded when the policy is `forbidden`. In this case the **`bucket sync run`** command must also be used after enabling the policy.

For more information about configuring an archive zone and bucket granular replication, see "Configuring the archive zone" on page 865.

Procedure

1. Modify the sync policy group or a bucket policy, using the `--bucket` option.

```
radosgw-admin sync group modify --bucket=BUCKET_NAME --group-id=GROUP_ID --
status=enabled | allowed | forbidden
```

   For example,

```
[ceph: root@host01 /]# radosgw-admin sync group modify --group-id=mygroup1 --
status=forbidden
```

2. Optional: Manually complete the sync process for bucket granular replication.

> **Note:** This step is mandatory when using as part of an archive zone with bucket granular replication if the policy has data written or the bucket was resharded when the policy was `forbidden`.

```
radosgw-admin bucket sync run
```

   For example,

```
[ceph: root@host01 /]# radosgw-admin bucket sync run
```

Getting a sync policy group

You can use the **`group get`** command to show the current sync policy group by group ID, or to show a specific bucket policy.

If the `--bucket` option is not provided, the groups created at zonegroup level is retrieved and not the groups at the bucket-level.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root or `sudo` access.
- The Ceph Object Gateway is installed.

Procedure

- Show the current sync policy group or bucket policy. To show a specific bucket policy, use the `--bucket` option:

  Syntax

```
radosgw-admin sync group get --bucket=BUCKET_NAME --group-id=GROUP_ID
```

  Example

```
[ceph: root@host01 /]# radosgw-admin sync group get --group-id=mygroup
```

Removing a sync policy group

You can use the **`group remove`** command to remove the current sync policy group by group ID, or to remove a specific bucket policy.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root or `sudo` access.
- The Ceph Object Gateway is installed.

Procedure

- Remove the current sync policy group or bucket policy. To remove a specific bucket policy, use the `--bucket` option:

  Syntax

  ```
  radosgw-admin sync group remove --bucket=BUCKET_NAME --group-id=GROUP_ID
  ```

  Example

  ```
  [ceph: root@host01 /]# radosgw-admin sync group remove --group-id=mygroup
  ```

Creating a sync flow

Create a sync flow for a multi-site sync policy.

You can create two different types of flows for a sync policy group or for a specific bucket:

- Directional sync flow
- Symmetrical sync flow

The `group flow create` command creates a sync flow. For an existing sync policy group or bucket, this command overwrites the existing settings for the sync flow and applies the settings that you specify.

Option	Description	Required/Optional
`--bucket`	Name of the bucket to which sync policy needs to be configured. Used only in bucket-level sync policy.	Optional
`--group-id`	ID of the sync group.	Required
`--flow-id`	ID of the flow.	Required
`--flow-type`	Types of flows for a sync policy group or for a specific bucket - directional or symmetrical.	Required
`--source-zone`	To specify the source zone from which sync should happen. Zone that send data to the sync group. Required if flow type of sync group is directional.	Optional

Option	Description	Required/Optional
--dest-zone	To specify the destination zone to which sync should happen. Zone that receives data from the sync group. Required if flow type of sync group is directional.	Optional
--zones	Zones that part of the sync group. Zones mention will be both sender and receiver zone. Specify zones separated by ",". Required if flow type of sync group is symmetrical.	Optional

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root or `sudo` access.
- The Ceph Object Gateway is installed.

Procedure

1. Create or update a directional sync flow. To create or update directional sync flow for a specific bucket, use the `--bucket` option.

   Syntax

   ```
   radosgw-admin sync group flow create --bucket=BUCKET_NAME --group-id=GROUP_ID --flow-id=FLOW_ID --flow-type=directional --source-zone=SOURCE_ZONE --dest-zone=DESTINATION_ZONE
   ```

2. Create or update a symmetrical sync flow. To specify multiple zones for a symmetrical flow type, use a comma-separated list for the `--zones` option.

   Syntax

   ```
   radosgw-admin sync group flow create --bucket=BUCKET_NAME --group-id=GROUP_ID --flow-id=FLOW_ID --flow-type=symmetrical --zones=ZONE_NAME_1,ZONE_NAME_2
   ```

`zones` are comma-separated lists of all zones that need to be added to the flow.

Removing sync flows and zones

Remove sync flows and zones from a sync policy or bucket.

The `group flow remove` command removes sync flows or zones from a sync policy group or bucket.

For sync policy groups or buckets using directional flows, `group flow remove` command removes the flow. For sync policy groups or buckets using symmetrical flows, you can use the `group flow remove` command to remove specified zones from the flow, or to remove the flow.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root or `sudo` access.
- The Ceph Object Gateway is installed.

Procedure

1. Remove a directional sync flow. To remove the directional sync flow for a specific bucket, use the `--bucket` option.

   Syntax

```
radosgw-admin sync group flow remove --bucket=BUCKET_NAME --group-id=GROUP_ID --flow-
id=FLOW_ID --flow-type=directional --source-zone=SOURCE_ZONE --dest-
zone=DESTINATION_ZONE
```

2. Remove specific zones from a symmetrical sync flow. To remove multiple zones from a symmetrical flow, use a comma-separated list for the `--zones` option.

   Syntax

   ```
   radosgw-admin sync group flow remove --bucket=BUCKET_NAME --group-id=GROUP_ID --flow-
   id=FLOW_ID --flow-type=symmetrical --zones=ZONE_NAME_1,ZONE_NAME_2
   ```

3. Remove a symmetrical sync flow. To remove the sync flow at the zonegroup level, use the `--bucket` option.

   Syntax

   ```
   radosgw-admin sync group flow remove --group-id=GROUP_ID --flow-id=FLOW_ID --flow-
   type=symmetrical --zones=ZONE_NAME_1,ZONE_NAME_2
   ```

Creating and modifying a sync group pipe

Define pipes to specify which buckets can use your configured data flows and the properties that are associated with those data flows. Use the **`sync group pipe create`** command to create or modify the sync group pipe.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Root or `sudo` access.
- The Ceph Object Gateway is installed.

About this task

As a storage administrator, you can use the **`sync group pipe create`** command to create pipes. These pipes are custom sync group data flows between specific buckets or groups of buckets, or between specific zones or groups of zones. The **`sync group pipe create`** command uses the options that are listed in "sync group pipe create Command options" on page 887.

Table 109: **`sync group pipe create`** *Command options*

Option	Description	Required/Optional
`--bucket`	Name of the bucket to which sync policy needs to be configured. Used only in bucket-level sync policy.	Optional
`--group-id`	ID of the sync group	Required
`--pipe-id`	ID of the pipe	Required
`--source-zones`	Zones that send data to the sync group. Use single quotation marks (') for value. Use commas to separate multiple zones. Use the wildcard * for all zones that match the data flow rules.	Required

Option	Description	Required/Optional
--source-bucket	Bucket or buckets that send data to the sync group. If bucket name is not mentioned, then * (wildcard) is taken as the default value. At bucket-level, source bucket will be the bucket for which the sync group created and at zonegroup-level, source bucket will be all buckets.	Optional
--source-bucket-id	ID of the source bucket. If this option is not specified, then * is the default value and is applied to all the buckets.	Optional
--dest-zones	Zone or zones that receive the sync data. Use single quotation marks (') for value. Use commas to separate multiple zones. Use the wildcard * for all zones that match the data flow rules.	Required
--dest-bucket	Bucket or buckets that receive the sync data. If bucket name is not mentioned, then * (wildcard) is taken as the default value. At bucket-level, destination bucket will be the bucket for which the sync group is created and at zonegroup-level, destination bucket will be all buckets.	Optional
--dest-bucket-id	ID of the destination bucket. If this option is not specified, then * is the default value and is applied to all the buckets.	Optional
--prefix	Bucket prefix. Use the wildcard * to filter for source objects.	Optional
--prefix-rm	Remove object prefix from filtering. Do not use bucket prefix for filtering.	Optional
--tags-add	Comma-separated list of key=value pairs.	Optional
--tags-rm	Removes one or more key=value pairs of tags.	Optional
--dest-owner	Destination owner of the objects from source.	Optional
--storage-class	Destination storage class for the objects from source.	Optional
--mode	Use `system` for system mode or `user` for user mode.	Optional

Option	Description	Required/Optional
--uid	Used for permissions validation in user mode. Specifies the user ID under which the sync operation will be issued.	Optional

Note: To enable or disable sync at zonegroup level for certain buckets, set zonegroup level sync policy to `enable` or `disable` state respectively, and create a pipe for each bucket with `--source-bucket` and `--dest-bucket` with its bucket name or with `bucket-id` i.e, `--source-bucket-id` and `--dest-bucket-id`.

Procedure

- Create the sync group pipe.
  The `create` command is also used to modify a command by creating the sync group pipe with only the relevant options.

```
radosgw-admin sync group pipe create --bucket=BUCKET_NAME --group-id=GROUP_ID --pipe-id=PIPE_ID --source-zones='ZONE_NAME','ZONE_NAME2'... --source-bucket=SOURCE_BUCKET1 --source-bucket-id=SOURCE_BUCKET_ID --dest-zones='ZONE_NAME1_','ZONE_NAME2'... --dest-bucket=DESTINATION_BUCKET1 --dest-bucket-id=DESTINATION_BUCKET_ID --prefix=SOURCE_PREFIX --prefix-rm --tags-add=KEY1=VALUE1,KEY2=VALUE2, ... --tags-rm=KEY1=VALUE1,KEY2=VALUE2, ... --dest-owner=OWNER_ID --storage-class=STORAGE_CLASS --mode=USER --uid=USER_ID
```

Modifying or deleting a sync group pipe

Use the **sync group pipe remove** command to modify or delete the sync group pipe.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Root or `sudo` access.
- The Ceph Object Gateway is installed.

About this task

As a storage administrator, you can use the **sync group pipe modify** command or **sync group pipe remove** command to modify the sync group pipe by removing certain options. You can also use this command to remove zones, buckets, or the sync group pipe completely.

Procedure

- Modify the sync group pipe options with the `modify` argument.

```
radosgw-admin sync group pipe modify --bucket=BUCKET_NAME --group-id=GROUP_ID --pipe-id=PIPE_ID --source-zones='ZONE_NAME_1','ZONE_NAME2'... --source-bucket=SOURCE_BUCKET --source-bucket-id=SOURCE_BUCKET_ID --dest-zones='ZONE_NAME_1','ZONE_NAME2'... --dest-bucket=DESTINATION_BUCKET1 --dest-bucket-id=DESTINATION_BUCKET-ID
```

Note: Ensure to put the zones in single quotes ('). The source bucket does not need the quotes.

```
[root@host01 ~]# radosgw-admin sync group pipe modify --group-id=zonegroup --pipe-id=pipe --dest-zones='primary','secondary','tertiary' --source-zones='primary','secondary','tertiary' --source-bucket=pri-bkt-1 --dest-bucket=pri-bkt-1
```

- Modify the sync group pipe options with the `remove` argument.

```
radosgw-admin sync group pipe remove --bucket=BUCKET_NAME --group-id=GROUP_ID --pipe-
id=PIPE_ID --source-zones='ZONE_NAME1','ZONE_NAME2'... --source-bucket=SOURCE_BUCKET --
source-bucket-id=SOURCE_BUCKET_ID --dest-zones='ZONE_NAME_1','ZONE_NAME2'... --dest-
bucket=DESTINATION_BUCKET --dest-bucket-id=DESTINATION_BUCKET-ID
```

```
[root@host01 ~]# radosgw-admin sync group pipe remove --group-id=zonegroup --pipe-
id=pipe --dest-zones='primary','secondary','tertiary' --source-
zones='primary','secondary','tertiary' --source-bucket=pri-bkt-1 --dest-bucket=pri-
bkt-1
```

– Delete a sync group pipe.

```
radosgw-admin sync group pipe remove --bucket=BUCKET_NAME --group-id=GROUP_ID --pipe-
id=PIPE_ID
```

```
[root@host01 ~]# radosgw-admin sync group pipe remove -bucket-name=mybuck --group-
id=zonegroup --pipe-id=pipe
```

Obtaining information about sync operations

The **sync info** command enables you to get information about the expected sync sources and targets, as defined by the sync policy.

When you create a sync policy for a bucket, that policy defines how data moves from that bucket toward a different bucket in a different zone. Creating the policy also creates a list of bucket dependencies that are used as hints whenever that bucket syncs with another bucket. Note that a bucket can refer to another bucket without actually syncing to it, since syncing depends on whether the data flow allows the sync to take place.

Both the `--bucket` and `effective-zone-name` parameters are optional. If you invoke the `sync info` command without specifying any options, the Ceph Object Gateway returns all of the sync operations defined by the sync policy in all zones.

Prerequisites

– A running Red Hat Ceph Storage cluster.

– Root or `sudo` access.

– The Ceph Object Gateway is installed.

– A group sync policy is defined.

Procedure

– Get information about sync operations for buckets.

Syntax

```
radosgw-admin sync info --bucket=BUCKET_NAME --effective-zone-name=ZONE_NAME
```

– Get information about sync operations at the zonegroup level.

Syntax

```
radosgw-admin sync info
```

Using destination parameters

Learn to use the destination parameters for multisite syn policy.

There are 3 types of destination parameters:

– Storage class

– Destination owner translation

– User mode

You can set the destination owner to force a specific destination owner for the objects. You can configure the destination storage class. If you select user mode, you can set only the destination bucket owner. You can set the user ID for the user mode to execute the sync operation for permissions validation.

Destination Params: Storage Class

Example for configuring storage class of the destination objects:

```
[ceph: root@host01 /]# radosgw-admin sync group create --bucket=buck10 \
 --group-id=buck10-default --status=enabled
[ceph: root@host01 /]# radosgw-admin sync group pipe create --bucket=buck10 \
 --group-id=buck10-default \
 --pipe-id=pipe-storage-class \
 --source-zones='*' --dest-zones=us-west-2 \
 --storage-class=CHEAP_AND_SLOW
```

Destination Params: Destination Owner Translation

Example for setting the destination objects owner as the destination bucket owner. This requires specifying the uid of the destination bucket:

```
[ceph: root@host01 /]# radosgw-admin sync group create --bucket=buck11 \
 --group-id=buck11-default --status=enabled
[ceph: root@host01 /]# radosgw-admin sync group pipe create --bucket=buck11 \
 --group-id=buck11-default --pipe-id=pipe-dest-owner \
 --source-zones='*' --dest-zones='*' \
 --dest-bucket=buck12 --dest-owner=joe
```

Destination Params: User Mode

User mode makes sure that the user has permissions to both read the objects, and write to the destination bucket. This requires that the uid of the user (which in its context the operation executes) is specified.

```
[ceph: root@host01 /]# radosgw-admin sync group pipe modify --bucket=buck11 \
                          --group-id=buck11-default --pipe-id=pipe-dest-owner \
                          --mode=user --uid=jenny
```

Bucket granular sync policies

You can set bucket granular sync policies for a Red Hat Ceph Storage cluster with Ceph Object Gateway configuration in multi-site.

When the sync policy of a bucket or zone group moves from `disabled` to `enabled` state, the following behavioral changes are observed.

Normal scenario

Zone group level

Data written when the sync policy is `disabled` catches up when it is `enabled` with no additional steps.

Bucket level

Data written when the sync policy is `disabled` does not catch up when the policy is `enabled`. In this case, either one of the following two workarounds can be applied:

- Writing new data to the bucket resynchronizes the old data.
- Running **`bucket sync run`** command syncs all the old data.

> **Note:** When you want to toggle from the sync policy to the legacy policy, you need to first run the **`sync init`** command followed by the **`radosgw-admin bucket sync run`** command to sync all the objects.

Reshard scenario

> **Note:** When the policy is set to `enabled` for the zone group and the policy is set to `enabled` or `allowed` for the bucket, the pipe configuration takes effect from zone group level and not at the bucket level. This is known issue BZ#2240719 (IBMCEPH-7545).

Zone group level

Any reshard that happens when the policy is `disabled` sync gets stuck after the policy is `enabled` again. New objects also do not sync at this point. Run the **`bucket sync run`** command as a workaround.

Bucket level

If any bucket is resharded when the policy is `disabled`, sync gets stuck after the policy is `enabled` again. New objects also do not sync. Run the **`bucket sync run`** command as a workaround.

Setting bi-directional policy for zonegroups

Set the policy for zone groups bidirectionally with the sync policy engine.

Before you begin

Before you begin setting bi-directional policies for zone groups, be sure to have at least two running Red Hat Ceph Storage clusters with Ceph Object Gateway configured on the same version.

About this task

In the following example, create a group policy and define a data flow for the movement of data from one zonegroup to another. Configure a pipe for the zonegroups to define the buckets that can use this data flow. The system in the following examples includes 3 zones: `us-east` (the primary zone), `us-west`, and `us-west-2`. Zonegroup sync policies are created with the new sync policy engine. Any change to the zonegroup sync policy requires a period update and a commit.

Procedure

1. Create a sync group with the status set to `allowed`.

   ```
   radosgw-admin sync group create --group-id=GROUP_ID --status=allowed
   ```

   For example,

   ```
   ceph: root@host01 /]# radosgw-admin sync group create --group-id=group1 --
   status=allowed
   ```

   > **Note:** Until a fully configured zonegroup replication policy is created, set the `--status` to `allowed` to prevent the replication from starting.

2. Create a flow policy for the newly created group with the `--flow-type` set as `symmetrical` to enable bidirectional replication.

   ```
   radosgw-admin sync group flow create --group-id=GROUP_ID --flow-id=FLOW_ID --flow-
   type=symmetrical --zones=ZONE_NAMES
   ```

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin sync group flow create --group-id=group1 --flow-
   id=flow-mirror --flow-type=symmetrical --zones=us-east,us-west
   ```

3. Create a pipe called `pipe`.

   ```
   radosgw-admin sync group pipe create --group-id=GROUP_ID
                            --pipe-id=PIPE_ID --source-zones='*' \
                            --source-bucket='*' --dest-zones='*' \
                            --dest-bucket='*'
   ```

For example,

```
[ceph: root@host01 /]# radosgw-admin sync group pipe create --group-id=group1 \
                         --pipe-id=pipe1 --source-zones='*' \
                         --source-bucket='*' --dest-zones='*' \
                         --dest-bucket='*'
```

Note:
Use the * wildcard for zones to include all zones set in the previous flow policy, and *for buckets to replicate all existing buckets in the zones.

4. Set the group policy `--status` to `enabled`.

```
radosgw-admin sync group modify --group-id=GROUP_ID --status=enabled
```

For example,

```
[ceph: root@host01 /]# radosgw-admin sync group modify --group-id=group1 --
status=enabled
```

5. Update and commit the new period.

```
radosgw-admin period update --commit
```

For example,

```
[ceph: root@host01 /]# radosgw-admin period update --commit
```

Note: Updating and committing the period is mandatory for a zonegroup policy.

6. Optional: Check the sync source, and destination for a specific bucket. All buckets in `us-east` and `us-west` zones replicate bi-directionally.

```
radosgw-admin sync info --bucket=BUCKET_NAME
```

The `id` field in the output reflects the pipe rule that generated that entry. A single rule can generate multiple sync entries as seen in the following example.

7. Optional: Run the **`sync info`** command to retrieve information about the expected bucket sync sources and targets, as defined in the policy.

```
radosgw-admin sync info --bucket=BUCKET_NAME
```

For example,

```
[ceph: root@host01 /]# radosgw-admin sync info --bucket=buck
{
    "sources": [
        {
            "id": "pipe1",
            "source": {
                "zone": "us-east",
                "bucket": "buck:115b12b3-....4409.1"
            },
            "dest": {
                "zone": "us-west",
                "bucket": "buck:115b12b3-....4409.1"
            },
            ...
        }
    ],
    "dests": [
        {
            "id": "pipe1",
            "source": {
                "zone": "us-west",
                "bucket": "buck:115b12b3-....4409.1"
            },
            "dest": {
                "zone": "us-east",
                "bucket": "buck:115b12b3-....4409.1"
            },
           ...
        }
    ],
    ...
}
```

Setting directional policy for zonegroups

Set the policy for zone groups uni directionally with the sync policy engine.

Before you begin

At least two running Red Hat Ceph Storage clusters with Ceph Object Gateway configured on the same version.

About this task

In the following example, create a group policy and configure the data flow for the movement of data from one zonegroup to another. Configure a pipe for the zonegroups to define the buckets that can use this data flow. The system in the following examples includes 3 zones: `us-east` (the primary zone), `us-west` (secondary zone), and `us-west-2` (backup zone). Here, `us-west-2` is a replica of `us-west`, but data is not replicated back from it. Zonegroup sync policies are created with the new sync policy engine. Any change to the zonegroup sync policy requires a period update and a commit.

Procedure

1. On the primary zone, create a sync group with the status set to `allowed`.

   ```
   radosgw-admin sync group create --group-id=GROUP_ID --status=allowed
   ```

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin sync group create --group-id=group1
          --status=allowed
   ```

   Note: Until a fully configured zonegroup replication policy is created, set the `--status` to `allowed` to prevent the replication from starting.

2. Create the flow.

   ```
   radosgw-admin sync group flow create --group-id=GROUP_ID --flow-id=FLOW_ID --flow-
   type=directional --source-zone=SOURCE_ZONE_NAME --dest-zone=DESTINATION_ZONE_NAME
   ```

```
[ceph: root@host01 /]# radosgw-admin sync group flow create --group-id=group1 --flow-
id=us-west-backup --flow-type=directional --source-zone=us-west --dest-zone=us-west-2
```

3. Create the pipe.

```
radosgw-admin sync group pipe create --group-id=GROUP_ID --pipe-id=PIPE_ID  --source-
zones='SOURCE_ZONE_NAME' --dest-zones='DESTINATION_ZONE_NAME'
```

```
[ceph: root@host01 /]# radosgw-admin sync group pipe create --group-id=group1 --pipe-
id=pipe1 --source-zones='us-west' --dest-zones='us-west-2'
```

4. Update and commit the new period.

```
radosgw-admin period update --commit
```

For example,

```
[ceph: root@host01 /]# radosgw-admin period update --commit
```

Note: Updating and committing the period is mandatory for a zonegroup policy.

5. On the secondary zone, check the status of the buckets. The secondary zone has two destinations.

```
radosgw-admin sync info
```

For example,

```
[ceph: root@host01 /]# radosgw-admin sync info --bucket=buck
{
    "sources": [
        {
            "id": "pipe1",
            "source": {
                "zone": "us-east",
                "bucket": "buck:115b12b3-....4409.1"
            },
            "dest": {
                "zone": "us-west",
                "bucket": "buck:115b12b3-....4409.1"
            },
            ...
        }
    ],
    "dests": [
        {
            "id": "pipe1",
            "source": {
                "zone": "us-west",
                "bucket": "buck:115b12b3-....4409.1"
            },
            "dest": {
                "zone": "us-east",
                "bucket": "buck:115b12b3-....4409.1"
            },
            ...
        },
        {
            "id": "pipe1",
            "source": {
                "zone": "us-west",
                "bucket": "buck:115b12b3-....4409.1"
            },
            "dest": {
                "zone": "us-west-2",
                "bucket": "buck:115b12b3-....4409.1"
            },
            ...
        }
    ],
    ...
}
```

6. On the backup of the secondary zone, check the status of the buckets. The backup zone has only one source and no destinations.

```
radosgw-admin sync info --bucket=BUCKET_NAME
```

For example,

```
[ceph: root@host01 /]# radosgw-admin sync info --bucket=buck
{
    "sources": [
        {
            "id": "pipe1",
            "source": {
                "zone": "us-west",
                "bucket": "buck:115b12b3-....4409.1"
            },
            "dest": {
                "zone": "us-west-2",
                "bucket": "buck:115b12b3-....4409.1"
            },
            ...
        }
    ],
    "dests": [],
    ...
}
```

Setting bi-directional policy for buckets

Set the sync policy for buckets in both the directions symmetrically.

Before you begin

At least two running Red Hat Ceph Storage 8.1 clusters with Ceph Object Gateway configured.

About this task

The data flow for the bucket-level policy is inherited from the zonegroup policy.
The data flow and pipes for bucket-level policy are a subset of the flow that is defined in the zonegroup policy. You need not change them.

Note:

- A bucket-level policy can `enable` pipes that are not enabled, except `forbidden`, at the zonegroup policy.
- Bucket-level policies do not require period updates.

Procedure

1. Set the zonegroup policy `--status` to `allowed` to allow per-bucket replication.

```
radosgw-admin sync group modify --group-id=GROUP_ID --status=allowed
```

For example,

```
[ceph: root@host01 /]# radosgw-admin sync group modify --group-id=group1 --
status=allowed
```

2. Update the period.

```
radosgw-admin period update --commit
```

For example,

```
[ceph: root@host01 /]# radosgw-admin period update --commit
```

3. Create a sync group for the bucket that you want to synchronize to and set `--status` to `enabled`.

```
radosgw-admin sync group create --bucket=BUCKET_NAME --group-id=GROUP_ID --
status=enabled
```

For example,

```
[ceph: root@host01 /]# radosgw-admin sync group create --bucket=buck --group-id=buck-
default --status=enabled
```

4. Create a pipe for the group that was created in the previous step. The flow is inherited from the zonegroup level policy where data flow is symmetrical.

```
radosgw-admin sync group pipe create --bucket=BUCKET_NAME --group-id=GROUP_ID --pipe-
id=PIPE_ID --source-zones='*' --dest-zones='*'
```

For example,

```
[ceph: root@host01 /]# radosgw-admin sync group pipe create --bucket=buck --group-
id=buck-default --pipe-id=pipe1 --source-zones='*' --dest-zones='*'
```

> **Note:** Use wildcards * to specify the source and destination zones for the bucket replication.

5. Optional: To retrieve information about the expected bucket sync sources and targets, as defined by the sync policy, run the **`radosgw-admin bucket sync info`** command with the `--bucket` flag.

```
radosgw-admin bucket sync info --bucket BUCKET_NAME
```

For example,

```
[ceph: root@host01 /]# radosgw-admin bucket sync info --bucket buck
realm 33157555-f387-44fc-b4b4-3f9c0b32cd66 (india)
zonegroup 594f1f63-de6f-4e1e-90b6-105114d7ad55 (shared)
zone ffaa5ba4-c1bd-4c17-b176-2fe34004b4c5 (primary)
bucket :buck[ffaa5ba4-c1bd-4c17-b176-2fe34004b4c5.16191.1]

source zone e0e75beb-4e28-45ff-8d48-9710de06dcd0
bucket :buck[ffaa5ba4-c1bd-4c17-b176-2fe34004b4c5.16191.1]
```

6. Optional: To retrieve information about the expected sync sources and targets, as defined by the sync policy, run the **`radosgw-admin sync info`** command with the `--bucket` flag.

```
radosgw-admin sync info --bucket BUCKET_NAME
```

For example,

```
[ceph: root@host01 /]# radosgw-admin sync info --bucket buck
{
    "id": "pipe1",
    "source": {
        "zone": "secondary",
        "bucket": "buck:ffaa5ba4-c1bd-4c17-b176-2fe34004b4c5.16191.1"
    },
    "dest": {
        "zone": "primary",
        "bucket": "buck:ffaa5ba4-c1bd-4c17-b176-2fe34004b4c5.16191.1"
    },
    "params": {
        "source": {
            "filter": {
                "tags": []
            }
        },
        "dest": {},
        "priority": 0,
        "mode": "system",
        "user": ""
    }
},
    {
    "id": "pipe1",
    "source": {
        "zone": "primary",
        "bucket": "buck:ffaa5ba4-c1bd-4c17-b176-2fe34004b4c5.16191.1"
    },
    "dest": {
        "zone": "secondary",
        "bucket": "buck:ffaa5ba4-c1bd-4c17-b176-2fe34004b4c5.16191.1"
    },
    "params": {
        "source": {
            "filter": {
                "tags": []
            }
        },
        "dest": {},
        "priority": 0,
        "mode": "system",
        "user": ""
    }
}
```

Setting directional policy for buckets

Set the policy for buckets uni directionally with the sync policy engine.

Before you begin

At least two running Red Hat Ceph Storage clusters with Ceph Object Gateway configured on the same version.

About this task

In the following example, create a group policy and configure the data flow for the movement of data from one zone to another. Also, configure a pipe for the zonegroups to define the buckets that can use this data flow. The system in the following examples includes 3 zones: `us-east` (the primary zone), `us-west` (secondary zone), and `us-west-2` (backup zone). Here, `us-west-2` is a replica of `us-west`, but data is not replicated back from it.

The difference between setting the directional policy for zonegroups and buckets is that you need to specify the `--bucket` option.

Procedure

1. On the primary zone, create a sync group with the status set to `allowed`.

   ```
   radosgw-admin sync group create --group-id=GROUP_ID --status=allowed
   ```

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin sync group create --group-id=group1 --
   status=allowed
   ```

> **Note:** Until a fully configured zonegroup replication policy is created, set the `--status` to `allowed` to prevent the replication from starting.

2. Create the flow.

```
radosgw-admin sync group flow create --bucket-name=BUCKET_NAME --group-id=GROUP_ID --
flow-id=FLOW_ID --flow-type=directional --source-zone=SOURCE_ZONE_NAME --dest-
zone=DESTINATION_ZONE_NAME
```

```
[ceph: root@host01 /]# radosgw-admin sync group flow create --bucket-name=buck --
group-id=group1 --flow-id=us-west-backup --flow-type=directional --source-zone=us-west
--dest-zone=us-west-2
```

3. Create the pipe.

```
radosgw-admin sync group pipe create --group-id=GROUP_ID --pipe-id=PIPE_ID  --source-
zones='SOURCE_ZONE_NAME' --dest-zones='DESTINATION_ZONE_NAME'
```

```
[ceph: root@host01 /]# radosgw-admin sync group pipe create --group-id=group1 --pipe-
id=pipe1 --source-zones='us-west' --dest-zones='us-west-2'
```

4. Verify source and destination of zonegroup using sync info in both the sites.

```
radosgw-admin sync info --bucket-name=BUCKET_NAME
```

Syncing between buckets

Sync data between source and destination buckets.

Before you begin

At least two running Red Hat Ceph Storage 8.1 clusters with Ceph Object Gateway configured.

About this task

You can sync data between the source and destination buckets across zones, but not within the same zone. Note that internally, data is still pulled from the source at the destination zone.

A wildcard bucket name means that the current bucket is in the context of the bucket sync policy.

There are two types of syncing between buckets:

- “Syncing from a different bucket” on page 899, where you need to specify the source bucket.
- “Syncing to a different bucket” on page 901, where you need to specify the destination bucket.

Syncing from a different bucket

Procedure

1. Create a sync group to pull data from a bucket of another zone.

```
radosgw-admin sync group create  --bucket=BUCKET_NAME --group-id=GROUP_ID --
status=enabled
```

For example,

```
[ceph: root@host01 /]# radosgw-admin sync group create --bucket=buck4 --group-
id=buck4-default --status=enabled
```

2. Pull the data.

```
radosgw-admin sync group pipe create --bucket-name=BUCKET_NAME --group-id=GROUP_ID --
pipe-id=PIPE_ID --source-zones=SOURCE_ZONE_NAME --source-bucket=SOURCE_BUCKET_NAME --
dest-zones=DESTINATION_ZONE_NAME
```

In the following example the source bucket is `buck5`.

```
[ceph: root@host01 /]# radosgw-admin sync group pipe create --bucket=buck4 \
                         --group-id=buck4-default --pipe-id=pipe1 \
                         --source-zones='*' --source-bucket=buck5 \
                         --dest-zones='*'
```

3. Optional: Sync from buckets in specific zones.

```
radosgw-admin sync group pipe modify --bucket=BUCKET_NAME
```

For example,

```
[ceph: root@host01 /]# radosgw-admin sync group pipe modify --bucket=buck4 \
                         --group-id=buck4-default --pipe-id=pipe1 \
                         --source-zones=us-west --source-bucket=buck5 \
                         --dest-zones='*'
```

4. Check sync status.

```
radosgw-admin sync info --bucket-name=BUCKET_NAME
```

For example,

```
[ceph: root@host01 /]# radosgw-admin sync info --bucket=buck5
{
    "sources": [],
    "dests": [],
    "hints": {
        "sources": [],
        "dests": [
            "buck4:115b12b3-....14433.2"
        ]
    },
    "resolved-hints-1": {
        "sources": [],
        "dests": [
            {
                "id": "pipe1",
                "source": {
                    "zone": "us-west",
                    "bucket": "buck5"
                },
                "dest": {
                    "zone": "us-east",
                    "bucket": "buck4:115b12b3-....14433.2"
                },
                ...
            },
            {
                "id": "pipe1",
                "source": {
                    "zone": "us-west",
                    "bucket": "buck5"
                },
                "dest": {
                    "zone": "us-west-2",
                    "bucket": "buck4:115b12b3-....14433.2"
                },
                ...
            }
        ]
    },
    "resolved-hints": {
        "sources": [],
        "dests": []
    }
```

Note: Note that there are `resolved-hints`, which means that the bucket `buck5` found about `buck4` syncing from it indirectly, and not from its own policy. The policy for `buck5` itself is empty.

Syncing to a different bucket

Procedure

1. Create a sync group.

```
radosgw-admin sync group create  --bucket=BUCKET_NAME --group-id=GROUP_ID --
status=enabled
```

   For example,

```
[ceph: root@host01 /]# radosgw-admin sync group create --bucket=buck6 --group-
id=buck6-default --status=enabled
```

2. Push the data.

```
radosgw-admin sync group pipe create --bucket-name=BUCKET_NAME --group-id=GROUP_ID --
pipe-id=PIPE_ID --source-zones=SOURCE_ZONE_NAME --dest-zones=DESTINATION_ZONE_NAME --
dest-bucket=DESTINATION_BUCKET_NAME
```

   In the following example the destination bucket is `buck5`.

```
[ceph: root@host01 /]# radosgw-admin sync group pipe create --bucket=buck6 \
                         --group-id=buck6-default --pipe-id=pipe1 \
                         --source-zones='*' --dest-zones='*' --dest-bucket=buck5
```

3. Optional: Sync to buckets in specific zones.

```
radosgw-admin sync group pipe modify --bucket=BUCKET_NAME
```

   For example,

```
[ceph: root@host01 /]# radosgw-admin sync group pipe modify --bucket=buck6 \
                         --group-id=buck6-default --pipe-id=pipe1 \
                         --source-zones='*' --dest-zones='us-west' --dest-
bucket=buck5
```

4. Check the sync status.

```
radosgw-admin sync info --bucket-name=BUCKET_NAME
```

   For example,

```
[ceph: root@host01 /]#  radosgw-admin sync info --bucket buck5
{
  "sources": [],
  "dests": [
    {
      "id": "pipe1",
      "source": {
        "zone": "us-west",
        "bucket": "buck6:c7887c5b-f6ff-4d5f-9736-aa5cdb4a15e8.20493.4"
      },
      "dest": {
        "zone": "us-east",
        "bucket": "buck5"
      },
      "params": {
        "source": {
          "filter": {
            "tags": []
          }
        },
        "dest": {},
        "priority": 0,
        "mode": "system",
        "user": "s3cmd"
      }
    },
  ],
  "hints": {
    "sources": [],
    "dests": [
      "buck5"
    ]
  },
  "resolved-hints-1": {
    "sources": [],
    "dests": []
  },
  "resolved-hints": {
    "sources": [],
    "dests": []
  }
}
```

Filtering objects

Filter objects within the bucket with prefixes and tags.

Before you begin

At least two running Red Hat Ceph Storage 99 clusters with Ceph Object Gateway configured and buckets that are created.

About this task

You can set object filter at zonegroup level policy as well. If the `--bucket` option is used, then it is set at bucket level for a bucket. In the following example, sync from `buck1` bucket from one zone is synced to `buck1` bucket in another zone with the objects that start with the prefix `foo/`. Similarly, you can filter objects that have tags such as `color=blue`.
Prefixes and tags can be combined, in which objects need to have both in the order to be synced. The priority parameter can also be passed, and it is used to determine when multiple different rules are there that are matches. This configuration determines that rules to be used.

Note:

1. If the tag in sync policy has more than one tags, while syncing objects, it sync objects that match at least one tag, the key value pair. Objects might not match all the tag sets.
2. If the prefix and tag is set, then to sync the object to another zone, the objects must have prefix and any one tag should match. Only then, it syncs with each other.

Procedure

1. On the primary zone, create a sync group. If you want to create at the bucket level, use the `--bucket` option.

```
radosgw-admin sync group create --bucket=BUCKET_NAME --group-id=GROUP_ID --
status=enabled
```

   For example,

```
[ceph: root@host01 /]# radosgw-admin sync group create --bucket=buck1 --group-
id=buck8-default --status=enabled
```

2. Sync between buckets where the object matches the tags. The flow is inherited from the zonegroup level policy where data flow is symmetrical.

```
radosgw-admin sync group pipe create --bucket=BUCKET_NAME --group-id=GROUP_ID --pipe-
id=PIPE_ID --tags-add=KEY1=VALUE1,KEY2=VALUE2 --source-zones='ZONE_NAME1','ZONE_NAME2'
--dest-zones='ZONE_NAME1','ZONE_NAME2'
```

   For example,

```
[ceph: root@host01 /]# radosgw-admin sync group pipe create --bucket=buck1 \
                         --group-id=buck1-default --pipe-id=pipe-prefix \
                         --tags-add=color=blue,color=red --source-zones='*' --dest-
zones='*'
```

3. Sync between buckets where the object matches the prefix. The flow is inherited from the zonegroup level policy where data flow is symmetrical.

```
radosgw-admin sync group pipe create --bucket=BUCKET_NAME --group-id=GROUP_ID --pipe-
id=PIPE_ID --prefix=PREFIX --source-zones='ZONE_NAME1','ZONE_NAME2' --dest-
zones='ZONE_NAME1','ZONE_NAME2'
```

   For example,

```
[ceph: root@host01 /]# radosgw-admin sync group pipe create --bucket=buck1 \
                         --group-id=buck1-default --pipe-id=pipe-tags \
                         --prefix=foo/ --source-zones='*' \
                         --dest-zones='*' --dest-bucket=buck1
```

4. Check the updated sync.

```
radosgw-admin sync info --bucket=BUCKET_NAME
```

   For example,

```
[ceph: root@host01 /]# radosgw-admin sync info --bucket=buck1
```

> **Note:** In the example, only two different destinations and no sources reflect, one each for configuration. When the sync process happens, it selects the relevant rule for each object it syncs.

Disabling policy between buckets

You can disable the policy between buckets along with sync information with the `forbidden` or `allowed` state.

Before you begin

At least two running Red Hat Ceph Storage 99 clusters with Ceph Object Gateway and sync policy configured.

About this task

See “Multi-site sync policy group state” on page 881for the different combinations that can used for zonegroup and bucket level sync policies. In certain cases, to interrupt the replication between two buckets, you can set the group policy for the bucket to be `forbidden`. You can also disable policy at zonegroup level, if the sync that is set does not happen for any of the buckets.

Note: You can also create a sync policy in disabled state with `allowed` or `forbidden` state using the `radosgw-admin sync group create` command.

Procedure

1. Run the **`sync group modify`** command to change the status from `allowed` to `forbidden`.

```
radosgw-admin sync group modify --group-id  GROUP_ID --status forbidden --
bucket BUCKET_NAME
```

For example,

```
ceph: root@host01 /]# radosgw-admin sync group modify --group-id  buck-default --
status forbidden --bucket buck
{
    "groups": [
        {
            "id": "buck-default",
            "data_flow": {},
            "pipes": [
                {
                    "id": "pipe1",
                    "source": {
                        "bucket": "*",
                        "zones": [
                            "*"
                        ]
                    },
                    "dest": {
                        "bucket": "*",
                        "zones": [
                            "*"
                        ]
                    },
                    "params": {
                        "source": {
                            "filter": {
                                "tags": []
                            }
                        },
                        "dest": {},
                        "priority": 0,
                        "mode": "system",
                    }
                }
            ],
            "status": "forbidden"
        }
    ]
}
```

In this example, the replication of the bucket `buck` is interrupted between zones `us-east` and `us-west`.

Note: No update and commit for the period is required as this is a bucket sync policy.

2. Optional: Run **`sync info command`** command to check the status of the sync for bucket.

```
radosgw-admin sync info --bucket BUCKET_NAME
```

For example,

```
[ceph: root@host01 /]# radosgw-admin sync info --bucket buck
{
    "sources": [],
    "dests": [],
    "hints": {
        "sources": [],
        "dests": []
    },
    "resolved-hints-1": {
        "sources": [],
        "dests": []
    },
    "resolved-hints": {
        "sources": [],
        "dests": []
    }
}
```

Note: No source and destination targets are there as the replication is interrupted.

Feature specific configuration

As a storage administrator, you can configure some of the more advanced features of the Ceph Object Gateway. You can configure a multi-site Ceph Object Gateway and integrate it with directory services, such as Microsoft Active Directory and OpenStack Keystone service.

Bucket management

As a storage administrator, when using the Ceph Object Gateway you can manage buckets by moving them between users and renaming them. You can create bucket notifications to trigger on specific events. Also, you can find orphan or leaky objects within the Ceph Object Gateway that can occur over the lifetime of a storage cluster.

Note:

- When millions of objects are uploaded to a Ceph Object Gateway bucket with a high ingest rate, incorrect **num_objects** are reported with the **radosgw-admin bucket stats** command. With the **radosgw-admin bucket list** command you can correct the value of **num_objects** parameter.
- The **radosgw-admin bucket stats** command does not return the `Unknown error 2002` error and explicitly translates to POSIX error 2 such as the `No such file or directory` error.

Related information

Developer

Renaming buckets

Learn how to rename buckets.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway software.
- An existing bucket.

About this task

If you want to allow underscores in bucket names, then set the `rgw_relaxed_s3_bucket_names` option to `true`.

Procedure

1. List the buckets.

   ```
   radosgw-admin bucket list
   ```

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin bucket list
   [
       "34150b2e9174475db8e191c188e920f6/swcontainer",
       "s3bucket1",
       "34150b2e9174475db8e191c188e920f6/swimpfalse",
       "c278edd68cfb4705bb3e07837c7ad1a8/ec2container",
       "c278edd68cfb4705bb3e07837c7ad1a8/demoten1",
       "c278edd68cfb4705bb3e07837c7ad1a8/demo-ct",
       "c278edd68cfb4705bb3e07837c7ad1a8/demopostup",
       "34150b2e9174475db8e191c188e920f6/postimpfalse",
       "c278edd68cfb4705bb3e07837c7ad1a8/demoten2",
       "c278edd68cfb4705bb3e07837c7ad1a8/postupsw"
   ]
   ```

2. Rename the bucket.

   Note: If the bucket is inside a tenant also specify the tenant.

   ```
   radosgw-admin bucket link --bucket=tenant/ORIGINAL_NAME --bucket-new-
   name=NEW_NAME --uid=TENANT$USER_ID
   ```

   ```
   radosgw-admin bucket link --bucket=ORIGINAL_NAME --bucket-new-name=NEW_NAME --
   uid=USER_ID
   ```

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin bucket link --bucket=s3bucket1 --bucket-new-
   name=s3newb --uid=testuser
   ```

3. Verify that the bucket was renamed.

   ```
   radosgw-admin bucket list
   ```

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin bucket list
   [
       "34150b2e9174475db8e191c188e920f6/swcontainer",
       "34150b2e9174475db8e191c188e920f6/swimpfalse",
       "c278edd68cfb4705bb3e07837c7ad1a8/ec2container",
       "s3newb",
       "c278edd68cfb4705bb3e07837c7ad1a8/demoten1",
       "c278edd68cfb4705bb3e07837c7ad1a8/demo-ct",
       "c278edd68cfb4705bb3e07837c7ad1a8/demopostup",
       "34150b2e9174475db8e191c188e920f6/postimpfalse",
       "c278edd68cfb4705bb3e07837c7ad1a8/demoten2",
       "c278edd68cfb4705bb3e07837c7ad1a8/postupsw"
   ]
   ```

Removing buckets

When the bucket does not have objects, you can run the `radosgw-admin bucket rm` command. For objects in the buckets, use the `--purge-objects` option to remove them.

About this task

For multi-site configuration, delete the buckets from the primary site.

For large-scale or ongoing data cleanup, implement an S3 Lifecycle (LC) policy instead of purging with `radosgw-admin`. LC policies apply good data-management practices and naturally throttle deletions because they run during the RGW lifecycle processing window. During each LC window:

- The lifecycle process marks objects for removal.
- Garbage collection later removes the deleted objects in smaller batches.
- The cycle repeats daily until the bucket is empty.

After all objects are removed, you can safely delete the bucket.

Procedure

1. List the buckets.

   ```
   radosgw-admin bucket list
   ```

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin bucket list
   [
       "34150b2e9174475db8e191c188e920f6/swcontainer",
       "s3bucket1",
       "34150b2e9174475db8e191c188e920f6/swimpfalse",
       "c278edd68cfb4705bb3e07837c7ad1a8/ec2container",
       "c278edd68cfb4705bb3e07837c7ad1a8/demoten1",
       "c278edd68cfb4705bb3e07837c7ad1a8/demo-ct",
       "c278edd68cfb4705bb3e07837c7ad1a8/demopostup",
       "34150b2e9174475db8e191c188e920f6/postimpfalse",
       "c278edd68cfb4705bb3e07837c7ad1a8/demoten2",
       "c278edd68cfb4705bb3e07837c7ad1a8/postupsw"
   ]
   ```

2. Remove the bucket.
   Remove the bucket by using the **`radosgw-admin bucket rm`** command.
   When objects exist in the bucket, remove objects using an S3 Lifecycle policy and wait for garbage collection to complete, then delete the bucket (recommended for large buckets). When necessary, use the **`--purge-objects`** option.

   ```
   radosgw-admin bucket rm --bucket=BUCKET_NAME  [--purge-objects]
   ```

   > **Note:** Use **`--bypass-gc`** with caution.
   > You can optionally use the **`--bypass-gc`** flag, but use it **with caution**. This flag causes deletions to skip the garbage collection mechanism, which can negatively impact cluster health and overall performance. The garbage collection process exists to regulate deletion workloads and maintain cluster stability. Skipping it removes that protection and can lead to performance degradation, especially during large or rapid deletion operations.

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin bucket rm --bucket=s3bucket1
   ```

What to do next

Verify that the bucket was removed.

```
[ceph: root@host01 /]# radosgw-admin bucket list
[
    "34150b2e9174475db8e191c188e920f6/swcontainer",
    "34150b2e9174475db8e191c188e920f6/swimpfalse",
    "c278edd68cfb4705bb3e07837c7ad1a8/ec2container",
    "c278edd68cfb4705bb3e07837c7ad1a8/demoten1",
    "c278edd68cfb4705bb3e07837c7ad1a8/demo-ct",
    "c278edd68cfb4705bb3e07837c7ad1a8/demopostup",
    "34150b2e9174475db8e191c188e920f6/postimpfalse",
    "c278edd68cfb4705bb3e07837c7ad1a8/demoten2",
    "c278edd68cfb4705bb3e07837c7ad1a8/postupsw"
]
```

Moving buckets

The `radosgw-admin bucket` utility provides the ability to move buckets between users. To do so, link the bucket to a new user and change the ownership of the bucket to the new user.

You can move buckets:

- Between two non-tenanted users
- Between two tenanted users
- Between a non-tenanted user to a tenanted user

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Ceph Object Gateway is installed.
- An S3 bucket.
- Various tenanted and non-tenanted users.

Moving buckets between non-tenanted users

The **`radosgw-admin bucket chown`** command provides the ability to change the ownership of buckets and all objects they contain from one user to another.

To move buckets between non-tenant users, unlink a bucket from the current user, link it to a new user, and change the ownership of the bucket to the new user.

Procedure

1. Link the bucket to a new user:

   Syntax

   ```
   radosgw-admin bucket link --uid=USER --bucket=BUCKET
   ```

   Example

   ```
   [ceph: root@host01 /]# radosgw-admin bucket link --uid=user2 --bucket=data
   ```

2. Verify that the bucket has been linked to `user2` successfully:

   Example

   ```
   [ceph: root@host01 /]# radosgw-admin bucket list --uid=user2
   [
       "data"
   ]
   ```

3. Change the ownership of the bucket to the new user:

   Syntax

   ```
   radosgw-admin bucket chown --uid=user --bucket=bucket
   ```

Example

```
[ceph: root@host01 /]# radosgw-admin bucket chown --uid=user2 --bucket=data
```

4. Verify that the ownership of the `data` bucket has been successfully changed by checking the `owner` line in the output of the following command:

   Example

   ```
   [ceph: root@host01 /]# radosgw-admin bucket list --bucket=data
   ```

Moving buckets between tenanted users

You can move buckets between one tenanted user and another.

Procedure

1. Link the bucket to a new user:

   Syntax

   ```
   radosgw-admin bucket link --bucket=CURRENT_TENANT/BUCKET --uid=NEW_TENANT$USER
   ```

   Example

   ```
   [ceph: root@host01 /]# radosgw-admin bucket link --bucket=test/data --uid=test2$user2
   ```

2. Verify that the bucket has been linked to `user2` successfully:

   ```
   [ceph: root@host01 /]# radosgw-admin bucket list --uid=test$user2
   [
       "data"
   ]
   ```

3. Change the ownership of the bucket to the new user:

   Syntax

   ```
   radosgw-admin bucket chown --bucket=NEW_TENANT/BUCKET --uid=NEW_TENANT$USER
   ```

   Example

   ```
   [ceph: root@host01 /]# radosgw-admin bucket chown --bucket='test2/data' --
   uid='test$tuser2'
   ```

4. Verify that the ownership of the `data` bucket has been successfully changed by checking the `owner` line in the output of the following command:

   ```
   [ceph: root@host01 /]# radosgw-admin bucket list --bucket=test2/data
   ```

Moving buckets from non-tenanted users to tenanted users

You can move buckets from a non-tenanted user to a tenanted user.

Procedure

1. Optional: If you do not already have multiple tenants, you can create them by enabling `rgw_keystone_implicit_tenants` and accessing the Ceph Object Gateway from an external tenant:

   Enable the `rgw_keystone_implicit_tenants` option:

   Example

   ```
   [ceph: root@host01 /]# ceph config set client.rgw rgw_keystone_implicit_tenants true
   ```

Access the Ceph Object Gateway from an eternal tenant using either the `s3cmd` or `swift` command:

Example

```
[ceph: root@host01 /]# swift list
```

Or use `s3cmd`:

Example

```
[ceph: root@host01 /]# s3cmd ls
```

The first access from an external tenant creates an equivalent Ceph Object Gateway user.

2. Move a bucket to a tenanted user:

   Syntax

   ```
   radosgw-admin bucket link --bucket=/BUCKET --uid=TENANT$USER
   ```

   Example

   ```
   [ceph: root@host01 /]# radosgw-admin bucket link --bucket=/data --uid='test$tenanted-
   user'
   ```

3. Verify that the `data` bucket has been linked to `tenanted-user` successfully:

   Example

   ```
   [ceph: root@host01 /]# radosgw-admin bucket list --uid='test$tenanted-user'
   [
       "data"
   ]
   ```

4. Change the ownership of the bucket to the new user:

   Syntax

   ```
   radosgw-admin bucket chown --bucket=tenant/bucket name --uid=tenant$user
   ```

   Example

   ```
   [ceph: root@host01 /]# radosgw-admin bucket chown --bucket='test/data' --
   uid='test$tenanted-user'
   ```

5. Verify that the ownership of the `data` bucket has been successfully changed by checking the `owner` line in the output of the following command:

   Example

   ```
   [ceph: root@host01 /]# radosgw-admin bucket list --bucket=test/data
   ```

Finding orphan and leaky objects

A healthy storage cluster does not have any orphan or leaky objects, but in some cases orphan or leaky objects can occur. Use this information to learn how to find orphan and leaky objects.

An orphan object exists in a storage cluster and has an object ID associated with the RADOS object. However, there is no reference of the RADOS object with the S3 object in the bucket index reference.

For example, if the Ceph Object Gateway goes down in the middle of an operation, this can cause some objects to become orphans. Also, an undiscovered bug can cause orphan objects to occur.

You can see how the Ceph Object Gateway objects map to the RADOS objects. The `radosgw-admin` command provides a tool to search for and produce a list of these potential orphan or leaky objects. Using the `radoslist` subcommand displays objects stored within buckets, or all buckets in the storage cluster. The `rgw-orphan-list` script displays orphan objects within a pool.

Note: The **`radoslist`** subcommand is replacing the deprecated **`orphans find`** and **`orphans finish`** subcommands.

Important: Do not use this command where `Indexless` buckets are in use as all the objects appear as `orphaned`.

Important: Another alternate way to identity orphaned objects is to run the **`rados -p <pool> ls | grep BUCKET_ID`** command.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- A running Ceph Object Gateway.

Procedure

1. Generate a list of objects that hold data within a bucket.

   Syntax

   ```
   radosgw-admin bucket radoslist --bucket BUCKET_NAME
   ```

   Example

   ```
   [root@host01 ~]# radosgw-admin bucket radoslist --bucket mybucket
   ```

   Note: If the *BUCKET_NAME* is omitted, then all objects in all buckets are displayed.

2. Check the version of `rgw-orphan-list`.

   Example

   ```
   [root@host01 ~]# head /usr/bin/rgw-orphan-list
   ```

   The version should be `2023-01-11` or newer.

3. Create a directory where you need to generate the list of orphans.

   Example

   ```
   [root@host01 ~]# mkdir orphans
   ```

4. Navigate to the directory created earlier.

   Example

   ```
   [root@host01 ~]# cd orphans
   ```

5. From the pool list, select the pool in which you want to find orphans. This script might run for a long time depending on the objects in the cluster.

   Example

   ```
   [root@host01 orphans]# rgw-orphan-list
   ```

   Example

```
Available pools:
    .rgw.root
    default.rgw.control
    default.rgw.meta
    default.rgw.log
    default.rgw.buckets.index
    default.rgw.buckets.data
    rbd
    default.rgw.buckets.non-ec
    ma.rgw.control
    ma.rgw.meta
    ma.rgw.log
    ma.rgw.buckets.index
    ma.rgw.buckets.data
    ma.rgw.buckets.non-ec
Which pool do you want to search for orphans?
```

Enter the pool name to search for orphans.

> **Important:** A data pool must be specified when using the `rgw-orphan-list` command, and not a metadata pool.

6. View the details of the `rgw-orphan-list` tool usage.

   Syntax

   ```
   rgw-orphan-list -h
   rgw-orphan-list POOL_NAME /DIRECTORY
   ```

   Example

   ```
   [root@host01 orphans]# rgw-orphan-list default.rgw.buckets.data /orphans

   2023-09-12 08:41:14 ceph-host01 Computing delta...
   2023-09-12 08:41:14 ceph-host01 Computing results...
   10 potential orphans found out of a possible 2412 (0%).         <<<<<<< orphans
  detected
   The results can be found in './orphan-list-20230912124113.out'.
       Intermediate files are './rados-20230912124113.intermediate' and './radosgw-
  admin-20230912124113.intermediate'.
   ***
   *** WARNING: This is EXPERIMENTAL code and the results should be used
   ***          only with CAUTION!
   ***
   Done at 2023-09-12 08:41:14.
   ```

7. Run the `ls -l` command to verify the files ending with error should be zero length indicating the script ran without any issues.

   Example

   ```
   [root@host01 orphans]# ls -l

   -rw-r--r--. 1 root root    770 Sep 12 03:59 orphan-list-20230912075939.out
   -rw-r--r--. 1 root root      0 Sep 12 03:59 rados-20230912075939.error
   -rw-r--r--. 1 root root 248508 Sep 12 03:59 rados-20230912075939.intermediate
   -rw-r--r--. 1 root root      0 Sep 12 03:59 rados-20230912075939.issues
   -rw-r--r--. 1 root root      0 Sep 12 03:59 radosgw-admin-20230912075939.error
   -rw-r--r--. 1 root root 247738 Sep 12 03:59 radosgw-admin-20230912075939.intermediate
   ```

8. Review the orphan objects listed.

   Example

   ```
   [root@host01 orphans]# cat ./orphan-list-20230912124113.out

   a9c042bc-be24-412c-9052-dda6b2f01f55.16749.1_key1.cherylf.433-bucky-4865-0.0
   a9c042bc-be24-412c-9052-dda6b2f01f55.16749.1_key1.cherylf.433-bucky-4865-0.1
   a9c042bc-be24-412c-9052-dda6b2f01f55.16749.1_key1.cherylf.433-bucky-4865-0.2
   a9c042bc-be24-412c-9052-dda6b2f01f55.16749.1_key1.cherylf.433-bucky-4865-0.3
   a9c042bc-be24-412c-9052-dda6b2f01f55.16749.1_key1.cherylf.433-bucky-4865-0.4
   ```

```
a9c042bc-be24-412c-9052-dda6b2f01f55.16749.1_key1.cherylf.433-bucky-4865-0.5
a9c042bc-be24-412c-9052-dda6b2f01f55.16749.1_key1.cherylf.433-bucky-4865-0.6
a9c042bc-be24-412c-9052-dda6b2f01f55.16749.1_key1.cherylf.433-bucky-4865-0.7
a9c042bc-be24-412c-9052-dda6b2f01f55.16749.1_key1.cherylf.433-bucky-4865-0.8
a9c042bc-be24-412c-9052-dda6b2f01f55.16749.1_key1.cherylf.433-bucky-4865-0.9
```

9. Remove orphan objects:

   Syntax

   ```
   rados -p POOL_NAME rm OBJECT_NAME
   ```

   Example

   ```
   [root@host01 orphans]# rados -p default.rgw.buckets.data rm myobject
   ```

   Warning: Verify you are removing the correct objects. Running the `rados rm` command removes data from the storage cluster.

Managing bucket index entries

Each bucket index entry related to a piece of a multipart upload object is matched against its corresponding `.meta` index entry. There should be one `.meta` entry for all the pieces of a given multipart upload. If it fails to find a corresponding `.meta` entry for a piece, it lists out the "orphaned" piece entries in a section of the output.

The stats for the bucket are stored in the bucket index headers. This phase loads those headers and also iterates through all the plain object entries in the bucket index and recalculates the stats. It then displays the actual and calculated stats in sections labeled "existing_header" and "calculated_header" respectively, so they can be compared.

If you use the `--fix` option with the `bucket check` sub-command, it removes the "orphaned" entries from the bucket index and also overwrites the existing stats in the header with those that it calculated. It causes all entries, including the multiple entries used in versioning, to be listed in the output.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- A running Ceph Object Gateway.
- A newly created bucket.

Procedure

1. Check the bucket index of a specific bucket:

   Syntax

   ```
   radosgw-admin bucket check --bucket=BUCKET_NAME
   ```

   Example

   ```
   [root@rgw ~]# radosgw-admin bucket check --bucket=mybucket
   ```

2. Fix the inconsistencies in the bucket index, including removal of orphaned objects:

   Syntax

   ```
   radosgw-admin bucket check --fix --bucket=BUCKET_NAME
   ```

   Example

```
[root@rgw ~]# radosgw-admin bucket check --fix --bucket=mybucket
```

Bucket notifications

Bucket notifications provide a way to send information out of the Ceph Object Gateway when certain events happen in the bucket.

Bucket notifications can be sent to HTTP, AMQP0.9.1, and Kafka endpoints. A notification entry must be created to send bucket notifications for events on a specific bucket and to a specific topic. A bucket notification can be created on a subset of event types or by default for all event types. The bucket notification can filter out events based on key prefix or suffix, regular expression matching the keys, and on the metadata attributes attached to the object, or the object tags. Bucket notifications have a REST API to provide configuration and control interfaces for the bucket notification mechanism.

> **Note:** The bucket notifications API is enabled by default. If **`rgw_enable_apis`** configuration parameter is explicitly set, ensure that `s3`, and `notifications` are included. To verify, run the following command, replacing *NAME* with the Ceph Object Gateway instance name.
>
> ```
> ceph --admin-daemon /var/run/ceph/ceph-client.rgw.NAME.asok config get rgw_enable_apis
> ```

By default, bucket notifications and topics that saved in fresh installation deployments (Greenfield) have their information synced between zones, due to the new notification_v2 feature. This provides enhanced troubleshooting of persistent topic issues. When upgrading from release versions earlier than Red Hat Ceph Storage 8.0, this enhancement needs to be added by enabling the `notification_v2` feature. Trigger a migration of any existing bucket notifications and topic configurations to the new format by running the **`zonegroup modify`** command.

```
radosgw-admin zonegroup modify --enable-feature=notification_v2
radosgw-admin period update
radosgw-admin period commit
```

Topic management using CLI

You can manage list, get, and remove topics for the Ceph Object Gateway buckets.

Table 110: Topic management for Ceph Object Gateway buckets

Action	Description	Command example
List topics	Run the command to list the configuration of all topics.	`[ceph: host01 /]# radosgw-admin topic list`
Get topics	Run the command to get the configuration of a specific topic.	`[ceph: host01 /]# radosgw-admin topic get --topic=topic1`
Remove topics	Run the command to remove the configuration of a specific topic. **Note:** The topic is removed even if the Ceph Object Gateway bucket is configured to that topic.	`[ceph: host01 /]# radosgw-admin topic rm --topic=topic1`

Creating bucket notifications

Create bucket notifications to send Ceph Object Gateway S3 events to an HTTP, RabbitMQ, or Kafka destination.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster with Ceph Object Gateway.
- A running HTTP server, a RabbitMQ server, or a Kafka server.
- Root-level access.
- User access key and secret key.
- Endpoint parameters.

About this task

Bucket notifications are S3 operations that you create at the bucket level. To send bucket notifications, publish the notification configuration with the destination.

Red Hat Ceph Storage supports the following S3 events:

`ObjectCreated`

Operations such as `put`, `post`, `multipartUpload`, and `copy`.

`ObjectRemoved`

Operations such as `object_delete` and `s3_multi_object_delete`.

`ObjectLifecycle:Expiration`

Object expiration by a lifecycle rule.

Create bucket notifications in one of two ways:

- “Creating bucket notifications by using the Boto3 script” on page 915
- “Creating bucket notifications by using the AWS CLI” on page 917

Creating bucket notifications by using the Boto3 script

Create an Amazon Simple Notification Service (SNS) topic and an S3 bucket notification by running Python scripts that use the Boto3 library.

Procedure

1. Install the `python3-boto3` package.

   ```
   dnf install python3-boto3
   ```

   For example,

   ```
   [user@client ~]$ dnf install python3-boto3
   ```

2. Create an S3 bucket.
3. Create a Python script named `topic.py` to create an SNS topic for the `http`, `amqp`, or `kafka` protocol. For example,

   ```
   import boto3
   from botocore.client import Config
   import sys

   # endpoint and keys from vstart
   endpoint = 'http://127.0.0.1:8000'
   access_key='0555b35654ad1656d804'
   secret_key='h7GhxuBLTrlhVUyxSPUKUV8r/2EI4ngqJxD7iBdBYLhwluN30JaT3Q=='

   client = boto3.client('sns',
           endpoint_url=endpoint,
   ```

```
        aws_access_key_id=access_key,
        aws_secret_access_key=secret_key,
        config=Config(signature_version='s3'))

attributes = {"push-endpoint": "amqp://localhost:5672", "amqp-exchange": "ex1", "amqp-
ack-level": "broker"}

client.create_topic(topic_name="mytopic", Attributes=attributes)
```

4. Run the Python script to create the topic.

```
python3 topic.py
```

5. Create a Python script named `notification.py` to create S3 bucket notifications.
   Create bucket notifications for the following events: `s3:objectCreate`, `s3:objectRemove`, and `s3:ObjectLifecycle:Expiration`.
   For example,

```
import boto3
import sys

# bucket name as first argument
bucketname = sys.argv[1]
# topic ARN as second argument
topic_arn = sys.argv[2]
# notification id as third argument
notification_id = sys.argv[3]

# endpoint and keys from vstart
endpoint = 'http://127.0.0.1:8000'
access_key='0555b35654ad1656d804'
secret_key='h7GhxuBLTrlhVUyxSPUKUV8r/2EI4ngqJxD7iBdBYLhwluN30JaT3Q=='

client = boto3.client('s3',
        endpoint_url=endpoint,
        aws_access_key_id=access_key,
        aws_secret_access_key=secret_key)

# regex filter on the object name and metadata based filtering are extension to AWS S3
API
# bucket and topic should be created beforehand

topic_conf_list = [{'Id': notification_id,
                    'TopicArn': topic_arn,
                    'Events': ['s3:ObjectCreated:*', 's3:ObjectRemoved:*'],
                    }]
    client.put_bucket_notification_configuration(
        Bucket=bucketname,
          NotificationConfiguration={
            'TopicConfigurations': [
              {
                'Id': notification_id,
                'TopicArn': topic_arn,
                'Events': ['s3:ObjectCreated:*', 's3:ObjectRemoved:*',
's3:ObjectLifecycle:Expiration:*']
              }]})
```

6. Run the Python script to create the bucket notification.

```
python3 notification.py
```

7. Create S3 objects in the bucket.
8. Fetch the notification configuration.

```
import boto3
endpoint = 'http://127.0.0.1:8000'
access_key='0555b35654ad1656d804'
secret_key='h7GhxuBLTrlhVUyxSPUKUV8r/2EI4ngqJxD7iBdBYLhwluN30JaT3Q=='

client = boto3.client('s3',
        endpoint_url=endpoint,
        aws_access_key_id=access_key,
        aws_secret_access_key=secret_key)
```

```
# getting a specific notification configuration is an extension to AWS S3 API

print(client.get_bucket_notification_configuration(Bucket=bucketname))
```

9. Optional: Delete the objects.
   Verify the object deletion events at the `http`, `rabbitmq`, or `kafka` receiver.

Creating bucket notifications by using the AWS CLI

Create a topic and an S3 bucket notification by running AWS CLI commands.

Procedure

1. Create a topic.

   ```
   aws --endpoint=AWS_END_POINT sns create-topic --name NAME --attributes=ATTRIBUTES_FILE
   ```

   For example,

   ```
   [user@client ~]$ aws --endpoint=http://localhost sns create-topic --

   _[Continuing response due to length limit...]_

   name test-kafka --attributes=file://topic.json

    sample topic.json:
    {"push-endpoint": "kafka://localhost","verify-ssl": "False", "kafka-ack-level":
   "broker", "persistent":"true"}
    ref: https://docs.aws.amazon.com/cli/latest/reference/sns/create-topic.html
   ```

2. Create the bucket notification.

   ```
   aws s3api put-bucket-notification-configuration --bucket BUCKET_NAME --notification-
   configuration NOTIFICATION_FILE
   ```

   For example,

   ```
   [user@client ~]$ aws s3api put-bucket-notification-configuration --bucket my-bucket --
   notification-configuration file://notification.json

    sample notification.json
    {
        "TopicConfigurations": [
            {
                "Id": "test_notification",
                "TopicArn": "arn:aws:sns:us-west-2:123456789012:test-kafka",
                "Events": [
                    "s3:ObjectCreated:*",
                    "s3:ObjectRestore:Post"
                ]
            }
        ]
    }
   ```

3. Fetch the notification configuration.

   ```
   aws s3api --endpoint=AWS_ENDPOINT get-bucket-notification-configuration --bucket
   BUCKET_NAME
   ```

   For example,

```
[user@client ~]$ aws s3api --endpoint=http://localhost get-bucket-notification-
configuration --bucket my-bucket
 {
     "TopicConfigurations": [
         {
             "Id": "test_notification",
             "TopicArn": "arn:aws:sns:default::test-kafka",
             "Events": [
                 "s3:ObjectCreated:*"
             ]
         }
     ]
 }
```

S3 bucket replication API

Create replication between different buckets in a storage cluster with Ceph Object Gateway configuration.

The S3 bucket replication API is implemented, and allows users to create replication rules between different buckets. While the AWS replication feature allows bucket replication within the same zone, Ceph Object Gateway does not allow it at the moment. However, the Ceph Object gateway API also added a `Zone` array that allows users to select to what zones the specific bucket is synced.

Creating S3 bucket replication

Create a replication configuration for a bucket or replace an existing one.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage 9 cluster with Multi-site Ceph object Gateway configured. For more information on creating multi-site sync policies, see Creating a sync policy group.
- Zonegroup-level policy is created. For more information on creating zone group policies, see Bucket granular sync policies.

About this task

You can create a replication configuration for a bucket or replace an existing one. Specify the replication configuration in the request body. Provide the name of the destination bucket or buckets where you want to replicate objects.

A replication configuration must include at least one rule. Each rule identifies a subset of objects to replicate by filtering the objects in the source bucket.

Procedure

1. Create a replication configuration file that contains the details of replication.

```
{
    "Role": "arn:aws:iam::account-id:role/role-name",
    "Rules": [
        {
            "ID": "String",
            "Status": "Enabled",
            "Priority": 1,
            "DeleteMarkerReplication": { "Status": "Enabled"|"Disabled" },
            "Destination": {
                "Bucket": "BUCKET_NAME"
            }
        }
    ]
}
```

   For example,

```
[root@host01 ~]# cat replication.json
{
    "Role": "arn:aws:iam::account-id:role/role-name",
    "Rules": [
        {
            "ID": "pipe-bkt",
            "Status": "Enabled",
            "Priority": 1,
            "DeleteMarkerReplication": { "Status": "Disabled" },
            "Destination": {
                "Bucket": "testbucket"
            }
        }
    ]
}
```

2. Create the S3 API put bucket replication.

```
aws --endpoint-url=RADOSGW_ENDPOINT_URL s3api put-bucket-replication --
bucket BUCKET_NAME --replication-configuration file://
REPLICATION_CONFIIRATION_FILE.json
```

For example,

```
[root@host01 ~]# aws --endpoint-url=http://host01:80 s3api put-bucket-replication --
bucket testbucket --replication-configuration file://replication.json
```

What to do next

– Verify the sync policy, by using the **`sync policy get`** command.
Syntax

```
radosgw-admin sync policy get --bucket BUCKET_NAME
```

Note: When applying replication policy, the rules are converted to sync-policy rules, that are known as *pipes*, and are categorized as`enabled`and`disabled`.

Enabled

These pipes are enabled and are active and the group status is set to `rgw_sync_policy_group:STATUS`.
Example,

`s3-bucket-replication:enabled`.

Disabled

The pipes under this set are not active and the group status is set to `rgw_sync_policy_group:STATUS`.
Example,

`s3-bucket-replication:disabled`.

Since there can be multiple configurable rules as part of replication policy, it has two separate groups (one with`enabled`and another with`allowed`state) for accurate mapping of each group.

Example

```
[host01 /]# radosgw-admin sync policy get --bucket testbucket

{
    "groups": [
        {
            "id": "s3-bucket-replication:disabled",
            "data_flow": {},
            "pipes": [],
            "status": "allowed"
        },
        {
            "id": "s3-bucket-replication:enabled",
            "data_flow": {},
            "pipes": [
                {
                    "id": "pipe-bkt",
                    "source": {
                        "bucket": "*",
                        "zones": [
                            "*"
                        ]
                    },
                    "dest": {
                        "bucket": "testbucket",
                        "zones": [
                            "*"
                        ]
                    },
                    "params": {
                        "source": {},
                        "dest": {},
                        "priority": 1,
                        "mode": "user",
                        "user": "s3cmd"
                    }
                }
            ],
            "status": "enabled"
        }
    ]
}
```

Related information

Using multi-site sync policies

Getting S3 bucket replication

Get a replication configuration for a bucket.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage 99 cluster with Multi-site Ceph object Gateway configured. For more information on creating multi-site sync policies, see Creating a sync policy group.
- Zonegroup-level policy is created. For more information on creating zone group policies, see Bucket granular sync policies.
- An S3 bucket replication created.

About this task

You can retrieve the replication configuration of the bucket.

Procedure

- Get the S3 API put bucket replication.

```
aws s3api get-bucket-replication --bucket BUCKET_NAME --endpoint-
url=RADOSGW_ENDPOINT_URL
```

  For example,

```
[root@host01 ~]# aws s3api get-bucket-replication --bucket testbucket --endpoint-
url=http://host01:80
{

"ReplicationConfiguration": {
    "Role": "",
    "Rules": [
        {
            "ID": "pipe-bkt",
            "Status": "Enabled",
            "Priority": 1,
            "Destination": {
                "Bucket": "testbucket"
            }
        }
    ]
  }

}
```

Deleting S3 bucket replication

Delete a replication configuration from a bucket.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage 99 cluster with Multi-site Ceph object Gateway configured. For more information on creating multi-site sync policies, see Creating a sync policy group.
- Zonegroup-level policy is created. For more information on creating zone group policies, see Bucket granular sync policies.

About this task

You can delete a replication configuration from a bucket. The bucket owner can grant permission to others to remove the replication configuration.

> **Note:** It can take a while to delete the replication configuration.

Procedure

- Delete the S3 API put bucket replication.

  ```
  aws s3api delete-bucket-replication --bucket BUCKET_NAME --endpoint-
  url=RADOSGW_ENDPOINT_URL
  ```

  For example,

  ```
  [root@host01 ~]# aws s3api delete-bucket-replication --bucket testbucket --endpoint-
  url=http://host01:80
  ```

What to do next

- Verify that the existing replication rules are deleted.

  ```
  radosgw-admin sync policy get --bucket=BUCKET_NAME
  ```

  For example,

  ```
  [ceph: root@host01 /]# radosgw-admin sync policy get --bucket=testbucket
  ```

Disabling S3 bucket replication for a user

As an administrator, you can set a user policy for other users to restrict them from performing any S3 replication API operations on buckets that reside under that particular user/users.

Before you begin

- A running Red Hat Ceph Storage 99 cluster with Multi-site Ceph object Gateway configured. For more information on creating multi-site sync policies, see Creating a sync policy group.
- Zonegroup-level policy is created. For more information on creating zone group policies, see Bucket granular sync policies.

Procedure

1. Create a user policy configuration file to deny access to the S3 bucket replication API.
   For example,

```
[root@host01 ~]# cat user_policy.json
{
    "Version":"2012-10-17",
    "Statement":
    {
        "Effect":"Deny",
        "Action":
        [
            "s3:PutReplicationConfiguration",
            "s3:GetReplicationConfiguration",
            "s3:DeleteReplicationConfiguration"
        ],
        "Resource": "arn:aws:s3:::*",
    }
}
```

2. As an admin user, set a user policy to user to disable user access to the S3 API.

```
aws --endpoint-url=ENDPOINT_URL iam put-user-policy --user-name USER_NAME --policy-
name USER_POLICY_NAME --policy-document POLICY_DOCUMENT_PATH
```

   For example,

```
[root@host01 ~]# aws --endpoint-url=http://host01:80 iam put-user-policy --user-name
newuser1 --policy-name userpolicy --policy-document file://user_policy.json
```

What to do next

- As an admin, verify the user policy set.

```
aws --endpoint-url=ENDPOINT_URL iam get-user-policy --user-name USER_NAME --policy-
name USER_POLICY_NAME --region us
```

  For example,

```
[root@host01 ~]# aws --endpoint-url=http://host01:80 iam get-user-policy --user-name
newuser1 --policy-name userpolicy --region us
```

- As a user, perform the below S3 bucket replication API operations to verify whether the action is denied as expected.
  - Creating S3 bucket replication
  - “Getting S3 bucket replication” on page 920
  - “Deleting S3 bucket replication” on page 921

Configuring Bucket lifecycle

As a storage administrator, you can use a bucket lifecycle configuration to manage your objects so they are stored effectively throughout their lifetime. For example, you can transition objects to less expensive storage classes, archive, or even delete them based on your use case.

RADOS Gateway supports S3 API object expiration by using rules defined for a set of bucket objects. Each rule has a prefix, which selects the objects, and a number of days after which objects become unavailable.

Creating a lifecycle management policy

You can manage a bucket lifecycle policy configuration using standard S3 operations rather than using the **`radosgw-admin`** command. RADOS Gateway supports only a subset of the Amazon S3 API policy language applied to buckets. The lifecycle configuration contains one or more rules defined for a set of bucket objects.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway.
- Root-level access to a Ceph Object Gateway node.
- An S3 bucket created.
- An S3 user created with user access.
- Access to a Ceph Object Gateway client with the `AWS CLI` package installed.

Procedure

1. Create a JSON file for lifecycle configuration:

   Example

   ```
   [user@client ~]$ vi lifecycle.json
   ```

2. Add the specific lifecycle configuration rules in the file:

   Example

   ```
   {
           "Rules": [
           {
                       "Filter": {
                               "Prefix": "images/"
                       },
                       "Status": "Enabled",
                       "Expiration": {
                               "Days": 1
                       },
                       "ID": "ImageExpiration"
               }
       ]
   }
   ```

   The lifecycle configuration example expires objects in the images directory after 1 day.

   The following example aborts incomplete multipart uploads that remain incomplete for more than 7 days:

   ```
   {
     "Rules": [
       {
         "ID": "AbortIncompleteMultipartUploads",
         "Status": "Enabled",
         "Filter": {
           "Prefix": ""
         },
         "AbortIncompleteMultipartUpload": {
           "DaysAfterInitiation": 7
         }
       }
     ]
   }
   ```

   Use lifecycle policies to automatically remove incomplete multipart uploads that are no longer required.

3. Set the lifecycle configuration on the bucket:

   Syntax

```
aws --endpoint-url=_RADOSGW_ENDPOINT_URL_:PORT s3api put-bucket-lifecycle-
configuration --bucket _BUCKET_NAME_ --lifecycle-configuration file://
_PATH_TO_LIFECYCLE_CONFIGURATION_FILE_/_LIFECYCLE_CONFIGURATION_FILE_.json
```

Example

```
[user@client ~]$ aws --endpoint-url=http://host01:80 s3api put-bucket-lifecycle-
configuration --bucket testbucket --lifecycle-configuration file://lifecycle.json
```

In this example, the `lifecycle.json` file exists in the current directory.

– Retrieve the lifecycle configuration for the bucket:

Syntax

```
aws --endpoint-url=_RADOSGW_ENDPOINT_URL_:PORT s3api get-bucket-lifecycle-configuration
--bucket _BUCKET_NAME_
```

Example

```
[user@client ~]$ aws --endpoint-url=http://host01:80 s3api get-bucket-lifecycle-
configuration --bucket testbucket
{
        "Rules": [
        {
                    "Expiration": {
                            "Days": 1
                    },
                    "ID": "ImageExpiration",
                    "Filter": {
                            "Prefix": "images/"
                    },
                    "Status": "Enabled"
            }
    ]
}
```

– Optional: From the Ceph Object Gateway node, log into the `cephadm` shell and retrieve the bucket lifecycle configuration:

Syntax

```
radosgw-admin lc get --bucket=BUCKET_NAME
```

Example

```
[ceph: root@host01 /]# radosgw-admin lc get --bucket=testbucket
{
        "prefix_map": {
                "images/": {
                        "status": true,
                        "dm_expiration": false,
                        "expiration": 1,
                        "noncur_expiration": 0,
                        "mp_expiration": 0,
                        "transitions": {},
                        "noncur_transitions": {}
                }
        },
        "rule_map": [
        {
                "id": "ImageExpiration",
                "rule": {
                        "id": "ImageExpiration",
                        "prefix": "",
                        "status": "Enabled",
                        "expiration": {
                                "days": "1",
                                "date": ""
                        },
                        "mp_expiration": {
                                "days": "",
                                "date": ""
                        },
```

```
                        "filter": {
                                "prefix": "images/",
                                "obj_tags": {
                                        "tagset": {}
                                }
                        },
                        "transitions": {},
                        "noncur_transitions": {},
                        "dm_expiration": false
                }
        }
  ]
}
```

Reference

For more information, see the following:

- S3 bucket lifecycle
- *Setting lifecycle configuration on a bucket*

Deleting a lifecycle management policy

You can delete the lifecycle management policy for a specified bucket by using the **s3api delete-bucket-lifecycle** command.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway.
- Root-level access to a Ceph Object Gateway node.
- An S3 bucket created.
- An S3 user created with user access.
- Access to a Ceph Object Gateway client with the `AWS CLI` package installed.

Procedure

- Delete a lifecycle configuration:

  Syntax

  ```
  aws --endpoint-url=_RADOSGW_ENDPOINT_URL_:PORT s3api delete-bucket-lifecycle --bucket
  _BUCKET_NAME_
  ```

  Example

  ```
  [user@client ~]$ aws --endpoint-url=http://host01:80 s3api delete-bucket-lifecycle --
  bucket testbucket
  ```

Verification

- Retrieve lifecycle configuration for the bucket:

  Syntax

  ```
  aws --endpoint-url=_RADOSGW_ENDPOINT_URL_:PORT s3api get-bucket-lifecycle-configuration
  --bucket _BUCKET_NAME_
  ```

  Example

  ```
  [user@client ~]# aws --endpoint-url=http://host01:80  s3api get-bucket-lifecycle-
  configuration --bucket testbucket
  ```

– Optional: From the Ceph Object Gateway node, retrieve the bucket lifecycle configuration:

  Syntax

  ```
  radosgw-admin lc get --bucket=BUCKET_NAME
  ```

  Example

  ```
  [ceph: root@host01 /]# radosgw-admin lc get --bucket=testbucket
  ```

  The command does not return any information if a bucket lifecycle policy is not present.

Updating a lifecycle management policy

You can update a lifecycle management policy by using the **`s3cmd put-bucket-lifecycle-configuration`** command.

> **Note:** The `put-bucket-lifecycle-configuration` overwrites an existing bucket lifecycle configuration. If you want to retain any of the current lifecycle policy settings, you must include them in the lifecycle configuration file.

Prerequisites

– A running Red Hat Ceph Storage cluster.
– Installation of the Ceph Object Gateway.
– Root-level access to a Ceph Object Gateway node.
– An S3 bucket created.
– An S3 user created with user access.
– Access to a Ceph Object Gateway client with the `AWS CLI` package installed.

Procedure

1. Create a JSON file for the lifecycle configuration:

   Example

   ```
   [user@client ~]$ vi lifecycle.json
   ```

2. Add the specific lifecycle configuration rules to the file:

   Example

   ```
   {
           "Rules": [
           {
                       "Filter": {
                               "Prefix": "images/"
                       },
                       "Status": "Enabled",
                       "Expiration": {
                               "Days": 1
                       },
                       "ID": "ImageExpiration"
               },
                   {
                       "Filter": {
                               "Prefix": "docs/"
                       },
                       "Status": "Enabled",
                       "Expiration": {
                               "Days": 30
                       },
                       "ID": "DocsExpiration"
   ```

```
                }
        ]
}
```

3. Update the lifecycle configuration on the bucket:

   Syntax

   ```
   aws --endpoint-url=RADOSGW_ENDPOINT_URL:PORT s3api put-bucket-lifecycle-configuration
   --bucket BUCKET_NAME --lifecycle-configuration file://
   PATH_TO_LIFECYCLE_CONFIGURATION_FILE/LIFECYCLE_CONFIGURATION_FILE.json
   ```

   Example

   ```
   [user@client ~]$ aws --endpoint-url=http://host01:80 s3api put-bucket-lifecycle-
   configuration --bucket testbucket --lifecycle-configuration file://lifecycle.json
   ```

– Retrieve the lifecycle configuration for the bucket:

  Syntax

  ```
  aws --endpointurl=RADOSGW_ENDPOINT_URL:PORT s3api get-bucket-lifecycle-configuration --
  bucket BUCKET_NAME
  ```

  Example

  ```
  [user@client ~]$ aws -endpoint-url=http://host01:80 s3api get-bucket-lifecycle-
  configuration --bucket testbucket

  {
      "Rules": [
          {
              "Expiration": {
                  "Days": 30
              },
              "ID": "DocsExpiration",
              "Filter": {
                  "Prefix": "docs/"
              },
              "Status": "Enabled"
          },
          {
              "Expiration": {
                  "Days": 1
              },
              "ID": "ImageExpiration",
              "Filter": {
                  "Prefix": "images/"
              },
              "Status": "Enabled"
          }
      ]
  }
  ```

– Optional: From the Ceph Object Gateway node, log into the `cephadm` shell and retrieve the bucket lifecycle configuration:

  Syntax

  ```
  radosgw-admin lc get --bucket=BUCKET_NAME
  ```

  Example

  ```
  [ceph: root@host01 /]# radosgw-admin lc get --bucket=testbucket
  {
          "prefix_map": {
          "docs/": {
                           "status": true,
                           "dm_expiration": false,
                           "expiration": 1,
                           "noncur_expiration": 0,
                           "mp_expiration": 0,
  ```

```
                        "transitions": {},
                        "noncur_transitions": {}
                },
                "images/": {
                        "status": true,
                        "dm_expiration": false,
                        "expiration": 1,
                        "noncur_expiration": 0,
                        "mp_expiration": 0,
                        "transitions": {},
                        "noncur_transitions": {}
                }
        },
        "rule_map": [
        {
        "id": "DocsExpiration",
        "rule": {
                "id": "DocsExpiration",
                "prefix": "",
                "status": "Enabled",
                "expiration": {
                        "days": "30",
                        "date": ""
                },
            "noncur_expiration": {
                "days": "",
                "date": ""
            },
                "mp_expiration": {
                        "days": "",
                        "date": ""
                },
                "filter": {
                        "prefix": "docs/",
                        "obj_tags": {
                                "tagset": {}
                        }
                },
                "transitions": {},
                "noncur_transitions": {},
                "dm_expiration": false
        }
    },
    {
                "id": "ImageExpiration",
                "rule": {
                        "id": "ImageExpiration",
                        "prefix": "",
                        "status": "Enabled",
                        "expiration": {
                                "days": "1",
                                "date": ""
                        },
                        "mp_expiration": {
                                "days": "",
                                "date": ""
                        },
                        "filter": {
                                "prefix": "images/",
                                "obj_tags": {
                                        "tagset": {}
                                }
                        },
                        "transitions": {},
                        "noncur_transitions": {},
                        "dm_expiration": false
                }
        }
  ]
}
```


Monitoring bucket lifecycles

You can monitor lifecycle processing and manually process the lifecycle of buckets with the **`radosgw-admin lc list`** and **`radosgw-admin lc process`** commands.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to a Ceph Object Gateway node.
- Creation of an S3 bucket with a lifecycle configuration policy applied.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. List bucket lifecycle progress:

   Example

   ```
   [ceph: root@host01 /]# radosgw-admin lc list

   [
      {
            “bucket”: “:testbucket:8b63d584-9ea1-4cf3-8443-a6a15beca943.54187.1”,
            “started”: “Thu, 01 Jan 1970 00:00:00 GMT”,
            “status” : “UNINITIAL”
      },
      {
            “bucket”: “:testbucket1:8b635499-9e41-4cf3-8443-a6a15345943.54187.2”,
            “started”: “Thu, 01 Jan 1970 00:00:00 GMT”,
            “status” : “UNINITIAL”
      }
   ]
   ```

   The bucket lifecycle processing status can be one of the following:

   - UNINITIAL - The process has not run yet.
   - PROCESSING - The process is currently running.
   - COMPLETE - The process has completed.

3. Optional: You can manually process bucket lifecycle policies:

   a. Process the lifecycle policy for a single bucket:

      Syntax

      ```
      radosgw-admin lc process --bucket=BUCKET_NAME
      ```

      Example

      ```
      [ceph: root@host01 /]# radosgw-admin lc process --bucket=testbucket1
      ```

   b. Process all bucket lifecycle policies immediately:

      Example

      ```
      [ceph: root@host01 /]# radosgw-admin lc process
      ```

Verification

- List the bucket lifecycle policies:

  ```
  [ceph: root@host01 /]# radosgw-admin lc list
  [
      {
            “bucket”: “:testbucket:8b63d584-9ea1-4cf3-8443-a6a15beca943.54187.1”,
            “started”: “Thu, 17 Mar 2022 21:48:50 GMT”,
            “status” : “COMPLETE”
      }
  ```

```
    {
          “bucket”: “:testbucket1:8b635499-9e41-4cf3-8443-a6a15345943.54187.2”,
          “started”: “Thu, 17 Mar 2022 20:38:50 GMT”,
          “status” : “COMPLETE”
    }
]
```

Reference

For more information, see the following:

- S3 bucket lifecycle

Configuring lifecycle expiration window

You can set the time that the lifecycle management process runs each day by setting the **rgw_lifecycle_work_time** parameter. By default, lifecycle processing occurs once per day, at midnight.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- A Ceph Object Gateway node.
- Installation of the Ceph Object Gateway.
- Root-level access to a Ceph Object Gateway node.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Set the lifecycle expiration time:

   Syntax

   ```
   ceph config set client.rgw rgw_lifecycle_work_time %D:%D-%D:%D
   ```

   Replace *%d:%d-%d:%d* with `start_hour:start_minute-end_hour:end_minute`.

   Example

   ```
   [ceph: root@host01 /]# ceph config set client.rgw rgw_lifecycle_work_time 06:00-08:00
   ```

Verification

- Retrieve the lifecycle expiration work time:

  Example

  ```
  [ceph: root@host01 /]# ceph config get client.rgw rgw_lifecycle_work_time
  06:00-08:00
  ```

Reference

For more information, see the following:

- S3 bucket lifecycle

S3 bucket lifecycle transition within a storage cluster

You can use a bucket lifecycle configuration to manage objects so objects are stored effectively throughout the object's lifetime. The object lifecycle transition rule allows you to manage, and effectively store the objects throughout the object's lifetime. You can transition objects to less expensive storage classes, archive, or even delete them.

You can create storage classes for:

- Fast media, such as SSD or NVMe for I/O sensitive workloads.
- Slow magnetic media, such as SAS or SATA for archiving.

You can create a schedule for data movement between a hot storage class and a cold storage class. You can schedule this movement after a specified time so that the object expires and is deleted permanently for example you can transition objects to a storage class 30 days after you have created or even archived the objects to a storage class one year after creating them. You can do this through a transition rule. This rule applies to an object transitioning from one storage class to another. The lifecycle configuration contains one or more rules using the `<Rule>` element.

Transitioning an object from one storage class to another

The object lifecycle transition rule allows you to transition an object from one storage class to another class.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway software.
- Root-level access to the Ceph Object Gateway node.
- An S3 user created with user access.

About this task

You can migrate data between replicated pools, erasure-coded pools, replicated to erasure-coded pools, or erasure-coded to replicated pools with the Ceph Object Gateway lifecycle transition policy.

> **Note:** Starting with 7.1, the policy-based lifecycle transition and storage class model is extended to support two more S3-compatible cloud platforms for cloud archival - **IBM COS** and **Red Hat Ceph Storage**.

Procedure

1. Create a new data pool.

   ```
   ceph osd pool create POOL_NAME
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph osd pool create test.hot.data
   ```

2. Add a new storage class.

   ```
   radosgw-admin zonegroup placement add  --rgw-zonegroup default --placement-
   id PLACEMENT_TARGET --storage-class STORAGE_CLASS
   ```

   For example,

```
[ceph: root@host01 /]# radosgw-admin zonegroup placement add  --rgw-zonegroup default
--placement-id default-placement --storage-class hot.test
{
        "key": "default-placement",
        "val": {
            "name": "default-placement",
            "tags": [],
            "storage_classes": [
                "STANDARD",
                "hot.test"
            ]
        }
    }
```

3. Provide the zone placement information for the new storage class.

```
radosgw-admin zone placement add --rgw-zone default --placement-id PLACEMENT_TARGET --
storage-class STORAGE_CLASS --data-pool DATA_POOL
```

For example,

```
[ceph: root@host01 /]# radosgw-admin zone placement add --rgw-zone default --
placement-id default-placement --storage-class hot.test --data-pool test.hot.data
{
           "key": "default-placement",
           "val": {
               "index_pool": "test_zone.rgw.buckets.index",
               "storage_classes": {
                   "STANDARD": {
                       "data_pool": "test.hot.data"
                   },
                   "hot.test": {
                       "data_pool": "test.hot.data",
                   }
               },
               "data_extra_pool": "",
               "index_type": 0
 }
```

> **Note:** Consider setting the `compression_type` when creating cold or archival data storage pools with write once.

4. Enable the `rgw` application on the data pool.

```
ceph osd pool application enable POOL_NAME rgw
```

For example,

```
[ceph: root@host01 /]# ceph osd pool application enable test.hot.data rgw
enabled application 'rgw' on pool 'test.hot.data'
```

5. Restart all the `rgw` daemons.
6. Create a bucket.

```
aws s3api create-bucket --bucket BUCKET_NAME --create-bucket-
configuration BUCKET_CONFIGURATION --endpoint-url ENDPOINT_URL
```

For example,

```
[ceph: root@host01 /]# aws s3api create-bucket --bucket testbucket10 --create-bucket-
configuration LocationConstraint=default:default-placement --endpoint-url http://
10.0.0.80:8080
```

7. Add the object.

```
aws --endpoint-url ENDPOINT_URL s3api put-object --bucket BUCKET_NAME
```

For example,

```
[ceph: root@host01 /]# aws --endpoint=http://10.0.0.80:8080 s3api put-object --bucket
testbucket10
```

8. Create a second data pool.

```
ceph osd pool create POOL_NAME
```

For example,

```
[ceph: root@host01 /]# ceph osd pool create test.cold.data
```

9. Add a new storage class.

```
radosgw-admin zonegroup placement add  --rgw-zonegroup default --placement-
id PLACEMENT_TARGET --storage-class STORAGE_CLASS
```

For example,

```
[ceph: root@host01 /]# radosgw-admin zonegroup placement add  --rgw-zonegroup default
--placement-id default-placement --storage-class cold.test
{
        "key": "default-placement",
        "val": {
            "name": "default-placement",
            "tags": [],
            "storage_classes": [
                "STANDARD",
                "cold.test"
            ]
        }
}
```

10. Provide the zone placement information for the new storage class.

```
radosgw-admin zone placement add --rgw-zone default --placement-id PLACEMENT_TARGET --
storage-class STORAGE_CLASS --data-pool DATA_POOL
```

For example,

```
[ceph: root@host01 /]# radosgw-admin zone placement add --rgw-zone default --
placement-id default-placement --storage-class cold.test --data-pool test.cold.data
```

11. Enable `rgw` application on the data pool.

```
ceph osd pool application enable POOL_NAME rgw
```

For example,

```
[ceph: root@host01 /]# ceph osd pool application enable test.cold.data rgw
enabled application 'rgw' on pool 'test.cold.data'
```

12. Restart all the `rgw` daemons.
13. View the zone group configuration.

```
radosgw-admin zonegroup get --rgw-zonegroup=ZONE_GROUP_NAME > JSON_FILE_NAME
```

For example,

```
radosgw-admin zonegroup get --rgw-zonegroup= us > rgw.json
{
    "id": "3019de59-ddde-4c5c-b532-7cdd29de09a1",
    "name": "default",
    "api_name": "default",
    "is_master": "true",
    "endpoints": [],
    "hostnames": [],
    "hostnames_s3website": [],
    "master_zone": "adacbe1b-02b4-41b8-b11d-0d505b442ed4",
    "zones": [
        {
            "id": "adacbe1b-02b4-41b8-b11d-0d505b442ed4",
            "name": "default",
            "endpoints": [],
            "log_meta": "false",
            "log_data": "false",
            "bucket_index_max_shards": 11,
            "read_only": "false",
            "tier_type": "",
            "sync_from_all": "true",
            "sync_from": [],
            "redirect_zone": ""
        }
    ],
    "placement_targets": [
        {
            "name": "default-placement",
            "tags": [],
            "storage_classes": [
                "hot.test",
                "cold.test",
                "STANDARD"
            ]
        }
    ],
    "default_placement": "default-placement",
    "realm_id": "",
    "sync_policy": {
        "groups": []
    }
}
```

14. View the zone configuration.

```
radosgw-admin zone get [--rgw-zone=ZONE_NAME]
```

For example,

```
radosgw-admin zone get
{
    "id": "adacbe1b-02b4-41b8-b11d-0d505b442ed4",
    "name": "default",
    "domain_root": "default.rgw.meta:root",
    "control_pool": "default.rgw.control",
    "gc_pool": "default.rgw.log:gc",
    "lc_pool": "default.rgw.log:lc",
    "log_pool": "default.rgw.log",
    "intent_log_pool": "default.rgw.log:intent",
    "usage_log_pool": "default.rgw.log:usage",
    "roles_pool": "default.rgw.meta:roles",
    "reshard_pool": "default.rgw.log:reshard",
    "user_keys_pool": "default.rgw.meta:users.keys",
    "user_email_pool": "default.rgw.meta:users.email",
    "user_swift_pool": "default.rgw.meta:users.swift",
    "user_uid_pool": "default.rgw.meta:users.uid",
    "otp_pool": "default.rgw.otp",
    "system_key": {
        "access_key": "",
        "secret_key": ""
    },
    "placement_pools": [
        {
            "key": "default-placement",
            "val": {
                "index_pool": "default.rgw.buckets.index",
                "storage_classes": {
                    "cold.test": {
                        "data_pool": "test.cold.data"
                    },
                    "hot.test": {
                        "data_pool": "test.hot.data"
                    },
                    "STANDARD": {
                        "data_pool": "default.rgw.buckets.data"
                    }
                },
                "data_extra_pool": "default.rgw.buckets.non-ec",
                "index_type": 0
            }
        }
    ],
    "realm_id": "",
    "notif_pool": "default.rgw.log:notif"
}
```

15. Create a bucket.

```
aws s3api create-bucket --bucket BUCKET_NAME --create-bucket-
configuration BUCKET_CONFIGURATION --endpoint-url ENDPOINT_URL
```

For example,

```
[ceph: root@host01 /]# aws s3api create-bucket --bucket testbucket10 --create-bucket-
configuration LocationConstraint=default:default-placement --endpoint-url http://
10.0.0.80:8080
```

16. List the objects prior to transition.

```
radosgw-admin bucket list --bucket BUCKET_NAME
```

```
[ceph: root@host01 /]# radosgw-admin bucket list --bucket testbucket10
{
            "ETag": "\"211599863395c832a3dfcba92c6a3b90\"",
            "Size": 540,
            "StorageClass": "STANDARD",
            "Key": "obj1",
            "VersionId": "W95teRsXPSJI4YWJwwSG30KxSCzSgk-",
            "IsLatest": true,
            "LastModified": "2023-11-23T10:38:07.214Z",
            "Owner": {
                "DisplayName": "test-user",
                "ID": "test-user"
            }
}
```

17. Create a JSON file for lifecycle configuration.

```
vi LICEYCLE_CONFGURATION_FILE.json
```

For example,

```
[ceph: root@host01 /]# vi lifecycle.json
```

18. Add the specific lifecycle configuration rule in the file.

```
{
    "Rules": [
        {
            "Filter": {
                "Prefix": ""
            },
            "Status": "Enabled",
            "Transitions": [
                {
                    "Days": 5,
                    "StorageClass": "hot.test"
                },
 {
                    "Days": 20,
                    "StorageClass": "cold.test"
                }
            ],
            "Expiration": {
                "Days": 365
            },
            "ID": "double transition and expiration"
        }
    ]
}
```

The lifecycle configuration example shows an object that transitions from the default `STANDARD` storage class to the `hot.test` storage class after 5 days, again transitions after 20 days to the `cold.test` storage class, and finally expires after 365 days in the `cold.test` storage class.

19. Set the lifecycle configuration on the bucket.

```
aws s3api put-bucket-lifecycle-configuration --bucket BUCKET_NAME --lifecycle-
configuration LICEYCLE_CONFGURATION_FILE.json
```

For example,

```
[ceph: root@host01 /]# aws s3api put-bucket-lifecycle-configuration --bucket
testbucket10 --lifecycle-configuration file://lifecycle.json
```

20. Retrieve the lifecycle configuration on the bucket.

```
aws s3api get-bucket-lifecycle-configuration --bucket BUCKET_NAME
```

For example,

```
[ceph: root@host01 /]# aws s3api get-bucket-lifecycle-configuration --bucket
testbucket10
{
   "Rules": [
       {
           "Expiration": {
               "Days": 365
           },
           "ID": "double transition and expiration",
           "Prefix": "",
           "Status": "Enabled",
           "Transitions": [
               {
                   "Days": 20,
                   "StorageClass": "cold.test"
               },
               {
                   "Days": 5,
                   "StorageClass": "hot.test"
               }
           ]
       }
   ]
}
```

21. Verify that the object is transitioned to the given storage class.

```
radosgw-admin bucket list --bucket BUCKET_NAME
```

For example,

```
[ceph: root@host01 /]# radosgw-admin bucket list --bucket testbucket10
{
            "ETag": "\"211599863395c832a3dfcba92c6a3b90\"",
            "Size": 540,
            "StorageClass": "cold.test",
            "Key": "obj1",
            "VersionId": "W95teRsXPSJI4YWJwwSG30KxSCzSgk-",
            "IsLatest": true,
            "LastModified": "2023-11-23T10:38:07.214Z",
            "Owner": {
                "DisplayName": "test-user",
                "ID": "test-user"
            }
}
```

Enabling object lock for S3

Using the S3 object lock mechanism, you can use object lock concepts like retention period, legal hold, and bucket configuration to implement Write-Once-Read_Many (WORM) functionality as part of the custom workflow overriding data deletion permissions.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root-level access to a Ceph Object Gateway node.
- S3 user with version-bucket creation access.

About this task

Important:

- The object version(s), not the object name, is the defining and required value for object lock to perform correctly to support the **GOVERNANCE** or **COMPLIANCE** mode. You need to know the version of the object when it is written so that you can retrieve it at a later time.
- Consulting company Cohasset concludes that Red Hat Ceph Storage, when properly configured and upon satisfying the additional considerations, meets the electronic record-keeping system requirements of **SEC Rules 17a-4(f)(2)**, **18a-6(e)(2)**, and **FINRA Rule 4511(c)**, as well as, supports

the regulated entity in its compliance with the audit system requirements in **SEC Rules 17a-4(f)(3)(iii)** and **18a-6(e)(3)(iii)**. In addition, the assessed capabilities meet the principles-based electronic records requirements of **CFTC Rule 1.31(c)-(d)**. See the Cohasset compliance certification for more information.
Red Hat Ceph Storage Compliance Assessment

Note: When S3 object lock is enabled on a bucket, delete behavior depends on whether a version ID is specified.

- Deleting an object without specifying a version ID creates a delete marker. Existing object versions remain intact and continue to be protected by object lock.
- Deleting an object by specifying a version ID for an object version that is under retention, in either GOVERNANCE or COMPLIANCE mode, fails with a forbidden by object lock error.

This behavior is expected and ensures that protected object versions cannot be removed before the retention period expires.

Procedure

1. Create a bucket with object lock enabled.

```
aws --endpoint=http://RGW_PORT:8080 s3api create-bucket --bucket BUCKET_NAME --object-lock-enabled-for-bucket
```

For example,

```
[root@rgw-2 ~]# aws --endpoint=http://rgw.ceph.com:8080 s3api create-bucket --bucket worm-bucket --object-lock-enabled-for-bucket
```

2. Set a retention period for the bucket.

```
aws --endpoint=http://RGW_PORT:8080 s3api put-object-lock-configuration --bucket BUCKET_NAME --object-lock-configuration { "ObjectLockEnabled": "Enabled", "Rule": { "DefaultRetention": { "Mode": "RETENTION_MODE", "Days": NUMBER_OF_DAYS }}}
```

You can choose either the GOVERNANCE or COMPLIANCE mode for the *RETENTION_MODE* in S3 object lock, to apply different levels of protection to any object version that is protected by object lock.

GOVERNANCE

In GOVERNANCE mode, users cannot overwrite or delete an object version or alter its lock settings unless they have special permissions.

COMPLIANCE

In COMPLIANCE mode, a protected object version cannot be overwritten or deleted by any user, including the root user in your AWS account. When an object is locked in COMPLIANCE mode, its *RETENTION_MODE* cannot be changed, and its retention period cannot be shortened. COMPLIANCE mode helps ensure that an object version cannot be overwritten or deleted for the duration of the period.

```
[root@rgw-2 ~]# aws --endpoint=http://rgw.ceph.com:8080 s3api put-object-lock-configuration --bucket worm-bucket --object-lock-configuration '{ "ObjectLockEnabled": "Enabled", "Rule": { "DefaultRetention": { "Mode": "COMPLIANCE", "Days": 10 }}}'
```

3. Put the object into the bucket with a retention time set.

```
aws --endpoint=http://RGW_PORT:8080 s3api put-object --bucket BUCKET_NAME --object-lock-mode RETENTION_MODE --object-lock-retain-until-date "DATE" --key compliance-upload --body TEST_FILE
```

For example,

```
[root@rgw-2 ~]# aws --endpoint=http://rgw.ceph.com:8080 s3api put-object --bucket
worm-bucket --object-lock-mode COMPLIANCE --object-lock-retain-until-date "2022-05-31"
--key compliance-upload --body test.dd
{
    "ETag": ""d560ea5652951637ba9c594d8e6ea8c1"",
    "VersionId": "Nhhk5kRS6Yp6dZXVWpZZdRcpSpBKToD"
}
```

4. Upload a new object using the same key.

```
aws --endpoint=http://RGW_PORT:8080 s3api put-object --bucket BUCKET_NAME --object-
lock-mode RETENTION_MODE --object-lock-retain-until-date "DATE" --key compliance-
upload --body PATH
```

For example,

```
[root@rgw-2 ~]# aws --endpoint=http://rgw.ceph.com:8080 s3api put-object --bucket
worm-bucket --object-lock-mode COMPLIANCE --object-lock-retain-until-date "2022-05-31"
--key compliance-upload --body /etc/fstab
{
    "ETag": ""d560ea5652951637ba9c594d8e6ea8c1"",
    "VersionId": "Nhhk5kRS6Yp6dZXVWpZZdRcpSpBKToD"
}
```

What to do next

You can use the following command-line options with object lock.

- Set an object lock legal hold on an object version.
  For example,

```
[root@rgw-2 ~]# aws --endpoint=http://rgw.ceph.com:8080 s3api put-object-legal-hold --
bucket worm-bucket --key compliance-upload --legal-hold Status=ON
```

> **Note:** Using the object lock legal hold operation, you can place a legal hold on an object version, thereby preventing an object version from being overwritten or deleted. A legal hold doesn't have an associated retention period and hence, remains in effect until removed.

- List the objects from the bucket to retrieve only the latest version of the object.
  For example,

```
[root@rgw-2 ~]# aws --endpoint=http://rgw.ceph.com:8080 s3api list-objects --bucket
worm-bucket
```

- List the object versions from the bucket.
  For example,

```
[root@rgw-2 ~]# aws --endpoint=http://rgw.ceph.com:8080 s3api list-objects --bucket
worm-bucket
{
    "Versions": [
        {
            "ETag": ""d560ea5652951637ba9c594d8e6ea8c1"",
            "Size": 288,
            "StorageClass": "STANDARD",
            "Key": "hosts",
            "VersionId": "Nhhk5kRS6Yp6dZXVWpZZdRcpSpBKToD",
            "IsLatest": true,
            "LastModified": "2022-06-17T08:51:17.392000+00:00",
            "Owner": {
                "DisplayName": "Test User in Tenant test",
                "ID": "test$test.user"
            }
            }
        }
    ]
}
```

- Access objects using version IDs.
  For example,

```
[root@rgw-2 ~]# aws --endpoint=http://rgw.ceph.com:8080 s3api get-object --bucket worm-
bucket  --key compliance-upload --version-id 'IGOU.vdIs3SPduZglrB-RBaK.sfXpcd'
download.1
{
    "AcceptRanges": "bytes",
    "LastModified": "2022-06-17T08:51:17+00:00",
    "ContentLength": 288,
    "ETag": ""d560ea5652951637ba9c594d8e6ea8c1"",
    "VersionId": "Nhhk5kRS6Yp6dZXVWpZZdRcpSpBKToD",
    "ContentType": "binary/octet-stream",
    "Metadata": {},
    "ObjectLockMode": "COMPLIANCE",
    "ObjectLockRetainUntilDate": "2023-06-17T08:51:17+00:00"
}
```

Bucket logging

Bucket logging records all access requests to a bucket and stores them as log objects in a separate log bucket. The log data can be used to monitor bucket activity, detect unauthorized access, analyze usage patterns, and maintain an audit trail of bucket changes.

Logging is configured at the bucket level and can be enabled or disabled at any time. The log bucket can collect logs from multiple source buckets. To distinguish logs from different buckets, configure a unique prefix for each source bucket so that the logs are stored in separate objects within the log bucket.Before enabling logging, ensure that the log bucket is created. The log bucket cannot be the same as the source bucket, and it cannot have logging, compression, or server-side encryption (SSE-S3 or SSE-C) enabled. It must not have RequestPayer enabled. Source and log buckets must be in the same zonegroup, but they can belong to different accounts if the log bucket policy allows it.

> **Note:** The log bucket may have object lock enabled with a default retention period. The 16-byte unique ID in the log object name is a lexicographically ordered value made up of a 10-byte counter followed by a 6-byte random alphanumeric string. If the counter is unavailable, the ID is generated entirely from a random alphanumeric string.

The logs are stored as objects in a dedicated log bucket, which can collect events from multiple source buckets. Each source bucket should be configured with a unique prefix so that logs are written to separate objects within the log bucket.

Logging modes

Bucket logging supports two modes that differ in reliability guarantees and in the bucket operations they record.

Standard mode

In Standard mode, log records are written to the log bucket after the bucket operation completes. Failures in logging do not affect the bucket operation. Logs follow the AWS-compatible standard record format, which includes information such as request ID, client IP, HTTP status, and user agent.

Journal mode

In Journal mode, log records are written before the bucket operation completes. If logging fails, the corresponding bucket operation also fails, except for specific operations where failure is ignored. The following bucket operations are supported in Journal mode:

- `PutObject, CopyObject, CompleteMultipartUpload, PutObjectAcl, PutObjectLegalHold, PutObjectRetention, and PutObjectTagging` — if logging fails, the operation fails.
- `DeleteObject, DeleteObjects, and DeleteObjectTagging` — the operation continues even if logging fails.

Journal mode uses a smaller, Ceph-specific record format and is intended for journaling changes and ensuring strong consistency. Journal mode supports filtering out records based on matches of the prefixes and suffixes of the logged object keys. Regular expression matching can also be used on these to create filters. Note that it may happen that the log records were successfully written but the bucket operation failed, since the logs are written.

Reliability and flushing

Log objects do not appear in the log bucket immediately. By default, log objects are rolled every five minutes or when they reach 128 MB in size.Log objects can be flushed manually. Automatic flush occurs when logging is

disabled, when the configuration changes, or when the source bucket is deleted. The time (in seconds) can be set per source bucket via a Ceph extension to the REST API , or globally via the `rgw_bucket_logging_obj_roll_time` configuration option. If not set, the default time is 5 minutes.

Multisite

In multisite deployments, each zone maintains temporary log objects before they are flushed to the log bucket. After flushing, log objects are then replicated across all zones similar to other objects sync in a multisite.

Policies and quotas

Only the bucket owner can enable or disable logging. To use a bucket as a log bucket, its policy must allow the `s3:PutObject` action for the `logging.s3.amazonaws.com` service principal. The policy should also specify the source bucket and account that are permitted to write logs.Bucket and user quotas apply to the log bucket:

- In Journal mode, exceeding quota causes both logging and the bucket operation to fail.
- In Standard mode, logging is skipped, but the bucket operation continues successfully.

For example,

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowLoggingFromSourceBucket",
      "Effect": "Allow",
      "Principal": {
        "Service": "logging.s3.amazonaws.com"
      },
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::log-bucket-name/prefix*",
      "Condition": {
        "StringEquals": {
          "aws:SourceAccount": "source-account-id"
        },
        "ArnLike": {
          "aws:SourceArn": "arn:aws:s3:::source-bucket-name"
        }
      }
    }
  ]
}
```

Log Objects Key Format

There are two types of log object key formats: Simple and Partitioned.

The Simple log object key follows this structure:

```
<prefix><year-month-day-hour-minute-second>-<16-byte-unique-id>
```

For example,

```
fish/2024-08-06-09-40-09-0000000002AGQ6W1
```

The Partitioned log object key uses a hierarchical structure that includes the bucket owner, zone group, and date components:

```
<prefix><source-bucket-owner>/<zone-group>/[tenant:]<source-bucket-name>/<year>/<month>/
<day>/<year-month-day-hour-minute-second>-<16-byte-unique-id>
```

For example,

```
fish/testid/default/fish-bucket/2024/08/06/2024-08-06-10-11-18-0000000011D1FGPA
```

Log record formats

Bucket logging supports two formats for log records, depending on the logging mode used.

Journal format

A minimal Ceph-specific record used for journaling bucket changes.

Standard format

AWS-compatible record format with extended metadata fields such as request ID, client IP, and HTTP status.

The Journal record format uses minimum amount of data for journaling bucket changes (this is a Ceph extension):

- bucket owner (or dash if empty)
- bucket name (or dash if empty), in the format: `[tenant:]<bucket name>`
- time in the following format: `[day/month/year:hour:minute:second timezone]`
- operation in the following format: `WEBSITE/REST.<HTTP method>.<resource>`
- object key (or dash if empty)
- object size (or dash if empty
- version id (or dash if empty)
- eTag (or dash if empty)

For example,

```
testid fish [06/Aug/2024:09:40:09 +0000] REST.PUT.OBJECT myfile - 512
4cfdfc1f58e762d3e116787cb92fac60
testid fish [06/Aug/2024:09:40:28 +0000] REST.DELETE.OBJECT myfile - -
4cfdfc1f58e762d3e116787cb92fac60
```

The Standard record format is based on AWS Logging Record Format.

- bucket owner (or dash if empty)
- bucket name (or dash if empty) in the format: `[tenant:]<bucket name>`
- time in the following format: `[day/month/year:hour:minute:second timezone]` where “timezone” is in UTC offset
- client IP address (or dash if empty)
- user or account (or dash if empty)
- request ID
- operation in the following format: `WEBSITE/REST.<HTTP method>.<resource>`
- object key (or dash if empty)
- request URI in the following format: `"<HTTP method> <URI> <HTTP version>"`
- HTTP status (or dash if zero). Note that in most cases log is written before the status is known
- error code (or dash if empty)
- bytes sent (or dash if zero)
- object size (or dash if zero)
- total time (not supported, always a dash)
- turnaround time in milliseconds
- referer (or dash if empty)
- user agent (or dash if empty) inside double quotes
- version id (or dash if empty)
- host id taken from `x-amz-id-2` (or dash if empty)
- signature version (or dash if empty)
- cipher suite (or dash if empty)

- authentication type (`AuthHeader` for regular auth, `QueryString` for presigned URL or dash if unauthenticated)
- host header (or dash if empty)
- TLS version (or dash if empty)
- access point ARN (not supported, always a dash)
- ACL flag (`Yes` if an ACL was required for authorization, otherwise dash)

For example,

```
testid fish [06/Aug/2024:09:30:25 +0000] - testid 9e369a15-5f43-4f07-b638-
de920b22f91b.4179.15085270386962380710 REST.PUT.OBJECT myfile "PUT /fish/myfile HTTP/1.1"
200 - 512 512 - - - - - - - - localhost - -
testid fish [06/Aug/2024:09:30:51 +0000] - testid 9e369a15-5f43-4f07-b638-
de920b22f91b.4179.7046073853138417766 REST.GET.OBJECT myfile "GET /fish/myfile HTTP/1.1"
200 - - 512 - - - - - - - - localhost - -
testid fish [06/Aug/2024:09:30:56 +0000] - testid 9e369a15-5f43-4f07-b638-
de920b22f91b.4179.10723158448701085570 REST.DELETE.OBJECT myfile "DELETE /fish/myfile1
HTTP/1.1" 200 - - 512 - - - - - - - - localhost - -
```

For information about enabling S2 bucket logging, see “S3 enable bucket logging” on page 1497.

Identity and Access Management (IAM)

The Ceph Object Gateway supports `user accounts` as an optional feature to enable the self-service management of users, groups and roles similar to those in AWS Identity and Access Management (IAM).

Account Root User

Each account is managed by an `account root user`. Like normal users and roles, accounts and account root users must be created by an administrator using radosgw-admin or the Admin Ops API. The account root user has default permissions on all resources owned by the account. The root user’s access and secret keys can be used with the Ceph Object Gateway IAM API to create and manage IAM users and roles for the Ceph Object Gateway S3 API, including their associated access keys and policies.

Account owners are encouraged to use this account root user for management only, and create users and roles with fine-grained permissions for specific applications.

> **Note:** The account root user does not require an IAM policy to access resources within the account. However, you can explicitly deny access by attaching a policy with Deny statements. Use Deny statements with caution, as they override all allow permissions.

Resource Ownership

When a normal (non-account) user creates buckets and uploads objects, those resources are owned by the user. The associated S3 ACLs name that user as both the owner and grantee, and those buckets are only visible to the owning user in a `s3:ListBuckets request`. In contrast, when users or roles belong to an account, the resources they create are instead owned by the account itself. The associated S3 ACLs name the account id as the owner and grantee, and those buckets are visible to `s3:ListBuckets` requests sent by any user or role in that account.

Because the resources are owned by the account rather than its users, all usage statistics and quota enforcement apply to the account as a whole rather than its individual users.

Account IDS

Account identifiers can be used in several places that otherwise accept User IDs or tenant names, so Account IDs use a special format to avoid ambiguity: the string RGW followed by 17 numeric digits like `RGW33567154695143645`. An Account ID in that format is randomly generated upon account creation if one is not specified.

Account IDs are commonly found in the Amazon Resource Names (ARNs) of IAM policy documents. For example, `arn:aws:iam::RGW33567154695143645:user/A` refers to an IAM user named A in that account. The Ceph Object Gateway also supports tenant names in that position. Accounts IDs can also be used in ACLs for a Grantee of type CanonicalUser. User IDs are also supported here.

IAM Policy

While non-account users are allowed to create buckets and upload objects by default, account users start with no permissions at all. Before an IAM user can perform API operations, some policy must be added to allow it. The account root user can add identity policies to its users in several ways.

- Add policy directly to the user with the iam:PutUserPolicy and iam:AttachUserPoliicy actions.
- Create an IAM group and add group policy with the `iam:PutGroupPolicy` and `iam:AttachGroupPolicy`actions. Users added to that group with the `iam:AddUserToGroup` action will inherit all of the group's policy.
- Create an IAM role and add role policy with the `iam:PutRolePolicy` and iam:AttachRolePoliicy actions. Users that assume this role with the`sts:AssumeRole` and `sts:AssumeRoleWithWebIdentity` actions will inherit all of the role's policy.
  These identity policies are evaluated according to the rules in Evaluating policies within a single account and Cross-account policy evaluation logic.

Principals

The "Principal" ARNs in policy documents refer to users differently when they belong to an account. Outside of an account, user principals are named by user id such as `arn:aws:iam:::user/uid` or `arn:aws:iam::tenantname:user/uid`, where `uid` corresponds to the `--uid` argument from `radosgw-admin`.

Within an account, user principals instead use the user name, such as `arn:aws:iam::RGW33567154695143645:user/name` where `name` corresponds to the `--display-name` argument from `radosgw-admin`. Account users continue to match the tenant form so that existing policy continues to work when users are migrated into accounts.

Tenant Isolation

Like users, accounts can optionally belong to a tenant for namespace isolation of buckets. For example, one account named "acct" can exist under a tenant "a", and a different account named "acct" can exist under tenant "b".

A tenanted account can only contain users with the same tenant name. Regardless of tenant, account IDs and email addresses must be globally unique.

Create an account

Use the `radosgw-admin account create` command to create an IAM user.

You MUST specify a user ID and a display name. You may also specify an email address.

```
radosgw-admin account create [--account-name={name}] [--account-id={id}] [--email={email}]
```

Example

```
radosgw-admin account create --account-name=user1 --account-id=12345 --
email=user1@example.com
```

Create an account root user

Use the `radosgw-admin user create` command to create an IAM user.

You MUST specify a user ID and a display name. You may also specify an email address. A **root user** is a privileged IAM user that has full access to the account, and can perform all operations, including managing other users.

```
radosgw-admin user create --uid={userid} --display-name={name} --account-id={accountid} --
account-root --gen-secret --gen-access-key
```

Example

```
radosgw-admin user create --uid=rootuser1 --display-name="Root User One" --account-
id=account123 --account-root --gen-secret --gen-access-key
```

Delete an account

Use the `radosgw-admin account rm` command to delete an IAM user.

Removing a user removes the user from the system..

```
radosgw-admin account rm --account-id={accountid}
```

Example

```
radosgw-admin account rm --account-id=account123
```

Managing account statistics and quotas

Retrieve detailed statistics for accounts, including storage usage and object counts. Configure storage quotas at the account level and bucket level to control resource usage.

Before you begin

Before you begin, make sure that you have administrative access to the Ceph Object Gateway (`radoswg`) cluster.

Viewing account statistics

To retrieve current usage statistics for an account, run the following command:

```
radosgw-admin account stats --account-id=ACCOUNT_ID --sync-stats
```

The command returns statistics in JSON format:

```
{
  "account": "account123",
  "data_size": 3145728000,
  "num_objects": 12000,
  "num_buckets": 5,
  "usage": {
    "total_size": 3145728000,
    "num_objects": 12000
  }
}
```

The response includes the following fields:

- `data_size` - Total storage size in bytes
- `num_objects` - Total number of objects stored
- `num_buckets` - Total number of buckets in the account

Setting and enabling account quotas

Account quotas limit the total storage size or number of objects across all buckets in an account.

1. Set the quota limit.

   ```
   radosgw-admin quota set --quota-scope=account --account-id=ACCOUNT_ID --max-size=SIZE
   ```

   Replace SIZE with a value like `10G`, `500M`, or `1T`.

2. Enable the quota.

   ```
   radosgw-admin quota enable --quota-scope=account --account-id=ACCOUNT_ID
   ```

Setting and enabling bucket quotas

Bucket quotas limit the number of objects that can be stored in individual buckets within an account.

1. Set the bucket quota:

   ```
   radosgw-admin quota set --quota-scope=bucket --account-id=ACCOUNT_ID --max-
   objects=NUMBER
   ```

2. Enable the bucket quota:

```
radosgw-admin quota enable --quota-scope=bucket --account-id=ACCOUNT_ID
```

Modifying account resource limits

You can set limits on the maximum number of buckets, users, roles, and other resources within an account.

```
radosgw-admin quota set --quota-scope=account --account-id=ACCOUNT_ID --max-buckets=VALUE
```

Available limit parameters:

- `--max-buckets`
- `--max-users`
- `--max-roles`
- `--max-groups`
- `--max-access-keys`

Example: Setting a bucket limit

The following example sets the maximum number of buckets to 10000 for account `RGW12345678901234568`:

```
$ radosgw-admin quota set --quota-scope=account --account-id=RGW12345678901234568 --max-
buckets=10000
{
    "id": "RGW12345678901234568",
    "tenant": "tenant1",
    "name": "account1",
    "email": "tenataccount1",
    "quota": {
        "enabled": true,
        "check_on_raw": false,
        "max_size": 10737418240,
        "max_size_kb": 10485760,
        "max_objects": 100
    },
    "bucket_quota": {
        "enabled": false,
        "check_on_raw": false,
        "max_size": -1,
        "max_size_kb": 0,
        "max_objects": -1
    },
    "max_users": 1000,
    "max_roles": 1000,
    "max_groups": 1000,
    "max_buckets": 10000,
    "max_access_keys": 4
}
```

Verifying quota settings

To verify the current quota configuration for an account:

```
radosgw-admin account get --account-id=ACCOUNT_ID
```

Migrate an existing user to an account

Migrate an existing IAM user to a Ceph Object Gateway account by using the **`radosgw-admin user modify`** command.

When you create a role associated with a Ceph Object Gateway account, you can list it by using the **`radosgw-admin role list`** command with the **`--account-id`** parameter.

To modify an existing user IAM account:

```
radosgw-admin user modify --uid=USER_ID --account-id=ACCOUNT_ID
```

Important:

- The migration transfers ownership of all the user's buckets to the Ceph Object Gateway account.
- Account membership is permanent. Once added, you cannot remove users from their account.
- The migration does not transfer ownership of the user's notification topics to the account. Notifications continue to work, but the topics are no longer visible to SNS Topic APIs.

Because account users have no permissions by default, you must add an identity policy to restore the user's original permissions. Alternatively, you can create a new account for each existing user and use the **`--account-root`** parameter to make each user the root user of their account.

Migrating notification topics

Account topics are supported only when the `notification_v2` feature is enabled.

Migration impact

When a non-account user is migrated to an account, the existing notification topics remain accessible through the Ceph Object Gateway admin API, but the user loses access to them through the SNS Topic API. Despite this, the topics remain functional, and bucket notifications will continue to be delivered as expected.

Re-creation of topics

The account user must re-create the topics using the same names. The old topics (now inaccessible) and the new account-owned topics coexist without interference.

Updating bucket notification configurations

Buckets that are subscribed to the old user-owned topics must be updated to use the new account-owned topics. To prevent duplicate notifications, maintain the same notification IDs.
For example, if a bucket's existing notification configuration is:

```
{"TopicConfigurations": [{ "Id": "ID1", "TopicArn": "arn:aws:sns:default::topic1",
"Events": ["s3:ObjectCreated:*"]}]}
```

The updated configuration would be:

```
{"TopicConfigurations": [{ "Id": "ID1", "TopicArn":
"arn:aws:sns:default:RGW00000000000000001:topic1", "Events": ["s3:ObjectCreated:*"]}]}
```

In this example, `RGW00000000000000001` is the account ID, `topic1` is the topic name, and `ID1` is the notification ID.

Removing old topics

Once no buckets are subscribed to the old user-owned topics, they can be removed by an administrator:

```
radosgw-admin topic rm --topic topic1
```

Migrating users from non-account to Ceph Object Gateway accounts

When migrating a user from a non-account to a Ceph Object Gateway (`rgw`) account, use the following steps to ensure a seamless I/O migration and appropriate IAM configuration.

1. Assign the **`--account-root`** flag during user migration. Ensure that the **`--account-root`** flag is added when migrating the user to an account. This grants the user the necessary privileges for migration.

   ```
   radosgw-admin user modify --uid USER_ID --account-id ACCOUNT_ID --account-root
   ```

2. Attach S3 access policy. After migrating the user, attach the appropriate IAM policy to grant the user S3 access. In the following example, the `AmazonS3FullAccess` policy is assigned.

   ```
   radosgw-admin user policy attach --uid USER_ID --policy-arn arn:aws:iam::aws:policy/
   AmazonS3FullAccess
   ```

3. Remove the **`--account-root`** privilege. After the migration and granting necessary access, remove the **`--account-root`** privilege from the user to limit the scope of their permissions.

```
radosgw-admin user modify --uid USER_ID --account-root=0
```

User management

Ceph Object Storage user management refers to users that are client applications of the Ceph Object Storage service; not the Ceph Object Gateway as a client application of the Ceph Storage Cluster. You must create a user, access key, and secret to enable client applications to interact with the Ceph Object Gateway service.

There are two user types:

- *User:* The term 'user' reflects a user of the S3 interface.
- *Subuser:* The term 'subuser' reflects a user of the Swift interface. A subuser is associated to a user .

You can create, modify, view, suspend, and remove users and subusers.

> **Important:** When managing users in a multi-site deployment, ALWAYS issue the `radosgw-admin` command on a Ceph Object Gateway node within the primary zone of the primary zone group to ensure that users synchronize throughout the multi-site cluster. DO NOT create, modify, or delete users on a multi-site cluster from a secondary zone or a secondary zone group.

In addition to creating user and subuser IDs, you may add a display name and an email address for a user. You can specify a key and secret, or generate a key and secret automatically. When generating or specifying keys, note that user IDs correspond to an S3 key type and subuser IDs correspond to a swift key type. Swift keys also have access levels of `read`, `write`, `readwrite` and `full`.

User management command line syntax generally follows the pattern `user COMMAND USER_ID` where `USER_ID` is either the `--uid=` option followed by the user's ID (S3) or the `--subuser=` option followed by the user name (Swift).

Syntax

```
radosgw-admin user <create|modify|info|rm|suspend|enable|check|stats> <--uid=USER_ID|--
subuser=SUB_USER_NAME> [other-options]
```

Additional options may be required depending on the command you issue.

Multi-tenant namespace

The Ceph Object Gateway supports multi-tenancy for both the S3 and Swift APIs, where each user and bucket lies under a "tenant." Multi tenancy prevents namespace clashing when multiple tenants are using common bucket names, such as "test" and "main".

Each user and bucket lies under a tenant. For backward compatibility, a "legacy" tenant with an empty name is added. Whenever referring to a bucket without specifically specifying a tenant, the Swift API will assume the "legacy" tenant. Existing users are also stored under the legacy tenant, so they will access buckets and objects the same way as earlier releases.

Tenants as such do not have any operations on them. They appear and disappear as needed, when users are administered. In order to create, modify, and remove users with explicit tenants, either an additional option `--tenant` is supplied, or a syntax `"_TENANT_$_USER_"` is used in the parameters of the `radosgw-admin` command.

To create a user `testx$tester` for S3, run the following command:

Example

```
[root@host01 ~]# radosgw-admin --tenant testx --uid tester
                    --display-name "Test User" --access_key TESTER
                    --secret test123 user create
```

To create a user `testx$tester` for Swift, run one of the following commands:

Example

```
[root@host01 ~]# radosgw-admin --tenant testx --uid tester
                    --display-name "Test User" --subuser tester:swift
```

```
                    --key-type swift --access full subuser create

[root@host01 ~]# radosgw-admin key create --subuser 'testx$tester:swift'
                    --key-type swift --secret test123
```

Note: The subuser with explicit tenant had to be quoted in the shell.

Create a user

Use the `user create` command to create an S3-interface user.

You must specify a user ID and a display name. You can also specify an email address. If you do not specify an access key or a secret key, `radosgw-admin` generates the credentials automatically. To create a user without generating an access key and a secret key, specify the `--no-credentials` option. Alternatively, you can specify an access key, a secret key, or both.

Note: Starting in IBM Storage Ceph 7.1z1, you can specify a default storage class and placement target for a user by using the `--storage-class` and `--placement-id` options.

Syntax

```
radosgw-admin user create --uid=USER_ID
[--key-type=KEY_TYPE]
[--gen-access-key | --access-key=ACCESS_KEY]
[--gen-secret | --secret=SECRET_KEY]
[--no-credentials]
[--email=EMAIL] --display-name=DISPLAY_NAME
[--storage-class=STORAGE_CLASS]
[--placement-id=PLACEMENT_ID]
```

The `--no-credentials` option creates the user without generating an S3 access key and a secret key.

Example

The following example creates a user by specifying an access key and a secret key:

```
[root@host01 ~]# radosgw-admin user create --uid=janedoe --access-key=11BS02LGFB6AL6H1ADMW
--secret=vzCEkuryfn060dfee4fgQPqFrncKEIkh3ZcdOANY --email=jane@example.com --display-
name="Jane Doe"

{ "user_id": "janedoe",
  "display_name": "Jane Doe",
  "email": "jane@example.com",
  "suspended": 0,
  "max_buckets": 1000,
  "auid": 0,
  "subusers": [],
  "keys": [
        { "user": "janedoe",
          "access_key": "11BS02LGFB6AL6H1ADMW",
          "secret_key": "vzCEkuryfn060dfee4fgQPqFrncKEIkh3ZcdOANY"}],
  "swift_keys": [],
  "caps": [],
  "op_mask": "read, write, delete",
  "default_placement": "",
  "placement_tags": [],
  "bucket_quota": { "enabled": false,
      "max_size_kb": -1,
      "max_objects": -1},
  "user_quota": { "enabled": false,
      "max_size_kb": -1,
      "max_objects": -1},
  "temp_url_keys": []}
```

Creating a user without credentials

The following example creates an S3-interface user without generating an access key and a secret key:

```
[root@host01 ~]# radosgw-admin user create --uid=johndoe --display-name="John Doe" --no-
credentials
```

The empty `keys` array in the command output indicates that no S3 credentials were generated:

```
{ "user_id": "johndoe",
  "display_name": "John Doe",
  "email": "",
  "suspended": 0,
  "max_buckets": 1000,
  "auid": 0,
  "subusers": [],
  "keys": [],
  "swift_keys": [],
  "caps": [],
  "op_mask": "read, write, delete",
  "default_placement": "",
  "placement_tags": [],
  "bucket_quota": { "enabled": false,
      "max_size_kb": -1,
      "max_objects": -1},
  "user_quota": { "enabled": false,
      "max_size_kb": -1,
      "max_objects": -1},
  "temp_url_keys": []}
```

The user cannot authenticate to the S3 interface by using access-key authentication until credentials are created for the user.

Creating a user with a default storage class and placement target

The following example creates a user with a default storage class and placement target:

```
[root@host01 ~]# radosgw-admin user create --uid=user1 --display-name="User One" --access-
key=user123 --secret=user123 --storage-class=cold --placement-id=default-placement
```

> **Note:** Ensure that the specified storage class and placement target exist in the cluster before you create the user.

> **Important:** Check the key output. Sometimes, `radosgw-admin` generates a JSON escape character (\) that some clients cannot process. To resolve this issue, remove the JSON escape character, enclose the string in quotation marks, regenerate the key without a JSON escape character, or specify the access key and secret key manually.

Create a subuser

To create a subuser (Swift interface), you must specify the user ID (`--uid=_USERNAME_`), a subuser ID and the access level for the subuser. If you DO NOT specify a key or secret, `radosgw-admin` generates them for you automatically. However, you can specify a key, a secret, or both if you prefer not to use generated key and secret pairs.

> **Note:** `full` is not `readwrite`, as it also includes the access control policy.

Syntax

```
 radosgw-admin subuser create --uid=USER_ID --subuser=SUB_USER_ID --access=[ read | write |
readwrite | full ]
```

Example

```
[root@host01 ~]# radosgw-admin subuser create --uid=janedoe --subuser=janedoe:swift --
access=full

{ "user_id": "janedoe",
  "display_name": "Jane Doe",
  "email": "jane@example.com",
  "suspended": 0,
  "max_buckets": 1000,
  "auid": 0,
  "subusers": [
        { "id": "janedoe:swift",
          "permissions": "full-control"}],
  "keys": [
        { "user": "janedoe",
          "access_key": "11BS02LGFB6AL6H1ADMW",
          "secret_key": "vzCEkuryfn060dfee4fgQPqFrncKEIkh3ZcdOANY"}],
  "swift_keys": [],
  "caps": [],
  "op_mask": "read, write, delete",
  "default_placement": "",
  "placement_tags": [],
  "bucket_quota": { "enabled": false,
      "max_size_kb": -1,
      "max_objects": -1},
  "user_quota": { "enabled": false,
      "max_size_kb": -1,
      "max_objects": -1},
  "temp_url_keys": []}
```

Get user information

Use the **`user info`** command and the user ID to retrieve user information.

To get information about a user, specify `user info` and the user ID (`--uid=_USERNAME_`).

Example

```
[root@host01 ~]# radosgw-admin user info --uid=janedoe
```

To get information about a tenanted user, specify both the user ID and the name of the tenant.

```
[root@host01 ~]# radosgw-admin user info --uid=janedoe --tenant=test
```

> **Note:** For STS federated shadow users authenticated through an OpenID Connect (OIDC) provider, specify the user namespace by using the **`--user_ns`** option with the value `oidc`.

```
[root@host01 ~]# radosgw-admin user info --uid=janedoe --user_ns=oidc
```

Modify user information

Modify user information, by using the **`user modify`** command. Typically, modifications are made to keys and secrets, email addresses, display names, and access levels.

To modify information about a user, you must specify the user ID (`--uid=_USERNAME_`) and the attributes you want to modify.

Example

```
[root@host01 ~]# radosgw-admin user modify --uid=janedoe --display-name="Jane E. Doe"
```

Example: Modify a user's default storage class and placement target.

```
[root@host01 ~]# radosgw-admin user modify --uid=user1 --storage-class=cold --placement-
id=default-placement
```

Note: Ensure that the specified storage class and placement target exist in the cluster before modifying the user.

To modify subuser values, specify `subuser modify` and the subuser ID.

Example

```
[root@host01 ~]# radosgw-admin subuser modify --subuser=janedoe:swift --access=full
```

Enable and suspend users

When you create a user, the user is enabled by default. However, you may suspend user privileges and re-enable them at a later time.

To suspend a user, specify `user suspend` and the user ID.

```
[root@host01 ~]# radosgw-admin user suspend --uid=johndoe
```

To re-enable a suspended user, specify `user enable` and the user ID:

```
[root@host01 ~]# radosgw-admin user enable --uid=johndoe
```

Note: Disabling the user disables the subuser.

Removing a user and subuser

Removing a user removes both the user and subuser from the system. A subuser can also be removed separately.

About this task

Remove a user only when wanting to remove both a subuser and subuser from the system.

When you remove a subuser, you are removing access to the Swift interface but the user remains in the system.

Removing a user

Procedure

- Remove a user and subuser from the system, by using the **`user rm`** command with the user ID.

  ```
  radosgw-admin user rm --uid=USER_ID [--purge-keys] [--purge-data]
  ```

  For example,

  ```
  [ceph: root@host01 /]# radosgw-admin user rm --uid=johndoe --purge-data
  ```

  The following command options are available:

  `--purge-data`

  The `--purge-data` option purges all data associated with the user ID.

  `--purge-keys`

  The `--purge-keys` option purges all keys associated with the user ID.

Removing a subuser

Procedure

- Remove a subuser from the system, by using the **`subuser rm`** command with the subuser name.

```
radosgw-admin subuser rm --subuser=SUBUSER_ID:swift --purge-keys
```

Note: Use the `--purge-keys` option to purge all subuser keys.

For example,

```
[root@host01 /]# radosgw-admin subuser rm --subuser=johndoe:swift --purge-keys
```

Rename a user

To change the name of a user, use the **`radosgw-admin user rename`** command.

The time that this command takes depends on the number of buckets and objects that the user has. If the number is large, Red Hat recommends using the command in the `Screen` utility provided by the `screen` package.

Prerequisites

- A working Red Hat Ceph Storage cluster.
- `root` or `sudo` access to the host running the Ceph Object Gateway.
- Installed Ceph Object Gateway.

Procedure

1. Rename a user:

   Syntax

   ```
   radosgw-admin user rename --uid=CURRENT_USER_NAME --new-uid=NEW_USER_NAME
   ```

   Example

   ```
   [ceph: root@host01 /]# radosgw-admin user rename --uid=user1 --new-uid=user2

   {
       "user_id": "user2",
       "display_name": "user 2",
       "email": "",
       "suspended": 0,
       "max_buckets": 1000,
       "auid": 0,
       "subusers": [],
       "keys": [
           {
               "user": "user2",
               "access_key": "59EKHI6AI9F8WOW8JQZJ",
               "secret_key": "XH0uY3rKCUcuL73X0ftjXbZqUbk0cavD11rD8MsA"
           }
       ],
       "swift_keys": [],
       "caps": [],
       "op_mask": "read, write, delete",
       "default_placement": "",
       "placement_tags": [],
       "bucket_quota": {
           "enabled": false,
           "check_on_raw": false,
           "max_size": -1,
           "max_size_kb": 0,
           "max_objects": -1
       },
       "user_quota": {
           "enabled": false,
           "check_on_raw": false,
           "max_size": -1,
           "max_size_kb": 0,
           "max_objects": -1
       },
       "temp_url_keys": [],
   ```

```
    "type": "rgw"
}
```

If a user is inside a tenant, specify both the user name and the tenant:

Syntax

```
radosgw-admin user rename --uid USER_NAME --new-uid NEW_USER_NAME --tenant TENANT
```

Example

```
[ceph: root@host01 /]# radosgw-admin user rename --uid=test$user1 --new-uid=test$user2
--tenant test

1000 objects processed in tvtester1. Next marker 80_tVtester1_99
2000 objects processed in tvtester1. Next marker 64_tVtester1_44
3000 objects processed in tvtester1. Next marker 48_tVtester1_28
4000 objects processed in tvtester1. Next marker 2_tVtester1_74
5000 objects processed in tvtester1. Next marker 14_tVtester1_53
6000 objects processed in tvtester1. Next marker 87_tVtester1_61
7000 objects processed in tvtester1. Next marker 6_tVtester1_57
8000 objects processed in tvtester1. Next marker 52_tVtester1_91
9000 objects processed in tvtester1. Next marker 34_tVtester1_74
9900 objects processed in tvtester1. Next marker 9_tVtester1_95
1000 objects processed in tvtester2. Next marker 82_tVtester2_93
2000 objects processed in tvtester2. Next marker 64_tVtester2_9
3000 objects processed in tvtester2. Next marker 48_tVtester2_22
4000 objects processed in tvtester2. Next marker 32_tVtester2_42
5000 objects processed in tvtester2. Next marker 16_tVtester2_36
6000 objects processed in tvtester2. Next marker 89_tVtester2_46
7000 objects processed in tvtester2. Next marker 70_tVtester2_78
8000 objects processed in tvtester2. Next marker 51_tVtester2_41
9000 objects processed in tvtester2. Next marker 33_tVtester2_32
9900 objects processed in tvtester2. Next marker 9_tVtester2_83
{
    "user_id": "test$user2",
    "display_name": "User 2",
    "email": "",
    "suspended": 0,
    "max_buckets": 1000,
    "auid": 0,
    "subusers": [],
    "keys": [
        {
            "user": "test$user2",
            "access_key": "user2",
            "secret_key": "123456789"
        }
    ],
    "swift_keys": [],
    "caps": [],
    "op_mask": "read, write, delete",
    "default_placement": "",
    "placement_tags": [],
    "bucket_quota": {
        "enabled": false,
        "check_on_raw": false,
        "max_size": -1,
        "max_size_kb": 0,
        "max_objects": -1
    },
    "user_quota": {
        "enabled": false,
        "check_on_raw": false,
        "max_size": -1,
        "max_size_kb": 0,
        "max_objects": -1
    },
    "temp_url_keys": [],
    "type": "rgw"
}
```

2. Verify that the user has been renamed successfully:

   Syntax

   ```
   radosgw-admin user info --uid=NEW_USER_NAME
   ```

Example

```
[ceph: root@host01 /]# radosgw-admin user info --uid=user2
```

If a user is inside a tenant, use the *TENANT*$*USER_NAME* format:

Syntax

```
radosgw-admin user info --uid= TENANT$USER_NAME
```

Example

```
[ceph: root@host01 /]# radosgw-admin user info --uid=test$user2
```

- The `screen(1)` manual page

Create a key

To create a key for a user, you must specify `key create`. For a user, specify the user ID and the `s3` key type.

To create a key for a subuser, you must specify the subuser ID and the `swift` keytype.

Example

```
[ceph: root@host01 /]# radosgw-admin key create --subuser=johndoe:swift --key-type=swift --
gen-secret

{ "user_id": "johndoe",
  "rados_uid": 0,
  "display_name": "John Doe",
  "email": "john@example.com",
  "suspended": 0,
  "subusers": [
     { "id": "johndoe:swift",
       "permissions": "full-control"}],
  "keys": [
    { "user": "johndoe",
      "access_key": "QFAMEDSJP5DEKJO0DDXY",
      "secret_key": "iaSFLDVvDdQt6lkNzHyW4fPLZugBAI1g17LO0+87"}],
  "swift_keys": [
    { "user": "johndoe:swift",
      "secret_key": "E9T2rUZNu2gxUjcwUBO8n/Ev4KX6/GprEuH4qhu1"}]}
```

Add and remove access keys

Users and subusers must have access keys to use the S3 and Swift interfaces. When you create a user or subuser and you do not specify an access key and secret, the key and secret get generated automatically. You may create a key and either specify or generate the access key and/or secret. You may also remove an access key and secret.

Options include:

- `--secret=SECRET_KEY` specifies a secret key, for example, manually generated.
- `--gen-access-key` generates a random access key (for S3 users by default).
- `--gen-secret` generates a random secret key.
- `--key-type=KEY_TYPE` specifies a key type. The options are: swift and s3.

To add a key, specify the user:

Example

```
[root@host01 ~]# radosgw-admin key create --uid=johndoe --key-type=s3 --gen-access-key --
gen-secret
```

You might also specify a key and a secret.

To remove an access key, you need to specify the user and the key:

1. Find the access key for the specific user:

Example

```
[root@host01 ~]# radosgw-admin user info --uid=johndoe
```

The access key is the `"access_key"` value in the output:

Example

```
[root@host01 ~]# radosgw-admin user info --uid=johndoe
{
    "user_id": "johndoe",
    ...
    "keys": [
        {
            "user": "johndoe",
            "access_key": "0555b35654ad1656d804",
            "secret_key": "h7GhxuBLTrlhVUyxSPUKUV8r/2EI4ngqJxD7iBdBYLhwluN30JaT3Q=="
        }
    ],
    ...
}
```

2. Specify the user ID and the access key from the previous step to remove the access key:

   Syntax

   ```
   radosgw-admin key rm --uid=USER_ID --access-key ACCESS_KEY
   ```

   Example

   ```
   [root@host01 ~]# radosgw-admin key rm --uid=johndoe --access-key 0555b35654ad1656d804
   ```

Add and remove admin capabilities

The Ceph Storage Cluster provides an administrative API that enables users to run administrative functions via the REST API. By default, users DO NOT have access to this API. To enable a user to exercise administrative functionality, provide the user with administrative capabilities.

To add administrative capabilities to a user, run the following command:

Syntax

```
radosgw-admin caps add --uid=USER_ID--caps=CAPS
```

You can add read, write, or all capabilities to users, buckets, metadata, and usage (utilization).

Syntax

```
--caps="[users|buckets|metadata|usage|zone]=[*|read|write|read, write]"
```

Example

```
[root@host01 ~]# radosgw-admin caps add --uid=johndoe --caps="users=*"
```

To remove administrative capabilities from a user, run the following command:

Example

```
[root@host01 ~]# radosgw-admin caps remove --uid=johndoe --caps={caps}
```

Role management

As a storage administrator, you can create, delete, or update a role and the permissions associated with that role with the **`radosgw-admin`** commands.

A role is similar to a user and has permission policies attached to it. It can be assumed by any identity. If a user assumes a role, a set of dynamically created temporary credentials are returned to the user. A role can be used

to delegate access to users, applications and services that do not have permissions to access some S3 resources.

Creating a role

Create a role for the user with the **`radosgw-admin role create`** command. You need to create a user with the **`assume-role-policy-doc`** parameter in the command, which is the trust relationship policy document that grants an entity the permission to assume the role.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway.
- Root-level access to a Ceph Object Gateway node.
- An S3 bucket created.
- An S3 user created with user access.

Procedure

- Create the role:

  Syntax

```
radosgw-admin role create --role-name=ROLE_NAME [--path=="PATH_TO_FILE"] [--assume-
role-policy-doc=TRUST_RELATIONSHIP_POLICY_DOCUMENT]
```

  Example

```
[root@host01 ~]# radosgw-admin role create --role-name=S3Access1 --path=/
application_abc/component_xyz/ --assume-role-policy-
doc={"Version":"2012-10-17","Statement":[{"Effect":"Allow","Principal":{"AWS":
["arn:aws:iam:::user/TESTER"]},"Action":["sts:AssumeRole"]}]}

{
  "RoleId": "ca43045c-082c-491a-8af1-2eebca13deec",
  "RoleName": "S3Access1",
  "Path": "/application_abc/component_xyz/",
  "Arn": "arn:aws:iam:::role/application_abc/component_xyz/S3Access1",
  "CreateDate": "2022-06-17T10:18:29.116Z",
  "MaxSessionDuration": 3600,
  "AssumeRolePolicyDocument": "{"Version":"2012-10-17","Statement":
[{"Effect":"Allow","Principal":{"AWS":["arn:aws:iam:::user/TESTER"]},"Action":
["sts:AssumeRole"]}]}"
}
```

  The value for `--path` is `/` by default.

Getting a role

Get the information about a role with the **`role get`** command.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway.
- Root-level access to a Ceph Object Gateway node.
- An S3 bucket created.
- A role created.
- An S3 user created with user access.

Procedure

- Getting the information about the role:

  Syntax

```
radosgw-admin role get --role-name=ROLE_NAME
```

  Example

```
[root@host01 ~]# radosgw-admin role get --role-name=S3Access1

{
  "RoleId": "ca43045c-082c-491a-8af1-2eebca13deec",
  "RoleName": "S3Access1",
  "Path": "/application_abc/component_xyz/",
  "Arn": "arn:aws:iam:::role/application_abc/component_xyz/S3Access1",
  "CreateDate": "2022-06-17T10:18:29.116Z",
  "MaxSessionDuration": 3600,
  "AssumeRolePolicyDocument": "{"Version":"2012-10-17","Statement":
[{"Effect":"Allow","Principal":{"AWS":["arn:aws:iam:::user/TESTER"]},"Action":
["sts:AssumeRole"]}]}"
}
```

Reference

For more information, see “Creating a role” on page 957.

Listing a role

You can list the roles in the specific path with the **`role list`** command.

With Ceph 8.0 release, the behavior of radosgw-admin role list has changed in relation to RGW (Rados Gateway) accounts. A simple radosgw-admin role list will **not** list the roles associated with RGW accounts. To list the roles for a specific RGW account, you must specify the account ID by running the following command:

```
radosgw-admin role list --account-id <RGWaccountID>
```

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway.
- Root-level access to a Ceph Object Gateway node.
- An S3 bucket created.
- A role created.
- An S3 user created with user access.

Procedure

- List the roles:

  Syntax

```
radosgw-admin role list --role-name=ROLE_NAME [--path-prefix =PATH_PREFIX]
```

  Example

```
[root@host01 ~]# radosgw-admin role list --role-name=S3Access1 --path-prefix="/
application"

[
    {
        "RoleId": "85fb46dd-a88a-4233-96f5-4fb54f4353f7",
        "RoleName": "kvm-sts",
        "Path": "/application_abc/component_xyz/",
```

```
        "Arn": "arn:aws:iam:::role/application_abc/component_xyz/kvm-sts",
        "CreateDate": "2022-09-13T11:55:09.39Z",
        "MaxSessionDuration": 7200,
        "AssumeRolePolicyDocument": "{"Version":"2012-10-17","Statement":
[{"Effect":"Allow","Principal":{"AWS":["arn:aws:iam:::user/kvm"]},"Action":
["sts:AssumeRole"]}]}"
    },
    {
        "RoleId": "9116218d-4e85-4413-b28d-cdfafba24794",
        "RoleName": "kvm-sts-1",
        "Path": "/application_abc/component_xyz/",
        "Arn": "arn:aws:iam:::role/application_abc/component_xyz/kvm-sts-1",
        "CreateDate": "2022-09-16T00:05:57.483Z",
        "MaxSessionDuration": 3600,
        "AssumeRolePolicyDocument": "{"Version":"2012-10-17","Statement":
[{"Effect":"Allow","Principal":{"AWS":["arn:aws:iam:::user/kvm"]},"Action":
["sts:AssumeRole"]}]}"
    }
]
```

Updating assume role policy document of a role

You can update the assume role policy document that grants an entity permission to assume the role with the **`role-trust-policy modify`** command.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway.
- Root-level access to a Ceph Object Gateway node.
- An S3 bucket created.
- A role created.
- An S3 user created with user access.

Procedure

- Modify the assume role policy document of a role:

  Syntax

```
radosgw-admin role-trust-policy modify --role-name=ROLE_NAME --assume-role-policy-
doc=TRUST_RELATIONSHIP_POLICY_DOCUMENT
```

  Example

```
[root@host01 ~]# radosgw-admin role-trust-policy modify --role-name=S3Access1 --assume-
role-policy-doc={"Version":"2012-10-17","Statement":[{"Effect":"Allow","Principal":
{"AWS":["arn:aws:iam:::user/TESTER"]},"Action":["sts:AssumeRole"]}]}

{
  "RoleId": "ca43045c-082c-491a-8af1-2eebca13deec",
  "RoleName": "S3Access1",
  "Path": "/application_abc/component_xyz/",
  "Arn": "arn:aws:iam:::role/application_abc/component_xyz/S3Access1",
  "CreateDate": "2022-06-17T10:18:29.116Z",
  "MaxSessionDuration": 3600,
  "AssumeRolePolicyDocument": "{"Version":"2012-10-17","Statement":
[{"Effect":"Allow","Principal":{"AWS":["arn:aws:iam:::user/TESTER"]},"Action":
["sts:AssumeRole"]}]}"
}
```

Getting permission policy attached to a role

You can get the specific permission policy attached to a role with the **`role-policy get`** command.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway.
- Root-level access to a Ceph Object Gateway node.
- An S3 bucket created.
- A role created.
- An S3 user created with user access.

Procedure

- Get the permission policy:

  Syntax

```
radosgw-admin role-policy get --role-name=ROLE_NAME --policy-name=POLICY_NAME
```

  Example

```
[root@host01 ~]# radosgw-admin role-policy get --role-name=S3Access1 --policy-
name=Policy1

{
  "Permission policy": "{"Version":"2012-10-17","Statement":
[{"Effect":"Allow","Action":["s3:*"],"Resource":"arn:aws:s3:::example_bucket"}]}"
}
```

Listing permission policy attached to a role

You can list the names of the permission policies attached to a role with the **`role-policy list`** command.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway.
- Root-level access to a Ceph Object Gateway node.
- An S3 bucket created.
- A role created.
- An S3 user created with user access.

Procedure

- List the names of the permission policies:

  Syntax

```
radosgw-admin role-policy list --role-name=ROLE_NAME
```

  Example

```
[root@host01 ~]# radosgw-admin role-policy list --role-name=S3Access1

[
  "Policy1"
]
```

Deleting policy attached to a role

You can delete the permission policy attached to a role with the **`role-policy delete`** command.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway.
- Root-level access to a Ceph Object Gateway node.
- An S3 bucket created.
- A role created.
- An S3 user created with user access.

Procedure

- Delete the permission policy:

  Syntax

  ```
  radosgw-admin role-policy delete --role-name=ROLE_NAME --policy-name=POLICY_NAME
  ```

  Example

  ```
  [root@host01 ~]# radosgw-admin role-policy delete --role-name=S3Access1 --policy-
  name=Policy1
  ```

Deleting a role

You can delete the role only after removing the permission policy attached to it.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway.
- Root-level access to a Ceph Object Gateway node.
- A role created.
- An S3 bucket created.
- An S3 user created with user access.

Procedure

1. Delete the policy attached to the role:

   Syntax

   ```
   radosgw-admin role policy delete --role-name=ROLE_NAME --policy-name=POLICY_NAME
   ```

   Example

   ```
   [root@host01 ~]# radosgw-admin role policy delete --role-name=S3Access1 --policy-
   name=Policy1
   ```

2. Delete the role:

   Syntax

   ```
   radosgw-admin role delete --role-name=ROLE_NAME
   ```

Example

```
[root@host01 ~]# radosgw-admin role delete --role-name=S3Access1
```

Reference

For more information, see “Deleting policy attached to a role” on page 961.

Updating the session duration of a role

You can update the session duration of a role with the **`role update`** command to control the length of time that a user can be signed into the account with the provided credentials.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway.
- Root-level access to a Ceph Object Gateway node.
- An S3 bucket created.
- A role created.
- An S3 user created with user access.

Procedure

- Update the max-session-duration using the `update` command:

  Syntax

  ```
  [root@node1 ~]# radosgw-admin role update --role-name=ROLE_NAME --max-session-
  duration=7200
  ```

  Example

  ```
  [root@node1 ~]# radosgw-admin role update --role-name=test-sts-role --max-session-
  duration=7200
  ```

Verification

- List the roles to verify the updates:

  Example

  ```
  [root@node1 ~]#radosgw-admin role list
  [
      {
          "RoleId": "d4caf33f-caba-42f3-8bd4-48c84b4ea4d3",
          "RoleName": "test-sts-role",
          "Path": "/",
          "Arn": "arn:aws:iam:::role/test-role",
          "CreateDate": "2022-09-07T20:01:15.563Z",
          "MaxSessionDuration": 7200,                              <<<<<<
          "AssumeRolePolicyDocument": "{"Version":"2012-10-17","Statement":
  [{"Effect":"Allow","Principal":{"AWS":["arn:aws:iam:::user/kvm"]},"Action":
  ["sts:AssumeRole"]}]}"
      }
  ]
  ```

Static web hosting

As a storage administrator, you can configure the Ceph Object Gateway to host static websites in S3 buckets. Traditional website hosting involves configuring a web server for each website, which can use resources inefficiently when content does not change dynamically. For example, sites that do not use server-side services

like PHP, servlets, databases, nodejs, and the like. This approach is substantially more economical than setting up virtual machines with web servers for each site.

Static web hosting requirements and assumptions

Static web hosting requires at least one running Red Hat Ceph Storage cluster, and at least two Ceph Object Gateway instances for the static web sites.

Red Hat assumes that each zone will have multiple gateway instances using a load balancer, such as high-availability (HA) Proxy and `keepalived`.

> **Important:** Red Hat **DOES NOT** support using a Ceph Object Gateway instance to deploy both standard S3/Swift APIs and static web hosting simultaneously.

Requirements

Use this information to understand static web hosting requirements.

Static web hosting functionality uses its own API, so configuring a gateway to use static web sites in S3 buckets requires the following:

1. S3 static web hosting uses Ceph Object Gateway instances that are separate and distinct from instances used for standard S3/Swift API use cases.
2. Gateway instances hosting S3 static web sites should have separate, non-overlapping domain names from the standard S3/Swift API gateway instances.
3. Gateway instances hosting S3 static web sites should use separate public-facing IP addresses from the standard S3/Swift API gateway instances.
4. Gateway instances hosting S3 static web sites load balance, and if necessary terminate SSL, using HAProxy/keepalive.

Reference

For more information, see the following:

- "High availability service" on page 829

Static web hosting gateway setup

Use this information to help enable Ceph Object GAteway for static web hosting.

To enable a Ceph Object Gateway for static web hosting, set the following options:

Syntax

```
ceph config set client.rgw OPTION VALUE
```

Example

```
[ceph: root@host01 /]# ceph config set client.rgw rgw_enable_static_website true
[ceph: root@host01 /]# ceph config set client.rgw rgw_enable_apis s3,s3website
[ceph: root@host01 /]# ceph config set client.rgw rgw_dns_name objects-
zonegroup.example.com
[ceph: root@host01 /]# ceph config set client.rgw rgw_dns_s3website_name objects-website-
zonegroup.example.com
[ceph: root@host01 /]# ceph config set client.rgw rgw_resolve_cname true
```

The `rgw_enable_static_website` setting MUST be `true`. The `rgw_enable_apis` setting MUST enable the `s3website` API. The `rgw_dns_name` and `rgw_dns_s3website_name` settings must provide their fully qualified domains. If the site uses canonical name extensions, then set the `rgw_resolve_cname` option to `true`.

> **Important:** The FQDNs of `rgw_dns_name` and `rgw_dns_s3website_name` **MUST NOT** overlap.

Static web hosting DNS configuration

Example of assumed DNS settings for a static web hosting.

This is an example of assumed DNS settings, where the first two lines specify the domains of the gateway instance using a standard S3 interface and point to the IPv4 and IPv6 addresses. The third line provides a wildcard CNAME setting for S3 buckets using canonical name extensions. The fourth and fifth lines specify the domains for the gateway instance using the S3 website interface and point to their IPv4 and IPv6 addresses.

```
objects-zonegroup.domain.com. IN    A 192.0.2.10
objects-zonegroup.domain.com. IN AAAA 2001:DB8::192:0:2:10
*.objects-zonegroup.domain.com. IN CNAME objects-zonegroup.domain.com.
objects-website-zonegroup.domain.com. IN    A 192.0.2.20
objects-website-zonegroup.domain.com. IN AAAA 2001:DB8::192:0:2:20
```

Note: The IP addresses in the first two lines differ from the IP addresses in the fourth and fifth lines.

If using Ceph Object Gateway in a multi-site configuration, consider using a routing solution to route traffic to the gateway closest to the client.

The Amazon Web Service (AWS) requires static web host buckets to match the host name. Ceph provides a few different ways to configure the DNS, and HTTPS will work if the proxy has a matching certificate.

Hostname to a bucket on a subdomain

To use AWS-style S3 sub-domains, use a wildcard in the DNS entry which can redirect requests to any bucket. A DNS entry might look like the following:

```
*.objects-website-zonegroup.domain.com. IN CNAME objects-website-zonegroup.domain.com.
```

Access the bucket name, where the bucket name is `bucket1`, in the following manner:

```
http://bucket1.objects-website-zonegroup.domain.com
```

Hostname to non-matching bucket

Ceph supports mapping domain names to buckets without including the bucket name in the request, which is unique to Ceph Object Gateway. To use a domain name to access a bucket, map the domain name to the bucket name. A DNS entry might look like the following, where the bucket name is `bucket2`:

```
www.example.com. IN CNAME bucket2.objects-website-zonegroup.domain.com.
```

Access the bucket in the following manner:

```
http://www.example.com
```

Hostname to long bucket with CNAME

AWS typically requires the bucket name to match the domain name. To configure the DNS for static web hosting using CNAME, the DNS entry might look like the following:

```
www.example.com. IN CNAME www.example.com.objects-website-zonegroup.domain.com.
```

Access the bucket in the following manner:

```
http://www.example.com
```

Hostname to long bucket without CNAME

If the DNS name contains other non-CNAME records, such as `SOA`, `NS`, `MX` or `TXT`, the DNS record must map the domain name directly to the IP address. For example:

```
www.example.com. IN A 192.0.2.20
www.example.com. IN AAAA 2001:DB8::192:0:2:20
```

Access the bucket in the following manner:

```
http://www.example.com
```

Creating a static web hosting site

Use this information for creating a static web hosting site.

To create a static website, perform the following steps:

1. Create an S3 bucket. The bucket name MIGHT be the same as the website's domain name. For example, `mysite.com` may have a bucket name of `mysite.com`. This is required for AWS, but it is NOT required for Ceph.

   For more information, see "Static web hosting DNS configuration" on page 964.

2. Upload the static website content to the bucket. Contents may include HTML, CSS, client-side JavaScript, images, audio/video content, and other downloadable files. A website MUST have an `index.html` file and might have an `error.html` file.

3. Verify the website's contents. At this point, only the creator of the bucket has access to the contents.

4. Set permissions on the files so that they are publicly readable.

Storage classes

Storage classes define how object data is placed and managed in Ceph Object Gateway (RGW). They map objects to specific placement targets and support cost- and performance-optimized data tiering, especially when used with S3 Bucket Lifecycle transitions.

All placement targets include a STANDARD storage class, which is applied to new objects by default. Users can override this default by setting the **`default_storage_class`** value. To store an object in a non-default storage class, include the storage class name in the request header.

S3 protocol

```
X-Amz-Storage-Class
```

Swift protocol

```
X-Object-Storage-Class
```

S3 Object Lifecycle Management can then be used to move object data between storage classes using Transition actions. When using AWS S3 SDKs, such as boto3, it is important that storage class names match those provided by AWS S3, or else the SDK will drop the request and raise an exception. Moreover, some S3 clients and libraries expect AWS-specific behavior when a storage class named or prefixed with `GLACIER` is used and thus will fail when accessing Ceph Object Gateway services. For this reason we advise that other storage class names be used with Ceph, including `INTELLIGENT-TIERING`, `STANDARD_IA`, `REDUCED_REDUNDANCY`, and `ONEZONE_IA`. Custom storage class names like `CHEAPNDEEP` are accepted by Ceph but might not be by some clients and libraries.

Use cases

Storage classes are commonly used in the following scenarios to optimize data placement, cost, and performance:

- Moving infrequently accessed objects to low-cost pools using automated lifecycle transitions.
- Assigning latency-sensitive or frequently accessed workloads to faster pools.
- Creating custom storage classes for compliance, isolation, or application-specific data placement (for example, APP_LOGS, ML_DATA).
- Automating multi-tier lifecycle transitions such as: STANDARD → STANDARD_IA → archival pool based on age or access patterns.
- Applying different durability or resiliency profiles by mapping storage classes to pools with varied replication or erasure coding.

- Separating workloads such as analytics, logging, or backup data into pools with optimized compression or cost models.

Adding a storage class

Add a new storage class to a placement target in Red Hat Ceph Storage Object Gateway and associate it with a data pool and compression settings.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- You have administrator privileges to run **`radosgw-admin`** commands.
- The zonegroup and zone are already configured in the Ceph Object Gateway.
- The required data pool (for example, `default.rgw.glacier.data`) exists in the Ceph cluster.
- The `radosgw-admin` tool is installed and available on the system where you will run the commands.

Procedure

1. Run the following command to add the new storage class to the zonegroup placement target.

```
radosgw-admin zonegroup placement add \
  --rgw-zonegroup default \

  --placement-id default-placement \

  --storage-class STANDARD_IA
```

   For example,

```
radosgw-admin zonegroup placement add \
  --rgw-zonegroup prod-zonegroup \
  --placement-id app-placement \
  --storage-class APP_LOGS
```

   This command updates the zonegroup's placement configuration to include the new STANDARD_IA storage class.

2. Run the following command to define the zone-specific placement configuration for the storage class.

```
radosgw-admin zone placement add \
  --rgw-zone DEFAULT \
  --placement-id DEFAULT_PLACEMENT \
  --storage-class STANDARD_IA \
  --data-pool DEFAULT.rgw.GLACIER.data \
  --compression lz4
```

   For example,

```
radosgw-admin zone placement add \
  --rgw-zone prod-zone \
  --placement-id app-placement \
  --storage-class APP_LOGS \
  --data-pool prod.rgw.logs.data \
  --compression lz4
```

   This command maps the STANDARD_IA storage class to the specified data pool with LZ4 compression.

Result

The new storage class STANDARD_IA is now available for use in the specified placement target. You can use it when uploading objects via S3 headers or configure transitions using S3 Bucket Lifecycle rules.

Ceph Object Gateway data layout

Ceph Object Gateway organizes its data into metadata, bucket index, and data.

While RADOS recognizes pools and objects by their Extended Attributes (`xattrs`) and object map (OMAP), conceptually Ceph Object Gateway organizes its data into three different kinds:

- "Metadata" on page 967
- "Bucket index" on page 967
- "Data" on page 968

Metadata

There are three sections of metadata:

`user`

Holds user information.

`bucket`

Holds a mapping between bucket name and bucket instance ID.

`bucket.instance`

Holds bucket instance information.

Use the following commands to view metadata entries:

```
radosgw-admin metadata get bucket:BUCKET_NAME
radosgw-admin metadata get bucket.instance:BUCKET:BUCKET_ID
radosgw-admin metadata get user:USER
radosgw-admin metadata set user:USER
```

For example,

```
[ceph: root@host01 /]# radosgw-admin metadata list
[ceph: root@host01 /]# radosgw-admin metadata list bucket
[ceph: root@host01 /]# radosgw-admin metadata list bucket.instance
[ceph: root@host01 /]# radosgw-admin metadata list user
```

Every metadata entry is kept on a single RADOS object.

Important: When using the `radosgw-admin` tool, ensure that the tool and the Ceph Cluster are of the same version. The use of mismatched versions is *not* supported.

Note: A Ceph Object Gateway object might consist of several RADOS objects. The first type of object is the head that contains the metadata, such as manifest, Access Control List (ACL), content type, ETag, and user-defined metadata. The metadata is stored in `xattrs`. The head might also contain up to 512 KB of object data, for efficiency and atomicity. The manifest describes how each object is laid out in RADOS objects.

Bucket index

Bucket index is a different kind of metadata, and kept separately. The bucket index holds a key-value map in RADOS objects. By default, it is a single RADOS object per bucket, but it is possible to shard the map over multiple RADOS objects.

The map itself is kept in OMAP associated with each RADOS object. The key of each OMAP is the name of the objects, and the value holds some basic metadata of that object, the metadata that appears when listing the bucket. Each OMAP holds a header, and some bucket accounting metadata is kept in that header such as number of objects, total size, and the like.

Note: OMAP is a key-value store, which is associated with an object, in a way similar to how extended attributes associate with a POSIX file. An object's OMAP is not physically located in the object's storage, but its precise implementation is invisible and immaterial to the Ceph Object Gateway.

Data

Objects data is kept in one or more RADOS objects for each Ceph Object Gateway object.

Object lookup path

When accessing objects, REST APIs come to Ceph Object Gateway with account information, bucket name or key, and object names. The Ceph Object Gateway uses the account information to find out the user ID and for access control. The bucket name and object key are used to address the object in a pool.

Account information

The account information has the access key in S3 or the account name in Swift.
The user ID in Ceph Object Gateway is a string, typically the actual username from the user credentials and not a hashed or mapped identifier.

When accessing a user's data, the user record is loaded from an object `USER_ID` in the `default.rgw.meta` pool with `users.uid` namespace.

Bucket or container name

The bucket or container names are represented in the `default.rgw.meta` pool with `root` namespace. Bucket record is loaded to obtain a marker, which serves as a bucket ID.

Object name or key

The object is located in the `default.rgw.buckets.data` pool. Object name is `MARKER_KEY`, for example `default.7593.4_image.png`, where the marker is `default.7593.4` and the key is `image.png`. These concatenated names are not parsed and are passed down to RADOS only. Therefore, the choice of the separator is not important and causes no ambiguity. For the same reason, you can use slashes in object names, such as keys.

Multiple data pools

It is possible to create multiple data pools so that different users' buckets are created in different RADOS pools by default, thus providing the necessary scaling. The layout and naming of these pools is controlled by a `policy` setting.

Optimize the Ceph Object Gateway's garbage collection

When new data objects are written into the storage cluster, the Ceph Object Gateway immediately allocates the storage for these new objects. After you delete or overwrite data objects in the storage cluster, the Ceph Object Gateway deletes those objects from the bucket index. Some time afterward, the Ceph Object Gateway then purges the space that was used to store the objects in the storage cluster. The process of purging the deleted object data from the storage cluster is known as Garbage Collection, or GC.

Garbage collection operations typically run in the background. You can configure these operations to either run continuously, or to run only during intervals of low activity and light workloads. By default, the Ceph Object Gateway conducts GC operations continuously. Because GC operations are a normal part of Ceph Object Gateway operations, deleted objects that are eligible for garbage collection exist most of the time.

Viewing the garbage collection queue

Before you purge deleted and overwritten objects from the storage cluster, use the **`radosgw-admin`** command to view the objects awaiting garbage collection.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph Object Gateway.

Procedure

- To view the queue of objects awaiting garbage collection:

  Example

```
[ceph: root@host01 /]# radosgw-admin gc list
```

Note: To list all entries in the queue, including unexpired entries, use the `--include-all` option.

Adjusting garbage collection settings

Adjust garbage collection settings to increase priority or tune the garbage collection queue.

The Ceph Object Gateway allocates storage for new and overwritten objects immediately. Also, the parts of a multi-part upload also use some storage.

The Ceph Object Gateway purges the storage space that is used for deleted objects after deleting the objects from the bucket index. Similarly, the Ceph Object Gateway will delete data that is associated with a multi-part upload after the multi-part upload completes or when the upload has gone inactive or failed to complete for a configurable amount of time.

Viewing the objects awaiting garbage collection can be done with the following command:

```
radosgw-admin gc list
```

Garbage collection is a background activity that runs continuously or during times of low loads, depending upon how the storage administrator configures the Ceph Object Gateway. By default, the Ceph Object Gateway conducts garbage collection operations continuously. Since garbage collection operations are a normal function of the Ceph Object Gateway, especially with object delete operations, objects eligible for garbage collection exist most of the time.

Some workloads can temporarily or permanently outpace the rate of garbage collection activity. This is especially true of delete-heavy workloads, where many objects get stored for a short period of time and then deleted. For these types of workloads, storage administrators can increase the priority of garbage collection operations relative to other operations with the following configuration parameters:

- The `rgw_gc_obj_min_wait` configuration option waits a minimum length of time, in seconds, before purging a deleted object's data. The default value is two hours, or 7200 seconds. The object is not purged immediately because a client might be reading the object. Under heavy workloads, this setting can use too much storage or have many deleted objects to purge. Do not set this value below 30 minutes, or 1800 seconds.
- The `rgw_gc_processor_period` configuration option is the garbage collection cycle run time. That is, the amount of time between the start of consecutive runs of garbage collection threads. If garbage collection runs longer than this period, the Ceph Object Gateway will not wait before running a garbage collection cycle again.
- The `rgw_gc_max_concurrent_io` configuration option specifies the maximum number of concurrent IO operations that the gateway garbage collection thread uses when purging deleted data. Under delete heavy workloads, consider increasing this setting to a larger number of concurrent IO operations.
- The `rgw_gc_max_trim_chunk` configuration option specifies the maximum number of keys to remove from the garbage collector log in a single operation. Under delete heavy operations, consider increasing the maximum number of keys so that more objects are purged during each garbage collection operation.

In addition, the following configuration parameters tune the garbage collection queue, as follows:

- The `rgw_gc_max_deferred_entries_size` configuration option sets the maximum size of deferred entries in the garbage collection queue.
- The `rgw_gc_max_queue_size` configuration option sets the maximum queue size that is used for garbage collection. Do not set this value greater than `osd_max_object_size` minus `rgw_gc_max_deferred_entries_size` minus 1 KB.
- The `rgw_gc_max_deferred` configuration option sets the maximum number of deferred entries that are stored in the garbage collection queue.

Note: In testing, with an evenly balanced delete-write workload, such as 50% delete and 50% write operations, the storage cluster fills completely in 11 hours. This is because Ceph Object Gateway garbage

collection fails to keep pace with the delete operations. The cluster status switches to the HEALTH_ERR state if this happens. Aggressive settings for parallel garbage collection tunables significantly delayed the onset of storage cluster fill in testing and can be helpful for many workloads. Typical real-world storage cluster workloads are not likely to cause a storage cluster fill primarily due to garbage collection.

Adjusting garbage collection for delete-heavy workloads

Delete-heavy workloads can outpace garbage collection when large numbers of short-lived objects are created and removed in a short time.

In some environments, garbage collection activity may not keep up with the rate at which objects are deleted, either temporarily or on an ongoing basis. This situation commonly occurs in delete-heavy workloads, where objects are stored briefly and then removed. In these cases, increasing the priority of garbage collection operations relative to other operations can help maintain system stability. For additional questions about Ceph Object Gateway garbage collection, contact Red Hat Support.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to all nodes in the storage cluster.

Procedure

1. Set the value of `rgw_gc_max_concurrent_io` to 20, and the value of `rgw_gc_max_trim_chunk` to 64:

   Example

   ```
   [ceph: root@host01 /]# ceph config set client.rgw rgw_gc_max_concurrent_io 20
   [ceph: root@host01 /]# ceph config set client.rgw rgw_gc_max_trim_chunk 64
   ```

2. Restart the Ceph Object Gateway to allow the changed settings to take effect.
3. Monitor the storage cluster during GC activity to verify that the increased values do not adversely affect performance.

Important: Never modify the value for the `rgw_gc_max_objs` option in a running cluster. You should only change this value before deploying the RGW nodes.

Reference

For more information, see the following:

- *Red Hat Ceph RGW - Garbage Collection (GC) Tuning Options*
- "General settings" on page 1088

Bucket and object listing

Buckets that belong to a given user are listed in an OMAP of an object named `USER_ID.buckets`, for example, foo.buckets, in the `default.rgw.meta` pool with `users.uid` namespace. These objects are accessed when listing buckets, when updating bucket contents, and updating and retrieving bucket statistics such as quota. These listings are kept consistent with buckets in the `.rgw` pool.

Note: See the user-visible, encoded class `cls_user_bucket_entry` and its nested class `cls_user_bucket` for the values of these OMAP entries.

Objects that belong to a given bucket are listed in a bucket index. The default naming for index objects is `.dir.MARKER` in the `default.rgw.buckets.index` pool.

Reference

For more information, see "Configure bucket index resharding" on page 1038.

Object Gateway data layout parameters

This information provides a list of data layout parameters for Ceph Object Gateway.

Known pools:

`.rgw.root` Unspecified region, zone, and global information records, one per object.

`ZONE.rgw.control` notify.*N*

`ZONE.rgw.meta` Multiple namespaces with different kinds of metadata

namespace: root BUCKET.bucket.meta.BUCKET:MARKER # see put_bucket_instance_info() The tenant is used to disambiguate buckets, but not bucket instances.

Example

```
.bucket.meta.prodtx:test%25star:default.84099.6
.bucket.meta.testcont:default.4126.1
.bucket.meta.prodtx:testcont:default.84099.4
prodtx/testcont
prodtx/test%25star
testcont
```

namespace: users.uid Contains per-user information (RGWUserInfo) in `USER` objects and per-user lists of buckets in omaps of `USER.buckets` objects. The `USER` might contain the tenant if non-empty.

Example

```
prodtx$prodt
test2.buckets
prodtx$prodt.buckets
test2
```

namespace: users.email Unimportant

namespace: users.keys 47UA98JSTJZ9YAN3OS3O

This allows Ceph Object Gateway to look up users by their access keys during authentication.

namespace: users.swift test:tester

`ZONE.rgw.buckets.index` Objects are named `.dir.MARKER`, each contains a bucket index. If the index is sharded, each shard appends the shard index after the marker. dow_].488urDFerTYXavx4yAd-Op8mxehnvTI_1 MARKER_pass:KEY

An example of a marker would be `default.16004.1` or `default.7593.4`. The current format is `ZONE.INSTANCE_ID.BUCKET`, but once generated, a marker is not parsed again, so its format might change freely in the future.

`ZONE.rgw.buckets.data` default.7593.4__shadow_.488urDFerTYXavx4yAd-Op8mxehnvTI_1 MARKER_KEY

An example of a marker would be `default.16004.1` or `default.7593.4`. The current format is `ZONE.INSTANCE_ID.BUCKET_ID`, but once generated, a marker is not parsed again, so its format might change freely in the future.

Reference

For more information, see the following:

- "Ceph Object Gateway data layout" on page 966

Rate limits for ingesting data

As a storage administrator, you can set rate limits on users and buckets based on the operations and bandwidth when saving an object in a Red Hat Ceph Storage cluster with a Ceph Object Gateway configuration. The rate

limit includes the maximum number of read operations, write operations per minute, and how many bytes per minute can be written or read per user or per bucket.

Requests that use GET or HEAD method in the REST are either read or write requests.

Important: Ensure to have the following permissions in place for ratelimit SET and GET operations through REST API:

- To get a rate limit, the user must have ratelimit capability set with read permission.

```
radosgw-admin caps add --uid=UID --caps='ratelimit=read'
```

- To set a rate limit, the user must have ratelimit capability set with write permission.

```
radosgw-admin caps add --uid=UID --caps='ratelimit=write'
```

The Ceph Object Gateway tracks the user and bucket requests separately and does not share with other gateways, which means that the desired limits configured should be divided by the number of active Object Gateways.

For example, if user A should be limited by ten ops per minute and there are two Ceph Object Gateways in the cluster, the limit over user A should be five, that is, ten ops per minute for two Ceph Object Gateways. If the requests are not balanced between Ceph Object Gateways, the rate limit may be underutilized. For example, if the ops limit is five and there are two Ceph Object Gateways, but the load balancer sends load only to one of those Ceph Object Gateways, the effective limit would be five ops, because this limit is enforced per Ceph Object Gateway.

If there is a limit reached for the bucket, but not for the user, or vice versa the request would be canceled as well.

The bandwidth counting happens after the request is accepted.As a result, this request proceeds even if the bucket or the user has reached its bandwidth limit in the middle of the request.

The Ceph Object Gateway keeps a debt of used bytes more than the configured value and prevents this user or bucket from sending more requests until their debt is paid. The debt maximum size is twice the max-read/write-bytes per minute. If user A has 1 byte read limit per minute and this user tries to GET 1 GB object, the user can do it.

After user A completes this 1 GB operation, the Ceph Object Gateway blocks the user request for up to two minutes until user A is able to send the GET request again.

The following are different options for limiting rates:

Bucket

The **`--bucket`** option allows you to specify a rate limit for a bucket.

User

The **`--uid`** option allows you to specify a rate limit for a user.

Maximum read ops

The **`--max-read-ops`** setting allows you to specify the maximum number of read ops per minute per Ceph Object Gateway.
A value of `0` disables this setting, which means unlimited access.

Maximum read bytes

The **`--max-read-bytes`** setting allows you to specify the maximum number of read bytes per minute per Ceph Object Gateway.
A value of `0` disables this setting, which means unlimited access.

Maximum write ops

The **`--max-write-ops`** setting allows you to specify the maximum number of write ops per minute per Ceph Object Gateway.
A value of `0` disables this setting, which means unlimited access.

Maximum write bytes

The **--max-write-bytes** setting allows you to specify the maximum number of write bytes per minute per Ceph Object Gateway.
A value of `0` disables this setting, which means unlimited access.

Rate limit scope

The **--rate-limit-scope** option sets the scope for the rate limit. The options are`bucket`,`user`, and `anonymous`. Bucket rate limit applies to buckets, user rate limit applies to a user, and anonymous applies to an unauthenticated user.
Anonymous scope is only available for global rate limit.

Managing user rate limits

You can set rate limits on users in a Ceph Object Gateway configuration. The rate limit on users include the maximum number of read operations, write operations per minute, and how many bytes per minute can be written or read per user.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- A Ceph Object Gateway installed.

Enabling user rate limits

About this task

You can enable the rate limit on users after setting the value of rate limits by using the **radosgw-admin ratelimit set** command with the **ratelimit-scope** set as `user`.

Procedure

1. Set the rate limit for the user.

```
radosgw-admin ratelimit set --ratelimit-scope=user --uid=USER_ID [--max-read-
ops=NUMBER_OF_OPERATIONS] [--max-read-bytes=NUMBER_OF_BYTES][--max-write-
ops=NUMBER_OF_OPERATIONS] [--max-write-bytes=NUMBER_OF_BYTES]
```

   Inputting a value of `0` for *NUMBER_OF_OPERATIONS* or *NUMBER_OF_BYTES* disables that specific rate limit attribute check.

   For example,

```
[ceph: root@host01 /]# radosgw-admin ratelimit set --ratelimit-scope=user --
uid=testing --max-read-ops=1024 --max-write-bytes=10240
```

2. Get the user rate limit.

```
radosgw-admin ratelimit get --ratelimit-scope=user --uid=USER_ID
```

   For example,

```
[ceph: root@host01 /]# radosgw-admin ratelimit get --ratelimit-scope=user --
uid=testing

{
    "user_ratelimit": {
        "max_read_ops": 1024,
        "max_write_ops": 0,
        "max_read_bytes": 0,
        "max_write_bytes": 10240,
        "enabled": false
    }
}
```

3. Enable the user rate limit.

```
radosgw-admin ratelimit enable --ratelimit-scope=user --uid=_USER_ID
```

For example,

```
[ceph: root@host01 /]# radosgw-admin ratelimit enable --ratelimit-scope=user --
uid=testing

{
    "user_ratelimit": {
        "max_read_ops": 1024,
        "max_write_ops": 0,
        "max_read_bytes": 0,
        "max_write_bytes": 10240,
        "enabled": true
    }
}
```

Disabling user rate limits

Procedure

– Disable user rate limits by running the **`ratelimit disable`** command.

```
radosgw-admin ratelimit disable --ratelimit-scope=user --uid=USER_ID
```

For example,

```
[ceph: root@host01 /]# radosgw-admin ratelimit disable --ratelimit-scope=user --
uid=testing
```

Managing bucket rate limits

You can set rate limits on buckets in a Ceph Object Gateway configuration. The rate limit on buckets include the maximum number of read operations, write operations per minute, and how many bytes per minute can be written or read per bucket.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

– A running Red Hat Ceph Storage cluster.

– A Ceph Object Gateway installed.

Enabling user rate limits

About this task

You can enable the rate limit on buckets after setting the value of rate limits by using the **`radosgw-admin ratelimit set`** command with the **`ratelimit-scope`** set as `bucket`.

Procedure

1. Set the rate limit for the bucket.

```
radosgw-admin ratelimit set --ratelimit-scope=bucket --bucket=BUCKET_NAME [--max-read-
ops=NUMBER_OF_OPERATIONS] [--max-read-bytes=NUMBER_OF_BYTES][--max-write-
ops=NUMBER_OF_OPERATIONS] [--max-write-bytes=NUMBER_OF_BYTES]
```

Inputting a value of `0` for *`NUMBER_OF_OPERATIONS`* or *`NUMBER_OF_BYTES`* disables that specific rate limit attribute check.

For example,

```
[ceph: root@host01 /]# radosgw-admin ratelimit set --ratelimit-scope=bucket --
bucket=mybucket --max-read-ops=1024 --max-write-bytes=10240
```

2. Get the bucket rate limit.

```
radosgw-admin ratelimit get --ratelimit-scope=bucket --bucket=BUCKET_NAME
```

For example,

```
[ceph: root@host01 /]# radosgw-admin ratelimit get --ratelimit-scope=bucket --
bucket=mybucket

{
    "bucket_ratelimit": {
        "max_read_ops": 1024,
        "max_write_ops": 0,
        "max_read_bytes": 0,
        "max_write_bytes": 10240,
        "enabled": false
    }
}
```

3. Enable the bucket rate limit.

```
radosgw-admin ratelimit enable --ratelimit-scope=bucket --bucket=BUCKET_NAME
```

For example,

```
[ceph: root@host01 /]# radosgw-admin ratelimit enable --ratelimit-scope=bucket --
bucket=mybucket

{
    "bucket_ratelimit": {
        "max_read_ops": 1024,
        "max_write_ops": 0,
        "max_read_bytes": 0,
        "max_write_bytes": 10240,
        "enabled": true
    }
}
```

Disabling the bucket rate limits

Procedure

- Disable bucket rate limits by running the **`ratelimit disable`** command.

```
radosgw-admin ratelimit disable --ratelimit-scope=bucket --bucket=BUCKET_NAME
```

For example,

```
[ceph: root@host01 /]# radosgw-admin ratelimit disable --ratelimit-scope=bucket --
bucket=mybucket
```

Managing global rate limits

You can read or write global rate limit settings in period configuration.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- A Ceph Object Gateway installed.

About this task

You can override the user or bucket rate limit configuration by manipulating the global rate limit settings with the **`global ratelimit`** parameter, which is the counterpart of **`ratelimit set`**, **`ratelimit enable`**, and **`ratelimit disable`** commands.

Note: In a multi-site configuration, where there is a realm and period present, changes to the global rate limit must be committed using **`period update --commit`** command. If there is no period present, the Ceph Object Gateways must be restarted for the changes to take effect.

Enabling user rate limits

About this task

You can enable the rate limit on buckets after setting the value of rate limits by using the **`radosgw-admin ratelimit set`** command with the **`ratelimit-scope`** set as `bucket`.

Procedure

1. Get the global rate limit.

```
radosgw-admin global ratelimit get
```

   For example,

```
[ceph: root@host01 /]# radosgw-admin global ratelimit get

{
    "bucket_ratelimit": {
        "max_read_ops": 1024,
        "max_write_ops": 0,
        "max_read_bytes": 0,
        "max_write_bytes": 0,
        "enabled": false
    },
    "user_ratelimit": {
        "max_read_ops": 0,
        "max_write_ops": 0,
        "max_read_bytes": 0,
        "max_write_bytes": 0,
        "enabled": false
    },
    "anonymous_ratelimit": {
        "max_read_ops": 0,
        "max_write_ops": 0,
        "max_read_bytes": 0,
        "max_write_bytes": 0,
        "enabled": false
    }
}
```

2. Configure and enable rate limit scope for the buckets.
   a. Set the global rate limits for bucket.

```
radosgw-admin global ratelimit set --ratelimit-scope=bucket [--max-read-
ops=NUMBER_OF_OPERATIONS] [--max-read-bytes=NUMBER_OF_BYTES][--max-write-
ops=NUMBER_OF_OPERATIONS] [--max-write-bytes=NUMBER_OF_BYTES]
```

      For example,

```
[ceph: root@host01 /]# radosgw-admin global ratelimit set --ratelimit-scope
bucket --max-read-ops=1024
```

   b. Enable the bucket rate limit.

```
radosgw-admin global ratelimit enable --ratelimit-scope=bucket
```

      For example,

```
[ceph: root@host01 /]# radosgw-admin global ratelimit enable --ratelimit-scope
bucket
```

3. Configure and enable rate limit scope for authenticated users.
   a. Set the global rate limits for users.

```
radosgw-admin global ratelimit set --ratelimit-scope=user [--max-read-
ops=NUMBER_OF_OPERATIONS] [--max-read-bytes=NUMBER_OF_BYTES][--max-write-
ops=NUMBER_OF_OPERATIONS] [--max-write-bytes=NUMBER_OF_BYTES]
```

For example,

```
[ceph: root@host01 /]# radosgw-admin global ratelimit set --ratelimit-scope=user
--max-read-ops=1024
```

b. Enable the user rate limit.

```
radosgw-admin global ratelimit enable --ratelimit-scope=user
```

For example,

```
[ceph: root@host01 /]# radosgw-admin global ratelimit enable --ratelimit-
scope=user
```

4. Configure and enable rate limit scope for unauthenticated users.
   a. Set the global rate limits for unauthenticated users.

```
radosgw-admin global ratelimit set --ratelimit-scope=anonymous [--max-read-
ops=NUMBER_OF_OPERATIONS] [--max-read-bytes=NUMBER_OF_BYTES][--max-write-
ops=NUMBER_OF_OPERATIONS] [--max-write-bytes=NUMBER_OF_BYTES]
```

For example,

```
[ceph: root@host01 /]# radosgw-admin global ratelimit set --ratelimit-
scope=anonymous --max-read-ops=1024
```

   b. Enable the user rate limit.

```
radosgw-admin global ratelimit enable --ratelimit-scope=anonymous
```

For example,

```
[ceph: root@host01 /]# radosgw-admin global ratelimit enable --ratelimit-
scope=anonymous
```

Authenticating with Ceph Object Gateway

Configure the Red Hat Directory Server to authenticate Ceph Object Gateway users.

> **Important:** Before configuring LDAP, ensure the `ceph` and `cephadm` users are created in LDAP.

To configure the Red Hat Directory Server to authenticate Ceph Object Gateway users, perform the following steps:

1. “Installing the Red Hat Directory Server” on page 978
2. “Configuring the Directory Server firewall” on page 979
3. “Labeling ports for SELinux” on page 979
4. “Configuring LDAPS” on page 979
5. “Adding a gateway user” on page 980
6. “Configuring the gateway to use LDAP” on page 980
7. “Using a custom search filter” on page 981
8. “Adding an S3 user to the LDAP server” on page 982
9. “Exporting an LDAP token” on page 982

10. “Testing the configuration with an S3 client” on page 982

LDAP integration

To integrate LDAP with the Ceph Object Gateway, you must perform several steps.

You must install the Red Hat Directory Server on a Red Hat Enterprise Linux 9 with a graphical user interface (GUI) to use the Java Swing GUI Directory and Administration consoles. Alternatively, you can still service the Directory Server exclusively from the command line interface (CLI). Next, you must configure the Directory Server firewall to allow access to the Directory Server's secure (636) port. After that, you need to label the ports for SELinux to ensure that they do not block requests.

After the LDAP is working, you need to configure the Ceph Object Gateway servers to trust the Directory Server’s certificate. You can extract/download a PEM-formatted certificate for the Certificate Authority (CA) that signed the LDAP server’s SSL certificate and add it to the store at `/etc/openldap/certs` using the certutil command.

After that, you must create an LDAP user for the Ceph Object Gateway and configure the gateway to use LDAP. You can use a custom search filter to limit user access by using the rgw_ldap_searchfilter setting.

Finally, you need to create at least one S3 user so that an S3 client can use the LDAP user credentials and export an LDAP token when running Ceph Object Gateway with LDAP. The access token is created from the access key and secret key.

Following are the steps to configure LDAP and Ceph Object Gateway::

1. “Installing the Red Hat Directory Server” on page 978
2. “Configuring the Directory Server firewall” on page 979
3. “Labeling ports for SELinux” on page 979
4. “Configuring LDAPS” on page 979
5. “Adding a gateway user” on page 980
6. “Configuring the gateway to use LDAP” on page 980
7. “Using a custom search filter” on page 981
8. “Adding an S3 user to the LDAP server” on page 982
9. “Exporting an LDAP token” on page 982
10. “Testing the configuration with an S3 client” on page 982

Installing the Red Hat Directory Server

Red Hat Directory Server should be installed on a Red Hat Enterprise Linux 9 with a graphical user interface (GUI) in order to use the Java Swing GUI Directory and Administration consoles. However, Red Hat Directory Server can still be serviced exclusively from the command line interface (CLI).

Prerequisites

- Red Hat Enterprise Linux (RHEL) is installed on the server.
- The Directory Server node’s FQDN is resolvable using DNS or the `/etc/hosts` file.
- Register the Directory Server node to the Red Hat subscription management service.
- A valid Red Hat Directory Server subscription is available in your Red Hat account.

Procedure

- Follow the instructions in *Installing the Directory Server packages* and *Setting up a new Directory Server instance* of the Red Hat Directory Server Installation Guide.

Reference

For more information, see the following:

– *Red Hat Director Server Installation Guide*

Configuring LDAPS

The Ceph Object Gateway uses a simple ID and password to authenticate with the LDAP server, so the connection requires an SSL certificate for LDAP. Once the LDAP is working, configure the Ceph Object Gateway servers to trust the Directory Server's certificate.

1. Extract/Download a PEM-formatted certificate for the Certificate Authority (CA) that signed the LDAP server's SSL certificate.
2. Confirm that `/etc/openldap/ldap.conf` does not have `TLS_REQCERT` set.
3. Confirm that `/etc/openldap/ldap.conf` contains a `TLS_CACERTDIR /etc/openldap/certs` setting.
4. Use the `certutil` command to add the AD CA to the store at `/etc/openldap/certs`. For example, if the CA is "msad-frog-MSAD-FROG-CA", and the PEM-formatted CA file is `ldap.pem`, use the following command:

   Example

   ```
   # certutil -d /etc/openldap/certs -A -t "TC,," -n "msad-frog-MSAD-FROG-CA" -i /path/
   to/ldap.pem
   ```

5. Update SELinux on all remote LDAP sites:

   Example

   ```
   # setsebool -P httpd_can_network_connect on
   ```

   > **Note:** This still has to be set even if SELinux is in permissive mode.

6. Make the `certs` database world-readable:

   Example

   ```
   # chmod 644 /etc/openldap/certs/*
   ```

7. Connect to the server using the "ldapwhoami" command as a non-root user.

   Example

   ```
   $ ldapwhoami -H ldaps://redhat-directory-server.example.com -d 9
   ```

   The `-d 9` option will provide debugging information in case something went wrong with the SSL negotiation.

Configuring the Directory Server firewall

On the LDAP host, make sure that the firewall allows access to the Directory Server's secure (636) port, so that LDAP clients can access the Directory Server.

Leave the default unsecure port (389) closed.

```
# firewall-cmd --zone=public --add-port=636/tcp
# firewall-cmd --zone=public --add-port=636/tcp --permanent
```

Labeling ports for SELinux

To ensure SELinux does not block requests, label the ports for SELinux.

For details see the *Changing the LDAP and LDAPS Port Numbers*.

Adding a gateway user

Create an LDAP user for the Ceph Object Gateway.

Before you begin

Before creating the gateway user, ensure that the Ceph Object Gateway does not already have the user.

```
radosgw-admin metadata list user
```

For example,

```
[ceph: root@host01 /]# radosgw-admin metadata list user
```

Look in the output to be sure that the username is not in this list of users.

Procedure

1. Create an LDAP user for the Ceph Object Gateway, and make note of the `binddn`.
   Since the Ceph Object Gateway uses the `ceph` user, consider using `ceph` as the username. The user needs to have permission to search the directory. The Ceph Object Gateway binds to this user as specified in `rgw_ldap_binddn`.
2. Test to verify that the user creation worked.
   You can run a search for the user, when the user ID (`uid`) is `ceph` for `People` with the `example.com` domain.

   ```
   # ldapsearch -x -D "uid=ceph,ou=People,dc=example,dc=com" -W -H ldaps://example.com -b
   "ou=People,dc=example,dc=com" -s sub 'uid=ceph'
   ```

3. On each gateway node, create a file for the user's secret.
   For example, the secret might get stored in a file entitled `/etc/bindpass`.
   For security, change the owner of this file to the `ceph` user and group to ensure that it is not globally readable.
4. Add the `rgw_ldap_secret` option.

   ```
   ceph config set client.rgw OPTION VALUE
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph config set client.rgw rgw_ldap_secret /etc/bindpass
   ```

5. Patch the bind password file to the Ceph Object Gateway container and reapply the Ceph Object Gateway specification.
   For example,

   ```
   service_type: rgw
   service_id: rgw.1
   service_name: rgw.rgw.1
   placement:
     label: rgw
     extra_container_args:
     - -v
     - /etc/bindpass:/etc/bindpass
   ```

   > **Note:** `/etc/bindpass` is not included automatically with Red Hat Ceph Storage. Check that the content is available on all the possible Ceph Object Gateway instance nodes.

Configuring the gateway to use LDAP

Configure the Ceoh Object Gateway to be able to use LDAP.

1. Change the Ceph configuration with the following commands on all the Ceph nodes.
   Syntax

```
ceph config set client.rgw OPTION VALUE
```

```
[ceph: root@host01 /]# ceph config set client.rgw rgw_ldap_uri ldaps://:636
[ceph: root@host01 /]# ceph config set client.rgw rgw_ldap_binddn
"ou=poc,dc=example,dc=local"
[ceph: root@host01 /]# ceph config set client.rgw rgw_ldap_searchdn
"ou=poc,dc=example,dc=local"
[ceph: root@host01 /]# ceph config set client.rgw rgw_ldap_dnattr "uid"
[ceph: root@host01 /]# ceph config set client.rgw rgw_s3_auth_use_ldap true
```

2. Restart the Ceph Object Gateway.

> **Note:** Use the output from the `ceph orch ps` command, under the `NAME` column, to get the `SERVICE_TYPE.ID` information.

a. To restart the Ceph Object Gateway on an individual node in the storage cluster.

Syntax

```
systemctl restart ceph-CLUSTER_ID@SERVICE_TYPE.ID.service
```

Example

```
[root@host01 ~]# systemctl restart ceph-c4b34c6f-8365-11ba-
dc31-529020a7702d@rgw.realm.zone.host01.gwasto.service
```

b. To restart the Ceph Object Gateways on all nodes in the storage cluster.

Syntax

```
ceph orch restart SERVICE_TYPE
```

Example

```
[ceph: root@host01 /]# ceph orch restart rgw
```

Using a custom search filter

You can create a custom search filter to limit user access by using the `rgw_ldap_searchfilter` setting.

There are two ways to use the `rgw_ldap_searchfilter` setting:

1. Specifying a partial filter:

Example

```
"objectclass=inetorgperson"
```

The Ceph Object Gateway generates the search filter with the user name from the token and the value of `rgw_ldap_dnattr`. The constructed filter is then combined with the partial filter from the `rgw_ldap_searchfilter` value. For example, the user name and the settings generate the final search filter:

Example

```
"(&(uid=joe)(objectclass=inetorgperson))"
```

User `joe` is only granted access if he is found in the LDAP directory, has an object class of `inetorgperson`, and specifies a valid password.

2. Specifying a complete filter:

A complete filter must contain a `USERNAME` token which is substituted with the user name during the authentication attempt. The `rgw_ldap_dnattr` setting is not used in this case. For example, to limit valid users to a specific group, use the following filter:

Example

```
"(&(uid=@USERNAME@)(memberOf=cn=ceph-users,ou=groups,dc=mycompany,dc=com))"
```

Adding an S3 user to the LDAP server

Create at least one S3 user so that an S3 client can use the LDAP user credentials.

In the administrative console on the LDAP server, create at least one S3 user so that an S3 client can use the LDAP user credentials. Make a note of the user name and secret for use when passing the credentials to the S3 client.

Exporting an LDAP token

When running Ceph Object Gateway with LDAP, the access token is all that is required. The access token is created from the access key and secret key.

Export the access key and secret key as an LDAP token.

1. Export the access key:

   Syntax

   ```
   export RGW_ACCESS_KEY_ID="USERNAME"
   ```

2. Export the secret key:

   Syntax

   ```
   export RGW_SECRET_ACCESS_KEY="PASSWORD"
   ```

3. Export the token. For LDAP, use `ldap` as the token type (`ttype`).

   Example

   ```
   radosgw-token --encode --ttype=ldap
   ```

   For Active Directory, use `ad` as the token type.

   Example

   ```
   radosgw-token --encode --ttype=ad
   ```

   The result is a base-64 encoded string, which is the access token. Provide this access token to S3 clients in lieu of the access key. The secret key is no longer required.

4. Optional: For added convenience, export the base-64 encoded string to the `RGW_ACCESS_KEY_ID` environment variable if the S3 client uses the environment variable.

   Example

   ```
   export
   RGW_ACCESS_KEY_ID="ewogICAgIlJHV19UT0tFTiI6IHsKICAgICAgICAidmVyc2lvbiI6IDEsCiAgICAgICA
   gInR5cGUiOiAibGRhcCIsCiAgICAgICAgImlkIjogImNlcGgiLAogICAgICAgICJrZXkiOiAiODAwI0dvcmlsb
   GEiCiAgICB9Cn0K"
   ```

Testing the configuration with an S3 client

Test the configuration with a Ceph Object Gateway client, using a script such as Python Boto.

Procedure

1. Use the `RGW_ACCESS_KEY_ID` environment variable to configure the Ceph Object Gateway client. Alternatively, you can copy the base-64 encoded string and specify it as the access key. Following is an example of the configured S3 client:

```
cat .aws/credentials

[default]
aws_access_key_id =
ewogICaGbnjlwe9UT0tFTiI6IHsKICAgICAgICAidmVyc2lvbiI6IDEsCiAgICAgICAgInR5cGUiOiAiYWQiLA
ogICAgICAgICJpZCI6ICJjZXBoIiwKICAgICAgICAia2V5IjogInBhc3M0Q2VwaCIKICAgIH0KfQo=
aws_secret_access_key =
```

> **Note:** The secret key is no longer required.

2. Run the **`aws s3 ls`** command to verify the user.

```
[root@host01 ~]# aws s3 ls --endpoint http://host03

2023-12-11 17:08:50 mybucket
2023-12-24 14:55:44 mybucket2
```

3. Optional: You can also run the **`radosgw-admin user`** command to verify the user in the directory.

```
[root@host01 ~]# radosgw-admin user info --uid dir1
{
    "user_id": "dir1",
    "display_name": "dir1",
    "email": "",
    "suspended": 0,
    "max_buckets": 1000,
    "subusers": [],
    "keys": [],
    "swift_keys": [],
    "caps": [],
    "op_mask": "read, write, delete",
    "default_placement": "",
    "default_storage_class": "",
    "placement_tags": [],
    "bucket_quota": {
        "enabled": false,
        "check_on_raw": false,
        "max_size": -1,
        "max_size_kb": 0,
        "max_objects": -1
    },
    "user_quota": {
        "enabled": false,
        "check_on_raw": false,
        "max_size": -1,
        "max_size_kb": 0,
        "max_objects": -1
    },
    "temp_url_keys": [],
    "type": "ldap",
    "mfa_ids": []
}
```

Active Directory integration

Configuring Active Directory (AD) for LDAPS is important to enable secure LDAP communication between the Ceph Object Gateway (RGW) and the LDAP server

To configure an Active Directory server to authenticate Ceph Object Gateway users, perform the following steps:

1. “Using Microsoft Active Directory” on page 984
2. “Configuring Active Directory for LDAPS” on page 984
3. “Adding a gateway user” on page 980
4. “Configuring the gateway to use Active Directory” on page 985
5. Add an S3 user to the LDAP server

6. Export an LDAP token
7. Test the configuration with an S3 client

Using Microsoft Active Directory

Ceph Object Gateway LDAP authentication is compatible with any LDAP-compliant directory service that can be configured for simple bind, including Microsoft Active Directory. By using Active Directory, the Ceph Object Gateway binds as the user configured in the `rgw_ldap_binddn` setting, and uses LDAPs to ensure security.

The process for configuring Active Directory is essentially identical to "Authenticating with Ceph Object Gateway" on page 977, but may have some Windows-specific usage.

Configuring Active Directory for LDAPS

Active Directory LDAP servers are configured to use LDAPs by default. Windows Server 2012 and higher can use Active Directory Certificate Services.

Instructions for generating and installing SSL certificates for use with Active Directory LDAP are available in the following MS TechNet article:LDAP over SSL (LDAPS) Certificate.

> **Note:** Ensure that port `636` is open on the Active Directory host.

Adding a gateway user

Create an LDAP user for the Ceph Object Gateway.

Before you begin

Before creating the gateway user, ensure that the Ceph Object Gateway does not already have the user.

```
radosgw-admin metadata list user
```

For example,

```
[ceph: root@host01 /]# radosgw-admin metadata list user
```

Look in the output to be sure that the username is not in this list of users.

Procedure

1. Create an LDAP user for the Ceph Object Gateway, and make note of the `binddn`.
   Since the Ceph Object Gateway uses the `ceph` user, consider using `ceph` as the username. The user needs to have permission to search the directory. The Ceph Object Gateway binds to this user as specified in `rgw_ldap_binddn`.
2. Test to verify that the user creation worked.
   You can run a search for the user, when the user ID (`uid`) is `ceph` for `People` with the `example.com` domain.

   ```
   # ldapsearch -x -D "uid=ceph,ou=People,dc=example,dc=com" -W -H ldaps://example.com -b
   "ou=People,dc=example,dc=com" -s sub 'uid=ceph'
   ```

3. On each gateway node, create a file for the user's secret.
   For example, the secret might get stored in a file entitled `/etc/bindpass`.
   For security, change the owner of this file to the `ceph` user and group to ensure that it is not globally readable.
4. Add the `rgw_ldap_secret` option.

   ```
   ceph config set client.rgw OPTION VALUE
   ```

   For example,

```
[ceph: root@host01 /]# ceph config set client.rgw rgw_ldap_secret /etc/bindpass
```

5. Copy the updated configuration file to each Ceph node.

```
scp /etc/ceph/ceph.conf NODE:/etc/ceph
```

Configuring the gateway to use Active Directory

Configure the Ceph Object Gateway to use the Active Directory.

1. Change the Ceph configuration with the following commands on all the Ceph nodes.

   Syntax

   ```
   ceph config set client.rgw OPTION VALUE
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph config set client.rgw rgw_ldap_uri ldaps://FQDN:636
   [ceph: root@host01 /]# ceph config set client.rgw rgw_ldap_binddn "BINDDN"
   [ceph: root@host01 /]# ceph config set client.rgw rgw_ldap_searchdn "SEARCHDN"
   [ceph: root@host01 /]# ceph config set client.rgw rgw_ldap_dnattr "cn"
   [ceph: root@host01 /]# ceph config set client.rgw rgw_s3_auth_use_ldap true
   ```

   For the `rgw_ldap_uri` setting, substitute `FQDN` with the fully qualified domain name of the LDAP server. If there is more than one LDAP server, specify each domain.

   For the `rgw_ldap_binddn` setting, substitute `BINDDN` with the bind domain. With a domain of `example.com` and a `ceph` user under `users` and `accounts`, it should look something like this:

   Example

   ```
   rgw_ldap_binddn "uid=ceph,cn=users,cn=accounts,dc=example,dc=com"
   ```

   For the `rgw_ldap_searchdn` setting, substitute `SEARCHDN` with the search domain. With a domain of `example.com` and users under `users` and `accounts`, it should look something like this:

   ```
   rgw_ldap_searchdn "cn=users,cn=accounts,dc=example,dc=com"
   ```

2. Restart the Ceph Object Gateway:

   > **Note:** Use the output from the `ceph orch ps` command, under the `NAME` column, to get the `SERVICE_TYPE.ID` information.

   a. To restart the Ceph Object Gateway on an individual node in the storage cluster:

      Syntax

      ```
      systemctl restart ceph-CLUSTER_ID@SERVICE_TYPE.ID.service
      ```

      Example

      ```
      [root@host01 ~]# systemctl restart ceph-c4b34c6f-8365-11ba-
      dc31-529020a7702d@rgw.realm.zone.host01.gwasto.service
      ```

   b. To restart the Ceph Object Gateways on all nodes in the storage cluster:

      Syntax

      ```
      ceph orch restart SERVICE_TYPE
      ```

      Example

```
[ceph: root@host01 /]# ceph orch restart rgw
```

Adding an S3 user to the LDAP server

Create at least one S3 user so that an S3 client can use the LDAP user credentials.

In the administrative console on the LDAP server, create at least one S3 user so that an S3 client can use the LDAP user credentials. Make a note of the user name and secret for use when passing the credentials to the S3 client.

Exporting an LDAP token

When running Ceph Object Gateway with LDAP, the access token is all that is required. The access token is created from the access key and secret key.

Export the access key and secret key as an LDAP token.

1. Export the access key:

   Syntax

   ```
   export RGW_ACCESS_KEY_ID="USERNAME"
   ```

2. Export the secret key:

   Syntax

   ```
   export RGW_SECRET_ACCESS_KEY="PASSWORD"
   ```

3. Export the token. For LDAP, use `ldap` as the token type (`ttype`).

   Example

   ```
   radosgw-token --encode --ttype=ldap
   ```

   For Active Directory, use `ad` as the token type.

   Example

   ```
   radosgw-token --encode --ttype=ad
   ```

   The result is a base-64 encoded string, which is the access token. Provide this access token to S3 clients in lieu of the access key. The secret key is no longer required.

4. Optional: For added convenience, export the base-64 encoded string to the `RGW_ACCESS_KEY_ID` environment variable if the S3 client uses the environment variable.

   Example

   ```
   export
   RGW_ACCESS_KEY_ID="ewogICAgIlJHV19UT0tFTiI6IHsKICAgICAgICAidmVyc2lvbiI6IDEsCiAgICAgICA
   gInR5cGUiOiAibGRhcCIsCiAgICAgICAgImlkIjogImNlcGgiLAogICAgICAgICJrZXkiOiAiODAwI0dvcmlsb
   GEiCiAgICB9Cn0K"
   ```

Testing the configuration with an S3 client

Test the configuration with a Ceph Object Gateway client, using a script such as Python Boto.

Procedure

1. Use the `RGW_ACCESS_KEY_ID` environment variable to configure the Ceph Object Gateway client. Alternatively, you can copy the base-64 encoded string and specify it as the access key. Following is an example of the configured S3 client:

```
cat .aws/credentials

[default]
aws_access_key_id =
ewogICaGbnjlwe9UT0tFTiI6IHsKICAgICAgICAidmVyc2lvbiI6IDEsCiAgICAgICAgInR5cGUiOiAiYWQiLA
ogICAgICAgICJpZCI6ICJjZXBoIiwKICAgICAgICAia2V5IjogInBhc3M0Q2VwaCIKICAgIH0KfQo=
aws_secret_access_key =
```

> **Note:** The secret key is no longer required.

2. Run the **`aws s3 ls`** command to verify the user.

```
[root@host01 ~]# aws s3 ls --endpoint http://host03

2023-12-11 17:08:50 mybucket
2023-12-24 14:55:44 mybucket2
```

3. Optional: You can also run the **`radosgw-admin user`** command to verify the user in the directory.

```
[root@host01 ~]# radosgw-admin user info --uid dir1
{
    "user_id": "dir1",
    "display_name": "dir1",
    "email": "",
    "suspended": 0,
    "max_buckets": 1000,
    "subusers": [],
    "keys": [],
    "swift_keys": [],
    "caps": [],
    "op_mask": "read, write, delete",
    "default_placement": "",
    "default_storage_class": "",
    "placement_tags": [],
    "bucket_quota": {
        "enabled": false,
        "check_on_raw": false,
        "max_size": -1,
        "max_size_kb": 0,
        "max_objects": -1
    },
    "user_quota": {
        "enabled": false,
        "check_on_raw": false,
        "max_size": -1,
        "max_size_kb": 0,
        "max_objects": -1
    },
    "temp_url_keys": [],
    "type": "ldap",
    "mfa_ids": []
}
```

OpenStack Keystone integration

As a storage administrator, you can use OpenStack's Keystone authentication service to authenticate users through the Ceph Object Gateway. Before you can configure the Ceph Object Gateway, you need to configure Keystone first. This enables the Swift service, and points the Keystone service to the Ceph Object Gateway. Next, you need to configure the Ceph Object Gateway to accept authentication requests from the Keystone service.

When using the OpenStack Keystone authentication service, be sure to configure the Ceph Object Gateway in the following order:

1. Understand Keystone authentication roles and usage. See “Keystone authentication and the Ceph Object Gateway” on page 988.
2. “Creating the Swift service” on page 989.
3. “Setting the Ceph Object Gateway endpoints” on page 989.
4. “Verifying Openstack is using the Ceph Object Gateway endpoints” on page 990.

5. Configure the Ceph Object Gateway in one of the following ways:
   - "Configuring the Ceph Object Gateway to use Keystone SSL" on page 991
   - "Configuring the Ceph Object Gateway to use Keystone authentication" on page 991
6. "Restarting the Ceph Object Gateway daemon" on page 992.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat OpenStack Platform environment.
- A running Red Hat Ceph Storage environment.
- A running Ceph Object Gateway environment.

Keystone authentication and the Ceph Object Gateway

Organizations that use OpenStack Keystone to authenticate users can integrate Keystone with the Ceph Object Gateway. The Ceph Object Gateway enables the gateway to accept a Keystone token, authenticate the user, and create a corresponding Ceph Object Gateway user. When Keystone validates a token, the gateway considers the user authenticated.

The following are the benefits of using Keystone authentication:

- Assigning `admin`, `member`, and `reader` roles to users with Keystone.
- Automatic user creation in the Ceph Object Gateway.
- Managing users with Keystone
- The Ceph Object Gateway queries Keystone periodically for a list of revoked tokens.

Roles for Keystone authentication

The OpenStack Keystone service provides three roles: `admin`, `member`, and `reader`. These roles are hierarchical. users with the `admin` role inherit the capabilities of the `member` role and users with the `member` role inherit the capabilities of the `reader` role.

> **Note:** The `member` role's read permissions only apply to objects of the project that it belongs to.

admin

The admin role is reserved for the highest level of authorization within a particular scope. This usually includes all the create, read, update, or delete operations on a resource or API.

member

The `member` role is not used directly by default. It provides flexibility during deployments and helps reduce responsibility for administrators.

For example, you can override a policy for a deployment by using the default `member` role and a simple policy override to allow system members to update services and endpoints. This provides a layer of authorization between `admin` and `reader` roles.

reader

The `reader` role is reserved for read-only operations regardless of the scope.

> **Important:** If you use a `reader` to access sensitive information such as image license keys, administrative image data, administrative volume metadata, application credentials, and secrets, you might unintentionally expose sensitive information. Hence, APIs that expose these resources should carefully consider the impact of the `reader` role and appropriately defer access to the `member` and `admin` roles.

Creating the Swift service

Before configuring the Ceph Object Gateway, configure Keystone so that the Swift service is enabled and pointing to the Ceph Object Gateway.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Access to the Ceph software repository.
- Root-level access to OpenStack controller node.

Procedure

- Create the Swift service:

```
[root@swift~]# openstack service create --name=swift --description="Swift Service"
object-store
```

  Creating the service will echo the service settings.

Example

Field	Value
description	Swift Service
enabled	True
id	37c4c0e79571404cb4644201a4a6e5ee
name	swift
type	object-store

Setting the Ceph Object Gateway endpoints

After creating the Swift service, point the service to a Ceph Object Gateway.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Access to the Ceph software repository.
- A running Swift service on a RHOSP 17.1 environment.

Procedure

- Create the OpenStack endpoints pointing to the Ceph Object Gateway:

  Syntax

```
openstack endpoint create --region REGION_NAME swift admin "URL"
openstack endpoint create --region REGION_NAME swift public "URL"
openstack endpoint create --region REGION_NAME swift internal "URL"
```

  Replace *REGION_NAME* with the name of the gateway's zone group name or region name. Replace *URL* with URLs appropriate for the Ceph Object Gateway.

  Example

```
[root@osp ~]# openstack endpoint create --region us-west swift admin "http://
radosgw.example.com:8080/swift/v1"
[root@osp ~]# openstack endpoint create --region us-west swift public "http://
radosgw.example.com:8080/swift/v1"
[root@osp ~]# openstack endpoint create --region us-west swift internal "http://
radosgw.example.com:8080/swift/v1"
```

Field	Value
adminurl	`http://radosgw.example.com:8080/swift/v1`
id	`e4249d2b60e44743a67b5e5b38c18dd3`
internalurl	`http://radosgw.example.com:8080/swift/v1`
publicurl	`http://radosgw.example.com:8080/swift/v1`
region	`us-west`
service_id	`37c4c0e79571404cb4644201a4a6e5ee`
service_name	`swift`
service_type	`object-store`

Setting the endpoints will output the service endpoint settings.

Verifying Openstack is using the Ceph Object Gateway endpoints

After creating the Swift service and setting the endpoints, show the endpoints to ensure that all settings are correct.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Access to the Ceph software repository.
- Set Ceph Object Gateway endpoints.

Procedure

1. List the endpoints under the Swift service:

```
[root@swift~]# openstack endpoint list --service=swift
```

2. Verify settings for the endpoints listed in the previous command:

```
[root@swift~] #openstack endpoint show ENDPOINT_ID
```

Showing the endpoints will echo the endpoints settings, and the service settings.

Table 113: Example

Field	Value
adminurl	http://radosgw.example.com:8080/swift/v1
enabled	True
id	e4249d2b60e44743a67b5e5b38c18dd3
internalurl	http://radosgw.example.com:8080/swift/v1
publicurl	http://radosgw.example.com:8080/swift/v1
region	us-west
service_id	37c4c0e79571404cb4644201a4a6e5ee
service_name	swift

Field	Value
service_type	object-store

Configuring the Ceph Object Gateway to use Keystone SSL

Converting the OpenSSL certificates that Keystone uses configures the Ceph Object Gateway to work with Keystone. When the Ceph Object Gateway interacts with OpenStack's Keystone authentication, Keystone will terminate with a self-signed SSL certificate.

Prerequisites

- A running, and healthy Red Hat Ceph Storage cluster.
- Access to the Ceph software repository.

Procedure

1. Convert the OpenSSL certificate to the `nss db` format:

   Example

   ```
   [root@osp ~]# mkdir /var/ceph/nss

   [root@osp ~]# openssl x509 -in /etc/keystone/ssl/certs/ca.pem -pubkey |
       certutil -d /var/ceph/nss -A -n ca -t "TCu,Cu,Tuw"

   [root@osp ~]# openssl x509 -in /etc/keystone/ssl/certs/signing_cert.pem -pubkey |
       certutil -A -d /var/ceph/nss -n signing_cert -t "P,P,P"
   ```

2. Install Keystone's SSL certificate in the node running the Ceph Object Gateway. Alternatively set the value of the configurable `rgw_keystone_verify_ssl` setting to `false`.

   Setting `rgw_keystone_verify_ssl` to `false` means that the gateway will not attempt to verify the certificate.

Configuring the Ceph Object Gateway to use Keystone authentication

Configure the Red Hat Ceph Storage to use OpenStack's Keystone authentication.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running, and healthy Red Hat Ceph Storage cluster.
- Access to the Ceph software repository.
- Have `admin` privileges to the production environment.

Procedure

1. For each gateway instance, set `nss_db_path` to the path where the NSS database is stored.
   For example,

   ```
   [ceph: root@host01 /]# ceph config set client.rgw nss_db_path "/var/lib/ceph/radosgw/
   ceph-rgw.rgw01/nss"
   ```

2. Provide authentication credentials.
   It is possible to configure a Keystone service tenant, user, and password for keystone for the OpenStack Identity API, similar to the way system administrators tend to configure OpenStack services. Providing a username and password avoids providing the shared secret to the `rgw_keystone_admin_token` setting.

   > **Important:** Disable authentication by admin token in production environments. The service tenant credentials should have `admin` privileges.

Set the following necessary configuration options:

```
ceph config set client.rgw rgw_keystone_verify_ssl TRUE/FALSE
ceph config set client.rgw rgw_s3_auth_use_keystone TRUE/FALSE
ceph config set client.rgw rgw_keystone_url KEYSTONE_URL:ADMIN_PORT
ceph config set client.rgw rgw_keystone_accepted_roles ACCEPTED_ROLES_
ceph config set client.rgw rgw_keystone_accepted_admin_roles ACCEPTED_ADMIN_ROLES
ceph config set client.rgw rgw_keystone_admin_domain default
ceph config set client.rgw rgw_keystone_admin_project SERVICE_NAME
ceph config set client.rgw rgw_keystone_admin_user KEYSTONE_TENANT_USER_NAME
ceph config set client.rgw rgw_keystone_admin_password KEYSTONE_TENANT_USER_PASSWORD
ceph config set client.rgw rgw_keystone_implicit_tenants KEYSTONE_IMPLICIT_TENANT_NAME
ceph config set client.rgw rgw_swift_versioning_enabled TRUE/FALSE
ceph config set client.rgw rgw_swift_enforce_content_length TRUE/FALSE
ceph config set client.rgw rgw_swift_account_in_url TRUE/FALSE
ceph config set client.rgw rgw_trust_forwarded_https TRUE/FALSE
ceph config set client.rgw rgw_max_attr_name_len MAXIMUM_LENGTH_OF_METADATA_NAMES
ceph config set client.rgw rgw_max_attrs_num_in_req MAXIMUM_NUMBER_OF_METADATA_ITEMS
ceph config set client.rgw rgw_max_attr_size MAXIMUM_LENGTH_OF_METADATA_VALUE
ceph config set client.rgw rgw_keystone_accepted_reader_roles SwiftSystemReader
```

For example,

```
[ceph: root@host01 /]# ceph config set client.rgw rgw_keystone_verify_ssl false
[ceph: root@host01 /]# ceph config set client.rgw rgw_s3_auth_use_keystone true
[ceph: root@host01 /]# ceph config set client.rgw rgw_keystone_url http://<public
Keystone endpoint>:5000/
[ceph: root@host01 /]# ceph config set client.rgw rgw_keystone_accepted_roles 'member,
Member, admin'
[ceph: root@host01 /]# ceph config set client.rgw rgw_keystone_accepted_admin_roles
'ResellerAdmin, swiftoperator'
[ceph: root@host01 /]# ceph config set client.rgw rgw_keystone_admin_domain default
[ceph: root@host01 /]# ceph config set client.rgw rgw_keystone_admin_project service
[ceph: root@host01 /]# ceph config set client.rgw rgw_keystone_admin_user swift
[ceph: root@host01 /]# ceph config set client.rgw rgw_keystone_admin_password password
[ceph: root@host01 /]# ceph config set client.rgw rgw_keystone_implicit_tenants true
[ceph: root@host01 /]# ceph config set client.rgw rgw_swift_versioning_enabled  true
[ceph: root@host01 /]# ceph config set client.rgw rgw_swift_enforce_content_length
true
[ceph: root@host01 /]# ceph config set client.rgw rgw_swift_account_in_url true
[ceph: root@host01 /]# ceph config set client.rgw rgw_trust_forwarded_https true
[ceph: root@host01 /]# ceph config set client.rgw rgw_max_attr_name_len 128
[ceph: root@host01 /]# ceph config set client.rgw rgw_max_attrs_num_in_req 90
[ceph: root@host01 /]# ceph config set client.rgw rgw_max_attr_size  1024
[ceph: root@host01 /]# ceph config set client.rgw rgw_keystone_accepted_reader_roles
SwiftSystemReader
```

A Ceph Object Gateway user is mapped into a Keystone `tenant`. A Keystone user has different roles assigned to it on possibly more than a single tenant. When the Ceph Object Gateway gets the ticket, it looks at the tenant, and the user roles that are assigned to that ticket, and accepts or rejects the request according to the `rgw_keystone_accepted_roles` configurable.

Related information

Users and Identity Management Guide

Restarting the Ceph Object Gateway daemon

Restarting the Ceph Object Gateway must be done to active configuration changes.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Access to the Ceph software repository.
- `admin` privileges to the production environment.

Procedure

- Once you have saved the Ceph configuration file and distributed it to each Ceph node, restart the Ceph Object Gateway instances:

Note: Use the output from the `ceph orch ps` command, under the `NAME` column, to get the *SERVICE_TYPE.ID* information.

- To restart the Ceph Object Gateway on an individual node in the storage cluster:

  Syntax

  ```
  systemctl restart ceph-CLUSTER_ID@SERVICE_TYPE.ID.service
  ```

  Example

  ```
  [root@host01 ~]# systemctl restart ceph-c4b34c6f-8365-11ba-
  dc31-529020a7702d@rgw.realm.zone.host01.gwasto.service
  ```

- To restart the Ceph Object Gateways on all nodes in the storage cluster:

  Syntax

  ```
  ceph orch restart SERVICE_TYPE
  ```

  Example

  ```
  [ceph: root@host01 /]# ceph orch restart rgw
  ```

Secure Token Service

The Amazon Web Services Secure Token Service (STS) is a set of APIs that return a set of temporary set of S3 access and secret keys to authenticate users.

Red Hat Ceph Storage Object Gateway supports a subset of Amazon STS application programming interfaces (APIs) for identity and access management (IAM).

Users first authenticate against STS and receive a short-lived S3 access key and secret key that can be used in subsequent requests.

Red Hat Ceph Storage Object Storage can authenticate S3 users by integrating with a Single Sign-On by configuring an OIDC provider. This feature enables Object Storage users to authenticate against an enterprise identity provider rather than the local Ceph Object Gateway database. For instance, if the SSO is connected to an enterprise IDP in the backend, Object Storage users can use their enterprise credentials to authenticate and get access to the Ceph Object Gateway S3 endpoint.

By using STS along with the IAM role policy feature, you can create finely tuned authorization policies to control access to your data. These policies allow you to implement either a Role-Based Access Control (RBAC) or Attribute-Based Access Control (ABAC) authorization model for your object storage data, giving you complete control over who can access the data.

Following is a simplified workflow to access S3 resources with STS:

1. The user wants access S3 resources in Red Hat Ceph Storage.
2. The user needs to authenticate against the SSO provider.
3. The SSO provider is federated with an IDP and checks if the user credentials are valid, the user gets authenticated and the SSO provides a Token to the user.
4. Using the Token provided by the SSO, the user accesses the Ceph Object Gateway STS endpoint, asking to assume an IAM role that provides the user with access to S3 resources.
5. The Red Hat Ceph Storage gateway receives the user token and asks the SSO to validate the token.
6. Once the SSO validates the token, the user is allowed to assume the role. Through STS, the user is provided with temporary access and secret keys that give the user access to the S3 resources.
7. Depending on the policies attached to the IAM role the user has assumed, the user can access a set of S3 resources.
8. For example, read for bucket A and write to bucket B.

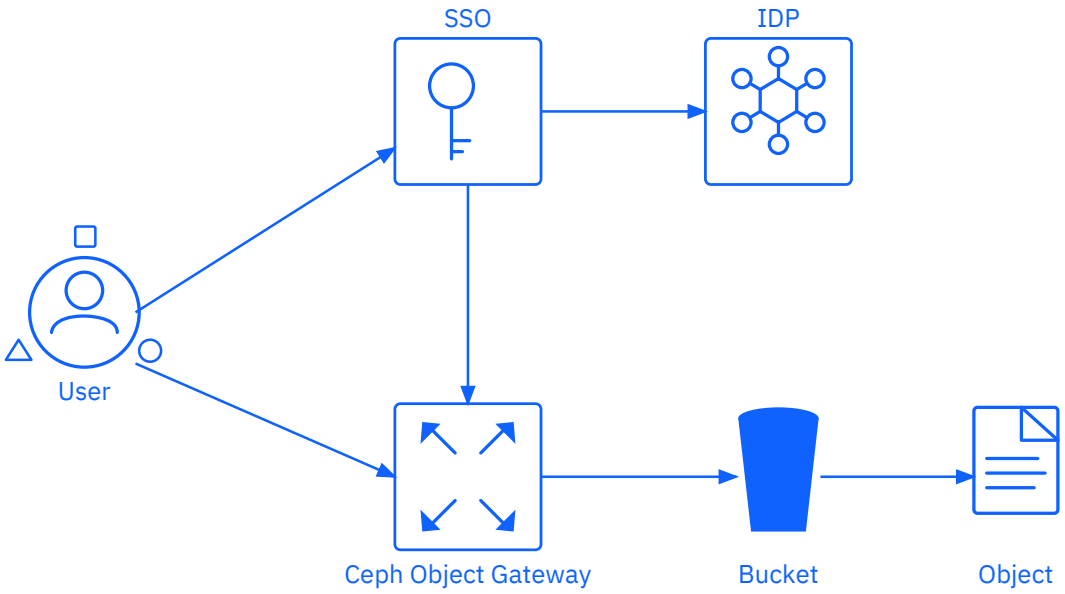


Figure 90: Secure Token Service

- For more information about STS Lite and Keystone, see "Configuring and using STS Lite with Keystone (Technology Preview)" on page 1004.
- For more information about STS Lite and Keystone limitations, see "Working around the limitations of using STS Lite with Keystone (Technology Preview)" on page 1007.

Related information

Welcome to the AWS Security Token Service API Reference

Secure Token Service application programming interfaces

The Ceph Object Gateway implements the AssumeRole and AssumeRoleWithWebIdentity Secure Token Service (STS) application programming interfaces (APIs).

AssumeRole

This API returns a set of temporary credentials for cross-account access. These temporary credentials allow for both, permission policies attached with Role and policies attached with AssumeRole API. The `RoleArn` and the `RoleSessionName` request parameters are required, but the other request parameters are optional.

`RoleArn`

Description

The role to assume for the Amazon Resource Name (ARN) with a length of 20 to 2048 characters.

Type

String

Required

Yes

`RoleSessionName`

Description

Identifying the role session name to assume. The role session name can uniquely identify a session when different principals or different reasons assume a role. This parameter's value has a length of 2 to 64 characters. The `=`, `,`, `.`, `@`, and `-` characters are allowed, but no spaces allowed.

Type

String

Required

Yes

`Policy`

Description

An identity and access management policy (IAM) in a JSON format for use in an inline session. This parameter's value has a length of 1 to 2048 characters.

Type

String

Required

No

`DurationSeconds`

Description

The duration of the session in seconds, with a minimum value of `900` seconds to a maximum value of `43200` seconds. The default value is `3600` seconds.

Type

Integer

Required

No

`ExternalId`

Description

When assuming a role for another account, provide the unique external identifier if available. This parameter's value has a length of 2 to 1224 characters.

Type

String

Required

No

`SerialNumber`

Description

A user's identification number from their associated multi-factor authentication (MFA) device. The parameter's value can be the serial number of a hardware device or a virtual device, with a length of 9 to 256 characters.

Type

String

Required

No

`TokenCode`

Description

The value generated from the multi-factor authentication (MFA) device, if the trust policy requires MFA. If an MFA device is required, and if this parameter's value is empty or expired, then AssumeRole call returns an "access denied" error message. This parameter's value has a fixed length of 6 characters.

Type

String

Required

No

AssumeRoleWithWebIdentity

This API returns a set of temporary credentials for users who have been authenticated by an application, such as OpenID Connect or OAuth 2.0 Identity Provider. The `RoleArn` and the `RoleSessionName` request parameters are required, but the other request parameters are optional.

`RoleArn`

Description

The role to assume for the Amazon Resource Name (ARN) with a length of 20 to 2048 characters.

Type

String

Required

Yes

`RoleSessionName`

Description

Identifying the role session name to assume. The role session name can uniquely identify a session when different principals or different reasons assume a role. This parameter's value has a length of 2 to 64 characters. The `=`, `,`, `.`, `@`, and `-` characters are allowed, but no spaces are allowed.

Type

String

Required

Yes

`Policy`

Description

An identity and access management policy (IAM) in a JSON format for use in an inline session. This parameter's value has a length of 1 to 2048 characters.

Type

String

Required

No

`DurationSeconds`

Description

The duration of the session in seconds, with a minimum value of `900` seconds to a maximum value of `43200` seconds. The default value is `3600` seconds.

Type

Integer

Required

No

`ProviderId`

Description

The fully qualified host component of the domain name from the identity provider. This parameter's value is only valid for OAuth 2.0 access tokens, with a length of 4 to 2048 characters.

Type

String

Required

No

`WebIdentityToken`

Description

The OpenID Connect identity token or OAuth 2.0 access token provided from an identity provider. This parameter's value has a length of 4 to 2048 characters.

Type

String

Required

No

Reference

For more information, see Examples using the Secure Token Service APIs.

Related information

Amazon Web Services Security Token Service: AssumeRole action
Amazon Web Services Security Token Service: AssumeRoleWithWebIdentity action

Configuring the Secure Token Service

Configure the Secure Token Service (STS) for use with the Ceph Object Gateway.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- A running Ceph Object Gateway.
- Root-level access to a Ceph Manager node.

About this task

Configure the Secure Token Service (STS) for use with the Ceph Object Gateway by setting the `rgw_sts_key`, and `rgw_s3_auth_use_sts` options.

> **Important:** Do not use the STS **`backward_compatibility`** configuration option in new Red Hat Ceph Storage 99 deployments. If the option is present in an existing configuration, remove it. This option is planned for deprecation along with tenant IAM/Roles. For more information, see Deprecated functionality .

> **Note:** The S3 and STS APIs co-exist in the same namespace, and both can be accessed from the same endpoint in the Ceph Object Gateway.

Procedure

1. Set the following configuration options for the Ceph Object Gateway client.

```
ceph config set RGW_CLIENT_NAME rgw_sts_key STS_KEY
ceph config set RGW_CLIENT_NAME rgw_s3_auth_use_sts true
```

The `rgw_sts_key` is the STS key for encrypting or decrypting the session token and is exactly 16 hex characters.

> **Important:** The STS key needs to be alphanumeric.

For example,

```
[root@mgr ~]# ceph config set client.rgw rgw_sts_key 7f8fd8dd4700mnop
[root@mgr ~]# ceph config set client.rgw rgw_s3_auth_use_sts true
```

2. Restart the Ceph Object Gateway for the added key to take effect.

> **Note:** Use the output from the **ceph orch ps** command, under the *NAME* column, to get the *SERVICE_TYPE.ID* information.

   - Restart the Ceph Object Gateway on an individual node in the storage cluster.

     ```
     systemctl restart ceph-CLUSTER_ID@SERVICE_TYPE.ID.service
     ```

     For example,

     ```
     [root@host01 ~]# systemctl restart ceph-c4b34c6f-8365-11ba-
     dc31-529020a7702d@rgw.realm.zone.host01.gwasto.service
     ```

   - Restart the Ceph Object Gateways on all nodes in the storage cluster.

     ```
     ceph orch restart SERVICE_TYPE
     ```

     For example,

     ```
     [ceph: root@host01 /]# ceph orch restart rgw
     ```

What to do next

- For more information about STS APIs, see “Secure Token Service application programming interfaces” on page 994.
- For more information about using the Ceph configuration database, see “Configuration database” on page 277.

Creating a user for an OpenID Connect provider

To establish trust between the Ceph Object Gateway and the OpenID Connect Provider create a user entity and a role trust policy.

- For more information about STS APIs, see “Secure Token Service application programming interfaces” on page 994.
- For more information, see “Examples using the Secure Token Service APIs” on page 1576.

Prerequisites

- User-level access to the Ceph Object Gateway node.
- Secure Token Service configured.

Procedure

1. Create a Ceph user.

   Syntax

```
radosgw-admin --uid USER_NAME --display-name "DISPLAY_NAME" --access_key USER_NAME --
secret SECRET user create
```

Example

```
[user@rgw ~]$ radosgw-admin --uid TESTER --display-name "TestUser" --access_key TESTER
--secret test123 user create
```

2. Configure the Ceph user capabilities.

   Syntax

   ```
   radosgw-admin caps add --uid="USER_NAME" --caps="oidc-provider=*"
   ```

   Example

   ```
   [user@rgw ~]$ radosgw-admin caps add --uid="TESTER" --caps="oidc-provider=*"
   ```

Related information

Obtaining the Root CA Thumbprint for an OpenID Connect Identity Provider

Obtaining a thumbprint of an OpenID Connect provider

Configure and get the OpenID Connect provider's (IDP) configuration document.

Any SSO that follows the OIDC protocol standards can be used but the following SSO providers have been tested:

- Red Hat Single Sign-on (RH-SSO) - An open-source identity and access management solution based on Keycloak, which provides enterprise-grade security and scalability for modern applications. For more information, see Red Hat Single Sign-On.
- Key cloak - A standalone tool for identity and access management, which allows users to create a user database with custom roles and groups. For more information, see Keycloak.
- IBM Security Verify - Provides end-to-end protection of resources over geographically dispersed intranets and extranet with deep, AI-powered context for both consumer and workforce identity and access management (IAM). A free 90-day trial is available with IBM Security Verify. Start your IBM Security Verify Free trial.
- For more information about STS APIs, see "Secure Token Service application programming interfaces" on page 994.
- For more information, see "Examples using the Secure Token Service APIs" on page 1576.

Prerequisites

- Installation of the `openssl` and `curl` packages.
- Secure Token Service configured.
- User created for an OIDC provider.

Procedure

1. Get the configuration document from the IDP's URL:

   Syntax

   ```
   curl -k -v \
        -X GET \
        -H "Content-Type: application/x-www-form-urlencoded" \
        "IDP_URL:8000/CONTEXT/realms/REALM/.well-known/openid-configuration" \
      | jq .
   ```

   Example

```
[user@client ~]$ curl -k -v \
     -X GET \
     -H "Content-Type: application/x-www-form-urlencoded" \
     "http://www.example.com:8000/auth/realms/quickstart/.well-known/openid-
configuration" \
   | jq .
```

2. Get the IDP certificate:

   Syntax

```
curl -k -v \
     -X GET \
     -H "Content-Type: application/x-www-form-urlencoded" \
     "IDP_URL/CONTEXT/realms/REALM/protocol/openid-connect/certs" \
     | jq .
```

   Example

```
[user@client ~]$ curl -k -v \
     -X GET \
     -H "Content-Type: application/x-www-form-urlencoded" \
     "http://www.example.com/auth/realms/quickstart/protocol/openid-connect/certs" \
     | jq .
```

   Note: The `x5c` cert can be available on the `/certs` path or in the `/jwks` path depending on the SSO provider.

3. Copy the result of the `x5c` response from the previous command and paste it into the `certificate.crt` file. Include `––BEGIN CERTIFICATE––` at the beginning and `––END CERTIFICATE––` at the end.

```
-----BEGIN CERTIFICATE-----
MIIDYjCCAkqgAwIBAgIEEEd2CDANBgkqhkiG9w0BAQsFADBzMQkwBwYDVQQGEwAxCTAHBgNVBAgTADEJMAcGA1
UEBxMAMQkwBwYDVQQKEwAxCTAHBgNVBAsTADE6MDgGA1UEAxMxYXV0aHN2Yy1pbmxpbmVtZmEuZGV2LnZlcmlm
eS5pYm1jbG91ZHNlY3VyaXR5LmNvbTAeFw0yMTA3MDUxMzU2MzZaFw0zMTA3MDMxMzU2MzZaMHMxCTAHBgNVBA
YTADEJMAcGA1UECBMAMQkwBwYDVQQHEwAxCTAHBgNVBAoTADEJMAcGA1UECxMAMTowOAYDVQQDEzFhdXRoc3Zj
LWlubGluZW1mYS5kZXYudmVyaWZ5LmlibWNsb3Vkc2VjdXJpdHkuY29tMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ
8AMIIBCgKCAQEAphyu3HaAZ14JH/
EXetZxtNnerNuqcnfxcmLhBz9SsTlFD59ta+BOVlRnK5SdYEqO3ws2iGEzTvC55rczF+hDVHFZEBJLVLQe8ABm
i22RAtG1P0dA/
Bq8ReFxpOFVWJUBc31QM+ummW0T4yw44wQJI51LZTMz7PznB0ScpObxKe+frFKd1TCMXPlWOSzmTeFYKzR83Fg
9hsnz7Y8SKGxi+RoBbTLT+ektfWpR7O+oWZIf4INe1VYJRxZvn+qWcwI5uMRCtQkiMknc3Rj6Eupiqq6FlAjDs
0p//
EzsHAlW244jMYnHCGq0UP3oE7vViLJyiOmZw7J3rvs3m9mOQiPLoQIDAQABMA0GCSqGSIb3DQEBCwUAA4IBAQC
eVqAzSh7Tp8LgaTIFUuRbdjBAKXC9Nw3+pRBHoiUTdhqO3ualyGih9m/js/clb8Vq/
39zl0VPeaslWl2NNX9zaK7xo+ckVIOY3ucCaTC04ZUn1KzZu/7azlN0C5XSWg/
CfXgU2P3BeMNzc1UNY1BASGyWn2lEplIVWKLaDZpNdSyyGyaoQAIBdzxeNCyzDfPCa2oSO8WH1czmFiNPqR5kd
knHI96CmsQdi+DT4jwzVsYgrLfcHXmiWyIAb883hR3Pobp+Bsw7LUnxebQ5ewccjYmrJzOk5Wb5FpXBhaJH1B3
AEd6RGalRUyc/zUKdvEy0nIRMDS9x2BP3NVvZSADD
-----END CERTIFICATE-----
```

4. Get the certificate thumbprint.

   Syntax

```
openssl x509 -in CERT_FILE -fingerprint -noout
```

   Example

```
[user@client ~]$ openssl x509 -in certificate.crt -fingerprint -noout
SHA1 Fingerprint=F7:D7:B3:51:5D:D0:D3:19:DD:21:9A:43:A9:EA:72:7A:D6:06:52:87
```

   Note: If the IDP/SSO uses https instead of http with self-signed certificates, for the Ceph Object Gateway to connect to the SSO/OIDC, it needs to have the CA to trust the self-signed certificates. Bind the CA certs into the Ceph Object Gateway container:
   Example

```
[root@host01 ~]# cat /tmp/rgw.yaml
---
service_type: rgw
service_id: objectgw
service_name: rgw.objectgw
placement:
count: 1
label: rgws
extra_container_args:
- -v
- /etc/pki/tls/certs:/etc/pki/tls/certs:z
- -v
- /etc/pki/ca-trust:/etc/pki/ca-trust:z
spec:
rgw_frontend_port: 8443
rgw_frontend_type: beast
rgw_realm: multisite
rgw_zone: zone1
ssl: true
```

5. Remove all the colons from the SHA1 fingerprint and use SHA1 as the input for creating the IDP entity in the IAM request.

Related information

Obtaining the Root CA Thumbprint for an OpenID Connect Identity Provider

Registering the OpenID Connect provider

Configure and get the OpenID Connect provider's (IDP) configuration document.

- For more information about STS APIs, see "Secure Token Service application programming interfaces" on page 994.
- For more information, see "Examples using the Secure Token Service APIs" on page 1576.

Prerequisites

- Installation of the `openssl` and `curl` packages.
- Secure Token Service configured.
- User created for an OIDC provider.
- Thumbprint of an OIDC obtained.

Procedure

1. Extract URL from the token.

```
[root@host01 ~]# bash check_token_isv.sh | jq .iss

"https://keycloak-sso.apps.ocp.example.com/auth/realms/ceph"
```

2. Register the OIDC provider with Ceph Object Gateway.

```
[root@host01 ~]# aws --endpoint https://cephproxy1.example.com:8443 iam create-open-
id-connect-provider --url https://keycloak-sso.apps.ocp.example.com/auth/realms/ceph
--thumbprint-list 00E9CFD697E0B16DD13C86B0FFDC29957E5D24DF
```

3. Verify that the OIDC provider is added to the Ceph Object Gateway.

```
[root@host01 ~]# aws --endpoint https://cephproxy1.example.com:8443 iam
list-open-id-connect-providers

{
 "OpenIDConnectProviderList": [
 {
```

```
 "Arn":
"arn:aws:iam:::oidc-provider/keycloak-sso.apps.ocp.example.com/auth/realms/ceph"
 }
 ]
}
```

Related information

Obtaining the Root CA Thumbprint for an OpenID Connect Identity Provider

Creating IAM roles and policies

Learn how to create IAM roles and policies.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Secure Token Service configured.
- User created for an OIDC provider.
- Thumbprint of an OIDC obtained.
- The OIDC provider in Ceph Object Gateway registered.

About this task

Note: Configuring OIDC federation and IAM roles at the tenant level is deprecated.
Tenant-based OIDC federation users should migrate their configurations to the new Ceph Object Gateway per-account model, before feature removal.

For migration options, see “Identity and Access Management (IAM)” on page 943 .

For more information, see Deprecated functionality .

Procedure

1. Retrieve and validate JWT token.

```
[root@host01 ~]# curl -k -q -L -X POST
"https://keycloak-sso.apps.example.com/auth/realms/ceph/protocol/openid-connect/
token" \
-H 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'client_id=ceph' \
--data-urlencode 'grant_type=password' \
--data-urlencode 'client_secret=XXXXXXXXXXXXXXXXXXXXXXX' \
--data-urlencode 'scope=openid' \
--data-urlencode "username=SSOUSERNAME" \
--data-urlencode "password=SSOPASSWORD"
```

2. Verify the token.

```
[root@host01 ~]# cat check_token.sh
USERNAME=$1
PASSWORD=$2
KC_CLIENT="ceph"
KC_CLIENT_SECRET="7sQXqyMSzHIeMcSALoKaljB6sNIBDRjU"
KC_ACCESS_TOKEN="$(./get_web_token.sh $USERNAME $PASSWORD | jq -r '.access_token')"
KC_SERVER="https://keycloak-sso.apps.ocp.stg.local"
KC_CONTEXT="auth"
KC_REALM="ceph"
curl -k -s -q \
-X POST \
-u "$KC_CLIENT:$KC_CLIENT_SECRET" \
-d "token=$KC_ACCESS_TOKEN" \
"$KC_SERVER/$KC_CONTEXT/realms/$KC_REALM/protocol/openid-connect/token/introspect" |
jq .
```

```
[root@host01 ~]# ./check_token.sh s3admin passw0rd | jq .sub
"ceph"
```

In this example, the `jq` filter is used by the subfield in the token and is set to `ceph`.

3. Create a JSON file with role properties.
   Set **`Statement`** to `Allow` and the **`Action`** as `AssumeRoleWithWebIdentity`. Allow access to any user with the JWT token that matches the condition with `sub:ceph`.

```
[root@host01 ~]# cat role-rgwadmins.json
{
 "Version": "2012-10-17",
 "Statement": [
   {
     "Effect": "Allow",
     "Principal": {
       "Federated": [
         "arn:aws:iam:::oidc-provider/keycloak-sso.apps.example.com/auth/realms/ceph"
       ]
     },
     "Action": [
       "sts:AssumeRoleWithWebIdentity"
     ],
     "Condition": {
       "StringLike": { "keycloak-sso.apps.example.com/auth/realms/ceph:sub":"ceph" }
     }
   }
 ]
}
```

4. Create a Ceph Object Gateway role using the JSON file.

```
[root@host01 ~]# radosgw-admin role create --role-name rgwadmins \
--assume-role-policy-doc=$(jq -rc . /root/role-rgwadmins.json)
```

5. Once the role is successfully created, specify the S3 resources you need access to after assuming the `rgwadmins` role.

```
[root@host01 ~]# cat policy-rgwadmin.json
{
"Version": "2012-10-17",
"Statement": [
  {
   "Effect": "Allow",
   "Action": [ "s3:*" ],
   "Resource": "arn:aws:s3:::*"
  }
 ]
}
```

6. Apply the role policy for the `rgwadmins` role.

```
[root@host01 ~]# radosgw-admin role policy put --role-name=rgwadmins --policy-
name=admin --policy-doc=$(jq -rc . /root/policy-rgwadmin.json)
Permission policy attached successfully
```

Related information

Obtaining the Root CA Thumbprint for an OpenID Connect Identity Provider
Role management

Accessing S3 resources

Verify the AssumeRole with STS credentials to access S3 resources.

Prerequisites

- Secure Token Service configured.
- User created for an OIDC provider.

- Thumbprint of an OIDC obtained.
- The OIDC provider in Ceph Object Gateway registered.
- IAM roles and policies created.

Procedure

1. Following is an example of AssumeRole with STS to get temporary access and secret key to access S3 resources.

```
[roo@host01 ~]# cat test-assume-role.sh
#!/bin/bash
 AWS_CA_BUNDLE="/etc/pki/ca-trust/source/anchors/cert.pem"
unset AWS_ACCESS_KEY_ID
unset AWS_SECRET_ACCESS_KEY
unset AWS_SESSION_TOKEN
KC_ACCESS_TOKEN=$(curl -k -q -L -X POST
"https://keycloak-sso.apps.ocp.example.com/auth/realms/ceph/protocol/openid-connect/
token" \
-H 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'client_id=ceph' \
--data-urlencode 'grant_type=password' \
--data-urlencode 'client_secret=XXXXXXXXXXXXXXXXXXXXXXXX' \
--data-urlencode 'scope=openid' \
--data-urlencode "<varname>SSOUSERNAME</varname>" \
--data-urlencode "<varname>SSOPASSWORD</varname>" | jq -r .access_token)
echo ${KC_ACCESS_TOKEN}
IDM_ASSUME_ROLE_CREDS=$(aws sts assume-role-with-web-identity --role-arn
"arn:aws:iam:::role/$3" --role-session-name testbr
--endpoint=https://cephproxy1.example.com:8443
--web-identity-token="$KC_ACCESS_TOKEN")
echo "aws sts assume-role-with-web-identity --role-arn "arn:aws:iam:::role/$3"
--role-session-name testb --endpoint=https://cephproxy1.example.com:8443
--web-identity-token="$KC_ACCESS_TOKEN""
echo $IDM_ASSUME_ROLE_CREDS
 AWS_ACCESS_KEY_ID=$(echo $IDM_ASSUME_ROLE_CREDS | jq -r
.Credentials.AccessKeyId)
 AWS_SECRET_ACCESS_KEY=$(echo $IDM_ASSUME_ROLE_CREDS | jq -r
.Credentials.SecretAccessKey)
 AWS_SESSION_TOKEN=$(echo $IDM_ASSUME_ROLE_CREDS | jq -r
.Credentials.SessionToken)
```

2. Run the script.

```
[root@host01 ~]# source ./test-assume-role.sh s3admin SSOPASSWORD rgwadmins
[root@host01 ~]# aws s3 mb s3://testbucket
[root@host01 ~]# aws s3 ls
```

For more information, see "Examples using the Secure Token Service APIs" on page 1576.

Related information

Obtaining the Root CA Thumbprint for an OpenID Connect Identity Provider

Configuring and using STS Lite with Keystone (Technology Preview)

The Amazon Secure Token Service (STS) and S3 APIs co-exist in the same namespace. The STS options can be configured in conjunction with the Keystone options.

Important: Technology Preview features are not supported with production service level agreements (SLAs), might not be functionally complete, and does not recommend using them for production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

Note: Both S3 and STS APIs can be accessed using the same endpoint in Ceph Object Gateway.

- For more information about installing the Boto Python module, see Test S3 Access.
- For more information, see Create a User.

Prerequisites

- Red Hat Ceph Storage 5.0 or later.
- A running Ceph Object Gateway.
- Installation of the Boto Python module, version 3 or higher.
- Root-level access to a Ceph Manager node.
- User-level access to an OpenStack node.

Procedure

1. Set the following configuration options for the Ceph Object Gateway client:

   Syntax

   ```
   ceph config set RGW_CLIENT_NAME rgw_sts_key STS_KEY
   ceph config set RGW_CLIENT_NAME rgw_s3_auth_use_sts true
   ```

   The `rgw_sts_key` is the STS key for encrypting or decrypting the session token and is exactly 16 hex characters.

   > **Important:** The STS key needs to be alphanumeric.

   Example

   ```
   [root@mgr ~]# ceph config set client.rgw rgw_sts_key 7f8fd8dd4700mnop
   [root@mgr ~]# ceph config set client.rgw rgw_s3_auth_use_sts true
   ```

2. Generate the EC2 credentials on the OpenStack node:

   Example

   ```
   [user@osp ~]$ openstack ec2 credentials create

   +------------+---------------------------------------------------------+
   | Field      | Value                                                   |
   +------------+---------------------------------------------------------+
   | access     | b924dfc87d454d15896691182fdeb0ef                        |
   | links      | {u'self': u'http://192.168.0.15/identity/v3/users/      |
   |            | 40a7140e424f493d8165abc652dc731c/credentials/           |
   |            | OS-EC2/b924dfc87d454d15896691182fdeb0ef'}                |
   | project_id | c703801dccaf4a0aaa39bec8c481e25a                        |
   | secret     | 6a2142613c504c42a94ba2b82147dc28                        |
   | trust_id   | None                                                    |
   | user_id    | 40a7140e424f493d8165abc652dc731c                        |
   +------------+---------------------------------------------------------+
   ```

3. Use the generated credentials to get back a set of temporary security credentials using *GetSessionToken* API:

   Example

   ```
   import boto3

   access_key = b924dfc87d454d15896691182fdeb0ef
   secret_key = 6a2142613c504c42a94ba2b82147dc28

   client = boto3.client('sts',
   aws_access_key_id=access_key,
   aws_secret_access_key=secret_key,
   endpoint_url=https://www.example.com/rgw,
   region_name='',
   )
   ```

```
response = client.get_session_token(
    DurationSeconds=43200
)
```

4. Obtaining the temporary credentials can be used for making S3 calls:

   Example

```
s3client = boto3.client('s3',
  aws_access_key_id = response['Credentials']['AccessKeyId'],
  aws_secret_access_key = response['Credentials']['SecretAccessKey'],
  aws_session_token = response['Credentials']['SessionToken'],
  endpoint_url=https://www.example.com/s3,
  region_name='')

bucket = s3client.create_bucket(Bucket='my-new-shiny-bucket')
response = s3client.list_buckets()
for bucket in response["Buckets"]:
  print "{name}\t{created}".format(
    name = bucket['Name'],
    created = bucket['CreationDate'],
  )
```

5. Create a new *S3Access* role and configure a policy.

   a. Assign a user with administrative CAPS:

      Syntax

```
radosgw-admin caps add --uid="USER" --caps="roles=*"
```

      Example

```
[root@mgr ~]# radosgw-admin caps add --uid="gwadmin" --caps="roles=*"
```

   b. Create the *S3Access* role:

      Syntax

```
radosgw-admin role create --role-name=ROLE_NAME --path=PATH --assume-role-policy-
doc=TRUST_POLICY_DOC
```

      Example

```
[root@mgr ~]# radosgw-admin role create --role-name=S3Access --path=/
application_abc/component_xyz/ --assume-role-policy-doc=\{\"Version\":
\"2012-10-17\",\"Statement\":\[\{\"Effect\":\"Allow\",\"Principal\":\{\"AWS\":\
[\"arn:aws:iam:::user/TESTER\"\]\},\"Action\":\[\"sts:AssumeRole\"\]\}\]\}
```

   c. Attach a permission policy to the *S3Access* role:

      Syntax

```
radosgw-admin role-policy put --role-name=ROLE_NAME --policy-name=POLICY_NAME --
policy-doc=PERMISSION_POLICY_DOC
```

      Example

```
[root@mgr ~]# radosgw-admin role-policy put --role-name=S3Access --policy-
name=Policy --policy-doc=\{\"Version\":\"2012-10-17\",\"Statement\":\[\
{\"Effect\":\"Allow\",\"Action\":\[\"s3:*\"\],\"Resource\":
\"arn:aws:s3:::example_bucket\"\}\]\}
```

   d. Now another user can assume the role of the `gwadmin` user. For example, the `gwuser` user can assume the permissions of the `gwadmin` user.

   e. Make a note of the assuming user's `access_key` and `secret_key` values.

      Example

```
[root@mgr ~]# radosgw-admin user info --uid=gwuser | grep -A1 access_key
```

6. Use the *AssumeRole* API call, providing the `access_key` and `secret_key` values from the assuming user:

   Example

```
import boto3

access_key = 11BS02LGFB6AL6H1ADMW
secret_key = vzCEkuryfn060dfee4fgQPqFrncKEIkh3ZcdOANY

client = boto3.client('sts',
aws_access_key_id=access_key,
aws_secret_access_key=secret_key,
endpoint_url=https://www.example.com/rgw,
region_name='',
)

response = client.assume_role(
RoleArn='arn:aws:iam:::role/application_abc/component_xyz/S3Access',
RoleSessionName='Bob',
DurationSeconds=3600
)
```

> **Important:** The *AssumeRole* API requires the *S3Access* role.

Working around the limitations of using STS Lite with Keystone (Technology Preview)

Working with Keystone has some limitations. Keystone does not support Secure Token Service (STS) requests and the payload hash is not included with the request. To work around these limitations the Boto authentication code must be modified.

> **Important:** Technology Preview features are not supported with production service level agreements (SLAs), might not be functionally complete, and does not recommend using them for production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

Prerequisites

- A running Red Hat Ceph Storage cluster, version 5.0 or later.
- A running Ceph Object Gateway.
- Installation of Boto Python module, version 3 or later.

For more information about installing the Boto Python module, see Test S3 Access.

Procedure

Open and edit Boto's `auth.py` file.

1. Add the following four lines to the code block:

```
f service_name == 'sts':
          self._service_name = 's3'
      else:
          self._service_name = service_name
```

   For example:

```
class SigV4Auth(BaseSigner):
  """
  Sign a request with Signature V4.
  """
  REQUIRES_REGION = True

  def __init__(self, credentials, service_name, region_name):
      self.credentials = credentials
      # We initialize these value here so the unit tests can have
      # valid values.  But these will get overriden in ``add_auth``
      # later for real requests.
      self._region_name = region_name
      if service_name == 'sts':
          self._service_name = 's3'
      else:
          self._service_name = service_name
```

2. Add the following two lines to the code block:

```
request.headers['X-Amz-Content-SHA256'] = UNSIGNED_PAYLOAD
        else:
```

For example:

```
def _modify_request_before_signing(self, request):
        if 'Authorization' in request.headers:
            del request.headers['Authorization']
        self._set_necessary_date_headers(request)
        if self.credentials.token:
            if 'X-Amz-Security-Token' in request.headers:
                del request.headers['X-Amz-Security-Token']
            request.headers['X-Amz-Security-Token'] = self.credentials.token

        if not request.context.get('payload_signing_enabled', True):
            if 'X-Amz-Content-SHA256' in request.headers:
                del request.headers['X-Amz-Content-SHA256']
            request.headers['X-Amz-Content-SHA256'] = UNSIGNED_PAYLOAD
        else:
            request.headers['X-Amz-Content-SHA256'] = self.payload(request)
```

Security

As a storage administrator, securing the storage cluster environment is important. Red Hat Ceph Storage provides encryption and key management to secure the Ceph Object Gateway access point.

Prerequisites

- A healthy running Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway software.

QAT acceleration for encryption and compression

Intel QuickAssist Technology (QAT) can provide extended accelerated encryption and compression services by offloading the actual encryption and compression requests to the hardware QuickAssist accelerators, which are more efficient in terms of cost and power than general purpose CPUs for those specific compute-intensive workloads.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Ceph Object gateway installed.
- *Grub* is configured to pass the **`intel_iommu`** parameter.

```
grubby --update-kernel=ALL --args="intel_iommu=on"
```

About this task

Important: Hardware accelerated compression in Ceph Object Gateway requires RHEL 9.4 on a Sapphire or Emerald Rapids Xeon CPU (or newer) with QAT devices. For more information, see Intel Ark.

Setting up the QAT service

You can set up the QAT service to encrypt and compress the Ceph Object Gateway objects.

Procedure

1. Install `qatlib-service`, `qatlib`, `qatzip`, and `qatengine` packages.

```
dnf install -y qatlib-service qatlib qatzip qatengine
```

2. Add `root` to the `QAT` group on the host.

```
usermod -aG qat root
```

3. Check that the `limits.conf` file exists with one of the following **`ServicesEnabled`** parameters in the configuration file.
   - For using data encryption, check that **`ServicesEnabled`** is set to `sym`.

```
cat /etc/sysconfig/qat

ServicesEnabled=sym
POLICY=8
```

   - For using data compression, check that **`ServicesEnabled`** is set to `dc`.

```
cat /etc/sysconfig/qat

ServicesEnabled=dc
POLICY=8
```

   - For using both data encryption and compression, check that **`ServicesEnabled`** is set to `sym;dc`.

```
cat /etc/sysconfig/qat

ServicesEnabled=sym;dc
POLICY=8
QAT_USER=ceph
```

4. Configure the `limits.conf` file with `memlock` information.

```
sudo vim /etc/security/limits.conf

...
root - memlock 500000
ceph - memlock 500000
...
```

5. Enable the configurations in the `limits.conf` file.

```
sudo su -l $USER
```

6. Enable the QAT service.

```
systemctl enable qat
```

7. Restart the node.

```
systemctl reboot
```

8. Create the specification file and provide additional arguments to Podman for Ceph Object Gateway.

Note: You can use the following command to generate the device list:

```
--device /dev/vfio --device /dev/qat_adf_ctl $(for i in `ls /dev/vfio/* | grep
'dev' | grep -v ':'` ; do echo --device $i;
```

– For Red Hat Ceph Storage 9.0 use the following specification file example.

```
service_type: rgw
service_id: rgw_qat
placement:
  label: rgw
extra_container_args:
  - "-v /etc/group:/etc/group:ro"
  - "--group-add=keep-groups"
  - "--cap-add=SYS_ADMIN"
  - "--cap-add=SYS_PTRACE"
  - "--cap-add=IPC_LOCK"
  - "--security-opt seccomp=unconfined"
  - "--ulimit memlock=209715200:209715200"
  - "--device=/dev/qat_adf_ctl:/dev/qat_adf_ctl"
  - "--device=/dev/vfio/vfio:/dev/vfio/vfio"
  - "--device=/dev/vfio/333:/dev/vfio/333"
  - "--device=/dev/vfio/334:/dev/vfio/334"
  - "--device=/dev/vfio/335:/dev/vfio/335"
  - "--device=/dev/vfio/336:/dev/vfio/336"
  - "--device=/dev/vfio/337:/dev/vfio/337"
  - "--device=/dev/vfio/338:/dev/vfio/338"
  - "--device=/dev/vfio/339:/dev/vfio/339"
  - "--device=/dev/vfio/340:/dev/vfio/340"
  - "--device=/dev/vfio/341:/dev/vfio/341"
  - "--device=/dev/vfio/342:/dev/vfio/342"
  - "--device=/dev/vfio/343:/dev/vfio/343"
  - "--device=/dev/vfio/344:/dev/vfio/344"
  - "--device=/dev/vfio/345:/dev/vfio/345"
  - "--device=/dev/vfio/346:/dev/vfio/346"
  - "--device=/dev/vfio/347:/dev/vfio/347"
  - "--device=/dev/vfio/348:/dev/vfio/348"
  - "--device=/dev/vfio/349:/dev/vfio/349"
  - "--device=/dev/vfio/350:/dev/vfio/350"
  - "--device=/dev/vfio/351:/dev/vfio/351"
  - "--device=/dev/vfio/352:/dev/vfio/352"
  - "--device=/dev/vfio/353:/dev/vfio/353"
  - "--device=/dev/vfio/354:/dev/vfio/354"
  - "--device=/dev/vfio/355:/dev/vfio/355"
  - "--device=/dev/vfio/356:/dev/vfio/356"
  - "--device=/dev/vfio/357:/dev/vfio/357"
  - "--device=/dev/vfio/358:/dev/vfio/358"
  - "--device=/dev/vfio/359:/dev/vfio/359"
  - "--device=/dev/vfio/360:/dev/vfio/360"
  - "--device=/dev/vfio/361:/dev/vfio/361"
  - "--device=/dev/vfio/362:/dev/vfio/362"
  - "--device=/dev/vfio/363:/dev/vfio/363"
  - "--device=/dev/vfio/364:/dev/vfio/364"
  - "--device=/dev/vfio/365:/dev/vfio/365"
  - "--device=/dev/vfio/366:/dev/vfio/366"
  - "--device=/dev/vfio/367:/dev/vfio/367"
  - "--device=/dev/vfio/368:/dev/vfio/368"
  - "--device=/dev/vfio/369:/dev/vfio/369"
  - "--device=/dev/vfio/370:/dev/vfio/370"
  - "--device=/dev/vfio/371:/dev/vfio/371"
  - "--device=/dev/vfio/372:/dev/vfio/372"
  - "--device=/dev/vfio/373:/dev/vfio/373"
  - "--device=/dev/vfio/374:/dev/vfio/374"
  - "--device=/dev/vfio/375:/dev/vfio/375"
  - "--device=/dev/vfio/376:/dev/vfio/376"
  - "--device=/dev/vfio/377:/dev/vfio/377"
  - "--device=/dev/vfio/378:/dev/vfio/378"
  - "--device=/dev/vfio/379:/dev/vfio/379"
  - "--device=/dev/vfio/380:/dev/vfio/380"
  - "--device=/dev/vfio/381:/dev/vfio/381"
  - "--device=/dev/vfio/382:/dev/vfio/382"
  - "--device=/dev/vfio/383:/dev/vfio/383"
  - "--device=/dev/vfio/384:/dev/vfio/384"
  - "--device=/dev/vfio/385:/dev/vfio/385"
  - "--device=/dev/vfio/386:/dev/vfio/386"
  - "--device=/dev/vfio/387:/dev/vfio/387"
```

```
  - "--device=/dev/vfio/388:/dev/vfio/388"
  - "--device=/dev/vfio/389:/dev/vfio/389"
  - "--device=/dev/vfio/390:/dev/vfio/390"
  - "--device=/dev/vfio/391:/dev/vfio/391"
  - "--device=/dev/vfio/392:/dev/vfio/392"
  - "--device=/dev/vfio/393:/dev/vfio/393"
  - "--device=/dev/vfio/394:/dev/vfio/394"
  - "--device=/dev/vfio/395:/dev/vfio/395"
  - "--device=/dev/vfio/396:/dev/vfio/396"
  - "--device=/dev/vfio/devices/vfio0:/dev/vfio/devices/vfio0"
  - "--device=/dev/vfio/devices/vfio1:/dev/vfio/devices/vfio1"
  - "--device=/dev/vfio/devices/vfio2:/dev/vfio/devices/vfio2"
  - "--device=/dev/vfio/devices/vfio3:/dev/vfio/devices/vfio3"
  - "--device=/dev/vfio/devices/vfio4:/dev/vfio/devices/vfio4"
  - "--device=/dev/vfio/devices/vfio5:/dev/vfio/devices/vfio5"
  - "--device=/dev/vfio/devices/vfio6:/dev/vfio/devices/vfio6"
  - "--device=/dev/vfio/devices/vfio7:/dev/vfio/devices/vfio7"
  - "--device=/dev/vfio/devices/vfio8:/dev/vfio/devices/vfio8"
  - "--device=/dev/vfio/devices/vfio9:/dev/vfio/devices/vfio9"
  - "--device=/dev/vfio/devices/vfio10:/dev/vfio/devices/vfio10"
  - "--device=/dev/vfio/devices/vfio11:/dev/vfio/devices/vfio11"
  - "--device=/dev/vfio/devices/vfio12:/dev/vfio/devices/vfio12"
  - "--device=/dev/vfio/devices/vfio13:/dev/vfio/devices/vfio13"
  - "--device=/dev/vfio/devices/vfio14:/dev/vfio/devices/vfio14"
  - "--device=/dev/vfio/devices/vfio15:/dev/vfio/devices/vfio15"
  - "--device=/dev/vfio/devices/vfio16:/dev/vfio/devices/vfio16"
  - "--device=/dev/vfio/devices/vfio17:/dev/vfio/devices/vfio17"
  - "--device=/dev/vfio/devices/vfio18:/dev/vfio/devices/vfio18"
  - "--device=/dev/vfio/devices/vfio19:/dev/vfio/devices/vfio19"
  - "--device=/dev/vfio/devices/vfio20:/dev/vfio/devices/vfio20"
  - "--device=/dev/vfio/devices/vfio21:/dev/vfio/devices/vfio21"
  - "--device=/dev/vfio/devices/vfio22:/dev/vfio/devices/vfio22"
  - "--device=/dev/vfio/devices/vfio23:/dev/vfio/devices/vfio23"
  - "--device=/dev/vfio/devices/vfio24:/dev/vfio/devices/vfio24"
  - "--device=/dev/vfio/devices/vfio25:/dev/vfio/devices/vfio25"
  - "--device=/dev/vfio/devices/vfio26:/dev/vfio/devices/vfio26"
  - "--device=/dev/vfio/devices/vfio27:/dev/vfio/devices/vfio27"
  - "--device=/dev/vfio/devices/vfio28:/dev/vfio/devices/vfio28"
  - "--device=/dev/vfio/devices/vfio29:/dev/vfio/devices/vfio29"
  - "--device=/dev/vfio/devices/vfio30:/dev/vfio/devices/vfio30"
  - "--device=/dev/vfio/devices/vfio31:/dev/vfio/devices/vfio31"
  - "--device=/dev/vfio/devices/vfio32:/dev/vfio/devices/vfio32"
  - "--device=/dev/vfio/devices/vfio33:/dev/vfio/devices/vfio33"
  - "--device=/dev/vfio/devices/vfio34:/dev/vfio/devices/vfio34"
  - "--device=/dev/vfio/devices/vfio35:/dev/vfio/devices/vfio35"
  - "--device=/dev/vfio/devices/vfio36:/dev/vfio/devices/vfio36"
  - "--device=/dev/vfio/devices/vfio37:/dev/vfio/devices/vfio37"
  - "--device=/dev/vfio/devices/vfio38:/dev/vfio/devices/vfio38"
  - "--device=/dev/vfio/devices/vfio39:/dev/vfio/devices/vfio39"
  - "--device=/dev/vfio/devices/vfio40:/dev/vfio/devices/vfio40"
  - "--device=/dev/vfio/devices/vfio41:/dev/vfio/devices/vfio41"
  - "--device=/dev/vfio/devices/vfio42:/dev/vfio/devices/vfio42"
  - "--device=/dev/vfio/devices/vfio43:/dev/vfio/devices/vfio43"
  - "--device=/dev/vfio/devices/vfio44:/dev/vfio/devices/vfio44"
  - "--device=/dev/vfio/devices/vfio45:/dev/vfio/devices/vfio45"
  - "--device=/dev/vfio/devices/vfio46:/dev/vfio/devices/vfio46"
  - "--device=/dev/vfio/devices/vfio47:/dev/vfio/devices/vfio47"
  - "--device=/dev/vfio/devices/vfio48:/dev/vfio/devices/vfio48"
  - "--device=/dev/vfio/devices/vfio49:/dev/vfio/devices/vfio49"
  - "--device=/dev/vfio/devices/vfio50:/dev/vfio/devices/vfio50"
  - "--device=/dev/vfio/devices/vfio51:/dev/vfio/devices/vfio51"
  - "--device=/dev/vfio/devices/vfio52:/dev/vfio/devices/vfio52"
  - "--device=/dev/vfio/devices/vfio53:/dev/vfio/devices/vfio53"
  - "--device=/dev/vfio/devices/vfio54:/dev/vfio/devices/vfio54"
  - "--device=/dev/vfio/devices/vfio55:/dev/vfio/devices/vfio55"
  - "--device=/dev/vfio/devices/vfio56:/dev/vfio/devices/vfio56"
  - "--device=/dev/vfio/devices/vfio57:/dev/vfio/devices/vfio57"
  - "--device=/dev/vfio/devices/vfio58:/dev/vfio/devices/vfio58"
  - "--device=/dev/vfio/devices/vfio59:/dev/vfio/devices/vfio59"
  - "--device=/dev/vfio/devices/vfio60:/dev/vfio/devices/vfio60"
  - "--device=/dev/vfio/devices/vfio61:/dev/vfio/devices/vfio61"
  - "--device=/dev/vfio/devices/vfio62:/dev/vfio/devices/vfio62"
  - "--device=/dev/vfio/devices/vfio63:/dev/vfio/devices/vfio63"
networks:
- 172.17.8.0/24
spec:
  rgw_frontend_port: 8000
```

– For 9.0z1 and later, specify eithersworhwfor the **compression** parameter in the specification file to enable the QAT service for Ceph Object Gateway.

compression: sw

Specify to enable QAT software fall back mode.

compression: hw

Enables hardware QAT offload if QAT hardware and virtual functions (VFs) are present on the node.

```
service_type: rgw
service_id: rgw_qat_auto
placement:
  label: rgw
spec:
  rgw_frontend_port: 8000
  rgw_realm: REALM
  rgw_zone: ZONE
  rgw_zonegroup: ZONE_GROUP
  qat:
    compression: sw | hw
  ssl: true
  generate_cert: true
```

For example,

```
[root@host01 ~]# cat rgw-test-qat.yaml

service_type: rgw
service_id: rgw_qat_auto
placement:
  label: rgw
spec:
  rgw_frontend_port: 8000
  rgw_realm: REALM
  rgw_zone: ZONE
  rgw_zonegroup: ZONE_GROUP
  qat:
    compression: sw | hw
  ssl: true
  generate_cert: true
```

`cephadm` automatically sets **`qat_compressor_enabled`** to `true` when `qat:compression` is configured.

What to do next

When using hardware compression (`compression: hw`), verify that the QAT compressor is enabled in the `/etc/sysconfig/qat` file.
For example,

```
[root@host01 sysconfig]# cat qat

ServicesEnabled=dc

POLICY=8

QAT_USER=ceph
```

Enabling QAT-based encryption

Encrypt objects in Ceph Object Gateway by using the QAT-based encryption for OpenSSL.

Procedure

- To enable QAT-based encryption, edit the Ceph configuration file to make use of QAT-based crypto plug-in.

```
ceph config set client.rgw.rgw_qat plugin_crypto_accelerator crypto_qat
```

Enabling QAT-based compression

Compress objects in Ceph Object Gateway by using the tool class for QAT acceleration.

About this task

When using Red Hat Ceph Storage 9.0z1 or later, compression is enabled when creating the QAT specification file. Compression is enabled when creating the QAT specification file. For more information, see "Setting up the QAT service" on page 1009.

For Red Hat Ceph Storage 9.0, QAT-based compression has to be enabled manually by using this procedure.

Procedure

- To enable QAT-based compression, edit the Ceph configuration file to enable QAT support for compression.
  Depending on the `compression: sw|hw` configuration option, the `qat_compressor_enabled` field is configured automatically.

Server-side encryption

The Ceph Object Gateway supports server-side encryption of uploaded objects for the S3 application programming interface (API). Server-side encryption means that the S3 client sends data over HTTPS in its unencrypted form, and the Ceph Object Gateway stores that data in the Red Hat Ceph Storage cluster in encrypted form.

> **Important:** To use encryption, client requests MUST send requests over an SSL connection. Red Hat does not support S3 encryption from a client unless the Ceph Object Gateway uses SSL. However, for testing purposes, administrators can disable SSL during testing by setting the **`rgw_crypt_require_ssl`** configuration setting to `false` at runtime, using the **`ceph config set client.rgw`** command, and then restarting the Ceph Object Gateway instance. In a production environment, it might not be possible to send encrypted requests over SSL. In such a case, send requests by using HTTP with server-side encryption.

Red Hat does not support S3 object encryption of Static Large Object (SLO) or Dynamic Large Object (DLO).

Currently, none of the Server-Side Encryption (SSE) modes have implemented support for CopyObject.

Encryption key management

You can use the following options to manage encryption keys.

Customer-provided keys

When using customer-provided keys, the S3 client passes an encryption key along with each request to read or write encrypted data. It is the customer's responsibility to manage those keys. Customers must remember which key the Ceph Object Gateway used to encrypt each object.

Ceph Object Gateway implements the customer-provided key behavior in the S3 API according to the Amazon server-side encryption with customer-provided keys (SSE-C) specification. The Ceph Object Gateway does not store the encryption key and uses it only for encryption and decryption operations associated with the request.

Because the customer handles the key management and the S3 client passes keys to the Ceph Object Gateway, the Ceph Object Gateway requires no special configuration to support this encryption mode.

Key management service

When using a key management service, the secure key management service stores the keys and the Ceph Object Gateway retrieves them on demand to serve requests to encrypt or decrypt data.

Ceph Object Gateway implements the key management service behavior in the S3 API according to the Amazon server-side encryption with AWS Key Management Service (SSE-KMS) specification. You can configure the Ceph Object Gateway to use an external key management service for SSE-KMS operations.

For IBM Guardium Key Lifecycle Manager (GKLM), see Configuring the Ceph Object Gateway to use IBM GKLM.

For Thales CipherTrust Manager, see Configuring the Ceph Object Gateway to use Thales CipherTrust Manager.

Important: Currently, the tested key management implementations are:

- HashiCorp Vault
- IBM Guardium Key Lifecycle Manager (GKLM)
- Thales CipherTrust Manager
- OpenStack Barbican (Technology Preview)

Important: Technology Preview features are not supported with production service level agreements (SLAs), might not be functionally complete, and does not recommend using them for production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

SSE-S3

When you use server-side encryption with Amazon S3 managed keys (SSE-S3), a vault stores the keys. Ceph automatically creates and deletes the keys and retrieves them as required to serve requests to encrypt or decrypt data.

Ceph Object Gateway implements the SSE-S3 behavior in the S3 API according to the Amazon SSE-S3 specification.

Amazon SSE-C

For Amazon SSE-C, the client provides the encryption key with each request. The Ceph Object Gateway does not store the encryption key and uses it only for encryption and decryption operations associated with the request.

Amazon SSE-KMS

For Amazon SSE-KMS, the Ceph Object Gateway retrieves encryption keys from a supported key management service.

For information about the supported key management implementations and configuring the Ceph Object Gateway to use an external key management service, see Key management service.

Encryption at rest and in transit

Understand how Red Hat Ceph Storage Object Gateway uses encryption at rest and in transit to protect sensitive data.

Data security is a critical priority in today's digital landscape. Red Hat Ceph Storage Object Gateway provides options for implementing encryption at rest and in transit, helping safeguard data whether stored or transferred. These mechanisms often rely on key management solutions such as IBM Guardium Key Lifecycle Manager (GKLM) to manage and store encryption keys securely. This topic explains how these encryption methods work and the benefits of using them together.

Encryption at rest with Ceph Object Gateway

Ceph Object Gateway offers robust encryption for data that is stored on disk, supporting granular object-level encryption. It is fully compatible with S3 encryption semantics and integrates with key management systems such as IBM Guardium Key Lifecycle Manager, Hashicorp Vault, and Thales CipherTrust.

Encryption at rest is essential for safeguarding stored data from unauthorized access by encrypting it while it resides on disk. Ceph facilitates this process through seamless integration with external Key Management Systems (KMS) like IBM Guardium Key Lifecycle Manager. By doing so, Ceph can automatically retrieve encryption keys from the key manager, helping ensure that objects are encrypted as they are uploaded. This integration provides data security throughout its lifecycle while centralizing key management, simplifying key rotation, auditing, and overall compliance efforts.

Encryption in transit with Ceph Object Gateway

Data can travel through various unknown, untrusted, or both unknown and untrusted networks as it moves between the object storage and the application. Encrypting the data as it travels over the network can effectively prevent tampering and helps ensure the security of the data, regardless of its path.

In Ceph Object, encryption in transit is typically achieved by using TLS/SSL encryption, which secures communication between clients and the Ceph Object Gateway. After the Object Gateways are configured, they provide an S3 RESTful API over HTTPS. The RESTful API encrypts and protects all data that is exchanged between clients and the object storage system.

From an architectural standpoint, SSL encryption can be applied at multiple layers: at the load balancer (ingress service), Ceph Object Gateway level, or both. For full client-to-Ceph Object Gateway HTTPS encryption, the two approaches for load balancer are *pass-through* and *reencrypt*.

Pass-through

Encrypted traffic is forwarded at level 3 directly to the Object Gateway service.

Reencrypt

The load balancer decrypts and reencrypts traffic before sending it to the Ceph Object Gateway.

Server-side encryption (SSE) options with Ceph Object Gateway

Ceph Object Gateway supports various server-side encryption (SSE) methods to secure data stored in its object storage. These methods allow Ceph to automatically encrypt data as it is written, helping ensure that data at rest is protected without requiring the user to manage encryption independently. See the following for the supported SSE methods and their differences.

SSE-C (server-side encryption with customer-provided keys)

SSE-C allows clients to manage their own encryption keys. The client provides Ceph with a key for each object request in this mode. Ceph then uses this key to encrypt the data before storing it and must decrypt it using the same key when data is requested.
This method suits users who require complete control over their encryption keys but are willing to manage them securely.

SSE-S3 (server-side encryption with S3 managed keys)

Red Hat Ceph Storage manages encryption and decryption in this configuration. The encryption and decryption are done by using external KMS to handle keys securely. SSE-S3 provides transparent encryption, automatically encrypting data when it is stored and decrypted when retrieved without requiring extra effort from the client. SSE-S3 is only supported with the HashiCorp Vault Integration Program.

SSE-KMS (server-side encryption with Key Management Service)

SSE-KMS provides server-side encryption where an external Key Management Service (KMS) keys, such as IBM Guardium Key Lifecycle Manager, manage keys. When using this method, Ceph communicates with a KMS to retrieve the encryption keys for encrypting and decrypting data. The keys are stored and managed externally, which can simplify compliance with encryption standards.

"SSE options advantages and disadvantages" on page 1015 compares and contrasts the different options.

Table 114: SSE options advantages and disadvantages

	SSE-C	SSE-KMS	SSE-S3
Key management	Handled by the client. The key is provided on every read-and-write request.	Handled by the KMS. For example, IBM GKLM.	SSE-S3.
Advantages	Full control over the keys remains with the user, allowing for more custom security policies.	Centralized and secure key management. The KMS takes care of the entire lifecycle of the encryption keys, including auditing, rotation, and deletion.	Provides transparent encryption, automatically encrypting data when it is stored and decrypted when retrieved without requiring extra effort from the client

	SSE-C	SSE-KMS	SSE-S3
Disadvantages	The user bears the burden of key management. Losing the key results in the permanent loss of data.	Requires integration with an external KMS, which can add complexity to the setup.	SSE-S3 is only supported by the Hashicorp Vault Integration Program.

Setting the default encryption for an existing S3 bucket

As a storage administrator, you can set the default encryption for an existing Amazon S3 bucket so that all objects are encrypted when they are stored in a bucket. You can use Bucket Encryption APIs to support server-side encryption with Amazon S3-managed keys (SSE-S3) or Amazon KMS customer primary keys (SSE-KMS).

You can manage default encryption for an existing Amazon S3 bucket using the `PutBucketEncryption` API. All files uploaded to this bucket will have this encryption by defining the default encryption at the bucket level.

Prerequisites

- A running Red Hat Ceph Storage
- Installation of the Ceph Object Gateway.
- An S3 bucket created.
- An S3 user created with user access.
- Access to a Ceph Object Gateway client with the AWS CLI package installed.

Procedure

1. Create a JSON file for the encryption configuration:

   Example

   ```
   [user@client ~]$ vi bucket-encryption.json
   ```

2. Add the encryption configuration rules to the file:

   Example

   ```
   {
           "Rules": [
           {
               "ApplyServerSideEncryptionByDefault": {
                       "SSEAlgorithm": "AES256"
               }
          }
       ]
   }
   ```

3. Set the default encryption for the bucket:

   Syntax

   ```
   aws --endpoint-url=RADOSGW_ENDPOINT_URL:PORT s3api put-bucket-encryption --bucket
   BUCKET_NAME --server-side-encryption-configuration file://
   PATH_TO_BUCKET_ENCRYPTION_CONFIGURATION_FILE/BUCKET_ENCRYPTION_CONFIGURATION_FILE.json
   ```

   Example

   ```
   [user@client ~]$ aws --endpoint-url=http://host01:80 s3api put-bucket-encryption --
   bucket testbucket --server-side-encryption-configuration file://bucket-encryption.json
   ```

Verification

- Retrieve the bucket encryption configuration for the bucket:

  Syntax

```
aws --endpoint-url=RADOSGW_ENDPOINT_URL:PORT s3api get-bucket-encryption --bucket
BUCKET_NAME
```

Example

```
[user@client ~]$  aws --profile ceph --endpoint=http://host01:80 s3api get-bucket-
encryption --bucket testbucket

{
    "ServerSideEncryptionConfiguration": {
        "Rules": [
            {
                "ApplyServerSideEncryptionByDefault": {
                    "SSEAlgorithm": "AES256"
                }
            }
        ]
    }
}
```

Note: If the bucket does not have a default encryption configuration, the `get-bucket-encryption` command returns `ServerSideEncryptionConfigurationNotFoundError`.

Deleting the default bucket encryption

You can delete the default bucket encryption for a specified bucket using the **`s3api delete-bucket-encryption`** command.

Prerequisites

- A running Red Hat Ceph Storage
- Installation of the Ceph Object Gateway.
- An S3 bucket created.
- An S3 user created with user access.
- Access to a Ceph Object Gateway client with the AWS CLI package installed.

Procedure

- Delete a bucket encryption configuration:

  Syntax

```
aws --endpoint-url=RADOSGW_ENDPOINT_URL:PORT s3api delete-bucket-encryption --bucket
BUCKET_NAME
```

  Example

```
[user@client ~]$ aws --endpoint-url=http://host01:80 s3api delete-bucket-encryption --
bucket testbucket
```

Verification

- Retrieve the bucket encryption configuration for the bucket:

  Syntax

```
aws --endpoint-url=RADOSGW_ENDPOINT_URL:PORT s3api get-bucket-encryption --bucket
BUCKET_NAME
```

  Example

```
[user@client ~]$  aws --endpoint=http://host01:80 s3api get-bucket-encryption --bucket
testbucket

An error occurred (ServerSideEncryptionConfigurationNotFoundError) when calling the
GetBucketEncryption operation:
The server side encryption configuration was not found
```

Server-side encryption requests

In a production environment, clients often contact the Ceph Object Gateway through a proxy. This proxy is referred to as a load balancer because it connects to multiple Ceph Object Gateways. When the client sends requests to the Ceph Object Gateway, the load balancer routes those requests to the multiple Ceph Object Gateways, thus distributing the workload.

In this type of configuration, it is possible that SSL terminations occur both at a load balancer and between the load balancer and the multiple Ceph Object Gateways. Communication occurs using HTTP only.

Configuring server-side encryption

You can set up server-side encryption to send requests to the Ceph Object Gateway using HTTP, in cases where it might not be possible to send encrypted requests over SSL.

This procedure uses HAProxy as proxy and load balancer.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to all nodes in the storage cluster.
- Installation of the Ceph Object Gateway software.
- Installation of the HAProxy software.

Procedure

1. Edit the `haproxy.cfg` file:

   Example

   ```
   frontend http_web *:80
       mode http
       default_backend rgw

   frontend rgw-https
     bind *:443 ssl crt /etc/ssl/private/example.com.pem
     default_backend rgw

   backend rgw
       balance roundrobin
       mode http
       server  rgw1 10.0.0.71:8080 check
       server  rgw2 10.0.0.80:8080 check
   ```

2. Comment out the lines that allow access to the `http` front end and add instructions to direct HAProxy to use the `https` front end instead:

   Example

   ```
   #     frontend http_web *:80
   #     mode http
   #     default_backend rgw

   frontend rgw-https
     bind *:443 ssl crt /etc/ssl/private/example.com.pem
     http-request set-header X-Forwarded-Proto https if { ssl_fc }
     http-request set-header X-Forwarded-Proto https
   # here we set the incoming HTTPS port on the load balancer (eg : 443)
     http-request set-header X-Forwarded-Port 443
     default_backend rgw

   backend rgw
   ```

```
    balance roundrobin
    mode http
    server  rgw1 10.0.0.71:8080 check
    server  rgw2 10.0.0.80:8080 check
```

3. Set the `rgw_trust_forwarded_https` option to `true`:

   Example

   ```
   [ceph: root@host01 /]# ceph config set client.rgw rgw_trust_forwarded_https true
   ```

4. Enable and start HAProxy:

   ```
   [root@host01 ~]# systemctl enable haproxy
   [root@host01 ~]# systemctl start haproxy
   ```

Reference

For more information, see the following:

- "High availability service" on page 829
- Installing

Using HashiCorp Vault

As a storage administrator, you can securely store keys, passwords, and certificates in the HashiCorp Vault for use with the Ceph Object Gateway. The HashiCorp Vault provides a secure key management service for server-side encryption used by the Ceph Object Gateway.

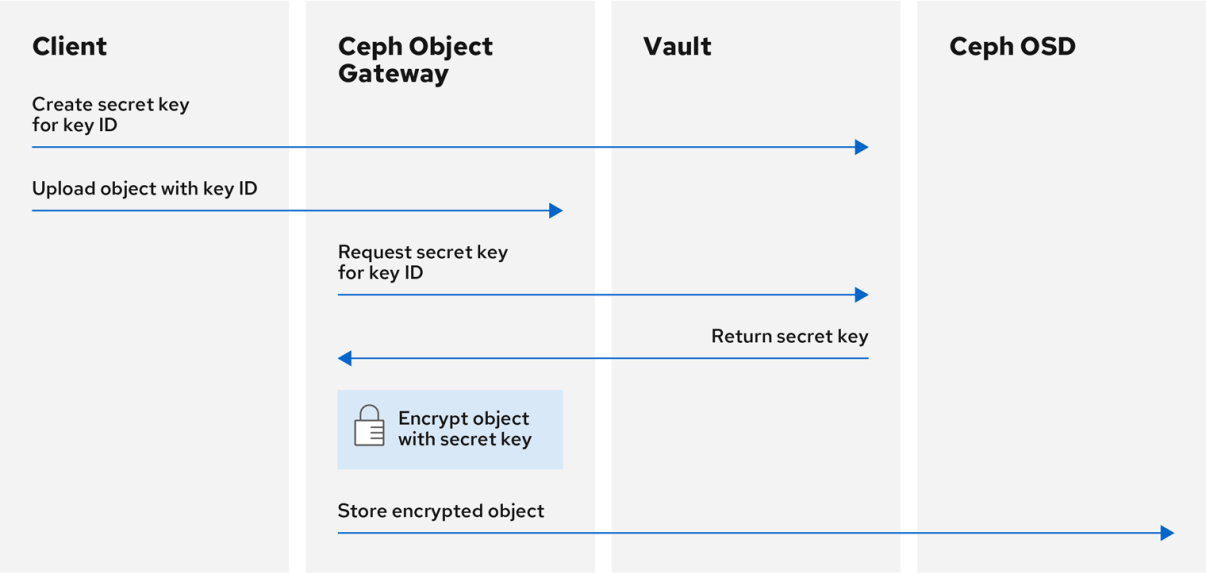


Figure 91: Ceph Vault integration

The basic workflow:

1. The client requests the creation of a secret key from the Vault based on an object's key ID.
2. The client uploads an object with the object's key ID to the Ceph Object Gateway.
3. The Ceph Object Gateway then requests the newly created secret key from the Vault.
4. The Vault replies to the request by returning the secret key to the Ceph Object Gateway.
5. Now the Ceph Object Gateway can encrypt the object using the new secret key.
6. After encryption is done the object is then stored on the Ceph OSD.

Important: Red Hat works with our technology partners to provide this documentation as a service to our customers. However, Red Hat does not provide support for this product. If you need technical assistance for this product, then contact HashiCorp for support.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway software.
- Installation of the HashiCorp Vault software.

Reference

For more information, see *Install Vault*.

Secret engines for Vault

The HashiCorp Vault provides several secret engines to generate, store, or encrypt data. The application programming interface (API) sends data calls to the secret engine requesting an action on that data, and the secret engine returns a result of that action request.

The Ceph Object Gateway supports two of the HashiCorp Vault secret engines:

- "Key/Value version 2" on page 1020
- "Transit" on page 1020

Important: The secret engines can be configured at any time but the engines are not supported simultaneously.

Key/Value version 2

The Key/Value secret engine stores random secrets within the Vault, on disk. With version 2 of the `kv` engine, a key can have a configurable number of versions. The default number of versions is 10. Deleting a version does not delete the underlying data, but marks the data as deleted, allowing deleted versions to be undeleted. You can use the API endpoint or the **`destroy`** command to permanently remove a version's data. To delete all versions and metadata for a key, you can use the **`metadata`** command or the API endpoint. The key names must be strings, and the engine converts non-string values into strings when using the command line interface. To preserve non-string values, provide a JSON file or use the HTTP application programming interface (API).

Note: For access control list (ACL) policies, the Key/Value secret engine recognizes the distinctions between the `create` and `update` capabilities.

For more information about the Vault's project site, see the *KV Secrets Engine* documentation.

Transit

The Transit secret engine performs cryptographic functions on in-transit data. The Transit secret engine can generate hashes, can be a source of random bytes, and can also sign and verify data. The Vault does not store data when using the Transit secret engine. The Transit secret engine supports key derivation, by allowing the same key to be used for multiple purposes. Also, the transit secret engine supports key versioning.

"Supported key types" on page 1020 lists the supported key types for the transit secret engine.

Table 115: Supported key types

Key type	Description
`aes128-gcm96`	– AES-GCM with a 128-bit AES key and a 96-bit nonce – Supports encryption, decryption, key derivation, and convergent encryption

Key type	Description
`aes256-gcm96`	– AES-GCM with a 256-bit AES key and a 96-bit nonce – Supports encryption, decryption, key derivation, and convergent encryption (default)
`chacha20-poly1305`	– ChaCha20-Poly1305 with a 256-bit key – Supports encryption, decryption, key derivation, and convergent encryption
`ed25519`	– Ed25519 – Supports signing, signature verification, and key derivation
`ecdsa-p256`	– ECDSA using curve P-256 – Supports signing and signature verification
`ecdsa-p384`	– ECDSA using curve P-384 – Supports signing and signature verification
`ecdsa-p521`	– ECDSA using curve P-521 – Supports signing and signature verification
`rsa-2048`	– 2048-bit RSA key – Supports encryption, decryption, signing, and signature verification
`rsa-3072`	– 3072-bit RSA key – Supports encryption, decryption, signing, and signature verification
`rsa-4096`	– 4096-bit RSA key – Supports encryption, decryption, signing, and signature verification

For more information about the Vault's project site, see the *Transit Secrets Engine* documentation.

Authentication for Vault

The HashiCorp Vault supports several types of authentication mechanisms. The Ceph Object Gateway currently supports the Vault Agent method.

The Ceph Object Gateway uses the `rgw_crypt_vault_auth`, and `rgw_crypt_vault_addr` options to configure the use of the HashiCorp Vault.

> **Important:** Red Hat Ceph Storage supports the usage of Vault Agent as the authentication method for containers and the usage of token authentication is not supported on containers.

Vault Agent

The Vault Agent is a daemon that runs on a client node and provides client-side caching, along with token renewal. The Vault Agent typically runs on the Ceph Object Gateway node. Run the Vault Agent and refresh the token file. When the Vault Agent is used in this mode, you can use file system permissions to restrict who has access to the usage of tokens. Also, the Vault Agent can act as a proxy server, that is, Vault will add a token

when required and add it to the requests passed to it before forwarding them to the actual server. The Vault Agent can still handle token renewal just as it would when storing a token in the Filesystem. It is required to secure the network that Ceph Object Gateways uses to connect with the Vault Agent, for example, the Vault Agent listens to only the localhost.

For more information, see the HashiCorp Developer Vault Agent documentation.

Namespaces for Vault

Using HashiCorp Vault as an enterprise service provides centralized management for isolated namespaces that teams within an organization can use.

These isolated namespace environments are known as *tenants*, and teams within an organization can utilize these tenants to isolate their policies, secrets, and identities from other teams. The namespace features of Vault help support secure multi-tenancy from within a single infrastructure.

Reference

For more information, see the following:

- *Vault Enterprise Namespaces*

Transit engine compatibility support

There is compatibility support for the previous versions of Ceph which used the Transit engine as a simple key store.

You can use the `compat` option in the Transit engine to configure the compatibility support. You can disable previous support with the following command:

Example

```
[ceph: root@host03 /]# ceph config set client.rgw rgw_crypt_vault_secret_engine transit
compat=0
```

Note: This is the default for future versions and you can use the current version for new installations.

The normal default with the current version is:

Example

```
[ceph: root@host03 /]# ceph config set client.rgw rgw_crypt_vault_secret_engine transit
compat=1
```

This enables the new engine for newly created objects and still allows the old engine to be used for the old objects. To access old and new objects, the Vault token must have both the old and new transit policies.

You can force use only the old engine with the following command:

Example

```
[ceph: root@host03 /]# ceph config set client.rgw rgw_crypt_vault_secret_engine transit
compat=2
```

This mode is selected by default if the Vault ends in `export/encryption-key`.

Important: After configuring the `client.rgw` options, you need to restart the Ceph Object Gateway daemons for the new values to take effect.

Reference

For more information, see the following:

- *Vault Agent*

Creating token policies for Vault

A token policy specifies the powers that all Vault tokens have. One token can have multiple policies. Use the required policy for the configuration.

Prerequisites

- A running Red Hat Ceph Storage cluster.

- Installation of the HashiCorp Vault software.
- Root-level access to the HashiCorp Vault node.

Procedure

1. Create a token policy:
   a. For the Key/Value secret engine:

      Example

      ```
      [root@vault ~]# vault policy write rgw-kv-policy -<<EOF
       path "secret/data/*" {
         capabilities = ["read"]
       }
      EOF
      ```

   b. For the Transit engine:

      Example

      ```
      [root@vault ~]# vault policy write rgw-transit-policy -<<EOF
        path "transit/keys/*" {
          capabilities = [ "create", "update" ]
          denied_parameters = {"exportable" = [], "allow_plaintext_backup" = [] }
        }

        path "transit/keys/*" {
          capabilities = ["read", "delete"]
        }

        path "transit/keys/" {
          capabilities = ["list"]
        }

        path "transit/keys/+/rotate" {
          capabilities = [ "update" ]
        }

        path "transit/*" {
          capabilities = [ "update" ]
        }
      EOF
      ```

      Note: If you have used the Transit secret engine on an older version of Ceph, the token policy is:

      Example

      ```
      [root@vault ~]# vault policy write old-rgw-transit-policy -<<EOF
        path "transit/export/encryption-key/*" {
          capabilities = ["read"]
        }
      EOF
      ```

If you are using both SSE-KMS and SSE-S3, you should point each to separate containers. You could either use separate Vault instances or separately mount transit instances or different branches under a common transit point. If you are not using separate Vault instances, you can point SSE-KMS or SSE-S3 to serparate containers using `rgw_crypt_vault_prefix` and `rgw_crypt_sse_s3_vault_prefix`. When granting Vault permissions to SSE-KMS bucket owners, you should not give them permission to SSE-S3 keys; only Ceph should have access to the SSE-S3 keys.

Configuring the Ceph Object Gateway to use SSE-S3 with Vault

To configure the Ceph Object Gateway to use the HashiCorp Vault with SSE-S3 for key management, it must be set as the encryption key store.

For SSE-S3 encryption with Vault, the Ceph Object Gateway supports the Vault Transit secret engine.

For more information, see:

- “Secret engines for Vault” on page 1020

- “Authentication for Vault” on page 1021

> **Note:** SSE-S3 supports only the Vault Transit secret engine. The Vault Key/Value (KV) secret engine is not supported for SSE-S3.

Before you begin Before you begin, make sure that you have the following prerequisites in place:

- A running, and healthy Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway software.
- Root-level access to a Ceph Object Gateway node.

Procedure

1. Log into the Cephadm shell.
   For example,

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Enable Vault as the secrets engine to retrieve SSE-S3 encryption keys.

   ```
   ceph config set client.rgw rgw_crypt_sse_s3_backend vault
   ```

3. Set the authentication method to use with SSE-S3 and Vault, by using one of the following methods:
   - **Method 1: Configuring with `agent` authentication**

     ```
     ceph config set client.rgw rgw_crypt_sse_s3_vault_auth agent
     ceph config set client.rgw rgw_crypt_sse_s3_vault_addr http://
     VAULT_AGENT:VAULT_AGENT_PORT
     ```

     For example,

     ```
     [ceph: root@host01 ~]# ceph config set client.rgw rgw_crypt_sse_s3_vault_auth
     agent
     [ceph: root@host01 ~]# ceph config set client.rgw rgw_crypt_sse_s3_vault_addr
     http://vaultagent:8100
     ```

     - Customize the policy as per your use case to set up a Vault agent.
     - Get the `role-id`.

       ```
       vault read auth/approle/role/rgw-ap/role-id -format=json | \ jq -r .rgw-ap-
       role-id > PATH_TO_FILE
       ```

     - Get the `secret-id`.

       ```
       vault read auth/approle/role/rgw-ap/role-id -format=json | \ jq -r .rgw-ap-
       secret-id > PATH_TO_FILE
       ```

     - Create the configuration for the Vault agent.
       For example,

       ```
       pid_file = "/run/rgw-vault-agent-pid"
       auto_auth {
         method "AppRole" {
           mount_path = "auth/approle"
           config = {
             role_id_file_path ="/usr/local/etc/vault/.rgw-ap-role-id"
             secret_id_file_path ="/usr/local/etc/vault/.rgw-ap-secret-id"
             remove_secret_id_file_after_reading ="false"
           }
         }
       }
       cache {
         use_auto_auth_token = true
       }
       listener "tcp" {
         address = "127.0.0.1:8100"
       ```

```
  tls_disable = true
}
vault {
  address = "https://vaultserver:8200"
}
```

- Use systemctl to run the persistent daemon.

```
/usr/local/bin/vault agent -config=/usr/local/etc/vault/rgw-agent.hcl
```

- A token file is populated with a valid token when the Vault agent runs.

- **Method 2: Configuring with `token` authentication**

```
ceph config set client.rgw rgw_crypt_sse_s3_vault_auth token
ceph config set client.rgw rgw_crypt_sse_s3_vault_token_file PATH_TO_TOKEN_FILE
ceph config set client.rgw rgw_crypt_sse_s3_vault_addr http://
VAULT_SERVER:VAULT_PORT
```

For example,

```
[ceph: root@host01 ~]# ceph config set client.rgw rgw_crypt_sse_s3_vault_auth
token
[ceph: root@host01 ~]# ceph config set client.rgw
rgw_crypt_sse_s3_vault_token_file /run/.rgw-vault-token
[ceph: root@host01 ~]# ceph config set client.rgw rgw_crypt_sse_s3_vault_addr
http://vaultserver:8200
```

> **Note:** For security reasons, the path to the token file should only be readable by the Ceph Object Gateway.

4. Set the Vault secret engine to `transit` to retrieve SSE-S3 encryption keys.

```
ceph config set client.rgw
```

```
rgw_crypt_sse_s3_vault_secret_engine transit
```

5. Optional: Configure the Ceph Object Gateway to access Vault within a particular namespace to retrieve the encryption keys:

Example

```
[ceph: root@host01 /]# ceph config set client.rgw rgw_crypt_sse_s3_vault_namespace
company/testnamespace1
```

> **Note:** Vault namespaces allow teams to operate within isolated environments known as tenants. The Vault namespaces feature is only available in the Vault Enterprise version.

6. Optional: Restrict access to a particular subset of the Vault secret space by setting a URL path prefix, where the Ceph Object Gateway retrieves the encryption keys from.

```
ceph config set client.rgw
```

```
rgw_crypt_sse_s3_vault_prefix /v1/transit
```

Assuming the domain name of the Vault server is `vaultserver`, the Ceph Object Gateway will fetch encrypted transit keys from the following URL:

```
http://vaultserver:8200/v1/transit
```

7. Configure the settings to authenticate with Vault to use custom SSL certification.

```
ceph config set client.rgw rgw_crypt_sse_s3_vault_verify_ssl true
ceph config set client.rgw rgw_crypt_sse_s3_vault_ssl_cacert PATH_TO_CA_CERTIFICATE
ceph config set client.rgw rgw_crypt_sse_s3_vault_ssl_clientcert
PATH_TO_CLIENT_CERTIFICATE
ceph config set client.rgw rgw_crypt_sse_s3_vault_ssl_clientkey PATH_TO_PRIVATE_KEY
```

   For example,

```
[ceph: root@host01 /]# ceph config set client.rgw rgw_crypt_sse_s3_vault_verify_ssl
true
[ceph: root@host01 /]# ceph config set client.rgw rgw_crypt_sse_s3_vault_ssl_cacert /
etc/ceph/vault.ca
[ceph: root@host01 /]# ceph config set client.rgw
rgw_crypt_sse_s3_vault_ssl_clientcert /etc/ceph/vault.crt
[ceph: root@host03 /]# ceph config set client.rgw
rgw_crypt_sse_s3_vault_ssl_clientkey /etc/ceph/vault.key
```

8. Restart the Ceph Object Gateway daemons.
   a. Restart the Ceph Object Gateway on an individual node in the storage cluster.

```
systemctl restart ceph-CLUSTER_ID@SERVICE_TYPE.ID.service
```

      For example,

```
[root@host01 ~]# systemctl restart ceph-c4b34c6f-8365-11ba-
dc31-529020a7702d@rgw.realm.zone.host01.gwasto.service
```

   b. Restart the Ceph Object Gateways on all nodes in the storage cluster.

```
ceph orch restart SERVICE_TYPE
```

      For example,

```
[ceph: root@host01 /]# ceph orch restart rgw
```

Configuring the Ceph Object Gateway to use Thales CipherTrust Manager

Configure the Ceph Object Gateway to retrieve encryption keys from Thales CipherTrust Manager by using the Key Management Interoperability Protocol (KMIP).

Before you begin

Make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- A deployed Ceph Object Gateway service.
- Thales CipherTrust Manager configured for KMIP access.
- A client certificate and private key that are trusted by Thales CipherTrust Manager.
- Network connectivity between the Ceph Object Gateway and the Thales CipherTrust Manager server.

About this task

To use Server-Side Encryption with customer-managed keys (SSE-KMS), configure the Ceph Object Gateway to communicate with Thales CipherTrust Manager by using KMIP. The Ceph Object Gateway uses a trusted client certificate to authenticate with the Thales CipherTrust Manager server and retrieve encryption keys for object encryption operations.

Red Hat Ceph Storage supports KMIP-based key management for SSE-KMS.

Supported KMIP-compatible key management implementations include:

- IBM Security Guardium Key Lifecycle Manager (GKLM)
- Thales CipherTrust Manager

Procedure

1. Log in to the **cephadm** shell.
   Example:

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Configure KMIP as the encryption key management backend.
   Syntax:

   ```
   ceph config set client.rgw.RGW_SERVICE rgw_crypt_s3_kms_backend kmip
   ```

   Example:

   ```
   [ceph: root@host01 ~]# ceph config set client.rgw.client-traffic \
   rgw_crypt_s3_kms_backend kmip
   ```

3. Configure the Thales CipherTrust Manager KMIP server address.
   Syntax:

   ```
   ceph config set client.rgw.RGW_SERVICE rgw_crypt_kmip_addr THALES_SERVER:5696
   ```

   Example:

   ```
   [ceph: root@host01 ~]# ceph config set client.rgw.client-traffic \
   rgw_crypt_kmip_addr 192.168.122.80:5696
   ```

4. Configure the path to the KMIP client private key.
   Syntax:

   ```
   ceph config set client.rgw.RGW_SERVICE rgw_crypt_kmip_client_key KEY_FILE_PATH
   ```

   Example:

   ```
   [ceph: root@host01 ~]# ceph config set client.rgw.client-traffic \
   rgw_crypt_kmip_client_key /var/run/ceph/rgw.example.com.key
   ```

5. Configure the path to the KMIP client certificate.
   Syntax:

   ```
   ceph config set
   client.rgw.RGW_SERVICE rgw_crypt_kmip_client_cert CERTIFICATE_FILE_PATH
   ```

   Example:

   ```
   [ceph: root@host01 ~]# ceph config set client.rgw.client-traffic \
   rgw_crypt_kmip_client_cert /var/run/ceph/rgw.example.com.cer
   ```

6. Copy the client certificate and private key to each host that runs the Ceph Object Gateway service.

   > **Important:** Ensure that the certificate and key files are accessible to the Object Gateway container and owned by the ceph user.

   Example:

   ```
   [root@host01 ~]# ls -l /var/run/ceph/

   -rw-------. 1 ceph ceph 3243 rgw.example.com.key
   -rw-r--r--. 1 ceph ceph 5930 rgw.example.com.cer
   ```

7. Configure SSL requirements for the KMIP connection.
   Syntax:

   ```
   ceph config set client.rgw rgw_crypt_require_ssl VALUE
   ```

Set **`rgw_crypt_require_ssl`** to one of the following values:

- `true` to require SSL for all KMIP connections. This setting is recommended for production environments.
- `false` to allow encryption keys and secrets to be sent over an unencrypted HTTP connection.

Example:

```
[ceph: root@host01 ~]# ceph config set client.rgw \
rgw_crypt_require_ssl true
```

8. Restart the Ceph Object Gateway service.

```
ceph orch restart rgw.RGW_SERVICE
```

9. Verify that the Ceph Object Gateway can communicate with Thales CipherTrust Manager.

```
ceph log last 20
```

Review the output for KMIP connection errors. A successful connection does not generate KMIP-related error messages.

Result

The Ceph Object Gateway is configured to retrieve encryption keys from Thales CipherTrust Manager through KMIP.

What to do next

Verify server-side encryption by uploading an object with SSE-KMS enabled.

For information about enabling bucket encryption, see Creating Ceph Object Gateway buckets.

Creating a key using the `kv` engine

Configure the HashiCorp Vault Key/Value secret engine (`kv`) so you can create a key for use with the Ceph Object Gateway. Secrets are stored as key-value pairs in the `kv` secret engine.

> **Important:** Keys for server-side encryption must be 256-bits long and encoded using `base64`.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- `root` or `sudo` access
- Ceph Object Gateway installed
- Installation of the HashiCorp Vault software.
- Root-level access to the HashiCorp Vault node.

Procedure

1. Enable the Key/Value version 2 secret engine:

   Example

```
vault secrets enable -path secret kv-v2
```

2. Create a new key:

   Syntax

```
vault kv put secret/PROJECT_NAME/BUCKET_NAME key=$(openssl rand -base64 32)
```

   Example

```
[root@vault ~]# vault kv put secret/myproject/mybucketkey key=$(openssl rand -base64
32)

====== Metadata ======
Key              Value
---              -----
created_time     2020-02-21T17:01:09.095824999Z
deletion_time    n/a
destroyed        false
version          1
```

Creating a key using the transit engine

Configure the HashiCorp Vault Transit secret engine (`transit`) so you can create a key for use with the Ceph Object Gateway. Creating keys with the Transit secret engine must be exportable to be used for server-side encryption with the Ceph Object Gateway.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Installation of the HashiCorp Vault software.
- Root-level access to the HashiCorp Vault node.

Procedure

1. Enable the Transit secret engine.

   ```
   vault secrets enable transit
   ```

   For example,

   ```
   [root@vault ~]# vault secrets enable transit
   ```

2. Create a new exportable key

   ```
   vault write -f transit/keys/BUCKET_NAME exportable=true
   ```

   > **Note:** By default this command creates a `aes256-gcm96` type key.

   For example,

   ```
   [root@vault ~]# vault write -f transit/keys/mybucketkey exportable=true
   ```

   a. Enable key rotation.

      ```
      vault write -f transit/keys/BUCKET_NAME/rotate exportable=true
      ```

      For example,

      ```
      [root@vault ~]# vault write -f transit/keys/mybucketkey/rotate exportable=true
      ```

   b. Specify the duration for key rotation.

      ```
      vault write -f transit/keys/BUCKET_NAME/config auto_rotate_period=DURATION
      ```

      For example,

      ```
      [root@vault ~]# vault write -f transit/keys/mybucketkey/config
      auto_rotate_period=30d
      ```

      In this example, **30d** specifies that the key is rotated after 30 days. To specify the key rotation duration in hours, use **auto_rotate_period=1h**. **1h** specifies that the key rotates every 1 hour.

   c. Verify that the key rotation is successful by checking the value of **latest_version** parameter. The value must be incremented.

```
vault read transit/export/encryption-key/BUCKET_NAME
```

For example,

```
[root@vault ~]# vault read transit/export/encryption-key/mybucketkey
```

3. Verify the creation of the key.

> **Note:** Providing the full key path, including the key version, is required.

```
vault read transit/export/encryption-key/BUCKET_NAME/VERSION_NUMBER
```

For example,

```
[root@vault ~]# vault read transit/export/encryption-key/mybucketkey/1

Key     Value
---     -----
keys    map[1:-gbTI9lNpqv/V/2lDcmH2Nq1xKn6FPDWarCmFM2aNsQ=]
name    mybucketkey
type    aes256-gcm96
```

Uploading an object using AWS and the Vault

When uploading an object to the Ceph Object Gateway, the Ceph Object Gateway fetches the key from the Vault, and then encrypts and stores the object in a bucket. When a request is made to download the object, the Ceph Object Gateway automatically retrieves the corresponding key from the Vault and decrypts the object. To upload an object, the Ceph Object Gateway fetches the key from the Vault and then encrypts the object and stores it in the bucket. The Ceph Object Gateway retrieves the corresponding key from the Vault and decrypts the object when there is a request to download the object.

> **Note:** The URL is constructed using the base address, set by the `rgw_crypt_vault_addr` option, and the path prefix, set by the `rgw_crypt_vault_prefix` option.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway software.
- Installation of the HashiCorp Vault software.
- Access to a Ceph Object Gateway client node.
- Access to Amazon Web Services (AWS).

Procedure

1. Upload an object using the AWS command line client and provide the Secure Side Encryption(SSE) key ID in the request:
   a. For the Key/Value secret engine:

      Example

      ```
      [user@client ~]$ aws --endpoint=http://radosgw:8000 s3 cp plaintext.txt s3://
      mybucket/encrypted.txt --sse=aws:kms --sse-kms-key-id myproject/mybucketkey
      ```

      > **Note:** In the example, the Ceph Object Gateway would fetch the secret from `http://vault-server:8200/v1/secret/data/myproject/mybucketkey`

   b. For the Transit engine:

      Example

```
[user@client ~]$ aws --endpoint=http://radosgw:8000 s3 cp plaintext.txt s3://
mybucket/encrypted.txt --sse=aws:kms --sse-kms-key-id mybucketkey
```

Note: In the example, the Ceph Object Gateway would fetch the secret from `http://vaultserver:8200/v1/transit/mybucketkey`

Multi-factor authentication

When a bucket is configured for object versioning, a developer can optionally configure the bucket to require multi-factor authentication (MFA) for delete requests. Using MFA, a time-based one time password (TOTP) token is passed as a key to the `x-amz-mfa` header. The tokens are generated with virtual MFA devices like Google Authenticator, or a hardware MFA device like those provided by Gemalto.

Use `radosgw-admin` to assign time-based one time password tokens to a user. You must set a secret seed and a serial ID. You can also use `radosgw-admin` to list, remove, and resynchronize tokens.

Important: In a multi-site environment it is advisable to use different tokens for different zones, because, while MFA IDs are set on the user's metadata, the actual MFA one time password configuration resides on the local zone's OSDs.

Term	Description
TOTP	Time-based One Time Password.
Token serial	A string that represents the ID of a TOTP token.
Token seed	The secret seed that is used to calculate the TOTP. It can be hexadecimal or base32.
TOTP seconds	The time resolution used for TOTP generation.
TOTP window	The number of TOTP tokens that are checked before and after the current token when validating tokens.
TOTP pin	The valid value of a TOTP token at a certain time.

Table: Terminology

Creating a seed for multi-factor authentication

To set up multi-factor authentication (MFA), you must create a secret seed for use by the one-time password generator and the back-end MFA system.

Prerequisites

- A Linux system.
- Access to the command line shell.

Procedure

1. Generate a 30 character seed from the `urandom` Linux device file and store it in the shell variable `SEED`:

   Example

   ```
   [user@host01 ~]$ SEED=$(head -10 /dev/urandom | sha512sum | cut -b 1-30)
   ```

2. Print the seed by running echo on the `SEED` variable:

   Example

```
[user@host01 ~]$ echo $SEED
492dedb20cf51d1405ef6a1316017e
```

Configure the one-time password generator and the back-end MFA system to use the same seed.

Reference

For more information, see the following:

- "Multi-factor authentication" on page 1031

Creating a new multi-factor authentication TOTP token

Create a new multi-factor authentication (MFA) time-based one time password (TOTP) token.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Ceph Object Gateway is installed.
- You have root access on a Ceph Monitor node.
- A secret seed for the one-time password generator and Ceph Object Gateway MFA was generated.

Procedure

- Create a new MFA TOTP token:

  Syntax

  ```
  radosgw-admin mfa create --uid=USERID --totp-serial=SERIAL --totp-seed=SEED --totp-
  seed-type=SEED_TYPE --totp-seconds=TOTP_SECONDS --totp-window=TOTP_WINDOW
  ```

  Set *USERID* to the user name to set up MFA on, set *SERIAL* to a string that represents the ID for the TOTP token, and set *SEED* to a hexadecimal or base32 value that is used to calculate the TOTP. The following settings are optional: Set the *SEED_TYPE* to `hex` or `base32`, set *TOTP_SECONDS* to the timeout in seconds, or set *TOTP_WINDOW* to the number of TOTP tokens to check before and after the current token when validating tokens.

  Example

  ```
  [root@host01 ~]# radosgw-admin mfa create --uid=johndoe --totp-serial=MFAtest --totp-
  seed=492dedb20cf51d1405ef6a1316017e
  ```

Reference

For more information, see the following:

- "Creating a seed for multi-factor authentication" on page 1031
- "Resynchronizing a multi-factor authentication TOTP token" on page 1033

Test a multi-factor authentication TOTP token

Test a multi-factor authentication (MFA) time-based one time password (TOTP) token.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the nodes.
- Ceph Object Gateway is installed.
- You have root access on a Ceph Monitor node.

– An MFA TOTP token was created using `radosgw-admin mfa create`.

Procedure

– Test the TOTP token PIN to verify that TOTP functions correctly:

Syntax

```
radosgw-admin mfa check --uid=USERID --totp-serial=SERIAL --totp-pin=PIN
```

Set *USERID* to the user name MFA is set up on, set *SERIAL* to the string that represents the ID for the TOTP token, and set *PIN* to the latest PIN from the one-time password generator.

Example

```
[root@host01 ~]# radosgw-admin mfa check  --uid=johndoe --totp-serial=MFAtest --totp-
pin=870305
ok
```

If this is the first time you have tested the PIN, it may fail. If it fails, resynchronize the token.

Reference

For more information, see the following:

– “Creating a seed for multi-factor authentication” on page 1031

– “Resynchronizing a multi-factor authentication TOTP token” on page 1033

Resynchronizing a multi-factor authentication TOTP token

Resynchronize a multi-factor authentication (MFA) time-based one time password token.

Prerequisites

– A running Red Hat Ceph Storage cluster.

– Ceph Object Gateway is installed.

– You have root access on a Ceph Monitor node.

– An MFA TOTP token was created using `radosgw-admin mfa create`.

Procedure

1. Resynchronize a multi-factor authentication TOTP token in case of time skew or failed checks.

   This requires passing in two consecutive pins: the previous pin, and the current pin.

   Syntax

   ```
   radosgw-admin mfa resync --uid=USERID --totp-serial=SERIAL --totp-pin=PREVIOUS_PIN --
   totp=pin=CURRENT_PIN
   ```

   Set *USERID* to the user name MFA is set up on, set *SERIAL* to the string that represents the ID for the TOTP token, set *PREVIOUS_PIN* to the user’s previous PIN, and set *CURRENT_PIN* to the user’s current PIN.

   Example

   ```
   [root@host01 ~]# radosgw-admin mfa resync --uid=johndoe --totp-serial=MFAtest --totp-
   pin=802021 --totp-pin=439996
   ```

2. Verify the token was successfully resynchronized by testing a new PIN:

   Syntax

   ```
   radosgw-admin mfa check --uid=USERID --totp-serial=SERIAL --totp-pin=PIN
   ```

Set *USERID* to the user name MFA is set up on, set *SERIAL* to the string that represents the ID for the TOTP token, and set *PIN* to the user's PIN.

Example

```
[root@host01 ~]# radosgw-admin mfa check  --uid=johndoe --totp-serial=MFAtest --totp-
pin=870305
ok
```

Reference

For more information, see "Creating a seed for multi-factor authentication" on page 1031.

Listing multi-factor authentication TOTP tokens

List all multi-factor authentication (MFA) time-based one time password (TOTP) tokens that a particular user has.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Ceph Object Gateway is installed.
- You have root access on a Ceph Monitor node.
- An MFA TOTP token was created using `radosgw-admin mfa create`.

Procedure

- List MFA TOTP tokens:

  Syntax

  ```
  radosgw-admin mfa list --uid=USERID
  ```

  Set *USERID* to the user name MFA is set up on.

  Example

  ```
  [root@host01 ~]# radosgw-admin mfa list --uid=johndoe
  {
      "entries": [
          {
              "type": 2,
              "id": "MFAtest",
              "seed": "492dedb20cf51d1405ef6a1316017e",
              "seed_type": "hex",
              "time_ofs": 0,
              "step_size": 30,
              "window": 2
          }
      ]
  }
  ```

Reference

For more information, see the following:

- *Creating a new multi-factor authentication TOTP token*

Display a multi-factor authentication TOTP token

Display a specific multi-factor authentication (MFA) time-based one time password (TOTP) token by specifying its serial.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Ceph Object Gateway is installed.
- You have root access on a Ceph Monitor node.
- An MFA TOTP token was created using `radosgw-admin mfa create`.

Procedure

- Show the MFA TOTP token:

  Syntax

  ```
  radosgw-admin mfa get --uid=USERID --totp-serial=SERIAL
  ```

  Set *USERID* to the user name MFA is set up on and set *SERIAL* to the string that represents the ID for the TOTP token.

Reference

For more information, see “Creating a new multi-factor authentication TOTP token” on page 1032.

Deleting a multi-factor authentication TOTP token or versioned-object

Delete a multi-factor authentication (MFA) time-based one time password (TOTP) token or versioned-object.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Ceph Object Gateway is installed.
- You have root access on a Ceph Monitor node.
- An MFA TOTP token that was created by using the **`radosgw-admin mfa create`** command.

Deleting an MFA TOTP token

Procedure

1. Delete the MFA TOTP token.

   ```
   radosgw-admin mfa remove --uid=USERID --totp-serial=SERIAL
   ```

   Set *USERID* to the user name MFA is set up on and set *SERIAL* to the string that represents the ID for the TOTP token.
   For example,

   ```
   [root@host01 ~]# radosgw-admin mfa remove --uid=johndoe --totp-serial=MFAtest
   ```

2. Verify that the MFA TOTP token was deleted.

   ```
   radosgw-admin mfa get --uid=USERID --totp-serial=SERIAL
   ```

   Set *USERID* to the user name MFA is set up on and set *SERIAL* to the string that represents the ID for the TOTP token.
   For example,

   ```
   [root@host01 ~]# radosgw-admin mfa get --uid=johndoe --totp-serial=MFAtest
   MFA serial id not found
   ```

Deleting an MFA-enabled versioned object

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root user must be authenticated with an MFA to delete a bucket that is versioning enabled.
- MFA-Object you want to delete is in a bucket with versioning enabled.

Procedure

1. Delete the MFA-enabled versioned object.

```
radosgw-admin object rm --bucket BUCKET_NAME --object OBJECT_NAME --object-version
OBJECT_VERSION --totp-pin TOTP_TOKEN
```

   Set *BUCKET_NAME* to the name of the bucket, *OBJECT_NAME* to the name of the object, and *VERSION_ID* to the version ID of the object to delete, along with the TOTP token.
   For example,

```
[root@host01 ~]# radosgw-admin object rm --bucket test-mfa --object obj1 --object-
version 4SdKdSmpVAarLLdsFukjUVEwr-2oWfC --totp-pin 562813
```

2. Verify that the MFA-enabled versioned object was deleted.

```
radosgw-admin bucket list --bucket BUCKET_NAME
```

   Set *USERID* to the user name MFA is set up on and set *SERIAL* to the string that represents the ID for the TOTP token.
   For example,

```
[root@host01 ~]# radosgw-admin bucket list --bucket test-mfa
```

Administrating Ceph Object Gateway

As a storage administrator, you can manage the Ceph Object Gateway using the `radosgw-admin` command line interface (CLI) or using the Red Hat Ceph Storage Dashboard.

> **Note:** Not all of the Ceph Object Gateway features are available to the Red Hat Ceph Storage Dashboard. For details about administrating the Ceph Object Gateway through the dashboard, see Managing Ceph Object Gateway.

Prerequisites

- A healthy running Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway software.

Creating storage policies

The Ceph Object Gateway stores the client bucket and object data by identifying placement targets, and storing buckets and objects in the pools associated with a placement target. If you don't configure placement targets and map them to pools in the instance's zone configuration, the Ceph Object Gateway will use default targets and pools, for example, `default_placement`.

Storage policies give Ceph Object Gateway clients a way of accessing a storage strategy, that is, the ability to target a particular type of storage, such as SSDs, SAS drives, and SATA drives, as a way of ensuring, for example, durability, replication, and erasure coding. For more information, see Storage strategies

To create a storage policy, use the following procedure:

1. Create a new pool `.rgw.buckets.special` with the desired storage strategy. For example, a pool customized with erasure-coding, a particular CRUSH ruleset, the number of replicas, and the `pg_num` and `pgp_num` count.
2. Get the zone group configuration and store it in a file:

   Syntax

   ```
   radosgw-admin zonegroup --rgw-zonegroup=ZONE_GROUP_NAME get > FILE_NAME.json
   ```

   Example

   ```
   [root@host01 ~]# radosgw-admin zonegroup --rgw-zonegroup=default get > zonegroup.json
   ```

3. Add a `special-placement` entry under `placement_target` in the `zonegroup.json` file:

   Example

   ```
   {
           "name": "default",
           "api_name": "",
           "is_master": "true",
           "endpoints": [],
           "hostnames": [],
           "master_zone": "",
           "zones": [{
                   "name": "default",
                   "endpoints": [],
                   "log_meta": "false",
                   "log_data": "false",
                   "bucket_index_max_shards": 5
           }],
           "placement_targets": [{
                   "name": "default-placement",
                   "tags": []
           }, {
                   "name": "special-placement",
                   "tags": []
           }],
           "default_placement": "default-placement"
   }
   ```

4. Set the zone group with the modified `zonegroup.json` file:

   Example

   ```
   [root@host01 ~]# radosgw-admin zonegroup set < zonegroup.json
   ```

5. Get the zone configuration and store it in a file, for example, `zone.json`:

   Example

   ```
   [root@host01 ~]# radosgw-admin zone get > zone.json
   ```

6. Edit the zone file and add the new placement policy key under `placement_pool`:

   Example

   ```
   {
           "domain_root": ".rgw",
           "control_pool": ".rgw.control",
           "gc_pool": ".rgw.gc",
           "log_pool": ".log",
           "intent_log_pool": ".intent-log",
           "usage_log_pool": ".usage",
           "user_keys_pool": ".users",
           "user_email_pool": ".users.email",
           "user_swift_pool": ".users.swift",
           "user_uid_pool": ".users.uid",
           "system_key": {
                   "access_key": "",
                   "secret_key": ""
           },
   ```

```
        "placement_pools": [{
                "key": "default-placement",
                "val": {
                        "index_pool": ".rgw.buckets.index",
                        "data_pool": ".rgw.buckets",
                        "data_extra_pool": ".rgw.buckets.extra"
                }
        }, {
                "key": "special-placement",
                "val": {
                        "index_pool": ".rgw.buckets.index",
                        "data_pool": ".rgw.buckets.special",
                        "data_extra_pool": ".rgw.buckets.extra"
                }
        }]
}
```

7. Set the new zone configuration:

   Example

   ```
   [root@host01 ~]# radosgw-admin zone set < zone.json
   ```

8. Update the zone group map:

   Example

   ```
   [root@host01 ~]# radosgw-admin period update --commit
   ```

   The `special-placement` entry is listed as a `placement_target`.

9. To specify the storage policy when making a request:

   Example

   ```
   $ curl -i http://10.0.0.1/swift/v1/TestContainer/file.txt -X PUT -H "X-Storage-Policy:
   special-placement" -H "X-Auth-Token: AUTH_rgwtxxxxxx"
   ```

Configure bucket index resharding

As a storage administrator, you can configure bucket index resharding in single-site and multi-site deployments to improve performance. Reshard a bucket index either manually offline or dynamically online.

The Ceph Object Gateway stores bucket index data in the index pool, which defaults to the **`.rgw.buckets.index`** parameter. When the client puts many objects in a single bucket without setting quotas for the maximum number of objects per bucket, the index pool can result in significant performance degradation.

- Bucket index resharding prevents performance bottlenecks when you add a high number of objects per bucket.
- You can configure bucket index resharding for new buckets or change the bucket index on the existing ones.
- You need to have the shard count as the nearest prime number to the calculated shard count. The bucket index shards that are prime numbers tend to work better in an evenly distributed bucket index entries across shards.
- Bucket index can be resharded manually or dynamically.
  During the process of resharding bucket index dynamically, there is a periodic check of all the Ceph Object Gateway buckets and it detects buckets that require resharding. If a bucket has grown larger than the value specified in the **`rgw_max_objs_per_shard`** parameter, the Ceph Object Gateway reshards the bucket dynamically in the background. The default value for **`rgw_max_objs_per_shard`** is 100,000 objects per shard. Resharding bucket index dynamically works as expected on the upgraded single-site configuration without any modification to the zone or the zone group. A single site-configuration can be any of the following:
  - A default zone configuration with no realm.
  - A non-default configuration with at least one realm.

- A multi-realm single-site configuration.

> **Note:** Versioned buckets may exhibit imbalanced indexes, especially if a small subset of objects are written and re-written. This issue may lead to large omaps on the versioned bucket when a large number of object uploads happen (around a million objects).

Enabling non-blocking bucket resharding

Enable bucket index resharding in both single-site and multi-site deployments to improve RGW client performance.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Red Hat Ceph Storage deployment uses the version that supports non-blocking resharding.
- All RGW, rgw-admin, and OSD nodes in the cluster are upgraded to the same supported version.
- Any ongoing bucket resharding operations are completed before performing the upgrade.

About this task

By default, bucket resharding temporarily blocks write and delete operations, especially on large buckets. Enabling non-blocking resharding allows write and delete operations to continue with minimal disruption while resharding happens in the background.

> **Note:** Dynamic resharding is enabled by default in this release. Verify that it remains enabled.

Procedure

1. Upgrade the Red Hat Ceph Storage cluster to a version that supports non-blocking resharding.

   > **Note:** Do not perform a partial upgrade. All relevant components must be upgraded. Mismatched versions between components may cause resharding to fail or revert to blocking mode.

2. Verify that all RGW, rgw-admin, and OSD nodes are running the same version.

Result

After the upgrade, bucket resharding occurs in the background without significantly disrupting write or delete operations. This improves performance during resharding events.

Recovering bucket index

Resharding a bucket that was created with **`bucket_index_max_shards = 0`**, removes the bucket's metadata. However, you can restore the bucket indexes by recovering the affected buckets.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- A Ceph Object Gateway is installed at a minimum of two sites.
- The package for `jq` installed.

About this task

The `/usr/bin/rgw-restore-bucket-index` tool creates temporary files in the `/tmp` directory. These temporary files consume space based on the bucket objects count from the previous buckets. The previous buckets with

more than 10 M objects need more than 4 GB of space in `/tmp` directory. If the storage space in `/tmp` is exhausted, the tool fails with the following message:

```
ln: failed to access '/tmp/rgwrbi-object-list.4053207': No such file or directory
```

The temporary objects are removed.

Note:

- The tool does not work for versioned buckets.

  ```
  [root@host01 ~]# time rgw-restore-bucket-index  --proceed serp-bu-ver-1
  default.rgw.buckets.data
  NOTICE: This tool is currently considered EXPERIMENTAL.
  marker is e871fb65-b87f-4c16-a7c3-064b66feb1c4.25076.5
  bucket_id is e871fb65-b87f-4c16-a7c3-064b66feb1c4.25076.5
  Error: this bucket appears to be versioned, and this tool cannot work with
  versioned buckets.
  ```

- The tool's scope is limited to a single site only and not multi-site, that is, if you run the **`rgw-restore-bucket-index`** command at Site A, it does not recover objects in Site B, and vice versa. On a multi-site, the recovery tool and the **`object re-index`** command must be run at both sites for a bucket.

Procedure

Run either of the following steps to recover the bucket index.

- Run **`radosgw-admin object reindex`** command.

  ```
  radosgw-admin object reindex --bucket BUCKET_NAME --object OBJECT_NAME
  ```

- Run the `rgw-restore-bucket-index` tool.

  ```
  /usr/bin/rgw-restore-bucket-index -b BUCKET_NAME -p DATA_POOL_NAME
  ```

  For example,

  ```
  [root@host01 ceph]# /usr/bin/rgw-restore-bucket-index -b bucket-large-1 -p local-
  zone.rgw.buckets.data

  marker is d8a347a4-99b6-4312-a5c1-75b83904b3d4.41610.2
  bucket_id is d8a347a4-99b6-4312-a5c1-75b83904b3d4.41610.2
  number of bucket index shards is 5
  data pool is local-zone.rgw.buckets.data
  NOTICE: This tool is currently considered EXPERIMENTAL.
  The list of objects that we will attempt to restore can be found in "/tmp/rgwrbi-
  object-list.49946".
  Please review the object names in that file (either below or in another window/
  terminal) before proceeding.
  Type "proceed!" to proceed, "view" to view object list, or "q" to quit: view
  Viewing...
  Type "proceed!" to proceed, "view" to view object list, or "q" to quit: proceed!
  Proceeding...
  NOTICE: Bucket stats are currently incorrect. They can be restored with the following
  command after 2 minutes:
      radosgw-admin bucket list --bucket=bucket-large-1 --allow-unordered --max-
  entries=1073741824
  Would you like to take the time to recalculate bucket stats now? [yes/no] yes
  Done

  real    2m16.530s
  user    0m1.082s
  sys    0m0.870s
  ```

Creating indexless buckets

You can configure a placement target where created buckets do not use the bucket index to store objects index; that is, indexless buckets. Placement targets that do not use data replication or listing might implement indexless buckets. Indexless buckets provide a mechanism in which the placement target does not track objects in specific buckets. Indexless buckets removes a resource contention that happens whenever an object write happens and reduces the number of round-trips that Ceph Object Gateway needs to make to the Ceph storage cluster. This can have a positive effect on concurrent operations and small object write performance.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway software.
- Root-level access to a Ceph Object Gateway node.

About this task

Important: The bucket index does not reflect the correct state of the bucket, and listing these buckets does not correctly return their list of objects, which affects multiple features. Specifically, these buckets are not synced in a multi-zone environment because the bucket index is not used to store change information. Do not to use S3 object versioning on indexless buckets because the bucket index is necessary for this feature.

Note: Using indexless buckets removes the limit of the max number of objects in a single bucket.

Procedure

1. Add a new placement target to the zonegroup by using the **`radosgw-admin zonegroup placement add`** command.
   For example,

   ```
   [ceph: root@host03 /]# radosgw-admin zonegroup placement add --rgw-
   zonegroup="default"
     --placement-id="indexless-placement"
   ```

2. Add a new placement target to the zone by using the **`radosgw-admin zone placement add`** command.
   For example,

   ```
   [ceph: root@host03 /]# radosgw-admin zone placement add --rgw-zone="default"
      --placement-id="indexless-placement"
      --data-pool="default.rgw.buckets.data"
      --index-pool="default.rgw.buckets.index"
      --data_extra_pool="default.rgw.buckets.non-ec"
      --placement-index-type="indexless"
   ```

3. Set the default placement of the zonegroup to `indexless-placement`.
   All buckets that are created in the `indexless-placement` target will be indexless buckets.

   ```
   radosgw-admin zonegroup placement default --placement-id "indexless-placement"
   ```

   For example,

   ```
   [ceph: root@host03 /]# radosgw-admin zonegroup placement default --placement-id
   "indexless-placement"
   ```

4. When the cluster is in a multi-site configuration, update and commit the period.

   ```
   radosgw-admin period update --commit
   ```

   For example,

```
[ceph: root@host03 /]# radosgw-admin period update --commit
```

5. Restart the Ceph Object Gateways on all nodes in the storage cluster for the change to take effect.

```
ceph orch restart SERVICE_TYPE
```

For example,

```
[ceph: root@host03 /]# ceph orch restart rgw
```

Limitations of bucket index resharding

Understand the limitations of bucket index shardings.

> **Important:** Use the following limitations with caution. There are implications related to your hardware choices and configuration, so you should always discuss these requirements with your Red Hat account team.

- **Maximum number of objects in one bucket before it needs resharding:** Use a maximum of 102,400 objects per bucket index shard. To take full advantage of resharding and maximize parallelism, provide a sufficient number of OSDs in the Ceph Object Gateway bucket index pool. This parallelization scales with the number of Ceph Object Gateway instances, and replaces the in-order index shard enumeration with a number sequence. The default locking timeout is extended from 60 seconds to 90 seconds.
- **Maximum number of objects imposed by bucket index sharding:** Based on prior testing, the number of bucket index shards currently supported is 65521.
- **You can reshard a bucket three times before the other zones catch-up:** Resharding is not recommended until the older generations synchronize. Around four generations of the buckets from previous reshards are supported. Once the limit is reached, dynamic resharding does not reshard the bucket again until at least one of the old log generations are fully trimmed. Using the command `radosgw-admin bucket reshard` throws the following error:

```
Bucket BUCKET_NAME already has too many log generations (4) from previous reshards that
peer zones haven't finished syncing.

Resharding is not recommended until the old generations sync, but you can force a
reshard with `--yes-i-really-mean-it`.
```

Scalability considerations for dynamic resharding
Ceph Object Gateway automatically reshares a bucket when it exceeds approximately 400 million objects, scaling up to a maximum of 1,999 shards. After reaching this limit, it requires manual resharding to support additional object uploads. If you do not manually reshard beyond this point, the object map may grow too large and degrade bucket performance.

> **Note:** Monitor the shard count proactively and plan for manual resharding before the bucket reaches the 1,999 shard limit to avoid performance issues.

Configuring bucket index resharding in simple deployments

To enable and configure bucket index resharding on all new buckets, use the **`rgw_override_bucket_index_max_shards`** parameter.

You can set the parameter to one of the following values:

- `0` to disable bucket index sharding, which is the default value.
- A value greater than `0` to enable bucket sharding and to set the maximum number of shards.

Prerequisites

- Two running Red Hat Ceph Storage clusters.
- A Ceph Object Gateway installed.

Procedure

1. Calculate the recommended number of shards:

```
number of objects expected in a bucket / 100,000
```

> **Note:** The maximum number of bucket index shards currently supported is 65,521.

2. Set the `rgw_override_bucket_index_max_shards` option accordingly:

   Syntax

```
ceph config set client.rgw rgw_override_bucket_index_max_shards VALUE
```

   Replace *VALUE* with the recommended number of shards calculated:

   Example

```
[ceph: root@host01 /]# ceph config set client.rgw rgw_override_bucket_index_max_shards
12
```

   - To configure bucket index resharding for all instances of the Ceph Object Gateway, set the `rgw_override_bucket_index_max_shards` parameter with the `global` option.
   - To configure bucket index resharding only for a particular instance of the Ceph Object Gateway, add `rgw_override_bucket_index_max_shards` parameter under the instance.

3. Restart the Ceph Object Gateways on all nodes in the cluster to take effect:

   Syntax

```
ceph orch restart SERVICE_TYPE
```

   Example

```
[ceph: root#host01 /]# ceph orch restart rgw
```

Reference

For more information, see the following:

- “Resharding bucket index dynamically” on page 1045
- “Resharding bucket index manually” on page 1050

Configuring bucket index resharding in multi-site deployments

In multi-site deployments, each zone can have a different `index_pool` setting to manage failover. To configure a consistent shard count for zones in one zone group, set the `bucket_index_max_shards` parameter in the configuration for that zone group.

The default value of `bucket_index_max_shards` parameter is 11.

You can set the parameter to one of the following values:

- `0` to disable bucket index sharding.
- A value greater than `0` to enable bucket sharding and to set the maximum number of shards.

Note: Mapping the index pool, for each zone, if applicable, to a CRUSH ruleset of SSD-based OSDs might also help with bucket index performance. See *CRUSH performance domains*

Important: To prevent sync issues in multi-site deployments, a bucket should not have more than three generation gaps.

Prerequisites

- Two running Red Hat Ceph Storage clusters.
- A Ceph Object Gateway installed at a minimum of two sites.

Procedure

1. Calculate the recommended number of shards:

```
number of objects expected in a bucket / 100,000
```

   Note: The maximum number of bucket index shards currently supported is 65,521.

2. Extract the zone group configuration to the `zonegroup.json` file:

   Example

```
[ceph: root@host01 /]# radosgw-admin zonegroup get > zonegroup.json
```

3. In the `zonegroup.json` file, set the `bucket_index_max_shards` parameter for each named zone:

   Syntax

```
bucket_index_max_shards = VALUE
```

   Replace *VALUE* with the recommended number of shards calculated:

   Example

```
bucket_index_max_shards = 12
```

4. Reset the zone group:

   Example

```
[ceph: root@host01 /]# radosgw-admin zonegroup set < zonegroup.json
```

5. Update the period:

   Example

```
[ceph: root@host01 /]# radosgw-admin period update --commit
```

6. Check if resharding is complete:

   Syntax

```
radosgw-admin reshard status --bucket BUCKET_NAME
```

   Example

```
[ceph: root@host01 /]# radosgw-admin reshard status --bucket data
```

Verification

- Check the sync status of the storage cluster:

  Example

```
[ceph: root@host01 /]# radosgw-admin sync status
```

Resharding bucket index dynamically

Dynamic resharding is a process where the system automatically reshards the bucket index without requiring manual intervention. The system monitors factors like data distribution and triggers resharding when necessary to optimize storage and performance.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- A Ceph Object Gateway installed.

About this task

When dynamic resharding is needed, the system automatically adds the bucket to the resharding queue, and background threads perform the resharding one bucket at a time. This ensures the system adapts to changing demands efficiently.

This process is seamless and happens in the background, allowing the system to adjust dynamically to varying workloads without admin intervention.

> **Note:**
> For more information, see the following:
> - “Cleaning stale instances of bucket entries after resharding” on page 1051
> - “Resharding bucket index manually” on page 1050
> - “Configuring bucket index resharding in simple deployments” on page 1042

Procedure

1. In the Ceph configuration file, set the **`rgw_dynamic_resharding`** parameter to `true`.

```
ceph config set client.rgw OPTION VALUE
```

   For example,

```
[ceph: root@host01 /]# ceph config set client.rgw rgw_dynamic_resharding true
```

2. Optional: Customize the Ceph configuration with any of the optional parameters.

```
ceph config set client.rgw OPTION VALUE
```

Table 117: Ceph configuration options for dynamic resharding

Parameter option	Description	Default value
`rgw_reshard_num_logs`	The number of shards for the resharding log.	`16`
`rgw_reshard_bucket_lock_duration`	The duration in seconds of the lock on a bucket during resharding.	`360`

Parameter option	Description	Default value
rgw_dynamic_resharding	A boolean value that enables or disables dynamic resharding.	true
rgw_max_objs_per_shard	The maximum number of objects per shard.	100000
rgw_reshard_thread_interval	The maximum time in seconds between rounds of reshard thread processing.	600
rgw_dynamic_resharding_may_reduce	Set totrueto allow shard reduction during dynamic resharding. Must be used with **rgw_dynamic_reshard** set to truefor dynamic resharding.	true
rgw_dynamic_resharding_reduction_wait	Defines the wait time in hours before performing a reduction after identifying a bucket for reduction. This option provides a protection mechanism to prevent constant resharding due to fluctuating object counts. **Note:** A low value can affect cluster performance for buckets with frequent object count fluctuations.	120

The following example increases the number of shards for the resharding log to 23.

```
[ceph: root@host01 /]# ceph config set client.rgw rgw_reshard_num_logs 23
```

3. Add a bucket to the resharding queue.

```
radosgw-admin reshard add --bucket BUCKET --num-shards NUMBER
```

For example,

```
[ceph: root@host01 /]# radosgw-admin reshard add --bucket data --num-shards 10
```

4. List the resharding queue.

```
radosgw-admin reshard list
```

For example,

```
[ceph: root@host01 /]# radosgw-admin reshard list
```

5. Check the bucket log generations and shards.

```
radosgw-admin bucket layout --bucket data
```

For example,

```
[ceph: root@host01 /]# radosgw-admin bucket layout --bucket data
{
    "layout": {
        "resharding": "None",
        "current_index": {
            "gen": 1,
            "layout": {
                "type": "Normal",
                "normal": {
                    "num_shards": 23,
                    "hash_type": "Mod"
                }
            }
        },
        "logs": [
            {
                "gen": 0,
                "layout": {
                    "type": "InIndex",
                    "in_index": {
                        "gen": 0,
                        "layout": {
                            "num_shards": 11,
                            "hash_type": "Mod"
                        }
                    }
                }
            },
            {
                "gen": 1,
                "layout": {
                    "type": "InIndex",
                    "in_index": {
                        "gen": 1,
                        "layout": {
                            "num_shards": 23,
                            "hash_type": "Mod"
                        }
                    }
                }
            }
        ]
    }
}
```

6. Check bucket resharding status.

```
radosgw-admin reshard status --bucket BUCKET
```

For example,

```
[ceph: root@host01 /]# radosgw-admin reshard status --bucket data
```

7. Immediately process task entries on the resharding queue.

```
radosgw-admin reshard process
```

For example,

```
[ceph: root@host01 /]# radosgw-admin reshard process
```

8. Optional: Cancel pending bucket resharding tasks.

> **Important:** You can only cancel `pending` resharding operations. Do not cancel `ongoing` resharding operations.

```
radosgw-admin reshard cancel --bucket BUCKET
```

For example,

```
[ceph: root@host01 /]# radosgw-admin reshard cancel --bucket data
```

What to do next

Verify the bucket resharding status.

```
radosgw-admin reshard status --bucket BUCKET
```

For example,

```
[ceph: root@host01 /]# radosgw-admin reshard status --bucket data
```

Resharding bucket index dynamically in multi-site configuration

The feature allows buckets to be resharded in a multi-site configuration without interrupting the replication of their objects.

Before you begin

- At least two running Red Hat Ceph Storage clusters.
- All the Ceph Object Gateway daemons enabled at both the sites are upgraded to the latest version.
- Root-level access to all the nodes.

About this task

When `rgw_dynamic_resharding` is enabled, it runs on each zone independently, and the zones might choose different shard counts for the same bucket.
You need to enable the `resharding` feature manually on the existing zones and the zone groups after upgrading the storage cluster.

Note:

- You can reshard a bucket three times before the other zones catch-up. For more information, see "Limitations of bucket index resharding" on page 1042.
- Zones and zone groups are supported and enabled by default.
- If a bucket is created and uploaded with more than the threshold number of objects for resharding dynamically, you need to continue to write I/Os to old buckets to begin the resharding process.

For more information, see "Resharding bucket index dynamically" on page 1045.

Procedure

1. Check if `resharding` is enabled on the zonegroup.

   ```
   radosgw-admin sync status
   ```

   For example,

   ```
   [ceph: root@host01 /]# radosgw-admin sync status
   ```

   If `zonegroup features enabled` is not enabled for resharding on the zonegroup, then continue with the procedure.

2. Enable the `resharding` feature on all the zonegroups in the multi-site configuration where Ceph Object Gateway is installed.

   ```
   radosgw-admin zonegroup modify --rgw-zonegroup=ZONEGROUP_NAME --enable-
   feature=resharding
   ```

   For example,

```
[ceph: root@host01 /]# radosgw-admin zonegroup modify --rgw-zonegroup=us --enable-
feature=resharding
```

3. Update the period and commit.

```
radosgw-admin period update --commit
```

For example,

```
[ceph: root@host01 /]# radosgw-admin period update --commit
```

4. Enable the `resharding` feature on all the zones in the multi-site configuration where Ceph Object Gateway is installed.

```
radosgw-admin zone modify --rgw-zone=ZONE_NAME --enable-feature=resharding
```

For example,

```
[ceph: root@host01 /]# radosgw-admin zone modify --rgw-zone=us-east --enable-
feature=resharding
```

5. Update the period and commit.

```
radosgw-admin period update --commit
```

6. Verify the `resharding` feature is enabled on the zones and zonegroups.
You can see that each zone lists its `supported_features` and the zonegroups lists its `enabled_features`.

```
radosgw-admin period get
```

For example,

```
[ceph: root@host01 /]# radosgw-admin period get

"zones": [
                    {
                        "id": "505b48db-6de0-45d5-8208-8c98f7b1278d",
                        "name": "us_east",
                        "endpoints": [
                            "http://10.0.208.11:8080"
                        ],
                        "log_meta": "false",
                        "log_data": "true",
                        "bucket_index_max_shards": 11,
                        "read_only": "false",
                        "tier_type": "",
                        "sync_from_all": "true",
                        "sync_from": [],
                        "redirect_zone": "",
                        "supported_features": [
                            "resharding"
                        ]
                "default_placement": "default-placement",
                "realm_id": "26cf6f23-c3a0-4d57-aae4-9b0010ee55cc",
                "sync_policy": {
                    "groups": []
                },
                "enabled_features": [
                    "resharding"
                ]
```

7. Check the sync status.

```
radosgw-admin sync status
```

For example,

```
[ceph: root@host01 /]# radosgw-admin sync status
          realm 26cf6f23-c3a0-4d57-aae4-9b0010ee55cc (usa)
      zonegroup 33a17718-6c77-493e-99fe-048d3110a06e (us)
           zone 505b48db-6de0-45d5-8208-8c98f7b1278d (us_east)
zonegroup features enabled: resharding
```

In this example, the `resharding` feature is enabled for the `us` zonegroup.

8. Optional: Disable the `resharding` feature for the zonegroups.

> **Important:** To disable resharding on any singular zone, set the `rgw_dynamic_resharding` configuration option to `false` on that specific zone.

a. Disable the feature on all the zonegroups in the multi-site where Ceph Object Gateway is installed.

```
radosgw-admin zonegroup modify --rgw-zonegroup=ZONEGROUP_NAME --disable-
feature=resharding
```

For example,

```
[ceph: root@host01 /]# radosgw-admin zonegroup modify --rgw-zonegroup=us --
disable-feature=resharding
```

b. Update the period and commit.

```
radosgw-admin period update --commit
```

Resharding bucket index manually

If a bucket has grown larger than the initial configuration for which it was optimized, reshard the bucket index pool by using the **`radosgw-admin bucket reshard`** command.

This command performs the following tasks:

- Creates a new set of bucket index objects for the specified bucket.
- Distributes object entries across these bucket index objects.
- Creates a new bucket instance.
- Links the new bucket instance with the bucket so that all new index operations go through the new bucket indexes.
- Prints the old and the new bucket ID to the command output.

Prerequisites

- At least two running Red Hat Ceph Storage clusters.
- A Ceph Object Gateway installed at a minimum of two sites.

Procedure

1. Back up the original bucket index:

Syntax

```
radosgw-admin bi list --bucket=BUCKET > BUCKET.list.backup
```

Example

```
[ceph: root@host01 /]# radosgw-admin bi list --bucket=data > data.list.backup
```

2. Reshard the bucket index:

Syntax

```
radosgw-admin bucket reshard --bucket=BUCKET --num-shards=NUMBER
```

Example

```
[ceph: root@host01 /]# radosgw-admin bucket reshard --bucket=data --num-shards=100
```

Verification

- Check bucket resharding status:

  Syntax

  ```
  radosgw-admin reshard status --bucket bucket
  ```

  Example

  ```
  [ceph: root@host01 /]# radosgw-admin reshard status --bucket data
  ```

Cleaning stale instances of bucket entries after resharding

The resharding process might not clean stale instances of bucket entries automatically and these instances can impact performance of the storage cluster.

Before you begin

Before you begin, make sure that you have a running Red Hat Ceph Storage cluster with a Ceph Object Gateway node.

About this task

Clean stale instances manually to prevent them from negatively impacting the performance of the storage cluster.

> **Important:**
>
> - Before cleaning stale instances, contact Red Hat Support.
> - Use this procedure only in simple deployments, not in multi-site clusters.

Procedure

1. List stale instances.

   ```
   radosgw-admin reshard stale-instances list
   ```

2. Clean the stale instances of the bucket entries.

   ```
   radosgw-admin reshard stale-instances delete
   ```

What to do next

Verify the bucket resharding status.

```
radosgw-admin reshard status --bucket BUCKET
```

For example,

```
[ceph: root@host01 /]# radosgw-admin reshard status --bucket data
```

Fixing lifecycle policies after resharding

For storage clusters with resharded instances, the old lifecycle processes would have flagged and deleted the lifecycle processing as the bucket instance changed during a reshard.

However, for older buckets that had lifecycle policies and have undergone resharding, you can fix such buckets with the `reshard fix` option.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- A Ceph Object Gateway installed.

Procedure

- Fix the lifecycle policies of the older bucket:

  Syntax

  ```
  radosgw-admin lc reshard fix --bucket BUCKET_NAME
  ```

  > **Important:** If you do not use the `--bucket` argument, then the command fixes lifecycle policies for all the buckets in the storage cluster.

  Example

  ```
  [ceph: root@host01 /]# radosgw-admin lc reshard fix --bucket mybucket
  ```

Creating indexless buckets

You can configure a placement target where created buckets do not use the bucket index to store objects index; that is, indexless buckets. Placement targets that do not use data replication or listing might implement indexless buckets. Indexless buckets provide a mechanism in which the placement target does not track objects in specific buckets. Indexless buckets removes a resource contention that happens whenever an object write happens and reduces the number of round-trips that Ceph Object Gateway needs to make to the Ceph storage cluster. This can have a positive effect on concurrent operations and small object write performance.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway software.
- Root-level access to a Ceph Object Gateway node.

About this task

> **Important:** The bucket index does not reflect the correct state of the bucket, and listing these buckets does not correctly return their list of objects, which affects multiple features. Specifically, these buckets are not synced in a multi-zone environment because the bucket index is not used to store change information. Do not to use S3 object versioning on indexless buckets because the bucket index is necessary for this feature.

> **Note:** Using indexless buckets removes the limit of the max number of objects in a single bucket.

Procedure

1. Add a new placement target to the zonegroup by using the **radosgw-admin zonegroup placement add** command.
   For example,

   ```
   [ceph: root@host03 /]# radosgw-admin zonegroup placement add --rgw-
   zonegroup="default"
     --placement-id="indexless-placement"
   ```

2. Add a new placement target to the zone by using the **radosgw-admin zone placement add** command.
   For example,

   ```
   [ceph: root@host03 /]# radosgw-admin zone placement add --rgw-zone="default"
      --placement-id="indexless-placement"
      --data-pool="default.rgw.buckets.data"
      --index-pool="default.rgw.buckets.index"
      --data_extra_pool="default.rgw.buckets.non-ec"
      --placement-index-type="indexless"
   ```

3. Set the default placement of the zonegroup to `indexless-placement`.
   All buckets that are created in the `indexless-placement` target will be indexless buckets.

   ```
   radosgw-admin zonegroup placement default --placement-id "indexless-placement"
   ```

   For example,

   ```
   [ceph: root@host03 /]# radosgw-admin zonegroup placement default --placement-id
   "indexless-placement"
   ```

4. When the cluster is in a multi-site configuration, update and commit the period.

   ```
   radosgw-admin period update --commit
   ```

   For example,

   ```
   [ceph: root@host03 /]# radosgw-admin period update --commit
   ```

5. Restart the Ceph Object Gateways on all nodes in the storage cluster for the change to take effect.

   ```
   ceph orch restart SERVICE_TYPE
   ```

   For example,

   ```
   [ceph: root@host03 /]# ceph orch restart rgw
   ```

Enabling compression

The Ceph Object Gateway supports server-side compression of uploaded objects using any of Ceph's compression plugins.

The supported compression plugins include the following:

- `zlib`
- `snappy`
- `zstd`

Configuring compression

To enable compression on a zone's placement target, provide the `--compression=TYPE` option to the **radosgw-admin zone placement modify** command. The compression `TYPE` refers to the name of the compression plugin to use when writing new object data.

Each compressed object stores the compression type. Changing the setting does not hinder the ability to decompress existing compressed objects, nor does it force the Ceph Object Gateway to re-compress existing objects.

This compression setting applies to all new objects uploaded to buckets using this placement target.

To disable compression on a zone's placement target, provide the `--compression=TYPE` option to the **`radosgw-admin zone placement modify`** command and specify an empty string or `none`.

Example

```
[root@host01 ~] radosgw-admin zone placement modify --rgw-zone=default --placement-
id=default-placement --compression=zlib
{
...
    "placement_pools": [
        {
            "key": "default-placement",
            "val": {
                "index_pool": "default.rgw.buckets.index",
                "data_pool": "default.rgw.buckets.data",
                "data_extra_pool": "default.rgw.buckets.non-ec",
                "index_type": 0,
                "compression": "zlib"
            }
        }
    ],
...
}
```

After enabling or disabling compression, restart the Ceph Object Gateway instance so the change will take effect.

> **Note:** Ceph Object Gateway creates a `default` zone and a set of pools. For production deployments, see "Creating a realm" on page 846.

Compression statistics

While all existing commands and APIs continue to report object and bucket sizes based on their uncompressed data, the **`radosgw-admin bucket stats`** command includes compression statistics for all buckets.

The usage types for the **`radosgw-admin bucket stats`** command are as follows:

- `rgw.main`, for regular entries or objects.
- `rgw.multimeta`, for the metadata of incomplete multipart uploads.
- `rgw.cloudtiered`, for objects that a lifecycle policy has transitioned to a cloud tier. When configured with `retain_head_object=true`, a head object that no longer contains data remains but can still serve the object's metadata with HeadObject requests. These stub head objects use the `rgw.cloudtiered` category. For more information, see Transitioning data to Amazon S3 cloud service.

Syntax

```
radosgw-admin bucket stats --bucket=BUCKET_NAME
{
...
    "usage": {
        "rgw.main": {
            "size": 1075028,
            "size_actual": 1331200,
            "size_utilized": 592035,
            "size_kb": 1050,
            "size_kb_actual": 1300,
            "size_kb_utilized": 579,
            "num_objects": 104
        }
    },
...
}
```

The `size` is the accumulated size of the objects in the bucket, uncompressed and unencrypted. The `size_kb` is the accumulated size in kilobytes and is calculated as `size/1024`. In this example, it is `1075028/1024 = 1050`.

The `size_actual` is the actual size of all the objects after each object is rounded up with the ceiling function to the nearest 4096 bytes since the object is stored in 4096-sized chunks. If a bucket has two objects, one of size 4100 bytes and the other of 8500 bytes, the first object is rounded up to 8192 bytes, and the second one

rounded 12288 bytes, and their total for the bucket is 20480 bytes. The `size_kb_actual` is the actual size in kilobytes and is calculated as `size_actual/1024`. In this example, it is `1331200/1024 = 1300`.

The `size_utilized` is the total size of the compressed data in bytes. This considers both compression and encryption. Encryption could increase the size of the object while compression could decrease it. The `size_kb_utilized` is the total size in kilobytes and is calculated as `size_utilized/1024`. In this example, it is `592035/1024= 579`.

Here, all the sizes in kilobytes is rounded up with the ceiling function.

Quota management

The Ceph Object Gateway enables you to set quotas on users and buckets owned by users. Quotas include the maximum number of objects in a bucket and the maximum storage size in megabytes.

- *Bucket:* The `--bucket` option allows you to specify a quota for buckets the user owns.
- *Maximum Objects:* The `--max-objects` setting allows you to specify the maximum number of objects. A negative value disables this setting.
- *Maximum Size:* The `--max-size` option allows you to specify a quota for the maximum number of bytes. A negative value disables this setting.
- *Quota Scope:* The `--quota-scope` option sets the scope for the quota. The options are `bucket` and `user`. Bucket quotas apply to buckets a user owns. User quotas apply to a user.

> **Important:** Buckets with a large number of objects can cause serious performance issues. The recommended maximum number of objects in a one bucket is 100,000. To increase this number, configure bucket index sharding. For more information, see "Limitations of bucket index resharding" on page 1042.

To start managing quotas, follow this procedure:

1. "Set user quotas" on page 1055.
2. "Enable and disable user quotas" on page 1055.
3. "Set bucket quotas" on page 1056.
4. "Enable and disable bucket quotas" on page 1056.

Set user quotas

Before you enable a quota, you must first set the quota parameters.

Syntax

```
radosgw-admin quota set --quota-scope=user --uid=USER_ID [--max-objects=NUMBER_OF_OBJECTS]
[--max-size=MAXIMUM_SIZE_IN_BYTES]
```

Example

```
[root@host01 ~]# radosgw-admin quota set --quota-scope=user --uid=johndoe --max-
objects=1024 --max-size=1024
```

A negative value for num objects and / or max size means that the specific quota attribute check is disabled.

Enable and disable user quotas

Once you set a user quota, you can enable it.

Syntax

```
radosgw-admin quota enable --quota-scope=user --uid=USER_ID
```

You may disable an enabled user quota.

Syntax

```
radosgw-admin quota disable --quota-scope=user --uid=USER_ID
```

Set bucket quotas

Bucket quotas apply to the buckets owned by the specified `uid` and are independent of the user.

Syntax

```
radosgw-admin quota set --uid=USER_ID --quota-scope=bucket --bucket=BUCKET_NAME [--max-
objects=NUMBER_OF_OBJECTS] [--max-size=MAXIMUM_SIZE_IN_BYTES]
```

A negative value for *NUMBER_OF_OBJECTS*, *MAXIMUM_SIZE_IN_BYTES*, or both means that the specific quota attribute check is disabled.

Enable and disable bucket quotas

Enable a bucket quota after it is set. Disable the quota, by using the **`diable`** command.

Syntax

```
radosgw-admin quota enable --quota-scope=bucket --uid=USER_ID
```

You may disable an enabled bucket quota.

Syntax

```
radosgw-admin quota disable --quota-scope=bucket --uid=USER_ID
```

Get quota settings

You can access each user's quota settings via the user information API.

To read user quota setting information with the CLI interface, run the following command:

Syntax

```
radosgw-admin user info --uid=USER_ID
```

To get quota settings for a tenanted user, specify the user ID and the name of the tenant:

Syntax

```
radosgw-admin user info --uid=USER_ID --tenant=TENANT
```

Update quota stats

Quota stats get updated asynchronously. You can update quota statistics for all users and all buckets manually to retrieve the latest quota stats.

Syntax

```
radosgw-admin user stats --uid=USER_ID --sync-stats
```

Get user quota usage stats

Get the latest user quota usage stats to see how much of the quota a user has used.

To see how much of the quota a user has consumed, run the following command:

Syntax

```
radosgw-admin user stats --uid=USER_ID
```

Note: You should run the `radosgw-admin user stats` command with the `--sync-stats` option to receive the latest data.

Quota cache

Quota statistics are cached for each Ceph Gateway instance. If there are multiple instances, then the cache can keep quotas from being perfectly enforced, as each instance will have a different view of the quotas.

The options that control how the cache keeps the quotas are `rgw bucket quota ttl`, `rgw user quota bucket sync interval`, and `rgw user quota sync interval`. The higher these values are, the more efficient quota operations are, but the more out-of-sync multiple instances will be. The lower these values are, the closer to perfect enforcement multiple instances will achieve. If all three are 0, then quota caching is effectively disabled, and multiple instances will have perfect quota enforcement.

For more details on these options, see "Configuration reference" on page 1088.

Reading and writing global quotas

You can read and write quota settings in a zonegroup map.

To get a zonegroup map:

```
[root@host01 ~]# radosgw-admin global quota get
```

The global quota settings can be manipulated with the `global quota` counterparts of the `quota set`, `quota enable`, and `quota disable` commands, for example:

```
[root@host01 ~]# radosgw-admin global quota set --quota-scope bucket --max-objects 1024
[root@host01 ~]# radosgw-admin global quota enable --quota-scope bucket
```

Note: In a multi-site configuration, where there is a realm and period present, changes to the global quotas must be committed using `period update --commit`. If there is no period present, the Ceph Object Gateways must be restarted for the changes to take effect.

Usage

The Ceph Object Gateway logs usage for each user. You can also track user usage within date ranges.

You can enable usage log by setting `rgw_enable_usage_log = true` in central config or in `ceph.conf` and restart all Ceph Object Gateway services.

Options include:

- *Start Date:* The `--start-date` option allows you to filter usage stats from a particular start date (*format:* `yyyy-mm-dd[HH:MM:SS]`).
- *End Date:* The `--end-date` option allows you to filter usage up to a particular date (*format:* `yyyy-mm-dd[HH:MM:SS]`).
- *Log Entries:* The `--show-log-entries` option allows you to specify whether or not to include log entries with the usage stats (options: `true` | `false`).

Note: You can specify time with minutes and seconds, but it is stored with 1 hour resolution.

Show usage

To show usage statistics, specify the `usage show`.

To show usage for a particular user, you must specify a user ID. You may also specify a start date, end date, and whether or not to show log entries.

Example

```
[ceph: root@host01 /]# radosgw-admin usage show
                --uid=johndoe --start-date=2022-06-01
                --end-date=2022-07-01
```

You may also show a summary of usage information for all users by omitting a user ID.

Example

```
[ceph: root@host01 /]# radosgw-admin usage show --show-log-entries=false
```

Trim usage

With heavy use, usage logs can begin to take up storage space. You can trim usage logs for all users and for specific users. You may also specify date ranges for trim operations.

Example

```
[ceph: root@host01 /]# radosgw-admin usage trim --start-date=2022-06-01
                     --end-date=2022-07-31

[ceph: root@host01 /]# radosgw-admin usage trim --uid=johndoe
[ceph: root@host01 /]# radosgw-admin usage trim --uid=johndoe --end-date=2021-04-31
```

Deduplication statistics

Use the Ceph Object Gateway deduplication statistics feature to estimate potential storage savings. This command does not perform deduplication or modify data.

The Ceph Object Gateway deduplication statistics utility helps administrators identify redundant data across Ceph Object Gateway buckets. It uses radosgw-admin subcommands to start and manage estimation tasks. The utility reports duplication estimates but does not modify data.

The utility scans bucket index metadata instead of object data. This design reduces I/O load and enables fast, scalable analysis.

Use this utility to assess whether deduplication can reduce storage consumption in your Ceph Object Gateway environment. It supports capacity planning, storage optimization, and analysis of object storage patterns.

How estimation works

The estimation process runs in parallel across multiple Ceph Object Gateway daemons. It reads each bucket index shard once, which allows the workload to scale efficiently. The process does not read object data or metadata, so it runs independently of storage media performance, whether SSD or HDD.

Skipped objects

The utility excludes these objects from estimation:

- Objects smaller than 4 MB, unless Multipart.
- Objects stored with different placement rules, pools, or storage classes.
- Deduplication skips data storage pools that use a replicated rule.

Note: Do not use RAID. Red Hat Ceph Storage uses object copies and erasure coding, which eliminate the need for RAID solutions. A degraded RAID negatively impacts performance, and data recovery through RAID is significantly slower compared to recovery with object replicas or erasure-coded chunks.

Note: The deduplication process skips compressed and user-encrypted objects. The estimation process includes them because it cannot detect compression or encryption.

Memory usage

The utility uses a small, predictable amount of memory based on the object count. The following table lists the reference values.

Ceph Object Gateway object count	Approximate memory usage
1 million	8 MB
4 million	16 MB
16 million	32 MB
64 million	64 MB
256 million	128 MB
1 billion	256 MB
4 billion	512 MB
16 billion	1024 MB (1 GB)

Running a deduplication estimate

Run the **`radosgw-admin dedup estimate`** command to start deduplication estimation. The command identifies duplicate objects but does not modify data.

Before you begin

Before you begin, make sure that you have the following prerequisites in place.

- Administrative access to the Ceph cluster.
- RGW service is running.
- radosgw-admin tool is installed and available in your system path.

Procedure

1. Open a terminal on a node that has access to the RGW service.
2. Run the **`radosgw-admin dedup estimate`** command to start the estimate. Confirm that the system displays a message indicating the estimate has started.

```
radosgw-admin dedup estimate
```

> **Note:** Starting a new estimate automatically aborts the current estimation process

Result

To check progress or view results, run the **`radosgw-admin dedup stats`** command.

Monitoring the estimate

Procedure

- Run the following command to check progress or view results.

```
radosgw-admin dedup stats
```

Managing the estimate

Procedure

- Use the following commands to manage the deduplication estimate:
  - Pause the estimate:

```
radosgw-admin dedup pause
```

- Resume a paused estimate:

```
radosgw-admin dedup resume
```

- Cancel the estimate and release resources:

```
radosgw-admin dedup abort
```

Optimize the Ceph Object Gateway's data object storage

Bucket lifecycle configuration optimizes data object storage to increase its efficiency and to provide effective storage throughout the lifetime of the data.

The S3 API in the Ceph Object Gateway currently supports a subset of the AWS bucket lifecycle configuration actions:

- Expiration
- NoncurrentVersionExpiration
- AbortIncompleteMultipartUpload

Parallel thread processing for bucket life cycles

The Ceph Object Gateway now allows for parallel thread processing of bucket life cycles across multiple Ceph Object Gateway instances. Increasing the number of threads that run in parallel enables the Ceph Object Gateway to process large workloads more efficiently. In addition, the Ceph Object Gateway now uses a numbered sequence for index shard enumeration instead of using in-order numbering.

Optimizing the bucket lifecycle

Optimize the bucket lifecycle for different types of workloads.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Root-level access to all nodes in the storage cluster.

About this task

Two options in the Ceph configuration file affect the efficiency of bucket lifecycle processing:

`rgw_lc_max_worker`

Specifies the number of lifecycle worker threads to run in parallel. This enables the simultaneous processing of both bucket and index shards. The default value for this option is 3.

`rgw_lc_max_wp_worker`

Specifies the number of threads in each lifecycle worker thread's work pool. This option helps to accelerate processing for each bucket. The default value for this option is 3.

For a workload with many buckets, for example a workload with thousands of buckets, consider increasing the value of the `rgw_lc_max_worker` option.

For a workload with a smaller number of buckets but with a higher number of objects in each bucket, such as in the hundreds of thousands, consider increasing the value of the `rgw_lc_max_wp_worker` option.

> **Note:** Before increasing the value of either of these options, validate current storage cluster performance and Ceph Object Gateway use. Do not assign a value of 10 or more for either of these options.

Procedure

1. Increase the number of threads.
   - To increase the number of threads to run in parallel, set the value of `rgw_lc_max_worker` to a value between `3` and `9`.

     ```
     ceph config set client.rgw rgw_lc_max_worker 7
     ```

   - To increase the number of threads in each thread's work pool, set the value of `rgw_lc_max_wp_worker` to a value between `3` and `9`.

     ```
     ceph config set client.rgw rgw_lc_max_wp_worker 7
     ```

2. Restart the Ceph Object Gateway.
   Restarting allows the changed settings to take effect.
3. Monitor the storage cluster to verify that the increased values do not adversely affect performance.

Policy based data archival and retrieval to S3 compatible platforms

The object lifecycle transition rule allows you to transition an object from one storage class to another S3-compatible platform.

You can migrate data to S3-compatible platforms with the Ceph Object Gateway lifecycle transition policy.

> **Note:** In a multi-site configuration, when the lifecycle transition rule is applied on the first site, to transition objects from one data pool to another in the same storage cluster, then the same rule is valid for the second site, if the second site has the respective data pool created and enabled with `rgw` application.

Transitioning data to Amazon S3 cloud service

You can transition data to a remote cloud service as part of the lifecycle configuration using storage classes to reduce cost and improve manageability. The transition is unidirectional and data cannot be transitioned back from the remote zone. This feature is to enable data transition to multiple cloud providers such as Amazon (S3).

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- An Red Hat Ceph Storage with Ceph Object Gateway installed.
- User credentials for the remote cloud service, Amazon S3.
- Target path created on Amazon S3.
- `s3cmd` installed on the bootstrapped node.
- Amazon AWS configured locally to download data.

About this task

Use `cloud-s3` as `tier-type` to configure the remote cloud S3 object store service to which the data needs to be transitioned. These do not need a data pool and are defined in terms of the zonegroup placement targets.

Procedure

1. From the `cephadm` shell, create a user with access key and secret key.

   ```
   radosgw-admin user create --uid=USER_NAME --display-name="DISPLAY_NAME" [--access-key ACCESS_KEY --secret-key SECRET_KEY]
   ```

   For example,

```
[ceph: root@host01 /]# radosgw-admin user create --uid=test-user --display-name="test-
user" --access-key a21e86bce636c3aa1 --secret-key cf764951f1fdde5e
{
    "user_id": "test-user",
    "display_name": "test-user",
    "email": "",
    "suspended": 0,
    "max_buckets": 1000,
    "subusers": [],
    "keys": [
        {
            "user": "test-user",
            "access_key": "a21e86bce636c3aa1",
            "secret_key": "cf764951f1fdde5e"
        }
    ],
    "swift_keys": [],
    "caps": [],
    "op_mask": "read, write, delete",
    "default_placement": "",
    "default_storage_class": "",
    "placement_tags": [],
    "bucket_quota": {
        "enabled": false,
        "check_on_raw": false,
        "max_size": -1,
        "max_size_kb": 0,
        "max_objects": -1
    },
    "user_quota": {
        "enabled": false,
        "check_on_raw": false,
        "max_size": -1,
        "max_size_kb": 0,
        "max_objects": -1
    },
    "temp_url_keys": [],
    "type": "rgw",
    "mfa_ids": []
}
```

2. On the bootstrapped node, add a storage class. with the tier type as `cloud-s3`.
   Add one of the following tier types. The tier type options are `cloud-s3` and `cloud-s3-glacier`.
   - Adding the `cloud-s3` tier type.

     Important: After a storage class is created with the **`--tier-type=cloud-s3`** option , it cannot be later modified to any other storage class type.

     ```
     radosgw-admin zonegroup placement add --rgw-zonegroup =ZONE_GROUP_NAME \
                                 --placement-id=PLACEMENT_ID \
                                 --storage-class =STORAGE_CLASS_NAME \
                                 --tier-type=cloud-s3
     ```

     For example,

```
[ceph: root@host01 /]# radosgw-admin zonegroup placement add --rgw-
zonegroup=default \
                                                    --placement-id=default-placement
\
                                                    --storage-class=CLOUDTIER \
                                                    --tier-type=cloud-s3
[
    {
        "key": "default-placement",
        "val": {
            "name": "default-placement",
            "tags": [],
            "storage_classes": [
                "CLOUDTIER",
                "STANDARD"
            ],
            "tier_targets": [
                {
                    "key": "CLOUDTIER",
                    "val": {
                        "tier_type": "cloud-s3",
                        "storage_class": "CLOUDTIER",
                        "retain_head_object": "false",
                        "s3": {
                            "endpoint": "",
                            "access_key": "",
                            "secret": "",
                            "host_style": "path",
                            "target_storage_class": "",
                            "target_path": "",
                            "acl_mappings": [],
                            "multipart_sync_threshold": 33554432,
                            "multipart_min_part_size": 33554432
                        }
                    }
                }
            ]
        }
    }
]
```

– Adding the `cloud-s3-glacier` tier type.

```
radosgw-admin zonegroup placement add --rgw-zonegroup =ZONE_GROUP_NAME \
                            --placement-id=PLACEMENT_ID \
                            --storage-class =CLOUDTIER-GLACIER \
                            --tier-type=cloud-s3-glacier
```

For example,

```
[ceph: root@host01 /]# radosgw-admin zonegroup placement add --rgw-
zonegroup=default \
                                                --placement-id=default-placement
\
                                                --storage-class=CLOUDTIER-
GLACIER \
                                                --tier-type=cloud-s3-glacier

[
    {
                "key": "CLOUDTIER-GLACIER",
                "val": {
                    "tier_type": "cloud-s3-glacier",
                    "storage_class": "CLOUDTIER-GLACIER",
                    "retain_head_object": true,
                    "s3": {
                        "endpoint": "http://s3.us-east-1.amazonaws.com",
                        "access_key": "XXXXXXXXXX",
                        "secret": "YYYYYYYYYY",
                        "region": "us-east-1",
                        "host_style": "path",
                        "target_storage_class": "GLACIER",
                        "target_path": "rgwbucket",
                        "acl_mappings": [],
                        "multipart_sync_threshold": 44432,
                        "multipart_min_part_size": 44432
                    },
                    "allow_read_through": false,
                    "read_through_restore_days": 10,
                    "restore_storage_class": "COLDTIER",
                    "s3-glacier": {
                        "glacier_restore_days": 2,
                        "glacier_restore_tier_type": "Expedited"
                    }
                }
            }

]
```

3. Update the **storage_class**.

> **Note:**
>
> - If the cluster is part of a multi-site setup, run the **period update --commit** command so that the zone group changes are propagated to all the zones in the multi-site.
> - On a multisite environment, the object restore gets replicated when the object is permanently restored.
> - Make sure access_key and secret do not start with a digit.

```
radosgw-admin zonegroup placement modify --rgw-zonegroup ZONE_GROUP_NAME \
                                         --placement-id PLACEMENT_ID \
                                         --storage-class STORAGE_CLASS_NAME  \
                                         --tier-config=endpoint=AWS_ENDPOINT_URL,\

 access_key=AWS_ACCESS_KEY,secret=AWS_SECRET_KEY,\
                                         target_path="TARGET_BUCKET_ON_AWS",\
                                         multipart_sync_threshold=44432,\
                                         multipart_min_part_size=44432,\
                                         retain_head_object=true,\
                                         region=REGION_NAME,\
                                         location_constraint=REGION_NAME
```

The following are mandatory parameters:

access_key

The remote cloud S3 access key used for a specific connection.

secret

The secret key for the remote cloud S3 service.

endpoint

The URL of the remote cloud S3 service endpoint.

region

The remote cloud S3 service region name for AWS.

location_constraint

For Amazon S3 buckets in non-default AWS Regions, specifies the AWS location constraint. Set this value to the same AWS Region specified by **region**.

> **Note:** When using an Amazon S3 bucket in a non-default AWS Region, set **location_constraint** to the same value as **region**. This setting is required for successful object transition operations to buckets in non-default AWS Regions.

The following are optional parameters:

target_path

Defines how the target path is created. The target path specifies a prefix to which the source `bucket-name/object-name` is appended. If not specified, the **target_path** created is `rgwx-ZONE_GROUP_NAME-STORAGE_CLASS_NAME-cloud-bucket`.

target_storage_class

Defines the target storage class to which the object transitions. If not specified, the object is transitioned toSTANDARDstorage class.

retain_head_object

Set to`true`to retain the metadata of the object transitioned to cloud. Set to`false`to delete the object after transition. The default setting is`false`.

> **Note:** This option is ignored for current versioned objects.

multipart_sync_threshold

Specifies that objects this size or larger are transitioned to the cloud using multipart upload.

multipart_min_part_size

Specifies the minimum part size to use when transitioning objects using multipart upload.

glacier_restore_days

For use with `cloud-s3-glacier`. Specifies the number of days that the object can be restored on the Glacier or Tape endpoint. The default setting is`1`(1 day).

glacier_restore_tier_type

For use with `cloud-s3-glacier`. Specifies the restore retrieval type. The options are`Standard`or `Expedited`. The default setting is`Standard`.

For example,

```
[ceph: root@host01 /]# radosgw-admin zonegroup placement modify --rgw-zonegroup
default
                                                                --placement-id
default-placement \
                                                                --storage-class
CLOUDTIER \
                                                                --tier-
config=endpoint=http://10.0.210.010:8080,\

access_key=a21e86bce636c3aa2,secret=cf764951f1fdde5f,\
                                                                target_path="dfqe-
bucket-01",\

multipart_sync_threshold=44432,\

multipart_min_part_size=44432,\

retain_head_object=true,\
                                                                region=us-east-1,\

location_constraint=us-east-1

[
    {
        "key": "default-placement",
        "val": {
            "name": "default-placement",
            "tags": [],
            "storage_classes": [
                "CLOUDTIER",
                "STANDARD",
                "cold.test",
                "hot.test"
            ],
            "tier_targets": [
                {
                    "key": "CLOUDTIER",
                    "val": {
                        "tier_type": "cloud-s3",
                        "storage_class": "CLOUDTIER",
                        "retain_head_object": "true",
                        "s3": {
                            "endpoint": "http://10.0.210.010:8080",
                            "access_key": "a21e86bce636c3aa2",
                            "secret": "cf764951f1fdde5f",
                            "region": "us-east-1",
                            "location_constraint": "us-east-1",
                            "host_style": "path",
                            "target_storage_class": "",
                            "target_path": "dfqe-bucket-01",
                            "acl_mappings": [],
                            "multipart_sync_threshold": 44432,
                            "multipart_min_part_size": 44432
                        }
                    }
                }
            ]
        }
    }
]
```

4. Restart the Ceph Object Gateway.

```
ceph orch restart CEPH_OBJECT_GATEWAY_SERVICE_NAME
```

For example,

```
[ceph: root@host 01 /]# ceph orch restart rgw.rgw.1

Scheduled to restart rgw.rgw.1.host03.vkfldf on host 'host03'
```

5. Exit the `cephadm` shell and configure Amazon S3 on your bootstrapped node, as a root user.

```
s3cmd --configure
```

For example,

```
[root@host01 ~]# s3cmd --configure

Enter new values or accept defaults in brackets with Enter.
Refer to user manual for detailed description of all options.

Access key and Secret key are your identifiers for Amazon S3. Leave them empty for
using the env variables.
Access Key: a21e86bce636c3aa2
Secret Key: cf764951f1fdde5f
Default Region [US]:

Use "s3.amazonaws.com" for S3 Endpoint and not modify it to the target Amazon S3.
S3 Endpoint [s3.amazonaws.com]: 10.0.210.78:80

Use "%(bucket)s.s3.amazonaws.com" to the target Amazon S3. "%(bucket)s" and "%
(location)s" vars can be used
if the target S3 system supports dns based buckets.
DNS-style bucket+hostname:port template for accessing a bucket [%
(bucket)s.s3.amazonaws.com]: 10.0.210.78:80

Encryption password is used to protect your files from reading
by unauthorized persons while in transfer to S3
Encryption password:
Path to GPG program [/usr/bin/gpg]:

When using secure HTTPS protocol all communication with Amazon S3
servers is protected from 3rd party eavesdropping. This method is
slower than plain HTTP, and can only be proxied with Python 2.7 or newer
Use HTTPS protocol [Yes]: No

On some networks all internet access must go through a HTTP proxy.
Try setting it here if you can't connect to S3 directly
HTTP Proxy server name:

New settings:
  Access Key: a21e86bce636c3aa2
  Secret Key: cf764951f1fdde5f
  Default Region: US
  S3 Endpoint: 10.0.210.78:80
  DNS-style bucket+hostname:port template for accessing a bucket: 10.0.210.78:80
  Encryption password:
  Path to GPG program: /usr/bin/gpg
  Use HTTPS protocol: False
  HTTP Proxy server name:
  HTTP Proxy server port: 0

Test access with supplied credentials? [Y/n] Y
Please wait, attempting to list all buckets...
Success. Your access key and secret key worked fine :-)

Now verifying that encryption works...
Not configured. Never mind.

Save settings? [y/N] y
Configuration saved to '/root/.s3cfg'
```

6. Create the S3 bucket.

```
s3cmd mb s3://NAME_OF_THE_BUCKET_FOR_S3
```

For example,

```
[root@host01 ~]# s3cmd mb s3://awstestbucket

Bucket 's3://awstestbucket/' created
```

7. Create your file, input the needed data, and move it to S3 service.

```
s3cmd put FILE_NAME  s3://NAME_OF_THE_BUCKET_ON_S3
```

For example,

```
[root@host01 ~]# s3cmd put test.txt s3://awstestbucket

upload: 'test.txt' -> 's3://awstestbucket/test.txt'  [1 of 1]
 21 of 21   100% in    1s    16.75 B/s  done
```

8. Create the lifecycle configuration transition policy.

```
<LifecycleConfiguration>
  <Rule>
    <ID>RULE_NAME</ID>
    <Filter>
      <Prefix></Prefix>
    </Filter>
    <Status>Enabled</Status>
    <Transition>
      <Days>DAYS</Days>
      <StorageClass>STORAGE_CLASS_NAME</StorageClass>
    </Transition>
  </Rule>
</LifecycleConfiguration>
```

> **Note:** If an object that is temporarily or permanently restored has an existing lifecycle Transition or Expiration rule that is set to expire or transition the object before the restore duration ends, the object will be expired or transitioned according to the defined lifecycle rule.

For example,

```
[root@host01 ~]# cat lc_cloud.xml
<LifecycleConfiguration>
  <Rule>
    <ID>Archive all objects</ID>
    <Filter>
      <Prefix></Prefix>
    </Filter>
    <Status>Enabled</Status>
    <Transition>
      <Days>2</Days>
      <StorageClass>CLOUDTIER</StorageClass>
    </Transition>
  </Rule>
</LifecycleConfiguration>
```

9. Set the lifecycle configuration transition policy.

```
s3cmd setlifecycle FILE_NAME s3://NAME_OF_THE_BUCKET_FOR_S3
```

For example,

```
[root@host01 ~]#  s3cmd setlifecycle lc_config.xml s3://awstestbucket

s3://awstestbucket/: Lifecycle Policy updated
```

10. Log in to the `cephadm` shell.

```
cephadm shell
```

11. Restart the Ceph Object Gateway.

```
ceph orch restart CEPH_OBJECT_GATEWAY_SERVICE_NAME
```

For example,

```
[ceph: root@host 01 /]# ceph orch restart rgw.rgw.1

Scheduled to restart rgw.rgw.1.host03.vkfldf on host `host03
```

What to do next

1. On the source cluster, verify if the data has moved to S3 with **`radosgw-admin lc list`** command.

```
[ceph: root@host01 /]# radosgw-admin lc list
[
    {
        "bucket": ":awstestbucket:552a3adb-39e0-40f6-8c84-00590ed70097.54639.1",
```

```
        "started": "Mon, 26 Sep 2022 18:32:07 GMT",
        "status": "COMPLETE"
    }
]
```

2. Verify object transition at cloud endpoint.

```
radosgw-admin bucket list
```

For example,

```
[root@client ~]$ radosgw-admin bucket list
[
    "awstestbucket"
]
```

3. List the objects in the bucket.

```
aws s3api list-objects --bucket awstestbucket --endpoint=AWS_ENDPOINT_URL
```

For example,

```
[root@client ~]$ aws s3api list-objects --bucket awstestbucket --endpoint=http://
10.0.209.002:8080
{
    "Contents": [
        {
            "Key": "awstestbucket/test",
            "LastModified": "2022-08-25T16:14:23.118Z",
            "ETag": "\"378c905939cc4459d249662dfae9fd6f\"",
            "Size": 29,
            "StorageClass": "STANDARD",
            "Owner": {
                "DisplayName": "test-user",
                "ID": "test-user"
            }
        }
    ]
}
```

4. List the contents of the S3 bucket.

```
s3cmd ls s3://awstestbucket
```

For example,

```
[root@host01 ~]# s3cmd ls s3://awstestbucket
2022-08-25 09:57            0  s3://awstestbucket/test.txt
```

5. Verify the file information.

```
s3cmd info s3://awstestbucket/test.txt
```

For example,

```
[root@host01 ~]# s3cmd info s3://awstestbucket/test.txt
s3://awstestbucket/test.txt (object):
   File size: 0
   Last mod:  Mon, 03 Aug 2022 09:57:49 GMT
   MIME type: text/plain
   Storage:   CLOUDTIER
   MD5 sum:   991d2528bb41bb839d1a9ed74b710794
   SSE:       none
   Policy:    none
   CORS:      none
   ACL:       test-user: FULL_CONTROL
   x-amz-meta-s3cmd-attrs: atime:1664790668/ctime:1664790668/gid:0/gname:root/
md5:991d2528bb41bb839d1a9ed74b710794/mode:33188/mtime:1664790668/uid:0/uname:root
```

6. Download data locally from Amazon S3.
    a. Configure AWS, using the **`aws configure`** command.

For example,

```
[client@client01 ~]$ aws configure

AWS Access Key ID [****************6VVP]:
AWS Secret Access Key [****************pXqy]:
Default region name [us-east-1]:
Default output format [json]:
```

b. List the contents of the AWS bucket.
For example,

```
[client@client01 ~]$ aws s3 ls s3://dfqe-bucket-01/awstest
PRE awstestbucket/
```

c. Download the data from S3.
For example,

```
[client@client01 ~]$ aws s3 cp s3://dfqe-bucket-01/awstestbucket/test.txt

download: s3://dfqe-bucket-01/awstestbucket/test.txt to ./test.txt
```

Transitioning data to IBM Cloud Object Store (COS) service

You can transition data to an IBM Cloud Object Store (COS) service as part of the lifecycle configuration using storage classes to reduce cost and improve manageability. The transition is unidirectional and data cannot be transitioned back from the remote zone. This feature is to enable data transition to multiple cloud providers such as IBM Cloud Object Store (COS) service.

Procedure

- The procedure for setting up cloud transition to IBM COS is similar to Transitioning data to Amazon S3 cloud service.

> **Important:**
>
> - The following parameters are different in IBM COS:
>   - **target_path**
>   - **tier-config**
>   - **access_key**
>   - **secret_key**
>   - **region**
> - The only supported option for `glacier_restore_tier_type` is **NoTier**.

Transitioning data to Azure cloud service

You can transition data to a remote cloud service as part of the lifecycle configuration using storage classes to reduce cost and improve manageability. The transition is unidirectional and data cannot be transitioned back from the remote zone. This feature is to enable data transition to multiple cloud providers such as Azure. One of the key differences with the AWS configuration is that you need to configure the multi-cloud gateway (MCG) and use MCG to translate from the S3 protocol to Azure Blob.

Use `cloud-s3` as `tier-type` to configure the remote cloud S3 object store service to which the data needs to be transitioned. These do not need a data pool and are defined in terms of the zonegroup placement targets.

Prerequisites

- A running Red Hat Ceph Storage cluster with Ceph Object Gateway installed.
- User credentials for the remote cloud service, Azure.

- Azure configured locally to download data.
- `s3cmd` installed on the bootstrapped node.
- Azure container for the for MCG namespace created. In this example, it is `mcgnamespace`.

Procedure

1. Create a user with access key and secret key:

   Syntax

   ```
   radosgw-admin user create --uid=USER_NAME --display-name="DISPLAY_NAME" [--access-
   key ACCESS_KEY --secret-key SECRET_KEY]
   ```

   Example

   ```
   [ceph: root@host01 /]# radosgw-admin user create --uid=test-user --display-name="test-
   user" --access-key a21e86bce636c3aa1 --secret-key cf764951f1fdde5e
   {
       "user_id": "test-user",
       "display_name": "test-user",
       "email": "",
       "suspended": 0,
       "max_buckets": 1000,
       "subusers": [],
       "keys": [
           {
               "user": "test-user",
               "access_key": "a21e86bce636c3aa1",
               "secret_key": "cf764951f1fdde5e"
           }
       ],
       "swift_keys": [],
       "caps": [],
       "op_mask": "read, write, delete",
       "default_placement": "",
       "default_storage_class": "",
       "placement_tags": [],
       "bucket_quota": {
           "enabled": false,
           "check_on_raw": false,
           "max_size": -1,
           "max_size_kb": 0,
           "max_objects": -1
       },
       "user_quota": {
           "enabled": false,
           "check_on_raw": false,
           "max_size": -1,
           "max_size_kb": 0,
           "max_objects": -1
       },
       "temp_url_keys": [],
       "type": "rgw",
       "mfa_ids": []
   }
   ```

2. As a root user, configure AWS CLI with the user credentials and create a bucket with default placement:

   Syntax

   ```
   aws s3 --ca-bundle CA_PERMISSION --profile rgw  --endpoint ENDPOINT_URL --region
   default mb s3://BUCKET_NAME
   ```

   Example

   ```
   [root@host01 ~]$ aws s3   --ca-bundle  /etc/pki/ca-trust/source/anchors/myCA.pem --
   profile rgw  --endpoint https://host02.example.com:8043 --region default mb s3://
   transition
   ```

3. Verify that the bucket is using `default-placement` with the placement rule:

   Example

```
[root@host01 ~]# radosgw-admin bucket stats --bucket transition
{
    "bucket": "transition",
    "num_shards": 11,
    "tenant": "",
    "zonegroup": "b29b0e50-1301-4330-99fc-5cdcfc349acf",
    "placement_rule": "default-placement",
    "explicit_placement": {
        "data_pool": "",
        "data_extra_pool": "",
        "index_pool": ""
    },
```

4. Log into the OpenShift Container Platform (OCP) cluster with OpenShift Data Foundation (ODF) deployed:

   Example

```
[root@host01 ~]$ oc project openshift-storage
[root@host01 ~]$ oc get clusterversion
NAME      VERSION   AVAILABLE   PROGRESSING   SINCE   STATUS
version   4.11.6    True        False         4d1h    Cluster version is 4.11.6

[root@host01 ~]$ oc get storagecluster
NAME                 AGE   PHASE   EXTERNAL   CREATED AT             VERSION
ocs-storagecluster   4d    Ready              2023-06-27T15:23:01Z   4.11.0
```

5. Configure the multi-cloud gateway (MCG) namespace Azure bucket running on an OCP cluster in Azure:

   Syntax

```
noobaa namespacestore create azure-blob az --account-key='ACCOUNT_KEY' --account-
name='ACCOUNT_NAME' --target-blob-container='AZURE_CONTAINER_NAME'
```

   Example

```
[root@host01 ~]$ noobaa namespacestore create azure-blob  az --account-
key='iq3+6hRtt9bQ46QfHKQ0nSm2aP+tyMzdn8dBSRW4XWrFhY+1nwfqEj4hk2q66nmD85E/
o5OrrUqo+AStkKwm9w==' --account-name='transitionrgw' --target-blob-
container='mcgnamespace'
```

6. Create an MCG bucket class pointing to the `namespacestore`:

   Example

```
[root@host01 ~]$ noobaa bucketclass create namespace-bucketclass single aznamespace-
bucket-class --resource az -n openshift-storage
```

7. Create an object bucket claim (OBC) for the transition to cloud:

   Syntax

```
noobaa obc create OBC_NAME --bucketclass aznamespace-bucket-class -n openshift-storage
```

   Example

```
[root@host01 ~]$ noobaa obc create rgwobc --bucketclass aznamespace-bucket-class -n
openshift-storage
```

> **Note:** Use the credentials provided by OBC to configure zonegroup placement on the Ceph Object Gateway.

8. On the bootstrapped node, create a storage class with the tier type as `cloud-s3` on the default placement within the default zonegroup on the previously configured MCG in Azure:

> **Note:** Once a storage class is created with the `--tier-type=cloud-s3` option , it cannot be later modified to any other storage class type.

Syntax

```
radosgw-admin zonegroup placement add --rgw-zonegroup =ZONE_GROUP_NAME \
                            --placement-id=PLACEMENT_ID \
                            --storage-class =STORAGE_CLASS_NAME \
                            --tier-type=cloud-s3
```

Example

```
[ceph: root@host01 /]# radosgw-admin zonegroup placement add --rgw-zonegroup=default \
                                                    --placement-id=default-placement \
                                                    --storage-class=AZURE \
                                                    --tier-type=cloud-s3
[
    {
        "key": "default-placement",
        "val": {
            "name": "default-placement",
            "tags": [],
            "storage_classes": [
                "AZURE",
                "STANDARD"
            ],
            "tier_targets": [
                {
                    "key": "AZURE",
                    "val": {
                        "tier_type": "cloud-s3",
                        "storage_class": "AZURE",
                        "retain_head_object": "false",
                        "s3": {
                            "endpoint": "",
                            "access_key": "",
                            "secret": "",
                            "host_style": "path",
                            "target_storage_class": "",
                            "target_path": "",
                            "acl_mappings": [],
                            "multipart_sync_threshold": 33554432,
                            "multipart_min_part_size": 33554432
                        }
                    }
                }
            ]
        }
    }
]
```

9. Configure the cloud S3 cloud storage class:

   Syntax

```
radosgw-admin zonegroup placement modify --rgw-zonegroup ZONE_GROUP_NAME \
                                         --placement-id PLACEMENT_ID \
                                         --storage-class STORAGE_CLASS_NAME  \
                                         --tier-config=endpoint=ENDPOINT_URL,\
                                         access_key=ACCESS_KEY,secret=SECRET_KEY,\
                                         target_path="TARGET_BUCKET_ON",\
                                         multipart_sync_threshold=44432,\
                                         multipart_min_part_size=44432,\
                                         retain_head_object=true
                                         region=REGION_NAME
```

   Important: Setting the `retain_head_object` parameter to `true` retains the metadata or the head of the object to list the objects that are transitioned.

10. Example

```
[ceph: root@host01 /]# radosgw-admin zonegroup placement modify --rgw-zonegroup
default
                                                                --placement-id
default-placement \
                                                                --storage-class AZURE
\
```

```
                                                                --tier-
config=endpoint="https://s3-openshift-storage.apps.ocp410.0e73azopenshift.com",\

access_key=a21e86bce636c3aa2,secret=cf764951f1fdde5f,\
                                                                target_path="dfqe-
bucket-01",\

multipart_sync_threshold=44432,\

multipart_min_part_size=44432,\

retain_head_object=true
                                                                region= us-east-1

[
    {
        "key": "default-placement",
        "val": {
            "name": "default-placement",
            "tags": [],
            "storage_classes": [
                "AZURE",
                "STANDARD",
                "cold.test",
                "hot.test"
            ],
            "tier_targets": [
                {
                    "key": "AZURE",
                    "val": {
                        "tier_type": "cloud-s3",
                        "storage_class": "AZURE",
                        "retain_head_object": "true",
                        "s3": {
                            "endpoint": "https://s3-openshift-
storage.apps.ocp410.0e73azopenshift.com",
                            "access_key": "a21e86bce636c3aa2",
                            "secret": "cf764951f1fdde5f",
                            "region": "",
                            "host_style": "path",
                            "target_storage_class": "",
                            "target_path": "dfqe-bucket-01",
                            "acl_mappings": [],
                            "multipart_sync_threshold": 44432,
                            "multipart_min_part_size": 44432
                        }
                    }
                }
            ]
        }
    }
]
```

11. Restart the Ceph Object Gateway:

    Syntax

    ```
    ceph orch restart CEPH_OBJECT_GATEWAY_SERVICE_NAME
    ```

    Example

    ```
    [ceph: root@host 01 /]# ceph orch restart client.rgw.objectgwhttps.host02.udyllp

    Scheduled to restart client.rgw.objectgwhttps.host02.udyllp on host 'host02
    ```

12. Create the lifecycle configuration transition policy for the bucket created previously. In this example, the bucket is `transition`:

    Syntax

    ```
    cat transition.json
    {
        "Rules": [
            {
                "Filter": {
                    "Prefix": ""
                },
    ```

```
            "Status": "Enabled",
            "Transitions": [
                {
                    "Days": 30,
                    "StorageClass": "STORAGE_CLASS"
                }
            ],
            "ID": "TRANSITION_ID"
        }
    ]
        }
```

Note: All the objects in the bucket older than 30 days are transferred to the cloud storage class called AZURE.

Example

```
[root@host01 ~]$ cat transition.json
{
    "Rules": [
        {
            "Filter": {
                "Prefix": ""
            },
            "Status": "Enabled",
            "Transitions": [
                {
                    "Days": 30,
                    "StorageClass": "AZURE"
                }
            ],
            "ID": "Transition Objects in bucket to AZURE Blob after 30 days"
        }
    ]
        }
```

13. Apply the bucket lifecycle configuration using AWS CLI:

    Syntax

    ```
     aws s3api --ca-bundle CA_PERMISSION --profile rgw --endpoint ENDPOINT_URL --region
    default put-bucket-lifecycle-configuration --lifecycle-configuration  file://
    BUCKET.json --bucket BUCKET_NAME
    ```

    Example

    ```
    [root@host01 ~]$ aws s3api --ca-bundle  /etc/pki/ca-trust/source/anchors/myCA.pem --
    profile rgw  --endpoint https://host02.example.com:8043 --region default put-bucket-
    lifecycle-configuration --lifecycle-configuration  file://transition.json --bucket
    transition
    ```

14. Optional: Get the lifecycle configuration:

    Syntax

    ```
     aws s3api --ca-bundle CA_PERMISSION --profile rgw --endpoint ENDPOINT_URL--region
    default get-bucket-lifecycle-configuration --lifecycle-configuration  file://
    BUCKET.json --bucket BUCKET_NAME
    ```

    Example

    ```
    [root@host01 ~]$ aws s3api   --ca-bundle  /etc/pki/ca-trust/source/anchors/myCA.pem --
    profile rgw  --endpoint https://host02.example.com:8043 --region default get-bucket-
    lifecycle-configuration --bucket transition
    {
        "Rules": [
            {
                "ID": "Transition Objects in bucket to AZURE Blob after 30 days",
                "Prefix": "",
                "Status": "Enabled",
                "Transitions": [
    ```

```
                {
                    "Days": 30,
                    "StorageClass": "AZURE"
                }
            ]
        }
    ]
}
```

15. Optional: Get the lifecycle configuration with the **`radosgw-admin lc list`** command:

```
[root@host 01 ~]# radosgw-admin lc list
[
    {
        "bucket": ":transition:d9c4f708-5598-4c44-9d36-849552a08c4d.169377.1",
        "started": "Thu, 01 Jan 1970 00:00:00 GMT",
        "status": "UNINITIAL"
    }
]
```

> **Note:** The `UNINITAL` status implies that the lifecycle configuration is not processed. It moves to `COMPLETED` state after the transition process is complete.

16. Log in to `cephadm shell`:

```
[root@host 01 ~]# cephadm shell
```

17. Restart the Ceph Object Gateway daemon:

    Syntax

```
ceph orch daemon CEPH_OBJECT_GATEWAY_DAEMON_NAME
```

    Example

```
[ceph: root@host 01 /]# ceph orch daemon restart rgw.objectgwhttps.host02.udyllp
[ceph: root@host 01 /]# ceph orch daemon restart rgw.objectgw.host02.afwvyq
[ceph: root@host 01 /]# ceph orch daemon restart rgw.objectgw.host05.ucpsrr
```

18. Migrate data from the source cluster to Azure:

    Example

```
[root@host 01 ~]# for i in 1 2 3 4 5
do
aws s3 --ca-bundle  /etc/pki/ca-trust/source/anchors/myCA.pem --profile rgw --endpoint
https://host02.example.com:8043 --region default cp /etc/hosts s3://transition/
transition$i
done
```

19. Verify transition of data:

    Example

```
[root@host 01 ~]# aws s3 --ca-bundle  /etc/pki/ca-trust/source/anchors/myCA.pem --
profile rgw  --endpoint https://host02.example.com:8043 --region default ls s3://
transition
2023-06-30 10:24:01       3847 transition1
2023-06-30 10:24:04       3847 transition2
2023-06-30 10:24:07       3847 transition3
2023-06-30 10:24:09       3847 transition4
2023-06-30 10:24:13       3847 transition5
```

20. Verify if the data has moved to Azure with `rados ls` command:

    Example

```
[root@host 01 ~]# rados ls -p default.rgw.buckets.data | grep transition
d9c4f708-5598-4c44-9d36-849552a08c4d.169377.1_transition1
d9c4f708-5598-4c44-9d36-849552a08c4d.169377.1_transition4
d9c4f708-5598-4c44-9d36-849552a08c4d.169377.1_transition2
d9c4f708-5598-4c44-9d36-849552a08c4d.169377.1_transition3
d9c4f708-5598-4c44-9d36-849552a08c4d.169377.1_transition5
```

21. If the data is not transitioned, you can run the `lc process` command:

    Example

    ```
    [root@host 01 ~]# radosgw-admin lc process
    ```

    This will force the lifecycle process to start and evaluates all the bucket lifecycle policies configured. It then starts the transition of data wherever needed.

Verification

1. Run the `radosgw-admin lc list` command to verify the completion of the transition:

   Example

   ```
   [root@host 01 ~]# radosgw-admin lc list
   [
       {
           "bucket": ":transition:d9c4f708-5598-4c44-9d36-849552a08c4d.170017.5",
           "started": "Mon, 30 Jun 2023-06-30 16:52:56 GMT",
           "status": "COMPLETE"
       }
   ]
   ```

2. List the objects in the bucket:

   Example

   ```
   [root@host01 ~]$ aws s3api list-objects --bucket awstestbucket --endpoint=http://
   10.0.209.002:8080
   {
       "Contents": [
           {
               "Key": "awstestbucket/test",
               "LastModified": "2023-06-25T16:14:23.118Z",
               "ETag": "\"378c905939cc4459d249662dfae9fd6f\"",
               "Size": 29,
               "StorageClass": "STANDARD",
               "Owner": {
                   "DisplayName": "test-user",
                   "ID": "test-user"
               }
           }
       ]
   }
   ```

3. List the objects on the cluster:

   Example

   ```
   [root@host01 ~]$ aws s3 --ca-bundle  /etc/pki/ca-trust/source/anchors/myCA.pem --
   profile rgw  --endpoint https://host02.example.com:8043 --region default ls s3://
   transition
   2023-06-30 17:52:56          0 transition1
   2023-06-30 17:51:59          0 transition2
   2023-06-30 17:51:59          0 transition3
   2023-06-30 17:51:58          0 transition4
   2023-06-30 17:51:59          0 transition5
   ```

   The objects are `0` in size. You can list the objects, but cannot copy them since they are transitioned to Azure.

4. Check the head of the object using the S3 API:

   Example

```
[root@host01 ~]$ aws s3api --ca-bundle  /etc/pki/ca-trust/source/anchors/myCA.pem --
profile rgw  --endpoint https://host02.example.com:8043 --region default head-object
--key transition1 --bucket transition
{
    "AcceptRanges": "bytes",
    "LastModified": "2023-06-31T16:52:56+00:00",
    "ContentLength": 0,
    "ETag": "\"46ecb42fd0def0e42f85922d62d06766\"",
    "ContentType": "binary/octet-stream",
    "Metadata": {},
    "StorageClass": "CLOUDTIER"
}
```

You can see that the storage class has changed from `STANDARD` to `CLOUDTIER`.

Restoring objects from S3 cloud-tier storage

You can restore the objects from the cloud back into Ceph Object Gateway cluster using S3 Restore API on the transitioned object.

Before you begin

Ensure the following prerequisites are met before restoring objects from S3 cloud-tier storage.

- Target path created on Amazon S3.
- **`retain_head_object`** should be set to`true`while configuring cloudtier storage-class to be able to restore from cloud service.
- `s3cmd` installed on the bootstrapped node.
- Ensure user credentials provided for that cloud-tier storage class remain valid for the `restore` function to work properly.
- For `cloud-s3-glacier` tier types, be sure that the **`glacier_restore_days`** and **`glacier_restore_tier_type`** options are correctly set. For more information, see “Transitioning data to Amazon S3 cloud service” on page 1061.

About this task

By default, objects are restored to STANDARD storage-class. However, you can configure the storage class to which the objects need to be restored to by setting the `restore_storage_class` option.

> **Note:**
> If the null version of an object is not the latest version, do not specify `version-id null` when issuing a `restore-object` request.
>
> For example, suppose an object named `object1` is uploaded to a bucket called `testbucket1`, transitioned to the cloud, and then restored. Later, versioning is enabled on `testbucket1`, and a new version of `object1` is uploaded. At this point, the original (null) version is no longer the latest.
>
> In this case, when restoring `object1`, you should omit the `--version-id null` parameter from the `restore-object`command.

Procedure

Restore an object, using one of the following procedures.

- Restore cloud transitioned objects by using the S3 **`restore-object`** command.

```
aws s3api restore-object
          --bucket <value>
          --key <value>
          [--version-id <value>]
          --restore-request (structure)
{
            // for temporary restore
            "Days": integer, // "Days" is optional and if not provided, the object will
```

```
be restored permanently.
          }
```

> **Note:** The restoration period of the temporary copies can be updated by reissuing the request with a new period.

The following example demonstrates how to restore the object `doc1.rtf` from the specified version for a period of 10 days.

```
aws s3api restore-object
--bucket bucket1
--key doc1.rtf [--version-id
3sL4kqtJlcpXroDTDmJ+rmSpXd3dIbrHY+MTRCxf3vjVBH40Nr8X8gdRQBpUMLUo]
--restore-request "Days": 10
```

The following example demonstrates how to restore the object `doc2.rtf` permanently, where it is treated as a regular object.

```
aws s3api restore-object  --bucket bucket1 --key doc2.rtf --restore-request {}
```

> **Note:** If a restore request for an object is already issued or in progress, the `s3api` restore request with the `--debug` flag displays a `RestoreAlreadyInProgress` message.

– Restore and read the transitioned objects by using the `read-through` feature.

> **Note:** The object copy restored via `read-through` is temporary and is retained only for the duration of `read_through_restore_days`.

Ensure that the following parameters are set:

- `"Allow_read_through": “enable"`
- `“read_through_restore_days”: 10`

```
--storage-class CLOUDTIER
--tier-config=endpoint=http://XX.XX.XX.XX:YY,
access_key=<access_key>,secret=<secret>,
retain_head_object=true,
restore_storage_class=COLDTIER,
allow_read_through=true,
read_through_restore_days=10
```

Following is an example to restore a object using the S3 Get Object when read-through is enabled.

```
aws s3api get-object  --bucket bucket1 --key doc3.rtf
```

Following are the examples of tier configuration.

Example for restoring an object from `cloud-s3`tier-type

```
tier-type = cloud-s3
tier-config =
{
"access_key": xxx,
"secret": xxx,
"endpoint": http://10.0.210.010:8080
“,
"region": “”,
"host_style": “path”,
"acls": [ { "type": “id”,
"source_id": “”,
"dest_id": “” } ... ],
"target_path": dfqe-bucket-01,
"target_storage_class": “”,
"multipart_sync_threshold": 44432,
"multipart_min_part_size": 44432,
"retain_head_object": “true”,
“Allow_read_through”: “enable”,
“read_through_restore_days”: 10
}
```

Note: The transition and restore logic works if the target storage class matches the actual storage class configured in the cloud endpoint that can archive objects.

Example for restoring an object from `cloud-s3-glacier` tier-type

```
tier-type = cloud-s3-glacier
tier-config =
{
"access_key": xxx,
"secret": xxx,
"endpoint": http://s3.us-east-1.amazonaws.com
“,
"region": “us-east-1”,
"host_style": “path”,
"acls": [ { "type": “id”,
"source_id": “”,
"dest_id": “” } ... ],
"target_path": rgwbucket,
"target_storage_class": “GLACIER”,
"multipart_sync_threshold": 44432,
"multipart_min_part_size": 44432,
"retain_head_object": “true”,
“Allow_read_through”: “enable”,
“read_through_restore_days”: 10
"restore_storage_class": "COLDTIER",
"s3-glacier": {
    "glacier_restore_days": 2,
    "glacier_restore_tier_type": "Expedited"
}
```

What to do next

Verify the status of the restore by running an S3 head-object request with the specified parameters. Check the status of the restore using the S3 `head-object` request.

For example,

```
[root@host01 ~]$ aws s3api --ca-bundle
/etc/pki/ca-trust/source/anchors/myCA.pem --profile rgw
--endpoint https://host02.example.com:8043 --region default head-object
--key transition1 --bucket transition
```

Using the radosgw-admin cli for cloud restore operations

Use the **`radosgw-admin`** cli to check the cloud-restore status for a specific object or to list all objects in a bucket that are restored or in progress.

Before you begin

- Administrator privileges to run radosgw-admin commands.
- radosgw-admin tool is installed and available in your $PATH.
- Target bucket exists in the cluster.
- Restore request is initiated for the object (for status checks).

.

Procedure

1. To display detailed restore status for a specific object in the bucket.

```
radosgw-admin restore status --bucket <bucket-name> --object <object-name>
```

```
{
    "name": "new",
    "RestoreExpiryDate": "2025-08-21T13:58:46.970540Z",
    "RestoreStatus": "CloudRestored",
    "RestoreType": "Temporary",
    "RestoreTime": "2025-08-21T13:36:39.618810Z"
}
```

   Following are the parameter descriptions:

   --bucket

   Name of the bucket.

   --object

   Name of the object whose restore status is queried.

2. To list objects in the bucket that are restored or are in the process of being restored.

```
radosgw-admin restore list --bucket <bucket-name>
```

   Following is a sample output (no filter):

```
{
    "file": "CloudRestored",
    "new": "CloudRestored"
}
```

   Following is a sample output (with filter):

```
radosgw-admin  restore list --bucket <bucket name> --restore-status=CloudRestored
 {
  "file": "CloudRestored"
  }
```

   Following are the parameter descriptions:

   --bucket

   Name of the bucket.

   --restore-status=<status>

   Filter objects by restore status: `RestoreAlreadyInProgress`, `CloudRestored`, or `RestoreFailed`.

   Note: `--restore-status=<status>` is an optional parameter.

3. To display the count of restore operations in the **`bucket stats`** command.

```
radosgw-admin bucket stats --bucket <bucket-name> --show-restore-stats
```

Following is the sample output:

```
{
  "restore_stats": {
    "restore_completed_count": 0,
    "restore_in_progress_count": 0,
    "restore_failed_count": 0
  }
}
```

Following are the parameter descriptions:

--bucket

Name of the bucket to query restore statistics.

--show-restore-stats

Displays restore statistics. Must be set explicitly, statistics are not shown by default.

Viewing Ceph Object Gateway per-user and per-bucket usage statistics (Technology Preview)

Use the Ceph Object Gateway performance counters to view per-user and per-bucket usage statistics, such as total bytes used and number of objects. Performance counters are available through the Ceph admin socket and use a cache to provide accurate, low-latency metrics with minimal overhead.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Ceph Object Gateway is installed and running.
- Object gateway login credentials are configured.
- At least one user and one bucket exist and are active (not suspended).
- Configure the following parameters to the Ceph Object Gateway service and then restart the `rgw` service to apply the changes, by using the **`ceph orch restart rgw`** command.

> **Note:** This feature is disabled by default. Configuring these parameters enables the per-user and per-bucket view feature.

```
ceph config set client.rgw rgw_enable_usage_perf_counters true
ceph config set client.rgw rgw_usage_cache_path /tmp/usage_cache.mdb
ceph config set client.rgw rgw_usage_cache_max_size 1073741824
```

About this task

> **Important:** Technology Preview features are not supported with production service level agreements (SLAs), might not be functionally complete, and does not recommend using them for production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

Procedure

1. Create a user.

```
radosgw-admin user create
```

For example,

```
radosgw-admin user create \
 --uid=testuser \
 --display-name="Test User" \
 --access-key=test \
 --secret-key=test
```

2. Configure a S3 client.
For example,

```
[default]
access_key = test
secret_key = test
host_base = localhost:8000
host_bucket = localhost:8000
use_https = False
signature_v2 = True
```

3. Create buckets and upload objects.
For example,

```
[default]
access_key = test
secret_key = test
host_base = localhost:8000
host_bucket = localhost:8000
use_https = False
signature_v2 = True
```

4. View usage counters through the admin socket.

> **Note:** Counter statistics are not real time. After uploading objects, counters may take up to 40 minutes to reflect changes.

```
SOCKET=out/radosgw.8000.asok
# ceph --admin-daemon $SOCKET perf dump | grep -A 20 rgw_usage
```

Example counters include:

- `rgw_usage` – Global counters
- `rgw_usage_cache` – Cache statistics
- `rgw_user_testuser` – User-specific statistics
- `rgw_bucket_bucket1` – Bucket-specific statistics

Example output for **`rgw_user_testuser`**

```
"used_bytes": 15728640,
"num_objects": 3
```

Example output for **`rgw_bucket_bucket1`**

```
"used_bytes": 5242880,
"num_objects": 2
```

5. Verify cache behavior.
   a. List a bucket multiple times.

```
s3cmd ls s3://bucket1
s3cmd ls s3://bucket1
s3cmd ls s3://bucket1
```

b. Check the cache hit counter.

```
ceph --admin-daemon $SOCKET perf dump | grep cache_hit
```

Result

You can now view `per-user` and `per-bucket usage` statistics for Ceph Object Gateway, confirm that usage counters are active, and verify cache behavior.

Testing and validation

As a storage administrator, you can do basic functionality testing to verify that the Ceph Object Gateway environment is working as expected. You can use the REST interfaces by creating an initial Ceph Object Gateway user for the S3 interface, and then create a subuser for the Swift interface.

Prerequisites

- A healthy running Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway software.

Create an S3 user

To test the gateway, create an S3 user and grant the user access. The **`man radosgw-admin`** command provides information on additional command options.

> **Note:** In a multi-site deployment, always create a user on a host in the primary zone of the primary zone group.

Prerequisites

- `root` or `sudo` access
- Ceph Object Gateway installed

Procedure

1. Create an S3 user:

   Syntax

   ```
   radosgw-admin user create --uid=name --display-name="USER_NAME"
   ```

   Replace *name* with the name of the S3 user:

   Example

   ```
   [root@host01 ~]# radosgw-admin user create --uid="testuser" --display-name="Jane Doe"
   {
       "user_id": "testuser",
       "display_name": "Jane Doe",
       "email": "",
       "suspended": 0,
       "max_buckets": 1000,
       "auid": 0,
       "subusers": [],
       "keys": [
           {
               "user": "testuser",
               "access_key": "CEP28KDIQXBKU4M15PDC",
               "secret_key": "MARoio8HFc8JxhEilES3dKFVj8tV3NOOYymihTLO"
           }
       ],
       "swift_keys": [],
       "caps": [],
   ```

```
    "op_mask": "read, write, delete",
    "default_placement": "",
    "placement_tags": [],
    "bucket_quota": {
        "enabled": false,
        "check_on_raw": false,
        "max_size": -1,
        "max_size_kb": 0,
        "max_objects": -1
    },
    "user_quota": {
        "enabled": false,
        "check_on_raw": false,
        "max_size": -1,
        "max_size_kb": 0,
        "max_objects": -1
    },
    "temp_url_keys": [],
    "type": "rgw"
}
```

2. Verify the output to ensure that the values of `access_key` and `secret_key` do not include a JSON escape character (\). These values are needed for access validation, but certain clients cannot handle if the values include JSON escape characters. To fix this problem, perform one of the following actions:
   - Remove the JSON escape character.
   - Encapsulate the string in quotes.
   - Regenerate the key and ensure that it does not include a JSON escape character.
   - Specify the key and secret manually.

   Do not remove the forward slash / because it is a valid character.

Create a Swift user

To test the Swift interface, create a Swift subuser. Creating a Swift user is a two-step process. The first step is to create the user. The second step is to create the secret key.

> **Note:** In a multi-site deployment, always create a user on a host in the primary zone of the primary zone group.

Prerequisites

- A running, and healthy Red Hat Ceph Storage cluster.
- Installation of the Ceph Object Gateway.
- Root-level access to the Ceph Object Gateway node.

Procedure

1. Create the Swift user:

   Syntax

   ```
   radosgw-admin subuser create --uid=NAME --subuser=NAME:swift --access=full
   ```

   Replace `NAME` with the Swift user name, for example:

   Example

   ```
   [root@host01 ~]# radosgw-admin subuser create --uid=testuser --subuser=testuser:swift
   --access=full
   {
       "user_id": "testuser",
       "display_name": "First User",
   ```

```
    "email": "",
    "suspended": 0,
    "max_buckets": 1000,
    "auid": 0,
    "subusers": [
        {
            "id": "testuser:swift",
            "permissions": "full-control"
        }
    ],
    "keys": [
        {
            "user": "testuser",
            "access_key": "O8JDE41XMI74O185EHKD",
            "secret_key": "i4Au2yxG5wtr1JK01mI8kjJPM93HNAoVWOSTdJd6"
        }
    ],
    "swift_keys": [
        {
            "user": "testuser:swift",
            "secret_key": "13TLtdEW7bCqgttQgPzxFxziu0AgabtOc6vM8DLA"
        }
    ],
    "caps": [],
    "op_mask": "read, write, delete",
    "default_placement": "",
    "placement_tags": [],
    "bucket_quota": {
        "enabled": false,
        "check_on_raw": false,
        "max_size": -1,
        "max_size_kb": 0,
        "max_objects": -1
    },
    "user_quota": {
        "enabled": false,
        "check_on_raw": false,
        "max_size": -1,
        "max_size_kb": 0,
        "max_objects": -1
    },
    "temp_url_keys": [],
    "type": "rgw"
}
```

2. Create the secret key:

   Syntax

   ```
   radosgw-admin key create --subuser=NAME:swift --key-type=swift --gen-secret
   ```

   Replace `NAME` with the Swift user name, for example:

   Example

   ```
   [root@host01 ~]# radosgw-admin key create --subuser=testuser:swift --key-type=swift --
   gen-secret
   {
       "user_id": "testuser",
       "display_name": "First User",
       "email": "",
       "suspended": 0,
       "max_buckets": 1000,
       "auid": 0,
       "subusers": [
           {
               "id": "testuser:swift",
               "permissions": "full-control"
           }
       ],
       "keys": [
           {
               "user": "testuser",
               "access_key": "O8JDE41XMI74O185EHKD",
               "secret_key": "i4Au2yxG5wtr1JK01mI8kjJPM93HNAoVWOSTdJd6"
           }
       ],
       "swift_keys": [
   ```

```
        {
            "user": "testuser:swift",
            "secret_key": "a4ioT4jEP653CDcdU8p4OuhruwABBRZmyNUbnSSt"
        }
    ],
    "caps": [],
    "op_mask": "read, write, delete",
    "default_placement": "",
    "placement_tags": [],
    "bucket_quota": {
        "enabled": false,
        "check_on_raw": false,
        "max_size": -1,
        "max_size_kb": 0,
        "max_objects": -1
    },
    "user_quota": {
        "enabled": false,
        "check_on_raw": false,
        "max_size": -1,
        "max_size_kb": 0,
        "max_objects": -1
    },
    "temp_url_keys": [],
    "type": "rgw"
}
```

Test S3 access

You need to write and run a Python test script for verifying S3 access. The S3 access test script will connect to the `radosgw`, create a new bucket, and list all buckets. The values for `aws_access_key_id` and `aws_secret_access_key` are taken from the values of `access_key` and `secret_key` returned by the **`radosgw_admin`** command.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the nodes.

Procedure

1. Enable high availability repository.
2. Install the `python3-boto3` package:

```
dnf install python3-boto3
```

3. Create the Python script:

```
vi s3test.py
```

4. Add the following contents to the file:

   Syntax

```
import boto3

endpoint = "" # enter the endpoint _URL_ along with the port "http://URL:PORT"

access_key = 'ACCESS'
secret_key = 'SECRET'

s3 = boto3.client(
        's3',
        endpoint_url=endpoint,
        aws_access_key_id=access_key,
        aws_secret_access_key=secret_key
        )

s3.create_bucket(Bucket='my-new-bucket')
```

```
response = s3.list_buckets()
for bucket in response['Buckets']:
    print("{name}\t{created}".format(
                name = bucket['Name'],
                created = bucket['CreationDate']
))
```

- Replace `endpoint` with the URL of the host where you have configured the gateway service. That is, the `gateway host`. Ensure that the `host` setting resolves with DNS. Replace `PORT` with the port number of the gateway.
- Replace `ACCESS` and `SECRET` with the `access_key` and `secret_key` values

5. Run the script:

```
python3 s3test.py
```

The output will be something like the following:

```
my-new-bucket 2022-05-31T17:09:10.000Z
```

Test Swift access

Swift access can be verified via the `swift` command line client. The **`man swift`** command provides more information on available command line options.

To install the `swift` client, run the following command:

```
sudo yum install python-setuptools
sudo easy_install pip
sudo pip install --upgrade setuptools
sudo pip install --upgrade python-swiftclient
```

To test swift access, run the following command:

Syntax

```
# swift -A http://IP_ADDRESS:PORT/auth/1.0 -U testuser:swift -K 'SWIFT_SECRET_KEY' list
```

Replace `IP_ADDRESS` with the public IP address of the gateway server and `SWIFT_SECRET_KEY` with its value from the output of the `radosgw-admin key create` command issued for the `swift` user. Replace *PORT* with the port number you are using with Beast. If you do not replace the port, it will default to port `80`.

For example:

```
swift -A http://10.10.143.116:80/auth/1.0 -U testuser:swift -K
'244+fz2gSqoHwR3lYtSbIyomyPHf3i7rgSJrF/IA' list
```

The output should be:

```
my-new-bucket
```

Configuration reference

As a storage administrator, you can set various options for the Ceph Object Gateway. These options contain default values. If you do not specify each option, then the default value is set automatically.

To set specific values for these options, update the configuration database by using the **`ceph config set client.rgw OPTION VALUE`** command.

General settings

Use this information to get general settings for configuring Ceph Object Gateway.

Table 119: Ceph Object Gateway general configuration settings

Name	Description	Type	Default
`rgw_data`	Sets the location of the data files for Ceph Object Gateway.	String	`/var/lib/ceph/radosgw/$cluster-$id`
`rgw_enable_apis`	Enables the specified APIs.	String	`s3, s3website, swift, swift_auth, admin, sts, iam, notifications`
`rgw_cache_enabled`	Whether the Ceph Object Gateway cache is enabled.	Boolean	`true`
`rgw_cache_lru_size`	The number of entries in the Ceph Object Gateway cache.	Integer	`10000`
`rgw_socket_path`	The socket path for the domain socket. `FastCgiExternalServer` uses this socket. If you do not specify a socket path, Ceph Object Gateway will not run as an external server. The path you specify here must be the same as the path specified in the `rgw.conf` file.	String	N/A
`rgw_host`	The host for the Ceph Object Gateway instance. Can be an IP address or a hostname.	String	`0.0.0.0`
`rgw_port`	Port the instance listens for requests. If not specified, Ceph Object Gateway runs external FastCGI.	String	None
`rgw_dns_name`	The DNS name of the served domain. See also the `hostnames` setting within zone groups.	String	None
`rgw_script_uri`	The alternative value for the `SCRIPT_URI` if not set in the request.	String	None
`rgw_request_uri`	The alternative value for the `REQUEST_URI` if not set in the request.	String	None
`rgw_print_continue`	Enable `100-continue` if it is operational.	Boolean	`true`
`rgw_remote_addr_param`	The remote address parameter. For example, the HTTP field containing the remote address, or the `X-Forwarded-For` address if a reverse proxy is operational.	String	`REMOTE_ADDR`
`rgw_op_thread_timeout`	The timeout in seconds for open threads.	Integer	600

Name	Description	Type	Default
`rgw_op_thread_suicide_timeout`	The `timeout` in seconds before a Ceph Object Gateway process dies. Disabled if set to `0`.	Integer	`0`
`rgw_thread_pool_size`	The size of the thread pool.	Integer	512 threads.
`rgw_num_control_oids`	The number of notification objects used for cache synchronization between different `rgw` instances.	Integer	`8`
`rgw_init_timeout`	The number of seconds before Ceph Object Gateway gives up on initialization.	Integer	`30`
`rgw_mime_types_file`	The path and location of the MIME types. Used for Swift auto-detection of object types.	String	`/etc/mime.types`
`rgw_gc_max_objs`	The maximum number of objects that may be handled by garbage collection in one garbage collection processing cycle.	Integer	`32`
`rgw_gc_obj_min_wait`	The minimum wait time before the object may be removed and handled by garbage collection processing.	Integer	`2 * 3600`
`rgw_gc_processor_max_time`	The maximum time between the beginning of two consecutive garbage collection processing cycles.	Integer	`3600`
`rgw_gc_processor_period`	The cycle time for garbage collection processing.	Integer	`3600`
`rgw_s3 success_create_obj_status`	The alternate success status response for `create-obj`.	Integer	`0`
`rgw_resolve_cname`	Whether `rgw` should use the DNS CNAME record of the request hostname field (if hostname is not equal to `rgw_dns name`).	Boolean	`false`
`rgw_object_stripe_size`	The size of an object stripe for Ceph Object Gateway objects.	Integer	`4 << 20`

Name	Description	Type	Default
`rgw_extended_http_attrs`	Add a new set of attributes that could be set on an object. These extra attributes can be set through HTTP header fields when putting the objects. If set, these attributes will return as HTTP fields when doing GET/HEAD on the object.	String	None. For example: "content_foo, content_bar"
`rgw_exit_timeout_secs`	Number of seconds to wait for a process before exiting unconditionally.	Integer	`120`
`rgw_get_obj_window_size`	The window size in bytes for a single object request.	Integer	`16 << 20`
`rgw_get_obj_max_req_size`	The maximum request size of a single get operation sent to the Ceph Storage Cluster.	Integer	`4 << 20`
`rgw_relaxed_s3_bucket_names`	Enables relaxed S3 bucket names rules for zone group buckets.	Boolean	`false`
`rgw_list buckets_max_chunk`	The maximum number of buckets to retrieve in a single operation when listing user buckets.	Integer	`1000`
`rgw_override_bucket_index_max_shards`	The number of shards for the bucket index object. A value of `0` indicates there is no sharding. Red Hat does not recommend setting a value too large (for example, `1000`) as it increases the cost for bucket listing. This variable should be set in the `[client]` or the `[global]` section so it is automatically applied to `radosgw-admin` commands.	Integer	`0`
`rgw_curl_wait_timeout_ms`	The timeout in milliseconds for certain `curl` calls.	Integer	`1000`
`rgw_copy_obj_progress`	Enables output of object progress during long copy operations.	Boolean	`true`
`rgw_copy_obj_progress_every_bytes`	The minimum bytes between copy progress output.	Integer	`1024 * 1024`
`rgw_admin_entry`	The entry point for an admin request URL.	String	`admin`

Name	Description	Type	Default
`rgw_content_length_compat`	Enable compatibility handling of FCGI requests with both CONTENT_LENGTH AND HTTP_CONTENT_LENGTH set.	Boolean	`false`
`rgw_bucket_default_quota_max_objects`	The default maximum number of objects per bucket. This value is set on new users if no other quota is specified. It has no effect on existing users. This variable should be set in the `[client]` or the `[global]` section so it is automatically applied to `radosgw-admin` commands.	Integer	`-1`
`rgw_bucket_quota_ttl`	The amount of time in seconds cached quota information is trusted. After this timeout, the quota information will be re-fetched from the cluster.	Integer	600
`rgw_user_quota_bucket_sync_interval`	The amount of time in seconds bucket quota information is accumulated before syncing to the cluster. During this time, other RGW instances will not see the changes in bucket quota stats from operations on this instance.	Integer	180
`rgw_user_quota_sync_interval`	The amount of time in seconds user quota information is accumulated before syncing to the cluster. During this time, other RGW instances will not see the changes in user quota stats from operations on this instance.	Integer	3600 * 24
`log_meta`	A zone parameter to determine whether or not the gateway logs the metadata operations.	Boolean	`false`
`log_data`	A zone parameter to determine whether or not the gateway logs the data operations.	Boolean	`false`

Name	Description	Type	Default
`sync_from_all`	A `radosgw-admin` command to set or unset whether zone syncs from all zonegroup peers.	Boolean	`false`
`rgw_curl_tcp_keepalive`	Enables TCP keepalive on the HTTP client sockets managed by libcurl. This does not apply to connections received by the HTTP frontend, but only to HTTP requests sent by radosgw. Examples include requests to Keystone for authentication, sync requests from multisite, and requests to key management servers for SSE.	Boolean	`false`

About pools

Red Hat Ceph Storage zones map to a series of Ceph Storage Cluster pools.

If the user key for the Ceph Object Gateway contains write capabilities, the gateway has the ability to create pools automatically. This is convenient for getting started. However, the Ceph Object Storage Cluster uses the placement group default values unless they were set in the Ceph configuration file. Additionally, Ceph will use the default CRUSH hierarchy. These settings are **NOT** ideal for production systems.

The default pools for the Ceph Object Gateway's default zone include:

- `.rgw.root`
- `.default.rgw.control`
- `.default.rgw.meta`
- `.default.rgw.log`
- `.default.rgw.buckets.index`
- `.default.rgw.buckets.data`
- `.default.rgw.buckets.non-ec`

The Ceph Object Gateway creates pools on a per zone basis. If you create the pools manually, prepend the zone name. The system pools store objects related to, for example, system control, logging, and user information. By convention, these pool names have the zone name prepended to the pool name.

- `.<zone-name>.rgw.control`: The control pool.
- `.<zone-name>.log`: The log pool contains logs of all bucket/container and object actions, such as create, read, update, and delete.
- `.<zone-name>.rgw.buckets.index`: This pool stores the index of the buckets.
- `.<zone-name>.rgw.buckets.data`: This pool stores the data of the buckets.
- `.<zone-name>.rgw.meta`: The metadata pool stores `user_keys` and other critical metadata.
- `.<zone-name>.meta:users.uid`: The user ID pool contains a map of unique user IDs.
- `.<zone-name>.meta:users.keys`: The keys pool contains access keys and secret keys for each user ID.
- `.<zone-name>.meta:users.email`: The email pool contains email addresses associated with a user ID.
- `.<zone-name>.meta:users.swift`: The Swift pool contains the Swift subuser information for a user ID.

Ceph Object Gateways store data for the bucket index (`index_pool`) and bucket data (`data_pool`) in placement pools. These may overlap; that is, you may use the same pool for the index and the data. The index pool for default placement is `{zone-name}.rgw.buckets.index` and for the data pool for default placement is `{zone-name}.rgw.buckets`.

Name	Description	Type	Default
`rgw_zonegroup_root_pool`	The pool for storing all zone group-specific information.	String	`.rgw.root`
`rgw_zone_root_pool`	The pool for storing zone-specific information.	String	`.rgw.root`

Lifecycle settings

As a storage administrator, you can set various bucket lifecycle options for a Ceph Object Gateway. These options contain default values. If you do not specify each option, then the default value is set automatically.

To set specific values for these options, update the configuration database by using the **`ceph config set client.rgw OPTION VALUE`** command.

Table 121: Ceph Object Gateway bucket lifecycle settings

Name	Description	Type	Default
`rgw_lc_debug_interval`	For developer use only to debug lifecycle rules by scaling expiration rules from days into an interval in seconds. Red Hat recommends that this option not be used in a production cluster.	Integer	`-1`
`rgw_lc_lock_max_time`	The timeout value used internally by the Ceph Object Gateway.	Integer	`90`
`rgw_lc_max_objs`	Controls the sharding of the RADOS Gateway internal lifecycle work queues, and should only be set as part of a deliberate resharding workflow. Red Hat recommends not changing this setting after the setup of your cluster, without first contacting Red Hat Support.	Integer	`32`
`rgw_lc_max_rules`	The number of lifecycle rules to include in one, per bucket, lifecycle configuration document. The Amazon Web Service (AWS) limit is 1000 rules.	Integer	`1000`

Name	Description	Type	Default
`rgw_lc_max_worker`	The number of lifecycle worker threads to run in parallel, processing bucket and index shards simultaneously. Red Hat does not recommend setting a value larger than 10 without contacting Red Hat Support.	Integer	`3`
`rgw_lc_max_wp_worker`	The number of buckets that each lifecycle worker thread can process in parallel. Red Hat does not recommend setting a value larger than 10 without contacting Red Hat Support.	Integer	`3`
`rgw_lc_thread_delay`	A delay, in milliseconds, that can be injected into shard processing at several points. The default value is 0. Setting a value from 10 to 100 ms would reduce CPU utilization on RADOS Gateway instances and reduce the proportion of workload capacity of lifecycle threads relative to ingest if saturation is being observed.	Integer	`0`

Swift settings

Use this information to get Ceph Object Gateway Swift settings.

Table 122: Ceph Object Gateway Swift settings

Name	Description	Type	Default
`rgw_enforce_swift_acls`	Enforces the Swift Access Control List (ACL) settings.	Boolean	`true`
`rgw_swift_token_expiration`	The time in seconds for expiring a Swift token.	Integer	`24 * 3600`
`rgw_swift_url`	The URL for the Ceph Object Gateway Swift API.	String	None
`rgw_swift_url_prefix`	The URL prefix for the Swift API, for example, `http://fqdn.com/swift`.	`swift`	N/A

Name	Description	Type	Default
rgw_swift_auth_url	Default URL for verifying v1 auth tokens (if not using internal Swift auth).	String	None
rgw_swift_auth_entry	The entry point for a Swift auth URL.	String	auth

Logging settings

Use this information to get Ceph Object Gateway logging settings.

Table 123: Ceph Object Gateway logging settings

Name	Description	Type	Default
debug_rgw_datacache	Low level D3N logs can be enabled by the debug_rgw_datacache subsystem (up to debug_rgw_datacache=30)	Integer	1/5
rgw_log_nonexistent_bucket	Enables Ceph Object Gateway to log a request for a non-existent bucket.	Boolean	false
rgw_log_object_name	The logging format for an object name. See manpage date for details about format specifiers.	Date	%Y-%m-%d-%H-%i-%n
rgw_log_object_name_utc	Whether a logged object name includes a UTC time. If false, it uses the local time.	Boolean	false
rgw_usage_max_shards	The maximum number of shards for usage logging.	Integer	32
rgw_usage_max_user_shards	The maximum number of shards used for a single user's usage logging.	Integer	1
rgw_enable_ops_log	Enable logging for each successful Ceph Object Gateway operation.	Boolean	false
rgw_enable_usage_log	Enable the usage log.	Boolean	false
rgw_ops_log_rados	Whether the operations log should be written to the Ceph Storage Cluster backend.	Boolean	true
rgw_ops_log_socket_path	The Unix domain socket for writing operations logs.	String	None

Name	Description	Type	Default
`rgw_ops_log_data-backlog`	The maximum data backlog data size for operations logs written to a Unix domain socket.	Integer	`5 << 20`
`rgw_usage_log_flush_threshold`	The number of dirty merged entries in the usage log before flushing synchronously.	Integer	1024
`rgw_usage_log_tick_interval`	Flush pending usage log data every `n` seconds.	Integer	`30`
`rgw_intent_log_object_name`	The logging format for the intent log object name. See manpage date for details about format specifiers.	Date	`%Y-%m-%d-%i-%n`
`rgw_intent_log_object_name_utc`	Whether the intent log object name includes a UTC time. If `false`, it uses the local time.	Boolean	`false`
`rgw_data_log_window`	The data log entries window in seconds.	Integer	`30`
`rgw_data_log_changes_size`	The number of in-memory entries to hold for the data changes log.	Integer	`1000`
`rgw_data_log_num_shards`	The number of shards (objects) on which to keep the data changes log. **Note:** Changing the value is not supported.	Integer	`128`
`rgw_data_log_obj_prefix`	The object name prefix for the data log.	String	`data_log`
`rgw_replica_log_obj_prefix`	The object name prefix for the replica log.	String	`replica log`
`rgw_md_log_max_shards`	The maximum number of shards for the metadata log.	Integer	`64`
`rgw_log_http_headers`	Comma-delimited list of HTTP headers to include with ops log entries. Header names are case insensitive, and use the full header name with words separated by underscores.	String	None

Keystone settings

Use this information to get Ceph Object Gateway Keystone settings.

Table 124: Ceph Object Gateway Keystone settings

Name	Description	Type	Default
`rgw_keystone_url`	The URL for the Keystone server.	String	None
`rgw_keystone_admin_token`	The Keystone admin token (shared secret).	String	None
`rgw_keystone_accepted_roles`	The roles required to serve requests.	String	`Member, admin`
`rgw_keystone_token_cache_size`	The maximum number of entries in each Keystone token cache.	Integer	`10000`

Keystone integration configuration options

Integrate your configuration options into Keystone.

See “Keystone integration configuration options” on page 1098 for details of the available Keystone integration configuration options.

> **Important:** After updating the Ceph configuration file, you must copy the new Ceph configuration file to all Ceph nodes in the storage cluster.

Table 125: Keystone integration configuration options

Name	Description	Type	Default
`rgw_s3_auth_use_keystone`	If set to `true`, the Ceph Object Gateway authenticates users by using Keystone.	Boolean	`false`
`nss_db_path`	The path to the NSS database.	String	`""`
`rgw_keystone_url`	The URL for the administrative RESTful API on the Keystone server.	String	`""`
`rgw_keystone_admin_token`	The token or shared secret that is configured internally in Keystone for administrative requests.	String	`""`
`rgw_keystone_admin_user`	The keystone admin username.	String	`""`
`rgw_keystone_admin_password`	The keystone admin user password.	String	`""`
`rgw_keystone_admin_tenant`	The Keystone admin user tenant for keystone v2.0.	String	`""`
`rgw_keystone_admin_project`	The Keystone admin user project for keystone v3.	String	`""`

Name	Description	Type	Default
`rgw_trust_forwarded_https`	When a proxy in front of the Ceph Object Gateway is used for SSL termination, it does not whether incoming HTTP connections are secure. Enable this option to trust the forwarded and X-forwarded headers sent by the proxy when determining when the connection is secure. This is mainly required for server-side encryption.	Boolean	`false`
`rgw_swift_account_in_url`	Whether the Swift account is encoded in the URL path. You *must* set this option to `true` and update the Keystone service catalog if you want the Ceph Object Gateway to support publicly-readable containers and temporary URLs.	Boolean	`false`
`rgw_keystone_admin_domain`	The Keystone admin user domain.	String	`""`
`rgw_keystone_api_version`	The version of the Keystone API to use. Valid options are `2` or `3`.	Integer	`2`
`rgw_keystone_accepted_roles`	The roles required to serve requests.	String	`"member, Member, admin"`
`rgw_keystone_accepted_admin_roles`	The list of roles allowing a user to gain administrative privileges.	String	`ResellerAdmin, swiftoperator`
`rgw_keystone_token_cache_size`	The maximum number of entries in the Keystone token cache.	Integer	`10000`
`rgw_max_attr_name_len`	The maximum length of metadata name. 0 skips the check.	Size	`0`
`rgw_max_attrs_num_in_req`	The maximum number of metadata items that can be put with a single request.	Unit	`0`
`rgw_max_attr_size`	The maximum length of metadata value. 0 skips the check.	Size	`0`
`rgw_swift_versioning_enabled`	Enabling Swift versioning.	Boolean	`0` or `1`
`rgw_keystone_accepted_reader_roles`	List of roles that can only be used for reads.	String	`""`
`rgw_swift_enforce_content_length`	Send content length when listing containers.	String	`false`

Name	Description	Type	Default
`rgw_keystone_verify_ssl`	If `true` Ceph tries to verify Keystone's SSL certificate.	Boolean	`true`
`rgw_keystone_implicit_tenants`	Create new users in their own tenants of the same name. Set this to `true` or `false` under most circumstances. This has the effect of splitting the identity space such that only the indicated protocol uses implicit tenants.	String	`false`

LDAP settings

Use this information to get Ceph Object Gateway LDAP settings.

Table 126: Ceph Object Gateway LDAP settings

Name	Description	Type	Example
`rgw_ldap_uri`	A space-separated list of LDAP servers in URI format.	String	`ldaps://<ldap.your.domain>`
`rgw_ldap_searchdn`	The LDAP search domain name, also known as base domain.	String	`cn=users,cn=accounts,dc=example,dc=com`
`rgw_ldap_binddn`	The gateway will bind with this LDAP entry (user match).	String	`uid=admin,cn=users,dc=example,dc=com`
`rgw_ldap_secret`	A file containing credentials for `rgw_ldap_binddn`.	String	`/etc/openldap/secret`
`rgw_ldap_dnattr`	LDAP attribute containing Ceph Object Gateway user names (to form binddns).	String	`uid`

Red Hat Ceph Storage NFS cluster and share management (OpenStack Manila users only)

Create and manage NFS file services, including clusters, shares, QoS, security, monitoring, backup and restore, and troubleshooting.

Important: The NFS gateway in Red Hat Ceph Storage is supported only when used with Red Hat OpenStack Services on OpenShift (RHOSO). Standalone use of the Ceph NFS service is not supported.

Before you begin, make sure that you have the following prerequisites in place:

- Installation and configuration of the Ceph Metadata Server daemons (`ceph-mds`).
- Create and mount a Ceph File System.

Note: This documentation uses *NFS share* instead of the earlier term *NFS export*. In the Ceph Dashboard, labels and controls may still refer to NFS export.

High availability for NFS services

Red Hat Ceph Storage supports highly available NFS deployments by running multiple NFS instances behind an ingress service.

A typical high-availability deployment includes:

- Multiple NFS service instances
- HAProxy for load balancing
- Keepalived for virtual IP (VIP) management

This architecture provides a single customer access endpoint while distributing requests across available NFS gateways.

NFSv4 ACL behavior and limitations

When a Ceph File System (CephFS) file system is shared through the NFS service, CephFS stores access control information as POSIX ACLs. The NFS service presents NFSv4 ACLs to customer systems by translating between the NFSv4 and POSIX ACL models. POSIX ACLs are the authoritative store. The NFSv4 ACL that the NFS client retrieves is a derived view.

"POSIX to NFSv4 permission mapping" on page 1101 shows how POSIX permission bits stored in CephFS map to the NFSv4 permissions that customer systems see. The ACL letters are the abbreviations that the **`nfs4_getfacl`** and **`nfs4_setfacl`** commands use.

Table 127: POSIX to NFSv4 permission mapping

POSIX permission	NFSv4 permissions	NFSv4 ACL letters
`r`	`READ_DATA` on files, or `LIST_DIRECTORY` on directories	`r`
`w`	`WRITE_DATA` and `APPEND_DATA`. On directories, also `DELETE_CHILD`.	`w`, `a`, and `D` on directories
`w` on the owner entry (`USER_OBJ`)	Also `WRITE_ACL` and `WRITE_ATTR`	`C`, `T`
`x`	`EXECUTE`	`x`
Any ALLOW entry	`READ_ACL`, `READ_ATTR`, and `SYNCHRONIZE`	`c`, `t`, `y`

POSIX ACL entries map to NFSv4 principals as follows:

USER_OBJ

Maps to `OWNER@`

GROUP_OBJ

Maps to `GROUP@`

OTHER

Maps to `EVERYONE@`

Named users

Map to their UID

Named groups

Map to their GID with the `GROUP_ID` flag (`g`)

Because the two ACL models use different permission and inheritance semantics, some NFSv4 ACL behavior cannot be represented exactly. The following specific behaviors apply:

- The NFS service applies the POSIX mask to the owning group (`GROUP_OBJ`), named user, and named group entries and does not emit it as a separate ACE. Permissions that the mask removes are absent from those entries as if they were never granted.
- POSIX ACLs have no DENY entry type. The NFS service advertises ALLOW-only ACL support. However, when a customer system retrieves an ACL by running **`nfs4_getfacl`**, the NFS service adds DENY ACEs to the returned NFSv4 ACL where needed. These DENY ACEs give the same effective permissions as the underlying POSIX ACL. They can restrict any permission, including read, write, and execute. They are not stored in CephFS.
- The NFS service adds `READ_ACL`, `READ_ATTR`, and `SYNCHRONIZE` to every ALLOW ACE unconditionally. These are not separate POSIX entries.
- The following NFSv4 permissions have no POSIX equivalent and are not preserved: `DELETE` (`d`), `WRITE_OWNER` (`o`), and the named attribute permissions (`n` and `N`). These permissions do not appear in the ACL that customers retrieve.
- Some NFSv4 permission combinations cannot be preserved exactly after translation.
- The NFS service converts default ACL entries with `FILE_INHERIT`, `DIR_INHERIT`, and `INHERIT_ONLY` flags. As a result, file-only and directory-only inheritance cannot be distinguished, because both use the same flags. Other inheritance behaviors that cannot be expressed in a POSIX default ACL are modified during translation.
- The NFSv4 ACL displayed to a customer system can differ from the ACL that was originally applied, including after a read-modify-write cycle.
- Effective access permissions are determined by the POSIX ACLs stored in CephFS, not by the NFSv4 view presented to the NFS client.

When the same CephFS data is accessed through multiple protocols, such as NFS, ACL representations can differ between customer systems because both protocols derive permissions from the underlying POSIX ACLs stored in CephFS.

NFS failover behavior

NFS maintains protocol state information, including:

- Locks
- Sessions
- Delegations

Traditional NFS failover relies on a grace-period-based recovery process. During the grace period, clients reclaim state information before normal operation resumes.

As a result:

- New requests can be delayed during recovery.
- Applications can experience temporary I/O delays.

- Client state must be recovered before normal operation resumes.

For deployments that use NFS transparent state migration, client state information can be maintained across participating NFS servers to reduce recovery delays during failover. For more information, see NFS transparent state migration (Technology Preview).

Load-balancing considerations

For multi-gateway deployments:

- Use HAProxy as the ingress load-balancing service.
- Configure client affinity when required to maintain NFS session consistency.
- Use a virtual IP (VIP) to provide a single service endpoint.
- Monitor gateway utilization and client distribution.

HAProxy and Keepalived provide a reference architecture for deploying highly available NFS services with multiple gateway instances.

Creating an NFS cluster

Create an NFS cluster with the **`ceph nfs cluster create`** command. This creates a common recovery pool for all NFS service daemons, new user based on the cluster name, and a common NFS service configuration RADOS object.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running, and healthy Red Hat Ceph Storage cluster.
- An existing Ceph File System.
- Root-level access to Ceph Monitor.
- Installation of the `nfs-pool`, `nfs-pool-ceph`, `nfspool-rados-grace`, and `nfs-pool-rados-urls` packages on the Ceph Manager hosts.
- Root-level access to the client.

About this task

> **Note:** NFS cluster creation can currently be done only with the command-line interface. Creating NFS clusters through the dashboard is not currently supported.

Procedure

1. Log into the `cephadm` shell.
   For example,

   ```
   [root@mds ~]# cephadm shell
   ```

2. Enable the Ceph Manager NFS module.

   ```
   ceph mgr module enable nfs
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph mgr module enable nfs
   ```

3. Create an NFS service cluster.

   ```
   ceph nfs cluster create CLUSTER_NAME [PLACEMENT] [--ingress] [--virtual_ip IP_ADDRESS]
   [--ingress-mode {default|keepalive-only|haproxy-standard|haproxy-protocol}] [--port
   PORT]
   ```

> **Important:** Red Hat only supports one NFS service daemon running per host.

For example,

```
[ceph: root@host01 /]# ceph nfs cluster create nfs-cephfs "host01 host02"
NFS Cluster Created Successfully
```

In this example, the NFS service cluster name is `nfs-cephfs` and the daemon containers are deployed to `host01`, and `host02`.

4. Verify the NFS service cluster information.

```
ceph nfs cluster info [CLUSTER_NAME]
```

> **Note:** Specifying the *CLUSTER_NAME* is optional.

For example,

```
[ceph: root@host01 /]# ceph nfs cluster info {
"nfs-cephfs": {
"backend": [
{
"hostname": "host01",
"ip": "10.8.129.4",
"port": 2049
},
{
"hostname": "host02",
"ip": "10.8.129.4",
"port": 2049
}
],
"virtual_ip": null
}
}
```

Customize an NFS configuration

Customize a Ceph-managed NFS cluster by supplying a user configuration file. The user configuration is stored and managed by `cephadm` and is merged with the generated defaults at runtime. The user configuration takes precedence only for supported parameters.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running, and healthy Red Hat Ceph Storage cluster.
- Root-level access to a Ceph Metadata Server (MDS) node.
- An NFS cluster created using the `ceph nfs cluster create` command.

About this task

Use this procedure to add or update supported NFS settings, such as logging, export defaults, and selected TLS policies for an existing NFS cluster. The configuration is stored by Ceph (in RADOS) and overlaid at runtime on the container-generated base configuration.

Procedure

1. Create a nfs-ganesha.conf file with custom parameters.

```
[root@ceph8-node1 ~]# cat ganesha2.conf
NFS_CORE_PARAM {
 Enable_NLM = true
 NFS_Protocols = 3,4;
```

```
NFS_Port = 2050;
 MNT_Port = 4048;
 mount_path_pseudo = true;
}
```

2. Apply the custom configuration using below command.

```
$ ceph nfs cluster config set singletest -i PATH_TO_CONFIG_FILE
```

For example,

```
$ ceph nfs cluster config set nfs-cephfs -i ganesha2.conf
```

3. Get the custom config, check port in netstat and check Ganesha.conf inside container.

```
[root@ceph8-node1 ~]# ceph nfs cluster config get singletest
NFS_CORE_PARAM {
 Enable_NLM = true
 NFS_Protocols = 3,4;
NFS_Port = 2050;
 MNT_Port = 4048;
 mount_path_pseudo = true;
}
[root@ceph8-node1 ~]# netstat -tnlp | grep -i nfs
tcp6    0   0 :::2049         :::*          LISTEN   1476200/ganesha.nfs
tcp6    0   0 :::50051        :::*          LISTEN   1476200/ganesha.nfs
tcp6    0   0 :::9587         :::*          LISTEN   1476200/ganesha.nfs
tcp6    0   0 :::46089        :::*          LISTEN   1476200/ganesha.nfs
[root@ceph8-node1 ~]# cephadm shell
[ceph: root@ceph8-node1 /]# /bin/rados -p .nfs -N singletest get userconf-
nfs.singletest -
NFS_CORE_PARAM {
 Enable_NLM = true
 NFS_Protocols = 3,4;
NFS_Port = 2050;
 MNT_Port = 4048;
 mount_path_pseudo = true;
}
```

Table 128: NFS customization parameters

Parameter	Default/Value	Description
Enable_NLM	true	Enables Network Lock Manager to support NFSv3 locking semantics; set to true when NFSv3 clients require file locks.
NFS_Protocols	3,4	Specifies the NFS protocol versions the server supports and listens for.
NFS_Port	2050	Intended TCP service port for NFS. In Ceph-managed NFS, port binding is controlled at the service layer (for example, during cluster creation with a service port) and this overlay value does not change the listening socket.
MNT_Port	4048	Intended mountd port. Not applied via the user configuration overlay when the service is managed by cephadm.
mount_path_pseudo	true	Enables pseudo-path mounting for NFSv3, aligning its behavior with NFSv4-style pseudo roots.

Result

– The NFS cluster applies the user-defined configuration successfully, and the updated settings can be verified through the **`ceph nfs cluster config get`** command and functional validation.

Remove customized NFS configurations

Remove previously applied user-defined NFS configuration overrides and restore the NFS cluster to its default configuration blocks.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running and healthy Red Hat Ceph Storage cluster.
- The NFS cluster name.
- You have administrative access to run Ceph CLI commands.

About this task

The reset commands removes all user-defined parameters that were previously applied with the **`ceph nfs cluster config set`** command. The reset operation reverts the NFS service to its default cephadm-managed configuration and removes the related user configuration object from the `.nfs` pool.

Procedure

1. Remove the user-defined configuration.

   ```
   ceph nfs cluster config reset CLUSTER_NAME
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph nfs cluster config reset nfs-cephfs
   ```

2. Optional: Verify that the configuration was removed.

   ```
   ceph nfs cluster config get CLUSTER_NAME
   ```

   > **Note:** If no user configuration is present, the command returns either an empty result or only the default values managed by cephadm.

Result

The user-defined NFS configuration is removed, and the cluster reverts to the default cephadm-generated settings. The verification command confirm that no custom configuration remains, and the service operates using only the default configuration blocks.

Sharing Ceph File System namespaces over the NFS protocol

Ceph File Systems (CephFS) namespaces can be shares over the NFS protocol using a NFS service file server. To share a CephFS namespace, you must first have a running NFS service cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running, and healthy Red Hat Ceph Storage cluster.
- An NFS cluster created using the **`ceph nfs cluster create`** command.

About this task

> **Important:**
>
> - NFSv3 (version 3) is supported and enabled for clients that do not yet support NFSv4 mounts for CephFS file shares.

Use NFSv4 for clients and servers that support NFSv4.2 or later.

- NFS clients are unable to create CephFS snapshots through their native NFS mount. They must use server-side operator tooling for their snapshot needs.

Procedure

1. Create a CephFS share.

   Important: Due to a known issue, do not use the **cmount_path** option when creating an NFS share. If a value other than / is used, NFS shares that were previously defined with the / path become inaccessible.

   ```
   ceph nfs export create cephfs CLUSTER_NAME BINDING FILE_SYSTEM_NAME [--readonly] [--
   path=PATH_WITHIN_CEPHFS]
   ```

   In the following example, the *BINDING* (/ceph) is the pseudo root path, which must be unique, and an absolute path.

   ```
   [ceph: root@host01 /]# ceph nfs export create cephfs nfs-cephfs /ceph cephfs01 --
   path=/
   {
   "bind": "/ceph",
   "cluster": "nfs-cephfs",
   "fs": "cephfs01",
   "mode": "RW",
   "path": "/"
   }
   ```

   Note:

   - The **--readonly** option shares a path with the read-only permission, the default being read, and write permissions.
   - The **PATH_WITHIN_CEPHFS** can be a subvolume. You can get the absolute subvolume path by using the **subvolume getpath** command.

     ```
     ceph fs subvolume getpath VOLUME_NAME SUBVOLUME_NAME [--
     group_name SUBVOLUME_GROUP_NAME]
     ```

     For example,

     ```
     [ceph: root@host01 /]# ceph fs subvolume getpath cephfs sub0
     ```

2. View the share block based on the pseudo root name.

   ```
   ceph nfs export get CLUSTER_NAME BINDING
   ```

   For example,

```
[ceph: root@host01 /]# ceph nfs export get nfs-cephfs /ceph
{
"access_type": "RW",
"clients": [],
"cluster_id": "nfsservice",
"export_id": 1,
"fsal": {
"cmount_path": "/", "fs_name": "cephfs", "name": "CEPH", "user_id":
"nfs.nfsservice.cephfs"
},
"path": "/",
"protocols": [
3, 4
],
"pseudo": "/ganesha1",
"security_label": true,
"squash": "none",
"transports": [
"TCP"
]
}
```

3. List the NFS shares.

```
ceph nfs export ls CLUSTER_NAME [--detailed]
```

For example,

```
[ceph: root@host01 /]# ceph nfs export ls nfs-cephfs --detailed
{
"access_type": "RW",
"clients": [],
"cluster_id": "nfsservice",
"export_id": 1,
"fsal": {
"cmount_path": "/", "fs_name": "cephfs", "name": "CEPH", "user_id":
"nfs.nfsservice.cephfs"
},
"path": "/",
"protocols": [
3, 4
],
"pseudo": "/service1",
"security_label": true,
"squash": "none",
"transports": [
"TCP"
]
}
```

4. Get the information of the NFS share.

```
ceph nfs export info CLUSTER_NAME [PSEUDO_PATH]
```

For example,

```
[ceph: root@host01 /]# ceph nfs export info nfs-cephfs /ceph
{
"access_type": "RW",
"clients": [],
"cluster_id": "nfsservice",
"export_id": 1,
"fsal": {
"cmount_path": "/", "fs_name": "cephfs", "name": "CEPH", "user_id":
"nfs.nfsservice.cephfs"
},
"path": "/",
"protocols": [
3, 4
],
"pseudo": "/ganesha1",
"security_label": true,
"squash": "none",
"transports": [
"TCP"
]
}
```

5. On a client host, mount the shared Ceph File System.

```
mount -t nfs -o port=GANESHA_PORT HOST_NAME:BINDING LOCAL_MOUNT_POINT
```

For example,

```
[root@client01 ~]# mount -t nfs -o port=2049 host01:/ceph/ /mnt/nfs-cephfs
```

a. To automatically mount on boot, open and edit the `/etc/fstab` file by adding a new line.

```
HOST_NAME:BINDING LOCAL_MOUNT_POINT nfs4
defaults,seclabel,vers=4.2,proto=tcp,port=2049  0  0
```

> **Note:** For NFSv3 mounts, use **`vers`** = 3.

For example,

```
[root@client01 ~]# host01:/ceph/   /mnt/nfs-cephfs   nfs4
defaults,seclabel,vers=4.2,proto=tcp,port=2049  0  0
```

6. On a client host, mount a shared NFS Ceph File System, created with an `ingress` service.

```
mount -t nfs VIRTUAL_IP_ADDRESS:BINDING LOCAL_MOUNT_POINT
```

- Replace *VIRTUAL_IP_ADDRESS* with **`--ingress --virtual-ip`** IP address used to create the NFS cluster.
- Replace *BINDING* with the pseudo root path.
- Replace *LOCAL_MOUNT_POINT* with the mount point to mount the share on.

The following example mounts the share `nfs-cephfs` that exists on a NFS cluster created with `--ingress --virtual-ip 10.10.128.75` on the `/mnt` mount point .

```
[root@client01 ~]# mount -t nfs 10.10.128.75:/nfs-cephfs /mnt
```

Modifying NFS shares

Modify Ceph File System share parameters with a configuration file.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running, and healthy Red Hat Ceph Storage cluster.
- An NFS share created.

About this task

You can modify the following parameters in a share with a configuration file:

`access_type`

This can be `RW`, `RO`, or `NONE`.

`squash`

This can be `No_Root_Squash`, `None`, or `Root_Squash`.

`security_label`

This can be `true` or `false`.

Note: This documentation uses *NFS share* instead of the earlier term *NFS export*. In the Ceph Dashboard, labels and controls may still refer to NFS export.

Procedure

1. View the share block based on the pseudo-root name.

```
ceph nfs export get CLUSTER_NAME BINDING
```

For example,

```
[ceph: root@host01 /]# ceph nfs export get nfs-cephfs /ceph
{
"access_type": "RW",
"clients": [],
"cluster_id": "nfsservice",
"export_id": 1,
"fsal": {
"cmount_path": "/", "fs_name": "cephfs", "name": "CEPH", "user_id":
"nfs.nfsservice.cephfs"
},
"path": "/",
"protocols": [
3, 4
],
"pseudo": "/ganesha1",
"security_label": true,
"squash": "none",
"transports": [
"TCP"
]
}
```

2. Share the configuration file.

```
ceph nfs export get nfs-cephfs /ceph > export.conf
```

For example,

```
[ceph: root@host01 /]# ceph nfs export get nfs-cephfs /ceph > export.conf
```

3. Edit the share information.

Note: For NFSv3 mounts (NFS version 3), use **`protocols`** = 3.

```
{
  "export_id": EXPORT_ID,
  "path": "PATH",
  "cluster_id": "CLUSTER_NAME",
  "pseudo": "CLUSTER_PSEUDO_PATH",
  "access_type": "RW/RO",
  "squash": "SQUASH",
  "security_label": SECURITY_LABEL,
  "protocols": [
    PROTOCOL_ID
  ],
  "transports": [
    "TCP"
  ],
  "fsal": {
    "name": "NAME",
    "user_id": "USER_ID",
    "fs_name": "FILE_SYSTEM_NAME",
    "sec_label_xattr": ""
  },
  "clients": []
}
```

In the following example, the `access_type` is modified from `RO` to `RW`.

```
[ceph: root@host01 /]# vi export.conf

{
"access_type": "RW",
"clients": [],
"cluster_id": "nfservice",
"export_id": 1,
"fsal": {
"cmount_path": "/", "fs_name": "cephfs", "name": "CEPH", "user_id":
"nfs.nfsservice.cephfs"
},
"path": "/",
"protocols": [
3, 4
],
"pseudo": "/ganesha1",
"security_label": true,
"squash": "none",
"transports": [
"TCP"
]
}
```

4. Apply the specification.

```
ceph nfs export apply CLUSTER_NAME PATH_TO_EXPORT_FILE
```

For example,

```
[ceph: root@host01 /]# ceph nfs export apply nfs-cephfs -i export.conf
Added export /ceph
```

5. Get the updated share information.

```
ceph nfs export get CLUSTER_NAME BINDING
```

For example,

```
[ceph: root@host01 /]# ceph nfs export get nfs-cephfs /ceph
{
"access_type": "RW",
"clients": [],
"cluster_id": "nfsservice",
"export_id": 1,
"fsal": {
"cmount_path": "/", "fs_name": "cephfs", "name": "CEPH", "user_id":
"nfs.nfsganesha.cephfs"
},
"path": "/",
"protocols": [
3, 4
],
"pseudo": "/ganesha1",
"security_label": true,
"squash": "none",
"transports": [
"TCP"
]
}
```

Creating custom NFS shares

You can customize the Ceph File System (CepFS) shares and apply the configuration.

> **Note:** This documentation uses *NFS share* instead of the earlier term *NFS export*. In the Ceph Dashboard, labels and controls may still refer to NFS export.

Prerequisites

- A running, and healthy Red Hat Ceph Storage cluster.
- An NFS cluster created using the `ceph nfs cluster create` command.
- A CephFS created.

Procedure

1. Create a custom file:

   Example

   ```
   [ceph: root@host01 /]# touch export_new.conf
   ```

2. Create a share using the custom file:

   Syntax

   > **Note:** For NFSv3 mounts, use 3 for the **`Protocols`** parameter.

   ```
   EXPORT {
     Export_Id = EXPORT_ID;
     Transports = TCP/UDP;
     Path = PATH;
     Pseudo = PSEUDO_PATH;
     Protocols = NFS_PROTOCOLS;
     Access_Type = ACCESS_TYPE;
     Attr_Expiration_Time = EXPIRATION_TIME;
     Squash = SQUASH;
     FSAL {
       Name = NAME;
       Filesystem = "CEPH_FILE_SYSTEM_NAME";
       User_Id = "USER_ID";
     }
   }
   ```

   Example

   ```
   [ceph: root@host01 /]# cat export_new.conf
   EXPORT {
     Export_Id = 2;
     Transports = TCP;
     Path = /;
     Pseudo = /ceph1/;
     Protocols = 4;
     Access_Type = RW;
     Attr_Expiration_Time = 0;
     Squash = None;
     FSAL {
       Name = CEPH;
       Filesystem = "cephfs";
       User_Id = "nfs.nfs-cephfs.2";
     }
   }
   ```

3. Apply the specification:

   Syntax

   ```
   ceph nfs export apply CLUSTER_NAME -i PATH_TO_EXPORT_FILE
   ```

   Example

   ```
   [ceph: root@host01 /]# ceph nfs export apply nfs-cephfs -i new_export.conf
   Added export /ceph1
   ```

4. Get the updated share information:

Syntax

```
ceph nfs export get CLUSTER_NAME BINDING
```

For example,

```
[ceph: root@host01 /]# ceph nfs export get nfs-cephfs /ceph1
{
  "export_id": 1,
  "path": "/",
  "cluster_id": "nfs-cephfs",
  "pseudo": "/ceph1",
  "access_type": "RW",
  "squash": "None",
  "security_label": true,
  "protocols": [
    4
  ],
  "transports": [
    "TCP"
  ],
  "fsal": {
    "name": "CEPH",
    "user_id": "nfs.nfs-cephfs.2",
    "fs_name": "cephfs",
    "sec_label_xattr": ""
  },
  "clients": []
}
```

Deleting NFS shares from the CLI

You can delete the Ceph File System (CephFS) NFS shares with the **ceph export rm** command.

Procedure

- Delete an NFS share.

  ```
  ceph nfs export rm CLUSTER_NAME BINDING
  ```

  For example,

  ```
  [ceph: root@host01 /]# ceph nfs export rm nfs-cephfs /ceph
  ```

Deleting an NFS cluster

Delete an NFS cluster with the **nfs cluster rm** command. This deletes the deployed cluster. The removal of NFS daemons and the ingress service is asynchronous. Check the status of removal with the **ceph orch ls** command.

Before you begin

- A running, and healthy Red Hat Ceph Storage cluster.
- Root-level access to a Ceph Metadata Server (MDS) node.
- NFS daemons deployed with **ceph nfs cluster create** command.

Procedure

1. Log into the `cephadm` shell.

   For example,

   ```
   [root@mds ~]# cephadm shell
   ```

2. Remove the NFS cluster.

```
ceph nfs cluster rm CLUSTER_NAME
```

For example,

```
[ceph: root@host01 /]# ceph nfs cluster rm nfs-cephfs
```

Windows 11 desktop client side mount options for NFSv3 fileshare

Use this information to understand the client side mount options for NFSv3 fileshare.

NFSv3 (version 3) is supported and enabled for clients that do not yet support NFSv4.2 mounts for CephFS file shares.

> **Note:** Mount a file share with NFSv3 Windows 11 desktop only when NFSv4.2 is not supported on the client or server side. For more information about configuring NFS, see Configuring NFS Service daemons.

Use the following mount client-side options to mount a file share with NFSv3 Windows 11 desktop:

```
Usage:  mount [-o options] [-u:username] [-p:<password | *>] <\\computername\sharename>
<devicename | *>
```

Table 129: Windows 11 mount options for NFSv3

Mount options	Description
-o rsize=size	Set the size of the read buffer in kilobytes.
-o wsize=size	Set the size of the write buffer in kilobytes.
-o timeout=time	Set the timeout value in seconds for an RPC call.
-o retry=number	Set the number of retries for a soft mount.
-o mtype=soft|hard	Set the mount type.
-o lang=euc-jp|euc-tw|euc-kr|shift-jis|big5|ksc5601|gb2312-80|ansi	Specify the encoding used for file and directory names.
-o fileaccess=mode	Specify the permission mode of the file. These are used for new files created on NFS servers. Specified using UNIX style mode bits.
-o nolock	Disable locking.
-o anon	Mount as an anonymous user.
-o casesensitive=yes|no	Specify case sensitivity of file lookup on server.
-o sec=sys|krb5|krb5i|krb5p	Specify the security mechanism for mounting an NFS share. This value can include any of the following: – **sec=sys:** No authentication or security checks are performed. Data transfers are not encrypted. – **sec=krb5:** Specifies Kerberos authentication for mounting the NFS share. – **sec=krb5i:** Extends Kerberos security by adding data integrity checks between the client and server hasn't been tampered. – **sec=krb5p:** Enhances Kerberos security by enabling data privacy, which encrypts data exchanged between the client and server.

Rate-limiting in NFS

With the rate-limiting or QoS (quality of service) feature in NFS you can control the client's read/write bandwidth and rate-limiting of the client's input/output operations (IOPS). This function operates at share and client levels,

controlling read/write requests and IOPS. QoS helps ensures that all clients accessing backend storage receives a fair share of resources.

You can enable QoS for a single node NFS server and multiple instances of NFS servers.

The QoS configuration must be stored in the RADOS object and be accessible by any NFS server within the server group. Also, each share must reside on only one NFS server within the group.

The QoS function in NFS supports both bandwidth control and IOPS control.

> **Note:** The rate-limiting feature is supported only for NFS version 4 and later.

Bandwidth rate-limiting

NFS bandwidth rate limiting allows administrators to control the amount of bandwidth that NFS clients use when accessing or transferring data from an NFS server. This helps prevent excessive network congestion, ensures fair bandwidth distribution, and optimizes network performance for all users and applications.

The following are supported options in bandwidth rate-limiting:

Per share *(export)* bandwidth control

Use this option to configure the maximum bandwidth that is allowed per share *(export)*. In this case, QoS makes sure that the cumulative bandwidth of all clients accessing this share does not exceed the set bandwidth. For example, if the bandwidth limit for a share is set to 100 Mbps, any client running any operation on this share is given only 100 Mbps speed.

Per client bandwidth control

Use this option to configure the maximum bandwidth that is allowed per client. In this case, even if the client accesses more than one share, the total bandwidth that is given to the client will not exceed the set limit. For example, if the client bandwidth is set to 100 Mbps, this client can perform operations on any share with only 100 Mbps speed.

Per share *(export)*-per client bandwidth control

Use this option to configure both per share maximum bandwidth and per client maximum bandwidth. In this case, the maximum client bandwidth is controlled to the set limit, while the maximum per share bandwidth is also maintained.
For example, if the per share bandwidth is set to 200 Mbps and the bandwidth per client is set to 100 Mbps, client A writing to this share will have a speed of 100 Mbps. If client B writes to the same share, client B will have a rate limit of 100 Mbps, as 100 Mbps is already in use by client A.

In bandwidth rate-limiting, you can configure the maximum read and write bandwidth individually. For example, you can set the maximum write bandwidth to 100 Mbps and the read bandwidth to 200 Mbps.

Input/output operations (IOPS) rate-limiting

IOPS rate-limiting is a feature that controls the number of input/output operations per second that NFS clients can run on an NFS server. This helps prevent performance degradation, ensures fair resource allocation, and optimizes storage system efficiency.

Read and write operations consist of multiple underlying tasks. For example, a write operation includes four tasks: get file, open file, write file, and put file. If you set the IOPS limit to 20, only 5 IOPS allowed for each of these tasks.

The following are supported options in IOPS rate-limiting are:

Per share operations control

Use this option to configure the maximum number of IOPS allowed per share. In this case, QoS makes sure that the cumulative number of operations that are performed by all clients accessing this share will not exceed the given limit. For example, if the IOPS limit is set to 20, then any client accessing this share can run only 5 IOPS.

Per client operations control

Use this option to configure the maximum number of operations that are allowed per client. Regardless of the number of shares that the client is accessing, the total number of operations that are performed by the client will not exceed this limit. For example, if the IOPS limit for a client is set to 20, then this client can perform only 5 IOPS, irrespective of the number of shares it is accessing.

Per share-per client operations control

Use this option to configure the maximum that is allowed per share and maximum that is allowed per client operations. The total number of operations that are run by the client is controlled to the set limit.

Simultaneously, QoS helps ensure the maximum operations per share limit is kept within the limits. For example, if the IOPS limit for share is set to 20 and the client limit is set to 40, then this client can run only 5 operations on the share as its limit is set to 20. The client can run the remaining 3 operations on other shares.

For IOPS rate-limiting, all operations that the client runs are counted together as a single unit.

Managing rate-limiting for NFS clusters

Learn to manage the rate-limiting or quality of service (QoS) for NFS clusters.

There are 2 types of rate-limiting for NFS clusters:

- Bandwidth rate-limiting.
- Input/output (IOPS) rate-limiting.

You can enable and disable the rate-limiting feature by using the command line.

Enabling bandwidth rate-limiting for NFS cluster

Learn to enable the bandwidth rate-limiting feature by using the command line.

About this task

You can define bandwidth limits per client or share (per share) to ensure a balanced workload distribution across the cluster.

> **Note:** The bandwidth rate-limiting, the limit ranges from a minimum of 1 Mbps to a maximum of 4 Gbps.

Understanding cluster-wide enforcement of bandwidth and IOPS limits
When you enable bandwidth or input/output operations (IOPS) control on an NFS cluster, the configured limits apply cumulatively across all hosts in the cluster, rather than per individual host.

To enforce these cumulative limits, NFS services coordinate with NFS daemons running on other hosts in the cluster. This coordination ensures that bandwidth and IOPS limits are enforced consistently, even when multiple NFS servers handle client requests at the same time.

> **Note:** You can configure the port used for this inter-node communication by using the cluster_qos_port field in the NFS specification. The default port is 31311.

You can control how frequently NFS daemons exchange coordination messages by using the cqos_msg_interval field in the NFS specification. The default interval is 100 ms.

Alternatively, you can update the message interval by using the command-line interface.

```
ceph nfs cluster qos set <cluster_id> <msg_interval>
```

This inter-node coordination ensures accurate enforcement of bandwidth and IOPS limits across the NFS cluster.

Cluster QoS TCP keepalive configuration
Cluster QoS uses TCP connections between NFS nodes. To keep these TCP connections open, configure TCP keepalive parameters in the `NFS_CORE_PARAMS` section of the nfs-ganesha.conf file on all NFS nodes.

Configure the following parameters:

```
TCP_KEEPCNT = 5;
TCP_KEEPIDLE = 30;
TCP_KEEPINTVL = 2;
```

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A healthy and running Red Hat Ceph Storage cluster.

– An NFS cluster.

Procedure

– Run the following command to enable bandwidth rate-limiting for an NFS cluster.

```
ceph nfs cluster qos enable bandwidth_control CLUSTER ID qos_type:PerShare|PerClient|
PerShare_PerClient [--combined-rw-bw-ctrl] [--max_export_write_bw VALUE] [--
max_export_read_bw VALUE] [--max_client_write_bw VALUE] [--max_client_read_bw VALUE]
[--max_export_combined_bw VALUE] [--max_client_combined_bw VALUE]
```

Where,

- *CLUSTER_ID* is the NFS cluster ID.
- **qos_type** is the type of bandwidth control. You can select either **PerShare** (per share), **PerClient** or **PerShare_PerClient** based on your requirement.
  - If the **PerShare** qos_type is selected, the cluster-level quality of service (QoS) configuration applies to all shares created on that NFS cluster. If **combined-rw-bw-ctrl** is set to false, you must specify both the **max_export_write_bw** and **max_export_read_bw** parameters. Otherwise, only the **max_export_combined_bw** parameter is required.
  - If the **PerClient** qos_type is selected, the cluster-level QoS configuration applies to all clients accessing shares on that cluster. If **combined-rw-bw-ctrl** is set to false, you must specify both the **max_client_write_bw** and **max_client_read_bw** parameters. Otherwise, only the **max_client_combined_bw** parameter is required
  - If the **PerShare_PerClient** qos_type is selected, the cluster-level configuration applies to all shares and all clients of that NFS cluster. If **combined-rw-bw-ctrl** is set to false, you must specify the **max_export_write_bw**, **max_export_read_bw**, **max_client_write_bw**, and **max_client_read_bw** parameters. Otherwise, only the **max_export_combined_bw** and **max_client_combined_bw** parameters are required.
- **combined-rw-bw-ctrl** This parameter enables combined read and write bandwidth. If **combined-rw-bw-ctrl** is set to true, then only **max_export_combined_bw** and **max_client_combined_bw** parameters are allowed based on QoS type.
- **max_export_write_bw** is the maximum bandwidth for share write per second.
- **max_export_read_bw** is the maximum bandwidth for share that is read per second.
- **max_client_write_bw** is the maximum bandwidth that the client can use for writing per second.
- **max_client_read_bw** is the maximum bandwidth that the client can use for reading per second.
- **max_export_combined_bw** is the maximum combined read/write bandwidth for share per second.
- **max_client_combined_bw** is the maximum combined read/write bandwidth for client per second.

> **Note:** You can specify the value as 10, 10 MB, 10 MiB, or 10 GB.

Example of bandwidth rate-limiting for PerShare, clust_nfs is the NFS cluster ID

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos enable bandwidth_control clust_nfs
PerShare --max_export_write_bw 100MB --max_export_read_bw 200MB
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs
{
  "combined_rw_bw_control": false,
  "enable_bw_control": true,
  "enable_iops_control": false,
  "enable_qos": true,
  "max_export_read_bw": "200.0MB",
  "max_export_write_bw": "100.0MB",
  "qos_type": "PerShare"
}
```

Example of bandwidth rate-limiting for PerShare with combined read/write share limit

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos enable bandwidth_control clust_nfs
PerShare --combined-rw-bw-ctrl --max_export_combined_bw 200MB
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs
{
  "combined_rw_bw_control": true,
  "enable_bw_control": true,
  "enable_iops_control": false,
  "enable_qos": true,
  "max_export_combined_bw": "200.0MB",
  "qos_type": "PerShare"
}
```

Example of bandwidth rate-limiting for `PerClient`

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos enable bandwidth_control clust_nfs
PerClient --max_client_write_bw 100MB --max_client_read_bw 200MB
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs
{
  "combined_rw_bw_control": false,
  "enable_bw_control": true,
  "enable_iops_control": false,
  "enable_qos": true,
  "max_client_read_bw": "200.0MB",
  "max_client_write_bw": "100.0MB",
  "qos_type": "PerClient"
}
```

Example of bandwidth rate-limiting for `PerClient` with combined read/write share limit.

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos enable bandwidth_control clust_nfs
PerClient --combined-rw-bw-ctrl --max_client_combined_bw 300MB
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs
{
  "combined_rw_bw_control": true,
  "enable_bw_control": true,
  "enable_iops_control": false,
  "enable_qos": true,
  "max_client_combined_bw": "300.0MB",
  "qos_type": "PerClient"
}
```

Example of bandwidth rate limiting for `PerShare_PerClient`

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos enable bandwidth_control clust_nfs
PerShare_PerClient --max_export_write_bw 100MB --max_export_read_bw 200MB --
max_client_write_bw 100MB --max_client_read_bw 200MB
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs
{
  "combined_rw_bw_control": false,
  "enable_bw_control": true,
  "enable_iops_control": false,
  "enable_qos": true,
  "max_client_read_bw": "200.0MB",
  "max_client_write_bw": "100.0MB",
  "max_export_read_bw": "200.0MB",
  "max_export_write_bw": "100.0MB",
  "qos_type": "PerShare_PerClient"
}
```

Example of bandwidth rate-limiting for `PerShare_PerClient` with combined read/write share limit

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos enable bandwidth_control clust_nfs
PerShare_PerClient --combined-rw-bw-ctrl --max_export_combined_bw 200MB --
max_client_combined_bw 200MB
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs
{
  "combined_rw_bw_control": true,
  "enable_bw_control": true,
  "enable_iops_control": false,
  "enable_qos": true,
  "max_client_combined_bw": "200.0MB",
  "max_export_combined_bw": "200.0MB",
  "qos_type": "PerShare_PerClient"
}
```

By default, all shares inherit the cluster-level QoS settings.

> **Note:** If you use the command in the procedure to update the **qos_type**, make sure to update all the shares with the required parameters.

Disabling bandwidth rate-limiting for NFS cluster

You can disable bandwidth rate-limiting by using the command line.

Procedure

- Run the following command to disable bandwidth-rate limiting.

```
ceph nfs cluster qos disable bandwidth_control CLUSTER_ID
```

Where,
CLUSTER_ID is the NFS cluster ID.

This command disables the bandwidth control in the cluster. If cluster-level QoS is disabled, then share-level QoS is not applicable, even if it is enabled.

Enabling input/output operations rate-limiting for NFS cluster

Learn how to enable the input/output operations (IOPS) rate-limiting for NFS cluster by using the command-line.

About this task

You can set input/output operations (IOPS) limits either per client or per share to ensure an even distribution of workload across the cluster.

> **Note:** The limit for IOPS rate-limiting is a minimum of 10 IOPS and a maximum of 16384 IOPS.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A healthy and running Red Hat Ceph Storage cluster.
- An NFS cluster.

Procedure

- Run the following command to enable bandwidth rate-limiting for an NFS cluster.

```
ceph nfs cluster qos enable ops_control CLUSTER_ID qos_type:PerShare|PerClient|
PerShare_PerClient [max_export_iops VALUE] [max_client_iops VALUE]
```

Where,

- *CLUSTER_ID* is the NFS cluster ID.
- **qos_type** is the type of ops control. It can be **PerShare**, **PerClient** or **PerShare_PerClient**.
-
  - If **PerShare qos_type** is selected, then the cluster-level QoS configuration applies to all shares created on that NFS cluster. It requires the **export_ops_count** parameter.
  - If **PerClient qos_type** is selected, then the cluster-level QoS configuration applies to all clients accessing shares created on that cluster. It requires the **client_ops_count** parameter.
  - If **PerShare_PerClient qos_type** is selected, then the cluster-level configuration applies to all shares as well as all clients of that NFS cluster. It requires both **export_ops_count** and **client_ops_count** parameters.
- **max_export_iops** is the count of IOPS operations for a share.
- **max_client_iops** is the count of IOPS operations for a client of a share.

> **Note:** If this command is used to update the **qos_type**, make sure to update all the shares with the required parameters.

Example of IOPS rate-limiting for PerShare_PerClient, clust_nfs is the NFS cluster ID

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos enable ops_control clust_nfs
PerShare_PerClient --max_export_iops 2000 --max_client_iops 3000
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs
{
  "combined_rw_bw_control": true,
  "enable_bw_control": true,
  "enable_iops_control": true,
  "enable_qos": true,
  "max_client_combined_bw": "200.0MB",
  "max_client_iops": 3000,
  "max_export_combined_bw": "200.0MB",
  "max_export_iops": 2000,
  "qos_type": "PerShare_PerClient"
}
```

Example of IOPS rate-limiting for PerShare

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos enable ops_control clust_nfs
PerShare --max_export_iops 4000
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs
{
  "combined_rw_bw_control": false,
  "enable_bw_control": false,
  "enable_iops_control": true,
  "enable_qos": true,
  "max_export_iops": 4000,
  "qos_type": "PerShare"
}
```

Example of IOPS rate-limiting for PerClient

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos enable ops_control clust_nfs
PerClient --max_client_iops 4000
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs
{
  "combined_rw_bw_control": false,
  "enable_bw_control": false,
  "enable_iops_control": true,
  "enable_qos": true,
  "max_client_iops": 4000,
  "qos_type": "PerClient"
}
```

Disabling input/output operations rate-limiting for NFS cluster

Learn to disable input/output operations (IOPS) rate-limiting for NFS clusters by using the command line.

Procedure

- You can disable IOPS rate-limiting for the NFS cluster by using the following command.

```
nfs cluster qos disable ops_control CLUSTER_ID
```

Where,
CLUSTER_ID is the NFS cluster ID.

Listing QoS configuration for NFS cluster

Learn how to list the quality of service (QoS) configuration for the NFS cluster by using the command line.

About this task

You can list the QoS configuration for the NFS clusters.

Procedure

- Run the following command to list the QoS configuration:

```
ceph nfs cluster qos get CLUSTER_ID [--format VALUE]
```

Where,

- *CLUSTER_ID* is the NFS cluster ID.
- *VALUE* is either YAML or JSON.

> **Note:** This command displays all the QoS global default configurations for a given cluster.

Example of listing QoS settings with JSON format

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs --format json
{
"combined_rw_bw_control": false,
"enable_bw_control": false,
"enable_iops_control": true,
"enable_qos": true,
"max_client_iops": 1000,
"qos_type": "PerClient"
}
```

Example of listing QoS settings with YAML format

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs --format yaml
combined_rw_bw_control: false
enable_bw_control: false
enable_iops_control: true
enable_qos: true
max_client_iops: 1000
qos_type: PerClient
```

Managing rate-limiting for NFS shares

Learn to manage the rate-limiting or quality of service (QoS) for NFS shares.

There are 2 types of rate-limiting for NFS shares:

- Bandwidth rate-limiting.
- Input/output (IOPS) rate-limiting.

You can enable and disable the rate-limiting feature by using the command line.

Enabling bandwidth rate-limiting for NFS shares

Learn how to enable bandwidth rate-limiting for NFS shares by using the command line.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A healthy and running Red Hat Ceph Storage cluster.
- NFS shares.

> **Note:** This documentation uses *NFS share* instead of the earlier term *NFS export*. In the Ceph Dashboard, labels and controls may still refer to NFS export.

About this task

Enabling bandwidth rate-limiting for NFS shares helps control network traffic by restricting data transfer rate for NFS clients. Bandwidth rate-limiting prevents network congestion, ensures fair bandwidth distribution, and optimizes system performance.

> **Note:**
>
> - When you set NFS QoS bandwidth control to a very low limit, other operations on the same file may experience delays. For example, if the QoS bandwidth is limited to 10 MB/sec and you execute the **`du -h FILE`** command while the file is under active write I/O, the command may take longer to complete.
> - This behavior is expected because the file is servicing throttled write operations, which can delay concurrent access.
> - You cannot enable share-level bandwidth control if the cluster-level **`qos_type`** is **`PerClient`**.

Procedure

- The following command enables and updates QoS bandwidth control for shares.
  To enable share-level bandwidth control, you must enable cluster-level bandwidth control with **`qos_type`** **`PerShare`** or **`Pershare_PerClient`**. This command creates QOS_BLOCK in the share block.

  ```
  nfs export qos enable bandwidth_control CLUSTER_ID <pseudo_path> [--combined-rw-bw-
  ctrl] [--max_export_write_bw VALUE] [--max_export_read_bw VALUE] [--max_client_write_bw
  VALUE] [--max_client_read_bw VALUE] [--max_export_combined_bw VALUE] [--
  max_client_combined_bw VALUE]
  ```

  Where,

  - *`CLUSTER_ID`* is the NFS cluster ID.
  - **`pseudo_path`** is the pseudo-root path. (The path must be an absolute path.)
  - **`combined-rw-bw-ctrl`** enables combined read and write bandwidth control. When set to True, only the **`max_export_combined_bw`** and **`max_client_combined_bw`** parameters are allowed, depending on the qos_type.
  - **`max_export_write_bw`** is the maximum bandwidth that can be used for share write per second.
  - **`max_export_read_bw`** is the maximum bandwidth that can be used for share that is read per second.
  - **`max_client_write_bw`** is the maximum bandwidth that the client can use for writing per second.
  - **`max_client_read_bw`** is the maximum bandwidth that the client can use for reading per second.
  - **`max_export_combined_bw`** is the maximum combined read/write bandwidth for share per second.
  - **`max_client_combined_bw`** is the maximum combined read/write bandwidth for client per second.

  Example for bandwidth rate-limiting for QoS type **`Pershare_PerClient`**

```
[ceph: root@ceph-node-0 /]# ceph nfs export qos enable bandwidth_control clust_nfs /
export1 --max_export_write_bw 100MB --max_export_read_bw 200MB --max_client_write_bw
300MB --max_client_read_bw 100MB
[ceph: root@ceph-node-0 /]# ceph nfs export qos get clust_nfs /export1
{
  "cluster_enable_bw_control": true,
  "cluster_enable_iops_control": false,
  "cluster_enable_qos": true,
  "combined_rw_bw_control": false,
  "enable_bw_control": true,
  "enable_qos": true,
  "max_client_read_bw": "100.0MB",
  "max_client_write_bw": "300.0MB",
  "max_export_read_bw": "200.0MB",
  "max_export_write_bw": "100.0MB"
}
```

Disabling bandwidth rate-limiting for NFS share

Learn how to disable bandwidth rate-limiting for NFS shares by using the command line.

About this task

Note: This documentation uses *NFS share* instead of the earlier term *NFS export*. In the Ceph Dashboard, labels and controls may still refer to NFS export.

Procedure

– You can disable bandwidth rate-limiting for the NFS shares by using the following command.

```
nfs export qos disable bandwidth_control CLUSTER_ID PSEUDO_PATH
```

Where,

- *CLUSTER_ID* is the NFS cluster ID.
- *PSEUDO_PATH* is the pseudo-root path. (The path must be an absolute path.)

This command disables QoS bandwidth control for shares, allowing them to use unlimited input/output bandwidth. Once share QoS is disabled, the shares will no longer follow the bandwidth values set at the cluster level.

Note: To use cluster level QoS bandwidth control values for share again, you can use the NFS share apply *CLUSTER_ID* command, with the share section not having QOS_BLOCK.

Enabling input/output operations rate-limiting for NFS share

Learn how to enable input/output operations (IOPS) rate-limiting for NFS share by using the command line.

About this task

Enabling IOPS rate-limiting for NFS shares helps control the number of input/output operations per second that clients can perform, preventing resource contention and ensuring fair usage.

To enable share-level IOPS control, you must enable cluster-level IOPS control with **qos_type** ,**PerShare**, or **PerShare_PerClient**.

Note: You cannot enable export-level IOPS control if the cluster-level **qos_type** is **PerClient**.

Understanding cluster-wide enforcement of bandwidth and IOPS limits

When you enable bandwidth or input/output operations (IOPS) control on an NFS cluster, the configured limits apply cumulatively across all hosts in the cluster, rather than per individual host.

To enforce these cumulative limits, NFS services coordinate with NFS daemons running on other hosts in the cluster. This coordination ensures that bandwidth and IOPS limits are enforced consistently, even when multiple NFS servers handle client requests at the same time.

> **Note:** You can configure the port used for this inter-node communication by using the cluster_qos_port field in the NFS specification. The default port is 31311.

You can control how frequently NFS daemons exchange coordination messages by using the cqos_msg_interval field in the NFS specification. The default interval is 100 ms.

Alternatively, you can update the message interval by using the command-line interface.

```
ceph nfs cluster qos set <cluster_id> <msg_interval>
```

This inter-node coordination ensures accurate enforcement of bandwidth and IOPS limits across the NFS cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A healthy and running Red Hat Ceph Storage cluster.
- An NFS cluster.
- NFS shares.

Procedure

- You can use the following command to enable IOPS rate-limiting to NFS shares. This command enables IOPS control for a specified share. You can also use this command to update the operations count.

  ```
  nfs export qos enable ops_control CLUSTER_ID PSEUDO_PATH [max_export_iops:INT]
  [max_client_iops:INT]
  ```

  Where,

  - *CLUSTER_ID* is the NFS cluster ID.
  - *PSEUDO_PATH* is the pseudo root path. The path must be an absolute path.
  - **max_export_iops** is the count of IOPS operations for share.
  - **max_client_iops** is the count of IOPS operations for a client of the share.

  IOPS rate-limiting for PerShare

  ```
  [ceph: root@ceph-node-0 /]# ceph nfs export qos enable ops_control clust_nfs /
  export1 --max_export_iops 2000
  [ceph: root@ceph-node-0 /]# ceph nfs export qos get clust_nfs /export1
  {
  "cluster_enable_bw_control": false,
  "cluster_enable_iops_control": true,
  "cluster_enable_qos": true,
  "combined_rw_bw_control": false,
  "enable_bw_control": false,
  "enable_iops_control": true,
  "enable_qos": true,
  "max_export_iops": 2000
  }
  ```

  IOPS rate-limiting for PerShare_Per Client

```
[ceph: root@ceph-node-0 /]# ceph nfs export qos enable ops_control clust_nfs /
export1 --max_export_iops 2000 --max_client_iops 1000
[ceph: root@ceph-node-0 /]# ceph nfs export qos get clust_nfs /export1
{
"cluster_enable_bw_control": false,
"cluster_enable_iops_control": true,
"cluster_enable_qos": true,
"combined_rw_bw_control": false,
"enable_bw_control": false,
"enable_iops_control": true,
"enable_qos": true,
"max_client_iops": 1000,
"max_export_iops": 2000
}
```

Disabling input/output operations rate-limiting for NFS share

Learn to disable input/output operations (IOPS) rate-limiting for NFS shares by using command line.

About this task

> **Note:** This documentation uses *NFS share* instead of the earlier term *NFS export*. In the Ceph Dashboard, labels and controls may still refer to NFS export.

Procedure

– The following command disables QoS operations control for sharing, removing any limits on the operations for shares. As a result, shares will not be subject to any cluster-level operation restrictions.

```
nfs export qos disable ops_control CLUSTER_ID PSEUDO_PATH
```

Where,

- *CLUSTER_ID* is the NFS cluster ID.
- *PSEUDO_PATH* is the pseudo-root path. The path must be an absolute path.

Listing QoS configuration for NFS share

Learn to list the quality of service (QoS) configuration for NFS shares by using the command line.

About this task

You can list the quality of service (QoS) configuration for the NFS shares.

> **Note:** This documentation uses *NFS share* instead of the earlier term *NFS export*. In the Ceph Dashboard, labels and controls may still refer to NFS export.

Procedure

– Run the **`nfs export qos get`** command.

```
nfs export qos get CLUSTER_ID PSEUDO_PATH [--format VALUE]
```

Where,

- *CLUSTER_ID* is the NFS cluster ID.
- *PSEUDO_PATH* is the pseudo-root path. (The path must be an absolute path.)
- *VALUE* is either JSON or YAML.

Example for QoS listing:

QoS listing with format as JSON

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs --format json
{
"combined_rw_bw_control": false,
"enable_bw_control": false,
"enable_iops_control": true,
"enable_qos": true,
"max_client_iops": 1000,
"qos_type": "PerClient"
}
```

QoS listing with format as YAML

```
[ceph: root@ceph-node-0 /]# ceph nfs cluster qos get clust_nfs --format yaml
combined_rw_bw_control: false
enable_bw_control: false
enable_iops_control: true
enable_qos: true
max_client_iops: 1000
qos_type: PerClient
```

QoS listing for a share

```
[ceph: root@ceph-node-0 /]# ceph nfs export qos get clust_nfs /export1
{
"cluster_enable_bw_control": true,
"cluster_enable_iops_control": true,
"cluster_enable_qos": true,
"combined_rw_bw_control": false,
"enable_bw_control": true,
"enable_iops_control": true,
"enable_qos": true,
"max_client_iops": 1000,
"max_client_read_bw": "100.0MB",
"max_client_write_bw": "100.0MB",
"max_export_iops": 2000,
"max_export_read_bw": "200.0MB",
"max_export_write_bw": "100.0MB"
}
```

The share will not have any bandwidth restriction in this case; that is, it will not have any cluster level bandwidth restrictions.

TLS security in NFS

Transport Layer Security (TLS), when used with Network File System (NFS), enables secure, encrypted communication for NFS traffic between the client and server.

TLS is a cryptographic protocol that provides confidentiality, integrity, and authentication for network communications. When used with NFS, TLS encrypts all traffic between the client and server. This ensures that data is not visible to intermediaries and cannot be altered in transit without detection.

TLS runs above Transmission Control Protocol (TCP) at the transport layer and establishes a secure channel between the client and server before any application data is exchanged.

A TLS handshake is the connection establishment phase. During the TLS handshake:

- The client validates the server's identity using its certificate.
- The server may optionally request a certificate from the client.

> **Note:** When NFS is configured to use TLS-based transport security, the mount negotiation and all subsequent Remote Procedure Call (RPC) operations occur over this encrypted channel.

TLS versus mTLS

The authentication models supported by TLS are TLS and mTLS.

TLS (one-way authentication)

In TLS authentication, only the server is authenticated. The client verifies the server's certificate against a trusted Certificate Authority (CA). The server does not verify the client's identity beyond network address and application-level authentication.

mTLS (mutual authentication)

In mTLS authentication, both endpoints present certificates during the TLS handshake. The client validates the server's certificate, and the server validates the client's certificate against its trusted CA list. This ensures only authorized clients can connect.

Use Cases

This feature is essential in scenarios where secure and compliant NFS communication is required, such as:

- SAP workloads requiring secure NFS communication.
- Migration from NetApp file services to Ceph-based NFS Regional File Services.
- Environments where compliance mandates encrypted data in transit.

Configuring share security options for NFS

Set share security (XprtSec) options in NFS to control how clients connect using TLS or mTLS and understand the expected behavior for different configurations.

About this task

Share security options determine whether NFS traffic uses plaintext or encrypted (TLS or mTLS) . These settings affect client mount behavior and enforce encryption policies for secure communication.

Procedure

- Share security options.

```
ceph nfs export create cephfs mynfs /export3 myfs --xprtsec tls
```

The expected behavior based on server share security and client mount options are listed in Table 1.

Table 130: Share options and behavior

Server share security	Client mount option	Expected behavior
XprtSec=none	(no xprtsec)	Mount succeeds in plaintext (unencrypted TCP/UDP).
XprtSec=none	xprtsec=tls	Mount succeeds over TLS; encryption used even though not required.
XprtSec=none	xprtsec=mtls	Mount succeeds over TLS; encryption used even though not required.
XprtSec=tls	(no xprtsec)	Mount fails with EACCES ("Permission denied") - server rejects non-TLS connections.
XprtSec=tls	xprtsec=tls	Mount succeeds over TLS; server certificate validated by client.
XprtSec=tls	xprtsec=mtls	Mount succeeds over TLS; client certificate validation is not enforced.
XprtSec=mtls	(no xprtsec)	Mount fails with EACCES - client not attempting mTLS.
XprtSec=mtls	xprtsec=tls	Mount fails with EACCES - mTLS required; client must provide valid certificate.

Server share security	Client mount option	Expected behavior
XprtSec=mtls	xprtsec=mtls	Mount succeeds only if client presents a valid certificate trusted by the server; otherwise fails with EACCES.

You can specify share security options manually. For more information, see Manually specify share security options for NFS.

Configuring TLS settings for NFS

Enable and customize TLS settings in NFS to secure NFS traffic with encryption and optional mutual authentication.

About this task

TLS settings in NFS control how encrypted communication is established between clients and the server. When you deploy the NFS service using this specification, certificates are generated and configured automatically based on the selected certificate source. The TLS settings also define the cipher suites, minimum protocol version, and optional features such as Kernel TLS (KTLS) and debugging.

Procedure

- Deploy or update the NFS service specification file with TLS and SSL configuration.
  The following example shows an NFS service specification with TLS and SSL configuration by using inline certificates.

```
.. code-block:: yaml

    service_type: nfs
    service_id: mynfs
    placement:
      hosts:
      - ceph-node-0
    spec:
      port: 12345
      ssl: true
      certificate_source: inline|reference|cephadm-signed
      ssl_cert: |
        -----BEGIN CERTIFICATE-----
        (PEM cert contents here)
        -----END CERTIFICATE-----
      ssl_key: |
        -----BEGIN PRIVATE KEY-----
        (PEM key contents here)
        -----END PRIVATE KEY-----
      ssl_ca_cert:
        -----BEGIN PRIVATE KEY-----
        (PEM key contents here)
        -----END PRIVATE KEY-----
      tls_ktls: true
      tls_debug: true
      tls_min_version: TLSv1.3
      tls_ciphers: AES-256
```

TLS configuration parameters and their default values are listed in Table 1.

Table 131: NFS TLS parameters (inline certificate example)

Parameter	Default	Description
service_type	nfs	Defines the service type to deploy.
service_id	mynfs	Identifier for the NFS service instance.

Parameter	Default	Description
placement	""	Defines where the NFS service is deployed, including host selection and scheduling constraints.
hosts	ceph-node-0	Host (or hosts) where the NFS service runs.
port	12345	TCP port that the NFS service listens on.
ssl	true	Enable or disable TLS for the NFS service.
certificate_source	inline | reference | cephadm-signed	Location of certificate material: inline content, external reference, or cephadm-signed.
ssl_cert	(PEM certificate)	Server certificate in PEM format (used when **`certificate_source`** is `inline`).
ssl_key	(PEM private key)	Server private key in PEM format matching **`ssl_cert`**.
ssl_ca_cert	(PEM CA certificate)	CA certificate (or chain) in PEM format used to validate the server certificate.
tls_ktls	true	Enable Kernel TLS acceleration if supported by the platform.
tls_debug	true	Enable detailed TLS debugging logs (use only for troubleshooting).
tls_min_version	TLSv1.3	Minimum supported TLS protocol version.
tls_ciphers	AES-256	Cipher suite policy for TLS connections.

Result

In case of configuration failure, you can configure TLS settings for NFS manually. For more information, see Manually configuring TLS settings for NFS.

Setting up TLS in NFS client using Stunnel

Learn how to configure TLS for NFS to protect data in transit and ensure secure communication between NFS clients and servers.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Install TLS backend library (OpenSSL) based on your build configuration.
- The TLS certificates and keys must be available.

> **Note:** You must have access to the following TLS certificates:
>
> - CA certificate (`ca.crt`)
> - Server certificate (`ganesha.crt`)
> - Private key (`ganesha.key`)

- Ensure client-side support for TLS or mTLS (NFS client tools that support `xprtsec=tls` or `xprtsec=mtls`). For more information on configuring the NFS client with TLS or mutual TLS (mTLS) support **How to › Enhancing security › Data encryption in transit > Establishing encryption in transit for regional file share. › Establishing encryption in transit for regional file share** in Virtual Private Cloud (VPC) documentation in IBM Cloud Docs.

About this task

TLS adds an encryption layer to NFS traffic, safeguarding sensitive information from interception or tampering during transmission. By enabling TLS in NFS, you can enforce secure connections, maintain data integrity, and comply with security best practices.

Use Stunnel to enable TLS in older clients without native TLS support.

Procedure

1. Install Stunnel on the client side.
2. Configure Stunnel (`/etc/stunnel/stunnel.conf`) to forward traffic to the NFS port.
   For example,

   ```
   pid = /var/run/stunnel/stunnel.pid
   cert = /home/dpp/workspace/TLS_certificates/vm1.crt
   key = /home/dpp/workspace/TLS_certificates/vm1.key
   CAfile = /home/dpp/workspace/TLS_certificates/ca.crt
   socket = r:TCP_NODELAY=1
   foreground = yes
   debug = 7
   output = /var/log/stunnel.log
   renegotiation = no
   [nfs4]
   client = yes
   accept = 127.0.0.1:49152
   connect = 23.13.161.10:2049
   sslVersion = TLSv1.3
   ciphers = ALL
   verify = 2
   ```

3. Change the `verify` value for TLS or mTLS as needed.

   > **Note:** Optional: Remove the `cert` and `key` entries if mTLS is not required on the client.

4. Start Stunnel.

   ```
   stunnel /etc/stunnel/stunnel.conf
   ```

5. Mount NFS through Stunnel.

   ```
   mount -vv -t nfs -o rw -o port=49152 127.0.0.1:/exportfs0 /mnt/
   ```

   You can enable TLS using tlshd (ktls-utils). For more information, see Setting up TLS in NFS clients using tlshd (ktls-utils).

Red Hat Ceph Storage File Systems

As a storage administrator, you can configure the Ceph Metadata Server (MDS) and how to create, mount, and work the Ceph File System (CephFS). Get to know the features, system components, and limitations to manage a CephFS environment.

The Ceph File System (CephFS) is a file system compatible with POSIX standards that is built on top of the Ceph distributed object store, RADOS (Reliable Autonomic Distributed Object Storage). CephFS provides file access to a Red Hat Ceph Storage cluster, and uses the POSIX semantics wherever possible. For example, in contrast to many other common network file systems, CephFS maintains strong cache coherency across clients. The goal is for processes that use the file system to behave the same when they are on different hosts as when they are on the same host. However, sometimes CephFS diverges from the strict POSIX semantics.

The Ceph File System has the following features:

Parallel data streams

CephFS is a parallel file system. Clients and storage servers can exchange data through multiple-streams in parallel, which helps increase throughput and scalability.

Scalability

The Ceph File System is highly scalable due to horizontal scaling of metadata servers and direct client reads and writes with individual OSD nodes.

Shared file system

The Ceph File System is a shared file system so multiple clients can work on the same file system simultaneously.

Multiple file systems

You can have multiple file systems active on one storage cluster. Each CephFS has its own set of pools and its own set of Metadata Server (MDS) ranks. When deploying multiple file systems this requires more running MDS daemons. This can increase metadata throughput, but also increases operational costs. You can also limit client access to certain file systems.

High availability

The Ceph File System provides a cluster of Ceph Metadata Servers (MDS). One is active and others are in standby mode. If the active MDS stops unexpectedly, one of the standby MDS becomes active. As a result, client mounts continue working through a server failure. This behavior makes the Ceph File System highly available. In addition, you can configure multiple active metadata servers.

Configurable file and directory layouts

The Ceph File System allows users to configure file and directory layouts to use multiple pools, pool namespaces, and file striping modes across objects.

POSIX Access Control Lists (ACL)

The Ceph File System supports the POSIX Access Control Lists (ACL). ACLs are enabled by default with the Ceph File Systems mounted as kernel clients with kernel version `kernel-3.10.0-327.18.2.el7` or newer. To use an ACL with the Ceph File Systems mounted as FUSE clients, you must enable them.

Client quotas

The Ceph File System supports setting quotas on any directory in a system. The quota can restrict the number of bytes or the number of files that are stored beneath that point in the directory hierarchy. CephFS client quotas are enabled by default.

- For more information about installing Ceph Metadata servers, see Managing the MDS service.
- For more information about creating Ceph File Systems, see “Deploying the CephFS” on page 1165.

Using CephFS

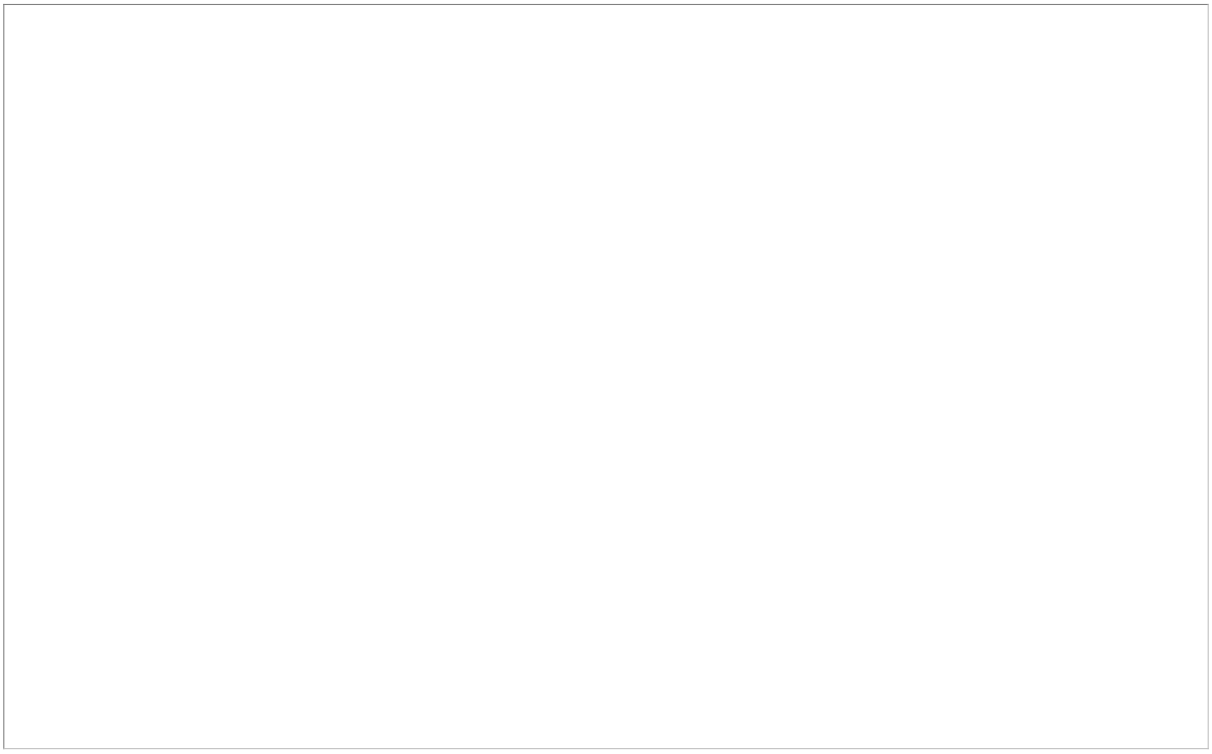
Understand how Ceph File Systems (CephFS) integrates within the Ceph cluster, its data flow, the role of the Metadata Server (MDS) and aspects of volume management.

Understanding the CephFS data flow

Understand the data flow in CephFS to recognize the interactions between the CephFS client, Metadata Server (MDS), and the Ceph Storage Cluster.

“Figure: CephFS Data Flow” on page 1132 illustrates the role of Metadata Server in CephFS.

Figure 92: CephFS Data Flow



The following are the primary functions of the CephFS Client and Metadata Server:

CephFS Client

Mounting

Connects to CephFS by using either `kcephfs` or `ceph-fuse`.

Metadata requests

Requests metadata from the MDS for file operations.

Metadata Server

Metadata Operations

Manages file system metadata including file creation, directory management, and access control.

Caching

Improves performance by caching frequently accessed metadata. Caching is distributed and coordinated with clients and other multi-MDS

Ceph Storage Cluster

RADOS Storage

Handles distributed object storage.

OSDs

Manage data storage, retrieval, and replication across the cluster.

Data Access and Storage

Accessing Data

After retrieving metadata from the MDS, the client accesses data directly from OSDs. Data is stored as objects within the Ceph Storage Cluster, with the CRUSH algorithm ensuring efficient distribution and redundancy.

Comparison with other FSaaS Solutions

Following are some of the key comparisons between CephFS and other SaaS solutions:

Scalability

CephFS inherits all the benefits of the core Ceph internal storage engine RADOS. File systems can be increased in capacity and performance by adding more storage nodes. The data is automatically distributed and protetected.

Performance

CephFS is a parallel filesystem. Datastreams can run in parallel from multiple CephFS clients to multiple Ceph storage servers and OSDs. Metadata operations are running independently of the data operations.

Integration

CephFS is a POSIX-compatible file system. The integration of applications and infrastructure software such as Backup and Recovery solutions can rely on the POSIX nature of CephFS.

Feature Set

CephFS provides parallel data access and throughput and performance can be scaled up and down by the number of storage nodes and storage devices. CephFS like the rest of Ceph is designed to have no single point of failure and data redundancy is a built-in feature of the Ceph RADOS architecture. Available physical storage capacities are managed in thinly provisioned pools which all share the same infrastructure.

Data Protection in CephFS

Because CephFS builds on top of RADOS, it inherits all the data durability available from that system. File data and metadata are regularly scrubbed and node failures are automatically handled. CephFS also provides metadata scrubbing that you can start.

For geographically resilient scale storage, you can use CephFS snapshots and your favorite file system backup solution to generate consistent versions. If a data center disaster occurs, CephFS provides fsck like tools to rebuild the file system from data objects. For help if your system reports data issues, contact Red Hat Support.

Configuring CephFS

Learn how to configure Ceph File Systems and understand health messages.

Health messages

Understand the different health messages for the Ceph File System.

For more information, see “Configuring Metadata Server daemons” on page 1135.

Cluster health checks

The Ceph Monitor daemons generate the following health messages in response to certain states of the Metadata Server (MDS):

mds rank(s) <ranks> have failed

One or more MDS ranks are not currently assigned to any MDS daemon. The storage cluster will not recover until a suitable replacement daemon starts.

mds rank(s) <ranks> are damaged

One or more MDS ranks has encountered severe damage to its stored metadata, and cannot start again until the metadata is repaired.

mds cluster is degraded

One or more MDS ranks are not currently up and running, clients might pause metadata I/O until this situation is resolved. This includes ranks being failed or damaged, and includes ranks which are running on an MDS but are not in the `active` state yet — for example, ranks in the `replay` state.

mds <names> are laggy

The MDS daemons are supposed to send beacon messages to the monitor in an interval specified by the `mds_beacon_interval` option, the default is 4 seconds. If an MDS daemon fails to send a message within the time specified by the `mds_beacon_grace` option, the default is 15 seconds. The Ceph Monitor marks the MDS daemon as `laggy` and automatically replaces it with a standby daemon if any is available.

Daemon-reported health checks

The MDS daemons can identify a variety of unwanted conditions, and return them in the output of the `ceph status` command. These conditions have human readable messages, and also have a unique code starting `MDS_HEALTH`, which appears in JSON output.

Behind on trimming...

Code: MDS_HEALTH_TRIM

CephFS maintains a metadata journal that is divided into log segments. The length of journal (in number of segments) is controlled by the `mds_log_max_segments` setting. When the number of segments exceeds that setting, the MDS starts writing back metadata so that it can remove (trim) the oldest segments. If this process is too slow, or a software bug is preventing trimming, then this health message appears. The threshold for this message to appear is for the number of segments to be double `mds_log_max_segments`.

> **Note:** Increasing `mds_log_max_segments` is recommended if the trim warning is encountered. However, ensure that this configuration is reset back to its default when the cluster health recovers and the trim warning is seen no more. It is recommended to set `mds_log_max_segments` to 256 to allow the MDS to catch up with trimming.

Client *NAME* failing to respond to capability release

Code: `MDS_HEALTH_CLIENT_LATE_RELEASE, MDS_HEALTH_CLIENT_LATE_RELEASE_MANY`

CephFS clients are issued capabilities by the MDS. The capabilities work like locks. Sometimes, for example, when another client needs access, the MDS requests clients to release their capabilities. If the client is unresponsive, it might fail to do so promptly, or fail to do so at all. This message appears if a client has taken a longer time to comply than the time specified by the `mds_revoke_cap_timeout` option (default is 60 seconds).

Client *NAME* failing to respond to cache pressure

Code: `MDS_HEALTH_CLIENT_RECALL, MDS_HEALTH_CLIENT_RECALL_MANY`

Clients maintain a metadata cache. Items, such as inodes, in the client cache, are also pinned in the MDS cache. When the MDS needs to shrink its cache to stay within its own cache size limits, the MDS sends messages to clients to shrink their caches too. If a client is unresponsive, it can prevent the MDS from properly staying within its cache size, and the MDS might eventually run out of memory and terminate unexpectedly. This message appears if a client has taken more time to comply than the time specified by the `mds_recall_state_timeout` option (default is 60 seconds). For more information, see "Metadata Server cache size limits" on page 1151.

Client *NAME* failing to advance its oldest client/flush tid

Code: `MDS_HEALTH_CLIENT_OLDEST_TID, MDS_HEALTH_CLIENT_OLDEST_TID_MANY`

The CephFS protocol for communicating between clients and MDS servers uses a field called **oldest tid** to inform the MDS of which client requests are fully complete so that the MDS can forget about them. If an unresponsive client is failing to advance this field, the MDS might be prevented from properly cleaning up resources used by client requests. This message appears if a client has more requests than the number specified by the `max_completed_requests` option (default is 100000) that are complete on the MDS side but have not yet been accounted for in the client's `oldest tid` value.

Metadata damage detected

Code: `MDS_HEALTH_DAMAGE`

Corrupt or missing metadata was encountered when reading from the metadata pool. This message indicates that the damage was sufficiently isolated for the MDS to continue operating, although client accesses to the damaged subtree return I/O errors. Use the `damage ls` administration socket command to view details on the damage. This message appears as soon as any damage is encountered.

MDS in read-only mode

Code: `MDS_HEALTH_READ_ONLY`

The MDS has entered into read-only mode and will return the EROFS error codes to client operations that attempt to modify any metadata. The MDS enters into read-only mode:

- If it encounters a write error while writing to the metadata pool.

– If the administrator forces the MDS to enter into read-only mode by using the **`force_readonly`** administration socket command.

***N* slow requests are blocked**

Code: `MDS_HEALTH_SLOW_REQUEST`

One or more client requests have not been completed promptly, indicating that the MDS is either running very slowly, or encountering a bug. Use the `ops` administration socket command to list outstanding metadata operations. This message appears if any client requests have taken more time than the value specified by the `mds_op_complaint_time` option (default is 30 seconds).

Too many inodes in cache

Code: `MDS_HEALTH_CACHE_OVERSIZED`

The MDS has failed to trim its cache to comply with the limit set by the administrator. If the MDS cache becomes too large, the daemon might exhaust available memory and terminate unexpectedly. By default, this message appears if the MDS cache size is 50% greater than its limit.

Configuring Metadata Server daemons

Use this list of commands for Metadata Server (MDS) daemon configuration.

Table 132: Metadata Server (MDS) daemon configuration commands

Command	Description	Type	Default
`mon_force_standby_active`	If set to `true`, monitors force MDS in standby replay mode to be active. Set under the `[mon]` or `[global]` section in the Ceph configuration file.	Boolean	`true`
`max_mds`	The number of active MDS daemons during cluster creation. Set under the `[mon]` or `[global]` section in the Ceph configuration file.	32-bit Integer	`1`
`mds_cache_memory_limit`	The memory limit that the MDS enforces for its cache. Use this parameter instead of the MDS cache size parameter.	64-bit Integer Unsigned	`1073741824`
`mds_cache_reservation`	The cache reservation, memory or inodes, for the MDS cache to maintain. The value is a percentage of the maximum cache configured. Once the MDS begins dipping into its reservation, it recalls client state until its cache size shrinks to restore the reservation.	Float	`0.05`

Command	Description	Type	Default
mds_cache_size	The number of inodes to cache. A value of 0 indicates an unlimited number. Red Hat recommends to use the `mds_cache_memory_limit` to limit the amount of memory the MDS cache uses.	32-bit Integer	`0`
mds_cache_mid	The insertion point for new items in the cache LRU, from the top.	Float	`0.7`
mds_dir_commit_ratio	The fraction of directory that contains erroneous information before Ceph commits using a full update instead of partial update.	Float	`0.5`
mds_dir_max_commit_size	The maximum size of a directory update in MB before Ceph breaks the directory into smaller transactions.	32-bit Integer	`10`
mds_decay_halflife	The half-life of the MDS cache temperature.	Float	`5`
mds_beacon_interval	The frequency, in seconds, of beacon messages sent to the monitor.	Float	`4`
mds_beacon_grace	The interval without beacons before Ceph declares a MDS `laggy` and possibly replaces it.	Float	`15`
mds_blacklist_interval	The blacklist duration for failed MDS daemons in the OSD map.	Float	`24.0*60.0`
mds_session_timeout	The interval, in seconds, of client inactivity before Ceph times out capabilities and leases.	Float	`60`
mds_session_autoclose	The interval, in seconds, before Ceph closes a `laggy` client's session.	Float	`300`
mds_reconnect_timeout	The interval, in seconds, to wait for clients to reconnect during a MDS restart.	Float	`45`
mds_tick_interval	How frequently the MDS performs internal periodic tasks.	Float	`5`
mds_dirstat_min_interval	The minimum interval, in seconds, to try to avoid propagating recursive statistics up the tree.	Float	`1`

Command	Description	Type	Default
mds_scatter_nudge_interval	How quickly changes in directory statistics propagate up.	Float	5
mds_client_prealloc_inos	The number of inode numbers to preallocate per client session.	32-bit Integer	1000
mds_early_reply	Determines whether the MDS allows clients to see request results before they commit to the journal.	Boolean	true
mds_use_tmap	Use trivialmap for directory updates.	Boolean	true
mds_default_dir_hash	The function to use for hashing files across directory fragments.	32-bit Integer	2, that is, rjenkins
mds_log	Set to true if the MDS should journal metadata updates. Disable for benchmarking only.	Boolean	true
mds_log_skip_corrupt_events	Determines whether the MDS tries to skip corrupt journal events during journal replay.	Boolean	false
mds_log_max_events	The maximum events in the journal before Ceph initiates trimming. Set to -1 to disable limits.	32-bit Integer	-1
mds_log_max_segments	The maximum number of segments or objects in the journal before Ceph initiates trimming. Set to -1 to disable limits.	32-bit Integer	30
mds_log_max_expiring	The maximum number of segments to expire in parallels.	32-bit Integer	20
mds_log_eopen_size	The maximum number of inodes in an EOpen event.	32-bit Integer	100
mds_bal_sample_interval	Determines how frequently to sample directory temperature when making fragmentation decisions.	Float	3
mds_bal_replicate_threshold	The maximum temperature before Ceph attempts to replicate metadata to other nodes.	Float	8000
mds_bal_unreplicate_threshold	The minimum temperature before Ceph stops replicating metadata to other nodes.	Float	0
mds_bal_frag	Determines whether or not the MDS fragments directories.	Boolean	false

Command	Description	Type	Default
mds_bal_split_size	The maximum directory size before the MDS splits a directory fragment into smaller bits. The root directory has a default fragment size limit of 10000.	32-bit Integer	10000
mds_bal_split_rd	The maximum directory read temperature before Ceph splits a directory fragment.	Float	25000
mds_bal_split_wr	The maximum directory write temperature before Ceph splits a directory fragment.	Float	10000
mds_bal_split_bits	The number of bits by which to split a directory fragment.	32-bit Integer	3
mds_bal_merge_size	The minimum directory size before Ceph tries to merge adjacent directory fragments.	32-bit Integer	50
mds_bal_merge_rd	The minimum read temperature before Ceph merges adjacent directory fragments.	Float	1000
mds_bal_merge_wr	The minimum write temperature before Ceph merges adjacent directory fragments.	Float	1000
mds_bal_interval	The frequency, in seconds, of workload exchanges between MDS nodes.	32-bit Integer	10
mds_bal_fragment_interval	The frequency, in seconds, of adjusting directory fragmentation.	32-bit Integer	5
mds_bal_idle_threshold	The minimum temperature before Ceph migrates a subtree back to its parent.	Float	0
mds_bal_max	The number of iterations to run balancer before Ceph stops. For testing purposes only.	32-bit Integer	-1
mds_bal_max_until	The number of seconds to run balancer before Ceph stops. For testing purposes only.	32-bit Integer	-1
mds_bal_mode	The method for calculating MDS load:	32-bit Integer	0
mds_bal_min_rebalance	The minimum subtree temperature before Ceph migrates.	Float	0.1

Command	Description	Type	Default
mds_bal_min_start	The minimum subtree temperature before Ceph searches a subtree.	Float	`0.2`
mds_bal_need_min	The minimum fraction of target subtree size to accept.	Float	`0.8`
mds_bal_need_max	The maximum fraction of target subtree size to accept.	Float	`1.2`
mds_bal_midchunk	Ceph migrates any subtree that is larger than this fraction of the target subtree size.	Float	`0.3`
mds_bal_minchunk	Ceph ignores any subtree that is smaller than this fraction of the target subtree size.	Float	`0.001`
mds_bal_target_removal_min	The minimum number of balancer iterations before Ceph removes an old MDS target from the MDS map.	32-bit Integer	`5`
mds_bal_target_removal_max	The maximum number of balancer iterations before Ceph removes an old MDS target from the MDS map.	32-bit Integer	`10`
mds_replay_interval	The journal poll interval when in `standby-replay` mode for a `hot standby`.	Float	`1`
mds_shutdown_check	The interval for polling the cache during MDS shutdown.	32-bit Integer	`0`
mds_thrash_exports	Ceph randomly exports subtrees between nodes. For testing purposes only.	32-bit Integer	`0`
mds_thrash_fragments	Ceph randomly fragments or merges directories.	32-bit Integer	`0`
mds_dump_cache_on_map	Ceph dumps the MDS cache contents to a file on each MDS map.	Boolean	`false`
mds_dump_cache_after_rejoin	Ceph dumps MDS cache contents to a file after rejoining the cache during recovery.	Boolean	`false`
mds_verify_scatter	Ceph asserts that various scatter/gather invariants are `true`. For developer use only.	Boolean	`false`
mds_debug_scatterstat	Ceph asserts that various recursive statistics invariants are `true`. For developer use only.	Boolean	`false`

Command	Description	Type	Default
mds_debug_frag	Ceph verifies directory fragmentation invariants when convenient. For developer use only.	Boolean	`false`
mds_debug_auth_pins	The debug authentication pin invariants. For developer use only.	Boolean	`false`
mds_debug_subtrees	Debugging subtree invariants. For developer use only.	Boolean	`false`
mds_kill_mdstable_at	Ceph injects a MDS failure in a MDS Table code. For developer use only.	32-bit Integer	`0`
mds_kill_export_at	Ceph injects a MDS failure in the subtree export code. For developer use only.	32-bit Integer	`0`
mds_kill_import_at	Ceph injects a MDS failure in the subtree import code. For developer use only.	32-bit Integer	`0`
mds_kill_link_at	Ceph injects a MDS failure in a hard link code. For developer use only.	32-bit Integer	`0`
mds_kill_rename_at	Ceph injects a MDS failure in the rename code. For developer use only.	32-bit Integer	`0`
mds_wipe_sessions	Ceph deletes all client sessions on startup. For testing purposes only.	Boolean	`0`
mds_wipe_ino_prealloc	Ceph deletes inode preallocation metadata on startup. For testing purposes only.	Boolean	`0`
mds_skip_ino	The number of inode numbers to skip on startup. For testing purposes only.	32-bit Integer	`0`
mds_standby_for_name	The MDS daemon is a standby for another MDS daemon of the name specified in this setting.	String	N/A
mds_standby_for_rank	An instance of the MDS daemon is a standby for another MDS daemon instance of this rank.	32-bit Integer	`-1`
mds_standby_replay	Determines whether the MDS daemon polls and replays the log of an active MDS when used as a `hot standby`.	Boolean	`false`

Configuring journaler

Use this list of commands for journaler configuration.

journaler_write_head_interval

Description

How frequently to update the journal head object.

Type

Integer

Required

No

Default

`15`

journaler_prefetch_periods

Description

How many stripe periods to read ahead on journal replay.

Type

Integer

Required

No

Default

`10`

journaler_prezero_periods

Description

How many stripe periods to zero ahead of write position.

Type

Integer

Required

No

Default

`10`

journaler_batch_interval

Description

Maximum additional latency in seconds to incur artificially.

Type

Double

Required

No

Default

`.001`

journaler_batch_max

Description

Maximum bytes that will be delayed flushing.

Type

64-bit Unsigned Integer

Required

No

Default

`0`

Configuring Ceph File System mirrors

Use this list of commands for Ceph File System (CephFS) mirror configuration.

`cephfs_mirror_max_concurrent_directory_syncs`

Description

Maximum number of directory snapshots that can be synchronized concurrently by `cephfs-mirror` daemon. Controls the number of synchronization threads.

Type

Integer

Default

`3`

Min

`1`

`cephfs_mirror_action_update_interval`

Description

Interval in seconds to process pending mirror update actions.

Type

`secs`

Default

`2`

Min

`1`

`cephfs_mirror_restart_mirror_on_blocklist_interval`

Description

Interval in seconds to restart blocklisted mirror instances. Setting to zero (`0`) disables restarting blocklisted instances.

Type

`secs`

Default

`30`

Min

`0`

`cephfs_mirror_max_snapshot_sync_per_cycle`

Description

Maximum number of snapshots to mirror when a directory is picked up for mirroring by worker threads.

Type

Integer

Default

3

Min

1

cephfs_mirror_directory_scan_interval

Description

Interval in seconds to scan configured directories for snapshot mirroring.

Type

Integer

Default

10

Min

1

cephfs_mirror_max_consecutive_failures_per_directory

Description

Number of consecutive snapshot synchronization failures to mark a directory as “failed”. Failed directories are retried for synchronization less frequently.

Type

Integer

Default

10

Min

0

cephfs_mirror_retry_failed_directories_interval

Description

Interval in seconds to retry synchronization for failed directories.

Type

Integer

Default

60

Min

1

cephfs_mirror_restart_mirror_on_failure_interval

Description

Interval in seconds to restart failed mirror instances. Setting to zero (0) disables restarting failed mirror instances.

Type

secs

Default

`20`

Min

`0`

`cephfs_mirror_mount_timeout`

Description

Timeout in seconds for mounting primary or secondary CephFS by the `cephfs-mirror` daemon. Setting this to a higher value could result in the mirror daemon getting stalled when mounting a file system if the cluster is not reachable. This option is used to override the usual `client_mount_timeout`.

Type

`secs`

Default

`10`

Min

`0`

CephFS and the Metadata Server

Ceph File System (CephFS) is a scalable distributed file system that relies on the Metadata Server (MDS) to efficiently manage metadata and coordinate file operations.

Role of Metadata Server in CephFS

The Metadata Server (MDS) is critical to CephFS, managing file system metadata and coordinating file operations across clients.

Figure 1 illustrates the role of Metadata Server in CephFS.

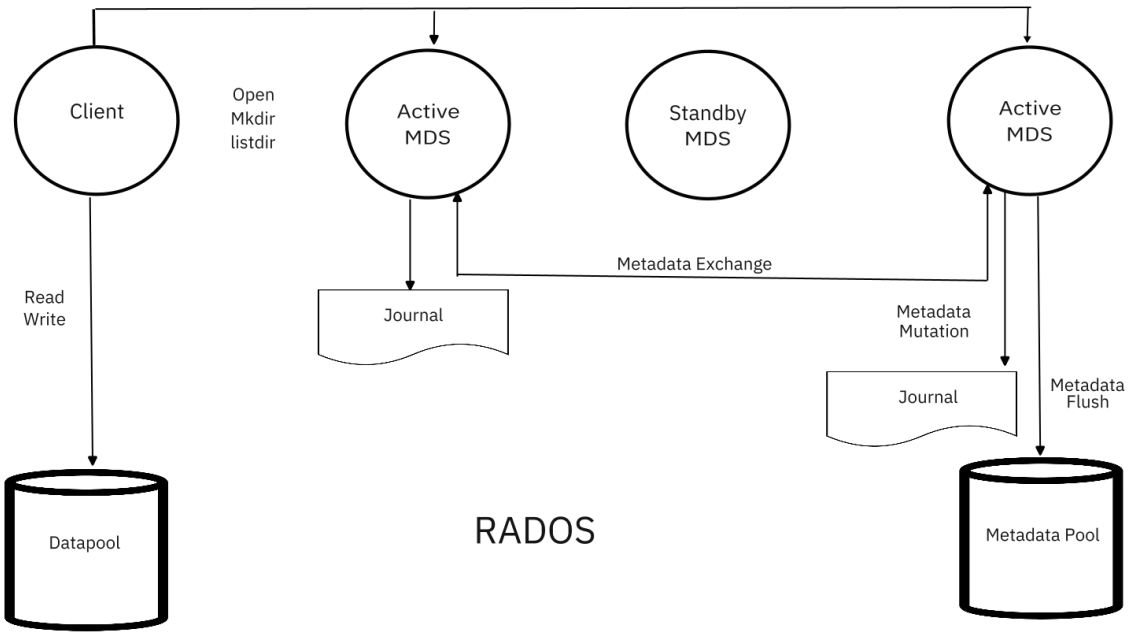


Figure 93: Role of Metadata Server in CephFS

The following outlines the key functions of the MDS:

Active MDS

Handles metadata-related client requests, such as file creation and directory management, while maintaining a cache to optimize performance.

Standby MDS

Provides high availability by taking over in the event of an active MDS failure. A standby MDS configured as standby-replay continuously applies journal changes to ensure a swift failover.

Optimizing MDS performance

Optimizing MDS performance is essential for maintaining a high-performing and reliable CephFS deployment. Key strategies include configuring single active MDS, multiple active MDS instances and Standby MDS. In CephFS, different MDS configurations can be used based on the scale and demands of the environment. The following outlines the various MDS deployment options:

Single active MDS

For smaller deployments or environments with limited metadata workloads, a single active MDS is typically sufficient. This configuration simplifies management and reduces overhead.

Multiple active MDS daemons

In larger deployments with substantial metadata workloads, multiple active MDS daemons can enhance performance by distributing the metadata load. Adjust the **`max_mds`** parameter to dynamically balance the load across multiple MDS instances, improving system responsiveness and reducing latency.

Standby MDS

Ensures high availability by automatically taking over if an active MDS fails. Configuring standby MDS as standby-replay can improve failover speed by continuously applying journal changes.

MDS memory sizing considerations

The MDS cache size directly affects how many file system objects CephFS can manage efficiently. MDS memory needs depend on the workload. They vary with the number of files and directories, the namespace structure, client activity, capability usage, and metadata access patterns. Base the **`mds_cache_memory_limit`** setting on your expected metadata workload, and watch cache utilization, cache pressure, and metadata latency over time.

Consider the following areas when you plan MDS memory and capacity:

Memory sizing

There is no fixed formula for MDS memory sizing. As a reference point, testing with a single active MDS gave these results under one specific workload:

- With the default **`mds_cache_memory_limit`** value of 4 GB, the MDS supported about 1.26 million inodes.
- With the limit raised to 8 GB, the MDS supported about 2.6 million inodes.

These results show that more MDS cache memory can greatly increase the amount of metadata that is cached and managed efficiently. They are observations from one workload, not general sizing rules. Actual results vary with workload characteristics, client activity, and directory structure.

MDS capacity scaling

When metadata performance needs to improve, increase MDS cache memory first. Add more active MDS daemons only if increasing cache memory doesn't solve the problem.

Indicators for increasing MDS memory

Consider increasing the **`mds_cache_memory_limit`** value when:

- The file system contains large numbers of files and directories.
- Metadata-intensive workloads show increased latency.
- The MDS cache regularly approaches its configured memory limit during normal operation.
- Cache pressure is consistently high.

Indicators for deploying multiple active MDS daemons

No fixed inode count determines when you need more active MDS daemons. The decision depends mainly on metadata workload characteristics, client concurrency, and MDS resource utilization. Consider configuring multiple active MDS daemons when:

- A single MDS becomes a performance bottleneck.

- Metadata latency stays high after you increase **`mds_cache_memory_limit`**.
- Large numbers of clients run metadata-intensive operations across multiple active areas of the namespace.

Metadata workload distribution

With multiple active MDS daemons configured, CephFS distributes metadata ownership automatically by using dynamic directory subtree partitioning. Different MDS ranks can manage different parts of the directory hierarchy, so the ranks process metadata operations in parallel. You don't need to assign directories or subvolumes to specific MDS daemons manually.

MDS daemon placement

For production environments, deploy active MDS daemons on separate nodes. Separate nodes give better CPU and memory isolation, better fault tolerance, and better scalability under heavy metadata workloads. You can run multiple active MDS daemons on the same node, but separate nodes are recommended for larger or performance-sensitive environments.

CephFS volumes, subvolumes, and subvolume groups

Ceph File System (CephFS) volumes, subvolumes, and subvolume groups are managed through a Ceph Manager (MGR) module.

In CephFS, several constructs are essential for efficiently managing data across various deployment scenarios and integrations. Each component serves a unique purpose, allowing for better organization, control, and policy application.

The following is a breakdown of these component constructs:

Volumes

Volumes represent the entire CephFS file system, serving as the foundational layer for file-based storage. Usually there is only one CephFS volume per Ceph cluster.

Subvolumes

Subvolumes act as per application or tenant directory trees within volumes, enabling you to organize and manage data more effectively within the larger file system. Subvolumes are usually the basis for user or application file-shares.

Subvolume groups

The subvolume groups act according to application or tenant directory trees within volumes, enabling you to organize and manage data more effectively within the larger file system. Subvolumes are usually the basis for user or application file shares.

Differences and Usage

To effectively manage data within CephFS, it’s important to understand the distinct roles played by volumes, subvolumes, and subvolume groups. Each of these constructs offers unique capabilities that contribute to the overall efficiency and organization of the file system.

The following lists the differences and usage of each component:

Volumes

Volumes provide a high-level overview and control of the entire file system, essential for overall management. Generally, there is one volume per Ceph cluster.

Subvolumes

Subvolumes allow for detailed data organization and quota management within volumes, helping to fine-tune resource allocation.
Subvolumes are ideal for creating shares for users, applications, and persistent shared file systems for Kubernetes.

Subvolume groups

Subvolume groups enable the efficient application of policies across groups of subvolumes, making it easier to manage large datasets and enforce administrative rules.
Subvolume groups can be used to simplify the management of multiple subvolumes that all belong to the same application or service.

Volume Management

Effective volume management is crucial for optimizing CephFS performance and security.

Single versus multiple volumes

Multiple file system support is stable and ready to use. This functionality allows configuring separate file systems with full data separation on separate pools. However, it is recommended to consolidate file system workloads onto a single CephFS file system, when possible. Consolidate the workloads to avoid over-allocating resources to MDS servers that can be underutilized.

Security considerations

Implement robust access controls and security policies for CephFS volumes, subvolumes, and subvolume groups to help ensure data protection and compliance for users, applications, and tenants.

File system components

The CephFS is comprised of clients and Metadata Servers (MDS). Understand the component layers of the Ceph File system and the underlying core storage cluster components.

The CephFS has two primary components:

Clients

The CephFS clients perform I/O operations on behalf of applications using CephFS, such as `ceph-fuse` for FUSE clients and `kcephfs` for kernel clients. CephFS clients send metadata requests to an active Metadata Server. In return, the CephFS client learns of the file metadata, and can begin safely caching both metadata and file data.

Metadata Servers (MDS)

The MDS does the following:

- Provides metadata to CephFS clients.
- Manages metadata related to files stored on the Ceph File System.
- Coordinates access to the shared Red Hat Ceph Storage cluster.
- Caches hot metadata to reduce requests to the backing metadata pool store.
- Manages the CephFS clients' caches to maintain cache coherence.
- Replicates hot metadata between active MDS.
- Coalesces metadata mutations to a compact journal with regular flushes to the backing metadata pool.
- CephFS requires at least one Metadata Server daemon (`ceph-mds`) to run.

Ceph File System components can be divided into four layers, as illustrated in “Figure: Ceph File System component layers” on page 1148.

The first, bottom layer represents the underlying core storage cluster components:

- Ceph OSDs (`ceph-osd`) where the Ceph File System data and metadata are stored.
- Ceph Metadata Servers (`ceph-mds`) that manages Ceph File System metadata.
- Ceph Monitors (`ceph-mon`) that manages the master copy of the cluster map.

The second level is the Ceph storage protocol layer. This represents the Ceph native `librados` library for interacting with the core storage cluster.

The third layer, on top of the Ceph storage protocol, is the CephFS library layer includes the CephFS `libcephfs` library that works on top of `librados` and represents the Ceph File System.

The top layer represents the two types of Ceph clients that can access the Ceph File Systems: CephFS kernel object and CephFS FUSE.

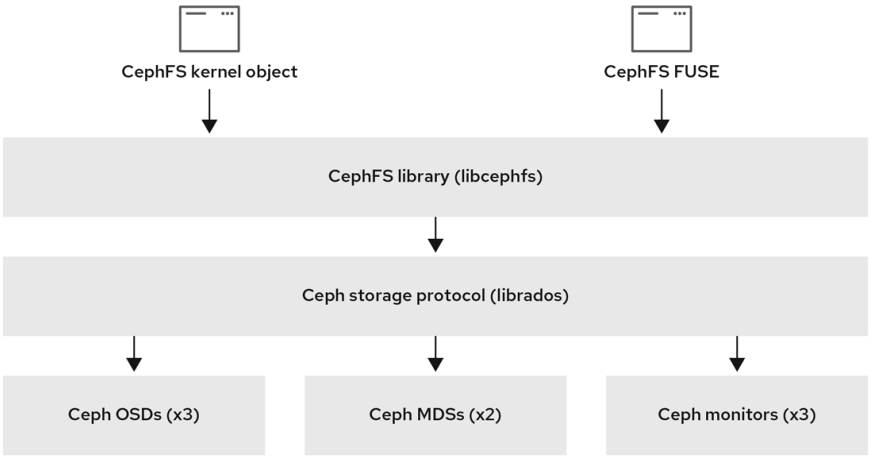


Figure 94: Ceph File System component layers

"Figure: Interaction between the Ceph File System components" on page 1148 shows more details on how the Ceph File System components interact with each other.

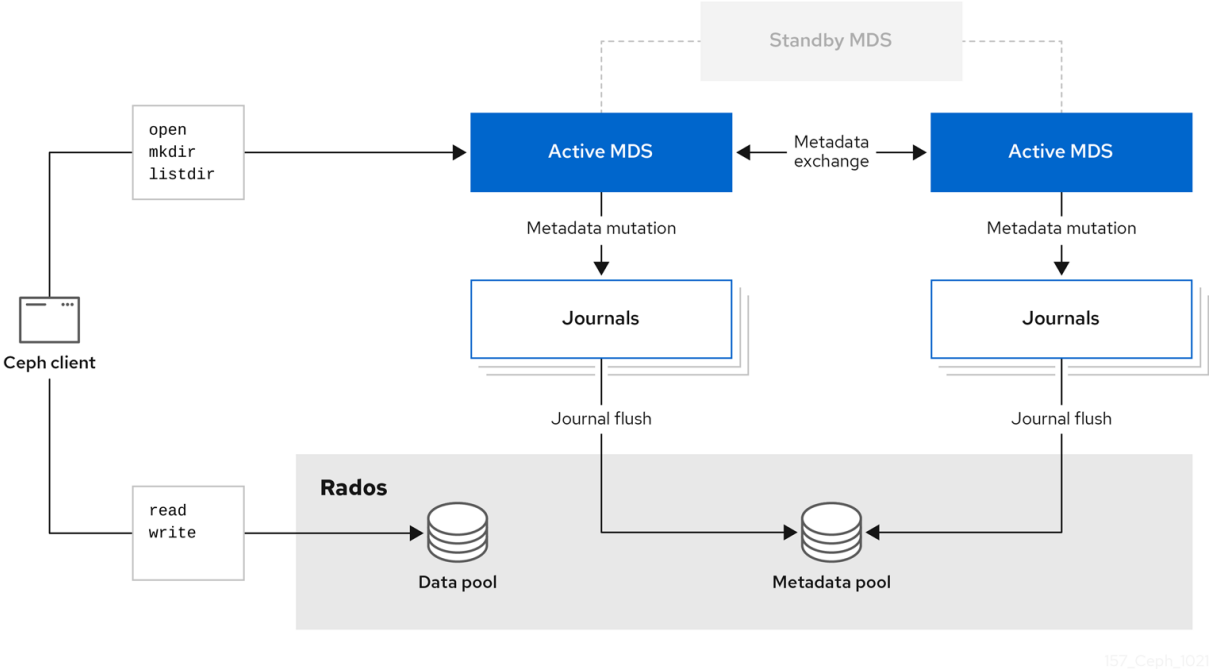


Figure 95: Interaction between the Ceph File System components

For more information, see:

- "Managing the MDS service using the Ceph Orchestrator" on page 619
- "Deploying the CephFS" on page 1165

CephFS and SELinux

Using Security-Enhanced Linux (SELinux) is supported on Ceph File Systems (CephFS) environments.

Set any SELinux file type with CephFS, along with assigning a particular SELinux type on individual files. This support applies to the CephFS Metadata Server (MDS), the CephFS File System in User Space (FUSE) clients, and the CephFS kernel clients.

Related information

Using SELinux

CephFS limitations and the POSIX standards

Learn about the CephFS limitations and the POSIX standards.

The CephFS diverges from the strict POSIX semantics in the following ways:

- If a client's attempt to write a file fails, the write operations are not necessarily atomic. For example, the client might call the `write()` system call on a file that is opened with the `O_SYNC` flag with an 8 MB buffer and then terminates unexpectedly. In this case, the write operation can be only partially applied. Almost all file systems, even local file systems, have this behavior.
- In situations when the write operations occur simultaneously, a write operation that exceeds object boundaries is not necessarily atomic. For example, writer *A* writes **"aa|aa"** and writer *B* writes **"bb|bb"** simultaneously, where **"|"** is the object boundary, and **"aa|bb"** is written rather than the proper **"aa|aa"** or **"bb|bb"**.
- POSIX includes the `telldir()` and `seekdir()` system calls that allow you to obtain the current directory offset and seek back to it. Because CephFS can fragment directories at any time, it is difficult to return a stable integer offset for a directory. As such, calling the `seekdir()` system call to a nonzero offset might often work but is not guaranteed to do so. Calling `seekdir()` to offset 0 will always work. This is equivalent to the `rewinddir()` system call.
- Sparse files propagate incorrectly to the `st_blocks` field of the `stat()` system call. CephFS does not explicitly track parts of a file that are allocated or written to because the `st_blocks` field is always populated by the quotient of file size divided by block size. This behavior causes utilities, such as `du`, to overestimate used space.
- When the `mmap()` system call maps a file into memory on multiple hosts, write operations are not coherently propagated to caches of other hosts. That is, if a page is cached on host A, and then updated on host B, host A page is not coherently invalidated.
- CephFS clients present a hidden `.snap` directory that is used to access, create, delete, and rename snapshots. Although this directory is excluded from the `readdir()` system call, any process that tries to create a file or directory with the same name returns an error. The name of this hidden directory can be changed at mount time with the `-o snapdirname=.<new_name>` option or by using the `client_snapdir` configuration option.

For more information, see:

- "Managing the MDS service using the Ceph Orchestrator" on page 619
- "Deploying the CephFS" on page 1165

CephFS Metadata Server

Learn about the different states of the Ceph File System (CephFS) Metadata Server (MDS), MDS ranking, standby MDS daemons, cache size limits, and MDS resource recommendations. Understanding these concepts helps you configure MDS daemons for your storage environment.

Before configuring the MDS memory cache limit, ensure that you have a running and operational IBM Storage Ceph cluster and that the Ceph Metadata Server (ceph-mds) daemons are installed. The MDS manages file-system metadata, including file names, directories, permissions, and file attributes, while the Ceph Object Storage Daemons (OSDs) handle the data workload.

MDS resource recommendations

The MDS resource requirements depend on the workload and the size of the metadata working set. Workloads that create, remove, access, or inspect many files require more MDS cache and CPU resources than workloads that primarily access large files sequentially.

Metadata-intensive workloads include:

- Software builds
- Continuous integration and continuous delivery pipelines
- AI and machine learning workloads that access many files
- Workloads with many small files

- Workloads that frequently use metadata operations such as `stat`, `readdir`, `create`, `unlink`, `chmod`, and `access`

Workloads with large files and primarily sequential I/O, such as video streaming and archival workloads, typically generate less metadata activity.

For metadata-intensive production workloads, consider use the following MDS configuration as a starting point:

- 128 GiB of MDS cache
- Four dedicated CPU cores for each active or standby MDS daemon
- Separate nodes for MDS daemons and other memory-intensive services

The 128 GiB cache and four-core configuration was validated with a metadata-intensive software-build workload. The configuration sustained approximately 10,000 to 15,000 operations per second during testing. Actual resource requirements depend on the application workload, metadata working set, client count, file count, directory structure, and storage-cluster performance. Test the configuration with a representative workload before you use it in production.

Increasing the MDS cache can reduce cache eviction and improve performance for metadata-intensive workloads. However, a larger cache uses more memory and can increase MDS failover time. Allocate sufficient memory to active and standby MDS daemons, and monitor the workload before increasing the cache further.

Additional CPU cores can improve MDS throughput and latency under load. During testing with a fixed 64 GiB cache, increasing the CPU allocation from one core to four cores improved sustainable throughput from approximately 9,000 operations per second to approximately 13,000 operations per second. Additional CPU resources cannot compensate for a severely undersized metadata cache.

For workloads that exceed the capacity of one active MDS daemon, configure multiple active MDS daemons. Consider using directory pinning or ephemeral pinning to distribute metadata-intensive directory trees across MDS ranks. Do not assume a fixed number of files per MDS daemon. MDS capacity depends on the workload, cache size, CPU allocation, directory structure, and metadata operation rate.

Use equivalent CPU and memory resources for active and standby MDS daemons. This configuration helps the standby daemon take over the workload if an active MDS daemon becomes unavailable.

Metadata Server daemon states

By default, a Ceph File System uses only one active MDS daemon. However, systems with many clients benefit from multiple active MDS daemons.

The Metadata Server (MDS) daemons operate in two states:

- `Active`: manages metadata for files and directories stores on the Ceph File System.
- `Standby`: serves as a backup, and becomes active when an active MDS daemon becomes unresponsive.

You can configure the file system to use multiple active MDS daemons so that you can scale metadata performance for larger workloads. The active MDS daemons dynamically share the metadata workload when metadata load patterns change. Systems with multiple active MDS daemons still require standby MDS daemons to remain highly available.

What happens when the active MDS daemon fails

When the active MDS becomes unresponsive, a Ceph Monitor daemon waits a few seconds equal to the value specified in the `mds_beacon_grace` option. If the active MDS is still unresponsive after the specified time period has passed, the Ceph Monitor marks the MDS daemon as `laggy`. One of the standby daemons becomes active, depending on the configuration.

> **Note:** To change the value of `mds_beacon_grace`, add this option to the Ceph configuration file and specify the new value.

Metadata Server ranks

Ranks define how the metadata workload is shared between multiple Metadata Server (MDS) daemons. The number of ranks is the maximum number of MDS daemons that can be active at one time. Each MDS daemon handles a subset of the CephFS metadata that is assigned to that rank.

Each Ceph File System (CephFS) has a number of ranks, one by default, which starts at zero.

Each MDS daemon initially starts without a rank. The Ceph Monitor assigns a rank to the daemon. The MDS daemon can only hold one rank at a time. Daemons only lose ranks when they are stopped.

The `max_mds` setting controls how many ranks will be created.

The actual number of ranks in the CephFS is only increased if a spare daemon is available to accept the new rank.

Rank states

Ranks can be as follows:

- Up: A rank that is assigned to the MDS daemon.
- `Failed`: A rank that is not associated with any MDS daemon.
- `Damaged`: A rank that is damaged; its metadata is corrupted or missing. Damaged ranks are not assigned to any MDS daemons until the operator fixes the problem, and uses the **`ceph mds repaired`** command on the damaged rank.

Metadata Server cache size limits

Learn how to limit the size of the Ceph File System (CephFS) Metadata Server (MDS) cache.

You can limit the size of the Ceph File System (CephFS) Metadata Server (MDS) cache by using either the memory limit or the inode count.

Memory limit

Use the `mds_cache_memory_limit` option. Use a value between 8 GB and 64 GB for `mds_cache_memory_limit`. Setting more cache can cause issues with recovery. This limit is approximately 66% of the desired maximum memory use of the MDS.

> **Important:** Use memory limits instead of inode count limits.

> **Note:** The default value for `mds_cache_memory_limit` is 4 GB. Since the default value is outside the recommended range, users are recommended to set the value within the mentioned range.

Inode count

Use the `mds_cache_size` option. By default, limiting the MDS cache by inode count is disabled.

You can also specify a cache reservation by using the `mds_cache_reservation` option for MDS operations. The cache reservation is limited as a percentage of the memory or inode limit and is set to 5% by default. The intent of this parameter is to have the MDS maintain an extra reserve of memory for its cache for new metadata operations to use. As a result, the MDS should operate below its memory limit because it recalls old state from clients to drop unused metadata in its cache.

The `mds_cache_reservation` option replaces the `mds_health_cache_threshold` option in all situations, except when MDS nodes send a health alert to the Ceph Monitors indicating the cache is too large. By default, `mds_health_cache_threshold` is 150% of the maximum cache size.

The cache limit is not a hard limit. Potential bugs in the CephFS client or MDS or misbehaving applications might cause the MDS to exceed its cache size. The `mds_health_cache_threshold` option configures the storage cluster health warning message so that operators can investigate why the MDS cannot shrink its cache.

For more information, see “Configuring Metadata Server daemons” on page 1135.

File system affinity

You can configure a Ceph File System (CephFS) to prefer a particular Ceph Metadata Server (MDS) over another Ceph MDS.

In some cases you may have two Ceph Metadata Servers, a newer and faster hardware and an older and slower one. You can specify the preference of the newer hardware, by setting the `mds_join_fs` option, which enforces this file system affinity. Ceph Monitors give preference to MDS standby daemons with `mds_join_fs` equal to the file system name with the failed rank. The standby-replay daemons are selected before choosing another standby daemon. If no standby daemon exists with the `mds_join_fs` option, then the Ceph Monitors choose an ordinary standby for replacement or any other available standby as a last resort. The Ceph Monitors periodically examine the Ceph File Systems to see whether a standby with a stronger affinity is available to replace the Ceph MDS that has a lower affinity.

Managing the MDS service using the Ceph Orchestrator

Use the Ceph Orchestrator with `cephadm` in the backend to deploy the MDS service.

By default, a Ceph File System (CephFS) uses only one active MDS daemon. However, systems with many clients benefit from multiple active MDS daemons.

When managing the MDS service by using the Ceph Orchestrator, be sure to have:

- A running Red Hat Ceph Storage cluster.
- Root-level access to all the nodes.
- Hosts added to the cluster.
- All manager, monitor, and OSD daemons deployed.

Deploying the MDS service with the command-line interface

Using the Ceph Orchestrator, you can deploy the Metadata Server (MDS) service by using the `placement` specification in the command-line interface.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Hosts added to the cluster.
- All manager, monitor, and OSD daemons deployed.

About this task

Ceph File System (CephFS) requires one or more MDS.

> **Note:** Ensure that you have at least two pools, one for Ceph file system (CephFS) data and one for CephFS metadata.

For more information about creating Ceph File Systems, see Creating Ceph File Systems.

To deploy the MDS service with the command-line interface, log in to the `cephadm` shell.

```
[root@host01 ~]# cephadm shell
```

Use one of the following options to deploy MDS daemons using placement specification.

Creating the MDS daemons through the command-line

Procedure

- Use **`ceph fs volume`** to create the MDS daemons.

Creating the MDS daemons creates the CephFS volume and pools associated with the CephFS, and also starts the MDS service on the hosts.

```
ceph fs volume create FILESYSTEM_NAME --
placement="NUMBER_OF_DAEMONS HOST_NAME_1 HOST_NAME_2 HOST_NAME_3"
```

Note: By default, replicated pools are created for this command.

For example,

```
[ceph: root@host01 /]# ceph fs volume create test --placement="2 host01 host02"
```

Creating the pools, CephFS, and then deploying MDS service using placement specification

Procedure

1. Create the pools for CephFS.

   ```
   ceph osd pool create DATA_POOL PG_NUM
   ceph osd pool create METADATA_POOL PG_NUM
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph osd pool create cephfs_data 64
   [ceph: root@host01 /]# ceph osd pool create cephfs_metadata 64
   ```

2. Create the file system for the data pools and metadata pools.

   ```
   ceph fs new FILESYSTEM_NAME METADATA_POOL DATA_POOL
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph fs new test cephfs_metadata cephfs_data
   ```

3. Deploy MDS service using the **ceph orch apply** command.

   ```
   ceph orch apply mds FILESYSTEM_NAME --
   placement="NUMBER_OF_DAEMONS HOST_NAME_1 HOST_NAME_2 HOST_NAME_3"
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch apply mds test --placement="2 host01 host02"
   ```

Verifying the MDS service deployment

Procedure

1. List the service.
   For example,

   ```
   [ceph: root@host01 /]# ceph orch ls
   ```

2. Check the CephFS status.
   For example,

   ```
   [ceph: root@host01 /]# ceph fs ls
   [ceph: root@host01 /]# ceph fs status
   ```

3. List the hosts, daemons, and processes.

   ```
   ceph orch ps --daemon_type=DAEMON_NAME
   ```

For example,

```
[ceph: root@host01 /]# ceph orch ps --daemon_type=mds
```

Deploying the MDS service using the service specification

Using the Ceph Orchestrator, you can deploy the MDS service using the service specification.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Hosts added to the cluster.
- All manager, monitor, and OSD daemons deployed.

About this task

Note: Ensure that you have at least two pools, one for the Ceph File System (CephFS) data and one for the CephFS metadata.

Procedure

1. Create the `mds.yaml` file.

   ```
   touch mds.yaml
   ```

2. Edit the `mds.yaml` file.

   ```
   service_type: mds
   service_id: FILESYSTEM_NAME
   placement:
     hosts:
     - HOST_NAME_1
     - HOST_NAME_2
     - HOST_NAME_3
   ```

   For example,

   ```
   service_type: mds
   service_id: fs_name
   placement:
     hosts:
     - host01
     - host02
   ```

3. Mount the YAML file under a directory in the container.
   For example,

   ```
   [root@host01 ~]# cephadm shell --mount mds.yaml:/var/lib/ceph/mds/mds.yaml
   ```

4. Navigate to the `ceph/mds/` directory.
   For example,

   ```
   [ceph: root@host01 /]# cd /var/lib/ceph/mds/
   ```

5. Log into the `cephadm` shell.

   ```
   cephadm shell
   ```

6. Navigate to the `/ceph/mds/` directory.
   For example,

   ```
   [ceph: root@host01 /]# cd /var/lib/ceph/mds/
   ```

7. Deploy MDS service using service specification.

```
ceph orch apply -i FILE_NAME.yaml
```

For example,

```
[ceph: root@host01 mds]# ceph orch apply -i mds.yaml
```

8. Once the MDS services is deployed and functional, create the CephFS.

```
ceph fs new CEPHFS_NAME METADATA_POOL DATA_POOL
```

For example,

```
[ceph: root@host01 /]# ceph fs new test metadata_pool data_pool
```

What to do next

Verify the MDS service deployment.

1. List the service.

```
[ceph: root@host01 /]# ceph orch ls
```

2. List the hosts, daemons, and processes.

```
ceph orch ps --daemon_type=DAEMON_NAME
```

For example,

```
[ceph: root@host01 /]# ceph orch ps --daemon_type=mds
```

Removing the MDS service using the Ceph Orchestrator

Remove the service either by using the **ceph orch rm** command or by removing the file system and the associated pools.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Hosts added to the cluster.
- Root-level access to all the nodes.
- At least one MDS daemon deployed on the hosts.

About this task

To remove the MDS service with the command-line interface, log in to the `cephadm` shell.

```
[root@host01 ~]# cephadm shell
```

Use one of the following options to remove MDS daemons.

Removing the MDS service from the entire cluster through the command line

Procedure

Use the **ceph orch rm** command to remove the MDS service from the entire cluster.

1. List the service.

```
ceph orch ls
```

For example,

```
[ceph: root@host01 /]# ceph orch ls
```

2. Remove the service.

```
ceph orch rm SERVICE_NAME
```

For example,

```
[ceph: root@host01 /]# ceph orch rm mds.test
```

Removing the CephFS volume, associated pools, and the services

Procedure

1. Set the configuration parameter **mon_allow_pool_delete** to `true`.

```
ceph config set mon mon_allow_pool_delete true
```

For example,

```
[ceph: root@host01 /]# ceph config set mon mon_allow_pool_delete true
```

2. Remove the file system, its data, and metadata pools.

```
ceph fs volume rm FILESYSTEM_NAME --yes-i-really-mean-it
```

This command also tries to remove the MDS using the enabled `ceph-mgr` Orchestrator module.

```
ceph fs volume rm cephfs-new --yes-i-really-mean-it
```

For example,

```
[ceph: root@host01 /]# ceph fs volume rm cephfs-new --yes-i-really-mean-it
```

Verifying the MDS service removal

Procedure

- List the hosts, daemons, and processes, by using the **ceph orch ps** command.
  For example,

```
[ceph: root@host01 /]# ceph orch ps
```

Configuring file system affinity

Set the Ceph File System (CephFS) affinity for a particular Ceph Metadata Server (MDS).

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A healthy, and running Red Hat Ceph Storage.
- A Ceph File System created.
- Root-level access to a Ceph Monitor node.

About this task

For more information about file system affinity, see “File system affinity” on page 1152.

Procedure

1. Check the current state of a Ceph File System.

```
[root@mon ~]# ceph fs dump
dumped fsmap epoch 399
...
Filesystem 'cephfs01' (27)
...
e399
max_mds 1
in      0
up      {0=20384}
failed
damaged
stopped
...
[mds.a{0:20384} state up:active seq 239 addr
[v2:127.0.0.1:6854/966242805,v1:127.0.0.1:6855/966242805]]

Standby daemons:

[mds.b{-1:10420} state up:standby seq 2 addr
[v2:127.0.0.1:6856/2745199145,v1:127.0.0.1:6857/2745199145]]
```

2. Set the file system affinity.

```
ceph config set STANDBY_DAEMON mds_join_fs FILE_SYSTEM_NAME
```

For example,

```
[root@mon ~]# ceph config set mds.b mds_join_fs cephfs01
```

After a Ceph MDS failover event, the file system favors the standby daemon for which the affinity is set, as shown in the following example.

Note the `mds.b` daemon now has the `join_fscid=27` in the file system dump output.

```
[root@mon ~]# ceph fs dump
dumped fsmap epoch 405
e405
...
Filesystem 'cephfs01' (27)
...
max_mds 1
in      0
up      {0=10420}
failed
damaged
stopped
...
[mds.b{0:10420} state up:active seq 274 join_fscid=27 addr
[v2:127.0.0.1:6856/2745199145,v1:127.0.0.1:6857/2745199145]]

Standby daemons:

[mds.a{-1:10720} state up:standby seq 2 addr
[v2:127.0.0.1:6854/1340357658,v1:127.0.0.1:6855/1340357658]]
```

> **Note:** If a file system is in a degraded or undersized state, then no failover occurs to enforce the file system affinity.

Configuring multiple active Metadata Server daemons

Configure multiple active Metadata Server (MDS) daemons to scale metadata performance for large systems.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Ceph administration capabilities on the MDS node.
- Root-level access to a Ceph Monitor node.

About this task

Important: Do not convert all standby MDS daemons to active ones. A Ceph File System (CephFS) requires at least one standby MDS daemon to remain highly available.

Procedure

1. Set the `max_mds` parameter to the desired number of active MDS daemons.

   ```
   ceph fs set NAME max_mds NUMBER
   ```

   For example, to increase the number of active MDS daemons to two in the CephFS called `cephfs`:

   ```
   [root@mon ~]# ceph fs set cephfs max_mds 2
   ```

   Note: Ceph only increases the actual number of ranks in the CephFS if a spare MDS daemon is available to take the new rank.

2. Verify the number of active MDS daemons.

   ```
   ceph fs status NAME
   ```

   For example:

   ```
   [root@mon ~]# ceph fs status cephfs
   cephfs - 0 clients
   ======
   +------+--------+-------+---------------+-------+-------+--------+--------+
   | RANK | STATE  |  MDS  |    ACTIVITY   |  DNS  |  INOS |  DIRS  |  CAPS  |
   +------+--------+-------+---------------+-------+-------+--------+--------+
   |  0   | active | node1 | Reqs:    0 /s |   10  |   12  |   12   |   0    |
   |  1   | active | node2 | Reqs:    0 /s |   10  |   12  |   12   |   0    |
   +------+--------+-------+---------------+-------+-------+--------+--------+
   +-----------------+----------+-------+-------+
   |       POOL      |   TYPE   |  USED | AVAIL |
   +-----------------+----------+-------+-------+
   | cephfs_metadata | metadata | 4638  | 26.7G |
   |   cephfs_data   |   data   |    0  | 26.7G |
   +-----------------+----------+-------+-------+

   +-------------+
   | STANDBY MDS |
   +-------------+
   |    node3    |
   +-------------+
   ```

Related information

Managing Ceph users

Configuring the number of standby daemons

Each Ceph File System (CephFS) can specify the required number of standby daemons to be considered healthy. This number also includes the standby-replay daemon waiting for a rank failure.

Before you begin

Be sure that you have root-level access to a Ceph Monitor node.

Procedure

– Set the expected number of standby daemons for a particular CephFS.

```
ceph fs set FS_NAME standby_count_wanted NUMBER
```

> **Note:** Setting the *NUMBER* to zero disables the daemon health check.

For example, to set the expected standby daemon count to two:

```
[root@mon ~]# ceph fs set cephfs standby_count_wanted 2
```

Configuring the standby-replay Metadata Server

Configure each Ceph File System (CephFS) by adding a standby-replay Metadata Server (MDS) daemon. Adding a standby-replay MDS daemon reduces failover time if the active MDS becomes unavailable.

Before you begin

Be sure that you have root-level access to a Ceph Monitor node.

About this task

This specific standby-replay daemon follows the active MDS's metadata journal. The standby-replay daemon is only used by the active MDS of the same rank, and is not available to other ranks.

> **Important:** If using standby-replay, then every active MDS must have a standby-replay daemon.

Procedure

– Set the standby-replay for a particular CephFS.

```
ceph fs set FS_NAME allow_standby_replay 1
```

> **Note:** Setting the *NUMBER* to zero disables the daemon health check.

For example, to enable the standby-replay daemons to be assigned to the active Ceph MDS daemons, set the Boolean value to1

```
[root@mon ~]# ceph fs set cephfs allow_standby_replay 1
```

For more information, see "Using the ceph mds fail command" on page 1212

Ephemeral pinning policies

An ephemeral pin is a static partition of subtrees, and can be set with a policy that uses extended attributes. A policy can automatically set ephemeral pins to directories. When setting an ephemeral pin to a directory, it is automatically assigned to a particular rank, as to be uniformly distributed across all Ceph MDS ranks.

Determining which rank gets assigned is done by a consistent hash and the directory's inode number. Ephemeral pins do not persist when the directory's inode is dropped from file system cache. When failing over a Ceph Metadata Server (MDS), the ephemeral pin is recorded in its journal so the Ceph MDS standby server does not lose this information.

> **Note:** Installation of the `attr` and `jq` package must be installed as a prerequisite for the ephemeral pinning policies.

There are two types of policies for using ephemeral pins:

Distributed

This policy enforces that all of a directory's immediate children must be ephemerally pinned. For example, use a distributed policy to spread a user's home directory across the entire Ceph File System cluster. Enable this policy by setting the `ceph.dir.pin.distributed` extended attribute.

Syntax

```
setfattr -n ceph.dir.pin.distributed -v 1 DIRECTORY_PATH
```

Example

```
[root@host01 mount]# setfattr -n ceph.dir.pin.distributed -v 1 dir1/
```

Random

This policy enforces a chance that any descendent subdirectory might be ephemerally pinned. You can customize the percent of directories that can be ephemerally pinned. Enable this policy by setting the `ceph.dir.pin.random` and setting a percentage. Set the percentage to a value smaller than 1% (`0.01`). Having too many subtree partitions can cause slow performance. You can set the maximum percentage by setting the `mds_export_ephemeral_random_max` Ceph MDS configuration option.
Syntax

```
setfattr -n ceph.dir.pin.random -v PERCENTAGE_IN_DECIMAL DIRECTORY_PATH
```

Example

```
[root@host01 mount]# setfattr -n ceph.dir.pin.random -v  0.01 dir1/
```

By default, both of these ephemeral pin policies are disabled. This feature can be enabled by setting either the `mds_export_ephemeral_distributed` or `mds_export_ephemeral_random` Ceph MDS configuration options.

After enabling pinning, you can **verify** by running either of the following commands.

Syntax

```
getfattr -n ceph.dir.pin.random DIRECTORY_PATH
getfattr -n ceph.dir.pin.distributed DIRECTORY_PATH
```

Example

```
[root@host01 mount]# getfattr -n ceph.dir.pin.distributed dir1/
# file: dir1/
ceph.dir.pin.distributed="1"

[root@host01 mount]# getfattr -n ceph.dir.pin.random dir1/
# file: dir1/
ceph.dir.pin.random="0.01"
```

Example

```
ceph tell mds.a get subtrees | jq '.[] | [.dir.path, .auth_first, .export_pin]'
```

If the directory is pinned, the value of `export_pin` is `0` if it is pinned to rank `0`, `1` if it is pinned to rank `1`, and so on. If the directory is not pinned, the value is `-1`.

To **remove** a partitioning policy, remove the extended attributes or set the value to `0`.

Syntax

```
 setfattr -n ceph.dir.pin.distributed -v 0 DIRECTORY_PATH
```

Example

```
[root@host01 mount]# setfattr -n ceph.dir.pin.distributed -v 0 dir1/
```

You can **verify** by running either of the following commands:

Syntax

```
getfattr -n ceph.dir.pin.distributed DIRECTORY_PATH
```

Example

```
[root@host01 mount]# getfattr -n ceph.dir.pin.distributed dir1/
```

For export pins, remove the extended attribute or set the extended attribute to `-1`.

Syntax

```
setfattr -n ceph.dir.pin -v -1 DIRECTORY_PATH
```

Example

```
[root@host01 mount]# setfattr -n ceph.dir.pin -v -1 dir1/
```

For more information about manually setting pins, see “Manually pinning directory trees to a particular rank” on page 1161.

Manually pinning directory trees to a particular rank

You can manually override the dynamic balancer with explicit mappings of metadata to a particular Ceph Metadata Server (MDS) rank. This can be done to evenly spread the load of an application or to limit the impact of users metadata requests on the CephFS cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- A running Ceph File System.
- Root-level access to the CephFS client.
- The `attr` package installed.

About this task

Manually pinning directories is also known as an export pin by setting the `ceph.dir.pin` extended attribute.

A directory’s export pin is inherited from its closest parent directory but can be overwritten by setting an export pin on that directory. Setting an export pin on a directory affects all of its subdirectories, for example:

```
[root@client ~]# mkdir -p a/b (1)
[root@client ~]# setfattr -n ceph.dir.pin -v 1 a/ (2)
[root@client ~]# setfattr -n ceph.dir.pin -v 0 a/b (3)”
```

(1) Directories `a/` and `a/b` both start without an export pin set.
(2) Directories `a/` and `a/b` are now pinned to rank `1`.
(3) Directory `a/b` is now pinned to rank `0` and directory `a/` and the rest of its subdirectories are still pinned to rank `1`.

For more information about automatically setting pins, see “Ephemeral pinning policies” on page 1159.

Procedure

- Set the export pin on a directory.

  ```
  setfattr -n ceph.dir.pin -v RANK PATH_TO_DIRECTORY
  ```

  For example:

  ```
  [root@client ~]# setfattr -n ceph.dir.pin -v 2 cephfs/home
  ```

Decreasing the number of active Metadata Server daemons

Learn how to decrease the number of active Ceph File System (CephFS) Metadata Server (MDS) daemons.

Before you begin

A rank that is to be removed, must first be active. This means that you must have the same number of MDS daemons as specified by the **max_mds** parameter.

Before you begin be sure that you have root-level access to a Ceph Monitor node.

About this task

For more information about Metadata Server daemons, see “Metadata Server daemon states” on page 1150 and “Configuring multiple active Metadata Server daemons” on page 1157.

Procedure

1. Set the same number of MDS daemons as specified by the **max_mds** parameter.

   ```
   ceph fs status NAME
   ```

   For example:

   ```
   [root@mon ~]# ceph fs status cephfs
   cephfs - 0 clients

   +------+--------+-------+---------------+-------+-------+--------+--------+
   | RANK | STATE  |  MDS  |    ACTIVITY   |  DNS  |  INOS |  DIRS  |  CAPS  |
   +------+--------+-------+---------------+-------+-------+--------+--------+
   |  0   | active | node1 | Reqs:    0 /s |   10  |   12  |   12   |   0    |
   |  1   | active | node2 | Reqs:    0 /s |   10  |   12  |   12   |   0    |
   +------+--------+-------+---------------+-------+-------+--------+--------+
   +-----------------+----------+-------+-------+
   |       POOL      |   TYPE   |  USED | AVAIL |
   +-----------------+----------+-------+-------+
   | cephfs_metadata | metadata | 4638  | 26.7G |
   |   cephfs_data   |   data   |    0  | 26.7G |
   +-----------------+----------+-------+-------+

   +-------------+
   | Standby MDS |
   +-------------+
   |    node3    |
   +-------------+
   ```

2. On a node with administration capabilities, change the **max_mds** parameter to the desired number of active MDS daemons.

   ```
   ceph fs set NAME max_mds NUMBER
   ```

   For example:

   ```
   [root@mon ~]# ceph fs set cephfs max_mds 1
   ```

3. Wait for the storage cluster to stabilize to the new **max_mds** value by watching the Ceph File System status.

4. Verify the number of active MDS daemons.

   ```
   ceph fs status NAME
   ```

   For example:

```
[root@mon ~]# ceph fs status cephfs
cephfs - 0 clients

+------+--------+-------+---------------+-------+-------+--------+--------+
| RANK | STATE  |  MDS  |    ACTIVITY   |  DNS  |  INOS |  DIRS  |  CAPS  |
+------+--------+-------+---------------+-------+-------+--------+--------+
|  0   | active | node1 | Reqs:    0 /s |   10  |   12  |   12   |   0    |
+------+--------+-------+---------------+-------+-------+--------|--------+
+-----------------+----------+-------+-------+
|       POOl      |   TYPE   |  USED | AVAIL |
+-----------------+----------+-------+-------+
| cephfs_metadata | metadata | 4638  | 26.7G |
|   cephfs_data   |   data   |    0  | 26.7G |
+-----------------+----------+-------+-------+

+-------------+
| Standby MDS |
+-------------+
|    node3    |
|    node2    |
+-------------+
```

Viewing metrics for Ceph metadata server clients

You can use the command-line interface to view the metrics for the Ceph metadata server (MDS).

Before you begin

- A running Red Hat Ceph Storage cluster.

About this task

CephFS uses `Perf Counters` to track metrics. You can view the metrics using the **`counter dump`** command.

Procedure

1. Get the name of the *mds* service:
   Syntax:

   ```
   [ceph: root@mds-host01 /]# ceph orch ps | grep mds
   ```

2. Check the MDS per client metrics:
   Syntax:

   ```
   [ceph: root@mds-host01 /]# ceph tell MDS_SERVICE_NAME counter dump
   ```

   Example:

   ```
   [root@ceph2-hk-n-0mfqao-node1-installer ~]# ceph tell mds.cephfs.ceph2-hk-n-0mfqao-
   node4.isztbk counter dump

   [
     {
       "key": "mds_client_metrics",
       "value": [
         {
           "labels": {
             "fs_name": "cephfs",
             "id": "24379"
           },
           "counters": {
             "num_clients": 4
           }
         }
       ]
     },
     {
       "key": "mds_client_metrics-cephfs",
       "value": [
         {
           "labels": {
             "client": "client.24413",
             "rank": "0"
   ```

```
    },
    "counters": {
      "cap_hits": 56,
      "cap_miss": 9,
      "avg_read_latency": 0E-9,
      "avg_write_latency": 0E-9,
      "avg_metadata_latency": 0E-9,
      "dentry_lease_hits": 2,
      "dentry_lease_miss": 12,
      "opened_files": 0,
      "opened_inodes": 9,
      "pinned_icaps": 4,
      "total_inodes": 9,
      "total_read_ops": 0,
      "total_read_size": 0,
      "total_write_ops": 0,
      "total_write_size": 0
    }
  },
  {
    "labels": {
      "client": "client.24502",
      "rank": "0"
    },
    "counters": {
      "cap_hits": 921403,
      "cap_miss": 102382,
      "avg_read_latency": 0E-9,
      "avg_write_latency": 0E-9,
      "avg_metadata_latency": 0E-9,
      "dentry_lease_hits": 17117,
      "dentry_lease_miss": 204710,
      "opened_files": 0,
      "opened_inodes": 9,
      "pinned_icaps": 7,
      "total_inodes": 9,
      "total_read_ops": 0,
      "total_read_size": 0,
      "total_write_ops": 1,
      "total_write_size": 132
    }
  },
  {
    "labels": {
      "client": "client.24508",
      "rank": "0"
    },
    "counters": {
      "cap_hits": 928694,
      "cap_miss": 103183,
      "avg_read_latency": 0E-9,
      "avg_write_latency": 0E-9,
      "avg_metadata_latency": 0E-9,
      "dentry_lease_hits": 17217,
      "dentry_lease_miss": 206348,
      "opened_files": 0,
      "opened_inodes": 9,
      "pinned_icaps": 7,
      "total_inodes": 9,
      "total_read_ops": 0,
      "total_read_size": 0,
      "total_write_ops": 1,
      "total_write_size": 132
    }
  },
  {
    "labels": {
      "client": "client.24520",
      "rank": "0"
    },
    "counters": {
      "cap_hits": 56,
      "cap_miss": 9,
      "avg_read_latency": 0E-9,
      "avg_write_latency": 0E-9,
      "avg_metadata_latency": 0E-9,
      "dentry_lease_hits": 2,
      "dentry_lease_miss": 12,
      "opened_files": 0,
      "opened_inodes": 9,
      "pinned_icaps": 4,
```

```
            "total_inodes": 9,
            "total_read_ops": 0,
            "total_read_size": 0,
            "total_write_ops": 0,
            "total_write_size": 0
          }
        }
      ]
    }
]
```

Client metrics description:

CephFS exports client metrics as `Labeled Perf Counters`, which you can use to monitor the client performance. CephFS exports the below client metric.

Table 133: Client metrics description

Name	Type	Description
cap_hits	Gauge	Percentage of file capability hits over total number of caps.
cap_miss	Gauge	Percentage of file capability misses over total number of caps.
avg_read_latency	Gauge	Mean value of the read latencies.
avg_write_latency	Gauge	Mean value of the write latencies.
avg_metadata_latency	Gauge	Mean value of the metadata latencies
dentry_lease_hits	Gauge	Percentage of dentry lease hits handed out over the total dentry lease request.
dentry_lease_miss	Gauge	Percentage of dentry lease misses handed out over the total dentry lease requests.
opened_files	Gauge	Number of opened files.
opened_inodes	Gauge	Number of opened inode.
pinned_icaps	Gauge	Number of pinned Inode Caps.
total_inodes	Gauge	Total number of Nodes.
total_read_ops	Gauge	Total number of read operations generated by all process.
total_read_size	Gauge	Number of bytes read in input/output operations generated by all process.
total_write_ops	Gauge	Total number of write operations generated by all process.
total_write_size	Gauge	Number of bytes written in input/output operations generated by all processes.

Deploying the CephFS

You can deploy Ceph File Systems (CephFS) in a storage environment and have clients mount those Ceph File Systems to meet the storage needs.

The deployment workflow is three steps:

1. Create CephFS on a Ceph Monitor node.
2. Create a Ceph client user with the appropriate capabilities and make the client key available on the node where the Ceph File System will be mounted.
3. Mount CephFS on a dedicated node, by using either a kernel client or a file system in User Space (FUSE) client.

Before deploying CephFS , be sure that you have a running and healthy Red Hat Ceph Storage cluster and the Ceph Metadata Server daemon (`ceph-mds`) installed and configured.

For more information, see:

– “Managing the MDS service using the Ceph Orchestrator” on page 619

- “Creating Ceph File Systems” on page 1167
- “Creating client users for a Ceph File System” on page 1171
- “Mounting the Ceph File System as a kernel client” on page 1174
- “Mounting the Ceph File System as a FUSE client” on page 1177
- “CephFS Metadata Server” on page 1149

Layout, quota, snapshot, and network restrictions

These user capabilities help restrict access to a Ceph File System (CephFS) based on the needed requirements.

> **Important:** All user capability flags, except `rw`, must be specified in alphabetical order.

For more information about setting the Ceph user capabilities, see “Creating client users for a Ceph File System” on page 1171.

Layouts and quotas

When using layouts or quotas clients require the `p` flag, in addition to `rw` capabilities. Setting the `p` flag restricts all the attributes being set by special extended attributes, those with a `ceph.` prefix. Also, this restricts other means of setting these fields, such as `openc` operations with layouts.

In the following example, `client.0` can modify layouts and quotas on the file system `cephfs_a`, but `client.1` cannot.

```
client.0
    key: AQAz7EVWygILFRAAdIcuJ10opU/JKyfFmxhuaw==
    caps: [mds] allow rwp
    caps: [mon] allow r
    caps: [osd] allow rw tag cephfs data=cephfs_a

client.1
    key: AQAz7EVWygILFRAAdIcuJ11opU/JKyfFmxhuaw==
    caps: [mds] allow rw
    caps: [mon] allow r
    caps: [osd] allow rw tag cephfs data=cephfs_a
```

Snapshots

When creating or deleting snapshots, clients require the `s` flag, in addition to `rw` capabilities. When the capability string also contains the `p` flag, the `s` flag must appear after it.

In the following example, `client.0` can create or delete snapshots in the `temp` directory of file system `cephfs_a`.

```
client.0
    key: AQAz7EVWygILFRAAdIcuJ10opU/JKyfFmxhuaw==
    caps: [mds] allow rw, allow rws path=/temp
    caps: [mon] allow r
    caps: [osd] allow rw tag cephfs data=cephfs_a
```

Network

Restrict clients connecting from a particular network.

The optional network and prefix length is in CIDR notation.

In the following example, the optional network and prefix length is `10.0.0.0/8`.

```
client.0
  key: AQAz7EVWygILFRAAdIcuJ10opU/JKyfFmxhuaw==
  caps: [mds] allow r network 10.0.0.0/8, allow rw path=/bar network 10.0.0.0/8
  caps: [mon] allow r network 10.0.0.0/8
  caps: [osd] allow rw tag cephfs data=cephfs_a network 10.0.0.0/8
```

Creating Ceph File Systems

Create multiple Ceph File Systems (CephFS) on a Ceph Monitor node.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running and healthy Red Hat Ceph Storage cluster.
- Root-level access to a Ceph Monitor node.
- Root-level access to a Ceph client node.

About this task

For more information, see the following:

- “Creating client users for a Ceph File System” on page 1171
- “Mounting the Ceph File System as a kernel client” on page 1174
- “Mounting the Ceph File System as a FUSE client” on page 1177
- “CephFS limitations and the POSIX standards” on page 1149
- Pools overview

Procedure

1. Configure the client node to use the Ceph storage cluster.
   a. Enable the latestRed Hat Ceph Storage 9 tools repository.

      ```
      subscription-manager repos --enable=rhceph-9-tools-for-rhel-9-x86_64-rpms
      ```

   b. Install the `ceph-fuse` package.

      ```
      dnf install ceph-fuse
      ```

   c. Copy the Ceph client keyring from the Ceph Monitor node to the client node.

      ```
      scp root@MONITOR_NODE_NAME:/etc/ceph/KEYRING_FILE /etc/ceph/
      ```

      Replace *MONITOR_NODE_NAME* with the Ceph Monitor hostname or IP address.
      For example,

      ```
      [root@client ~]# scp root@192.168.0.1:/etc/ceph/ceph.client.1.keyring /etc/ceph/
      ```

   d. Copy the Ceph configuration file from a Ceph Monitor node to the client node.

      ```
      scp root@MONITOR_NODE_NAME:/etc/ceph/ceph.conf /etc/ceph/ceph.conf
      ```

      Replace *MONITOR_NODE_NAME* with the Ceph Monitor hostname or IP address.
      For example,

      ```
      [root@client ~]# scp root@192.168.0.1:/etc/ceph/ceph.conf /etc/ceph/ceph.conf
      ```

   e. Set the appropriate permissions for the configuration file.

      ```
      chmod 644 /etc/ceph/ceph.conf
      ```

2. Create a Ceph File System on a Ceph Monitor node.

   ```
   ceph fs volume create FILE_SYSTEM_NAME
   ```

   For example,

   ```
   [root@mon ~]# ceph fs volume create cephfs01
   ```

Repeat this step to create more file systems.

> **Note:** By running this command, Ceph automatically creates the new pools and deploys a new Ceph Metadata Server (MDS) daemon to support the new file system. This command also configures the MDS affinity.

3. Verify access to the new Ceph File System from a Ceph client.
   a. Authorize a Ceph client to access the new file system.

```
ceph fs authorize FILE_SYSTEM_NAME CLIENT_NAME DIRECTORY PERMISSIONS
```

> **Important:** The Ceph client can only see the CephFS it is authorized for.

   For example,

```
[root@mon ~]# ceph fs authorize cephfs01 client.1 / rw
[client.1]
    key = BQAmthpf81M+JhAAiHDYQkMiCq3x+J0n9e8REK==

[root@mon ~]# ceph auth get client.1
exported keyring for client.1
[client.1]
    key = BQAmthpf81M+JhAAiHDYQkMiCq3x+J0n9e8REK==
    caps mds = "allow rw fsname=cephfs01"
    caps mon = "allow r fsname=cephfs01"
    caps osd = "allow rw tag cephfs data=cephfs01"
```

> **Note:** Optionally, you can add a safety measure by specifying the `root_squash` option. This prevents accidental deletion scenarios by disallowing clients with a `uid=0` or `gid=0` to do write operations, but still allows read operations.
> In the following example, `root_squash` is enabled for the file system `cephfs01`, except within the `/volumes` directory tree.
>
> ```
> [root@mon ~]# ceph fs authorize cephfs01 client.1 / rw root_squash /volumes
> rw
> [client.1]
>     key = BQAmthpf81M+JhAAiHDYQkMiCq3x+J0n9e8REK==
>
> [root@mon ~]# ceph auth get client.1
> [client.1]
>     key = BQAmthpf81M+JhAAiHDYQkMiCq3x+J0n9e8REK==
>     caps mds = "allow rw fsname=cephfs01 root_squash, allow rw
> fsname=cephfs01 path=/volumes"
>     caps mon = "allow r fsname=cephfs01"
>     caps osd = "allow rw tag cephfs data=cephfs01"
> ```

   b. Copy the Ceph user's keyring to the Ceph client node.

```
ceph auth get CLIENT_NAME > OUTPUT_FILE_NAME
scp OUTPUT_FILE_NAMETARGET_NODE_NAME:/etc/ceph
```

   For example,

```
[root@mon ~]# ceph auth get client.1 > ceph.client.1.keyring
exported keyring for client.1
[root@mon ~]# scp ceph.client.1.keyring client:/etc/ceph
root@client's password:
ceph.client.1.keyring                     100%  178   333.0KB/s   00:00
```

   c. On the Ceph client node, create a new directory.

```
mkdir PATH_TO_NEW_DIRECTORY_NAME
```

For example,

```
[root@client ~]# mkdir /mnt/mycephfs
```

d. On the Ceph client node, mount the new Ceph File System.

```
ceph-fuse PATH_TO_NEW_DIRECTORY_NAME -n CEPH_USER_NAME --client-
fs=FILE_SYSTEM_NAME
```

For example,

```
[root@client ~]# ceph-fuse /mnt/mycephfs/ -n client.1 --client-fs=cephfs01
ceph-fuse[555001]: starting ceph client
2022-05-09T07:33:27.158+0000 7f11feb81200 -1 init, newargv = 0x55fc4269d5d0
newargc=15
ceph-fuse[555001]: starting fuse
```

e. On the Ceph client node, either list the directory contents of the new mount point or create a file on the new mount point.

Adding an erasure-coded pool to a Ceph File System

By default, Ceph uses replicated pools for data pools. However, another erasure-coded data pool to the Ceph File System can be added, as needed.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- An existing Ceph File System.
- Pools that use BlueStore OSDs.
- Root-level access to a Ceph Monitor node.
- The `attr` package installed.

About this task

Ceph File Systems (CephFS) backed by erasure-coded pools use less overall storage compared to Ceph File Systems that are backed by replicated pools. While erasure-coded pools use less overall storage, they also use more memory and processor resources than replicated pools.

> **Important:** For production environments, use the default replicated data pool for CephFS. The creation of inodes in CephFS creates at least one object in the default data pool. It is better to use a replicated pool for the default data to improve small-object write performance, and to improve read performance for updating backtraces.

For more information, see:

- “CephFS Metadata Server” on page 1149
- “Creating Ceph File Systems” on page 1167
- “Erasure code pools overview” on page 169

Procedure

1. Create an erasure-coded data pool for CephFS.

```
ceph osd pool create DATA_POOL_NAME erasure
```

For example:

```
[root@mon ~]# ceph osd pool create cephfs-data-ec01 erasure
pool 'cephfs-data-ec01' created
```

2. Verify that the pool was added.

```
[root@mon ~]# ceph osd lspools
```

3. Enable overwrites on the erasure-coded pool.

```
ceph osd pool set DATA_POOL_NAME allow_ec_overwrites true
```

For example:

```
[root@mon ~]# ceph osd pool set cephfs-data-ec01 allow_ec_overwrites true
set pool 15 allow_ec_overwrites to true
```

4. Verify the status of the Ceph File System.

```
ceph fs status FILE_SYSTEM_NAME
```

For example:

```
[root@mon ~]# ceph fs status cephfs-ec
cephfs-ec - 14 clients
=========
RANK  STATE              MDS                ACTIVITY     DNS    INOS   DIRS   CAPS
 0    active  cephfs-ec.example.ooymyq  Reqs:    0 /s  8231   8233    891    921
       POOL           TYPE     USED  AVAIL
cephfs-metadata-ec  metadata   787M  8274G
  cephfs-data-ec      data    2360G  12.1T

       STANDBY MDS
cephfs-ec.example.irsrql
cephfs-ec.example.cauuaj
```

5. Add the erasure-coded data pool to the existing CephFS.

```
ceph fs add_data_pool FILE_SYSTEM_NAME DATA_POOL_NAME
```

In the following example, the new data pool, `cephfs-data-ec01`, is added to the existing erasure-coded file system, `cephfs-ec`.

```
[root@mon ~]# ceph fs add_data_pool cephfs-ec cephfs-data-ec01
```

6. Verify that the erasure-coded pool was added to the Ceph File System.

```
ceph fs status FILE_SYSTEM_NAME
```

For example:

```
[root@mon ~]# ceph fs status cephfs-ec
cephfs-ec - 14 clients
=========
RANK  STATE              MDS                ACTIVITY     DNS    INOS   DIRS   CAPS
 0    active  cephfs-ec.example.ooymyq  Reqs:    0 /s  8231   8233    891    921
       POOL           TYPE     USED  AVAIL
cephfs-metadata-ec  metadata   787M  8274G
  cephfs-data-ec      data    2360G  12.1T
 cephfs-data-ec01     data       0   12.1T

       STANDBY MDS
cephfs-ec.example.irsrql
cephfs-ec.example.cauuaj
```

7. Set the file layout on a new directory.

```
mkdir PATH_TO_DIRECTORY
setfattr -n ceph.dir.layout.pool -v DATA_POOL_NAME PATH_TO_DIRECTORY
```

In the following example, all new files that are created in the `/mnt/cephfs/newdir` directory inherit the directory layout and places the data in the newly added erasure-coded pool.

```
[root@mon ~]# mkdir /mnt/cephfs/newdir
[root@mon ~]# setfattr -n ceph.dir.layout.pool -v cephfs-data-ec01 /mnt/cephfs/newdir
```

Creating client users for a Ceph File System

Red Hat Ceph Storage uses `cephx` for authentication, which is enabled by default. To use `cephx` with the Ceph File System, create a user with the correct authorization capabilities on a Ceph Monitor node. Also, make its key available on the node where the Ceph File System will be mounted.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- The Ceph Metadata Server daemon (`ceph-mds`) installed and configured.
- Root-level access to a Ceph Monitor node.
- Root-level access to a Ceph client node.

About this task

For more information, see Ceph user management.

Procedure

1. Log in to the `cephadm` shell on the monitor node.

   ```
   cephadm shell
   ```

2. Create a client user on a Ceph Monitor node.

   ```
   ceph fs authorize FILE_SYSTEM_NAME client.CLIENT_NAME /DIRECTORY CAPABILITY [/
   DIRECTORY CAPABILITY] PERMISSIONS ...
   ```

   Example for restricting the client to only writing in the `temp` directory of filesystem `cephfs_a`

   ```
   [ceph: root@host01 ~]# ceph fs authorize cephfs_a client.1 / r /temp rw

   client.1
     key = AQBSdFhcGZFUDRAAcKhG9Cl2HPiDMMRv4DC43A==
   ```

   Example for completely restricting the client to the `temp` directory, by removing the root (/) directory

   ```
   [ceph: root@host01 ~]# ceph fs authorize cephfs_a client.1 /temp rw
   ```

   Note: Supplying `all` or asterisk (*) as the file system name grants access to every file system. Typically, it is necessary to put the asterisk in quotations to protect it from the shell.

3. Verify the created key.

   ```
   ceph auth get client.ID
   ```

   For example,

```
[ceph: root@host01 ~]# ceph auth get client.1

client.1
  key = AQBSdFhcGZFUDRAAcKhG9Cl2HPiDMMRv4DC43A==
  caps mds = "allow r, allow rw path=/temp"
  caps mon = "allow r"
  caps osd = "allow rw tag cephfs data=cephfs_a"
```

4. Copy the keyring to the client.
   a. On the Ceph Monitor node, export the keyring to a file.

      ```
      ceph auth get client.ID -o ceph.client.ID.keyring
      ```

      For example,

      ```
      [ceph: root@host01 ~]# ceph auth get client.1 -o ceph.client.1.keyring
      exported keyring for client.1
      ```

   b. Copy the client keyring from the Ceph Monitor node to the `/etc/ceph/` directory on the client node.
      Replace *CLIENT_NODE_NAME* with the Ceph client node name or IP.
      For example,

      ```
      [ceph: root@host01 ~]# scp /ceph.client.1.keyring root@client01:/etc/ceph/
      ceph.client.1.keyring
      ```

5. From the client node, set the appropriate permissions for the keyring file.

   ```
   chmod 644 ceph.client.ID.keyring
   ```

   For example,

   ```
   [root@client01 ~]# chmod 644 /etc/ceph/ceph.client.1.keyring
   ```

CephFS client capabilities

Learn how capability strings control authentication and authorization for Ceph File System clients across Monitor, OSD, and Metadata Server daemons.

In Red Hat Ceph Storage, the `cephx` authentication system uses capability strings (*caps*) to restrict client access to storage daemons. When mounting and using a Ceph File System (CephFS), a client requires distinct capabilities for Ceph Monitors (MON), Ceph OSDs, and Ceph Metadata Servers (MDS).

Capability requirements by daemon type

Each Ceph daemon type enforces permissions differently based on its role in the cluster:

Monitor (MON) capabilities

Grant access to cluster maps and authentication services. CephFS clients require read access (`allow r`) to MON daemons to retrieve the file system map and cluster status.

OSD capabilities

Control direct read, write, and execute permissions on RADOS object pools containing CephFS data. CephFS clients require read-write access to the data pools (for example, `allow rw pool=DATA_POOL_NAME`).

Metadata Server (MDS) capabilities

Control access to directory hierarchies, file metadata, and namespace paths within CephFS.

MDS capability syntax and constraints

MDS capabilities do not support a write-only (`allow w`) flag. All file system operations require metadata traversal and directory inspection before modification, so write access always implies read access.

The syntax for an MDS capability string is:

```
allow [r|rw] [path=/PATH] [root_squash] [fs_name=FS_NAME] [uid=UID [gid=GID]]
```

Supported MDS capability parameters:

allow r

Grants read-only access to file system metadata. The client can traverse directories and inspect files but cannot create, modify, or delete metadata.

allow rw

Grants read and write access to file system metadata across the entire file system hierarchy.

path=*/PATH*

Restricts client metadata operations to the specified directory subtree (for example, `path=/volumes/group1`). The client cannot access directories outside this path.

root_squash

Maps client requests from `uid=0` (root) to the anonymous user `nobody` (`uid=65534`), preventing unauthorized administrative actions over the mount point.

fs_name=*FS_NAME*

Restricts access to the named file system in clusters that host multiple CephFS instances.

uid=*UID* [gid=*GID*]

Restricts metadata access to the specified user identifier and optional group identifier.

Security considerations for path restrictions and OSD access

Restricting access to an MDS path (`path=/PATH`) secures metadata operations, but file data blocks reside directly in OSD pools. A client with broad OSD capabilities (`allow rw`) could bypass path-level restrictions by reading or modifying raw RADOS objects directly.

To enforce complete isolation in multi-tenant environments:

- Isolate file system subtrees into dedicated RADOS data pools or pool namespaces.
- Restrict OSD capabilities to specific pool names or namespaces (`allow rw pool=DATA_POOL_NAME namespace=NAMESPACE`).
- Use the **ceph fs authorize** command to generate combined MON, MDS, and OSD capabilities automatically with proper path and pool scoping.

Authorizing CephFS clients

Create client credentials and assign Monitor, OSD, and Metadata Server capabilities for Ceph File System access.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root or administrative access to the IBM Storage Ceph cluster.
- An active Ceph File System.

About this task

Ceph File System (CephFS) clients require authentication keyrings containing appropriate capabilities for the Ceph Monitor (MON), OSD, and Ceph Metadata Server (MDS) daemons. You can configure capabilities automatically using the **ceph fs authorize** command or manually using the **ceph auth get-or-create** command.

Procedure

1. Log in to the IBM Storage Ceph admin node.
2. Authorize the client and generate a keyring by using the **ceph fs authorize** command.

   ```
   ceph fs authorize FS_NAME client.CLIENT_ID /DIRECTORY_PATH PERMISSIONS
   ```

   Where:

   - *FS_NAME* is the name of the CephFS file system.
   - *CLIENT_ID* is the client authentication name (for example, `user1`).

- *DIRECTORY_PATH* is the target directory path. Use `/` to grant access to the entire file system.
- *PERMISSIONS* specifies the access mode: `r` for read-only or `rw` for read-write.

For example, to grant `client.user1` read-write permissions to `/volumes/group1` and read-only permissions to `/public` on file system `cephfs`:

```
[root@admin ~]# ceph fs authorize cephfs client.user1 /volumes/group1 rw /public r
[client.user1]
    key = AQB...==
    caps mds = "allow rw path=/volumes/group1, allow r path=/public"
    caps mon = "allow r fs_name=cephfs"
    caps osd = "allow rw tag cephfs data=cephfs"
```

Because the **ceph fs authorize** output contains the keyring, you can write it directly to a file by redirecting stdout:

```
ceph fs authorize FS_NAME client.CLIENT_ID /DIRECTORY_PATH PERMISSIONS > /etc/ceph/
ceph.client.CLIENT_ID.keyring
```

3. Optional: Update the capabilities for an existing client by running the **ceph fs authorize** command again with the new path and permissions.

```
ceph fs authorize FS_NAME client.CLIENT_ID /DIRECTORY_PATH PERMISSIONS
```

For example, to grant `client.user1` read-write access to `/public2` on file system `cephfs`:

```
[root@admin ~]# ceph fs authorize cephfs client.user1 /public2 rw
```

> **Note:** If you previously saved the keyring to a file, update the file with the output of this command so that it reflects the latest client capabilities.

Result

> **Note:** You can also use the **ceph auth get-or-create** command to create CephFS clients and update their capabilities. However, this approach requires you to construct the MDS, MON, and OSD capabilities fully and precisely by hand. The **ceph fs authorize** command is simpler because it requires only the MDS path and permission; it derives the MON and OSD capabilities automatically. The following example shows the equivalent manual command:
>
> ```
> ceph auth get-or-create client.CLIENT_ID mon 'allow r' mds 'allow rw path=/
> DIRECTORY_PATH root_squash' osd 'allow rw pool=DATA_POOL_NAME'
> ```

Mounting the Ceph File System as a kernel client

The Ceph File System (CephFS) can be mounted as a kernel client, either manually or automatically on system start.

Before you begin

Before mounting a Ceph File System as a kernel client, be sure that you have:

- Root-level access to a Linux-based client node.
- Root-level access to a Ceph Monitor node.
- An existing Ceph File System.

About this task

Important: Clients running on other Linux distributions, aside from Red Hat Enterprise Linux, are permitted but not supported. If issues are found in the CephFS Metadata Server or other parts of the storage cluster when using these clients, Red Hat will address them. If the cause is found to be on the client side, the kernel vendor of the Linux distribution must address the issue.

- See the `mount(8)` manual page.
- For more information about creating a Ceph user, see "Ceph user management" on page 445.
- For more information about creating Ceph File Systems, see "Creating Ceph File Systems" on page 1167.

Procedure

Configure the client node to use the Ceph storage cluster.

1. Enable the Red Hat Ceph Storage 9 Tools repository.

```
subscription-manager repos --enable=rhceph-9-tools-for-rhel-9-x86_64-rpms
```

Repeat on all client nodes of the Red Hat Ceph Storage.

2. Install the `ceph-common` package.

```
dnf install ceph-common
```

3. Copy the Ceph client keyring from the Ceph Monitor node to the client node.

Note: The client user should already be created for the keyring file to exist. For information about creating client users for Ceph File Systems, see "Creating client users for a Ceph File System" on page 1171.

```
scp /etc/ceph/ceph.client.admin.keyring root@CLIENT_NODE_NAME:/etc/ceph/
ceph.client.admin.keyring
```

Replace *CLIENT_NODE_NAME* with the Ceph client host name or IP address.
For example,

```
[ceph: root@host01 ~]# scp /etc/ceph/ceph.client.admin.keyring
root@ceph.client.admin.keyring:/etc/ceph/ceph.client.admin.keyring
```

4. Copy the Ceph configuration file from a Ceph Monitor node to the client node.

```
scp /etc/ceph/ceph.conf root@CLIENT_NODE_NAME:/etc/ceph/ceph.conf
```

Replace *CLIENT_NODE_NAME* with the Ceph client host name or IP address.
For example,

```
[ceph: root@host01 ~]# scp /etc/ceph/ceph.conf root@client01:/etc/ceph/ceph.conf
```

5. Configure the client node to use the Ceph storage cluster.

```
chmod 644 /etc/ceph/ceph.conf
```

6. Choose either `automatically` or `manually`.
   - For automatic mounting, continue to "Mounting automatically" on page 1176.
   - For manual mounting, continue to "Mounting manually" on page 1176 .

Mounting automatically

Procedure

1. On the client host, create a new directory for mounting the Ceph File System.

```
mkdir -p MOUNT_POINT
```

For example:

```
[root@client01 ~]# mkdir -p /mnt/cephfs
```

2. Edit the `/etc/fstab` file.

```
#DEVICE                 PATH           TYPE        OPTIONS
MON_0_HOST:PORT,               MOUNT_POINT        ceph      name=CLIENT_ID,
MON_1_HOST:PORT,                                             ceph.client_mountpoint=/
VOL/SUB_VOL_GROUP/SUB_VOL/UID_SUB_VOL, fs=FILE_SYSTEM_NAME, [ADDITIONAL_OPTIONS]
MON_2_HOST:PORT:/VOL/SUB_VOL/UID_SUB_VOL
```

In the `/etc/fstab` file,

- The first column (`#DEVICE`) sets the Ceph Monitor host names and the port number.
- The second column (`PATH`) set the mount point.
- The third column (`TYPE`) sets the file system type. In this case, `ceph`, for CephFS.
- The fourth column (`OPTIONS`) sets the various options. These include the user name and the secret file that use the `name` and `secretfile` options. You can also set specific volumes, subvolume groups, and subvolumes that use the `ceph.client_mountpoint` option.
  Set the `_netdev` option to ensure that the file system is mounted after the networking subsystem starts to prevent hanging and networking issues. If you do not need access time information, then setting the `noatime` option can increase performance.
- Set the fifth and sixth columns (`DUMP` and `FSCK`) to `0`.

For example,

```
#DEVICE         PATH                    TYPE    OPTIONS         DUMP  FSCK
mon1:6789,      /mnt/cephfs             ceph    name=1,            0     0
mon2:6789,                                      ceph.client_mountpoint=/my_vol/
my_sub_vol_group/my_sub_vol/0,
mon3:6789:/                                     fs=cephfs01,
                                                _netdev,noatime
```

The Ceph File System is mounted on the next system start.

Note:

- `mount.ceph` can read keyring files directly. As such, a secret file is no longer necessary. Specify the client ID with `name=`*`CLIENT_ID`*, and `mount.ceph` will find the correct keyring file.
- You can replace the Monitor host names with the string `:/` and `mount.ceph` will read the Ceph configuration file to determine which Monitors to connect to.

Mounting manually

Procedure

1. Create a mount directory on the client node.

```
mkdir -p MOUNT_POINT
```

For example,

```
[root@client01 ~]# mkdir -p /mnt/cephfs
```

2. Mount the Ceph File System.
   To specify multiple Ceph Monitor addresses, in the **mount** command:
   a. Separate them with commas.
   b. Specify the mount point.
   c. Set the client name.

> **Note:** `mount.ceph` can read keyring files directly, therefore a secret file is not necessary. It is enough to specify the client ID with `name=CLIENT_ID`, and `mount.ceph` finds the correct keyring file.

```
mount -t ceph MONITOR-1_NAME:6789,MONITOR-2_NAME:6789,MONITOR-3_NAME:6789:/
MOUNT_POINT -o name=CLIENT_ID,fs=FILE_SYSTEM_NAME
```

For example:

```
[root@client01 ~]# mount -t ceph mon1:6789,mon2:6789,mon3:6789:/ /mnt/cephfs -o
name=1,fs=cephfs01
```

> **Note:**
> - Configure a DNS server so that a single host name resolves to multiple IP addresses. That single host name can then be used with the **mount** command, instead of supplying a comma-separated list.
> - You can also replace the Monitor host names with the string `:/` and `mount.ceph` reads the Ceph configuration file to determine which Monitors to connect to.
> - Set the `nowsync` option to asynchronously run file creation and removal on the Red Hat Ceph Storage clusters. This improves the performance of some workloads by avoiding round-trip latency for these system calls without impacting consistency. The `nowsync` option requires kernel clients with Red Hat Enterprise Linux 9.2 or later.
>   For example,
>
> ```
> [root@client01 ~]# mount -t ceph mon1:6789,mon2:6789,mon3:6789:/ /mnt/
> cephfs -o nowsync,name=1,fs=cephfs01
> ```

3. Verify that the file system is successfully mounted.

```
stat -f MOUNT_POINT
```

For example,

```
[root@client01 ~]# stat -f /mnt/cephfs
```

Related information

Rotating CephX keys

Mounting the Ceph File System as a FUSE client

The Ceph File System (CephFS) can be mounted as a FUSE client, either manually or automatically on system start.

Before you begin

Before mounting a Ceph File System as a FUSE client, be sure that you have:

- Root-level access to a Linux-based client node.
- Root-level access to a Ceph Monitor node.
- An existing Ceph File System.

About this task

- For more information, see the `ceph-fuse(8)` manual page.
- For more information about creating a Ceph user, see "Ceph user management" on page 445.
- For more information about creating Ceph File Systems, see "Creating Ceph File Systems" on page 1167.

Procedure

Configure the client node to use the Ceph storage cluster.

1. Enable the Red Hat Ceph Storage Tools repository.

```
subscription-manager repos --enable=rhceph-9-tools-for-rhel-9-x86_64-rpms
```

   Repeat on client nodes of the Red Hat Ceph Storage storage cluster.

2. Install the `ceph-fuse` package.

```
dnf install ceph-fuse
```

3. Copy the Ceph client keyring from the Ceph Monitor node to the client node.

> **Note:** The client user must already be created.

```
scp /etc/ceph/ceph.client.admin.keyring root@CLIENT_NODE_NAME:/etc/ceph/
ceph.client.admin.keyring
```

   Replace *CLIENT_NODE_NAME* with the Ceph client host name or IP address.

   For example,

```
[ceph: root@host01 ~]# scp /etc/ceph/ceph.client.admin.keyring root@client01:/etc/
ceph/ceph.client.admin.keyring
```

4. Copy the Ceph configuration file from a Ceph Monitor node to the client node.

```
scp /etc/ceph/ceph.conf root@CLIENT_NODE_NAME:/etc/ceph/ceph.conf
```

   Replace *CLIENT_NODE_NAME* with the Ceph client host name or IP address.

   For example,

```
[ceph: root@host01 ~]# scp /etc/ceph/ceph.conf root@client01:/etc/ceph/ceph.conf
```

5. From the client node, set the appropriate permissions for the configuration file.

```
chmod 644 /etc/ceph/ceph.conf
```

6. Choose either `automatically` or `manually`.
   - For automatic mounting, continue to "Mounting automatically" on page 1178.
   - For manual mounting, continue to "Mounting manually" on page 1179.

Mounting automatically

Procedure

1. On the client node, create a directory for the mount point.

Note: If you used the `path` option with MDS capabilities, then the mount point must be within what is specified by the `path`.

```
mkdir PATH_TO_MOUNT_POINT
```

For example,

```
[root@client01 ~]# mkdir /mnt/mycephfs
```

2. Edit the `/etc/fstab` file.

```
DEVICE                 PATH           TYPE          OPTIONS                   DUMP
 FSCK
HOST_NAME:PORT,     MOUNT_POINT fuse.ceph     ceph.id=CLIENT_ID,        0     0
HOST_NAME:PORT,                                  ceph.client_mountpoint=/VOL/
SUB_VOL_GROUP/SUB_VOL/UID_SUB_VOL,
HOST_NAME:PORT:/
ceph.client_fs=FILE_SYSTEM_NAME,ceph.name=USERNAME,ceph.keyring=/etc/ceph/
KEYRING_FILE,
                                                      [ADDITIONAL_OPTIONS]
```

In the `/etc/fstab` file,

- The first column (`#DEVICE`) sets the Ceph Monitor host names and the port number.
- The second column (`PATH`) set the mount point.
- The third column (`TYPE`) sets the file system type. In this case, `fuse.ceph`, for CephFS.
- The fourth column (`OPTIONS`) sets the various options. These include the user name and the keyring that use the `ceph.name` and `ceph.keyring` options. You can also set specific volumes, subvolume groups, and subvolumes that use the `ceph.client_mountpoint` option.
  To specify which Ceph File System to access, use the `ceph.client_fs` option. Set the `_netdev` option to ensure that the file system is mounted after the networking subsystem starts to prevent hanging and networking issues. If you do not need access time information, then setting the `noatime` option can increase performance. If you want to automatically reconnect after an eviction, then set the `client_reconnect_stale=true` option.
- Set the fifth and sixth columns (`DUMP` and `FSCK`) to `0`.

For example,

```
# DEVICE         PATH              TYPE         OPTIONS         DUMP  FSCK
 mon1:6789,      /mnt/mycephfs       fuse.ceph    ceph.id=1,         0     0
 mon2:6789,                                       ceph.client_mountpoint=/my_vol/
my_sub_vol_group/my_sub_vol/0,
 mon3:6789:/
 ceph.client_fs=cephfs01,ceph.name=client.1,ceph.keyring=/etc/ceph/client1.keyring,
                                              _netdev,defaults
```

The Ceph File System is mounted on the next system start.

Mounting manually

Procedure

1. On the client node, create a directory for the mount point.

Note: If you used the `path` option with MDS capabilities, then the mount point must be within what is specified by the `path`.

```
mkdir PATH_TO_MOUNT_POINT
```

For example,

```
[root@client01 ~]# mkdir /mnt/mycephfs
```

2. Use the `ceph-fuse` utility to mount the Ceph File System.

```
ceph-fuse -n client.CLIENT_ID --client_fs FILE_SYSTEM_NAME MOUNT_POINT
```

For example,

```
[root@client01 ~]# ceph-fuse -n client.1 --client_fs cephfs01 /mnt/mycephfs
```

Note:

- If you do not use the default name and location of the user keyring (`/etc/ceph/ceph.client.CLIENT_ID.keyring`) then use the **`--keyring`** option to specify the path to the user keyring. For example,

```
[root@client01 ~]# ceph-fuse -n client.1 --keyring=/etc/ceph/
client.1.keyring /mnt/mycephfs
```

- Use the **`-r`** option to instruct the client to treat that path as its root.

```
ceph-fuse -n client.CLIENT_ID MOUNT_POINT -r PATH
```

For example,

```
[root@client01 ~]# ceph-fuse -n client.1 /mnt/cephfs -r /home/cephfs
```

- If you want to automatically reconnect an evicted Ceph client, then add the **`--client_reconnect_stale=true`** option.
For example,

```
[root@client01 ~]# ceph-fuse -n client.1 /mnt/cephfs --
client_reconnect_stale=true
```

3. Verify that the file system is successfully mounted.

```
stat -f MOUNT_POINT
```

For example,

```
[root@client01 ~]# stat -f /mnt/cephfs
```

Related information

Rotating CephX keys

Managing CephFS volumes

The `volumes` module for the Ceph Manager daemon (`ceph-mgr`) implements the ability to export Ceph File Systems (CephFS).

The Ceph Manager volumes module implements the following file system export abstractions:

- CephFS volumes
- CephFS subvolumes
- CephFS subvolume groups

For more information, see “Managing Ceph users” on page 447.

CephFS volumes

You can create, list, and remove Ceph File System (CephFS) volumes. CephFS volumes are an abstraction for Ceph File Systems.

Before working with CephFS volume be sure that you have:

- A working Red Hat Ceph Storage cluster with Ceph File System deployed.
- A minimum read access on the Ceph Monitor.
- Read and write capability on the Ceph Manager nodes.

Creating a CephFS volume

Ceph Orchestrator is a module for Ceph Manager that creates a Metadata Server (MDS) for the Ceph File System (CephFS). This section describes how to create a CephFS volume.

About this task

Creating a CephFS volume creates the Ceph File system along with the data and metadata pools.

Procedure

- Create a CephFS volume on the monitor node.

  ```
  ceph fs volume create VOLUME_NAME
  ```

  For example,

  ```
  [ceph: root@host01 /]# ceph fs volume create cephfs
  ```

Listing CephFS volumes

Learn how to list Ceph File System volumes.

Before you begin

Be sure that you have a CephFS volume.

Procedure

- List the CephFS volume.

  ```
  [ceph: root@host01 /]# ceph fs volume ls
  ```

Viewing information about a CephFS volume

List basic details about a Ceph File System (CephFS) volume, such as attributes of data and metadata pools of the CephFS volume, pending subvolumes deletion count, and the like.

Before you begin

Be sure that you have a CephFS volume.

Procedure

- View information about a CephFS volume.

  ```
  ceph fs volume info VOLUME_NAME
  ```

  For example:

```
[ceph: root@host01 /]# ceph fs volume info cephfs
{
    "mon_addrs": [
        "192.168.1.7:40977",
    ],
    "pending_subvolume_deletions": 0,
    "pools": {
        "data": [
            {
                "avail": 106288709632,
                "name": "cephfs.cephfs.data",
                "used": 4096
            }
        ],
        "metadata": [
            {
                "avail": 106288709632,
                "name": "cephfs.cephfs.meta",
                "used": 155648
            }
        ]
    },
    "used_size": 0
}
```

The output of the **ceph fs volume info** command includes:

mon_addrs

List of monitor addresses.

pending_subvolume_deletions

Number of subvolumes pending deletion.

pools

Attributes of data and metadata pools.

avail

The amount of free space available in bytes.

name

Name of the pool.

used

The amount of storage consumed in bytes.

used_size

Current used size of the CephFS volume in bytes.

Removing a CephFS volume

Ceph Orchestrator is a module for Ceph Manager that removes the Metadata Server (MDS) for the Ceph File System (CephFS). Learn how to remove the Ceph File System (CephFS) volume.

Before you begin

Be sure that you have a CephFS volume.

Procedure

1. If the `mon_allow_pool_delete` option is not set to `true`, then set it to `true` before removing the CephFS volume.

   ```
   [ceph: root@host01 /]# ceph config set mon mon_allow_pool_delete true
   ```

2. Remove the CephFS volume.

   ```
   ceph fs volume rm VOLUME_NAME [--yes-i-really-mean-it]
   ```

   For example:

```
[ceph: root@host01 /]# ceph fs volume rm cephfs --yes-i-really-mean-it
```

CephFS subvolumes

You can create, list, fetch absolute path, fetch metadata, and remove Ceph File System (CephFS) subvolumes. You can also create, list, and remove snapshots of these subvolumes. CephFS subvolumes are an abstraction for independent Ceph File Systems directory trees.

Before working with CephFS subvolumes be sure that you have:

- A working Red Hat Ceph Storage cluster with CephFS deployed.
- A minimum read access on the Ceph Monitor.
- Read and write capability on the Ceph Manager nodes.

Creating a file system subvolume

Learn how to create a Ceph File System (CephFS) subvolume.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A volume.
  For more information, see Creating a CephFS volume.
- A subvolume group.
  For more information, see Creating a file system subvolume group.

About this task

When creating a subvolume, you can specify its subvolume group, data pool layout, uid, gid, file mode in octal numerals, and size in bytes. The subvolume can be created in a separate RADOS namespace by specifying the `--namespace-isolated` option. By default, a subvolume is created within the default subvolume group, and with an octal file mode 755, uid of its subvolume group, gid of its subvolume group, data pool layout of its parent directory, and no size limit.

Note:

- you can specify its data pool layout, UID, GID, and file mode in octal numerals. By default, the subvolume group is created with an octal file mode ‘755’, uid ‘0’, gid ‘0’, and data pool layout of its parent directory.
- To set quotas while creating a subvolume group, see Setting and managing quotas on a file system subvolume group.

You can also specify an unicode normalization form by using the `normalization` option. This will be used to internally mangle file names so that unicode characters that can be represented by different unicode code point sequences are all mapped to the representation, which means that they will all access the same file. However, users will continue to see the same name that they used when the file was created. The valid values for the unicode normalization form are:

nfd

canonical decomposition (default)

nfc

canonical decomposition, followed by canonical composition

nfkd

compatibility decomposition

nfkc

compatibility decomposition, followed by canonical composition

To learn more about unicode normalization forms see https://unicode.org/reports/tr15

Procedure

- Create a CephFS subvolume.

> **Note:** The `--group_name` parameter is optional. If the subvolume needs to be created in a subvolume group, then `--group_name` needs to be passed in the command.

> **Note:**
>
> - Subvolume names containing whitespace must be enclosed in quotes when using the CLI. If whitespace is not quoted, the command-line parser interprets the name as multiple arguments and may lead to unintended behavior or incorrect subvolume creation.
> - To correctly specify a subvolume name with spaces, use quotes. For example,
>
>   ```
>   ceph fs subvolume create cephfs "sv 1" --group_name smb
>   ```

```
ceph fs subvolume create VOLUME_NAME SUBVOLUME_NAME[--size SIZE_IN_BYTES--
group_name SUBVOLUME_GROUP_NAME--pool_layout DATA_POOL_NAME --uid UID --gid GID--
mode OCTAL_MODE] [--namespace-isolated][--normalization <form>] [--casesensitive bool]
```

For example:

```
[root@mon ~]# ceph fs subvolume create cephfs01 sub0 --group_name subgroup0 --
namespace-isolated
```

The command succeeds even if the subvolume exists.

Listing a file system subvolume

Learn how to list a Ceph File System (CephFS) subvolume.

Before you begin

Be sure that you have a CephFS subvolume.

Procedure

- List the CephFS subvolume.

> **Note:** The `--group_name` parameter is optional. If the subvolume needs to be listed from within a subvolume group, then `--group_name` needs to be passed in the command.

```
ceph fs subvolume ls VOLUME_NAME[--group_name SUBVOLUME_GROUP_NAME]
```

For example,

```
[root@mon ~]# ceph fs subvolume ls cephfs01 --group_name subgroup0
[

{

"name": "sub0"

}

]
```

Resizing a file system subvolume

Learn how to resize a Ceph File System (CephFS) subvolume.

Before you begin

Be sure that you have a CephFS subvolume.

About this task

> **Note:** The **`ceph fs subvolume resize`** command resizes the subvolume quota by using the size that is specified by `new_size`. The `--no_shrink` flag prevents the subvolume from shrinking below the currently used size of the subvolume. The subvolume can be resized to an infinite size by passing `inf` or `infinite` as the `new_size`.

Procedure

– Resize a CephFS subvolume.

> **Note:** The `--group_name` parameter is optional. If the subvolume needs to be resized in a subvolume group, then `--group_name` needs to be passed in the command.

```
ceph fs subvolume resize VOLUME_NAME SUBVOLUME_NAME NEW_SIZE [--
group_name SUBVOLUME_GROUP_NAME] [--no_shrink]
```

For example:

```
[root@mon ~]# ceph fs subvolume resize cephfs01 sub0 1024000000 --group_name subgroup0
--no_shrink
```

Fetching absolute path of a file system subvolume

Learn how to fetch the absolute path of a Ceph File System (CephFS) subvolume.

Before you begin

Be sure that you have a CephFS subvolume.

Procedure

– Fetch the absolute path of the CephFS subvolume.

> **Note:** The `--group_name` parameter is optional. If the subvolume needs to be fetched from within a subvolume group, then `--group_name` needs to be passed in the command.

```
ceph fs subvolume getpath VOLUME_NAME SUBVOLUME_NAME [--
group_name SUBVOLUME_GROUP_NAME]
```

For example,

```
[root@mon ~]# ceph fs subvolume getpath cephfs01 sub0 --group_name subgroup0

/volumes/subgroup0/sub0/81992fd4-9765-420e-85f9-2545d31ff136
```

Fetching metadata of a file system subvolume

Learn how to fetch metadata of a Ceph File System (CephFS) subvolume.

Before you begin

Be sure that you have a CephFS subvolume.

Procedure

– Fetch the metadata of a CephFS subvolume.

> **Note:** The `--group_name` parameter is optional. If the subvolume needs to be fetched from within a subvolume group, then `--group_name` needs to be passed in the command.

```
ceph fs subvolume info VOLUME_NAME SUBVOLUME_NAME [--group_name SUBVOLUME_GROUP_NAME]
```

For example:

```
[root@mon ~]# ceph fs subvolume info cephfs01 sub0 --group_name subgroup0
```

Example output:

```
# ceph fs subvolume info cephfs01 sub0
{
    "atime": "2023-07-14 08:52:46",
    "bytes_pcent": "0.00",
    "bytes_quota": 1024000000,
    "bytes_used": 0,
    "created_at": "2023-07-14 08:52:46",
    "ctime": "2023-07-14 08:53:54",
    "data_pool": "cephfs.cephfs01.data",
    "features": [
        "snapshot-clone",
        "snapshot-autoprotect",
        "snapshot-retention"
    ],
    "flavor": "2",
    "gid": 0,
    "mode": 16877,
    "mon_addrs": [
        "10.0.208.172:6789",
        "10.0.211.197:6789",
        "10.0.209.212:6789"
    ],
    "mtime": "2023-07-14 08:52:46",
    "path": "/volumes/_nogroup/sub0/834c5cbc-f5db-4481-80a3-aca92ff0e7f3",
    "pool_namespace": "",
    "state": "complete",
    "type": "subvolume",
    "uid": 0
}
```

The output format is JSON and contains the following fields:

- `atime`: access time of subvolume path in the format "YYYY-MM-DD HH:MM:SS".
- `bytes_pcent`: quota used in percentage if quota is set, else displays "undefined".
- `bytes_quota`: quota size in bytes if quota is set, else displays "infinite".
- `bytes_used`: current used size of the subvolume in bytes.
- `created_at`: time of creation of subvolume in the format "YYYY-MM-DD HH:MM:SS".
- `ctime`: change time of subvolume path in the format "YYYY-MM-DD HH:MM:SS".
- `data_pool`: data pool the subvolume belongs to.
- `features`: features supported by the subvolume, such as, "snapshot-clone", "snapshot-autoprotect", or "snapshot-retention".
- `flavor`: subvolume version, either `1` for version one or `2` for version two.
- `gid`: group ID of subvolume path.
- `mode`: mode of subvolume path.

- `mon_addrs`: list of monitor addresses.
- `mtime`: modification time of subvolume path in the format "YYYY-MM-DD HH:MM:SS".
- `path`: absolute path of a subvolume.
- `pool_namespace`: RADOS namespace of the subvolume.
- `state`: current state of the subvolume, such as, "complete" or "snapshot-retained".
- `type`: subvolume type indicating whether it is clone or subvolume.
- `uid`: user ID of subvolume path.

Creating snapshot of a file system subvolume

Learn how to create snapshots of a Ceph File System (CephFS) subvolume.

Before you begin

Be sure that you have a CephFS subvolume.
In addition to read (`r`) and write (`w`) capabilities, clients also require `s` flag on a directory path within the file system.

Verify that the `s` flag is set on the directory.

```
ceph auth get CLIENT_NAME
```

Output example:

```
CLIENT_NAME
    key = AQAz7EVWygILFRAAdIcuJ12opU/JKyfFmxhuaw==
    caps mds = allow rw, allow rws path=DIRECTORY_PATH
    caps mon = allow r
    caps osd = allow rw tag cephfs01 data=DIRECTORY_NAME
```

In the following example, `client.0` can create or delete snapshots in the `bar` directory of file system `cephfs_a`.

```
[root@mon ~]# ceph auth get client.0
[client.0]
    key = AQAz7EVWygILFRAAdIcuJ12opU/JKyfFmxhuaw==
    caps mds = "allow rw, allow rws path=/bar"
    caps mon = "allow r"
    caps osd = "allow rw tag cephfs data=cephfs_a"
```

Procedure

– Create a snapshot of the Ceph File System subvolume.

> **Note:** The `--group_name` parameter is optional. If the subvolume is present within a subvolume group, then `--group_name` needs to be passed in the command.

```
ceph fs subvolume snapshot create VOLUME_NAME SUBVOLUME_NAME SNAP_NAME [--
group_name GROUP_NAME]
```

For example:

```
[root@mon ~]# ceph fs subvolume snapshot create cephfs01 sub0 snap0 --group_name
subgroup0
```

Cloning subvolumes from snapshots

Create subvolumes by cloning subvolume snapshots, an asynchronous operation that copies data from a snapshot to a subvolume.

Before you begin

Be sure that you have a Ceph File System (CephFS) subvolume.
In addition to read (`r`) and write (`w`) capabilities, Ceph clients also require `s` flag on a directory path within the file system.

Verify that the `s` flag is set on the directory.

```
ceph auth get CLIENT_NAME
```

Output example:

```
CLIENT_NAME
    key = AQAz7EVWygILFRAAdIcuJ12opU/JKyfFmxhuaw==
    caps mds = allow rw, allow rws path=DIRECTORY_PATH
    caps mon = allow r
    caps osd = allow rw tag cephfs01 data=DIRECTORY_NAME
```

In the following example, `client.0` can create or delete snapshots in the `bar` directory of file system `cephfs_a`.

```
[root@mon ~]# ceph auth get client.0
[client.0]
    key = AQAz7EVWygILFRAAdIcuJ12opU/JKyfFmxhuaw==
    caps mds = "allow rw, allow rws path=/bar"
    caps mon = "allow r"
    caps osd = "allow rw tag cephfs data=cephfs_a"
```

About this task

For more information, see “Managing Ceph users” on page 447. To pause clone operations, for example during cluster recovery, see “Pausing asynchronous subvolume purge and clone operations” on page 1217.

> **Note:** Cloning is inefficient for very large data sets.

Procedure

1. Create a CephFS volume.
   This creates the CephFS file system, its data, and metadata pools.

   ```
   ceph fs volume create VOLUME_NAME
   ```

   For example,

   ```
   [root@mon ~]# ceph fs volume create cephfs01
   ```

2. Create a subvolume group.
   By default, the subvolume group is created with an octal file mode '755', and data pool layout of its parent directory.

   ```
   ceph fs subvolumegroup create VOLUME_NAME GROUP_NAME [--pool_layout DATA_POOL_NAME --
   uid UID --gid GID --mode OCTAL_MODE]
   ```

   For example,

   ```
   [root@mon ~]# ceph fs subvolumegroup create cephfs01 subgroup0
   ```

3. Create a subvolume.
   By default, a subvolume is created within the default subvolume group, and with an octal file mode ‘755’, uid of its subvolume group, gid of its subvolume group, data pool layout of its parent directory, and no size limit.

   > **Note:** The **`--group_name`** parameter is optional. If the subvolume needs to be created in a subvolume group, then **`--group_name`** needs to be passed in the command.

```
ceph fs subvolume create VOLUME_NAME SUBVOLUME_NAME [--size SIZE_IN_BYTES --group_name
SUBVOLUME_GROUP_NAME --pool_layout DATA_POOL_NAME --uid UID --gid GID --mode
OCTAL_MODE]
```

For example,

```
[root@mon ~]# ceph fs subvolume create cephfs01 sub0 --group_name subgroup0
```

4. Create a snapshot of a subvolume.

```
ceph fs subvolume snapshot create VOLUME_NAME SUBVOLUME_NAME SNAP_NAME [--group_name
SUBVOLUME_GROUP_NAME]
```

For example,

```
[root@mon ~]# ceph fs subvolume snapshot create cephfs01 sub0 snap0 --group_name
subgroup0
```

5. Initiate a clone operation.

> **Note:** By default, cloned subvolumes are created in the default group.

   a. If the source subvolume and the target clone are in the default group, run the following command.

```
ceph fs subvolume snapshot clone VOLUME_NAME SUBVOLUME_NAME SNAP_NAME
TARGET_CLONE_NAME
```

   For example,

```
[root@mon ~]# ceph fs subvolume snapshot clone cephfs01 sub0 snap0 clone0
```

   b. If the source subvolume is in the non-default group, then specify the source subvolume group in the following command.

```
ceph fs subvolume snapshot clone VOLUME_NAME SUBVOLUME_NAME SNAP_NAME
TARGET_CLONE_NAME --group_name SUBVOLUME_GROUP_NAME
```

   For example,

```
[root@mon ~]# ceph fs subvolume snapshot clone cephfs01 sub0 snap0 clone0 --
group_name subgroup0
```

   c. If the target clone is to a non-default group, then specify the target group in the following command.

```
ceph fs subvolume snapshot clone VOLUME_NAME SUBVOLUME_NAME SNAP_NAME
TARGET_CLONE_NAME --target_group_name SUBVOLUME_GROUP_NAME
```

   For example,

```
[root@mon ~]# ceph fs subvolume snapshot clone cephfs01 sub0 snap0 clone0 --
target_group_name subgroup1
```

6. Check the status of the clone operation.

```
ceph fs clone status VOLUME_NAME CLONE_NAME [--group_name TARGET_GROUP_NAME]
```

For example,

```
[root@mon ~]# ceph fs clone status cephfs01 clone0 --group_name subgroup1

{
  "status": {
    "state": "complete"
  }
}
```

Listing snapshots of a file system subvolume

Learn how to list the snapshots of a Ceph File system (CephFS) subvolume.

Before you begin

Be sure that you have a CephFS subvolume and snapshots of the subvolume.

Procedure

– List the snapshots of a CephFS subvolume.

> **Note:** The `--group_name` parameter is optional. If the subvolume is present within the subvolume group, then `--group_name` needs to be passed in the command.

```
ceph fs subvolume snapshot ls VOLUME_NAME SUBVOLUME_NAME [--
group_name SUBVOLUME_GROUP_NAME]
```

For example:

```
[root@mon ~]# ceph fs subvolume snapshot ls cephfs01 sub0 --group_name subgroup0

[
{
"name": "snap0"
}
]
```

Fetching metadata of the snapshots of a file system subvolume

Learn how to list the snapshots of a Ceph File system (CephFS) subvolume.

Before you begin

Be sure that you have a CephFS subvolume and snapshots of the subvolume.

Procedure

– Fetch the metadata of the snapshots of a CephFS subvolume.

> **Note:** The `--group_name` parameter is optional. If the subvolume is present within a subvolume group, then `--group_name` needs to be passed in the command.

```
ceph fs subvolume snapshot info VOLUME_NAME SUBVOLUME_NAME SNAP_NAME [--
group_name SUBVOLUME_GROUP_NAME]
```

For example:

```
[root@mon ~]# ceph fs subvolume snapshot info cephfs01 sub0 snap0 --group_name
subgroup0
```

Example output:

```
{
    "created_at": "2022-05-09 06:18:47.330682",
    "data_pool": "cephfs_data",
    "has_pending_clones": "no"
}
```

The output format is JSON and contains the following fields:

- `created_at`: time of creation of snapshot in the format `"YYYY-MM-DD HH:MM:SS:ffffff"`.
- `data_pool`: data pool the snapshot belongs to.
- `has_pending_clones`: `"yes"` if snapshot clone is in progress otherwise `"no"`.
- `size`: snapshot size in bytes.

Removing a file system subvolume

Learn how to remove the Ceph File System (CephFS) subvolume.

Before you begin

Be sure that you have a CephFS subvolume.

About this task

> **Note:** The **`ceph fs subvolume rm`** command removes the subvolume and its contents in two steps. First, it moves the subvolume to a trash folder, and then asynchronously purges its contents.

A subvolume can be removed while retaining existing snapshots of the subvolume that use the `--retain-snapshots` option. If snapshots are retained, the subvolume is considered empty for all operations not involving the retained snapshots. Retained snapshots can be used as a clone source to re-create the subvolume, or cloned to a newer subvolume.

Procedure

– Remove a CephFS subvolume.

> **Note:** The `--group_name` parameter is optional. If the subvolume needs to be removed from within in a subvolume group, then `--group_name` needs to be passed in the command.

```
ceph fs subvolume rm VOLUME_NAME SUBVOLUME_NAME [--group_name SUBVOLUME_GROUP_NAME] [--
force] [--retain-snapshots]
```

For example:

```
[root@mon ~]# ceph fs subvolume rm cephfs01 sub0 --group_name subgroup0 --retain-
snapshots
```

Recreating a subvolume from a retained snapshot

To recreate a subvolume from a retained snapshot, use

```
ceph fs subvolume snapshot
clone VOLUME_NAME DELETED_SUBVOLUME RETAINED_SNAPSHOT NEW_SUBVOLUME --
group_name SUBVOLUME_GROUP_NAME --target_group_name SUBVOLUME_TARGET_GROUP_NAME
```

NEW_SUBVOLUME can either be the same subvolume, which was deleted earlier or clone it to a new subvolume.

For example:

```
[root@mon ~]# ceph fs subvolume snapshot clone cephfs01 sub0 snap0 sub1 --group_name
subgroup0 --target_group_name subgroup0
```

Verifying the state of a file subvolume

Verify the state of a file subvolume, by using the **`subvolume info`** command.
For example,

```
[root@mon ~]# ceph fs subvolume info cephfs01 sub1 subgroup0

{
"atime": "2025-01-08 04:21:37",
"bytes_pcent": "undefined",
"bytes_quota": "infinite",
"bytes_used": 0,
"created_at": "2025-01-08 04:42:46",
"ctime": "2025-01-08 04:42:47",
"data_pool": "cephfs.cephfs01.data",
"earmark": "",
"features": [
"snapshot-clone",
"snapshot-autoprotect",
"snapshot-retention"
],
"flavor": 2,
"gid": 0,
"mode": 16877,
"mon_addrs": [
"10.0.64.12:6789",
"10.0.67.102:6789",
"10.0.66.135:6789"
],
"mtime": "2025-01-08 04:21:37",
"path": "/volumes/subgroup0/sub1/6bd615e9-8985-4c25-8dd9-c80d9ebb8033",
"pool_namespace": "fsvolumens_sub0",
"state": "complete",
"type": "clone",
"uid": 0
}
```

Removing snapshot of a file system subvolume

Learn how to remove snapshots of a Ceph File System (CephFS) subvolume group.

Before you begin

Be sure that you have a CephFS subvolume and a snapshot of the subvolume group.

About this task

Using the `--force` flag allows the command to succeed that would otherwise fail if the snapshot did not exist.

Procedure

– Remove the snapshot of the CephFS subvolume.

> **Note:** The `--group_name` parameter is optional. If the subvolume is present within a subvolume group, then `--group_name` needs to be passed in the command.

```
ceph fs subvolume snapshot rm VOLUME_NAME SUBVOLUME_NAME SNAP_NAME [--
group_name GROUP_NAME --force]
```

For example:

```
[root@mon ~]# ceph fs subvolume snapshot rm cephfs01 sub0 snap0 --group_name subgroup0
--force

[]
```

Managing Character Mapping for subvolume

Learn how to manage character mapping in CephFS.

The CephFS subvolume and subvolumegroup interfaces provide a porcelain API to manage character mapping `ceph.dir.charmap` settings. These settings support name normalization and case sensitivity options.

Configuring Character Mapping

Configure character mapping for subvolume.

To configure character mapping for subvolume:

```
ceph fs subvolume charmap set vol_name subvol group_name=name  setting value
```

Reading Character Mapping

Read character mapping for subvolume.

To read the character mapping for subvolume:

```
ceph fs subvolume charmap get vol_name subvol group_name=name setting
```

For example,

```
ceph fs subvolume charmap get vol subvol group_name=csi casesensitive
```

To read the full character mapping for subvolume:

```
ceph fs subvolume charmap get vol_name subvol group_name=name
```

For example,

```
ceph fs subvolumegroup charmap get vol csi
```

Outputs:

```
{"casesensitive":false,"normalization":"nfd","encoding":"utf8"}
```

Removing Character Mapping

Remove character mapping for subvolume.

> **Note:** A charmap can only be removed when a subvolume is empty.

To remove character mapping for subvolume:

```
ceph fs subvolume charmap rm vol_name subvol group_name=name
```

CephFS subvolume groups

You can create, list, fetch absolute path, and remove Ceph File System (CephFS) subvolume groups. subvolume groups are abstractions at a directory level that effects policies. For example, file layouts, across a set of subvolumes.

> **Note:** The subvolume group snapshot feature is not supported. You can only list and remove the existing snapshots of these subvolume groups.

Before working with Ceph File System subvolume groups be sure that you have:

- A working Red Hat Ceph Storage cluster with CephFS deployed.
- A minimum read access on the Ceph Monitor.
- Read and write capability on the Ceph Manager nodes.

Creating a file system subvolume group

Learn how to create a Ceph File System (CephFS) subvolume group.

About this task

Note:

- When creating a subvolume group, you can specify its data pool layout, UID, GID, and file mode in octal numerals. By default, the subvolume group is created with an octal file mode '755', uid '0', gid '0', and data pool layout of its parent directory.
- To set quotas while creating a subvolume group, see "Setting and managing quotas on a file system subvolume group" on page 1194.

Procedure

- Create a CephFS subvolume group.

```
ceph fs subvolumegroup create VOLUME_NAME GROUP_NAME [--pool_layout DATA_POOL_NAME --
uid UID --gid GID --mode OCTAL_MODE]
```

For example,

```
[ceph: root@host01 /]# ceph fs subvolumegroup create cephfs01 subgroup0
```

The command succeeds even if the subvolume group already exists.

Setting and managing quotas on a file system subvolume group

Learn how to set and manage quotas on a Ceph File System (CephFS) subvolume group.

Procedure

1. Set quotas while creating a subvolume group by providing size in bytes.

```
ceph fs subvolumegroup create VOLUME_NAME GROUP_NAME [--size SIZE_IN_BYTES] [--
pool_layout DATA_POOL_NAME] [--uid UID] [--gid GID] [--mode OCTAL_MODE]
```

For example:

```
[ceph: root@host01 /]# ceph fs subvolumegroup create cephfs01 subvolgroup_2
10737418240
```

2. Resize a subvolume group.

```
ceph fs subvolumegroup resize VOLUME_NAME GROUP_NAME new_size [--no_shrink]
```

For example:

```
[ceph: root@host01 /]# ceph fs subvolumegroup resize cephfs01 subvolgroup_2
20737418240
[
    {
        "bytes_used": 10768679044
    },
    {
        "bytes_quota": 20737418240
    },
    {
        "bytes_pcent": "51.93"
    }
]
```

3. Fetch the metadata of a subvolume group.

```
ceph fs subvolumegroup info VOLUME_NAME GROUP_NAME
```

For example:

```
[ceph: root@host01 /]# ceph fs subvolumegroup info cephfs01 subvolgroup_2
{
    "atime": "2022-10-05 18:00:39",
    "bytes_pcent": "51.85",
    "bytes_quota": 20768679043,
    "bytes_used": 10768679044,
    "created_at": "2022-10-05 18:00:39",
    "ctime": "2022-10-05 18:21:26",
    "data_pool": "cephfs.cephfs01.data",
    "gid": 0,
    "mode": 16877,
    "mon_addrs": [
        "60.221.178.236:1221",
        "205.64.75.112:1221",
        "20.209.241.242:1221"
    ],
    "mtime": "2022-10-05 18:01:25",
    "uid": 0
}
```

Listing file system subvolume groups

Learn how to list the Ceph File System (CephFS) subvolume groups.

Before you begin

Be sure that you have a CephFS subvolume group.

Procedure

– List the CephFS subvolume groups.

```
ceph fs subvolumegroup ls VOLUME_NAME
```

For example:

```
[ceph: root@host01 /]# ceph fs subvolumegroup ls cephfs01

[

{

"name": "subgroup0"

}

]
```

Fetching the absolute path of a file system subvolume group

Learn how to fetch the absolute path of a Ceph File System (CephFS) subvolume group.

Before you begin

Be sure that you have a CephFS subvolume group.

Procedure

– Fetch the absolute path of the CephFS subvolume group.

```
ceph fs subvolumegroup getpath VOLUME_NAME GROUP_NAME
```

For example:

```
[ceph: root@host01 /]# ceph fs subvolumegroup getpath cephfs01 subgroup0
/volumes/subgroup0
```

Listing snapshots of a file system subvolume group

Learn how to list the snapshots of a Ceph File System (CephFS) subvolume group.

Before you begin

Be sure that you have a CephFS subvolume group and snapshots of a subvolume group.

Procedure

- List the snapshots of a CephFS subvolume group.

  ```
  ceph fs subvolumegroup snapshot ls VOLUME_NAME GROUP_NAME
  ```

  For example:

  ```
  [ceph: root@host01 /]# ceph fs subvolumegroup snapshot ls cephfs01 subgroup0
  []
  ```

Removing snapshot of a file system subvolume group

Learn how to remove snapshots of a Ceph File System (CephFS) subvolume group.

Before you begin

Be sure that you have a CephFS subvolume group and snapshots of a subvolume group.

About this task

> **Note:** Using the `--force` flag allows the command to succeed that would otherwise fail if the snapshot did not exist.

Procedure

- Remove the snapshot of the CephFS subvolume group.

  ```
  ceph fs subvolumegroup snapshot rm VOLUME_NAME GROUP_NAMESNAP_NAME  [--force]
  ```

  For example:

  ```
  [ceph: root@host01 /]# ceph fs subvolumegroup snapshot rm cephfs01 subgroup0 snap0 --
  force
  ```

Removing a file system subvolume group

Learn how to remove the Ceph File System (CephFS) subvolume group.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- No subvolumes under the targeted subvolume group.
  For more information, see “Removing a file system subvolume” on page 1191.
- No retained snaphshots under the targeted subvolume group.
  For more information, see “Removing snapshot of a file system subvolume” on page 1192.

About this task

Note: The removal of a subvolume group fails if it is not empty or nonexistent. The `--force` flag allows the nonexistent subvolume group to be removed.

Procedure

- Remove the CephFS subvolume group.

```
ceph fs subvolumegroup rm VOLUME_NAME GROUP_NAME [--force]
```

For example:

```
[ceph: root@host01 /]# ceph fs subvolumegroup rm cephfs01 subgroup0 --force
```

Managing Character Mapping for subvolume group

Learn how to manage character mapping in CephFS.

The CephFS subvolume and subvolumegroup interfaces provide a porcelain API to manage character mapping `ceph.dir.charmap` settings. These settings support name normalization and case sensitivity options.

Configuring Character Mapping

Configure character mapping for a subvolumegroup.

To configure the character mapping for subvolumegroup:

```
ceph fs subvolumegroup charmap set vol_name group_name setting value
```

For example,

```
ceph fs subvolumegroup charmap set vol csi normalization nfd
```

Outputs:

```
{"casesensitive":true,"normalization":"nfd","encoding":"utf8"}
```

Reading Character Mapping

Read character mapping for a subvolumegroup.

To read the character mapping for subvolumegroup:

```
ceph fs subvolumegroup charmap get vol_name group_name setting
```

To read the full character mapping for subvolumegroup:

```
ceph fs subvolumegroup charmap get vol_name group_name
```

For example,

```
ceph fs subvolumegroup charmap get vol csi
```

Outputs:

```
{"casesensitive":false,"normalization":"nfd","encoding":"utf8"}
```

Removing Character Mapping

Remove character mapping for a subvolumegroup.

Note: A charmap can only be removed when a subvolumegroup is empty.

To remove the character mapping for subvolumegroup:

```
ceph fs subvolumegroup charmap rm <vol_name> <group_name>
```

For example,

```
ceph fs subvolumegroup charmap rm vol csi
```

Outputs:

```
{}
```

Metadata information on CephFS subvolumes

You can set, get, list, and remove metadata information of Ceph File System (CephFS) subvolumes.

The custom metadata is for users to store their metadata in subvolumes. Users can store the key-value pairs similar to `xattr` in a Ceph File System.

Before working with the metadata on CephFS subvolumes be sure that you have:

- A running Red Hat Ceph Storage cluster.
- A CephFS volume, subvolume group, and subvolume created.

Viewing subvolume metrics for CephFS metadata server clients

To access subvolume-level metrics, a client can mount a Ceph File System (CephFS) or a specific subvolume.

To access client-specific metrics, a user mounts the complete CephFS. This exposes client-specific metrics of all the operations at the CephFS volume level.

> **Note:** When a user mounts an entire CephFS, it leads to performance degradation of the specific file system. To avoid this, users should perform a subvolume level mount.

Using a subvolume level mount has the following benefits:

- It improves the file system performance.
- It enables clients to identify and isolate their operations to specific subvolume paths.

Metrics for throughput, IOPS, and latency (read and write) are implemented at the subvolume level. These subvolume metrics are collected from all Metadata Servers (MDSs), shared with the metrics aggregator, and aggregated within a sliding window.

Sliding window

A sliding window is a fixed-size time interval that moves forward incrementally. In CephFS, the sliding window monitors continuous streams of file system activity and performance data. A subvolume is considered active if operations occur within the sliding window. The default window size is 30 seconds.

> **Note:** If no activity is detected during this interval, the subvolume is considered inactive.

The subvolume state (active or inactive) impacts on metrics generation.

Active

The subvolume generates metrics immediately.

Inactive

The subvolume does not generate any metrics and no value is reported.

Sliding window interval for subvolume metrics

The sliding window algorithm includes the `subv_metrics_window_interval` output value. Both the default and minimum values are defined at 30 seconds.

Note:

- The sliding window is only available for subvolume metrics, not for client metrics.
- An administrator can configure different default sliding window values for different file systems.

```
name: subv_metrics_window_interval
  type: secs
  level: dev
  desc: subvolume metrics sliding window interval, seconds
  long_desc: interval in seconds to hold values in sliding window for subvolume metrics
  default: 30
  min: 30
  services:
  - mds
```

- Get the `subv_metrics_window_interval` default value by using the **`ceph config get mds`** command.

  ```
  ceph config get mds subv_metrics_window_interval
  ```

  For example,

  ```
  [root@mon ~]# ceph config get mds subv_metrics_window_interval
  30
  ```

- Change the `subv_metrics_window_interval` default value by using the **`ceph config set mds`** command.

  ```
  ceph config set mds subv_metrics_window_interval 60
  ```

  For example,

  ```
  [root@mon ~]# ceph config get mds subv_metrics_window_interval
  30
  [root@mon ~]# ceph config set mds subv_metrics_window_interval 60
  [root@mon ~]# ceph config get mds subv_metrics_window_interval
  60
  ```

Getting subvolume metrics

View aggregated performance and usage metrics for CephFS subvolumes by using the MDS counter dump command.

To view aggregated subvolume metrics, including the `mds_subvolume_metrics` section, query MDS rank `0` by using the following command:

```
ceph tell mds.0 counter dump
```

CephFS exports gauge subvolume metrics, as defined in “Subvolume performance metrics description” on page 1199.

Table 134: Subvolume performance metrics description

Name	Type	Description
`mds_subvolume_metrics`	Array	List of subvolume metric objects .
`labels`	Object	Metadata identifying the file system and subvolume path.
`fs_name`	String	Name of the file system.
`subvolume_path`	String	Path of the subvolume within the file system.
`counters`	Object	Performance metrics for the subvolume.
`avg_read_iops`	Gauge	Average read IOPS (input/output operations per second) over the sliding window.

Name	Type	Description
avg_read_tp_Bps	Gauge	Average read throughput in bytes per second.
avg_read_lat_msec	Gauge	Average read latency in milliseconds.
avg_write_iops	Gauge	Average write IOPS over the sliding window.
avg_write_tp_Bps	Gauge	Average write throughput in bytes per second.
avg_write_lat_msec	Gauge	Average write latency in milliseconds.
last_window_end_sec	Gauge	End time of the last measurement window in seconds since the epoch.
last_window_dur_sec	Gauge	Duration of the last measurement window in seconds.

The subvolume metrics are dumped as part of the same command. The `mds_subvolume_metrics` section in the output of the **`counter dump`** command displays metrics for each subvolume.

For example,

```
"mds_subvolume_metrics": [
        {
            "labels": {
                "fs_name": "cephfs",
                "subvolume_path": "/volumes/_nogroup/test_subvolume"
            },
            "counters": {
                "avg_read_iops": 0,
                "avg_read_tp_Bps": 11,
                "avg_read_lat_msec": 0,
                "avg_write_iops": 1564,
                "avg_write_tp_Bps": 6408316,
                "avg_write_lat_msec": 338
                "last_window_end_sec": 0
                "last_window_dur_secc": 10
            }
        }
```

Setting custom metadata on the file system subvolume

Learn how to set custom metadata on the file system subvolume as a key-value pair.

About this task

Important: Custom metadata on a subvolume is not preserved when creating a snapshot of the subvolume, and hence, is also not preserved when cloning the subvolume snapshot.

Note:

- If the `key_name` exists, then the old value is replaced by the new value.
- Be sure that the *`KEY_NAME`* and *`VALUE`* is a string of ASCII characters, as specified in python's `string.printable`. The *`KEY_NAME`* is case-insensitive and is always stored in lowercase.

Procedure

1. Set the metadata on the CephFS subvolume.

   ```
   ceph fs subvolume metadata set VOLUME_NAME SUBVOLUME_NAME KEY_NAME VALUE [--
   group_name SUBVOLUME_GROUP_NAME]
   ```

   For example:

```
[ceph: root@host01 /]# ceph fs subvolume metadata set cephfs01 sub0 test_meta cluster
--group_name subgroup0
```

2. Optional: Set the custom metadata with a space in the *KEY_NAME*.
   This creates another metadata with *KEY_NAME* as `test meta` for the *VALUE* `cluster`.

   For example:

   ```
   [ceph: root@host01 /]# ceph fs subvolume metadata set cephfs01 sub0 "test meta"
   cluster --group_name subgroup0
   ```

3. Optional: Set the same metadata with a different value.
   For example:

   ```
   [ceph: root@host01 /]# ceph fs subvolume metadata set cephfs01 sub0 "test_meta"
   cluster2 --group_name subgroup0
   ```

Getting custom metadata on the file system subvolume

Learn how to get the custom metadata, the key-value pairs, of a Ceph File System (CephFS) in a volume, and optionally, in a specific subvolume group.

What to do next

Get the metadata on the CephFS subvolume:

```
ceph fs subvolume metadata get VOLUME_NAME SUBVOLUME_NAME KEY_NAME [--
group_name SUBVOLUME_GROUP_NAME]
```

For example:

```
[ceph: root@host01 /]# ceph fs subvolume metadata get cephfs01 sub0 test_meta --group_name
subgroup0

cluster
```

Listing custom metadata on the file system subvolume

Learn how to list the custom metadata associated with the key of a Ceph File System (CephFS) in a volume, and optionally, in a specific subvolume group.

List the metadata on the CephFS subvolume.

```
ceph fs subvolume metadata ls VOLUME_NAME SUBVOLUME_NAME [--
group_name SUBVOLUME_GROUP_NAME]
```

For example:

```
[ceph: root@host01 /]# ceph fs subvolume metadata ls cephfs01 sub0
{
    "test_meta": "cluster"
}
```

Removing custom metadata from the file system subvolume

Learn how to remove the custom metadata, the key-value pairs, of a Ceph File System (CephFS) in a volume, and optionally, in a specific subvolume group.

Procedure

1. Remove the custom metadata on the CephFS subvolume.

   ```
   ceph fs subvolume metadata rm VOLUME_NAME SUBVOLUME_NAME KEY_NAME [--
   group_name SUBVOLUME_GROUP_NAME]
   ```

   For example:

```
[ceph: root@host01 /]# ceph fs subvolume metadata rm cephfs01 sub0 test_meta --
group_name subgroup0
```

2. List the metadata.
   For example:

```
[ceph: root@host01 /]# ceph fs subvolume metadata ls cephfs01 sub0 --group_name
subgroup0

{}
```

Administering CephFS

Learn how to perform common Ceph File System (CephFS) administrative tasks.

Common CephFS administrative tasks are:

- Monitoring CephFS metrics in real time
- Mapping a directory to a particular MDS rank
- Disassociating a directory from an MDS rank
- Adding a new data pool
- Working with files and directory layouts
- Removing a Ceph File System
- Client features
- Using the **`ceph mds fail`** command
- Manually evict a CephFS client

Before performing administrative tasks on Ceph File Systems, be sure that you have the following:

- A running, and healthy Red Hat Ceph Storage cluster.
- Ceph Metadata Server daemons (`ceph-mds`) installed and configured.
- A Ceph File System that is created and mounted.

For more information, see:

- “Deploying the CephFS” on page 1165
- Installing

Using the `cephfs-top` utility

The Ceph File System (CephFS) provides a `top`-like utility to display metrics on Ceph File Systems in real-time. The `cephfs-top` utility is a `curses`-based Python script that uses the Ceph Manager `stats` module to fetch and display client performance metrics.

Before you begin

Before using the `cephfs-top` utility, be sure that you have the following:

- A healthy and running Red Hat Ceph Storage cluster.
- Deployment of a Ceph File System.
- Root-level access to a Ceph client node.
- `cephfs-top` package installed.

About this task

Currently, the `cephfs-top` utility supports nearly 10,000 clients.

Note: Not all of the performance stats are available in the Red Hat Enterprise Linux 9 kernel.

Procedure

1. Enable the Red Hat Ceph Storage 9 Tools repository:

```
subscription-manager repos --enable=rhceph-9-tools-for-rhel-9-x86_64-rpms
```

2. Install the `cephfs-top` package.

```
dnf install cephfs-top
```

For example,

```
[root@client ~]# dnf install cephfs-top
```

3. Enable the Ceph Manager `stats` plug-in.

```
ceph mgr module enable stats
```

For example,

```
[root@client ~]# ceph mgr module enable stats
```

4. Create the `client.fstop` Ceph user.

```
ceph auth get-or-create client.fstop mon 'allow r' mds 'allow r' osd 'allow r' mgr
'allow r' > /etc/ceph/ceph.client.fstop.keyring
```

Note: Optionally, use the `--id` argument to specify a different Ceph user, other than `client.fstop`.

5. Start the `cephfs-top` utility, by using the **`cephfs-top`** command.
For example:

```
[root@client ~]# cephfs-top
cephfs-top - Thu Oct 20 07:29:35 2022
Total Client(s): 3 - 2 FUSE, 1 kclient, 0 libcephfs
HELP: All filesystem info [Press 'm' to select a filesystem and 'q' to quit]

Filesystem: cephfs01 - 2 client(s)

  client_id mount_root chit(%) dlease(%) ofiles oicaps oinodes rtio(MB) raio(MB)
rsp(MB/s) wtio(MB) waio(MB) wsp(MB/s) rlatavg(ms) rlatsd(ms) wlatavg(ms) wlatsd(ms)
mlatavg(ms) mlatsd(ms) mount_point@host/addr
  4880     /          100.00   0.0          0      1    0       0.0      0.0      0.0
       0.0     0.00     0.0       0.0         0.0        0.0          0.0        0.27
        0.0       /mnt/cephfs@example/127.0.0.1
  4885     /          100.00   0.0          0      1    0       0.0      0.0      0.0
       0.0     0.00     0.0       0.0         0.0        0.0          0.0        0.35
        0.0        N/A@cephfs@example/127.0.0.1

Filesystem: cephfs02 - 2 client(s)

  client_id mount_root chit(%) dlease(%) ofiles oicaps oinodes rtio(MB) raio(MB)
rsp(MB/s) wtio(MB) waio(MB) wsp(MB/s) rlatavg(ms) rlatsd(ms) wlatavg(ms) wlatsd(ms)
mlatavg(ms) mlatsd(ms) mount_point@host/addr
  4889     /          100.0   0.0          0      1      0       0.0      0.0
 0.0           0.0      0.0      0.0       0.0         0.0        0.0         0.0
   0.45        0.0        /mnt/cephfs@example/127.0.0.1
```

`cephfs-top` utility interactive commands

Select a particular file system and view the metrics related to that file system with the `cephfs-top` utility interactive commands.

m

Description

Displays a menu of file systems for selection.

q

Description

Exits the utility if you are at the home screen with all file system information. If you are not at the home screen, it redirects you back to the home screen.

s

Description

Designates the sort field. **cap_hit** is the default.

l

Description

Sets the limit on the number of clients to be displayed.

r

Description

Resets the sort field and limit value to the default.

The metrics display can be scrolled using the Arrow Keys, PgUp/PgDn, Home/End and mouse.

Example of entering and exiting the file system selection menu

```
[root@client ~]# m

                        Filesystems
Press "q" to go back to home (all filesystem info) screen



                         cephfs01
                         cephfs02
[root@client ~]# q

cephfs-top - Thu Oct 20 07:29:35 2022
Total Client(s): 3 - 2 FUSE, 1 kclient, 0 libcephfs
```

`cephfs-top` utility options

Use the `cephfs-top` utility command with various options.

Example

```
[root@client ~]# cephfs-top --selftest
selftest ok
```

This list describes the different options and their descriptions.

`--cluster` *`NAME_OF_THE_CLUSTER`*

With this option, you can connect to the non-default cluster name. The default name is `ceph`.

`--id` *`USER`*

This is a client which connects to the Ceph cluster and is `fstop` by default.

`--selftest`

With this option, you can perform a selftest. This mode performs a sanity check of `stats` module.

`--conffile` *`PATH_TO_THE_CONFIGURATION_FILE`*

With this option, you can provide a path to the Ceph cluster configuration file.

`-d/--delay` *`INTERVAL_IN_SECONDS`*

The `cephfs-top` utility refreshes statistics every second by default. With this option, you can change a refresh interval.

> **Note:** Interval should be greater than or equal to 1 seconds. Fractional seconds are honored.

`--dump`

With this option, you can dump the metrics to stdout without creating a curses display use.

`--dumpfs` *`FILE_SYSTEM_NAME`*

With this option, you can dump the metrics of the given filesystem to stdout without creating a curses display use.

Using the MDS autoscaler module

The MDS Autoscaler Module monitors the Ceph File System (CephFS) to ensure that sufficient MDS daemons are available. It works by adjusting the placement specification for the Orchestrator backend of the MDS service.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A healthy and running Red Hat Ceph Storage cluster.
- Deployment of a Ceph File System.
- Root-level access to a Ceph Monitor node.

About this task

The module monitors the following file system settings to inform placement count adjustments:

- `max_mds` file system setting
- `standby_count_wanted` file system setting

The Ceph monitor daemons are still responsible for promoting or stopping MDS according to these settings. The `mds_autoscaler` adjusts the number of MDS that are created by the orchestrator.

Procedure

- Enable the MDS autoscaler module.
  For example:

```
[ceph: root@host01 /]# ceph mgr module enable mds_autoscaler
```

Unmounting Ceph File Systems mounted as kernel clients

Learn how to unmount a Ceph File System that is mounted as a kernel client.

About this task

For more information, see the `umount(8)` manual page.

Before you begin

Before you begin, be sure that you have root-level access to the node that is mounting.

Procedure

- Unmount a Ceph File System mounted as a kernel client.

```
umount MOUNT_POINT
```

  For example:

```
[root@client ~]# umount /mnt/cephfs
```

Related information

Rotating CephX keys

Unmounting Ceph File Systems mounted as FUSE clients

Learn how to unmount a Ceph File System that is mounted as a File System in User Space (FUSE) client.

Before you begin

Before beginning this task, be sure that you have root-level access to the FUSE client node.

About this task

For more information, see the `ceph-fuse(8)` manual page.

Procedure

- Unmount a Ceph File System mounted in FUSE.

  ```
  fusermount -u MOUNT_POINT
  ```

  For example,

  ```
  [root@client ~]# fusermount -u /mnt/cephfs
  ```

Related information

Rotating CephX keys

Mapping directory trees to Metadata Server daemon ranks

You can map a directory and its subdirectories to a particular active Metadata Server (MDS) rank. By doing this, its metadata is only managed by the MDS daemon holding that rank. This approach enables you to evenly spread application load or the limit impact of users' metadata requests to the entire storage cluster.

Before you begin

Before you begin, verify that the `attr` package is installed on the CephFS client node with a mounted Ceph File System. Also, be sure that you have the following:

- At least two active MDS daemons.
- Installation of `attr` package.
- User access to the CephFS client node.

About this task

> **Important:** An internal balancer already dynamically spreads the application load. Therefore, only map directory trees to ranks for certain carefully chosen applications.
> In addition, when a directory is mapped to a rank, the balancer cannot split it. Therefore, a large number of operations within the mapped directory can overload the rank and the MDS daemon that manages it.

For more information, see:

- “Layout, quota, snapshot, and network restrictions” on page 1166
- “Manually pinning directory trees to a particular rank” on page 1161
- “Configuring multiple active Metadata Server daemons” on page 1157

Procedure

1. Add the p flag to the Ceph user's capabilities.

```
ceph fs authorize FILE_SYSTEM_NAME client.CLIENT_NAME /
DIRECTORYCAPABILITYDIRECTORY [/ CAPABILITY] ...
```

```
[user@client ~]$ ceph fs authorize cephfs_a client.1 /temp rwp

client.1
  key: AQBSdFhcGZFUDRAAcKhG9Cl2HPiDMMRv4DC43A==
  caps: [mds] allow r, allow rwp path=/temp
  caps: [mon] allow r
  caps: [osd] allow rw tag cephfs data=cephfs_a
```

2. Set the `ceph.dir.pin` extended attribute on a directory.

```
setfattr -n ceph.dir.pin -v RANK DIRECTORY
```

In the following example, the `/temp` directory is assigned and all of its subdirectories to rank 2.

```
[user@client ~]$ setfattr -n ceph.dir.pin -v 2 /temp
```

Disassociating directory trees from Metadata Server daemon ranks

Disassociate a directory from a particular active Metadata Server (MDS) rank.

Before you begin

Before disassociating directory trees, be sure that you have user access to the Ceph File System (CephFS) client node.
Also, ensure that the `attr` package is installed on the client node with a mounted CephFS.

Procedure

– Set the `ceph.dir.pin` extended attribute to `-1` on a directory:

```
setfattr -n ceph.dir.pin -v -1 DIRECTORY
```

For example:

```
[user@client ~]$ setfattr -n ceph.dir.pin -v -1 /home/ceph-user
```

> **Note:** Any separately mapped subdirectories of `/home/ceph-user/` are not affected.

What to do next

– Syntax

```
setfattr -n ceph.dir.pin -v -1 DIRECTORY
```

Example

```
[user@client ~]$ setfattr -n ceph.dir.pin -v -1 /home/ceph-user
```

Related information

Mapping directory trees to Metadata Server daemon ranks

Multiple data pools use-cases

Ceph File System (CephFS) supports creating multiple data pools in a CephFS volume. Each data pool can be assigned to a dedicated subvolume or directory in the same file system. This helps administrators optimize their Ceph cluster's performance and storage behavior.

CephFS is a distributed, POSIX-compliant file system. All CephFS deployments require a metadata RADOS pool. By default, a single, additional RADOS pool is provisioned for file data. Deploying additional data pools can be beneficial for supporting multiple workloads, storage media types, and data protection strategies.

Note: When creating a new CephFS, it is recommended to configure the initial data pool as a replicated SSD pool, rather than using Erasure Coding (EC) or deploying on HDDs. The CephFS metadata and default data pools may be colocated on the same SSDs.

Note: If you do not configure the first data pool as a replicated SSD pool, the configurations cannot be retrofitted unless you:

- Recreate the entire CephFS filesystem
- Migrate all existing data to the new setup

The following lists the use-cases for multiple data pools:

File System as a Service (FSaaS)

Service providers can offer differentiated storage tiers by mapping subvolumes to pools with distinct performance and durability characteristics.

Workload segmentation

Assign different workloads to subvolumes or directories backed by pools optimized for their specific needs. For optimized performance and cost efficiency:

- Use fast media (SSDs) and replicated pools for throughput-intensive workloads requiring low latency and high IOPS.
- Store archival or infrequently accessed data in subvolumes or directories backed by erasure-coded (EC) HDD or Quad-Level Cell (QLC) SSD pools to reduce total cost of ownership (TCO) while maintaining durability.

Benefits of using multiple pools

Administrators can optimize their Ceph clusters by aligning subvolume configurations with workload priorities by selecting appropriate attributes.

Table 135: Attribute selection considerations for CephFS optimization

Attributes	Types	
Data Protection Scheme	Replicated pools	EC pools
	Recommended for performance-sensitive workloads.	Suitable for cost-efficient, durable storage.
Storage Media	SSD	HDD or QLC SSD
	High throughput and low latency.	Cost-effective capacity for less demanding workloads.
Performance vs. Cost Trade-offs	Throughput-optimized	TCO-optimized
	SSDs with replication for high-performance workloads.	HDDs or QLC SSDs with erasure coding for archival or backup workloads.

By leveraging these attributes and following recommendations, administrators can optimize CephFS deployments to meet diverse operational requirements.

Adding data pools

The Ceph File System (CephFS) supports adding more than one pool to be used for storing data.

Before you begin

Before you begin, be sure that you have root-level access to the Ceph Monitor node.

About this task

Adding data pools can be useful for:

- Storing log data on reduced redundancy pools.
- Storing user home directories on an SSD or NVMe pool.
- Basic data segmentation.

Before using another data pool in the Ceph File System, you must add a data pool.

By default, for storing file data, CephFS uses the initial data pool that was specified during its creation. To use a secondary data pool, you must also configure a part of the file system hierarchy to store file data in that pool or optionally within a namespace of that pool, using file and directory layouts.

Procedure

1. Create a data pool.

   ```
   ceph osd pool create POOL_NAME
   ```

   Replace *POOL_NAME* with the name of the pool.
   For example:

   ```
   [ceph: root@host01 /]# ceph osd pool create cephfs_data_ssd

   pool 'cephfs_data_ssd' created
   ```

2. Add the newly created pool under the control of the Metadata Servers.

   ```
   ceph fs add_data_pool FS_NAME POOL_NAME
   ```

   - Replace *FS_NAME* with the name of the file system.
   - Replace *POOL_NAME* with the name of the pool.

   For example:

   ```
   [ceph: root@host01 /]# ceph fs add_data_pool cephfs cephfs_data_ssd

   added data pool 6 to fsmap
   ```

3. Verify that the pool was successfully added.
   For example:

   ```
   [ceph: root@host01 /]# ceph fs ls

   name: cephfs, metadata pool: cephfs_metadata, data pools: [cephfs_data
   cephfs_data_ssd]
   ```

4. Optional: Remove a data pool from the file system.

   ```
   ceph fs rm_data_pool FS_NAME POOL_NAME
   ```

   For example:

   ```
   [ceph: root@host01 /]# ceph fs rm_data_pool cephfs cephfs_data_ssd

   removed data pool 6 from fsmap
   ```

   After removing a data pool, verify that the pool was successfully removed.

For example:

```
[ceph: root@host01 /]# ceph fs ls

name: cephfs, metadata pool: cephfs_metadata, data pools: [cephfs_data]
```

5. If you use the `cephx` authentication, make sure that clients can access the new pool.

Related information

Overview of file and directory layouts
Creating client users for a Ceph File System

Taking down a CephFS cluster

You can take down Ceph File System (CephFS) cluster by setting the **down** flag to `true`. Setting the flag to `true` gracefully shuts down the Metadata Server (MDS) daemons by flushing journals to the metadata pool and stopping all client I/O.

Before you begin

Be sure that you have root-level access to a Ceph Monitor node.

About this task

You can take the CephFS cluster down quickly to test the deletion of a file system and bring the Metadata Server (MDS) daemons down. For example, when practicing a disaster recovery scenario. Doing this sets the `jointable` flag to prevent the MDS standby daemons from activating the file system.

Procedure

Use one of the following procedures to take down a CephFS cluster. To bring the CephFS cluster back up, see "Bringing the CephFS cluster back up" on page 1210.

- Mark the CephFS cluster down.

```
ceph fs set FS_NAME down true
```

For example:

```
[ceph: root@host01 /]# ceph fs set cephfs down true
```

- Quickly take down a CephFS cluster.

```
ceph fs fail FS_NAME
```

For example:

```
[ceph: root@host01 /]# ceph fs fail cephfs
```

Bringing the CephFS cluster back up

Procedure

Use one of the following procedures to bring a CephFS cluster back up after taking it down.

- Bring the CephFS cluster back up.

```
ceph fs set FS_NAME down false
```

For example:

```
[ceph: root@host01 /]# ceph fs set cephfs down false
```

- Get the CephFS cluster back up, by setting `cephfs` to `joinable`.

```
ceph fs set FS_NAME joinable true
```

For example:

```
[ceph: root@host01 /]# ceph fs set cephfs joinable true

cephfs marked joinable; MDS may join as newly active.
```

Removing a CephFS

Learn how to remove a Ceph File System.

Before you begin

Before removing a CephFS, be sure that you have root-level access to a Ceph Monitor node and to back up your data.

About this task

You can remove a Ceph File System (CephFS). Before doing so, consider backing up all the data and verifying that all clients have unmounted the file system locally.

> **Warning:** This operation is destructive and will make the data that is stored on the Ceph File System permanently inaccessible.

Procedure

1. Mark the storage cluster as down:

   ```
   ceph fs set FS_NAME down true
   ```

   Replace *FS_NAME* with the name of the Ceph File System that you want to remove.
   For example:

   ```
   [ceph: root@host01 /]# ceph fs set cephfs down true

   cephfs marked down.
   ```

2. Display the status of the Ceph File System.

   ```
   ceph fs status
   ```

   For example:

   ```
   [ceph: root@host01 /]# ceph fs status

   cephfs - 0 clients
   ======
   +-------------------+----------+-------+-------+
   |       POOL        |   TYPE   |  USED | AVAIL |
   +-----------------+------------+-------+-------+
   |cephfs.cephfs.meta | metadata | 31.5M |  52.6G|
   |cephfs.cephfs.data |   data   |    0  |  52.6G|
   +-----------------+----------+-------+---------+
                  STANDBY MDS
   cephfs.ceph-host01
   cephfs.ceph-host02
   cephfs.ceph-host03
   ```

3. Remove the Ceph File System.

   ```
   ceph fs rm FS_NAME --yes-i-really-mean-it
   ```

   Replace *FS_NAME* with the name of the Ceph File System that you want to remove.
   For example:

```
[ceph: root@host01 /]# ceph fs rm cephfs --yes-i-really-mean-it
```

4. Verify that the file system has been successfully removed.
   For example:

```
[ceph: root@host01 /]# ceph fs ls
```

5. Optional: Remove data and metadata pools associated with the removed file system.

Related information

Deleting a pool

Using the `ceph mds fail` command

Use the **`ceph mds fail`** command to mark an MDS daemon as failed or restart a running MDS daemon.

Before you begin

Before using the **`ceph mds fail`** command, make sure that the Ceph MDS daemons are installed and configured.

About this task

Marking an MDS daemon as failed

If the daemon was active and a suitable standby daemon was available, and if the standby daemon was active after disabling the `standby-replay` configuration, using this command forces a failover to the standby daemon. By disabling the `standby-replay` daemon, this prevents new `standby-replay` daemons from being assigned.

Restarting a running MDS daemon

If the daemon was active and a suitable standby daemon was available, the "failed" daemon becomes a standby daemon.

For more information about MDS daemons, see:

- “Decreasing the number of active Metadata Server daemons” on page 1161
- “Configuring the number of standby daemons” on page 1158
- “Metadata Server ranks” on page 1151

Procedure

- Fail an MDS daemon.

```
ceph mds fail MDS_NAME
```

Replace the *MDS_NAME* with the name of the `standby-replay` MDS node.

> **Note:** Use the **`ceph fs status`** command to find the Ceph MDS name.

For example:

```
[ceph: root@host01 /]# ceph mds fail example01
```

Client features

At times you might want to set Ceph File System (CephFS) features that clients must support to enable them to use Ceph File Systems. Clients without these features might disrupt other CephFS clients, or behave in unexpected ways. Also, you might want to require new features to prevent older, and possibly buggy clients from connecting to a Ceph File System.

Important: CephFS clients missing newly added features are evicted automatically. You can list all the CephFS features by using the **`fs features ls`** command. You can add or remove requirements by using the **`fs required_client_features`** command.

Use the following syntax:

```
fs required_client_features FILE_SYSTEM_NAME add FEATURE_NAME
fs required_client_features FILE_SYSTEM_NAME rm FEATURE_NAME
```

Table 136: Client features

Client feature	Description
`reply_encoding`	The Ceph Metadata Server (MDS) encodes reply requests in extensible format, if the client supports this feature.
`reclaim_client`	The Ceph MDS allows a new client to reclaim another, perhaps a dead, client's state.
`lazy_caps_wanted`	When a stale client resumes, the Ceph MDS only needs to re-issue the capabilities that are explicitly wanted, if the client supports this feature.
`multi_reconnect`	After a Ceph MDS failover event, the client sends a reconnect message to the MDS to reestablish cache states. A client can split large reconnect messages into multiple messages.
`deleg_ino`	A Ceph MDS delegates inode numbers to a client, if the client supports this feature. Delegating inode numbers is a prerequisite for a client to do async file creation.
`metric_collect`	CephFS clients can send performance metrics to a Ceph MDS.
`alternate_name`	CephFS clients can set and understand alternative names for directory entries. This feature allows for encrypted file names.

CephFS client evictions

When a Ceph File System (CephFS) client is unresponsive or misbehaving, it might be necessary to forcibly stop, or evict it from accessing the CephFS.

Evicting a CephFS client prevents it from communicating further with Metadata Server (MDS) daemons and Ceph OSD daemons. If a CephFS client is buffering I/O to the CephFS at the time of eviction, then any un-flushed data will be lost. The CephFS client eviction process applies to all client types: FUSE mounts, kernel mounts, and any process that uses `libcephfs` API library.

You can evict CephFS clients automatically, if they fail to communicate promptly with the MDS daemon, or manually. For more information, see "Manually evicting a CephFS client" on page 1214.

Automatic client eviction

These scenarios cause an automatic CephFS client eviction:

- If a CephFS client has not communicated with the active MDS daemon for over the default of 300 seconds, or as set by the **`session_autoclose`** option.
- If the **`mds_cap_revoke_eviction_timeout`** option is set, and a CephFS client has not responded to the cap revoke messages for over the set number of seconds. The **`mds_cap_revoke_eviction_timeout`** option is disabled by default.

– During MDS startup or failover, the MDS daemon goes through a reconnect phase waiting for all the CephFS clients to connect to the new MDS daemon. If any CephFS clients fail to reconnect within the default time window of 45 seconds, or as set by the **`mds_reconnect_timeout`** option.

Blocklist CephFS clients

Ceph File System (CephFS) client blocklisting is enabled by default. When you send an eviction command to a single Metadata Server (MDS) daemon, it propagates the blocklist to the other MDS daemons. This is to prevent the CephFS client from accessing any data objects, so it is necessary to update the other CephFS clients, and MDS daemons with the latest Ceph OSD map, which includes the blocklisted client entries.

An internal “osdmap epoch barrier” mechanism is used when updating the Ceph OSD map. The purpose of the barrier is to verify the CephFS clients receiving the capabilities have a sufficiently recent Ceph OSD map before any capabilities are assigned that might allow access to the same RADOS objects, as to not race with canceled operations. For example, from ENOSPC or blocklisted clients from evictions.

If you are experiencing frequent CephFS client evictions due to slow nodes or an unreliable network and you cannot fix the underlying issue, you can set the MDS to be less strict. It is possible to respond to slow CephFS clients by dropping their MDS sessions, but permit the CephFS client to re-open sessions and to continue talking to Ceph OSDs. Setting the `mds_session_blocklist_on_timeout` and `mds_session_blocklist_on_evict` options to `false` enables this mode.

> **Note:** When blocklisting is disabled, the evicted CephFS client has only an effect on the MDS daemon you send the command to. On a system with multiple active MDS daemons, you need to send an eviction command to each active daemon.

Manually evicting a CephFS client

You might want to manually evict a Ceph File System (CephFS) client if the client is misbehaving and you do not have access to the client node, or if a client dies and you do not want to wait for the client session to time out.

Before you begin

Before evicting a Ceph File System client, be sure that you have root-level access to the Ceph Monitor node.

Procedure

1. Review the client list.

   ```
   ceph tell DAEMON_NAME client ls
   ```

   For example:

```
[ceph: root@host01 /]# ceph tell mds.0 client ls
[
    {
        "id": 47385,
        "entity": {
            "name": {
                "type": "client",
                "num": 47385
            },
            "addr": {
                "type": "any",
                "addr": "10.8.128.23:0",
                "nonce": 428447873
            }
        },
        "state": "open",
        "num_leases": 0,
        "num_caps": 0,
        "request_load_avg": 0,
        "uptime": 829961.12384965899,
        "requests_in_flight": 0,
        "num_completed_requests": 0,
        "num_completed_flushes": 0,
        "reconnecting": false,
        "recall_caps": {
            "value": 0,
            "halflife": 60
        },
        "release_caps": {
            "value": 0,
            "halflife": 60
        },
        "recall_caps_throttle": {
            "value": 0,
            "halflife": 1.5
        },
        "recall_caps_throttle2o": {
            "value": 0,
            "halflife": 0.5
        },
        "session_cache_liveness": {
            "value": 0,
            "halflife": 300
        },
        "cap_acquisition": {
            "value": 0,
            "halflife": 10
        },
        "delegated_inos": [],
        "inst": "client.47385 10.8.128.23:0/428447873",
        "completed_requests": [],
        "prealloc_inos": [],
        "client_metadata": {
            "client_features": {
                "feature_bits": "0x00000000000fffff"
            },
            "metric_spec": {
                "metric_flags": {
                    "feature_bits": "0x000000000000ffff"
                }
            },
            "ceph_sha1": "3c9b67d46bf428c8eb52f31dfd4c722a2e896cf7",
            "ceph_version": "ceph version 17.2.6-96.el9cp
(3c9b67d46bf428c8eb52f31dfd4c722a2e896cf7) quincy (stable)",
            "entity_id": "admin",
            "hostname": "node03",
            "mount_point": "/mnt/cephfs_io_tbmutle114_1",
            "pid": "16699",
            "root": "/"
        }
]
```

2. Evict the specified CephFS client.

```
ceph tell DAEMON_NAME client evict id=ID_NUMBER
```

For example:

```
[ceph: root@host01 /]# ceph tell mds.0 client evict id=4305
```

Removing a CephFS client from the blocklist

In some situations, it can be useful to allow a previously blocklisted Ceph File System (CephFS) client to reconnect to the storage cluster.

Before you begin

Before removing a Ceph File System client, be sure that you have root-level access to the Ceph Monitor node.

About this task

Important: Removing a CephFS client from the blocklist puts data integrity at risk, and does not guarantee a fully healthy, and functional CephFS client as a result. The best way to get a fully healthy CephFS client back after an eviction, is to unmount the CephFS client and do a fresh mount. If other CephFS clients are accessing files that the blocklisted CephFS client was buffering I/O to, it can result in data corruption.

Procedure

1. Review the blocklist.
   For example:

   ```
   [ceph: root@host01 /]# ceph osd blocklist ls

   listed 1 entries
   127.0.0.1:0/3710147553 2022-05-09 11:32:24.716146
   ```

2. Remove the CephFS client from the blocklist.

   ```
   ceph osd blocklist rm CLIENT_NAME_OR_IP_ADDR
   ```

   For example:

   ```
   [ceph: root@host01 /]# ceph osd blocklist rm 127.0.0.1:0/3710147553

   un-blocklisting 127.0.0.1:0/3710147553
   ```

3. Optional: Set kernel-based CephFS clients automatically to reconnect when removing them from the blocklist.
   On the kernel-based CephFS client, set the following option to `clean` either when doing a manual mount, or automatically mounting with an entry in the `/etc/fstab` file.

   ```
   recover_session=clean
   ```

4. Optional: Set FUSE-based CephFS clients automatically to reconnect when removing them from the blocklist.
   On the FUSE client, set the following option to `true` either when doing a manual mount, or automatically mounting with an entry in the `/etc/fstab` file.

   ```
   client_reconnect_stale=true
   ```

Related information

Mounting the Ceph File System as a FUSE client

Disabling volumes plug-in

Learn how to disable volume plug-ins.

By default the volumes plug-in is enabled and set to always on. However, in certain cases, it might be appropriate to disable it. For example, when a CephFS is in a degraded state, the volumes plug-in commands can accumulate in MGR instead of getting served. This eventually causes policy throttles to take effect and the MGR becomes unresponsive.

In this event, you can disable the volumes plug-in even though it is an always on module in MGR.

Run the following command:

```
ceph mgr module disable volumes --yes-i-really-mean-i
```

> **Warning:** This command disables operations and removes commands of the volumes plug-in as it disables all CephFS services on the Ceph cluster that is accessed through this plug-in.

Another way for disabling volume plug-ins is to disable asynchronous threads that are started by volumes plug-ins for cloning and purging trash.

Pausing asynchronous subvolume purge and clone operations

Pause the Ceph Manager threads that purge deleted subvolumes and clone snapshots to reduce load on the Ceph Metadata Server during cluster recovery.

About this task

The CephFS volumes plugin runs two sets of background threads, one for purging removed subvolumes and other for cloning subvolume snapshots. Either or both of the thread sets can be paused during cluster recovery, or when they cause too much strain on the Ceph Manager (MGR) or Ceph Metadata Server (MDS). Pausing the threads is optional.

Procedure

1. Pause the threads that purge deleted subvolumes.

   ```
   ceph config set mgr mgr/volumes/pause_purging true
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph config set mgr mgr/volumes/pause_purging true
   ```

2. Pause the threads that clone subvolume snapshots.

   ```
   ceph config set mgr mgr/volumes/pause_cloning true
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph config set mgr mgr/volumes/pause_cloning true
   ```

3. Verify the current values.

   ```
   ceph config get mgr mgr/volumes/pause_purging
   ceph config get mgr mgr/volumes/pause_cloning
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph config get mgr mgr/volumes/pause_purging
   true
   ```

What to do next

To resume the threads, set the corresponding **`mgr/volumes/pause_purging`** or **`mgr/volumes/pause_cloning`** parameter to `false`:

```
ceph config set mgr mgr/volumes/pause_purging false
ceph config set mgr mgr/volumes/pause_cloning false
```

CephFS subvolume quiesce

The subvolume quiesce provides enterprise-level consistency assurance for multi-client applications with one or more subvolumes. With this feature, you can pause the input/output (IO) to a set of subvolumes of a volume (file system). The ability to pause across all clients helps ensure that any persistent checkpoints the application reaches before the pause are recoverable from the snapshots that are made during the pause.

In cases where many write operations are run, CephFS snapshots do not provide strong consistent backup and disaster recovery for distributed applications. The subvolume quiesce helps provide the consistency that is needed at an enterprise level.

The `volumes` plug-in provides a command-line interface (CLI) to initiate and await the pause for a set of subvolumes. This pause is called a quiesce, which is also the command name.

The `fs quiesce` function is based on a lower-level `quiesce db` service that is provided by the Metadata Server (MDS) daemons, which operates at a file system path for granularity. The `volumes` plug-in maps the subvolume names to their corresponding paths on the file system and then issues the corresponding `quiesce db` command to the MDS.

Example

```
ceph fs quiesce VOLUME_NAME --set-id ID-NAME [GROUP_NAME/]SUB_NAME... --await

# Now, you can perform actions while the IO pause is active, like taking snapshots.

ceph fs quiesce VOLUME_NAME --set-id ID-NAME --release --await
```

After the release command is successful, all members of the set are confirmed as still paused and are released.

Operations

You can request the quiesce for a set of one or more subvolumes (that is paths in a file system). This set is referred to as the quiesce set. A unique set ID identifies every quiesce set. You can manipulate a quiesce set in the following ways:

include

Include one or more subvolumes - quiesce set members.

exclude

Exclude one or more members.

cancel

Cancel the set asynchronously aborting the pause on all its current members.

release

Release the set requesting the end of the pause from all members and expecting an acknowledgment from all clients.

query

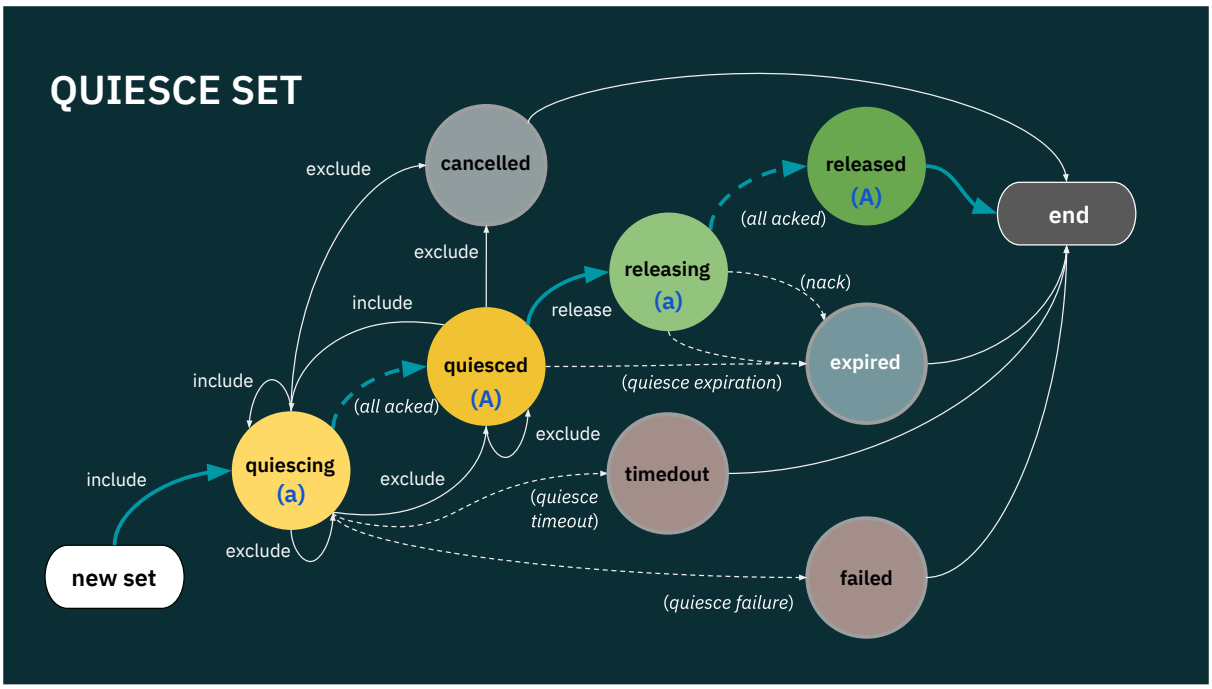
Query the current state of a set by ID or all active or known sets.

cancel all

Cancel all active sets in case an immediate resume of IO is required.

The operations that are listed before are non-blocking that is they attempt the intended modification and return with an up-to-date version of the target set, whether the operation was successful or not. The set can change states due to the modification, and the version that is returned in the response is guaranteed to be consistent with other potentially successful operations from the same control loop batch.

Some states are set `awaitable`. Any of the commands that are mentioned before can be amended with an await modifier, which causes the commands to block on the set after applying their intended modification, as long as the resulting set state is `awaitable`. The command remains blocked until the set reaches the awaited state, gets modified by another command, or transitions into another state. The return code unambiguously identifies the exit condition, and the contents of the response always carry the latest known set state.



`Awaitable` states on the diagram are marked with (a) or (A). Blocking versions of the operations pends while the set is in an (a) state and completes with success if it reaches an (A) state. If the set is already at an (A) state, the operation completes immediately with success.

Most of the operations require a set-ID. The exceptions are:

- Creation of a new set without specifying a set ID.
- Query of active or all known sets.
- The cancel all.

You can create a new set by including members through the **`include`** or **`reset`** commands. You can specify a set ID, and if it is a new ID then the set is created with the specified members in the `QUIESCING` state.

When no set ID is specified while including or resetting the members, then a new set with a unique set ID is created. The set ID is known to the caller by inspecting the output through the following command:

Example

```
ceph fs quiesce fs1 sub1 --set-id=unique-id
{
    "epoch": 3,
    "set_version": 1,
    "sets": {
        "unique-id": {
            "version": 1,
            "age_ref": 0.0,
            "state": {
                "name": "TIMEDOUT",
                "age": 0.0
            },
            "timeout": 0.0,
            "expiration": 0.0,
            "members": {
                "file:/volumes/_nogroup/sub1/b1fcce76-3418-42dd-aa76-f9076d047dd3": {
                    "excluded": false,
                    "state": {
                        "name": "QUIESCING",
                        "age": 0.0
                    }
                }
            }
        }
    }
}
```

However, in the output of this example, the set that is created successfully is already `TIMEDOUT`. This is expected since the timeout parameter is not specified for this quiesce, and in the output it is initialized to 0 by default, along with the expiration.

CephFS subvolume quiesce parameters

Learn about the different parameters that can be used while quiescing the CephFS subvolumes.

Use the following parameters that can be used while quiescing:

- "Timeout" on page 1220
- "Awaiting" on page 1221
- "Quiesce-Await and Expiration" on page 1222
- "If Version" on page 1223

Timeout

The two timeout parameters, **`timeout`** and **`expiration`**, are the main guards against accidentally causing a DOS condition for the application. You can add the **`--timeout`** or **`--expiration`** arguments to any commands to update the values for an active set. If these arguments are present, the values are applied before the action requested by the command.

In the following example, `unique-id` set is in a terminal state. No changes are allowed to sets that are in their terminal states that is which are inactive.

```
$ ceph fs quiesce fs1 --set-id=unique-id --timeout=10 > /dev/null
Error EPERM:
```

The following example shows a set-ID that is not specified so the system created a new one. In the output, the set ID is `8988b419`. The command is successful and the set is `QUIESCING`.

```
ceph fs quiesce fs1 sub1 --timeout 60
{
    "epoch": 3,
    "set_version": 2,
    "sets": {
        "8988b419": {
            "version": 2,
            "age_ref": 0.0,
            "state": {
                "name": "QUIESCING",
                "age": 0.0
            },
            "timeout": 60.0,
            "expiration": 0.0,
            "members": {
                "file:/volumes/_nogroup/sub1/b1fcce76-3418-42dd-aa76-f9076d047dd3": {
                    "excluded": false,
                    "state": {
                        "name": "QUIESCING",
                        "age": 0.0
                    }
                }
            }
        }
    }
}
```

The following example is adding more members to the set while quiescing.

> **Note:** The **`--include`** parameter is optional. Include parameter is assumed if no operation is given while members are provided.

```
ceph fs quiesce fs1 --set-id 8988b419 --include sub2 sub3
{
    "epoch": 3,
    "set_version": 3,
    "sets": {
        "8988b419": {
            "version": 3,
            "age_ref": 0.0,
            "state": {
                "name": "QUIESCING",
                "age": 30.7
            },
            "timeout": 60.0,
            "expiration": 0.0,
            "members": {
                "file:/volumes/_nogroup/sub1/b1fcce76-3418-42dd-aa76-f9076d047dd3": {
                    "excluded": false,
                    "state": {
                        "name": "QUIESCING",
                        "age": 30.7
                    }
                },
                "file:/volumes/_nogroup/sub2/bc8f770e-7a43-48f3-aa26-d6d76ef98d3e": {
                    "excluded": false,
                    "state": {
                        "name": "QUIESCING",
                        "age": 0.0
                    }
                },
                "file:/volumes/_nogroup/sub3/24c4b57b-e249-4b89-b4fa-7a810edcd35b": {
                    "excluded": false,
                    "state": {
                        "name": "QUIESCING",
                        "age": 0.0
                    }
                }
            }
        }
    }
}
```

The timeout argument specifies how much time is to be given to the system to reach the QUIESCED state on the set. However, since new members can be added to an active set at any time, the timeout is not measured from the set creation time. Hence, the timeout is tracked per member and every member has timeout seconds to quiesce. If any member takes longer than the timeout seconds, the whole set is marked as `timedout` and the pause is released.

After the set is in the `quiesced` state, it begins the expiration timer. This timer is tracked per set as a whole, not per member. After the expiration seconds elapse, the set changes into an `expired` state, unless a dedicated operation actively releases it or cancels it.

It is possible to add new members to a QUIESCED set. In this case, the set transitions back to `quiescing`, and the new members have their timeout to quiesce. If they succeed, then the set is again `quiesced` and the expiration timer restarts.

> **Warning:**
> The expiration timer does not apply when a set is `quiescing`. It is reset to the value of the expiration property when the set becomes `quiesced`.
>
> Timeout does not apply to members that are `quiesced`.

Awaiting

To wait for the quiesce set to reach the `quiesced` state, you must await it at some point. You can give the **`--await`** argument along with other arguments to let the system know that it must wait for the set to reach the quiesced state.

There are two types of `await`: **`quiesce await`** and **`release await`**. **`Quiesce await`** is the default, and **`release await`** requires the **`--release`** parameter to be present in the argument list.

> **Note:** Issuing a **`quiesce await`** when the set is not `quiescing` is not allowed.

Trying to **`--release`** a set that is not `quiesced` gives an EPERM error regardless of whether await is requested alongside. However, it is not an error to **`release await`** an already released set or to **`quiesce await`** a `quiesced` set.

Since a set is awaited after the application of the **`--await-augmented`** command, the awaited operation can mask a successful result with its error.

The following example shows to **`cancel-await`** a set:

```
ceph fs quiesce fs1 --set-id set1 --cancel --await
{
    // ...
    "sets": {
        "set1": {
            // ...
            "state": {
                "name": "CANCELED",
                "age": 0
            },
            // ...
        }
    }
}
Error EPERM:
```

Awaiting a canceled set is not allowed and results in an EINVAL error. The `--cancel` operation succeeds synchronously for a set in an active state.

When awaiting, you can specify a maximum duration, you want this await request to block. If the target awaited state is not reached within the specified duration, then an error EINPROGRESS is returned.

To resolve the EINPROGRESS error, use the argument **`--await-for`**=*SECONDS*. **`--await`** argument is equivalent to **`--await-for=infinity`**. If both **`--await`** and **`--await-for`** are present, then the time limit from **`--await-for`** is considered.

For example,

```
time ceph fs quiesce fs1 sub1 --timeout=10 --await-for=2
{
    "epoch": 6,
    "set_version": 3,
    "sets": {
        "c3c1d8de": {
            "version": 3,
            "age_ref": 0.0,
            "state": {
                "name": "QUIESCING",
                "age": 2.0
            },
            "timeout": 10.0,
            "expiration": 0.0,
            "members": {
                "file:/volumes/_nogroup/sub1/b1fcce76-3418-42dd-aa76-f9076d047dd3": {
                    "excluded": false,
                    "state": {
                        "name": "QUIESCING",
                        "age": 2.0
                    }
                }
            }
        }
    }
}
Error EINPROGRESS:
ceph fs quiesce fs1 sub1 --timeout=10 --await-for=2  0.41s user 0.04s system 17% cpu 2.563
total
```

Quiesce-Await and Expiration

`Quiesce await` resets the internal expiration timer. This parameter allows for a watchdog approach to a multistep process under the IO pause by repeatedly **`--awaiting`** an already `quiesced` set. Consider the following example script:

```
set -e   # (1)
ceph fs quiesce fs1 sub1 sub2 sub3 --timeout=30 --expiration=10 --set-id="snapshots" --
await # (2)
ceph fs subvolume snapshot create a sub1 snap1-sub1  # (3)
ceph fs quiesce fs1 --set-id="snapshots" --await  # (4)
ceph fs subvolume snapshot create a sub2 snap1-sub2  # (3)
ceph fs quiesce fs1 --set-id="snapshots" --await  # (4)
ceph fs subvolume snapshot create a sub3 snap1-sub3  # (3)
ceph fs quiesce fs1 --set-id="snapshots" --release --await  # (5)
```

> **Warning:** This example uses arbitrary timeouts to convey the concept. In real life, the values must be carefully chosen following the actual system requirements and specifications

The goal of the script is to take consistent snapshots of 3 subvolumes. In the example, the `bash -e` option (1) is set to exit the script if any or the following commands return with a nonzero status.

An input/output (IO) pause is requested for the three subvolumes (2). The timeouts are set to allow the system to spend up to 30 seconds to reach the quiesced state across all members. The members stay quiesced for up to 10 seconds before the quiesce expires and the IO is resumed. The **`--await`** is also specified to only proceed after the quiesce is reached.

The script proceeds with the set of command pairs that take the snapshot and call **`--await`** on set to extend the expiration timeout for 10 more seconds (3,4). This approach gives up to 10 seconds for every snapshot but also allows taking as many snapshots as needed without losing the IO pause. This approach also helps get consistent snapshots. If you want, you can update the expiration every time you call for `await`.

If any of the snapshots get stuck and takes longer than 10 seconds to complete, then the next call to **`--await`** returns an error since the set is in the `expired` state, which is not an `awaitable` state. This helps to limit the impact on the applications in such scenarios.

If Version

It is not enough to only observe the successful quiesce or release. Another client can make a concurrent change to the set.

Consider this example:

```
ceph fs quiesce fs1 sub1 sub2 sub3 --timeout=30 --expiration=60 --set-id="snapshots" --
await  #(1)
ceph fs subvolume snapshot create a sub1 snap1-sub1  # (2)
ceph fs subvolume snapshot create a sub2 snap1-sub2  # (3)
ceph fs subvolume snapshot create a sub3 snap1-sub3  # (4)
ceph fs quiesce fs1 --set-id="snapshots" --release --await  # (5)
```

The sequence looks good, and the release (5) completes successfully. However, it can be that before snap for sub3 (4) is taken, another session excludes sub3 from the set, resuming its input/output (IO):

```
ceph fs quiesce fs1 --set-id="snapshots" --exclude sub3
```

As removing a member from a set does not affect its `quiesced` state, the release command (5) has no reason to fail. It acknowledges the two unexcluded members sub1 and sub2 and reports success.

To address this or similar problems, the quiesce command supports an optimistic concurrency mode. To activate it, pass an **`--if-version=<version>`** that compares the database version of the set and the operation only proceeds if the values match. Otherwise, the command is not run and the return status is `ESTALE`.

Every command that modifies a set returns this set on the stdout, regardless of the exit status. In the example, notice that every set carries a version property, which gets updated whenever this set is modified.

In this example, the initial quiesce command (1) would return the newly created set with ID snapshots and some version, for example, consider 13. Since no other changes are expected to the set while taking the snapshots with the commands (2,3,4), the release command (5) might look like this:

```
ceph fs quiesce fs1 --set-id="snapshots" --release --await --if-version=13 # (5)
```

This way, the release command returns ESTALE instead of 0, indicating that something is not right with the quiesce set and the snapshots are not consistent.

> **Note:** When **`--if-version`** and the command returns `ESTALE`, the requested action is not run. It means that the script wants to run some unconditional command on the set to adjust the state according to the requirements.

The other use of the **`--if-version`** argument is for automation software. It is possible to create a new quiesce set with a given set ID. Drivers like the CSI for Kubernetes can use their internal request ID to eliminate the need to keep an extra mapping to the quiesce set ID. However, to guarantee uniqueness, the driver verifies that the set is new. For that, **`if-version=0`** can be used, and it only creates the new set if no other set with this ID is present in the database:

```
ceph fs quiesce fs1 sub1 sub2 sub3 --set-id="external-id" --if-version=0
```

CephFS quotas

Use Ceph File System (CephFS) quotas to restrict the number of bytes or the number of files that are stored in the directory structure. Ceph File System quotas are fully supported by using a FUSE client or by using Kernel clients, version 4.17 or newer. Learn how to view, set, and remove quotas on any directory in the file system.

When configuring quotas, be sure that you have the following:

- A running, and healthy Red Hat Ceph Storage cluster.
- Deployment of a Ceph File System.
- Root-level access to the Ceph client node.
- The `attr` package installed.

For more information, see:

- “Deploying the CephFS” on page 1165.
- The `getfattr(1)` manual page.
- The `setfattr(1)` manual page.

Limitations

- CephFS quotas rely on the cooperation of the client mounting the file system to stop writing data when it reaches the configured limit. However, quotas alone cannot prevent an adversarial, untrusted client from filling the file system.
- Once processes that write data to the file system reach the configured limit, a short period of time elapses between when the amount of data reaches the quota limit, and when the processes stop writing data. The time period generally measures in the tenths of seconds. However, processes continue to write data during that time. The amount of additional data that the processes write depends on the amount of time elapsed before they stop.
- When using path-based access restrictions, be sure to configure the quota on the directory to which the client is restricted, or to a directory nested beneath it. If the client has restricted access to a specific path based on the MDS capability, and the quota is configured on an ancestor directory that the client cannot access, the client will not enforce the quota. For example, if the client cannot access the `/home/` directory and the quota is configured on `/home/`, the client cannot enforce that quota on the directory `/home/user/`.
- Snapshot file data that has been deleted or changed does not count towards the quota.

Viewing quotas

Use the **`getfattr`** command and the `ceph.quota` extended attributes to view the quota settings for a directory.

About this task

> **Note:** If the attributes appear on a directory inode, then that directory has a configured quota. If the attributes do not appear on the inode, then the directory does not have a quota set, although its parent directory might have a quota that is configured. If the value of the extended attribute is `0`, the quota is not set.

For more information, see the `getfattr(1)` manual page.

Procedure

View CephFS quotes in one of the following ways.

- Using a byte-limit quota.

```
getfattr -n ceph.quota.max_bytes DIRECTORY
```

For example:

```
[root@client ~]# getfattr -n ceph.quota.max_bytes /mnt/cephfs/
getfattr: Removing leading '/' from absolute path names
# file: mnt/cephfs/
ceph.quota.max_bytes="100000000"
```

In this example, `100000000` equals 100 MB.

- Using a file-limit quota.

```
getfattr -n ceph.quota.max_files DIRECTORY
```

For example:

```
[root@client ~]# getfattr -n ceph.quota.max_files /mnt/cephfs/
getfattr: Removing leading '/' from absolute path names
# file: mnt/cephfs/
ceph.quota.max_files="10000"
```

In this example, `10000` equals 10,000 files.

Setting quotas

Learn how to use the **`setfattr`** command and the `ceph.quota` extended attributes to set the quota for a directory.

About this task

For more information, see the `setfattr(1)` manual page.

Procedure

Set CephFS quotes in one of the following ways.

- Using a byte-limit quota.

> **Note:** The following values are supported for byte-limit quota: K, Ki, M, Mi, G, Gi, T, Ti.

```
setfattr -n ceph.quota.max_bytes -v LIMIT_VALUE DIRECTORY
```

For example:

```
[root@client ~]# setfattr -n ceph.quota.max_bytes -v 2T /mnt/cephfs/
```

- Using a file-limit quota.

```
setfattr -n ceph.quota.max_files -v LIMIT_VALUE DIRECTORY
```

For example:

```
[root@client ~]# setfattr -n ceph.quota.max_files -v 10000 /mnt/cephfs/
```

In this example, `10000` equals 10,000 files.

> **Note:** Only numerical values are supported for the file *LIMIT_VALUE*.

Removing quotas

Learn how to use the **`setfattr`** command and the `ceph.quota` extended attributes to remove a quota from a directory.

About this task

For more information, see the `setfattr(1)` manual page.

Procedure

Remove CephFS quotes in one of the following ways.

- Using a byte-limit quota.

  ```
  setfattr -n ceph.quota.max_bytes -v 0 DIRECTORY
  ```

  For example:

  ```
  [root@client ~]# setfattr -n ceph.quota.max_bytes -v 0 /mnt/cephfs/
  ```

  In this example, `100000000` equals 100 MB.

- Using a file-limit quota.

  ```
  setfattr -n ceph.quota.max_files -v 0 DIRECTORY
  ```

  For example:

  ```
  [root@client ~]# setfattr -n ceph.quota.max_files -v 0 /mnt/cephfs/
  ```

File and directory layouts

You can control how file or directory data is mapped to objects.

Be sure that you have the following when working with file and directory layouts:

- A running, and healthy Red Hat Ceph Storage cluster.
- Deployment of a Ceph File System.
- Root-level access to the node.
- The `attr` package installed.

A layout of a file or directory controls how its content is mapped to Ceph RADOS objects. The directory layouts serve primarily for setting an inherited layout for new files in that directory.

To view and set a file or directory layout, use virtual extended attributes or extended file attributes (`xattrs`). The name of the layout attributes depends on whether a file is a regular file or a directory:

- Regular files layout attributes are called `ceph.file.layout`.
- Directories layout attributes are called `ceph.dir.layout`.

For more information, see:

- “Deploying the CephFS” on page 1165
- The `getfattr(1)` manual page.

- The `setfattr(1)` manual page.

Layouts inheritance

Files inherit the layout of their parent directory when you create them. However, subsequent changes to the parent directory layout do not affect children. If a directory does not have any layouts set, files inherit the layout from the closest directory to the layout in the directory structure.

Setting file and directory layout fields

Use the **`setfattr`** command to set layout fields on a file or directory.

About this task

Important: When you modify the layout fields of a file, the file must be empty, otherwise an error occurs.

For more information, see the `setfattr(1)` manual page.

Procedure

- Modify layout fields on a file or directory.

```
setfattr -n ceph.TYPE.layout.FIELD -v VALUE PATH
```

Replace:

- *TYPE* with `file` or `dir`.
- *FIELD* with the name of the field.
- *VALUE* with the new value of the field.
- *PATH* with the path to the file or directory.

For example:

```
[root@mon ~]# setfattr -n ceph.file.layout.stripe_unit -v 1048576 test
```

Viewing file and directory layout fields

Use the **`getfattr`** command to view layout fields on a file or directory.

About this task

For more information, see the `getfattr(1)` manual page.

Procedure

- View layout fields on a file or directory as a single string.

Note: A directory does not have an explicit layout until you set it. Therefore, attempting to view the layout without first setting it fails because there are no changes to display.

```
getfattr -n ceph.TYPE.layout PATH
```

Replace:

- *TYPE* with `file` or `dir`.
- *PATH* with the path to the file or directory.

For example:

```
[root@mon ~]# getfattr -n ceph.dir.layout /home/test
ceph.dir.layout="stripe_unit=4194304 stripe_count=2 object_size=4194304
pool=cephfs_data"
```

Viewing individual layout fields

Use the **getfattr** command to view individual layout fields for a file or directory.

About this task

For more information, see the `getfattr(1)` manual page.

Procedure

– View individual layout fields on a file or directory.

```
getfattr -n ceph.TYPE.layout.FIELD PATH
```

Replace:

- *TYPE* with `file` or `dir`.
- *FIELD* with the name of the field.
- *PATH* with the path to the file or directory.

For example:

```
[root@mon ~]# getfattr -n ceph.file.layout.pool test
ceph.file.layout.pool="cephfs_data"
```

> **Note:** Pools in the `pool` field are indicated by name. However, newly created pools can be indicated by ID.

Removing directory layouts

Use the **setfattr** command to remove layouts from a directory.

About this task

> **Note:** When you set a file layout, you cannot change or remove it.

For more information, see the `setfattr(1)` manual page.

Procedure

1. Remove a layout from a directory.

   ```
   setfattr -x ceph.dir.layout DIRECTORY_PATH
   ```

   For example:

   ```
   [user@client ~]$ setfattr -x ceph.dir.layout /home/test
   ```

2. Remove the `pool_namespace` field.

   > **Note:** The `pool_namespace` field is the only field that you can remove separately.

```
setfattr -x ceph.dir.layout.pool_namespace DIRECTORY_PATH
```

For example:

```
[user@client ~]$ setfattr -x ceph.dir.layout.pool_namespace /home/test
```

CephFS snapshots

Use CephFS snapshots to create an immutable, point-in-time view of the file system. CephFS directory. CephFS snapshots are asynchronous, and you can choose which directory snapshots are created in.

Create snapshots in one of the following ways:

- With the Ceph Dashboard.
  For more information, see "Managing CephFS snapshots" on page 766.
- By invoking the **`mkdir`** command within the `.snap` hidden directory
- By using CLI commands.

Snapshots can also be automatically created at specific intervals through snapshot schedules and policies.

Before creating CephFS snapshots, be sure to have CephFS deployed and root-level access to the Ceph MetaData Server (MDS) node.

The Ceph File System (CephFS) snapshotting feature is enabled by default on new Ceph File Systems, but must be manually enabled on existing Ceph File Systems. CephFS snapshots are asynchronous and are kept in a special hidden directory in the CephFS directory named `.snap`. You can specify snapshot creation for any directory within a Ceph File System. When specifying a directory, the snapshot also includes all the subdirectories within it.

> **Warning:** Each MDS cluster allocates the snap identifiers independently. Using snapshots for multiple Ceph File Systems that are sharing a single pool causes snapshot collisions, and results in missing file data.

For more information, see "Deploying the CephFS" on page 1165 and "CephFS snapshot schedules" on page 1232.

CephFS snapshots usage

Use CephFS snapshots for data protection and disaster recovery.

Data protection

Snapshots allow users to recover files from specific points in time. It is useful for creating backups without disrupting ongoing operations.

Disaster recovery

Snapshots are used for CephFS mirroring which enables to replicate CephFS data asynchronously to another Ceph cluster in another location.

CephFS limitations

Before using CephFS, get to know the CephFS limitations.

Snapshot names

Snapshot names cannot start with an underscore (_) and cannot exceed 240 characters.

Performance impact

While creating snapshots is fast, the performance impact can be noticeable during heavy I/O operations due to the asynchronous nature of data flushing.

Creating a snapshot for a Ceph File System

You can create an immutable, point-in-time view of a Ceph File System (CephFS) by creating a snapshot.

About this task

> **Note:** For a new Ceph File System, snapshots are enabled by default.

Procedure

1. Log into the `cephadm` shell.

```
[root@mds ~]# cephadm shell
```

2. For existing Ceph File Systems, enable the snapshot feature.

```
ceph fs set FILE_SYSTEM_NAME allow_new_snaps true
```

For example:

```
[ceph: root@mds ~]# ceph fs set cephfs01 allow_new_snaps true
```

3. Create a snapshot subdirectory under the `.snap` directory.

```
mkdir NEW_DIRECTORY_PATH/.snap/SNAPNAME
```

The following example creates the `new-snaps` subdirectory, informing the Ceph Metadata Server (MDS) to start making snapshots.

```
[ceph: root@mds ~]# mkdir /directory1/.snap/new-snap1
```

Deleting snapshots

Procedure

> **Important:** Attempting to delete root-level snapshots, which might contain underlying snapshots, will fail.

– Delete snapshots.

```
rmdir NEW_DIRECTORY_PATH
```

For example:

```
[ceph: root@mds ~]# rmdir /.snap/new-snaps
```

Cloning subvolumes from snapshots

Create subvolumes by cloning subvolume snapshots, an asynchronous operation that copies data from a snapshot to a subvolume.

Before you begin

Be sure that you have a Ceph File System (CephFS) subvolume.
In addition to read (`r`) and write (`w`) capabilities, Ceph clients also require `s` flag on a directory path within the file system.

Verify that the `s` flag is set on the directory.

```
ceph auth get CLIENT_NAME
```

Output example:

```
CLIENT_NAME
    key = AQAz7EVWygILFRAAdIcuJ12opU/JKyfFmxhuaw==
    caps mds = allow rw, allow rws path=DIRECTORY_PATH
    caps mon = allow r
    caps osd = allow rw tag cephfs01 data=DIRECTORY_NAME
```

In the following example, `client.0` can create or delete snapshots in the `bar` directory of file system `cephfs_a`.

```
[root@mon ~]# ceph auth get client.0
[client.0]
    key = AQAz7EVWygILFRAAdIcuJ12opU/JKyfFmxhuaw==
    caps mds = "allow rw, allow rws path=/bar"
    caps mon = "allow r"
    caps osd = "allow rw tag cephfs data=cephfs_a"
```

About this task

For more information, see “Managing Ceph users” on page 447. To pause clone operations, for example during cluster recovery, see “Pausing asynchronous subvolume purge and clone operations” on page 1217.

> **Note:** Cloning is inefficient for very large data sets.

Procedure

1. Create a CephFS volume.
   This creates the CephFS file system, its data, and metadata pools.

   ```
   ceph fs volume create VOLUME_NAME
   ```

   For example,

   ```
   [root@mon ~]# ceph fs volume create cephfs01
   ```

2. Create a subvolume group.
   By default, the subvolume group is created with an octal file mode '755', and data pool layout of its parent directory.

   ```
   ceph fs subvolumegroup create VOLUME_NAME GROUP_NAME [--pool_layout DATA_POOL_NAME --
   uid UID --gid GID --mode OCTAL_MODE]
   ```

   For example,

   ```
   [root@mon ~]# ceph fs subvolumegroup create cephfs01 subgroup0
   ```

3. Create a subvolume.
   By default, a subvolume is created within the default subvolume group, and with an octal file mode ‘755’, uid of its subvolume group, gid of its subvolume group, data pool layout of its parent directory, and no size limit.

   > **Note:** The **`--group_name`** parameter is optional. If the subvolume needs to be created in a subvolume group, then **`--group_name`** needs to be passed in the command.

   ```
   ceph fs subvolume create VOLUME_NAME SUBVOLUME_NAME [--size SIZE_IN_BYTES --group_name
   SUBVOLUME_GROUP_NAME --pool_layout DATA_POOL_NAME --uid UID --gid GID --mode
   OCTAL_MODE]
   ```

   For example,

   ```
   [root@mon ~]# ceph fs subvolume create cephfs01 sub0 --group_name subgroup0
   ```

4. Create a snapshot of a subvolume.

```
ceph fs subvolume snapshot create VOLUME_NAME SUBVOLUME_NAME SNAP_NAME [--group_name
SUBVOLUME_GROUP_NAME]
```

For example,

```
[root@mon ~]# ceph fs subvolume snapshot create cephfs01 sub0 snap0 --group_name
subgroup0
```

5. Initiate a clone operation.

> **Note:** By default, cloned subvolumes are created in the default group.

   a. If the source subvolume and the target clone are in the default group, run the following command.

```
ceph fs subvolume snapshot clone VOLUME_NAME SUBVOLUME_NAME SNAP_NAME
TARGET_CLONE_NAME
```

      For example,

```
[root@mon ~]# ceph fs subvolume snapshot clone cephfs01 sub0 snap0 clone0
```

   b. If the source subvolume is in the non-default group, then specify the source subvolume group in the following command.

```
ceph fs subvolume snapshot clone VOLUME_NAME SUBVOLUME_NAME SNAP_NAME
TARGET_CLONE_NAME --group_name SUBVOLUME_GROUP_NAME
```

      For example,

```
[root@mon ~]# ceph fs subvolume snapshot clone cephfs01 sub0 snap0 clone0 --
group_name subgroup0
```

   c. If the target clone is to a non-default group, then specify the target group in the following command.

```
ceph fs subvolume snapshot clone VOLUME_NAME SUBVOLUME_NAME SNAP_NAME
TARGET_CLONE_NAME --target_group_name SUBVOLUME_GROUP_NAME
```

      For example,

```
[root@mon ~]# ceph fs subvolume snapshot clone cephfs01 sub0 snap0 clone0 --
target_group_name subgroup1
```

6. Check the status of the clone operation.

```
ceph fs clone status VOLUME_NAME CLONE_NAME [--group_name TARGET_GROUP_NAME]
```

For example,

```
[root@mon ~]# ceph fs clone status cephfs01 clone0 --group_name subgroup1

{
  "status": {
    "state": "complete"
  }
}
```

CephFS snapshot schedules

A Ceph File System (CephFS) can schedule snapshots of a file system directory. The scheduling of snapshots is managed by the Ceph Manager, and relies on Python Timers. The snapshot schedule data is stored as an object in the CephFS metadata pool, and at runtime, all the schedule data lives in a serialized SQLite database.

Important: The scheduler is precisely based on the specified time to keep snapshots apart when a storage cluster is under normal load. When the Ceph Manager is under a heavy load, it's possible that a snapshot might not get scheduled immediately, resulting in a slightly delayed snapshot. If this happens, then the next scheduled snapshot acts as if there was no delay. Scheduled snapshots that are delayed do not cause drift in the overall schedule.

Scheduling snapshots for a Ceph File System (CephFS) is managed by the `snap_schedule` Ceph Manager module. This module provides an interface to add, query, and delete snapshot schedules, and to manage the retention policies. This module also implements the **`ceph fs snap-schedule`** command, with several subcommands to manage schedules, and retention policies. All subcommands take the CephFS volume path and subvolume path arguments to specify the file system path when using multiple Ceph File Systems. Not specifying the CephFS volume path, the argument defaults to the first file system listed in the `fs_map`, and not specifying the subvolume path argument defaults to nothing.

Snapshot schedules are identified by the file system path, the repeat interval, and the start time. The repeat interval defines the time between two subsequent snapshots. The interval format is a number plus a time designator: `h`(our), `d`(ay), or `w`(eek). For example, having an interval of `4h`, means one snapshot every four hours. The start time is a string value in the ISO format, `%Y-%m-%dT%H:%M:%S`, and if not specified, the start time uses a default value of last midnight. For example, you schedule a snapshot at `14:45`, using the default start time value, with a repeat interval of `1h`, the first snapshot will be taken at 15:00.

Retention policies are identified by the file system path, and the retention policy specifications. Defining a retention policy consist of either a number plus a time designator or a concatenated pair in the format of *`COUNTTIME_PERIOD`*. The policy ensures that a number of snapshots are kept, and the snapshots are at least for a specified time period apart. The time period designators are: `h`(our), `d`(ay), `w`(eek), `M`(onth), `y`(ear), and `n`. The `n` time period designator is a special modifier, which means to keep the last number of snapshots regardless of timing. For example, `4d` means keeping four snapshots that are at least one day, or longer apart from each other.

For more information about snapshots, see "CephFS snapshots" on page 1229.

Adding a snapshot schedule for a CephFS

Add a snapshot schedule for a CephFS path that does not exist yet. You can create one or more schedules for a single path. Schedules are considered different, if their repeat interval and start times are different.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running, and healthy Red Hat Ceph Storage cluster.
- Deployment of a Ceph File System.
- Root-level access to a Ceph Manager and Metadata Server (MDS) nodes.
- CephFS snapshots enabled on the file system.

About this task

A CephFS path can have only one retention policy, but a retention policy can have multiple count-time period pairs.

Note: Once the scheduler module is enabled, running the **`ceph fs snap-schedule`** command displays the available subcommands and their usage format.

Procedure

1. Log in to the `cephadm` shell on a Ceph Manager node.
   For example,

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Enable the `snap_schedule` module.

For example,

```
[ceph: root@host01 ~]# ceph mgr module enable snap_schedule
```

3. Log in to the `cephadm` shell on a Ceph Metadata Server node.
For example,

```
[root@host02 ~]# cephadm shell
```

4. Add a schedule for a Ceph File System.

> **Note:** *START_TIME* is represented in ISO 8601 format.

```
ceph fs snap-schedule add FILE_SYSTEM_VOLUME_PATH REPEAT_INTERVAL [START_TIME] --fs
CEPH_FILE_SYSTEM_NAME
```

In the following example, a snapshot schedule is created for the CephFS `/cephfs` volume, snapshotting every hour, and starts on 27 June 2022 9:50 PM.

```
[ceph: root@host02 ~]# ceph fs snap-schedule add /cephfs_kernelf739cwtus2/pmo9axbwsi
1h 2022-06-27T21:50:00 --fs cephfs01
```

5. Add a retention policy for snapshots of a CephFS volume path.

> **Note:**
> Snapshot retention counts are limited by the **`mds_max_snaps_per_dir`** configuration option. The snapshot scheduler can keep at most `mds_max_snaps_per_dir - 1` scheduled snapshots in a directory. With the default value of `100`, the maximum number of scheduled snapshots is `99`.
>
> If your retention policy requires more snapshots than the configured limit, increase the value before creating or updating the retention policy. For example, to retain 120 snapshots, set **`mds_max_snaps_per_dir`** to at least `121`.

```
ceph config set mds mds_max_snaps_per_dir VALUE

ceph fs snap-schedule retention add FILE_SYSTEM_VOLUME_PATH [COUNT_TIME_PERIOD_PAIR]
TIME_PERIOD COUNT
```

For example:

```
[ceph: root@host02 ~]# ceph config set mds mds_max_snaps_per_dir 121
[ceph: root@host02 ~]# ceph fs snap-schedule retention add /cephfs h 14 (1)
[ceph: root@host02 ~]# ceph fs snap-schedule retention add /cephfs d 4 (2)
[ceph: root@host02 ~]# ceph fs snap-schedule retention add /cephfs 14h4w (3)
```

(1) This example keeps 14 snapshots at least one hour apart.
(2) This example keeps 4 snapshots at least one day apart.
(3) This example keeps up to 18 snapshots: 14 hourly snapshots and 4 weekly snapshots.

6. List the snapshot schedules to verify that the new schedule is created.

```
ceph fs snap-schedule list FILE_SYSTEM_VOLUME_PATH [--format=plain|json] [--
recursive=true]
```

The following example lists all schedules in the directory tree.

```
[ceph: root@host02 ~]# ceph fs snap-schedule list /cephfs --recursive=true
```

7. Check the status of a snapshot schedule.

```
ceph fs snap-schedule status FILE_SYSTEM_VOLUME_PATH [--format=plain|json]
```

The following example displays the status of the snapshot schedule for the CephFS `/cephfs` path in JSON format. If not specified, the default format is plain text.

```
[ceph: root@host02 ~]# ceph fs snap-schedule status /cephfs --format=json
```

Adding a snapshot schedule for CephFS subvolume

Create snapshot schedules and retention policies for directories in a subvolume, subvolume within a default group, and for subvolumes within a non-default group.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running and healthy Red Hat Ceph Storage cluster with Ceph File System (CephFS) deployed.
- A minimum of read access on the Ceph Monitor.
- Read and write capability on the Ceph Manager nodes.
- A CephFS subvolume and subvolume group created.

About this task

Schedules are considered different, if their repeat interval and start times are different.

Add a snapshot schedule for a CephFS file path that does not exist yet. A CephFS path can have only one retention policy, but a retention policy can have multiple count-time period pairs.

Note:

- After the scheduler module is enabled, running the **`ceph fs snap-schedule`** command displays the available subcommands and their usage format.
- Subvolumes belonging to the default subvolume group and non-default subvolume group can be scheduled for creating snapshots. However, the commands are different.

Procedure

You can add snapshot schedules to the directories within a subvolume, subvolume in the default group and to the subvolume in a non-default group.

- Add a snapshot schedule to a directory in a subvolume.
  - Get the absolute path of the subvolume where directory exists:
    Syntax:

    ```
    ceph fs subvolume getpath VOLUME_NAME SUBVOLUME_NAME --group-
    name SUBVOLUME_GROUP_NAME
    ```

    Example:

    ```
    [ceph: root@host02 /]# ceph fs subvolume getpath cephfs01 sub0 --group_name
    subgroup0

    /volumes/subgroup0/sub0/db992be0-aacd-414e-8abd-e1bfeb37a4a2
    ```

  - Add a snapshot schedule to a directory in the subvolume:

    Note: *START_TIME* is represented in ISO 8601 format.

> **Note:** Path in snap-schedule command is <absolute_path_of_ subvolume>/<relative_path_of_test_dir>. Get the absolute_path of the subvolume, from step "1" on page 1235.

Syntax:

```
ceph fs snap-schedule add SUBVOLUME_DIR_PATH SNAP_SCHEDULE [START_TIME] --
fs CEPH_FILE_SYSTEM_NAME
```

In this example, test_dir is the directory for which the schedule is added.

```
[ceph: root@host02 /]# ceph fs snap-schedule add /volumes/subgroup0/
sub0/4dee0788-32a0-47e6-ac4b-bd8568a6989b/test_dir 1h --fs cephfs01

Schedule set for path /volumes/subgroup0/sub0/4dee0788-32a0-47e6-ac4b-
bd8568a6989b/test_dir
```

- Add a snapshot schedule for a CephFS subvolume in the default group:
  Syntax:

```
ceph fs snap-schedule add /.. SNAP_SCHEDULE [START_TIME] --fs CEPH_FILE_SYSTEM_NAME --
subvol SUBVOLUME_NAME
```

> **Important:** The path (/) must be defined and cannot be left empty. There is no dependency on path string value, you can define it with one of the following characters:
> / - /..

In this example, a snapshot schedule for the CephFS subvolume sub0 volume is created, taking snapshots every hour, and starts on 27 June 2022 9:50 PM.

```
[ceph: root@host02 /]# ceph fs snap-schedule add / 1h 2022-06-27T21:50:00 --fs cephfs01
--subvol sub0
```

- Add a snapshot schedule for a CephFS subvolume in the non-default group:
  Syntax:

```
ceph fs snap-schedule add /.. SNAP_SCHEDULE [START_TIME] --fs CEPH_FILE_SYSTEM_NAME  --
subvol SUBVOLUME_NAME --group NON_DEFAULT_SUBVOLGROUP_NAME
```

> **Important:** The path (/) must be defined and cannot be left empty. There is no dependency on path string value, you can define it with one of the following characters:
> / - /..

In this example, a snapshot schedule is added for the Ceph subvolume sv_non_def_1 belonging to the non default group subgroup0.

```
[ceph: root@host02 /]# ceph fs snap-schedule add / 2M --subvol sub0 --group subgroup0
```

Adding retention policy for snapshot schedules of a CephFS volume path

Before you begin

Before adding a retention policy, you must have a snapshot schedule in place.

About this task

To define how many snapshots to retain in the volume path at any time, you must add a retention policy after you have created a snapshot schedule.

You can create retention policies for directories within a subvolume, subvolume in a default group and to the subvolume in a non-default group.

Procedure

- Add a retention policy to a snapshot schedule created for a directory in a subvolume.

```
ceph fs snap-schedule retention
add SUBVOLUME_DIR_PATH [COUNT_TIME_PERIOD_PAIR] TIME_PERIOD COUNT
```

  For example,

```
[ceph: root@host02 /]# ceph fs snap-schedule retention add /volumes/subvolgroup_1/
subvol_1 h 14 (1)
[ceph: root@host02 /]# ceph fs snap-schedule retention add /volumes/subvolgroup_1/
subvol_1 d 4 (2)
[ceph: root@host02 /]# ceph fs snap-schedule retention add /volumes/subvolgroup_1/
subvol_1 14h4w (3)
```

  (1) This example keeps 14 snapshots at least one hour apart.
  (2) This example keeps 4 snapshots at least one day apart.
  (3) This example keeps 14 hourly, and 4 weekly snapshots.

- Add a retention policy to a snapshot schedule created for a subvolume in the default group.

```
ceph fs snap-schedule retention add / [COUNT_TIME_PERIOD_PAIR] TIME_PERIOD COUNT --
fs CEPH_FILE_SYSTEM_NAME --subvol SUBVOLUME_NAME
```

  Important: The `path ( / )` must be defined and cannot be left empty. There is no dependency on `path` string value, you can define it with one of the following characters:
  / - /..

  For example,

```
[ceph: root@host02 /]# ceph fs snap-schedule retention add / 5h --fs cephfs --subvol
sv_sched
Retention added to path /volumes/sv_sched/e704342a-ff07-4763-bb0b-a46d9dda6f27/..
```

- Add a retention policy to the snapshot schedule created for a subvolume group in the non-default group.

```
ceph fs snap-schedule retention add / [COUNT_TIME_PERIOD_PAIR] TIME_PERIOD COUNT --
fs CEPH_FILE_SYSTEM_NAME --subvol SUBVOLUME_NAME --group NON_DEFAULT_SUBVOLGROUP_NAME
```

  Important: The `path ( / )` must be defined and cannot be left empty. There is no dependency on `path` string value, you can define it with one of the following characters:
  / - /..

  For example,

```
[ceph: root@host02 /]# ceph fs snap-schedule retention add / 5h --fs cephfs --subvol
sv_sched --group subvolgroup_cg
Retention added to path /volumes/subvolgroup_cg/sv_sched/e704342a-ff07-4763-bb0b-
a54j0dda7f16/..
```

Listing CephFS snapshot schedules

Before you begin

Before listing CephFS snapshots schedules, be sure that you have both read and write capability on the Ceph Manager nodes.

About this task

By listing and adhering to snapshot schedules, you can ensure robust data protection and efficient management.

Before you begin

Before listing CephFS snapshots schedules, be sure that you have both read and write capability on the Ceph Manager nodes.

Procedure

- List the snapshot schedules.

```
ceph fs snap-schedule list SUBVOLUME_PATH [--format=plain|json] [--recursive=true]
```

  The following example lists all schedules in the directory tree.

```
[ceph: root@host02 /]# ceph fs snap-schedule list /volumes/subgroup0/
sub0/4dee0788-32a0-47e6-ac4b-bd8568a6989b/test_dir --recursive=true

/volumes/subgroup0/sub0/4dee0788-32a0-47e6-ac4b-bd8568a6989b/test_dir 1h
```

Checking status of CephFS snapshot schedules

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Read and write capability on the Ceph Manager nodes.
- A snapshot schedule.

About this task

You can check the status of snapshot schedules using the command in the procedure for snapshots created in a directory of a subvolume, for a subvolume in the default subvolume group and for a subvolume created in a non-default group.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Read and write capability on the Ceph Manager nodes.
- A snapshot schedule.

Procedure

- Check the status of a snapshot schedule created for a directory in a subvolume:

```
ceph fs snap-schedule status SUBVOLUME_DIR_PATH [--format=plain|json]
```

  The following example displays the status of the snapshot schedule for the `/volumes/subvolgroup_1/subvol_1` path in JSON format. If not specified, the default format is plain text.

```
/volumes/subgroup0/sub0/4dee0788-32a0-47e6-ac4b-bd8568a6989b/test_dir

{"fs": "cephfs01", "subvol": null, "group": null, "path": "/volumes/subgroup0/
sub0/4dee0788-32a0-47e6-ac4b-bd8568a6989b/test_dir", "rel_path": "/volumes/subgroup0/
sub0/4dee0788-32a0-47e6-ac4b-bd8568a6989b/test_dir", "schedule": "1h", "retention": {},
"start": "2025-01-09T00:00:00", "created": "2025-01-09T10:00:29", "first": null,
"last": null, "last_pruned": null, "created_count": 0, "pruned_count": 0, "active":
true}
```

- Check the status of the snapshot schedule created for a subvolume in the default group:

```
ceph fs snap-schedule status --fs CEPH_FILE_SYSTEM_NAME --subvol SUBVOLUME_NAME
```

> **Important:** The `path ( / )` must be defined and cannot be left empty. There is no dependency on `path` string value, you can define it with one of the following characters:
> / - /..

For example,

```
[ceph: root@host02 /]# ceph fs snap-schedule status --fs cephfs --subvol sv_sched
{"fs": "cephfs", "subvol": "sv_sched", "group": "subvolgroup_cg", "path": "/volumes/
subvolgroup_cg/sv_sched/e704342a-ff07-4763-bb0b-a46d9dda6f27/..", "rel_path": "/
volumes/subvolgroup_cg/sv_sched/e704342a-ff07-4763-bb0b-a46d9dda6f27/..", "schedule":
"1h", "retention": {"h": 5}, "start": "2024-05-21T00:00:00", "created":
"2024-05-21T09:18:58", "first": null, "last": null, "last_pruned": null,
"created_count": 0, "pruned_count": 0, "active": true}proc_cephfs_adding-snapshot-
schedule-of-a-file-system-subvolumeproc_cephfs_adding-snapshot-schedule-of-a-file-
system-subvolumeproc_cephfs_adding-snapshot-schedule-of-a-file-system-subvolume
```

- Check the status of the snapshot schedule created for a subvolume in the non-default group:

```
ceph fs snap-schedule status --fs CEPH_FILE_SYSTEM_NAME --subvol SUBVOLUME_NAME --
group NON_DEFAULT_SUBVOLGROUP_NAME
```

> **Important:** The `path ( / )` must be defined and cannot be left empty. There is no dependency on `path` string value, you can define it with one of the following characters:
> / - /..

For example,

```
[ceph: root@host02 /]# ceph fs snap-schedule status --fs cephfs --subvol sv_sched --
group subvolgroup_cg
{"fs": "cephfs", "subvol": "sv_sched", "group": "subvolgroup_cg", "path": "/volumes/
subvolgroup_cg/sv_sched/e564329a-kj87-4763-gh0y-b56c8sev7t23/..", "rel_path": "/
volumes/subvolgroup_cg/sv_sched/e704342a-ff07-4763-bb0b-a46d9dda6f27/..", "schedule":
"1h", "retention": {"h": 5}, "start": "2024-05-21T00:00:00", "created":
"2024-05-21T09:18:58", "first": null, "last": null, "last_pruned": null,
"created_count": 0, "pruned_count": 0, "active": true}
```

Activating snapshot schedule for a CephFS

Learn how to manually set the snapshot schedule to active for a Ceph File System (CephFS).

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running, and healthy Red Hat Ceph Storage cluster with Ceph File System (CephFS) deployed.
- A minimum of read access on the Ceph Monitor.
- Read and write capability on the Ceph Manager nodes.

Procedure

- Activate the snapshot schedule.

```
ceph fs snap-schedule activate FILE_SYSTEM_VOLUME_PATH [REPEAT_INTERVAL]
```

In the following example, all schedules for the CephFS `/cephfs` path are activated.

```
[ceph: root@host01 ~]# ceph fs snap-schedule activate /cephfs
```

What to do next

Verify that the snapshot schedule is activated, by using the **`snap-schedule status`** command. For more information, see “Checking status of CephFS snapshot schedules” on page 1238.

Activating snapshot schedule for a CephFS subvolume

Learn how to manually set the snapshot schedule to active for a Ceph File System (CephFS) sub volume.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A working Red Hat Ceph Storage cluster with a Ceph File System (CephFS) deployed.
- A minimum of read access on the Ceph Monitor.
- Read and write capability on the Ceph Manager nodes.

Procedure

- Activate the snapshot schedule created for a directory in a subvolume group.

```
ceph fs snap-schedule activate SUB_VOLUME_PATH REPEAT_INTERVAL
```

  In the following example, all schedules for the CephFS `/volumes/_nogroup/subvol_1/85a615da-e8fa-46c1-afc3-0eb8ae64a954/..` path are activated.

```
[ceph: root@host01 ~]# ceph fs snap-schedule activate /volumes/_nogroup/
subvol_1/85a615da-e8fa-46c1-afc3-0eb8ae64a954/..
```

- Activate the snapshot schedule created for a subvolume in the default group.

```
ceph fs snap-schedule activate /.. REPEAT_INTERVAL --fs CEPH_FILE_SYSTEM_NAME --
subvol SUBVOLUME_NAME
```

  Important: The `path ( / )` must be defined and cannot be left empty. There is no dependency on `path` string value, you can define it with one of the following characters:
  / - /..

  For example,

```
[ceph: root@host02 /]# ceph fs snap-schedule activate / --fs cephfs --subvol sv_sched
Schedule activated for path /volumes/subvolgroup_cg/sv_sched/e704342a-ff07-4763-bb0b-
a46d9dda6f27/..
```

- Activate the snapshot schedule created for a subvolume in the non-default group:

```
ceph fs snap-schedule activate /.. [_REPEAT_INTERVAL_] --fs CEPH_FILE_SYSTEM_NAME --
subvol SUBVOLUME_NAME --group NON_DEFAULT_GROUP_NAME
```

  Important: The `path ( / )` must be defined and cannot be left empty. There is no dependency on `path` string value, you can define it with one of the following characters:
  / - /..

  For example,

```
[ceph: root@host02 /]# ceph fs snap-schedule activate / --fs cephfs --subvol sv_sched
--group subvolgroup_cg
Schedule activated for path /volumes/subvolgroup_cg/sv_sched/e704342a-ff07-4763-bb0b-
a46d9dda6f27/..
```

What to do next

Verify that the snapshot schedule is activated, by using the **`snap-schedule status`** command. For more information, see “Checking status of CephFS snapshot schedules” on page 1238.

Deactivating snapshot schedule for a CephFS

Learn how to manually set the snapshot schedule to inactive for a Ceph File System (CephFS). This action excludes the snapshot from scheduling until it is activated again.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A working Red Hat Ceph Storage cluster with a Ceph File System (CephFS) deployed.
- Minimum of read access on the Ceph Monitor.
- Read and write capability on the Ceph Manager nodes.
- A snapshot schedule that is created and in active state.

Procedure

- Deactivate a snapshot schedule for a CephFS path.

```
ceph fs snap-schedule deactivate FILE_SYSTEM_VOLUME_PATH [REPEAT_INTERVAL]
```

This example deactivates the daily snapshots for the `/cephfs` path, pausing any further snapshot creation.

```
[ceph: root@host02 ~]# ceph fs snap-schedule deactivate /cephfs 1d
```

What to do next

Verify that the snapshot schedule is deactivated, by using the **`snap-schedule status`** command. For more information, see “Checking status of CephFS snapshot schedules” on page 1238.

Deactivating snapshot schedule for a CephFS subvolume

Learn how to manually set the snapshot schedule to inactive for a Ceph File System (CephFS) subvolume. This action will exclude the snapshot from scheduling until it is activated again.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A working Red Hat Ceph Storage cluster with a Ceph File System (CephFS) deployed.
- Minimum of read access on the Ceph Monitor.
- Read and write capability on the Ceph Manager nodes.
- A snapshot schedule that is created and in active state.

Procedure

- Deactivate a snapshot schedule for a CephFS directory in the subvolume.

```
ceph fs snap-schedule deactivate SUBVOLUME_DIR_PATH REPEAT_INTERVAL
```

> **Important:** The `path ( / )` must be defined and cannot be left empty. There is no dependency on `path` string value, you can define it with one of the following characters:
> / - /..

The following example deactivates the daily snapshots for the `/volumes/_nogroup/subvol_1/85a615da-e8fa-46c1-afc3-0eb8ae64a954/..` path, pausing any further snapshot creation.

```
[ceph: root@host02 ~]# ceph fs snap-schedule deactivate /volumes/_nogroup/
subvol_1/85a615da-e8fa-46c1-afc3-0eb8ae64a954/.. 1d
```

- Deactivate the snapshot schedule created for a subvolume in a default group.

```
ceph fs snap-schedule deactivate / REPEAT_INTERVAL --fs CEPH_FILE_SYSTEM_NAME --
subvol SUBVOLUME_NAME
```

> **Important:** The `path ( / )` must be defined and cannot be left empty. There is no dependency on `path` string value, you can define it with one of the following characters:
> / - /..

For example,

```
[ceph: root@host02 ~]# ceph fs snap-schedule deactivate / --fs cephfs --subvol
sv_sched
Schedule deactivated for path /volumes/subvolgroup_cg/sv_sched/e704342a-ff07-4763-bb0b-
a46d9dda6f27/..
```

- Deactivate the snapshot schedule created for a subvolume in the non-default group.

```
ceph fs snap-schedule deactivate / REPEAT_INTERVAL --fs CEPH_FILE_SYSTEM_NAME --
subvol SUBVOLUME_NAME --group NON-DEFAULT_GROUP_NAME
```

> **Important:** The `path ( / )` must be defined and cannot be left empty. There is no dependency on `path` string value, you can define it with one of the following characters:
> / - /..

For example,

```
[ceph: root@host02 ~]# ceph fs snap-schedule deactivate / --fs cephfs --subvol sv_sched
--group subvolgroup_cg
Schedule deactivated for path /volumes/subvolgroup_cg/sv_sched/e704342a-ff07-4763-bb0b-
a46d9dda6f27/..
```

What to do next

Verify that the snapshot schedule is deactivated, by using the **`snap-schedule status`** command. For more information, see "Checking status of CephFS snapshot schedules" on page 1238.

Removing a snapshot schedule for a CephFS

Learn how to remove snapshot schedule of a Ceph File System (CephFS).

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A working Red Hat Ceph Storage cluster with a Ceph File System (CephFS) deployed.
- Minimum of read access on the Ceph Monitor.
- Read and write capability on the Ceph Manager nodes.
- A snapshot schedule that is created.

Procedure

1. Remove a specific snapshot schedule.

```
ceph fs snap-schedule remove FILE_SYSTEM_VOLUME_PATH [REPEAT_INTERVAL] [START_TIME]
```

This example removes the specific snapshot schedule for the `/cephfs` volume that is snapshotting every four hours and that started on 16 May 2023 2:00 PM.

```
[ceph: root@host02 ~]# ceph fs snap-schedule remove /cephfs 4h 2023-05-16T14:00:00
```

2. Remove all snapshot schedules for a specific CephFS volume.

```
ceph fs snap-schedule remove FILE_SYSTEM_VOLUME_PATH
```

This example removes all the snapshot schedules for the /cephfs volume.

```
[ceph: root@host02 ~]# ceph fs snap-schedule remove /cephfs
```

What to do next

Validate the snapshot schedule, by using the **snap-schedule list** command. For more information, see "Listing CephFS snapshot schedules" on page 1237.

Removing a snapshot schedule for a CephFS subvolume

Learn how to remove snapshot schedule of a Ceph File System (CephFS) subvolume.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A working Red Hat Ceph Storage cluster with a Ceph File System (CephFS) deployed.
- Minimum of read access on the Ceph Monitor.
- Read and write capability on the Ceph Manager nodes.
- A snapshot schedule.

Procedure

Remove specific snapshots schedules in any of the following ways.

- Remove a specific snapshot schedule created for a directory in the subvolume.

```
ceph fs snap-schedule remove SUBVOLUME_DIR_PATH REPEAT_INTERVAL START_TIME
```

The following example removes the specific snapshot schedule for the /volumes/_nogroup/subvol_1/85a615da-e8fa-46c1-afc3-0eb8ae64a954/.. volume that is taking snapshots every four hours and started on 16 May 2022 2:00 PM.

```
[ceph: root@host02 ~]# ceph fs snap-schedule remove /volumes/_nogroup/
subvol_1/85a615da-e8fa-46c1-afc3-0eb8ae64a954/.. 4h 2022-05-16T14:00:00
```

- Remove a specific snapshot schedule created for a subvolume in a default group.

```
ceph fs snap-schedule remove / --fs CEPH_FILESYSTEM_NAME --subvol SUBVOLUME_NAME
```

> **Important:** The path (/) must be defined and cannot be left empty. There is no dependency on path string value, you can define it with one of the following characters:
> / - /..

For example,

```
[ceph: root@host02 ~]# ceph fs snap-schedule remove / --fs cephfs --subvol sv_sched
Schedule removed for path /volumes/subvolgroup_cg/sv_sched/e704342a-ff07-4763-bb0b-
a46d9dda6f27/..
```

- Remove a specific snapshot schedule created for a subvolume in a non-default group.

```
ceph fs snap-schedule remove / --fs CEPH_FILESYSTEM_NAME --subvol SUBVOLUME_NAME --
group NON-DEFAULT_GROUP_NAME
```

> **Important:** The `path ( / )` must be defined and cannot be left empty. There is no dependency on `path` string value, you can define it with one of the following characters:
> / - /..

For example,

```
[ceph: root@host02 ~]# ceph fs snap-schedule remove / --fs cephfs --subvol sv_sched --
group subvolgroup_cg
Schedule removed for path /volumes/subvolgroup_cg/sv_sched/e704342a-ff07-4763-bb0b-
a46d9dda6f27/..
```

What to do next

Validate the snapshot schedule, by using the **`snap-schedule list`** command. For more information, see "Listing CephFS snapshot schedules" on page 1237.

Removing snapshot schedule retention policy for a CephFS

Learn how to remove the retention policy of the scheduled snapshots for a Ceph File System (CephFS).

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A working Red Hat Ceph Storage cluster with a Ceph File System (CephFS) deployed.
- Minimum of read access on the Ceph Monitor.
- Read and write capability on the Ceph Manager nodes.
- A snapshot schedule that is created for a CephFS volume path.

Procedure

- Remove a retention policy on a CephFS path.

```
ceph fs snap-schedule retention
remove FILE_SYSTEM_VOLUME_PATH [COUNT_TIME_PERIOD_PAIR] TIME_PERIOD COUNT
```

For example,

```
[ceph: root@host02 ~]# ceph fs snap-schedule retention remove /cephfs h 4 (1)
[ceph: root@host02 ~]# ceph fs snap-schedule retention remove /cephfs 14d4w (2)
```

(1) This example removes 4 hourly snapshots.
(2) This example removes 14 daily and 4 weekly snapshots.

What to do next

Verify the retention policy, by using the **`snap-schedule status`** command. For more information, see "Checking status of CephFS snapshot schedules" on page 1238.

Removing snapshot schedule retention policy for a CephFS subvolume

Learn how to remove the retention policy of the scheduled snapshots for a Ceph File System (CephFS) subvolume.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A working Red Hat Ceph Storage cluster with a Ceph File System (CephFS) deployed.
- Minimum of read access on the Ceph Monitor.
- Read and write capability on the Ceph Manager nodes.

– A snapshot schedule created either for a CephFS subvolume directory or a subvolume in default group or a subvolume in non-default group.

Procedure

Remove retention policy in any of the following ways.

– Remove a retention policy created on snapshot schedule for a directory in the subvolume.

```
ceph fs snap-schedule retention
remove SUBVOLUME_DIR_PATH TIME_PERIOD_PAIR TIME_PERIOD COUNT
```

For example,

```
[ceph: root@host02 ~]# ceph fs snap-schedule retention remove /volumes/_nogroup/
subvol_1/85a615da-e8fa-46c1-afc3-0eb8ae64a954/.. h 4 (1)
[ceph: root@host02 ~]# ceph fs snap-schedule retention remove /volumes/_nogroup/
subvol_1/85a615da-e8fa-46c1-afc3-0eb8ae64a954/.. 14d4w (2)
```

(1) This example removes 4 hourly snapshots.
(2) This example removes 14 daily and 4 weekly snapshots.

– Remove a retention policy created on snapshot schedule for a subvolume in a default group.

```
ceph fs snap-schedule retention remove / TIME_PERIOD_PAIR TIME_PERIOD COUNT --
fs CEPH_FILESYSTEM_NAME --subvol SUBVOLUME_NAME
```

> **Important:** The `path ( / )` must be defined and cannot be left empty. There is no dependency on `path` string value, you can define it with one of the following characters:
> / - /..

For example,

```
[ceph: root@host02 ~]# ceph fs snap-schedule retention remove / 5h --fs cephfs --subvol
sv_sched --group subvolgroup_cg
Retention removed from path /volumes/subvolgroup_cg/sv_sched/e704342a-ff07-4763-bb0b-
a46d9dda6f27/..
```

– Remove a retention policy created on snapshot schedule for a subvolume in a non-default group.

```
ceph fs snap-schedule retention remove / TIME_PERIOD_PAIR TIME_PERIOD COUNT --
fs CEPH_FILESYSTEM_NAME --subvol SUBVOLUME_NAME --group NON-DEFAULT_GROUP_NAME
```

> **Important:** The `path ( / )` must be defined and cannot be left empty. There is no dependency on `path` string value, you can define it with one of the following characters:
> / - /..

For example,

```
[ceph: root@host02 ~]# ceph fs snap-schedule retention remove / 5h --fs cephfs --subvol
sv_sched --group subvolgroup_cg
Retention removed from path /volumes/subvolgroup_cg/sv_sched/e704342a-ff07-4763-bb0b-
a46d9dda6f27/..
```

What to do next

Verify the retention policy, by using the **`snap-schedule status`** command. For more information, see “Checking status of CephFS snapshot schedules” on page 1238.

CephFS snapshot mirroring (Technology Preview)

You can replicate a Ceph File System (CephFS) to a remote Ceph File System on another Red Hat Ceph Storage cluster. A Ceph File System supports asynchronous replication of snapshots directories.

Important: Technology Preview features are not supported with production service level agreements (SLAs), might not be functionally complete, and does not recommend using them for production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

Note: CephFS mirroring is currently available as a Technology Preview feature. In production environments, use other synchronization methods, such as `rsync`. You can combine these methods with periodic CephFS snapshots to help ensure data consistency. For more information, see Recovering a Ceph Monitor when using BlueStore.

To use CephFS snapshot mirroring, both the source and the target storage clusters must be running Red Hat Ceph Storage 7.0 or later.

The Ceph File System (CephFS) supports asynchronous replication of snapshots to a remote CephFS on another Red Hat Ceph Storage cluster. Snapshot synchronization copies snapshot data to a remote Ceph File System, and creates a new snapshot on the remote target with the same name. You can configure specific directories for snapshot synchronization.

Managing CephFS snapshot mirroring is done by the CephFS mirroring daemon (`cephfs-mirror`). This snapshot data is synchronized by doing a bulk copy to the remote CephFS. The chosen order of synchronizing snapshot pairs is based on the creation using the `snap-id`.

Important: Synchronizing hard links is not supported. Hard linked files get synchronized as regular files.

The CephFS snapshot mirroring includes features, for example snapshot incarnation or high availability. These can be managed through Ceph Manager `mirroring` module, which is the recommended control interface.

Ceph Manager Module and interfaces

The Ceph Manager `mirroring` module is disabled by default. It provides interfaces for managing mirroring of directory snapshots. Ceph Manager interfaces are mostly wrappers around monitor commands for managing CephFS mirroring. They are the recommended control interface.

The Ceph Manager `mirroring` module is implemented as a Ceph Manager plugin. It is responsible for assigning directories to the `cephfs-mirror` daemons for synchronization.

The Ceph Manager `mirroring` module also provides a family of commands to control mirroring of directory snapshots. The `mirroring` module does not manage the `cephfs-mirror` daemons. The stopping, starting, restarting, and enabling of the `cephfs-mirror` daemons is controlled by `systemctl`, but managed by `cephadm`.

Note: Mirroring module commands use the `fs snapshot mirror` prefix as compared to the monitor commands with the `fs mirror` prefix. Assure that you are using the module command prefix to control the mirroring of directory snapshots.

Snapshot incarnation

A snapshot might be deleted and created again with the same name and different content. The user could synchronize an "old" snapshot earlier and create the snapshot again when the mirroring was disabled. Using snapshot names to infer the point-of-continuation results in the “new” snapshot, an incarnation, never getting picked up for synchronization.

Snapshots on the secondary file system store the `snap-id` of the snapshot it was synchronized from. This metadata is stored in the `SnapInfo` structure on the Ceph Metadata Server.

High availability

You can deploy multiple `cephfs-mirror` daemons on two or more nodes to achieve concurrency in synchronization of directory snapshots. When `cephfs-mirror` daemons are deployed or terminated, the Ceph Manager `mirroring` module discovers the modified set of `cephfs-mirror` daemons and rebalances the directory assignment amongst the new set thus providing high availability.

`cephfs-mirror` daemons share the synchronization load by using a simple M/N policy, where M is the number of directories and N is the number of `cephfs-mirror` daemons.

Readdition of Ceph File System mirror peers

When readding or reassigning a peer to a CephFS in another cluster, make sure that all mirror daemons stopped synchronization to the peer. You can verify readdition of CephFS mirror peers with the **`fs mirror status`** command. The Peer UUID does not show up in the command output.

Purge synchronized directories from the peer before readding it to another CephFS, especially those directories that might exist in the new primary file system. Purging synchronized directories is not required if you are readding a peer to the same primary file system it was earlier synchronized from.

For more information about the **`fs mirror status`** command, see "Viewing the mirror status for a Ceph File System" on page 1250.

Configuring a snapshot mirror for a Ceph File System

Learn how to configure a Ceph File System (CephFS) for mirroring to replicate snapshots to another CephFS on a remote Red Hat Ceph Storage cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- The source and the target storage clusters must be healthy and running the latest Red Hat Ceph Storage version.
- Root-level access to a Ceph Monitor node in the source and the target storage clusters.
- At least one CephFS deployed on your storage cluster.

About this task

The time taken for synchronizing to a remote storage cluster depends on the file size and the total number of files in the mirroring path.

Procedure

1. Log into the `cephadm` shell.
   For example,

   ```
   [root@host01 ~]# cephadm shell
   ```

2. On the source storage cluster, deploy the CephFS mirroring daemon.
   This command creates a Ceph user called `cephfs-mirror` and deploys the `cephfs-mirror` daemon on the given node.

   ```
   ceph orch apply cephfs-mirror ["NODE_NAME"]
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph orch apply cephfs-mirror "node1.example.com"
   Scheduled cephfs-mirror update...
   ```

   a. Deploy multiple CephFS mirroring daemons and achieve high availability.

      ```
      ceph orch apply cephfs-mirror --placement="PLACEMENT_SPECIFICATION"
      ```

      The following example deploys three `cephfs-mirror` daemons on different hosts.

> **Warning:** Do not separate the hosts with commas. Using commas results in the following error:
>
> `Error EINVAL: name component must include only a-z, 0-9, and -`

```
[ceph: root@host01 /]# ceph orch apply cephfs-mirror --placement="3 host1 host2
host3"
Scheduled cephfs-mirror update...
```

3. On the source storage cluster, enable the CephFS mirroring module.

```
ceph mgr module enable mirroring
```

For example,

```
[ceph: root@host01 /]# ceph mgr module enable mirroring
```

4. On the source storage cluster, enable mirroring on a Ceph File System.

```
ceph fs snapshot mirror enable FILE_SYSTEM_NAME
```

For example,

```
[ceph: root@host01 /]# ceph fs snapshot mirror enable cephfs
```

a. Disable snapshot mirroring.

> **Warning:** Disabling snapshot mirroring on a file system removes the configured peers. You have to import the peers again by bootstrapping them.

```
ceph fs snapshot mirror disable FILE_SYSTEM_NAME
```

For example,

```
[ceph: root@host01 /]# ceph fs snapshot mirror disable cephfs
```

5. Prepare the target peer storage cluster.

a. On a target node, enable the `mirroring` Ceph Manager module.

```
ceph mgr module enable mirroring
```

For example,

```
[ceph: root@host01 /]# ceph mgr module enable mirroring
```

b. On the same target node, create the peer bootstrap.

```
ceph fs snapshot mirror peer_bootstrap
create FILE_SYSTEM_NAME CLIENT_NAME SITE_NAME
```

The *SITE_NAME* is a user-defined string to identify the target storage cluster.

For example,

```
[ceph: root@host01 /]# ceph fs snapshot mirror peer_bootstrap create cephfs
client.mirror_remote remote-site
{"token":
"eyJmc2lkIjogIjBkZjE3MjE3LWRmY2QtNDAzMC05MDc5LTM2Nzk4NTVkNDJlZiIsICJmaWxlc3lzdGVt
IjogImJhY2t1cF9mcyIsICJ1c2VyIjogImNsaWVudC5taXJyb3JfcGVlc19ib290c3RyYXAiLCAic2l0Z
V9uYW1lIjogInNpdGUtcmVtb3RlIiwgImtleSI6ICJBUUFhcDBCZ0xtRmpOeEFBVnNyZXozai9YYUV0T2
UrbUJEZlJDZz09IiwgIm1vbl9ob3N0IjogIlt2MjoxOTIuMTY4LjAuNTo0MDkxOCx2MToxOTIuMTY4LjA
uNTo0MDkxOV0ifQ=="}
```

c. Copy the token string between the double quotes, from step "5.b" on page 1248 for use in step "6" on page 1249.

6. On the source storage cluster, import the bootstrap token from the target storage cluster.

```
ceph fs snapshot mirror peer_bootstrap import FILE_SYSTEM_NAME TOKEN
```

For example,

```
[ceph: root@host01 /]# ceph fs snapshot mirror peer_bootstrap import cephfs
eyJmc2lkIjogIjBkZjE3MjE3LWRmY2QtNDAzMC05MDc5LTM2Nzk4NTVkNDJlZiIsICJmaWxlc3lzdGVtIjogIm
JhY2t1cF9mcyIsICJ1c2VyIjogImNsaWVudC5taXJyb3JfcGVlcl9ib290c3RyYXAiLCAic2l0ZV9uYW1lIjog
InNpdGUtcmVtb3RlIiwgImtleSI6ICJBUUFhcDBCZ0xtRmpOeEFBVnNyZXozai9YYUV0T2UrbUJEZlJDZz09Ii
wgIm1vbl9ob3N0IjogIlt2MjoxOTIuMTY4LjAuNTo0MDkxOCx2MToxOTIuMTY4LjAuNTo0MDkxOV0ifQ==
```

7. On the source storage cluster, list the CephFS mirror peers.

```
ceph fs snapshot mirror peer_list FILE_SYSTEM_NAME
```

For example,

```
[ceph: root@host01 /]# ceph fs snapshot mirror peer_list cephfs
{"e5ecb883-097d-492d-b026-a585d1d7da79": {"client_name": "client.mirror_remote",
"site_name": "remote-site", "fs_name": "cephfs", "mon_host":
"[v2:10.0.211.54:3300/0,v1:10.0.211.54:6789/0]
[v2:10.0.210.56:3300/0,v1:10.0.210.56:6789/0]
[v2:10.0.210.65:3300/0,v1:10.0.210.65:6789/0]"}}
```

a. Remove a snapshot peer.

```
ceph fs snapshot mirror peer_remove FILE_SYSTEM_NAME PEER_UUID
```

> **Note:** For more information about how to find the peer UUID value, see step "4" on page 1251 of "Viewing the mirror status for a Ceph File System" on page 1250.

For example,

```
[ceph: root@host01 /]# ceph fs snapshot mirror peer_remove cephfs
e5ecb883-097d-492d-b026-a585d1d7da79
```

8. On the source storage cluster, configure a directory for snapshot mirroring.

```
ceph fs snapshot mirror add FILE_SYSTEM_NAME PATH
```

> **Important:** Only absolute paths inside the Ceph File System are valid.

> **Note:** The Ceph Manager `mirroring` module normalizes the path. For example, the `/d1/d2/../dN` directories are equivalent to `/d1/d2`. Once a directory has been added for mirroring, its ancestor directories and subdirectories are prevented from being added for mirroring.

For example,

```
[ceph: root@host01 /]# ceph fs snapshot mirror add cephfs /volumes/_nogroup/subvol_1
```

a. Stop snapshot mirroring for a directory.

```
ceph fs snapshot mirror remove FILE_SYSTEM_NAME PATH
```

For example,

```
[ceph: root@host01 /]# ceph fs snapshot mirror remove cephfs /home/user1
```

What to do next

To initiate snapshot mirroring, create a snapshot of the configured directory in the primary storage cluster. For more information, see “Creating a snapshot for a Ceph File System” on page 1229.

Viewing the mirror status for a Ceph File System

The Ceph File System (CephFS) mirror daemon (`cephfs-mirror`) gets asynchronous notifications about changes in the CephFS mirroring status, along with peer updates. The CephFS mirroring module provides a mirror daemon status interface to check mirror daemon status.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- A minimum of one deployment of a Ceph File System with mirroring enabled.
- Root-level access to the node running the CephFS mirroring daemon.

About this task

For more detailed information, you can query the `cephfs-mirror` admin socket with commands to retrieve the mirror status and peer status.

Procedure

1. Log in to the `cephadm` shell.
   For example,

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Check the `cephfs-mirror` daemon status.

   > **Note:** For more detailed information, use the admin socket interface as detailed in step “5” on page 1252.

   ```
   ceph fs snapshot mirror daemon status
   ```

   For example,

```
[ceph: root@host01 /]$ ceph fs snapshot mirror daemon status
[
  {
    "daemon_id": 284167,
    "filesystems": [
      {
        "filesystem_id": 1,
        "name": "a",
        "directory_count": 1,
        "peers": [
          {
            "uuid": "02117353-8cd1-44db-976b-eb20609aa160",
            "remote": {
              "client_name": "client.mirror_remote",
              "cluster_name": "ceph",
              "fs_name": "backup_fs",
              "mon_host": "[v2:192.168.64.5:40183,v1:192.168.64.5:40184]",
              "fsid": "5682c8e5-50cd-4cfd-b75c-5354dcdda487"
            },
            "stats": {
              "failure_count": 1,
              "recovery_count": 0
            }
          }
        ]
      }
    ]
  }
]
```

3. Find the Ceph File System ID on the node running the CephFS mirroring daemon.

```
ceph --admin-daemon PATH_TO_THE_ASOK_FILE help
```

> **Note:** When mirroring is disabled, the respective **`fs mirror status`** command for the file system does not show up in the **`help`** command.

In the following example, the Ceph File System ID is `cephfs@11`.

```
[ceph: root@host01 /]# ceph --admin-daemon /var/run/ceph/1011435c-9e30-4db6-
b720-5bf482006e0e/ceph-client.cephfs-mirror.node1.bndvox.asok help
{
    ...
    "fs mirror peer status cephfs@11 1011435c-9e30-4db6-b720-5bf482006e0e": "get peer
mirror status",
    "fs mirror status cephfs@11": "get filesystem mirror status",
    ...
}
```

4. View the mirror status.

```
ceph --admin-daemon PATH_TO_THE_ASOK_FILE fs mirror
status FILE_SYSTEM_NAME @_FILE_SYSTEM_ID
```

For example,

```
[ceph: root@host01 /]# ceph --admin-daemon /var/run/ceph/1011435c-9e30-4db6-
b720-5bf482006e0e/ceph-client.cephfs-mirror.node1.bndvox.asok fs mirror status
cephfs@11
{
  "rados_inst": "192.168.0.5:0/1476644347",
  "peers": {
      "1011435c-9e30-4db6-b720-5bf482006e0e": { (1)
          "remote": {
              "client_name": "client.mirror_remote",
              "cluster_name": "remote-site",
              "fs_name": "cephfs"
          }
      }
  },
  "snap_dirs": {
      "dir_count": 1
  }
}
```

(1) This is the unique peer UUID.

5. View the peer status.

```
ceph --admin-daemon PATH_TO_ADMIN_SOCKET fs mirror
status FILE_SYSTEM_NAME@FILE_SYSTEM_ID PEER_UUID
```

For example,

```
[ceph: root@host01 /]# ceph --admin-daemon /var/run/ceph/cephfs-mirror.asok fs mirror
peer status cephfs@11 1011435c-9e30-4db6-b720-5bf482006e0e
{
  "/home/user1": {
      "state": "idle", (1)
      "last_synced_snap": {
          "id": 120,
          "name": "snap1",
          "sync_duration": 0.079997898999999997,
          "sync_time_stamp": "274900.558797s"
      },
      "snaps_synced": 2, (2)
      "snaps_deleted": 0, (3)
      "snaps_renamed": 0
  }
}
```

The `state` can be one of these three values:

(1) `idle` means that the directory is currently not being synchronized.

(2) `syncing` means that the directory is currently being synchronized.

(3) `failed` means that the directory has reached the upper limit of consecutive failures.

The default number of consecutive failures is 10, and the default retry interval is 60 seconds.

Displaying the directory to which `cephfs-mirror` daemon is mapped

To display the directory to which `cephfs-mirror` daemon is mapped:

```
ceph fs snapshot mirror dirmap FILE_SYSTEM_NAME PATH
```

Example 1 (mirror daemons are running)

```
[ceph: root@host01 /]# ceph fs snapshot mirror dirmap cephfs /volumes/_nogroup/subvol_1
{
  "instance_id": "25184", (1)
  "last_shuffled": 1661162007.012663,
  "state": "mapped"
}
```

(1) `instance_id` is the RADOS instance-ID that is associated with a `cephfs-mirror` daemon.

Example 2 (no mirror daemons are running)

```
[ceph: root@host01 /]# ceph fs snapshot mirror dirmap cephfs /volumes/_nogroup/subvol_1
{
  "reason": "no mirror daemons running",
  "state": "stalled" (1)
}
```

(1) `stalled` state means the CephFS mirroring is stalled.

Viewing metrics for Ceph File System snapshot mirroring

Monitor Ceph File System snapshot mirroring performance and synchronization progress by viewing available mirroring metrics.

Before you begin

- A running Red Hat Ceph Storage cluster.
- A minimum of one deployment of a Ceph File System snapshot mirroring enabled.
- Root-level access to the node running the Ceph File System mirroring daemon.

About this task

Check Ceph File System snapshot mirror health and volume metrics by using the **`counter dump`**.

Procedure

- Check the mirroring metrics and synchronization status by using the **`counter dump`** command.
  a. Run the counter dump command on the node running the CephFS mirroring daemon.

```
ceph --admin-daemon ASOK_FILE_NAME counter dump
```

> **Note:** The CephFS mirror daemon admin socket (ASOK) file is available only on the node where the active `cephfs-mirror` daemon is running.

For example,

```
[ceph: root@mirror-host01 /]# ceph --admin-daemon ceph-client.cephfs-
mirror.ceph1-hk-n-0mfqao-node7.pnbrlu.2.93909288073464.asok counter dump

[
  {
    "key": "cephfs_mirror",
    "value": [
      {
        "labels": {},
        "counters": {
          "mirrored_filesystems": 1,
          "mirror_enable_failures": 0
        }
      }
    ]
  },
  {
    "key": "cephfs_mirror_mirrored_filesystems",
    "value": [
      {
        "labels": {
          "filesystem": "cephfs"
        },
        "counters": {
          "mirroring_peers": 1,
          "directory_count": 1
        }
      }
    ]
  },
  {
    "key": "cephfs_mirror_peers",
    "value": [
      {
        "labels": {
          "peer_cluster_filesystem": "cephfs",
          "peer_cluster_name": "remote_site",
          "source_filesystem": "cephfs",
          "source_fscid": "1"
        },
        "counters": {
          "snaps_synced": 1,
          "snaps_deleted": 0,
          "snaps_renamed": 0,
          "sync_failures": 0,
          "avg_sync_time": {
            "avgcount": 1,
            "sum": 4.216959457,
            "avgtime": 4.216959457
          },
          "sync_bytes": 132
        }
      }
    ]
  }
]
```

Metrics description:

Labeled `Perf Counters` generate metrics that the OCP/ODF dashboard can use to provide monitoring of geo-replication in the OCP and ACM dashboard.

These metrics show the synchronization progress of `cephfs_mirror` and provide monitoring capability. The exported metrics enable monitoring based on the following alerts.

mirroring_peers

The number of peers involved in mirroring.

directory_count

The total number of directories being synchronized.

mirrored_filesystems

The total number of file systems which are mirrored.

mirror_enable_failures

The total number of failures when enabling mirroring.

snaps_synced

The total number of snapshots successfully synchronized.

sync_bytes

The total bytes being synchronized.

sync_failures

The total number of failed snapshot synchronizations.

snaps_deleted

The total number of snapshots deleted.

snaps_renamed

The total number of snapshots renamed.

avg_synced_time

The average time taken by all snapshot synchronizations.

last_synced_start

The sync start time of the last synced snapshot.

last_synced_end

The sync end time of the last synced snapshot.

last_synced_duration

The time duration of the last synchronization.

last_synced_bytes

The total bytes being synchronized for the last synced snapshot.

Red Hat Ceph Storage Block Devices

Learn to manage, create, configure, and use Red Hat Ceph Storage Block Devices.

A block is a set length of bytes in a sequence, for example, a 512-byte block of data. Combining many blocks together into a single file can be used as a storage device that you can read from and write to. Block-based storage interfaces are the most common way to store data with rotating media such as:

- Hard drives
- CD/DVD discs
- Floppy disks
- Traditional 9-track tapes

Ceph Block Devices are thin-provisioned, resizable and store data striped over multiple Object Storage Devices (OSD) in a Ceph storage cluster. Ceph Block Devices are also known as Reliable Autonomic Distributed Object Store (RADOS) Block Devices (RBDs). Ceph Block Devices leverage RADOS capabilities such as:

- Snapshots
- Replication
- Data consistency

Ceph Block Devices interact with OSDs by using the `librbd` library.

Ceph Block Devices deliver high performance with infinite scalability to Kernel Virtual Machines (KVMs), such as Quick Emulator (QEMU), and cloud-based computing systems, like OpenStack, that rely on the `libvirt` and QEMU utilities to integrate with Ceph Block Devices. You can use the same storage cluster to operate the Ceph Object Gateway and Ceph Block Devices simultaneously.

> **Important:** Using Ceph Block Devices requires access to a running Ceph storage cluster. For more information about installing a Red Hat Ceph Storage cluster, see Initial installation.

Using Ceph Block Devices

As a storage administrator, being familiar with Ceph Block Device commands can help you effectively manage the Red Hat Ceph Storage. You can create and manage block devices pools and images, along with enabling and disabling the various features of Ceph Block Devices.

Displaying command help

Display command, and sub-command online help from the command-line interface.

> **Note:** The `-h` option still displays help for all available commands.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the client node.

Procedure

1. Use the `rbd help` command to display help for a particular `rbd` command and its subcommand:

   Syntax

```
rbd help COMMAND SUBCOMMAND
```

2. To display help for the `snap list` command:

```
[root@rbd-client ~]# rbd help snap list
```

Creating block device pools

Learn how to create a block device pool.

Before you begin

Before using the block device client, ensure a pool for `rbd` exists, is enabled and initialized.

> **Note:** You **MUST** create a pool first before you can specify it as a source.

Before beginning this procedure, ensure that you have the following:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the client node.

About this task

For more information, see Pools.

Procedure

- Create a Ceph Block Device (`rbd`) pool.

```
ceph osd pool create POOL_NAME PG_NUM

ceph osd pool application enable POOL_NAME rbd
rbd pool init -p POOL_NAME
```

For example,

```
[root@rbd-client ~]# ceph osd pool create pool1
[root@rbd-client ~]# ceph osd pool application enable pool1 rbd
[root@rbd-client ~]# rbd pool init -p pool1
```

Creating images

Before adding a block device to a node, create a Ceph Block Device image for it in the Ceph storage cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the client node.

Procedure

- Create a block device image.

> **Note:** Ensure the pool exists before creating an image.

```
rbd create IMAGE_NAME --size MEGABYTES --pool POOL_NAME
```

For example,

```
[root@rbd-client ~]# rbd create image1 --size 1024 --pool pool1
```

This example creates a 1 GB image named `image1` that stores information in a pool named `pool1`.

Related information

Creating block device pools

Listing images

List the Ceph Block Device images with the **`rbd ls`** command.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the client node.

Procedure

1. To list block devices in the `rbd` pool, run the following command:

   > **Note:** `rbd` is the default pool name.

   Example

   ```
   [root@rbd-client ~]# rbd ls
   ```

2. To list block devices in a specific pool:

   Syntax

   ```
   rbd ls POOL_NAME
   ```

   Example

   ```
   [root@rbd-client ~]# rbd ls pool1
   ```

Retrieving image information

Retrieve information on the block device image.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the client node.

Procedure

1. To retrieve information from a particular image in the default `rbd` pool, run the following command:

   Syntax

   ```
   rbd --image IMAGE_NAME info
   ```

   Example

   ```
   [root@rbd-client ~]# rbd --image image1 info
   ```

2. To retrieve information from an image within a pool:

   Syntax

   ```
   rbd --image IMAGE_NAME -p POOL_NAME info
   ```

   Example

   ```
   [root@rbd-client ~]# rbd --image image1 -p pool1 info
   ```

Resizing images

Ceph Block Device images are thin-provisioned. They do not actually use any physical storage until you begin saving data to them. However, they do have a maximum capacity that you set with the `--size` option.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Root-level access to the client node.

Procedure

1. To increase the maximum size of a Ceph Block Device image for the default `rbd` pool:
   Syntax

   ```
   rbd resize --image IMAGE_NAME --size SIZE
   ```

   Example

   ```
   [root@rbd-client ~]# rbd resize --image image1 --size 1024
   ```

2. To decrease the maximum size of a Ceph Block Device image for the default `rbd` pool:
   Syntax

   ```
   rbd resize --image IMAGE_NAME --size SIZE --allow-shrink
   ```

   Example

   ```
   [root@rbd-client ~]# rbd resize --image image1 --size 1024 --allow-shrink
   ```

3. To increase the maximum size of a Ceph Block Device image for a specific pool:
   Syntax

   ```
   rbd resize --image POOL_NAME/IMAGE_NAME --size SIZE
   ```

   Example

   ```
   [root@rbd-client ~]# rbd resize --image pool1/image1 --size 1024
   ```

4. To decrease the maximum size of a Ceph Block Device image for a specific pool:
   Syntax

   ```
   rbd resize --image POOL_NAME/IMAGE_NAME --size SIZE --allow-shrink
   ```

   Example

   ```
   [root@rbd-client ~]# rbd resize --image pool1/image1 --size 1024 --allow-shrink
   ```

Removing images

Remove a Ceph Block Device image with the **`rbd rm`** command.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the client node.

Procedure

- Remove a block device from the default `rbd` pool.

```
rbd rm IMAGE_NAME
```

For example,

```
[root@rbd-client ~]# rbd rm image1
```

- Remove a block device from a specific pool.

```
rbd rm IMAGE_NAME -p POOL_NAME
```

```
[root@rbd-client ~]# rbd rm image1 -p pool1
```

Moving images to trash

RADOS Block Device (RBD) images can be moved to the trash using the **`rbd trash`** command.

The **`rbd trash`** command provides more options than the **`rbd rm`** command.

Once an image is moved to the trash, it can be removed from the trash at a later time. This helps to avoid accidental deletion.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the client node.

Procedure

1. Move an image to the trash:

   Syntax

   ```
   rbd trash mv POOL_NAME/IMAGE_NAME
   ```

   Example

   ```
   [root@rbd-client ~]# rbd trash mv pool1/image1
   ```

   Once an image is in the trash, a unique image ID is assigned.

   > **Note:** You need this image ID to specify the image later if you need to use any of the trash options.

2. Run the `rbd trash list POOL_NAME` for a list of IDs of the images in the trash. This command also returns the image's pre-deletion name. In addition, there is an optional `--image-id` argument that can be used with `rbd info` and `rbd snap` commands. Use `--image-id` with the `rbd info` command to see the properties of an image in the trash, and with `rbd snap` to remove an image's snapshots from the trash.
3. To remove an image from the trash run the following:

   Syntax

```
rbd trash rm POOL_NAME/IMAGE_ID
```

Example

```
[root@rbd-client ~]# rbd trash rm pool1/d35ed01706a0
```

> **Important:** Once an image is removed from the trash, it cannot be restored.

4. Run the `rbd trash restore` command to restore the image:

   Syntax

   ```
   rbd trash restore POOL_NAME/IMAGE_ID
   ```

   Example

   ```
   [root@rbd-client ~]# rbd trash restore pool1/d35ed01706a0
   ```

5. To remove all expired images from trash:

   Syntax

   ```
   rbd trash purge POOL_NAME
   ```

   Example

   ```
   [root@rbd-client ~]# rbd trash purge pool1
   Removing images: 100% complete...done.
   ```

Schedule automatic trash purge

You can schedule periodic trash purge operations on a pool.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the client node.

Procedure

1. To add a trash purge schedule, run:

   Syntax

   ```
   rbd trash purge schedule add --pool POOL_NAME INTERVAL
   ```

   Example

   ```
   [ceph: root@host01 /]# rbd trash purge schedule add --pool pool1 10m
   ```

2. To list the trash purge schedule, run:

   Syntax

   ```
   rbd trash purge schedule ls --pool POOL_NAME
   ```

   Example

```
[ceph: root@host01 /]# rbd trash purge schedule ls --pool pool1
every 10m
```

3. To know the status of trash purge schedule, run:

   Example

```
[ceph: root@host01 /]# rbd trash purge schedule status
POOL   NAMESPACE   SCHEDULE  TIME
pool1             2021-08-02 11:50:00
```

4. To remove the trash purge schedule, run:

   Syntax

```
rbd trash purge schedule remove --pool POOL_NAME INTERVAL
```

   Example

```
[ceph: root@host01 /]# rbd trash purge schedule remove --pool pool1 10m
```

Enabling and disabling image features

The Ceph Block Device image features, such as `fast-diff`, `exclusive-lock`, `object-map`, or `deep-flatten`, are enabled by default. You can enable or disable these image features on already existing images.

Note: The `deep flatten` feature can be only disabled on already existing images but not enabled. To use `deep flatten`, enable it when creating images.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the client node.

Procedure

1. Retrieve information from a particular image in a pool:

   Syntax

```
rbd --image POOL_NAME/IMAGE_NAME info
```

   Example

```
[ceph: root@host01 /]# rbd --image pool1/image1 info
```

2. Enable a feature:

   Syntax

```
rbd feature enable POOL_NAME/IMAGE_NAME FEATURE_NAME
```

   - To enable the `exclusive-lock` feature on the `image1` image in the `pool1` pool:
     Example

```
[ceph: root@host01 /]# rbd feature enable pool1/image1 exclusive-lock
```

Important: If you enable the `fast-diff` and `object-map` features, then rebuild the object map:

```
rbd object-map rebuild POOL_NAME/IMAGE_NAME
```

3. Disable a feature:

   Syntax

   ```
   rbd feature disable POOL_NAME/IMAGE_NAME FEATURE_NAME
   ```

   - To disable the `fast-diff` feature on the `image1` image in the `pool1` pool:
     Example

     ```
     [ceph: root@host01 /]# rbd feature disable pool1/image1 fast-diff
     ```

Working with image metadata

Ceph supports adding custom image metadata as key-value pairs. The pairs do not have any strict format. By using metadata you can also set the RADOS Block Device (RBD) configuration parameters for particular images. Use the **`rbd image-meta`** command to work with metadata.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the client node.

Procedure

1. To set a new metadata key-value pair:

   Syntax

   ```
   rbd image-meta set POOL_NAME/IMAGE_NAME KEY VALUE
   ```

   Example

   ```
   [ceph: root@host01 /]# rbd image-meta set pool1/image1 last_update 2021-06-06
   ```

   This example sets the `last_update` key to the `2021-06-06` value on the `image1` image in the `pool1` pool.

2. To view a value of a key:

   Syntax

   ```
   rbd image-meta get POOL_NAME/IMAGE_NAME KEY
   ```

   Example

   ```
   [ceph: root@host01 /]# rbd image-meta get pool1/image1 last_update
   ```

   This example views the value of the `last_update` key.

3. To show all metadata on an image:

   Syntax

   ```
   rbd image-meta list POOL_NAME/IMAGE_NAME
   ```

   Example

```
[ceph: root@host01 /]# rbd image-meta list pool1/image1
```

This example lists the metadata set for the `image1` image in the `pool1` pool.

4. To remove a metadata key-value pair:

   Syntax

   ```
   rbd image-meta remove POOL_NAME/IMAGE_NAME KEY
   ```

   Example

   ```
   [ceph: root@host01 /]# rbd image-meta remove pool1/image1 last_update
   ```

   This example removes the `last_update` key-value pair from the `image1` image in the `pool1` pool.

5. To override the RBD image configuration settings set in the Ceph configuration file for a particular image:

   Syntax

   ```
   rbd config image set POOL_NAME/IMAGE_NAME  PARAMETER VALUE
   ```

   Example

   ```
   [ceph: root@host01 /]# rbd config image set pool1/image1 rbd_cache false
   ```

   This example disables the RBD cache for the `image1` image in the `pool1` pool.

Reference

For a list of possible configuration options, see “General options” on page 1341.

Moving images between pools

You can move RADOS Block Device (RBD) images between different pools within the same cluster. This process copies the source image to the target image with all snapshot history and optionally with link to the source image’s parent to help preserve sparseness.

The source image is read only, the target image is writable. The target image is linked to the source image while the migration is in progress.

You can safely run this process in the background while the new target image is in use. However, stop all clients using the source image before the preparation step to ensure that clients using the image are updated to point to the new target image.

> **Important:** The `krbd` kernel module does not support live migration at this time.

Prerequisites

- Stop all clients that use the source image.
- Root-level access to the client node.

Procedure

1. Prepare for migration by creating the new target image that cross-links the source and target images:

   Syntax

   ```
   rbd migration prepare SOURCE_IMAGE TARGET_IMAGE
   ```

   Replace:

- SOURCE_IMAGE with the name of the image to be moved. Use the POOL_NAME/IMAGE_NAME format.
- TARGET_IMAGE with the name of the new image. Use the POOL_NAME/IMAGE_NAME format.

Example

```
[root@rbd-client ~]# rbd migration prepare pool1/image1 pool2/image2
```

2. Verify the state of the new target image, which is supposed to be prepared:

   Syntax

   ```
   rbd status TARGET_IMAGE
   ```

   Example

   ```
   [root@rbd-client ~]# rbd status pool2/image2
   Watchers: none
   Migration:
               source: pool1/image1 (5e2cba2f62e)
               destination: pool2/image2 (5e2ed95ed806)
               state: prepared
   ```

3. Optionally, restart the clients using the new target image name.
4. Copy the source image to target image:

   Syntax

   ```
   rbd migration execute TARGET_IMAGE
   ```

   Example

   ```
   [root@rbd-client ~]# rbd migration execute pool2/image2
   ```

5. Ensure that the migration is completed:

   Example

   ```
   [root@rbd-client ~]# rbd status pool2/image2
   Watchers:
       watcher=1.2.3.4:0/3695551461 client.123 cookie=123
   Migration:
               source: pool1/image1 (5e2cba2f62e)
               destination: pool2/image2 (5e2ed95ed806)
               state: executed
   ```

6. Commit the migration by removing the cross-link between the source and target images, and this also removes the source image:

   Syntax

   ```
   rbd migration commit TARGET_IMAGE
   ```

   Example

   ```
   [root@rbd-client ~]# rbd migration commit pool2/image2
   ```

   If the source image is a parent of one or more clones, use the --force option after ensuring that the clone images are not in use:

   Example

   ```
   [root@rbd-client ~]# rbd migration commit pool2/image2 --force
   ```

7. If you did not restart the clients after the preparation step, restart them using the new target image name.

Migrating pools

You can migrate or copy RADOS Block Device (RBD) images. In migrating pools the source image is exported and then imported.

Before you begin

- Stop all active I/O in the RBD images which are being exported and imported.
- Root-level access to the client node.

About this task

Important:

- Use this migration process if the workload contains only RBD images. No `rados cppool` images can exist in the workload. If `rados cppool` images exist in the workload, see Migrating a pool.
- While running the export and import commands, be sure that there is no active I/O in the related RBD images. It is recommended to take production down during this pool migration time.

Procedure

- Migrate the volume.

  ```
  rbd export volumes/VOLUME_NAME - | rbd import --image-format 2 - volumes_new/
  VOLUME_NAME
  ```

  For example,

  ```
  [root@rbd-client ~]# rbd export volumes/volume-3c4c63e3-3208-436f-9585-fee4e2a3de16 - |
  rbd import --image-format 2 - volumes_new/volume-3c4c63e3-3208-436f-9585-fee4e2a3de16
  ```

- If using the local drive for import or export is necessary, the commands can be divided, first exporting to a local drive and then importing the files to a new pool.

  ```
  rbd export volume/VOLUME_NAME FILE_PATH
  rbd import --image-format 2 FILE_PATH volumes_new/VOLUME_NAME
  ```

  For example,

  ```
  [root@rbd-client ~]# rbd export volumes/volume-3c4c63e3-3208-436f-9585-fee4e2a3de16
   <path of export file>
  [root@rbd-client ~]# rbd import --image-format 2 <path> volumes_new/
  volume-3c4c63e3-3208-436f-9585-fee4e2a3de16
  ```

`rbdmap` service

The `systemd` unit file, `rbdmap.service`, is included with the `ceph-common` package. The `rbdmap.service` unit runs the `rbdmap` shell script. This script automates the mapping and unmapping of RADOS Block Devices (RBD) for one or more RBD images.

The `rbdmap` script can be ran manually at any time, but the typical use case is to automatically mount RBD images at boot time, and unmount at shutdown. The script takes a single argument, which can be either `map`, for mounting or `unmap`, for unmounting RBD images. The script parses a configuration file, the default is `/etc/ceph/rbdmap`, but can be overridden using an environment variable called `RBDMAPFILE`. Each line of the configuration file corresponds to an RBD image.

The format of the configuration file format is as follows:

```
IMAGE_SPEC RBD_OPTS
```

Where `IMAGE_SPEC` specifies the `POOL_NAME / IMAGE_NAME`, or just the `IMAGE_NAME`, in which case the `POOL_NAME` defaults to `rbd`. The `RBD_OPTS` is an optional list of options to be passed to the underlying `rbd map` command. These parameters and their values should be specified as a comma-separated string:

```
OPT1=VAL1,OPT2=VAL2,...,OPT_N=VAL_N
```

This will cause the script to issue an `rbd map` command like the following:

Syntax

```
rbd map POOL_NAME/IMAGE_NAME --OPT1 VAL1 --OPT2 VAL2
```

> **Note:** For options and values which contain commas or equality signs, a simple apostrophe can be used to prevent replacing them.

When successful, the `rbd map` operation maps the image to a `/dev/rbdX` device, at which point a `udev` rule is triggered to create a friendly device name symlink, for example, `/dev/rbd/POOL_NAME/IMAGE_NAME`, pointing to the real mapped device. For mounting or unmounting to succeed, the friendly device name must have a corresponding entry in `/etc/fstab` file. When writing `/etc/fstab` entries for RBD images, it is a good idea to specify the `noauto` or `nofail` mount option. This prevents the init system from trying to mount the device too early, before the device exists.

Reference

- See the `rbd` manpage for a full list of possible options.

Configuring `rbdmap` service

Automatically map and mount, or unmap and unmount, RADOS Block Devices (RBD) at boot time, or at shutdown respectively.

Prerequisites

- Root-level access to the node doing the mounting.
- Installation of the `ceph-common` package.

Procedure

1. Open for editing the `/etc/ceph/rbdmap` configuration file.
2. Add the RBD image or images to the configuration file:

   Example

   ```
   foo/bar1    id=admin,keyring=/etc/ceph/ceph.client.admin.keyring
   foo/bar2    id=admin,keyring=/etc/ceph/
   ceph.client.admin.keyring,options='lock_on_read,queue_depth=1024'
   ```

3. Save changes to the configuration file.
4. Enable the RBD mapping service:

   Example

   ```
   [root@client ~]# systemctl enable rbdmap.service
   ```

Reference

For more information about the RBD system service, see “rbdmap service” on page 1266.

Persistent write log cache

Red Hat Ceph Storage cluster, Persistent Write Log (PWL) cache provides a persistent, fault-tolerant write-back cache for librbd-based RBD clients.

PWL cache uses a log-ordered write-back design which maintains checkpoints internally so that writes that get flushed back to the cluster are always crash consistent. If the client cache is lost entirely, the disk image is still consistent but the data appears stale. You can use PWL cache with persistent memory (PMEM) or solid-state disks (SSD) as cache devices.

For PMEM, the cache mode is replica write log (RWL) and for SSD, the cache mode is (SSD). Currently, PWL cache supports RWL and SSD modes and is disabled by default.

Primary benefits of PWL cache

- PWL cache can provide high performance when the cache is not full. The larger the cache, the longer the duration of high performance.
- PWL cache provides persistence and is not much slower than RBD cache. RBD cache is faster but volatile and cannot guarantee data order and persistence.
- In a steady state, where the cache is full, performance is affected by the number of I/Os in flight. For example, PWL can provide higher performance at low io_depth, but at high io_depth, such as when the number of I/Os is greater than 32, the performance is often worse than that in cases without cache.

Use cases for PMEM caching

- Different from RBD cache, PWL cache has non-volatile characteristics and is used in scenarios where you do not want data loss and need performance.
- RWL mode provides low latency. It has a stable low latency for burst I/Os and it is suitable for those scenarios with high requirements for stable low latency.
- RWL mode also has high continuous and stable performance improvement in scenarios with low I/O depth or not too much inflight I/O.

Use case for SSD caching

- The advantages of SSD mode are similar to RWL mode. SSD hardware is relatively cheap and popular, but its performance is slightly lower than PMEM.

Limitations

Understand the limitations when using Persistent Write Log (PWL) cache.

When using Persistent Write Log (PWL) cache, there are several limitations that should be considered.

- The underlying implementation of persistent memory (PMEM) and solid-state disks (SSD) is different, with PMEM having higher performance. At present, PMEM can provide `persist on write` and SSD is `persist on flush or checkpoint`. In future releases, these two modes will be configurable.
- When users switch frequently and open and close images repeatedly, Ceph displays poor performance. If PWL cache is enabled, the performance is worse. It is not recommended to set `num_jobs` in a Flexible I/O (fio) test, but instead setup multiple jobs to write different images.

Enabling

You can enable persistent write log cache (PWL) on a Red Hat Ceph Storage cluster by setting the Ceph RADOS block device (RBD) `rbd_persistent_cache_mode` and `rbd_plugins` options.

> **Important:** The exclusive-lock feature must be enabled to enable persistent write log cache. The cache can be loaded only after the exclusive-lock is acquired. Exclusive-locks are enabled on newly created images by default unless overridden by the `rbd_default_features` configuration option or the `--image-feature` flag for the **`rbd create`** command. For more information about the `exclusive-lock` feature, see "Enabling and disabling image features" on page 1262.

Set the persistent write log cache options at the host level by using the **`ceph config set`** command. Set the persistent write log cache options at the pool or image level by using the **`rbd config pool set`** or the **`rbd config image set`** commands.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the monitor node.
- The exclusive-lock feature is enabled.
- Client-side disks are persistent memory (PMEM) or solid-state disks (SSD).
- RBD cache is disabled.

Procedure

1. Enable PWL cache:

   i. At the host level, use the `ceph config set` command:

   Syntax

   ```
   ceph config set client rbd_persistent_cache_mode CACHE_MODE
   ceph config set client rbd_plugins pwl_cache
   ```

   Replace *CACHE_MODE* with `rwl` or `ssd`.

   Example

   ```
   [ceph: root@host01 /]# ceph config set client rbd_persistent_cache_mode ssd
   [ceph: root@host01 /]# ceph config set client rbd_plugins pwl_cache
   ```

   ii. At the pool level, use the `rbd config pool set` command:

   Syntax

   ```
   rbd config pool set POOL_NAME rbd_persistent_cache_mode CACHE_MODE
   rbd config pool set POOL_NAME rbd_plugins pwl_cache
   ```

   Replace *CACHE_MODE* with `rwl` or `ssd`.

   Example

   ```
   [ceph: root@host01 /]# rbd config pool set pool1 rbd_persistent_cache_mode ssd
   [ceph: root@host01 /]# rbd config pool set pool1 rbd_plugins pwl_cache
   ```

   iii. At the image level, use the `rbd config image set` command:

   Syntax

   ```
   rbd config image set POOL_NAME/IMAGE_NAME rbd_persistent_cache_mode CACHE_MODE
   rbd config image set POOL_NAME/IMAGE_NAME rbd_plugins pwl_cache
   ```

   Replace *CACHE_MODE* with `rwl` or `ssd`.

   Example

   ```
   [ceph: root@host01 /]# rbd config image set pool1/image1 rbd_persistent_cache_mode ssd
   [ceph: root@host01 /]# rbd config image set pool1/image1 rbd_plugins pwl_cache
   ```

2. Optional: Set the additional RBD options at the host, the pool, or the image level:

   Syntax

   ```
   rbd_persistent_cache_mode CACHE_MODE
   rbd_plugins pwl_cache
   rbd_persistent_cache_path /PATH_TO_DAX_ENABLED_FOLDER/WRITE_BACK_CACHE_FOLDER <1>
   rbd_persistent_cache_size PERSISTENT_CACHE_SIZE <2>
   ```

   <1> `rbd_persistent_cache_path` - A file folder to cache data that must have direct access (DAX) enabled when using the `rwl` mode to avoid performance degradation.

<2> `rbd_persistent_cache_size` - The cache size per image, with a minimum cache size of 1 GB. The larger the cache size, the better the performance.

Example

```
rbd_cache false
rbd_persistent_cache_mode rwl
rbd_plugins pwl_cache
rbd_persistent_cache_path /mnt/pmem/cache/
rbd_persistent_cache_size 1073741824
```

Reference

- See Direct Access for files article on *kernel.org* for more details on using DAX.

Checking status

You can check the status of the Persistent Write Log (PWL) cache. The cache is used when an exclusive lock is acquired, and when the exclusive-lock is released, the persistent write log cache is closed. The cache status shows information about the cache size, location, type, and other cache-related information. Updates to the cache status are done when the cache is opened and closed.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the monitor node.
- A running process with PWL cache enabled.

Procedure

- View the PWL cache status:

  Syntax

  ```
  rbd status POOL_NAME/IMAGE_NAME
  ```

  Example

  ```
  [ceph: root@host01 /]# rbd status pool1/image1
  Watchers:
          watcher=10.10.0.102:0/1061883624 client.25496 cookie=140338056493088
  Persistent cache state:
          host: host02
          path: /mnt/nvme0/rbd-pwl.rbd.101e5824ad9a.pool
          size: 1 GiB
          mode: ssd
          stats_timestamp: Mon Apr 18 13:26:32 2022
          present: true   empty: false    clean: false
          allocated: 509 MiB
          cached: 501 MiB
          dirty: 338 MiB
          free: 515 MiB
          hits_full: 1450 / 61%
          hits_partial: 0 / 0%
          misses: 924
          hit_bytes: 192 MiB / 66%
          miss_bytes: 97 MiB
  ```

Flushing

You can flush the cache file with the `rbd` command, specifying `persistent-cache flush`, the pool name, and the image name before discarding the persistent write log (PWL) cache. The `flush` command can explicitly write cache files back to the OSDs. If there is a cache interruption or the application dies unexpectedly, all the entries in the cache are flushed to the OSDs so that you can manually flush the data and then `invalidate` the cache.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the monitor node.
- PWL cache is enabled.

Procedure

- Flush the PWL cache:

  Syntax

  ```
  rbd persistent-cache flush POOL_NAME/IMAGE_NAME
  ```

  Example

  ```
  [ceph: root@host01 /]# rbd persistent-cache flush pool1/image1
  ```

Reference

For more information, see “Discarding” on page 1271.

Discarding

You might need to manually discard the Persistent Write Log (PWL) cache, for example, if the data in the cache has expired. You can discard a cache file for an image by using the `rbd persistent-cache invalidate` command. The command removes the cache metadata for the specified image, disables the cache feature, and deletes the local cache file, if it exists.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the monitor node.
- PWL cache is enabled.

Procedure

- Discard PWL cache:

  Syntax

  ```
  rbd persistent-cache invalidate POOL_NAME/IMAGE_NAME
  ```

  Example

  ```
  [ceph: root@host01 /]# rbd persistent-cache invalidate pool1/image1
  ```

Monitoring performance of Ceph Block Devices

Learn how to monitor performance of Ceph Block Devices using the command-line interface.

This framework provides a built-in method to generate and process performance metrics upon which other Ceph Block Device performance monitoring solutions are built.

A new Ceph Manager module,`rbd_support`, aggregates the performance metrics when enabled. The `rbd` command has two new actions: `iotop` and `iostat`.

> **Note:** The initial use of these actions can take around 30 seconds to populate the data fields.

Prerequisites

- User-level access to a Ceph Monitor node.

Procedure

1. Ensure the `rbd_support` Ceph Manager module is enabled:

   Example

   ```
   [ceph: root@host01 /]# ceph mgr module ls

   {
          "always_on_modules": [
             "balancer",
              "crash",
              "devicehealth",
              "orchestrator",
              "pg_autoscaler",
              "progress",
              "rbd_support",   <--
              "status",
              "telemetry",
              "volumes"
   }
   ```

2. To display an "iotop" style of images:

   Example

   ```
   [user@mon ~]$ rbd perf image iotop
   ```

   Note: The write ops, read-ops, write-bytes, read-bytes, write-latency, and read-latency columns can be sorted dynamically by using the right and left arrow keys.

3. To display an "iostat" style of images:

   Example

   ```
   [user@mon ~]$ rbd perf image iostat
   ```

   Note: The output from this command can be in JSON or XML format, and then can be sorted using other command-line tools.

Ceph user and keyring

When `cephx` is enabled, you must specify a user name or ID and a path to the keyring containing the corresponding key for the user.

Note: `cephx` is enabled by default.

You might also add the `CEPH_ARGS` environment variable to avoid re-entry of the following parameters:

Syntax

```
rbd --id USER_ID --keyring=/path/to/secret [commands]
rbd --name USERNAME --keyring=/path/to/secret [commands]
```

Example

```
[root@rbd-client ~]# rbd --id admin --keyring=/etc/ceph/ceph.keyring [commands]
[root@rbd-client ~]# rbd --name client.admin --keyring=/etc/ceph/ceph.keyring [commands]
```

Tip: Add the user and secret to the `CEPH_ARGS` environment variable so that you do not need to enter them each time.

Live migration of images

As a storage administrator, you can live-migrate RBD images between different pools or even with the same pool, within the same storage cluster. You can migrate between different images formats or layouts and even from external data sources. When live migration is initiated, the source image is deep copied to the destination image, pulling all snapshot history while preserving the sparse allocation of data where possible.

Images with encryption support live migration.

Important: Currently, the `krbd` kernel module does not support live migration.

By default, during the live migration of the RBD images with the same storage cluster, the source image is marked read-only. All clients redirect the Input/Output (I/O) to the new target image. Additionally, this mode can preserve the link to the source image's parent to preserve sparseness, or it can flatten the image during the migration to remove the dependency on the source image's parent.

You can use the live migration process in an import-only mode, where the source image remains unmodified. You can link the target image to an external data source, such as a backup file, HTTP(s) file, or an S3 object or NBD export. The live migration copy process can safely run in the background while the new target image is being used.

The live migration process consists of three steps:

1. Prepare migration.
   The first step is to create new target image and link the target image to the source image. If the import-only mode is not configured, the source image will also be linked to the target image and marked read-only. Attempts to read uninitialized data extents within the target image will internally redirect the read to the source image, and writes to uninitialized extents within the target image will internally deep copy, the overlapping source image extents to the target image.

   For migration preparation instructions, see "Preparing for default live migration" on page 1276 or "Preparing import-only migration" on page 1278, according to your needs..

2. Run migration.
   This is a background operation that deep-copies all initialized blocks from the source image to the target. You can run this step when clients are actively using the new target image.

   For migration instructions, see "Running the live migration" on page 1280.

3. Finish migration.
   You can commit or stop (abort) the migration, once the background migration process is completed.

   Committing the migration removes the cross-links between the source and target images, and will remove the source image if not configured in the import-only mode. For committing instructions, see "Committing the live migration" on page 1281.

   Stopping the migration remove the cross-links, and will remove the target image. For instructions to stop the migration, see "Stopping a live migration" on page 1282.

Live migration formats

`Native`, `qcow`, and `raw` are supported live migration format types and layouts.

Use the following `source-spec` JSON example file for live migration.

```
{
 "type": "native",
```

```
 ["mon_host": "<mon-host>",]
 Specify if the image is in another cluster. Must be specified together
 with "key". Mutually exclusive with "cluster_name".

 ["key": "<key or key-ref>",]
 For connecting to another cluster. Must be specified together with
 "mon_host". Mutually exclusive with "cluster_name".

 ["cluster_name": "<cluster-name>",]
 Specify if the image is in another cluster. Requires a
 "<cluster-name>.conf" file. Mutually exclusive with "mon_host" and "key".

 ["client_name": "<client-name>",]
 For connecting to another cluster. Default: "client.admin".

 "pool_name": "<pool-name>",

 ["pool_id": <pool-id>,]
 Optional alternative to "pool_name".

 ["pool_namespace": "<pool-namespace>",]
 Optional.

 "image_name": "<image-name>",

 ["image_id": "<image-id>",]
 Specify if the image is in trash.

 "snap_name": "<snap-name>",

 ["snap_id": "<snap-id>",]
 Optional alternative to "snap_name".
}
```

`native` format

The `native` format does not include the stream object, since it utilizes native Ceph operations. For example, to import from the `rbd/ns1/image1@snap1` image, encode the `source-spec` as follows:

```
{
"type": "native",
"pool_name": "rbd",
"pool_namespace": "ns1",
"image_name": "image1",
"snap_name": "snap1"
}
```

`qcow` format

Use the `qcow` format to describe a QEMU copy-on-write (QCOW) block device. Both the QCOW v1 and v2 formats are currently supported with the exception of advanced features such as compression, encryption, backing files, and external data files.

Link the `qcow` format data to any supported stream source, by using the following `source-spec` file:

```
{
    "type": "qcow",
    "stream": {
      "type": "file",
  "file_path": "/mnt/image.qcow"
    }
}
```

`raw` format

Use the `raw` format to describe a thick-provisioned, raw block device export that is **`rbd export –export-format 1 SNAP_SPEC`**.

Link the `raw` format data to any supported stream source, by using the following `source-spec` file:

```
{
    "type": "raw",
    "stream": {
      "type": "file",
      "file_path": "/mnt/image-head.raw"
    },
    "snapshots": [
```

```
        {
            "type": "raw",
            "name": "snap1",
            "stream": {
              "type": "file",
       "file_path": "/mnt/image-snap1.raw"
            }
        },
    ] (optional oldest to newest ordering of snapshots)
}
```

The inclusion of the `snapshots` array is optional and currently only supports thick-provisioned raw snapshot exports.

Live migration streams

Use `file`, `HTTP`, `S3`, and `NBD` streams for live migration.

File stream

You can use the `file` stream to import from a locally accessible POSIX file source.

```
{
    <format unique parameters>
    "stream": {
        "type": "file",
        "file_path": "FILE_PATH"
    }
}
```

The following is a `source-spec` JSON file that imports a raw-format image from a file located at `/mnt/image.raw`.

```
{
    "type": "raw",
    "stream": {
        "type": "file",
        "file_path": "/mnt/image.raw"
    }
}
```

HTTP stream

You can use the `HTTP` stream to import from a remote HTTP or HTTPS web server.

```
{
    <format unique parameters>
    "stream": {
        "type": "http",
        "url": "URL_PATH"
    }
}
```

The following is a `source-spec` JSON file that imports a raw-format image from a file located at `http://download.ceph.com/image.raw`.

```
{
    "type": "raw",
    "stream": {
        "type": "http",
        "url": "http://download.ceph.com/image.raw"
    }
}
```

S3 stream

You can use the `s3` stream to import from a remote S3 bucket.

```
{
    <format unique parameters>
    "stream": {
        "type": "s3",
```

```
        "url": "URL_PATH",
        "access_key": "ACCESS_KEY",
        "secret_key": "SECRET_KEY"
    }
}
```

The following is a `source-spec` JSON file that imports a raw-format image from a file located at `http://s3.ceph.com/bucket/image.raw`.

```
{
    "type": "raw",
    "stream": {
        "type": "s3",
        "url": "http://s3.ceph.com/bucket/image.raw",
        "access_key": "NX5QOQKC6BH2IDN8HC7A",
        "secret_key": "LnEsqNNqZIpkzauboDcLXLcYaWwLQ3Kop0zAnKIn"
    }
}
```

NBD stream

You can use the `NBD` stream to import from a remote NBD export.

`nbd-uri` parameter must follow the NBD URI format. The default NBD port is `10809`.

For more information, see The NBD Uniform Resource Indicator (URI) format.

```
{
            <format unique parameters>
            "stream": {
                "type": "nbd",
                "uri": "<nbd-uri>",
            }
        }
```

The following is a `source-spec` JSON file that imports a raw-format image from an NBD export located at `nbd://nbd.ceph.com/image.raw`.

```
{
            "type": "raw",
            "stream": {
                "type": "nbd",
                "uri": "nbd://nbd.ceph.com/image.raw",
            }
        }
```

Preparing for default live migration

Prepare the default live migration process for RADOS Block Device (RBD) images within the same Red Hat Ceph Storage cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage.
- Two block device pools.
- One block device image.

About this task

The **`rbd migration prepare`** command accepts all the same layout options as the **`rbd create`** command. The **`rbd create`** command allows changes to the on-disk layout of the immutable image. If you only want to change the on-disk layout and want to keep the original image name, skip the `migration_target` argument. All clients using the source image must be stopped before preparing a live migration. The `prepare` step will fail if it finds any running clients with the image open in read/write mode. You can restart the clients using the new target image once the `prepare` step is completed.

Note: You cannot restart the clients using the source image as it results in a failure.

Important: Cloned images are implicitly flattened during live importing (using the **`--import-only`** and these images are disassociated from any parent chain in the source cluster when migrated to another Ceph cluster.

Procedure

1. Optional: If you are migrating the image from one Ceph cluster to another, copy the `ceph.conf` and `ceph.client.admin.keyring` of both the clusters to a common node. This ensures the client node has access to both clusters for migration.
   - Example of copying `ceph.conf` and `ceph.client.admin.keyring` of cluster `c1` to a common node.

     ```
     [root@rbd1-client /]# scp /etc/ceph/ceph.conf root@10.0.67.67:/etc/ceph/c1.conf
     root@10.0.67.67's password:
     ceph.conf
       100%  263     1.2MB/s   00:00

     [root@rbd1-client /]# scp /etc/ceph/ceph.client.admin.keyring  root@10.0.67.67:/
     etc/ceph/c1.keyring
     root@10.0.67.67's password:
     ceph.client.admin.keyring
     ```

   - Example of copying `ceph.conf` and `ceph.client.admin.keyring` of cluster `c2` to a common node.

     ```
     [root@rbd2-client]# scp /etc/ceph/ceph.conf root@10.0.67.67:/etc/ceph/c2.conf
     ceph.conf
       100%  261   864.5KB/s   00:00

     [root@rbd2-client]# scp /etc/ceph/ceph.client.admin.keyring root@10.0.67.67:/etc/
     ceph/c2.keyring
     root@10.0.67.67's password:
     ceph.client.admin.keyring
     ```

2. Prepare the live migration within the storage cluster or rename the source image.
   - Prepare the live migration.

     ```
     rbd migration prepare SOURCE_POOL_NAME/SOURCE_IMAGE_NAME TARGET_POOL_NAME/
     SOURCE_IMAGE_NAME
     ```

     For example,

     ```
     [ceph: root@rbd-client /]# rbd migration prepare sourcepool1/sourceimage1
     targetpool1/sourceimage1
     ```

   - Rename the source image.

     ```
     rbd migration prepare SOURCE_POOL_NAME/SOURCE_IMAGE_NAME TARGET_POOL_NAME/
     NEW_SOURCE_IMAGE_NAME
     ```

     In the following example, `newsourceimage1` is the renamed source image.

     ```
     [ceph: root@rbd-client /]# rbd migration prepare sourcepool1/sourceimage1
     targetpool1/newsourceimage1
     ```

What to do next

1. Check the current state of the live migration process.

   ```
   rbd status TARGET_POOL_NAME/SOURCE_IMAGE_NAME
   ```

   For example,

```
[ceph: root@rbd-client /]# rbd status targetpool1/sourceimage1
Watchers: none
Migration:
source: sourcepool1/sourceimage1 (adb429cb769a)
destination: targetpool2/testimage1 (add299966c63)
state: prepared
```

Important: During the migration process, the source image is moved into the RBD trash to prevent mistaken usage.
For example:

- ```
  [ceph: root@rbd-client /]# rbd info sourceimage1
  rbd: error opening image sourceimage1: (2) No such file or directory
  ```

- ```
  [ceph: root@rbd-client /]# rbd trash ls --all sourcepool1
  adb429cb769a sourceimage1
  ```

2. Run the live migration. For more information, see “Running the live migration” on page 1280.

Preparing import-only migration

You can initiate the import-only live migration process by running the **rbd migration prepare** command with the **--import-only** flag and either the **--source-spec** or **--source-spec-path** options. The import-only process is implemented by passing a JSON document that describes how to access the source image data directly on the command line or from a file.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- The destination pool exists.
- A bucket and an S3 object are created.

Procedure

1. Create a JSON file.
   For example,

   ```
   [ceph: root@rbd-client /]# cat testspec.json
    {
      "type": "raw",
       "stream": {
           "type": "s3",
           "url": "http:10.74.253.18:80/testbucket1/image.raw",
          "access_key": "RLJOCP6345BGB38YQXI5",
          "secret_key": "oahWRB2ote2rnLy4dojYjDrsvaBADriDDgtSfk6o"
     }
   ```

2. Prepare the import-only live migration process.

   ```
   rbd migration prepare --import-only --source-spec-path "JSON_FILE" TARGET_POOL_NAME/
   SOURCE_IMAGE_NAME
   ```

   For example,

   ```
   [ceph: root@rbd-client /]# rbd migration prepare --import-only --source-spec-path
   "testspec.json" targetpool1/sourceimage1
   ```

3. Create a JSON file that describes the source image.
   To import data from an S3 source:

```
{
  "type": "raw",
  "stream": {
    "type": "s3",
    "url": "http://10.74.253.18:80/testbucket1/image.raw",
    "access_key": "RLJOCP6345BGB38YQXI5",
    "secret_key": "oahWRB2ote2rnLy4dojYjDrsvaBADriDDgtSfk6o"
  }
}
```

To import data from an image in another Ceph cluster by using a cluster configuration file:

```
{
  "cluster_name": "c1",
  "type": "native",
  "pool_name": "pool1",
  "image_name": "image1",
  "snap_name": "snap1"
}
```

Alternatively, specify monitor endpoints and authentication credentials directly in the source-spec file:

```
{
  "type": "native",
  "mon_host": "10.0.0.11,10.0.0.12,10.0.0.13",
  "client_name": "client.migration",
  "key": "AQAz7EVWygILFRAAdIcuJ12opU/JKyfFmxhuaw==",
  "pool_name": "pool1",
  "image_name": "image1",
  "snap_name": "snap1"
}
```

To avoid storing authentication credentials directly in the source-spec file, store the key in the MON config-key store and reference it by using the `config://` prefix.

Store the key:

```
[ceph: root@rbd-client /]# ceph config-key set rbd/native/my_key
AQAz7EVWygILFRAAdIcuJ12opU/JKyfFmxhuaw==
```

Reference the stored key in the source-spec file:

```
{
  "type": "native",
  "mon_host": "10.0.0.11,10.0.0.12,10.0.0.13",
  "client_name": "client.migration",
  "key": "config://rbd/native/my_key",
  "pool_name": "pool1",
  "image_name": "image1",
  "snap_name": "snap1"
}
```

Important:

- Specify **`mon_host`** and **`key`** together.
- The **`cluster_name`** parameter is mutually exclusive with **`mon_host`** and **`key`**.
- If **`client_name`** is not specified, the default value is `client.admin`.

4. Prepare the import-only migration process.

```
rbd migration prepare --import-only --source-spec-path JSON_FILE TARGET_POOL_NAME/
SOURCE_IMAGE_NAME
```

For example,

```
[ceph: root@rbd-client /]# rbd migration prepare \
--import-only \
--source-spec-path /tmp/native_spec \
c2pool1/c2image1 \
--cluster c2
```

5. Run the migration.

```
[ceph: root@rbd-client /]# rbd migration execute c2pool1/c2image1 --cluster c2
Image migration: 100% complete...done.
```

6. Commit the migration.

```
[ceph: root@rbd-client /]# rbd migration commit c2pool1/c2image1 --cluster c2
Commit image migration: 100% complete...done.
```

What to do next

1. Check the current state of the import-only live migration process.

```
rbd status TARGET_POOL_NAME/SOURCE_IMAGE_NAME
```

For example,

```
[ceph: root@rbd-client /]# rbd status targetpool1/sourceimage1
Watchers: none
Migration:
source: {"stream":
{"access_key":"RLJOCP6345BGB38YQXI5","secret_key":"oahWRB2ote2rnLy4dojYjDrsvaBADriDDgt
Sfk6o","type":"s3","url":"http://10.74.253.18:80/testbucket1/image.raw"},"type":"raw"}
destination: targetpool1/sourceimage1 (b13865345e66)
state: prepared
```

The following example shows migrating data from Ceph cluster `c1` to Ceph cluster `c2`:

```
[ceph: root@rbd-client /]# cat /tmp/native_spec
{
  "cluster_name": "c1",
  "type": "native",
  "pool_name": "pool1",
  "image_name": "image1",
  "snap_name": "snap1"
}
ceph: root@rbd-client /]# rbd migration prepare --import-only --source-spec-path /tmp/
native_spec c2pool1/c2image1 --cluster c2
ceph: root@rbd-client /]# rbd migration execute c2pool1/c2image1 --cluster c2
Image migration: 100% complete...done.
ceph: root@rbd-client /]# rbd migration commit c2pool1/c2image1 --cluster c2
Commit image migration: 100% complete...done.
```

> **Note:** Use either the **`cluster_name`** parameter or the **`mon_host and key`** parameters to connect to the source cluster.

2. Run the live migration. For more information, see “Running the live migration” on page 1280.

Running the live migration

After you prepare for the live migration, you must copy the image blocks from the source image to the target image.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Two block device pools.

- One block device image with migration, as prepared in either "Preparing for default live migration" on page 1276 or "Preparing import-only migration" on page 1278.

About this task

> **Note:** The sub-commands help users copy the image blocks. You are not required to take any further action other than the **execute** command.

Procedure

- Run the live migration.

```
rbd migration execute TARGET_POOL_NAME/SOURCE_IMAGE_NAME
```

For example,

```
[ceph: root@rbd-client /]# rbd migration execute targetpool1/sourceimage1
Image migration: 100% complete...done.
```

What to do next

1. You can check the feedback on the progress of the migration block deep-copy process.

```
rbd status TARGET_POOL_NAME/SOURCE_IMAGE_NAME
```

For example,

```
[ceph: root@rbd-client /]# rbd status targetpool1/sourceimage1
Watchers: none
Migration:
source: sourcepool1/testimage1 (adb429cb769a)
destination: targetpool1/testimage1 (add299966c63)
state: executed
```

2. Commit the live migration. For more information, see "Committing the live migration" on page 1281.

Committing the live migration

You can commit the migration, once the live migration has completed deep-copying all the data blocks from the source image to the target image.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Two block device pools.
- One block device image, that was used while running the live migration.
- A completed migration block deep-copy process, in the `executed` state.

Procedure

- Commit the migration.

```
rbd migration commit TARGET_POOL_NAME/SOURCE_IMAGE_NAME
```

For example,

```
[ceph: root@rbd-client /]# rbd migration commit targetpool1/sourceimage1
Commit image migration: 100% complete...done.
```

Committing the live migration removes the cross-links between the source and target images and removes the source image from the source pool.

What to do next

To see which source images were removed from the source pool, use the **`trash list`** command.

```
rbd trash list --all SOURCE_POOL_NAME
```

For example,

```
[ceph: root@rbd-client /]# rbd trash list --all sourcepool1
```

Stopping a live migration

You can revert the live migration process.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Two block device pools.
- One block device image.

About this task

Stopping the live migration reverts the prepare and run steps.

Procedure

- Stop the live migration process.

  ```
  rbd migration abort TARGET_POOL_NAME/SOURCE_IMAGE_NAME
  ```

  For example,

  ```
  [ceph: root@rbd-client /]# rbd migration abort targetpool1/sourceimage1
  Abort image migration: 100% complete...done.
  ```

What to do next

After the live migration process is stopped, the target image is deleted and access to the original source image is restored in the source pool.

Verify that the source image is restored, by using the **`rbd ls`** command.

```
[ceph: root@rbd-client /]# rbd ls sourcepool1
sourceimage1
```

Image encryption

Learn how to set a secret key that is used to encrypt a specific RADOS Block Device (RBD) image. Image level encryption is handled internally by RBD clients.

> **Note:**
> - The `krbd` module does not support image level encryption.
> - You can use external tools such as `dm-crypt` or `QEMU` to encrypt an RBD image.

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- `root` level permissions.

Encryption format

Ceph Block Device images are not encrypted by default. You can encrypt an RBD image by formatting to one of the supported encryption formats. The format operation persists the encryption metadata to the RBD image. The encryption metadata includes information such as the encryption format and version, cipher algorithm and mode specifications, as well as the information used to secure the encryption key.

The encryption key is protected by a user kept secret that is a passphrase, which is never stored as persistent data in the RBD image. The encryption format operation requires you to specify the encryption format, cipher algorithm, and mode specification as well as a passphrase. The encryption metadata is stored in the RBD image, currently as an encryption header that is written at the start of the raw image. This means that the effective image size of the encrypted image would be lower than the raw image size.

Note:

- Currently you can only encrypt flat Ceph Block Device images. Clones of an encrypted RBD image are inherently encrypted using the same encryption profile and passphrase.
- Any data written to the RBD image before formatting might become unreadable, even though it might still occupy storage resources. Ceph Block Device images with the journal feature enabled cannot be encrypted.

Encryption load

By default, all RBD APIs treat encrypted Ceph Block Device images the same way as unencrypted Ceph Block Device images. You can read or write raw data anywhere in the image.

Writing raw data into the image might risk the integrity of the encryption format. For example, the raw data could override the encryption metadata located at the beginning of the image. To safely perform encrypted Input/Output(I/O) or maintenance operations on the encrypted RBD image, an additional encryption load operation must be applied immediately after opening the image.

The encryption load operation requires you to specify the encryption format and a passphrase. All I/Os for the opened RBD image are encrypted or decrypted, for a cloned RBD image, this includes I/Os for the parent images. The encryption key is stored in memory by the RBD client until the image is closed.

Note: Once the encryption is loaded on the RBD image, no other encryption load or format operation can be applied. Additionally, API calls for retrieving the RBD image size using the opened image context return the effective image size. The encryption is loaded automatically when mapping the Ceph Block Device images as block devices through `rbd-nbd`.

Note: API calls for retrieving the image size and the parent overlap using the opened image context returns the effective image size and the effective parent overlap.

Note: If a clone of an encrypted image is explicitly formatted, flattening or shrinking of the cloned image ceases to be transparent since the parent data must be re-encrypted according to the cloned image format as it is copied from the parent snapshot. If encryption is not loaded before the flatten operation is issued, any parent data that was previously accessible in the cloned image might become unreadable.

Note: If a clone of an encrypted image is explicitly formatted, the operation of shrinking the cloned image ceases to be transparent. This is because, in scenarios such as the cloned image containing snapshots or the cloned image being shrunk to a size that is not aligned with the object size, the action of copying some data from the parent snapshot, similar to flattening is involved. If encryption is not loaded before the shrink operation is issued, any parent data that was previously accessible in the cloned image might become unreadable.

Supported formats

Both Linux Unified Key Setup (LUKS) 1 and 2 are supported. The data layout is fully compliant with the LUKS specification. External LUKS compatible tools such as `dm-crypt` or `QEMU` can safely perform encrypted Input/Output (I/O) on encrypted RBD images. Additionally, you can import existing LUKS images created by external tools, by copying the raw LUKS data into the RBD image.

Currently, only Advanced Encryption Standards (AES) 128 and 256 encryption algorithms are supported. xts-plain64 is currently the only supported encryption mode.

To use the LUKS format, format the RBD image with the following command:

> **Note:** You need to create a file named `passphrase.txt` and enter a passphrase. You can randomly generate the passphrase, which might contain NULL characters. If the passphrase ends with a newline character, it will be stripped off.

```
rbd encryption format POOL_NAME/LUKS_IMAGE luks1|luks2 passphrase.txt
```

For example,

```
[ceph: root@host01 /]# rbd encryption format pool1/luksimage1 luks1 passphrase.txt
```

> **Note:** You can select either `luks1` or `luks2` encryption format.

The encryption format operation generates a LUKS header and writes it at the start of the RBD image. A single keyslot is appended to the header. The keyslot holds a randomly generated encryption key, and is protected by the passphrase read from the passphrase file. By default, AES-256 in xts-plain64 mode, which is the current recommended mode and the default for other LUKS tools, is used. Adding or removing additional passphrases is currently not supported natively, but can be achieved using LUKS tools such as `cryptsetup`. The LUKS header size can vary that is up to 136 MiB in LUKS, but it is usually upto 16 MiB, dependent on the version of `libcryptsetup` installed. For optimal performance, the encryption format will set the data offset to be aligned with the image object size. For example, expect a minimum overhead of 8 MiB if using an image configured with an 8 MiB object size.

In LUKS1, sectors, which are the minimal encryption units, are fixed at 512 bytes. LUKS2 supports larger sectors, and for better performance, the default sector size is set to the maximum of 4KiB. Writes which are either smaller than a sector, or are not aligned to a sector start, will trigger a guarded `read-modify-write` chain on the client, with a considerable latency penalty. A batch of such unaligned writes can lead to I/O races which will further deteriorate performance. Avoid using using RBD encryption in cases where incoming writes cannot be guaranteed to be LUKS sector aligned.

To map a LUKS encrypted image, run the following command:

```
rbd device map -t nbd -o encryption-format=luks1|luks2,encryption-passphrase-
file=passphrase.txt POOL_NAME/LUKS_IMAGE
```

For example,

```
[ceph: root@host01 /]# rbd device map -t nbd -o encryption-format=luks1,encryption-
passphrase-file=passphrase.txt pool1/luksimage1
```

> **Note:**
>
> - You can select either `luks1` or `luks2` encryption format.
> - For security reasons, both the encryption format and encryption load operations are CPU-intensive, and may take a few seconds to complete. For encrypted I/O, assuming AES-NI is enabled, a relative small microseconds latency might be added, as well as a small increase in CPU utilization.

Adding encryption format to images and clones

Add encryption format to images and clones with the rbd encryption format command. Given a LUKS2-formatted image, you can create both a LUKS2-formatted clone and a LUKS1-formatted clone.

Layered-client-side encryption is supported. The cloned images can be encrypted with their own format and passphrase, potentially different from that of the parent image.

Prerequisites

- A running Red Hat Ceph Storage cluster with RADOS Block Device (RBD) configured.
- Root-level access to the node.

Procedure

1. Create a LUKS2-formatted image:

   Syntax

   ```
   rbd create --size SIZE POOL_NAME/LUKS_IMAGE
   rbd encryption format POOL_NAME/LUKS_IMAGE luks1|luks2 PASSPHRASE_FILE
   rbd resize --size 50G --encryption-passphrase-file PASSPHRASE_FILE POOL_NAME/
   LUKS_IMAGE
   ```

   Example

   ```
   [ceph: root@host01 /]# rbd create --size 50G mypool/myimage
   [ceph: root@host01 /]# rbd encryption format mypool/myimage luks2 passphrase.txt
   [ceph: root@host01 /]# rbd resize --size 50G --encryption-passphrase-file
   passphrase.txt mypool/myimage
   ```

   The rbd resize command grows the image to compensate for the overhead associated with the LUKS2 header.

2. With the LUKS2-formatted image, create a LUKS2-formatted clone with the same effective size:

   Syntax

   ```
   rbd snap create POOL_NAME/IMAGE_NAME@SNAP_NAME
   rbd snap protect POOL_NAME/IMAGE_NAME@SNAP_NAME
   rbd clone POOL_NAME/IMAGE_NAME@SNAP_NAME POOL_NAME/CLONE_NAME
   rbd encryption format POOL_NAME/CLONE_NAME luks2 CLONE_PASSPHRASE_FILE
   ```

   Example

   ```
   [ceph: root@host01 /]# rbd snap create mypool/myimage@snap
   [ceph: root@host01 /]# rbd snap protect mypool/myimage@snap
   [ceph: root@host01 /]# rbd clone mypool/myimage@snap mypool/myclone
   [ceph: root@host01 /]# rbd encryption format mypool/myclone luks2 clone-
   passphrase.bin
   ```

3. With the LUKS2-formatted image, create a LUKS1-formatted clone with the same effective size:

   Syntax

   ```
   rbd snap create POOL_NAME/IMAGE_NAME@SNAP_NAME
   rbd snap protect POOL_NAME/IMAGE_NAME@SNAP_NAME
   rbd clone POOL_NAME/IMAGE_NAME@SNAP_NAME POOL_NAME/CLONE_NAME
   rbd encryption format POOL_NAME/CLONE_NAME luks1 CLONE_PASSPHRASE_FILE
   rbd resize --size SIZE --allow-shrink --encryption-passphrase-file
   CLONE_PASSPHRASE_FILE --encryption-passphrase-file PASSPHRASE_FILE POOL_NAME/
   CLONE_NAME
   ```

   Example

   ```
   [ceph: root@host01 /]# rbd snap create mypool/myimage@snap
   [ceph: root@host01 /]# rbd snap protect mypool/myimage@snap
   [ceph: root@host01 /]# rbd clone mypool/myimage@snap mypool/myclone
   [ceph: root@host01 /]# rbd encryption format mypool/myclone luks1 clone-
   ```

```
passphrase.bin
 [ceph: root@host01 /]# rbd resize --size 50G --allow-shrink --encryption-passphrase-
file clone-passphrase.bin --encryption-passphrase-file passphrase.bin mypool/myclone
```

Since LUKS1 header is usually smaller than LUKS2 header, the rbd resize command at the end shrinks the cloned image to get rid of unwanted space allowance.

4. With the LUKS-1-formatted image, create a LUKS2-formatted clone with the same effective size:

   Syntax

```
 rbd resize --size SIZE POOL_NAME/LUKS_IMAGE
 rbd snap create POOL_NAME/IMAGE_NAME@SNAP_NAME
 rbd snap protect POOL_NAME/IMAGE_NAME@SNAP_NAME
 rbd clone POOL_NAME/IMAGE_NAME@SNAP_NAME POOL_NAME/CLONE_NAME
 rbd encryption format POOL_NAME/CLONE_NAME luks2 CLONE_PASSPHRASE_FILE
 rbd resize --size SIZE --allow-shrink --encryption-passphrase-file PASSPHRASE_FILE
POOL_NAME/LUKS_IMAGE
 rbd resize --size SIZE --allow-shrink --encryption-passphrase-file
CLONE_PASSPHRASE_FILE --encryption-passphrase-file PASSPHRASE_FILE POOL_NAME/
CLONE_NAME
```

   Example

```
 [ceph: root@host01 /]# rbd resize --size 51G mypool/myimage
 [ceph: root@host01 /]# rbd snap create mypool/myimage@snap
 [ceph: root@host01 /]# rbd snap protect mypool/myimage@snap
 [ceph: root@host01 /]# rbd clone mypool/my-image@snap mypool/myclone
 [ceph: root@host01 /]# rbd encryption format mypool/myclone luks2 clone-
passphrase.bin
 [ceph: root@host01 /]# rbd resize --size 50G --allow-shrink --encryption-passphrase-
file passphrase.bin mypool/myimage
 [ceph: root@host01 /]# rbd resize --size 50G --allow-shrink --encryption-passphrase-
file clone-passphrase.bin --encryption-passphrase-file passphrase.bin mypool/myclone
```

   Since LUKS2 header is usually bigger than LUKS1 header, the rbd resize command at the beginning temporarily grows the parent image to reserve some extra space in the parent snapshot and consequently the cloned image. This is necessary to make all parent data accessible in the cloned image. The rbd resize command at the end shrinks the parent image back to its original size and does not impact the parent snapshot and the cloned image to get rid of the unused reserved space.

   The same applies to creating a formatted clone of an unformatted image, since an unformatted image does not have a header at all.

Managing snapshots

A snapshot is a read-only copy of the state of an image at a particular point in time. As a storage administrator, being familiar with Ceph's snapshotting feature can help you manage the snapshots and clones of images stored in the Red Hat Ceph Storage cluster.

One of the advanced features of Ceph Block Devices is that you can create snapshots of the images to retain a history of an image's state. Ceph also supports snapshot layering, which allows you to clone images quickly and easily, for example a virtual machine image. Ceph supports block device snapshots using the `rbd` command and many higher level interfaces, including `QEMU`, `libvirt`, OpenStack and CloudStack.

> **Note:** If a snapshot is taken while `I/O` is occurring, then the snapshot might not get the exact or latest data of the image and the snapshot might have to be cloned to a new image to be mountable. Stop any `I/O` usage before taking a snapshot of an image. If the image contains a filesystem, the filesystem must be in a consistent state before taking a snapshot. To stop `I/O` you can use `fsfreeze` command. For virtual machines, the `qemu-guest-agent` can be used to automatically freeze filesystems when creating a snapshot.

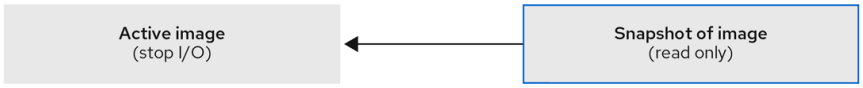


Figure 96: Ceph Block device snapshots

For more information see the `fsfreeze(8)` man page for more details.

Creating snapshots

Create a snapshot of a Ceph Block Device.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

1. Specify the `snap create` option, the pool name, and the image name:
   - Method 1:

     Syntax

     ```
     rbd --pool POOL_NAME snap create --snap SNAP_NAME IMAGE_NAME
     ```

     Example

     ```
     [root@rbd-client ~]# rbd --pool pool1 snap create --snap snap1 image1
     ```

   - Method 2:

     Syntax

     ```
     rbd snap create POOL_NAME/IMAGE_NAME@SNAP_NAME
     ```

     Example

     ```
     [root@rbd-client ~]# rbd snap create pool1/image1@snap1
     ```

Listing snapshots

List the block device snapshots.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

1. Specify the pool name and the image name:

   Syntax

   ```
   rbd --pool POOL_NAME --image IMAGE_NAME snap ls
   rbd snap ls POOL_NAME/IMAGE_NAME
   ```

   Example

```
[root@rbd-client ~]# rbd --pool pool1 --image image1 snap ls
[root@rbd-client ~]# rbd snap ls pool1/image1
```

Roll back snapshots

Learn how to rollback a block device snapshot.

> **Note:** Rolling back an image to a snapshot means overwriting the current version of the image with data from a snapshot. The time it takes to run a rollback increases with the size of the image. It is **faster to clone** from a snapshot **than to rollback** an image to a snapshot, and it is the preferred method of returning to a pre-existing state.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

1. Specify the `snap rollback` option, the pool name, the image name and the snap name:

   Syntax

   ```
   rbd --pool POOL_NAME snap rollback --snap SNAP_NAME IMAGE_NAME
   rbd snap rollback POOL_NAME/IMAGE_NAME@SNAP_NAME
   ```

   Example

   ```
   [root@rbd-client ~]# rbd --pool pool1 snap rollback --snap snap1 image1
   [root@rbd-client ~]# rbd snap rollback pool1/image1@snap1
   ```

Deleting snapshot

Delete a snapshot for Ceph Block Devices.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

1. To delete a block device snapshot, specify the `snap rm` option, the pool name, the image name and the snapshot name:

   Syntax

   ```
   rbd --pool POOL_NAME snap rm --snap SNAP_NAME IMAGE_NAME
   rbd snap rm POOL_NAME-/IMAGE_NAME@SNAP_NAME
   ```

   Example

   ```
   [root@rbd-client ~]# rbd --pool pool1 snap rm --snap snap2 image1
   [root@rbd-client ~]# rbd snap rm pool1/image1@snap1
   ```

Important: If an image has any clones, the cloned images retain reference to the parent image snapshot. To delete the parent image snapshot, you must flatten the child images first.

Note: Ceph OSD daemons delete data asynchronously, so deleting a snapshot does not free up the disk space immediately.

Reference

For more information, see "Flattening cloned images" on page 1294.

Purging snapshots

Purge block device snapshots.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

About this task

Note: Purging a snapshot means deleting all snapshots in the image, where deleting a snapshot refers to removing a specific snapshot from the image.

Procedure

- Specify the **snap purge** option and the image name on a specific pool.

```
rbd --pool POOL_NAME snap purge IMAGE_NAME
rbd snap purge POOL_NAME/IMAGE_NAME
```

For example,

```
[root@rbd-client ~]# rbd --pool pool1 snap purge image1
[root@rbd-client ~]# rbd snap purge pool1/image1
```

Renaming snapshots

Rename a block device snapshot.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

1. To rename a snapshot:

   Syntax

```
rbd snap rename POOL_NAME/IMAGE_NAME@ORIGINAL_SNAPSHOT_NAME POOL_NAME/
IMAGE_NAME@NEW_SNAPSHOT_NAME
```

Example

```
[root@rbd-client ~]# rbd snap rename data/dataset@snap1 data/dataset@snap2
```

This renames `snap1` snapshot of the `dataset` image on the `data` pool to `snap2`.

2. Run the `rbd help snap rename` command to display additional details on renaming snapshots.

Ceph Block Device layering

Ceph supports the ability to create many copy-on-write (COW) or copy-on-read (COR) clones of a block device snapshot. Snapshot layering enables Ceph Block Device clients to create images very quickly.

For example, you might create a block device image with a Linux VM written to it. Then, snapshot the image, protect the snapshot, and create as many clones as you like. A snapshot is read-only, so cloning a snapshot simplifies semantics—making it possible to create clones rapidly.

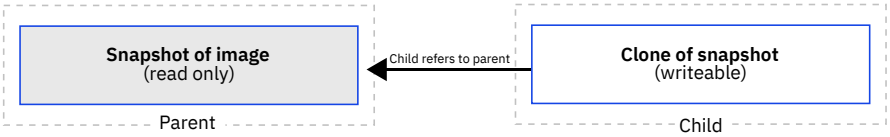


Figure 97: Ceph Block Device layering

> **Note:** The terms `parent` and `child` mean a Ceph Block Device snapshot, parent, and the corresponding image cloned from the snapshot, child. These terms are important for the command line usage below.

Each cloned image, the child, stores a reference to its parent image, which enables the cloned image to open the parent snapshot and read it. This reference is removed when the clone is `flattened` that is, when information from the snapshot is completely copied to the clone.

A clone of a snapshot behaves exactly like any other Ceph Block Device image. You can read to, write from, clone, and resize the cloned images. There are no special restrictions with cloned images. However, the clone of a snapshot refers to the snapshot, so you **MUST** protect the snapshot before you clone it.

A clone of a snapshot can be a copy-on-write (COW) or copy-on-read (COR) clone. Copy-on-write (COW) is always enabled for clones while copy-on-read (COR) has to be enabled explicitly. Copy-on-write (COW) copies data from the parent to the clone when it writes to an unallocated object within the clone. Copy-on-read (COR) copies data from the parent to the clone when it reads from an unallocated object within the clone. Reading data from a clone will only read data from the parent if the object does not yet exist in the clone. Rados block device breaks up large images into multiple objects. The default is set to 4 MB and all copy-on-write (COW) and copy-on-read (COR) operations occur on a full object, that is writing 1 byte to a clone will result in a 4 MB object being read from the parent and written to the clone if the destination object does not already exist in the clone from a previous COW/COR operation.

Whether or not copy-on-read (COR) is enabled, any reads that cannot be satisfied by reading an underlying object from the clone will be rerouted to the parent. Since there is practically no limit to the number of parents, meaning that you can clone a clone, this reroute continues until an object is found or you hit the base parent image. If copy-on-read (COR) is enabled, any reads that fail to be satisfied directly from the clone result in a full object read from the parent and writing that data to the clone so that future reads of the same extent can be satisfied from the clone itself without the need of reading from the parent.

This is essentially an on-demand, object-by-object flatten operation. This is specially useful when the clone is in a high-latency connection away from it's parent, that is the parent in a different pool, in another geographical location. Copy-on-read (COR) reduces the amortized latency of reads. The first few reads will have high latency because it will result in extra data being read from the parent, for example, you read 1 byte from the clone but now 4 MB has to be read from the parent and written to the clone, but all future reads will be served from the clone itself.

To create copy-on-read (COR) clones from snapshot you have to explicitly enable this feature by adding `rbd_clone_copy_on_read = true` under `[global]` or `[client]` section in the `ceph.conf` file.

Reference

- For more information on `flattening`, see “Flattening cloned images” on page 1294.

Protecting snapshots

Learn how to protect a block device snapshot from being accidentally deleted when deleting the parent snapshot.

Clones access the parent snapshots. All clones would break if a user inadvertently deleted the parent snapshot.

You can set the `set-require-min-compat-client` parameter to greater than or equal to mimic versions of Ceph.

Example

```
ceph osd set-require-min-compat-client mimic
```

This creates clone v2, by default. However, clients older than mimic cannot access those block device images.

> **Note:** Clone v2 does not require protection of snapshots.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

1. Specify `POOL_NAME`, `IMAGE_NAME`, and `SNAP_SHOT_NAME` in the following command:

   Syntax

   ```
   rbd --pool POOL_NAME snap protect --image IMAGE_NAME --snap SNAPSHOT_NAME
   rbd snap protect POOL_NAME/IMAGE_NAME@SNAPSHOT_NAME
   ```

   Example

   ```
   [root@rbd-client ~]# rbd --pool pool1 snap protect --image image1 --snap snap1
   [root@rbd-client ~]# rbd snap protect pool1/image1@snap1
   ```

   > **Note:** You cannot delete a protected snapshot.

Cloning block device snapshot

Cloning block device snapshots provides an efficient way to create writable copies of existing block device snapshots without duplicating data. You can either clone a single block device snapshot or a group block device snapshot based on your requirement.

Cloning a single block device snapshot

Clone a block device snapshot to create a read or write child image of the snapshot within the same pool or in another pool. One use case would be to maintain read-only images and snapshots as templates in one pool, and writable clones in another pool.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.

- Root-level access to the node.

About this task

> **Note:** Clone v2 does not require protection of snapshots.

Procedure

- Clone a snapshot.
  To clone a snapshot, specify the parent pool, snapshot, child pool, and image name.

```
rbd clone --pool POOL_NAME --image PARENT_IMAGE --snap SNAP_NAME --dest-pool POOL_NAME
--dest CHILD_IMAGE_NAME
rbd clone POOL_NAME/PARENT_IMAGE@SNAP_NAME  POOL_NAME/CHILD_IMAGE_NAME
```

  For example,

```
[root@rbd-client ~]# rbd clone --pool pool1 --image image1 --snap snap1 --dest-pool
pool1 --dest childimage1
[root@rbd-client ~]# rbd clone pool1/image1@snap1 pool1/childimage1
```

Cloning block device group snapshot

You can clone new groups from group snapshots created with the **`rbd group snap create`** command with the latest **`--snap-id`** option for the **`rbd clone`** command.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.
- A group snapshot.

About this task

Cloning from group snapshots is supported only with clone v2 `--rbd-default-clone-format 2`.

```
[root@rbd-client]# ] rbd clone --snap-id 4 pool1/image1 pool1/i1clone1 --rbd-default-clone-
format 2
```

Procedure

1. Get the snap ID of the group snapshot.

```
rbd snap ls --all POOL_NAME/PARENT_IMAGE_NAME
```

   The following example has a group snapshot, indicated with `SNAPID` listed as **4** and the `NAMESPACE` as `group`.

```
[root@rbd-client] # rbd snap ls --all pool1/image1
SNAPID     NAME             SIZE     PROTECTED   TIMESTAMP
 NAMESPACE
3          snap1            10 GiB   yes         Thu Jul 25 06:21:33 2024     user
4          .group.2_39d     10 GiB               Wed Jul 31 02:28:49 2024     group
(pool1/group1@p1g1snap1)
```

2. Create a clone of the group snapshot using the **`--snap-id`** option.

```
rbd clone --snap-id <SNAP_ID> POOL_NAME/IMAGE_NAME POOL_NAME/CLONE_IMAGE_NAME --rbd-
default-clone-format 2
```

```
[root@rbd-client]# rbd clone --snap-id 4 pool1/image1 pool2/clone2 --rbd-default-
clone-format 2
```

What to do next

Use the **`rbd ls`** command to verify the clone image of the group snapshot is created successfully.

```
[root@rbd-client]# rbd ls -p pool2
clone2
```

Unprotecting snapshots

Before you can delete a snapshot, you must unprotect it first. Additionally, you may *NOT* delete snapshots that have references from clones. You must flatten each clone of a snapshot, before you can delete the snapshot.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

1. Run the following commands:

   Syntax

   ```
   rbd --pool POOL_NAME snap unprotect --image IMAGE_NAME --snap SNAPSHOT_NAME
   rbd snap unprotect POOL_NAME/IMAGE_NAME@SNAPSHOT_NAME
   ```

   Example

   ```
   [root@rbd-client ~]# rbd --pool pool1 snap unprotect --image image1 --snap snap1

   [root@rbd-client ~]# rbd snap unprotect pool1/image1@snap1
   ```

Listing children of a snapshot

List the children of a snapshot.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

1. To list the children of a snapshot, run the following:

   Syntax

   ```
   rbd --pool POOL_NAME children --image IMAGE_NAME --snap SNAP_NAME
   rbd children POOL_NAME/IMAGE_NAME@SNAPSHOT_NAME
   ```

   Example

   ```
   [root@rbd-client ~]# rbd --pool pool1 children --image image1 --snap snap1
   [root@rbd-client ~]# rbd children pool1/image1@snap1
   ```

Flattening cloned images

Cloned images retain a reference to the parent snapshot. When you remove the reference from the child clone to the parent snapshot, you flatten the image by copying the data from the snapshot to the clone. The time required to flatten a clone increases with the size of the snapshot. Because a flattened image contains all the data from the snapshot, the flattened image uses more storage space than a layered clone.

> **Note:** If the `deep flatten` feature is enabled on an image, the image clone is dissociated from its parent by default.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

> **Warning:**
> Stop all client workloads on the cloned image before you flatten the image. The image can remain mapped. However, flattening an image during live, high-volume I/O can cause excessive I/O wait times while the flattening operation and the RBD client switch the exclusive lock.
>
> If I/O for a kernel RBD client stalls for more than 30 seconds, the client can be blocklisted. Blocklisting can make all RBD images that are mapped on the affected node unavailable. You must restart the node to restore access to the mapped images. Perform the flattening operation during a maintenance window.

1. To delete a parent image snapshot that is associated with child images, flatten the child images first.

   Syntax

   ```
   rbd --pool POOL_NAME flatten --image IMAGE_NAME
   ```

   Alternatively, use the following syntax:

   ```
   rbd flatten POOL_NAME/IMAGE_NAME
   ```

   Example

   ```
   [root@rbd-client ~]# rbd --pool pool1 flatten --image childimage1
   ```

   Alternatively, use the following command:

   ```
   [root@rbd-client ~]# rbd flatten pool1/childimage1
   ```

Mirroring Ceph Block Devices

As a storage administrator, you can add another layer of redundancy to Ceph Block Devices by mirroring data images between Red Hat Ceph Storage clusters. Understanding and using Ceph Block Device mirroring can provide you protection against data loss, such as a site failure.

The `rbd-mirror` daemon can run either on a single Ceph storage cluster for one-way mirroring or two Ceph storage clusters for two-way mirroring.

The `rbd-mirror` daemon is responsible for synchronizing images from one Ceph storage cluster to another Ceph storage cluster by pulling changes from the remote primary image and writing those changes to the local, non-primary image. You can configure Ceph Block Device mirroring either by one-way or two-way mirroring. These mirroring configurations can be done either on pools or on individual images.

For Ceph Block Device mirroring to work, either by using one-way or two-way replication, a couple of assumptions are made:

- A pool with the same name exists on both storage clusters.
- For journal-based mirroring, a pool contains journal-enabled images that you want to mirror.

> **Important:** In one-way or two-way replication, each instance of `rbd-mirror` must be able to connect to the other Ceph storage cluster simultaneously. Also, the network must have sufficient bandwidth between the two data center sites to handle mirroring.

Mirroring is configured on a per-pool basis with mirror peering storage clusters. Red Hat Ceph Storage supports two mirroring modes, depending on the type of images in the pool.

Pool mode

All images in a pool with the journaling feature enabled are mirrored.

Image mode

Only a specific subset of images within a pool are mirrored. You must enable mirroring for each image separately.

Whether or not an image can be modified depends on its state:

- You can modify images in the primary state.
- You can not modify images in the non-primary state.

Images are automatically promoted to primary when mirroring is enabled on an image. The promotion can happen:

- Implicitly by enabling mirroring in pool mode and only for journal-based images.
- Explicitly by enabling the mirroring of a specific image.

It is possible to demote primary images and promote non-primary images.

One-way replication

One-way mirroring implies that a primary image or pool of images in one storage cluster gets replicated to a secondary storage cluster. One-way mirroring also supports replicating to multiple secondary storage clusters.

On the secondary storage cluster, the image is the non-primary replicate; that is, Ceph clients cannot write to the image. When data is mirrored from a primary storage cluster to a secondary storage cluster, the `rbd-mirror` runs only on the secondary storage cluster.

For one-way mirroring to work, a couple of assumptions are made:

- You have two Red Hat Ceph Storage storage clusters and you want to replicate images from a primary storage cluster to a secondary storage cluster.
- The secondary storage cluster contains the `rbd-mirror` daemon that can run on one of the cluster nodes. The `rbd-mirror` daemon connects to the primary storage cluster to sync images to the secondary storage cluster.

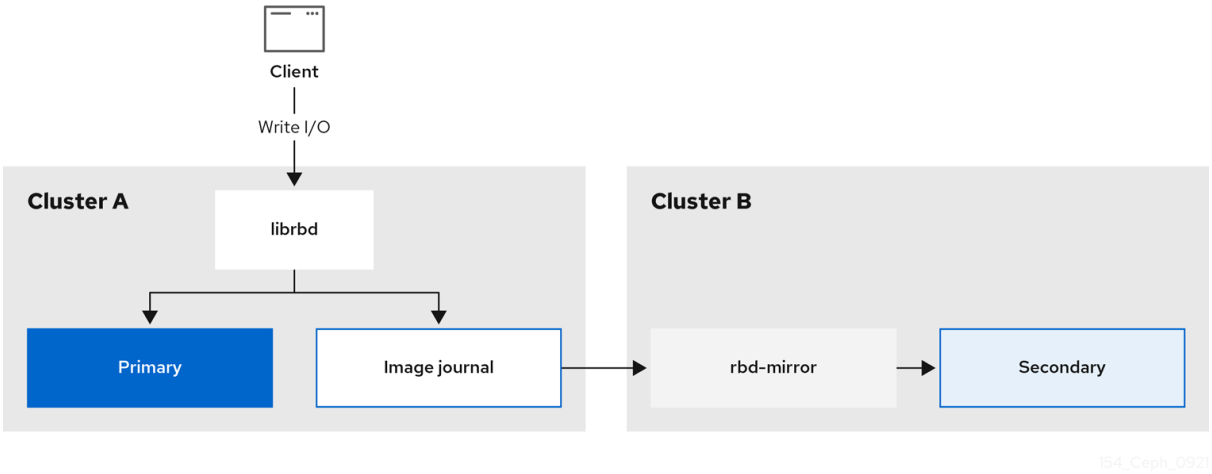


Figure 98: One-way mirroring

Two-way replication

Two-way replication adds an `rbd-mirror` daemon on the primary cluster so images can be demoted on it and promoted on the secondary cluster. Changes can then be made to the images on the secondary cluster and they will be replicated in the reverse direction, from secondary to primary. Both clusters must have `rbd-mirror` running to allow promoting and demoting images on either cluster. Currently, two-way replication is only supported between two sites.

For two-way mirroring to work, a couple of assumptions are made:

- You have two storage clusters and you want to be able to replicate images between them in either direction.
- Both storage clusters have `rbd-mirror` daemon running. The images that are primary on cluster1 are synced by the `rbd-mirror` daemon on the remote cluster and the images on the remote cluster are synced to the primary.

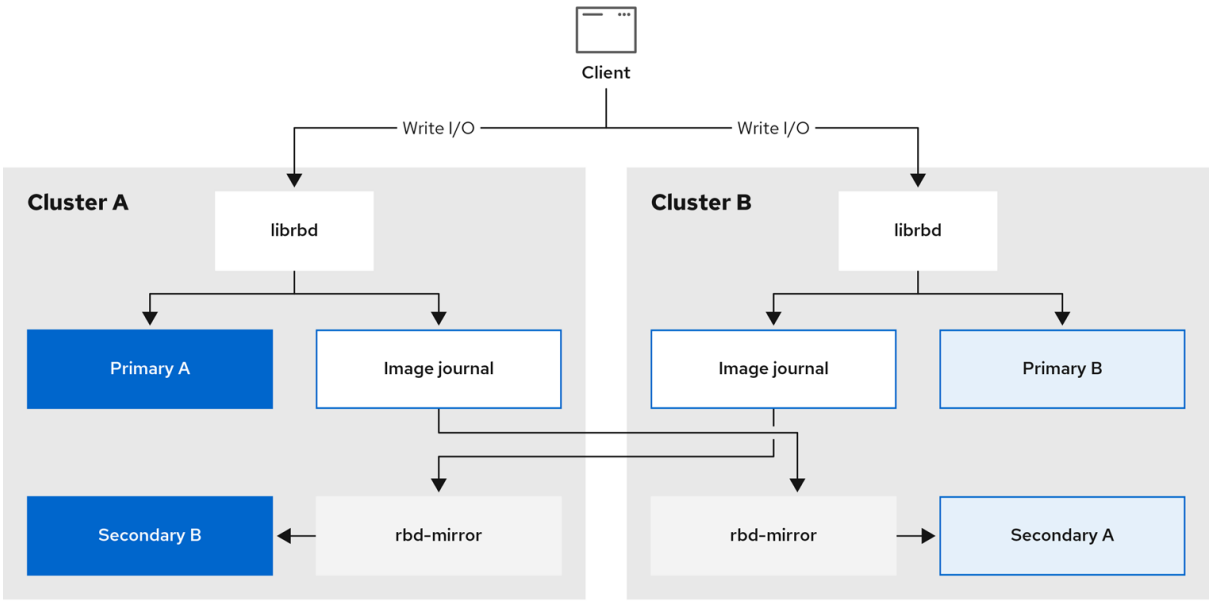


Figure 99: Two-way mirroring

Configuring Ceph Block Device mirroring

Configure one-way and two-way replication of a Ceph Block Device (`rbd`) pool to enable data synchronization between the primary storage cluster and a secondary cluster by using the command-line interface.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A minimum of two running Red Hat Ceph Storage clusters.
- Root-level access to a Ceph client node for each storage cluster.
- A CephX user with administrator-level capabilities.

About this task

Note:

- When using one-way replication, you can mirror to multiple secondary storage clusters.
- When using two-way replication you can only mirror between two storage clusters.

Examples in this section distinguish between two storage clusters by referring to the primary storage cluster with the primary images as `site-a`, and the secondary storage cluster you are replicating the images to, as `site-b`. The pool name that is used in these examples is called `data`.

Journal-based mirroring

Configure asynchronous replication for a block image using Ceph Block Device journaling to ensure near real-time, crash-consistent synchronization between clusters.

Procedure

1. Log in to the `cephadm` shell on both the sites.
   For example,

   ```
   [root@site-a ~]# cephadm shell
   [root@site-b ~]# cephadm shell
   ```

2. On `site-b`, schedule the deployment of mirror daemon on the secondary cluster.

   Note:

   - For two-way replication, repeat this step on site-a.
   - For one-way replication, continue to step “3” on page 1297.

   ```
   ceph orch apply rbd-mirror --placement=NODENAME
   ```

   Where *NODENAME* is the host where you want to configure mirroring in the secondary cluster.

   For example,

   ```
   [ceph: root@site-b /]# ceph orch apply rbd-mirror --placement=host04
   ```

3. Enable journaling features on an image on `site-a`.
   a. Use the **`--image-feature`** option for new images.

      ```
      rbd create IMAGE_NAME --size MEGABYTES --pool POOL_NAME --image-feature FEATURE
      FEATURE
      ```

> **Note:** If `exclusive-lock` is already enabled, use `journaling` as the only argument, else it returns the following error:`one or more requested features are already enabled (22) Invalid argument`

For example,

```
[ceph: root@site-a /]# rbd create image1 --size 1024 --pool data --image-feature
exclusive-lock,journaling
```

b. Use the **`rbd feature enable`** command for existing images.
Syntax,

```
rbd feature enable POOL_NAME/IMAGE_NAME FEATURE, FEATURE
```

For example,

```
[ceph: root@site-a /]# rbd feature enable data/image1 exclusive-lock, journaling
```

c. Enable journaling on all new images by default.
Set the configuration parameter by using the **`ceph config set`** command.
For example,

```
[ceph: root@site-a /]# ceph config set global rbd_default_features 125
[ceph: root@site-a /]# ceph config show mon.host01 rbd_default_features
```

4. Choose the mirroring mode on both the storage clusters.
The mode can be either`pool`or`image`.

```
rbd mirror pool enable POOL_NAME MODE
```

Use one of the following options, enabling either with pool or image, based on your requirements.

- The following example enables mirroring of the whole pool named `data`.

```
[ceph: root@site-a /]# rbd mirror pool enable data pool
[ceph: root@site-b /]# rbd mirror pool enable data pool
```

- The following example enables image mode mirroring on the pool named `data`.

```
[ceph: root@site-a /]# rbd mirror pool enable data image
[ceph: root@site-b /]# rbd mirror pool enable data image
```

5. Verify that mirroring is successfully enabled at both the sites.

```
rbd mirror pool info POOL_NAME
```

For example,

```
[ceph: root@site-a /]# rbd mirror pool info data
Mode: pool
Site Name: c13d8065-b33d-4cb5-b35f-127a02768e7f

Peer Sites: none

[ceph: root@site-b /]# rbd mirror pool info data
Mode: pool
Site Name: a4c667e2-b635-47ad-b462-6faeeee78df7

Peer Sites: none
```

6. On the Ceph client node, bootstrap the storage cluster peers.
   a. Create Ceph user accounts, and register the storage cluster peer to the pool.

```
rbd mirror pool peer bootstrap create --site-name PRIMARY_LOCAL_SITE_NAME
POOL_NAME > PATH_TO_BOOTSTRAP_TOKEN
```

The following example bootstrap command creates the `client.rbd-mirror.site-a` and the `client.rbd-mirror-peer` Ceph users.

```
[ceph: root@rbd-client-site-a /]# rbd mirror pool peer bootstrap create --site-
name site-a data > /root/bootstrap_token_site-a
```

b. Copy the bootstrap token file to the `site-b` storage cluster.

c. Import the bootstrap token on the `site-b` storage cluster.

```
bd mirror pool peer bootstrap import --site-name SECONDARY_LOCAL_SITE_NAME --
direction rx-only POOL_NAME PATH_TO_BOOTSTRAP_TOKEN
```

For example,

```
[ceph: root@rbd-client-site-b /]# rbd mirror pool peer bootstrap import --site-
name site-b --direction rx-only data /root/bootstrap_token_site-a
```

What to do next

Continue with verifying the mirroring status, see "What to do next" on page 1301.

Snapshot-based mirroring

Set up periodic replication for a block image by creating and syncing snapshots to provide point-in-time consistency across clusters.

Procedure

1. Log in to the `cephadm` shell on both the sites.
   For example,

```
[root@site-a ~]# cephadm shell
[root@site-b ~]# cephadm shell
```

2. On `site-b`, schedule the deployment of mirror daemon on the secondary cluster.

> **Note:**
> - For two-way replication, repeat this step on site-a.
> - For one-way replication, continue to step "3" on page 1299.

```
ceph orch apply rbd-mirror --placement=NODENAME
```

Where *NODENAME* is the host where you want to configure mirroring in the secondary cluster.

For example,

```
[ceph: root@site-b /]# ceph orch apply rbd-mirror --placement=host04
```

3. Enable image mode mirroring on the pool.

```
rbd mirror pool enable POOL_NAME MODE
```

The following example enables image mode mirroring on the pool named `data`.

```
[ceph: root@site-a /]# rbd mirror pool enable data image
[ceph: root@site-b /]# rbd mirror pool enable data image
```

4. Verify that mirroring is successfully enabled at both the sites.

```
rbd mirror pool info POOL_NAME
```

For example,

```
[ceph: root@site-a /]# rbd mirror pool data
Mode: pool
Site Name: c13d8065-b33d-4cb5-b35f-127a02768e7f
Mirror UUID: be874515-c563-4bb0-a24a-962a2784c665
Remote Namesapace:

Peer Sites: none

[ceph: root@site-b /]# rbd mirror pool info data
Mode: pool
Site Name: a4c667e2-b635-47ad-b462-6faeeee78df7
Mirror UUID: be874515-c563-4bb0-a24a-962a2784c665

Peer Sites: none
```

5. On the Ceph client node, bootstrap the storage cluster peers.

> **Note:** For two-way mirroring, repeat these substeps in opposite order. First creating a bootstrap on `site-b` and then importing on `site-a`.

   a. Create Ceph user accounts, and register the storage cluster peer to the pool.

   ```
   rbd mirror pool peer bootstrap create --site-name PRIMARY_LOCAL_SITE_NAME
   POOL_NAME > PATH_TO_BOOTSTRAP_TOKEN
   ```

   The following example bootstrap command creates the `client.rbd-mirror.site-a` and the `client.rbd-mirror-peer` Ceph users.

   ```
   [ceph: root@rbd-client-site-a /]# rbd mirror pool peer bootstrap create --site-
   name site-a data > /root/bootstrap_token_site-a
   ```

   b. Copy the bootstrap token file to the `site-b` storage cluster.

   c. Import the bootstrap token on the `site-b` storage cluster.

   ```
   rbd mirror pool peer bootstrap import --site-name SECONDARY_LOCAL_SITE_NAME --
   direction rx-only POOL_NAME PATH_TO_BOOTSTRAP_TOKEN
   ```

   For example,

   ```
   [ceph: root@rbd-client-site-b /]# rbd mirror pool peer bootstrap import --site-
   name site-b --direction rx-only data /root/bootstrap_token_site-a
   ```

6. On the Primary site, create a new Ceph Block Device image in a specific pool.

```
rbd create POOL_NAME/IMAGE_NAME --size SIZE
```

For example,

```
[ceph: root@rbd-client-site-a /]# rbd create data/image1 --size 1024
```

7. Enable snapshot image mirroring.

```
rbd mirror image enable POOL_NAME/IMAGE_NAME snapshot
```

For example,

```
[ceph: root@rbd-client-site-a /]# rbd mirror image enable data/image1 snapshot
```

8. Create a snapshot mirror schedule.

```
rbd --cluster CLUSTER_NAME mirror snapshot schedule add --pool POOL_NAME --image
IMAGE_NAME INTERVAL
```

For more information, see “Scheduling mirror snapshots” on page 1313.

For example,

```
[ceph: root@rbd-client-site-a /]# rbd --cluster site-a mirror snapshot schedule add --
pool data --image image1 6h
```

What to do next

Continue with verifying the mirroring status, see "What to do next" on page 1301.

What to do next

Procedure

- Verify the mirroring status from a Ceph Monitor node on the primary and secondary sites.

  ```
  rbd mirror image status POOL_NAME/IMAGE_NAME
  ```

  In the following example, `up` means the `rbd-mirror` daemon is running, and `stopped` means that this image is not the target for replication from another storage cluster. The status is `stopped` because the image is primary on this storage cluster.

  ```
  [ceph: root@mon-site-a /]# rbd mirror image status data/image1
  image1:
      global_id:   c13d8065-b33d-4cb5-b35f-127a02768e7f
      state:       up+stopped
      description: remote image is non-primary
      service:     host03.yuoosv on host03
      last_update: 2021-10-06 09:13:58

  [ceph: root@mon-site-b /]# rbd mirror image status data/image1
  image1:
      global_id: c13d8065-b33d-4cb5-b35f-127a02768e7f
  ```

Administering Ceph Block Device mirroring

As a storage administrator, you can perform a range of tasks to manage and maintain the Ceph Block Device (RBD) mirroring environment.

These tasks include:

- Viewing information about storage cluster peers.
- Add or remove a storage cluster peer.
- Getting mirroring status for a pool, image and group.
- Enabling mirroring on a pool, image and group.
- Disabling mirroring on a pool, image and group.
- Delaying block device replication.
- Promoting and demoting an image and group.

These tasks help ensure smooth replication operations and support failover and recovery scenarios in multi-site deployments.

Viewing peer information

Learn to view information about storage cluster peers using the command-line.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

About this task

View information about storage cluster peers.

Procedure

- View information about the peers.
  Syntax,

  ```
  rbd mirror pool info POOL_NAME
  ```

  Example,

  ```
  [root@rbd-client ~]# rbd mirror pool info data
  Mode: pool
  Site Name: a

  Peer Sites:

  UUID: 950ddadf-f995-47b7-9416-b9bb233f66e3
  Name: b
  Mirror UUID: 4696cd9d-1466-4f98-a97a-3748b6b722b3
  Direction: rx-tx
  Client: client.rbd-mirror-peer
  ```

Managing mirroring on a pool

Learn to enable and disable mirroring on a Ceph Block Device pool by using the command-line interface. You can also get the mirroring status for a pool.

Before you begin

Before you begin, make sure that you have root-level access to the node.

Enabling mirroring on a pool

Procedure

- Enable mirroring on a pool, by using the **`mirror pool enable`** command.
  Be sure to run the command on both peer clusters.

  ```
  rbd mirror pool enable POOL_NAME MODE
  ```

  The following example enables `pool` mode mirroring of the whole pool named `data`.

  ```
  [root@rbd-client ~]# rbd mirror pool enable data pool
  ```

Disabling mirroring on a pool

Before you begin

Before disabling mirroring, remove any related peer clusters.

About this task

When you disable mirroring on a pool, you also disable it on any images within the pool for which mirroring was enabled separately in image mode.

Procedure

- Disable mirroring on a pool, by using the **`mirror pool disable`** command.

  ```
  rbd mirror pool disable POOL_NAME
  ```

  The following example disables mirroring of a pool named `data`.

  ```
  [root@rbd-client ~]# rbd mirror pool disable data
  ```

Getting mirroring status for a pool

About this task

Get the mirroring pool status summary, by using the **mirror pool status** command.

Procedure

- Run the following command to get the pool status.
  Syntax

  ```
  rbd mirror pool status POOL_NAME
  ```

  To output status details for every mirroring image in a pool, use the **--verbose** option.

  Example

  ```
  [root@site-a ~]# rbd mirror pool status data
  health: OK
  daemon health: OK
  image health: OK
  images: 1 total
      1 replaying
  ```

Managing mirroring on images

Learn to enable and disable mirroring on Ceph Block Device images by using the command-line interface. You can also get the mirroring status for an image. Also, learn to promote, demote, and resynchronize an image.

Before you begin

Before you begin, make sure that you have root-level access to the node.

Enabling mirroring on an image

About this task

Enable mirroring on an image by using the command in the procedure.

Procedure

- Enable mirroring for a specific image within the pool, by using the **mirror image enable** command.
  Syntax

  ```
  rbd mirror image enable POOL_NAME/IMAGE_NAME
  ```

  The following example enables mirroring for the `image2` image in the `data` pool.

  ```
  [root@rbd-client ~]# rbd mirror image enable data/image2
  ```

Disabling mirroring on an image

About this task

Disable mirroring on an image by using the command in the procedure.

Procedure

- Disable mirroring for a specific image, by using the **mirror image disable** command.
  Syntax

  ```
  rbd mirror image disable POOL_NAME/IMAGE_NAME
  ```

  The following example disables mirroring of the `image2` image in the `data` pool.

  ```
  [root@rbd-client ~]# rbd mirror image disable data/image2
  ```

Getting mirroring status for an image

About this task

Get the mirroring status for an image by using the command in the procedure.

Procedure

- Get the mirror status for an image, by running the **`mirror image status`** command.
  Syntax

  ```
  rbd mirror image status POOL_NAME/IMAGE_NAME
  ```

  The following example gets the status of the `image2` image in the `data` pool.

  ```
  [root@site-a ~]# rbd mirror image status data/image2
  image2:
    global_id:   1e3422a2-433e-4316-9e43-1827f8dbe0ef
    state:       up+unknown
    description: remote image is non-primary
    service:     pluto008.yuoosv on pluto008
    last_update: 2025-05-01 09:37:58
  ```

Promoting an image

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster with snapshot-based mirroring is configured.
- Root-level access to the node.

About this task

Promote an image by using the command in the procedure.

> **Warning:** Do not force promote non-primary images that are still syncing as the images are not valid after the promotion.

Procedure

- Promote an image to primary, by using the **`mirror image promote`** command.
  Syntax

  ```
  rbd mirror image promote POOL_NAME/IMAGE_NAME
  ```

  The following example promotes `image2` in the `data` pool.

  ```
  [root@rbd-client ~]# rbd mirror image promote data/image2
  ```

- Use forced promotion when the demotion cannot be propagated to the peer Ceph storage cluster because of cluster failure or communication outage.
  Syntax

  ```
  rbd mirror image promote --force POOL_NAME/IMAGE_NAME
  ```

  ```
  [root@rbd-client ~]# rbd mirror image promote --force data/image2
  ```

Demoting an image

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster with snapshot-based mirroring is configured.

- Root-level access to the node.

About this task

Demote an image by using the command in the procedure.

> **Warning:** Do not force promote non-primary images that are still syncing as the images are not valid after the promotion.

Procedure

- Demote an image to non-primary, by using the **`mirror image demote`** command.
  Syntax

  ```
  rbd mirror image demote POOL_NAME/IMAGE_NAME
  ```

  The following example demotes the `image2` image in the `data` pool.

  ```
  [root@rbd-client ~]# rbd mirror image demote data/image2
  ```

Resynchronizing an image

About this task

You can resynchronize an image if an inconsistency occurs between the two peer clusters. When an image enters an inconsistent state, the `rbd-mirror` daemon skips mirroring for that image until the issue is resolved.

Procedure

- Resynchronize to the primary image, by using the **`mirror image resync`** command.
  Syntax

  ```
  rbd mirror image resync POOL_NAME/IMAGE_NAME
  ```

  The following example requests resynchronization of `image2` in the `data` pool.

  ```
  [root@rbd-client ~]# rbd mirror image resync data/image2
  ```

Managing mirroring on a consistency group

Enable, disable, promote, demote, and resynchronize mirroring on a Ceph Block Device consistency group, and get its mirroring status.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root-level access to the node.
- Pool mirroring is enabled in `image` mode. For more information, see step 5 in Configuring two-way Ceph device block mirroring.

About this task

The commands in this topic support the optional **namespace** parameter. Specify it as `POOL_NAME/NAMESPACE/GROUP_NAME`.

> **Restriction:** You cannot remove images from a mirror-enabled group. Only adding images to a mirror-enabled group is supported.

Enabling mirroring on a group

Enable mirroring for a group in a pool by using the **`rbd mirror group enable`** command.

Procedure

- Run the following command to enable mirroring on a group.

```
rbd mirror group enable POOL_NAME/GROUP_NAME
```

  The following example enables mirroring for the `test_group` group in the `test_pool` pool.

```
[root@rbd-client ~]# rbd mirror group enable test_pool/test_group
```

Result

After you enable mirroring for a group, you can add new images to the group by using the rbd group image add command. The newly added image is automatically onboarded into the mirror-enabled group without requiring mirroring to be disabled and re-enabled.

Disabling mirroring on a group

Disable mirroring for a group in a pool by using the **`rbd mirror group disable`** command.

Procedure

- Run the following command to disable mirroring on a group.

```
rbd mirror group disable POOL_NAME/GROUP_NAME
```

  The following example disables mirroring of the `test_group` group in the `test_pool` pool.

```
[root@rbd-client ~]# rbd mirror group disable test_pool/test_group
```

Getting mirroring status for a group

Get the mirroring status for a group by running the **`rbd mirror group status`** command.

Procedure

- Run the following command to get the group mirroring status.

```
rbd mirror group status POOL_NAME/GROUP_NAME
```

  The following example gets the status of the `test_group` group in the `test_pool` pool.

```
[root@site-a ~]# rbd mirror group status test_pool/test_group
test_group:
  global_id:   8206677f-00e5-4b28-a6cc-121be028cb84
  state:       up+stopped
  description: local group is primary
  service:     admin on dhcp53-181.lab.eng.abc.pqr.com
  last_update: 2025-06-13 17:59:16
  images:
    image:       3/9c6c0e9b-abbb-4219-891a-b451396487a4
    state:       up+stopped
    description: local image is primary
  peer_sites:
    name: site-b
    state: up+replaying
    description: replaying,
{"last_snapshot_bytes":0,"last_snapshot_complete_seconds":10,"local_snapshot_timestamp"
:1749817730,"remote_snapshot_timestamp":1749817730}
    last_update: 2025-06-13 17:59:16
    images:
      image:       3/9c6c0e9b-abbb-4219-891a-b451396487a4
      state:       up+replaying
      description: replaying,
{"bytes_per_second":0.0,"bytes_per_snapshot":0.0,"last_snapshot_bytes":0,"last_snapshot
_sync_seconds":0,"local_snapshot_timestamp":1749817730,"remote_snapshot_timestamp":1749
817730,"replay_state":"idle"}
  snapshots:
    .mirror.primary.8206677f-00e5-4b28-a6cc-121be028cb84.104dda662be0
```

Promoting a group

Promote a group to primary by using the **`rbd mirror group promote`** command.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster with snapshot-based group mirroring is configured.
- Root-level access to the node.

Procedure

- Run the following command to promote a group to primary.

  ```
  rbd mirror group promote POOL_NAME/GROUP_NAME
  ```

  The following example promotes the group `test_group` in the `test_pool` pool to primary.

  ```
  [root@rbd-client ~]# rbd mirror group promote test_pool/test_group
  ```

- Optional: If the demotion cannot be propagated to the peer Ceph storage cluster because of cluster failure or communication outage, force the promotion.

  > **Important:** Before you run the **`rbd mirror group promote`** command with the **`--force`** flag, ensure that the secondary mirror daemon is stopped. This step is a temporary workaround for known issues and might not be necessary after those issues are resolved.

  - Run the following commands to stop and kill the secondary mirror daemon.

    ```
    ceph orch stop rbd-mirror

    kill -SIGKILL PID_OF_SECONDARY_RBD_MIRROR
    ```

  - Run the **`rbd mirror group promote`** command with the **`--force`** flag.

    ```
    rbd mirror group promote --force POOL_NAME/GROUP_NAME
    ```

    The following example forcibly promotes the `test_group` group in the `test_pool` pool to primary.

```
[root@rbd-client ~]# rbd mirror group promote --force test_pool/test_group
```

- After the promotion completes, restart the mirror daemon.

```
ceph orch start rbd-mirror
```

Demoting a group

Demote a group to non-primary by using the **rbd mirror group demote** command.

Procedure

- Run the following command to demote a group to non-primary.

```
rbd mirror group demote POOL_NAME/GROUP_NAME
```

The following example demotes the `test_group` group in the `test_pool` pool to non-primary.

```
[root@rbd-client ~]# rbd mirror group demote test_pool/test_group
```

Resynchronizing a group

Resynchronize a group when an inconsistency occurs between the two peer clusters by using the **rbd mirror group resync** command.

About this task

You can resynchronize a group if an inconsistency occurs between the two peer clusters. When a group enters an inconsistent state, the `rbd-mirror` daemon skips mirroring for that group until the issue is resolved.

Procedure

- Run the following command to resynchronize a group.

```
rbd mirror group resync POOL_NAME/GROUP_NAME
```

The following example requests resynchronization of `test_group` in the `test_pool` pool.

```
[root@rbd-client ~]# rbd mirror group resync test_pool/test_group
```

Managing mirroring on a namespace

Enable and disable Ceph Block Device mirroring on namespaces by using the command-line interface.

Before you begin

Before you begin, make sure that you have root-level access to the node.

About this task

Before configuring a namespace to mirror with a different remote namespace, mirroring must be disabled for both the local and remote namespaces on both the clusters.

Enabling mirroring on a namespace

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- 2 running Red Hat Ceph Storage clusters.
- `rbd-mirror` daemon service enabled on both clusters.
- Mirroring is in enabled for the pool in which the namespace is added.

Note:

About this task

You can mirror a namespace to a namespace with a different name in the remote pool by using the **`--remote-namespace`** option. By default, the namespace is mirrored with the same name on the remote pool.

Procedure

- Enable mirroring on a namespace, by using the **`mirror pool enable`** command. Be sure to run the command on both peer clusters.
  Syntax

```
rbd mirror pool enable POOL_NAME/LOCAL_NAMESPACE_NAMEMODE --remote-
namespace REMOTE_NAMESPACE_NAME
```

  The mirroring mode options are `image` or `pool`. The default mirroring mode is `pool`.

  `image`

  When configured in `image` mode, mirroring must be explicitly enabled on each image.

  `pool`

  When configured in `pool` mode, all images in the namespace with the journaling feature enabled are mirrored.

  The namespace and remote namespace on the first cluster must match the remote namespace and namespace respectively on the remote cluster.

  Note: If the **`--remote-namespace`** option is not provided, the namespace is mirrored to a namespace with the same name in the remote pool.

  The following example enables image mode mirroring between `image-pool/namespace-a` on the first cluster and `image-pool/namespace-b` on the second, remote, cluster.

```
[root@rbd-client ~]#  rbd mirror pool enable image-pool/namespace-a image --remote-
namespace namespace-b

[root@rbd-client ~]#  rbd mirror pool enable image-pool/namespace-b image --remote-
namespace namespace-a
```

Disabling mirroring on a namespace

Before you begin

Before disabling mirroring, any consistency group mirror enabled images in the namespace must be explicitly disabled before disabling mirroring on the namespace if configured in `image` mode.

About this task

Disable mirroring on a namespace, by using the **`mirror pool disable`** command.

Procedure

- Run the following command to disable mirroring on a namespace.
  Syntax

```
rbd mirror pool disable POOL_NAME/NAMESPACE_NAME
```

  Example

```
[root@rbd-client ~]# rbd mirror pool disable image-pool/namespace-a
[root@rbd-client ~]# rbd mirror pool disable image-pool/namespace-b
```

Adding a storage cluster peer

Learn to add a storage cluster peer by using the command-line.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

About this task

Add a storage cluster peer to enable the `rbd-mirror` daemon to discover the remote cluster.

For example, to add the `site-a` storage cluster as a peer to `site-b`, perform the following steps from a client node in the `site-b` cluster.

Procedure

- Register the peer to the pool.
  Syntax,

```
rbd --cluster CLUSTER_NAME mirror pool peer add POOL_NAME
PEER_CLIENT_NAME@PEER_CLUSTER_NAME -n CLIENT_NAME
```

```
[root@rbd-client ~]# rbd --cluster site-b mirror pool peer add data client.site-a@site-
a -n client.site-b
```

Removing a storage cluster peer

Learn to remove a storage cluster peer by using the command-line.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

About this task

Specify the peer UUID to remove a storage cluster peer.

Procedure

- Specify the pool name and the peer Universally Unique Identifier (UUID).
  Syntax:

```
rbd mirror pool peer remove POOL_NAME PEER_UUID
```

```
[root@rbd-client ~]# rbd mirror pool peer remove data 7e90b4ce-
e36d-4f07-8cbc-42050896825d
```

  To view the peer UUID, use the `rbd mirror pool info` command.

Delaying block device replication

Learn to delay block device replication by using the command-line.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

About this task

Whether using one-way or two-way replication, you can configure delayed replication for RADOS Block Device (RBD) mirrored images. Delayed replication provides a buffer period, allowing you to revert unintended changes on the primary image before they are propagated to the secondary image.

> **Note:** Delaying block device replication is only applicable with journal-based mirroring.

To implement delayed replication, the `rbd-mirror` daemon within the destination storage cluster should set the `rbd_mirroring_replay_delay = _MINIMUM_DELAY_IN_SECONDS_` configuration option. This setting can either be applied globally within the `ceph.conf` file utilized by the `rbd-mirror` daemons, or on an individual image basis.

Procedure

- Utilize delayed replication for a specific image, on the primary image by running the following **rbd** command.
  Syntax,

  ```
  rbd image-meta set POOL_NAME/
  IMAGE_NAME conf_rbd_mirroring_replay_delay MINIMUM_DELAY_IN_SECONDS
  ```

  The following example sets a 10 minute minimum replication delay on image `vm-1` in the `vms` pool.

  ```
  [root@rbd-client ~]# rbd image-meta set vms/vm-1 conf_rbd_mirroring_replay_delay 600
  ```

Converting journal-based mirroring to snapshot-based mirroring

Learn to convert journal-based mirroring to snapshot-based mirroring by using the command-line.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

About this task

Convert journal-based mirroring to snapshot-based mirroring by disabling mirroring and enabling snapshots.

Procedure

1. Log in to the `Cephadm` shell.
   Example,

   ```
   [root@rbd-client ~]# cephadm shell
   ```

2. Disable mirroring for a specific image within the pool.
   Syntax,

   ```
   rbd mirror image disable POOL_NAME/IMAGE_NAME
   ```

   Example,

   ```
   [ceph: root@rbd-client /]# rbd mirror image disable mirror_pool/mirror_image
   Mirroring disabled
   ```

3. Enable snapshot-based mirroring for the image.
   Syntax,

   ```
   rbd mirror image enable POOL_NAME/IMAGE_NAME snapshot
   ```

The following example enables snapshot-based mirroring for the `mirror_image` image in the `mirror_pool` pool.

```
ceph: root@rbd-client /]# rbd mirror image enable mirror_pool/mirror_image snapshot
Mirroring enabled
```

Creating an image mirror-snapshot

Learn to create an image mirror-snapshot by using the command-line.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A minimum of two running Red Hat Ceph Storage clusters.
- Root-level access to the Ceph client nodes for the Red Hat Ceph Storage clusters.
- A CephX user with administrator-level capabilities.
- Access to the Red Hat Ceph Storage cluster where a snapshot mirror will be created.

About this task

Create a mirror snapshot to replicate changes in a Ceph Block Device image when using snapshot-based mirroring.

> **Important:** By default, a maximum of 5 image mirror-snapshots are retained. The most recent image mirror-snapshot is automatically removed if the limit is reached. If required, the limit can be overridden through the `rbd_mirroring_max_mirroring_snapshots` configuration. Image mirror-snapshots are automatically deleted when the image is removed or when mirroring is disabled.

Procedure

- Create an image-mirror snapshot.
  Syntax

  ```
  rbd --cluster CLUSTER_NAME mirror image snapshot POOL_NAME/IMAGE_NAME
  ```

  Example

  ```
  [root@site-a ~]# rbd mirror image snapshot data/image1
  ```

Creating consistency group mirror snapshot

Learn to create and get the status consistency group mirror snapshots by using the command-line interface.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A minimum of two running Red Hat Ceph Storage clusters.
- Root-level access to the Ceph client nodes for the Red Hat Ceph Storage clusters.
- A CephX user with administrator-level capabilities.
- Access to the Red Hat Ceph Storage cluster where a snapshot mirror will be created.

About this task

Create a mirror group snapshot to replicate changes in a Ceph Block Device when using snapshot-based mirroring.

Procedure

– Create a mirror group mirroring snapshot.

```
rbd --cluster CLUSTER_NAME mirror group snapshot POOL_NAME/GROUP_NAME
```

For example,

```
[root@site-a ~]# rbd --cluster site-a mirror group snapshot test_pool/test_group
```

What to do next

View the created mirror group snapshot by running the **snap list** command.

```
rbd group snap list --group GROUP_NAME --pool POOL_NAME
```

Verify that the snapshot is in the `copied` state. Check the snapshot `copied` or `not copied` state from the output in the secondary cluster.

`copied`

The data is completely synced to the secondary cluster

`not copied`

The data is still syncing to the secondary cluster. Continue monitoring the snapshot until it reaches the `copied` state.

Example output for primary cluster

```
[ceph: root@site-a]# rbd group snap list --group group_1 --pool pool_1 --debug_rbd 0
ID            NAME               STATE   NAMESPACE
3a08a33beae1 .mirror.primary.123 created mirror (primary peer_uuids:[0b44e1fd-d4e9-4c65-
b5e6-489ad0a87762])
[ceph: root@site-a]#
```

Example output for secondary cluster

```
[ceph: root@site-b]# rbd group snap list --group group_1 --pool pool_1 --debug_rbd 0

ID            NAME                     STATE   NAMESPACE
3a08a33beae1 .mirror.non-primary.123 created mirror (non-primary peer_uuids:[]
75aeb329-4891-495e-9f4d-f938fbf33fb1:3a08a33beae1 **copied**)
[ceph: root@site-b]#
```

Scheduling mirror snapshots

Mirror snapshots can be automatically created when mirror-snapshot schedules are defined. The mirror-snapshot can be scheduled globally, per-pool or per-image levels. Multiple mirror snapshot schedules can be defined at any level but only the most specific snapshot schedules that match an individual mirrored image will run.

Before you begin

– A minimum of two running Red Hat Ceph Storage clusters.

– Root-level access to the Ceph client nodes for the Red Hat Ceph Storage clusters.

– A CephX user with administrator-level capabilities.

– Access to the Red Hat Ceph Storage cluster where the mirror image needs to be scheduled.

Creating a schedule

About this task

Create a mirror-snapshot schedule, by using the **snapshot schedule** command.

Procedure

– Run the following command to create a schedule for mirror snapshots.

Syntax

```
rbd --cluster CLUSTER_NAME mirror snapshot schedule add --pool POOL_NAME --
image IMAGE_NAME INTERVAL [START_TIME]
```

- Only use **--cluster** when the cluster name is different from the default name, ceph.
- Specify the interval in days, hours, or minutes using d, h, or m suffix, respectively.
- The optional *START_TIME* must be specified using the ISO 8601 (hh:mm:ss+|–hh:mm) time format.

The following example creates a snapshot schedule with the default ceph cluster, every 6 hours, and does not specify a start time.

```
[root@site-a ~]# rbd mirror snapshot schedule add --pool data --image image1 6h
```

The following example creates a snapshot schedule with the default ceph cluster, every 24 hours, with a start time of 2:00 PM at -5 GMT.

```
root@site-a ~]# rbd mirror snapshot schedule add --pool data --image image1 24h
14:00:00-05:00
```

Listing snapshot schedules by level

About this task

List all snapshot schedules for a specific global, pool or image level, with an optional pool or image name, by using the **snapshot schedule ls** command.

Procedure

– Run the following command to view the schedules by level.
Syntax

```
rbd --cluster CLUSTER_NAME mirror snapshot schedule ls --pool POOL_NAME --recursive
```

Use the **--recursive** option to list all schedules at the specified level

Example

```
[root@rbd-client ~]# rbd mirror snapshot schedule ls --pool data --recursive
POOL        NAMESPACE IMAGE  SCHEDULE
data         -         -      every 1d starting at 14:00:00-05:00
data         -        image1  every 6h
```

Removing a schedule

About this task

Remove a mirror-snapshot schedule, by using the **snapshot schedule** command.

Procedure

– Run the following command to remove the mirror snapshot schedule.
Syntax

```
rbd --cluster CLUSTER_NAME mirror snapshot schedule remove --pool POOL_NAME --
image IMAGE_NAME INTERVAL START_TIME
```

- Only use **--cluster** when the cluster name is different from the default name, ceph.
- Specify the interval in days, hours, or minutes using d, h, or m suffix, respectively.
- The optional *START_TIME* must be specified using the ISO 8601 (hh:mm:ss+|–hh:mm) time format.

The following example removes a snapshot schedule with the default ceph cluster, every 6 hours, and does not specify a start time.

```
[root@site-a ~]# rbd mirror snapshot schedule remove --pool data --image image1 6h
```

The following example removes a snapshot schedule with the default `ceph` cluster, every 24 hours, with a start time of 2:00 PM at -5 GMT.

```
[root@site-a ~]# rbd mirror snapshot schedule remove --pool data --image image1 24h
14:00:00-05:00
```

Viewing upcoming snapshot schedule

About this task

View the status for the next snapshots to be created for snapshot-based mirroring Ceph Block Device images.

Procedure

- View the status for the next snapshots to be created.
  Syntax

  ```
  rbd --cluster CLUSTER_NAME mirror snapshot schedule status --pool POOL_NAME --
  image IMAGE_NAME
  ```

  Example

  ```
  [root@rbd-client ~]# rbd --cluster site-a mirror snapshot schedule status --ppol data
  --group group1
  SCHEDULE    TIME       IMAGE
  2025-05-01 18:00:00 data/image1
  ```

Scheduling consistency group mirror snapshots

Learn to create, remove, list, and check the status of group mirror snapshot schedule by using the command-line interface.

Before you begin

- A minimum of two running Red Hat Ceph Storage clusters.
- Root-level access to the Ceph client nodes for the Red Hat Ceph Storage clusters.
- A CephX user with administrator-level capabilities.
- Access to the Red Hat Ceph Storage cluster where creation of group mirror-snapshots need to be scheduled.

Creating a schedule

About this task

Create a group snapshot schedule, by using the **`mirror group snapshot schedule add`** command.

Procedure

- Run the following command to create mirror snapshot schedule for group.
  Syntax

  ```
  rbd --cluster CLUSTER_NAME mirror group snapshot schedule add --pool POOL_NAME --
  group GROUP_NAME INTERVAL [START_TIME]
  ```

  - Only use **`--cluster`** when the cluster name is different from the default name, `ceph`.
  - Specify the interval in days, hours, or minutes using `d`, `h`, or `m` suffix, respectively.
  - The optional *START_TIME* must be specified using the ISO 8601 (hh:mm:ss+|–hh:mm) time format.

  The following example creates a mirror group snapshot schedule with the default `ceph` cluster, every 6 hours, and does not specify a start time.

  ```
  [root@site-a ~]# rbd mirror group snapshot schedule add --pool test_pool --group
  test_group 6h
  ```

The following example creates a mirror group snapshot schedule with the default `ceph` cluster, every 24 hours, with a start time of 2:00 PM at -5 GMT.

```
root@site-a ~]# rbd mirror group snapshot schedule add --pool test_pool --group
test_group 24h 14:00:00-05:00
```

Listing snapshot schedules by level

About this task

List all snapshot schedules at the global, pool, namespace or group level by using the **`mirror group snapshot schedule ls`** command.

Procedure

- List all snapshot schedules at the global, pool, namespace or group level by running the following command.
  Syntax

  ```
  rbd --cluster CLUSTER_NAME mirror group snapshot schedule ls --pool POOL_NAME --
  namespace NAMESPACE --group GROUP_NAME --recursive
  ```

  Use the **`--recursive`** option to list all schedules at the specified level.

  Example

  ```
  [root@rbd-client ~]# # rbd --cluster site-a mirror group snapshot schedule ls --pool
  test_pool --recursive
  POOL NAMESPACE GROUP SCHEDULE
  test_pool test_group every 30m
  ```

- To list the mirror group snapshot schedule, run the following command.
  Syntax

  ```
  rbd --cluster CLUSTER_NAME mirror group snapshot schedule ls --pool POOL_NAME --
  group GROUP_NAME --recursive
  ```

  Example

  ```
  [root@rbd-client ~]# rbd --cluster site-a mirror group snapshot schedule ls --pool
  test_pool --group test_group --recursive

  POOL NAMESPACE GROUP SCHEDULE
  test_pool test_group every 30m
  ```

Removing a schedule

About this task

Remove a group snapshot schedule, by using the **`mirror group snapshot schedule remove`** command.

Procedure

- Run the following command to remove the snapshot schedule for a group.
  Syntax

  ```
  rbd --cluster CLUSTER_NAME mirror group snapshot schedule remove --pool POOL_NAME --
  group GROUP_NAME INTERVAL START_TIME
  ```

  - Only use **`--cluster`** when the cluster name is different from the default name, `ceph`.
  - Specify the interval in days, hours, or minutes using `d`, `h`, or `m` suffix, respectively.
  - The optional *START_TIME* must be specified using the ISO 8601 (hh:mm:ss+|–hh:mm) time format.

  The following example removes a snapshot schedule with the default `ceph` cluster, every 6 hours, and does not specify a start time.

```
[root@site-a ~]# rbd mirror group snapshot schedule remove --pool test_pool --group
test_group 6h
```

The following example removes a snapshot schedule with the default ceph cluster, every 24 hours, with a start time of 2:00 PM at -5 GMT.

```
[root@site-a ~]# rbd mirror group snapshot schedule remove --pool test_pool --group
test_group 24h 14:00:00-05:00
```

Viewing upcoming snapshot schedule

About this task

View the status for the next snapshots to be created for snapshot-based mirroring Ceph Block Device consistency group.

Procedure

- View the status for the next snapshots to be created.
  Syntax

```
rbd --cluster CLUSTER_NAME mirror group snapshot schedule status --pool POOL_NAME --
namespace NAMESPACE_NAME --group GROUP_NAME
```

  Example

```
[root@rbd-client ~]# rbd --cluster site-a mirror group snapshot schedule status --pool
test_pool --group test_group
SCHEDULE    TIME       GROUP
2025-05-01 18:00:00 test_pool/test_group
```

Configure graceful shutdown for `rbd-mirror`

Configure `rbd-mirror` for immediate shutdown to interrupt Ceph Block Device replication tasks and enable fast failover during maintenance or orchestration updates.

When you manage `rbd-mirror` services, you might need to control how quickly the daemon shuts down during orchestration updates or failover events. Use the **`termination_grace_period_seconds`** option in the service specification to set the time, in seconds, that a daemon can terminate gracefully before it is forcefully stopped.

A long grace period can delay `rbd-mirror` shutdown and affect automated failover processes.

To ensure fast failover, set **`termination_grace_period_seconds`** to `0` in the rbd-mirror service specification.

```
service_type: rbd-mirror
placement:
  count: 1
spec:
  termination_grace_period_seconds: 0
```

Important: External tools rely on quick termination to avoid hung states. When using external tools, configure **`termination_grace_period_seconds`** to `0` so the daemon exits immediately when required.

Note:

- Other services, such as OSDs, can retain defaults as high as 30 seconds for a clean shutdown.
- This setting is propagated into systemd unit files and affects daemon termination during orchestration updates or failover events.

Setting up mirroring between the default and non-default namespaces

Learn to set up RBD mirroring between the default namespace on one cluster and a non-default namespace on the other cluster.

Before you begin

- All nodes of both clusters must be running Red Hat Ceph Storage 8.1.
- A pool with same name exists on both the clusters and is initialized for RBD.
- A namespace is created in the pool on the cluster.

About this task

To enable setting up mirroring between the default and non-default namespaces, a new mode `init-only` is introduced in Red Hat Ceph Storage 8.1.

When a pool is configured in `init-only` mode, images in the default namespace are not mirrored. However, mirroring can still be configured for other namespaces.

This configuration is useful when you want to mirror images from a different namespace into the default namespace of a remote pool. While this is the primary use case, `init-only` mode is beneficial in scenarios where selective mirroring is required.

> **Note:** `init-only` mode does not prevent configuration of mirroring rules for non-default namespaces. It simply disables automatic mirroring of images in the default namespace.

The procedure for setting up RBD mirroring from a non-default namespace to the default namespace is effectively the same as configuring mirroring from the default namespace to a non-default namespace, with the clusters reversed.

The steps in the procedure set up RBD mirroring from the default namespace in the `test-pool` pool on Cluster 1 to the non-default `test-pool/test-ns` namespace on Cluster 2.

> **Warning:**
>
> - All nodes in both clusters must run Red Hat Ceph Storage 8.1 or later. Mixed-version clusters are not supported.
> - If a namespace is disabled on one cluster, you must disable the corresponding paired namespace on the other cluster before performing any operations on it.

Procedure

1. On cluster 1, enable mirroring on the default namespace for the pool, with the remote namespace and mode set to **`image`**.
   Syntax

   ```
   rbd mirror pool enable POOL_NAME image --site-name SITE_NAME --remote-
   namespace REMOTE_NAMESPACE_NAME
   ```

   In this example, the pool name is `test-pool` and the remote namespace name is `test-ns`.

   ```
   [root@test-server1]# rbd mirror pool enable test-pool image --site-name site-a --
   remote-namespace test-ns
   ```

2. Check mirror information for the pool.
   Syntax

   ```
   rbd mirror pool info POOL_NAME
   ```

```
[root@test-server1]# rbd mirror pool info test-pool
Mode: image
Site Name: site-a
Mirror UUID: e2bb7e8a-3385-48f3-a01c-cb9b546d4a21
Remote Namespace: test-ns
Peer Sites: none
```

3. On cluster 2, enable the pool for mirroring using the **`init-only`** mode. This allows the pool to be configured for mirroring without enabling mirroring on the default namespace.
   Syntax

   ```
   rbd mirror pool enable --site-name SITE_NAME POOL_NAME init-only
   ```

   In this example, mirroring is configured for the pool but is not enabled on the default namespace with mode **`init-only`**.

   ```
   [root@test-server2]# rbd mirror pool enable --site-name site-b test-pool init-only
   ```

4. On the Ceph client node, bootstrap the storage cluster peers.
   a. Create Ceph user accounts, and register the storage cluster peer to the pool.
      Syntax

      ```
      rbd mirror pool peer bootstrap create --site-
      name PRIMARY_LOCAL_SITE_NAME POOL_NAME > PATH_TO_BOOTSTRAP_TOKEN
      ```

      The following example bootstrap command creates the `client.rbd-mirror.site-a` and the `client.rbd-mirror-peer` Ceph users.

      ```
      [ceph: root@rbd-client-site-a /]# rbd mirror pool peer bootstrap create --site-
      name site-a test-pool > /root/bootstrap_token_site-a
      ```

   b. Copy the bootstrap token file to the `site-b` storage cluster.
   c. Import the bootstrap token on the `site-b` storage cluster.
      Syntax

      ```
      rbd mirror pool peer bootstrap import --site-
      name SECONDARY_LOCAL_SITE_NAME POOL_NAME PATH_TO_BOOTSTRAP_TOKEN
      ```

      Note:
      The **`--direction`** argument is optional, as two-way mirroring is the default when bootstrapping peers.

      Example

      ```
      [ceph: root@rbd-client-site-b /]# rbd mirror pool peer bootstrap import --site-
      name site-b test-pool /root/bootstrap_token_site-a
      ```

What to do next

1. Check that the mirroring is set up as expected. Run the **`mirror pool info`** command on both the clusters.
   a. Cluster 1:

```
[root@test-server1]# rbd mirror pool info test-pool
Mode: image
Site Name: site-a
Mirror UUID: e2bb7e8a-3385-48f3-a01c-cb9b546d4a21
Remote Namespace: test-ns

Peer Sites:

UUID: 7ca5c1a2-4831-4625-8191-fa0c9064dc48
Name: site-b
Mirror UUID: 1e5e1b21-441b-4111-a475-5eb7fd19ab1b
Direction: rx-tx
Client: client.rbd-mirror-peer
```

b. Cluster 2:

```
[root@test-server2]# rbd mirror pool info test-pool
Mode: init-only
Site Name: site-b
Mirror UUID: 1e5e1b21-441b-4111-a475-5eb7fd19ab1b
Remote Namespace:

Peer Sites:

UUID: e35fa86d-a37f-4f39-9696-c0ec2a976dac
Name: site-a
Mirror UUID: e2bb7e8a-3385-48f3-a01c-cb9b546d4a21
Direction: rx-tx
Client: client.rbd-mirror-peer
```

2. Configure mirroring on `pool/remote_namespace` to mirror to the default namespace on cluster 1. If the `Remote Namespace` field is not empty, the default namespace is not configured correctly.

```
[root@test-server2]# rbd  mirror pool enable test-pool/test-ns --remote-namespace ""
image
[root@test-server2]# rbd mirror pool info test-pool/test-ns
Mode: image
Mirror UUID: 1e5e1b21-441b-4111-a475-5eb7fd19ab1b
Remote Namespace:
```

> **Note:** Make sure the value passed to the remote-namespace is a properly quoted empty space.

3. **Optional:** To check whether the mirroring is working correctly, create and mirror enable an image in the default namespace on cluster 1 and verify that the image is mirrored to the `test-ns` namespace in cluster 2.

a. Cluster 1:

```
[root@test-server1]# rbd create -s 12M test-pool/img-1
[root@test-server1]# rbd mirror image enable test-pool/img-1 snapshot
Mirroring enabled
[root@test-server1]# rbd ls test-pool
img-1
```

b. Cluster 2:

```
[root@test-server2]# rbd ls test-pool   <-- the image is not mirrored to the
default ns on cluster2
[root@test-server2]# rbd ls test-pool/test-ns  <-- the image is mirrored to test-
pool/test-ns on cluster2
img-1
```

c. Compare the global IDs of the images on both clusters.

```
[root@test-server1]# rbd info test-pool/img-1 | grep "global id"
    mirroring global id: 6923936e-1e5d-4844-a4af-6b20258edf67
[root@test-server2]# rbd info test-pool/test-ns/img-1 | grep "global id"
    mirroring global id: 6923936e-1e5d-4844-a4af-6b20258edf67
```

4. To disable mirroring on a namespace with `rbd`, specify the **`mirror pool disable`** command and the namespace.

```
rbd mirror pool disable POOL_NAME/NAMESPACE
```

When configured in `image` mode, any mirror enabled images in the namespace must be explicitly disabled before disabling mirroring on the namespace.

To disable mirroring on the default namespace, run the **`mirror pool enable`** command with the **`init-only`** mode.

Example

```
[root@test-server1]# rbd --cluster site-a mirror pool enable test-pool init-only
[root@test-server2]# rbd --cluster site-b mirror pool disable test-pool/test-ns
```

Disaster recovery

As a storage administrator, you can be prepared for eventual hardware failure by knowing how to recover the data from another storage cluster where mirroring was configured.

In the examples, the primary storage cluster is known as the `site-a`, and the secondary storage cluster is known as the `site-b`. Additionally, the storage clusters both have a `data` pool with two images, `image1` and `image2`.

These failures have a widespread impact, also referred as a **large blast radius**, and can be caused by impacts to the power grid and natural disasters.

Customer data needs to be protected during these scenarios. Volumes must be replicated with consistency and efficiency and also within Recovery Point Objective (RPO) and Recovery Time Objective (RTO) targets. This solution is called a Wide Area Network- Disaster Recovery (WAN-DR).

In such scenarios it is hard to restore the primary system and the data center. The solutions that are used to recover from these failure scenarios are guided by the application:

- **Recovery Point Objective (RPO)**: The amount of data loss, an application tolerate in the worst case.
- **Recovery Time Objective (RTO)**: The time taken to get the application back on line with the latest copy of the data available.
- For more information, see Mirroring Ceph Block Devices.
- For more information about data transmission over the wire in an encrypted state, see Encryption in transit.

Recover with one-way mirroring

Learn how to recover from a disaster with one-way mirroring.

To recover from a disaster when using one-way mirroring use the following procedures. They show how to fail over to the secondary cluster after the primary cluster terminates, and how to fail back. The shutdown can be orderly or non-orderly.

> **Important:** One-way mirroring supports multiple secondary sites. If you are using additional secondary clusters, choose one of the secondary clusters to fail over to. Synchronize from the same cluster during fail back.

Recover with two-way mirroring

To recover from a disaster when using two-way mirroring use the following procedures. They show how to fail over to the mirrored data on the secondary cluster after the primary cluster terminates, and how to failback. The shutdown can be orderly or non-orderly.

Failover after an orderly shutdown

Learn how to perform a failover to a secondary storage cluster after an orderly shutdown.

Prerequisites

- Minimum of two running Red Hat Ceph Storage clusters.
- Root-level access to the node.
- Pool mirroring or image mirroring configured with one-way mirroring.

Procedure

1. Stop all clients that use the primary image. This step depends on which clients use the image. For example, detach volumes from any OpenStack instances that use the image.
2. Demote the primary images located on the `site-a` cluster by running the following commands on a monitor node in the `site-a` cluster:

   Syntax

   ```
   rbd mirror image demote POOL_NAME/IMAGE_NAME
   ```

   Example

   ```
   [root@rbd-client ~]# rbd mirror image demote data/image1
   [root@rbd-client ~]# rbd mirror image demote data/image2
   ```

3. Promote the non-primary images located on the `site-b` cluster by running the following commands on a monitor node in the `site-b` cluster:

   Syntax

   ```
   rbd mirror image promote POOL_NAME/IMAGE_NAME
   ```

   Example

   ```
   [root@rbd-client ~]# rbd mirror image promote data/image1
   [root@rbd-client ~]# rbd mirror image promote data/image2
   ```

4. After some time, check the status of the images from a monitor node in the `site-b` cluster. They should show a state of `up+stopped` and be listed as primary:

   ```
   [root@rbd-client ~]# rbd mirror image status data/image1
   image1:
     global_id:   08027096-d267-47f8-b52e-59de1353a034
     state:       up+stopped
     description: local image is primary
     last_update: 2019-04-17 16:04:37
   [root@rbd-client ~]# rbd mirror image status data/image2
   image2:
     global_id:   596f41bc-874b-4cd4-aefe-4929578cc834
     state:       up+stopped
     description: local image is primary
     last_update: 2019-04-17 16:04:37
   ```

5. Resume the access to the images. This step depends on which clients use the image.

Reference

- See the Block Storage and Volumeschapter in the *Red Hat OpenStack Platform Storage Guide*.

Failover after a non-orderly shutdown

Learn how to perform a failover to a secondary storage cluster after a non-orderly shutdown.

Prerequisites

- Minimum of two running Red Hat Ceph Storage clusters.
- Root-level access to the node.

– Pool mirroring or image mirroring configured with one-way mirroring.

Procedure

1. Verify that the primary storage cluster is down.
2. Stop all clients that use the primary image. This step depends on which clients use the image. For example, detach volumes from any OpenStack instances that use the image.
3. Promote the non-primary images from a Ceph Monitor node in the `site-b` storage cluster. Use the `--force` option, because the demotion cannot be propagated to the `site-a` storage cluster:

   Syntax

   ```
   rbd mirror image promote --force POOL_NAME/IMAGE_NAME
   ```

   Example

   ```
   [root@rbd-client ~]# rbd mirror image promote --force data/image1
   [root@rbd-client ~]# rbd mirror image promote --force data/image2
   ```

4. Check the status of the images from a Ceph Monitor node in the `site-b` storage cluster. They should show a state of `up+stopping_replay`. The description should say `force promoted`, meaning it is in the intermittent state. Wait until the state comes to `up+stopped` to validate the successful promotion of the site.

   Example

   ```
   [root@rbd-client ~]# rbd mirror image status data/image1
   image1:
     global_id:   08027096-d267-47f8-b52e-59de1353a034
     state:       up+stopping_replay
     description: force promoted
     last_update: 2023-04-17 13:25:06

   [root@rbd-client ~]# rbd mirror image status data/image1
   image1:
     global_id:   08027096-d267-47f8-b52e-59de1353a034
     state:       up+stopped
     description: force promoted
     last_update: 2023-04-17 13:25:06
   ```

Preparing for fail back

If two storage clusters were originally configured only for one-way mirroring, in order to fail back, configure the primary storage cluster for mirroring in order to replicate the images in the opposite direction.

During failback scenario, the existing peer that is inaccessible must be removed before adding a new peer to an existing cluster.

Prerequisites

– A running Red Hat Ceph Storage cluster.

– Root-level access to the client node.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@rbd-client ~]# cephadm shell
   ```

2. On the `site-a` storage cluster , run the following command:

   Example

   ```
   [ceph: root@rbd-client /]# ceph orch apply rbd-mirror --placement=host01
   ```

3. Remove any inaccessible peers.

> **Important:** This step must be run on the peer site which is up and running.

> **Note:** Multiple peers are supported only for one way mirroring.

   a. Get the pool UUID:
   Syntax

```
rbd mirror pool info POOL_NAME
```

   Example

```
[ceph: root@host01 /]# rbd mirror pool info pool_failback
```

   b. Remove the inaccessible peer:
   Syntax

```
rbd mirror pool peer remove POOL_NAME PEER_UUID
```

   Example

```
[ceph: root@host01 /]# rbd mirror pool peer remove pool_failback
f055bb88-6253-4041-923d-08c4ecbe799a
```

4. Create a block device pool with a name same as its peer mirror pool.

   a. To create an rbd pool, run the following:
   Syntax

```
ceph osd pool create POOL_NAME PG_NUM
```

```
ceph osd pool application enable POOL_NAME rbd
```

```
rbd pool init -p POOL_NAME
```

   Example

```
[root@rbd-client ~]# ceph osd pool create pool1
```

```
[root@rbd-client ~]# ceph osd pool application enable pool1 rbd
```

```
[root@rbd-client ~]# rbd pool init -p pool1
```

5. On a Ceph client node, bootstrap the storage cluster peers.

   a. Create Ceph user accounts, and register the storage cluster peer to the pool:

   Syntax

```
rbd mirror pool peer bootstrap create --site-name LOCAL_SITE_NAME POOL_NAME >
PATH_TO_BOOTSTRAP_TOKEN
```

   Example

```
[ceph: root@rbd-client-site-a /]# rbd mirror pool peer bootstrap create --site-
name site-a data > /root/bootstrap_token_site-a
```

> **Note:** This example bootstrap command creates the `client.rbd-mirror.site-a` and the `client.rbd-mirror-peer` Ceph users.

b. Copy the bootstrap token file to the `site-b` storage cluster.

c. Import the bootstrap token on the `site-b` storage cluster:

Syntax

```
rbd mirror pool peer bootstrap import --site-name LOCAL_SITE_NAME --direction rx-only POOL_NAME PATH_TO_BOOTSTRAP_TOKEN
```

Example

```
[ceph: root@rbd-client-site-b /]# rbd mirror pool peer bootstrap import --site-name site-b --direction rx-only data /root/bootstrap_token_site-a
```

> **Note:** For one-way RBD mirroring, you must use the `--direction rx-only` argument, as two-way mirroring is the default when bootstrapping peers.

6. From a monitor node in the `site-a` storage cluster, verify the `site-b` storage cluster was successfully added as a peer:

Example

```
[ceph: root@rbd-client /]# rbd mirror pool info -p data
Mode: image
Peers:
  UUID                                 NAME   CLIENT
  d2ae0594-a43b-4c67-a167-a36c646e8643 site-b client.site-b
```

Reference

For more information, see Ceph user management

Fail back to the primary storage cluster

When the formerly primary storage cluster recovers, fail back to the primary storage cluster.

> **Note:** If you have scheduled snapshots at the image level, then you need to re-add the schedule as image resync operations changes the RBD Image ID and the previous schedule becomes obsolete.

Prerequisites

- Minimum of two running Red Hat Ceph Storage clusters.
- Root-level access to the node.
- Pool mirroring or image mirroring configured with one-way mirroring.

Procedure

1. Check the status of the images from a monitor node in the `site-b` cluster again. They should show a state of `up-stopped` and the description should say `local image is primary`:

Example

```
[root@rbd-client ~]# rbd mirror image status data/image1
image1:
  global_id:   08027096-d267-47f8-b52e-59de1353a034
```

```
  state:       up+stopped
  description: local image is primary
  last_update: 2019-04-22 17:37:48
[root@rbd-client ~]# rbd mirror image status data/image2
image2:
  global_id:   08027096-d267-47f8-b52e-59de1353a034
  state:       up+stopped
  description: local image is primary
  last_update: 2019-04-22 17:38:18
```

2. From a Ceph Monitor node on the `site-a` storage cluster determine if the images are still primary:

   Syntax

   ```
   rbd mirror pool info POOL_NAME/IMAGE_NAME
   ```

   Example

   ```
   [root@rbd-client ~]# rbd info data/image1
   [root@rbd-client ~]# rbd info data/image2
   ```

   In the output from the commands, look for `mirroring primary: true` or `mirroring primary: false`, to determine the state.

3. Demote any images that are listed as primary by running a command like the following from a Ceph Monitor node in the `site-a` storage cluster:

   Syntax

   ```
   rbd mirror image demote POOL_NAME/IMAGE_NAME
   ```

   Example

   ```
   [root@rbd-client ~]# rbd mirror image demote data/image1
   ```

4. Resynchronize the images ONLY if there was a non-orderly shutdown. Run the following commands on a monitor node in the `site-a` storage cluster to resynchronize the images from `site-b` to `site-a`:

   Syntax

   ```
   rbd mirror image resync POOL_NAME/IMAGE_NAME
   ```

   Example

   ```
   [root@rbd-client ~]# rbd mirror image resync data/image1
   Flagged image for resync from primary
   [root@rbd-client ~]# rbd mirror image resync data/image2
   Flagged image for resync from primary
   ```

5. After some time, ensure resynchronization of the images is complete by verifying they are in the `up+replaying` state. Check their state by running the following commands on a monitor node in the `site-a` storage cluster:

   Syntax

   ```
   rbd mirror image status POOL_NAME/IMAGE_NAME
   ```

   Example

   ```
   [root@rbd-client ~]# rbd mirror image status data/image1
   [root@rbd-client ~]# rbd mirror image status data/image2
   ```

6. Demote the images on the `site-b` storage cluster by running the following commands on a Ceph Monitor node in the `site-b` storage cluster:

   Syntax

```
rbd mirror image demote POOL_NAME/IMAGE_NAME
```

Example

```
[root@rbd-client ~]# rbd mirror image demote data/image1
[root@rbd-client ~]# rbd mirror image demote data/image2
```

> **Note:** If there are multiple secondary storage clusters, this only needs to be done from the secondary storage cluster where it was promoted.

7. Promote the formerly primary images located on the `site-a` storage cluster by running the following commands on a Ceph Monitor node in the `site-a` storage cluster:

   Syntax

   ```
   rbd mirror image promote POOL_NAME/IMAGE_NAME
   ```

   Example

   ```
   [root@rbd-client ~]# rbd mirror image promote data/image1
   [root@rbd-client ~]# rbd mirror image promote data/image2
   ```

8. Check the status of the images from a Ceph Monitor node in the `site-a` storage cluster. They should show a status of `up+stopped` and the description should say `local image is primary`:

   Syntax

   ```
   rbd mirror image status POOL_NAME/IMAGE_NAME
   ```

   Example

   ```
   [root@rbd-client ~]# rbd mirror image status data/image1
   image1:
     global_id:   08027096-d267-47f8-b52e-59de1353a034
     state:       up+stopped
     description: local image is primary
     last_update: 2019-04-22 11:14:51
   [root@rbd-client ~]# rbd mirror image status data/image2
   image2:
     global_id:   596f41bc-874b-4cd4-aefe-4929578cc834
     state:       up+stopped
     description: local image is primary
     last_update: 2019-04-22 11:14:51
   ```

Removing two-way mirroring

After fail back is complete, you can remove two-way mirroring and disable the Ceph Block Device mirroring service.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

1. Remove the `site-b` storage cluster as a peer from the `site-a` storage cluster:

   Example

   ```
   [root@rbd-client ~]# rbd mirror pool peer remove data client.remote@remote --cluster
   local
   ```

```
[root@rbd-client ~]# rbd --cluster site-a mirror pool peer remove data client.site-
b@site-b -n client.site-a
```

2. Stop and disable the `rbd-mirror` daemon on the `site-a` client:

   Syntax

```
systemctl stop ceph-rbd-mirror@CLIENT_ID
systemctl disable ceph-rbd-mirror@CLIENT_ID
systemctl disable ceph-rbd-mirror.target
```

   Example

```
[root@rbd-client ~]# systemctl stop ceph-rbd-mirror@site-a
[root@rbd-client ~]# systemctl disable ceph-rbd-mirror@site-a
[root@rbd-client ~]# systemctl disable ceph-rbd-mirror.target
```

Managing `ceph-immutable-object-cache` daemons

Use `ceph-immutable-object-cache` daemons to cache parent image content in a local directory, so that future reads on that data use the local cache.

"Figure: Ceph immutable cache daemon" on page 1328 shows how a cloned image reads data from the cluster or from the local cache of parent image data.

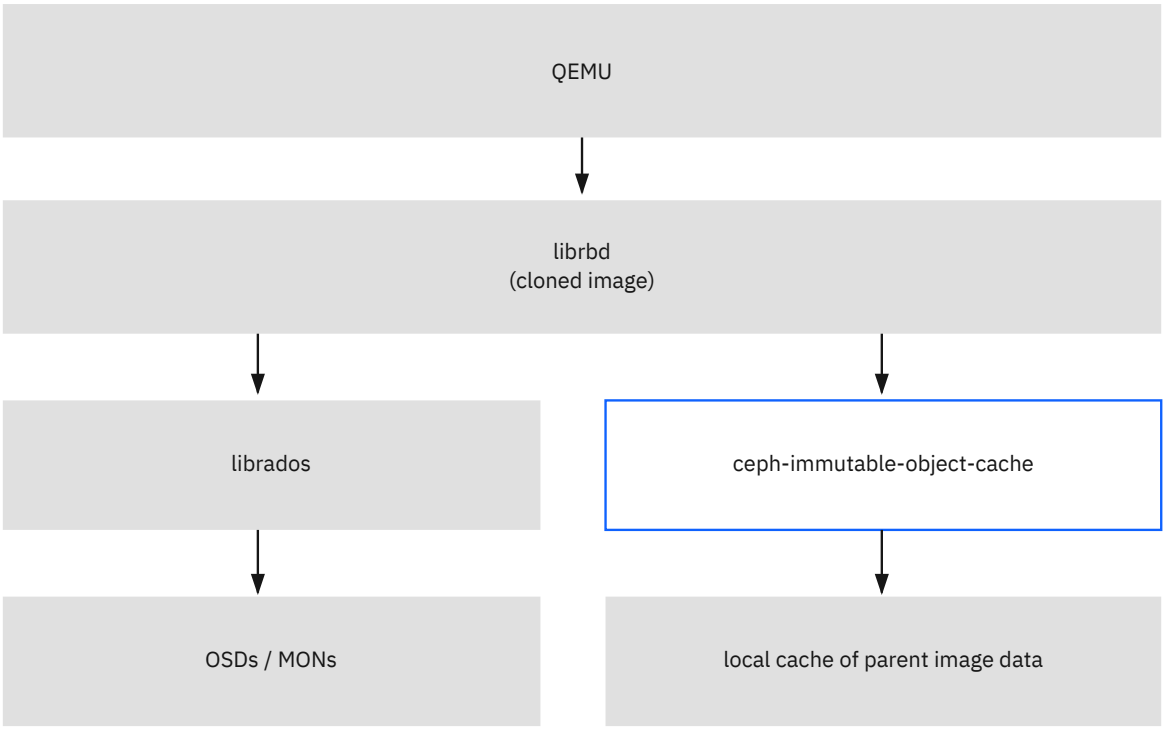


Figure 100: Ceph immutable cache daemon

Reasons to use `ceph-immutable-object-cache` daemons

Cloned Ceph Block Device images usually modify only a small fraction of the parent image. For example, in a virtual desktop infrastructure (VDI), the virtual machines are cloned from the same base image and initially differ only by the hostname and the IP address. During boot-up, a local cache of the parent image speeds up reads on the caching host. This cache also reduces network traffic between Ceph clients and the cluster.

Connection to the cluster

The `ceph-immutable-object-cache` daemon connects to the local cluster by using the Reliable Autonomic Distributed Object Store (RADOS) protocol. The daemon relies on default search paths to find the `ceph.conf` file, the Ceph Monitor addresses, and the authentication information, for example `/etc/ceph/CLUSTER.conf`,

`/etc/ceph/CLUSTER.keyring`, and `/etc/ceph/CLUSTER.NAME.keyring`. In these paths, *CLUSTER* is the human-friendly name of the cluster, and *NAME* is the RADOS user to connect as, for example `client.ceph-immutable-object-cache`.

Key components of the daemon

The `ceph-immutable-object-cache` daemon has the following parts:

Domain socket-based inter-process communication (IPC)

The daemon listens on a local domain socket at startup and waits for connections from `librbd` clients.

Least recently used (LRU) promotion and demotion policy

The daemon maintains in-memory statistics of cache hits on each cache file. When capacity reaches the configured threshold, the daemon demotes the cold cache files.

File-based caching store

The daemon maintains a simple file-based cache store. On promotion, the daemon fetches the RADOS objects from the cluster and stores them in the local caching directory.

How `librbd` uses the cache daemon

When you open each cloned Ceph Block Device image, `librbd` tries to connect to the cache daemon through its Unix domain socket. After the connection succeeds, `librbd` coordinates with the daemon on subsequent reads.

If a read is not cached, the daemon promotes the RADOS object to the local caching directory, so the next read on that object is serviced from the cache. The daemon also maintains simple LRU statistics so that, under capacity pressure, it evicts cold cache files as needed.

> **Tip:** For better performance, use solid-state drives (SSDs) as the underlying storage.

Configuration

The `ceph-immutable-object-cache` is a daemon for object cache of RADOS objects among Ceph clusters.

Before you begin

- A running Red Hat Ceph Storage cluster.
- At least one SSD for the cache.

About this task

> **Important:** To use the `ceph-immutable-object-cache` daemon, you must be able to connect RADOS clusters.

The daemon promotes the objects to a local directory. These cache objects service the future reads. You can configure the daemon by installing the `ceph-immutable-object-cache` package.

Procedure

1. Enable the RBD shared read only parent image cache. Add the following parameters under `[client]` in the `/etc/ceph/ceph.conf` file:
   Example

   ```
   [root@ceph-host01 ~]# vi /etc/ceph/ceph.conf

   [client]
   rbd parent cache enabled = true
   rbd plugins = parent_cache
   ```

2. Restart the cluster.
3. Install the `ceph-immutable-object-cache` package:
   Example

```
[root@ceph-host1 ~]# dnf install ceph-immutable-object-cache
```

4. Create a unique Ceph user ID, the keyring:
   Syntax

   ```
   ceph auth get-or-create client.ceph-immutable-object-cache.USER_NAME mon profile rbd
   osd profile rbd-read-only
   ```

   Example

   ```
   [root@ceph-host1 ~]# ceph auth get-or-create client.ceph-immutable-object-cache.user
   mon 'profile rbd' osd 'profile rbd-read-only'

   [client.ceph-immutable-object-cache.user]
           key = AQCVPH1gFgHRAhAAp8ExRIsoxQK4QSYSRoVJLw==
   ```

   Copy this keyring.

5. In the `/etc/ceph` directory, create a file and paste the keyring:
   Example

   ```
   [root@ceph-host1 ]# vi /etc/ceph/ceph.client.ceph-immutable-object-cache.user.keyring

   [client.ceph-immutable-object-cache.user]
           key = AQCVPH1gFgHRAhAAp8ExRIsoxQK4QSYSRoVJLw
   ```

6. Enable the daemon:
   Syntax

   ```
   systemctl enable ceph-immutable-object-cache@ceph-immutable-object-cache.USER_NAME
   ```

   Specify the **USER_NAME** as the daemon instance.

   Example

   ```
   [root@ceph-host1 ~]# systemctl enable ceph-immutable-object-cache@ceph-immutable-
   object-cache.user

   Created symlink /etc/systemd/system/ceph-immutable-object-cache.target.wants/ceph-
   immutable-object-cache@ceph-immutable-object-cache.user.service → /usr/lib/systemd/
   system/ceph-immutable-object-cache@.service.
   ```

7. Start the `ceph-immutable-object-cache` daemon:
   Syntax

   ```
   systemctl start ceph-immutable-object-cache@ceph-immutable-object-cache.USER_NAME
   ```

   Example

   ```
   [root@ceph-host1 ~]# systemctl start ceph-immutable-object-cache@ceph-immutable-
   object-cache.user
   ```

8. Verification: Check the status of the configuration:
   Syntax

   ```
   systemctl status ceph-immutable-object-cache@ceph-immutable-object-cache.USER_NAME
   ```

   Example

```
[root@ceph-host1 ~]# systemctl status ceph-immutable-object-cache@ceph-immutable-
object-cache.user

● ceph-immutable-object-cache@ceph-immutable-object-cache.user>
  Loaded: loaded (/usr/lib/systemd/system/ceph-immutable-objec>
  Active: active (running) since Mon 2021-04-19 13:49:06 IST; >
Main PID: 85020 (ceph-immutable-)
   Tasks: 15 (limit: 49451)
  Memory: 8.3M
  CGroup: /system.slice/system-ceph\x2dimmutable\x2dobject\x2d>
          └─85020 /usr/bin/ceph-immutable-object-cache -f --cl>
```

Generic settings

Understand the generic options for `ceph-immutable-object-cache` Ceph Block Device daemons.

"Ceph Block Device ceph-immutable-object-cache generic settings" on page 1331 lists the different generic settings for `ceph-immutable-object-cache` Ceph Block Device daemons.

Table 137: Ceph Block Device `ceph-immutable-object-cache` generic settings

Option	Description	Type	Default
immutable_object_cache_max_size	The maximum size for immutable cache.	Size	1G
immutable_object_cache_path	The immutable object cache data directory.	String	`/tmp/ceph_immutable_object_cache`
immutable_object_cache_sock	The path to the domain socket used for communication between librbd clients and the ceph-immutable-object-cache daemon.	String	`/var/run/ceph/immutable_object_cache_sock`
immutable_object_cache_watermark	The high-water mark for the cache. The value is between zero and one. If the cache size reaches this threshold the daemon starts to delete cold cache based on LRU statistics.	Float	0.9

QoS settings for `ceph-immutable-object-cache` daemons

Use these quality of service (QoS) settings to limit the I/O bytes and I/O operations per second of `ceph-immutable-object-cache` daemons.

The `ceph-immutable-object-cache` daemons support throttling. "Quality of service settings for ceph-immutable-object-cache daemons" on page 1331 lists the supported settings.

Table 138: Quality of service settings for `ceph-immutable-object-cache` daemons

Setting	Description	Type	Default
immutable_object_cache_qos_schedule_tick_min	Minimum schedule tick for the immutable object cache.	Milliseconds	`50`
immutable_object_cache_qos_bps_limit	Limit on immutable object cache I/O bytes per second.	Integer	`0`
immutable_object_cache_qos_bps_burst	Burst limit for immutable object cache I/O bytes.	Integer	`0`
immutable_object_cache_qos_bps_burst_seconds	Desired burst limit of read operations.	Seconds	`1`

Setting	Description	Type	Default
immutable_object_cache_qos_iops_limit	Limit on immutable object cache I/O operations per second.	Integer	0
immutable_object_cache_qos_iops_burst	Burst limit for immutable object cache I/O operations.	Integer	0
immutable_object_cache_qos_iops_burst_seconds	Burst duration, in seconds, for immutable object cache I/O operations.	Seconds	1

rbd kernel module

Access Ceph Block Devices through the rbd kernel module. Through the module, you can map and unmap a Block Device and display the mappings. A list of images can also be retrieved through the rbd kernel module.

> **Important:** Kernel clients on Linux distributions other than Red Hat Enterprise Linux (RHEL) are permitted but not supported. If issues are found in the storage cluster when using these kernel clients, Red Hat addresses them, but if the root cause is found to be on the kernel client side, the issue will have to be addressed by the software vendor.

Creating a Ceph Block Device and using it from a Linux kernel module client

Create a Ceph Block Device for a Linux kernel module client on the Red Hat Ceph Storage Dashboard. As a system administrator, you can map that block device on a Linux client, and partition, format, and mount it, using the command line. After this, you can read and write files to it.

Creating a Ceph Block device for a Linux kernel module client requires:

- A running Red Hat Ceph Storage cluster.
- A Red Hat Enterprise Linux client.

Creating a Ceph Block Device for a Linux kernel module client using dashboard

You can create a Ceph Block Device specifically for a Linux kernel module client using the dashboard web interface by enabling only the features it supports.

Kernel module client supports features like Deep flatten, Layering, Exclusive lock, Object map, and Fast diff.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- A replicated RBD pool created and enabled.

Procedure

1. From the **Block** drop-down menu, select **Images**.
2. Click **Create**.
3. In the **Create RBD** window, enter a image name, select the RBD enabled pool, select the supported features.

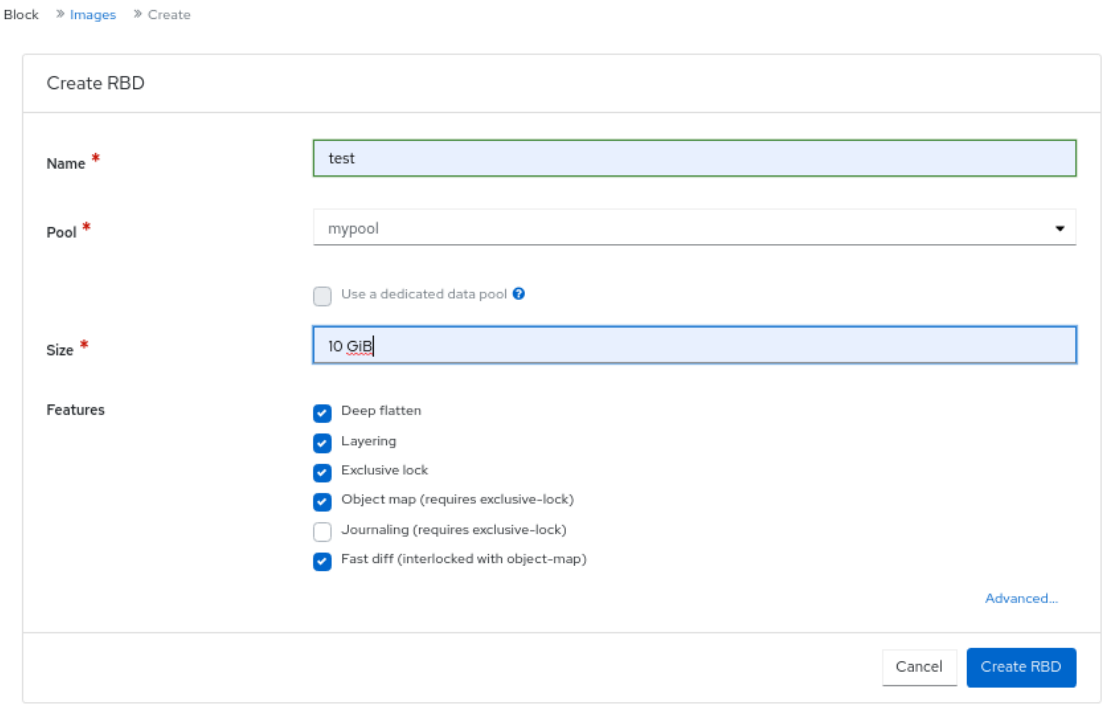


Figure 101: Create RBD window

4. Click **Create RBD**.

Verification

- You get a notification that the image is created successfully.

Map and mount a Ceph Block Device on Linux using the command-line

Map and mount a Ceph Block Device to enable writing files to it by using the command-line interface.

Before you begin

- A running Red Hat Ceph Storage cluster.
- A Ceph Block Device for a Linux kernel module client that uses the dashboard is created.
- A Red Hat Enterprise Linux client.

Procedure

1. On the Red Hat Enterprise Linux client node, enable the Red Hat Ceph Storage tools repository.

```
subscription-manager repos --enable=rhceph-9-tools-for-rhel-9-x86_64-rpms
```

   For example,

```
[root@rbd-client ~]# subscription-manager repos --enable=rhceph-9-tools-for-rhel-9-x86_64-rpms
```

2. Install the `ceph-common` RPM package.

```
dnf install ceph-common
```

3. Copy the Ceph configuration file from a Monitor node to the Client node.

```
scp root@MONITOR_NODE:/etc/ceph/ceph.conf /etc/ceph/ceph.conf
```

   For example,

```
[root@rbd-client ~]# scp root@cluster1-node2:/etc/ceph/ceph.conf /etc/ceph/ceph.conf
root@192.168.0.32's password:
ceph.conf                                                     100%  497
724.9KB/s   00:00
```

4. Copy the key file from a Monitor node to the Client node.

```
scp root@MONITOR_NODE:/etc/ceph/ceph.client.admin.keyring /etc/ceph/
ceph.client.admin.keyring
```

For example,

```
[root@rbd-client ~]# scp root@cluster1-node2:/etc/ceph/ceph.client.admin.keyring /etc/
ceph/ceph.client.admin.keyring
root@192.168.0.32's password:
ceph.client.admin.keyring
 100%  151   265.0KB/s   00:00
```

5. Map the image.

```
rbd map --pool POOL_NAME IMAGE_NAME --id admin
```

For example,

```
[root@rbd-client ~]# rbd map --pool block-device-pool image1 --id admin
/dev/rbd0
```

6. Create a partition table on the block device.

```
parted /dev/MAPPED_BLOCK_DEVICE mklabel msdos
```

For example,

```
[root@rbd-client ~]# parted /dev/rbd0 mklabel msdos
Information: You may need to update /etc/fstab.
```

7. Create a partition for an XFS file system.

```
parted /dev/MAPPED_BLOCK_DEVICE mkpart primary xfs 0% 100%
```

For example,

```
[root@rbd-client ~]# parted /dev/rbd0 mkpart primary xfs 0% 100%
Information: You may need to update /etc/fstab.
```

8. Format the partition.

```
mkfs.xfs /dev/MAPPED_BLOCK_DEVICE_WITH_PARTITION_NUMBER
```

For example,

```
[root@rbd-client ~]# mkfs.xfs /dev/rbd0p1
meta-data=/dev/rbd0p1            isize=512    agcount=16, agsize=163824 blks
         =                       sectsz=512   attr=2, projid32bit=1
         =                       crc=1        finobt=1, sparse=1, rmapbt=0
         =                       reflink=1
data     =                       bsize=4096   blocks=2621184, imaxpct=25
         =                       sunit=16     swidth=16 blks
naming   =version 2              bsize=4096   ascii-ci=0, ftype=1
log      =internal log           bsize=4096   blocks=2560, version=2
         =                       sectsz=512   sunit=16 blks, lazy-count=1
realtime =none                   extsz=4096   blocks=0, rtextents=0
```

9. Create a directory to mount the new file system on.

```
mkdir PATH_TO_DIRECTORY
```

For example,

```
[root@rbd-client ~]# mkdir /mnt/ceph
```

10. Mount the file system.

```
mount /dev/MAPPED_BLOCK_DEVICE_WITH_PARTITION_NUMBER PATH_TO_DIRECTORY
```

For example,

```
[root@rbd-client ~]# mount /dev/rbd0p1 /mnt/ceph/
```

11. Verify that the file system is mounted and showing the correct size.

```
df -h PATH_TO_DIRECTORY
```

For example,

```
[root@rbd-client ~]# df -h /mnt/ceph/
Filesystem      Size  Used Avail Use% Mounted on
/dev/rbd0p1      10G  105M  9.9G   2% /mnt/ceph
```

Mapping a block device

Use the **rbd** command to map an image name to a kernel module. You must specify the image name, the pool name and the user name. `rbd` will load the RBD kernel module if it is not already loaded.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

1. Return a list of the images.
   Example

```
[root@rbd-client ~]# rbd list
```

2. Following are the two options to map the image:
   - Map an image name to a kernel module:

     Syntax

```
rbd device map POOL_NAME/IMAGE_NAME --id USER_NAME
```

     Example

```
[root@rbd-client ~]# rbd device map rbd/myimage --id admin
```

   - Specify a secret when using `cephx` authentication by either the keyring or a file containing the secret:

     Syntax

```
[root@rbd-client ~]# rbd device map POOL_NAME/IMAGE_NAME --id USER_NAME --keyring
PATH_TO_KEYRING
```

     or

```
[root@rbd-client ~]# rbd device map POOL_NAME/IMAGE_NAME --id USER_NAME --keyfile
PATH_TO_FILE
```

Displaying mapped block devices

You can display which block device images are mapped to the kernel module with the **rbd** command.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

- Display the mapped block devices:

```
[root@rbd-client ~]# rbd device list
```

Unmapping a block device

You can unmap a block device image with the **rbd** command, by using theunmapoption and providing the device name.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.
- An image that is mapped.

Procedure

1. Get the specification of the device.
   Example

```
[root@rbd-client ~]# rbd device list
```

2. Unmap the block device image:

   Syntax

```
rbd device unmap /dev/rbd/POOL_NAME/IMAGE_NAME
```

   Example

```
[root@rbd-client ~]# rbd device unmap /dev/rbd/pool1/image1
```

Ceph Block Device python module

The `rbd` python module provides file-like access to Ceph Block Device images. In order to use this built-in tool, import the `rbd` and `rados` python modules.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

Procedure

1. Connect to RADOS and open an IO context:

```
cluster = rados.Rados(conffile='my_ceph.conf')
cluster.connect()
ioctx = cluster.open_ioctx('mypool')
```

2. Instantiate an :class:rbd.RBD object, which you use to create the image:

```
rbd_inst = rbd.RBD()
size = 4 * 1024**3  # 4 GiB
rbd_inst.create(ioctx, 'myimage', size)
```

3. To perform I/O on the image, instantiate an :class:rbd.Image object:

```
image = rbd.Image(ioctx, 'myimage')
data = 'foo' * 200
image.write(data, 0)
```

This writes *foo* to the first 600 bytes of the image. Note that data cannot be :type:unicode - librbd does not know how to deal with characters wider than a :c:type:char.

4. Close the image, the IO context and the connection to RADOS:

```
image.close()
ioctx.close()
cluster.shutdown()
```

To be safe, each of these calls must to be in a separate :finally block:

```
import rados
import rbd

cluster = rados.Rados(conffile='my_ceph_conf')
try:
    ioctx = cluster.open_ioctx('my_pool')
    try:
        rbd_inst = rbd.RBD()
        size = 4 * 1024**3  # 4 GiB
        rbd_inst.create(ioctx, 'myimage', size)
        image = rbd.Image(ioctx, 'myimage')
        try:
            data = 'foo' * 200
            image.write(data, 0)
        finally:
            image.close()
    finally:
        ioctx.close()
finally:
    cluster.shutdown()
```

This can be cumbersome, so the **Rados**, **Ioctx**, and **Image** classes can be used as context managers that close or shut down automatically. Using them as context managers, the above example becomes:

```
with rados.Rados(conffile='my_ceph.conf') as cluster:
    with cluster.open_ioctx('mypool') as ioctx:
        rbd_inst = rbd.RBD()
        size = 4 * 1024**3  # 4 GiB
        rbd_inst.create(ioctx, 'myimage', size)
        with rbd.Image(ioctx, 'myimage') as image:
            data = 'foo' * 200
            image.write(data, 0)
```

Ceph Block Device configuration reference

As a storage administrator, you can fine tune the behavior of Ceph Block Devices through the various options that are available. You can use this reference for viewing such things as the default Ceph Block Device options, and Ceph Block Device caching options.

Default options

Ceph creates images with the 2 format and with no striping.

“Ceph Block Device default options” on page 1338 lists the different default options for Ceph Block Device configuration.

> **Note:** The default settings can be overridden.

Table 139: Ceph Block Device default options

Option	Description	Type	Default	Notes
`rbd_default_format`	The default format (2) if no other format is specified. Format `1` is the original format for a new image, which is compatible with all versions of `librbd` and the kernel module, but does not support newer features like cloning. Format `2` is supported by `librbd` and the kernel module since version 3.11 (except for striping). Format `2` adds support for cloning and is more easily extensible to allow more features in the future.	Integer	`2`	
`rbd_default_order`	The default order if no other order is specified.	Integer	`22`	
`rbd_default_stripe_count`	The default stripe count if no other stripe count is specified. Changing the default value requires striping v2 feature.	64-bit Unsigned Integer	`0`	
`rbd_default_stripe_unit`	The default stripe unit if no other stripe unit is specified. Changing the unit from `0` (that is, the object size) requires the striping v2 feature.	64-bit Unsigned Integer	`0`	

Option	Description	Type	Default	Notes
`rbd_default_features`	The default features enabled when creating a block device image. The settings only apply to format two images. The settings are as follows: **1: Layering support** Layering enables you to use cloning. **2:Striping v2 support** Striping spreads data across multiple objects. Striping helps with parallelism for sequential read/write workloads. **4: Exclusive locking support** When enabled, it requires a client to get a lock on an object before making a write. **8: Object map support**	Integer	`61`	Layering, exclusive-lock, object-map, fast-diff, and deep-flatten are enabled. **Important:** The current default setting is not compatible with the RADOS Block Device kernel driver or older RADOS Block Device clients.

Option	Description	Type	Default	Notes
	Block devices are thin-provisioned—meaning, they only store data that actually exists. Object map support helps track which objects actually exist (have data stored on a drive). Enabling object map support speeds up I/O operations for cloning, or importing and exporting a sparsely populated image. **16: Fast-diff support** Fast-diff support depends on object map support and exclusive lock support. It adds another property to the object map, which makes it much faster to generate diffs between snapshots of an image, and the actual data usage of a snapshot much faster. **32: Deep-flatten support**			

Option	Description	Type	Default	Notes
	Deep-flatten makes `rbd flatten` work on all the snapshots of an image, in addition to the image itself. Without it, snapshots of an image will still rely on the parent, so the parent will not be delete-able until the snapshots are deleted. Deep-flatten makes a parent independent of its clones, even if they have snapshots. **64: Journaling support** Journaling records all modifications to an image in the order they occur. This ensures that a crash-consistent mirror of the remote image is available locally. The enabled features are the sum of the numeric settings.			
`rbd_default_map_options`	Most of the options are useful mainly for debugging and benchmarking. For more information, see `man rbd` under “Map Options”.	String	`""`	

General options

Understand the general options for Ceph Block Devices.

“Ceph Block Device general options” on page 1342 lists the different default options for Ceph Block Device configuration.

Table 140: Ceph Block Device general options

Option	Description	Type	Default
`rbd_clone_copy_on_read`	When set to `true`, copy-on-read cloning is enabled.	Boolean	`false`
`rbd_concurrent_management_ops`	The maximum number of concurrent management operations in flight (for example, deleting or resizing an image).	Integer	`10`
`rbd_enable_alloc_hint`	If `true`, allocation hinting is enabled, and the block device will issue a hint to the OSD back end to indicate the expected size object.	Boolean	`true`
`rbd_non_blocking_aio`	If `true`, Ceph will process block device asynchronous I/O operations from a worker thread to prevent blocking.	Boolean	`true`
`rbd_op_thread_timeout`	The timeout (in seconds) for block device operation threads.	Integer	`60`
`rbd_op_threads`	The number of block device operation threads. **Warning:** Do not change the default value of `rbd_op_threads` because setting it to a number higher than `1` might cause data corruption.	Integer	`1`
`rbd_request_timed_out_seconds`	The number of seconds before a maintenance request times out.	Integer	`30`
`rbd_skip_partial_discard`	If `true`, the block device will skip zeroing a range when trying to discard a range inside an object.	Boolean	`true`
`rbd_tracing`	Set this option to `true` to enable the Linux Trace Toolkit Next Generation User Space Tracer (LTTng-UST) tracepoints. For more information, see Tracing RADOS Block Device (RBD) Workloads with the RBD Replay Feature within the Red Hat Customer Portal.	Boolean	`false`

Option	Description	Type	Default
rbd_validate_names	Set this option to true to validate image specifications.	Boolean	true
rbd_validate_pool	Set this option to true to validate empty pools for RBD compatibility.	Boolean	true

Caching options

The user space implementation of the Ceph Block Device, librbd, cannot take advantage of the Linux page cache, so it includes its own in-memory caching, called RBD caching. Ceph Block Device caching behaves just like well-behaved hard disk caching.

When the operating system sends a barrier or a flush request, all dirty data is written to the Ceph OSDs. This means that using write-back caching is as safe as using a well-behaved physical hard disk with a virtual machine that properly sends flushes, that is, Linux kernel version 2.6.32 or higher. The cache uses a Least Recently Used (LRU) algorithm, and in write-back mode it can coalesce contiguous requests for better throughput.

Ceph Block Devices support write-back caching. To enable write-back caching, set rbd_cache = true to the [client] section of the Ceph configuration file. By default, librbd does not perform any caching. Writes and reads go directly to the storage cluster, and writes return only when the data is on disk on all replicas. With caching enabled, writes return immediately, unless there are more than rbd_cache_max_dirty unflushed bytes. In this case, the write triggers a write-back and blocks until enough bytes are flushed.

Ceph Block Devices support write-through caching. You can set the size of the cache, and you can set targets and limits to switch from write-back caching to write-through caching. To enable write-through mode, set rbd_cache_max_dirty to 0. This means writes return only when the data is on disk on all replicas, but reads can come from the cache. The cache is in memory on the client, and each Ceph Block Device image has its own. Since the cache is local to the client, there is no coherency if there are others accessing the image. Running other file systems, such as GFS or OCFS, on top of Ceph Block Devices will not work with caching enabled.

The Ceph configuration settings for Ceph Block Devices must be set in the [client] section of the Ceph configuration file, by default /etc/ceph/ceph.conf.

“Red Hat Ceph Storage cache option settings” on page 1343 lists the available cache settings.

Table 141: Red Hat Ceph Storage cache option settings

Option	Description	Type	Required	Constraint	Default
rbd_cache	Enable caching for RADOS Block Device (RBD).	Boolean	No	N/A	true
rbd_cache_size	The RADOS Block Devices (RBD) cache size in bytes.	64-bit integer	No	N/A	32 MiB
rbd_cache_max_dirty	The dirty limit in bytes at which the cache triggers a write-back. If 0, uses write-through caching.	64-bit integer	No	Must be less than rbd cache size.	24 MiB

Option	Description	Type	Required	Constraint	Default
`rbd_cache_target_dirty`	The `dirty target` before the cache begins writing data to the data storage. Does not block writes to the cache.	64-bit integer	No	Must be less than `rbd cache max dirty`.	`16 MiB`
`rbd_cache_max_dirty_age`	The number of seconds dirty data is in the cache before writeback starts.	Float	No	N/A	`1.0`
`rbd_cache_max_dirty_object`	The dirty limit for objects. Set to `0` to auto calculate from `rbd_cache_size`.	Integer	N/A	N/A	`0`
`rbd_cache_block_writes_upfront`	If `true`, it will block writes to the cache before the `aio_write` call completes. If `false`, it blocks before the `aio_completion` is called.	Boolean	N/A	N/A	`false`
`rbd_cache_writethrough_until_flush`	Start out in write-through mode, and switch to write-back after the first flush request is received. Enabling this is a conservative but safe setting in case VMs running on rbd are too old to send flushes, like the virtio driver in Linux before 2.6.32.	Boolean	No	N/A	`true`

Parent and child read options

Understand the parent and child read options for Ceph Block Devices.

“Red Hat Ceph Storage cache option settings” on page 1343 lists the parent and child read options for Ceph Block Device configuration.

Table 142: Parent and child read options

Option	Description	Type	Default
`rbd_balance_snap_reads`	Ceph typically reads objects from the primary OSD. Since reads are immutable, you can enable this feature to balance snap reads between the primary OSD and the replicas. This option randomizes the replica for reading a snapshot.	Boolean	`false`
`rbd_localize_snap_reads`	The block device looks to the CRUSH map to find the closest or local OSD for reading the snapshot.	Boolean	`false`
`rbd_balance_parent_reads`	Ceph typically reads objects from the primary OSD. Since reads are immutable, you can enable this feature to balance parent reads between the primary OSD and the replicas. This option randomizes the replica for reading a parent.	Boolean	`false`
`rbd_localize_parent_reads`	The block device looks to the CRUSH map to find the closest or local OSD for reading the parent.	Boolean	`true`

Read ahead options

Ceph Block Devices support read-ahead and prefetching to optimize small, sequential reads. This is generally handled by the guest OS in the case of a virtual machine (VM), but boot loaders potentially do not issue efficient reads. Read-ahead is automatically disabled if caching is disabled.

"Read-ahead configuration options" on page 1345 lists the read ahead configuration options for Ceph Block Devices.

Table 143: Read-ahead configuration options

Option	Description	Type	Required	Default
`rbd_readahead_trigger_requests`	Number of sequential read requests necessary to trigger read-ahead.	Integer	No	`10`
`rbd_readahead_max_bytes`	Maximum size of a read-ahead request. If zero, read-ahead is disabled.	64-bit integer	No	`512 KiB`

Option	Description	Type	Required	Default
`rbd_readahead_disable_after_bytes`	After this many bytes are read from an RBD image, read-ahead is disabled for that image until it is closed. This allows the guest OS to take over read-ahead after it is started. If zero, read-ahead stays enabled.	64-bit integer	No	`50 MiB`

Block-list options

Understand the block-list options for Ceph Block Devices.

"Block-list configuration options" on page 1346 lists the block-list configuration options for Ceph Block Devices.

Table 144: Block-list configuration options

Option	Description	Type	Default
`rbd_blocklist_on_break_lock`	Whether to block-list clients whose lock was broken.	Boolean	`true`
`rbd_blocklistexpire_seconds`	The number of seconds to block-list - set to `0` for OSD default.	Integer	`0`

Journal options

Understand the journal options for Ceph Block Devices.

"Journal configuration options" on page 1346 lists the journal configuration options for Ceph Block Devices.

Table 145: Journal configuration options

Option	Description	Type	Default
`rbd_journal_order`	The number of bits to shift to compute the journal object maximum size. The value is between `12` and `64`.	32-bit unsigned integer	`24`
`rbd_journal_splay_width`	The number of active journal objects.	32-bit unsigned integer	`4`
`rbd_journal_commit_age`	The commit time interval in seconds.	Double Precision Floating Point Number	`5`
`rbd_journal_object_flush_interval`	The maximum number of pending commits per a journal object.	Integer	`0`
`rbd_journal_object_flush_bytes`	The maximum number of pending bytes per a journal object.	Integer	`0`
`rbd_journal_object_flush_age`	The maximum time interval in seconds for pending commits.	Double Precision Floating Point Number	`0`
`rbd_journal_pool`	Specifies a pool for journal objects.	String	`""`

Configuration override options

Use these configuration override options for Ceph Block Devices on global and pool levels.

Global level

"Global level override key options" on page 1347 lists the override key options on a global level.

Table 146: Global level override key options

Key	Description	Type	Default
`rbd_qos_bps_burst`	The desired burst limit of I/O bytes.	Integer	0
`rbd_qos_bps_limit`	The desired limit of I/O bytes per second.	Integer	0
`rbd_qos_iops_burst`	The desired burst limit of I/O operations.	Integer	0
`rbd_qos_iops_limit`	The desired limit of I/O operations per second.	Integer	0
`rbd_qos_read_bps_burst`	The desired burst limit of read bytes.	Integer	0
`rbd_qos_read_bps_limit`	The desired limit of read bytes per second.	Integer	0
`rbd_qos_read_iops_burst`	The desired burst limit of read operations.	Integer	0
`rbd_qos_read_iops_limit`	The desired limit of read operations per second.	Integer	0
`rbd_qos_write_bps_burst`	The desired burst limit of write bytes.	Integer	0
`rbd_qos_write_bps_limit`	The desired limit of write bytes per second.	Integer	0
`rbd_qos_write_iops_burst`	The desired burst limit of write operations.	Integer	0
`rbd_qos_write_iops_limit`	The desired burst limit of write operations per second.	Integer	0

"Global override command usage" on page 1347 describes how to use the global override options listed in "Global level override key options" on page 1347.

- Replace *`CONFIG_ENTITY`* with the global client or client ID.
- Replace *`KEY`* with the configuration key.
- Replace *`VALUE`* with the configuration value.

Table 147: Global override command usage

Command syntax	Description
`rbd config global set CONFIG_ENTITY KEY VALUE`	Set a global level configuration override.
`rbd config global get CONFIG_ENTITY KEY`	Get a global level configuration override.
`rbd config global list CONFIG_ENTITY`	List the global level configuration overrides.

Command syntax	Description
`rbd config global remove CONFIG_ENTITY KEY`	Remove a global level configuration override.

Pool level

“Pool level override command usage” on page 1348 describes how to override pool options.

- Replace *CONFIG_ENTITY* with the global client or client ID.
- Replace *KEY* with the configuration key.
- Replace *VALUE* with the configuration value.
- Replace *POOL_NAME* with the name of the pool.

Table 148: Pool level override command usage

Command syntax	Description
`rbd config pool set POOL_NAME KEY VALUE`	Set a pool level configuration override.
`rbd config pool get POOL_NAME KEY`	Get a pool level configuration override.
`rbd config pool list POOL_NAME`	List the pool level configuration overrides.
`rbd config pool remove POOL_NAME KEY`	Remove a pool level configuration override.

Input and output options

Understand the input and output (I/O) options for Ceph Block Devices.

“I/O configuration options” on page 1349 lists the block-list configuration options for Ceph Block Devices.

Table 149: I/O configuration options

Option	Description	Type	Required	Values	Default
`rbd_compression_hint`	Hint to send to the OSDs on write operations. If set to `compressible` and the OSD `bluestore_compression_mode` setting is `passive`, the OSD attempts to compress data. If set to `incompressible` and the OSD `bluestore_compression_mode` setting is `aggressive`, the OSD does not attempt to compress data.	Enum	No	– `none` – `compressible` – `incompressible`	`none`

Option	Description	Type	Required	Values	Default
`rbd_read_from_replica_policy`	Policy for determining which OSD receives read operations. If set to `default`, each PG's primary OSD will always be used for read operations. If set to `balance`, read operations are sent to a randomly selected OSD within the replica set. If set to `localize`, read operations are sent to the closest OSD as determined by the CRUSH map and the `crush_location` configuration option, where the `crush_location` is denoted by using `key=value`. The `key` aligns with the CRUSH map keys. **Note:** This feature requires the storage cluster to be configured with a minimum compatible OSD release of the latest version of Red Hat Ceph Storage.	Enum	No	– `default` – `balance` – `localize`	`default`

Red Hat Ceph Storage Block Devices and OpenStack

Red Hat Ceph Storage integration with Red Hat OpenStack provides scalable and resilient block storage for cloud workloads, supporting virtual machine images, volumes, and guest disks.

Red Hat Ceph Storage integrates with OpenStack distributed block storage backend that scales horizontally and delivers high availability for cloud workloads. In an integrated environment, Ceph Block Devices are consumed by multiple OpenStack services, to store and manage images, volumes, persistent and virtual machine disks efficiently.

This section describes the supported integration models between Ceph and OpenStack, the OpenStack services that can use Ceph block storage, and key considerations to review before configuring the environment.

For detailed deployment guidance and service-specific configuration options, see the Red Hat OpenStack Platform documentation for your specific OpenStack release.

Ceph and OpenStack integration models

OpenStack can use Ceph Block Devices in the following deployment models:

OpenStack-managed Ceph cluster

In this model, OpenStack creates and manages the Ceph storage cluster as part of the deployment. OpenStack installs and configures Ceph components, including monitors and object storage daemons (OSDs). This approach simplifies deployment by allowing OpenStack to automate Ceph installation and configuration.

Existing Ceph cluster integration

OpenStack connects to a preconfigured Ceph storage cluster and uses existing Ceph monitors. Ceph block devices are configured as a storage backend for OpenStack services. This model is typically used when a Ceph cluster already exists or when storage and cloud infrastructure are managed separately.

> **Note:** The procedures described in this guide assume that you are manually configuring OpenStack to use an existing Ceph cluster rather than using automated deployment tools such as an OpenStack director.

OpenStack services that use Ceph Block Devices

Ceph Block Devices can be used as storage backends for several OpenStack services:

Image storage (Glance)

OpenStack Glance manages virtual machine images, which are treated as immutable objects. When Glance is configured to use Ceph, images are stored in a Ceph Block Device pool and can be efficiently cloned using copy-on-write mechanisms.

Volume storage (Cinder and Cinder Backup)

OpenStack Cinder provides persistent block storage volumes that can be attached to running virtual machines, used for boot-from-volume instances, or backed up using Cinder Backup. Ceph can be used as a backend for both Cinder volumes and Cinder Backup storage.

Guest virtual machine disks (Nova)

Ceph block devices can store guest virtual machine disks directly. Instead of maintaining disk files on the hypervisor file system, Nova accesses disks from Ceph, improving scalability and enabling features such as live migration.

Image format and storage considerations

Ceph does not support the QCOW2 image format for hosting virtual machine disks. To boot virtual machines using Ceph block storage, images configured in Glance must use the RAW format.

This requirement applies to both ephemeral disks and boot-from-volume instances.

OpenStack can use Ceph for any combination of image storage, volume storage, or guest disk storage. There is no requirement to use Ceph for all three services.

Installing and configuring Ceph for OpenStack

Install and configure Red Hat Ceph Storage so that OpenStack can use Ceph Block Devices as a storage backend.

Before you begin, make sure that you have the following prerequisites in place:

- A new or existing Red Hat Ceph Storage cluster is installed and operational.
- Administrative access to the Ceph cluster.
- Access to the OpenStack environment that will use Ceph storage.

Complete all tasks in this section before proceeding to coßnfigure OpenStack services to use Ceph Block Devices. The following is a high-level workflow of the tasks required:

1. Install Red Hat Ceph Storage and verify that the cluster reports a healthy status.
   For more information, see "Installing the Ceph client on OpenStack" on page 1353.
2. Create and configure Ceph storage pools for OpenStack services.
3. Configure authentication and access for OpenStack.
4. Validate connectivity between OpenStack and the Ceph cluster.

Creating Ceph pools for OpenStack

Create Ceph storage pools to store images, volumes, virtual machine disks, and backup data for OpenStack services.

Before you begin

Before you begin, make sure that you have the prerequisites defined in "Installing and configuring Ceph for OpenStack" on page 1352 and access to a Ceph Monitor node.

About this task

Ceph pools are used by OpenStack services to store different types of data such as images, volumes, guest virtual machine disks, and backups. Creating separate pools helps isolate workloads and allows tuning based on usage patterns.

Procedure

1. Verify that the Ceph cluster is running and in a healthy state.
   Run the following command on a Ceph monitor node:

   ```
   ceph -s
   ```

   Ensure that the cluster reports a **HEALTH_OK** status.
2. Create Ceph pools for OpenStack workloads.
   The following example creates pools used by OpenStack services, where 128 represents the number of placement groups (PGs) configured for each pool.

   ```
   [root@rbd-client ~]# ceph osd pool create volumes 128
   [root@rbd-client ~]# ceph osd pool create backups 128
   [root@rbd-client ~]# ceph osd pool create images 128
   [root@rbd-client ~]# ceph osd pool create vms 128
   ```

Result

Ceph storage pools required for OpenStack services are created and available. These pools can now be configured as backends for Glance, Cinder, and Nova.

What to do next

Red Hatrecommends calculating an appropriate number of placement groups based on the size, number of OSDs, and workload characteristics of the Ceph cluster.

Installing the Ceph client on OpenStack

Install the Ceph client packages on OpenStack nodes so that OpenStack services can communicate with the Ceph cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

1. Prerequisites defined in "Installing and configuring Ceph for OpenStack" on page 1352.
2. Access to the Ceph software repositories.
3. Root access to the OpenStack nodes.

About this task

OpenStack services require Ceph client libraries to communicate with the Ceph cluster. These packages enable OpenStack components to access and manage Ceph Block Devices by using the Ceph Block Device (RBD) interface.

Procedure

1. Install the required Ceph client packages on each of the Nova, Cinder, and Cinder Backup OpenStack nodes.

   ```
   dnf install ceph-common python-rbd
   ```

2. Install the Ceph RBD Python package on the Glance node.

   ```
   dnf install python-rbd
   ```

Result

The required Ceph client packages are installed on the OpenStack nodes. OpenStack services can now communicate with the Ceph cluster.

Copying the Ceph configuration file to OpenStack

Copy the Ceph configuration file to OpenStack nodes so that they can locate and connect to the Ceph cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

1. Prerequisites defined in Installing and configuring Ceph for OpenStack.
2. Root access to the Ceph Monitor node.
3. Root access to the OpenStack nodes.

About this task

OpenStack services require the Ceph configuration file to identify the Ceph cluster monitors and connection settings. This file must be present on each OpenStack node that communicates with Ceph.

Procedure

1. Copy the Ceph configuration file from a Ceph monitor node to all OpenStack nodes.

   ```
   scp /etc/ceph/ceph.conf OPENSTACK_NODES:/etc/ceph/
   ```

   Replace `OPENSTACK_NODES` with the hostnames or IP addresses of the OpenStack nodes.

2. Verify that the Ceph configuration file exists on each OpenStack node.
   Confirm that the `/etc/ceph/ceph.conf` file is present and readable.

Result

The Ceph configuration file is available on all required OpenStack nodes. OpenStack services can now locate and connect to the Ceph cluster.

Configuring Ceph client authentication

Configure Ceph authentication so that OpenStack services can securely access Ceph Block Devices.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

1. Prerequisites defined in Installing and configuring Ceph for OpenStack.
2. Root access to a Ceph Monitor node.
3. Root access to specific OpenStack service nodes (Nova, Cinder, Cinder Backup, Glance).

About this task

Ceph uses authenticated clients to control access to storage pools. Before OpenStack services can use Ceph Block Devices, you must create Ceph users, distribute keyrings, and configure the libvirt secret used by Nova.

Procedure

1. Create Ceph users for OpenStack services.
   Run the following commands on a Ceph Monitor node:

   ```
   ceph auth get-or-create client.cinder \
     mon 'allow r' \
     osd 'allow class-read object_prefix rbd_children, allow rwx pool=volumes, allow rwx
   pool=vms, allow rx pool=images'
   ```

   ```
   ceph auth get-or-create client.cinder-backup \
     mon 'allow r' \
     osd 'allow class-read object_prefix rbd_children, allow rwx pool=backups'
   ```

   ```
   ceph auth get-or-create client.glance \
     mon 'allow r' \
     osd 'allow class-read object_prefix rbd_children, allow rwx pool=images'
   ```

2. Copy the keyrings to the appropriate OpenStack nodes and set ownership.
   For the Cinder volume service, run

   ```
   ceph auth get-or-create client.cinder | \
     ssh CINDER_VOLUME_NODE sudo tee /etc/ceph/ceph.client.cinder.keyring

   ssh CINDER_VOLUME_NODE sudo chown cinder:cinder \
     /etc/ceph/ceph.client.cinder.keyring
   ```

   Repeat this step for `client.cinder-backup` and `client.glance` on their respective OpenStack nodes.

3. Copy the Cinder keyring to each Nova compute node.

   ```
   ceph auth get-or-create client.cinder | \
     ssh NOVA_NODE sudo tee /etc/ceph/ceph.client.cinder.keyring
   ```

4. Retrieve the Cinder secret key on a Nova compute node.

   ```
   ceph auth get-key client.cinder > client.cinder.key
   ```

5. Generate a UUID for the libvirt secret.

   ```
   uuidgen > uuid-secret.txt
   ```

6. Define and set the Ceph secret in libvirt.

```
cat > secret.xml <<EOF
<secret ephemeral='no' private='no'>
  <uuid>$(cat uuid-secret.txt)</uuid>
  <usage type='ceph'>
    <name>client.cinder secret</name>
  </usage>
</secret>
EOF
```

```
virsh secret-define --file secret.xml

virsh secret-set-value \
  --secret $(cat uuid-secret.txt) \
  --base64 $(cat client.cinder.key)
```

7. Remove the temporary files.

```
rm -f client.cinder.key secret.xml
```

Result

Ceph client authentication is configured for OpenStack services. OpenStack components can now authenticate securely and access Ceph Block Devices.

Configuring OpenStack to use Ceph Block Devices

Configure OpenStack services to use Ceph block devices as a scalable and resilient storage backend after the Ceph cluster is prepared.

As a storage administrator, you must configure OpenStack services to use Ceph Block Devices after installing and preparing the Ceph cluster. This configuration enables OpenStack components to consume Ceph storage for images, volumes, backups, and virtual machine disks.

This section describes the tasks required to configure OpenStack components to integrate with Ceph block storage. Complete all tasks in this section only after Ceph is fully installed and configured for OpenStack.

Before you begin, make sure that you have the following prerequisites in place:

- Red Hat Ceph Storage is installed and configured for use with OpenStack.
- Ceph storage pools required for OpenStack service integration are created.
- Ceph client authentication is configured.
- A running OpenStack environment is available.

Configuring Cinder to use Ceph Block Devices

Configure OpenStack Cinder to use Ceph Block Devices as the backend for volume storage.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

1. Prerequisites defined in “Installing and configuring Ceph for OpenStack” on page 1352.
2. Root access to the Cinder node.
3. A Ceph pool is created for storing Cinder volumes.
4. The Ceph user and UUID secret required for Cinder are configured.

About this task

Cinder uses a backend driver to provision and manage block storage volumes. When configured to use Ceph, Cinder stores volumes as Ceph Block Devices (RBDs) in a designated Ceph pool.

Procedure

1. Edit the Cinder configuration file.

For example, to open the configuration file on the Cinder node:

```
vi /etc/cinder/cinder.conf
```

2. In the `[DEFAULT]` section, enable Ceph as the backend.
   Add or update the following configuration options, as required.

```
enabled_backends = ceph
glance_api_version = 2
```

3. Create a `[ceph]` section and configure the Ceph volume driver.

```
[ceph]
volume_driver = cinder.volume.drivers.rbd.RBDDriver
rbd_cluster_name = ceph
rbd_pool = volumes
rbd_user = cinder
rbd_ceph_conf = /etc/ceph/ceph.conf
rbd_secret_uuid = UUID
rbd_flatten_volume_from_snapshot = false
rbd_max_clone_depth = 5
rbd_store_chunk_size = 4
rados_connect_timeout = -1
```

   Replace *`UUID`* with the UUID of the libvirt secret configured for the Cinder service.

4. Save the file and exit the editor.

Result

Cinder is configured to use Ceph Block Devices as its volume backend. Volumes can now be created and managed by using Ceph storage.

What to do next

Restart the OpenStack service to complete the configuration. For more information, see “Restarting the OpenStack services” on page 1359.

Configuring Cinder Backup to use Ceph Block Devices

Configure OpenStack Cinder Backup to store volume backup data in Ceph Block Storage.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root access to the Cinder node.
- A Ceph pool is created for storing backup data.

About this task

Cinder Backup enables backup and restore of Cinder volumes. When configured to use Ceph, backup data is stored as RADOS objects in a designated Ceph pool.

Procedure

1. Edit the Cinder configuration file.
   For example, to open the configuration file on the Cinder node:

```
vi /etc/cinder/cinder.conf
```

2. Add the Cinder Backup configuration to the `[ceph]` section of the configuration file.

```
backup_driver = cinder.backup.drivers.ceph
backup_ceph_conf = /etc/ceph/ceph.conf
backup_ceph_pool = backups
backup_ceph_user = cinder-backup
backup_ceph_chunk_size = 134217728
backup_ceph_stripe_unit = 0
```

```
backup_ceph_stripe_count = 0
restore_discard_excess_bytes = true
```

3. Verify that the Cinder Backup service is enabled in the OpenStack dashboard configuration.

Result

Cinder Backup is configured to use Ceph Block Devices for storing backup data. Volume backups can now be created and managed by using the Ceph backend.

What to do next

Restart the OpenStack service to complete the configuration. For more information, see “Restarting the OpenStack services” on page 1359.

Configuring Glance to use Ceph Block Devices

Configure OpenStack Glance to store virtual machine images in Ceph block storage.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root access to the Glance API node.
- A Ceph pool is created for storing images.

About this task

Glance manages virtual machine images in OpenStack. When configured to use Ceph, images are stored as Ceph Block Devices (RBDs), which enables efficient cloning and improved performance through copy-on-write operations.

Procedure

1. Edit the Glance API configuration file.
   For example, to open the configuration file on the Glance API node:

   ```
   vi /etc/glance/glance-api.conf
   ```

2. Configure Glance to use Ceph as the default image store.

   ```
   stores = rbd
   default_store = rbd
   rbd_store_pool = images
   rbd_store_user = glance
   rbd_store_ceph_conf = /etc/ceph/ceph.conf
   rbd_store_chunk_size = 8
   ```

3. Enable copy-on-write image cloning.

   ```
   show_image_direct_url = True
   ```

4. If enabled, disable cache management.
   Ensure the following option is set:

   ```
   flavor = keystone
   ```

Result

Glance is configured to use Ceph Block Devices as the backend for virtual machine images. Images can now be stored, cloned, and managed efficiently by using Ceph.

What to do next

Restart the OpenStack service to complete the configuration. For more information, see “Restarting the OpenStack services” on page 1359.

Configuring Nova to use Ceph Block Devices

Configure OpenStack Nova to store virtual machine disks in Ceph block storage.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root access to all Nova compute nodes.
- Ceph authentication is configured for Nova.

About this task

Nova uses the libvirt driver to manage virtual machine disks. When configured to use Ceph, Nova stores instance disks as Ceph Block Devices (RBDs), which improves scalability and enables features such as live migration.

Procedure

1. Edit the Ceph configuration file on the Nova compute nodes.
   For example, to open the configuration file on the Nova compute nodes:

   ```
   vi /etc/ceph/ceph.conf
   ```

   a. Add the required client configuration settings.

   ```
   [client]
   rbd cache = true
   rbd cache writethrough until flush = true
   rbd concurrent management ops = 20
   admin socket = /var/run/ceph/guests/$cluster-$type.$id.$pid.$cctid.asok
   log file = /var/log/ceph/qemu-guest-$pid.log
   ```

2. Create the required directories and set permissions.
   Run the following commands on each Nova compute node:

   ```
   mkdir -p /var/run/ceph/guests /var/log/ceph
   chown qemu:libvirt /var/run/ceph/guests /var/log/ceph
   ```

3. Edit the Nova configuration file.
   For example, to open the configuration file on the Nova compute nodes:

   ```
   vi /etc/nova/nova.conf
   ```

   a. Configure the `[libvirt]` section to use Ceph.
   Ensure the following options are present:

   ```
   [libvirt]
   images_type = rbd
   images_rbd_pool = vms
   images_rbd_ceph_conf = /etc/ceph/ceph.conf
   rbd_user = cinder
   rbd_secret_uuid = UUID
   disk_cachemodes = network=writeback
   hw_disk_discard = unmap
   ```

   Replace *`UUID`* with the UUID of the Ceph secret configured for Nova.

Result

Nova is configured to store virtual machine disks in Ceph Block Devices. New instances can now use Ceph-backed disks, enabling scalable and resilient compute storage.

What to do next

Restart the OpenStack service to complete the configuration. For more information, see “Restarting the OpenStack services” on page 1359.

Restarting the OpenStack services

Restart OpenStack services to apply the Ceph configuration changes.

Before you begin

Before you begin, make sure that you have completed the configuration of all OpenStack services that use Ceph, as detailed in Installing and configuring Ceph for OpenStack.

1. You have completed configuration of all OpenStack services that use Ceph.

About this task

After configuring OpenStack services to use Ceph Block Devices, the services must be restarted so that the configuration changes take effect.

Procedure

1. Restart the OpenStack services by running the following commands on each of the OpenStack nodes.

```
systemctl restart openstack-cinder-volume
systemctl restart openstack-cinder-backup
systemctl restart openstack-glance-api
systemctl restart openstack-nova-compute
```

2. Verify that all services start successfully.
   Check the service status and logs to confirm that no errors are reported.

Result

All OpenStack services are restarted successfully, and the Ceph configuration changes are active.

Red Hat Ceph Storage Developer

Use the various application programming interfaces (APIs) for Red Hat Ceph Storage running on AMD64 and Intel 64 architectures.

Ceph RESTful API

As a storage administrator, you can use the Ceph RESTful API, or simply the Ceph API, provided by the Red Hat Ceph Storage Dashboard to interact with the Red Hat Ceph Storage cluster. The Ceph API provides programmatic access to dashboard functionality and cluster resources, enabling administrators and automation tools to monitor, configure, and manage Red Hat Ceph Storage deployments. Most functionality that is available through the Dashboard user interface is also available through API endpoints.

The Ceph API uses the following standards:

- HTTP 1.1
- JSON
- MIME and HTTP Content Negotiation
- JWT

The API is OpenAPI 3.0 compliant and uses these standards to define syntax, content encoding, versioning, authentication, and authorization.

API documentation

The Red Hat Ceph Storage Dashboard provides OpenAPI-compliant API documentation that describes available endpoints, request parameters, request and response schemas, and supported HTTP status codes.

You can access the interactive OpenAPI (Swagger) documentation at:

```
https://<dashboard-hostname>:<dashboard-port>/#/api-docs
```

You can use the API through the interactive OpenAPI (Swagger) interface or access API endpoints directly by using tools such as `curl`, scripts, and automation frameworks.

The OpenAPI interface provides:

- Endpoint documentation
- Request parameters
- Request and response schemas
- Parameter descriptions
- Example payloads
- Supported HTTP status codes

Before using an API endpoint, review the API specification to understand:

- The purpose of the endpoint.
- Required permissions.
- Required and optional parameters.
- Supported parameter values.
- Expected responses and error conditions.

The API specification also provides request and response examples that can help you develop automation and integrations.

Understanding API parameters

API parameters can be required or optional. Parameter documentation typically includes the following information:

- Parameter name
- Data type
- Description
- Allowed values
- Default value, if applicable
- Whether the parameter is required or optional

Review parameter requirements before sending requests to avoid validation errors.

Understanding API responses

API responses are returned in JSON format. Depending on the operation, responses can include:

- Configuration information
- Resource details
- Status information
- Validation results
- Error messages

Review the endpoint documentation to understand the structure of the returned data.

Understanding API errors

The Red Hat Ceph Storage Dashboard API uses standard HTTP status codes:

Status code	Description
`200`	Request completed successfully.
`400`	Invalid request or parameter validation failure.
`401`	Authentication required.
`403`	Permission denied.
`404`	Resource not found.
`409`	Resource conflict.
`500`	Internal server error.

When troubleshooting API failures:

- Verify authentication credentials.
- Verify user permissions.
- Verify parameter values.
- Verify resource identifiers.

> **Tip:** For HTTP `500` errors, review the Dashboard and Ceph Manager logs for additional diagnostic information. Increasing the log level can provide more detailed troubleshooting information to help identify the root cause of a failure.

Authentication and authorization for the Ceph API

Access to the Ceph RESTful API requires authentication and authorization. You can access APIs through the Dashboard OpenAPI (Swagger) interface after logging in to the dashboard, or by using JWT authentication when accessing API endpoints directly through tools such as `curl`, scripts, and automation frameworks. For more information, see Authentication and authorization for the Ceph API.

Enabling and securing the Ceph API

The Ceph RESTful API provides secure programmatic access to Red Hat Ceph Storage cluster resources. API access is secured through HTTPS and can be controlled through Dashboard authentication and role-based access control (RBAC). For more information, see “Enabling and Securing the Ceph API” on page 1364

Using the Ceph RESTful API

The Ceph API can be used to view, monitor, configure, and manage Red Hat Ceph Storage cluster resources through API endpoints.

Versioning for the Ceph API

The main goal for the Ceph RESTful API is to provide a stable interface, which is achieved by versioning.

To achieve a stable interface, the Ceph API is built on the following principles:

- A mandatory explicit default version for all endpoints to avoid implicit defaults.
- Fine-grain change control per-endpoint.
  - The expected version from a specific endpoint is stated in the HTTP header.

    Syntax

    ```
    Accept: application/vnd.ceph.api.vMAJOR.MINOR+json
    ```

    Example

    ```
    Accept: application/vnd.ceph.api.v1.0+json
    ```

    If the current Ceph API server is not able to address that specific version, a `415 - Unsupported Media Type` response will be returned.
- Using semantic versioning.
  - Major changes are backwards incompatible. Changes might result in non-additive changes to the request, and to the response formats for a specific endpoint.
  - Minor changes are backwards and forwards compatible. Changes consist of additive changes to the request or response formats for a specific endpoint.

Authentication and authorization for the Ceph API

Access to the Ceph RESTful API involves two stages: authentication, which verifies the identity of the user making the request, and authorization, which determines whether the authenticated user can perform a specific action through an API endpoint.

You can access the Ceph API through the Dashboard OpenAPI (Swagger) interface or by using tools such as **`curl`**, scripts, and automation frameworks to access API endpoints directly. The OpenAPI (Swagger) interface is available from the dashboard through the **Help** menu.

For an overview of API access methods and the OpenAPI documentation interface, see “Ceph RESTful API specifications” on page 1571.

API versioning

The Ceph RESTful API uses versioned media types to provide a stable interface and support API compatibility across releases.

When accessing API endpoints directly, you must specify the API version in the Accept header. All endpoints require an explicit API version, beginning with version 1.0.

```
Accept: application/vnd.ceph.api.v1.0+json
```

The API version is specified in the `Accept` header using the following format.

```
application/vnd.ceph.api.vMAJOR.MINOR+json
```

If the requested API version is not supported, the API returns an `HTTP 415 Unsupported Media Type` response.

For example,

```
curl -X GET "https://example.com:8443/api/osd" \
-H "Accept: application/vnd.ceph.api.v1.0+json" \
-H "Authorization: Bearer JWT"
```

Authentication

Authentication verifies the identity of the user that is making the API request.

API requests can be authenticated through the dashboard OpenAPI (Swagger) interface after logging in to the dashboard, or by using a JSON Web Token (JWT) when accessing API endpoints directly.

Applications and automation tools that interact with the API outside of the dashboard must obtain and present a valid JWT before accessing protected resources.

Obtaining a JSON Web Token (JWT)

When accessing API endpoints directly, you must obtain a valid JSON Web Token (JWT). Use the `/api/auth` endpoint to authenticate and retrieve a token.

For example,

```
curl -X POST "https://example.com:8443/api/auth" \
-H "Accept: application/vnd.ceph.api.v1.0+json" \
-H "Content-Type: application/json" \
-d '{"username":"user1","password":"password1"}'
```

Include the resulting token in the `Authorization` HTTP header of every API request.

JWTs expire and must be renewed by authenticating again through the /api/auth endpoint. By default, a JWT is valid for 8 hours. You can specify a custom token lifetime when requesting a token by using the **`ttl`** parameter.

You can specify a custom token lifetime when requesting a token.

```
curl -X POST "https://example.com:8443/api/auth" \
-H "Accept: application/vnd.ceph.api.v1.0+json" \
-H "Content-Type: application/json" \
-d '{"username":"user1","password":"password1","ttl":24}'
```

You can also configure the default token lifetime for the dashboard.

```
ceph dashboard set-jwt-token-ttl SECONDS
```

Authorization

Authorization determines whether an authenticated user has permission to perform an operation on a resource.

Access to API endpoints is controlled by the Red Hat Ceph Storage dashboard role-based access control (RBAC) framework. Users can perform only the operations that are permitted by their assigned roles and capabilities.

For example, a user might be authorized to view cluster information but not to create or modify resources.

Before developing automation or integrations, ensure that the user account has the permissions required for the target API operations.

For more information about users and roles, see “Ceph user management” on page 445.

Protecting API credentials

JWTs grant access to API resources and should be protected in the same manner as user credentials.

Use the following practices to protect your user credentials:

- Store tokens securely.
- Limit access to tokens.
- Do not share tokens between users.
- Do not expose tokens in scripts, logs, or troubleshooting output.

- Replace tokens if you suspect they have been compromised.

HTTP status codes

The API returns standard HTTP status codes to indicate the result of a request.

Status code	Description
`201`	Resource created.
`202`	Operation is still executing. Check the task queue.
`204`	Resource deleted.
`401`	Authentication required, invalid authentication credentials, or an expired JSON Web Token (JWT). A 401 response can indicate that the JWT has expired and must be renewed through the /api/auth endpoint.
`403`	Authenticated user does not have permission to perform the requested operation.
`415`	Unsupported API version or media type specified in the `Accept` header.

When troubleshooting access failures:

- Verify that the JWT is valid.
- Verify that the token is included in the `Authorization` header.
- Verify that the correct API version is specified in the `Accept` header.
- Verify that the user account has the required Dashboard roles and capabilities.
- Verify that the requested operation is permitted for the user.
- Verify that the JWT has not expired. Expired tokens must be renewed through the `/api/auth` endpoint.

Enabling and Securing the Ceph API

The Red Hat Ceph Storage Dashboard module providesthe RESTful API access to the storage cluster over an SSL-secured connection.

> **Important:** If disabling SSL, then user names and passwords are sent unencrypted to the Red Hat Ceph Storage Dashboard.

Prerequisites

- Root-level access to a Ceph Monitor node.
- Ensure that you have at least one `ceph-mgr` daemon active.
- If you use a firewall, ensure that TCP port `8443`, for SSL, and TCP port `8080`, without SSL, are open on the node with the active `ceph-mgr` daemon.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   root@host01 ~]# cephadm shell
   ```

2. Enable the RESTful plug-in:

   ```
   [ceph: root@host01 /]# ceph mgr module enable dashboard
   ```

3. Configure an SSL certificate.

a. If your organization's certificate authority (CA) provides a certificate, then set using the certificate files:

Syntax

```
ceph dashboard set-ssl-certificate HOST_NAME -i CERT_FILE
ceph dashboard set-ssl-certificate-key HOST_NAME -i KEY_FILE
```

Example

```
[ceph: root@host01 /]# ceph dashboard set-ssl-certificate -i dashboard.crt
[ceph: root@host01 /]# ceph dashboard set-ssl-certificate-key -i dashboard.key
```

If you want to set unique node-based certificates, then add a *HOST_NAME* to the commands:

Example

```
[ceph: root@host01 /]# ceph dashboard set-ssl-certificate host01 -i dashboard.crt
[ceph: root@host01 /]# ceph dashboard set-ssl-certificate-key host01 -i
dashboard.key
```

b. Alternatively, you can generate a self-signed certificate. However, using a self-signed certificate does not provide full security benefits of the HTTPS protocol:

```
[ceph: root@host01 /]# ceph dashboard create-self-signed-cert
```

> **Warning:** Most modern web browsers will complain about self-signed certificates, which require you to confirm before establishing a secure connection.

4. Create a user, set the password, and set the role:

Syntax

```
echo -n "PASSWORD" > PATH_TO_FILE/PASSWORDFILE
ceph dashboard ac-user-create USERNAME -i PASSWORDFILE ROLE
```

Example

```
[ceph: root@host01 /]# echo -n "p@ssw0rd" > /root/dash-password.txt
[ceph: root@host01 /]# ceph dashboard ac-user-create user1 -i /root/dash-password.txt
administrator
```

This example creates a user named `user1` with the `administrator` role.

5. Connect to the RESTful plug-in web page. Open a web browser and enter the following URL:

Syntax

```
https://HOST_NAME:8443
```

Example

```
https://host01:8443
```

If you used a self-signed certificate, confirm a security exception.

Reference

- The **`ceph dashboard --help`** command.
- The `https://HOST_NAME:8443/doc` page, where *HOST_NAME* is the IP address or name of the node with the running `ceph-mgr` instance.
- The *Security hardening* guide for your Red Hat Enterprise Linux version on Red Hat Documentation.

Using the Ceph RESTful API

Learn how to use the Ceph API to view information about the storage cluster, Ceph Monitors, OSDs, pools, and hosts.

Viewing information can be done by either using the command line interface (CLI), Python, or a web browser.

Viewing cluster configuration options

Understand how to use the RESTful plug-in to view cluster configuration options and their values.

Viewing cluster configuration options using the `curl` command

On the command line, run the following command:

```
curl -X GET \
"https://CEPH_MANAGER:CEPH_MANAGER_PORT/api/cluster_conf" \
-H "Accept: application/vnd.ceph.api.v1.0+json" \
-H "Authorization: Bearer <JWT_TOKEN>"
```

Replace:

- *USER* with the username.
- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *CEPH_MANAGER_PORT* with the TCP port number.
  The default TCP port number is 8443.

Enter the user's password when prompted.

> **Note:** If using a self-signed certificate, use the `--insecure` option.
>
> ```
> curl --silent --insecure --user USER 'https://CEPH_MANAGER:8443/api/cluster_conf'
> ```

Viewing cluster configuration options using Python

In the Python interpreter, enter the following:

```
import requests

headers = {
    "Accept": "application/vnd.ceph.api.v1.0+json",
    "Authorization": "Bearer <JWT_TOKEN>"
}

result = requests.get(
    "https://CEPH_MANAGER:8080/api/cluster_conf",
    headers=headers
)

print(result.json())
```

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *USER* with the username.
- *PASSWORD* with the user's password.

> **Note:** If using a self-signed certificate, use `verify=False`.
>
> ```
> $ python
> >> import requests
> >> result = requests.get('https://CEPH_MANAGER:8443/api/cluster_conf', auth=("USER",
> ```

```
"PASSWORD"), verify=False)
>> print(result.json())
```

Viewing cluster configuration options using a web browser

In the web browser, go to `https://CEPH_MANAGER:8443/api/cluster_conf`.

Replace *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.

Enter the username and password when prompted.

Related information

Configuring

Viewing a particular cluster configuration option

Understand how to view a particular cluster option and its value.

Viewing a particular cluster configuration option using the `curl` command

On the command line, run the following command.

```
curl --silent --user USER 'https://CEPH_MANAGER:8080/api/cluster_conf/ARGUMENT'
```

Replace:

- *USER* with the username.
- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *ARGUMENT* with the configuration option you want to view.

Enter the user's password when prompted.

Note: If you used a self-signed certificate, use the `--insecure` option.

```
curl --silent --insecure --user USER 'https://CEPH_MANAGER:8080/api/cluster_conf/
ARGUMENT'
```

Viewing a particular cluster configuration option using Python

In the Python interpreter, enter the following:

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/cluster_conf/ARGUMENT',
auth=("USER", "PASSWORD"))
>> print(result.json())
```

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *ARGUMENT* with the configuration option you want to view.
- *USER* with the username.
- *PASSWORD* with the user's password.

Note: If using a self-signed certificate, use `verify=False`.

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/cluster_conf/ARGUMENT',
auth=("USER", "PASSWORD"), verify=False)
>> print(result.json())
```

Viewing a particular cluster configuration option using a web browser

In the web browser, go to `https://CEPH_MANAGER:8080/api/cluster_conf/ARGUMENT`.

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *ARGUMENT* with the configuration option you want to view.

Enter the username and password when prompted.

Related information

Configuring

Viewing all configuration options for OSDs

Understand how to view all configuration options and their values for OSDs.

Viewing all configuration options for OSDs using the `curl` command

On the command line, run the following command.

```
curl --silent --user USER 'https://CEPH_MANAGER:8080/api/osd/flags'
```

Replace:

- *USER* with the username.
- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.

Enter the user's password when prompted.

Note: If you used a self-signed certificate, use the `--insecure` option.

```
curl --silent --insecure --user USER 'https://CEPH_MANAGER:8080/api/osd/flags'
```

Viewing all configuration options for OSDs using Python

In the Python interpreter, enter the following:

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/osd/flags', auth=("USER",
"PASSWORD"))
>> print(result.json())
```

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *USER* with the username.
- *PASSWORD* with the user's password.

If you used a self-signed certificate, use the `verify=False` option.

Note: If using a self-signed certificate, use `verify=False`.

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/osd/flags', auth=("USER",
"PASSWORD"), verify=False))
>> print(result.json())
```

Viewing all configuration options for OSDs using a web browser

In the web browser, go to `userhttps://CEPH_MANAGER:8080/api/osd/flags`.

Replace *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.

Enter the username and password when prompted.

Related information

Configuring

Viewing CRUSH rules

Understand how to view CRUSH rules.

Viewing CRUSH rules using the `curl` command

On the command line, run the following command:

```
curl --silent --user USER 'https://CEPH_MANAGER:8080/api/crush_rule'
```

Replace:

- *USER* with the username.
- *CEPH_MANAGER* with the IP address or short host name of the node with the active `ceph-mgr` instance.

Enter the user's password when prompted.

Note: If using a self-signed certificate, use the `--insecure` option.

```
curl --silent --insecure --user USER 'https://CEPH_MANAGER:8080/api/crush_rule'
```

Viewing CRUSH rules using Python

In the Python interpreter, enter the following:

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/crush_rule', auth=("USER",
"PASSWORD"))
>> print(result.json())
```

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *USER* with the username.
- *PASSWORD* with the user's password.

Note: If using a self-signed certificate, use `verify=False`.

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/crush_rule', auth=("USER",
"PASSWORD"), verify=False)
>> print(result.json())
```

Viewing CRUSH rules using a web browser

In the web browser, go to `https://CEPH_MANAGER:8080/api/crush_rule`.

Replace *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.

Enter the username and password when prompted.

Related information

CRUSH rules

Viewing Monitor information

Understand how to use the RESTful plug-in to view Monitor information, such as IP address, name, and quorum status.

Viewing Monitor information using the `curl` command

On the command line, run the following command:

```
curl --silent --user USER 'https://CEPH_MANAGER:CEPH_MANAGER_PORT/api/monitor'
```

Replace:

- *USER* with the username.
- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *CEPH_MANAGER_PORT* with the TCP port number.
  The default TCP port number is 8443.

Enter the user's password when prompted.

Note: If using a self-signed certificate, use the `--insecure` option.

```
curl --silent --insecure --user USER 'https://CEPH_MANAGER:CEPH_MANAGER_PORT/api/
monitor'
```

Viewing Monitor information using Python

In the Python interpreter, enter the following:

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/monitor', auth=("USER",
"PASSWORD"))
>> print(result.json())
```

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *USER* with the username.
- *PASSWORD* with the user's password.

Note: If using a self-signed certificate, use `verify=False`.

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/monitor', auth=("USER",
"PASSWORD"), verify=False)
>> print(result.json())
```

Viewing Monitor information using a web browser

In the web browser, go to `https://CEPH_MANAGER:8080/api/monitor`.

Replace *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.

Enter the username and password when prompted.

Viewing information about a particular Monitor

Understand how to use the RESTful plug-in to view information about a particular Monitor, such as IP address, name, and quorum status.

Viewing information about a particular Monitor using the `curl` command

On the command line, run the following command:

```
curl --silent --user USER 'https://CEPH_MANAGER:8080/api/monitor/MON_NAME'
```

Replace:

- *USER* with the username.
- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *MON_NAME* with the short hostname of the Monitor.

Enter the user's password when prompted.

Note: If using a self-signed certificate, use the `--insecure` option.

```
curl --silent --insecure --user USER 'https://CEPH_MANAGER:8080/api/monitor/MON_NAME'
```

Viewing information about a particular Monitor using Python

In the Python interpreter, enter the following:

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/monitor/MON_NAME', auth=("USER",
"PASSWORD"))
>> print(result.json())
```

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *MON_NAME* with the short hostname of the Monitor
- *USER* with the username.
- *PASSWORD* with the user's password.

Note: If using a self-signed certificate, use `verify=False`.

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/monitor/MON_NAME',
auth=("USER", "PASSWORD"), verify=False)
>> print(result.json())
```

Viewing information about a particular Monitor using a web browser

In the web browser, go to `https://CEPH_MANAGER:8080/api/monitor/MON_NAME`.

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *MON_NAME* with the short hostname of the Monitor

Enter the username and password when prompted.

Viewing OSD information

Understand how to use the RESTful plug-in to view OSD information, such as IP address, Its pools, affinity, and weight.

Viewing OSD information using the `curl` command

On the command line, run the following command:

```
curl --silent --user USER 'https://CEPH_MANAGER:8080/api/osd'
```

Replace:

- *USER* with the username.
- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.

Enter the user's password when prompted.

Note: If using a self-signed certificate, use the `--insecure` option.

```
curl --silent --insecure --user USER 'https://CEPH_MANAGER:8080/api/osd'
```

Viewing OSD information using Python

In the Python interpreter, enter the following:

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/osd/', auth=("USER", "PASSWORD"))
>> print(result.json())
```

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *USER* with the username.
- *PASSWORD* with the user's password.

Note: If using a self-signed certificate, use `verify=False`.

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/osd/', auth=("USER",
```

```
"PASSWORD"), verify=False)
>> print(result.json())
```

Viewing OSD information using a web browser

In the web browser, go to `https://CEPH_MANAGER:8080/api/osd`.

Replace *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.

Enter the username and password when prompted.

Viewing information about a particular OSD

Understand how to use the RESTful plug-in to view information about a particular OSD, such as IP address, Its pools, affinity, and weight.

Viewing information about a particular OSD using the `curl` command

On the command line, run the following command:

```
curl --silent --user USER 'https://CEPH_MANAGER:8080/api/osd/ID'
```

Replace:

- *USER* with the username.
- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *ID* with the ID of the OSD listed in the `osd` field

Enter the user's password when prompted.

Note: If using a self-signed certificate, use the `--insecure` option.

```
curl --silent --insecure --user USER 'https://CEPH_MANAGER:8080/api/osd/ID'
```

Viewing information about a particular OSD using Python

In the Python interpreter, enter the following:

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/osd/ID', auth=("USER", "PASSWORD"))
>> print(result.json())
```

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *ID* with the ID of the OSD listed in the `osd` field
- *USER* with the username.
- *PASSWORD* with the user's password.

Note: If using a self-signed certificate, use `verify=False`.

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/osd/ID', auth=("USER",
```

```
"PASSWORD"), verify=False)
>> print(result.json())
```

Viewing information about a particular OSD using a web browser

In the web browser, go to `https://CEPH_MANAGER:8080/api/osd/ID`.

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *ID* with the ID of the OSD listed in the `osd` field

Enter the username and password when prompted.

Determining processes that can be scheduled on an OSD

Understand how to use the RESTful plug-in to view which processes, such as scrubbing or deep scrubbing, can be scheduled on an OSD.

Determining processes that can be scheduled on an OSD using the `curl` command

On the command line, run the following command:

```
curl --silent --user USER 'https://CEPH_MANAGER:8080/api/osd/ID/command'
```

Replace:

- *USER* with the username.
- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *ID* with the ID of the OSD listed in the `osd` field

Enter the user's password when prompted.

Note: If using a self-signed certificate, use the `--insecure` option.

```
curl --silent --insecure --user USER 'https://CEPH_MANAGER:8080/api/osd/ID/command'
```

Determining processes that can be scheduled on an OSD using Python

In the Python interpreter, enter the following:

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/osd/ID/command', auth=("USER",
"PASSWORD"))
>> print(result.json())
```

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *ID* with the ID of the OSD listed in the `osd` field
- *USER* with the username.
- *PASSWORD* with the user's password.

Note: If using a self-signed certificate, use `verify=False`.

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/osd/ID/command', auth=("USER",
"PASSWORD"), verify=False)
>> print(result.json())
```

Determining processes that can be scheduled on an OSD using a web browser

In the web browser, go to `https://CEPH_MANAGER:8080/api/osd/ID/command`.

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *ID* with the ID of the OSD listed in the `osd` field

Enter the username and password when prompted.

Viewing pool information

Understand how to use the RESTful plug-in to view pool information, such as flags, size, and number of placement groups (PGs).

Viewing pool information using the `curl` command

On the command line, run the following command:

```
curl --silent --user USER 'https://CEPH_MANAGER:8080/api/pool'
```

Replace:

- *USER* with the username.
- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.

Enter the user's password when prompted.

Note: If using a self-signed certificate, use the `--insecure` option.

```
curl --silent --insecure --user USER 'https://CEPH_MANAGER:8080/api/pool'
```

Viewing pool information using Python

In the Python interpreter, enter the following:

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/pool', auth=("USER", "PASSWORD"))
>> print(result.json())
```

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *USER* with the username.
- *PASSWORD* with the user's password.

Note: If using a self-signed certificate, use `verify=False`.

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/pool', auth=("USER",
```

```
"PASSWORD"), verify=False)
>> print(result.json())
```

Viewing pool information using a web browser

In the web browser, go to `https://CEPH_MANAGER:8080/api/pool`.

Replace *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.

Enter the username and password when prompted.

Viewing information about a particular pool

Understand how to use the RESTful plug-in to view information about a particular pool, such as flags, size, and number of placement groups (PGs).

Viewing information about a particular pool using the `curl` command

On the command line, run the following command:

```
curl --silent --user USER 'https://CEPH_MANAGER:8080/api/pool/ID'
```

Replace:

- *USER* with the username.
- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *ID* with the ID of the OSD listed in the `osd` field

Enter the user's password when prompted.

Note: If using a self-signed certificate, use the `--insecure` option.

```
curl --silent --insecure --user USER 'https://CEPH_MANAGER:8080/api/pool/ID'
```

Viewing information about a particular pool using Python

In the Python interpreter, enter the following:

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/pool/ID', auth=("USER",
"PASSWORD"))
>> print(result.json())
```

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *ID* with the ID of the OSD listed in the `pool` field
- *USER* with the username.
- *PASSWORD* with the user's password.

Note: If using a self-signed certificate, use `verify=False`.

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/pool/ID', auth=("USER",
```

```
"PASSWORD"), verify=False)
>> print(result.json())
```

Viewing information about a particular pool using a web browser

In the web browser, go to `https://CEPH_MANAGER:8080/api/pool/ID`.

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *ID* with the ID of the OSD listed in the `pool` field

Enter the username and password when prompted.

Viewing host information

Understand how to use the RESTful plug-in to view host information, such as hostnames, Ceph daemons and their IDs, and Ceph versions.

Viewing host information using the `curl` command

On the command line, run the following command:

```
curl --silent --user USER 'https://CEPH_MANAGER:8080/api/host'
```

Replace:

- *USER* with the username.
- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.

Enter the user's password when prompted.

Note: If using a self-signed certificate, use the `--insecure` option.

```
curl --silent --insecure --user USER 'https://CEPH_MANAGER:8080/api/host'
```

Viewing host information using Python

In the Python interpreter, enter the following:

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/host', auth=("USER", "PASSWORD"))
>> print(result.json())
```

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *USER* with the username.
- *PASSWORD* with the user's password.

Note: If using a self-signed certificate, use `verify=False`.

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/host', auth=("USER",
```

```
"PASSWORD"), verify=False)
>> print(result.json())
```

Viewing host information using a web browser

In the web browser, go to `https://CEPH_MANAGER:8080/api/host`.

Replace *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.

Enter the username and password when prompted.

Viewing information about a particular host

Understand how to use the RESTful plug-in to view information about a particular host, such as hostnames, Ceph daemons and their IDs, and Ceph versions.

Viewing information about a particular host using the `curl` command

On the command line, run the following command:

```
curl --silent --user USER 'https://CEPH_MANAGER:8080/api/host/HOSTNAME'
```

Replace:

- *USER* with the username.
- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *HOSTNAME* with the host name of the host listed in the `host` field.

Enter the user's password when prompted.

Note: If using a self-signed certificate, use the `--insecure` option.

```
curl --silent --insecure --user USER 'https://CEPH_MANAGER:8080/api/host/HOSTNAME'
```

Viewing information about particular host using Python

In the Python interpreter, enter the following:

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/host/HOSTNAME', auth=("USER",
"PASSWORD"))
>> print(result.json())
```

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *HOSTNAME* with the host name of the host listed in the `host` field
- *USER* with the username.
- *PASSWORD* with the user's password.

Note: If using a self-signed certificate, use `verify=False`.

```
$ python
>> import requests
>> result = requests.get('https://CEPH_MANAGER:8080/api/host', auth=("USER",
```

```
"PASSWORD"), verify=False)
>> print(result.json())
```

Viewing information about a particular host using a web browser

In the web browser, go to `https://CEPH_MANAGER:8080/api/host/HOSTNAME`.

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *HOSTNAME* with the host name of the host listed in the `host` field.

Enter the username and password when prompted.

Changing OSD configurations

Learn how to use the Ceph API to change OSD configuration options, the state of an OSD, and information about pools.

Changing the OSD configurations can be done by using either the command line interface (CLI) or Python.

Changing OSD configuration options

Understand how to use the RESTful plug-in to change OSD configuration options.

Changing OSD configuration options using the `curl` command

On the command line, run the following command:

```
echo -En '{"=OPTION": VALUE}' | curl --request PATCH --data @- --silent --
user USER 'https://CEPH_MANAGER:8080/api/osd/flags'
```

Replace:

- *OPTION* with one of the modifying options:`pause`,`noup`,`nodown`,`noout`,`noin`,`nobackfill`,`norecover`, `noscrub`, or`nodeep-scrub`.
- *VALUE* with`true`or`false`.
- *USER* with the username.
- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.

Enter the user's password when prompted.

Note: If using a self-signed certificate, use the`--insecure`option.

```
echo -En '{"=OPTION": VALUE}' | curl --request PATCH --data @- --silent --insecure --
user USER 'https://CEPH_MANAGER:8080/api/osd/flags'
```

Changing OSD configuration options using Python

In the Python interpreter, enter the following:

```
$ python
>> import requests
>> result = requests.patch('https://CEPH_MANAGER:8080/api/osd/flags',
json={"OPTION": VALUE}, auth=("USER", "PASSWORD"))
>> print(result.json())
```

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.

- *OPTION* with one of the modifying options:`pause`,`noup`,`nodown`,`noout`,`noin`,`nobackfill`,`norecover`, `noscrub`, or`nodeep-scrub`.
- *VALUE* with`true`or`false`.
- *USER* with the username.
- *PASSWORD* with the user's password.

Note: If using a self-signed certificate, use `verify=False`.

```
$ python
>> import requests
>> result = requests.patch('https://CEPH_MANAGER:8080/api/osd/flags',
json={"OPTION": VALUE}, auth=("USER", "PASSWORD"), verify=False)
>> print(result.json())
```

Changing an OSD state

Understand how to use the RESTful plug-in to change the state of an OSD.

Changing the state of an OSD using the `curl` command

On the command line, run the following command:

```
echo -En '{"=STATE": VALUE}' | curl --request PATCH --data @- --silent --
user USER 'https://CEPH_MANAGER:8080/api/osd/ID'
```

Replace:

- *STATE* with either`in`or`up`.
- *VALUE* with`true`or`false`.
- *USER* with the username.
- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *ID* with the ID of the OSD listed in the `osd` field.

Enter the user's password when prompted.

Note: If using a self-signed certificate, use the`--insecure`option.

```
echo -En '{"=STATE": VALUE}' | curl --request PATCH --data @- --silent --insecure --
user USER 'https://CEPH_MANAGER:8080/api/osd/ID'
```

Changing the state of an OSD using Python

In the Python interpreter, enter the following:

```
$ python
>> import requests
>> result = requests.patch('https://CEPH_MANAGER:8080/api/osd/ID', json={"STATE": VALUE},
auth=("USER", "PASSWORD"))
>> print(result.json())
```

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *STATE* with either`in`or`up`.
- *VALUE* with`true`or`false`.
- *USER* with the username.

- *PASSWORD* with the user's password.
- *ID* with the ID of the OSD listed in the `osd` field.

Note: If using a self-signed certificate, use `verify=False`.

```
$ python
>> import requests
>> result = requests.patch('https://CEPH_MANAGER:8080/api/osd/ID',
json={"STATE": VALUE}, auth=("USER", "PASSWORD"), verify=False)
>> print(result.json())
```

Changing the weight of an OSD

Understand how to use the RESTful plug-in to change the weight of an OSD.

Changing the weight of an OSD using the `curl` command

On the command line, run the following command:

```
echo -En '{"=reweight": VALUE}' | curl --request PATCH --data @- --silent --
user USER 'https://CEPH_MANAGER:8080/api/osd/ID'
```

Replace:

- *VALUE* with `true` or `false`.
- *USER* with the username.
- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *ID* with the ID of the OSD listed in the `osd` field.

Enter the user's password when prompted.

Note: If using a self-signed certificate, use the `--insecure` option.

```
echo -En '{"=reweight": VALUE}' | curl --request PATCH --data @- --silent --insecure
--user USER 'https://CEPH_MANAGER:8080/api/osd/ID'
```

Changing the weight of an OSD using Python

In the Python interpreter, enter the following:

```
$ python
>> import requests
>> result = requests.patch('https://CEPH_MANAGER:8080/api/osd/ID',
json={"reweight": VALUE}, auth=("USER", "PASSWORD"))
>> print(result.json())
```

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *VALUE* with `true` or `false`.
- *USER* with the username.
- *PASSWORD* with the user's password.
- *ID* with the ID of the OSD listed in the `osd` field.

Note: If using a self-signed certificate, use `verify=False`.

```
$ python
>> import requests
>> result = requests.patch('https://CEPH_MANAGER:8080/api/osd/ID',
json={"reweight": VALUE}, auth=("USER", "PASSWORD"), verify=False)
>> print(result.json())
```

Changing pool information

Understand how to use the RESTful plug-in to change pool information.

Changing pool information using the `curl` command

On the command line, run the following command:

```
echo -En '{"=OPTION": VALUE}' | curl --request PATCH --data @- --silent --
user USER 'https://CEPH_MANAGER:8080/api/pool/ID'
```

Replace:

- *OPTION* with the option to modify.
- *VALUE* with the new value of the option.
- *USER* with the username.
- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *ID* with the ID of the pool listed in the `pool` field.

Enter the user's password when prompted.

Note: If using a self-signed certificate, use the`--insecure`option.

```
echo -En '{"=OPTION": VALUE}' | curl --request PATCH --data @- --silent --insecure --
user USER 'https://CEPH_MANAGER:8080/api/pool/ID'
```

Changing pool information using Python

In the Python interpreter, enter the following:

```
$ python
>> import requests
>> result = requests.patch('https://CEPH_MANAGER:8080/api/pool/ID', json={"OPTION": VALUE},
auth=("USER", "PASSWORD"))
>> print(result.json())
```

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *ID* with the ID of the pool listed in the `pool` field.
- *OPTION* with one of the modifying options:`pause`,`noup`,`nodown`,`noout`,`noin`,`nobackfill`,`norecover`, `noscrub`, or`nodeep-scrub`.
- *VALUE* with`true`or`false`.
- *USER* with the username.
- *PASSWORD* with the user's password.

Note: If using a self-signed certificate, use `verify=False`.

```
$ python
>> import requests
>> result = requests.patch('https://CEPH_MANAGER:8080/api/pool/ID',
json={"OPTION": VALUE}, auth=("USER", "PASSWORD"), verify=False)
>> print(result.json())
```

Administering the cluster

Learn how to use the Ceph API to initialize scrubbing or deep scrubbing on an OSD, create a pool or remove data from a pool, remove requests, or create a request.

Running a scheduled process on an OSD

Understand how to use the RESTful plug-in to run a scheduled process on an OSD, such as scrubbing or deep scrubbing.

Running a scheduled process on an OSD using the `curl` command

On the command line, run the following command:

```
echo -En '{"command": "COMMAND"}' | curl --request POST --data @- --silent --
user USER 'https://CEPH_MANAGER:8080/api/osd/ID/command'
```

Replace:

- *COMMAND* with the process you want to start. The options are `scrub`, `deep-scrub`, or `repair`. Verify if the process is supported on the OSD. For more information, see "Determining processes that can be scheduled on an OSD" on page 1374.
- *USER* with the username.
- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *ID* with the ID of the OSD listed in the `osd` field

Enter the user's password when prompted.

Note: If using a self-signed certificate, use the `--insecure` option.

```
echo -En '{"command": "COMMAND"}' | curl --request POST --data @- --silent --insecure
--user USER 'https://CEPH_MANAGER:8080/api/osd/ID/command'
```

Running a scheduled process on an OSD using Python

In the Python interpreter, enter the following:

```
$ python
>> import requests
>> result = requests.post('https://CEPH_MANAGER:8080/api/osd/ID/command', json={"command":
"COMMAND"}, auth=("USER", "PASSWORD"))
>> print(result.json())
```

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *ID* with the ID of the OSD listed in the `osd` field
- *COMMAND* with the process you want to start. The options are `scrub`, `deep-scrub`, or `repair`. Verify if the process is supported on the OSD. For more information, see "Determining processes that can be scheduled on an OSD" on page 1374.
- *USER* with the username.

– *PASSWORD* with the user's password.

Note: If using a self-signed certificate, use `verify=False`.

```
$ python
>> import requests
>> result = requests.post('https://CEPH_MANAGER:8080/api/osd/ID/command',
json={"command": "COMMAND"}, auth=("USER", "PASSWORD"), verify=False)
>> print(result.json())
```

Creating a pool

Understand how to use the RESTful plug-in to create a pool.

Creating a pool using the `curl` command

On the command line, run the following command:

```
echo -En '{"name": "NAME", "pg_num": NUMBER}' | curl --request POST --data @- --silent --
user USER 'https://CEPH_MANAGER:8080/api/pool'
```

Replace:

– *NAME* with the name of the new pool.

– *NUMBER* with the number of the placement groups.

– *USER* with the username.

– *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.

Enter the user's password when prompted.

Note: If using a self-signed certificate, use the `--insecure` option.

```
echo -En '{"name": "NAME", "pg_num": NUMBER}' | curl --request POST --data @- --silent
--insecure --user USER 'https://CEPH_MANAGER:8080/api/pool'
```

Creating a pool using Python

In the Python interpreter, enter the following:

```
$ python
>> import requests
>> result = requests.post('https://CEPH_MANAGER:8080/api/pool', json={"NAME": NUMBER},
auth=("USER", "PASSWORD"))
>> print(result.json())
```

Replace:

– *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.

– *NAME* with the name of the new pool.

– *NUMBER* with the number of the placement groups.

– *USER* with the username.

– *PASSWORD* with the user's password.

Note: If using a self-signed certificate, use `verify=False`.

```
$ python
>> import requests
>> result = requests.post('https://CEPH_MANAGER:8080/api/pool', json={"NAME": NUMBER},
auth=("USER", "PASSWORD"), verify=False)
>> print(result.json())
```

Removing a pool

Understand how to use the RESTful plug-in to remove a pool.

Removing a pool is not allowed by default. To allow the removal of a pool, in the Ceph configuration file, add the following parameter:

```
mon_allow_pool_delete = true
```

Removing a pool using the `curl` command

On the command line, run the following command:

```
curl --request DELETE --silent --user USER 'https://CEPH_MANAGER:8080/api/pool/ID'
```

Replace:

- *USER* with the username.
- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *ID* with the ID of the pool listed in the `pool` field.

Enter the user's password when prompted.

Note: If using a self-signed certificate, use the `--insecure` option.

```
curl --request DELETE --silent --insecure --user USER 'https://CEPH_MANAGER:8080/api/
pool/ID'
```

Removing a pool using Python

In the Python interpreter, enter the following:

```
$ python
>> import requests
>> result = requests.delete('https://CEPH_MANAGER:8080/api/pool/ID', auth=("USER",
"PASSWORD"))
>> print(result.json())
```

Replace:

- *CEPH_MANAGER* with the IP address or short hostname of the node with the active `ceph-mgr` instance.
- *ID* with the ID of the pool listed in the `pool` field.
- *USER* with the username.
- *PASSWORD* with the user's password.

Note: If using a self-signed certificate, use `verify=False`.

```
$ python
>> import requests
>> result = requests.delete('https://CEPH_MANAGER:8080/api/pool/ID', auth=("USER",
```

```
"PASSWORD"), verify=False)
>> print(result.json())
```

Ceph Object Gateway administrative API

As a developer, you can administer the Ceph Object Gateway by interacting with the RESTful application programming interface (API). The Ceph Object Gateway makes available the features of the **`radosgw-admin`** command in a RESTful API. You can manage users, data, quotas, and usage which you can integrate with other management platforms.

Note: Use the command-line interface when configuring the Ceph Object Gateway.

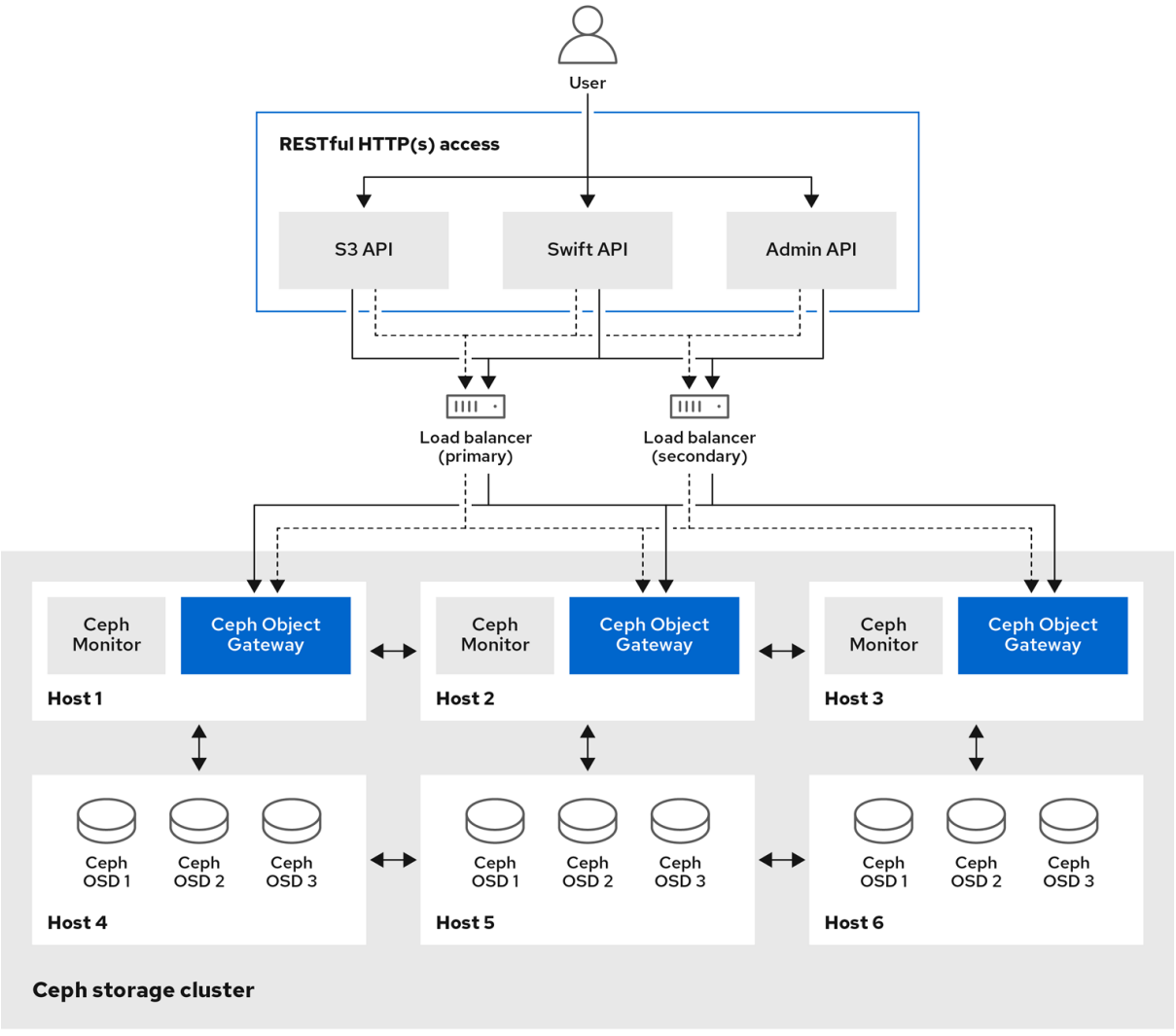


Figure 102: Basic access

Before administering with the RESTful application programming interface (API), be sure that you have both a running Red Hat Ceph Storage cluster and a RESTful client.

The administrative API provides the following functionality:

Administration operations

An administrative Application Programming Interface (API) request will be done on a URI that starts with the configurable *admin* resource entry point. Authorization for the administrative API duplicates the S3 authorization mechanism.

Some operations require that the user holds special administrative capabilities. The response entity type, either XML or JSON, might be specified as the *format* option in the request and defaults to JSON if not specified.

Example

```
PUT /admin/user?caps&format=json HTTP/1.1
Host: FULLY_QUALIFIED_DOMAIN_NAME
Content-Type: text/plain
Authorization: AUTHORIZATION_TOKEN

usage=read
```

Administration authentication requests

Amazon's S3 service uses the access key and a hash of the request header and the secret key to authenticate the request. It has the benefit of providing an authenticated request, especially large uploads, without SSL overhead.

Most use cases for the S3 API involve using open-source S3 clients such as the `AmazonS3Client` in the Amazon SDK for Java or Python Boto. These libraries do not support the Ceph Object Gateway Admin API. You can subclass and extend these libraries to support the Ceph Admin API. Alternatively, you can create a unique Gateway client.

Creating an `execute()` method

The CephAdminAPI example class in this section illustrates how to create an `execute()` method that can take request parameters, authenticate the request, call the Ceph Admin API and receive a response.

> **Important:** The `CephAdminAPI` class example is not supported or intended for commercial use. It is for illustrative purposes only.

Calling the Ceph Object Gateway

The client code contains five calls to the Ceph Object Gateway to demonstrate CRUD operations:

- Create a User
- Get a User
- Modify a User
- Create a Subuser
- Delete a User

To use this example, get the `httpcomponents-client-4.5.3` Apache HTTP components. You can download it for example here: http://hc.apache.org/downloads.cgi. Then unzip the tar file, navigate to its `lib` directory and copy the contents to the `/jre/lib/ext` directory of the `JAVA_HOME` directory, or a custom classpath.

As you examine the CephAdminAPI class example, notice that the `execute()` method takes an HTTP method, a request path, an optional subresource, `null` if not specified, and a map of parameters. To run with subresources, for example, `subuser`, and `key`, you will need to specify the subresource as an argument in the `execute()` method.

The example method:

1. Builds a URI.
2. Builds an HTTP header string.
3. Instantiates an HTTP request, for example, `PUT`, `POST`, `GET`, `DELETE`.
4. Adds the `Date` header to the HTTP header string and the request header.
5. Adds the `Authorization` header to the HTTP request header.

6. Instantiates an HTTP client and passes it the instantiated HTTP request.
7. Makes a request.
8. Returns a response.

Building the header string

Building the header string is the portion of the process that involves Amazon's S3 authentication procedure. Specifically, the example method does the following:

1. Adds a request type, for example, `PUT`, `POST`, `GET`, `DELETE`.
2. Adds the date.
3. Adds the requestPath.

The request type should be uppercase with no leading or trailing white space. If you do not trim white space, authentication will fail. The date MUST be expressed in GMT, or authentication will fail.

The exemplary method does not have any other headers. The Amazon S3 authentication procedure sorts `x-amz` headers lexicographically. So if you are adding `x-amz` headers, be sure to add them lexicographically.

Once you have built the header string, the next step is to instantiate an HTTP request and pass it the URI. The exemplary method uses `PUT` for creating a user and subuser, `GET` for getting a user, `POST` for modifying a user and `DELETE` for deleting a user.

Once you instantiate a request, add the `Date` header followed by the `Authorization` header. Amazon's S3 authentication uses the standard `Authorization` header, and has the following structure:

```
Authorization: AWS _ACCESS_KEY_:_HASH_OF_HEADER_AND_SECRET_
```

The CephAdminAPI example class has a `base64Sha1Hmac()` method, which takes the header string and the secret key for the admin user, and returns a SHA1 HMAC as a base-64 encoded string. Each `execute()` call will invoke the same line of code to build the `Authorization` header:

```
httpRequest.addHeader("Authorization", "AWS " + this.getAccessKey() + ":" +
base64Sha1Hmac(headerString.toString(), this.getSecretKey()));
```

The following `CephAdminAPI` example class requires you to pass the access key, secret key, and an endpoint to the constructor. The class provides accessor methods to change them at runtime.

Example

```
import java.io.IOException;
import java.net.URI;
import java.net.URISyntaxException;
import java.time.OffsetDateTime;
import java.time.format.DateTimeFormatter;
import java.time.ZoneId;

import org.apache.http.HttpEntity;
import org.apache.http.NameValuePair;
import org.apache.http.Header;
import org.apache.http.client.entity.UrlEncodedFormEntity;
import org.apache.http.client.methods.CloseableHttpResponse;
import org.apache.http.client.methods.HttpRequestBase;
import org.apache.http.client.methods.HttpGet;
import org.apache.http.client.methods.HttpPost;
import org.apache.http.client.methods.HttpPut;
import org.apache.http.client.methods.HttpDelete;
import org.apache.http.impl.client.CloseableHttpClient;
import org.apache.http.impl.client.HttpClients;
import org.apache.http.message.BasicNameValuePair;
import org.apache.http.util.EntityUtils;
import org.apache.http.client.utils.URIBuilder;

import java.util.Base64;
import java.util.Base64.Encoder;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import javax.crypto.spec.SecretKeySpec;
import javax.crypto.Mac;

import java.util.Map;
```

```
import java.util.Iterator;
import java.util.Set;
import java.util.Map.Entry;

public class CephAdminAPI {

        /*
         * Each call must specify an access key, secret key, endpoint and format.
         */
        String accessKey;
        String secretKey;
        String endpoint;
        String scheme = "http"; //http only.
        int port = 80;

        /*
         * A constructor that takes an access key, secret key, endpoint and format.
         */
        public CephAdminAPI(String accessKey, String secretKey, String endpoint){
                this.accessKey = accessKey;
                this.secretKey = secretKey;
                this.endpoint = endpoint;
        }

        /*
         * Accessor methods for access key, secret key, endpoint and format.
         */
        public String getEndpoint(){
                return this.endpoint;
        }

        public void setEndpoint(String endpoint){
                this.endpoint = endpoint;
        }

        public String getAccessKey(){
                return this.accessKey;
        }

        public void setAccessKey(String accessKey){
                this.accessKey = accessKey;
        }

        public String getSecretKey(){
                return this.secretKey;
        }

        public void setSecretKey(String secretKey){
                this.secretKey = secretKey;
        }

        /*
         * Takes an HTTP Method, a resource and a map of arguments and
         * returns a CloseableHTTPResponse.
         */
        public CloseableHttpResponse execute(String HTTPMethod, String resource,
                                        String subresource, Map arguments) {

                String httpMethod = HTTPMethod;
                String requestPath = resource;
                StringBuffer request = new StringBuffer();
                StringBuffer headerString = new StringBuffer();
                HttpRequestBase httpRequest;
                CloseableHttpClient httpclient;
                URI uri;
                CloseableHttpResponse httpResponse = null;

                try {

                        uri = new URIBuilder()
                                .setScheme(this.scheme)
                                .setHost(this.getEndpoint())
                                .setPath(requestPath)
                                .setPort(this.port)
                                .build();


                        if (subresource != null){
                                uri = new URIBuilder(uri)
                                        .setCustomQuery(subresource)
                                        .build();
```

```
                        }


                        for (Iterator iter = arguments.entrySet().iterator();
                        iter.hasNext();) {
                                Entry entry = (Entry)iter.next();
                                uri = new URIBuilder(uri)
                                        .setParameter(entry.getKey().toString(),
                                 entry.getValue().toString())
                                        .build();

                        }

                        request.append(uri);

                        headerString.append(HTTPMethod.toUpperCase().trim() + "\n\n\n");

                        OffsetDateTime dateTime = OffsetDateTime.now(ZoneId.of("GMT"));
                        DateTimeFormatter formatter = DateTimeFormatter.RFC_1123_DATE_TIME;
                        String date = dateTime.format(formatter);

                        headerString.append(date + "\n");
                        headerString.append(requestPath);

                        if (HTTPMethod.equalsIgnoreCase("PUT")){
                                httpRequest = new HttpPut(uri);
                        } else if (HTTPMethod.equalsIgnoreCase("POST")){
                                httpRequest = new HttpPost(uri);
                        } else if (HTTPMethod.equalsIgnoreCase("GET")){
                                httpRequest = new HttpGet(uri);
                        } else if (HTTPMethod.equalsIgnoreCase("DELETE")){
                                httpRequest = new HttpDelete(uri);
                        } else {
                                System.err.println("The HTTP Method must be PUT,
                                POST, GET or DELETE.");
                                throw new IOException();
                        }

                        httpRequest.addHeader("Date", date);
                        httpRequest.addHeader("Authorization", "AWS " + this.getAccessKey()
                        + ":" + base64Sha1Hmac(headerString.toString(),
                        this.getSecretKey()));

                        httpclient = HttpClients.createDefault();
                        httpResponse = httpclient.execute(httpRequest);

                }       catch  (URISyntaxException e){
                        System.err.println("The URI is not formatted properly.");
                        e.printStackTrace();
                }  catch (IOException e){
                        System.err.println("There was an error making the request.");
                        e.printStackTrace();
                }
                        return httpResponse;
        }

        /*
         * Takes a uri and a secret key and returns a base64-encoded
         * SHA-1 HMAC.
         */
        public String base64Sha1Hmac(String uri, String secretKey) {
                try {

                        byte[] keyBytes = secretKey.getBytes("UTF-8");
                        SecretKeySpec signingKey = new SecretKeySpec(keyBytes, "HmacSHA1");

                        Mac mac = Mac.getInstance("HmacSHA1");
                        mac.init(signingKey);

                        byte[] rawHmac = mac.doFinal(uri.getBytes("UTF-8"));

                        Encoder base64 = Base64.getEncoder();
                        return base64.encodeToString(rawHmac);

                } catch (Exception e) {
                        throw new RuntimeException(e);
                }
        }

}
```

The subsequent `CephAdminAPIClient` example illustrates how to instantiate the `CephAdminAPI` class, build a map of request parameters, and use the `execute()` method to create, get, update and delete a user.

Example

```
import java.io.IOException;
import org.apache.http.client.methods.CloseableHttpResponse;
import org.apache.http.HttpEntity;
import org.apache.http.util.EntityUtils;
import java.util.*;


public class CephAdminAPIClient {

        public static void main (String[] args){

                CephAdminAPI adminApi = new CephAdminAPI ("FFC6ZQ6EMIF64194158N",
                                                "Xac39eCAhlTGcCAUreuwe1ZuH5oVQFa51lbEMVoT",
                                                "ceph-client");

                /*
                 * Create a user
                 */
                Map requestArgs = new HashMap();
                requestArgs.put("access", "usage=read, write; users=read, write");
                requestArgs.put("display-name", "New User");
                requestArgs.put("email", "new-user@email.com");
                requestArgs.put("format", "json");
                requestArgs.put("uid", "new-user");

                CloseableHttpResponse response =
                        adminApi.execute("PUT", "/admin/user", null, requestArgs);

                System.out.println(response.getStatusLine());
                HttpEntity entity = response.getEntity();

                try {
                        System.out.println("\nResponse Content is: "
                                + EntityUtils.toString(entity, "UTF-8") + "\n");
                        response.close();
                } catch (IOException e){
                        System.err.println ("Encountered an I/O exception.");
                        e.printStackTrace();
                }

                /*
                 * Get a user
                 */
                requestArgs = new HashMap();
                requestArgs.put("format", "json");
                requestArgs.put("uid", "new-user");

                response = adminApi.execute("GET", "/admin/user", null, requestArgs);

                System.out.println(response.getStatusLine());
                entity = response.getEntity();

                try {
                        System.out.println("\nResponse Content is: "
                                + EntityUtils.toString(entity, "UTF-8") + "\n");
                        response.close();
                } catch (IOException e){
                        System.err.println ("Encountered an I/O exception.");
                        e.printStackTrace();
                }

                /*
                 * Modify a user
                 */
                requestArgs = new HashMap();
                requestArgs.put("display-name", "John Doe");
                requestArgs.put("email", "johndoe@email.com");
                requestArgs.put("format", "json");
                requestArgs.put("uid", "new-user");
                requestArgs.put("max-buckets", "100");

                response = adminApi.execute("POST", "/admin/user", null, requestArgs);

                System.out.println(response.getStatusLine());
                entity = response.getEntity();
```

```
                try {
                        System.out.println("\nResponse Content is: "
                                + EntityUtils.toString(entity, "UTF-8") + "\n");
                        response.close();
                } catch (IOException e){
                        System.err.println ("Encountered an I/O exception.");
                        e.printStackTrace();
                }


                /*
                 * Create a subuser
                 */
                requestArgs = new HashMap();
                requestArgs.put("format", "json");
                requestArgs.put("uid", "new-user");
                requestArgs.put("subuser", "foobar");

                response = adminApi.execute("PUT", "/admin/user", "subuser", requestArgs);
                System.out.println(response.getStatusLine());
                entity = response.getEntity();

                try {
                        System.out.println("\nResponse Content is: "
                                + EntityUtils.toString(entity, "UTF-8") + "\n");
                        response.close();
                } catch (IOException e){
                        System.err.println ("Encountered an I/O exception.");
                        e.printStackTrace();
                }


                /*
                 * Delete a user
                 */
                requestArgs = new HashMap();
                requestArgs.put("format", "json");
                requestArgs.put("uid", "new-user");

                response = adminApi.execute("DELETE", "/admin/user", null, requestArgs);
                System.out.println(response.getStatusLine());
                entity = response.getEntity();

                try {
                        System.out.println("\nResponse Content is: "
                                + EntityUtils.toString(entity, "UTF-8") + "\n");
                        response.close();
                } catch (IOException e){
                        System.err.println ("Encountered an I/O exception.");
                        e.printStackTrace();
                }
        }
}
```

Reference

- For a more extensive explanation of the Amazon S3 authentication procedure, consult the Signing and Authenticating REST Requests section of Amazon Simple Storage Service documentation.
- For more information, see S3 Authentication.

Creating an administrative user

Learn how to create an administrative API user.

> **Important:** To run the **`radosgw-admin`** command from the Ceph Object Gateway node, ensure the node has the admin key. The admin key can be copied from any Ceph Monitor node.

Prerequisites

- Root-level access to the Ceph Object Gateway node.

Procedure

1. Create an object gateway user:

   Syntax

   ```
   radosgw-admin user create --uid="USER_NAME" --display-name="DISPLAY_NAME"
   ```

   Example

   ```
   [user@client ~]$ radosgw-admin user create --uid="admin-api-user" --display-
   name="Admin API User"
   ```

   The `radosgw-admin` command-line interface will return the user.

   Example output

   ```
   {
       "user_id": "admin-api-user",
       "display_name": "Admin API User",
       "email": "",
       "suspended": 0,
       "max_buckets": 1000,
       "auid": 0,
       "subusers": [],
       "keys": [
           {
               "user": "admin-api-user",
               "access_key": "NRWGT19TWMYOB1YDBV1Y",
               "secret_key": "gr1VEGIV7rxcP3xvXDFCo4UDwwl2YoNrmtRlIAty"
           }
       ],
       "swift_keys": [],
       "caps": [],
       "op_mask": "read, write, delete",
       "default_placement": "",
       "placement_tags": [],
       "bucket_quota": {
           "enabled": false,
           "max_size_kb": -1,
           "max_objects": -1
       },
       "user_quota": {
           "enabled": false,
           "max_size_kb": -1,
           "max_objects": -1
       },
       "temp_url_keys": []
   }
   ```

2. Assign administrative capabilities to the user you create:

   Syntax

   ```
   radosgw-admin caps add --uid="USER_NAME" --caps="users=*"
   ```

   Example

   ```
   [user@client ~]$ radosgw-admin caps add --uid=admin-api-user --caps="users=*"
   ```

   The `radosgw-admin` command-line interface will return the user. The `"caps":` will have the capabilities you assigned to the user:

   Example output

   ```
   {
       "user_id": "admin-api-user",
       "display_name": "Admin API User",
       "email": "",
       "suspended": 0,
       "max_buckets": 1000,
       "auid": 0,
       "subusers": [],
   ```

```
    "keys": [
        {
            "user": "admin-api-user",
            "access_key": "NRWGT19TWMYOB1YDBV1Y",
            "secret_key": "gr1VEGIV7rxcP3xvXDFCo4UDwwl2YoNrmtRlIAty"
        }
    ],
    "swift_keys": [],
    "caps": [
        {
            "type": "users",
            "perm": "*"
        }
    ],
    "op_mask": "read, write, delete",
    "default_placement": "",
    "placement_tags": [],
    "bucket_quota": {
        "enabled": false,
        "max_size_kb": -1,
        "max_objects": -1
    },
    "user_quota": {
        "enabled": false,
        "max_size_kb": -1,
        "max_objects": -1
    },
    "temp_url_keys": []
}
```

Now you have a user with administrative privileges.

Get user information

Learn how to get user information.

Cap `users` or `user-info-without-keys` must be set to `read` to run this operation. If cap `user-info-without-keys` is set to `read` or `*`, S3 keys and Swift keys will not be included in the response unless the user running this operation is the system user, an admin user, or the cap `users` is set to `read`.

Capabilities

```
users=read or user-info-without-keys=read
```

Syntax

```
GET /admin/user?format=json HTTP/1.1
Host: FULLY_QUALIFIED_DOMAIN_NAME
```

Request Parameters

`uid`

Description

The user for which the information is requested.

Type

String

Example

`foo_user`

Required

Yes

`access-key`

Description

The S3 access key of the user for which the information is requested.

Type

String

Example

`ABCD0EF12GHIJ2K34LMN`

Required

No

Response Entities

`user`

Description

A container for the user data information.

Type

Container

Parent

N/A

`user_id`

Description

The user ID.

Type

String

Parent

`user`

`display_name`

Description

Display name for the user.

Type

String

Parent

`user`

`suspended`

Description

True if the user is suspended.

Type

Boolean

Parent

`user`

`max_buckets`

Description

The maximum number of buckets to be owned by the user.

Type

Integer

Parent

user

subusers

Description

Subusers associated with this user account.

Type

Container

Parent

user

keys

Description

S3 keys associated with this user account.

Type

Container

Parent

user

swift_keys

Description

Swift keys associated with this user account.

Type

Container

Parent

user

caps

Description

User capabilities.

Type

Container

Parent

user

If successful, the response contains the user information.

Special Error Responses

None.

Create a user

Create a new user. By default, an S3 key pair is automatically created and returned in the response.

If only a `access-key` or `secret-key` is provided, the omitted key will be automatically generated. By default, a generated key is added to the keyring without replacing an existing key pair. If `access-key` is specified and refers to an existing key owned by the user then it will be modified.

Capabilities

```
`users=write`
```

Syntax

```
PUT /admin/user?format=json HTTP/1.1
Host: FULLY_QUALIFIED_DOMAIN_NAME
```

Request Parameters

`uid`

Description

The user ID to be created.

Type

String

Example

`foo_user`

Required

Yes

`display-name`

Description

The display name of the user to be created.

Type

String

Example

`foo_user`

Required

Yes

`default-placement`

Description

Specify the default placement target for the user.

Type

String

Example

`default-placement`

Required

No

`default-storage-class`

Description

Specify the default storage class for the user.

Type

String

Example

`cold`

Required

No

`email`

Description

The email address associated with the user.

Type

String

Example

`foo@bar.com`

Required

No

`key-type`

Description

Key type to be generated, options are: swift, s3 (default).

Type

String

Example

`s3` [`S3`]

Required

No

`access-key`

Description

Specify access key.

Type

String

Example

`ABCD0EF12GHIJ2K34LMN`

Required

No

`secret-key`

Description

Specify secret key.

Type

String

Example

`0AbCDEFg1h2i34JklM5nop6QrSTUV+WxyzaBC7D8`

Required

No

`user-caps`

Description

User capabilities.

Type

String

Example

usage=read, write; users=read

Required

No

`generate-key`

Description

Generate a new key pair and add to the existing keyring.

Type

Boolean

Example

True [`True`]

Required

No

`max-buckets`

Description

Specify the maximum number of buckets the user can own.

Type

Integer

Example

500 [1000]

Required

No

`suspended`

Description

Specify whether the user should be suspended

Type

Boolean

Example

False [False]

Required

No

Example: Create a user with a default storage class and placement target.

```
$ curl --ipv4 --http1.1 --aws-sigv4 aws:amz:us-east-1:s3 \
-u 'ACCESS_KEY:SECRET_KEY' \
-H x-amz-content-sha256:UNSIGNED-PAYLOAD \
-X PUT \
"http://HOSTNAME/admin/user?default-placement=default-placement&default-storage-
class=cold&display-name=test4&uid=curl4"
```

> **Note:** Ensure that the specified storage class and placement target exist in the cluster before creating the user.

Response Entities

`user`

Description

Specify whether the user should be suspended

Type

Boolean

Parent

No

`user_id`

Description

The user ID.

Type

String

Parent

`user`

`display_name`

Description

Display name for the user.

Type

String

Parent

`user`

`suspended`

Description

True if the user is suspended.

Type

Boolean

Parent

`user`

`max_buckets`

Description

The maximum number of buckets to be owned by the user.

Type

Integer

Parent

`user`

`subusers`

Description

Subusers associated with this user account.

Type

Container

Parent

`user`

`keys`

Description

S3 keys associated with this user account.

Type

Container

Parent

`user`

`swift_keys`

Description

Swift keys associated with this user account.

Type

Container

Parent

`user`

`caps`

Description

User capabilities.

Type

Container

Parent

If successful, the response contains the user information.

Special Error Responses

`UserExists`

Description

Attempt to create existing user.

Code

409 Conflict

`InvalidAccessKey`

Description

Invalid access key specified.

Code

400 Bad Request

`InvalidKeyType`

Description

Invalid key type specified.

Code

400 Bad Request

`InvalidSecretKey`

Description

Invalid secret key specified.

Code

400 Bad Request

`KeyExists`

Description

Provided access key exists and belongs to another user.

Code

409 Conflict

`EmailExists`

Description

Provided email address exists.

Code

409 Conflict

`InvalidCap`

Description

Attempt to grant invalid admin capability.

Code

400 Bad Request

Reference

For more information about creating subusers, see “Create a subuser” on page 950.

Modify a user

Modify an existing user.

Capabilities

```
`users=write`
```

Syntax

```
POST /admin/user?format=json HTTP/1.1
Host: FULLY_QUALIFIED_DOMAIN_NAME
```

Request Parameters

`uid`

Description

The user ID to be created.

Type

String

Example

`foo_user`

Required

Yes

`display-name`

Description

The display name of the user to be created.

Type

String

Example

`foo_user`

Required

Yes

`email`

Description

The email address associated with the user.

Type

String

Example

`foo@bar.com`

Required

No

`generate-key`

Description

Generate a new key pair and add to the existing keyring.

Type

Boolean

Example

True [False]

Required

No

`access-key`

Description

Specify access key.

Type

String

Example

`ABCD0EF12GHIJ2K34LMN`

Required

No

`secret-key`

Description

Specify secret key.

Type

String

Example

`0AbCDEFg1h2i34JklM5nop6QrSTUV+WxyzaBC7D8`

Required

No

`key-type`

Description

Key type to be generated, options are: swift, s3 (default).

Type

String

Example

`s3`

Required

No

`user-caps`

Description

User capabilities.

Type

String

Example

`usage=read, write; users=read`

Required

No

`max-buckets`

Description

Specify the maximum number of buckets the user can own.

Type

Integer

Example

500 [1000]

Required

No

`suspended`

Description

Specify whether the user should be suspended

Type

Boolean

Example

False [False]

Required

No

Response Entities

`user`

Description

Specify whether the user should be suspended

Type

Boolean

Parent

No

`user_id`

Description

The user ID.

Type

String

Parent

`user`

`display_name`

Description

Display name for the user.

Type

String

Parent

`user`

`suspended`

Description

True if the user is suspended.

Type

Boolean

Parent

`user`

`max_buckets`

Description

The maximum number of buckets to be owned by the user.

Type

Integer

Parent

`user`

`subusers`

Description

Subusers associated with this user account.

Type

Container

Parent

`user`

`keys`

Description

S3 keys associated with this user account.

Type

Container

Parent

`user`

`swift_keys`

Description

Swift keys associated with this user account.

Type

Container

Parent

`user`

`caps`

Description

User capabilities.

Type

Container

Parent

If successful, the response contains the user information.

Special Error Responses

`InvalidAccessKey`

Description

Invalid access key specified.

Code

400 Bad Request

`InvalidKeyType`

Description

Invalid key type specified.

Code

400 Bad Request

`InvalidSecretKey`

Description

Invalid secret key specified.

Code

400 Bad Request

`KeyExists`

Description

Provided access key exists and belongs to another user.

Code

409 Conflict

`EmailExists`

Description

Provided email address exists.

Code

409 Conflict

`InvalidCap`

Description

Attempt to grant invalid admin capability.

Code

400 Bad Request

Reference

For more information, see “Modify a subuser” on page 1410.

Remove a user

Remove an existing user.

Capabilities

```
`users=write`
```

Syntax

```
DELETE /admin/user?format=json HTTP/1.1
Host: FULLY_QUALIFIED_DOMAIN_NAME
```

Request Parameters

`uid`

Description

The user ID to be removed.

Type

String

Example

`foo_user`

Required

Yes

`purge-data`

Description

When specified the buckets and objects belonging to the user will also be removed.

Type

Boolean

Example

True

Required

No

Response Entities

None.

Special Error Responses

None.

Reference

For more information, see Remove a subuser.

Create a subuser

Learn how to create a new subuser, primarily useful for clients using the Swift API.

> **Note:** Either `gen-subuser` or `subuser` is required for a valid request. In general, for a subuser to be useful, it must be granted permissions by specifying `access`. As with user creation if `subuser` is specified without `secret`, then a secret key is automatically generated.

Capabilities

```
`users=write`
```

Syntax

```
PUT /admin/user?subuser&format=json HTTP/1.1
Host FULLY_QUALIFIED_DOMAIN_NAME
```

Request Parameters

`uid`

Description

The user ID under which a subuser is to be created.

Type

String

Example

`foo_user`

Required

Yes

`subuser`

Description

Specify the subuser ID to be created.

Type

String

Example

`sub_foo`

Required

Yes (or `gen-subuser`)

`gen-subuser`

Description

Specify the subuser ID to be created.

Type

String

Example

`sub_foo`

Required

Yes (or `gen-subuser`)

`secret-key`

Description

Specify secret key.

Type

String

Example

`0AbCDEFg1h2i34JklM5nop6QrSTUV+WxyzaBC7D8`

Required

No

`key-type`

Description

Key type to be generated, options are: swift (default), s3.

Type

String

Example

`swift` [`swift`]

Required

No

`access`

Description

Set access permissions for sub-user, should be one of `read, write, readwrite, full`.

Type

String

Example

`read`

Required

No

`generate-secret`

Description

Generate the secret key.

Type

Boolean

Example

True [False]

Required

No

Response Entities

`subusers`

Description

Subusers associated with the user account.

Type

Container

Parent

N/A

`permissions`

Description

Subuser access to user account.

Type

String

Parent

`subusers`

If successful, the response contains the subuser information.

`SubuserExists`

Description

Specified subuser exists.

Code

409 Conflict

`InvalidKeyType`

Description

Invalid key type specified.

Code

400 Bad Request

`InvalidSecretKey`

Description

Invalid secret key specified.

Code

400 Bad Request

`InvalidAccess`

Description

Invalid subuser access specified

Code

400 Bad Request

Modify a subuser

Learn how to modify an existing subuser.

Capabilities

```
`users=write`
```

Syntax

```
POST /admin/user?subuser&format=json HTTP/1.1
Host FULLY_QUALIFIED_DOMAIN_NAME
```

Request Parameters

`uid`

Description

The user ID under which a subuser is to be created.

Type

String

Example

`foo_user`

Required

Yes

`subuser`

Description

The subuser ID to be modified.

Type

String

Example

`sub_foo`

Required

No

`generate-secret`

Description

Generate a new secret key for the subuser, replacing the existing key.

Type

Boolean

Example

True [False]

Required

No

`secret`

Description

Specify secret key.

Type

String

Example

`0AbCDEFg1h2i34JklM5nop6QrSTUV+WxyzaBC7D8`

Required

No

`key-type`

Description

Key type to be generated, options are: swift (default), s3.

Type

String

Example

`swift` [`swift`]

Required

No

`access`

Description

Set access permissions for sub-user, should be one of `read, write, readwrite, full`.

Type

String

Example

`read`

Required

No

Response Entities

`subusers`

Description

Subusers associated with the user account.

Type

Container

Parent

N/A

`id`

Description

Subuser ID

Type

String

Parent

`subusers`

`permissions`

Description

Subuser access to user account.

Type

String

Parent

`subusers`

If successful, the response contains the subuser information.

`InvalidKeyType`

Description

Invalid key type specified.

Code

400 Bad Request

`InvalidSecretKey`

Description

Invalid secret key specified.

Code

400 Bad Request

`InvalidAccess`

Description

Invalid subuser access specified

Code

400 Bad Request

Remove a subuser

Learn how to remove a subuser.

Capabilities

```
`users=write`
```

Syntax

```
DELETE /admin/user?subuser&format=json HTTP/1.1
Host FULLY_QUALIFIED_DOMAIN_NAME
```

Request Parameters

`uid`

Description

The user ID to be removed.

Type

String

Example

`foo_user`

Required

Yes

`subuser`

Description

The subuser ID to be removed.

Type

String

Example

sub_foo

Required

Yes

purge-keys

Description

Remove keys belonging to the subuser.

Type

Boolean

Example

True [True]

Required

No

Response Entities

None.

Special Error Responses

None.

Add capabilities to a user

Learn how to add an administrative capability to a specified user.

Capabilities

```
`users=write`
```

Syntax

```
PUT /admin/user?caps&format=json HTTP/1.1
Host FULLY_QUALIFIED_DOMAIN_NAME
```

uid

Description

The user ID to add an administrative capability to.

Type

String

Example

foo_user

Required

Yes

user-caps

Description

The administrative capability to add to the user.

Type

String

Example

`usage=read, write`

Required

Yes

`user`

Description

A container for the user data information.

Type

Container

Parent

N/A

`user_id`

Description

The user ID

Type

String

Parent

`user`

`caps`

Description

User capabilities

Type

Container

Parent

`user`

If successful, the response contains the user's capabilities.

Special Error Responses

`InvalidCap`

Description

Attempt to grant invalid admin capability.

Code

400 Bad Request

Remove capabilities from a user

Learn how to remove an administrative capability from a specified user

Capabilities

```
`users=write`
```

Syntax

```
DELETE /admin/user?caps&format=json HTTP/1.1
Host FULLY_QUALIFIED_DOMAIN_NAME
```

Request Parameters

`uid`

Description

The user ID to remove an administrative capability from.

Type

String

Example

`foo_user`

Required

Yes

`user-caps`

Description

The administrative capabilities to remove from the user.

Type

String

Example

`usage=read, write`

Required

Yes

Response Entities

`user`

Description

A container for the user data information.

Type

Container

Parent

N/A

`user_id`

Description

The user ID.

Type

String

Parent

`user`

`caps`

Description

User capabilities.

Type

Container

Parent

`user`

If successful, the response contains the user's capabilities.

`InvalidCap`

Description

Attempt to remove an invalid admin capability.

Code

400 Bad Request

`NoSuchCap`

Description

User does not possess specified capability.

Code

404 Not Found

Create a key

Learn how to create a new key.

If a `subuser` is specified then by default created keys will be swift type. If only one of `access-key` or `secret-key` is provided the committed key will be automatically generated, that is if only `secret-key` is specified then `access-key` will be automatically generated. By default, a generated key is added to the keyring without replacing an existing key pair. If `access-key` is specified and refers to an existing key owned by the user then it will be modified. The response is a container listing all keys of the same type as the key created.

> **Note:** When creating a swift key, specifying the option `access-key` will have no effect. Additionally, only one swift key might be held by each user or subuser.

Capabilities

```
`users=write`
```

Syntax

```
PUT /admin/user?key&format=json HTTP/1.1
Host FULLY_QUALIFIED_DOMAIN_NAME
```

Request Parameters

`uid`

Description

The user ID to receive the new key.

Type

String

Example

`foo_user`

Required

Yes

`subuser`

Description

The subuser ID to receive the new key.

Type

String

Example

`sub_foo`

Required

No

`key-type`

Description

Key type to be generated, options are: swift, s3 (default).

Type

String

Example

`s3` [`s3`]

Required

No

`access-key`

Description

Specify access key.

Type

String

Example

`AB01C2D3EF45G6H7IJ8K`

Required

No

`secret-key`

Description

Specify secret key.

Type

String

Example

`0ab/CdeFGhij1klmnopqRSTUv1WxyZabcDEFgHij`

Required

No

`generate-key`

Description

Generate a new key pair and add to the existing keyring.

Type

Boolean

Example

True [`True`]

Required

No

Response Entities

`keys`

Description

Keys of type created associated with this user account.

Type

Container

Parent

N/A

`user`

Description

The user account associated with the key.

Type

String

Parent

`keys`

`access-key`

Description

The access key.

Type

String

Parent

`keys`

`secret-key`

Description

The secret key.

Type

String

Parent

`keys`

Special Error Responses

`InvalidAccessKey`

Description

Invalid access key specified.

Code

400 Bad Request

`InvalidKeyType`

Description

Invalid key type specified.

Code

400 Bad Request

`InvalidSecretKey`

Description

Invalid secret key specified.

Code

400 Bad Request

`InvalidKeyType`

Description

Invalid key type specified.

Code

400 Bad Request

`KeyExists`

Description

Provided access key exists and belongs to another user.

Code

409 Conflict

Remove a key

Learn how to remove a key.

Capabilities

```
`users=write`
```

Syntax

```
DELETE /admin/user?key&format=json HTTP/1.1
Host FULLY_QUALIFIED_DOMAIN_NAME
```

Request Parameters

`access-key`

Description

The S3 access key belonging to the S3 key pair to remove.

Type

String

Example

`AB01C2D3EF45G6H7IJ8K`

Required

Yes

`uid`

Description

The user to remove the key from.

Type

String

Example

`foo_user`

Required

No

`subuser`

Description

The subuser to remove the key from.

Type

String

Example

`sub_foo`

Required

No

`key-type`

Description

Key type to be removed, options are: swift, s3.

> **Note:** Required to remove swift key.

Type

String

Example

`swift`

Required

No

Special Error Responses

None.

Response Entities

None.

Bucket notifications

Bucket notification APIs can be used to provide configuration and control interfaces for the bucket notification mechanism.

Bucket notifications associate topics with a specific bucket. For more information about bucket notifications, see “S3 bucket operations” on page 1464. Before using bucket notification APIs, create bucket notifications on the Ceph Object Gateway.

> **Note:**
> - In all topic actions, the parameters are URL encoded, and sent in the message body using `application/x-www-form-urlencoded` content type.
> - Any bucket notification already associated with the topic needs to be re-created for the topic update to take effect.

Bucket notifications provide a way to send information out of the Ceph Object Gateway when certain events happen in the bucket. Bucket notifications can be sent to HTTP, AMQP0.9.1, and Kafka endpoints. A notification entry must be created to send bucket notifications for events on a specific bucket and to a specific topic. A bucket notification can be created on a subset of event types or by default for all event types. The bucket notification can filter out events based on key prefix or suffix, regular expression matching the keys, and the

metadata attributes attached to the object, or the object tags. Bucket notifications have a REST API to provide configuration and control interfaces for the bucket notification mechanism.

Sending a bucket notification when an object is synced to a zone lets the external system get information into the zone syncing status at the object level. The bucket notification event types `s3:ObjectSynced:*` and `s3:ObjectSynced:Created`, when configured via the bucket notification mechanism, send a notification event from the synced RGW upon successful sync of an object. Both the topics and the notification configuration should be done separately in each zone from which the notification events are being sent.

Persistent notifications

Persistent notifications enable reliable and asynchronous delivery of notifications from the Ceph Object Gateway to the endpoint configured at the topic.

Regular notifications are also reliable because the delivery to the endpoint is performed synchronously during the request. With persistent notifications, the Ceph Object Gateway retries sending notifications even when the endpoint is down or there are network issues during the operations, that is notifications are retried if not successfully delivered to the endpoint. Notifications are sent only after all other actions related to the notified operation are successful. If an endpoint goes down for a longer duration, the notification queue fills up and the S3 operations that have configured notifications for these endpoints will fail.

> **Note:** With `kafka-ack-level=none`, there is no indication for message failures, and therefore messages sent while broker is down are not retried, when the broker is up again. After the broker is up again, only new notifications are seen.

Creating a topic

Create a topic before creating bucket notifications.

A topic is a Simple Notification Service (SNS) entity and all the topic operations, that is, `create`, `delete`, `list`, and `get`, are SNS operations. The topic needs to have endpoint parameters that are used when a bucket notification is created. Once the request is successful, the response includes the topic Amazon Resource Name (ARN) that can be used later to reference this topic in the bucket notification request.

> **Note:** A `topic_arn` provides the bucket notification configuration and is generated after a topic is created.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access.
- Installation of the Ceph Object Gateway.
- User access key and secret key.
- Endpoint parameters.

1. Create a topic with the following request format:

   Syntax

```
POST
Action=CreateTopic
&Name=TOPIC_NAME
[&Attributes.entry.1.key=amqp-exchange&Attributes.entry.1.value=EXCHANGE]
[&Attributes.entry.2.key=amqp-ack-level&Attributes.entry.2.value=none|broker|routable]
[&Attributes.entry.3.key=verify-ssl&Attributes.entry.3.value=true|false]
[&Attributes.entry.4.key=kafka-ack-level&Attributes.entry.4.value=none|broker]
[&Attributes.entry.5.key=use-ssl&Attributes.entry.5.value=true|false]
[&Attributes.entry.6.key=ca-location&Attributes.entry.6.value=FILE_PATH]
[&Attributes.entry.7.key=OpaqueData&Attributes.entry.7.value=OPAQUE_DATA]
[&Attributes.entry.8.key=push-endpoint&Attributes.entry.8.value=ENDPOINT]
[&Attributes.entry.9.key=persistent&Attributes.entry.9.value=true|false]
```

Here are the request parameters:

- `Endpoint`: URL of an endpoint to send notifications to.
- `OpaqueData`: opaque data is set in the topic configuration and added to all notifications triggered by the topic.
- `persistent`: indication of whether notifications to this endpoint are persistent that is asynchronous or not. By default the value is `false`.
- HTTP endpoint:
  - `URL`: https://*FQDN:PORT*
  - `port defaults to`: Use 80/443 for HTTP[S] accordingly.
  - `verify-ssl`: Indicates whether the server certificate is validated by the client or not. By default , it is `true`.
- AMQP0.9.1 endpoint:
  - `URL`: amqp://*USER:PASSWORD@FQDN:PORT[/VHOST]*.
  - User and password defaults to: `guest` and `guest` respectively.
  - User and password details should be provided over HTTPS, otherwise the topic creation request is rejected.
  - `port defaults to`: 5672.
  - `vhost` defaults to: “/”
  - `amqp-exchange`: The exchanges must exist and be able to route messages based on topics. This is a mandatory parameter for AMQP0.9.1. Different topics pointing to the same endpoint must use the same exchange.
  - `amqp-ack-level`: No end to end acknowledgment is required, as messages may persist in the broker before being delivered into their final destination. Three acknowledgment methods exist:
    - `none`: Message is considered `delivered` if sent to the broker.
    - `broker`: By default, the message is considered `delivered` if acknowledged by the broker.
    - `routable`: Message is considered `delivered` if the broker can route to a consumer.

    Note:
    - The key and value of a specific parameter do not have to reside in the same line, or in any specific order, but must use the same index. Attribute indexing does not need to be sequential or start from any specific value.
    - The `topic-name` is used for the AMQP topic.
- Kafka endpoint:
  - `URL`: kafka:*USER:PASSWORD@FQDN:PORT*.
  - `use-ssl` is set to `false` by default. If `use-ssl` is set to `true`, secure connection is used for connecting with the broker.
  - If `ca-location` is provided, and secure connection is used, the specified CA will be used, instead of the default one, to authenticate the broker.
  - User and password can only be provided over HTTP[S]. Otherwise, the topic creation request is rejected.
  - User and password may only be provided together with `use-ssl`, otherwise, the connection to the broker will fail.
  - `port defaults to`: 9092.

- `kafka-ack-level`: no end to end acknowledgment required, as messages may persist in the broker before being delivered into their final destination. Two acknowledgment methods exist:
  - `none`: message is considered `delivered` if sent to the broker.
  - `broker`: By default, the message is considered `delivered` if acknowledged by the broker.

The following is an example of the response format:

Example

```
<CreateTopicResponse xmlns="https://sns.amazonaws.com/doc/2010-03-31/">
    <CreateTopicResult>
        <TopicArn></TopicArn>
    </CreateTopicResult>
    <ResponseMetadata>
        <RequestId></RequestId>
    </ResponseMetadata>
</CreateTopicResponse>
```

Note: The topic Amazon Resource Name (ARN) in the response will have the following format: `arn:aws:sns:ZONE_GROUP:TENANT:TOPIC`

The following is an example of AMQP0.9.1 endpoint:

Example

```
client.create_topic(Name='my-topic' , Attributes={'push-endpoint': 'amqp://127.0.0.1:5672',
'amqp-exchange': 'ex1', 'amqp-ack-level': 'broker'}) "
```

Getting topic information

Getting topic information provides information about a specific topic, which can include endpoint information if it is provided.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access.
- Installation of the Ceph Object Gateway.
- User access key and secret key.
- Endpoint parameters.

Procedure

1. Get topic information with the following request format:

   Syntax

   ```
   POST
   Action=GetTopic
   &TopicArn=TOPIC_ARN
   ```

   Here is an example of the response format:

   ```
   <GetTopicResponse>
   <GetTopicRersult>
   <Topic>
   <User></User>
   <Name></Name>
   <EndPoint>
   <EndpointAddress></EndpointAddress>
   ```

```
<EndpointArgs></EndpointArgs>
<EndpointTopic></EndpointTopic>
<HasStoredSecret></HasStoredSecret>
<Persistent></Persistent>
</EndPoint>
<TopicArn></TopicArn>
<OpaqueData></OpaqueData>
</Topic>
</GetTopicResult>
<ResponseMetadata>
<RequestId></RequestId>
</ResponseMetadata>
</GetTopicResponse>
```

The following are the tags and definitions:

- `User`: Name of the user that created the topic.
- `Name`: Name of the topic.
- JSON formatted endpoints include:
  - `EndpointAddress`: The endpoint URL. If the endpoint URL contains user and password information, the request must be made over HTTPS. Otheriwse, the topic get request is rejected.
    - `EndPointArgs`: The endpoint arguments.
    - `EndpointTopic`: The topic name that is be sent to the endpoint can be different than the example topic name.
    - `HasStoredSecret`: `true` when the endpoint URL contains user and password information.
    - `Persistent`: `true` when the topic is persistent.
- `TopicArn`: Topic ARN.
- `OpaqueData`: This is an opaque data set on the topic.

Listing topics

Learn how to list topics that a user has defined.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access.
- Installation of the Ceph Object Gateway.
- User access key and secret key.
- Endpoint parameters.

Procedure

1. List topic information with the following request format:

Syntax

```
POST
Action=ListTopics
```

Here is an example of the response format:

```
<ListTopicdResponse xmlns="https://sns.amazonaws.com/doc/2020-03-31/">
<ListTopicsRersult>
<Topics>
<member>
<User></User>
```

```
<Name></Name>
<EndPoint>
<EndpointAddress></EndpointAddress>
<EndpointArgs></EndpointArgs>
<EndpointTopic></EndpointTopic>
</EndPoint>
<TopicArn></TopicArn>
<OpaqueData></OpaqueData>
</member>
</Topics>
</ListTopicsResult>
<ResponseMetadata>
<RequestId></RequestId>
</ResponseMetadata>
</ListTopicsResponse>
```

Note: If endpoint URL contains user and password information, in any of the topics, the request must be made over HTTPS. Otherwise, the topic list request is rejected.

Deleting topics

Learn how to delete a topic.

Note: Removing a deleted topic results in no operation and is not a failure

.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access.
- Installation of the Ceph Object Gateway.
- User access key and secret key.
- Endpoint parameters.

Procedure

- Delete a topic with the following request format:

  Syntax

  ```
  POST
  Action=DeleteTopic
  &TopicArn=TOPIC_ARN
  ```

  Here is an example of the response format:

  ```
  <DeleteTopicResponse xmlns="https://sns.amazonaws.com/doc/2020-03-31/">
  <ResponseMetadata>
  <RequestId></RequestId>
  </ResponseMetadata>
  </DeleteTopicResponse>
  ```

Using the command-line interface for topic management

List, get, and remove topics by using the command-line interface.

Prerequisites

- Root-level access to the Ceph Object Gateway node.

1. To get a list of all topics of a user:

   Syntax

   ```
   radosgw-admin topic list --uid=USER_ID
   ```

   Example

   ```
   [root@rgw ~]# radosgw-admin topic list --uid=example
   ```

2. To get configuration of a specific topic:

   Syntax

   ```
   radosgw-admin topic get --uid=USER_ID --topic=TOPIC_NAME
   ```

   Example

   ```
   [root@rgw ~]# radosgw-admin topic get --uid=example --topic=example-topic
   ```

3. To remove a specific topic:

   Syntax

   ```
   radosgw-admin topic rm --uid=USER_ID --topic=TOPIC_NAME
   ```

   Example

   ```
   [root@rgw ~]# radosgw-admin topic rm --uid=example --topic=example-topic
   ```

Managing notification configuration

Manage notification configuration of buckets with the command-line interface.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A Red Hat Ceph Storage cluster.
- A Ceph Object Gateway installed.

About this task

You can list, get, and remove a bucket notification configuration with the **`radosgw-admin`** command.

Procedure

- List all the bucket notification configuration.

  ```
  radosgw-admin notification list --bucket=BUCKET_NAME
  ```

  For example,

```
[root@host04 ~]# radosgw-admin notification list --bucket bkt2
{
    "notifications": [
        {
            "TopicArn": "arn:aws:sns:default::topic1",
            "Id": "notif1",
            "Events": [
                "s3:ObjectCreated:*",
                "s3:ObjectRemoved:*"
            ],
            "Filter": {
                "S3Key": {},
                "S3Metadata": {},
                "S3Tags": {}
            }
        },
        {
            "TopicArn": "arn:aws:sns:default::topic1",
            "Id": "notif2",
            "Events": [
                "s3:ObjectSynced:*"
            ],
            "Filter": {
                "S3Key": {},
                "S3Metadata": {},
                "S3Tags": {}
            }
        }
    ]
}
```

– Get the bucket notification configuration.

```
radosgw-admin notification get --bucket BUCKET_NAME --notification-id NOTIFICATION_ID
```

For example,

```
[root@host04 ~]# radosgw-admin notification get --bucket bkt2 --notification-id notif2
{
    "TopicArn": "arn:aws:sns:default::topic1",
    "Id": "notif2",
    "Events": [
        "s3:ObjectSynced:*"
    ],
    "Filter": {
        "S3Key": {},
        "S3Metadata": {},
        "S3Tags": {}
    }
}
```

– Remove a specific bucket notification configuration.

```
radosgw-admin notification rm --bucket BUCKET_NAME [--notification-id NOTIFICATION_ID]
```

Here, *NOTIFICATION_ID* is optional. If it is not specified, the command removes all the notification configurations of that bucket.
For example,

```
[root@host04 ~]# radosgw-admin notification rm --bucket bkt2 --notification-id notif1
```

Event record

An event holds information about the operation done by the Ceph Object Gateway and is sent as a payload over the chosen endpoint, such as HTTP, HTTPS, Kafka, or AMQ0.9.1. The event record is in JSON format.

The following `ObjectLifecycle:Expiration` events are supported:

– `ObjectLifecycle:Expiration:Current`

– `ObjectLifecycle:Expiration:NonCurrent`

– `ObjectLifecycle:Expiration:DeleteMarker`

- `ObjectLifecycle:Expiration:AbortMultipartUpload`

Example

```
{"Records":[
    {
        "eventVersion":"2.1",
        "eventSource":"ceph:s3",
        "awsRegion":"us-east-1",
        "eventTime":"2019-11-22T13:47:35.124724Z",
        "eventName":"ObjectCreated:Put",
        "userIdentity":{
            "principalId":"tester"
        },
        "requestParameters":{
            "sourceIPAddress":""
        },
        "responseElements":{
            "x-amz-request-id":"503a4c37-85eb-47cd-8681-2817e80b4281.5330.903595",
            "x-amz-id-2":"14d2-zone1-zonegroup1"
        },
        "s3":{
            "s3SchemaVersion":"1.0",
            "configurationId":"mynotif1",
            "bucket":{
                "name":"mybucket1",
                "ownerIdentity":{
                    "principalId":"tester"
                },
                "arn":"arn:aws:s3:us-east-1::mybucket1",
                "id":"503a4c37-85eb-47cd-8681-2817e80b4281.5332.38"
            },
            "object":{
                "key":"myimage1.jpg",
                "size":"1024",
                "eTag":"37b51d194a7513e45b56f6524f2d51f2",
                "versionId":"",
                "sequencer": "F7E6D75DC742D108",
                "metadata":[],
                "tags":[]
            }
        },
        "eventId":"",
        "opaqueData":"me@example.com"
    }
]}
```

These are the event record keys and their definitions:

`awsRegion`

Zonegroup.

`eventTime`

Timestamp that indicates when the event was triggered.

`eventName`

The type of the event. It can be `ObjectCreated`, `ObjectRemovedObjectLifecycle:Expiration`

`userIdentity.principalId`

The identity of the user that triggered the event.

`requestParameters.sourceIPAddress`

The IP address of the client that triggered the event. This field is not supported.

`responseElements.x-amz-request-id`

The request ID that triggered the event.

`responseElements.x_amzID2`

The identity of the Ceph Object Gateway on which the event was triggered. The identity format is RGWID*-ZONE-ZONEGROUP*.

`s3.configurationId`

The notification ID that created the event.

s3.bucket.name

The name of the bucket.

s3.bucket.ownerIdentity.principalId

The owner of the bucket.

s3.bucket.arn

Amazon Resource Name (ARN) of the bucket.

s3.bucket.id

Identity of the bucket.

s3.object.key

The object key.

s3.object.size

The size of the object.

s3.object.eTag

The object etag.

s3.object.version

The object version in a versioned bucket.

s3.object.sequencer

Monotonically increasing identifier of the change per object in the hexadecimal format.

s3.object.metadata

Any metadata set on the object sent as `x-amz-meta`.

s3.object.tags

Any tags set on the object.

s3.eventId

Unique identity of the event.

s3.opaqueData

Opaque data is set in the topic configuration and added to all notifications triggered by the topic.

Reference

- See the Event Message Structure for more information.

Supported event types

Review the S3 event types that Ceph Object Gateway supports for bucket notifications.

The following event types are supported:

- `s3:ObjectCreated:*`
- `s3:ObjectCreated:Put`
- `s3:ObjectCreated:Post`
- `s3:ObjectCreated:Copy`
- `s3:ObjectCreated:CompleteMultipartUpload`

> **Note:** In multipart upload, an ObjectCreated:CompleteMultipartUpload notification is sent at the end of the process.

- `s3:ObjectRemoved:*`
- `s3:ObjectRemoved:Delete`
- `s3:ObjectRemoved:DeleteMarkerCreated`
- `s3:ObjectLifecycle:Expiration:Current`
- `s3:ObjectLifecycle:Expiration:NonCurrent`
- `s3:ObjectLifecycle:Expiration:DeleteMarker`
- `s3:ObjectLifecycle:Expiration:AbortMultipartUpload`
- `s3:ObjectLifecycle:Transition:Current`
- `s3:ObjectLifecycle:Transition:NonCurrent`
- `s3:ObjectSynced:Create`

Get bucket information

Get information about a subset of the existing buckets.

If `uid` is specified without `bucket` then all buckets belonging to the user will be returned. If `bucket` alone is specified, information for that particular bucket will be retrieved.

Capabilities

```
`buckets=read`
```

Syntax

```
GET /admin/bucket?format=json HTTP/1.1
Host FULLY_QUALIFIED_DOMAIN_NAME
```

Request Parameters

`bucket`

Description

The bucket to return info on.

Type

String

Example

`foo_bucket`

Required

No

`uid`

Description

The user to retrieve bucket information for.

Type

String

Example

`foo_user`

Required

No

`stats`

Description

Return bucket statistics.

Type

Boolean

Example

True [False]

Required

No

Response Entities

`stats`

Description

Per bucket information.

Type

Container

Parent

N/A

`buckets`

Description

Contains a list of one or more bucket containers.

Type

Container

Parent

`buckets`

`bucket`

Description

Container for single bucket information.

Type

Container

Parent

`buckets`

`name`

Description

The name of the bucket.

Type

String

Parent

`bucket`

`pool`

Description

The pool the bucket is stored in.

Type

String

Parent

`bucket`

`id`

Description

The unique bucket ID.

Type

String

Parent

`bucket`

`marker`

Description

Internal bucket tag.

Type

String

Parent

`bucket`

`owner`

Description

The user ID of the bucket owner.

Type

String

Parent

`bucket`

`usage`

Description

Storage usage information.

Type

Container

Parent

`bucket`

`index`

Description

Status of bucket index.

Type

String

Parent

`bucket`

If successful, then the request returns a bucket's container with the bucket information.

Special Error Responses

`IndexRepairFailed`

Description

Bucket index repair failed.

Code

409 Conflict

Check a bucket index

Check the index of an existing bucket.

> **Note:** To check multipart object accounting with `check-objects`, `fix` must be set to True.

Capabilities

`buckets=write`

Syntax

```
GET /admin/bucket?index&format=json HTTP/1.1
Host FULLY_QUALIFIED_DOMAIN_NAME
```

Request Parameters

`bucket`

Description

The bucket to return info on.

Type

String

Example

`foo_bucket`

Required

Yes

`check-objects`

Description

Check multipart object accounting.

Type

Boolean

Example

True [False]

Required

No

`fix`

Description

Also fix the bucket index when checking.

Type

Boolean

Example

False [False]

Required

No

Response Entities

`index`

Description

Status of bucket index.

Type

String

Special Error Responses

`IndexRepairFailed`

Description

Bucket index repair failed.

Code

409 Conflict

Remove a bucket

Learn how to remove an existing bucket.

Capabilities

```
`buckets=write`
```

Syntax

```
DELETE /admin/bucket?format=json HTTP/1.1
Host FULLY_QUALIFIED_DOMAIN_NAME
```

Request Parameters

`bucket`

Description

The bucket to remove.

Type

String

Example

`foo_bucket`

Required

Yes

`purge-objects`

Description

Type

Boolean

Example

True [False]

Required

No

Response Entities

None.

Special Error Responses

`BucketNotEmpty`

Description

Attempted to delete non-empty bucket.

Code

409 Conflict

`ObjectRemovalFailed`

Description

Unable to remove objects.

Code

409 Conflict

Link a bucket

Link a bucket to a specified user, unlinking the bucket from any previous user.

Capabilities

```
`buckets=write`
```

Syntax

```
PUT /admin/bucket?format=json HTTP/1.1
Host FULLY_QUALIFIED_DOMAIN_NAME
```

Request Parameters

`bucket`

Description

The bucket to unlink.

Type

String

Example

`foo_bucket`

Required

Yes

`uid`

Description

The user ID to link the bucket to.

Type

String

Example

`foo_user`

Required

Yes

Response Entities

`bucket`

Description

Container for single bucket information.

Type

Container

Parent

N/A

`name`

Description

The name of the bucket.

Type

String

Parent

`bucket`

`pool`

Description

The pool the bucket is stored in.

Type

String

Parent

`bucket`

`id`

Description

The unique bucket ID.

Type

String

Parent

`bucket`

`marker`

Description

Internal bucket tag.

Type

String

Parent

`bucket`

`owner`

Description

The user ID of the bucket owner.

Type

String

Parent

`bucket`

`usage`

Description

Storage usage information.

Type

Container

Parent

`bucket`

`index`

Description

Status of bucket index.

Type

String

Parent

`bucket`

Special Error Responses

`BucketUnlinkFailed`

Description

Unable to unlink bucket from specified user.

Code

409 Conflict

`BucketLinkFailed`

Description

Unable to link bucket to specified user.

Code

409 Conflict

Unlink a bucket

Unlink a bucket from a specified user. Unlinking is primarily useful for changing bucket ownership.

Capabilities

```
`buckets=write`
```

Syntax

```
POST /admin/bucket?format=json HTTP/1.1
Host FULLY_QUALIFIED_DOMAIN_NAME
```

`bucket`

Description

The bucket to unlink.

Type

String

Example

`foo_bucket`

Required

Yes

`uid`

Description

The user ID to link the bucket to.

Type

String

Example

`foo_user`

Required

Yes

Response Entities None.

`BucketUnlinkFailed`

Description

Unable to unlink bucket from specified user.

Type

409 Conflict

Get a bucket or object policy

Get the policy of a bucket or object.

Capabilities

```
`buckets=read`
```

Syntax

```
GET /admin/bucket?policy&format=json HTTP/1.1
Host FULLY_QUALIFIED_DOMAIN_NAME
```

Request Parameters

`bucket`

Description

The bucket to read the policy from.

Type

String

Example

`foo_bucket`

Required

Yes

`object`

Description

The object to read the policy from.

Type

String

Example

`foo.txt`

Required

No

Response Entities

`policy`

Description

Access control policy.

Type

Container

Parent

N/A

If successful, returns the object or bucket policy.

Special Error Responses

`IncompleteBody`

Description

Either bucket was not specified for a bucket policy request or bucket and object were not specified for an object policy request.

Code

400 Bad Request

Remove an object

Learn how to remove an existing object.

> **Note:** Removing an object does not require owner to be non-suspended.

Capabilities

```
`buckets=write`
```

Syntax

```
DELETE /admin/bucket?object&format=json HTTP/1.1
Host FULLY_QUALIFIED_DOMAIN_NAME
```

Request Parameters

`bucket`

Description

The bucket containing the object to be removed.

Type

String

Example

`foo_bucket`

Required

Yes

`object`

Description

The object to remove

Type

String

Example

`foo.txt`

Required

Yes

Response Entities

None.

Special Error Responses

`NoSuchObject`

Description

Specified object does not exist.

Code

404 Not Found

`ObjectRemovalFailed`

Description

Unable to remove objects.

Code

409 Conflict

Ceph Object Gateway quota operation

Ceph Object Gateway administrative API quota operations let you view, set, modify, and disable quotas on users and on buckets that users own.

Required capabilities

Quotas include the maximum number of objects in a bucket and the maximum storage size. To view quotas, you must have the `users=read` capability. To set, modify, or disable a quota, you must have the `users=write` capability.

Quota parameters

The following parameters are valid for quotas:

`bucket`

Specifies a quota for buckets that a user owns.

`max-objects`

Specifies the maximum number of objects. A negative value disables this setting.

max-size

Specifies the maximum number of bytes. A negative value disables this setting.

max-size-kb

Specifies the maximum size in KiB. A negative value disables this setting.

quota-type

Sets the scope of the quota. The options are `bucket` and `user`.

enabled

Specifies whether the quota is enabled. The value must be `True` or `False`.

Get a user quota

To get a user quota, you must have the `users` capability with `read` permission.

```
GET /admin/user?quota&uid=UID&quota-type=user
```

Replace *`UID`* with the ID of the user.

Set a user quota

To set a user quota, you must have the `users` capability with `write` permission.

```
PUT /admin/user?quota&uid=UID&quota-type=user
```

Replace *`UID`* with the ID of the user.

The request body must include a JSON representation of the quota settings, in the same format that the corresponding get operation returns.

Get a bucket quota

To get a bucket quota, you must have the `users` capability with `read` permission.

```
GET /admin/user?quota&uid=UID&quota-type=bucket
```

Replace *`UID`* with the ID of the user who owns the buckets.

To retrieve other information about buckets, such as the owner, pool, and usage, see “Get bucket information” on page 1431.

Set a bucket quota

To set a bucket quota, you must have the `users` capability with `write` permission.

```
PUT /admin/user?quota&uid=UID&quota-type=bucket
```

Replace *`UID`* with the ID of the user who owns the buckets.

The request body must include a JSON representation of the quota settings, in the same format that the corresponding get operation returns.

Set a quota for an individual bucket

To set a quota for an individual bucket, you must have the `buckets` capability with `write` permission.

```
PUT /admin/bucket?quota&uid=UID&bucket=BUCKET_NAME
```

Replace *`UID`* with the ID of the user who owns the bucket. Replace *`BUCKET_NAME`* with the name of the bucket.

The request body must include a JSON representation of the quota settings, in the same format as for setting a bucket quota.

Get usage information

Learn how to get bandwidth usage information.

Capabilities

```
`usage=read`
```

Syntax

```
GET /admin/usage?format=json HTTP/1.1
Host: FULLY_QUALIFIED_DOMAIN_NAME
```

Request Parameters

`uid`

Description

The user for which the information is requested.

Type

String

Required

Yes

`start`

Description

The date, and optionally, the time of when the data request started. For example, `2012-09-25 16:00:00`.

Type

String

Required

No

`end`

Description

The date, and optionally, the time of when the data request ended. For example, `2012-09-25 16:00:00`.

Type

String

Required

No

`show-entries`

Description

Specifies whether data entries should be returned.

Type

Boolean

Required

No

`show-summary`

Description

Specifies whether data entries should be returned.

Type

Boolean

Required

No

Response Entities

`usage`

Description

A container for the usage information.

Type

Container

`entries`

Description

A container for the usage entries information.

Type

Container

`user`

Description

A container for the user data information.

Type

Container

`owner`

Description

The name of the user that owns the buckets.

Type

String

`bucket`

Description

The bucket name.

Type

String

`time`

Description

Time lower bound for which data is being specified that is rounded to the beginning of the first relevant hour.

Type

String

`epoch`

Description

The time specified in seconds since `1/1/1970`.

Type

String

`categories`

Description

A container for stats categories.

Type

Container

`entry`

Description

A container for stats entry.

Type

Container

`category`

Description

Name of request category for which the stats are provided.

Type

String

`bytes_sent`

Description

Number of bytes sent by the Ceph Object Gateway.

Type

Integer

`bytes_received`

Description

Number of bytes received by the Ceph Object Gateway.

Type

Integer

`ops`

Description

Number of operations.

Type

Integer

`successful_ops`

Description

Number of successful operations.

Type

Integer

`summary`

Description

Number of successful operations.

Type

Container

`total`

Description

A container for stats summary aggregated total.

Type

Container

If successful, the response contains the requested information.

Remove usage information

Learn how to remove usage information.

> **Important:** If no dates are specified, all usage information is removed.

Capabilities

```
`usage=write`
```

Syntax

```
DELETE /admin/usage?format=json HTTP/1.1
Host: FULLY_QUALIFIED_DOMAIN_NAME
```

Request Parameters

`uid`

Description

The user for which the information is requested.

Type

String

Example

`foo_user`

Required

Yes

`start`

Description

The date, and optionally, the time of when the data request started.

Type

String

Example

`2012-09-25 16:00:00`

Required

No

`end`

Description

The date, and optionally, the time of when the data request ended.

Type

String

Example

`2012-09-25 16:00:00`

Required

No

`remove-all`

Description

Required when `uid` is not specified, in order to acknowledge multi-user data removal.

Type

Boolean

Example

True [False]

Required

No

Standard error responses

Get to know the standard error responses and their descriptions.

`AccessDenied`

Description

Access denied.

Code

403 Forbidden

`InternalError`

Description

Internal server error.

Code

500 Internal Server Error

`NoSuchUser`

Description

User does not exist.

Code

404 Not Found

`NoSuchBucket`

Description

Bucket does not exist.

Code

404 Not Found

`NoSuchKey`

Description

No such access key.

Code

404 Not Found

Ceph Object Gateway and the S3 API

As a developer, you can use a RESTful application programming interface (API) that is compatible with the Amazon S3 data access model. You can manage the buckets and objects stored in an Red Hat Ceph Storage cluster through the Ceph Object Gateway.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- A RESTful client.

S3 limitations

Understand Amazon S3 limitations.

> **Important:** These limitations should be used with caution as there are implications that are related to your hardware selections. Always discuss these requirements with your Red Hat account team.

Maximum object size when using Amazon S3

Individual Amazon S3 objects can range in size from a minimum of 0 B to a maximum of 5 TB. The largest object that can be uploaded in a single `PUT` is 5 GB. For objects larger than 100 MB, consider using the Multipart Upload capability.

Maximum metadata size when using Amazon S3

There is no defined limit on the total size of user metadata that can be applied to an object, but a single HTTP request is limited to 16,000 bytes.

The amount of data overhead Red Hat Ceph Storage cluster produces to store S3 objects and metadata

The estimate is 200-300 bytes plus the length of the object name. Versioned objects use additional space proportional to the number of versions. Also, transient overhead is produced during multi-part upload and other transactional updates, but these overheads are recovered during garbage collection.

For more information, see “S3 unsupported header fields” on page 1575.

Accessing the Ceph Object Gateway with the S3 API

As a developer, you must configure access to the Ceph Object Gateway and the Secure Token Service (STS) before you can start using the Amazon S3 API.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- A running Ceph Object Gateway.
- A RESTful client.

S3 authentication

Requests to the Ceph Object Gateway can be either authenticated or unauthenticated. Ceph Object Gateway assumes that unauthenticated requests are sent by an anonymous user. Ceph Object Gateway supports canned ACLs.

For most use cases, clients use existing open source libraries like the Amazon SDK’s `AmazonS3Client` for Java, and Python Boto. With open source libraries, you pass in the access key and secret key and the library builds the request header and authentication signature for you. However, you can create requests and sign them too.

Authenticating a request requires including an access key and a base 64-encoded Hash-based Message Authentication Code (HMAC) in the request before it is sent to the Ceph Object Gateway server. Ceph Object Gateway uses an S3-compatible authentication approach.

Example

```
HTTP/1.1
PUT /buckets/bucket/object.mpeg
Host: cname.domain.com
Date: Mon, 2 Jan 2012 00:01:01 +0000
Content-Encoding: mpeg
Content-Length: 9999999

Authorization: AWS _ACCESS_KEY_:_HASH_OF_HEADER_AND_SECRET_
```

In the example, replace `ACCESS_KEY` with the value for the access key ID followed by a colon (`:`). Replace `HASH_OF_HEADER_AND_SECRET` with a hash of a canonicalized header string and the secret corresponding to the access key ID.

Generate a hash of header string and secret.

To generate the hash of the header string and secret:

1. Get the value of the header string.
2. Normalize the request header string into canonical form.
3. Generate an HMAC with an SHA-1 hashing algorithm.
4. Encode the `hmac` result as base-64.

Normalizing header

Normalize the header into canonical form.

1. Get all `content-` headers.
2. Remove all `content-` headers except for `content-type` and `content-md5`.
3. Ensure the `content-` header names are lowercase.
4. Sort the `content-` headers lexicographically.
5. Ensure you have a `Date` header AND ensure the specified date uses GMT and not an offset.
6. Get all headers that begins with `x-amz-`.
7. Ensure that the `x-amz-` headers are all lowercase.
8. Sort the `x-amz-` headers lexicographically.
9. Combine multiple instances of the same field name into a single field and separate the field values with a comma.
10. Replace white space and line breaks in header values with a single space.
11. Remove white space before and after colons.
12. Append a new line after each header.
13. Merge the headers back into the request header.

Replace the *HASH_OF_HEADER_AND_SECRET* with the base-64 encoded HMAC string.

Reference

– Signing and Authenticating REST Requests

S3 server-side encryption

The Ceph Object Gateway supports server-side encryption of uploaded objects for the S3 application programming interface (API). Server-side encryption means that the S3 client sends data over HTTP in its unencrypted form, and the Ceph Object Gateway stores that data in the Red Hat Ceph Storage cluster in encrypted form.

Important: To use encryption, client requests must send requests over an SSL connection. S3 encryption from a client is not supported unless the Ceph Object Gateway uses SSL. For testing purposes, administrators can disable SSL during testing. SSL can be disabled by setting the `rgw_crypt_require_ssl` configuration setting to `false` at run time by using the **`ceph config set client.rgw`** command and then restarting the Ceph Object Gateway instance.

Note: S3 object encryption of Static Large Object (SLO) and Dynamic Large Object (DLO) are not supported.

In a production environment, it might not be possible to send encrypted requests over SSL. In such a case, send requests by using HTTP with server-side encryption.

There are two options for the management of encryption keys:

Customer-provided keys

When using customer-provided keys, the S3 client passes an encryption key along with each request to read or write encrypted data. It is the customer's responsibility to manage those keys. Customers must remember which key the Ceph Object Gateway used to encrypt each object.

Ceph Object Gateway implements the customer-provided key behavior in the S3 API according to the Amazon SSE-C specification.

Since the customer handles the key management and the S3 client passes keys to the Ceph Object Gateway, the Ceph Object Gateway requires no special configuration to support this encryption mode.

Key management service

Note: For the latest supported key managers, see "Compatibility matrix for Red Hat Ceph Storage 9.0" on page 13.

When using a key management service, the secure key management service stores the keys and the Ceph Object Gateway retrieves them on demand to serve requests to encrypt or decrypt data.

Ceph Object Gateway implements the key management service behavior in the S3 API according to the Amazon SSE-KMS specification.

For more information, see Configuring server-side encryption and HashiCorp Vault.

Related information

Amazon SSE-C
Amazon SSE-KMS

S3 access control lists

Ceph Object Gateway supports S3-compatible Access Control Lists (ACL) functionality.

An ACL is a list of access grants that specify which operations a user can perform on a bucket or on an object. Each grant has a different meaning when applied to a bucket versus applied to an object, as listed in "User operations" on page 1450.

Table 152: User operations

Permission	Bucket	Object
`READ`	Grantee can list the objects in the bucket.	Grantee can read the object.
`WRITE`	Grantee can write or delete objects in the bucket.	N/A

Permission	Bucket	Object
READ_ACP	Grantee can read bucket ACL.	Grantee can read the object ACL.
WRITE_ACP	Grantee can write bucket ACL.	Grantee can write to the object ACL.
FULL_CONTROL	Grantee has full permissions for object in the bucket.	Grantee can read or write to the object ACL.

Preparing access to the Ceph Object Gateway using S3

Prepare the Ceph Object Gatway node before attempting to access the gateway server.

Prerequisites

- Installation of the Ceph Object Gateway software.
- Root-level access to the Ceph Object Gateway node.

Procedure

1. As root, open port 8080 on the firewall:

```
[root@rgw ~]# firewall-cmd --zone=public --add-port=8080/tcp --permanent
[root@rgw ~]# firewall-cmd --reload
```

2. Add a wildcard to the DNS server that you are using for the gateway, as mentioned in Add a wildcard to the DNS.

   You can also set up the gateway node for local DNS caching. To do so, run the following steps:

   a. As root, install and setup dnsmasq:

```
[root@rgw ]# yum install dnsmasq
[root@rgw ]# echo "address=/.FQDN_OF_GATEWAY_NODE/IP_OF_GATEWAY_NODE" | tee --
append /etc/dnsmasq.conf
[root@rgw ]# systemctl start dnsmasq
[root@rgw ]# systemctl enable dnsmasq
```

   Replace IP_OF_GATEWAY_NODE and FQDN_OF_GATEWAY_NODE with the IP address and FQDN of the gateway node.

   b. As root, stop NetworkManager:

```
[root@rgw ~]# systemctl stop NetworkManager
[root@rgw ~]# systemctl disable NetworkManager
```

   c. As root, set the gateway server's IP as the nameserver:

```
[root@rgw ]# echo "DNS1=IP_OF_GATEWAY_NODE" | tee --append /etc/sysconfig/
network-scripts/ifcfg-eth0
[root@rgw ]# echo "IP_OF_GATEWAY_NODE FQDN_OF_GATEWAY_NODE" | tee --append /etc/
hosts
[root@rgw ]# systemctl restart network
[root@rgw ]# systemctl enable network
[root@rgw ~]# systemctl restart dnsmasq
```

   Replace IP_OF_GATEWAY_NODE and FQDN_OF_GATEWAY_NODE with the IP address and FQDN of the gateway node.

   d. Verify subdomain requests:

```
[user@rgw ~]$ ping mybucket.FQDN_OF_GATEWAY_NODE
```

   Replace FQDN_OF_GATEWAY_NODE with the FQDN of the gateway node.

> **Warning:** Setting up the gateway server for local DNS caching is for testing purposes only. You won't be able to access the outside network after doing this. It is strongly recommended to use a proper DNS server for the Red Hat Ceph Storage cluster and gateway node.

3. Create the `radosgw` user for `S3` access carefully and copy the generated `access_key` and `secret_key`.You will need these keys for `S3` access and subsequent bucket management tasks. For more information, see Create an S3 user

Accessing the Ceph Object Gateway that uses Ruby AWS S3

Use Ruby programming language along with `aws-s3` gem for `S3` access. Run the steps that are described on the node that is used for accessing the Ceph Object Gateway server with `Ruby AWS::S3`.

Prerequisites

- User-level access to Ceph Object Gateway.
- Root-level access to the node that can access the Ceph Object Gateway.
- Internet access.

Procedure

1. Install the `ruby` package:

```
[root@dev ~]# yum install ruby
```

> **Note:** The command installs `ruby` and its essential dependencies like `rubygems` and `ruby-libs`. If somehow the command does not install all the dependencies, install them separately.

2. Install the `aws-s3` Ruby package:

```
[root@dev ~]# gem install aws-s3
```

3. Create a project directory:

```
[user@dev ~]$ mkdir ruby_aws_s3
[user@dev ~]$ cd ruby_aws_s3
```

4. Create the connection file:

```
[user@dev ~]$ vim conn.rb
```

5. Paste the following contents into the `conn.rb` file:

   Syntax

```
#!/usr/bin/env ruby

require aws/s3
require resolv-replace

AWS::S3::Base.establish_connection!(
        :server            => FQDN_OF_GATEWAY_NODE,
        :port           => 8080,
        :access_key_id     => MY_ACCESS_KEY,
        :secret_access_key => MY_SECRET_KEY
)
```

   Replace `FQDN_OF_GATEWAY_NODE` with the FQDN of the Ceph Object Gateway node.

   Example

```
#!/usr/bin/env ruby

require 'aws/s3'
require 'resolv-replace'

AWS::S3::Base.establish_connection!(
        :server            => 'testclient.englab.pnq.redhat.com',
        :port           => '8080',
        :access_key_id     => '98J4R9P22P5CDL65HKP8',
        :secret_access_key => '6C+jcaP0dp0+FZfrRNgyGA9EzRy25pURldwje049'
)
```

Save the file and exit the editor.

6. Make the file executable:

```
[user@dev ~]$ chmod +x conn.rb
```

7. Run the file:

```
[user@dev ~]$ ./conn.rb | echo $?
```

If you have provided the values correctly in the file, the output of the command is `0`.

8. Create a file for creating a bucket:

```
[user@dev ~]$ vim create_bucket.rb
```

Paste the following contents into the file:

```
#!/usr/bin/env ruby

load 'conn.rb'

AWS::S3::Bucket.create('my-new-bucket1')
```

Save the file and exit the editor.

9. Make the file executable:

```
[user@dev ~]$ chmod +x create_bucket.rb
```

10. Run the file:

```
[user@dev ~]$ ./create_bucket.rb
```

If the output of the command is `true` it would mean that a bucket `my-new-bucket1` was created successfully.

11. Create a file for listing owned buckets:

```
[user@dev ~]$ vim list_owned_buckets.rb
```

Paste the following content into the file:

```
#!/usr/bin/env ruby

load 'conn.rb'

AWS::S3::Service.buckets.each do |bucket|
        puts "{bucket.name}\t{bucket.creation_date}"
end
```

Save the file and exit the editor.

12. Make the file executable:

```
[user@dev ~]$ chmod +x list_owned_buckets.rb
```

13. Run the file:

```
[user@dev ~]$ ./list_owned_buckets.rb
```

The output should look something like this:

```
my-new-bucket1 2020-01-21 10:33:19 UTC
```

14. Create a file for creating an object:

```
[user@dev ~]$ vim create_object.rb
```

Paste the following contents into the file:

```
#!/usr/bin/env ruby

load 'conn.rb'

AWS::S3::S3Object.store(
        'hello.txt',
        'Hello World!',
        'my-new-bucket1',
        :content_type => 'text/plain'
)
```

Save the file and exit the editor.

15. Make the file executable:

```
[user@dev ~]$ chmod +x create_object.rb
```

16. Run the file:

```
[user@dev ~]$ ./create_object.rb
```

This command creates a file `hello.txt` with the string `Hello World!`.

17. Create a new file for listing a bucket's content:

```
[user@dev ~]$ vim list_bucket_content.rb
```

Paste the following content into the file:

```
#!/usr/bin/env ruby

load 'conn.rb'

new_bucket = AWS::S3::Bucket.find('my-new-bucket1')
new_bucket.each do |object|
        puts "{object.key}\t{object.about['content-length']}\t{object.about['last-
modified']}"
end
```

Save the file and exit the editor.

18. Make the file executable.

```
[user@dev ~]$ chmod +x list_bucket_content.rb
```

19. Run the file:

```
[user@dev ~]$ ./list_bucket_content.rb
```

The output looks something like this:

```
hello.txt    12    Fri, 22 Jan 2020 15:54:52 GMT
```

20. Create a file for deleting an empty bucket:

```
[user@dev ~]$ vim del_empty_bucket.rb
```

Paste the following contents into the file:

```
#!/usr/bin/env ruby
load 'conn.rb'
AWS::S3::Bucket.delete('my-new-bucket1')
```

Save the file and exit the editor.

21. Make the file executable:

```
[user@dev ~]$ chmod +x del_empty_bucket.rb
```

22. Run the file:

```
[user@dev ~]$ ./del_empty_bucket.rb | echo $?
```

If the bucket is successfully deleted, the command returns `0` as output.

> **Note:** Edit the `create_bucket.rb` file to create empty buckets, for example, `my-new-bucket4`, `my-new-bucket5`. Next, edit the `del_empty_bucket.rb` file before deleting the empty buckets.

23. Create a file for deleting nonempty buckets:

```
[user@dev ~]$ vim del_non_empty_bucket.rb
```

Paste the following contents into the file:

```
#!/usr/bin/env ruby
load 'conn.rb'
AWS::S3::Bucket.delete('my-new-bucket1', :force => true)
```

Save the file and exit the editor.

24. Make the file executable:

```
[user@dev ~]$ chmod +x del_non_empty_bucket.rb
```

25. Run the file:

```
[user@dev ~]$ ./del_non_empty_bucket.rb | echo $?
```

If the bucket is successfully deleted, the command returns `0` as output.

26. Create a file for deleting an object:

```
[user@dev ~]$ vim delete_object.rb
```

Paste the following contents into the file:

```
#!/usr/bin/env ruby
load 'conn.rb'
```

```
AWS::S3::S3Object.delete('hello.txt', 'my-new-bucket1')
```

Save the file and exit the editor.

27. Make the file executable:

```
[user@dev ~]$ chmod +x delete_object.rb
```

28. Run the file:

```
[user@dev ~]$ ./delete_object.rb
```

This command deletes the object `hello.txt`.

Accessing the Ceph Object Gateway using Ruby AWS SDK

Use the Ruby programming language along with `aws-sdk` gem for `S3` access. Run the steps on the node used for accessing the Ceph Object Gateway server with `Ruby AWS::SDK`.

Prerequisites

- User-level access to Ceph Object Gateway.
- Root-level access to the node accessing the Ceph Object Gateway.
- Internet access.

Procedure

1. Install the `ruby` package:

```
[root@dev ~]# yum install ruby
```

> **Note:** This command installs `ruby` and its essential dependencies like `rubygems` and `ruby-libs`. If somehow the command does not install all the dependencies, install them separately.

2. Install the `aws-sdk` Ruby package:

```
[root@dev ~]# gem install aws-sdk
```

3. Create a project directory:

```
[user@dev ~]$ mkdir ruby_aws_sdk
[user@dev ~]$ cd ruby_aws_sdk
```

4. Create the connection file:

```
[user@dev ~]$ vim conn.rb
```

5. Paste the following contents into the `conn.rb` file:

Syntax

```
#!/usr/bin/env ruby

require aws-sdk
require resolv-replace

Aws.config.update(
        endpoint: http://FQDN_OF_GATEWAY_NODE:8080,
        access_key_id: MY_ACCESS_KEY,
```

```
        secret_access_key: MY_SECRET_KEY,
        force_path_style: true,
        region: us-east-1
)
```

Replace `FQDN_OF_GATEWAY_NODE` with the FQDN of the Ceph Object Gateway node. Replace `MY_ACCESS_KEY` and `MY_SECRET_KEY` with the `access_key` and `secret_key` that were generated when you created the `radosgw` user for `S3` access as mentioned in the Create an S3 user section.

Example

```
#!/usr/bin/env ruby

require 'aws-sdk'
require 'resolv-replace'

Aws.config.update(
        endpoint: 'http://testclient.englab.pnq.redhat.com:8080',
        access_key_id: '98J4R9P22P5CDL65HKP8',
        secret_access_key: '6C+jcaP0dp0+FZfrRNgyGA9EzRy25pURldwje049',
        force_path_style: true,
        region: 'us-east-1'
)
```

Save the file and exit the editor.

6. Make the file executable:

```
[user@dev ~]$ chmod +x conn.rb
```

7. Run the file:

```
[user@dev ~]$ ./conn.rb | echo $?
```

If you have provided the values correctly in the file, the output of the command will be `0`.

8. Create a file for creating a bucket:

```
[user@dev ~]$ vim create_bucket.rb
```

Paste the following contents into the file:

Syntax

```
#!/usr/bin/env ruby

load 'conn.rb'

s3_client = Aws::S3::Client.new
s3_client.create_bucket(bucket: 'my-new-bucket2')
```

Save the file and exit the editor.

9. Make the file executable:

```
[user@dev ~]$ chmod +x create_bucket.rb
```

10. Run the file:

```
[user@dev ~]$ ./create_bucket.rb
```

If the output of the command is `true`, this means that bucket `my-new-bucket2` was created successfully.

11. Create a file for listing owned buckets:

```
[user@dev ~]$ vim list_owned_buckets.rb
```

Paste the following content into the file:

```
#!/usr/bin/env ruby

load 'conn.rb'

s3_client = Aws::S3::Client.new
s3_client.list_buckets.buckets.each do |bucket|
        puts "{bucket.name}\t{bucket.creation_date}"
end
```

Save the file and exit the editor.

12. Make the file executable:

```
[user@dev ~]$ chmod +x list_owned_buckets.rb
```

13. Run the file:

```
[user@dev ~]$ ./list_owned_buckets.rb
```

The output should look something like this:

```
my-new-bucket2 2020-01-21 10:33:19 UTC
```

14. Create a file for creating an object:

```
[user@dev ~]$ vim create_object.rb
```

Paste the following contents into the file:

```
#!/usr/bin/env ruby

load 'conn.rb'

s3_client = Aws::S3::Client.new
s3_client.put_object(
        key: 'hello.txt',
        body: 'Hello World!',
        bucket: 'my-new-bucket2',
        content_type: 'text/plain'
)
```

Save the file and exit the editor.

15. Make the file executable:

```
[user@dev ~]$ chmod +x create_object.rb
```

16. Run the file:

```
[user@dev ~]$ ./create_object.rb
```

This will create a file `hello.txt` with the string `Hello World!`.

17. Create a file for listing a bucket's content:

```
[user@dev ~]$ vim list_bucket_content.rb
```

Paste the following content into the file:

```
#!/usr/bin/env ruby

load 'conn.rb'

s3_client = Aws::S3::Client.new
s3_client.list_objects(bucket: 'my-new-bucket2').contents.each do |object|
        puts "{object.key}\t{object.size}"
end
```

Save the file and exit the editor.

18. Make the file executable.

```
[user@dev ~]$ chmod +x list_bucket_content.rb
```

19. Run the file:

```
[user@dev ~]$ ./list_bucket_content.rb
```

The output will look something like this:

```
hello.txt    12    Fri, 22 Jan 2020 15:54:52 GMT
```

20. Create a file for deleting an empty bucket:

```
[user@dev ~]$ vim del_empty_bucket.rb
```

Paste the following contents into the file:

```
#!/usr/bin/env ruby

load 'conn.rb'

s3_client = Aws::S3::Client.new
s3_client.delete_bucket(bucket: 'my-new-bucket2')
```

Save the file and exit the editor.

21. Make the file executable:

```
[user@dev ~]$ chmod +x del_empty_bucket.rb
```

22. Run the file:

```
[user@dev ~]$ ./del_empty_bucket.rb | echo $?
```

If the bucket is successfully deleted, the command will return `0` as output.

> **Note:** Edit the `create_bucket.rb` file to create empty buckets, for example, `my-new-bucket6`, `my-new-bucket7`. Next, edit the `del_empty_bucket.rb` file prior to deleting the empty buckets.

23. Create a file for deleting a non-empty bucket:

```
[user@dev ~]$ vim del_non_empty_bucket.rb
```

Paste the following contents into the file:

```
#!/usr/bin/env ruby

load 'conn.rb'

s3_client = Aws::S3::Client.new
Aws::S3::Bucket.new('my-new-bucket2', client: s3_client).clear!
s3_client.delete_bucket(bucket: 'my-new-bucket2')
```

Save the file and exit the editor.

24. Make the file executable:

```
[user@dev ~]$ chmod +x del_non_empty_bucket.rb
```

25. Run the file:

```
[user@dev ~]$ ./del_non_empty_bucket.rb | echo $?
```

If the bucket is successfully deleted, the command will return `0` as output.

26. Create a file for deleting an object:

```
[user@dev ~]$ vim delete_object.rb
```

Paste the following contents into the file:

```
#!/usr/bin/env ruby

load 'conn.rb'

s3_client = Aws::S3::Client.new
s3_client.delete_object(key: 'hello.txt', bucket: 'my-new-bucket2')
```

Save the file and exit the editor.

27. Make the file executable:

```
[user@dev ~]$ chmod +x delete_object.rb
```

28. Run the file:

```
[user@dev ~]$ ./delete_object.rb
```

This will delete the object `hello.txt`.

Accessing the Ceph Object Gateway using PHP

Use PHP scripts for S3 access. This procedure provides some example PHP scripts to do various tasks, such as deleting a bucket or an object.

> **Important:** The examples are tested against `php v5.4.16` and `aws-sdk v2.8.24`.

Prerequisites

- Root-level access to a development workstation.
- Internet access.

1. Install the `php` package:

```
[root@dev ~]# yum install php
```

2. Download the zip archive of `aws-sdk` for PHP and extract it.
3. Create a project directory:

```
[user@dev ~]$ mkdir php_s3
[user@dev ~]$ cd php_s3
```

4. Copy the extracted `aws` directory to the project directory. For example:

```
[user@dev ~]$ cp -r ~/Downloads/aws/ ~/php_s3/
```

5. Create the connection file:

```
[user@dev ~]$ vim conn.php
```

6. Paste the following contents in the `conn.php` file:

Syntax

```
<?php
define(AWS_KEY, MY_ACCESS_KEY);
define(AWS_SECRET_KEY, MY_SECRET_KEY);
define(HOST, FQDN_OF_GATEWAY_NODE);
define(PORT, 8080);

// require the AWS SDK for php library
require /PATH_TO_AWS/aws-autoloader.php;

use Aws\S3\S3Client;

// Establish connection with host using S3 Client
client = S3Client::factory(array(
    base_url => HOST,
    port => PORT,
    key      => AWS_KEY,
    secret   => AWS_SECRET_KEY
));
?>
```

Replace `FQDN_OF_GATEWAY_NODE` with the FQDN of the gateway node. Replace `PATH_TO_AWS` with the absolute path to the extracted `aws` directory that you copied to the `php` project directory.

Save the file and exit the editor.

7. Run the file:

```
[user@dev ~]$ php -f conn.php | echo $?
```

If you have provided the values correctly in the file, the output of the command will be `0`.

8. Create a new file for creating a bucket:

```
[user@dev ~]$ vim create_bucket.php
```

Paste the following contents into the new file:

Syntax

```
<?php

include 'conn.php';

client->createBucket(array('Bucket' => 'my-new-bucket3'));

?>
```

Save the file and exit the editor.

9. Run the file:

```
[user@dev ~]$ php -f create_bucket.php
```

10. Create a new file for listing owned buckets:

```
[user@dev ~]$ vim list_owned_buckets.php
```

Paste the following content into the file:

Syntax

```
<?php

include 'conn.php';

blist = client->listBuckets();
echo "Buckets belonging to " . blist['Owner']['ID'] . ":\n";
foreach (blist['Buckets'] as b) {
    echo "{b['Name']}\t{b['CreationDate']}\n";
}
```

```
?>
```

Save the file and exit the editor.

11. Run the file:

```
[user@dev ~]$ php -f list_owned_buckets.php
```

The output should look similar to this:

```
my-new-bucket3 2020-01-21 10:33:19 UTC
```

12. Create an object by first creating a source file named `hello.txt`:

```
[user@dev ~]$ echo "Hello World!" > hello.txt
```

13. Create a new php file:

```
[user@dev ~]$ vim create_object.php
```

Paste the following contents into the file:

Syntax

```
<?php

include 'conn.php';

key         = 'hello.txt';
source_file = './hello.txt';
acl         = 'private';
bucket      = 'my-new-bucket3';
client->upload(bucket, key, fopen(source_file, 'r'), acl);

?>
```

Save the file and exit the editor.

14. Run the file:

```
[user@dev ~]$ php -f create_object.php
```

This will create the object `hello.txt` in bucket `my-new-bucket3`.

15. Create a new file for listing a bucket's content:

```
[user@dev ~]$ vim list_bucket_content.php
```

Paste the following content into the file:

Syntax

```
<?php

include 'conn.php';

o_iter = client->getIterator('ListObjects', array(
    'Bucket' => 'my-new-bucket3'
));
foreach (o_iter as o) {
    echo "{o['Key']}\t{o['Size']}\t{o['LastModified']}\n";
}
?>
```

Save the file and exit the editor.

16. Run the file:

```
[user@dev ~]$ php -f list_bucket_content.php
```

The output will look similar to this:

```
hello.txt    12    Fri, 22 Jan 2020 15:54:52 GMT
```

17. Create a new file for deleting an empty bucket:

```
[user@dev ~]$ vim del_empty_bucket.php
```

Paste the following contents into the file:

Syntax

```
<?php

include 'conn.php';

client->deleteBucket(array('Bucket' => 'my-new-bucket3'));
?>
```

Save the file and exit the editor.

18. Run the file:

```
[user@dev ~]$ php -f del_empty_bucket.php | echo $?
```

If the bucket is successfully deleted, the command will return `0` as output.

> **Important:** Deleting a non-empty bucket is currently not supported in PHP 2 and newer versions of `aws-sdk`.

> **Note:** Edit the `create_bucket.php` file to create empty buckets, for example, `my-new-bucket4`, `my-new-bucket5`. Next, edit the `del_empty_bucket.php` file accordingly prior to deleting the empty buckets.

19. Create a new file for deleting an object:

```
[user@dev ~]$ vim delete_object.php
```

Paste the following contents into the file:

Syntax

```
<?php

include 'conn.php';

client->deleteObject(array(
    'Bucket' => 'my-new-bucket3',
    'Key'    => 'hello.txt',
));
?>
```

Save the file and exit the editor.

20. Run the file:

```
[user@dev ~]$ php -f delete_object.php
```

This will delete the object `hello.txt`.

S3 bucket operations

Perform bucket operations with the Amazon S3 application programming interface (API) through the Ceph Object Gateway.

To use S3 bucket operations, a running Red Hat Ceph Storage cluster and a RESTful client are required.

"S3 functional operations and support status for buckets" on page 1464 describes the Amazon S3 functional operations for buckets, along with the function's support status.

Table 153: S3 functional operations and support status for buckets

Feature	Status	Notes
List Buckets	Supported	
Create a Bucket	Supported	Different set of canned ACLs.
Put Bucket Website	Supported	
Get Bucket Website	Supported	
Delete Bucket Website	Supported	
Bucket Lifecycle	Partially Supported	`Expiration`, `NoncurrentVersionExpiration` and `AbortIncompleteMultipartUpload` supported.
Put Bucket Lifecycle	Partially Supported	`Expiration`, `NoncurrentVersionExpiration` and `AbortIncompleteMultipartUpload` supported.
Delete Bucket Lifecycle	Supported	
Get Bucket Objects	Supported	
Bucket Location	Supported	
Get Bucket Version	Supported	
Put Bucket Version	Supported	
Delete Bucket	Supported	
Get Bucket ACLs	Supported	Different set of canned ACLs
Put Bucket ACLs	Supported	Different set of canned ACLs
Get Bucket cors	Supported	
Put Bucket cors	Supported	
Delete Bucket cors	Supported	
List Bucket Object Versions	Supported	
Head Bucket	Supported	
List Bucket Multipart Uploads	Supported	
Bucket Policies	Partially Supported	
Get a Bucket Request Payment	Supported	
Put a Bucket Request Payment	Supported	
GET `PublicAccessBlock`	Supported	
PUT `PublicAccessBlock`	Supported	
Delete `PublicAccessBlock`	Supported	

Bucket basics

Use these S3 API operations to create, list, inspect, and delete buckets in a Ceph Object Gateway.

Accessing bucket host names

There are two different modes of accessing the buckets. The first, and preferred method identifies the bucket as the first level directory in the URL.

Example

```
GET /mybucket HTTP/1.1
Host: cname.domain.com
```

The second method identifies the bucket via a virtual bucket host name.

Example

```
GET / HTTP/1.1
Host: mybucket.cname.domain.com
```

> **Tip:** While both methods are available, it is preferable to use the first method, because the second method requires expensive domain certification and DNS wild cards.

S3 list buckets

`GET /` returns a list of buckets created by the user making the request. `GET /` only returns buckets created by an authenticated user. You cannot make an anonymous request.

Syntax

```
GET / HTTP/1.1
Host: cname.domain.com

Authorization: AWS_ACCESS_KEY_:_HASH_OF_HEADER_AND_SECRET_
```

Response Entities

`Buckets`

Description

Container for list of buckets.

Type

Container

`Bucket`

Description

Container for bucket information.

Type

Container

`Name`

Description

Bucket name.

Type

String

`CreationDate`

Description

UTC time when the bucket was created.

Type

Date

`ListAllMyBucketsResult`

Description

A container for the result.

Type

Container

`Owner`

Description

A container for the bucket owner's `ID` and `DisplayName`.

Type

Container

`ID`

Description

The bucket owner's ID.

Type

String

`DisplayName`

Description

The bucket owner's display name.

Type

String

S3 create a new bucket

Create a new bucket. To create a bucket, you must have a user ID and a valid AWS Access Key ID to authenticate requests. You cannot create buckets as an anonymous user.

Constraints

In general, bucket names follow domain name constraints.

- Bucket names must be unique.
- Bucket names cannot be formatted as IP address.
- Bucket names can be between 3 and 63 characters long.
- Bucket names must not contain uppercase characters or underscores.
- Bucket names must start with a lowercase letter or number.
- Bucket names can contain a dash (-).
- Bucket names must be a series of one or more labels. Adjacent labels are separated by a single period (.). Bucket names can contain lowercase letters, numbers, and hyphens. Each label must start and end with a lowercase letter or a number.

Note: The constraints are relaxed if `rgw_relaxed_s3_bucket_names` is set to `true`. The bucket names must still be unique, cannot be formatted as IP address, and can contain letters, numbers, periods, dashes, and underscores of up to 255 characters long.

Syntax

```
PUT /_BUCKET_ HTTP/1.1
Host: cname.domain.com
x-amz-acl: public-read-write

Authorization: AWS _ACCESS_KEY_:_HASH_OF_HEADER_AND_SECRET_
```

Parameters `x-amz-acl`

Description

Canned ACLs.

Valid Values

`private`, `public-read`,`public-read-write`, `authenticated-read`

Required

No

HTTP Response

If the bucket name is unique, within constraints, and unused, the operation succeeds. If a bucket with the same name exists and the user is the bucket owner, the operation will succeed. If the bucket name is already in use, the operation fails.

`409`

Status Code

`BucketAlreadyExists`

Description

Bucket exists under different user's ownership.

S3 head bucket

Calls HEAD on a bucket to determine if it exists and if the caller has access permissions. Returns `200 OK` if the bucket exists and the caller has permissions; `404 Not Found` if the bucket does not exist; and, `403 Forbidden` if the bucket exists but the caller does not have access permissions.

Syntax

```
HEAD /_BUCKET_ HTTP/1.1
Host: cname.domain.com
Date: date
Authorization: AWS _ACCESS_KEY_:_HASH_OF_HEADER_AND_SECRET_
```

S3 delete a bucket

Deletes a bucket. You can reuse bucket names following a successful bucket removal.

Syntax

```
DELETE /_BUCKET_ HTTP/1.1
Host: cname.domain.com

Authorization: AWS _ACCESS_KEY_:_HASH_OF_HEADER_AND_SECRET_
```

HTTP Response

`204`

Status Code

`No Content`

Description

Bucket removed.

S3 create bucket notifications

Create bucket notifications at the bucket level. The notification configuration has the Red Hat Ceph Storage Object Gateway S3 events, `ObjectCreated`, `ObjectRemoved`, and `ObjectLifecycle:Expiration`.

These need to be published and the destination to send the bucket notifications. Bucket notifications are S3 operations.

To create a bucket notification for `s3:objectCreate`, `s3:objectRemove`, and `s3:ObjectLifecycle:Expiration` events, use PUT:

Example

```
client.put_bucket_notification_configuration(
   Bucket=bucket_name,
   NotificationConfiguration={
       'TopicConfigurations': [
           {
               'Id': notification_name,
               'TopicArn': topic_arn,
               'Events': ['s3:ObjectCreated:*', 's3:ObjectRemoved:*',
's3:ObjectLifecycle:Expiration:*']
           }]})
```

Important: `ObjectCreate` events, such as `put`, `post`, `multipartUpload`, and `copy` are supported. `ObjectRemove` events, such as `object_delete` and `s3_multi_object_delete` are also supported.

Request Entities

`NotificationConfiguration`

Description

list of `TopicConfiguration` entities.

Type

Container

Required

Yes

`TopicConfiguration`

Description

`Id`, `Topic`, and `list` of Event entities.

Type

Container

Required

Yes

`id`

Description

Name of the notification.

Type

String

Required

Yes

`Topic`

Description

Topic Amazon Resource Name(ARN)

> **Note:** The topic must be created beforehand.

Type

String

Required

Yes

`Event`

Description

List of supported events. Multiple event entities can be used. If omitted, all events are handled.

Type

String

Required

No

`Filter`

Description

`S3Key`, `S3Metadata` and `S3Tags` entities.

Type

Container

Required

No

`S3Key`

Description

A list of `FilterRule` entities, for filtering based on the object key. At most, 3 entities may be in the list, for example `Name` would be `prefix`, `suffix`, or `regex`. All filter rules in the list must match for the filter to match.

Type

Container

Required

No

`S3Metadata`

Description

A list of `FilterRule` entities, for filtering based on object metadata. All filter rules in the list must match the metadata defined on the object. However, the object still matches if it has other metadata entries not listed in the filter.

Type

Container

Required

No

`S3Tags`

Description

A list of `FilterRule` entities, for filtering based on object tags. All filter rules in the list must match the tags defined on the object. However, the object still matches if it has other tags not listed in the filter.

Type

Container

Required

No

`S3Key.FilterRule`

Description

`Name` and `Value` entities. Name is : `prefix`, `suffix`, or `regex`. The `Value` would hold the key prefix, key suffix, or a regular expression for matching the key, accordingly.

Type

Container

Required

Yes

`S3Metadata.FilterRule`

Description

`Name` and `Value` entities. Name is the name of the metadata attribute for example `x-amz-meta-xxx`. The value is the expected value for this attribute.

Type

Container

Required

Yes

`S3Tags.FilterRule`

Description

`Name` and `Value` entities. Name is the tag key, and the value is the tag value.

Type

Container

Required

Yes

HTTP response

`400`

Status Code

`MalformedXML`

Description

The XML is not well-formed.

`400`

Status Code

`InvalidArgument`

Description

Missing Id or missing or invalid topic ARN or invalid event.

404

Status Code

NoSuchBucket

Description

The bucket does not exist.

404

Status Code

NoSuchKey

Description

The topic does not exist.

S3 get bucket notifications

Get a specific notification or list all the notifications configured on a bucket.

Syntax

```
Get /_BUCKET_?notification=_NOTIFICATIONID HTTP/1.1
Host: cname.domain.com
Date: date
Authorization: AWS _ACCESS_KEY_:_HASH_OF_HEADER_AND_SECRET_
```

Example

```
Get /testbucket?notification=testnotificationID HTTP/1.1
Host: cname.domain.com
Date: date
Authorization: AWS _ACCESS_KEY_:_HASH_OF_HEADER_AND_SECRET_
```

Example Response

```
<NotificationConfiguration xmlns="http://s3.amazonaws.com/doc/2006-03-01/">
    <TopicConfiguration>
        <Id></Id>
        <Topic></Topic>
        <Event></Event>
        <Filter>
            <S3Key>
                <FilterRule>
                    <Name></Name>
                    <Value></Value>
                </FilterRule>
                 </S3Key>
             <S3Metadata>
                 <FilterRule>
                     <Name></Name>
                     <Value></Value>
                 </FilterRule>
             </S3Metadata>
             <S3Tags>
                 <FilterRule>
                     <Name></Name>
                     <Value></Value>
                 </FilterRule>
             </S3Tags>
         </Filter>
    </TopicConfiguration>
</NotificationConfiguration>
```

> **Note:** The `notification` subresource returns the bucket notification configuration or an empty `NotificationConfiguration` element. The caller must be the bucket owner.

Request Entities

`notification-id`

Description

Name of the notification. All notifications are listed if the ID is not provided.

Type

String

`NotificationConfiguration`

Description

list of `TopicConfiguration` entities.

Type

Container

Required

Yes

`TopicConfiguration`

Description

`Id`, `Topic`, and `list` of Event entities.

Type

Container

Required

Yes

`id`

Description

Name of the notification.

Type

String

Required

Yes

`Topic`

Description

Topic Amazon Resource Name(ARN)

> **Note:** The topic must be created beforehand.

Type

String

Required

Yes

`Event`

Description

Handled event. Multiple event entities may exist.

Type

String

Required

Yes

`Filter`

Description

The filters for the specified configuration.

Type

Container

Required

No

HTTP response

`404`

Status Code

`NoSuchBucket`

Description

The bucket does not exist.

`404`

Status Code

`NoSuchKey`

Description

The notification does not exist if it has been provided.

S3 delete bucket notifications

Delete a specific or all notifications from a bucket.

> **Note:** Notification deletion is an extension to the S3 notification API. Any defined notifications on a bucket are deleted when the bucket is deleted. Deleting an unknown notification for example `double delete`, is not considered an error.

To delete a specific or all notifications use DELETE:

Syntax

```
DELETE /BUCKET?notification=NOTIFICATION_ID HTTP/1.1
```

Example

```
DELETE /testbucket?notification=testnotificationID HTTP/1.1
```

Request Entities

`notification-id`

Description

Name of the notification. All notifications on the bucket are deleted if the notification ID is not provided.

Type

String

HTTP response

`404`

Status Code

`NoSuchBucket`

Description

The bucket does not exist.

Bucket configuration

Use these S3 API operations to configure bucket policies, versioning, location, payment settings, and multi-tenancy on a Ceph Object Gateway bucket.

S3 bucket policies

The Ceph Object Gateway supports a subset of the Amazon S3 policy language that controls access to buckets.

The Ceph Object Gateway supports a subset of the Amazon S3 policy language that is applied to buckets.

Creation and removal

Ceph Object Gateway manages S3 bucket policies through standard S3 operations rather than using the **`radosgw-admin`** CLI tool.

You can use the **`s3cmd`** command to set or delete a policy.

The following is an example of setting and deleting an S3 bucket policy:

```
$ cat > examplepol
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"AWS": ["arn:aws:iam::usfolks:user/fred"]},
    "Action": "s3:PutObjectAcl",
    "Resource": [
      "arn:aws:s3:::happybucket/*"
    ]
  }]
}

$ s3cmd setpolicy examplepol s3://happybucket
$ s3cmd delpolicy s3://happybucket
```

Limitations

Ceph Object Gateway supports the following S3 actions only:

- `s3:AbortMultipartUpload`
- `s3:CreateBucket`
- `s3:DeleteBucketPolicy`
- `s3:DeleteBucket`
- `s3:DeleteBucketReplication`
- `s3:DeleteReplicationConfiguration`
- `s3:DeleteBucketWebsite`
- `s3:DeleteObject`
- `s3:DeleteObjectVersion`

- `s3:GetBucketAcl`
- `s3:GetBucketCORS`
- `s3:GetBucketLocation`
- `s3:GetBucketPolicy`
- `s3:GetBucketRequestPayment`
- `s3:GetBucketVersioning`
- `s3:GetBucketWebsite`
- `s3:GetBucketReplication`
- `s3:GetReplicationConfiguration`
- `s3:GetLifecycleConfiguration`
- `s3:GetObjectAcl`
- `s3:GetObject`
- `s3:GetObjectTorrent`
- `s3:GetObjectVersionAcl`
- `s3:GetObjectVersion`
- `s3:GetObjectVersionTorrent`
- `s3:ListAllMyBuckets`
- `s3:ListBucketMultiPartUploads`
- `s3:ListBucket`
- `s3:ListBucketVersions`
- `s3:ListMultipartUploadParts`
- `s3:PutBucketAcl`
- `s3:PutBucketCORS`
- `s3:PutBucketPolicy`
- `s3:PutBucketRequestPayment`
- `s3:PutBucketVersioning`
- `s3:PutBucketWebsite`
- `s3:PutBucketReplication`
- `s3:PutReplicationConfiguration`
- `s3:PutLifecycleConfiguration`
- `s3:PutObjectAcl`
- `s3:PutObject`
- `s3:PutObjectVersionAcl`

The Ceph Object Gateway uses the Ceph Object Gateway tenant identifier in place of the Amazon twelve-digit account ID. To use policies between Amazon Web Services (AWS) S3 and Ceph Object Gateway S3, you must use the Amazon account ID as the tenant ID during user creation.

With AWS S3, all tenants share a single namespace. By contrast, Ceph Object Gateway gives every tenant its own namespace of buckets. If you access a bucket that belongs to another tenant, you must address it as `tenant:bucket` in the S3 request.

In AWS, a bucket policy can grant access to another account, and that account owner can then grant access to individual users with user permissions. Because Ceph Object Gateway does not support user, role, and group permissions, you must grant access directly to individual users.

Important:

- Granting an entire account access to a bucket grants access to all users in that account.
- Bucket policies do not support string interpolation.

Ceph Object Gateway supports the following condition keys:

- `aws:CurrentTime`
- `aws:EpochTime`
- `aws:PrincipalType`
- `aws:Referer`
- `aws:SecureTransport`
- `aws:SourceIp`
- `aws:UserAgent`
- `aws:username`

Ceph Object Gateway supports only the following condition keys for the `ListBucket` action:

- `s3:prefix`
- `s3:delimiter`
- `s3:max-keys`

Impact on Swift

Ceph Object Gateway provides no functions to set bucket policies under the Swift API. However, bucket policies that are set with the S3 API govern Swift and S3 operations.

Ceph Object Gateway matches Swift credentials against principals that are specified in a policy.

S3 get bucket location

Retrieves the bucket's zone group. The user needs to be the bucket owner to call this. A bucket can be constrained to a zone group by providing `LocationConstraint` during a PUT request.

Add the `location` subresource to the bucket resource.

Syntax

```
GET /BUCKET?location HTTP/1.1
Host: cname.domain.com

Authorization: AWS ACCESS_KEY:HASH_OF_HEADER_AND_SECRET
```

Response Entities

`LocationConstraint`

Description

The zone group where bucket resides, an empty string for default zone group.

Type

String

S3 get bucket versioning

Retrieves the versioning state of a bucket. The user needs to be the bucket owner to call this.

Add the `versioning` subresource to the bucket resource.

Syntax

```
GET /BUCKET?versioning HTTP/1.1
Host: cname.domain.com

Authorization: AWS ACCESS_KEY:HASH_OF_HEADER_AND_SECRET
```

S3 put bucket versioning

This subresource set the versioning state of an existing bucket. The user needs to be the bucket owner to set the versioning state. If the versioning state has never been set on a bucket, then it has no versioning state. Doing a GET versioning request does not return a versioning state value.

Setting the bucket versioning state:

`Enabled`: Enables versioning for the objects in the bucket. All objects added to the bucket receive a unique version ID. `Suspended`: Disables versioning for the objects in the bucket. All objects added to the bucket receive the version ID null.

Syntax

```
PUT /BUCKET?versioning HTTP/1.1
```

Example

```
PUT /testbucket?versioning HTTP/1.1
```

Bucket Request Entities

`VersioningConfiguration`

Description

A container for the request.

Type

Container

`Status`

Description

Sets the versioning state of the bucket. Valid Values: Suspended/Enabled

Type

String

S3 get the request payment configuration on a bucket

Uses the `requestPayment` subresource to return the request payment configuration of a bucket. The user needs to be the bucket owner or to have been granted `READ_ACP` permission on the bucket.

Add the `requestPayment` subresource to the bucket request.

Syntax

```
GET /BUCKET?requestPayment HTTP/1.1
Host: cname.domain.com

Authorization: AWS ACCESS_KEY:HASH_OF_HEADER_AND_SECRET
```

S3 set the request payment configuration on a bucket

Uses the `requestPayment` subresource to set the request payment configuration of a bucket. By default, the bucket owner pays for downloads from the bucket. This configuration parameter enables the bucket owner to specify that the person requesting the download will be charged for the request and the data download from the bucket.

Add the `requestPayment` subresource to the bucket request.

Syntax

```
PUT /BUCKET?requestPayment HTTP/1.1
Host: cname.domain.com
```

Request Entities

`Payer`

Description

Specifies who pays for the download and request fees.

Type

Enum

`RequestPaymentConfiguration`

Description

A container for `Payer`.

Type

Container

Multi-tenant bucket operations

When a client application accesses buckets, it always operates with the credentials of a particular user. Consequently, every bucket operation has an implicit tenant in its context if no tenant is specified explicitly. Thus multi-tenancy is completely backward compatible with previous releases, as long as the referred buckets and referring user belong to the same tenant.

Extensions employed to specify an explicit tenant differ according to the protocol and authentication system used.

In the following example, a colon character separates tenant and bucket. Thus a sample URL would be:

```
https://rgw.domain.com/tenant:bucket
```

By contrast, a simple Python example separates the tenant and bucket in the bucket method itself:

Example

```
from boto.s3.connection import S3Connection, OrdinaryCallingFormat
  c = S3Connection(
    aws_access_key_id="TESTER",
    aws_secret_access_key="test123",
    host="rgw.domain.com",
    calling_format = OrdinaryCallingFormat()
  )
  bucket = c.get_bucket("tenant:bucket")
```

Note: It's not possible to use S3-style subdomains using multi-tenancy, since host names cannot contain colons or any other separators that are not already valid in bucket names. Using a period creates an ambiguous syntax. Therefore, the `bucket-in-URL-path` format has to be used with multi-tenancy.

Reference

- For more information, see Multi-tenant namespace.

Bucket objects and lifecycle

Use these S3 API operations to list objects and versions in a bucket, manage multipart uploads, and configure lifecycle rules on a Ceph Object Gateway bucket.

S3 return a list of bucket objects

Returns a list of bucket objects.

Syntax

```
GET /BUCKET?max-keys=25 HTTP/1.1
Host: cname.domain.com
```

Parameters

`prefix`

Description

Only returns objects that contain the specified prefix.

Type

String

`delimiter`

Description

The delimiter between the prefix and the rest of the object name.

Type

String

`marker`

Description

A beginning index for the list of objects returned.

Type

String

`max-keys`

Description

The maximum number of keys to return. Default is 1000.

Type

Integer

HTTP Response

`200`

Status Code

`OK`

Description

Buckets retrieved.

`GET /_BUCKET` returns a container for buckets with the following fields:

Bucket Response Entities

`ListBucketResult`

Description

The container for the list of objects.

Type

Entity

`Name`

Description

The name of the bucket whose contents will be returned.

Type

String

`Prefix`

Description

A prefix for the object keys.

Type

String

`Marker`

Description

A beginning index for the list of objects returned.

Type

String

`MaxKeys`

Description

The maximum number of keys returned.

Type

Integer

`Delimiter`

Description

If set, objects with the same prefix will appear in the `CommonPrefixes` list.

Type

String

`IsTruncated`

Description

If `true`, only a subset of the bucket's contents were returned.

Type

Boolean

`CommonPrefixes`

Description

If multiple objects contain the same prefix, they will appear in this list.

Type

Container

The `ListBucketResult` contains objects, where each object is within a `Contents` container.

Object Response Entities

`Contents`

Description

A container for the object.

Type

Object

`Key`

Description

The object's key.

Type

String

`LastModified`

Description

The object's last-modified date and time.

Type

Date

`ETag`

Description

An MD-5 hash of the object. Etag is an entity tag.

Type

String

`Size`

Description

The object's size.

Type

Integer

`StorageClass`

Description

Should always return `STANDARD`.

Type

String

S3 list bucket object versions

The *GET /BUCKET?versions* request returns a list of metadata about all the version of objects within a bucket.

The *GET /BUCKET?versions* request requires READ access to the bucket.

Add the versions subresource to the bucket request.

```
GET /BUCKET?versions HTTP/1.1
Host: cname.domain.com

Authorization: AWS ACCESS_KEY:HASH_OF_HEADER_AND_SECRET
```

You can specify parameters for *GET /BUCKET?versions*, but none of them are required. "GET /BUCKET?versions parameters" on page 1481 lists the optional parameters.

Table 154: GET /BUCKET?versions parameters

Parameter	Description	Type
`prefix`	Returns in-progress uploads whose keys contain the specified prefix.	String
`delimiter`	The delimiter between the prefix and the rest of the object name.	String
`key-marker`	The beginning marker for the list of uploads.	String

Parameter	Description	Type
`max-keys`	The maximum number of in-progress uploads. The default is 1000.	Integer
`version-id-marker`	Specifies the object version to begin the list.	String

“GET /BUCKET?versions response entities” on page 1482 lists the response entities.

Table 155: GET /BUCKET?versions response entities

Response Entity	Description	Type
`KeyMarker`	The key marker specified by the `key-marker` request parameter, if any.	String
`NextKeyMarker`	The key marker to use in a subsequent request if `IsTruncated` is `true`.	String
`NextUploadIdMarker`	The upload ID marker to use in a subsequent request if `IsTruncated` is `true`.	String
`IsTruncated`	If `true`, only a subset of the bucket’s upload contents were returned.	Boolean
`Size`	The size of the uploaded part.	Integer
`DisplayName`	The owner’s display name.	String
`ID`	The owner’s ID.	String
`Owner`	A container for the `ID` and `DisplayName` of the user who owns the object.	Container
`StorageClass`	The method used to store the resulting object. `STANDARD` or `REDUCED_REDUNDANCY`.	String
`Version`	Container for the version information.	Container
`versionId`	Version ID of an object.	String
`versionIdMarker`	The last version of the key in a truncated response.	String

S3 list multipart uploads

`GET /?uploads` returns a list of the current in-progress multipart uploads, that is, the application initiates a multipart upload, but the service hasn’t completed all the uploads yet.

Syntax

```
GET /_BUCKET_?uploads HTTP/1.1
```

You can specify parameters for `GET /_BUCKET_?uploads`, but none of them are required.

Parameters

`prefix`

Description

Returns in-progress uploads whose keys contain the specified prefix.

Type

String

`delimiter`

Description

The delimiter between the prefix and the rest of the object name.

Type

String

`key-marker`

Description

The beginning marker for the list of uploads.

Type

String

`max-keys`

Description

The maximum number of in-progress uploads. The default is 1000.

Type

Integer

`max-uploads`

Description

The maximum number of multipart uploads. The range is from 1-1000. The default is 1000.

Type

Integer

`version-id-marker`

Description

Ignored if `key-marker` isn't specified. Specifies the `ID` of the first upload to list in lexicographical order at or following the `ID`.

Type

String

Response Entities

`ListMultipartUploadsResult`

Description

A container for the results.

Type

Container

`ListMultipartUploadsResult.Prefix`

Description

The prefix specified by the `prefix` request parameter, if any.

Type

String

`Bucket`

Description

The bucket that will receive the bucket contents.

Type

String

`KeyMarker`

Description

The key marker specified by the `key-marker` request parameter, if any.

Type

String

`UploadIdMarker`

Description

The marker specified by the `upload-id-marker` request parameter, if any.

Type

String

`NextKeyMarker`

Description

The key marker to use in a subsequent request if `IsTruncated` is `true`.

Type

String

`NextUploadIdMarker`

Description

The upload ID marker to use in a subsequent request if `IsTruncated` is `true`.

Type

String

`MaxUploads`

Description

The max uploads specified by the `max-uploads` request parameter.

Type

Integer

`Delimiter`

Description

If set, objects with the same prefix will appear in the `CommonPrefixes` list.

Type

String

`IsTruncated`

Description

If `true`, only a subset of the bucket's upload contents were returned.

Type

Boolean

`Upload`

Description

A container for `Key`, `UploadId`, `InitiatorOwner`, `StorageClass`, and `Initiated` elements.

Type

Container

`Key`

Description

The key of the object once the multipart upload is complete.

Type

String

`UploadId`

Description

The `ID` that identifies the multipart upload.

Type

String

`Initiator`

Description

Contains the `ID` and `DisplayName` of the user who initiated the upload.

Type

Container

`DisplayName`

Description

The initiator's display name.

Type

String

`ID`

Description

The initiator's ID.

Type

String

`Owner`

Description

A container for the `ID` and `DisplayName` of the user who owns the uploaded object.

Type

Container

`StorageClass`

Description

The method used to store the resulting object. `STANDARD` or `REDUCED_REDUNDANCY`

Type

String

`Initiated`

Description

The date and time the user initiated the upload.

Type

Date

`CommonPrefixes`

Description

If multiple objects contain the same prefix, they will appear in this list.

Type

Container

`CommonPrefixes.Prefix`

Description

The substring of the key after the prefix as defined by the `prefix` request parameter.

Type

String

S3 bucket lifecycle

You can use a bucket lifecycle configuration to manage your objects so they are stored effectively throughout their lifetime. The S3 API in the Ceph Object Gateway supports a subset of the AWS bucket lifecycle actions:

`Expiration`

This action defines the lifespan of objects within a bucket. It takes the number of days the object lives or expiration date, at which point Ceph Object Gateway deletes the object. If the bucket doesn't enable versioning, Ceph Object Gateway deletes the object permanently. If the bucket enables versioning, Ceph Object Gateway creates a delete marker for the current version, and then delete the current version.

`NoncurrentVersionExpiration`

This action defines the lifespan of noncurrent object versions within a bucket. To use this feature, you must enable bucket versioning. It takes the number of days a noncurrent object lives, at which point Ceph Object Gateway deletes the noncurrent object.

`NewerNoncurrentVersions`

This action specifies how many noncurrent object versions to retain. You can specify up to 100 noncurrent versions to retain. If the specified number to retain is more than 100, additional noncurrent versions are deleted.

`AbortIncompleteMultipartUpload`

This action defines the number of days an incomplete multipart upload lives before it is cancelled.

`BlockPublicPolicy reject`

This action is for public access block. It calls PUT access point policy and PUT bucket policy that are made through the access point if the specified policy (for either the access point or the underlying bucket) allows public access. It provides settings for access points, buckets, and accounts to help you manage public access to Amazon S3 resources. By default, new buckets, access points, and objects do not allow public access. However, you can modify bucket policies, access point policies, or object permissions to allow public access. S3 Block Public Access settings override these policies and permissions so that you can limit public access to these resources.

The lifecycle configuration contains one or more rules that use the `<Rule>` element.

Example

```
<LifecycleConfiguration>
    <Rule>
      <Prefix/>
      <Status>Enabled</Status>
      <Expiration>
        <Days>10</Days>
      </Expiration>
    </Rule>
</LifecycleConfiguration>
```

A lifecycle rule can apply to all or a subset of objects in a bucket based on the `<Filter>` element that you specify in the lifecycle rule. You can specify a filter in several ways:

- Key prefixes
- Object tags
- Both key prefixes and one or more object tags

Key prefixes

You can apply a lifecycle rule to a subset of objects based on the key name prefix. For example, specifying `<keypre/>` applies to objects that begin with `keypre/`:

```
<LifecycleConfiguration>
    <Rule>
        <Status>Enabled</Status>
        <Filter>
           <Prefix>keypre/</Prefix>
        </Filter>
    </Rule>
</LifecycleConfiguration>
```

You can also apply different lifecycle rules to objects with different key prefixes:

```
<LifecycleConfiguration>
    <Rule>
        <Status>Enabled</Status>
        <Filter>
           <Prefix>keypre/</Prefix>
        </Filter>
    </Rule>
    <Rule>
        <Status>Enabled</Status>
        <Filter>
           <Prefix>mypre/</Prefix>
        </Filter>
    </Rule>
</LifecycleConfiguration>
```

Object tags

You can apply a lifecycle rule to only objects with a specific tag that uses the `<Key>` and `<Value>` elements:

```
<LifecycleConfiguration>
    <Rule>
        <Status>Enabled</Status>
        <Filter>
           <Tag>
              <Key>key</Key>
              <Value>value</Value>
           </Tag>
        </Filter>
    </Rule>
</LifecycleConfiguration>
```

Both prefixes and one or more tags

In a lifecycle rule, you can specify a filter based on both the key prefix and one or more tags. They must be wrapped in the `<And>` element. A filter can have only one prefix, and zero or more tags:

```
<LifecycleConfiguration>
    <Rule>
        <Status>Enabled</Status>
        <Filter>
          <And>
             <Prefix>key-prefix</Prefix>
             <Tag>
                <Key>key1</Key>
                <Value>value1</Value>
             </Tag>
             <Tag>
                <Key>key2</Key>
                <Value>value2</Value>
             </Tag>
              ...
          </And>
        </Filter>
```

```
    </Rule>
</LifecycleConfiguration>
```

Related information

S3 deletes a bucket lifecycle
S3 GET bucket lifecycle
S3 create or replace a bucket lifecycle

S3 GET bucket lifecycle

To get a bucket lifecycle, use `GET` and specify a destination bucket.

Syntax

```
GET /BUCKET?lifecycle HTTP/1.1
Host: cname.domain.com

Authorization: AWS _ACCESS_KEY_:_HASH_OF_HEADER_AND_SECRET_
```

Request Headers

For more information about request headers, see S3 common request headers.

Response

The response contains the bucket lifecycle and its elements.

S3 create or replace a bucket lifecycle

To create or replace a bucket lifecycle, use `PUT` and specify a destination bucket and a lifecycle configuration. The Ceph Object Gateway only supports a subset of the S3 lifecycle functionality.

Syntax

```
PUT /_BUCKET_?lifecycle HTTP/1.1
Host: cname.domain.com

Authorization: AWS _ACCESS_KEY_:_HASH_OF_HEADER_AND_SECRET_
<LifecycleConfiguration>
  <Rule>
    <Expiration>
      <Days>10</Days>
    </Expiration>
  </Rule>
    ...
  <Rule>
  </Rule>
</LifecycleConfiguration>
```

Request Headers

`content-md5`

Description

A base64 encoded MD-5 hash of the message

Valid Values

String No defaults or constraints.

Required

No

Reference

- For more information about Amazon S3 common request headers, see S3 common request headers.
- For more information about Amazon S3 bucket lifecycles, see S3 bucket lifecycles.

S3 delete a bucket lifecycle

To delete a bucket lifecycle, use `DELETE` and specify a destination bucket.

Syntax

```
DELETE /_BUCKET_?lifecycle HTTP/1.1
Host: cname.domain.com

Authorization: AWS _ACCESS_KEY_:_HASH_OF_HEADER_AND_SECRET_
```

Request Headers

The request does not contain any special elements.

Response

The response returns common response status.

Reference

- For more information about Amazon S3 common request headers, see S3 common request headers.
- For more information about Amazon S3 common response status codes, see S3 common response status codes.

Bucket access control

Use these S3 API operations to manage access control lists, CORS rules, Block Public Access settings, and replication configuration on a Ceph Object Gateway bucket.

S3 get bucket access control lists

Retrieves the bucket access control list. The user needs to be the bucket owner or to have been granted `READ_ACP` permission on the bucket.

Add the `acl` subresource to the bucket request.

Syntax

```
GET /BUCKET?acl HTTP/1.1
Host: cname.domain.com

Authorization: AWS ACCESS_KEY:HASH_OF_HEADER_AND_SECRET
```

Response Entities

`AccessControlPolicy`

Description

A container for the response.

Type

Container

`AccessControlList`

Description

A container for the ACL information.

Type

Container

`Owner`

Description

A container for the bucket owner's `ID` and `DisplayName`.

Type

Container

`ID`

Description

The bucket owner's ID.

Type

String

`DisplayName`

Description

The bucket owner's display name.

Type

String

`Grant`

Description

A container for `Grantee` and `Permission`.

Type

Container

`Grantee`

Description

A container for the `DisplayName` and `ID` of the user receiving a grant of permission.

Type

Container

`Permission`

Description

The permission given to the `Grantee` bucket.

Type

String

S3 put bucket Access Control Lists

Sets an access control to an existing bucket. The user needs to be the bucket owner or to have been granted `WRITE_ACP` permission on the bucket.

Add the `acl` subresource to the bucket request.

Syntax

```
PUT /_BUCKET_?acl HTTP/1.1
```

Request Entities S3 list multipart uploads

`AccessControlList`

Description

A container for the ACL information.

Type

Container

`Owner`

Description

A container for the bucket owner's `ID` and `DisplayName`.

Type

Container

`ID`

Description

The bucket owner's ID.

Type

String

`DisplayName`

Description

The bucket owner's display name.

Type

String

`Grant`

Description

A container for `Grantee` and `Permission`.

Type

Container

`Grantee`

Description

A container for the `DisplayName` and `ID` of the user receiving a grant of permission.

Type

Container

`Permission`

Description

The permission given to the `Grantee` bucket.

Type

String

S3 get bucket cors

Retrieves the cors configuration information set for the bucket. The user needs to be the bucket owner or to have been granted `READ_ACP` permission on the bucket.

Add the `cors` subresource to the bucket request.

Syntax

```
GET /BUCKET?cors HTTP/1.1
Host: cname.domain.com

Authorization: AWS ACCESS_KEY:HASH_OF_HEADER_AND_SECRET
```

S3 put bucket cors

Sets the cors configuration for the bucket. The user needs to be the bucket owner or to have been granted `READ_ACP` permission on the bucket.

Add the `cors` subresource to the bucket request.

Syntax

```
PUT /BUCKET?cors HTTP/1.1
Host: cname.domain.com

Authorization: AWS ACCESS_KEY:HASH_OF_HEADER_AND_SECRET
```

S3 delete a bucket cors

Deletes the cors configuration information set for the bucket. The user needs to be the bucket owner or to have been granted `READ_ACP` permission on the bucket.

Add the `cors` subresource to the bucket request.

Syntax

```
DELETE /BUCKET?cors HTTP/1.1
Host: cname.domain.com

Authorization: AWS ACCESS_KEY:HASH_OF_HEADER_AND_SECRET
```

S3 Block Public Access

You can use the S3 Block Public Access feature to set buckets and users to help you manage public access to Red Hat Ceph Storage object storage S3 resources.

Using this feature, bucket policies, access point policies, and object permissions can be overridden to limit public access. By default, new buckets, access points, and objects do not allow public access.

The S3 API in the Ceph Object Gateway supports a subset of the AWS public access settings:

`BlockPublicPolicy`

This defines the setting to allow users to manage access point and bucket policies. This setting does not allow the users to publicly share the bucket or the objects it contains. Existing access point and bucket policies are not affected by enabling this setting. Setting this option to `TRUE` causes the Ceph Object Gateway:

- To reject calls to PUT Bucket policy if the specified bucket policy allows public access.
- To reject calls to PUT access point policy for all of the bucket's same-account access points.

> **Important:** Apply this setting at the user level so that users cannot alter a specific bucket's block public access setting.

> **Note:** The `TRUE` setting only works if the specified policy allows public access.

`RestrictPublicBuckets`

This defines the setting to restrict access to a bucket or access point with public policy.
The restriction applies to only authorized users within the bucket owner's account within the bucket owner's account and access point owner's account. This blocks cross-account access to the access point or bucket, except for the cases specified, while still allowing users within the account to manage the access points or buckets.

Enabling this setting does not affect existing access point or bucket policies. It only defines that Ceph Object Gateway blocks public and cross-account access derived from any public access point or bucket policy, including non-public delegation to specific accounts.

`BlockPublicAcls`

When this setting is set to `TRUE`, requests that include public access control lists (ACLs) are blocked. For example, operations such as `PutBucketAcl`, `PutObjectAcl`, and `PutObject` fail if the request specifies a public ACL.

This setting prevents any new ACLs from being created or modified if they would grant public access. It does not affect existing public ACLs, so objects that are already public via an ACL remain public and accessible via `GetObject` requests.

> **Note:** This setting affects only requests that include public ACLs and does not remove or alter existing ACLs on buckets or objects.

`IgnorePublicAcls`

When this setting is set to `TRUE`, all public access control lists (ACLs) applied to buckets and objects are ignored when evaluating access permissions.
This setting is the one that addresses existing public ACLs. When this setting is set to `TRUE`, all public access control lists (ACLs) applied to buckets and objects are ignored while evaluating access permissions.

Existing ACLs remain unchanged and new public ACLs can still be set. This setting prevents public access granted through ACLs by ignoring them.

> **Note:** This setting affects how access permissions are evaluated but does not modify or remove existing ACLs.

Bucket policies are assumed to be public unless defined otherwise. To block public access a bucket policy must give access only to fixed values for one or more of the following:

> **Note:** A fixed value does not contain a wildcard (`*`) or an AWS Identity and Access Management Policy Variable.

- An AWS principal, user, role, or service principal
- A set of Classless Inter-Domain Routings (CIDRs), using `aws:SourceIp`
- `aws:SourceArn`
- `aws:SourceVpc`
- `aws:SourceVpce`
- `aws:SourceOwner`
- `aws:SourceAccount`
- `s3:x-amz-server-side-encryption-aws-kms-key-id`
- `aws:userid` outside the pattern `ARIOLEID:*`
- `s3:DataAccessPointArn`

  > **Note:** When used in a bucket policy, this value can contain a wildcard for the access point name without rendering the policy public, as long as the account ID is fixed.

- `s3:DataAccessPointPointAccount`

Example of a policy that is considered public

```
{
        "Principal": "*",
        "Resource": "*",
        "Action": "s3:PutObject",
        "Effect": "Allow",
        "Condition": { "StringLike": {"aws:SourceVpc": "vpc-*"}}
    }
```

To make a policy non-public, include any of the condition keys with a fixed value.

Example of a policy that is considered non-public

```
{
        "Principal": "*",
        "Resource": "*",
        "Action": "s3:PutObject",
        "Effect": "Allow",
        "Condition": {"StringEquals": {"aws:SourceVpc": "vpc-91237329"}}
    }
```

For more information, see the following:

- S3 GET PublicAccessBlock
- S3 PUT PublicAccessBlock
- S3 delete PublicAccessBlock

Related information

Blocking public access to your Amazon S3 storage

S3 get bucket replication

The get bucket replication API returns the replication configuration of a bucket.

```
GET /BUCKET_NAME?replication HTTP/1.1
```

Reference

- For more information about this API call, see S3 API.

S3 put bucket replication

The put bucket replication API configures replication configuration for a bucket or replaces an existing one.

```
PUT /BUCKET_NAME?replication HTTP/1.1
```

Reference

- For more information about this API call, see S3 API.

S3 delete bucket replication

The delete bucket replication API deletes the replication configuration from a bucket.

```
DELETE /BUCKET_NAME?website-configuration=HTTP/1.1
```

Reference

- For more information about this API call, see S3 API.

Bucket ownership and tagging

Use these S3 API operations to manage bucket tags and object ownership settings on a Ceph Object Gateway bucket.

Managing bucket tags

Add, view, update, and remove key-value bucket tags on an Red Hat Ceph Storage Object Gateway S3 bucket by using the AWS CLI.

Before you begin

Ensure that the AWS CLI is installed and configured on a host with network access and appropriate permissions for the Ceph Object Gateway endpoint.

About this task

Bucket tags are key-value pairs attached to an S3 bucket to categorize, organize, and track storage resources. You can perform any of the following tagging management operations:

> **Important:** Applying a new bucket tagging configuration replaces the entire existing tag set on the bucket.

Adding tags to a bucket

Add one or more key-value tags to an existing Ceph Object Gateway bucket.

Procedure

- Run the **put-bucket-tagging** command with the desired tag set.

```
aws --profile ceph --endpoint-url https://rgw.example.com s3api put-bucket-tagging \
  --bucket finance-data \
--tagging '
    "TagSet": [
      {
        "Key": "environment",
        "Value": "production"
      },
      {
        "Key": "department",
        "Value": "finance"
      }
    ]'
```

Viewing bucket tags

Display the active tag set configured on a bucket.

Procedure

- Run the **get-bucket-tagging** command to display the active tag set.

```
aws --profile ceph --endpoint-url https://rgw.example.com s3api get-bucket-tagging --
bucket finance-data
```

Updating bucket tags

Update the tag configuration by submitting the complete replacement tag set.

Procedure

- Run the **put-bucket-tagging** command with the complete replacement tag set.

```
aws --profile ceph --endpoint-url https://rgw.example.com s3api put-bucket-tagging \
  --bucket finance-data \
  --tagging '{
    "TagSet": [
      {
        "Key": "environment",
        "Value": "test"
      }
    ]
  }'
```

Removing bucket tags

Remove all tags configured on a bucket.

Procedure

- Run the **`delete-bucket-tagging`** command.

```
aws --profile ceph --endpoint-url https://rgw.example.com s3api delete-bucket-tagging
--bucket finance-data
```

Bucket website and logging

Use these S3 API operations to configure static website hosting and audit logging on a Ceph Object Gateway bucket.

S3 put bucket website

The put bucket website API sets the configuration of the website that is specified in the `website` subresource. To configure a bucket as a website, the `website` subresource can be added on the bucket.

> **Note:** `Put` operation requires `S3:PutBucketWebsite` permission. By default, only the bucket owner can configure the website attached to a bucket.

Syntax

```
PUT /BUCKET?website-configuration=HTTP/1.1
```

Example

```
PUT /testbucket?website-configuration=HTTP/1.1
```

Reference

- For more information about this API call, see S3 API.

S3 get bucket website

The get bucket website API retrieves the configuration of the website that is specified in the `website` subresource.

> **Note:** `Get` operation requires the `S3:GetBucketWebsite` permission. By default, only the bucket owner can read the bucket website configuration.

Syntax

```
GET /BUCKET?website-configuration=HTTP/1.1
```

Example

```
GET /testbucket?website-configuration=HTTP/1.1
```

Reference

- For more information about this API call, see S3 API.

S3 delete bucket website

The delete bucket website API removes the website configuration for a bucket.

Syntax

```
DELETE /BUCKET?website-configuration=HTTP/1.1
```

Example

```
DELETE /testbucket?website-configuration=HTTP/1.1
```

Reference

– For more information about this API call, see S3 API.

S3 enable bucket logging

Enables bucket logging for a bucket. Use this operation to configure the target log bucket, prefix, and logging type (Standard or Journal).

Syntax

```
PUT /_BUCKET_?logging HTTP/1.1
```

Request entities

Parameters are XML-encoded in the body of the request, in the following format:

```
<BucketLoggingStatus xmlns="http://s3.amazonaws.com/doc/2006-03-01/">
  <LoggingEnabled>
    <TargetBucket>string</TargetBucket>
    <TargetGrants>
      <Grant>
        <Grantee>
          <DisplayName>string</DisplayName>
          <EmailAddress>string</EmailAddress>
          <ID>string</ID>
          <xsi:type>string</xsi:type>
          <URI>string</URI>
        </Grantee>
        <Permission>string</Permission>
      </Grant>
    </TargetGrants>
    <TargetObjectKeyFormat>
      <PartitionedPrefix>
        <PartitionDateSource>DeliveryTime|EventTime</PartitionDateSource>
      </PartitionedPrefix>
      <SimplePrefix/>
    </TargetObjectKeyFormat>
    <TargetPrefix>string</TargetPrefix>
    <LoggingType>Standard|Journal</LoggingType>
    <ObjectRollTime>integer</ObjectRollTime>
    <Filter>
      <S3Key>
        <FilterRule>
          <Name>suffix/prefix/regex</Name>
          <Value></Value>
        </FilterRule>
      </S3Key>
    </Filter>
  </LoggingEnabled>
</BucketLoggingStatus>
```

Request Parameters

Following table lists the request parameters:

Name	Type	Description	Required
BucketLoggingStatus	Container	Enables or disables logging configuration for the bucket.	Yes

Name	Type	Description	Required
LoggingEnabled	Container	Contains the logging configuration details for the bucket.	Yes
TargetBucket	String	Specifies the log bucket where access logs are stored. The log bucket cannot have its own logging enabled.	Yes
TargetGrants	Container	Not supported. The log bucket owner is automatically the owner of the log objects.	No
TargetObjectKeyFormat	Container	Defines the format of the log object key. Can include `PartitionedPrefix` or `SimplePrefix`.	No
PartitionedPrefix	Container	Indicates a partitioned log object key format. The `PartitionDateSource` field is fixed as `DeliveryTime`.	No
SimplePrefix	Container	Indicates a simple (default) log object key format.	No
TargetPrefix	String	Prefix used for log object names. Useful to distinguish logs from multiple source buckets.	No
LoggingType	String	Logging mode. Valid values: `Standard` (default) or `Journal`. Journal mode logs operations before completion.	No
ObjectRollTime	Integer	Time in seconds after which a new log object is created. Default: 3600 (1 hour).	No
Filter	Container	Optional filter for selecting log entries based on object key or metadata.	No

Response entities

The response is XML encoded in the body of the request, only if a configuration change triggers flushing of the current logging object. In this case it will return the name of the flushed logging object in following format:

```
<PostBucketLoggingOutput xmlns="http://s3.amazonaws.com/doc/2006-03-01/">
  <FlushedLoggingObject>string</FlushedLoggingObject>
</PostBucketLoggingOutput>
```

HTTP Response

The following HTTP response might be returned:

HTTP Response

`400`

Status Code

`MalformedXML`

Description

The XML is not well-formed.

HTTP Response

`400`

Status Code

`InvalidArgument`

Description

Missing mandatory value or invalid value.

HTTP Response

`404`

Status Code

`NoSuchBucket`

Description

The bucket does not exist.

S3 disable bucket logging

Disables bucket logging for a bucket. Use this operation to remove the logging configuration from a source bucket.

Syntax

```
PUT /{bucket}?logging HTTP/1.1
```

Request entities

Parameters are XML encoded in the body of the request, in the following format:

```
<BucketLoggingStatus xmlns="http://s3.amazonaws.com/doc/2006-03-01/">
</BucketLoggingStatus>
```

HTTP Response

`404`

Status Code

`NoSuchBucket`

Description

The bucket does not exist.

S3 get bucket logging configuration

Retrieves the current bucket logging configuration, including the target bucket, prefix, and logging type.

Syntax

```
GET /{bucket}?logging HTTP/1.1
```

Response entities

The response header contains `Last-Modified` date/time of the logging configuration. Logging configuration is XML encoded in the body of the response, in the following format:

```
<BucketLoggingStatus xmlns="http://s3.amazonaws.com/doc/2006-03-01/">
  <LoggingEnabled>
    <TargetBucket>string</TargetBucket>
    <TargetGrants>
      <Grant>
        <Grantee>
          <DisplayName>string</DisplayName>
          <EmailAddress>string</EmailAddress>
          <ID>string</ID>
          <xsi:type>string</xsi:type>
          <URI>string</URI>
        </Grantee>
        <Permission>string</Permission>
      </Grant>
    </TargetGrants>
    <TargetObjectKeyFormat>
      <PartitionedPrefix>
        <PartitionDateSource>DeliveryTime|EventTime</PartitionDateSource>
      </PartitionedPrefix>
      <SimplePrefix>
```

```
      </SimplePrefix>
    </TargetObjectKeyFormat>
    <TargetPrefix>string</TargetPrefix>
    <LoggingType>Standard|Journal</LoggingType>
    <ObjectRollTime>integer</ObjectRollTime>
    <Filter>
      <S3Key>
        <FilterRule>
          <Name>suffix/prefix/regex</Name>
          <Value></Value>
        </FilterRule>
      </S3Key>
    </Filter>
  </LoggingEnabled>
</BucketLoggingStatus>
```

HTTP Response

```
404
```

Status Code

```
NoSuchBucket
```

Description

The bucket does not exist.

Using `radosgw-admin` to view bucket logging configuration

To view the same logging configuration that the S3 API returns, use the **`radosgw-admin`** command.

Following is an example for viewing logging configuration on the source bucket

```
[root@ceph14 ~]# radosgw-admin bucket logging info --bucket src-bkt1
{
    "bucketLoggingStatus": {
        "loggingEnabled": {
            "targetBucket": "dest-bkt1",
            "targetPrefix": "src-bkt1-logs-",
            "objectRollTime": 75,
            "loggingType": "Journal",
            "recordsBatchSize": 0,
            "targetObjectKeyFormat": {
                "simplePrefix": {}
            }
        }
    }
}
[root@ceph14 ~]## View logging configuration on the target (destination) bucket
[root@ceph14 ~]# radosgw-admin bucket logging info --bucket dest-bkt1
[
    {
        "name": "src-bkt1",
        "marker": "870dab95-5bb1-42b1-b50a-1a6b4a7c5e77.218451.1",
        "bucket_id": "870dab95-5bb1-42b1-b50a-1a6b4a7c5e77.218451.1",
        "tenant": "",
        "explicit_placement": {
            "data_pool": "",
            "data_extra_pool": "",
            "index_pool": ""
        }
    }
]
[root@ceph14 ~]#
```

Following is an example for view logging configuration on the target (destination) bucket.

```
[root@ceph14 ~]# radosgw-admin bucket logging info --bucket dest-bkt1
[
    {
        "name": "src-bkt1",
        "marker": "870dab95-5bb1-42b1-b50a-1a6b4a7c5e77.218451.1",
        "bucket_id": "870dab95-5bb1-42b1-b50a-1a6b4a7c5e77.218451.1",
        "tenant": "",
        "explicit_placement": {
            "data_pool": "",
            "data_extra_pool": "",
            "index_pool": ""
        }
    }
]
[root@ceph14 ~]#
```

S3 flush bucket logging

Flushes pending logging objects for a bucket. Use this operation to immediately commit log data to the target log bucket. If no log records exist to flush, then the flush operation flushes an empty log object to the target bucket.

Syntax

```
POST /{bucket}?logging HTTP/1.1
```

Response entities

The response body is XML encoded in the following format. It returns the name of the object that was flushed:

```
<PostBucketLoggingOutput xmlns="http://s3.amazonaws.com/doc/2006-03-01/">
  <FlushedLoggingObject>string</FlushedLoggingObject>
</PostBucketLoggingOutput>
```

HTTP Response

404

Status Code

NoSuchBucket

Description

The bucket does not exist.

Using **radosgw-admin**
Use the **radosgw-admin** command to immediately flush and commit pending logging objects for the bucket.

The following example shows how to flush pending logging objects by using the **radosgw-admin** command.

```
# Flush pending logging objects for the source bucket
[root@ceph14 ~]# radosgw-admin bucket logging flush --bucket src-bkt1
flushed pending logging object 'src-bkt1-logs-2025-11-18-20-43-19-0000000011AEI402' to
target bucket 'dest-bkt1'
[root@ceph14 ~]#
```

S3 object operations

Perform object operations with the Amazon S3 application programming interface (API) through the Ceph Object Gateway.

To use these APIs a running Red Hat Ceph Storage cluster and RESTful client are required.

"S3 functional operations and support status for objects" on page 1501 describes the Amazon S3 functional operations for objects, along with the function's support status.

Table 157: S3 functional operations and support status for objects

Feature	Status
Get Object	Supported
Head object	Supported

Feature	Status
Put Object Lock	Supported
Get Object Lock	Supported
Put Object Legal Hold	Supported
Get Object Legal Hold	Supported
Put Object Retention	Supported
Get Object Retention	Supported
Put Object Tagging	Supported
Get Object Tagging	Supported
Delete Object Tagging	Supported
Put Object	Supported
Delete Object	Supported
Delete Multiple Objects	Supported
Get Object ACLs	Supported
Put Object ACLs	Supported
Copy Object	Supported
Post Object	Supported
Options Object	Supported
Initiate Multipart Upload	Supported
Add a Part to a Multipart Upload	Supported
List Parts of a Multipart Upload	Supported
Assemble Multipart Upload	Supported
Copy Multipart Upload	Supported
Abort Multipart Upload	Supported
Multi-Tenancy	Supported

S3 select operations

As a developer, you can use the S3 select API for high-level analytic applications like Spark-SQL to improve latency and throughput.

There are three S3 select workflow - CSV, Apache Parquet (Parquet), and JSON that provide S3 select operations with CSV, Parquet, and JSON objects:

- A CSV file stores tabular data in plain text format. Each line of the file is a data record.
- Parquet is an open-source, column-oriented data file format designed for efficient data storage and retrieval. It provides highly efficient data compression and encoding schemes with enhanced performance to handle complex data in bulk. It can be used for single, multi, and archive site. Parquet enables the S3 select-engine to skip columns and chunks, thereby reducing IOPS dramatically (contrary to CSV and JSON format).
- JSON uses SQL statements to scan and extract information from JSON documents. It can be nested in various ways, such as within objects or arrays. These objects and arrays can be further nested within each other without any limitations.
- JSON is a format structure. The S3 select engine enables the use of SQL statements on top of the JSON format input data using the JSON reader, enabling the scanning of highly nested and complex JSON formatted data.

For example, a CSV, Parquet, or JSON S3 object with several gigabytes of data allows the user to extract a single column which is filtered by another column using the following query:

```
select customerid from s3Object where age>30 and age<65;
```

Currently, the S3 object must retrieve data from the Ceph OSD through the Ceph Object Gateway before filtering and extracting data. There is improved performance when the object is large and the query is more specific.

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- A RESTful client.
- A S3 user created with user access.

S3 select content from an object

The select object content API filters the content of an object through the structured query language (SQL).

Requests

In the request, you must specify the data serialization format as, comma-separated values (CSV) of the object to retrieve the specified content. Amazon Web Services (AWS) command-line interface (CLI) select object content uses the CSV format to parse object data into records and returns only the records specified in the query.

For more information and an example of the description of what should reside in the inventory object, see **Metadata collected by Inventory**, within the AWS Systems Manager User Guide.

> **Note:** You must specify the data serialization format for the response. You must have `s3:GetObject` permission for this operation.

```
POST /BUCKET/KEY?select&select-type=2 HTTP/1.1\r\n
```

For example,

```
POST /testbucket/sample1csv?select&select-type=2 HTTP/1.1\r\n
```

Table 158: Request entities

Request	Description	Type	Required
`Bucket`	The bucket to select object content from.	String	Yes
`Key`	The object key. Required minimum length of 1.	String	Yes
`SelectObjectContentRequest`	Root level tag for the select object content request parameters.	String	Yes
`Expression`	The expression that is used to query the object.	String	Yes
`ExpressionType`	The type of the provided expression for example SQL. Valid value: `SQL`	String	Yes
`InputSerialization`	Describes the format of the data in the object that is being queried.	String	Yes
`OutputSerialization`	Format of data returned in comma separator and new-line.	String	Yes

Responses

If the action is successful, the service sends back `HTTP 200` response. Data is returned in XML format by the service:

Table 159: Response entities

Response	Description	Type	Required
`Payload`	Root level tag for the payload parameters.	String	Yes
`Records`	The records event.	Base64-encoded binary data object	No
`Stats`	The stats event.	Long	No

Ceph Object Gateway supports the following response type:

```
{:event-type,records} {:content-type,application/octet-stream} {:message-type,event}
```

CSV requests

```
aws --endpoint-URL http://localhost:80 s3api select-object-content
 --bucket BUCKET NAME
 --expression-type SQL
 --input-serialization
 {"CSV": {"FieldDelimiter": "," , "QuoteCharacter": "\"" , "RecordDelimiter" : "\n" ,
"QuoteEscapeCharacter" : "\\" , "FileHeaderInfo": "USE" }, "CompressionType": "NONE"}
 --output-serialization {"CSV": {}}
 --key OBJECT_NAME.csv
 --expression "select count(0) from s3object where int(_1)<10;" output.csv
```

Note: The object names in `--key OBJECT_NAME` do not require `.csv`, `.json`, or `.parquet`.

For example,

```
aws --endpoint-url http://localhost:80 s3api select-object-content
 --bucket testbucket
 --expression-type 'SQL'
 --input-serialization
 '{"CSV": {"FieldDelimiter": "," , "QuoteCharacter": "\"" , "RecordDelimiter" : "\n" ,
"QuoteEscapeCharacter" : "\\" , "FileHeaderInfo": "USE" }, "CompressionType": "NONE"}'
 --output-serialization '{"CSV": {}}'
 --key testobject.csv
 --expression "select count(0) from s3object where int(_1)<10;" output.csv
```

Parquet requests

Note: The only supported output serialization is `.csv`.

```
aws --endpoint-url http://localhost:80 s3api select-object-content
 --bucket BUCKET NAME
 --expression-type 'SQL'
 --input-serialization
 '{"Parquet": {}, {"CompressionType": "NONE"}'
 --output-serialization '{"CSV": {}}}'
 --key OBJECT_NAME.parquet
 --expression "select count(0) from s3object where int(_1)<10;" output.csv
```

For example,

```
aws --endpoint-url http://localhost:80 s3api select-object-content
 --bucket testbucket
 --expression-type 'SQL'
 --input-serialization
 '{"Parquet":{}, "CompressionType": "NONE"}'
 --output-serialization '{"CSV": {}}'
 --key testobject.parquet
 --expression "select count(0) from s3object where int(_1)<10;" output.csv
```

JSON requests

```
aws --endpoint-url http://localhost:80 s3api select-object-content
 --bucket BUCKET_NAME
 --key OBJECT_NAME.json
 --expression-type 'SQL'
 --input-serialization
 '{"JSON": {"Type": "DOCUMENT"}, "CompressionType": "NONE"}'
 --output-serialization '{"CSV": {}}'
 --expression "select count(*) from S3Object[*];" /dev/stdout
```

For example,

```
aws s3api --endpoint-url http://localhost:80
select-object-content --bucket test
--key testobject.json
--expression-type 'SQL'
--input-serialization
'{"JSON": {"Type": "DOCUMENT"}, "CompressionType": "NONE"}'
--output-serialization '{"CSV": {}}'
--expression "select count(*) from S3Object[*];" /dev/stdout
```

BOTO3 requests

```
import pprint
import boto3
from botocore.exceptions import ClientError

def run_s3select(bucket,key,query,column_delim=",",row_delim="\n",quot_char='"',esc_char='\
\',csv_header_info="NONE"):

   s3 = boto3.client('s3',
       endpoint_url=endpoint,
       aws_access_key_id=access_key,
       region_name=region_name,
       aws_secret_access_key=secret_key)

   result = ""
   try:
       r = s3.select_object_content(
       Bucket=bucket,
       Key=key,
       ExpressionType='SQL',
       InputSerialization = {"CSV": {"RecordDelimiter" : row_delim, "FieldDelimiter" :
column_delim,"QuoteEscapeCharacter": esc_char, "QuoteCharacter": quot_char,
"FileHeaderInfo": csv_header_info}, "CompressionType": "NONE"},
       OutputSerialization = {"CSV": {}},
       Expression=query,
       RequestProgress = {"Enabled": progress})

   except ClientError as c:
       result += str(c)
       return result

   for event in r['Payload']:
            if 'Records' in event:
                result = ""
                records = event['Records']['Payload'].decode('utf-8')
                result += records
            if 'Progress' in event:
                print("progress")
                pprint.pprint(event['Progress'],width=1)
            if 'Stats' in event:
                print("Stats")
                pprint.pprint(event['Stats'],width=1)
            if 'End' in event:
                print("End")
                pprint.pprint(event['End'],width=1)
```

```
   return result



 run_s3select(
 "my_bucket",
 "my_csv_object",
 "select int(_1) as a1, int(_2) as a2 , (a1+a2) as a3 from s3object where a3>100 and
a3<300;")
```

Supported features

Currently, only part of the AWS s3 select command is supported. For a full list of supported features, see "AWS s3 select command supported features" on page 1506.

Table 160: AWS s3 select command supported features

Features	Details	Description	Example
Arithmetic operators	`^ * % / + - ( )`	N/A	`select (int(_1)+int(_2))*int(_9) from s3object;`
Arithmetic operators	`% modulo`	N/A	`select count(*) from s3object where cast(_1 as int)%2 = 0;`
Arithmetic operators	`^ power-of`	N/A	`select cast(2^10 as int) from s3object;`
Compare operators	`> < >= <= == !=`	N/A	`select _1,_2 from s3object where (int(_1)+int(_3))>int(_5);`
logical operator	`AND OR NOT`	N/A	`select count(*) from s3object where not (int(1)>123 and int(_5)<200);`
logical operator	`is null`	Returns true/false for null indication in expression	N/A
logical operator and NULL	`is not null`	Returns true/false for null indication in expression	N/A
logical operator and NULL	unknown state	Review null-handle and observe the results of logical operations with NULL. The query returns `0`.	`select count(*) from s3object where null and (3>2);`
Arithmetic operator with NULL	unknown state	Review null-handle and observe the results of binary operations with NULL. The query returns `0`.	`select count(*) from s3object where (null+1) and (3>2);`
Compare with NULL	unknown state	Review null-handle and observe results of compare operations with NULL. The query returns `0`.	`select count(*) from s3object where (null*1.5) != 3;`
missing column	unknown state	N/A	`select count(*) from s3object where _1 is null;`

Features	Details	Description	Example
projection column	Similar to if or then or else	N/A	`select case when (1+1==(2+1)*3) then 'case_1' when ((4*3)==(12)) then 'case_2' else 'case_else' end, age*2 from s3object;`
projection column	Similar to switch/case default	N/A	`select case cast(_1 as int) + 1 when 2 then “a” when 3 then “b” else “c” end from s3object;`
logical operator	N/A	`coalesce` returns first non-null argument	`select coalesce(nullif(5,5), nullif(1,1.0),age+12) from s3object;`
logical operator	N/A	`nullif` returns null in case both arguments are equal, or else the first one,`nullif(1,1)=NULL nullif(null,1)=NULL nullif(2,1)=2`	`select nullif(cast(_1 as int),cast(_2 as int)) from s3object;`
logical operator	N/A	`{expression} in ( .. {expression} ..)`	`select count(*) from s3object where 'ben' in (trim(_5),substring(_1,char_length(_1)-3,3),last_name);`
logical operator	N/A	`{expression} between {expression} and {expression}`	`select _1 from s3object where cast(_1 as int) between 800 and 900;; select count(*) from stdin where substring(_3,char_length(_3),1) between “x” and trim(_1) and substring(_3,char_length(_3)-1,1) = “:”;`
logical operator	N/A	`{expression} like {match-pattern}`	`select count(*) from s3object where first_name like '%de_'; select count(*) from s3object where _1 like "%a[r-s];`
casting operator	N/A	N/A	`select cast(123 as int)%2 from s3object;`
casting operator	N/A	N/A	`select cast(123.456 as float)%2 from s3object;`
casting operator	N/A	N/A	`select cast('ABC0-9' as string),cast(substr('ab12cd',3,2) as int)*4 from s3object;`

Features	Details	Description	Example
casting operator	N/A	N/A	`select cast(substring('publish on 2007-01-01',12,10) as timestamp) from s3object;`
non AWS casting operator	N/A	N/A	`select int(_1),int( 1.2 + 3.4) from s3object;`
non AWS casting operator	N/A	N/A	`select float(1.2) from s3object;`
non AWS casting operator	N/A	N/A	`select to_timestamp('1999-10-10T12:23:44Z') from s3object;`
Aggregation Function	`sun`	N/A	`select sum(int(_1)) from s3object;`
Aggregation Function	`avg`	N/A	`select avg(cast(_1 as float) + cast(_2 as int)) from s3object;`
Aggregation Function	`min`	N/A	`select avg(cast(_1 a float) + cast(_2 as int)) from s3object;`
Aggregation Function	`max`	N/A	`select max(float(_1)),min(int(_5)) from s3object;`
Aggregation Function	`count`	N/A	`select count(*) from s3object where (int(1)+int(_3))>int(_5);`
Timestamp Functions	`extract`	N/A	`select count(*) from s3object where extract(year from to_timestamp(_2)) > 1950 and extract(year from to_timestamp(_1)) < 1960;`
Timestamp Functions	`dateadd`	N/A	`select count(0) from s3object where date_diff(year,to_timestamp(_1),date_add(day,366,to_timestamp(_1))) = 1;`
Timestamp Functions	`datediff`	N/A	`select count(0) from s3object where date_diff(month,to_timestamp(_1),to_timestamp(_2)) = 2;`
Timestamp Functions	`utcnow`	N/A	`select count(0) from s3object where date_diff(hour,utcnow(),date_add(day,1,utcnow())) = 24`

Features	Details	Description	Example
Timestamp Functions	`to_string`	N/A	`select to_string( to_timestamp("2009-09-17T17:56:06.234567Z"), "yyyyMMdd-H:m:s") from s3object;`
String Functions	`substring`	N/A	`select count(0) from s3object where int(substring(_1,1,4))>1950 and int(substring(_1,1,4))<1960;`
String Functions	`substring`	substring with from negative number is valid considered as first	`select substring("123456789" from -4) from s3object;`
String Functions	`substring`	substring with from zero for out-of-bound number is valid just as (first,last)	`select substring("123456789" from 0 for 100) from s3object;`
String Functions	`trim`	N/A	`select trim(' foobar ') from s3object;`
String Functions	`trim`	N/A	`select trim(trailing from ' foobar ') from s3object;`
String Functions	`trim`	N/A	`select trim(leading from ' foobar ') from s3object;`
String Functions	`trim`	N/A	`select trim(both '12' from '1112211foobar22211122') from s3object;`
String Functions	`lower or upper`	N/A	`select lower('ABcD12#$e') from s3object;`
String Functions	`char_length, character_length`	N/A	`select count(*) from s3object where char_length(_3)=3;`
Complex queries	N/A	N/A	`select sum(cast(_1 as int)),max(cast(_3 as int)), substring('abcdefghijklm', (2-1)*3+sum(cast(_1 as int))/sum(cast(_1 as int))+1, (count() + count(0))/count(0)) from s3object;`
alias support	N/A	N/A	`select int(_1) as a1, int(_2) as a2 , (a1+a2) as a3 from s3object where a3>100 and a3<300;`

Related information

Amazon's S3 Select Object Content API

S3 supported select functions

Get to know the different S3 select supported functions.

S3 select supports the following functions:

- “Timestamp” on page 1510
- “Aggregation” on page 1512
- “String” on page 1512
- “NULL” on page 1513

For more information about S3 select functions, see **SelectObjectContent** within the Amazon Simple Storage Service (S3) API Reference section of AWS Documentation.

Timestamp

`to_timestamp` (string)
Description

Converts string to timestamp basic type. In the string format, any missing 'time' value is populated with zero; for missing month and day value, 1 is the default value. 'Timezone' is in format `+/-HH:mm` or `Z` , where the letter 'Z' indicates Coordinated Universal Time (UTC). Value of timezone can range between - 12:00 and +14:00.

Supported

Currently it can convert the following string formats into timestamp:

- `YYYY-MM-DDTHH:mm:ss.SSSSSS+/-HH:mm`
- `YYYY-MM-DDTHH:mm:ss.SSSSSSZ`
- `YYYY-MM-DDTHH:mm:ss+/-HH:mm`
- `YYYY-MM-DDTHH:mm:ssZ`
- `YYYY-MM-DDTHH:mm+/-HH:mm`
- `YYYY-MM-DDTHH:mmZ`
- `YYYY-MM-DDT`
- `YYYYT`

`to_string` (timestamp, format_pattern)
Description

Returns a string representation of the input timestamp in the given input string format.

Parameters

Table 161: `to_string` parameters

Format	Example	Description
yy	69	2-digit year
y	1969	4-digit year
yyyy	1969	Zero-padded 4-digit year
M	1	Month of the year
MM	01	Zero-padded month of the year
MMM	Jan	Abbreviated month of the year name
MMMM	January	Full month of the year name

Format	Example	Description
MMMMM	J	Month of the year first letter. **Note:** Not valid for use with the `to_timestamp` function.
d	2	Day of the month (1-31)
dd	02	Zero-padded day of the month (01-31)
a	AM	AM or PM of day
h	3	Hour of day (1-12)
hh	03	Zero-padded hour of the day (01-12)
H	3	Hour of the day (0-24)
HH	03	Zero-padded hour of the day (00-23)
m	4	Minute of the hour (0-59)
mm	04	Zero-padded minute of the hour (00-59)
s	5	Second of the minute (0-59)
ss	05	Zero padded second of the minute (00-59)
S	1	Fraction of the second (precision 0.1, range: 0.0-0.9)
SS	12	Fraction of the second (precision 0.01, range: 0.0-0.99)
SSS	123	Fraction of the second (precision: 0.01, range: 0.0-0.999).
SSSS	1234	Fraction of the second (precision: 0.001, range: 0.0-0.9999).
SSSSSS	123456	Fraction of the second (maximum precision: 1 nanosecond, range: 0.0-0.999999)
n	60000000	Nano of second
X	+07 or Z	Offset in hours, or “Z” if the offset is 0
XX or XXXX	+0700 or Z	Offset in hours and minutes or “Z” if the offset is 0
XXX or XXXXX	+07:00 or Z	Offset in hours and minutes or “Z” if the offset is 0
x	7	Offset in hours
xx or xxxx	700	Offset in hours and minutes
xxx or xxxxx	+07:00	Offset in hours and minutes

`extract` (date-part from timestamp)
Description

Returns integer according to date-part extract from input timestamp.

Supported

year, month, week, day, hour, minute, second, timezone_hour, timezone_minute.

`date_add` (date-part ,integer,timestamp)
Description

Returns timestamp, a calculation based on the results of input timestamp and date-part.

Supported

year, month, day, hour, minute, second.

`date_diff` (date-part,timestamp,timestamp)
Description

Return an integer, a calculated result of the difference between two timestamps according to date-part.

Supported

year, month, day, hour, minute, second.

`utcnow()`
Description

Return timestamp of current time.

Aggregation

`count()`
Description

Returns integers based on the number of rows that match a condition if there is one.

`sum(expression)`
Description

Returns a summary of expression on each row that matches a condition if there is one.

`avg(expression)`
Description

Returns an average expression on each row that matches a condition if there is one.

`max` (expression)
Description

Returns the maximal result for all expressions that match a condition if there is one.

`min` (expression)
Description

Returns the minimal result for all expressions that match a condition if there is one.

String

`substring` (string,from,for)
Description

Returns a string extract from the input string according to from, for inputs.

`Char_length`
Description

Returns a number of characters in string. `Character_length` also does the same.

`trim([[leading | trailing | both remove_chars] from] string )`
Description

Trims leading/trailing (or both) characters from the target string. The default value is a blank character.

`Upper\lower`
Description

Converts characters into uppercase or lowercase.

NULL

The NULL value indicates a missing or unknown that is NULL and that cannot produce a value on any arithmetic operations.

"The NULL use case" on page 1513 depicts various NULL use cases.

Table 162: The NULL use case

A is NULL	Result (NULL=UNKNOWN)
Not A	NULL
A or False	NULL
A or True	True
A or A	NULL
A and False	False
A and True	NULL
A and A	NULL

S3 alias programming construct

Alias programming construct is an essential part of the s3 select language because it enables better programming with objects that contain many columns or complex queries.

When you parse a statement with alias construct, it replaces the alias with a reference to the right projection column. On querying, the reference is evaluated like any other expression. Alias maintains result-cache. If an alias is used more than one time, the same expression is not evaluated. The same result is returned because the result from the cache is used. Currently, Red Hat supports the column alias.

There is a risk that self or cyclic reference might occur causing stack-overflow (endless-loop), for that concern upon evaluating an alias, it is validated for cyclic reference.

With each new row the cache is invalidated as the results may then differ.

Example

```
select int(_1) as a1, int(_2) as a2 , (a1+a2) as a3 from s3object where a3>100 and a3<300;
```

S3 parsing explained

The S3 select engine has parsers for CSV, Parquet, and JSON file formats. The parsers separate the commands into more processable components and are then attached to tags that define each component.

S3 CSV parsing

Use this information to get to know about how CSV parses S3-objects.

You can define the CSV definitions with input serialization uses these default values:

- Use {\n} for row-delimiter.
- Use {"} for quote.
- Use {\} for escape characters.

The csv-header-info is parsed upon USE appearing in the AWS-CLI; this is the first row in the input object containing the schema. Currently, output serialization and compression-type is not supported. The S3 select engine has a CSV parser which parses S3-objects:

- Each row ends with a row-delimiter.
- The field-separator separates the adjacent columns.
- The successive field separator defines the NULL column.
- The quote-character overrides the field-separator; that is, the field separator is any character between the quotes.

– The escape character disables any special character except the row delimiter.

The following are examples of CSV parsing rules:

Table 163: CSV parsing

Feature	Description	Input (Tokens)
`NULL`	Successive field delimiter	`,,1,,2, ==> {null}{null}{1}{null}{2}{null}`
`QUOTE`	The quote character overrides the field delimiter.	`11,22,"a,b,c,d",last ==> {11}{22}{"a,b,c,d"}{last}`
`Escape`	The escape character overrides the meta-character.	`11,22,str=\"abcd\"\,str2=\"123\",last ==> {11}{22}{str="abcd",str2="123"}{last}`
`row delimiter`	There is no closed quote; row delimiter is the closing line.	`11,22,a="str,44,55,66 ==> {11}{22}{a="str,44,55,66}`
`csv header info`	FileHeaderInfo tag	USE value means each token on the first line is the column-name; IGNORE value means to skip the first line.

Reference

– See Amazon's S3 Select Object Content API for more details.

S3 Parquet parsing

Apache Parquet is an open-source, columnar data file format designed for efficient data storage and retrieval. Use this information to get to know about how Apache Parquet parses S3-objects.

The S3 select engine's Parquet parser parses S3-objects as follows:

```
4-byte magic number "PAR1"
<Column 1 Chunk 1 + Column Metadata>
<Column 2 Chunk 1 + Column Metadata>
...
<Column N Chunk 1 + Column Metadata>
<Column 1 Chunk 2 + Column Metadata>
<Column 2 Chunk 2 + Column Metadata>
...
<Column N Chunk 2 + Column Metadata>
...
<Column 1 Chunk M + Column Metadata>
<Column 2 Chunk M + Column Metadata>
...
<Column N Chunk M + Column Metadata>
File Metadata
4-byte length in bytes of file metadata
4-byte magic number "PAR1"
```

– In the above example, there are N columns in this table, split into M row groups. The file metadata contains the locations of all the column metadata start locations.

– Metadata is written after the data to allow for single pass writing.

– All the column chunks can be found in the file metadata which should later be read sequentially.

– The format is explicitly designed to separate the metadata from the data. This allows splitting columns into multiple files, as well as having a single metadata file reference multiple parquet files.

S3 JSON parsing

Use this information to get to know about how JSON parses S3-objects.

The values in a JSON document can be nested within objects or arrays without limitations. When querying a specific value in a JSON document in the S3 select engine, the location of the value is specified via a path in the `SELECT` statement.

The generic structure of a JSON document does not have a row and column structure like CSV and Parquet. Instead, it is the SQL statement itself that defines the rows and columns when querying a JSON document.

The S3 select engine's JSON parser parses S3-objects as follows:

- The `FROM` clause in the `SELECT` statement defines the row boundaries
- A row in a JSON document is similar to how the row delimiter is used to define rows for CSV objects, and how row groups are used to define rows for Parquet objects
- Consider the following example:

```
{
"firstName": "Joe",
"lastName": "Jackson",
"gender": "male",
"age": "twenty",
"address":
  {
   "streetAddress": "101",
   "city": "San Diego",
   "state": "CA"
  },
"firstName": "Joe_2",
"lastName": "Jackson_2",
"gender": "male",
"age": 21,
"address":
 {
  "streetAddress": "101",
  "city": "San Diego",
  "state": "CA"
 },
"phoneNumbers":
 [
  { "type": "home1", "number": "734928_1","addr": 11 },
  { "type": "home2", "number": "734928_2","addr": 22 }
 ],
"key_after_array": "XXX",
"description" :
 {
  "main_desc" : "value_1",
  "second_desc" : "value_2"
 }
}

 # the from-clause define a single row.
 # _1 points to root object level.
 # _1.age appears twice in Documnet-row, the last value is used for the operation.
 query = "select
_1.firstname,_1.key_after_array,_1.age+4,_1.description.main_desc,_1.description.second
_desc from s3object[*];";
 expected_result = Joe_2,XXX,25,value_1,value_2


 # the from-clause points the phonenumbers array (it defines the _1)
 # each element in phoneNumbers array define a row.
 # in this case each element is an object contains 3 keys/values.
 # the query "can not access" values outside phonenumbers array, the query can access
only values appears on _1.phonenumbers path.
 query = "select cast(substring(_1.number,1,6) as int) *10 from
s3object[*].phonenumbers where _1.type='home2';";
 expected_result = 7349280
```

  - The statement instructs the reader to search for the path *aa.bb.cc* and defines the row boundaries based on the occurrence of this path.
  - A row begins when the reader encounters the path, and it ends when the reader exits the innermost part of the path, which in this case is the object *cc*.

S3 SQL limit operator

The SQL limit operator is used to limit the number of rows processed by the query.

Upon reaching the limit set by the user, the Ceph Object Gateway stops fetching additional chunks.

The following are examples of SQL limit operator for CSV, Parquet, and JSON.

Example of SQL limit operator for CSV

```
[cephuser@host ~]$ aws s3api --endpoint-url http://extensa027.ceph.rh.com:80 select-object-
content  --bucket bkt1 --key placement_data1 --expression-type 'SQL' --input-serialization
'{"CSV": {"FileHeaderInfo": "USE"}, "CompressionType": "NONE"}' --output-serialization
'{"CSV": {}}' --expression "select * from s3object limit 10;"

/dev/stdout
1,M,67.00,Others,91.00,Others,Commerce,58.00,Sci&Tech,No,55,Mkt&HR,58.8,Placed,270000
2,M,79.33,Central,78.33,Others,Science,77.48,Sci&Tech,Yes,86.5,Mkt&Fin,66.28,Placed,200000
3,M,65.00,Central,68.00,Central,Arts,64.00,Comm&Mgmt,No,75,Mkt&Fin,57.8,Placed,250000
4,M,56.00,Central,52.00,Central,Science,52.00,Sci&Tech,No,66,Mkt&HR,59.43,Not Placed,
5,M,85.80,Central,73.60,Central,Commerce,73.30,Comm&Mgmt,No,96.8,Mkt&Fin,55.5,Placed,425000
6,M,55.00,Others,49.80,Others,Science,67.25,Sci&Tech,Yes,55,Mkt&Fin,51.58,Not Placed,
7,F,46.00,Others,49.20,Others,Commerce,79.00,Comm&Mgmt,No,74.28,Mkt&Fin,53.29,Not Placed,
8,M,82.00,Central,64.00,Central,Science,66.00,Sci&Tech,Yes,67,Mkt&Fin,62.14,Placed,252000
9,M,73.00,Central,79.00,Central,Commerce,72.00,Comm&Mgmt,No,91.34,Mkt&Fin,61.29,Placed,2310
00
10,M,58.00,Central,70.00,Central,Commerce,61.00,Comm&Mgmt,No,54,Mkt&Fin,52.21,Not Placed,
```

Example of SQL limit operator for Parquet

```
[cephuser@extensa022 ~]$ aws s3api --endpoint-url http://extensa027.ceph.rh.com:80 select-
object-content  --bucket parquetbkt2 --key smallfile --expression-type 'SQL' --input-
serialization '{"Parquet": {}, "CompressionType": "NONE"}' --output-serialization '{"CSV":
{}}'  --expression "select * from s3object limit 5;"

/dev/stdout
4819,90340852.424672872,exile,0
4638,95603700.561133489,contradiction,1
1667,99440023.596563846,slam,2
7113,33213151.245887276,publisher,3
2872,43556628.954562038,demonstrate,4
```

Example of SQL limit operator for JSON

```
[cephuser@extensa027 ~]$ aws s3api --endpoint-url http://extensa027.ceph.rh.com:80 select-
object-content  --bucket bkt1 --key books_data.json --expression-type 'SQL' --input-
serialization '{"JSON": {"Type": "DOCUMENT"}, "CompressionType": "NONE"}' --output-
serialization '{"CSV": {}}' --expression "select * from s3object[*];"

/dev/stdout
id. : 1091
authors.name. : Makoto Satoh
authors.org. : Shinshu University
authors.id. : 2312688602
authors.name. : Ryo Muramatsu
authors.org. : Shinshu University
authors.id. : 2482909946
title. : Preliminary Design of a Network Protocol Learning Tool Based on the Comprehension
of High School Students: Design by an Empirical Study Using a Simple Mind Map
year. : 2013
#=== 0 ===#
id. : 1388
authors.name. : Mizue Kayama
authors.org. : Shinshu University
authors.id. : 2128134587
authors.name. : Pranava K. Jha
authors.id. : 2718958994
authors.name. : Kazunori Itoh
authors.org. : Shinshu University
authors.id. : 2101782692
title. : Further Results on Independence in Direct-Product Graphs.
year. : 2000
#=== 1 ===#
id. : 5781
authors.name. : Jovan Dj. Golic
authors.id. : 1237859792
authors.name. : Guglielmo Morgari
authors.id. : 220887178
title. : Vectorial fast correlation attacks.
year. : 2004
#=== 2 ===#
id. : 6522
authors.name. : Güzin Ulutas
authors.org. : Karadeniz Technical Univ.
authors.id. : 2022192081
authors.name. : Mustafa Ulutas
authors.org. : Karadeniz Technical Univ.
authors.id. : 2023460672
authors.name. : Vasif V. Nabiyev
authors.org. : Karadeniz Technical Univ.
authors.id. : 2174205032
title. : Improved Secret Image Sharing Method By Encoding Shared Values With Authentication
Bits
year. : 2011
#=== 3 ===#
```

Integrating Ceph Object Gateway with Trino

Integrate the Ceph Object Gateway with Trino, an important utility that enables the user to run SQL queries 9x faster on S3 objects.

Before you begin

- A running cluster with Ceph Object Gateway installed.
- Docker or Podman installed.
- Buckets created.
- Objects are uploaded.

About this task

You can integrate Ceph Object Gateway with Trino for S3 select operations.

The following are some benefits of using Trino:

- Trino is a complete SQL engine.

- Pushes down S3 select requests wherein the Trino engine identifies part of the SQL statement that is cost effective to run on the server-side.
- Uses the optimization rules of Ceph/S3select to enhance performance.
- Leverages scalability and divides the original object into multiple equal parts, performs S3 select requests, and merges the request.

Important: If the `s3select` syntax does not work while querying through trino, use the SQL syntax.

Procedure

1. Deploy Trino and hive.
   a. Clone the repository.

```
[cephuser@host01 ~]$ git clone https://github.com/ceph/s3select.git
[cephuser@host01 ~]$ cd s3select
```

   b. Modify the `hms_trino.yaml` file with S3 endpoint, access key, and secret key.

```
[cephuser@host01 s3select]$ cat container/trino/hms_trino.yaml
version: '3'
services:
  hms:
    image: galsl/hms:dev
    container_name: hms
    environment:
      # S3_ENDPOINT the CEPH/RGW end-point-url
      - S3_ENDPOINT=http://rgw_ip:port
      - S3_ACCESS_KEY=abc
      - S3_SECRET_KEY=abc
    # the container starts with booting the hive metastore
    command: sh -c '. ~/.bashrc; start_hive_metastore'
    ports:
      - 9083:9083
    networks:
      - trino_hms

  trino:
    image: trinodb/trino:405
    container_name: trino
    volumes:
      # the trino directory contains the necessary configuration
      - ./trino:/etc/trino
    ports:
      - 8080:8080
    networks:
      - trino_hms

networks:
  trino_hm
```

   c. Modify the `hive.properties` file with S3 endpoint, access key, and secret key.

```
[cephuser@host01 s3select]$ cat container/trino/trino/catalog/hive.properties
connector.name=hive
hive.metastore.uri=thrift://hms:9083

#hive.metastore.warehouse.dir=s3a://hive/

hive.allow-drop-table=true
hive.allow-rename-table=true
hive.allow-add-column=true
hive.allow-drop-column=true
hive.allow-rename-column=true

hive.non-managed-table-writes-enabled=true
hive.s3select-pushdown.enabled=true
hive.s3.aws-access-key=abc
hive.s3.aws-secret-key=abc

# should modify per s3-endpoint-url
hive.s3.endpoint=http://rgw_ip:port
```

```
#hive.s3.max-connections=1
#hive.s3select-pushdown.max-connections=1

hive.s3.connect-timeout=100s
hive.s3.socket-timeout=100s
hive.max-splits-per-second=10000
hive.max-split-size=128MB
```

2. Start a Trino container to integrate Ceph Object Gateway.

```
[cephuser@host01 s3select]$ sudo docker compose -f ./container/trino/hms_trino.yaml up
-d
```

3. Verify integration.

```
[cephuser@host01 s3select]$ sudo docker exec -it trino /bin/bash
trino@66f753905e82:/$ trino
trino> create schema hive.csvbkt1schema;
trino> create table hive.csvbkt1schema.polariondatacsv(c1 varchar,c2 varchar, c3
varchar, c4 varchar, c5 varchar, c6 varchar, c7 varchar, c8 varchar, c9 varchar) WITH
( external_location = 's3a://csvbkt1/',format = 'CSV');
trino> select * from hive.csvbkt1schema.polariondatacsv;
```

Note: The external location must point to the bucket name or a directory, and not the end file.

Object operations

Use these S3 API operations to add, retrieve, copy, and delete objects in a Ceph Object Gateway bucket.

S3 add an object to a bucket

Use the S3 **PUT** command to add an object to a bucket, or the **POST** command to add an object using an HTML form. Write permissions on the bucket are required.

Adds an object to a bucket. You must have write permissions on the bucket to perform this operation.

```
PUT /BUCKET/OBJECT HTTP/1.1
```

Request headers

"PUT request headers" on page 1519 lists the supported request headers.

Table 164: PUT request headers

Header	Description	Valid values	Required
content-md5	A base64-encoded MD5 hash of the message.	String. No defaults or constraints.	No
content-type	A standard MIME type.	Any MIME type. Default: `binary/octet-stream`.	No
x-amz-meta-*	User metadata stored with the object.	A string up to 8 KB. No defaults.	No
x-amz-acl	A canned ACL.	`private` | `public-read` | `public-read-write` | `authenticated-read`	No

Header	Description	Valid values	Required
if-match	Uploads the object only if the current object's ETag matches the specified ETag value. If the ETag does not match, the request fails with `HTTP 412 (Precondition Failed)`. If multiple requests attempt to modify the same object concurrently, the server can return `HTTP 409 (Conflict)`.	Entity Tag (ETag)	No
if-none-match	Uploads the object only if the specified ETag does not match the current object's ETag. If the ETag matches, the request fails with `HTTP 412 (Precondition Failed)`.	Entity Tag (ETag)	No

Note: Conditional headers are not supported for multipart upload initialization requests.

Response headers

"PUT response headers" on page 1520 lists the supported response headers.

Table 165: PUT response headers

Response header	Description
x-amz-version-id	Returns the version ID or null.

Adding an object using HTML forms

Adds an object to a bucket using an HTML form. You must have write permissions on the bucket to perform this operation.

```
POST /BUCKET/OBJECT HTTP/1.1
```

S3 get an object from a bucket

Use the S3 **GET** command to retrieve an object from a bucket.

Retrieve an object from a bucket.

```
GET /BUCKET/OBJECT HTTP/1.1
```

Add the **versionId** subresource to retrieve a particular version of the object.

```
GET /BUCKET/OBJECT?versionId=VERSION_ID HTTP/1.1
```

Request headers

"GET request headers" on page 1520 lists the supported request headers.

Table 166: GET request headers

Header	Description	Valid values	Required
partNumber	Part number of the object being read. This enables a ranged **GET** request for the specified part. Use this parameter to download only a part of an object.	A positive integer between 1 and 10,000.	No

Header	Description	Valid values	Required
range	The range of the object to retrieve. **Note:** Multiple ranges of data per GET request are not supported.	`Range:bytes=BEGINBYTE-ENDBYTE`	No
if-modified-since	Gets only if modified since the timestamp.	Timestamp	No
if-unmodified-since	Gets only if not modified since the timestamp.	Timestamp	No
if-match	Gets only if the object ETag does not match.	Entity Tag	No
if-none-match	Gets only if the object ETag matches.	Entity Tag	No

Example syntax with request headers:

```
GET /BUCKET/OBJECT?partNumber=PARTNUMBER&versionId=VERSION_ID HTTP/1.1
Host: Bucket.s3.amazonaws.com
If-Match: IFMATCH
If-Modified-Since: IFMODIFIEDSINCE
If-None-Match: IFNONEMATCH
If-Unmodified-Since: IFUNMODIFIEDSINCE
Range: RANGE
```

Response headers

"GET response headers" on page 1521 lists the supported response headers.

Table 167: GET response headers

Response header	Description
Content-Range	Data range. Only returned if the range header field was specified in the request.
x-amz-version-id	Returns the version ID or null.
x-rgw-replicated-from	Returns the source zone and any intermediate zones involved in an object's replication path within a Ceph multi-zone environment. This header is included in `GetObject` and `HeadObject` responses.
x-rgw-replicated-at	Returns a timestamp indicating when the object was replicated to its current location. Calculate the replication duration by using this header with the **Last-Modified** header.

Note: As of now, **x-rgw-replicated-from** and **x-rgw-replicated-at** are supported by client tools such as s3cmd or curl at the replicated zone. These tools can be used in addition to the **radosgw-admin** command for verification. There is a known issue [BZ-2312552] with the **radosgw-admin** object stat command where the **x-rgw-replicated-from** header key is missing.

S3 get object attributes

Use the S3 `GetObjectAttributes` API to retrieve the metadata of an object without returning the object's data.

`GetObjectAttributes` combines the functionality of `HeadObject` and `ListParts`. It provides all the information returned by these two calls in a single request, reducing the number of API calls needed.

```
GET /BUCKET/OBJECT?attributes&versionId=VERSION_ID
```

The **versionId** subresource retrieves a particular version of the object.

Request headers

The following example shows the syntax for a `GetObjectAttributes` request:

```
GET /BUCKET/OBJECT?attributes&versionId=VERSION_ID HTTP/1.1
Host: BUCKET.s3.amazonaws.com
x-amz-max-parts: MAX_PARTS
x-amz-part-number-marker: PART_NUMBER_MARKER
x-amz-server-side-encryption-customer-algorithm: SSE_CUSTOMER_ALGORITHM
x-amz-server-side-encryption-customer-key: SSE_CUSTOMER_KEY
x-amz-server-side-encryption-customer-key-MD5: SSE_CUSTOMER_KEY_MD5
x-amz-request-payer: REQUEST_PAYER
x-amz-expected-bucket-owner: EXPECTED_BUCKET_OWNER
x-amz-object-attributes: OBJECT_ATTRIBUTES
```

"GET request headers" on page 1522 lists the supported request headers.

Table 168: GET request headers

Request header	Description	Type / Valid values	Required
Bucket	The name of the bucket that contains the object.	String	Yes
Key	The object key.	String	Yes
versionId	The version ID used to reference a specific version of the object.	String	No
x-amz-max-parts	Sets the maximum number of parts to return.	String	No
x-amz-object-attributes	Specifies the fields at the root level to return in the response. Fields that you do not specify are not returned.	`ETag` | `Checksum` | `ObjectParts` | `StorageClass` | `ObjectSize`	Yes
x-amz-part-number-marker	Specifies the part after which listing begins. Only parts with higher part numbers are listed.	String	No

Response entities

The following example shows a `GetObjectAttributes` response:

```
HTTP/1.1 200
x-amz-delete-marker: DELETE_MARKER
Last-Modified: LAST_MODIFIED
x-amz-version-id: VERSION_ID
x-amz-request-charged: REQUEST_CHARGED
<?xml version="1.0" encoding="UTF-8"?>
<GetObjectAttributesOutput>
   <ETag>string</ETag>
   <Checksum>
      <ChecksumCRC32>string</ChecksumCRC32>
      <ChecksumCRC32C>string</ChecksumCRC32C>
      <ChecksumSHA1>string</ChecksumSHA1>
      <ChecksumSHA256>string</ChecksumSHA256>
   </Checksum>
   <ObjectParts>
      <IsTruncated>boolean</IsTruncated>
      <MaxParts>integer</MaxParts>
      <NextPartNumberMarker>integer</NextPartNumberMarker>
      <PartNumberMarker>integer</PartNumberMarker>
      <Part>
         <ChecksumCRC32>string</ChecksumCRC32>
         <ChecksumCRC32C>string</ChecksumCRC32C>
         <ChecksumSHA1>string</ChecksumSHA1>
         <ChecksumSHA256>string</ChecksumSHA256>
         <PartNumber>integer</PartNumber>
         <Size>long</Size>
      </Part>
      ...
      <PartsCount>integer</PartsCount>
   </ObjectParts>
   <StorageClass>string</StorageClass>
   <ObjectSize>long</ObjectSize>
</GetObjectAttributesOutput>
```

Response headers

"GET response headers and output fields" on page 1523 lists the supported response headers and output fields.

Table 169: GET response headers and output fields

Response header / field	Description	Type
Last-Modified	The date and time that the object was last modified.	Timestamp
x-amz-delete-marker	Specifies whether the object retrieved was (`true`) or was not (`false`) a delete marker. If `false`, this response header does not appear in the response.	Boolean
x-amz-request-charged	If present, indicates that the requester was successfully charged for the request.	String
x-amz-version-id	The version ID of the object.	String
GetObjectAttributesOutput	Root-level container for the `GetObjectAttributesOutput` response fields.	Structure
Checksum	The checksum or digest of the object.	Structure
ChecksumCRC32	The base64-encoded, 32-bit CRC-32 checksum of the object. This value is only present if it was uploaded with the object. For objects uploaded using multipart uploads, this value is calculated from the checksum values of each individual part, not the full object.	String
ChecksumCRC32C	The base64-encoded, 32-bit CRC-32C checksum of the object. This value is only present if it was uploaded with the object. For objects uploaded using multipart uploads, this value is calculated from the checksum values of each individual part, not the full object.	String
ChecksumSHA1	The base64-encoded, 160-bit SHA-1 digest of the object. This value is only present if it was uploaded with the object. For objects uploaded using multipart uploads, this value is calculated from the checksum values of each individual part, not the full object.	String
ChecksumSHA256	The base64-encoded, 256-bit SHA-256 digest of the object. This value is only present if it was uploaded with the object. For objects uploaded using multipart uploads, this value is calculated from the checksum values of each individual part, not the full object.	String
ETag	An opaque identifier assigned to a specific version of an object.	String
ObjectParts	A collection of parts associated with a multipart upload.	Structure
TotalPartsCount	The total number of parts.	Integer
PartNumberMarker	The marker for the current part.	Integer
NextPartNumberMarker	When a list is truncated, specifies the last part in the list and the value to use for the **PartNumberMarker** request parameter in a subsequent request.	Integer
MaxParts	The maximum number of parts allowed in the response.	Integer
IsTruncated	Indicates whether the returned list of parts is truncated. A value of `true` indicates that the list was truncated. A list can be truncated if the number of parts exceeds the limit returned in the **MaxParts** field.	Boolean

Response header / field	Description	Type
Parts	A container for elements related to a particular part. A response can contain zero or more **Parts** elements. **Note:** – General purpose buckets: For `GetObjectAttributes`, if an additional checksum (including **x-amz-checksum-crc32**, **x-amz-checksum-crc32c**, **x-amz-checksum-sha1**, or **x-amz-checksum-sha256**) is not applied to the object specified in the request, the response does not return **Part**. – Directory buckets: For `GetObjectAttributes`, regardless of whether an additional checksum is applied to the object specified in the request, the response returns **Part**.	List
PartNumber	The part number identifying the part. This value is a positive integer between 1 and 10,000.	Integer
Size	The size of the uploaded part in bytes.	Long
ObjectSize	The size of the object in bytes.	Long
StorageClass	The storage class of the object. Returned for all objects except those in the Standard storage class.	String

S3 copy an object

Use the S3 **PUT** command to copy an object to a destination bucket.

To copy an object, use the **PUT** command and specify a destination bucket and object name.

```
PUT /DEST_BUCKET/DEST_OBJECT HTTP/1.1
x-amz-copy-source: SOURCE_BUCKET/SOURCE_OBJECT
```

Request headers

"Copy object request headers" on page 1524 lists the supported request headers.

Table 170: Copy object request headers

Header	Description	Valid values	Required
x-amz-copy-source	The source bucket name and object name.	`SOURCE_BUCKET/SOURCE_OBJECT`	Yes
x-amz-acl	A canned ACL.	`private` | `public-read` | `public-read-write` | `authenticated-read`	No
x-amz-copy-if-modified-since	Copies only if modified since the timestamp.	Timestamp	No
x-amz-copy-if-unmodified-since	Copies only if not modified since the timestamp.	Timestamp	No
x-amz-copy-if-match	Copies only if the object ETag matches.	Entity Tag	No

Header	Description	Valid values	Required
x-amz-copy-if-none-match	Copies only if the object ETag does not match.	Entity Tag	No

Response entities

"Copy object response entities" on page 1525 lists the response entities.

Table 171: Copy object response entities

Entity	Description	Type
CopyObjectResult	A container for the response elements.	Container
LastModified	The last modified date of the source object.	Date
ETag	The ETag of the new object.	String

S3 delete an object

Use the S3 **DELETE** command to remove an object from a bucket.

Removes an object. Requires WRITE permission on the containing bucket. If bucket versioning is enabled, the delete operation creates a *delete marker*. To permanently delete a specific version, specify the **versionId** subresource.

Delete the latest version of an object:

```
DELETE /BUCKET/OBJECT HTTP/1.1
```

Delete a specific version of an object:

```
DELETE /BUCKET/OBJECT?versionId=VERSION_ID HTTP/1.1
```

Conditional delete

You can perform a conditional delete so that the object is deleted only if certain metadata conditions are met. This helps prevent accidental deletion if the object has changed since it was last read.

"Conditional delete options" on page 1525 lists the supported conditional delete options.

Table 172: Conditional delete options

Option	Description	Supported bucket type
if-match	Deletes the object only if the specified ETag matches the current ETag of the object. Returns `412 Precondition Failed` if it does not match.	General purpose and directory buckets.
if-match-last-modified-time	Deletes the object only if its last modified time matches the specified timestamp. Returns `412 Precondition Failed` if it does not match.	Directory buckets only.
if-match-size	Deletes the object only if the object size matches the specified size in bytes. Returns `412 Precondition Failed` if it does not match.	Directory buckets only.

> **Note:** You can use these conditional parameters individually or in combination. If the condition fails, the server returns HTTP status code `412 Precondition Failed`.

Examples

Example 1: Delete an object unconditionally.

```
aws s3api delete-object \
  --bucket my-example-bucket \
  --key logs/2025/system.log
```

Example 2: Delete a specific version of an object.

```
aws s3api delete-object \
  --bucket my-versioned-bucket \
  --key data/file.txt \
  --version-id 3HL4kqCxf3vjVBH40NrjfkdXh6xYzl7l
```

Example 3: Conditional delete using ETag.

```
aws s3api delete-object \
  --bucket my-dir-bucket \
  --key reports/january.csv \
  --if-match "b1946ac92492d2347c6235b4d2611184"
```

Example 4: Conditional delete using size and last modified time (directory bucket only).

```
aws s3api delete-object \
  --bucket amzn-s3-demo-bucket--usw2-az1--x-s3 \
  --key analytics/summary.json \
  --if-match-last-modified-time 2025-09-20T10:25:00Z \
  --if-match-size 2048
```

S3 delete multiple objects

Use the S3 **DELETE** command to remove multiple objects from a bucket in a single request.

Deletes multiple objects from a bucket in a single request. Each object entry can include optional metadata fields to perform conditional deletes.

Conditional delete

When deleting multiple objects, you can include optional metadata for each object to ensure that deletion occurs only if the specified conditions are met:

`ETag`

Deletes the object only if the current ETag matches the specified value.

`LastModifiedTime`

Deletes the object only if the last modified time matches the specified timestamp.

`Size`

Deletes the object only if the size matches the specified size in bytes.

> **Note:** If any condition fails, that object entry is skipped and the response includes the error for that object. Other objects in the batch are still processed.

Examples

Example 1: Delete three objects in one request.

```
aws s3api delete-objects \
  --bucket my-batch-bucket \
  --delete '{
    "Objects": [
      {"Key": "images/photo1.jpg"},
      {"Key": "images/photo2.jpg"},
      {"Key": "images/photo3.jpg"}
    ],
    "Quiet": false
  }'
```

Example 2: Conditional delete of multiple objects using ETag, LastModifiedTime, and Size. Deletes each object only if the specified metadata matches.

```
aws s3api delete-objects \
  --bucket my-dir-bucket--usw2-az1--x-s3 \
  --delete '{
    "Objects": [
      {
        "Key": "logs/2025/january.log",
        "ETag": "b1946ac92492d2347c6235b4d2611184",
        "LastModifiedTime": "2025-09-22T14:12:00Z",
        "Size": 4096
      },
      {
        "Key": "logs/2025/february.log",
        "ETag": "5d41402abc4b2a76b9719d911017c592"
      }
    ],
    "Quiet": false
  }'
```

Object metadata and tagging

Use these S3 API operations to manage access control lists (ACLs) and tags on objects in a Ceph Object Gateway bucket.

S3 get an object's Access Control List (ACL)

Use the S3 **GET** command to retrieve the Access Control List (ACL) for an object in a bucket.

Returns the ACL for the current version of the object:

```
GET /BUCKET/OBJECT?acl HTTP/1.1
```

Add the **versionId** subresource to retrieve the ACL for a particular version:

```
GET /BUCKET/OBJECT?versionId=VERSION_ID&acl HTTP/1.1
```

Response headers

"GET ACL response headers" on page 1527 lists the supported response headers.

Table 173: GET ACL response headers

Response header	Description
x-amz-version-id	Returns the version ID or null.

Response entities

"GET ACL response entities" on page 1527 lists the response entities.

Table 174: GET ACL response entities

Entity	Description	Type
AccessControlPolicy	A container for the response.	Container
AccessControlList	A container for the ACL information.	Container
Owner	A container for the bucket owner's **ID** and **DisplayName**.	Container
ID	The bucket owner's ID.	String
DisplayName	The bucket owner's display name.	String
Grant	A container for **Grantee** and **Permission**.	Container
Grantee	A container for the **DisplayName** and **ID** of the user receiving a grant of permission.	Container
Permission	The permission granted to the **Grantee**.	String

S3 set an object's Access Control List (ACL)

Use the S3 **PUT** command to set the Access Control List (ACL) for an object in a bucket.

Sets the ACL for the current version of the object.

```
PUT /BUCKET/OBJECT?acl HTTP/1.1
```

Request entities

"PUT ACL request entities" on page 1528 lists the request entities.

Table 175: PUT ACL request entities

Entity	Description	Type
AccessControlPolicy	A container for the request.	Container
AccessControlList	A container for the ACL information.	Container
Owner	A container for the bucket owner's **ID** and **DisplayName**.	Container
ID	The bucket owner's ID.	String
DisplayName	The bucket owner's display name.	String
Grant	A container for **Grantee** and **Permission**.	Container
Grantee	A container for the **DisplayName** and **ID** of the user receiving a grant of permission.	Container
Permission	The permission granted to the **Grantee**.	String

S3 put object tagging

Use the S3 **PUT** command to associate tags with an object.

The put object tagging API associates tags with an object. A tag is a key-value pair. To tag a specific version, use the **versionId** query parameter. You must have permission to perform the `s3:PutObjectTagging` action. By default, the bucket owner has this permission and can grant it to others.

Syntax

```
PUT /BUCKET/OBJECT?tagging&versionId=VERSION_ID HTTP/1.1
```

The **versionId** parameter is optional. Omit it to tag the current version of the object.

Request entities

"PUT object tagging request entities" on page 1528 lists the request entities.

Table 176: PUT object tagging request entities

Entity	Description	Type	Required
Tagging	A container for the request.	Container	Yes
TagSet	A collection of tags.	String	Yes

Related information

S3 API reference

S3 get object tagging

Use the S3 **GET** command to retrieve the tags associated with an object.

The get object tagging API returns the tags of an object. By default, the **GET** operation returns information about the current version of an object.

Note: For a versioned bucket, you can have multiple versions of an object in your bucket. To retrieve tags of a specific version, add the **versionId** query parameter to the request.

Syntax

```
GET /BUCKET/OBJECT?tagging&versionId=VERSION_ID HTTP/1.1
```

The **versionId** parameter is optional. Omit it to retrieve tags for the current version of the object.

Related information

S3 API reference

S3 delete object tagging

Use the S3 **DELETE** command to delete the tags associated with an object.

The delete object tagging API removes the entire tag set from the specified object. You must have permission to perform the `s3:DeleteObjectTagging` action, to use this operation.

Note: To delete tags of a specific object version, add the **versionId** query parameter in the request.

Syntax

```
DELETE /BUCKET/OBJECT?tagging&versionId= HTTP/1.1
```

Related information

S3 API reference

Object lock and retention

Use these S3 API operations to configure object lock, legal hold, and retention policies for compliance and governance on a Ceph Object Gateway bucket.

S3 put object lock

Use the S3 **PUT** command to place an object lock configuration on a bucket.

The put object lock API places a lock configuration on the selected bucket. With object lock, you can store objects using a Write-Once-Read-Many (WORM) model. Object lock ensures that an object is not deleted or overwritten for a fixed amount of time or indefinitely. The rule specified in the object lock configuration is applied by default to every new object placed in the selected bucket.

For more information about the WORM model, see Ceph Object Gateway.

Note: Enable object lock when creating the bucket. If object lock is not enabled at bucket creation, this operation fails.

Syntax

```
PUT /BUCKET?object-lock HTTP/1.1
```

Request entities

“PUT object lock request entities” on page 1530 lists the request entities.

Table 177: PUT object lock request entities			
Entity	**Description**	**Type**	**Required**
ObjectLockConfiguration	A container for the request.	Container	Yes
ObjectLockEnabled	Indicates whether this bucket has an object lock configuration enabled.	String	Yes
Rule	The object lock rule in place for the specified bucket.	Container	No
DefaultRetention	The default retention period applied to new objects placed in the specified bucket.	Container	No
Mode	The default object lock retention mode.	String	Yes
Days	The number of days specified for the default retention period.	Integer	No
Years	The number of years specified for the default retention period.	Integer	No

HTTP response codes

"PUT object lock HTTP response codes" on page 1530 lists the HTTP response codes returned by this operation.

Table 178: PUT object lock HTTP response codes		
HTTP code	**Status code**	**Description**
400	MalformedXML	The XML is not well-formed.
409	InvalidBucketState	The bucket object lock is not enabled.

Related information

S3 API reference

S3 get object lock

Use the S3 **GET** command to retrieve the object lock configuration for a bucket.

The get object lock API retrieves the lock configuration for a bucket.

Syntax

```
GET /BUCKET?object-lock HTTP/1.1
```

Response entities

"GET object lock response entities" on page 1530 lists the response entities.

Table 179: GET object lock response entities			
Entity	**Description**	**Type**	**Required**
ObjectLockConfiguration	A container for the response.	Container	Yes
ObjectLockEnabled	Indicates whether this bucket has an object lock configuration enabled.	String	Yes
Rule	The object lock rule in place for the specified bucket.	Container	No
DefaultRetention	The default retention period applied to new objects placed in the specified bucket.	Container	No
Mode	The default object lock retention mode.	String	Yes

Entity	Description	Type	Required
Days	The number of days specified for the default retention period.	Integer	No
Years	The number of years specified for the default retention period.	Integer	No

Related information

S3 API reference

S3 put object legal hold

Use the S3 **PUT** command to apply a legal hold to an object.

The put object legal hold API applies a legal hold configuration to the selected object. With a legal hold in place, you cannot overwrite or delete an object version. A legal hold does not have an associated retention period and remains in place until you explicitly remove it.

Syntax

```
PUT /BUCKET/OBJECT?legal-hold&versionId=VERSION_ID HTTP/1.1
```

The **versionId** parameter is optional. Use it to specify the version of the object to apply the legal hold to.

Request entities

"PUT object legal hold request entities" on page 1531 lists the request entities.

Table 180: PUT object legal hold request entities

Entity	Description	Type	Required
LegalHold	A container for the request.	Container	Yes
Status	Indicates whether the specified object has a legal hold in place.	String	Yes

Related information

S3 API reference

S3 get object legal hold

Use the S3 **GET** command to retrieve the current legal hold status of an object.

The get object legal hold API retrieves an object's current legal hold status.

Syntax

```
GET /BUCKET/OBJECT?legal-hold&versionId=VERSION_ID HTTP/1.1
```

The **versionId** parameter is optional. Use it to retrieve the legal hold status of a specific version of the object.

Response entities

"GET object legal hold response entities" on page 1531 lists the response entities.

Table 181: GET object legal hold response entities

Entity	Description	Type	Required
LegalHold	A container for the response.	Container	Yes
Status	Indicates whether the specified object has a legal hold in place.	String	Yes

Related information

S3 API reference

S3 put object retention

Use the S3 **PUT** command to place a retention configuration on an object.

The put object retention API places an object retention configuration on an object. A retention period protects an object version for a fixed amount of time. There are two retention modes: `GOVERNANCE` and `COMPLIANCE`. These two modes apply different levels of protection to your objects.

> **Note:** During this period, your object is Write-Once-Read-Many (WORM-protected) and cannot be overwritten or deleted. For more information about the WORM model, see Ceph Object Gateway.

Syntax

```
PUT /BUCKET/OBJECT?retention&versionId=VERSION_ID HTTP/1.1
```

The **`versionId`** parameter is optional. Use it to specify the version of the object to apply the retention configuration to.

Request entities

"PUT object retention request entities" on page 1532 lists the request entities.

Table 182: PUT object retention request entities

Entity	Description	Type	Required
`Retention`	A container for the request.	Container	Yes
`Mode`	Retention mode for the specified object. Valid values: `GOVERNANCE` | `COMPLIANCE`.	String	Yes
`RetainUntilDate`	The date until which the object is retained. Use the format `2020-01-05T00:00:00.000Z`.	Timestamp	Yes

Related information

S3 API reference

S3 get object retention

Use the S3 **`GET`** command to retrieve the retention configuration of an object.

The get object retention API retrieves the retention configuration of an object.

Syntax

```
GET /BUCKET/OBJECT?retention&versionId=VERSION_ID HTTP/1.1
```

The **`versionId`** parameter is optional. Use it to retrieve the retention configuration of a specific version of the object.

Response entities

"GET object retention response entities" on page 1532 lists the response entities.

Table 183: GET object retention response entities

Entity	Description	Type	Required
`Retention`	A container for the response.	Container	Yes
`Mode`	Retention mode for the specified object. Valid values: `GOVERNANCE` | `COMPLIANCE`.	String	Yes

Entity	Description	Type	Required
RetainUntilDate	The date until which the object is retained. Use the format `2020-01-05T00:00:00.000Z`.	Timestamp	Yes

Related information

S3 API reference

Multipart uploads

Use these S3 API operations to upload large objects in parts to a Ceph Object Gateway bucket. Initiate the upload, add parts, then assemble or abort the upload.

S3 initiate a multipart upload

Use the S3 **POST** command to initiate a multipart upload.

Initiates a multipart upload process. Returns an **UploadId**, which you can specify when adding parts, listing parts, and completing or aborting a multipart upload.

```
POST /BUCKET/OBJECT?uploads HTTP/1.1
```

Request headers

"Initiate multipart upload request headers" on page 1533 lists the supported request headers.

Table 184: Initiate multipart upload request headers

Header	Description	Valid values	Required
content-md5	A base64-encoded MD5 hash of the message.	String. No defaults or constraints.	No
content-type	A standard MIME type.	Any MIME type. Default: `binary/octet-stream`.	No
x-amz-meta-*	User metadata stored with the object.	A string up to 8 KB. No defaults.	No
x-amz-acl	A canned ACL.	`private` | `public-read` | `public-read-write` | `authenticated-read`	No

Response entities

"Initiate multipart upload response entities" on page 1533 lists the response entities.

Table 185: Initiate multipart upload response entities

Entity	Description	Type
InitiatedMultipartUploadsResult	A container for the results.	Container
Bucket	The bucket that receives the object contents.	String
Key	The key specified by the **key** request parameter.	String
UploadId	The ID that identifies the multipart upload, as specified by the **upload-id** request parameter.	String

S3 add a part to a multipart upload

Use the S3 **PUT** command to add a part to an existing multipart upload.

Adds a part to a multipart upload. Specify the **uploadId** subresource and the upload ID to identify the multipart upload to add the part to.

```
PUT /BUCKET/OBJECT?partNumber=PART_NUMBER&uploadId=UPLOAD_ID HTTP/1.1
```

HTTP response codes

"Add part HTTP response codes" on page 1534 lists the HTTP response codes returned by this operation.

Table 186: Add part HTTP response codes

HTTP code	Status code	Description
404	NoSuchUpload	The specified upload ID does not match any initiated upload on this object.

S3 copy a multipart upload

Use the S3 **PUT** command to upload a part by copying data from an existing object.

Uploads a part by copying data from an existing object as the data source. Specify the **uploadId** subresource and the upload ID to perform a multipart upload copy.

```
PUT /BUCKET/OBJECT?partNumber=PART_NUMBER&uploadId=UPLOAD_ID HTTP/1.1
```

Request headers

"Copy multipart upload request headers" on page 1534 lists the supported request headers.

Table 187: Copy multipart upload request headers

Header	Description	Valid values	Required
x-amz-copy-source	The source bucket name and object name.	BUCKET/OBJECT	Yes
x-amz-copy-source-range	The range of bytes to copy from the source object. For example, bytes=0-9 copies the first 10 bytes of the source.	bytes=FIRST-LAST, where **FIRST** and **LAST** are zero-based byte offsets.	No

Response entities

"Copy multipart upload response entities" on page 1534 lists the response entities.

Table 188: Copy multipart upload response entities

Entity	Description	Type
CopyPartResult	A container for all response elements.	Container
ETag	The ETag of the new part.	String
LastModified	The date that the part was last modified.	Date

Related information

S3 API reference — UploadPartCopy

S3 list the parts of a multipart upload

Use the S3 **GET** command to list the parts of a multipart upload.

Specify the **uploadId** subresource and the upload ID to list the parts of a multipart upload.

```
GET /BUCKET/OBJECT?uploadId=UPLOAD_ID HTTP/1.1
```

Response entities

"List parts response entities" on page 1535 lists the response entities.

Table 189: List parts response entities		
Entity	**Description**	**Type**
InitiatedMultipartUploadsResult	A container for the results.	Container
Bucket	The bucket that will receive the object contents.	String
Key	The key specified by the **key** request parameter, if any.	String
UploadId	The ID specified by the **upload-id** request parameter identifying the multipart upload, if any.	String
Initiator	Contains the **ID** and **DisplayName** of the user who initiated the upload.	Container
ID	The initiator's ID.	String
DisplayName	The initiator's display name.	String
Owner	A container for the **ID** and **DisplayName** of the user who owns the uploaded object.	Container
StorageClass	The method used to store the resulting object. `STANDARD` or `REDUCED_REDUNDANCY`.	String
PartNumberMarker	The part marker to use in a subsequent request if **IsTruncated** is `true`. Precedes the list.	String
NextPartNumberMarker	The next part marker to use in a subsequent request if **IsTruncated** is `true`. The end of the list.	String
IsTruncated	If `true`, only a subset of the object's upload contents were returned.	Boolean
Part	A container for **Key**, **Part**, **InitiatorOwner**, **StorageClass**, and **Initiated** elements.	Container
PartNumber	A container for **Key**, **Part**, **InitiatorOwner**, **StorageClass**, and **Initiated** elements.	Integer
ETag	The part's entity tag.	String
Size	The size of the uploaded part.	Integer

S3 assemble the uploaded parts

Use the S3 **POST** command to assemble uploaded parts and complete a multipart upload.

Assembles uploaded parts and creates a new object, thereby completing a multipart upload. Specify the **uploadId** subresource and the upload ID to complete a multipart upload.

```
POST /BUCKET/OBJECT?uploadId=UPLOAD_ID HTTP/1.1
```

Request entities

“Assemble multipart upload request entities” on page 1535 lists the request entities.

Table 190: Assemble multipart upload request entities			
Entity	**Description**	**Type**	**Required**
CompleteMultipartUpload	A container consisting of one or more parts.	Container	Yes
Part	A container for the **PartNumber** and **ETag**.	Container	Yes
PartNumber	The identifier of the part.	Integer	Yes
ETag	The part's entity tag.	String	Yes

Response entities

“Assemble multipart upload response entities” on page 1536 lists the response entities.

Table 191: Assemble multipart upload response entities		
Entity	**Description**	**Type**
CompleteMultipartUploadResult	A container for the response.	Container
Location	The resource identifier (path) of the new object.	URI
Bucket	The name of the bucket that contains the new object.	String
Key	The object's key.	String
ETag	The entity tag of the new object.	String

S3 abort a multipart upload

Use the S3 **DELETE** command to abort a multipart upload.

Aborts a multipart upload. Specify the **uploadId** subresource and the upload ID to abort a multipart upload.

```
DELETE /BUCKET/OBJECT?uploadId=UPLOAD_ID HTTP/1.1
```

S3 determine options for a request

Use the S3 **OPTIONS** command to send a preflight request to determine if an actual request can be sent with the specified origin, HTTP method, and headers.

```
OPTIONS /OBJECT HTTP/1.1
```

Retrieve sync replication Headers of object

Use the S3 **HEAD** command to retrieve sync replication headers for an object in a Ceph multi-zone environment.

Use the following command to retrieve the current version of the object:

```
HEAD /BUCKET/OBJECT HTTP/1.1
```

Add the **versionId** subresource to retrieve information for a particular version:

```
HEAD /BUCKET/OBJECT?versionId=VERSION_ID HTTP/1.1
```

Request headers

"HEAD request headers" on page 1536 lists the supported request headers.

Table 192: HEAD request headers			
Request header	**Description**	**Valid values**	**Required**
range	The range of the object to retrieve.	`Range:bytes=BEGINBYTE-ENDBYTE`	No
if-modified-since	Gets only if modified since the timestamp.	Timestamp	No
if-match	Gets only if the object ETag matches.	Entity tag	No
if-none-match	Gets only if the object ETag does not match.	Entity tag	No

Response headers

"HEAD response headers" on page 1536 lists the supported response headers.

Table 193: HEAD response headers	
Response header	**Description**
x-amz-version-id	Returns the version ID or null.

Response header	Description
x-rgw-replicated-from	Returns the source zone and any intermediate zones involved in an object's replication path within a Ceph multi-zone environment. This header is included in `GetObject` and `HeadObject` responses.
x-rgw-replicated-at	Returns a timestamp indicating when the object was replicated to its current location. Calculate the replication duration by using this header with the **Last-Modified** header.

Note: **x-rgw-replicated-from** and **x-rgw-replicated-at** are supported by client tools such as s3cmd or curl at the replicated zone. These tools can be used in addition to the **radosgw-admin** command for verification. There is a known issue [BZ-2312552] with the **radosgw-admin** object stat command where the **x-rgw-replicated-from** header key is missing.

S3 Hadoop interoperability

The Ceph Object Gateway supports Hadoop Distributed File System (HDFS) access through the Apache S3A connector.

For data analytics applications that require Hadoop Distributed File System (HDFS) access, the Ceph Object Gateway can be accessed using the Apache S3A connector for Hadoop. The S3A connector is an open-source tool that presents S3 compatible object storage as an HDFS file system. It provides HDFS file system read and write semantics to applications while data is stored in the Ceph Object Gateway.

Ceph Object Gateway supports the S3A connector that ships with Hadoop 2.7.3.

S3 select operations

As a developer, you can use the S3 select API for high-level analytic applications like Spark-SQL to improve latency and throughput.

There are three S3 select workflow - CSV, Apache Parquet (Parquet), and JSON that provide S3 select operations with CSV, Parquet, and JSON objects:

- A CSV file stores tabular data in plain text format. Each line of the file is a data record.
- Parquet is an open-source, column-oriented data file format designed for efficient data storage and retrieval. It provides highly efficient data compression and encoding schemes with enhanced performance to handle complex data in bulk. It can be used for single, multi, and archive site. Parquet enables the S3 select-engine to skip columns and chunks, thereby reducing IOPS dramatically (contrary to CSV and JSON format).
- JSON uses SQL statements to scan and extract information from JSON documents. It can be nested in various ways, such as within objects or arrays. These objects and arrays can be further nested within each other without any limitations.
- JSON is a format structure. The S3 select engine enables the use of SQL statements on top of the JSON format input data using the JSON reader, enabling the scanning of highly nested and complex JSON formatted data.

For example, a CSV, Parquet, or JSON S3 object with several gigabytes of data allows the user to extract a single column which is filtered by another column using the following query:

```
select customerid from s3Object where age>30 and age<65;
```

Currently, the S3 object must retrieve data from the Ceph OSD through the Ceph Object Gateway before filtering and extracting data. There is improved performance when the object is large and the query is more specific.

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- A RESTful client.
- A S3 user created with user access.

S3 select content from an object

The select object content API filters the content of an object through the structured query language (SQL).

Requests

In the request, you must specify the data serialization format as, comma-separated values (CSV) of the object to retrieve the specified content. Amazon Web Services (AWS) command-line interface (CLI) select object content uses the CSV format to parse object data into records and returns only the records specified in the query.

For more information and an example of the description of what should reside in the inventory object, see **Metadata collected by Inventory**, within the AWS Systems Manager User Guide.

> **Note:** You must specify the data serialization format for the response. You must have `s3:GetObject` permission for this operation.

```
POST /BUCKET/KEY?select&select-type=2 HTTP/1.1\r\n
```

For example,

```
POST /testbucket/sample1csv?select&select-type=2 HTTP/1.1\r\n
```

Table 194: Request entities

Request	Description	Type	Required
`Bucket`	The bucket to select object content from.	String	Yes
`Key`	The object key. Required minimum length of 1.	String	Yes
`SelectObjectContentRequest`	Root level tag for the select object content request parameters.	String	Yes
`Expression`	The expression that is used to query the object.	String	Yes
`ExpressionType`	The type of the provided expression for example SQL. Valid value: `SQL`	String	Yes
`InputSerialization`	Describes the format of the data in the object that is being queried.	String	Yes
`OutputSerialization`	Format of data returned in comma separator and new-line.	String	Yes

Responses

If the action is successful, the service sends back `HTTP 200` response. Data is returned in XML format by the service:

Table 195: Response entities

Response	Description	Type	Required
`Payload`	Root level tag for the payload parameters.	String	Yes
`Records`	The records event.	Base64-encoded binary data object	No

Response	Description	Type	Required
`Stats`	The stats event.	Long	No

Ceph Object Gateway supports the following response type:

```
{:event-type,records} {:content-type,application/octet-stream} {:message-type,event}
```

CSV requests

```
aws --endpoint-URL http://localhost:80 s3api select-object-content
 --bucket BUCKET_NAME
 --expression-type SQL
 --input-serialization
 {"CSV": {"FieldDelimiter": "," , "QuoteCharacter": "\"" , "RecordDelimiter" : "\n" ,
"QuoteEscapeCharacter" : "\\" , "FileHeaderInfo": "USE" }, "CompressionType": "NONE"}
 --output-serialization {"CSV": {}}
 --key OBJECT_NAME.csv
 --expression "select count(0) from s3object where int(_1)<10;" output.csv
```

Note: The object names in `--key OBJECT_NAME` do not require `.csv`, `.json`, or `.parquet`.

For example,

```
aws --endpoint-url http://localhost:80 s3api select-object-content
 --bucket testbucket
 --expression-type 'SQL'
 --input-serialization
 '{"CSV": {"FieldDelimiter": "," , "QuoteCharacter": "\"" , "RecordDelimiter" : "\n" ,
"QuoteEscapeCharacter" : "\\" , "FileHeaderInfo": "USE" }, "CompressionType": "NONE"}'
 --output-serialization '{"CSV": {}}'
 --key testobject.csv
 --expression "select count(0) from s3object where int(_1)<10;" output.csv
```

Parquet requests

Note: The only supported output serialization is `.csv`.

```
aws --endpoint-url http://localhost:80 s3api select-object-content
 --bucket BUCKET_NAME
 --expression-type 'SQL'
 --input-serialization
 '{"Parquet": {}, {"CompressionType": "NONE"}'
 --output-serialization '{"CSV": {}}}'
 --key OBJECT_NAME.parquet
 --expression "select count(0) from s3object where int(_1)<10;" output.csv
```

For example,

```
aws --endpoint-url http://localhost:80 s3api select-object-content
 --bucket testbucket
 --expression-type 'SQL'
 --input-serialization
 '{"Parquet":{}, "CompressionType": "NONE"}'
 --output-serialization '{"CSV": {}}}'
 --key testobject.parquet
 --expression "select count(0) from s3object where int(_1)<10;" output.csv
```

JSON requests

```
aws --endpoint-url http://localhost:80 s3api select-object-content
 --bucket BUCKET_NAME
 --key OBJECT_NAME.json
 --expression-type 'SQL'
 --input-serialization
 '{"JSON": {"Type": "DOCUMENT"}, "CompressionType": "NONE"}'
```

```
 --output-serialization '{"CSV": {}}'
 --expression "select count(*) from S3Object[*];" /dev/stdout
```

For example,

```
aws s3api --endpoint-url http://localhost:80
select-object-content --bucket test
--key testobject.json
--expression-type 'SQL'
--input-serialization
'{"JSON": {"Type": "DOCUMENT"}, "CompressionType": "NONE"}'
--output-serialization '{"CSV": {}}'
--expression "select count(*) from S3Object[*];" /dev/stdout
```

BOTO3 requests

```
import pprint
import boto3
from botocore.exceptions import ClientError

def run_s3select(bucket,key,query,column_delim=",",row_delim="\n",quot_char='"',esc_char='\
\',csv_header_info="NONE"):

   s3 = boto3.client('s3',
       endpoint_url=endpoint,
       aws_access_key_id=access_key,
       region_name=region_name,
       aws_secret_access_key=secret_key)

   result = ""
   try:
       r = s3.select_object_content(
       Bucket=bucket,
       Key=key,
       ExpressionType='SQL',
       InputSerialization = {"CSV": {"RecordDelimiter" : row_delim, "FieldDelimiter" :
column_delim,"QuoteEscapeCharacter": esc_char, "QuoteCharacter": quot_char,
"FileHeaderInfo": csv_header_info}, "CompressionType": "NONE"},
       OutputSerialization = {"CSV": {}},
       Expression=query,
       RequestProgress = {"Enabled": progress})

   except ClientError as c:
       result += str(c)
       return result

   for event in r['Payload']:
            if 'Records' in event:
                result = ""
                records = event['Records']['Payload'].decode('utf-8')
                result += records
            if 'Progress' in event:
                print("progress")
                pprint.pprint(event['Progress'],width=1)
            if 'Stats' in event:
                print("Stats")
                pprint.pprint(event['Stats'],width=1)
            if 'End' in event:
                print("End")
                pprint.pprint(event['End'],width=1)

   return result



 run_s3select(
 "my_bucket",
 "my_csv_object",
 "select int(_1) as a1, int(_2) as a2 , (a1+a2) as a3 from s3object where a3>100 and
a3<300;")
```

Supported features

Currently, only part of the AWS s3 select command is supported. For a full list of supported features, see “AWS s3 select command supported features” on page 1506.

Table 196: AWS s3 select command supported features

Features	Details	Description	Example
Arithmetic operators	`^ * % / + - ( )`	N/A	`select (int(_1)+int(_2))*int(_9) from s3object;`
Arithmetic operators	`% modulo`	N/A	`select count(*) from s3object where cast(_1 as int)%2 = 0;`
Arithmetic operators	`^ power-of`	N/A	`select cast(2^10 as int) from s3object;`
Compare operators	`> < >= <= == !=`	N/A	`select _1,_2 from s3object where (int(_1)+int(_3))>int(_5);`
logical operator	`AND OR NOT`	N/A	`select count(*) from s3object where not (int(1)>123 and int(_5)<200);`
logical operator	`is null`	Returns true/false for null indication in expression	N/A
logical operator and NULL	`is not null`	Returns true/false for null indication in expression	N/A
logical operator and NULL	unknown state	Review null-handle and observe the results of logical operations with NULL. The query returns `0`.	`select count(*) from s3object where null and (3>2);`
Arithmetic operator with NULL	unknown state	Review null-handle and observe the results of binary operations with NULL. The query returns `0`.	`select count(*) from s3object where (null+1) and (3>2);`
Compare with NULL	unknown state	Review null-handle and observe results of compare operations with NULL. The query returns `0`.	`select count(*) from s3object where (null*1.5) != 3;`
missing column	unknown state	N/A	`select count(*) from s3object where _1 is null;`
projection column	Similar to if or then or else	N/A	`select case when (1+1==(2+1)*3) then 'case_1' when ((4*3)==(12)) then 'case_2' else 'case_else' end, age*2 from s3object;`
projection column	Similar to switch/case default	N/A	`select case cast(_1 as int) + 1 when 2 then “a” when 3 then “b” else “c” end from s3object;`

Features	Details	Description	Example
logical operator	N/A	`coalesce` returns first non-null argument	`select coalesce(nullif(5,5), nullif(1,1.0),age+12) from s3object;`
logical operator	N/A	`nullif` returns null in case both arguments are equal, or else the first one,`nullif(1,1)=NULL nullif(null,1)=NULL nullif(2,1)=2`	`select nullif(cast(_1 as int),cast(_2 as int)) from s3object;`
logical operator	N/A	`{expression} in ( .. {expression} ..)`	`select count(*) from s3object where 'ben' in (trim(_5),substring(_1,char_length(_1)-3,3),last_name);`
logical operator	N/A	`{expression} between {expression} and {expression}`	`select _1 from s3object where cast(_1 as int) between 800 and 900;; select count(*) from stdin where substring(_3,char_length(_3),1) between “x” and trim(_1) and substring(_3,char_length(_3)-1,1) = “:”;`
logical operator	N/A	`{expression} like {match-pattern}`	`select count(*) from s3object where first_name like '%de_'; select count(*) from s3object where _1 like "%a[r-s];`
casting operator	N/A	N/A	`select cast(123 as int)%2 from s3object;`
casting operator	N/A	N/A	`select cast(123.456 as float)%2 from s3object;`
casting operator	N/A	N/A	`select cast('ABC0-9' as string),cast(substr('ab12cd',3,2) as int)*4 from s3object;`
casting operator	N/A	N/A	`select cast(substring('publish on 2007-01-01',12,10) as timestamp) from s3object;`
non AWS casting operator	N/A	N/A	`select int(_1),int( 1.2 + 3.4) from s3object;`
non AWS casting operator	N/A	N/A	`select float(1.2) from s3object;`

Features	Details	Description	Example
non AWS casting operator	N/A	N/A	`select to_timestamp('1999-10-10T12:23:44Z') from s3object;`
Aggregation Function	`sun`	N/A	`select sum(int(_1)) from s3object;`
Aggregation Function	`avg`	N/A	`select avg(cast(_1 as float) + cast(_2 as int)) from s3object;`
Aggregation Function	`min`	N/A	`select avg(cast(_1 a float) + cast(_2 as int)) from s3object;`
Aggregation Function	`max`	N/A	`select max(float(_1)),min(int(_5)) from s3object;`
Aggregation Function	`count`	N/A	`select count(*) from s3object where (int(1)+int(_3))>int(_5);`
Timestamp Functions	`extract`	N/A	`select count(*) from s3object where extract(year from to_timestamp(_2)) > 1950 and extract(year from to_timestamp(_1)) < 1960;`
Timestamp Functions	`dateadd`	N/A	`select count(0) from s3object where date_diff(year,to_timestamp(_1),date_add(day,366,to_timestamp(_1))) = 1;`
Timestamp Functions	`datediff`	N/A	`select count(0) from s3object where date_diff(month,to_timestamp(_1),to_timestamp(_2)) = 2;`
Timestamp Functions	`utcnow`	N/A	`select count(0) from s3object where date_diff(hour,utcnow(),date_add(day,1,utcnow())) = 24`
Timestamp Functions	`to_string`	N/A	`select to_string( to_timestamp(“2009-09-17T17:56:06.234567Z”), “yyyyMMdd-H:m:s”) from s3object;`
String Functions	`substring`	N/A	`select count(0) from s3object where int(substring(_1,1,4))>1950 and int(substring(_1,1,4))<1960;`

Features	Details	Description	Example
String Functions	`substring`	substring with from negative number is valid considered as first	`select substring("123456789" from -4) from s3object;`
String Functions	`substring`	substring with from zero for out-of-bound number is valid just as (first,last)	`select substring("123456789" from 0 for 100) from s3object;`
String Functions	`trim`	N/A	`select trim(' foobar ') from s3object;`
String Functions	`trim`	N/A	`select trim(trailing from ' foobar ') from s3object;`
String Functions	`trim`	N/A	`select trim(leading from ' foobar ') from s3object;`
String Functions	`trim`	N/A	`select trim(both '12' from '1112211foobar22211122') from s3object;`
String Functions	`lower or upper`	N/A	`select lower('ABcD12#$e') from s3object;`
String Functions	`char_length, character_length`	N/A	`select count(*) from s3object where char_length(_3)=3;`
Complex queries	N/A	N/A	`select sum(cast(_1 as int)),max(cast(_3 as int)), substring('abcdefghijklm', (2-1)*3+sum(cast(_1 as int))/sum(cast(_1 as int))+1, (count() + count(0))/count(0)) from s3object;`
alias support	N/A	N/A	`select int(_1) as a1, int(_2) as a2 , (a1+a2) as a3 from s3object where a3>100 and a3<300;`

Related information

Amazon's S3 Select Object Content API

S3 supported select functions

Get to know the different S3 select supported functions.

S3 select supports the following functions:

- "Timestamp" on page 1510
- "Aggregation" on page 1512
- "String" on page 1512
- "NULL" on page 1513

For more information about S3 select functions, see **SelectObjectContent** within the Amazon Simple Storage Service (S3) API Reference section of AWS Documentation.

Timestamp

`to_timestamp` (string)

Description

Converts string to timestamp basic type. In the string format, any missing 'time' value is populated with zero; for missing month and day value, 1 is the default value. 'Timezone' is in format `+/-HH:mm` or `Z` , where the letter 'Z' indicates Coordinated Universal Time (UTC). Value of timezone can range between - 12:00 and +14:00.

Supported

Currently it can convert the following string formats into timestamp:

- `YYYY-MM-DDTHH:mm:ss.SSSSSS+/-HH:mm`
- `YYYY-MM-DDTHH:mm:ss.SSSSSSZ`
- `YYYY-MM-DDTHH:mm:ss+/-HH:mm`
- `YYYY-MM-DDTHH:mm:ssZ`
- `YYYY-MM-DDTHH:mm+/-HH:mm`
- `YYYY-MM-DDTHH:mmZ`
- `YYYY-MM-DDT`
- `YYYYT`

`to_string` (timestamp, format_pattern)

Description

Returns a string representation of the input timestamp in the given input string format.

Parameters

Table 197: `to_string` parameters

Format	Example	Description
yy	69	2-digit year
y	1969	4-digit year
yyyy	1969	Zero-padded 4-digit year
M	1	Month of the year
MM	01	Zero-padded month of the year
MMM	Jan	Abbreviated month of the year name
MMMM	January	Full month of the year name
MMMMM	J	Month of the year first letter. **Note:** Not valid for use with the `to_timestamp` function.
d	2	Day of the month (1-31)
dd	02	Zero-padded day of the month (01-31)
a	AM	AM or PM of day
h	3	Hour of day (1-12)

Format	Example	Description
hh	03	Zero-padded hour of the day (01-12)
H	3	Hour of the day (0-24)
HH	03	Zero-padded hour of the day (00-23)
m	4	Minute of the hour (0-59)
mm	04	Zero-padded minute of the hour (00-59)
s	5	Second of the minute (0-59)
ss	05	Zero padded second of the minute (00-59)
S	1	Fraction of the second (precision 0.1, range: 0.0-0.9)
SS	12	Fraction of the second (precision 0.01, range: 0.0-0.99)
SSS	123	Fraction of the second (precision: 0.01, range: 0.0-0.999).
SSSS	1234	Fraction of the second (precision: 0.001, range: 0.0-0.9999).
SSSSSS	123456	Fraction of the second (maximum precision: 1 nanosecond, range: 0.0-0.999999)
n	60000000	Nano of second
X	+07 or Z	Offset in hours, or “Z” if the offset is 0
XX or XXXX	+0700 or Z	Offset in hours and minutes or “Z” if the offset is 0
XXX or XXXXX	+07:00 or Z	Offset in hours and minutes or “Z” if the offset is 0
x	7	Offset in hours
xx or xxxx	700	Offset in hours and minutes
xxx or xxxxx	+07:00	Offset in hours and minutes

`extract` (date-part from timestamp)

Description

Returns integer according to date-part extract from input timestamp.

Supported

year, month, week, day, hour, minute, second, timezone_hour, timezone_minute.

`date_add` (date-part ,integer,timestamp)

Description

Returns timestamp, a calculation based on the results of input timestamp and date-part.

Supported

year, month, day, hour, minute, second.

`date_diff` (date-part,timestamp,timestamp)

Description

Return an integer, a calculated result of the difference between two timestamps according to date-part.

Supported

year, month, day, hour, minute, second.

`utcnow()`

Description

Return timestamp of current time.

Aggregation

`count()`

Description

Returns integers based on the number of rows that match a condition if there is one.

`sum(expression)`

Description

Returns a summary of expression on each row that matches a condition if there is one.

`avg(expression)`

Description

Returns an average expression on each row that matches a condition if there is one.

`max` (expression)

Description

Returns the maximal result for all expressions that match a condition if there is one.

`min` (expression)

Description

Returns the minimal result for all expressions that match a condition if there is one.

String

`substring` (string,from,for)

Description

Returns a string extract from the input string according to from, for inputs.

`Char_length`

Description

Returns a number of characters in string. `Character_length` also does the same.

`trim([[leading | trailing | both remove_chars] from] string )`

Description

Trims leading/trailing (or both) characters from the target string. The default value is a blank character.

`Upper\lower`

Description

Converts characters into uppercase or lowercase.

NULL

The `NULL` value indicates a missing or unknown that is `NULL` and that cannot produce a value on any arithmetic operations.

"The NULL use case" on page 1513 depicts various NULL use cases.

Table 198: The NULL use case	
A is NULL	**Result (NULL=UNKNOWN)**
Not A	NULL
A or False	NULL
A or True	True
A or A	NULL
A and False	False
A and True	NULL
A and A	NULL

S3 alias programming construct

Alias programming construct is an essential part of the s3 select language because it enables better programming with objects that contain many columns or complex queries.

When you parse a statement with alias construct, it replaces the alias with a reference to the right projection column. On querying, the reference is evaluated like any other expression. Alias maintains result-cache. If an alias is used more than one time, the same expression is not evaluated. The same result is returned because the result from the cache is used. Currently, Red Hat supports the column alias.

There is a risk that self or cyclic reference might occur causing stack-overflow (endless-loop), for that concern upon evaluating an alias, it is validated for cyclic reference.

With each new row the cache is invalidated as the results may then differ.

Example

```
select int(_1) as a1, int(_2) as a2 , (a1+a2) as a3 from s3object where a3>100 and a3<300;
```

S3 parsing explained

The S3 select engine has parsers for CSV, Parquet, and JSON file formats. The parsers separate the commands into more processable components and are then attached to tags that define each component.

S3 CSV parsing

Use this information to get to know about how CSV parses S3-objects.

You can define the CSV definitions with input serialization uses these default values:

- Use `{\n}` for row-delimiter.
- Use `{"}` for quote.
- Use `{\}` for escape characters.

The `csv-header-info` is parsed upon `USE` appearing in the AWS-CLI; this is the first row in the input object containing the schema. Currently, output serialization and compression-type is not supported. The S3 select engine has a CSV parser which parses S3-objects:

- Each row ends with a row-delimiter.
- The field-separator separates the adjacent columns.
- The successive field separator defines the `NULL` column.
- The quote-character overrides the field-separator; that is, the field separator is any character between the quotes.
- The escape character disables any special character except the row delimiter.

The following are examples of CSV parsing rules:

Table 199: CSV parsing		
Feature	**Description**	**Input (Tokens)**
`NULL`	Successive field delimiter	`,,1,,2, ==> {null}{null}{1}{null}{2}{null}`
`QUOTE`	The quote character overrides the field delimiter.	`11,22,”a,b,c,d”,last ==> {11}{22}{“a,b,c,d”}{last}`
`Escape`	The escape character overrides the meta-character.	`11,22,str=\”abcd\”\,str2=\”123\”,last ==> {11}{22}{str=”abcd”,str2=”123”}{last}`
`row delimiter`	There is no closed quote; row delimiter is the closing line.	`11,22,a=”str,44,55,66 ==> {11}{22}{a=”str,44,55,66}`
`csv header info`	FileHeaderInfo tag	USE value means each token on the first line is the column-name; IGNORE value means to skip the first line.

Reference

- See Amazon’s S3 Select Object Content API for more details.

S3 Parquet parsing

Apache Parquet is an open-source, columnar data file format designed for efficient data storage and retrieval. Use this information to get to know about how Apache Parquet parses S3-objects.

The S3 select engine's Parquet parser parses S3-objects as follows:

```
4-byte magic number "PAR1"
<Column 1 Chunk 1 + Column Metadata>
<Column 2 Chunk 1 + Column Metadata>
...
<Column N Chunk 1 + Column Metadata>
<Column 1 Chunk 2 + Column Metadata>
<Column 2 Chunk 2 + Column Metadata>
...
<Column N Chunk 2 + Column Metadata>
...
<Column 1 Chunk M + Column Metadata>
<Column 2 Chunk M + Column Metadata>
...
<Column N Chunk M + Column Metadata>
File Metadata
4-byte length in bytes of file metadata
4-byte magic number "PAR1"
```

- In the above example, there are N columns in this table, split into M row groups. The file metadata contains the locations of all the column metadata start locations.
- Metadata is written after the data to allow for single pass writing.
- All the column chunks can be found in the file metadata which should later be read sequentially.
- The format is explicitly designed to separate the metadata from the data. This allows splitting columns into multiple files, as well as having a single metadata file reference multiple parquet files.

S3 JSON parsing

Use this information to get to know about how JSON parses S3-objects.

The values in a JSON document can be nested within objects or arrays without limitations. When querying a specific value in a JSON document in the S3 select engine, the location of the value is specified via a path in the `SELECT` statement.

The generic structure of a JSON document does not have a row and column structure like CSV and Parquet. Instead, it is the SQL statement itself that defines the rows and columns when querying a JSON document.

The S3 select engine's JSON parser parses S3-objects as follows:

- The `FROM` clause in the `SELECT` statement defines the row boundaries
- A row in a JSON document is similar to how the row delimiter is used to define rows for CSV objects, and how row groups are used to define rows for Parquet objects
- Consider the following example:

```
{
"firstName": "Joe",
"lastName": "Jackson",
"gender": "male",
"age": "twenty",
"address":
  {
   "streetAddress": "101",
   "city": "San Diego",
   "state": "CA"
  },
"firstName": "Joe_2",
"lastName": "Jackson_2",
"gender": "male",
"age": 21,
"address":
 {
  "streetAddress": "101",
  "city": "San Diego",
  "state": "CA"
 },
"phoneNumbers":
 [
  { "type": "home1", "number": "734928_1","addr": 11 },
  { "type": "home2", "number": "734928_2","addr": 22 }
 ],
"key_after_array": "XXX",
"description" :
 {
  "main_desc" : "value_1",
  "second_desc" : "value_2"
 }
}

 # the from-clause define a single row.
 # _1 points to root object level.
 # _1.age appears twice in Documnet-row, the last value is used for the operation.
 query = "select
_1.firstname,_1.key_after_array,_1.age+4,_1.description.main_desc,_1.description.second
_desc from s3object[*];";
 expected_result = Joe_2,XXX,25,value_1,value_2


 # the from-clause points the phonenumbers array (it defines the _1)
 # each element in phoneNumbers array define a row.
 # in this case each element is an object contains 3 keys/values.
 # the query "can not access" values outside phonenumbers array, the query can access
only values appears on _1.phonenumbers path.
 query = "select cast(substring(_1.number,1,6) as int) *10 from
s3object[*].phonenumbers where _1.type='home2';";
 expected_result = 7349280
```

  - The statement instructs the reader to search for the path *aa.bb.cc* and defines the row boundaries based on the occurrence of this path.
  - A row begins when the reader encounters the path, and it ends when the reader exits the innermost part of the path, which in this case is the object *cc*.

S3 SQL limit operator

The SQL limit operator is used to limit the number of rows processed by the query.

Upon reaching the limit set by the user, the Ceph Object Gateway stops fetching additional chunks.

The following are examples of SQL limit operator for CSV, Parquet, and JSON.

Example of SQL limit operator for CSV

```
[cephuser@host ~]$ aws s3api --endpoint-url http://extensa027.ceph.rh.com:80 select-object-
content  --bucket bkt1 --key placement_data1 --expression-type 'SQL' --input-serialization
'{"CSV": {"FileHeaderInfo": "USE"}, "CompressionType": "NONE"}' --output-serialization
'{"CSV": {}}' --expression "select * from s3object limit 10;"

/dev/stdout
1,M,67.00,Others,91.00,Others,Commerce,58.00,Sci&Tech,No,55,Mkt&HR,58.8,Placed,270000
2,M,79.33,Central,78.33,Others,Science,77.48,Sci&Tech,Yes,86.5,Mkt&Fin,66.28,Placed,200000
3,M,65.00,Central,68.00,Central,Arts,64.00,Comm&Mgmt,No,75,Mkt&Fin,57.8,Placed,250000
4,M,56.00,Central,52.00,Central,Science,52.00,Sci&Tech,No,66,Mkt&HR,59.43,Not Placed,
5,M,85.80,Central,73.60,Central,Commerce,73.30,Comm&Mgmt,No,96.8,Mkt&Fin,55.5,Placed,425000
6,M,55.00,Others,49.80,Others,Science,67.25,Sci&Tech,Yes,55,Mkt&Fin,51.58,Not Placed,
7,F,46.00,Others,49.20,Others,Commerce,79.00,Comm&Mgmt,No,74.28,Mkt&Fin,53.29,Not Placed,
8,M,82.00,Central,64.00,Central,Science,66.00,Sci&Tech,Yes,67,Mkt&Fin,62.14,Placed,252000
9,M,73.00,Central,79.00,Central,Commerce,72.00,Comm&Mgmt,No,91.34,Mkt&Fin,61.29,Placed,2310
00
10,M,58.00,Central,70.00,Central,Commerce,61.00,Comm&Mgmt,No,54,Mkt&Fin,52.21,Not Placed,
```

Example of SQL limit operator for Parquet

```
[cephuser@extensa022 ~]$ aws s3api --endpoint-url http://extensa027.ceph.rh.com:80 select-
object-content  --bucket parquetbkt2 --key smallfile --expression-type 'SQL' --input-
serialization '{"Parquet": {}, "CompressionType": "NONE"}' --output-serialization '{"CSV":
{}}'  --expression "select * from s3object limit 5;"

/dev/stdout
4819,90340852.424672872,exile,0
4638,95603700.561133489,contradiction,1
1667,99440023.596563846,slam,2
7113,33213151.245887276,publisher,3
2872,43556628.954562038,demonstrate,4
```

Example of SQL limit operator for JSON

```
[cephuser@extensa027 ~]$ aws s3api --endpoint-url http://extensa027.ceph.rh.com:80 select-
object-content  --bucket bkt1 --key books_data.json --expression-type 'SQL' --input-
serialization '{"JSON": {"Type": "DOCUMENT"}, "CompressionType": "NONE"}' --output-
serialization '{"CSV": {}}' --expression "select * from s3object[*];"

/dev/stdout
id. : 1091
authors.name. : Makoto Satoh
authors.org. : Shinshu University
authors.id. : 2312688602
authors.name. : Ryo Muramatsu
authors.org. : Shinshu University
authors.id. : 2482909946
title. : Preliminary Design of a Network Protocol Learning Tool Based on the Comprehension
of High School Students: Design by an Empirical Study Using a Simple Mind Map
year. : 2013
#=== 0 ===#
id. : 1388
authors.name. : Mizue Kayama
authors.org. : Shinshu University
authors.id. : 2128134587
authors.name. : Pranava K. Jha
authors.id. : 2718958994
authors.name. : Kazunori Itoh
authors.org. : Shinshu University
authors.id. : 2101782692
title. : Further Results on Independence in Direct-Product Graphs.
year. : 2000
#=== 1 ===#
id. : 5781
authors.name. : Jovan Dj. Golic
authors.id. : 1237859792
authors.name. : Guglielmo Morgari
authors.id. : 220887178
title. : Vectorial fast correlation attacks.
year. : 2004
#=== 2 ===#
id. : 6522
authors.name. : Güzin Ulutas
authors.org. : Karadeniz Technical Univ.
authors.id. : 2022192081
authors.name. : Mustafa Ulutas
authors.org. : Karadeniz Technical Univ.
authors.id. : 2023460672
authors.name. : Vasif V. Nabiyev
authors.org. : Karadeniz Technical Univ.
authors.id. : 2174205032
title. : Improved Secret Image Sharing Method By Encoding Shared Values With Authentication
Bits
year. : 2011
#=== 3 ===#
```

Integrating Ceph Object Gateway with Trino

Integrate the Ceph Object Gateway with Trino, an important utility that enables the user to run SQL queries 9x faster on S3 objects.

Before you begin

- A running cluster with Ceph Object Gateway installed.
- Docker or Podman installed.
- Buckets created.
- Objects are uploaded.

About this task

You can integrate Ceph Object Gateway with Trino for S3 select operations.

The following are some benefits of using Trino:

- Trino is a complete SQL engine.

- Pushes down S3 select requests wherein the Trino engine identifies part of the SQL statement that is cost effective to run on the server-side.
- Uses the optimization rules of Ceph/S3select to enhance performance.
- Leverages scalability and divides the original object into multiple equal parts, performs S3 select requests, and merges the request.

Important: If the `s3select` syntax does not work while querying through trino, use the SQL syntax.

Procedure

1. Deploy Trino and hive.
   a. Clone the repository.

```
[cephuser@host01 ~]$ git clone https://github.com/ceph/s3select.git
[cephuser@host01 ~]$ cd s3select
```

   b. Modify the `hms_trino.yaml` file with S3 endpoint, access key, and secret key.

```
[cephuser@host01 s3select]$ cat container/trino/hms_trino.yaml
version: '3'
services:
  hms:
    image: galsl/hms:dev
    container_name: hms
    environment:
      # S3_ENDPOINT the CEPH/RGW end-point-url
      - S3_ENDPOINT=http://rgw_ip:port
      - S3_ACCESS_KEY=abc
      - S3_SECRET_KEY=abc
    # the container starts with booting the hive metastore
    command: sh -c '. ~/.bashrc; start_hive_metastore'
    ports:
      - 9083:9083
    networks:
      - trino_hms

  trino:
    image: trinodb/trino:405
    container_name: trino
    volumes:
      # the trino directory contains the necessary configuration
      - ./trino:/etc/trino
    ports:
      - 8080:8080
    networks:
      - trino_hms

networks:
  trino_hm
```

   c. Modify the `hive.properties` file with S3 endpoint, access key, and secret key.

```
[cephuser@host01 s3select]$ cat container/trino/trino/catalog/hive.properties
connector.name=hive
hive.metastore.uri=thrift://hms:9083

#hive.metastore.warehouse.dir=s3a://hive/

hive.allow-drop-table=true
hive.allow-rename-table=true
hive.allow-add-column=true
hive.allow-drop-column=true
hive.allow-rename-column=true

hive.non-managed-table-writes-enabled=true
hive.s3select-pushdown.enabled=true
hive.s3.aws-access-key=abc
hive.s3.aws-secret-key=abc

# should modify per s3-endpoint-url
hive.s3.endpoint=http://rgw_ip:port
```

```
#hive.s3.max-connections=1
#hive.s3select-pushdown.max-connections=1

hive.s3.connect-timeout=100s
hive.s3.socket-timeout=100s
hive.max-splits-per-second=10000
hive.max-split-size=128MB
```

2. Start a Trino container to integrate Ceph Object Gateway.

```
[cephuser@host01 s3select]$ sudo docker compose -f ./container/trino/hms_trino.yaml up
-d
```

3. Verify integration.

```
[cephuser@host01 s3select]$ sudo docker exec -it trino /bin/bash
trino@66f753905e82:/$ trino
trino> create schema hive.csvbkt1schema;
trino> create table hive.csvbkt1schema.polariondatacsv(c1 varchar,c2 varchar, c3
varchar, c4 varchar, c5 varchar, c6 varchar, c7 varchar, c8 varchar, c9 varchar) WITH
( external_location = 's3a://csvbkt1/',format = 'CSV');
trino> select * from hive.csvbkt1schema.polariondatacsv;
```

Note: The external location must point to the bucket name or a directory, and not the end file.

Ceph Object Gateway and the Swift API

Use a RESTful application programming interface (API) that is compatible with the Swift API data access model.

Manage the buckets and objects stored in Red Hat Ceph Storage cluster through the Ceph Object Gateway.

To use these APIs a running Red Hat Ceph Storage cluster and RESTful client are required.

Swift API limitations

Important: The following limitations should be used with caution. There are implications related to your hardware selections, so you should always discuss these requirements with your Red Hat account team.

Maximum object size

When using Swift API, the maximum object size is 5 GB.

Maximum metadata size

There is no defined limit on the total size of user metadata that can be applied to an object when using Swift API, but a single HTTP request is limited to 16,000 bytes.

Swift function feature support

"Support status for Swift functional features" on page 1554 describes the support status for current Swift functional features.

Table 200: Support status for Swift functional features

Feature	Status	Notes
Authentication	Supported	None
Get Account Metadata	Supported	No custom metadata
Swift ACLs	Supported	Supports a subset of Swift ACLs
List Containers	Supported	None
List Container's Objects	Supported	None
Create Container	Supported	None

Feature	Status	Notes
Delete Container	Supported	None
Get Container Metadata	Supported	None
Add/Update Container Metadata	Supported	None
Delete Container Metadata	Supported	None
Get Object	Supported	None
Create/Update an Object	Supported	None
Create Large Object	Supported	None
Delete Object	Supported	None
Copy Object	Supported	None
Get Object Metadata	Supported	None
Add/Update Object Metadata	Supported	None
Temp URL Operations	Supported	None
CORS	Not Supported	None
Expiring Objects	Supported	None
Object Versioning	Not Supported	None
Static Website	Not Supported	None

Create a Swift user

Create a Swift subuser and a secret key to test the Swift interface.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Installation of the Ceph Object Gateway.
- Root-level access to the Ceph Object Gateway node.

About this task

To test the Swift interface, create a Swift subuser. Creating a Swift user is a two-step process. The first step is to create the user. The second step is to create the secret key.

> **Note:** In a multi-site deployment, always create a user on a host in the primary zone of the primary zone group.

Procedure

1. Create the Swift user.

   ```
   radosgw-admin subuser create --uid=NAME --subuser=NAME:swift --access=full
   ```

   Replace `NAME` with the Swift user name.
   For example,

```
[root@host01 ~]# radosgw-admin subuser create --uid=testuser --subuser=testuser:swift
--access=full
{
    "user_id": "testuser",
    "display_name": "First User",
    "email": "",
    "suspended": 0,
    "max_buckets": 1000,
    "auid": 0,
    "subusers": [
        {
            "id": "testuser:swift",
            "permissions": "full-control"
        }
    ],
    "keys": [
        {
            "user": "testuser",
            "access_key": "O8JDE41XMI74O185EHKD",
            "secret_key": "i4Au2yxG5wtr1JK01mI8kjJPM93HNAoVWOSTdJd6"
        }
    ],
    "swift_keys": [
        {
            "user": "testuser:swift",
            "secret_key": "13TLtdEW7bCqgttQgPzxFxziu0AgabtOc6vM8DLA"
        }
    ],
    "caps": [],
    "op_mask": "read, write, delete",
    "default_placement": "",
    "placement_tags": [],
    "bucket_quota": {
        "enabled": false,
        "check_on_raw": false,
        "max_size": -1,
        "max_size_kb": 0,
        "max_objects": -1
    },
    "user_quota": {
        "enabled": false,
        "check_on_raw": false,
        "max_size": -1,
        "max_size_kb": 0,
        "max_objects": -1
    },
    "temp_url_keys": [],
    "type": "rgw"
}
```

2. Create the secret key.

```
radosgw-admin key create --subuser=NAME:swift --key-type=swift --gen-secret
```

Replace `NAME` with the Swift user name.
For example,

```
[root@host01 ~]# radosgw-admin key create --subuser=testuser:swift --key-type=swift --
gen-secret
{
    "user_id": "testuser",
    "display_name": "First User",
    "email": "",
    "suspended": 0,
    "max_buckets": 1000,
    "auid": 0,
    "subusers": [
        {
            "id": "testuser:swift",
            "permissions": "full-control"
        }
    ],
    "keys": [
        {
            "user": "testuser",
            "access_key": "O8JDE41XMI74O185EHKD",
            "secret_key": "i4Au2yxG5wtr1JK01mI8kjJPM93HNAoVWOSTdJd6"
        }
    ],
    "swift_keys": [
        {
            "user": "testuser:swift",
            "secret_key": "a4ioT4jEP653CDcdU8p4OuhruwABBRZmyNUbnSSt"
        }
    ],
    "caps": [],
    "op_mask": "read, write, delete",
    "default_placement": "",
    "placement_tags": [],
    "bucket_quota": {
        "enabled": false,
        "check_on_raw": false,
        "max_size": -1,
        "max_size_kb": 0,
        "max_objects": -1
    },
    "user_quota": {
        "enabled": false,
        "check_on_raw": false,
        "max_size": -1,
        "max_size_kb": 0,
        "max_objects": -1
    },
    "temp_url_keys": [],
    "type": "rgw"
}
```

Swift authenticating a user

To authenticate a user, make a request containing an `X-Auth-User` and a `X-Auth-Key` in the header.

Syntax

```
GET /auth HTTP/1.1
Host: swift.example.com
X-Auth-User: johndoe
X-Auth-Key: R7UUOLFDI2ZI9PRCQ53K
```

Example Response

```
HTTP/1.1 204 No Content
Date: Mon, 16 Jul 2012 11:05:33 GMT
Server: swift
X-Storage-Url: https://swift.example.com
X-Storage-Token: UOlCCC8TahFKlWuv9DB09TWHF0nDjpPElha0kAa
Content-Length: 0
Content-Type: text/plain; charset=UTF-8
```

Note: You can retrieve data about Ceph's Swift-compatible service by executing `GET` requests using the `X-Storage-Url` value during authentication.

Reference

For more information, see:

- "Swift request headers" on page 1576
- "Swift response headers" on page 1576

Swift container operations

Swift container operations provide mechanisms to manage data storage, access controls, and object metadata through the Ceph Object Gateway.

To use Swift container operations, a running Red Hat Ceph Storage cluster and a REST client are required.

A container is a storage mechanism for data objects within an account. An account can have multiple containers, but container names must be unique. Through the Ceph Object Gateway Swift application programming interface (API), you can list, create, update, and delete containers, as well as manage container metadata.

Because the Swift API processes requests that access account information, all requests must be authenticated unless container access control is explicitly configured for public or anonymous access.

> **Important:** When uploading large objects to versioned Swift containers, specify the **`--leave-segments`** option with the **`python-swiftclient`** utility. Omitting **`--leave-segments`** overwrites the manifest file and the existing object, which results in data loss.

S3 and Swift terminology comparison

The Amazon Simple Storage Service (Amazon S3) API uses the term *bucket* to describe a data container. In Ceph Object Gateway, a bucket in the S3 API is equivalent to a container in the Swift API.

Pseudo-hierarchical containers

Object storage does not use hierarchical directories. Instead, storage consists of a single level of one or more containers, where each container holds objects. The Ceph Object Gateway Swift API supports pseudo-hierarchical containers, which use object naming conventions to emulate directory structures without implementing a physical directory hierarchy.

You can name objects with pseudo-hierarchical paths, such as `photos/buildings/empire-state.jpg`. However, container names cannot contain a forward slash (`/`) character.

Swift update a container's Access Control List (ACL)

When a user creates a container, the user has read and write access to the container by default. To allow other users to read a container's contents or write to a container, you must specifically enable the user. You can also specify `*` in the `X-Container-Read` or `X-Container-Write` settings, which effectively enables all users to either read from or write to the container. Setting `*` makes the container public. That is it enables anonymous users to either read from or write to the container.

Syntax

```
POST /_AP_VERSION_/_ACCOUNT_/_TENANT_:_CONTAINER_ HTTP/1.1
Host: FULLY_QUALIFIED_DOMAIN_NAME
X-Auth-Token: _AUTH_TOKEN_
X-Container-Read: *
X-Container-Write: _UID1_, _UID2_, _UID3_
```

`X-Container-Read`

Description

The user IDs with read permissions for the container.

Type

Comma-separated string values of user IDs.

Required

No

`X-Container-Write`

Description

The user IDs with write permissions for the container.

Type

Comma-separated string values of user IDs.

Required

No

Swift list containers

A `GET` request that specifies the API version and the account will return a list of containers for a particular user account. Since the request returns a particular user's containers, the request requires an authentication token. The request cannot be made anonymously.

Syntax

```
GET /API_VERSION/ACCOUNT HTTP/1.1
Host: FULLY_QUALIFIED_DOMAIN_NAME
X-Auth-Token: AUTH_TOKEN
```

Request Parameters

`limit`

Description

Limits the number of results to the specified value.

Type

Integer

Valid Values

N/A

Required

Yes

`format`

Description

Limits the number of results to the specified value.

Type

Integer

Valid Values

`json` or `xml`

Required

No

`marker`

Description

Returns a list of results greater than the marker value.

Type

String

Valid Values

N/A

Required

No

The response contains a list of containers, or returns with an HTTP `204` response code.

Response Entities

`account`

Description

A list for account information.

Type

Container

`container`

Description

The list of containers.

Type

Container

`name`

Description

The name of a container.

Type

String

`bytes`

Description

The size of the container.

Type

Integer

Swift list a container's objects

To list the objects within a container, make a `GET` request with the API version, account, and the name of the container. You can specify query parameters to filter the full list, or leave out the parameters to return a list of the first 10,000 object names stored in the container.

Syntax

```
GET /_AP_VERSION_/_TENANT_:_CONTAINER_ HTTP/1.1
Host: FULLY_QUALIFIED_DOMAIN_NAME
X-Auth-Token: _AUTH_TOKEN_
```

Request Parameters

`format`

Description

Limits the number of results to the specified value.

Type

Integer

Valid Values

`json` or `xml`

Required

No

prefix

Description

Limits the result set to objects beginning with the specified prefix.

Type

String

Valid Values

N/A

Required

No

marker

Description

Returns a list of results greater than the marker value.

Type

String

Valid Values

N/A

Required

No

limit

Description

Limits the number of results to the specified value.

Type

Integer

Valid Values

0 - 10,000

Required

No

delimiter

Description

The delimiter between the prefix and the rest of the object name.

Type

String

Valid Values

N/A

Required

No

path

Description

The pseudo-hierarchical path of the objects.

Type

String

Valid Values

N/A

Required

No

Response Entities

`container`

Description

The container.

Type

Container

`object`

Description

An object within the container.

Type

Container

`name`

Description

The name of an object within the container.

Type

String

`hash`

Description

A hash code of the object's contents.

Type

String

`last_modified`

Description

The last time the object's contents were modified.

Type

Date

`content_type`

Description

The type of content within the object.

Type

String

Swift create a container

To create a new container, make a `PUT` request with the API version, account, and the name of the new container. The container name must be unique, must not contain a forward-slash (/) character, and should be less than 256 bytes. You can include access control headers and metadata headers in the request. You can also

include a storage policy identifying a key for a set of placement pools. For example, run `radosgw-admin zone get` to see a list of available keys under `placement_pools`. A storage policy enables you to specify a special set of pools for the container, for example, SSD-based storage. The operation is idempotent. If you make a request to create a container that already exists, it will return with a HTTP 202 return code, but will not create another container.

Syntax

```
PUT /_AP_VERSION_/_ACCOUNT_/_TENANT_:_CONTAINER_ HTTP/1.1
Host: FULLY_QUALIFIED_DOMAIN_NAME
X-Auth-Token: _AUTH_TOKEN_
X-Container-Read: _COMMA_SEPARATED_UIDS_
X-Container-Write: _COMMA_SEPARATED_UIDS_
X-Container-Meta-_KEY_:VALUE
X-Storage-Policy: _PLACEMENT_POOLS_KEY_
```

Headers

`X-Container-Read`

Description

The user IDs with read permissions for the container.

Type

Comma-separated string values of user IDs.

Required

No

`X-Container-Write`

Description

The user IDs with write permissions for the container.

Type

Comma-separated string values of user IDs.

Required

No

`X-Container-Meta-_KEY`

Description

A user-defined metadata key that takes an arbitrary string value.

Type

String

Required

No

`X-Storage-Policy`

Description

The key that identifies the storage policy under `placement_pools` for the Ceph Object Gateway. Run `radosgw-admin zone get` for available keys.

Type

String

Required

No

If a container with the same name already exists, and the user is the container owner then the operation will succeed. Otherwise, the operation will fail.

HTTP Response

409

Status Code

BucketAlreadyExists

Description

The container already exists under a different user's ownership.

Swift delete a container

To delete a container, make a DELETE request with the API version, account, and the name of the container. The container must be empty. If you'd like to check if the container is empty, run a HEAD request against the container. Once you've successfully removed the container, you'll be able to reuse the container name.

Syntax

```
DELETE /_AP_VERSION_/_ACCOUNT_/_TENANT_:_CONTAINER_ HTTP/1.1
Host: FULLY_QUALIFIED_DOMAIN_NAME
X-Auth-Token: _AUTH_TOKEN_
```

HTTP Response

204

Status Code

NoContent

Description

The container was removed.

Swift add or update the container metadata

To add metadata to a container, make a POST request with the API version, account, and container name. You must have write permissions on the container to add or update metadata.

Syntax

```
POST /_AP_VERSION_/_ACCOUNT_/_TENANT_:_CONTAINER_ HTTP/1.1
Host: FULLY_QUALIFIED_DOMAIN_NAME
X-Auth-Token: _AUTH_TOKEN_
X-Container-Meta-Color: red
X-Container-Meta-Taste: salty
```

Request Headers

X-Container-Meta-_KEY

Description

A user-defined metadata key that takes an arbitrary string value.

Type

String

Required

No

Swift object operations

Swift object operations provide mechanisms to create, retrieve, update, and delete data objects and manage object metadata through the Ceph Object Gateway.

To use these APIs a running Red Hat Ceph Storage cluster and REST client are required.

An object stores data and its associated metadata within a container. A container can hold multiple objects, but object names within the same container must be unique. Through the Ceph Object Gateway Swift application programming interface (API), you can list, create, update, and delete objects, as well as manage object metadata and access controls.

Because the Swift API processes requests that access account information, all requests must be authenticated unless the container or object access control is explicitly configured for public or anonymous access.

Swift get an object

To retrieve an object, make a `GET` request with the API version, account, container, and object name. You must have read permissions on the container to retrieve an object within it.

Syntax

```
GET /_AP_VERSION_/_ACCOUNT_/_TENANT_:_CONTAINER_/_OBJECT_ HTTP/1.1
Host: FULLY_QUALIFIED_DOMAIN_NAME
X-Auth-Token: _AUTH_TOKEN_
```

Request Headers

`range`

Description

To retrieve a subset of an object's contents, you can specify a byte range.

Type

Date

Required

No

`If-Modified-Since`

Description

Only copies if modified since the date and time of the source object's `last_modified` attribute.

Type

Date

Required

No

`If-Unmodified-Since`

Description

Only copies if not modified since the date and time of the source object's `last_modified` attribute.

Type

Date

Required

No

`Copy-If-Match`

Description

Copies only if the ETag in the request matches the source object's ETag.

Type

ETag

Required

No

`Copy-If-None-Match`

Description

Copies only if the `ETag` in the request does not match the source object's ETag.

Type

ETag

Required

No

Response Headers

`Content-Range`

Description

The range of the subset of object contents. Returned only if the range header field was specified in the request.

Swift create or update an object

To create a new object, make a `PUT` request with the API version, account, container name, and the name of the new object. You must have write permission on the container to create or update an object. The object name must be unique within the container. The `PUT` request is not idempotent, so if you do not use a unique name, the request will update the object. However, you can use pseudo-hierarchical syntax in the object name to distinguish it from another object of the same name if it is under a different pseudo-hierarchical directory. You can include access control headers and metadata headers in the request.

Syntax

```
PUT /_AP_VERSION_/_ACCOUNT_/_TENANT_:_CONTAINER_ HTTP/1.1
Host: FULLY_QUALIFIED_DOMAIN_NAME
X-Auth-Token: _AUTH_TOKEN_
```

Request Headers

`ETag`

Description

An MD5 hash of the object's contents. Recommended.

Type

String

Valid Values

N/A

Required

No

`Content-Type`

Description

An MD5 hash of the object's contents.

Type

String

Valid Values

N/A

Required

No

`Transfer-Encoding`

Description

Indicates whether the object is part of a larger aggregate object.

Type

String

Valid Values

`chunked`

Required

No

Swift delete an object

To delete an object, make a `DELETE` request with the API version, account, container, and object name. You must have write permissions on the container to delete an object within it. Once you've successfully deleted the object, you will be able to reuse the object name.

Syntax

```
DELETE /_API_VERSION_/_ACCOUNT_/_TENANT_:_CONTAINER_/_OBJECT_ HTTP/1.1
Host: FULLY_QUALIFIED_DOMAIN_NAME
X-Auth-Token: _AUTH_TOKEN_
```

Swift copy an object

Copying an object allows you to make a server-side copy of an object, so that you do not have to download it and upload it under another container. To copy the contents of one object to another object, you can make either a `PUT` request or a `COPY` request with the API version, account, and the container name.

For a `PUT` request, use the destination container and object name in the request, and the source container and object in the request header.

For a `Copy` request, use the source container and object in the request, and the destination container and object in the request header. You must have write permission on the container to copy an object. The destination object name must be unique within the container. The request is not idempotent, so if you do not use a unique name, the request will update the destination object. You can use pseudo-hierarchical syntax in the object name to distinguish the destination object from the source object of the same name if it is under a different pseudo-hierarchical directory. You can include access control headers and metadata headers in the request.

Syntax

```
PUT /_AP_VERSION_/_ACCOUNT_/_TENANT_:_CONTAINER_ HTTP/1.1
X-Copy-From: _TENANT_:_SOURCE_CONTAINER_/_SOURCE_OBJECT_
Host: FULLY_QUALIFIED_DOMAIN_NAME
X-Auth-Token: _AUTH_TOKEN_
```

or alternatively:

Syntax

```
COPY /_AP_VERSION_/_ACCOUNT_/_TENANT_:_SOURCE_CONTAINER_/_SOURCE_OBJECT_ HTTP/1.1
Destination: _TENANT_:_DEST_CONTAINER_/_DEST_OBJECT_
```

Request Headers

`X-Copy-From`

Description

Used with a `PUT` request to define the source container/object path.

Type

String

Required

Yes, if using `PUT`.

`Destination`

Description

Used with a `COPY` request to define the destination container/object path.

Type

String

Required

Yes, if using `COPY`.

`If-Modified-Since`

Description

Only copies if modified since the date and time of the source object's `last_modified` attribute.

Type

Date

Required

No

`If-Unmodified-Since`

Description

Only copies if not modified since the date and time of the source object's `last_modified` attribute.

Type

Date

Required

No

`Copy-If-Match`

Description

Copies only if the ETag in the request matches the source object's ETag.

Type

ETag

Required

No

`Copy-If-None-Match`

Description

Copies only if the `ETag` in the request does not match the source object's ETag.

Type

ETag

Required

No

Swift get object metadata

To retrieve an object's metadata, make a `HEAD` request with the API version, account, container, and object name. You must have read permissions on the container to retrieve metadata from an object within the container. This request returns the same header information as the request for the object itself, but it does not return the object's data.

Syntax

```
HEAD /_AP_VERSION_/_ACCOUNT_/_TENANT_:_CONTAINER_/_OBJECT_ HTTP/1.1
Host: FULLY_QUALIFIED_DOMAIN_NAME
X-Auth-Token: _AUTH_TOKEN_
```

Swift add or update object metadata

To add metadata to an object, make a `POST` request with the API version, account, container, and object name. You must have write permissions on the parent container to add or update metadata.

Syntax

```
POST /_AP_VERSION_/_ACCOUNT_/_TENANT_:_CONTAINER_/_OBJECT_ HTTP/1.1
Host: FULLY_QUALIFIED_DOMAIN_NAME
X-Auth-Token: _AUTH_TOKEN_
```

Request Headers

`X-Object-Meta-_KEY`

Description

A user-defined meta data key that takes an arbitrary string value.

Type

String

Required

No

Swift temporary URL operations

To allow temporary access, temp url functionality is supported by swift endpoint of `radosgw`. For example GET requests, to objects without the need to share credentials.

For this functionality, initially the value of `X-Account-Meta-Temp-URL-Key` and optionally `X-Account-Meta-Temp-URL-Key-2` should be set. The Temp URL functionality relies on a HMAC-SHA1 signature against these secret keys.

Swift get temporary URL objects

Temporary URL uses a cryptographic HMAC-SHA1 signature, which includes the following elements:

- The value of the Request method, `GET` for instance.
- The expiry time, in the format of seconds since the epoch, that is, Unix time.
- The request path that starts from `v1` onwards.

The items are normalized with newlines that are appended between them. An HMAC is generated that uses the SHA-1 hashing algorithm against one of the Temp URL Keys posted earlier.

The following example is a sample python script to demonstrate this.

Example

```
import hmac
from hashlib import sha1
from time import time

method = 'GET'
host = 'https://objectstore.example.com'
duration_in_seconds = 300  # Duration for which the url is valid
expires = int(time() + duration_in_seconds)
path = '/v1/your-bucket/your-object'
key = 'secret'
hmac_body = '%s\n%s\n%s' % (method, expires, path)
hmac_body = hmac.new(key, hmac_body, sha1).hexdigest()
sig = hmac.new(key, hmac_body, sha1).hexdigest()
rest_uri = "{host}{path}?temp_url_sig={sig}&temp_url_expires={expires}".format(
     host=host, path=path, sig=sig, expires=expires)
print rest_uri
```

Example Output

```
https://objectstore.example.com/v1/your-bucket/your-object?
temp_url_sig=ff4657876227fc6025f04fcf1e82818266d022c6&temp_url_expires=1423200992
```

Swift POST temporary URL keys

A `POST` request to the swift account with the required Key will set the secret temp URL key for the account against which temporary URL access can be provided to accounts. Up to two keys are supported, and signatures are checked against both the keys, if present, so that keys can be rotated without invalidating the temporary URLs.

Syntax

```
POST /API_VERSION/ACCOUNT HTTP/1.1
Host: FULLY_QUALIFIED_DOMAIN_NAME
X-Auth-Token: AUTH_TOKEN
```

Request Headers

`X-Account-Meta-Temp-URL-Key`

Description

A user-defined key that takes an arbitrary string value.

Type

String

Required

Yes

`X-Account-Meta-Temp-URL-Key-2`

Description

A user-defined key that takes an arbitrary string value.

Type

String

Required

No

Swift multi-tenancy container operations

When a client application accesses containers, it always operates with credentials of a particular user. In a Red Hat Ceph Storage cluster, every user belongs to a tenant. Consequently, every container operation has an implicit tenant in its context if no tenant is specified explicitly. Thus multi-tenancy is completely backward compatible with previous releases, as long as the referred containers and referring user belong to the same tenant.

Extensions employed to specify an explicit tenant differ according to the protocol and authentication system used.

A colon character separates tenant and container, thus a sample URL would be:

Example

```
https://rgw.domain.com/tenant:container
```

By contrast, in a `create_container()` method, simply separate the tenant and container in the container method itself:

Example

```
create_container("tenant:container")
```

Ceph RESTful API specifications

As a storage administrator, you can access the various Ceph sub-systems through the Ceph RESTful API endpoints. This is a reference section for the available Ceph RESTful API methods.

Before you begin, make sure that you have the following prerequisites in place:

- An understanding of how to use a RESTful API.
- A healthy running Red Hat Ceph Storage cluster.
- The Ceph Manager `dashboard` module is enabled.

S3 common request headers

The following table lists the valid common request headers and their descriptions.

Table 201: Request Headers

Request Header	Description
`CONTENT_LENGTH`	Length of the request body.
`DATE`	Request time and date (in UTC).
`HOST`	The name of the host server.
`AUTHORIZATION`	Authorization token.

S3 common response status codes

The following table lists the valid common HTTP response status and its corresponding code.

Table 202: Response Status

HTTP Status	Response Code
`100`	Continue
`200`	Success
`201`	Created
`202`	Accepted
`204`	NoContent
`206`	Partial content
`304`	NotModified
`400`	InvalidArgument
`400`	InvalidDigest
`400`	BadDigest
`400`	InvalidBucketName
`400`	InvalidObjectName
`400`	UnresolvableGrantByEmailAddress
`400`	InvalidPart

HTTP Status	Response Code
400	InvalidPartOrder
400	RequestTimeout
400	EntityTooLarge
403	AccessDenied
403	UserSuspended
403	RequestTimeTooSkewed
404	NoSuchKey
404	NoSuchBucket
404	NoSuchUpload
405	MethodNotAllowed
408	RequestTimeout
409	BucketAlreadyExists
409	BucketNotEmpty
411	MissingContentLength
412	PreconditionFailed
416	InvalidRange
422	UnprocessableEntity
500	InternalError

S3 supported and unsupported verbs

Use this information to know the latest supported and unsupported S3 verbs.

- Supported verbs: lists the S3 API verbs that are fully supported by Red Hat Ceph Storage Object Gateway and are available for use in customer applications and integrations
- Unsupported verbs: lists the S3 API verbs that are not currently supported by Red Hat Ceph Storage Object Gateway and cannot be used.

Table 203: Supported verbs

Action	API
AbortMultipartUpload	S3
CompleteMultipartUpload	S3
CopyObject	S3
CreateBucket	S3
CreateMultipartUpload	S3
DeleteBucket	S3
DeleteBucketCORS	S3
DeleteBucketEncryption	S3

Action	API
DeleteBucketLifecycle	S3
DeleteBucketPolicy	S3
DeleteBucketReplication	S3
DeleteBucketTagging	S3
DeleteBucketWebsite	S3
DeleteObject	S3
DeleteObjects	S3
DeleteObjectTagging	S3
DeletePublicAccessBlock	S3
GetBucketAcl	S3
GetBucketCORS	S3
GetBucketEncryption	S3
GetBucketLocation	S3
GetBucketLogging	S3
GetBucketNotificationConfiguration	S3
GetBucketPolicy	S3
GetBucketPolicyStatus	S3
GetBucketReplication	S3
GetBucketRequestPayment	S3
GetBucketTagging	S3
GetBucketVersioning	S3
GetBucketWebsite	S3
GetObject	S3
GetObjectAcl	S3
GetObjectAttributes	S3
GetObjectLegalHold	S3
GetObjectLockConfiguration	S3
GetObjectRetention	S3
GetObjectTagging	S3
GetObjectTorrent	S3
GetPublicAccessBlock	S3
HeadBucket	S3
HeadObject	S3
ListBuckets	S3
ListMultipartUploads	S3
ListObjects	S3
ListObjectsV2	S3
ListObjectVersions	S3
ListParts	S3
PutBucketAcl	S3
PutBucketCORS	S3
PutBucketEncryption	S3

Action	API
PutBucketLifecycle	S3
PutBucketLifecycleConfiguration	S3
PutBucketLogging	S3
PutBucketNotificationConfiguration	S3
PutBucketPolicy	S3
PutBucketReplication	S3
PutBucketRequestPayment	S3
PutBucketTagging	S3
PutBucketVersioning	S3
PutBucketWebsite	S3
PutObject	S3
PutObjectAcl	S3
PutObjectLegalHold	S3
PutObjectLockConfiguration	S3
PutObjectRetention	S3
PutObjectTagging	S3
PutPublicAccessBlock	S3
RestoreObject	S3
SelectObjectContent	S3
UploadPart	S3
UploadPartCopy	S3
AssumeRole	STS
AssumeRoleWithWebIdentity	STS
GetSessionToken	STS
AddClientIdToOpenIDConnectProvider	IAM
AttachGroupPolicy	IAM
AttachRolePolicy	IAM
AttachUserPolicy	IAM
CreateOpenIDConnectProvider	IAM
CreateRole	IAM
DeleteOpenIDConnectProvider	IAM
DeleteRole	IAM
GetAccountSummary	IAM
GetOpenIDConnectProvider	IAM
GetRole	IAM
ListOpenIDConnectProviders	IAM
ListRoles	IAM
UpdateOpenIDConnectProviderThumbprint	IAM
DeleteBucketNotification	S3 extension
CreateTopic	SNS
GetTopicAttributes	SNS
DeleteTopic	SNS

Action	API
ListTopics	SNS
GetCallerIdentity	STS

Table 204: Unsupported verbs

Action	API
DeleteBucketInventoryConfiguration	S3
DeleteIntelligentTieringConfiguration	S3
GetBucketInventoryConfiguration	S3
GetIntelligentTieringConfiguration	S3
ListBucketIntelligentTieringConfigurations	S3
ListBucketInventoryConfigurations	S3
PutBucketIntelligentTieringConfiguration	S3
PutBucketInventoryConfiguration	S3
AssumeRoleWithSAML	STS
DecodeAuthorizationMessage	STS
GetAccessKeyInfo	STS
GetFederationToken	STS
CreatePolicy	IAM
CreateSAMLProvider	IAM
DeletePolicy	IAM
DeleteSAMLProviders	IAM
GetPolicy	IAM
GetSAMLProviders	IAM
ListPolicies	IAM
ListSAMLProviders	IAM

S3 unsupported header fields

Use this information to get to know S3 unsupported header fields.

Table 205: Unsupported header fields

Name	Type
x-amz-security-token	Request
Server	Response
x-amz-delete-marker	Response
x-amz-id-2	Response
x-amz-request-id	Response
x-amz-version-id	Response

Swift request headers

Table 206: Request headers			
Name	**Description**	**Type**	**Required**
`X-Auth-User`	The key Ceph Object Gateway username to authenticate.	String	Yes
`X-Auth-Key`	The key associated to a Ceph Object Gateway username.	String	Yes

Swift response headers

The response from the server should include an `X-Auth-Token` value. The response might also contain a `X-Storage-Url` that provides the `API_VERSION_/_ACCOUNT` prefix that is specified in other requests throughout the API documentation.

Table 207: Response Headers		
Name	**Description**	**Type**
`X-Storage-Token`	The authorization token for the `X-Auth-User` specified in the request.	String
`X-Storage-Url`	The URL and `_API_VERSION_/_ACCOUNT_` path for the user.	String

Examples using the Secure Token Service APIs

These examples are using Python's `boto3` module to interface with the Ceph Object Gateway's implementation of the Secure Token Service (STS). In these examples, `TESTER2` assumes a role created by `TESTER1`, as to access S3 resources owned by `TESTER1` based on the permission policy attached to the role.

The *AssumeRole* example creates a role, assigns a policy to the role, then assumes a role to get temporary credentials and access to S3 resources using those temporary credentials.

The *AssumeRoleWithWebIdentity* example authenticates users using an external application with Keycloak, an OpenID Connect identity provider, assumes a role to get temporary credentials and access S3 resources according to the permission policy of the role.

AssumeRole Example

```
import boto3

iam_client = boto3.client(iam,
aws_access_key_id=ACCESS_KEY_OF_TESTER1,
aws_secret_access_key=SECRET_KEY_OF_TESTER1,
endpoint_url=<IAM URL>,
region_name='
)

policy_document = "{\"Version\":\"2012-10-17\",\"Statement\":[{\"Effect\":\"Allow\",
\"Principal\":{\"AWS\":[\"arn:aws:iam:::user/TESTER1\"]},\"Action\":
[\"sts:AssumeRole\"]}]}"

role_response = iam_client.create_role(
AssumeRolePolicyDocument=policy_document,
Path=/,
RoleName='S3Access,
)

role_policy = "{\"Version\":\"2012-10-17\",\"Statement\":{\"Effect\":\"Allow\",\"Action\":
\"s3:*\",\"Resource\":\"arn:aws:s3:::*\"}}"

response = iam_client.putROLEpolicy(
```

```
RoleName=S3Access,
PolicyName=Policy1,
PolicyDocument=role_policy
)

sts_client = boto3.client(sts,
aws_access_key_id=ACCESS_KEY_OF_TESTER2,
aws_secret_access_key=SECRET_KEY_OF_TESTER2,
endpoint_url=<STS URL>,
region_name=',
)

response = sts_client.assume_role(
RoleArn=role_response['Role][Arn],
RoleSessionName=Bob,
DurationSeconds=3600
)

s3client = boto3.client(s3,
aws_access_key_id = response[Credentials][AccessKeyId],
aws_secret_access_key = response[Credentials][SecretAccessKey],
aws_session_token = response[Credentials][SessionToken],
endpoint_url=<S3 URL>,
region_name=',)

bucket_name = 'my-bucket
s3bucket = s3client.create_bucket(Bucket=bucket_name)
resp = s3client.list_buckets()
```

AssumeRoleWithWebIdentity Example

```
import boto3

iam_client = boto3.client('iam',
aws_access_key_id=ACCESS_KEY_OF_TESTER1,
aws_secret_access_key=SECRET_KEY_OF_TESTER1,
endpoint_url=<IAM URL>,
region_name=''
)

oidc_response = iam_client.create_openIDconnect_provider(
    Url=<URL of the OpenID Connect Provider>,
    ClientIDList=[
        <Client id registered with the IDP>
    ],
    ThumbprintList=[
        <IDP THUMBPRINT>
 ]
)

policy_document = "{\"Version\":\"2012-10-17\",\"Statement\":\[\{\"Effect\":\"Allow\",
\"Principal\":\{\"Federated\":\[\"arn:aws:iam:::oidc-provider/localhost:8080/auth/realms/
demo\"\]\},\"Action\":\[\"sts:AssumeRoleWithWebIdentity\"\],\"Condition\":\
{\"StringEquals\":\{\"localhost:8080/auth/realms/demo:app_id\":\"customer-portal\"\}\}\}\]
\}"
role_response = iam_client.create_role(
AssumeRolePolicyDocument=policy_document,
Path='/',
RoleName='S3Access',
)

role_policy = "{\"Version\":\"2012-10-17\",\"Statement\":{\"Effect\":\"Allow\",\"Action\":
\"s3:*\",\"Resource\":\"arn:aws:s3:::*\"}}"

response = iam_client.put_role_policy(
    RoleName='S3Access',
    PolicyName='Policy1',
    PolicyDocument=role_policy
)

sts_client = boto3.client('sts',
aws_access_key_id=ACCESS_KEY_OF_TESTER2,
aws_secret_access_key=SECRET_KEY_OF_TESTER2,
endpoint_url=<STS URL>,
region_name='',
)

response = sts_client.assume_role_with_web_identity(
RoleArn=role_response['Role']['Arn'],
RoleSessionName='Bob',
```

```
DurationSeconds=3600,
WebIdentityToken=<Web Token>
)

s3client = boto3.client(s3,
aws_access_key_id = response[Credentials][AccessKeyId],
aws_secret_access_key = response[Credentials][SecretAccessKey],
aws_session_token = response[Credentials][SessionToken],
endpoint_url=<S3 URL>,
region_name=',)

bucket_name = 'my-bucket
s3bucket = s3client.create_bucket(Bucket=bucket_name)
resp = s3client.list_buckets()
```

Reference

For more information about using Python's boto module, see Test S3 access.

Red Hat Ceph Storage Troubleshooting

Troubleshoot and resolve common problems with Red Hat Ceph Storage.

Initial troubleshooting

As a storage administrator, you can do the initial troubleshooting of a Red Hat Ceph Storage cluster before contacting Red Hat Support.

Initial troubleshooting requires a running Red Hat Ceph Storage cluster.

For more information about contacting Red Hat Support, see .

Identifying problems

Identify any possible causes of errors within the Red Hat Ceph Storage cluster.

Before you begin

- A running Red Hat Ceph Storage cluster.

Procedure

Determine possible causes of the error with the Red Hat Ceph Storage cluster, by answering the following questions.

1. Certain problems can arise when using unsupported configurations.
   Ensure that your configuration is supported. For more information, see Compatibility matrix.
2. Do you know which Ceph component caused the problem?
   - No.
     See “Diagnosing health” on page 1579.
   - Ceph Monitors.
     See “Troubleshooting Ceph Monitors” on page 1599.
   - Ceph OSDs.
     See “Troubleshooting Ceph OSDs” on page 1609.
   - Ceph placement groups.
     See “Troubleshooting Ceph placement groups” on page 1628.
   - Multi-site Ceph Object Gateway.
     See “Troubleshooting multi-site Ceph Object Gateway” on page 1624.

What to do next

For more information, see Red Hat Ceph Storage: Supported configurations, on Red Hat Customer Portal.

Diagnosing health

Learn how to run a basic diagnostic of the health of an Red Hat Ceph Storage cluster.

Before you begin

- A running Red Hat Ceph Storage cluster.

Procedure

1. Log in to the `cephadm` shell.

   ```
   cephadm shell
   ```

2. Check the overall status of the storage cluster.

For example,

```
[ceph: root@host01 /]# ceph health detail
```

If the command returns a HEALTH_WARN or HEALTH_ERR message, see “Understanding Ceph health” on page 1580.

3. Monitor the logs of the storage cluster.
For example,

```
[ceph: root@host01 /]# ceph -w cephadm
```

4. Run the following commands to capture the logs of the cluster to a file.

```
ceph config set global log_to_file true
```

```
ceph config set global mon_cluster_log_to_file true
```

By default, the logs are located in the `/var/log/ceph/` directory.

5. Check the Ceph logs for any error messages that are listed in “Understanding Ceph logs” on page 1582.
6. If the logs do not include enough information, increase the debugging level and try to reproduce the action that failed.
For more information, see “Configuring logging at runtime” on page 1586.

Understanding Ceph health

Learn about the different Ceph health messages.

The **`ceph health`** command returns information about the status of the Red Hat Ceph Storage cluster.

HEALTH_OK

Indicates that the cluster is healthy.

HEALTH_WARN

Indicates a warning. Sometimes the Ceph status returns to HEALTH_OK automatically such as when Red Hat Ceph Storage cluster finishes the rebalancing process.
Consider further troubleshooting if a cluster is in the HEALTH_WARN state for an extended time.

HEALTH_ERR

Indicates a more serious problem that requires your immediate attention.

Use the `ceph health detail` and `ceph -s` commands to get a more detailed output.

> **Note:** A health warning is displayed if there is no `mgr` daemon running. In case the last `mgr` daemon of an Red Hat Ceph Storage cluster was removed, you can manually deploy a `mgr` daemon on a random host of the Red Hat Ceph Storage cluster.
> For more information, see Manually deploying a mgr daemon.

For more information, see “Ceph Monitor error messages” on page 1600, “Ceph OSD error messages” on page 1610, and “Placement group error messages” on page 1629.

Muting health alerts

In certain scenarios, users might want to temporarily mute some warnings because they are already aware of the warning and cannot act on it immediately. You can mute health checks so that they do not affect the overall reported status of the Ceph cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.

- Root-level of access to the nodes.
- A health warning message.

About this task

Alerts are specified by using the health check codes. One example is, when an OSD is brought down for maintenance, OSD_DOWN warnings are expected. You can choose to mute the warning until the maintenance is over because those warnings put the cluster in HEALTH_WARN instead of HEALTH_OK for the entire duration of maintenance.

Most health mutes also disappear if the extent of an alert gets worse. For example, if there is one OSD down, and the alert is muted, the mute disappears if one or more additional OSDs go down. This is true for any health alert that involves a count indicating how much or how many of something is triggering the warning or error.

For more information, see “Health messages of a Ceph cluster” on page 1657.

Procedure

1. Log in to the `cephadm` shell.

   ```
   cephadm shell
   ```

2. Check the health of the Red Hat Ceph Storage cluster by running the **`ceph health detail`** command. For example,

   ```
   [ceph: root@host01 /]# ceph health detail

    HEALTH_WARN 1 osds down; 1 OSDs or CRUSH {nodes, device-classes} have
   {NOUP,NODOWN,NOIN,NOOUT} flags set
    [WRN] OSD_DOWN: 1 osds down
        osd.1 (root=default,host=host01) is down
    [WRN] OSD_FLAGS: 1 OSDs or CRUSH {nodes, device-classes} have
   {NOUP,NODOWN,NOIN,NOOUT} flags set
        osd.1 has flags noup
   ```

   In this example, the storage cluster is in *HEALTH_WARN* status, indicated as one of the OSDs is down.

3. Mute the alert.

   ```
   ceph health mute HEALTH_MESSAGE
   ```

   The following example mutes the OSD_DOWN message.

   ```
   [ceph: root@host01 /]# ceph health mute OSD_DOWN
   ```

   Note: A health check mute can have a time to live (TTL) associated with it, such that the mute automatically expires after the specified time is over. You can optionally specify the TTL as an optional duration argument in the command.

   ```
   ceph health mute HEALTH_MESSAGE DURATION
   ```

   DURATION can be specified in any of the following value types: `s`, `sec`, `m`, `min`, `h`, or `hour`.
   The following example mutes the OSD_DOWN message for 10 minutes:

   ```
   [ceph: root@host01 /]# ceph health mute OSD_DOWN 10m
   ```

4. Verify that the Red Hat Ceph Storage cluster status has changed to HEALTH_OK.
   The following example shows that the alerts OSD_DOWN and OSD_FLAG are muted and the mute is active for nine minutes.

```
[ceph: root@host01 /]# ceph -s
   cluster:
     id:     81a4597a-b711-11eb-8cb8-001a4a000740
     health: HEALTH_OK
             (muted: OSD_DOWN(9m) OSD_FLAGS(9m))

   services:
     mon: 3 daemons, quorum host01,host02,host03 (age 33h)
     mgr: host01.pzhfuh(active, since 33h), standbys: host02.wsnngf, host03.xwzphg
     osd: 11 osds: 10 up (since 4m), 11 in (since 5d)

   data:
     pools:   1 pools, 1 pgs
     objects: 13 objects, 0 B
     usage:   85 MiB used, 165 GiB / 165 GiB avail
     pgs:     1 active+clean
```

5. Optional: Retain the mute even after the alert is cleared by making it **sticky**.

```
ceph health mute HEALTH_MESSAGE DURATION --sticky
```

For example,

```
[ceph: root@host01 /]# ceph health mute OSD_DOWN 1h --sticky
```

What to do next

Remove the mute by running the following command:

```
ceph health unmute HEALTH_MESSAGE
```

For example,

```
[ceph: root@host01 /]# ceph health unmute OSD_DOWN
```

Understanding Ceph logs

Ceph stores its logs in the `/var/log/ceph/` directory after the logging is enabled.

The `CLUSTER_NAME.log` is the main storage cluster log file that includes global events. By default, the log file name is `ceph.log`. Only the Ceph Monitor nodes include the main storage cluster log.

Each Ceph OSD and Monitor has its own log file named `CLUSTER_NAME-osd.NUMBER.log` and `CLUSTER_NAME-mon.HOSTNAME.log` respectively.

When you increase debugging level for Ceph subsystems, Ceph generates new log files for those subsystems as well.

For more information about logging, see “Configuring logging” on page 1584.

For more information about error messages, see “Common Ceph Monitor error messages in Ceph logs” on page 1600 and “Common Ceph OSD error messages in Ceph logs” on page 1610.

Generating an `sos report`

Run the **`sos report`** command to collect the configuration details, system information, and diagnostic information of an Red Hat Ceph Storage cluster from a Red Hat Enterprise Linux system.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the nodes.

About this task

Red Hat Support uses the logs that are generated for further troubleshooting of the storage cluster.

For more information, see the *What is an sos report and how to create one in Red Hat Enterprise Linux?* KnowledgeBase article.

Procedure

1. Install the `sos` package.
   For example,

   ```
   [root@host01 ~]# dnf install sos
   ```

2. Run the **`sos report`** command to get the system information of the storage cluster.

   ```
   sos report -a --all-logs
   ```

   For example,

   ```
   [root@host01 ~]# sos report -a --all-logs
   ```

   The report is saved in the `/var/tmp` file.
   Run the following command for specific Ceph daemon information.

   ```
   sos report --all-logs -e CEPH_SERVICE1,CEPH_SERVICE2
   ```

   For example,

   ```
   [root@host01 ~]# sos report --all-logs -e
   ceph_mgr,ceph_common,ceph_mon,ceph_osd,ceph_ansible,ceph_mds,ceph_rgw
   ```

Providing information for support

To get help troubleshooting Red Hat Ceph Storage problems, contact Red Hat Support and provide enough information for the support engineers to diagnose the issue.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root-level access to the node.
- Red Hat Support account.

Procedure

1. Open a support ticket with Red Hat Support.
2. Attach an `sos report` to the ticket.
   For more information, see Generating an sos report.
3. Optional: Generate a readable core dump file.
   If the Ceph daemons fail with a segmentation fault, a core dump file can help diagnose the problem. For more information, see Generating readable core dump files.

Generating readable core dump files

When a Ceph daemon terminates unexpectedly with a segmentation fault, gather the information about its failure and provide it to the IBM Support Engineers.

Before you begin

Before generating the readable core dump files, enable and install the debuginfo packages, if they are not already installed.

```
subscription-manager repos --enable=ibmceph-9-tools-for-rhel-9-x86_64-rpms
yum --enable=ibmceph-9-tools-for-rhel-9-x86_64-debug-rpms
```

Once the repository is enabled, you can install the debug info packages that you need from this list of supported packages:

```
ceph-base-debuginfo
ceph-common-debuginfo
ceph-debugsource
ceph-fuse-debuginfo
ceph-immutable-object-cache-debuginfo
ceph-mds-debuginfo
ceph-mgr-debuginfo
ceph-mon-debuginfo
ceph-osd-debuginfo
ceph-radosgw-debuginfo
cephfs-mirror-debuginfo
```

About this task

Gathering the information about failures speeds up the initial investigation. The Support Engineers can also compare the information from the core dump files with Red Hat Ceph Storage cluster known issues.

Procedure

- Ensure that the `gdb` package is installed.
  Example

```
[root@host01 ~]# dnf install gdb
```

Configuring logging

Learn how to configure logging for various Ceph subsystems.

> **Important:** Logging is resource-intensive. Also, verbose logging can generate a huge amount of data in a relatively short time. If you are encountering problems in a specific subsystem of the cluster, enable logging only of that subsystem. For more information, see "Ceph subsystems" on page 1584.
> Consider also setting up a rotation of log files. For more information, see "Accelerating log rotation" on page 1588.
>
> Once you fix any problems you encounter, change the subsystems log and memory levels to their default values. For a list of all Ceph subsystems and their default values, see "Ceph subsystems default logging level values" on page 1655.

You can configure Ceph logging by:

- Using the `ceph` command at runtime. This is the most common approach. For more information, see "Configuring logging at runtime" on page 1586.
- Updating the Ceph configuration file. Use this approach if you are encountering problems when starting the cluster. For more information, see "Configuring logging in configuration file" on page 1587.

> **Note:** When configuring logging, be sure to have a running Red Hat Ceph Storage cluster.

Ceph subsystems

This section contains information about Ceph subsystems and their logging levels.

Understanding Ceph subsystems and their logging levels

Ceph consists of several subsystems.

Each subsystem has a logging level:

- Output logs that are stored by default in `/var/log/ceph/` directory (log level)
- Logs that are stored in a memory cache (memory level)

In general, Ceph does not send logs stored in memory to the output logs unless:

- A fatal signal is raised

- An assert in source code is triggered
- You request it

You can set different values for each of these subsystems. Ceph logging levels operate on a scale of `1` to `20`, where `1` is terse and `20` is verbose.

Use a single value for the log level and memory level to set them both to the same value. For example, `debug_osd = 5` sets the debug level for the `ceph-osd` daemon to `5`.

To use different values for the output log level and the memory level, separate the values with a forward slash (`/`). For example, `debug_mon = 1/5` sets the debug log level for the `ceph-mon` daemon to `1` and its memory log level to `5`.

Ceph Subsystems and the Logging Default Values

Table 208: Default logging values of sub systems

Subsystem	Log Level	Memory Level	Description
`asok`	1	5	The administration socket
`auth`	1	5	Authentication
`client`	0	5	Any application or library that uses `librados` to connect to the cluster.
`bluestore`	1	5	The BlueStore OSD backend.
`journal`	1	5	The OSD journal
`mds`	1	5	The Metadata Servers
`monc`	0	5	The Monitor client handles communication between most Ceph daemons and Monitors
`mon`	1	5	Monitors
`ms`	0	5	The messaging system between Ceph components
`osd`	0	5	The OSD Daemons
`paxos`	0	5	The algorithm that Monitors use to establish a consensus
`rados`	0	5	Reliable Autonomic Distributed Object Store, a core component of Ceph
`rbd`	0	5	The Ceph Block Devices
`rgw`	1	5	The Ceph Object Gateway

Example Log Outputs

The following examples show the type of messages in the logs when you increase the verbosity for the Monitors and OSDs.

Monitor Debug Settings

```
debug_ms = 5
debug_mon = 20
debug_paxos = 20
debug_auth = 20
```

Example Log Output of Monitor Debug Settings

```
2022-05-12 12:37:04.278761 7f45a9afc700 10 mon.cephn2@0(leader).osd e322 e322: 2 osds: 2
up, 2 in
2022-05-12 12:37:04.278792 7f45a9afc700 10 mon.cephn2@0(leader).osd e322
 min_last_epoch_clean 322
2022-05-12 12:37:04.278795 7f45a9afc700 10 mon.cephn2@0(leader).log v1010106 log
2022-05-12 12:37:04.278799 7f45a9afc700 10 mon.cephn2@0(leader).auth v2877 auth
2022-05-12 12:37:04.278811 7f45a9afc700 20 mon.cephn2@0(leader) e1 sync_trim_providers
2022-05-12 12:37:09.278914 7f45a9afc700 11 mon.cephn2@0(leader) e1 tick
2022-05-12 12:37:09.278949 7f45a9afc700 10 mon.cephn2@0(leader).pg v8126 v8126: 64 pgs: 64
active+clean; 60168 kB data, 172 MB used, 20285 MB / 20457 MB avail
2022-05-12 12:37:09.278975 7f45a9afc700 10 mon.cephn2@0(leader).paxosservice(pgmap
7511..8126) maybe_trim trim_to 7626 would only trim 115 < paxos_service_trim_min 250
2022-05-12 12:37:09.278982 7f45a9afc700 10 mon.cephn2@0(leader).osd e322 e322: 2 osds: 2
up, 2 in
2022-05-12 12:37:09.278989 7f45a9afc700  5 mon.cephn2@0(leader).paxos(paxos active c
1028850..1029466) is_readable = 1 - now=2021-08-12 12:37:09.278990 lease_expire=0.000000
has v0 lc 1029466
....
2022-05-12 12:59:18.769963 7f45a92fb700  1 -- 192.168.0.112:6789/0 <== osd.1
192.168.0.114:6800/2801 5724 ==== pg_stats(0 pgs tid 3045 v 0) v1 ==== 124+0+0 (2380105412
0 0) 0x5d96300 con 0x4d5bf40
2022-05-12 12:59:18.770053 7f45a92fb700  1 -- 192.168.0.112:6789/0 -->
192.168.0.114:6800/2801 -- pg_stats_ack(0 pgs tid 3045) v1 -- ?+0 0x550ae00 con 0x4d5bf40
2022-05-12 12:59:32.916397 7f45a9afc700  0 mon.cephn2@0(leader).data_health(1) update_stats
avail 53% total 1951 MB, used 780 MB, avail 1053 MB
....
2022-05-12 13:01:05.256263 7f45a92fb700  1 -- 192.168.0.112:6789/0 -->
192.168.0.113:6800/2410 -- mon_subscribe_ack(300s) v1 -- ?+0 0x4f283c0 con 0x4d5b440
```

Example log output of OSD debug settings

```
debug_ms = 5
debug_osd = 20
```

```
2022-05-12 11:27:53.869151 7f5d55d84700  1 -- 192.168.17.3:0/2410 -->
192.168.17.4:6801/2801 -- osd_ping(ping e322 stamp 2021-08-12 11:27:53.869147) v2 -- ?+0
0x63baa00 con 0x578dee0
2022-05-12 11:27:53.869214 7f5d55d84700  1 -- 192.168.17.3:0/2410 -->
192.168.0.114:6801/2801 -- osd_ping(ping e322 stamp 2021-08-12 11:27:53.869147) v2 -- ?+0
0x638f200 con 0x578e040
2022-05-12 11:27:53.870215 7f5d6359f700  1 -- 192.168.17.3:0/2410 <== osd.1
192.168.0.114:6801/2801 109210 ==== osd_ping(ping_reply e322 stamp 2021-08-12
11:27:53.869147) v2 ==== 47+0+0 (261193640 0 0) 0x63c1a00 con 0x578e040
2022-05-12 11:27:53.870698 7f5d6359f700  1 -- 192.168.17.3:0/2410 <== osd.1
192.168.17.4:6801/2801 109210 ==== osd_ping(ping_reply e322 stamp 2021-08-12
11:27:53.869147) v2 ==== 47+0+0 (261193640 0 0) 0x6313200 con 0x578dee0
....
2022-05-12 11:28:10.432313 7f5d6e71f700  5 osd.0 322 tick
2022-05-12 11:28:10.432375 7f5d6e71f700 20 osd.0 322 scrub_random_backoff lost coin flip,
randomly backing off
2022-05-12 11:28:10.432381 7f5d6e71f700 10 osd.0 322 do_waiters -- start
2022-05-12 11:28:10.432383 7f5d6e71f700 10 osd.0 322 do_waiters -- finish
```

Reference

For more information, see “Configuring logging at runtime” on page 1586 and “Configuring logging in configuration file” on page 1587.

Configuring logging at runtime

Configure the logging of Ceph subsystems at system runtime to help troubleshoot any issues that might occur.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Access to Ceph debugger.

About this task

For more information, see:

- “Ceph subsystems” on page 1584
- “Configuring logging in configuration file” on page 1587
- Debugging and logging configuration options

Procedure

- Activate the Ceph debugging output `dout()`, at runtime.

  ```
  ceph tell TYPE.ID injectargs --debug-SUBSYSTEM VALUE [--NAME VALUE]
  ```

  Replace as follows:

  - *TYPE* with the type of Ceph daemons (`osd`, `mon`, or `mds`).
  - *ID* with a specific ID of the Ceph daemon. Alternatively, use `*` to apply the runtime setting to all daemons of a particular type.
  - *SUBSYSTEM* with a specific subsystem.
  - *VALUE* with a number from `1` to `20`, where `1` is terse and `20` is verbose.

  The following example shows setting the log level for the OSD subsystem on the OSD name `osd.0` to 0 and the memory level to 5.

  ```
  # ceph tell osd.0 injectargs --debug-osd 0/5
  ```

Seeing the configuration settings at runtime

Procedure

1. Log in to the host with a running Ceph daemon.
   For example, `ceph-osd` or `ceph-mon`.
2. Display the configuration.

   ```
   ceph daemon NAME config show | less
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph daemon osd.0 config show | less
   ```

Configuring logging in configuration file

Configure Ceph subsystems to log information, warning, and error messages to the log file.

About this task

You can specify the debugging level in the Ceph configuration file, which is found by default in `/etc/ceph/ceph.conf`.

For more information, see:

- “Ceph subsystems” on page 1584
- “Configuring logging at runtime” on page 1586
- Debugging and logging configuration options

Procedure

To activate the Ceph debugging output `dout()` at boot time, add the debugging settings to the Ceph configuration file.

1. For subsystems common to each daemon, add the settings under the `[global]` section.

2. For subsystems for particular daemons, add the settings under a daemon section, such as `[mon]`, `[osd]`, or `[mds]`.
For example,

```
[global]
            debug_ms = 1/5

    [mon]
            debug_mon = 20
            debug_paxos = 1/5
            debug_auth = 2

    [osd]
            debug_osd = 1/5
            debug_monc = 5/20

    [mds]
            debug_mds = 1
```

Accelerating log rotation

Accelerate log rotation by modifying the Ceph log rotation file.

Before you begin

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

About this task

Increasing debugging level for Ceph components might generate a huge amount of data. If you have almost full disks, you can accelerate log rotation by modifying the Ceph log rotation file at `/etc/logrotate.d/ceph-<fsid>`. The Cron job scheduler uses this file to schedule log rotation.

Procedure

1. Add the size setting after the rotation frequency to the log rotation file.

```
rotate 7
 weekly
 size SIZE
 compress
 sharedscripts
```

The following is an example to rotate a log file when it reaches 500 MB.

```
rotate 7
 weekly
 size 500 MB
 compress
 sharedscripts
 size 500M
```

> **Note:** The *SIZE* value can be expressed as *500 MB* or *500M*.

2. Open the `crontab` editor.
For example,

```
[root@mon ~]# crontab -e
```

3. Add an entry to check the `/etc/logrotate.d/ceph-<fsid>` file.
The following example shows how to instruct Cron to check the `/etc/logrotate.d/ceph-<fsid>` file every 30 minutes.

```
30 * * * * /usr/sbin/logrotate /etc/logrotate.d/ceph-d3bb5396-
c404-11ee-9e65-002590fc2a2e >/dev/null 2>&1
```

Creating and collecting operation logs for Ceph Object Gateway

Track user identities reliably by S3 request in all versions of the Ceph Object Gateway operation log.

About this task

User identity information is added to the operation log output. This is used to enable customers to access this information for auditing of S3 access.

Procedure

1. Find where the logs are located.

```
logrotate -f
```

For example,

```
[root@host01 ~]# logrotate -f
 /etc/logrotate.d/ceph-12ab345c-1a2b-11ed-b736-fa163e4f6220
```

2. List the logs within the specified location.

```
ll LOG_LOCATION
```

For example,

```
[root@host01 ~]# ll /var/log/ceph/12ab345c-1a2b-11ed-b736-fa163e4f6220
 -rw-r--r--. 1 ceph ceph    412 Sep 28 09:26 opslog.log.1.gz
```

3. Create a bucket.

```
/usr/local/bin/s3cmd mb s3://NEW_BUCKET_NAME
```

For example,

```
[root@host01 ~]# /usr/local/bin/s3cmd mb s3://bucket1
 Bucket `s3://bucket1` created
```

4. Collect the logs.

```
tail -f LOG_LOCATION/opslog.log
```

For example,

```
[root@host01 ~]# tail -f /var/log/ceph/12ab345c-1a2b-11ed-b736-fa163e4f6220/opslog.log

 {"bucket":"","time":"2022-09-29T06:17:03.133488Z","time_local":"2022-09-
 29T06:17:03.133488+0000","remote_addr":"10.0.211.66","user":"test1",
 "operation":"list_buckets","uri":"GET /
 HTTP/1.1","http_status":"200","error_code":"","bytes_sent":232,
 "bytes_received":0,"object_size":0,"total_time":9,"user_agent":"","referrer":
 "","trans_id":"tx00000c80881a9acd2952a-006335385f-175e5-primary",
 "authentication_type":"Local","access_key_id":"1234","temp_url":false}

 {"bucket":"cn1","time":"2022-09-29T06:17:10.521156Z","time_local":"2022-09-
 29T06:17:10.521156+0000","remote_addr":"10.0.211.66","user":"test1",
 "operation":"create_bucket","uri":"PUT /cn1/
 HTTP/1.1","http_status":"200","error_code":"","bytes_sent":0,
 "bytes_received":0,"object_size":0,"total_time":106,"user_agent":"",
 "referrer":"","trans_id":"tx0000058d60c593632c017-0063353866-175e5-primary",
 "authentication_type":"Local","access_key_id":"1234","temp_url":false}
```

Troubleshooting networking issues

Learn basic troubleshooting procedures connected with networking and chrony for Network Time Protocol (NTP).

In order to troubleshoot networking issues, a running Red Hat Ceph Storage cluster is required.

Basic networking troubleshooting

Red Hat Ceph Storage depends heavily on a reliable network connection. Ceph Storage nodes use the network for communicating with each other. Networking issues can cause many problems with Ceph OSDs, such as them flapping, or being incorrectly reported as `down`. Networking issues can also cause the Ceph Monitor's clock skew errors. In addition, packet loss, high latency, or limited bandwidth can impact the cluster performance and stability.

Before you begin

Before you begin, be sure that you have root-level access to the node.

About this task

For more information, see the following resources on the Red Hat Customer Portal:

- *Basic Network troubleshooting*
- What is the *ethtool* command and how can I use it to obtain information about my network devices and interfaces
- *RHEL network interface dropping packets*
- *What are the performance benchmarking tools available for Red Hat Ceph Storage?*.
- *Knowledgebase articles and solutions* related to troubleshooting networking issues

Procedure

1. Install the `net-tools` and `telnet` packages.
   The `net-tools` and `telnet` packages can help troubleshoot network issues that can occur in a Ceph storage cluster.
   For example,

   ```
   [root@host01 ~]# dnf install net-tools
   [root@host01 ~]# dnf install telnet
   ```

2. Log in to the `cephadm` shell and verify that the **`public_network`** parameters in the Ceph configuration file include the correct values.
   For example,

   ```
   [ceph: root@host01 /]# cat /etc/ceph/ceph.conf
   # minimal ceph.conf for 57bddb48-ee04-11eb-9962-001a4a000672
   [global]
           fsid = 57bddb48-ee04-11eb-9962-001a4a000672
           mon_host = [v2:10.74.249.26:3300/0,v1:10.74.249.26:6789/0]
   [v2:10.74.249.163:3300/0,v1:10.74.249.163:6789/0]
   [v2:10.74.254.129:3300/0,v1:10.74.254.129:6789/0]
   [mon.host01]
   public network = 10.74.248.0/21
   ```

3. Exit the shell and verify that the network interfaces are up.
   For example,

   ```
   [root@host01 ~]# ip link list
   1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group
   default qlen 1000
       link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP mode DEFAULT
   group default qlen 1000
       link/ether 00:1a:4a:00:06:72 brd ff:ff:ff:ff:ff:ff
   ```

4. Verify that the Ceph nodes are able to reach each other using their short hostnames .
   Do this check on each node in the storage cluster.

   ```
   ping SHORT_HOST_NAME
   ```

   For example,

   ```
   [root@host01 ~]# ping host02
   ```

5. If you use a firewall, ensure that that Ceph nodes are able to reach each other on their appropriate ports. If you do not use a firewall, continue to step "6" on page 1591. The `firewall-cmd` tool validates the port status and the `telnet` tool validates if the port is open.

   ```
   firewall-cmd --info-zone=ZONE
   telnet IP_ADDRESS PORT
   ```

   For example,

   ```
   [root@host01 ~]# firewall-cmd --info-zone=public
   public (active)
     target: default
     icmp-block-inversion: no
     interfaces: ens3
     sources:
     services: ceph ceph-mon cockpit dhcpv6-client ssh
     ports: 9283/tcp 8443/tcp 9093/tcp 9094/tcp 3000/tcp 9100/tcp 9095/tcp
     protocols:
     masquerade: no
     forward-ports:
     source-ports:
     icmp-blocks:
     rich rules:

   [root@host01 ~]# telnet 192.168.0.22 9100
   ```

6. Verify that there are no errors on the interface counters.
   Check that the network connectivity between nodes has expected latency, and that there is no packet loss.

   a. Use the **ethtool** command.

      ```
      ethtool -S INTERFACE
      ```

      For example,

      ```
      [root@host01 ~]# ethtool -S ens3 | grep errors
      NIC statistics:
           rx_fcs_errors: 0
           rx_align_errors: 0
           rx_frame_too_long_errors: 0
           rx_in_length_errors: 0
           rx_out_length_errors: 0
           tx_mac_errors: 0
           tx_carrier_sense_errors: 0
           tx_errors: 0
           rx_errors: 0
      ```

   b. Use the **ifconfig** command.
      For example,

      ```
      [root@host01 ~]# ifconfig
      ens3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
              inet 10.74.249.26  netmask 255.255.248.0  broadcast 10.74.255.255
              inet6 fe80::21a:4aff:fe00:672  prefixlen 64  scopeid 0x20<link>
              inet6 2620:52:0:4af8:21a:4aff:fe00:672  prefixlen 64  scopeid 0x0<global>
              ether 00:1a:4a:00:06:72  txqueuelen 1000  (Ethernet)
              RX packets 150549316  bytes 56759897541 (52.8 GiB)
              RX errors 0  dropped 176924  overruns 0  frame 0
              TX packets 55584046  bytes 62111365424 (57.8 GiB)
              TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

      lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
              inet 127.0.0.1  netmask 255.0.0.0
              inet6 ::1  prefixlen 128  scopeid 0x10<host>
              loop  txqueuelen 1000  (Local Loopback)
              RX packets 9373290  bytes 16044697815 (14.9 GiB)
              RX errors 0  dropped 0  overruns 0  frame 0
              TX packets 9373290  bytes 16044697815 (14.9 GiB)
              TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
      ```

   c. Use the **netstat** command.
      For example,

```
[root@host01 ~]# netstat -ai
Kernel Interface table
Iface   MTU    RX-OK      RX-ERR RX-DRP RX-OVR TX-OK      TX-ERR TX-DRP TX-OVR Flg
ens3    1500   311847720  0      364903 0      114341918  0      0      0
 BMRU
lo      65536  19577001   0      0      0      19577001   0      0      0      LRU
```

7. For performance issues, use the `iperf3` tool to verify the network bandwidth between all nodes of the storage cluster.
   The `iperf3` tool does a simple point-to-point network bandwidth test between a server and a client.
   a. Install the `iperf3` package on the Red Hat Ceph Storage nodes that you want to check the bandwidth.
      For example,

      ```
      [root@host01 ~]# dnf install iperf3
      ```

   b. On an Red Hat Ceph Storage, start the `iperf3` server.

      > **Note:** The default port is 5201, but can be set by using the `-P` command argument.

      For example,

      ```
      [root@host01 ~]# iperf3 -s
       Server listening on 5201
      ```

   c. On a different Red Hat Ceph Storage node, start the `iperf3` client.

      ```
      iperf3 -c mon
      ```

      For example,

      ```
      [root@host02 ~]# iperf3 -c mon
       Connecting to host mon, port 5201
       [  4] local xx.x.xxx.xx port 52270 connected to xx.x.xxx.xx port 5201
       [ ID] Interval           Transfer     Bandwidth       Retr  Cwnd
       [  4]   0.00-1.00   sec   114 MBytes   954 Mbits/sec    0    409 KBytes
       [  4]   1.00-2.00   sec   113 MBytes   945 Mbits/sec    0    409 KBytes
       [  4]   2.00-3.00   sec   112 MBytes   943 Mbits/sec    0    454 KBytes
       [  4]   3.00-4.00   sec   112 MBytes   941 Mbits/sec    0    471 KBytes
       [  4]   4.00-5.00   sec   112 MBytes   940 Mbits/sec    0    471 KBytes
       [  4]   5.00-6.00   sec   113 MBytes   945 Mbits/sec    0    471 KBytes
       [  4]   6.00-7.00   sec   112 MBytes   937 Mbits/sec    0    488 KBytes
       [  4]   7.00-8.00   sec   113 MBytes   947 Mbits/sec    0    520 KBytes
       [  4]   8.00-9.00   sec   112 MBytes   939 Mbits/sec    0    520 KBytes
       [  4]   9.00-10.00  sec   112 MBytes   939 Mbits/sec    0    520 KBytes
       - - - - - - - - - - - - - - - - - - - - - - - - -
       [ ID] Interval           Transfer     Bandwidth       Retr
       [  4]   0.00-10.00  sec  1.10 GBytes   943 Mbits/sec    0             sender
       [  4]   0.00-10.00  sec  1.10 GBytes   941 Mbits/sec                  receiver

       iperf Done.
      ```

      This output shows a network bandwidth of 1.1 Gbits/second between the Red Hat Ceph Storage nodes, along with no retransmissions (`Retr`) during the test. It is recommended to validate the network bandwidth between all the nodes in the storage cluster.

8. Ensure that all nodes have the same network interconnect speed.
   Slower attached nodes might slow down the faster connected ones. Also, ensure that the inter-switch links can handle the aggregated bandwidth of the attached nodes.

   ```
   ethtool INTERFACE
   ```

   For example,

```
[root@host01 ~]# ethtool ens3
 Settings for ens3:
 Supported ports: [ TP ]
 Supported link modes:   10baseT/Half 10baseT/Full
                         100baseT/Half 100baseT/Full
                         1000baseT/Half 1000baseT/Full
 Supported pause frame use: No
 Supports auto-negotiation: Yes
 Supported FEC modes: Not reported
 Advertised link modes:  10baseT/Half 10baseT/Full
                         100baseT/Half 100baseT/Full
                         1000baseT/Half 1000baseT/Full
 Advertised pause frame use: Symmetric
 Advertised auto-negotiation: Yes
 Advertised FEC modes: Not reported
 Link partner advertised link modes:  10baseT/Half 10baseT/Full
                                      100baseT/Half 100baseT/Full
                                      1000baseT/Full
 Link partner advertised pause frame use: Symmetric
 Link partner advertised auto-negotiation: Yes
 Link partner advertised FEC modes: Not reported
 Speed: 1000Mb/s
 Duplex: Full
 Port: Twisted Pair
 PHYAD: 1
 Transceiver: internal
 Auto-negotiation: on
 MDI-X: off
 Supports Wake-on: g
 Wake-on: d
 Current message level: 0x000000ff (255)
        drv probe link timer ifdown ifup rx_err tx_err
 Link detected: yes
```

Basic chrony NTP troubleshooting

Use this information to run basic chrony NTP troubleshooting steps.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph Monitor node.

About this task

For more information, see the following:

- “Clock skew” on page 1602
- Checking if chrony is synchronized on Red Hat Documentation.
- How to troubleshoot chrony issues solution on the Red Hat Customer Portal for advanced chrony NTP troubleshooting steps.

Procedure

1. Verify that the `chrony` daemon is running on the Ceph Monitor hosts:
   For example,

   ```
   [root@mon ~]# systemctl status chrony
   ```

   a. If `chrony` is not running, enable and start it.
      For example,

      ```
      [root@mon ~]# systemctl enable chrony
      [root@mon ~]# systemctl start chrony
      ```

2. Ensure that `chrony` is synchronizing the clocks correctly.
   For example,

```
[root@mon ~]# chronyc sources
[root@mon ~]# chronyc sourcestats
[root@mon ~]# chronyc tracking
```

3. To configure NTP. see NTP section in Installation Guide for RHEL (x86_64)
4. To configure using chrony, open the NTP configuration file `/etc/chrony.con` in a text editor such as vi or nano, or create a new one if it does not already exist.
5. Add the NTP server entry details `server <NTP_Server> iburst`
6. Restart the chrony service `systemctl restart chrony`.
7. Repeat the same steps on all nodes to configure NTP and to sync time across all nodes.

cephadm troubleshooting

As a storage administrator, you can troubleshoot the Red Hat Ceph Storage cluster. Sometimes there is a need to investigate why a `cephadm` command failed or why a specific service does not run properly.

Before troubleshooting `cephadm`, be sure that you have a running Red Hat Ceph Storage cluster

Pause or disable cephadm

If `cephadm` does not behave as expected, you can pause most of the background activity or disable it completely.

Pause `cephadm` with the following commands:

Example

```
[ceph: root@host01 /]# ceph orch pause
```

This stops any changes, but `cephadm` periodically checks hosts to refresh it's inventory of daemons and devices.

If you want to disable `cephadm` completely, run the following commands:

Example

```
[ceph: root@host01 /]# ceph orch set backend ''
[ceph: root@host01 /]# ceph mgr module disable cephadm
```

Note that previously deployed daemon containers continue to exist and start as they did before.

To re-enable `cephadm` in the cluster, run the following commands:

Example

```
[ceph: root@host01 /]# ceph mgr module enable cephadm
[ceph: root@host01 /]# ceph orch set backend cephadm
```

Per service and per daemon event

`cephadm` stores events per service and per daemon in order to aid in debugging failed daemon deployments. These events often contain relevant information.

Per service

Syntax

```
ceph orch ls --service_name SERVICE_NAME --format yaml
```

Example

```
[ceph: root@host01 /]# ceph orch ls --service_name alertmanager --format yaml
service_type: alertmanager
service_name: alertmanager
placement:
  hosts:
```

```
  - unknown_host
status:
  ...
  running: 1
  size: 1
events:
- 2021-02-01T08:58:02.741162 service:alertmanager [INFO] "service was created"
- '2021-02-01T12:09:25.264584 service:alertmanager [ERROR] "Failed to apply: Cannot
  place <AlertManagerSpec for service_name=alertmanager> on unknown_host: Unknown hosts"'
```

Per daemon

Syntax

```
ceph orch ps --service-name SERVICE_NAME --daemon-id DAEMON_ID --format yaml
```

Example

```
[ceph: root@host01 /]# ceph orch ps --service-name mds --daemon-id cephfs.hostname.ppdhsz
--format yaml
daemon_type: mds
daemon_id: cephfs.hostname.ppdhsz
hostname: hostname
status_desc: running
...
events:
- 2021-02-01T08:59:43.845866 daemon:mds.cephfs.hostname.ppdhsz [INFO] "Reconfigured
  mds.cephfs.hostname.ppdhsz on host 'hostname'"
```

Check `cephadm` logs

Use the `cephadm` logs for monitoring and troubleshooting cephadm.

You can monitor the `cephadm` log in real time with the following command:

Example

```
[ceph: root@host01 /]# ceph -W cephadm
```

You can see the last few messages with the following command:

Example

```
[ceph: root@host01 /]# ceph log last cephadm
```

If you have enabled logging to files, you can see a `cephadm` log file called `ceph.cephadm.log` on the monitor hosts.

Gather log files

Gather log files to help troubleshoot `cephadm`.

Note:

- Be sure to run all log file commands outside the `cephadm` shell.
- By default, `cephadm` stores logs in `journald`.

Table 209: Gathering `cephadm` ogs

Action needed	Command	Example	Notes
Gather log files for all daemons	**`journalctl`**		

Action needed	Command	Example	Notes
Read the log file of a specific daemon	`cephadm logs --name DAEMON_NAME`	`[root@host01 ~]# cephadm logs --name cephfs.hostname.ppdhsz`	This command works when run on the same hosts where the daemon is running.
Read the log file of a specific daemon running on a different host	`cephadm logs --fsid FSID --name DAEMON_NAME`	`[root@host01 ~]# cephadm logs --fsid 2d2fd136-6df1-11ea-ae74-002590e526e8 --name cephfs.hostname.ppdhsz`	*FSID* is the cluster ID provided by the **ceph status** command.
Fetch all log files of all the daemons on a given host	`for name in $(cephadm ls \| python3 -c "import sys, json; [print(i[name]) for i in json.load(sys.stdin)]") ; do cephadm logs --fsid FSID_OF_CLUSTER --name "$name" > $name; done`	`[root@host01 ~]# for name in $(cephadm ls \| python3 -c "import sys, json; [print(i['name']) for i in json.load(sys.stdin)]") ; do cephadm logs --fsid 57bddb48-ee04-11eb-9962-001a4a000672 --name "$name" > $name; done`	

Collect systemd status

Collect the systemd status for `cephadm` troubleshooting.

To print the state of a systemd unit, run the following command:

Example

```
[root@host01 ~]$ systemctl status ceph-a538d494-fb2a-48e4-82c8-
b91c37bb0684@mon.host01.service
```

List all downloaded container images

List all the container images that are downloaded on a host.

To list all the container images that are downloaded on a host, run the following command:

```
[ceph: root@host01 /]# podman ps -a --format json | jq '.[].Image'
"docker.io/library/rhel9"
"registry.redhat.io/rhceph-alpha/
rhceph-9-rhel9@sha256:9aaea414e2c263216f3cdcb7a096f57c3adf6125ec9f4b0f5f65fa8c43987155"
```

Manually run containers

`cephadm` writes small wrappers that runs a container. Refer to `/var/lib/ceph/CLUSTER_FSID/SERVICE_NAME/unit` to run the container command.

Analyzing SSH errors

If you get the following error:

Example

```
    execnet.gateway_bootstrap.HostNotFound: -F /tmp/cephadm-conf-73z09u6g -i /tmp/cephadm-
identity-ky7ahp_5 root@10.10.1.2

    ...

    raise OrchestratorError(msg) from e
    orchestrator._interface.OrchestratorError: Failed to connect to 10.10.1.2 (10.10.1.2).

    Please make sure that the host is reachable and accepts connections using the cephadm
SSH key
```

Try the following options to troubleshoot the issue:

- To ensure `cephadm` has a SSH identity key, run the following command:

  Example

  ```
  [ceph: root@host01 /]# ceph config-key get mgr/cephadm/ssh_identity_key > ~/
  cephadm_private_key
  INFO:cephadm:Inferring fsid f8edc08a-7f17-11ea-8707-000c2915dd98
  INFO:cephadm:Using recent ceph image docker.io/ceph/ceph:v15 obtained 'mgr/cephadm/
  ssh_identity_key'
  [root@mon1 ~] # chmod 0600 ~/cephadm_private_key
  ```

  If the command fails, `cephadm` does not have a key. To generate a SSH key, run the following command:

  Example

  ```
  [ceph: root@host01 /]# chmod 0600 ~/cephadm_private_key
  ```

  Or

  Example

  ```
  [ceph: root@host01 /]# cat ~/cephadm_private_key | ceph cephadm set-ssk-key -i-
  ```

- To ensure that the SSH configuration is correct, run the following command:

  Example

  ```
  [ceph: root@host01 /]# ceph cephadm get-ssh-config
  ```

- To verify the connection to the host, run the following command:

  Example

  ```
  [ceph: root@host01 /]# ssh -F config -i ~/cephadm_private_key root@host01
  ```

Verify public key is in `authorized_keys`.

To verify that the public key is in the `authorized_keys` file, run the following commands:

Example

```
[ceph: root@host01 /]# ceph cephadm get-pub-key
[ceph: root@host01 /]# grep "`cat ~/ceph.pub`" /root/.ssh/authorized_keys
```

CIDR network error

Classless inter domain routing (CIDR) also known as supernetting, is a method of assigning Internet Protocol (IP) addresses, the `cephadm` log entries shows the current state that improves the efficiency of address distribution and replaces the previous system based on Class A, Class B and Class C networks.

If you see one of the following errors:

```
ERROR: Failed to infer CIDR network for mon ip ***; pass --skip-mon-network to configure it
later
```

Or

```
Must set public_network config option or specify a CIDR network, ceph addrvec, or plain IP
```

You need to run the following command:

Example

```
[ceph: root@host01 /]# ceph config set host public_network hostnetwork
```

Access admin socket

Each Ceph daemon provides an admin socket that bypasses the MONs.

To access the admin socket, enter the daemon container on the host:

Example

```
[ceph: root@host01 /]# cephadm enter --name cephfs.hostname.ppdhsz
[ceph: root@mon1 /]# ceph --admin-daemon /var/run/ceph/ceph-cephfs.hostname.ppdhsz.asok
config show
```

Manually deploying a `mgr` daemon

`cephadm` requires a `mgr` daemon in order to manage the Red Hat Ceph Storage cluster. In case the last `mgr` daemon of an Red Hat Ceph Storage cluster was removed, you can manually deploy a `mgr` daemon, on a random host of the Red Hat Ceph Storage cluster.

Prerequisites

- A running Red Hat Ceph Storage cluster.
- Root-level access to all the nodes.
- Hosts are added to the cluster.

Procedure

1. Log into the `cephadm` shell:

   Example

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Disable the `cephadm` scheduler to prevent `cephadm` from removing the new MGR daemon, with the following command:

   Example

   ```
   [ceph: root@host01 /]# ceph config-key set mgr/cephadm/pause true
   ```

3. Get or create the `auth` entry for the new MGR daemon:

   Example

   ```
   [ceph: root@host01 /]# ceph auth get-or-create mgr.host01.smfvfd1 mon "profile mgr"
   osd "allow *" mds "allow *"
   [mgr.host01.smfvfd1]
   key = AQDhcORgW8toCRAAlMzlqWXnh3cGRjqYEa9ikw==
   ```

4. Open `ceph.conf` file:

   Example

   ```
   [ceph: root@host01 /]# ceph config generate-minimal-conf
   # minimal ceph.conf for 8c9b0072-67ca-11eb-af06-001a4a0002a0
   [global]
   fsid = 8c9b0072-67ca-11eb-af06-001a4a0002a0
   ```

```
mon_host = [v2:10.10.200.10:3300/0,v1:10.10.200.10:6789/0]
[v2:10.10.10.100:3300/0,v1:10.10.200.100:6789/0]
```

5. Get the container image:

   Example

   ```
   [ceph: root@host01 /]# ceph config get "mgr.host01.smfvfd1" container_image
   ```

6. Create a `config-json.json` file and add the following:

   > **Note:** Use the values from the output of the **`ceph config generate-minimal-conf`** command.

   Example

   ```
   {
     {
     "config": "# minimal ceph.conf for 8c9b0072-67ca-11eb-af06-001a4a0002a0\n[global]
   \n\tfsid = 8c9b0072-67ca-11eb-af06-001a4a0002a0\n\tmon_host =
    [v2:10.10.200.10:3300/0,v1:10.10.200.10:6789/0]
   [v2:10.10.10.100:3300/0,v1:10.10.200.100:6789/0]\n",
     "keyring": "[mgr.Ceph5-2.smfvfd1]\n\tkey =
   AQDhcORgW8toCRAAlMzlqWXnh3cGRjqYEa9ikw==\n"
   }
   }
   ```

7. Exit from the `cephadm` shell:

   Example

   ```
   [ceph: root@host01 /]# exit
   ```

8. Deploy the MGR daemon:

   Example

   ```
   [root@host01 ~]# cephadm --image registry.redhat.io/rhceph-alpha/
   rhceph-9-rhel9:latest  deploy --fsid  8c9b0072-67ca-11eb-af06-001a4a0002a0 --name
   mgr.host01.smfvfd1 --config-json config-json.json
   ```

Verification

- In the `cephadm` shell, run the following command:

  Example

  ```
  [ceph: root@host01 /]# ceph -s
  ```

  You can see a new `mgr` daemon has been added.

Troubleshooting Ceph Monitors

Use this information to learn how to fix the most common errors that are related to Ceph Monitors.

Before troubleshooting Ceph Monitors verify all network connections.

Most common Ceph Monitor errors

Learn the most common error messages that are returned by the **`ceph health detail`** command and that are included in the Ceph logs.

Use the **`ceph health detail`** command on a running Red Hat Ceph Storage cluster.

Ceph Monitor error messages

Understand Ceph Monitor error messages and how to fix them.

"Ceph Monitor error messages" on page 1600 provides links to corresponding sections that explain the errors and point to specific procedures to fix the problems.

A table of Ceph Monitor error messages, and links in the documentation of potential fixes.

Table 210: Ceph Monitor error messages

Error message	See
HEALTH_WARN	
`mon.X is down (out of quorum)`	"Ceph Monitor is out of quorum" on page 1600
`clock skew`	"Clock skew" on page 1602
`store is getting too big!`	"Ceph Monitor store is getting too big" on page 1603

Common Ceph Monitor error messages in Ceph logs

Understand Ceph Monitor error messages in the Ceph logs and how to fix them.

"Ceph Monitor error messages in the Ceph logs" on page 1600 provides links to corresponding sections that explain the errors and point to specific procedures to fix the problems.

A table of Ceph Monitor error messages in the Ceph logs and links in the documentation of potential fixes.

Table 211: Ceph Monitor error messages in the Ceph logs

Error message	Log file	See
clock skew	Main cluster log	"Clock skew" on page 1602
clocks not synchronized	Main cluster log	"Clock skew" on page 1602
Corruption: error in middle of record	Monitor log	– "Ceph Monitor is out of quorum" on page 1600 – Recovering the Ceph Monitor store when using BlueStore
Corruption: 1 missing files	Monitor log	– "Ceph Monitor is out of quorum" on page 1600 – Recovering the Ceph Monitor store when using BlueStore
Caught signal (Bus error)	Monitor log	"Ceph Monitor is out of quorum" on page 1600

Ceph Monitor is out of quorum

Understand and troubleshoot Ceph Monitors out of quorum.

A Ceph Monitor is out of quorum if one or more Ceph Monitors are marked as *down* but the other Ceph Monitors are still able to form a quorum. In addition, the **`ceph health detail`** command returns an error message similar to the following example:

```
HEALTH_WARN 1 mons down, quorum 1,2 mon.b,mon.c, mon.a (rank 0) addr 127.0.0.1:6789/0 is
down (out of quorum)
```

What this means

Ceph marks a Ceph Monitor as *down* due to various reasons.

If the `ceph-mon` daemon is not running, it might have a corrupted store or some other error is preventing the daemon from starting. Also, the `/var/` partition might be full. As a consequence, `ceph-mon` is not able to run any operations to the store located by default at `/var/lib/ceph/mon-SHORT_HOST_NAME/store.db` and ends.

If the `ceph-mon` daemon is running but the Ceph Monitor is out of quorum and marked as *down*, the cause of the problem depends on the Ceph Monitor state:

- If the Ceph Monitor is in the *probing* state longer than expected, it cannot find the other Ceph Monitors. This problem can be caused by networking issues, or the Ceph Monitor can have an outdated Ceph Monitor map (`monmap`) and be trying to reach the other Ceph Monitors on incorrect IP addresses. Alternatively, if the `monmap` is up-to-date, the Ceph Monitor clock might not be synchronized.
- If the Ceph Monitor is in the *electing* state longer than expected, the Ceph Monitor's clock might not be synchronized.
- If the Ceph Monitor changes its state from *synchronizing* to *electing* and back, the cluster state is advancing. This means that it is generating new maps faster than the synchronization process can handle.
- If the Ceph Monitor marks itself as the *leader* or a *peon*, then it believes to be in a quorum, while the remaining cluster is sure that it is not. This problem can be caused by failed clock synchronization.

For more information, see:

- Understanding Ceph Monitor status
- Starting, stopping, and restarting all Ceph daemons using the systemctl command
- Using Ceph administration socket

Troubleshooting this problem

Verify that the `ceph-mon` daemon is running. If needed, start the daemon.

```
systemctl status ceph-mon@HOST_NAME
systemctl start ceph-mon@HOST_NAME
```

Replace *HOST_NAME* with the short name of the host where the daemon is running.

> **Note:** If the hostname is not known, use the **`hostname -s`** command.

- If you are not able to start the `ceph-mon` daemon, go to "The ceph-mon daemon cannot start" on page 1601.
- If you are able to start the `ceph-mon` daemon but it is marked as *down*, go to "The ceph-mon daemon is running, but still marked as down" on page 1602.

The `ceph-mon` daemon cannot start

1. Check the corresponding Ceph Monitor log located in the `/var/log/ceph/CLUSTER_FSID/` directory.

   > **Note:** By default, the monitor logs are not present in the log folder. You need to enable logging to files for the logs to appear in the folder. See the Ceph daemon logs to enable logging to files.

2. If the log contains error messages similar to the following ones, the Ceph Monitor might have a corrupted store.

   ```
   Corruption: error in middle of record
   Corruption: 1 missing files; example: /var/lib/ceph/mon/mon.0/store.db/1234567.ldb
   ```

   To fix this problem, replace the Ceph Monitor. For more information, see Replacing a failed monitor.

3. If the log contains an error message similar to the following one, the `/var/` partition might be full. Delete any unnecessary data from `/var/`.

   ```
   Caught signal (Bus error)
   ```

> **Important:** Do not delete any data from the Monitor directory manually. Instead, use the `ceph-monstore-tool` to compact it.

4. If you see any other error messages, open a support ticket. For more information, contact Red Hat Support.

The `ceph-mon` daemon is running, but still marked as `down`

From the Ceph Monitor host that is out of the quorum, use the **`mon_status`** command to check its state.

```
ceph daemon ID mon_status
```

Replace *`ID`* with the ID of the Ceph Monitor

For example,

```
[ceph: root@host01 /]# ceph daemon mon.host01 mon_status
```

- If the status is *probing*, verify the locations of the other Ceph Monitors in the **`mon_status`** output.
  - If the addresses are incorrect, the Ceph Monitor has an incorrect Ceph Monitor map (`monmap`). To fix this problem, see "Injecting a monmap" on page 1605.
  - If the addresses are correct, verify that the Ceph Monitor clocks are synchronized. For more information, see "Clock skew" on page 1602. To troubleshoot any networking issues, see "Troubleshooting networking issues" on page 1589.
- If the status is *electing*, verify that the Ceph Monitor clocks are synchronized. For more information, see "Clock skew" on page 1602.
- If the status changes from *electing* to *synchronizing*, open a support ticket. For more information, contact Red Hat Support.
- If the Ceph Monitor is the *leader* or a *peon*, verify that the Ceph Monitor clocks are synchronized. Open a support ticket if synchronizing the clocks does not solve the problem. For more information, contact Red Hat Support.

Clock skew

Understand and troubleshoot clock skew.

A Ceph Monitor is out of quorum, and the **`ceph health detail`** command output contains error messages similar to the following examples:

```
mon.a (rank 0) addr 127.0.0.1:6789/0 is down (out of quorum)
mon.a addr 127.0.0.1:6789/0 clock skew 0.08235s > max 0.05s (latency 0.0045s)
```

In addition, Ceph logs contain error messages similar to the following examples:

```
2022-05-04 07:28:32.035795 7f806062e700 0 log [WRN] : mon.a 127.0.0.1:6789/0 clock skew
0.14s > max 0.05s
2022-05-04 04:31:25.773235 7f4997663700 0 log [WRN] : message from mon.1 was stamped
0.186257s in the future, clocks not synchronized
```

What this means

The clock skew error message indicates that Ceph Monitors' clocks are not synchronized. Clock synchronization is important because Ceph Monitors depend on time precision and behave unpredictably if their clocks are not synchronized.

The **`mon_clock_drift_allowed`** parameter determines what disparity between the clocks is tolerated. By default, this parameter is set to 0.05 seconds.

Important: Do not change the default value of **mon_clock_drift_allowed** without previous testing. Changing this value might affect the stability of the Ceph Monitors and the Ceph Storage Cluster in general.

Possible causes of the clock skew error include network problems or problems with chrony Network Time Protocol (NTP) synchronization if that is configured. In addition, time synchronization does not work properly on Ceph Monitors deployed on virtual machines.

For more information, see:

- "Understanding Ceph Monitor status" on page 1604
- "Ceph Monitor is out of quorum" on page 1600

Troubleshooting this problem

Note: Ceph evaluates time synchronization every five minutes, resulting in delays between fixing the problem and clearing the clock skew messages.

1. Verify that your network works correctly. For more information, see "Troubleshooting networking issues" on page 1589. If you use chrony for NTP, see "Basic chrony NTP troubleshooting" on page 1593.
2. If you use a remote NTP server, consider deploying your own chrony NTP server on your network. For more information, see **Using the Chrony Suite to Configure NTP** chapter within the *Configuring basic system settings* guide within the Product Documentation for Red Hat Enterprise Linux for your OS version, on Red Hat Documentation.

Ceph Monitor store is getting too big

Understand and troubleshoot Ceph Monitors that are getting too big.

The **ceph health** command returns an error message similar to the following example when the Ceph Monitor is getting too big:

```
mon.ceph1 store is getting too big! 48031 MB >= 15360 MB -- 62% avail
```

What this means

Red Hat Ceph Storage Monitor store is a RocksDB database that stores entries as key–values pairs. The database includes a cluster map and is located by default at `/var/lib/ceph/mon/CLUSTER_NAME-SHORT_HOST_NAME/store.db`.

Querying a large Monitor store can take time, which can cause the Ceph Monitor to be delayed in responding to client queries.

In addition, if the `/var/` partition is full, the Ceph Monitor cannot perform any write operations to the store and terminates. For more information about troubleshooting this issue, see "Ceph Monitor is out of quorum" on page 1600.

Troubleshooting this problem

1. Check the size of the database.

   ```
   du -sch /var/lib/ceph/mon/CLUSTER_NAME-SHORT_HOST_NAME/store.db
   ```

   Specify the name of the cluster and the short host name of the host where the `ceph-mon` is running. For example,

   ```
   # du -sch /var/lib/ceph/mon/ceph-host1/store.db
   47G     /var/lib/ceph/mon/ceph-ceph1/store.db/
   47G     total
   ```

2. Compact the Ceph Monitor store. For more information, see "Compacting Monitor store" on page 1607.

Understanding Ceph Monitor status

Understand the different Ceph Monitor statuses.

The **mon_status** command returns information about a Ceph Monitor. Information provided includes:

- State
- Rank
- Elections epoch
- Monitor map (monmap)

If Ceph Monitors are able to form a quorum, use the **mon_status** command with the ceph command-line utility.

If Ceph Monitors are not able to form a quorum, but the ceph-mon daemon is running, use the administration socket to run the **mon_status** command.

For more information, see Using Ceph administration socket

Example output of mon_status

```
{
    "name": "mon.3",
    "rank": 2,
    "state": "peon",
    "election_epoch": 96,
    "quorum": [
        1,
        2
    ],
    "outside_quorum": [],
    "extra_probe_peers": [],
    "sync_provider": [],
    "monmap": {
        "epoch": 1,
        "fsid": "d5552d32-9d1d-436c-8db1-ab5fc2c63cd0",
        "modified": "0.000000",
        "created": "0.000000",
        "mons": [
            {
                "rank": 0,
                "name": "mon.1",
                "addr": "172.25.1.10:6789\/0"
            },
            {
                "rank": 1,
                "name": "mon.2",
                "addr": "172.25.1.12:6789\/0"
            },
            {
                "rank": 2,
                "name": "mon.3",
                "addr": "172.25.1.13:6789\/0"
            }
        ]
    }
}
```

Ceph Monitor states

Leader

During the electing phase, Ceph Monitors are electing a leader. The leader is the Ceph Monitor with the highest rank, that is the rank with the lowest value. In the example provided, the leader is mon.1.

Peon

Peons are the Ceph Monitors in the quorum that are not leaders. If the leader fails, the peon with the highest rank becomes a new leader.

Probing

A Ceph Monitor is in the probing state if it is looking for other Ceph Monitors. For example, after you start the Ceph Monitors, they are *probing* until they find enough Ceph Monitors specified in the Ceph Monitor map (monmap) to form a quorum.

Electing

A Ceph Monitor is in the electing state if it is in the process of electing the leader. Usually, this status changes quickly.

Synchronizing

A Ceph Monitor is in the synchronizing state if it is synchronizing with the other Ceph Monitors to join the quorum. The smaller the Ceph Monitor store it, the faster the synchronization process. Therefore, if you have a large store, synchronization takes a longer time.

Injecting a monmap

If a Ceph Monitor has an outdated or corrupted Ceph Monitor map (`monmap`), it cannot join a quorum because it is trying to reach the other Ceph Monitors on incorrect IP addresses. The safest way to fix this problem is to obtain and inject the actual Ceph Monitor map from other Ceph Monitors.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Access to the Ceph Monitor Map.
- Root-level access to the Ceph Monitor node.

About this task

> **Note:** This action overwrites the existing Ceph Monitor map that is kept by the Ceph Monitor.

Use this procedure to inject the Ceph Monitor map when the other Ceph Monitors are able to form a quorum, or when at least one Ceph Monitor has a correct Ceph Monitor map. If all Ceph Monitors have corrupted stores, and therefore also a corrupted Ceph Monitor map, see Recovering the Ceph Monitor store when using BlueStore.

For more information, see “Ceph Monitor is out of quorum” on page 1600.

Procedure

1. If the remaining Ceph Monitors are able to form a quorum, get the Ceph Monitor map by using the **ceph mon getmap** command.
   If the remaining Ceph Monitors are not able to form the quorum, go to step “2” on page 1605.
   For example,

   ```
   [ceph: root@host01 /]# ceph mon getmap -o /tmp/monmap
   ```

2. If the remaining Ceph Monitors are not able to form the quorum and you have at least one Ceph Monitor with a correct Ceph Monitor map, copy it from that Ceph Monitor.
   a. Stop the Ceph Monitor which you want to copy the Ceph Monitor map from.

      ```
      systemctl stop ceph-mon@HOST_NAME
      ```

      For example, to stop the Ceph Monitor running on a host with the `host01` short hostname, run the following command:

      ```
      [root@mon ~]# systemctl stop ceph-mon@host01
      ```

   b. Copy the Ceph Monitor map.

      ```
      ceph-mon -i ID --extract-monmap /tmp/monmap
      ```

      Replace *ID* with the ID of the Ceph Monitor which you want to copy the Ceph Monitor map from.
      For example,

      ```
      [ceph: root@host01 /]#  ceph-mon -i mon.a  --extract-monmap /tmp/monmap
      ```

3. Stop the Ceph Monitor which you want to copy the Ceph Monitor map from.

```
systemctl stop ceph-mon@HOST_NAME
```

For example, to stop the Ceph Monitor running on a host with the `host01` short hostname, run the following command:

```
[root@mon ~]# systemctl stop ceph-mon@host01
```

4. Inject the Ceph Monitor map.

```
ceph-mon -i ID --inject-monmap /tmp/monmap
```

Replace *ID* with the ID of the Ceph Monitor with the corrupted or outdated Ceph Monitor map.
For example,

```
[root@mon ~]# ceph-mon -i mon.host01 --inject-monmap /tmp/monmap
```

5. Start the Ceph Monitor.
For example,

```
[root@mon ~]# systemctl start ceph-mon@host01
```

> **Note:** If you copied the Ceph Monitor map from another Ceph Monitor, also start that Ceph Monitor.

Replacing a failed Monitor

When a Ceph Monitor has a corrupted store, replace the Monitor in the storage cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster
- Able to form a quorum
- Root-level access to Ceph Monitor node

About this task

For more information, see “Ceph Monitor is out of quorum” on page 1600.

Procedure

1. From the Monitor host, remove the Monitor store.
The Monitor store is located, by default, in `/var/lib/ceph/mon/CLUSTER_NAME-SHORT_HOST_NAME`.

```
rm -rf /var/lib/ceph/mon/CLUSTER_NAME-SHORT_HOST_NAME
```

Specify the short hostname of the Monitor host and the cluster name.
For example, to remove the Monitor store of a Monitor running on `host1` from a cluster called `remote`, run the following command:

```
[root@mon ~]# rm -rf /var/lib/ceph/mon/remote-host1
```

2. Remove the Monitor from the Monitor map (`monmap`).

```
ceph mon remove SHORT_HOST_NAME
```

Specify the short hostname of the Monitor host.
For example, to remove the Monitor running on `host01`, run the following command:

```
[ceph: root@host01 /]# ceph mon remove host01
```

3. Troubleshoot and fix any problems that are related to the underlying file system or hardware of the Monitor host.

Compacting Monitor store

Compact a Monitor store when it has grown too large.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph Monitor node.

About this task

The Monitor store can be compacted in any of the following ways:

- Dynamically by using the **`ceph tell`** command.
- Upon the start of the `ceph-mon` daemon.
- By using the `ceph-monstore-tool` when the `ceph-mon` daemon is not running. Use this method when the other methods fail to compact the Monitor store or when the Monitor is out of quorum and its log contains the Caught signal (Bus error) error message.

Important: Monitor store size changes when the cluster is not in the `active+clean` state or during the rebalancing process. For this reason, compact the Monitor store when rebalancing is completed. Also, ensure that the placement groups are in the `active+clean` state.

For more information, see “Ceph Monitor store is getting too big” on page 1603 and “Ceph Monitor is out of quorum” on page 1600.

Procedure

1. While the `ceph-mon` daemon is running, compact the Monitor store.

```
ceph tell mon.HOST_NAME compact
```

Replace *HOST_NAME* with the short hostname of the host where the `ceph-mon` is running.

Note: If the hostname is not known, use the **`hostname -s`** command.

For example,

```
[ceph: root@host01 /]# ceph tell mon.host01 compact
```

2. Add the **`mon_compact_on_start = true`** parameter to the Ceph configuration under the `[mon]` section.
For example,

```
[mon]
mon_compact_on_start = true
```

3. Restart the `ceph-mon` daemon.

```
systemctl restart ceph-mon@HOST_NAME
```

Replace *HOST_NAME* with the short name of the host where the daemon is running.

> **Note:** If the hostname is not known, use the **hostname -s** command.

For example,

```
[root@mon ~]# systemctl restart ceph-mon@host01
```

4. Ensure that Monitors have formed a quorum.

```
ceph mon stat
```

For example,

```
[ceph: root@host01 /]# ceph mon stat
```

5. Optional: If needed, repeat steps “1” on page 1607 through “4” on page 1608 on any other Monitors.

> **Note:** Before starting, be sure that the `ceph-test` package is installed.

6. Verify that the `ceph-mon` daemon with the large store is not running.
If needed, stop the daemon.

```
systemctl status ceph-mon@HOST_NAME
```

```
systemctl stop ceph-mon@HOST_NAME
```

Replace *HOST_NAME* with the short name of the host where the daemon is running.

> **Note:** If the hostname is not known, use the **hostname -s** command.

For example,

```
[root@mon ~]# systemctl status ceph-mon@host01
[root@mon ~]# systemctl stop ceph-mon@host01
```

7. Compact the Monitor store.

```
ceph-monstore-tool /var/lib/ceph/mon/mon.HOST_NAME compact
```

Replace *HOST_NAME* with a short hostname of the Monitor host.
For example,

```
[ceph: root@host01 /]# ceph-monstore-tool /var/lib/ceph/mon/mon.host01 compact
```

8. Start `ceph-mon` again.

```
systemctl start ceph-mon@HOST_NAME
```

For example,

```
[root@mon ~]# systemctl start ceph-mon@host01
```

Opening port for Ceph Manager

Understand how to open ports for Ceph Manager. The ports must be open to keep the clusters working correctly.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to Ceph Manager.

About this task

The `ceph-mgr` daemons receive placement group information from OSDs on the same range of ports as the `ceph-osd` daemons. If these ports are not open, a cluster devolves from `HEALTH_OK` to `HEALTH_WARN` and indicates that placement groups (PGs) are *unknown* with a percentage count of the PGs unknown.

Procedure

Use this procedure to open ports.

1. For each host running `ceph-mgr` daemons, open ports 6800-7300.
   For example,

   ```
   [root@ceph-mgr] # firewall-cmd --add-port 6800-7300/tcp
   [root@ceph-mgr] # firewall-cmd --add-port 6800-7300/tcp --permanent
   ```

2. Restart the `ceph-mgr` daemons.

Recovering Ceph Monitor store

If the Ceph Monitors experience corruption, you can recover them with information that is stored on the OSD nodes.

Ceph Monitors store the cluster map in a key-value store such as LevelDB. If the store is corrupted on a Monitor, the Monitor stops unexpectedly and fails to start again. The Ceph logs might include the following errors:

```
Corruption: error in middle of record
Corruption: 1 missing files; e.g.: /var/lib/ceph/mon/mon.0/store.db/1234567.ldb
```

The Red Hat Ceph Storage clusters use at least three Ceph Monitors so that if one fails, it can be replaced with another one. However, under certain circumstances, all Ceph Monitors can have corrupted stores. For example, when the Ceph Monitor nodes have incorrectly configured disk or file system settings, a power outage can corrupt the underlying file system.

If there is corruption on all Ceph Monitors, you can recover it with information that is stored on the OSD nodes by using the `ceph-monstore-tool` and `ceph-objectstore-tool` utilities.

Important:

- These procedures cannot recover the following information:
  - Metadata Daemon Server (MDS) keyrings and maps.
  - Placement group (PG) settings.
    `full ratio`

    `full ratio` set, by using the **`ceph pg set_full_ratio`** command.

    `nearfull ratio`

    `nearfull ratio` set, by using the **`ceph pg set_nearfull_ratio`** command.
- Never restore the Ceph Monitor store from an old backup. Rebuild the Ceph Monitor store from the current cluster state by using the steps detailed in the following sections, and restore from that.

Troubleshooting Ceph OSDs

Use this information to learn how to fix the most common errors that are related to Ceph OSDs.

Before you begin troubleshooting Ceph OSDs do the following:

- Verify your network connection. For more information, see “Troubleshooting networking issues” on page 1589.

- Verify that Monitors have a quorum by using the **ceph health** command. If the command returns a health status (HEALTH_OK, HEALTH_WARN, or HEALTH_ERR), the Monitors are able to form a quorum. If not, address any Monitor problems first.
  - For more information about Ceph Monitors, see "Troubleshooting Ceph Monitors" on page 1599.
  - For more information about `ceph health`, see "Understanding Ceph health" on page 1580.
- Optionally, stop the rebalancing process to save time and resources. For more information, see "Stopping and starting rebalancing" on page 1619.

Most common Ceph OSD errors

Learn the most common Ceph OSD errors that are returned by the **ceph health detail** command and that are included in the Ceph logs.

Use the `ceph health detail` command with root-level access to the Ceph OSD nodes.

Ceph OSD error messages

Understand Ceph OSD error messages and how to fix them.

This section lists Ceph OSD errors and warnings with links to troubleshooting topics. Each topic includes an explanation and procedures to resolve the problem.

OSD errors (HEALTH_ERR)

full osds

For information and troubleshooting , see "Full OSDs" on page 1611.

OSD warnings (HEALTH_WARN)

nearfull osds

For information and troubleshooting see, "Nearfull OSDs" on page 1612.

osds are down

For information and troubleshooting, see the following:

- "Down OSDs" on page 1614
"Flapping OSDs" on page 1615

requests are blocked

For information and troubleshooting, see "Slow requests or requests are blocked" on page 1617.

slow requests

For information and troubleshooting, see "Slow requests or requests are blocked" on page 1617.

Common Ceph OSD error messages in Ceph logs

Understand Ceph OSD error messages in the Ceph logs and how to fix them.

"Ceph OSD error messages in the Ceph logs" on page 1610 provides links to corresponding sections that explain the errors and point to specific procedures to fix the problems.

This table lists Ceph OSD error messages in the Ceph logs and links in the documentation of potential fixes.

Table 212: Ceph OSD error messages in the Ceph logs

Error message	Log file	See
heartbeat_check: no reply from osd.X	Main cluster log	"Flapping OSDs" on page 1615
wrongly marked me down	Main cluster log	"Flapping OSDs" on page 1615
osds have slow requests	Main cluster log	"Slow requests or requests are blocked" on page 1617
FAILED assert(0 == "hit suicide timeout")	OSD log	"Down OSDs" on page 1614

Full OSDs

Understand and troubleshoot full OSDs.

When OSD nodes are considered *full*, the **ceph health detail** command returns an error message similar to the following example:

```
HEALTH_ERR 1 full osds
osd.3 is full at 95%
```

What this means

Ceph prevents clients from running I/O operations on full OSD nodes to avoid losing data. It returns the `HEALTH_ERR full osds` message when the cluster reaches the capacity set by the **mon_osd_full_ratio** parameter. By default, this parameter is set to `0.95`, which means 95% of the cluster capacity.

Troubleshooting this problem

To diagnose and resolve full OSDs, complete the following steps:

1. Run **ceph osd df** to check how full each OSD is. When invoked without arguments, it reports on the entire cluster. You can also filter by device class or a single OSD.

   ```
   ceph osd df# ceph osd df ssd# ceph osd df osd.1701
   ```

2. Review the `%USE` column to see how full each OSD is. The `VAR` column shows how each OSD compares to the average.
   The `STDDEV` value indicates how evenly OSDs are filled.

   If **ceph osd df ssd** or **ceph osd df nvme** reports a standard deviation greater than 2.0, the balancer might not be enabled or functioning correctly.

3. If **ceph df** shows that the device class of the full OSD is much less full on average than the full OSD, this is likely a balancer issue. For more information, see “Using the Ceph Manager balancer module” on page 570.

4. If OSDs within the device class are balanced, use the following approaches to reduce usage:

   a. Check for benchmark detritus.
      If **rados bench** was run against this pool, orphaned data might remain. Check by running the following command:

      ```
      rados -p mypoolname ls | grep -c bench
      ```

      If this command returns a large number, consult Red Hat Support for safe removal by using the **rados rm** command.

   b. Request that users remove temporary, outdated, or non-essential data.

   c. Run **fstrim** on Ceph Block Device clients that use conventional file systems on volumes in a pool that uses the full OSD. This discards unused blocks and frees capacity.

   d. Add nodes or OSDs of the appropriate device class so the balancer can redistribute data and reduce fullness.

5. As an emergency measure, you can temporarily raise the full ratio.

   ```
   ceph osd set-full-ratio 0.98
   ```

   Important: Raising the full ratio is a temporary emergency measure. Restore the default setting as soon as possible to reduce the risk of cluster instability.

For more information, see “Nearfull OSDs” on page 1612 and “Deleting data from a full storage cluster” on page 1623.

Backfillfull OSDs

Understand and troubleshoot backfillfull OSDs.

If OSDs are considered *backfillfull*, the **ceph health detail** command returns an error message similar to the following example:

```
health: HEALTH_WARN
3 backfillfull osd(s)
Low space hindering backfill (add storage if this doesn't resolve itself): 32 pgs
backfill_toofull
```

What this means

When one or more OSDs has exceeded the backfillfull threshold, Ceph prevents data from rebalancing to this device. This is an early warning that rebalancing might not be complete and that the cluster is approaching full. The default for the backfullfull threshold is 90%.

Troubleshooting this problem

Determine what percent of raw storage (`%RAW USED`) is used, by using the **ceph df** command.

If `%RAW USED` is higher than 70-75%, troubleshoot by using any of the following options:

- For a long-term solution, scale the cluster by adding an OSD node.
- Delete unnecessary data.

  > **Note:** This is a short-term solution to avoid production downtime.

  For Ceph Block Device pools, also run the **fstrim** command on clients with conventional file systems on volumes served by pools that use the backfillfull OSD.
- Increase the `backfillfull` ratio for the OSDs that contain the PGs stuck in `backfull_toofull` to allow the recovery process to continue. Add new storage to the cluster as soon as possible or remove data to prevent filling more OSDs.

  > **Important:** Increasing the backfillfull ratio is a temporary emergency action. Restore the default setting as soon as possible to reduce the risk of cluster instability.

  ```
  ceph osd set-backfillfull-ratio VALUE
  ```

  The range for *VALUE* is 0.0 to 1.0.

  For example,

  ```
  [ceph: root@host01/]# ceph osd set-backfillfull-ratio 0.92
  ```

For more information and additional resolution strategies, see the following:

- “Nearfull OSDs” on page 1612
- “Full OSDs” on page 1611
- “Deleting data from a full storage cluster” on page 1623

Nearfull OSDs

Understand and troubleshoot nearfull OSDs.

If OSDs are considered *nearfull*, the **ceph health detail** command returns an error message, similar to the following example:

```
HEALTH_WARN 1 nearfull osds
osd.2 is near full at 95%
```

What this means

Ceph returns the `nearfull osds` message when the cluster reaches the capacity set by the **`ceph osd set-nearfull-ratio VALUE`** command.

For the latest default parameter settings and percentages, see "Full OSDs" on page 1611.

Ceph distributes data based on the CRUSH hierarchy in the best possible way but it cannot guarantee equal distribution. The main causes of the uneven data distribution and the `nearfull osds` messages are:

- The OSDs are not balanced among the OSD nodes in the cluster. That is, some OSD nodes host more aggregate capacity than others.
- The placement group (PG) count is not proper as per the number of the OSDs, use case, target PGs per OSD, and OSD utilization.
- The cluster uses inappropriate CRUSH tunables.
- The back-end storage for OSDs is almost full.
- The OSDs have different sizes.

> **Note:** This is only an issue if the pool has too few PGs or if the balancer module is not enabled. Ensure that the target autoscaler ratio is changed from the default and set to `300`.

For more information, see the following:

- "Full OSDs" on page 1611.
- How can I test the impact CRUSH map tunable modifications will have on my PG distribution across OSDs in Red Hat Ceph Storage? within the Red Hat Customer Portal.

Troubleshooting this problem

> **Note:** Ratios can trigger even if the **`ceph osd df`** command shows OSDs slightly below the thresholds. A buffer of 1 - 1.5% is common.

1. Verify that the PG count is sufficient, by using the **`ceph osd df`** command.
   The **PG** column output should be between `100` and `300`.

   If needed, increase the count. For more information, see "Increasing the placement group" on page 1639.

   > **Note:** When ratio values are changed it can take time to take effect. To expedite the value change, you can restart the affected OSDs in a safe rolling fashion.

2. Verify that you are using `optimal` CRUSH tunables to the cluster version. If needed, adjust the CRUSH tunables. For more information, see "CRUSH tuning" on page 132.
3. Use the Ceph Manager balancer module to balance the OSDs.
   For more information, see"Using the Ceph Manager balancer module" on page 570 .
4. Determine how much space remains on the disks that are used by OSDs.
   - To view how much space OSDs use in general.

     ```
     ceph osd df
     ```

     For example,

     ```
     [ceph: root@host01 /]# ceph osd df
     ```

   - To view how much space OSDs use on a particular node, use the **`df`** command from the node containing `nearfull` OSDs.
     For example,

```
[ceph: root@host01 /]# df
```

5. If needed, add an OSD node.

Additional considerations and best practices

- If ratios are raised during an emergency, restore them afterward. Leaving them elevated increases the risk of outages.
- Very large clusters may raise ratios by a few percent to reserve capacity.
- Always check that the balancer is working and that there are no overlapping CRUSH roots. Overlaps often occur when some CRUSH rules specify a device class and others do not.
- Run **fstrim** on Ceph Block Device clients weekly by using `cron` or one-time during crisis. Avoid using the **discard mount** option as it impacts performance continuously.
- Check for unused RADOS pools, empty the Ceph Block Device trash, and orphaned rados bench objects. Remove carefully by using the **rados rm** command.

Down OSDs

Understand and troubleshoot OSDs that are down.

If OSDs are considered *down*, the `ceph health detail` command returns an error similar to the following example:

```
HEALTH_WARN 1/3 in osds are down
```

What this means

One of the `ceph-osd` processes is unavailable due to a possible service failure or problems with communication with other OSDs. As a consequence, the surviving `ceph-osd` daemons reported this failure to the Monitors.

If the `ceph-osd` daemon is not running, the underlying OSD drive or file system is either corrupted, or some other error, such as a missing keyring, is preventing the daemon from starting.

Usually, networking issues cause the situation when the `ceph-osd` daemon is running but still marked as `down`.

For more information, see Stale placement groups. To enable logging files, see Ceph daemon logs.

Troubleshooting this problem

1. Determine which OSD is down, by using the **ceph health detail** command.

```
[ceph: root@host01 /]# ceph health detail
HEALTH_WARN 1/3 in osds are down
osd.0 is down since epoch 23, last address 192.168.106.220:6800/11080
```

2. Restart the OSD daemon, by using one of the following commands:
   - ```
     ceph orch daemon restart osd.ID
     ```

     Replace *ID* with the ID of the OSD that is in a `down` state.
     For example,

     ```
     [root@host01 ~]# ceph orch daemon restart osd.0
     ```

   - ```
     systemctl restart ceph-FSID@osd.ID.service
     ```

     Replace the *FSID* and *ID* with the FSID and ID respectively of the OSD that is in a `down` state.

     ```
     [root@host01 ~]# systemctl restart ceph-b1f94d70-6a2e-4893-a1c3-
     d5e8b9f74261@osd.0.service
     ```

   - If you are not able to start `ceph-osd`, go to "The ceph-osd daemon cannot start" on page 1615.
   - If you are able to start the `ceph-osd` daemon but it is marked in a `down` state, go to "The ceph-osd is running but still marked as down" on page 1615.

The `ceph-osd` daemon cannot start

1. If you have a node containing several OSDs (generally, more than twelve), verify that the default maximum number of threads (PID count) is sufficient. For more information, see "Increasing PID count" on page 1623.
2. Verify that the OSD data and journal partitions are mounted properly. You can use the **`ceph-volume lvm list`** command to list all devices and volumes that are associated with the Ceph Storage Cluster and then manually inspect if they are mounted properly. See the `mount(8)` manual page for details.
3. If you got the ERROR: missing keyring, cannot use cephx for authentication error message, the OSD is a missing keyring.
4. If you got the ERROR: unable to open OSD superblock on /var/lib/ceph/osd/ceph-1 error message, the `ceph-osd` daemon cannot read the underlying file system.
   - Check the corresponding log file to determine the cause of the failure. By default, Ceph stores log files in the `/var/log/ceph/` directory.
   - An EIO error message indicates a failure of the underlying disk. To fix this problem, replace the underlying OSD disk. For more information, "Replacing an OSD drive" on page 1620.
   - If the log includes any other FAILED assert errors, such as the following one, open a support ticket. For more information, contact Red Hat Support.
     See the following FAILED assert message example:

     ```
     FAILED assert(0 == "hit suicide timeout")
     ```
5. Check the `dmesg` output for the errors with the underlying file system or disk:
   - The error -5 error message similar to the one in the following example, indicates corruption of the underlying XFS file system.

     ```
     xfs_log_force: error -5 returned
     ```

     To resolve this issue, unmount the volume and then recover using the **`xfs_repair`** command tool-set. For more information and help using the **`xfs_repair`** command, contact Red Hat Support, referring to this error and documentation.
   - If the `dmesg` output includes any SCSI error error messages, see the SCSI Error Codes Solution Finder solution to determine the best way to fix the problem.
   - If you are still unable to fix the underlying file system, replace the OSD drive. For more information, see "Replacing an OSD drive" on page 1620.
6. If the OSD failed with a segmentation fault, such as the one in the following example, gather the required information and open a support ticket. For more information, contact Red Hat Support.

   ```
   Caught signal (Segmentation fault)
   ```

The `ceph-osd` is running but still marked as `down`

Check the corresponding log file to determine the cause of the failure. By default, Ceph stores log files in the `/var/log/ceph/` directory.

- If the log includes any error messages similar to the following example, see Flapping OSDs.

  ```
  wrongly marked me down
      heartbeat_check: no reply from osd.2 since back
  ```
- If there are any other error types, open a support ticket. For more information, contact Red Hat Support.

Flapping OSDs

Flapping OSDs are when OSDs change repeatedly between down and up within a short period of time. Understand and troubleshoot flapping OSDs.

Flapping OSDs are when the **`ceph -w | grep osds`** command shows OSDs repeatedly as `down` and then `up` again within a short period of time, such as in the following example.

```
ceph -w | grep osds
2022-05-05 06:27:20.810535 mon.0 [INF] osdmap e609: 9 osds: 8 up, 9 in
2022-05-05 06:27:24.120611 mon.0 [INF] osdmap e611: 9 osds: 7 up, 9 in
2022-05-05 06:27:25.975622 mon.0 [INF] HEALTH_WARN; 118 pgs stale; 2/9 in osds are down
2022-05-05 06:27:27.489790 mon.0 [INF] osdmap e614: 9 osds: 6 up, 9 in
2022-05-05 06:27:36.540000 mon.0 [INF] osdmap e616: 9 osds: 7 up, 9 in
2022-05-05 06:27:39.681913 mon.0 [INF] osdmap e618: 9 osds: 8 up, 9 in
2022-05-05 06:27:43.269401 mon.0 [INF] osdmap e620: 9 osds: 9 up, 9 in
2022-05-05 06:27:54.884426 mon.0 [INF] osdmap e622: 9 osds: 8 up, 9 in
2022-05-05 06:27:57.398706 mon.0 [INF] osdmap e624: 9 osds: 7 up, 9 in
2022-05-05 06:27:59.669841 mon.0 [INF] osdmap e625: 9 osds: 6 up, 9 in
2022-05-05 06:28:07.043677 mon.0 [INF] osdmap e628: 9 osds: 7 up, 9 in
2022-05-05 06:28:10.512331 mon.0 [INF] osdmap e630: 9 osds: 8 up, 9 in
2022-05-05 06:28:12.670923 mon.0 [INF] osdmap e631: 9 osds: 9 up, 9 in
```

In addition, the Ceph log contains error messages similar to the following example:

```
2022-05-25 03:44:06.510583 osd.50 127.0.0.1:6801/149046 18992 : cluster [WRN] map e600547
wrongly marked me down

2022-05-25 19:00:08.906864 7fa2a0033700 -1 osd.254 609110 heartbeat_check: no reply from
osd.2 since back 2021-07-25 19:00:07.444113 front 2021-07-25 18:59:48.311935 (cutoff
2021-07-25 18:59:48.906862)
```

What this means

The main causes of flapping OSDs are:

- Certain storage cluster operations, such as scrubbing or recovery, take an abnormal amount of time. For example, if you run these operations on objects with a large index or large placement groups. Usually, after these operations finish, the flapping OSDs' problem is solved.
- Problems with the underlying physical hardware. In this case, the **`ceph health detail`** command also returns the `slow requests` error message.
- Problems with the network.

Ceph OSDs cannot manage situations where the private network for the storage cluster fails, or significant latency is on the public client-facing network.

Ceph OSDs use the private network for sending heartbeat packets to each other to indicate that they are `up` and `in`. If the private storage cluster network does not work properly, OSDs are unable to send and receive the heartbeat packets. As a consequence, they report each other as being `down` to the Ceph Monitors, while marking themselves as `up`.

"Ceph configuration parameters that can influence the OSD status" on page 1616 lists parameters in the Ceph configuration file that can influence this behavior.

Table 213: Ceph configuration parameters that can influence the OSD status

Parameter	Description	Default value
`osd_heartbeat_grace_time`	How long OSDs wait for the heartbeat packets to return before reporting an OSD as `down` to the Ceph Monitors.	20 seconds
`mon_osd_min_down_reporters`	How many OSDs must report another OSD as down before the Ceph Monitors mark the OSD as `down`	2

"Ceph configuration parameters that can influence the OSD status" on page 1616 shows that in the default configuration, the Ceph Monitors mark an OSD as `down` if only one OSD made three distinct reports about the first OSD being `down`. In some cases, if one single host encounters network issues, the entire cluster can experience flapping OSDs. This is because the OSDs that are on the host reports other OSDs in the cluster as `down`.

> **Note:** The flapping OSDs scenario does not include the situation when the OSD processes are started and then immediately stopped.

Troubleshooting this problem

Important: Flapping OSDs can be caused by MTU misconfiguration on Ceph OSD nodes, at the network switch level, or both. To resolve the issue, set MTU to a uniform size on all storage cluster nodes, including on the core and access network switches with a planned downtime. Do not tune `osd heartbeat min size` because changing this setting can hide issues within the network, and it does not solve actual network inconsistency.

1. Check the output of the **`ceph health detail`** command again.

   Note: If the output includes the `slow requests` error message, see "Slow requests or requests are blocked" on page 1617 for information on how to troubleshoot this issue.

   ```
   ceph health detail
   HEALTH_WARN 30 requests are blocked > 32 sec; 3 osds have slow requests
   30 ops are blocked > 268435 sec
   1 ops are blocked > 268435 sec on osd.11
   1 ops are blocked > 268435 sec on osd.18
   28 ops are blocked > 268435 sec on osd.39
   3 osds have slow requests
   ```

2. Determine which OSDs are marked as `down` and on what nodes they reside.

   ```
   ceph osd tree | grep down
   ```

3. On the nodes containing the flapping OSDs, troubleshoot and fix any networking problems. For more information, see Troubleshooting networking issues.

4. Alternatively, you can temporarily force Monitors to stop marking the OSDs as `down` and `up` by setting the `noup` and `nodown` flags.

   Important: Using the `noup` and `nodown` flags does not fix the root cause of the problem but only prevents OSDs from flapping.

   ```
   ceph osd set noup
   ceph osd set nodown
   ```

Related information

Heartbeat

Slow requests or requests are blocked

Understand and troubleshoot slow requests or if requests are blocked.

When the `ceph-osd` daemon is slow to respond to a request, the `ceph health detail` command returns an error message similar to the following example:

```
HEALTH_WARN 30 requests are blocked > 32 sec; 3 osds have slow requests
30 ops are blocked > 268435 sec
1 ops are blocked > 268435 sec on osd.11
1 ops are blocked > 268435 sec on osd.18
28 ops are blocked > 268435 sec on osd.39
3 osds have slow requests
```

In addition to the error message, the Ceph logs include an error message similar to the following messages:

```
2022-05-24 13:18:10.024659 osd.1 127.0.0.1:6812/3032 9 : cluster [WRN] 6 slow requests, 6
included below; oldest blocked for > 61.758455 secs

2022-05-25 03:44:06.510583 osd.50 [WRN] slow request 30.005692 seconds old, received at
{date-time}: osd_op(client.4240.0:8 benchmark_data_ceph-1_39426_object7 [write 0~4194304]
0.69848840) v4 currently waiting for subops from [610]
```

What this means

An OSD with slow requests is every OSD that is not able to service the I/O operations per second (IOPS) in the queue within the time defined by the **osd_op_complaint_time** parameter. By default, this parameter is set to 30 seconds.

The main causes of OSDs having slow requests are:

- Problems with the underlying hardware, such as disk drives, hosts, racks, or network switches.
- Problems with the network. These problems are usually connected with flapping OSDs. For more information, see “Flapping OSDs” on page 1615.
- System load.

“Slow request types” on page 1618 shows the types of slow requests. Use the **dump_historic_ops** administration socket command to determine the type of a slow request.

Table 214: Slow request types

Slow request type	Description
`waiting for rw locks`	The OSD is waiting to acquire a lock on a placement group for the operation.
`waiting for subops`	The OSD is waiting for replica OSDs to apply the operation to the journal.
`no flag points reached`	The OSD did not reach any major operation milestone.
`waiting for degraded object`	The OSDs have not yet replicated an object the specified number of times.

For more information about the administration socket, see Using the Ceph administration socket.

Troubleshooting this problem

1. Determine whether the OSDs with slow or block requests share a common piece of hardware, for example, a disk drive, host, rack, or network switch.
2. If the OSDs share a disk:
   a. Use the `smartmontools` utility to check the health of the disk or the logs to determine any errors on the disk.

      Note: The `smartmontools` utility is included in the `smartmontools` package.

   b. Use the `iostat` utility to get the I/O wait report (`%iowai`) on the OSD disk to determine whether the disk is under heavy load.

      Note: The `iostat` utility is included in the `sysstat` package.
3. If the OSDs share the node with another service:
   a. Check the RAM and CPU usage.
   b. Use the `netstat` utility to see the network statistics on the Network Interface Controllers (NICs) and troubleshoot any networking issues.
4. If the OSDs share a rack, check the network switch for the rack. For example, if you use jumbo frames, verify that the NIC in the path has jumbo frames set.

5. If you are unable to determine a common piece of hardware shared by OSDs with slow requests, or to troubleshoot and fix hardware and networking problems, open a support ticket with Red Hat Support.

Stopping and starting rebalancing

When an OSD fails or you stop it, the CRUSH algorithm automatically starts the rebalancing process to redistribute data across the remaining OSDs.

Before you begin

Before you begin, be sure that you have root-level access to the Ceph Monitor node.

About this task

Rebalancing can take time and resources, therefore, consider stopping rebalancing during troubleshooting or maintaining OSDs.

> **Note:** Placement groups within the stopped OSDs become *degraded* during troubleshooting and maintenance.

For more information, see Rebalancing and recovery.

Procedure

1. Log in to the `cephadm` shell.
   For example,

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Set the `noout` flag before stopping the OSD.

   ```
   ceph osd set noout
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph osd set noout
   ```

3. When you finish troubleshooting or maintenance, unset the `noout` flag to start rebalancing.
   For example,

   ```
   [ceph: root@host01 /]# ceph osd unset noout
   ```

Mounting the OSD data partition

Learn how to properly mount the OSD data partiion.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Access to the `ceph-osd` daemon.
- Root-level access to the Ceph Monitor node.

About this task

If the OSD data partition is not mounted correctly, the `ceph-osd` daemon cannot start. If you discover that the partition is not mounted as expected, use this information to properly mount the partition.

For more information, see Down OSDs.

Procedure

1. Mount the partition.

```
mount -o noatime PARTITION /var/lib/ceph/osd/CLUSTER_NAME-OSD_NUMBER
```

- Replace *PARTITION* with the path to the partition on the OSD drive that is dedicated to OSD data.
- Specify the *CLUSTER_NAME* and *OSD_NUMBER*.

For example,

```
[root@host01 ~]# mount -o noatime /dev/sdd1 /var/lib/ceph/osd/ceph-0
```

2. Try to start the failed `ceph-osd` daemon.

```
systemctl start ceph-osd@OSD_ID
```

Replace the *OSD_ID* with the ID of the OSD.
For example,

```
[root@host01 ~]# systemctl start ceph-osd@0
```

Replacing an OSD drive

Ceph is designed for fault tolerance, which means that it can operate in a *degraded* state without losing data. Therefore, Ceph can operate even if a data storage drive fails. In the context of a failed drive, the *degraded* state means that the extra copies of the data that is stored on other OSDs backfill automatically to other OSDs in the cluster. If this occurs, replace the failed OSD drive and re-create the OSD manually.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the Ceph Monitor node.
- At least one OSD is *down*.

About this task

When a drive fails, Ceph reports the OSD as *down*.

For example,

```
HEALTH_WARN 1/3 in osds are down
osd.0 is down since epoch 23, last address 192.168.106.220:6800/11080
```

> **Note:** Ceph can mark an OSD as *down* also as a consequence of networking or permissions problems.

Modern servers typically deploy with hot-swappable drives so you can pull a failed drive and replace it with a new one without bringing down the node. The procedure includes these steps:

1. “Removing an OSD from the Ceph cluster” on page 1621
2. “Replacing the physical drive” on page 1622
3. “Adding an OSD to the Ceph cluster” on page 1622

For more information, see:

- Deploying Ceph OSDs on all available devices
- Deploying Ceph OSDs on specific devices and hosts
- “Down OSDs” on page 1614
- Installing

Removing an OSD from the Ceph cluster

Procedure

1. Log in to the `cephadm` shell.

```
[root@host01 ~]# cephadm shell
```

2. Determine which OSD is *down*.

```
ceph osd tree | grep -i down
```

For example,

```
[ceph: root@host01 /]# ceph osd tree | grep -i down
ID  CLASS  WEIGHT   TYPE NAME  STATUS  REWEIGHT  PRI-AFF
0   hdd    0.00999  osd.0      down    1.00000   1.00000
```

Determine which OSD is *down*.

3. Mark the OSD as `out` for the cluster to rebalance and copy its data to other OSDs

```
ceph osd out osd.OSD_ID
```

For example,

```
[ceph: root@host01 /]# ceph osd out osd.0
marked out osd.0.
```

> **Note:** If the OSD is *down*, Ceph marks it as *out* automatically after 600 seconds when it does not receive any heartbeat packet from the OSD based on the **`mon_osd_down_out_interval`** parameter. When this happens, other OSDs with copies of the failed OSD data begin backfilling to ensure that the required number of copies exists within the cluster. While the cluster is backfilling, the cluster is in a *degraded* state.

4. Ensure that the failed OSD is backfilling.

```
ceph -w | grep backfill
```

For example,

```
[ceph: root@host01 /]# ceph -w | grep backfill
2022-05-02 04:48:03.403872 mon.0 [INF] pgmap v10293282: 431 pgs: 1
active+undersized+degraded+remapped+backfilling, 28 active+undersized+degraded, 49
active+undersized+degraded+remapped+wait_backfill, 59 stale+active+clean, 294
active+clean; 72347 MB data, 101302 MB used, 1624 GB / 1722 GB avail; 227 kB/s rd,
1358 B/s wr, 12 op/s; 10626/35917 objects degraded (29.585%); 6757/35917 objects
misplaced (18.813%); 63500 kB/s, 15 objects/s recovering
2022-05-02 04:48:04.414397 mon.0 [INF] pgmap v10293283: 431 pgs: 2
active+undersized+degraded+remapped+backfilling, 75
active+undersized+degraded+remapped+wait_backfill, 59 stale+active+clean, 295
active+clean; 72347 MB data, 101398 MB used, 1623 GB / 1722 GB avail; 969 kB/s rd,
6778 B/s wr, 32 op/s; 10626/35917 objects degraded (29.585%); 10580/35917 objects
misplaced (29.457%); 125 MB/s, 31 objects/s recovering
2022-05-02 04:48:00.380063 osd.1 [INF] 0.6f starting backfill to osd.0 from (0'0,0'0]
MAX to 2521'166639
2022-05-02 04:48:00.380139 osd.1 [INF] 0.48 starting backfill to osd.0 from (0'0,0'0]
MAX to 2513'43079
2022-05-02 04:48:00.380260 osd.1 [INF] 0.d starting backfill to osd.0 from (0'0,0'0]
MAX to 2513'136847
2022-05-02 04:48:00.380849 osd.1 [INF] 0.71 starting backfill to osd.0 from (0'0,0'0]
MAX to 2331'28496
2022-05-02 04:48:00.381027 osd.1 [INF] 0.51 starting backfill to osd.0 from (0'0,0'0]
MAX to 2513'87544
```

The placement group states change from `active+clean` to `active`, some degraded objects, and then changes to `active+clean` when migration completes.

5. Stop the OSD.

```
ceph orch daemon stop OSD_ID
```

For example,

```
[ceph: root@host01 /]# ceph orch daemon stop osd.0
```

6. Remove the OSD from the storage cluster.

> **Note:** The *OSD_ID* is preserved.

```
ceph orch osd rm OSD_ID --replace
```

For example,

```
[ceph: root@host01 /]# ceph orch osd rm 0 --replace
```

Replacing the physical drive

Procedure

See the documentation for the hardware node for details on replacing the physical drive.

- If the drive is hot-swappable, replace the failed drive with a new one.
- If the drive is not hot-swappable and the node contains multiple OSDs, you might have to shut down the whole node and replace the physical drive. Consider preventing the cluster from backfilling. For more information, see "Stopping and starting rebalancing" on page 1619.
- When the drive appears under the `/dev/` directory, make a note of the drive path.
- If you want to add the OSD manually, find the OSD drive and format the disk.

Adding an OSD to the Ceph cluster

Procedure

1. After the new drive is inserted, deploy the OSDs by using one of the following options.
   - The OSDs are deployed automatically by the Ceph Orchestrator if the **--unmanaged** parameter is not set.

     ```
     ceph orch apply osd --all-available-devices
     ```

   - Deploy the OSDs on all the available devices with the **unmanaged** parameter set to `true`.

     ```
     ceph orch apply osd --all-available-devices --unmanaged=true
     ```

   - Deploy the OSDs on specific devices and hosts.

     ```
     ceph orch daemon add osd HOST:PATH_TO_DRIVE
     ```

     For example,

     ```
     [ceph: root@host01 /]# ceph orch daemon add osd host02:/dev/sdb
     ```

2. Ensure that the CRUSH hierarchy is accurate.

   ```
   ceph osd tree
   ```

Increasing PID count

Learn how to increase the maximum number of threads (PID count).

About this task

The default PID count can be sufficient with nodes containing more than 12 Ceph OSDs, especially during recovery. As a result, some `ceph-osd` daemons can terminate and fail to start again. If this happens, increase the maximum possible number of threads allowed.

Temporarily increase the maximum number of threads

Temporarily increase the maximum number of threads.

```
sysctl -w kernel.pid.max=PID_COUNT
```

For example,

```
[root@mon ~]# sysctl -w kernel.pid.max=4194303
```

Permanently increase the maximum number of threads

Permanently increase the maximum number of threads by updating the `/etc/sysctl.conf` file.
For example,

```
[root@mon ~]# sysctl -w kernel.pid.max=4194303
```

Deleting data from a full storage cluster

Ceph automatically prevents any I/O operations on OSDs that reached the capacity that is specified by the **`mon_osd_full_ratio`** parameter and returns the full osds error message. To fix this error, delete any unnecessary data.

Before you begin

Before you begin, be sure that you have root-level access to the Ceph Monitor node.

About this task

> **Note:** The **`mon_osd_full_ratio`** parameter sets the value of the **`full_ratio`** parameter when creating a cluster. You cannot change the value of **`mon_osd_full_ratio`** afterward. To temporarily increase the **`full_ratio`** value, increase the `set-full-ratio` instead.

Procedure

1. Log in to the `cephadm` shell.

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Determine the current value of **`full_ratio`**.
   By default, the **`full_ratio`** value is set to `0.95`.

   ```
   ceph osd dump | grep -i full
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph osd dump | grep -i full
   full_ratio 0.95
   ```

3. Temporarily change the value of `set-full-ratio` to `0.97`.

> **Important:** Do not set the **`set-full-ratio`** to a value higher than 0.97. Setting this parameter to a higher value makes the recovery process harder. As a result, you might not be able to recover full OSDs at all.

```
ceph osd set-full-ratio 0.97
```

4. Check that the parameter was successfully changed to `0.97`.
   For example

```
[ceph: root@host01 /]# ceph osd dump | grep -i full
full_ratio 0.97
```

5. Monitor the cluster state, by using the **`ceph -w`** command.
   As soon as the cluster changes its state from *full* to *nearfull*, delete any unnecessary data.
   For more information about these states, see "Full OSDs" on page 1611 and "Nearfull OSDs" on page 1612.
6. After any unnecessary data is deleted, change the `full_ratio` value back to `0.95`.

```
ceph osd set-full-ratio 0.95
```

7. Check that the parameter was successfully changed to `0.95`.
   For example,

```
[ceph: root@host01 /]# ceph osd dump | grep -i full
full_ratio 0.95
```

Troubleshooting multi-site Ceph Object Gateway

Understand how to troubleshoot and learn how to fix the most common errors that are related to multi-site Ceph Object Gateway configuration and operational conditions.

> **Note:** When the **`rados-admin bucket sync status`** command reports that the bucket is behind on shards even if the data is consistent across multi-site, run more writes to the bucket. It synchronizes the `sync status` reports and displays the message bucket is caught up with source.

To troubleshoot multi-site Ceph Object Gateway errors make sure to have:

- A running Red Hat Ceph Storage cluster.
- A running Ceph Object Gateway.

Error code definitions for Ceph Object Gateway

The Ceph Object Gateway logs contain error and warning messages to help troubleshooting your environment.

This information lists the common Ceph Object Gateway common error and warning messages and suggested resolutions. For more information, contact Red Hat Support.

- "Common error messages" on page 1624
- "Syncing error messages" on page 1625

Table 215: Common error messages

Error message	Description
data_sync: ERROR: a sync operation returned error	A high-level data sync process is indicating that lower-level bucket sync process returned an error. This message is redundant; the bucket sync error appears previously it in the log.

Error message	Description
data sync: ERROR: failed to sync object: BUCKET_NAME:OBJECT_NAME	Either the process failed to fetch the required object over HTTP from a remote gateway or the process failed to write that object to RADOS and it will be tried again.
data sync: ERROR: failure in sync, backing out (sync_status=2)	A low-level message that shows one of the previous conditions, specifically that the data was deleted before it was able to sync and thus showing a `-2 ENOENT` status.
data sync: ERROR: failure in sync, backing out (sync_status=-5)	A low-level message that shows one of the previous conditions, specifically that Ceph failed to write that object to RADOS and thus showing a `-5 EIO`.
ERROR: failed to fetch remote data log info: ret=11	This is the `EAGAIN` generic error code from `libcurl` showing an error condition from another gateway. By default, the system tries again.
meta sync: ERROR: failed to read mdlog info with (2) No such file or directory	The shard of the mdlog was never created so there is nothing to sync.

Table 216: Syncing error messages

Error message	Description
failed to sync object	Either the process failed to fetch this object over HTTP from a remote gateway or it failed to write that object to RADOS and it will be tried again.
failed to sync bucket instance: (11) Resource temporarily unavailable	A connection issue between primary and secondary zones.
failed to sync bucket instance: (125) Operation canceled	A racing condition exists between writes to the same RADOS object.

Syncing a multi-site Ceph Object Gateway

A multi-site sync reads the change log from other zones.

To get a high-level view of the sync progress from the metadata and the data logs, you can use the following command:

```
radosgw-admin sync status
```

This command lists which log shards, if any, which are behind their source zone.

> **Note:** Sometimes you might observe recovering shards when running the **`radosgw-admin sync status`** command. For data sync, there are 128 shards of replication logs that are each processed independently. If any of the actions triggered by these replication log events result in any error from the network, storage, or elsewhere, those errors get tracked so the operation can retry again later. While a given shard has errors that need a retry, **`radosgw-admin sync status`** command reports that shard as `recovering`. This recovery happens automatically, so the operator does not need to intervene to resolve them.

If the results of the sync status you have previously run reports log shards are behind, run the following command substituting the shard-id for *X*.

Buckets within a multi-site object can also be monitored on the Ceph dashboard. For more information, see Monitoring buckets of a multi-site object.

```
radosgw-admin data sync status --shard-id=X --source-zone=ZONE_NAME
```

For example,

```
[ceph: root@host01 /]# radosgw-admin data sync status --shard-id=27 --source-zone=us-east
{
  "shard_id": 27,
  "marker": {
        "status": "incremental-sync",
        "marker": "1_1534494893.816775_131867195.1",
        "next_step_marker": "",
        "total_entries": 1,
        "pos": 0,
        "timestamp": "0.000000"
   },
   "pending_buckets": [],
   "recovering_buckets": [
        "pro-registry:4ed07bb2-a80b-4c69-aa15-fdc17ae6f5f2.314303.1:26"
   ]
}
```

The output lists which buckets are next to sync and which buckets, if any, are going to be retried due to previous errors.

Inspect the status of individual buckets with the following command, substituting the bucket ID for *X*.

```
radosgw-admin bucket sync status --bucket=X
```

Replace *X* with the ID number of the bucket.

The result shows which bucket index log shards are behind their source zone.

A common error in sync is EBUSY, which means the sync is already in progress, often on another gateway. Read errors that are written to the sync error log, which can be read with the following command:

```
radosgw-admin sync error list
```

The syncing process tries again until it is successful. Errors can still occur that can require intervention.

Performance counters for multi-site Ceph Object Gateway data sync

Use performance counters to measure data sync for multi-site configurations of the Ceph Object Gateway.

About this task

The following performance counters are available for multi-site configurations of the Ceph Object Gateway to measure data sync:

- `poll_latency` measures the latency of requests for remote replication logs.
- `fetch_bytes` measures the number of objects and bytes fetched by data sync.

For more information about performance encounters, see “Ceph performance counters” on page 469.

Procedure

- Use the **`ceph --admin-daemon`** command to view the current metric data for the performance counters.

  > **Note:** Run the **`ceph --admin-daemon`** command from the node running the daemon.

  ```
  ceph --admin-daemon /var/run/ceph/cluster-id/ceph-client.rgw.RGW_ID.asok perf dump
  data-sync-from-ZONE_NAME
  ```

  For example,

```
[ceph: root@host01 /]# ceph --admin-daemon  /var/run/ceph/cluster-id/ceph-
client.rgw.host02-rgw0.103.94309060818504.asok perf dump data-sync-from-us-west

{
    "data-sync-from-us-west": {
        "fetch bytes": {
            "avgcount": 54,
            "sum": 54526039885
        },
        "fetch not modified": 7,
        "fetch errors": 0,
        "poll latency": {
            "avgcount": 41,
            "sum": 2.533653367,
            "avgtime": 0.061796423
        },
        "poll errors": 0
    }
}
```

Synchronizing data in a multi-site Ceph Object Gateway configuration

In a multi-site Ceph Object Gateway configuration of a storage cluster, failover and failback causes data synchronization to stop. The **`radosgw-admin sync status`** command reports that the data sync is behind for an extended period of time.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Ceph Object Gateway configured at two sites at least.

About this task

You can run the **`radosgw-admin data sync init`** command to synchronize data between the sites and then restart the Ceph Object Gateway. This command does not touch any actual object data and initiates data sync for a specified source zone. It causes the zone to restart a full sync from the source zone.

Important: Contact Red Hat Support before running the **`data sync init`** command. If you are going for a full restart of sync, and if there is a lot of data that needs to be synced on the source zone, then the bandwidth consumption is high and then you have to plan accordingly.

Note: If a user accidentally deletes a bucket on the secondary site, you can use the **`metadata sync init`** command on the site to synchronize data.

Procedure

1. Check the sync status between the sites.
   For example,

```
[ceph: host04 /]# radosgw-admin sync status
          realm d713eec8-6ec4-4f71-9eaf-379be18e551b (india)
      zonegroup ccf9e0b2-df95-4e0a-8933-3b17b64c52b7 (shared)
          zone 04daab24-5bbd-4c17-9cf5-b1981fd7ff79 (primary)
 current time 2022-09-15T06:53:52Z
 zonegroup features enabled: resharding
 metadata sync no sync (zone is master)
     data sync source: 596319d2-4ffe-4977-ace1-8dd1790db9fb (secondary)
                       syncing
                       full sync: 0/128 shards
                       incremental sync: 128/128 shards
                       data is caught up with source
```

2. Synchronize data from the secondary zone.

```
radosgw-admin data sync init --source-zone primary
```

For example,

```
[ceph: root@host04 /]# radosgw-admin data sync init --source-zone primary
```

3. Restart all the Ceph Object Gateway daemons at the site.

```
ceph orch restart rgw.myrgw
```

Troubleshooting `radosgw-admin` commands after upgrading a cluster

Troubleshoot using **`radosgw-admin`** commands inside the `cephadm` shell after upgrading a cluster.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Ceph Object Gateway configured at two sites at least.

About this task

The following is an example of errors that could be emitted after trying to run **`radosgw-admin`** commands inside the `cephadm` shell after upgrading a cluster.

```
2024-05-13T09:05:30.607+0000 7f4e7c4ea500  0 ERROR: failed to decode obj
from .rgw.root:periods.91d2a42c-735b-492a-bcf3-05235ce888aa.3
2024-05-13T09:05:30.607+0000 7f4e7c4ea500  0 failed reading current period info:  (5) Input/
output error
2024-05-13T09:05:30.607+0000 7f4e7c4ea500  0 ERROR: failed to start notify service ((5) Input/
output error
2024-05-13T09:05:30.607+0000 7f4e7c4ea500  0 ERROR: failed to init services (ret=(5) Input/
output error)
couldn't init storage provider
```

For example,

```
[ceph: root@host01 /]# date;radosgw-admin bucket list
Mon May 13 09:05:30 UTC 2024
2024-05-13T09:05:30.607+0000 7f4e7c4ea500  0 ERROR: failed to decode obj
from .rgw.root:periods.91d2a42c-735b-492a-bcf3-05235ce888aa.3
2024-05-13T09:05:30.607+0000 7f4e7c4ea500  0 failed reading current period info:  (5)
Input/output error
2024-05-13T09:05:30.607+0000 7f4e7c4ea500  0 ERROR: failed to start notify service ((5)
Input/output error
2024-05-13T09:05:30.607+0000 7f4e7c4ea500  0 ERROR: failed to init services (ret=(5) Input/
output error)
couldn't init storage provider
```

Procedure

- Repair the connection by running the command again with the **`-- radosgw-admin`** syntax.

```
cephadm shell -- radosgw-admin COMMAND
```

For example,

```
[root@host01 /]# cephadm shell -- radosgw-admin bucket list
```

Troubleshooting Ceph placement groups

Learn to troubleshoot the most common errors that are related to the Ceph Placement groups (PGs).

Before troubleshooting Ceph placement groups:

- Verify your network connection.
- Ensure that Monitors are able to form a quorum.

– Ensure that all healthy OSDs are *up* and *in*, and the backfilling and recovery processes are finished.

Most common Ceph placement groups errors

Understand the most common error messages that are returned by the **`ceph health detail`** command.

Placement groups that are stuck in a state that is not optimal can be listed, as described in "Listing placement groups stuck in stale, inactive, or unclean states" on page 1635.

> **Note:** When troubleshooting Ceph placement group errors, be sure you have both a running Red Hat Ceph Storage cluster and Ceph Object Gateway.

Placement group error messages

Understand placement group error messages and how to fix them.

"Placement group error messages" on page 1629 provides links to corresponding sections that explain the errors and point to specific procedures to fix the problems.

A table of common placement group error messages, and links in the documentation of potential fixes.

Table 217: Placement group error messages

Error message	See
HEALTH_ERR	
`pgs down`	"Placement groups are down" on page 1632
`pgs inconsistent`	"Inconsistent placement groups" on page 1630
`scrub errors`	"Inconsistent placement groups" on page 1630
HEALTH_WARN	
`pgs stale`	"Stale placement groups" on page 1629
`unfound`	"Unfound objects" on page 1633

Stale placement groups

The **`ceph health`** command lists some placement groups (PGs) as `stale`.

The following is an example of a placement group marked as *stale*:

```
HEALTH_WARN 24 pgs stale; 3/300 in osds are down
```

What this means

The Monitor marks a placement group as *stale* when it does not receive any status update from the primary OSD of the placement group's acting set or when other OSDs reported that the primary OSD is *down*.

Usually, PGs enter the *stale* state after you start the storage cluster and until the peering process completes. However, when the PGs remain *stale* for longer than expected, it might indicate that the primary OSD for those PGs is *down* or not reporting PG statistics to the Monitor. When the primary OSD storing *stale* PGs is back *up*, Ceph starts to recover the PGs.

The `mon_osd_report_timeout` setting determines how often OSDs report PGs statistics to Monitors. By default, this parameter is set to `0.5`, which means that OSDs report the statistics every half a second.

For more information, see Monitoring Placement Group Sets and "Down OSDs" on page 1614.

Troubleshooting this problem

1. Use the **`ceph health detail`** to identify which PGs are *stale* and on what OSDs they are stored. The error message includes information similar to the following example:

```
[ceph: root@host01 /]# ceph health detail
HEALTH_WARN 24 pgs stale; 3/300 in osds are down
...
pg 2.5 is stuck stale+active+remapped, last acting [2,0]
...
osd.10 is down since epoch 23, last address 192.168.106.220:6800/11080
osd.11 is down since epoch 13, last address 192.168.106.220:6803/11539
osd.12 is down since epoch 24, last address 192.168.106.220:6806/11861
```

2. Troubleshoot any problems with the OSDs that are marked as *down*.

Inconsistent placement groups

Understand and troubleshoot inconsistent placement groups (PGs).

Some placement groups are marked as active + clean + inconsistent and the **ceph health detail** command returns an error message similar to the following example:

```
HEALTH_ERR 1 pgs inconsistent; 2 scrub errors
pg 0.6 is active+clean+inconsistent, acting [0,1,2]
2 scrub errors
```

What this means

When Ceph detects inconsistencies in one or more replicas of an object in a placement group, it marks the placement group as *inconsistent*. The most common inconsistencies are:

- Objects have an incorrect size.
- Objects are missing from one replica after a recovery finished.

In most cases, errors during scrubbing cause inconsistency within placement groups.

For more information, see the following:

- “Listing placement group inconsistencies” on page 1635
- Data integrity
- Scrubbing the OSD
- “Repairing inconsistent placement groups” on page 1639

Troubleshooting this problem

1. Log in to the `cephadm` shell.

   ```
   cephadm shell
   ```

2. Use the **ceph health detail** command to determine which placement group is in the *inconsistent* state.

   ```
   [ceph: root@host01 /]# ceph health detail
   HEALTH_ERR 1 pgs inconsistent; 2 scrub errors
   pg 0.6 is active+clean+inconsistent, acting [0,1,2]
   2 scrub errors
   ```

3. Determine why the placement group is *inconsistent*.
   a. Start the deep scrubbing process on the placement group.

      ```
      ceph pg deep-scrub ID
      ```

      Replace *ID* with the ID of the *inconsistent* placement group.
      For example,

      ```
      [ceph: root@host01 /]# ceph pg deep-scrub 0.6
      instructing pg 0.6 on osd.0 to deep-scrub
      ```

   b. Use the **ceph -w** command to find messages related to the *inconsistent* placement group.

```
ceph -w | grep ID
```

Replace *ID* with the ID of the *inconsistent* placement group.

For example,

```
[ceph: root@host01 /]# ceph -w | grep 0.6
2022-05-26 01:35:36.778215 osd.106 [ERR] 0.6 deep-scrub stat mismatch, got
636/635 objects, 0/0 clones, 0/0 dirty, 0/0 omap, 0/0 hit_set_archive, 0/0
whiteouts, 1855455/1854371 bytes.
2022-05-26 01:35:36.788334 osd.106 [ERR] 0.6 deep-scrub 1 errors
```

4. If the output includes any error messages similar to the following ones, you can repair the `inconsistent` placement group.

```
PG.ID shard OSD: soid OBJECT missing attr , missing attr ATTRIBUTE_TYPE
PG.ID shard OSD: soid OBJECT digest 0 != known digest DIGEST, size 0 != known size
SIZE
PG.ID shard OSD: soid OBJECT size 0 != known size SIZE
PG.ID deep-scrub stat mismatch, got MISMATCH
PG.ID shard OSD: soid OBJECT candidate had a read error, digest 0 != known digest
DIGEST
```

– If the output includes any error messages similar to the following ones, you can repair the *inconsistent* placement group.

```
PG.ID shard OSD: soid OBJECT missing attr , missing attr ATTRIBUTE_TYPE
PG.ID shard OSD: soid OBJECT digest 0 != known digest DIGEST, size 0 != known size SIZE
PG.ID shard OSD: soid OBJECT size 0 != known size SIZE
PG.ID deep-scrub stat mismatch, got MISMATCH
PG.ID shard OSD: soid OBJECT candidate had a read error, digest 0 != known digest
DIGEST
```

– If the output includes any error messages similar to the following ones, it is not safe to repair the *inconsistent* placement group because you can lose data.

```
PG.ID shard OSD: soid OBJECT digest DIGEST != known digest DIGEST
PG.ID shard OSD: soid OBJECT omap_digest DIGEST != known omap_digest DIGEST
```

If this occurs, open a support ticket. For more information, contact Red Hat Support.

Unclean placement groups

Understand and troubleshoot unclean placement groups (PGs).

The **ceph health** command returns an error message similar to the following example:

```
HEALTH_WARN 197 pgs stuck unclean
```

What this means

Ceph marks a placement group as *unclean* if it has not achieved the *active+clean* state for the number of seconds specified in the **mon_pg_stuck_threshold** parameter in the Ceph configuration file. The default value of **mon_pg_stuck_threshold** is `300` seconds.

If a placement group is *unclean*, it contains objects that are not replicated the number of times specified in the **osd_pool_default_size** parameter. The default value of **osd_pool_default_size** is `3`, which means that Ceph creates three replicas.

Usually, *unclean* placement groups indicate that some OSDs might be *down*.

For more information, see Listing placement groups stuck in stale inactive or unclean state.

Troubleshooting this problem

1. Use the **ceph osd tree** command to determine which OSDs are in a *down* state.
   For example,

```
[ceph: root@host01 /]# ceph osd tree
```

2. Troubleshoot and fix any problems with the OSDs. For more information, see "Down OSDs" on page 1614.

Inactive placement groups

Understand and troubleshoot inactive placement groups (PGs).

The **ceph health** command returns an error message similar to the following example:

```
HEALTH_WARN 197 pgs stuck inactive
```

What this means

Ceph marks a placement group as *inactive* if it has not be active for the number of seconds specified in the **mon_pg_stuck_threshold** parameter in the Ceph configuration file. The default value of **mon_pg_stuck_threshold** is `300` seconds.

Usually, `inactive` placement groups indicate that some OSDs might be *down*.

For more information, see "Listing placement groups stuck in stale, inactive, or unclean states" on page 1635 and "Down OSDs" on page 1614.

Troubleshooting this problem

1. Use the **ceph osd tree** command to determine which OSDs are in a *down* state.
   For example,

   ```
   [ceph: root@host01 /]# ceph osd tree
   ```

2. Troubleshoot and fix any problems with the OSDs. For more information, see "Down OSDs" on page 1614.

Placement groups are down

Understand and troubleshoot placement groups that are in a *down* state.

The **ceph health detail** command reports that some placement groups are *down*. For example,

```
HEALTH_ERR 7 pgs degraded; 12 pgs down; 12 pgs peering; 1 pgs recovering; 6 pgs stuck
unclean; 114/3300 degraded (3.455%); 1/3 in osds are down
...
pg 0.5 is down+peering
pg 1.4 is down+peering
...
osd.1 is down since epoch 69, last address 192.168.106.220:6801/8651
```

What this means

In certain cases, the peering process can be blocked, which prevents a placement group from becoming active and usable. Usually, a failure of an OSD causes the peering failures.

For more information, see Ceph OSD peering.

Troubleshooting this problem

Determine what blocks the peering process.

```
ceph pg ID query
```

Replace *ID* with the ID of the placement group that is *down*.

For example,

```
[ceph: root@host01 /]#  ceph pg 0.5 query

{ "state": "down+peering",
  ...
  "recovery_state": [
       { "name": "Started\/Primary\/Peering\/GetInfo",
         "enter_time": "2021-08-06 14:40:16.169679",
         "requested_info_from": []},
       { "name": "Started\/Primary\/Peering",
         "enter_time": "2021-08-06 14:40:16.169659",
         "probing_osds": [
               0,
               1],
         "blocked": "peering is blocked due to down osds",
         "down_osds_we_would_probe": [
               1],
         "peering_blocked_by": [
               { "osd": 1,
                 "current_lost_at": 0,
                 "comment": "starting or marking this osd lost may let us proceed"}]},
       { "name": "Started",
         "enter_time": "2021-08-06 14:40:16.169513"}
   ]
}
```

The `recovery_state` section includes information on why the peering process is blocked.

- If the output includes the peering is blocked due to down osds error message, see "Down OSDs" on page 1614.
- If you see any other error message, open a support ticket with Red Hat Support.

Unfound objects

Understand and troubleshoot unfound objects.

The **ceph health** command returns an error message similar to the following one, containing the `unfound` keyword, as in the following example:

```
HEALTH_WARN 1 pgs degraded; 78/3778 unfound (2.065%)
```

What this means

Ceph marks objects as *unfound* when it knows these objects or their newer copies exist but it is unable to find them. As a result, Ceph cannot recover such objects and proceed with the recovery process.

Example situation

A placement group stores data on `osd.1` and `osd.2`.

1. `osd.1` goes *down*.
2. `osd.2` handles some write operations.
3. `osd.1` comes *up*.
4. A peering process between `osd.1` and `osd.2` starts, and the objects missing on `osd.1` are queued for recovery.
5. Before Ceph copies new objects, `osd.2` goes `down`.

As a result, `osd.1` knows that these objects exist, but there is no OSD that has a copy of the objects.

In this scenario, Ceph is waiting for the failed node to be accessible again, and the *unfound* objects blocks the recovery process.

Troubleshooting this problem

1. Log in to the `cephadm` shell.
   For example,

   ```
   [root@host01 ~]# cephadm shell
   ```

2. Use the **`ceph health detail`** command to determine which placement group contains *unfound* objects.
   For example,

```
[ceph: root@host01 /]# ceph health detail
HEALTH_WARN 1 pgs recovering; 1 pgs stuck unclean; recovery 5/937611 objects degraded
(0.001%); 1/312537 unfound (0.000%)
pg 3.8a5 is stuck unclean for 803946.712780, current state active+recovering, last
acting [320,248,0]
pg 3.8a5 is active+recovering, acting [320,248,0], 1 unfound
recovery 5/937611 objects degraded (0.001%); **1/312537 unfound (0.000%)**
```

3. List more information about the placement group.

```
ceph pg ID query
```

   Replace *ID* with the ID of the placement group containing the *unfound* objects.

   For example,

```
[ceph: root@host01 /]# ceph pg 3.8a5 query
{ "state": "active+recovering",
  "epoch": 10741,
  "up": [
        320,
        248,
        0],
  "acting": [
        320,
        248,
        0],
<snip>
  "recovery_state": [
        { "name": "Started\/Primary\/Active",
          "enter_time": "2021-08-28 19:30:12.058136",
          "might_have_unfound": [
                { "osd": "0",
                  "status": "already probed"},
                { "osd": "248",
                  "status": "already probed"},
                { "osd": "301",
                  "status": "already probed"},
                { "osd": "362",
                  "status": "already probed"},
                { "osd": "395",
                  "status": "already probed"},
                { "osd": "429",
                  "status": "osd is down"}],
          "recovery_progress": { "backfill_targets": [],
              "waiting_on_backfill": [],
              "last_backfill_started": "0\/\/0\/\/-1",
              "backfill_info": { "begin": "0\/\/0\/\/-1",
                  "end": "0\/\/0\/\/-1",
                  "objects": []},
              "peer_backfill_info": [],
              "backfills_in_flight": [],
              "recovering": [],
              "pg_backend": { "pull_from_peer": [],
                  "pushing": []}},
          "scrub": { "scrubber.epoch_start": "0",
              "scrubber.active": 0,
              "scrubber.block_writes": 0,
              "scrubber.finalizing": 0,
              "scrubber.waiting_on": 0,
              "scrubber.waiting_on_whom": []}},
        { "name": "Started",
          "enter_time": "2021-08-28 19:30:11.044020"}],
```

   The `might_have_unfound` section includes OSDs where Ceph tried to locate the *unfound* objects:

   - The `already probed` status indicates that Ceph cannot locate the *unfound* objects in that OSD.
   - The `osd is down` status indicates that Ceph cannot contact that OSD.

4. Troubleshoot the OSDs that are marked as *down*. See “Down OSDs” on page 1614 for details.
5. If you are unable to fix the problem that causes the OSD to be *down*, open a support ticket. For more information, see Red Hat Support.

Listing placement groups stuck in *stale*, *inactive*, or *unclean* states

After a failure, placement groups enter states such as *degraded* or *peering*. These states indicate normal progression through the failure recovery process. If a placement group stays in one of these states for a longer time than expected, it can be an indication of a larger problem.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Root-level access to the node.

About this task

The Monitors report when placement groups (PGs) get stuck in a state that is not optimal.

The `mon_pg_stuck_threshold` option in the Ceph configuration file determines the number of seconds after which placement groups are considered *inactive*, *unclean*, or *stale*.

"Placement group states" on page 1635 lists the `mon_pg_stuck_threshold` states along with their descriptions.

Table 218: Placement group states

State	What it means	Most common causes	See
`inactive`	The PG is not able to service read/write requests.	Peering problems	"Inactive placement groups" on page 1632
`unclean`	The PG contains objects that are not replicated the desired number of times. Something is preventing the PG from recovering.	– Unfound objects – OSDs are down – Incorrect configuration	"Unclean placement groups" on page 1631
`stale`	The status of the PG has not been updated by a `ceph-osd` daemon.	OSDs are down	"Stale placement groups" on page 1629

Procedure

1. Log in to the `cephadm` shell.
   For example,

   ```
   [root@host01 ~]# cephadm shell
   ```

2. List the stuck PGs.
   For example,

   ```
   [ceph: root@host01 /]# ceph pg dump_stuck inactive
   [ceph: root@host01 /]# ceph pg dump_stuck unclean
   [ceph: root@host01 /]# ceph pg dump_stuck stale
   ```

   For for more information, see Placement Group states.

Listing placement group inconsistencies

Find and troubleshoot Ceph placement group (PG) issues by finding inconsistencies.

Use the `rados` utility to list inconsistencies in various replicas of objects. Use the `--format=json-pretty` option to list a more detailed output.

Before listing PG inconsistencies, be sure to have a running a Red Hat Ceph Storage cluster running correctly and root-level access to the node.

For more information, see "Inconsistent placement groups" on page 1630 and "Repairing inconsistent placement groups" on page 1639.

Troubleshoot placement group inconsistencies, by using the following procedures:

- “Listing inconsistent placement groups in a pool” on page 1636
- “Listing inconsistent objects in a placement group” on page 1636
- “Listing inconsistent snapshot sets in a placement group” on page 1638

Listing inconsistent placement groups in a pool

To list all the inconsistent placement groups in a pool, run the following command:

```
rados list-inconsistent-pg POOL --format=json-pretty
```

For example,

```
[ceph: root@host01 /]# rados list-inconsistent-pg data --format=json-pretty
 [0.6]
```

Listing inconsistent objects in a placement group

To list all inconsistent objects in a placement group with ID, run the following command:

```
rados list-inconsistent-obj PLACEMENT_GROUP_ID
```

For example,

```
[ceph: root@host01 /]# rados list-inconsistent-obj 0.6
 {
     "epoch": 14,
     "inconsistents": [
         {
             "object": {
                 "name": "image1",
                 "nspace": "",
                 "locator": "",
                 "snap": "head",
                 "version": 1
             },
             "errors": [
                 "data_digest_mismatch",
                 "size_mismatch"
             ],
             "union_shard_errors": [
                 "data_digest_mismatch_oi",
                 "size_mismatch_oi"
             ],
             "selected_object_info": "0:602f83fe:::foo:head(16'1 client.4110.0:1 dirty|
data_digest|omap_digest s 968 uv 1 dd e978e67f od ffffffff alloc_hint [0 0 0])",
             "shards": [
                 {
                     "osd": 0,
                     "errors": [],
                     "size": 968,
                     "omap_digest": "0xffffffff",
                     "data_digest": "0xe978e67f"
                 },
                 {
                     "osd": 1,
                     "errors": [],
                     "size": 968,
                     "omap_digest": "0xffffffff",
                     "data_digest": "0xe978e67f"
                 },
                 {
                     "osd": 2,
                     "errors": [
                         "data_digest_mismatch_oi",
                         "size_mismatch_oi"
                     ],
                     "size": 0,
                     "omap_digest": "0xffffffff",
                     "data_digest": "0xffffffff"
                 }
             ]
         }
     ]
 }
```

Use the following fields to determine what is causing the inconsistency.

`name`

The name of the object with inconsistent replicas.

`nspace`

The namespace that is a logical separation of a pool. The value is empty by default.

`locator`

The key that is used as the alternative of the object name for placement.

`snap`

The snapshot ID of the object. The only writable version of the object is called `head`. If an object is a clone, this field includes its sequential ID.

`version`

The version ID of the object with inconsistent replicas. Each write operation to an object increments it.

`errors`

A list of errors that indicate inconsistencies between shards without determining which shard or shards are incorrect. See the `shard` array to further investigate the errors.

data_digest_mismatch

The digest of the replica that is read from one OSD is different from the other OSDs.

size_mismatch

The size of a clone or the `head` object does not match the expectation.

read_error

This error indicates inconsistencies that are most likely caused by disk errors.

union_shard_error

The union of all errors specific to shards. These errors are connected to a faulty shard. The errors that end with `oi` indicate that you must compare the information from a faulty object to information with selected objects. See the `shard` array to further investigate the errors.

In this example, the object replica that is stored on `osd.2` has a different digest than the replicas stored on `osd.0` and `osd.1`. Specifically, the digest of the replica is not `0xffffffff` as calculated from the shard read from `osd.2`, but `0xe978e67f`. In addition, the size of the replica that is read from `osd.2` is 0, while the size reported by `osd.0` and `osd.1` is 968.

Listing inconsistent snapshot sets in a placement group

To list all the inconsistent sets of snapshots, run the following command:

```
rados list-inconsistent-snapset PLACEMENT_GROUP_ID
```

For example,

```
[ceph: root@host01 /]# rados list-inconsistent-snapset 0.23 --format=json-pretty
 {
     "epoch": 64,
     "inconsistents": [
         {
             "name": "obj5",
             "nspace": "",
             "locator": "",
             "snap": "0x00000001",
             "headless": true
         },
         {
             "name": "obj5",
             "nspace": "",
             "locator": "",
             "snap": "0x00000002",
             "headless": true
         },
         {
             "name": "obj5",
             "nspace": "",
             "locator": "",
             "snap": "head",
             "ss_attr_missing": true,
             "extra_clones": true,
             "extra clones": [
                 2,
                 1
             ]
         }
     ]
```

This command returns the following error messages.

ss_attr_missing

One or more attributes are missing. Attributes are information about snapshots encoded into a snapshot set as a list of key-value pairs.

ss_attr_corrupted

One or more attributes fail to decode.

clone_missing

A clone is missing.

snapset_mismatch

The snapshot set is inconsistent by itself.

head_mismatch

The snapshot set indicates that `head` exists or not, but the scrub results report otherwise.

headless

The `head` of the snapshot set is missing.

size_mismatch

The size of a clone or the `head` object does not match the expectation.

Repairing inconsistent placement groups

Due to an error during deep scrubbing, some placement groups (PGs) can include inconsistencies. Ceph reports such placement groups as *inconsistent*.

Before you begin

Before you begin, be sure that you have root-level access to the Ceph Monitor node.

About this task

The following is an example of a report with an inconsistent placement group.

```
HEALTH_ERR 1 pgs inconsistent; 2 scrub errors
pg 0.6 is active+clean+inconsistent, acting [0,1,2]
2 scrub errors
```

> **Important:** Only certain inconsistencies can be repaired.
> Do not repair the placement groups if the Ceph logs include the following errors:
>
> ```
> PG.ID shard OSD: soid OBJECT digest DIGEST != known digest DIGEST
> PG.ID shard OSD: soid OBJECT omap_digest DIGEST != known omap_digest DIGEST
> ```
>
> If any of these errors occur, contact Red Hat Support.

For more information, see “Inconsistent placement groups” on page 1630 and “Listing placement group inconsistencies” on page 1635.

Procedure

- Repair the inconsistent placement group.

  ```
  ceph pg repair ID
  ```

  Replace *ID* with the ID of the *inconsistent* placement group.

Increasing the placement group

Learn how to increase the placement group.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster in a healthy state.
- Root-level access to the node.

About this task

Insufficient placement group (PG) count impacts the performance of the Ceph cluster and data distribution. It is one of the main causes of the nearfull osds error messages.

The recommended ratio is between 100 and 300 PGs per OSD. This ratio can decrease when you add more OSDs to the cluster.

The **pg_num** and **pgp_num** parameters determine the PG count. These parameters are configured per each pool, and therefore, you must adjust each pool with low PG count separately.

> **Important:** Increasing the PG count is the most intensive process that you can run on a Ceph cluster. This process might have a serious performance impact if not done in a slow and methodical way. After you increase **pgp_num**, you will not be able to stop or reverse the process and you must complete it. Consider increasing the PG count outside of business-critical processing time allocation, and alert all clients about the potential performance impact. Do not change the PG count if the cluster is in the HEALTH_ERR state.

For more information, see "Nearfull OSDs" on page 1612 and Monitoring placement group sets.

Procedure

1. Reduce the impact of data redistribution and recovery on individual OSDs and OSD hosts.
   a. Lower the value of the **osd max backfills**, **osd_recovery_max_active**, and **osd_recovery_op_priority** parameters.

      ```
      [ceph: root@host01 /]# ceph tell osd.* injectargs '--osd_max_backfills 1 --
      osd_recovery_max_active 1 --osd_recovery_op_priority 1'
      ```

   b. Disable the shallow and deep scrubbing.

      ```
      ceph osd set noscrub
      ```

      ```
      ceph osd set nodeep-scrub
      ```

      For example,

      ```
      [ceph: root@host01 /]# ceph osd set noscrub
      [ceph: root@host01 /]# ceph osd set nodeep-scrub
      ```

2. Calculate the optimal value of the **pg_num** and **pgp_num** parameters.
   Use the Ceph Placement Groups (PGs) per Pool Calculator to calculate the optimal values.
3. Increase the **pg_num** value in small increments until you reach the wanted value.
   a. Determine the starting increment value.
   Use a very low value that is a power of two, and increase it when you determine the impact on the cluster. The optimal value depends on the pool size, OSD count, and client I/O load.
   b. Increment the **pg_num** value.
   Specify the pool name and the new value.

      ```
      ceph osd pool set POOL pg_num VALUE
      ```

      For example,

      ```
      [ceph: root@host01 /]# ceph osd pool set data pg_num 4
      ```

   c. Monitor the status of the cluster by using the **ceph -s** command.
   The PGs state changes from *creating* to *active+clean*. Wait until all PGs are in the *active+clean* state.
4. Increase the **pgp_num** value in small increments until you reach the wanted value.
   a. Determine the starting increment value.
   Use a very low value that is a power of two, and increase it when you determine the impact on the cluster. The optimal value depends on the pool size, OSD count, and client I/O load.
   b. Increment the **pgp_num** value.
   Specify the pool name and the new value.

```
ceph osd pool set POOL pgp_num VALUE
```

For example,

```
[ceph: root@host01 /]# ceph osd pool set data pgp_num 4
```

c. Monitor the status of the cluster by using the **ceph -s** command.
The PGs state changes through *peering*, *wait_backfill*, *backfilling*, *recover*, and others. Wait until all PGs are in the *active+clean* state.

5. Repeat steps “1” on page 1640 though “4” on page 1640 for all pools with insufficient PG count.
6. Set **osd max backfills**, **osd_recovery_max_active**, and **osd_recovery_op_priority** to their default values.
For example,

```
[ceph: root@host01 /]# ceph tell osd.* injectargs '--osd_max_backfills 1 --
osd_recovery_max_active 3 --osd_recovery_op_priority 3'
```

7. Enable the shallow and deep scrubbing.

```
ceph osd unset noscrub
```

```
ceph osd unset nodeep-scrub
```

For example,

```
[ceph: root@host01 /]# ceph osd unset noscrub
[ceph: root@host01 /]# ceph osd unset nodeep-scrub
```

Interpreting placement group dump output

The **ceph pg dump** command displays a wealth of information regarding placement groups.

Because placement groups (PGs) typically range from hundreds to tens of thousands, redirecting the command output to a file or piping it through a pager is advised for easier readability.

See the following output example for the **ceph pg dump** command.

See the following tables for descriptions of each of the output types:

- PG stats
- Pool stats
- OSD stats

```
; PG stats
    PG_STAT  OBJECTS  MISSING_ON_PRIMARY  DEGRADED  MISPLACED  UNFOUND  BYTES  OMAP_BYTES*
 OMAP_KEYS*  LOG    DISK_LOG  STATE         STATE_STAMP                       VERSION
 REPORTED    UP       UP_PRIMARY  ACTING   ACTING_PRIMARY  LAST_SCRUB  SCRUB_STAMP
             LAST_DEEP_SCRUB  DEEP_SCRUB_STAMP                SNAPTRIMQ_LEN
    5.1b           0                   0         0          0        0      0           0
          0      0         0  active+clean  2024-06-05T22:12:41.648713+0000        0'0
255:294  [0,7,8]           0  [0,7,8]               0         0'0
 2024-06-01T16:34:39.253348+0000             0'0  2024-05-30T05:04:41.133279+0000
     0
    4.1a           0                   0         0          0        0      0           0
          0      0         0  active+clean  2024-06-05T22:12:44.045134+0000        0'0
255:291  [3,4,5]           3  [3,4,5]               3         0'0
 2024-06-01T07:43:59.302980+0000             0'0  2024-05-27T16:45:10.130663+0000
     0
    1.1f           0                   0         0          0        0      0           0
          0      0         0  active+clean  2024-06-05T22:12:44.044842+0000        0'0
255:262  [4,1,3]           4  [4,1,3]               4         0'0
 2024-06-01T08:52:22.029352+0000             0'0  2024-05-27T16:44:47.679985+0000
     0
    ....
    ....
    ; Pool stats
    POOLID OBJECTS MISSING_ON_PRIMARY DEGRADED MISPLACED UNFOUND BYTES OMAP_BYTES*
OMAP_KEYS* LOG     DISK_LOG
    5      4       0                  0        0         0       763   0           0
   8       8
    4      8       0                  0        0         0       0     0           0
   1145    1145
    3      209     0                  0        0         0       3702  0           0
   250271  250271
    1      4       0                  0        0         0       1403  0           0
   4       4
    2      0       0                  0        0         0       0     0           0
   0       0
    ...
    ...
    ; OSD stats
    OSD_STAT  USED     AVAIL    USED_RAW  TOTAL   HB_PEERS           PG_SUM  PRIMARY_PG_SUM
    4          70 MiB  9.9 GiB    70 MiB  10 GiB  [0,1,2,3,5,6,7,8]      56              21
    7          44 MiB   10 GiB    44 MiB  10 GiB  [0,1,2,3,4,5,6,8]      39               9
    2          44 MiB   10 GiB    44 MiB  10 GiB  [0,1,3,4,5,6,7,8]      34              11
```

Table 219: PG stats

PG stats column name	Description
PG_STAT	PG ID
OBJECTS	Number of RADOS objects associated with this PG
MISSING_ON_PRIMARY	Number of missing RADOS objects on primary OSD
DEGRADED	Number of degraded (missing desired replicas) RADOS objects
MISPLACED	Number of misplaced (wrong location in the cluster) RADOS objects
UNFOUND	Number of unfound RADOS objects
BYTES	Total number of bytes stored in RADOS objects
OMAP_BYTES*	Total number of bytes of all omaps
OMAP_KEYS*	Total number of omap keys
LOG	Number of PG log entries
LOG_DUPS	Number of PG log entries for duplicate ops
DISK_LOG	Number of stored PG log entries
STATE	Current state of the PG
STATE_STAMP	Timestamp of the last state change
VERSION	Version of the most recent write to the placement group
REPORTED	Version of pg_stat reported for this PG

PG stats column name	Description
UP	The set of OSDs responsible for transferring data
UP_PRIMARY	The primary OSD for this PG; handles client requests
ACTING	The set of OSDs that currently have a full and working version of the PG
ACTING_PRIMARY	Primary OSD responsible for handling client requests
LAST_SCRUB	Last PG version that completed a shallow scrub
SCRUB_STAMP	Timestamp when this PG last completed a shallow scrub
LAST_DEEP_SCRUB	Last PG version that completed a deep scrub
DEEP_SCRUB_STAMP	Timestamp when this PG last completed a deep-scrub
SNAPTRIMQ_LEN	Size of the snaptrim queue
LAST_SCRUB_DURATION	The duration (in seconds) of the last completed scrub
SCRUB_SCHEDULING	Indicates whether this PG is scheduled/queued/being scrubbed
OBJECTS_SCRUBBED	The number of RADOS objects scrubbed in a PG after a scrub begins
OBJECTS_TRIMMED	The number of RADOS objects trimmed

Table 220: Pool stats

Field	Description
POOLID	Numeric pool ID
OBJECTS	Number of RADOS objects stored in this pool
MISSING_ON_PRIMARY	Number of RADOS objects missing from this pool
DEGRADED	Number of degraded (missing desired replicas) RADOS objects in this pool
MISPLACED	Number of misplaced (wrong location in the cluster) RADOS objects in the pool
UNFOUND	Number of unfound RADOS objects in this pool
BYTES	Total number of bytes stored in RADOS objects in this pool
OMAP_BYTES*	Total number of bytes stored in OMAPs in this pool
OMAP_KEYS*	Total number of OMAP keys stored in this pool
LOG	Sum of log entries across all PGs within this pool
DISK_LOG	Sum of all persisted log entries across all PGs within this pool

Table 221: OSD stats

Field	Description
OSD_STAT	OSD ID
USED	The amount of data stored on the OSD
AVAIL	The amount of free space available on the OSD
USED_RAW	The amount of raw storage consumed by user data, internal overhead, and reserved capacity
TOTAL	The total storage capacity of the OSD
HB_PEERS	Peer OSDs that this OSD heartbeats

Field	Description
PG_SUM	The number of placement group replicas or shards on this OSD
PRIMARY_PG_SUM	The number of placement groups for which this OSD acts as primary

Troubleshooting Ceph objects

Use the `ceph-objectstore-tool` utility to perform high-level or low-level object operations. The `ceph-objectstore-tool` utility can help you troubleshoot problems related to objects within a particular OSD or placement group.

> **Important:** Manipulating objects can cause unrecoverable data loss. Before using the `ceph-objectstore-tool` utility, contact Red Hat Support.

Before troubleshooting Ceph objects, verify that there are no network related issues.

Troubleshooting high-level object operations

Use the `ceph-objectstore-tool` utility to run high-level object operations.

> **Important:** Manipulating objects can cause unrecoverable data loss. Before using the `ceph-objectstore-tool` utility, contact Red Hat Support.

The `ceph-objectstore-tool` utility supports the following high-level object operations:

- List objects
- List lost objects
- Fix lost objects

Before troubleshooting high-level object operations be sure to have root-level access to the Ceph OSD nodes.

Listing objects

The OSD can contain zero to many placement groups, and zero to many objects within a placement group (PG). The `ceph-objectstore-tool` utility allows you to list objects stored within an OSD.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root-level access to the Ceph OSD node.
- Stopping the `ceph-osd` daemon.

Procedure

1. Verify that the appropriate OSD is down.

   ```
   systemctl status ceph-osd@OSD_NUMBER
   ```

   For example,

   ```
   [root@host01 ~]# systemctl status ceph-osd@1
   ```

2. Log in to the OSD container.

```
cephadm shell --name osd.OSD_ID
```

For example,

```
[root@host01 ~]# cephadm shell --name osd.0
```

3. Identify all the objects within an OSD, regardless of their placement group.

```
ceph-objectstore-tool --data-path PATH_TO_OSD --op list
```

For example,

```
[ceph: root@host01 /]# ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0 --op
list
```

4. Identify all the objects within a placement group.

```
ceph-objectstore-tool --data-path PATH_TO_OSD --pgid PG_ID --op list
```

For example,

```
[ceph: root@host01 /]# ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0 --
pgid 0.1c --op list
```

5. Identify the PG that an object belongs to.

```
ceph-objectstore-tool --data-path PATH_TO_OSD --op list OBJECT_ID
```

For example,

```
[ceph: root@host01 /]# ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0 --op
list default.region
```

Fixing lost objects

Use the `ceph-objectstore-tool` utility to list and fix *lost* and *unfound* legacy objects that are stored within a Ceph OSD.

Before you begin

- Root-level access to the Ceph OSD node.
- Stopping the `ceph-osd` daemon.

About this task

> **Note:** This procedure applies only to legacy objects.

Procedure

1. Verify that the appropriate OSD is down.

```
systemctl status ceph-osd@OSD_NUMBER
```

For example,

```
[root@host01 ~]# systemctl status ceph-osd@1
```

2. Log in to the OSD container.

```
cephadm shell --name osd.OSD_ID
```

For example,

```
[root@host01 ~]# cephadm shell --name osd.0
```

3. List all lost legacy objects.

```
ceph-objectstore-tool --data-path PATH_TO_OSD --op fix-lost --dry-run
```

For example,

```
[ceph: root@host01 /]# ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0 --op
fix-lost --dry-run
```

4. Fix the *lost* and *unfound* objects by using the `ceph-objectstore-tool` utility.
Use the following procedures, depending on the circumstance.
   - Fix all lost objects.

     ```
     ceph-objectstore-tool --data-path PATH_TO_OSD --op fix-lost
     ```

     For example,

     ```
     [ceph: root@host01 /]# ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0
     --op fix-lost
     ```

   - Fix all lost objects within a placement group.

     ```
     ceph-objectstore-tool --data-path PATH_TO_OSD --pgid PG_ID --op fix-lost
     ```

     For example,

     ```
     [ceph: root@host01 /]# ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0
     --pgid 0.1c --op fix-lost
     ```

   - Fix a lost object by its identifier.

     ```
     ceph-objectstore-tool --data-path PATH_TO_OSD --op fix-lost OBJECT_ID
     ```

     For example,

     ```
     [ceph: root@host01 /]# ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0
     --op fix-lost default.region
     ```

Troubleshooting low-level object operations

Use the `ceph-objectstore-tool` utility to run low-level object operations.

> **Important:** Manipulating objects can cause unrecoverable data loss. Before using the `ceph-objectstore-tool` utility, contact Red Hat Support.

Before using the `ceph-objectstore-tool` utility, verify that you have root-level access to the Ceph OSD nodes.

Manipulating object content

With the `ceph-objectstore-tool` utility, you can get or set bytes on an object.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root-level access to the Ceph OSD node.
- Stopping the `ceph-osd` daemon.

Procedure

1. Verify that the appropriate OSD is down.

   ```
   systemctl status ceph-osd@OSD_ID
   ```

   For example,

   ```
   [root@host01 ~]# systemctl status ceph-osd@1
   ```

2. Find the object by listing the objects of the OSD or placement group (PG).

3. Log in to the OSD container.

   ```
   cephadm shell --name osd.OSD_ID
   ```

   For example,

   ```
   [root@host01 ~]# cephadm shell --name osd.0
   ```

4. Create a backup and working copy of the object.
   This must be done before setting bytes on an object.

   ```
   ceph-objectstore-tool --data-path PATH_TO_OSD --pgid PG_ID \
   OBJECT \
   get-bytes > OBJECT_FILE_NAME
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0 --
   pgid 0.1c \
   '{"oid":"zone_info.default","key":"","snapid":-2,"hash":235010478,"max":0,"pool":11,"n
   amespace":""}'  \
   get-bytes > zone_info.default.backup

   [ceph: root@host01 /]#  ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0 --
   pgid 0.1c \
   '{"oid":"zone_info.default","key":"","snapid":-2,"hash":235010478,"max":0,"pool":11,"n
   amespace":""}'  \
   get-bytes > zone_info.default.working-copy
   ```

5. Edit the working copy object file and modify the object contents.

6. Set the bytes of the object.

   ```
   ceph-objectstore-tool --data-path PATH_TO_OSD --pgid PG_ID \
   OBJECT \
   set-bytes < OBJECT_FILE_NAME
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0 --
   pgid 0.1c \
   '{"oid":"zone_info.default","key":"","snapid":-2,"hash":235010478,"max":0,"pool":11,"n
   amespace":""}' \
   set-bytes < zone_info.default.working-copy
   ```

Removing an object

Use the `ceph-objectstore-tool` utility to remove an object. By removing an object, its contents and references are removed from the placement group (PG).

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root-level access to the Ceph OSD node.
- Stopping the `ceph-osd` daemon.

About this task

Important: You cannot recreate an object after it is removed.

Procedure

1. Log in to the OSD container.

```
cephadm shell --name osd.OSD_ID
```

For example,

```
[root@host01 ~]# cephadm shell --name osd.0
```

2. Remove an object.

```
ceph-objectstore-tool --data-path PATH_TO_OSD --pgid PG_ID \
OBJECT \
remove
```

For example,

```
[ceph: root@host01 /]# ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0 --
pgid 0.1c \
'{"oid":"zone_info.default","key":"","snapid":-2,"hash":235010478,"max":0,"pool":11,"n
amespace":""}' \
remove
```

Listing object map

Use the `ceph-objectstore-tool` utility to list the contents of the object map (OMAP).

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root-level access to the Ceph OSD node.
- Stopping the `ceph-osd` daemon.

About this task

Listing the contents of the OMAP provides a list of keys.

Procedure

1. Verify that the appropriate OSD is down.

```
systemctl status ceph-osd@OSD_ID
```

For example,

```
[root@host01 ~]# systemctl status ceph-osd@1
```

2. Log in to the OSD container.

```
cephadm shell --name osd.OSD_ID
```

For example,

```
[root@host01 ~]# cephadm shell --name osd.0
```

3. List the object map.

```
ceph-objectstore-tool --data-path PATH_TO_OSD --pgid PG_ID OBJECT list-omap
```

For example,

```
[ceph: root@host01 /]# ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0 --
pgid 0.1c \
'{"oid":"zone_info.default","key":"","snapid":-2,"hash":235010478,"max":0,"pool":11,"n
amespace":""}' \
list-omap
```

Manipulating object map header

The `ceph-objectstore-tool` utility outputs the object map (OMAP) header with the values that are associated with the object's keys.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root-level access to the Ceph OSD node.
- Stopping the `ceph-osd` daemon.

Procedure

1. Verify that the appropriate OSD is down.

   ```
   systemctl status ceph-osd@OSD_ID
   ```

   For example,

   ```
   [root@host01 ~]# systemctl status ceph-osd@1
   ```

2. Log in to the OSD container.

   ```
   cephadm shell --name osd.OSD_ID
   ```

   For example,

   ```
   [root@host01 ~]# cephadm shell --name osd.0
   ```

3. Get the object map header.

   ```
   ceph-objectstore-tool --data-path PATH_TO_OSD \
   --pgid PG_ID OBJECT \
   get-omaphdr > OBJECT_MAP_FILE_NAME
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0 \
   --pgid 0.1c
   '{"oid":"zone_info.default","key":"","snapid":-2,"hash":235010478,"max":0,"pool":11,"n
   amespace":""}'  \
   get-omaphdr > zone_info.default.omaphdr.txt
   ```

4. Set the object map header.

   ```
   ceph-objectstore-tool --data-path PATH_TO_OSD \
   --pgid PG_ID OBJECT \
   set-omaphdr > OBJECT_MAP_FILE_NAME
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0 \
   --pgid 0.1c
   '{"oid":"zone_info.default","key":"","snapid":-2,"hash":235010478,"max":0,"pool":11,"n
   amespace":""}'  \
   set-omaphdr > zone_info.default.omaphdr.txt
   ```

Manipulating object map key

Use the `ceph-objectstore-tool` utility to change the object map (OMAP) key.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root-level access to the Ceph OSD node.
- Stopping the `ceph-osd` daemon.

About this task

To change the OMAP, provide the data path, the placement group identifier (PG ID), the object, and the key.

Procedure

1. Log in to the OSD container.

   ```
   cephadm shell --name osd.OSD_ID
   ```

   For example,

   ```
   [root@host01 ~]# cephadm shell --name osd.0
   ```

2. Get the object map key.

   ```
   ceph-objectstore-tool --data-path PATH_TO_OSD \
    --pgid PG_ID OBJECT \
    get-omap KEY > OBJECT_MAP_FILE_NAME
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0 \
    --pgid 0.1c
   '{"oid":"zone_info.default","key":"","snapid":-2,"hash":235010478,"max":0,"pool":11,"n
   amespace":""}'  \
    get-omap "" > zone_info.default.omap.txt
   ```

3. Set the object map key.

   ```
   ceph-objectstore-tool --data-path PATH_TO_OSD \
    --pgid PG_ID OBJECT \
    set-omap KEY > OBJECT_MAP_FILE_NAME
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0 \
    --pgid 0.1c
   '{"oid":"zone_info.default","key":"","snapid":-2,"hash":235010478,"max":0,"pool":11,"n
   amespace":""}'  \
    set-omap "" > zone_info.default.omap.txt
   ```

4. Remove the object map key.

   ```
   ceph-objectstore-tool --data-path PATH_TO_OSD \
    --pgid PG_ID OBJECT \
    rm-omap KEY
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0 \
    --pgid 0.1c
   '{"oid":"zone_info.default","key":"","snapid":-2,"hash":235010478,"max":0,"pool":11,"n
   amespace":""}'  \
    rm-omap ""
   ```

Listing object attributes

Use the `ceph-objectstore-tool` utility to list object attributes.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root-level access to the Ceph OSD node.
- Stopping the `ceph-osd` daemon.

About this task

The output provides you with the object's keys and values.

Procedure

1. Verify that the appropriate OSD is down.

   ```
   systemctl status ceph-osd@OSD_ID
   ```

   For example,

   ```
   [root@host01 ~]# systemctl status ceph-osd@1
   ```

2. Log in to the OSD container.

   ```
   cephadm shell --name osd.OSD_ID
   ```

   For example,

   ```
   [root@host01 ~]# cephadm shell --name osd.0
   ```

3. List the object attributes.

   ```
   ceph-objectstore-tool --data-path PATH_TO_OSD \
   --pgid PG_ID OBJECT \
   list-attrs
   ```

   For example,

   ```
   [ceph: root@host01 /]# ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0 \
   --pgid 0.1c
   '{"oid":"zone_info.default","key":"","snapid":-2,"hash":235010478,"max":0,"pool":11,"n
   amespace":""}' \
   list-attrs
   ```

Manipulating object attribute key

Use the `ceph-objectstore-tool` utility to change object attributes.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- Root-level access to the Ceph OSD node.
- Stop the `ceph-osd` daemon.

About this task

To manipulate object attributes you need the data paths, the placement group identifier (PG ID), the object, and the key in the object attribute.

Procedure

1. Verify that the appropriate OSD is down.

```
systemctl status ceph-osd@OSD_ID
```

For example,

```
[root@host01 ~]# systemctl status ceph-osd@1
```

2. Log in to the OSD container.

```
cephadm shell --name osd.OSD_ID
```

For example,

```
[root@host01 ~]# cephadm shell --name osd.0
```

3. Get the attributes of the object.

```
ceph-objectstore-tool --data-path PATH_TO_OSD \
--pgid PG_ID OBJECT \
get-attr KEY > OBJECT_ATTRS_FILE_NAME
```

For example,

```
[ceph: root@host01 /]# ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0 \
--pgid 0.1c
'{"oid":"zone_info.default","key":"","snapid":-2,"hash":235010478,"max":0,"pool":11,"n
amespace":""}' \
get-attr "oid" > zone_info.default.attr.txt
```

4. Set the attributes of the object.

```
ceph-objectstore-tool --data-path PATH_TO_OSD \
--pgid PG_ID OBJECT \
set-attr KEY > OBJECT_ATTRS_FILE_NAME
```

For example,

```
[ceph: root@host01 /]# ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0 \
--pgid 0.1c
'{"oid":"zone_info.default","key":"","snapid":-2,"hash":235010478,"max":0,"pool":11,"n
amespace":""}' \
set-attr "oid" > zone_info.default.attr.txt
```

5. Remove the attributes of the object.

```
ceph-objectstore-tool --data-path PATH_TO_OSD \
--pgid PG_ID OBJECT  \
rm-attr KEY
```

For example,

```
[ceph: root@host01 /]# ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0 \
--pgid 0.1c
'{"oid":"zone_info.default","key":"","snapid":-2,"hash":235010478,"max":0,"pool":11,"n
amespace":""}' \
rm-attr "oid"
```

Snapset metadata options

Use the `ceph-objectstore-tool` utility to modify snapset metadata for troubleshooting and recovery operations.

> **Important:** The `ceph-objectstore-tool` utility is a low-level troubleshooting and recovery tool. Incorrect use can cause unrecoverable data loss. Perform these operations only under Red Hat Support guidance.

`clear-snapset` command

Use the `clear-snapset` command to clear or modify snapset metadata associated with an object. These operations are intended for advanced troubleshooting scenarios.

1. Before running `ceph-objectstore-tool`, verify that the appropriate OSD is down and log into the OSD container.
   a. Verify that the OSD is down.

   ```
   systemctl status ceph-osd@OSD_ID
   ```

   b. Log in to the OSD container.

   ```
   cephadm shell --name osd.OSD_ID
   ```

2. Clear or modify snapset metadata associated with an object.

   ```
   ceph-objectstore-tool ... OBJECT clear-snapset [corrupt|seq|snaps|clones|clone_size|
   clone_overlap|size]
   ```

`clear-snapset` options

Get to know all of the `clear-snapset` command options for `ceph-objectstore-tool`.

`clear-snapset`

By default, clears `clones`, `clone_size`, and `clone_overlap`.

`clear-snapset corrupt`

Clear the entire snapset.

`clear-snapset seq`

Clear `snapset.seq`.

`clear-snapset snaps`

Clear `clone_snaps`.

`clear-snapset clones`

Clear `clones`.

`clear-snapset clone_size`

Clear `clone_size`.

`clear-snapset clone_overlap`

Clear `clone_overlap`.

`clear-snapset size`

Break all clone sizes by adding 1.

Example

The following example clears the entire snapset for an object:

```
ceph-objectstore-tool --data-path /var/lib/ceph/osd/ceph-0 'OBJECT-JSON' clear-snapset
corrupt
```

Troubleshooting clusters in stretch mode

You can replace and remove the failed tiebreaker monitors. You can also force the cluster into the recovery or healthy mode if needed.

Forcing stretch cluster into recovery or healthy mode

When in stretch degraded mode, the cluster goes into the recovery mode automatically after the disconnected data center comes back. If that does not happen, or you want to enable recovery mode early, you can force the stretch cluster into the recovery mode.

Before you begin

Before you begin, make sure that you have the following prerequisites in place:

- A running Red Hat Ceph Storage cluster.
- Stretch mode in enabled on a cluster.

Procedure

1. Force the stretch cluster into the recovery mode.

   > **Note:** The recovery state puts the cluster in the *HEALTH_WARN* state.

   For example,

   ```
   [ceph: root@host01 /]#  ceph osd force_recovery_stretch_mode --yes-i-really-mean-it
   ```

   When in recovery mode, the cluster should go back into normal stretch mode after the placement groups are healthy. In cases that the cluster does not go back into normal stretch mode you can force the stretch cluster into the healthy mode. to force the stretch cluster into the healthy mode, go to step “2” on page 1654.

2. If needed, force the stretch cluster into the healthy mode.

   > **Note:**
   > - You can also run this command if you want to force the cross-data-center peering early and you are willing to risk data downtime, or you have verified separately that all the placement groups can peer, even if they are not fully recovered. You might also want to start the healthy mode to remove the `HEALTH_WARN` state, which is generated by the recovery state.
   > - The **`force_recovery_stretch_mode`** and **`force_recovery_healthy_mode`** commands are not necessary, as they are included in the process of managing unanticipated situations.

   ```
   ceph osd force_healthy_stretch_mode --yes-i-really-mean-it
   ```

Troubleshooting scrub and deep-scrub issues

Learn to troubleshoot the scrub and deep-scrub issues.

Addressing scrub slowness issue after upgrading from Red Hat Ceph Storage 6 to Red Hat Ceph Storage 8 and Red Hat Ceph Storage 7 to Red Hat Ceph Storage 8

Learn to troubleshoot the scrub slowness issue which seen after upgrading from Red Hat Ceph Storage 6 to Red Hat Ceph Storage 8 and Red Hat Ceph Storage 7 to Red Hat Ceph Storage 8.

Before you begin

- A running Red Hat Ceph Storage cluster in a healthy state.
- Root-level access to the node.

About this task

Scrub slowness is caused by the automated OSD benchmark setting a very low value for **osd_mclock_max_capacity_iops_hdd**. Due to this, scrub operations are impacted since the IOPS capacity of an OSD plays a significant role in determining the bandwidth the scrub operation receives. To further increase the problem, scrubs receive only a fraction of the total IOPS capacity based on the QoS allocation defined by the **mClock** profile.

Due to this, the Ceph cluster reports the expected scrub completion time in multiples of days or weeks.

Procedure

- Detect low measured IOPS reported by OSD bench during OSD boot-up and fallback to default IOPS setting defined for **osd_mclock_max_capacity_iops_[hdd|ssd]**. The fallback is triggered if the reported IOPS falls below a threshold determined by **osd_mclock_iops_capacity_low_threshold_[hdd|ssd]**. A cluster warning is also logged.
  Example:

```
$ ceph config rm osd.X osd_mclock_max_capacity_iops_[hdd|ssd]
```

  - Perform the following steps if you have not yet upgraded to Red Hat Ceph Storage 7 from Red Hat Ceph Storage 6 (before the upgrade):

  a. For clusters already affected by the issue, remove the IOPS capacity setting on the OSD(s) before upgrading to the release with the fix by running the following command:
  For example:

```
$ ceph config rm osd.X osd_mclock_max_capacity_iops_[hdd|ssd]
```

  b. Set the **osd_mclock_force_run_benchmark_on_init** option for the affected OSD to `true` before the upgrade:
  For example:

```
$ ceph config set osd.X osd_mclock_force_run_benchmark_on_init true
```

  After upgrading to the release with this fix, the IOPS capacity reflects the default setting or the new one reported by the OSD bench.

  - Perform the following steps:

  a. If you were unable to perform the above steps before upgrade, you re-run the OSD bench again after upgrading by removing the **osd_mclock_max_capacity_iops_[hdd|ssd]** setting:
  Example:

```
$ ceph config rm osd.X osd_mclock_max_capacity_iops_[hdd|ssd]
```

  b. Set **osd_mclock_force_run_benchmark_on_init**to `true`.
  Example:

```
$ ceph config set osd.X osd_mclock_force_run_benchmark_on_init true
```

  c. Restart the OSD.
  After the OSD restarts, the IOPS capacity reflects the default setting or the new setting reported by the OSD bench.

Ceph subsystems default logging level values

Ceph subsystems default log and memory levels.

"Ceph subsystems default logging level values" on page 1655 details default logging level values for the various Ceph subsystems.

Table 222: Ceph subsystems default logging level values

Subsystem	Log Level	Memory Level
asok	1	5

Subsystem	Log Level	Memory Level
auth	1	5
buffer	0	0
client	0	5
context	0	5
crush	1	5
default	0	5
filer	0	5
bluestore	1	5
finisher	1	5
heartbeatmap	1	5
javaclient	1	5
journaler	0	5
journal	1	5
lockdep	0	5
mds balancer	1	5
mds locker	1	5
mds log expire	1	5
mds log	1	5
mds migrator	1	5
mds	1	5
monc	0	5
mon	1	5
ms	0	5
objclass	0	5
objectcacher	0	5
objecter	0	0
optracker	0	5
osd	0	5
paxos	0	5
perfcounter	1	5
rados	0	5

Subsystem	Log Level	Memory Level
rbd	0	5
rgw	1	5
throttle	1	5
timer	0	5
tp	0	5

Troubleshooting upgrade error messages

Use this information to help troubleshoot and understand common cephadm upgrade error messages.

"Troubleshooting upgrade error messages" on page 1657 shows some cephadm upgrade error messages. If the cephadm upgrade fails for any reason, an error message appears in the storage cluster health status.

Table 223: Troubleshooting upgrade error messages

Error Message	Description
UPGRADE_NO_STANDBY_MGR	Ceph requires both active and standby manager daemons to proceed, but there is currently no standby.
UPGRADE_FAILED_PULL	Ceph was unable to pull the container image for the target version. This can happen if you specify a version or container image that does not exist (e.g., 1.2.3), or if the container registry is not reachable from one or more hosts in the cluster.

Health messages of a Ceph cluster

Each health check has a unique identifier. The identifier is a terse pseudo-human-readable string that is intended to enable tools to make sense of health checks, and present them in a way that reflects their meaning.

These tables describe each of the different types of health messages of a Ceph cluster:

- Monitor
- Manager
- OSDs
- Device health
- Pools and placements groups
- Miscellaneous

Table 224: Monitor

Health code	Description
DAEMON_OLD_VERSION	Warn if old versions of Ceph are running on any daemons. If multiple versions are detected, a health error is generated.
MON_DOWN	One or more Ceph Monitor daemons are currently down.
MON_CLOCK_SKEW	The clocks on the nodes running the ceph-mon daemons are not sufficiently synchronized. Resolve it by synchronizing the clocks by using ntpd or chrony.

Health code	Description
MON_NETSPLIT	A network partition has occurred among Ceph Monitors. This health check is raised by the Monitor leader when its connectivity-based election data indicates that at least two Ceph Monitors have lost connectivity or reachability. The leader evaluates the connection reports collected from the Monitors. This warning appears only when the cluster is provisioned with at least three Ceph Monitors and uses the `connectivity` election strategy. Network partitions are reported as location-level netsplits, for example, "Netsplit detected between dc1 and dc2", when all Monitors in one location cannot communicate with all Monitors in another location. They are reported as individual Monitor netsplits, for example, "Netsplit detected between mon.a and mon.d", when only specific Monitors are disconnected across locations. The system prioritizes reporting at the highest topology level, such as `datacenter` or `rack`, when possible to help operators identify infrastructure-level network issues.
MON_MSGR2_NOT_ENABLED	The `ms_bind_msgr2` option is enabled but one or more Ceph Monitors is not configured to bind to a v2 port in the cluster's monmap. Resolve this by running **`ceph mon enable-msgr2`** command.
MON_DISK_LOW	One or more Ceph Monitors are low on disk space.
MON_DISK_CRIT	One or more Ceph Monitors are critically low on disk space.
MON_DISK_BIG	The database size for one or more Ceph Monitors are very large.
AUTH_INSECURE_GLOBAL_ID_RECLAIM	One or more clients or daemons are connected to the storage cluster that are not securely reclaiming their `global_id` when reconnecting to a Ceph Monitor.
AUTH_INSECURE_GLOBAL_ID_RECLAIM_ALLOWED	Ceph is configured to allow clients to reconnect to monitors by using an insecure process to reclaim their previous `global_id` because the setting **`auth_allow_insecure_global_id_reclaim`** is set to `true`.

Table 225: Manager

Health code	Description
MGR_DOWN	All Ceph Manager daemons are down.
MGR_MODULE_DEPENDENCY	An enabled Ceph Manager module is failing its dependency check.
MGR_MODULE_ERROR	A Ceph Manager module encountered an unexpected error. Typically, this means that an unhandled exception was raised from the module's serve function.

Table 226: OSDs

Health code	Description
OSD_DOWN	One or more OSDs are marked down.

Health code	Description
OSD_CRUSH_TYPE_DOWN	All the OSDs within a particular CRUSH subtree are marked down, for example all OSDs on a host. For example, `OSD_HOST_DOWN` and `OSD_ROOT_DOWN`
OSD_ORPHAN	An OSD is referenced in the CRUSH map hierarchy but does not exist. Remove the OSD by running **ceph osd crush rm osd._OSD_ID** command.
OSD_OUT_OF_ORDER_FULL	The utilization thresholds for `nearfull`, `backfillfull`, `full`, or, `failsafefull` are not ascending. Adjust the thresholds by running **ceph osd set-nearfull-ratio RATIO**, **ceph osd set-backfillfull-ratio RATIO**, and **ceph osd set-full-ratio RATIO** commands.
OSD_FULL	One or more OSDs exceeded the full threshold and is preventing the storage cluster from servicing writes. Restore write availability by raising the full threshold by a small margin **ceph osd set-full-ratio RATIO**.
OSD_BACKFILLFULL	One or more OSDs exceeded the backfillfull threshold, which prevents data from being allowed to rebalance to this device.
OSD_NEARFULL	One or more OSDs exceeded the nearfull threshold.
OSDMAP_FLAGS	One or more storage cluster flags of interest has been set. These flags include`full`,`pauserd`, `pausewr`,`noup`,`nodown`,`noin`,`noout`,`nobackfill`, `norecover`,`norebalance`,`noscrub`, `nodeep_scrub`, and`notieragent`. Except for`full`, the flags can be cleared with the **ceph osd set FLAG** and **ceph osd unset FLAG** commands.
OSD_FLAGS	One or more OSDs or CRUSH has a flag of interest set. These flags include`noup`,`nodown`,`noin`, and `noout`.
OLD_CRUSH_TUNABLES	The CRUSH map is using old settings and should be updated.
OLD_CRUSH_STRAW_CALC_VERSION	The CRUSH map is using an older, non-optimal method for calculating intermediate weight values for `straw` buckets.
CACHE_POOL_NO_HIT_SET	One or more cache pools are not configured with a hit set to track usage, which prevents the tiering agent from identifying cold objects to flush and evict from the cache. Configure the hit sets on the cache pool with the following commands: – **ceph osd pool set POOL_NAME hit_set_type TYPE** – **ceph osd pool set POOL_NAME hit_set_period PERIOD_IN_SECONDS** – **ceph osd pool set POOL_NAME hit_set_count NUMBER_OF_HIT_SETS** – **ceph osd pool set POOL_NAME hit_set_fpp TARGET_FALSE_POSITIVE_RATE**
OSD_NO_SORTBITWISE	`sortbitwise` flag is not set. Set the flag with **ceph osd set sortbitwise** command.

Health code	Description
POOL_FULL	One or more pools reached their quota and is no longer allowing writes. Increase the pool quota with the **ceph osd pool set-quota POOL_NAME max_objects NUMBER_OF_OBJECTS** and **ceph osd pool set-quota POOL_NAME max_bytes BYTES** commands or delete some existing data to reduce utilization.
BLUEFS_SPILLOVER	One or more OSDs that use the BlueStore backend is allocated db partitions but that space is filled, such that metadata has “spilled over” onto the normal slow device. Disable this with the **ceph config set osd bluestore_warn_on_bluefs_spillover false** command.
BLUEFS_AVAILABLE_SPACE	This output gives three values that are `BDEV_DB free`, `BDEV_SLOW free`, and `available_from_bluestore`.
BLUEFS_LOW_SPACE	If the BlueStore File System (BlueFS) is running low on available free space and there is little `available_from_bluestore` one can consider reducing BlueFS allocation unit size.
BLUESTORE_FRAGMENTATION	As BlueStore works, free space on underlying storage gets fragmented. This is normal and unavoidable but excessive fragmentation causes slowdown. Use the `BLUESTORE_WARN_ON_FREE_FRAGMENTATION` and `BLUESTORE_FRAGMENTATION_CHECK_PERIOD` parameters to control and configure the `BLUESTORE_FRAGMENTATION` health warning. The default value for `BLUESTORE_WARN_ON_FREE_FRAGMENTATION` is 0.8 and the default for `BLUESTORE_FRAGMENTATION_CHECK_PERIOD` is 3600 milliseconds.
BLUESTORE_LEGACY_STATFS	BlueStore tracks its internal usage statistics on a per-pool granular basis, and one or more OSDs have BlueStore volumes. Disable the warning with the **ceph config set global bluestore_warn_on_legacy_statfs false** command.
BLUESTORE_NO_PER_POOL_OMAP	BlueStore tracks omap space utilization by pool. Disable the warning with the **ceph config set global bluestore_warn_on_no_per_pool_omap false** command.
BLUESTORE_NO_PER_PG_OMAP	BlueStore tracks omap space utilization by PG. Disable the warning with **ceph config set global bluestore_warn_on_no_per_pg_omap false** command.
BLUESTORE_DISK_SIZE_MISMATCH	One or more OSDs using BlueStore has an internal inconsistency between the size of the physical device and the metadata tracking its size.
BLUESTORE_NO_COMPRESSION	One or more OSDs is unable to load a BlueStore compression plug-in. This can be caused by a broken installation, in which the `ceph-osd` binary does not match the compression plug-ins, or a recent upgrade that did not include a restart of the `ceph-osd` daemon.

Health code	Description
BLUESTORE_SPURIOUS_READ_ERRORS	One or more OSDs using BlueStore detects spurious read errors on the main device. BlueStore recovers from these errors by retrying disk reads.

Table 227: Device health

Health code	Description
DEVICE_HEALTH	One or more devices is expected to fail soon, where the warning threshold is controlled by the `mgr/devicehealth/warn_threshold` config option. Mark the device `out` to migrate the data and replace the hardware.
DEVICE_HEALTH_IN_USE	One or more devices is expected to fail soon and has been marked `out` of the storage cluster based on `mgr/devicehealth/mark_out_threshold`, but it is still participating in one more PGs.
DEVICE_HEALTH_TOOMANY	Too many devices are expected to fail soon and the `mgr/devicehealth/self_heal` behavior is enabled, such that marking out all of the ailing devices would exceed the clusters `mon_osd_min_in_ratio` ratio that prevents too many OSDs from being automatically marked `out`.

Table 228: Pools and placement groups

Health code	Description
PG_AVAILABILITY	Data availability is reduced, meaning that the storage cluster is unable to service potential read or write requests for some data in the cluster.
PG_DEGRADED	Data redundancy is reduced for some data, meaning the storage cluster does not have the desired number of replicas for replicated pools or erasure code fragments.
PG_RECOVERY_FULL	Data redundancy might be reduced or at risk for some data due to a lack of free space in the storage cluster, specifically, one or more PGs has the `recovery_toofull` flag set, which means that the cluster is unable to migrate or recover data because one or more OSDs is over the `full` threshold.
PG_BACKFILL_FULL	Data redundancy might be reduced or at risk for some data due to a lack of free space in the storage cluster, specifically, one or more PGs has the `backfill_toofull` flag set, which means that the cluster is unable to migrate or recover data because one or more OSDs is over the `backfillfull` threshold.
PG_DAMAGED	Data scrubbing has discovered some problems with data consistency in the storage cluster, specifically, one or more PGs has the inconsistent or `snaptrim_error` flag is set, indicating an earlier scrub operation found a problem, or that the `repair` flag is set, meaning a repair for such an inconsistency is currently in progress.
OSD_SCRUB_ERRORS	Recent OSD scrubs have uncovered inconsistencies.
OSD_TOO_MANY_REPAIRS	When a read error occurs and another replica is available it is used to repair the error immediately, so that the client can get the object data.

Health code	Description
LARGE_OMAP_OBJECTS	One or more pools contain large omap objects as determined by `osd_deep_scrub_large_omap_object_key_threshold` or `osd_deep_scrub_large_omap_object_value_sum_threshold` or both. Adjust the thresholds with the **ceph config set osd osd_deep_scrub_large_omap_object_key_threshold KEYS** and **ceph config set osd osd_deep_scrub_large_omap_object_value_sum_threshold BYTES** commands.
CACHE_POOL_NEAR_FULL	A cache tier pool is nearly full. Adjust the cache pool target size with the **ceph osd pool set CACHE_POOL_NAME target_max_bytes BYTES** and **ceph osd pool set CACHE_POOL_NAME target_max_bytes BYTES** commands.
TOO_FEW_PGS	The number of PGs in use in the storage cluster is less than the configurable threshold of `mon_pg_warn_min_per_osd` PGs per OSD.
POOL_PG_NUM_NOT_POWER_OF_TWO	One or more pools has a `pg_num` value that is not a power of two. Disable the warning with the **ceph config set global mon_warn_on_pool_pg_num_not_power_of_two false** command.
POOL_TOO_FEW_PGS	One or more pools should probably have more PGs, based on the amount of data that is currently stored in the pool. You can either disable auto-scaling of PGs with the **ceph osd pool set POOL_NAME pg_autoscale_mode off** command, automatically adjust the number of PGs with the **ceph osd pool set POOL_NAME pg_autoscale_mode on** command or manually set the number of PGs with **ceph osd pool set POOL_NAME pg_num NEW_PG_NUMBER** command.
TOO_MANY_PGS	The number of PGs in use in the storage cluster is over the configurable threshold of `mon_max_pg_per_osd` PGs per OSD. Increase the number of OSDs in the cluster by adding more hardware.
POOL_TOO_MANY_PGS	One or more pools should probably have more PGs, based on the amount of data that is currently stored in the pool. You can either disable auto-scaling of PGs with the **ceph osd pool set POOL_NAME pg_autoscale_mode off** command, automatically adjust the number of PGs with the **ceph osd pool set POOL_NAME pg_autoscale_mode on** command or manually set the number of PGs with the **ceph osd pool set POOL_NAME pg_num NEW_PG_NUMBER** command.
POOL_TARGET_SIZE_BYTES_OVERCOMMITTED	One or more pools have a `target_size_bytes` property set to estimate the expected size of the pool, but the values exceed the total available storage. Set the value for the pool to zero with the **ceph osd pool set POOL_NAME target_size_bytes 0** command.

Health code	Description
POOL_HAS_TARGET_SIZE_BYTES_AND_RATIO	One or more pools have both `target_size_bytes` and `target_size_ratio` set to estimate the expected size of the pool. Set the value for the pool to zero with `ceph osd pool set POOL_NAME target_size_bytes 0` command.
TOO_FEW_OSDS	The number of OSDs in the storage cluster is less than the configurable threshold of `osd_pool_default_size`.
SMALLER_PGP_NUM	One or more pools has a `pgp_num` value less than `pg_num`. This is normally an indication that the PG count was increased without also increasing the placement behavior. Resolve this by setting `pgp_num` to match with `pg_num` with the **ceph osd pool set POOL_NAME pgp_num PG_NUM_VALUE** command.
MANY_OBJECTS_PER_PG	One or more pools has an average number of objects per PG that is significantly higher than the overall storage cluster average. The specific threshold is controlled by the `mon_pg_warn_max_object_skew` configuration value.
POOL_APP_NOT_ENABLED	A pool exists that contains one or more objects but has not been tagged for use by a particular application. Resolve this warning by labeling the pool for use by an application with **rbd pool init POOL_NAME** command.
POOL_FULL	One or more pools has reached its quota. The threshold to trigger this error condition is controlled by the **mon_pool_quota_crit_threshold** configuration option.
POOL_NEAR_FULL	One or more pools is approaching a configured fullness threshold. Adjust the pool quotas with the **ceph osd pool set-quota POOL_NAME max_objects NUMBER_OF_OBJECTS** and **ceph osd pool set-quota POOL_NAME max_bytes BYTES** commands.
OBJECT_MISPLACED	One or more objects in the storage cluster is not stored on the node that the storage cluster should be stored on. This is an indication that data migration due to some recent storage cluster change has not yet completed.
OBJECT_UNFOUND	One or more objects in the storage cluster cannot be found, specifically, the OSDs know that a new or updated copy of an object should exist, but a copy of that version of the object has not been found on OSDs that are currently online.
SLOW_OPS	One or more OSD or monitor requests is taking a long time to process. This can be an indication of extreme load, a slow storage device, or a software bug.
PG_NOT_SCRUBBED	One or more PGs have not been scrubbed recently. PGs are normally scrubbed within every configured interval that is specified by `osd_scrub_max_interval` globally. Initiate the scrub with the **ceph pg scrub PG_ID** command.

Health code	Description
PG_NOT_DEEP_SCRUBBED	One or more PGs have not been deep scrubbed recently. Initiate the scrub with **ceph pg deep-scrub PG_ID** command. PGs are normally scrubbed every `osd_deep_scrub_interval` seconds, and this warning triggers when `mon_warn_pg_not_deep_scrubbed_ratio` percentage of interval has elapsed without a scrub since it was due.
PG_SLOW_SNAP_TRIMMING	The snapshot trim queue for one or more PGs has exceeded the configured warning threshold. This indicates that either an extremely large number of snapshots were recently deleted, or that the OSDs are unable to trim snapshots quickly enough to keep up with the rate of new snapshot deletions.

Table 229: Miscellaneous

Health code	Description
RECENT_CRASH	One or more Ceph daemons has crashed recently, and the crash has not yet been acknowledged by the administrator.
TELEMETRY_CHANGED	Telemetry has been enabled, but the contents of the telemetry report have changed since that time, so telemetry reports will not be sent.
AUTH_BAD_CAPS	One or more authorized users have capabilities that cannot be parsed by the monitor. Update the capabilities of the user with **ceph auth ENTITY_NAME DAEMON_TYPE CAPS** command.
OSD_NO_DOWN_OUT_INTERVAL	The `mon_osd_down_out_interval` option is set to zero, which means that the system will not automatically run any repair or healing operations after an OSD fails. Silence the interval with the **ceph config global mon mon_warn_on_osd_down_out_interval_zero false** command.
DASHBOARD_DEBUG	The Dashboard debug mode is enabled. This means, if there is an error while processing a REST API request, the HTTP error response contains a Python traceback. Disable the debug mode with the **ceph dashboard debug disable** command.

Published date: 2026-11-

© Copyright International Business Machines Corporation
US Government Users Restricted Rights - Use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM Corp

